Privacy-Preserving Machine Learning

Manuel V. Loureiro

Version 2026-10-27

Table of contents

Preface **1**
How this book is organized . . . 1

I. Prerequisites **2**
Who this part is for . . . 3
The common thread . . . 3
How to read this part . . . 4
Notation . . . 5
What you will be able to do . . . 6

1. Probability distributions **7**
1.1. Moments . . . 7
1.2. The moment generating function . . . 9
1.3. Bernoulli and binomial . . . 11
1.4. Poisson . . . 12
1.5. Exponential . . . 15
1.6. Laplace . . . 18
1.7. The normal distribution . . . 20
1.8. Normal vectors in many dimensions . . . 24
1.9. A density against its shifted copy . . . 26
1.10. Code . . . 29
1.11. Summary . . . 29
1.12. References . . . 30

2. Concentration inequalities **31**
2.1. Markov and Chebyshev . . . 31
2.2. The Chernoff method . . . 35
2.3. Sub-Gaussian variables and Hoeffding . . . 38
2.4. Many estimates at once . . . 42
2.5. Gaussian and Laplace tails . . . 44
2.6. Azuma and McDiarmid . . . 47
2.7. Code . . . 51
2.8. Summary . . . 52
2.9. References . . . 53

3. Divergences **54**
3.1. KL divergence . . . 55
3.2. Rényi divergence . . . 58
3.3. Two Gaussians with equal variance . . . 62

3.4. Two Laplace distributions with shifted location . . . 64
3.5. Total variation . . . 65
3.6. Hockey-stick divergence . . . 67
3.7. f-divergences . . . 70
3.8. The data-processing inequality . . . 72
3.9. Pinsker's inequality . . . 74
3.10. Code . . . 76
3.11. Summary . . . 77
3.12. References . . . 78

4. Hypothesis testing **79**
4.1. Tests and their two errors . . . 79
4.2. The Neyman–Pearson lemma . . . 80
4.3. Trade-off functions and ROC curves . . . 84
4.4. The Gaussian trade-off curve . . . 86
4.5. The Laplace trade-off curve . . . 88
4.6. Best advantage equals total variation . . . 90
4.7. The area under the ROC curve . . . 93
4.8. The Bayesian view . . . 94
4.9. Estimating error rates from finite trials . . . 97
4.10. Code . . . 98
4.11. Summary . . . 100
4.12. References . . . 100

5. Information theory **102**
5.1. Shannon entropy . . . 102
5.2. Mutual information . . . 106
5.3. Channels and capacity . . . 109
5.4. Min-entropy and guessing . . . 111
5.5. Conditional min-entropy and min-entropy leakage . . . 112
5.6. Shannon leakage versus min-entropy leakage . . . 115
5.7. Fano's inequality . . . 119
5.8. Code . . . 122
5.9. Summary . . . 122
5.10. References . . . 123

6. Optimization **125**
6.1. Convexity, Lipschitz continuity and smoothness . . . 125
6.2. Gradient descent and SGD . . . 129
6.3. SGD for convex Lipschitz objectives . . . 131
6.4. Strongly convex objectives . . . 136
6.5. Gradient noise and minibatches . . . 137
6.6. Non-convex objectives . . . 141
6.7. Sensitivity of regularized ERM . . . 142
6.8. Code . . . 146
6.9. Summary . . . 147
6.10. References . . . 148

7. Algorithmic stability and generalization **149**
7.1. Risk, empirical risk and the gap . . . 149
7.2. Uniform stability and the expected gap . . . 150
7.3. Regularized empirical risk minimization . . . 153
7.4. High probability: the Bousquet–Elisseeff bound . . . 156
7.5. Stochastic gradient descent . . . 158
7.6. An unstable algorithm: 1-nearest neighbour . . . 161
7.7. Memorization of individual records . . . 164
7.8. Choosing what to measure after looking . . . 164
7.9. Code . . . 167
7.10. Summary . . . 168
7.11. References . . . 170

8. Cryptography basics **171**
8.1. Distinguishers and advantage . . . 172
8.2. Negligible functions . . . 173
8.3. Indistinguishability and the hybrid argument . . . 174
8.4. Concrete security . . . 177
8.5. Pseudorandom generators and functions . . . 179
8.6. The birthday bound . . . 180
8.7. Block ciphers and the switching lemma . . . 181
8.8. Perfect secrecy and the one-time pad . . . 184
8.9. Chosen-plaintext security . . . 185
8.10. The simulation paradigm . . . 187
8.11. Learning with errors . . . 188
8.12. Regev encryption and its noise budget . . . 189
8.13. Code . . . 192
8.14. Summary . . . 192
8.15. References . . . 193

II. Threat models and attacks **194**
Who this part is for . . . 195
The common thread . . . 195
How to read this part . . . 197
Notation . . . 197
What you will be able to do . . . 198

9. Threat models and privacy games **199**
9.1. Two-world games . . . 200
9.2. Threat models . . . 202
9.3. Privacy games . . . 205
9.4. Advantage, total variation and the trade-off curve . . . 209
9.5. Average case and worst case . . . 211
9.6. Code . . . 214
9.7. Summary . . . 215
9.8. References . . . 216

10. Membership inference **217**
10.1. The loss attack and the generalization gap . . . 217
10.2. Shadow models . . . 220
10.3. LiRA: a likelihood-ratio test per record . . . 222
10.4. Evaluating attacks: TPR at a low FPR . . . 227
10.5. Why outliers are most exposed . . . 229
10.6. The full-size pipeline: LiRA on CIFAR-10 . . . 232
10.7. Code . . . 234
10.8. Summary . . . 235
10.9. References . . . 236

11. Attribute inference and model inversion **237**
11.1. The attribute inference game . . . 238
11.2. The imputation baseline . . . 240
11.3. Model inversion . . . 244
11.4. Confidence masking . . . 249
11.5. Code . . . 252
11.6. Summary . . . 252
11.7. References . . . 253

12. Property inference **255**
12.1. The property-inference game . . . 256
12.2. Why a property leaks when no record does . . . 258
12.3. Shadow models and meta-classifiers . . . 262
12.4. Permutation invariance . . . 267
12.5. Code . . . 270
12.6. Summary . . . 271
12.7. References . . . 271

13. Memorization and training-data extraction **273**
13.1. Verbatim, approximate, discoverable, extractable . . . 274
13.2. Duplication and model size . . . 277
13.3. Canaries and the exposure metric . . . 280
13.4. Extraction without the prefix: generate, rank, verify . . . 285
13.5. Code . . . 288
13.6. Summary . . . 288
13.7. References . . . 289

14. Gradient inversion **291**
14.1. The reconstruction game . . . 292
14.2. What a gradient is made of . . . 293
14.3. Exact recovery from a fully connected layer . . . 296
14.4. Label recovery from the last layer . . . 297
14.5. Deep Leakage from Gradients . . . 300
14.6. Batches and averaging . . . 303
14.7. How often a unit isolates an example . . . 305
14.8. Code . . . 308
14.9. Summary . . . 309

14.10. References 310

15. Linkage and re-identification **311**
15.1. Quasi-identifiers and the linkage attack 311
15.2. Pseudonyms and hashed identifiers 313
15.3. How many people are unique 315
15.4. Uniqueness from marginals and from one population 317
15.5. Sparse, high-dimensional data 320
15.6. Does releasing a sample protect? 323
15.7. Code 327
15.8. Summary 328
15.9. References 329

16. Syntactic anonymity and how it breaks **331**
16.1. k-anonymity 332
16.2. Homogeneity, background knowledge and l-diversity 336
16.3. Skewness, similarity and t-closeness 341
16.4. The composition attack 343
16.5. Code 347
16.6. Summary 348
16.7. References 349

17. Reconstruction attacks **350**
17.1. Subset sums and the exponential attack 350
17.2. The linear-programming decoder 352
17.3. Least squares under worst-case and random error 356
17.4. The Fundamental Law of Information Recovery 359
17.5. Attacking a deployed query system 363
17.6. Reconstructing a census block 365
17.7. Code 368
17.8. Summary 368
17.9. References 369

III. Differential privacy foundations **370**
Who this part is for 371
The common thread 371
How to read this part 373
Notation 373
What you will be able to do 375

18. The definition of differential privacy **376**
18.1. Neighbouring datasets 376
18.2. Pure and approximate differential privacy 377
18.3. Randomized response 380
18.4. Divergence forms and the privacy loss 382
18.5. The hypothesis-testing interpretation 385
18.6. Bound on the membership advantage 387

18.7. The Bayesian interpretation . . . 389
18.8. Why δ must be much smaller than 1/n . . . 392
18.9. What differential privacy does not protect . . . 394
18.10. Is it a privacy violation? . . . 395
18.11. Code . . . 396
18.12. Summary . . . 397
18.13. References . . . 398

19. Post-processing, group privacy and composition **400**
19.1. Post-processing . . . 400
19.2. Group privacy . . . 404
19.3. Basic sequential composition . . . 406
19.4. Parallel composition . . . 412
19.5. What composes and what doesn't . . . 413
19.6. Code . . . 415
19.7. Summary . . . 416
19.8. References . . . 416

20. The Laplace and Gaussian mechanisms **418**
20.1. Sensitivity . . . 418
20.2. The Laplace mechanism . . . 420
20.3. The Gaussian mechanism: classic calibration . . . 425
20.4. The exact privacy profile of the Gaussian mechanism . . . 428
20.5. Laplace against Gaussian in many dimensions . . . 431
20.6. Code . . . 433
20.7. Summary . . . 434
20.8. References . . . 435

21. Advanced and optimal composition **437**
21.1. The privacy loss of a composition . . . 437
21.2. Advanced composition . . . 441
21.3. The worst mechanism . . . 443
21.4. Optimal composition . . . 446
21.5. Adaptive composition . . . 449
21.6. Code . . . 451
21.7. Summary . . . 452
21.8. References . . . 453

22. Selection: exponential mechanism, report-noisy-max, sparse vector **455**
22.1. The exponential mechanism . . . 455
22.2. Utility of the exponential mechanism . . . 457
22.3. Applications of the exponential mechanism . . . 459
22.4. Report-noisy-max . . . 463
22.5. Gumbel noise: report-noisy-max is the exponential mechanism . . . 465
22.6. The sparse vector technique . . . 467
22.7. When the sparse vector technique goes wrong . . . 469
22.8. Code . . . 472
22.9. Summary . . . 473

22.10. References 474

23. Query release and private multiplicative weights **476**
23.1. Linear queries as inner products 476
23.2. Noise on every answer 478
23.3. Multiplicative weights and its regret bound 479
23.4. From regret to a mistake bound 481
23.5. Private multiplicative weights 483
23.6. Online private multiplicative weights 487
23.7. SmallDB: the exponential mechanism over small datasets 489
23.8. Code 491
23.9. Summary 491
23.10. References 492

24. Beyond global sensitivity **494**
24.1. Local sensitivity, and why it leaks 494
24.2. Smooth sensitivity 497
24.3. Admissible noise for smooth sensitivity 499
24.4. Propose-test-release 502
24.5. Inverse sensitivity 504
24.6. Stability-based release 505
24.7. Subsample-and-aggregate 507
24.8. Clipping 508
24.9. Code 511
24.10. Summary 512
24.11. References 513

25. Discrete and floating-point-safe mechanisms **514**
25.1. Floating-point Laplace breaks differential privacy 514
25.2. The snapping mechanism 516
25.3. An exact coin with an exponential bias 517
25.4. The discrete Laplace mechanism 520
25.5. The discrete Gaussian mechanism 524
25.6. Sampling the discrete Gaussian exactly 529
25.7. Code 531
25.8. Summary 532
25.9. References 533

26. Local, shuffle and distributed trust models **534**
26.1. Central and local differential privacy 535
26.2. The price of local privacy 537
26.3. Frequency estimation 541
26.4. Large and unknown domains 543
26.5. Estimating a mean 545
26.6. Collecting every day 547
26.7. The shuffle model 549
26.8. Distributed noise 551
26.9. Code 554

26.10. Summary . . . 554
26.11. References . . . 556

27. Lower bounds: what privacy costs **558**
27.1. Packing: pure DP pays the logarithm of the number of candidates . . . 559
27.2. Sample complexity: how many records privacy costs . . . 564
27.3. Fingerprinting: the $\sqrt{d}$ lower bound for approximate DP . . . 567
27.4. Tracing attacks: the lower bound is a membership inference attack . . . 572
27.5. Dinur–Nissim as a lower bound . . . 575
27.6. Code . . . 577
27.7. Summary . . . 577
27.8. References . . . 578

28. Private mean estimation **580**
28.1. Bounded data . . . 581
28.2. Gaussian data with a known range . . . 583
28.3. Coarse then fine . . . 585
28.4. Many dimensions: shrinking balls . . . 589
28.5. Heavy tails: clip into the data . . . 592
28.6. Code . . . 595
28.7. Summary . . . 596
28.8. References . . . 597

IV. Privacy accounting **599**
Who this part is for . . . 600
The common thread . . . 600
The running example . . . 602
How to read this part . . . 602
Notation . . . 602
What you will be able to do . . . 604

29. Rényi differential privacy **605**
29.1. Rényi differential privacy . . . 606
29.2. RDP curves of the basic mechanisms . . . 608
29.3. Composition . . . 611
29.4. Post-processing and group privacy . . . 613
29.5. The moments accountant . . . 617
29.6. From RDP to (ε, δ) . . . 618
29.7. Code . . . 621
29.8. Summary . . . 621
29.9. References . . . 623

30. From Rényi DP to (ε, δ) and to tests **624**
30.1. Where Mironov's conversion loses . . . 624
30.2. The improved conversion . . . 627
30.3. RDP as a constraint on tests . . . 630
30.4. Choosing the order . . . 634

30.5. The slack no conversion removes . . . 636
30.6. Code . . . 640
30.7. Summary . . . 640
30.8. References . . . 641

31. Concentrated differential privacy **643**
31.1. The definition of zCDP . . . 643
31.2. Pure DP implies zCDP . . . 645
31.3. From zCDP to (ε, δ) . . . 649
31.4. Group privacy . . . 651
31.5. Approximate zCDP . . . 653
31.6. Subsampling, and truncated CDP . . . 655
31.7. Code . . . 660
31.8. Summary . . . 660
31.9. References . . . 661

32. f-DP and Gaussian differential privacy **663**
32.1. f-DP . . . 663
32.2. Gaussian differential privacy . . . 665
32.3. Duality with (ε, δ) . . . 668
32.4. Composition . . . 670
32.5. The central limit theorem for composition . . . 673
32.6. The subsampled Gaussian and DP-SGD . . . 676
32.7. RDP as a lossy summary of the curve . . . 678
32.8. Code . . . 680
32.9. Summary . . . 682
32.10. References . . . 682

33. Amplification by subsampling **684**
33.1. Amplification for (ε, δ)-DP under Poisson sampling . . . 684
33.2. Sampling schemes and neighbouring relations . . . 688
33.3. RDP of the Poisson-subsampled Gaussian . . . 692
33.4. Small q and the moments accountant . . . 695
33.5. Accounting for DP-SGD . . . 697
33.6. Amplification for a general mechanism . . . 698
33.7. Code . . . 700
33.8. Summary . . . 701
33.9. References . . . 702

34. Numerical accounting with privacy loss distributions **704**
34.1. The privacy loss distribution . . . 705
34.2. Composition is convolution . . . 707
34.3. Pessimistic and optimistic discretization . . . 709
34.4. Composition by FFT . . . 712
34.5. The subsampled Gaussian . . . 714
34.6. Accounting for DP-SGD . . . 717
34.7. Code . . . 719
34.8. Summary . . . 721

34.9. References 721

35. Amplification by shuffling and by iteration **723**
35.1. Shuffled randomized response, exactly 723
35.2. The clone bound for any local randomizer 726
35.3. Amplification by iteration 729
35.4. Many steps on a bounded set 733
35.5. Noisy gradient descent on a quadratic 736
35.6. Code 738
35.7. Summary 739
35.8. References 740

36. Adaptive composition: filters, odometers and individual accounting **742**
36.1. When the parameters are chosen during the run 743
36.2. The Rényi filter 746
36.3. Filters for pure differential privacy 749
36.4. Odometers 752
36.5. Individual privacy accounting in DP-SGD 755
36.6. Code 758
36.7. Summary 759
36.8. References 760

37. Choosing an accountant **761**
37.1. Every accountant on one run 762
37.2. The inverse problem: noise for a target ε 766
37.3. Where each accountant is tight 768
37.4. Matching the accountant to the code 770
37.5. What to report 772
37.6. Checking your accountant 774
37.7. Code 775
37.8. Summary 776
37.9. References 777

V. Differential privacy in machine learning **779**
Who this part is for 780
The common thread 780
The running examples 781
How to read this part 782
Notation 782
What you will be able to do 783

38. Private empirical risk minimization **784**
38.1. Output perturbation 784
38.2. Objective perturbation 788
38.3. Gradient perturbation: noisy gradient descent 792
38.4. Noisy projected gradient descent without a ridge 795
38.5. How the three compare 798

38.6. When the model is not convex . . . 799
38.7. Code . . . 802
38.8. Summary . . . 802
38.9. References . . . 803

39. DP-SGD **805**
39.1. The algorithm and its guarantee . . . 806
39.2. Clipping: bias, noise and the choice of C . . . 809
39.3. Batch size and the number of steps at a fixed budget . . . 811
39.4. Per-example gradient norms without per-example gradients . . . 814
39.5. Microbatches and vectorized per-example gradients . . . 816
39.6. The running example: privacy against accuracy . . . 818
39.7. Code . . . 819
39.8. Summary . . . 820
39.9. References . . . 821

40. Hyperparameter tuning and private selection **824**
40.1. The cost of releasing the best run . . . 824
40.2. Paying by composition . . . 827
40.3. Random stopping . . . 828
40.4. Rényi bounds for a random number of runs . . . 830
40.5. Selecting by a noisy validation score . . . 835
40.6. Tuning the running example . . . 837
40.7. What to do and what to report . . . 839
40.8. Code . . . 840
40.9. Summary . . . 841
40.10. References . . . 842

41. DP-FTRL and matrix factorization **843**
41.1. Training is a stream of prefix sums . . . 844
41.2. The matrix mechanism . . . 846
41.3. Tree aggregation and DP-FTRL . . . 849
41.4. The square-root factorization . . . 850
41.5. Several participations . . . 855
41.6. The running example: matrix mechanisms against DP-SGD . . . 856
41.7. Code . . . 859
41.8. Summary . . . 861
41.9. References . . . 862

42. Private prediction: PATE and in-context learning **864**
42.1. The noisy vote: LNMax and GNMax . . . 865
42.2. How accurate is a noisy vote . . . 867
42.3. Private answers against a private model . . . 869
42.4. PATE: from private answers to a private model . . . 870
42.5. Data-dependent analysis and Confident-GNMax . . . 873
42.6. Stable aggregation . . . 877
42.7. Private in-context learning . . . 879
42.8. Code . . . 881

42.9. Summary . . . 883
42.10. References . . . 883

43. Public data and private fine-tuning **885**
43.1. The cost of each trainable direction . . . 886
43.2. The last layer on frozen features . . . 889
43.3. Low-rank adapters . . . 892
43.4. Gradients near a public subspace . . . 893
43.5. Pretraining and fine-tuning the running example . . . 896
43.6. What public data does not buy . . . 898
43.7. Code . . . 900
43.8. Summary . . . 901
43.9. References . . . 902

44. User-level differential privacy **904**
44.1. The privacy unit . . . 905
44.2. A user with k records in example-level DP-SGD . . . 907
44.3. User-level DP-SGD . . . 911
44.4. Contribution bounding . . . 915
44.5. Heterogeneous users . . . 917
44.6. Comparing the strategies on the running example . . . 918
44.7. Code . . . 920
44.8. Summary . . . 921
44.9. References . . . 922

45. Disparate impact **923**
45.1. Measuring disparate impact . . . 924
45.2. Clipping reweights the groups . . . 925
45.3. The noise floor of a small group . . . 930
45.4. Measuring per-group accuracy . . . 934
45.5. Mitigations and what they cost . . . 936
45.6. Code . . . 939
45.7. Summary . . . 940
45.8. References . . . 941

46. Choosing ε **942**
46.1. What one (ε, δ) pair says about a membership test . . . 943
46.2. The whole curve of DP-SGD . . . 944
46.3. The Bayesian reading . . . 948
46.4. Reconstruction . . . 952
46.5. Interpreting δ . . . 953
46.6. Deployed values and their units . . . 955
46.7. Justifying a choice . . . 956
46.8. Code . . . 958
46.9. Summary . . . 959
46.10. References . . . 960

47. Auditing private training **962**
47.1. From an attack to a lower bound on ε . . . 963
47.2. Canaries and threat models . . . 965
47.3. Tight audits of the mechanism . . . 968
47.4. One-run auditing . . . 971
47.5. Black-box audit of the trained network . . . 974
47.6. Finding bugs . . . 976
47.7. Code . . . 978
47.8. Summary . . . 979
47.9. References . . . 980

48. Adaptive data analysis **982**
48.1. Reusing a holdout set overfits . . . 983
48.2. Sample splitting and its cost . . . 985
48.3. Differential privacy implies generalization . . . 987
48.4. Answering with Gaussian noise: the $\sqrt{k}$ rate . . . 991
48.5. The reusable holdout: Thresholdout . . . 995
48.6. What it means for machine-learning practice . . . 997
48.7. Code . . . 999
48.8. Summary . . . 1000
48.9. References . . . 1001

Appendices **1002**

A. How much noise to add **1002**
A.1. What differential privacy promises, and the privacy loss . . . 1003
A.2. Sensitivity: how much one person can move the answer . . . 1006
A.3. Laplace noise: scale Δ_1/ε gives ε-DP . . . 1008
A.4. Randomized response: one bit, ε-DP . . . 1010
A.5. The exponential mechanism: choosing the best candidate . . . 1014
A.6. Gaussian noise, the classic formula . . . 1017
A.7. Gaussian noise, the exact calibration . . . 1020
A.8. How ε, δ, σ and the sensitivity trade off . . . 1023
A.9. All the recipes in one table . . . 1031
A.10. Code . . . 1031
A.11. References . . . 1032

B. Composing private releases **1034**
B.1. Basic composition: the budgets add . . . 1035
B.2. Advanced composition: $\sqrt{k}$ instead of k . . . 1039
B.3. Gaussian releases: k of them are one . . . 1044
B.4. Rényi DP: a summary of the loss that adds . . . 1047
B.5. Comparing the four accountants . . . 1051
B.6. Code . . . 1053
B.7. References . . . 1054

C. Accounting for DP-SGD **1055**
C.1. Clipping gives the sum a sensitivity . . . 1056
C.2. One noisy step is a Gaussian mechanism . . . 1058
C.3. Random batches make each step cheaper . . . 1060
C.4. The Rényi curve of a sampled Gaussian step . . . 1063
C.5. From batch, noise and epochs to the final ε . . . 1068
C.6. Code . . . 1072
C.7. References . . . 1073

D. What ε means, and converting between definitions **1075**
D.1. What ε means for any test . . . 1076
D.2. What ε means for an attacker's belief . . . 1080
D.3. From Rényi DP to (ε, δ) . . . 1083
D.4. From zero-concentrated DP to (ε, δ) . . . 1087
D.5. Gaussian DP and its exact (ε, δ) curve . . . 1089
D.6. Summary . . . 1091
D.7. Code . . . 1092
D.8. References . . . 1093

E. Sums, limits and counting **1094**
E.1. A sum's distribution, found by convolution . . . 1095
E.2. Stirling's formula and the peak of a binomial . . . 1099
E.3. How fast a Poisson count falls far above its mean . . . 1106
E.4. The central limit theorem, by swapping one term at a time . . . 1108
E.5. The length of a normal vector stays near $\sqrt{d}$ . . . 1113
E.6. The difference of two Laplace variables . . . 1117
E.7. Code . . . 1118
E.8. References . . . 1119

F. Tail bounds, tests and divergences **1122**
F.1. Bernstein's inequality: the variance sets the width . . . 1122
F.2. Chernoff's exponent is exact, for tails and for tests . . . 1125
F.3. The Clopper–Pearson limits exist and are unique . . . 1133
F.4. Rényi divergence tends to KL at order 1 . . . 1136
F.5. Perspective functions are jointly convex . . . 1139
F.6. A noisy count has an increasing likelihood ratio . . . 1143
F.7. Code . . . 1146
F.8. References . . . 1147

G. Optimization and estimation **1148**
G.1. Convexity three ways, and the linear rate of gradient descent . . . 1148
G.2. The gradient of a convex smooth function is co-coercive . . . 1153
G.3. When to stop SGD: balancing optimization against stability . . . 1154
G.4. No gradient method beats $RL/\sqrt{T}$ on every convex Lipschitz function . . . 1157
G.5. The 1-nearest-neighbour error tends to $\mathbb{E}[2p(1-p)]$ . . . 1162
G.6. Fitted parameters are approximately Gaussian, and one record moves them by its gradient . . . 1165
G.7. The best membership attack looks at the parameters through the loss . . . 1170

G.8. Code . . . 1174
G.9. References . . . 1175

H. Random matrices and factorizations **1176**
H.1. The average inverse of a random Gram matrix . . . 1177
H.2. A random subset matrix shrinks no direction much . . . 1182
H.3. The prefix-sum matrix . . . 1187
H.4. The optimal factorization for the root-mean-square error . . . 1193
H.5. Streaming: triangular factors and adaptive participations . . . 1195
H.6. Code . . . 1200
H.7. References . . . 1200

I. Pseudorandom functions and learning with errors **1202**
I.1. The GGM tree is a pseudorandom function . . . 1203
I.2. Search LWE reduces to decision LWE . . . 1210
I.3. Code . . . 1214
I.4. References . . . 1215

J. Mechanisms and their limits **1216**
J.1. Every approximate-DP pair is post-processed four-point randomized response . . . 1217
J.2. Approximate DP implies approximate zCDP . . . 1221
J.3. The stability-based histogram . . . 1223
J.4. A lower bound for heavy-tailed means . . . 1226
J.5. Sparse vector with c answers above . . . 1232
J.6. Gumbel top-c is the exponential mechanism without replacement . . . 1234
J.7. A Gumbel noisy max is private by the Laplace bar argument . . . 1237
J.8. The alarm mechanism is (ε, δ)-DP, with one condition . . . 1240
J.9. Code . . . 1241
J.10. References . . . 1243

K. Composition of trade-off functions **1244**
K.1. Every trade-off function has a canonical pair . . . 1245
K.2. Composition of f-DP, adaptive steps included . . . 1248
K.3. Dominating pairs survive adaptive composition . . . 1252
K.4. The f-DP central limit theorem . . . 1258
K.5. The central limit theorem for the subsampled Gaussian . . . 1262
K.6. Code . . . 1268
K.7. References . . . 1269

L. Subsampling, shuffling and iteration **1270**
L.1. The remove direction dominates in Rényi divergence . . . 1270
L.2. Dominating pairs for Poisson subsampling . . . 1276
L.3. The clone reduction for shuffling . . . 1278
L.4. Shift reduction and contraction . . . 1284
L.5. The random stopping time . . . 1288
L.6. Code . . . 1290
L.7. References . . . 1291

M. Private learning and tuning **1292**
M.1. The optimal rates of private convex learning 1292
M.2. The data-dependent bound for a noisy vote 1300
M.3. The best of a random number of runs 1306
M.4. A user with many records in record-level DP-SGD 1311
M.5. How one step of DP-SGD changes a group's loss 1315
M.6. Code 1318
M.7. References 1319

N. Auditing, reconstruction and adaptive analysis **1320**
N.1. One-run auditing: correct guesses are dominated by a binomial 1321
N.2. Private answers stay accurate with high probability 1326
N.3. The Ladder: a leaderboard that rarely changes cannot be overfitted 1334
N.4. Unbiased reconstruction has a floor set by Rényi DP of order 2 1337
N.5. Code 1341
N.6. References 1342

O. Glossary **1343**

P. Index **1347**

Preface

This is the full edition of *Privacy-Preserving Machine Learning*: every result is derived step by step. If differential privacy is new to you, read the course in differential privacy first: it teaches the definition and its core algorithms one topic per chapter, with complete proofs and little else, and this book then goes deeper and wider. The practitioner edition shares this book's parts and chapter files, states the same results without proofs and shows how to use them with real libraries; read it next if you build or review private systems.

The figures and the numbers quoted in the text are produced by each chapter's script, in the repository's `code/` directory, one folder per part.

How this book is organized

The book has five parts, each building on the ones before it.

1. **Prerequisites** collects the mathematics: distributions, concentration, divergences and hypothesis testing, then information theory, optimization and stability, and the basics of cryptography.
2. **Threat models and attacks** shows how models and data leak before any defence is introduced. It first attacks trained models, from one bit of membership up to whole training inputs, then attacks released data, from records with names removed to records hidden in crowds to aggregate statistics alone.
3. **Differential privacy foundations** defines the guarantee those attacks call for, builds its mechanisms, composes them, and ends with what privacy costs.
4. **Privacy accounting** tracks the guarantee through many steps of training, as accurately as possible.
5. **Differential privacy in machine learning** trains models with the guarantee: where the noise goes, how the knobs of noisy, clipped gradient descent trade privacy for accuracy, what else spends the budget, who pays for the noise, which ε to choose, and how to audit the result.

The appendices derive, in plain steps, the results a chapter cites or sketches, and the book ends with a glossary of acronyms and an index.

Part I.

Prerequisites

This part collects the mathematics the rest of the book stands on. It contains no new ideas about privacy. Everything here is standard probability, statistics, optimization, information theory and cryptography, and you may have met most of it before. What is different is the selection and the angle: each topic is chosen because a later part needs it, and each is developed only as far as that need goes, but always with full derivations.

Who this part is for

You are expected to know undergraduate calculus and linear algebra, the basics of probability (random variables, expectation, variance, independence, conditional probability), and enough machine learning to recognise a loss function, a training set and stochastic gradient descent. You should be able to read short Python programs that use NumPy. Nothing beyond that is assumed. Every inequality, divergence or definition that the later parts rely on is stated and derived here, so you can check each step for yourself instead of taking it on trust.

If you already know a topic well, read the Summary table at the end of its chapter, and move on if every result in it is familiar.

The common thread

Later parts study algorithms whose output is random, and keep asking the same few questions about them. How far is a random output from its expected value? How different are the output distributions on two slightly different inputs, and how would anyone tell them apart? How much does a trained model depend on a single training record? How much does an observation reveal about something hidden, and what can an observer with limited computing power learn from it? Each chapter of this part supplies the vocabulary and tools for one of these questions (Table 1).

Table 1.: The chapters of this part, the question each one answers, and the earlier chapters it uses.

Chapter	The question it answers	Builds on
Probability distributions	Which few distributions model counts, waiting times and additive noise, and what are their moments and tails?	—
Concentration inequalities	How far can a random quantity be from its mean, and with what probability?	1
Divergences	How different are two probability distributions, as a single number?	1, 2
Hypothesis testing	Given one observation, how well can you tell which of two distributions produced it?	1, 2, 3

Chapter	The question it answers	Builds on
Information theory	How much does an observation tell you about a hidden value, on average and for a single best guess?	3
Optimization	How fast does (stochastic) gradient descent converge, and how far does the minimizer move when one record changes?	1, 2
Algorithmic stability and generalization	Why does a model that depends little on each training record generalize?	2, 6
Cryptography basics	What does it mean that an efficient observer learns nothing it was not supposed to learn?	2, 3, 4

Two ideas run through all eight chapters. The first is the **likelihood ratio** $p(x)/q(x)$: how many times more likely an observation is under one distribution than under another. Divergences are averages of functions of it, the best test thresholds it, and mutual information is a divergence in disguise. The second is the **effect of one input**: how much one record shifts an average, a minimizer, a loss or a test statistic. Concentration inequalities bound it for sums, stability bounds it for learning algorithms, and the sensitivity of regularized empirical risk minimization makes it a concrete number. Watch for both as you read. Most of the later parts are built from them.

How to read this part

Order. Read chapters 1 to 4 in order, since each one uses the previous ones. Together they form the core of this part: tail bounds, then a number for how different two distributions are, then what that number means for anyone trying to tell them apart. Chapter 5, information theory, continues that thread: it measures how much an observation reveals, on average and for a single best guess. Then read chapters 6 and 7, which go together: how training moves when one record changes, and why that matters for generalization. Chapter 8 is used by three later chapters: chapter 9 on threat models uses its security games and advantage, chapter 15 on linkage uses its birthday bound, and chapter 26 on trust models builds secure aggregation from its secret sharing. A planned later part on cryptographic approaches will use all of it. You can defer it until you reach chapter 9.

Structure of a chapter. Every chapter has the same shape, so you always know where to look. It opens with the question it answers, the one idea that organizes the answer, and a running example with its numbers. Then comes one section per topic, in teaching order. Each section opens with its question, and its definitions, theorems and worked examples are numbered so that later chapters can point to them. Every theorem is followed at once by its proof in numbered steps, and then by what it means and how tight it is. A **Code** section runs the script that checks

every quoted number, a **Summary** table lists each result with where it is used, and **References** close the chapter.

Abstract steps are made concrete. Where a derivation takes an abstract step, such as writing a probability as the expectation of an indicator or changing the distribution an expectation is taken under, the text works the step out on a small case first, usually a die or a coin with a table of every outcome. If a step feels like a trick, look for that table. It is there to show that the step is just bookkeeping.

Run the code. Each chapter has a script in `code/` that reproduces every number and figure in its text. From the repository root, run, for example,

```
uv run python code/part01-prerequisites/concentration.py
```

The scripts use fixed random seeds, so your output matches the tables in the text. Change the parameters and watch how the bounds respond. That is the quickest way to develop a feel for how loose each bound is.

Notation

The chapters share the conventions in Table 2. Where a chapter departs from them, it says so at its start.

Table 2.: Notation shared by the chapters of this part.

Symbol	Meaning
$\Pr[A]$, $\mathbb{E}[X]$, $\mathrm{Var}(X)$	Probability, expectation, variance
$\mathbf{1}[A]$	Indicator of the event A: 1 if A happens, 0 otherwise
$\mathcal{N}(\mu, \sigma^2)$, $\mathrm{Lap}(b)$	Normal distribution with mean μ and variance σ^2; Laplace distribution with scale b
P, Q and p, q	Two distributions and their probability mass functions or densities
$\ln$, $\log_2$	Natural logarithm (in nats); base-2 logarithm (in bits), used in information theory
$\lVert \cdot \rVert$	Euclidean norm, $\lVert x \rVert = \lVert x \rVert_2$
$D = (z_1, \dots, z_n)$, $z = (x, y)$	A dataset of n records; a labelled record with features x and label y
$\theta \in \mathbb{R}^d$, $\ell(\theta; z)$	Model parameters of dimension d; the loss of θ on the record z
$\mathcal{L}(\theta; D)$, $\mathcal{L}_{\mathcal{D}}(\theta)$	Empirical risk on D; population risk under the data distribution $\mathcal{D}$
η, T	Learning rate; number of steps
L, β, λ	Lipschitz, smoothness and strong convexity constants

What you will be able to do

By the end of this part you should be able to do each of the following on paper:

- derive exponential tail bounds for sums of independent variables, and for normal and Laplace noise;
- compute the Kullback–Leibler (KL), Rényi, total-variation and hockey-stick divergences between two normal or two Laplace distributions, and explain why processing data cannot increase any of them;
- derive the optimal test between two distributions and its trade-off curve between the two kinds of error;
- prove convergence rates for stochastic gradient descent, and bound how far a regularized minimizer moves when one training record is replaced;
- prove that a uniformly stable algorithm generalizes;
- compute entropy, mutual information and min-entropy leakage, and say when the last two disagree;
- state a security definition as a game between two worlds, and prove a simple scheme secure with a hybrid argument or a simulator.

Each of these is a result derived in one of the chapters that follow.

1. Probability distributions

A handful of distributions do almost all the work in this book. Counts of events are binomial, and Poisson when the events are rare. Waiting times are exponential, and additive noise is usually normal or Laplace. Every later chapter is stated in terms of these few families, so this chapter asks what each one is, and what its mean, variance, moment generating function (MGF) and tail are.

The idea that organizes the answer is that the families are built from one another. A binomial is a sum of coin flips, and a Poisson is a binomial with many unlikely trials. An exponential is the wait for the first event in that same setting, and a Laplace is two exponentials back to back. A normal is what any sum of many small independent pieces looks like. So each family's moments can be computed from its pieces.

The running example is a classifier tested on $n = 2{,}000$ records, which errs on each one independently with probability $p = 0.05$. It makes about 100 errors, give or take 9.7, and the count is binomial. When errors are rare it is close to a Poisson count, the wait for a first error in deployment is exponential, and the normal approximation says how the measured error rate spreads. The last section compares a normal and a Laplace density with copies of themselves moved sideways, which is where the next chapters begin.

1.1. Moments

Every family below is described by the same few summaries, and the first two say where the distribution sits and how widely it spreads. This section defines them and proves the rules that compute them for sums, which every later section uses.

Definition 1.1 (Mean, variance and covariance)**.** For a random variable X, the **mean** $\mathbb{E}[X]$ is the average of its values weighted by their probabilities. The **variance** is the average squared distance from the mean,

$$\operatorname{Var}(X) = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big] = \mathbb{E}[X^2] - \mathbb{E}[X]^2, \tag{1.1}$$

and its square root is the **standard deviation**. The k-th **moment** of X is $\mathbb{E}[X^k]$. For two variables X and Y, the **covariance** is $\operatorname{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X]\,\mathbb{E}[Y]$.

The two forms of the variance agree, because expanding the square gives $X^2 - 2X\,\mathbb{E}[X] + \mathbb{E}[X]^2$, and averaging each term gives $\mathbb{E}[X^2] - 2\mathbb{E}[X]^2 + \mathbb{E}[X]^2$. Three rules cover almost every calculation in this chapter.

Lemma 1.1 (Means and variances of sums)**.** *Let X and Y be random variables and c a constant. The mean of a sum is the sum of the means, whether or not X and Y are independent. Scaling by c multiplies the variance by c^2. If X and Y are independent, their variances add:*

$$\mathbb{E}[X+Y] = \mathbb{E}[X] + \mathbb{E}[Y], \qquad \operatorname{Var}(cX) = c^2 \operatorname{Var}(X), \qquad \operatorname{Var}(X+Y) = \operatorname{Var}(X) + \operatorname{Var}(Y). \tag{1.2}$$

Proof.

1. *Means add.* An expectation is a weighted sum over outcomes, and the weighted sum of $X+Y$ splits into the weighted sum of X plus that of Y. No step uses independence, and the same splitting gives $\mathbb{E}[cX] = c\,\mathbb{E}[X]$.
2. *Scaling.* Multiplying X by c multiplies its mean by c, so every deviation from the mean is multiplied by c. Squaring the deviation turns that factor into c^2.
3. *Expand the square.* The deviation of $X+Y$ is the sum of the two deviations. Its square is the two squares plus twice their product, and the mean of that product expands to $\mathbb{E}[XY] - \mathbb{E}[X]\,\mathbb{E}[Y]$, the covariance, so

$$\operatorname{Var}(X+Y) = \operatorname{Var}(X) + \operatorname{Var}(Y) + 2\operatorname{Cov}(X,Y). \tag{1.3}$$

4. *Independence removes the cross term.* For independent variables the mean of a product is the product of the means, so the covariance is 0.

□

The third rule needs independence. If $X_1, \dots, X_n$ are all the same variable, their sum is nX_1, whose variance is $n^2 \operatorname{Var}(X_1)$, not $n \operatorname{Var}(X_1)$. With independent copies, the rules give the law that governs every average.

Corollary 1.1 (The square-root law for averages)**.** *The average of n independent copies of X has mean $\mathbb{E}[X]$ and variance $\operatorname{Var}(X)/n$, so its standard deviation is that of one draw divided by $\sqrt{n}$.*

Proof.

1. The average is the sum of the copies times $1/n$. By the first rule of Lemma 1.1 its mean is $n\,\mathbb{E}[X]/n = \mathbb{E}[X]$.
2. By the third rule the sum has variance $n \operatorname{Var}(X)$, and by the second, multiplying by $1/n$ divides that by n^2.

□

Example 1.1. A fair die has mean $(1+\cdots+6)/6 = 3.5$ and mean of the square $(1+4+\cdots+36)/6 = 91/6$. So its variance is $91/6 - 3.5^2 = 35/12 \approx 2.92$, and its standard deviation is about 1.71.

1.2. The moment generating function

The mean and variance summarize a distribution by two numbers. Later sections need more: the distribution of a sum of independent variables, and how fast large values become rare. One function does both jobs, and the idea behind it is to average not X itself but an exponential of it.

Definition 1.2 (Moment generating function)**.** The **moment generating function** of a random variable X is the function

$$M_X(\lambda) = \mathbb{E}[e^{\lambda X}] \tag{1.4}$$

of a real number λ, wherever this average is finite. Its logarithm is written $K_X(\lambda) = \ln M_X(\lambda)$.

The number λ is not a parameter of the distribution but a dial that you set. With $\lambda > 0$ large values of X dominate the average, and at $\lambda = 0$ every weight is 1, so $M_X(0) = 1$.

Example 1.2. One test record of the running example is a coin $X \sim \text{Bern}(p)$, which is 1 when the classifier errs, with probability p, and 0 otherwise. To compute $\mathbb{E}[e^{\lambda X}]$, list each outcome, the value $e^{\lambda X}$ takes there, and its probability (Table 1.1).

Table 1.1.: The MGF of a coin, outcome by outcome.

Outcome x	$\Pr[X = x]$	$e^{\lambda x}$
0	$1 - p$	1
1	p	e^{λ}

An expectation adds up the value at each outcome times its probability, so

$$M_X(\lambda) = (1 - p) \cdot 1 + p \cdot e^{\lambda} = 1 + p(e^{\lambda} - 1). \tag{1.5}$$

So the e^{λ} in the MGFs of all the counting distributions below is the weight of the outcome 1. With $p = 0.3$ and $\lambda = 1$, for instance, $M_X(1) = 0.7 + 0.3e \approx 1.52$.

The MGF gets its name from the first thing it does. Its derivatives at 0 are the moments, and the derivatives of its logarithm are the mean and the variance directly.

Proposition 1.1 (The MGF generates the moments)**.** *If M_X is finite on an interval around 0, then its k-th derivative at 0 is the k-th moment, $M_X^{(k)}(0) = \mathbb{E}[X^k]$. The first two derivatives of K_X at 0 are the mean and the variance,*

$$K_X'(0) = \mathbb{E}[X], \qquad K_X''(0) = \text{Var}(X). \tag{1.6}$$

Proof. The plan is to expand the exponential as a power series and read the moments off its coefficients, then to differentiate the logarithm.

1. *Expand.* Put $u = \lambda X$ in the power series $e^u = 1 + u + u^2/2! + \cdots$ and average term by term, using the first rule of Lemma 1.1:

$$M_X(\lambda) = 1 + \lambda\,\mathbb{E}[X] + \frac{\lambda^2}{2!}\mathbb{E}[X^2] + \frac{\lambda^3}{3!}\mathbb{E}[X^3] + \cdots. \tag{1.7}$$

2. *Differentiate k times at 0.* Differentiating removes the lower powers of λ, and setting $\lambda = 0$ removes the higher ones. Only the term $\lambda^k\,\mathbb{E}[X^k]/k!$ is left, and it contributes $\mathbb{E}[X^k]$.
3. *First derivative of the logarithm.* The chain rule gives $K'_X = M'_X/M_X$, and at 0, where $M_X = 1$, this is $\mathbb{E}[X]$.
4. *Second derivative.* The quotient rule gives $K''_X = M''_X/M_X - (M'_X/M_X)^2$. At 0 this is $\mathbb{E}[X^2] - \mathbb{E}[X]^2$, the variance.

□

The main reason to use MGFs, though, is that adding independent variables multiplies them, which is the first step of every exponential tail bound in Concentration inequalities.

Lemma 1.2 (MGFs of independent sums multiply)**.** *If X and Y are independent, then at every λ where both MGFs are finite,*

$$M_{X+Y}(\lambda) = M_X(\lambda)\,M_Y(\lambda), \qquad K_{X+Y}(\lambda) = K_X(\lambda) + K_Y(\lambda). \tag{1.8}$$

Proof.

1. The exponential turns the sum into a product, $e^{\lambda(X+Y)} = e^{\lambda X}e^{\lambda Y}$.
2. The two factors are independent because X and Y are, and the mean of a product of independent variables is the product of the means. Taking logarithms turns the product into a sum.

□

To conclude which family a sum belongs to, the chapter needs one more fact: the MGF determines the distribution.

Theorem 1.1 (The MGF determines the distribution)**.** *If X and Y have MGFs that are finite and equal on an interval around 0, then X and Y have the same distribution.*

This is a standard theorem of probability, proved in Blitzstein & Hwang (listed in the references). Its proof needs more analysis than this book uses, so take it as given. Section E.1 in the appendices checks each sum this chapter identifies without it, by summing or integrating directly. The chapter uses the theorem to identify a sum whose MGF, computed as a product, is that of a known family. For heavy-tailed distributions the MGF is infinite for every $\lambda > 0$, and then the theorem says nothing.

The last summary is the **tail** of X, the probability $\Pr[X \geq t]$ as a function of a point t well above the mean. It describes the rare, large outcomes that a worst-case guarantee has to control. At $t = 5$, a polynomial decay $1/t^2$ gives 0.04 and an exponential decay e^{-t} about 6.7×10^{-3}. A Gaussian (normal) decay $e^{-t^2/2}$ gives about 3.7×10^{-6}, because its exponent grows like t^2. The MGF controls the tail in two lines.

Proposition 1.2 (The MGF bounds the tail)**.** *For every t and every $\lambda > 0$ at which $M_X(\lambda)$ is finite,*

$$\Pr[X \geq t] \leq M_X(\lambda)\, e^{-\lambda t}. \tag{1.9}$$

Proof.

1. Write $\mathbf{1}[X \geq t]$ for the indicator that is 1 when $X \geq t$ and 0 otherwise. Whenever $X \geq t$, the weight $e^{\lambda X}$ is at least $e^{\lambda t}$, because $\lambda > 0$, and elsewhere it is still positive. So $e^{\lambda X} \geq e^{\lambda t}\, \mathbf{1}[X \geq t]$ at every outcome.
2. Averaging both sides turns the indicator into a probability, which gives $M_X(\lambda) \geq e^{\lambda t} \Pr[X \geq t]$.

□

So a finite MGF at some $\lambda > 0$ makes the tail decay at least exponentially in t, and Concentration inequalities turns this into explicit bounds by choosing λ well.

1.3. Bernoulli and binomial

The simplest random quantity is a yes/no outcome, such as whether a test record is misclassified. Count the yeses over n independent trials and you get a binomial. Every estimated rate in machine learning, such as an accuracy or a false positive rate, is a binomial count divided by n, so this is the first family the running example needs.

Definition 1.3 (Bernoulli and binomial distributions)**.** A **Bernoulli** variable $X \sim \mathrm{Bern}(p)$ takes the value 1 with probability p and 0 with probability $1 - p$. A **binomial** variable $S \sim \mathrm{Bin}(n, p)$ is the number of ones among n independent $\mathrm{Bern}(p)$ trials.

The binomial's probability mass function (PMF), the table of probabilities of each count, follows by counting sequences of trials. Its moments follow from the rules of Section 1.1, without any sum over that table.

Proposition 1.3 (Binomial PMF and moments)**.** *For $S \sim \mathrm{Bin}(n, p)$ and each count $k = 0, 1, \ldots, n$,*

$$\Pr[S = k] = \binom{n}{k} p^k (1-p)^{n-k}, \tag{1.10}$$

$$\mathbb{E}[S] = np, \qquad \mathrm{Var}(S) = np(1-p), \qquad M_S(\lambda) = \left(1 + p(e^{\lambda} - 1)\right)^n. \tag{1.11}$$

Proof. The plan is to count sequences for the PMF, then to compute the moments of one trial and combine them over the n trials.

1. *One sequence.* A particular sequence with exactly k ones has probability $p^k(1-p)^{n-k}$, because the trials are independent and each one contributes p or $1 - p$.
2. *All sequences.* There are $\binom{n}{k}$ such sequences, one for each choice of the k positions that hold the ones. They are disjoint events, so their probabilities add.

3. *One trial.* A Bernoulli has mean p. Since $X^2 = X$, its mean square is also p, so its variance is $p - p^2 = p(1-p)$. Its MGF is $1 + p(e^\lambda - 1)$ by Example 1.2.
4. *Combine the trials.* Write $S = X_1 + \cdots + X_n$ with independent Bernoullis. By Lemma 1.1 the means add to np and the variances to $np(1-p)$. By Lemma 1.2 the MGF is the product of n identical Bernoulli MGFs.

□

The variance $p(1-p)$ is largest at $p = \frac{1}{2}$ and small near 0 or 1, so in absolute terms a rate near 0 is estimated more precisely than one near $\frac{1}{2}$. The estimated rate $\hat{p} = S/n$ is an average of n independent trials, so by Corollary 1.1 it has standard deviation $\sqrt{p(1-p)/n}$.

Example 1.3. The classifier errs on each test record independently with probability $p = 0.05$. On $n = 2{,}000$ records its error count is $\mathrm{Bin}(2000, 0.05)$, with mean $2000 \times 0.05 = 100$. Its standard deviation is $\sqrt{2000 \times 0.05 \times 0.95} \approx 9.7$, so the measured rate has standard deviation about $9.7/2000 \approx 0.005$. So the measured error rate is typically within about half a percentage point of the true 5%.

A better classifier, with $p = 0.001$, makes 2 errors on average, with standard deviation $\sqrt{2000 \times 0.001 \times 0.999} \approx 1.4$. Now the *relative* error, 1.4/2, is about 70%, and a single test set can easily show no errors at all. The probability of that is $0.999^{2000} \approx e^{-2}$, about one test set in seven. In general $(1-p)^n \approx e^{-np}$, because $1 - p \approx e^{-p}$ for small p, so you need n on the order of $1/p$ before you expect to see any error.

1.4. Poisson

In the running example at $p = 0.001$, the classifier makes about 2 errors in 2,000 records: many trials, each unlikely. In that regime the binomial depends on n and p only through the mean np, and its limit is the Poisson distribution, the law of rare events.

Definition 1.4 (Poisson distribution)**.** A **Poisson** variable $N \sim \mathrm{Poisson}(\mu)$ with mean $\mu > 0$ takes the values $k = 0, 1, 2, \ldots$ with probabilities

$$\Pr[N = k] = e^{-\mu} \frac{\mu^k}{k!}. \tag{1.12}$$

The factor $e^{-\mu}$ makes the probabilities add up to 1, since $\sum_k \mu^k/k! = e^\mu$. The limit that produces this distribution holds the expected count μ fixed while the number of trials grows.

Theorem 1.2 (Poisson limit of the binomial)**.** *Fix a mean $\mu > 0$ and a count $k \geq 0$. As $n \to \infty$,*

$$\Pr\left[\mathrm{Bin}(n, \mu/n) = k\right] \to e^{-\mu} \frac{\mu^k}{k!}. \tag{1.13}$$

Proof. The plan is to write the binomial PMF at $p = \mu/n$ as a product of four factors, each of which has a simple limit.

1. *Split into four factors.* Write $\binom{n}{k} = n(n-1)\cdots(n-k+1)/k!$ and group the terms:

$$\binom{n}{k}\left(\frac{\mu}{n}\right)^k\left(1-\frac{\mu}{n}\right)^{n-k} = \frac{n(n-1)\cdots(n-k+1)}{n^k}\cdot\frac{\mu^k}{k!}\cdot\left(1-\frac{\mu}{n}\right)^n\cdot\left(1-\frac{\mu}{n}\right)^{-k}. \tag{1.14}$$

2. *The first factor tends to 1.* It is a product of k fractions $n/n, (n-1)/n, \ldots$, each tending to 1. The second factor does not depend on n.
3. *The third factor tends to $e^{-\mu}$.* Its logarithm is $n\ln(1-\mu/n)$. Since $\ln(1-u) = -u + O(u^2)$ for small u, that is $-\mu + O(1/n)$.
4. *The fourth factor tends to 1.* It has a fixed exponent and a base that tends to 1.

□

Figure 1.1 shows the convergence. At $n = 10$ the binomial is visibly narrower than the Poisson, while at $n = 100$ the two almost coincide, and the gap shrinks like $1/n$.

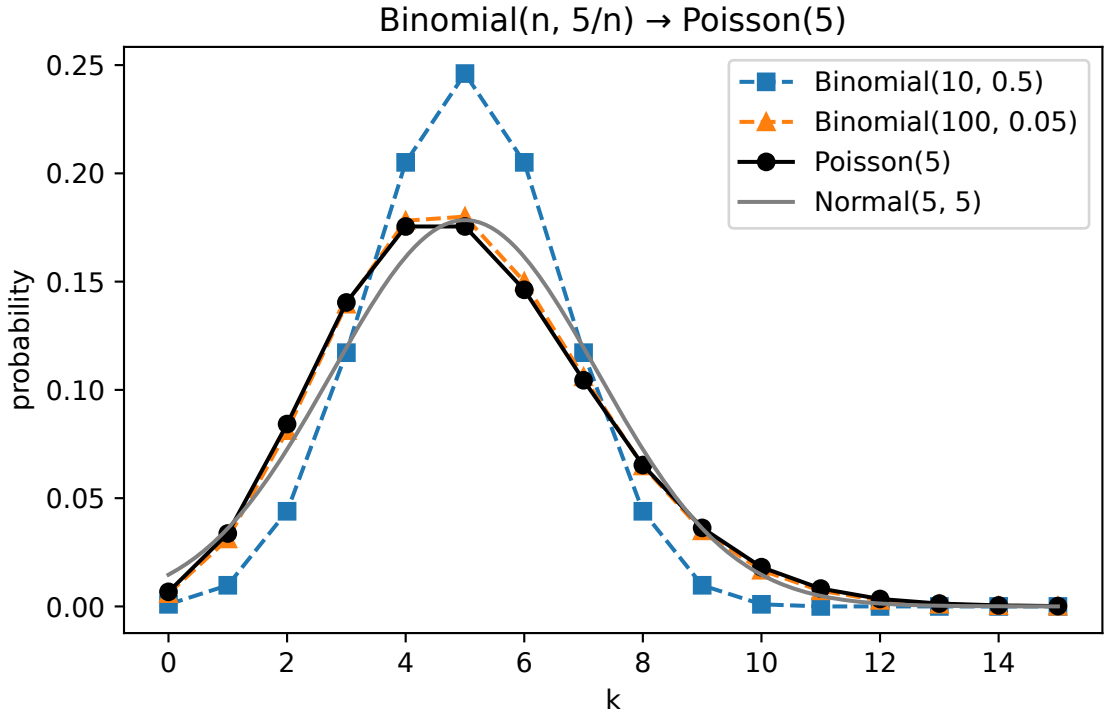


Figure 1.1.: $\mathrm{Bin}(n, 5/n)$ for $n = 10$ and 100, against Poisson(5) and its normal approximation.

Proposition 1.4 (Poisson MGF, moments and sums)**.** *For $N \sim \mathrm{Poisson}(\mu)$,*

$$M_N(\lambda) = \exp\left(\mu(e^\lambda - 1)\right), \qquad \mathbb{E}[N] = \mathrm{Var}(N) = \mu. \tag{1.15}$$

If $N_1 \sim \mathrm{Poisson}(\mu_1)$ and $N_2 \sim \mathrm{Poisson}(\mu_2)$ are independent, then $N_1 + N_2 \sim \mathrm{Poisson}(\mu_1 + \mu_2)$.

Proof.

1. *The MGF.* Weight each count k by $e^{\lambda k}$. Gathering the powers into $(\mu e^{\lambda})^k$ leaves the power series of the exponential at μe^{λ}:

$$M_N(\lambda) = e^{-\mu} \sum_{k \geq 0} \frac{(\mu e^{\lambda})^k}{k!} = e^{-\mu} e^{\mu e^{\lambda}}. \tag{1.16}$$

2. *Mean and variance.* So $K_N(\lambda) = \mu(e^{\lambda} - 1)$, all of whose derivatives equal μ at 0. By Proposition 1.1 the mean and the variance are both μ.
3. *Sums.* By Lemma 1.2 the MGF of $N_1 + N_2$ is the product of the two MGFs, $\exp\left((\mu_1 + \mu_2)(e^{\lambda} - 1)\right)$. That is the Poisson$(\mu_1 + \mu_2)$ MGF, so Theorem 1.1 identifies the sum.

□

The binomial variance $np(1-p)$ tends to μ as well, because $1-p$ tends to 1. The spread relative to the mean, $1/\sqrt{\mu}$, shrinks only once you expect many events, so with $\mu = 4$ a count of 0 or 8, two standard deviations of 2 from the mean, is unremarkable. Far above the mean the probabilities fall like $(e\mu/k)^k$, faster than any exponential but slower than a normal tail, as Section E.3 derives.

The Chernoff bound in Concentration inequalities bounds the tail of a count of independent yes/no events through its MGF, as in Proposition 1.2. Its first step is that, as far as the MGF can tell, such a count is dominated by a Poisson count with the same mean.

Proposition 1.5 (A Bernoulli sum is dominated by a Poisson count)**.** *Let $X_1, \dots, X_n$ be independent, with $X_i \sim \text{Bern}(p_i)$, where the probabilities p_i may differ, and let $\mu = p_1 + \cdots + p_n$. For every λ,*

$$M_{X_1 + \cdots + X_n}(\lambda) \leq \exp\left(\mu(e^{\lambda} - 1)\right). \tag{1.17}$$

Proof.

1. *The tangent line.* The inequality $1 + x \leq e^x$ holds for every x, because the exponential is convex and $1 + x$ is its tangent line at 0, which a convex curve lies above.
2. *One trial.* With $x = p_i(e^{\lambda} - 1)$ it bounds the Bernoulli MGF $1 + p_i(e^{\lambda} - 1)$ by $\exp\left(p_i(e^{\lambda} - 1)\right)$.
3. *The sum.* By Lemma 1.2 the MGF of the sum is the product of the n positive Bernoulli MGFs. Multiplying their bounds adds the exponents, to $\mu(e^{\lambda} - 1)$.

□

Example 1.4. Some training procedures form a minibatch by including each of $n = 60{,}000$ records independently with probability $q = 0.01$. The batch size is Bin$(60000, 0.01)$, so by Theorem 1.2 it is very close to Poisson(600), with mean $60000 \times 0.01 = 600$ and standard deviation $\sqrt{600}$, about 24.5. So batch sizes between 550 and 650, about two standard deviations either side, are typical. The classifier's errors at $p = 0.001$ are the same kind of count, close to Poisson(2).

1.5. Exponential

The Poisson counts how many rare events land in a fixed window. Turn the question around: how long until the first one, such as a deployed classifier's first error? The answer is the exponential distribution, which comes out of the same construction as the Poisson and is the building block of the Laplace.

The construction chops time into short slots. Fix a scale $b > 0$, which will turn out to be the average wait, and split each unit of time into n slots. As the slots get finer, each must get less likely to hold an event. So let each slot hold an event independently with probability $p = 1/(bn)$, which keeps the expected number of events per unit of time at $1/b$.

Theorem 1.3 (The waiting time in the Poisson limit)**.** *Let each slot of width $1/n$ hold an event independently with probability $1/(bn)$, and let T_n be the time of the first event. For every time $t \geq 0$, as $n \to \infty$,*

$$\Pr[T_n > t] \to e^{-t/b}. \tag{1.18}$$

Proof. The plan is to find the wait in slots, convert it to time, and rewrite the result in the form whose limit Theorem 1.2 already found.

1. *The wait in slots is geometric.* Let G be the index of the first slot that holds an event. The event $G > j$ means that the first j slots are all empty, and independent slots multiply:

   $$\Pr[G > j] = (1-p)^j, \qquad j = 0, 1, 2, ... \tag{1.19}$$

   This is the **geometric distribution** $\mathrm{Geom}(p)$, the number of coin flips up to and including the first head (Table 1.2).

   Table 1.2.: The waiting time G, in slots, until the first event.

j	Slots until the first event	$\Pr[G = j]$	$\Pr[G > j]$
1	event	p	$1-p$
2	empty, event	$(1-p)p$	$(1-p)^2$
3	empty, empty, event	$(1-p)^2p$	$(1-p)^3$

2. *Convert slots to time.* Each slot lasts $1/n$, so $T_n = G/n$. Time t covers $\lfloor nt \rfloor$ whole slots, where the floor rounds down to a whole number, so

   $$\Pr[T_n > t] = \left(1 - \frac{1}{bn}\right)^{\lfloor nt \rfloor}. \tag{1.20}$$

3. *Rewrite.* The goal is the form $(1-\mu/m)^m$, whose limit $e^{-\mu}$ appeared in step 3 of the proof of Theorem 1.2. Write the exponent as $nt - r$, where $0 \leq r < 1$ is the fractional part of nt, and write $\frac{1}{bn}$ as $\frac{t/b}{nt}$:

   $$\left(1 - \frac{1}{bn}\right)^{\lfloor nt \rfloor} = \left(1 - \frac{t/b}{nt}\right)^{nt} \cdot \left(1 - \frac{1}{bn}\right)^{-r}. \tag{1.21}$$

4. *Take the limit.* The first factor is $(1-\mu/m)^m$ with $\mu = t/b$ and $m = nt$, so it tends to $e^{-t/b}$. The second has a bounded exponent and a base that tends to 1.

□

Counting gives the same answer. The window $[0, t]$ holds about nt slots with $1/(bn)$ chance each, so by Theorem 1.2 its count of events tends to Poisson(t/b), which is 0 with probability $e^{-t/b}$. The limit has $\Pr[T \le t] = 1 - e^{-t/b}$, and its derivative is the density that defines the family.

Definition 1.5 (Exponential distribution)**.** A variable $Y \sim \mathrm{Exp}(b)$ with **scale** $b > 0$ has density

$$f(y) = \frac{1}{b} e^{-y/b}, \qquad y \ge 0. \tag{1.22}$$

The rate of events is $1/b$. Figure 1.2 shows the limit for $b = 2$. With one slot per unit of time the curve is a staircase that halves at every integer, and with five it already hugs $e^{-t/2}$. Table 1.3 gives the values at $t = 4$ from the chapter's script (Section 1.10). The first two follow from step 2 of the proof: $(\frac{1}{2})^4 = 0.0625$, and with five slots the probability per slot is 0.1, so $0.9^{20} = 0.1216$.

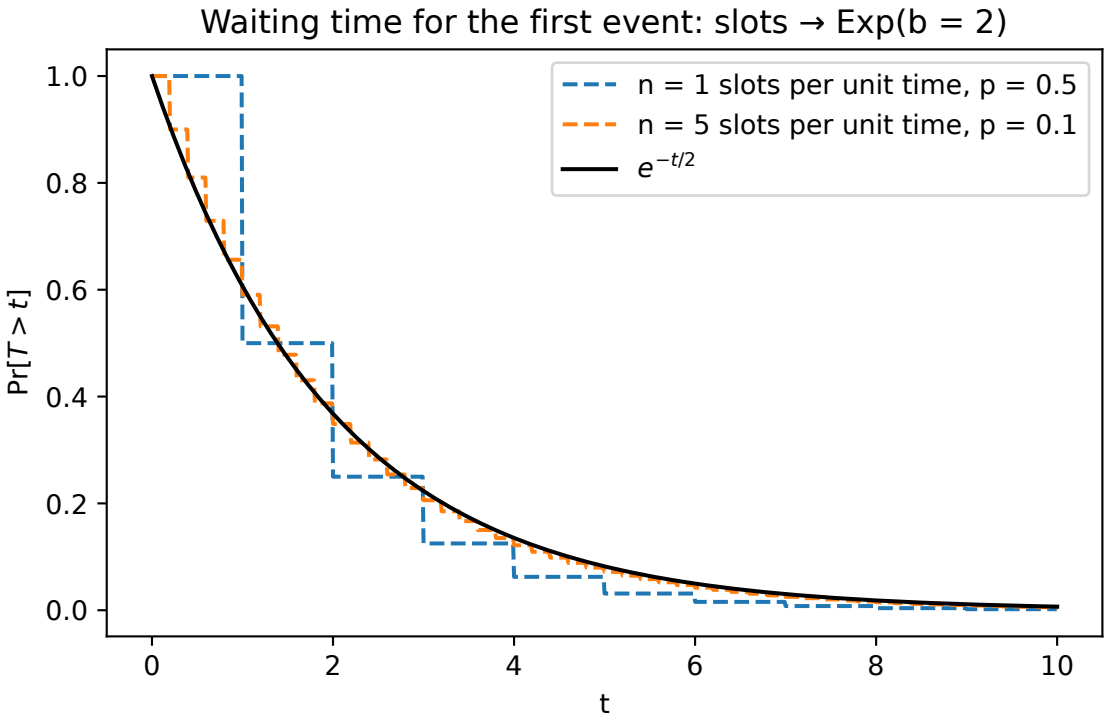


Figure 1.2.: $\Pr[T_n > t]$ for the slotted waiting time with $b = 2$ and $n = 1$ or 5 slots per unit of time, against the exponential tail $e^{-t/2}$.

Table 1.3.: The slotted waiting time with $b = 2$. The probability is exact; the mean and variance are over 1,000,000 draws (seed 0).

Slots per unit of time	$\Pr[T_n > 4]$	Mean of T_n	Variance of T_n
1	0.0625	1.998	2.001
5	0.1216	2.000	3.596

Slots per unit of time	$\Pr[T_n > 4]$	Mean of T_n	Variance of T_n
100	0.1347	2.001	3.984
Limit	0.1353	2	4

The simulated mean is close to b for every n, and the variance approaches b^2 as the slots shrink. The exponential's other summaries follow from its density by short integrals.

Proposition 1.6 (Exponential tail, moments and MGF)**.** *For $Y \sim \mathrm{Exp}(b)$, every $t \geq 0$, every $k = 1, 2, \ldots$ and every $\lambda < 1/b$,*

$$\Pr[Y \geq t] = e^{-t/b}, \qquad \mathbb{E}[Y^k] = k!\, b^k, \qquad M_Y(\lambda) = \frac{1}{1 - b\lambda}. \tag{1.23}$$

So $\mathbb{E}[Y] = b$ and $\mathrm{Var}(Y) = b^2$, and $M_Y(\lambda)$ is infinite for $\lambda \geq 1/b$.

Proof.

1. *Tail.* Integrating the density from t to infinity gives $e^{-t/b}$, which is Equation 1.18 again.
2. *Moments.* The substitution $u = y/b$, then k integrations by parts, give the Gamma integral

$$\int_0^\infty y^k e^{-y/b}\,\mathrm{d}y = b^{k+1} \int_0^\infty u^k e^{-u}\,\mathrm{d}u = k!\, b^{k+1}. \tag{1.24}$$

   With the factor $\frac{1}{b}$ of the density, $\mathbb{E}[Y^k] = k!\, b^k$, so the mean is b and the variance $2b^2 - b^2 = b^2$.
3. *MGF.* Multiplying the density by $e^{\lambda y}$ combines the exponents into $-(1/b - \lambda)y$, which is negative for $\lambda < 1/b$:

$$M_Y(\lambda) = \frac{1}{b} \int_0^\infty e^{-(1/b - \lambda)y}\,\mathrm{d}y = \frac{1}{1 - b\lambda}. \tag{1.25}$$

   For $\lambda \geq 1/b$ the integrand does not shrink, and the integral diverges.

□

The MGF exists only on part of the line, which is the hallmark of an exponential tail rather than a Gaussian one. The exponential also forgets how long you have waited.

Proposition 1.7 (Memorylessness)**.** *For $Y \sim \mathrm{Exp}(b)$ and all times $s, t \geq 0$,*

$$\Pr[Y \geq s + t \mid Y \geq s] = \Pr[Y \geq t]. \tag{1.26}$$

Proof.

1. The event $Y \geq s + t$ lies inside the event $Y \geq s$, so the conditional probability is the ratio of the two tails.
2. By Proposition 1.6 that ratio is $e^{-(s+t)/b} / e^{-s/b} = e^{-t/b}$.

□

In the slotted picture, the slots after time s are independent of the empty ones before it, so the wait starts afresh.

Example 1.5. A deployed classifier serves 50 requests per second and misclassifies each one independently with probability 10^{-4}. Each request is a slot, so errors arrive at rate $50 \times 10^{-4} = 0.005$ per second, and the wait is approximately Exp(200) in seconds. So a 10-minute window is clean with probability $e^{-600/200} = e^{-3} \approx 0.050$. The exact answer, $(1 - 10^{-4})^{30000}$ with one factor per request, agrees to three decimals. By Proposition 1.7, after 10 quiet minutes the expected wait is still 200 seconds.

1.6. Laplace

The Laplace distribution is symmetric noise with exponential tails. In machine learning it is the noise model behind absolute-error loss, and in later parts it is one of the two standard noise distributions. This section builds it from the exponential, which gives its moments almost for free.

Definition 1.6 (Laplace distribution)**.** A variable $X \sim \mathrm{Lap}(\mu, b)$ with **location** μ and **scale** $b > 0$ has density

$$f(x) = \frac{1}{2b} e^{-|x-\mu|/b}. \tag{1.27}$$

We write $\mathrm{Lap}(b)$ for $\mathrm{Lap}(0, b)$.

The density is an exponential density on each side of the centre, mirrored, with each side carrying half the probability. So its cumulative distribution function (CDF) is $\frac{1}{2}e^{(x-\mu)/b}$ left of the centre and $1 - \frac{1}{2}e^{-(x-\mu)/b}$ right of it.

Proposition 1.8 (Laplace tail, moments and MGF)**.** *Let $X \sim \mathrm{Lap}(b)$. Its size $|X|$ has the* Exp(b) *distribution, so*

$$\Pr[|X| \geq t] = e^{-t/b}, \qquad \mathbb{E}[|X|] = b, \qquad \mathrm{Var}(X) = 2b^2. \tag{1.28}$$

For $|\lambda| < 1/b$ its MGF is

$$M_X(\lambda) = \frac{1}{1 - b^2\lambda^2}. \tag{1.29}$$

Proof.

1. *The size is exponential.* The density takes the same value at x and $-x$, so on $x \geq 0$ the density of $|X|$ is $2 \cdot \frac{1}{2b}e^{-x/b} = \frac{1}{b}e^{-x/b}$, the Exp$(b)$ density. Its tail and mean are then those of Proposition 1.6.
2. *Variance.* Since $X^2 = |X|^2$, the mean square is the exponential second moment $2b^2$, and the mean is 0 by symmetry.
3. *MGF.* Split the integral at 0. The right half is $\frac{1}{2b} \cdot \frac{1}{1/b - \lambda}$, which needs $\lambda < 1/b$, and the left half is the same with $-\lambda$ in place of λ. Over the common denominator $1/b^2 - \lambda^2$,

$$M_X(\lambda) = \frac{1}{2b}\Big(\frac{1}{1/b - \lambda} + \frac{1}{1/b + \lambda}\Big) = \frac{1}{1 - b^2\lambda^2}. \tag{1.30}$$

□

So a Laplace variable with variance σ^2 has scale $b = \sigma/\sqrt{2}$, and by symmetry one side alone has half of the tail. Since the Laplace is two exponentials back to back, it can also be built by subtracting one exponential from another.

Proposition 1.9 (A Laplace variable is a difference of exponentials)**.** *If* $Y_1, Y_2 \sim \text{Exp}(b)$ *are independent, then* $Y_1 - Y_2 \sim \text{Lap}(b)$.

Proof.

1. *Negation flips the dial.* The MGF of $-Y_2$ at λ is $M_{Y_2}(-\lambda)$.
2. *Multiply.* For $|\lambda| < 1/b$, Lemma 1.2 and Proposition 1.6 give the MGF of $Y_1 - Y_2$:

$$M_{Y_1}(\lambda)\, M_{Y_2}(-\lambda) = \frac{1}{1 - b\lambda} \cdot \frac{1}{1 + b\lambda} = \frac{1}{1 - b^2\lambda^2}. \tag{1.31}$$

3. *Identify.* That is the Laplace MGF of Proposition 1.8, so Theorem 1.1 gives $Y_1 - Y_2 \sim \text{Lap}(b)$.

□

Over a million samples, the empirical CDF of $\text{Exp}(1) - \text{Exp}(1)$ is within 0.0006 of the $\text{Lap}(1)$ CDF everywhere on $[-5, 5]$ (Figure 1.3).

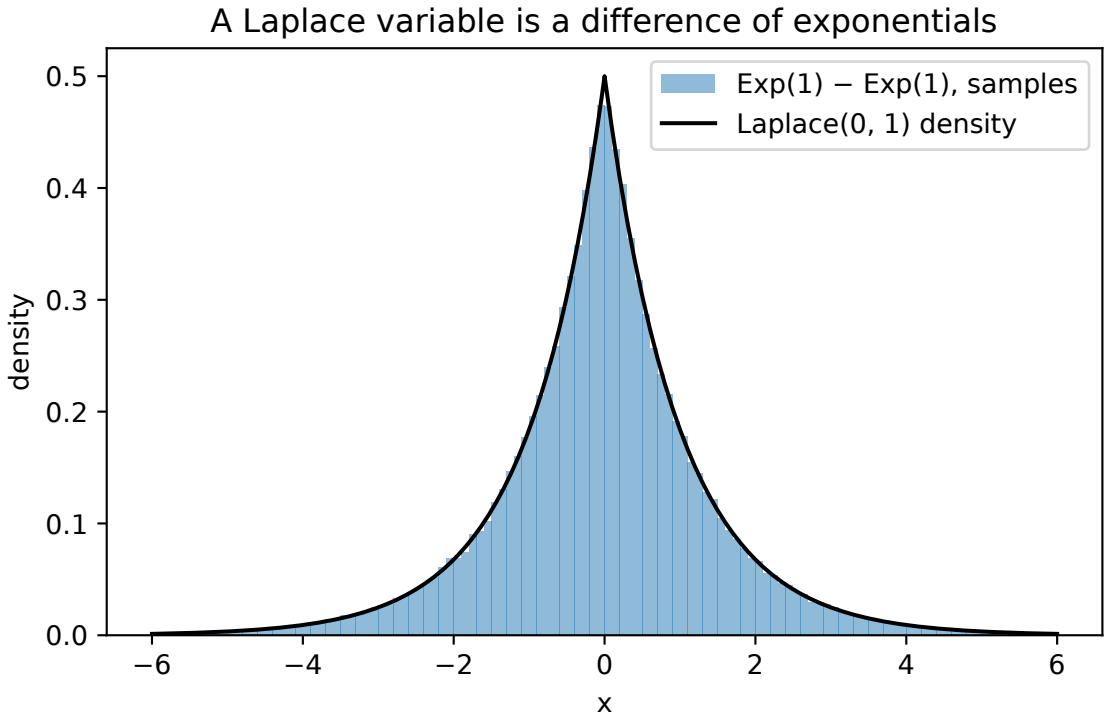


Figure 1.3.: Histogram of $\text{Exp}(1) - \text{Exp}(1)$ samples against the $\text{Lap}(0, 1)$ density.

Put a Laplace with $b = 1$ and a normal (Section 1.7) with standard deviation $\sqrt{2}$ side by side, both with variance 2 (Figure 1.4). The Laplace has a sharp peak at 0 and fatter tails. On a log scale the difference is structural, because the normal's log density is a parabola, $-x^2/(2\sigma^2)$, while the Laplace's is two straight lines, $-|x|/b$. Far out, a parabola always falls below a line. At $3b$, for instance, the Laplace has $\Pr[|X| \geq 3b] = e^{-3} \approx 0.050$, while for the normal with the same variance, $3b$ is $3/\sqrt{2}$ standard deviations and the probability is about 0.034.

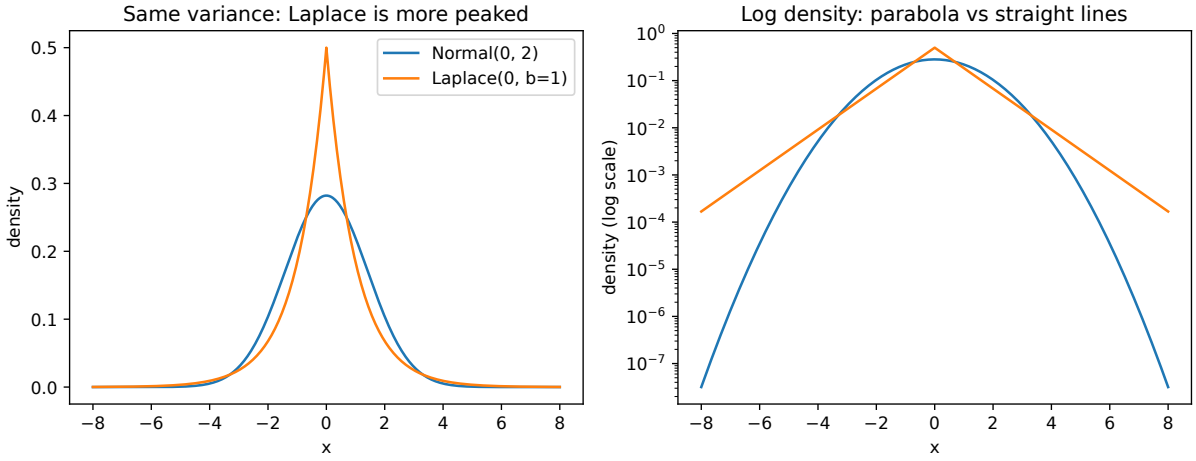


Figure 1.4.: Normal and Laplace densities with variance 2, on linear and log scales.

Example 1.6. The classifier's error counts are not Laplace, so this example shows what the Laplace model of noise implies when you fit a model. Suppose $y_i = \theta^\top x_i + \xi_i$, with parameters θ and independent noise $\xi_i \sim \text{Lap}(b)$. The log-likelihood of the data is the sum of the log densities of the residuals,

$$-n \ln(2b) - \frac{1}{b} \sum_i |y_i - \theta^\top x_i|. \tag{1.32}$$

Only the second term depends on θ, so the maximum-likelihood θ, the one that makes the observed data most probable, minimizes the sum of absolute errors. When the model is a single constant m in place of $\theta^\top x_i$, the best m is the *median*, where half the points lie on each side. The median is far less sensitive to outliers than the mean, because moving one extreme point further out does not change which point is in the middle.

1.7. The normal distribution

The normal distribution appears whenever many small independent effects add up, and it is the default model for measurement noise. This section derives its MGF and shows that it is closed under sums. It then states the central limit theorem (CLT), the reason the normal appears everywhere, including in the running example's error count.

Definition 1.7 (Normal distribution)**.** A variable $X \sim \mathcal{N}(\mu, \sigma^2)$ with mean μ and standard deviation $\sigma > 0$ has density

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}. \tag{1.33}$$

The **standard normal** $\mathcal{N}(0,1)$ has density φ and CDF Φ.

Subtracting the mean and dividing by σ gives a standard normal, so $\Pr[X \le x] = \Phi((x-\mu)/\sigma)$. There is no closed form for Φ. In code, compute it as $\Phi(x) = \frac{1}{2}\operatorname{erfc}(-x/\sqrt{2})$ with the complementary error function, which stays accurate far into the tail, where $1-\Phi$ would lose all its digits.

The MGF integrates $e^{\lambda x}$ against the density, so the two exponents add. The trick is to recognize their sum as the exponent of another normal density.

Proposition 1.10 (Normal MGF)**.** *For $X \sim \mathcal{N}(\mu, \sigma^2)$ and every real λ,*

$$M_X(\lambda) = e^{\mu\lambda + \sigma^2\lambda^2/2}. \tag{1.34}$$

So $K_X(\lambda) = \mu\lambda + \sigma^2\lambda^2/2$, whose third and higher derivatives are 0.

Proof.

1. *Centre first.* Take $\mu = 0$, and complete the square in the combined exponent:
$$\lambda x - \frac{x^2}{2\sigma^2} = -\frac{(x-\lambda\sigma^2)^2}{2\sigma^2} + \frac{\lambda^2\sigma^2}{2}. \tag{1.35}$$
2. *Integrate.* The first term is the exponent of a normal density centred at $\lambda\sigma^2$, which integrates to 1 against the normalizer, so only the factor $e^{\lambda^2\sigma^2/2}$ survives.
3. *Add the mean.* A mean μ adds the constant μ to X, which multiplies $e^{\lambda X}$ by $e^{\lambda\mu}$.
4. *Higher derivatives.* So K_X is a quadratic, whose third and higher derivatives are 0.

□

Proposition 1.11 (Linear combinations of independent normals)**.** *For independent $X_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$ and constants a_i,*

$$\sum_i a_i X_i \sim \mathcal{N}\Big(\sum_i a_i\mu_i,\ \sum_i a_i^2\sigma_i^2\Big). \tag{1.36}$$

Proof.

1. *Scaling.* Scaling X_i by a_i evaluates its MGF at $a_i\lambda$.
2. *Adding.* By Lemma 1.2 the MGF of the sum is the product, which adds the exponents of Proposition 1.10:
$$M_{\sum_i a_i X_i}(\lambda) = \exp\Big(\lambda\sum_i a_i\mu_i + \frac{\lambda^2}{2}\sum_i a_i^2\sigma_i^2\Big). \tag{1.37}$$
3. *Identify.* That is a normal MGF, so Theorem 1.1 identifies the sum.

□

The Laplace has no such property. By Lemma 1.2 the sum of two independent Lap(b) variables has MGF $1/(1-b^2\lambda^2)^2$, which is not of the form $1/(1-b'^2\lambda^2)$ for any scale b', so the sum is not Laplace.

The running example's error count is a sum of 2,000 independent pieces. By Lemma 1.1, a sum of n independent draws with mean μ and variance σ^2 has mean $n\mu$ and standard deviation $\sigma\sqrt{n}$, so standardizing it always leaves mean 0 and variance 1. The CLT says more: whatever the distribution of each draw, the *shape* of the standardized sum approaches the standard normal.

Theorem 1.4 (Central limit theorem)**.** *Let $X_1, X_2, \ldots$ be independent and identically distributed (i.i.d.) with mean μ and finite variance $\sigma^2 > 0$. Then for every z, as $n \to \infty$,*

$$\Pr\left[\frac{X_1 + \cdots + X_n - n\mu}{\sigma\sqrt{n}} \le z\right] \to \Phi(z). \tag{1.38}$$

The proof needs more than this chapter's tools. Section E.4 in the appendices proves the theorem by swapping one term at a time for a normal one, with a slower rate than the one below. The best way to see that the shape of each draw does not matter is to start from one that looks nothing like a normal.

Take $X_i \sim \text{Exp}(1)$, with mean and variance 1, which is badly skewed. The sum $S_n = X_1 + \cdots + X_n$ has the Gamma$(n, 1)$ density $x^{n-1}e^{-x}/(n-1)!$, whose MGF $(1-\lambda)^{-n}$ is the product of n exponential MGFs. So you can watch the standardized sum change shape as n grows (Figure 1.5). At $n = 1$ it has a hard wall at -1, from the constraint $X_1 \ge 0$. At $n = 30$ the curve is close to $\mathcal{N}(0, 1)$, though a small lean remains.

The lean is measured by the **skewness** $\mathbb{E}[(X-\mu)^3]/\sigma^3$, the third central moment (the mean cubed deviation) over the cubed standard deviation, which is 0 for every normal. The third central moment adds over independent terms, because each cross term in the cube of a sum of deviations contains some deviation $X_i - \mu$ to the first power, which has mean 0 and is independent of the rest. For Exp(1), expanding $(Y-1)^3$ and using $\mathbb{E}[Y^k] = k!$ from Proposition 1.6 gives $6 - 6 + 3 - 1 = 2$. So S_n has third central moment $2n$ and standard deviation $\sqrt{n}$, and skewness $2n/n^{3/2} = 2/\sqrt{n}$. The largest gap between the CDF of the standardized sum and Φ, computed exactly from the Gamma CDF, falls at the same rate (Table 1.4).

Table 1.4.: How far the standardized sum of n Exp(1) variables is from $\mathcal{N}(0, 1)$. The CDF gap is the maximum of $|F_n(z) - \Phi(z)|$ over $z \in [-5, 5]$ in steps of 0.01.

n	Skewness $2/\sqrt{n}$	Largest CDF gap to $\mathcal{N}(0,1)$
1	2.000	0.1587
2	1.414	0.0945
5	0.894	0.0596
30	0.365	0.0243

Multiply each gap by $\sqrt{n}$ and you get 0.159, 0.134, 0.133 and 0.133, so the error decays like $1/\sqrt{n}$. In general the Berry–Esseen theorem bounds the gap by $c\,\mathbb{E}[|X-\mu|^3]/(\sigma^3\sqrt{n})$ for a constant $c \le 0.4748$ (Shevtsova, 2011). The CLT is a statement about the *bulk*, the region within a few standard deviations of the mean. In the far tails it can be badly wrong, because a gap of order

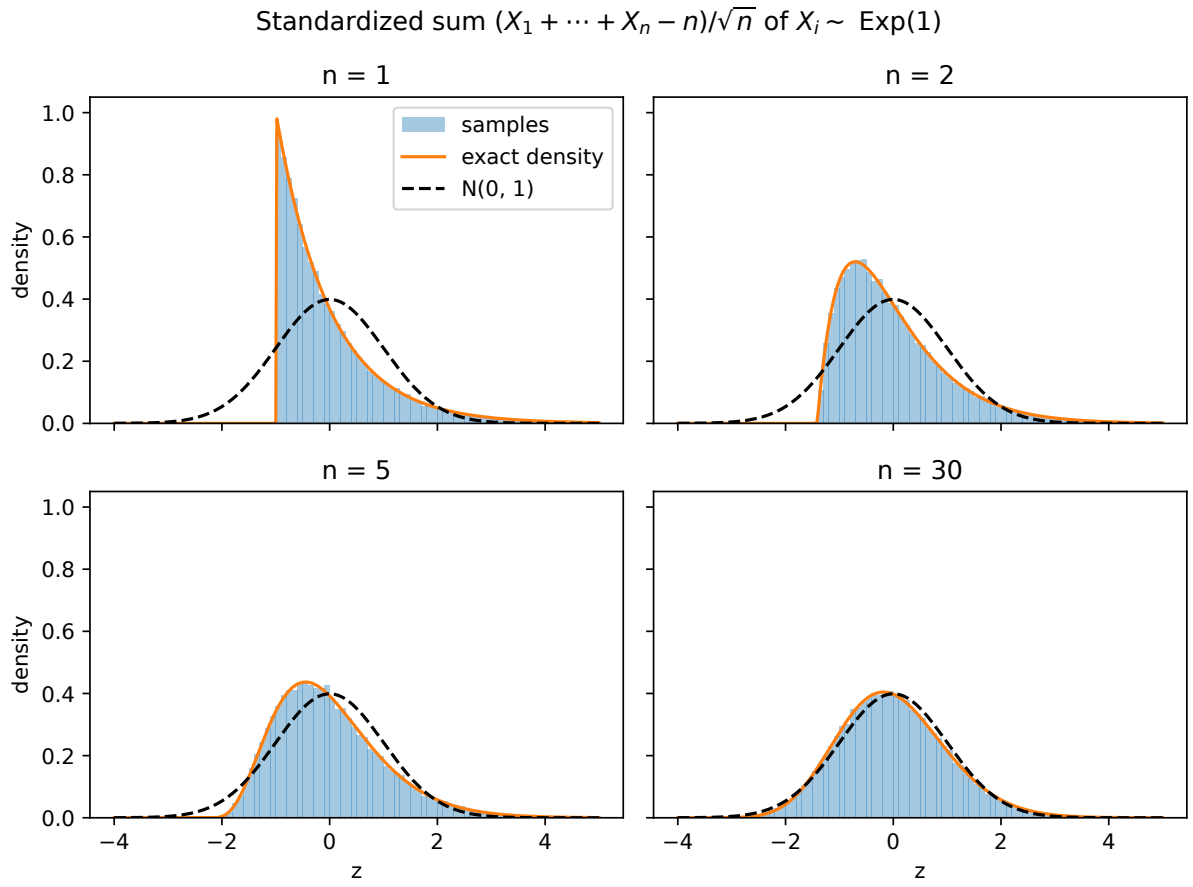


Figure 1.5.: The standardized sum of n independent Exp(1) variables for $n = 1, 2, 5, 30$: histogram of 50,000 simulated sums, the exact density from Gamma$(n, 1)$, and the $\mathcal{N}(0, 1)$ density.

$1/\sqrt{n}$ is far larger than the tiny tail probabilities themselves, and bounding those is the job of the next chapter.

Example 1.7. A common rule of thumb is that the normal approximation to a binomial is reasonable once $np(1-p) \gtrsim 10$. The running example's error count has $np(1-p) = 95$, so in the bulk it is close to $\mathcal{N}(100, 95)$, with the standard deviation 9.7 of Example 1.3. A normal variable is within 1.96 standard deviations of its mean with probability $\Phi(1.96) - \Phi(-1.96) = 0.95$, so about 95% of test sets show 81 to 119 errors. At $p = 0.001$, by contrast, $np(1-p)$ is about 2, and the count is better described by Poisson(2).

1.8. Normal vectors in many dimensions

A random weight initialization, or noise added to every weight of a model, is a normal vector with many coordinates. Later parts use two facts about it: it looks the same in every direction, and its length is almost fixed. Write $X \sim \mathcal{N}(0, \sigma^2 I_d)$ for a vector whose d coordinates are independent $\mathcal{N}(0, \sigma^2)$, where I_d is the $d \times d$ identity matrix. The first fact turns a question about one direction into a one-dimensional question.

Proposition 1.12 (Rotation invariance)**.** *Let $X \sim \mathcal{N}(0, \sigma^2 I_d)$ and let u be a unit vector, one with $\sum_i u_i^2 = 1$. Whatever the direction u, the projection $\langle u, X \rangle = \sum_i u_i X_i$ has the $\mathcal{N}(0, \sigma^2)$ distribution.*

Proof.

1. The projection is a linear combination of independent normals, so by Proposition 1.11 it is normal with mean 0 and variance $\sigma^2 \sum_i u_i^2 = \sigma^2$.

□

The second fact is about the length. The squared length $\|X\|^2 = \sum_i X_i^2$ is a sum of d independent terms, so its mean and variance are d times those of one term.

Proposition 1.13 (The norm concentrates)**.** *For $X \sim \mathcal{N}(0, \sigma^2 I_d)$,*

$$\mathbb{E}[\|X\|^2] = d\sigma^2, \qquad \mathrm{Var}\left(\|X\|^2\right) = 2d\sigma^4. \tag{1.39}$$

So the ratio $\|X\|^2/(d\sigma^2)$ has mean 1 and standard deviation $\sqrt{2/d}$.

Proof. The plan is to find the mean and variance of one squared coordinate, which needs the fourth moment of a standard normal, and then to add over coordinates.

1. *One coordinate.* Write $X_i = \sigma Z$ with Z standard normal. Then $\mathbb{E}[X_i^2] = \sigma^2$ and $\mathrm{Var}(X_i^2) = \sigma^4(\mathbb{E}[Z^4] - 1)$.
2. *The fourth moment.* The MGF of Z expands as $e^{\lambda^2/2} = 1 + \lambda^2/2 + \lambda^4/8 + \cdots$. By Proposition 1.1, $\mathbb{E}[Z^4]$ is 4! times the coefficient of λ^4, which is $24/8 = 3$, so $\mathrm{Var}(X_i^2) = 2\sigma^4$.
3. *Add the coordinates.* By Lemma 1.1 the d independent squares add to mean $d\sigma^2$ and variance $2d\sigma^4$.

4. *Divide by the mean.* By the scaling rule, the ratio has standard deviation $\sqrt{2d\sigma^4}/(d\sigma^2) = \sqrt{2/d}$.

□

In high dimension a normal vector is almost never near the origin, because it lives on a thin shell of radius $\sigma\sqrt{d}$. The length itself has a relative spread of about $1/\sqrt{2d}$, half that of the squared length, because a square root halves small relative deviations, $\sqrt{1+e} \approx 1 + e/2$. The script measures $\|X\|/\sqrt{d}$ for $\sigma = 1$ (Table 1.5 and Figure 1.6).

Table 1.5.: Mean and standard deviation of $\|X\|/\sqrt{d}$ over 2,000 draws of $X \sim \mathcal{N}(0, I_d)$.

d	1	10	100	1,000	10,000
mean of $\|X\|/\sqrt{d}$	0.802	0.977	0.998	1.000	1.000
std of $\|X\|/\sqrt{d}$	0.611	0.220	0.069	0.022	0.007

At $d = 10{,}000$, for instance, $1/\sqrt{20000} \approx 0.007$, and at $d = 1$ the exact mean is $\mathbb{E}[|Z|] = \sqrt{2/\pi} \approx 0.798$. The mean approaches 1 from below because the square root is concave. Jensen's inequality says that $\mathbb{E}[g(W)] \le g(\mathbb{E}[W])$ for a concave g, and with $W = \|X\|^2$ it gives $\mathbb{E}[\|X\|] \le \sqrt{d}$.

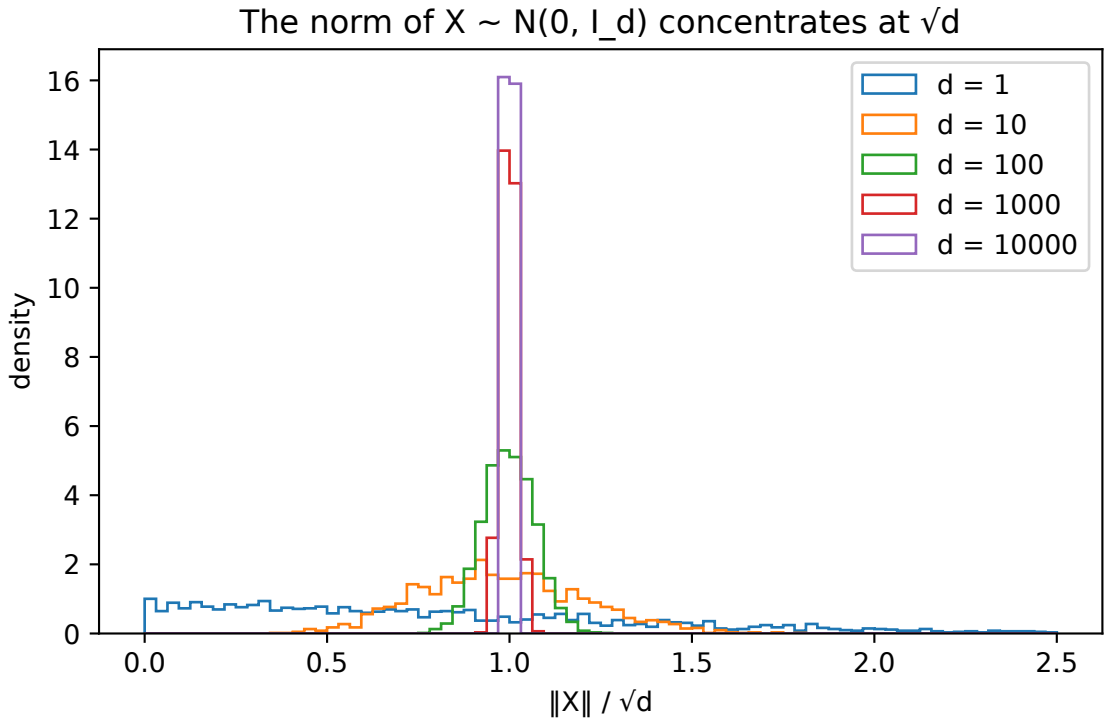


Figure 1.6.: Histograms of $\|X\|/\sqrt{d}$ for $X \sim \mathcal{N}(0, I_d)$ and d from 1 to 10,000.

Example 1.8. The error counts are not vectors, so take the classifier's weights instead. Add $\mathcal{N}(0, \sigma^2 I_d)$ noise to a model with $d = 10^6$ parameters and $\sigma = 0.01$. By Proposition 1.13 the noise vector has length about $\sigma\sqrt{d} = 10$, with a relative spread $1/\sqrt{2 \times 10^6}$ under 0.1%. Yet by Proposition 1.12 its projection onto any fixed unit vector is only $\mathcal{N}(0, 0.01^2)$. The same two facts show that two independent random directions are almost orthogonal.

1.9. A density against its shifted copy

You take a noisy measurement $x = \mu + X$ of an unknown value μ, such as a sensor reading. The noise X has density f centred at 0, and μ can only be 0 or a known **shift** s. This section asks how strongly one observation can favour one candidate, and the normal and the Laplace give opposite answers. Divergences and hypothesis testing build on that answer.

If $\mu = 0$, then x has density $f(x)$. If $\mu = s$, the CDF of $s + X$ at x is $\Pr[X \le x - s] = F(x - s)$, where F is the CDF of X. Differentiating gives the density $f(x - s)$, the same curve moved right by s. The ratio $f(x)/f(x - s)$ says how many times more likely x is under $\mu = 0$ than under $\mu = s$, and its logarithm turns the ratio into a difference.

Definition 1.8 (Log ratio of a density and its shifted copy)**.** For a density f and a shift s, the **log ratio** at an observation x is

$$L(x) = \ln \frac{f(x)}{f(x - s)} = \ln f(x) - \ln f(x - s). \tag{1.40}$$

If the log ratio is bounded, no single observation can make one candidate much more likely than the other. The answer follows from the shapes of the log densities. A normal log density is a parabola, and a parabola minus its shifted copy is a straight line, which grows without bound. A Laplace log density is a pair of straight lines with slopes $\pm 1/b$, and shifting it changes its value at any point by at most $|s|/b$.

Theorem 1.5 (Normal and Laplace against their shifted copies)**.** *Let s be any real shift. If f is the $\mathcal{N}(0, \sigma^2)$ density, then*

$$L(x) = \frac{s^2 - 2sx}{2\sigma^2}, \tag{1.41}$$

which is linear in x and unbounded when $s \neq 0$. If f is the Lap(b) *density, then*

$$L(x) = \frac{|x - s| - |x|}{b}, \qquad |L(x)| \le \frac{|s|}{b} \quad \text{for every } x, \tag{1.42}$$

and for $s > 0$ the bound is reached: $L(x) = s/b$ for every $x \le 0$, and $L(x) = -s/b$ for every $x \ge s$.

Proof. The plan is to cancel the constants in each ratio and simplify the difference of the exponents, then, for the Laplace, to bound that difference above and below with the triangle inequality.

1. *Normal: the ratio.* Replacing x by $x - s$ changes only the exponent, so the constants cancel, and the logarithm of a ratio of exponentials is the difference of the exponents:

$$L(x) = \frac{(x - s)^2 - x^2}{2\sigma^2}. \tag{1.43}$$

2. *Normal: expand the square.* Since $(x-s)^2 = x^2 - 2sx + s^2$, the x^2 cancels, which leaves $(s^2 - 2sx)/(2\sigma^2)$. A line with nonzero slope $-s/\sigma^2$ has no bound.
3. *Laplace: the ratio.* As for the normal, the constants cancel and $L(x) = (|x-s| - |x|)/b$.
4. *Laplace: bound from above.* The triangle inequality $|a+c| \le |a| + |c|$ with $a = x$ and $c = -s$ gives $|x-s| \le |x| + |s|$, so $|x-s| - |x| \le |s|$.
5. *Laplace: bound from below.* The same inequality with $a = x - s$ and $c = s$ gives $|x| \le |x-s| + |s|$, so $|x-s| - |x| \ge -|s|$.
6. *Laplace: equality.* Take $s > 0$. For $x \le 0$ both x and $x - s$ are at most 0, so $|x-s| - |x| = (s-x) - (-x) = s$. For $x \ge s$ both are at least 0, and the difference is $(x-s) - x = -s$.

□

So two Laplace copies are within the factor $e^{|s|/b}$ of each other everywhere, while two normal copies have an unbounded ratio in the tails. There is very little probability in those tails, though. Since the normal log ratio is linear in x, it is itself normal when x is drawn from f.

Corollary 1.2 (The normal log ratio is normal)**.** *If f is the $\mathcal{N}(0, \sigma^2)$ density and the observation x is drawn from f, then*

$$L(x) \sim \mathcal{N}\Big(\frac{s^2}{2\sigma^2},\ \frac{s^2}{\sigma^2}\Big). \tag{1.44}$$

Proof.

1. By Theorem 1.5, $L(x)$ is the constant $s^2/(2\sigma^2)$ minus the multiple s/σ^2 of x.
2. By Proposition 1.11 with a single term, that multiple is normal with mean 0 and variance $(s/\sigma^2)^2\sigma^2 = s^2/\sigma^2$, and adding the constant moves the mean.

□

Example 1.9. Table 1.6 puts numbers on Theorem 1.5 for $s = 1$, $\sigma = 1$ and $b = 1$.

Table 1.6.: The log ratio $L(x)$ of a density and its copy shifted by $s = 1$, for $\mathcal{N}(0, 1)$ and Lap(1).

x	-2	0	0.5	1	3	10
Normal, $L(x) = (1 - 2x)/2$	2.5	0.5	0	-0.5	-2.5	-9.5
Laplace, $L(x) = \lvert x-1\rvert - \lvert x\rvert$	1	1	0	-1	-1	-1

Both vanish at the midpoint $x = 0.5$, where the two densities cross. At $x = 10$ the normal log ratio is -9.5, so the observation is $e^{9.5} \approx 13{,}360$ times more likely under $\mu = 1$ than under $\mu = 0$. The Laplace never says more than $e^1 \approx 2.72$ times (Figure 1.7). Such far-out observations are rare, because by Corollary 1.2 the normal log ratio under $\mu = 0$ is $\mathcal{N}(0.5, 1)$. The next three chapters make "rare" precise.

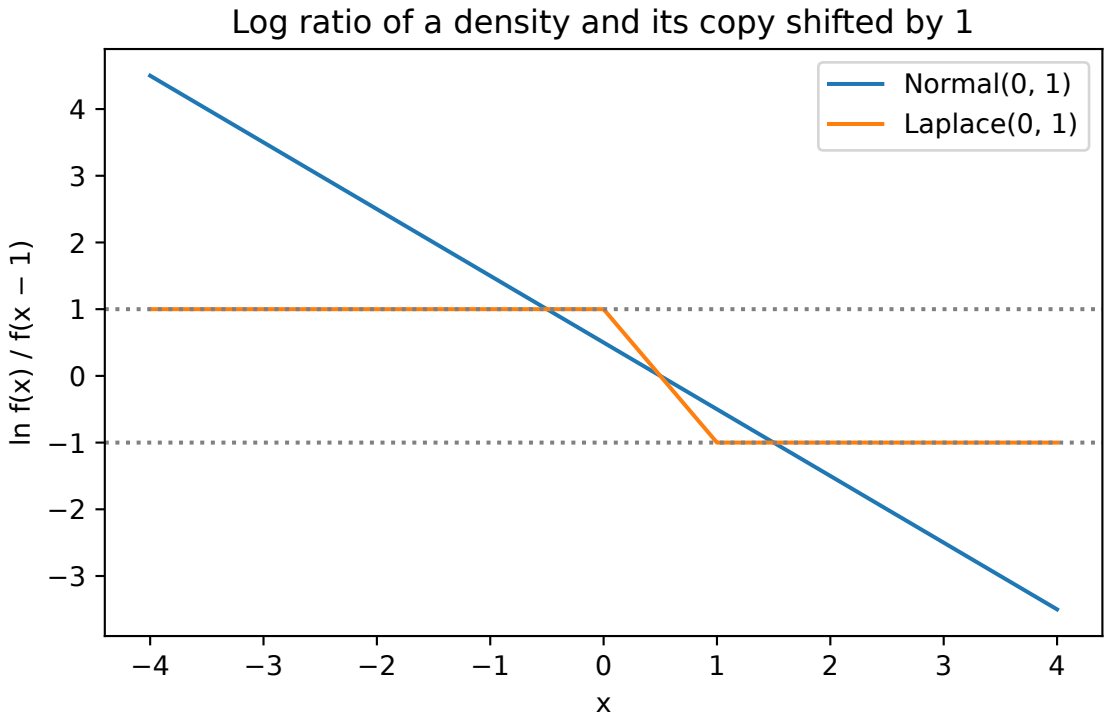


Figure 1.7.: Log ratio $L(x)$ of a density and its copy shifted by $s = 1$, for the standard normal and Lap(1).

1.10. Code

The script code/distributions.py draws every figure in this chapter and checks the formulas against simulation. From the repository root:

```
uv run python code/part01-prerequisites/distributions.py
```

Table 1.7.: Mean and variance from the formulas against 1,000,000 samples (seed 0). The script's other checks are reported in their sections.

Distribution	Mean (formula / sample)	Variance (formula / sample)
Bern(0.3)	0.300 / 0.301	0.210 / 0.210
Bin(20, 0.3)	6.000 / 6.001	4.200 / 4.198
Poisson(4)	4.000 / 3.998	4.000 / 4.001
Exp(2)	2.000 / 2.001	4.000 / 4.019
Lap(0, 2)	0.000 / −0.000	8.000 / 7.993
$\mathcal{N}(1, 3^2)$	1.000 / 1.002	9.000 / 8.977

tests/test_distributions.py checks that the output still matches.

1.11. Summary

Table 1.8.: The results of this chapter, and where each is used.

Result	Statement	Use
Lemma 1.1	Means add; $\mathrm{Var}(cX) = c^2 \mathrm{Var}(X)$; independent variances add	Moments of every sum
Corollary 1.1	Averages of n copies: standard deviation over $\sqrt{n}$	Four times the data halves the error
Proposition 1.1	$K'_X(0) = \mathbb{E}[X]$, $K''_X(0) = \mathrm{Var}(X)$	Moments in one line
Lemma 1.2	$M_{X+Y} = M_X M_Y$ for independent X, Y	Sums; every Chernoff bound
Theorem 1.1	Equal MGFs near 0, equal distributions	Identify a sum from its MGF
Proposition 1.2	$\Pr[X \geq t] \leq M_X(\lambda) e^{-\lambda t}$	A finite MGF means an exponential tail
Proposition 1.3	Mean np, variance $np(1-p)$	Error counts; rates near 0 are precise

Result	Statement	Use
Theorem 1.2	$\mathrm{Bin}(n, \mu/n) \to \mathrm{Poisson}(\mu)$	Rare events, with error of order $1/n$
Proposition 1.4	Mean and variance μ; Poissons add	Sampled batch sizes, spread $\sqrt{\mu}$
Proposition 1.5	Bernoulli-sum MGF $\leq e^{\mu(e^{\lambda}-1)}$	First step of the Chernoff bound
Theorem 1.3	Waiting time $\to \mathrm{Exp}(b)$	Time to a first event
Proposition 1.6	$\mathbb{E}[Y^k] = k!\, b^k$	Building block of the Laplace
Proposition 1.7	The wait starts afresh	Waiting after a quiet period
Proposition 1.8	$\lvert X\rvert \sim \mathrm{Exp}(b)$, variance $2b^2$	Scale $b = \sigma/\sqrt{2}$ for variance σ^2
Proposition 1.9	$\mathrm{Exp}(b) - \mathrm{Exp}(b) \sim \mathrm{Lap}(b)$	Sampling Laplace noise
Proposition 1.10	$M_X(\lambda) = e^{\mu\lambda + \sigma^2\lambda^2/2}$	Gaussian tails
Proposition 1.11	Independent normals combine into a normal	The Laplace does not
Theorem 1.4	Standardized i.i.d. sums tend to $\mathcal{N}(0,1)$	The bulk, once $np(1-p) \gtrsim 10$; never the far tails
Proposition 1.12	Each projection is $\mathcal{N}(0, \sigma^2)$	Noise is small in every direction
Proposition 1.13	$\lVert X\rVert \approx \sigma\sqrt{d}$, relative spread $1/\sqrt{2d}$	Noise is large in length
Theorem 1.5	Normal $L(x)$ unbounded; Laplace $\lvert L(x)\rvert \leq \lvert s\rvert/b$	Shifted Laplace copies differ by a bounded factor
Corollary 1.2	$L \sim \mathcal{N}(s^2/(2\sigma^2), s^2/\sigma^2)$	Large normal log ratios are rare

1.12. References

- Blitzstein & Hwang, *Introduction to Probability* (2nd ed., 2019), ch. 3–5 and 7: every distribution here, with worked examples, and the proofs of Theorem 1.1 and the central limit theorem.
- Wasserman, *All of Statistics* (2004), ch. 2–3: a compact reference.
- Vershynin, *High-Dimensional Probability* (2018), §3.1: concentration of the norm of a normal vector.
- Shevtsova, "On the absolute constants in the Berry–Esseen type inequalities for identically distributed summands" (2011): the constant $c \leq 0.4748$ in the Berry–Esseen bound.

2. Concentration inequalities

Almost every number in machine learning is random. A test error depends on which examples happened to land in the test set, and the size of a randomly sampled minibatch changes from one step to the next. Before you rely on such a number, you need to know **how far a random quantity can be from what you expect, and with what probability**. Concentration inequalities answer that question.

The whole subject is one idea: to bound the chance that a quantity lands far out, you compare it with something whose average you know, and the more you know, the sharper the comparison. The mean alone gives Markov's inequality, whose bound decays like $1/t$ at a distance t out, and the variance gives Chebyshev's, which decays like $1/t^2$. The **moment generating function (MGF)**, an average of an exponential of the quantity, gives the Chernoff bound, which decays exponentially, and the later sections specialize it to bounded variables, to random noise added to a number, and to sums of dependent terms.

The running example is a classifier whose true error rate is 10%. You test it on $n = 200$ examples drawn independently, and its measured error rate is the fraction of them it gets wrong. How likely is the measured rate to reach 15%, half again its true value? A simulation of 200,000 test sets says 9.6×10^{-3}. The bounds of this chapter put that probability at most at 0.67 (Markov), 0.18 (Chebyshev), 0.37 (Hoeffding) and 0.135 (Chernoff). Each section explains one number, including why Hoeffding's does worse than Chebyshev's here.

2.1. Markov and Chebyshev

Start with the crudest version of the question. If all you know about a quantity is its mean, or its mean and its variance, how often can it land far from where you expect? The answers are weak, but every stronger bound in this chapter is the same argument applied to a better function.

The argument turns a probability into an average, and a die shows how. Write $\mathbf{1}[A]$ for the **indicator** of an event A, the variable that is 1 on outcomes where A happens and 0 elsewhere. Its average is $\Pr[A]$, because averaging a 0/1 variable counts the fraction of outcomes where the event happens. Averages also keep order: if one variable is at most another on every outcome, its average is at most the other's average. Now roll a fair die, let X be the roll, and compare $5 \cdot \mathbf{1}[X \geq 5]$ with X itself, outcome by outcome (Table 2.1).

Table 2.1.: Outcome-by-outcome comparison of $5 \cdot \mathbf{1}[X \geq 5]$ with the die roll X.

Roll	1	2	3	4	5	6	Average
$5 \cdot \mathbf{1}[X \geq 5]$	0	0	0	0	5	5	$5\Pr[X \geq 5]$
X	1	2	3	4	5	6	3.5

Every entry in the top row is at most the one below it, so the averages are ordered as well, and $5\Pr[X \geq 5] \leq 3.5$. Dividing by 5 gives $\Pr[X \geq 5] \leq 0.7$, while the true value is $1/3$. The bound is weak, but it used only the mean (Figure 2.1).

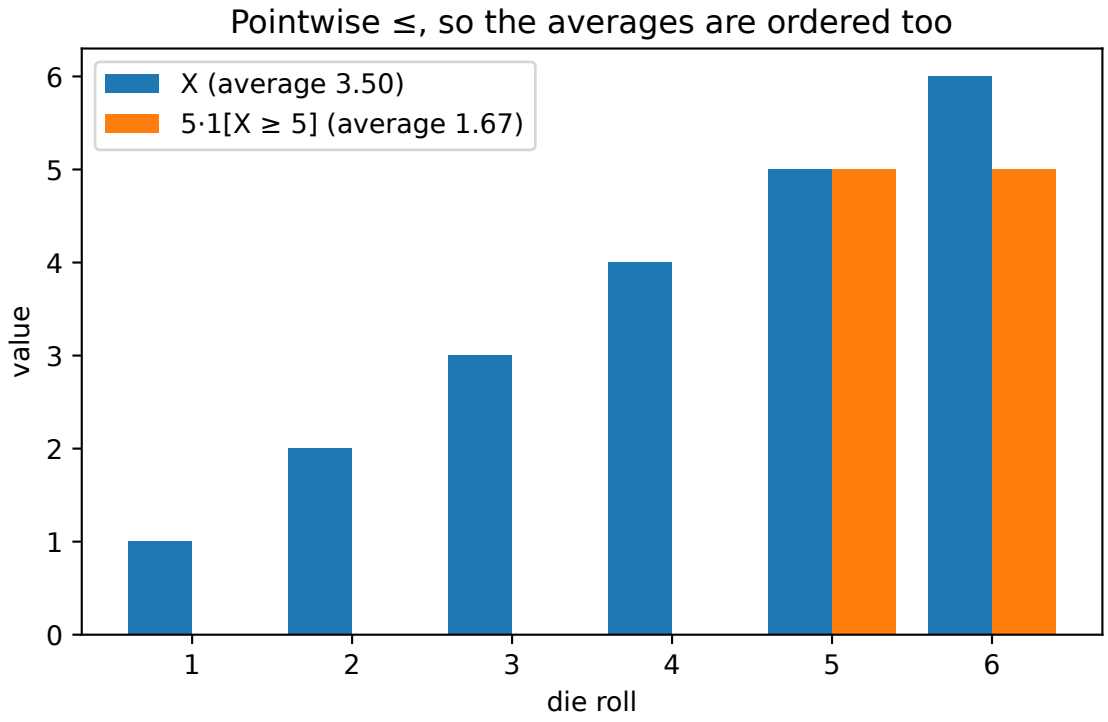


Figure 2.1.: The scaled indicator $5 \cdot \mathbf{1}[X \geq 5]$ never exceeds the die roll X, so its average never exceeds 3.5.

Markov's inequality is the same comparison with any threshold in place of 5, and it needs the quantity never to be negative, because negative values could balance large ones in the average.

Theorem 2.1 (Markov's inequality)**.** *Let X be a random variable that is never negative, and let $a > 0$ be a threshold. Then*

$$\Pr[X \geq a] \leq \frac{\mathbb{E}[X]}{a}. \tag{2.1}$$

Proof.

1. *Compare outcome by outcome.* If $X \geq a$, the scaled indicator $a\,\mathbf{1}[X \geq a]$ equals a, which is at most X. If $X < a$, it equals 0, which is at most X because X is never negative. This is the only place the hypothesis is used, and it gives, at every outcome,

$$a\,\mathbf{1}[X \geq a] \leq X. \tag{2.2}$$

2. *Average both sides.* Averages keep a pointwise order, and the average of the indicator is the probability of its event, so

$$a\Pr[X \geq a] \leq \mathbb{E}[X]. \tag{2.3}$$

Dividing by $a > 0$ gives the bound.

□

No bound that uses only the mean can do better. When $\mathbb{E}[X] \leq a$, the variable that equals a with probability $\mathbb{E}[X]/a$ and 0 otherwise has that mean and meets the bound with equality.

Markov's inequality applies to any quantity that is never negative, so you can apply it to a transformed version of X instead of X itself.

Corollary 2.1 (Markov's inequality through a function)**.** *Let X be a random variable, let g be a function that is positive and strictly increasing, and let t be a real number. Then*

$$\Pr[X \geq t] \leq \frac{\mathbb{E}[g(X)]}{g(t)}. \tag{2.4}$$

Proof.

1. An increasing function keeps the order of numbers, so $X \geq t$ and $g(X) \geq g(t)$ are the same event.
2. The variable $g(X)$ is never negative and the threshold $g(t)$ is positive, so Theorem 2.1 applies to $g(X)$ at the threshold $a = g(t)$.

□

The rest of the chapter is a search for good functions g, which grow fast but keep $\mathbb{E}[g(X)]$ computable. The square gives Chebyshev's inequality below, and the exponential gives the Chernoff bound of Section 2.2.

A tail bound answers the question "how likely is a deviation of size t?". In practice you usually want the reverse: how large a deviation must you allow so that it is exceeded with probability at most β? That number is the **width** at failure probability β, found by setting the bound equal to β and solving for t. Most bounds here have the form $e^{-h(t)}$, so the lemma below does this once.

Lemma 2.1 (From a tail bound to a width)**.** *Let V be a random deviation, such as the distance of a measured average from its mean, and let h be a continuous, strictly increasing function such that for every t*

$$\Pr[V \geq t] \leq e^{-h(t)}. \tag{2.5}$$

Let $\beta \in (0, 1)$ be a failure probability such that $\ln(1/\beta)$ is a value of h, so that $h^{-1}(\ln(1/\beta))$ exists. Then, with probability at least $1 - \beta$,

$$V < h^{-1}(\ln(1/\beta)). \tag{2.6}$$

Proof.

1. At the deviation $t^* = h^{-1}(\ln(1/\beta))$ the exponent is $h(t^*) = \ln(1/\beta)$, so the bound equals $e^{-\ln(1/\beta)} = \beta$.
2. So the event $V \geq t^*$ has probability at most β, and its complement $V < t^*$ has probability at least $1 - \beta$.

□

Chebyshev's inequality applies Markov's inequality to a square. The **variance** σ^2 is the average squared distance from the mean μ. A deviation of t or more makes the squared distance at least t^2, and if that happened often, the average squared distance would exceed σ^2.

Theorem 2.2 (Chebyshev's inequality)**.** *Let X have mean μ and variance σ^2, and let $t > 0$. Then*

$$\Pr[|X - \mu| \geq t] \leq \frac{\sigma^2}{t^2}. \tag{2.7}$$

If $\bar{X}$ is the mean of n independent copies of X, then for every $\beta \in (0, 1)$, with probability at least $1 - \beta$,

$$|\bar{X} - \mu| < \frac{\sigma}{\sqrt{n\beta}}. \tag{2.8}$$

Proof. The plan is to rewrite the two-sided event as an event about a square, apply Markov's inequality to the square, and then repeat the bound for a mean of samples and invert it.

1. *Square the event.* The distance $|X - \mu|$ is at least t exactly when its square $(X - \mu)^2$ is at least t^2, because squaring keeps the order of numbers that are not negative.
2. *Apply Markov.* The square $(X-\mu)^2$ is never negative, and its average is σ^2 by the definition of the variance. So Theorem 2.1 at the threshold t^2 gives the first bound.
3. *The variance of a mean.* Variances add for independent terms, because the covariances between them vanish, and dividing by n divides the variance by n^2, as Probability distributions shows. So $\bar{X}$ has mean μ and variance σ^2/n, and step 2 applied to $\bar{X}$ gives

$$\Pr[|\bar{X} - \mu| \geq t] \leq \frac{\sigma^2}{nt^2}. \tag{2.9}$$

4. *Invert.* Setting $\sigma^2/(nt^2) = \beta$ and solving gives $t = \sigma/\sqrt{n\beta}$, the width.

□

Only step 3 used independence, and only to make the covariances vanish, so pairwise uncorrelated samples suffice. The price of knowing only the variance shows in the width. For a single variable, setting $\sigma^2/t^2 = \beta$ gives $t = \sigma/\sqrt{\beta}$. Most tails of later sections have the Gaussian shape $e^{-t^2/(2\sigma^2)}$, so Lemma 2.1 with $h(t) = t^2/(2\sigma^2)$ gives $t = \sigma\sqrt{2\ln(1/\beta)}$, and Table 2.2 compares the two in units of σ.

Table 2.2.: Width needed to reach failure probability β, in units of σ, with only the variance known (Chebyshev) and with a Gaussian-shaped tail.

β	Chebyshev, $1/\sqrt{\beta}$	Gaussian-shaped, $\sqrt{2\ln(1/\beta)}$
0.05	4.5	2.4
10^{-3}	31.6	3.7
10^{-6}	1,000	5.3

The Chebyshev width grows like a power of $1/\beta$, the Gaussian-shaped one only like the square root of a logarithm. So polynomial bounds are fine for a quick 95% error bar, but useless when the failure probability must be tiny, as it must be when many estimates have to hold at once (Section 2.4).

Example 2.1. The measured error rate of the running example is the mean $\bar{X}$ of $n = 200$ independent errors, each 1 with probability 0.1 and 0 otherwise. Its mean is 0.1, and one error has variance $0.1 \times 0.9 = 0.09$. Markov's inequality at the threshold 0.15 gives $0.1/0.15 = 0.67$. Chebyshev's inequality with the deviation $t = 0.05$ gives

$$\Pr[|\bar{X} - 0.1| \geq 0.05] \leq \frac{0.09}{200 \times 0.05^2} = 0.18, \tag{2.10}$$

which covers the upper tail you ask about, since reaching 0.15 is one way to be 0.05 away. Against the simulated 9.6×10^{-3}, Markov is about 70 times too large and Chebyshev about 19 times, because neither uses more than one or two numbers about the distribution.

2.2. The Chernoff method

Failure probabilities like 10^{-5} are out of reach for polynomial bounds at sensible widths, as Table 2.2 showed. This section reaches them by applying Corollary 2.1 with an exponential, $g(x) = e^{\lambda x}$, for a chosen $\lambda > 0$. The average of $e^{\lambda X}$ is the MGF $M_X(\lambda) = \mathbb{E}[e^{\lambda X}]$ of Probability distributions, which contains every moment of X at once.

A larger λ weights large values of X more, which helps, but also enlarges $M_X(\lambda)$, which hurts. With the **log-MGF** $K_X(\lambda) = \ln M_X(\lambda)$, the bound is e to minus an exponent, and the best λ makes that exponent largest.

Theorem 2.3 (The Chernoff bound)**.** *Let X be a random variable, let t be a real number, and let $\lambda > 0$ be such that $M_X(\lambda)$ is finite. Then*

$$\Pr[X \geq t] \leq e^{-\lambda t} M_X(\lambda) = \exp\big(-(\lambda t - K_X(\lambda))\big). \tag{2.11}$$

Since this holds for every such λ,

$$\Pr[X \geq t] \leq \exp\Big(-\sup_{\lambda > 0} \big(\lambda t - K_X(\lambda)\big)\Big). \tag{2.12}$$

Proof.

1. For $\lambda > 0$ the function $e^{\lambda x}$ is positive and strictly increasing, so Corollary 2.1 applies with $g(x) = e^{\lambda x}$. Its numerator is $M_X(\lambda)$ and its denominator is $e^{\lambda t}$, which gives the first form.
2. Writing $M_X(\lambda) = e^{K_X(\lambda)}$ gathers both factors into one exponent.
3. The bound holds for every allowed λ, so it holds for the smallest of the bounds, which is the one with the largest exponent.

□

For large sums the best exponent is also the right one. For a sum of n independent copies of a variable, Cramér's theorem says that the probability of a large deviation is e^{-nI} up to factors that grow slower than exponentially, where I is the optimized exponent for one copy, and Section F.2 in the appendices derives that matching lower bound. In practice every Chernoff bound in this chapter is computed by the same five steps:

1. Write down the MGF of X.
2. If X is a sum of independent terms, factorize the MGF into one factor per term.
3. Bound each factor by something simple.
4. Minimize the exponent over λ by setting its derivative to zero.
5. Simplify the result into a usable form.

Figure 2.2 shows the trade-off in λ for a binomial count. The bound has a clear minimum, and bounding each factor with $1 + x \le e^x$, as step 1 of the next proof does, costs a little but puts the minimizing λ in closed form.

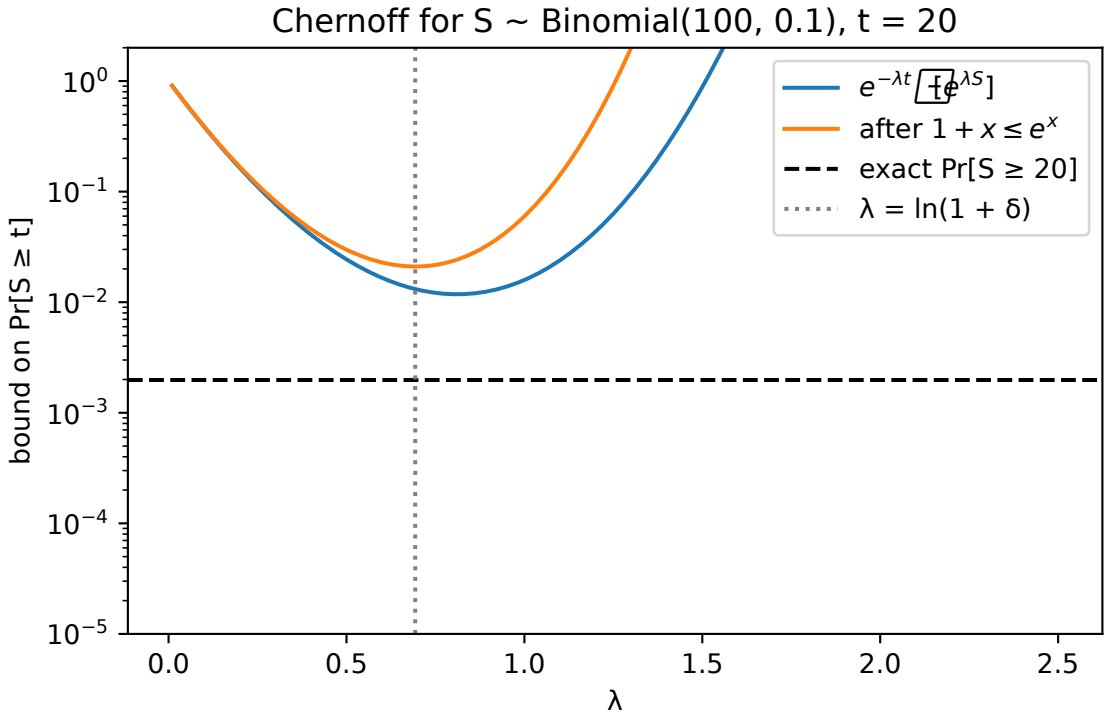


Figure 2.2.: The Chernoff bound $e^{-\lambda t}\mathbb{E}[e^{\lambda S}]$ for $S \sim \mathrm{Bin}(100, 0.1)$ and $t = 20$, as a function of λ, before and after the relaxation $1 + x \le e^x$. Both stay above the exact tail.

The method is used most for a count of independent yes/no events, such as errors on a test set or records drawn into a batch. Each event is a variable $X_i \sim \mathrm{Bern}(p_i)$, equal to 1 with probability p_i, and the question is how likely the count S is to land a fraction r above or below its mean μ.

Theorem 2.4 (Chernoff bounds for a count)**.** *Let $X_1, \dots, X_n$ be independent with $X_i \sim \mathrm{Bern}(p_i)$, let $S = \sum_i X_i$, and let $\mu = \sum_i p_i$. For every $r > 0$, the upper tail obeys*

$$\Pr[S \ge (1+r)\mu] \le \exp\big(-\mu[(1+r)\ln(1+r) - r]\big) \le \exp\left(-\frac{r^2\mu}{2+r}\right). \tag{2.13}$$

For every $0 < r < 1$, the lower tail obeys

$$\Pr[S \le (1-r)\mu] \le \exp\left(-\frac{r^2\mu}{2}\right). \tag{2.14}$$

Proof. The plan is to bound the MGF of the count by that of a Poisson count with the same mean, apply the Chernoff bound, choose the best λ, and simplify. The lower tail repeats the same steps for $-S$.

1. *One factor.* Each X_i is 1 with probability p_i, so its MGF is $p_i e^\lambda + (1-p_i) = 1 + p_i(e^\lambda - 1)$. The inequality $1 + x \le e^x$ holds for every real x, because the convex curve e^x lies above its tangent line $1 + x$ at 0 (Probability distributions). With $x = p_i(e^\lambda - 1)$ it gives
$$M_{X_i}(\lambda) \le \exp\left(p_i(e^\lambda - 1)\right). \tag{2.15}$$
The right side is the MGF of a Poisson count with mean p_i.
2. *Product.* The MGF of a sum of independent variables is the product of their MGFs, because the exponential turns the sum into a product and independence lets the average of that product split (Probability distributions). The exponents of step 1 then add up to $\mu(e^\lambda - 1)$:
$$M_S(\lambda) \le \exp\left(\mu(e^\lambda - 1)\right). \tag{2.16}$$
3. *Chernoff bound.* Putting this into Equation 2.11 at $t = (1+r)\mu$ gives
$$\Pr[S \ge (1+r)\mu] \le \exp\left(\mu(e^\lambda - 1) - \lambda(1+r)\mu\right). \tag{2.17}$$
4. *Minimize.* The derivative of the exponent, $\mu e^\lambda - (1+r)\mu$, vanishes at $e^\lambda = 1 + r$, so $\lambda = \ln(1+r)$, which is positive as required. Substituting it turns the exponent into $\mu r - (1+r)\mu \ln(1+r)$, which is the first upper-tail bound.
5. *Simplify.* A width needs the exponent solved for r, and $(1+r)\ln(1+r) - r$ has no closed-form inverse. So replace the logarithm by the lower bound $\ln(1+r) \ge 2r/(2+r)$, valid for $r \ge 0$, which turns the bracket into a ratio of polynomials. The difference $\ln(1+r) - 2r/(2+r)$ is 0 at $r = 0$, and it never decreases, because its derivative is never negative:
$$\frac{1}{1+r} - \frac{4}{(2+r)^2} = \frac{r^2}{(1+r)(2+r)^2} \ge 0. \tag{2.18}$$
Putting the lower bound into the bracket gives $(1+r)\frac{2r}{2+r} - r = \frac{r^2}{2+r}$, which is the second upper-tail bound.
6. *Lower tail.* A small S is a large $-S$. For $\lambda > 0$ the function $e^{-\lambda x}$ is decreasing, so $S \le s$ exactly when $e^{-\lambda S} \ge e^{-\lambda s}$. Markov's inequality applied to $e^{-\lambda S}$, with steps 1 and 2 at $-\lambda$ in place of λ, gives
$$\Pr[S \le s] \le e^{\lambda s} M_S(-\lambda) \le \exp\left(\lambda s + \mu(e^{-\lambda} - 1)\right). \tag{2.19}$$
7. *Minimize.* With $s = (1-r)\mu$, the derivative $s - \mu e^{-\lambda}$ vanishes at $e^{-\lambda} = 1 - r$, that is, at $\lambda = -\ln(1-r) > 0$. Substituting it gives
$$\Pr[S \le (1-r)\mu] \le \exp\left(-\mu[r + (1-r)\ln(1-r)]\right). \tag{2.20}$$
8. *Simplify.* For $0 < r < 1$, which keeps $1 - r$ positive and the series below convergent, multiply the Taylor series $\ln(1-r) = -\sum_{j \ge 1} r^j/j$ by $1 - r$. Each power r^j with $j \ge 2$ gets $-1/j$ from the 1 and $1/(j-1)$ from the $-r$, together $1/(j(j-1))$, so
$$(1-r)\ln(1-r) = -r + \sum_{j \ge 2} \frac{r^j}{j(j-1)}. \tag{2.21}$$
Every term of the sum is non-negative, so keeping only the first one, $r^2/2$, gives a lower bound. Adding r, the bracket is then at least $r^2/2$, which is the lower-tail bound.

□

The bounds depend on the expected count μ, not on the number of trials n, so a relative deviation r becomes unlikely only once μ is large. Because step 1 keeps each p_i, a small μ, and with it a small variance $\sum_i p_i(1-p_i) \le \mu$, gives a small MGF bound, which is why this bound will beat Hoeffding's range-only bound of the next section. For large r the simplified upper-tail exponent $r^2\mu/(2+r)$ grows like $r\mu$, not $r^2\mu$, so counts overshoot their mean more easily than they undershoot it.

Example 2.2. In the running example the number of errors S is a sum of 200 independent Bern(0.1) variables, so $\mu = 20$. A measured rate of 15% means 30 errors, which is $(1+r)\mu$ with $r = 0.5$. The simplified upper-tail bound gives

$$\Pr[S \ge 30] \le \exp\left(-\frac{0.5^2 \times 20}{2.5}\right) = e^{-2} = 0.135. \tag{2.22}$$

That is the best bound so far, below Chebyshev's 0.18, but still 14 times the simulated 9.6×10^{-3}. Part of the gap is the simplification in step 5 of the proof, and the rest is a factor in front of the exponential, shrinking like $1/\sqrt{\mu}$, that the Chernoff method cannot see because it bounds only the exponent.

2.3. Sub-Gaussian variables and Hoeffding

Redoing the Chernoff calculation for every new distribution is tedious. A normal variable $\mathcal{N}(0, \sigma^2)$, with mean 0 and variance σ^2, has the MGF $e^{\lambda^2\sigma^2/2}$ (Probability distributions), a quadratic log-MGF, and the Chernoff calculation for it uses nothing else. So this section defines a class of variables by that one property, proves tail and sum bounds for the class once, and applies them to bounded variables, which gives Hoeffding's inequality for averages.

Definition 2.1 (Sub-Gaussian variable)**.** A random variable X with mean μ is **σ-sub-Gaussian** if, for every real λ,

$$\mathbb{E}[e^{\lambda(X-\mu)}] \le e^{\lambda^2\sigma^2/2}. \tag{2.23}$$

The square σ^2 is its **variance proxy**.

The next result shows that it is never smaller than the true variance, and it can be much larger, which is where these bounds lose accuracy.

Proposition 2.1 (Properties of sub-Gaussian variables)**.** *Let X be σ-sub-Gaussian with mean μ, and let $t > 0$.*

1. *Its variance is at most σ^2.*
2. *Its tails are no heavier than those of a normal variable with variance σ^2:*

$$\Pr[X - \mu \ge t] \le e^{-t^2/(2\sigma^2)}, \qquad \Pr[|X-\mu| \ge t] \le 2e^{-t^2/(2\sigma^2)}. \tag{2.24}$$

3. *If $X_1, \dots, X_n$ are independent and each X_i is σ_i-sub-Gaussian with mean μ_i, their sum is $\sqrt{\sum_i \sigma_i^2}$-sub-Gaussian.*

4. *For a constant c, the variable cX is $|c|\sigma$-sub-Gaussian. So the mean of n independent σ-sub-Gaussian variables is $(\sigma/\sqrt{n})$-sub-Gaussian.*

Proof.

1. *Variance.* Expand both sides of the definition around $\lambda = 0$. The left side is $1+\lambda^2 \operatorname{Var}(X)/2+$ ..., with no linear term because $X - \mu$ has mean 0, and the right side is $1 + \lambda^2\sigma^2/2 + ...$. Subtracting 1, dividing by $\lambda^2/2$ and letting $\lambda \to 0$ forces $\operatorname{Var}(X) \le \sigma^2$.
2. *Tail.* Putting the bound of the definition into Equation 2.11 for $X - \mu$ gives, for every $\lambda > 0$,

$$\Pr[X - \mu \ge t] \le \exp\big(-\lambda t + \lambda^2\sigma^2/2\big). \tag{2.25}$$

   The exponent is a parabola in λ, whose derivative $-t + \lambda\sigma^2$ vanishes at $\lambda = t/\sigma^2 > 0$, and substituting it gives the exponent $-t^2/\sigma^2 + t^2/(2\sigma^2) = -t^2/(2\sigma^2)$. Replacing λ by $-\lambda$ in the definition shows that $-X$ is also σ-sub-Gaussian, which gives the same bound for the lower tail, and adding the two gives the two-sided bound.
3. *Sums.* The centred MGF $\mathbb{E}[e^{\lambda(X-\mu)}]$ of a sum of independent terms is the product of their centred MGFs, as in step 2 of the proof of Theorem 2.4. Each factor is at most a normal MGF, and the exponents of those add up:

$$\mathbb{E}\big[e^{\lambda\sum_i (X_i-\mu_i)}\big] \le \prod_i e^{\lambda^2\sigma_i^2/2} = e^{\lambda^2(\sum_i \sigma_i^2)/2}. \tag{2.26}$$

4. *Scaling and means.* Multiplying X by c is the same as multiplying λ by c, so the definition at $c\lambda$ gives the bound $e^{c^2\lambda^2\sigma^2/2}$. The sum of n independent σ-sub-Gaussian variables is $\sigma\sqrt{n}$-sub-Gaussian by item 3, and dividing by n makes their mean $(\sigma/\sqrt{n})$-sub-Gaussian.

□

Hoeffding's inequality also needs the fact that bounded variables are sub-Gaussian. A variable trapped in an interval cannot have a heavy tail, since it cannot go beyond the interval at all. The lemma makes this quantitative, with a variance proxy of half the interval's length, squared.

Lemma 2.2 (Hoeffding's lemma)**.** *Let X take values in an interval $[l, u]$ and have mean μ. Then X is $\frac{u-l}{2}$-sub-Gaussian, that is, for every real λ,*

$$\mathbb{E}\big[e^{\lambda(X-\mu)}\big] \le \exp\big(\lambda^2(u-l)^2/8\big). \tag{2.27}$$

Proof. Shifting X and the interval by μ changes neither side, so assume $\mu = 0$. A variable with mean 0 takes values on both sides of 0, so $l \le 0 \le u$. The plan is to bound $e^{\lambda X}$ by a straight line in X, whose average depends only on the mean, and then to show that the logarithm of the result is at most a quadratic.

1. *Use convexity.* On $[l, u]$ the convex curve $e^{\lambda x}$ lies below the chord joining its values at the two endpoints. That chord is the weighted average of the endpoint values, with weights that say how close x is to each end:

$$e^{\lambda x} \le \frac{u-x}{u-l}e^{\lambda l} + \frac{x-l}{u-l}e^{\lambda u}. \tag{2.28}$$

2. *Take expectations.* The right side is linear in x, so its average needs only $\mathbb{E}[X] = 0$:

$$\mathbb{E}[e^{\lambda X}] \leq \frac{u}{u-l} e^{\lambda l} - \frac{l}{u-l} e^{\lambda u}. \tag{2.29}$$

   This is exactly the MGF of the two-point variable that sits at l or u with mean 0, so that variable is the worst case on the interval.
3. *Reduce to one variable.* Let $p = -l/(u-l)$, a number in $[0,1]$, so that $\frac{u}{u-l} = 1-p$. Write $w = \lambda(u-l)$, so that $\lambda l = -wp$ and $e^{\lambda u} = e^{\lambda l} e^{w}$. Factoring $e^{\lambda l} = e^{-wp}$ out of the right side of step 2 leaves $1 - p + pe^{w}$, so that side equals $e^{\Lambda(w)}$, where

$$\Lambda(w) = -wp + \ln\left(1 - p + pe^{w}\right). \tag{2.30}$$

   It remains to show that $\Lambda(w) \leq w^2/8$, by a Taylor expansion around $w = 0$.
4. *Value and slope at 0.* At $w = 0$ the value is $\Lambda(0) = \ln 1 = 0$. The derivative is $\Lambda'(w) = -p + \frac{pe^{w}}{1-p+pe^{w}}$, so $\Lambda'(0) = -p + p = 0$.
5. *Bound the curvature.* Write $v = \frac{pe^{w}}{1-p+pe^{w}}$, a number in $[0,1]$, so that $\Lambda'(w) = -p + v$. By the quotient rule the derivative of v is $v \cdot \frac{1-p}{1-p+pe^{w}} = v(1-v)$, so $\Lambda''(w) = v(1-v)$. A product of two non-negative numbers that add up to 1 is largest when both are $\frac{1}{2}$, so $\Lambda''(w) \leq \frac{1}{4}$.
6. *Taylor with remainder.* Taylor's theorem gives $\Lambda(w) = \Lambda(0) + w\Lambda'(0) + \frac{w^2}{2}\Lambda''(\xi)$ for some ξ between 0 and w. With value and slope 0 and curvature at most $\frac{1}{4}$, this is at most $w^2/8 = \lambda^2(u-l)^2/8$.

□

The constant $\frac{1}{8}$ cannot be improved. A fair coin on the two endpoints has variance $(u-l)^2/4$, and step 1 of Proposition 2.1 shows that no variance proxy can be below the variance. Combining the lemma with items 3 and 4 of Proposition 2.1 gives Hoeffding's inequality: the mean of n bounded independent variables has a variance proxy that shrinks like $1/n$.

Theorem 2.5 (Hoeffding's inequality)**.** *Let $X_1, \dots, X_n$ be independent, each with values in $[l,u]$, and let $\bar{X}_n$ be their mean, with $\mathbb{E}[\bar{X}_n] = \mu$. For every $t > 0$,*

$$\Pr\left[\bar{X}_n - \mu \geq t\right] \leq \exp\left(-\frac{2nt^2}{(u-l)^2}\right). \tag{2.31}$$

For every $\beta \in (0,1)$, with probability at least $1-\beta$,

$$|\bar{X}_n - \mu| < (u-l)\sqrt{\frac{\ln(2/\beta)}{2n}}. \tag{2.32}$$

Proof.

1. *Each term.* By Lemma 2.2 each X_i is $\frac{u-l}{2}$-sub-Gaussian.
2. *The mean.* By items 3 and 4 of Proposition 2.1 their mean is $\frac{u-l}{2\sqrt{n}}$-sub-Gaussian, with variance proxy $(u-l)^2/(4n)$.
3. *Tail.* Item 2 of Proposition 2.1 with this proxy in place of σ^2 gives the exponent $t^2 \big/ (2(u-l)^2/(4n)) = 2nt^2/(u-l)^2$, which is the one-sided bound.
4. *Width.* The two-sided bound is twice this. Setting it equal to β and solving for t, as in Lemma 2.1, gives the width.

□

Hoeffding's inequality needs only the range, not the variance. That is its strength, since there is nothing to estimate, and its weakness, since it assumes the worst-case variance $(u-l)^2/4$ of the fair coin on the endpoints.

Example 2.3. Each error of the running example lies in $[0,1]$, so $u-l=1$, and the deviation is $t=0.05$ with $n=200$. Hoeffding's inequality gives

$$\Pr[\bar{X}-0.1\geq 0.05]\leq e^{-2\times 200\times 0.05^2}=e^{-1}=0.37. \tag{2.33}$$

That is worse than Chebyshev's 0.18, because the true variance of one error, $0.1\times 0.9=0.09$, is far below Hoeffding's worst case of $1/4$. Figure 2.3 puts the bounds side by side with the simulated tail.

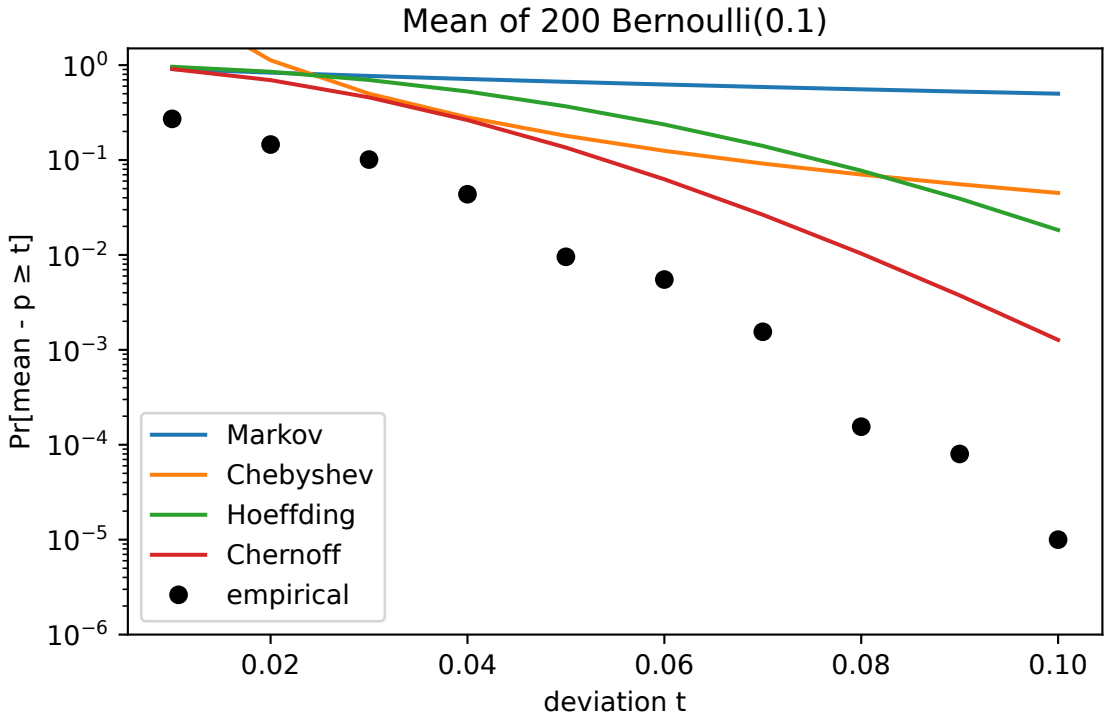


Figure 2.3.: Empirical tail of the mean of 200 Bernoulli(0.1) variables against the Markov, Chebyshev, Hoeffding and Chernoff bounds.

When the true variance is much smaller than the worst case, Bernstein's inequality keeps it in the bound. Write σ^2 for the variance of each term and R for the largest distance a term can be from its mean.

Theorem 2.6 (Bernstein's inequality)**.** *Let $X_1,\dots,X_n$ be independent, each with mean μ and variance σ^2 and with $|X_i-\mu|\leq R$, and let $\bar{X}_n$ be their mean. For every $t>0$,*

$$\Pr\left[\bar{X}_n-\mu\geq t\right]\leq \exp\left(-\frac{nt^2}{2\sigma^2+\frac{2}{3}Rt}\right). \tag{2.34}$$

For every $\beta \in (0,1)$, with probability at least $1-\beta$ the deviation $\bar{X}_n - \mu$ is below

$$\sqrt{\frac{2\sigma^2 \ln(1/\beta)}{n}} + \frac{2R\ln(1/\beta)}{3n}. \tag{2.35}$$

The proof is the same Chernoff recipe, with a sharper bound on each factor's MGF. Boucheron, Lugosi & Massart (ch. 2) give it, and Section F.1 in the appendices derives it and the width step by step. For small t the $2\sigma^2$ term of the denominator dominates, and the bound looks Gaussian with the true variance. For large t the $\frac{2}{3}Rt$ term dominates, and the bound decays like e^{-ct}, more slowly than the e^{-ct^2} of the Gaussian regime.

Example 2.4. The variance matters most when the rate is near zero. A classifier makes 1 false positive on 1,000 negative records, and you want a 95% upper bound on its true false-positive rate, so $\beta = 0.05$ and $\ln(1/\beta) = \ln 20 \approx 3.0$. Setting the one-sided bound of Theorem 2.5 equal to β gives the half-width $\sqrt{\ln(1/\beta)/(2n)} = \sqrt{\ln 20/2000} \approx 0.039$, added to the measured rate of 0.001, because Hoeffding assumes a variance of 0.25. Bernstein's inequality with the variance $\sigma^2 = 0.001 \times 0.999 \approx 0.001$ of the measured rate, $R = 1$ and $n = 1{,}000$ gives the width

$$\sqrt{\frac{2 \times 0.001 \times 3.0}{1000}} + \frac{2 \times 3.0}{3000} \approx 0.0045, \tag{2.36}$$

almost ten times tighter, though only as a heuristic, because σ^2 was estimated from the same data. The exact **Clopper–Pearson** bound, the largest rate under which 1 or fewer false positives in 1,000 trials still has probability at least 0.05, is 0.0047.

2.4. Many estimates at once

A learner rarely relies on one estimate. It compares many models or runs many training steps, and it needs all of them to behave at the same time. The union bound says that the chance that at least one of several events happens is at most the sum of their separate chances, because the sum counts an outcome that lies in several events several times. The running example is one estimate, so this section uses a training loop.

Lemma 2.3 (The union bound)**.** *For any events $A_1, \dots, A_k$, independent or not,*

$$\Pr[A_1 \cup \cdots \cup A_k] \le \sum_{i=1}^{k} \Pr[A_i]. \tag{2.37}$$

Proof.

1. *Compare indicators outcome by outcome.* Where none of the events happens, the indicator of the union and the sum of the indicators are both 0. Where at least one happens, the first is 1 and the second counts the events that happen, which is at least 1. So at every outcome

$$\mathbf{1}[A_1 \cup \cdots \cup A_k] \le \sum_{i=1}^{k} \mathbf{1}[A_i]. \tag{2.38}$$

2. *Average.* Averages keep this order, and the average of each indicator is the probability of its event, as in the proof of Theorem 2.1.

□

If each of k estimates fails with probability at most β/k, all of them succeed together except with probability at most $k \cdot \beta/k = \beta$. In an exponential bound, asking for β/k instead of β replaces $\ln(1/\beta)$ by $\ln(k/\beta) = \ln k + \ln(1/\beta)$, and that is the $\ln k$ cost.

The first use is a question every learner faces. A learner picks its model after seeing the data, so you need its training loss to be close to its population loss for every model it could pick, not just for one fixed model.

Corollary 2.2 (Uniform convergence over a finite class)**.** *Let Θ be a finite set of models, and let the loss of each model on each record lie in $[0,1]$. Let D be a dataset of n records drawn independently from a population $\mathcal{D}$. Write $\mathcal{L}(\theta; D)$ for the* ***empirical risk*** *of a model θ, its average loss on D, and $\mathcal{L}_{\mathcal{D}}(\theta)$ for its* ***true risk****, its average loss on the population. For every $\beta \in (0,1)$, with probability at least $1-\beta$,*

$$|\mathcal{L}(\theta; D) - \mathcal{L}_{\mathcal{D}}(\theta)| \le \sqrt{\frac{\ln(2|\Theta|/\beta)}{2n}} \quad \text{for all } \theta \in \Theta. \tag{2.39}$$

Proof.

1. *One model.* For a fixed θ, the empirical risk is a mean of n independent terms in $[0,1]$ whose average is the true risk. So Theorem 2.5 with $u - l = 1$, on both tails, gives
$$\Pr\left[|\mathcal{L}(\theta; D) - \mathcal{L}_{\mathcal{D}}(\theta)| \ge t\right] \le 2e^{-2nt^2}. \tag{2.40}$$
2. *All models.* By Lemma 2.3 over the $|\Theta|$ models, the chance that any of them is off by t or more is at most $2|\Theta|e^{-2nt^2}$.
3. *Invert.* Setting this equal to β and solving for t, as in Lemma 2.1, gives the width.

□

The size of the class enters only through $\ln|\Theta|$, so squaring the number of models only doubles that term.

Example 2.5. A training loop includes each of $n = 60{,}000$ records in a minibatch independently with probability $q = 0.01$, so the batch size B is a count with mean $\mu = 600$. For a batch more than 20% off, $r = 0.2$, $r^2 = 0.04$ and $2 + r = 2.2$, and the two tails of Theorem 2.4 give

$$\Pr[B \ge 720] \le e^{-0.04\times 600/2.2} = e^{-10.9} \approx 1.8 \times 10^{-5}, \qquad \Pr[B \le 480] \le e^{-0.04\times 600/2} = e^{-12} \approx 6.1 \times 10^{-6}. \tag{2.41}$$

The two events cannot happen together, so B stays within $600 \pm 20\%$ except with probability about 2.4×10^{-5} per step. Over 1,000 steps, Lemma 2.3 adds the per-step failure probabilities, so every batch stays in that range except with probability about $1{,}000 \times 2.4 \times 10^{-5} = 0.024$.

2.5. Gaussian and Laplace tails

The normal and Laplace distributions are the two standard models of additive noise, and later chapters add such noise to computed numbers on purpose, so they need to know how large it can be. Both densities are known exactly, so this section bounds their tails directly and more sharply, and gives the width of the largest of many draws. The running example is a count, not a noise model, so this section's examples use the noise itself.

Write $Z \sim \mathcal{N}(0,1)$ for a standard normal variable and $\varphi(z) = e^{-z^2/2}/\sqrt{2\pi}$ for its density. Of the two bounds below, the second, the **Mills ratio** bound, named after the ratio of tail to density, is sharper far out. Its proof weights the density by a factor that is at least 1 on the tail and makes the integral easy.

Proposition 2.2 (Gaussian tail bounds)**.** *Let $Z \sim \mathcal{N}(0,1)$ have density φ, and let $t > 0$. Then*

$$\Pr[Z \geq t] \leq \tfrac{1}{2} e^{-t^2/2} \qquad \text{and} \qquad \Pr[Z \geq t] \leq \frac{\varphi(t)}{t}. \tag{2.42}$$

If $X_1, \dots, X_k$ each have the distribution $\mathcal{N}(0, \sigma^2)$, independent or not, then for every $\beta \in (0,1)$, with probability at least $1-\beta$, every $|X_i|$ is below $\sigma\sqrt{2\ln(k/\beta)}$.

Proof. Steps 1 to 3 prove the first bound, steps 4 and 5 the second, and step 6 the width.

1. *Shift the tail to start at 0.* Substituting $y = z - t$ makes the tail integral run over $y \geq 0$:
$$\Pr[Z \geq t] = \int_0^\infty \varphi(t+y)\,\mathrm{d}y. \tag{2.43}$$
2. *Split the density.* Expanding $(t+y)^2 = t^2 + 2ty + y^2$ in the exponent splits the density into the factor $e^{-t^2/2}$, a factor e^{-ty} and the density $\varphi(y)$, which keeps the constant $1/\sqrt{2\pi}$:
$$\Pr[Z \geq t] = e^{-t^2/2} \int_0^\infty e^{-ty}\varphi(y)\,\mathrm{d}y. \tag{2.44}$$
3. *Drop the middle factor.* The factor e^{-ty} is at most 1 for $t, y \geq 0$, and the integral of φ over $y \geq 0$ is $\frac{1}{2}$ because the density is symmetric about 0. Together they give the first bound.
4. *Weight the density.* On $z \geq t$ the factor z/t is at least 1, so multiplying the density by it can only increase the integral:
$$\Pr[Z \geq t] = \int_t^\infty \varphi(z)\,\mathrm{d}z \leq \int_t^\infty \frac{z}{t}\varphi(z)\,\mathrm{d}z. \tag{2.45}$$
5. *Integrate.* Differentiating the density gives $\varphi'(z) = -z\varphi(z)$, so $z\varphi(z)$ has the antiderivative $-\varphi(z)$. The integral of step 4 is therefore $\frac{1}{t}\big[-\varphi(z)\big]_t^\infty = \varphi(t)/t$.
6. *Width of k draws.* Each X_i is σZ, so the first bound, doubled for both tails, gives $\Pr[|X_i| \geq t] \leq e^{-t^2/(2\sigma^2)}$. Setting this equal to β/k gives $t = \sigma\sqrt{2\ln(k/\beta)}$ by Lemma 2.1, and Lemma 2.3 over the k draws gives a total failure probability of at most β.

□

A standard normal is 1-sub-Gaussian, since its MGF is $e^{\lambda^2/2}$, so Proposition 2.1 gives the Chernoff bound $e^{-t^2/2}$, and the first bound beats it by a factor of $\frac{1}{2}$. The ratio of the Mills bound to the exact tail tends to 1 as t grows, because the tail mass sits ever closer to t, where the factor z/t is close to 1.

Example 2.6. At $t = 3$ the exact standard normal tail is 1.35×10^{-3}. Against it, Chebyshev's inequality allows $1/3^2 = 0.111$ and the Chernoff bound $e^{-9/2} = 0.0111$, while the two bounds of Proposition 2.2 allow 0.0056 (the halved Chernoff bound) and $\varphi(3)/3 = 0.00148$ (the Mills ratio). So the Mills ratio is within 10% of the exact tail, while the Chernoff bound is 8 times too large (Figure 2.4).

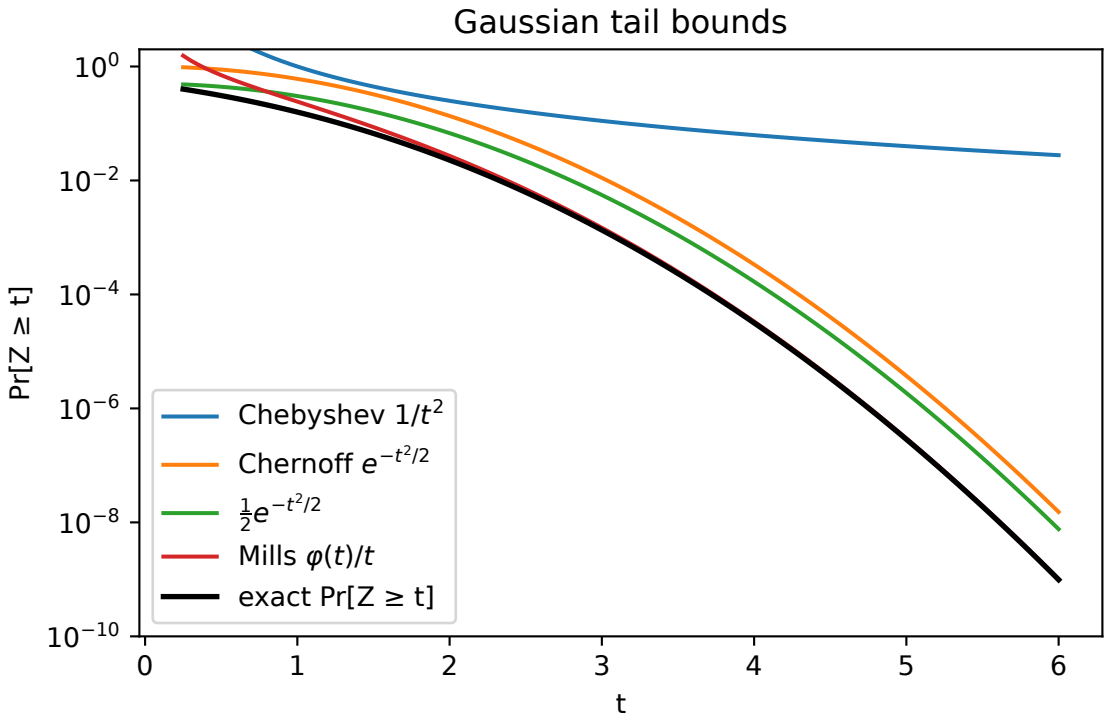


Figure 2.4.: The exact standard normal tail against the Chebyshev, Chernoff, halved Chernoff and Mills-ratio bounds.

For the Laplace distribution the tail can be computed exactly, so no bound is needed. The **Laplace distribution** $\mathrm{Lap}(b)$ with scale $b > 0$ has density $\frac{1}{2b}e^{-|x|/b}$, symmetric around 0.

Proposition 2.3 (Laplace tails and widths)**.** *Let $X \sim \mathrm{Lap}(b)$ and $t \geq 0$. Then*

$$\Pr[X \geq t] = \tfrac{1}{2}e^{-t/b}, \qquad \Pr[|X| \geq t] = e^{-t/b}. \tag{2.46}$$

For $\beta \in (0,1)$, one draw satisfies $|X| \leq b\ln(1/\beta)$ with probability at least $1-\beta$. Draws $X_1, \ldots, X_k \sim \mathrm{Lap}(b)$, independent or not, all satisfy $|X_i| \leq b\ln(k/\beta)$ with probability at least $1-\beta$.

Proof.

1. *The tail.* Integrating the density $\frac{1}{2b}e^{-x/b}$ from t to infinity gives $\frac{1}{2}e^{-t/b}$. The density is symmetric, so the two-sided tail is twice that.
2. *One draw.* Setting $e^{-t/b} = \beta$ and solving gives $t = b\ln(1/\beta)$.
3. *Many draws.* Setting $e^{-t/b} = \beta/k$ gives $t = b\ln(k/\beta)$, and Lemma 2.3 over the k draws gives a total failure probability of at most β.

□

For independent draws the union bound is nearly exact, because they rarely fail together when each fails rarely. For $k = 100$ draws of Lap(1) at $\beta = 0.05$, the width is $\ln(100/0.05) = 7.60$, and simulated draws exceed it at the rate 0.0486, close to the target (Figure 2.5).

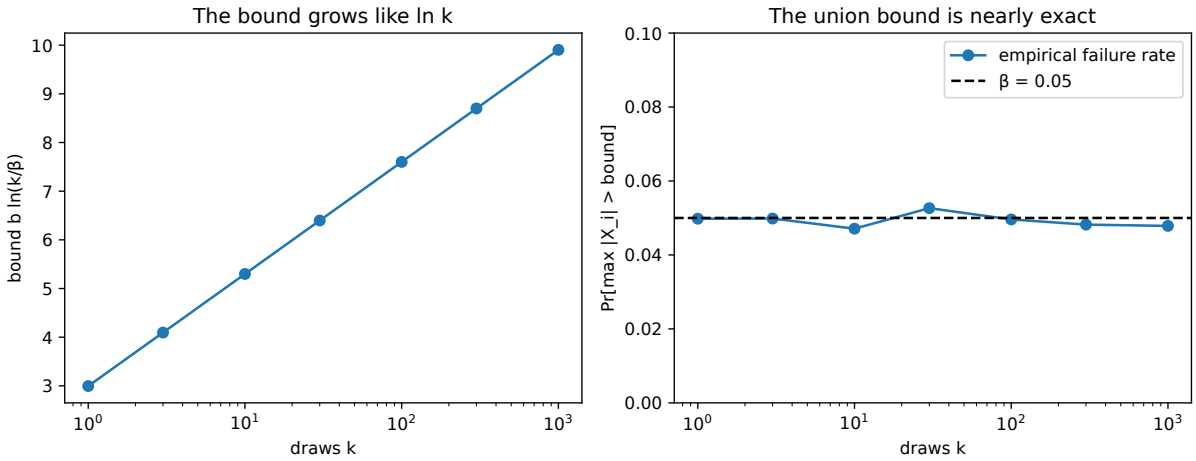


Figure 2.5.: Left: the bound $b\ln(k/\beta)$ grows like $\ln k$. Right: the empirical probability that the maximum of k Laplace draws exceeds it stays at $\beta = 0.05$.

2.5.1. Sums of Laplace variables

Noise is often added many times and summed, so how large can a sum of k Laplace draws get? The Laplace MGF is $1/(1-\lambda^2 b^2)$ for $|\lambda| < 1/b$ and infinite beyond (Probability distributions), so no bound of the form $e^{\lambda^2\sigma^2/2}$ holds for every λ, and a Laplace variable is not sub-Gaussian. For $|\lambda| \le 1/(2b)$, though, its log-MGF is still at most a quadratic, and the proof of Proposition 2.4 below runs the Chernoff bound on that range only.

Lemma 2.4 (The Laplace MGF near zero)**.** *Let $X \sim \mathrm{Lap}(b)$. For every λ with $|\lambda| \le 1/(2b)$,*

$$M_X(\lambda) \le e^{2\lambda^2 b^2}. \tag{2.47}$$

Proof.

1. *Rewrite the log-MGF.* With $x = \lambda^2 b^2$, the log-MGF is $-\ln(1-x)$, and the condition on λ gives $x \le \frac{1}{4}$.
2. *Compare series.* The series $-\ln(1-x) = x + x^2/2 + x^3/3 + \ldots$ is term by term at most $x + x^2 + x^3 + \ldots$, which is the geometric series $x/(1-x)$.

3. *Bound the denominator.* Since $x \le \frac{1}{4}$, the factor $1/(1-x)$ is at most $\frac{4}{3}$, rounded up to 2 to keep the constants simple, so the log-MGF is at most $2x = 2\lambda^2 b^2$.

□

Proposition 2.4 (Sums of Laplace variables)**.** *Let S be the sum of k independent* Lap(b) *variables, and let $t > 0$. Then*

$$\Pr[S \ge t] \le \exp\left(-\min\left(\frac{t^2}{8kb^2},\ \frac{t}{4b}\right)\right). \tag{2.48}$$

Proof. The plan is to bound the MGF of S on the range of λ where Lemma 2.4 applies, run the Chernoff bound with λ restricted to that range, and split by whether the best λ lies inside it.

1. *The MGF of the sum.* The MGF of S is the product of k factors, each at most $e^{2\lambda^2 b^2}$ by the lemma, so $M_S(\lambda) \le e^{2k\lambda^2 b^2}$ for $0 < \lambda \le 1/(2b)$.
2. *The unconstrained optimum.* By Equation 2.11 the exponent to minimize is $-\lambda t + 2k\lambda^2 b^2$, a parabola whose derivative $-t + 4k\lambda b^2$ vanishes at $\lambda^* = t/(4kb^2)$.
3. *Small deviations.* The optimum λ^* is allowed exactly when $t \le 2kb$. Substituting it gives the exponent $-t^2/(4kb^2) + t^2/(8kb^2) = -t^2/(8kb^2)$, so the bound is $e^{-t^2/(8kb^2)}$.
4. *Large deviations.* If $t > 2kb$, the parabola is still decreasing at $1/(2b)$, so the best allowed λ is that boundary, where the exponent is $-t/(2b) + 2kb^2/(4b^2) = -t/(2b) + k/2$. In this regime $k/2 < t/(4b)$, so the exponent is at most $-t/(4b)$.
5. *Combine.* Since $t^2/(8kb^2) \le t/(4b)$ exactly when $t \le 2kb$, the minimum is $t^2/(8kb^2)$ in the regime of step 3 and $t/(4b)$ in that of step 4.

□

Small deviations look Gaussian, with a variance of order kb^2, and large deviations decay only exponentially. That two-regime shape is what **sub-exponential** means.

Example 2.7. Take $k = 10$ and $b = 1$. The central limit theorem approximates the sum by a normal variable with the same mean 0 and variance $10 \times 2b^2 = 20$, since one Lap(b) draw has variance $2b^2$. It matches the bulk, with $\Pr[S \ge 10]$ equal to 1.42×10^{-2} in simulation and 1.27×10^{-2} from the normal. But $\Pr[S \ge 20]$ is 3.4×10^{-5} in simulation and only 3.9×10^{-6} from the normal, a factor of 9 too small. Both deviations are in the Gaussian regime of Proposition 2.4, so the bound gives $e^{-100/80} = 0.287$ at $t = 10$ and $e^{-5} = 6.7 \times 10^{-3}$ at $t = 20$, loose but a guarantee (Figure 2.6).

2.6. Azuma and McDiarmid

So far every quantity was a sum of independent terms, and two common situations break that. In sequential procedures each step can be chosen after seeing earlier outcomes, so the terms are dependent and the MGF of their sum no longer factorizes. And the risk of a model trained on the data is a complicated function of all the records, not a sum of one term per record. Azuma's inequality handles the first situation, and McDiarmid's inequality, which follows from it, the second. Azuma's needs a **martingale**, a running total whose next step is zero on average given the past, like your fortune in a sequence of fair bets that you choose as you go.

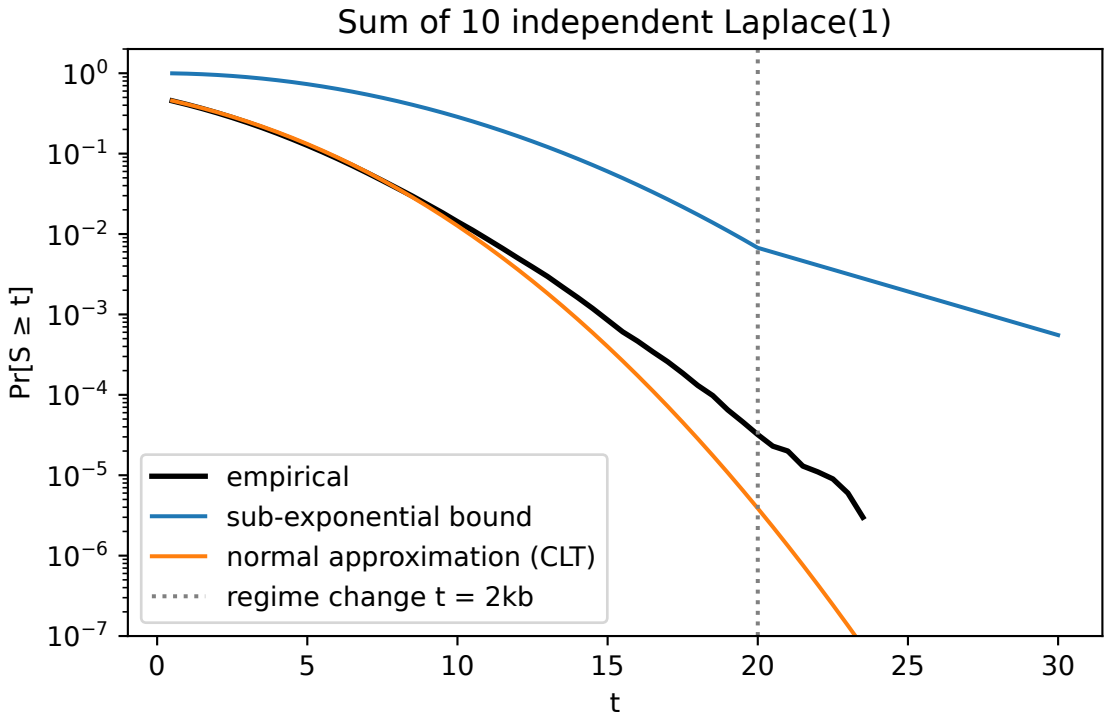


Figure 2.6.: Empirical tail of a sum of 10 Laplace(1) variables, against the sub-exponential bound and the normal approximation.

Definition 2.2 (Martingale)**.** A sequence $W_0, W_1, \dots, W_n$ of random variables is a **martingale** if each increment $U_i = W_i - W_{i-1}$ has mean zero given the past, that is, given everything observed before step i:

$$\mathbb{E}[U_i \mid \text{past}] = 0, \qquad i = 1, \dots, n. \tag{2.49}$$

Azuma's inequality says that the total drift $W_n - W_0$ obeys the same tail bound as a sum of independent bounded terms, with the lengths of the steps' intervals in place of the ranges.

Theorem 2.7 (Azuma–Hoeffding inequality)**.** *Let $W_0, \dots, W_n$ be a martingale whose increments satisfy $U_i \in [A_i, A_i + d_i]$, where the position A_i may depend on the past but the length d_i is fixed in advance, so that the bound of each step is a constant. For every $t > 0$,*

$$\Pr[W_n - W_0 \geq t] \leq \exp\left(-\frac{2t^2}{\sum_i d_i^2}\right). \tag{2.50}$$

Proof. The plan is to bound it one step at a time instead, starting from the last step and working back, and then to apply the Chernoff bound.

1. *Bound one step given the past.* Given the past, U_i is a variable with mean zero on an interval of length d_i. So Lemma 2.2 applies to it, with every expectation taken given the past, and its proof runs unchanged:
$$\mathbb{E}\big[e^{\lambda U_i} \mid \text{past}\big] \leq e^{\lambda^2 d_i^2/8}. \tag{2.51}$$
2. *Peel off the last increment.* The **tower property** says that you can average in two stages, first over the last step with the past held fixed and then over the past, and get the ordinary average. Split $W_n - W_0 = (W_{n-1} - W_0) + U_n$. With the past held fixed, $e^{\lambda(W_{n-1}-W_0)}$ is a constant and comes out of the inner average:
$$\mathbb{E}\big[e^{\lambda(W_n - W_0)}\big] = \mathbb{E}\Big[e^{\lambda(W_{n-1}-W_0)}\,\mathbb{E}\big[e^{\lambda U_n} \mid \text{past}\big]\Big]. \tag{2.52}$$
Step 1 bounds the inner average by a constant, because d_n is fixed in advance, and a constant comes out of the outer average:
$$\mathbb{E}\big[e^{\lambda(W_n - W_0)}\big] \leq e^{\lambda^2 d_n^2/8}\,\mathbb{E}\big[e^{\lambda(W_{n-1}-W_0)}\big]. \tag{2.53}$$
3. *Repeat.* Applying the same step to $W_{n-1} - W_0$, then to $W_{n-2} - W_0$, and so on, n times, gives
$$\mathbb{E}\big[e^{\lambda(W_n - W_0)}\big] \leq \exp\Big(\lambda^2 \sum_i d_i^2/8\Big). \tag{2.54}$$
4. *Chernoff.* By Equation 2.11 the exponent to minimize is $-\lambda t + \lambda^2 \sum_i d_i^2/8$. Its derivative vanishes at $\lambda = 4t/\sum_i d_i^2$, and substituting this λ gives the exponent $-2t^2/\sum_i d_i^2$.

□

Hoeffding's inequality is the special case of independent increments.

Example 2.8. You have two coins, with heads probabilities 0.3 and 0.7. You flip 200 times, and before each flip you pick the 0.7 coin if the previous flip was heads and the 0.3 coin otherwise. The flips are dependent, so Hoeffding's inequality does not apply. What you can control is how far the heads count drifts from what your own choices predicted. Let X_i be 1 if flip i is heads, and p_i the heads probability of the coin chosen for it. Let $W_n = \sum_{i \le n} (X_i - p_i)$ be the number of heads minus the number you expected. Given the past the coin is fixed, so $X_i - p_i$ has mean $p_i - p_i = 0$ and lies in $[-p_i, 1 - p_i]$, an interval of length 1. So W is a martingale with $d_i = 1$ and $\sum_i d_i^2 = 200$. At $t = 20$, Theorem 2.7 gives

$$\Pr[W_{200} \ge 20] \le e^{-2 \times 400/200} = e^{-4} = 1.83 \times 10^{-2}, \tag{2.55}$$

while simulation gives 1.12×10^{-3} (Figure 2.7). The bound pays for assuming the worst-case variance, exactly as Hoeffding's does. Each step's true variance is $p_i(1 - p_i) = 0.21$, not the 0.25 the interval allows.

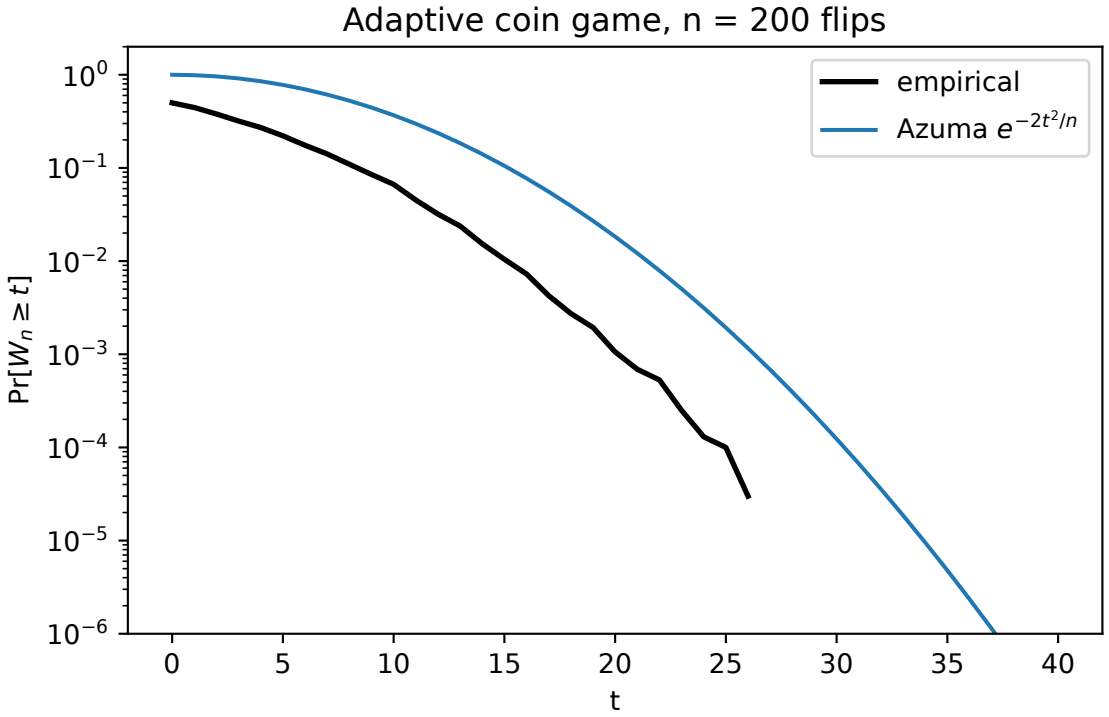


Figure 2.7.: Empirical tail of the martingale in the adaptive coin game against the Azuma bound.

McDiarmid's inequality says that a function of many independent inputs that no single input can move much concentrates as tightly as a sum of bounded terms would. The proof turns the function into a martingale by revealing the inputs one at a time and tracking your best guess of its value. Each revealed input moves the guess by a bounded amount, and Azuma's inequality does the rest.

Theorem 2.8 (McDiarmid's inequality)**.** *Let $X = (X_1, \dots, X_n)$ have independent coordinates, and let f be a function of them such that changing only the i-th input, with all others held fixed,*

changes f by at most c_i. For every $t > 0$,

$$\Pr\left[f(X) - \mathbb{E}[f(X)] \geq t\right] \leq \exp\left(-\frac{2t^2}{\sum_i c_i^2}\right). \tag{2.56}$$

Proof.

1. *Build the martingale.* After seeing $X_1, \ldots, X_i$, your best guess of $f(X)$ is its conditional average $W_i = \mathbb{E}[f(X) \mid X_1, \ldots, X_i]$. Before seeing anything the guess is $W_0 = \mathbb{E}[f(X)]$, and after seeing everything it is $W_n = f(X)$, so $W_n - W_0$ is exactly the deviation to bound. By the tower property, averaging the next guess over the input not yet seen gives back the current guess, so each increment has mean zero given the past.
2. *Bound the range of each increment.* Because the inputs are independent, seeing $X_1, \ldots, X_i$ does not change the distribution of the others, so W_i is the average of f over fresh copies of the unseen inputs $X_{i+1}, \ldots, X_n$, with $X_1, \ldots, X_i$ fixed at their observed values. With $X_1, \ldots, X_{i-1}$ held fixed, changing X_i changes every term of that average by at most c_i, so it moves the average by at most c_i. So W_i stays in an interval of length at most c_i whose position depends on the past. Given the past, W_{i-1} is a fixed number, so $U_i = W_i - W_{i-1}$ stays in an interval of the same length.
3. *Apply Azuma.* Theorem 2.7 with $d_i = c_i$ gives the bound.

□

The bounded-difference constant c_i measures how much one input can move the output. Its real use is for functions that are not averages, such as the risk of a trained model (Algorithmic stability and generalization).

Example 2.9. One test example moves the running example's error rate by at most $c_i = 1/200$, so $\sum_i c_i^2 = 1/200$. McDiarmid's inequality at $t = 0.05$ then gives $e^{-2\times 0.05^2 \times 200} = e^{-1} = 0.37$, which is Hoeffding's bound of Example 2.3 again.

2.7. Code

The script code/part01-prerequisites/concentration.py draws every figure in this chapter and checks each bound it quotes against simulation. From the repository root:

```
uv run python code/part01-prerequisites/concentration.py
```

Table 2.3 shows the output with seed 0.

Table 2.3.: Results of the chapter's script with seed 0.

Experiment	Result
Bernoulli mean, $n = 200$, $p = 0.1$, $t = 0.05$	Empirical 9.6×10^{-3}; Markov 0.67; Chebyshev 0.18; Hoeffding 0.37; Chernoff 0.135
Maximum of $k = 100$ draws of Lap(1), $\beta = 0.05$	Bound 7.60; empirical failure rate 0.0486
Standard normal tail at $t = 3$	Exact 1.35×10^{-3}; Chebyshev 0.111; Chernoff 0.0111; halved 0.0056; Mills 0.00148
Sum of $k = 10$ draws of Lap(1)	$t = 10$: empirical 1.42×10^{-2}, bound 0.287, normal 1.27×10^{-2}. $t = 20$: empirical 3.4×10^{-5}, bound 6.7×10^{-3}, normal 3.9×10^{-6}
Adaptive coin game, $n = 200$, $t = 20$	Empirical 1.12×10^{-3}; Azuma 1.83×10^{-2}

tests/test_concentration.py checks that the output still matches this table.

2.8. Summary

Table 2.4.: The results of this chapter, and where each is used.

Result	Statement	Use
Theorem 2.1	$\Pr[X \geq a] \leq \mathbb{E}[X]/a$ for $X \geq 0$	The base step of every other bound
Lemma 2.1	A tail $e^{-h(t)}$ gives the width $h^{-1}(\ln(1/\beta))$	Any tail bound as an error bar
Theorem 2.2	$\Pr[\lvert X - \mu \rvert \geq t] \leq \sigma^2/t^2$	Quick error bars; variance-only bounds cost $1/\sqrt{\beta}$
Theorem 2.3	$\Pr[X \geq t] \leq e^{-\lambda t} M_X(\lambda)$, $\lambda > 0$	Every exponential bound; its optimized exponent is exact for large sums
Theorem 2.4	Count with mean μ: $e^{-r^2\mu/(2+r)}$ above, $e^{-r^2\mu/2}$ below	Error counts, batch sizes, rare outcomes; depends on μ, not n
Proposition 2.1	Tail $e^{-t^2/(2\sigma^2)}$; proxies add; means shrink by $\sqrt{n}$	Sub-Gaussian widths cost $\sqrt{\ln(1/\beta)}$
Lemma 2.2	A variable in $[l, u]$ is $\frac{u-l}{2}$-sub-Gaussian	Bounded variables; martingale steps
Theorem 2.5	$\Pr[\bar{X}_n - \mu \geq t] \leq e^{-2nt^2/(u-l)^2}$	Confidence intervals for bounded averages; generalization

Result	Statement	Use
Theorem 2.6	$\Pr[\bar{X}_n - \mu \geq t] \leq e^{-nt^2/(2\sigma^2+\frac{2}{3}Rt)}$	Small variance or small rates: use it or Chernoff, not Hoeffding
Lemma 2.3	$\Pr[\cup_i A_i] \leq \sum_i \Pr[A_i]$	k estimates add $\ln k$ inside the logarithm
Corollary 2.2	All of a finite class within $\sqrt{\ln(2\lvert\Theta\rvert/\beta)/(2n)}$	Generalization; the class costs $\ln\lvert\Theta\rvert$
Proposition 2.2	$\Pr[Z \geq t] \leq \frac{1}{2}e^{-t^2/2}$ and $\leq \varphi(t)/t$	Normal noise; maxima grow like $\sqrt{\ln k}$
Proposition 2.3	$\Pr[\lvert X\rvert \geq t] = e^{-t/b}$	Laplace noise; exponential tails cost $\ln(1/\beta)$
Proposition 2.4	Sum of k: $e^{-\min(t^2/(8kb^2),\ t/(4b))}$	The normal approximation is not a tail bound
Theorem 2.7	$\Pr[W_n - W_0 \geq t] \leq e^{-2t^2/\sum_i d_i^2}$	Sums whose terms depend on the past
Theorem 2.8	$\Pr[f(X) - \mathbb{E}[f(X)] \geq t] \leq e^{-2t^2/\sum_i c_i^2}$	Functions of many independent inputs; stability

2.9. References

- Mitzenmacher & Upfal, *Probability and Computing*, ch. 4: the clearest Chernoff chapter, with worked computer-science examples
- Vershynin, *High-Dimensional Probability* (2018), ch. 2: sub-Gaussian and sub-exponential variables in depth. Read it after this chapter rather than before.
- Boucheron, Lugosi & Massart, *Concentration Inequalities* (2013), ch. 2 and 6: reference depth
- Bousquet & Elisseeff, "Stability and Generalization" (JMLR 2002): McDiarmid applied to learning algorithms

3. Divergences

Someone hands you a die that is either fair or loaded so that a six comes up half the time, and you may roll it as often as you like. How much does one roll tell you, on average, about which die you hold? How strongly can a single roll point the wrong way? How often can the best guess from one roll be right? Each answer is a **divergence**, a number that is zero when two distributions, call them P and Q, are equal and grows as they become easier to tell apart. The same questions measure how far a fitted model is from the truth and how many samples an A/B test needs.

One idea organizes the answers. To decide which distribution produced an outcome x, compare how likely x is under each. Write p and q for the probabilities, or densities, of P and Q. Their quotient is the **likelihood ratio** $r(x) = p(x)/q(x)$. Its logarithm $L(x) = \ln r(x)$ is the **evidence** for P, in nats, and it adds over independent observations. Every divergence is a summary of r. The Kullback–Leibler (KL) divergence averages the evidence over draws from P, the max-divergence takes its largest value, and the total variation (TV) distance measures the mass of p above q. Processing data, such as binning it, can only average likelihood ratios together, so no divergence grows under processing, and this **data-processing inequality** is what later parts use most.

The running example is the die, with P loaded and Q fair (Table 3.1). A six is three times more likely under P, evidence of $\ln 3 = 1.10$ nats for "loaded". Any other roll gives $\ln 0.6 = -0.51$ nats, weak evidence for "fair". Two continuous pairs recur, a normal and a Laplace density each against a shifted copy, whose evidence is unbounded and bounded (Probability distributions). Sums $\sum_x$ are integrals for densities, and Table 3.2 lists the symbols.

Table 3.1.: The loaded die P against the fair die Q, outcome by outcome. This is the running example of the chapter.

Roll x	1 to 5 (each)	6
$p(x)$ (loaded)	0.1	0.5
$q(x)$ (fair)	1/6	1/6
$r(x) = p(x)/q(x)$	0.6	3
$\ln r(x)$	-0.511	1.099

Table 3.2.: The symbols of this chapter.

Symbol	Meaning
$r = p/q$; $L = \ln r$	the likelihood ratio; the evidence for P
$\mathbb{E}_P$, $\mathbb{E}_Q$	an average over draws from P, from Q
D_{KL}, D_α, D_∞, TV, E_γ, D_f	the divergences: KL, Rényi of order α, max, total variation, hockey-stick, f

Symbol	Meaning
$K_L(\lambda)$	the logarithm of the moment generating function of L under P
$\mu = s/\sigma$; $t = s/b$	a shift s between two normals in standard deviations; between two Laplace distributions in units of the scale b
$(y)_+$; K	the positive part $\max(y, 0)$; a channel, written without a subscript

3.1. KL divergence

Add up the evidence of n rolls of the die. With the loaded die the sum climbs, with the fair die it falls, and each family of runs drifts along a straight line (Figure 3.1). This section names its slope, the average evidence per roll, and proves the two facts every later use rests on: it is never negative, and it adds over independent observations.

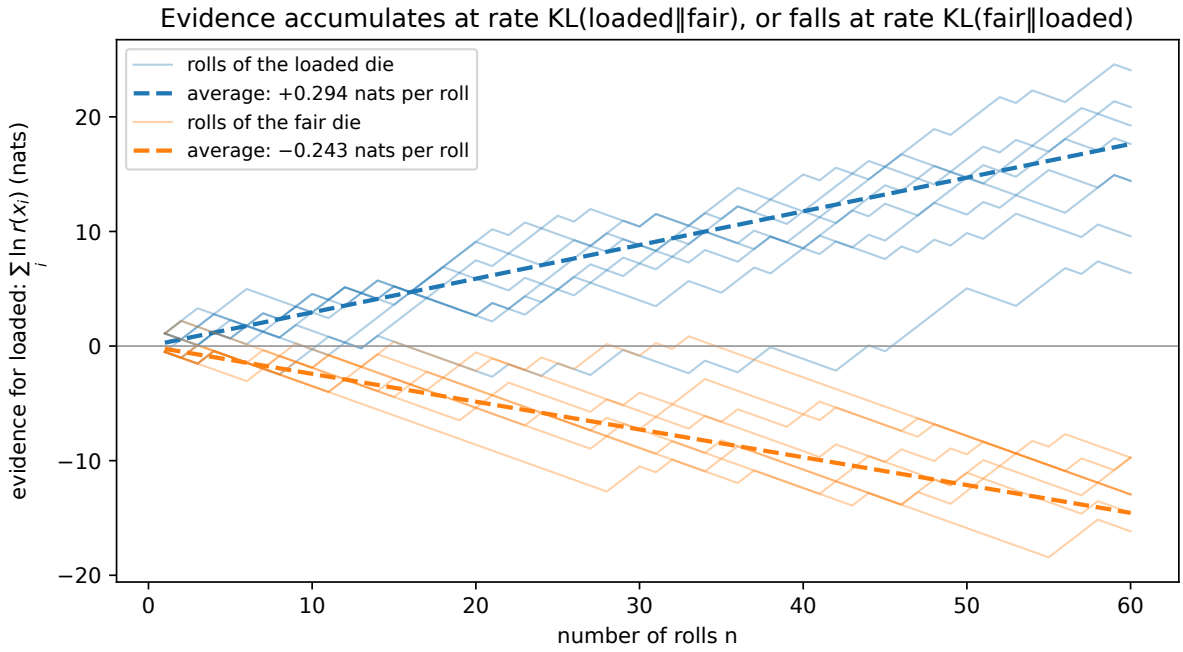


Figure 3.1.: Cumulative evidence for "loaded" over 60 rolls of the loaded die (blue) and the fair die (orange).

Definition 3.1 (Kullback–Leibler divergence)**.** The **KL divergence** of P from Q is the average evidence for P that one observation gives when it comes from P:

$$D_{\mathrm{KL}}(P\|Q) = \mathbb{E}_P[L] = \sum_x p(x) \ln \frac{p(x)}{q(x)}. \tag{3.1}$$

It is infinite when P puts mass where Q puts none.

That data from P cannot favour Q on average sounds obvious, but the proof needs a tool that compares a function of an average with the average of the function. A function f is **convex** when its graph bends upward, so that every chord lies above it, and **strictly convex** when the graph has no straight pieces. The next proof uses Jensen's inequality at step 2, and later sections use it again.

Lemma 3.1 (Jensen's inequality)**.** *Let f be convex and let X be a random variable with a finite average. Then*

$$f(\mathbb{E}[X]) \le \mathbb{E}[f(X)]. \tag{3.2}$$

If f is strictly convex, equality holds only when X is constant. For a concave f, whose graph bends downward, the inequality is reversed.

Proof.

1. *Use a tangent.* A convex function lies above its tangent lines. Take the tangent at $\mathbb{E}[X]$, with slope a, so that $f(x) \ge f(\mathbb{E}[X]) + a(x - \mathbb{E}[X])$ for every x.
2. *Average.* Put $x = X$ and average. The last term averages to 0, because X averages to $\mathbb{E}[X]$, which leaves $\mathbb{E}[f(X)] \ge f(\mathbb{E}[X])$.
3. *Equality.* A strictly convex graph touches the tangent only at $\mathbb{E}[X]$, so equality forces $X = \mathbb{E}[X]$ with probability 1. A concave function is minus a convex one, which flips the inequality.

□

For the die's two ratios, 0.6 and 3 each with probability $\frac{1}{2}$ under P, and $f(x) = x^2$, the average is 1.8 and the two sides are $1.8^2 = 3.24$ and $\frac{1}{2}(0.36 + 9) = 4.68$.

Theorem 3.1 (Gibbs' inequality and additivity)**.** *For any two distributions, $D_{\mathrm{KL}}(P\|Q) \ge 0$, with equality only if $P = Q$. Write $P_1 \times P_2$ for the distribution of a pair drawn independently from P_1 and P_2. Then*

$$D_{\mathrm{KL}}(P_1 \times P_2 \,\|\, Q_1 \times Q_2) = D_{\mathrm{KL}}(P_1\|Q_1) + D_{\mathrm{KL}}(P_2\|Q_2), \tag{3.3}$$

so n independent samples carry n times the KL divergence of one.

Proof. The plan is to write KL as the average of a convex function and apply Jensen, then to split the evidence of a pair into a sum.

1. *Flip the ratio.* Since $\ln(p/q) = -\ln(q/p)$, KL is the average under P of the strictly convex function $-\ln$ of q/p.
2. *Apply Lemma 3.1.* The average of $-\ln$ is at least $-\ln$ of the average:

$$D_{\mathrm{KL}}(P\|Q) \ge -\ln \mathbb{E}_P\Big[\frac{q(X)}{p(X)}\Big]. \tag{3.4}$$

3. *The inner average.* Each $p(x)$ cancels, so it is $\sum_{x:\,p(x)>0} q(x)$, the mass Q puts where P can land. That is at most 1, so minus its logarithm is at least 0.

4. *Equality.* Step 2 needs q/p constant where $p > 0$, and step 3 needs Q to put no mass elsewhere. Two distributions with a constant ratio on the same outcomes are equal, because both sum to 1.
5. *Additivity.* Under a product distribution the probability of a pair is the product of the probabilities, so the evidence of a pair is the sum $L_1(x_1) + L_2(x_2)$. Averages are linear, and the average of $L_i(X_i)$ involves only P_i, so it is $D_{\mathrm{KL}}(P_i \| Q_i)$.

□

KL also measures a model's fit. For a model q_θ with parameter θ, only the term $-\mathbb{E}_P[\ln q_\theta(X)]$ of $D_{\mathrm{KL}}(P\|Q_\theta)$ depends on θ, so minimizing this forward KL is maximum likelihood. Because KL averages under its first argument, the forward fit must cover every region the data visit, while the reverse divergence $D_{\mathrm{KL}}(Q_\theta\|P)$ of variational inference may lock onto one mode (Figure 3.2).

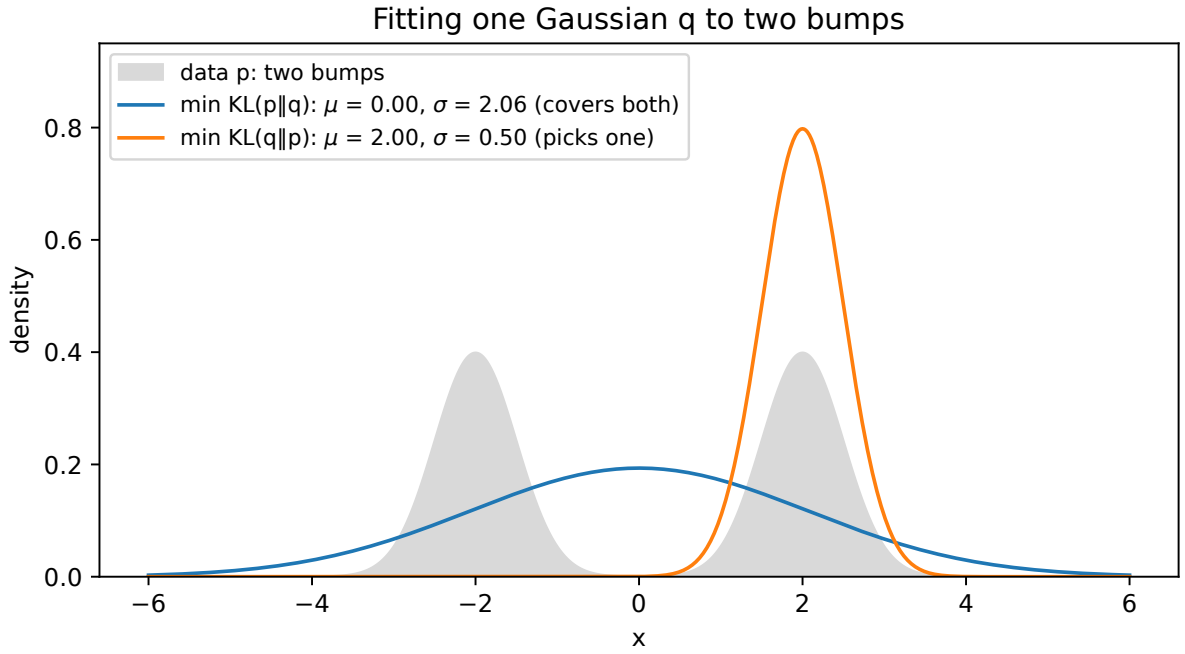


Figure 3.2.: One Gaussian fitted to two bumps by forward and by reverse KL.

Example 3.1. Under P the five non-sixes together have probability 0.5 and evidence $\ln 0.6$ each, and the six has probability 0.5 and evidence $\ln 3$, so

$$D_{\mathrm{KL}}(P\|Q) = 0.5 \ln 0.6 + 0.5 \ln 3 = 0.5 \ln 1.8 = 0.2939. \tag{3.5}$$

In the other direction the fair die weights the non-sixes $\frac{5}{6}$ and the six $\frac{1}{6}$, and the evidence is for "fair":

$$D_{\mathrm{KL}}(Q\|P) = \tfrac{5}{6} \ln \tfrac{1}{0.6} + \tfrac{1}{6} \ln \tfrac{1}{3} = 0.4257 - 0.1831 = 0.2426. \tag{3.6}$$

A loaded die gives itself away faster than a fair one proves its innocence, because its sixes carry strong evidence. In general, telling two distributions apart takes a number of observations of order $1/D_{\mathrm{KL}}$, which Section 3.9 makes precise.

3.2. Rényi divergence

KL is an average, so it cannot say how often the evidence is large, and later parts need that. Since $e^{D_{\mathrm{KL}}}$ is the geometric mean of r under P, other means can weight large ratios more. The **power mean** with exponent λ raises each value to the power λ, averages, and undoes the power:

$$m_\lambda = \left(\mathbb{E}_P[r^\lambda]\right)^{1/\lambda}. \tag{3.7}$$

Exponent -1 gives the harmonic mean, the limit 0 the geometric mean, 1 the arithmetic mean, and $\lambda \to \infty$ the largest value (Table 3.3). The Rényi divergence takes the logarithm of the power mean with exponent $\alpha - 1$, so its **order** α is a dial from the typical evidence to the worst case.

Table 3.3.: Means of the die's likelihood ratio under P. Their logarithms are the Rényi divergences of orders 0, 1, 2 and ∞.

Mean of r under P	Value	Its logarithm	Divergence
Harmonic, $1/\mathbb{E}_P[1/r]$	1.0000	0	D_0
Geometric, $e^{\mathbb{E}_P[\ln r]}$	1.3416	0.2939	$D_1 = D_{\mathrm{KL}}$
Arithmetic, $\mathbb{E}_P[r]$	1.8000	0.5878	D_2
Largest value, $\max r$	3.0000	1.0986	D_∞

Definition 3.2 (Rényi divergence)**.** For an order $\alpha \in (0,1) \cup (1,\infty)$, the **Rényi divergence** of P from Q is

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \sum_x p(x)^\alpha q(x)^{1-\alpha}. \tag{3.8}$$

The **max-divergence** is $D_\infty(P\|Q) = \ln \max_x r(x)$, an essential supremum for densities, and D_1 is defined as D_{KL}, which Theorem 3.2 shows is the limit at order 1.

The forms used below move an average under Q to one under P: an outcome three times as likely under P must count three times as much.

Lemma 3.2 (Change of measure)**.** *Suppose Q gives positive probability to every outcome that P does. Then for any function g,*

$$\mathbb{E}_P[g(X)] = \mathbb{E}_Q[r(X)\, g(X)]. \tag{3.9}$$

In particular $\mathbb{E}_Q[r] = 1$, and $\Pr_P[A] = \mathbb{E}_Q[r\, \mathbf{1}[A]]$ for every event A, with $\mathbf{1}[A]$ equal to 1 on A and 0 elsewhere.

Proof. Multiply and divide each term of $\sum_x p(x)g(x)$ by $q(x)$. The special cases take $g = 1$ and $g = \mathbf{1}[A]$. □

A second form links the divergence to the Chernoff method of Concentration inequalities, which bounds tails through the logarithm of a moment generating function. For the evidence under P that logarithm, the cumulant generating function of Probability distributions, is

$$K_L(\lambda) = \ln \mathbb{E}_P\left[e^{\lambda L}\right]. \tag{3.10}$$

Proposition 3.1 (Three forms of the Rényi divergence)**.** *For every order $\alpha \in (0,1) \cup (1,\infty)$,*

$$D_\alpha(P\|Q) = \frac{1}{\alpha-1} \ln \mathbb{E}_Q[r^\alpha] = \frac{1}{\alpha-1} \ln \mathbb{E}_P[r^{\alpha-1}]. \tag{3.11}$$

So the divergence is the logarithm of a power mean, and a chord slope of K_L:

$$D_\alpha(P\|Q) = \ln m_{\alpha-1}, \tag{3.12}$$

$$D_\alpha(P\|Q) = \frac{K_L(\alpha-1)}{\alpha-1}. \tag{3.13}$$

Proof.

1. Each term $p^\alpha q^{1-\alpha}$ equals $q\, r^\alpha$, so the sum is $\mathbb{E}_Q[r^\alpha]$, and Lemma 3.2 with $g = r^{\alpha-1}$ turns it into $\mathbb{E}_P[r^{\alpha-1}]$.
2. Dividing a logarithm by $\alpha - 1$ takes the logarithm of the power $1/(\alpha-1)$, which gives Equation 3.12.
3. Since $r^{\alpha-1} = e^{(\alpha-1)L}$, the logarithm of $\mathbb{E}_P[r^{\alpha-1}]$ is $K_L(\alpha-1)$, which gives Equation 3.13.

□

The behaviour of the family therefore follows from the shape of K_L, and the limit at order 1 needs its slope at 0.

Lemma 3.3 (Shape of the log moment generating function)**.** *If K_L is finite near 0, then $K_L(0) = 0$, $K_L'(0) = D_{\mathrm{KL}}(P\|Q)$, and K_L is convex.*

Proof.

1. *Value.* $K_L(0) = \ln \mathbb{E}_P[1] = 0$.
2. *Slope.* Differentiate the logarithm of the average, passing the derivative inside it:

$$K_L'(\lambda) = \frac{\mathbb{E}_P[Le^{\lambda L}]}{\mathbb{E}_P[e^{\lambda L}]}. \tag{3.14}$$

   This is the average of L under the **tilted** distribution, which weights each outcome x in proportion to $p(x)r(x)^\lambda$. At $\lambda = 0$ there is no tilt, and the slope is $\mathbb{E}_P[L]$, the KL divergence.
3. *Curvature.* Differentiating once more gives the variance of L under the tilted distribution, which is never negative, so K_L is convex.

□

Theorem 3.2 (Properties of the Rényi family)**.** *For any two distributions P and Q:*

1. Order 1. *If $D_\alpha(P\|Q)$ is finite for some $\alpha > 1$, then $D_\alpha \to D_{\mathrm{KL}}(P\|Q)$ as $\alpha \to 1$.*
2. Order ∞. *$D_\alpha \to D_\infty(P\|Q)$ as $\alpha \to \infty$, and the max-divergence is the largest log-ratio over events:*

$$D_\infty(P\|Q) = \ln \max_{A:\, \Pr_Q[A]>0} \frac{\Pr_P[A]}{\Pr_Q[A]}. \tag{3.15}$$

3. Order 0. $D_\alpha \to -\ln \Pr_Q[p(X) > 0]$ *as* $\alpha \to 0$.
4. Monotonicity. D_α *is non-decreasing in* α.
5. Additivity. $D_\alpha(P_1 \times P_2 \| Q_1 \times Q_2) = D_\alpha(P_1\|Q_1) + D_\alpha(P_2\|Q_2)$ *for every order.*
6. Skew symmetry. *For* $0 < \alpha < 1$, $D_\alpha(P\|Q) = \frac{\alpha}{1-\alpha} D_{1-\alpha}(Q\|P)$, *so* $D_{1/2}$ *is symmetric.*

Proof. The plan is to take the limits from the forms of Proposition 3.1 and from the sum, monotonicity from Jensen, and the last two claims from the algebra of the sum.

1. *Order 1.* The assumption makes K_L finite near 0, so Equation 3.13 is a difference quotient of K_L at 0, and it tends to $K_L'(0)$, which is KL by Lemma 3.3. Section F.4 in the appendices proves the limit from below without the assumption.
2. *Order* ∞*, upper bound.* Let $r_{\max}$ be the largest likelihood ratio on the support of P. Every $r \le r_{\max}$, so $\mathbb{E}_P[r^{\alpha-1}] \le r_{\max}^{\alpha-1}$, and Equation 3.12 gives $D_\alpha \le \ln r_{\max}$ for $\alpha > 1$.
3. *Order* ∞*, lower bound.* Keep only the term of a point x^* where $r = r_{\max}$, which makes the average smaller, so $\mathbb{E}_P[r^{\alpha-1}] \ge p(x^*) r_{\max}^{\alpha-1}$. Taking logarithms and dividing by $\alpha - 1$,

$$D_\alpha \ge \ln r_{\max} + \frac{\ln p(x^*)}{\alpha - 1}. \tag{3.16}$$

   The last term vanishes as $\alpha \to \infty$, so steps 2 and 3 squeeze D_α to $\ln r_{\max}$. For densities, replace x^* by a set of positive probability on which r is within any margin of $r_{\max}$.
4. *Events.* By Lemma 3.2, $\Pr_P[A] = \mathbb{E}_Q[r\,\mathbf{1}[A]] \le r_{\max} \Pr_Q[A]$, and the event $\{x^*\}$ reaches the bound.
5. *Order 0.* As $\alpha \to 0$, $p^\alpha \to 1$ wherever $p > 0$, so the sum tends to $\Pr_Q[p(X) > 0]$, and its logarithm is divided by $\alpha - 1 \to -1$.
6. *Monotonicity, positive exponents.* By Equation 3.12 it is enough that the power mean grows with its exponent. For $0 < \lambda_1 < \lambda_2$ the function $y \mapsto y^{\lambda_2/\lambda_1}$ is convex, so Lemma 3.1 applied to $Y = r^{\lambda_1}$ gives

$$\left(\mathbb{E}_P[r^{\lambda_1}]\right)^{\lambda_2/\lambda_1} \le \mathbb{E}_P[r^{\lambda_2}], \tag{3.17}$$

   and raising both sides to the positive power $1/\lambda_2$ keeps the order, so $m_{\lambda_1} \le m_{\lambda_2}$.
7. *Monotonicity, other exponents.* The logarithm is concave, so Jensen reversed gives $\ln \mathbb{E}_P[r^\lambda] \ge \lambda\, \mathbb{E}_P[\ln r]$. Dividing by λ puts $\ln m_\lambda$ above the logarithm of the geometric mean when $\lambda > 0$ and below it when $\lambda < 0$. For two negative exponents the power λ_2/λ_1 lies in $(0, 1)$, so the function is concave and step 6 reverses, but the negative power $1/\lambda_2$ reverses it back.
8. *Additivity.* The evidence of a pair is the sum of two independent terms, and the moment generating function of such a sum is the product of theirs (Lemma 1.2). So $K_L = K_{L_1} + K_{L_2}$, and Equation 3.13 divides by $\alpha - 1$.
9. *Skew symmetry.* Swapping p with q and α with $1 - \alpha$ leaves each term $p^\alpha q^{1-\alpha}$ unchanged. So the two sums are equal, and their logarithms are $(\alpha - 1) D_\alpha(P\|Q)$ and $-\alpha D_{1-\alpha}(Q\|P)$.

□

Figure 3.3 draws the chords for the die, and Figure 3.4 the tilted distribution with $\lambda = \alpha - 1$.

The family can also bound how often a sample from Q looks like strong evidence for P. The tool is Markov's inequality, which says that a quantity that cannot be negative exceeds a level with probability at most its average divided by that level.

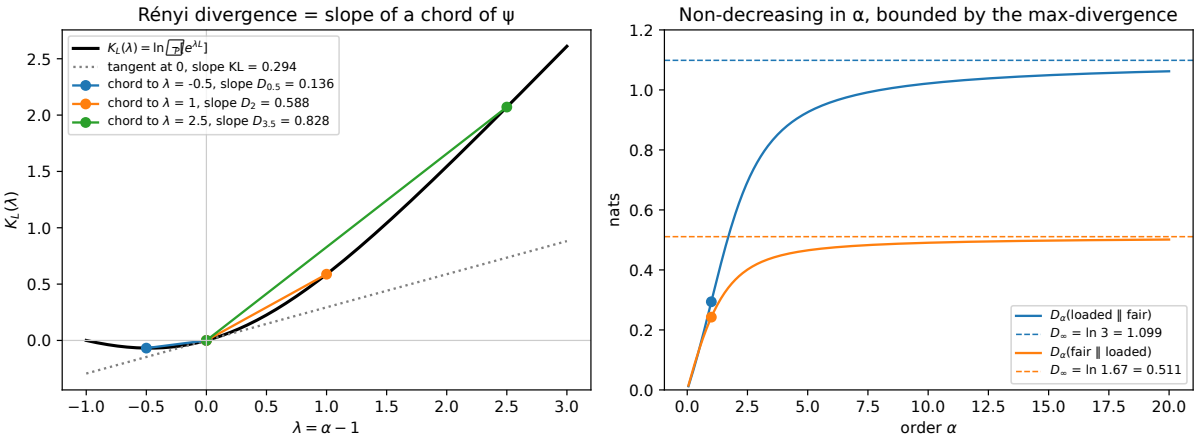


Figure 3.3.: Left: K_L for the die, whose chord to $\lambda = \alpha - 1$ has slope D_α. Right: D_α against α.

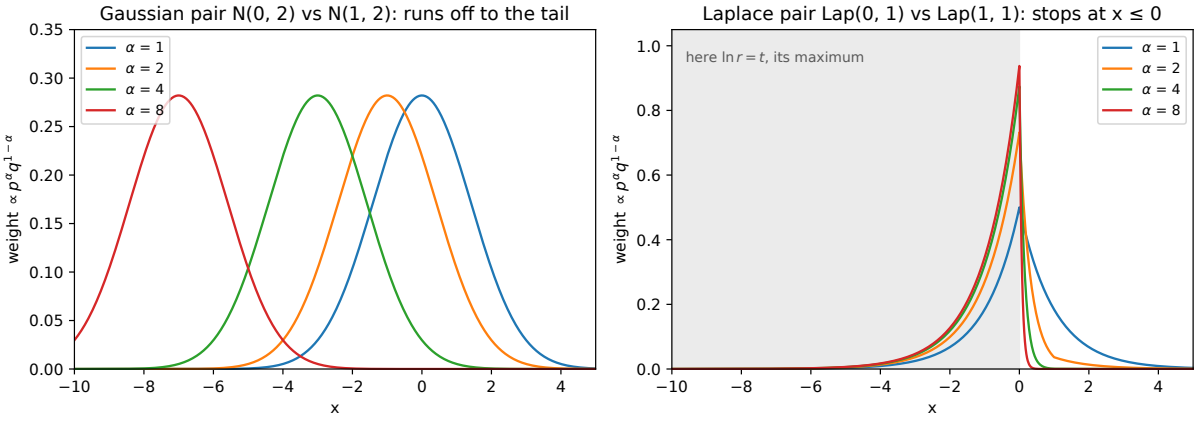


Figure 3.4.: The tilted weights $p^\alpha q^{1-\alpha}$ for a Gaussian pair (left) and a Laplace pair (right).

Proposition 3.2 (Tail of the evidence)**.** *For every evidence level c and every order $\alpha > 0$,*

$$\Pr_Q[L \geq c] \leq \exp\big(-\alpha c + (\alpha - 1) D_\alpha(P\|Q)\big). \tag{3.18}$$

Proof.

1. The quantity $r^\alpha = e^{\alpha L}$ cannot be negative, and it is at least $e^{\alpha c}$ exactly when $L \geq c$, so Markov's inequality gives $\Pr_Q[L \geq c] \leq e^{-\alpha c}\,\mathbb{E}_Q[r^\alpha]$.
2. By Proposition 3.1, $\mathbb{E}_Q[r^\alpha] = e^{(\alpha-1)D_\alpha(P\|Q)}$, and at $\alpha = 1$ it is 1 by Lemma 3.2.

□

At order 1 the bound is e^{-c} for every pair, and larger orders use more about the pair to do much better (Example 3.3).

Example 3.2. Table 3.4 gives the die's Rényi divergences in both directions.

Table 3.4.: Rényi divergences between the loaded and the fair die.

α	0.5	1 (KL)	2	5	10	∞
$D_\alpha(P\|Q)$, loaded ‖ fair	0.1362	0.2939	0.5878	0.9257	1.0216	1.0986
$D_\alpha(Q\|P)$, fair ‖ loaded	0.1362	0.2426	0.3677	0.4653	0.4906	0.5108

At order 2, r is 0.6 with probability $\frac{5}{6}$ and 3 with probability $\frac{1}{6}$ under the fair die, so $D_2(P\|Q) = \ln(\frac{5}{6} \times 0.36 + \frac{1}{6} \times 9) = \ln 1.8 = 0.5878$. Each row increases, as monotonicity requires, and the entries at order 0.5 agree, as skew symmetry requires.

3.3. Two Gaussians with equal variance

The most common pair in practice is a distribution and a shifted copy of it, such as a feature before and after a drift. For two normals with equal variance every divergence has a closed form in the shift measured in standard deviations: KL is half its square, each Rényi divergence is α times the KL, and the max-divergence is infinite, because the evidence is a line with no top.

Proposition 3.3 (Divergences of two shifted normals)**.** *Let $P = \mathcal{N}(0, \sigma^2)$ and $Q = \mathcal{N}(s, \sigma^2)$, and write $\mu = s/\sigma$ for the shift in standard deviations. Then for every order α,*

$$D_{\mathrm{KL}}(P\|Q) = \frac{\mu^2}{2}, \qquad D_\alpha(P\|Q) = \frac{\alpha\mu^2}{2}, \qquad D_\infty(P\|Q) = \infty. \tag{3.19}$$

The same holds for any two means a distance s apart, since shifting both changes nothing.

Proof. The plan is to compute the evidence, average it for KL, and complete the square in the Rényi integral.

1. *The evidence.* The normal density with mean m is $\frac{1}{\sigma\sqrt{2\pi}}e^{-(x-m)^2/(2\sigma^2)}$. The constants cancel in the ratio, and expanding $(x-s)^2$ cancels the x^2 terms:
$$L(x) = \frac{(x-s)^2 - x^2}{2\sigma^2} = \frac{s^2 - 2sx}{2\sigma^2}. \tag{3.20}$$
2. *KL.* The evidence is linear in x, so its average under P is its value at the mean 0, which is $s^2/(2\sigma^2) = \mu^2/2$.
3. *Constants.* The factors $\frac{1}{\sigma\sqrt{2\pi}}$ raised to the powers α and $1-\alpha$ multiply back to one such factor. What remains of $p^\alpha q^{1-\alpha}$ is $\exp(-B(x)/(2\sigma^2))$, with $B(x) = \alpha x^2 + (1-\alpha)(x-s)^2$.
4. *Complete the square* around $(1-\alpha)s$. The constant left over is $(1-\alpha)s^2 - (1-\alpha)^2 s^2$, so
$$B(x) = \big(x - (1-\alpha)s\big)^2 + \alpha(1-\alpha)s^2. \tag{3.21}$$
5. *Integrate.* The squared term, with the factor $\frac{1}{\sigma\sqrt{2\pi}}$, is the density of $\mathcal{N}((1-\alpha)s, \sigma^2)$ and integrates to 1, so
$$\int p^\alpha q^{1-\alpha}\,\mathrm{d}x = \exp\Big(-\frac{\alpha(1-\alpha)s^2}{2\sigma^2}\Big). \tag{3.22}$$
6. *Divide.* Taking the logarithm and dividing by $\alpha - 1$ gives $\alpha s^2/(2\sigma^2)$, since $-\alpha(1-\alpha)/(\alpha-1) = \alpha$.
7. *Max-divergence.* The line L has a non-zero slope, so it is unbounded above.

□

Doubling the noise is the same as halving the shift. By additivity, independent coordinates with shifts s_i give $\alpha\sum_i s_i^2/(2\sigma^2)$, which depends only on the length of the shift vector (Section E.1 in the appendices proves the rotation invariance behind this). Every finite order is finite while the worst case is infinite, because large ratios exist but are rare, and Equation 3.18 measures how rare.

Example 3.3. Take $\mu = 1$, so $D_{\mathrm{KL}} = 0.5$ and $D_\alpha = \alpha/2$. How often does a draw from Q give at least $c = 2$ nats of evidence for P? The evidence decreases in x, so $L \geq c$ exactly when $x \leq \frac{s}{2} - \frac{c\sigma^2}{s}$. Under Q, $x = s + \sigma Z$ with Z standard normal. Write Φ for the standard normal cumulative distribution function, so that $\Phi(z)$ is the probability that $Z \leq z$. Then
$$\Pr_Q[L \geq c] = \Phi\Big(-\frac{\mu}{2} - \frac{c}{\mu}\Big) = \Phi(-2.5) = 0.0062. \tag{3.23}$$
At order 1, Equation 3.18 gives Markov's bound $e^{-2} = 0.1353$. With $D_\alpha = \alpha\mu^2/2$ the exponent of the bound is the parabola $-\alpha c + \alpha(\alpha-1)\mu^2/2$, smallest at $\alpha = c/\mu^2 + \frac{1}{2} = 2.5$. There it equals -3.125, so the bound is $e^{-3.125} = 0.0439$, 7 times the exact value against 22 times for Markov's.

3.4. Two Laplace distributions with shifted location

The Laplace distribution, with density $\frac{1}{2b}e^{-|x-m|/b}$ for centre m and scale b, is the other standard noise model. Its evidence against a shifted copy is bounded, so this section computes the whole Rényi family to see what a bounded ratio changes. The max-divergence turns out to be the shift in units of the scale, and KL slightly less.

Proposition 3.4 (Divergences of two shifted Laplace distributions)**.** *Let $P = \mathrm{Lap}(0, b)$ and $Q = \mathrm{Lap}(s, b)$ with $s \geq 0$, and write $t = s/b$. Then*

$$D_{\mathrm{KL}}(P\|Q) = t + e^{-t} - 1, \qquad D_{\infty}(P\|Q) = t, \tag{3.24}$$

and for every order $\alpha \neq \frac{1}{2}, 1$,

$$D_{\alpha}(P\|Q) = \frac{1}{\alpha - 1} \ln\left(\frac{\alpha}{2\alpha - 1}e^{(\alpha-1)t} + \frac{\alpha - 1}{2\alpha - 1}e^{-\alpha t}\right). \tag{3.25}$$

At $\alpha = \frac{1}{2}$ the bracket is replaced by its limit $(1 + \frac{t}{2})e^{-t/2}$.

Proof. The plan is to rescale, read the max-divergence off the evidence, average the evidence for KL, and integrate the Rényi sum piece by piece.

1. *Rescale.* Dividing by b is a bijection, which changes no divergence (apply Theorem 3.5 to it and to its inverse), so take $b = 1$ and $s = t$. The evidence is the difference of the exponents, $L(x) = |x - t| - |x|$.
2. *Max-divergence.* Two numbers a distance t apart have absolute values that differ by at most t, by the triangle inequality, so $L \leq t$. For $x \leq 0$ both x and $x - t$ are negative, and $L(x) = t$ exactly.
3. *KL, set up.* KL is $\mathbb{E}[|X - t|] - \mathbb{E}[|X|]$ with $X \sim \mathrm{Lap}(0, 1)$, and the second average is 1, the mean of the exponential density of $|X|$.
4. *KL, remove the absolute value.* Every number satisfies $|y| = y + 2(-y)_+$. Put $y = X - t$ and use $\mathbb{E}[X] = 0$:
$$\mathbb{E}[|X - t|] = -t + 2\,\mathbb{E}[(t - X)_+]. \tag{3.26}$$
5. *KL, the positive part.* $(t - X)_+$ is the length of the interval from X up to t, so its average integrates the distribution function, $\frac{1}{2}e^{x}$ below 0 and $1 - \frac{1}{2}e^{-x}$ above, over $x < t$:
$$\mathbb{E}[(t - X)_+] = \int_{-\infty}^{0} \tfrac{1}{2}e^{x}\,\mathrm{d}x + \int_{0}^{t} (1 - \tfrac{1}{2}e^{-x})\,\mathrm{d}x = t + \tfrac{1}{2}e^{-t}. \tag{3.27}$$
So $\mathbb{E}[|X - t|] = t + e^{-t}$, and subtracting 1 gives the KL.
6. *Rényi.* The integral $I(\alpha) = \frac{1}{2}\int \exp(-\alpha|x| - (1 - \alpha)|x - t|)\,\mathrm{d}x$ has an exponent linear in x on each of the pieces $x < 0$, $0 \leq x \leq t$ and $x > t$, which integrate to $e^{-(1-\alpha)t}$, $\frac{e^{-\alpha t} - e^{-(1-\alpha)t}}{1 - 2\alpha}$ and $e^{-\alpha t}$. Adding the pieces and halving, $e^{-\alpha t}$ gets the coefficient $\frac{1}{2}(1 + \frac{1}{1-2\alpha}) = \frac{\alpha - 1}{2\alpha - 1}$, and $e^{(\alpha-1)t}$ gets $\frac{1}{2}(1 - \frac{1}{1-2\alpha}) = \frac{\alpha}{2\alpha-1}$. Then $\frac{1}{\alpha-1}\ln I(\alpha)$ is Equation 3.25.

□

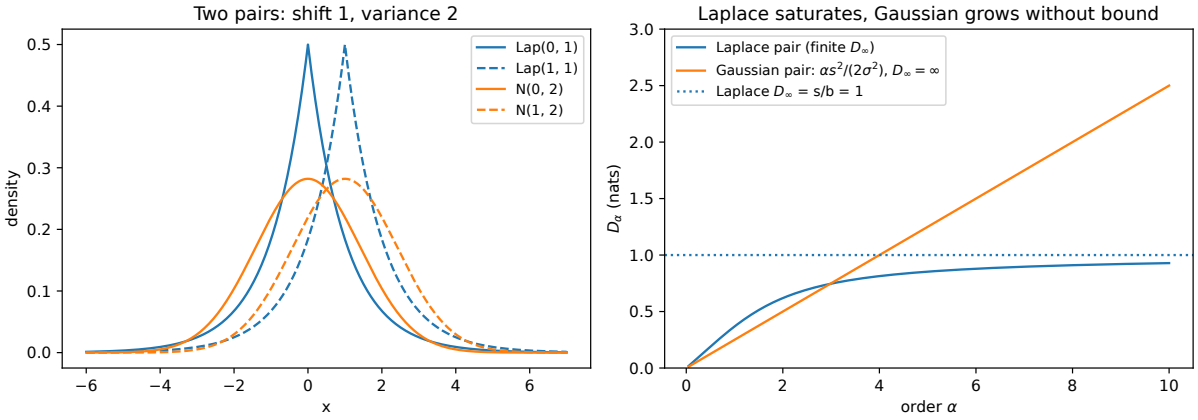


Figure 3.5.: A Laplace and a Gaussian pair with shift 1 and variance 2, and their Rényi divergences.

The worst case is not rare here: the evidence equals t on all of $x \le 0$, which has probability $\frac{1}{2}$ under P. Against a Gaussian pair of the same variance $2b^2$, the Laplace pair is further apart at low orders, but the Gaussian D_α grows linearly while the Laplace one levels off at t, and at $t = 1$ they cross at order 2.98 (Figure 3.5).

Example 3.4. Table 3.5 compares the shift $t = 1$ with the Gaussian pair of variance 2, for which $D_\alpha = \alpha/4$.

Table 3.5.: Rényi divergences of a Laplace pair and a Gaussian pair with the same shift and variance.

	KL	$D_{0.5}$	D_2	D_{10}	D_∞
Laplace, $t = 1$	0.3679	0.1891	0.6191	0.9287	1.0000
Gaussian, same variance	0.2500	0.1250	0.5000	2.5000	∞

The Laplace KL is $e^{-1} = 0.3679$. At order 2 the coefficients in Equation 3.25 are $\frac{2}{3}$ and $\frac{1}{3}$, so $D_2 = \ln(\frac{2}{3}e + \frac{1}{3}e^{-2}) = \ln 1.8573 = 0.6191$.

3.5. Total variation

KL and Rényi divergences are convenient to compute, but total variation answers the operational questions. It says how well the best possible test tells P from Q with one observation, and how far any bounded statistic can move when Q is replaced by P.

Definition 3.3 (Total variation distance)**.** The **total variation distance** is the largest difference between the probabilities that P and Q give to one event:

$$\mathrm{TV}(P, Q) = \max_A \left| \Pr_P[A] - \Pr_Q[A] \right|. \tag{3.28}$$

The hockey-stick divergence of Section 3.6 asks the same question with a weight on Q. Which event, or which soft event that includes each outcome with some probability, makes $\Pr_P[A] - \gamma \Pr_Q[A]$ largest? The next lemma answers it once for both sections.

Lemma 3.4 (The best event)**.** *Let $\gamma \geq 0$. The largest value of $\Pr_P[A] - \gamma \Pr_Q[A]$ over events A is*

$$\max_A \left[\Pr_P[A] - \gamma \Pr_Q[A] \right] = \sum_x \left(p(x) - \gamma q(x) \right)_+, \tag{3.29}$$

reached at $A^* = \{x : r(x) > \gamma\}$. A soft event, which includes each outcome x with probability $\phi(x) \in [0,1]$, does no better: $\sum_x \phi(x)(p(x) - \gamma q(x))$ is at most the same sum.

Proof.

1. The quantity is $\sum_{x \in A}(p(x) - \gamma q(x))$, and an outcome adds a positive term exactly when $p(x) > \gamma q(x)$. So the sum is largest when A holds those outcomes and no others.
2. Each term of the soft sum is $\phi(x)$ times the bracket, which is at most the positive part of the bracket. So the soft sum is at most the sum of the positive parts.

□

Theorem 3.3 (Forms and uses of total variation)**.** *The total variation is the largest gap over events, the mass of p above q, half the total absolute difference, and the average under Q of how far r exceeds 1:*

$$\mathrm{TV}(P,Q) = \max_A |\Pr_P[A] - \Pr_Q[A]| = \sum_x (p-q)_+ = \frac{1}{2}\sum_x |p-q| = \mathbb{E}_Q[(r-1)_+]. \tag{3.30}$$

The overlap is $\sum_x \min(p,q) = 1 - \mathrm{TV}(P,Q)$. Moreover:

1. Bounded statistics. *Every function g with values in $[0,1]$ satisfies $|\mathbb{E}_P[g] - \mathbb{E}_Q[g]| \leq \mathrm{TV}(P,Q)$.*
2. Best single-shot test. *If nature picks P or Q with probability $\frac{1}{2}$ each and shows you one draw, the best guess is correct with probability*

$$\Pr[\textit{correct}] = \frac{1}{2} + \frac{1}{2}\mathrm{TV}(P,Q). \tag{3.31}$$

3. Metric. *TV is symmetric, zero only when $P = Q$, at most 1, and satisfies the triangle inequality.*

Proof. The plan is to get the forms from Lemma 3.4 with weight 1, then to treat the statistic and the test as soft events.

1. *Forms.* The lemma with $\gamma = 1$ gives $\max_A[\Pr_P[A] - \Pr_Q[A]] = \sum(p-q)_+$. The differences $p - q$ sum to 0, so their positive and negative parts have equal mass. So the best event for Q over P gives the same value, which justifies the absolute value, and the half-sum of $|p-q|$ is that common value. Since $p - q = q(r-1)$, the sum is also $\mathbb{E}_Q[(r-1)_+]$.
2. *Overlap.* At each x, $\min(p,q) = p - (p-q)_+$, and summing over x gives $1 - \mathrm{TV}$.
3. *Bounded statistics.* The gap $\mathbb{E}_P[g] - \mathbb{E}_Q[g] = \sum_x g(x)(p(x) - q(x))$ is a soft event, so the lemma bounds it by TV, and $1 - g$ bounds the other sign.

4. *Best test.* Saying "P" on A is right with probability $\frac{1}{2}\Pr_P[A] + \frac{1}{2}(1 - \Pr_Q[A])$, which is $\frac{1}{2} + \frac{1}{2}(\Pr_P[A] - \Pr_Q[A])$. Step 1 maximizes the bracket, and a randomized test is a soft event that does no better.
5. *Metric.* The half-sum is half the ℓ_1 distance between two probability tables, which has the three properties and is at most 1.

□

So no classifier beats $\frac{1}{2} + \frac{1}{2}$TV accuracy from one draw, and a data shift of TV 0.01 moves any fixed classifier's accuracy by at most 0.01. TV is not additive, so for n samples you compute KL and convert it with Section 3.9.

Example 3.5. On the die only the six has $p > q$, so TV $= 0.5 - \frac{1}{6} = \frac{1}{3}$. The best test says "loaded" exactly when it sees a six, and it is right with probability $\frac{2}{3}$. For the Gaussian pair the curves cross at the midpoint $s/2$, so the best event is $\{x < s/2\}$. It has probability $\Phi(\frac{\mu}{2})$ under P and $\Phi(-\frac{\mu}{2}) = 1 - \Phi(\frac{\mu}{2})$ under Q, so

$$\mathrm{TV} = 2\Phi\left(\frac{\mu}{2}\right) - 1. \tag{3.32}$$

At $\mu = 1$ this is 0.3829, and the best test is right with probability $\Phi(0.5) = 0.6915$ (Figure 3.6, left). At $\mu = 0.1$ it is only 0.0399.

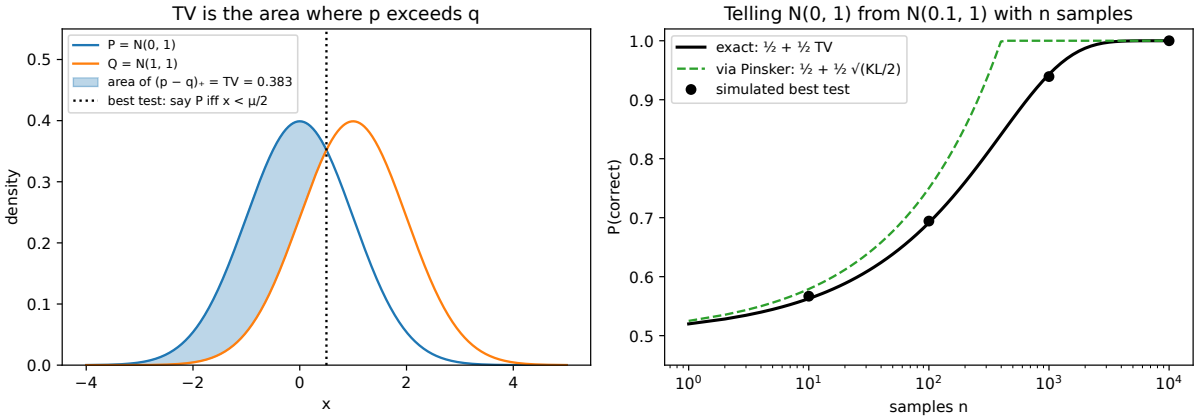


Figure 3.6.: Left: the total variation of a Gaussian pair, shaded. Right: the best accuracy for n samples (Example 3.10).

3.6. Hockey-stick divergence

TV measures the mass of p above q. Now scale q up by a factor $\gamma \geq 1$ and measure the mass of p that still sticks out above γq (Figure 3.7). Only likelihood ratios above γ count, so this measures how much of the difference comes from large ratios. It appears in tests with unequal priors, and in **rejection sampling**, which draws from a target p by proposing from q under an envelope γq.

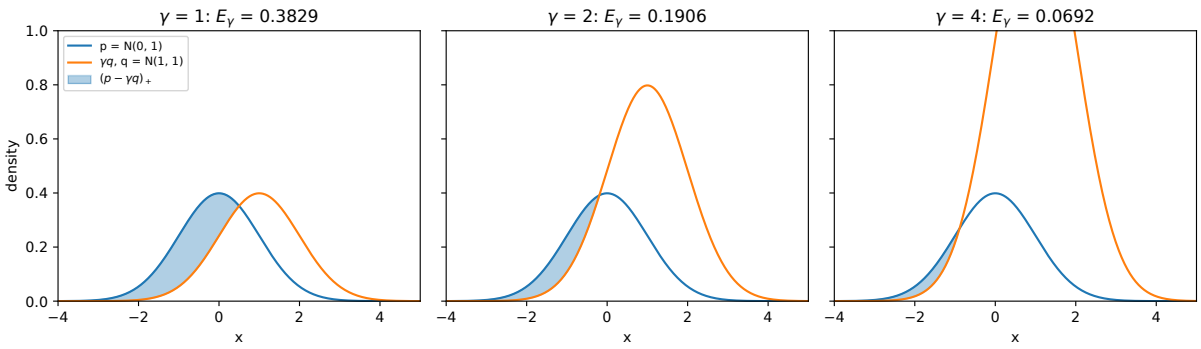


Figure 3.7.: The mass of $\mathcal{N}(0,1)$ above γ times $\mathcal{N}(1,1)$: 0.3829, 0.1906 and 0.0692 at $\gamma = 1, 2, 4$.

Definition 3.4 (Hockey-stick divergence)**.** For $\gamma \geq 1$, which makes it vanish when $P = Q$, the **hockey-stick divergence** is

$$E_\gamma(P\|Q) = \max_A \left[\Pr_P[A] - \gamma \Pr_Q[A]\right]. \tag{3.33}$$

Proposition 3.5 (Properties of the hockey-stick divergence)**.** *Let $\gamma \geq 1$ and $c = \ln \gamma$. Then:*

1. Forms. $E_\gamma(P\|Q) = \sum_x (p - \gamma q)_+ = \mathbb{E}_Q[(r - \gamma)_+]$, *and* $E_1(P\|Q) = \mathrm{TV}(P, Q)$.
2. Shape. E_γ *is non-increasing and convex in* γ, *and it is 0 exactly when* $\gamma \geq e^{D_\infty(P\|Q)}$.
3. Tails. $E_\gamma(P\|Q) = \Pr_P[L > c] - \gamma \Pr_Q[L > c]$.
4. Unequal priors. *If P has prior probability $\pi \leq \frac{1}{2}$, the smallest probability of a wrong guess from one observation is $\pi(1 - E_\gamma(P\|Q))$ with $\gamma = (1 - \pi)/\pi$, the prior odds against P.*

Proof.

1. *Forms.* Lemma 3.4 with weight γ gives the sum, and $p - \gamma q = q(r - \gamma)$ turns it into an average under Q.
2. *Shape.* Each term $(r(x) - \gamma)_+$ is a descending line in γ that flattens at 0, so it is non-increasing and convex, and so is their average. All terms vanish exactly when γ is at least the largest ratio.
3. *Tails.* The terms are non-zero only on $\{r > \gamma\} = \{L > c\}$, where p sums to $\Pr_P[L > c]$ and γq to $\gamma \Pr_Q[L > c]$.
4. *Priors.* Guessing "P" on A errs when a draw from P falls outside A or one from Q falls inside it. Factoring out π, the error is

$$\pi(1 - \Pr_P[A]) + (1 - \pi)\Pr_Q[A] = \pi - \pi\left[\Pr_P[A] - \gamma \Pr_Q[A]\right], \tag{3.34}$$

which is smallest when the bracket is largest, that is equal to E_γ.

□

Always guessing Q errs with probability π, and one observation removes a fraction E_γ of that error. For the shifted pairs, property 3 gives the whole curve in closed form, and later parts use exactly these two curves.

Proposition 3.6 (Hockey-stick divergence of the shifted pairs)**.** *Let $\gamma \geq 1$ and $c = \ln \gamma$. For the normal pair of Proposition 3.3,*

$$E_\gamma(P\|Q) = \Phi\Big(\frac{\mu}{2} - \frac{c}{\mu}\Big) - \gamma\,\Phi\Big(-\frac{\mu}{2} - \frac{c}{\mu}\Big), \tag{3.35}$$

which is positive for every γ. For the Laplace pair of Proposition 3.4, $E_\gamma = 0$ for $\gamma > e^t$, and

$$E_\gamma(P\|Q) = 1 - e^{-(t-c)/2} \quad \text{for } 1 \leq \gamma \leq e^t. \tag{3.36}$$

Proof.

1. *Normal pair.* The evidence is the line of Proposition 3.3, so $L > c$ exactly when $x/\sigma < \frac{\mu}{2} - \frac{c}{\mu}$. Under P, x/σ is standard normal, and under Q it is shifted by μ, so property 3 gives the formula. It is positive by property 2, because $D_\infty = \infty$.
2. *Laplace pair, the event.* With $b = 1$, the evidence is t on $x \leq 0$, $-t$ on $x \geq t$ and $t - 2x$ in between. So for $c < t$, $L > c$ exactly when $x < \frac{t-c}{2}$, a point above 0.
3. *Laplace pair, the tails.* That event has probability $1 - \frac{1}{2}e^{-(t-c)/2}$ under P. Under Q it needs a $\mathrm{Lap}(0, 1)$ variable below $-\frac{t+c}{2}$, which has probability $\frac{1}{2}e^{-(t+c)/2}$.
4. *Laplace pair, combine.* Since $\gamma e^{-(t+c)/2} = e^{-(t-c)/2}$, property 3 gives the formula. Beyond e^t, the largest ratio, property 2 gives 0.

□

The curve $\gamma \mapsto E_\gamma$ starts at TV and reaches zero at the largest ratio, which the Gaussian curve never does (Figure 3.8).

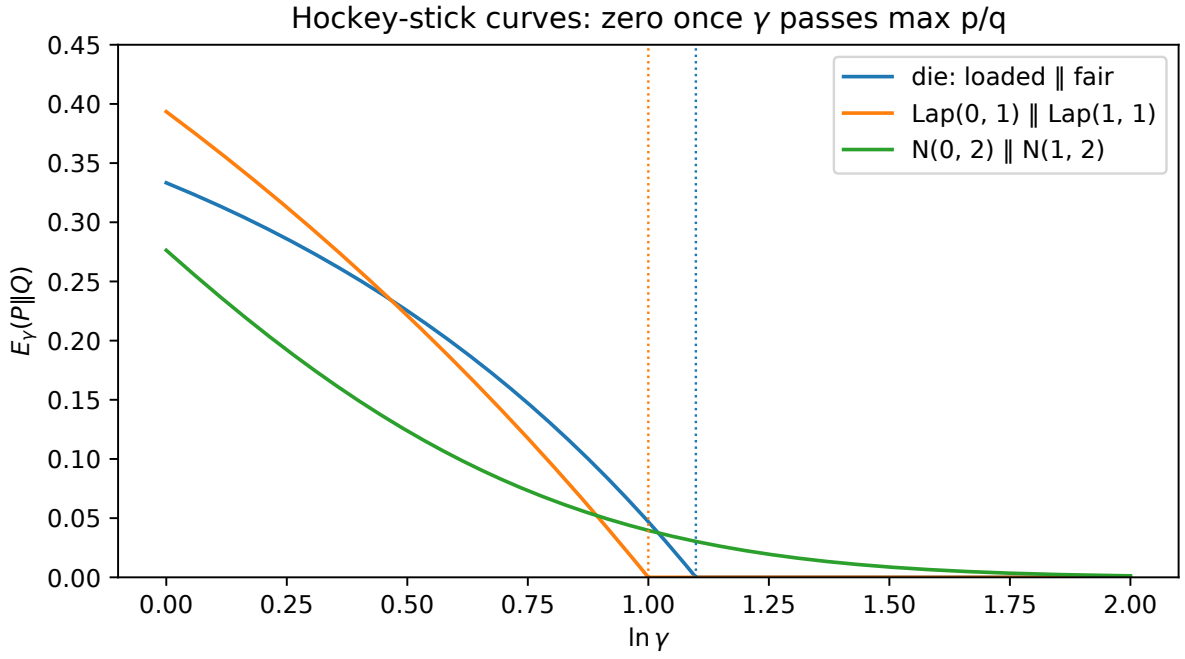


Figure 3.8.: E_γ against $\ln \gamma$ for the die, a Laplace pair and a Gaussian pair.

Example 3.6. On the die, only the six has $r > \gamma$ for $1 \le \gamma < 3$, so $E_\gamma = 0.5 - \gamma/6$ falls linearly to zero at $\gamma = 3$, the largest ratio. In rejection sampling of the loaded die from fair rolls, an envelope $2q$ leaves the mass $E_2(P\|Q) = \frac{1}{6}$ of the six uncovered, and the sampled six comes up 40% of the time instead of 50%. The error in TV is never more than E_γ.

3.7. f-divergences

KL, TV and E_γ were defined separately, but each is an average under Q of some function of the likelihood ratio. Naming that function gives one family, with one proof of non-negativity here and one of the data-processing inequality in Section 3.8. A ratio of 1 is no evidence either way, so the function must vanish there.

Definition 3.5 (f-divergence)**.** Let the **generator** $f\colon (0,\infty) \to \mathbb{R}$ be convex with $f(1) = 0$. The **f-divergence** is

$$D_f(P\|Q) = \mathbb{E}_Q\big[f(r(X))\big] = \sum_x q(x)\, f\Big(\frac{p(x)}{q(x)}\Big). \tag{3.37}$$

Example 3.7. Table 3.6 lists the common generators with their values on the die. The generator $u \ln u$ gives KL because $\mathbb{E}_Q[r \ln r] = \mathbb{E}_P[\ln r]$ by Lemma 3.2.

Table 3.6.: Common f-divergences, their generators, and their values for the loaded die against the fair die.

Divergence	$f(u)$	Die, loaded ‖ fair
KL, $D_{\mathrm{KL}}(P\|Q)$	$u \ln u$	0.2939
Reverse KL, $D_{\mathrm{KL}}(Q\|P)$	$-\ln u$	0.2426
Total variation	$\frac{1}{2}\lvert u-1\rvert$	0.3333
Hockey-stick E_γ $(\gamma = 2)$	$(u-\gamma)_+$	0.1667
χ^2	$(u-1)^2$	0.8000
Squared Hellinger H^2	$(\sqrt{u}-1)^2$	0.1317

Rényi divergence is not a member, but it is an increasing function of one, so it inherits what the family proves.

Proposition 3.7 (Non-negativity, and Rényi as a transform)**.** *Every f-divergence satisfies $D_f(P\|Q) \ge 0$, with equality when $P = Q$. For $\alpha > 1$ the generator $u^\alpha - 1$, and for $0 < \alpha < 1$ the generator $1 - u^\alpha$, give f-divergences with*

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln\big(1 + D_f(P\|Q)\big) \ \ (\alpha > 1), \qquad D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln\big(1 - D_f(P\|Q)\big) \ \ (\alpha < 1), \tag{3.38}$$

each an increasing function of D_f.

Proof.

1. *Non-negativity.* By Lemma 3.1 and Lemma 3.2, $\mathbb{E}_Q[f(r)] \geq f(\mathbb{E}_Q[r]) = f(1) = 0$, with equality when $r = 1$ everywhere.
2. *Orders above 1.* The generator $u^\alpha - 1$ is convex and gives $D_f = \mathbb{E}_Q[r^\alpha] - 1$, so Proposition 3.1 gives the first formula.
3. *Orders below 1.* Now u^α is concave, so $1 - u^\alpha$ is convex, with $D_f = 1 - \mathbb{E}_Q[r^\alpha]$. The logarithm decreases in D_f and $\frac{1}{\alpha - 1}$ is negative, so the second formula increases in D_f.

□

A convex function is a sum of hockey-stick shapes weighted by its curvature, so every f-divergence is a weighted sum of the curves E_γ, which therefore carry all the information about a pair.

Theorem 3.4 (f-divergences from the hockey-stick curves)**.** *If the generator f is twice differentiable, then*

$$D_f(P\|Q) = \int_1^\infty f''(\gamma)\, E_\gamma(P\|Q)\, \mathrm{d}\gamma + \int_0^1 \gamma f''(\gamma)\, E_{1/\gamma}(Q\|P)\, \mathrm{d}\gamma. \tag{3.39}$$

Proof. The plan is to write $f(u)$ as hockey sticks for each fixed u, then put in $u = r(X)$ and average under Q.

1. *Taylor's theorem around 1.* With an integral remainder, $f(u)$ is its value at 1, plus the tangent line, plus the curvature accumulated between 1 and u:
$$f(u) = f(1) + f'(1)(u - 1) + \int_1^u f''(\gamma)(u - \gamma)\, \mathrm{d}\gamma. \tag{3.40}$$
2. *Hockey sticks.* The remainder equals $\int_1^\infty f''(\gamma)(u - \gamma)_+ \, \mathrm{d}\gamma + \int_0^1 f''(\gamma)(\gamma - u)_+ \, \mathrm{d}\gamma$. For $u \geq 1$ only the first integral is non-zero and it stops at $\gamma = u$. For $u < 1$, swapping the limits of $\int_1^u$ gives the second.
3. *Average.* Put $u = r(X)$ and average under Q. The first two terms vanish, because $f(1) = 0$ and $\mathbb{E}_Q[r - 1] = 0$.
4. *The two integrals.* $\mathbb{E}_Q[(r - \gamma)_+] = E_\gamma(P\|Q)$ by Proposition 3.5. Also $\mathbb{E}_Q[(\gamma - r)_+] = \sum_x (\gamma q - p)_+ = \gamma \sum_x (q - p/\gamma)_+$, which is $\gamma E_{1/\gamma}(Q\|P)$.

□

Each generator weights the ratio differently (Figure 3.9); the quadratic one of χ^2 lets large ratios dominate. An f-divergence can also be estimated from samples alone, through a variational form that a generative adversarial network (GAN) fits with a neural network, as in the f-GAN of Nowozin, Cseke & Tomioka (2016).

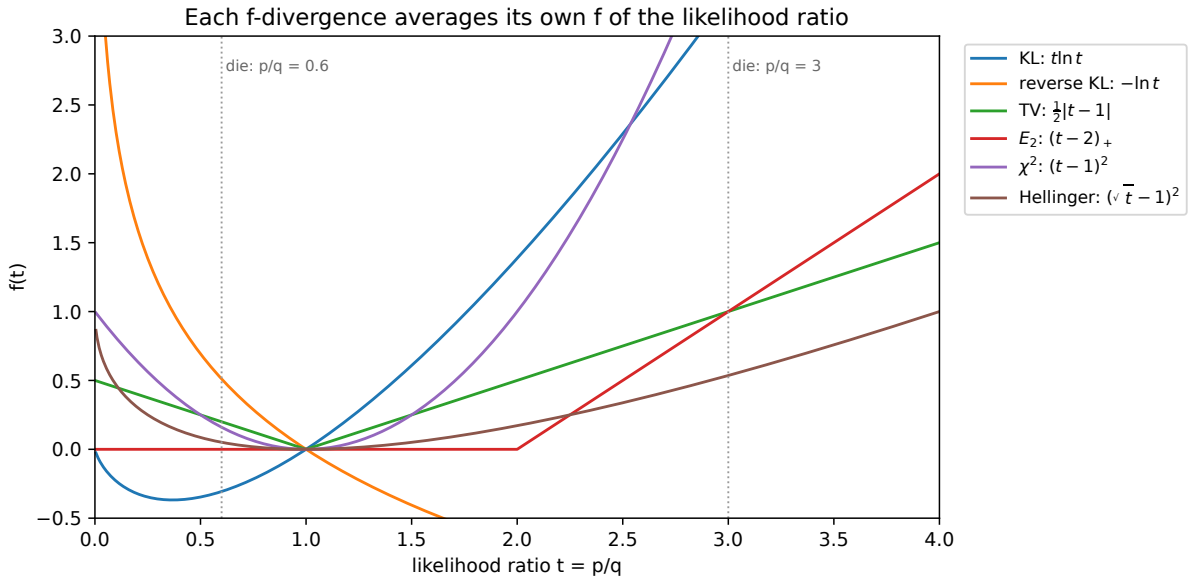


Figure 3.9.: The generators of six f-divergences.

3.8. The data-processing inequality

You rarely look at raw data. You bin it, compute features, or pass it through a randomized algorithm, and the data-processing inequality says that none of this can make two distributions easier to tell apart. When processing maps several inputs to one output, the output's likelihood ratio is an average of theirs. Every f-divergence is a convex summary of the ratios, and by Jensen averaging first can only make it smaller.

Definition 3.6 (Channel)**.** A **channel** K maps each input x to a distribution $K(\cdot \mid x)$ over outputs y. Feeding $X \sim P$ through it gives the output distribution $PK(y) = \sum_x p(x)K(y \mid x)$, and likewise QK.

Theorem 3.5 (Data-processing inequality)**.** *For every f-divergence, every Rényi divergence D_α including D_∞, and every channel K,*

$$D_f(PK\|QK) \le D_f(P\|Q), \qquad D_\alpha(PK\|QK) \le D_\alpha(P\|Q). \tag{3.41}$$

Equality holds when $r(x)$ can be computed from the output, which is then a ***sufficient statistic****.*

Proof. The plan is to show that the output's ratio is a conditional average of the input's, apply Jensen conditionally, and average over the output.

1. *The output's ratio.* Write $p(x) = q(x)r(x)$ inside $PK(y)$ and divide by $QK(y)$:

$$r_Y(y) = \frac{PK(y)}{QK(y)} = \sum_x \frac{q(x)K(y \mid x)}{QK(y)}\, r(x). \tag{3.42}$$

   By Bayes' rule the weights are the probabilities of X given $Y = y$ when $X \sim Q$ goes through K, so $r_Y(y) = \mathbb{E}[r(X) \mid Y = y]$.

2. *Jensen, conditionally.* Lemma 3.1 for the distribution of X given $Y = y$ gives $f(r_Y(y)) \le \mathbb{E}[f(r(X)) \mid Y = y]$.
3. *Average over the output.* Over $Y \sim QK$ the left side averages to $D_f(PK\|QK)$, and the conditional averages on the right average to $\mathbb{E}_Q[f(r)] = D_f(P\|Q)$.
4. *Rényi.* D_α is an increasing function of an f-divergence by Proposition 3.7, and $\alpha \to \infty$ gives D_∞.
5. *Equality.* If $r(x)$ is a function of y, then $r(X)$ is constant given $Y = y$, and step 2 is an equality.

□

The channel may use its own randomness, as long as it is the same whether the input came from P or Q. The coarsest processing reports only whether an event happened, and this gives a lower bound on KL from any single event.

Corollary 3.1 (KL bounds every event)**.** *For every event A, write $p_A = \Pr_P[A]$ and $q_A = \Pr_Q[A]$. Then*

$$D_{\mathrm{KL}}(P\|Q) \ge \mathrm{kl}(p_A\|q_A) = p_A \ln \frac{p_A}{q_A} + (1 - p_A) \ln \frac{1 - p_A}{1 - q_A}, \tag{3.43}$$

*where the **binary KL** kl is the KL divergence between coins with heads probabilities p_A and q_A.*

Proof. Reporting whether A happened is the channel $\mathbf{1}[x \in A]$, which turns P and Q into those two coins, and Theorem 3.5 with the generator of KL bounds their divergence by $D_{\mathrm{KL}}(P\|Q)$. □

So if KL is small, no event can have very different probabilities under P and Q, and Hypothesis testing uses this to bound the errors of any test.

Example 3.8. Report only whether the die roll is odd or even (Table 3.7).

Table 3.7.: Processing the die roll into its parity averages the likelihood ratio within each output.

y	Rolls	$PK(y)$	$QK(y)$	$r_Y = PK(y)/QK(y)$	Average of r over the rolls, Q-weighted
even	2, 4, 6	0.7	0.5	1.4	$(0.6 + 0.6 + 3)/3 = 1.4$
odd	1, 3, 5	0.3	0.5	0.6	$(0.6 + 0.6 + 0.6)/3 = 0.6$

The twos and fours dilute the six's evidence, and the KL drops from 0.2939 to $0.7 \ln 1.4 + 0.3 \ln 0.6 = 0.0823$. Reporting only whether the roll is a six loses nothing. The ratio depends only on that bit, so the bit is sufficient, and its KL is $\mathrm{kl}(0.5\|\frac{1}{6}) = 0.5 \ln 3 + 0.5 \ln 0.6 = 0.2939$.

Different divergences can lose different amounts. Round draws of $\mathcal{N}(0, 1)$ and $\mathcal{N}(1, 1)$ into bins of width w with an edge at $\frac{1}{2}$, which flattens the evidence into steps (Figure 3.10).

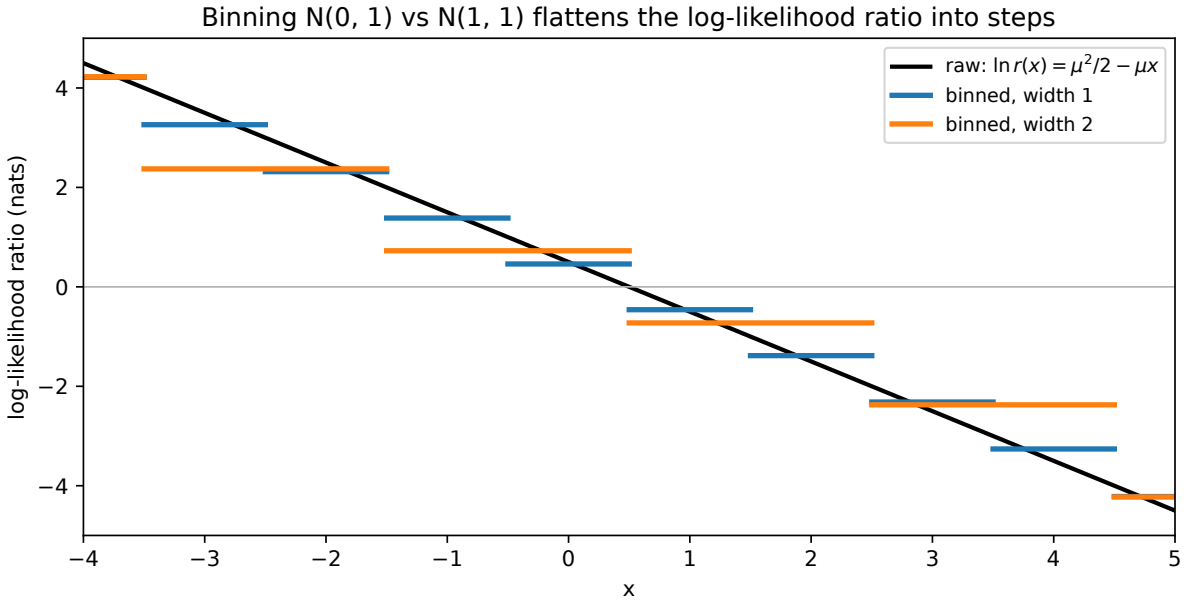


Figure 3.10.: The evidence of a Gaussian pair, raw and after binning.

Example 3.9. Table 3.8 gives the KL after binning.

Table 3.8.: KL between $\mathcal{N}(0,1)$ and $\mathcal{N}(1,1)$ after rounding x into bins.

Bin width w	0.5	1	2	one bit: $x < \frac{1}{2}$?	raw
KL after binning	0.4898	0.4618	0.3781	0.3090	0.5

The single bit at the midpoint keeps 62% of the KL (Figure 3.11), but it is the best event of Theorem 3.3, so it keeps all of the TV.

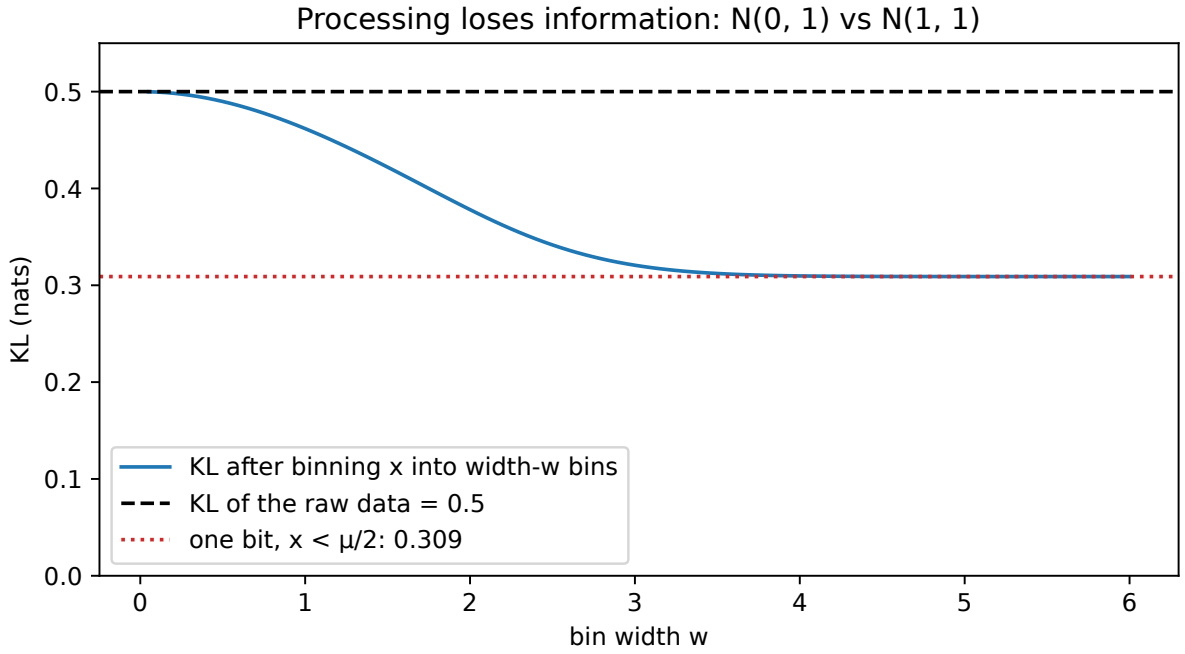


Figure 3.11.: KL after binning, against the bin width.

3.9. Pinsker's inequality

KL adds up over samples, while TV is what you want, the accuracy of the best test. Pinsker's

1. *Reduce to coins.* The event $A^* = \{p > q\}$ attains the TV by Theorem 3.3, so $\mathrm{TV} = p_A - q_A$ for it, and Corollary 3.1 gives $D_{\mathrm{KL}}(P\|Q) \geq \mathrm{kl}(p_A\|q_A)$. So it is enough that $\mathrm{kl}(p_A\|q_A) \geq 2(p_A - q_A)^2$.
2. *The gap.* Fix p_A and let $g(q_A) = \mathrm{kl}(p_A\|q_A) - 2(p_A - q_A)^2$. Two equal coins have zero KL, so $g(p_A) = 0$.
3. *Differentiate.* The derivative of $\mathrm{kl}(p_A\|q_A)$ in q_A is $\frac{q_A - p_A}{q_A(1-q_A)}$, and that of $-2(p_A - q_A)^2$ is $4(p_A - q_A)$, so
$$g'(q_A) = (q_A - p_A)\Big(\frac{1}{q_A(1-q_A)} - 4\Big). \tag{3.45}$$
4. *Sign.* Two numbers that add to 1 have a product of at most $\frac{1}{4}$, so the bracket is non-negative. Then g decreases until $q_A = p_A$, where it is 0, and increases after, so $g \geq 0$. With step 1, $D_{\mathrm{KL}} \geq 2\,\mathrm{TV}^2$, which rearranges to Equation 3.44.

□

Above $\mathrm{KL} = 2$ this bound exceeds 1 and says nothing, so a second inequality, weaker near zero, stays below 1.

Theorem 3.7 (Bretagnolle–Huber inequality)**.** *For any two distributions,*
$$\mathrm{TV}(P, Q) \leq \sqrt{1 - e^{-D_{\mathrm{KL}}(P\|Q)}}. \tag{3.46}$$

Proof. The plan is to bound the Bhattacharyya sum $\sum_x \sqrt{pq}$ from above by TV and from below by KL.

1. *Cauchy–Schwarz.* At each x, $pq = \min(p, q)\max(p, q)$, so the Cauchy–Schwarz inequality gives
$$\Big(\sum_x \sqrt{pq}\Big)^2 \leq \sum_x \min(p, q) \cdot \sum_x \max(p, q). \tag{3.47}$$
2. *Express in TV.* The minimum sums to $1 - \mathrm{TV}$ by Theorem 3.3, and the maximum to $1 + \mathrm{TV}$, because the two add to $p + q$. So the right side is $1 - \mathrm{TV}^2$.
3. *Express in KL.* Since $\sqrt{pq} = p\,e^{-L/2}$, the sum is $\mathbb{E}_P[e^{-L/2}]$, and $e^{-y/2}$ is convex, so Lemma 3.1 puts it above $e^{-D_{\mathrm{KL}}/2}$. Squaring and using step 2 gives $e^{-D_{\mathrm{KL}}} \leq 1 - \mathrm{TV}^2$.

□

Pinsker's inequality is tight for two nearly fair coins, passes 1 at $\mathrm{KL} = 2$, and loses to Bretagnolle–Huber once KL is above 1.59 (Figure 3.12). Neither can be reversed: if Q puts a tiny mass where P puts none, TV is tiny but $D_{\mathrm{KL}}(Q\|P)$ is infinite.

Example 3.10. How many samples tell $\mathcal{N}(0, 1)$ from $\mathcal{N}(0.1, 1)$, where one sample has TV 0.0399 and KL 0.005? By additivity n samples have KL $0.005\,n$, which Pinsker turns into a cap on TV and, by Equation 3.31, on accuracy. The exact TV is known too, because the evidence of n samples depends only on their mean, a Gaussian pair with shift $0.1\sqrt{n}$ (Table 3.9).

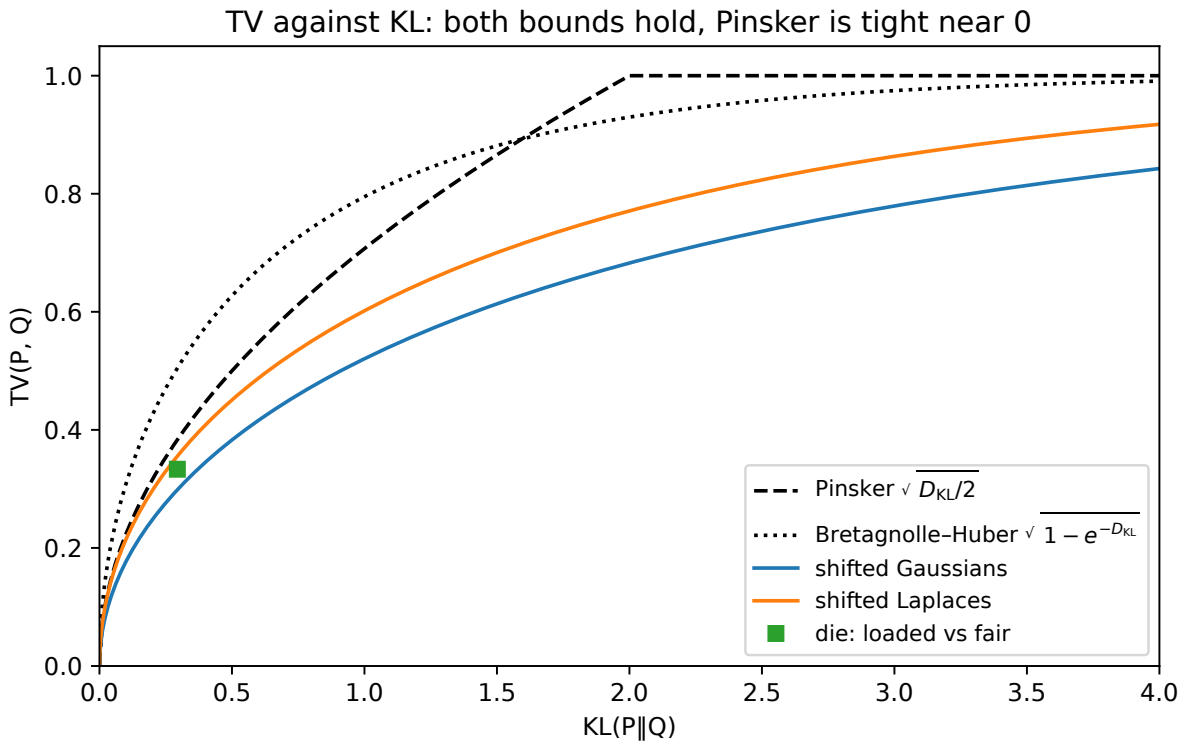


Figure 3.12.: Total variation against KL, with the two bounds.

Table 3.9.: n samples of $\mathcal{N}(0,1)$ against $\mathcal{N}(0.1,1)$: exact TV, best accuracy (simulated over 20,000 trials), and the two bounds on TV.

n	KL	Exact TV	Best accuracy (simulated)	Pinsker	Bretagnolle–Huber
100	0.50	0.3829	0.6915 (0.6916)	0.5000	0.6273
400	2.00	0.6827	0.8413 (0.8424)	1.0000	0.9299

Any test needs n of order $1/D_{\mathrm{KL}}$. At $n = 100$ Pinsker's bound overstates TV by 31%, and at $n = 400$ it is vacuous while Bretagnolle–Huber is not.

3.10. Code

The script code/divergences.py computes every number in this chapter and draws every figure. From the repository root:

```
uv run python code/part01-prerequisites/divergences.py
```

Table 3.10.: The script's output with the default seed.

Experiment	Result
Die	Table 3.4, Table 3.6; KL from the E_γ curves 0.2939; after parity 0.0823
Gaussians, $\mu = 1$	KL 0.5000 (Monte Carlo 0.4990); TV 0.3829; E_2 0.1906 (Monte Carlo 0.1892); $\Pr_Q[L \geq 2]$ 0.0062
Laplace, $t = 1$	KL 0.3679 (Monte Carlo 0.3684); Table 3.5
Two-bump fit	Forward $\sigma = 2.06$, reverse $\sigma = 0.50$

tests/test_divergences.py checks that the output still matches.

3.11. Summary

Table 3.11.: The results of this chapter, and where each is used.

Result	Statement	Use
Theorem 3.1	$D_{\mathrm{KL}} = \mathbb{E}_P[L] \geq 0$, additive	About $1/D_{\mathrm{KL}}$ samples tell two distributions apart
Lemma 3.1, Lemma 3.2	Jensen; $\mathbb{E}_P[g] = \mathbb{E}_Q[r\,g]$	Tools for every proof here
Proposition 3.1, Lemma 3.3	$D_\alpha = \ln m_{\alpha-1} = K_L(\alpha-1)/(\alpha-1)$	The order is a dial from typical to worst case
Theorem 3.2, Proposition 3.2	Non-decreasing, additive, from KL to $\ln \max r$; bounds the tail of L	Compute in Rényi, where samples add
Proposition 3.3, Proposition 3.4	$D_\alpha = \alpha\mu^2/2$; $D_\infty = t$	Unbounded against bounded ratio
Lemma 3.4	The best event is $\{r > \gamma\}$	TV, E_γ, Pinsker
Theorem 3.3	Accuracy $\frac{1}{2} + \frac{1}{2}\mathrm{TV}$	Not additive: convert from KL at the end
Proposition 3.5, Proposition 3.6	$E_\gamma = \mathbb{E}_Q[(r-\gamma)_+]$	Unequal priors
Proposition 3.7, Theorem 3.4	$D_f = \mathbb{E}_Q[f(r)] \geq 0$, a weighted sum of E_γ	One proof for every divergence here
Theorem 3.5, Corollary 3.1	$D_f(PK\|QK) \leq D_f(P\|Q)$	Processing only loses; sufficient statistics lose nothing
Theorem 3.6, Theorem 3.7	$\mathrm{TV} \leq \sqrt{D_{\mathrm{KL}}/2}$, $\mathrm{TV} \leq \sqrt{1 - e^{-D_{\mathrm{KL}}}}$	Pinsker near zero, Bretagnolle–Huber above KL 1.59

3.12. References

- van Erven & Harremoës, "Rényi Divergence and Kullback–Leibler Divergence" (IEEE Transactions on Information Theory, 2014), §§1–3: Rényi divergence for general measures.
- Cover & Thomas, *Elements of Information Theory* (2nd ed., 2006), ch. 2: KL and Jensen's inequality; §11.6 for Pinsker's inequality.
- Polyanskiy & Wu, *Information Theory: From Coding to Learning* (2025), ch. 7: f-divergences and the data-processing inequality, at reference depth.
- Tsybakov, *Introduction to Nonparametric Estimation* (2009), §2.4: Pinsker, Bretagnolle–Huber and other inequalities between divergences.
- Bishop, *Pattern Recognition and Machine Learning* (2006), §10.1: forward against reverse KL in variational inference.
- Liese & Vajda, "On Divergences and Informations in Statistics and Information Theory" (IEEE Transactions on Information Theory, 2006): f-divergences and their representation through the curves E_γ.
- Nowozin, Cseke & Tomioka, "f-GAN: Training Generative Neural Samplers using Variational Divergence Minimization" (NeurIPS 2016): f-divergences estimated from samples with neural networks.

4. Hypothesis testing

A spam filter must say "ham" or "spam", a medical test "healthy" or "ill", a monitoring job "calibrated" or "drifted". Each faces the same problem: an observation came from one of two distributions, and you must decide which. This chapter asks what the best decision rule is, how good it can be, how to summarize real rules, and how to measure their error rates from finite data. Later parts need exactly this: how well *any* test can tell two distributions apart.

One idea organizes every answer. Write $p(x)$ for the probability (or density) of an observation x under the first distribution and $q(x)$ for its probability under the second. Their ratio $\Lambda(x) = q(x)/p(x)$, the **likelihood ratio**, says how many times more likely x is under the second, and Section 4.2 proves that it is the only thing about x that matters. The best rule compares this ratio with a cut-off. Plotting the two error rates of the best rule at every cut-off gives a curve that depends on the two distributions alone, and the chapter reads every summary of a rule off that curve.

Two running examples carry the chapter. In the first, a spam filter counts suspicious words in an email, capped at 3, and Table 4.1 gives the count's distribution P for ham and Q for spam. A count of 3 is four times as likely in spam, and a count of 0 a quarter as likely. Spam's probability goes in the numerator of the ratio, the reverse of the convention in Divergences. The second example is a biomarker that is normal with standard deviation 1, with mean 0 in healthy patients and mean 2 in ill ones.

Table 4.1.: Word-count distributions under ham (P) and spam (Q), and their likelihood ratio.

x	0	1	2	3
$p(x)$, ham	0.4	0.3	0.2	0.1
$q(x)$, spam	0.1	0.2	0.3	0.4
$\Lambda(x) = q(x)/p(x)$	0.250	0.667	1.500	4.000

4.1. Tests and their two errors

Before asking which decision rule is best, you need to say what a rule is and how to score it. This chapter compares two candidate distributions that are each fully specified, called **simple** hypotheses, unlike a composite one such as "the mean is positive". The **null hypothesis** H_0, the default you keep unless the evidence is strong, is that the observation comes from P, and the **alternative** H_1 is that it comes from Q. A rule may also reject H_0 (say "spam") on some outcomes only with some probability, by flipping a biased coin, and Example 4.2 shows why that matters.

Definition 4.1 (Test, errors and power)**.** Let P and Q be distributions on a set $\mathcal{X}$ of outcomes, and test $H_0 : X \sim P$ against $H_1 : X \sim Q$. A **test** is a function $\phi : \mathcal{X} \to [0, 1]$, where $\phi(x)$ is the probability of rejecting H_0 after observing x. It is **deterministic** if it takes only the values 0 and 1, and so is the indicator of its **rejection region**. Its **type I error**, or **size**, and its **type II error** are

$$\alpha_\phi = \mathbb{E}_P[\phi(X)], \qquad \beta_\phi = 1 - \mathbb{E}_Q[\phi(X)], \tag{4.1}$$

also called its false positive rate (FPR) and false negative rate (FNR). Its **power**, or true positive rate (TPR), is $1 - \beta_\phi$. A test has **level** α if $\alpha_\phi \leq \alpha$.

For a deterministic test the two averages are the masses that P and Q give to the rejection region, because the average of an indicator is the probability of its event (Concentration inequalities). Since H_0 is the default, a false alarm, rejecting H_0 when it holds, is the error you control first, so you fix the level and then look for the most powerful test within it.

Table 4.2.: Errors of deterministic tests in the spam example. The first five are thresholds, which reject when the count is at least some value; the last two are not.

Rejection region	α	β	Power
none	0.00	1.00	0.00
$\{3\}$	0.10	0.60	0.40
$\{2, 3\}$	0.30	0.30	0.70
$\{1, 2, 3\}$	0.60	0.10	0.90
all	1.00	0.00	1.00
$\{0\}$	0.40	0.90	0.10
$\{0, 3\}$	0.50	0.50	0.50

Down the first five rows of Table 4.2 the region grows, and each added outcome adds its ham mass to α and its spam mass to the power. So along the thresholds you can only trade one error for the other. The yardstick is the coin-flip test $\phi \equiv \alpha$, which ignores the email and rejects with a fixed probability α, so its power equals its size. The region $\{0\}$ does worse than that coin, with power 0.10 at size 0.40, because it rejects on the outcome most typical of ham.

Example 4.1. A screening test flags a patient when the biomarker exceeds $\tau = 1.645$, chosen so that $\alpha = 0.05$ (Figure 4.1). Write Φ for the standard normal cumulative distribution function (CDF), so that $\Phi^{-1}(0.95) = 1.6449$. A healthy patient's biomarker is standard normal, so it exceeds τ with probability $1 - \Phi(1.645) = 0.05$. An ill patient's biomarker is centred at 2, so it falls below τ with probability $\beta = \Phi(\tau - 2) = \Phi(-0.3551) = 0.3612$. So the test catches about two ill patients in three, $1 - \beta = 0.6388$, while sending one healthy patient in 20 for a follow-up. A lower threshold catches more ill patients and alarms more healthy ones.

4.2. The Neyman–Pearson lemma

Since the two errors trade against each other, the question is which test gives the most power for a given false positive budget. Rejecting an outcome x costs $p(x)$ of budget and buys $q(x)$ of

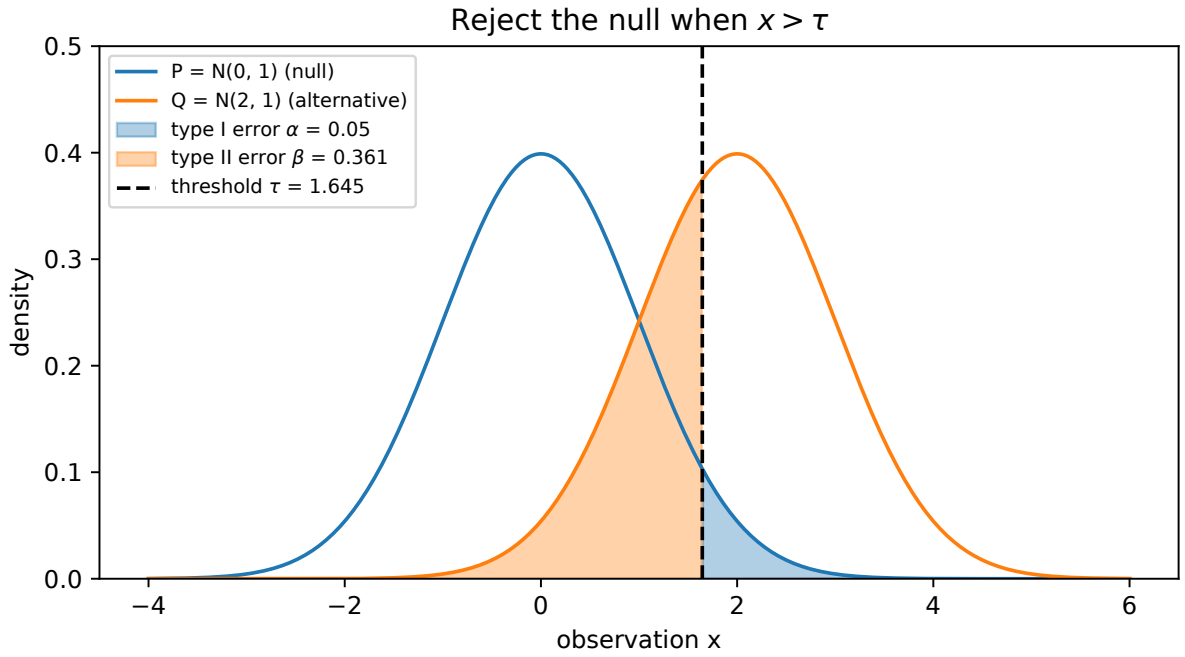


Figure 4.1.: The two errors of the threshold test "reject when $x > \tau$" for $P = \mathcal{N}(0,1)$ against $Q = \mathcal{N}(2,1)$, with τ set so that $\alpha = 0.05$. The blue area under P beyond τ is α; the orange area under Q before τ is $\beta = 0.361$.

power, an exchange rate of $\Lambda(x)$. The best buyer spends on the best rates first, which means rejecting exactly where Λ is above some cut-off. The test below writes that as $q(x) > c\,p(x)$, so that it always rejects outcomes that P never produces, $p(x) = 0$, which cost no budget.

Theorem 4.1 (Neyman–Pearson lemma)**.** *Let P and Q have densities, or mass functions, p and q. For a cut-off $c \geq 0$, a* ***likelihood ratio (LR) test*** *rejects where q exceeds c times p, keeps H_0 where it falls short, and on the tie set, where $q(x) = c\,p(x)$, rejects with some probability $\gamma(x) \in [0,1]$:*

$$\phi^*(x) = \begin{cases} 1 & \text{if } q(x) > c\,p(x), \\ \gamma(x) & \text{if } q(x) = c\,p(x), \\ 0 & \text{if } q(x) < c\,p(x). \end{cases} \tag{4.2}$$

1. Optimality. *Every test ϕ with $\alpha_\phi \leq \alpha_{\phi^*}$ has $\beta_\phi \geq \beta_{\phi^*}$.*
2. Existence. *For every $\alpha \in (0,1]$ there is an LR test with a constant γ and size exactly α.*

So the most powerful test of level α is the LR test of size α.

Proof. The plan is to prove optimality by comparing ϕ^* with any other test outcome by outcome, and existence by choosing the lowest cut-off whose outcomes strictly above it still fit in the budget, then randomizing on the ties.

1. *A pointwise inequality.* Weight each outcome by $q(x) - c\,p(x)$, which is positive exactly where ϕ^* rejects. Where it is positive $\phi^* = 1 \geq \phi$, where it is negative $\phi^* = 0 \leq \phi$, and on

the tie set it is 0. So for every x, using only that ϕ takes values in $[0, 1]$,

$$(\phi^*(x) - \phi(x))(q(x) - c\,p(x)) \geq 0. \tag{4.3}$$

2. *Sum over the outcomes.* A sum of non-negative terms is non-negative, and summing ϕq and ϕp gives $\mathbb{E}_Q[\phi]$ and $\mathbb{E}_P[\phi]$, so

$$\mathbb{E}_Q[\phi^*] - \mathbb{E}_Q[\phi] \geq c(\mathbb{E}_P[\phi^*] - \mathbb{E}_P[\phi]). \tag{4.4}$$

3. *Use the budget.* The other test spends no more and $c \geq 0$, so the right side is non-negative. Then ϕ^* has at least the power of ϕ, which is optimality.
4. *The size of a strict cut-off.* For existence, write $G(t) = \Pr_P[\Lambda(X) > t]$ for the size of the test that rejects strictly above a cut-off t. Raising t removes outcomes, so G is non-increasing. As t' falls to t the sets $\{\Lambda > t'\}$ grow to $\{\Lambda > t\}$, and the probability of growing sets tends to that of their union, so G is right-continuous.
5. *Choose the cut-off.* Let $c = \inf\{t \geq 0 : G(t) \leq \alpha\}$, the smallest cut-off whose strict region fits in the budget. Right-continuity gives $G(c) \leq \alpha$. If $c > 0$, then $G(t) > \alpha$ for every $t < c$, and as t rises to c the sets $\{\Lambda > t\}$ shrink to $\{\Lambda \geq c\}$, so $\Pr_P[\Lambda \geq c] \geq \alpha$; if $c = 0$ this probability is 1, because $\Lambda \geq 0$ everywhere.
6. *Randomize on the ties.* The LR test with constant γ has size $G(c) + \gamma \Pr_P[\Lambda = c]$, which runs from $G(c) \leq \alpha$ at $\gamma = 0$ to $\Pr_P[\Lambda \geq c] \geq \alpha$ at $\gamma = 1$. It is linear in γ, so some $\gamma \in [0, 1]$ makes it exactly α.

□

The optimal test looks at x only through $\Lambda(x)$. When Λ increases with a simple statistic, such as the word count or the biomarker, the optimal test is a plain threshold on the statistic. A calibrated classifier score, an estimate of the probability of H_1 given x, is such a statistic, by Bayes' rule (Section 4.8), which is why thresholding it is right.

How fast does the error fall with more data? For n independent observations the ratio of the sample is the product of the individual ratios, and at a fixed false positive rate the best type II error decays like $e^{-n\,D_{\mathrm{KL}}(P\|Q)}$, with the Kullback–Leibler (KL) divergence of Divergences as the rate. Section F.2 derives this Chernoff–Stein lemma.

Example 4.2. Take the spam example at budget $\alpha = 0.25$, a size no threshold has. A cut-off between 1.5 and 4 rejects only the count 3, with size 0.1, which fits. Just below 1.5 the count 2 joins and the size is 0.3, which does not, so $c = 1.5$. The test spends the remaining 0.15 on the tie set $\{2\}$, whose ham mass is 0.2, so it rejects the count 2 with probability $\gamma = 0.75$. Its power is $0.4 + 0.75 \times 0.3 = 0.625$. Randomizing matters only for discrete data like these, since for a continuous score a threshold alone can hit any size.

A brute-force check agrees. Of 200,000 random tests, each $\phi(x)$ uniform on $[0, 1]$, 11,727 have size at most 0.25, and the best of them has power only 0.587 (Figure 4.2).

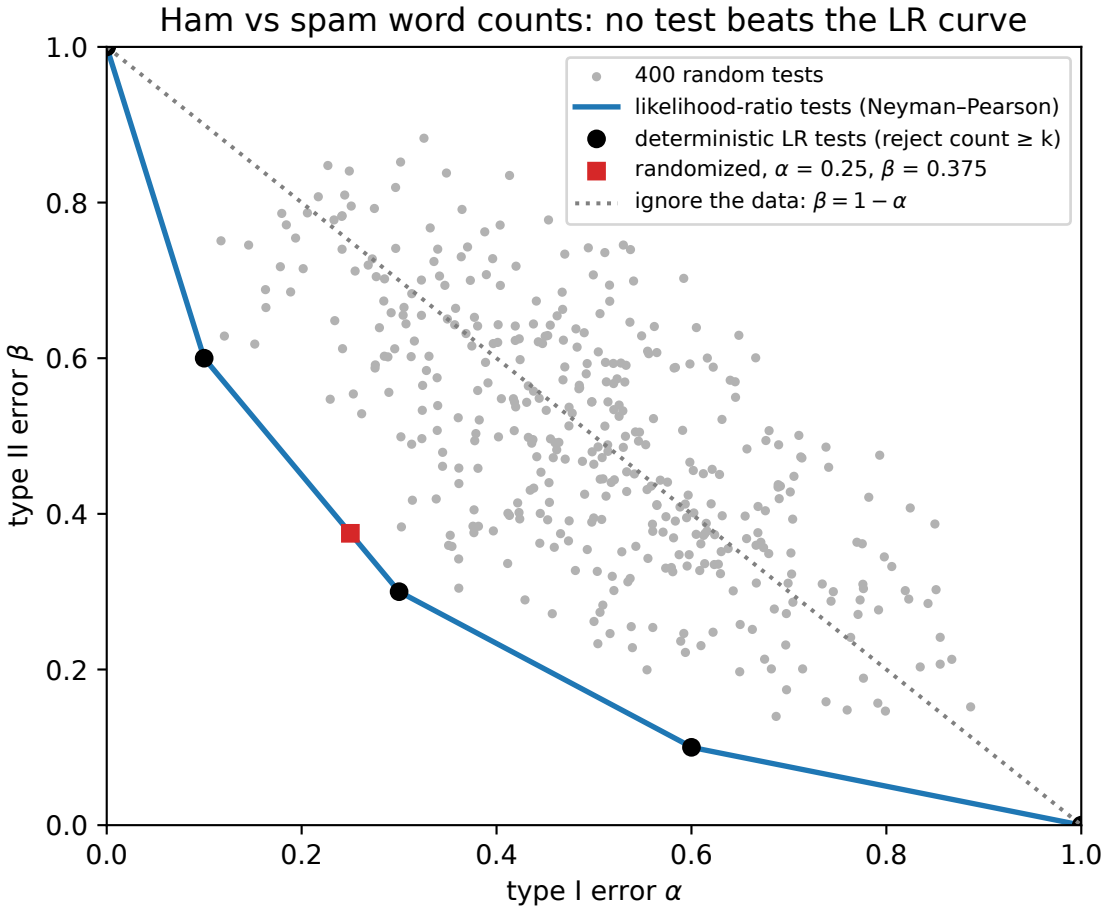


Figure 4.2.: Errors (α, β) of 400 of the random tests on the spam example (grey), against the curve of likelihood-ratio tests. Black dots are the deterministic thresholds, the red square is the randomized test of size 0.25, and the dotted line is the coin-flip test.

4.3. Trade-off functions and ROC curves

A single test gives one point (α, β). To say how hard it is to tell P from Q you want the best β at every α at once. That curve depends only on the pair, and later parts use it to describe how distinguishable two distributions are.

Definition 4.2 (Trade-off function)**.** The **trade-off function** of P and Q gives, at each false positive budget α, the smallest type II error of any test within it:

$$T(P, Q)(\alpha) = \inf\{\beta_\phi : \alpha_\phi \le \alpha\}, \qquad \alpha \in [0, 1]. \tag{4.5}$$

Write T for $T(P, Q)$ when the pair is clear. The **receiver operating characteristic (ROC) curve** of the optimal tests plots its power against its size, $\alpha \mapsto 1 - T(\alpha)$.

By Theorem 4.1 the infimum is attained by the LR test of size α, so sweeping the LR cut-off traces T. The proof that every trade-off function is convex and continuous needs two tools. Convexity comes from mixing two tests, and the first tool says what mixing does to their errors.

Lemma 4.1 (Mixing tests mixes their errors)**.** *Let ϕ_1 and ϕ_2 be tests and $\lambda \in [0, 1]$. The test $\phi = \lambda\phi_1 + (1 - \lambda)\phi_2$, which runs ϕ_1 with probability λ and ϕ_2 otherwise, has errors $\alpha_\phi = \lambda\alpha_{\phi_1} + (1 - \lambda)\alpha_{\phi_2}$ and $\beta_\phi = \lambda\beta_{\phi_1} + (1 - \lambda)\beta_{\phi_2}$.*

Proof.

1. On an outcome x the mixed rule rejects with probability $\lambda\phi_1(x) + (1 - \lambda)\phi_2(x)$, which lies in $[0, 1]$, so it is a test.
2. Both errors are averages of ϕ by Definition 4.1, and the average of a weighted sum is the weighted sum of the averages.

□

Continuity at $\alpha = 0$ needs a second tool, because a convex function can jump at an end of its interval. It comes from bounding a test's power, an average under Q, by averages under P weighted by Λ. Divergences proved this **change of measure** when both distributions give positive probability to the same outcomes. Here Q may also put mass where P has none, which needs a term of its own.

Lemma 4.2 (Change of measure)**.** *For every function $g \ge 0$ on the outcomes,*

$$\mathbb{E}_Q[g(X)] = \mathbb{E}_P[g(X)\Lambda(X)] + \mathbb{E}_Q\big[g(X)\mathbf{1}[p(X) = 0]\big]. \tag{4.6}$$

In particular, the likelihood ratio has average $\mathbb{E}_P[\Lambda(X)] = 1 - \Pr_Q[p(X) = 0]$, which is at most 1.

Proof.

1. Split the sum (or integral) $\sum_x g(x)q(x)$ into the outcomes with $p(x) > 0$ and those with $p(x) = 0$. The second part is the second term.
2. Where $p(x) > 0$ the ratio's definition gives $q(x) = \Lambda(x)p(x)$, so the first part is $\mathbb{E}_P[g\Lambda]$.

3. With $g = 1$ the left side is 1 and the second term is $\Pr_Q[p = 0]$.

□

Theorem 4.2 (Properties of trade-off functions)**.** *Every trade-off function T is non-increasing, satisfies $T(\alpha) \le 1-\alpha$, is convex, and is continuous on $[0, 1]$. Its value at 0 is $T(0) = 1-\Pr_Q[p(X) = 0]$.*

Proof. The first two properties come from the definition, convexity from mixing, and continuity from convexity inside the interval and from separate arguments at the ends.

1. *Non-increasing.* A larger budget allows every test a smaller one does, so the infimum is no larger.
2. *At most $1 - \alpha$.* The coin flip $\phi \equiv \alpha$ is within budget and misses with probability $1 - \alpha$.
3. *Convex.* Fix budgets α_1, α_2 and a weight λ. By Theorem 4.1 (and step 5 for a zero budget) some tests ϕ_i have $\alpha_{\phi_i} \le \alpha_i$ and $\beta_{\phi_i} = T(\alpha_i)$. By Lemma 4.1 their mixture has size at most $\lambda\alpha_1 + (1-\lambda)\alpha_2$ and type II error $\lambda T(\alpha_1) + (1-\lambda)T(\alpha_2)$, so the curve at the mixed budget is at most the mixed value.
4. *Continuous inside and at 1.* A convex function is continuous at every interior point of its interval, because its chords pin it between two lines (Rockafellar, *Convex Analysis*, §10). At 1, the test $\phi \equiv 1$ gives $T(1) = 0$, and step 2 squeezes $T(\alpha)$ between 0 and $1 - \alpha$.
5. *The value at 0.* A test of size 0 has $\mathbb{E}_P[\phi] = 0$, so apart from a set of P-probability 0 it rejects only where $p = 0$. So the best one rejects exactly where $p = 0$, and it misses the rest of Q's mass, $1 - \Pr_Q[p = 0]$.
6. *A bound on the power.* To rule out a jump at 0, take any test with $\alpha_\phi \le \alpha$ and a cut-off $K > 0$. By Lemma 4.2 the power is $\mathbb{E}_P[\phi\Lambda]$ plus at most $\Pr_Q[p = 0]$. Where $\Lambda \le K$ the term $\phi\Lambda$ averages to at most $K\alpha$, and where $\Lambda > K$ it is at most Λ because $\phi \le 1$, so

$$1 - \beta_\phi \le \Pr_Q[p = 0] + K\alpha + \mathbb{E}_P[\Lambda\,\mathbf{1}[\Lambda > K]]. \tag{4.7}$$

7. *Let the budget shrink.* Since β_ϕ is 1 minus the power and $T(0) = 1 - \Pr_Q[p = 0]$ by step 5, the bound says $\beta_\phi \ge T(0) - K\alpha - \mathbb{E}_P[\Lambda\,\mathbf{1}[\Lambda > K]]$ for every test within budget, and so for their infimum $T(\alpha)$. The average of Λ is finite by Lemma 4.2, so its part from $\Lambda > K$ vanishes as K grows. Choosing K large, then α small, and using $T(\alpha) \le T(0)$ from step 1, $T(\alpha)$ tends to $T(0)$.

□

Conversely, every function with these four properties is the trade-off function of some pair (Section K.1 derives it). A pair is **harder to distinguish** than another if its curve is higher everywhere. The hardest curve is $1-\alpha$, for $P = Q$, and the easiest is $T \equiv 0$, for distributions that never produce the same outcome. The curve starts below 1 when Q has mass where P has none, and swapping P and Q reflects it across the line $\beta = \alpha$, because the test $1 - \phi$ for the swapped pair has the errors of ϕ in the other order.

The curves of the next sections differ in how fast they fall, and one rule explains their shapes. The slope of the curve is minus the exchange rate of Section 4.2, the likelihood ratio of the outcomes the test is currently adding.

Proposition 4.1 (The slope of the curve is the likelihood ratio)**.** *If P and Q are discrete, T is piecewise linear, and the segment on which the LR test adds the outcome x has slope $-\Lambda(x)$. If they have continuous positive densities and Λ increases with the observation, the slope at the point whose LR test is $\mathbf{1}[x > \tau]$ is $T'(\alpha) = -\Lambda(\tau)$.*

Proof.

1. *Discrete case.* Between two consecutive deterministic LR tests, the LR test rejects the next outcome x with a probability γ that rises from 0 to 1. That adds $\gamma p(x)$ to α and removes $\gamma q(x)$ from β, a straight segment of slope $-q(x)/p(x)$.
2. *Continuous case.* Moving the threshold from τ to $\tau + \mathrm{d}\tau$ lowers $\alpha = \Pr_P[X > \tau]$ by $p(\tau)\,\mathrm{d}\tau$ and raises $\beta = \Pr_Q[X \le \tau]$ by $q(\tau)\,\mathrm{d}\tau$, so $\mathrm{d}\beta/\mathrm{d}\alpha = -q(\tau)/p(\tau)$.

□

Example 4.3. In the spam example T is the broken line through the error pairs of the thresholds in Table 4.2 (Figure 4.2). Its first segment adds the count 3: β falls by 0.4 while α rises by 0.1, a slope of -4, the ratio of that count. The next segments have slopes -1.5, -0.667 and -0.25, as Proposition 4.1 predicts.

4.4. The Gaussian trade-off curve

The most common pair in practice is a shift of a normal: a score, a biomarker or a sample mean that is normal under both hypotheses, with the same spread and different centres. Its curve has a closed form, which later parts use as the reference curve for normal noise.

Theorem 4.3 (Gaussian trade-off curve)**.** *Let $P = \mathcal{N}(0,1)$ and $Q = \mathcal{N}(\mu,1)$ with $\mu \ge 0$. Then*

$$T(P,Q)(\alpha) = \Phi(\Phi^{-1}(1-\alpha) - \mu). \tag{4.8}$$

More generally, $\mathcal{N}(m_0,\sigma^2)$ against $\mathcal{N}(m_1,\sigma^2)$ has this curve with $\mu = |m_1 - m_0|/\sigma$.

Proof. The plan is to compute the likelihood ratio, see that the LR tests are thresholds, read both errors off normal tails, and then reduce the general pair to this one.

1. *The likelihood ratio.* With the standard normal density $\varphi(x) = e^{-x^2/2}/\sqrt{2\pi}$, the density of Q is $\varphi(x-\mu)$. The constants cancel, and $x^2 - (x-\mu)^2 = 2\mu x - \mu^2$, so

$$\Lambda(x) = \exp\Big(\mu x - \frac{\mu^2}{2}\Big). \tag{4.9}$$

2. *The LR tests are thresholds.* For $\mu > 0$ the ratio is strictly increasing in x, so it exceeds a cut-off exactly when x exceeds some τ. A tie happens only at that one point, which has probability 0, so by Theorem 4.1 the tests $\mathbf{1}[x > \tau]$ are optimal. At $\mu = 0$ the curve is the diagonal, as the formula says.
3. *The size fixes the threshold.* The size is $\Pr_P[X > \tau] = 1 - \Phi(\tau)$, and setting it to α gives $\tau = \Phi^{-1}(1-\alpha)$.

4. *The type II error.* Under Q the variable $X - \mu$ is standard normal, so the test misses with probability $\Pr_Q[X \le \tau] = \Phi(\tau - \mu)$. Substituting step 3 gives Equation 4.8.
5. *The general pair.* Standardizing by $x \mapsto (x - m_0)/\sigma$ relabels the outcomes one to one, so every test keeps its errors, and the shift becomes $(m_1 - m_0)/\sigma$. A negative shift reverses the threshold and gives the same curve.

□

The curve depends on the pair only through μ, the distance between the means in standard deviations (Figure 4.3). Averaging repeated readings increases that distance, and the next result says by how much.

Corollary 4.1 (Averaging n observations)**.** *For n independent draws from $\mathcal{N}(0, \sigma^2)$ against n independent draws from $\mathcal{N}(h, \sigma^2)$, the trade-off function is Equation 4.8 with*

$$\mu = \frac{h\sqrt{n}}{\sigma}. \tag{4.10}$$

Proof.

1. Step 1 of the proof of Theorem 4.3, with the spread put back, gives each draw the log-ratio $(hx_i - h^2/2)/\sigma^2$. Log-ratios of independent draws add, so the sample's log-ratio depends on the data only through the sample mean $\bar{x}$, and the LR tests are thresholds on $\bar{x}$.
2. The sample mean is $\mathcal{N}(0, \sigma^2/n)$ against $\mathcal{N}(h, \sigma^2/n)$, a shift of h in units of $\sigma/\sqrt{n}$, and Theorem 4.3 applies.

□

So distinguishability grows like $\sqrt{n}$. Near $\alpha = 0$ the curve is vertical, because by Proposition 4.1 its slope is $-\Lambda(\tau)$, which grows without bound as the threshold moves out. Swapping P and Q gives a shift pair at the same distance μ, so the curve is also its own reflection across the line $\beta = \alpha$.

Example 4.4. A monitoring job averages $n = 100$ readings from a sensor with noise σ and must detect a drift of $h = 0.2\sigma$ in the mean. By Corollary 4.1 the shift is $\mu = 0.2 \times 10 = 2$, the biomarker's, and Table 4.3 evaluates the curve there and at $\mu = 1$. At $\alpha = 0.05$ the threshold is $\Phi^{-1}(0.95) = 1.6449$, and $\beta = \Phi(-0.3551) = 0.3612$, so the job catches the drift with probability 0.6388. A simulation with 1,000,000 draws per hypothesis gives $\hat{\alpha} = 0.0503$ and $\hat{\beta} = 0.3617$.

Table 4.3.: The Gaussian trade-off curve at two shifts and two levels.

μ	α	Threshold $\Phi^{-1}(1-\alpha)$	$\beta = T(\alpha)$	Power
1	0.05	1.6449	0.7405	0.2595
1	10^{-3}	3.0902	0.9817	0.0183
2	0.05	1.6449	0.3612	0.6388
2	10^{-3}	3.0902	0.8622	0.1378

Turned around, the curve says how many readings a target needs. Power 0.9 at $\alpha = 0.01$ needs

$$\mu \geq \Phi^{-1}(0.99) - \Phi^{-1}(0.1) = 2.3263 + 1.2816, \tag{4.11}$$

and for a drift of $h = 0.25\sigma$, $\mu = 0.25\sqrt{n}$ reaches this at 209 readings.

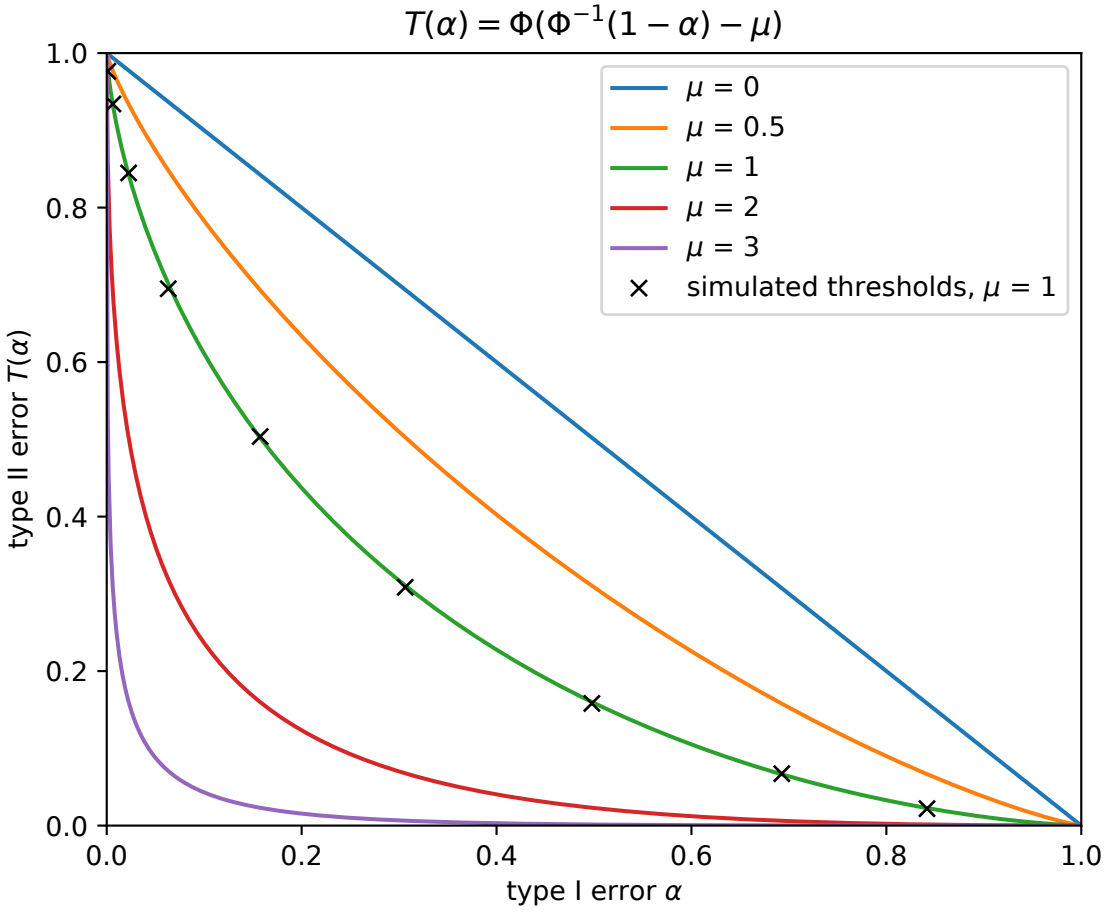


Figure 4.3.: The Gaussian trade-off curve $\Phi(\Phi^{-1}(1-\alpha) - \mu)$ for $\mu = 0, 0.5, 1, 2, 3$. Crosses are simulated errors of threshold tests at $\mu = 1$.

4.5. The Laplace trade-off curve

Does the shape of the noise change the curve? Replacing the normal by a Laplace distribution shows that it does, at small and at large α. The Laplace distribution $\mathrm{Lap}(m, b)$ with centre m and scale b has density $\frac{1}{2b}e^{-|x-m|/b}$. Moving the centre by m_1 changes the distance $|x - m|$ by at most m_1, so the likelihood ratio stays between $e^{-m_1/b}$ and $e^{m_1/b}$ (Probability distributions). It equals these bounds on whole tails, so by the slope rule of Proposition 4.1 the two ends of the curve are straight lines.

Theorem 4.4 (Laplace trade-off curve)**.** *Let $P = \mathrm{Lap}(0, b)$ and $Q = \mathrm{Lap}(m_1, b)$ with $m_1 > 0$, and write $\kappa = m_1/b$ for the shift in units of the scale. Then*

$$T(P,Q)(\alpha) = \begin{cases} 1 - e^{\kappa}\alpha, & \alpha < \frac{1}{2}e^{-\kappa}, \\ \dfrac{e^{-\kappa}}{4\alpha}, & \frac{1}{2}e^{-\kappa} \le \alpha \le \frac{1}{2}, \\ e^{-\kappa}(1-\alpha), & \alpha > \frac{1}{2}. \end{cases} \tag{4.12}$$

Proof. The plan is the one used for the normal, with three cases for where the threshold falls: above the alternative's centre, between the centres, or below the null's centre.

1. *Rescale.* Dividing x by b changes no test's errors, so take $b = 1$ and $m_1 = \kappa$. Integrating $\frac{1}{2}e^{-|x|}$ gives the CDF $F(x) = \frac{1}{2}e^{x}$ for $x < 0$ and $1 - \frac{1}{2}e^{-x}$ for $x \ge 0$.
2. *The likelihood ratio.* The constants cancel, so $\ln \Lambda(x) = |x| - |x - \kappa|$, which, once the absolute values are removed, is
$$\ln \Lambda(x) = \begin{cases} -\kappa, & x \le 0, \\ 2x - \kappa, & 0 < x < \kappa, \\ \kappa, & x \ge \kappa. \end{cases} \tag{4.13}$$
It is non-decreasing and clipped to $[-\kappa, \kappa]$ (Figure 4.4, left).
3. *Thresholds are LR tests.* Because Λ is non-decreasing, the set $\{x > \tau\}$ contains every point whose ratio exceeds $\Lambda(\tau)$, plus part of the tie set where the ratio equals $\Lambda(\tau)$. Rejecting part of a tie set is an allowed $\gamma(x)$ in Equation 4.2, so every threshold is optimal, and its errors $\alpha = 1 - F(\tau)$ and $\beta = F(\tau - \kappa)$ trace T.
4. *Threshold at or above κ.* Both τ and $\tau - \kappa$ are non-negative, so $\alpha = \frac{1}{2}e^{-\tau} \le \frac{1}{2}e^{-\kappa}$ and $\beta = 1 - \frac{1}{2}e^{-(\tau-\kappa)} = 1 - e^{\kappa}\alpha$.
5. *Threshold between 0 and κ.* Now $\alpha = \frac{1}{2}e^{-\tau}$ lies in $(\frac{1}{2}e^{-\kappa}, \frac{1}{2}]$ and $\beta = \frac{1}{2}e^{\tau-\kappa}$. Substituting $e^{\tau} = 1/(2\alpha)$ gives $\beta = e^{-\kappa}/(4\alpha)$.
6. *Threshold below 0.* Now $\alpha = 1 - \frac{1}{2}e^{\tau}$ exceeds $\frac{1}{2}$, so $\frac{1}{2}e^{\tau} = 1 - \alpha$ and $\beta = \frac{1}{2}e^{\tau-\kappa} = e^{-\kappa}(1-\alpha)$.

□

The upper bound e^{κ} on the ratio limits every test, not only thresholds. Here $p > 0$ everywhere, so by Lemma 4.2 any test's power is $\mathbb{E}_P[\phi\Lambda] \le e^{\kappa}\alpha$, and the first piece shows that the best test reaches this bound.

To compare the shapes fairly, match the pairs by their best advantage, the largest gap TPR$-$FPR of any test, which Section 4.6 shows is their total variation. That is 0.3935 for the Laplace pair at $\kappa = 1$, and solving $2\Phi(\mu/2) - 1 = 0.3935$ (Corollary 4.2) gives the matching Gaussian pair, $\mu = 1.0301$. The two curves nearly coincide in the bulk but not at small α (Table 4.4). At $\alpha = 10^{-6}$ the Gaussian test is 36.2 times more powerful, because its unbounded ratio lets a test find rare observations that are overwhelmingly more likely under Q (Figure 4.4).

Table 4.4.: Best power at small false positive rates for two pairs with the same total variation.

α	Laplace power, $\kappa = 1$	Gaussian power, $\mu = 1.0301$
10^{-3}	2.718×10^{-3}	1.969×10^{-2}
10^{-6}	2.718×10^{-6}	9.829×10^{-5}

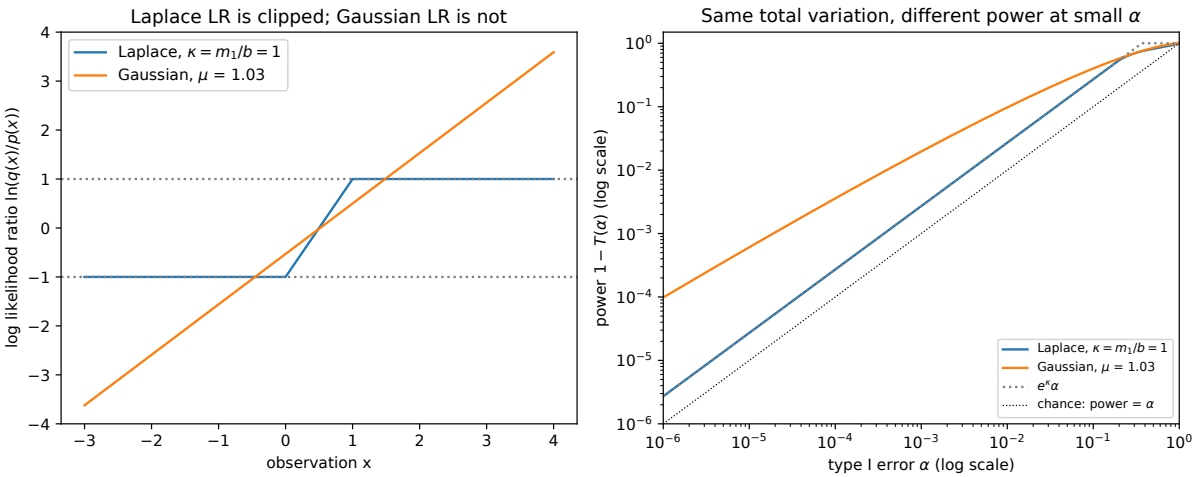


Figure 4.4.: Left: the log likelihood ratio of $\mathrm{Lap}(1,1)$ against $\mathrm{Lap}(0,1)$ is clipped at ± 1, while the Gaussian one with the same total variation is a straight line. Right: best power $1 - T(\alpha)$ on log axes. The Laplace curve follows $e\alpha$ for small α; the Gaussian curve rises far above it.

Example 4.5. Suppose a regression model's residuals are Laplace with scale b, and you test whether a batch's residuals are centred at 0 (P) or shifted by $m_1 = b$ (Q), so $\kappa = 1$. The first breakpoint of Equation 4.12 is $\frac{1}{2}e^{-1} = 0.1839$, so at $\alpha = 0.05$ the first piece gives $\beta = 1 - e \times 0.05 = 0.8641$, while at $\alpha = 0.25$ the middle piece gives $\beta = e^{-1} = 0.3679$ and at $\alpha = 0.75$ the last gives 0.0920. Simulating the threshold test at $\alpha = 0.25$ with 1,000,000 draws per hypothesis gives $\hat{\alpha} = 0.2498$ and $\hat{\beta} = 0.3685$.

4.6. Best advantage equals total variation

Sometimes you want one number for how distinguishable P and Q are. A natural one is the **advantage** of a test, $1 - \alpha_\phi - \beta_\phi = \mathrm{TPR} - \mathrm{FPR}$, how much more often it fires under Q than under P, which is 0 for the coin flip. The advantage adds up $\phi(x)$ times $q(x) - p(x)$ over the outcomes, so the best test fires exactly where q exceeds p. Its advantage is the total excess of q over p, the total variation (TV) distance of Divergences.

Theorem 4.5 (Best advantage equals total variation)**.** *For any distributions P and Q with mass functions p and q,*

$$\max_{\phi} (1 - \alpha_\phi - \beta_\phi) = \mathrm{TV}(P, Q) = \sup_{A} \left(\Pr_{Q}[A] - \Pr_{P}[A] \right) = \frac{1}{2} \sum_{x} |q(x) - p(x)|, \tag{4.14}$$

with an integral for densities. The LR test with cut-off $c = 1$, which rejects where $q > p$, attains the maximum.

Proof.

1. *Advantage as one sum.* By Definition 4.1 the advantage is $\mathbb{E}_Q[\phi] - \mathbb{E}_P[\phi] = \sum_x \phi(x)(q(x) - p(x))$.
2. *Maximize term by term.* Each $\phi(x)$ appears in one term, so the maximum takes $\phi(x) = 1$ where $q(x) > p(x)$ and 0 where $q(x) < p(x)$. It equals $\Pr_Q[A^*] - \Pr_P[A^*]$ for $A^* = \{q > p\}$. Each event A gives the deterministic test $\mathbf{1}[x \in A]$, so the supremum over events is at most this maximum, and A^* attains it.
3. *Half the absolute difference.* Both tables sum to 1, so the positive and negative parts of $q - p$ have equal totals, and each is half of $\sum_x |q - p|$.

□

On the curve, the advantage at level α is the gap $1 - \alpha - T(\alpha)$ between the diagonal and the curve. For a continuous pair it peaks where the slope of T is -1, which by Proposition 4.1 is where the ratio at the threshold is 1 (Figure 4.5). The next result evaluates the peak for the two shift pairs.

Corollary 4.2 (Total variation of the two shift pairs)**.** *For* $\mathcal{N}(0,1)$ *against* $\mathcal{N}(\mu,1)$*,* TV $= 2\Phi(\mu/2) - 1$*, reached at* $\alpha = 1 - \Phi(\mu/2)$*. For* $\mathrm{Lap}(0,b)$ *against* $\mathrm{Lap}(m_1,b)$ *with* $\kappa = m_1/b$*,* $\mathrm{TV} = 1 - e^{-\kappa/2}$*, reached at* $\alpha = \frac{1}{2}e^{-\kappa/2}$*.*

Proof.

1. *Gaussian.* By step 1 of the proof of Theorem 4.3, $q > p$ exactly when $x > \mu/2$, halfway between the means. That test has $\alpha = \beta = 1 - \Phi(\mu/2)$, so its advantage is $2\Phi(\mu/2) - 1$.
2. *Laplace, the ends.* On the first piece of Equation 4.12 the gap is $(e^{\kappa} - 1)\alpha$, which grows, and on the last it is $(1 - \alpha)(1 - e^{-\kappa})$, which shrinks, so the peak is on the middle piece.
3. *Laplace, the middle.* There the gap is $1 - \alpha - e^{-\kappa}/(4\alpha)$, whose derivative vanishes at $\alpha = \frac{1}{2}e^{-\kappa/2}$. At that point $e^{-\kappa}/(4\alpha) = \frac{1}{2}e^{-\kappa/2}$ too, so the gap is $1 - e^{-\kappa/2}$.

□

TV measures one point of the curve and says nothing about its ends, as Table 4.4 showed, so if false alarms must be very rare, the whole curve is the right summary.

Example 4.6. In the spam example half the sum of the differences $|q-p|$ in Table 4.1 is TV $= 0.40$, and rejecting the counts where spam is more likely, $\{2, 3\}$, has advantage $1 - 0.3 - 0.3 = 0.4$ (Table 4.2). For the Gaussian pair at $\mu = 1$, Corollary 4.2 gives 0.3829 at $\alpha = 0.3085$, and so does a grid search over 100,001 values of α. For the Laplace pair at $\kappa = 1$ both give 0.3935 at $\alpha = 0.3033$.

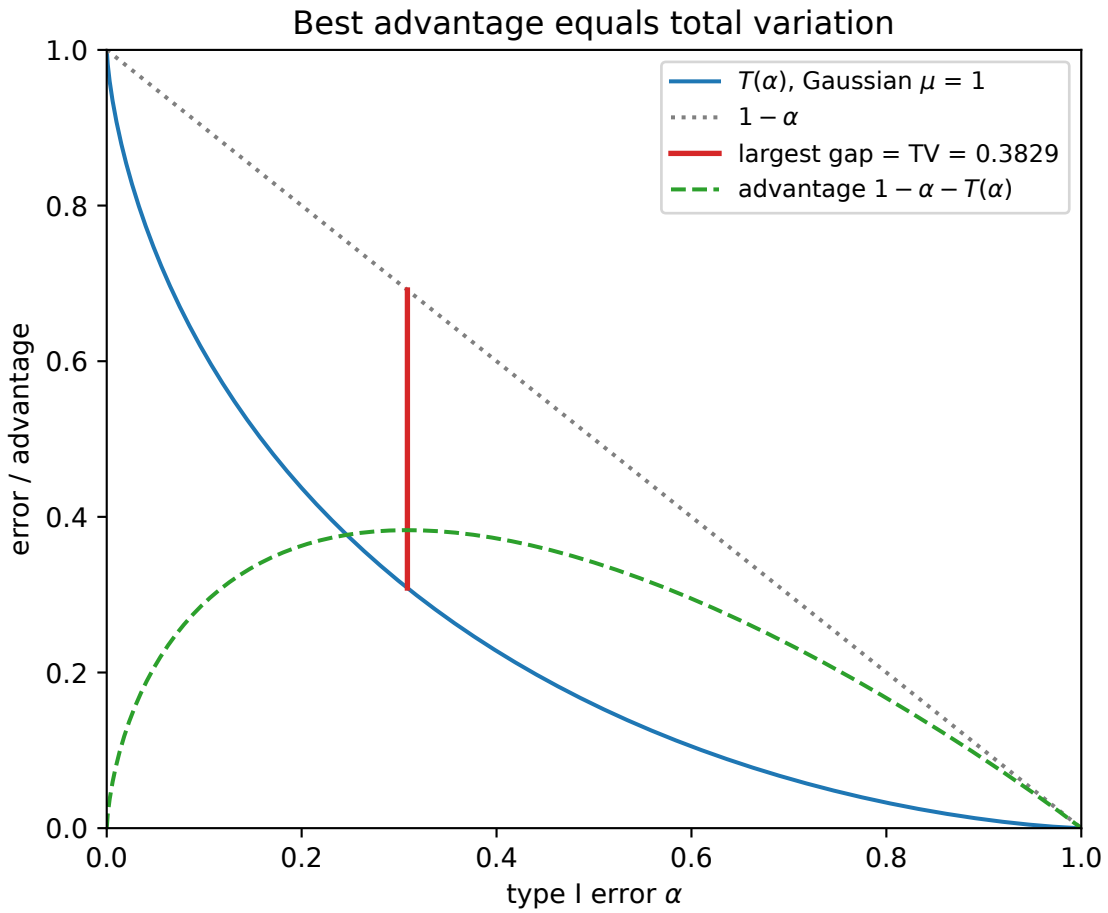


Figure 4.5.: The Gaussian trade-off curve at $\mu = 1$, the diagonal $1 - \alpha$, and the advantage $1 - \alpha - T(\alpha)$ (dashed). The red segment is the largest gap, at $\alpha = 0.3085$, and its length is the total variation 0.3829.

4.7. The area under the ROC curve

A real detector outputs a score and fires above a threshold. Detectors are compared on held-out data by their ROC curves, usually summarized by the area under the curve (AUC). Yet a spam filter, say, cares only about the corner of very low false positive rates. This section shows that the AUC is the probability that the detector scores a random positive above a random negative, and that two detectors with the same AUC can differ tenfold in that corner.

Proposition 4.2 (The AUC is a ranking probability)**.** *Let a detector's score be S_- on negatives and S_+ on positives, independent and continuous. The test "positive when the score exceeds τ" has* $\mathrm{FPR}(\tau) = \Pr[S_- > \tau]$ *and* $\mathrm{TPR}(\tau) = \Pr[S_+ > \tau]$*, and the area under the curve they trace is*

$$\mathrm{AUC} = \int_0^1 \mathrm{TPR}\,\mathrm{d}(\mathrm{FPR}) = \Pr[S_+ > S_-]. \tag{4.15}$$

In the ***binormal model****, $S_- \sim \mathcal{N}(0,1)$ and $S_+ \sim \mathcal{N}(m, \sigma^2)$, the AUC is $\Phi(m/\sqrt{1+\sigma^2})$ and*

$$\mathrm{TPR}(\mathrm{FPR}) = 1 - \Phi\Big(\frac{\Phi^{-1}(1-\mathrm{FPR}) - m}{\sigma}\Big). \tag{4.16}$$

Proof. The plan is to parametrize the curve by the threshold, read the area as an average over a negative's score, and then evaluate both formulas for normal scores.

1. *Parametrize by the threshold.* As τ goes from $+\infty$ to $-\infty$ the FPR goes from 0 to 1. With p_- the density of S_-, $\mathrm{FPR}(\tau) = \int_\tau^\infty p_-(s)\,\mathrm{d}s$, so $\mathrm{d}\,\mathrm{FPR}(\tau) = -p_-(\tau)\,\mathrm{d}\tau$.
2. *Change variables.* Substituting, and flipping the limits to absorb the minus sign,
$$\mathrm{AUC} = \int_{-\infty}^{\infty} \Pr[S_+ > \tau]\, p_-(\tau)\,\mathrm{d}\tau. \tag{4.17}$$
3. *Read it as an average.* A positive beats a negative with score τ with probability $\Pr[S_+ > \tau]$, and the integral averages this over the negatives' scores, which gives $\Pr[S_+ > S_-]$.
4. *Binormal AUC.* The difference of independent normals is normal, with the means subtracted and the variances added, so $S_+ - S_- \sim \mathcal{N}(m, 1+\sigma^2)$, which is positive with probability $\Phi(m/\sqrt{1+\sigma^2})$.
5. *Binormal ROC.* As in step 3 of the proof of Theorem 4.3, the threshold is $\tau = \Phi^{-1}(1-\mathrm{FPR})$, and a positive exceeds it when $(S_+ - m)/\sigma$ exceeds $(\tau - m)/\sigma$.

□

Example 4.7. Detector A has positives that score $\mathcal{N}(1,1)$, so $\mathrm{AUC} = \Phi(1/\sqrt{2}) = 0.7602$. Detector B has positives that score $\mathcal{N}(m, 2^2)$, more spread out, with $m/\sqrt{1+2^2} = 1/\sqrt{2}$, so that $m = 1.5811$ gives the same AUC. Table 4.5 compares them. At FPR 10^{-3} the threshold is 3.0902, so A's TPR is $1 - \Phi(3.0902 - 1) = 0.0183$, while B's is $1 - \Phi((3.0902 - 1.5811)/2) = 1 - \Phi(0.7546) = 0.2253$, which is $0.2253/0.0183 = 12.3$ times more. (Figure 4.6).

Table 4.5.: Two binormal detectors with equal AUC. The simulated AUC counts correctly ranked pairs among 1,000,000.

Detector	AUC (formula / simulated)	TPR at FPR $= 0.1$	TPR at FPR $= 10^{-3}$
A, $\mathcal{N}(1, 1)$	0.7602 / 0.7605	0.3891	0.0183
B, $\mathcal{N}(1.5811, 2^2)$	0.7602 / 0.7601	0.5595	0.2253

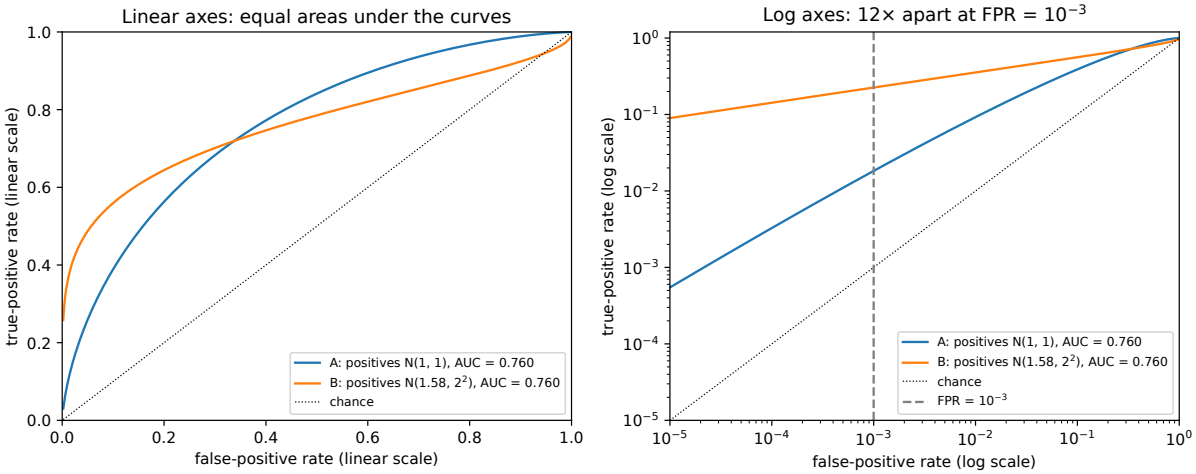


Figure 4.6.: ROC curves of two detectors with the same AUC of 0.760. Left, linear axes: the curves cross and enclose equal areas. Right, log axes: at FPR $= 10^{-3}$ detector B has 12 times the TPR of detector A.

The AUC hides this because it weights all FPRs equally, and FPRs up to 10^{-3} are a tenth of a percent of the axis. A few of B's spread-out positives score far above every negative, and a strict threshold catches exactly those. So report the TPR at a fixed low FPR chosen by the application, and plot ROC curves on log–log axes. An extreme case is a detector with AUC 0.525 that catches 5% of positives at FPR 10^{-3}.

A's threshold is optimal, but B's is not. With unequal variances the x^2 terms of the two log densities no longer cancel, so the log likelihood ratio is quadratic in the score, so the optimal test of Theorem 4.1 also flags very *low* scores.

4.8. The Bayesian view

Neyman–Pearson uses no prior, but often you know a base rate, such as 1% of patients being ill. Then you ask how likely H_1 is after the observation, and which rule makes the fewest mistakes. The likelihood ratio answers both, and shows why a low false positive rate matters most when positives are rare.

Theorem 4.6 (Posterior odds and the Bayes-optimal test)**.** *Let H_1 have prior probability $\pi \in (0,1)$. The odds of H_1 after seeing x are the odds before seeing it, multiplied by the likelihood ratio of x:*

$$\underbrace{\frac{\Pr[H_1 \mid x]}{\Pr[H_0 \mid x]}}_{\text{posterior odds}} = \underbrace{\frac{\pi}{1-\pi}}_{\text{prior odds}} \times \underbrace{\frac{q(x)}{p(x)}}_{\Lambda(x)}. \tag{4.18}$$

The test that minimizes the probability of error, $(1-\pi)\alpha_\phi + \pi\beta_\phi$, is the LR test with cut-off $c = (1-\pi)/\pi$.

Proof.

1. *Bayes' rule.* The probability of H_1 and x together is $\pi q(x)$, and that of H_0 and x is $(1-\pi)p(x)$. Each posterior is one of these divided by the total probability of x, which cancels in their ratio and leaves Equation 4.18.
2. *The error of a test.* A test errs when H_0 holds and it rejects, or when H_1 holds and it misses. Writing both errors as sums by Definition 4.1,

$$(1-\pi)\alpha_\phi + \pi\beta_\phi = \pi + \sum_x \phi(x)\big((1-\pi)p(x) - \pi q(x)\big). \tag{4.19}$$

3. *Minimize term by term.* Each $\phi(x)$ appears in one term, so the minimum sets $\phi(x) = 1$ where the bracket is negative, that is where $\Lambda(x) > (1-\pi)/\pi$, and 0 where it is positive.

□

With equal priors the error is $\frac{1}{2}(\alpha_\phi + \beta_\phi)$, one half minus half the advantage, so its smallest value is $\frac{1}{2}(1 - \mathrm{TV})$ by Theorem 4.5, which is 0.3085 for the Gaussian pair at $\mu = 1$. The theorem also applies to coarser evidence: if all you know is that a test fired, the likelihood ratio of that evidence is $(1-\beta)/\alpha$, the power over the false positive rate.

Example 4.8. Use the biomarker test with $\mu = 2$ from Table 4.3 in a population where 1% are ill, so the prior odds are $1/99$. To get the probability of illness after a positive result, multiply the prior odds by the likelihood ratio of a positive, then turn the odds o into a probability $o/(1+o)$.

At $\alpha = 0.05$ the power is 0.6388, so a positive has likelihood ratio $0.6388/0.05 = 12.8$. The posterior odds are $12.8/99 = 0.129$, so the probability of illness is 0.114, and nine positives in ten are false alarms. At $\alpha = 10^{-3}$ the likelihood ratio is $0.1378/0.001 = 137.8$, and the probability of illness rises to 0.582. The stricter test catches fewer ill patients, but its positives are far more informative (Figure 4.7).

The test with the fewest errors at this prior rejects where $\Lambda(x) > 99$, which for the biomarker means $x > 3.2976$. It has $\alpha = 4.88 \times 10^{-4}$ and power only 0.0972, because when both errors count equally and the disease is rare, it is cheapest to rarely say "ill".

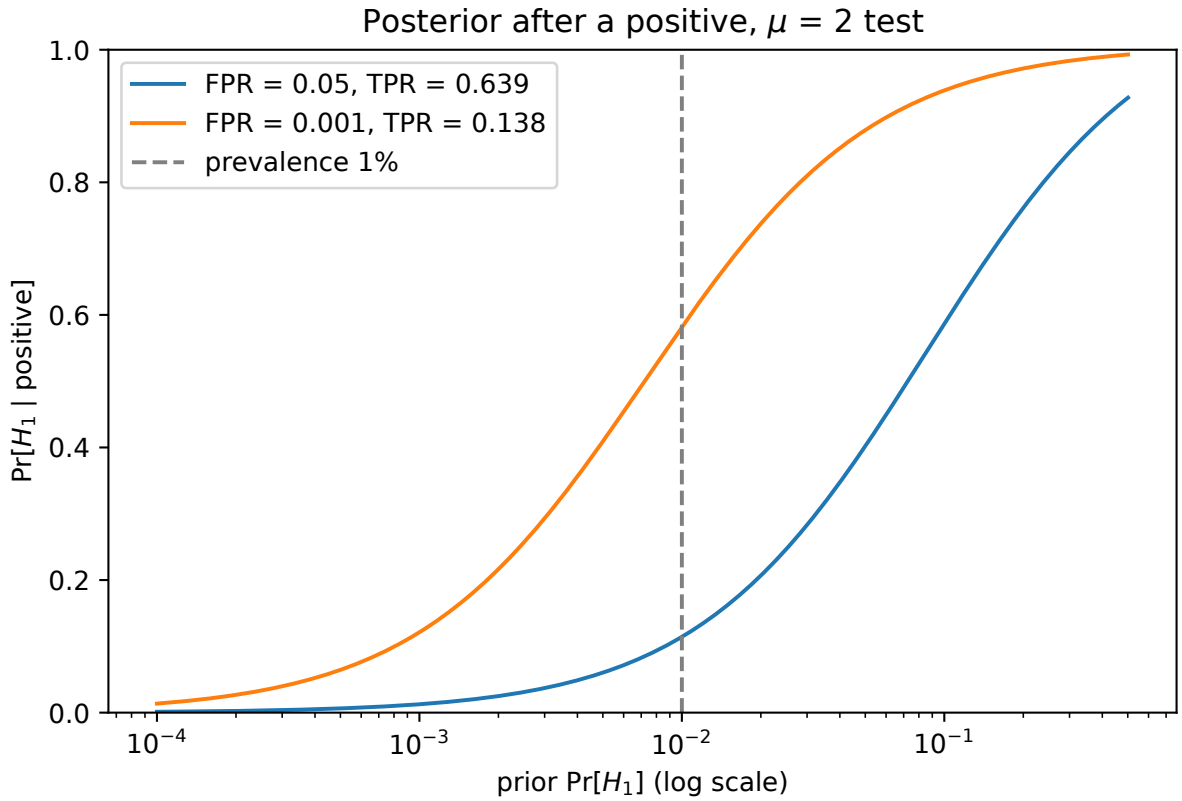


Figure 4.7.: Posterior probability of illness after a positive result, as a function of the prior, for the $\mu = 2$ biomarker test at two thresholds. At 1% prevalence (dashed line) the stricter test gives 0.582 against 0.114.

4.9. Estimating error rates from finite trials

Everything so far assumed that you know P and Q. In practice you measure a detector's rates on test data, for example k false positives among n negatives. If each negative is flagged independently with probability p, here a rate rather than a density, then k is a draw from $\mathrm{Bin}(n, p)$ (Probability distributions), and at low rates its uncertainty is large compared with the rate itself.

An interval for the rate collects the true rates that would make the observed count unsurprising. A rate far above k/n would make a count as small as k very unlikely, so the upper end u is the rate at which seeing k or fewer has probability $\alpha/2$. Likewise the lower end l is the rate at which seeing k or more has probability $\alpha/2$. Here α is the chance that the interval misses, not a test's size, and the confidence level is $1 - \alpha$.

Theorem 4.7 (Clopper–Pearson interval)**.** *For k successes in n independent trials and a level $1 - \alpha$, let l and u solve*

$$\Pr[\mathrm{Bin}(n, l) \geq k] = \frac{\alpha}{2}, \qquad \Pr[\mathrm{Bin}(n, u) \leq k] = \frac{\alpha}{2}, \tag{4.20}$$

with $l = 0$ if $k = 0$ and $u = 1$ if $k = n$. Then $\Pr[l \leq p \leq u] \geq 1 - \alpha$ for every true rate p.

Proof. The interval is random, because it depends on the count, and the claim is about how often it misses the fixed true rate. The plan is to show that $p \geq u$, which includes every miss above, has probability at most $\alpha/2$; the lower end is symmetric. Write $F_p(k) = \Pr[\mathrm{Bin}(n, p) \leq k]$ and let $K \sim \mathrm{Bin}(n, p)$ be the observed count.

1. *Monotonicity.* $F_p(k)$ is continuous and strictly decreasing in p for $k < n$: a larger success probability makes small counts less likely (Section F.3 proves it). So for each k there is a single rate $u(k)$ with $F_{u(k)}(k) = \alpha/2$, and $p \geq u(K)$ exactly when $F_p(K) \leq \alpha/2$.
2. *A CDF evaluated at its own draw is rarely small.* Let k^* be the largest k with $F_p(k) \leq \alpha/2$. Since F_p is increasing in k, $F_p(K) \leq \alpha/2$ exactly when $K \leq k^*$, which has probability $F_p(k^*) \leq \alpha/2$ (or 0 if there is no such k^*).
3. *Combine the two ends.* The lower end misses with probability at most $\alpha/2$ in the same way, and by the union bound the interval misses with probability at most α.

□

The interval is exact, since it uses the binomial itself, and conservative, since its coverage may exceed $1 - \alpha$. For a zero count, a one-sided upper bound at level 0.95, which puts all of the 0.05 on the upper side, solves $(1 - u)^n = 0.05$. Since $\ln(1 - u) \approx -u$ for small u, this gives $u \approx \ln(20)/n \approx 3/n$, the **rule of three**. With 3,000 negatives and no false positive, $u = 9.981 \times 10^{-4}$, and certifying an FPR below 10^{-3} takes 2,995 clean negatives.

The interval's width relative to the rate is set mainly by the expected count np, since a binomial count with a small rate has variance close to its mean, so a count near k fluctuates by about $\sqrt{k}$. Ten false positives leave the rate uncertain by a factor of two, and a hundred, at $n = 100{,}000$, by about 20%: $[0.00081, 0.00122]$ around 10^{-3}. So measuring an FPR near p to within 20% takes about $100/p$ negatives. A threshold tuned to hit a target rate on some data also fits that data's noise, so choose thresholds on one split and measure rates on another.

Example 4.9. Take the $\mu = 2$ biomarker test at the threshold 3.0902, with true FPR 10^{-3} and true TPR 0.1378, and evaluate it on 10,000 negatives and 10,000 positives. Table 4.6 gives one simulated run. The TPR is pinned down to about ± 0.007, but the data are consistent with a true FPR from about half to almost twice the target.

Table 4.6.: One simulated evaluation of the $\mu = 2$ test at its 10^{-3} threshold, with 10,000 examples per class.

Quantity	Count	Estimate	95% Clopper–Pearson interval
False positives	10	0.0010	$[0.00048, 0.00184]$
True positives	1,350	0.1350	$[0.1284, 0.1419]$

Two alternatives do worse. The **Wald** interval, from the normal approximation, gives $[0.00038, 0.00162]$, too narrow and symmetric for a skewed count, and at $k = 0$ it collapses to $[0, 0]$. The interval from Hoeffding's inequality (Concentration inequalities) has half-width $\sqrt{\ln(2/0.05)/(2n)} = 0.0136$, over 13 times the rate itself, because it uses only the range $[0, 1]$ and ignores the tiny variance $p(1 - p)$. Clopper–Pearson is conservative but not loose: over 2,000 simulated evaluations it covered the true rate 97.20% of the time. Figure 4.8 shows how slowly it tightens as n grows.

4.10. Code

The script code/hypothesis_testing.py computes every number quoted in this chapter and draws every figure. It checks the Neyman–Pearson lemma against random tests, and the closed-form curves and the interval's coverage by simulation. From the repository root:

```
uv run python code/part01-prerequisites/hypothesis_testing.py
```

Table 4.7.: Results printed by the script with seed 0.

Experiment	Result
Spam example, LR test at $\alpha = 0.25$	Cut-off 1.5, $\gamma = 0.75$, power 0.625
200,000 random tests on the spam example	11,727 with size ≤ 0.25; best power 0.587
Gaussian $\mu = 2$, $\alpha = 0.05$	$\beta = 0.3612$; simulated $\hat{\alpha} = 0.0503$, $\hat{\beta} = 0.3617$
Laplace $\kappa = 1$, $\alpha = 0.25$	$\beta = 0.3679$; simulated $\hat{\alpha} = 0.2498$, $\hat{\beta} = 0.3685$
Same TV 0.3935, $\alpha = 10^{-6}$	Laplace power 2.718×10^{-6}; Gaussian 9.829×10^{-5}
Equal-AUC detectors A and B	AUC 0.7602; TPR at 10^{-3}: 0.0183 and 0.2253
Screening at 1% prevalence, $\mu = 2$	Posterior 0.114 at $\alpha = 0.05$; 0.582 at $\alpha = 10^{-3}$
Clopper–Pearson, $k = 10$, $n = 10{,}000$	$[0.00048, 0.00184]$; coverage 0.9720 over 2,000 runs

tests/test_hypothesis_testing.py checks that the output still matches Table 4.7 and every other number in the text.

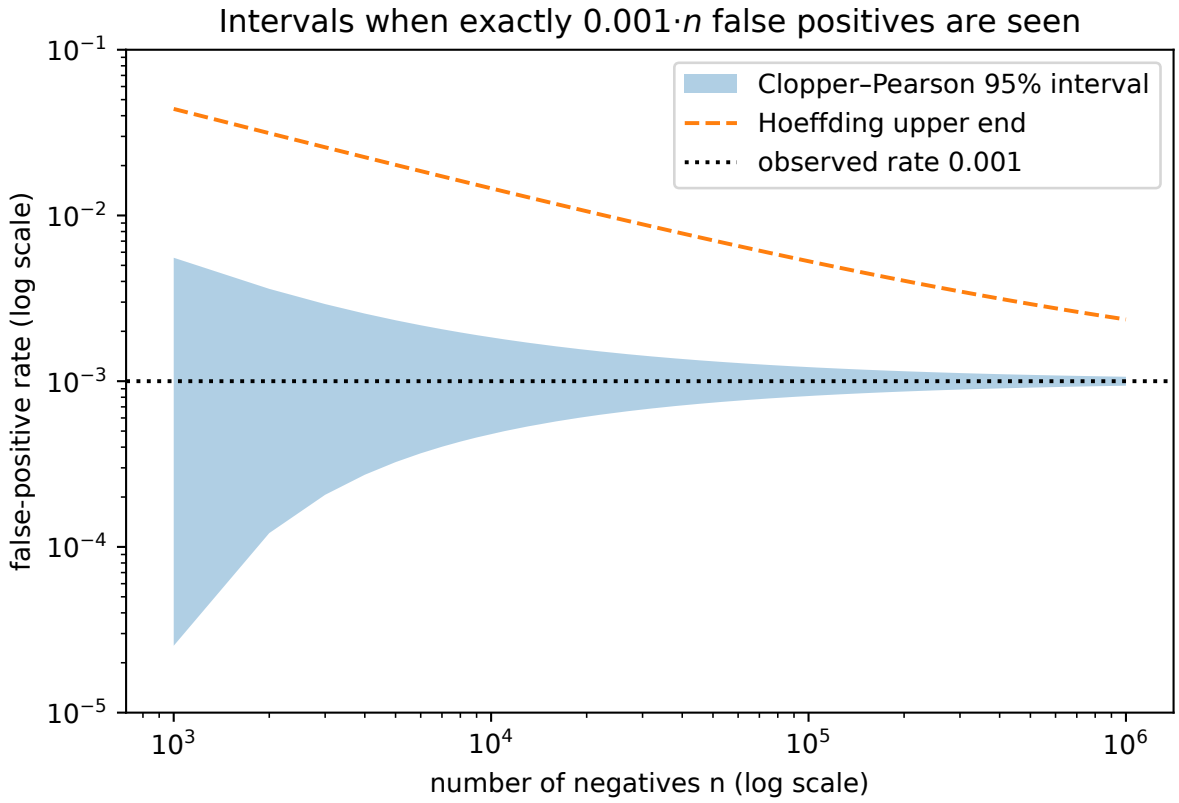


Figure 4.8.: 95% Clopper–Pearson intervals for an observed false positive rate of exactly 10^{-3}, as the number of negatives grows from 10^3 to 10^6, with the upper end of the two-sided Hoeffding interval for comparison.

4.11. Summary

Table 4.8.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 4.1	$\alpha_\phi = \mathbb{E}_P[\phi]$, $\beta_\phi = 1 - \mathbb{E}_Q[\phi]$	Reading any detector's errors
Theorem 4.1	The LR test of size α is most powerful at level α	To find the best test, threshold Λ, or a statistic it increases with
Definition 4.2	$T(\alpha) = \inf\{\beta_\phi : \alpha_\phi \le \alpha\}$; ROC curve $1 - T$	The whole trade-off as one curve
Lemma 4.1	Mixing tests mixes their errors	Randomizing fills the gaps between thresholds
Lemma 4.2	$\mathbb{E}_Q[g] = \mathbb{E}_P[g\Lambda] + \mathbb{E}_Q[g\,\mathbf{1}[p = 0]]$	A ratio at most e^κ caps power at $e^\kappa\alpha$
Theorem 4.2	Non-increasing, $\le 1 - \alpha$, convex, continuous	Exactly the possible trade-offs
Proposition 4.1	Slope of T is $-\Lambda$ at the threshold	Reading a curve's shape off the ratio
Theorem 4.3	$\Phi(\Phi^{-1}(1-\alpha) - \mu)$	Reference curve for normal noise
Corollary 4.1	$\mu = h\sqrt{n}/\sigma$	Half the shift needs four times the data
Theorem 4.4	Straight ends of slopes $-e^{\pm\kappa}$, $\kappa = m_1/b$	Power at most $e^\kappa\alpha$ at small α
Theorem 4.5	Best advantage $= \mathrm{TV}(P, Q)$	One point of the curve; silent about the ends
Corollary 4.2	Gaussian $2\Phi(\mu/2) - 1$; Laplace $1 - e^{-\kappa/2}$	Matching pairs by distinguishability
Proposition 4.2	$\mathrm{AUC} = \Pr[S_+ > S_-]$	Also report the TPR at a fixed low FPR
Theorem 4.6	Posterior odds = prior odds $\times\,\Lambda$	Whether an alarm is worth acting on at a base rate
Theorem 4.7	Exact binomial interval; $u \approx 3/n$ at $k = 0$	Report every measured rate with it; plan $100/p$ trials

4.12. References

- Lehmann & Romano, *Testing Statistical Hypotheses* (3rd ed., 2005), ch. 3: the rigorous Neyman–Pearson lemma (§3.2) with its converse, and monotone likelihood ratios.
- Casella & Berger, *Statistical Inference* (2nd ed., 2002), §8.3: a gentler proof of the lemma, and the Karlin–Rubin theorem for one-sided composite tests.
- Wasserman, *All of Statistics* (2004), ch. 10: a compact tour of tests, power and p-values.

- Cover & Thomas, *Elements of Information Theory* (2nd ed., 2006), §11.7–11.9: the Chernoff–Stein lemma and Chernoff information.
- Fawcett, "An introduction to ROC analysis" (Pattern Recognition Letters, 2006): ROC curves, AUC as a ranking probability, and the convex hull of a classifier's ROC.
- Clopper & Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial" (Biometrika, 1934), and Brown, Cai & DasGupta, "Interval Estimation for a Binomial Proportion" (Statistical Science, 2001): the exact interval, and why the Wald interval fails.

5. Information theory

You observe something, such as a word in an email, and you want to know how much it tells you about something you cannot see, such as whether the email is spam. Information theory turns "how much" into a number in two ways, because there are two questions you might be asking. The first is how many bits, on average, the observation saves you when you describe the hidden value, and Shannon entropy and mutual information answer it. The second is how much more likely a single best guess is to be right once you have seen the observation, and min-entropy answers that one. Later parts need both views, and the places where they disagree.

The idea that organizes the chapter is that every quantity here is built from the logarithm of a probability, either averaged over the outcomes or taken at the most likely one. So almost every proof either compares the average of a logarithm with the logarithm of an average, which Jensen's inequality does because the logarithm is concave, or takes logarithms of the product rule for probabilities to turn it into a sum. Logarithms are base 2, so quantities are in **bits**. Divergences uses the natural logarithm ln, whose unit is the nat, so a value in nats is this chapter's value in bits times $\ln 2$.

The running example is a spam filter's keyword. One email in eight is spam, every spam email contains the word "offer", and so does one in seven legitimate (ham) emails. The first two sections measure what the keyword says about the label, and Section 5.5 shows that it does not help someone who must guess the label.

5.1. Shannon entropy

How uncertain is a random variable? A fair coin should count as one unit of uncertainty, two fair coins as two, and a coin that always lands heads as zero. Shannon entropy is the measure with these properties. The rarer an outcome, the more it tells you when it happens, so the definition gives each outcome a number that grows as its probability shrinks, and averages it.

Definition 5.1 (Entropy)**.** Let X be a discrete random variable with distribution p. The **surprisal** of an outcome x is $\log_2(1/p(x))$ bits, so an outcome of probability $\frac{1}{4}$ carries 2 bits. The **entropy** of X is its average surprisal,

$$H(X) = -\sum_x p(x) \log_2 p(x) = \mathbb{E}\left[\log_2 \frac{1}{p(X)}\right], \tag{5.1}$$

with the convention $0 \log_2 0 = 0$, so that impossible outcomes contribute nothing. For a variable that is 1 with probability t and 0 otherwise, the sum has two terms and is the **binary entropy function**

$$h(t) = -t \log_2 t - (1-t) \log_2 (1-t). \tag{5.2}$$

Entropy cannot be negative, because no surprisal is, and it should be largest when no outcome is favoured. The proof of the upper bound measures how far the distribution is from uniform. For two distributions p and q on the same finite set, with q positive wherever p is, the **Kullback–Leibler (KL) divergence** is

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)}, \tag{5.3}$$

the average, over outcomes drawn from p, of the log of how much more likely each outcome is under p than under the other distribution q. Divergences proves Gibbs' inequality for it: the divergence is never negative, and it is zero only when $p = q$.

Theorem 5.1 (Range of the entropy)**.** *Let X take values in a finite set $\mathcal{X}$ with $|\mathcal{X}|$ elements. Then*

$$0 \le H(X) \le \log_2 |\mathcal{X}|. \tag{5.4}$$

The lower bound holds with equality exactly when X is constant, and the upper bound exactly when X is uniform on $\mathcal{X}$.

Proof.

1. *Lower bound.* Every probability is at most 1, so every surprisal and every term of the sum is non-negative. A sum of non-negative terms is zero only if every term is, and a term is zero only when its probability is 0 or 1. So zero entropy forces one value to have probability 1.
2. *Rewrite the constant.* Write u for the uniform distribution, which gives each value probability $1/|\mathcal{X}|$. The probabilities $p(x)$ sum to 1, so the constant $\log_2 |\mathcal{X}|$ is the average of $\log_2(1/u(x))$ under p, and subtracting $H(X)$ term by term combines two logarithms in each term:

$$\log_2 |\mathcal{X}| - H(X) = \sum_x p(x) \log_2 \frac{p(x)}{u(x)} = D_{\mathrm{KL}}(p \,\|\, u). \tag{5.5}$$

3. *Upper bound.* Gibbs' inequality makes the right side non-negative, and zero only when $p = u$. So $H(X) \le \log_2 |\mathcal{X}|$, with equality only for the uniform distribution.

□

For two variables you also need the uncertainty left about one once the other is known. Their joint distribution is a table $p(x, y)$, with one row for each value of X and one column for each value of Y. Adding up a row gives the **marginal** probability $p(x)$, and adding up a column gives $p(y)$. Before you observe Y you do not know which value you will see, so the uncertainty left about X is summarized by an average over those values.

Definition 5.2 (Joint and conditional entropy)**.** Let X and Y take finitely many values, with joint distribution $p(x, y)$. The **joint entropy** $H(X, Y)$ is the entropy of the pair. Once you observe $Y = y$, the uncertainty about X is the entropy $H(X \mid Y = y)$ of its distribution $p(x \mid y)$ given that value. The **conditional entropy** is its average over the observation,

$$H(X \mid Y) = \sum_y p(y)\, H(X \mid Y = y). \tag{5.6}$$

The uncertainty in a pair should be the uncertainty in one variable plus what remains of the other once the first is known. Logarithms turn the product rule for probabilities into a sum rule for surprisals, and averaging keeps sums.

Theorem 5.2 (Chain rule)**.** *For any two discrete random variables X and Y,*

$$H(X,Y) = H(Y) + H(X \mid Y) = H(X) + H(Y \mid X). \tag{5.7}$$

Proof.

1. *Turn the product into a sum.* For every pair with $p(x,y) > 0$, the product rule gives $p(x,y) = p(y)\,p(x \mid y)$. Taking $\log_2(1/\cdot)$ of both sides gives
$$\log_2 \frac{1}{p(x,y)} = \log_2 \frac{1}{p(y)} + \log_2 \frac{1}{p(x \mid y)}. \tag{5.8}$$
2. *Average over the pair.* The left side averages to $H(X,Y)$ and the first term on the right to $H(Y)$. Grouped by y, the second term averages to $H(X \mid Y = y)$ within each column, and then over y to $H(X \mid Y)$.
3. *Swap the roles.* Starting from $p(x,y) = p(x)\,p(y \mid x)$ instead, the same two steps give the second form.

□

Applying Equation 5.7 repeatedly, one variable at a time, extends the chain rule to any number of variables. Figure 5.1 shows the binary entropy, which is 1 bit for a fair coin and zero for a certain one. It rises steeply near the ends, so a coin with only an 11% chance of heads already keeps $h(0.11) = 0.4999$ bits, half the fair coin's.

Learning something should not make you more uncertain on average, as Section 5.2 proves, but a rare observation can push you from a confident belief into a toss-up.

Example 5.1. Let X be the label of an email, ham or spam, and Y whether it contains the word "offer". Ham emails are $\frac{7}{8}$ of the total and one in seven of them contains the word, so ham emails with the word are $\frac{1}{8}$ of all emails and those without it are $\frac{3}{4}$ (Table 5.1).

Table 5.1.: Joint distribution of an email's label and whether it contains the keyword.

	$Y = 0$ (absent)	$Y = 1$ (present)	$p(x)$
ham	3/4	1/8	7/8
spam	0	1/8	1/8
$p(y)$	3/4	1/4	

Before you look at the email, the label is spam with probability $\frac{1}{8}$, so $H(X) = h(1/8) = 0.5436$ bits. If the word is absent, the email is certainly ham, so $H(X \mid Y = 0) = 0$. If the word is present, ham and spam each have probability $\frac{1}{8}$ out of $\frac{1}{4}$, so they are equally likely and $H(X \mid Y = 1) = 1$ bit. Seeing the word made you more uncertain than before.

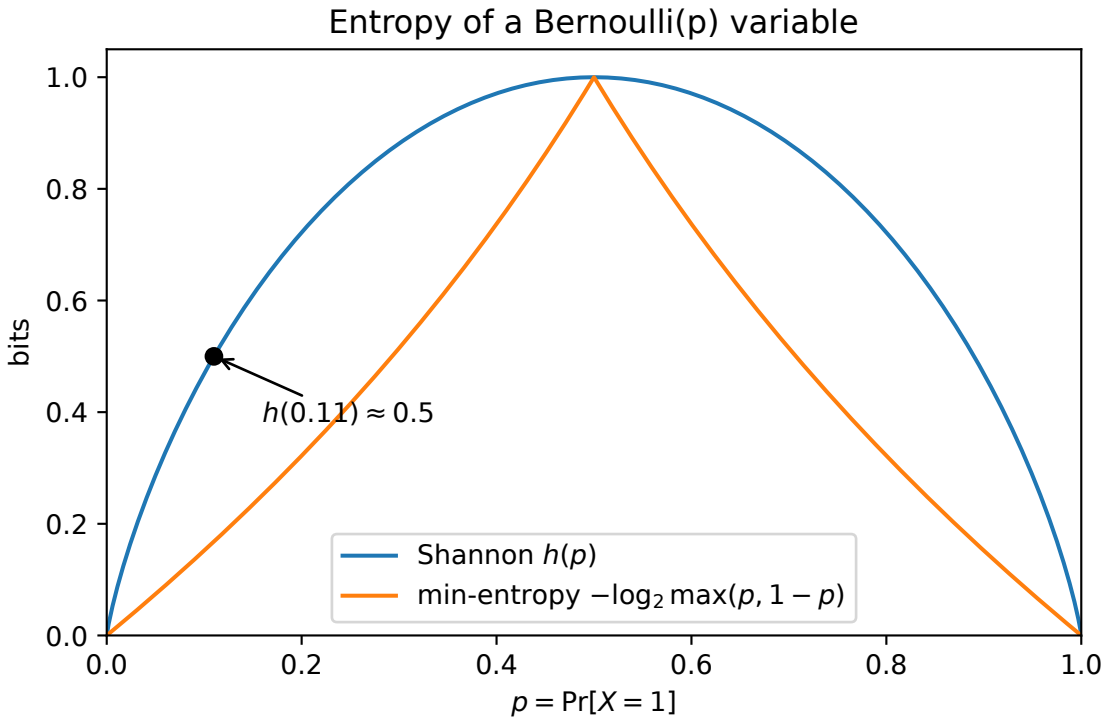


Figure 5.1.: The binary entropy $h(t)$ against the min-entropy $-\log_2 \max(t, 1-t)$ of the same Bernoulli variable. Both are 1 bit at $t = \frac{1}{2}$; the min-entropy, defined in Section 5.4, is lower everywhere else.

On average, though, the word helps. It is absent with probability $\frac{3}{4}$, so $H(X \mid Y) = \frac{3}{4} \cdot 0 + \frac{1}{4} \cdot 1 = 0.25$ bits, less than $H(X)$ (Figure 5.2). The chain rule checks out on the same table, since $H(Y) = h(1/4) = 0.8113$ and $H(Y) + H(X \mid Y) = 1.0613$ is the entropy of the three non-zero cells $(\frac{3}{4}, \frac{1}{8}, \frac{1}{8})$.

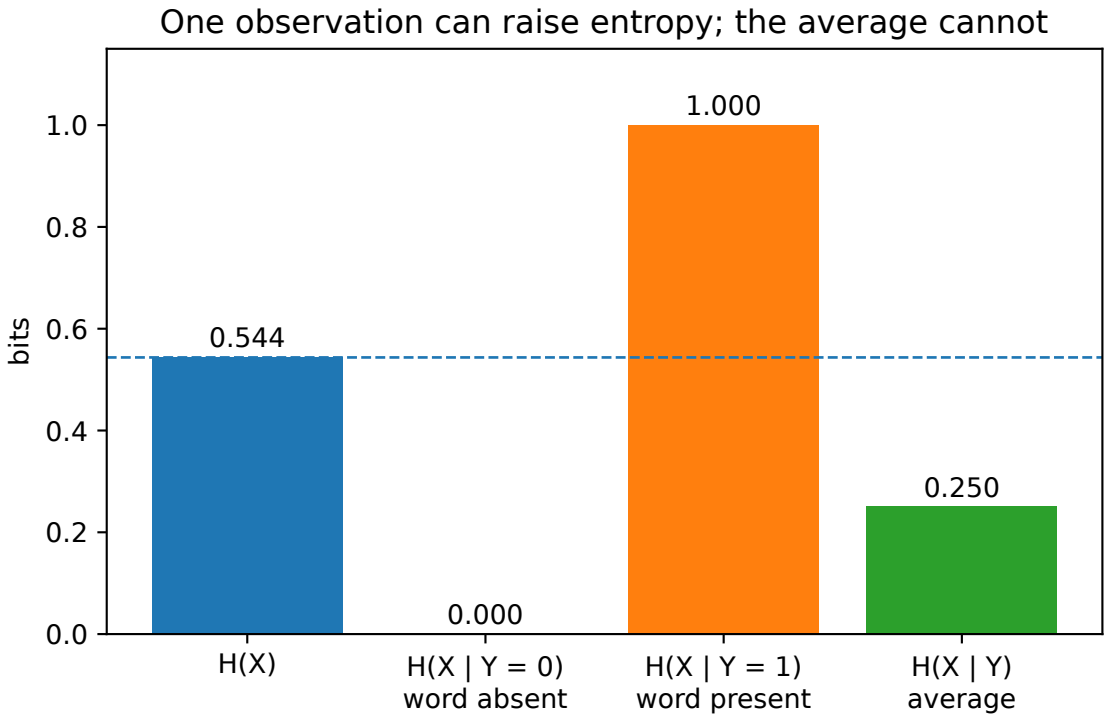


Figure 5.2.: Entropy of the spam label before observing the keyword, after each of the two possible observations, and on average. The rare observation raises the entropy above $H(X)$ (dashed line); the average falls below it.

5.2. Mutual information

How much does observing Y tell you about X? A natural answer is the drop in uncertainty it causes on average, the **information gain**. The spam example showed that one observation can raise the uncertainty, so you need to know that the average drop is never negative. It turns out to be a KL divergence, which makes it non-negative and symmetric in X and Y. It also obeys a rule that much of the book relies on, that processing cannot create information.

Definition 5.3 (Mutual information)**.** The **mutual information** between two discrete random variables X and Y is the average drop in uncertainty about X from seeing Y,

$$I(X;Y) = H(X) - H(X \mid Y). \tag{5.9}$$

Given a third variable Z, the **conditional mutual information** $I(X;Y \mid Z) = H(X \mid Z) - H(X \mid Y, Z)$, where conditioning on Y, Z means conditioning on the pair, is what Y adds about X beyond what Z already told you.

Mutual information can also be read as a measure of dependence, by comparing the joint table with the product of its marginals, the table that X and Y would have if they were independent.

Theorem 5.3 (Mutual information is a KL divergence)**.** *Let X and Y be discrete, write P_{XY} for their joint distribution and $P_X P_Y$ for the product of its marginals. Then*

$$I(X;Y) = D_{\mathrm{KL}}(P_{XY} \,\|\, P_X P_Y) = \sum_{x,y} p(x,y) \log_2 \frac{p(x,y)}{p(x)\,p(y)}. \tag{5.10}$$

So mutual information is symmetric, $I(X;Y) = I(Y;X) = H(Y) - H(Y \mid X)$. It is non-negative, and it is zero if and only if X and Y are independent. Conditional mutual information is the average over z of the same divergence computed from the distributions given $Z = z$, marked with a subscript z, so it is non-negative too:

$$I(X;Y \mid Z) = \sum_{z} p(z)\, D_{\mathrm{KL}}(P_{XY|z} \,\|\, P_{X|z} P_{Y|z}) \geq 0. \tag{5.11}$$

Proof. The plan is to write both entropies as averages over the same pairs (x, y), combine them term by term into one sum, and read the properties off that sum.

1. *Average over pairs.* Each $p(x)$ is the sum of its row, so $H(X)$ is an average over pairs. So is $H(X \mid Y)$, by Definition 5.2 and $p(y)\,p(x \mid y) = p(x,y)$:

$$H(X) = \sum_{x,y} p(x,y) \log_2 \frac{1}{p(x)}, \qquad H(X \mid Y) = \sum_{x,y} p(x,y) \log_2 \frac{1}{p(x \mid y)}. \tag{5.12}$$

2. *Subtract.* Combining the two logarithms in each term gives

$$I(X;Y) = \sum_{x,y} p(x,y) \log_2 \frac{p(x \mid y)}{p(x)}. \tag{5.13}$$

3. *Bring in the joint.* Multiply the fraction by $p(y)/p(y)$. The product rule $p(x \mid y)\,p(y) = p(x,y)$ turns it into $\frac{p(x,y)}{p(x)\,p(y)}$, which gives Equation 5.10.
4. *Read off the properties.* The sum in Equation 5.10 does not change when X and Y swap roles, and steps 1 to 3 with the roles swapped give $H(Y) - H(Y \mid X)$. Gibbs' inequality makes the divergence non-negative, and zero only if the joint table equals the product of its marginals, which is the definition of independence.
5. *Condition on Z.* Both entropies in $I(X;Y \mid Z)$ are averages over z of entropies computed from the distribution given $Z = z$. For $H(X \mid Y, Z)$ this is Definition 5.2 with the pair's values grouped by z. So it is the average over z of a mutual information, and steps 1 to 4 make each one a non-negative divergence.

□

The theorem settles the question the spam example left open, whether the average drop in uncertainty can be negative. Its corollary also bounds the uncertainty of a pair by the sum of the two uncertainties, which Section 5.7 needs.

Corollary 5.1 (Conditioning reduces entropy on average)**.** *For any two discrete random variables X and Y,*

$$H(X \mid Y) \leq H(X), \qquad H(X,Y) \leq H(X) + H(Y), \tag{5.14}$$

each with equality if and only if X and Y are independent.

Proof.

1. *The first inequality.* The gap $H(X) - H(X \mid Y)$ is $I(X;Y)$, which Theorem 5.3 shows is non-negative and zero exactly under independence.
2. *The second inequality.* By Equation 5.7, $H(X,Y) = H(Y) + H(X \mid Y)$, and step 1 bounds the last term by $H(X)$, with the same case of equality.

□

What a pair (Y, Z) tells you about X is what Y tells you, plus what Z adds once Y is known. The data-processing inequality below uses this split twice.

Proposition 5.1 (Chain rule for mutual information)**.** *For any three discrete random variables X, Y and Z,*

$$I(X;Y,Z) = I(X;Y) + I(X;Z \mid Y). \tag{5.15}$$

Proof.

1. *The total drop.* By Definition 5.3, $I(X;Y,Z) = H(X) - H(X \mid Y,Z)$, the drop in uncertainty from knowing nothing to knowing the pair.
2. *Pass through an intermediate stage.* Add and subtract the uncertainty $H(X \mid Y)$ that remains when only Y is known:

$$I(X;Y,Z) = \big[H(X) - H(X \mid Y)\big] + \big[H(X \mid Y) - H(X \mid Y,Z)\big]. \tag{5.16}$$

3. *Name the two brackets.* The first is the drop from learning Y, which is $I(X;Y)$, and the second is the further drop from learning Z when Y is known, which is $I(X;Z \mid Y)$.

□

Denoising, feature extraction and training a model all compute something from data. If the result Z is computed from Y alone, it should not know more about X than Y does.

Theorem 5.4 (Data-processing inequality)**.** *Let X, Y and Z be discrete random variables such that X and Z are independent given Y. This is called a* ***Markov chain****, written $X \to Y \to Z$, and it holds whenever Z is computed from Y alone, possibly with randomness independent of X and Y. Then*

$$I(X;Z) \leq I(X;Y), \tag{5.17}$$

with equality if and only if $I(X;Y \mid Z) = 0$.

Proof. The plan is to expand what the pair (Y, Z) tells you about X in two orders, and to notice that in one order the part Z adds after Y is zero.

1. *Expand twice.* Proposition 5.1 with Z first and with Y first gives
$$I(X;Z) + I(X;Y \mid Z) = I(X;Y,Z) = I(X;Y) + I(X;Z \mid Y). \tag{5.18}$$
2. *Use the Markov chain.* Given each value of Y, the variables X and Z are independent, so each divergence in the conditional form of Theorem 5.3 compares two identical distributions, and $I(X;Z \mid Y) = 0$.
3. *Drop a non-negative term.* Hence $I(X;Z) = I(X;Y) - I(X;Y \mid Z)$. The subtracted term is non-negative by Theorem 5.3, and it is zero exactly in the case of equality.

□

In the case of equality, Z has kept everything in Y that matters for X, and it is called a **sufficient statistic** for X. An invertible function Z of Y is one. Since Y can be computed back from Z, the chain $X \to Z \to Y$ is also Markov, so Theorem 5.4 also gives $I(X;Y) \le I(X;Z)$, and the two are equal.

Mutual information is an average over both x and y. So a small value can hide a few outcomes in which Y reveals X completely, and Section 5.6 shows how far apart the average and the worst case can be.

Example 5.2. In the spam table, the keyword reduced the uncertainty from 0.5436 bits to 0.25 bits on average, so $I(X;Y) = 0.2936$ bits. The script (Section 5.8) gets the same value from Equation 5.10 and from $H(Y) - H(Y \mid X)$.

Now suppose the keyword is detected by a sloppy tokenizer that gets the answer wrong with probability 0.1, and call its output Z. The tokenizer looks only at the word, so $X \to Y \to Z$ is a Markov chain, and the script finds $I(X;Z) = 0.1668$ bits, below $I(X;Y)$ as Theorem 5.4 predicts. The noisy feature carries 57% of the information of the clean one, so a 10% error rate costs over 40% of the information.

5.3. Channels and capacity

The sloppy tokenizer is a channel, which turns an input into a random output, and so is a noisy storage device. You control how often each input is sent, but not the noise. How much information can one use of the channel carry at best? That maximum is the **capacity**, and Shannon's channel coding theorem, not proved here, says that it is the largest number of data bits per use of the channel that long codes can send with an error probability as small as you like.

Definition 5.4 (Channel and capacity)**.** A **channel** from X to Y is a table K whose entry $K[x,y] = p(y \mid x)$ says how likely the output y is given the input x. Each row sums to 1, because some output must occur. A **prior** $\pi(x)$ says how likely each input is, and the product rule gives the joint table $p(x,y) = \pi(x)\,K[x,y]$.

The **capacity** of the channel is the largest mutual information between input and output over all priors,
$$\mathcal{C} = \max_{\pi} I(X;Y), \qquad X \sim \pi, \quad Y \mid X = x \sim K[x,\cdot]. \tag{5.19}$$
The **binary symmetric channel** BSC(p) has inputs and outputs in $\{0,1\}$, and it flips the input with probability p, a number here rather than a distribution.

The tokenizer is BSC(0.1). The capacity of a binary symmetric channel should be one bit, the most a binary output can carry, minus the uncertainty that the noise adds.

Theorem 5.5 (Capacity of the binary symmetric channel)**.** *For every flip probability $p \in [0, 1]$, the capacity of* BSC(p) *is*

$$\mathcal{C}_{\mathrm{BSC}}(p) = 1 - h(p), \tag{5.20}$$

and the uniform input attains it.

Proof. The plan is to split the information into the output's uncertainty minus the noise's, show that the noise part does not depend on the prior, and then make the output as uncertain as possible. Write π_1 for the probability that the input is 1.

1. *Split the information.* By Theorem 5.3, $I(X;Y) = H(Y) - H(Y \mid X)$.
2. *The noise part is fixed.* Given either input, the output is flipped with probability p, so $H(Y \mid X = x) = h(p)$ for both inputs, and their average $H(Y \mid X)$ is $h(p)$ whatever π_1 is.
3. *Bound the output part.* The output takes two values, so $H(Y) \le \log_2 2 = 1$ by Theorem 5.1, with equality if and only if the output is uniform.
4. *The uniform input attains the bound.* The output is 1 when the input is 1 and is not flipped, or the input is 0 and is flipped, so

$$\Pr[Y = 1] = \pi_1(1 - p) + (1 - \pi_1)p. \tag{5.21}$$

   At $\pi_1 = \frac{1}{2}$ this is $\frac{1}{2}$, so the uniform input makes the output uniform, and the largest information is $1 - h(p)$.

□

Figure 5.3 (left) shows the information as a function of the input prior, a hump that peaks at the uniform input for every p and falls fast away from it. At $p = 0.11$ and $\pi_1 = 0.2$, the output is 1 with probability 0.266, so the channel carries only $h(0.266) - h(0.11) = 0.3358$ bits, against 0.5001 at the uniform input. The right panel shows that the capacity is zero at $p = \frac{1}{2}$, where the output is a fair coin whatever the input. The capacity is symmetric about $\frac{1}{2}$, because a receiver can flip every output of a channel that almost always flips, which turns it into one that rarely flips.

For channels without this symmetry, the maximum over priors has no closed form. The **Blahut–Arimoto** algorithm computes it numerically, by alternately choosing the best posterior over inputs for the current prior and the best prior for that posterior, which never lowers the information. The script uses it to check Equation 5.20.

Example 5.3. A cheap storage medium flips each stored bit independently with probability 0.11. Its capacity is $1 - h(0.11) = 0.5001$ bits per stored bit. So codes that add redundant bits to correct errors can store about one bit of data per two raw bits, with an error rate as small as you like, and by the channel coding theorem no code can do better. At a flip probability of 0.25 the capacity drops to $1 - h(0.25) = 0.1887$, about five raw bits per data bit.

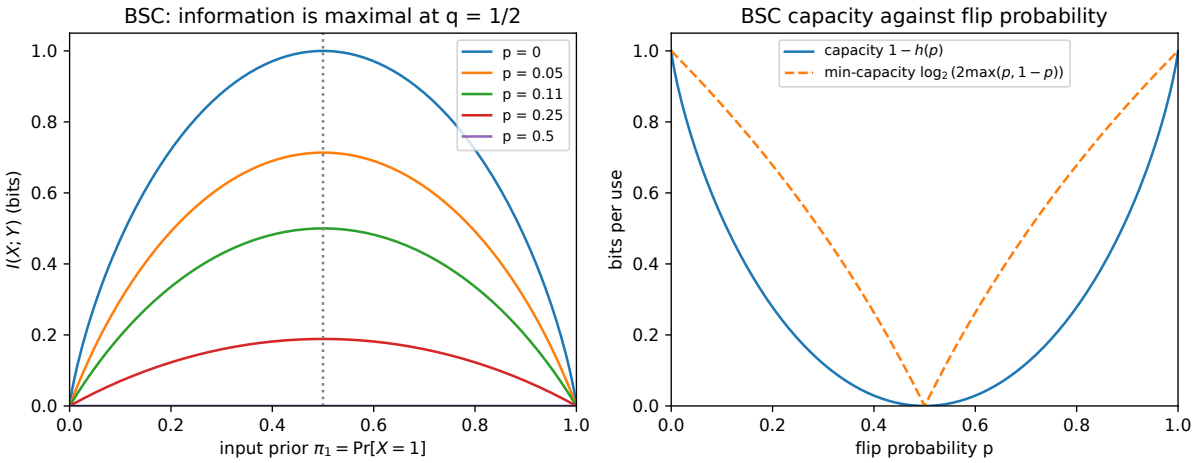


Figure 5.3.: Left: mutual information of a binary symmetric channel as a function of the input prior, for several flip probabilities; every curve peaks at $\pi_1 = \frac{1}{2}$. Right: the capacity $1 - h(p)$ against p, with the min-capacity of Section 5.5 for comparison.

5.4. Min-entropy and guessing

Shannon entropy counts the bits needed to describe a value on average. A different question is how likely you are to guess the value in **one try**. A password chosen by a person might have high entropy and still be guessed one time in ten, because one popular choice dominates. A guess g is right with probability $p(g)$, so the best single guess is the most likely value, and the next two sections build the one-guess versions of mutual information and capacity on its probability.

Definition 5.5 (Vulnerability and min-entropy)**.** Let X be a discrete random variable with distribution p. The **vulnerability** of X is the success probability of the best single guess, and the **min-entropy** puts it on the same bit scale as entropy by taking its negative logarithm:

$$V(X) = \max_x p(x), \qquad H_\infty(X) = -\log_2 V(X). \tag{5.22}$$

A fair coin has $V = \frac{1}{2}$ and $H_\infty = 1$ bit, the same as its Shannon entropy. In general, min-entropy is the surprisal of the most likely outcome, which is the smallest surprisal, while Shannon entropy averages all of them. An average is at least its smallest value, so min-entropy is never larger.

Proposition 5.2 (Min-entropy is at most Shannon entropy)**.** *Let X take values in a finite set $\mathcal{X}$. Then*

$$H_\infty(X) \le H(X) \le \log_2 |\mathcal{X}|, \tag{5.23}$$

*with equality in the first inequality if and only if X is uniform on its **support**, the set of values it takes with positive probability.*

Proof.

1. *Compare each surprisal with the smallest.* Every outcome has $p(x) \le V(X)$, so its surprisal is at least $\log_2(1/V(X)) = H_\infty(X)$.

2. *Average.* The entropy $H(X)$ is the average of the surprisals, so it is at least $H_\infty(X)$. The second inequality is Theorem 5.1.
3. *Equality.* Equality in step 2 needs every surprisal in the support to equal the smallest one, that is $p(x) = V(X)$ for every x with $p(x) > 0$.

□

Min-entropy has a direct reading: a value with a min-entropy of r bits is exactly as hard to guess in one try as a uniform string of r bits, however large its Shannon entropy is. For a coin, Figure 5.1 shows the gap, with a kink in the min-entropy where the most likely outcome switches.

Both entropies are the ends of one family, so a result proved for the family covers both. The **Rényi entropy** of order α is

$$H_\alpha(X) = \frac{1}{1-\alpha} \log_2 \sum_x p(x)^\alpha. \tag{5.24}$$

As α grows, the sum is dominated more and more by the largest probability, and as $\alpha \to \infty$ the formula tends to H_∞. As $\alpha \to 1$ the formula becomes $0/0$, and its limit is Shannon's H. Section F.4 derives both limits, and the family parallels the Rényi divergences of Divergences.

Example 5.4. A password shows the gap between the two entropies better than the two-valued spam label. A uniform 4-digit PIN has $H = H_\infty = \log_2 10{,}000 = 13.2877$ bits. Now suppose 10% of people choose 1234 and the rest choose uniformly among the other 9,999 PINs.

The Shannon entropy is $H = 12.4278$ bits, less than a bit below the uniform case. The chain rule shows where it comes from, applied to X and the indicator B of "the PIN is 1234". Since B is a function of X,

$$H(X) = H(X, B) = H(B) + H(X \mid B). \tag{5.25}$$

Revealing whether the PIN is 1234 costs $h(0.1) = 0.4690$ bits, and then, in the 90% of cases where it is not, revealing which of the others it is costs $\log_2 9{,}999$ bits, or $0.9 \log_2 9{,}999$ on average. That gives $0.4690 + 11.9588 = 12.4278$.

The min-entropy is $H_\infty = \log_2 10 = 3.3219$ bits, because the best guess, 1234, succeeds with probability 0.1. So the Shannon entropy says that this PIN distribution is almost as good as uniform, while the min-entropy says that one guess in ten succeeds.

5.5. Conditional min-entropy and min-entropy leakage

Now the guesser sees an observation Y before guessing the hidden value X, which this section calls the secret. How much does that help a guesser? Smith (2009) answers with a success probability again, that of the best guessing strategy that may depend on what was observed. Comparing it with the vulnerability before the observation gives the leakage.

Definition 5.6 (Posterior vulnerability and min-entropy leakage)**.** A **guessing strategy** g assigns a guess $g(y)$ to each observation y. The **posterior vulnerability** $V(X \mid Y)$ is the largest

success probability $\Pr[X = g(Y)]$ of any strategy, and the **conditional min-entropy** is its negative logarithm,

$$H_\infty(X \mid Y) = -\log_2 V(X \mid Y). \tag{5.26}$$

For a prior π and a channel K, the **min-entropy leakage** $\mathcal{L}(\pi, K)$ is the factor by which the observation multiplies the chance of a correct guess, measured in bits, and the **min-capacity** $\mathcal{ML}(K)$ is its worst case over priors:

$$\mathcal{L}(\pi, K) = \log_2 \frac{V(X \mid Y)}{V(X)} = H_\infty(X) - H_\infty(X \mid Y), \qquad \mathcal{ML}(K) = \max_\pi \mathcal{L}(\pi, K). \tag{5.27}$$

The guesser's choice for one observation does not affect its success on any other, so it can make the best choice one column of the joint table at a time.

Proposition 5.3 (The best strategy is the MAP guess)**.** *For a joint distribution $p(x, y)$, the posterior vulnerability is the sum of the column maxima of the joint table,*

$$V(X \mid Y) = \sum_y \max_x p(x, y) = \sum_y p(y) \max_x p(x \mid y), \tag{5.28}$$

and the strategy that attains it guesses, for each y, the value with the largest posterior probability $p(x \mid y)$. That guess is the ***maximum a posteriori (MAP) guess****.*

Proof.

1. *Write the success as a sum over columns.* A strategy g succeeds with probability $\sum_y p(g(y), y)$, so it picks one cell from each column of the joint table and its success is the sum of the picked cells.
2. *Optimize each column.* The guess $g(y)$ affects only the term for y, so the best choice in that column is the row with the largest entry. Dividing the column by $p(y)$ does not change which row is largest, so this row is also the MAP guess.
3. *Rewrite.* Summing the column maxima gives the first form, and $p(x, y) = p(y)\, p(x \mid y)$ gives the second, the average over y of the vulnerability of the posterior distribution.

□

The min-capacity turns out to need no maximization at all. The idea is to bound the posterior vulnerability by pulling the prior out of each column maximum, and then to notice that the uniform prior loses nothing in that step.

Theorem 5.6 (Min-capacity)**.** *Let K be a channel from a finite set $\mathcal{X}$ of secrets to a finite set $\mathcal{Y}$ of observations. Then*

$$\mathcal{ML}(K) = \log_2 \sum_y \max_x K[x, y], \tag{5.29}$$

the uniform prior attains it, and $0 \le \mathcal{ML}(K) \le \log_2 \min(|\mathcal{X}|, |\mathcal{Y}|)$.

Proof. The plan is to bound the leakage for every prior, to show that the uniform prior attains the bound, and then to bound the sum of the column maxima from both sides.

1. *Pull the prior out.* By Proposition 5.3 with $p(x, y) = \pi(x)K[x, y]$, the posterior vulnerability is a sum of column maxima. In each column, the largest product is at most the largest prior times the largest channel entry:

$$\max_x \pi(x)K[x, y] \leq V(X) \max_x K[x, y]. \tag{5.30}$$

2. *Bound the leakage.* Summing over y and dividing by $V(X)$ gives $V(X \mid Y)/V(X) \leq \sum_y \max_x K[x, y]$ for every prior. Taking $\log_2$ bounds $\mathcal{L}(\pi, K)$ by the right side of Equation 5.29.
3. *The uniform prior attains it.* Under the uniform prior, $\pi(x) = V(X)$ for every x, so the constant comes out of the maximum, step 1 is an equality, and the bound is reached.
4. *The range.* Each column maximum is at most 1, so the sum is at most $|\mathcal{Y}|$. Each is also at most its column sum, so the sum is at most the total of all entries, which is $|\mathcal{X}|$ because each row sums to 1. Finally, the column maxima are at least the entries of any one row, so the sum is at least 1.

□

A leakage of 1 bit means that the observation doubles the chance of a correct first guess. This is why min-entropy is the usual measure in quantitative information flow, the field that studies what a program's outputs reveal about its secret inputs. The min-capacity bounds the leakage for every prior with one number computed from the channel alone. For BSC(p) both column maxima are $\max(p, 1-p)$, so at $p = 0.11$ the min-capacity is $\log_2(2 \times 0.89) = 0.8319$ bits against a Shannon capacity of 0.5001 (Figure 5.3, right).

A natural alternative would average the min-entropy $H_\infty(X \mid Y = y)$ left after each observation. Smith's value takes minus the logarithm of the averaged vulnerability instead, and because $-\log_2$ is convex, Jensen's inequality makes Smith's value never larger than the alternative. Only Smith's is the logarithm of an actual success probability.

Example 5.5. In the spam table, the column maxima are $\frac{3}{4}$ and $\frac{1}{8}$, so by Equation 5.28, $V(X \mid Y) = \frac{7}{8}$. That is exactly $V(X)$, so the leakage is 0. The keyword carries 0.2936 bits of Shannon information, yet a guesser who must name the label says "ham" whatever it sees and gains nothing.

A table where guessing does improve shows each step. A secret X takes four values with prior $\pi = (0.4, 0.3, 0.2, 0.1)$, and an observation Y takes three values, through the channel in Table 5.2. Multiplying each row by its prior gives the joint table (Table 5.3, Figure 5.4).

Table 5.2.: The channel of the guessing walk-through: rows are secret values, columns are observations.

$K[x, y]$	y_1	y_2	y_3
x_1	0.8	0.2	0
x_2	0.1	0.8	0.1
x_3	0	0.5	0.5
x_4	0	0.2	0.8

Table 5.3.: The joint distribution $p(x, y) = \pi(x)K[x, y]$ of the walk-through, with the column maxima in bold.

$p(x,y)$	y_1	y_2	y_3
x_1	**0.32**	0.08	0
x_2	0.03	**0.24**	0.03
x_3	0	0.10	**0.10**
x_4	0	0.02	0.08
$p(y)$	0.35	0.44	0.21

Without the observation, the best guess is x_1, right with probability $V(X) = 0.4$. With it, the MAP guesser picks the bold cell in each column and succeeds with probability $0.32 + 0.24 + 0.10 = 0.66$, so $H_\infty(X \mid Y) = 0.5995$ bits. On y_3 it picks x_3 although the channel entry for x_4 is larger, because x_3 is twice as likely a priori. A million simulated draws give a success rate of 0.6594.

The leakage is $\log_2(0.66/0.4) = 0.7225$ bits, while the mutual information of the same table is 0.6865 bits, so here the two measures nearly agree. Every column of the channel has maximum 0.8, so by Equation 5.29 the min-capacity is $\log_2 2.4 = 1.2630$ bits, against a Shannon capacity of 0.8269 from Blahut–Arimoto. The alternative average of $H_\infty(X \mid Y = y)$ is 0.6548 bits, above Smith's 0.5995, as Jensen's inequality predicts.

5.6. Shannon leakage versus min-entropy leakage

Shannon leakage, which is the mutual information $I(X;Y)$, and min-entropy leakage both measure how much an observation reveals about a secret, and the spam keyword already showed that they can disagree. The disagreement can be huge, and it comes from channels that reveal the secret completely on a few outcomes and nothing on the rest. Later parts state guarantees in one measure or the other, so you need to know which channels separate them.

Definition 5.7 (Two families of deterministic channels)**.** Let the secret X be uniform on the strings of w bits. A channel is **deterministic** when its output is a fixed function of its input. The channel **reveal-low(m)** always outputs the lowest m bits of X. The channel **reveal-if-zero(j)** outputs X itself if its lowest j bits are all zero, which happens with probability 2^{-j}, and a fixed symbol $\perp$ otherwise.

The first family reveals part of the secret every time, and the second reveals all of it, but rarely. For deterministic channels both leakages come down to counting outputs, which makes them easy to compare.

Proposition 5.4 (Leakages of the two families)**.** *For reveal-low(m), both leakages equal m bits. For reveal-if-zero(j) with $1 \le j \le w$,*

$$I(X;Y) = 2^{-j}w + (1 - 2^{-j}) \log_2 \frac{1}{1 - 2^{-j}}, \qquad \mathcal{L} = \log_2 (2^{w-j} + 1). \tag{5.31}$$

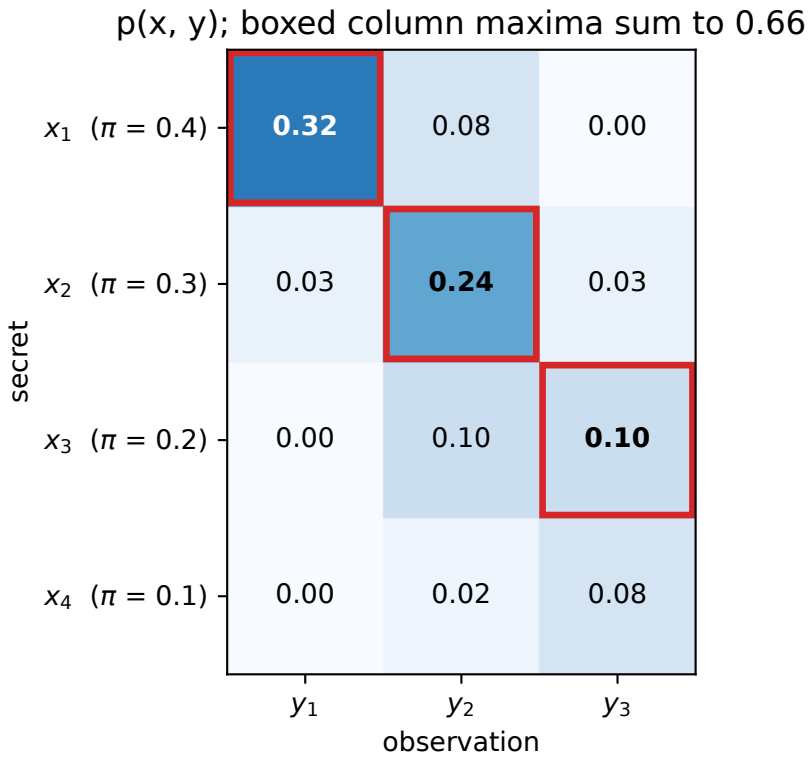


Figure 5.4.: The joint distribution $p(x, y) = \pi(x)K[x, y]$ of the walk-through. The MAP guesser picks the boxed cell in each column, and its success probability is their sum $V(X \mid Y) = 0.66$.

Proof. The plan is to reduce Shannon leakage to the entropy of the output and min-entropy leakage to the number of outputs, and then to count for each family.

1. *Shannon leakage is the output's entropy.* The output is fixed once the input is known, so $H(Y \mid X) = 0$, and by Theorem 5.3, $I(X;Y) = H(Y)$.
2. *Min-entropy leakage counts outputs.* Under the uniform prior every input has probability 2^{-w}, so every non-zero entry of the joint table is 2^{-w}. Each column maximum is then $V(X)$, and the ratio $V(X \mid Y)/V(X)$ is the number of distinct outputs.
3. *reveal-low(m).* The lowest m bits of a uniform string are uniform on 2^m values, so $H(Y) = m$, and there are 2^m distinct outputs.
4. *reveal-if-zero(j), the outputs.* There are 2^{w-j} strings whose lowest j bits are zero, each output as itself with probability 2^{-w}, and every other string gives $\perp$. So there are $2^{w-j} + 1$ distinct outputs, which gives $\mathcal{L}$.
5. *reveal-if-zero(j), the entropy.* The revealed outputs each have surprisal w and together have probability 2^{-j}, while $\perp$ has the remaining probability. Averaging the surprisals gives the stated $H(Y)$.

□

The script checks both closed forms against the explicit channel tables for $w = 6$, and they agree to 7.8×10^{-16}. Figure 5.5 plots the two leakages for $w = 64$, where the reveal-if-zero family at large j has almost no Shannon leakage and almost total min-entropy leakage.

The reason is where the logarithm sits. Shannon leakage averages the logarithm over outcomes, so an outcome of probability 2^{-j} that reveals w bits contributes only $2^{-j}w$. Min-entropy leakage takes the logarithm of an averaged probability, and the same outcome raises the guessing success from 2^{-w} to more than 2^{-j}, a gain of $w - j$ bits.

Example 5.6. On a 64-bit secret, compare reveal-if-zero(3), which outputs the whole secret one time in eight, with reveal-low(9). For reveal-if-zero(3), the Shannon leakage is $2^{-3} \cdot 64 + \frac{7}{8} \log_2 \frac{8}{7} = 8.1686$ bits, and the guesser's success is just above $\frac{1}{8}$, since it wins whenever the secret is revealed. For reveal-low(9), both leakages are 9 bits, and the guesser must still guess the other 55 bits. Table 5.4 puts these side by side.

Table 5.4.: Shannon and min-entropy leakage of two deterministic channels on a uniform 64-bit secret.

Channel	Shannon leakage	Min-entropy leakage	One-guess success
reveal-if-zero(3)	8.1686 bits	61.0000 bits	0.1250
reveal-low(9)	9.0000 bits	9.0000 bits	2.78×10^{-17}

By Shannon leakage, reveal-low(9) is the leakier channel. By one-guess success, reveal-if-zero(3) is worse by a factor of $0.125/(2.78 \times 10^{-17}) \approx 4.5 \times 10^{15}$. Smith (2009) used a pair of programs like these to argue that Shannon leakage is the wrong measure for one-try guessing.

The extreme case is $j = w$, a channel that reveals the secret only if it is all zeros. For a 4-bit secret the Shannon leakage is $h(1/16) = 0.3373$ bits, and the min-entropy leakage is exactly 1 bit, because the guesser wins on seeing 0000 and otherwise picks one of 15 values, so its success

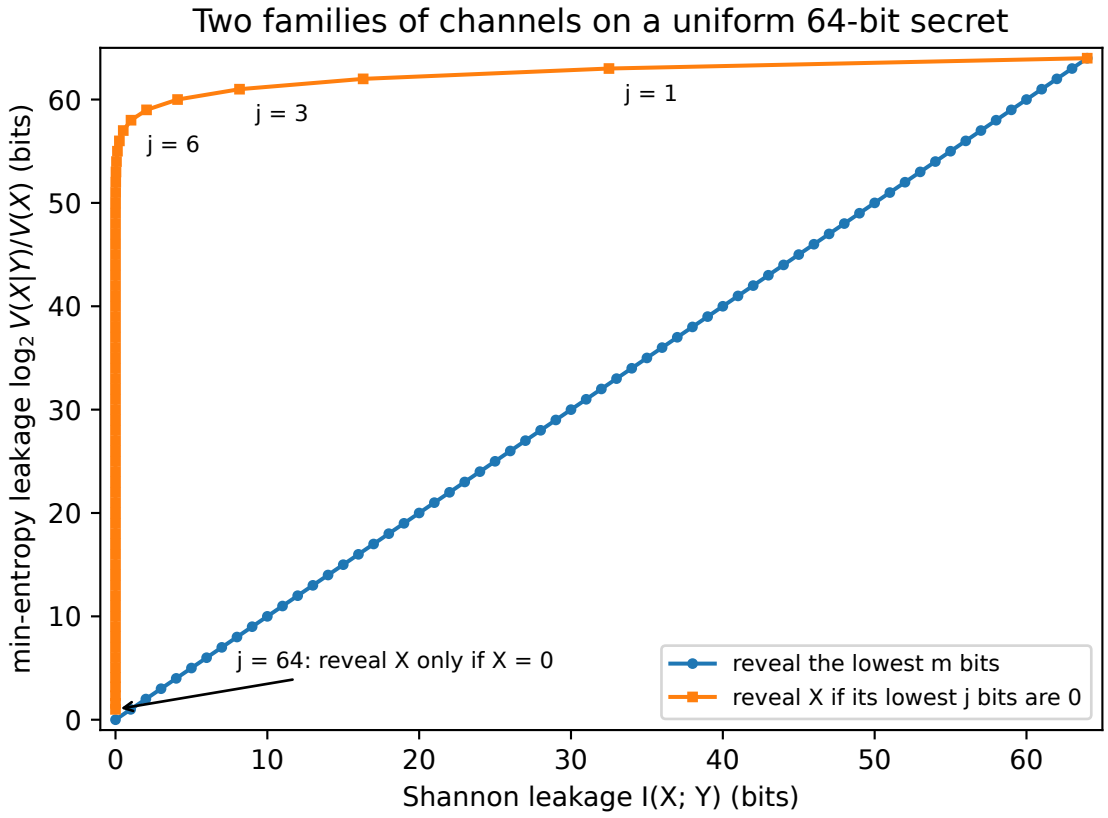


Figure 5.5.: Shannon leakage against min-entropy leakage for two families of deterministic channels on a uniform 64-bit secret. Revealing the low m bits gives equal leakages (diagonal). Revealing the whole secret with probability 2^{-j} gives a small Shannon leakage but a min-entropy leakage close to $64 - j$.

doubles to $\frac{1}{16} + \frac{15}{16} \cdot \frac{1}{15}$. For a 32-bit secret the Shannon leakage falls to 7.786×10^{-9} bits, while the min-entropy leakage stays at 1 bit. Both numbers are small next to the 4 or 32 bits of the secret, and both miss that for the one secret value 0000 the channel reveals everything. Any measure that averages over the secret can hide a fully exposed value, so a worst-case question needs a worst-case measure, which later parts build.

5.7. Fano's inequality

Mutual information also gives **lower bounds**, statements that *no* estimator can do better than some error rate, which you need to show that a problem is hard rather than that a method is good. Fano's inequality is the basic tool. It says that if X is still uncertain after seeing Y, every guess of X from Y must often be wrong. The reason is that describing X once the guess is known takes only two pieces: whether the guess was right, and, when it was wrong, which other value X is.

Theorem 5.7 (Fano's inequality)**.** *Let X take values in a finite set of k elements, and let $\hat{X}$ be any estimate of X computed from Y, possibly with extra randomness independent of X and Y. Write P_e for its error probability,*

$$P_e = \Pr[\hat{X} \neq X]. \tag{5.32}$$

Then

$$H(X \mid Y) \leq h(P_e) + P_e \log_2(k-1), \tag{5.33}$$

and as a consequence

$$P_e \geq \frac{H(X \mid Y) - 1}{\log_2 k}. \tag{5.34}$$

Proof. The plan is to expand the uncertainty of the error together with X in two orders, bound the pieces of one order, carry the bound from $\hat{X}$ to Y, and weaken it. Write $E = \mathbf{1}[\hat{X} \neq X]$ for the error indicator, which is 1 when the guess is wrong.

1. *Expand in two orders.* Every entropy in steps 1 to 4 is conditioned on $\hat{X}$. The chain rule Equation 5.7 holds given each value of $\hat{X}$, so it also holds for the average over $\hat{X}$, and revealing X first or E first gives

$$H(X \mid \hat{X}) + H(E \mid X, \hat{X}) = H(E, X \mid \hat{X}) = H(E \mid \hat{X}) + H(X \mid E, \hat{X}). \tag{5.35}$$

   Given both X and $\hat{X}$ you know whether they differ, so $H(E \mid X, \hat{X}) = 0$.
2. *Bound the error's uncertainty.* By Corollary 5.1, $H(E \mid \hat{X}) \leq H(E)$, which is $h(P_e)$ because E is 1 with probability P_e.
3. *Bound what remains of X.* If $E = 0$, then $X = \hat{X}$ and no uncertainty is left. If $E = 1$, then X is one of the $k-1$ other values, so its entropy is at most $\log_2(k-1)$ by Theorem 5.1. Averaging over E bounds $H(X \mid E, \hat{X})$ by $P_e \log_2(k-1)$.
4. *Combine.* Steps 1 to 3 give

$$H(X \mid \hat{X}) \leq h(P_e) + P_e \log_2(k-1). \tag{5.36}$$

5. *Replace $\hat{X}$ by Y.* The estimate is computed from Y, so $X \to Y \to \hat{X}$ is a Markov chain, and Theorem 5.4 gives $I(X;\hat{X}) \le I(X;Y)$. Writing both sides as $H(X)$ minus a conditional entropy turns this into $H(X \mid Y) \le H(X \mid \hat{X})$, and the bound of step 4 carries over.
6. *Weaken.* Using $h(P_e) \le 1$ and $\log_2(k-1) \le \log_2 k$ gives $H(X \mid Y) \le 1 + P_e \log_2 k$, and solving for P_e gives Equation 5.34.

□

The right side of Equation 5.33 increases with P_e on $[0, (k-1)/k]$, the range from a perfect estimator to blind guessing, because its derivative is positive there:

$$\frac{\mathrm{d}}{\mathrm{d}P_e}\Big(h(P_e) + P_e \log_2(k-1)\Big) = \log_2 \frac{(1-P_e)(k-1)}{P_e} > 0. \tag{5.37}$$

So the **Fano bound**, the smallest P_e that satisfies it, can be found by bisection. The bound is tight when step 3 is, that is when the true value after a wrong guess is uniform over the other values. The k-ary **symmetric channel** keeps its input with probability $1-p$ and otherwise outputs one of the other symbols uniformly. It has exactly this structure, and Fano's inequality is an equality for it (Figure 5.6, left).

If errors go to a few wrong values, the bound is loose, because little uncertainty is left after a wrong guess. The **shift channel** outputs $x+1 \bmod k$ instead of x with probability p, and given the output the input is one of only two values. With uniform X, $k = 8$ and $p = 0.2$, both channels have MAP error 0.2000, because the best guess is the input that keeps the output unchanged, which is wrong with probability p. The Fano bound matches it for the symmetric channel but is only 0.0954 for the shift channel (Figure 5.6, right). The weak form Equation 5.34 gives 0.0945 for the symmetric channel and 0 for the shift channel, whose $H(X \mid Y)$ is below 1.

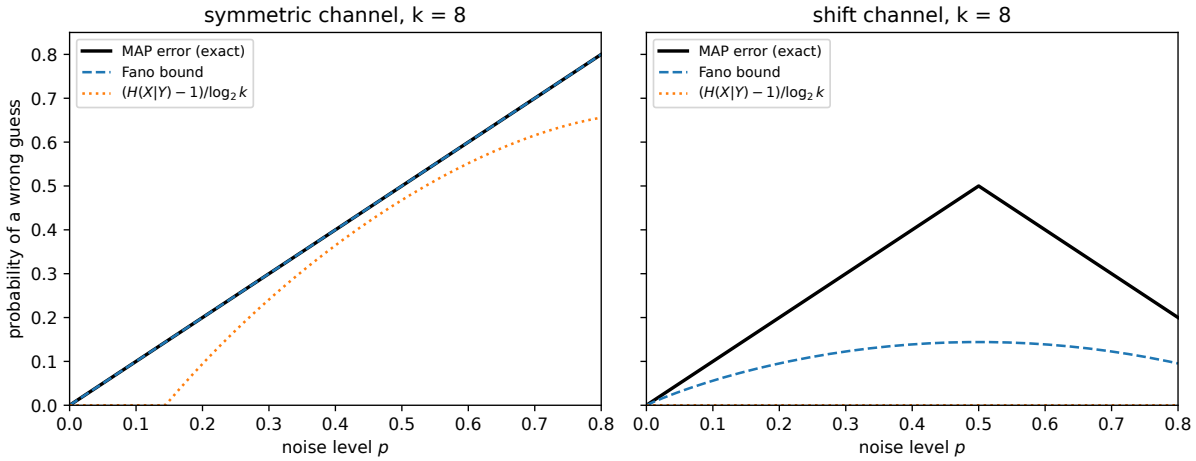


Figure 5.6.: MAP error against Fano's lower bound, for $k = 8$ and a uniform X, as the noise level p grows. Left: on the symmetric channel the bound is exact. Right: on the shift channel, where errors go to a single wrong value, the bound is loose and the weak form is zero.

The same idea shows that estimating a parameter needs many samples, by turning estimation into identification. Pick candidate parameter values far apart, so that estimating the parameter

well would identify the candidate, yet with sample distributions close together. If the samples carry little information about which candidate produced them, no estimator can identify it.

Proposition 5.5 (Fano's method)**.** *Let $\theta_1, \dots, \theta_k$ be candidate parameter values, with distributions P_{θ_x} for the samples. Let the index X be uniform on $\{1, \dots, k\}$, and given X let the n samples $Y^n = (Y_1, \dots, Y_n)$ be independent draws from P_{θ_X}. Suppose that a number κ bounds the divergence between any two candidates,*

$$D_{\mathrm{KL}}(P_{\theta_x} \,\|\, P_{\theta_{x'}}) \le \kappa \quad \text{bits for all } x, x'. \tag{5.38}$$

Then every estimator of X from Y^n has error probability

$$P_e \ge 1 - \frac{n\kappa + 1}{\log_2 k}. \tag{5.39}$$

Proof. The plan is to bound the information in n samples by n times the information in one, to bound the information in one sample by κ, and to put the result into the weak form of Fano's inequality.

1. *Information adds at most over samples.* Equation 5.7, applied one sample at a time, splits $H(Y^n \mid X)$ into the terms $H(Y_i \mid X, Y_1, \dots, Y_{i-1})$ for $i = 1, \dots, n$. Given X the samples are independent, so each term is $H(Y_i \mid X)$. Without X, the chain rule splits $H(Y^n)$ the same way, and Corollary 5.1 bounds each term only by $H(Y_i)$. Subtracting the first from the second, and using that all samples have the same distribution,
$$I(X; Y^n) \le \sum_i I(X; Y_i) = n\, I(X; Y_1). \tag{5.40}$$
2. *Write one sample's information as an average divergence.* Let $\bar{P} = \frac{1}{k}\sum_x P_{\theta_x}$ be the average of the candidate distributions, which is the distribution of Y_1 when the index is random. With $p(x, y) = P_{\theta_x}(y)/k$ and $p(y) = \bar{P}(y)$, Equation 5.10 reads
$$I(X; Y_1) = \frac{1}{k}\sum_x D_{\mathrm{KL}}(P_{\theta_x} \,\|\, \bar{P}). \tag{5.41}$$
3. *Bound it by κ.* For any distribution Q, and later for $Q = P_{\theta_{x'}}$, splitting $\log_2 \frac{P_{\theta_x}}{Q} = \log_2 \frac{P_{\theta_x}}{\bar{P}} + \log_2 \frac{\bar{P}}{Q}$ and averaging over x gives
$$\frac{1}{k}\sum_x D_{\mathrm{KL}}(P_{\theta_x} \,\|\, Q) = \frac{1}{k}\sum_x D_{\mathrm{KL}}(P_{\theta_x} \,\|\, \bar{P}) + D_{\mathrm{KL}}(\bar{P} \,\|\, Q), \tag{5.42}$$
and the last term is non-negative. So the average divergence to $\bar{P}$ is at most the average divergence to any single candidate $P_{\theta_{x'}}$, which is at most κ.
4. *Apply Fano.* The index is uniform, so $H(X) = \log_2 k$, and steps 1 to 3 give $H(X \mid Y^n) = \log_2 k - I(X; Y^n) \ge \log_2 k - n\kappa$. Putting this into Equation 5.34 gives the bound.

□

The bound fades as $n\kappa$ approaches $\log_2 k$, the number of bits needed to name the candidate. Choosing the candidates is the art of the method, and Wainwright (2019) works through many cases.

Example 5.7. A label is uniform over $k = 8$ classes, so it has 3 bits of entropy, and the features carry at most 1 bit about it. Then $H(X \mid Y) \geq 2$ bits, so every classifier, however it is trained, errs at least as often as the P_e that solves $h(P_e) + P_e \log_2 7 = 2$, which is 0.3730. The weak form gives $(2 - 1)/3 = 0.3333$.

For estimation, take $k = 1024$ candidates, so $\log_2 k = 10$ bits, within $\kappa = 0.01$ bits of each other. Then $n = 100$ samples leave $P_e \geq 1 - (1 + 1)/10 = 0.8000$, and even 400 samples leave $P_e \geq 1 - (4 + 1)/10 = 0.5000$.

5.8. Code

The script code/part01-prerequisites/information_theory.py computes every number quoted in this chapter and draws its figures. The script computes mutual information in two ways, checks the capacity formula with Blahut–Arimoto and the posterior vulnerability with a simulated guesser, and checks the leakage formulas against explicit channel tables. From the repository root:

```
uv run python code/part01-prerequisites/information_theory.py
```

Table 5.5.: Output of the chapter's script with seed 0; only the guessing simulation uses the seed. All quantities are in bits except probabilities.

Experiment	Result
Spam keyword	$H(X) = 0.5436$, $H(X \mid Y = 1) = 1.0000$, $H(X \mid Y) = 0.2500$, $I(X;Y) = 0.2936$ both ways; $I(X;Z) = 0.1668$ through BSC(0.1)
PIN, 10% choose 1234	$H = 12.4278$, $H_\infty = 3.3219$ (uniform: 13.2877)
BSC(0.11)	$1 - h(p) = 0.5001$; Blahut–Arimoto 0.5001 at $\Pr[X = 1] = 0.5000$; min-capacity 0.8319
Guessing walk-through	$V(X) = 0.4000$, $V(X \mid Y) = 0.6600$, simulated 0.6594; $\mathcal{L} = 0.7225$; $\mathcal{ML} = 1.2630$; Shannon capacity 0.8269
Reveal-if-zero(3) vs reveal-low(9), $w = 64$	Shannon 8.1686 vs 9.0000; min-entropy 61.0000 vs 9.0000
Fano, $k = 8$, $p = 0.2$	Symmetric: bound 0.2000, MAP 0.2000. Shift: bound 0.0954, MAP 0.2000

tests/test_information_theory.py checks that the output still matches.

5.9. Summary

Table 5.6.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 5.1	$H(X) = -\sum_x p(x) \log_2 p(x)$	Average uncertainty in bits
Theorem 5.1	$0 \le H(X) \le \log_2 \lvert\mathcal{X}\rvert$	The scale of every other quantity
Theorem 5.2	$H(X,Y) = H(Y) + H(X \mid Y)$	Splitting uncertainty into pieces
Theorem 5.3	$I(X;Y) = D_{\mathrm{KL}}(P_{XY} \Vert P_X P_Y)$	Information gain; symmetric, zero iff independent
Corollary 5.1	$H(X \mid Y) \le H(X)$	On average only: one observation can raise it
Proposition 5.1	$I(X;Y,Z) = I(X;Y) + I(X;Z \mid Y)$	Information gathered in stages
Theorem 5.4	$I(X;Z) \le I(X;Y)$ if $X \to Y \to Z$	Processing cannot add information
Theorem 5.5	$\mathcal{C}_{\mathrm{BSC}}(p) = 1 - h(p)$	Other channels need Blahut–Arimoto
Proposition 5.2	$H_\infty(X) \le H(X)$	Password strength; a high H can hide a guessable value
Proposition 5.3	$V(X \mid Y) = \sum_y \max_x p(x,y)$	Success of the best guesser
Theorem 5.6	$\mathcal{ML}(K) = \log_2 \sum_y \max_x K[x,y]$	Leakage bound for every prior
Proposition 5.4	Rare full reveals leak little Shannon information	The two leakages can rank channels oppositely; neither sees one exposed value
Theorem 5.7	$H(X \mid Y) \le h(P_e) + P_e \log_2(k-1)$	No estimator does better; tight when errors spread evenly
Proposition 5.5	$P_e \ge 1 - (n\kappa + 1)/\log_2 k$	Bound the information in the data, then apply Fano

5.10. References

- Cover & Thomas, *Elements of Information Theory* (2nd ed., 2006), ch. 2 (entropy, mutual information, data processing, Fano) and ch. 7 (channel capacity and the coding theorem): the standard textbook for every Shannon result here.
- MacKay, *Information Theory, Inference, and Learning Algorithms* (2003), ch. 2 and 8–10: the same material with a machine-learning slant, free online.
- Smith, "On the Foundations of Quantitative Information Flow" (FoSSaCS 2009): vulnerability, conditional min-entropy, and the argument against Shannon leakage for one-try guessing.
- Braun, Chatzikokolakis & Palamidessi, "Quantitative Notions of Leakage for One-try Attacks" (MFPS 2009): min-capacity and the fact that the uniform prior attains it.

- Alvim, Chatzikokolakis, McIver, Morgan, Palamidessi & Smith, *The Science of Quantitative Information Flow* (2020): the full theory of vulnerability, leakage and capacity.
- Wainwright, *High-Dimensional Statistics* (2019), ch. 15: minimax lower bounds, including Fano's method.
- Yu, "Assouad, Fano, and Le Cam" (1997): the three classical lower-bound methods side by side.

6. Optimization

Training a model means minimizing an average loss over the training set, and two questions come up every time. The first is **how fast the optimizer gets there**: how many steps gradient descent (GD) and stochastic gradient descent (SGD) need, and which learning rate to pick. SGD estimates the gradient from a few random records, so its steps are noisy and it needs more of them. The second is **how much the answer depends on any one training record**, which the next chapter turns into a guarantee that the model works on new data.

Both questions have the same kind of answer, because **a few constants of the loss decide everything**: how steep it can be, how sharply it can curve at most, how much it must curve at least, and how noisy SGD's gradient estimates are. Every rate and bound in this chapter combines these four.

The running example is logistic regression, a linear classifier, trained on 1,000 records with five features, and each result of the chapter is checked on it. For example, after 10,000 steps of SGD its loss is within 7.71×10^{-4} of the minimum, while the guarantee of Theorem 6.2 only promises 0.050, and the chapter explains why the guarantee is so cautious. Table 6.1 lists the symbols.

Table 6.1.: The symbols of this chapter.

Symbol	Meaning
$D = (z_1, \dots, z_n)$; $\theta \in \mathbb{R}^d$; $\ell(\theta; z)$	n training records; the parameters; the loss on record z
$\mathcal{L}(\theta)$, or $\mathcal{L}(\theta; D)$; θ^*, $\mathcal{L}^*$	the empirical risk $\frac{1}{n}\sum_i \ell(\theta; z_i)$; its minimizer and minimum
L, β, λ; $\kappa = \beta/\lambda$	Lipschitz, smoothness, strong convexity constants; condition number
η, η_t; T	learning rate (step size); number of steps
g_t; G^2; σ^2	random gradient at step t; bound on its mean squared length; its variance
$\mathcal{W}$, Π; R	set SGD stays in, projection onto it; bound on the starting distance to θ^*, or the radius of a ball $\mathcal{W}$ around 0

6.1. Convexity, Lipschitz continuity and smoothness

Every guarantee below rests on assumptions about the loss, and four properties are enough. Lipschitz continuity bounds the slope and smoothness the curvature from above, while convexity

and strong convexity bound the curvature from below. This section defines each, relates them, and checks them on the logistic loss.

Definition 6.1 (Convex function)**.** A function $f\colon \mathbb{R}^d \to \mathbb{R}$ is **convex** when the straight segment (the chord) between any two points of its graph never dips below the graph. That is, for all θ, θ' and every mixing weight $s \in [0,1]$,

$$f(s\theta' + (1-s)\theta) \le sf(\theta') + (1-s)f(\theta). \tag{6.1}$$

The proofs use a second picture, through tangents. The tangent at θ predicts the value at θ' by adding to $f(\theta)$ the inner product $\langle \nabla f(\theta), \theta' - \theta \rangle$ of the gradient with the step, and for a convex function this prediction never overshoots.

Lemma 6.1 (A convex function lies above its tangents)**.** *If f is convex and differentiable, then for all θ and θ'*

$$f(\theta') \ge f(\theta) + \langle \nabla f(\theta), \theta' - \theta \rangle. \tag{6.2}$$

Proof.

1. *Shrink the chord to a short step.* Write the mixed point as $\theta + s(\theta' - \theta)$, a step of size s towards θ', and the chord's height there as $f(\theta) + s(f(\theta') - f(\theta))$. Subtracting $f(\theta)$ from both sides of the chord condition and dividing by $s > 0$ gives

$$\frac{f(\theta + s(\theta' - \theta)) - f(\theta)}{s} \le f(\theta') - f(\theta). \tag{6.3}$$

2. *Let the step vanish.* The left side is the average slope over a short step towards θ'. As $s \to 0$ it becomes the slope in that direction, which for a differentiable f is the inner product $\langle \nabla f(\theta), \theta' - \theta \rangle$, and this gives Equation 6.2.

□

Every convex convergence proof below uses the lemma with $\theta' = \theta^*$. Moving the inner product to the other side and flipping its sign, it says that $\langle \nabla f(\theta), \theta - \theta^* \rangle$ is at least the current error $f(\theta) - f(\theta^*)$. So the gradient points away from the minimizer, and the further θ is from optimal, the more a small step against the gradient shortens the distance to θ^*. Section G.1 derives the converse, from tangents to chords.

Definition 6.2 (Lipschitz, smooth and strongly convex functions)**.** Let $f\colon \mathbb{R}^d \to \mathbb{R}$ be differentiable.

- f is **L-Lipschitz** when it never changes faster than L per unit of distance: $|f(\theta) - f(\theta')| \le L\|\theta - \theta'\|$.
- f is **β-smooth** when its gradient is β-Lipschitz: $\|\nabla f(\theta) - \nabla f(\theta')\| \le \beta\|\theta - \theta'\|$.
- f is **λ-strongly convex**, for some $\lambda > 0$, when it lies above each tangent by at least a bowl of curvature λ:

$$f(\theta') \ge f(\theta) + \langle \nabla f(\theta), \theta' - \theta \rangle + \frac{\lambda}{2}\|\theta' - \theta\|^2. \tag{6.4}$$

For a function that is both smooth and strongly convex, $\kappa = \beta/\lambda$ is its **condition number**, which Section 6.2 shows sets how slowly gradient descent converges.

The next result collects the working forms of these definitions, with a test of smoothness by the **Hessian** $\nabla^2 f$, the matrix of second derivatives, whose eigenvalues are the curvatures of f. The proof bounds inner products by the Cauchy–Schwarz inequality $|\langle a, b\rangle| \le \|a\|\,\|b\|$: an inner product is the product of the lengths times a cosine.

Proposition 6.1 (Working forms of the constants)**.** *Let $f\colon \mathbb{R}^d \to \mathbb{R}$ be differentiable.*

(a) f is L-Lipschitz exactly when $\|\nabla f(\theta)\| \le L$ for every θ.

*(b) If f is β-smooth, the error of its tangent grows at most like a parabola of curvature β (the **descent lemma**):*

$$f(\theta') \le f(\theta) + \langle \nabla f(\theta), \theta' - \theta\rangle + \frac{\beta}{2}\|\theta' - \theta\|^2. \tag{6.5}$$

(c) If f is twice differentiable and every eigenvalue of $\nabla^2 f$ lies in $[-\beta, \beta]$ everywhere, then f is β-smooth.

*(d) If f is λ-strongly convex, its gradient is **strongly monotone**:*

$$\langle \nabla f(\theta) - \nabla f(\theta'), \theta - \theta'\rangle \ge \lambda\|\theta - \theta'\|^2. \tag{6.6}$$

Proof. Steps 1 to 4 walk along the segment $\theta_s = \theta + s(\theta' - \theta)$, $s \in [0, 1]$, and add up the changes by the fundamental theorem of calculus. Step 5 adds two inequalities.

1. *A bounded gradient makes f Lipschitz.* The change of f is the integral of the slope in the direction of travel,
$$f(\theta') - f(\theta) = \int_0^1 \langle \nabla f(\theta_s), \theta' - \theta\rangle \,\mathrm{d}s, \tag{6.7}$$
and by Cauchy–Schwarz each inner product, and so the integral, is at most $L\|\theta' - \theta\|$ in size.
2. *A Lipschitz f has a bounded gradient.* Where the gradient is not zero, take the unit vector $u = \nabla f(\theta)/\|\nabla f(\theta)\|$. The Lipschitz condition says $f(\theta + hu) - f(\theta) \le Lh$, and divided by h the left side tends to $\langle \nabla f(\theta), u\rangle = \|\nabla f(\theta)\|$ as $h \to 0$, so $\|\nabla f(\theta)\| \le L$.
3. *Smoothness gives the descent lemma.* The error of the tangent is the integral of how much the slope has changed:
$$f(\theta') - f(\theta) - \langle \nabla f(\theta), \theta' - \theta\rangle = \int_0^1 \langle \nabla f(\theta_s) - \nabla f(\theta), \theta' - \theta\rangle \,\mathrm{d}s. \tag{6.8}$$
At position s you are a distance $s\|\theta' - \theta\|$ from θ, so by smoothness and Cauchy–Schwarz the integrand is at most $\beta s\|\theta' - \theta\|^2$. Since $\int_0^1 \beta s \,\mathrm{d}s = \beta/2$, the error is at most $\frac{\beta}{2}\|\theta' - \theta\|^2$.
4. *A bounded Hessian gives smoothness.* The change of the gradient is the integral of the Hessian applied to the direction of travel,
$$\nabla f(\theta') - \nabla f(\theta) = \int_0^1 \nabla^2 f(\theta_s)(\theta' - \theta) \,\mathrm{d}s. \tag{6.9}$$
A symmetric matrix with eigenvalues in $[-\beta, \beta]$ stretches no vector by more than β, and an integral is no longer than the integral of the lengths, so it has length at most $\beta\|\theta' - \theta\|$.

5. *Strong convexity gives strong monotonicity.* Write the strong convexity inequality from θ to θ' and from θ' to θ, and add the two. The function values cancel and the two bowl terms add up to $\lambda\|\theta - \theta'\|^2$, which leaves

$$0 \geq \langle \nabla f(\theta) - \nabla f(\theta'), \theta' - \theta \rangle + \lambda\|\theta - \theta'\|^2, \tag{6.10}$$

and moving the inner product to the left gives Equation 6.6.

□

The converses hold too, for twice-differentiable f: a β-smooth f has no Hessian eigenvalue above β, and f is λ-strongly convex exactly when every eigenvalue of $\nabla^2 f$ is at least λ (Nesterov 2018, §2.1; Section G.1). A Hessian with no negative eigenvalue likewise makes f convex. So the two constants sandwich the curvature, and $\kappa \geq 1$.

Two consequences of strong convexity are used later, the first in Section 6.7 and the second whenever a ridge term $\frac{\lambda}{2}\|\theta\|^2$ is added.

Corollary 6.1 (Two consequences of strong convexity)**.** *Let $\lambda > 0$.*

(a) If f is differentiable and λ-strongly convex with a minimizer θ^*, then for every θ

$$\|\theta - \theta^*\| \leq \frac{\|\nabla f(\theta)\|}{\lambda}, \tag{6.11}$$

and in particular the minimizer is unique.

(b) If f is convex and differentiable, then $f(\theta) + \frac{\lambda}{2}\|\theta\|^2$ is λ-strongly convex.

Proof.

1. *Part (a).* The gradient of a differentiable function on all of $\mathbb{R}^d$ vanishes at its minimizer. Put $\theta' = \theta^*$ in Equation 6.6 and bound its left side by Cauchy–Schwarz:

$$\lambda\|\theta - \theta^*\|^2 \leq \|\nabla f(\theta)\|\,\|\theta - \theta^*\|. \tag{6.12}$$

Divide by $\lambda\|\theta - \theta^*\|$, or note that there is nothing to prove when $\theta = \theta^*$. A second point with zero gradient would be at distance 0 from θ^*, so the minimizer is unique.

2. *Part (b).* Expanding a square gives $\|\theta'\|^2 = \|\theta\|^2 + 2\langle\theta, \theta' - \theta\rangle + \|\theta' - \theta\|^2$. Multiplied by $\frac{\lambda}{2}$, this says that $\frac{\lambda}{2}\|\theta\|^2$, whose gradient is $\lambda\theta$, meets the strong convexity inequality with equality, term by term. Adding it to the tangent inequality Equation 6.2 of f gives that inequality for the sum.

□

Neither of Lipschitz and smooth implies the other: $|\theta|$ is 1-Lipschitz but not smooth, and $\theta^2/2$ is 1-smooth but not Lipschitz (Figure 6.1). By part (a) of Corollary 6.1 the gradient of a strongly convex function grows at least like λ times the distance to the minimizer, so **no strongly convex function is Lipschitz on all of** $\mathbb{R}^d$, and the strongly convex results below stay in a bounded set by the projection defined in Section 6.2.

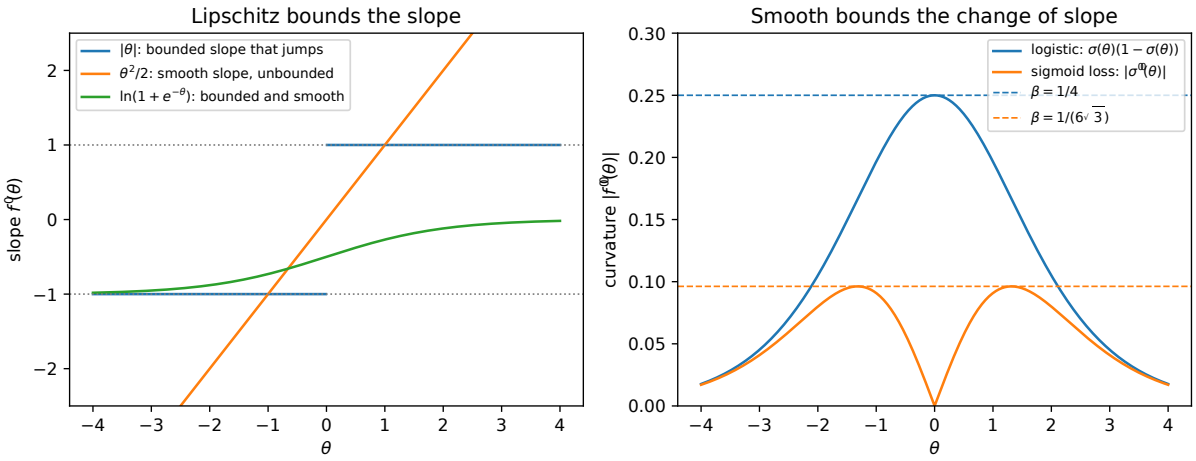


Figure 6.1.: Left: the slopes of $|\theta|$ (bounded, jumps), $\theta^2/2$ (unbounded) and the logistic loss (bounded, continuous). Right: the curvatures of the logistic loss and of the sigmoid loss of Section 6.6.

Example 6.1. In the running example, features have $\|x\| \le 1$ and labels $y \in \{-1, +1\}$. The **margin** $m = y\langle\theta, x\rangle$ is positive when a record is classified correctly, and the logistic loss is $\ln(1 + e^{-m})$. Write $\mathrm{sig}(m) = 1/(1 + e^{-m})$ for the logistic function.

The derivative of $\ln(1+e^{-m})$ in m is $-\mathrm{sig}(-m)$, so by the chain rule the gradient is $-\mathrm{sig}(-m)\, y\, x$, of length at most 1, so the loss is 1-Lipschitz by part (a) of Proposition 6.1. Differentiating again, with $\mathrm{sig}'(-m) = \mathrm{sig}(m)(1 - \mathrm{sig}(m))$, the Hessian is $\mathrm{sig}(m)(1 - \mathrm{sig}(m))\, xx^\top$. Here $xx^\top$ has eigenvalue $\|x\|^2 \le 1$ along x and 0 across it, and the factor in front, $p(1-p)$ with p in $(0, 1)$, is at most $\frac{1}{4}$. So the loss is convex and $\frac{1}{4}$-smooth by part (c), but not strongly convex, because its curvature is 0 across x and tends to 0 as the margin grows.

For $x = (0.6, 0.8)$ and $y = +1$, the loss, gradient norm and curvature are 0.6931, 0.5000 and 0.2500 at $\theta = 0$, but 0.0067, 0.0067 and 0.0066 at $\theta = (3, 4)$, where the margin is 5. Confidently classified records add almost no curvature, and at the minimizer most records are classified confidently. So on the running dataset, with features drawn on the unit sphere and labels from a logistic model, the Hessian of the average loss at θ^* has smallest eigenvalue only 0.0233.

6.2. Gradient descent and SGD

Gradient descent steps against the full gradient, which costs one gradient evaluation per record. SGD uses the gradient of one random record, or a small batch, which is cheap and right on average but noisy. This section defines both and shows on a quadratic what a fast rate looks like without noise, so that you can see later what the noise costs.

Definition 6.3 (Gradient descent and projected SGD)**. Gradient descent** with learning rate η takes the step $\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t)$.

Projected SGD runs inside a convex set $\mathcal{W}$, such as a ball, that is closed (contains its boundary), and its **projection** $\Pi(u)$ is the point of $\mathcal{W}$ nearest to u. At step t the algorithm draws an index

i_t uniformly from $\{1, \dots, n\}$, takes that record's gradient $g_t = \nabla \ell(\theta_t; z_{i_t})$, and updates

$$\theta_{t+1} = \Pi(\theta_t - \eta_t g_t). \tag{6.13}$$

With a **minibatch** of B indices, g_t is the average gradient over the batch. Plain SGD is the case $\mathcal{W} = \mathbb{R}^d$.

With one record or a batch, the estimate is right on average. Write "past" for the random choices made before step t, which fix θ_t; a uniform index averages to the full gradient, so

$$\mathbb{E}[g_t \mid \text{past}] = \nabla \mathcal{L}(\theta_t). \tag{6.14}$$

On a quadratic risk the gradient is linear in the error $\theta - \theta^*$, so each GD step multiplies the error by a fixed matrix, and the next theorem reads the rate off its eigenvalues.

Theorem 6.1 (Gradient descent on a strongly convex quadratic)**.** *Let $\mathcal{L}(\theta) = \mathcal{L}^* + \frac{1}{2}(\theta - \theta^*)^\top H(\theta - \theta^*)$, where the symmetric matrix H has eigenvalues in $[\lambda, \beta]$ with $0 < \lambda \le \beta$, and let $\kappa = \beta/\lambda$. Then GD with $\eta = 1/\beta$ from any θ_0 satisfies*

$$\|\theta_t - \theta^*\| \le \left(1 - \frac{1}{\kappa}\right)^t \|\theta_0 - \theta^*\|. \tag{6.15}$$

Proof.

1. *One step multiplies the error by a matrix.* The gradient is $H(\theta - \theta^*)$, so subtracting θ^* from both sides of the update gives
$$\theta_{t+1} - \theta^* = (I - H/\beta)(\theta_t - \theta^*). \tag{6.16}$$
2. *Find the factor in each direction.* The matrix $I - H/\beta$ has the principal directions (eigenvectors) of H, and turns a curvature $h \in [\lambda, \beta]$ into the factor $1 - h/\beta \in [0, 1 - 1/\kappa]$.
3. *Bound the shrinkage, then repeat.* In the orthonormal basis of principal directions, each coordinate of the error is multiplied by its own factor, so the length is multiplied by at most $1 - 1/\kappa$. Applying this t times gives Equation 6.15.

□

For any β-smooth risk, convex or not, a weaker statement holds, about the risk's value rather than the distance, and Section 6.6 starts from it. The descent lemma puts the risk below a parabola that touches it at θ_t, and a step of $1/\beta$ lands at that parabola's lowest point, so the risk falls at least as much as the parabola does.

Proposition 6.2 (One gradient step lowers a smooth function)**.** *If $\mathcal{L}$ is β-smooth, one GD step with $\eta = 1/\beta$ satisfies*

$$\mathcal{L}(\theta_{t+1}) \le \mathcal{L}(\theta_t) - \frac{1}{2\beta}\|\nabla \mathcal{L}(\theta_t)\|^2. \tag{6.17}$$

Proof.

1. *Put the step into the descent lemma.* With $\theta_{t+1} - \theta_t = -\nabla\mathcal{L}(\theta_t)/\beta$, the tangent term of Equation 6.5 is $-\frac{1}{\beta}\|\nabla\mathcal{L}(\theta_t)\|^2$, the progress. The parabola term is $\frac{\beta}{2} \cdot \frac{1}{\beta^2}\|\nabla\mathcal{L}(\theta_t)\|^2$, the price of overshooting.
2. *Add them.* Their sum is $-\frac{1}{2\beta}\|\nabla\mathcal{L}(\theta_t)\|^2$: the overshoot eats half of the progress.

□

Theorem 6.1 says that GD converges **linearly**, by a constant factor per step, and Section G.1 extends Equation 6.15 to every twice-differentiable strongly convex smooth risk. Since $1-x \le e^{-x}$, the factor after t steps is at most $e^{-t/\kappa}$, so shrinking the error to a fraction τ takes about $\kappa \ln(1/\tau)$ steps. A large κ makes GD slow, because the step must stay small for the steepest direction, where the error is multiplied by $1 - \eta\beta$, which must stay between -1 and 1.

SGD with a constant step cannot do this: near θ^* the gradients of single records do not vanish, so its iterates keep jumping around θ^*.

Example 6.2. The logistic risk is not a quadratic, so this example uses least squares, whose risk $\frac{1}{2n}\sum_i (y_i - \langle\theta, x_i\rangle)^2$ is a quadratic with H the average of the $x_i x_i^\top$. The script draws 200 records with $x \sim \mathcal{N}(0, \mathrm{diag}(1, 0.1))$ and $y = x_1 + x_2$ plus noise, and H has eigenvalues $\lambda = 0.101$ and $\beta = 0.969$, so $\kappa = 9.58$.

From $\theta_0 = (-1, 3)$, Equation 6.15 predicts that GD needs at most 126 steps to shrink $\|\theta - \theta^*\|$ by 10^6, since $(1 - 1/9.58)^t \le 10^{-6}$ needs

$$t \ge \frac{\ln(10^6)}{-\ln(1 - 1/9.58)} = \frac{13.8155}{0.11024} = 125.3. \tag{6.18}$$

GD takes 122, close to the bound, because the starting error lies largely along the flat direction, which shrinks at exactly the worst-case rate. GD crawls along the flat valley while SGD jitters around θ^* (Figure 6.2).

6.3. SGD for convex Lipschitz objectives

The cleanest guarantee for SGD needs only convexity and bounded gradients, so it covers the logistic loss and, with the subgradients defined after the proof, losses with kinks. Its proof, the template for the rest of the chapter, follows the squared distance to the minimizer. It needs two tools first: the projection never undoes progress, and a random gradient behaves like the true one in the cross term $\langle g_t, \theta_t - \theta^*\rangle$ but not in its squared length.

The first tool is used where the new point is pulled back into $\mathcal{W}$. Seen from $\Pi(u)$, the directions to u and to any θ in the set make an angle of at least 90°, so, as in a right-angled triangle, u is farther from θ than $\Pi(u)$ is.

Lemma 6.2 (Projection never moves a point away from the points of the set)**.** *Let $\mathcal{W} \subseteq \mathbb{R}^d$ be convex and closed, so that a nearest point exists, and let $\Pi(u)$ be its point nearest to u. Then for every u and every $\theta \in \mathcal{W}$,*

$$\|\Pi(u) - \theta\| \le \|u - \theta\|. \tag{6.19}$$

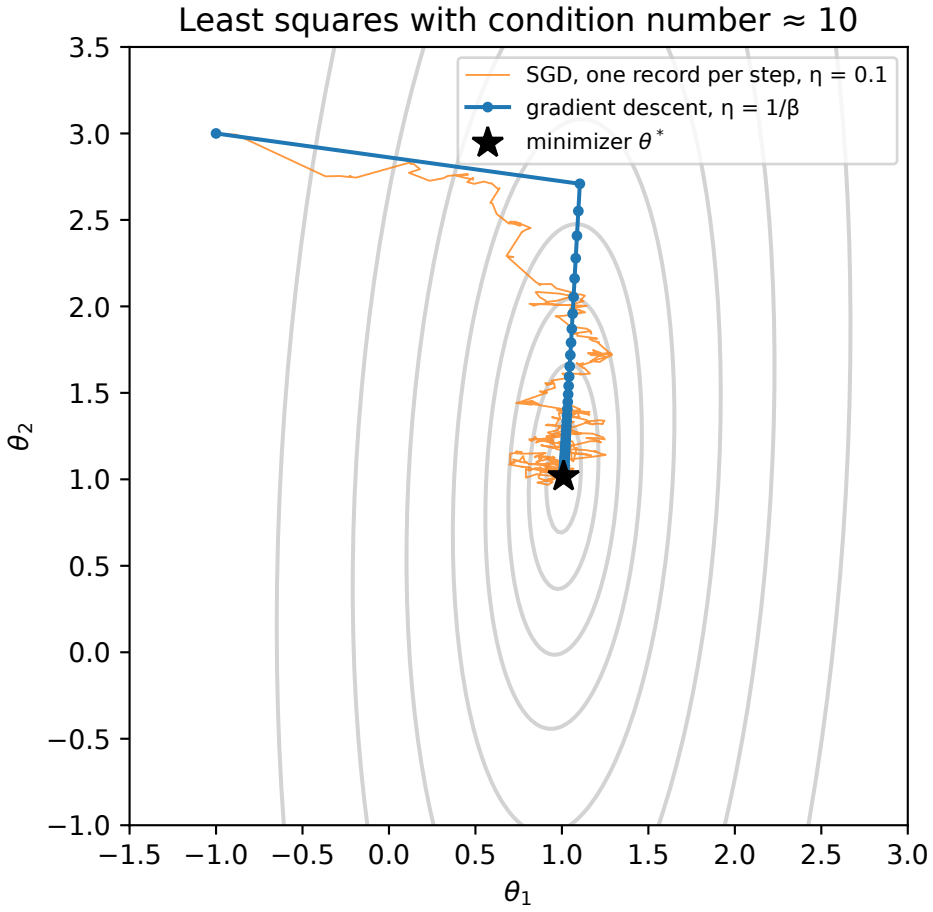


Figure 6.2.: Gradient descent (step $1/\beta$) and single-record SGD (step 0.1) on the least-squares risk of Example 6.2, over its contours.

Proof.

1. *Walk from the nearest point towards θ.* Let $p = \Pi(u)$. The points $p + s(\theta - p)$, $s \in [0, 1]$, lie in $\mathcal{W}$ because it is convex, and their squared distance to u is $r(s) = \|u - p - s(\theta - p)\|^2$.
2. *The angle at p is obtuse.* Since p is the closest point of the set, r is smallest at $s = 0$, so its derivative there, $r'(0) = -2\langle u - p, \theta - p\rangle$, is not negative. That says

$$\langle u - p, p - \theta\rangle \geq 0. \tag{6.20}$$

3. *Pythagoras.* Expanding the square of $u - \theta = (u - p) + (p - \theta)$ gives

$$\|u - \theta\|^2 = \|u - p\|^2 + \|p - \theta\|^2 + 2\langle u - p, p - \theta\rangle, \tag{6.21}$$

   where $\|u - p\|^2$ and, by step 2, the inner product are non-negative, so $\|u - \theta\| \geq \|p - \theta\|$.

□

The second tool is used where the proof averages over the random index, and its second half is what every SGD rate pays for.

Lemma 6.3 (Unbiased gradients: the cross term averages out, the square does not)**.** *Let $\mathbb{E}[g_t \mid \textit{past}] = \nabla\mathcal{L}(\theta_t)$, and let v_t be a vector fixed by the past, such as $\theta_t - \theta^*$. Then*

$$\mathbb{E}\langle g_t, v_t\rangle = \mathbb{E}\langle \nabla\mathcal{L}(\theta_t), v_t\rangle, \tag{6.22}$$

*and the mean squared length of g_t exceeds that of the true gradient by its **variance**, the mean squared error of the estimate:*

$$\mathbb{E}[\|g_t\|^2 \mid \textit{past}] = \|\nabla\mathcal{L}(\theta_t)\|^2 + \mathbb{E}[\|g_t - \nabla\mathcal{L}(\theta_t)\|^2 \mid \textit{past}]. \tag{6.23}$$

Proof.

1. *The cross term.* Given the past, v_t is fixed and the inner product with it is linear, so the conditional expectation passes inside it:

$$\mathbb{E}[\langle g_t, v_t\rangle \mid \text{past}] = \langle \nabla\mathcal{L}(\theta_t), v_t\rangle. \tag{6.24}$$

   Averaging this over the past gives the first claim.
2. *The square.* Write $g_t = \nabla\mathcal{L}(\theta_t) + e_t$, with the error e_t of conditional mean zero, and expand $\|g_t\|^2 = \|\nabla\mathcal{L}(\theta_t)\|^2 + 2\langle e_t, \nabla\mathcal{L}(\theta_t)\rangle + \|e_t\|^2$. The middle term pairs the error with a vector fixed by the past, so by step 1 it averages to zero.

□

For three records $z = (1, 2, 6)$ with the loss $\frac{1}{2}(\theta - z)^2$, at $\theta = 4$ the single-record gradients $\theta - z_i$ are 3, 2 and -2. They average to the true gradient 1, but their squares average to $17/3 = 1^2 + 14/3$, where $14/3$ is the variance.

The guarantee below is for the average of the iterates, as step 7 of its proof explains, and it has two costs. Travelling the distance R takes long with small steps, while each step's random error grows with η, so the right step balances them.

Theorem 6.2 (SGD on convex objectives)**.** *Let $\mathcal{L}$ be convex, and let projected SGD run in a closed convex set $\mathcal{W}$ that contains θ^*, starting at $\theta_1 \in \mathcal{W}$ with $\|\theta_1 - \theta^*\| \le R$. Let the random gradients be unbiased with $\mathbb{E}\|g_t\|^2 \le G^2$. With a constant step η for T steps, the averaged iterate $\bar{\theta}_T = \frac{1}{T}\sum_{t=1}^{T} \theta_t$ satisfies*

$$\mathbb{E}[\mathcal{L}(\bar{\theta}_T)] - \mathcal{L}^* \le \frac{R^2}{2\eta T} + \frac{\eta G^2}{2}. \tag{6.25}$$

The step $\eta = R/(G\sqrt{T})$ makes the right side $RG/\sqrt{T}$. For an L-Lipschitz loss every per-record gradient has length at most L, so the bound is $RL/\sqrt{T}$.

Proof. We follow the squared distance $a_t = \|\theta_t - \theta^*\|^2$: convexity ties the cross term of one step to the error, and summing over the steps cancels the changes in distance.

1. *Projection can only help.* By Lemma 6.2, since $\theta^* \in \mathcal{W}$,

$$a_{t+1} \le \|\theta_t - \eta g_t - \theta^*\|^2. \tag{6.26}$$

2. *Expand the square*, with $\theta_t - \theta^*$ and $-\eta g_t$ as the two vectors:

$$\|\theta_t - \eta g_t - \theta^*\|^2 = a_t - 2\eta\langle g_t, \theta_t - \theta^*\rangle + \eta^2\|g_t\|^2. \tag{6.27}$$

3. *Isolate the progress.* Steps 1 and 2, solved for the cross term, give

$$\langle g_t, \theta_t - \theta^*\rangle \le \frac{a_t - a_{t+1}}{2\eta} + \frac{\eta}{2}\|g_t\|^2. \tag{6.28}$$

4. *Take expectations.* By Lemma 6.3 the inner product averages to that with the true gradient, and $\mathbb{E}\|g_t\|^2 \le G^2$, so

$$\mathbb{E}\langle \nabla\mathcal{L}(\theta_t), \theta_t - \theta^*\rangle \le \frac{\mathbb{E}a_t - \mathbb{E}a_{t+1}}{2\eta} + \frac{\eta G^2}{2}. \tag{6.29}$$

5. *Convexity turns progress into error.* By Lemma 6.1 with $\theta = \theta_t$ and $\theta' = \theta^*$, the error $\mathcal{L}(\theta_t) - \mathcal{L}^*$ is at most the inner product on the left, which can therefore be replaced by $\mathbb{E}[\mathcal{L}(\theta_t) - \mathcal{L}^*]$.
6. *Sum over $t = 1, \dots, T$.* The distance terms cancel in a chain (they **telescope**) to $a_1 - a_{T+1} \le R^2$, and the noise term is added T times:

$$\sum_{t=1}^{T} \mathbb{E}[\mathcal{L}(\theta_t) - \mathcal{L}^*] \le \frac{R^2}{2\eta} + \frac{\eta T G^2}{2}. \tag{6.30}$$

7. *Pass to the averaged iterate.* Divide by T. Jensen's inequality (Lemma 3.1) says that a convex function of an average is at most the average of the function, so $\mathcal{L}(\bar{\theta}_T) \le \frac{1}{T}\sum_t \mathcal{L}(\theta_t)$, which gives Equation 6.25.
8. *Choose η.* The derivative in η, $-\frac{R^2}{2\eta^2 T} + \frac{G^2}{2}$, vanishes at $\eta = R/(G\sqrt{T})$, where both terms equal $RG/(2\sqrt{T})$.

□

For a loss with kinks, a **subgradient** replaces the gradient: any vector g such that the linear function $\mathcal{L}(\theta) + \langle g, \theta' - \theta \rangle$ stays below the graph. Step 5 then holds by this definition. The rate cannot be improved, since some convex L-Lipschitz functions force every method that only sees gradients to an error of order $RL/\sqrt{T}$ after T of them (Bubeck 2015, §3.5; Section G.4).

The guarantee is also a worst case. When the risk curves near the optimum, SGD does much better.

Example 6.3. On the running example the exact minimizer, found by Newton's method, has $\|\theta^*\| = 3.152$ and $\mathcal{L}^* = 0.5198$. Projected SGD runs on the ball of radius 5 from $\theta_1 = 0$, so θ^* is inside and $R = 5$. With $L = 1$ the step is $\eta = R/(L\sqrt{T})$, 0.5 at $T = 100$. Table 6.2 gives the mean over 20 runs.

Table 6.2.: Projected SGD on logistic regression against the convex Lipschitz bound Equation 6.25.

Steps T	Step η	Observed $\mathcal{L}(\bar{\theta}_T) - \mathcal{L}^*$	Bound $RL/\sqrt{T}$
100	0.5000	4.44×10^{-2}	0.500
1,000	0.1581	6.46×10^{-3}	0.158
10,000	0.0500	7.71×10^{-4}	0.050

The bound holds, loose by a factor of 11 at $T = 100$ and 65 at $T = 10{,}000$,

$$\frac{0.500}{0.0444} = 11.3, \qquad \frac{0.050}{0.000771} = 64.9. \tag{6.31}$$

The gap grows because, as T grows 100-fold, the error falls by $0.0444/0.000771 = 57.6$, nearer the 100 of a $1/T$ rate than the 10 of $1/\sqrt{T}$. The risk curves near θ^*, with the curvature 0.0233 of Example 6.1, and the next section shows what curvature buys (Figure 6.3, left).

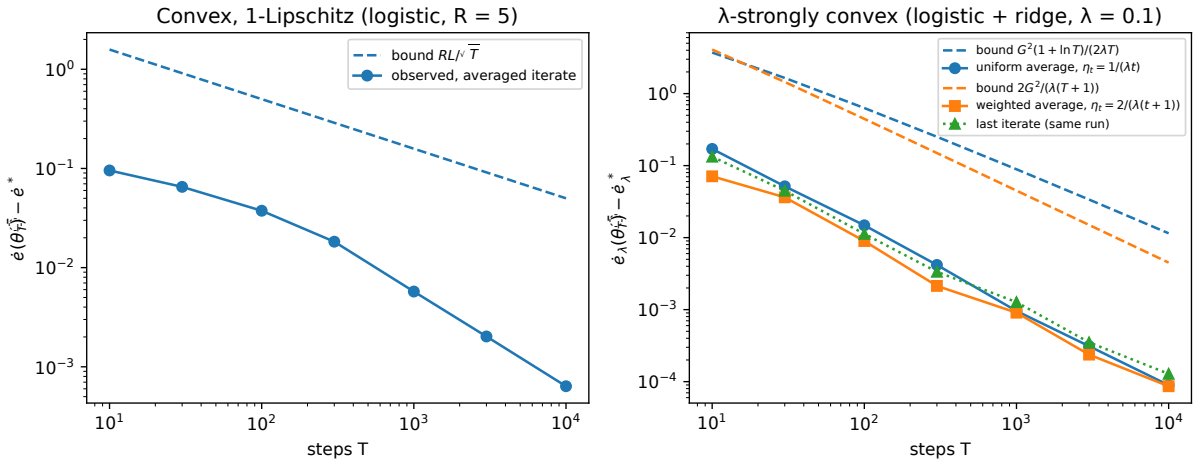


Figure 6.3.: Left: projected SGD on logistic regression, averaged iterate, against $RL/\sqrt{T}$. Right: with a ridge term $\lambda = 0.1$, the uniform and the t-weighted averages and the last iterate, against the two bounds of Theorem 6.3.

6.4. Strongly convex objectives

A convex loss plus a ridge term is strongly convex (Corollary 6.1), and strong convexity pulls the iterates towards θ^*, harder the farther they are. With steps that shrink like $1/(\lambda t)$ and an average of the iterates, that turns the $1/\sqrt{T}$ rate of Section 6.3 into $1/T$. The two choices below differ in how much weight the average gives to later iterates.

Theorem 6.3 (SGD on strongly convex objectives)**.** *Let $\mathcal{L}$ be λ-strongly convex, let projected SGD run in a closed convex set $\mathcal{W}$ that contains θ^*, and let the random gradients be unbiased with $\mathbb{E}\|g_t\|^2 \le G^2$.*

(a) With $\eta_t = \frac{1}{\lambda t}$ and the uniform average $\bar{\theta}_T = \frac{1}{T}\sum_t \theta_t$,

$$\mathbb{E}[\mathcal{L}(\bar{\theta}_T)] - \mathcal{L}^* \le \frac{G^2(1+\ln T)}{2\lambda T}. \tag{6.32}$$

(b) With $\eta_t = \frac{2}{\lambda(t+1)}$ and the average $\tilde{\theta}_T = \sum_t \frac{2t}{T(T+1)}\theta_t$, which weights iterate t in proportion to t,

$$\mathbb{E}[\mathcal{L}(\tilde{\theta}_T)] - \mathcal{L}^* \le \frac{2G^2}{\lambda(T+1)}. \tag{6.33}$$

Proof. This is the proof of Theorem 6.2 with a better step 5, whose extra distance term makes the distances cancel completely. Write again $a_t = \|\theta_t - \theta^*\|^2$.

1. *The per-step inequality.* Steps 1 to 4 of that proof hold with a step η_t that changes over time:
$$\mathbb{E}\langle \nabla\mathcal{L}(\theta_t), \theta_t - \theta^*\rangle \le \frac{\mathbb{E}a_t - \mathbb{E}a_{t+1}}{2\eta_t} + \frac{\eta_t G^2}{2}. \tag{6.34}$$
2. *Strong convexity adds a distance term.* Strong convexity at $\theta = \theta_t$, $\theta' = \theta^*$, rearranged as after Lemma 6.1, says that the inner product is at least $\mathcal{L}(\theta_t) - \mathcal{L}^* + \frac{\lambda}{2}a_t$. Using this and moving the distance term to the right,
$$\mathbb{E}[\mathcal{L}(\theta_t) - \mathcal{L}^*] \le \Big(\frac{1}{2\eta_t} - \frac{\lambda}{2}\Big)\mathbb{E}a_t - \frac{1}{2\eta_t}\mathbb{E}a_{t+1} + \frac{\eta_t G^2}{2}. \tag{6.35}$$
3. *Uniform schedule.* With $\eta_t = 1/(\lambda t)$ the two coefficients are $\frac{1}{2\eta_t} - \frac{\lambda}{2} = \frac{\lambda(t-1)}{2}$ and $\frac{1}{2\eta_t} = \frac{\lambda t}{2}$, so the distance terms are $\frac{\lambda}{2}[(t-1)\mathbb{E}a_t - t\,\mathbb{E}a_{t+1}]$. Summed over t, each term subtracted at step t is added back at step $t+1$, leaving $\frac{\lambda}{2}[0\cdot a_1 - T\,\mathbb{E}a_{T+1}] \le 0$. The noise terms $\frac{G^2}{2\lambda t}$ sum to at most $\frac{G^2(1+\ln T)}{2\lambda}$, because $\sum_{t\le T} 1/t$ is at most $1 + \int_1^T \mathrm{d}s/s = 1 + \ln T$. Dividing by T and applying Jensen gives (a).
4. *Weighted schedule.* With $\eta_t = 2/(\lambda(t+1))$ the coefficients are $\frac{\lambda(t-1)}{4}$ and $\frac{\lambda(t+1)}{4}$. Multiply the inequality of step 2 by t, which turns the coefficients into the consecutive products $(t-1)t$ and $t(t+1)$, so the distance terms become $\frac{\lambda}{4}[(t-1)t\,\mathbb{E}a_t - t(t+1)\mathbb{E}a_{t+1}]$ and telescope in the same way. The noise term becomes $\frac{t}{t+1}\cdot\frac{G^2}{\lambda} \le \frac{G^2}{\lambda}$, so no harmonic sum appears and the total is at most TG^2/λ. Dividing by $\sum_t t = T(T+1)/2$ and applying Jensen with the weights $2t/(T(T+1))$, which are non-negative and sum to 1, gives (b).

□

The harmonic sum of step 3 is where the $\ln T$ comes from: the early iterates, taken with huge steps, are bad, and the uniform average gives them full weight. The bound is divided by λ, so with a weak regularizer it beats $RG/\sqrt{T}$ only after many steps. The $1/(\lambda T)$ rate is not an artefact of the proof. On the risk $\frac{\lambda}{2}\|\theta\|^2$ with gradient noise of variance σ^2, the last iterate of schedule (a) is minus the average noise divided by λ, and its error is exactly $\sigma^2/(2\lambda T)$.

The constant G depends on the set $\mathcal{W}$. For $\ell + \frac{\lambda}{2}\|\theta\|^2$ with ℓ L-Lipschitz, the gradient on the ball of radius R is at most $L + \lambda R$.

Example 6.4. On the running example with $\lambda = 0.1$, the minimizer has $\|\theta^*\| = 0.839$, inside the ball of radius 5, and $G = 1 + 0.1 \times 5 = 1.5$. After $T = 10{,}000$ steps, averaged over 20 runs, the uniform average has error 9.09×10^{-5} and the weighted average 7.85×10^{-5}, against the bounds of parts (a) and (b),

$$\frac{G^2(1+\ln T)}{2\lambda T} = \frac{2.25 \times 10.21}{2000} = 1.15 \times 10^{-2}, \qquad \frac{2G^2}{\lambda(T+1)} = \frac{2 \times 2.25}{0.1 \times 10{,}001} = 4.50 \times 10^{-3}. \tag{6.36}$$

The bounds are loose by factors of about 130 and 60,

$$\frac{0.0115}{0.0000909} = 127, \qquad \frac{0.0045}{0.0000785} = 57, \tag{6.37}$$

mainly because G is the worst case over the whole ball (Figure 6.3, right).

6.5. Gradient noise and minibatches

Every SGD bound so far paid for $\mathbb{E}\|g_t\|^2$, which by Lemma 6.3 includes the variance. Variance comes from sampling records and, in later parts, from noise added on purpose, so this section asks how the batch size changes it and how the learning rate should follow it. Averaging B independent gradients divides their variance by B, as for the mean of a sample, and drawing without replacement does a little better.

Proposition 6.3 (Minibatch variance)**.** *Fix θ, and let $\sigma_1^2 = \frac{1}{n}\sum_i \|\nabla\ell(\theta; z_i) - \nabla\mathcal{L}(\theta)\|^2$ be the variance of a per-record gradient. The average gradient g_B of B indices drawn uniformly satisfies*

$$\mathbb{E}\|g_B - \nabla\mathcal{L}(\theta)\|^2 = \begin{cases} \dfrac{\sigma_1^2}{B} & \text{with replacement,} \\ \dfrac{\sigma_1^2}{B} \cdot \dfrac{n-B}{n-1} & \text{without replacement.} \end{cases} \tag{6.38}$$

Proof. Write $u_i = \nabla\ell(\theta; z_i) - \nabla\mathcal{L}(\theta)$ for the deviation of record i, so that the u_i sum to zero and the mean of $\|u_i\|^2$ is σ_1^2, and let $I_1, \dots, I_B$ be the indices drawn. We expand the squared mean deviation into squares and cross terms and average each.

1. *Each square.* Each draw on its own is uniform, so $\mathbb{E}\|u_{I_j}\|^2 = \sigma_1^2$.
2. *Cross terms with replacement.* The draws are independent and each deviation has mean zero, so each cross term has mean zero.

3. *Cross terms without replacement.* Two draws form a uniform pair of distinct indices, so a cross term averages $\langle u_i, u_{i'} \rangle$ over the $n(n-1)$ ordered pairs $i \neq i'$. Expanding $\| \sum_i u_i \|^2$ shows that the sum of these terms is $\| \sum_i u_i \|^2 - \sum_i \|u_i\|^2 = 0 - n\sigma_1^2$, so

$$\mathbb{E}\langle u_{I_j}, u_{I_k} \rangle = -\frac{\sigma_1^2}{n-1}. \tag{6.39}$$

   Once a record that deviates one way is drawn, the rest deviate on average slightly the other way.
4. *Add up.* The squared length of $\frac{1}{B}\sum_j u_{I_j}$ is $\frac{1}{B^2}$ times B squares and $B(B-1)$ cross terms. With replacement this averages to σ_1^2/B, and without replacement to

$$\frac{1}{B^2}\Big[B\sigma_1^2 - B(B-1)\frac{\sigma_1^2}{n-1}\Big] = \frac{\sigma_1^2}{B} \cdot \frac{n-B}{n-1}. \tag{6.40}$$

□

The factor $\frac{n-B}{n-1}$ is 1 for $B = 1$ and 0 for $B = n$. For the three records of Section 6.3 at $\theta = 4$, $\sigma_1^2 = 14/3$, and over the three possible batches of two the batch average has variance $7/6$, half of the $\sigma_1^2/2 = 7/3$ of drawing with replacement.

Noise added on top of the sampling, independent and with mean zero, simply adds its mean squared size to the mean squared length of the gradient.

Proposition 6.4 (SGD with added gradient noise)**.** *In the setting of Theorem 6.2 with an L-Lipschitz loss, let each per-record gradient be perturbed by noise ξ_t, independent of everything else, with mean zero and $\mathbb{E}\|\xi_t\|^2 = \sigma^2$ (here σ^2 is the size of the added noise alone). Then Equation 6.25 holds with $G^2 = L^2 + \sigma^2$, so the best step is $\eta^* = R/\sqrt{(L^2+\sigma^2)T}$ and the bound there is $R\sqrt{L^2+\sigma^2}/\sqrt{T}$.*

Proof.

1. *The second moment.* Expand the squared length of $\nabla\ell + \xi_t$. The cross term has mean zero, because ξ_t is independent of the gradient and has mean zero, so $\mathbb{E}\|\nabla\ell + \xi_t\|^2 \leq L^2 + \sigma^2$, and the noisy gradient is still unbiased.
2. *Substitute.* Put $G^2 = L^2 + \sigma^2$ into Equation 6.25 and choose η as in step 8 of that proof.

□

So noisier gradients call for smaller steps and longer runs. Setting the bound $R\sqrt{L^2+\sigma^2}/\sqrt{T}$ equal to a target error and solving for T, the steps needed grow in proportion to $L^2 + \sigma^2$, so noise of size $\sigma = 3L$ multiplies them by 10. A batch of B costs B gradient evaluations to divide the variance by B, so it pays off only through parallelism.

Example 6.5. At the logistic minimizer of the running example, $\sigma_1^2 = 0.1722$. The script measures the variance of the average gradient over 4,000 batches of each size from 1 to 500, and Equation 6.38 matches it within the Monte Carlo error (Figure 6.4). At $B = 500 = n/2$, for example, it measures 3.47×10^{-4} with replacement against 3.44×10^{-4} predicted. Without replacement it measures 1.74×10^{-4} against 1.72×10^{-4}, half as much.

The script then adds Gaussian noise to every per-record gradient and runs projected SGD for $T = 10{,}000$ steps, 10 runs per setting (against 20 in Table 6.2), over 17 values of η from 10^{-3} to 10. With $R = 5$ and $L = 1$ the recommended step is $\eta^* = 5/\sqrt{(1+\sigma^2)\cdot 10^4}$ (Table 6.3).

Table 6.3.: Projected SGD with added gradient noise, $T = 10{,}000$: suboptimality $\mathcal{L}(\bar{\theta}_T) - \mathcal{L}^*$ at the learning rate the bound recommends and at the best learning rate on the grid.

Noise σ	η^* from the bound	Bound at η^*	Observed at η^*	Best η on the grid	Observed there
0	0.0500	0.050	4.99×10^{-4}	0.178	1.85×10^{-4}
1	0.0354	0.071	1.58×10^{-3}	0.056	1.38×10^{-3}
3	0.0158	0.158	9.58×10^{-3}	0.056	5.10×10^{-3}

At the recommended step the error grows about 20-fold from $\sigma = 0$ to $\sigma = 3$ ($9.58 \times 10^{-3} / 4.99 \times 10^{-4} = 19.2$), and the best step falls from 0.178 to 0.056 (Figure 6.5). The bound assumes gradients as large as L and a start as far as R, so its step is cautious: without noise the best step is $0.178/0.05 = 3.56$ times larger.

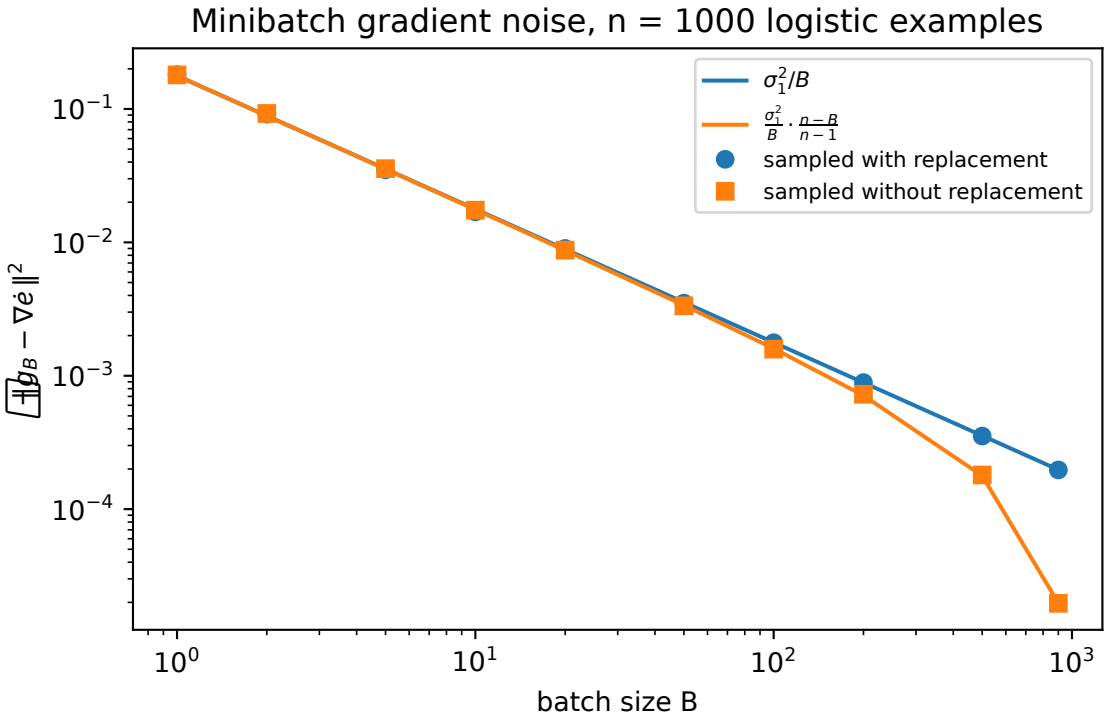


Figure 6.4.: Minibatch gradient variance at the logistic minimizer against batch size, measured and from Equation 6.38.

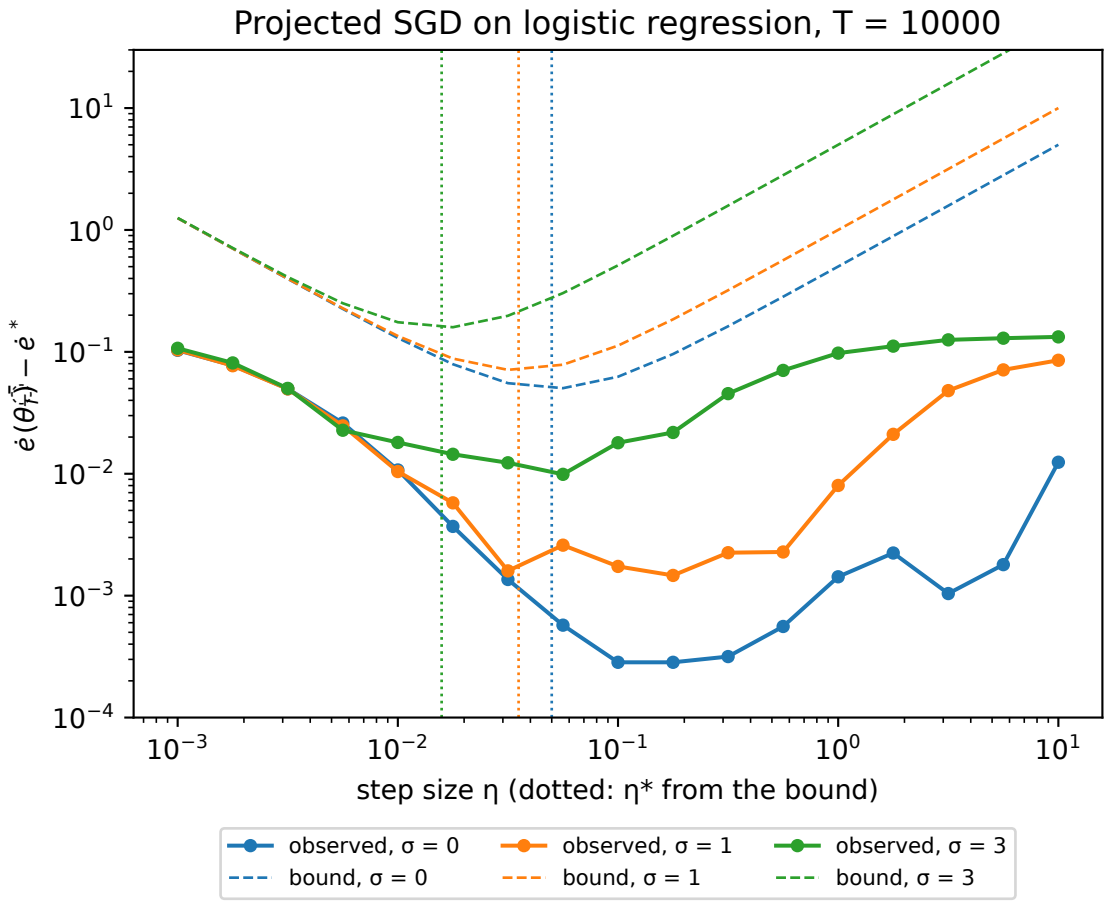


Figure 6.5.: Suboptimality of projected SGD on logistic regression after $T = 10{,}000$ steps against the learning rate, for three levels of added gradient noise. Dashed: the bound of Proposition 6.4; dotted: its minimizer η^*.

6.6. Non-convex objectives

Neural network losses are not convex, and no gradient method can promise their global minimum. SGD can still guarantee a point with a small gradient, an **approximate stationary point**. As in Proposition 6.2, the descent lemma shows that each step lowers $\mathcal{L}$ in proportion to the squared gradient, minus a penalty for the noise when the gradients are random. If the risk is bounded below, as the theorem assumes, it cannot fall forever, so the squared gradients must be small on average.

Theorem 6.4 (SGD on smooth non-convex objectives)**.** *Let $\mathcal{L}$ be β-smooth and bounded below by $\mathcal{L}_{\inf}$, and let $\Gamma = \mathcal{L}(\theta_1) - \mathcal{L}_{\inf}$ be the most it can fall. Let the random gradients be unbiased with $\mathbb{E}[\|g_t - \nabla\mathcal{L}(\theta_t)\|^2 \mid \text{past}] \le \sigma^2$. SGD without projection, with a constant step $\eta \le 1/\beta$, satisfies*

$$\min_{t\le T} \mathbb{E}\|\nabla\mathcal{L}(\theta_t)\|^2 \le \frac{1}{T}\sum_{t=1}^{T} \mathbb{E}\|\nabla\mathcal{L}(\theta_t)\|^2 \le \frac{2\Gamma}{\eta T} + \beta\eta\sigma^2. \tag{6.41}$$

With $\eta = \min\left(1/\beta,\ \sqrt{2\Gamma/(\beta\sigma^2 T)}\right)$ the right side is at most $\frac{4\beta\Gamma}{T} + 2\sqrt{\frac{2\beta\Gamma\sigma^2}{T}}$.

Proof. Bound one step by the descent lemma, average over the random gradient, then sum so that the decreases telescope.

1. *Descent lemma for one step.* By Equation 6.5 with $\theta' = \theta_t - \eta g_t$,

$$\mathcal{L}(\theta_{t+1}) \le \mathcal{L}(\theta_t) - \eta\langle\nabla\mathcal{L}(\theta_t), g_t\rangle + \frac{\beta\eta^2}{2}\|g_t\|^2. \tag{6.42}$$

2. *Average over the random gradient.* By Lemma 6.3 the inner product averages to $\|\nabla\mathcal{L}(\theta_t)\|^2$, and $\mathbb{E}\|g_t\|^2$ is at most $\|\nabla\mathcal{L}(\theta_t)\|^2 + \sigma^2$. Collecting the squared gradients,

$$\mathbb{E}\mathcal{L}(\theta_{t+1}) \le \mathbb{E}\mathcal{L}(\theta_t) - \eta\Big(1 - \frac{\beta\eta}{2}\Big)\mathbb{E}\|\nabla\mathcal{L}(\theta_t)\|^2 + \frac{\beta\eta^2\sigma^2}{2}. \tag{6.43}$$

3. *Make sure the step makes progress.* With $\eta \le 1/\beta$ the factor $1 - \beta\eta/2$ is at least $\frac{1}{2}$, so

$$\frac{\eta}{2}\mathbb{E}\|\nabla\mathcal{L}(\theta_t)\|^2 \le \mathbb{E}\mathcal{L}(\theta_t) - \mathbb{E}\mathcal{L}(\theta_{t+1}) + \frac{\beta\eta^2\sigma^2}{2}. \tag{6.44}$$

4. *Sum and telescope.* Over $t = 1, \dots, T$ the differences of $\mathcal{L}$ add up to $\mathcal{L}(\theta_1) - \mathbb{E}\mathcal{L}(\theta_{T+1}) \le \Gamma$, and the noise terms to $T\beta\eta^2\sigma^2/2$.
5. *Divide by $\eta T/2$.* This gives the average in Equation 6.41, and the minimum is at most the average.
6. *Choose η.* If $\sqrt{2\Gamma/(\beta\sigma^2 T)} \le 1/\beta$, that step makes both terms equal to $\sqrt{2\beta\Gamma\sigma^2/T}$. Otherwise $\eta = 1/\beta$, the first term is $2\beta\Gamma/T$, and the second, $\beta\eta\sigma^2$, is σ^2. Squaring the failed condition $\sqrt{2\Gamma/(\beta\sigma^2 T)} > 1/\beta$ gives $\sigma^2 < 2\beta\Gamma/T$, so the sum is below $4\beta\Gamma/T$.

□

This is the argument of Ghadimi and Lan (2013). Without noise the rate is of order $\beta\Gamma/T$, that of GD, while with noise the $\sqrt{\sigma^2/T}$ term dominates for large T. The squared gradient then falls like $1/\sqrt{T}$, so the gradient itself only like $T^{-1/4}$.

A small gradient is all you get. A stationary point can be a local minimum, a saddle point or a flat region far from anything good, and nothing bounds $\mathcal{L}(\theta_T) - \mathcal{L}^*$. Without convexity there is no tangent inequality (Lemma 6.1) to turn the progress into a gap in function value.

Example 6.6. The **sigmoid loss** $\mathrm{sig}(-m)$, with the margin m of Example 6.1, is a smooth stand-in for the 0–1 loss, which counts misclassified records. It is not convex, since sig curves both ways. The margin is linear in θ, so its Hessian is $\mathrm{sig}''(-m)\,xx^\top$, and its smoothness constant is the largest $|\,\mathrm{sig}''\,|$, reached where $\mathrm{sig} = (3 \pm \sqrt{3})/6$, which is $\beta = \frac{1}{6\sqrt{3}} = 0.0962$. The loss lies in $(0, 1)$ and equals $\frac{1}{2}$ at $\theta_1 = 0$, so $\Gamma \le \frac{1}{2}$. Every gradient has length at most $\max \mathrm{sig}' = \frac{1}{4}$, and a variance is at most the mean squared length, so $\sigma^2 \le \frac{1}{16}$.

On the logistic data, with $2\Gamma = 1$, the step of the theorem at $T = 100$ is $\sqrt{1/(0.0962 \times 0.0625 \times 100)} = 1.289$, well below $1/\beta = 10.4$. Table 6.4 averages 5 runs.

Table 6.4.: SGD on the non-convex sigmoid loss against the stationarity bound.

Steps T	Step η	Observed $\frac{1}{T}\sum_t \|\nabla\mathcal{L}(\theta_t)\|^2$	Bound Equation 6.41
100	1.289	1.64×10^{-3}	1.55×10^{-2}
1,000	0.408	5.22×10^{-4}	4.90×10^{-3}
10,000	0.129	1.87×10^{-4}	1.55×10^{-3}

The bound is loose by a factor of 8 to 10 ($0.00155/0.000187 = 8.3$, $0.0155/0.00164 = 9.5$), since Γ, σ^2 and β are worst cases, but falls at the right rate (Figure 6.6).

6.7. Sensitivity of regularized ERM

This section asks how much the minimizer itself depends on the data, which decides whether the model does as well on new records as on its training set (Algorithmic stability and generalization). Regularized empirical risk minimization (ERM) returns the minimizer $\theta^*(D)$ of the **regularized empirical risk**

$$\mathcal{L}_\lambda(\theta; D) = \frac{1}{n}\sum_{i=1}^{n} \ell(\theta; z_i) + \frac{\lambda}{2}\|\theta\|^2. \tag{6.45}$$

If you replace one record, how far can the minimizer move? Later parts call this quantity *sensitivity*. Changing one record changes one of the n terms of the average, so the gradient of the objective moves by at most $2L/n$, and strong convexity turns a small gradient into a small distance.

Theorem 6.5 (Sensitivity of regularized ERM)**.** *Let each loss $\ell(\cdot; z)$ be convex, differentiable and L-Lipschitz, let $\lambda > 0$, and let the datasets D and D' of size n differ in one replaced record. Then*

$$\|\theta^*(D) - \theta^*(D')\| \le \frac{2L}{\lambda n}. \tag{6.46}$$

The bound is reached by the linear loss $\ell(\theta; z) = z\theta$ with $d = 1$ and labels $z \in \{-1, +1\}$.

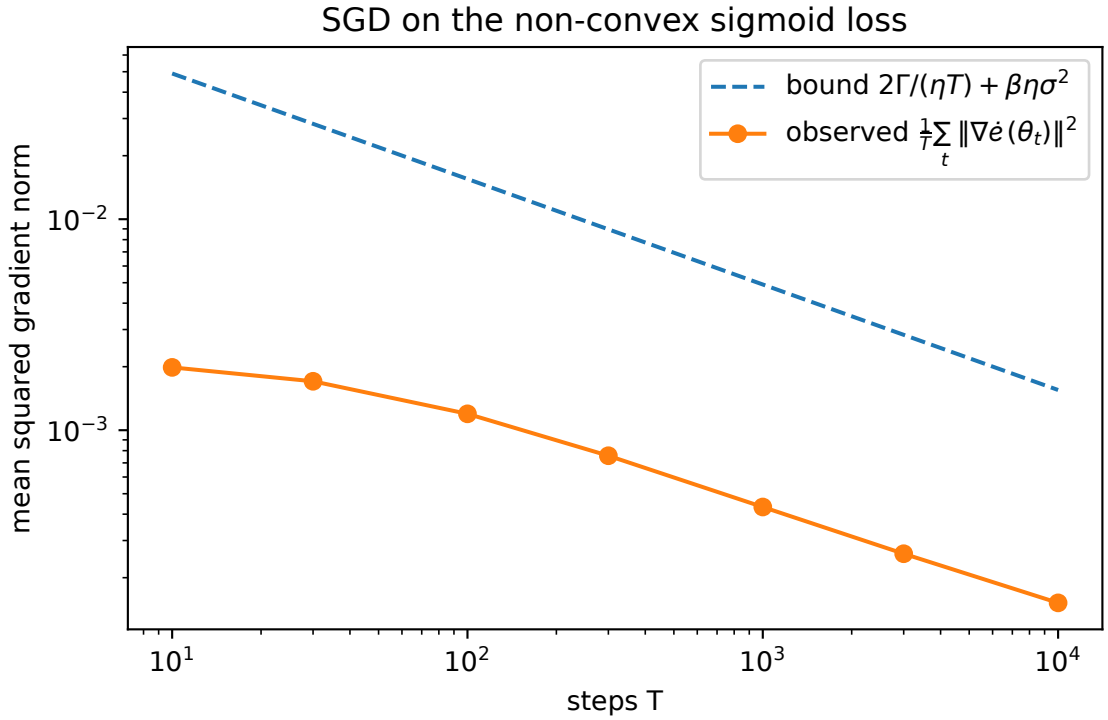


Figure 6.6.: Average squared gradient norm along SGD on the sigmoid loss, against the bound $2\Gamma/(\eta T) + \beta\eta\sigma^2$ with the step chosen for each horizon T.

Proof. Let D' replace z_i by z', and write $\theta = \theta^*(D)$ and $\theta' = \theta^*(D')$. We show that the old objective has a small gradient at θ', which strong convexity turns into a small distance from θ.

1. *The objectives differ by one term.* The new objective is the old one plus $\psi(v) = \frac{1}{n}(\ell(v; z') - \ell(v; z_i))$. Each loss has gradient of length at most L, so by the triangle inequality $\|\nabla\psi(v)\| \le \frac{2L}{n}$.
2. *At θ' the old objective has a small gradient.* The new objective has zero gradient at its minimizer θ', so
$$\nabla\mathcal{L}_\lambda(\theta'; D) = -\nabla\psi(\theta'), \tag{6.47}$$
whose length is at most $2L/n$.
3. *The old objective is strongly convex.* It is an average of convex losses plus $\frac{\lambda}{2}\|\theta\|^2$, so part (b) of Corollary 6.1 applies.
4. *A small gradient means close to the minimizer.* Part (a) of Corollary 6.1, for the old objective at the point θ', gives
$$\|\theta' - \theta\| \le \frac{\|\nabla\mathcal{L}_\lambda(\theta'; D)\|}{\lambda} \le \frac{2L}{\lambda n}. \tag{6.48}$$
5. *Tightness.* For the linear loss the objective is $\bar{z}\theta + \frac{\lambda}{2}\theta^2$, with $\bar{z}$ the mean label, so $\theta^*(D) = -\bar{z}/\lambda$. Changing one label from $+1$ to -1 lowers $\bar{z}$ by $2/n$ and moves the minimizer by exactly $2/(\lambda n)$: the two gradients, $+1$ and -1, are opposite and of full length, and the loss adds no curvature, so neither step 1 nor step 4 loses anything.

□

The move shrinks like $1/n$, because one record carries weight $1/n$. Each loss is L-Lipschitz, so the trained model's loss on any record changes by at most L times the move, $\frac{2L^2}{\lambda n}$, a bound called **uniform stability**, which the next chapter turns into a generalization bound.

The bound is loose in two places. It uses only the regularizer's curvature, while real data add their own, and it uses the worst case $2L$ for the gradient difference.

Example 6.7. The script solves regularized logistic regression exactly with Newton's method, replaces one random record with a fresh one drawn the same way, solves again, and records the largest move over 20 replacements. The bound is $2/(\lambda n)$, for example $2/(0.1 \times 1{,}000) = 0.02$ (Table 6.5).

Table 6.5.: Largest movement of the regularized logistic minimizer over 20 single-record replacements, against $2L/(\lambda n)$ with $L = 1$.

n	$\lambda = 0.01$: observed / bound	$\lambda = 0.1$: observed / bound	$\lambda = 1$: observed / bound
100	2.52×10^{-1} / 2.00	5.98×10^{-2} / 0.200	8.55×10^{-3} / 0.0200
1,000	2.13×10^{-2} / 0.200	6.66×10^{-3} / 0.0200	8.54×10^{-4} / 0.00200
10,000	2.06×10^{-3} / 0.0200	6.35×10^{-4} / 0.00200	7.80×10^{-5} / 0.000200

Down each column the move shrinks tenfold, as $1/n$ predicts, and the ratio of observed to bound stays put: 0.39–0.43 at $\lambda = 1$, 0.30–0.33 at $\lambda = 0.1$ and 0.10–0.13 at $\lambda = 0.01$. At $\lambda = 1$ the regularizer supplies nearly all the curvature, and the remaining factor of about 2.5 (1/0.4) comes

from two records' gradients rarely pointing in opposite directions. At $\lambda = 0.01$ the data supply more curvature than the regularizer, about 0.0233 as in Example 6.1, so as $\lambda \to 0$ the bound blows up while the observed move levels off (Figure 6.7).

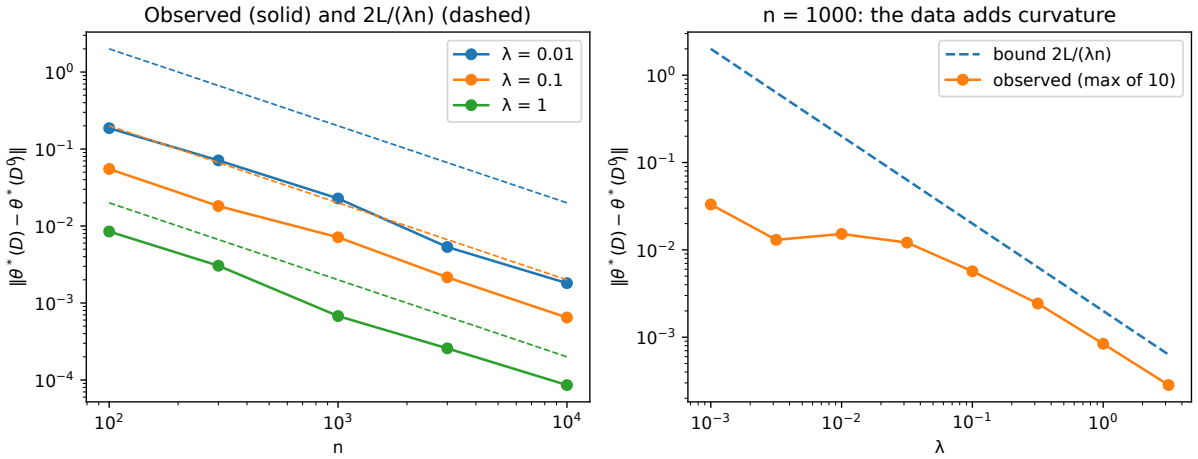


Figure 6.7.: Left: largest observed move of the regularized logistic minimizer (solid) against $2L/(\lambda n)$ (dashed), for three values of λ. Right: the same at $n = 1000$ against λ.

6.7.1. Without strong convexity

Convexity alone does not keep the minimizer from jumping, because a convex function can be flat, and on a flat stretch a tiny change in the data can move the minimizer far.

Proposition 6.5 (A convex Lipschitz loss whose minimizer jumps)**.** *Let $\ell(\theta; z) = |\theta - z|$, which is convex and 1-Lipschitz, and let D hold $n = 2k + 1$ values, $k + 1$ of them 0 and k of them 1. Replacing one 0 by a 1 moves the minimizer of $\frac{1}{n}\sum_i |\theta - z_i|$ from 0 to 1, for every n.*

Proof.

1. *The minimizer is the median.* Moving θ up a little increases $|\theta - z_i|$ for each value below θ and decreases it for each value above. So the risk falls while more values lie above θ, and stops falling when half lie on each side.
2. *Before and after.* In D the majority is 0, so the minimizer is 0. After the replacement the majority is 1, a move of 1 whatever n is.

□

With $\frac{\lambda}{2}\theta^2$ added, Equation 6.46 caps the move at $2/(\lambda n)$, with subgradients in place of gradients. The actual move is half of that, $1/(\lambda n)$, where the new subgradient $-1/n + \lambda\theta$ vanishes.

For a non-convex loss even a regularizer may not help. With labels $z \in \{-1, +1\}$, the double-well loss $\frac{1}{4}(\theta^2 - 1)^2 - z\theta$ has the risk $\frac{1}{4}(\theta^2 - 1)^2 - \bar{z}\theta$: two valleys near ± 1, tilted towards the sign of the mean label $\bar{z}$. With 11 records, six labelled $+1$, replacing one $+1$ by -1 flips that sign and

moves the minimizer from 1.0427 to −1.0427 (Figure 6.8), and with 1,001 records the move is still 2.0010. A ridge term fixes this only if it makes the objective convex, as step 3 of the proof of Theorem 6.5 needs. A fixed number of SGD steps from a fixed start, by contrast, is a composition of smooth maps of the records, so for fixed random indices it changes continuously with the data. That is why, for non-convex models, one studies the influence of a record on the *algorithm*.

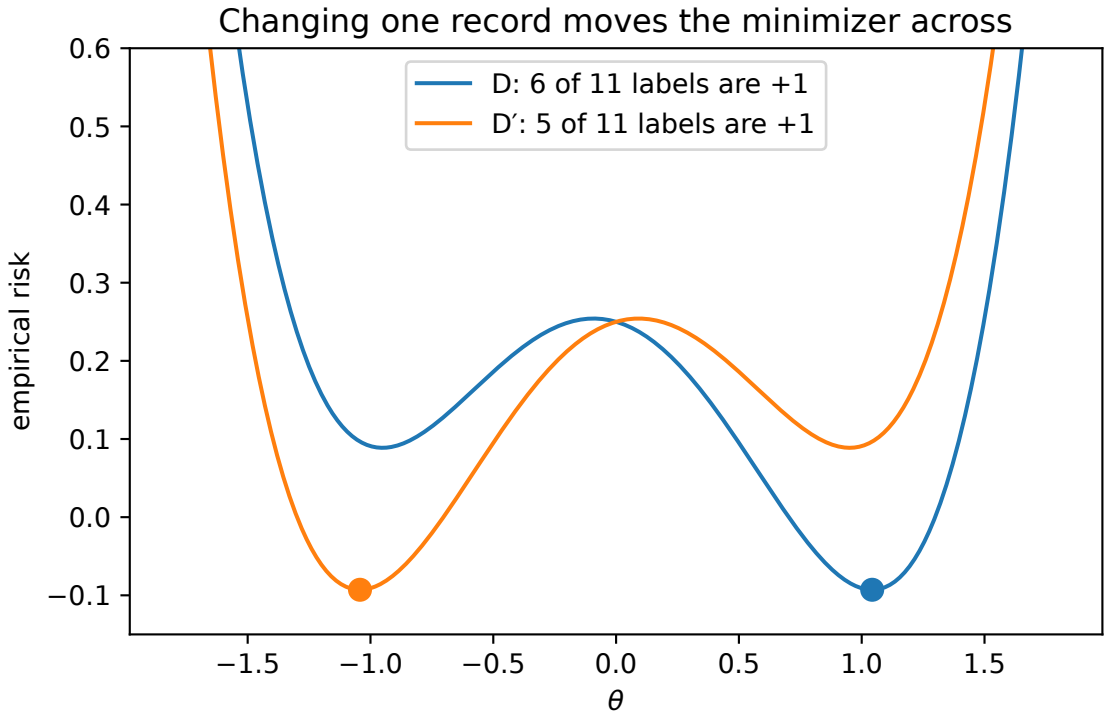


Figure 6.8.: A double-well empirical risk on 11 records before (D, six labels +1) and after (D', five labels +1) one label changes. The dots mark the global minimizers, which jump from one well to the other.

6.8. Code

The script code/part01-prerequisites/optimization.py computes every number and figure in this chapter, checking each bound by simulation. From the repository root:

```
uv run python code/part01-prerequisites/optimization.py
```

Table 6.6.: The script's output with the default seed.

Experiment	Result
Logistic loss at $x = (0.6, 0.8)$, $y = +1$	Loss, gradient norm, curvature 0.6931, 0.5000, 0.2500 at $\theta = 0$; 0.0067, 0.0067, 0.0066 at $(3, 4)$

Experiment	Result
GD on least squares, $\kappa = 9.58$	122 steps to shrink the error by 10^6, against at most 126
Minibatch variance at θ^*, $\sigma_1^2 = 0.1722$	Matches Equation 6.38 for $B = 1$ to 500
Projected SGD, convex, $T = 10^4$	7.71×10^{-4} against the bound 0.050 (Table 6.2)
Projected SGD, $\lambda = 0.1$, $T = 10^4$	Uniform 9.09×10^{-5} (bound 1.15×10^{-2}), weighted 7.85×10^{-5} (bound 4.50×10^{-3}), last iterate 9.61×10^{-5}
Added gradient noise, $T = 10^4$	Best grid step 0.178, 0.056, 0.056 for $\sigma = 0, 1, 3$ (Table 6.3)
Non-convex sigmoid loss, $T = 10^4$	1.87×10^{-4} against the bound 1.55×10^{-3} (Table 6.4)
Regularized ERM, one record replaced	Observed/bound between 0.10 and 0.43 (Table 6.5)
Double well, $n = 11$ and 1,001	Minimizer moves by 2.0854 and 2.0010

tests/test_optimization.py checks that the output still matches Table 6.6 and the tables above.

6.9. Summary

Table 6.7.: The results of this chapter, and where each is used.

Result	Statement	Use
Lemma 6.1	A convex f lies above its tangents	Every convex proof
Proposition 6.1	Bounded gradient; descent lemma; Hessian test; strong monotonicity	Lipschitz governs SGD; smoothness governs learning rates
Corollary 6.1	$\|\theta - \theta^*\| \le \|\nabla f(\theta)\|/\lambda$; ridge adds strong convexity	ERM sensitivity; uniqueness of the minimizer
Example 6.1	1-Lipschitz, $\frac{1}{4}$-smooth, convex	Ridge: $G = 1 + \lambda R$
Theorem 6.1	Error shrinks by $1 - 1/\kappa$ per step	About $\kappa \ln(1/\tau)$ steps
Proposition 6.2	A step gains $\|\nabla \mathcal{L}\|^2/(2\beta)$	Non-convex analysis
Lemma 6.2	Projection never increases distances to the set	Free if $\theta^* \in \mathcal{W}$
Lemma 6.3	Square gains the variance	Every SGD rate pays for it
Theorem 6.2	$RG/\sqrt{T}$ at $\eta = R/(G\sqrt{T})$	Average the iterates

Result	Statement	Use
Theorem 6.3	$G^2(1+\ln T)/(2\lambda T)$; $2G^2/(\lambda(T+1))$ weighted	Regularized models
Proposition 6.3	σ_1^2/B, times $\frac{n-B}{n-1}$ without replacement	Batches save nothing per evaluation
Proposition 6.4	$G^2 = L^2 + \sigma^2$	Noise multiplies steps by $1+\sigma^2/L^2$
Theorem 6.4	Mean squared gradient $\le 2\Gamma/(\eta T) + \beta\eta\sigma^2$	Neural networks: a small gradient only
Theorem 6.5	$2L/(\lambda n)$, tight	Uniform stability
Proposition 6.5	Without strong convexity the minimizer jumps	Analyse the algorithm instead

6.10. References

- Shalev-Shwartz & Ben-David, *Understanding Machine Learning* (2014), ch. 12–14 (convexity, regularization and stability, SGD): the closest match to this chapter, at the same level.
- Bubeck, "Convex Optimization: Algorithms and Complexity" (2015), ch. 3 and §6.1: gradient methods under each set of assumptions, with lower bounds (§3.5).
- Nesterov, *Lectures on Convex Optimization* (2018), §2.1: the equivalent forms of smoothness and strong convexity, with proofs.
- Ghadimi & Lan, "Stochastic First- and Zeroth-Order Methods for Nonconvex Stochastic Programming" (SIAM J. Optim. 2013): the non-convex SGD rate.
- Lacoste-Julien, Schmidt & Bach, "A Simpler Approach to Obtaining an O(1/t) Convergence Rate for the Projected Stochastic Subgradient Method" (2012); Rakhlin, Shamir & Sridharan, "Making Gradient Descent Optimal for Strongly Convex Stochastic Optimization" (ICML 2012): averaging schemes without the $\ln T$.
- Bottou, Curtis & Nocedal, "Optimization Methods for Large-Scale Machine Learning" (SIAM Review 2018): noise, learning rates and batch sizes in practice.
- Bousquet & Elisseeff, "Stability and Generalization" (JMLR 2002): the stability of regularized ERM and what it implies.

7. Algorithmic stability and generalization

You train a model and its loss on the training data is low. Will its loss on new data be low too? The difference between the two is the **generalization gap**, and bounding it is the central question of learning theory. The classical answer counts the models an algorithm could output; this chapter looks at the algorithm instead.

The idea that organizes it fits in one sentence: a model generalizes when it does not depend much on any single training record. So if replacing one record can move the loss by at most γ at every test point, a property called uniform stability, the expected gap is at most γ. The chapter computes γ for regularized empirical risk minimization (ERM) and for stochastic gradient descent (SGD), and ends with the same bias reappearing when a holdout set is reused to choose what to measure.

The running example is logistic regression in ten dimensions. Features x are uniform on the unit sphere in $\mathbb{R}^{10}$, and the label $y = \pm 1$ is $+1$ with probability $\sigma(5x_1)$, where $\sigma(t) = 1/(1+e^{-t})$ is the logistic function. The model is a weight vector θ, chosen to minimize the average logistic loss $\ln(1+e^{-y\langle\theta,x\rangle})$ plus a penalty $\frac{\lambda}{2}\|\theta\|^2$. With $n = 50$ records and $\lambda = 0.01$, averaged over 100 training sets, the training loss is 0.4603 and the test loss is 0.5781. With $n = 1{,}000$ records the gap shrinks to about 0.006, and the chapter explains why.

7.1. Risk, empirical risk and the gap

Before bounding the gap you need to say exactly what it is, and why it is not zero: training loss and future loss agree on average only for a model chosen before looking at the data.

Definition 7.1 (Risk, empirical risk and the gap)**.** A **training set** $D = (z_1, \dots, z_n)$ is a list of n records drawn independently from a distribution $\mathcal{D}$, the source of all data, past and future. A **training algorithm** $\mathcal{T}$ maps D to a model $\mathcal{T}(D)$, and a **loss** $\ell(\theta; z) \geq 0$ says how badly the model θ does on the record z. The **population risk**, or test loss, of θ is its average loss on a new record, and its **empirical risk**, or training loss, is its average loss on the training set:

$$\mathcal{L}_{\mathcal{D}}(\theta) = \mathbb{E}_{z\sim\mathcal{D}}\big[\ell(\theta; z)\big], \qquad \mathcal{L}(\theta; D) = \frac{1}{n}\sum_{i=1}^{n} \ell(\theta; z_i). \tag{7.1}$$

The **generalization gap** of the trained model is the test loss minus the training loss,

$$\operatorname{gap}(D) = \mathcal{L}_{\mathcal{D}}(\mathcal{T}(D)) - \mathcal{L}(\mathcal{T}(D); D). \tag{7.2}$$

The test loss cannot be computed, because $\mathcal{D}$ is unknown, so the training loss stands in for it, and for a fixed model it does so without bias.

Proposition 7.1 (A fixed model's training loss is unbiased)**.** *Let θ be a model that does not depend on the training set D. Then*

$$\mathbb{E}_D[\mathcal{L}(\theta; D)] = \mathcal{L}_{\mathcal{D}}(\theta). \tag{7.3}$$

Proof.

1. The expectation of an average is the average of the expectations, by linearity, so $\mathbb{E}_D[\mathcal{L}(\theta; D)] = \frac{1}{n}\sum_i \mathbb{E}[\ell(\theta; z_i)]$.
2. Each z_i is a draw from $\mathcal{D}$ and θ is fixed, so each term $\mathbb{E}[\ell(\theta; z_i)]$ is the average loss of θ on a draw from $\mathcal{D}$, which is $\mathcal{L}_{\mathcal{D}}(\theta)$. The average of n copies of it is itself.

□

Step 2 is exactly what fails for the trained model $\mathcal{T}(D)$. The record z_i helped to choose $\mathcal{T}(D)$, so it is no longer an independent draw from the model's point of view. The algorithm chose a model that does well on exactly these records, so the training loss is biased downward.

Example 7.1. In the running example with $n = 50$ and $\lambda = 0.01$, the average training loss over 100 training sets is 0.4603 and the average test loss, computed by numerical integration (Section 7.9), is 0.5781. The expected gap is about 0.119, not exactly their difference, because Example 7.4 estimates it with less noise. With $n = 1{,}000$ it is about 0.006.

7.2. Uniform stability and the expected gap

To control the gap you need to say how much $\mathcal{T}(D)$ depends on the training set: replace one record, retrain, and see how much the loss changes at any test point. This section turns that into a definition and proves that it bounds the expected gap.

The tool is a training set that differs from D in one record. The **replace-one neighbour** $D^{(i)} = (z_1, \dots, z_{i-1}, z_i', z_{i+1}, \dots, z_n)$ is D with record i replaced by z_i'. In probability statements, the replacements $z_1', \dots, z_n'$ are fresh draws from $\mathcal{D}$, independent of each other and of D.

Definition 7.2 (Uniform stability)**.** A training algorithm $\mathcal{T}$ is **γ-uniformly stable** if for every training set D, every index i, every replacement z_i' and every test point z,

$$\left|\ell(\mathcal{T}(D); z) - \ell(\mathcal{T}(D^{(i)}); z)\right| \le \gamma. \tag{7.4}$$

If $\mathcal{T}$ is randomized, as SGD is through the order in which it visits records, the condition is on the expectation over its internal randomness:

$$\left|\mathbb{E}_{\mathcal{T}}[\ell(\mathcal{T}(D); z) - \ell(\mathcal{T}(D^{(i)}); z)]\right| \le \gamma. \tag{7.5}$$

Usually γ falls with n, because each record carries less weight in a larger training set (the literature writes it β, which this book keeps for a failure probability). In Figure 7.1 one record's label is flipped, and the fitted model moves a lot when the penalty λ is small and barely when it is large.

The main result needs one fact about independent draws: swapping two of them changes no expectation. Step 2 of the proof below uses it to exchange a training record with a fresh one.

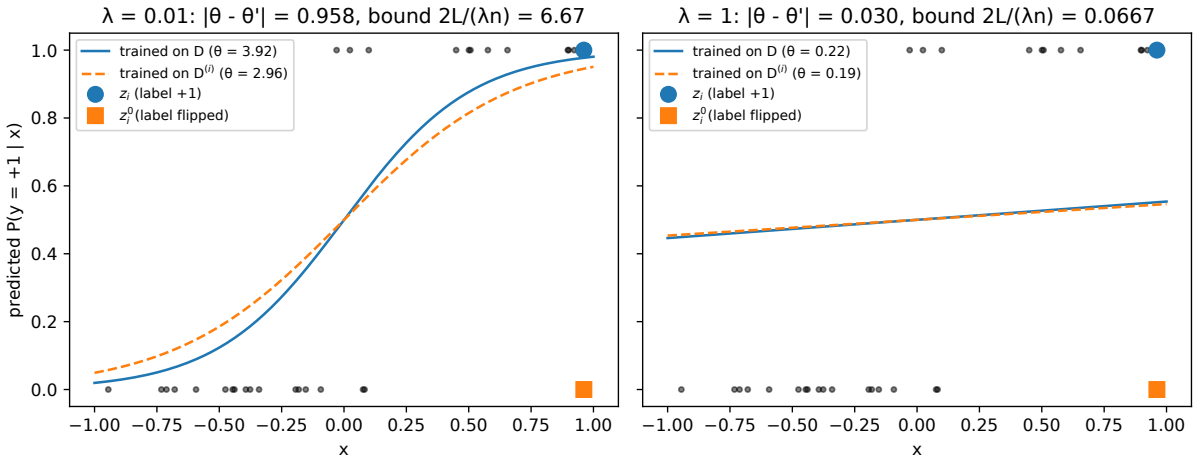


Figure 7.1.: One-dimensional logistic regression on 30 points, fitted on D and on $D^{(i)}$, where one positive record has its label flipped, at $\lambda = 0.01$ (left) and $\lambda = 1$ (right). The titles compare the weight change with the bound of Theorem 7.2.

Lemma 7.1 (Swapping two independent draws)**.** *Let U and V be independent and identically distributed (i.i.d.), and independent of a random variable W. Then (U, V, W) and (V, U, W) have the same distribution, so $\mathbb{E}[g(U, V, W)] = \mathbb{E}[g(V, U, W)]$ for every function g whose expectation exists.*

Proof.

1. By independence, the joint density of (U, V, W) at (u, v, w) is the product $\rho(u)\,\rho(v)\,\rho_W(w)$, where ρ is the common density of U and V, and ρ_W is that of W. For discrete variables, read mass functions for densities.
2. Exchanging u and v leaves this product unchanged, because U and V share the density ρ. So the swapped triple has the same distribution, and every expectation computed from it agrees.

□

The test loss of the trained model can be measured on a fresh point z_i', and swapping the roles of z_i and z_i' keeps its average while turning it into the loss on z_i of a model trained with z_i' in its place, which never saw z_i.

Theorem 7.1 (The expected gap is an average replace-one effect)**.** *For any training algorithm $\mathcal{T}$, with the records of D and the replacements $z_1', \dots, z_n'$ all i.i.d. from $\mathcal{D}$, and $\mathbb{E}$ averaging over all of them and over any randomness of $\mathcal{T}$,*

$$\mathbb{E}_D[\mathrm{gap}(D)] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}\Big[\ell(\mathcal{T}(D^{(i)}); z_i) - \ell(\mathcal{T}(D); z_i)\Big]. \tag{7.6}$$

If $\mathcal{T}$ is moreover γ-uniformly stable, then

$$\big|\mathbb{E}_D[\mathrm{gap}(D)]\big| \le \gamma. \tag{7.7}$$

Proof. The plan is to write the expected test loss with a fresh point, exchange that point with a training record by Lemma 7.1, subtract the training loss, and bound each term by stability.

1. *Write the test loss with a fresh point.* The point z_i' is independent of D and has distribution $\mathcal{D}$, so given D its average loss is the test loss of $\mathcal{T}(D)$. Averaging over D too gives
$$\mathbb{E}\big[\mathcal{L}_{\mathcal{D}}(\mathcal{T}(D))\big] = \mathbb{E}\big[\ell(\mathcal{T}(D); z_i')\big] \quad \text{for every } i. \tag{7.8}$$
2. *Exchange z_i and z_i'.* The two points are i.i.d. and independent of the other records, so by Lemma 7.1 exchanging them changes no expectation. The exchange turns D into $D^{(i)}$ and z_i' into z_i, so
$$\mathbb{E}\big[\ell(\mathcal{T}(D); z_i')\big] = \mathbb{E}\big[\ell(\mathcal{T}(D^{(i)}); z_i)\big]. \tag{7.9}$$
3. *Average over i.* Steps 1 and 2 hold for every i, so the expected test loss is also the average $\frac{1}{n}\sum_i \mathbb{E}[\ell(\mathcal{T}(D^{(i)}); z_i)]$.
4. *Subtract the training loss.* By definition, the expected training loss is $\frac{1}{n}\sum_i \mathbb{E}[\ell(\mathcal{T}(D); z_i)]$. Both sums run over the same i, so their difference is the average of the differences, which is Equation 7.6.
5. *Bound each term.* Uniform stability at the test point $z = z_i$ puts each difference $\ell(\mathcal{T}(D^{(i)}); z_i) - \ell(\mathcal{T}(D); z_i)$ in $[-\gamma, \gamma]$ for every outcome. Expectations and averages of numbers in that interval stay in it, which is Equation 7.7. For a randomized $\mathcal{T}$, take the expectation over its randomness first.

□

The identity is not a bound. It says that the expected gap *is* the average **replace-one self-influence**, how much the loss on z_i rises when z_i leaves the training set. So a large gap means that the model fits many records only because they were there, and Section 7.7 measures this record by record.

The bound, by contrast, takes the worst case over everything, which is why γ can sit far above the true gap, and why weaker notions that average over the data exist (Bousquet & Elisseeff 2002; Shalev-Shwartz et al. 2010). No count of models appears in it, so it covers SGD on large models.

Example 7.2. Step 2 is clearest with every outcome listed. Take $n = 1$: one record $z_1 \in \{0, 1\}$ with $\Pr[z_1 = 1] = 0.3$, and a fresh copy z_1'. The algorithm shrinks the data towards zero, $\mathcal{T}(D) = z_1/2$, and the loss is the squared error $\ell(\theta; z) = (\theta - z)^2$. Table 7.1 lists the four outcomes with the three losses of the proof.

Table 7.1.: Every outcome of the one-record case. The renamed column is the test column with the rows $(0, 1)$ and $(1, 0)$ exchanged.

(z_1, z_1')	Probability	Train $\ell(\mathcal{T}(D); z_1)$	Test $\ell(\mathcal{T}(D); z_1')$	Renamed $\ell(\mathcal{T}(D^{(1)}); z_1)$
$(0, 0)$	$0.7 \times 0.7 = 0.49$	0	0	0
$(0, 1)$	$0.7 \times 0.3 = 0.21$	0	$(0-1)^2 = 1$	$(\frac{1}{2}-0)^2 = 0.25$
$(1, 0)$	$0.3 \times 0.7 = 0.21$	$(\frac{1}{2}-1)^2 = 0.25$	$(\frac{1}{2}-0)^2 = 0.25$	$(0-1)^2 = 1$
$(1, 1)$	$0.3 \times 0.3 = 0.09$	0.25	0.25	0.25

(z_1, z_1')	Probability	Train $\ell(\mathcal{T}(D); z_1)$	Test $\ell(\mathcal{T}(D); z_1')$	Renamed $\ell(\mathcal{T}(D^{(1)}); z_1)$
Average		0.075	0.285	0.285

Row by row, the test and renamed columns differ, but the renamed column holds the same values attached to the mirror-image outcome, which has the same probability. That is all step 2 says. The expected gap, test minus train, is $0.285 - 0.075 = 0.21$, and so is renamed minus train. Changing z_1 from 0 to 1 moves the loss at $z = 1$ from 1 to 0.25 and at $z = 0$ from 0 to 0.25, so the largest change is $\gamma = 0.75$, above the gap. The running example has too many outcomes to list; Section 7.3 takes it up.

7.3. Regularized empirical risk minimization

The bound of Theorem 7.1 is only as good as γ, so the next question is which algorithms have a small one. Regularized ERM is the first answer. Its penalty $\frac{\lambda}{2}\|\theta\|^2$ makes the objective a bowl with curvature at least λ, and one record, with weight $1/n$ in the objective, can only tilt such a bowl so far.

A loss is L-**Lipschitz** in the weights (Definition 6.2) when moving the weights by a distance r changes it by at most Lr at every record, so its gradient never exceeds L in norm. Regularized ERM returns the minimizer θ_D of the **regularized empirical risk**,

$$\mathcal{L}_\lambda(\theta; D) = \mathcal{L}(\theta; D) + \frac{\lambda}{2}\|\theta\|^2. \tag{7.10}$$

Replacing one record changes the objective's gradient by at most $2L/n$. The penalty makes the objective λ-**strongly convex**, curved by at least λ in every direction, so a point where its gradient is small lies close to its minimizer, and the Lipschitz constant turns the weight change into a loss change.

Theorem 7.2 (Stability of regularized ERM)**.** *Let $\ell(\cdot; z)$ be convex, differentiable and L-Lipschitz for every record z, and let $\lambda > 0$. Then for every training set D, index i, replacement z_i' and test point z,*

$$\|\theta_D - \theta_{D^{(i)}}\| \le \frac{2L}{\lambda n} \quad \textit{and} \quad \left|\ell(\theta_D; z) - \ell(\theta_{D^{(i)}}; z)\right| \le \frac{2L^2}{\lambda n}. \tag{7.11}$$

So regularized ERM is γ-uniformly stable with $\gamma = 2L^2/(\lambda n)$. Its weights also satisfy $\|\theta_D\| \le L/\lambda$, which bounds the loss in Example 7.5.

Proof.

1. *The weights.* The first bound is the sensitivity of regularized ERM, Theorem 6.5, and its proof runs as follows. The two objectives differ only in the i-th term, and the gradient of the second is zero at its own minimizer $\theta_{D^{(i)}}$. So there the gradient of $\mathcal{L}_\lambda(\cdot; D)$ is the difference of two per-record gradients divided by n, of norm at most $2L/n$. A λ-strongly convex function's minimizer lies within the gradient norm divided by λ of any point (Corollary 6.1), here $2L/(\lambda n)$.

2. *The losses.* The loss is L-Lipschitz, so a weight change of at most $2L/(\lambda n)$ moves the loss at every test point by at most $2L^2/(\lambda n)$. This holds for every D, i, z_i' and z, which is Definition 7.2.
3. *The norm.* The gradient of $\mathcal{L}_\lambda(\cdot; D)$ is zero at its minimizer, so $\lambda\theta_D = -\frac{1}{n}\sum_i \nabla\ell(\theta_D; z_i)$. The right side is an average of vectors of norm at most L, so dividing by λ gives $\|\theta_D\| \le L/\lambda$.

□

Doubling λ halves γ, but a larger λ also pulls the solution away from the best fit, so the test loss is U-shaped in λ (Figure 7.3, right). The bound is loose in three places: it charges the gradient difference its worst case $2L$, it ignores the curvature of the training loss $\mathcal{L}(\theta; D)$ itself, and step 2 assumes the worst test point.

Example 7.3. By the chain rule, the gradient of the logistic loss is $-y\,\sigma(-y\langle\theta, x\rangle)\,x$, of norm at most $\|x\| = 1$, so $L = 1$ and $\gamma = 2/(\lambda n)$ in the running example. The script refits on $D^{(i)}$ for 10 random i, with z_i replaced by the same point with its label flipped. It records the largest weight change, and the largest loss change over 2,000 random test points and the four points along the direction the weights moved, where the loss changes most.

Table 7.2.: Replace-one changes of regularized logistic regression against Equation 7.11. With $L = 1$ the weight and loss bounds coincide. Ratio is the loss change over its bound.

n	λ	Largest weight change	Bound $2L/(\lambda n) = 2L^2/(\lambda n)$	Largest loss change	Ratio
100	0.01	0.406	2.0	0.340	0.17
100	0.1	0.0824	0.2	0.0474	0.24
100	1	0.00976	0.02	0.00505	0.25
1,000	0.1	0.00809	0.02	0.00499	0.25
10,000	0.01	0.00336	0.02	0.00285	0.14
10,000	1	9.76×10^{-5}	2×10^{-4}	5.00×10^{-5}	0.25

For $\lambda \ge 0.1$ the changes follow $1/(\lambda n)$ closely (Figure 7.2). At $\lambda = 0.01$ the training loss's own curvature adds to λ, so the ratio drops to about 0.15.

For $\lambda = 1$ the ratio is 0.25, made of two factors of 2. At the same θ, the gradients for labels y and $-y$ are $-y\,\sigma(-y\langle\theta, x\rangle)x$ and $y\,\sigma(y\langle\theta, x\rangle)x$, and since $\sigma(-t) + \sigma(t) = 1$ their difference is $-yx$, of norm 1 rather than $2L = 2$. So the weights move by 0.00976, half the bound. With $\|\theta\| \le 1/\lambda = 1$ the weights are small and test margins are near zero, where the slope of the logistic loss is $\sigma(0) = \frac{1}{2}$, so the loss moves by 0.00505, half the weight change.

Example 7.4. Equation 7.7 bounds the expected gap by the same $\gamma = 2/(\lambda n)$. The script averages the gap over 100 training sets, and subtracts the gap of a fixed reference model, fitted once on a separate sample of 20,000 records, on the same training set. That gap has mean zero by Proposition 7.1 but moves with the same lucky or unlucky training sets, so subtracting it cancels most of the noise.

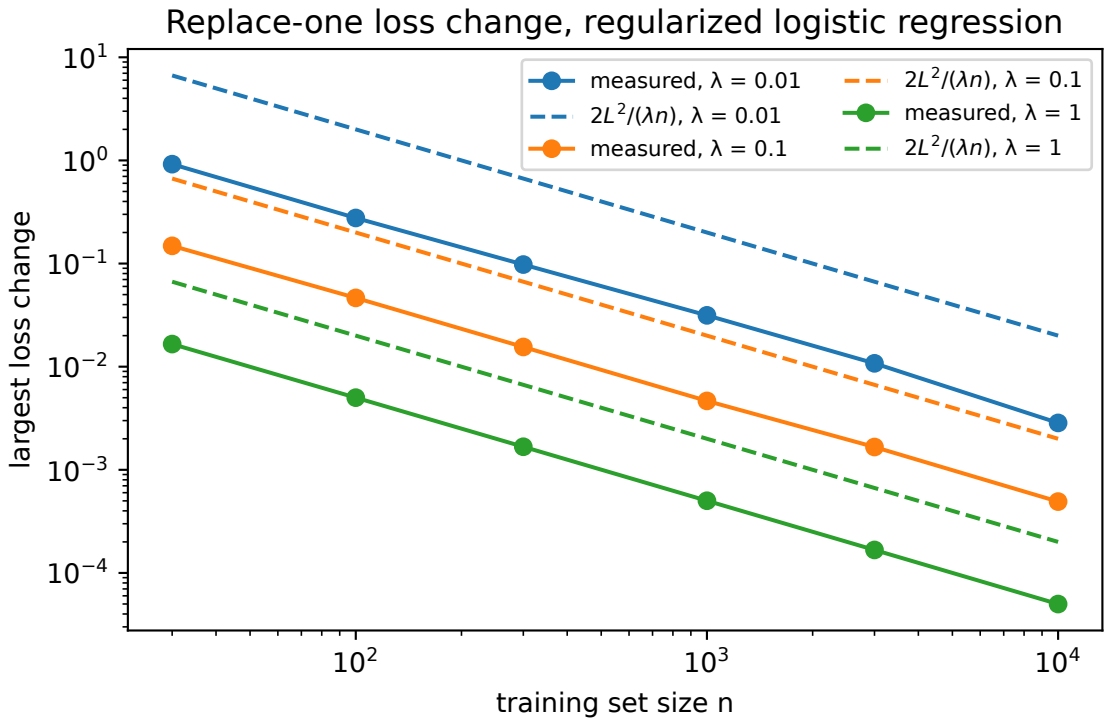


Figure 7.2.: Largest replace-one loss change of regularized logistic regression against n, for three values of λ (solid), with the bound $2L^2/(\lambda n)$ (dashed).

Table 7.3.: Expected generalization gap of regularized logistic regression, averaged over 100 training sets, against the stability bound.

λ	n	Train loss	Test loss	Expected gap	Bound $2/(\lambda n)$	Gap / bound
0.01	50	0.4603	0.5781	0.119	4.0	0.030
0.01	1,000	0.5311	0.5381	0.00596	0.2	0.030
0.1	50	0.6073	0.6401	0.0319	0.4	0.080
0.1	200	0.6288	0.6373	0.00830	0.1	0.083
0.1	1,000	0.6346	0.6365	0.00178	0.02	0.089
1	50	0.6806	0.6854	0.00459	0.04	0.115
1	1,000	0.6852	0.6855	0.000253	0.002	0.127

The gap falls like $1/n$ and stays a fixed fraction of the bound: about 3% for $\lambda = 0.01$, and 11% to 13% for $\lambda = 1$. Part of the slack is the ratio of about 0.25 in Table 7.2, and the rest is that the gap averages over test points and replacements, while the bound takes the worst of both.

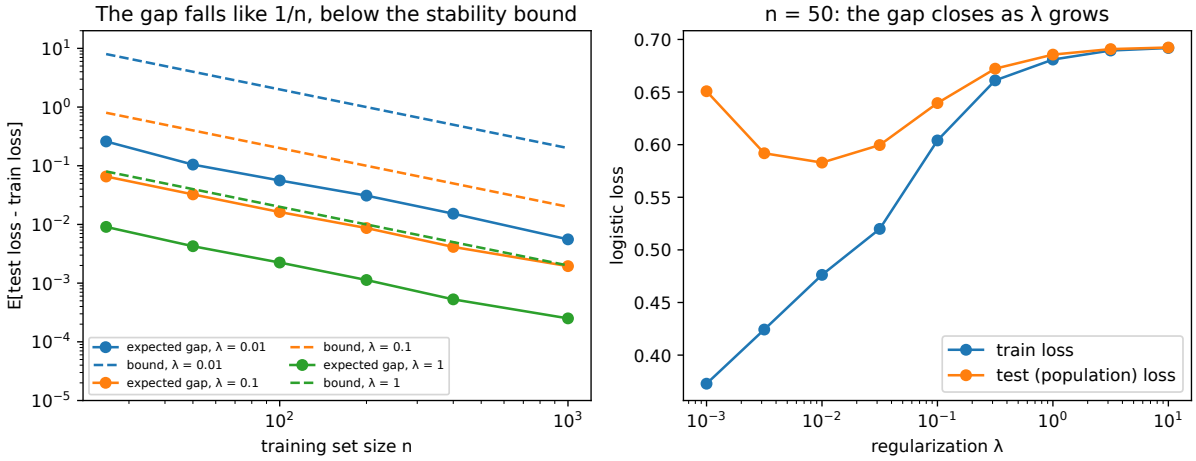


Figure 7.3.: Left: expected gap of regularized logistic regression against n (solid), with the bound $2/(\lambda n)$ (dashed). Right: train and test loss against λ at $n = 50$.

7.4. High probability: the Bousquet–Elisseeff bound

Equation 7.7 controls the gap on average, but you train once, so you need the gap of your model to be small with high probability. The gap is a function of n independent records, and stability bounds how much one record can move it, which is the setting of **McDiarmid's inequality** (Theorem 2.8). Suppose that changing only the record z_i changes a function $g(D)$ by at most c_i, whatever the values of the other records. Then for every $t > 0$,

$$\Pr\left[g(D) - \mathbb{E}\, g(D) \geq t\right] \leq \exp\Big(-\frac{2t^2}{\sum_i c_i^2}\Big). \tag{7.12}$$

Setting the right side equal to a failure probability β and solving for t gives the deviation $t = \sqrt{\frac{1}{2}\sum_i c_i^2 \ln(1/\beta)}$.

The theorem below adds this deviation and γ to the training loss. It is Theorem 12 of Bousquet & Elisseeff (2002), who define stability by removing a record; replace-one stability is at most twice that.

Theorem 7.3 (Bousquet–Elisseeff bound)**.** *Let $\mathcal{T}$ be γ-uniformly stable, let the loss take values in $[0, K]$, and let $\beta \in (0, 1)$. Then with probability at least $1 - \beta$ over the training set D,*

$$\mathcal{L}_{\mathcal{D}}(\mathcal{T}(D)) \le \mathcal{L}(\mathcal{T}(D); D) + \gamma + (2n\gamma + K)\sqrt{\frac{\ln(1/\beta)}{2n}}. \tag{7.13}$$

Proof. First bound how much one record can move the gap, then feed that to McDiarmid's inequality with $g = \text{gap}$, and bound the mean by Theorem 7.1.

1. *One record and the test loss.* Replacing z_i moves $\ell(\mathcal{T}(D); z)$ by at most γ at every z, by uniform stability. The test loss is an average of these losses over z, so it also moves by at most γ.
2. *One record and the training loss.* The training losses on D and on $D^{(i)}$ share $n - 1$ terms. Each shared term $\frac{1}{n}\ell(\cdot; z_j)$ is evaluated at the same point under two models, so by stability it moves by at most γ/n. The two unshared terms are evaluated at different points, z_i and z_i', so stability says nothing about them, but both lie in $[0, K/n]$. The total change is at most $\frac{n-1}{n}\gamma + \frac{K}{n} \le \gamma + \frac{K}{n}$.
3. *Bounded differences.* The gap is the test loss minus the training loss, so by the triangle inequality its change is at most $c_i = 2\gamma + K/n$. Uniform stability holds for every training set and replacement, so this holds for all values of the records, as McDiarmid's inequality requires.
4. *Apply McDiarmid's inequality.* All n constants are equal, so the deviation $\sqrt{\frac{1}{2}\sum_i c_i^2 \ln(1/\beta)}$ is $\sqrt{n}\,(2\gamma + K/n)\sqrt{\ln(1/\beta)/2}$. Multiplying inside and out by $\sqrt{n}$ rewrites it, so with probability at least $1 - \beta$,
$$\text{gap}(D) < \mathbb{E}[\text{gap}(D)] + (2n\gamma + K)\sqrt{\frac{\ln(1/\beta)}{2n}}. \tag{7.14}$$
5. *Bound the mean.* By Equation 7.7 the expected gap is at most γ. Substitute it, and move the training loss to the right-hand side.

□

Call the **width** of the bound everything it adds to the training loss. Its part $K\sqrt{\ln(1/\beta)/(2n)}$ is what Hoeffding's inequality gives for one fixed model. Its stability part, $2n\gamma\sqrt{\ln(1/\beta)/(2n)} = \gamma\sqrt{2n\ln(1/\beta)}$, goes to zero only if $\gamma\sqrt{n} \to 0$. With γ of order $1/n$, as for regularized ERM, the width falls like $1/\sqrt{n}$, and with γ of order $1/\sqrt{n}$ it does not fall at all (Figure 7.4).

The looseness comes from step 3, which charges every record the full 2γ although their effects largely average out. Later work (Feldman & Vondrák 2018, 2019; Bousquet, Klochkov & Zhivotovskiy 2020) reached, for $K = 1$ and up to constant factors (which is what $\lesssim$ means),

$$\text{gap} \lesssim \gamma \ln n \, \ln(1/\beta) + \sqrt{\frac{\ln(1/\beta)}{n}}, \tag{7.15}$$

so that γ of order $1/\sqrt{n}$ also gives a gap of order $1/\sqrt{n}$ up to log factors.

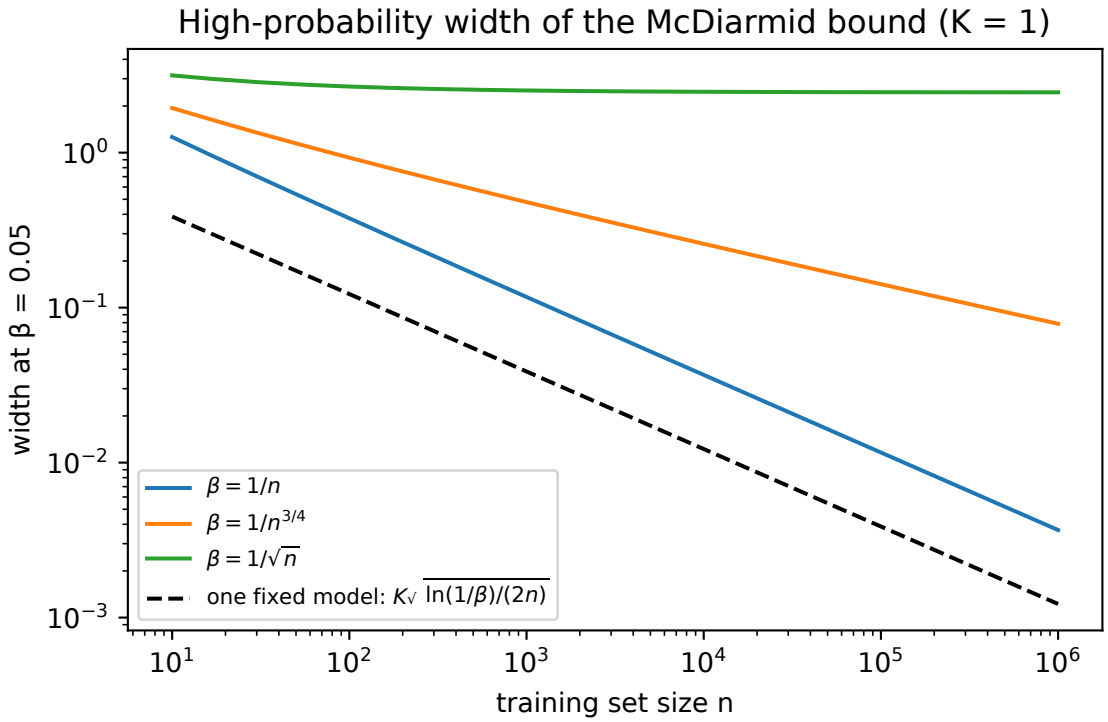


Figure 7.4.: Width of the high-probability bound, Equation 7.13, with $K = 1$ and $\beta = 0.05$, for stability $\gamma = 1/n$, $n^{-3/4}$ and $n^{-1/2}$, against the Hoeffding width for a single fixed model.

Example 7.5. By Theorem 7.2, regularized logistic regression in the running example has $\gamma = 2/(\lambda n)$ and weights of norm at most $1/\lambda$. By the Cauchy–Schwarz inequality with $\|x\| = 1$, the margin $y\langle\theta, x\rangle$ is at least $-1/\lambda$, so the largest loss is $K = \ln(1 + e^{1/\lambda})$. With $n = 10{,}000$ and $\beta = 0.05$ the common factor is $\sqrt{\ln 20/20{,}000} = 0.01224$, and with $2n\gamma = 4/\lambda$ the width is $\gamma + (4/\lambda + K) \times 0.01224$. At $\lambda = 1$, with $K = 1.313$, that is $2 \times 10^{-4} + (4 + 1.313) \times 0.01224 = 0.065$. At $\lambda = 0.1$, with $K = 10.000$, it is $0.002 + (40 + 10) \times 0.01224 = 0.614$.

In both cases the stability term, $2n\gamma \times 0.01224$, is three quarters or more of the width, while the expected gap at $\lambda = 1$ is only 2.53×10^{-4} even at the smaller $n = 1{,}000$ (Table 7.3). So the bound is valid but loose.

7.5. Stochastic gradient descent

Large models are trained by SGD, whose steps each pick one training record at random and move the weights a little against the gradient of its loss. Hardt, Recht & Singer (2016) showed that SGD is stable, with a γ that grows with the sum of the learning rates.

Two SGD runs, one on D and one on $D^{(i)}$, can only drift apart on the steps that pick record i. Each such step pushes them apart by at most $2\eta_t L$, where η_t is the learning rate at step t, and it

happens with probability $1/n$. On every other step both runs take a gradient step on the same loss, and the lemma below says that such a step never pushes two points apart. It needs the loss to be β_s**-smooth**: its gradient changes by at most β_s per unit of distance (Definition 6.2; the subscript keeps β for the failure probability).

Lemma 7.2 (A short gradient step is non-expansive)**.** *Let h be convex and β_s-smooth, let $0 \le \eta \le 2/\beta_s$, and let $\psi(\theta) = \theta - \eta\nabla h(\theta)$ be one gradient step. Then for all θ and θ',*

$$\|\psi(\theta) - \psi(\theta')\| \le \|\theta - \theta'\|. \tag{7.16}$$

Proof. We expand the squared distance after the step and bound its cross term by a property of the gradient.

1. *Co-coercivity.* For a convex β_s-smooth function, two gradients that differ by a lot must come from points far apart along the direction of their difference. Writing $g = \nabla h(\theta) - \nabla h(\theta')$,

$$\langle g, \theta - \theta' \rangle \ge \frac{1}{\beta_s}\|g\|^2. \tag{7.17}$$

   This is Theorem 2.1.5 of Nesterov (2004), and Section G.2 in the appendices derives it step by step.
2. *Expand the square.* The difference of the two steps is $(\theta - \theta') - \eta g$, so

$$\|\psi(\theta) - \psi(\theta')\|^2 = \|\theta - \theta'\|^2 - 2\eta\langle g, \theta - \theta' \rangle + \eta^2\|g\|^2. \tag{7.18}$$

3. *Use co-coercivity.* Replacing the inner product by the smaller $\|g\|^2/\beta_s$ of step 1 can only increase the right side, so

$$\|\psi(\theta) - \psi(\theta')\|^2 \le \|\theta - \theta'\|^2 - \eta\Big(\frac{2}{\beta_s} - \eta\Big)\|g\|^2. \tag{7.19}$$

4. *Conclude.* When $\eta \le 2/\beta_s$ the subtracted term is not negative, so the distance does not grow.

□

The condition on η is needed: on $h(\theta) = \frac{\beta_s}{2}\theta^2$ the step multiplies distances by $|1 - \eta\beta_s|$, which exceeds 1 when $\eta > 2/\beta_s$.

The stability of SGD, Theorem 7.4, is Theorem 3.8 of Hardt, Recht & Singer (2016).

Theorem 7.4 (Stability of SGD)**.** *Let $\ell(\cdot; z)$ be convex, β_s-smooth and L-Lipschitz for every record z. Run SGD for T steps from a starting point θ_0. Step t draws an index I_t uniformly from $\{1, \dots, n\}$, independently of the earlier steps, and updates*

$$\theta_t = \theta_{t-1} - \eta_t\nabla\ell(\theta_{t-1}; z_{I_t}), \qquad \eta_t \le \frac{2}{\beta_s}. \tag{7.20}$$

Let θ'_t be the run on $D^{(i)}$ with the same indices and starting point, and $r_t = \|\theta_t - \theta'_t\|$ the distance between the runs. Then $\mathbb{E}\, r_T \le \frac{2L}{n}\sum_{t=1}^{T}\eta_t$, and the output θ_T is uniformly stable in expectation over the indices, with

$$\gamma \le \frac{2L^2}{n}\sum_{t=1}^{T}\eta_t. \tag{7.21}$$

Proof. We couple the two runs, bound the growth of their distance in one step, average over the index, and add up the steps. Sharing the indices is allowed because the stability condition involves only the difference of the two runs' average losses, and each run's average is the same whichever indices the other run uses. Both runs start at θ_0, so $r_0 = 0$.

1. *Same record picked.* With probability $1 - 1/n$, the index I_t is not i, and both runs take a gradient step on the same loss. That step does not increase distances, by Lemma 7.2, so $r_t \le r_{t-1}$.
2. *Differing record picked.* With probability $1/n$, one run steps on z_i and the other on z_i', and the lemma does not apply. The new difference is the old one minus $\eta_t \nabla \ell(\theta_{t-1}; z_i)$ plus $\eta_t \nabla \ell(\theta_{t-1}'; z_i')$, and each gradient has norm at most L, so the triangle inequality gives
$$r_t \le r_{t-1} + 2\eta_t L. \tag{7.22}$$
3. *Average over the index.* The index I_t is drawn fresh, so it is independent of r_{t-1}. Weighting the two cases by their probabilities,
$$\mathbb{E}\, r_t \le \Big(1 - \frac{1}{n}\Big)\mathbb{E}\, r_{t-1} + \frac{1}{n}(\mathbb{E}\, r_{t-1} + 2\eta_t L) = \mathbb{E}\, r_{t-1} + \frac{2\eta_t L}{n}. \tag{7.23}$$
4. *Add up the steps.* Starting from $r_0 = 0$ and adding the increments of steps 1 to T gives $\mathbb{E}\, r_T \le \frac{2L}{n} \sum_t \eta_t$.
5. *From weights to losses.* The loss is L-Lipschitz, so at every test point the loss difference is at most $L\, r_T$ in absolute value, and its expectation is at most $L\, \mathbb{E}\, r_T$, which is Equation 7.21.

□

So stability degrades linearly with training time (Figure 7.5). With a constant step η and m epochs (passes over the data), so $T = mn$ steps, the bound is $\gamma \le 2L^2 \eta m$, and for a fixed number of epochs it does not improve with more data. Training longer lowers the training loss but raises γ, and the test loss is the training loss plus the gap, so there is a case for stopping early. Section G.3 in the appendices finds the training time that balances the two.

The bound is loose because step 1 only uses that a step does not expand distances. A step on a curved loss often contracts them, and a strongly convex loss makes the bound independent of T.

Example 7.6. Take $n = 100$ records of the running example, no penalty, and a constant step $\eta = 0.25$. At margin t the logistic loss has second derivative $\sigma(t)(1 - \sigma(t))$, a product of two numbers that sum to 1, so at most $\frac{1}{4}$. With $\|x\| \le 1$ that gives $\beta_s = \frac{1}{4}$, and $\eta \le 2/\beta_s = 8$ holds. The script averages over 100 coupled pairs of runs, with z_1 replaced by a fresh draw, and with $L = 1$ both bounds are $0.5m$.

Table 7.4.: SGD on unregularized logistic regression, $n = 100$, $\eta = 0.25$, averaged over 100 runs. With $L = 1$ the distance and loss bounds coincide.

Epochs	Train loss	Test loss	Gap	Bound $2L\eta m = 2L^2\eta m$	Mean $\|\theta_T - \theta_T'\|$
1	0.559	0.596	0.036	0.5	0.130
2	0.515	0.567	0.052	1.0	0.189

Epochs	Train loss	Test loss	Gap	Bound $2L\eta m = 2L^2\eta m$	Mean $\|\theta_T - \theta'_T\|$
5	0.478	0.557	0.079	2.5	0.277
10	0.470	0.567	0.097	5.0	0.363
20	0.467	0.582	0.115	10.0	0.355
40	0.467	0.588	0.121	20.0	0.395

The gap grows with the epochs, and the test loss is lowest after about 5 (Figure 7.6). The bound is loose by a factor of 14 after one epoch (0.5/0.036) and of 165 after 40 (20.0/0.121), because the distance between the runs levels off near 0.4 while the bound on it grows linearly.

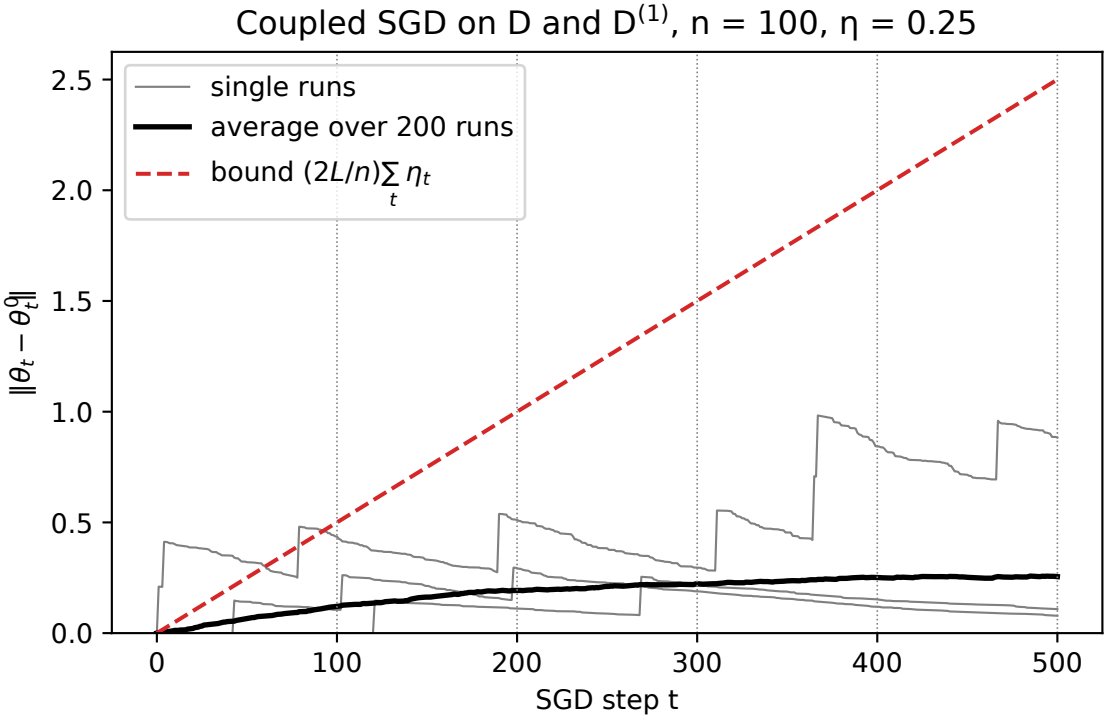


Figure 7.5.: Distance between coupled SGD runs on D and $D^{(1)}$ ($n = 100$, $\eta = 0.25$), three single runs and a separate average of 200, against the bound $(2L/n)\sum_t \eta_t$. Single runs jump when record 1 is drawn.

7.6. An unstable algorithm: 1-nearest neighbour

An algorithm without stability shows what stability rules out. The nearest neighbour (NN) rule with one neighbour, 1-NN, predicts the label of the closest training point, so it fits every training record by using that record itself. It is scored with the **0-1 loss**, which is 1 for a wrong prediction and 0 for a right one. With $p(x) = \Pr[y = 1 \mid x]$, the best rule, the **Bayes classifier**, predicts the more likely label at each x, with error $\mathbb{E}[\min(p(x), 1 - p(x))]$.

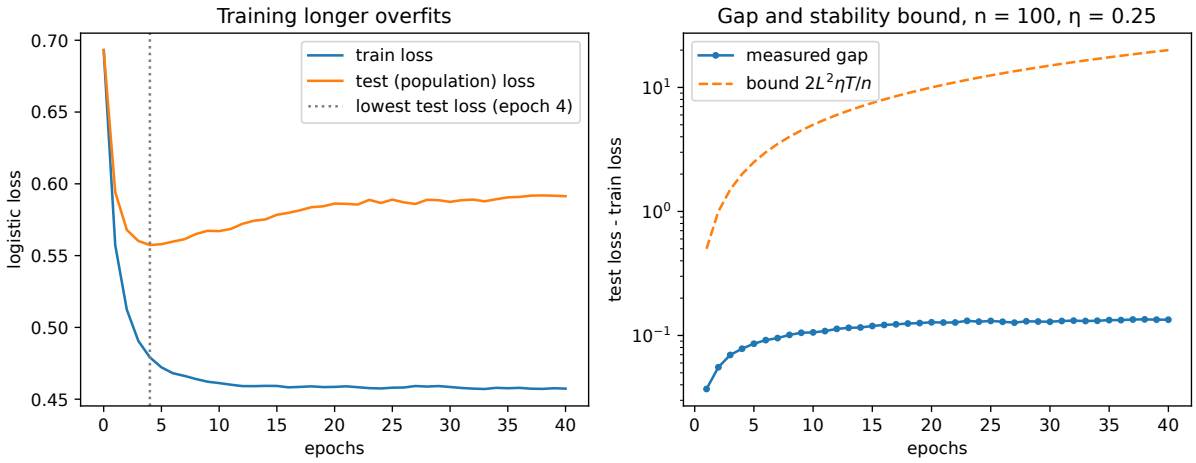


Figure 7.6.: Left: train and test loss of SGD against the number of epochs. Right: the measured gap against the bound $2L^2\eta T/n$, on a log scale.

Proposition 7.2 (1-NN is maximally unstable)**.** *Score 1-NN with the 0-1 loss, and assume the training points are distinct. Then its training error is 0, and it is γ-uniformly stable only for $\gamma = 1$, the largest possible value. If moreover the features take values in a closed and bounded set on which p is continuous, its expected test error tends to $\mathbb{E}[2p(x)(1-p(x))]$ as $n \to \infty$ (Cover & Hart 1967). So when $p(x)$ lies strictly between 0 and 1 with positive probability, its gap stays bounded away from zero.*

Proof.

1. *Training error.* Each training point is its own nearest neighbour, so it is predicted with its own label.
2. *Stability.* Take any D and the test point $z_i = (x_i, y_i)$. On D the prediction at x_i is y_i, with loss 0. Replace z_i by $(x_i, -y_i)$, the same point with the opposite label. On $D^{(i)}$ that point is the nearest neighbour of x_i, so the prediction is $-y_i$, with loss 1.
3. *The limit.* As n grows, the nearest neighbour of a test point x lies arbitrarily close to x, so its label is $+1$ with probability close to $p(x)$. The test label is an independent draw with probability $p(x)$, and the two disagree with probability $p(1-p) + (1-p)p = 2p(1-p)$. Making "arbitrarily close" precise is Cover and Hart's theorem, which Section G.5 in the appendices proves under the stated assumptions.
4. *The gap.* By step 1 the gap equals the test error, whose limit is positive under that condition on p.

□

So zero training error says nothing here. 1-NN is a reasonable classifier: $2p(1-p)$ is $2\min(p, 1-p)$ times $\max(p, 1-p) \leq 1$, so its limiting error is within twice the Bayes error. But its training error does not tell you how close it gets. With $\gamma = 1$, Equation 7.7 only says that the gap is at most 1, which every 0-1 loss satisfies anyway.

A majority vote over several nearest neighbours rarely changes when one record is replaced, although its worst-case γ is still 1, and notions of stability that average over the data capture its behaviour (Devroye & Wagner 1979).

Example 7.7. In the running example the Bayes error is 0.250 and the 1-NN limit $\mathbb{E}[2p(1-p)]$ is 0.338, both computed by numerical integration. Table 7.5 compares 1-NN with regularized logistic regression, averaging over 10 training sets, with 0-1 error measured on 2,000 test points.

Table 7.5.: 0-1 error of 1-nearest neighbour and of regularized logistic regression.

n	1-NN train	1-NN test	Logistic ($\lambda = 0.01$) train	Logistic test
50	0.000	0.404	0.170	0.301
200	0.000	0.380	0.232	0.274
1,000	0.000	0.367	0.244	0.266
4,000	0.000	0.352	0.249	0.259

The gap of 1-NN is its test error, which falls towards 0.338 and no further. The gap of the stable logistic regression falls from $0.301 - 0.170 = 0.131$ at $n = 50$ to $0.259 - 0.249 = 0.010$ at $n = 4{,}000$ (Figure 7.7).

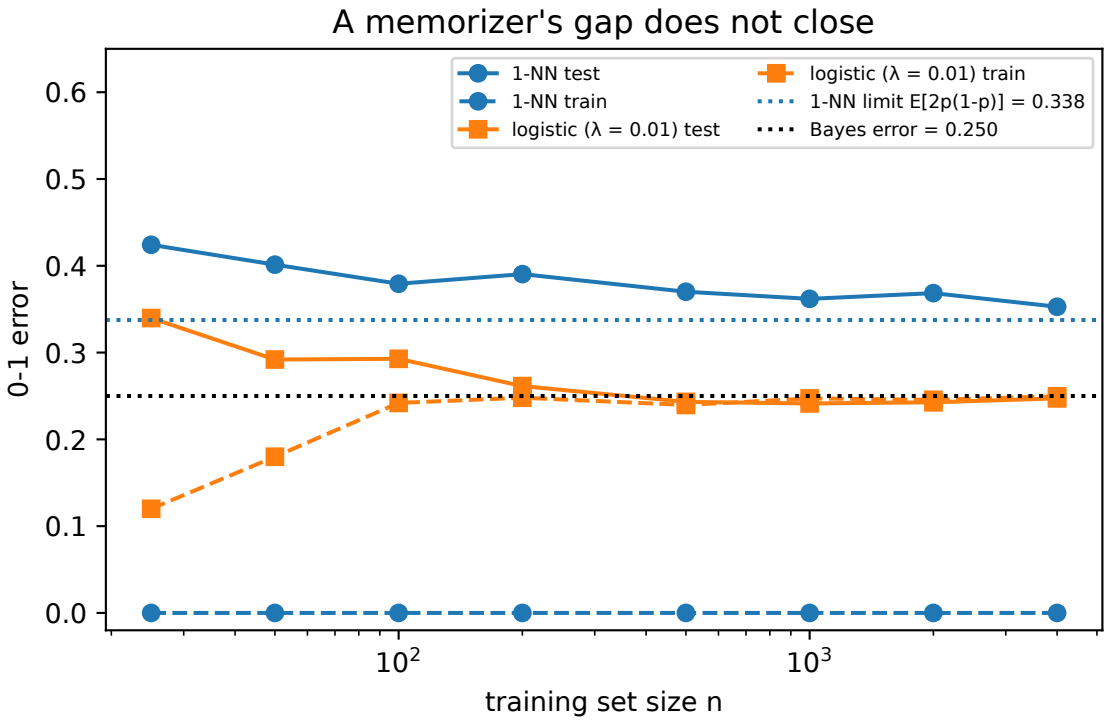


Figure 7.7.: Train (dashed) and test (solid) 0-1 error of 1-nearest neighbour and of regularized logistic regression against n, with the Bayes error and the 1-NN limit $\mathbb{E}[2p(1-p)]$.

7.7. Memorization of individual records

Theorem 7.1 says that the expected gap is an average over training records of how much each record's own loss rises when it is replaced. That quantity can be measured directly by retraining, and measuring it shows which records carry the gap.

Definition 7.3 (Self-influence)**.** The **self-influence**, or **memorization score**, of training record i is its loss under the model trained with a fresh point in its place, minus its loss under the model trained with it:

$$\mathrm{mem}_i(D) = \ell(\mathcal{T}(D^{(i)}); z_i) - \ell(\mathcal{T}(D); z_i). \tag{7.24}$$

By Equation 7.6, the expected gap is the average of the expected self-influences. Uniform stability caps each of them at γ, but only their average matters. Feldman (2020) uses the same idea with removal instead of replacement and accuracy instead of loss.

The contributions are uneven. Typical records are fitted well whether or not they are present, because similar records carry the same information, while atypical records, such as mislabelled points, are fitted only if they are in the training set. Memorization can still be necessary: Feldman (2020) shows that when the data contain many rare subpopulations, each seen about once, an algorithm that does not memorize them loses accuracy on them. Deep networks even fit random labels (Zhang et al. 2017), and score highest on atypical images (Feldman & Zhang 2020).

Example 7.8. The script draws training sets of $n = 40$ points and, for each i, replaces z_i by a fresh draw, refits, and records mem_i. Over 100 training sets, with $\pm$ one standard error, the measured gap and the average self-influence agree, as the identity requires. At $\lambda = 0.01$ they are 0.1482 ± 0.0073 and 0.1422 ± 0.0025, and at $\lambda = 0.1$ they are 0.0420 ± 0.0017 and 0.0416 ± 0.0003.

To see which records carry the gap, split them by whether the label goes with the trend ($y_i x_{i1} \geq 0$, the more likely label, since $\sigma(5x_1) > \frac{1}{2}$ exactly when $x_1 > 0$) or against it. About a quarter go against it (24% and 25% at the two values of λ). At $\lambda = 0.01$ their average self-influence is 0.284, three times the 0.097 of the others. At $\lambda = 0.1$ the two are 0.055 and 0.037, so regularization flattens the profile (Figure 7.8).

7.8. Choosing what to measure after looking

A holdout set is a sample kept aside for judging models, and by Proposition 7.1 it judges a model fixed in advance without bias. In practice you look at its verdict, change a feature, and ask again, so each new question depends on the holdout set. This section shows that the guarantee then fails, and what restores it.

Definition 7.4 (Questions to a holdout set)**.** A **holdout set** $H = (z_1, \dots, z_n)$ is a list of n records drawn independently from $\mathcal{D}$. A **question** is a function f that maps one record to a number in $[0, 1]$, such as "is this record classified correctly by a given model?". Its answer on the holdout set, written $f(H)$, and its true value are

$$f(H) = \frac{1}{n} \sum_{i=1}^{n} f(z_i), \qquad f(\mathcal{D}) = \mathbb{E}_{z \sim \mathcal{D}}[f(z)]. \tag{7.25}$$

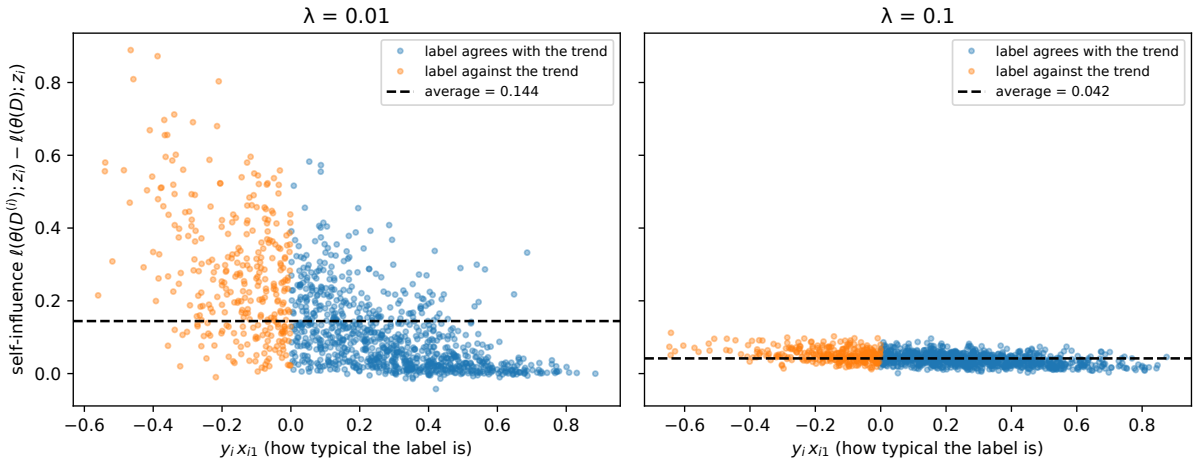


Figure 7.8.: Self-influence of each training record against $y_i x_{i1}$, which measures how typical its label is, for $n = 40$ and two values of λ. The dashed line is the average, which equals the expected gap.

A question is **fixed in advance** if it was chosen before H was drawn, and **chosen after looking** if it may depend on answers already given on H.

Many questions fixed in advance can share one holdout set, because the largest error among k answers grows only with the logarithm of k. The proof needs Hoeffding's lemma (Lemma 2.2): a centred random variable with values in an interval of length c has moment generating function at most $e^{s^2c^2/8}$ at every real s.

Theorem 7.5 (Questions fixed in advance)**.** *If k questions $f_1, \dots, f_k$ are all fixed in advance, then*

$$\mathbb{E}\Big[\max_{j \le k} \big|f_j(H) - f_j(\mathcal{D})\big|\Big] \le \sqrt{\frac{\ln(2k)}{2n}}. \tag{7.26}$$

Proof. The plan is to bound the moment generating function of one error, combine the $2k$ signed errors through the exponential of their maximum, and choose s to balance the two resulting terms.

1. *One error.* Write $Z_j = f_j(H) - f_j(\mathcal{D})$ for the error of question j. Because f_j is fixed while the records are drawn, Z_j is an average of n independent values $f_j(z_i)$ in $[0,1]$ minus their common mean. Each centred term $f_j(z_i)/n$ lies in an interval of length $1/n$, so Hoeffding's lemma bounds its moment generating function by $e^{s^2/(8n^2)}$. Multiplying the n independent bounds gives, for every real s,
$$\mathbb{E}\big[e^{sZ_j}\big] \le e^{s^2/(8n)}. \tag{7.27}$$
Since s may be negative, the same bound holds for $-Z_j$.
2. *The largest of $2k$.* The largest error is the maximum of the $2k$ variables $\pm Z_j$. For $s > 0$, Jensen's inequality for the convex function e^x moves the expectation inside the exponential, and the exponential of a maximum is at most the sum of the exponentials:
$$e^{s\,\mathbb{E}\max} \le \mathbb{E}\big[e^{s\max}\big] \le \sum_{\text{all } 2k} \mathbb{E}\big[e^{\pm sZ_j}\big] \le 2k\, e^{s^2/(8n)}. \tag{7.28}$$

3. *Take logarithms.* Dividing the logarithm by s bounds the expected maximum by a sum of two terms,
$$\mathbb{E}\max \le \frac{\ln(2k)}{s} + \frac{s}{8n}. \tag{7.29}$$
4. *Balance.* The two terms are equal, each $\sqrt{\ln(2k)/(8n)}$, at $s = \sqrt{8n\ln(2k)}$, and their sum is Equation 7.26. The argument never used independence between the questions.

□

Step 1 is where looking breaks the argument. A question chosen from earlier answers depends on H, so its values on H are not independent draws, just as in Section 7.1, and Example 7.9 shows how large the error can be. The remedy is to never let a record answer a question it helped to choose.

Proposition 7.3 (Sample splitting)**.** *Split H into k disjoint parts $H_1, \dots, H_k$ of n/k records each, and answer question j on H_j only. Let each f_j depend only on the answers on $H_1, \dots, H_{j-1}$ and on randomness of the analyst that is independent of H. Then, however the questions are chosen,*
$$\mathbb{E}\Big[\max_{j\le k} \big|f_j(H_j) - f_j(\mathcal{D})\big|\Big] \le \sqrt{\frac{k\ln(2k)}{2n}}. \tag{7.30}$$

Proof.

1. *Each question is fixed for its own part.* Fix the parts $H_1, \dots, H_{j-1}$ and the analyst's randomness. Then f_j is fixed and H_j is independent of it, so step 1 of the proof of Theorem 7.5 holds given those parts, with n/k records in place of n.
2. *Remove the conditioning.* That bound holds for every value of the earlier parts, so it holds after averaging over them.
3. *Finish as before.* Steps 2 to 4 of that proof then apply unchanged, with n/k in place of n.

□

A question chosen with the holdout set's help is like a model trained on it: the analyst is a learning algorithm whose input is H, and the answer to the final question plays the part of a training loss, optimistic for the same reason. Splitting removes the bias at the price of a factor k. If the questions come in r rounds, and every question of a round is fixed before that round's answers are seen, one part per round suffices, and the factor k in front of $\ln(2k)$ becomes r. The classical rule is one round: keep a test set for one final verdict, and use it once.

An analyst's choices ("keep the feature if it looks predictive") are thresholds, which one record can flip, so analysts are rarely stable. A later chapter reuses one holdout set safely for many adaptive questions with a randomized form of stability.

Example 7.9. The script builds a holdout set of $n = 1{,}000$ records with d random ± 1 features and a random label independent of them all, so no rule beats an accuracy of $\frac{1}{2}$ on new data. An analyst measures how often each feature agrees with the label, which is d questions fixed in advance, An agreement rate of a feature independent of the label is an average of n fair coin flips, with standard deviation $\sqrt{1/(4n)} = 0.0158$, and the analyst keeps the features more than that

far from $\frac{1}{2}$. The kept features then vote. A feature that agreed less than half the time votes the opposite of its value, and a tie counts as +1. Then the analyst asks one question chosen after looking: the vote's accuracy on the same holdout set (Table 7.6, 100 repetitions).

Table 7.6.: A holdout set of 1,000 records reused for a question chosen after looking, averaged over 100 repetitions. Every feature is independent of the label, so the true accuracy of any vote is $\frac{1}{2}$.

Features d	Kept	Largest error of the d fixed questions	Bound Equation 7.26	Vote, measured on H	Vote, chosen on one half and measured on the other	Vote on new data
100	33.3	0.0436	0.0515	0.6080	0.5032	0.5
1,000	325.9	0.0538	0.0616	0.8053	0.5018	0.5

The d fixed questions stay below the bound, while the question chosen after looking is off by 0.31 with 1,000 features. Every kept feature agrees with the holdout labels slightly more often than chance, and the vote adds these small agreements up. Choosing the features on one half and measuring on the other brings the accuracy back to $\frac{1}{2}$ (Figure 7.9), and two parts suffice because the questions come in two rounds, the d agreement rates and then the vote. This is Freedman's paradox (Freedman 1983), also a trap in cross-validation (Hastie, Tibshirani & Friedman 2009, §7.10.2).

7.9. Code

The script code/stability.py computes every number in this chapter and draws every figure. By symmetry the test loss depends only on θ_1 and the norm of the other coordinates, so it is computed exactly as a two-dimensional integral. From the repository root:

```
uv run python code/part01-prerequisites/stability.py
```

Table 7.7.: The script's output with the default seed.

Experiment	Result
Renaming (Table 7.1)	Train 0.0750; test 0.2850; renamed 0.2850; gap 0.2100; $\gamma = 0.75$
Replace-one change (Table 7.2)	0.14 to 0.25 of $2L^2/(\lambda n)$
Expected gap (Table 7.3)	About 3% of the bound at $\lambda = 0.01$, 11% to 13% at $\lambda = 1$
High-probability width, $n = 10{,}000$	0.065 at $\lambda = 1$; 0.614 at $\lambda = 0.1$

Experiment	Result
Self-influence, $n = 40$	Gap 0.1482 ± 0.0073 against 0.1422 ± 0.0025 ($\lambda = 0.01$); 0.0420 ± 0.0017 against 0.0416 ± 0.0003 ($\lambda = 0.1$)
SGD (Table 7.4)	Gap 0.036 after 1 epoch, 0.121 after 40; bound 0.5 and 20.0
1-NN (Table 7.5)	Test error 0.404 → 0.352; limit 0.338; Bayes 0.250
Reused holdout set (Table 7.6)	Vote 0.8053 on the set that chose it, 0.5018 with a split

tests/test_stability.py checks that the output still matches.

7.10. Summary

Table 7.8.: The results of this chapter, and where each is used.

Result	Statement	Use
Proposition 7.1	A fixed model's training loss is unbiased	Holdout sets used once; variance reduction
Lemma 7.1	Swapping two i.i.d. draws changes no expectation	The renaming step
Theorem 7.1	Expected gap = average replace-one effect; $\lvert\mathbb{E}[\text{gap}]\rvert \le \gamma$	Overfitting as dependence on single records; needs $\gamma \to 0$
Theorem 7.2	$\lVert\theta_D - \theta_{D^{(i)}}\rVert \le 2L/(\lambda n)$, $\gamma = 2L^2/(\lambda n)$	Choosing λ; $\gamma \propto 1/\lambda$
Theorem 7.3	gap $\le \gamma + (2n\gamma + K)\sqrt{\ln(1/\beta)/(2n)}$ w.p. $1 - \beta$	The model you trained; needs $\gamma\sqrt{n} \to 0$
Lemma 7.2	A step with $\eta \le 2/\beta_s$ on a convex β_s-smooth function is non-expansive	Coupled SGD runs
Theorem 7.4	$\gamma \le (2L^2/n)\sum_t \eta_t$	Epochs and early stopping; $\gamma \propto \eta T$
Proposition 7.2	1-NN: training error 0, $\gamma = 1$, test error → $\mathbb{E}[2p(1-p)]$	Training error 0 is a warning, not a result
Definition 7.3	Average self-influence = expected gap	Atypical records are memorized most
Theorem 7.5	$\sqrt{\ln(2k)/(2n)}$ for k questions fixed in advance	Comparing fixed models on one holdout set
Proposition 7.3	$\sqrt{k\ln(2k)/(2n)}$ with one part per question	Give each round its own data; use the test set once

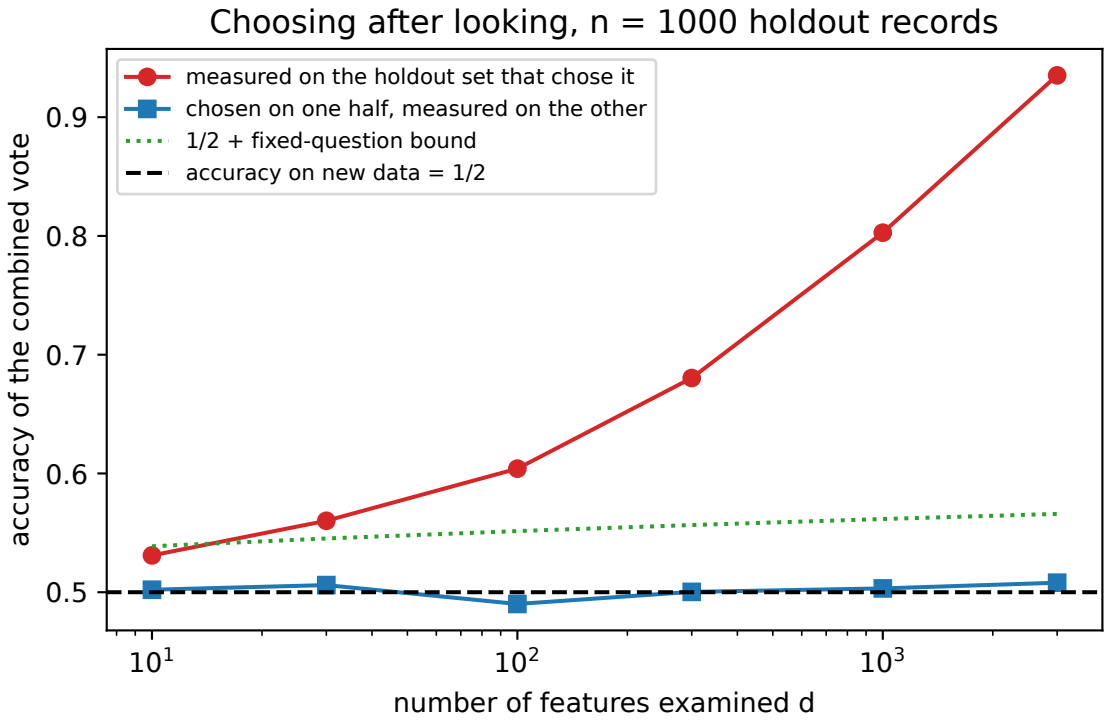


Figure 7.9.: Accuracy of the vote against the number of features examined, on a holdout set of 1,000 records with labels independent of every feature: measured on the set that chose the features, and chosen on one half and measured on the other.

7.11. References

- Bousquet & Elisseeff, “Stability and Generalization” (JMLR 2002): defines uniform and weaker stability notions; Theorem 12 is the high-probability bound.
- Shalev-Shwartz & Ben-David, *Understanding Machine Learning* (2014), ch. 13: the clearest textbook treatment; Theorem 13.2 is Equation 7.6 and Corollary 13.6 the regularized-ERM bound. Chapter 14 covers SGD convergence.
- Hardt, Recht & Singer, “Train Faster, Generalize Better: Stability of Stochastic Gradient Descent” (ICML 2016): SGD stability for convex (Theorem 3.8), strongly convex and non-convex losses.
- Feldman & Vondrák, “Generalization Bounds for Uniformly Stable Algorithms” (NeurIPS 2018) and “High Probability Generalization Bounds for Uniformly Stable Algorithms with Nearly Optimal Rate” (COLT 2019): the improved high-probability bounds.
- Bousquet, Klochkov & Zhivotovskiy, “Sharper Bounds for Uniformly Stable Algorithms” (COLT 2020): removes a log factor from the 2019 bound.
- Shalev-Shwartz, Shamir, Srebro & Sridharan, “Learnability, Stability and Uniform Convergence” (JMLR 2010): stability as the characterization of learnability.
- Cover & Hart, “Nearest Neighbor Pattern Classification” (IEEE Trans. Information Theory 1967): the 1-NN limit.
- Devroye & Wagner, “Distribution-Free Performance Bounds for Potential Function Rules” (IEEE Trans. Information Theory 1979): stability arguments for nearest-neighbour and local rules.
- Nesterov, *Introductory Lectures on Convex Optimization* (2004), §2.1: co-coercivity and smooth convex functions, used in Lemma 7.2.
- Feldman, “Does Learning Require Memorization? A Short Tale about a Long Tail” (STOC 2020): why memorizing rare records can be necessary for accuracy.
- Feldman & Zhang, “What Neural Networks Memorize and Why” (NeurIPS 2020): memorization scores measured on image benchmarks.
- Zhang, Bengio, Hardt, Recht & Vinyals, “Understanding Deep Learning Requires Rethinking Generalization” (ICLR 2017): networks fit random labels.
- Freedman, “A Note on Screening Regression Equations” (The American Statistician 1983): selecting variables and testing them on the same data, the paradox of Section 7.8.
- Hastie, Tibshirani & Friedman, *The Elements of Statistical Learning* (2nd ed. 2009), §7.10.2: “the wrong and the right way to do cross-validation”, the same trap in model selection.

8. Cryptography basics

Some computations must hide something, such as a key, a message, or a value split among several holders. **What does it mean that an observer with limited computing power learns nothing it was not supposed to learn?** This chapter gives cryptography's answer as mathematics, with toy constructions you can run.

One idea organizes the answer: **security is a game**. A challenger flips a fair coin to pick one of two worlds, runs a procedure in that world, and shows the result to an adversary, who guesses the world. An adversary that learns nothing is right half the time, and its **advantage** measures how much better it does. A scheme is secure when every efficient adversary's advantage is tiny. Proofs walk from one world to the other through intermediate worlds called **hybrids**, each differing from the next in one detail, and add up the advantage gained at each step.

Two examples run through the chapter. The first is a coin with heads probability 0.51, to be told apart from a fair coin. One flip gives an advantage of at most 0.01, and 10,000 flips give 0.683, far below the 1 that adding 0.01 per flip, as the hybrid argument does, would suggest. The second is the 128-bit size of keys and cipher blocks used in practice. Testing 10^{18} keys a second for a year finds a 128-bit key with probability 9.3×10^{-14}, yet a cipher that encrypts 128-bit blocks loses its security guarantee after about 2^{64} blocks. Table 8.1 lists the symbols, with Q and ν for the number of queries an adversary makes and its advantage, which cryptography texts call q and ε.

Table 8.1.: The symbols of this chapter.

Symbol	Meaning
$\mathcal{A}$; $\mathcal{B}$	an adversary; one built from it by a reduction
ν; λ	an advantage; the security parameter
Q	the number of samples, queries or draws
P_0, P_1	the two worlds a game compares
ξ	the coin's extra heads probability, 0.01
n	the bit length of messages and inputs
K; m; y	a random key; a message; a ciphertext
p; h	a modulus, not a density; the number of share holders
d, s, e	the dimension, secret and error of learning with errors

8.1. Distinguishers and advantage

Every definition in this chapter compares two worlds, so you first need a score for an observer who tries to tell them apart. This section defines that score and shows that it measures the same thing as winning a guessing game.

Definition 8.1 (Distinguisher and advantage)**.** A **distinguisher** $\mathcal{A}$ is any procedure, efficient or not, that receives one sample and outputs a bit. Its **advantage** in telling a distribution P_0 from a distribution P_1 is

$$\mathrm{Adv}_{\mathcal{A}}(P_0, P_1) = \big|\Pr[\mathcal{A}(P_1) = 1] - \Pr[\mathcal{A}(P_0) = 1]\big|, \tag{8.1}$$

where $\Pr[\mathcal{A}(P_1) = 1]$ is the probability that $\mathcal{A}$ outputs 1 on a sample from P_1. In the **distinguishing game**, a challenger flips a fair bit b and gives $\mathcal{A}$ a sample from P_b, and $\mathcal{A}$ **wins** if its guess b' equals b.

The coin in the game is fair, so an adversary with nothing to go on wins half the time whatever it answers. The next lemma turns win rates into advantages and back.

Lemma 8.1 (Winning and advantage)**.** *For any distinguisher $\mathcal{A}$ in the distinguishing game,*

$$\Pr[b' = b] = \frac{1}{2} + \frac{1}{2}\big(\Pr[\mathcal{A}(P_1) = 1] - \Pr[\mathcal{A}(P_0) = 1]\big). \tag{8.2}$$

So a distinguisher with advantage ν, flipped if needed as in step 3 below, yields one that wins with probability $\frac{1}{2} + \frac{\nu}{2}$.

Proof.

1. *Split the win by world.* Each world has probability $\frac{1}{2}$, and $\mathcal{A}$ wins in world 1 by saying 1 and in world 0 by saying 0, so
$$\Pr[b' = b] = \tfrac{1}{2}\Pr[\mathcal{A}(P_1) = 1] + \tfrac{1}{2}\Pr[\mathcal{A}(P_0) = 0]. \tag{8.3}$$
2. *Count only the outputs of 1.* Saying 0 is the complement of saying 1, so replacing $\Pr[\mathcal{A}(P_0) = 0]$ by $1 - \Pr[\mathcal{A}(P_0) = 1]$ and collecting terms gives Equation 8.2.
3. *Flip a losing distinguisher.* If the bracket is negative, the distinguisher that outputs the opposite bit negates the bracket, keeps the advantage, and wins with probability $\frac{1}{2} + \frac{\nu}{2}$.

□

Write $\mathrm{TV}(P_0, P_1)$ for the **total variation (TV) distance**, the sum, over the outcomes that P_1 makes likelier than P_0 does, of the extra probability P_1 gives them. Divergences shows that the best distinguisher, which says 1 exactly on those outcomes, wins with probability $\frac{1}{2} + \frac{1}{2}\mathrm{TV}(P_0, P_1)$. Since an advantage ν gives the win probability $\frac{1}{2} + \frac{\nu}{2}$ by Lemma 8.1, no advantage exceeds the TV distance, however much time the distinguisher uses:

$$\mathrm{Adv}_{\mathcal{A}}(P_0, P_1) \le \mathrm{TV}(P_0, P_1). \tag{8.4}$$

Two worlds at TV distance 0 give **perfect** security, and at a small one **statistical** security. Cryptography's main business is the third case, where the TV distance is large but no *efficient* distinguisher can exploit it.

Example 8.1. Let P_0 be a fair coin and P_1 a coin with heads probability 0.6. The distinguisher outputs 1 on heads. Each row of Table 8.2 is one outcome, with probability $\frac{1}{2}$ for the challenger's bit times the probability of the coin's outcome.

Table 8.2.: The distinguishing game for a fair against a 0.6-biased coin, with the distinguisher "output 1 on heads".

b	Coin	Outcome	Probability	$\mathcal{A}$ outputs	$\mathcal{A}$ wins?
0	fair	tails	0.25	0	yes
0	fair	heads	0.25	1	no
1	biased	tails	0.20	0	no
1	biased	heads	0.30	1	yes

The adversary wins with probability $0.25 + 0.30 = 0.55$. Its advantage is $0.6 - 0.5 = 0.1$, and indeed $0.55 = \frac{1}{2} + \frac{0.1}{2}$. The excess of heads, 0.1, is also the TV distance, so no distinguisher does better from one flip, and for the running example's coin the best is 0.01.

8.2. Negligible functions

A key of 40 bits can be found by trying every key, and a key of 256 bits cannot. So security says how fast the attacker's success shrinks as a **security parameter** λ, the key length in bits, grows. Calling a scheme secure first needs a notion of efficient attackers and of tiny success probabilities.

Definition 8.2 (Efficient adversaries and negligible functions)**.** An adversary is **probabilistic polynomial time (PPT)** if it is a randomized algorithm whose running time is bounded by a polynomial in λ. A function $\mu\colon \mathbb{N} \to [0,1]$ is **negligible** if, for every exponent c, it eventually falls below λ^{-c} and stays there:

$$\forall c > 0 \;\; \exists \lambda_0 \;\; \forall \lambda \geq \lambda_0 : \quad \mu(\lambda) < \lambda^{-c}. \tag{8.5}$$

A scheme is **secure** if every PPT adversary has negligible advantage in its security game.

An efficient adversary can repeat its attack polynomially many times, which by the union bound of Concentration inequalities multiplies its success by at most that polynomial. So "tiny" must stay tiny after multiplication by any polynomial, and the next proposition says that it does.

Proposition 8.1 (Negligible functions survive polynomial repetition)**.** *If μ is negligible and* poly *is a polynomial, then* $\mathrm{poly}(\lambda)\mu(\lambda)$ *is negligible. The sum of two negligible functions is negligible.*

Proof.

1. *Bound the polynomial by a power.* A polynomial of degree $c' - 1$ is eventually below $\lambda^{c'}$, so $\mathrm{poly}(\lambda) \leq \lambda^{c'}$ for all $\lambda \geq \lambda_1$.
2. *Ask μ for a steeper decay.* Fix any $c > 0$. Applying Equation 8.5 with the exponent $c + c'$ gives $\mu(\lambda) < \lambda^{-(c+c')}$ for all $\lambda \geq \lambda_2$.

3. *Multiply the two bounds.* For $\lambda \geq \max(\lambda_1, \lambda_2)$,

$$\mathrm{poly}(\lambda)\mu(\lambda) < \lambda^{c'}\lambda^{-(c+c')} = \lambda^{-c}, \tag{8.6}$$

   and since c was arbitrary, the product is negligible.
4. *Handle a sum.* With the exponent $c+1$, each of two negligible functions is eventually below $\lambda^{-(c+1)}$, so their sum is below $2\lambda^{-(c+1)}$, which is at most λ^{-c} once $\lambda \geq 2$.

□

Brute force, trying t of the 2^λ keys, succeeds with probability $t/2^\lambda$, negligible for polynomial t but never zero. In logarithms, μ is negligible when $-\log_2 \mu(\lambda)$ outgrows every multiple $c \log_2 \lambda$. So $2^{-\lambda}$, $\lambda^{-\log_2 \lambda}$ and $2^{-\sqrt{\lambda}}$ are negligible, but λ^{-10}, with $-\log_2 \mu = 10 \log_2 \lambda$, is never below λ^{-11}.

Example 8.2. At a fixed key length, "negligible" can mislead. Table 8.3 evaluates the four functions at $\lambda = 128 = 2^7$.

Table 8.3.: Four functions at $\lambda = 128$.

$\mu(\lambda)$	Negligible?	Value at $\lambda = 128$
$2^{-\lambda}$	yes	$2^{-128} = 2.9 \times 10^{-39}$
$\lambda^{-\log_2 \lambda}$	yes	$2^{-49} = 1.8 \times 10^{-15}$
$2^{-\sqrt{\lambda}}$	yes	$2^{-11.31} = 3.9 \times 10^{-4}$
λ^{-10}	no	$2^{-70} = 8.5 \times 10^{-22}$

The non-negligible λ^{-10} is smaller than two of the negligible functions, and $2^{-\sqrt{\lambda}}$ drops below it for good only at $\lambda = 20{,}520$. Figure 8.1 plots the **bits of security** $-\log_2 \mu(\lambda)$, the number of halvings from 1 down to $\mu(\lambda)$.

8.3. Indistinguishability and the hybrid argument

Most security definitions say that two things look the same, such as a ciphertext and random bits. This section makes that precise and answers the first question every proof meets: if one sample cannot be told apart, can many?

Definition 8.3 (Computational indistinguishability)**.** Two families of distributions $P_0 = \{P_{0,\lambda}\}$ and $P_1 = \{P_{1,\lambda}\}$, one distribution for each λ, are **computationally indistinguishable**, written $P_0 \approx_c P_1$, if every PPT distinguisher has negligible advantage $\mathrm{Adv}_{\mathcal{A}}(P_{0,\lambda}, P_{1,\lambda})$. If their TV distance is negligible they are **statistically indistinguishable**, written $\approx_s$, and if they are equal they are **perfectly indistinguishable**, written $\equiv$.

By Equation 8.4, $\equiv$ implies $\approx_s$, which implies $\approx_c$. A hybrid argument needs to know how advantages combine along a chain of worlds, and the next lemma says they add at most.

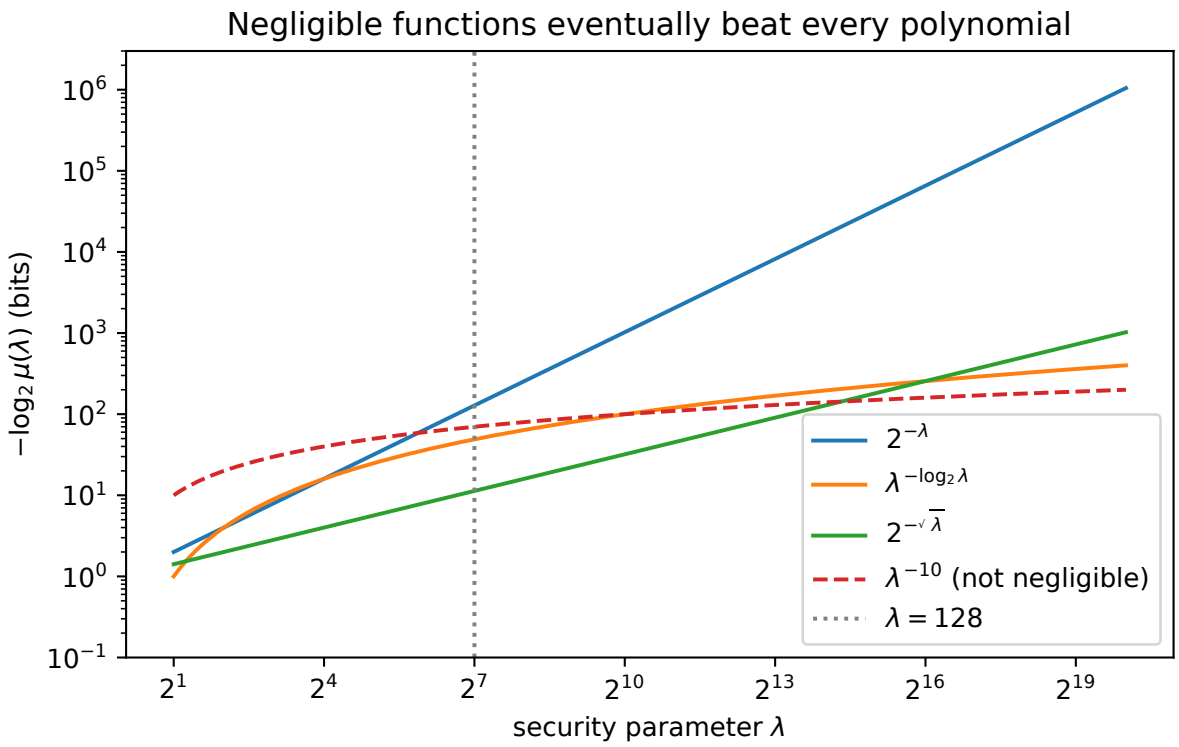


Figure 8.1.: Bits of security $-\log_2 \mu(\lambda)$ for three negligible functions and the non-negligible λ^{-10}, on log–log axes. Each negligible curve eventually overtakes the dashed one.

Lemma 8.2 (Advantages add along a chain)**.** *For any distributions $H_0, H_1, \dots, H_Q$ and any distinguisher $\mathcal{A}$,*

$$\mathrm{Adv}_{\mathcal{A}}(H_0, H_Q) \le \sum_{i=1}^{Q} \mathrm{Adv}_{\mathcal{A}}(H_{i-1}, H_i). \tag{8.7}$$

Proof.

1. *Telescope.* In the sum over i of $\Pr[\mathcal{A}(H_{i-1}) = 1] - \Pr[\mathcal{A}(H_i) = 1]$, each intermediate probability appears once with each sign, so the sum is $\Pr[\mathcal{A}(H_0) = 1] - \Pr[\mathcal{A}(H_Q) = 1]$.
2. *Take absolute values.* The absolute value of a sum is at most the sum of the absolute values.

□

The hybrid lemma walks from Q samples of P_0 to Q samples of P_1 by swapping one sample at a time. If the two ends are ν apart, the Q steps share that ν, so a step is worth ν/Q on average, and since a step changes one sample, it is a one-sample problem. Write $P^{\otimes Q}$ for Q independent samples from P.

Theorem 8.1 (The hybrid lemma)**.** *Let $P_0 \approx_c P_1$ be families that an efficient algorithm can sample from, and let $Q = Q(\lambda)$ be a polynomial. Then $P_0^{\otimes Q} \approx_c P_1^{\otimes Q}$. Quantitatively, a distinguisher with advantage ν on Q samples yields a distinguisher with advantage ν/Q on one sample, with about the same running time.*

Proof. The proof builds a one-sample distinguisher that lands at a random link of a chain of hybrids, computes its advantage by telescoping, and concludes by contradiction.

1. *Define the hybrids.* Let H_i be Q independent entries, the first i from P_1 and the rest from P_0, so $H_0 = P_0^{\otimes Q}$ and $H_Q = P_1^{\otimes Q}$.
2. *Build a one-sample distinguisher.* On input w, let $\mathcal{B}$ pick a position i uniformly from $\{1, \dots, Q\}$, draw the other entries itself as in H_i, which needs efficient sampling, put w in position i, and run $\mathcal{A}$. The tuple is distributed as H_{i-1} if w comes from P_0, and as H_i if it comes from P_1.
3. *Average over the position.* Each i has probability $\frac{1}{Q}$, so

$$\Pr[\mathcal{B}(P_0) = 1] - \Pr[\mathcal{B}(P_1) = 1] = \frac{1}{Q} \sum_{i=1}^{Q} \left(\Pr[\mathcal{A}(H_{i-1}) = 1] - \Pr[\mathcal{A}(H_i) = 1] \right). \tag{8.8}$$

4. *Telescope.* As in the proof of Lemma 8.2, the sum equals $\Pr[\mathcal{A}(H_0) = 1] - \Pr[\mathcal{A}(H_Q) = 1]$, so $\mathcal{B}$ has advantage ν/Q.
5. *Conclude by contradiction.* If ν were not negligible, there would be a c with $\nu(\lambda) \ge \lambda^{-c}$ for infinitely many λ. With $Q(\lambda) \le \lambda^{c'}$, the efficient $\mathcal{B}$ would then have advantage at least $\lambda^{-(c+c')}$ for those λ, which contradicts $P_0 \approx_c P_1$.

□

Example 8.3. Let P_0 be a fair coin and P_1 the running example's coin, with heads probability $\frac{1}{2} + \xi$. One flip gives an advantage of at most ξ, so the hybrid lemma, turned around, allows at most $Q\xi$ from Q flips. The factor Q is pessimistic, because the best attacker pools its flips into one count of heads. Table 8.4 and Figure 8.2 compare that bound with the exact best advantage and with majority vote, which outputs 1 when more than half the flips are heads.

Table 8.4.: Advantage in telling Q flips of a 0.51-coin from Q fair flips (seed 0, 200,000 simulated games per entry).

Flips Q	Majority vote (simulation)	Best possible (TV)	Hybrid bound $\min(1, Q\xi)$
1	0.008	0.010	0.010
100	0.082	0.080	1.000
10,000	0.481	0.683	1.000

Majority vote's 0.082, above the best possible 0.080, is simulation noise. Both counts of heads have a standard deviation of about $\sqrt{Q}/2$, and their means differ by $Q\xi$, so they are close to bell curves whose centres are $2\xi\sqrt{Q}$ standard deviations apart. Two normal curves whose centres are δ standard deviations apart have TV distance $2\Phi(\delta/2) - 1$, with Φ the standard normal distribution function, so the best advantage is about

$$2\Phi(\xi\sqrt{Q}) - 1, \tag{8.9}$$

which grows like $\xi\sqrt{Q}$ rather than ξQ. Majority vote stalls near $\frac{1}{2}$, because its threshold sits at the centre of the fair count's curve, so on fair flips it still says 1 about half the time.

8.4. Concrete security

Real systems fix λ, and a statement for all large enough λ does not say whether 128 is large enough. Concrete security replaces "efficient" and "negligible" by explicit numbers, so that a proof says how much a key can withstand.

Definition 8.4 (Concrete security and reductions)**.** A primitive is (t, Q, ν)**-secure** if every adversary that runs in time at most t and makes at most Q queries to its **oracles**, the procedures the game lets it call, has advantage at most ν. It has κ**-bit security** if, roughly, every adversary with time t has advantage at most $t/2^\kappa$, as for brute force over κ-bit keys. A **reduction** turns an adversary that breaks a scheme with (t, Q, ν) into one that breaks a primitive the scheme is built from with (t', Q', ν'), and it is **tight** when $\nu' \approx \nu$.

So a reduction says that a hard primitive makes a hard scheme, and a loose one costs bits. If an attack of advantage ν on the scheme gives only $\nu/2^{20}$ on a 128-bit primitive, the scheme keeps only $128 - 20 = 108$ bits.

Example 8.4. An exascale machine that tests 10^{18} keys per second for a year of about 3.16×10^7 seconds tries $2^{84.7}$ keys. Against a 128-bit key it succeeds with probability $2^{84.7-128} = 9.3 \times 10^{-14}$, and trying all 2^{128} keys would take 1.08×10^{13} years.

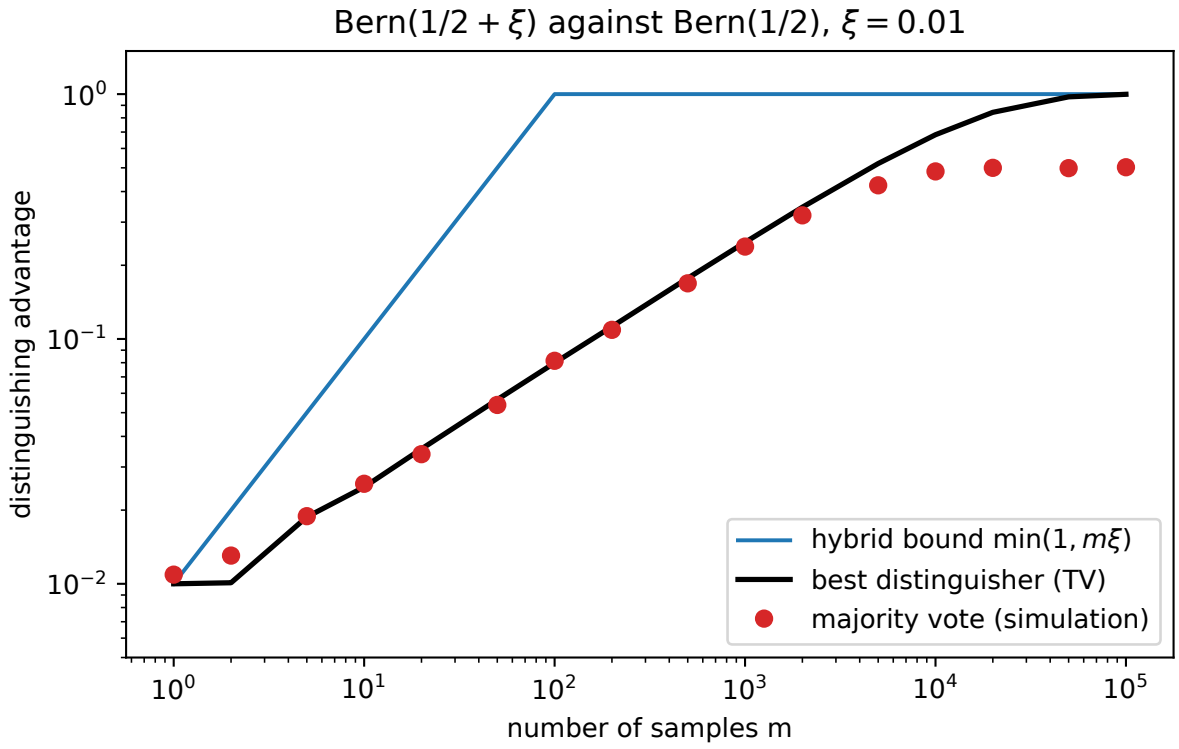


Figure 8.2.: Advantage from Q flips of the 0.51 coin: majority vote (simulation), the best possible distinguisher (exact TV distance), and the hybrid bound $Q\xi$.

8.5. Pseudorandom generators and functions

The schemes later in this chapter need many random-looking values from one short random key, because truly random bits are expensive to share. This section defines the two objects that supply them, generators and functions, and builds the second from the first.

Definition 8.5 (Pseudorandom generator)**.** Write U_ℓ for the uniform distribution on ℓ-bit strings. An efficient deterministic map G from λ-bit strings to $\ell(\lambda)$-bit strings, with $\ell(\lambda) > \lambda$, is a **pseudorandom generator (PRG)** if

$$G(U_\lambda) \approx_c U_{\ell(\lambda)}. \tag{8.10}$$

Pseudorandomness is a statement about efficient distinguishers only, because the outputs of G form a tiny subset of all strings.

Proposition 8.2 (A pseudorandom generator is never statistically secure)**.** *For every G as in Definition 8.5,* $\mathrm{TV}(G(U_\lambda), U_\ell) \geq \frac{1}{2}$.

Proof.

1. *An exhaustive distinguisher.* Let $\mathcal{A}$ output 1 exactly when its input equals $G(u)$ for some seed u. On $G(U_\lambda)$ it always outputs 1.
2. *Few strings are outputs.* At most 2^λ of the 2^ℓ strings are outputs, so on U_ℓ, $\mathcal{A}$ outputs 1 with probability at most $2^{\lambda-\ell} \leq \frac{1}{2}$.
3. *Bound the distance.* So $\mathcal{A}$ has advantage at least $\frac{1}{2}$, and Equation 8.4 gives the bound.

□

This distinguisher loops over 2^λ seeds, so it is not efficient,. A **random function** from a set $\mathcal{X}$ to a set $\mathcal{Y}$ gives each input its own independent, uniform output. A pseudorandom function must look like one even to an adversary who chooses its questions.

Definition 8.6 (Pseudorandom function)**.** A keyed function $F_K \colon \mathcal{X} \to \mathcal{Y}$, with a λ-bit key K, is a **pseudorandom function (PRF)** if every PPT adversary has negligible advantage in this game. The challenger flips b, and its oracle is F_K for a uniform key K if $b = 0$ and a random function if $b = 1$. The adversary sends Q queries, each of which may depend on earlier answers, and guesses b.

A random function on 128-bit inputs is a table of 2^{128} entries, which nobody can store, and a PRF gives the same appearance from a 128-bit key. The toy code uses the hash-based message authentication code (HMAC), a keyed hash that practice treats as a PRF.

8.5.1. The GGM tree

A PRF can be built from any PRG that doubles its input's length. The construction of Goldreich, Goldwasser & Micali (1986), the GGM tree, uses the generator to grow a binary tree of keys, picking the left or right half of the output by the next bit of the input.

Theorem 8.2 (The GGM tree is a pseudorandom function)**.** *Let G map λ bits to 2λ bits, with output halves $G_0(u)$ and $G_1(u)$. For an input $x = x_1 \cdots x_n$, start at the root K, replace the current node v by $G_{x_i}(v)$ for $i = 1, \ldots, n$, and let $F_K(x)$ be the leaf reached. With advantages in the games of Definition 8.6 and Definition 8.5 written* $\mathrm{Adv}_{\mathrm{PRF}}$ *and* $\mathrm{Adv}_{\mathrm{PRG}}$*, for every adversary $\mathcal{A}$ making Q queries there is a PRG adversary $\mathcal{B}$ with about the same running time such that*

$$\mathrm{Adv}_{\mathrm{PRF}}(\mathcal{A}) \le nQ \cdot \mathrm{Adv}_{\mathrm{PRG}}(\mathcal{B}). \tag{8.11}$$

Goldreich (2001, §3.6) proves it, and Appendix Section I.1 derives it step by step. It makes the tree random one level at a time, and each of the n levels costs at most Q times the PRG advantage, because the queries reach at most Q nodes per level, which Theorem 8.1 swaps one at a time.

Example 8.5. On the 2-bit input 10, the tree computes $G_1(K)$ and then outputs the left half of G applied to that node, $G_0(G_1(K))$. For 128-bit inputs, one evaluation walks 128 levels and costs 128 generator calls.

8.6. The birthday bound

The proofs of the next two sections fail only if two random values coincide, such as a repeated nonce, a random value meant to be used once, so you need the probability of a coincidence. Among Q values there are about $Q^2/2$ pairs, and each coincides with probability $1/N$, so collisions become likely after about $\sqrt{N}$ draws rather than N.

Theorem 8.3 (The birthday bound)**.** *Draw Q values independently and uniformly from a set of size N. Then*

$$1 - e^{-Q(Q-1)/(2N)} \;\le\; \Pr[\mathit{collision}] \;\le\; \frac{Q(Q-1)}{2N} \le \frac{Q^2}{2N}, \tag{8.12}$$

and the collision probability reaches $\frac{1}{2}$ after about $1.18\sqrt{N}$ draws.

Proof. The upper bound counts pairs. The lower bound bounds the exact probability of no collision by an exponential.

1. *Upper bound.* Each of the $Q(Q-1)/2$ pairs coincides with probability $1/N$, so the union bound gives the upper bound.
2. *Exact value.* Draw the values one at a time. If the first i are distinct, the next avoids them with probability $1 - i/N$, so

$$\Pr[\text{no collision}] = \prod_{i=0}^{Q-1} \Big(1 - \frac{i}{N}\Big). \tag{8.13}$$

3. *Lower bound.* The inequality $1 - x \le e^{-x}$ of Concentration inequalities bounds each factor, and the exponents add to $Q(Q-1)/(2N)$. Subtracting from 1 gives the lower bound.
4. *The 50% point.* Setting the lower bound to $\frac{1}{2}$, with $Q(Q-1) \approx Q^2$, gives $Q^2/(2N) = \ln 2$, so $Q \approx \sqrt{2N \ln 2} \approx 1.18\sqrt{N}$.

□

The two bounds are close, because $1 - e^{-x}$ bends downwards and so lies above its chord $(1 - 1/e)x$ on $[0, 1]$, which makes the lower bound at least $1 - 1/e$ times the upper one. With $N = 2^{128}$, random 128-bit values start to collide after about 2^{64} draws.

Example 8.6. Table 8.5 gives the exact probability and both bounds for three cases. For the class of 23, the union bound is $23 \times 22/730 = 0.693$.

Table 8.5.: Collision probability, exact against the two bounds of Equation 8.12.

Case	Q	N	Exact	Union bound	$1 - e^{-Q(Q-1)/(2N)}$
Shared birthday in a class	23	365	0.507	0.693	0.500
Hashing trick: 1,000 features into 2^{20} buckets	1,000	2^{20}	0.379	0.476	0.379
Random 32-bit identifiers for 100,000 records	100,000	2^{32}	0.688	1.164	0.688

Figure 8.3 checks Equation 8.12 for $N = 2^{16}$. At $Q = 256$ the exact probability is 0.3927, a simulation of 100,000 trials gives 0.3920, and the union bound gives 0.4980. The 50% point is $Q = 301$, against $1.18 \times 256 \approx 302$.

8.7. Block ciphers and the switching lemma

A block cipher such as the Advanced Encryption Standard (AES) is a keyed *permutation*, which never gives two inputs the same output, while most proofs assume a PRF. This section bridges the gap and turns the birthday bound into a limit on how much data one key may encrypt. A random function sometimes repeats an output and a random permutation never does, so an adversary can tell them apart only about as often as Q random values collide.

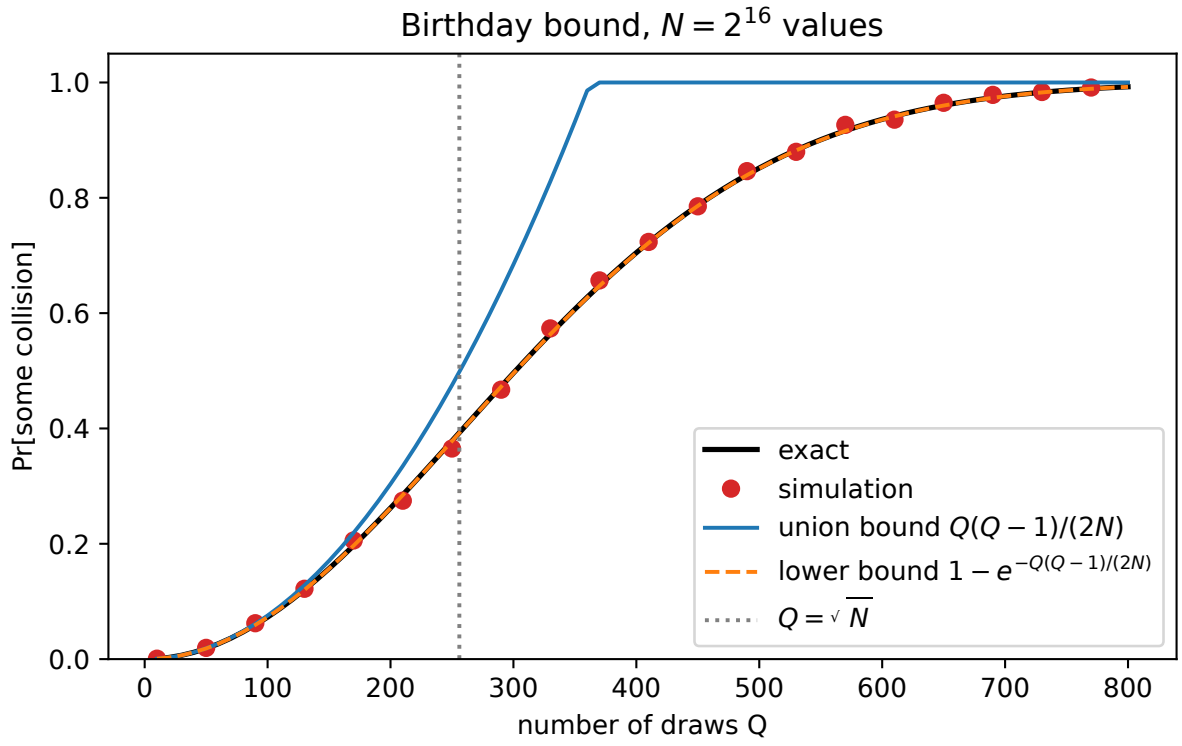


Figure 8.3.: Probability of a collision among Q uniform draws from $N = 2^{16}$ values: exact, simulated, the union bound $Q(Q-1)/(2N)$ and the lower bound $1 - e^{-Q(Q-1)/(2N)}$.

Definition 8.7 (Pseudorandom permutation)**.** A keyed permutation of $\mathcal{X}$ is a **pseudorandom permutation (PRP)** if it satisfies Definition 8.6 with the random function replaced by a uniformly random permutation of $\mathcal{X}$.

Theorem 8.4 (The switching lemma)**.** *Let $\mathcal{A}$ make Q queries, and write $\mathcal{A}^{\mathrm{RF}}$ and $\mathcal{A}^{\mathrm{RP}}$ for $\mathcal{A}$ run with a random function and with a random permutation of $\mathcal{X}$ as its oracle. Then*

$$\left|\Pr[\mathcal{A}^{\mathrm{RF}}=1]-\Pr[\mathcal{A}^{\mathrm{RP}}=1]\right| \le \frac{Q(Q-1)}{2|\mathcal{X}|} \le \frac{Q^2}{2|\mathcal{X}|}. \tag{8.14}$$

Proof. The plan is to run both oracles from one pre-drawn list of answers and show that they behave identically unless the list has a repeat.

1. *Simplify the adversary.* Assume the Q queries are distinct, since both oracles answer a repeated query the same way.
2. *Pre-sample the function's answers.* Draw $Y_1, \ldots, Y_Q$ independently and uniformly from $\mathcal{X}$ and answer the i-th query with Y_i. On distinct queries this is exactly a random function.
3. *Do the same for the permutation.* A random permutation answers each new query with a value uniform over those not yet used, so it can be run from a uniform list of Q *distinct* values.
4. *Compare the two.* Let Col be the event that the list of step 2 has a repeat. Every list is equally likely, so conditioning on no repeat gives a uniform list of distinct values, exactly the permutation's. Hence $\Pr[\mathcal{A}^{\mathrm{RF}}=1 \mid \text{no Col}] = \Pr[\mathcal{A}^{\mathrm{RP}}=1]$, which we call ρ.
5. *Bound the difference.* Splitting on Col, and using step 4 for the case without a repeat,

$$\Pr[\mathcal{A}^{\mathrm{RF}}=1] = \Pr[\mathrm{Col}]\Pr[\mathcal{A}^{\mathrm{RF}}=1 \mid \mathrm{Col}] + (1-\Pr[\mathrm{Col}])\rho. \tag{8.15}$$

   Subtracting ρ from both sides gives

$$\Pr[\mathcal{A}^{\mathrm{RF}}=1]-\rho = \Pr[\mathrm{Col}](\Pr[\mathcal{A}^{\mathrm{RF}}=1 \mid \mathrm{Col}]-\rho), \tag{8.16}$$

   and the bracket is a difference of probabilities, so the whole is at most $\Pr[\mathrm{Col}]$ in absolute value.
6. *Count the repeats.* Equation 8.12 with $N = |\mathcal{X}|$ bounds $\Pr[\mathrm{Col}]$.

□

Since Col depends only on the pre-drawn list, step 4 holds even when each query depends on earlier answers (Bellare & Rogaway 2006). The bound is tight, because the adversary that outputs 1 when two answers collide does so with probability $\Pr[\mathrm{Col}]$ against a function and never against a permutation.

Corollary 8.1 (A pseudorandom permutation is a pseudorandom function)**.** *A PRP with advantage at most ν against Q-query adversaries is a PRF with advantage at most $\nu + Q^2/(2|\mathcal{X}|)$.*

Proof. Go from F_K to a random permutation, at cost at most ν by assumption, and then to a random function, at cost at most $Q^2/(2|\mathcal{X}|)$ by Theorem 8.4. By Lemma 8.2 the total is at most the sum. □

Example 8.7. For a cipher on 128-bit blocks, the extra term $Q^2/(2|\mathcal{X}|)$ of Corollary 8.1 after $Q = 2^k$ blocks is $Q^2/2^{129} = 2^{2k-129}$. After 2^{32} blocks of 16 bytes, which is 64 GiB, it is at most 2^{-65}, and after 2^{48} blocks, about 4 PiB, at most 2^{-33}. At 2^{64} blocks it reaches $\frac{1}{2}$, so keys must be changed long before.

8.8. Perfect secrecy and the one-time pad

Encryption should hide the message. The strongest possible answer is a ciphertext that tells even an unbounded adversary nothing, and Shannon (1949) showed what it costs: a key as long as the data.

Definition 8.8 (Perfect secrecy)**.** An **encryption scheme** turns a message m into a ciphertext $\mathrm{Enc}_K(m)$ under a random key K, and back. It has **perfect secrecy** if, for all messages m, m' and ciphertexts y,

$$\Pr[\mathrm{Enc}_K(m) = y] = \Pr[\mathrm{Enc}_K(m') = y]. \tag{8.17}$$

Equivalently, by Bayes' rule, seeing the ciphertext does not change the probability of any message, for any prior on messages that is independent of the key. The proof for the one-time pad rests on one counting fact, which secret sharing and learning with errors use again.

Lemma 8.3 (A bijection maps uniform to uniform)**.** *If U is uniform on a finite set S and $f\colon S \to S$ is a bijection, then $f(U)$ is uniform on S.*

Proof. Each $t \in S$ comes from exactly one element, $f^{-1}(t)$, so $\Pr[f(U) = t] = \Pr[U = f^{-1}(t)] = 1/|S|$. □

Theorem 8.5 (The one-time pad has perfect secrecy)**.** *For n-bit messages, let K be uniform on n-bit strings and $\mathrm{Enc}_K(m) = m \oplus K$, where $\oplus$ is bitwise exclusive or (XOR). Then every ciphertext has probability 2^{-n} for every message, so the scheme has perfect secrecy.*

Proof.

1. *The pad is a bijection.* For fixed m, the map $k \mapsto m \oplus k$ is its own inverse, since $m \oplus m = 0$.
2. *Apply the lemma.* By Lemma 8.3, $m \oplus K$ is uniform whatever m is.

□

No scheme can do better on key length, so a 128-bit message needs a fresh 128-bit key.

Proposition 8.3 (Perfect secrecy needs as many keys as messages)**.** *If a scheme with key set $\mathcal{K}$ and message set $\mathcal{M}$ has perfect secrecy and correct decryption, then $|\mathcal{K}| \geq |\mathcal{M}|$.*

Proof.

1. *Few messages reach one ciphertext.* Fix a ciphertext y that occurs. Each key decrypts y to one message, so at most $|\mathcal{K}|$ messages can produce y.

2. *Fewer keys break secrecy.* If $|\mathcal{K}| < |\mathcal{M}|$, some message never produces y while another does, so their ciphertext distributions differ.

□

The pad must also never be reused, because XOR-ing two ciphertexts under one pad cancels it and reveals $m_1 \oplus m_2$. English text is so redundant that both messages can be pulled apart from $m_1 \oplus m_2$.

Example 8.8. Table 8.6 shows the pad on 2-bit messages. Each message reaches every ciphertext under exactly one key.

Table 8.6.: The one-time pad on 2-bit messages.

Key K	00	01	10	11
Ciphertext for $m = 01$	01	00	11	10
Ciphertext for $m = 10$	10	11	00	01

8.9. Chosen-plaintext security

Practical encryption accepts a negligible advantage in exchange for a short key. This section gives the definition used in practice, shows that it forces encryption to be randomized, and proves a randomized scheme secure. In the game, the adversary picks two messages and must say which one a ciphertext encrypts, which Goldwasser & Micali (1984) showed is equivalent to their **semantic security**, under which a ciphertext helps compute nothing about the message.

Definition 8.9 (Indistinguishability under chosen-plaintext attack)**.** A scheme is indistinguishable under chosen-plaintext attack (CPA), or **IND-CPA secure**, if every PPT adversary has negligible advantage in this game. The challenger draws a key K and a fair bit b. The adversary may ask for encryptions of messages of its choice at any time. It submits two messages m_0 and m_1 of equal length, receives the challenge $y^* = \mathrm{Enc}_K(m_b)$, and outputs a guess b'.

Taking $b = 0$ and $b = 1$ as the two worlds, Lemma 8.1 makes this advantage $2|\Pr[b' = b] - \frac{1}{2}|$. The messages have equal length because ciphertexts usually reveal lengths. If encryption always maps a message to the same ciphertext, the adversary can encrypt m_0 itself and compare.

Proposition 8.4 (Deterministic encryption is never IND-CPA secure)**.** *If Enc_K is a deterministic function of the message and decryption is correct, some adversary that makes one encryption query has advantage 1.*

Proof.

1. *The attack.* Ask for $y_0 = \mathrm{Enc}_K(m_0)$, submit m_0 and m_1, and output 0 if $y^* = y_0$ and 1 otherwise.
2. *World 0.* If $b = 0$, determinism gives $y^* = y_0$, so the guess is 0.

3. *World 1.* If $b = 1$, correct decryption of the distinct messages forces $\mathrm{Enc}_K(m_1) \neq \mathrm{Enc}_K(m_0)$, so the guess is 1.

□

The fix is a fresh pad for every encryption. Draw a uniform r from a set $\mathcal{R}$, use $F_K(r)$ as a one-time pad, and send r along. The scheme fails only if the PRF is distinguishable from random or the challenge reuses a seen r, and the bound adds the two.

Theorem 8.6 (A randomized scheme from a PRF)**.** *Let* $\mathrm{Enc}_K(m) = (r,\ m \oplus F_K(r))$ *with* r *uniform on* $\mathcal{R}$*. For every adversary* $\mathcal{A}$ *making* Q *encryption queries there is a PRF adversary* $\mathcal{B}$ *with about the same running time such that*

$$\Pr[\mathcal{A} \textit{ wins}] \leq \frac{1}{2} + \mathrm{Adv}_{\mathrm{PRF}}(\mathcal{B}) + \frac{Q}{2|\mathcal{R}|}. \tag{8.18}$$

Proof. The proof replaces the PRF by a random function at the cost of the PRF advantage, and then shows that the challenge is a one-time pad unless one bad event happens.

1. *Two hybrids.* Hybrid 0 is the real game, and hybrid 1 replaces F_K by a random function f.
2. *The step costs the PRF advantage.* $\mathcal{B}$ runs the game for $\mathcal{A}$ with its own oracle in place of F_K and outputs 1 when $\mathcal{A}$ wins. With oracle F_K it simulates hybrid 0, and with f hybrid 1, so the win probabilities differ by at most $\mathrm{Adv}_{\mathrm{PRF}}(\mathcal{B})$.
3. *The bad event.* Let E be the event that the challenge's r^* equals one of the Q values r_i that the encryption oracle drew. Each equals it with probability $1/|\mathcal{R}|$, so the union bound gives $\Pr[E] \leq Q/|\mathcal{R}|$.
4. *Without it, a one-time pad.* Then $f(r^*)$ is uniform and independent of everything else $\mathcal{A}$ sees, so by Lemma 8.3 the challenge is uniform whatever b is, and $\mathcal{A}$ wins with probability $\frac{1}{2}$.
5. *Combine.* Bounding the win probability by 1 when E happens,

$$\Pr[\text{win in hybrid 1}] \leq \tfrac{1}{2}\Pr[\text{not } E] + \Pr[E] = \tfrac{1}{2} + \tfrac{1}{2}\Pr[E] \leq \tfrac{1}{2} + \frac{Q}{2|\mathcal{R}|}, \tag{8.19}$$

   and adding the loss of step 2 gives the bound.

□

By Lemma 8.1, in advantage units the bound is $2\,\mathrm{Adv}_{\mathrm{PRF}} + Q/|\mathcal{R}|$, and the term $Q/|\mathcal{R}|$ is real, not an artefact of the proof.

Example 8.9. Against the deterministic scheme $m \oplus F_K(0)$, the attack of Proposition 8.4 has measured advantage 1.000 over 20,000 games for each b. Against the randomized scheme with 8-bit nonces, $|\mathcal{R}| = 256$, the adversary asks for $Q = 16$ encryptions of m_0, and XOR-ing each reply with m_0 reveals its pad $F_K(r_i)$. If r^* matches one of those nonces r_i, it reads the challenge and outputs the true b, and otherwise it outputs 1. So it errs only when $b = 0$ and there is no match, and its advantage is $1 - (1 - \Pr[E]) = \Pr[E]$. Since r^* misses each nonce with probability $255/256$,

$$\Pr[E] = 1 - (255/256)^{16} = 0.0607. \tag{8.20}$$

The simulation measures 0.0605, and the bound $Q/|\mathcal{R}|$ is 0.0625. With 128-bit nonces the same attack has advantage below $16/2^{128} = 2^{-124}$.

8.10. The simulation paradigm

Encryption hides everything, but many procedures must reveal *something*, such as a participant's own share of a secret, and nothing else. The simulation paradigm defines "nothing else" indirectly: whatever the participant sees, it could have produced by itself from what it is allowed to know (Lindell 2017).

Definition 8.10 (Revealing nothing beyond a leakage)**.** A randomized procedure run on the inputs x of all participants gives one participant a **view** $\mathrm{View}(x)$, everything it sees during the run. The participant is allowed to learn a function $L(x)$, its **leakage**. The procedure **reveals nothing beyond** L if an efficient **simulator** S given only $L(x)$ reproduces the view's distribution:

$$S(L(x)) \equiv \mathrm{View}(x) \quad \text{for every input } x. \tag{8.21}$$

Replacing $\equiv$ by $\approx_s$ or $\approx_c$ gives statistical or computational security.

The definition captures "nothing else" because anything computed from the view has the same distribution when computed from the simulator's output, which uses $L(x)$ alone. The simplest procedure it covers splits a number into random pieces that add up to it, in $\mathbb{Z}_p$, the integers $\{0, \dots, p-1\}$ with arithmetic mod p.

Definition 8.11 (Additive secret sharing)**.** To share a secret $x \in \mathbb{Z}_p$ among h holders, draw the shares $z_1, \dots, z_{h-1}$ independently and uniformly from $\mathbb{Z}_p$, and set

$$z_h = x - (z_1 + \cdots + z_{h-1}) \bmod p. \tag{8.22}$$

Any $h-1$ shares should look uniform whatever the secret is, because the missing share is what ties them to x.

Theorem 8.7 (Any h − 1 shares reveal nothing)**.** *For every secret x and every coalition of $h-1$ holders, the coalition's shares are independent and uniform on $\mathbb{Z}_p^{h-1}$. So a simulator that outputs $h-1$ uniform elements of $\mathbb{Z}_p$, with no leakage, satisfies Equation 8.21 with $\equiv$.*

Proof.

1. *The randomness is uniform.* Each of the p^{h-1} tuples $(z_1, \dots, z_{h-1})$ has probability $p^{-(h-1)}$.
2. *The coalition's shares determine the randomness.* Fix x and a coalition that misses holder j. If $j = h$, the coalition holds $z_1, \dots, z_{h-1}$ themselves. If $j < h$, it holds z_h and every other z_i, and the definition of z_h gives back $z_j = x - z_h - \sum_{i \neq j,h} z_i$.
3. *So the map is a bijection.* The map from the randomness to the coalition's shares can be undone, so by Lemma 8.3 every tuple of shares has probability $p^{-(h-1)}$ for every x.

□

Example 8.10. Table 8.7 makes step 3 concrete for $h = 2$ and $p = 5$, and the script checks the uniformity for $h = 3$ by listing every case.

Table 8.7.: Two-holder additive sharing in $\mathbb{Z}_5$. Each row is a permutation of $\mathbb{Z}_5$, so the share is uniform for every secret.

First share z_1	0	1	2	3	4
Second share $x - z_1 \bmod 5$ for $x = 2$	2	1	0	4	3
Second share $x - z_1 \bmod 5$ for $x = 4$	4	3	2	1	0

Without the reduction mod p the proof fails. With a first share uniform on $\{0, \dots, R-1\}$, the second share $x - z_1$ is uniform on $\{x - R + 1, \dots, x\}$, so the ranges for secrets x and x' differ in $|x - x'|$ values and the TV distance is $|x - x'|/R$. Figure 8.4 compares the secrets 0 and 7. With $p = 11$ the empirical TV distance over 100,000 shares each is 0.0045, sampling noise around the exact 0, while over the integers with $R = 16$ it is 0.4423, against the exact $7/16 = 0.4375$.

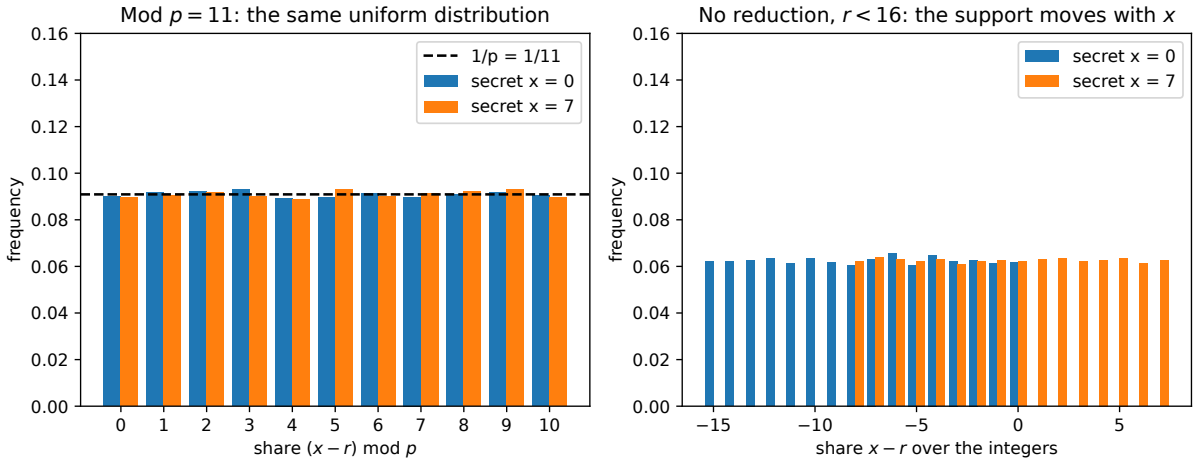


Figure 8.4.: Histograms of the second share for secrets $x = 0$ and $x = 7$. Left: reduced mod $p = 11$, both are uniform. Right: over the integers with a first share below 16, the support shifts with the secret.

8.11. Learning with errors

Every primitive so far is defined by what it hides rather than by how. Learning with errors (LWE), introduced by Regev (2005), is a specific hard problem with no known efficient attack even by a quantum computer, and encryption built on it lets you add ciphertexts. It is linear algebra with noise.

Definition 8.12 (Learning with errors)**.** Fix a dimension d, a modulus p and an error distribution χ on small integers. Draw a secret s uniformly from $\mathbb{Z}_p^d$. An **LWE sample** is a uniform $a \in \mathbb{Z}_p^d$ together with the inner product $\langle a, s \rangle$ plus an error e drawn from χ:

$$(a,\ \langle a, s \rangle + e \bmod p). \tag{8.23}$$

Search LWE asks for s from many samples. **Decision LWE** asks whether given pairs are LWE samples or uniform pairs.

Solving search solves decision, since you can check whether the residuals $b - \langle a, s\rangle$ of a candidate s are small. Regev shows the converse for a prime p polynomial in d, and Appendix Section I.2 derives it step by step. For suitably wide errors drawn from a normal distribution rounded to integers, Regev (2009) proves LWE as hard as the hardest instances of well-studied geometric problems on lattices, regular grids of points in $\mathbb{R}^d$. These results are cited, not proved here, and they do not cover the uniform errors used in the toy scheme below.

Example 8.11. Without noise, LWE is a linear system that Gaussian elimination solves. Take $d = 2$, $p = 7$, $s = (3, 5)$, $a_1 = (1, 2)$ and $a_2 = (3, 1)$, so the right-hand sides are $13 \equiv 6$ and $14 \equiv 0$. The second equation gives $s_2 = -3s_1 \equiv 4s_1$, and the first then gives $s_1 + 8s_1 = 9s_1 \equiv 2s_1 = 6$, so $s_1 = 3$ and $s_2 = 12 \equiv 5$. With the error $(1, 0)$ both right-hand sides are 0, and the same steps give $(0, 0)$, completely wrong, because elimination multiplies the error by the inverse matrix, which mod 7 sends $(1, 0)$ to $(4, 2)$, and $(3, 5) + (4, 2) \equiv (0, 0)$. At $d = 16$ and $p = 4096$ with 32 samples, the script's elimination recovers all 16 coordinates of s without noise, and 0 with errors in $\{-1, 0, 1\}$.

8.12. Regev encryption and its noise budget

The hardness of LWE becomes encryption once a message bit hides in a sample. Picture $\mathbb{Z}_p$ as a clock, and move the sample's second component halfway round if the bit is 1. Whoever knows s subtracts $\langle a, s\rangle$ and sees whether the rest is near 0 or near halfway. This section proves the scheme correct and secure, and asks how many ciphertexts can be added.

Definition 8.13 (Regev's bit encryption)**.** Take p even and a key $s \in \mathbb{Z}_p^d$. To encrypt a bit m, draw a fresh uniform a and an error e from χ, and output

$$\mathrm{Enc}_s(m) = (a,\ w = \langle a, s\rangle + e + m \cdot \tfrac{p}{2} \bmod p). \tag{8.24}$$

To decrypt, compute $v = w - \langle a, s\rangle \bmod p$ and output 1 if $p/4 < v < 3p/4$.

Proposition 8.5 (Correctness)**.** *Decryption returns m whenever $|e| < p/4$.*

Proof.

1. *Remove the inner product.* By the definition of w, $v = e + m\frac{p}{2} \bmod p$.
2. *A 0 stays near 0.* If $m = 0$, then $v \equiv e$ lies in $[0, p/4)$ or, for negative e, in $(3p/4, p)$, and the output is 0.
3. *A 1 stays near halfway.* If $m = 1$, then $v = e + p/2$ lies in $(p/4, 3p/4)$, and the output is 1.

□

Without s, an LWE sample looks uniform, so the bit is added to something that looks uniform, as in a one-time pad.

Proposition 8.6 (Security under decision LWE)**.** *If an IND-CPA adversary $\mathcal{A}$ against Regev's scheme has advantage ν, there is a decision-LWE distinguisher $\mathcal{B}$ with about the same running time and advantage $\nu/2$.*

Proof.

1. *The distinguisher.* $\mathcal{B}$ runs the IND-CPA game for $\mathcal{A}$, builds each ciphertext from one of its given pairs by adding $m \cdot \frac{p}{2}$ to the second component, and outputs 1 when $\mathcal{A}$ wins.
2. *On LWE samples.* The ciphertexts are real, so by Lemma 8.1 $\mathcal{B}$ outputs 1 with probability $\frac{1}{2} + \frac{\nu}{2}$.
3. *On uniform pairs.* Adding a fixed value is a bijection of $\mathbb{Z}_p$, so by Lemma 8.3 each second component is uniform whatever m is, and $\mathcal{A}$ wins with probability $\frac{1}{2}$. The difference is $\frac{\nu}{2}$.

□

A ciphertext is linear in a, the error and the encoded bit, so adding ciphertexts adds all three, and two half-turns make a full turn.

Proposition 8.7 (Adding ciphertexts)**.** *The componentwise sum of encryptions of $m_1, \dots, m_k$ under one key is an encryption of $m_1 \oplus \cdots \oplus m_k$ with error $e_1 + \cdots + e_k$. If every $|e_i| \le B$, it decrypts correctly whenever*

$$k < \frac{p}{4B}. \tag{8.25}$$

Proof.

1. *Linearity.* By linearity of the inner product,

$$\sum_i w_i - \Big\langle \sum_i a_i, s \Big\rangle = \sum_i e_i + \Big(\sum_i m_i \Big) \frac{p}{2} \bmod p. \tag{8.26}$$

2. *Only the parity survives.* Because p is even, $2 \cdot \frac{p}{2} \equiv 0$, so only $m_1 \oplus \cdots \oplus m_k$ remains in the last term.
3. *Bound the error.* By the triangle inequality the summed error is at most kB in absolute value, and by Proposition 8.5 $kB < p/4$ suffices.

□

Equation 8.25 assumes that every error is as large as possible, with the same sign. Independent errors with mean zero mostly cancel, so k errors of standard deviation σ sum to a standard deviation of $\sigma\sqrt{k}$. For χ uniform on the $2B+1$ integers $\{-B, \dots, B\}$ the variance is $((2B+1)^2 - 1)/12 = B(B+1)/3$. By the central limit theorem, with Z a standard normal variable and the reduction mod p taken into $(-p/2, p/2]$,

$$\Pr[\text{failure after } k \text{ additions}] \approx \Pr\Big[\big| Z\sqrt{kB(B+1)/3} \bmod p \big| > p/4 \Big]. \tag{8.27}$$

Example 8.12. Take $d = 3$, $p = 4096$ and $s = (5, 1000, 3000)$. Encrypting $m = 1$ with $a_1 = (17, 2, 1)$ and $e_1 = -7$ gives $\langle a_1, s \rangle = 5085 \equiv 989$ and $w_1 = 989 - 7 + 2048 = 3030$, which decrypts to $v = 2041$, near 2048, so to 1. Encrypting $m = 1$ with $a_2 = (1, 1, 1)$ and $e_2 = 12$ gives $\langle a_2, s \rangle = 4005$ and $w_2 = 4005 + 12 + 2048 \equiv 1969$. The sum has $a_1 + a_2 = (18, 3, 2)$, whose inner product with s is $9090 \equiv 898$, and $w_1 + w_2 \equiv 903$, so decryption finds $v = 5 = e_1 + e_2$ and outputs $0 = 1 \oplus 1$.

Table 8.8 and Figure 8.5 compare the worst case with the typical case for $p = 4096$ and $B = 32$, where Equation 8.25 guarantees $k < 32$. The simulation adds k toy ciphertexts of random bits, with $d = 4$, 20,000 times.

Table 8.8.: Decryption failure rate of a sum of k Regev ciphertexts, $p = 4096$, errors uniform on $\{-32, \dots, 32\}$ (seed 0, 20,000 trials).

Ciphertexts added k	Empirical failure rate	Normal approximation
1	0.0000	0.0000
31	0.0000	0.0000
256	0.0007	0.0006
1,024	0.0868	0.0881
4,096	0.3814	0.3833

At $k = 1024$ the summed error has standard deviation $\sqrt{1024 \times 32 \times 33/3} \approx 600$, so $p/4 = 1024$ is about 1.7 standard deviations, exceeded with probability about 0.088. Failures become measurable only around $k = 256$, eight times the guaranteed 32, but the budget is finite, and the failure rate tends to $\frac{1}{2}$ as the error spreads evenly round the clock.

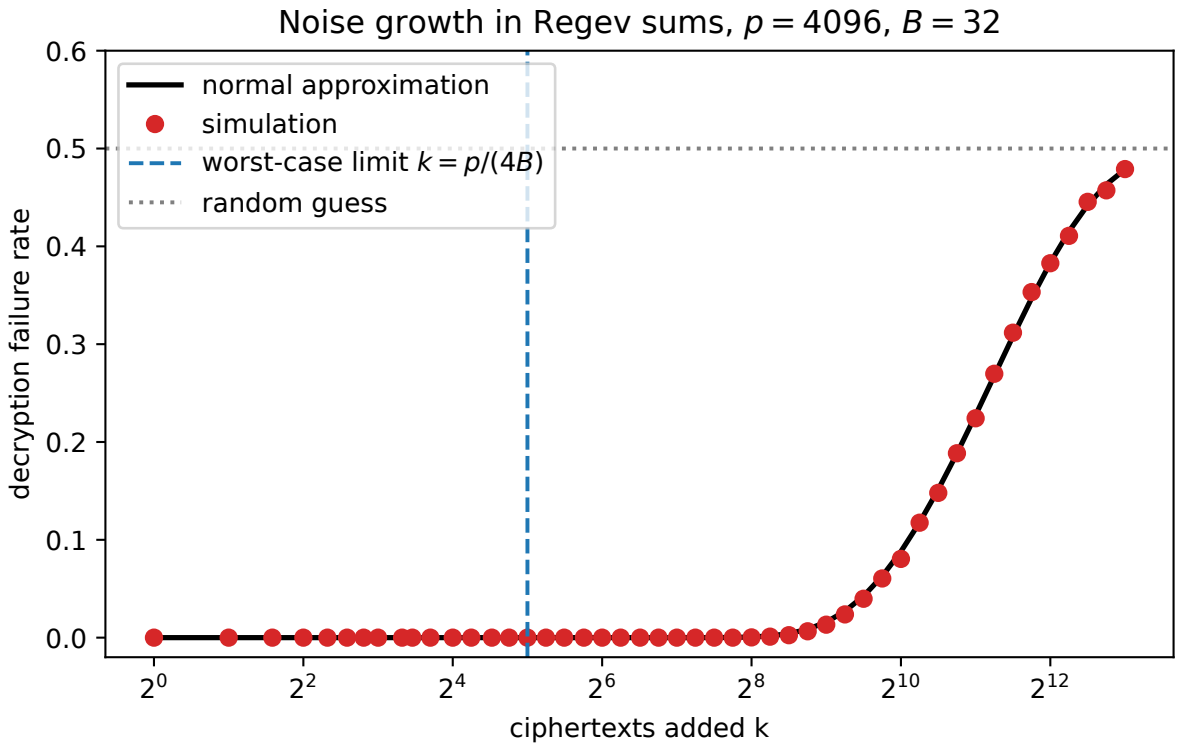


Figure 8.5.: Decryption failure rate of a sum of k Regev ciphertexts ($p = 4096$, errors uniform on $\{-32, \dots, 32\}$): simulation against the normal approximation. The dashed line is the worst-case guarantee $k < p/(4B) = 32$.

8.13. Code

The script code/part01-prerequisites/crypto_basics.py computes every number quoted in this chapter and draws its figures. From the repository root:

```
uv run python code/part01-prerequisites/crypto_basics.py
```

Table 8.9.: Output of the chapter's script (seed 0).

Experiment	Result
Negligible functions	$2^{-\sqrt{\lambda}} < \lambda^{-10}$ from $\lambda = 20{,}520$
Brute force, 10^{18} keys/s for a year	Success 9.3×10^{-14}
Birthday, $N = 2^{16}$, $Q = 256$	Exact 0.3927; simulated 0.3920; union bound 0.4980; 50% at $Q = 301$
Biased coin, Table 8.4	At $Q = 10{,}000$: majority 0.481, best 0.683, hybrid 1.000
IND-CPA, deterministic scheme	Advantage 1.000
IND-CPA, PRF scheme, $Q = 16$, 8-bit nonce	Simulated 0.0605; exact 0.0607; bound 0.0625
Secret shares, $x = 0$ against $x = 7$	Mod 11: TV 0.0045. Integers, $R = 16$: TV 0.4423 (exact 0.4375)
Gaussian elimination, $d = 16$, $p = 4096$	16/16 coordinates without noise; 0/16 with noise

tests/test_crypto_basics.py checks that the output still matches this table.

8.14. Summary

Table 8.10.: The results of this chapter, and where each is used.

Result	Statement	Use
Lemma 8.1	Win probability $\frac{1}{2} + \frac{1}{2}$Adv, at most $\frac{1}{2} + \frac{1}{2}$TV	Scoring every game
Proposition 8.1	Negligible survives $\times$ poly	Repeated attacks stay negligible
Theorem 8.1	Q samples cost a factor Q	Often loose by $\sqrt{Q}$
Definition 8.4	κ bits: advantage $t/2^{\kappa}$	Loose reductions cost bits
Theorem 8.2	GGM is a PRF, loss nQ	Many values from one key
Theorem 8.3	Collisions after $1.18\sqrt{N}$ draws	κ-bit nonces collide after $2^{\kappa/2}$
Theorem 8.4	Permutation versus function: $Q^2/(2\lvert\mathcal{X}\rvert)$	Data per cipher key

Result	Statement	Use
Proposition 8.3	Perfect secrecy needs $\lvert\mathcal{K}\rvert \geq \lvert\mathcal{M}\rvert$	A fresh key per message
Theorem 8.6	Fresh nonces cost $Q/\lvert\mathcal{R}\rvert$	Randomize encryption
Theorem 8.7	Any $h-1$ shares are uniform	Shares must be reduced mod p
Proposition 8.7	Correct if $kB < p/4$	Noise is a budget

8.15. References

- Katz & Lindell, *Introduction to Modern Cryptography* (3rd ed., 2020), ch. 2–3: this chapter's topics, with complete proofs at its level
- Boneh & Shoup, *A Graduate Course in Applied Cryptography*, ch. 2–5: the same topics in the concrete-security style, with the switching lemma and the GGM tree
- Goldreich, *Foundations of Cryptography*, Vol. 1 (2001), ch. 3: pseudorandomness in depth, including the GGM proof (§3.6)
- Shannon, "Communication Theory of Secrecy Systems" (1949): perfect secrecy and the key-length bound
- Goldwasser & Micali, "Probabilistic Encryption" (JCSS 1984): semantic security and the need for randomized encryption
- Goldreich, Goldwasser & Micali, "How to Construct Random Functions" (JACM 1986): the GGM construction
- Bellare & Rogaway, "The Security of Triple Encryption and a Framework for Code-Based Game-Playing Proofs" (EUROCRYPT 2006): game-playing proofs and a careful switching lemma
- Lindell, "How to Simulate It — A Tutorial on the Simulation Proof Technique" (2017): the simulation paradigm, step by step
- Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography" (STOC 2005; JACM 2009): LWE and the original encryption scheme
- Peikert, "A Decade of Lattice Cryptography" (2016): survey of LWE, its variants and reductions

Part II.

Threat models and attacks

This part teaches you to break models before it teaches you to protect them. Every defence in the rest of the book is measured against an attack: a guarantee is only as meaningful as the strongest adversary it stops, and an evaluation is only as honest as the attack it runs. So you start here, with the adversaries, what they see, what they want, and how well the best of them can do.

Who this part is for

You are expected to have read the Prerequisites, or to know their content. This part leans hardest on hypothesis testing (the Neyman–Pearson lemma, trade-off curves, receiver operating characteristic (ROC) curves read at a low false positive rate), on the total variation distance of Divergences, and on the link between stability and generalization. Information theory appears in the chapters on attribute inference, memorization and linkage. You should also be comfortable with logistic regression, gradients of a small neural network, and an n-gram language model at the level of writing them in NumPy.

Differential privacy, the defence most of the book is about, is introduced in the next part, and it is designed against exactly the games defined here. The only privacy definitions in this part are the syntactic ones of the last chapter, k-anonymity and its successors, and they appear as targets: the chapter shows what each one promises and the attack that breaks it.

The common thread

Every attack in this part is the same object: a **game** in which a challenger trains or releases something that depends on a hidden **secret**, and an adversary guesses the secret from what the **threat model** lets it see. What changes from chapter to chapter is the secret: one bit of membership, a sensitive attribute, a memorized string, the whole dataset, a client's input, a property of the population behind the data, or a person's name. Table 8.11 lists them.

Table 8.11.: The chapters of this part, the question each one answers, and the earlier chapters it uses.

Chapter	The question it answers	Builds on
Threat models and privacy games	What can an adversary see and do, and how is an attack stated and scored?	3, 4, 8
Membership inference	Was this record in the training set?	4, 7, 9
Attribute inference and model inversion	Does the model reveal a person's sensitive attribute beyond what the population already does?	3–5, 9, 10
Property inference	Can a model reveal a fact about its training population that no single record reveals?	1, 3, 4, 9, 10

Chapter	The question it answers	Builds on
Memorization and training-data extraction	Can the model be made to emit its training text, and when?	4, 5, 7, 9, 10
Gradient inversion	What does a gradient computed on private data reveal about that data?	6, 9
Linkage and re-identification	When does removing or hashing names fail to hide who a record belongs to?	1, 2, 4, 5, 8, 9
Syntactic anonymity and how it breaks	What do k-anonymity, l-diversity and t-closeness guarantee, and which attacks get past each?	1, 3, 5, 11, 15
Reconstruction attacks	How many accurate statistics does it take to rebuild a whole private dataset?	2, 5, 9

Two ideas run through all nine chapters. The first is that **an attack is a hypothesis test**. When the secret is one bit, the best attack is the likelihood-ratio test of the Neyman–Pearson lemma, its **advantage** (true positive rate (TPR) minus false positive rate (FPR)) is at most the total variation distance between the adversary's two possible views, and its whole error trade-off is bounded by one curve. Membership inference, attribute inference and property inference are each this statement with a different pair of views. The strongest practical attacks, such as the likelihood ratio attack (LiRA) on membership, are explicit approximations of the optimal test.

The second idea is that **the worst case is not the average case**. Attacks succeed first on outliers, on duplicated strings, on unique combinations of attributes and on the one example in a batch that a neuron isolates. An average over records can look harmless while a few records are fully exposed. This is why attacks are reported as the TPR at a low FPR rather than as accuracy or area under the curve (AUC), and why the defence of the next part has to hold for every record and every dataset.

A third idea closes the part and frames what comes after it. Reconstruction attacks show that answering too many statistics too accurately destroys privacy no matter how the answers are computed: this is the **Fundamental Law of Information Recovery**. Some noise is not optional, and the question becomes how little noise is enough.

The second half of the part also tells one story about releasing less. Releasing records with the names removed or replaced by codes fails to linkage. Releasing only coarsened records fails to attacks on the sensitive attribute. Releasing only statistics fails to reconstruction, and adding a fixed amount of noise to each statistic fails once enough of them are asked, as an attack on a deployed query system shows. A few hundred published averages can even reveal whether one person is in the data, the tracing attack of Lower bounds. Each step releases less and still fails, which is the motivation for the definition of the next part.

How to read this part

Order. The part opens with chapter 9, which sets up the vocabulary, and then has two halves. Chapters 10 to 14 attack trained models, from the smallest secret to the largest: one bit of membership, a sensitive attribute, a property of the training population, memorized text, and whole inputs recovered from gradients. Chapters 15 to 17 attack released data, from the weakest defence to the strongest: names removed (linkage), records hidden in crowds (syntactic anonymity), and only aggregate statistics published (reconstruction).

Read chapter 9 first. It fixes the vocabulary (challenger, adversary, view, game, advantage, baseline) that the others use without redefining. Then read chapter 10, since membership inference is the reference attack that chapters 11, 12 and 13 extend. Chapters 11 to 14 can then be read in any order. Read chapters 15 to 17 in order: chapter 16 starts from the linkage game of chapter 15 and uses the imputation baseline of chapter 11, and chapter 17 shows that even exact aggregate statistics, with no records released at all, give the data away. Chapters 16 and 17 matter most for what comes next: the attacks of chapter 16 are the requirements the next part's definition is built to meet, and the results of chapter 17 return as lower bounds there.

Structure of a chapter. Every chapter has the same shape as in the prerequisites. It opens with the question it answers and a running example, then gives one section per topic, whose definitions, theorems and worked examples are numbered, each theorem followed by its proof. A Code section, a Summary table of the chapter's results and References close it.

Toy experiments, real pipelines. The attacks in the literature run on image classifiers and large language models trained on GPUs. Here every experiment is a NumPy toy that runs in seconds: logistic regression on synthetic data, Gaussian mean estimation, a small multilayer perceptron written by hand, an n-gram language model with planted canaries. Each toy is chosen so that the optimal attack can be computed exactly and compared with the practical one. Where the published experiment needs a full pipeline (such as LiRA with 16 shadow models on CIFAR-10), the chapter describes that pipeline in prose and runs a faithful scaled-down version.

Run the code. From the repository root, for example,

```
uv run python code/part02-attacks/membership_inference.py
```

The scripts use fixed seeds, so their output matches the tables in the text.

Notation

The chapters share the conventions in Table 8.12, set in Threat models and privacy games. Where a chapter departs from them, it says so at its start.

Table 8.12.: Notation shared by the chapters of this part.

Symbol	Meaning
D, $\mathcal{D}$	A dataset of records; the data distribution (the population) they are drawn from

Symbol	Meaning
$\mathcal{T}$, $\theta = \mathcal{T}(D)$	A (possibly randomized) training algorithm, and the model it outputs
$\mathcal{A}$	The adversary
b, b'	The hidden bit of a two-world game, and the adversary's guess; $b = 1$ is the world where the target record is used (the member world)
π	A prior over a secret; π_1 is the prior probability of $b = 1$
V; P, Q	The adversary's view; its distributions when $b = 0$ and $b = 1$
TPR, FPR	$\Pr[b' = 1 \mid b = 1]$ and $\Pr[b' = 1 \mid b = 0]$
Adv	The advantage, TPR $-$ FPR, at most $\mathrm{TV}(P, Q)$
$T(P, Q)$	The trade-off function: the smallest false negative rate $1 - \mathrm{TPR}$ of any test at each FPR
$\ell(\theta; z)$	The loss of model θ on record z

What you will be able to do

By the end of this part you should be able to:

- state any privacy attack as a game, name its threat model and its baseline, and bound its advantage by a total variation distance;
- derive the loss-threshold membership attack from the generalization gap and LiRA from the Neyman–Pearson lemma, and evaluate both by the TPR at a low FPR;
- separate attribute inference from what the population already reveals, and say why masking confidences does not stop it;
- explain how a property of the training population can leak when no individual record does;
- define verbatim, discoverable and extractable memorization, and compute a canary's exposure;
- recover an input exactly from the gradient of a fully connected layer, and say why averaging a batch does not remove the leak;
- invert a hashed identifier by enumeration, compute the fraction of a population made unique by a set of quasi-identifiers, and run a Netflix-style de-anonymization;
- say what k-anonymity, l-diversity and t-closeness each guarantee, and break each one with homogeneity, skewness and composition attacks;
- prove that answers with error $o(\sqrt{n})$ to random subset sums allow almost all of a dataset to be reconstructed, and predict how many queries an attack on a deployed query system needs.

Each of these is a result derived in one of the chapters that follow.

9. Threat models and privacy games

A hospital trains a model on patient records and offers it to doctors through a web service. Is the model "private"? The question has no answer until you say **against whom**. Can the attacker only read predicted labels, or does it hold the weights? Does it know the other patients or only the population, and can it slip crafted records into the training data? And does it want to learn that one person was a patient, their diagnosis, or the whole training table? Without a **threat model** that answers them, "the model leaks" and "the model is safe" are claims nobody can test.

Once the threat model is fixed, an attack can be written down as a **game**, as cryptographers do for encryption. A challenger trains the model with a hidden secret, such as whether one patient's record was used, and shows the adversary what the threat model allows, and the adversary guesses the secret. That is the one idea of this chapter: **an attack is a guess about a hidden secret, made from a view that the threat model fixes.** When the secret is one bit, an attack is a test between two hypotheses, so the limits that Hypothesis testing puts on every test, such as the total variation bound, become ceilings that no attack can break.

The running example is a count of people with a sensitive condition, published with standard normal noise. An adversary wants to know whether one person with the condition is in the dataset. If it knows the other 100 records, the best attack flags members at a rate 0.3829 higher than it flags non-members. If it does not know them, the best attack manages only 0.0779, because their unknown count varies far more than the one the target adds. That gap between the uninformed and the best-informed adversary runs through every later chapter of this part.

Table 9.1.: The symbols of this chapter.

Symbol	Meaning
b, b'; π_1, π_0	the hidden bit (1: the target is in the training data) and the guess; the priors of $b = 1$ and $b = 0$
V; P, Q; p, q	the view; its distributions when $b = 0$ and $b = 1$; the probability of one view under each
TPR, FPR, Adv	true and false positive rates, advantage
$\mathcal{T}$, θ, D, $\mathcal{D}$	training algorithm, model, dataset, population
TV, E_γ, $T(P, Q)$	total variation, hockey-stick divergence, trade-off function
S	side information
ω; $\bar{P}$, $\bar{Q}$	what the adversary may not know; the views averaged over it

Symbol	Meaning
Φ	the standard normal distribution function

9.1. Two-world games

Before threat models can be compared, you need a way to score a guess. This section scores the simplest game, whose secret is one bit, and shows why the score is the advantage, defined below, rather than the accuracy.

Definition 9.1 (Two-world game)**.** A challenger flips a hidden bit b, equal to 1 with probability π_1 and to 0 with probability $\pi_0 = 1 - \pi_1$. In the world $b = 1$ the target's record is in the training data ("member"), and in the world $b = 0$ it is not. The adversary receives its **view** V, which is everything it gets to look at (from Section 9.2 on, what it knew beforehand is written separately as S). The view has distribution P when $b = 0$ and Q when $b = 1$. The adversary outputs a guess b', where 1 means "member". Its **true positive rate** (TPR) is the fraction of members it flags, and its **false positive rate** (FPR) is the fraction of non-members it flags:

$$\mathrm{TPR} = \Pr[b' = 1 \mid b = 1], \qquad \mathrm{FPR} = \Pr[b' = 1 \mid b = 0]. \tag{9.1}$$

Its **advantage** is how much more often it says "member" to members than to non-members,

$$\mathrm{Adv} = \mathrm{TPR} - \mathrm{FPR}. \tag{9.2}$$

The adversary wins a game when its guess matches the hidden bit. Play 1,000 games with an attack whose TPR is 0.6 and FPR is 0.2. Table 9.2 counts the wins for a fair bit, $\pi_1 = \frac{1}{2}$, and for a bit that is 1 only one time in ten.

Table 9.2.: Expected wins in 1,000 games of an attack with TPR 0.6 and FPR 0.2. The attack fixes the win rate in each world, and the prior decides how many games each world gets.

Prior π_1	Games with $b = 1$	Wins there ($b' = 1$, rate TPR)	Games with $b = 0$	Wins there ($b' = 0$, rate $1 - \mathrm{FPR}$)	Wins in total
$\frac{1}{2}$	500	$0.6 \times 500 = 300$	500	$0.8 \times 500 = 400$	700, so $\Pr[b' = b] = 0.7$
0.1	100	$0.6 \times 100 = 60$	900	$0.8 \times 900 = 720$	780, so $\Pr[b' = b] = 0.78$

The next result says the same in general.

Proposition 9.1 (Winning probability)**.** *In a two-world game, the probability that the guess is right, over the challenger's bit and everything else that is random, is*

$$\Pr[b' = b] = \pi_1 \,\mathrm{TPR} + \pi_0 \,(1 - \mathrm{FPR}). \tag{9.3}$$

With a fair bit, $\pi_1 = \frac{1}{2}$*, it equals* $\frac{1}{2} + \frac{1}{2}\mathrm{Adv}$*.*

Proof. The plan is to split the games by the world the challenger chose, count the wins in each world, and add.

1. *Split the win into the two worlds.* A game is won when the bit is 1 and the guess is 1, or when the bit is 0 and the guess is 0. A game has only one bit, so the two cannot happen together, and the probability of one or the other is the sum:

$$\Pr[b' = b] = \Pr[b = 1 \text{ and } b' = 1] + \Pr[b = 0 \text{ and } b' = 0]. \tag{9.4}$$

2. *The member world.* A joint probability is the prior times the conditional probability, so the first term is $\Pr[b = 1]\Pr[b' = 1 \mid b = 1]$, which is $\pi_1\mathrm{TPR}$.
3. *The other world.* In the same way the second term is $\pi_0 \Pr[b' = 0 \mid b = 0]$. The guess is either 0 or 1, so $\Pr[b' = 0 \mid b = 0]$ is $1 - \mathrm{FPR}$, and adding the two terms gives Equation 9.3.
4. *The fair bit.* With $\pi_0 = \pi_1 = \frac{1}{2}$ the right side is $\frac{1}{2}\mathrm{TPR} + \frac{1}{2} - \frac{1}{2}\mathrm{FPR}$, which is $\frac{1}{2} + \frac{1}{2}\mathrm{Adv}$.

□

An attack that ignores its view, such as a coin flip, says 1 equally often in both worlds, so its advantage is 0 and it wins half the games. A perfect attack has advantage 1 and wins every game. In between, the fair-bit form reads $\mathrm{Adv} = 2\Pr[b' = b] - 1$, twice the amount by which the attack beats a coin flip.

Unlike the win rate, the advantage does not depend on the prior, because each of the two rates is measured inside one world. In the second row of Table 9.2, always saying 0 already wins 900 games of 1,000, so the attack's 780 wins look worse than knowing nothing. Yet the attack still learns from its view exactly what it learns in the first row, and its advantage of 0.4 says so. Because the advantage measures only what the view reveals, every two-world game in this part is scored by it.

Example 9.1. A data team releases $Y = c + b + Z$, where $c = 50$ is the number of people with the condition among the 100 other records and Z is standard normal noise. The target has the condition, so it adds 1 to the count exactly when it is a member, $b = 1$. The adversary knows the other records, so it knows c, and it says "member" when $Y > 50.5$, halfway between the two possible centres. Write Φ for the standard normal cumulative distribution function (CDF). A member's count, centred at 51, lands above 50.5 when $Z > -\frac{1}{2}$, which by the symmetry of the normal curve has probability $\Phi(\frac{1}{2}) = 0.6915$, the TPR. A non-member's, centred at 50, needs $Z > \frac{1}{2}$, with probability $1 - \Phi(\frac{1}{2}) = 0.3085$, the FPR. The advantage is $2\Phi(\frac{1}{2}) - 1 = 0.3829$, and by Proposition 9.1 the attack wins with probability 0.6915. Over 200,000 simulated games in each world, it scores TPR 0.6901 and FPR 0.3092, an advantage of 0.3809 and a winning probability of 0.6905.

9.2. Threat models

Papers on privacy attacks report numbers such as "membership advantage 0.2", which mean nothing until you know what the attacker was assumed to see, know and do. This section lists what a threat model has to specify, then proves the one ordering that always holds: more access never makes an adversary weaker.

Definition 9.2 (Threat model)**.** A **threat model** answers five questions.

1. **Goal: what is the secret?** It can be whether a record was in the training data (membership), a sensitive field of a known record (attribute), the records themselves (reconstruction), or a statistic of the whole dataset (property).
2. **Access: what does the adversary observe?** From least to most, it sees the model's decisions on inputs it chooses (**label-only**), the model's scores on them, such as confidences or losses (**black-box**), the parameters θ (**white-box**), or everything the training process exposed to it, such as per-step gradients (the **transcript**).
3. **Behaviour: does it follow the rules?** An **honest-but-curious** adversary follows the protocol and only analyses what it sees. A **malicious** adversary may deviate, for example by inserting crafted records into the training data (**poisoning**).
4. **Trust: who sees the raw data?** In the **central** model a trusted curator holds the data, and the adversary sees only what is released. In the **local** model nobody is trusted, and the adversary may be the party that collects the data. Distributed settings, where many parties each hold a slice, sit in between, because each party trusts itself with its own slice but nobody with the rest.
5. **Side information: what does it know already?** It may know the population, a public dataset, some or all of the other records, or the target's non-sensitive fields.

Label-only and black-box access describe a model behind an application programming interface (API). White-box access describes released weights, and also federated learning, where devices train a shared model on their own data, send updates to a server and all receive the global model. The transcript describes that server, which sees each client's updates, the setting of Gradient inversion.

Trust works differently from the other axes, because it decides *who* the adversary is rather than how strong it is. In the local model the collector may itself be the adversary, so protection has to be applied before data leave the user, and Part 3 compares the models. Side information is the hardest axis to bound, because the defender rarely knows what the adversary knows. A guarantee that survives the strongest assumption, "the adversary knows every record except the target's", survives every weaker one, so that is the assumption to design for (Section 9.5).

Of the five axes, access is the one with a natural order, because from the weights the adversary can compute any score, and from a score the label. The proof of the order uses one tool for each step down that chain: whatever the adversary computes from its view cannot raise its best advantage, because it could have computed the same thing itself. Write $\mathrm{Adv}^*(V) = \max_{\mathcal{A}} \mathrm{Adv}(\mathcal{A})$ for the best advantage of any attack $\mathcal{A}$ whose input is the view V.

Lemma 9.1 (Processing the view cannot help)**.** *Let $V' = h(V, R)$ for a function h that the adversary knows and randomness R that it can draw itself, independent of b and V. Then every*

attack on V' has the same TPR and FPR as some attack on V. So $\mathrm{Adv}^*(V') \le \mathrm{Adv}^*(V)$, and the best TPR at every FPR from V' is at most the best from V.

Proof.

1. *Simulate the processing.* Given an attack $\mathcal{A}'$ on V', build an attack $\mathcal{A}$ on V that draws R, computes $h(V, R)$ and runs $\mathcal{A}'$ on the result.
2. *Compare the rates.* Because R does not depend on b, the input that $\mathcal{A}$ passes to $\mathcal{A}'$ has the distribution of V' in each world. So $\mathcal{A}$'s guess has the same distribution as $\mathcal{A}'$'s in each world, with the same TPR and FPR.
3. *Compare the best attacks.* Every pair of rates that some attack on V' reaches is reached by some attack on V, and a maximum over a larger set is at least as large.

□

This is the data processing inequality of Divergences, seen from the attacker's side. So a service that adds noise to the scores it returns can only lower the best advantage.

Proposition 9.2 (More access never weakens the adversary)**.** *For the same model and the same side information S, the best advantages of attacks that see the labels, the scores, the parameters and the training transcript are ordered:*

$$\mathrm{Adv}^*(\mathit{labels}, S) \le \mathrm{Adv}^*(\mathit{scores}, S) \le \mathrm{Adv}^*(\theta, S) \le \mathrm{Adv}^*(\mathit{transcript}, S). \tag{9.5}$$

The same chain holds for the best TPR at every FPR. More side information never lowers the best advantage, $\mathrm{Adv}^*(V) \le \mathrm{Adv}^*(V, S)$, and neither does allowing malicious behaviour.

Proof. The plan is to compute each view from the next, so that Lemma 9.1 applies at every link.

1. *Each view can be computed from the next.* A label is a function of the scores, such as the class with the largest score. A score on a query x is a function of θ and x, and the adversary chose x itself. The final θ is the last entry of the transcript.
2. *The chain.* Each view on the left of Equation 9.5 is $h(V, R)$ for the view V to its right, so Lemma 9.1 orders their best advantages and their best TPR at every FPR.
3. *Side information.* The view V is what you get from the pair (V, S) by forgetting S, so Lemma 9.1 applies again.
4. *Behaviour.* This step needs no processing, because a malicious adversary changes the data rather than the view. An honest-but-curious adversary is a malicious one that chooses not to deviate, and a maximum over more strategies is at least as large.

□

The chain compares the *best* attacks at each level, not the attacks people actually run. In the example below, the gap between two attacks inside one access level is larger than the gap between levels.

Example 9.2. The noisy count has no weights, so this example uses the simplest model that has them. Take a centroid anomaly detector: the mean $\theta = \frac{1}{n}\sum_i x_i$ of $n = 100$ records $x_i \sim \mathcal{N}(0, I_d)$ with $d = 50$ features. A point x is flagged when its anomaly score $\|x - \theta\|^2$ exceeds d, the typical squared distance from the centre. The adversary holds a target z, which is one of the n records if $b = 1$ and a fresh draw if $b = 0$.

With the weights (white-box), the adversary knows θ, and it asks how much more likely this θ is in the member world than in the other. Write $q(\theta \mid z)$ and $p(\theta \mid z)$ for the density of θ given the target z in those two worlds. By the Neyman–Pearson lemma of Hypothesis testing, no attack catches more members at a given false positive rate than one that thresholds this **likelihood ratio**. If z was used, then $\theta = (z + W)/n$, where $W \sim \mathcal{N}(0, (n-1)I_d)$ is the sum of the other records. If it was not, then $\theta \sim \mathcal{N}(0, I_d/n)$. Expanding the squares in the two Gaussian densities gives the log of the likelihood ratio (Section G.7 does it step by step):

$$\ln \frac{q(\theta \mid z)}{p(\theta \mid z)} = \frac{n}{2(n-1)}(2\langle z, \theta\rangle - \|\theta\|^2) - \frac{\|z\|^2}{2(n-1)} + \text{const.} \tag{9.6}$$

The constant depends only on n and d. The bracket grows when θ points towards z, which is what training on z does.

Black-box access gives only the score, $\|z - \theta\|^2 = \|z\|^2 - (2\langle z, \theta\rangle - \|\theta\|^2)$. The adversary knows z, so the **calibrated score** $\|z\|^2 - \|z - \theta\|^2$ gives back exactly the bracket. The log-ratio adds only the term $-\|z\|^2/(2(n-1))$, which shifts the best threshold slightly from one target to the next, so thresholding the calibrated score is almost the likelihood-ratio test. A simpler black-box attack says "member" when the raw score, the target's loss, is low, and it is confused by targets far from the mean, which score high whether or not they were used. With only the flag (label-only), the adversary says "member" when z is not flagged. Table 9.3 reports the advantage of each attack at its best threshold, over 100,000 games in each world.

The best attacks at the label-only, black-box and white-box levels give $0.0400 \le 0.2789 \le 0.2789$, so the chain holds, and the two best attacks agree to four decimals. But the large gap lies *inside* the black-box level: calibrating each target against its own distance from the mean, $\|z\|^2$, multiplies the advantage by about seven, the idea behind the attacks of Membership inference. Sablayrolles et al. (2019) show that whenever training picks parameters with probability proportional to $e^{-\text{training loss}}$, the best attack looks at the parameters mainly through the target's calibrated loss (Section G.7). Figure 9.1 shows the best advantage climbing from 0.0326 at $d = 1$ to 0.7407 at $d = 500$, because with more parameters per record one record moves the model in more directions.

Table 9.3.: Membership advantage against the centroid detector with $n = 100$ and $d = 50$, from 100,000 games per world.

View	Attack	Best advantage TPR − FPR
White-box	likelihood ratio of θ	0.2789
Black-box	calibrated score $\|z\|^2 - \|z - \theta\|^2$	0.2789
Black-box	raw loss $-\|z - \theta\|^2$	0.0407
Label-only	not flagged, threshold d	0.0400

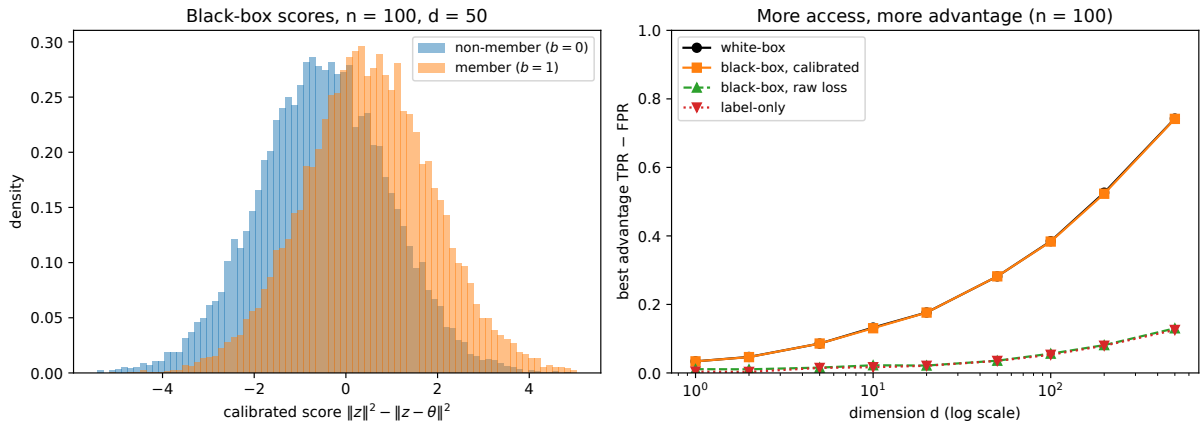


Figure 9.1.: Left: the calibrated black-box score $\|z\|^2 - \|z-\theta\|^2$ for members and non-members of the centroid detector, $n = 100$, $d = 50$. Right: the best advantage at each access level against the dimension d, from 20,000 games per world and point. White-box and calibrated black-box coincide; raw loss and label-only are far below.

9.3. Privacy games

Salem et al. (2023) write each attack in the literature as a game between a challenger and an adversary, because one name, such as "membership inference", covers attacks with different assumptions and scores. This section defines the games of this part, then works out the best possible guess in two cases: a rare secret bit, for which Section 9.1 found no best guess, and many secret bits behind one aggregate.

Definition 9.3 (Privacy game)**.** A **privacy game** has four steps. The challenger draws a secret s from a known distribution, together with a dataset D whose distribution may depend on s. It trains $\theta = \mathcal{T}(D)$ and gives the adversary the view V that the threat model allows, along with the side information S. The adversary outputs a guess s' and earns a **gain** $g(s, s')$, usually $\mathbf{1}[s' = s]$. The attack is compared with a **baseline**, the best expected gain of a guess made from S alone, without looking at V.

Figure 9.2 draws the membership game, and Table 9.4 lists the others. Every row can be played at every access level, as in "label-only membership".

Table 9.4.: The privacy games of this part. The first three are average-case, because the data come from the population; the distinguishing game is worst-case, because the adversary picks the data.

Game	Secret s	Data in the two worlds	Gain	Baseline
Membership	bit b	$b = 1$: target z drawn from D; $b = 0$: z fresh from $\mathcal{D}$	$\mathbf{1}[b' = b]$	$\frac{1}{2}$

Game	Secret s	Data in the two worlds	Gain	Baseline
Attribute	field a of a known member $z = (u, a)$, with other fields u	$z \in D$; adversary knows u	$\mathbf{1}[a' = a]$	guess of a from u and $\mathcal{D}$, or the attack on non-members
Reconstruction	the records of D	D drawn from $\mathcal{D}$	fraction recovered	guessing from $\mathcal{D}$
Distinguishing	bit b	adversary picks two datasets D_0, D_1 that differ in one record; challenger trains on D_b	$\mathbf{1}[b' = b]$	$\frac{1}{2}$
Property	bit b	D drawn from population $\mathcal{D}_{t_0}$ or $\mathcal{D}_{t_1}$, which differ in a statistic t	$\mathbf{1}[b' = b]$	$\frac{1}{2}$

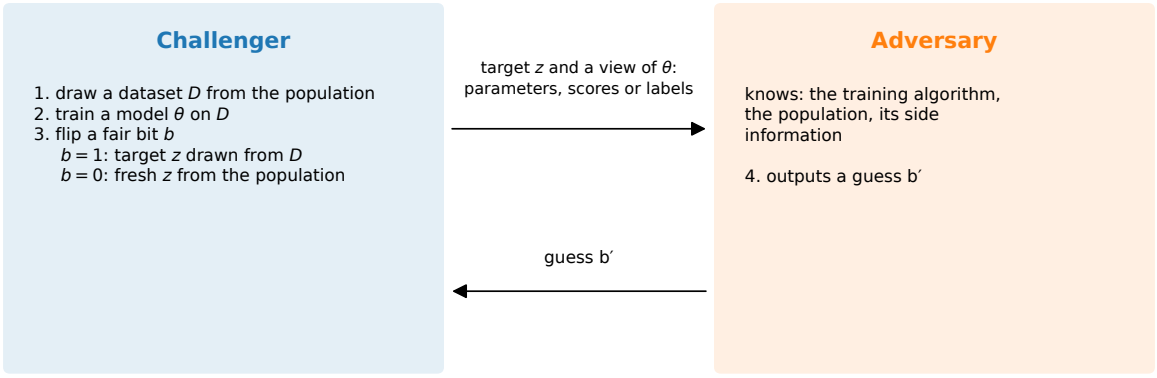


Figure 9.2.: The membership game. The challenger trains on a dataset, flips a bit that decides whether the target was in it, and shows the adversary the target and a view of the model.

Say the secret bit is rare, 1 with prior $\pi_1 < \frac{1}{2}$. Then the safe default is to guess 0, which already wins with probability π_0. The view should overturn that default only where it is strong enough evidence to outweigh the prior odds $\gamma = \pi_0/\pi_1$ against a 1, that is, where it is more than γ times as likely when the bit is 1 as when it is 0. How much those views add to the default's wins is measured by the **hockey-stick divergence** of Divergences, the total by which the probability q of a view when the bit is 1 exceeds γ times its probability p when the bit is 0 (for continuous

views, sums become integrals of densities):

$$E_\gamma(Q\|P) = \sum_x \max\left(q(x) - \gamma p(x),\, 0\right). \tag{9.7}$$

At $\gamma = 1$ it is the **total variation (TV) distance**, half the total absolute difference between the two probability tables:

$$\mathrm{TV}(P, Q) = \tfrac{1}{2} \sum_x |p(x) - q(x)|. \tag{9.8}$$

Proposition 9.3 (Bayes-optimal guess)**.** *In a two-world game with prior $\pi_1 \le \frac{1}{2}$ on $b = 1$, so that guessing 0 is the default, write $\gamma = \pi_0/\pi_1$. The best attack wins with probability*

$$\max_{\mathcal{A}} \Pr[b' = b] = \pi_0 + \pi_1\, E_\gamma(Q\|P), \tag{9.9}$$

and it says 1 exactly on the views x where $q(x) > \gamma p(x)$.

Proof. The plan is to write the winning probability of any attack as the baseline plus a sum over views, and then to choose the views that make the sum largest.

1. *Write an attack as a set.* A deterministic attack says 1 on some set A of views. By Equation 9.3 it wins with probability $\pi_1 Q(A) + \pi_0(1 - P(A))$.
2. *Pull out the baseline.* Factoring out π_1 from the terms that depend on A, with $\gamma = \pi_0/\pi_1$, gives
$$\pi_1 Q(A) + \pi_0(1 - P(A)) = \pi_0 + \pi_1(Q(A) - \gamma P(A)). \tag{9.10}$$
3. *Choose the best set.* The bracket is a sum over the views in A of $q(x) - \gamma p(x)$. It is largest when A holds exactly the views whose term is positive, and then it is the sum of the positive terms, which is $E_\gamma(Q\|P)$. In words, the best attack says 1 when the likelihood ratio q/p exceeds the prior odds γ against the bit being 1.
4. *Randomizing does not help.* A randomized attack says 1 on view x with some probability $\phi(x)$ and gets $\sum_x \phi(x)(q(x) - \gamma p(x))$ in the bracket. This is also largest with $\phi = 1$ on the positive terms and 0 elsewhere.

□

The first term of Equation 9.9 is the baseline of always guessing 0, and the second is everything the view adds. For a fair bit, $\gamma = 1$ and E_1 is the TV distance, so the best win probability is $\frac{1}{2} + \frac{1}{2}\mathrm{TV}$, and matching it with $\frac{1}{2} + \frac{1}{2}\mathrm{Adv}$ from Proposition 9.1 makes the best advantage the TV distance, a ceiling that Section 9.4 extends.

The reconstruction game has many secret bits rather than one. Its simplest case shows how little a single aggregate reveals about each record.

Proposition 9.4 (Reconstruction from one exact count)**.** *Let the secret be m independent uniform bits $x_1, \ldots, x_m$, and let the release be their sum K. The best reconstruction recovers in expectation the fraction of the bits below, whose two forms agree because* $\max(k, m-k) = \frac{m}{2} + |k - \frac{m}{2}|$*:*

$$\frac{\mathbb{E}[\max(K, m-K)]}{m} = \frac{1}{2} + \frac{\mathbb{E}\left[|K - \frac{m}{2}|\right]}{m}. \tag{9.11}$$

For large m this is close to $\frac{1}{2} + 1/\sqrt{2\pi m}$.

Proof. The plan is to find what the count says about each bit, guess each bit from that, and then average over the count.

1. *What the count says about each bit.* Given $K = k$, every arrangement of k ones among m places is equally likely, so each bit is 1 with probability k/m.
2. *The best guess for each bit.* The gain is the fraction of bits guessed right, so each bit can be guessed separately. The best guess is 1 when $k/m > \frac{1}{2}$ and 0 otherwise, so every bit gets the majority value, and the expected number right is $\max(k, m-k)$.
3. *Average over the count.* The count K is $\mathrm{Bin}(m, \frac{1}{2})$, and averaging $\max(K, m-K)/m$ over it gives the fraction in the statement.
4. *How large the gain is.* By the central limit theorem, $K - \frac{m}{2}$ is approximately normal with standard deviation $\sqrt{m}/2$. The average distance of a normal variable from its mean is its standard deviation times $\sqrt{2/\pi}$, and dividing $\frac{\sqrt{m}}{2}\sqrt{2/\pi}$ by m gives $1/\sqrt{2\pi m}$. Section E.2 computes the fraction exactly, as $\frac{1}{2} + \frac{1}{2}\binom{2j}{j}4^{-j}$ with $j = \lfloor m/2 \rfloor$, and proves the approximation without the central limit theorem.

□

A model that lets an attacker guess a rare diagnosis "with 90% accuracy" reveals nothing if 90% of people do not have that diagnosis. Many aggregates are a different matter: Reconstruction attacks shows how noisy counts over random subsets of the records, a few times as many as there are bits, give away almost every bit.

Attribute inference has two possible baselines. A guess from the population asks whether the model helps, while the attack's success on non-members asks whether it helps *because this person was in the training data.* Only the second measures a breach of that person's privacy, as Attribute inference and model inversion develops.

Example 9.3. Take the noisy count $Y = c + b + Z$ of Example 9.1, with the rest of the count c known, and attack it in three games.

In the **membership** game, the count is $\mathcal{N}(c, 1)$ without the target and $\mathcal{N}(c+1, 1)$ with it, and by Example 9.1 the best attack wins with probability $\Phi(\frac{1}{2}) = 0.6915$.

In the **attribute** game with a fair prior, the target is known to be in D, and the secret is whether it has the condition. The count is $c + 1 + Z$ if it does and $c + Z$ if not, the same two distributions as before, so the best guess is again right with probability 0.6915 (0.6925 in 200,000 simulated games). Now make the condition rare, with prior $\pi_1 = 0.1$, so that always guessing "no" wins with probability 0.9. By Proposition 9.3 the release overturns that guess only where the likelihood ratio exceeds $\gamma = 9$. The ratio of the two normal densities at y is $e^{-(y-c-1)^2/2}/e^{-(y-c)^2/2} = e^{(y-c)-1/2}$, so this happens where $y > c + \frac{1}{2} + \ln 9$. A normal variable with mean μ and variance 1 exceeds a with probability $\Phi(\mu - a)$, so the world with the condition puts mass $\Phi(\frac{1}{2} - \ln 9)$ there and the other world mass $\Phi(-\frac{1}{2} - \ln 9)$, and

$$E_9(Q\|P) = \Phi(\tfrac{1}{2} - \ln 9) - 9\,\Phi(-\tfrac{1}{2} - \ln 9) = 0.0134. \tag{9.12}$$

The best accuracy is $0.9 + 0.1 \times 0.0134 = 0.9013$ (simulated: 0.9028). The same release that lifts a fair guess from 0.5 to 0.69 lifts this one by about a tenth of a percentage point, because the likelihood ratio rarely overturns 9-to-1 odds.

In the **reconstruction** game, release the exact count of m uniform bits instead. By Proposition 9.4 the best reconstruction recovers a fraction 0.6230 of the bits for $m = 10$ (0.6236 in simulation), 0.5398 for $m = 100$ and 0.5126 for $m = 1000$. The approximation $\frac{1}{2} + 1/\sqrt{2\pi m}$ gives 0.6262, 0.5399 and 0.5126.

9.4. Advantage, total variation and the trade-off curve

A two-world game is a test of the hypothesis "not a member", under which the view has distribution P, against "member", under which it has distribution Q. So everything Hypothesis testing proved about tests becomes a statement about attacks. This section draws out three: a ceiling on the advantage, a ceiling on the whole receiver operating characteristic (ROC) curve, and a warning about which point of that curve to report.

The first ceiling is the TV distance, which Proposition 9.3 already found for a fair bit. The second bounds the attack's TPR at *every* false positive rate, not just where TPR − FPR is largest. It comes from the **trade-off function** $T(P, Q)$ of Hypothesis testing. For each false positive rate, $T(P, Q)(\text{FPR})$ is the smallest fraction of members that any attack with that false positive rate must miss, so an attack can catch at most the rest.

Proposition 9.5 (Ceilings on every attack)**.** *For every attack in a two-world game with views P and Q,*

$$\text{Adv} \le \text{TV}(P, Q), \qquad \Pr[b' = b] \le \tfrac{1}{2} + \tfrac{1}{2}\text{TV}(P, Q), \tag{9.13}$$

where the second bound is for a fair bit, and

$$\text{TPR} \le 1 - T(P, Q)(\text{FPR}). \tag{9.14}$$

All three bounds are attained by likelihood-ratio attacks, which flag first the views with the largest likelihood ratio q/p.

Proof.

1. *The attack is a test.* An attack says "member", rejecting the hypothesis P, with probability $\phi(V)$. Its type I error is its FPR, and its type II error is $1 - \text{TPR}$.
2. *The advantage ceiling.* Every test has $\text{TPR} - \text{FPR}$ at most $\text{TV}(P, Q)$, with equality for the test that says "member" exactly where $q > p$ (Hypothesis testing). That gives the first bound of Equation 9.13, and Proposition 9.1 turns it into the second.
3. *The curve ceiling.* By the definition of the trade-off function, no attack with false positive rate FPR misses fewer than $T(P, Q)(\text{FPR})$ of the members, so $1 - \text{TPR} \ge T(P, Q)(\text{FPR})$. By the Neyman–Pearson lemma, the attack that attains it flags views in decreasing order of the likelihood ratio q/p until its false positive rate reaches the chosen value, flagging the last views only at random if that is needed to hit it exactly.

□

The warning needs one more quantity, which answers the question a flagged person cares about: given that the attack says "member", how likely is that to be true? The **precision** is the members' share of everyone the attack flags.

Proposition 9.6 (Precision)**.** *An attack with rates TPR and FPR, in a game with prior π_1, is right when it says "member" with probability*

$$\Pr[b = 1 \mid b' = 1] = \frac{\pi_1 \,\mathrm{TPR}}{\pi_1 \,\mathrm{TPR} + \pi_0 \,\mathrm{FPR}}. \tag{9.15}$$

Proof.

1. *A conditional probability is a ratio.* The precision is the probability that a target is a member and flagged, divided by the probability that it is flagged. The numerator is $\pi_1\mathrm{TPR}$, as in step 2 of the proof of Proposition 9.1.
2. *The denominator.* A target is flagged either as a member, with probability $\pi_1\mathrm{TPR}$, or as a non-member, with probability $\pi_0\mathrm{FPR}$. The two cannot happen together, so the probability of being flagged is their sum.

□

The advantage is only one point of the ROC curve, the plot of an attack's TPR against its FPR as its threshold varies. It is the largest height of that curve above the diagonal, reached at one threshold, so it says nothing about the low false-positive end, where an attack identifies people with confidence. An attack that flags 1% of members with no false alarms has an advantage of only about 0.01, yet it identifies those members with certainty. Carlini et al. (2022) argue from this that membership attacks should be reported by their TPR at a low FPR, such as 10^{-3}, on ROC curves with log axes, and the rest of this part does so. Precision points the same way when members are rare (Jayaraman et al. 2021), as Example 9.4 shows.

Both ceilings depend only on P and Q, so they bound attacks nobody has invented yet, and a training pipeline whose ROC curve stays close to the diagonal for every target defeats every attack at once.

Example 9.4. In the noisy count of Example 9.1, $P = \mathcal{N}(50, 1)$ and $Q = \mathcal{N}(51, 1)$. The attack there says "member" exactly where $q > p$, so by Proposition 9.5 its advantage 0.3829 is the TV distance, and no attack does better.

Now hold the false positive rate at 10^{-3}, so that the threshold τ must rise until only one non-member in a thousand exceeds it. With Φ^{-1} the inverse of Φ, $\Pr[\mathcal{N}(50, 1) > \tau] = 10^{-3}$ gives $\tau = 50 + \Phi^{-1}(0.999) = 50 + 3.0902 = 53.09$. A member's count, centred one higher, exceeds τ with probability $1 - \Phi(3.0902 - 1) = 0.0183$. That is the TPR, and by Equation 9.14 it is the best possible at this FPR (simulated: 0.0184, at an FPR of 0.00109). The attack now catches fewer than 2% of members, but when it fires, Equation 9.15 gives a precision of $0.0183/(0.0183 + 0.001) = 0.948$ at a fair prior. If only one target in a hundred is a member, the precision drops to $0.01 \times 0.0183/(0.01 \times 0.0183 + 0.99 \times 0.001) = 0.156$.

The black curve in Figure 9.3 is this ROC curve on log axes, and the green one is the same game against an adversary who does not know the other records (Section 9.5).

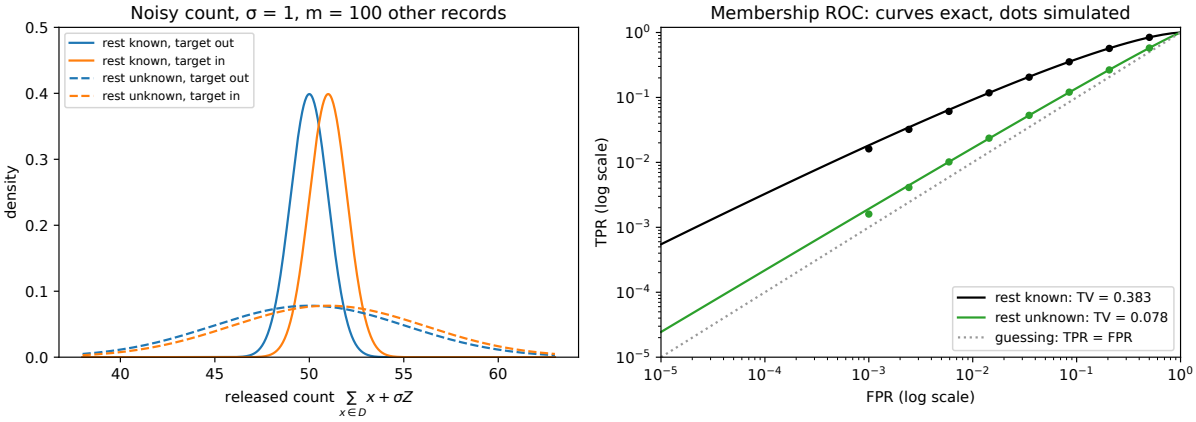


Figure 9.3.: Left: the released count in the two worlds when the other 100 records are known (solid) and unknown (dashed). Right: the membership ROC curves of the best attacks on log axes, exact (lines) and from 100,000 simulated games per world (dots). The dotted diagonal is random guessing.

9.5. Average case and worst case

A membership game can be scored in two ways. In the **average case**, the data and the target come from the population, the adversary knows only the population, and the advantage is an average over all of that randomness. In the **worst case**, the adversary knows, or even picks, the rest of the data and the target, and the advantage is the maximum over those choices. Evaluations of real models usually measure the first, but privacy guarantees have to control the second, and the two can be far apart.

When the view depends on something the adversary does not know, such as the other records, not knowing it works like extra noise, because averaging two distributions can only blur their difference. Knowing it removes that noise, and choosing it picks the situation where the difference is largest. Call the unknown ω, taking finitely many values with probabilities w_ω that sum to 1, and let P_ω and Q_ω be the views in the two worlds when the unknown is ω.

Lemma 9.2 (Mixing hides, and knowing the mixture undoes it)**.** *Let $\bar{P} = \sum_\omega w_\omega P_\omega$ and $\bar{Q} = \sum_\omega w_\omega Q_\omega$ be the views averaged over the unknown, and write $(\omega, V)_b$ for the distribution of the pair of the unknown and the view in world b. The chain below has three links, and each step of the proof gives one:*

$$\mathrm{TV}(\bar{P}, \bar{Q}) \;\le\; \sum_\omega w_\omega \,\mathrm{TV}(P_\omega, Q_\omega) \;=\; \mathrm{TV}\big((\omega, V)_{b=0}, (\omega, V)_{b=1}\big) \;\le\; \max_\omega \mathrm{TV}(P_\omega, Q_\omega). \tag{9.16}$$

Proof.

1. *Averaging first can only cancel.* The TV distance of the averages is half the sum over views of $\big|\sum_\omega w_\omega (p_\omega(x) - q_\omega(x))\big|$. Differences can cancel when they are added before taking absolute values, but not after, by the triangle inequality. So this is at most half the sum over views of $\sum_\omega w_\omega |p_\omega(x) - q_\omega(x)|$, and swapping the two sums gives the middle term.

2. *Knowing the unknown.* The pair (ω, V) has probability $w_\omega p_\omega(x)$ in world 0 and $w_\omega q_\omega(x)$ in world 1. Half the total absolute difference between these two tables is $\sum_\omega w_\omega \mathrm{TV}(P_\omega, Q_\omega)$, which is the equality.
3. *Choosing it.* An average is at most its largest term, which gives the last inequality.

□

A tiny case shows how much the unknown can hide. Let there be one other record, whose bit ω is 0 or 1 with probability $\frac{1}{2}$ each, and let the mechanism release the exact count $\omega + b$. Knowing ω, the adversary subtracts it and reads off b, so the TV distance is 1. Without it, the count is 0 or 1 in world 0 and 1 or 2 in world 1, each with probability $\frac{1}{2}$. Only the count 1 appears in both, so the TV distance is $\frac{1}{2}(\frac{1}{2} + 0 + \frac{1}{2}) = \frac{1}{2}$.

Now let ω collect everything that the average-case game draws at random and that a worst-case adversary is allowed to fix: the other records, and the target, which until now was fixed. The three terms of Equation 9.16 are then the best advantages of three adversaries.

Proposition 9.7 (Average case against worst case)**.** *The best advantages of an adversary who does not know ω, one who knows it and one who chooses it are ordered:*

$$\underbrace{\mathrm{TV}(\bar{P}, \bar{Q})}_{\text{does not know } \omega} \;\le\; \underbrace{\mathbb{E}_\omega \,\mathrm{TV}(P_\omega, Q_\omega)}_{\text{knows } \omega} \;\le\; \underbrace{\max_\omega \,\mathrm{TV}(P_\omega, Q_\omega)}_{\text{chooses } \omega}. \tag{9.17}$$

The first gap can be extreme. For the exact count of m unknown fair bits plus the target's membership bit b, the middle term is 1 while the left term is $\binom{m}{\lfloor m/2 \rfloor} 2^{-m}$, about $\sqrt{2/(\pi m)}$.

Proof. The plan is to identify each term as a best advantage, so that Lemma 9.2 orders them, and then to compute the exact count.

1. *Three adversaries, three views.* An adversary that does not know ω sees V, distributed as $\bar{P}$ or $\bar{Q}$, so by Proposition 9.5 its best advantage is $\mathrm{TV}(\bar{P}, \bar{Q})$. One that knows ω sees the pair (ω, V), and by the equality of Equation 9.16 its best advantage is $\mathbb{E}_\omega \mathrm{TV}(P_\omega, Q_\omega)$. One that chooses ω picks the most revealing value, which gives the maximum, and Equation 9.16 orders the three.
2. *The exact count, known crowd.* An adversary who knows the other bits subtracts their sum from the count and reads off b, so the middle term is 1.
3. *The exact count, unknown crowd.* An adversary who does not know them sees K in world 0 and $K+1$ in world 1, where $K \sim \mathrm{Bin}(m, \frac{1}{2})$ is the unknown sum. So with $p(k) = \Pr[K = k]$, the other world has $q(k) = p(k-1)$, the same list of binomial weights shifted one place right. Their TV distance is the total excess of q over p where it is positive.
4. *Telescope.* The binomial weights rise up to the middle value $\lfloor m/2 \rfloor$ and fall after it (Section E.2 proves it). So q is larger exactly to the right of the middle, with excess $p(k-1) - p(k)$ at k. Adding these differences from the middle to the end, every weight cancels except the one at the middle, which is the largest weight $\binom{m}{\lfloor m/2 \rfloor} 2^{-m}$.
5. *Its size.* For large m the binomial is close to a normal curve with standard deviation $\sqrt{m}/2$, whose peak height is $\sqrt{2/(\pi m)}$, and this tends to 0 as m grows.

□

Averaging over datasets hides the target in a crowd, but only from an adversary who does not know the crowd, and a malicious adversary can plant the other records itself. Averaging over targets hides the worst-treated person in the same way. If a mechanism exposes one target in a thousand completely and the others not at all, the middle term is $1/1000 = 0.001$ and the right term is 1. Outliers and duplicated records are the usual victims, as Membership inference shows.

An audit can estimate average-case numbers from data, but they are lower bounds on the worst case: an audit that reports advantage 0.01 shows only that the worst case is at least 0.01. An audit narrows the gap by planting **canaries**, records built to be as exposed as possible (Memorization and training-data extraction). A guarantee that holds for every pair of datasets differing in one record bounds the right side of Equation 9.17, and therefore bounds every game in this part whose two worlds differ in one record. The definition in Part 3 is a guarantee of this kind.

Example 9.5. Keep the noisy count of Example 9.1, with noise of standard deviation $\sigma = 1$ and a target who has the condition, but now let the $m = 100$ other bits be fresh $\mathrm{Bern}(\frac{1}{2})$ draws that the adversary does not see. The count is then $K + b + Z$ with $K \sim \mathrm{Bin}(100, \frac{1}{2})$, and its likelihood ratio between the two worlds increases with the observed count (Section F.6). So the best attack is still a threshold. The member world's count is the other world's shifted by 1, and both are symmetric, about 51 and 50, so their densities cross halfway, at 50.5. Table 9.5 compares the adversaries of Equation 9.17, and adds a target whose condition the adversary does not know, which it has with probability $\frac{1}{2}$.

Not knowing the crowd cuts the advantage by a factor of five, and a normal approximation explains why: treating K as $\mathcal{N}(50, 25)$ makes the two worlds Gaussians with shift 1 and standard deviation $\sqrt{25+1}$, which gives $2\Phi(1/(2\sqrt{26})) - 1 = 0.0781$. When the target has the condition only half the time, the half without it gives the same count in both worlds, so only the other half of each world's mixture differs, and the advantage halves to $0.0779/2 = 0.0390$. At an FPR of 10^{-3} the average-case attack has TPR 0.0019 (simulated 0.0016), against 0.0183 for the adversary who knows the rest, and these are the green and black curves of Figure 9.3. Figure 9.4 shows the average-case advantage falling as the crowd grows, while the worst case does not move.

Table 9.5.: Membership advantage on a count of 100 other records plus the target's bit, with normal noise of standard deviation σ, for adversaries with different knowledge.

Adversary	Best advantage (exact)	Simulated (200,000 games per world)
Knows the other records ($\sigma = 1$)	0.3829	0.3809
Does not know them, target has the condition	0.0779	0.0751
Does not know them, target has it with probability $\frac{1}{2}$	0.0390	0.0398
Knows the other records, exact count ($\sigma = 0$)	1	—
Does not know them, exact count ($\sigma = 0$)	0.0796	—

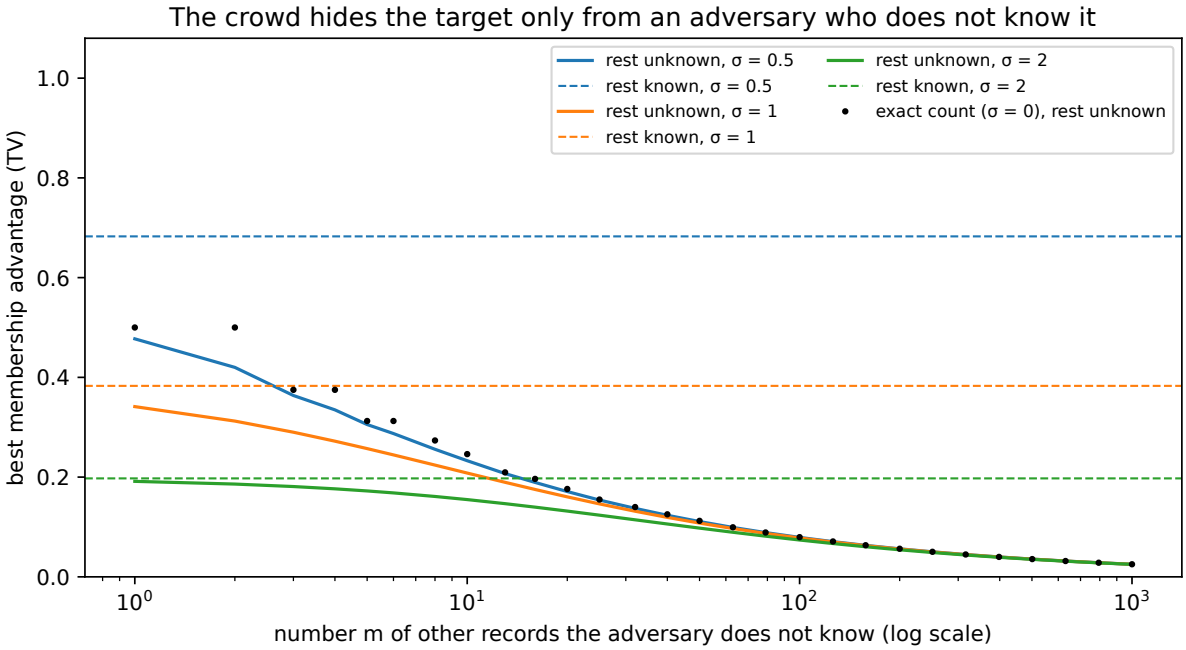


Figure 9.4.: Best membership advantage on the noisy count against the number m of other records the adversary does not know, for three noise levels (solid), and the same mechanism against an adversary who knows them (dashed). Dots: the exact count, $\sigma = 0$, whose worst case is 1.

A small advantage is not always a small risk, even in the worst case, as a release of one record shows.

Example 9.6. A mechanism publishes one record of D, chosen uniformly among $n = 1000$ records that are all different. The attack says "member" if the published record is z. If z is in D, it is published with probability $1/n$, so TPR $= 0.001$, and if not, it is never published, so FPR $= 0$. Simulation gives TPR $= 0.00098$ and FPR $= 0$ over 200,000 games per world. The advantage is $1/n = 0.001$ for every target and every dataset, so the mechanism looks almost perfectly safe in the average case and in the worst case alike. Yet every run exposes one person completely, because when the attack fires its precision is 1. Only the low false-positive end of Section 9.4 reveals the problem. The published record is possible in one world and impossible in the other, so a guarantee has to bound how much more likely an output is in one world than in the other.

9.6. Code

The script code/part02-attacks/threat_models.py computes every number in this chapter from closed forms, checks each against games played by simulation, and draws every figure. From the repository root:

```
uv run python code/part02-attacks/threat_models.py
```

Table 9.6.: The script's output with the default seed.

Experiment	Result
Centroid detector, $n = 100$, $d = 50$	Best advantage: white-box 0.2789; calibrated black-box 0.2789; raw loss 0.0407; label-only 0.0400
Noisy count, rest known, $\sigma = 1$	TV 0.3829; simulated TPR 0.6901, FPR 0.3092, advantage 0.3809, winning probability 0.6905
Same, at FPR 10^{-3}	TPR 0.0183 (simulated 0.0184); precision 0.948 at prior $\frac{1}{2}$, 0.156 at prior 0.01
Noisy count, 100 unknown others	TV 0.0779 (normal approximation 0.0781; simulated 0.0751); TPR at FPR 10^{-3} 0.0019 (simulated 0.0016)
Same, random target bit	Advantage 0.0390 (simulated 0.0398)
Exact count, 100 unknown others	TV 0.0796; $\sqrt{2/(\pi m)} = 0.0798$
Attribute from the noisy count	Prior $\frac{1}{2}$: 0.6915 (simulated 0.6925). Prior 0.1: baseline 0.9, best 0.9013, $E_9 = 0.0134$ (simulated 0.9028)
Reconstruction from one exact count	$m = 10$: 0.6230 (simulated 0.6236); $m = 100$: 0.5398; $m = 1000$: 0.5126
Random-record release, $n = 1000$	Simulated TPR 0.00098, FPR 0; $1/n = 0.001$

tests/test_threat_models.py checks that the output still matches.

9.7. Summary

Table 9.7.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 9.2	Goal, access, behaviour, trust, side information	Read an attack's threat model before its number
Proposition 9.1	$\Pr[b' = b] = \pi_1 \mathrm{TPR} + \pi_0(1 - \mathrm{FPR})$; fair bit: $\frac{1}{2} + \frac{1}{2}\mathrm{Adv}$	Score games by $\mathrm{Adv} = \mathrm{TPR} - \mathrm{FPR}$, which ignores the prior
Lemma 9.1, Proposition 9.2	Labels $\leq$ scores $\leq$ parameters $\leq$ transcript, for the best attack	A better attack on the same view, calibrated per target, can matter more than more access
Definition 9.3, Proposition 9.3	$\max \Pr[b' = b] = \pi_0 + \pi_1 E_{\pi_0/\pi_1}(Q\|P)$ for $\pi_1 \leq \frac{1}{2}$	Compare every attack with its baseline; accuracy near the prior is no attack
Proposition 9.4	One exact count of m bits: about $\frac{1}{2} + 1/\sqrt{2\pi m}$ recovered	One aggregate reveals little about each record

Result	Statement	Use
Proposition 9.5	$\text{Adv} \le \text{TV}(P, Q)$; $\text{TPR} \le 1 - T(P, Q)(\text{FPR})$	The ceiling of every attack, and the target of every defence
Proposition 9.6	$\pi_1\text{TPR}/(\pi_1\text{TPR} + \pi_0\text{FPR})$	A low FPR keeps precision high when members are rare
Lemma 9.2, Proposition 9.7	Not knowing $\omega \le$ knowing it $\le$ choosing it	Average-case numbers are lower bounds; noisy count: $2\Phi(\frac{1}{2\sigma}) - 1$ with the rest known, about $2\Phi(1/(2\sqrt{m/4 + \sigma^2})) - 1$ with m unknown (Example 9.5)

9.8. References

- Salem, Cherubin, Evans, Köpf, Paverd, Suri, Tople & Zanella-Béguelin, "SoK: Let the Privacy Games Begin! A Unified Treatment of Data Inference Privacy in Machine Learning" (IEEE S&P 2023). The source of this chapter's games and the relations between them; read §§3–5.
- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (IEEE CSF 2018). The membership and attribute games with advantage TPR − FPR, and the loss-threshold attack.
- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022). Why attacks should be reported by TPR at low FPR on log-scaled ROC curves; the attack of Membership inference.
- Sablayrolles, Douze, Schmid, Ollivier & Jégou, "White-box vs Black-box: Bayes Optimal Strategies for Membership Inference" (ICML 2019). Conditions under which the optimal attack depends only on the loss, so white-box access does not help.
- Jayaraman, Wang, Knipmeyer, Gu & Evans, "Revisiting Membership Inference Under Realistic Assumptions" (PETS 2021). Membership inference when members are rare: priors, precision and threshold choice.
- Choquette-Choo, Tramèr, Carlini & Papernot, "Label-Only Membership Inference Attacks" (ICML 2021). Attacks that see only predicted labels, and how close they come to score-based ones.
- Nasr, Shokri & Houmansadr, "Comprehensive Privacy Analysis of Deep Learning: Passive and Active White-box Inference Attacks against Centralized and Federated Learning" (IEEE S&P 2019). White-box and federated adversaries, passive and active.
- Boneh & Shoup, *A Graduate Course in Applied Cryptography*, ch. 2–3: security games, advantage and the challenger–adversary vocabulary this chapter borrows.

10. Membership inference

A hospital trains a model to predict a disease and publishes it. You hold one patient's record. **Was that record in the training set?** If it was, you have learned that the patient was treated at that hospital for that disease, although the model never output a diagnosis. This is **membership inference**. It is the simplest question an attacker can ask about a trained model, which makes it the standard measure of how much a model leaks. An attack is scored by two rates. Its **true positive rate (TPR)** is the fraction of members it flags as members, and its **false positive rate (FPR)** is the fraction of non-members it wrongly flags.

The one idea of this chapter is that membership inference is a statistical test about one record. Training is random, because the rest of the training set is drawn at random and the optimization varies from run to run. So the model's output on a record, such as its confidence in the record's label, has one distribution over models trained with that record, its **IN distribution**, and another over models trained without it, its **OUT distribution**. The attacker sees one output and must say which of the two produced it. A typical record is predicted well either way, so its two distributions nearly coincide, while an atypical record is predicted badly unless the model saw it. An attack that says "member" whenever the model is confident therefore flags typical records, which get confident outputs even when never trained on, and the best attack compares each record's output with that record's own two distributions.

The running example is a scaled-down, linear version of an experiment by Carlini et al. (2022). A pool of 1,000 labelled points in $\mathbb{R}^{100}$ holds 950 typical points and 50 outliers, whose labels nothing else in the data predicts. Logistic-regression models train on random halves of the pool. The attacker targets 128 of these models. Pairing a model with a pool point gives one guess, and each model trained on 500 of the 1,000 points, so there are $128 \times 500 = 64{,}000$ member pairs and as many non-member pairs. At an FPR of 10^{-3}, the attack that thresholds the loss flags a fraction 0.0009 of members, which is what flagging at random would do, since a random flag hits members and non-members at the same rate. The per-record test of Section 10.3 flags 0.0149, and 87% of the members it finds are outliers.

10.1. The loss attack and the generalization gap

A model is fitted to its training data, so its loss on a training point is usually lower than on a fresh one. This section asks how much membership that difference alone gives away.

Attacks are scored in the membership game of Threat models and privacy games. A challenger draws a training set D and flips a fair coin for a hidden bit b. If $b = 1$, the **target** $z = (x, y)$, a feature vector and its label, is drawn uniformly from D; if $b = 0$, it is a fresh record from the same population. The adversary sees z and the trained model, with weights θ, and guesses b'. Half the games have $b = 1$ and are won with probability TPR, and the other half are won with probability $1 - \text{FPR}$. So by Proposition 9.1 the adversary wins with probability $\frac{1}{2} + \frac{1}{2}\text{Adv}$, where

the **advantage** $\text{Adv} = \text{TPR} - \text{FPR}$ measures how much more often the attack says "member" to members than to non-members.

Yeom et al. (2018) turned the lower loss on members into an attack. Its cleanest version turns the loss into a probability, so the loss must lie in a known range.

Definition 10.1 (Loss attacks)**.** Let the loss $\ell(\theta; z)$ of the model on the target take values in $[0, B]$. The **bounded-loss attack** says "member" with probability $1 - \ell(\theta; z)/B$. The **loss threshold attack** says "member" when $\ell(\theta; z) < \tau$, for a threshold τ such as the mean training loss, which a model's report often states.

The randomized rule makes the probability of saying "member" linear in the loss, so, as the next proposition proves, the advantage is the difference between the average loss of non-members and of members, divided by B. That difference is the **generalization gap**. Write θ_D for the model trained on the training set D, and $\mathcal{D}$ (calligraphic) for the population it is drawn from. The gap is the model's expected loss on fresh data, its **population risk** $\mathcal{L}_{\mathcal{D}}(\theta_D)$, minus its average loss on its own training set, its **empirical risk** $\mathcal{L}(\theta_D; D)$.

Proposition 10.1 (The loss attack's advantage is the generalization gap)**.** *In the membership game, with expectations over the training set, the training run and the target, the bounded-loss attack has an advantage that is first a difference of average losses and then, renamed, the generalization gap:*

$$\text{Adv} = \frac{\mathbb{E}[\ell(\theta; z) \mid b = 0] - \mathbb{E}[\ell(\theta; z) \mid b = 1]}{B} = \frac{\mathbb{E}_D\big[\mathcal{L}_{\mathcal{D}}(\theta_D) - \mathcal{L}(\theta_D; D)\big]}{B}, \tag{10.1}$$

the expected generalization gap divided by B. For the 0-1 loss, which is 0 on a correct prediction and 1 on a wrong one, $B = 1$, the attack says "member" exactly when the model classifies z correctly, and its advantage is the test error minus the training error.

Proof. The plan is to write the TPR and the FPR as average losses, subtract them, and then recognize the two averages as the test loss and the training loss.

1. *The TPR.* Given $b = 1$, the attack says "member" with probability $1 - \ell/B$ for whatever loss the target has. Averaging over targets and training runs gives $\text{TPR} = 1 - \mathbb{E}[\ell \mid b = 1]/B$.
2. *The FPR.* The same calculation with $b = 0$ gives $\text{FPR} = 1 - \mathbb{E}[\ell \mid b = 0]/B$.
3. *Subtract.* The advantage is $\text{TPR} - \text{FPR}$. The two 1s cancel, which leaves the first form of Equation 10.1.
4. *Name the two averages.* When $b = 1$, z is a uniformly chosen training point, so its expected loss is the expected empirical risk. When $b = 0$, z is a fresh draw independent of D, so its expected loss is the expected population risk. Substituting both gives the second form.
5. *The 0-1 loss.* With $B = 1$, the probability $1 - \ell$ is 1 on a correct prediction and 0 on a wrong one. So $\text{TPR} = 1 - \text{train error}$ and $\text{FPR} = 1 - \text{test error}$, and their difference is the test error minus the training error.

□

So **overfitting implies leakage**, by an amount the training and test errors give without any attack. By Theorem 7.1, the expected gap is the average **self-influence** of the training records (Definition 7.3): how much each record's own loss rises when it is replaced in training by a fresh record. But **a small gap does not mean a private model.** The loss attack ignores *which* record it is looking at, and the per-record test of Section 10.3 beats it by a wide margin.

Each attack below reads one of three numbers from the model's output on z. The **confidence** p_y is the probability the model gives to the true label, and the usual classification loss is the cross-entropy $\ell = -\ln p_y$. The **logit of the confidence** is the log of the odds the model gives to the true label,

$$\phi = \ln \frac{p_y}{1 - p_y}. \tag{10.2}$$

The confidence crowds against 1, while ϕ keeps confident outputs apart. That matters because Section 10.3 fits bell curves to these values, and the next lemma shows why a bell curve fits ϕ: for logistic regression, ϕ is linear in the weights. It also shows that a threshold on ϕ is a threshold on the loss, so the rest of the chapter works with ϕ.

Lemma 10.1 (For logistic regression, the logit of the confidence is the margin)**.** *Let a logistic-regression model with weights θ give the true label $y \in \{-1, 1\}$ of a record $z = (x, y)$ the probability $p_y = 1/(1 + e^{-y\langle\theta, x\rangle})$. Then ϕ is the **margin** $y\langle\theta, x\rangle$, how far the point lies on the correct side of the decision boundary, and*

$$\phi = \ln \frac{p_y}{1 - p_y} = y\langle\theta, x\rangle \qquad \text{and} \qquad \ell = -\ln p_y = \ln\left(1 + e^{-\phi}\right). \tag{10.3}$$

So a threshold on any of the confidence, the loss and ϕ is a threshold on the others.

Proof.

1. *The odds.* Write $t = y\langle\theta, x\rangle$. Then $1 - p_y = e^{-t}/(1 + e^{-t})$, so the factor $1 + e^{-t}$ cancels in the ratio $p_y/(1 - p_y)$, which leaves e^t. Its logarithm is $\phi = t$.
2. *The loss.* The reciprocal of the confidence is $1/p_y = 1 + e^{-\phi}$, so the loss $-\ln p_y$ is $\ln(1 + e^{-\phi})$.
3. *Monotone.* The confidence rises with ϕ, and by step 2 the loss falls as ϕ rises. So "ϕ above a cut" is the same event as "loss below the matching cut".

□

Example 10.1. In the running example, each of the 950 typical points has features x drawn from the normal distribution $\mathcal{N}(0, I/100)$, with I the identity, so its 100 independent coordinates each have variance $1/100$ and $\|x\| \approx 1$. Its label $y = \pm 1$ follows the first coordinate with some noise, and the factor 30 makes it follow closely, since x_1 is of order $1/10$:

$$\Pr[y = 1 \mid x] = \frac{1}{1 + e^{-30x_1}}. \tag{10.4}$$

The 50 outliers have the same kind of x scaled by 5 and a label drawn by a fair coin, so a model gets an outlier's label right only by fitting that point itself. Each model is logistic regression with the penalty $\frac{\lambda}{2}\|\theta\|^2$, $\lambda = 10^{-3}$, trained on a random half of the pool ($n = 500$). Each model trained on a random half is followed by one trained on the other half, so every point is a member of exactly half the models.

Over the 128 target models, the training error is 0.1495, and the test error, on the half of the pool each model did not see, is 0.2663. "Member if correctly classified" therefore has TPR $= 1 - 0.1495 = 0.8505$ and FPR $= 1 - 0.2663 = 0.7337$. Its advantage, $0.8505 - 0.7337 = 0.1168$, is exactly the gap $0.2663 - 0.1495$.

The logistic loss is unbounded, so the bounded-loss attack needs it clipped: any loss above $B = 2$ is replaced by 2. The mean clipped loss is 0.4158 on members and 0.5433 on non-members. So Equation 10.1 predicts an advantage of $(0.5433 - 0.4158)/2 = 0.0638$, and simulating the randomized attack gives 0.0639. The loss threshold attack at the mean unclipped training loss, $\tau = 0.4159$ (almost the clipped mean, because few training losses exceed 2), has TPR $= 0.5936$, FPR $= 0.4834$ and advantage 0.1102.

By the win rate $\frac{1}{2} + \frac{1}{2}\text{Adv}$, the three attacks win between 53% and 56% of the games. None of them identifies anyone with confidence, because each flags a large share of non-members too, with an FPR of 0.7337 for the error rule and 0.4834 for the threshold. Figure 10.1 shows why: the two distributions of ϕ are shifted, but they almost entirely overlap.

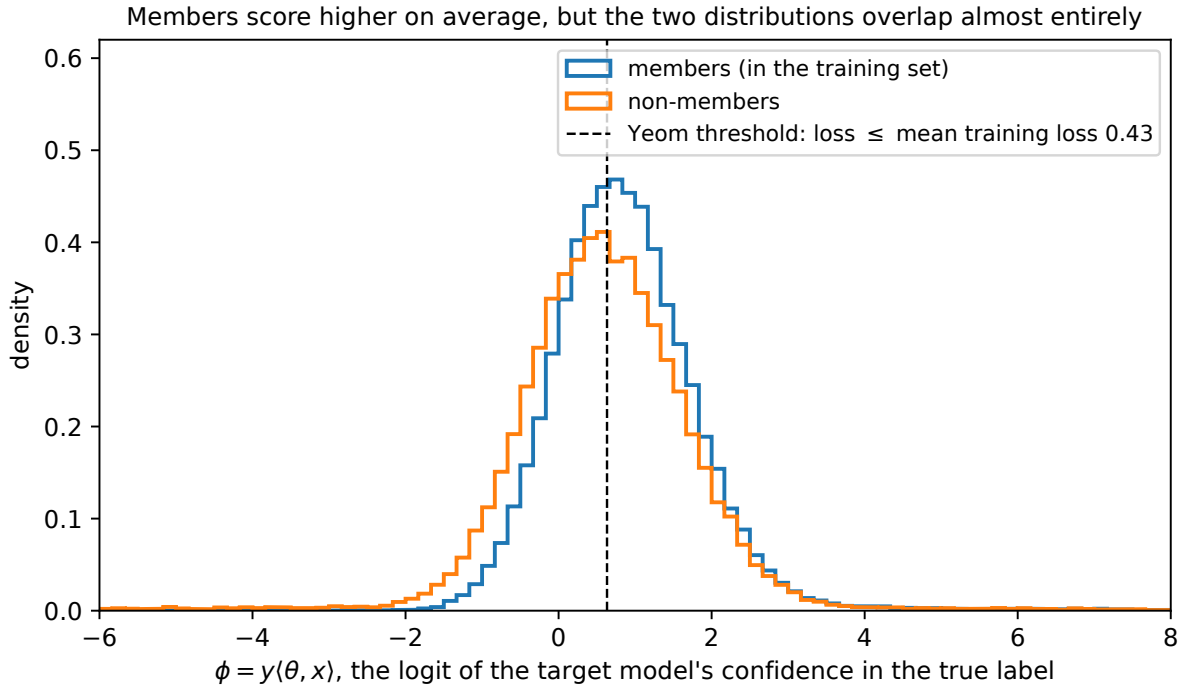


Figure 10.1.: Distribution of ϕ, the logit of the target model's confidence in the true label, over member pairs and non-member pairs of the 128 target models. The dashed line is Yeom's threshold: the attack says "member" to its right, where the loss is below the mean training loss.

10.2. Shadow models

An attacker who wants a better rule than "low loss means member" has to know what members' outputs look like. It has no labelled examples, because it does not know who is in the target's

training set. This section shows how the attacker can learn that from models of its own, whose training sets it chooses (Shokri et al. 2017), and what the best rule learned this way can achieve.

Definition 10.2 (Shadow models and the attack model)**.** Suppose the attacker holds records from the target's population. A **shadow model** is a model that the attacker trains like the target, with the same recipe, on a training set that includes each of its records with probability $\frac{1}{2}$. For each shadow model and record, the attacker keeps the shadow model's output o on the record, such as its confidence, and the membership label m, with $m = 1$ for "in". The **attack model** $h(o) \in (0, 1)$ is a classifier trained on these pairs by minimizing the cross-entropy, $-\ln h$ on "in" pairs and $-\ln(1-h)$ on "out" pairs. It says "member" when $h(o)$ exceeds a threshold.

In the running example the output is $o = (y, \phi)$, the record's label and the logit of the confidence. With enough shadow data, the attack model learns the best rule available from o alone, as the next proposition proves.

Proposition 10.2 (The attack model is the best record-blind attack)**.** *Let $q(o)$ and $p(o)$ be the densities of the output for members and for non-members, each pooled over all the records that could produce o (they are unrelated to the confidence p_y), and let the attack model range over all functions of o. As the number of shadow pairs grows, $h(o)$ tends to the* ***posterior probability of membership****, and with a fair prior the posterior odds are the likelihood ratio:*

$$h(o) \to \Pr[b = 1 \mid o], \qquad \frac{\Pr[b = 1 \mid o]}{\Pr[b = 0 \mid o]} = \frac{q(o)}{p(o)}. \tag{10.5}$$

So no attack that sees only o has a higher TPR, at any FPR, than a threshold on h.

Proof. The proof has four links: the shadow pairs look like the real game, cross-entropy recovers the posterior, the posterior ranks outputs by the likelihood ratio, and a threshold on the likelihood ratio is optimal.

1. *The shadow pairs have the right distribution.* A shadow model is trained like the target, and each record is in or out with probability $\frac{1}{2}$. So a shadow pair (o, m) has the same joint distribution as a pair of target output and membership bit in the game.
2. *Cross-entropy is minimized by the posterior.* Fix one output o, and let $\rho = \Pr[m = 1 \mid o]$ be the probability of "in" given that output. If the attack model predicts h there, its expected loss is $-\rho \ln h - (1-\rho)\ln(1-h)$. This is convex in h, and its derivative $-\rho/h + (1-\rho)/(1-h)$ is zero at $h = \rho$, so that is the minimum. The same holds at every o separately, so the best function is $h(o) = \Pr[m = 1 \mid o]$, which by step 1 is $\Pr[b = 1 \mid o]$.
3. *Bayes' rule.* The joint probability of a world, $b = 1$ or $b = 0$, and an output is the prior of the world times the density of the output in that world. So the posterior odds are the prior odds times $q(o)/p(o)$, and with a fair prior the prior odds are 1. The odds $h/(1-h)$ rise with h, so a threshold on h is a threshold on $q(o)/p(o)$.
4. *Neyman–Pearson.* The Neyman–Pearson lemma says that a threshold on the likelihood ratio has the highest TPR at its FPR among all tests that see the same observation.

□

The attack model is **optimal for the wrong question.** Seeing (y, ϕ) and not which record produced it, it cannot know that $\phi = 2$ is high for one record and low for another. When members tend to have the higher ϕ within each class, h rises with ϕ there, so a threshold on h is a threshold on ϕ and, by Lemma 10.1, on the loss: the loss attack with a threshold for each class.

What shadow models really buy is **calibration.** The attacker picks the threshold at which the desired fraction of its own shadow non-members pass, and trusts that real non-members behave the same way. Salem et al. (2019) found that one shadow model, or shadow data from another distribution, often suffices.

Example 10.2. For each class, the attack model is a logistic regression of "in" on ϕ with an intercept, trained on the roughly 32,000 shadow pairs of that class (the 64 shadow models times the roughly 500 pool points with that label). Both slopes are positive, 0.238 for $y = -1$ and 0.158 for $y = +1$. So within each class the attack is a threshold on the loss.

To compare it with the loss attack over all thresholds, draw each attack's **receiver operating characteristic (ROC) curve**, its TPR against its FPR as the threshold sweeps. The **area under the curve (AUC)** equals the probability that a random member scores higher than a random non-member (Hypothesis testing). The attack model and the loss attack both have AUC 0.577, and the same TPR at every FPR to within sampling noise (Table 10.3).

Calibration works: a threshold set on the shadow models for an FPR of 1% gives an FPR of 0.0095 on the 128 unseen target models. But the TPR there is 0.0098, no better than chance, because a threshold on ϕ that ignores which record it sees cannot single out any member at a low FPR.

10.3. LiRA: a likelihood-ratio test per record

The attacks of the last two sections compare a record's output with everyone else's outputs. The right comparison is with the *same record's* output under models that did and did not train on it. This section builds that test and proves that it is the best one.

Carlini et al. (2022) built the **likelihood ratio attack (LiRA)** on that comparison. For a record z, the attacker has a few dozen shadow values of ϕ_z, the logit of the confidence on z, and summarizes them by bell curves, a shape justified after the theorem below.

Definition 10.3 (LiRA, online and offline)**.** For a record z, let $\mu_{\text{in},z}$ and $\sigma_{\text{in},z}$ be the mean and standard deviation of ϕ_z over the shadow models that trained on z, and $\mu_{\text{out},z}$ and $\sigma_{\text{out},z}$ those over the shadow models that did not. Write $\mathcal{N}(\phi; \mu, \sigma^2)$ for the normal density at ϕ. **Online LiRA** scores the target model's ϕ_z by the log of the ratio of the IN fit to the OUT fit,

$$\ln \Lambda_z(\phi_z) = \ln \mathcal{N}(\phi_z;\, \mu_{\text{in},z}, \sigma^2_{\text{in},z}) - \ln \mathcal{N}(\phi_z;\, \mu_{\text{out},z}, \sigma^2_{\text{out},z}). \tag{10.6}$$

Offline LiRA uses only the OUT fit, and scores how many OUT standard deviations the output lies above its usual value,

$$\frac{\phi_z - \mu_{\text{out},z}}{\sigma_{\text{out},z}}. \tag{10.7}$$

Each says "member" when its score exceeds a threshold that is **the same for every record**.

Online LiRA needs shadow models that trained on each new target record, while offline LiRA's OUT models can be trained once and reused. One threshold serves all records because, as the theorem below shows, the record is drawn the same way in both worlds, so it drops out of the likelihood ratio that the Neyman–Pearson lemma (Theorem 4.1) thresholds.

Theorem 10.1 (One threshold on the per-record likelihood ratio is optimal)**.** *In the membership game, suppose the target record z is drawn from the same distribution whether $b = 1$ or $b = 0$, and let q_z and p_z be the IN and OUT densities of ϕ_z. Among all attacks that see z and ϕ_z, the highest TPR at each FPR is reached by saying "member" when $\Lambda_z(\phi_z) = q_z(\phi_z)/p_z(\phi_z)$ exceeds one threshold, the same for every record. Online LiRA estimates this true ratio by its Gaussian fits, so when the fits are exact the test is online LiRA.*

Proof. The plan is to write the density of the adversary's view in each world, see that the record's part cancels in the likelihood ratio, and apply the Neyman–Pearson lemma.

1. *The view is a pair.* The density of the pair is the probability of drawing the record, call it $g(z)$, times the density of the output given the record. Write it as $g(z)\, q_z(\phi)$ when $b = 1$ and as $g(z)\, p_z(\phi)$ when $b = 0$.
2. *The record's factor is shared.* By hypothesis, the record is drawn the same way whether or not it ends up in the training set, so g is the same function in both worlds. The running example satisfies this, since every pool point is a member of each model with probability $\frac{1}{2}$.
3. *The ratio.* Since the factor $g(z)$ is the same in both worlds, it cancels:

$$\frac{g(z)\, q_z(\phi)}{g(z)\, p_z(\phi)} = \frac{q_z(\phi)}{p_z(\phi)} = \Lambda_z(\phi). \tag{10.8}$$

4. *Neyman–Pearson.* The best test of the pair thresholds its likelihood ratio at one value, by Theorem 4.1. So different records are compared through their ratios, never through their raw ϕ. With exact Gaussian fits, the log of the ratio is Equation 10.6.

□

The hypothesis is needed, since if members came from one hospital and non-members from another, the record alone would reveal b. The theorem is an instance of Lemma 9.2 in Threat models and privacy games, which says that an attacker who does not know a hidden part of the data does no better than one who knows it. An attacker that ignores the record sees the IN and OUT distributions averaged over all records, where the large shift of an atypical record is diluted among the small shifts of the typical ones. The bell-curve shape is justified because, by Lemma 10.1, ϕ_z is linear in the weights. The regularized estimate θ sets the gradient of the training objective to zero, and that gradient averages n independent per-record terms, so θ is approximately Gaussian over the training randomness, by the central limit theorem that Section G.6 proves for such estimates. So ϕ_z is close to normal, and for deep networks Carlini et al. found the same.

Offline LiRA is a weaker test, because without IN models it cannot write down q_z. It only asks whether ϕ_z lies unusually far above its OUT values, the direction in which training on z moves it, since fitting z raises its margin. The two scores can be compared exactly in the simplest case, where the IN and OUT fits have the same standard deviation σ_z. Write t_z for the offline score,

the output measured in units of σ_z from the OUT mean. Then t_z is standard normal under OUT, and under IN it is normal with variance 1 and mean the **separation**

$$s_z = \frac{\mu_{\text{in},z} - \mu_{\text{out},z}}{\sigma_z}. \tag{10.9}$$

Measured in t_z, the OUT density is proportional to $e^{-t_z^2/2}$ and the IN density to $e^{-(t_z - s_z)^2/2}$, with the same constant, which cancels in Equation 10.6. Expanding the square leaves $\ln \Lambda_z = s_z t_z - s_z^2/2$. For one record s_z is a fixed positive number, so both scores rank that record's outputs alike. Across records they differ, because online LiRA weights each t_z by s_z, while offline LiRA treats every t_z the same.

How much each record can be caught at a given FPR is then a question about two unit-variance Gaussians. Write Φ for the standard normal distribution function, so that $\Phi^{-1}(1-\alpha)$ is the point a standard normal exceeds with probability α. Theorem 4.3 in Hypothesis testing shows that when an observation is $\mathcal{N}(0,1)$ in one world and $\mathcal{N}(s,1)$ in the other, the most members any test can catch at FPR α is

$$\text{TPR}(\alpha) = 1 - \Phi(\Phi^{-1}(1-\alpha) - s). \tag{10.10}$$

At $\alpha = 10^{-3}$ the cut-off is 3.0902. With a separation of 0.5, a member must still clear $3.0902 - 0.5 = 2.59$ standard deviations above its own mean, so the TPR is $1 - \Phi(2.59) = 0.0048$. With a separation of 4, the cut-off sits below the member mean, and the TPR is $1 - \Phi(-0.91) = 0.8185$.

Example 10.3. On a population with two kinds of record, this formula shows in closed form how much the per-record comparison gains (Table 10.1). A typical record is predicted confidently whether or not the model trained on it. An atypical record has a negative ϕ, which means it is misclassified, unless the model trained on it.

Table 10.1.: Two kinds of record. The last column is Equation 10.10.

Kind	Share	OUT: $\phi \sim$	IN: $\phi \sim$	Separation s	Best TPR at FPR 10^{-3}, this kind alone
Typical	95%	$\mathcal{N}(3,1)$	$\mathcal{N}(3.5,1)$	0.5	0.0048
Atypical	5%	$\mathcal{N}(-2,1)$	$\mathcal{N}(2,1)$	4	0.8185

Now set three attacks to an FPR of 10^{-3} pooled over both kinds, that is, counted over all non-members together.

One threshold on ϕ is the loss attack. Typical records make up 95% of non-members and their OUT values are high, so the threshold must sit high. Solving for the point that 0.1% of all non-members exceed puts it at $\phi = 6.075$. Atypical members score around 2 and almost never pass it. Only typical members occasionally do, and the pooled TPR is 0.0048. That matches the typical kind's best TPR alone, because typical records dominate the non-members, so the threshold sits almost where the typical kind alone would put it, at $3 + 3.0902$.

One threshold on the z-score is offline LiRA. Each record is standardized against its own OUT distribution, so every record gets FPR 10^{-3}, at $t \geq 3.0902$. Each kind then gets its own best TPR from the table, and the pooled TPR is their weighted average, $0.95 \times 0.0048 + 0.05 \times 0.8185 = 0.0455$.

One threshold on $\ln \Lambda$ is online LiRA. A one-dimensional numerical search for a pooled FPR of 10^{-3} puts the threshold at 1.497. Since $\ln \Lambda = st - s^2/2$, the cut $\ln \Lambda \geq 1.497$ is the z-score cut $t \geq (1.497 + s^2/2)/s$. That is $t \geq 3.244$ for typical records, with an FPR of only 5.90×10^{-4}. For atypical ones it is $t \geq 2.374$, with FPR 0.0088. The pooled FPR is $0.95 \times 5.90 \times 10^{-4} + 0.05 \times 0.0088 = 10^{-3}$, and the pooled TPR is 0.0503, the best possible at that FPR by Theorem 10.1.

The global threshold is ten times worse than either per-record test. The likelihood ratio beats the z-score because it spends its allowance of false positives where they pay. A typical record's IN and OUT curves overlap, so admitting more of its false positives admits almost no extra members. An atypical record's curves lie far apart, so a few more false positives there admit many members.

Example 10.4. Figure 10.2 shows the IN and OUT distributions of a typical point and an outlier of the running example. On the 128 target models, online LiRA reaches a TPR of 0.0149 at FPR 10^{-3}, against 0.0009 for the loss attack (Table 10.3).

With few shadow models, each record's estimated standard deviations are noisy, and the noise hurts most at a low FPR. A record whose spread is underestimated has its OUT curve drawn too narrow, so an ordinary output looks far out in the tail and becomes a false positive at a strict threshold. Carlini et al. therefore recommend a **global variance** when shadow models are few: one IN variance and one OUT variance, each the average over all records (Figure 10.3 compares online and offline LiRA, each with both kinds of variance). With 16 shadow models, online LiRA with variances per record reaches 0.0023, barely above chance, while with a global variance it reaches 0.0151.

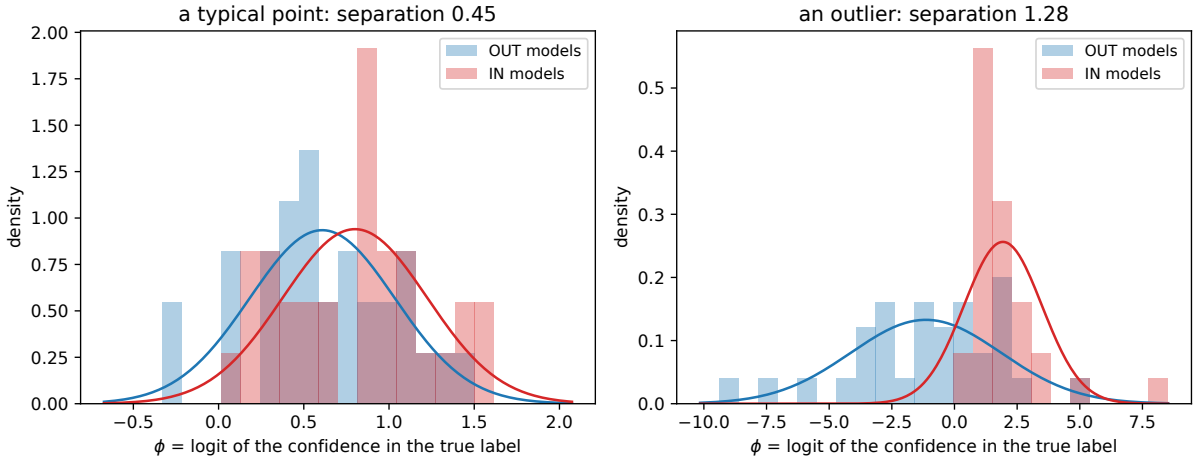


Figure 10.2.: The logit ϕ of one point over the 64 shadow models, split into the 32 that trained on it (IN) and the 32 that did not (OUT), with the fitted Gaussians. Left: a typical point, whose two distributions overlap. Right: an outlier, which the models predict wrongly unless they trained on it.

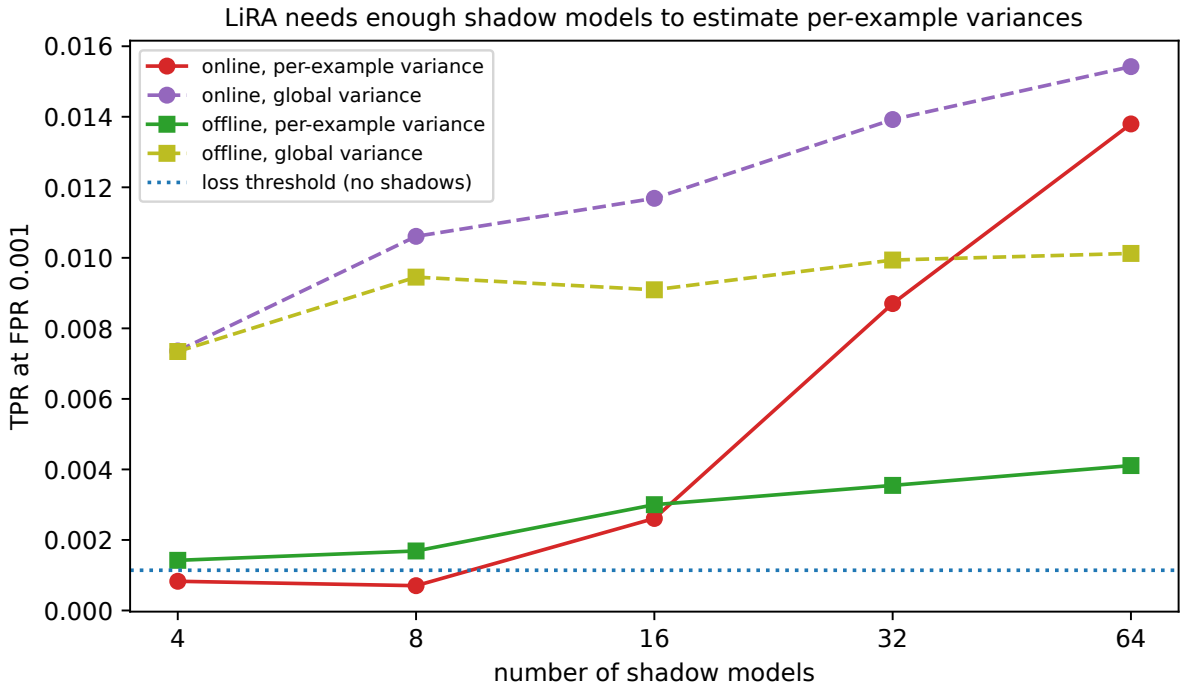


Figure 10.3.: TPR at FPR 10^{-3} of four LiRA variants on the 128 target models, as the attacker uses the first 4, 8, 16, 32 or 64 of its shadow models. With a global variance the attack is strong even with few shadow models; with a variance per record it needs many. The dotted line is the loss attack, which needs no shadow models.

10.4. Evaluating attacks: TPR at a low FPR

Older papers report an attack's accuracy, which is $\frac{1}{2} + \frac{1}{2}\text{Adv}$ in the membership game, or its AUC, and both average over all members and non-members. This section asks which number says whether an attack is a threat, starting from the fact that a privacy breach is not an average. It happens when an attacker can say *with confidence* that a particular person was in the data, and that needs a low FPR, where almost every "member" verdict is right.

Definition 10.4 (TPR at a low FPR)**.** An attack's **TPR at FPR** α is the height of its ROC curve at α, the largest fraction of members it flags with a threshold that flags at most a fraction α of non-members. Carlini et al. (2022) report it at a low α such as 10^{-3}, together with the whole ROC curve on log axes.

Two attacks can have the same AUC and best advantages (the largest TPR − FPR over all thresholds) within 0.015 of each other, yet TPRs at FPR 10^{-3} that differ by a factor of 38. In the pair below, attack A exposes a few members completely and learns nothing about anyone else, while attack B learns a little about everyone.

Proposition 10.3 (Same AUC, different exposure)**.** *Let $r \in (0, 1)$ be a fraction of members. **Attack A** gives a fraction r of members a score above every other record's, and gives every other record, member or not, a random score from one common distribution. **Attack B** gives members a score $\mathcal{N}(\mu, 1)$ and non-members a score $\mathcal{N}(0, 1)$, with $\mu > 0$. Then A has AUC $\frac{1}{2} + \frac{r}{2}$, best advantage r and*

$$\text{TPR}_A(\alpha) = r + (1 - r)\alpha, \tag{10.11}$$

while B has AUC $\Phi(\mu/\sqrt{2})$, best advantage $2\Phi(\mu/2) - 1$, and the TPR of Equation 10.10 with $s = \mu$. The two AUCs are equal when $\Phi(\mu/\sqrt{2}) = \frac{1}{2} + \frac{r}{2}$.

Proof. The plan is to compute each attack's AUC from its definition as a ranking probability, A's TPR by splitting members into recognized and not, and B's best advantage from the total variation distance.

1. *AUC of A.* The AUC is the probability that a random member outscores a random non-member, with ties counted as one half (Proposition 4.2). With probability r the member is a recognized one, and it wins. Otherwise both scores have the same distribution, so the member wins with probability $\frac{1}{2}$. The total is $r + (1 - r)\frac{1}{2} = \frac{1}{2} + \frac{r}{2}$.
2. *TPR of A.* At a threshold with FPR α, the recognized members always pass. The other members have the same score distribution as non-members, so they pass with the same probability α, which gives $\text{TPR}_A(\alpha)$. Then $\text{TPR} - \text{FPR} = r(1 - \alpha)$, which is at most r.
3. *AUC of B.* A member outscores a non-member when the difference of their scores is positive. That difference is $\mathcal{N}(\mu, 1)$ minus an independent $\mathcal{N}(0, 1)$, so it is normal with mean μ and variance 2, and it is positive with probability $\Phi(\mu/\sqrt{2})$.
4. *Advantage of B.* The best advantage of a test between two distributions is their **total variation distance**, the most by which the two can differ in the probability of one set of scores (Theorem 4.5). The best set is where the member density exceeds the non-member density, which for two unit-variance Gaussians at distance μ is the scores above the midpoint $\mu/2$. Its TPR is $\Phi(\mu/2)$ and its FPR is $1 - \Phi(\mu/2)$, and their difference is $2\Phi(\mu/2) - 1$.

□

With $r = 0.05$, both AUCs are 0.5250 when $\mu = 0.0887$ (Table 10.2). At FPR 10^{-3}, A has TPR $0.05 + 0.95 \times 10^{-3} = 0.05095$, while B has $1 - \Phi(3.0902 - 0.0887) = 0.00134$, which is 38 times less.

Table 10.2.: Two attacks with the same AUC. Only the TPR at a low FPR tells them apart.

	AUC	Best advantage	TPR at FPR 10^{-3}
A: 5% of members exposed	0.5250	0.0500	0.05095
B: every member shifted by $\mu = 0.0887$	0.5250	0.0354	0.00134

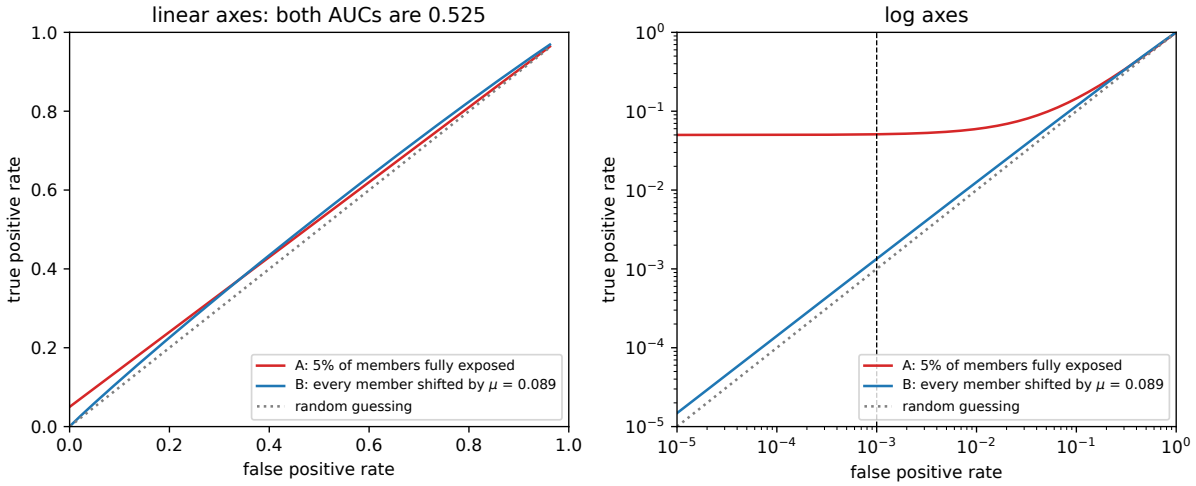


Figure 10.4.: ROC curves of attacks A and B. Left, linear axes: the two curves are indistinguishable from each other and from the diagonal. Right, log axes: A identifies 5% of members at every FPR, B is barely above chance. The dashed line marks FPR 10^{-3}.

Who is exposed matters more than how many. Only the low-FPR end of the ROC curve shows the difference, and only on log axes, since linear axes squeeze everything below FPR 0.01 into one pixel column (Figure 10.4).

Measuring a low FPR takes many non-members. An FPR is estimated by counting false positives, and a count of k events has a relative error of about $1/\sqrt{k}$ (Hypothesis testing). To see k false positives at FPR α you need about k/α non-members, which is why the running example uses 64,000 non-member pairs.

A low FPR makes the verdicts reliable. With as many members as non-members, the **precision**, the fraction of "member" verdicts that are right, is $\mathrm{TPR}/(\mathrm{TPR} + \mathrm{FPR})$, so a TPR of 0.05 at FPR 10^{-3} gives $0.05/0.051 = 0.98$.

Example 10.5. Table 10.3 scores every attack on the 128 target models, and Figure 10.5 draws their ROC curves on log axes.

Table 10.3.: Every attack on the running example: 128 target models, 64 shadow models, 64,000 member and 64,000 non-member pairs.

Attack	AUC	Best advantage	TPR at FPR 10^{-3}	TPR at FPR 10^{-2}
Loss threshold (Yeom)	0.577	0.122	0.0009	0.0103
Shadow attack model (Shokri)	0.577	0.121	0.0008	0.0104
LiRA offline	0.632	0.189	0.0030	0.0273
LiRA online	0.629	0.175	0.0149	0.0386
LiRA online, global variance	0.623	0.169	0.0177	0.0396

By AUC the attacks are close, 0.577 against about 0.63, but at FPR 10^{-3} they are not. The loss and shadow attacks are at chance, since their TPRs of 0.0009 and 0.0008 are no higher than the FPR of 0.001. Online LiRA finds 952 of the 64,000 members, a TPR of 0.0149 and 17.0 times the loss attack's TPR before rounding.

The **Clopper–Pearson interval**, an exact 95% confidence interval for a proportion estimated from a count, is $[0.0140, 0.0158]$ for that TPR. The FPR rests on 64 false positives among 64,000 non-members, and its interval is $[0.00077, 0.00128]$. One caveat is that **the threshold was chosen on the same data** that measure the rates, as is usual in this literature, while a real attacker would set it on its shadow models.

10.5. Why outliers are most exposed

Example 10.3 showed that the low-FPR end of the ROC curve is made of the records with a large separation between their IN and OUT distributions. This section asks which records those are, and finds that they are the ones the model cannot predict without having seen them.

Proposition 10.4 (Exposure of one record)**.** *Let the IN and OUT distributions of ϕ_z for a record z be Gaussian, with means $\mu_{\text{in},z}$ and $\mu_{\text{out},z}$ and a common standard deviation σ_z. The best TPR of any test on z alone at FPR α is*

$$\text{TPR}_z(\alpha) = 1 - \Phi(\Phi^{-1}(1-\alpha) - s_z), \qquad s_z = \frac{\mu_{\text{in},z} - \mu_{\text{out},z}}{\sigma_z}, \tag{10.12}$$

*and the numerator $\mu_{\text{in},z} - \mu_{\text{out},z}$ is the **self-influence** of z measured in ϕ: the average amount by which training on z raises the model's ϕ on z.*

Proof.

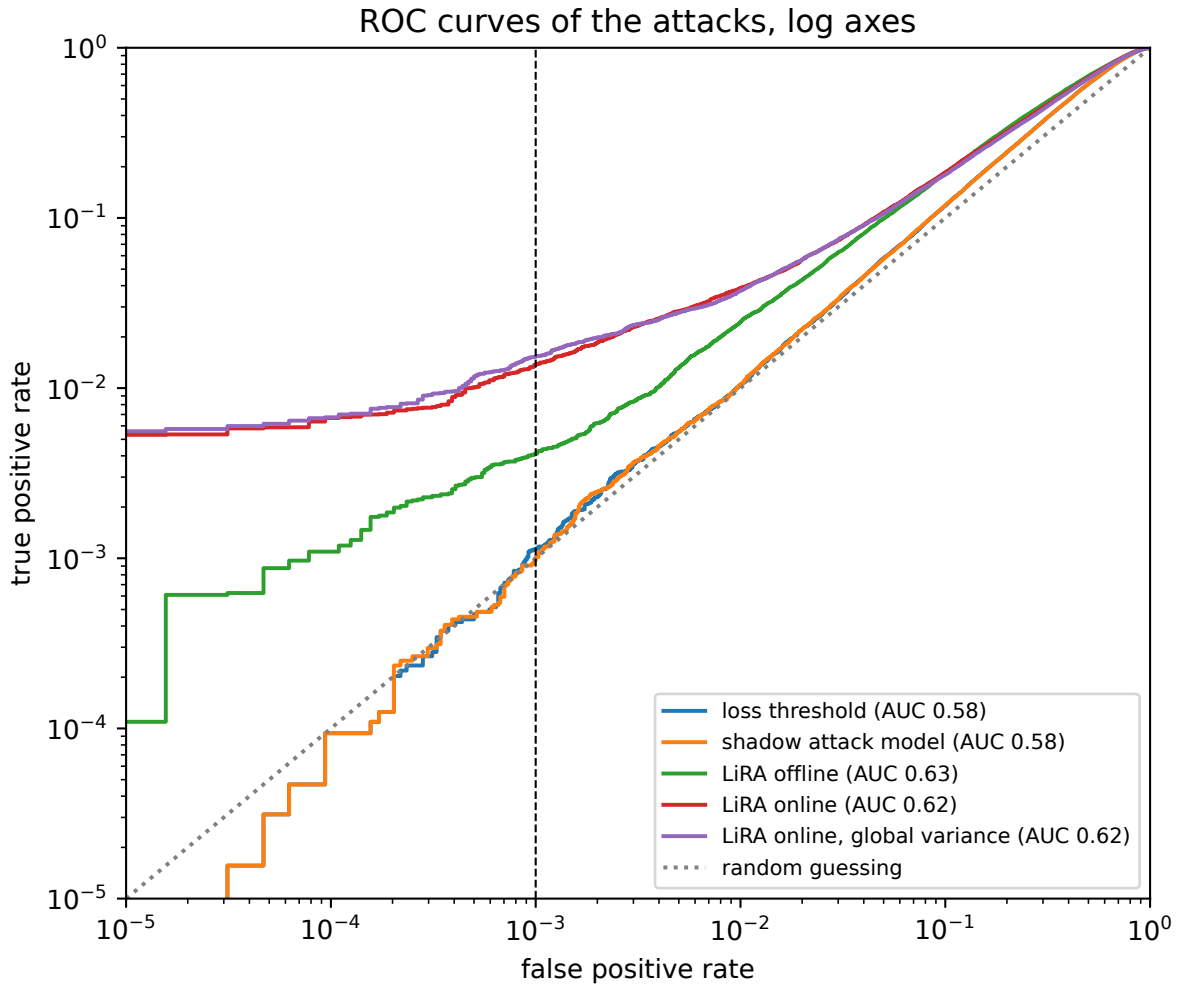


Figure 10.5.: ROC curves of the five attacks on log axes, over the 128 target models. The loss threshold and the shadow attack model follow the diagonal below FPR 0.01; online LiRA stays far above it. The dashed line marks FPR 10^{-3}.

1. *The test.* Standardizing the output, as in Section 10.3, gives a standard normal under OUT and $\mathcal{N}(s_z, 1)$ under IN. So Equation 10.10 with $s = s_z$ is the best TPR.
2. *Self-influence.* Definition 7.3 defines the self-influence of a training record z_i as $\ell(\theta_{D^{(i)}}; z_i) - \ell(\theta_D; z_i)$. That is the loss of the model trained on $D^{(i)}$, the training set with z_i replaced by a fresh record, minus the loss of the model trained on D itself. Measured with ϕ instead of ℓ, the sign flips, because ϕ rises where the loss falls.
3. *Average it.* Take $z_i = z$ and average over the rest of the training set, the fresh record and the training run. The models trained on D contain z, so they give ϕ_z the mean $\mu_{\text{in},z}$. The models trained on $D^{(i)}$ do not, so they give $\mu_{\text{out},z}$. The averaged self-influence in ϕ is the difference of the two means.

□

Which records have a large self-influence? If the rest of the data already predict z's label, $\mu_{\text{out},z}$ is high and training on z adds little. If they predict the opposite label, $\mu_{\text{out},z}$ is low, and the model must move toward z to fit it. The regularization limits how far the model can move. By Theorem 7.2, one record moves the weights of a regularized fit by at most $2G/(\lambda n)$, where G bounds the length of a record's loss gradient, and its proof shows that each record's pull scales with the length of its own gradient. For logistic regression that gradient is x times a probability, so the pull of z scales with at most $\|x_z\|$. A move of the weights changes the margin $\phi_z = y\langle\theta, x_z\rangle$ by at most $\|x_z\|$ times the length of the move. So a long x_z both pulls the weights harder and turns that pull into a larger change in its own margin, and the outliers of the running example have long features and unpredictable labels.

Leakage is concentrated. The loss attack's advantage reflects the *average* self-influence, while the low-FPR end of the ROC curve is governed by the *largest* ones, because a strict threshold is passed only by the records with the largest separations. So a model with a small generalization gap can still expose a few records completely. Memorizing them can even be what makes it accurate: Feldman (2020) shows that on long-tailed data, a rare subpopulation seen once in training must be memorized for good test accuracy.

Removing the outliers does not fix it. Carlini et al. (2022b) removed the most exposed records and retrained, and the next layer of records became exposed, which they call the privacy onion effect. Exposure is relative to the rest of the dataset, because a record's OUT distribution depends on what the other records predict about it. A guarantee that protects every record, the subject of Part 3, has to bound the self-influence of the worst record, not the average.

Example 10.6. Table 10.4 splits the running example by kind of point.

Table 10.4.: The running example, split into typical points and outliers. Separations and self-influences are estimated from the 64 shadow models; TPR and FPR are measured on the 128 target models.

	Typical points (950)	Outliers (50)
Mean separation s_z	0.461	1.539
Mean loss self-influence, $\mathbb{E}[\ell \mid \text{out}] - \mathbb{E}[\ell \mid \text{in}]$	0.0897	2.0999

	Typical points (950)	Outliers (50)
Mean of Equation 10.12 at $\alpha = 10^{-3}$	0.0074	0.2145
Online LiRA at pooled FPR 10^{-3}: TPR	0.0020	0.2597
Online LiRA at pooled FPR 10^{-3}: FPR	0.00086	0.00375

Outliers carry the gap. The mean loss self-influence over all points weights the two kinds by their shares, $0.95 \times 0.0897 + 0.05 \times 2.0999 = 0.1902$. This is a gap in logistic loss on the shadow models, not the error gap of Example 10.1. The outliers' part of that is $0.05 \times 2.0999 \approx 0.105$, a share of 0.552 of the total, from 5% of the points.

Outliers are the true positives. At FPR 10^{-3}, online LiRA flags 26% of outlier members and 0.2% of typical members. Each target model has about 25 outlier members and 475 typical members, so it finds about $0.26 \times 25 = 6.5$ outliers and $0.002 \times 475 = 0.95$ typical points, and over the 128 target models a share of 0.873 of the members it finds are outliers. As in Example 10.3, the likelihood ratio spends its false positives on them. Typical points get an FPR of 0.00086 and outliers 0.00375, whose weighted average is the pooled 10^{-3}.

Not every outlier is exposed. Figure 10.6 plots each point's separation against $\mu_{\text{out},z}$. About half the outliers have a coin-flip label that happens to agree with the rule that labels typical points by x_1, so they are predicted correctly without being seen and are no more exposed than typical points.

10.6. The full-size pipeline: LiRA on CIFAR-10

The usual full-scale experiment runs LiRA with 16 shadow models on an image classifier for CIFAR-10, a benchmark of small colour images in ten classes. This section states the protocol and what changes at full size. The running example is this protocol at small scale, with logistic regression in place of a ResNet, a deep convolutional network.

Definition 10.5 (The LiRA evaluation protocol)**.** The protocol has six steps.

1. *Data split.* Take the 50,000 CIFAR-10 training images as the pool, and give every model, target and shadows alike, a random half of it. One model plays the target and the others the shadows, and rotating the target gives many pairs to evaluate.
2. *Training.* Train every model with the target's recipe, including its architecture (wide ResNets), optimizer, epochs and data augmentation, which is what gives the shadow pairs the target's distribution (Proposition 10.2, step 1).
3. *Statistic.* For each model and image, compute the network's raw class scores f on the image and on a few augmented copies, such as horizontal flips and small shifts. Convert each to ϕ as f_y minus the log of the sum of $e^{f_{y'}}$ over the other classes. This equals $\ln(p_y/(1-p_y))$,

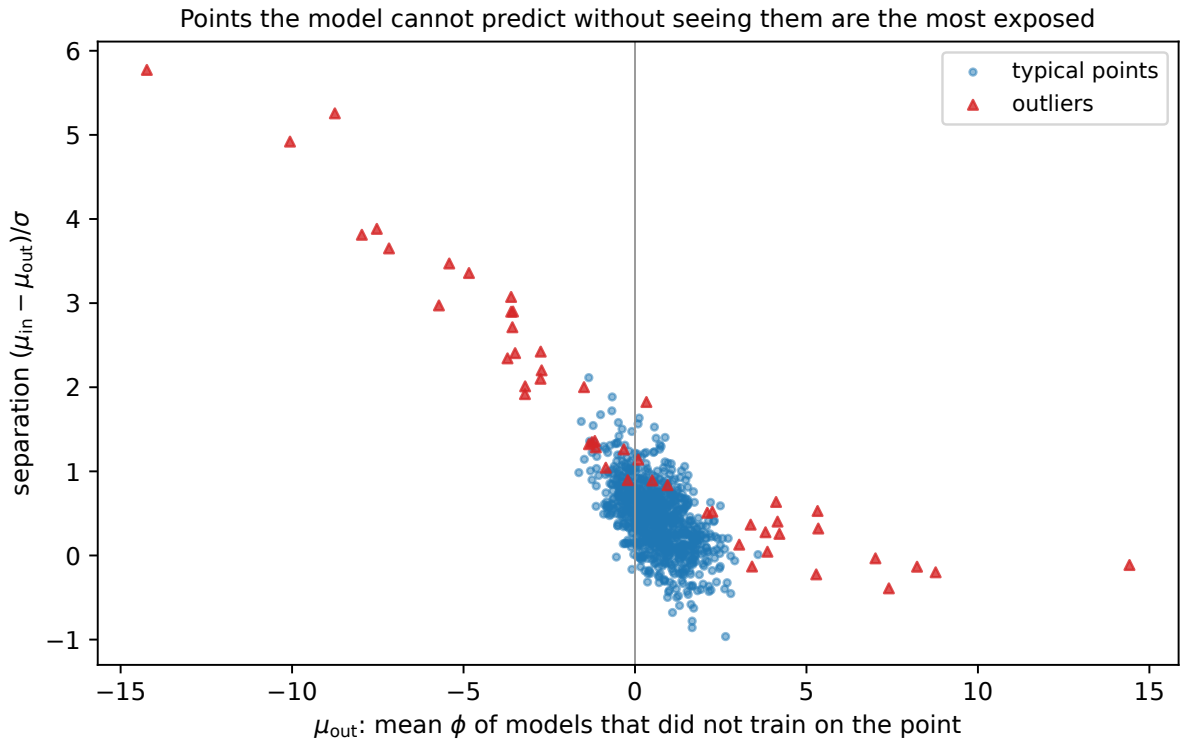


Figure 10.6.: Each pool point's separation $(\mu_{\text{in}} - \mu_{\text{out}})/\sigma$ against μ_{out}, the mean ϕ of the shadow models that did not train on it. Outliers (triangles) whose label the rest of the data contradict have the largest separations.

because the softmax sets $p_y = e^{f_y} / \sum_{y'} e^{f_{y'}}$, but it never forms $1 - p_y$, a number that rounds to 0 for a confident model.

4. *Fits.* For each image, fit IN and OUT Gaussians over the shadow models, with one mean per augmented copy and variances per record or global.
5. *Scores.* For each target model and image, compute the online score Equation 10.6 or the offline score Equation 10.7.
6. *Evaluation.* Pool all pairs of target model and image, sweep one threshold, and plot the ROC curve on log axes. Report the TPR at FPR 10^{-3}, with the number of non-member pairs it rests on.

With 16 shadow models, use a global variance, because each image is then out of only about 8 shadows, and Figure 10.3 shows that variances per record from so few values fail at the low-FPR end.

Carlini et al. report that online LiRA is about ten times more powerful than earlier attacks at a low FPR on CIFAR-10, and that the loss attack is close to chance there, as Table 10.3 shows in miniature. The auditing part of the book reuses this pipeline to measure privacy leakage. A privately trained model takes the place of the target, and inserted **canaries**, records such as mislabelled images chosen to be as exposed as possible, play the members.

10.7. Code

The script code/membership_inference.py builds the pool, trains the 128 target and 64 shadow models, runs every attack, and prints every number in this chapter, including the closed-form examples. From the repository root:

```
uv run python code/part02-attacks/membership_inference.py
```

Table 10.5.: The script's output with the default seed.

Experiment	Result
Generalization gap	Train error 0.1495, test error 0.2663; "correct means member": TPR 0.8505, FPR 0.7337, advantage 0.1168
Bounded loss, $B = 2$	Clipped loss: members 0.4158, non-members 0.5433; gap/B 0.0638; simulated advantage 0.0639
Yeom threshold	$\tau = 0.4159$: TPR 0.5936, FPR 0.4834, advantage 0.1102
Shadow attack model	Slopes 0.238 and 0.158; threshold set on shadows for FPR 1%: target FPR 0.0095, TPR 0.0098
Attacks	Table 10.3

Experiment	Result
LiRA online at FPR 10^{-3}	952 true positives of 64,000, interval $[0.0140, 0.0158]$; 64 false positives of 64,000, interval $[0.00077, 0.00128]$; 17.0 times the loss attack
Two kinds of record	Global threshold at $\phi = 6.075$: TPR 0.0048. z-score: 0.0455. Likelihood ratio, $\ln \Lambda \geq 1.497$: 0.0503, with FPRs 5.90×10^{-4} and 0.0088
Same AUC	A: AUC 0.5250, advantage 0.0500, TPR 0.05095. B: $\mu = 0.0887$, advantage 0.0354, TPR 0.00134
Outliers	Table 10.4; mean loss self-influence 0.1902, outliers' share 0.552; outliers' share of true positives 0.873
Shadow models, TPR at 10^{-3} for 4, 8, 16, 32, 64	Online, per-record variance: 0.0015, 0.0009, 0.0023, 0.0086, 0.0149. Online, global: 0.0092, 0.0127, 0.0151, 0.0171, 0.0177. Offline, per-record: 0.0018, 0.0023, 0.0029, 0.0035, 0.0030. Offline, global: 0.0098, 0.0111, 0.0112, 0.0114, 0.0118

tests/test_membership_inference.py checks that the output still matches.

10.8. Summary

Table 10.6.: The results of this chapter, and where each is used.

Result	Statement	Use
Proposition 10.1	$\text{Adv} = \mathbb{E}[\text{gap}]/B$	Overfitting implies leakage, but no gap does not mean no leakage
Lemma 10.1	$\phi = y\langle\theta, x\rangle$	Confidence, loss and ϕ give one attack; fit Gaussians to ϕ
Proposition 10.2	The attack model tends to $\Pr[b = 1 \mid o]$	Best record-blind attack; shadows calibrate thresholds
Theorem 10.1	One threshold on Λ_z is optimal	Optimal when the Gaussian fits are right; a global loss threshold is near chance at a low FPR
Definition 10.3	Online and offline scores	Offline reuses OUT models; with few shadow models (16 here), use a global variance

Result	Statement	Use
Proposition 10.3	Equal AUCs, TPRs a factor 38 apart	Report TPR at FPR 10^{-3} on log axes with the non-member count, never AUC alone
Proposition 10.4	Best TPR set by s_z, a scaled self-influence	The exposed are records the rest of the data mispredict, such as outliers
Definition 10.5	LiRA on CIFAR-10	Audits private training with canaries

10.9. References

- Shokri, Stronati, Song & Shmatikov, "Membership Inference Attacks against Machine Learning Models" (IEEE S&P 2017). Shadow models and attack models, one per class.
- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (IEEE CSF 2018). The membership experiment, the loss attack and its advantage as the generalization gap; also attribute inference, used in Attribute inference and model inversion.
- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022). LiRA online and offline, logit scaling, global variance, and the case for TPR at low FPR on log-scale ROC curves.- Salem, Zhang, Humbert, Berrang, Fritz & Backes, "ML-Leaks: Model and Data Independent Membership Inference Attacks and Defenses on Machine Learning Models" (NDSS 2019). One shadow model, or none, is often enough.
- Sablayrolles, Douze, Schmid, Ollivier & Jégou, "White-box vs Black-box: Bayes Optimal Strategies for Membership Inference" (ICML 2019). Under assumptions on training noise, the optimal attack depends on the parameters only through the loss, calibrated per example.
- Watson, Guo, Cormode & Sablayrolles, "On the Importance of Difficulty Calibration in Membership Inference Attacks" (ICLR 2022). Subtracting a per-example reference loss: the step from the loss attack toward offline LiRA.
- Ye, Maddi, Murakonda, Bindschaedler & Shokri, "Enhanced Membership Inference Attacks against Machine Learning Models" (CCS 2022). A hypothesis-testing framework that unifies population, reference-model and per-example attacks.
- Zarifzadeh, Liu, Backes & Shokri, "Low-Cost High-Power Membership Inference Attacks" (ICML 2024). Comparing the target with population records gives strong attacks with few reference models.
- Feldman, "Does Learning Require Memorization? A Short Tale about a Long Tail" (STOC 2020). Why accurate models on long-tailed data must memorize rare examples.
- Carlini, Jagielski, Zhang, Papernot, Terzis & Tramèr, "The Privacy Onion Effect: Memorization is Relative" (NeurIPS 2022). Removing the most exposed records exposes the next layer.

11. Attribute inference and model inversion

A hospital trains a model that predicts whether a patient will develop a lung disease from their age band and whether they smoke, and lets anyone query it. An insurer knows your age and, from a claim, that you have the disease. Can it use the model to find out whether you smoke? It can make a better guess than it could without the model, but that alone is not a privacy failure. If smokers get the disease more often, a published study would allow the same guess about everyone with the disease, whether or not their record was used. The question that matters for *you* is different: **does the model reveal more about your attribute because your record was in its training data?**

Attribute inference is the attack that recovers one sensitive field of a record, such as smoking status, a genotype or an income band, from the record's other fields and access to a model. The one idea of this chapter is to compare every such attack with the **imputation baseline**, the success of a guesser who never sees the model but knows the population's exact joint distribution of attributes, features and labels. A model trained without your record was built from other people's data, so it can teach an attacker only what holds for the population at large, nothing about you beyond that. The chapter proves that, as a result, no attack beats the baseline on records outside the training set. Only on training records can an attack beat it, and that excess is the leak about individuals. The excess is a difference between the success on people who were in the training set and on people who were not, so attribute inference is a membership question in disguise, which ties this chapter to Membership inference.

The running example has two layers. A small smoking table, in which 30% of the population smokes, shows the baseline exactly: the best guess from age and diagnosis is right with probability 0.748, against 0.7 with no observation at all. A toy population with ten numeric features then shows an attack crossing the baseline. Its baseline is 0.7517, and the model inversion attack of Section 11.3, run against a logistic regression trained on 100 of its records, is right on 0.7832 of the training records, above that baseline, but on only 0.7262 of fresh records.

Table 11.1.: The symbols of this chapter.

Symbol	Meaning
$z = (s, x, y)$	a record: sensitive attribute s, public features x, label y
S, X, Y	the same three fields of a random record drawn from the population
$\mathcal{D}$; $p(s, x, y)$	the population distribution; the probability (or density) of a record
D, n; $\mathcal{T}$, θ	a training set of n records; the training algorithm and the model $\theta = \mathcal{T}(D)$
$f_y(x, s)$	the model's confidence in label y at input (x, s)

Symbol	Meaning
b; $\hat{s}$	the hidden bit of the game of Definition 11.1 (1 for a member); the adversary's guess of s
$V(S \mid O)$	the success probability of the best guess of S from an observation O, the one most often right
P_{in}, P_{out}	the distributions of the pair (target record, model), for a member and for a non-member
F; h	the model's answers at every candidate s; a function that coarsens them, merging answers that differed, such as rounding

11.1. The attribute inference game

A claim such as "the attack recovered 78% of smoking statuses" means nothing until you know two more numbers. One is how often the attribute could be guessed without the model, and the other is how often the attack succeeds on people outside the training set. This section sets up the game of Yeom et al. (2018), a two-world game like those of Threat models, which measures the second and compares it with the success on people inside the training set. A challenger flips a coin to decide whether the target comes from the training set or fresh from the population. The coin is not a real-world event, but a way to run both situations under one set of rules, so that the attack's success in one can be compared with its success in the other. The score subtracts the success on non-members from the success on members, and Section 11.2 proves that this difference is what the model reveals about its training records beyond the population.

Definition 11.1 (The attribute inference game)**.** Fix a training algorithm $\mathcal{T}$, a population $\mathcal{D}$, a training-set size n and an adversary $\mathcal{A}$. The game has three steps.

1. The challenger draws a training set $D \sim \mathcal{D}^n$ of n independent records from the population and trains a model $\theta = \mathcal{T}(D)$ on it.
2. It draws a hidden bit $b \in \{0, 1\}$, each value with probability $\frac{1}{2}$. If $b = 1$, it picks the target $z = (s, x, y)$ uniformly from D, so the target is a **member** of the training set. If $b = 0$, it draws $z \sim \mathcal{D}$ afresh, so the target is a **non-member**.
3. The adversary receives the target's public fields and label (x, y), but not s, and access to θ, either its weights or queries to it. It may also know the population $\mathcal{D}$, fully or in part. This auxiliary knowledge is fixed before the game starts, so it is the same in both worlds. It outputs a guess $\hat{s}$ of the sensitive attribute.

The **attribute advantage** of $\mathcal{A}$ is its success rate on members minus its success rate on non-members,

$$\text{Adv}^{\text{attr}}(\mathcal{A}) = \Pr[\hat{s} = s \mid b = 1] - \Pr[\hat{s} = s \mid b = 0], \tag{11.1}$$

where each probability is over everything random in the game.

Any attribute attack can be turned into a membership attack. If the attribute attack is right more often on members, then "the attack was right" is itself evidence of membership. A membership adversary may check this, because the membership game of Membership inference hands it the

whole record, sensitive field included, and asks only whether the record was used. A membership attack is described by two rates. Its true positive rate (TPR) is the probability that it says "member" for a member, and its false positive rate (FPR) is the probability that it says "member" for a non-member. Its advantage is TPR minus FPR.

Membership attacks also have a known ceiling, which the reduction below, Proposition 11.1, passes on to every attribute attack. Hypothesis testing proves that no test, randomized or not, has an advantage above the **total variation distance** TV between the two distributions it must tell apart. That distance is half the sum (an integral for continuous outcomes), over all outcomes, of the absolute differences between their two probabilities.

Proposition 11.1 (Attribute advantage is membership advantage)**.** *Let $\mathcal{A}$ be an adversary in the game of Definition 11.1. Let $\mathcal{B}$ be the membership adversary that receives the whole record $z = (s, x, y)$ and θ, runs $\mathcal{A}$ on (x, y) and θ, and says "member" exactly when $\hat{s} = s$. Then*

$$\mathrm{TPR}_{\mathcal{B}} - \mathrm{FPR}_{\mathcal{B}} = \mathrm{Adv}^{\mathrm{attr}}(\mathcal{A}). \tag{11.2}$$

Write P_{in} and P_{out} for the distributions of the pair (z, θ), the target together with the model, over the draw of the training set, the target and the training randomness, when z is a member and when it is not. Then every adversary satisfies

$$\mathrm{Adv}^{\mathrm{attr}}(\mathcal{A}) \le \mathrm{TV}(P_{\mathrm{in}}, P_{\mathrm{out}}). \tag{11.3}$$

Proof. The plan is to read off the two rates of $\mathcal{B}$, subtract them, and then apply the bound by total variation.

1. *The membership test fires when the guess is right.* $\mathcal{B}$ is given s, so it can check whether $\hat{s} = s$, and it says "member" on exactly that event.
2. *Its two rates.* The TPR is the probability of "member" when $b = 1$, and "member" means that the guess was right, so the TPR is $\Pr[\hat{s} = s \mid b = 1]$. In the same way, the FPR is $\Pr[\hat{s} = s \mid b = 0]$.
3. *Subtract.* TPR minus FPR is then the right-hand side of Equation 11.1, term by term.
4. *Bound by total variation.* $\mathcal{B}$ sees exactly the pair (z, θ), and it runs $\mathcal{A}$ with the same auxiliary knowledge in both worlds, so it is a test between P_{in} and P_{out}, randomized by $\mathcal{A}$'s coins. Its advantage is therefore at most $\mathrm{TV}(P_{\mathrm{in}}, P_{\mathrm{out}})$, and step 3 turns that into Equation 11.3.

□

The subtraction in Equation 11.1 is what makes the advantage a measure of privacy. Success that comes from knowing the population, such as "people with the disease tend to smoke", is the same for members and non-members, as Proposition 11.2 below proves, so it cancels. The cancelled part still matters to the people attacked, so report the success on members and on non-members as well, as a membership attack reports its TPR and FPR. The reduction also says that a model with no membership leak has no attribute leak. If P_{in} and P_{out} are equal, Equation 11.3 forces the advantage of every attribute attack to 0. The converse fails. A model whose predictions do not depend on s at all gives no attribute attack anything to work with, yet it can still reveal who was in its training set, for example through a lower loss on its training records.

The membership test that the reduction builds is a weak one, though. It outputs a single bit, so all it can tell you is whether it fired, and how much a firing should raise your belief in membership

is the likelihood ratio TPR/FPR, how many times more often it fires on members than on non-members. For an attack that is right on most of the population, members or not, that ratio is close to 1. To lower its FPR, a test that sees only this bit can only fire on a random share r of the right guesses. That multiplies both rates by r, so the ratio stays near 1 and the TPR stays near the FPR. Membership inference judges attacks at a low FPR, because a few confident identifications are what harm people, and there the dedicated attacks do much better.

Example 11.1. The toy experiment of this chapter (Section 11.5) has a binary sensitive attribute with prior $\Pr[s = 1] = 0.3$, ten public features and a binary label. A logistic regression that keeps a separate weight vector for each value of s, and predicts a record's label with the vector of its s, is trained on $n = 100$ records, and this is repeated 200 times. Each time, the maximum a posteriori (MAP) inversion attack, which Section 11.3 defines and which queries the model at each candidate value of s, guesses s for the 100 members and for 100 fresh records. It is right on 0.7832 of members and 0.7262 of non-members. The attribute advantage is their difference,

$$\mathrm{Adv}^{\mathrm{attr}} = 0.7832 - 0.7262 \approx 0.0569, \tag{11.4}$$

where the last digit comes from the unrounded rates. The standard error of this average over the 200 models is 0.0045.

Read as a membership test, the same attack has TPR 0.7832 at FPR 0.7262. Its likelihood ratio when it fires is $0.7832/0.7262 = 1.08$, so being right is barely more likely for a member than for a non-member. Its balanced accuracy is the average of its accuracy on members, the TPR, and on non-members, one minus the FPR. That average is $\frac{1}{2}(\mathrm{TPR} + 1 - \mathrm{FPR}) = \frac{1}{2} + \frac{1}{2}(\mathrm{TPR} - \mathrm{FPR})$, one half plus half the advantage, here $\frac{1}{2} + \frac{1}{2} \times 0.0569 = 0.5285$, hardly better than the $\frac{1}{2}$ of a coin flip. The attack is right on 0.7262 of people whose records the model never saw, so most of its 78% success would be there even if the person had never been in the training data.

11.2. The imputation baseline

The game removes population inference by subtracting it. This section measures it directly, because it is the part of an attack's success that any model trained on the same population would give, and so would published statistics about that population. The measure is the success of the best guess of s from (x, y) by someone who knows the population distribution exactly. Statisticians call filling in a missing field from the others **imputation**, so this is the imputation baseline. Jayaraman & Evans (2022) found that many published attribute inference attacks do no better than imputation fitted on similar data, a practical estimate of this exact baseline, and argued that this is the baseline an attack must beat.

The best guess has a simple form. Whatever you answer after an observation is right exactly when the true s is your answer, so the best answer is the value of s with the largest joint probability $p(s, o)$ with that observation, which is also the value with the largest posterior $p(s \mid o)$. Information theory proves that this **MAP guess** beats every deterministic guessing rule. A randomized rule picks a deterministic rule with coins independent of the secret, so its success is an average of theirs and never above the best one.

Definition 11.2 (Vulnerability and the imputation baseline)**.** Let S be a secret and O an observation with joint probability $p(s, o)$. The **vulnerability** of S given O is the success of the

MAP guess $\arg\max_s p(s \mid o)$. Since $p(s, o) = p(o)\, p(s \mid o)$ and the factor $p(o)$ does not depend on s, it can be summed as the largest joint probability per observation or averaged as the largest posterior:

$$V(S \mid O) = \sum_o \max_s p(s, o) = \mathbb{E}_O\Big[\max_s p(s \mid O)\Big]. \tag{11.5}$$

With S, X and Y the sensitive attribute, features and label of a record drawn from $\mathcal{D}$, the **imputation baseline** is $V(S \mid X, Y)$. With no observation, the vulnerability is $V(S) = \max_s p(s)$.

For continuous features the sum becomes an integral, while the second form, an average over records, covers both cases. For a concrete case, take no observation and a prior of 0.3 on smoking. The rule "always say non-smoker" is right 70% of the time and "always say smoker" 30%, so $V(S) = 0.7$.

On a non-member, the model was trained without the target, so beyond what the adversary already knows about the population it should help no more than a coin the adversary flips before guessing. Step 2 of the proof of Theorem 11.1, below, needs that idea made precise: an observation that is independent of the secret cannot raise the best success.

Lemma 11.1 (An independent observation is useless)**.** *If an observation U is independent of the pair (S, O), then $V(S \mid O, U) = V(S \mid O)$.*

Proof.

1. *Independence factorizes the joint.* Independence means that the probabilities multiply, $p(s, o, u) = p(s, o)\, p(u)$.
2. *Pull $p(u)$ out of the maximum.* The factor $p(u)$ is not negative and does not depend on s, so Definition 11.2 with the observation (o, u) gives

$$V(S \mid O, U) = \sum_{o,u} \max_s p(s, o)\, p(u) = \sum_u p(u) \sum_o \max_s p(s, o). \tag{11.6}$$

3. *The weights add up to one.* The inner sum is $V(S \mid O)$, and the $p(u)$ sum to 1, so the whole is $V(S \mid O)$.

□

The lemma turns the model into a useless observation for a fresh target. The best any adversary can do on a non-member is then the best guess from (x, y) alone, which gives the **non-member ceiling**.

Theorem 11.1 (The non-member ceiling)**.** *For every adversary in the game of Definition 11.1, the success on non-members is at most the imputation baseline:*

$$\Pr[\hat{s} = s \mid b = 0] \le V(S \mid X, Y) = \sum_{x,y} \max_s p(s, x, y). \tag{11.7}$$

The bound also holds for an adversary that sees the whole training set D instead of the model, so even publishing the training data reveals nothing about outsiders beyond the population.

Proof.

1. *A non-member is independent of the model.* When $b = 0$, the target z is drawn afresh, separately from D. The model depends only on D and the randomness of training, so θ is independent of $z = (s, x, y)$, and so is D itself. The adversary's auxiliary knowledge and its own coins are fixed before the target is drawn, so they are independent of z too.
2. *So the model is a useless observation.* Lemma 11.1 with $O = (X, Y)$ and $U = \theta$ (or $U = D$) gives $V(S \mid X, Y, \theta) = V(S \mid X, Y)$.
3. *No rule beats the MAP guess.* The adversary's rule is a function of (x, y, θ), possibly randomized. By the optimality of the MAP guess, recalled before Definition 11.2, its success is at most the vulnerability given that observation, which step 2 showed is $V(S \mid X, Y)$.

□

The ceiling is about non-members, while the next result is about attacks that never look at the model, on both kinds of target. A training record, picked at random from a training set drawn from the population, looks exactly like a fresh draw if you ignore the model, so such an attack cannot tell the two worlds apart.

Proposition 11.2 (Model-free attacks have zero advantage)**.** *If the guess $\hat{s} = g(x, y)$ depends on (x, y) only, and not on θ, then* $\Pr[\hat{s} = s \mid b = 1] = \Pr[\hat{s} = s \mid b = 0]$. *The* ***Bayes imputation*** *guess* $\arg\max_s p(s \mid x, y)$*, which is the MAP guess of Definition 11.2 under the true population, has success exactly $V(S \mid X, Y)$ in both worlds, so it attains the ceiling Equation 11.7 on non-members.*

Proof.

1. *A member is a population record.* D consists of n independent draws from $\mathcal{D}$, and the target is one of them, chosen uniformly at random. Whichever one is chosen was drawn from $\mathcal{D}$, so on its own, ignoring the model, the target has distribution $\mathcal{D}$, the same as a fresh draw.
2. *A rule that ignores θ sees one distribution.* Its success is $\Pr[g(X, Y) = S]$ for a record drawn from $\mathcal{D}$ in either world, so the two terms of Equation 11.1 are equal and the advantage is 0.
3. *Imputation reaches the ceiling.* With g the MAP guess under $\mathcal{D}$, this common success is $V(S \mid X, Y)$ by Definition 11.2.

□

On non-members, then, the model only teaches you the population. An adversary who does not know $\mathcal{D}$ can learn it from the model, and so raise its success on non-members towards $V(S \mid X, Y)$, but no further. This does not contradict Lemma 11.1, which takes the population as known: the model teaches what is true of everyone, never anything about this target beyond it. The smoking example in Dwork & Roth (2014, §1) makes the same point. A study that shows smoking causes cancer lets an insurer infer more about any smoker's health, whether or not that smoker took part. The study disclosed a fact about the world, not about its participants, and a privacy definition that forbade this would forbid learning anything from data.

That does not make population inference harmless, since being guessed to be a smoker hurts whether or not the guess used your record. But no protection that limits what one record can change, such as differential privacy in Part 3, can prevent it, because the harm would be there with your record removed. Preventing it is a matter for policy and fairness, not for the defences against the attacks of this part. The ceiling is also a property of the population, not of the model,

and it can be high. If x and y nearly determine s, every model and every statistic about the population "reveals" s, which is why attacks should be reported relative to it.

Example 11.2. Let $s = 1$ mean "smoker", x the age band (young or old) and $y = 1$ "has the disease". The population has $\Pr[s = 1] = 0.3$. Among non-smokers a fraction $\Pr[\text{old} \mid s = 0] = 0.4$ is old, and among smokers a fraction 0.6. The disease probabilities $\Pr[y = 1 \mid x, s]$ are 0.1 (young non-smoker), 0.3 (old non-smoker), 0.4 (young smoker) and 0.7 (old smoker). The probability of a full record is the chance of the smoking status, times the chance of the age band given it, times the chance of the disease outcome given both. Multiplying gives the joint table, Table 11.2. For example, $p(\text{smoker}, \text{old}, \text{disease}) = 0.3 \times 0.6 \times 0.7 = 0.126$, and $p(\text{non-smoker}, \text{young}, \text{no disease}) = 0.7 \times 0.6 \times 0.9 = 0.378$.

Table 11.2.: The smoking table: the joint distribution of smoking status s, age band x and disease y, with the column maxima in bold and the posterior probability of smoking in each column.

$p(s, x, y)$	young, no disease	young, disease	old, no disease	old, disease
non-smoker ($s = 0$)	**0.378**	0.042	**0.196**	0.084
smoker ($s = 1$)	0.072	**0.048**	0.054	**0.126**
$\Pr[\text{smoker} \mid x, y]$	0.1600	0.5333	0.2160	0.6000

The last row is each column's smoker entry divided by the column total. For young people with the disease, it is $0.048/(0.042 + 0.048) = 0.5333$. Now watch the best guess improve as the guesser sees more.

With no observation, the best guess is "non-smoker", right with probability $V(S) = 0.7$.

With age alone, the guesser adds the two disease columns of each age band. The two disease probabilities add to 1, so each sum is the prior times the age probability. Young smokers have probability $0.3 \times 0.4 = 0.12$ and young non-smokers $0.7 \times 0.6 = 0.42$, so a young person is a smoker with probability $0.12/0.54 = 0.2222$. Old smokers have $0.3 \times 0.6 = 0.18$ and old non-smokers $0.7 \times 0.4 = 0.28$, so an old person is a smoker with probability $0.18/0.46 = 0.3913$. Both are below $\frac{1}{2}$, so the guess stays "non-smoker" in both age bands and $V(S \mid X) = 0.7$. Age changes the odds but never the decision, so it adds nothing to the success.

With age and disease, the posterior crosses $\frac{1}{2}$ in the two disease columns (Figure 11.1), so the MAP guess becomes "smoker" there. The success is the sum of the largest entry in each column, the bold ones, so $V(S \mid X, Y) = 0.378 + 0.048 + 0.196 + 0.126 = 0.748$. The entry a guess misses in each column is the one not in bold, and the largest of these is the 0.084 of old non-smokers with the disease.

The **min-entropy leakage** of Information theory turns this gain into bits, a unit that compares leaks of secrets with different priors. It is the base-2 logarithm of how many times more likely the best guess is to succeed with the observation than without. Here seeing age and disease raises the success from 0.7 to 0.748 (age alone added nothing), a leakage of $\log_2(0.748/0.7) = 0.0957$ bits.

Every one of these numbers comes from the population. A model trained on 10,000 patients from it, if it predicts well, lets an insurer reach 0.748 on *anyone* drawn from the population. The ceiling

Equation 11.7 says that nobody outside the training set can be attacked more successfully than that.

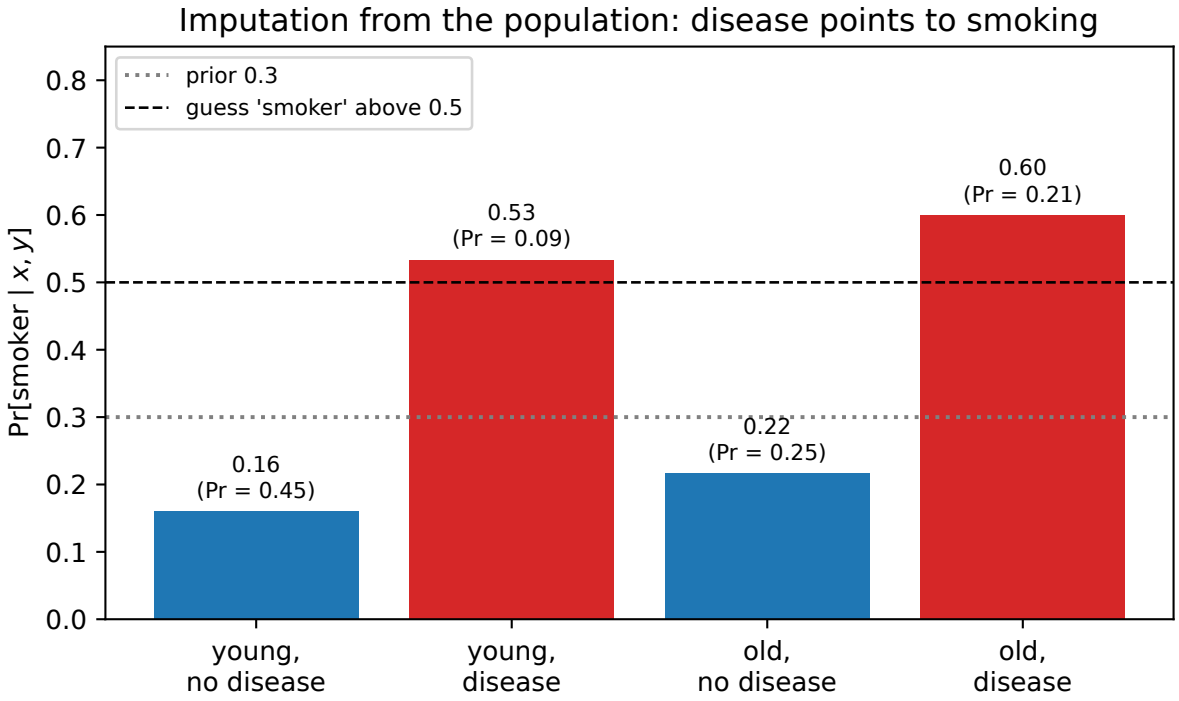


Figure 11.1.: The posterior probability of smoking in each cell of the smoking table, with each cell's probability in brackets. Red bars cross $\frac{1}{2}$, where imputation guesses "smoker". Switching these two cells to "smoker" gains $(0.048-0.042)+(0.126-0.084)=0.048$ over the 0.7 of the blind guess.

11.3. Model inversion

The imputation baseline assumes a guesser who knows the population exactly, and a real adversary does not. In particular, it usually does not know $p(y \mid x, s)$, the link between the label and the sensitive attribute, and Bayes' rule needs that link to compute the posterior of s. But a model that predicts y from x and s is an estimate of exactly that link. Fredrikson et al. (2014) used this on a linear model that predicted the stable dose of the drug warfarin from demographics and two genetic markers, so that an adversary who knew a patient's demographics and dose could infer their genotype. They called the attack **model inversion**, running the model "backwards" from its output to an input. This section states the attack as Bayes' rule and shows when it is no more than imputation.

By Bayes' rule, written out in the proof of Proposition 11.3, the value of s that best explains what you see has the largest product of two factors. The first is how plausible s is for someone with these features, and the second is how likely the observed label is if the attribute were s. The adversary supplies the first factor from what it knows about the population. It gets the second

by querying the model with each candidate value of s, and Fredrikson, Jha & Ristenpart (2015) showed that models releasing **confidences**, the predicted probability of each label, make this much more effective, because a confidence says how likely the label is under each candidate, not just which label wins.

Definition 11.3 (MAP model inversion)**.** Let $p(s \mid x)$ be what the adversary knows about how s relates to the public features, and let $f_y(x, s)$ be the model's confidence in the observed label y when it is fed the candidate s. The **MAP inversion attack** guesses

$$\hat{s}_{\text{inv}}(x, y) = \arg\max_s \; p(s \mid x)\, f_y(x, s). \tag{11.8}$$

Fredrikson et al. (2014) used the marginal $p(s)$ in place of $p(s \mid x)$, which treats s as independent of x. That throws away the evidence x carries about s and loses success even with a perfect model.

Even with $p(s \mid x)$, a perfect model gives the attack nothing beyond imputation. If the model's confidences were exactly the population's probabilities, the second factor would be the true likelihood. This chapter calls a model with that property **calibrated**, a stronger demand than the usual meaning of calibration, that among inputs given confidence 0.7 about 70% have the label. For a calibrated model the attack is the imputation guess.

Proposition 11.3 (For a calibrated model, inversion is imputation)**.** *Suppose the adversary's $p(s \mid x)$ is the population's, and that for every training set the model satisfies $f_y(x, s) = p(y \mid x, s)$ for all s, x and y, so that its answers do not depend on which records it was trained on. Then $\hat{s}_{\text{inv}}$ is the Bayes imputation guess of Proposition 11.2. Its success is $V(S \mid X, Y)$ on members and non-members alike.*

Proof. The plan is to write the posterior of s by Bayes' rule, drop the factor that does not depend on s, substitute the model, and then apply the result on model-free attacks.

1. *Bayes' rule.* The joint probability of s and y given x can be built in two stages, first s from x and then y from both, so $p(s, y \mid x) = p(s \mid x)\, p(y \mid x, s)$. Dividing by $p(y \mid x)$ turns it into the posterior of s:
$$p(s \mid x, y) = \frac{p(s \mid x)\, p(y \mid x, s)}{p(y \mid x)}. \tag{11.9}$$
2. *The denominator does not depend on s.* It is the same number for every candidate, so it cannot change which candidate is largest. The MAP guess is therefore $\arg\max_s p(s \mid x)\, p(y \mid x, s)$.
3. *Plug in the model.* Replacing the unknown $p(y \mid x, s)$ by the model's estimate $f_y(x, s)$ gives Equation 11.8. For a calibrated model the replacement changes nothing, so $\hat{s}_{\text{inv}}$ is the MAP guess, with success $V(S \mid X, Y)$.
4. *Members and non-members.* The guess uses θ only through answers that are the same for every training set, so it is a fixed function of (x, y). It is a model-free attack, so Proposition 11.2 gives equal success in both worlds.

□

Everything the attack does well or badly therefore comes from how the model's $f_y(x, s)$ differs from the population's $p(y \mid x, s)$, and the difference helps the attack on members and hurts it on non-members. On members, the model is too sure of the truth. Training lowers the log-loss $-\ln f_y(x, s)$ on each training record at its *true* s, which pushes $f_y(x, s)$ up towards 1. The other candidate value s' was never paired with this record during training, so $f_y(x, s')$ stays closer to an honest prediction. In the toy model the record never touches the weights used at s', because each value of s has its own weight vector. The ratio $f_y(x, s)/f_y(x, s')$ that decides the attack is therefore tilted towards the truth for members. This is the generalization gap of Membership inference, the difference between a model's loss on its training records and on fresh ones, seen through the sensitive attribute.

On non-members, an overfit model does worse than imputation. Its confidences carry the noise of a small training set rather than the population's exact probabilities, so on some records its guess differs from the imputation guess, and by Theorem 11.1 no departure from imputation can win on average. More data closes both gaps, the excess on members and the shortfall on non-members. As n grows, a correctly specified model, one whose family contains the population's $p(y \mid x, s)$, approaches $p(y \mid x, s)$, and both success rates approach $V(S \mid X, Y)$ (Figure 11.3; Section G.6 in the appendices shows why the fit converges). The population inference on non-members gets better and the individual leak gets smaller.

Example 11.3. Start with a calibrated model on the smoking table, and take an old patient with the disease. From Example 11.2, the adversary knows Pr[smoker | old] = 0.3913, so Pr[non-smoker | old] = $1 - 0.3913 = 0.6087$. It queries the model at both values of s, and a calibrated model returns the population's disease probabilities, $f_1(\text{old}, \text{smoker}) = 0.7$ and $f_1(\text{old}, \text{non-smoker}) = 0.3$. The scores of Equation 11.8 are $0.3913 \times 0.7 = 0.2739$ for "smoker" and $0.6087 \times 0.3 = 0.1826$ for "non-smoker", so it guesses "smoker". Dividing the smoker score by the sum of the two restores the normalization that step 2 of the proof dropped, and gives $0.2739/(0.2739 + 0.1826) = 0.6000$, the posterior of Table 11.2. The attack is imputation.

The toy model shows what a trained model changes. Its population draws s with $\Pr[s = 1] = 0.3$ and ten standard normal features, the first shifted by 0.5 when $s = 1$. The label follows a logistic model with a different, fixed weight vector for each value of s, listed in the script. Guessing from x alone is right 0.7045 of the time, and Bayes imputation, which knows the true logistic weights, is right 0.7517 of the time, both measured on a million fresh records. In the 200 games with $n = 100$, imputation, which never looks at the models, is right on 0.7508 of the members and 0.7555 of the non-members. The difference, -0.0046, is about one standard error of that difference from zero, as Proposition 11.2 predicts, and the 0.7555 on non-members sits above the ceiling of 0.7517 only by the same sampling noise.

The trained models overfit, with a mean log-loss of 0.2965 on members and 0.6690 on non-members. MAP inversion, given the population's $p(s \mid x)$ so that only the model's contribution varies, and the trained models' confidences, is right on 0.7832 of members, $0.7832 - 0.7517 = 0.0315$ *above* the imputation ceiling, and on 0.7262 of non-members, 0.0254 below it (computed before rounding). The member success above the ceiling is something no population-level knowledge could produce, since the ceiling is the best that perfect population knowledge achieves.

Figure 11.2, from a separate, smaller run of 50 models that is enough to show the shape, shows where the excess on members comes from. It plots the attack's evidence for the true value s over the other value s', the log of the ratio of their two scores, $\ln \frac{p(s|x) f_y(x,s)}{p(s'|x) f_y(x,s')}$. The attack is right

when this is positive. For members the distribution has a thinner left tail, because training has removed most of the cases where the model strongly prefers the wrong value.

Figure 11.3 repeats the game for training sets from 25 to 1600 records. At $n = 25$ the attack is right on 0.8420 of members and 0.6810 of non-members, an advantage of $0.8420 - 0.6810 = 0.1610$. At $n = 400$ the advantage is 0.0111, and at $n = 1600$ it is 0.0052.

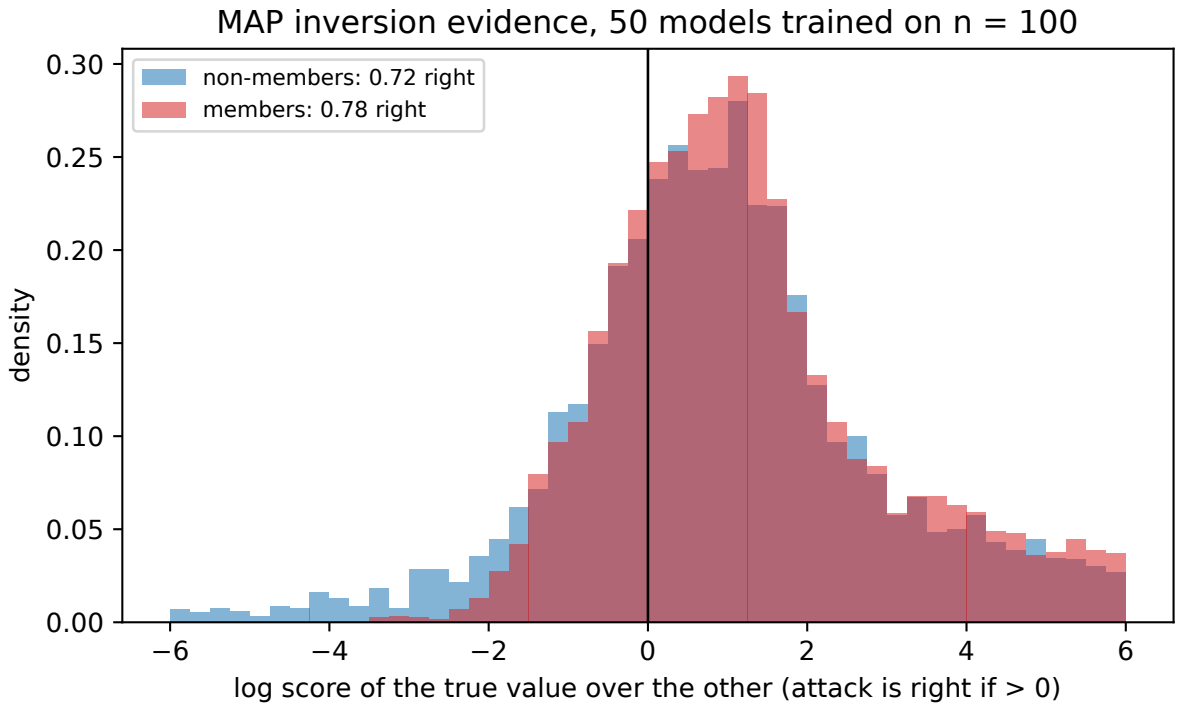


Figure 11.2.: The evidence of MAP inversion for the true attribute, the log of the score of the true value over the other, for members and non-members of 50 models trained on 100 records each. The attack is right when the evidence is positive; members have far fewer large negative values.

The same idea can invert a whole input rather than one field. Fredrikson, Jha & Ristenpart (2015) also attacked a face recognition model in which each class is one person. They followed the gradient of a chosen class's confidence with respect to the image, searching for the input that maximizes it, and obtained a recognizable, blurred image of that person. Here the whole input plays the role of s. Nothing about the person is known beforehand, so an image regularizer that keeps the search among plausible images plays the role of the prior. What the attack recovers is a typical input of the class, which by this chapter's measure is population inference about the class. It is still a privacy problem, because the class is a single person and its population is that person's photos.

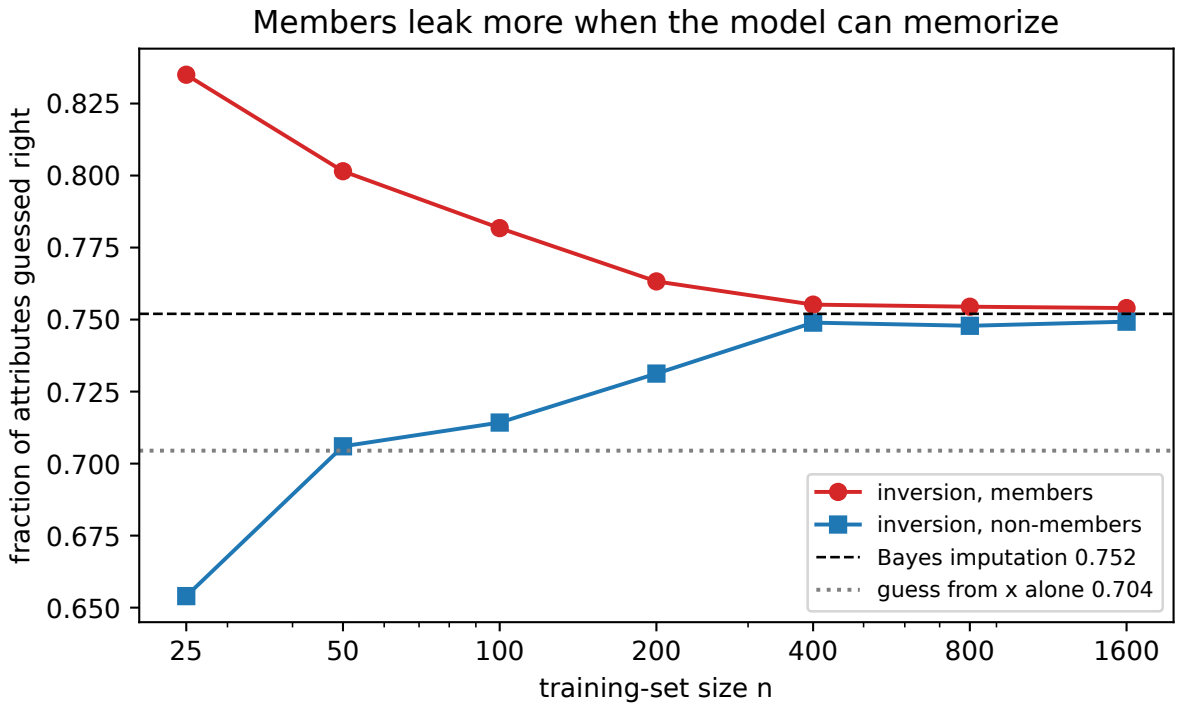


Figure 11.3.: Success of MAP inversion on members and non-members against the training-set size n, 40 models per point. Members start above the Bayes imputation ceiling and non-members below it; both converge to the ceiling as the model approaches the population.

11.4. Confidence masking

If confidences make inversion stronger, why release them? Fredrikson, Jha & Ristenpart (2015) suggested rounding the confidences an interface returns, and many deployed interfaces return only the top label. Such **confidence masking** is cheap and leaves the predicted label unchanged. This section asks whether it closes the leak. It shows that masking can never help the best attacker, but also that it need not shrink the gap between members and non-members.

Masking only coarsens what the adversary sees. The proof of Proposition 11.4 below needs the general form of that fact: whatever you compute from an observation, the best guess from the result is no better than the best guess from the observation itself.

Lemma 11.2 (Processing cannot help the best guesser)**.** *Let h be any function of an observation O, possibly randomized with randomness independent of (S, O). Then $V(S \mid h(O)) \leq V(S \mid O)$.*

Proof.

1. *A rule on $h(O)$ is a rule on O.* Whoever sees o can compute $h(o)$, drawing any randomness itself, and then guess. So a rule g that guesses from $h(o)$ is also a possibly randomized rule on o, namely "apply h, then g".
2. *No rule on O beats the MAP guess.* By the optimality of the MAP guess recalled before Definition 11.2, none of the rules on O succeeds more often than $V(S \mid O)$, and that includes every rule on $h(O)$.

□

For example, rounding a confidence of 0.62 to 0.6 can only merge observations that a guesser could have told apart, and it never separates two that were the same. This is the guessing version of the data-processing inequality of Divergences.

To apply the lemma to masking, write $f(x, s)$, without a label subscript, for the model's whole vector of confidences over the labels at input (x, s), whose entry for label y is $f_y(x, s)$. Let $F = \big(f(x, s)\big)_s$ be the list of these vectors that the adversary gets by querying the model at every candidate value of s.

Proposition 11.4 (Masking cannot help the best attacker)**.** *Let h be any masking function applied to the answers F, such as rounding, keeping only the top label or adding noise. In each world of the game separately, with V computed under that world's joint distribution of the target and the model rather than under the population alone, the best attack on the masked answers does no better than the best attack on the full ones:*

$$V(S \mid X, Y, h(F)) \leq V(S \mid X, Y, F). \tag{11.10}$$

Proof.

1. *A masked observation is a processed one.* Anyone holding the full answers can apply the mask themselves, so $(x, y, h(F))$ is computed from (x, y, F).

2. *Apply the lemma.* Lemma 11.2 with $O = (X, Y, F)$ and the processing $(x, y, F) \mapsto (x, y, h(F))$ gives Equation 11.10. The argument never used which world the target came from, so it holds for $b = 0$ and for $b = 1$ separately.

□

The inequality says nothing about the *difference* between the two worlds, and labels alone still leak, because a label can still be checked at each candidate. With only the top label, the adversary still sees, for each candidate s, whether the predicted label equals the known label y, so the confidence factor of Equation 11.8 becomes 1 on a match and 0 otherwise. On a member, the model usually predicts the training label at the true s, because it fit that record, while at the other value it gives an honest prediction that matches y less often. So a label that matches at only one candidate points to that candidate, and on members it usually points to the truth. A non-member rarely gets this help. Its two predicted labels usually agree, so both match or both miss, the confidence factors are equal, and the prior $p(s \mid x)$ alone decides, which is the guess from x alone. This asymmetry is why masking hurts the population inference more than the individual leak.

Label-only membership inference (Choquette-Choo et al. 2021) exploits the same asymmetry between members and non-members.

Masking lowers the success in both worlds by amounts that depend on h, and nothing forces the two drops to be equal, so the gap need not shrink, and Example 11.4 shows it growing. Do not read much into small changes either way. Equation 11.10 is about the optimal attacker, and MAP inversion with an overfit model is not optimal, so a particular mask can even raise its success slightly. Above all, masking is obfuscation, not a guarantee. It removes one attack's input without bounding what any attack can learn about a member. Part 3 introduces differential privacy, which bounds the difference between members and non-members itself, for every attacker and every output format.

Example 11.4. The same 200 models as in Example 11.3, attacked through three interfaces, give Table 11.3 and Figure 11.4.

Table 11.3.: MAP inversion success through three interfaces, averaged over 200 models trained on 100 records each. Guessing from x alone gives 0.7045 and Bayes imputation 0.7517.

Interface	Members	Non-members	Advantage
Full confidences	0.7832	0.7262	0.0569
Confidences rounded to one decimal	0.7836	0.7253	0.0583
Top label only	0.7741	0.7092	0.0650

Rounding to one decimal changes almost nothing. The attack only compares the scores of the two candidates, and most confidences are far enough from the rounding boundaries that the comparison keeps its sign. With labels only, non-member success drops from 0.7262 to 0.7092, close to the 0.7045 of guessing from x alone. Member success drops less, to 0.7741, still $0.7741 - 0.7517 = 0.0224$ above the imputation ceiling. So the advantage *grows*, from 0.0569 to 0.0650. The interface now leaks less about the population, while members are still guessed well above what the population allows.

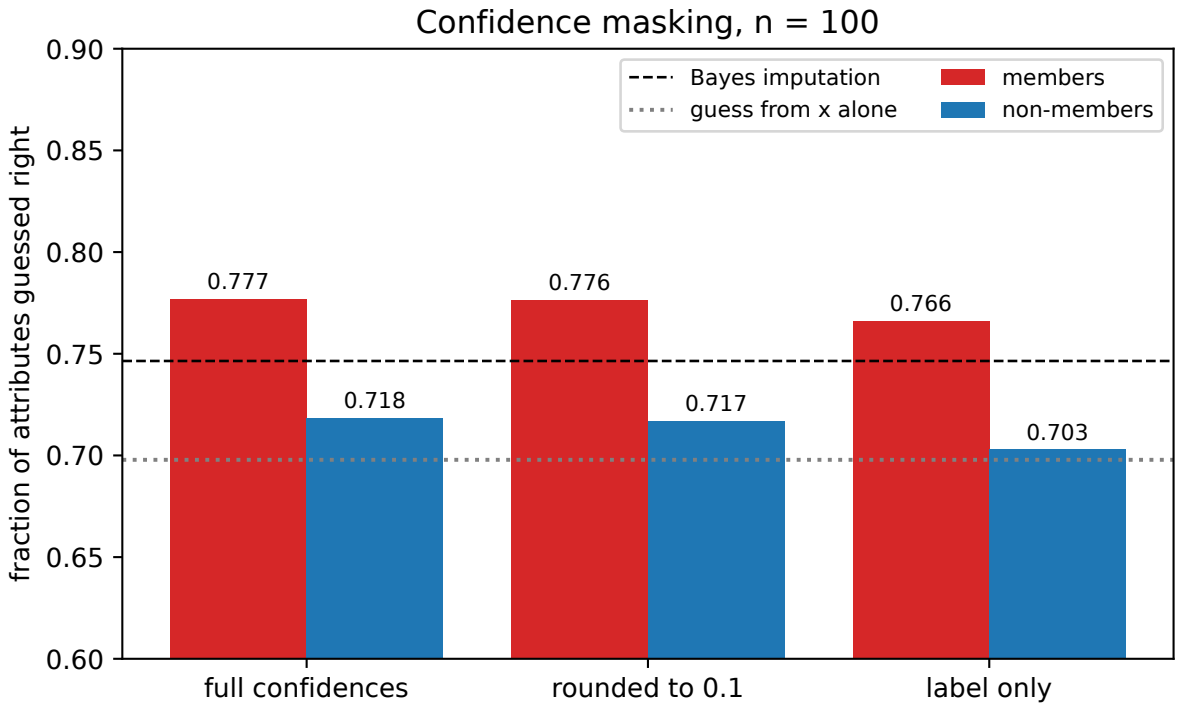


Figure 11.4.: MAP inversion success on members and non-members through three interfaces, for models trained on 100 records. The dashed line is Bayes imputation, the dotted line the guess from the public features alone.

11.5. Code

The script code/attribute_inference.py computes every number in this chapter and draws every figure. It works out the smoking table exactly. For the toy model it samples the population and trains a ridge-regularized logistic regression with one weight vector per value of s by Newton's method. It then plays the game, attacking the n members and n fresh non-members of each model with MAP inversion, Bayes imputation and the guess from x alone. From the repository root:

```
uv run python code/part02-attacks/attribute_inference.py
```

Table 11.4.: Output of the chapter's script (seed 0). Pairs are success on members / non-members.

Experiment	Result
Smoking table	$V(S) = 0.7000$; $V(S \mid X) = 0.7000$; $V(S \mid X, Y) = 0.7480$; leakage 0.0957 bits; posteriors 0.1600, 0.5333, 0.2160, 0.6000
Toy population, 10^6 fresh records	Guess from x alone 0.7045; Bayes imputation 0.7517
Game, $n = 100$, 200 models: mean log-loss	Members 0.2965; non-members 0.6690; gap 0.3725
Guess from x alone	Members 0.7017; non-members 0.7089
Bayes imputation	Members 0.7508; non-members 0.7555; advantage -0.0046
MAP inversion	Members 0.7832; non-members 0.7262; advantage 0.0569 (standard error 0.0045)
MAP inversion against the ceiling	Members $+0.0315$; non-members -0.0254
As a membership test	TPR 0.7832; FPR 0.7262; balanced accuracy 0.5285
Masking	Rounded: 0.7836 / 0.7253, advantage 0.0583. Label only: 0.7741 / 0.7092, advantage 0.0650
Training-set size, 40 models each	$n = 25$: 0.8420 / 0.6810. $n = 400$: 0.7611 / 0.7500. $n = 1600$: 0.7550 / 0.7497

tests/test_attribute_inference.py checks that the output still matches.

11.6. Summary

Table 11.5.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 11.1	Attribute advantage = success on members − success on non-members, Equation 11.1	Report both rates and the imputation baseline; a single success rate says nothing about privacy
Proposition 11.1	Fire when $\hat{s} = s$: TPR − FPR = attribute advantage $\leq \mathrm{TV}(P_{\mathrm{in}}, P_{\mathrm{out}})$	No membership leak means no attribute leak; an attribute attack is a one-bit membership test, weak at low FPR
Definition 11.2	$V(S \mid O) = \sum_o \max_s p(s, o)$, the success of the MAP guess	The baseline every attribute attack is measured against
Lemma 11.1	An observation independent of (S, O) leaves $V(S \mid O)$ unchanged	The non-member ceiling
Theorem 11.1	Success on non-members $\leq V(S \mid X, Y)$	Success up to the baseline is population inference, which any model or statistic of the population gives
Proposition 11.2	Attacks that ignore the model succeed equally on members and non-members	Imputation shows no advantage; use it to check an experiment
Definition 11.3, Proposition 11.3	$\hat{s} = \arg\max_s p(s \mid x)\, f_y(x, s)$; for a calibrated model it is imputation	In the toy, success on members beyond non-members comes from overfitting and shrinks as the training set grows (Figure 11.3)
Lemma 11.2, Proposition 11.4	$V(S \mid X, Y, h(F)) \leq V(S \mid X, Y, F)$, but the gap between members and non-members can grow	Rounding confidences or returning labels only changes the attacks, not the fact that the model fits its training records more closely than fresh ones

11.7. References

- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (IEEE CSF 2018). The attribute inference game with members and non-members, its advantage, the reduction between attribute and membership advantage, and the link to the generalization gap.
- Fredrikson, Lantz, Jha, Lin, Page & Ristenpart, "Privacy in Pharmacogenetics: An End-to-End Case Study of Personalized Warfarin Dosing" (USENIX Security 2014). The first model inversion attack: inferring genetic markers from a dosing model and a patient's demographics and dose.

- Fredrikson, Jha & Ristenpart, "Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures" (CCS 2015). Inversion from confidences for decision trees and for face recognition, and rounding as a countermeasure.
- Wu, Fredrikson, Jha & Naughton, "A Methodology for Formalizing Model-Inversion Attacks" (IEEE CSF 2016). Model inversion as a game, with the distinction between what a model reveals about its training data and about the population.
- Jayaraman & Evans, "Are Attribute Inference Attacks Just Imputation?" (CCS 2022). The imputation baseline made central: many attacks do no better, and what it takes to beat it.
- Choquette-Choo, Tramèr, Carlini & Papernot, "Label-Only Membership Inference Attacks" (ICML 2021). Why returning only labels does not stop inference about members.
- Salem, Cherubin, Evans, Köpf, Paverd, Suri, Tople & Zanella-Béguelin, "SoK: Let the Privacy Games Begin! A Unified Treatment of Data Inference Privacy in Machine Learning" (IEEE S&P 2023). Attribute inference among the other privacy games, with their relationships.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §1. The smoking example and why learning about the population is not a privacy breach of the individual.

12. Property inference

A hospital trains a readmission model on its 1,000 patients and shares it with a research partner. Suppose you have checked that no attack on the model can tell, much better than a coin flip, whether any one patient was in the training set. The partner can still learn whether 45% or 55% of the hospital's patients are over 65, and be right almost every time. That secret is not about a record but a **property** of the training set as a whole, a statistic of the dataset such as the fraction of records that belong to some subgroup. **Property inference** is the attack that recovers such a statistic from a trained model.

This chapter asks why an attacker can learn a property from a model when it cannot learn a record. The one idea is that a model behaves like a noisy average over its n records, which Section 12.3 makes precise for fitted models. Changing one record, whose values are bounded, moves an average by about $1/n$, while the randomness of which records were sampled moves it by about $1/\sqrt{n}$. So one record's signal is about $1/\sqrt{n}$ in units of the noise, and it shrinks as n grows. A property is different, because raising the subgroup's share from t_0 to t_1 changes about $n|t_1 - t_0|$ records at once, which together move the average by about the gap $|t_1 - t_0|$. In units of the noise that is $\sqrt{n}\,|t_1 - t_0|$, which grows with n. So property inference gets easier with more data, while membership inference gets harder.

The running example is the hospital, whose records state each patient's age group, and the secret is whether 45% or 55% of its patients are over 65. An attacker's **advantage** is the probability that it answers "55%" when 55% is true minus the probability that it answers "55%" when 45% is true. At $n = 1{,}000$, no attacker on any model trained on these records has advantage above 0.9985, a ceiling that simply publishing the number of patients over 65 reaches, while on the same published number the best attacker asking whether one patient took part has advantage 0.0126. Logistic regressions and small networks trained on a synthetic population then show the practical attack. Table 12.1 lists the symbols.

Table 12.1.: The symbols of this chapter.

Symbol	Meaning
$\mathcal{D}_t$; t_0, t_1, $\bar{t}$	population with subgroup share t; the two candidate shares; their midpoint
$D \sim \mathcal{D}_t^n$; S	dataset of n records; its number of subgroup records
$\mathcal{T}$; θ; $P_{n,t}$	training algorithm; trained parameters; their distribution
$\theta(t)$; $\Sigma_1(t)$	population fit; its noise covariance per record
γ; κ	separation of two Gaussians in standard deviations; γ per unit of $\sqrt{n}\,\lvert t_1 - t_0\rvert$
Φ	standard normal distribution function

Symbol	Meaning
h; u_j; g	hidden units; parameters of unit j; a symmetry of the units

12.1. The property-inference game

To ask whether an attacker can learn the property, you must fix who draws the data, what the attacker sees and knows, and how its guess is scored. Threat models and privacy games writes attacks as games between a **challenger**, who runs the training pipeline with a hidden secret, and an **adversary**, who tries to guess the secret (Salem et al. 2023). In the property game the secret is one bit, which of two subgroup shares the population that produced the training set has. This section sets up that game and finds its ceiling, against which every later attack is measured.

Records come from a population that mixes a majority group A and a subgroup B. Write $\mathcal{D}_A$ and $\mathcal{D}_B$ for the distributions of a record from each group. A population in which a fraction t of people belong to B is the mixture $\mathcal{D}_t = (1-t)\mathcal{D}_A + t\,\mathcal{D}_B$. To draw a record from it, first pick B with probability t and A otherwise, then draw from that group.

Definition 12.1 (Property-inference game)**.** Fix a dataset size n, two shares $t_0 \neq t_1$, and a training algorithm $\mathcal{T}$ that maps a dataset to parameters and may use random choices of its own, such as its initialization. The **property-inference game** runs in four steps.

1. The challenger draws a hidden bit b, which is 0 or 1 with probability $\frac{1}{2}$ each.
2. It draws a dataset $D \sim \mathcal{D}_{t_b}^n$ of n independent records and trains $\theta = \mathcal{T}(D)$.
3. The adversary receives a **view** of θ: the parameters themselves (white-box), or the model's predictions on inputs of its choosing (black-box). It knows $\mathcal{D}_A$, $\mathcal{D}_B$, n, t_0, t_1 and $\mathcal{T}$.
4. It outputs a guess b' and wins if $b' = b$.

With a fair bit, an attacker who learns nothing scores exactly $\frac{1}{2}$. Knowing $\mathcal{D}_A$, $\mathcal{D}_B$ and $\mathcal{T}$ means, in practice, holding auxiliary data from both groups and being able to train models the way the victim does, as the attack of Section 12.3, which trains its own models on such data, requires. Suri & Evans (2022) call this game **distribution inference**.

An attack's **true positive rate** $\mathrm{TPR} = \Pr[b' = 1 \mid b = 1]$ is how often it says 1 when the bit is 1, and its **false positive rate** $\mathrm{FPR} = \Pr[b' = 1 \mid b = 0]$ is how often it says 1 when the bit is 0. Its **advantage** is $\mathrm{TPR} - \mathrm{FPR}$. Half the games have $b = 1$, where the attacker wins with probability TPR, and half have $b = 0$, where it wins with probability $1 - \mathrm{FPR}$. So its accuracy is

$$\Pr[b' = b] = \tfrac{1}{2}(1 + \mathrm{TPR} - \mathrm{FPR}). \tag{12.1}$$

Write $P_{n,t}$ for the distribution of θ, over both the random data and the training randomness. Two results from Part 1 bound what the attacker can do. The first says that the best advantage at telling two distributions apart is their total variation (TV) distance, which is half the sum over all views of the absolute difference between their two probabilities, with an integral of densities in place of the sum for continuous views. The test that attains it says 1 wherever the second distribution is more likely than the first (Hypothesis testing).

The second result is the **data processing inequality** (Divergences). Passing a view through any random function whose behaviour does not depend on the secret cannot increase the TV distance between the view's two distributions. Equality holds when the output is a **sufficient statistic**, meaning that the likelihood ratio of the view between the two worlds can be computed from the output alone. The best test only thresholds that ratio, so it loses nothing.

The model is computed from the data, so the attacker can never learn more from θ than from D itself. When each record shows its group, the theorem also shows that only the count of subgroup records matters.

Theorem 12.1 (The ceiling on property inference)**.** *In the game of Definition 12.1, every adversary has*

$$\mathrm{TPR} - \mathrm{FPR} \le \mathrm{TV}(P_{n,t_0}, P_{n,t_1}) \le \mathrm{TV}(\mathcal{D}_{t_0}^n, \mathcal{D}_{t_1}^n). \tag{12.2}$$

If in addition $\mathcal{D}_A$ and $\mathcal{D}_B$ have disjoint supports, so that no record could come from both groups and each record shows its group, then the right-hand side equals

$$\mathrm{TV}(\mathrm{Bin}(n, t_0), \mathrm{Bin}(n, t_1)). \tag{12.3}$$

Proof. The plan is to bound the advantage by the TV distance of the models, then by that of the datasets, and then to show that with visible groups the count S of subgroup records carries everything the dataset says about the share.

1. *The game is a test.* The view has distribution P_{n,t_0} when $b = 0$ and P_{n,t_1} when $b = 1$. No test between two distributions has advantage above their TV distance, which gives the left inequality of Equation 12.2.
2. *Training is a channel.* The view is a randomized function of D, and its randomness does not depend on b. By the data processing inequality such processing cannot increase the TV distance, which gives the right inequality.
3. *Write the likelihood ratio of a dataset.* Let $\mathcal{D}_A(x)$ and $\mathcal{D}_B(x)$ also denote the two groups' densities at a record x. In general x has density $(1-t)\mathcal{D}_A(x) + t\,\mathcal{D}_B(x)$ under $\mathcal{D}_t$, but with disjoint supports one of the two terms is zero. So a record x from B has density $t\,\mathcal{D}_B(x)$ under $\mathcal{D}_t$, and a record from A has $(1-t)\mathcal{D}_A(x)$. The records are independent, so the density of D is the product over its records. In the ratio of the density under t_1 to that under t_0, the factors $\mathcal{D}_A(x)$ and $\mathcal{D}_B(x)$ cancel. Each of the S subgroup records leaves a factor t_1/t_0, and each of the $n - S$ others leaves $(1-t_1)/(1-t_0)$, so the likelihood ratio is
$$\Big(\frac{t_1}{t_0}\Big)^S \Big(\frac{1-t_1}{1-t_0}\Big)^{n-S}. \tag{12.4}$$
4. *Conclude that the count is sufficient.* The likelihood ratio depends on D only through S, so S is sufficient, and the data processing inequality from D to S holds with equality. Each record is from the subgroup with probability t, independently, so S has law $\mathrm{Bin}(n, t)$, and the TV distance between the datasets is Equation 12.3.

□

The theorem says that **releasing the count is the worst case**. When the group is visible in each record, no model trained on D can leak more about the share than the count does. A model

that never takes the group as an input, like the logistic regression of Section 12.3, is still bound by this ceiling, since it is computed from the same records.

The ceiling bounds accuracy, which is also the right score here, because the secret belongs to a group, not a person. A membership attacker faces many targets, and exposing a few with high confidence is already a harm, so it is scored by its true positive rate at a low false positive rate, while here each dataset carries one secret.

Example 12.1. If age group is part of each of the hospital's $n = 1{,}000$ records, Equation 12.3 gives the ceiling $\mathrm{TV}(\mathrm{Bin}(1000, 0.45), \mathrm{Bin}(1000, 0.55)) = 0.9985$ for every model trained on the data. It is close to 1 because the count has mean 450 in one world and 550 in the other, with standard deviation $\sqrt{1000 \times 0.45 \times 0.55} \approx 15.7$. The two means are more than six standard deviations apart, so the two distributions barely overlap.

At $n = 100$ the means are 45 and 55 and the standard deviation is about 5, only two standard deviations apart, and Proposition 12.2 below turns that into a ceiling of 0.6827. Over 200,000 simulated games, the best test on the count has advantage 0.6811, within sampling error.

12.2. Why a property leaks when no record does

This section proves the opening's claim, that more data hides each record better and the property worse, for the simplest release of all, the count S of subgroup records. Two attackers look at the same count, one asking whether a subgroup target is in D and the other whether D was drawn with share t_0 or t_1, and both best advantages can be computed exactly.

The membership attacker compares the count when a subgroup target is in D with the count when a fresh record takes its place. Its advantage, worked out in Proposition 12.1 below, comes down to how far the distribution of a count moves when the count is shifted by one, and that depends on the shape of the distribution. A probability mass function is **unimodal** if it rises to a most likely value, the mode, and then falls.

Lemma 12.1 (Shifting a unimodal count by one)**.** *Let X be an integer-valued random variable with finite support, whose probability mass function $p_k = \Pr[X = k]$ is unimodal. Then*

$$\mathrm{TV}(\mathrm{Law}(X + 1), \mathrm{Law}(X)) = \max_k p_k, \tag{12.5}$$

where $\mathrm{Law}(\cdot)$ *means "the distribution of".*

Proof.

1. *Write the TV distance as steps.* The shifted variable takes the value k with probability $\Pr[X + 1 = k] = p_{k-1}$. So the TV distance is half the total size of the steps between neighbouring probabilities,

$$\mathrm{TV}(\mathrm{Law}(X + 1), \mathrm{Law}(X)) = \frac{1}{2} \sum_k |p_k - p_{k-1}|. \tag{12.6}$$

2. *Add the climb and the descent.* Let k^* be the mode. The support is finite, so the probabilities start and end at 0. Up to k^* every step goes up, so the steps add to the climb from 0 to the top, p_{k^*}. After k^* every step goes down, so they add to the descent back to 0, also p_{k^*}.
3. *Halve.* The total is $2p_{k^*}$, and half of it is $p_{k^*} = \max_k p_k$. Table 12.2 walks through $X \sim \mathrm{Bin}(4, \frac{1}{2})$.

□

Table 12.2.: Shifting $X \sim \mathrm{Bin}(4, \frac{1}{2})$ by one. The positive differences add up to 6/16, the height of the mode, and so do the negative ones. Half the total absolute difference is $\frac{1}{2} \times \frac{12}{16} = \frac{6}{16} = 0.375 = \max_k \Pr[X = k]$.

k	0	1	2	3	4	5
$16 \Pr[X = k]$	1	4	6	4	1	0
$16 \Pr[X + 1 = k]$	0	1	4	6	4	1
$16\times$ difference	1	3	2	-2	-3	-1

Unimodality matters, because a distribution that rises twice climbs and descends twice, so its steps add up to more than twice its peak. The next proposition shows that the membership advantage is the height of the binomial's most likely value times $1 - t$, the chance that a fresh record is not from the subgroup. That height shrinks like $1/\sqrt{n}$, because the count's standard deviation grows like $\sqrt{n}$ and its probability is spread over that many likely values.

Proposition 12.1 (Membership from a released count)**.** *Let the dataset hold $n - 1$ records from $\mathcal{D}_t$ plus one more, either a target from the subgroup or a fresh record from $\mathcal{D}_t$, and let the release be the count S of subgroup records. Write $X \sim \mathrm{Bin}(n - 1, t)$ for the count among the $n - 1$ other records. The best advantage at telling the two cases apart is*

$$\mathrm{Adv}_{\mathrm{mem}}(n) = (1 - t) \max_k \Pr[X = k] \approx \frac{1 - t}{\sqrt{2\pi(n - 1)t(1 - t)}}. \tag{12.7}$$

Proof. The plan is to write the two views as distributions of the count, to notice that one of them is a mixture containing the other, and to reduce the TV distance to the effect of shifting a binomial by one.

1. *Write the two views.* With the target in D, the count is $S = X + 1$. With a fresh record, the count is $X + 1$ when the fresh record is from the subgroup, which happens with probability t, and X otherwise.
2. *See the second view as a mixture.* So the law of the second view is $t\,\mathrm{Law}(X + 1) + (1 - t)\,\mathrm{Law}(X)$.
3. *Mixing scales the TV distance.* Write $P = \mathrm{Law}(X + 1)$ and $Q = \mathrm{Law}(X)$, with mass functions p and q. In each term of the TV distance between P and $tP + (1 - t)Q$, the difference is $p(x) - tp(x) - (1 - t)q(x) = (1 - t)(p(x) - q(x))$, so the factor $1 - t$ comes out:

$$\mathrm{TV}(P, tP + (1 - t)Q) = \frac{1}{2} \sum_x (1 - t)|p(x) - q(x)| = (1 - t)\,\mathrm{TV}(P, Q). \tag{12.8}$$

The best advantage is this TV distance, so it is $(1 - t)\,\mathrm{TV}(\mathrm{Law}(X + 1), \mathrm{Law}(X))$.

4. *Shift the binomial.* A binomial mass function is unimodal, so Lemma 12.1 turns the TV distance into $\max_k \Pr[X = k]$, which gives the exact form of Equation 12.7.
5. *Estimate the height of the mode.* By the normal approximation of Probability distributions, X is close to a normal with the same mean and variance $(n-1)t(1-t)$. A normal density with variance s^2 has height $1/\sqrt{2\pi s^2}$ at its peak. The probability of the most likely integer is close to that height, because the integers are spaced one apart and the density is nearly flat over a unit interval at the top. That gives the approximate form. Section E.2 proves both facts.

□

The property attacker sees the same count, but its two worlds centre the fraction S/n on t_0 and on t_1, with spread about $\sqrt{t(1-t)/n}$ at share t. Its advantage therefore depends on how many spreads apart the two centres are, and that number grows like $\sqrt{n}$. The best test cuts halfway between the centres, which is where the factor 2 below comes from.

Proposition 12.2 (The property from a released count)**.** *Let the release be the count S of a dataset drawn from $\mathcal{D}^n_{t_0}$ or $\mathcal{D}^n_{t_1}$, with the groups visible, and let $\bar{t} = \frac{t_0+t_1}{2}$ be the midpoint share. With Φ the standard normal distribution function, the best advantage is*

$$\mathrm{Adv}_{\mathrm{prop}}(n) = \mathrm{TV}(\mathrm{Bin}(n, t_0), \mathrm{Bin}(n, t_1)) \approx 2\Phi\Big(\frac{\sqrt{n}\,|t_1 - t_0|}{2\sqrt{\bar{t}(1-\bar{t})}}\Big) - 1. \tag{12.9}$$

Proof.

1. *Start from the exact answer.* The count is sufficient (Theorem 12.1), so the best advantage is the TV distance between the two binomials.
2. *Approximate by two normals.* By the normal approximation, S/n is close to $\mathcal{N}(t, t(1-t)/n)$. The two variances are close when the gap is small, so replace both by the one at the midpoint, $\bar{t}(1-\bar{t})/n$. Measured in units of that standard deviation, the two means are a distance

$$\gamma = \frac{\sqrt{n}\,|t_1 - t_0|}{\sqrt{\bar{t}(1-\bar{t})}} \tag{12.10}$$

   apart, so the problem is two unit-variance normals with means γ apart.
3. *Use the Gaussian pair.* For two unit-variance normals with means γ apart, the densities cross halfway between the means, and the best advantage is $2\Phi(\gamma/2) - 1$ (Hypothesis testing). Substituting γ gives Equation 12.9.

□

The two results say that **more data protects records and exposes properties**. The membership advantage falls like $1/\sqrt{n}$, a line of slope $-\frac{1}{2}$ on the log axes of Figure 12.1 (right). The property advantage climbs to 1 once $\sqrt{n}\,|t_1 - t_0|$ is a few times $\sqrt{\bar{t}(1-\bar{t})}$, sooner for a larger gap. Neither argument is special to counts, because models fitted by minimizing an average loss behave like averages of their records, as Proposition 12.3 shows.

The reason is that **a property is a group of records**. Moving from t_0 to t_1 changes about $n|t_1 - t_0|$ records, which for the hospital is $1{,}000 \times 0.1 = 100$ of its 1,000 patients. A protection

measured one record at a time says little about a change to 100 records at once, and Part 3 makes this precise with group privacy, which extends a guarantee for one record to a group at a cost that grows with the group's size. To hide the property, a released count must carry noise of about $n|t_1 - t_0|$, the distance between its values in the two worlds, and that much noise also hides the count the release was for.

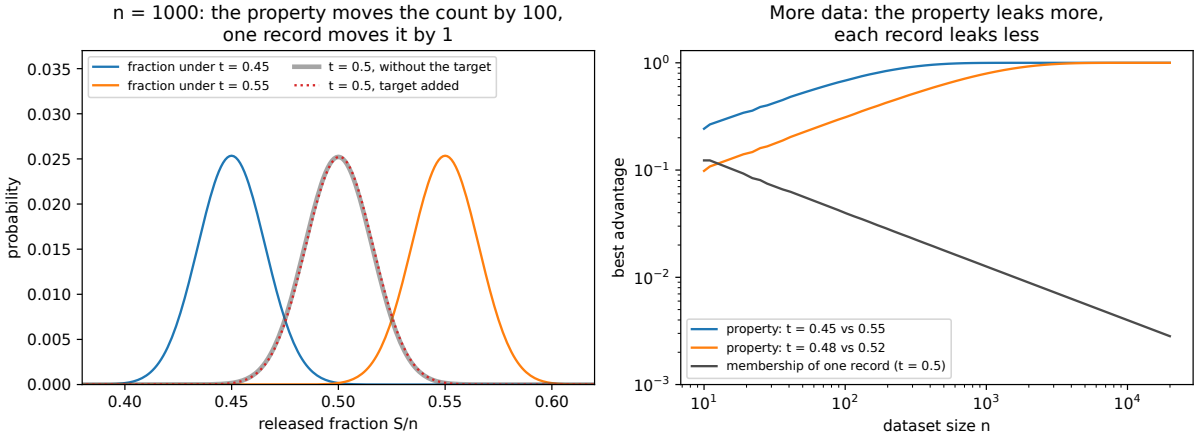


Figure 12.1.: Left: the released fraction S/n at $n = 1{,}000$ under $t = 0.45$ and $t = 0.55$, and under $t = 0.5$ with and without one target from the subgroup; the last two curves are indistinguishable. Right: the best property advantage for two gaps, and the best membership advantage of one record, against n.

Example 12.2. Table 12.3 gives both advantages, and you can check the first row by hand. For the property, $\bar{t} = 0.5$, so $\sqrt{\bar{t}(1-\bar{t})} = 0.5$ and the argument of Φ in Equation 12.9 is $\sqrt{100}\times 0.1/(2\times 0.5) = 1$. Then $2\Phi(1)-1 = 2(0.8413)-1 = 0.6827$. For membership, Equation 12.7 gives $0.5/\sqrt{2\pi \times 99 \times 0.25} = 0.5/12.47 = 0.0401$. Both approximations are within 1% of the exact values.

At $n = 10{,}000$, an attacker who wants to know whether one patient is in the data has advantage 0.0040, so with a fair bit it is right with probability $\frac{1}{2} + \frac{1}{2}(0.0040) = 0.502$. An attacker who wants the share of patients over 65 has advantage 1.0000 to four decimals, though the data, the release and its knowledge are the same.

Table 12.3.: Best advantages from a released count of subgroup records. Property: $t_0 = 0.45$ against $t_1 = 0.55$. Membership: one subgroup record, population share $t = 0.5$.

n	Property, exact	Property, Equation 12.9	Membership, exact	Membership, Equation 12.7
100	0.6827	0.6827	0.0398	0.0401
1,000	0.9985	0.9984	0.0126	0.0126
10,000	1.0000	1.0000	0.0040	0.0040

12.3. Shadow models and meta-classifiers

For a real model nobody can write down $P_{n,t}$, the distribution of the trained parameters, so the best test of Theorem 12.1 is out of reach. Ateniese et al. (2015) gave the recipe every later attack uses: if you cannot compute the two distributions, sample from them. This section states that recipe, and then answers the two questions it leaves open, which classifier to train and how accurate it can be.

Definition 12.2 (Shadow-model attack)**.** The attacker draws many datasets of n records from $\mathcal{D}_{t_0}$ and from $\mathcal{D}_{t_1}$, using auxiliary data, and trains a **shadow model** on each with the victim's algorithm $\mathcal{T}$. It labels each shadow model's parameters with the bit that produced them, trains a classifier on these labelled parameter vectors, the **meta-classifier**, and applies it to the victim's model.

This is the shadow-model idea of Membership inference, with whole models as the data points. Choosing the best meta-classifier needs the distribution of the parameters, which the next proposition gives.

Let θ minimize the average loss $\frac{1}{n}\sum_i \ell(\theta; x_i)$ over the training records, and let $\theta(t)$ minimize the expected loss under $\mathcal{D}_t$, the model you would get from infinitely many records. How noisy θ is around $\theta(t)$ depends on two things. A sharply curved loss pins the minimizer down, and records whose gradients at $\theta(t)$ point in different directions pull it about.

Proposition 12.3 (Fitted parameters are approximately Gaussian)**.** *Let the average curvature of the loss be $H = \mathbb{E}_{\mathcal{D}_t} \nabla^2 \ell(\theta(t); x)$ and the second moment of the per-record gradients $G = \mathbb{E}_{\mathcal{D}_t}[\nabla \ell(\theta(t); x) \nabla \ell(\theta(t); x)^\top]$, with H invertible, and define the **sandwich covariance** $\Sigma_1(t) = H^{-1} G H^{-1}$. Under the regularity conditions of van der Vaart (1998, ch. 5), roughly that the loss is smooth, $\theta(t)$ is its unique minimizer and these moments exist, for large n*

$$\theta \approx \mathcal{N}(\theta(t), \Sigma_1(t)/n), \tag{12.11}$$

meaning that $\sqrt{n}\,(\theta - \theta(t))$ converges in distribution to $\mathcal{N}(0, \Sigma_1(t))$.

Proof. This is the asymptotic normality of **M-estimators**, estimators defined as the minimizer of an average loss. The plan is to write the fit as a linear function of an average of per-record gradients, which is approximately Gaussian. Section G.6 proves each approximation for logistic regression.

1. *The fit solves an average equation.* At a minimizer the gradient is zero, so $\frac{1}{n}\sum_i \nabla \ell(\theta; x_i) = 0$.
2. *Linearize around $\theta(t)$.* For large n, θ is close to $\theta(t)$, so a first-order Taylor expansion of the gradient is accurate. The sample average of the Hessians, taken at points between θ and $\theta(t)$, is close to its expectation H by the law of large numbers, so $0 \approx \frac{1}{n}\sum_i \nabla \ell(\theta(t); x_i) + H(\theta - \theta(t))$. Solving for θ,

$$\theta - \theta(t) \approx -H^{-1} \frac{1}{n} \sum_i \nabla \ell(\theta(t); x_i). \tag{12.12}$$

   So the fit is an average, as in Section 12.2, in which each record pulls by its gradient scaled by the inverse curvature.

3. *The average gradient has mean zero.* $\theta(t)$ minimizes the expected loss, so the expected gradient there is zero. Each gradient therefore has mean zero, so its second moment G is its covariance, and an average of n independent copies has covariance G/n.
4. *Apply the central limit theorem.* The average gradient is approximately $\mathcal{N}(0, G/n)$. Multiplying a Gaussian vector by the symmetric matrix $-H^{-1}$ multiplies its covariance by H^{-1} on both sides, which gives $\mathcal{N}(0, H^{-1}GH^{-1}/n)$.

□

Under the two hypotheses, then, the parameters are two Gaussians with nearly equal covariances when the gap is small, and the next lemma gives the best test between such Gaussians. In d dimensions the noise can be larger in some directions than in others, so a shift along a noisy direction counts for less than the same shift along a quiet one. The **Mahalanobis distance** measures a shift in exactly this way, in standard deviations of the noise along the direction of the shift.

Lemma 12.2 (Two Gaussians with a shared covariance)**.** *Let $P = \mathcal{N}(m_0, \Sigma)$ and $Q = \mathcal{N}(m_1, \Sigma)$ on $\mathbb{R}^d$ with Σ invertible, with densities p and q, and let*

$$\gamma = \sqrt{(m_1 - m_0)^\top \Sigma^{-1} (m_1 - m_0)} \tag{12.13}$$

be their Mahalanobis distance. Then the log-likelihood ratio is linear in θ,

$$\ln \frac{q(\theta)}{p(\theta)} = \omega^\top \Big(\theta - \frac{m_0 + m_1}{2}\Big), \qquad \omega = \Sigma^{-1}(m_1 - m_0), \tag{12.14}$$

and the best test with a fair bit has accuracy

$$\Phi(\gamma/2). \tag{12.15}$$

Proof. The plan is to write out the two log-densities, let the quadratic parts cancel because the covariances are equal, and find the distribution of what is left.

1. *Expand the two quadratics.* A Gaussian log-density is $-\frac{1}{2}(\theta - m)^\top \Sigma^{-1}(\theta - m)$ plus a constant that depends only on Σ, so the constants cancel in $\ln q - \ln p$. Multiplying out, the terms $\theta^\top \Sigma^{-1}\theta$ appear with opposite signs and cancel too, which leaves $(m_1 - m_0)^\top \Sigma^{-1}\theta - \frac{1}{2}(m_1^\top \Sigma^{-1} m_1 - m_0^\top \Sigma^{-1} m_0)$.
2. *Factor the constant.* Because Σ^{-1} is symmetric, the cross terms $m_1^\top \Sigma^{-1} m_0$ and $m_0^\top \Sigma^{-1} m_1$ are equal, so the bracketed constant factors as $(m_1 - m_0)^\top \Sigma^{-1}(m_1 + m_0)$. With its factor $-\frac{1}{2}$ it becomes $-(m_1 - m_0)^\top \Sigma^{-1} \frac{m_0 + m_1}{2}$. Pulling out $\omega^\top = (m_1 - m_0)^\top \Sigma^{-1}$ gives the linear form.
3. *Find its distribution.* Call the linear form Λ. As a linear function of a Gaussian it is Gaussian. Under Q the mean of $\theta - \frac{m_0 + m_1}{2}$ is $m_1 - \frac{m_0 + m_1}{2} = \frac{m_1 - m_0}{2}$, so the mean of Λ is $\omega^\top \frac{m_1 - m_0}{2} = \frac{\gamma^2}{2}$, by the definition of γ, and under P the same computation gives $-\frac{\gamma^2}{2}$. Under both, its variance is $\omega^\top \Sigma \omega = (m_1 - m_0)^\top \Sigma^{-1}(m_1 - m_0) = \gamma^2$.
4. *Compute the accuracy.* The best test says 1 where $q > p$, that is, where $\Lambda > 0$. Under Q the threshold 0 lies $\frac{\gamma^2/2}{\gamma} = \frac{\gamma}{2}$ standard deviations below the mean, so the test is right with probability $\Phi(\gamma/2)$. Under P it is right when $\Lambda < 0$, which by symmetry has the same probability. Averaging the two worlds gives Equation 12.15.

□

The lemma and Proposition 12.3 together give the best attack on a fitted model, which is linear.

Theorem 12.2 (The best attack on a fitted model)**.** *Suppose the parameters satisfy Proposition 12.3 under both hypotheses, and that the gap is small enough to replace $\Sigma_1(t_0)$ and $\Sigma_1(t_1)$ by their average $\bar{\Sigma}_1$. Then the best attack is a linear test in θ, and its accuracy, with γ the Mahalanobis distance of Lemma 12.2, is*

$$\Phi(\gamma/2), \qquad \gamma = \sqrt{n\,(\theta(t_1)-\theta(t_0))^\top \bar{\Sigma}_1^{-1}(\theta(t_1)-\theta(t_0))}. \tag{12.16}$$

For a small gap, with $\theta'(\bar{t}) = \frac{\mathrm{d}\theta}{\mathrm{d}t}$ the slope of the population fit at the midpoint,

$$\gamma \approx \kappa\sqrt{n}\,|t_1 - t_0|, \qquad \kappa = \sqrt{\theta'(\bar{t})^\top \Sigma_1(\bar{t})^{-1}\theta'(\bar{t})}. \tag{12.17}$$

Proof.

1. *Two Gaussians.* Under the two hypotheses θ is approximately $\mathcal{N}(\theta(t_b), \bar{\Sigma}_1/n)$, for $b = 0, 1$.
2. *Apply the lemma.* Lemma 12.2 with $m_b = \theta(t_b)$ and $\Sigma = \bar{\Sigma}_1/n$ says the best test is linear, with accuracy $\Phi(\gamma/2)$. Since $\Sigma^{-1} = n\,\bar{\Sigma}_1^{-1}$, the factor n comes out of the Mahalanobis distance, which gives Equation 12.16.
3. *Linearize in the share.* For a small gap the population fit changes almost linearly in t, so $\theta(t_1)-\theta(t_0) \approx \theta'(\bar{t})(t_1-t_0)$, and the covariance hardly changes, so $\bar{\Sigma}_1 \approx \Sigma_1(\bar{t})$. Substituting both into Equation 12.16 gives $\gamma^2 \approx n(t_1-t_0)^2\,\theta'(\bar{t})^\top \Sigma_1(\bar{t})^{-1}\theta'(\bar{t})$, whose square root is Equation 12.17.

□

The count of Section 12.2 fits the same mould, which checks the theorem against an exact answer.

Corollary 12.1 (The released fraction)**.** *For the released fraction $\theta = S/n$, the constant is $\kappa = 1/\sqrt{\bar{t}(1-\bar{t})}$, which is 2 at $\bar{t} = \frac{1}{2}$, and the advantage $2\Phi(\gamma/2) - 1$ is Equation 12.9.*

Proof.

1. *The fraction is a fitted model.* Take the loss $\ell(\theta; x) = \frac{1}{2}(\theta - a)^2$, where $a \in \{0, 1\}$ marks the subgroup. The minimizer of its average is the mean of the marks, $\theta = S/n$, and its population version is $\theta(t) = t$, so $\theta' = 1$.
2. *Compute the sandwich.* The second derivative of the loss is $H = 1$. The gradient $\theta - a$, taken at $\theta(t) = t$, is $t - a$ and has variance $G = t(1-t)$, the variance of a Bern(t) variable. So $\Sigma_1 = t(1-t)$, and Equation 12.17 gives $\kappa = 1/\sqrt{\bar{t}(1-\bar{t})}$.
3. *Turn accuracy into advantage.* Advantage is twice accuracy minus 1, so $\Phi(\gamma/2)$ becomes $2\Phi(\gamma/2) - 1$, which with this κ is Equation 12.9.

□

Two numbers set the leak. The slope θ' says how much the property moves what the model learns, and Σ_1 says how noisy that is. If the property does not affect what the model fits, then $\theta' = 0$ and, to first order, nothing leaks however large n is. The constant also compares any model with the count. By Corollary 12.1 the count on n' records has $\gamma = \sqrt{n'}\,|t_1 - t_0|/\sqrt{\bar{t}(1-\bar{t})}$. Setting this equal to the model's $\kappa\sqrt{n}\,|t_1 - t_0|$ and solving for n' shows that the model leaks as much as the count on $n' = n(\kappa\sqrt{\bar{t}(1-\bar{t})})^2$ records, which is how Suri & Evans (2022) measure leakage.

The Gaussian picture fails where its approximations do. With small n the parameters are skewed, so the linear meta-classifier is no longer optimal. A meta-classifier trained on few shadow models also estimates the direction ω with error, most of all when the effect is small and the shadow models' noise swamps the shift.

Example 12.3. Each record has two features $x \sim \mathcal{N}(0, I_2)$ and a label. The majority labels with $\Pr[y = 1 \mid x] = \sigma(2x_1 + x_2)$ and the subgroup with $\sigma(2x_1 - x_2)$, where $\sigma(z) = 1/(1 + e^{-z})$ is the logistic function. Think of a symptom x_2 that raises risk in most patients and lowers it in a minority. **The group is never a feature**, so the model sees (x_1, x_2, y) only. A logistic regression with weights (θ_1, θ_2) and an intercept is trained on n records by Newton's method, with a tiny ridge penalty 10^{-3}. The chapter's script (Section 12.5) computes $\theta(t)$, shown in Table 12.4, and $\Sigma_1(t)$ to numerical precision, by Gauss–Hermite quadrature over x (a numerical rule for integrals against a Gaussian).

The property moves θ_2. At $t = \frac{1}{2}$ the two rules cancel in θ_2, because half the population pushes it up and half down. As t grows, the subgroup's rule wins and θ_2 goes negative. The script computes the slope $\theta'(\frac{1}{2}) = (0, -1.704, 0)$ for $(\theta_1, \theta_2, \text{intercept})$. Mixing the rules also shrinks θ_1 from 2 to 1.704, but symmetrically in t, so θ_1 is flat at $\frac{1}{2}$ and carries no information there. The intercept is 0 for every t, by the label-flipping symmetry below, so it carries none either.

The noise $\Sigma_1(\frac{1}{2})$ has diagonal $(12.59, 6.94, 6.04)$ and zero off-diagonal entries, by symmetry. A change of the data that leaves the population unchanged and flips the sign of one parameter makes that parameter's covariance with the others equal its own negative, hence zero. Flipping the sign of x_2 swaps the two groups' rules, leaves the population at $t = \frac{1}{2}$ unchanged and flips only θ_2, and flipping the sign of x and swapping the labels flips only the intercept. Two thousand models trained on $n = 1{,}000$ records at $t = \frac{1}{2}$ have variances, times n, of $(11.92, 6.80, 6.08)$, and their mean θ_1 is 1.693, so the Gaussian picture is right to within about 5%. Most of the difference in the variance of θ_1 comes from the ridge penalty, which the population fit omits. The penalty is added to the average loss, so it does not fade as n grows, and including it lowers the predicted value from 12.59 to 11.83 (Section G.6).

Only θ_2 has a nonzero slope and Σ_1 is diagonal, so Equation 12.17 reduces to the slope of θ_2 over its standard deviation, $\kappa = 1.704/\sqrt{6.94} = 0.647$, against 2 for the count (Corollary 12.1). With $\sqrt{\bar{t}(1-\bar{t})} = \frac{1}{2}$, the formula for n' above says that the model leaks the property as well as $(0.647/2)^2 = 0.105$ of its records would, about one record in ten.

Table 12.4.: The population fit $\theta(t)$ of the logistic regression, against the subgroup share t. The intercept is 0 for every t.

t	0	0.25	0.5	0.75	1
$\theta_1(t)$	2.000	1.767	1.704	1.767	2.000
$\theta_2(t)$	1.000	0.442	0.000	−0.442	−1.000

In Figure 12.2, models trained at $t_0 = 0.4$ and $t_1 = 0.6$, a wider gap for visibility, overlap at $n = 100$ and separate along θ_2 at $n = 1{,}000$.

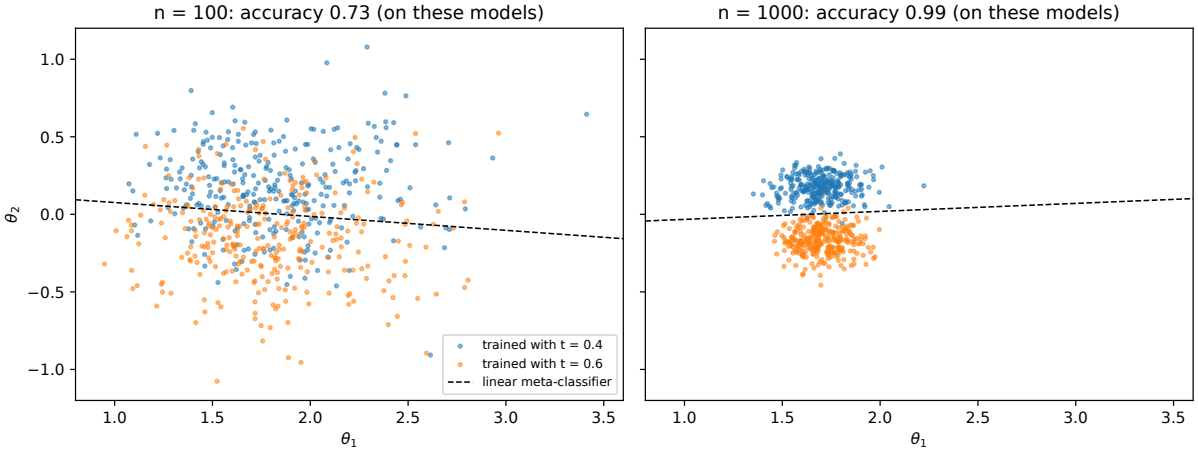


Figure 12.2.: The parameters (θ_1, θ_2) of 300 logistic regressions trained with subgroup share $t = 0.4$ and 300 with $t = 0.6$, at $n = 100$ (left) and $n = 1{,}000$ (right), with the linear meta-classifier fitted to them. The clouds shrink like $1/\sqrt{n}$; the gap between their centres stays fixed.

The script trains 150 shadow models per hypothesis for each n and gap, with t_0 and t_1 centred on $\frac{1}{2}$. It fits the linear test of Lemma 12.2, with means and covariance estimated from the shadow models' three parameters, and measures its accuracy on 150 fresh target models per hypothesis (Table 12.5). Each accuracy averages 300 outcomes, so its standard error is at most $\sqrt{0.25/300} = 0.029$. Most cells agree with $\Phi(\gamma/2)$ to within two standard errors. The exceptions are two small effects, gap 0.1 at $n = 50$ (0.520 against 0.590) and gap 0.05 at $n = 200$ (0.503 against 0.590), where the meta-classifier finds the direction of ω badly.

Table 12.5.: Accuracy of the meta-classifier against logistic regressions, by dataset size n and gap $|t_1 - t_0|$, with the prediction $\Phi(\gamma/2)$ of Equation 12.16 in parentheses.

n	gap 0.05	gap 0.1	gap 0.2	gap 0.4
50	0.523 (0.546)	0.520 (0.590)	0.653 (0.676)	0.830 (0.818)
100	0.523 (0.564)	0.670 (0.627)	0.733 (0.741)	0.863 (0.901)
200	0.503 (0.590)	0.670 (0.676)	0.803 (0.819)	0.957 (0.965)
500	0.633 (0.641)	0.787 (0.765)	0.910 (0.926)	0.997 (0.998)
1,000	0.713 (0.695)	0.867 (0.847)	0.980 (0.979)	1.000 (1.000)

By Equation 12.17 the accuracy depends on n and the gap only through $\sqrt{n}\,|t_1 - t_0|$, and plotted against that product the four curves fall on one (Figure 12.3). So a gap of 0.05 at $n = 1{,}000$ is about as easy as a gap of 0.1 at $n = 200$, since $\sqrt{1000} \times 0.05 = 1.58$ against $\sqrt{200} \times 0.1 = 1.41$.

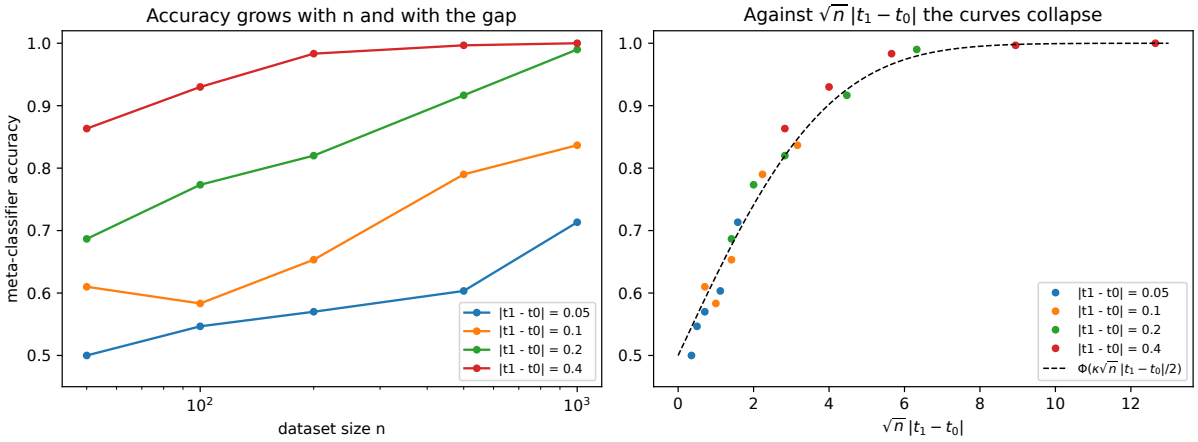


Figure 12.3.: Left: accuracy of the meta-classifier against n for four gaps $|t_1 - t_0|$ (150 shadow and 150 target models per hypothesis, seed 1). Right: the same accuracies against $\sqrt{n}\,|t_1 - t_0|$, with the Gaussian prediction $\Phi(\kappa\sqrt{n}\,|t_1 - t_0|/2)$, $\kappa = 0.647$.

12.4. Permutation invariance

A logistic regression has one parameter vector per function, since different weights give different predictions, but a neural network has many. Swap two hidden units, together with their incoming and outgoing weights, and the network computes exactly the same function. Ganju et al. (2018) observed that a meta-classifier fed the flattened weights then sees each unit in an arbitrary position of the vector, and does poorly. This section works out why for one hidden layer of tanh units, and which representations bring the property back.

Definition 12.3 (A one-layer tanh network and its symmetries)**.** A network with h hidden units computes

$$f_\theta(x) = \sigma(v^\top \tanh(Wx + \beta) + c), \tag{12.18}$$

where row j of the matrix W and entry β_j of the hidden bias vector β compute unit j's input, v_j is its outgoing weight, and c is the output bias. Unit j's parameters are $u_j = (W_{j\cdot}, \beta_j, v_j)$. A **symmetry** is a matrix $g = F\Pi$, where the permutation matrix Π reorders the units and the diagonal matrix F with entries ± 1 flips the sign of some of them. It acts on θ by applying g to the rows of W and to the entries of β and v, leaving c alone.

There are $h!$ orders and 2^h sign patterns, so a generic network, one whose units are nonzero and pairwise neither equal nor opposite, has $h!\,2^h$ parameter vectors that compute the same function: $4! \times 2^4 = 384$ for $h = 4$. The next proposition shows that the symmetry of the function carries over to the distribution of the trained weights, provided training treats every arrangement of the units alike, and that this has a sharp consequence for a linear attacker.

Proposition 12.4 (Symmetries hide the property from raw weights)**.** *Every symmetry g leaves the network's function unchanged,*

$$f_{(gW,\ g\beta,\ gv,\ c)} = f_{(W,\ \beta,\ v,\ c)}. \tag{12.19}$$

Suppose the initial parameters have the same law as their image under every g (for example, independent entries drawn from a distribution symmetric about 0) and training is gradient descent. Then, under both hypotheses of the game,

1. *the law of the trained parameters is invariant under every g;*
2. *every coordinate of every u_j has mean zero, so the mean of a linear score of the raw parameters can differ between the hypotheses only through the output bias c;*
3. *the sum of the units' outer products, $\sum_j u_j u_j^\top$, is invariant under every g, so its entries can serve as features, the* ***second-moment features****.*

Proof. The plan is to show that g passes through tanh, so it leaves the output unchanged, then that it passes through every step of gradient descent, and then to read off what this means for averages.

1. *tanh commutes with g.* tanh acts entry by entry, so permuting its input permutes its output in the same way. tanh is odd, $\tanh(-z) = -\tanh(z)$, so flipping the sign of an entry flips the sign of its output. Hence $\tanh(gz) = g\tanh(z)$.
2. *The output is unchanged.* By step 1 the symmetry can be pulled out of tanh, and $g^\top g = \Pi^\top F^2 \Pi = I$, because each entry of F is ± 1 and undoing a reordering gives back the original. So $(gv)^\top \tanh(gWx + g\beta) = v^\top g^\top g \tanh(Wx + \beta) = v^\top \tanh(Wx + \beta)$, which is Equation 12.19.
3. *Gradient descent commutes with g.* Let $\mathcal{L}$ be the training loss, and let g also denote its action on the whole parameter vector, which only reorders entries and flips signs, so it is linear and orthogonal. Step 2 says $\mathcal{L}(g\theta) = \mathcal{L}(\theta)$ for every θ. Differentiating both sides with the chain rule gives $g^\top \nabla\mathcal{L}(g\theta) = \nabla\mathcal{L}(\theta)$. Multiplying by g, which undoes $g^\top$ because g is orthogonal, gives $\nabla\mathcal{L}(g\theta) = g\nabla\mathcal{L}(\theta)$. One step with learning rate η from $g\theta$ therefore lands at $g\theta - \eta g\nabla\mathcal{L}(\theta) = g(\theta - \eta\nabla\mathcal{L}(\theta))$. Repeating, if T steps from θ_0 reach θ_T, then from $g\theta_0$ they reach $g\theta_T$.
4. *The law is invariant.* The initial parameters θ_0 have the same law as $g\theta_0$. Training maps them to θ_T and, by step 3, to $g\theta_T$, so θ_T and $g\theta_T$ have the same law for every dataset, and averaging over datasets under either hypothesis keeps this. That is claim 1.
5. *Raw means vanish.* Take the g that flips unit j only. By claim 1, u_j and $-u_j$ have the same law, so $\mathbb{E}[u_j] = -\mathbb{E}[u_j]$, which forces $\mathbb{E}[u_j] = 0$ under both hypotheses. A linear score $w^\top\theta$ has mean shift $w^\top(m_1 - m_0)$ between the hypotheses, where m_0 and m_1 are the mean parameter vectors, and $m_1 - m_0$ is zero in every unit coordinate, so the score's mean shift comes only from the output bias c, which no g touches. That is claim 2.
6. *Second moments are invariant.* A permutation only reorders the terms of $\sum_j u_j u_j^\top$, which does not change the sum. A sign flip turns u_j into $-u_j$, and $(-u_j)(-u_j)^\top = u_j u_j^\top$. That is claim 3.

□

Raw weights hide the property because each position in the parameter vector holds a random one of the network's units, with a random sign. The property changes the set of units the network learns, not their positions, so averaging position by position washes it out.

There are three ways to undo the symmetry. A **canonical form** picks one parameter vector per function: flip each unit so that $v_j > 0$, then sort the units by v_j. Sorting is the first remedy of Ganju et al., but it is fragile, because two units with nearly equal v_j swap places from one network to the next. A **set representation** feeds each unit through the same map ϕ, adds the results and applies a second map, $\rho(\sum_j \phi(u_j))$, which does not depend on the order (Zaheer et al. 2017). The features $\sum_j u_j u_j^\top$ are the case $\phi(u) = uu^\top$, with ρ the identity. **Query features** use the network's outputs on probe inputs that the attacker fixes once, such as random points; they are invariant automatically and need only black-box access.

For the best attack, white-box access is never weaker, because whoever holds the weights can compute any query output. A fixed linear meta-classifier is not the best attack, though, and for it a better representation can matter more than more access, so in the example below **black-box beats white-box**.

Example 12.4. The data are as in Example 12.3, with $t_0 = 0.2$ and $t_1 = 0.8$. Each network has $h = 4$ tanh units and is trained on $n = 200$ records by 300 steps of full-batch gradient descent from a random start. For each hypothesis, 150 shadow networks fit the linear test of Lemma 12.2 and 150 target networks measure its accuracy; Figure 12.4 repeats this on a separate run of 100 each.

Each unit has four parameters (two input weights, a bias and an output weight), so the raw parameters number $4 \times 4 + 1 = 17$ with the output bias. The symmetric second-moment matrix has 10 distinct entries, which with the output bias gives 11 features. Table 12.6 also reports each representation's largest **standardized mean gap**: for each feature, the difference of its means under t_0 and t_1, divided by its pooled standard deviation.

Raw parameters are at chance, as claim 2 predicts. Their largest standardized gap, 0.34, is noise: a difference of two means of 150 standardized values has standard deviation $\sqrt{2/150} = 0.115$, and by chance alone the largest of 17 such values is typically two to three times that. Canonical order lifts the accuracy to 0.807, mostly through one feature (Figure 12.4, left), and second moments reach 0.877 with fewer features, because they need no tie-breaking. Probe outputs do best, because they measure directly the function that the property changes.

Table 12.6.: The linear meta-classifier against tanh networks, by representation, with gaps measured on the shadow networks.

Representation	Features	Largest standardized mean gap	Accuracy
Raw parameters	17	0.34	0.497
Canonical order (flip so $v_j > 0$, sort by v_j)	17	0.66	0.807
Unit second moments $\sum_j u_j u_j^\top$	11	2.24	0.877

Representation	Features	Largest standardized mean gap	Accuracy
Outputs on 16 probe inputs	16	2.81	1.000

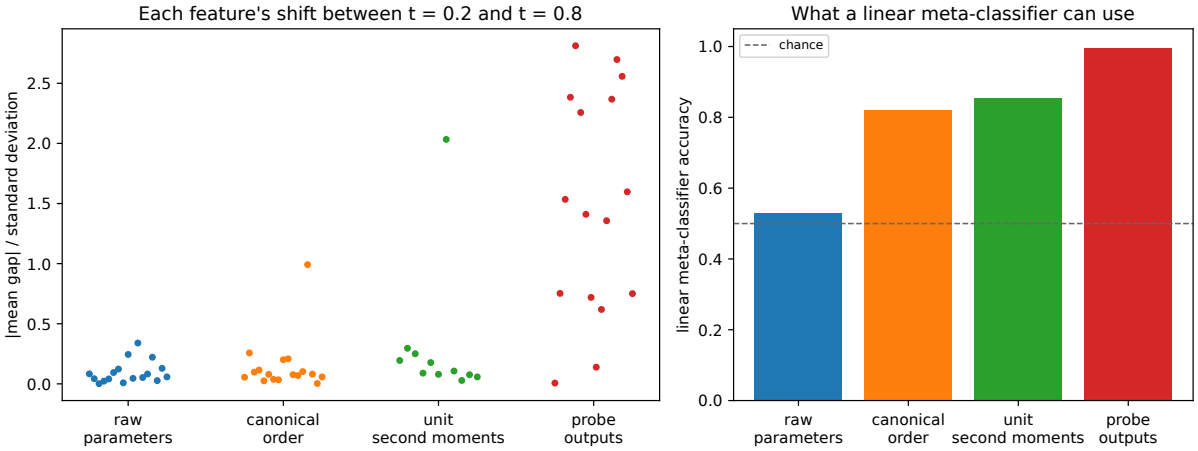


Figure 12.4.: Left: for each representation of 100 tanh networks per hypothesis ($t = 0.2$ against $t = 0.8$), the standardized mean gap of every feature. Right: accuracy of the linear meta-classifier on 100 further networks per hypothesis (seed 1). Raw parameters carry no mean signal; invariant representations do.

12.5. Code

The script code/property_inference.py computes every number in this chapter and draws every figure. The binomial advantages are exact sums, and the logistic regression's population fit and sandwich covariance come from Gauss–Hermite quadrature on a 40×40 grid. From the repository root:

```
uv run python code/part02-attacks/property_inference.py
```

Table 12.7.: The script's output with the default seed.

Experiment	Result
Released count, $t_0 = 0.45$, $t_1 = 0.55$	Property advantage 0.6827, 0.9985, 1.0000 at $n = 100$, 1,000, 10,000 (Table 12.3)
Released count, membership at $t = 0.5$	0.0398, 0.0126, 0.0040 at the same n
Simulated property game, $n = 100$	Advantage 0.6811 over 200,000 games (exact 0.6827)
Logistic regression, population view at $t = \frac{1}{2}$	$\theta = (1.704, 0, 0)$; $\theta' = (0, -1.704, 0)$; Σ_1 diagonal $(12.59, 6.94, 6.04)$; $\kappa = 0.647$

Experiment	Result
2,000 logistic regressions, $n = 1{,}000$, $t = \frac{1}{2}$	Mean $\theta_1 = 1.693$; $n\times$ variances $(11.92, 6.80, 6.08)$ Table 12.5
Meta-classifier sweep tanh networks, $h = 4$, $n = 200$, $t = 0.2$ against 0.8	Accuracy 0.497 (raw), 0.807 (canonical), 0.877 (second moments), 1.000 (probes); Equation 12.19 checked numerically

tests/test_property_inference.py checks that the output still matches.

12.6. Summary

Table 12.8.: The results of this chapter, and where each is used.

Result	Statement	Use
Theorem 12.1	$\text{Adv} \le \text{TV}(P_{n,t_0}, P_{n,t_1}) \le \text{TV}(\mathcal{D}^n_{t_0}, \mathcal{D}^n_{t_1})$, the binomial TV when groups are visible	The ceiling; a released count is the worst case; score attacks by accuracy
Lemma 12.1	A unimodal count shifted by one moves by its largest probability	Membership on a count
Proposition 12.1	$(1-t)\max_k \Pr[\text{Bin}(n-1,t) = k]$	Membership gets harder like $1/\sqrt{n}$
Proposition 12.2	$2\Phi(\sqrt{n}\,\lvert t_1 - t_0\rvert/(2\sqrt{\bar{t}(1-\bar{t})})) - 1$	Properties get easier with n; defences sized for one record do not cover them
Proposition 12.3	$\theta \approx \mathcal{N}(\theta(t), H^{-1}GH^{-1}/n)$	The law of a fitted model
Lemma 12.2	Best accuracy $\Phi(\gamma/2)$, by a linear test	Train a linear meta-classifier
Theorem 12.2, Corollary 12.1	Accuracy $\Phi(\kappa\sqrt{n}\,\lvert t_1 - t_0\rvert/2)$; $\kappa = 2$ for the count at $\bar{t} = \frac{1}{2}$	Predict an attack from θ' and Σ_1; dropping the group column does not hide the property
Proposition 12.4	$h!\,2^h$ equivalent parameter vectors; raw means vanish for tanh	Use canonical order, sums over units or probe outputs

12.7. References

- Ateniese, Mancini, Spognardi, Villani, Vitali & Felici, "Hacking Smart Machines with Smarter Ones: How to Extract Meaningful Data from Machine Learning Classifiers" (International

Journal of Security and Networks, 2015). The first property-inference attack: shadow models trained with and without the property, and a meta-classifier on their parameters.

- Ganju, Wang, Yang, Gunter & Borisov, "Property Inference Attacks on Fully Connected Neural Networks using Permutation Invariant Representations" (CCS 2018). The permutation problem for neural networks, neuron sorting, and the DeepSets meta-classifier.
- Suri & Evans, "Formalizing and Estimating Distribution Inference Risks" (PoPETs 2022). The property game as a test between two training distributions, and leakage measured in records.
- Zhang, Tople & Ohrimenko, "Leakage of Dataset Properties in Multi-Party Machine Learning" (USENIX Security 2021). Black-box property inference from query outputs alone.
- Mahloujifar, Ghosh & Chase, "Property Inference from Poisoning" (IEEE S&P 2022). Adding a few records makes a property far easier to infer.
- Melis, Song, De Cristofaro & Shmatikov, "Exploiting Unintended Feature Leakage in Collaborative Learning" (IEEE S&P 2019). Properties inferred from the updates of collaborative training.
- Salem, Cherubin, Evans, Köpf, Paverd, Suri, Tople & Zanella-Béguelin, "SoK: Let the Privacy Games Begin! A Unified Treatment of Data Inference Privacy in Machine Learning" (IEEE S&P 2023). Property inference as one game among the others of this part.
- Zaheer, Kottur, Ravanbakhsh, Póczos, Salakhutdinov & Smola, "Deep Sets" (NeurIPS 2017). The $\rho(\sum_j \phi(u_j))$ form of permutation-invariant functions.
- van der Vaart, *Asymptotic Statistics* (1998), ch. 5. Asymptotic normality of M-estimators and the sandwich covariance $H^{-1}GH^{-1}$.

13. Memorization and training-data extraction

A language model trained on scraped text sees phone numbers, addresses, API keys and private messages. Ask it to continue "Anna Smith's phone number is" and it may finish the sentence with the real number. Nobody queried a database or broke into anything, because the number was stored in the weights and came out through the model's normal interface. That is **training-data extraction**. Membership inference needs the record in hand and asks whether it was in the training set, while extraction gives the attacker the record itself, often without knowing that it existed.

The one idea of this chapter is the **rank** of a secret under the model. Fix the context in which the secret appears, say "my pin is", and list every value that could fill it, here all five-digit pins, from most to least likely under the model. The rank of the true secret is its position in that list, which is also the number of guesses an attacker needs if it tries the values in that order. Every measure in the chapter reads this rank. Greedy decoding, which always writes the most likely next symbol, reproduces the secret when each of its symbols is the most likely one at its step. Sampling at random from the model finds the secret quickly when it ranks near the top. Exposure counts in bits how far up the list the model has moved it.

The running example is a toy. The training set is a contact list, in which each line is a user name of three random syllables, a space and a five-digit pin, such as `toneka 14524`. There are 5,000 background lines, each seen once, and 600 secret lines, of which 100 each are inserted 1, 2, 4, 8, 16 and 32 times. The model is a character n-gram model, which predicts each character by counting what followed the last $n-1$ characters before it. Its order n plays the role of model size, because a higher order stores counts for more distinct n-grams, strings of n characters, from 2,734 at order 3 to 54,614 at order 8. When a context is written out, `_` stands for a space, so `gule_` is the five characters before the first digit of `migule 16430`.

Table 13.1.: The symbols of this chapter.

Symbol	Meaning
$p(w \mid c)$, $p(x \mid c)$	the model's probability of a next symbol w, or of a string x, after the text c
c; $x = x_1 \dots x_L$; $x_{<t}$	the text before (a prompt, or an n-gram context); a suffix of L symbols; its first $t-1$ symbols
n; $N(c, w)$; α; V	the order of an n-gram model; its counts; the smoothing constant; the alphabet size
k; k^*	the copies of a line in the training set; the copies it needs for greedy decoding to reproduce it
q; m; β	the probability that one sample is the string; the number of samples; the failure probability

Symbol	Meaning
$\mathcal{R}$; $\hat{r}$	a canary's space of possible values; the planted value
p_θ; p_{ref}; Λ	the target model; a reference model; their log-likelihood ratio

13.1. Verbatim, approximate, discoverable, extractable

"The model memorized its training data" can mean several things. Completing a famous quotation leaks nothing private, while emitting a phone number seen once in training does. Some definitions assume that the attacker knows the beginning of the training string, and others ask whether any prompt at all brings it out. You need to say which notion you measure, because one model can score very differently under two of them, as Example 13.4 shows. All of them rest on a language model's probabilities, which come first.

A language model scores a string one symbol at a time, by giving the probability of each possible next symbol after the text so far, and the chain rule multiplies these into the probability of the whole string.

Definition 13.1 (Language model)**.** A **language model** gives, for every preceding text c and every symbol w, a probability $p(w \mid c)$ that the next symbol is w. Write $x_{<t} = x_1 \dots x_{t-1}$ for the symbols before step t of a string $x = x_1 \dots x_L$ of length L. The probability of x after a prompt c is the product of its one-step probabilities, where $c, x_{<t}$ means c followed by $x_{<t}$, and on a log scale the product becomes a sum:

$$p(x \mid c) = \prod_{t=1}^{L} p(x_t \mid c, x_{<t}), \qquad \ln p(x \mid c) = \sum_{t=1}^{L} \ln p(x_t \mid c, x_{<t}). \tag{13.1}$$

Greedy decoding from a prompt c appends the most likely next symbol at each step, and greedy(c) is the string it produces. **Sampling** draws each next symbol at random with the model's probabilities, until an end-of-line symbol or a length limit.

For the pin `16430` after the prompt `migule`, the model multiplies five numbers, the probabilities of `1` after `migule_`, of `6` after `migule_1`, and so on up to `0` after `migule_1643`.

The toy computes these probabilities by counting. An order-n model looks only at the last $n-1$ symbols, which it calls the **context**. The natural prediction is the fraction of times w followed the context in training. But that gives probability 0 to anything never seen, and one zero in the chain rule makes a whole string impossible. Smoothing removes the zeros by pretending that every symbol was seen a fraction of an extra time in every context.

Definition 13.2 (An n-gram model with add-α smoothing)**.** Let the alphabet have V symbols, and let $\alpha > 0$ be a smoothing constant. An **order-n model** stores, for every context c of $n-1$

symbols and every symbol w, the count $N(c, w)$ of times that w followed c in the training text. **Add-α smoothing** adds α to each of the V counts, so the total grows by αV:

$$p(w \mid c) = \frac{N(c, w) + \alpha}{N(c) + \alpha V}, \qquad N(c) = \sum_{w} N(c, w). \tag{13.2}$$

Each symbol has a **code**, its position in a fixed order of the alphabet, and greedy decoding breaks a tie toward the lower code.

The toy uses $\alpha = 0.1$ and $V = 39$ symbols, which are the 26 letters, the 10 digits, the space, an end-of-line symbol and a start symbol, which pads the start of each line to give its first symbols a full context and has the lowest code.

Definition 13.3 (Notions of memorization)**.** Let a language model be trained on a dataset of documents.

1. **Extractable** (Carlini et al. 2021). A string s of the training data is extractable if there is some prompt c, any prompt at all, such that greedy(c) begins with s.
2. **k-eidetic** (Carlini et al. 2021). A string s is k-eidetically memorized if it is extractable and it appears in at most k documents of the dataset (in the toy, lines).
3. **Discoverable** (Carlini et al. 2023, named by Nasr et al. 2023). Suppose the training data contain a prefix c followed by a suffix x. Then x is discoverably memorized if prompting with that same prefix makes greedy decoding produce the suffix, so that greedy(c) begins with x.
4. **Approximate** (Ippolito et al. 2023). The suffix x is approximately memorized if greedy(c) is within distance τ of x in some string distance. In the toy the distance is the number of wrong digits and $\tau = 1$, so a pin with at most one wrong digit counts.

Notions 1 to 3 are **verbatim**, because the output must match exactly.

A discoverable suffix is extractable, because its training prefix is one allowed prompt. The converse fails, because an extractable string may come out after some other prompt while its own prefix leads elsewhere. The most dangerous k-eidetic strings are those with $k = 1$, because a string found in one document only is typically one person's secret, and the model still gives it back.

The notions answer different threat models. Discoverable memorization assumes an attacker who holds the training prefix, which is strong but easy to measure, while extractable memorization fits the real attacker, who has only the model, but needs a search over all prompts. So papers report discoverable rates, and attacks then show how much of the discoverable text an outsider can reach. Verbatim matching also misses near copies, and Ippolito et al. (2023) bypass a filter that blocks verbatim training text by asking for a changed style, such as different capitalization, which returns the text with small edits the filter does not match. A verbatim rate is therefore a lower bound on the leak, not a measure of it.

Greedy decoding is a convenience for measurement, while deployed systems sample, so you also need to know how many samples bring a string out. Each sample is an independent try, and the chance that all of them miss falls exponentially in their number.

Proposition 13.1 (Samples needed to extract a string)**.** *Let $q = p(x \mid c) > 0$ be the probability of the string x after the prompt c, and let $\beta \in (0, 1)$ be a failure probability. Draw m independent samples from c at temperature 1, that is, with each next symbol drawn with exactly the model's probabilities. If*

$$m \geq \frac{\ln(1/\beta)}{q}, \tag{13.3}$$

then at least one of the samples begins with x with probability at least $1 - \beta$.

Proof. The plan is to compute the probability that every sample misses, bound it by an exponential, and solve for m.

1. *One sample hits with probability q.* A sample draws each next symbol x_t with probability $p(x_t \mid c, x_{<t})$. By the chain rule, Equation 13.1, the chance that it begins with x is the product of those probabilities, which is q.
2. *All m samples miss with probability $(1 - q)^m$.* Each sample misses with probability $1 - q$, and the samples are independent, so the misses multiply.
3. *Replace $1 - q$ by an exponential.* The curve e^{-q} is convex, and the line $1 - q$ is its tangent at $q = 0$, since both equal 1 there with slope -1. A convex curve lies above its tangent lines, so $1 - q \leq e^{-q}$. Raising both sides to the power m keeps the order, because both are non-negative:
$$(1 - q)^m \leq e^{-mq}. \tag{13.4}$$
4. *Solve for m.* The miss probability is therefore at most β as soon as $e^{-mq} \leq \beta$. Taking logarithms gives $mq \geq \ln(1/\beta)$, and dividing by $q > 0$ gives Equation 13.3.

□

The proposition converts a probability into an attack budget, so that a string with probability 10^{-3} after its prefix comes out with 99% confidence ($\beta = 0.01$) within about 4,600 samples. It is slightly pessimistic, because it replaces $1 - q$ by the larger e^{-q}, but for rare strings it agrees closely with the exact budget, $\ln \beta / \ln(1 - q)$ rounded up, the smallest m with $(1 - q)^m \leq \beta$.

Example 13.1. Take the secret line `migule 16430`, inserted 4 times, and prompt each model with its prefix `migule`. Greedy decoding gives only `07` and then ends the line at order 3, `81554` at order 4, `14032` at order 5, and the true `16430` at orders 6, 7 and 8. The switch happens at order 6 because an order-6 model reads five characters of context. From the first digit on, its contexts (`gule_`, then `ule_1`, and so on) occur in this line and nowhere else, as the script checks. In such a context the line's own next digit has count 4 and every rival digit has count 0, so greedy decoding emits the line's digit every time. Shorter contexts, such as the four characters `ule_` that an order-5 model reads, are shared with other lines, whose digits can outvote the four copies.

Sampling gives a graded version of the same picture. At order 5, where greedy decoding fails for most secrets, the probability q that one sample from the true prefix reproduces a secret's pin, averaged over the 100 secrets of each group, is 1.4×10^{-3} at 4 copies and 0.142 at 32 copies. With $\beta = 0.01$, the exact budgets $\ln \beta / \ln(1 - q)$ for these mean probabilities, computed before rounding q, are 3,252 and 31 samples.

13.2. Duplication and model size

Example 13.1 showed one line becoming memorized as the model grew. This section asks which lines get memorized in general, because that decides which data a defence must protect first. Carlini et al. (2023) found that discoverable memorization grows linearly in the logarithm of how often a string is duplicated, of model size and of prompt length. Kandpal, Wallace & Raffel (2022) found that a sequence repeated 10 times is generated about 1,000 times more often than one seen once. The n-gram toy lets you derive why, for every line, from its counts.

The idea is a vote. Greedy decoding follows the largest count at each step, and each copy of a line adds one vote for the line's own next symbol in every context the line passes through. The next lemma states how copies change the counts and which symbol wins a step, which the proof of Theorem 13.1 needs, and the size of one copy's effect, which the canary of Section 13.3 uses.

Lemma 13.1 (What counting implies)**.** *In an order-n model with add-α smoothing (Definition 13.2), the following hold.*

1. *Training on k copies of a line adds k to the count of a (context, next symbol) pair for each time the line passes through it, and changes no other count.*
2. *In a fixed context c, the most likely symbol is the one with the largest count, that is,* $\arg\max_w p(w \mid c) = \arg\max_w N(c, w)$.
3. *Suppose that a context c occurs in the training data only in the k copies of one line. Then the line's own next symbol and each other symbol get, respectively,*

$$p(\mathit{true} \mid c) = \frac{k+\alpha}{k+\alpha V}, \qquad p(\mathit{other} \mid c) = \frac{\alpha}{k+\alpha V}, \tag{13.5}$$

so the true symbol is $(k+\alpha)/\alpha$ times as likely as any other, whatever V is. A context never seen at all gives every symbol probability $1/V$.

Proof.

1. *Training is counting.* A line passes through a sequence of (context, next symbol) pairs, and training on it adds 1 to the count of a pair each time the pair occurs. Training on k copies repeats this k times.
2. *Smoothing keeps the order.* In a fixed context, Equation 13.2 adds the same α to every count and divides by the same $N(c) + \alpha V$. Both operations are increasing, so the symbol with the largest count still has the largest probability.
3. *A context of one line.* At such a context the line's next symbol has count k, every other symbol has count 0, and the total is $N(c) = k$. Putting those counts into Equation 13.2 gives Equation 13.5, and the two probabilities share a denominator, so their ratio is $(k+\alpha)/\alpha$. A context never seen has all counts 0, which gives $\alpha/(\alpha V) = 1/V$.

□

Table 13.2 evaluates Equation 13.5 for the toy. For one copy, $\alpha V = 3.9$, so $p(\text{true} \mid c) = 1.1/4.9 = 0.2245$ and $p(\text{other} \mid c) = 0.1/4.9 = 0.0204$, a ratio of $1.1/0.1 = 11$. The last column writes the ratio in bits, $\log_2 11 = 3.46$, the unit that exposure uses in Section 13.3. That is the mechanism

of memorization in this toy: a context that belongs to one training line alone, followed by that line's next symbol.

Table 13.2.: Add-α probabilities at a context that occurs only in the k copies of one line ($\alpha = 0.1$, $V = 39$).

Copies k	$p(\text{true} \mid c)$	$p(\text{other} \mid c)$	Ratio	Bits per digit, $\log_2$ ratio
1	0.2245	0.0204	11.0	3.46
4	0.5190	0.0127	41.0	5.36
16	0.8090	0.0050	161.0	7.33

To turn the vote into a number, count the rest of the corpus separately from the line's copies. At each step, the **gap** is how far the strongest rival symbol is ahead of the true symbol in those counts. The copies must close the gap at every step, so the step where the rival is furthest ahead sets the number of copies needed.

Theorem 13.1 (Copies needed for greedy extraction)**.** *Let s be a training line made of a prefix c and a suffix $x = x_1 \dots x_L$, inserted k times into the training set of an order-n model with add-α smoothing. Let c_t be the context at step t, the last $n-1$ symbols of $c\,x_{<t}$. Let N_- be the counts of the rest of the training set, without the copies of s. Assume that each context c_t occurs only once within s, so that a copy adds to no rival pair. The gap at step t is the bracket below, reduced by one at a step where the true symbol has a lower code than the strongest rival, because there a tie is already a win. Then greedy decoding from c emits x if and only if $k \geq k^*(s)$, where*

$$k^*(s) = \max_{t \leq L} \Big[\max_{w \neq x_t} N_-(c_t, w) - N_-(c_t, x_t) \Big] + 1. \tag{13.6}$$

A line in the training set has at least one copy, so a k^* below 1 counts as 1. A line whose contexts are unique to it, with $N_- = 0$ at every step, has $k^* = 1$, since the start symbol, with the lowest code, would win a tie at such a context. Over a population of lines, the fraction discoverably memorized at k copies is the fraction with $k^*(s) \leq k$.

Proof. The plan is to find when the true symbol wins one step, then require every step to win, and finally read off the special cases.

1. *Greedy picks the largest count.* By part 2 of Lemma 13.1, at context c_t greedy decoding emits the symbol with the largest count $N(c_t, w)$.
2. *What the copies add.* By part 1 of Lemma 13.1, the k copies add k to $N(c_t, x_t)$. Because c_t occurs only once in s, they add nothing to a rival pair (c_t, w) with $w \neq x_t$. So the true symbol's count is $N_-(c_t, x_t) + k$, and each rival's is still $N_-(c_t, w)$.
3. *Winning one step.* The true symbol wins outright when its count beats the largest rival count:
$$N_-(c_t, x_t) + k > \max_{w \neq x_t} N_-(c_t, w). \tag{13.7}$$
Moving $N_-(c_t, x_t)$ to the right turns this into $k > \text{gap}_t$, where gap_t is the bracket of Equation 13.6. Both sides are whole numbers, so this is $k \geq \text{gap}_t + 1$. When the true symbol has the lower code, a tie already wins, which removes the $+1$ at that step.

4. *Every step must win.* If some step loses, greedy decoding emits a wrong symbol there, so its output does not begin with x. So x comes out exactly when $k \geq \text{gap}_t + 1$ at every step, which is $k \geq k^*(s)$.
5. *Unique contexts.* If $N_{_}(c_t, \cdot) = 0$ at every step, every gap is $0 - 0 = 0$, so $k^* = 1$.
6. *The population rate.* Each line is memorized at k copies exactly when $k^*(s) \leq k$, so the fraction of lines memorized is the fraction for which this holds.

□

Figure 13.1 draws steps 3 and 4 for one line. Each copy counts at every step at once, because it adds one to every pair on the line's path, so the largest gap decides and not the sum of the gaps.

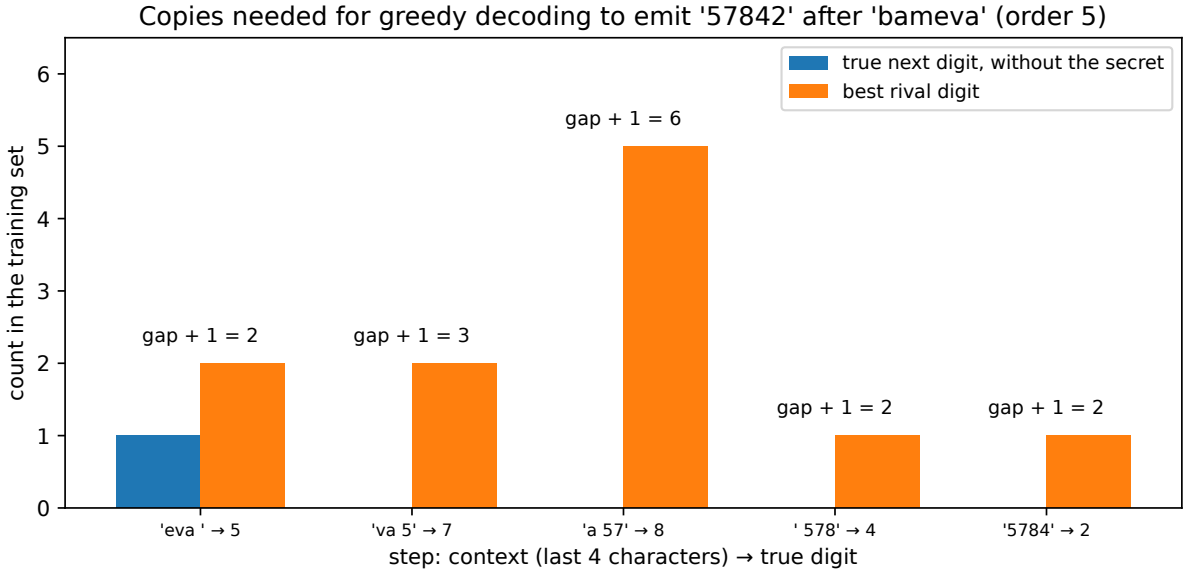


Figure 13.1.: One secret line inserted once, and an order-5 model. At each step of the pin, the bars show the count of the true next digit (blue) and of its strongest rival (orange) in the rest of the training set. The label is the gap plus one, the copies that step needs. The line needs as many copies as its hardest step.

Duplication works as a vote, and model size works by splitting the vote. A longer context occurs at fewer places in the training text, so its rival counts are smaller, while the line's own k votes follow it into every context. In the limit every line has contexts of its own and $k^* = 1$, which is the toy version of "larger models memorize more". For the same reason rare data are exposed first, since a line whose name shares few syllables with other names reaches a context of its own at a lower order. Real models show the same pattern. The change in the loss on an example when that example leaves the training set is its **self-influence** (Definition 7.3 in Algorithmic stability and generalization), and atypical examples, the analogue of a line whose contexts nobody else shares, have the largest (Feldman 2020).

Deduplication is the cheapest defence, because it lowers k for every repeated string. Lee et al. (2022) found that models trained on deduplicated data emit memorized text about ten times

less often, without losing quality. By Theorem 13.1 it does nothing for singletons in a high-capacity model, where $k = 1$ already and $k^* = 1$.

Example 13.2. The script prompts each of the 600 secrets with its true prefix, decodes five symbols greedily, and compares them with the pin (Table 13.3). It also computes k^* for every secret from Equation 13.6, and the prediction $\Pr[k^* \le k]$ matches the verbatim rate exactly in all 36 cells, as Theorem 13.1 says it must (the script also handles contexts that recur within a line).

Along a row of the table, more copies mostly means more memorization, so that at order 5 the rate climbs from 0 at one copy to 0.70 at 32, and the small dips, such as 0.99 to 0.97 at order 7, come from the different 100 secrets of each column. Down a column, a larger model memorizes more. A line seen once is safe at orders up to 5, memorized 8% of the time at order 6, 70% at order 7 and 98% at order 8. Those singletons of the order-8 model are 1-eidetic in the sense of Definition 13.3.

Table 13.3.: Fraction of 100 secret lines per cell whose pin greedy decoding reproduces from the true prefix, verbatim / with at most one digit wrong. Rows are model orders, columns copies in the training set.

Order n	$k = 1$	$k = 2$	$k = 4$	$k = 8$	$k = 16$	$k = 32$
3	0.00 / 0.00	0.00 / 0.03	0.00 / 0.00	0.00 / 0.00	0.00 / 0.02	0.00 / 0.05
4	0.00 / 0.00	0.00 / 0.00	0.00 / 0.01	0.00 / 0.00	0.00 / 0.00	0.13 / 0.17
5	0.00 / 0.00	0.00 / 0.00	0.05 / 0.06	0.11 / 0.15	0.42 / 0.42	0.70 / 0.70
6	0.08 / 0.13	0.65 / 0.69	0.76 / 0.78	0.80 / 0.85	0.90 / 0.91	0.96 / 0.97
7	0.70 / 0.78	0.95 / 0.96	0.98 / 0.98	0.98 / 0.99	0.99 / 0.99	0.97 / 0.98
8	0.98 / 0.98	1.00 / 1.00	1.00 / 1.00	0.99 / 0.99	1.00 / 1.00	1.00 / 1.00

Figure 13.2 plots the same rates for an independent draw of the corpus, with the same pattern. The approximate rates are barely above the verbatim ones, because a pin that loses one contest puts greedy decoding in the wrong context, where it wanders off. Natural text has many near-ties, such as synonyms and punctuation, and there approximate memorization adds more (Ippolito et al. 2023).

The experiment of Carlini et al. (2023) has the same shape at scale. They prompted GPT-Neo and GPT-J models, trained on the Pile, a public collection of web and book text, with the first 50 tokens, the word pieces those models use as symbols, of training sequences and counted exact greedy matches of the next 50. The largest model, with 6 billion parameters, reproduces at least 1% of its training data this way.

13.3. Canaries and the exposure metric

Discoverable memorization is all or nothing. An auditor wants a graded answer: how much closer to the secret does the model put an attacker? Carlini et al. (2019, “The Secret Sharer”) proposed planting random secrets of a fixed format in the training data and measuring where the trained model ranks each among all secrets of the same format. Planting the secret gives the right answer and the size of the space, so memorization can be measured exactly.

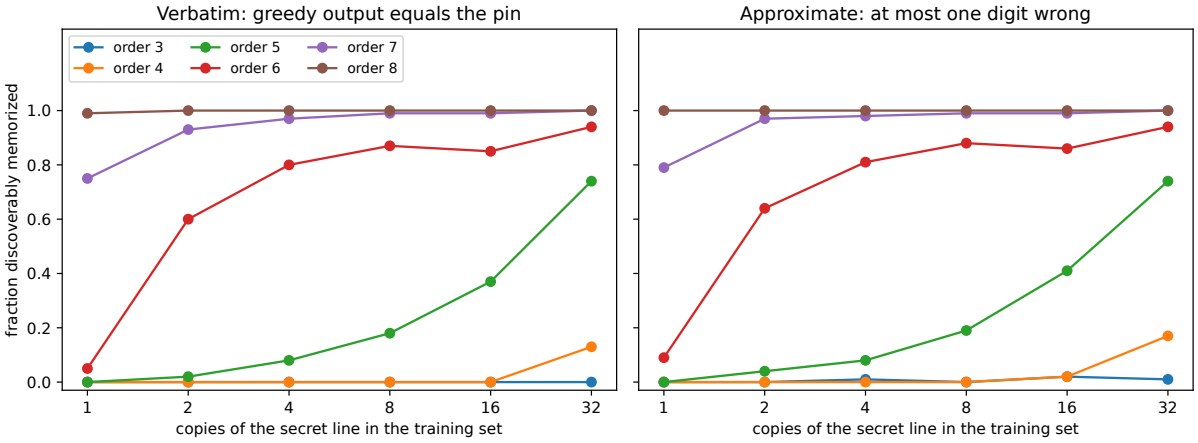


Figure 13.2.: Discoverable memorization against copies of the secret in the training set, for n-gram models of order 3 to 8, on an independent draw of the corpus. Left: greedy decoding reproduces the pin exactly. Right: at most one digit is wrong.

An attacker who guesses from the highest score down needs as many guesses as the rank of the true secret. Without the model the attacker faces every candidate, so the ratio of the rank to the size of the space measures what the model gave away, and on a log scale it becomes a difference of bits.

Definition 13.4 (Canary, rank and exposure)**.** A **canary** is a sentence with a hole, together with a **randomness space** $\mathcal{R}$ of values that can fill the hole. In the toy, the sentence is "my pin is" followed by a value r, one of the $|\mathcal{R}| = 10^5$ pins. A value $\hat{r}$ is drawn uniformly at random from $\mathcal{R}$, and the sentence filled with $\hat{r}$ is inserted k times into the training data. After training, the **rank** of a candidate r is the number of candidates at least as likely as it under the model, itself and every tie included:

$$\operatorname{rank}(r) = \left|\{r' \in \mathcal{R} : p(r' \mid \text{sentence}) \geq p(r \mid \text{sentence})\}\right|, \tag{13.8}$$

where $p(r \mid \text{sentence})$ is the model's probability of r in the hole, after the sentence's text. The **exposure** of the canary is the number of bits of search the model saves:

$$\operatorname{exposure}(\hat{r}) = \log_2 |\mathcal{R}| - \log_2 \operatorname{rank}(\hat{r}). \tag{13.9}$$

Ties are counted against the attacker, who may have to try every tied candidate first. Because the rank lies between 1 and $|\mathcal{R}|$, the exposure lies between 0 and $\log_2 |\mathcal{R}|$. Solving Equation 13.9 for the rank gives $2^{\log_2 |\mathcal{R}| - \text{exposure}}$ guesses instead of $|\mathcal{R}|$, so each bit of exposure halves the work. The maximum, $\log_2 10^5 = 16.61$ bits for a pin, is the **min-entropy** of a uniform canary, the base-2 logarithm of its number of equally likely values, in the sense of Information theory (Definition 5.5), so a canary at full exposure has lost all of it.

Exposure is an audit tool because you know how it behaves when nothing was memorized, as a space of four candidates shows. Suppose the model was trained without the canary and gives the four candidates different scores. The drawn value is then independent of the model's order, so it is equally likely to sit at rank 1, 2, 3 or 4, and its exposure is $2 - \log_2(\text{rank})$, as Table 13.4 lists.

Table 13.4.: The four equally likely exposures of a canary from a space of four candidates that was never inserted.

Rank	1	2	3	4
Exposure $2 - \log_2(\text{rank})$	2	1	0.415	0

The mean of the four is $\frac{1}{4}(2 + 1 + 0.415 + 0) = 0.854$. The probability of exposure at least 1 is $\frac{1}{2} = 2^{-1}$, from ranks 1 and 2, and of exposure at least 2 is $\frac{1}{4} = 2^{-2}$, from rank 1 only. The next theorem shows that the tail 2^{-h} holds, as a bound, for every space.

Theorem 13.2 (Exposure of a canary that was not memorized)**.** *Suppose the model does not depend on the drawn value $\hat{r}$, for instance because the canary was never inserted. Then for every threshold $h \geq 0$,*

$$\Pr\left[\text{exposure}(\hat{r}) \geq h\right] \leq 2^{-h}. \tag{13.10}$$

If moreover no two candidates tie, the average exposure is

$$\mathbb{E}[\text{exposure}(\hat{r})] = \log_2 |\mathcal{R}| - \frac{\log_2(|\mathcal{R}|!)}{|\mathcal{R}|}, \tag{13.11}$$

which tends to $\log_2 e = 1.4427$ bits as $|\mathcal{R}|$ grows.

Proof. The plan is to show that the canary lands at a uniformly random position in a fixed order, count the positions with high exposure, check that ties only help, and average the exposure over the positions.

1. *The order is fixed.* The model does not depend on $\hat{r}$, so its scores $p(r \mid \cdot)$ are fixed numbers before $\hat{r}$ is drawn. With no ties they order $\mathcal{R}$ into positions $1, \dots, |\mathcal{R}|$.
2. *The canary's position is uniform.* The value $\hat{r}$ is uniform on $\mathcal{R}$ and independent of the scores, so its position is equally likely to be any of $1, \dots, |\mathcal{R}|$.
3. *Count the positions with high exposure.* By Equation 13.9, exposure at least h means $\log_2 \text{rank} \leq \log_2 |\mathcal{R}| - h$, that is rank $\leq |\mathcal{R}|2^{-h}$. There are $\lfloor |\mathcal{R}|2^{-h} \rfloor$ such positions, each with probability $1/|\mathcal{R}|$, so
$$\Pr\left[\text{exposure}(\hat{r}) \geq h\right] = \frac{\lfloor |\mathcal{R}|2^{-h} \rfloor}{|\mathcal{R}|} \leq 2^{-h}. \tag{13.12}$$
4. *Ties keep the bound valid.* With ties, break them in a fixed order to get positions as before. The rank counts every candidate at least as likely as $\hat{r}$, so it is never below the position of $\hat{r}$. A higher rank means a lower exposure, so the probability of a high exposure can only fall.
5. *The mean.* With the rank uniform on $1, \dots, |\mathcal{R}|$, the average of $\log_2 \text{rank}$ is $\frac{1}{|\mathcal{R}|} \sum_{i=1}^{|\mathcal{R}|} \log_2 i$. A sum of logarithms is the logarithm of the product, and $1 \cdot 2 \cdots |\mathcal{R}| = |\mathcal{R}|!$, so the average is $\log_2(|\mathcal{R}|!)/|\mathcal{R}|$. Subtracting it from $\log_2 |\mathcal{R}|$ gives Equation 13.11.
6. *The limit.* Stirling's approximation of the factorial, derived in Section E.2, says that $\ln(|\mathcal{R}|!)$ is $|\mathcal{R}| \ln |\mathcal{R}| - |\mathcal{R}|$ up to a correction that grows no faster than a multiple of $\ln |\mathcal{R}|$. Dividing by $|\mathcal{R}| \ln 2$ turns this into base-2 logarithms per candidate, and $1/\ln 2 = \log_2 e$, so
$$\frac{\log_2(|\mathcal{R}|!)}{|\mathcal{R}|} = \log_2 |\mathcal{R}| - \log_2 e + O\Big(\frac{\ln |\mathcal{R}|}{|\mathcal{R}|}\Big). \tag{13.13}$$
The correction vanishes as the space grows, so the mean exposure tends to $\log_2 e$.

□

The bound is tight whenever $|\mathcal{R}|2^{-h}$ is a whole number, as in the space of four. Read with h set to the observed exposure, it says that $2^{-\text{exposure}}$ is a **p-value**, the probability, if nothing was memorized, of a result at least as extreme as the one observed. "Was this canary inserted?" is a membership question, and the test that says "inserted" when the exposure is at least h has false positive rate at most 2^{-h}, by Equation 13.10. Membership inference calibrates such tests with **shadow models**, trained with and without a record, but none are needed here, because Equation 13.10 already says how a random canary's exposure behaves.

Canaries also test defences, because you can compare exposure across two training runs that differ in one thing, such as deduplication or unlearning. With differentially private training, which Part 3 defines and which limits how much any one record can change the model, exposure near the null mean says that the planted secrets were not memorized, as Carlini et al. (2019) found for their canaries under private training.

Example 13.3. The script plants "my pin is 58745" and computes the exact exposure by scoring all 10^5 pins, for orders 2, 3 and 4, which do not saturate at once (Table 13.5). At $k = 0$ the canary is one draw that was never inserted, so its exposure is a single sample of the null distribution of Theorem 13.2. The jump from $k = 0$ to $k = 1$ copies depends on whether a digit of the canary sits in a context that only the canary's copies fill. No background line contains `s_`, an `s` followed by a space, because names are built from consonant-vowel syllables and so end in a vowel.

At order 2 the first digit's context is a single space, shared by all 11,300 lines, so one copy is a single vote among thousands and moves the exposure by $0.41 - 0.40 = 0.01$ bits. At order 3 the first digit's context is `s_`, which occurs only in the canary. By part 3 of Lemma 13.1, one copy then makes the first digit 11 times as likely as each other first digit, a factor worth $\log_2 11 = 3.46$ bits. The exposure rises from 0.89 to 4.05, by $4.05 - 0.89 = 3.16$ bits. That is close to 3.46 but not equal, because multiplying the canary's probability by 11 does not divide its rank by 11. The rank falls only by the number of pins the canary overtakes, and that number depends on how densely the scores lie just above it. At order 4 the first two digits have the unique contexts `is_` and `s_5`, worth about $2 \times 3.46 = 6.92$ bits, of which the exposure, going from 3.76 to 9.98, gains $9.98 - 3.76 = 6.22$. The remaining digits share their contexts with the 5,000 background pins, and they move only as the copies outvote them.

Table 13.5.: Exposure in bits of the canary "my pin is 58745" against the number of copies k in the training set. The maximum is $\log_2 10^5 = 16.61$.

Order n	$k = 0$	$k = 1$	$k = 2$	$k = 4$	$k = 8$	$k = 16$	$k = 32$	$k = 64$
2	0.40	0.41	0.43	0.47	0.55	0.73	1.21	2.54
3	0.89	4.05	4.44	4.71	5.24	6.59	9.69	15.61
4	3.76	9.98	12.15	15.02	15.61	16.61	16.61	16.61

Figure 13.3 shows the same movement on the full distribution of scores, and Figure 13.4 averages several canaries.

To check Theorem 13.2, treat each of the 10^5 pins in turn as a canary that was never inserted. For orders 2, 3 and 4 the scores have no ties, so their exposures are exactly the uniform-rank values.

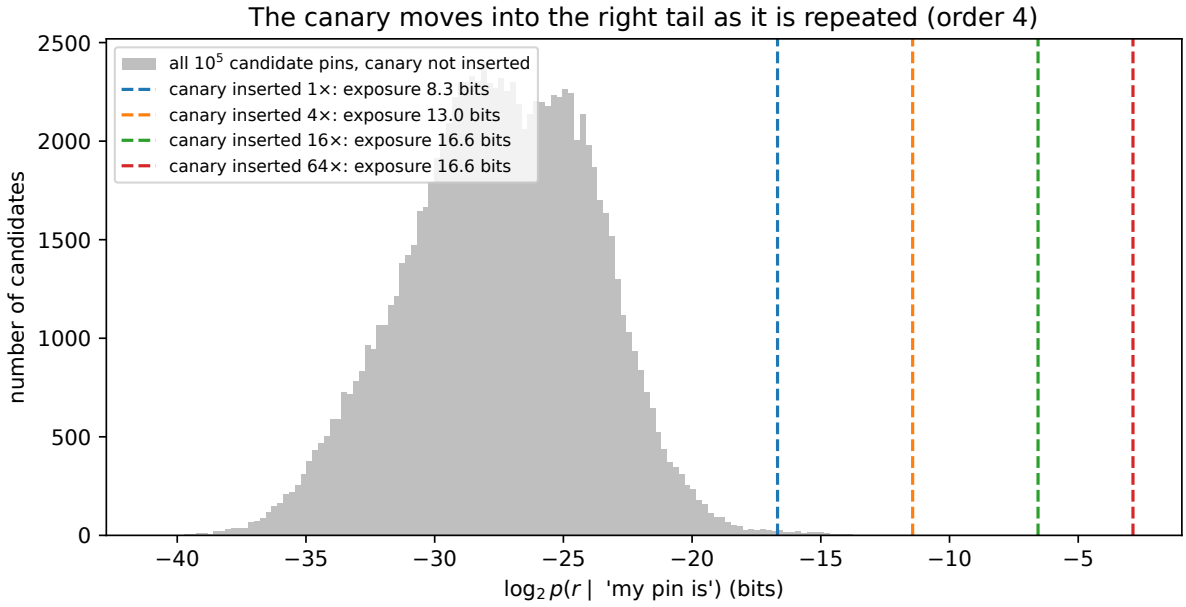


Figure 13.3.: Histogram of $\log_2 p(r \mid \text{my pin is})$ over all 10^5 pins for an order-4 model without the canary (grey), and the score of the canary after it is inserted 1, 4, 16 and 64 times (dashed). Each insertion pushes the canary further into the right tail.

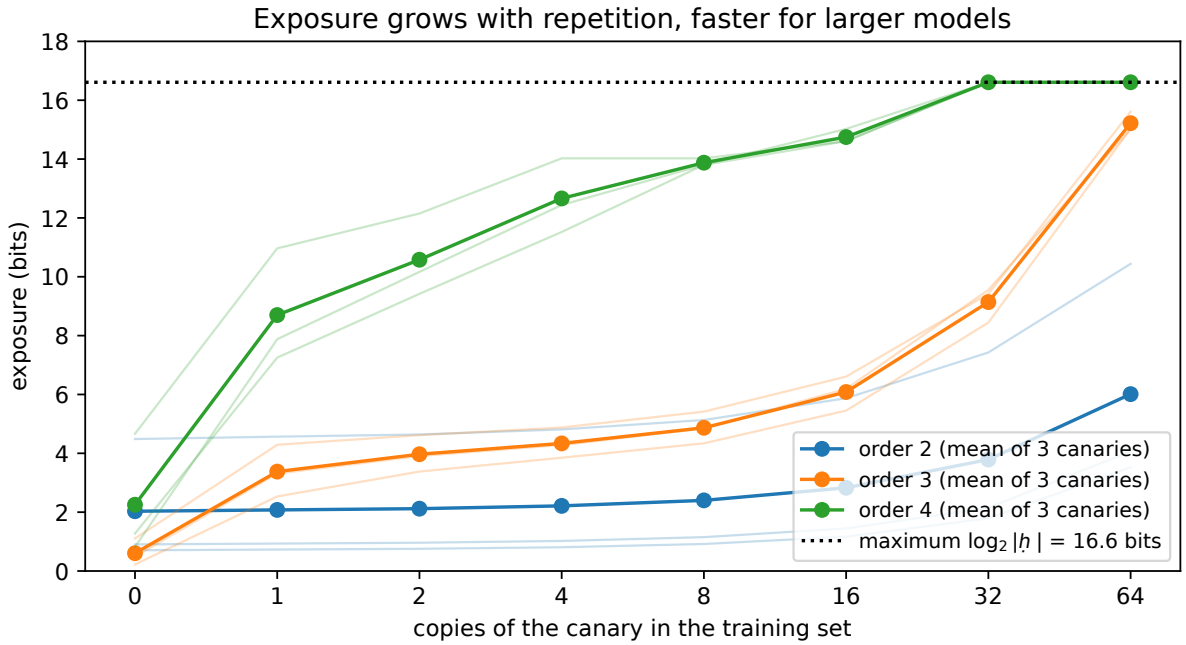


Figure 13.4.: Exposure against copies of the canary for n-gram models of order 2, 3 and 4. Thin lines are individual canaries and markers their mean; the dotted line is the maximum, $\log_2 10^5$. Larger models reach the maximum with fewer copies.

Their mean is 1.443 bits, against 1.4426 from Equation 13.11, and a fraction $0.03125 = 1/32 = 2^{-5}$ has exposure at least 5, exactly on the bound Equation 13.10. Ties work the other way, because the rank counts every tied candidate, so a model that ties many pins shows low exposure whatever it stores. The order-8 model trained on the contact list alone has never seen the contexts after "my pin is", so it gives every pin the same probability, every rank is $|\mathcal{R}|$ and every exposure is 0.

13.3.1. Estimating exposure in a large space

For a large space, such as 9-digit numbers, exposure has to be estimated from a sample of candidates. The counting estimate of rank $/|\mathcal{R}|$ is the fraction of sampled candidates at least as likely as the canary. With m samples its smallest non-zero value is $1/m$, so it cannot resolve exposures above $\log_2 m$. Extrapolation fits a distribution to the sampled scores and reads off its tail beyond the canary's score, and Carlini et al. (2019) fit a skew-normal, a normal with one heavier tail. At order 3, the script estimates the canary's exposure from 10,000 random pins in both ways, fitting a plain normal for the second (Table 13.6).

Table 13.6.: Exposure of the canary at order 3, computed exactly over all 10^5 pins and estimated from 10,000 random pins, by counting and by fitting a normal to their scores.

Copies k	Exact exposure	Rank	Sampling estimate	Normal fit
1	4.05	6,018	4.13	4.47
2	4.44	4,606	4.43	5.57
4	4.71	3,823	4.68	6.81
8	5.24	2,648	5.35	8.28

Counting is accurate here, because every exposure is well below $\log_2 10^4 = 13.3$ bits, the largest value that 10,000 samples can resolve. The normal fit overestimates by up to 3 bits (at 8 copies, $8.28 - 5.24 = 3.04$), because the scores have a heavier right tail than a normal. It therefore puts too few pins above the canary, so the rank looks smaller and the exposure larger, and that tail is the part you cannot sample.

13.4. Extraction without the prefix: generate, rank, verify

The attacker of Section 13.1 had the training prefix, but a real attacker has only the model. This section asks what such an attacker can extract, and how it tells memorized text from text that is merely likely. Carlini et al. (2021) answered with a three-stage attack on GPT-2, whose hard stage, the ranking, is a hypothesis test.

Definition 13.5 (Generate-rank-verify attack)**.** The attacker **generates** a large number of samples from the model, **ranks** the distinct generations by a score of how likely each is to be memorized, and **verifies** the top of the ranking against a source of training text. Its quality is the **precision** of the top of the ranking, the fraction of the top-ranked generations that are real training data.

Verifying is a search, against the training data or public sources. Ranking needs more thought, because a string may be likely as ordinary text that any model finds likely, or because the target memorized it, and only the second is a leak. So for a generated string x, the attacker tests the hypothesis H_0, "x was drawn from common text", against H_1, "x was drawn from what the target memorized". A **reference model** p_{ref}, trained on data from the same distribution but not on the target's training set, gives the distribution of common text, and the target model p_θ itself stands in for H_1, since it gives memorized strings high probability. The evidence for memorization is then the ratio of the two probabilities, a difference on a log scale.

Proposition 13.2 (Rank generations by the likelihood ratio)**.** *Suppose that under H_0 a string has probability $p_{\text{ref}}(x)$ and under H_1 probability $p_\theta(x)$. Call the fraction of H_0 strings a test flags its false positive rate (FPR), and the fraction of H_1 strings its true positive rate (TPR). Among all tests with a given FPR, the one with the highest TPR flags a string when the log-likelihood ratio*

$$\Lambda(x) = \ln p_\theta(x) - \ln p_{\text{ref}}(x) \tag{13.14}$$

is above a threshold. Ranking by $\ln p_\theta(x)$ alone gives the same order when p_{ref} is constant.

Proof.

1. *The best test thresholds the ratio.* The Neyman–Pearson lemma (Theorem 4.1) says that, among all tests with a given FPR, the one with the highest TPR flags x when the likelihood ratio $p_\theta(x)/p_{\text{ref}}(x)$ is above a threshold. Each flagged string costs false positives in proportion to $p_{\text{ref}}(x)$ and gains true positives in proportion to $p_\theta(x)$, so for a fixed budget of false positives you catch the most by flagging the largest ratios first.
2. *Take logarithms.* The logarithm is increasing, so it keeps the order of the ratios, and thresholding the ratio is the same as thresholding its logarithm, Equation 13.14.
3. *Raw likelihood.* If p_{ref} is a constant, $\ln p_{\text{ref}}(x)$ is the same for every string, so Λ and $\ln p_\theta$ order the strings in the same way.

□

Ranking by raw likelihood, $\ln p_\theta(x)$ alone, treats every string as equally common a priori. That assumption is wrong exactly for strings that are likely under any model, such as software licences, news boilerplate and number sequences, and those are the strings raw likelihood puts first. Extracting them reveals nothing, and since a reference model knows them too, their ratio is close to 1 and Equation 13.14 cancels them. Membership inference calibrates in the same way, comparing the target's loss on a record, minus the log of its probability, with the loss from models that did not see it.

The reference can be crude. Carlini et al. (2021) used a smaller model of the same family, and also zlib, a general-purpose compressor whose output length on the text measures its predictability without any model and stands in for $-\ln p_{\text{ref}}(x)$. The attacker cares about precision more than recall, because it checks the top few candidates by hand. Ordinary generations vastly outnumber memorized ones, so flagging a few strings with high precision needs a tiny FPR, the same operating point that matters in membership inference.

Example 13.4. The attacker samples 20,000 lines from the order-6 model, starting each at the beginning of a line. To make ranking hard, 50 **public** lines, such as the entries of a published directory, are added 100 times each to the target's training set. The reference model is trained on an independent contact list of 5,000 lines plus the same 50 public lines, 100 times each. A generation counts as a hit if it is a private training line, background or secret.

Of the 20,000 samples, 14,852 are distinct, and among them are 197 private training lines and all 50 public lines. By duplication group, the samples contain none of the secrets inserted 1, 2 or 4 times, 11% of those inserted 8 times, 86% of those inserted 16 times and all of those inserted 32 times. With the prefix, 8% of the singletons and 65% of the lines seen twice come out at order 6 (Table 13.3). Without the prefix, a sample must first produce the right name before it can produce the pin, and only heavily duplicated lines make their names likely enough.

Ranking decides the precision (Figure 13.5). Of the top 100 distinct generations by target likelihood, only 45 are private training lines, and the rest are mostly the 50 public lines, the most likely strings the model knows. Ranking by Equation 13.14, 85 of the top 100 are private training lines, because the reference model also finds the public lines likely and cancels them.

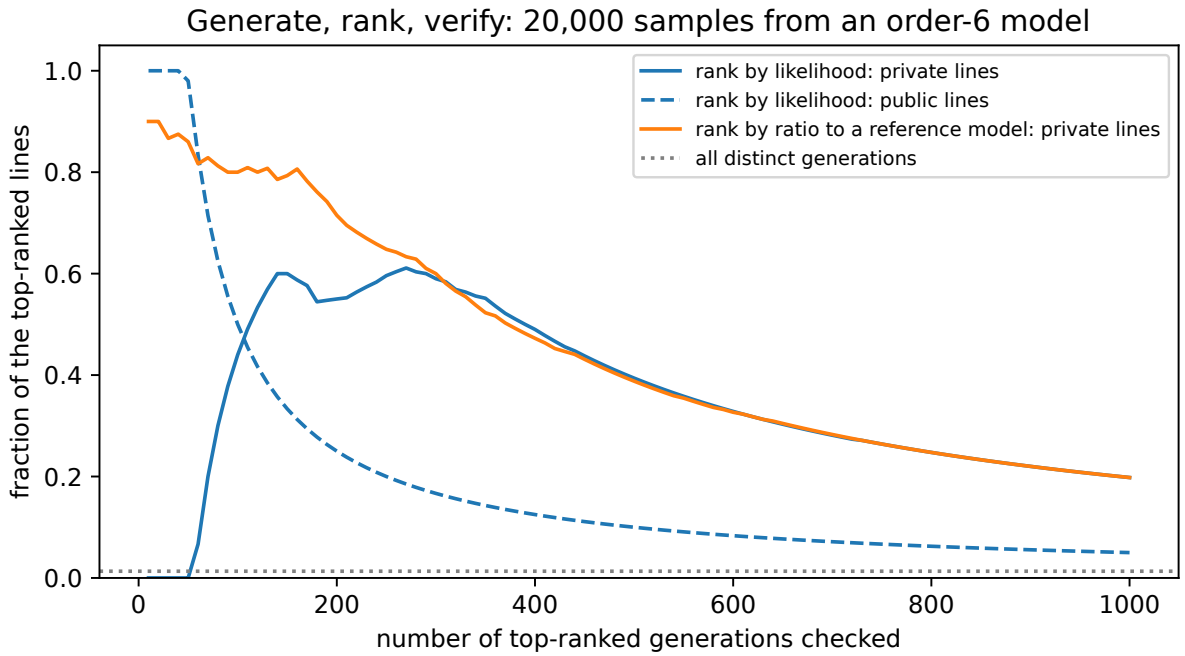


Figure 13.5.: Fraction of the top-ranked distinct generations that are private training lines (solid) or public lines (dashed), from 20,000 samples of an order-6 model trained with 50 public lines repeated 100 times. Ranking by likelihood puts the public lines first, and ranking by the ratio to the reference model puts the private lines first.

13.4.1. The divergence attack on a production model

A chat model that has been **aligned**, further trained to answer as an assistant rather than continue documents, rarely emits training data when sampled, so the toy's generate stage fails on it. Nasr

et al. (2023) found a prompt that brings the data out anyway, verifying generations against the training sets of open models and, for closed ones, against a 9-terabyte corpus of public web text. Their **divergence attack** asked ChatGPT (gpt-3.5-turbo) to repeat a single word forever. After many repetitions the model left its chat behaviour and began to emit text from its pretraining data, the web text it learned from before alignment, much of it matching the web corpus verbatim. They report training data emitted about 150 times as often as in normal use, and over 10,000 unique memorized examples, including contact details, for about 200 US dollars.

13.5. Code

The script code/memorization.py builds the contact list, trains the n-gram models by counting, and runs every experiment in this chapter. From the repository root:

```
uv run python code/part02-attacks/memorization.py
```

Table 13.7.: The script's output with the default seed.

Experiment	Result
Corpus	5,000 background lines, 600 secrets (100 each at 1, 2, 4, 8, 16, 32 copies), 11,300 lines, 146,900 symbols
Model size, distinct n-grams	Order 3: 2,734; 4: 19,129; 5: 42,407; 6: 52,610; 7: 54,334; 8: 54,614
One secret, `migule 16430` $\times$ 4	$k^* = 151, 43, 9, 1, 1, 1$ for orders 3 to 8; greedy reproduces it from order 6
Discoverable memorization	Table 13.3; $\Pr[k^* \le k]$ matches greedy in 36 of 36 cells
Sampling from the prefix, order 5	Mean $p(\text{pin} \mid \text{prefix})$ 1.4×10^{-3} at 4 copies, 0.142 at 32; 3,252 and 31 samples for 99%
Exposure null, orders 2–4	Mean 1.443 bits (theory 1.4426); $\Pr[\text{exposure} \ge 5] = 0.03125$ (bound 2^{-5}); order 8: all tied, exposure 0
Canary exposure	Table 13.5
Exposure estimates, order 3	Table 13.6
Extraction, order 6	14,852 distinct of 20,000 samples; 197 private lines; top 100 precision 0.45 by likelihood, 0.85 by ratio

tests/test_memorization.py checks that the output still matches.

13.6. Summary

Table 13.8.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 13.1, Definition 13.2	A string's probability is the product of its next-symbol probabilities	Every score of the chapter is a sum of per-symbol log-probabilities
Definition 13.3	Extractable, k-eidetic, discoverable, approximate	Discoverable is easy to measure; verbatim rates understate the leak
Proposition 13.1	$m \geq \ln(1/\beta)/q$ samples succeed with probability $1-\beta$	About $4.6/q$ samples for 99%
Lemma 13.1	A context of one line favours its symbol by $(k+\alpha)/\alpha$	One copy: 11 times as likely, 3.46 bits per digit
Theorem 13.1	k^* is the largest gap on the line's path plus one	The most contested context decides; larger models memorize from fewer copies; deduplication does not help singletons in a large model
Definition 13.4	$\log_2\lvert\mathcal{R}\rvert - \log_2 \text{rank}$	Bits of guessing saved; graded, unlike extraction
Theorem 13.2	$\Pr[\text{exposure} \geq h] \leq 2^{-h}$; mean $\to \log_2 e$	A p-value without shadow models
Definition 13.5, Proposition 13.2	Rank generations by $\Lambda(x)$	Never by raw likelihood; without the prefix an attacker gets less, but still thousands of records in production

13.7. References

- Carlini, Liu, Erlingsson, Kos & Song, "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks" (USENIX Security 2019): canaries, the exposure metric, its estimation by sampling and skew-normal extrapolation, and extraction of planted secrets by search.
- Carlini et al., "Extracting Training Data from Large Language Models" (USENIX Security 2021): extractable and k-eidetic memorization; the generate-rank-verify attack on GPT-2 with perplexity, reference-model, zlib and lowercase ranking.
- Carlini, Ippolito, Jagielski, Lee, Tramèr & Zhang, "Quantifying Memorization Across Neural Language Models" (ICLR 2023): memorization measured by prompting with training prefixes; log-linear growth with model size, duplication and prompt length.
- Nasr et al., "Scalable Extraction of Training Data from (Production) Language Models" (2023): discoverable against extractable memorization, measurement on open models against their training sets, and the divergence attack on an aligned chat model.
- Kandpal, Wallace & Raffel, "Deduplicating Training Data Mitigates Privacy Risks in Language Models" (ICML 2022): how the rate of regenerated sequences grows with duplication.

- Lee et al., “Deduplicating Training Data Makes Language Models Better” (ACL 2022): near-duplicate detection at scale and its effect on emitted training text and model quality.
- Ippolito et al., “Preventing Generation of Verbatim Memorization in Language Models Gives a False Sense of Privacy” (INLG 2023): approximate memorization and why verbatim filters fail.
- Feldman, “Does Learning Require Memorization? A Short Tale about a Long Tail” (STOC 2020): why accuracy on rare examples requires memorizing them.
- Jurafsky & Martin, *Speech and Language Processing* (3rd ed. draft), ch. 3: n-gram language models, smoothing and perplexity.

14. Gradient inversion

A phone keyboard learns from what its users type, but the raw text never leaves the phones. A server sends the current model θ to each phone, the phone computes the gradient of the loss on its own few examples, and only that gradient goes back. The server averages the gradients of many phones and takes a gradient step on the model. This is the setting of **federated learning (FL)**, and it is often described as private because "the data stay on the device". This chapter asks one question about it: **what does a single client's gradient reveal about the client's data?**

The answer is often everything, and one idea explains why. In a fully connected layer, each row of the weight gradient is the layer's input multiplied by a single number, the rate at which the loss changes with that row's output. The bias gradient is the list of those numbers, one per row. So with one example, dividing a row of the weight gradient by the matching entry of the bias gradient gives back the input exactly. The chapter takes four steps. First, one division recovers the input of one example. Second, the signs of the last layer's gradient give its label, because the true class is the only one whose entry is negative. Third, for layers that offer no such division, such as convolutions, **Deep Leakage from Gradients (DLG)** searches for an input whose gradient matches the one observed. Fourth, the chapter asks which of these attacks survive when the client averages over a batch.

The running example comes in two sizes. A network with three inputs and two hidden units is small enough to compute by hand, and the division gives back its input $(0.5, 1, 0.2)$ exactly from each of its two hidden units. The experiments then use synthetic 8×8 images and a network with 32 hidden units, whose gradient has 2,212 entries against 64 unknown pixels. There the division recovers an image to rounding error, DLG recovers one with about 13 correct digits after 600 steps, and in a batch of 4 the same division, applied at each unit that responds to only one image, still recovers all four images.

Table 14.1.: The symbols of this chapter.

Symbol	Meaning
$x \in \mathbb{R}^d$; y	an input with d pixels; its label, one of K classes
W_1, b_1; W_2, b_2	the weights and biases of the first and the second layer
a, h, z, p	pre-activations, hidden activations, logits and predicted probabilities
σ; m	the activation function; the number of hidden units
ℓ	the cross-entropy loss of one example
γ_1, γ_2	the backward signals $\partial \ell / \partial a$ and $\partial \ell / \partial z$

Symbol	Meaning
e_y; $\odot$; $A_{j,:}$	the one-hot vector of the label; the entrywise product; row j of a matrix A
B; g	the batch size; the gradient the client sends
R	the matching loss of Deep Leakage from Gradients

14.1. The reconstruction game

Before any attack can be judged, you need to know who sees what and what counts as success. This section fixes both for one round of federated training, in the vocabulary of Threat models and privacy games, and then fixes the network that every experiment uses.

The server is **honest-but-curious** in the sense of that chapter: it runs the protocol as specified and only analyses what it receives. Its access is the **transcript**. It sees each client's own update, here the gradient the client sends, and it knows the architecture and every parameter of θ because it chose them. The client's loss is the average of the losses of its examples, and the derivative of an average is the average of the derivatives, so the gradient it sends is the average of its per-example gradients.

Definition 14.1 (The gradient reconstruction game)**.** Fix a model θ, known to the server, and a population of labelled examples. The challenger, who holds the secret, is the client, and the adversary, who guesses it, is the server. The game has three steps.

1. The challenger draws a batch of B labelled examples $(x_1, y_1), \dots, (x_B, y_B)$ from the population. The inputs are the secret.
2. The client sends the gradient of its average loss with respect to all the parameters,

$$g = \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell(\theta; x_i, y_i). \tag{14.1}$$

3. The adversary sees (θ, g) and outputs reconstructions $\hat{x}_1, \dots, \hat{x}_B$.

Each true input x is matched to its closest reconstruction $\hat{x}$ in Euclidean distance (two inputs may share one), and the pair is scored by its **relative error** $\|\hat{x} - x\|/\|x\|$.

The relative error is 0 for a perfect reconstruction and 1 for a mistake as large as the input itself. The matching is needed because an average does not change when you reorder the examples, so the gradient carries no record of which example came first. The attacks below either recover the client's exact pixels or, for large batches, produce blends, images that mix several of the client's inputs.

All experiments use one small network, written in NumPy. Its first fully connected layer gives one number per hidden unit, the unit's **pre-activation**, and an activation is applied to each unit. A second fully connected layer turns the result into one score per class, called a **logit**. The **softmax** turns the logits into probabilities by exponentiating and normalizing, so that every probability is

positive and they sum to 1. The loss is the **cross-entropy**, minus the log of the probability the model gives to the true label, which is small when the model is confidently right.

Definition 14.2 (The network)**.** An input $x \in \mathbb{R}^d$ has a label y, one of K classes. It passes through a first layer with weights $W_1 \in \mathbb{R}^{m \times d}$ and biases b_1, one per hidden unit. An activation σ is applied to each of the m hidden units. A second layer with weights $W_2 \in \mathbb{R}^{K \times m}$ and biases b_2 gives one logit per class, and then

$$a = W_1 x + b_1, \quad h = \sigma(a), \quad z = W_2 h + b_2, \quad p = \operatorname{softmax}(z), \quad \ell = -\ln p_y, \tag{14.2}$$

where the softmax has entries

$$p_k = \frac{e^{z_k}}{\sum_{k'} e^{z_{k'}}}. \tag{14.3}$$

The **backward signals** are the rates at which the loss changes with each hidden unit's pre-activation and with each logit, $\gamma_1 = \partial \ell / \partial a$ and $\gamma_2 = \partial \ell / \partial z$.

The activation σ is either the ReLU $\max(a, 0)$, which passes positive values and replaces negative ones by 0, or the sigmoid $1/(1 + e^{-a})$, which squashes every value smoothly into $(0, 1)$.

Example 14.1. The client's inputs are synthetic 8×8 grey-scale images, so $d = 64$. Each shows one of $K = 4$ shapes (a horizontal bar, a vertical bar, a diagonal and a square outline), shifted at random and with pixel noise. The network has $m = 32$ hidden units and is at its random initialization, as in the first round of training. The first layer has $64 \cdot 32$ weights and 32 biases, and the second has $32 \cdot 4$ weights and 4 biases, so the network has

$$64 \cdot 32 + 32 + 32 \cdot 4 + 4 = 2{,}212 \tag{14.4}$$

parameters. The server receives 2,212 numbers about an example that has only 64 unknown pixels.

14.2. What a gradient is made of

Every attack in this chapter reads the input out of the gradient, so you first need to know where in the gradient the input sits. Two results answer this, one placing the input in every row of the weight gradient and one giving the backward signals that scale those rows.

The reason the input appears is the chain rule. The weight W_{jk} multiplies the input x_k and feeds only the output a_j, so the loss changes with it at the rate at which the loss changes with a_j, times x_k. So the gradient of a weight is the signal flowing backward into its output times the input flowing forward into it. Collected over all j and k, these products form a matrix whose (j, k) entry is $\gamma_j x_k$, called the **outer product** of γ and x.

Proposition 14.1 (The gradient of a fully connected layer is an outer product)**.** *Let a fully connected layer compute $a = Wx + b$ with $W \in \mathbb{R}^{m \times d}$, and let the loss ℓ depend on W and b only through a, with backward signal $\gamma = \partial \ell / \partial a$. Then*

$$\nabla_W \ell = \gamma \, x^\top, \qquad \nabla_b \ell = \gamma. \tag{14.5}$$

Proof.

1. *Each weight feeds one output.* Written out, output j is $a_j = \sum_k W_{jk} x_k + b_j$. The weight W_{jk} appears in a_j only, with coefficient x_k, so $\partial a_j / \partial W_{jk} = x_k$.
2. *The chain rule multiplies the two rates.* The loss depends on W_{jk} only through a_j, so

$$\frac{\partial \ell}{\partial W_{jk}} = \frac{\partial \ell}{\partial a_j} \frac{\partial a_j}{\partial W_{jk}} = \gamma_j x_k, \tag{14.6}$$

   which is the (j, k) entry of $\gamma x^\top$.
3. *The bias has coefficient 1.* The bias b_j enters a_j with coefficient 1, so the same rule gives $\partial \ell / \partial b_j = \gamma_j$.

□

Row j of the outer product is the input multiplied by the number γ_j. The signal γ depends on everything above the layer (the rest of the network, the loss, the label), but the factor $x^\top$ does not. So the rest of the network can only rescale the rows, never change their direction.

To use Proposition 14.1 at the first layer you need its backward signal γ_1, and the chain rule obtains it by sending the last layer's signal γ_2 back through the network. At the last layer the signal has a simple form. Raising the logit of a wrong class k raises the loss at the rate p_k, the probability the model already gives that class, and raising the logit of the true class lowers the loss at the rate $1 - p_y$. Each hidden unit then collects the logits' signals, weighted by its connections to them, and multiplies by the slope of its activation.

Proposition 14.2 (The backward signals)**.** *For the network of Definition 14.2, write e_y for the one-hot vector of the label, with 1 in position y and 0 elsewhere, and $\odot$ for the entrywise product of two vectors. Then*

$$\gamma_2 = p - e_y, \tag{14.7}$$

so the entry of γ_2 at the true label is negative and every other entry is positive, and

$$\gamma_1 = (W_2^\top \gamma_2) \odot \sigma'(a). \tag{14.8}$$

Proof. The plan is to differentiate the loss in the logits, read off the signs, and then pass the result back through the second layer and the activation.

1. *Remove the fraction.* Since $p_y = e^{z_y} / \sum_k e^{z_k}$, taking minus its log gives

$$\ell = -z_y + \ln \sum_k e^{z_k}. \tag{14.9}$$

2. *Differentiate in one logit.* The first term depends only on z_y, so it contributes -1 when $k = y$ and 0 otherwise. The second term is the log of a sum, whose derivative is the derivative of the sum, e^{z_k}, divided by the sum itself, which is p_k. Together,

$$\frac{\partial \ell}{\partial z_k} = p_k - \mathbf{1}[k = y], \tag{14.10}$$

   which is entry k of $p - e_y$.

3. *Read off the signs.* Every e^{z_k} is positive, so every p_k lies strictly between 0 and 1. The entry at the label is $p_y - 1 < 0$, and every other entry is $p_k > 0$.
4. *Pass back through the second layer.* Logit k is $z_k = \sum_j W_{2,kj} h_j + b_{2,k}$, so h_j enters each logit with coefficient $W_{2,kj}$, and the chain rule sums over the logits:

$$\frac{\partial \ell}{\partial h_j} = \sum_k \gamma_{2,k} W_{2,kj} = (W_2^\top \gamma_2)_j. \tag{14.11}$$

5. *Pass through the activation.* The output $h_j = \sigma(a_j)$ changes with a_j at the rate $\sigma'(a_j)$, so multiplying step 4 by it gives entry j of Equation 14.8.

□

For ReLU the slope is $\sigma'(a_j) = \mathbf{1}[a_j > 0]$, which is 1 if the unit is switched on and 0 if it is switched off. So a unit that is switched off on an input has $\gamma_{1,j} = 0$ and, by Proposition 14.1, a zero row in the first-layer weight gradient.

Example 14.2. Take $d = 3$ inputs, $m = 2$ ReLU units and $K = 2$ classes, with

$$W_1 = \begin{pmatrix} 0.4 & -0.2 & 0.1 \\ 0.3 & 0.5 & -0.4 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix}, \quad W_2 = \begin{pmatrix} 1 & -1 \\ 0.5 & 0.5 \end{pmatrix}, \quad b_2 = 0, \tag{14.12}$$

and the example $x = (0.5, 1, 0.2)$ with label $y = 1$, the first class. Table 14.2 walks through the forward pass, the backward pass and the gradient.

Table 14.2.: Forward pass, backward pass and gradient of a two-unit ReLU network, by Proposition 14.2 and Proposition 14.1. The server sees only the "Gradient" rows.

Step	Quantity	Value
Forward	$a = W_1 x + b_1$	$(0.2 - 0.2 + 0.02 + 0.1,\ 0.15 + 0.5 - 0.08 - 0.1) = (0.12, 0.47)$
	$h = \max(a, 0)$	$(0.12, 0.47)$, both units on
	$z = W_2 h$	$(0.12 - 0.47,\ 0.06 + 0.235) = (-0.35, 0.295)$
	$p = \mathrm{softmax}(z)$	$(0.3441, 0.6559)$
Backward	$\gamma_2 = p - e_y$	$(-0.6559, 0.6559)$
	$\gamma_1 = (W_2^\top \gamma_2) \odot \sigma'(a)$	$(1 \cdot (-0.6559) + 0.5 \cdot 0.6559,\ (-1) \cdot (-0.6559) + 0.5 \cdot 0.6559) = (-0.3279, 0.9838)$
Gradient	$\nabla_{W_1} \ell = \gamma_1 x^\top$	rows $(-0.1640, -0.3279, -0.0656)$ and $(0.4919, 0.9838, 0.1968)$
	$\nabla_{b_1} \ell = \gamma_1$	$(-0.3279, 0.9838)$

The first probability is $p_1 = e^{-0.35}/(e^{-0.35} + e^{0.295})$, and dividing top and bottom by $e^{-0.35}$ gives $p_1 = 1/(1 + e^{0.645}) = 1/(1 + 1.906) = 0.3441$, so $p = (0.3441, 0.6559)$. The two entries of γ_2 sum to 0, and only the one at the true class is negative. Both units are on, since both entries of a are positive, so γ_1 is just $W_2^\top \gamma_2$, and the two rows of the weight gradient are -0.3279 times x and 0.9838 times x.

14.3. Exact recovery from a fully connected layer

The server sees g. Is there any computation, without a search, that gives back the client's input? For a fully connected first layer with a bias, the most common kind, there is one division, which Phong et al. (2018) pointed out for this federated setting.

By Proposition 14.1, row j of the first-layer weight gradient is x scaled by $\gamma_{1,j}$, and entry j of the bias gradient is that same number, so dividing removes the scale.

Theorem 14.1 (Exact recovery from one example)**.** *Suppose the client sends the gradient of a single example, and the first layer is fully connected with a bias, $a = W_1 x + b_1$, whatever the layers above it and the loss. Then for every unit j with $(\nabla_{b_1}\ell)_j \neq 0$,*

$$x = \frac{(\nabla_{W_1}\ell)_{j,:}}{(\nabla_{b_1}\ell)_j}. \tag{14.13}$$

*Moreover, if $\gamma_1 \neq 0$, the matrix $\nabla_{W_1}\ell$ has **rank one**, and its row space is the line through x.*

Proof.

1. *Write out what the server sees.* Proposition 14.1 applied to the first layer gives $(\nabla_{W_1}\ell)_{j,:} = \gamma_{1,j}\, x^\top$ and $(\nabla_{b_1}\ell)_j = \gamma_{1,j}$.
2. *Divide one by the other.* When $\gamma_{1,j} \neq 0$, the ratio is $\gamma_{1,j}x^\top/\gamma_{1,j} = x^\top$. Nothing about the rest of the network enters, because the depth, the activations above, the loss and the label all sit inside the number $\gamma_{1,j}$, and that number cancels.
3. *Read off the rank.* All rows of $\nabla_{W_1}\ell = \gamma_1 x^\top$ are multiples of the same vector $x^\top$, and at least one is nonzero when $\gamma_1 \neq 0$. A matrix whose rows are all multiples of one nonzero vector has rank one, and its row space is spanned by that vector.

□

The attack is exact and needs no optimization, no prior knowledge of the data and no label. It also checks itself, because every usable unit must give the same answer.

Four things get in its way, and only one stops it fully. With ReLU, switched-off units give nothing, but one active unit is enough. Without a bias there is no denominator, but the rank-one structure still gives the direction of x, so only the scale remains unknown, one number that matching gradients as in Section 14.5 finds. A convolutional first layer really breaks the outer-product form. It applies the same small filter at every position of the image, so its gradient is a sum of outer products over all the image patches, and only the search of Section 14.5 can undo it. A batch of more than one example also breaks the formula, but only partly, as Section 14.6 shows.

Example 14.3. In the hand network of Example 14.2, the server never sees a, h, z or p. It sees only the gradient rows of Table 14.2. Dividing the first row by -0.3279 and the second by 0.9838 gives $(0.5, 1, 0.2)$ both times, which is the input.

On the images, Figure 14.1 shows the attack on one input. Of the 32 ReLU units, 13 are switched on and have a nonzero bias gradient. Every one of the 13 ratios equals the image to a relative error of at most 6.7×10^{-17}, which is rounding error. To check the rank, look at the **singular values**

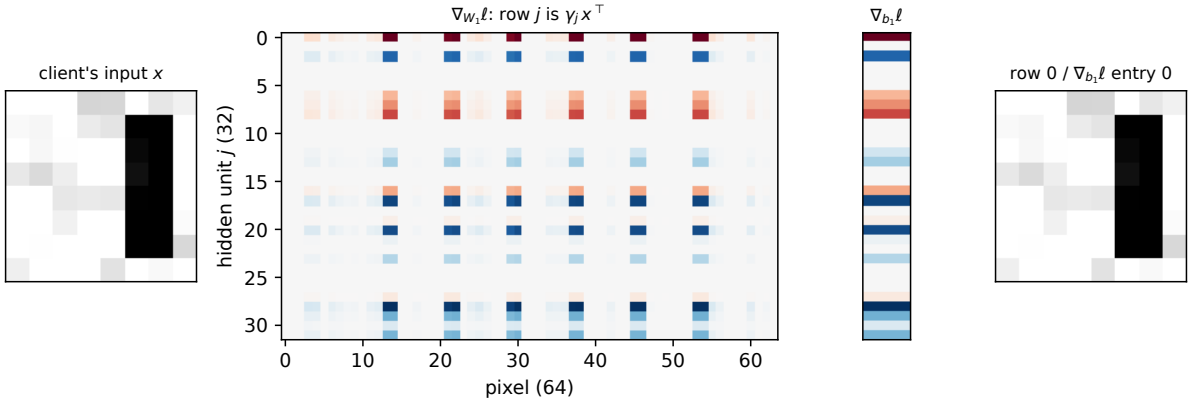


Figure 14.1.: Exact recovery from the first-layer gradient of a ReLU network with 32 hidden units. Left: the client's 8×8 input. Middle: $\nabla_{W_1}\ell$, one row per hidden unit and one column per pixel (blue positive, red negative), next to $\nabla_{b_1}\ell$. Right: one row divided by its bias entry, the input exactly.

of $\nabla_{W_1}\ell$. They measure how much of the matrix lies along each of its independent directions, so a rank-one matrix has exactly one that is not zero. The largest is 1.6155 and the second is 9.5×10^{-17} times the first, so the gradient is rank one, as Theorem 14.1 says.

14.4. Label recovery from the last layer

The gradient depends on the label too, and the label is often the most sensitive part: the diagnosis, the word typed next, the class of a photo. Zhao, Mopuri & Bilen (2020) noticed that the label can be read off the last layer in closed form. Their attack, iDLG ("improved DLG"), then searches only for the input, which makes the gradient-matching search of Section 14.5, below, easier and more reliable.

The next result gives three rules, one from the bias, one from the weights and one for batches. All three come from Proposition 14.2, which says that the last layer's backward signal is a vector of probabilities with 1 subtracted at the true label. A batch gradient averages these vectors. Before the division by the batch size, the subtracted 1s in each position add up to the number of examples of that class.

Proposition 14.3 (Labels from the last layer)**.** *Let the network of Definition 14.2 keep its last-layer bias b_2. For a single example, the label is the position of the only negative entry of the last-layer bias gradient,*

$$y = \arg\min_k (\nabla_{b_2}\ell)_k. \tag{14.14}$$

If in addition the hidden activations satisfy $h \geq 0$ and $h \neq 0$, as for ReLU or the sigmoid, then row y of $\nabla_{W_2}\ell$ is the only row whose entries are all non-positive, and the only row with a negative sum.

For a batch of B examples, write n_k for the number of examples of class k and p_{ik} for the probability the model gives class k on example i. Then

$$n_k = \sum_{i=1}^{B} p_{ik} - B\,(\nabla_{b_2}\ell)_k. \tag{14.15}$$

Proof.

1. *The bias rule.* Proposition 14.1 at the last layer gives $\nabla_{b_2}\ell = \gamma_2$, and Proposition 14.2 gives $\gamma_2 = p - e_y$, with entry $p_y - 1 < 0$ at the label and $p_k > 0$ elsewhere. The smallest entry is therefore at y.
2. *The weight rule.* Proposition 14.1 gives $(\nabla_{W_2}\ell)_{k,:} = \gamma_{2,k}\,h^\top$. With $h \geq 0$, row y is a negative number times non-negative entries, so it is non-positive entrywise, and every other row is $p_k h^\top \geq 0$. Summing a row multiplies $\gamma_{2,k}$ by $\sum_j h_j$, which is positive because $h \geq 0$ and $h \neq 0$, so the sum keeps the sign of $\gamma_{2,k}$.
3. *The counts.* The batch gradient is the average of the per-example ones, so $\nabla_{b_2}\ell = \frac{1}{B}\sum_i (p_i - e_{y_i})$. The vector e_{y_i} has a 1 in position k exactly when example i has class k, so $\sum_i e_{y_i}$ counts the examples of each class, and entry k is

$$(\nabla_{b_2}\ell)_k = \frac{1}{B}\Big(\sum_i p_{ik} - n_k\Big). \tag{14.16}$$

   Multiplying by B and solving for n_k gives Equation 14.15.

□

For one example the bias rule makes no errors, at any point of training, because the sign argument never depends on how good the model is. A confident model gives a tiny negative entry, which floating point hides only when p_y rounds to 1 and the entry becomes 0. That is one reason attacks are run on models early in training. Without a bias, and even for an activation such as tanh whose outputs can be negative, step 2 of the proof still shows that row y of $\nabla_{W_2}\ell$ is a negative multiple of h and every other row a positive one, so the label's row points the opposite way.

For a batch, the gradient leaks the label histogram. Equation 14.15 is exact, so the only unknown is the total prediction $\sum_i p_{ik}$, and the attacker can estimate it well. If it holds data of its own from a similar population, which this attack assumes, it can run the model on them and average the predictions into a guess $\bar{p}$, then replace the total by $B\bar{p}_k$ and round. The true counts are integers, so rounding gives them exactly whenever the estimate of the total is off by less than $\frac{1}{2}$ for every class. The cruder guess $\bar{p}_k = 1/K$ of evenly spread predictions does worse, because even an untrained network favours some classes over others.

Example 14.4. In the hand network of Example 14.2, the last-layer bias gradient is $\gamma_2 = (-0.6559, 0.6559)$. Its smallest entry is the first, so the server reads off the label 1. On 1,000 random images and random networks, both the bias rule and the weight rule name the label correctly in a fraction 1.000 of the trials.

For a batch, take 16 images with true counts $(4, 2, 4, 6)$ over the four shapes. Table 14.3 reads them off the bias gradient of the 32-unit ReLU network, with three guesses for the total prediction in Equation 14.15.

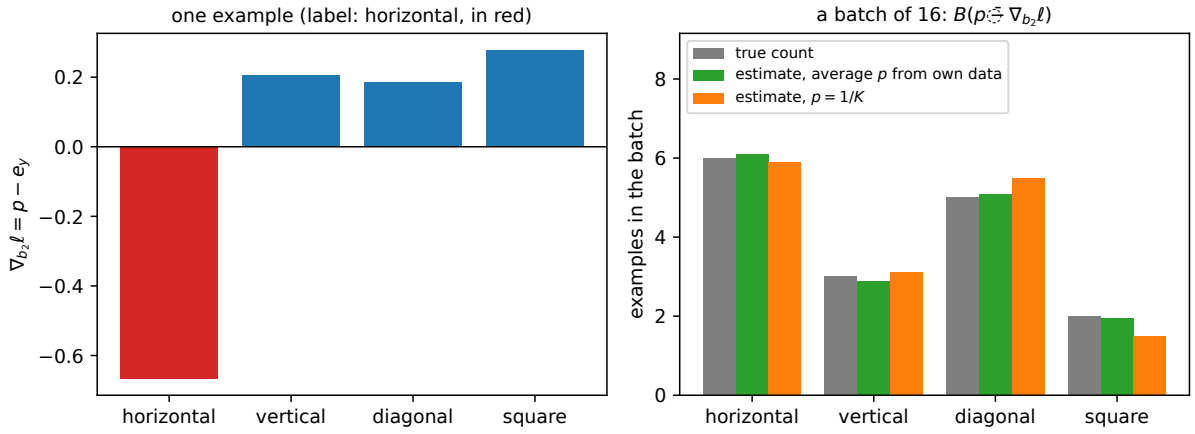


Figure 14.2.: Labels from the last-layer bias gradient. Left: for one example, $\nabla_{b_2}\ell = p - e_y$ has a single negative entry, at the true label (red). Right: for a batch of 16, the counts $B(\bar{p} - \nabla_{b_2}\ell)$ with $\bar{p}$ the model's average prediction on the attacker's own images (green) are close to the true counts (grey); with $\bar{p} = 1/K$ (orange) they are off by up to about half an example.

Table 14.3.: Label counts of one batch of 16 recovered from $\nabla_{b_2}\ell$ by Equation 14.15, under three guesses of the summed predictions.

Guess of $\sum_i p_{ik}$	Estimated counts	Rounded	All correct?
Exact, $\sum_i p_{ik}$ (not available to the attacker)	$(4.00, 2.00, 4.00, 6.00)$	$(4, 2, 4, 6)$	yes
$B\bar{p}$, $\bar{p}$ averaged over 200 of the attacker's own images	$(3.88, 2.00, 4.19, 5.92)$	$(4, 2, 4, 6)$	yes
$B/K = 4$ for every class	$(2.12, 3.01, 4.27, 6.60)$	$(2, 3, 4, 7)$	no

With the averaged guess every estimate is within $\frac{1}{2}$ of the truth, the largest miss being $4.19 - 4 = 0.19$, so rounding fixes it. With the uniform guess the first class is off by $4 - 2.12 = 1.88$, and rounding cannot help.

Over 200 random batches and networks, the averaged guess gets all the rounded counts right in 1.000 of batches of 4 and 0.975 of batches of 16. For batches of 64 the rate falls to 0.525, and with the uniform guess the three rates are 0.925, 0.095 and 0.000. The rates fall because the error of the estimate is a sum of B deviations, one per example, and that sum grows with B. At $B = 16$ the largest error over the four classes, before rounding, has a median over batches of 0.233 with the averaged guess, against 1.102 with the uniform one.

14.5. Deep Leakage from Gradients

The closed forms need a fully connected layer with a bias. Most image models start with convolutions, and many architectures have layers that rescale or mix their inputs, such as normalization and attention, or no biases at all. Zhu, Liu & Han (2019) gave a generic attack that needs none of these: **guess an input, compute its gradient, and move the guess until the gradients match**. Zhu et al. also searched for a soft label, a probability vector standing in for the unknown label, and that joint search often failed. With the label fixed by Equation 14.14, the search is over the input only.

DLG measures how badly a guess matches by one number, the squared distance between the gradient the guessed batch would produce and the observed gradient, summed over all the gradient's entries. It is zero for a perfect match and positive otherwise.

Definition 14.3 (Deep Leakage from Gradients)**.** Given the model θ, the observed gradient g and the labels $y_1, \dots, y_B$, write $G(x') = \frac{1}{B}\sum_i \nabla_\theta \ell(\theta; x'_i, y_i)$ for the gradient that a guessed batch $x' = (x'_1, \dots, x'_B)$ would produce. The **matching loss** is $R(x') = \|G(x') - g\|^2$, and DLG outputs

$$\hat{x}_1, \dots, \hat{x}_B = \arg\min_{x'_1,\dots,x'_B} R(x'), \qquad R(x') = \Big\| \frac{1}{B}\sum_{i=1}^{B} \nabla_\theta \ell(\theta; x'_i, y_i) - g \Big\|^2, \tag{14.17}$$

searching by a gradient method that starts from random noise and repeatedly moves the guess a little downhill on R.

For a batch, the label rules of Proposition 14.3 give only the counts of each class, not which example has which label. That is enough, because the guesses are interchangeable: the attacker hands out the recovered labels to the guesses in any order, and each guess is then pulled towards a true example with its label.

Two facts make this a sensible search. The true batch scores the best value the matching loss can take, and the slope of the matching loss can be computed, as long as the network can be differentiated twice. The second fact needs the second derivatives because G is itself made of first derivatives of the network.

Proposition 14.4 (The true batch is a global minimizer)**.** *The true batch $(x_1, \dots, x_B)$ has $R = 0$, the smallest value R takes. If the network is twice differentiable, then R is differentiable in the guess, with*

$$\nabla_{x'} R = 2\, J_G(x')^\top (G(x') - g), \tag{14.18}$$

*where J_G is the **Jacobian** of G, the matrix of rates at which each gradient entry changes with each input pixel.*

Proof.

1. *The true batch is a global minimizer.* R is a squared length, so $R \geq 0$ for every guess. Plugging in the true inputs reproduces g exactly, so $R = 0$ there, and no guess can do better.
2. *The slope of a squared length.* A squared length $\|u\|^2$ changes with u at the rate $2u$. Here $u = G(x') - g$, and g does not depend on the guess.

3. *The chain rule passes the slope back to the guess.* u changes with x' at the rates collected in $J_G(x')$, so the chain rule multiplies the transpose of that matrix by $2u$, which is Equation 14.18. The entries of J_G are derivatives of the gradient, so they are second derivatives of the network, and they exist because the network is twice differentiable.

□

The need for second derivatives is why Zhu et al. replaced ReLU by the sigmoid, and why the experiments here do the same. The gradient G contains the factor $\sigma'(a)$ of Equation 14.8, and the search differentiates G, so it differentiates that factor too. The ReLU's slope is 0 or 1, constant on each side of zero, so the factor does not change as the guess moves, and the part of the slope of R that would flow through it is zero. The sigmoid's slope varies smoothly, so that part is not lost.

Nobody builds J_G itself, which has one row per parameter and one column per pixel. Automatic-differentiation libraries instead differentiate the gradient computation, running backpropagation through the backward pass, which is called **double backpropagation**. That needs only a twice-differentiable network, not a fully connected first layer, so DLG also works for convolutional networks and on real images. Zhu et al. recover pictures from the CIFAR-100 image dataset and text tokens from single-example gradients.

Proposition 14.4 does not say that the search finds the true batch. R is not convex in the guess, and for such a function Optimization shows that a gradient method with a suitable step only drives the slope to zero. That gives a **stationary point**, where the surface is flat, which may or may not be the global minimum, so success is an empirical question. The final value of R tells you which case you are in: $R = 0$ up to rounding means an exact match, while a value stuck well above zero means the search is stuck, and restarts or image priors, penalties that favour natural-looking images, may help.

The minimizer is also plausibly unique, since matching g imposes one equation per parameter, 2,212 here against $64B$ unknown pixels, though that count is not a proof. For a single example the sigmoid network still has a fully connected first layer with a bias, so Theorem 14.1 shows that its gradient determines the input. Geiping et al. (2020) improved the search by comparing gradients by a cosine distance and penalizing differences between neighbouring pixels, and so recovered recognizable images from trained networks and from batches.

Example 14.5. The chapter's script (Section 14.8) runs DLG on one image with the sigmoid network of 32 hidden units, computing the slope of Equation 14.18 by double backpropagation written out by hand. A finite-difference check, which nudges one pixel slightly and sees how much R changes, agrees with it to within 1.2×10^{-10}. The search starts from uniform noise and runs 600 steps of the Adam optimizer, with learning rate 0.05 decaying linearly to 0. Adam is a variant of gradient descent that scales each pixel's step by a running estimate of the size of that pixel's gradient. The label comes from Equation 14.14.

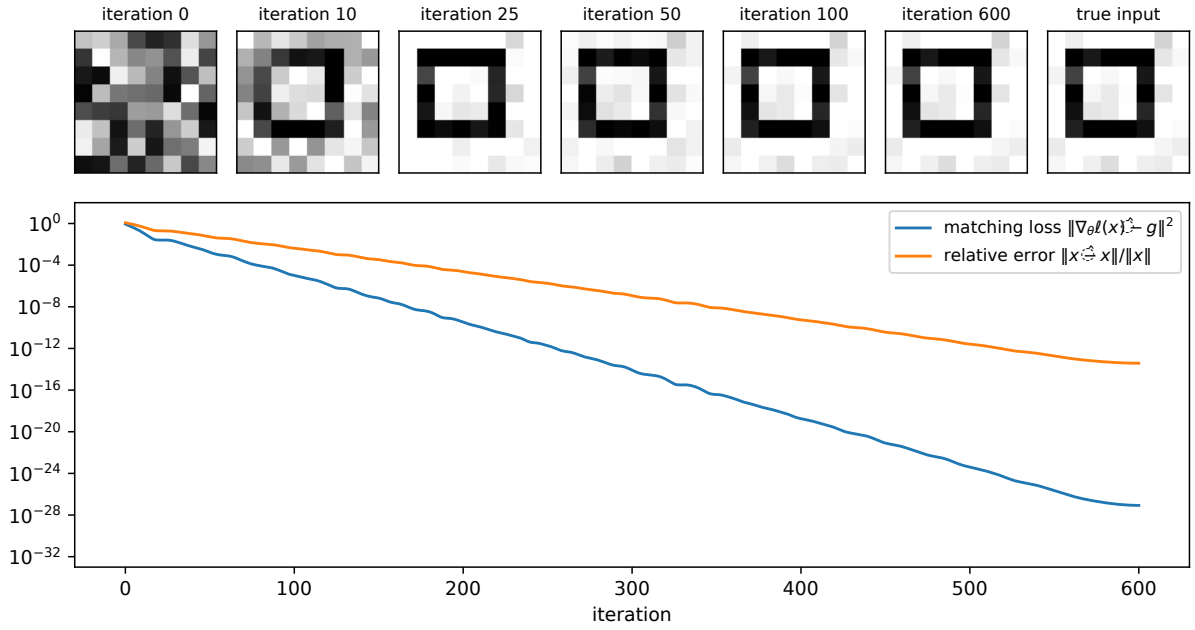


Figure 14.3.: Deep Leakage from Gradients on one 8×8 image, sigmoid network with 32 hidden units. Top: the reconstruction $\hat{x}$ at selected iterations of the Adam optimizer (Example 14.5), starting from uniform noise, and the true input. Bottom: the matching loss $R(\hat{x})$ and the relative error, on a log scale. Both fall geometrically until they reach rounding error.

Table 14.4.: Deep Leakage from Gradients on one image, sigmoid network with 32 hidden units: relative error of the reconstruction and the matching loss over the iterations.

Iteration	0	10	50	100	200	400	600
Relative error	1.6	0.67	7.5×10^{-2}	5.7×10^{-3}	3.4×10^{-5}	7.8×10^{-10}	5.8×10^{-14}
Matching loss R	1.3	0.20	2.5×10^{-3}	1.3×10^{-5}	4.6×10^{-10}	2.5×10^{-19}	1.3×10^{-27}

By iteration 50 the image is visually correct, with 7.5% error, and by iteration 600 it is exact to about 13 digits (Figure 14.3). The matching loss falls about as the square of the error, which is what you should expect close to the answer. Near the true input x the map G is nearly linear, so the mismatch is about the Jacobian times the error, and its squared length is quadratic in the error:

$$R(x') \approx \|J_G(x)(x' - x)\|^2. \tag{14.19}$$

In the table, between iterations 50 and 100 the error shrinks by a factor $0.075/0.0057 \approx 13$ and R by $0.0025/0.000013 \approx 190$, close to $13^2 \approx 170$.

14.6. Batches and averaging

Clients rarely send the gradient of one example. They send the average over a batch, or the change in parameters after several local training steps, which the end of this section treats. Averaging is often said to protect the individual examples, and this section measures that claim. Averaging does destroy the one-line formula of Equation 14.13, and it does make DLG harder. But the span of the batch survives, and so does every example to which some hidden unit responds alone.

The structure of a batch gradient follows from Proposition 14.1. Each example contributes one outer product, whose rows are multiples of its own input, so every row of the sum is a weighted combination of the B inputs.

Proposition 14.5 (The gradient of a batch)**.** *Take a batch of B examples and a fully connected first layer, and write $\gamma_{1,i}$ for the backward signal of example i at that layer, with entries $\gamma_{1,ij}$, one per unit j. Then*

$$\nabla_{W_1}\ell = \frac{1}{B}\sum_{i=1}^{B} \gamma_{1,i}\, x_i^\top, \tag{14.20}$$

so $\nabla_{W_1}\ell$ has rank at most B, and every row lies in $\operatorname{span}\{x_1, \dots, x_B\}$. For every unit j with $(\nabla_{b_1}\ell)_j \neq 0$, the unit ratio is a weighted combination of the inputs,

$$\frac{(\nabla_{W_1}\ell)_{j,:}}{(\nabla_{b_1}\ell)_j} = \sum_{i=1}^{B} \frac{\gamma_{1,ij}}{\sum_{i'}\gamma_{1,i'j}}\, x_i^\top, \tag{14.21}$$

with weights that sum to 1 (some may be negative). It equals x_i exactly when $\gamma_{1,ij} \neq 0$ and $\gamma_{1,i'j} = 0$ for every $i' \neq i$.

Proof.

1. *A sum of outer products.* The batch gradient is the average of the per-example gradients, and by Proposition 14.1 each of those is $\gamma_{1,i} x_i^\top$.
2. *Rank and span.* Stack the $x_i^\top$ as the rows of a matrix X with B rows, and the $\gamma_{1,i}^\top$ as the rows of a matrix Γ. The sum of outer products is then the product $\frac{1}{B}\Gamma^\top X$. A product has rank at most that of either factor, and X has only B rows, so the rank is at most B. Row j of the product is $\frac{1}{B}\sum_i \Gamma_{ij} x_i^\top$, a combination of the inputs, so it lies in their span.
3. *The ratios.* By Proposition 14.1 again, entry j of the bias gradient is $\frac{1}{B}\sum_i \gamma_{1,ij}$. Dividing row j by it cancels the factors $\frac{1}{B}$ and gives Equation 14.21. The weights are each example's share of the same total, so they sum to 1, and if only $\gamma_{1,ij}$ is nonzero the weight on x_i is 1 and all the others are 0.

□

For ReLU the condition for exact recovery, that only example i has a nonzero signal at unit j, has a simple form. By Proposition 14.2, $\gamma_{1,i'j} = 0$ whenever unit j is switched off on $x_{i'}$, so a unit that is switched on for example i and for no other example gives back x_i exactly. Such a unit **isolates** example i, and Section 14.7 asks how often that happens.

The span leaks even when nothing is isolated. When the gradient has rank B, as it does for the batch of 4 below, its rows span exactly the span of the inputs. Every example is then a combination of B directions the attacker can read off the gradient, so a search such as DLG can look for each example as B unknown coefficients of those directions instead of d unknown pixels. That is 4 unknowns instead of 64 here, or of hundreds of thousands for real images.

Local steps do not help either. If the client takes several steps of stochastic gradient descent (SGD) on its batch and sends the total change in parameters, only the backward signals change from step to step. For one example and a learning rate η, step s changes W_1 by $-\eta\gamma_1^{(s)} x^\top$ and b_1 by $-\eta\gamma_1^{(s)}$, where $\gamma_1^{(s)}$ is that step's backward signal. Summed over the steps, the change in W_1 is the vector $-\eta\sum_s \gamma_1^{(s)}$ times $x^\top$, still rank one, and the change in b_1 is that same vector. So the scale still cancels and Equation 14.13 still recovers x exactly.

Example 14.6. A batch of 4 images through the 32-unit ReLU network gives a first-layer gradient whose singular values, relative to the largest, are 1.0, 0.84, 0.69, 0.039 and then 1.5×10^{-16}. Four directions are nonzero and the rest are rounding error, so the rank is 4, the most Proposition 14.5 allows. Every row lies in the span of the four images to within 4.6×10^{-16}, and the unit ratios recover all 4 of the 4 images exactly.

DLG does much worse on batches of the same size. Table 14.5 runs it on the sigmoid network it needs for 6 random networks and batches of each size.

Table 14.5.: Deep Leakage from Gradients on batches, sigmoid network with 32 hidden units, 600 Adam steps, 6 trials per batch size.

B	Median relative error	Examples recovered (error < 0.05)	Median final matching loss
1	0.000	1.00	1.5×10^{-27}
2	0.000	0.67	1.3×10^{-27}
4	0.363	0.29	3.3×10^{-5}

B	Median relative error	Examples recovered (error < 0.05)	Median final matching loss
8	0.711	0.00	7.7×10^{-5}

Batches of one and two are recovered to rounding error in most runs. From $B = 4$ the median run gets stuck, with the matching loss near 10^{-5} instead of 10^{-27}, and the reconstructions are blends of the batch (Figure 14.4). Two facts suggest that the search, not the information, is what fails. At $B = 4$ there are only $64 \cdot 4 = 256$ unknown pixels against 2,212 equations, though that count is not a proof. And on a ReLU network of the same width, which also averages four examples, the unit ratios recovered all four images, so averaging four examples does not by itself hide them.

14.7. How often a unit isolates an example

Every isolated example comes out exactly, so the share of a batch that a gradient exposes is the share isolated by at least one unit. How does that share depend on the batch size and on the width of the layer? To answer, you need a model of when units switch on, and the simplest one treats each unit's response to each example as a fair coin.

For a single unit, the coin model is exact in one case. Suppose the first-layer bias is zero and the inputs are drawn from a distribution that is symmetric about zero, such as $\mathcal{N}(0, I)$. Unit j, with weights $(W_1)_{j,:}$, is on for x_i when $(W_1)_{j,:}\, x_i > 0$. With the weights fixed, symmetry makes this product equally likely to be positive or negative, it is exactly zero with probability 0, and independent inputs make the signs independent across the batch. So each unit's on/off pattern over the batch is a sequence of fair coin flips. Different units look at the same inputs, though, so independence between units is only approximate, and real data are not symmetric at all.

Proposition 14.6 (Isolation under independent units)**.** *Take a ReLU first layer with m units and a batch of B examples. Suppose that each unit is on for each example with probability $\frac{1}{2}$, independently across examples and across units, and that every unit j active on an example i passes on a nonzero backward signal, $(W_2^\top \gamma_{2,i})_j \neq 0$, where $\gamma_{2,i}$ is the last-layer signal of example i. That fails only by coincidence. Then each example of the batch is recovered exactly by some unit ratio with probability*

$$P_{iso}(B, m) = 1 - (1 - 2^{-B})^m. \tag{14.22}$$

Proof.

1. *One unit.* By Proposition 14.5 and the ReLU slope, unit j gives back x_i exactly when it is on for x_i and off for each of the other $B - 1$ examples, since an active unit has a nonzero signal by hypothesis. Each of these B events has probability $\frac{1}{2}$, and they are independent, so their probability together is 2^{-B}.
2. *Many units.* The example is missed only if all m units miss it. The units are independent by hypothesis, so this has probability $(1 - 2^{-B})^m$, and subtracting from 1 gives Equation 14.22.

□

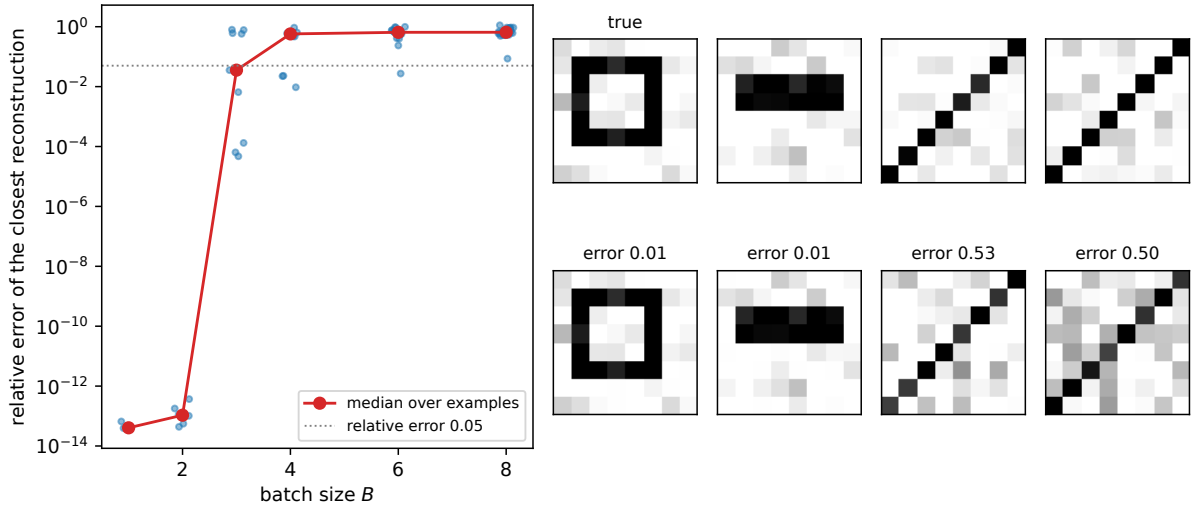


Figure 14.4.: Deep Leakage from Gradients against batch size, sigmoid network with 32 hidden units. Left: relative error of the closest reconstruction for every example (dots) and the median (red), on a log scale; the dotted line marks an error of 0.05. Right: a batch of 4 (top) and its closest reconstructions (bottom), some exact and some blends.

The formula pits an exponential in B against a count that grows linearly in m. A unit isolates a given example with probability 2^{-B}, so among m units the expected number that isolate it is $m2^{-B}$, and almost every example is isolated when m is much larger than 2^B. To find the width that gives an even chance, require the miss probability $(1-2^{-B})^m$ to be at most $\frac{1}{2}$ and take logs. The log of $1-2^{-B}$ is negative, so dividing by it flips the inequality and gives

$$m \geq \frac{\ln 2}{-\ln(1-2^{-B})}. \tag{14.23}$$

At $B=10$ the denominator is $-\ln(1-2^{-10}) = 0.000977$, close to 2^{-10} itself, and the bound is $0.6931/0.000977 \approx 709.4$, so you need $m \geq 710$ units. Real first layers have thousands of units, so if units behaved like these coins, batches of 10 would not be safe from this attack.

Data whose examples resemble one another, such as images of a few shapes, are easier for the attacker, not harder. With such images, a unit tends to fire on many examples of a batch or on few. The number of examples it fires for is more spread out than the coin model predicts, which statisticians call **overdispersion**. In particular some units fire only on rare inputs, and when such a unit fires on a large batch it usually fires on just one example, so isolation is much more common at large B. Take one extreme model, in which each unit has its own firing rate r, drawn uniformly between 0 and 1. A unit with rate r isolates a given example with probability $r(1-r)^{B-1}$, and averaging over r gives $\int_0^1 r(1-r)^{B-1}\,dr = 1/(B(B+1))$. That shrinks only like $1/B^2$ instead of halving with each extra example.

The game of this chapter has an honest-but-curious server. A malicious server does better still, because it chooses θ to attack rather than at random. It can plant first-layer units that fire on rare inputs only (“trap weights”, Boenisch et al. 2023), or that sort inputs into bins by a feature such as brightness (Fowl et al. 2022), so that most examples of even a large batch are isolated by construction.

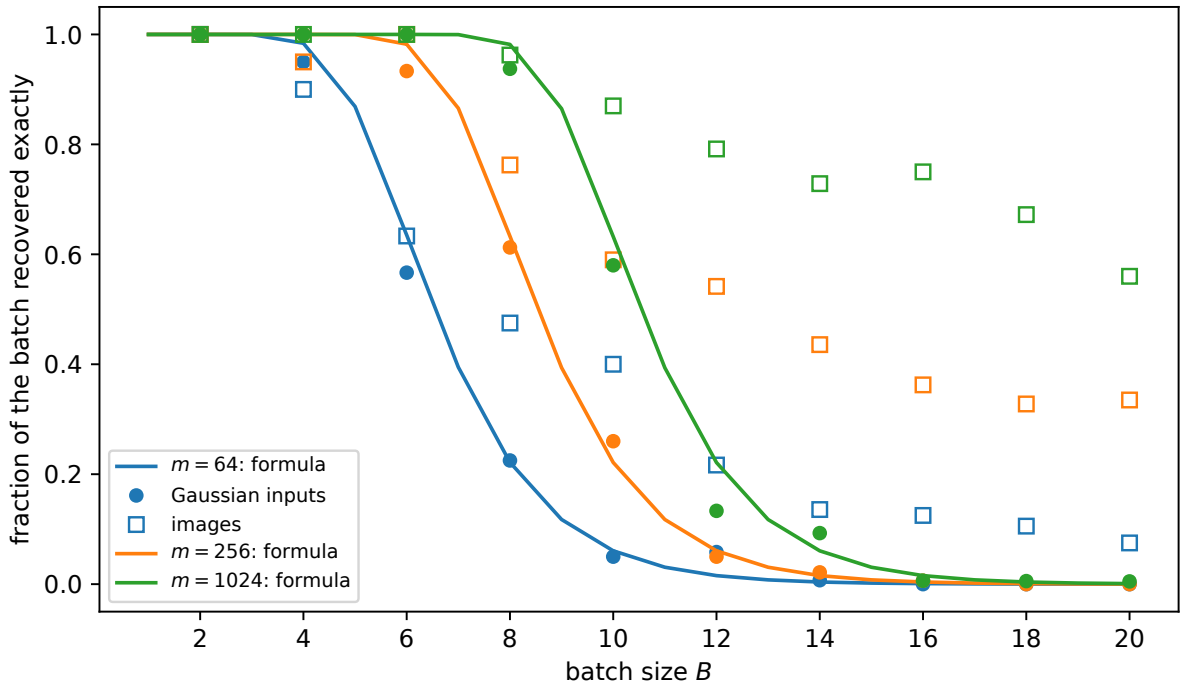


Figure 14.5.: Fraction of a batch recovered exactly by first-layer unit ratios, for ReLU networks with $m = 64$, 256 and 1024 hidden units. Lines: Equation 14.22. Filled circles: Gaussian inputs and zero first-layer bias. Open squares: the 8×8 images, where isolation decays much more slowly.

Example 14.7. For the batch of 4 in Example 14.6, the network has $m = 32$ units and $1 - 2^{-4} = 15/16$, so Equation 14.22 predicts

$$P_{\text{iso}}(4, 32) = 1 - (15/16)^{32} = 1 - e^{32 \ln(15/16)} = 1 - e^{-2.065} = 0.873 \tag{14.24}$$

per example under independence. All 4 of the 4 images were recovered. Table 14.6 compares Equation 14.22 with simulation over a range of widths and batch sizes, with 20 random networks and batches per cell (Figure 14.5 shows the full range).

Table 14.6.: Fraction of batch examples recovered exactly from first-layer unit ratios of a random ReLU network, against Equation 14.22.

m	B	Equation 14.22	Gaussian inputs	Images
64	4	0.984	0.975	0.838
64	8	0.222	0.188	0.550
64	16	0.001	0.000	0.144
256	8	0.633	0.562	0.769
256	12	0.061	0.046	0.542
256	16	0.004	0.006	0.375
1024	12	0.221	0.196	0.850
1024	16	0.016	0.009	0.691

For Gaussian inputs the formula matches to within the simulation noise, overestimating slightly in seven of the eight rows because units see the same inputs, so they are not quite independent. For images the formula is far too reassuring. At $B = 16$ it predicts that 1.6% of the batch is exposed in the 1024-unit network, and 69.1% is.

At $B = 4$, by contrast, the images are recovered less often than the formula predicts, 0.838 against 0.984 for $m = 64$. Overdispersion cuts both ways. The same spread that makes "on for exactly one image" more common at large B also makes "on for all four images" and "on for none" common at small B, and units like that isolate nothing.

14.8. Code

The script code/part02-attacks/gradient_inversion.py computes every number in this chapter and draws every figure. From the repository root:

```
uv run python code/part02-attacks/gradient_inversion.py
```

Table 14.7.: The script's output with the default seed. Here s_1 and s_2 are the two largest singular values of the first-layer gradient.

Experiment	Result
Hand example (Table 14.2)	$p = (0.3441, 0.6559)$, $\gamma_1 = (-0.3279, 0.9838)$; both unit ratios give $(0.5, 1, 0.2)$; label 1 (index 0 in the script)
Exact recovery, one image, ReLU, $m = 32$	13 usable units of 32; largest relative error 6.7×10^{-17}; $s_2/s_1 = 9.5 \times 10^{-17}$; 2,212 gradient entries for 64 pixels
Labels, one example, 1,000 trials	Bias rule 1.000; weight rule 1.000
Label counts, all exact, 200 batches	Averaged guess: $B = 4$: 1.000, $B = 16$: 0.975, $B = 64$: 0.525. Uniform guess: 0.925, 0.095, 0.000
DLG, one image (Table 14.4)	Relative error 5.8×10^{-14} after 600 steps; matching loss 1.3×10^{-27}
Double backpropagation against finite differences	Largest gap 1.2×10^{-10}
DLG on batches (Table 14.5)	Recovered: $B = 1$: 1.00, $B = 2$: 0.67, $B = 4$: 0.29, $B = 8$: 0.00
Batch of 4, ReLU	Rank 4; rows in the span to 4.6×10^{-16}; 4 of 4 images isolated
Width for isolation probability $\frac{1}{2}$ at $B = 10$	710

tests/test_gradient_inversion.py checks that the output still matches.

14.9. Summary

Table 14.8.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 14.1	An honest-but-curious server sees (θ, g) and outputs reconstructions, scored by relative error	Every attack here needs no malice
Proposition 14.1	$\nabla_W \ell = \gamma x^\top$ and $\nabla_b \ell = \gamma$ for a fully connected layer	A fully connected layer's input sits in its gradient
Proposition 14.2	$\gamma_2 = p - e_y$ and $\gamma_1 = (W_2^\top \gamma_2) \odot \sigma'(a)$	The sign pattern for labels; a switched-off ReLU unit has a zero row
Theorem 14.1	One example: $x = (\nabla_{W_1} \ell)_{j,:} / (\nabla_{b_1} \ell)_j$, and $\nabla_{W_1} \ell$ has rank one	The input costs one division; fails for a convolutional first layer

Result	Statement	Use
Proposition 14.3	$y = \arg\min_k (\nabla_{b_2} \ell)_k$; $n_k = \sum_i p_{ik} - B(\nabla_{b_2} \ell)_k$	The label costs one sign; batch counts are exact for small batches
Definition 14.3, Proposition 14.4	Minimize $R(x') = \|G(x') - g\|^2$; the true batch has $R = 0$	Needs only a twice-differentiable network; a final R well above zero means a stuck search
Proposition 14.5	$\operatorname{rank} \nabla_{W_1} \ell \leq B$, rows in the span of the batch; unit ratios are weighted combinations	The span and isolated examples still leak
Proposition 14.6	$P_{\text{iso}} = 1 - (1 - 2^{-B})^m$ for independent units	Most of a batch is exposed once $m \gg 2^B$; real data more often

14.10. References

- Zhu, Liu & Han, "Deep Leakage from Gradients" (NeurIPS 2019). The gradient-matching attack of Section 14.5, on images and text.
- Zhao, Mopuri & Bilen, "iDLG: Improved Deep Leakage from Gradients" (arXiv 2020). Label recovery from the sign of the last-layer gradient, and DLG with the label fixed.
- Phong, Aono, Hayashi, Wang & Moriai, "Privacy-Preserving Deep Learning via Additively Homomorphic Encryption" (IEEE Transactions on Information Forensics and Security, 2018). The division of Equation 14.13.
- Geiping, Bauermeister, Dröge & Moeller, "Inverting Gradients — How easy is it to break privacy in federated learning?" (NeurIPS 2020). Cosine matching with an image prior, on trained networks, local steps and batches.
- Fowl, Geiping, Czaja, Goldblum & Goldstein, "Robbing the Fed: Directly Obtaining Private Data in Federated Learning with Modified Models" (ICLR 2022). A malicious server's binning units isolate most of a large batch.
- Boenisch, Dziedzic, Schuster, Shamsabadi, Shumailov & Papernot, "When the Curious Abandon Honesty: Federated Learning Is Not Private" (IEEE EuroS&P 2023). Trap weights that isolate examples of large batches.
- Wainakh et al., "User-Level Label Leakage from Gradients in Federated Learning" (PETS 2022). Label counts from batch gradients, as in Equation 14.15.
- Huang, Gupta, Song, Li & Arora, "Evaluating Gradient Inversion Attacks and Defenses in Federated Learning" (NeurIPS 2021). How the attacks depend on known labels, normalization statistics and untrained models.
- McMahan, Moore, Ramage, Hampson & Agüera y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data" (AISTATS 2017). Federated averaging.

15. Linkage and re-identification

A hospital shares its discharge records with researchers, after deleting the names and keeping ZIP code, date of birth, sex and diagnosis. Is anyone in the file identifiable? Usually, yes, because most people are the only person with their combination of ZIP code, birth date and sex, so anyone who holds a voter roll with names and those three columns can join the two tables and name most rows. This is a **linkage attack**, and putting a name back on a "de-identified" record is **re-identification**. Unlike the rest of this part, it attacks the data itself rather than a trained model.

The one idea of the chapter is that identifying information is measured in bits, and bits add up. Picking one person out of n takes $\log_2 n$ bits, because each bit halves the number of candidates, so 27.9 bits single out one of the 248.7 million people in the United States in 1990, the census year of the classic study below. An attribute with 12 equally likely values, such as a birth month, cuts the candidates twelvefold and so carries $\log_2 12 \approx 3.6$ bits. When attributes are independent their cuts multiply, so their bits add. **Once the attributes you know carry a few bits more than $\log_2 n$, almost everyone is unique.**

The running example is a synthetic census of $n = 10{,}000$ people with eight attributes, which an adversary learns one at a time. Once the known attributes carry 12.90 bits, an average over people that Section 15.4 defines, a third of the people are already unique, even though that is below the $\log_2 n = 13.29$ bits needed to name one person, because skewed attributes leave some people in rare combinations. At 20.39 bits 97.5% are unique.

15.1. Quasi-identifiers and the linkage attack

Data protection practice sorts a table's columns into three kinds. **Direct identifiers**, such as a name, name a person on their own, while **sensitive attributes**, such as a diagnosis, are what must not be tied to a person. Between them sit **quasi-identifiers**, such as a birth date. Each names nobody alone, but together they often single out one person, and they appear next to names in other datasets. "De-identification" usually removes only the direct identifiers. This section scores a join on the quasi-identifiers with a game in the style of Threat models, in which a challenger hides a secret and an adversary guesses it.

Definition 15.1 (The linkage game)**.** A release R holds n records, each with quasi-identifiers and a sensitive attribute. Records with identical quasi-identifiers form an **equivalence class**, and the J classes have sizes $k_1, \dots, k_J$. A challenger draws a target uniformly from the n records and gives the adversary the target's quasi-identifiers, as a voter roll would. The adversary outputs one record of R and wins if it is the target's. The **linkage attack** outputs a record drawn uniformly from the target's class.

A target in a class of k look-alikes is found one time in k, but it lands in that class k times as often as in a class of one, so each class adds the same $1/n$ to the success, whatever its size.

Proposition 15.1 (Linkage success)**.** *In the game of Definition 15.1, the linkage attack wins with probability*

$$\Pr[win] = \frac{1}{n} \sum_{j=1}^{J} k_j \cdot \frac{1}{k_j} = \frac{J}{n}, \tag{15.1}$$

the number of distinct quasi-identifier combinations divided by the number of records. No adversary that sees only the target's quasi-identifiers does better.

Proof.

1. *Condition on the target's class.* A target in class j shares its quasi-identifiers with $k_j - 1$ other records, so picking one of the k_j at random wins with probability $1/k_j$.
2. *Nothing beats this.* Within a class, the target is equally likely to be any of the k_j records, because it was drawn uniformly and the adversary sees only quasi-identifiers, which are identical within the class. So every guess within the class wins with probability exactly $1/k_j$. A guess outside the class never wins, since its quasi-identifiers differ from the target's.
3. *Average over the target.* The target lands in class j with probability k_j/n, so the success is $\sum_j \frac{k_j}{n} \cdot \frac{1}{k_j} = \frac{J}{n}$.

□

Each unique record is a sure win, so the fraction of unique records is a lower bound on the success. Conversely, if every class has at least k records, then n records fill at most n/k classes, so $J \leq n/k$ and the success is at most $1/k$, the idea that Syntactic anonymity studies.

The game assumes the adversary knows that the target is in the release. Otherwise a unique match may belong to someone who is not in R, and whether that doubt protects anyone is the question of Section 15.6.

Example 15.1. Table 15.1 is a hospital release with names removed. Its last column shows the name a public voter roll gives each combination of the three quasi-identifiers.

Table 15.1.: A de-identified hospital release, three quasi-identifiers and one sensitive attribute, joined with part of a voter roll.

ZIP code	Birth date	Sex	Diagnosis	Name on the voter roll
02138	1961-03-14	F	asthma	Ana Silva
02138	1972-11-02	M	diabetes	Ben Cole
02138	1972-11-02	M	hypertension	Ben Cole
02139	1958-06-21	F	depression	Carla Diaz
02139	1990-01-01	F	migraine	
02139	1990-01-01	F	asthma	
02139	1990-01-01	F	fracture	
02141	1983-09-30	M	pneumonia	Dan Eto

The join gives Ana's asthma, Carla's depression and Dan's pneumonia with certainty, because each of them matches exactly one record. Ben matches two records, his own and a stranger's with the same quasi-identifiers, so the adversary is right about him with probability $\frac{1}{2}$. The three records born on 1990-01-01 count too, since a target drawn from them is found one time in three, though this part of the roll names none of them. With $J = 5$ classes for $n = 8$ records, Equation 15.1 gives a success of $5/8 = 0.625$.

This is, in miniature, how the computer scientist Latanya Sweeney re-identified the governor of Massachusetts in the late 1990s: in the Cambridge voter roll, six people shared his birth date, three of them were men, and only one lived in his ZIP code.

Sweeney (2000) then estimated from 1990 census tables that 87% of the US population were likely unique on {5-digit ZIP code, date of birth, sex}. The bits explain why (Figure 15.1). Treating each attribute as uniform gives an upper bound on its bits, because equally likely values are the hardest to guess, so about 40,000 ZIP codes, $365 \times 80 = 29{,}200$ birthdays in about 80 birth years, and two sexes carry at most

$$\log_2 40{,}000 + \log_2 29{,}200 + 1 = 15.3 + 14.8 + 1.0 = 31.1 \text{ bits}, \tag{15.2}$$

against the 27.9 bits needed to single out one American, which leaves room for most people to be unique.

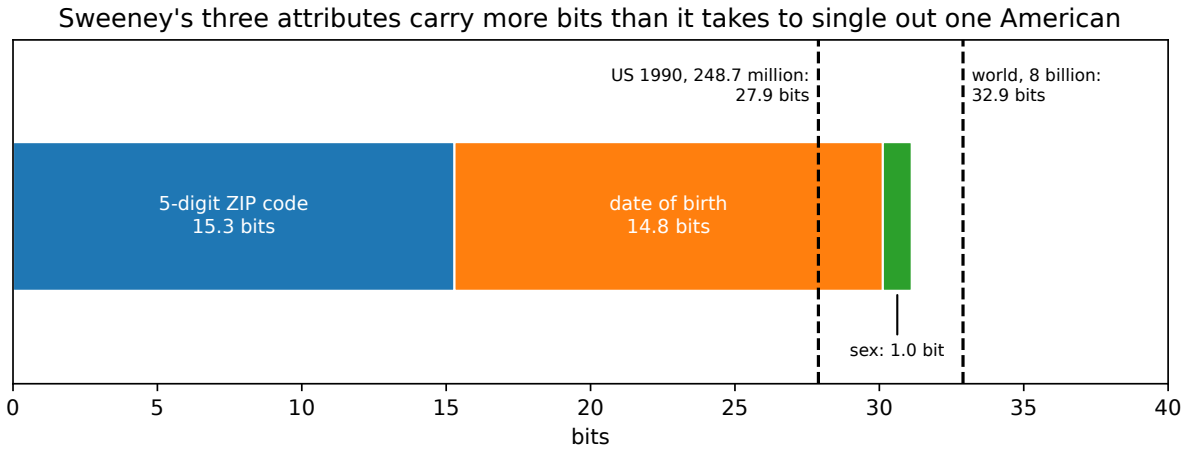


Figure 15.1.: The bits of the three classic quasi-identifiers, upper bounds that treat each as uniform, against the bits needed to single out one person in the US of 1990 and in the world.

15.2. Pseudonyms and hashed identifiers

Deleting an identifier also deletes the ability to follow one person through the data. A common alternative is to replace the identifier by a code that stays the same on all of that person's records, often its MD5 hash, a standard 128-bit hash. This section shows two separate ways in which the code fails to hide the person.

Definition 15.2 (Pseudonyms)**.** A **pseudonym** replaces an identifier x by a code that is the same on every record of the same person. A **hashed pseudonym** is $h(x)$ for a public hash function h that anyone can compute. A **keyed hash** mixes a secret key into h, so that only the key holder can compute it.

A good hash cannot be run backwards, so the way to invert it is to hash candidate inputs until one matches, and an identifier with a public format can be guessed by listing every value. Even a pseudonym that nobody can invert has a second weakness, because all of one person's records carry it.

Proposition 15.2 (Two ways a pseudonym fails)**.** *Suppose the identifiers come from a public set of N values.*

1. *If the pseudonyms are $h(x)$ for a public hash with 128-bit outputs that behaves like a random function, giving each input an independent uniform output, an adversary who computes h on all N values recovers the identifier behind every pseudonym. It fails only if two identifiers share a hash, which has probability at most $N^2/2^{129}$. The cost is N hash computations for the whole release, which doubles with each bit of the identifier.*
2. *Whatever the pseudonyms, an adversary who links any one record to a person learns that person's pseudonym, and with it every record that carries it.*

Proof.

1. *Part 1: the adversary can list every pseudonym.* With no secret key, the possible pseudonyms are the values $h(x)$ over the N identifiers, which take N computations. The adversary keeps a table from each value back to its identifier.
2. *Each pseudonym maps back to one identifier.* Two identifiers with the same hash would make the lookup ambiguous. The birthday bound, Theorem 8.3, says that N values of a random function with 2^{128} outputs collide with probability at most $N^2/(2 \cdot 2^{128}) = N^2/2^{129}$.
3. *Part 2: a consistent pseudonym groups a person's records.* It is the same on every record of one person, so learning it from one record identifies all of them.

□

Hashing therefore protects only identifiers that are already hard to guess. Phone, licence and identity numbers carry between about 20 and 33 bits (ten-digit phone numbers carry 33.2), and a laptop computes millions of MD5 hashes a second, so it lists any of these sets in seconds to hours. A keyed hash stops the enumeration, but not the second failure. Keeping a person's rows together, the reason to use a pseudonym, makes them one long record, which Section 15.5 shows to be identifying.

Example 15.2. In 2014 a freedom-of-information request obtained every New York City taxi trip of 2013, about 173 million trips, with each medallion number, the permit number of a cab, and each driver's licence number replaced by its MD5 hash. One hashed licence seemed to earn thousands of dollars a day. Its code, `cfcd208495d565ef66e7dff9f98764da`, is the MD5 of the string "0", as the script of Section 15.7 confirms, the placeholder that every trip with a missing licence carried, so the codes were plain MD5 with no key.

Vijay Pandurangan then inverted all the pseudonyms, as part 1 of Proposition 15.2 predicts. Licence numbers have six digits, or seven starting with 5, about 2 million possibilities or 20.9 bits, and medallion numbers, which follow a few fixed patterns of letters and digits, about 22 million or 24.4 bits. He hashed every possibility in minutes. In miniature, the script hashes all 10^6 six-digit numbers, 19.9 bits' worth, and recovers five secret identifiers.

Passengers had no pseudonym, yet each trip is itself a record of rare details, of the kind Section 15.5 studies. A pickup minute (one of 525,600 in the year) carries 19.0 bits and a pickup square of 100 metres (one of 78,000) carries 16.3 bits, together at most 35.3 bits, more than the 27.4 bits that single out one trip among 173 million. Anthony Tockar showed this in 2014 with photographs of celebrities entering taxis. And by part 2 of Proposition 15.2, a passenger who finds their own trip and knows who drove it learns the driver's pseudonym, and with it the year's trips and income.

15.3. How many people are unique

A **custodian**, the organization that holds and releases data, can count the unique records in its own table, but not in the population behind it, nor for columns an adversary might combine from elsewhere. So it needs a model that predicts the count. It is the birthday problem of Cryptography basics asked the other way round: how many people share their cell with *nobody*?

Definition 15.3 (Cells and uniqueness)**.** A population has n people, and a release may cover all of them. A person's combination of attribute values is their **cell**. Under the **population model**, each person falls in cell c with probability p_c, independently of everyone else, and K is the number of cells. A person is **unique** when nobody else is in their cell, and U is the number of unique people.

A person is unique when all the others avoid their cell, and averaging that chance over the person's own cell gives the expected fraction unique.

Theorem 15.1 (Expected fraction unique)**.** *Under the population model of Definition 15.3, the expected fraction of unique people is*

$$\frac{\mathbb{E}[U]}{n} = \sum_{c} p_c (1 - p_c)^{n-1}. \tag{15.3}$$

Proof. The plan is to write U as a sum of one 0/1 counter per person, find the chance that one person is unique, and add.

1. *Count with indicators.* U is the sum over people of $\mathbf{1}$[person i is unique]. The mean of a sum is the sum of the means even when the terms depend on each other (Lemma 1.1), and the mean of an indicator is its probability. So $\mathbb{E}[U]$ is the sum of the n probabilities, which are equal because everyone draws a cell from the same probabilities.
2. *Condition on the person's own cell.* Call it c, which has probability p_c.
3. *Ask the others to stay away.* Given c, the person is unique if each of the other $n-1$ people lands outside c. They are independent and each avoids c with probability $1 - p_c$, so this has probability $(1 - p_c)^{n-1}$.

4. *Average over the cell.* One person is unique with probability $\sum_c p_c(1-p_c)^{n-1}$, and by step 1, $\mathbb{E}[U]$ is n times this.

□

When all K cells are equally likely, the formula is a single power, close to $e^{-n/K}$ because $1-p$ is close to e^{-p} when p is small, and the proof of Corollary 15.1, after the next lemma, needs to know how close.

Lemma 15.1 (A power and its exponential)**.** *For $0 \le p < 1$ and any exponent $\ell \ge 0$,*

$$e^{-p\ell/(1-p)} \le (1-p)^{\ell} \le e^{-p\ell}. \tag{15.4}$$

Proof.

1. *Take logarithms.* The power becomes $\ell \ln(1-p)$, so the claim is $-\frac{p}{1-p} \le \ln(1-p) \le -p$.
2. *The upper bound.* The logarithm is concave and $x-1$ is its tangent line at $x=1$, so $\ln x \le x-1$ for every $x>0$. With $x = 1-p$ this is $\ln(1-p) \le -p$.
3. *The lower bound.* With $x = \frac{1}{1-p}$ the same inequality gives $\ln \frac{1}{1-p} \le \frac{1}{1-p} - 1 = \frac{p}{1-p}$, and flipping the sign gives $\ln(1-p) \ge -\frac{p}{1-p}$.

□

Corollary 15.1 (Uniform cells)**.** *If all K cells are equally likely, write $b = \log_2 K$ for the bits a cell carries. The expected fraction unique is $(1-1/K)^{n-1}$, and it equals the exponential below up to a relative error of order $1/K + n/K^2$,*

$$\frac{\mathbb{E}[U]}{n} \approx e^{-n/K} = \exp\big(-2^{\log_2 n - b}\big). \tag{15.5}$$

Proof.

1. *Substitute uniform cells.* With $p_c = 1/K$, Equation 15.3 has K equal terms $\frac{1}{K}(1-\frac{1}{K})^{n-1}$, which add up to $(1-\frac{1}{K})^{n-1}$.
2. *Replace the power by an exponential.* Lemma 15.1 with $p = 1/K$ and $\ell = n-1$ puts the power between $e^{-(n-1)/(K-1)}$ and $e^{-(n-1)/K}$. These exponents differ by $\frac{n-1}{K(K-1)}$, of order n/K^2, and replacing $n-1$ by n moves the exponent by only $1/K$. Two exponentials whose exponents differ by a small amount u differ by a factor $e^u \approx 1+u$, a relative error of u.
3. *Write the exponent in bits.* The average occupancy is $n/K = 2^{\log_2 n}/2^b = 2^{\log_2 n - b}$.

□

For uniform cells, then, the fraction unique depends only on $b - \log_2 n$, the bits beyond those needed to name one person, and the curve is the same for a town and for the planet (Figure 15.2, right). Setting $\exp(-2^{\log_2 n - b})$ equal to a fraction f and solving gives $b - \log_2 n = -\log_2 \ln(1/f)$, so half the people are unique once b exceeds $\log_2 n$ by 0.53 bits, 90% at 3.25 bits more and 99% at 6.64 bits more.

Real attributes are skewed, and skew usually lowers uniqueness, because many people crowd into a few common cells while many rare cells stay empty. This is not a theorem. With 10 equally

likely cells and 100 people, nobody is unique, but making one cell rare lets the few people in it be unique.

Example 15.3. Sweeney's three attributes have $K = 40{,}000 \times 29{,}200 \times 2 = 2.34 \times 10^9$ cells for $n = 248.7$ million people, so $n/K = 0.106$ people per cell. If everyone fell into a uniformly random cell, Equation 15.5 says that a fraction $e^{-0.106} = 0.899$ would be unique. Sweeney's 87% is a little lower because ZIP codes and birth years are not equally common. Golle (2006) repeated the estimate with the 2000 census and found 63%, and both estimates depend on how they spread birth dates within age groups, since the census publishes only the groups.

The often-quoted "33 bits identify anyone" is the point $b = \log_2 n$. With 2^{33} uniform cells for 8 billion people, $n/K = 8 \times 10^9/2^{33} = 0.93$, and only a fraction $e^{-0.93} = 0.394$ is unique; 99% needs $32.9 + 6.64 = 39.5$ bits. The running census is skewed, so uniform cells misjudge it, and Section 15.4 compares it with the exact formula.

15.4. Uniqueness from marginals and from one population

Theorem 15.1 needs every cell probability, and there can be billions of cells, while a custodian often knows only how common each value of each attribute is. Its answer is also an average over imagined populations, while the custodian has one. This section closes both gaps, with a floor from the per-attribute frequencies and a bound on how far one population strays.

The floor uses the chance that two random people share a cell. Both land in cell c with probability $p_c \cdot p_c$, so this **collision probability** is $\sum_c p_c^2$, a measure of how crowded a typical cell is.

Proposition 15.3 (A floor from the collision probability)**.** *Under the population model, for any cell probabilities,*

$$\frac{\mathbb{E}[U]}{n} \geq \Big(1 - \sum_c p_c^2\Big)^{n-1}. \tag{15.6}$$

*If a cell is a combination of independent attributes, and attribute j takes the value v with probability $p_j(v)$, then $\sum_c p_c^2 = \prod_j \sum_v p_j(v)^2$. So the floor needs only each attribute's frequencies, its **marginal**.*

Proof. The plan is to write Equation 15.3 as the average of a convex function, move the average inside the function, and factorize what is left.

1. *Write the fraction as an average.* Draw a random person and note the probability of the cell they landed in. That number is random, and it equals p_c with probability p_c. With $g(x) = (1-x)^{n-1}$, Equation 15.3 is the average of g at that probability.
2. *Check that g bends upwards.* Its second derivative, $(n-1)(n-2)(1-x)^{n-3}$, is never negative on $[0, 1]$ when $n \geq 2$, so g is convex there.
3. *Apply Jensen's inequality.* A convex curve lies above its tangent line at the average point, so the average of g is at least the average of the tangent line. A line's average is its value at the average point, which is g of the average.
4. *Compute the inner average.* The average cell probability is $\sum_c p_c \cdot p_c = \sum_c p_c^2$, which gives Equation 15.6.

5. *Factorize for independent attributes.* Then $p_c = \prod_j p_j(c_j)$, where c_j is the value the cell gives attribute j. Expanding the product of sums gives one term per choice of a value for each attribute, that is, one term per cell, and that term is $\prod_j p_j(c_j)^2 = p_c^2$:

$$\prod_j \sum_v p_j(v)^2 = \sum_c \prod_j p_j(c_j)^2 = \sum_c p_c^2. \tag{15.7}$$

□

Step 3 is an equality when all cells with positive probability are equally likely, so the floor is tight for uniform cells. It is loose when a few crowded cells push the collision probability up while people in rare cells stay unique.

The **entropy** $H = \sum_c p_c \log_2(1/p_c)$ is the average number of bits a cell carries, and it adds over independent attributes, by the chain rule of Information theory, Theorem 5.2. The entropy rule $e^{-n2^{-H}}$ replaces the cells by 2^H equally likely ones. It is not a bound in either direction, but it is often closer to the exact value than the floor.

The second gap is that the custodian has one population, in which no single person moves the fraction unique by much. McDiarmid's inequality, Theorem 2.8, makes this a bound. If changing input i alone moves a function of independent inputs by at most d_i, it strays more than a margin $\gamma > 0$ above (or below) its mean with probability at most $\exp(-2\gamma^2 / \sum_i d_i^2)$.

Proposition 15.4 (One population is enough)**.** *Under the population model, for every margin* $\gamma > 0$*,*

$$\Pr\left[\left|\frac{U}{n} - \frac{\mathbb{E}[U]}{n}\right| \geq \gamma\right] \leq 2e^{-n\gamma^2/2}. \tag{15.8}$$

Proof.

1. *Change one input.* U is a function of the n independent cells. Moving one person from cell u to cell u' changes only the counts of those two cells.
2. *Bound the change.* A cell holds a unique person exactly when its count is 1. The move can switch that on or off in each of the two cells, so U changes by at most 2 and U/n by at most $d_i = 2/n$.
3. *Apply McDiarmid.* With $d_i = 2/n$ for all n people, $\sum_i d_i^2 = 4/n$, so each tail is at most $\exp(-2\gamma^2 n/4) = e^{-n\gamma^2/2}$. The two tails give the factor 2.

□

Setting the right side equal to a failure probability β says that the fraction unique is within $\sqrt{2\ln(2/\beta)/n}$ of its expectation with probability $1 - \beta$. The bound is loose, because it charges every person the worst case, while most moves change U by 0 or 1.

Example 15.4. The running census has eight independent attributes, with region and occupation skewed. Table 15.2 reveals them one at a time and compares the exact fraction unique of Equation 15.3 with simulation and three approximations.

Table 15.2.: Fraction of 10,000 people who are unique as attributes are revealed: exact, simulated (mean of 20 populations), and three approximations. $\log_2 n = 13.29$ bits.

Attributes known	Bits H	Cells K	Exact	Simulated	Collision bound	Entropy rule	Uniform cells
sex, age band, region	9.31	1,280	0.0158	0.0160	0.0000	0.0000	0.0004
+ birth month	12.90	15,360	0.3328	0.3332	0.0834	0.2692	0.5215
+ marital status	14.60	61,440	0.5789	0.5797	0.4237	0.6690	0.8498
+ children	16.48	368,640	0.7939	0.7918	0.7443	0.8967	0.9732
+ occupation	20.39	7,372,800	0.9747	0.9737	0.9736	0.9928	0.9986
+ blood group	22.57	58,982,400	0.9925	0.9926	0.9924	0.9984	0.9998

With 20.39 bits, $20.39 - 13.29 = 7.1$ bits beyond $\log_2 n$, 97.5% of people are unique, a little below the 99% that uniform cells give at 6.64 bits beyond, because skew lowers uniqueness. For the same reason, uniform cells overstate uniqueness once birth month is known, by up to $0.8498 - 0.5789 = 0.27$.

Simulation and the exact formula agree to within 0.003. For $\beta = 0.05$, Equation 15.8 gives a half-width of $\sqrt{2 \ln 40/10^4} = 0.0272$, about five times the largest standard deviation over the 20 simulated populations, 0.0053, so the bound is safe but loose.

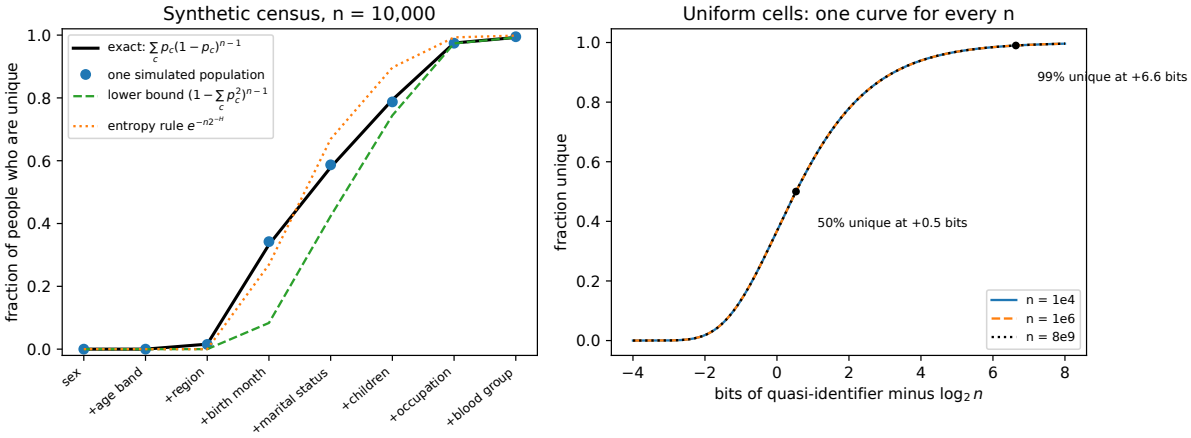


Figure 15.2.: Left: the fraction unique in the synthetic census as attributes are added, exact and simulated, with the collision floor and the entropy rule. Right: with uniform cells, the curves for $n = 10^4$, 10^6 and 8×10^9 against the bits beyond $\log_2 n$ coincide.

15.5. Sparse, high-dimensional data

In 2006 Netflix released about 100 million movie ratings by 480,189 subscribers, with names replaced by random numbers. Each record is a set of (movie, rating, date) triples, 209 on average out of 17,770 movies. There are no columns to join on, and what an adversary knows about a person is partial and noisy. Narayanan & Shmatikov (2008) still matched Netflix records to named reviews on the Internet Movie Database. This section asks how to match noisy information to a record, and when to trust the match.

No record matches exactly, so the attack scores how well each record agrees with what the adversary knows, item by item. An agreement on a film that half the subscribers rated says little, while one on an obscure film says a lot, so rarer items get larger weights.

Definition 15.4 (The Narayanan–Shmatikov score)**.** The adversary's auxiliary information aux is a set of items i, such as films, with approximate values aux_i, such as ratings. For a record r with value r_i on item i, the agreement $\text{Sim}(\text{aux}_i, r_i)$ is 1 if r rated i within one star of aux_i, and 0 otherwise. With a weight w_i for each item, the **score** of the record is

$$\text{Score}(\text{aux}, r) = \sum_{i \in \text{aux}} w_i \, \text{Sim}(\text{aux}_i, r_i). \tag{15.9}$$

Narayanan and Shmatikov use $w_i = 1/\ln|\text{supp}(i)|$, where the **support** $|\text{supp}(i)|$ is the number of records that rated item i. Their attack also scores dates, which this chapter leaves out.

The weights can be derived. For each record, the adversary weighs "this record is the target" against "this record is someone unrelated" by the **likelihood ratio**, the probability of what it observed under the first explanation divided by that under the second. For independent items its logarithm is a sum over items, the form of Equation 15.9.

Proposition 15.5 (The score is a log-likelihood ratio)**.** *Suppose an unrelated record agrees with item i of the auxiliary information with probability s_i, and the target's record with probability $1-\rho$, independently across items, where ρ covers wrong or stale information. Then the logarithm of a record's likelihood ratio is its score Equation 15.9 with the weights*

$$w_i = \ln\frac{1-s_i}{s_i} + \ln\frac{1-\rho}{\rho} \approx \ln\frac{1}{s_i} + \ln\frac{1-\rho}{\rho}, \tag{15.10}$$

plus a constant that is the same for every record; the approximation holds for rare items. If also the target is one of the n records, each equally likely a priori, and records are independent given which is the target, so that the patterns of all records multiply, then record t is the target with posterior probability

$$\Pr[\textit{target} = t \mid \textit{data}] = \frac{\text{LR}(t)}{\sum_j \text{LR}(j)}, \tag{15.11}$$

where $\text{LR}(t)$ is the likelihood ratio of record t. So the top-scoring record is the most probable target.

Proof. The plan is to write a record's pattern of agreements under both explanations, take the log of the ratio, drop the terms shared by all records, and apply Bayes' rule.

1. *Record the agreements.* Write $a_i(r) \in \{0,1\}$ for whether record r agrees with the auxiliary information on item i.
2. *The pattern under each explanation.* If r is the target, each item agrees with probability $1-\rho$, independently, so the pattern has probability $P_1(r) = \prod_i (1-\rho)^{a_i}\rho^{1-a_i}$, the exponents picking the right factor. If r is unrelated, it has probability $P_0(r) = \prod_i s_i^{a_i}(1-s_i)^{1-a_i}$.
3. *Take the log of the ratio.* The logarithm turns the ratio of products into a sum over items:

$$\ln \frac{P_1(r)}{P_0(r)} = \sum_i \Big[a_i \ln \frac{1-\rho}{s_i} + (1-a_i) \ln \frac{\rho}{1-s_i} \Big]. \tag{15.12}$$

4. *Keep what depends on the record.* Collecting the terms with a_i leaves $\sum_i a_i \big[\ln \frac{1-\rho}{s_i} - \ln \frac{\rho}{1-s_i} \big]$ plus $\sum_i \ln \frac{\rho}{1-s_i}$, which is the same for every record. Splitting each logarithm of a ratio turns the bracket into $\ln \frac{1-s_i}{s_i} + \ln \frac{1-\rho}{\rho}$, which is Equation 15.10.
5. *Bayes' rule.* If t is the target, record t's pattern follows P_1 and every other record's follows P_0, so all the patterns together have probability $P_1(t) \prod_{j \neq t} P_0(j)$. The prior is the same for every t, so the posterior of t is proportional to this. Dividing by $\prod_j P_0(j)$, which does not depend on t, leaves $\mathrm{LR}(t) = P_1(t)/P_0(t)$, and normalizing gives Equation 15.11.

□

For rare items these weights are about $\ln(1/s_i)$. They and the Narayanan–Shmatikov weights both grow as an item gets rarer, at different rates, and Example 15.5 compares the two. For one record, "target or unrelated" is a hypothesis test, and by the Neyman–Pearson lemma, Theorem 4.1, thresholding the likelihood ratio is the most powerful test.

Because the score adds up evidence, noise costs little: with a wrong item the target loses one term, while unrelated records rarely gain one. What decides success is the bits, and each item contributes its **surprisal** $\log_2(1/s_i)$, the bits that single out an event of probability s_i.

Proposition 15.6 (False matches and bits)**.** *Take reliable information ($\rho \to 0$) with exact agreement required. If an unrelated record agrees on each item i of a set I independently with probability s_i, the expected number of the n records that agree on all of I by chance is*

$$n \prod_{i \in I} s_i = 2^{\log_2 n - \sum_{i \in I} \log_2(1/s_i)}. \tag{15.13}$$

Proof.

1. *One record.* By independence, an unrelated record agrees on every item of I with probability $\prod_{i \in I} s_i$.
2. *All records.* The expected number of false matches is the sum of these probabilities over records (Lemma 1.1), n times step 1.
3. *Write it in bits.* Writing $n = 2^{\log_2 n}$ and $s_i = 2^{-\log_2(1/s_i)}$ turns the product into a power of 2.

□

So false matches vanish once the items' bits exceed $\log_2 n$, as in Section 15.3. A film that 1% of subscribers rated carries $\log_2 100 = 6.64$ bits, and three of them carry $3 \times 6.64 = 19.9$ bits, more than the $\log_2 480{,}189 = 18.9$ bits that single out one Netflix subscriber. This is why sparsity is the weakness: most items are rated by few people, so each one a person rated is worth many bits.

The top score is the best guess *if the target is in the release*. If it may not be, the adversary must also decide whether to trust the guess, and a real match should stand out from the other scores.

Definition 15.5 (The eccentricity test)**.** Let $\text{top}_1 \geq \text{top}_2$ be the two largest scores and σ the standard deviation of all the scores. The **eccentricity** is the gap between them in units of σ, and the eccentricity test declares a match to the top record only if it clears a threshold τ,

$$\frac{\text{top}_1 - \text{top}_2}{\sigma} \geq \tau, \tag{15.14}$$

and otherwise says "no match". Narayanan and Shmatikov used $\tau = 1.5$.

How often the top score stands out by chance depends on the shape of the score distribution, so τ has to be calibrated by running the test for people known to be absent from the release, at a small false positive rate (Hypothesis testing). On Netflix, Narayanan and Shmatikov found that 8 ratings, 2 of them wrong, with dates known to within 14 days (their score uses dates), identify 99% of records, and 2 ratings with dates within 3 days identify 68%.

Example 15.5. The script draws a scaled-down Netflix of 5,000 users and 4,000 items, with 203,029 ratings and density 0.0102 (the share of user–item pairs rated), close to Netflix's 0.0118. An item drawn from a random user's record carries a median of 5.83 bits, against $\log_2 5000 = 12.29$ bits needed.

The auxiliary information is a few items from the target's record, with the rating off by one star half the time, and in the noisy setting a quarter of the items are replaced by random popular items. Table 15.3 gives the fraction of 400 targets whose record gets the top score. The third weighting is Equation 15.10 in its approximate form, with $s_i = |\text{supp}(i)|/n$ and $\rho = \frac{1}{2}$, which makes the term in ρ zero.

Table 15.3.: Re-identification rate in a synthetic ratings matrix with 5,000 users: the fraction of 400 targets whose record gets the top score (ties broken at random), for three weightings of Equation 15.9.

Auxiliary information	Equal weights	$1/\ln\lvert\text{supp}(i)\rvert$	$\ln(n/\lvert\text{supp}(i)\rvert)$
2 items	0.428	0.440	0.402
4 items	0.936	0.941	0.924
8 items	1.000	1.000	1.000
8 items, 2 of them wrong	0.943	0.989	0.978

Two items carry about $2 \times 5.83 = 11.7$ bits, a little under $\log_2 n$, and re-identify about 43% of targets, while four carry about 23 bits and re-identify 93%. With 8 items of which 2 are wrong,

the rarity weightings re-identify 98–99%, as Narayanan and Shmatikov found. Equal weights fall to 94%, because a wrong popular item helps many unrelated records. The two rarity weightings differ by less than two standard errors, each $\sqrt{0.98 \times 0.02/400} = 0.007$. The derived weighting of Proposition 15.5 trails with 2 and 4 items, by $0.440 - 0.402 = 0.038$ and $0.941 - 0.924 = 0.017$, within two standard errors, since its $\rho = \frac{1}{2}$ and independence model do not describe this noise.

To test the eccentricity (Figure 15.3), the script also scores 400 outsiders drawn from the same model, so that any match declared for them is false. With 8 items, 2 of them wrong, and $\tau = 1.5$, 94.8% of insiders, the targets in the release, are correctly re-identified, but 26.3% of outsiders are also "matched". The median eccentricity is 7.32 for insiders and 0.72 for outsiders, but the outsiders' distribution has a long tail. Holding false matches to 1% of outsiders takes $\tau = 4.83$, which still re-identifies 73.0% of insiders (Figure 15.4).

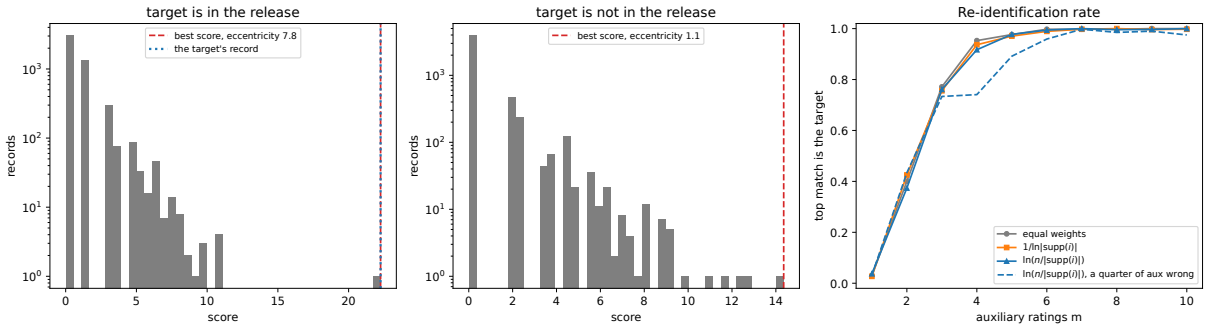


Figure 15.3.: Left and centre: the scores of all 5,000 records for one target (8 items, 2 wrong), on a log scale; the target's record stands far out when it is in the release, and the best score is the edge of the crowd when it is not. Right: re-identification rate against the number of items, for three weightings, and with a quarter of the items wrong.

15.6. Does releasing a sample protect?

Statistical agencies often release a sample, such as 1% of a census. The argument is plausible deniability: a record unique in the sample need not be unique in the population, because the person you have in mind may share their attributes with someone who was not sampled. Yet Rocher, Hendrickx & de Montjoye (2019) predicted, from a statistical model of the population fitted to a sample, that 99.98% of Americans would be correctly re-identified in any dataset with 15 demographic attributes. This section derives when a match found in a sample can be trusted.

Each of n people is in the release independently with probability q, the **sampling rate**. Take a cell with probability p, and let $\lambda = (n-1)p$ be the expected number of *other* people in it. The adversary can start from a person or from a record.

From a person, the adversary knows a target's cell and finds exactly one record there. Either the target was sampled and no one else in the cell was, or exactly one other person was sampled instead, which grows more likely in a crowded cell.

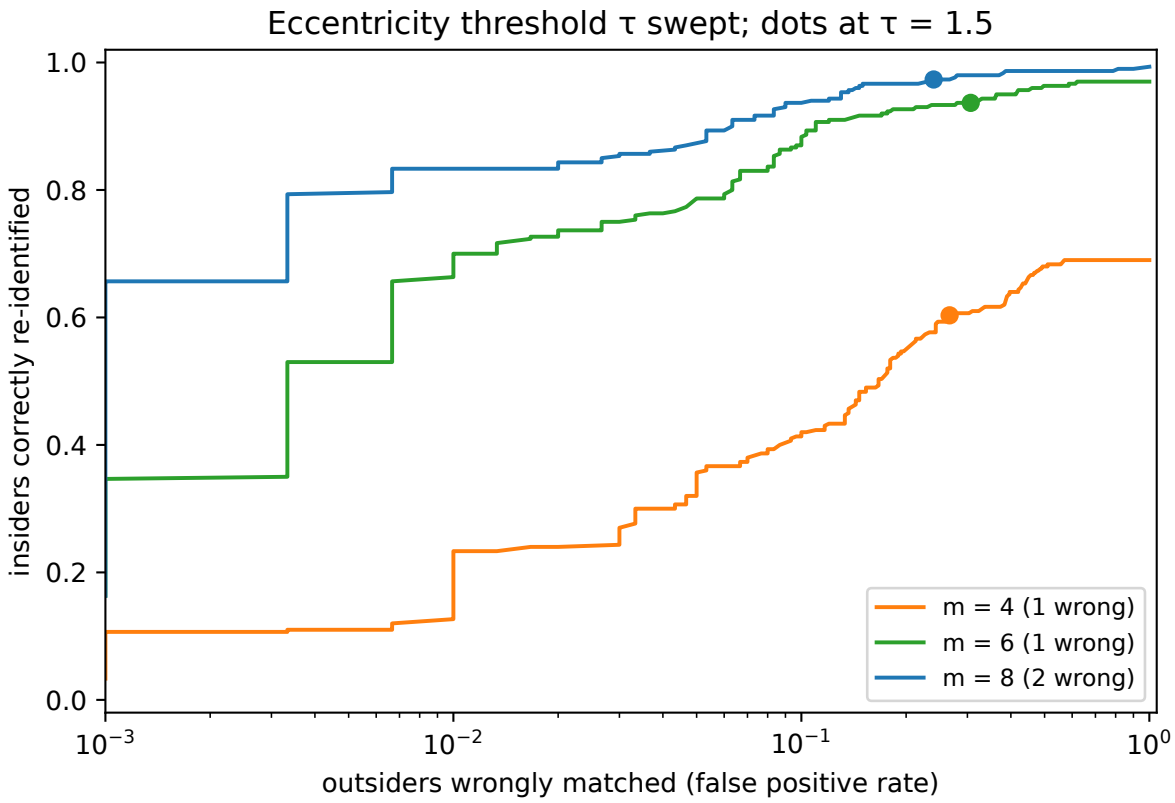


Figure 15.4.: True against false positive rates of the eccentricity test as the threshold τ is swept, for 4, 6 and 8 auxiliary items with 1, 1 and 2 of them wrong. The dots mark $\tau = 1.5$. The false positive rate is on a logarithmic axis, with 0 drawn at its left edge.

Proposition 15.7 (A match from a person)**.** *Given that the target's cell holds exactly one record of the release, that record is the target's with probability*

$$\kappa = \frac{1}{1 + \frac{(n-1)p(1-q)}{1-pq}} \approx \frac{1}{1 + \lambda(1-q)}, \tag{15.15}$$

where the approximation holds for small p.

Proof. The plan is to compute the right and the wrong story given the number of others in the cell, and then average both over that number.

1. *Name the crowd.* Each of the $n-1$ others is in the target's cell with probability p, independently, so their number is $W \sim \mathrm{Bin}(n-1, p)$.
2. *The two stories.* Given W, the right match needs the target sampled and none of the others, with probability $q(1-q)^W$. The wrong one needs the target unsampled and exactly one of the W others sampled, with probability $(1-q) \cdot Wq(1-q)^{W-1} = Wq(1-q)^W$.
3. *Condition on one match.* By the definition of a conditional probability, a correct match has probability $\Pr[\text{right}]/(\Pr[\text{right}] + \Pr[\text{wrong}])$, and each of these unconditional probabilities is its story averaged over W. The factor q cancels, leaving

$$\kappa = \frac{\mathbb{E}[(1-q)^W]}{\mathbb{E}[(1+W)(1-q)^W]}. \tag{15.16}$$

4. *The two averages.* Both come from the generating function $\mathbb{E}[z^W]$. Each of the $n-1$ others contributes an independent factor z to z^W if it is in the cell and 1 if not, so $\mathbb{E}[z^W] = (1-p+pz)^{n-1}$, which is $(1-pq)^{n-1}$ at $z = 1-q$. For the second average, $Wz^W = z\frac{\mathrm{d}}{\mathrm{d}z}z^W$, and the derivative of $(1-p+pz)^{n-1}$ is $(n-1)p(1-p+pz)^{n-2}$, so multiplying by z and setting $z = 1-q$ gives $\mathbb{E}[W(1-q)^W] = (n-1)p(1-q)(1-pq)^{n-2}$.
5. *Simplify.* The denominator of step 3 is the first average plus the second. Dividing both by $(1-pq)^{n-1}$ turns the first into 1 and the second into $\frac{(n-1)p(1-q)}{1-pq}$, which gives Equation 15.15. For small p, $1-pq \approx 1$ and $(n-1)p = \lambda$.

□

From a record, the adversary takes a record alone in its cell in the release and looks the cell up in a population register, such as a voter roll. Being alone in the sample says that nobody else in the cell was sampled, so what remains is whether anyone in the cell was left out.

Proposition 15.8 (A match from a record)**.** *A record that is alone in its cell in the release is alone in the population with probability*

$$\xi = \left(\frac{1-p}{1-pq}\right)^{n-1} \approx e^{-\lambda(1-q)}. \tag{15.17}$$

Proof.

1. *Sort the others into three groups.* Each other person is, independently, in the cell and sampled (probability pq), in the cell and not sampled ($p(1-q)$), or outside the cell ($1-p$).

2. *Condition on being alone in the sample.* This rules out the first group for every other person. People are independent, so for each of them the chances of the other two groups are divided by their total, $1 - pq$.
3. *Ask that nobody hides in the cell.* All $n - 1$ others must be outside the cell, with probability $\left(\frac{1-p}{1-pq}\right)^{n-1}$. Subtracting the numerator from the denominator writes the ratio as $1 - \frac{p(1-q)}{1-pq}$, close to $1 - p(1 - q)$ for small p, so by Lemma 15.1 the power is about $e^{-\lambda(1-q)}$.

□

Over a whole release, the share of sample-unique records that are unique in the population compares two expected counts, each a sum of indicators as in Theorem 15.1.

Proposition 15.9 (Sample uniques that are real)**.** *Under the population model, the expected number of records unique in both the release and the population, divided by the expected number unique in the release, is*

$$\frac{\sum_c p_c(1-p_c)^{n-1}}{\sum_c p_c(1-p_c q)^{n-1}}. \tag{15.18}$$

It tends to the population's fraction unique, Equation 15.3, as $q \to 0$ and is 1 at $q = 1$.

Proof.

1. *Records unique in both.* A population unique is unique in the sample if it is sampled, and sampling is independent of the cells. So the expected number is q times that of population uniques, $nq \sum_c p_c(1-p_c)^{n-1}$ by Theorem 15.1.
2. *Records unique in the sample.* Person i in cell c is sampled, and each other person must be outside c or unsampled, which rules out only being in c and sampled, probability $p_c q$. Adding over people gives $nq \sum_c p_c(1-p_c q)^{n-1}$.
3. *Divide.* The factor nq cancels. As $q \to 0$ the denominator tends to $\sum_c p_c = 1$, and at $q = 1$ the two sums are equal.

□

Sampling only helps people in crowded cells. It enters Equation 15.15 and Equation 15.17 through the factor $1 - q$ on λ, so for rare combinations, $\lambda \ll 1$, both are close to 1 whatever q is. Since shrinking q only raises $\lambda(1 - q)$ towards λ, neither falls below its value at $q = 0$: $\kappa \geq 1/(1 + \lambda)$ and $\xi \geq e^{-\lambda}$.

At small sampling rates, by Equation 15.18, the share of sample uniques that are unique in the population is the population's own fraction unique. So a 1% sample gives deniability only to the people who were not unique anyway. The adversary can also estimate each record's p from the sample and compute ξ, which tells it which matches to trust.

Example 15.6. Table 15.4 gives both approximate probabilities for a single cell. With $\lambda = 0.1$ and $q = 0.01$, for instance, $\kappa = 1/(1 + 0.1 \times 0.99) = 1/1.099 = 0.910$.

Table 15.4.: Probability that a match in a sample is correct: from a person to the release (κ) and from a sample-unique record to the population (ξ), for three crowding levels and two sampling rates.

λ (expected others in the cell)	κ, $q = 0.01$	κ, $q = 0.5$	ξ, $q = 0.01$	ξ, $q = 0.5$
0.1	0.910	0.952	0.906	0.951
0.5	0.669	0.800	0.610	0.779
2	0.336	0.500	0.138	0.368

Going from a 50% to a 1% sample lowers κ by $0.952 - 0.910 = 0.04$ for a rare cell and by $0.500 - 0.336 = 0.16$ for a crowded one. A larger running census has $n = 100{,}000$ people and its first seven attributes, and 82.8% of them are unique. Equation 15.18 gives 0.8301 of sample uniques being population-unique at $q = 0.01$ (simulated: 0.8334 over 19,951 sample uniques in 20 samples), 0.8493 at $q = 0.1$ and 0.9235 at $q = 0.5$.

The adversary's own estimate works too (Figure 15.5, right). From one 1% sample with 993 sample-unique records, it treats the attributes as independent and multiplies their estimated frequencies to get each record's cell probability, and computes ξ by Equation 15.17. Its mean prediction is 0.817 against an actual 0.824. The 415 records it rates above 0.95 average a prediction of 0.980 and are population-unique 99.3% of the time, while the 119 it rates below 0.5 average 0.284 and are unique 30.3% of the time.

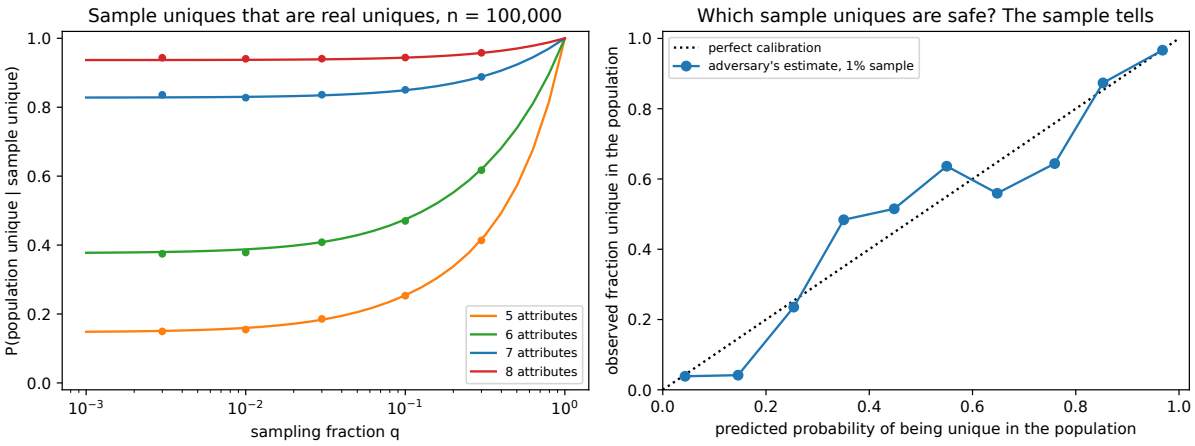


Figure 15.5.: Left: the fraction of sample uniques that are unique in a population of 100,000, against the sampling rate, for 5 to 8 attributes; lines are Equation 15.18, dots simulations. Right: the adversary's predicted ξ from a 1% sample against the observed fraction, in bins of width 0.1.

15.7. Code

The script code/linkage.py computes every number in this chapter and draws every figure. From the repository root:

```
uv run python code/part02-attacks/linkage.py
```

Table 15.5.: The script's output with the default seed.

Experiment	Result
Bits	$\log_2(8 \times 10^9) = 32.9$; $\log_2(248.7 \times 10^6) = 27.9$; ZIP 15.3 + birth date 14.8 + sex 1.0 = 31.1 bits
Sweeney, uniform cells	$K = 2.34 \times 10^9$; $n/K = 0.106$; fraction unique 0.899
World, uniform cells	33 bits: fraction unique 0.394. Bits needed for 50%, 90%, 99%: 33.4, 36.1, 39.5
Toy release	Classes of sizes 1, 1, 1, 2, 3: linkage success 0.625
Hashed identifiers	MD5 of "0": `cfcd2084…`. All 10^6 six-digit identifiers (19.9 bits) hashed, 5 of 5 inverted. Taxi: 173 million trips (27.4 bits), licences 20.9 bits, medallions 24.4 bits; pickup minute 19.0 + pickup square 16.3 = 35.3 bits; phone numbers 33.2 bits
Synthetic census, $n = 10^4$	Table 15.2; McDiarmid 95% half-width 0.0272
Synthetic ratings, 5,000 users	203,029 ratings, density 0.0102; median bits per item 5.83; rates in Table 15.3
Eccentricity, 8 items, 2 wrong	$\tau = 1.5$: insiders 0.948, outsiders 0.263. $\tau = 4.83$: insiders 0.730, outsiders 0.010
Sampling, $n = 10^5$, 7 attributes	Population unique 0.828. Sample uniques that are unique: 0.8301 ($q = 0.01$; simulated 0.8334), 0.8493 ($q = 0.1$), 0.9235 ($q = 0.5$)
κ and ξ	Table 15.4
Adversary's estimate of ξ, 1% sample	Mean predicted 0.817, actual 0.824; above 0.95: predicted 0.980, actual 0.993; below 0.5: predicted 0.284, actual 0.303

tests/test_linkage.py checks that the output still matches.

15.8. Summary

Table 15.6.: The results of this chapter, and where each is used.

Result	Statement	Use
Proposition 15.1	Linkage success is J/n	Scoring a release; at most $1/k$ if every class has k or more records
Proposition 15.2	N hashes invert every unkeyed pseudonym; a pseudonym links a person's records	Hashing protects only identifiers with many bits; licence, phone and identity numbers (20 to 33 bits) are listed in hours; a key stops enumeration, not linkage
Theorem 15.1	$\mathbb{E}[U]/n = \sum_c p_c(1-p_c)^{n-1}$	Risk from a model of the population; compute the exact sum
Lemma 15.1	$e^{-p\ell/(1-p)} \le (1-p)^\ell \le e^{-p\ell}$	"Everyone avoids my cell" as an exponential
Corollary 15.1	Uniform cells: $\exp(-2^{\log_2 n - b})$	$\log_2 n$ bits name one person (33 for the world); 6.64 bits more make 99% unique; skew usually lowers uniqueness
Proposition 15.3	$\mathbb{E}[U]/n \ge (1-\sum_c p_c^2)^{n-1}$	A floor from the marginals; tight for uniform cells
Proposition 15.4	U/n is within γ of its mean except with probability $2e^{-n\gamma^2/2}$	One population is enough
Proposition 15.5	The score with $w_i \approx \ln(1/s_i) + \ln\frac{1-\rho}{\rho}$ is the log-likelihood ratio	The top score is the most probable target
Proposition 15.6	$n\prod_{i\in I} s_i$ expected false matches	Each rare item is worth its surprisal, so a few facts suffice, noise or not
Definition 15.5	Match only if $(\text{top}_1 - \text{top}_2)/\sigma \ge \tau$	Calibrate τ on outsiders at a small false positive rate
Proposition 15.7, Proposition 15.8	$\kappa \approx 1/(1+\lambda(1-q))$, $\xi \approx e^{-\lambda(1-q)}$	A sample protects only people in crowded cells
Proposition 15.9	The share of sample uniques that are real	The population's rate as $q \to 0$; removing names, pseudonymizing or sampling does not anonymize a dataset

15.9. References

- Sweeney, "Simple Demographics Often Identify People Uniquely" (Carnegie Mellon University, Data Privacy Working Paper 3, 2000): the 87% estimate for {ZIP code, birth date, sex}

and its variants with coarser geography. Sweeney, "k-Anonymity: A Model for Protecting Privacy" (International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems, 2002) tells the Massachusetts story in its introduction.

- Golle, "Revisiting the Uniqueness of Simple Demographics in the US Population" (Workshop on Privacy in the Electronic Society, 2006): the same estimate from the 2000 census.
- Narayanan & Shmatikov, "Robust De-anonymization of Large Sparse Datasets" (IEEE Symposium on Security and Privacy, 2008): the score, the eccentricity test and the Netflix experiments.
- Rocher, Hendrickx & de Montjoye, "Estimating the success of re-identifications in incomplete datasets using generative models" (Nature Communications, 2019): why sampling and incompleteness do not protect.
- de Montjoye, Hidalgo, Verleysen & Blondel, "Unique in the Crowd: The privacy bounds of human mobility" (Scientific Reports, 2013): the same sparsity argument for location traces.
- Pandurangan, "On Taxis and Rainbows" (blog post, 2014), and Tockar, "Riding with the Stars: Passenger Privacy in the NYC Taxicab Dataset" (Neustar Research blog, 2014): the two taxi attacks.
- Barth-Jones, "The 'Re-Identification' of Governor William Weld's Medical Information: A Critical Re-Examination of Health Data Identification Risks and Privacy Protections, Then and Now" (2012): what the Massachusetts story does and does not show about ordinary people.
- Ohm, "Broken Promises of Privacy: Responding to the Surprising Failure of Anonymization" (UCLA Law Review, 2010): the legal side.
- Dwork, McSherry, Nissim & Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): where the response to these attacks starts, with a guarantee that holds whatever auxiliary information the adversary has.

16. Syntactic anonymity and how it breaks

Linkage and re-identification showed that deleting names does not anonymize a table. The columns that remain, such as age, ZIP code and sex, are **quasi-identifiers**. None of them names anyone, but together they single out most people, so anyone with a list of names and the same columns can join that list to the release. Between 1998 and 2008, researchers answered with definitions that constrain the released table itself. k-anonymity hides every record in a crowd of at least k records with the same quasi-identifiers. l-diversity asks each crowd to hold several different values of the sensitive column, such as a diagnosis, and t-closeness asks each crowd to have the same mix of those values as the whole table. They are called **syntactic** because each is a property of the published table that you can check by reading it. Statistics offices and open-source anonymization tools such as ARX still use them, so you need to know where each promise ends.

The one idea of this chapter is that a syntactic definition fixes the adversary's knowledge in advance. k-anonymity assumes that the adversary knows the target's quasi-identifiers and nothing else, l-diversity lets it also rule out a few of the target's sensitive values, and t-closeness lets it know how the sensitive values are spread over the whole table. Each attack below gives the adversary one piece of knowledge more than its definition assumes, and the guarantee then no longer applies. A second release, the most ordinary extra knowledge there is, breaks all three definitions.

The running example is a synthetic hospital table of 10,000 patients, with age, ZIP code and sex as quasi-identifiers and one of eight diagnoses as the sensitive attribute. Released with exact quasi-identifiers, it lets an adversary who knows a random patient's three columns pick out that patient's record with probability 0.933, guessing at random among the records that share those values. Two independent anonymized releases of it reveal more than three times as many diagnoses as either alone.

Table 16.1.: The symbols of this chapter.

Symbol	Meaning
n; S, m	records; the sensitive attribute and its number of values
E, $\lvert E\rvert$; J	an equivalence class (records with the same released quasi-identifiers) and its size; the number of classes
$n_E(v)$; $P_E(v)$, $P(v)$	records of E with $S = v$; their fraction in E, and in the table
k; K, λ_k	the smallest class size; cells of the uniform model, and the mean number of people per cell it needs

Symbol	Meaning
ℓ, c; $r_1 \geq ... \geq r_m$	the parameters of l-diversity; a class's sorted value counts
t; $P_E(A)$	the parameter of t-closeness; the probability of a set A of values

16.1. k-anonymity

Sweeney and Samarati's 1998 answer to the linkage attack is to make sure that every combination of quasi-identifiers in the release is shared by at least k people. The data holder, the **custodian**, gets there by generalizing, so that age 37 becomes 35–39 and ZIP code 02139 becomes 021**, and by suppressing the few records that still stand out. This section asks what the definition buys against linkage, and what it costs in the detail the quasi-identifiers can keep.

Definition 16.1 (k-anonymity)**.** A **generalization** replaces each quasi-identifier of a record by a coarser value, such as an age by a range or a ZIP code by a prefix, and different records may be coarsened differently. **Suppression** deletes records. An **equivalence class** E of the released table is a set of records with the same generalized quasi-identifiers, and $|E|$ is its number of records. A release is **k-anonymous** if every equivalence class has at least k records.

The linkage game of Linkage and re-identification draws a target uniformly from a release of n records and gives the adversary the target's quasi-identifiers, and the adversary wins if it picks out the target's record. It finds the target's class but cannot tell the records inside apart, so it guesses one of them at random. In a class of $|E|$ records it is then right with probability $1/|E|$, and the target lands there with probability $|E|/n$. So each class contributes $1/n$, and summing over J classes gives the success J/n of Equation 15.1. For a fixed n, large classes mean few classes, and that is the whole guarantee.

Proposition 16.1 (k-anonymity caps linkage)**.** *If a release of n records is k-anonymous, then the adversary's success in the linkage game is at most $1/k$:*

$$\Pr[\mathit{win}] = \frac{J}{n} \leq \frac{1}{k}. \tag{16.1}$$

Proof.

1. *Count records by class.* The J classes split the n records between them, each record in exactly one class, so $n = \sum_j |E_j|$.
2. *Use the definition.* Every class has at least k records, so the sum is at least Jk. Then $n \geq Jk$, which is $J/n \leq 1/k$.
3. *Read off the linkage success.* Equation 15.1 says that the linkage success is J/n, so it is at most $1/k$.

□

The guarantee is about identity only. Equation 16.1 bounds the chance of picking out the target's *record*, and says nothing about the sensitive attribute, which the next two sections attack. It also needs the right columns treated as quasi-identifiers. In data with many columns and few filled entries, such as movie ratings, every column is one, because a few ratings already single a person out (Section 15.5). Every column must then be coarsened until k records agree on all of them, which leaves little of the table (Aggarwal, 2005).

Coarse quasi-identifiers protect more but say less, so the next question is how coarse they must be. Suppose the generalized quasi-identifiers split n people into K cells, each person falling into a cell independently and every cell equally likely, and no record is suppressed, so each cell is a class. Naming a person's cell then takes $\log_2 K$ bits, to compare with the $\log_2 n$ bits that single out one person of n. Each of the other $n-1$ people lands in a given person's cell with probability $1/K$, so the number of others there is a $\text{Bin}(n-1, 1/K)$ count. With many people and many cells, Theorem 1.2 in Probability distributions shows that this count is close to a Poisson count with the same mean, $(n-1)/K \approx n/K$.

Proposition 16.2 (The cost of k-anonymity for uniform cells)**.** *Suppose n people fall independently into K equally likely cells, and treat the number of other people in a person's cell as a* $\text{Poisson}(n/K)$ *count. A person's class is smaller than k when at most $k-2$ others share the cell, so the expected fraction of people in classes smaller than k is*

$$\Pr[\text{Poisson}(n/K) \leq k-2]. \tag{16.2}$$

Suppose the custodian accepts that in expectation at most 1% of records fall in such classes, to be suppressed. Let λ_k be the mean at which the fraction is exactly 1%, that is $\Pr[\text{Poisson}(\lambda_k) \leq k-2] = 0.01$. *The fraction falls as the mean grows, so it is at most 1% exactly when $n/K \geq \lambda_k$, and the quasi-identifiers may carry at most*

$$\log_2 n - \log_2 \lambda_k \text{ bits.} \tag{16.3}$$

Proof. The plan is to compute the chance that one person's class is too small, note that this chance is also the expected fraction of such people, and then solve for the coarseness.

1. *The size of a person's class.* A person's class is their cell, which holds the person plus N others, with N a $\text{Poisson}(n/K)$ count by assumption.
2. *When it is too small.* The class is smaller than k when $1+N \leq k-1$, that is when $N \leq k-2$, which has probability $\Pr[\text{Poisson}(n/K) \leq k-2]$.
3. *From one person to the fraction.* The expected number of people in small classes is the sum, over people, of each one's chance of being in one, because the expected count of events is the sum of their probabilities. Every person has the same chance, so the sum is n times it, and the expected fraction is that chance.
4. *Solve for the coarseness.* A Poisson count with a larger mean is more likely to exceed any fixed level, so the chance that it is at most $k-2$ falls as the mean grows. The fraction is therefore at most 1% exactly when $n/K \geq \lambda_k$, that is $K \leq n/\lambda_k$. Taking $\log_2$ of both sides gives $\log_2 K \leq \log_2 n - \log_2 \lambda_k$.

□

So k-anonymity costs more than the naive $\log_2 k$ bits below $\log_2 n$, which is what exactly k people in each of $K = n/k$ cells would allow. Cells vary in size around their mean, and the unlucky small cells violate the definition, so you need λ_k people per cell *on average*, more than k. For $k = 2$ a person violates the definition when $N = 0$, which has probability $e^{-n/K}$. Setting $e^{-n/K} = 0.01$ gives $\lambda_2 = \ln 100 = 4.61$, more than twice k.

Example 16.1. Table 16.2 applies Equation 16.2 to a population of $n = 10{,}000$, with $\log_2 n = 13.29$ bits. For $k = 2$, $\lambda_2 = 4.61$ from the calculation above, $\log_2 4.61 = 2.20$, and $13.29 - 2.20 = 11.08$ bits are allowed.

Table 16.2.: Uniform cells, $n = 10{,}000$: how many bits of quasi-identifier k-anonymity allows if at most 1% of records may fall in classes smaller than k.

k	People per cell λ_k	Bits allowed	Below $\log_2 n$ by	$\log_2 k$
2	4.61	11.08	2.20	1.00
5	10.05	9.96	3.33	2.32
10	17.40	9.17	4.12	3.32
20	30.58	8.35	4.93	4.32
50	66.74	7.23	6.06	5.64

The last two columns compare the true cost with the naive $\log_2 k$. The gap is about 1.2 bits at $k = 2$ $(2.20 - 1.00)$ and shrinks to about 0.4 bits at $k = 50$ $(6.06 - 5.64)$, because a Poisson count with a large mean varies little relative to its mean, so λ_k/k approaches 1 and $\log_2 \lambda_k$ approaches $\log_2 k$. Figure 16.1 draws the whole curve for each k.

At the other extreme, Section 15.3 showed that 99% of people are unique once the quasi-identifiers carry $\log_2 n + 6.64$ bits. Hiding 99% in pairs needs 2.20 bits below $\log_2 n$, so only $6.64 + 2.20 = 8.84$ bits, about 9, separate "almost everyone is unique" from "almost everyone is hidden in a pair", and every published attribute uses some of those bits.

Real tables are not uniform, so real anonymizers search for a generalization that works on the data at hand. This chapter's script uses the algorithm of LeFevre, DeWitt & Ramakrishnan (2006).

Definition 16.2 (Mondrian)**.** Given a table and a criterion that a class must meet, such as having at least k records, **Mondrian** starts with the whole table as one box. Each quasi-identifier is coded as ordered numbers (ZIP codes in a fixed order, sex as 0 and 1). Mondrian splits a box at the median of the quasi-identifier whose values in the box span the largest share of their full range, as long as both halves still meet the criterion. If that split fails, it tries the next widest quasi-identifier, and a box whose every split fails is final. Each final box is a class, published as a range per quasi-identifier.

Because Mondrian splits until a half would fail the criterion, its classes stay close to size k. With no tied values they have between k and $2k - 1$ records, because a box of $2k$ or more records splits at its median into two halves of at least k.

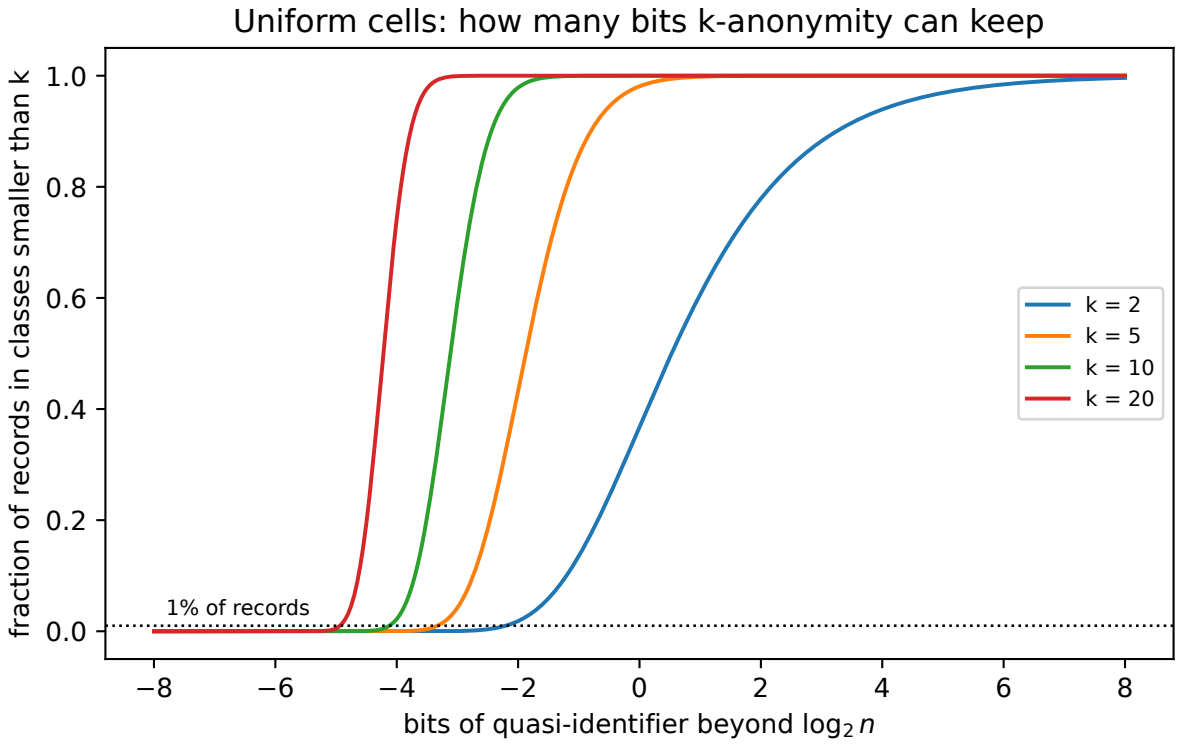


Figure 16.1.: The fraction of records in classes smaller than k, against the bits of quasi-identifier minus $\log_2 n$ (negative when below $\log_2 n$), for uniform cells. Each curve is a Poisson CDF (cumulative distribution function); the dotted line is 1% of records.

Example 16.2. In the hospital table of $n = 10{,}000$ patients, age runs from 0 to 89, the ZIP code takes 800 values with a few areas far more common than the rest, and sex has two values. The diagnosis depends on age, from asthma in childhood to cancer late in life. Flu is equally common at every age, and HIV is rare (1.0% of records). The exact linkage success of 0.933 means, by Equation 15.1, that the patients fall into $0.933 \times 10{,}000 = 9{,}330$ distinct combinations, so most are alone in theirs. Table 16.3 gives the result of Mondrian with k-anonymity as the criterion. Its last two columns concern the diagnosis and are read in Section 16.2.

Table 16.3.: Mondrian k-anonymization of a synthetic hospital table of 10,000 patients. Mean width is the average, over records and quasi-identifiers, of the class's range as a fraction of the full range. Certain disclosure is the fraction of records in a class with a single diagnosis. Guess accuracy is the fraction of records whose diagnosis is the most common one in their class.

k	Classes	Sizes	Mean width	Linkage J/n	Certain disclosure	Guess accuracy
2	4,091	2–5	0.005	0.4091	0.2797	0.675
5	1,542	5–11	0.016	0.1542	0.0232	0.593
10	748	10–20	0.028	0.0748	0.0000	0.562
20	366	20–41	0.043	0.0366	0.0000	0.540
50	129	50–99	0.075	0.0129	0.0000	0.506

The linkage success sits between $1/(2k)$ and $1/k$ for every k (Figure 16.2, left), because J/n is one over the average class size. No class is smaller than k, so the upper bound $1/k$ is guaranteed by Proposition 16.1. The lower bound is not guaranteed, because ties in age and sex let Mondrian leave classes larger than $2k$, up to 41 records at $k = 20$, but they are rare enough that the average size stays below $2k$ in every row, which keeps J/n above $1/(2k)$. The cost in precision is small, since at $k = 5$ the average range covers 1.6% of each quasi-identifier's span.

16.2. Homogeneity, background knowledge and l-diversity

Machanavajjhala, Kifer, Gehrke & Venkitasubramaniam (2006) pointed out that k-anonymity hides who a record belongs to but not what it says. The release exists to show a **sensitive attribute** S, such as a diagnosis. This section asks what an adversary learns about a target's sensitive value from a k-anonymous release, and what condition on the classes limits it. Table 16.4 is a 3-anonymous release that shows the two attacks.

Table 16.4.: A 3-anonymous release with three classes of three records. Age 3* means 30–39, and 4* means 40–49.

ZIP code	Age	Diagnosis
130**	< 30	heart disease
130**	< 30	heart disease
130**	< 30	heart disease

ZIP code	Age	Diagnosis
148**	4*	cancer
148**	4*	flu
148**	4*	heart disease
130**	3*	cancer
130**	3*	cancer
130**	3*	flu

Both attacks, and the guarantee that answers them, compute the adversary's **posterior**, its belief about the target's sensitive value after it has seen the release.

Lemma 16.1 (Within a class, the posterior is the frequency)**.** *Suppose the adversary knows the target's quasi-identifiers, knows that the target is in the release, and knows nothing else about the target. Let E be the target's class, and let $n_E(v)$ be the number of records of E with sensitive value v. Then the adversary's posterior is the class's frequency of v,*

$$\Pr[S_{target} = v \mid release] = \frac{n_E(v)}{|E|} = P_E(v). \tag{16.4}$$

If the adversary also knows that the target's value is not in a set B of values, then for every value v outside B,

$$\Pr[S_{target} = v \mid release, S_{target} \notin B] = \frac{n_E(v)}{|E| - \sum_{u \in B} n_E(u)}. \tag{16.5}$$

Proof.

1. *Every record of the class is equally likely to be the target's.* All records of E show the same generalized quasi-identifiers and differ only in the sensitive value, which the adversary does not know. The adversary knows nothing else about the target, so, as for a target drawn at random, it has no reason to favour any record, and each has probability $1/|E|$.
2. *Count the records with value v.* The target has value v exactly when its record is one of the $n_E(v)$ records with that value. Adding $1/|E|$ over them gives $n_E(v)/|E|$.
3. *Cross out the excluded values.* Knowing that the target's value is not in B rules out the records with values in B. The records that are left still look alike, so the adversary spreads its belief evenly over them. There are $|E| - \sum_{u \in B} n_E(u)$ of them, and step 2 applied to these records gives the second formula.

□

Alice is 26 and lives in ZIP code 13053, so her class is the first. Every record in it says heart disease, and by Lemma 16.1 her posterior for heart disease is $\frac{3}{3} = 1$, so the **homogeneity attack** discloses her diagnosis without re-identifying anyone. Bob is 35, lives in 13068, and is in the third class, where two of three records say cancer, so his posterior for cancer is $\frac{2}{3}$. His neighbour knows that Bob never had flu that season and rules it out. Crossing out the flu record leaves two cancer records, and cancer then has posterior $\frac{2}{2} = 1$, which is the **background knowledge attack**.

Both attacks work because one value dominates the class, either from the start or after a few values are ruled out. l-diversity therefore asks for enough different values in each class, and its

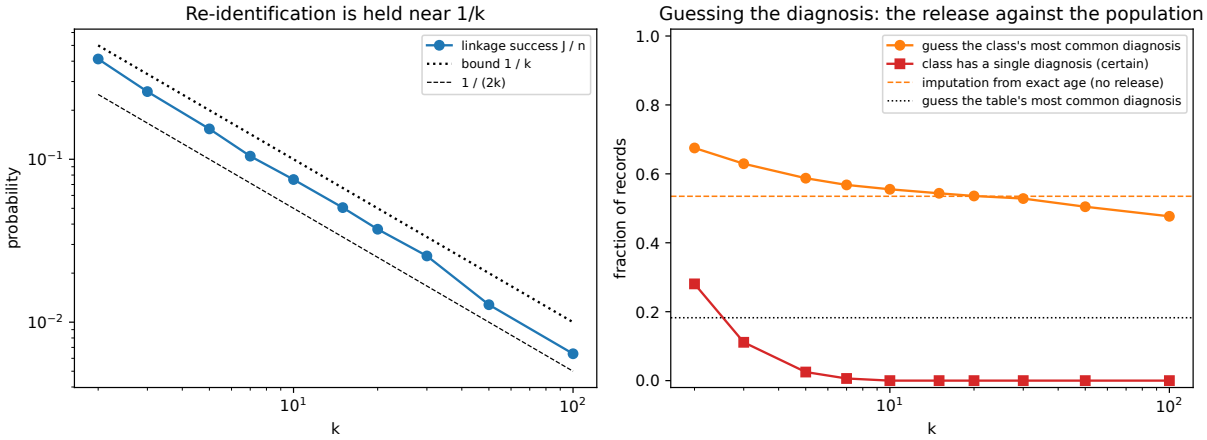


Figure 16.2.: Left: the linkage success of Mondrian releases of the synthetic hospital table against k, between $1/(2k)$ and the bound $1/k$. Right: how often the diagnosis is the most common one in the record's class, and how often the class holds a single diagnosis, against two baselines that use no release, the table's most common diagnosis and the best guess from age.

three versions differ in how they measure "enough". Sort a class's value counts from largest to smallest as $r_1 \geq r_2 \geq \ldots \geq r_m$, so that r_1 is the count of the most common value. The entropy $H(P_E) = -\sum_v P_E(v) \log_2 P_E(v)$ measures in bits how spread out the values are, and ℓ equally common values have entropy exactly $\log_2 \ell$. The third version is built to survive the background knowledge attack.

Definition 16.3 (l-diversity)**.** A class E is

- **distinct ℓ-diverse** if it holds at least ℓ different values;
- **entropy ℓ-diverse** if $H(P_E) \geq \log_2 \ell$, so that the class is at least as spread out as ℓ equally common values;
- **recursive (c, ℓ)-diverse** if $r_1 < c\,(r_\ell + r_{\ell+1} + \cdots + r_m)$, so that the most common value has fewer than c times as many records as all the values outside the $\ell - 1$ most common have together.

A release is ℓ-diverse in one of these senses if every class is.

The right-hand side of the recursive condition is c times the tail, the records left after the $\ell - 1$ most common values are set aside. The guarantee allows the adversary to rule out $\ell - 2$ values, the most any guarantee can allow, since ruling out $\ell - 1$ values of a class with exactly ℓ values leaves one. Ruling out values only removes counts, so the top count cannot grow and the rest cannot shrink below the tail.

Proposition 16.3 (What recursive diversity guarantees)**.** *Let a class be recursive (c, ℓ)-diverse, and let the adversary know, besides the target's quasi-identifiers, that the target's value is not one of some $\ell - 2$ values. Then every remaining value v has*

$$\Pr[S_{target} = v \mid \textit{release}, \textit{background}] < \frac{c}{1 + c}. \tag{16.6}$$

Proof. The plan is to show that the defining inequality survives elimination, and then let Lemma 16.1 turn it into a posterior.

1. *What is left.* Rule out a set B of $\ell - 2$ values, all occurring in the class (a value that does not occur removes nothing, and leaves more counts in the tail). The remaining $m - \ell + 2$ values keep their original counts. Call the largest of them r'_1, and the total of the other $m - \ell + 1$ remaining values R.
2. *The top does not grow.* Ruling out values removes records but never adds any, so r'_1 is one of the original counts. No original count exceeds r_1, so $r'_1 \leq r_1$.
3. *The tail does not shrink below the original tail.* R is a sum of $m-\ell+1$ of the original counts. The smallest such sum is that of the $m - \ell + 1$ smallest counts, which is $r_\ell + \cdots + r_m$. So $R \geq r_\ell + \cdots + r_m$.
4. *Combine with the definition.* Chaining step 2, the definition and step 3 gives

$$r'_1 \leq r_1 < c\,(r_\ell + \cdots + r_m) \leq c\,R. \tag{16.7}$$

5. *Turn counts into a posterior.* By Lemma 16.1, after elimination the posterior of a remaining value is its count over the total count left, $r'_1 + R$, so it is at most $r'_1/(r'_1 + R)$. The share $x/(x + R)$ grows with x, like a larger slice of a pie whose other part is fixed. With $r'_1 < cR$ from step 4, the share is therefore below

$$\frac{cR}{cR + R} = \frac{c}{1 + c}. \tag{16.8}$$

□

l-diversity rules out certainty but still allows high likelihood. The bound $c/(1 + c)$ is $\frac{1}{2}$ at $c = 1$, $\frac{2}{3}$ at $c = 2$ and $\frac{3}{4}$ at $c = 3$, so even recursive diversity lets the adversary be fairly sure. Distinct ℓ-diversity only says that the adversary must rule out $\ell-1$ values to be certain, while one value can still hold all but $\ell-1$ of the records. Entropy ℓ-diversity does not limit the top value's share directly either. An entropy 3-diverse class over 8 values can have one value in 73% of its records, because the other 27% spread evenly over seven values give entropy $-0.73 \log_2 0.73 - 0.27 \log_2 \frac{0.27}{7} = 1.60$ bits, above $\log_2 3 = 1.58$.

The guarantee assumes how much the adversary knows rather than limiting it. It covers $\ell - 2$ ruled-out values, and one more can give certainty whatever c is. An adversary who knows something softer, such as "probably not flu", is not covered either.

Some tables cannot be made diverse at all. The whole table's distribution P is the average of its classes' distributions P_E, each weighted by the class's share of the records. In entropy, such an average is at least as spread out as its parts are on average, so a concentrated table forces some class to be concentrated too.

Proposition 16.4 (A table with low entropy has no entropy-diverse release)**.** *If the whole table's distribution P of the sensitive attribute has entropy below $\log_2 \ell$, then no partition of the table into classes is entropy ℓ-diverse.*

Proof.

1. *The table is a mixture of its classes.* Pick a record uniformly at random, and let C be its class and S its sensitive value. Then S has distribution P, and given that C is the class E, it has distribution P_E.
2. *Conditioning lowers entropy on average.* Information theory proves in Corollary 5.1 that $H(S \mid C) \le H(S)$. Here $H(S \mid C)$ is, by the definition of conditional entropy, the average of the class entropies $H(P_E)$, weighted by the class shares, and $H(S)$ is $H(P)$.
3. *Conclude.* If every class had $H(P_E) \ge \log_2 \ell$, their weighted average would too, and step 2 would give $H(P) \ge \log_2 \ell$, against the hypothesis.

□

Example 16.3. In Table 16.4 the three classes hold 1, 3 and 2 different values, so they are distinct 1-, 3- and 2-diverse. The largest ℓ for which a class is entropy ℓ-diverse is $2^{H(P_E)}$, because ℓ equally common values have entropy $\log_2 \ell$. The first class has one value, entropy 0 and $\ell = 1.00$, and the second has three equally common values and $\ell = 3.00$. The third, $\frac{2}{3}$ cancer and $\frac{1}{3}$ flu, has entropy $\frac{2}{3}\log_2 \frac{3}{2} + \frac{1}{3}\log_2 3 = 0.918$ bits and $\ell = 2^{0.918} = 1.89$. The release as a whole is only 1-diverse, because of Alice's class.

In the hospital table, the "certain disclosure" column of Table 16.3 counts the patients in homogeneous classes, 28.0% of all patients at $k = 2$ and 2.3% at $k = 5$, which is 232 patients whose diagnosis the release states outright. Knowing a patient's age never gives that certainty, since at no age does the population's distribution give any diagnosis more than 0.705.

A posterior has to be read against what the adversary would believe with no release, since 0.6 for a diagnosis is no leak if the population already gives 0.6 to people of the target's age. The right comparison is the **imputation baseline** of Attribute inference and model inversion, the best guess from the target's own quasi-identifiers when the population is known exactly. Here the diagnosis depends only on age, so imputation guesses from age alone. Guessing the most common diagnosis of the whole table is right for 18.5% of patients. Imputation is right for 53.6% (Figure 16.2, right). A 5-anonymous release is right for 59.3%. So most of what the release teaches the adversary is the age–diagnosis pattern, which any useful study of this table would report. The excess over imputation, $0.593 - 0.536 = 0.057$ at $k = 5$, comes from classes so small that their most common diagnosis is often the member's own, and it is down to $0.540 - 0.536 = 0.004$ at $k = 20$.

Table 16.5 adds diversity to $k = 5$.

Table 16.5.: Mondrian with l-diversity on the same hospital table: each split is allowed only if both halves satisfy the criterion.

Criterion	Classes	Sizes	Mean width	Linkage J/n	Certain disclosure	Guess accuracy
$k = 5$	1,542	5–11	0.016	0.1542	0.0232	0.593
$k = 5$, distinct $\ell = 3$	1,132	5–33	0.022	0.1132	0.0000	0.574

Criterion	Classes	Sizes	Mean width	Linkage J/n	Certain disclosure	Guess accuracy
$k = 5$, recursive $(c = 2, \ell = 3)$	389	5–641	0.113	0.0389	0.0000	0.417

Distinct 3-diversity removes the certain disclosures for little cost. Recursive $(2, 3)$-diversity forces some classes to cover hundreds of patients and pushes the guess accuracy below imputation. Diagnosis depends on age, so to keep any diagnosis from dominating a class, Mondrian must mix ages, and that erases the pattern the data were released to show.

16.3. Skewness, similarity and t-closeness

l-diversity counts values, but what matters is how far the adversary's belief moves. Li, Li & Venkatasubramanian (2007) found two attacks that diversity does not see. In the **skewness attack**, 1% of the table has HIV, and a class that is half HIV is 2-diverse in every sense, yet by Lemma 16.1 it raises the belief that a member has HIV fifty-fold. In the **similarity attack**, a class holds gastric ulcer, gastritis and stomach cancer, so it is 3-diverse, and still reveals a stomach disease. This section measures a class by its distance from the whole table's distribution, and asks what a bound on that distance guarantees.

Definition 16.4 (t-closeness)**.** Let d be a distance between distributions on the values of the sensitive attribute. A release is **t-close** if every class's distribution is within distance t of the whole table's, that is $d(P_E, P) \le t$ for every class E.

The first choice of distance is total variation. Theorem 3.3 in Divergences proves that the **total variation (TV)** distance, half the total absolute difference between two probability tables, equals the largest difference between the probabilities the two tables give to one set of values:

$$\mathrm{TV}(P_E, P) = \frac{1}{2}\sum_{v} |P_E(v) - P(v)| = \max_{A} |P_E(A) - P(A)|. \tag{16.9}$$

Here the maximum is over all sets of values A, and $P_E(A)$ is the probability under P_E that the value lands in A.

Proposition 16.5 (What t-closeness in total variation guarantees)**.** *If a release is t-close with the total variation distance, then for every class E and every set of values A,*

$$|P_E(A) - P(A)| \le t. \tag{16.10}$$

In particular, guessing the most common value of the target's class is right with probability at most $\max_v P(v) + t$, the best guess from the table alone plus t.

Proof.

1. *A distance bound is an event bound.* By Theorem 3.3, $\mathrm{TV}(P_E, P)$ is the largest of $|P_E(A) - P(A)|$ over all sets A, so a bound t on the distance bounds every one of them by t.
2. *The adversary's belief is the class distribution.* By Lemma 16.1, the adversary's belief about a target in class E is P_E. So every question it can ask about the target's value, "is it in A?", gets an answer within t of the answer from the table as a whole.
3. *Apply it to the best guess.* Take $A = \{v\}$ for the class's most common value v. Then step 1 gives $P_E(v) \le P(v) + t$, and $P(v)$ is at most $\max_u P(u)$.

□

The bound is additive, but rare values need a ratio. Equation 16.10 lets a 1% value rise to $1\% + t$, which at $t = 0.15$ is 16%, a sixteen-fold increase. A guarantee for rare values has to bound the *ratio* of the adversary's beliefs with and without the release, not their difference, and that is the shape of differential privacy, the definition of Part 3.

Total variation treats every pair of values as equally different, so it does not see the similarity attack. The **earth mover's distance (EMD)**, the choice of Li et al., makes similar values close. Think of each distribution as piles of sand on the values, and of a ground distance between values as the cost of moving a unit of sand from one to the other.

Definition 16.5 (Earth mover's distance)**.** Given a ground distance between values, the **earth mover's distance** $\mathrm{EMD}(P_E, P)$ is the least total cost of moving the mass of P_E until it becomes P, where moving mass f from value i to value j costs f times their ground distance.

For ordered values, such as salary bands or age groups, the least cost has a closed form. On a line, sand can only get from one side of a gap to the other by crossing it, so the work can be counted gap by gap.

Proposition 16.6 (Earth mover's distance on ordered values)**.** *Number m ordered values 1 to m, and place them at ground distance $|i-j|/(m-1)$, so that the two ends are distance 1 apart. Then*

$$\mathrm{EMD}(P_E, P) = \frac{1}{m-1} \sum_{i=1}^{m-1} \Big| \sum_{j \le i} \big(P_E(j) - P(j)\big) \Big|. \tag{16.11}$$

Proof.

1. *Cost per gap.* A move from i to j crosses each of the $|i-j|$ gaps between them once, so its cost is $1/(m-1)$ per unit of mass per gap crossed.
2. *How much must cross each gap.* Look at the gap between values i and $i+1$. The values up to i hold $\sum_{j \le i} P_E(j)$ under the class and must end up holding $\sum_{j \le i} P(j)$. Their excess, or shortfall, is $\sum_{j \le i} (P_E(j) - P(j))$, and that net mass has to cross the gap, rightwards if it is positive and leftwards if it is negative. So any plan moves at least this much across the gap.
3. *A plan that does no more.* Move exactly the net amount of step 2 across each gap, between neighbouring values. Value i then gains the net mass entering through its two gaps, which is $P(i) - P_E(i)$, so the plan turns P_E into P. So the least cost is $1/(m-1)$ times the sum, over the $m-1$ gaps, of the absolute net mass that crosses each one, which is Equation 16.11.

□

The choice of distance is a choice of what counts as similar. EMD with a ground distance that puts diagnoses of the same organ close together treats gastritis and gastric ulcer as close, so a class full of stomach diseases is far from the table and fails the definition. On salary bands, if the table is spread evenly, a class bunched at the low end is farther from it than a class spread across the range.

Example 16.4. Take total variation first. In Table 16.4, the table's distribution is $\frac{4}{9}$ heart disease, $\frac{3}{9}$ cancer and $\frac{2}{9}$ flu. Both distributions sum to 1, so the excesses and shortfalls are equal, and the half-sum of absolute differences is the total excess of the class over the table. Alice's class is all heart disease, an excess of $1 - \frac{4}{9} = \frac{5}{9} = 0.556$. The second class is $\frac{1}{3} = \frac{3}{9}$ of each value, an excess of $\frac{3}{9} - \frac{2}{9} = \frac{1}{9} = 0.111$ on flu only. Bob's class, $\frac{6}{9}$ cancer and $\frac{3}{9}$ flu, has excesses $\frac{3}{9}$ and $\frac{1}{9}$, together $\frac{4}{9} = 0.444$. So the release is t-close only for $t \geq 0.556$.

For the skewness attack (Figure 16.3), the class that is 50% HIV is at total variation $0.50 - 0.01 = 0.49$ from a table with 1% HIV, the excess on HIV. A class with 10% HIV is at $0.10 - 0.01 = 0.09$, which passes $t = 0.1$, and still makes HIV ten times as likely for its members.

Table 16.6 adds t-closeness in total variation to $k = 5$ on the hospital table.

Table 16.6.: Mondrian with t-closeness (total variation) on the hospital table. The table's most common diagnosis has frequency 0.185.

Criterion	Classes	Sizes	Mean width	Largest TV	Guess accuracy	Bound $\max_v P(v)+t$
$k = 5$	1,542	5–11	0.016	0.937	0.593	—
$k = 5$, $t = 0.3$	495	6–60	0.291	0.300	0.275	0.485
$k = 5$, $t = 0.15$	136	14–304	0.324	0.149	0.222	0.335

The bound column is $0.185 + t$, and the guess accuracy respects it, as Proposition 16.5 says it must. But the classes now span about 30% of each quasi-identifier's range, and the guess accuracy of 0.222 to 0.275 is far below the 0.536 of imputation from age. Making every class t-close mixed ages, as recursive diversity did.

16.4. The composition attack

Every definition so far is about one release, and Ganta, Kasiviswanathan & Smith (2008) asked what happens when there are two. Two hospitals in one city publish their patient tables, each properly k-anonymous or ℓ-diverse, with no coordination, and a patient treated at both appears in a class of each.

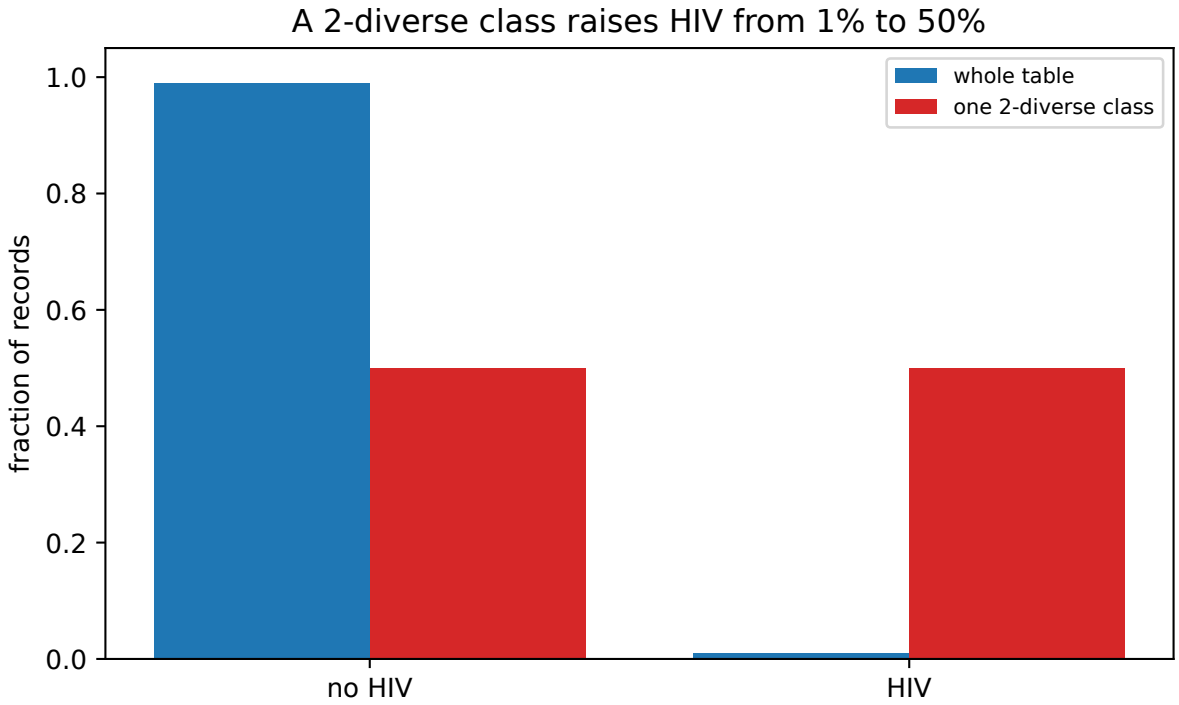


Figure 16.3.: The skewness attack: a table in which 1% of patients have HIV, and a class of it that is half HIV. The class is 2-diverse but at total variation 0.49 from the table.

Definition 16.6 (The composition attack)**.** Let a target appear in two releases, in class E_1 of the first and class E_2 of the second. The attack's **candidates** are the values that appear in both classes,

$$\{v : n_{E_1}(v) > 0\} \cap \{v : n_{E_2}(v) > 0\}. \tag{16.12}$$

The target's value is **pinned down** when a single candidate is left.

The true value is always a candidate, because the target's own record carries it in each class, so a single candidate is the truth. None of the three definitions **composes**, that is, survives the combination of two releases, because each is a condition on one table, and nobody checked the intersection of two classes. To prevent it, a hospital would have to check its classes against every other release, which it cannot see, so a definition that survives must bound what two releases leak together, whatever the adversary knows. Differential privacy, the subject of Part 3, does.

To see how often the attack succeeds, model the classes as random. Let the target's class have k_1 records in the first release and k_2 in the second, and let the values of the other members be independent draws from the population, with p_v the probability of value v. Given the true value v, let W_1 be the set of wrong values, those other than v, that appear among the $k_1 - 1$ other members of the first class, and W_2 the same for the second. These are the wrong values each release leaves open, and the target is pinned exactly when no wrong value is left open by both. Write $p(W) = \sum_{u \in W} p_u$ for the probability of a set of values W. The formula below fixes what the first class leaves open and asks whether the second class avoids it.

Proposition 16.7 (How often two releases pin down a value)**.** *In this model, the probability that the two releases pin down the target's value is*

$$\Pr[\textit{pinned}] = \sum_{v} p_v \, \mathbb{E}\big[(1 - p(W_1))^{k_2 - 1}\big], \tag{16.13}$$

where the expectation is over the random set W_1 for the given v. For every set W of values other than v, the probability that W_1 is exactly W is

$$\Pr[W_1 = W] = \sum_{T \subseteq W} (-1)^{|W| - |T|} \big(p_v + p(T)\big)^{k_1 - 1}. \tag{16.14}$$

Proof. The plan is to fix the true value and what the first class contains, compute the chance that the second class avoids it, and then average.

1. *Condition on the true value.* The target's true value is v with probability p_v. Everything below is for a fixed v, and the outer sum $\sum_v p_v(\cdot)$ averages over it.
2. *When the value is pinned.* The candidates are v itself together with the wrong values that appear in both classes, the set $W_1 \cap W_2$. So the value is pinned exactly when W_1 and W_2 are disjoint.
3. *Fix what the first class leaves open.* Condition on $W_1 = W$ for a particular set W. The $k_2 - 1$ others of the second class are independent draws, and each avoids W with probability $1 - p(W)$, so
$$\Pr[W_2 \cap W = \varnothing] = (1 - p(W))^{k_2 - 1}. \tag{16.15}$$
Averaging over the possible sets W, weighted by $\Pr[W_1 = W]$, gives the expectation in Equation 16.13.
4. *The distribution of the first class's set.* $W_1 = W$ means two things. Every one of the $k_1 - 1$ draws lands in $W \cup \{v\}$, and each value of W is hit at least once. For any set T, the event that all draws land in $T \cup \{v\}$ has probability $(p_v + p(T))^{k_1 - 1}$. **Inclusion–exclusion** then handles the second requirement. Start from "all draws land in $W \cup \{v\}$", subtract the cases where some value of W is missed, which means all draws land in a smaller set, add back the cases subtracted twice, and continue alternating over every number of missed values. Each term is labelled by a subset T of W that the draws are allowed to hit, and the values of W outside T are the ones forced to be missed. The sign $(-1)^{|W| - |T|}$ is plus when an even number are missed and minus when an odd number are.

□

For a single wrong value u, the sum has two terms,

$$\Pr[W_1 = \{u\}] = (p_v + p_u)^{k_1 - 1} - p_v^{k_1 - 1}. \tag{16.16}$$

That is the probability that every draw is v or u, minus the probability that every draw is v, in which case u was never hit.

The model understates the risk, because it draws the other members of a class from the whole population. In real releases a class collects people of similar ages, who are more likely to share a diagnosis, so each class holds fewer distinct wrong values and the intersection is smaller.

The sum over subsets grows quickly with the number of values, so a simpler bound helps. The target escapes only if some wrong value turns up in both classes, and the chance of "some" is at most the sum of the chances of each.

Proposition 16.8 (A union bound on pinning)**.** *In the same model,*

$$\Pr[\textit{pinned}] \geq 1 - \sum_{v} p_v \sum_{u \neq v} (1 - (1 - p_u)^{k_1 - 1})(1 - (1 - p_u)^{k_2 - 1}). \tag{16.17}$$

Proof.

1. *When it fails.* Given v, the value is not pinned when some $u \neq v$ is in both W_1 and W_2.
2. *The chance for one wrong value.* The value u is missing from the first class when all $k_1 - 1$ others avoid it, which has probability $(1 - p_u)^{k_1 - 1}$. So u is in W_1 with probability $1 - (1 - p_u)^{k_1 - 1}$, and likewise for W_2. The two classes are drawn independently, so the probability that u is in both is the product.
3. *Add over wrong values.* The expected number of wrong values in both sets is the sum of these products, because the expected count of events is the sum of their probabilities. A count that is at least 1 whenever the event happens has an expectation at least the event's probability, so the chance that there is at least one shared wrong value is at most this sum. Subtracting from 1 and averaging over v gives Equation 16.17.

□

The union bound charges every wrong value separately, so it is accurate only when two wrong values rarely turn up in both classes at once. With classes of 2, each class holds at most one wrong value, and the bound is exact. With classes of 5, the sum in Equation 16.17 already exceeds 1, so the bound is negative and Equation 16.13 is needed.

Example 16.5. With the population's diagnosis distribution of the hospital table, the model's probability that two releases pin down the diagnosis is 0.8734 for classes of 2, 0.1615 for classes of 5, 0.0288 for classes of 7 and 0.0015 for classes of 10. It falls fast because a larger class contains more wrong values, and each one is another chance for the two classes to share one. Simulation agrees to within 0.001 (Figure 16.4, left). The union bound gives 0.8734 at $k = 2$ and is negative from $k = 5$ on.

The script then runs the attack. A city has 15,000 people drawn from the model of the hospital table. Two hospitals each treat a random 10,000 of them, so 6,649 people are treated at both. Each hospital publishes a 5-anonymous Mondrian release of its own patients. Table 16.7 gives what the adversary learns about the 6,649.

Table 16.7.: The composition attack on two independent hospital releases, for the 6,649 people treated at both. "Pinned" is the fraction left with a single possible diagnosis.

Releases	Pinned by release 1 alone	Pinned by release 2 alone	Pinned by both	Mean candidates, release 1 alone	Mean candidates, both
$k = 5$	0.0289	0.0262	0.0985	2.79	2.38
$k = 5$, distinct $\ell = 3$	0.0000	0.0000	0.0027	3.22	2.92

Two 5-anonymous releases pin down the diagnosis of 9.9% of the shared patients, more than three times either release alone (0.0985 against 0.0289 and 0.0262). At $k = 5$ a patient's class has 6.8 records, averaged over patients rather than over classes, so the nearest model case is classes of 7, and it predicts only 2.9%. The release does better because real classes gather similar ages, as noted after Proposition 16.7. With distinct 3-diversity, no single release pins anyone down, and the two together pin down 18 patients (0.27% of 6,649).

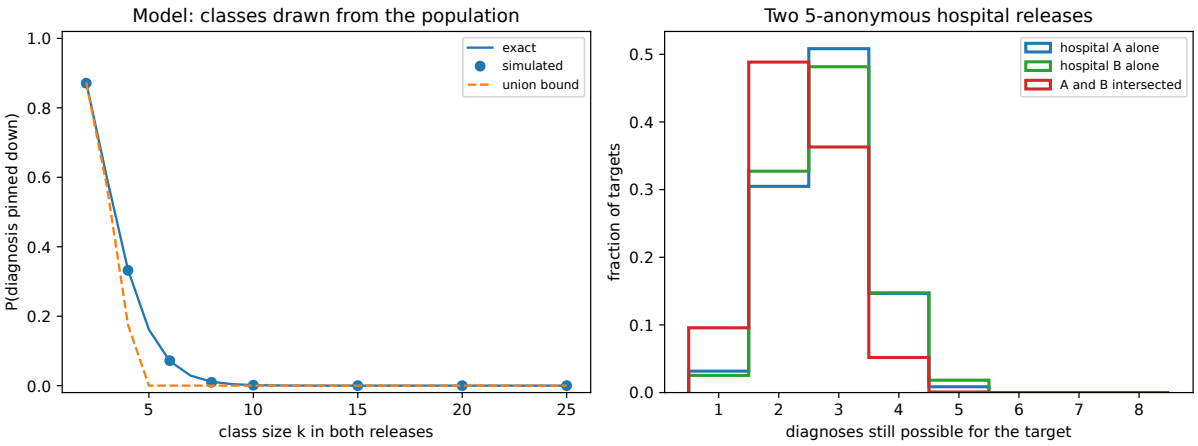


Figure 16.4.: Left: the probability that two releases pin down the diagnosis, against the class size, when the other members of each class are drawn from the population: exact (Equation 16.13), simulated, and the union bound (Equation 16.17). Right: the number of diagnoses still possible for a patient treated at both hospitals, from either 5-anonymous release alone and from their intersection.

16.5. Code

The script code/syntactic_anonymity.py computes every number in this chapter and draws every figure. It implements Mondrian with each criterion, the uniform-cells calculation and every attack of the chapter. From the repository root:

```
uv run python code/part02-attacks/syntactic_anonymity.py
```

Table 16.8.: Output of the chapter's script (seed 0).

Experiment	Result
Toy release	Posteriors 1 (homogeneous class), $\frac{2}{3}$ cancer (Bob), 1 after ruling out flu; entropy $\ell = 1.00$, 3.00, 1.89; TV to the table 0.556, 0.111, 0.444
Recursive (c, ℓ) bound	$c/(1+c) = 0.500$, 0.667, 0.750 for $c = 1$, 2, 3
Skewness	50% HIV against 1%: TV 0.49, ratio 50. 10% HIV: TV 0.09, ratio 10

Experiment	Result
Uniform cells, $n = 10^4$	Table 16.2
Hospital table, $n = 10^4$	Exact quasi-identifiers: linkage 0.933. Baselines: most common diagnosis 0.185, imputation from age 0.536, largest population posterior 0.705
Mondrian	Table 16.3, Table 16.5, Table 16.6
Two hospitals	Table 16.7
Composition model	Pinned: 0.8734 ($k = 2$), 0.1615 ($k = 5$), 0.0288 ($k = 7$), 0.0015 ($k = 10$); simulation within 0.001

tests/test_syntactic_anonymity.py checks that the output still matches.

16.6. Summary

Table 16.9.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 16.1, Proposition 16.1	Every class has at least k records, so linkage $J/n \leq 1/k$	Protects names, not attributes: ask what a class's values reveal
Proposition 16.2	Uniform cells: a fraction $\Pr[\text{Poisson}(n/K) \leq k-2]$ is in classes below k	99% 2-anonymous at $\log_2 n - 2.20$ bits; more than $\log_2 k$ bits
Definition 16.2	Split at medians while both halves meet the criterion	Linkage between $1/(2k)$ and $1/k$ at little cost in precision
Lemma 16.1	The posterior is the class's frequency; ruling out values renormalizes	Every attribute attack, read against imputation
Definition 16.3, Proposition 16.3	Recursive (c, ℓ)-diversity keeps the posterior below $c/(1+c)$ after $\ell - 2$ values are ruled out	Distinct and entropy diversity do not cap the top value's share
Proposition 16.4	Table entropy below $\log_2 \ell$ rules out entropy ℓ-diversity	Check the table's entropy before choosing ℓ
Definition 16.4, Proposition 16.5	Total variation t moves every event's probability by at most t	A 1% value may rise to $1\% + t$: rare values need a bound on the ratio
Definition 16.5, Proposition 16.6	On ordered values, EMD sums the net mass crossing each gap	Similar values are close, which catches the similarity attack

Result	Statement	Use
Definition 16.6, Proposition 16.7, Proposition 16.8	Candidates are the values in both classes	No syntactic definition prevents pinning, so assume another release exists

16.7. References

- Samarati & Sweeney, "Protecting Privacy when Disclosing Information: k-Anonymity and Its Enforcement through Generalization and Suppression" (SRI technical report, 1998), and Sweeney, "k-Anonymity: A Model for Protecting Privacy" (International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems, 2002): the definition and the linkage attack it answers.
- LeFevre, DeWitt & Ramakrishnan, "Mondrian Multidimensional k-Anonymity" (ICDE, 2006): the median-split algorithm the script uses.
- Machanavajjhala, Kifer, Gehrke & Venkitasubramaniam, "l-Diversity: Privacy Beyond k-Anonymity" (ACM Transactions on Knowledge Discovery from Data, 2007; ICDE 2006): the homogeneity and background knowledge attacks, and the three versions of l-diversity.
- Li, Li & Venkatasubramanian, "t-Closeness: Privacy Beyond k-Anonymity and l-Diversity" (ICDE, 2007): the skewness and similarity attacks, and the earth mover's distance.
- Ganta, Kasiviswanathan & Smith, "Composition Attacks and Auxiliary Information in Data Privacy" (KDD, 2008): the intersection attack on independent releases, and why a definition must survive other releases.
- Aggarwal, "On k-Anonymity and the Curse of Dimensionality" (VLDB, 2005): why high-dimensional data cannot be k-anonymized without destroying it.
- Prasser, Kohlmayer and colleagues, the ARX data anonymization tool (arx.deidentifier.org): an open implementation of these criteria.
- Article 29 Working Party, "Opinion 05/2014 on Anonymisation Techniques": how European regulators judge these techniques against singling out, linkability and inference.

17. Reconstruction attacks

A hospital keeps one sensitive bit per patient, whether they tested positive, and never releases the bits. It only answers counting questions, such as "how many of patients 3, 17 and 42 tested positive?", and it adds a little noise to every answer. Is that enough? This is the **reconstruction game** of Threat models and privacy games, where the adversary is scored by the fraction of the secret bits it recovers. One exact count of all n bits tells the attacker only which value is in the majority, and guessing that value for everyone lifts the score from $\frac{1}{2}$ to about $\frac{1}{2} + 1/\sqrt{2\pi n}$. This chapter asks what many counts do.

The one idea is that every answer is a linear equation in the secret bits, so only the error of the answers stands between the attacker and the solution. A wrong guess survives only if the error is as large as the amount by which the guess changes some answer. For a guess with a constant fraction of its bits wrong, that amount is about $\sqrt{n}$ for a random question and about n for the question that names exactly the wrong bits. Errors well below these scales reveal the bits (Dinur and Nissim 2003). Errors that are independent from answer to answer fail too, because averaging many answers cancels them. Dwork and Roth call this the **Fundamental Law of Information Recovery**: overly accurate answers to too many questions destroy privacy.

The running example is the hospital with $n = 500$ patients and an attacker who asks $m = 2000$ random counting questions. With independent normal noise of standard deviation $0.25\sqrt{n} = 5.59$ on each answer, a quarter of the $\sqrt{n}$ scale, a least-squares fit recovers 0.9655 of the bits, where guessing recovers $\frac{1}{2}$.

17.1. Subset sums and the exponential attack

Consider first the strongest attacker, who may ask every possible counting question. How large must the error be before even this attacker cannot pin the bits down?

Definition 17.1 (Subset-sum queries and the curator)**.** The secret database is a bit vector $d = (d_1, \dots, d_n) \in \{0,1\}^n$, one bit per person. A **subset-sum query** is a set $S \subseteq \{1, \dots, n\}$ of people, written as its indicator vector f, with $f_i = 1$ when person i is in S. Its true answer is

$$\sum_{i \in S} d_i = \langle f, d \rangle. \tag{17.1}$$

The attacker asks m queries $f_1, \dots, f_m$, the rows of an $m \times n$ matrix A. The **curator**, the party that holds d, returns $a = Ad + e$, where the error vector e has one entry per query. The error is at most E when $|e_j| \le E$ for every query j. Throughout, the subscript j numbers queries and i numbers people.

Take a wrong guess, a bit vector c. The published answer to a query f is within E of $\langle f, d\rangle$, and would also have to be within E of $\langle f, c\rangle$ if c were the database. So if these differ by more than $2E$, c is ruled out.

Definition 17.2 (Consistent candidates and blatant non-privacy)**.** A candidate $c \in \{0,1\}^n$ is **consistent** with the answers if $|a_j - \langle f_j, c\rangle| \le E$ for every query j. Its number of wrong bits is the **Hamming distance** $\mathrm{dist}(c, d)$, the number of people with $c_i \neq d_i$. A curator is **blatantly non-private** if some attacker, even one that runs in exponential time, recovers all but $o(n)$ of the bits.

The true d is always consistent, so an attacker who asks all 2^n queries can check every candidate against every answer and output any consistent one. The candidate's wrong bits form two subsets, the ones it missed (true ones set to 0) and the ones it invented (zeros set to 1), and each subset is itself a query the attacker has asked.

Theorem 17.1 (The exponential attack)**.** *Suppose the curator answers all 2^n subset-sum queries with error at most E. Then every consistent candidate c satisfies*

$$\mathrm{dist}(c, d) \le 4E. \tag{17.2}$$

Proof.

1. *Count the ones that the candidate missed.* Let S_0 be the set of people with $d_i = 1$ and $c_i = 0$. On S_0, d sums to $|S_0|$ and c sums to 0. Both sums are within E of the one answer to S_0, so by the triangle inequality they are within $2E$ of each other, and $|S_0| \le 2E$.
2. *Count the ones that the candidate invented.* On the set S_1 of people with $d_i = 0$ and $c_i = 1$, the candidate sums to $|S_1|$ and d sums to 0, and the same argument gives $|S_1| \le 2E$.
3. *Add.* Every disagreement lies in S_0 or in S_1, so $\mathrm{dist}(c, d) = |S_0| + |S_1| \le 4E$.

□

The proof never uses that the errors are random, so it holds even against a curator that picks them to make a chosen wrong candidate look consistent, and such a curator attains the bound when $2E$ is a whole number.

Proposition 17.1 (The bound is attained)**.** *Let $k = \lfloor 2E \rfloor$, and suppose d has at least k ones and at least k zeros. Then some curator with error at most E on all 2^n queries makes a candidate at distance $2k$ from d consistent.*

Proof.

1. *Build the decoy.* Let c flip k of the ones of d to 0 and k of the zeros to 1.
2. *Its sums stay close to the true ones.* For any S, the sum of $c_i - d_i$ over S is the number of flipped zeros in S minus the number of flipped ones in S, so it lies between $-k$ and k.
3. *Answer the midpoint.* A curator that answers each query with the midpoint of the two databases' sums is within $k/2 \le E$ of both, so c is consistent.

□

So a curator whose error on all 2^n queries is $o(n)$ is blatantly non-private, because $4E$ is then $o(n)$. A curator can also be weaker than the bound suggests. One that rounds every answer to a multiple of 4 has error at most 2, yet rounding each candidate's answers the same way and keeping exact matches recovers d.

Example 17.1. For a hospital of ten patients, $d = (1, 1, 1, 0, 0, 0, 0, 0, 0, 1)$, the script (Section 17.7) checks all 1024 candidates against all 1024 subsets. Some curator with error at most E makes c consistent exactly when every subset sum of $c - d$ is at most $2E$ in absolute value, by the midpoint construction in one direction and the triangle inequality in the other (Table 17.1).

Table 17.1.: The exponential attack on $n = 10$ bits. Every surviving candidate is within $4E$ of d, and the bound is attained.

Error E	0	0.5	1	1.5	2
Candidates that survive	1	35	242	630	912
Largest distance to d	0	2	4	6	8
Bound $4E$	0	2	4	6	8

The bound is attained at every E, as Proposition 17.1 predicts. At $E = 2$, 912 candidates survive, which says almost nothing. Figure 17.1 draws both rows over a finer grid of E, for a random database of ten bits.

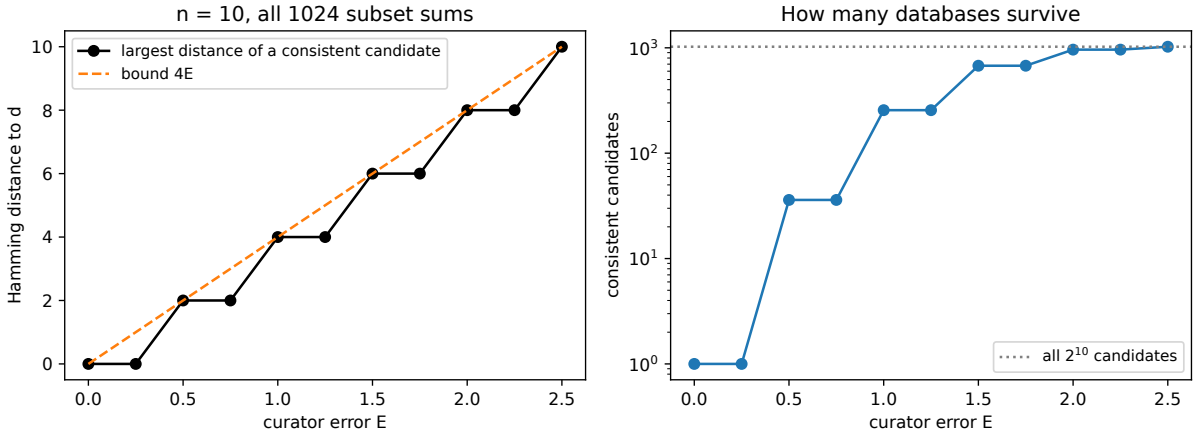


Figure 17.1.: The exponential attack on ten bits. Left: the largest distance from d of a candidate that some curator with error E makes consistent, against $4E$. Right: the number of surviving candidates (log scale).

17.2. The linear-programming decoder

Asking 2^n questions is not realistic, but Dinur and Nissim showed that about $n \ln n$ random subsets suffice, with an efficient decoder, as long as the error is $o(\sqrt{n})$. The threshold drops because a random subset cannot aim at the wrong bits. If a candidate differs from d in k places,

a random subset picks up each with probability $\frac{1}{2}$, so the change in its answer is a sum of about k random terms, which fluctuates by about $\sqrt{k}$, not k. A candidate with a constant fraction of wrong bits has k of order n, hence the scale $\sqrt{n}$.

Searching all 2^n candidates is too slow, so the decoder relaxes each bit to the interval $[0, 1]$. That turns the search for a consistent candidate into a **linear program**, a search for a point that satisfies a list of linear inequalities, which linear programming (LP) solves in polynomial time.

Definition 17.3 (Linear-programming decoder)**.** Given the answers a to m queries $f_1, \dots, f_m$ and the error bound E, the **linear-programming decoder** finds some c with

$$c \in [0,1]^n \quad \text{and} \quad \bigl|\langle f_j, c\rangle - a_j\bigr| \le E \text{ for } j = 1, \dots, m, \tag{17.3}$$

and rounds each c_i to the nearest bit.

The true d is a solution, and Theorem 17.2, after two lemmas, shows that no solution has many wrong bits. Its key step needs one random query to catch any difference vector with many coordinates of size at least $\frac{1}{3}$, by moving its answer far from 0 with constant probability. That is an **anti-concentration** statement, a lower bound on how often a variable is not close to zero. The tool is the Paley–Zygmund inequality, Lemma 17.1 below, applied to the squared answer. It needs the answer's fourth moment to be at most a constant times its squared second moment.

A random subset is a constant plus half a sum of random signs (proof of Lemma 17.2), so the fourth moment comes from one fact about such sums. Take independent fair signs s_i, each $+1$ or -1 with probability $\frac{1}{2}$, and fixed weights w_i. The sum $X = \sum_i s_i w_i$ has odd moments 0, because flipping every sign leaves its distribution unchanged. Its fourth moment is at most 3 times its squared second moment, as for a normal variable:

$$\mathbb{E}[X^2] = \sum_i w_i^2, \qquad \mathbb{E}[X^4] \le 3\,\mathbb{E}[X^2]^2. \tag{17.4}$$

Expanding X^4, a product of four signs has mean 1 when its indices pair up and 0 otherwise, which gives exactly $3(\sum_i w_i^2)^2 - 2\sum_i w_i^4$, so dropping the negative term gives the inequality.

Markov's inequality (Theorem 2.1) says that a variable that cannot be negative is rarely far *above* its mean, while the attack needs it to be rarely far *below*. The next lemma gives that whenever the second moment is not much larger than the squared mean.

Lemma 17.1 (Paley–Zygmund inequality)**.** *Let $W \ge 0$ be a random variable with finite second moment, and let $0 \le \lambda < 1$. Then*

$$\Pr\left[W > \lambda\,\mathbb{E}[W]\right] \ge (1-\lambda)^2 \frac{\mathbb{E}[W]^2}{\mathbb{E}[W^2]}. \tag{17.5}$$

Proof. The plan is to split the mean of W between the event and its complement, and to bound the part on the event with Cauchy–Schwarz.

1. *Split the mean.* Write B for the event $W > \lambda\,\mathbb{E}[W]$. Then $\mathbb{E}[W]$ is the sum of $\mathbb{E}[W\mathbf{1}[B^c]]$ and $\mathbb{E}[W\mathbf{1}[B]]$.
2. *Bound the complement.* On the complement $W \le \lambda\,\mathbb{E}[W]$, so the first part is at most $\lambda\,\mathbb{E}[W]$.

3. *Bound the event.* The Cauchy–Schwarz inequality $\mathbb{E}[UV] \le \sqrt{\mathbb{E}[U^2]\,\mathbb{E}[V^2]}$, with $U = W$ and $V = \mathbf{1}[B]$, for which $\mathbb{E}[V^2] = \Pr[B]$, gives

$$\mathbb{E}[W\mathbf{1}[B]] \le \sqrt{\mathbb{E}[W^2]\Pr[B]}. \tag{17.6}$$

4. *Solve for the probability.* Steps 1 to 3 give $(1-\lambda)\mathbb{E}[W] \le \sqrt{\mathbb{E}[W^2]\Pr[B]}$. Squaring and dividing by $\mathbb{E}[W^2]$, which is positive unless W is always 0, gives Equation 17.5.

□

Think of z as the difference between d and a candidate with many wrong bits. A wrong bit makes a coordinate of size at least $\frac{1}{2}$, and the theorem's proof snaps candidates to a grid, which can shrink that size to $\frac{1}{3}$.

Lemma 17.2 (One random query catches a bad difference)**.** *Let $z \in \mathbb{R}^n$ have at least γn coordinates with $|z_i| \ge \frac{1}{3}$, for a fraction γ, and let f be a uniformly random subset, which includes each person independently with probability $\frac{1}{2}$. Then*

$$\Pr\left[|\langle f, z\rangle| > \frac{\sqrt{\gamma n}}{12}\right] \ge \frac{3}{16}. \tag{17.7}$$

Proof. The plan is to apply Lemma 17.1 to the squared answer, with Equation 17.4 supplying its moments, and then to let the large coordinates set the scale. Write $Y = \langle f, z\rangle$ for the answer, and write the random subset as $f_i = (1 + s_i)/2$ with independent fair signs s_i.

1. *Split the sum into a constant and a random-sign part.* $Y = \mu + X$, with the constant $\mu = \frac{1}{2}\sum_i z_i$ and the random-sign sum $X = \sum_i s_i z_i/2$, whose weights are $w_i = z_i/2$.
2. *Compute the second moment.* X has mean 0, and by Equation 17.4 its variance, written τ^2, is $\frac{1}{4}\sum_i z_i^2$. The cross term $2\mu X$ has mean 0, so $\mathbb{E}[Y^2] = \mu^2 + \tau^2$.
3. *Bound the fourth moment.* Expanding $(\mu + X)^4$, the terms with the odd moments of X vanish, the cross term has mean $6\mu^2\tau^2$, and Equation 17.4 gives $\mathbb{E}[X^4] \le 3\tau^4$. Raising μ^4 to $3\mu^4$ only increases the total and completes the square, so

$$\mathbb{E}[Y^4] \le 3(\mu^2 + \tau^2)^2. \tag{17.8}$$

4. *Apply Paley–Zygmund.* Step 3 makes the ratio $\mathbb{E}[Y^2]^2/\mathbb{E}[Y^4]$ at least $\frac{1}{3}$, so Equation 17.5 with $W = Y^2$ gives

$$\Pr\left[Y^2 > \lambda(\mu^2 + \tau^2)\right] \ge \frac{(1-\lambda)^2}{3}. \tag{17.9}$$

5. *Drop the mean.* Since $\mu^2 + \tau^2 \ge \tau^2$, the event of step 4 implies $|Y| > \sqrt{\lambda}\,\tau$, which therefore has the same lower bound. A non-zero mean only helps, because it pushes Y away from 0.
6. *Use the large coordinates.* At least γn coordinates have $z_i^2 \ge \frac{1}{9}$, so $\tau^2 \ge \gamma n/36$ and $\tau \ge \sqrt{\gamma n}/6$.
7. *Choose $\lambda = \frac{1}{4}$.* Then $\sqrt{\lambda}\,\tau \ge \sqrt{\gamma n}/12$, and the probability is at least $(3/4)^2/3 = 3/16$, which is Equation 17.7.

□

A candidate with more than γn wrong bits solves the linear program only if no query moves its answer past $\sqrt{\gamma n}/12$, which has probability at most $(13/16)^m$. Candidates are infinitely many, so the proof snaps them to a finite grid, which costs the $\frac{1}{2}$ and the $n\ln(n+1)$ in the theorem.

Theorem 17.2 (Dinur–Nissim, in the form proved here)**.** *Fix d, a fraction γ and a failure probability β, let $n \geq 3$, and let the m queries be independent uniformly random subsets. Suppose the curator's error is at most E, with $2E + \frac{1}{2} \leq \sqrt{\gamma n}/12$, and*

$$m \geq \frac{n\ln(n+1) + \ln(1/\beta)}{\ln(16/13)}. \tag{17.10}$$

Then with probability at least $1 - \beta$ over the queries, every solution of the linear program rounds to a vector with at most γn wrong bits, whatever errors the curator chose, even after seeing the queries.

Proof. The plan is the one just sketched: snap to a grid, catch each bad grid point, then take a union bound.

1. *Consistent candidates differ from d by little on every query.* Both c and d are within E of every answer, so by the triangle inequality $|\langle f_j, c - d\rangle| \leq 2E$ for every query j.
2. *A wrong bit is a large coordinate.* Rounding c_i gives the wrong bit only if $|c_i - d_i| \geq \frac{1}{2}$. So more than γn wrong bits means more than γn coordinates of $c - d$ of size at least $\frac{1}{2}$.
3. *Move to a finite set.* Snap each c_i to the nearest multiple of $1/n$, a finer step than rounding to bits, giving a grid point c'. A subset sum adds at most n coordinates, each moved by at most $\frac{1}{2n}$, so it moves by at most $\frac{1}{2}$, and $z' = c' - d$ has $|\langle f_j, z'\rangle| \leq 2E + \frac{1}{2}$ for every j. It still has more than γn coordinates of size at least $\frac{1}{2} - \frac{1}{2n}$, which is at least $\frac{1}{3}$ because $n \geq 3$.
4. *One grid point passes all queries only with small probability.* Fix a grid point whose z' has more than γn large coordinates. By Lemma 17.2, each query has $|\langle f_j, z'\rangle| > \sqrt{\gamma n}/12$ with probability at least $\frac{3}{16}$, and by the assumption on E such a query violates step 3. The queries are independent, so z' passes all m of them with probability at most $(13/16)^m$.
5. *Union bound over the grid.* There are at most $(n+1)^n$ grid points, since each coordinate takes one of $n+1$ values. By the union bound (Lemma 2.3), the chance that one of several events happens is at most the sum of their chances, so some bad grid point passes every query with probability at most $(n+1)^n(13/16)^m$, and taking logarithms shows that this is at most β exactly when Equation 17.10 holds.
6. *Conclude.* Outside that event no bad grid point passes, so by step 3 no solution with more than γn wrong bits exists. The event depends only on the queries, so the guarantee holds for every curator at once.

□

Letting γ shrink slowly to 0 as n grows, error $o(\sqrt{n})$ on $O(n\log n)$ random subset sums lets the attacker recover all but $o(n)$ bits in polynomial time. Step 5 of Lemma 17.2 relies on the fluctuation alone, so it also covers the hard case of a balanced difference, whose positive and negative entries cancel in the mean (Figure 17.2).

The constants are poor, but the scaling in n is right, and sharper analyses need only $O(n)$ queries. A variant that minimizes the total absolute error $\|Ac - a\|_1$ tolerates a constant fraction of arbitrarily wrong answers (Dwork, McSherry and Talwar 2007).

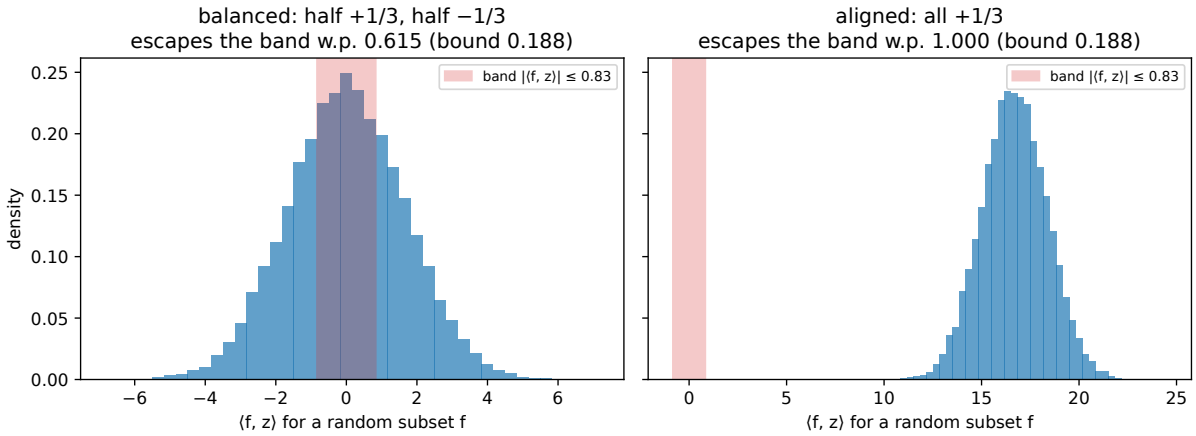


Figure 17.2.: A random subset sum of a difference vector with 100 entries of size $\frac{1}{3}$ ($n = 1000$), over 20 000 random subsets, against the lemma's band ± 0.83 (red). Left: half the entries positive and half negative. Right: all entries positive.

Example 17.2. Take a hospital of $n = 1000$ patients and target $\gamma = 0.1$, at most 100 wrong bits. The lemma's threshold is $\sqrt{100}/12 = 0.8333$. Over 100 000 random subsets, more than Figure 17.2 draws, the answer leaves that band with probability 0.6161 for the balanced vector and 1.0000 for the one with all entries positive, against the guaranteed 0.1875.

The theorem's condition allows $E \le \sqrt{\gamma n}/24 - \frac{1}{4} = 0.1667$. With $\beta = 0.01$, Equation 17.10 asks for

$$m \ge \frac{1000 \ln 1001 + \ln 100}{\ln(16/13)} = 33\,295.04, \tag{17.11}$$

so at least 33 296 queries, about $33n$, while the least-squares decoder of the next section works with $4n$.

17.3. Least squares under worst-case and random error

The linear program needs a solver and a value of E. Least squares needs neither, and its estimate is linear in the answers, so its error is linear in the curator's and can be analysed exactly against both adversarial and random errors.

Definition 17.4 (Least-squares decoder)**.** Let $m \ge n$ and let A have full column rank. The **least-squares decoder** picks the vector whose answers come closest to the published ones in squared distance,

$$\hat{x} = \arg\min_{x} \|Ax - a\|^2 = (A^\top A)^{-1} A^\top a, \tag{17.12}$$

then clips each coordinate to $[0, 1]$ and rounds it to the nearest bit.

How much the decoder amplifies the curator's error depends on how much A can shrink a vector. The **smallest singular value** $s_{\min}(A)$ is the smallest length $\|Ax\|$ over unit vectors x. Undoing A, as the decoder does, stretches a vector by at most $1/s_{\min}(A)$. Each wrong bit is a coordinate off by at least $\frac{1}{2}$, so it adds at least $\frac{1}{4}$ to the squared length of the decoder's error.

Proposition 17.2 (Least squares under worst-case error)**.** *If the curator's error is at most E on each of the m queries, the least-squares decoder gets at most*

$$\frac{4\|e\|^2}{s_{\min}(A)^2} \le \frac{4mE^2}{s_{\min}(A)^2} \tag{17.13}$$

bits wrong.

Proof. Bound the length of the decoder's error, then count how many coordinates it can push past $\frac{1}{2}$.

1. *The decoder's error is linear in the curator's.* Substituting $a = Ad + e$ gives $\hat{x} = d + A^+e$, with the pseudo-inverse $A^+ = (A^\top A)^{-1}A^\top$.
2. *Bound its size.* In the singular value decomposition $A = U\,\mathrm{diag}(s_1, \dots, s_n)\,V^\top$, with orthonormal columns in U and V, the pseudo-inverse is $V\,\mathrm{diag}(1/s_1, \dots, 1/s_n)\,U^\top$. Its largest singular value is $1/s_{\min}(A)$, so $\|\hat{x} - d\| \le \|e\|/s_{\min}(A)$.
3. *Clipping only helps.* Each d_i lies in $[0, 1]$, so moving $\hat{x}_i$ to the nearest point of $[0, 1]$ cannot move it away from d_i.
4. *Count the wrong bits.* A wrong bit has $|\hat{x}_i - d_i| \ge \frac{1}{2}$, so it contributes at least $\frac{1}{4}$ to $\|\hat{x} - d\|^2$. There are therefore at most $4\|e\|^2/s_{\min}(A)^2$ of them, and $\|e\|^2 \le mE^2$.

□

The bound is useful only if $s_{\min}(A)$ is of order $\sqrt{m}$, so that m cancels. For rows of independent entries with mean 0 and variance 1, $s_{\min}$ concentrates near $\sqrt{m} - \sqrt{n}$ (Vershynin 2018, ch. 4). An entry of a random subset is $\frac{1}{2}$ plus a random sign times $\frac{1}{2}$, and its common mean affects only one direction, which suggests $s_{\min}(A) \approx \frac{1}{2}(\sqrt{m} - \sqrt{n})$. Appendix Section H.2 proves $s_{\min}(A) \ge \sqrt{m}/14$ once m is of order $n \ln n$, which gives at most $4 \times 14^2 E^2 = 784E^2$ wrong bits, so error $o(\sqrt{n})$ leaves $o(n)$ wrong bits.

Now let the errors be independent $\mathcal{N}(0, \sigma^2)$ instead. The decoder's error is then normal. Its variance per coordinate is about σ^2 divided by the number $m - n$ of equations beyond the n needed, because extra equations average the noise. It is multiplied by 4, because one person's membership in a random query, 0 or 1 with equal chance, has variance only $\frac{1}{4}$. The exact constant needs the average inverse of $A^\top A$. For m independent normal rows with mean 0 and second-moment matrix $\Sigma = \mathbb{E}[ff^\top]$, the sum $A^\top A$ is a **Wishart matrix**, and its inverse has mean $\Sigma^{-1}/(m - n - 1)$ (Muirhead 1982, ch. 3). Appendix Section H.1 derives this and finds 0/1 rows within about 3% of it, so the next result assumes it.

Proposition 17.3 (Least squares under random noise)**.** *Let the m queries be uniformly random subsets and the errors independent $\mathcal{N}(0, \sigma^2)$. Suppose that $(A^\top A)^{-1}$ has the mean it would have for normal rows with mean 0 and the same $\Sigma = \mathbb{E}[ff^\top]$. Then each coordinate of $\hat{x} - d$ is approximately $\mathcal{N}(0, \frac{4\sigma^2}{m-n})$, and with Φ the standard normal cumulative distribution function, the expected fraction of bits recovered is approximately*

$$\Phi\left(\frac{\sqrt{m-n}}{4\sigma}\right). \tag{17.14}$$

Proof. First find the variance of each coordinate of $\hat{x} - d$, then ask how often that normal error crosses $\frac{1}{2}$.

1. *The error is normal.* Given A, the error $A^{+}e$ is a linear map of a normal vector, so it is normal. A linear map G turns covariance $\sigma^2 I$ into $\sigma^2 GG^{\top}$, so the covariance is $\sigma^2 A^{+}(A^{+})^{\top} = \sigma^2 (A^{\top}A)^{-1}$.
2. *Compute the average Gram matrix.* $A^{\top}A$ is the sum of $f_j f_j^{\top}$ over the queries. For a random subset, $f_i^2 = f_i$ has mean $\frac{1}{2}$, and two different memberships are independent, so $\mathbb{E}[f_i f_k] = \frac{1}{4}$ for $i \neq k$. Hence
$$\mathbb{E}[A^{\top}A] = \frac{m}{4}(I + \mathbf{1}\mathbf{1}^{\top}). \tag{17.15}$$
3. *Invert it.* Multiplying out shows that $(I + \mathbf{1}\mathbf{1}^{\top})^{-1} = I - \frac{1}{n+1}\mathbf{1}\mathbf{1}^{\top}$. Its diagonal is $\frac{n}{n+1} \approx 1$, so inverting the mean of $A^{\top}A$ would give a variance of about $4\sigma^2/m$, which step 4 corrects.
4. *Correct for finite m.* The inverse of a fluctuating matrix is on average larger than the inverse of its mean, as $\mathbb{E}[1/Z] \geq 1/\mathbb{E}[Z]$ for a positive number Z. The hypothesis says that the mean of $(A^{\top}A)^{-1}$ is $\Sigma^{-1}/(m-n-1)$, while step 2 gave $\mathbb{E}[A^{\top}A] = m\Sigma$, so m is replaced by $m-n-1$ and the variance is
$$4\sigma^2 \cdot \frac{n}{n+1} \cdot \frac{1}{m-n-1} \approx \frac{4\sigma^2}{m-n}. \tag{17.16}$$
5. *Find when a bit goes wrong.* Clipping to $[0,1]$ never moves a coordinate across $\frac{1}{2}$, so it can be ignored. A 0 rounds to 1 when its error exceeds $\frac{1}{2}$, and a 1 rounds to 0 when its error is below $-\frac{1}{2}$, so only one tail counts. The error has standard deviation $2\sigma/\sqrt{m-n}$, so a bit goes wrong with probability $\Phi(-\sqrt{m-n}/(4\sigma))$. The symmetry $1 - \Phi(-t) = \Phi(t)$ gives Equation 17.14.

□

With m a fixed multiple of n, the prediction depends only on $\sigma/\sqrt{n}$ (Figure 17.3). Noise of order $\sqrt{n}$, the threshold of Theorem 17.2, does not stop least squares either (Example 17.3). At a fixed σ, more queries average the noise away, so to hold the attack at a fixed accuracy the noise must grow like $\sqrt{m}$ (Figure 17.4).

Worst-case errors hurt more than random ones of the same size, because a curator can push every answer along the direction that least squares amplifies most. The bound Equation 17.13 is loose by more than a factor of 10 in Example 17.3 below, but it is the only statement that holds for every curator.

Example 17.3. In the running example, $n = 500$ and $m = 2000$, so $\sqrt{n} = 22.36$, and the script averages over four random databases and query matrices (Table 17.2).

Table 17.2.: Least-squares reconstruction of $n = 500$ bits from $m = 2000$ random subset sums with Gaussian noise.

$\sigma/\sqrt{n}$	0.1	0.25	0.5	1	2
σ	2.24	5.59	11.18	22.36	44.72
Recovered, simulated	1.0000	0.9655	0.8130	0.6550	0.5875

$\sigma/\sqrt{n}$	0.1	0.25	0.5	1	2
Recovered, Equation 17.14	1.0000	0.9585	0.8069	0.6676	0.5858

The prediction is within 0.013 of every simulated value. For worst-case errors, the script measures $s_{\min}(A) = 11.43$, against the approximation $\frac{1}{2}(\sqrt{2000} - \sqrt{500}) = \frac{1}{2}(44.72 - 22.36) = 11.18$. The script's worst-case curator adds to each answer E times the sign of the matching entry of Av, where v is the direction that A shrinks most. At $E = 1$, 2 and 5.59 the bound Equation 17.13 allows 61.2, 245.0 and 1914.0 wrong bits, and the curator causes 0, 20 and 143. Gaussian noise with $\sigma = 5.59$ causes only about 17.

17.4. The Fundamental Law of Information Recovery

Two questions remain before the attacks add up to a law. Is error of order $\sqrt{n}$ enough to stop reconstruction, or is it just where the proofs stop? And how many queries does any attack need?

Definition 17.5 (Sampling curator)**.** The **sampling curator** fixes a sampling rate q and draws a secret set T of people, keeping each person independently with probability q. It answers the query that names the set S_j with

$$a_j = \frac{1}{q} \sum_{i \in S_j \cap T} d_i. \tag{17.17}$$

Dividing by q makes each answer correct on average, and its error, a sum of up to n independent terms, is of order $\sqrt{n}$. Yet the people left out of T are hidden completely.

Proposition 17.4 (What the sampling curator hides)**.** *With probability at least $1 - \beta$, all m answers of the sampling curator have error at most*

$$\max_j |a_j - \langle f_j, d \rangle| \le \sqrt{\frac{n \ln(2m/\beta)}{2q^2}}. \tag{17.18}$$

If d is uniform on $\{0,1\}^n$, no attacker recovers an expected fraction of more than $q + \frac{1-q}{2}$ of the bits.

Proof. Hoeffding's inequality bounds one answer's error and the union bound all of them, and then a person outside T turns out to be a fair coin to the attacker.

1. *Write the error as a sum.* With $t_i = \mathbf{1}[i \in T]$, the error of query j is the sum of $d_i(t_i/q - 1)$ over $i \in S_j$. Only the $k \le n$ people with $d_i = 1$ in S_j contribute, with independent terms that have mean 0 and lie in an interval of length $\frac{1}{q}$.

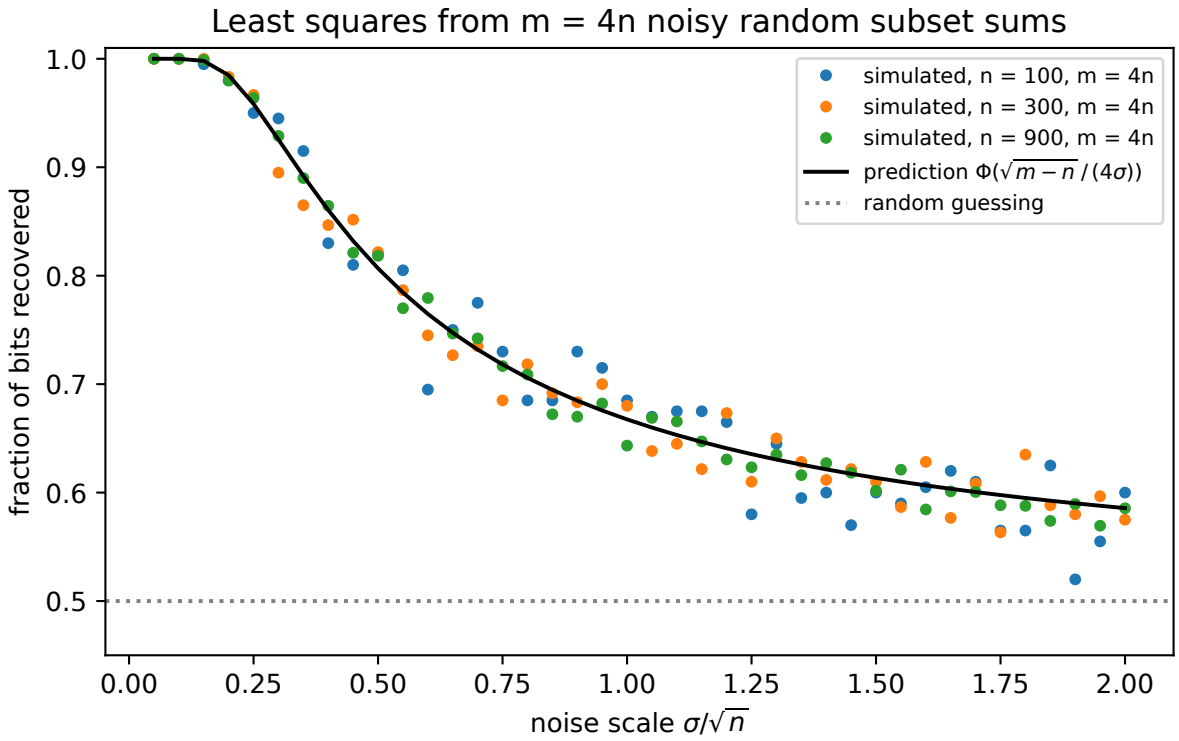


Figure 17.3.: Fraction of bits recovered by least squares from $m = 4n$ noisy random subset sums, against the noise in units of $\sqrt{n}$, for three database sizes. The black curve is Equation 17.14, and the curves coincide.

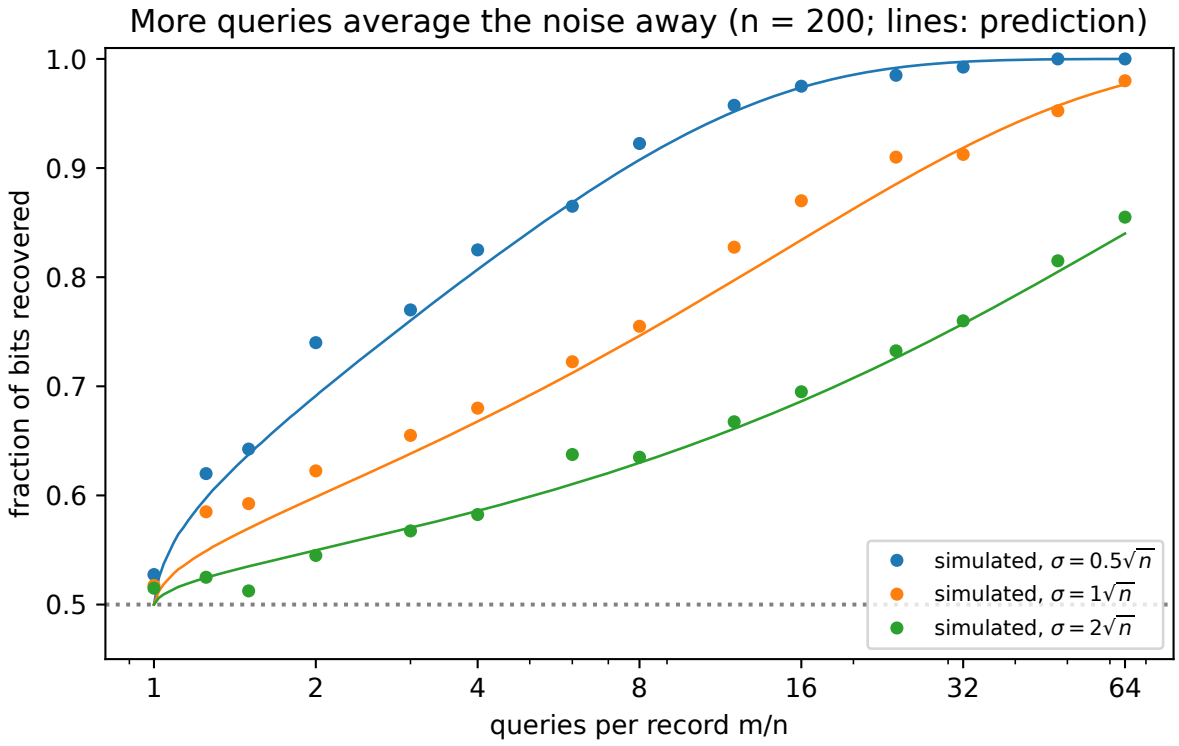


Figure 17.4.: Fraction of bits recovered by least squares at $n = 200$ as the number of queries grows, for three noise levels. Lines are Equation 17.14; dots are simulations.

2. *Bound one error.* Hoeffding's inequality (Theorem 2.5) says that the mean of such terms exceeds its expectation by t with probability at most $\exp(-2kt^2q^2)$. The sum deviates by u exactly when the mean deviates by u/k, which gives $\exp(-2q^2u^2/k)$. Since $k \le n$ this is at most $\exp(-2q^2u^2/n)$, and the lower tail doubles it:

$$\Pr\left[|\text{error}| \ge u\right] \le 2\exp(-2q^2u^2/n). \tag{17.19}$$

3. *Bound all errors.* By the union bound (Lemma 2.3), some answer has error at least u with probability at most $2m\exp(-2q^2u^2/n)$. Setting this equal to β and solving for u gives Equation 17.18.
4. *The answers ignore the people outside T.* They depend only on T and on the bits of the people in it, so for $i \notin T$ any guess of the fair bit d_i is right with probability $\frac{1}{2}$.
5. *Average over T.* Each person is in T with probability q, where at best the attacker is always right, and outside with probability $1-q$, where it is right half the time, so the expected fraction is at most $q + \frac{1-q}{2}$.

□

At $q = \frac{1}{2}$ the ceiling is three quarters, with error of order $\sqrt{n \ln m}$, only a logarithmic factor above the $o(\sqrt{n})$ the attacks need. But this is not privacy, because the people in T enter every answer without noise, so for them the answers are exact linear equations, which least squares with enough queries solves.

To bound the number of queries any attack needs, count bits. The secret is n uniform bits, and an answer with at most V possible values carries at most $\log_2 V$ bits. By Fano's inequality, a guess that is wrong on a fraction γ of the bits leaves at most the uncertainty of a coin that lands heads with probability γ, its **binary entropy** $h(\gamma) = -\gamma\log_2\gamma - (1-\gamma)\log_2(1-\gamma)$ per bit. So the answers must carry at least $n(1-h(\gamma))$ bits.

Theorem 17.3 (The information bound)**.** *Let d be uniform on $\{0,1\}^n$, fix the m queries in advance, and let each answer take at most V values. If an attacker's guess $\hat{d}$, computed from the answers, has an expected fraction $\gamma \le \frac{1}{2}$ of wrong bits (where h still increases), then*

$$m \ge \frac{n\,(1-h(\gamma))}{\log_2 V}. \tag{17.20}$$

Proof. The plan is the count of bits above, made exact. Write Y for the answers and use the entropy H and mutual information I of Information theory.

1. *Bound what the answers can carry.* Mutual information is at most the entropy of the output, a tuple's entropy is at most the sum of its entries' entropies, and a variable with V values has entropy at most $\log_2 V$ (Theorem 5.1). So

$$I(d;Y) \le H(Y) \le m\log_2 V. \tag{17.21}$$

2. *Processing cannot add information.* The guess is computed from Y, so $I(d;\hat{d}) \le I(d;Y)$ by the data-processing inequality (Theorem 5.4).

3. *Write what a good guess requires.* Since d has entropy n, $I(d;\hat{d}) = n - H(d \mid \hat{d})$. The chain rule (Theorem 5.2) splits $H(d \mid \hat{d})$ into the terms $H(d_i \mid \hat{d}, d_1, \dots, d_{i-1})$, one per bit. Conditioning on less cannot reduce entropy (Corollary 5.1), so each term is at most $H(d_i \mid \hat{d}_i)$, and
$$H(d \mid \hat{d}) \le \sum_i H(d_i \mid \hat{d}_i). \tag{17.22}$$
4. *Apply Fano's inequality to each bit.* With two values, Fano's inequality (Theorem 5.7) reads $H(d_i \mid \hat{d}_i) \le h(P_i)$, where P_i is the probability that the guess of bit i is wrong.
5. *Average with concavity.* The binary entropy is concave, so by Jensen's inequality the average of the $h(P_i)$ is at most $h(\gamma)$, where γ is the average of the P_i.
6. *Combine.* Steps 3 to 5 give $I(d;\hat{d}) \ge n(1 - h(\gamma))$, and steps 1 and 2 give $I(d;\hat{d}) \le m \log_2 V$.

□

The attacks use about as few queries as possible, since exact counts take $V = n + 1$ values, so any attack needs of order $n/\ln n$ of them, and least squares with exact answers needs only n queries whose matrix has full rank. The law has two halves, then, and a defence must control both. "Overly accurate" means error below $\sqrt{n}$ for random queries, or n for all queries, and "too many" means enough queries to average independent noise away.

Example 17.4. In the running example the sampling curator with $q = \frac{1}{2}$ keeps 250 of the 500 people. Its largest error over the 2000 queries is 28.00, or $1.25\sqrt{n}$, well under the Hoeffding bound of 106.25 at $\beta = 0.05$. Least squares on these answers recovers 0.7560 of the bits, within sampling error of the ceiling $\frac{3}{4}$ on the expected fraction.

For the information bound, take $n = 1000$ and at most 5% wrong bits. Then $h(0.05) = 0.2864$, so the answers must carry $1 - 0.2864 = 0.7136$ bits per person. An exact answer carries at most $\log_2 1001 \approx 9.97$ bits, so any attack needs at least 72 queries, from the bound 71.6. Rounding every count to a multiple of 10 leaves at most 102 values and raises the bound to 106.9, and multiples of 32 give 141.5.

A curator with $\sigma = 2\sqrt{n}$ lets least squares recover only 0.5875 of the bits at $m = 4n$ (Table 17.2), but by Equation 17.14 $106.1n$ queries recover 90%.

17.5. Attacking a deployed query system

Do the attacks survive a system built to resist them? In 2017 Aircloak offered a bounty for re-identifying data behind Diffix, its commercial query system, which answered unlimited counting queries in a restricted form of SQL. Each condition in a query, such as "status is C", added its own layer of Gaussian noise with standard deviation 1. The noise was **sticky**, computed from the query itself, so asking a query twice returned the same answer. And the language forbade OR, so a random subset of people, which only a long list of identifiers joined by OR describes, could not be asked. Cohen and Nissim (2020) instead drew random short descriptions and took whatever sets they selected.

Definition 17.6 (Digit-predicate query)**.** Each record has a public identifier i, a whole number. A **digit-predicate query** is fixed by a prime p, a digit position, a fractional exponent x and a modulus r. It selects record i when the digit at that position after the decimal point of $(p\,i)^x$ is divisible by r.

Diffix counted the whole digit test as one condition, so it added one layer of noise. The first 25 primes, the digit positions 1 to 5, the 14 exponents $0.5, 0.6, \ldots, 1.9$ other than 1, and the moduli 2 and 5 make 3500 queries. A digit is divisible by 2 for five of its ten values and by 5 for two, so the fraction ρ of records that a query includes is about $\frac{1}{2}$ or $\frac{1}{5}$. If the selections behave like coin flips, least squares works as before, with $\rho(1-\rho)$ in place of $\frac{1}{4}$.

Proposition 17.5 (Least squares with inclusion rates)**.** *Let query j include each record independently with probability ρ_j, write v for the average of $\rho_j(1-\rho_j)$ over the m queries, and let each answer carry independent noise of standard deviation σ. Suppose, as in Proposition 17.3, that $(A^\top A)^{-1}$ has the mean it would have for normal rows. Then each coordinate of the least-squares error has standard deviation about $\sigma/\sqrt{v(m-n-1)}$, keeping the $m-n-1$ that Proposition 17.3 rounded to $m-n$, and the expected fraction of bits recovered is about*

$$\Phi\left(\frac{\sqrt{v\,(m-n-1)}}{2\sigma}\right). \tag{17.23}$$

Proof. The proof of Proposition 17.3 used the inclusion probability $\frac{1}{2}$ only in the average Gram matrix, so only that part changes.

1. *The average Gram matrix of one query.* A query with rate ρ has $\mathbb{E}[f_i^2] = \rho$ and $\mathbb{E}[f_i f_k] = \rho^2$ for $i \neq k$, so $\mathbb{E}[f f^\top] = \rho(1-\rho)I + \rho^2 \mathbf{1}\mathbf{1}^\top$.
2. *Add up the queries.* Summing gives $\mathbb{E}[A^\top A] = m(vI + \overline{\rho^2}\,\mathbf{1}\mathbf{1}^\top)$, where $\overline{\rho^2}$ is the average of ρ_j^2.
3. *Invert it.* As in step 3 of Proposition 17.3, the all-ones part changes each diagonal entry of the inverse by a fraction below $\frac{1}{n}$, as the factor $\frac{n}{n+1}$ did there, so each is about $1/(mv)$.
4. *Correct for finite m and finish.* As in steps 4 and 5 of Proposition 17.3, the inverse-Wishart mean replaces m by $m-n-1$, which gives the variance $\sigma^2/(v(m-n-1))$, and a bit goes wrong when its error passes $\frac{1}{2}$.

□

Digits are a fixed function of i, not coins, so independence is an assumption that the example checks. Diffix's noise was not small by this chapter's standard, because each attack query also had conditions on a range of identifiers and on the secret loan status, and Cohen and Nissim report a total standard deviation of 4. For a range of $n = 73$ records that is $0.47\sqrt{n}$, and what defeated it was the number of queries. The noise did not grow with m, so 3,500 distinct queries averaged it away, and sticky noise did not help because the attack never repeats a query.

Restricting the query language did not help either, because any short formula that splits the records independently of their secrets serves as a random query. Keeping least squares down to 75% of the 73 bits with 3,500 queries takes a standard deviation of about 19.5, 4.9 times what Diffix added, and the noise must keep growing like $\sqrt{m}$. The privacy definition of Part 3 makes that growth automatic, because it charges every answer against a fixed total allowance of privacy loss.

Example 17.5. Cohen and Nissim asked the 3,500 queries about the loans of a public banking dataset in each of four ranges of identifiers, holding between 73 and 142 loans, and the absolute-error linear program reconstructed all four ranges perfectly. The script repeats it with least squares on 73 synthetic identifiers, each with a fair secret bit, answering all 3,500 queries with Gaussian noise of standard deviation 4. Averaging $\rho(1-\rho)$ over the queries gives $v = 0.2016$. Over 50 runs the attack recovered every bit in 49, an average of 0.02 wrong bits per run, while Equation 17.23 predicts 0.036. With 142 records it was perfect in 45 of 50 runs, with 0.10 wrong bits per run against 0.077 predicted. So the digit predicates are close enough to coin flips, and even with noise of $2\sqrt{n} = 17.1$ the prediction for 3,500 queries is 0.779 of the bits (Figure 17.5).

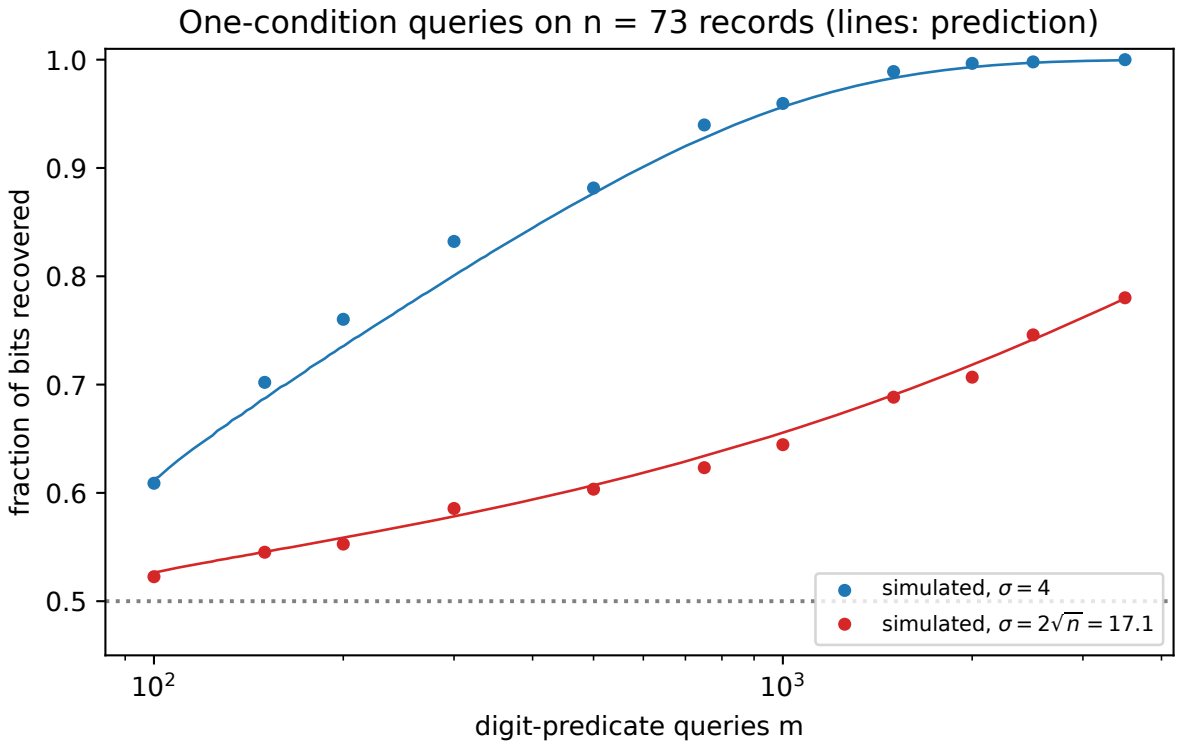


Figure 17.5.: Fraction of 73 secret bits recovered by least squares from the first m of the 3,500 digit-predicate queries, for noise of standard deviation 4 and $2\sqrt{n}$, averaged over 20 runs. Lines are Equation 17.23.

17.6. Reconstructing a census block

National statistical offices publish many tables of counts, medians and means for every block, all computed from the same confidential records. Each cell is a query, so do the tables determine the records? Garfinkel, Abowd and Martindale (2019) showed with a toy block of seven people that they can, and this section repeats the experiment on a block of its own.

The fictional block has seven people, each with an age from 0 to 99, a sex, a race (Black or White) and a marital status, and nobody under 18 is married. Table 17.3 is everything the agency

publishes. Means are given to two decimals, which recovers each sum of ages exactly because the sum is a whole number. For example $34.29 \times 7 = 240.03$, so the total is 240.

Table 17.3.: The published statistics for a fictional census block of seven people. A median over an even count is the average of the two middle ages.

Group	Count	Median age	Mean age
Total	7	33	34.29
Female	4	31	30.25
Male	3	33	39.67
Black	4	33.5	36.25
White	3	33	31.67
Married	4	43.5	47.75

Proposition 17.6 (The block is determined)**.** *Exactly one block of seven people is consistent with Table 17.3, and it is the true block. Its women are aged 7, 27, 35 and 52 and its men 15, 33 and 71, the people aged 7, 15, 52 and 71 are Black, and those aged 33, 35, 52 and 71 are married.*

Proof. Enumerate the age lists that fit the sex statistics, then discard those with no race split or no marital split that fits the table. Steps 4 to 7 are done by the script's exhaustive search.

1. *Turn means into sums.* Mean times count gives the sums of ages: total 240, female 121, male 119, Black 145, White 95, married 191.
2. *Enumerate the women's ages.* With four women the two middle ages sum to $2 \times 31 = 62$, so the youngest and oldest sum to 59. The order of the ages requires the second youngest b to be at most 31. The oldest, 59 minus the youngest, must be at least the third age $62 - b$, so the youngest is at most $b - 3$. For each b from 3 to 31 that leaves $b - 2$ choices, so there are $1 + 2 + \cdots + 29 = 435$ age lists.
3. *Enumerate the men's ages.* Three ages with median 33 and sum 119 have the other two summing to 86. Every youngest age from 0 to 33 gives an oldest between 53 and 86, so there are 34 lists.
4. *Combine and check the total median.* The $435 \times 34 = 14\,790$ combined age vectors include 12 906 that have overall median 33.
5. *Check the race split.* For each age vector, try the $\binom{7}{4} = 35$ ways of choosing 4 of the 7 people as Black. Only 387 age vectors admit a choice with the published Black and White medians and sums.
6. *Check the marital split* in the same way, with married median 43.5, married sum 191 and every married person at least 18. Only 33 age vectors admit one.
7. *Require all three at once.* Exactly one age vector admits a valid sex split, race split and marital split, and its splits are unique, which gives the block of the statement.

□

No single statistic determines the block, only their intersection, and real tables cross-classify many variables, so the same people appear in thousands of cells. The Census Bureau's own reconstruction of the 2010 Census rebuilt a record for each of the 308,745,538 people counted, and

about 46% matched a real person exactly on block, sex, age, race and ethnicity (Abowd 2021). The Bureau then based its method for the 2020 Census on the privacy definition of Part 3.

Example 17.6. To test whether withholding one number helps, Figure 17.6 repeats the search with each of the twelve medians and means withheld in turn. Eight leave the block uniquely determined, because the rest of the table pins it down, and Table 17.4 lists the other four.

Table 17.4.: Blocks consistent with the published table when one statistic is withheld, and the share of them whose sex, race and marital labels are the true ones.

Withheld	Consistent blocks	Distinct sex/race/marital patterns	Share with the true pattern
Female median	7	6	0.14
Black median	36	3	0.64
Married median	26	4	0.46
Married mean	45	7	0.51
Any of the other eight	1	1	1.00

Withholding the married mean, the only statistic that splits the total age between married and single people, leaves the most consistent blocks, and still only 45 remain.

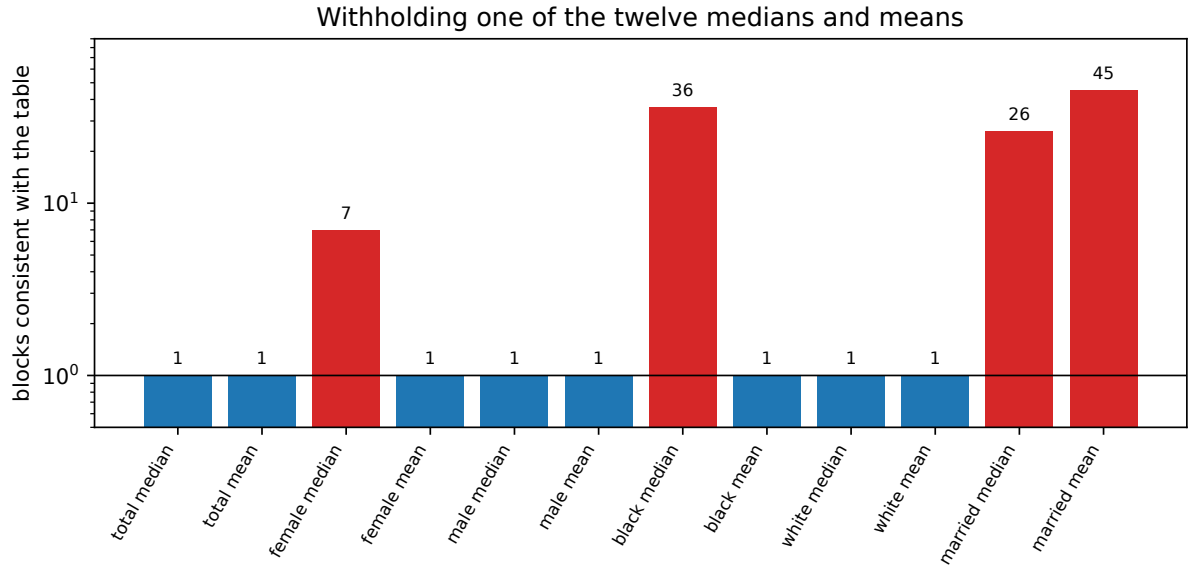


Figure 17.6.: Number of seven-person blocks consistent with Table 17.3 when one median or mean is withheld (log scale).

17.7. Code

The script code/reconstruction.py computes every number in this chapter and draws every figure. It checks the attacks and the census block by exhaustive search, and the lemma and the predictions by simulation. From the repository root:

```
uv run python code/part02-attacks/reconstruction.py
```

Table 17.5.: The script's output with the default seed.

Experiment	Result
Exponential attack, $n = 10$	Table 17.1; rounding to multiples of 2 or 4: one survivor
Lemma 17.2, $n = 1000$	Threshold 0.8333; escape 0.6161, 1.0000; bound 0.1875
Theorem 17.2, $\beta = 0.01$	$E \leq 0.1667$; $m \geq 33\,296$ $(33.3n)$
Least squares, Gaussian noise	Table 17.2; m/n for 90%: 106.1 at $\sigma = 2\sqrt{n}$, 657.9 at $5\sqrt{n}$
Least squares, worst case	$s_{\min} = 11.43$ (11.18); wrong bits 0, 20, 143, bounds 61.2, 245.0, 1914.0
Sampling curator, $q = \frac{1}{2}$	Kept 250; max error 28.00; bound 106.25; recovered 0.7560
Information bound, $n = 1000$	$\gamma = 0.05$: 71.6, 106.9, 141.5; $\gamma = 0.01$: 92.2
Digit predicates, $\sigma = 4$	$v = 0.2016$; $n = 73$: 49 of 50 perfect, predicted 0.99950; $n = 142$: 45 of 50, predicted 0.99946; 19.5 for 75%
Census block	435 and 34 age lists; 14 790, 12 906, 387, 33, 1; Table 17.4

tests/test_reconstruction.py checks that the output still matches.

17.8. Summary

Table 17.6.: The results of this chapter, and where each is used.

Result	Statement	Use
Theorem 17.1	All 2^n subsets with error E: at most $4E$ wrong bits	Against unlimited queries the error must be of order n
Proposition 17.1	Some curator attains distance $2\lfloor 2E \rfloor$	Tight only against adversarial errors
Lemma 17.1	$\Pr[W > \lambda\,\mathbb{E}[W]] \geq (1-\lambda)^2\,\mathbb{E}[W]^2/\mathbb{E}[W^2]$	Anti-concentration; Appendix Section H.2

Result	Statement	Use
Lemma 17.2	$\Pr[\lvert\langle f, z\rangle\rvert > \sqrt{\gamma n}/12] \geq \frac{3}{16}$	A random query's threat scale is $\sqrt{n}$
Theorem 17.2	Error $o(\sqrt{n})$, $O(n \log n)$ random queries: $o(n)$ wrong bits	The Dinur–Nissim attack; Lower bounds
Proposition 17.2	Wrong bits $\leq 4mE^2/s_{\min}(A)^2$	Errors aimed at the weakest direction hurt most
Proposition 17.3	Recovered $\approx \Phi(\sqrt{m-n}/(4\sigma))$	Independent noise must grow like $\sqrt{m}$
Proposition 17.4	Error $\leq \sqrt{n \ln(2m/\beta)/(2q^2)}$; recovery $\leq q + \frac{1-q}{2}$	Stopping reconstruction is not privacy
Theorem 17.3	$m \geq n(1 - h(\gamma))/\log_2 V$	Any attack needs of order $n/\ln n$ exact counts
Proposition 17.5	Recovered $\approx \Phi(\sqrt{v(m-n-1)}/(2\sigma))$	Restricting query syntax is no defence
Proposition 17.6	One block fits the published table	Overlapping exact tables are a solvable system

17.9. References

- Dinur & Nissim, "Revealing Information while Preserving Privacy" (PODS 2003). The original attacks and the "blatantly non-private" definition.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 8. Reconstruction as a lower bound, and the Fundamental Law.
- Dwork, McSherry & Talwar, "The Price of Privacy and the Limits of LP Decoding" (STOC 2007). Decoding with a constant fraction of arbitrarily wrong answers.
- Dwork & Yekhanin, "New Efficient Attacks on Statistical Disclosure Control Mechanisms" (CRYPTO 2008). Structured queries and fast decoders.
- Kasiviswanathan, Rudelson & Smith, "The Power of Linear Reconstruction Attacks" (SODA 2013). Reconstruction from released statistics, and the role of the smallest singular value.
- Dwork, Smith, Steinke & Ullman, "Exposed! A Survey of Attacks on Private Data" (Annual Review of Statistics and Its Application, 2017). A readable survey.
- Cohen & Nissim, "Linear Program Reconstruction in Practice" (Journal of Privacy and Confidentiality, 2020). The Diffix attack of Section 17.5.
- Homer et al., "Resolving Individuals Contributing Trace Amounts of DNA to Highly Complex Mixtures Using High-Density SNP Genotyping Microarrays" (PLoS Genetics, 2008). The tracing attack of Lower bounds.
- Garfinkel, Abowd & Martindale, "Understanding Database Reconstruction Attacks on Public Data" (CACM 2019). The model for Section 17.6.
- Abowd, declaration in *State of Alabama v. US Department of Commerce* (2021), Appendix B. The 2010 Census reconstruction experiment.
- Vershynin, *High-Dimensional Probability* (2018), ch. 4, for singular values of random matrices, and Muirhead, *Aspects of Multivariate Statistical Theory* (1982), ch. 3, for the inverse-Wishart mean.

Part III.

Differential privacy foundations

The previous part showed that models and statistics leak: membership, attributes, training text, whole datasets. This part introduces the defence that the rest of the book is built on, **differential privacy (DP)**, and develops it far enough that you can prove a new mechanism private, calibrate its noise, compose it with others, and say what any such mechanism must cost. The finer tools for tracking privacy over many steps (Rényi DP, numerical accountants, amplification by subsampling) come in the next part, which builds directly on this one.

Who this part is for

You are expected to have read the Prerequisites and Threat models and attacks. This part uses the max-divergence, the hockey-stick divergence and the data processing inequality of Divergences, the trade-off function of Hypothesis testing, and the tail bounds for the Laplace and normal distributions of Concentration inequalities. It uses the games and the advantage of Threat models and privacy games, since the definition of DP is a bound on exactly those games. Each chapter restates in one sentence every fact it takes from an earlier page, and links to the page that proves it.

The common thread

A randomized algorithm is differentially private when changing one person's record changes the distribution of its output by at most a factor e^{ε}, up to a small slack δ. Everything in this part follows from reading that one sentence in three ways.

- **As a bound on a divergence.** Pure ε-DP bounds the max-divergence between the outputs on neighbouring datasets, and (ε, δ)-DP bounds their hockey-stick divergence. The data processing inequality then gives post-processing for free, and the **privacy loss** $L_{D,D'}$, the log of the likelihood ratio, is the quantity whose sum governs composition.
- **As a bound on every attack.** By the hypothesis-testing view, $\mathrm{FPR} + e^{\varepsilon}\,\mathrm{FNR} \geq 1 - \delta$ for any test of "was this record used". Every game of the attacks part, from membership inference to reconstruction, is therefore capped by a number you can compute before the data is touched.
- **As a budget to spend.** Privacy is lost a little at each release. Noise is calibrated to how much one record can move the answer (its **sensitivity**), losses add up under composition, and lower bounds say how much accuracy no mechanism can avoid paying.

Table 17.7 lists the chapters.

Table 17.7.: The chapters of this part, the question each one answers, and the earlier chapters it uses, by number (chapters 1–8 are the prerequisites, 9–17 the attacks part).

Chapter	The question it answers	Builds on
The definition of differential privacy	What exactly does DP promise, to whom, and what does it not protect? How do you tell a privacy violation from learning about the population?	3, 4, 9–11, 13
Post-processing, group privacy and composition	Which operations keep a guarantee, and how do guarantees add up?	2, 3, 9, 10, 18
The Laplace and Gaussian mechanisms	How much noise does a numerical query need, and what is the exact privacy of Gaussian noise?	1–4, 18, 19
Advanced and optimal composition	How does privacy degrade over many releases, exactly?	2–4, 18–20
Selection: exponential mechanism, report-noisy-max, sparse vector	How do you privately choose the best of many options, or find the few queries above a threshold?	1, 2, 18–21
Query release and private multiplicative weights	How do you answer millions of linear queries, or publish a synthetic dataset, when noise on every answer would drown them?	2, 17, 18, 20–22
Beyond global sensitivity	What if the worst case is far from the data you actually have?	1, 2, 3, 9, 18–20, 22
Discrete and floating-point-safe mechanisms	Why do textbook mechanisms break on real computers, and how is that fixed?	1, 3, 18–20
Local, shuffle and distributed trust models	What changes when no one is trusted with the raw data?	1–4, 8, 9, 18–20
Lower bounds: what privacy costs	How much accuracy must every private mechanism give up?	2, 4, 5, 10, 17–20, 22–24
Private mean estimation	How many records does a private estimate of a population mean need, and what does not knowing the range, the dimension or the tails cost?	1, 2, 18–22, 24, 27

How to read this part

Order. Chapters 18 to 21 are the core and should be read in order: the definition, the properties that make it usable, the two mechanisms everything else is built from, and advanced and optimal composition, which finishes the story of composition that chapter 19 starts and introduces the privacy loss random variable that the next part uses throughout. Chapter 22 adds mechanisms for selection, and chapter 23 builds on them to answer many queries at once, by releasing synthetic data with private multiplicative weights. Chapter 24 handles data whose worst case is far from typical. Chapter 25 is short and practical: read it before you implement any mechanism yourself. Chapter 26 changes who is trusted with the raw data. Chapter 27 is the most theoretical; it ties back to reconstruction and membership attacks and says what privacy must cost. Chapter 28 closes the part by putting these tools together on the most common statistical task, estimating a mean, and matches each of its error rates with a lower bound from chapter 27.

Structure of a chapter. Every chapter opens with the question it answers and a running example with its numbers, such as the counting query of the Laplace and Gaussian mechanisms or the dashboard of a hundred counts in Advanced and optimal composition. Then comes one section per topic. Its definitions, theorems, lemmas and worked examples are numbered environments, so that later chapters can cite them by number, and every theorem is followed at once by its proof in numbered steps. A Code section runs the script that checks every quoted number. A Summary table then lists each result of the chapter, its statement in a line and where it is used, and References close the chapter.

Proofs you can check by computer. Most privacy claims in this part are checked numerically as well as proved: the exact privacy profile of a mechanism is computed and compared with its bound, a composition formula is compared with brute-force enumeration, and a buggy mechanism is shown to leak by computing its privacy loss. Running the scripts is the fastest way to see how loose each bound is. From the repository root, for example,

```
uv run python code/part03-dp-foundations/laplace_gaussian.py
```

Notation

The chapters share the conventions in Table 17.8, set in the definition of differential privacy and fixed for the whole book. The next part uses the same notation. There the noise multiplier, the noise standard deviation per unit of sensitivity, is usually written σ; it equals the standard deviation when the sensitivity is 1.

Table 17.8.: Notation shared by the chapters of this part and the next.

Symbol	Meaning
$D = (x_1, \dots, x_n)$, $\mathcal{X}$	A dataset of n records, each from the data universe $\mathcal{X}$, so $D \in \mathcal{X}^n$
M; $M_1, \dots, M_k$	A mechanism, a randomized algorithm whose privacy is analysed; k mechanisms in a composition

Symbol	Meaning
$y \in \mathcal{Y}$, $S \subseteq \mathcal{Y}$	An output, the output space and an event; $Y = M(D)$ is the random output
$D \sim D'$	Neighbouring datasets: by default one is the other with one record added or removed; "replace one record" is named explicitly when used
$D' = D \cup \{x\}$	Under add/remove, D' is the dataset that contains the record x and D the one without it. A membership test says "D'" when it guesses that the record is in; its false positive rate is the chance of saying "D'" when the data were D, and its true positive rate the chance of saying "D'" when they were D'
ε, δ	The privacy parameters: ε-DP is pure DP, (ε, δ)-DP approximate DP
f; $\Delta_1 f$, $\Delta_2 f$	A query $f\colon \mathcal{X}^n \to \mathbb{R}^d$ with output dimension d; its ℓ_1 and ℓ_2 sensitivities, written Δf when the norm is clear
$L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}$	The privacy loss of output y; $L = L_{D,D'}(Y)$ with $Y \sim M(D)$ is the privacy loss random variable
$E_\gamma(P \,\|\, Q)$, $\delta(\varepsilon)$	The hockey-stick divergence at level $\gamma \geq 1$; (ε, δ)-DP means $E_{e^\varepsilon}(M(D) \,\|\, M(D')) \leq \delta$ for all $D \sim D'$, and the privacy profile $\delta(\varepsilon)$ is the smallest such δ
$\mathrm{Lap}(b)$, $\mathcal{N}(0, \sigma^2)$	Laplace noise with scale $b = \Delta_1 f/\varepsilon$; normal noise with standard deviation σ
$r \in \mathcal{R}$, $u(D, r)$, Δu	The exponential mechanism's candidates, its score function and the score's sensitivity
$\mathcal{Q}$, τ, $\perp$, $\top$	A workload of counting queries; the sparse vector technique's threshold and its answers "below" and "above"
(α, β)	An (α, β)-accurate mechanism errs by at most α with probability at least $1 - \beta$; β is a failure probability
FPR, FNR	A test's false positive and false negative rates, its type I and type II errors, written α and β only on a page where neither letter has another meaning
$R\colon \mathcal{X} \to \mathcal{Y}$, ε_0	A local randomizer and its privacy parameter

What you will be able to do

By the end of this part you should be able to:

- state pure and approximate DP for a stated neighbouring relation and unit of privacy, translate a guarantee into a bound on membership advantage and on posterior odds, and say why δ must be much smaller than $1/n$;
- prove post-processing, group privacy and basic composition, and say what does not compose;
- calibrate the Laplace and Gaussian mechanisms, including the exact (analytic) Gaussian calibration;
- derive advanced composition and compute optimal composition exactly;
- prove the exponential mechanism, report-noisy-max and the sparse vector technique private, and recognize the common broken variants of the last;
- answer a large workload of linear queries with private multiplicative weights, and bound its error through the regret of multiplicative weights;
- handle data-dependent sensitivity with smooth sensitivity, propose-test-release, inverse sensitivity, subsample-and-aggregate and clipping;
- implement discrete Laplace and discrete Gaussian samplers exactly, and explain why naive floating-point noise is not private;
- design frequency estimation in the local model and explain what shuffling and secure aggregation buy;
- prove packing and fingerprinting lower bounds, and run the tracing attack they come from;
- estimate a mean privately when its range, its dimension or its tails are unknown, and match each error rate with a lower bound.

Each of these is a result derived in one of the chapters that follow.

18. The definition of differential privacy

Each attack of Part 2 was a game in which the challenger ran an algorithm on one of two datasets that differ in one person, and the adversary guessed which (Threat models and privacy games). Each worked because a test could tell the two datasets apart from the output. **Differential privacy (DP)** forbids exactly that: for every pair of datasets that differ in one person, the distribution of the output barely changes. Then every attack of Part 2 fails at once, whatever the adversary knows and computes.

The one idea is the **privacy loss**. For one output of a randomized algorithm, it is the logarithm of how much more likely that output was under one dataset than under its neighbour, the log-likelihood ratio of Divergences. Differential privacy bounds it. Pure DP bounds it at every output, and approximate DP lets it exceed the bound on outputs of small total probability δ.

The running example is a survey. Alice, Bob and Carol are asked whether they smoke, and each answers with two coins. On heads they tell the truth, and on tails they toss again and say "yes" on heads and "no" on tails. So each person tells the truth with probability $\frac{3}{4}$, and every "yes" has an excuse. This survey is $\ln 3$-DP, as Section 18.3 shows. A count released with Gaussian noise of standard deviation 2 is the contrast case, because its privacy loss is unbounded and it needs a δ.

18.1. Neighbouring datasets

"Differ in one person" needs a precise meaning before anything can be proved. A dataset D is a multiset of **records** drawn from a universe $\mathcal{X}$ of possible records. The **neighbouring relation** says which pairs of datasets the guarantee must protect, and so it fixes the **unit of privacy**, the thing whose presence or value the output must not reveal.

Definition 18.1 (Neighbouring datasets)**.** Datasets D and D' are **neighbours**, written $D \sim D'$, in one of two senses, which every guarantee must name.

- **Add/remove** (also called unbounded DP): D' is D with one record added or removed. The size n of the dataset is itself private.
- **Replace** (also called bounded DP, or substitution): $|D| = |D'|$, and they differ in the value of one record. The size n is public.

Both relations are symmetric: if $D \sim D'$ then $D' \sim D$.

Add/remove protects participation, which is the question membership inference asks. Replace protects the value of your record. Separately, you choose what one record is. At **example-level** a record is one row, and at **user-level** it is all the data of one person, so example-level DP on 200 messages from one user protects each message, not the user. Neighbours also range over absurd

records, such as a person aged 900, so the guarantee must hide even such a record, which is why mechanisms clip their inputs to a fixed range before adding noise.

Example 18.1. Take $D = (\text{Alice: smoker}, \text{Bob: non-smoker}, \text{Carol: smoker})$. Table 18.1 checks five candidate datasets against both relations.

Table 18.1.: Five candidate datasets, checked against the add/remove and the replace relation.

Candidate D'	Add/remove neighbour?	Replace neighbour?
(Alice: smoker, Bob: non-smoker)	yes: Carol removed	no: size differs
(Alice: smoker, Bob: non-smoker, Carol: smoker, Dan: non-smoker)	yes: Dan added	no: size differs
(Alice: non-smoker, Bob: non-smoker, Carol: smoker)	no: two steps	yes: Alice's answer changed
(Alice: smoker, Bob: non-smoker, Dan: smoker)	no: two steps	yes: Carol replaced by Dan
(Alice: non-smoker, Bob: smoker, Carol: smoker)	no	no: two records changed

18.2. Pure and approximate differential privacy

"The output distribution barely changes" needs a measure. An average such as the Kullback–Leibler divergence can hide one rare output with an enormous loss. A difference such as the total variation (TV) distance fails another way. Any two datasets of n records are n replacements apart, so a tolerance τ per replacement bounds their outputs' difference by $n\tau$, and a useful output needs $n\tau$ near 1. But τ near $1/n$ already allows publishing a random record in full (Section 18.8). Differential privacy therefore bounds a ratio, in the worst case over every pair and every event, and allows a small difference only on top. Section A.1 in the appendices derives the definition's forms from scratch.

Write M for a **mechanism**, a randomized algorithm that maps a dataset to a random output in a set $\mathcal{Y}$. The probability that the output lands in an event $S \subseteq \mathcal{Y}$ is taken over the mechanism's own coins.

Definition 18.2 (Differential privacy)**.** A mechanism M is (ε, δ)**-differentially private**, for $\varepsilon \geq 0$ and $\delta \in [0, 1]$, if for every pair of neighbours $D \sim D'$ and every event $S \subseteq \mathcal{Y}$,

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{18.1}$$

Because the neighbouring relation is symmetric, the inequality also holds with D and D' swapped. The case $\delta = 0$ is **pure DP**, written ε**-DP**. The case $\delta > 0$ is **approximate DP** (Dwork, Kenthapadi, McSherry, Mironov & Naor, 2006).

The definition bounds events because an adversary's decision rule, "say D' when the output is in S", is an event. The dataset is fixed and the randomness is the mechanism's own, so privacy comes only from added noise. A deterministic algorithm that is not constant fails the definition for every $\delta < 1$. Somewhere along a chain of neighbours its output changes, and the event "the output equals its value on D" then has probability 1 on D and 0 on D'. So rounding or suppressing small cells gives no guarantee by itself.

The privacy loss of the opening checks the definition output by output.

Definition 18.3 (Privacy loss)**.** Write $p_{M(D)}$ for the probability mass function or density of $M(D)$. The **privacy loss** of an output y for the ordered pair (D, D') is

$$L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}. \tag{18.2}$$

It is 0 when y is equally likely in both worlds, large when y is strong evidence for D, and infinite when y is impossible under D'.

Pure DP can be checked one output at a time, because grouping outcomes into an event can only dilute the worst ratio. Approximate DP cannot, because the slack δ is paid once per event, not once per outcome.

Proposition 18.1 (Pure DP is pointwise, approximate DP is not)**.** *Let M have a discrete output set. Then M is ε-DP if and only if $|L_{D,D'}(y)| \le \varepsilon$ for every neighbouring pair and every output y, and for densities the same holds at almost every y. For $\delta > 0$, (ε, δ)-DP implies the pointwise bound*

$$p_{M(D)}(y) \le e^{\varepsilon} p_{M(D')}(y) + \delta \quad \text{for every } y, \tag{18.3}$$

but the pointwise bound does not imply (ε, δ)-DP.

Proof. The plan is to show that events and outcomes say the same thing when $\delta = 0$, then to see the slack add up over the outcomes of an event when $\delta > 0$.

1. *Events imply outcomes.* A single outcome is an event, so taking $S = \{y\}$ in Equation 18.1 gives the pointwise bound. With $\delta = 0$, together with the same bound for the swapped pair, this is $|L_{D,D'}(y)| \le \varepsilon$.
2. *Outcomes imply events when $\delta = 0$.* An event's probability is the sum of the probabilities of its outcomes, so summing the pointwise bound over $y \in S$ gives

$$\Pr[M(D) \in S] = \sum_{y \in S} p_{M(D)}(y) \le e^{\varepsilon} \sum_{y \in S} p_{M(D')}(y) = e^{\varepsilon} \Pr[M(D') \in S]. \tag{18.4}$$

   For densities, changing the density on a set of probability zero changes no event's probability, which is why almost every y is enough.
3. *Outcomes do not imply events when $\delta > 0$.* Each of the $|S|$ outcomes of an event brings its own δ, so summing the pointwise bound over S gives only

$$\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + |S|\,\delta. \tag{18.5}$$

4. *A counterexample.* Let $M(D)$ be uniform on $\{1, \dots, m\}$ and $M(D')$ uniform on $\{m+1, \dots, 2m\}$. Every outcome has probability at most $1/m$, so the pointwise bound holds with $\delta = 1/m$. But the event $\{1, \dots, m\}$ has probability 1 under $M(D)$ and 0 under $M(D')$, so Equation 18.1 needs $\delta = 1$. With $m = 1000$ the chapter's script (Section 18.11) finds a pointwise δ of 0.0010 and an event δ of 1.0000.

□

For small ε the factor e^{ε} is about $1 + \varepsilon$, so every event changes probability by at most about a fraction ε of itself. Along a chain of datasets or releases the factors multiply and the exponents add, which is how group privacy and composition work in Post-processing, group privacy and composition. At $\varepsilon = 0.1$, an event of probability 0.5 under $M(D')$ has probability at most $0.5e^{0.1} = 0.553$ under $M(D)$, while at $\varepsilon = 1$ the bound is 1.36 and says nothing.

In Figure 18.1, Laplace noise added to a count keeps the density on D inside the band that pure DP allows, while Gaussian noise leaves it in the tail, where δ must pay.

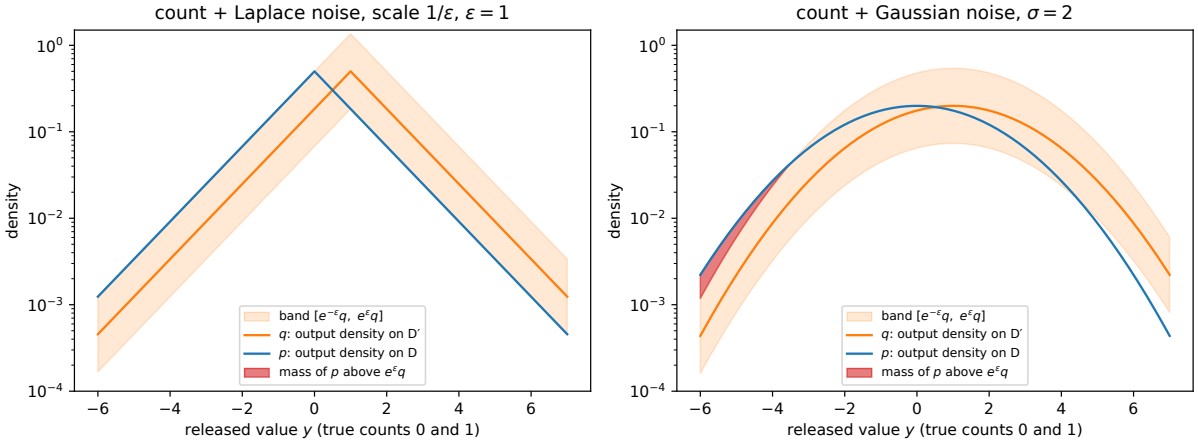


Figure 18.1.: The band from $e^{-\varepsilon}$ to e^{ε} times the density on D' (orange), at $\varepsilon = 1$. Left: Laplace noise keeps the density on D (blue) inside. Right: Gaussian noise ($\sigma = 2$) leaves it in the tail, and the red mass is what δ covers.

The chain of factors also converts between the neighbouring relations, because a replace neighbour is two add/remove steps away.

Proposition 18.2 (From add/remove to replace)**.** *If M is ε-DP under add/remove, then it is 2ε-DP under replace:*

$$\varepsilon\text{-DP under add/remove} \implies 2\varepsilon\text{-DP under replace.} \tag{18.6}$$

The converse fails: a mechanism can be 0-DP under replace and not ε-DP under add/remove for any ε.

Proof.

1. *Build the two-step path.* Let $D \sim D'$ under replace, with D holding the record x where D' holds x', and let $D_0 = D \setminus \{x\}$. Then D and D_0 differ by one removal, and D_0 and D' by one addition.
2. *Chain the ratio bound along the path.* Pure DP bounds each step by a factor e^{ε}, so applying it to (D, D_0) and then to (D_0, D') gives, for every event S,

$$\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D_0) \in S] \le e^{2\varepsilon} \Pr[M(D') \in S]. \tag{18.7}$$

   Walking the path from D' back to D gives the other order.
3. *The counterexample for the converse.* Let M output the number of records. It is 0-DP under replace, because datasets of the same size give the same output. Under add/remove, the event $\{n+1\}$ has probability 1 on a dataset of $n+1$ records and 0 on its neighbour, and no finite e^{ε} times 0 reaches 1.

□

The factor 2 is reached when one replacement changes two counts, as changing Alice's answer does in the survey's histogram (smokers, non-smokers). With Laplace noise of scale $1/\varepsilon$ in each bin, the script finds largest losses of 1.0000 under add/remove and 2.0000 under replace at $\varepsilon = 1$. For approximate DP the same path gives $(2\varepsilon, (1 + e^{\varepsilon})\delta)$, because the second step multiplies the first step's δ by e^{ε}.

Example 18.2. In the survey, take the true bits $(1, 0, 1)$ for Alice, Bob and Carol and the replace neighbour $(0, 0, 1)$. The script checks all 256 events of the 8 report vectors. The largest ratio over events is 3.0000, the same as over single outcomes, as Proposition 18.1 says. Every replace pair gives the same, so the survey is $\ln 3$-DP under replace and no better, and Theorem 18.1 shows why only Alice's coins matter.

18.3. Randomized response

You want the fraction of people who have done something embarrassing, but asked directly, people lie. Warner (1965) found the answer four decades before the definition. Each respondent tells the truth only with a known probability, and the randomization averages out over the population. The two coins of the running example are the **forced-response** variant credited to Boruch (1971). Section A.4 in the appendices derives the mechanism from scratch.

The idea is to tell the truth with a probability t a little above $\frac{1}{2}$, so that the ratio $t/(1-t)$ of the chances of a report under the two true bits is e^{ε}.

Definition 18.4 (Randomized response)**. Randomized response at level** ε reports a person's bit $x \in \{0, 1\}$ truthfully with probability $t = e^{\varepsilon}/(1 + e^{\varepsilon})$ and flipped with probability $1 - t$. On a dataset of n bits it is applied to each person independently, and the output is the vector of n reports.

With two coins, a respondent tells the truth on heads, or on tails when the second coin gives the true answer, so $t = \frac{3}{4}$, $t/(1-t) = 3$, and the survey is randomized response at $\varepsilon = \ln 3 = 1.0986$ (Table 18.2).

Table 18.2.: Randomized response as a table of output probabilities. The ratios between the two rows are the privacy loss.

True bit	Pr[report 1]	Pr[report 0]	At $\varepsilon = \ln 3$
1	$t = \frac{e^\varepsilon}{1+e^\varepsilon}$	$1-t = \frac{1}{1+e^\varepsilon}$	0.75, 0.25
0	$1-t$	t	0.25, 0.75
Ratio, row 1 / row 0	e^ε	$e^{-\varepsilon}$	3, 1/3

Write $\bar{x}$ for the true fraction of ones and $\bar{y}$ for the mean of the reports. The coins push $\bar{y}$ toward $\frac{1}{2}$ in a known way, and undoing the push scales up the noise by $1/(2t-1)$.

Theorem 18.1 (Privacy and accuracy of randomized response)**.** *Randomized response at level ε on n people is ε-DP under replace. The estimate*

$$\hat{\bar{x}} = \frac{\bar{y} - (1-t)}{2t-1} \tag{18.8}$$

of the true fraction is unbiased, and its standard deviation is

$$\mathrm{sd}(\hat{\bar{x}}) = \frac{\sqrt{t(1-t)}}{(2t-1)\sqrt{n}} = \frac{e^{\varepsilon/2}}{(e^\varepsilon - 1)\sqrt{n}} \approx \frac{1}{\varepsilon\sqrt{n}} \quad \textit{for small } \varepsilon. \tag{18.9}$$

Proof. The plan is to read one person's privacy off Table 18.2, to show that the other people cancel out of the likelihood ratio, and then to compute the mean and the variance of the estimate.

1. *One person.* Every ratio between the two rows of Table 18.2 is e^ε or its inverse, so the privacy loss of each report is $\pm\varepsilon$, and one report is ε-DP by Proposition 18.1.
2. *The output distribution factors.* People toss their own coins independently, so the probability of a report vector is a product of one factor per person, $\prod_i p_i(y_i \mid x_i)$, where $p_i(y_i \mid x_i)$ is the chance that person i reports y_i when their true bit is x_i.
3. *Neighbours differ in one factor.* Under replace, D and D' differ only in person j's bit, so every other factor cancels in the ratio. The loss is person j's alone, so it is $\pm\varepsilon$, and the vector is ε-DP by Proposition 18.1.
4. *The mean of the reports.* A report is 1 with probability t if the true bit is 1 and $1-t$ if it is 0. Averaging over a population in which a fraction $\bar{x}$ have bit 1 gives

$$\mathbb{E}[\bar{y}] = \bar{x}\,t + (1-\bar{x})(1-t) = (1-t) + \bar{x}(2t-1). \tag{18.10}$$

   Solving for $\bar{x}$ gives Equation 18.8, which is unbiased because it inverts this linear map and expectations pass through linear maps.
5. *The spread.* Each report is a Bernoulli variable with variance $t(1-t)$, whatever its true bit, and the reports are independent, so

$$\mathrm{Var}(\bar{y}) = \frac{t(1-t)}{n}. \tag{18.11}$$

   Dividing $\bar{y}$ by $2t-1$ divides the variance by $(2t-1)^2$, which gives the first form. Substituting $t = e^\varepsilon/(1+e^\varepsilon)$ gives the closed form, because

$$t(1-t) = \frac{e^\varepsilon}{(1+e^\varepsilon)^2}, \qquad 2t-1 = \frac{e^\varepsilon - 1}{e^\varepsilon + 1}. \tag{18.12}$$

□

Deniability is the privacy, and the noise is added before the data leave each person, the **local** model of Local, shuffle and distributed trust models. Under add/remove the number of reports would reveal whether you took part, so the guarantee holds under replace only.

Privacy costs samples: halving ε doubles the error, so recovering the accuracy takes four times as many people, and much of that is the price of trusting nobody. A trusted curator can instead add Laplace noise of scale $1/\varepsilon$ to the true count once, which is ε-DP by the Laplace and Gaussian mechanisms. Its error on the fraction, the Laplace standard deviation $\sqrt{2}/\varepsilon$ divided by n, falls like $1/n$ rather than $1/\sqrt{n}$.

Example 18.3. Of $n = 1000$ respondents, a fraction $\bar{x} = 0.3$ smoke, so the expected fraction of "yes" reports is $0.3 \times 0.75 + 0.7 \times 0.25 = 0.4$, which Equation 18.8 turns back into 0.3. The formula gives $\mathrm{sd}(\hat{\bar{x}}) = \sqrt{0.1875}/(0.5\sqrt{1000}) = 0.0274$, and ten thousand simulated surveys give a mean of 0.2996 and a standard deviation of 0.0278. At $\varepsilon = 0.5$ the formula gives 0.0626 (Figure 18.2). A trusted curator at $\varepsilon = \ln 3$ would reach $\sqrt{2}/(1000 \ln 3) = 0.00129$, which is 21.3 times smaller.

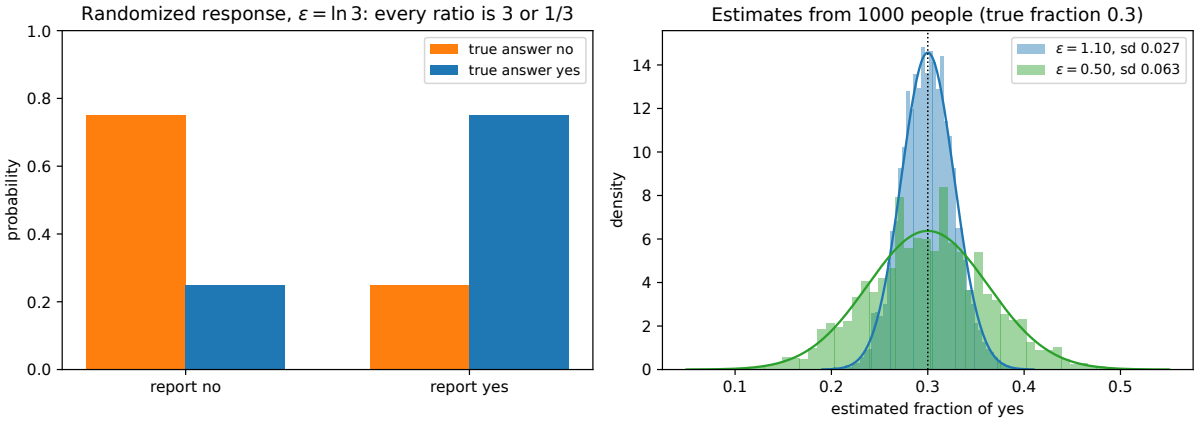


Figure 18.2.: Left: randomized response at $\varepsilon = \ln 3$ as output probabilities. Right: estimates from 4000 simulated surveys of 1000 people with true fraction 0.3, at $\varepsilon = \ln 3$ and 0.5, with the normal curves the formula predicts.

18.4. Divergence forms and the privacy loss

Equation 18.1 quantifies over all events, which is unusable for continuous outputs. A divergence of Divergences turns "for all events" into one formula and shows what δ measures. The **max-divergence** $D_\infty(P\|Q) = \ln \max_y p(y)/q(y)$ is the largest privacy loss (Section 3.2). The **hockey-stick divergence** $E_\gamma(P\|Q) = \sum_y (p(y) - \gamma q(y))_+$ is the mass of P that sticks out above γ times Q (Section 3.6), where $(x)_+ = \max\{x, 0\}$.

Theorem 18.2 (Differential privacy as a divergence bound)**.** *For every pair of neighbours $D \sim D'$ write $P = M(D)$ and $Q = M(D')$, with probabilities $p(y)$ and $q(y)$ (densities when the output is continuous, with sums read as integrals), and $L = L_{D,D'}$.*

1. *M is ε-DP if and only if $D_\infty(P\|Q) \le \varepsilon$ for every ordered pair of neighbours.*
2. *M is (ε, δ)-DP if and only if, for every ordered pair of neighbours,*

$$E_{e^\varepsilon}(P\|Q) = \sum_y \big(p(y) - e^\varepsilon q(y)\big)_+ = \mathbb{E}_{y\sim P}\Big[\big(1 - e^{\varepsilon - L(y)}\big)_+\Big] \le \delta. \tag{18.13}$$

3. *If $\Pr_{y\sim P}[L(y) > \varepsilon] \le \delta$ for every ordered pair, then M is (ε, δ)-DP.*

Proof. The plan is to find the worst event for the definition's difference, to rewrite the resulting sum as an average under P, and to bound that average by a tail probability.

1. *Pure DP.* By Proposition 18.1, ε-DP is $|L(y)| \le \varepsilon$ at every output and in both orders of the pair, and the largest value of L is $D_\infty(P\|Q)$ by definition.
2. *The worst event for a difference.* The definition asks $\Pr_P[S] - e^\varepsilon \Pr_Q[S] \le \delta$ for every event. Each outcome in S adds $p(y) - e^\varepsilon q(y)$ to the left side, which is positive exactly when $L(y) > \varepsilon$. So the worst event takes every outcome with a positive contribution and no other, and the definition holds for every S exactly when it holds for this one:

$$\max_S \big[\Pr_P[S] - e^\varepsilon \Pr_Q[S]\big] = \sum_y \big(p(y) - e^\varepsilon q(y)\big)_+ = E_{e^\varepsilon}(P\|Q). \tag{18.14}$$

3. *As an average under P.* Factor $p(y)$ out of each term and use $q/p = e^{-L}$ from Equation 18.2, so that each term becomes $p\,(1 - e^{\varepsilon - L})_+$. A sum of $p(y)$ times a function of y is the average of that function under P, and outcomes with $p(y) = 0$ contribute nothing.
4. *The sufficient condition.* The weight $(1 - e^{\varepsilon - L})_+$ is 0 when $L \le \varepsilon$ and less than 1 otherwise, so it is at most the indicator $\mathbf{1}[L > \varepsilon]$. Averaging both sides under P gives

$$E_{e^\varepsilon}(P\|Q) \le \Pr_P[L > \varepsilon] \le \delta. \tag{18.15}$$

□

So the smallest valid δ is not the probability that the loss exceeds ε, but an average that counts an output with loss just above ε for almost nothing and an infinite loss in full. What δ bounds exactly is the chance of a total failure. The outputs that are impossible on D' have infinite loss, and Equation 18.1 applied to them says that their total probability on D is at most δ.

A mechanism has a curve, not a pair: it is $(\varepsilon, \delta(\varepsilon))$-DP for every ε (Figure 18.3). The curve reaches 0 only if the loss is bounded. The distribution of L with y drawn from $M(D)$ is the **privacy loss distribution**, which the accountants of Numerical accounting work with directly.

Example 18.4. For each mechanism, add up the excess mass where the likelihood ratio exceeds e^ε. For the survey only the report with ratio 3 counts, which gives $\delta(\varepsilon) = (3 - e^\varepsilon)_+/4$, so $\delta(0.5) = 0.3378$ and $\delta(1) = 0.0704$. For the Laplace count of scale 1, the closed form of Section 3.6 gives $\delta(0.5) = 0.2212$ and $\delta(1) = 0$.

For the Gaussian count, whose true value moves by 1 between neighbours, write $\mu = 1/\sigma = 0.5$ for that shift in units of the noise. The privacy loss is normal with variance μ^2 and mean $\mu^2/2$ under $M(D)$, $-\mu^2/2$ under $M(D')$ (Section 3.3). By step 2 of the proof, $\delta(\varepsilon)$ is the P-probability

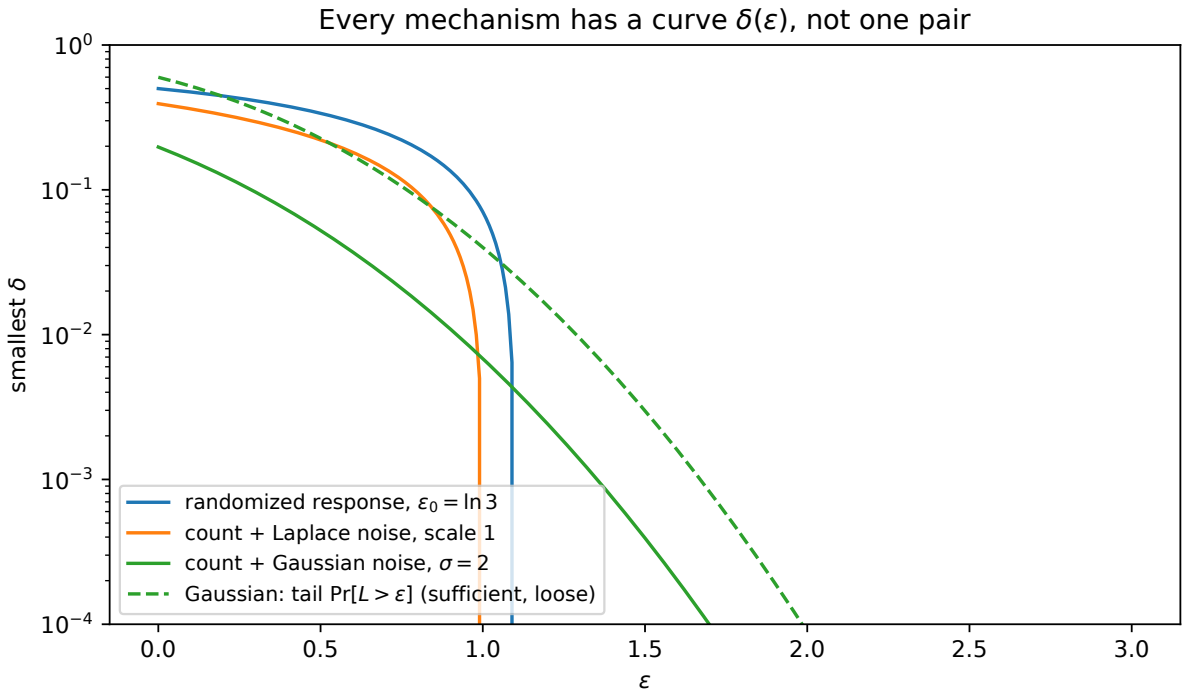


Figure 18.3.: The smallest δ against ε for randomized response at $\ln 3$, the Laplace count of scale 1 and the Gaussian count with $\sigma = 2$. Only the Gaussian curve never reaches 0. Dashed: its loose tail $\Pr[L > \varepsilon]$.

of $\{L > \varepsilon\}$ minus e^{ε} times its Q-probability, and standardizing the loss with the standard normal distribution function Φ gives

$$\delta(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big). \tag{18.16}$$

Table 18.3 compares it with the tail, the first term alone.

Table 18.3.: The exact $\delta(\varepsilon)$ of a count with Gaussian noise of standard deviation 2, against the sufficient condition $\Pr[L > \varepsilon]$.

ε	0.5	1	2	3
Exact $\delta(\varepsilon)$, Equation 18.13	5.244×10^{-2}	6.830×10^{-3}	9.439×10^{-6}	3.401×10^{-10}
Tail $\Pr[L > \varepsilon]$	2.266×10^{-1}	4.006×10^{-2}	8.842×10^{-5}	4.462×10^{-9}
Tail / exact	4.32	5.87	9.37	13.12

A Monte Carlo average over a million draws of L gives 6.859×10^{-3} at $\varepsilon = 1$. The tail is loose by a factor of 4 to 13, because most outputs in it have a loss only a little above ε.

18.5. The hypothesis-testing interpretation

A membership attacker is a test that sees the output and decides between "the data were D" and "the data were D'". Hypothesis testing described the best achievable pairs of error rates by the **trade-off function**. Differential privacy is exactly a lower bound on that function, uniform over all neighbours (Wasserman & Zhou, 2010; Kairouz, Oh & Viswanath, 2015), and Section D.1 in the appendices derives it from scratch.

Let a test say "D'" on the event S. Its **false positive rate (FPR)** is $\alpha = \Pr[M(D) \in S]$, and its **false negative rate (FNR)** is $\beta = \Pr[M(D') \notin S]$. A randomized test says "D'" with probability $\phi(y) \in [0, 1]$ after seeing y (Section 4.1).

Theorem 18.3 (Differential privacy as a bound on every test)**.** *M is (ε, δ)-DP if and only if every test, randomized or not, for every pair of neighbours, satisfies*

$$\alpha + e^{\varepsilon}\beta \geq 1 - \delta \qquad \text{and} \qquad e^{\varepsilon}\alpha + \beta \geq 1 - \delta. \tag{18.17}$$

Equivalently, the trade-off function of every pair of neighbours lies above

$$f_{\varepsilon,\delta}(\alpha) = \max\big\{0,\ 1 - \delta - e^{\varepsilon}\alpha,\ e^{-\varepsilon}(1 - \delta - \alpha)\big\}. \tag{18.18}$$

*In terms of the **true positive rate (TPR)** $1 - \beta$, the second inequality reads* $\mathrm{TPR} \leq e^{\varepsilon}\alpha + \delta$.

Proof. The plan is to apply the definition to the complement of S and to the swapped pair, to extend both bounds to randomized tests, and to read them backwards for the converse.

1. *The first inequality.* Apply Equation 18.1 to the pair (D, D') and the complement S^c, on which the test says "D". The left side is $1-\alpha$ and the probability on the right is β, so
$$1-\alpha \le e^{\varepsilon}\beta + \delta. \tag{18.19}$$
2. *The second inequality.* Apply Equation 18.1 to the swapped pair (D', D) and the event S. The left side is then $1-\beta$ and the probability on the right is α.
3. *Randomized tests.* A test ϕ is not an event, but the gap $\mathbb{E}_{M(D)}[\phi] - e^{\varepsilon}\mathbb{E}_{M(D')}[\phi]$ is linear in each $\phi(y)$, with coefficient $p(y) - e^{\varepsilon}q(y)$. So it is largest at the worst event of Theorem 18.2, and at most δ, and steps 1 and 2 go through with ϕ and $1-\phi$ in place of the events.
4. *The converse.* Every event is a test, so if Equation 18.17 holds for all tests, steps 1 and 2 read backwards give Equation 18.1.
5. *The trade-off form.* Solved for β, the two inequalities give two lower bounds on β, and $\beta \ge 0$ is a third, so the smallest β at level α is at least the largest of the three.

□

The result draws a forbidden region: every attack lies on or above the two lines of Equation 18.17 (Figure 18.4). Part 2 judged membership attacks by their TPR at a low FPR, and that is where DP bites hardest, since it caps the TPR at $e^{\varepsilon}\alpha + \delta$ and as $\alpha \to 0$ the δ term dominates.

Auditing runs this in reverse. Solving the cap for ε at an attack's measured FPR and TPR gives the smallest ε consistent with the measurement. The neighbouring pair is a training set with and without a **canary**, a record planted to be easy to detect (Memorization and extraction). The bound is tight for randomized response and loose for most mechanisms, and keeping a mechanism's whole trade-off curve is the idea of f-DP in Gaussian differential privacy.

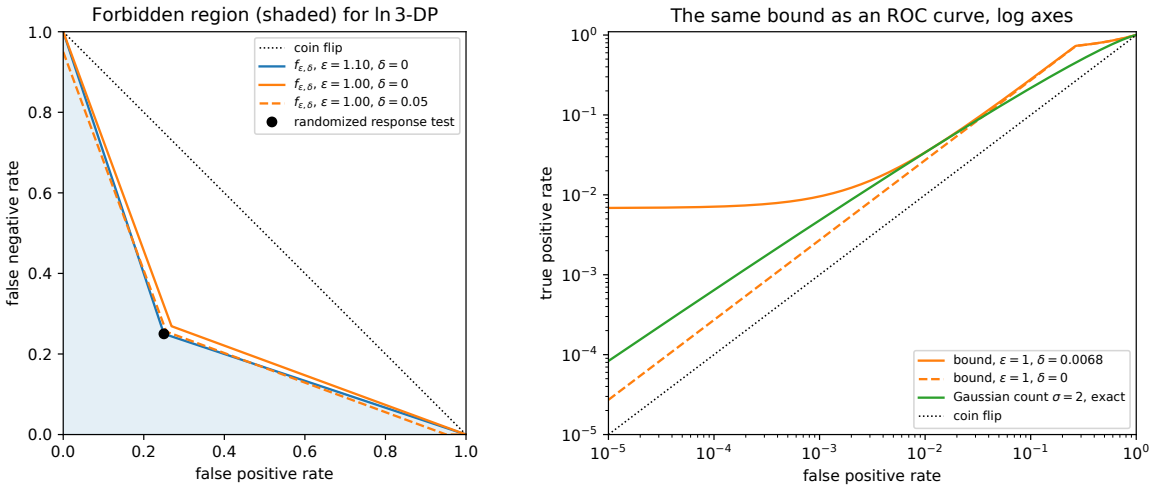


Figure 18.4.: Left: the region below $f_{\varepsilon,\delta}$ that no test can enter, for $\varepsilon = \ln 3$ and for $\varepsilon = 1$ with $\delta = 0$ and 0.05. Right: the largest TPR against FPR at $\varepsilon = 1$, with the best TPR against the Gaussian count.

Example 18.5. In the survey, the best test says "D'" when Alice reports 0. Its FPR and FNR are both 0.25, so $\alpha + 3\beta = 1$, the corner of the region.

The Gaussian count is $(1, 0.00683)$-DP, so at FPR 10^{-3} the bound allows TPR up to $e \times 10^{-3} + 0.00683 = 0.0095$, against 0.002728 under $(1, 10^{-5})$-DP. The best test thresholds y (Neyman–Pearson), and the Gaussian trade-off curve of Section 4.4 gives its TPR as $1-\Phi(\Phi^{-1}(0.999)-0.5) = 0.0048$.

18.6. Bound on the membership advantage

Part 2 measured a membership attacker by its **advantage**. In the membership game the challenger flips a fair coin to pick D or D' and runs M on it, and the adversary guesses which. The advantage is $2\Pr[\text{correct}] - 1$, which with a fair coin equals $\text{TPR} - \text{FPR} = 1 - \alpha - \beta$ (Section 4.6).

Corollary 18.1 (Membership advantage)**.** *If M is (ε, δ)-DP, every adversary in the membership game between any two neighbours has*

$$\text{Adv} = \text{TPR} - \text{FPR} \le \frac{e^\varepsilon - 1 + 2\delta}{e^\varepsilon + 1}, \qquad \Pr[\textit{correct}] \le \frac{e^\varepsilon + \delta}{e^\varepsilon + 1}. \tag{18.20}$$

For $\delta = 0$ the bound is $\tanh(\varepsilon/2)$, about $\varepsilon/2$ for small ε, and randomized response attains it. The bound also holds when the game draws the dataset at random before the coin flip.

Proof. The plan is to turn the advantage into a sum of error rates, to find the smallest sum over the region that Theorem 18.3 allows, and to check that randomized response sits at the corner.

1. *Turn the advantage into a sum of errors.* The advantage is $1 - (\alpha + \beta)$, so the largest advantage comes from the smallest $\alpha + \beta$ allowed by Equation 18.17 with $\alpha, \beta \ge 0$.
2. *Draw the feasible region.* Its lower boundary has corners where the two lines meet the axes, at $(0, 1-\delta)$ and $(1-\delta, 0)$, and where they cross, at $\alpha = \beta = (1-\delta)/(1+e^\varepsilon)$.
3. *Check the corners.* A linear function such as $\alpha + \beta$ changes at a constant rate along a straight edge, so its smallest value is at a corner. At the first two corners $\alpha + \beta = 1 - \delta$, and the crossing point is smaller because $e^\varepsilon \ge 1$:

$$\alpha + \beta = \frac{2(1-\delta)}{1+e^\varepsilon} \le 1 - \delta. \tag{18.21}$$

4. *Compute the advantage.* One minus the smallest sum is

$$1 - \frac{2(1-\delta)}{1+e^\varepsilon} = \frac{e^\varepsilon - 1 + 2\delta}{e^\varepsilon + 1}, \tag{18.22}$$

   and $\Pr[\text{correct}] = \frac{1}{2} + \frac{1}{2}\text{Adv}$ gives the second bound. Multiplying the top and bottom of the $\delta = 0$ bound by $e^{-\varepsilon/2}$ turns it into $\tanh(\varepsilon/2)$.
5. *Tightness and random datasets.* Against randomized response, the test "say D' on report 0" has $\alpha = \beta = 1/(1+e^\varepsilon)$, the crossing point, so it attains the bound. A bound that holds for each fixed pair also holds for an average over random pairs.

□

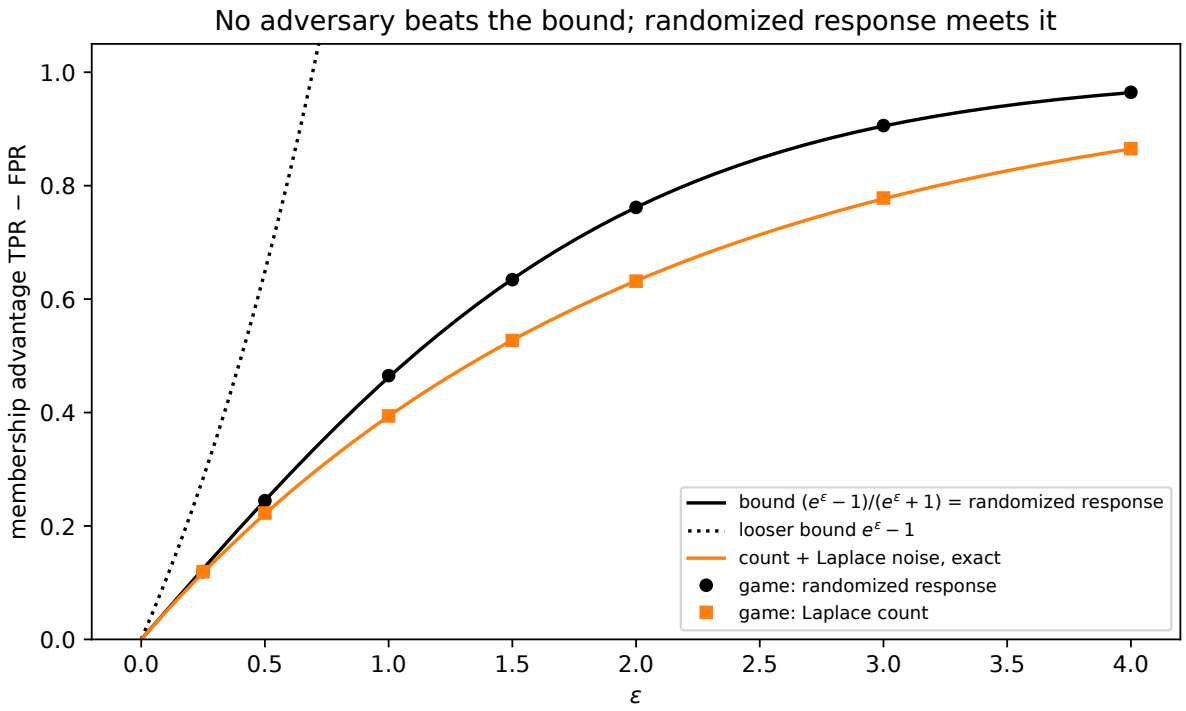


Figure 18.5.: The largest membership advantage under ε-DP (black), the looser bound $e^\varepsilon - 1$ (dotted) and the exact best advantage against the Laplace count (orange). Markers: 100,000 simulated games with the likelihood-ratio adversary.

At $\varepsilon = 0.1$ the best attacker is right with probability at most 0.525, while at $\varepsilon = 5$ it may be right 99.3% of the time. The older bound $e^{\varepsilon} - 1$ (Yeom et al., 2018) is about twice as large and says nothing once $e^{\varepsilon} \geq 2$. A count with Laplace noise of scale $1/\varepsilon$ has best advantage $1 - e^{-\varepsilon/2}$, from Section 3.6, which stays below $\tanh(\varepsilon/2)$ (Figure 18.5).

Example 18.6. Table 18.4 compares the bound with the best advantage against two mechanisms. In 200,000 games with the likelihood-ratio adversary, which guesses D when the privacy loss is positive, the script measures 0.4624 against randomized response (exact 0.4621) and 0.3953 against the Laplace count (exact 0.3935) at $\varepsilon = 1$.

Table 18.4.: The membership advantage bound for ε-DP and the exact best advantage against two ε-DP mechanisms.

ε	0.1	0.5	1	$\ln 3$	2	5
Bound $\tanh(\varepsilon/2)$	0.0500	0.2449	0.4621	0.5000	0.7616	0.9866
Looser bound $e^{\varepsilon} - 1$	0.1052	0.6487	1.7183	2.0000	6.3891	147.4
Randomized response	0.0500	0.2449	0.4621	0.5000	0.7616	0.9866
Laplace count	0.0488	0.2212	0.3935	0.4226	0.6321	0.9179

18.7. The Bayesian interpretation

People often ask a different question: after seeing the output, how sure can someone be about me? Bayes' rule in odds form (Section 4.8) says that the posterior odds are the prior odds times the likelihood ratio, which for an output y is $e^{L_{D,D'}(y)}$. So a bound on the privacy loss is a bound on how far any belief can move, whatever the prior (Kasiviswanathan & Smith, 2014). Section D.2 in the appendices derives it with a two-hypothesis table.

Take an adversary who knows that the data are D or D' and gives D the prior probability π. Under approximate DP, outputs whose loss exceeds a level $\varepsilon' > \varepsilon$ are rare, because each adds a fixed fraction of its probability to the excess that δ bounds.

Theorem 18.4 (What one output does to a belief)**.** *If M is ε-DP, then after any output y,*

$$e^{-\varepsilon}\frac{\pi}{1-\pi} \leq \frac{\Pr[D \mid y]}{\Pr[D' \mid y]} \leq e^{\varepsilon}\frac{\pi}{1-\pi}, \qquad \Pr[D \mid y] \leq \frac{e^{\varepsilon}\pi}{e^{\varepsilon}\pi + 1 - \pi}. \tag{18.23}$$

If M is (ε, δ)-DP, then for every $\varepsilon' > \varepsilon$ the upper bounds hold with ε' in place of ε for every output except those in the set $B = \{y : L_{D,D'}(y) > \varepsilon'\}$, which has

$$\Pr_{y \sim M(D)}[y \in B] \leq \frac{\delta}{1 - e^{\varepsilon - \varepsilon'}}. \tag{18.24}$$

Proof. The plan is to write the posterior odds as prior odds times e^L, to bound e^L under pure DP, and then to show that the outputs where L is large are rare under approximate DP.

1. *Odds.* By Bayes' rule, $\Pr[D \mid y]$ is $\pi\, p_{M(D)}(y)$ divided by the overall probability of y, and the same holds for D' with the same denominator. The ratio cancels it and leaves the prior odds times $e^{L_{D,D'}(y)}$.
2. *Pure DP.* By Proposition 18.1, e^L lies between $e^{-\varepsilon}$ and e^{ε}, and multiplying the prior odds by these extremes gives the odds bound.
3. *From odds to a probability.* Odds o convert to the probability $o/(1+o)$, which increases with o. Taking $o = e^{\varepsilon}\pi/(1-\pi)$ and multiplying top and bottom by $1-\pi$ gives the bound on $\Pr[D \mid y]$.
4. *The bad set is rare.* Write $P = M(D)$ and $Q = M(D')$, and apply Equation 18.1 to B:

$$\Pr_P[B] - e^{\varepsilon} \Pr_Q[B] \le \delta. \tag{18.25}$$

   On B the loss exceeds ε', so $q < e^{-\varepsilon'} p$ and $\Pr_Q[B] < e^{-\varepsilon'} \Pr_P[B]$. Putting this larger quantity in place of $\Pr_Q[B]$ makes the left side smaller, which leaves

$$\Pr_P[B](1 - e^{\varepsilon - \varepsilon'}) < \delta, \tag{18.26}$$

   and the factor is positive because $\varepsilon' > \varepsilon$.
5. *Outside the bad set.* There the loss is at most ε', and steps 1 to 3 with ε' in place of ε give the upper bounds.

□

DP bounds the *change* in belief, not the belief, so an adversary who is already 99% sure stays about that sure. Against adversaries who do not know every other record, the same bound holds relative to the posterior computed without the person's record (Kasiviswanathan & Smith, 2014).

Example 18.7. In the survey, the interviewer knows every answer but Alice's and gives a 50% chance that she smokes. Her "yes" is 3 times as likely from a true "yes", so the posterior odds are at most 3 and the interviewer is at most 75% sure. From a prior of 1%, the posterior is at most 0.0294 (Table 18.5).

Table 18.5.: The largest posterior probability after one output of an ε-DP mechanism, from Equation 18.23.

ε	Prior 0.01	Prior 0.1	Prior 0.5
0.1	0.0110	0.1094	0.5250
1	0.0267	0.2320	0.7311
$\ln 3$	0.0294	0.2500	0.7500
3	0.1687	0.6906	0.9526

For the $(1, 0.0068)$-DP Gaussian count, Equation 18.24 with $\varepsilon' = 2$ bounds the probability of a loss above 2 by $0.0068/(1-e^{-1}) = 0.0108$, while the exact value is 8.842×10^{-5}. The bound uses only the pair (ε, δ), not the shape of the tail.

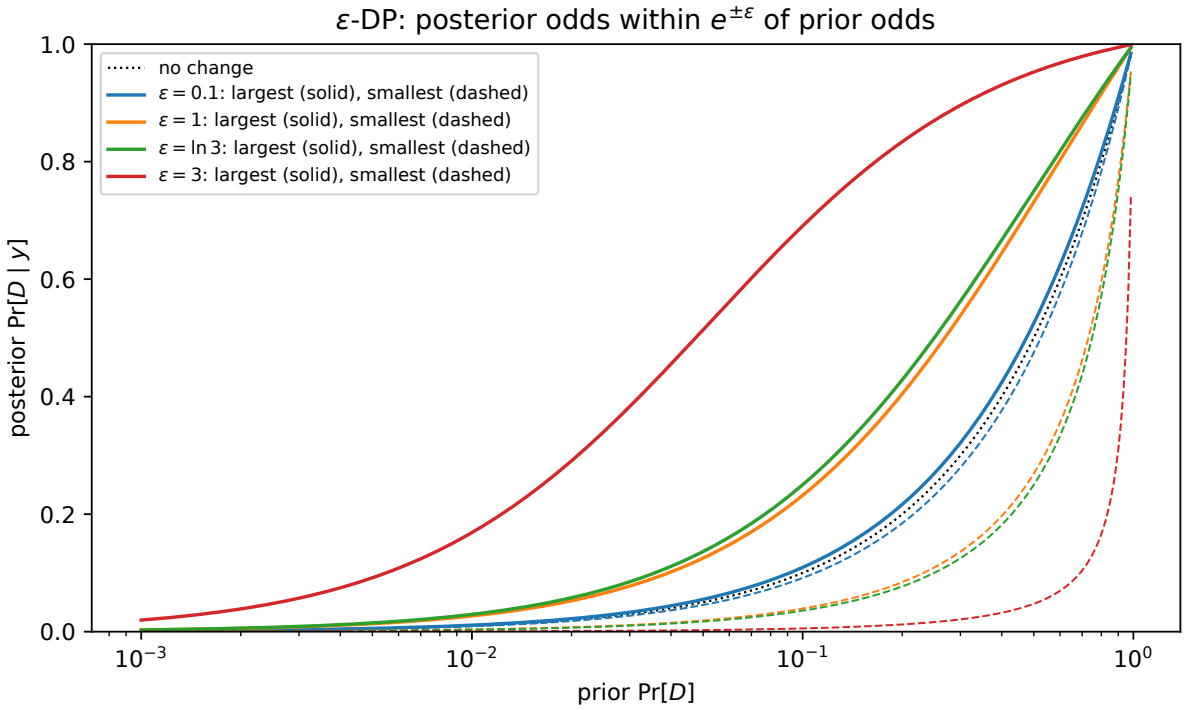


Figure 18.6.: The largest (solid) and smallest (dashed) posterior after one output of an ε-DP mechanism, against the prior, for four values of ε. Even at $\varepsilon = 3$ a prior of 0.01 rises only to 0.169.

18.8. Why δ must be much smaller than 1/n

How small must δ be? A deliberately terrible mechanism shows the danger. It is DP with $\varepsilon = 0$ and yet publishes records verbatim, because over a whole dataset its small chances add up.

Proposition 18.3 (Release each record with probability δ)**.** *The mechanism that publishes each record of the dataset, in full, independently with probability δ is $(0, \delta)$-DP under add/remove. On a dataset of n records it publishes $n\delta$ records on average, and at least one record with probability*

$$1 - (1 - \delta)^n \approx 1 - e^{-n\delta}, \tag{18.27}$$

so $\delta = 1/n$ exposes someone with probability about $1 - e^{-1} = 0.632$.

Proof. The plan is to run the mechanism on D and on $D' = D \cup \{x\}$ with the same coins, so that the outputs differ only through one extra coin, and then to count the exposures.

1. *Couple the two runs.* Run the mechanism on D' by first running it on D and then tossing one more coin for x. The two outputs are equal unless that coin publishes x, which happens with probability δ.
2. *Bound one direction.* The output on D' lands in an event S either because the output on D did or because the extra coin changed it, so
$$\Pr[M(D') \in S] \leq \Pr[M(D) \in S] + \delta. \tag{18.28}$$
3. *Bound the other direction.* With probability $1 - \delta$ the extra coin does nothing, and a probability is at most 1, so
$$\Pr[M(D') \in S] \geq (1 - \delta) \Pr[M(D) \in S] \geq \Pr[M(D) \in S] - \delta. \tag{18.29}$$
Both directions match Equation 18.1 with factor $e^0 = 1$ and slack δ.
4. *Count the exposures.* The number of records published is $\mathrm{Bin}(n, \delta)$, with mean $n\delta$, and none is published with probability $(1 - \delta)^n$. For small δ, $\ln(1 - \delta) \approx -\delta$, which gives
$$(1 - \delta)^n = e^{n \ln(1-\delta)} \approx e^{-n\delta}. \tag{18.30}$$

□

So δ is a per-person budget for catastrophe. Pick it well below $1/n$, with n the number of people rather than records. A common choice is $\delta = 1/n^{1.5}$, and fixed values need checking: Abadi et al. (2016) used $\delta = 10^{-5}$ for 60,000 training examples, so $n\delta = 0.6$.

One pair (ε, δ) can describe mechanisms that fail in very different ways (Figure 18.7). The Gaussian count with $\sigma = 2$ is $(1, 6.830 \times 10^{-3})$-DP, and its loss exceeds 3 with probability 4.462×10^{-9}. Publishing a person's bit with probability 6.830×10^{-3}, and otherwise running randomized response at $\varepsilon = 1$, gives the same pair, with the whole δ on outputs of infinite loss. From the pair alone, Equation 18.24 at level 3 allows

$$\Pr[L > 3] \leq \frac{6.830 \times 10^{-3}}{1 - e^{1-3}} = 7.899 \times 10^{-3}, \tag{18.31}$$

and the catastrophic mechanism comes close to it. So read a quoted pair pessimistically unless you know the whole curve $\delta(\varepsilon)$.

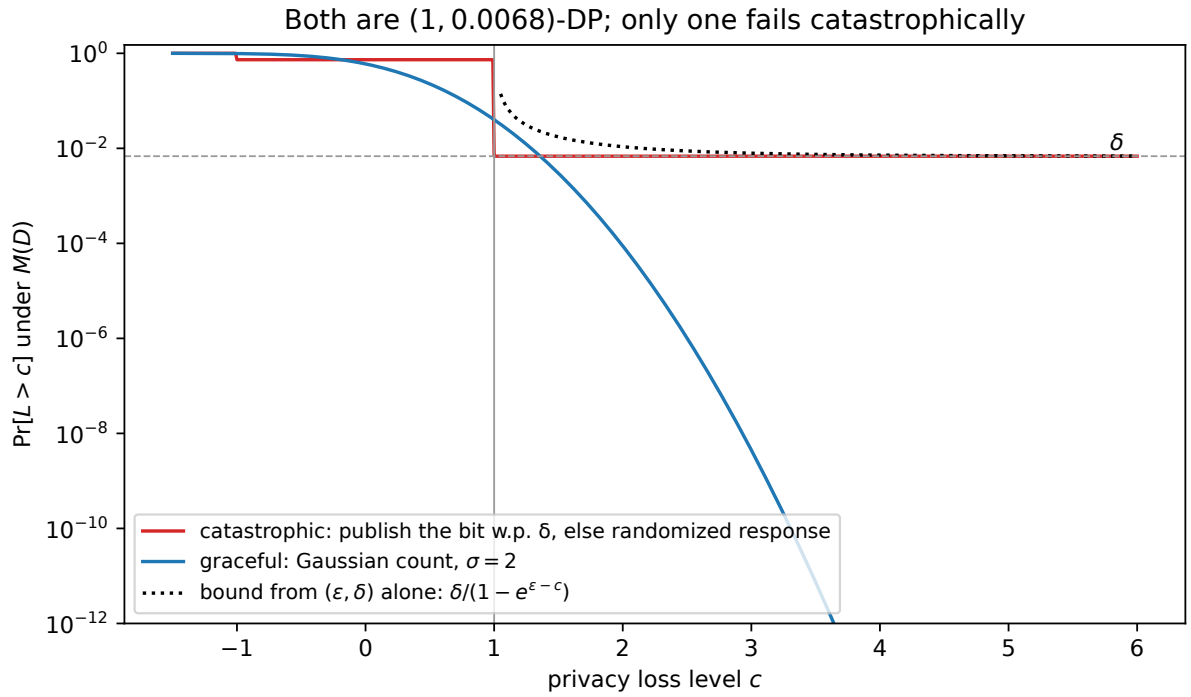


Figure 18.7.: The tail $\Pr[L > c]$ of two $(1, 0.0068)$-DP mechanisms on a log scale. Blue: the Gaussian count, whose tail falls fast. Red: publish the bit with probability δ, else randomized response; its tail stays at δ. Dotted: the largest tail the pair allows, from Equation 18.24.

Example 18.8. With $n = 3$ and $\delta = 0.1$ the script enumerates every output on D and on $D \cup \{x\}$ and finds a largest gap over events of 0.1000, so the mechanism is $(0, 0.1)$-DP and no better. Table 18.6 takes $n = 10{,}000$ records. In 100,000 runs at $\delta = 1/n$ it published 1.0077 records on average and at least one in a fraction 0.6351 of the runs, against 1 and 0.6321 from the formulas (Figure 18.8).

Table 18.6.: Exposure under the $(0, \delta)$-DP mechanism that releases each of 10,000 records with probability δ.

δ	Expected records published, $n\delta$	Pr[at least one]
$10^{-4} = 1/n$	1	0.6321
10^{-6}	0.01	9.950×10^{-3}
10^{-8}	0.0001	9.999×10^{-5}

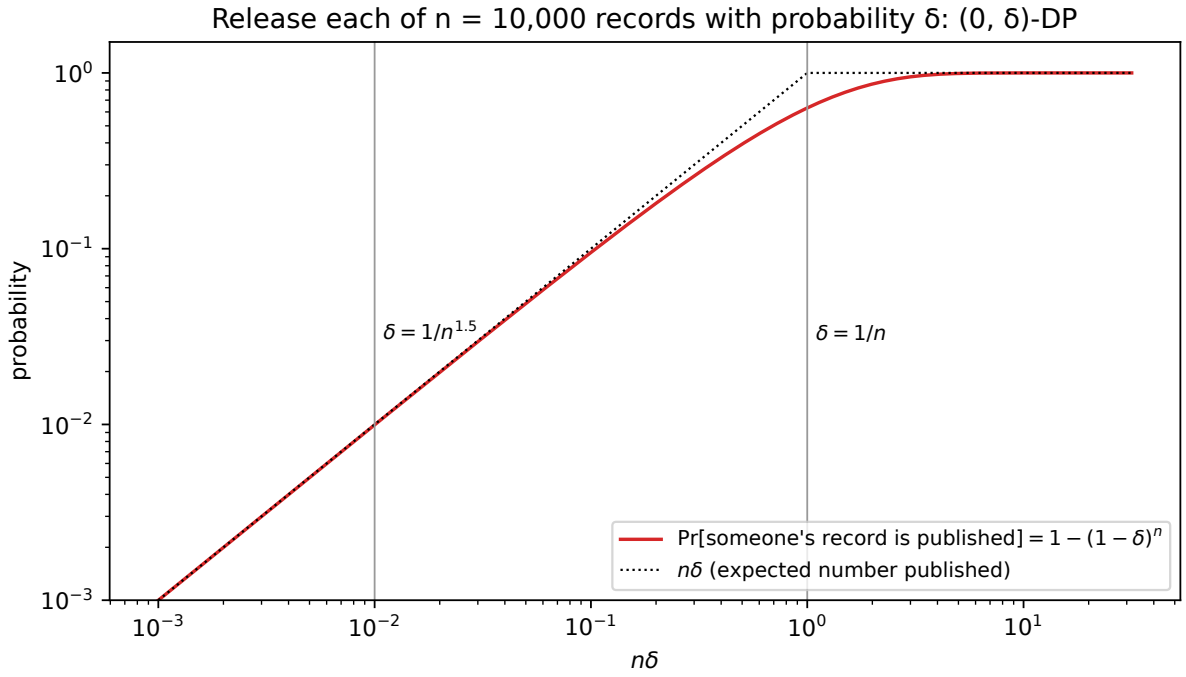


Figure 18.8.: The probability of publishing at least one of 10,000 records against $n\delta$, on log axes. Below $n\delta \approx 0.1$ it is essentially $n\delta$, and at $\delta = 1/n$ it is already 0.63.

18.9. What differential privacy does not protect

The definition compares two worlds that differ in one record, not the world before and after the release. Kifer & Machanavajjhala (2011) showed that no definition can block every inference without assumptions on how the data were generated. Four limits matter in practice.

Population inference. Suppose the cancer rate among smokers is 0.1 or 0.3, equally likely, so your belief that Alice, a smoker, will get cancer is 0.2. A study that excludes her reveals the rate 0.3, and your belief rises to 0.3. Yet the study never saw her record, so nothing happened to her *because she took part*.

Correlated records. If k records move together, as in a household, their joint privacy loss can reach $k\varepsilon$, so the unit of privacy must match the unit of correlation.

Everything outside the mathematical mechanism. Floating-point leaks (Discrete and floating-point-safe mechanisms), timing side channels and other releases need separate care.

Tasks about one person. Contact tracing must find whom one infected person met, so its output must depend on that person, which DP forbids. DP protects your contribution, not secrets that other people's data reveal.

Example 18.9. A count with Laplace noise of scale $1/\varepsilon$ meets a household of $k = 5$ people who always answer alike. At $\varepsilon = 1$ the script finds a largest privacy loss of 5.0000 for the household, which is protected only as if by a 5-DP mechanism. The exact best advantage against it is $1 - e^{-5/2} = 0.9179$, under the 5-DP bound $\tanh(5/2) = 0.9866$. An attacker who asks whether the household took part is right with probability up to 0.959, although each record is "protected at $\varepsilon = 1$".

18.10. Is it a privacy violation?

Some reported privacy failures show that a release reveals something about the people whose data it used, and others that it reveals something true about the world, which then applies to particular people. DP is designed to stop only the first kind. Three questions from Kamath's course on private data analysis sort a claim into one kind or the other.

1. **Does it depend on this person's record?** DP bounds an inference's probability by e^{ε} times its probability without the record, plus δ, so it has nothing against an inference that is as likely without the record.
2. **Does the inference succeed exactly when the model is accurate?** Then it comes from patterns shared by many records, and blocking it is a decision not to learn.
3. **When, and by whom, was the information given up?** An inference from what the person disclosed elsewhere, such as her purchases, is the same whether or not her record was used.

The first question decides, and the other two explain a no. To measure a claimed attack, compare its success on members with its success on non-members, because only the excess can be a privacy loss (Attribute inference). An inference that is no violation can still be unacceptable, since norms of information flow (Nissenbaum, 2004) cover far more than DP.

Example 18.10. Train a regression model on points that all lie on the line $y = x + 1$, among them $(2.7, 3.7)$. Asked about $x = 2.7$ it answers the training label 3.7, which looks like a leak. But without that point the line and the answer stay the same, so the first question says no. Table 18.7 applies the test to four well-known cases.

Table 18.7.: Four claimed privacy failures and the three-question test.

Case	A violation in the DP sense?	The question that decides it
Facebook likes predict politics (Kosinski, Stillwell & Graepel, 2013)	No	First: the patterns hold without any one user
A retailer infers a shopper's pregnancy from her purchases (Duhigg, 2012; anecdotal)	No	First: a model not trained on her infers the same
Netflix Prize data linked to public reviews (Narayanan & Shmatikov, 2008)	Yes	First: without her record there is nothing to find
A language model completes one user's identification number (Carlini et al., 2019)	Yes	First: only her messages make it likely

The last two rows are the contrast that matters for machine learning: generalizing passes the first question, and memorizing a rare string fails it (Memorization and extraction).

18.11. Code

The script code/definition.py computes every number in this chapter and draws every figure. It checks the definition by brute force over small outcome sets, and the closed forms by simulation. From the repository root:

```
uv run python code/part03-dp-foundations/definition.py
```

Table 18.8.: The script's output with the default seed.

Experiment	Result
Histogram, Laplace noise $1/\varepsilon$ per bin, $\varepsilon = 1$	Largest loss 1.0000 (add/remove), 2.0000 (replace)
Disjoint uniform distributions, $m = 1000$	Pointwise δ 0.0010; event δ 1.0000
Additive-only definition, $n = 1000$	TV 0.1000, accuracy 0.5500; exposure 0.6323
Randomized response, 3 people, 256 events	Event ratio 3.0000 = outcome ratio; hockey-stick at $\gamma = 2$ 0.2500
Survey, $n = 1000$, $\varepsilon = \ln 3 = 1.0986$	sd 0.0274, simulated 0.0278 (mean 0.2996); central 0.00129
$\delta(0.5)$, $\delta(1)$	Randomized response 0.3378, 0.0704; Laplace 0.2212, 0; Gaussian Table 18.3
TPR at FPR 10^{-3}	Cap 0.002728 at $(1, 10^{-5})$; Gaussian 0.0048, cap 0.0095

Experiment	Result
Membership advantage	Table 18.4; games at $\varepsilon = 1$: 0.4624, 0.3953
Bayesian	Table 18.5; Gaussian $\Pr[L > 2] = 8.842 \times 10^{-5}$, bound 0.0108
Release each record	$(0, 0.1)$-DP at $n = 3$; 1.0077 records, $\Pr[\geq 1] = 0.6351$ at $n = 10^4$
$\Pr[L > 3]$ at $(1, 6.830 \times 10^{-3})$	Gaussian 4.462×10^{-9}, catastrophic 6.830×10^{-3}, bound 7.899×10^{-3}
Household of 5, $\varepsilon = 1$	Largest loss 5.0000; bound 0.9866; Laplace 0.9179

tests/test_definition.py checks that the output still matches.

18.12. Summary

Table 18.9.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 18.1	Add/remove (n private) or replace (n public)	Name it with every ε; subsampling
Definition 18.2	$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta$	Every mechanism of this part
Definition 18.3	$L_{D,D'}(y) = \ln\left(p_{M(D)}(y)/p_{M(D')}(y)\right)$	What every reading of the definition bounds
Proposition 18.1	ε-DP iff $\lvert L\rvert \leq \varepsilon$ everywhere	Prove pure DP output by output, never approximate DP
Proposition 18.2	Add/remove ε gives replace 2ε	With δ: $(2\varepsilon, (1 + e^{\varepsilon})\delta)$
Theorem 18.1	Truth with probability $e^{\varepsilon}/(1 + e^{\varepsilon})$; error about $1/(\varepsilon\sqrt{n})$	The local model; worst case for composition
Theorem 18.2	(ε, δ)-DP iff $E_{e^{\varepsilon}}(P\|Q) \leq \delta$	Quote the curve $\delta(\varepsilon)$; numerical accounting
Theorem 18.3	$\text{TPR} \leq e^{\varepsilon}\,\text{FPR} + \delta$	Auditing; f-DP
Corollary 18.1	$\text{Adv} \leq (e^{\varepsilon} - 1 + 2\delta)/(e^{\varepsilon} + 1)$	ε below 1 is strong; around 10 it says little
Theorem 18.4	Posterior odds within $e^{\pm\varepsilon}$ of prior odds	Explaining ε
Proposition 18.3	$(0, \delta)$-DP, yet exposes someone with probability $1 - (1 - \delta)^n$	Pick $\delta \ll 1/n$
Example 18.9	k correlated records cost $k\varepsilon$	Match the unit of privacy to the unit of correlation

Result	Statement	Use
Example 18.10	Record? Accuracy? Disclosure?	Only the excess over non-members is a privacy loss

18.13. References

- Dwork, McSherry, Nissim & Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (TCC 2006; journal version in the Journal of Privacy and Confidentiality, 2016). The original definition of pure DP, with sensitivity and the Laplace mechanism.
- Dwork, Kenthapadi, McSherry, Mironov & Naor, "Our Data, Ourselves: Privacy via Distributed Noise Generation" (EUROCRYPT 2006). Introduces approximate (ε, δ)-DP.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 2: the definition, randomized response, what DP promises and does not (§2.3), and the "release a random record" argument for small δ. The standard reference for this whole chapter.
- Warner, "Randomized Response: A Survey Technique for Eliminating Evasive Answer Bias" (Journal of the American Statistical Association, 1965). The original spinner design, which answers truthfully with a known probability, and its estimator.
- Boruch, "Assuring Confidentiality of Responses in Social Research: A Note on Strategies" (The American Sociologist, 1971). The forced-response design, of which the two-coin protocol is an instance.
- Vadhan, "The Complexity of Differential Privacy" (2017), §1.6. Why a purely additive (total variation) definition cannot be both useful and private.
- Wasserman & Zhou, "A Statistical Framework for Differential Privacy" (Journal of the American Statistical Association, 2010). DP as a bound on the power of tests.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015). The hypothesis-testing characterization Equation 18.17 and its use for optimal composition (Advanced and optimal composition).
- Dong, Roth & Su, "Gaussian Differential Privacy" (JRSS Series B, 2022). Trade-off functions as the privacy definition itself (f-DP); Equation 18.18 is their $f_{\varepsilon,\delta}$.
- Kasiviswanathan & Smith, "On the 'Semantics' of Differential Privacy: A Bayesian Formulation" (Journal of Privacy and Confidentiality, 2014). The posterior reading for arbitrary priors, and the bad-set argument for approximate DP.
- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (CSF 2018). Membership advantage and the bound $e^{\varepsilon} - 1$.
- Kifer & Machanavajjhala, "No Free Lunch in Data Privacy" (SIGMOD 2011). Why DP cannot protect correlated data without assumptions on how it was generated.
- Desfontaines & Pejó, "SoK: Differential Privacies" (PETS 2020). A map of the many variants of the definition, including the choice of neighbouring relation.
- Abadi et al., "Deep Learning with Differential Privacy" (CCS 2016). Example-level DP for training neural networks, with $\delta = 10^{-5}$ for 60,000 examples.
- Kamath, "Algorithms for Private Data Analysis" (University of Waterloo, recorded lectures, 2020). The three-question test for privacy violations, and the catastrophic and graceful readings of δ.
- Kosinski, Stillwell & Graepel, "Private Traits and Attributes Are Predictable from Digital

Records of Human Behavior" (PNAS, 2013); Duhigg, "How Companies Learn Your Secrets" (New York Times Magazine, 2012). Inferences from public or volunteered data, the "not a violation" side of Table 18.7.

- Narayanan & Shmatikov, "Robust De-anonymization of Large Sparse Datasets" (IEEE S&P 2008); Carlini, Liu, Erlingsson, Kos & Song, "The Secret Sharer" (USENIX Security 2019). Two failures that depend on a person's own record.
- Nissenbaum, "Privacy as Contextual Integrity" (Washington Law Review, 2004). Norms of information flow, which cover far more than DP.

19. Post-processing, group privacy and composition

You have proved that one mechanism satisfies differential privacy (DP), and now you use it: you round its output, you protect users who each contributed fifty records, and you answer a hundred queries rather than one. So three questions come up. Is a processed output still private? What does a guarantee about one record give for a group of k records? And what is the guarantee of many mechanisms run on the same data?

The answers are the three properties that make DP usable as an engineering tool. Two datasets D and D' are neighbours, written $D \sim D'$, when they differ by adding or removing one record, unless a section says otherwise. One idea organizes all three properties, the **privacy loss** of an output y from the definition of differential privacy, the log of how much more likely y is on D than on D':

$$L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}. \tag{19.1}$$

Here $p_{M(D)}$ is the probability, or density, that the mechanism M gives to y on D. Pure ε-DP, the case $\delta = 0$ of (ε, δ)-DP, keeps this loss within $[-\varepsilon, \varepsilon]$ for every output and every neighbouring pair. When you walk through several datasets, or run mechanisms whose coins (their internal randomness) are independent, likelihood ratios multiply, so privacy losses add, and that is what costs. Processing instead merges outputs and averages their likelihood ratios, which is free.

The running example is **randomized response** at level $\varepsilon_0 = \ln 3$: a person whose record is one private bit reports it truthfully with probability $\frac{3}{4}$ and flips it with probability $\frac{1}{4}$. If the bit is 1 the report 1 has probability $\frac{3}{4}$, and if it is 0 it has probability $\frac{1}{4}$, a ratio of 3, while the report 0 has the ratio $\frac{1}{3}$. So the privacy loss is $\pm \ln 3$ on every output. The output distributions $M(D)$ and $M(D')$ are written P and Q, with mass functions or densities p and q, and a sum over continuous outputs means an integral.

19.1. Post-processing

A private output is rarely published as is. You round a noisy count, fit a model to noisy marginals, or let an analyst decide what to do next, each time without looking at the data again. This section asks whether such processing can undo the protection. It cannot, because a processed output merges outputs, and its likelihood ratio is an average of theirs, which stays in the band $[e^{-\varepsilon}, e^{\varepsilon}]$. For approximate DP the privacy loss is not bounded, so the proof works with events and needs one small lemma first.

In its event form, from the definition chapter, (ε, δ)-DP says that $\Pr_P[S] \le e^{\varepsilon} \Pr_Q[S] + \delta$ for every set S of outputs and every ordered neighbouring pair. Step 1 of the proof of post-processing turns an event about the processed output into a soft rule on the original output, one that accepts each output with some probability, and the lemma says that soft rules obey the same bound.

Lemma 19.1 (Soft tests obey the definition)**.** *Let M be (ε, δ)-DP, fix a neighbouring pair with output distributions $P = M(D)$ and $Q = M(D')$, and let g be any function of the output with values in $[0, 1]$. Then*

$$\mathbb{E}_P[g(Y)] \le e^{\varepsilon}\, \mathbb{E}_Q[g(Y)] + \delta. \tag{19.2}$$

Proof.

1. *Write the gap as a sum.* The difference $\mathbb{E}_P[g] - e^{\varepsilon}\mathbb{E}_Q[g]$ equals $\sum_y g(y)(p(y) - e^{\varepsilon}q(y))$, a sum in which each value $g(y)$ multiplies a fixed bracket.
2. *Push it to the extreme.* The sum is largest when $g(y) = 1$ wherever the bracket is positive and $g(y) = 0$ wherever it is negative, because any value in between gives up part of a positive term or adds part of a negative one. That choice of g is the indicator of the event $B = \{y : p(y) > e^{\varepsilon}q(y)\}$, so the sum is at most $\Pr_P[B] - e^{\varepsilon}\Pr_Q[B]$.
3. *Apply the definition* to the event B, which bounds $\Pr_P[B] - e^{\varepsilon}\Pr_Q[B]$ by δ.

□

The lemma is the randomized version of the worst-event argument of Divergences. The largest value of $\Pr_P[S] - e^{\varepsilon}\Pr_Q[S]$ over events S is the **hockey-stick divergence**,

$$E_{e^{\varepsilon}}(P\|Q) = \sum_y \big(p(y) - e^{\varepsilon}q(y)\big)_+, \tag{19.3}$$

where $(x)_+ = \max(x, 0)$ keeps the positive parts. So M is (ε, δ)-DP exactly when that divergence is at most δ for every ordered neighbouring pair. The smallest such δ at each ε is the **privacy profile** $\delta(\varepsilon)$ of M, and the script computes it exactly for every mechanism in this chapter.

To cover randomized processing, describe it as a **channel** K: $K(z \mid y)$ is the probability that the processing outputs z when its input is y. A deterministic function f is the case $K(z \mid y) = \mathbf{1}[z = f(y)]$, which is 1 when $z = f(y)$ and 0 otherwise. The processed output on D has the distribution $PK(z) = \sum_y p(y)K(z \mid y)$, which weights each y by how likely it is to become z, and $L^f_{D,D'}(z)$ is its privacy loss. The theorem says that processing never makes D and D' easier to tell apart.

Theorem 19.1 (Post-processing)**.** *Let M be (ε, δ)-DP and let f be any randomized function of its output whose coins are independent of the data and of M's coins. Then $f \circ M$, the mechanism $D \mapsto f(M(D))$, is (ε, δ)-DP. In terms of the channel K of f, for every neighbouring pair the hockey-stick divergence can only go down, and so can the largest privacy loss:*

$$E_{e^{\varepsilon}}(PK \,\|\, QK) \le E_{e^{\varepsilon}}(P\|Q), \qquad \max_z \big|L^f_{D,D'}(z)\big| \le \max_y \big|L_{D,D'}(y)\big|. \tag{19.4}$$

Proof. The plan is to turn any event about the processed output into a soft test on the original output and apply Lemma 19.1, then to show the privacy-loss form by the averaging argument of the data processing inequality (DPI).

1. *Fix an event S of processed outputs.* Let $g(y) = \sum_{z \in S} K(z \mid y)$, the probability that processing y lands in S. It is a probability, so it takes values in $[0, 1]$, and it does not depend on the data, because f does not look at the data: the same g serves for D and for D'.

2. *Average over the output.* To land in S, the mechanism first produces some y, and then processing sends it into S with probability $g(y)$, so $\Pr[f(M(D)) \in S] = \sum_y p(y)g(y) = \mathbb{E}_P[g(Y)]$, and the same with q for D'.
3. *Apply the lemma to g.* Lemma 19.1 gives $\mathbb{E}_P[g] \le e^{\varepsilon}\mathbb{E}_Q[g] + \delta$, that is $\Pr[f(M(D)) \in S] \le e^{\varepsilon}\Pr[f(M(D')) \in S] + \delta$. This holds for every S and every ordered neighbouring pair, which is the event form of (ε, δ)-DP for $f \circ M$. The hockey-stick inequality of Equation 19.4 is the same statement for the worst event S.
4. *The privacy-loss form.* The likelihood ratio of z is $PK(z)/QK(z)$. In the numerator, for outputs with $q(y) > 0$, write each $p(y)$ as $q(y)$ times its likelihood ratio $e^{L(y)} = p(y)/q(y)$. That turns the ratio into a weighted average of the likelihood ratios of the y that can produce z:

$$e^{L^f(z)} = \frac{PK(z)}{QK(z)} = \sum_y \frac{q(y)K(z \mid y)}{QK(z)}\, e^{L(y)}. \tag{19.5}$$

   The weight of each y is the share of the D'-probability of z that comes through y. The weights are non-negative and sum to 1, because $\sum_y q(y)K(z \mid y) = QK(z)$. So $e^{L^f(z)}$ lies between the smallest and the largest $e^{L(y)}$, and $|L^f(z)|$ is at most the largest $|L(y)|$, which is the second inequality of Equation 19.4. This is step 1 of the DPI derivation in Divergences.

□

So nothing downstream needs a new proof. A model trained on DP statistics, or a plot of them, never reads the data again and inherits the guarantee. Attackers are covered too: a membership-inference attacker of Part 2 applies a channel to y, so the bounds of the definition chapter on its error rates hold for every attack. An analyst who picks the next question from earlier private answers is post-processing them, though answering that question is a new mechanism (Section 19.3; Adaptive data analysis).

Invertible processing, such as a change of units, keeps the privacy loss exactly, since nothing is averaged. Processing that merges outputs with different losses lowers it, though usually only a little. The one condition is that f must not read the data, because step 1 needed the same g under D and D'. Looking up the exact count of the noisy winning category is therefore a second, non-private mechanism (Section 19.5).

Example 19.1. Collect answers with randomized response at $\varepsilon_0 = \ln 3$ and then, before publishing, flip each answer with probability $r = \frac{1}{4}$ using a fresh coin. The published answer $z = 1$ can be reached in two ways: the first answer was 1 and was kept, or it was 0 and was flipped. Each path's probability is the product of its two steps, and Table 19.1 follows both paths.

Table 19.1.: Randomized response with $\varepsilon_0 = \ln 3$, then a fresh coin that flips the answer with probability $\frac{1}{4}$. The published $z = 1$ is reached by two paths.

Path to $z = 1$	Probability under bit 1 (D)	Probability under bit 0 (D')	$e^{L(y)}$ of the first answer
$y = 1$, kept	$\frac{3}{4} \cdot \frac{3}{4} = \frac{9}{16}$	$\frac{1}{4} \cdot \frac{3}{4} = \frac{3}{16}$	3
$y = 0$, flipped	$\frac{1}{4} \cdot \frac{1}{4} = \frac{1}{16}$	$\frac{3}{4} \cdot \frac{1}{4} = \frac{3}{16}$	$\frac{1}{3}$
total	$\frac{10}{16} = 0.625$	$\frac{6}{16} = 0.375$	—

The likelihood ratio of $z = 1$ is $0.625/0.375 = \frac{5}{3}$. Step 4 of the proof predicts it as the average of 3 and $\frac{1}{3}$, each weighted by its share $\frac{1}{2}$ of the D' column. The published $z = 0$ has the ratio $\frac{3}{5}$, so the published answers are $\ln \frac{5}{3} = 0.5108$-DP, less than $\ln 3 = 1.0986$. The right panel of Figure 19.1 shows ε for every flip probability: it falls to 0 at $r = \frac{1}{2}$, where the published bit is a fair coin.

A count with noise shows that processing can also lower δ while keeping ε. Add Laplace noise of scale 1 to a count whose neighbouring values are 0 and 1, so that the outputs are $\mathrm{Lap}(0, 1)$ and $\mathrm{Lap}(1, 1)$, with densities $\frac{1}{2}e^{-|y|}$ and $\frac{1}{2}e^{-|y-1|}$. Their privacy loss $|y - 1| - |y|$ lies within $[-1, 1]$ by the triangle inequality, so the noisy count is 1-DP (The Laplace and Gaussian mechanisms). Table 19.2 and the left panel of Figure 19.1 give the exact privacy profiles of the noisy count, of the count rounded to an integer, and of the bit "is the noisy count above $\frac{1}{2}$?".

Table 19.2.: Exact privacy profiles of a count with $\mathrm{Lap}(0, 1)$ noise, before and after two post-processings. Each row is on or below the one above it, as Equation 19.4 requires.

Published	Smallest pure ε	$\delta(0.25)$	$\delta(0.5)$
Noisy count $c + Z$	1.0000	0.3127	0.2212
Rounded to an integer	1.0000	0.3073	0.1967
Only "is $c + Z > \frac{1}{2}$?"	0.8318	0.3073	0.1967

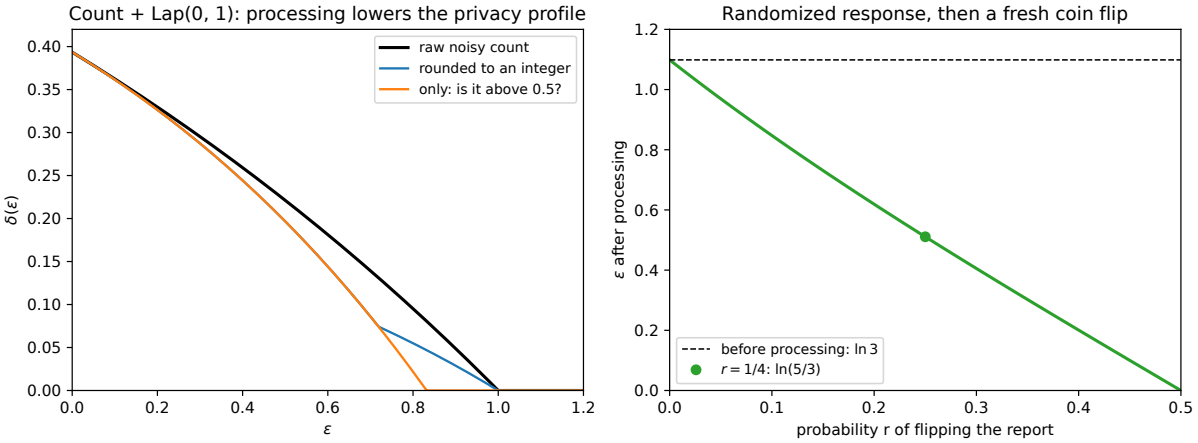


Figure 19.1.: Left: the privacy profile $\delta(\varepsilon)$ of a count with $\mathrm{Lap}(0, 1)$ noise (black), after rounding to an integer (blue), and after keeping only whether it exceeds $\frac{1}{2}$ (orange). Post-processing can only lower the curve. Right: randomized response with $\varepsilon_0 = \ln 3$ followed by a fresh coin that flips the answer with probability r; the dot is $r = \frac{1}{4}$, where $\varepsilon = \ln \frac{5}{3}$.

Rounding keeps the pure ε. Each rounded value at or below -1 gathers outputs that all lie left of 0, where the densities have the constant ratio e, and an average of equal ratios is that ratio, so those rounded values still have loss 1. It still lowers $\delta(\varepsilon)$, because the cell $[-\frac{1}{2}, \frac{1}{2}]$ mixes losses from -1 to 1. The one-bit summary is the most private. Its "yes" has probability $\frac{1}{2}e^{-1/2} = 0.3033$ when the count is 0 and 0.6967 when it is 1, a privacy loss of $\ln(0.6967/0.3033) = 0.8318$.

19.2. Group privacy

The definition protects one record, but people are not rows. A user of a messaging app contributes hundreds of messages, and a household shares an address. To bound what an adversary learns about all k records of one user, you need a guarantee for datasets that differ in k records. Group privacy derives it from the one-record guarantee, at a price that is linear in k for ε and exponential in k for δ.

Say that D and D'' are at **distance** k if a path $D = D_0 \sim D_1 \sim \cdots \sim D_k = D''$ joins them, changing one record per step. The idea is to apply the one-record guarantee once per step. Each step multiplies the bound by e^{ε} and adds a slack δ, which later steps multiply again, so the slacks pile up in a geometric sum.

Theorem 19.2 (Group privacy)**.** *Let M be (ε, δ)-DP. Then for every pair of datasets D and D'' at distance k and every event S,*

$$\Pr[M(D) \in S] \le e^{k\varepsilon} \Pr[M(D'') \in S] + \delta_k, \qquad \delta_k = \delta \sum_{j=0}^{k-1} e^{j\varepsilon} = \delta \, \frac{e^{k\varepsilon} - 1}{e^{\varepsilon} - 1} \le k \, e^{(k-1)\varepsilon} \delta. \tag{19.6}$$

In particular, pure ε-DP gives pure $k\varepsilon$-DP for groups of size k.

Proof. The plan is to prove the pure case through the privacy loss, which adds along the path, and the approximate case through events, chaining the one-step bound k times.

1. *Pure DP: telescope the likelihood ratio along the path.* Multiply and divide by the densities at the intermediate datasets $D_1, \dots, D_{k-1}$. Each appears once in a numerator and once in a denominator, so they cancel and only the two ends remain:
$$\frac{p_{M(D_0)}(y)}{p_{M(D_k)}(y)} = \prod_{i=1}^{k} \frac{p_{M(D_{i-1})}(y)}{p_{M(D_i)}(y)}, \qquad \text{so} \qquad L_{D_0, D_k}(y) = \sum_{i=1}^{k} L_{D_{i-1}, D_i}(y). \tag{19.7}$$
2. *Bound each term.* Each pair $D_{i-1} \sim D_i$ is neighbouring, so $|L_{D_{i-1}, D_i}(y)| \le \varepsilon$ by pure DP.
3. *Add.* A sum of k terms, each between $-\varepsilon$ and ε, lies between $-k\varepsilon$ and $k\varepsilon$, so $|L_{D_0, D_k}(y)| \le k\varepsilon$ for every y, which is pure $k\varepsilon$-DP.
4. *Approximate DP: one step.* With $\delta > 0$ the privacy loss is no longer bounded, so there is nothing to add up, and the proof chains the event form instead. Fix an event S and write s_i for the probability $\Pr[M(D_i) \in S]$. The definition applied to the neighbouring pair $D_{i-1} \sim D_i$ gives $s_{i-1} \le e^{\varepsilon} s_i + \delta$.
5. *Substitute the next step* into the right side. The bound on s_1 goes in place of s_1:
$$s_0 \le e^{\varepsilon} s_1 + \delta \le e^{\varepsilon}\big(e^{\varepsilon} s_2 + \delta\big) + \delta = e^{2\varepsilon} s_2 + \delta(1 + e^{\varepsilon}). \tag{19.8}$$
6. *Repeat k times.* By induction, $s_0 \le e^{k\varepsilon} s_k + \delta(1 + e^{\varepsilon} + \cdots + e^{(k-1)\varepsilon})$. Each new δ enters before $k-1, k-2, \dots, 0$ more multiplications by e^{ε}, which is where the geometric sum comes from.
7. *Simplify.* The geometric sum is $\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1}$, because multiplying it by $e^{\varepsilon} - 1$ cancels all terms but the first and last. Each of its k terms is at most the largest one, $e^{(k-1)\varepsilon}$, which gives the simpler but looser $k \, e^{(k-1)\varepsilon} \delta$.

□

No better bound holds for all (ε, δ)-DP mechanisms. The mechanism below shows it, because it has an event on which every one-step bound of steps 4 to 6 is an equality.

Proposition 19.1 (The alarm mechanism attains the bound)**.** *Fix $\varepsilon > 0$, $\delta \in (0, 1)$ and a cap K on the count of some kind of record. On a count $c \le K$ the* ***alarm mechanism*** *outputs "alarm" with probability*

$$a_c = \delta \sum_{j=0}^{K-c-1} e^{j\varepsilon} \tag{19.9}$$

and "quiet" otherwise, and on a count above K it always outputs "quiet". If $a_0 \le (e^\varepsilon + \delta)/(e^\varepsilon + 1)$, the mechanism is (ε, δ)-DP for neighbouring counts, which differ by 1, and for the counts 0 and $k \le K$ it meets Equation 19.6 with equality on the event "alarm".

Proof.

1. *One step is tight.* Pulling the first term δ out of the sum and a factor e^ε out of the rest shows that $a_c = e^\varepsilon a_{c+1} + \delta$ for $c < K$. So each neighbouring pair meets the event form with equality on "alarm".
2. *The mechanism is private.* The other three checks, "alarm" in the reverse order and "quiet" in both orders, hold exactly when $a_0 \le (e^\varepsilon + \delta)/(e^\varepsilon + 1)$. Section J.8 in the appendices proves it, and a simpler sufficient condition is $a_0 \le 1 - e^{-\varepsilon}$. For $\varepsilon = 0.5$ and $\delta = 10^{-5}$ the condition allows $K \le 21$.
3. *The chain is tight.* Along the path of counts $0, 1, \dots, k$, step 5 of the proof of Theorem 19.2 substitutes equalities into equalities, so $a_0 - e^{k\varepsilon} a_k = \delta \sum_{j=0}^{k-1} e^{j\varepsilon} = \delta_k$.

□

Pure DP degrades linearly. A 0.1-DP guarantee per record is 10-DP for a user with 100 records, which is almost nothing, since a likelihood ratio of up to $e^{10} \approx 22{,}000$ lets an adversary be nearly certain. This is the formal version of the "correlated records" caveat of the definition chapter. For approximate DP the δ term grows exponentially: with $\varepsilon = 1$ and $k = 10$ it is $\delta_k = \frac{e^{10}-1}{e-1}\,\delta = 12818.3\,\delta$. Since δ must already be much smaller than $1/n$ for n records (the definition chapter), group privacy for large groups is often vacuous.

A specific mechanism can do much better than the generic bound. With Gaussian noise on a count, a group of k records shifts the count by k, so its exact group profile is that of a Gaussian pair with a larger shift (Part 4). The better fix is often to define neighbours as datasets that differ in one user's records, bound each user's contribution, and prove DP at that level.

Example 19.2. For randomized response a group of k records is k people, and the reports in which all k favour the same side have privacy loss exactly $k \ln 3$, so the pure bound is exact. A noisy count shows the telescoping of step 1 at work. Take $\mathrm{Lap}(0, 1)$ noise and the path of counts $0, 1, 2, 3$, so $k = 3$. With density $\frac{1}{2}e^{-|y-c|}$ at true count c, the privacy loss between counts c and $c + 1$ is $L_{c,c+1}(y) = |y - c - 1| - |y - c|$, how much farther y is from $c + 1$ than from c. Table 19.3 adds the steps at three outputs.

Table 19.3.: Privacy losses along a path of three neighbouring counts, with $\mathrm{Lap}(0, 1)$ noise. The step losses add up to the loss between the ends.

Output y	$L_{0,1}(y)$	$L_{1,2}(y)$	$L_{2,3}(y)$	Sum	Direct: $\lvert y-3\rvert - \lvert y\rvert$
-2	$3-2=1$	$4-3=1$	$5-4=1$	3	$5-2=3$
0.5	$0.5-0.5=0$	$1.5-0.5=1$	$2.5-1.5=1$	2	$2.5-0.5=2$
1.5	$0.5-1.5=-1$	$0.5-0.5=0$	$1.5-0.5=1$	0	$1.5-1.5=0$

At $y=-2$ the output lies left of every count on the path, so every step points the same way and the sum reaches 3. At $y=1.5$ the steps point in different directions and cancel. The script confirms that the rounded noisy count has pure ε exactly k for counts that differ by $k=1,2,3,5$, so for a count group privacy is tight.

For approximate DP, protect each record at $(\varepsilon, \delta) = (0.5, 10^{-5})$. Table 19.4 compares the bound with the exact δ_k of the alarm mechanism with $K=10$ and of a count with Gaussian noise of standard deviation σ. Two normal distributions with standard deviation σ and means s apart have an exact δ in closed form, a difference of two normal tail probabilities (Divergences). The script solves that closed form for the σ that gives $\delta(0.5)=10^{-5}$ at shift 1, which is $\sigma=7.0318$, and evaluates it at the shift $s=k$. Figure 19.2 plots the same numbers up to $k=12$.

Table 19.4.: Group privacy from a $(0.5, 10^{-5})$ guarantee per record: the bound, its simple form, and the exact δ_k of two mechanisms at $\varepsilon_k = k\varepsilon$.

Group size k	$\varepsilon_k = k\varepsilon$	δ_k from Equation 19.6	Simple form $ke^{(k-1)\varepsilon}\delta$	Alarm mechanism, exact	Gaussian count, exact
1	0.5	1.000×10^{-5}	1.000×10^{-5}	1.000×10^{-5}	1.000×10^{-5}
2	1	2.649×10^{-5}	3.297×10^{-5}	2.649×10^{-5}	2.551×10^{-5}
3	1.5	5.367×10^{-5}	8.155×10^{-5}	5.367×10^{-5}	4.858×10^{-5}
5	2.5	1.724×10^{-4}	3.695×10^{-4}	1.724×10^{-4}	1.288×10^{-4}
10	5	2.272×10^{-3}	9.002×10^{-3}	2.272×10^{-3}	7.592×10^{-4}

The alarm mechanism attains the geometric sum at every k. The simple form is looser by a factor $3.297/2.649 = 1.24$ at $k=2$ and $9.002/2.272 = 3.96$ at $k=10$, so use the sum. The Gaussian count is a factor $2.272/0.7592 = 2.99$ below the bound at $k=10$. Even so, for a user with 10 records the record-level $(0.5, 10^{-5})$ becomes $(5, 7.6\times10^{-4})$, a weak guarantee.

19.3. Basic sequential composition

An analyst asks k queries of the same dataset, each with fresh noise. Each output alone is private, but an adversary sees all of them. This section asks what guarantee holds for the whole transcript $(y_1, \dots, y_k)$, even when each mechanism is chosen after seeing the previous outputs. For pure DP the answer rests on one fact about independent coins, which step 2 of the proof needs and parallel composition reuses: probabilities multiply, so privacy losses add.

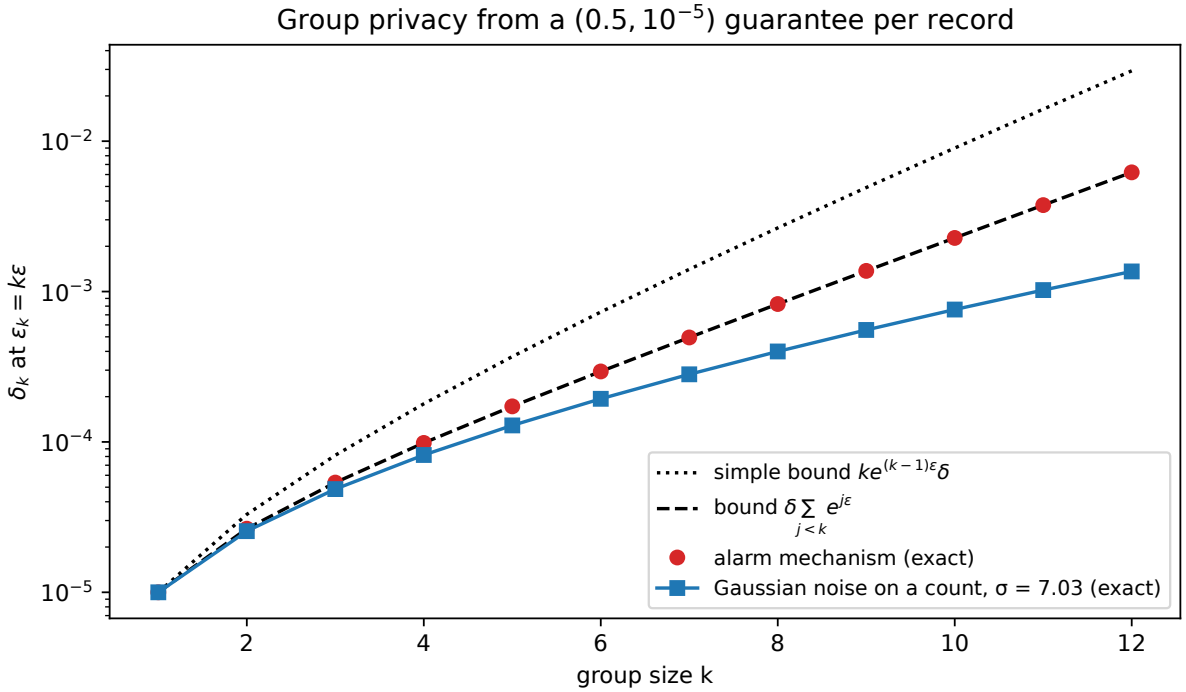


Figure 19.2.: The δ of group privacy at $\varepsilon_k = k\varepsilon$, for a per-record guarantee of $(0.5, 10^{-5})$, on a log scale. The geometric-sum bound (dashed) is attained by the alarm mechanism (red dots); the simple form (dotted) is looser by a factor that grows with k; Gaussian noise on a count (blue squares) does better than the bound but still grows quickly with k.

Lemma 19.2 (Privacy losses of independent outputs add)**.** *Let $Y = (Y_1, Y_2)$, where Y_1 and Y_2 are produced with independent coins, so that $p(y_1, y_2) = p_1(y_1)p_2(y_2)$ on D and likewise $q(y_1, y_2) = q_1(y_1)q_2(y_2)$ on D'. Then the privacy loss of the pair is the sum of the privacy losses of its parts,*

$$L(y_1, y_2) = \ln \frac{p_1(y_1)p_2(y_2)}{q_1(y_1)q_2(y_2)} = L_1(y_1) + L_2(y_2). \tag{19.10}$$

The same holds when the second output depends on the first, with the conditional probabilities $p_2(y_2 \mid y_1)$ and $q_2(y_2 \mid y_1)$ in place of $p_2(y_2)$ and $q_2(y_2)$.

Proof.

1. *Independent outputs.* The probability of seeing both outputs is the product of their probabilities, and the logarithm of a product of two ratios is the sum of the two logarithms.
2. *Dependent outputs.* The chain rule of probability writes any joint probability as "first, times second given first", $p(y_1, y_2) = p_1(y_1)\, p_2(y_2 \mid y_1)$, and the same on D'. The same logarithm then gives the same sum, with the conditional loss $\ln \frac{p_2(y_2|y_1)}{q_2(y_2|y_1)}$ as its second term.

□

In the theorem below, each mechanism M_i may also read the previous outputs, the way an analyst picks the next query after reading the last answer. This is **adaptive** composition, and **non-adaptive** composition is the case where M_i ignores them. The condition is that each step is private however the earlier outputs turned out. Section B.1 in the appendices derives the theorem from scratch, with worked numbers.

Theorem 19.3 (Basic composition)**.** *Let $M_1, \dots, M_k$ run on the same dataset with independent coins, where M_i may read the previous outputs $y_1, \dots, y_{i-1}$. If, for every fixed value of the previous outputs, $D \mapsto M_i(D; y_1, \dots, y_{i-1})$ is $(\varepsilon_i, \delta_i)$-DP, then the mechanism that releases $(y_1, \dots, y_k)$ is*

$$\Big(\sum_{i=1}^{k} \varepsilon_i,\ \sum_{i=1}^{k} \delta_i\Big)\text{-DP}. \tag{19.11}$$

For pure DP the bound $\sum_i \varepsilon_i$ is the exact worst-case privacy loss whenever each M_i attains its own worst case at outputs that can occur together.

Proof. The plan is to prove the pure case by adding privacy losses with Lemma 19.2. The approximate case takes two mechanisms, since induction on k does the rest. That case splits an event over the first output, applies the second mechanism's guarantee for each value of it, and applies the first mechanism's guarantee to what is left, a soft test of the first output.

1. *Pure DP: factor the probability of a transcript.* By the chain rule, the probability of $(y_1, \dots, y_k)$ is the probability of y_1, times that of y_2 given y_1, and so on. On D,

$$p_D(y_1, \dots, y_k) = \prod_{i=1}^{k} p_{M_i(D;\, y_{<i})}(y_i), \tag{19.12}$$

where $y_{<i} = (y_1, \dots, y_{i-1})$ and each factor uses fresh coins. The same holds on D', with the same previous outputs plugged in.

2. *Take the log-ratio.* By Lemma 19.2, applied $k-1$ times in its dependent form, $L_{D,D'}(y_1,\dots,y_k)=\sum_{i=1}^{k} L^{(i)}(y_i;\, y_{<i})$, where $L^{(i)}$ is the privacy loss of M_i with its previous outputs fixed.
3. *Bound each term and add.* For the fixed $y_{<i}$, M_i is ε_i-DP by assumption, so $|L^{(i)}|\le\varepsilon_i$, and the sum is at most $\sum_i \varepsilon_i$ at every transcript, which is pure $\sum_i \varepsilon_i$-DP. When every M_i attains ε_i at outputs that can occur together, the sum is attained, which is the exactness claim.
4. *Approximate DP: split an event over the first output.* Take $k=2$, and write P and Q for the distribution of the transcript (Y_1,Y_2) on D and on D'. For a set S of transcripts, let $S_{y_1}=\{y_2 : (y_1,y_2)\in S\}$, the second outputs that complete y_1 into S. The transcript lands in S when the first output is some y_1 and the second lands in S_{y_1}, so

$$P(S)=\sum_{y_1} p_1(y_1)\, P_2^{y_1}(S_{y_1}), \tag{19.13}$$

   where p_1 is the mass function of $M_1(D)$ and $P_2^{y_1}$ the distribution of $M_2(D;y_1)$. Write q_1 and $Q_2^{y_1}$ for the same objects on D'.
5. *Apply the second mechanism's guarantee inside the sum.* For each y_1, $P_2^{y_1}(S_{y_1})\le e^{\varepsilon_2}Q_2^{y_1}(S_{y_1})+\delta_2$, and also $P_2^{y_1}(S_{y_1})\le 1$ because it is a probability. So $P_2^{y_1}(S_{y_1})\le g(y_1)+\delta_2$ with

$$g(y_1)=\min\left(1,\ e^{\varepsilon_2}Q_2^{y_1}(S_{y_1})\right)\in[0,1]. \tag{19.14}$$

6. *Sum over y_1.* The weights $p_1(y_1)$ add up to 1, so δ_2 comes out once: $P(S)\le \mathbb{E}_{p_1}[g]+\delta_2$.
7. *Apply the first mechanism's guarantee to g.* The function g takes values in $[0,1]$, which is why it was capped at 1, so Lemma 19.1 gives $\mathbb{E}_{p_1}[g]\le e^{\varepsilon_1}\mathbb{E}_{q_1}[g]+\delta_1$.
8. *Undo the cap.* Since $g(y_1)\le e^{\varepsilon_2}Q_2^{y_1}(S_{y_1})$, the average $\mathbb{E}_{q_1}[g]$ is at most $e^{\varepsilon_2}\sum_{y_1} q_1(y_1)Q_2^{y_1}(S_{y_1})$, which by step 4 on D' is $e^{\varepsilon_2}Q(S)$.
9. *Combine.* Steps 6 to 8 give $P(S)\le e^{\varepsilon_1+\varepsilon_2}Q(S)+\delta_1+\delta_2$. Nothing used that M_2 ignores y_1, so this covers adaptive composition.

□

Basic composition is exact for the worst case, and the worst case is rare. For k randomized-response answers about the same bit, "every answer is true" has loss $k\ln 3$ but probability $(\frac{3}{4})^k$. Typical transcripts mix truths with lies, whose losses cancel. Under D, each answer contributes $+\varepsilon_0$ when it tells the truth, with probability $t=e^{\varepsilon_0}/(1+e^{\varepsilon_0})$, and $-\varepsilon_0$ otherwise. The mean of each answer's loss is therefore $\varepsilon_0(2t-1)$, which is $\varepsilon_0/2$ for $\varepsilon_0=\ln 3$. A sum of k such independent terms stays within a spread of order $\varepsilon_0\sqrt{k}$ of its mean (Concentration). An (ε,δ) guarantee may ignore losses above ε that occur with probability at most δ, so a small δ writes off the unlikely tail, and the ε you need grows like $\sqrt{k}$ rather than k, which is the advanced composition of Advanced and optimal composition.

The δs add without the inflation of group privacy, because each mechanism compares the *same* pair D, D', and step 6 of the proof just averages the second slack. Adaptivity is free, but only with budgets fixed in advance. Choosing the budgets from the outputs ("stop when the answer is good enough") needs the privacy filters of Advanced and optimal composition.

Example 19.3. Start with an adaptive survey. Ask a person for a randomized-response answer with $\varepsilon_1=\ln 3$, then ask again with $\ln 3$ if the answer was 1 and with 0.5 if it was 0. Level 0.5

tells the truth with probability $e^{0.5}/(1+e^{0.5}) = 0.6225$. So the second step depends on y_1, but it is at most $\ln 3$-DP for each value of y_1. In Table 19.5 each probability is the product of the two steps. For example, $(0,1)$ under bit 1 is a lie and then the truth, $\frac{1}{4} \times 0.6225 = 0.1556$.

Table 19.5.: An adaptive composition of two randomized-response answers: the second one uses $\varepsilon = \ln 3$ after a 1 and $\varepsilon = 0.5$ after a 0. Each transcript's privacy loss is the sum of the two step losses.

(y_1, y_2)	Under bit 1 (D)	Under bit 0 (D')	$L = L^{(1)} + L^{(2)}$
$(1,1)$	0.5625	0.0625	$\ln 3 + \ln 3 = 2.1972$
$(1,0)$	0.1875	0.1875	$\ln 3 - \ln 3 = 0$
$(0,1)$	0.1556	0.2832	$-\ln 3 + 0.5 = -0.5986$
$(0,0)$	0.0944	0.4668	$-\ln 3 - 0.5 = -1.5986$

The answers $(1,1)$ are two pieces of evidence pointing the same way, and $(1,0)$ are two that cancel. The worst loss is $2.1972 = \ln 3 + \ln 3$, exactly the bound with $\varepsilon_2 = \max(\ln 3, 0.5)$, the largest budget the second step can have.

Now ask the same question k times with fresh randomized response at $\varepsilon_0 = \ln 3$. If j answers equal 1, the privacy loss is $(2j-k)\ln 3$, and j is binomial with k trials and success probability $\frac{3}{4}$ when the bit is 1, or $\frac{1}{4}$ when it is 0. Figure 19.3 shows the loss for $k = 10$. Table 19.6 gives the script's exact numbers, with "exact ε at $\delta = 10^{-6}$" the smallest ε whose $\delta(\varepsilon)$ is at most 10^{-6}.

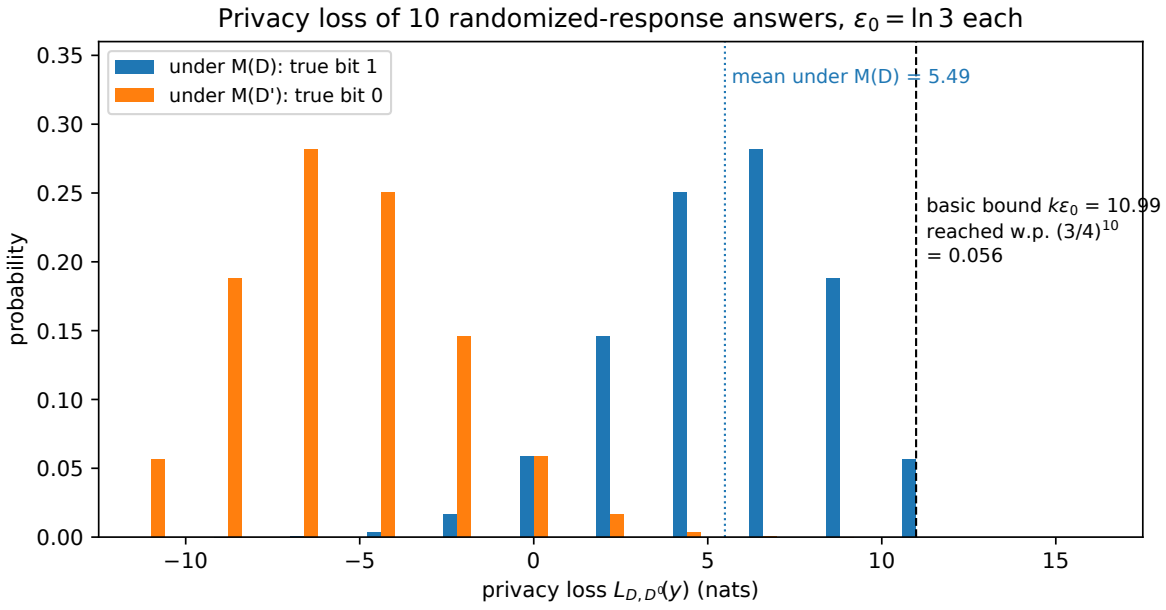


Figure 19.3.: Distribution of the privacy loss $L_{D,D'}$ of 10 randomized-response answers with $\varepsilon_0 = \ln 3$ each, when the true bit is 1 (blue) or 0 (orange). The loss takes the values $(2j-10)\ln 3$, where j is the number of answers equal to 1. The basic bound $10\ln 3$ is reached, but only with probability $(3/4)^{10} = 0.056$; the mean under D is half of it.

Table 19.6.: k randomized-response answers about the same bit, $\varepsilon_0 = \ln 3$ each: the basic bound, the exact worst case, how likely the worst case is, the mean privacy loss, and the exact ε when $\delta = 10^{-6}$ is allowed.

k	Basic bound $k\varepsilon_0$	Exact worst-case loss	$\Pr_D[L = k\varepsilon_0] = (\frac{3}{4})^k$	Mean loss under D	Exact ε at $\delta = 10^{-6}$
1	1.099	1.099	0.75	0.549	1.099
10	10.986	10.986	5.631×10^{-2}	5.493	10.986
20	21.972	21.972	3.171×10^{-3}	10.986	21.972
50	54.931	54.931	5.663×10^{-7}	27.465	52.680
100	109.861	109.861	3.207×10^{-13}	54.931	94.290

A slack δ helps only if transcripts with large loss have total probability below δ. Lowering ε by a small ξ below $k\varepsilon_0$ makes the worst transcript alone contribute about $(\frac{3}{4})^k\xi$ to the hockey-stick divergence, because its term $p - e^{\varepsilon}q$ is then $p(1 - e^{-\xi}) \approx p\,\xi$, with $p = (\frac{3}{4})^k$. So at $k = 20$ the allowed ξ is only about $10^{-6}/(3.171 \times 10^{-3}) \approx 3 \times 10^{-4}$, and the exact ε equals the basic bound to three decimals. At $k = 50$ the worst case drops below 10^{-6}, and at $k = 100$ the slack buys $109.861 - 94.290 = 15.6$. With a smaller per-answer budget the gain is much larger: at $\varepsilon_0 = 0.1$ and $k = 1000$ the basic bound is 100, and the exact ε at $\delta = 10^{-6}$ is 19.345 (Figure 19.4).

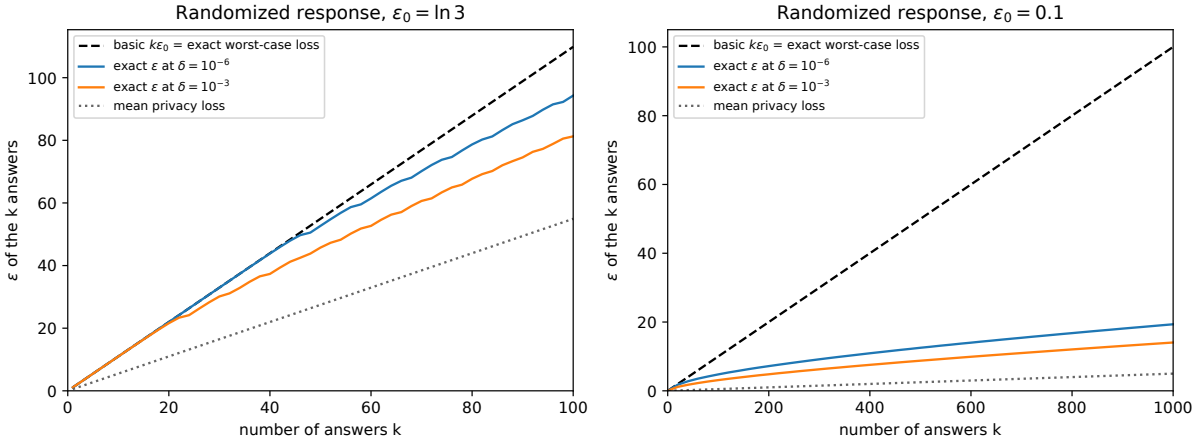


Figure 19.4.: The ε of k randomized-response answers: the basic bound $k\varepsilon_0$, which is also the exact worst-case loss (dashed), the exact ε at $\delta = 10^{-6}$ and 10^{-3}, and the mean privacy loss (dotted). Left: $\varepsilon_0 = \ln 3$; the exact curves leave the basic line only once $(3/4)^k$ drops below δ. Right: $\varepsilon_0 = 0.1$; the exact curves grow like $\sqrt{k}$.

The same gap appears for approximate-DP mechanisms. Take k counts with Gaussian noise, each calibrated to $(0.5, 10^{-5})$ with $\sigma = 7.0318$. One record shifts each noisy count by 1, and k independent Gaussians shifted by 1 carry the same evidence as one shifted by $\sqrt{k}$ (Section B.3 in the appendices derives this). By Table 19.7, basic composition overstates ε for ten queries by a factor of $5/1.504 = 3.3$.

Table 19.7.: k counts with Gaussian noise, each $(0.5, 10^{-5})$-DP: basic composition against the exact ε at the same total δ.

k	Basic: $(k\varepsilon, k\delta)$	Exact ε at the same $\delta = k \times 10^{-5}$
1	$(0.5, 10^{-5})$	0.500
2	$(1, 2 \times 10^{-5})$	0.694
4	$(2, 4 \times 10^{-5})$	0.966
10	$(5, 10^{-4})$	1.504

19.4. Parallel composition

A histogram counts people in disjoint bins. Sequential composition would charge $d\varepsilon$ for d bins, which is far too pessimistic, because one person's record lands in one bin only. Parallel composition makes that precise, with care about the two common neighbouring relations. Under **add/remove**, neighbours differ by adding or removing one record. Under **replace**, neighbours have the same size and differ in the content of one record, which is a removal followed by an addition.

Split the data universe into disjoint parts $U_1, \dots, U_d$ by a rule that may depend on a record's content but not on the rest of the dataset, and run M_i on $D \cap U_i$, the records in part i. A changed record touches only its own part, so you pay only that part's budget.

Theorem 19.4 (Parallel composition)**.** *Let $U_1, \dots, U_d$ be a partition of the data universe fixed in advance, and let M_i run on $D \cap U_i$ with independent coins, each $(\varepsilon_i, \delta_i)$-DP under add/remove. Then the release $(M_1(D \cap U_1), \dots, M_d(D \cap U_d))$ is*

$$(\max_i \varepsilon_i,\ \max_i \delta_i)\text{-DP under add/remove.} \tag{19.15}$$

Under replace, a record that moves from part a to part $b \neq a$ costs $(\varepsilon_a + \varepsilon_b, \delta_a + \delta_b)$, and one replaced within part a costs that part's guarantee for two add/remove steps, at most group privacy with $k = 2$; for pure DP with equal budgets both cases give 2ε. If the parts are fixed slots, so that record i always goes to part i, a replacement changes one part only and the cost is $(\max_i \varepsilon_i, \max_i \delta_i)$ again.

Proof.

1. *Add/remove changes one part.* If $D' = D \cup \{x\}$, then x lies in exactly one part U_j, because the parts are disjoint and cover the universe. For $i \neq j$, $D \cap U_i = D' \cap U_i$, so M_i gets the same input and has the same output distribution on both.
2. *Pure DP: the other losses are zero.* By Lemma 19.2 the privacy loss of the whole release is $\sum_i L^{(i)}$, and $L^{(i)} \equiv 0$ for $i \neq j$ because the two distributions are equal and their ratio is 1. So $|L| = |L^{(j)}| \leq \varepsilon_j \leq \max_i \varepsilon_i$.
3. *Approximate DP: condition on the unchanged parts.* Let W be the outputs of the parts $i \neq j$, which have the same distribution on D and D' and are independent of Y_j. Given W, the release lands in an event S when Y_j falls in S_W, the set of y_j completing W into S. So the probability of S is the average over W of $\Pr[Y_j \in S_W]$. Inside that average, M_j's

guarantee bounds the probability on D by e^{ε_j} times that on D', plus δ_j, and averaging over the same W carries the bound to S.

4. *Replace changes up to two parts.* If D' replaces a record $x \in U_a$ by $x' \in U_b$, then $D \cap U_a$ loses a record and $D \cap U_b$ gains one: two add/remove steps on two independent mechanisms, which is a composition of two mechanisms (Theorem 19.3, with the other parts handled as in step 3), giving $(\varepsilon_a + \varepsilon_b, \delta_a + \delta_b)$. If $a = b$, part a sees a removal and an addition, two neighbouring steps, which is Theorem 19.2 with $k = 2$ inside one part. In fixed slots a replacement changes only slot i, and steps 1 to 3 apply again.

□

So a histogram costs one ε, not d. Laplace noise of scale $1/\varepsilon$ on each of d bins is ε-DP under add/remove, because adding a record changes one bin by 1. Under replace it is 2ε-DP, since moving a record changes two bins, which is why a histogram's sensitivity is quoted as 2 under replace (The Laplace and Gaussian mechanisms).

The partition must not depend on the data. "Split the data at its median" changes when one record changes, which can move many records between parts, and then step 1 of the proof fails. Use fixed bins, or account for the data-dependent split as a mechanism of its own.

Example 19.4. Local randomized response composes in parallel. Each of n people reports their own bit at $\varepsilon_0 = \ln 3$, and each person is one slot, so the whole vector of n reports is $\ln 3$-DP for any n, not $n \ln 3$, under either neighbouring relation.

A two-bin histogram shows the factor of 2. Give each bin rounded $\mathrm{Lap}(0, 1)$ noise, and let the script compute the exact pure ε of the pair under each relation. Adding or removing a record changes one bin by 1, so only that bin contributes privacy loss, and $\varepsilon = 1.0000$. Replacing a record in bin 1 by one in bin 2 moves bin 1 up by 1 and bin 2 down by 1. The worst output then has both noisy counts far out on the side that favours D, where both step losses equal 1 at once, so $\varepsilon = 2.0000$.

19.5. What composes and what doesn't

The three properties hold under weak assumptions, which makes it tempting to apply them where even those fail. Each assumption appeared in a proof. Post-processing needs f not to read the data, composition needs independent coins and one neighbouring relation, and group privacy and composition need budgets fixed in advance. Table 19.8 collects the ways these break in practice, and the section then proves that the commonest break, two releases sharing one noise draw, is a total failure.

Table 19.8.: What composes and what doesn't.

Situation	Composes?	Why
Function of the output, own coins	Yes, at no cost	Post-processing (Theorem 19.1)

Situation	Composes?	Why
Another query on the same data, fresh noise, even chosen from the first answer	Yes, budgets add	Basic composition (Theorem 19.3)
Two queries sharing one noise draw	**No**	The noise cancels in a difference (Proposition 19.2)
"Post-processing" that reads the data again	**No**	It is a new mechanism, often a non-private one
Budgets chosen from earlier outputs	Not by Equation 19.11	Needs privacy filters (Advanced and optimal composition)
Proved under add/remove, released under replace	Yes, at $(2\varepsilon, (1+e^{\varepsilon})\delta)$	Group privacy with $k=2$, as in Equation 18.6 of the definition chapter
A DP release plus an exact statistic of the same data	**No**	The exact statistic is not DP

When two releases share one noise draw, the noise can be subtracted away, and privacy fails completely.

Proposition 19.2 (Shared noise is not private)**.** *Release two counts with the same Laplace draw Z of scale b: $c_1(D)+Z$ and $c_2(D)+Z$, where removing some record lowers c_1 by 1 and leaves c_2 unchanged. Each count alone is $1/b$-DP, but the pair is not (ε,δ)-DP for any ε and any $\delta<1$.*

Proof.

1. *The difference is noise-free.* $(c_1+Z)-(c_2+Z)=c_1(D)-c_2(D)$, a deterministic function of the data.
2. *It changes between neighbours.* If the difference is c on D, it is $c-1$ on the neighbour D' without that record, since removing the record lowers c_1 by 1 and leaves c_2.
3. *One event separates the two.* The event "the difference equals c" has probability 1 on D and 0 on D', so the event form $1 \le e^{\varepsilon}\cdot 0+\delta$ fails for every ε unless $\delta \ge 1$.

□

What broke is step 1 of the proof of Theorem 19.3: the transcript's probability is no longer a product of the two mechanisms' probabilities. So fresh randomness is part of the mechanism. Seeding two mechanisms with the same seed, re-adding a cached noise draw to a related query, or reusing noise across shards are all the shared-noise bug. An exact total published elsewhere can be differenced against a DP release in the same way.

The neighbouring relation is part of the budget too. A guarantee proved under add/remove and reported under replace can be off by a factor of 2, and mixing the two inside one accountant, the code that totals the budget, is a bug that comes back in Subsampling.

Example 19.5. Randomized response survives shared coins, because a second answer that reuses the first answer's coins repeats it, which is post-processing. Additive noise does not. The script simulates the **differencing attack** of Proposition 19.2 on two Laplace counts, the number of smokers and the number of smokers aged 40 or over. Alice is a smoker aged 30, so she counts only in the first. She is in the data with probability $\frac{1}{2}$, and the attacker guesses "in" when the difference of the two counts exceeds the midpoint of its two possible true values. Over 200,000 runs with $\mathrm{Lap}(0, 1)$ noise, the attack with shared noise has accuracy 1.0000, because the difference is exact.

With fresh noise the same attack has accuracy 0.6206. The second count no longer depends on Alice, so the best any attack can reach is that of the first count alone, $\frac{1}{2}$ plus half the total variation distance between what it sees in the two cases (Divergences). For $\mathrm{Lap}(0, 1)$ against $\mathrm{Lap}(1, 1)$ the best event is "the output is above $\frac{1}{2}$", whose probabilities differ by $1 - e^{-1/2}$, so the best accuracy is $\frac{1}{2} + \frac{1}{2}(1 - e^{-1/2}) = 0.6967$. The difference attack falls short of it because it adds the noise of the second count.

19.6. Code

The script code/properties.py computes every number in this chapter and draws every figure, from each mechanism's exact output distributions on two neighbouring datasets. From the repository root:

```
uv run python code/part03-dp-foundations/properties.py
```

Table 19.9.: The script's output with the default seed.

Experiment	Result
Randomized response $\ln 3$, then flip with probability r	$r = 0.1$: $\varepsilon = 0.8473$; $r = 0.25$: $0.5108 = \ln \frac{5}{3}$; $r = 0.5$: 0
Count + $\mathrm{Lap}(0, 1)$, post-processed	Raw: $\varepsilon = 1$, $\delta(0.5) = 0.2212$; rounded: 1.0000, 0.1967; threshold: 0.8318, 0.1967
Group privacy, pure	Rounded Laplace count, counts k apart: $\varepsilon = k$ for $k = 1, 2, 3, 5$
Group privacy, $(0.5, 10^{-5})$ per record	Table 19.4; the alarm mechanism attains the bound at every k
k randomized-response answers	Table 19.6; $\varepsilon_0 = 0.1$, $k = 1000$: basic 100, exact 19.345 at $\delta = 10^{-6}$
Simulation, 10 answers, 200,000 runs	Mean loss 5.500 (exact 5.493); worst case in 0.0559 of runs (exact 0.0563)
Adaptive two-step survey	Table 19.5; worst loss $2.1972 = 2 \ln 3$
k Gaussian counts, $(0.5, 10^{-5})$ each	Table 19.7
Two-bin histogram, rounded $\mathrm{Lap}(0, 1)$	Add/remove: $\varepsilon = 1.0000$; replace across bins: 2.0000
Shared against fresh noise, 200,000 runs	Attack accuracy 1.0000 against 0.6206; best possible with fresh noise 0.6967

tests/test_properties.py checks that the output still matches.

19.7. Summary

Table 19.10.: The results of this chapter, and where each is used.

Result	Statement	Use
Lemma 19.1	$\mathbb{E}_P[g] \le e^{\varepsilon}\mathbb{E}_Q[g] + \delta$ for every g with values in $[0,1]$	Approximate-DP proofs where a later stage depends on an earlier output
Theorem 19.1	$f \circ M$ is (ε,δ)-DP for any f that does not read the data	Privatize once, early; everything after is free, every attack of Part 2 included
Theorem 19.2	$k\varepsilon$ for datasets k records apart, with $\delta_k = \delta\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1} \le ke^{(k-1)\varepsilon}\delta$	Per-user guarantees; cheap only for small groups, so prefer user-level neighbours
Proposition 19.1	The alarm mechanism meets Equation 19.6 with equality	No better generic bound exists, though Gaussian noise does better
Lemma 19.2	$L(y_1,y_2) = L_1(y_1) + L_2(y_2)$ for independent coins, also given y_1	The fact behind both compositions; it needs fresh noise
Theorem 19.3	$(\sum \varepsilon_i, \sum \delta_i)$, also adaptive with budgets fixed in advance	Budgeting many queries; the worst case is exponentially rare, so for large k use the accountants of Advanced and optimal composition
Theorem 19.4	$(\max \varepsilon_i, \max \delta_i)$ under add/remove for a fixed partition; twice that under replace	Histograms, per-group statistics, local randomized response
Proposition 19.2	Two counts with one noise draw are not DP for any $\delta < 1$	Every composed mechanism needs its own fresh noise

19.8. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014): Proposition 2.1 (post-processing), Theorem 2.2 (group privacy, pure), Theorem 3.16 (basic composition, with the proof for approximate DP in Appendix B), and §3.5 for composition in general. The standard reference for every statement here.

- Dwork, McSherry, Nissim & Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (TCC 2006): the paper that defined DP, where the guarantee for groups of records first appears.
- McSherry, "Privacy Integrated Queries" (SIGMOD 2009): parallel composition, and a system built on post-processing and composition as its only rules.
- Vadhan, "The Complexity of Differential Privacy" (2017), the opening sections: the basic properties of DP (post-processing, group privacy, composition) with careful proofs, at a more theoretical level than Dwork & Roth.
- Near & Abuah, *Programming Differential Privacy*, chapters on the properties of DP: sequential and parallel composition and post-processing, with code.
- Kifer & Machanavajjhala, "No Free Lunch in Data Privacy" (SIGMOD 2011): why correlated records defeat a per-record guarantee, the problem group privacy addresses.
- Dwork, Rothblum & Vadhan, "Boosting and Differential Privacy" (FOCS 2010), and Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015): advanced and optimal composition, the subject of Advanced and optimal composition.
- Rogers, Roth, Ullman & Vadhan, "Privacy Odometers and Filters: Pay-as-you-Go Composition" (NeurIPS 2016): what changes when the budgets themselves are chosen adaptively.

20. The Laplace and Gaussian mechanisms

You run a hospital database and want to publish how many patients have diabetes, with a differential privacy (DP) guarantee. The definition of differential privacy says what the release must satisfy: changing one person's record must barely change the distribution of what you publish. It does not say how to build such a release. The two mechanisms of this chapter are the answer used almost everywhere. Both compute the exact answer, call it $f(D)$ for the dataset D, and blur it with random noise Z drawn fresh each time, so that what you publish is

$$M(D) = f(D) + Z. \tag{20.1}$$

If the noise is wide compared with how far one person can move the answer, an observer cannot tell from the output whether that person was there.

Three questions decide everything. The first is how much one record can move the answer, the **sensitivity** of f, which sets the scale of the noise. The second is which noise distribution to use. Laplace noise gives pure ε-DP, while Gaussian noise gives only (ε, δ)-DP but scales better when many numbers are released at once. The third is how much noise exactly. Laplace noise needs a one-line formula, while for Gaussian noise the textbook formula is loose, and the exact answer comes from solving one equation numerically.

The running example is the hospital's count: 1,234 of its $n = 10{,}000$ patients have diabetes, and the count must be released under add/remove neighbours.

20.1. Sensitivity

To hide one record, the noise has to cover the largest change one record can make to the answer. That change is the sensitivity, and it is the first thing to compute for any release. It depends on the query, on the norm you measure the change in, and on the neighbouring relation of the definition of differential privacy: under **add/remove** a neighbour has one record more or less, so the dataset size changes, and under **replace** one record is swapped for another, so the size n is fixed and public.

For a vector answer you need a way to measure a change, and two are used. The ℓ_1 norm $\|v\|_1 = \sum_i |v_i|$ is the total change summed over coordinates, and Laplace noise is calibrated to it. The ℓ_2 norm $\|v\|_2 = \sqrt{\sum_i v_i^2}$ is the straight-line length, and Gaussian noise is calibrated to it.

Definition 20.1 (Sensitivity)**.** Let $f\colon \mathcal{X}^n \to \mathbb{R}^d$ be a query with values in $\mathbb{R}^d$. For $p \in \{1, 2\}$, the ℓ_p **sensitivity** of f is the largest distance between its answers on two neighbouring datasets,

$$\Delta_p f = \max_{D \sim D'} \big\| f(D) - f(D') \big\|_p. \tag{20.2}$$

The two sensitivities are never far apart, and how far depends on how one record's change spreads over the coordinates. The next result says exactly how far, and Section 20.5 turns it into the rule for choosing between the two mechanisms.

Proposition 20.1 (The two sensitivities)**.** *For every query f with values in $\mathbb{R}^d$,*

$$\Delta_2 f \le \Delta_1 f \le \sqrt{d}\,\Delta_2 f. \tag{20.3}$$

The left bound is reached when one record moves a single coordinate, the right one when it moves all d coordinates equally.

Proof.

1. Fix a vector $v \in \mathbb{R}^d$. Squaring the sum $\sum_i |v_i|$ gives every v_i^2 plus cross terms that are never negative, so $\|v\|_2^2 \le \|v\|_1^2$, that is $\|v\|_2 \le \|v\|_1$.
2. The Cauchy–Schwarz inequality applied to the sum $\sum_i 1 \cdot |v_i|$ gives $\|v\|_1 \le \sqrt{d}\,\|v\|_2$.
3. Both inequalities hold for the change $v = f(D) - f(D')$ of every neighbouring pair, so they hold for the maximum over pairs. If only one coordinate of v is non-zero, both norms equal its absolute value; if all d coordinates equal c, then $\|v\|_1 = dc$ and $\|v\|_2 = \sqrt{d}\,c$.

□

The sensitivities of the standard queries follow by asking what the single worst record can do. Table 20.1 collects them; records take values in a domain $\mathcal{X}$, and B bounds a numerical record.

Table 20.1.: Sensitivities of the standard queries under the two neighbouring relations.

Query	Add/remove Δ_1	Add/remove Δ_2	Replace Δ_1	Replace Δ_2
Count $\lvert D\rvert$	1	1	0	0
Sum, records in $[0, B]$	B	B	B	B
Mean, records in $[0, B]$, n public	—	—	B/n	B/n
Histogram with d bins	1	1	2	$\sqrt{2}$
d threshold counts $\#\{x \ge j\}$, $j = 1..d$	d	$\sqrt{d}$	d	$\sqrt{d}$

Proposition 20.2 (Sensitivities of the standard queries)**.** *The sensitivities in Table 20.1 hold, and each is reached by some neighbouring pair.*

Proof.

1. *Count.* Adding or removing a record changes $|D|$ by exactly 1. Replacing a record leaves it unchanged, which is why under replace the size is not private and a count of all records needs no noise.

2. *Sum.* Adding a record $x \in [0, B]$ changes the sum by $x \leq B$, and replacing x by x' changes it by $|x - x'| \leq B$. Both bounds are reached with $x = B$ and $x' = 0$.
3. *Mean with public n.* Under replace the mean is the sum divided by the fixed n, so its sensitivity is the sum's divided by n, which is B/n. Under add/remove the denominator moves too, so there is no such simple bound; the remedy is to release a noisy sum and a noisy count instead.
4. *Histogram.* Under add/remove the record lands in one bin, so that bin changes by 1 and the others not at all. The change vector is $\pm e_j$, where e_j has a 1 in bin j and 0 elsewhere, and both of its norms are 1. Under replace one bin loses 1 and another gains 1, so the change is $e_{j'} - e_j$, with ℓ_1 norm $1 + 1 = 2$ and ℓ_2 norm $\sqrt{1^2 + 1^2} = \sqrt{2}$.
5. *Threshold counts.* The count for threshold j is the number of records with $x \geq j$, so a record x adds 1 to every count with $j \leq x$. A record with $x \geq d$ moves all d counts by 1, so the change vector is $(1, \ldots, 1)$, with ℓ_1 norm d and ℓ_2 norm $\sqrt{d}$. Under replace, changing x to x' moves only the counts with j between the two values, at most d of them, by 1 each, so the same bounds hold.

□

Sensitivity is a worst case over datasets. It does not depend on the data you hold, only on what the domain allows, which is why it can be computed before looking at the data. It is also why the sum of arbitrary real numbers has infinite sensitivity, since one record can be as large as it likes; clipping, in Beyond global sensitivity, forces a finite B. The neighbouring relation changes the answer by up to a factor of 2, because replacing is removing one record and adding another, so a guarantee stated without its neighbouring relation is ambiguous by that factor in ε.

Example 20.1. The hospital also releases the mean age of its $n = 10{,}000$ patients, with ages clipped to $[0, 100]$, under replace neighbours. By row 3 of Table 20.1 the sensitivity is $B/n = 100/10{,}000 = 0.01$ year, so one patient can move the mean by at most a hundredth of a year, however old they are. For the diabetes count under add/remove, row 1 gives a sensitivity of 1.

20.2. The Laplace mechanism

The Laplace mechanism is the first mechanism to try (Dwork, McSherry, Nissim & Smith 2006). It adds independent Laplace noise to each coordinate, with scale proportional to the ℓ_1 sensitivity. Section A.3 in the appendices derives the same result from scratch for a single count, with every number shown.

Everything rests on the **privacy loss** of the definition of differential privacy. For an output y, it is the log of how much more likely y is when the data are D than when they are D',

$$L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}, \tag{20.4}$$

where $p_{M(D)}$ is the density of the published output on dataset D. Write P and Q for the two output distributions $M(D)$ and $M(D')$. In its section on pure DP being pointwise, that chapter

proved that a mechanism whose loss is at most ε at every output, for every ordered neighbouring pair, is ε-DP. The Laplace mechanism is proved private exactly this way.

For additive noise the loss has a simple shape. On neighbouring datasets the mechanism outputs $f(D) + Z$ and $f(D') + Z$, the same noise placed around two different centres, so the density of Q is the density of P slid over by the shift $f(D') - f(D)$. How far apart two distributions are, at a factor $\gamma \geq 1$, is measured by the **hockey-stick divergence** of Divergences, $E_\gamma(P\|Q)$, the largest value of $\Pr_P[S] - \gamma \Pr_Q[S]$ over events S. At $\gamma = e^\varepsilon$ it is the smallest δ that works for the pair. The first result says that under this measure the true answer drops out, and only the shift matters. That tells you what the Laplace scale must cover, and Section 20.4 uses the same lemma to compute the Gaussian mechanism's exact δ.

Lemma 20.1 (Additive noise reduces to one shifted pair)**.** *Let $M(D) = f(D) + Z$, where Z has a density on $\mathbb{R}^d$, and let $v = f(D') - f(D)$ be the shift between the two answers on a neighbouring pair. Then for every $\gamma \geq 1$,*

$$E_\gamma(M(D)\|M(D')) = E_\gamma(Z\|Z + v). \tag{20.5}$$

Proof.

1. The map $y \mapsto y - f(D)$ sends $M(D)$ to Z and $M(D')$ to $Z + v$, and it is a bijection: every output corresponds to exactly one relabelled output, and back.
2. So an event S about M's output is the same as the event $S - f(D)$ about the relabelled output, with the same probability under each dataset. The hockey-stick divergence is a maximum of $\Pr_P[S] - \gamma \Pr_Q[S]$ over events, and both sides range over the same values, so they are equal.

□

For the hospital's count, the true value 1234 drops out and only the shift $v = 1$ remains. Privacy is a property of the noise and of the set of shifts that neighbouring datasets can produce, which Section 20.1 bounded. The Laplace mechanism chooses its scale so that the largest shift one record can cause, divided by the scale, is exactly ε.

Theorem 20.1 (The Laplace mechanism)**.** *The **Laplace distribution** $\mathrm{Lap}(b)$ with scale $b > 0$ has density $\frac{1}{2b}e^{-|z|/b}$, a two-sided exponential peaked at 0. Let $f\colon \mathcal{X}^n \to \mathbb{R}^d$ have ℓ_1 sensitivity $\Delta_1 f$ and let $\varepsilon > 0$. The **Laplace mechanism***

$$M(D) = f(D) + (Z_1, \ldots, Z_d), \qquad Z_i \overset{i.i.d.}{\sim} \mathrm{Lap}(b), \quad b = \frac{\Delta_1 f}{\varepsilon}, \tag{20.6}$$

is ε-DP (i.i.d. means independent and identically distributed). Its error per coordinate is about b, and for every failure probability $\beta \in (0, 1)$ the largest error over the d coordinates obeys

$$\mathbb{E}[|Z_i|] = b, \qquad \mathrm{Var}(Z_i) = 2b^2, \qquad \Pr\Big[\max_i |Z_i| > b \ln \frac{d}{\beta}\Big] \leq \beta. \tag{20.7}$$

The scale is the smallest that works in general: for a count, $\mathrm{Lap}(b)$ noise with $b < 1/\varepsilon$ is not ε-DP.

Proof. The plan is to write the privacy loss down and bound it at every output by the triangle inequality, then to read the error off the Laplace distribution, and finally to exhibit an event on which the privacy bound is reached.

1. *Write the two densities.* Fix $D \sim D'$ and write $a = f(D)$ and $a' = f(D')$ for the two true answers. The coordinates of the noise are independent, so the density of $M(D)$ at y is the product $\prod_i \frac{1}{2b} e^{-|y_i - a_i|/b}$, and that of $M(D')$ is the same with a' in place of a.
2. *Take the log-ratio.* The constants $\frac{1}{2b}$ cancel, and the log of a product of exponentials is the sum of the exponents, so
$$L_{D,D'}(y) = \frac{1}{b} \sum_{i=1}^{d} \left(|y_i - a'_i| - |y_i - a_i| \right). \tag{20.8}$$
3. *Bound each term.* The distance from y_i to a'_i is at most the distance from y_i to a_i plus the distance from a_i to a'_i, by the triangle inequality $|y_i - a'_i| \le |y_i - a_i| + |a_i - a'_i|$. So each term of the sum is at most $|a_i - a'_i|$, whatever y is.
4. *Add the terms.* Summing over i gives the ℓ_1 norm of the shift, which is at most the sensitivity, so for every y
$$L_{D,D'}(y) \le \frac{\|a - a'\|_1}{b} \le \frac{\Delta_1 f}{b} = \varepsilon. \tag{20.9}$$
The same bound for the swapped pair gives $L_{D,D'}(y) \ge -\varepsilon$, so $|L_{D,D'}(y)| \le \varepsilon$ at every output, and the mechanism is ε-DP by the pointwise criterion recalled above.
5. *Accuracy.* The absolute value $|Z_i|$ has density $\frac{1}{b} e^{-z/b}$ on $z \ge 0$, an exponential distribution with mean b and second moment $2b^2$; since Z_i has mean 0, that gives $\mathbb{E}[|Z_i|] = b$ and $\mathrm{Var}(Z_i) = 2b^2$. Integrating the same density from t to infinity gives the tail $\Pr[|Z_i| > t] = e^{-t/b}$.
6. *All coordinates at once.* Give each coordinate a share β/d of the failure probability: setting $e^{-t/b} = \beta/d$ gives $t = b \ln(d/\beta)$. The probability that at least one of the d coordinates fails is at most the sum of their failure probabilities, by the union bound, which is $d \cdot \beta/d = \beta$.
7. *Tightness.* The outputs where the loss is largest are those left of both answers, so take a count with $f(D) = 0$ and $f(D') = 1$ and the event $S = \{y \le 0\}$. Under $\mathrm{Lap}(b)$ noise, $\Pr[M(D) \in S] = \Pr[Z \le 0] = \frac{1}{2}$ by the symmetry of the noise about 0, and $\Pr[M(D') \in S] = \Pr[Z \le -1] = \frac{1}{2} e^{-1/b}$ from the tail of step 5. So the ratio of the two probabilities is exactly $e^{1/b}$, and on S the bound of step 4 holds with equality at every point. If $b < 1/\varepsilon$, the ratio exceeds e^{ε} and the definition fails on this event.

□

The noise does not depend on n. A count gets noise of scale $1/\varepsilon$ whether the database has a hundred records or a hundred million, because one record moves a count by 1 either way. The *relative* error of a count of size n is therefore about $1/(n\varepsilon)$, so large populations are cheap to protect and small subgroups are expensive. The worst case is not rare, either: the privacy loss equals ε on the event S of step 7, which has probability $\frac{1}{2}$ (Figure 20.1, left), so the guarantee is tight in the strongest sense and not just at some unlikely output.

Two more facts about the Laplace mechanism matter later. Drawn as a random number, the privacy loss of an output from $M(D)$ never leaves $[-\varepsilon, \varepsilon]$. It has an atom of probability $\frac{1}{2}$ at $+\varepsilon$, an atom of probability $\frac{1}{2} e^{-\varepsilon}$ at $-\varepsilon$, and a continuous ramp between them (Figure 20.2, left), and Part 4's accountants track exactly this distribution. The mechanism also satisfies a whole curve

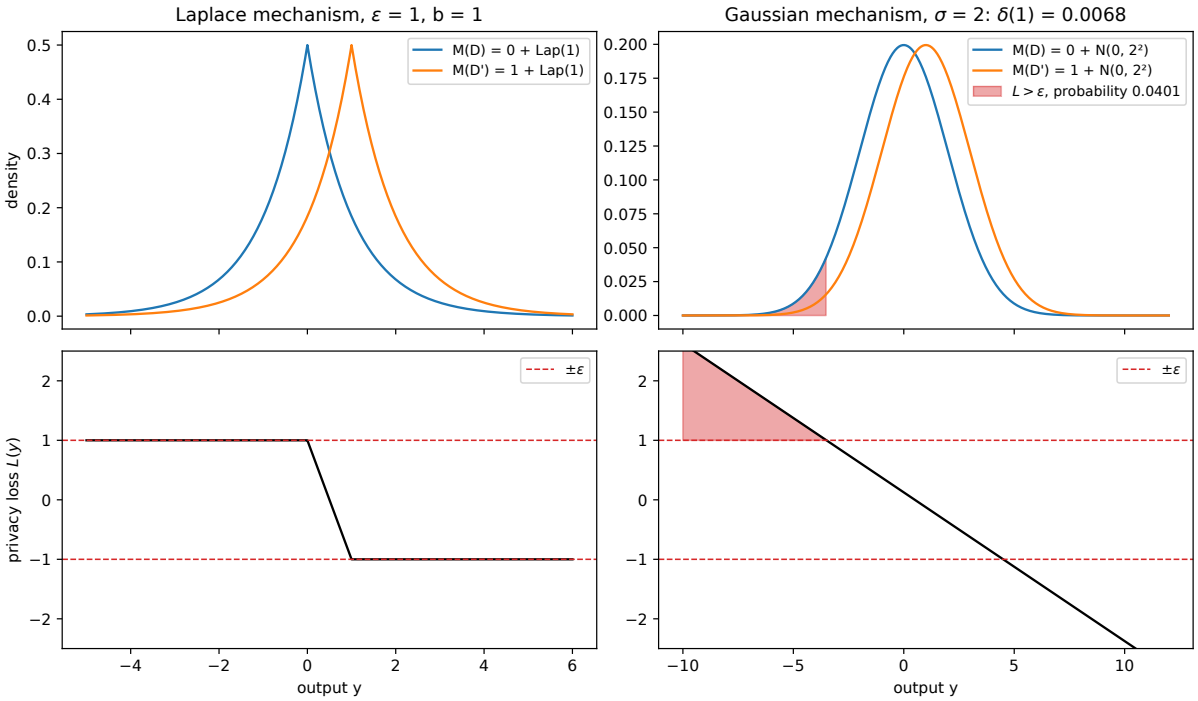


Figure 20.1.: Left column: a count released with Lap(1) noise on neighbouring datasets with true counts 0 and 1, and the privacy loss $L(y)$. It equals $+\varepsilon$ for every $y \leq 0$ and $-\varepsilon$ for every $y \geq 1$, never more. Right column: the same count with $\mathcal{N}(0, 2^2)$ noise. The privacy loss is a straight line, and exceeds $\varepsilon = 1$ on the shaded tail, which has probability 0.0401 under $M(D)$. The exact $\delta(1) = 0.0068$ is smaller, because the hockey-stick subtracts e^{ε} times the mass of $M(D')$ on the same tail.

of weaker guarantees. For every $\varepsilon' < \varepsilon$ it is (ε', δ)-DP with $\delta = 1 - e^{-(\varepsilon-\varepsilon')/2}$, the hockey-stick divergence of two shifted Laplace distributions from Divergences, drawn as the dashed curve of Figure 20.3.

As for the shape of the noise, the proof used only that the log-density changes by at most $\varepsilon/\Delta_1 f$ per unit of shift, and the Laplace log-density $-|z|/b$ has exactly that slope everywhere. Other shapes are slightly better for large ε (Geng & Viswanath's staircase mechanism), but the Laplace is within a small factor and is the standard choice.

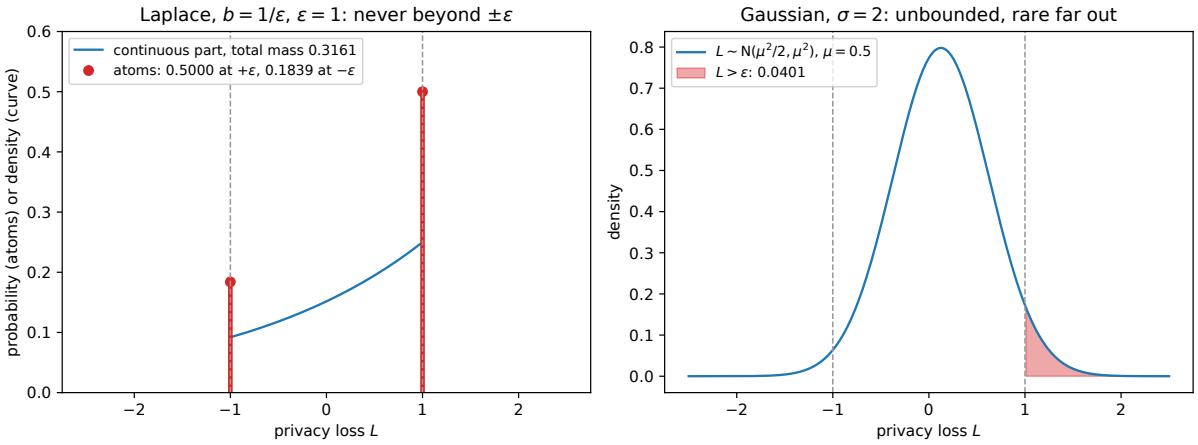


Figure 20.2.: The privacy loss distribution under $M(D)$ for a count, with $\varepsilon = 1$. Left: Laplace noise of scale $1/\varepsilon$. The loss never leaves $[-\varepsilon, \varepsilon]$; it has an atom of probability 0.5000 at $+\varepsilon$, an atom of 0.1839 at $-\varepsilon$, and a continuous ramp of total probability 0.3161 in between (density shown). Right: Gaussian noise of standard deviation 2. The loss is normal with mean $\mu^2/2$ and variance μ^2, here $\mu = 0.5$, and exceeds ε with probability 0.0401 (shaded).

Example 20.2. Release the diabetes count, 1,234 of $n = 10{,}000$ patients, with $\varepsilon = 0.1$ under add/remove. The sensitivity is 1, so the noise is Lap(10) and the expected absolute error is $b = 10$ (simulated over a million releases: 9.978). For a 95% guarantee take $d = 1$ and $\beta = 0.05$ in the accuracy bound. With probability 0.95 the error is then below $b \ln 20 = 29.96$ (simulated 95th percentile: 29.85), a relative error of $29.96/1234 = 0.0243$.

To see the privacy loss itself, take a count with $b = 1$, so $\varepsilon = 1$, and true answers 0 and 1. Table 20.2 evaluates the two densities and the loss at a few outputs. Left of both true answers the ratio of the densities is stuck at e, right of them at e^{-1}, and in between the loss falls linearly from 1 to -1. Once y is past both centres, moving it further away multiplies both densities by the same factor, so however far out an output lands, it never favours one dataset by more than a factor of e. A million simulated draws reproduce the tight event's probabilities to three decimals (Table 20.7).

Table 20.2.: The Laplace mechanism for a count with $b = 1$: densities and privacy loss at five outputs.

Output y	-2	0	0.5	1	3
$p_{M(D)}(y) = \frac{1}{2}e^{-\lvert y\rvert}$	0.0677	0.5	0.3033	0.1839	0.0249
$p_{M(D')}(y) = \frac{1}{2}e^{-\lvert y-1\rvert}$	0.0249	0.1839	0.3033	0.5	0.0677
$L(y) = \lvert y-1\rvert - \lvert y\rvert$	1	1	0	-1	-1

The hospital's histogram of diagnoses, with 20 bins, $\varepsilon = 1$ and add/remove, has $\Delta_1 = 1$, so each bin gets Lap(1) noise. With $d = 20$ and $\beta = 0.05$, every bin is within $\ln(20/0.05) = 5.99$ of its true count with probability 0.95. In 20,000 simulated releases the maximum error exceeded 5.99 in a fraction 0.0464 of them, just under β: independent coordinates rarely fail together, so the union bound is nearly exact.

20.3. The Gaussian mechanism: classic calibration

Gaussian noise is the other natural choice. It has lighter tails than Laplace noise, a sum of independent Gaussians is again Gaussian, and it is calibrated to the ℓ_2 sensitivity, which can be much smaller than the ℓ_1 sensitivity. The price is that its privacy loss is a straight line in the output and so is unbounded (Figure 20.1, right), so the pointwise criterion never applies and Gaussian noise cannot give pure DP. Instead you accept a small probability δ that the loss exceeds ε. This section derives the calibration found in the textbook of Dwork & Roth and in many papers and libraries, and shows where it is loose; Section A.6 in the appendices derives the same formula from scratch, with worked numbers.

The smallest δ that works for one pair is its hockey-stick divergence at $\gamma = e^{\varepsilon}$, and the largest of these over all pairs, as a function of ε, is the **privacy profile** $\delta_M(\varepsilon)$ of M. The first tool says which event is the worst one: the set of outputs where the loss is too large.

Lemma 20.2 (The bad event and the exact δ)**.** *Fix a neighbouring pair, write P and Q for the distributions of $M(D)$ and $M(D')$, with densities p and q, and let L be the privacy loss. Call $B = \{y : L(y) > \varepsilon\}$ the **bad event**, the outputs that favour D over D' by more than a factor e^{ε}. Then*

$$E_{e^{\varepsilon}}(P\|Q) = \Pr_{P}[B] - e^{\varepsilon}\Pr_{Q}[B] \le \Pr_{P}[B] = \Pr_{y\sim P}[L(y) > \varepsilon]. \tag{20.10}$$

Equivalently, writing $(x)_+ = \max(x, 0)$ for the positive part,

$$E_{e^{\varepsilon}}(P\|Q) = \mathbb{E}_Q[(e^L - e^{\varepsilon})_+] = \mathbb{E}_P[(1 - e^{\varepsilon - L})_+]. \tag{20.11}$$

Proof.

1. The quantity $\Pr_P[S] - e^\varepsilon \Pr_Q[S]$ is the integral over S of $p(y) - e^\varepsilon q(y)$. To make the integral as large as possible, include every output where the integrand is positive and no other. The integrand is positive exactly where $p > e^\varepsilon q$, that is where $L > \varepsilon$, which is B. This is the argument of Divergences for the worst event of a hockey-stick divergence, and it gives the equality.
2. The inequality drops the term $e^\varepsilon \Pr_Q[B]$, which is never negative.
3. For the expectation forms, the hockey-stick is the integral of $(p - e^\varepsilon q)_+$. Factoring out q gives $q\,(e^L - e^\varepsilon)_+$, and factoring out p gives $p\,(1 - e^{\varepsilon - L})_+$; integrating each against its density is the stated expectation.

□

So the bad event's probability is always a valid δ, though usually a loose one, and the last form needs only the distribution of the loss under P, the starting point of Numerical accounting. For Gaussian noise that distribution is itself normal.

Lemma 20.3 (The privacy loss of Gaussian noise)**.** *Let $M(D) = f(D) + Z$ with $Z \sim \mathcal{N}(0, \sigma^2 I_d)$, that is d independent normal coordinates with mean 0 and standard deviation σ, and let $v = f(D') - f(D)$ be the shift of a neighbouring pair. Write $\mu = \|v\|_2/\sigma$ for the shift measured in standard deviations of the noise. Under $M(D)$ the privacy loss is normal,*

$$L \sim \mathcal{N}\Big(\frac{\mu^2}{2}, \mu^2\Big), \tag{20.12}$$

and under $M(D')$ it is $\mathcal{N}(-\mu^2/2, \mu^2)$.

Proof.

1. Under $M(D)$, write the output as $y = f(D) + z$, where z is the noise. The density of $M(D)$ at y is proportional to $e^{-\|z\|^2/(2\sigma^2)}$, and that of $M(D')$ to $e^{-\|z-v\|^2/(2\sigma^2)}$, since y sits at distance $z - v$ from $f(D')$. The log of their ratio is the difference of the exponents, and expanding $\|z - v\|^2 = \|z\|^2 - 2\langle z, v\rangle + \|v\|^2$ gives
$$L(y) = \frac{\|v\|^2 - 2\langle z, v\rangle}{2\sigma^2}. \tag{20.13}$$
2. The loss depends on z only through the inner product $\langle z, v\rangle = \sum_i z_i v_i$, the component of the noise along the shift. That is a sum of independent normals, so it is normal, with mean 0 and variance $\sum_i \sigma^2 v_i^2 = \sigma^2 \|v\|^2$.
3. A constant minus a multiple of a normal is normal, so L is normal with mean $\|v\|^2/(2\sigma^2) = \mu^2/2$ and variance $4\sigma^2\|v\|^2/(4\sigma^4) = \mu^2$. The other $d - 1$ directions of z, perpendicular to v, do not enter.
4. Under $M(D')$ the noise is centred one shift further along v, which lowers $\langle z, v\rangle$ by $\|v\|^2$ on average and so lowers the loss by μ^2; the variance is unchanged.

□

Unlike the Laplace loss, the Gaussian loss has no atoms and no largest value (Figure 20.2, right). The classic calibration bounds δ by the probability that it exceeds ε and chooses σ to make that tail small.

Theorem 20.2 (Classic Gaussian calibration)**.** *Let f have ℓ_2 sensitivity $\Delta_2 f$, and let $\varepsilon \in (0,1)$ and $\delta \in (0,1)$. The* ***Gaussian mechanism*** *$M(D) = f(D) + Z$ with $Z \sim \mathcal{N}(0, \sigma^2 I_d)$ and*

$$\sigma = \frac{\Delta_2 f \sqrt{2\ln(1.25/\delta)}}{\varepsilon} \tag{20.14}$$

is (ε, δ)-DP (Dwork & Roth 2014, Theorem A.1).

Proof. The plan is to bound δ by the bad-event probability of Lemma 20.2, which by Lemma 20.3 is a normal tail, and to show with a standard tail bound that this tail is below δ for the σ above. The route is Dwork & Roth's, in a one-sided form that is a little shorter. Write $c = \sqrt{2\ln(1.25/\delta)}$, so $\sigma = c\,\Delta_2 f/\varepsilon$.

1. *Bound δ by the bad event.* By Lemma 20.2, $\delta \le \Pr_P[L > \varepsilon]$. By Lemma 20.3 the loss is normal with mean $\mu^2/2$ and standard deviation μ, so $L > \varepsilon$ exactly when a standard normal exceeds $(\varepsilon - \mu^2/2)/\mu = \frac{\varepsilon}{\mu} - \frac{\mu}{2}$. Writing Φ for the standard normal cumulative distribution function (CDF), and using its symmetry $\Phi(-t) = 1 - \Phi(t)$,
$$\Pr_P[L > \varepsilon] = \Phi(-t), \qquad t = \frac{\varepsilon}{\mu} - \frac{\mu}{2}. \tag{20.15}$$
2. *Take the worst shift.* A larger shift makes the bad event more likely, because t decreases as μ grows. So the worst neighbouring pair has the largest shift, $\|v\|_2 = \Delta_2 f$, that is $\mu = \Delta_2 f/\sigma = \varepsilon/c$, and then $t = c - \frac{\varepsilon}{2c}$.
3. *Bound the normal tail.* The Mills-ratio bound of Concentration inequalities says $\Phi(-t) \le \varphi(t)/t$ for $t > 0$, where $\varphi(t) = e^{-t^2/2}/\sqrt{2\pi}$ is the standard normal density. The reason is that beyond t every point z has $z/t \ge 1$, so $\int_t^\infty \varphi(z)\,\mathrm{d}z \le \int_t^\infty \frac{z}{t}\varphi(z)\,\mathrm{d}z$, and since the derivative of φ is $-z\varphi(z)$ the right side integrates to $\varphi(t)/t$.
4. *Expand t^2.* Squaring $t = c - \varepsilon/(2c)$ gives $t^2 = c^2 - \varepsilon + \frac{\varepsilon^2}{4c^2} \ge c^2 - \varepsilon$, dropping the last term because it is positive. By the definition of c, $e^{-c^2/2} = \delta/1.25$, so
$$e^{-t^2/2} \le \frac{\delta}{1.25}\, e^{\varepsilon/2}. \tag{20.16}$$
5. *Combine.* Steps 3 and 4 give $\Pr_P[L > \varepsilon] \le \delta\, e^{\varepsilon/2}/(1.25\sqrt{2\pi}\,t)$, which is at most δ when $t \ge e^{\varepsilon/2}/(1.25\sqrt{2\pi})$. For $\varepsilon \le 1$ the right side is at most $e^{1/2}/(1.25\sqrt{2\pi}) = 0.5262$.
6. *Check that t is large enough.* For $\varepsilon \le 1$, $t = c - \varepsilon/(2c) \ge c - 1/(2c)$, and this lower bound grows with c. It equals 0.5262 at the positive root of $c^2 - 0.5262c - \frac{1}{2} = 0$, which is where $\delta = 1.25e^{-c^2/2} = 0.7448$; since c grows as δ shrinks, $t \ge 0.5262$ for every $\delta \le 0.7448$, and every δ of practical interest qualifies.

□

The constant 1.25 absorbs $e^{\varepsilon/2} \le e^{1/2}$ and the $1/(t\sqrt{2\pi})$ factor of step 5, and the condition $\varepsilon < 1$ keeps $e^{\varepsilon/2}$ under control. The proof made three approximations in a row, the bad-event bound, the Mills ratio and the constants, and the example follows the chain to see what each one costs.

Example 20.3. Release the diabetes count with $\varepsilon = 1$ and $\delta = 10^{-5}$. The sensitivity is 1, so $c = \sqrt{2\ln(1.25 \times 10^5)}$ gives $\sigma = 4.8448$ and $t = c - 1/(2c) = 4.7416$. Table 20.3 follows the proof step by step.

Table 20.3.: The classic proof at $\varepsilon = 1$, $\delta = 10^{-5}$, step by step.

Quantity	Value	What was dropped to get the next line
Exact $\delta(\varepsilon)$, the hockey-stick	4.11×10^{-8}	the term $e^{\varepsilon} \Pr_Q[B]$ of Equation 20.10
$\Pr[L > \varepsilon] = \Phi(-t)$	1.06×10^{-6}	the Mills-ratio approximation
Mills bound $\varphi(t)/t$	1.10×10^{-6}	the constants $e^{\varepsilon/2}$, 1.25 and $t\sqrt{2\pi}$
Target δ	10^{-5}	

The classic σ delivers a δ about 240 times smaller than asked for, since $10^{-5}/(4.11 \times 10^{-8}) \approx 243$. Most of the slack is in two places. The bad-event bound throws away the $e^{\varepsilon} \Pr_Q[B]$ correction, a factor of $1.06 \times 10^{-6}/(4.11 \times 10^{-8}) \approx 26$ here, and the constants of steps 4 and 5 lose another factor of $10^{-5}/(1.10 \times 10^{-6}) \approx 9$. The Mills ratio itself costs almost nothing.

So you could use less noise for the same (ε, δ), and Section 20.4 finds exactly how much less. The classic theorem is stated for $\varepsilon < 1$, and its constants are designed for that range. The formula does not fail immediately above 1: at $\delta = 10^{-5}$ the classic σ still gives a valid guarantee up to $\varepsilon = 8.42$ and violates it beyond (Figure 20.4, right), because the classic σ shrinks like $1/\varepsilon$ while the exact one shrinks more slowly. Nothing in the classic proof tells you that, so for $\varepsilon \geq 1$ you need the exact curve.

20.4. The exact privacy profile of the Gaussian mechanism

Lemma 20.2 says that the smallest valid δ is the bad-event probability minus the correction $e^{\varepsilon} \Pr_Q[B]$, and Lemma 20.3 says that for Gaussian noise both terms are normal tail probabilities. So the exact δ has a closed form, the one Divergences computed for two Gaussians with the same variance. Balle & Wang (2018) turned that observation into a calibration: instead of bounding δ, compute it exactly, and pick the smallest σ for which it is below the target. Section A.7 in the appendices derives the formula from scratch, with worked numbers.

Theorem 20.3 (Exact privacy profile of the Gaussian mechanism)**.** *Let f have ℓ_2 sensitivity $\Delta_2 f$, reached by some neighbouring pair, and let $M(D) = f(D) + Z$ with $Z \sim \mathcal{N}(0, \sigma^2 I_d)$. Write $\mu = \Delta_2 f/\sigma$. Then M is (ε, δ)-DP if and only if $\delta \geq \delta_\mu(\varepsilon)$, where*

$$\delta_\mu(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\, \Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big). \tag{20.17}$$

The function $\delta_\mu(\varepsilon)$ is strictly decreasing in ε and strictly increasing in μ. At $\varepsilon = 0$ it is the total variation (TV) distance $2\Phi(\mu/2) - 1$ between the two output distributions, and it tends to 0 as $\varepsilon \to \infty$.

Proof. The plan is to reduce the worst pair to one dimension, apply the two lemmas there, and show that the largest shift is the worst one; the monotonicity claims then follow from the formula.

1. *Reduce to a shifted pair.* By Lemma 20.1, for a neighbouring pair with shift v the smallest δ at this ε is $E_{e^\varepsilon}(\mathcal{N}(0,\sigma^2 I_d)\|\mathcal{N}(v,\sigma^2 I_d))$.
2. *Reduce to one dimension.* Rotate coordinates so that v points along the first axis. A rotation is a bijection, so as in Lemma 20.1 it changes no divergence, and the normal distribution with independent equal-variance coordinates looks the same after any rotation (Section E.1 in the appendices proves it). Now both distributions are products: the first coordinate is $\mathcal{N}(0,\sigma^2)$ or $\mathcal{N}(\|v\|_2,\sigma^2)$, and the other $d-1$ coordinates have the same density h under both. The hockey-stick is the integral of $(p-\gamma q)_+$, and $(p_1 h-\gamma q_1 h)_+ = h\,(p_1-\gamma q_1)_+$ because $h\ge 0$, so integrating out the last $d-1$ coordinates leaves E_γ of the first coordinate alone. Coordinates that are identical under both hypotheses carry no evidence.
3. *Use the two lemmas.* By Lemma 20.2, $E_{e^\varepsilon} = \Pr_P[B] - e^\varepsilon \Pr_Q[B]$ with $B=\{L>\varepsilon\}$. By Lemma 20.3 with $\mu_v = \|v\|_2/\sigma$, the loss is $\mathcal{N}(\mu_v^2/2,\mu_v^2)$ under P and $\mathcal{N}(-\mu_v^2/2,\mu_v^2)$ under Q, so standardizing each tail gives

$$\Pr_P[B] = \Phi\Big(\frac{\mu_v}{2}-\frac{\varepsilon}{\mu_v}\Big), \qquad \Pr_Q[B] = \Phi\Big(-\frac{\mu_v}{2}-\frac{\varepsilon}{\mu_v}\Big). \tag{20.18}$$

   This is Equation 20.17 with μ_v in place of μ.
4. *The worst pair has the largest shift.* A smaller shift is harder to detect, and the proof shows that the pair with a smaller shift is a post-processing of the pair with a larger one, which cannot help anyone tell them apart. Let $\mu' < \mu_v$, take Y from the unit-variance pair with shift μ_v, and output $Y' = \frac{\mu'}{\mu_v}Y + W$ with independent $W\sim\mathcal{N}(0,1-\mu'^2/\mu_v^2)$. If $Y\sim\mathcal{N}(0,1)$ then Y' has mean 0 and variance $\mu'^2/\mu_v^2 + (1-\mu'^2/\mu_v^2) = 1$; if $Y\sim\mathcal{N}(\mu_v,1)$ the mean becomes μ' and the variance is still 1. By the data-processing inequality of Divergences, the hockey-stick of the post-processed pair is no larger, so $\delta_{\mu_v}(\varepsilon)$ is non-decreasing in μ_v and the maximum over neighbours is at $\|v\|_2 = \Delta_2 f$, that is at μ.
5. *If and only if.* The hockey-stick at $\gamma = e^\varepsilon$ is the smallest δ that works for one pair, and steps 1 to 4 show that the pair at shift $\Delta_2 f$ is the worst. So $\delta \ge \delta_\mu(\varepsilon)$ is sufficient, since it covers the worst pair and hence all, and necessary, since any smaller δ fails on that pair.
6. *Monotone in ε and in μ.* Differentiating Equation 20.17 gives $\frac{\mathrm{d}}{\mathrm{d}\varepsilon}\delta_\mu(\varepsilon) = -e^\varepsilon\Phi(-\frac{\mu}{2}-\frac{\varepsilon}{\mu}) < 0$ and $\frac{\mathrm{d}}{\mathrm{d}\mu}\delta_\mu(\varepsilon) = \varphi(\frac{\mu}{2}-\frac{\varepsilon}{\mu}) > 0$.
7. *The two ends.* At $\varepsilon = 0$ the factor e^ε is 1 and Equation 20.17 reads $\Phi(\frac{\mu}{2}) - \Phi(-\frac{\mu}{2}) = 2\Phi(\mu/2)-1$, which is the TV distance because the TV distance is the hockey-stick with $\gamma = 1$. As $\varepsilon\to\infty$ the first term $\Phi(\frac{\mu}{2}-\frac{\varepsilon}{\mu})$ goes to 0 and the second is non-negative, so $\delta_\mu(\varepsilon)\to 0$.

□

Because δ_μ grows with μ, less noise always means a larger δ, and there is a single noise level that hits the target exactly.

Corollary 20.1 (Analytic calibration)**.** *For $0<\delta<1$ the equation $\delta_\mu(\varepsilon)=\delta$ has exactly one root μ^* in μ, and*

$$\sigma^*(\varepsilon,\delta) = \frac{\Delta_2 f}{\mu^*} \tag{20.19}$$

is the smallest σ for which the Gaussian mechanism is (ε,δ)-DP.

Proof.

1. As a function of μ, $\delta_\mu(\varepsilon)$ is continuous, tends to 0 as $\mu \to 0$ (both arguments of Φ go to $-\infty$, since there is no signal) and to 1 as $\mu \to \infty$ (the first Φ goes to 1 and the second to 0), and is strictly increasing by Theorem 20.3. So it crosses every $\delta \in (0,1)$ exactly once.
2. By Theorem 20.3, a noise level σ works exactly when $\delta_{\Delta_2 f/\sigma}(\varepsilon) \le \delta$, that is when $\Delta_2 f/\sigma \le \mu^*$, so the smallest such σ is $\Delta_2 f/\mu^*$.

□

The root is found by bisection: start with an interval of μ whose ends put $\delta_\mu(\varepsilon)$ on either side of δ, evaluate at the midpoint, and keep the half where the sign changes. Monotonicity is what makes this safe, since there is exactly one crossing to find. The script halves in $\ln \mu$, because μ^* ranges over several orders of magnitude, and 60 halvings pin it to machine precision.

Example 20.4. Release the diabetes count at $\varepsilon = 0.1$, $\delta = 10^{-5}$ with Gaussian noise. The sensitivity is 1, so the analytic calibration gives $\sigma^* = 30.75$, the row $\varepsilon = 0.1$ of Table 20.5. A normal variable stays within 1.96 standard deviations of its mean with probability 0.95, so the 95% error is $1.96\sigma^* = 60.27$, against 29.96 for Laplace noise with $\delta = 0$: for a single count, Laplace noise is better on both error and guarantee, and Section 20.5 says when the Gaussian earns its place.

To see the two terms of Equation 20.10 at work, take $\mu = 1$ and $\varepsilon = 1$ and measure the noise z in units of σ. With $\mu = 1$ the loss is $L = \frac{1}{2} - z$, so the bad event $B = \{L > 1\}$ is $\{z < -\frac{1}{2}\}$. Under $M(D)$ the noise is standard normal, so $\Pr_P[B] = \Phi(-0.5)$; under $M(D')$ the output is centred one unit higher, so in the same coordinates z is $\mathcal{N}(1,1)$ and $\Pr_Q[B] = \Phi(-1.5)$. Table 20.4 evaluates both, and the integration and Monte Carlo checks of Table 20.7 agree with the result.

Table 20.4.: The exact δ at $\mu = 1$, $\varepsilon = 1$ as the bad-event probability minus its correction.

Term	Formula	Value
$\Pr_P[B]$, the bad-event bound	$\Phi(\frac{\mu}{2} - \frac{\varepsilon}{\mu}) = \Phi(-0.5)$	0.3085
$e^\varepsilon \Pr_Q[B]$, the correction	$e\,\Phi(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}) = e\,\Phi(-1.5)$	0.1816
$\delta_1(1)$, the exact δ	difference	0.1269

One Gaussian mechanism gives a whole curve of guarantees, not one: it is $(\varepsilon, \delta_\mu(\varepsilon))$-DP for every $\varepsilon \ge 0$ simultaneously, trading a larger ε for a smaller δ (Figure 20.3). At $\sigma = \Delta_2 f$, so $\mu = 1$, the curve passes through $\delta = 0.3829$ at $\varepsilon = 0$ (the TV distance), 0.1269 at $\varepsilon = 1$, 0.0209 at $\varepsilon = 2$ and 4.71×10^{-5} at $\varepsilon = 4$, and reaches 10^{-5} at $\varepsilon = 4.3772$. The whole curve, not a single point, is what composes tightly in Part 4. Only the ratio $\mu = \Delta_2 f/\sigma$ matters, because Equation 20.17 depends on $\Delta_2 f$ and σ only through it: doubling the sensitivity and doubling the noise give the same curve. Read μ as a signal-to-noise ratio: it is the parameter of the Gaussian trade-off curve in Hypothesis testing.

Table 20.5 compares the two calibrations at $\delta = 10^{-5}$. Over the rows with $\varepsilon \le 1$ the classic formula uses between 1.30 and 1.68 times the necessary noise, and the gap grows as ε shrinks. Since variance is σ^2, the factor at $\varepsilon = 1$ means 69% more variance than needed, which you could instead spend on a smaller ε. The last column shows the other failure: beyond $\varepsilon = 8.42$ the true δ at the classic σ exceeds the target.

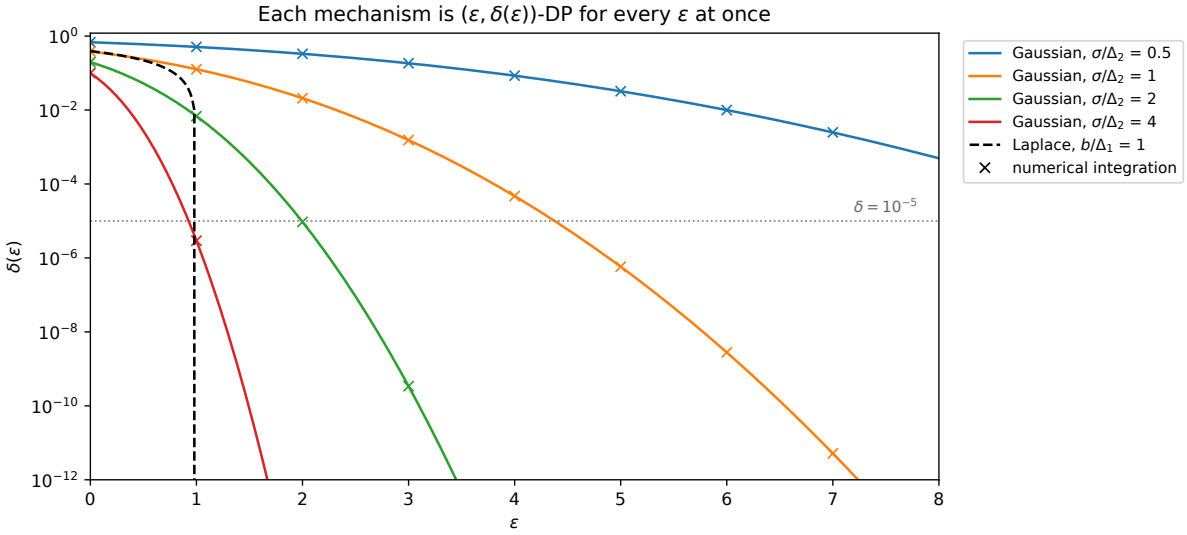


Figure 20.3.: The privacy profile $\delta(\varepsilon)$ of the Gaussian mechanism for four noise levels $\sigma/\Delta_2 f$, on a log scale, and of the Laplace mechanism with $b = \Delta_1 f$ (dashed). Crosses are the hockey-stick divergence computed by numerical integration. The Laplace curve drops to zero at $\varepsilon = 1$, its pure-DP level; every Gaussian curve stays positive but falls faster than exponentially. The dotted line marks $\delta = 10^{-5}$.

Table 20.5.: Classic against analytic calibration of the Gaussian mechanism at $\delta = 10^{-5}$. The last column is Equation 20.17 evaluated at the classic σ.

ε	Classic $\sigma/\Delta_2 f$	Analytic $\sigma^*/\Delta_2 f$	Ratio	True δ at the classic σ
0.05	96.8961	57.7707	1.677	1.29×10^{-9}
0.1	48.4481	30.7496	1.576	2.64×10^{-9}
0.25	19.3792	13.2855	1.459	7.10×10^{-9}
0.5	9.6896	7.0318	1.378	1.61×10^{-8}
1	4.8448	3.7306	1.299	4.11×10^{-8}
2	2.4224	1.9938	1.215	1.34×10^{-7}
4	1.2112	1.0812	1.120	6.84×10^{-7}
8	0.6056	0.6002	1.009	7.97×10^{-6}

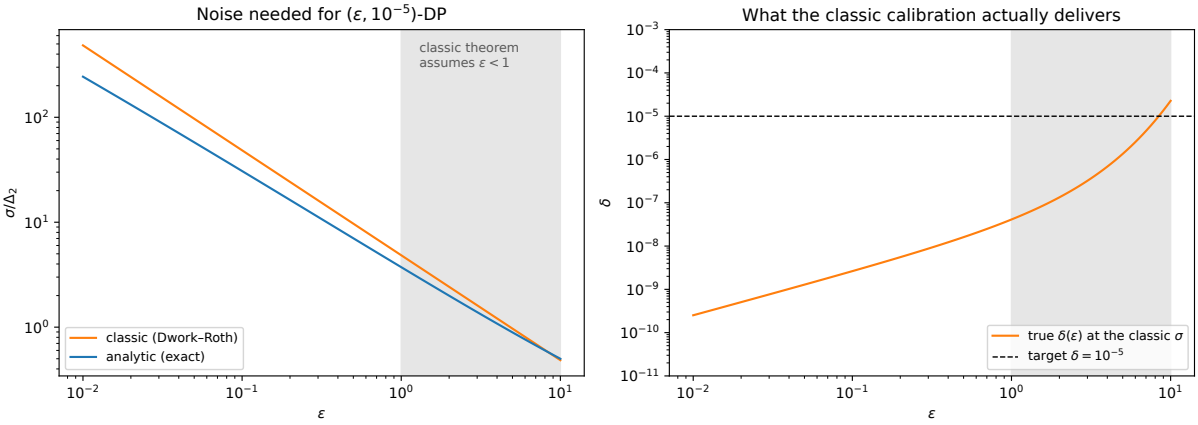


Figure 20.4.: Left: the noise needed for $(\varepsilon, 10^{-5})$-DP by the classic formula and by the analytic calibration. Right: the true δ that the classic σ delivers, from Equation 20.17. It is orders of magnitude below the target for small ε and crosses the target at $\varepsilon = 8.42$, in the shaded region where the classic theorem makes no claim.

20.5 Laplace against Gaussian in many dimensions

*For a **sparse query**, where one record moves one coordinate, $\Delta_1 f = \Delta_2 f$, and Laplace noise is more accurate whenever $\sqrt{2} < \varepsilon\,\sigma_1(\varepsilon, \delta)$, whatever d is. For a **dense query**, where one record moves every coordinate by 1, $\Delta_1 f = d$ and $\Delta_2 f = \sqrt{d}$, and Laplace noise is more accurate only while*

$$d < \frac{\left(\varepsilon\,\sigma_1(\varepsilon, \delta)\right)^2}{2}. \tag{20.21}$$

Proof.

1. *Laplace error.* A Lap(b) coordinate has mean 0 and variance $2b^2$ by Theorem 20.1, so its root-mean-square error is $\sqrt{2}\,b$, which with $b = \Delta_1 f/\varepsilon$ is $\sqrt{2}\,\Delta_1 f/\varepsilon$.
2. *Gaussian error.* A Gaussian coordinate has variance σ^2, so its root-mean-square error is σ. The analytic calibration gives $\sigma^* = \Delta_2 f\,\sigma_1(\varepsilon, \delta)$, because only $\mu = \Delta_2 f/\sigma$ enters Equation 20.17, so scaling the sensitivity scales the solution by the same factor.
3. *Sparse queries.* With $\Delta_1 f = \Delta_2 f$, Laplace is better when $\sqrt{2}\,\Delta_2 f/\varepsilon < \Delta_2 f\,\sigma_1$; dividing by $\Delta_2 f$ and multiplying by ε leaves $\sqrt{2} < \varepsilon\sigma_1$, with no d.
4. *Dense queries.* Substituting $\Delta_1 f = d$ and $\Delta_2 f = \sqrt{d}$, Laplace is better when $\sqrt{2}\,d/\varepsilon < \sqrt{d}\,\sigma_1$. Dividing both sides by $\sqrt{d}$ and rearranging gives $\sqrt{d} < \varepsilon\sigma_1/\sqrt{2}$, and squaring both sides gives the bound on d.

□

So use Laplace noise for sparse queries and Gaussian noise for dense ones. When a record touches one coordinate, the two sensitivities agree, and the stronger guarantee of Laplace, with $\delta = 0$, comes with less noise too. When a record touches all coordinates, $\Delta_1 f$ pays d while $\Delta_2 f$ pays $\sqrt{d}$, and the Gaussian wins by a factor that grows like $\sqrt{d}$ (Figure 20.5). This is why private training of neural networks uses Gaussian noise: one example's gradient moves every coordinate of a model with millions of parameters, the densest query there is, and clipping bounds its ℓ_2 norm, which is exactly what the Gaussian mechanism is calibrated to.

Keep in mind that the comparison is at equal ε but unequal guarantees, since the Laplace release is ε-DP and the Gaussian release has an extra δ. The Gaussian mechanism also has a second advantage that the comparison does not show: many Gaussian releases compose into another Gaussian mechanism, exactly, because stacking independent Gaussian answers gives one Gaussian answer whose ℓ_2 sensitivity is the root-sum-of-squares of theirs.

Example 20.5. The hospital releases the empirical cumulative distribution function (CDF) of its patients' ages, read off at d thresholds, at $\varepsilon = 1$ and $\delta = 10^{-5}$. There $\sigma_1 = 3.7306$, the value found by bisection in Section 20.4. For one count, Laplace noise has root-mean-square error $\sqrt{2} = 1.4142$ against 3.7306 for Gaussian noise, so Laplace wins. For threshold counts the crossover is at $d = (3.7306)^2/2 = 6.96$. Table 20.6 shows the errors, exactly (from $\sqrt{2}\,d$ and $3.7306\sqrt{d}$) and by simulation: at 100 thresholds Gaussian noise is 3.79 times more accurate, and at 1,000 thresholds 11.99 times.

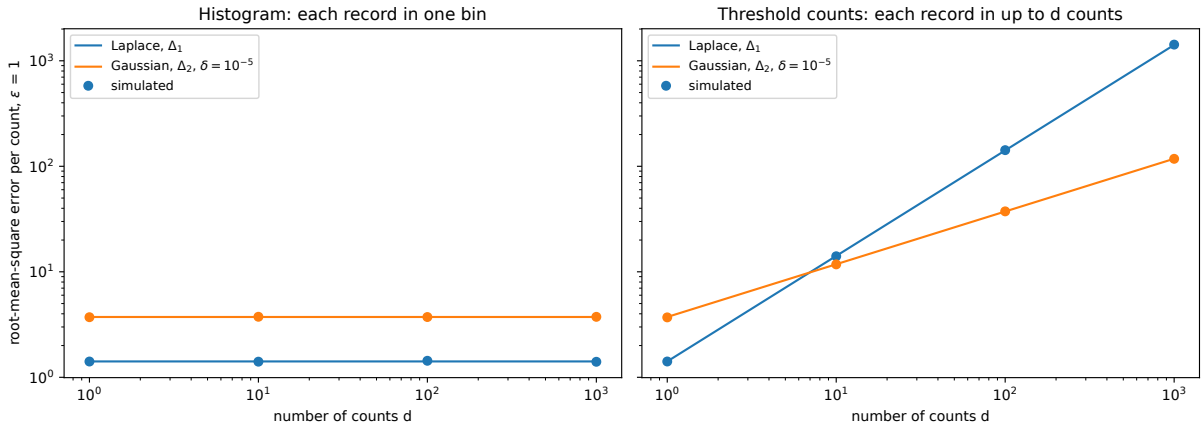


Figure 20.5.: Root-mean-square error per count at $\varepsilon = 1$ for the Laplace mechanism (calibrated to Δ_1) and the Gaussian mechanism (analytic calibration to Δ_2, $\delta = 10^{-5}$), as the number of released counts d grows; dots are simulations. Left: a histogram, where each record falls in one bin, so neither error depends on d and Laplace noise is smaller. Right: threshold counts, where a record can move all d counts; the Laplace error grows like d, the Gaussian like $\sqrt{d}$, and they cross near $d = 7$.

Table 20.6.: Root-mean-square error per count for d threshold counts at $\varepsilon = 1$, $\delta = 10^{-5}$.

d	Laplace, exact	Laplace, simulated	Gaussian, exact	Gaussian, simulated	Laplace / Gaussian
1	1.41	1.42	3.73	3.73	0.38
10	14.14	14.09	11.80	11.78	1.20
100	141.42	141.07	37.31	37.28	3.79
1000	1414.21	1412.90	117.97	118.11	11.99

20.6. Code

The script code/laplace_gaussian.py computes every number in this chapter and draws every figure. Sensitivities are found by brute force over every dataset of one to three records with values in $\{0, 1, 2, 3\}$. The Laplace mechanism's privacy and accuracy are checked by simulation. For the Gaussian mechanism, the closed form Equation 20.17 is checked against Simpson's rule on the two densities and against a Monte Carlo average of the privacy loss, and the analytic calibration is solved by bisection on $[e^{-12}, e^{8}]$. From the repository root:

```
uv run python code/part03-dp-foundations/laplace_gaussian.py
```

Table 20.7.: The script's output with the default seed.

Experiment	Result
Sensitivities, brute force over datasets of up to 3 records	Every row of Table 20.1, e.g. threshold counts $\ell_1 = 3$, $\ell_2 = 1.7321$
Laplace privacy loss, count, $\varepsilon = 1$	Ranges over exactly $[-1, 1]$
Laplace tight event	$\Pr[M(D) \le 0] = 0.5002$, $\Pr[M(D') \le 0] = 0.1838$, ratio 2.7221 against $e = 2.7183$
Laplace privacy loss distribution, $\varepsilon = 1$	Atoms 0.5000 and 0.1839 (simulated 0.5007, 0.1841), ramp 0.3161; mean 0.3679 (simulated 0.3682)
Laplace count, $\varepsilon = 0.1$	Mean absolute error 9.978 (exact 10); 95th percentile 29.85 (bound 29.96)
Laplace histogram, 20 bins	Maximum error above 5.99 in a fraction 0.0464 of releases ($\beta = 0.05$)
Classic proof chain, $\varepsilon = 1$, $\delta = 10^{-5}$	Exact δ 4.11×10^{-8}; $\Pr[L > \varepsilon]$ 1.06×10^{-6}; Mills bound 1.10×10^{-6}
Gaussian closed form vs integration	Largest relative difference 3.1×10^{-6} over 24 pairs (ε, μ) with $\delta > 10^{-12}$
Gaussian, $\mu = 1$, $\varepsilon = 1$	$\delta = 0.1269$ (integrated 0.1269, Monte Carlo 0.1270)
Analytic calibration, $\varepsilon = 1$, $\delta = 10^{-5}$	$\sigma^* = 3.7306$, classic 4.8448
Classic σ fails at $\delta = 10^{-5}$	For $\varepsilon > 8.42$
Laplace vs Gaussian, threshold counts	Crossover at $d = 6.96$; Table 20.6

tests/test_laplace_gaussian.py checks that the output still matches.

20.7. Summary

Table 20.8.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 20.1	$\Delta_p f = \max_{D \sim D'} \lVert f(D) - f(D') \rVert_p$	Compute it first, from the domain, and say which neighbouring relation it is for
Proposition 20.1	$\Delta_2 f \le \Delta_1 f \le \sqrt{d}\,\Delta_2 f$	Sparse queries sit at the left end, dense ones at the right
Proposition 20.2	Count 1; sum in $[0, B]$: B; mean with public n: B/n; histogram 1, or 2 and $\sqrt{2}$ under replace	Calibrating every release

Result	Statement	Use
Lemma 20.1	$E_\gamma(M(D)\|M(D')) = E_\gamma(Z\|Z+v)$	Only the shift matters, so analyse the noise against a shifted copy of itself
Theorem 20.1	$f(D) + \mathrm{Lap}(\Delta_1 f/\varepsilon)^d$ is ε-DP, tightly; error b per coordinate, $b\ln(d/\beta)$ for all at once	Counts, histograms, low-dimensional statistics; report-noisy-max
Lemma 20.2	$\delta_{\text{exact}} = \Pr_P[L > \varepsilon] - e^\varepsilon \Pr_Q[L > \varepsilon] \le \Pr_P[L > \varepsilon]$	The exact δ of any mechanism from its privacy loss; Numerical accounting
Lemma 20.3	$L \sim \mathcal{N}(\mu^2/2, \mu^2)$ under $M(D)$, with $\mu = \|v\|_2/\sigma$	Unbounded, so no pure DP; the object Part 4's accountants compose
Theorem 20.2	$\sigma = \Delta_2 f\sqrt{2\ln(1.25/\delta)}/\varepsilon$ for $\varepsilon < 1$	Reading papers and libraries that still use it; 1.3 to 1.7 times too much noise, and wrong for large ε
Theorem 20.3	(ε,δ)-DP iff $\delta \ge \Phi(\frac{\mu}{2} - \frac{\varepsilon}{\mu}) - e^\varepsilon \Phi(-\frac{\mu}{2} - \frac{\varepsilon}{\mu})$	Every Gaussian release; quote the profile, or μ, not one (ε,δ) pair
Corollary 20.1	$\sigma^* = \Delta_2 f/\mu^*$ with $\delta_{\mu^*}(\varepsilon) = \delta$, one bisection	At $\varepsilon = 1$, $\delta = 10^{-5}$: $\sigma^* \approx 3.73\,\Delta_2 f$
Proposition 20.3	Per coordinate: $\sqrt{2}\,\Delta_1 f/\varepsilon$ against $\Delta_2 f\,\sigma_1(\varepsilon,\delta)$	One number or a sparse vector: Laplace, pure DP. A dense vector: Gaussian, calibrated to Δ_2; gradients in Part 5

20.8. References

- Dwork, McSherry, Nissim & Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (TCC 2006): the paper that introduced global sensitivity and the Laplace mechanism.
- Dwork, Kenthapadi, McSherry, Mironov & Naor, "Our Data, Ourselves: Privacy via Distributed Noise Generation" (EUROCRYPT 2006): approximate (ε,δ)-DP and Gaussian noise.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3 for the Laplace mechanism and its accuracy, and Appendix A for the classic Gaussian calibration (Theorem A.1) with its two-sided proof.
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising" (ICML 2018): the exact privacy profile Equation 20.17, the bisection calibration, and why the classic formula fails for large ε.

- Near & Abuah, *Programming Differential Privacy*, the chapters on sensitivity and on the Laplace and Gaussian mechanisms: the same material with code, and the practical pitfalls of computing sensitivity.
- Geng & Viswanath, "The Optimal Noise-Adding Mechanism in Differential Privacy" (IEEE Transactions on Information Theory, 2016): the staircase mechanism, optimal for pure DP, and how close the Laplace mechanism comes to it.
- Dong, Roth & Su, "Gaussian Differential Privacy" (JRSS-B 2022): the Gaussian mechanism as a trade-off curve, which is where Gaussian differential privacy takes $\mu = \Delta_2 f/\sigma$ next.

21. Advanced and optimal composition

A differentially private system rarely answers one query: a dashboard refreshes a hundred counts, and a training run takes thousands of noisy gradient steps. Each step satisfies (ε, δ)-differential privacy (DP) on its own, and you need a guarantee for the whole run. Post-processing, group privacy and composition gave the first answer, **basic composition**: k mechanisms that are each ε-DP are together $k\varepsilon$-DP. For large k that answer is far too pessimistic. A hundred queries at $\varepsilon = 0.1$ give $k\varepsilon = 10$, which lets an observer multiply its odds between two neighbouring datasets by up to $e^{10} \approx 22{,}000$. Yet at $\delta = 10^{-5}$ the same hundred queries are $(4.31, 10^{-5})$-DP, and no analysis that knows only each query's ε can say less.

The one idea behind the gap is that the privacy loss of a composition is a sum. Each output is evidence for one dataset or the other, measured by the logarithm of how much likelier the output is under D than under its neighbour D', and the evidence of k outputs adds up. Basic composition charges the largest value that sum can take, as if every step produced its worst output. But a sum of k bounded random terms rarely strays from its mean by more than a multiple of $\sqrt{k}$. Allowing a small δ lets you ignore the rare extreme sums and charge for the typical one, and that is **advanced composition**. The best bound of this kind, **optimal composition**, can be computed exactly, because randomized response is the mechanism whose composition is hardest to protect. Both survive an analyst who chooses each query after seeing the earlier answers, which is **adaptive composition**.

The running example is the dashboard: $k = 100$ counts, each released by a mechanism that is 0.1-DP, under add/remove neighbours. Basic composition charges $\varepsilon = 10$, advanced composition 5.85 at $\delta' = 10^{-5}$, and optimal composition 4.31, and the chapter derives all three from the distribution of the total privacy loss.

21.1. The privacy loss of a composition

Every bound in this chapter is a statement about one random variable, the total privacy loss of the run. This section defines it, shows that it is a sum over the steps, and works out its distribution for composed randomized response, which Section 21.3 proves is the worst case.

Definition 21.1 (The privacy loss random variable)**.** Let M be a mechanism and $D \sim D'$ two neighbouring datasets. Write $P = M(D)$ and $Q = M(D')$ for the two output distributions, with mass functions or densities p and q. The **privacy loss** of an output y is the log-ratio $L_{D,D'}(y) = \ln\left(p(y)/q(y)\right)$, which is $+\infty$ where $q(y) = 0 < p(y)$, and the **privacy loss random variable** is the loss of a random output drawn under D,

$$L = L_{D,D'}(Y) = \ln \frac{p(Y)}{q(Y)}, \qquad Y \sim P. \tag{21.1}$$

Its distribution is the **privacy loss distribution** of the pair. Once the pair is fixed, $L(y)$ is short for $L_{D,D'}(y)$.

(ε, δ)-DP asks that $\Pr_P[S] \le e^\varepsilon \Pr_Q[S] + \delta$ for every set of outputs S; with $\delta = 0$ this says $|L| \le \varepsilon$ for every output (The definition of differential privacy). For one ordered pair of datasets, the smallest δ that works at a given ε is the largest gap $\Pr_P[S] - e^\varepsilon \Pr_Q[S]$, which Divergences shows is reached on the outputs with $p(y) > e^\varepsilon q(y)$. That largest gap is the **hockey-stick divergence**,

$$\delta(\varepsilon) = E_{e^\varepsilon}(P\|Q) = \sum_y \big(p(y) - e^\varepsilon q(y)\big)_+, \qquad (x)_+ = \max\{x, 0\}, \tag{21.2}$$

and as a function of ε it is the **privacy profile** of the pair in one direction. DP asks for it in both directions, for every pair of neighbours.

The smallest case is **randomized response** at level ε (The definition of differential privacy), which reports a person's bit truthfully with probability $t = e^\varepsilon/(1 + e^\varepsilon)$ and flips it otherwise. For a person whose bit is 0 in D and 1 in D', at $\varepsilon = \ln 3$, a report of 0 has probability $\frac{3}{4}$ under D and $\frac{1}{4}$ under D', so its loss is $\ln 3$, and a report of 1 has loss $-\ln 3$. The first lemma says that the profile is an average of a function of the loss, so a composition theorem only has to control the distribution of the total loss.

Lemma 21.1 (The privacy profile is an average of the privacy loss)**.** *For every pair (P, Q) and every $\varepsilon \ge 0$,*

$$\delta(\varepsilon) = E_{e^\varepsilon}(P\|Q) = \sum_y \big(p(y) - e^\varepsilon q(y)\big)_+ = \mathbb{E}\big[(1 - e^{\varepsilon - L})_+\big] \le \Pr[L > \varepsilon], \tag{21.3}$$

where the expectation and the probability are over $Y \sim P$. In particular, $\delta(\varepsilon)$ depends on the pair only through the distribution of L under P.

Proof.

1. *Factor out p.* Each term of the hockey-stick sum is $(p - e^\varepsilon q)_+ = p\,(1 - e^\varepsilon q/p)_+$, and the ratio in the bracket is $e^\varepsilon q/p = e^{\varepsilon - \ln(p/q)} = e^{\varepsilon - L(y)}$. A sum of $p(y)$ times a function of y is the expectation of that function when $Y \sim P$, which gives the equality with the expectation. An output with $q = 0$ has $L = +\infty$ and bracket 1, as it should.
2. *Bound the bracket.* When $L \le \varepsilon$ the exponent $\varepsilon - L$ is non-negative, so $e^{\varepsilon - L} \ge 1$ and the bracket is 0. When $L > \varepsilon$ the bracket is $1 - e^{\varepsilon - L}$, which lies strictly between 0 and 1. So the bracket is at most the indicator $\mathbf{1}[L > \varepsilon]$, and its average is at most $\Pr[L > \varepsilon]$.

□

An output whose loss is at most ε contributes nothing to $\delta(\varepsilon)$, and an output that is impossible under D' contributes its whole probability. For randomized response at $\ln 3$, only the report 0, with loss $\ln 3$, has a positive loss, so the lemma at $\varepsilon = 0$ gives $\delta(0) = \frac{3}{4}(1 - \frac{1}{3}) = \frac{1}{2}$, the total variation distance $\frac{3}{4} - \frac{1}{4}$, as it should. Advanced composition uses the tail bound of the lemma, and optimal composition uses the equality.

Now run k mechanisms. The probability of a whole sequence of outputs is built one step at a time, and the logarithm turns that product into a sum.

Lemma 21.2 (The privacy loss of a composition is a sum)**.** *Run $M_1, \dots, M_k$ in sequence, where M_i may depend on the earlier outputs $y_{<i} = (y_1, \dots, y_{i-1})$. Write $p_i(y_i \mid y_{<i})$ for the probability that step i outputs y_i on D after the history $y_{<i}$, and $q_i(y_i \mid y_{<i})$ for the same on D'. Then the privacy loss of the whole sequence is the sum of the per-step losses,*

$$L(y_1, \dots, y_k) = \sum_{i=1}^{k} L_i(y_i \mid y_{<i}), \qquad L_i(y_i \mid y_{<i}) = \ln \frac{p_i(y_i \mid y_{<i})}{q_i(y_i \mid y_{<i})}, \tag{21.4}$$

where L_i is the privacy loss of step i alone, for the mechanism that was run after that history. If step i is ε-DP for every history, then $|L_i| \le \varepsilon$, and so $|L| \le k\varepsilon$.

Proof.

1. *Chain rule.* The probability of a sequence of outputs is the probability of y_1, times the probability of y_2 given y_1, and so on. So the joint output has probability $p(y_1, \dots, y_k) = \prod_i p_i(y_i \mid y_{<i})$ on D, and likewise with q_i on D'.
2. *Log of a ratio of products.* The ratio of the two products is the product of the step-by-step ratios, and the logarithm of a product is the sum of the logarithms, which is Equation 21.4.
3. *Basic composition.* If step i is ε-DP whatever the history, each term lies in $[-\varepsilon, \varepsilon]$, and adding k such bounds gives $|L| \le k\varepsilon$.

□

Table 21.1 shows the sum for two independent rounds of randomized response at $\varepsilon = \ln 3$: the output $(0, 1)$ has loss 0 because one report favours D as much as the other favours D'.

Table 21.1.: Two rounds of randomized response at $\varepsilon = \ln 3$. The privacy loss of each output is the sum of the two per-round losses.

Output (y_1, y_2)	$(0,0)$	$(0,1)$	$(1,0)$	$(1,1)$
p (bit 0)	$9/16$	$3/16$	$3/16$	$1/16$
q (bit 1)	$1/16$	$3/16$	$3/16$	$9/16$
$L = \ln(p/q)$	$2\ln 3$	0	0	$-2\ln 3$

For randomized response the sum is a random walk with steps $\pm\varepsilon$, and its whole distribution can be written down. A report of 0 is the truth under D, so it **points to** D; under D a report points to D' only when it is a lie.

Proposition 21.1 (The loss of composed randomized response)**.** *Compose k independent rounds of randomized response at level ε, on a person whose bit is 0 in D and 1 in D', and let J be the number of rounds whose report points to D'. Under D,*

$$J \sim \mathrm{Bin}(k, 1 - t), \qquad L = (k - 2J)\varepsilon, \tag{21.5}$$

and the loss has mean, variance and worst-case probability

$$\mathbb{E}[L] = k\varepsilon \tanh(\varepsilon/2), \qquad \mathrm{Var}(L) = k\varepsilon^2(1 - \tanh^2(\varepsilon/2)), \qquad \Pr[L = k\varepsilon] = t^k. \tag{21.6}$$

Proof. The plan is to find the loss of one round, add k of them, and then compute the mean and variance of one round and multiply by k.

1. *One round.* A truthful report has probability t under D and $1-t$ under D', so its loss is $\ln \frac{t}{1-t} = \ln e^{\varepsilon} = \varepsilon$, because $t/(1-t) = e^{\varepsilon}$ by the definition of t. A lie has the two probabilities swapped and loss $-\varepsilon$.
2. *k rounds.* By Lemma 21.2 the total is the sum of the k per-round losses. The J rounds that point to D' contribute $-\varepsilon$ each and the other $k-J$ contribute $+\varepsilon$, so $L = (k-J)\varepsilon - J\varepsilon = (k-2J)\varepsilon$. Each round points to D' exactly when it lies, independently with probability $1-t$, so J counts the successes in k independent trials and is binomial.
3. *Mean.* One round has mean $\varepsilon t - \varepsilon(1-t) = \varepsilon(2t-1)$. Substituting t gives $2t - 1 = \frac{e^{\varepsilon}-1}{e^{\varepsilon}+1}$, and this is $\tanh(\varepsilon/2)$, because $\tanh x = \frac{e^{2x}-1}{e^{2x}+1}$. The mean of a sum is the sum of the means, so multiply by k.
4. *Variance.* One round takes the two values $\pm\varepsilon$, so its square is always ε^2 and its variance is $\varepsilon^2 - (\text{mean})^2 = \varepsilon^2(1 - \tanh^2(\varepsilon/2))$. Variances of independent terms add, so the total variance is k times this.
5. *The worst case.* The value $L = k\varepsilon$ needs $J = 0$, every report truthful, which has probability t^k.

□

For small ε, $\tanh(\varepsilon/2) \approx \varepsilon/2$, so the mean is about $k\varepsilon^2/2$ and the standard deviation about $\sqrt{k}\,\varepsilon$, both far below the worst case $k\varepsilon$, which needs all k reports to be truthful. Basic composition guards against an event that will never happen.

By the central limit theorem the total loss is nearly normal (Figure 21.1), the starting point of the Gaussian view of privacy in Privacy accounting. Advanced composition bounds its tail, and optimal composition evaluates Equation 21.3 on the binomial exactly.

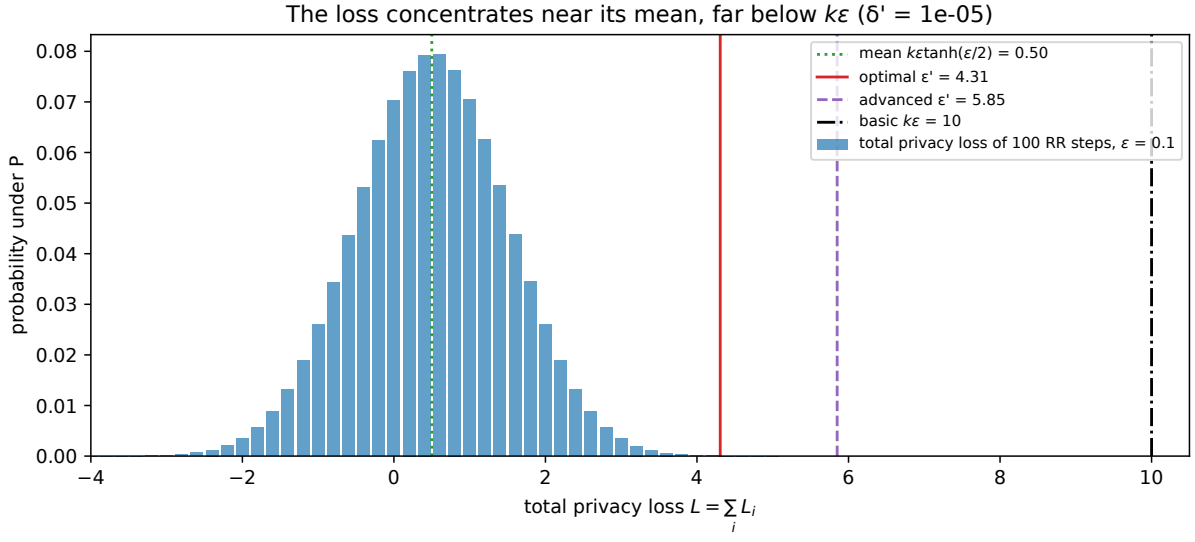


Figure 21.1.: The total privacy loss of 100 rounds of randomized response at $\varepsilon = 0.1$, under D, with its mean, the optimal and advanced ε' at $\delta' = 10^{-5}$, and basic composition's $k\varepsilon = 10$.

Example 21.1. Release each of the dashboard's $k = 100$ counts by randomized response at $\varepsilon = 0.1$, the worst 0.1-DP mechanism by Section 21.3. Each round is truthful with probability $t = e^{0.1}/(1+e^{0.1}) = 0.5250$. By Proposition 21.1 the total loss has mean $100 \times 0.1 \tanh(0.05) = 0.4996$ and standard deviation 0.9988. Basic composition's $k\varepsilon = 10$ sits $(10 - 0.50)/1.00 = 9.5$ standard deviations above the mean, and it needs all 100 reports to be truthful, which has probability $t^{100} = 1.0 \times 10^{-28}$. The loss $(100 - 2J) \times 0.1$ exceeds 4 only when $J < 30$, about 3.5 standard deviations of J (which are 5) below its mean $100(1 - t) = 47.5$. The optimal ε' at $\delta' = 10^{-5}$ that Section 21.4 computes, 4.3068, sits $(4.3068 - 0.4996)/0.9988 = 3.8$ standard deviations above the mean of the loss.

21.2. Advanced composition

Advanced composition, due to Dwork, Rothblum & Vadhan (2010), charges for the typical value of the sum rather than its largest: it needs only each step's ε, holds for mechanisms chosen in any adaptive way, and grows like $\sqrt{k}$. Its tool is Azuma's inequality, a version of Hoeffding's inequality for sums whose terms may depend on the earlier terms. Azuma controls how far the sum strays from its mean, so the theorem also needs a bound on the mean of each term, the drift of the sum, and that bound comes first.

The loss can be as large as ε, but on average it is much smaller: when ε is small, positive and negative losses nearly cancel, and what is left is of order ε^2. The average loss under P is $\sum_y p(y) \ln(p(y)/q(y))$, the Kullback–Leibler (KL) divergence $D_{\mathrm{KL}}(P\|Q)$ of Divergences.

Lemma 21.3 (The mean privacy loss of an ε-DP pair is second order in ε)**.** *If $|L| \le \varepsilon$ for every output, then*

$$\mathbb{E}[L] = D_{\mathrm{KL}}(P\|Q) \le \varepsilon(e^{\varepsilon} - 1). \tag{21.7}$$

Proof.

1. *Symmetrize.* The KL divergence is never negative (Divergences): by Jensen's inequality for the concave logarithm, $-D_{\mathrm{KL}}(Q\|P) = \sum_y q \ln(p/q) \le \ln \sum_y q\,(p/q) = \ln 1 = 0$. So adding $D_{\mathrm{KL}}(Q\|P) \ge 0$ to $\mathbb{E}[L]$ can only increase it, and the two sums combine term by term:

$$\mathbb{E}[L] \le \sum_y p \ln \frac{p}{q} + \sum_y q \ln \frac{q}{p} = \sum_y (p - q) \ln \frac{p}{q}. \tag{21.8}$$

2. *Use the bound on the loss.* Each $|\ln(p/q)|$ is at most ε, so the sum is at most $\varepsilon \sum_y |p - q|$, and the total difference $\sum_y |p - q|$ is itself small when P and Q are close.
3. *Bound the total difference.* Write $\sum_y |p - q| = \sum_y q\,|p/q - 1|$. The ratio p/q lies in $[e^{-\varepsilon}, e^{\varepsilon}]$, and on that interval $|p/q - 1|$ is largest at the upper end, where it is $e^{\varepsilon} - 1$ (at the lower end it is $1 - e^{-\varepsilon}$, which is smaller). Since the $q(y)$ sum to 1, $\sum_y |p - q| \le e^{\varepsilon} - 1$, and the bound follows.

□

So the mean is of order ε^2, and at $\varepsilon = 0.1$ the bound is $0.1(e^{0.1} - 1) = 0.0105$. It is not tight. Theorem 21.3 in Section 21.3 gives the best bound, $\varepsilon \tanh(\varepsilon/2)$, which is 0.0050 at $\varepsilon = 0.1$ and 0.4621 at $\varepsilon = 1$, where the lemma gives 1.7183. Section B.2 in the appendices derives the same bound from scratch for a single count, with every number shown.

Theorem 21.1 (Advanced composition (Dwork, Rothblum & Vadhan 2010))**.** *Fix k and $\varepsilon > 0$ in advance, and let $M_1, \dots, M_k$ be mechanisms, each chosen after seeing the earlier outputs, such that every M_i is ε-DP for every history. Then for every $\delta' > 0$ the composition is (ε', δ')-DP with*

$$\varepsilon' = \varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon(e^{\varepsilon} - 1). \tag{21.9}$$

Proof. The plan is to write the loss as a sum, centre each term by its conditional mean to get a martingale, bound the fluctuation of the martingale by Azuma's inequality and the drift by Lemma 21.3, and convert the tail bound into a δ by Lemma 21.1.

1. *The loss is a sum.* By Lemma 21.2, $L = \sum_i L_i$, where L_i is the loss of step i given the history $y_{<i}$.
2. *Each term is bounded, given the past.* For every fixed history, step i is ε-DP, so $L_i \in [-\varepsilon, \varepsilon]$.
3. *Each term has a small conditional mean.* Given the history, the average of L_i over step i's output is $m_i = \mathbb{E}[L_i \mid y_{<i}] = \sum_{y_i} p_i \ln(p_i/q_i)$, the KL divergence of step i's pair. So $m_i \le \varepsilon(e^{\varepsilon} - 1)$ by Lemma 21.3.
4. *Centre the terms to get a martingale.* A **martingale** is a running sum whose next increment has mean zero given everything that happened before. Subtracting each term's conditional mean makes one: $W_k = \sum_i (L_i - m_i)$ has increments with conditional mean zero. Given the past, $L_i - m_i$ lies in $[-\varepsilon - m_i, \varepsilon - m_i]$, an interval of length $c_i = 2\varepsilon$, whose position depends on the past but whose length does not.
5. *Azuma.* The Azuma–Hoeffding inequality of Concentration inequalities says that if each increment of a martingale has conditional mean zero and lies, given the past, in an interval of length c_i fixed in advance, then the sum exceeds s with probability at most $\exp(-2s^2/\sum_i c_i^2)$. It holds because Hoeffding's lemma bounds each increment's conditional moment generating function using only its mean and the length of its range, and these bounds multiply one step at a time. With $c_i = 2\varepsilon$, so that $\sum_i c_i^2 = 4k\varepsilon^2$,

$$\Pr[W_k \ge s] \le \exp\left(-\frac{2s^2}{4k\varepsilon^2}\right) = \exp\left(-\frac{s^2}{2k\varepsilon^2}\right). \tag{21.10}$$

6. *Choose the level.* Setting the right side to δ' and solving, $s^2 = 2k\varepsilon^2 \ln(1/\delta')$, gives $s = \varepsilon\sqrt{2k\ln(1/\delta')}$. The loss is the fluctuation plus the drift, $L = W_k + \sum_i m_i \le W_k + k\varepsilon(e^{\varepsilon} - 1)$. So if L exceeds $\varepsilon' = s + k\varepsilon(e^{\varepsilon} - 1)$, then W_k must exceed s, and $\Pr[L > \varepsilon'] \le \delta'$.
7. *Convert.* By Lemma 21.1, $\delta(\varepsilon') \le \Pr[L > \varepsilon'] \le \delta'$. The same argument with D and D' swapped gives the other direction, so the composition is (ε', δ')-DP.

□

The gain is $\sqrt{k}$ instead of k. The price is δ' and the factor $\sqrt{2\ln(1/\delta')}$, which is $\sqrt{2 \times 11.51} = 4.80$ at $\delta' = 10^{-5}$, so the theorem beats basic composition only once k is large. It wins when

$\varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon(e^{\varepsilon}-1) < k\varepsilon$. Dividing by $k\varepsilon$ leaves $\sqrt{2\ln(1/\delta')/k} < 2 - e^{\varepsilon}$, and squaring, which needs $\varepsilon < \ln 2$ so that $2 - e^{\varepsilon} > 0$, gives the crossover:

$$k > \frac{2\ln(1/\delta')}{(2-e^{\varepsilon})^2}. \tag{21.11}$$

Setting the leading term equal to a target $\varepsilon_{\text{total}}$ gives each step $\varepsilon \approx \varepsilon_{\text{total}}/\sqrt{2k\ln(1/\delta')}$, where basic composition would allow only $\varepsilon_{\text{total}}/k$. Corollary 21.1 in Section 21.4 extends the theorem to (ε, δ)-DP steps at a cost of $k\delta$, with the sharper drift $k\varepsilon\tanh(\varepsilon/2)$.

The bound is loose in three places: Azuma assumes each increment has the largest variance its range allows, the mean bound of Lemma 21.3 is a factor 2 too large for small ε, and the tail bound of Lemma 21.1 counts every output above ε' in full. At $k = 100$ and $\varepsilon = 0.1$, the exact δ at the advanced $\varepsilon' = 5.85$ is 1.96×10^{-9}, not 10^{-5}.

Example 21.2. Take the dashboard's $k = 100$ counts at $\varepsilon = 0.1$ and $\delta' = 10^{-5}$. The Azuma term is $0.1\sqrt{200\ln 10^5} = 0.1\sqrt{2302.6} = 4.7985$, and the mean term is $100 \times 0.1(e^{0.1} - 1) = 1.0517$, so $\varepsilon' = 5.8502$, against 10 for basic composition. The crossover is $23.03/0.8948^2 = 28.8$, so advanced composition wins from $k = 29$ on (Figure 21.2, and Table 21.4 in Section 21.4). At $k = 10{,}000$ the mean term dominates: it costs 105 of the total 153, against 48 for the Azuma term, and the sharper drift of Corollary 21.1 costs 50 instead.

Laplace noise in many dimensions. The same arithmetic explains where the Gaussian mechanism's advantage in high dimension comes from. Take d dense counts, where one record can move every count by 1, at $\varepsilon = 1$ and $\delta = 10^{-5}$. Laplace noise of scale b has root-mean-square error $\sqrt{2}\,b$, and on the whole vector it must use the ℓ_1 sensitivity d, so $b = d/\varepsilon$. Composing d separate Laplace mechanisms instead, at the largest ε_0 whose composition stays at 1, gives error $\sqrt{2}/\varepsilon_0$: for $d = 1000$, $\varepsilon_0 = 0.00645$ by Equation 21.9, or 0.00849 by the optimal composition of Section 21.4. Table 21.2 adds the Gaussian mechanism of the Laplace and Gaussian mechanisms. Composed Laplace noise grows like $\sqrt{d}$ and stays within a factor of 1.9 of the Gaussian, while pure-DP Laplace noise falls behind by a factor of 38 at $d = 10{,}000$. So the $\sqrt{d}$ gain comes from allowing $\delta > 0$, not from the Gaussian density.

Table 21.2.: Root-mean-square error per count for d dense counts at $\varepsilon = 1$ and, except in the first column, $\delta = 10^{-5}$.

d	Laplace, pure DP	Laplace, advanced composition	Laplace, optimal composition	Gaussian
100	141.4	69.3	52.3	37.3
1,000	1,414.2	219.2	166.6	118.0
10,000	14,142.1	693.0	527.5	373.1

21.3. The worst mechanism

Advanced composition is one bound among many, and the question is which is best. Kairouz, Oh & Viswanath (2015) answered it with post-processing: every (ε, δ)-DP mechanism is a post-processing of one version of randomized response, so no composition of such mechanisms can be

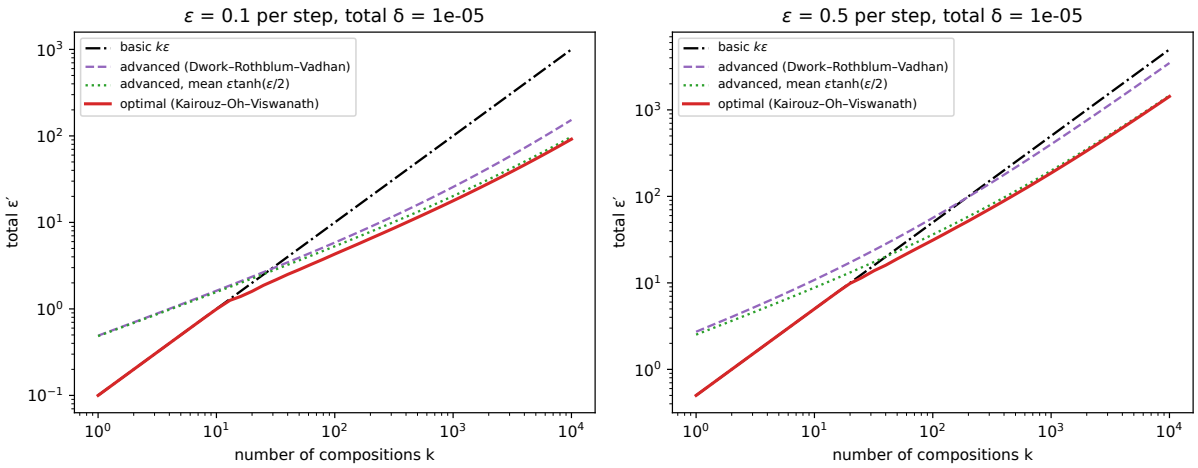


Figure 21.2.: Total ε' at $\delta = 10^{-5}$ after k steps that are each ε-DP, for $\varepsilon = 0.1$ (left) and 0.5 (right), on log axes. Basic composition grows like k, the others like $\sqrt{k}$ until the mean term takes over.

worse than composed randomized response. This section builds that mechanism and proves the reduction, and Section 21.4 composes it.

A **channel** K is a table of probabilities $K(y \mid z)$: given an input z, it draws an output y without looking at the data, and feeding a distribution $P^\star$ through it gives $P^\star K(y) = \sum_z P^\star(z)K(y \mid z)$. The data-processing inequality of Divergences says that a channel never increases the hockey-stick divergence or the KL divergence. For the hockey-stick the reason is short: the gap $P^\star K(S) - e^{\varepsilon'} Q^\star K(S)$ on a set S is $\sum_z K(S \mid z)(P^\star(z) - e^{\varepsilon'} Q^\star(z))$ with weights in $[0, 1]$, so it is at most the sum of the positive brackets, which is the hockey-stick before the channel.

The worst mechanism must spend both parameters. It spends δ by announcing, with probability δ, which dataset it ran on, and ε by otherwise running randomized response at level ε.

Definition 21.2 (The four-point randomized response)**.** The **four-point pair** $(P^\star, Q^\star)$ at level (ε, δ) is the pair of output distributions in Table 21.3: with probability δ the mechanism announces its dataset outright, and otherwise it reports the person's bit through randomized response at level ε, with $t = e^\varepsilon/(1 + e^\varepsilon)$.

Table 21.3.: The four-point randomized response of Kairouz, Oh & Viswanath at level (ε, δ).

Output	"flag D"	0	1	"flag D'"
$P^\star$ (on D)	δ	$(1-\delta)t$	$(1-\delta)(1-t)$	0
$Q^\star$ (on D')	0	$(1-\delta)(1-t)$	$(1-\delta)t$	δ
Privacy loss	$+\infty$	$+\varepsilon$	$-\varepsilon$	$-\infty$

Every (ε, δ)-DP pair can be produced from the four-point pair by one channel, and for $\delta = 0$ the channel has a closed form.

Theorem 21.2 (Randomized response is the worst (ε, δ)-DP mechanism)**.** *Let (P, Q) be a pair of distributions on a common set of outputs that satisfies (ε, δ)-DP in both directions. Then there is a channel K with $P = P^\star K$ and $Q = Q^\star K$, where $(P^\star, Q^\star)$ is the four-point pair of Definition 21.2. For $\delta = 0$ the channel is*

$$K(y \mid 0) = \frac{e^\varepsilon p(y) - q(y)}{e^\varepsilon - 1}, \qquad K(y \mid 1) = \frac{e^\varepsilon q(y) - p(y)}{e^\varepsilon - 1}. \tag{21.12}$$

Proof. For $\delta = 0$, solve for the channel and check that it reproduces the pair; for $\delta > 0$, the last step says why the channel exists and where it is built.

1. *Solve for the channel.* Plain randomized response outputs 0, the report that points to D, with probability t on D and $1 - t$ on D'. So the channel must satisfy, for every output y of the target pair, $p(y) = tK(y \mid 0) + (1 - t)K(y \mid 1)$ and $q(y) = (1 - t)K(y \mid 0) + tK(y \mid 1)$. These are two linear equations in the two unknowns $K(y \mid 0)$ and $K(y \mid 1)$, and solving them with $t/(1 - t) = e^\varepsilon$ gives Equation 21.12.
2. *The rows are distributions.* A channel's rows must be non-negative and sum to 1. $K(y \mid 0) \geq 0$ exactly when $e^\varepsilon p(y) \geq q(y)$, and $K(y \mid 1) \geq 0$ exactly when $e^\varepsilon q(y) \geq p(y)$. Both hold for every y because the pair is ε-DP. Since p and q each sum to 1, each row sums to $(e^\varepsilon - 1)/(e^\varepsilon - 1) = 1$.
3. *They reproduce P.* Mixing the rows with randomized response's probabilities on D gives

$$t\,K(y \mid 0) + (1 - t)K(y \mid 1) = \frac{(te^\varepsilon - (1 - t))p(y) + ((1 - t)e^\varepsilon - t)q(y)}{e^\varepsilon - 1}. \tag{21.13}$$

   With $t = e^\varepsilon/(1 + e^\varepsilon)$, $(1 - t)e^\varepsilon = t$, so the coefficient of q is 0. The coefficient of p is $\frac{te^\varepsilon - (1-t)}{e^\varepsilon - 1} = \frac{(e^{2\varepsilon} - 1)/(1 + e^\varepsilon)}{e^\varepsilon - 1} = 1$, since $e^{2\varepsilon} - 1 = (e^\varepsilon - 1)(e^\varepsilon + 1)$.
4. *They reproduce Q.* The same computation with t and $1 - t$ swapped.
5. *The case $\delta > 0$, cited.* The **trade-off function** of a pair gives, for each false positive rate, the smallest false negative rate any test between P and Q can reach. The definition of differential privacy shows that a pair is (ε, δ)-DP exactly when its trade-off function lies above a boundary $f_{\varepsilon,\delta}$ made of two straight lines, on which the four-point pair sits (Figure 21.3, left). So every (ε, δ)-DP pair is at least as hard to test as $(P^\star, Q^\star)$, at every level. Blackwell's theorem (1953) says that a pair is at least as hard to test as another at every level exactly when it is a post-processing of the other. Kairouz, Oh & Viswanath (2015) construct the channel, and Section J.1 in the appendices derives it step by step, by splitting each distribution into an ε-DP part of mass $1 - \delta$ and a leftover of mass δ.

□

The same reduction sharpens the drift term of advanced composition, because the KL divergence, like the hockey-stick, can only shrink under a channel.

Theorem 21.3 (The sharp mean loss)**.** *If the pair (P, Q) is ε-DP in both directions, then*

$$\mathbb{E}[L] = D_{\mathrm{KL}}(P \| Q) \leq \varepsilon \tanh(\varepsilon/2), \tag{21.14}$$

with equality for randomized response at level ε.

Proof.

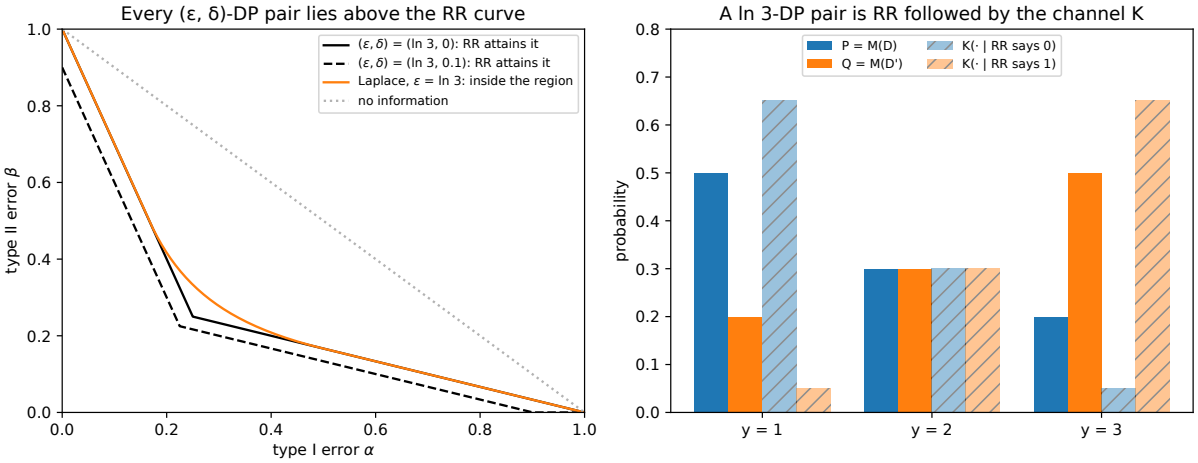


Figure 21.3.: Left: the boundaries $f_{\varepsilon,\delta}$ at $\varepsilon = \ln 3$ with $\delta = 0$ and 0.1, attained by randomized response, and a Laplace mechanism's trade-off curve inside the allowed region. Right: the pair of Example 21.3 and the two rows of its channel.

1. *Reduce to randomized response.* By Theorem 21.2 with $\delta = 0$, the pair is a post-processing of randomized response at level ε through the channel Equation 21.12. Passing two distributions through the same channel can only make them more alike, which is the data-processing inequality for the KL divergence, so $D_{\mathrm{KL}}(P\|Q)$ is at most the KL divergence of randomized response.
2. *The KL divergence of randomized response.* Its mean loss is $\varepsilon t - \varepsilon(1-t) = \varepsilon \tanh(\varepsilon/2)$, by step 3 of the proof of Proposition 21.1 with $k = 1$, and randomized response itself reaches it.

□

Example 21.3. The dashboard's randomized-response steps need no channel, so take another pair: $P = (0.5, 0.3, 0.2)$ and $Q = (0.2, 0.3, 0.5)$ have largest loss $\ln(0.5/0.2) = \ln 2.5 = 0.9163$, below $\ln 3 = 1.0986$, so the pair is $\ln 3$-DP. With $e^{\varepsilon} = 3$, Equation 21.12 gives $K(\cdot \mid 0) = (3P - Q)/2 = (0.65, 0.30, 0.05)$ and $K(\cdot \mid 1) = (3Q - P)/2 = (0.05, 0.30, 0.65)$ (Figure 21.3, right). Randomized response at $\ln 3$ has $t = \frac{3}{4}$, and the first output comes back as $\frac{3}{4} \times 0.65 + \frac{1}{4} \times 0.05 = 0.5$. As Theorem 21.3 requires, the pair's KL divergence, $0.5 \ln 2.5 + 0.3 \ln 1 + 0.2 \ln 0.4 = 0.2749$, is below randomized response's $\ln 3 \times \frac{1}{2} = 0.5493$, and its $\delta(0)$, the total variation $0.5 - 0.2 = 0.3$, is below 0.5.

21.4. Optimal composition

With the worst mechanism in hand, the best bound for k mechanisms known only by their (ε, δ) is the exact profile of k copies of the four-point pair. Either some step raises a flag, and the output counts in full, or none does, and the loss is binomial by Equation 21.5. Lemma 21.1 turns that binomial loss into a δ, written $\delta_{\mathrm{RR}}(\varepsilon')$.

Theorem 21.4 (Optimal composition (Kairouz, Oh & Viswanath 2015))**.** *For every composition of k mechanisms that are each (ε, δ)-DP, adaptive or not, and every $\varepsilon' \geq 0$,*

$$\delta(\varepsilon') \leq 1 - (1-\delta)^k(1 - \delta_{\mathrm{RR}}(\varepsilon')), \qquad \delta_{\mathrm{RR}}(\varepsilon') = \sum_{j=0}^{k} \binom{k}{j} t^{k-j}(1-t)^j \left(1 - e^{\varepsilon' - (k-2j)\varepsilon}\right)_+, \tag{21.15}$$

with equality for k independent copies of the four-point pair. The binomial weight $\binom{k}{j}t^{k-j}(1-t)^j$ is the probability that exactly j of the k reports point to D'. At the points where ε' equals one of the values the loss can take, $\varepsilon' = (k-2i)\varepsilon$ for $i = 0, \dots, \lfloor k/2 \rfloor$, the positive part can be dropped and the sum becomes a ratio of exponentials,

$$\delta_{\mathrm{RR}}((k-2i)\varepsilon) = \frac{\sum_{\ell=0}^{i-1} \binom{k}{\ell}\left(e^{(k-\ell)\varepsilon} - e^{(k-2i+\ell)\varepsilon}\right)}{(1+e^{\varepsilon})^k}, \tag{21.16}$$

which is the formula of Kairouz, Oh & Viswanath's main theorem.

Proof. The plan is to show that a composition, adaptive or not, is a post-processing of k independent four-point pairs, so that the data-processing inequality reduces the bound to the four-point product, and then to evaluate Lemma 21.1 on that product.

1. *Non-adaptive.* If step i is $P_i = P^\star K_i$ and $Q_i = Q^\star K_i$ by Theorem 21.2, then the product $P_1 \times \cdots \times P_k$ is $(P^\star)^k$ followed by the channel that applies K_i to coordinate i, independently. So $\delta(\varepsilon')$ of the composition is at most that of $(P^\star)^k$ against $(Q^\star)^k$, by the data-processing inequality.
2. *Adaptive.* Now the mechanism of step i, and so its channel $K_i^{y_{<i}}$, may depend on the earlier outputs. The trick is to draw all the four-point outputs z_i first and apply the channels afterwards. Draw $z_1, \dots, z_k$ from $(P^\star)^k$, then $y_1 \sim K_1(\cdot \mid z_1)$, then $y_2 \sim K_2^{y_1}(\cdot \mid z_2)$, and so on. Given $y_{<i}$, z_i is still a fresh draw from $P^\star$, because it was drawn independently of $z_{<i}$ and $y_{<i}$ depends only on those. So y_i has law $P^\star K_i^{y_{<i}} = P_i(\cdot \mid y_{<i})$, and the sequence has exactly the law of the adaptive run on D. The same sampler fed with $(Q^\star)^k$ produces the run on D', so the adaptive run is still one post-processing of $(P^\star)^k$ against $(Q^\star)^k$.
3. *The four-point product.* With probability $1-(1-\delta)^k$ some coordinate of $(P^\star)^k$ shows "flag D", which $(Q^\star)^k$ never produces, so there $L = +\infty$, the bracket $(1 - e^{\varepsilon' - L})_+$ is 1, and the output contributes its full probability to Equation 21.3. Outputs with "flag D'" have probability 0 under $(P^\star)^k$. Otherwise, with probability $(1-\delta)^k$, all coordinates are plain randomized response; the factor $(1-\delta)$ per coordinate cancels in the ratio, so $L = (k-2J)\varepsilon$ by Equation 21.5, and Lemma 21.1 on this part gives $(1-\delta)^k\delta_{\mathrm{RR}}(\varepsilon')$. Adding the two parts gives $1-(1-\delta)^k + (1-\delta)^k\delta_{\mathrm{RR}}(\varepsilon')$, which is Equation 21.15.
4. *Equality.* The four-point pair is itself a mechanism, on a dataset of one bit, and it is (ε, δ)-DP. Composing k independent copies of it gives equality, so no better bound can hold for the whole class.
5. *The grid formula.* At $\varepsilon' = (k-2i)\varepsilon$, the terms of δ_{RR} with $j \geq i$ vanish, because their loss $(k-2j)\varepsilon$ is at most ε' and the bracket is 0. For $j = \ell < i$ the bracket is positive and the positive part can be dropped. Since $t = e^\varepsilon/(1+e^\varepsilon)$ and $1-t = 1/(1+e^\varepsilon)$, the weight is $\binom{k}{\ell}t^{k-\ell}(1-t)^\ell = \binom{k}{\ell}e^{(k-\ell)\varepsilon}/(1+e^\varepsilon)^k$. Multiplying it by $e^{\varepsilon' - (k-2\ell)\varepsilon} = e^{(2\ell-2i)\varepsilon}$ turns $e^{(k-\ell)\varepsilon}$ into $e^{(k-2i+\ell)\varepsilon}$, which is Equation 21.16.

□

"Optimal" means optimal for the class: Equation 21.15 is the best you can say about k mechanisms *if all you know about each is its* (ε, δ). A Laplace mechanism with the same ε does a little better, because its loss often lies strictly inside $(-\varepsilon, \varepsilon)$: its mean loss at $\varepsilon = 1$ is 0.3679, against 0.4621 for randomized response.

A single (ε, δ) also throws information away. The privacy loss of a Gaussian step is normal (The Laplace and Gaussian mechanisms), so k steps with noise σ on a sensitivity-1 query have exactly the loss of one step with noise $\sigma/\sqrt{k}$. Figure 21.4 shows what a point summary costs: with $\sigma = 20$, $k = 1{,}000$ steps are exactly $(7.51, 10^{-5})$-DP, but the best per-step split of δ followed by optimal composition gives 62.7. Composing the curves themselves is the subject of Privacy accounting.

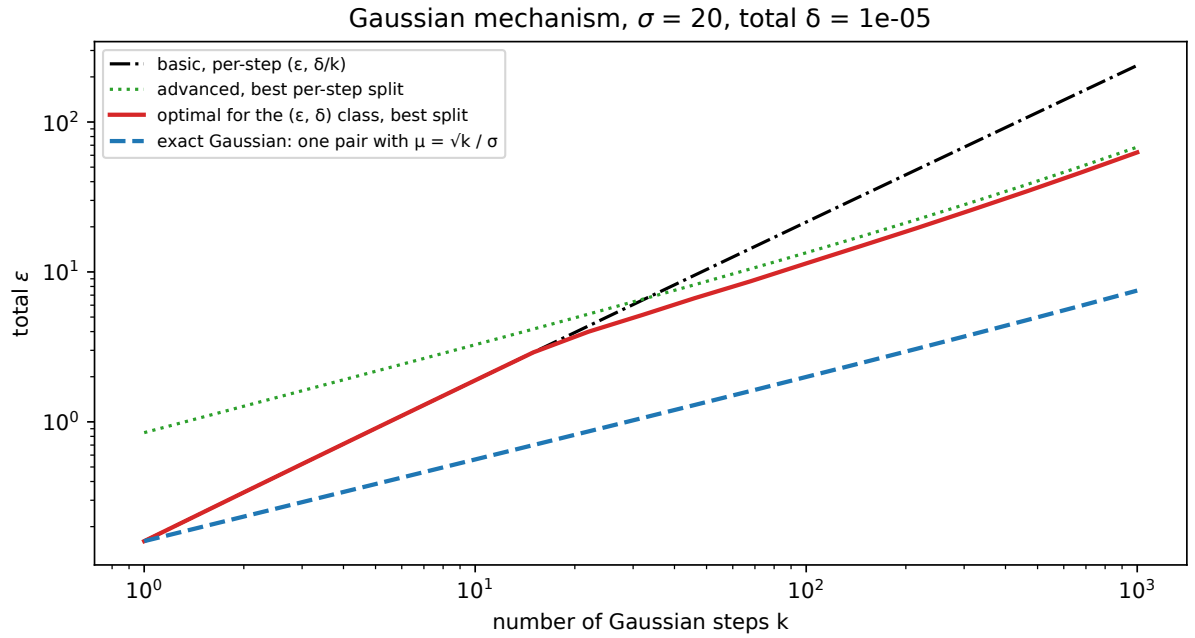


Figure 21.4.: Total ε at $\delta = 10^{-5}$ after k Gaussian steps with $\sigma = 20$ on a sensitivity-1 query. Three curves summarize each step by one (ε, δ) point and compose it with basic, advanced or optimal composition; the dashed curve is the exact composition, one Gaussian step with noise $\sigma/\sqrt{k}$.

The four-point product also extends advanced composition to steps with $\delta > 0$, and with the sharper drift: the flags cost their probability, and the rest is randomized response.

Corollary 21.1 (Advanced composition for (ε, δ)-DP steps, with the sharp drift)**.** *Fix k, $\varepsilon > 0$ and $\delta \geq 0$ in advance, and let $M_1, \ldots, M_k$ be mechanisms, each chosen after seeing the earlier outputs, such that every M_i is (ε, δ)-DP for every history. Then for every $\delta' > 0$ the composition is $(\varepsilon', k\delta + \delta')$-DP with ε' from Equation 21.9, and also with*

$$\varepsilon' = \varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon\tanh(\varepsilon/2). \tag{21.17}$$

Proof.

1. *Reduce to the four-point product.* By step 2 of the proof of Theorem 21.4, the adaptive run is a post-processing of k independent copies of the four-point pair, and a channel never increases $\delta(\varepsilon')$, so it suffices to bound the k-fold four-point product.
2. *The flags.* Some coordinate shows "flag D" with probability $1-(1-\delta)^k$, which is at most $k\delta$ because the chance that at least one of k events happens is at most the sum of their chances. Those outputs are impossible under D' and contribute their whole probability to $\delta(\varepsilon')$, so the flags cost at most $k\delta$.
3. *The rest.* Otherwise all k coordinates are plain randomized response at level ε, which are k independent ε-DP steps, so Theorem 21.1 bounds their contribution at the ε' of Equation 21.9 by δ'. Their drift is exactly $k\varepsilon\tanh(\varepsilon/2)$ by Proposition 21.1, so step 6 of that proof with this drift in place of $k\varepsilon(e^{\varepsilon}-1)$ gives the same δ' at the ε' of Equation 21.17. Adding the two parts gives $k\delta+\delta'$.

□

Example 21.4. For the dashboard's $k=100$ steps at $\varepsilon=0.1$, the optimal ε' at $\delta'=10^{-5}$ is 4.3068, and the sharper advanced bound of Equation 21.17 is $4.7985+0.4996=5.2981$. The optimal value is 19% below it ($4.3068/5.2981=0.81$), 26% below the original advanced bound ($4.3068/5.8502=0.74$) and 57% below basic composition's 10. Table 21.4 compares the four bounds over k; at $k=10$, below the crossover, both advanced bounds are worse than basic composition, while the optimal bound is slightly better, 0.994 against 1. If each step also has $\delta=5\times10^{-8}$, the flags use up $1-(1-\delta)^{100}=5.0\times10^{-6}$ of a total $\delta=10^{-5}$, and the optimal ε' rises to 4.4469; advanced composition with the remaining $\delta'=5\times10^{-6}$ gives 5.9926.

Table 21.4.: Total ε' at $\delta'=10^{-5}$ for k mechanisms that are each 0.1-DP.

k	Basic $k\varepsilon$	Advanced, Equation 21.9	Advanced, Equation 21.17	Optimal
10	1.000	1.623	1.567	0.994
100	10.000	5.850	5.298	4.307
1,000	100.000	25.691	20.170	17.787
10,000	1,000.000	153.156	97.944	91.691

Computing the bound. Equation 21.16 as printed divides two numbers that overflow: at $\varepsilon=1$ and $k=1{,}000$, $(1+e)^{1000}\approx10^{570}$, beyond the largest double. The chapter's script (Section 21.6) adds the non-negative terms of Equation 21.15 in log space with a log-sum-exp, so nothing overflows and nothing cancels. Between two grid points $\delta_{\mathrm{RR}}(\varepsilon')=A-e^{\varepsilon'}B$, where A and B are the masses under P and Q of the outputs with loss above ε'. Both are constant between grid points, so the optimal ε' for a target δ is $\ln((A-\delta)/B)$ on the right interval, exactly. With a different ε_i per step the loss depends on which steps lied, not just how many, and the exact bound is #P-complete to compute (Murtagh & Vadhan, 2016); for a few steps you can still enumerate.

21.5. Adaptive composition

In practice the next query depends on the last answer: an analyst zooms into the largest bin of a noisy histogram, and a training loop computes each gradient at parameters that came from

earlier noisy gradients. The proofs above were written to cover this; this section makes the model explicit, collects the guarantees in one statement, and says where adaptivity does break things.

Definition 21.3 (The k-fold adaptive composition experiment)**.** An adversary plays k rounds against a hidden bit $b \in \{0, 1\}$, fixed for the whole run (Dwork, Rothblum & Vadhan 2010). In round i it looks at the outputs $y_1, \dots, y_{i-1}$ so far and, as any function of them, chooses a pair of neighbouring datasets $D_i^0 \sim D_i^1$ and a mechanism M_i from a class of (ε, δ)-DP mechanisms; it then receives $y_i \sim M_i(D_i^b)$. Everything the adversary sees, its **view**, is the sequence $V = (y_1, \dots, y_k)$. A composition theorem holds for the experiment when, for every adversary, the distributions of the view under $b = 0$ and under $b = 1$ satisfy its guarantee in both directions.

The claim is that every bound of this chapter holds in the experiment.

Theorem 21.5 (Every composition bound holds under adaptive choice)**.** *In the experiment of Definition 21.3, with k, ε and δ fixed before the interaction starts, the views under $b = 0$ and $b = 1$ satisfy basic composition, $(k\varepsilon, k\delta)$-DP; advanced composition, $(\varepsilon', k\delta + \delta')$-DP with ε' from Equation 21.9 or Equation 21.17; and optimal composition, Equation 21.15.*

Proof. Every argument above looked at one step at a time, given the history so far, and once the history is fixed, so is the adversary's choice.

1. *The loss of the view is a sum.* Lemma 21.2 applies with each term $L_i(y_i \mid y_{<i})$ computed for the mechanism and the pair of datasets chosen after $y_{<i}$. Nothing in Lemma 21.1, Lemma 21.2 or Lemma 21.3 needs the pair to be the same in every round; each step only needs its own pair to be neighbours.
2. *Basic composition.* For $\delta = 0$, $|L_i| \le \varepsilon$ for every history, so $|L| \le k\varepsilon$ by Lemma 21.2. For $\delta > 0$, step 1 of the proof of Corollary 21.1 reduces the run to k independent four-point pairs: a flag shows with probability at most $k\delta$, and otherwise $|L| \le k\varepsilon$, so $\delta(k\varepsilon) \le k\delta$.
3. *Advanced composition.* Azuma's inequality needs only conditional statements, given the past: a conditional mean of zero and a range of fixed length. That is the setting of steps 2 to 5 of the proof of Theorem 21.1, and the $\delta > 0$ extension goes through the same reduction as step 2.
4. *Optimal composition.* Step 2 of the proof of Theorem 21.4 builds the adaptive run from the four-point product with history-dependent channels.

□

Adaptivity does not allow choosing the privacy parameters on the fly. If the analyst picks ε_i after seeing earlier outputs, the range length $c_i = 2\varepsilon_i$ in the proof of Theorem 21.1 depends on the history, and Azuma's inequality needs it fixed in advance. A rule such as "keep querying until the budget is spent" needs its own theorem, a **privacy filter** (Rogers, Roth, Ullman & Vadhan, 2016), and reporting the privacy spent so far at every moment needs a **privacy odometer**; both are treated in Adaptive composition. For pure DP with basic composition the naive rule is fine, because $|L| \le \sum_i \varepsilon_i$ holds pointwise whatever order the ε_i came in.

Nor does adaptivity allow choices that depend on the data rather than the outputs. A system that answers private counts *until the true count of some group crosses a threshold* reveals that count through the number of answers it gives, so every answer can be ε-DP and the whole run

still not be DP for any finite ε. A rule that stops early within a fixed k, looking only at the noisy answers, is covered, because the stopping time is then a post-processing of the outputs.

Example 21.5. The script simulates an analyst who, at each of the dashboard's $k = 100$ steps, chooses the next mechanism from the last output. If the previous output pointed to D, meaning its privacy loss was positive, the analyst asks a Laplace query at $\varepsilon = 0.1$; otherwise a randomized-response query at $\varepsilon = 0.1$. Over 100,000 simulated runs on D, Table 21.5 estimates $\delta(2)$ through Equation 21.3, by averaging $(1 - e^{2-L})_+$ over the simulated total losses L. The exact Laplace mean is 100 times the per-step KL divergence of two Laplace densities whose centres are ε scale units apart, $e^{-\varepsilon} + \varepsilon - 1$.

Table 21.5.: The privacy loss of 100 steps of 0.1-DP mechanisms, composed non-adaptively and by an adaptive analyst.

Composition of 100 steps at $\varepsilon = 0.1$	Mean total loss	$\delta(\varepsilon' = 2)$
Randomized response, exact (Equation 21.15)	0.4996	0.0201
Randomized response, simulated	—	0.0198
Laplace, simulated	0.4838 (exact 0.4837)	0.0189
Adaptive mix, simulated	0.4971	0.0196

The adaptive analyst's $\delta(2)$ lies between the two non-adaptive compositions, and below the optimal bound, as Theorem 21.5 requires. Figure 21.5 shows the whole curves: the three compositions nearly coincide, and the advanced-composition points lie far to the right of them.

21.6. Code

The script code/composition.py computes every number in this chapter and draws every figure. It checks the log-space optimal bound against brute-force enumeration, in floating point and in exact fractions, and compares simulated Laplace and adaptive compositions with it. From the repository root:

```
uv run python code/part03-dp-foundations/composition.py
```

Table 21.6.: The script's output with the default seed.

Experiment	Result
One step, $\varepsilon = 0.1$	$t = 0.5250$; mean loss $\varepsilon \tanh(\varepsilon/2) = 0.004996$; bound $\varepsilon(e^{\varepsilon} - 1) = 0.010517$

Experiment	Result
One step, $\varepsilon = 1$	tanh form 0.4621; $\varepsilon(e^{\varepsilon} - 1) = 1.7183$; $\varepsilon^2/2 = 0.5$; Laplace mean loss 0.3679
Three-output pair	Largest loss 0.9163; channel rows $(0.65, 0.3, 0.05)$ and $(0.05, 0.3, 0.65)$; KL 0.2749 against 0.5493; $\delta(0)$ 0.3 against 0.5
100 rounds of randomized response, $\varepsilon = 0.1$	Mean 0.4996; standard deviation 0.9988; $k\varepsilon$ is 9.5 standard deviations above the mean; $t^k = 1.033 \times 10^{-28}$
100 steps, $\varepsilon = 0.1$, $\delta' = 10^{-5}$	Basic 10; advanced 5.8502; advanced (tanh) 5.2981; optimal 4.3068
Exact δ at the advanced ε'	1.956×10^{-9} at 5.8502; 6.249×10^{-8} at 5.2981
Crossover	Advanced beats basic from $k = 29$
Brute-force checks of Equation 21.15	4,096 sequences at $\varepsilon = 0.5$, $\delta = 0$, $k = 12$ and at $\varepsilon = 0.3$, $\delta = 10^{-3}$, $k = 6$; 1,024 at $\varepsilon = 1$, $\delta = 0.01$, $k = 5$: largest difference 3.9×10^{-16}
Exact fractions, $\varepsilon = \ln 3$, $k = 8$	$\delta(0) = 1759/2048$, error 2.2×10^{-16}
Overflow, $\varepsilon = 1$, $k = 1{,}000$	The grid formula raises `OverflowError`; log space gives $\delta(550) = 6.094 \times 10^{-4}$
$\varepsilon = 0.01$, $k = 100{,}000$, $\delta' = 10^{-6}$	Optimal 19.4228; advanced (tanh) 21.6225, 11% above
Gaussian, $\sigma = 5$, $k = 100$, $\delta = 10^{-5}$	Basic 93.178; advanced 89.843; optimal 76.223; exact 9.997
Gaussian, $\sigma = 20$, $k = 1{,}000$, $\delta = 10^{-5}$	Basic 238.578; advanced 68.008; optimal 62.720; exact 7.511
Dense counts, $\varepsilon = 1$, $\delta = 10^{-5}$	Table 21.2; per-count ε_0 at $d = 1000$: 0.00645 (advanced), 0.00849 (optimal)

tests/test_composition.py checks that the output still matches.

21.7. Summary

Table 21.7.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 21.1	$L = \ln \frac{p(Y)}{q(Y)}$ with $Y \sim P$	The object every accountant of Privacy accounting composes
Lemma 21.1	$\delta(\varepsilon) = \mathbb{E}[(1 - e^{\varepsilon - L})_+] \leq \Pr[L > \varepsilon]$	The exact δ from the loss distribution; the tail bound is the cheap version
Lemma 21.2	$L = \sum_i L_i(y_i \mid y_{<i})$	Any sequence, adaptive or not; $\lvert L \rvert \leq k\varepsilon$ is basic composition

Result	Statement	Use
Proposition 21.1	$L = (k - 2J)\varepsilon$, $J \sim \mathrm{Bin}(k, 1-t)$	Mean about $k\varepsilon^2/2$, spread about $\sqrt{k}\,\varepsilon$; the worst case $k\varepsilon$ has probability t^k
Lemma 21.3	$\mathbb{E}[L] \le \varepsilon(e^\varepsilon - 1)$ for an ε-DP pair	The drift of advanced composition; a factor 2 loose for small ε
Theorem 21.1	$\varepsilon' = \varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon(e^\varepsilon - 1)$ at slack δ'	Beats basic composition from $k = 29$ at $\varepsilon = 0.1$, $\delta' = 10^{-5}$; budget $\varepsilon_{\text{total}}/\sqrt{2k\ln(1/\delta')}$ per step
Theorem 21.2	Every (ε, δ)-DP pair is a post-processing of the four-point pair of Table 21.3	Reduces any question about the class to randomized response
Theorem 21.3	$\mathbb{E}[L] \le \varepsilon\tanh(\varepsilon/2) \le \min\{\varepsilon^2/2,\ \varepsilon(e^\varepsilon - 1)\}$	The sharp drift; the mean loss of Concentrated DP
Theorem 21.4	$\delta(\varepsilon') \le 1-(1-\delta)^k(1-\delta_{\mathrm{RR}}(\varepsilon'))$, tight for the class	Knowing only each (ε, δ), compute it exactly in log space; knowing the mechanism, compose its whole profile, since the point summary can be off by a factor of 8 (62.7 against 7.51)
Corollary 21.1	$(\varepsilon', k\delta + \delta')$-DP with Equation 21.9 or Equation 21.17	The per-step δ is spent outright; composed Laplace steps cost like $\sqrt{d}$, and the $\sqrt{d}$ comes from δ
Theorem 21.5	Every bound holds when each mechanism is chosen from the earlier outputs, with k, ε, δ fixed in advance	Adaptive privacy parameters, or stopping rules that read the raw data, need the filters of Adaptive composition

21.8. References

- Dwork, Rothblum & Vadhan, "Boosting and Differential Privacy" (FOCS 2010). Introduces the adaptive composition experiment and advanced composition, Equation 21.9, with the bound $\varepsilon(e^\varepsilon - 1)$ on the mean loss.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.5: basic and advanced composition with full proofs (Theorem 3.20), including the $\delta > 0$ case through a decomposition of approximate DP pairs.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015; IEEE Transactions on Information Theory, 2017). The hypothesis-testing view, the

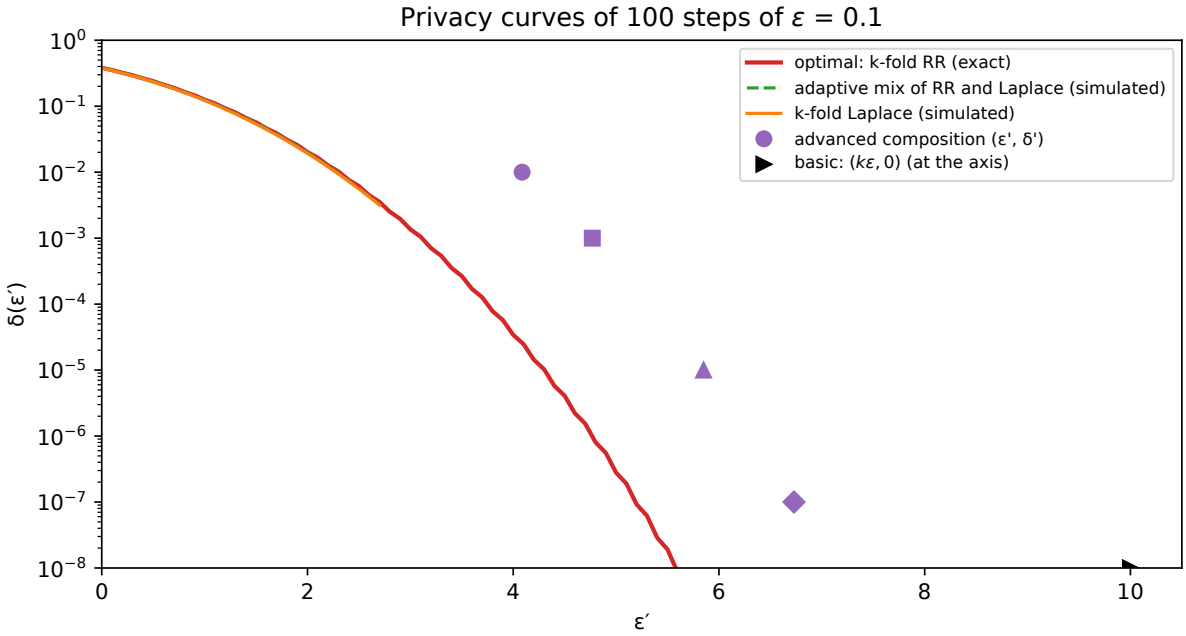


Figure 21.5.: The privacy profile $\delta(\varepsilon')$ of 100 steps at $\varepsilon = 0.1$: exact for composed randomized response, simulated for composed Laplace mechanisms and for the adaptive analyst, with the advanced-composition pairs for $\delta' = 10^{-2}, 10^{-3}, 10^{-5}, 10^{-7}$ and basic composition at $(10, 0)$.

four-point randomized response, the optimal composition formula Equation 21.16, and improved closed-form bounds of the advanced kind.

- Blackwell, "Equivalent Comparisons of Experiments" (Annals of Mathematical Statistics, 1953). The theorem that one experiment is uniformly harder to test than another exactly when it is a post-processing of it.
- Murtagh & Vadhan, "The Complexity of Computing the Optimal Composition of Differential Privacy" (TCC 2016). Optimal composition with different ε_i is #P-complete, with an efficient approximation.
- Vadhan, "The Complexity of Differential Privacy" (2017): composition theorems in the order used here, with the privacy loss as the central object.
- Rogers, Roth, Ullman & Vadhan, "Privacy Odometers and Filters: Pay-as-you-Go Composition" (NeurIPS 2016). Why adaptively chosen privacy parameters need new theorems, and the first filters and odometers.
- Dong, Roth & Su, "Gaussian Differential Privacy" (JRSS Series B, 2022). Composition of whole trade-off curves, and the central limit theorem for composition that the loss distribution of Section 21.1 foreshadows.

22. Selection: exponential mechanism, report-noisy-max, sparse vector

Many questions ask you to pick something rather than to compute a number, such as the best of 50 trained models or the first statistic to cross an alarm level. The answer is a name from a list, and adding noise to a name makes no sense. The Laplace mechanism of differential privacy (DP) could release a noisy score for every candidate. But if one record can move each of the $|\mathcal{R}|$ candidates' scores by 1, it moves the vector by $|\mathcal{R}|$ in the ℓ_1 norm, so the noise would grow with the number of candidates.

This chapter shows that you never have to pay for the losers. The one idea is to release the position, not the values. Each mechanism adds noise to scores that are never published and reports only which candidate came out on top. The privacy proof asks how far one record can move the bar that the chosen candidate must clear. That bar is a maximum over the other candidates, and a maximum of numbers that each move by at most some amount moves by at most that amount, however many there are. So the number of candidates never enters the privacy cost, and enters the accuracy only through its logarithm.

The **exponential mechanism** samples a candidate with probability exponential in its score. **Report-noisy-max** adds noise to every score and names the winner. The **sparse vector technique** reports the first query of a stream above a threshold. The running example is model selection. Fifty models are scored on a private validation set of 1,000 examples, and model r gets $900 - r$ right. A vote of six people checks the exponential mechanism by hand, and the sparse vector technique watches 1,000 counts for an alarm. Neighbours differ by adding or removing one record.

22.1. The exponential mechanism

You want to output a good candidate, where "good" is measured by a score that one record can move only a little. The natural private answer is a random candidate tilted toward high scores, and this section finds how much tilting a budget of ε allows.

A selection problem has a finite set $\mathcal{R}$ of candidates, fixed before the data are seen, and a **score function** $u(D, r)$ saying how good candidate r is on the dataset D. Its **sensitivity** is the largest change of any one score between two neighbours,

$$\Delta u = \max_{r \in \mathcal{R}} \max_{D \sim D'} \big| u(D, r) - u(D', r) \big|, \tag{22.1}$$

written Δ when the score is clear. McSherry and Talwar (2007) weighted each candidate exponentially in its score, at a rate that lets one record change every weight by at most $e^{\varepsilon/2}$. Section A.5 in the appendices derives the mechanism and its utility from scratch.

Definition 22.1 (The exponential mechanism)**.** For $\varepsilon > 0$, the **exponential mechanism** M_E outputs candidate r with probability proportional to $\exp\left(\varepsilon u(D, r)/(2\Delta)\right)$. The total weight $Z(D)$ is the **normalizer**:

$$\Pr[M_E(D) = r] = \frac{\exp\left(\frac{\varepsilon\, u(D,r)}{2\Delta}\right)}{Z(D)}, \qquad Z(D) = \sum_{s \in \mathcal{R}} \exp\Big(\frac{\varepsilon\, u(D, s)}{2\Delta}\Big). \tag{22.2}$$

The scores are **monotone** if, for every pair of neighbours, either every score goes up from D to D' or every score goes down.

Counts under add/remove are monotone, because adding a record can only raise each count. A candidate's probability is a weight divided by a total, and one record can change both, so the proof bounds each part. With monotone scores the two parts cannot both grow, which lets the 2 go.

Theorem 22.1 (Privacy of the exponential mechanism)**.** *The exponential mechanism is ε-DP. If the scores are monotone, the mechanism with weights $\exp\left(\varepsilon u(D, r)/\Delta\right)$, without the 2, is also ε-DP.*

Proof. The plan is to split the ratio of the probabilities of one candidate into its own weight and the normalizer, and bound each factor by sensitivity.

1. *Reduce to single outputs.* The output set is finite, so by Proposition 18.1 it is enough that every single output's probability changes by at most e^{ε} between neighbours. Fix $D \sim D'$ and a candidate r. Dividing the two probabilities of Equation 22.2 separates the two effects of one record:
$$\frac{\Pr[M_E(D) = r]}{\Pr[M_E(D') = r]} = \exp\Big(\frac{\varepsilon\big(u(D, r) - u(D', r)\big)}{2\Delta}\Big) \cdot \frac{Z(D')}{Z(D)}. \tag{22.3}$$
2. *The candidate's own weight.* By sensitivity, $u(D, r) - u(D', r) \le \Delta$, so the first factor is at most $e^{\varepsilon/2}$.
3. *The normalizer.* For every candidate s, sensitivity gives $u(D', s) \le u(D, s) + \Delta$, so each term of $Z(D')$ is at most $e^{\varepsilon/2}$ times the matching term of $Z(D)$. Adding the terms gives $Z(D') \le e^{\varepsilon/2} Z(D)$, and the product is at most e^{ε}.
4. *Monotone scores.* With weights $e^{\varepsilon u/\Delta}$, each factor alone could reach e^{ε}. If every score goes up from D to D', the first factor is at most 1 and the normalizer ratio at most e^{ε}, by step 3 with Δ for 2Δ. If every score goes down, the first factor is at most e^{ε} and the normalizer ratio at most 1. Either way the product is at most e^{ε}.

□

The 2 pays for two effects that can act together. One record can raise the score of r and lower its competitors', so r's weight grows while the normalizer shrinks. Without the 2, there are neighbours on which the ratio approaches $e^{2\varepsilon}$. The mechanism is a softmax of the scores with temperature $2\Delta/\varepsilon$, uniform as $\varepsilon \to 0$ and the argmax as $\varepsilon \to \infty$ (Figure 22.1).

The candidate set must not depend on the data, because the proof compares the same list on D and D'. It may be infinite, though. On a continuous range the sum becomes an integral against

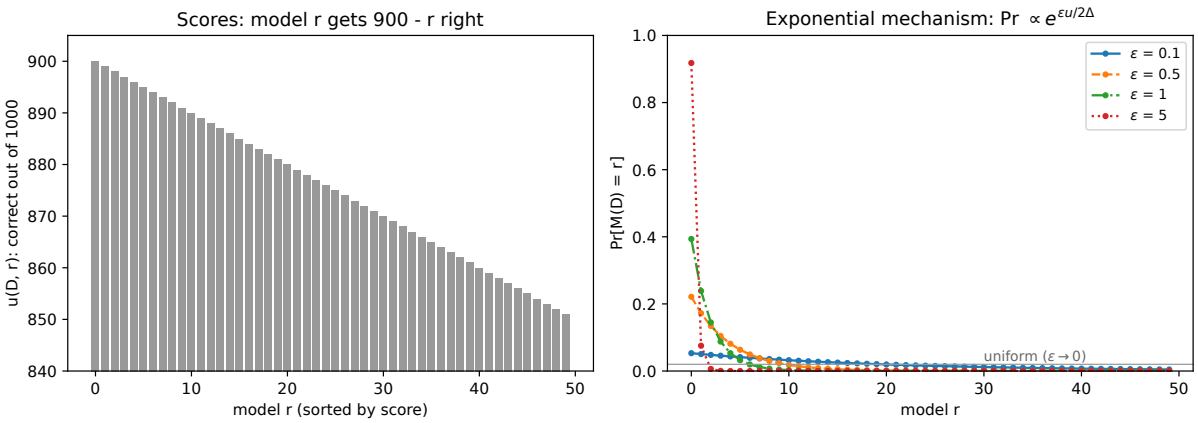


Figure 22.1.: Left: the scores of 50 models, model r getting $900 - r$ of 1,000 validation examples right. Right: the probability that the exponential mechanism selects each model, for four values of ε. Small ε spreads the probability toward uniform; large ε concentrates it on the best model.

a fixed base measure, such as length, and the two factors bound the ratio of densities at every point. Nothing else uses the structure of $\mathcal{R}$, so the candidates can be models, split points, strings or graphs.

Example 22.1. Six people vote among options A, B, C and X, with counts $(3, 2, 1, 0)$ on the dataset D. The neighbour D' has one more vote, for B, so $\Delta = 1$. With $\varepsilon = 1$ the weights are $e^{u/2}$, and Table 22.1 shows the output of the chapter's script (Section 22.8).

Table 22.1.: The exponential mechanism with $\varepsilon = 1$ on a vote and its neighbour with one extra vote for B.

Option	A	B	C	X
$\Pr[M_E(D) = r]$	0.4551	0.2760	0.1674	0.1015
$\Pr[M_E(D') = r]$	0.3859	0.3859	0.1420	0.0861
Ratio D/D'	1.1790	0.7151	1.1790	1.1790

The weights on D sum to $Z(D) = 9.8487$, and on D' the weight of B rises to $e^{1.5}$, so $Z(D') = 11.6121$. Only B's score changed, so for A, C and X the ratio is the normalizer ratio 1.1790, below its bound $e^{1/2} = 1.6487$. For B its own factor $e^{-1/2}$ multiplies that, giving 0.7151. The largest privacy loss is $|\ln 0.7151| = 0.3353$, a third of ε, because only one factor is at work for each option. Counts are monotone, so the weights e^{u} are also allowed, with largest privacy loss 0.6585.

22.2. Utility of the exponential mechanism

Privacy came for free in the number of candidates, but accuracy does not, because more candidates means more bad ones to pick by accident. This section bounds how far below the best score the

choice can fall. The idea is that a bad candidate weighs little next to the best one, whose weight is part of the normalizer.

Write $\mathrm{OPT}(D) = \max_r u(D, r)$ for the best score, and call the **gap** the difference between it and the score of the chosen candidate.

Theorem 22.2 (Utility of the exponential mechanism)**.** *For every dataset D and every $t > 0$,*

$$\Pr\left[u(D, M_E(D)) \le \mathrm{OPT}(D) - \frac{2\Delta}{\varepsilon}(\ln|\mathcal{R}| + t)\right] \le e^{-t}, \tag{22.4}$$

and the expected gap is at most $\frac{2\Delta}{\varepsilon}(\ln|\mathcal{R}| + 1)$. With monotone scores and weights $e^{\varepsilon u/\Delta}$, 2Δ can be replaced by Δ.

Proof. The plan is to bound the bad candidates' weight from above and the normalizer from below, solve for the gap, and integrate the tail. Write G for the gap. It is at least a exactly when the mechanism picks a candidate at least a below the best.

1. *Bad candidates are light.* The weight grows with the score, so each such candidate weighs at most $e^{\varepsilon(\mathrm{OPT}-a)/(2\Delta)}$.
2. *The normalizer is heavy.* It includes the weight of an optimal candidate, so $Z(D) \ge e^{\varepsilon\,\mathrm{OPT}/(2\Delta)}$.
3. *Divide.* There are at most $|\mathcal{R}|$ bad candidates, so dividing their total weight by the normalizer gives
$$\Pr[G \ge a] \le |\mathcal{R}|\, e^{-\varepsilon a/(2\Delta)}. \tag{22.5}$$
4. *Solve for the gap.* Setting the right side to e^{-t} and taking logarithms gives $a = \frac{2\Delta}{\varepsilon}(\ln|\mathcal{R}| + t)$, which is Equation 22.4.
5. *Expectation.* The mean of a non-negative quantity is the integral of its tail. Bound the tail by 1 up to $a_0 = \frac{2\Delta}{\varepsilon}\ln|\mathcal{R}|$, where step 3 reaches 1, and by step 3 beyond. The second piece is an exponential integral equal to $\frac{2\Delta}{\varepsilon}$, so $\mathbb{E}[G] \le a_0 + \frac{2\Delta}{\varepsilon}$. With monotone weights the same steps run with Δ for 2Δ.

□

Doubling the candidates adds only $\frac{2\Delta}{\varepsilon}\ln 2$ to the gap, and a lower bound shows that no ε-DP mechanism avoids the $\ln|\mathcal{R}|$. The proof counted every candidate as bad at the largest bad weight and kept one term of the normalizer. Both are nearly exact when one best candidate faces 49 runners-up all exactly a below it (Figure 22.2, right). At $\varepsilon = 1$ and $a = 20$ the bound is 0.0023 and the truth 0.0022.

When scores are spread out, most bad candidates sit far below the best and are much lighter than the bound assumes. Figure 22.2 (left) shows the bound orders of magnitude above the truth for the 50 models.

Example 22.2. Release the best of the 50 models. The score counts correct predictions, so one validation example changes each score by at most 1 and $\Delta = 1$. Table 22.2 shows the script's output.

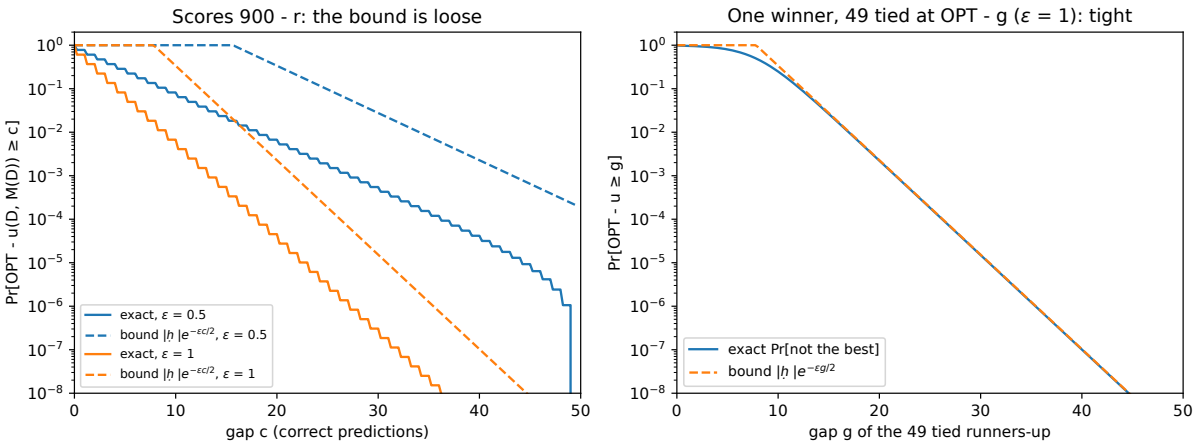


Figure 22.2.: Probability that the exponential mechanism's choice is at least a below the best score, exact against the bound $|\mathcal{R}|e^{-\varepsilon a/2\Delta}$. Left: the 50 models with scores $900 - r$, for $\varepsilon = 0.5$ and 1; the bound is loose because the scores are spread out. Right: one best model and 49 runners-up all the same distance below it, with $\varepsilon = 1$; the bound is tight.

Table 22.2.: The exponential mechanism selecting among 50 models with scores $900 - r$. The gap is in correct predictions out of 1,000.

ε	Pr[best]	$\mathbb{E}$[gap]	Bound $2(\ln 50 + 1)/\varepsilon$	$\mathbb{E}$[gap], monotone weights
0.1	0.0531	15.03	98.24	9.17
1	0.3935	1.54	9.82	0.58
5	0.9179	0.09	1.96	0.01

At $\varepsilon = 1$ you lose 1.54 correct predictions on average, although you pick the best model only 39% of the time, because the mechanism mostly picks a near-tie. The tail bound with $t = 3$ says the gap exceeds 13.82 with probability at most 0.0498, while the exact probability is 9.12×10^{-4}. The scores are monotone, since one more example can only add correct predictions, and the weights $e^{\varepsilon u}$ cut the expected gap at $\varepsilon = 1$ from 1.54 to 0.58.

22.3. Applications of the exponential mechanism

To use the exponential mechanism, you choose a fixed candidate set, a score and its sensitivity. This section makes those choices three times. The first recovers the Laplace mechanism and shows a second way to drop the 2. The second picks a classifier from a finite class, and the third finds a median over an interval, which private training in Part 5 uses to set its clipping norm.

22.3.1. The Laplace mechanism

Take a query f of sensitivity Δ, let every real number h be a candidate, and score it by $u(D,h) = -|f(D)-h|$. With length as the base measure, the density is proportional to $e^{-\varepsilon|f(D)-h|/(2\Delta)}$, a Laplace density of scale $2\Delta/\varepsilon$ centred at $f(D)$. Changing the data only slides it along the line, so the normalizer cannot change and cannot work against any candidate.

Proposition 22.1 (A data-independent normalizer allows the full rate)**.** *If $Z(D) = Z(D')$ for every pair of neighbours, the exponential mechanism with weights $e^{\varepsilon u(D,h)/\Delta}$ is ε-DP. For the score $-|f(D)-h|$ on the real line, the normalizer does not depend on the data, and the mechanism outputs $f(D) + \mathrm{Lap}(\Delta/\varepsilon)$.*

Proof.

1. *General claim.* When $Z(D) = Z(D')$, the ratio of the densities at h is the own factor of Theorem 22.1, now at most e^{ε}, and integrating bounds every event.
2. *Sensitivity.* By the triangle inequality, $\big||f(D)-h| - |f(D')-h|\big| \le |f(D)-f(D')| \le \Delta$.
3. *The normalizer.* Shifting the variable of integration by $f(D)$ removes the data:
$$Z(D) = \int_{-\infty}^{\infty} e^{-\varepsilon|f(D)-h|/\Delta}\,\mathrm{d}h = \int_{-\infty}^{\infty} e^{-\varepsilon|h|/\Delta}\,\mathrm{d}h = \frac{2\Delta}{\varepsilon}. \tag{22.6}$$
4. *It is the Laplace mechanism.* A density proportional to $e^{-\varepsilon|h-f(D)|/\Delta}$ is the $\mathrm{Lap}(\Delta/\varepsilon)$ density shifted to $f(D)$.

□

So monotone scores keep the two factors from both being large, and a data-independent normalizer makes the second one 1. A constant normalizer needs candidates that look the same after a shift and data that only shift the scores, which is why the Laplace mechanism needs only half the scale $2\Delta/\varepsilon$ of the exponential mechanism.

22.3.2. A private learner

A learning problem has a finite class $\mathcal{H}$ of classifiers and n labelled examples drawn independently from one unknown distribution. The **training error** $\mathrm{err}_D(h)$ is the fraction of training examples that h gets wrong, and the **population error** $\mathrm{err}(h)$ is its error probability on a fresh example. The learner of Kasiviswanathan, Lee, Nissim, Raskhodnikova and Smith (2008) is the exponential mechanism over $\mathcal{H}$ with score $-n\,\mathrm{err}_D(h)$, minus the number of mistakes. Its excess error over the best classifier has a sampling term, paid without privacy too, and a privacy term.

Theorem 22.3 (A private learner for a finite class)**.** *The learner is ε-DP. For a failure probability β, with probability at least $1-2\beta$ its choice $\hat{h}$ satisfies*

$$\mathrm{err}(\hat{h}) \le \min_{h\in\mathcal{H}} \mathrm{err}(h) + 2\sqrt{\frac{\ln(2|\mathcal{H}|/\beta)}{2n}} + \frac{2\ln(|\mathcal{H}|/\beta)}{\varepsilon n}. \tag{22.7}$$

Proof. The plan is to get privacy from Theorem 22.1, then chain two bounds: the choice is nearly the best on the training set, and training errors are close to population errors.

1. *Privacy.* One example changes every classifier's number of mistakes by at most 1, so $\Delta = 1$. This holds for any example, so privacy does not assume independence.
2. *Nearly the best on the training set.* Equation 22.4 with $t = \ln(1/\beta)$, divided by n, gives $\operatorname{err}_D(\hat{h}) \le \min_h \operatorname{err}_D(h) + c_E$ with $c_E = 2\ln(|\mathcal{H}|/\beta)/(\varepsilon n)$, with probability at least $1 - \beta$.
3. *Close to the population.* A training error is an average of n independent indicators, so Hoeffding's inequality (Concentration inequalities) bounds the chance that it strays by more than s by $2e^{-2ns^2}$. A union bound over the class and setting the total to β give $s = \sqrt{\ln(2|\mathcal{H}|/\beta)/(2n)}$ for all classifiers at once.
4. *Chain.* Let $h^\star$ have the smallest population error. On both events, step 3 moves to the training set, step 2 compares with $h^\star$, and step 3 moves back:

$$\operatorname{err}(\hat{h}) \le \operatorname{err}_D(\hat{h}) + s \le \operatorname{err}_D(h^\star) + c_E + s \le \operatorname{err}(h^\star) + c_E + 2s. \tag{22.8}$$

□

The sampling term falls like $1/\sqrt{n}$ and the privacy term like $1/n$, so for fine accuracy privacy costs little. The class must be finite. Alon, Livni, Malliaris and Moran (2019) showed that thresholds at every real number cannot be learned privately in the probably approximately correct (PAC) sense.

22.3.3. A private median

Take records $x_1, \dots, x_n$ in an interval $[0, B]$ fixed before looking at the data, and score every point r by $-\big|\#\{i : x_i < r\} - \#\{i : x_i > r\}\big|$. The absolute value is the **imbalance** of r. Here the 2 stays, because a new record raises the scores on one side of the median and lowers them on the other. Assume no two records are equal, and write $x_{(1)} < \cdots < x_{(n)}$ for them in increasing order. They cut the interval into $n + 1$ **pieces**, with $x_{(0)} = 0$ and $x_{(n+1)} = B$ as end points. Write ℓ_j for the length of the piece from $x_{(j)}$ to $x_{(j+1)}$, and $\ell^\star$ for the total length of the best pieces.

Proposition 22.2 (The private median)**.** *Under add/remove the score has sensitivity 1, and the exponential mechanism with length as base measure is ε-DP. It picks piece j with probability proportional to $\ell_j e^{-\varepsilon|2j-n|/2}$, then a uniform point inside it. For even n and any $t > 0$, with probability at least $1 - e^{-t}$ its output r satisfies*

$$\big|\#\{i : x_i < r\} - \#\{i : x_i > r\}\big| \le \frac{2}{\varepsilon}\Big(\ln\frac{B}{\ell^\star} + t\Big). \tag{22.9}$$

For odd n the right side gains 1, because no point splits the data evenly.

Proof.

1. *Sensitivity.* One record changes the count below r or the count above it by 1, or neither, so the imbalance moves by at most 1. Theorem 22.1 gives privacy.
2. *Sampling.* Inside piece j, exactly j records lie below r and $n - j$ above, so the score is $-|2j - n|$ and the density is flat. The piece's probability is its length times its weight, over Z.

3. *Utility.* Repeat Theorem 22.2 with lengths for counts. The points at least a below the best have total length at most B, so they weigh at most $Be^{\varepsilon(\mathrm{OPT}-a)/2}$, while the best pieces weigh $\ell^\star e^{\varepsilon\,\mathrm{OPT}/2}$. Their ratio $(B/\ell^\star)e^{-\varepsilon a/2}$, set to e^{-t}, bounds the imbalance by the best imbalance plus $\frac{2}{\varepsilon}(\ln(B/\ell^\star)+t)$. The best imbalance is 0 for even n and 1 for odd n.

□

The range enters only through $\ln(B/\ell^\star)$, so a generous range is cheap, but it must be fixed without looking at the data, as a clamping range must be in Sensitivity. The guarantee controls the output's rank, not its value. For the quantile q, the score $-\big|(1-q)\#\{x_i < r\} - q\,\#\{x_i > r\}\big|$ works the same way.

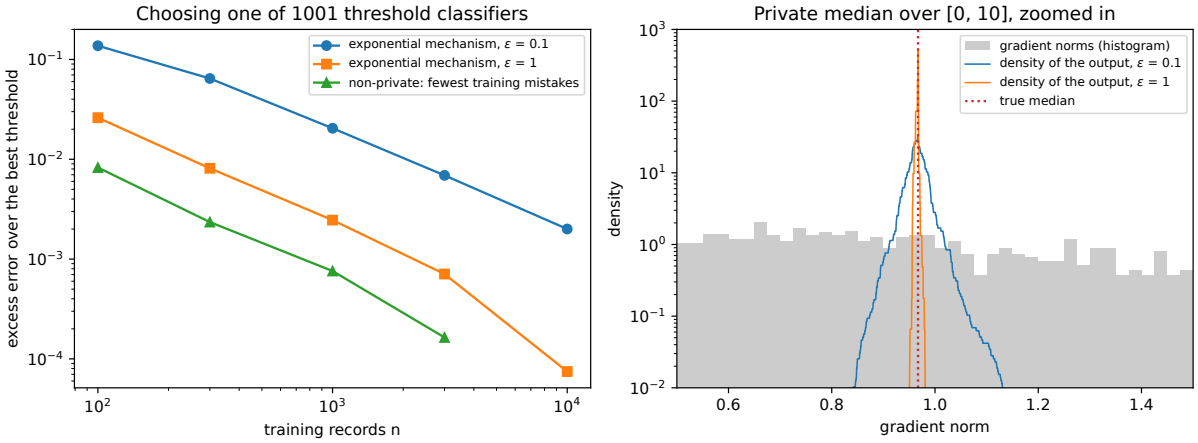


Figure 22.3.: Left: excess population error of the learner that picks one of 1,001 threshold classifiers with the exponential mechanism, against the non-private learner that picks the fewest training mistakes, averaged over 100 datasets per point (the text's numbers use 200); the non-private learner finds the best threshold exactly at $n = 10{,}000$, which a log axis cannot show. Right: 1,000 lognormal gradient norms (histogram) and the density of the private median over $[0, 10]$ for $\varepsilon = 0.1$ and $\varepsilon = 1$, zoomed in around the true median.

Example 22.3. *A threshold classifier.* The data are x uniform on $[0, 1]$, labelled 1 when $x \geq 0.3$, with each label flipped with probability 0.1. The class holds the 1,001 thresholds $0, 0.001, \ldots, 1$. Over 200 datasets of $n = 1{,}000$, the private learner's average excess error is 0.0024 at $\varepsilon = 1$ and 0.0206 at $\varepsilon = 0.1$, against 0.0008 without privacy (Figure 22.3, left). At $\beta = 0.05$, Equation 22.7 has a sampling term of 0.1456 and a privacy term of 0.0198, both far above the observed errors, because the union bound guards against all 1,001 classifiers at once.

A clipping norm. Take 1,000 gradient norms drawn from a lognormal distribution with median 1, capped at $B = 10$. Their median is 0.9804. At $\varepsilon = 1$ the private median is on average 0.0013 from it, and 0.0157 at $\varepsilon = 0.1$ (Figure 22.3, right). With $t = \ln 20$, Equation 22.9 allows an imbalance of 26.4 at $\varepsilon = 1$ and 264.5 at $\varepsilon = 0.1$.

22.4. Report-noisy-max

The obvious way to select privately is to add independent noise to every score and report only the index of the largest. This section proves that counts then need noise of scale $1/\varepsilon$, for any number of candidates, with the bar argument of the opening. Its three tools, used again for the sparse vector technique, come first as lemmas.

The proof looks at one candidate i and freezes the noise of all the others. The first lemma says that freezing is allowed when the frozen noise is drawn the same way on both datasets.

Lemma 22.1 (Fix the data-independent randomness)**.** *Let W be randomness inside a mechanism whose distribution does not depend on the data, and A an event. If $\Pr[A \mid D, W = w] \le e^{\varepsilon} \Pr[A \mid D', W = w]$ for every w, then $\Pr[A \mid D] \le e^{\varepsilon} \Pr[A \mid D']$.*

Proof. Both probabilities average the conditional ones (a sum, or an integral for continuous noise) with the same weights $\Pr[W = w]$, because W does not depend on the data. Bounding each term of the first average by e^{ε} times the matching term of the second bounds the averages. □

Once the other noise is frozen, candidate i wins when its own noise clears a bar built from a maximum over the others. The second lemma bounds how far one record moves that maximum.

Lemma 22.2 (A maximum moves no more than its terms)**.** *If $|v'_j - v_j| \le \Delta$ for every j, then $|\max_j v'_j - \max_j v_j| \le \Delta$, for any number of terms. If every $v'_j - v_j$ lies in $[0, \Delta]$, so does $\max_j v'_j - \max_j v_j$.*

Proof. Raising every term to its highest value gives $\max_j v'_j \le \max_j v_j + \Delta$, and swapping v and v' gives the other side. When every term moves up, the maximum cannot fall. □

The third lemma pays for the move. The Laplace density falls by $e^{-1/b}$ per unit of distance from 0, so sliding it by s changes its height anywhere by at most $e^{|s|/b}$.

Lemma 22.3 (Moving a Laplace threshold)**.** *For $Z \sim \mathrm{Lap}(b)$ and all real t and s,*

$$\Pr[Z > t - s] \le e^{|s|/b} \Pr[Z > t]. \tag{22.10}$$

Proof.

1. *Compare densities.* With $p(z) = \frac{1}{2b} e^{-|z|/b}$, the ratio $p(z-s)/p(z)$ is $e^{(|z|-|z-s|)/b}$. Two points s apart differ in distance from 0 by at most $|s|$, so $p(z-s) \le e^{|s|/b} p(z)$.
2. *Change variables.* Substituting $z \mapsto z - s$ writes $\Pr[Z > t - s]$ as $\int_t^{\infty} p(z-s)\,\mathrm{d}z$, which step 1 bounds by $e^{|s|/b} \Pr[Z > t]$.

□

With $b = s = 1$, the ratio of the two tails is 1.6321 at $t = 0$ and exactly $e = 2.7183$ at $t = 2$, where t and $t - s$ both lie above 0 and each tail is $\frac{1}{2} e^{-x/b}$.

Theorem 22.4 (Report-noisy-max)**.** ***Report-noisy-max*** *adds independent* $Z_r \sim \mathrm{Lap}(b)$ *to every score and outputs* $M_N(D) = \arg\max_r (u(D,r) + Z_r)$*. With* $b = 2\Delta/\varepsilon$ *it is* ε*-DP, and with monotone scores* $b = \Delta/\varepsilon$ *suffices (Dwork and Roth, Claim 3.9). With probability at least* $1-\beta$*, its gap is at most* $2b\ln(|\mathcal{R}|/\beta)$*.*

Proof. The plan is to freeze the other noise, write the output as one Laplace noise clearing a bar, bound how far the bar moves, and pay for the move.

1. *Freeze the others.* Fix $D \sim D'$, a candidate i and the noise z_{-i} of the others. By Lemma 22.1 it is enough to bound the conditional probability that i wins.
2. *A bar.* Candidate i wins on D exactly when $Z_i > t(D)$, with
$$t(D) = \max_{j \neq i} (u(D,j) + z_j) - u(D,i). \tag{22.11}$$
3. *The bar moves by at most* 2Δ. Each term $u(\cdot, j) + z_j$ moves by at most Δ, so the maximum does too by Lemma 22.2, however many terms it has. The score $u(\cdot, i)$ adds another Δ.
4. *Monotone scores.* If every score goes up by an amount in $[0, \Delta]$, then by Lemma 22.2 so does the maximum, and their difference lies in $[-\Delta, \Delta]$. The same holds when every score goes down.
5. *Pay.* Lemma 22.3 with $s = t(D) - t(D')$ gives a factor at most $e^{2\Delta/b} = e^{\varepsilon}$ in general and $e^{\Delta/b} = e^{\varepsilon}$ for monotone scores. Lemma 22.1 removes the conditioning, and Proposition 18.1 gives ε-DP.
6. *Accuracy.* By the accuracy bound of Theorem 20.1, a union bound over the candidates' Laplace tails, every $|Z_r|$ is below $x = b\ln(|\mathcal{R}|/\beta)$ with probability at least $1-\beta$. Then the winner's noisy score beat that of a best candidate, so $u(D,i) + x > \mathrm{OPT} - x$, and the gap is below $2x$.

□

Figure 22.4 draws steps 2 to 5 with $b = 1$ and a bar lowered from 0.7 by 1. The winning probability rises from 0.2483 to 0.6296, a ratio of 2.5357, below e because the lowered bar left the tail where Lemma 22.3 is exact.

The losers enter only through one maximum in step 3, which is why they cost nothing. The 2 is needed in general because competitors can rise while i falls. A histogram under replace neighbours, where one record is swapped for another, does exactly that, so it is not monotone.

Release only the index, because the winning noisy score would publish Z_i, the noise the proof averages over. Neither Laplace report-noisy-max nor the exponential mechanism always selects better.

Example 22.4. On the vote of Table 22.1 the counts are monotone, so $b = 1$ is allowed at $\varepsilon = 1$. The script's exact winning probabilities give a largest privacy loss of 0.6674, and 0.3626 with $b = 2$. On the 50 models at $\varepsilon = 1$, the general mechanism ($b = 2$) picks the best with probability 0.4241, against 0.3935 for the exponential mechanism, and has expected gap 1.40 against 1.54. The monotone one ($b = 1$) has expected gap 0.54. The accuracy bound at $\beta = 0.05$, 27.63 for $b = 2$, is loose for spread-out scores.

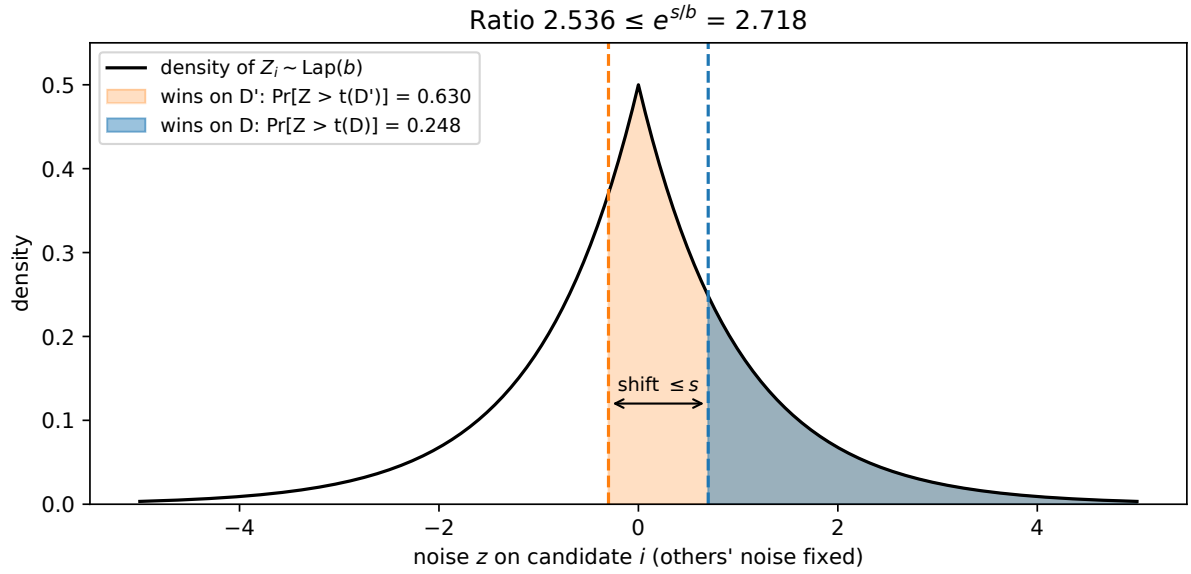


Figure 22.4.: The proof of report-noisy-max in one picture. With the other candidates' noise fixed, candidate i wins when its own noise $Z_i \sim \mathrm{Lap}(1)$ exceeds a bar t. Moving to a neighbouring dataset moves the bar by at most s, and the winning probability changes by at most a factor $e^{s/b}$.

22.5. Gumbel noise: report-noisy-max is the exponential mechanism

Report-noisy-max with Laplace noise is close to the exponential mechanism but not equal to it. With Gumbel noise the noisy argmax gives exactly the probabilities of Equation 22.2, which is the **Gumbel-max trick**. The **Gumbel distribution** with scale κ, Gumbel(κ), has cumulative distribution function (CDF) $\Pr[G \le g] = \exp(-e^{-g/\kappa})$, and NumPy's `rng.gumbel(0, kappa)` samples it.

Theorem 22.5 (The Gumbel-max trick)**.** *For scores u_r and independent $G_r \sim \mathrm{Gumbel}(\kappa)$,*

$$\Pr\left[\arg\max_r \left(u_r + G_r\right) = i\right] = \frac{e^{u_i/\kappa}}{\sum_r e^{u_r/\kappa}}. \tag{22.12}$$

With $\kappa = 2\Delta/\varepsilon$ this is the exponential mechanism.

Proof. Conditioning on the winner's noise lets you multiply the chances that each other candidate falls short, and the integral can then be done exactly. Dividing everything by κ keeps the argmax, so take $\kappa = 1$, with CDF $H(g) = \exp(-e^{-g})$ and density $H'(g) = e^{-g}H(g)$.

1. *Condition.* If $G_i = g$, candidate i wins when every other $G_j < g + u_i - u_j$. These events are independent, so
$$\Pr[i \text{ wins}] = \int_{-\infty}^{\infty} H'(g) \prod_{j \neq i} H(g + u_i - u_j)\,\mathrm{d}g. \tag{22.13}$$

2. *Collect.* Each factor is $\exp(-e^{-g}e^{u_j-u_i})$, and the $H(g)$ inside $H'(g)$ is the factor for $j=i$. The product is $\exp(-e^{-g}A)$, with $A = e^{-u_i}\sum_j e^{u_j}$.
3. *Substitute* $w = e^{-g}$. The factor e^{-g} and $\mathrm{d}g$ together are $-\mathrm{d}w$, and the minus sign flips the limits, so

$$\Pr[i \text{ wins}] = \int_0^\infty e^{-wA}\,\mathrm{d}w = \frac{1}{A} = \frac{e^{u_i}}{\sum_j e^{u_j}}. \tag{22.14}$$

□

So the Gumbel noisy max inherits the privacy of Theorem 22.1, and Section J.7 proves it directly too, because the Gumbel tail obeys the bound of Lemma 22.3 and the bar argument of Theorem 22.4 then applies. It works in score units, while $e^{\varepsilon u/(2\Delta)}$ overflows for scores in the thousands. Taking the c largest of $u_r + G_r$ at once has the distribution of c exponential mechanisms run in turn without replacement, at $c\varepsilon$ (Durfee and Rogers 2019; Section J.6 derives it).

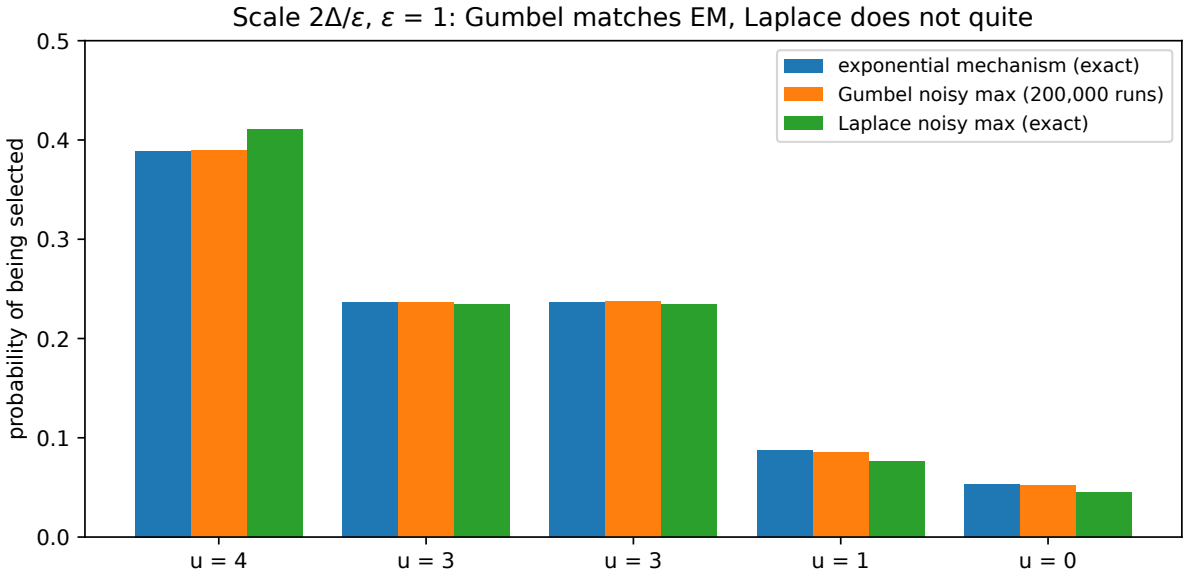


Figure 22.5.: Selection probabilities for five candidates with scores $(4, 3, 3, 1, 0)$ and noise scale $2\Delta/\varepsilon$ with $\varepsilon = 1$: the exponential mechanism's exact probabilities, the frequencies of the Gumbel noisy max in 200,000 runs, and the exact probabilities of the Laplace noisy max.

Example 22.5. Take five candidates with scores $(4, 3, 3, 1, 0)$ and noise scale 2, which is $\varepsilon = 1$ and $\Delta = 1$. Table 22.3 and Figure 22.5 compare the three mechanisms.

Table 22.3.: Selection probabilities with noise scale 2 for five candidates.

Candidate score	4	3	3	1	0
Exponential mechanism, exact	0.3889	0.2359	0.2359	0.0868	0.0526
Gumbel noisy max, 10^6 runs	0.3891	0.2351	0.2362	0.0871	0.0525
Laplace noisy max, exact	0.4105	0.2341	0.2341	0.0764	0.0449

The Gumbel frequencies are within 0.0008 of the exponential mechanism, about the sampling error of a million draws. The Laplace noisy max is at total variation distance 0.0216, the extra probability it puts on the top candidate.

22.6. The sparse vector technique

Now the queries arrive one at a time, and you care only about the first one above a threshold. Answering each with the Laplace mechanism would cost ε per query. Report-noisy-max needs the whole list in advance, but here each query must be answered before the next is seen. The **sparse vector technique** (Dwork, Naor, Reingold, Rothblum and Vadhan 2009) compares each query with one noisy threshold and pays only for the one above it. Its basic form is Algorithm 1 of Dwork and Roth (2014).

Definition 22.2 (AboveThreshold)**.** The input is a dataset D, a stream of k queries f_i of sensitivity Δ, possibly adaptive, a threshold τ and budgets $\varepsilon_1, \varepsilon_2 > 0$. Draw $\rho \sim \mathrm{Lap}(\Delta/\varepsilon_1)$ once. For each query, draw fresh $\nu_i \sim \mathrm{Lap}(2\Delta/\varepsilon_2)$. If $f_i(D) + \nu_i \geq \tau + \rho$, output $\top$ (“above”) and halt; otherwise output $\bot$ (“below”). The output $\bot^{j-1}\top$ stops at query j.

The output “stop at query j” says that the noisy threshold is above every earlier noisy answer and at or below the j-th. The proof slides ρ by however much the largest earlier answer moved, which costs at most Δ by Lemma 22.2, and slides ν_j to keep query j in place, which costs at most 2Δ.

Theorem 22.6 (AboveThreshold)**.** *AboveThreshold is $(\varepsilon_1 + \varepsilon_2)$-DP for any number of queries (Dwork and Roth, Theorem 3.23; Lyu, Su and Li 2017). For monotone queries, which all move the same way between neighbours as counts under add/remove do, $\nu_i \sim \mathrm{Lap}(\Delta/\varepsilon_2)$ suffices. With $\varepsilon_1 = \varepsilon_2 = \varepsilon/2$ and k fixed before the run, with probability at least $1 - \beta$ every $\bot$ query has $f_i(D) < \tau + \alpha$ and the $\top$ query has $f_j(D) > \tau - \alpha$, where*

$$\alpha = \frac{8\Delta(\ln k + \ln(2/\beta))}{\varepsilon}. \tag{22.15}$$

Proof. The plan is to freeze the earlier noise, write the output as two conditions on (ρ, ν_j), shift both noises to turn the conditions for D into those for D', and pay with the Laplace density; accuracy then comes from Laplace tails. Fix $D \sim D'$ and write $g(D) = \max_{i<j} (f_i(D) + \nu_i)$.

1. *Freeze.* Fix $\nu_1, \dots, \nu_{j-1}$, which do not depend on the data, and use Lemma 22.1. Adaptive queries are then fixed too, since they depend only on earlier answers, all $\perp$.
2. *Two conditions.* The earlier queries say $\perp$ exactly when $g(D) < \tau + \rho$, and query j says $\top$ when $f_j(D) + \nu_j \geq \tau + \rho$. With noise densities p_ρ and p_ν, the output has probability
$$\iint p_\rho(\rho)\, p_\nu(\nu)\, \mathbf{1}[g(D) < \tau + \rho]\, \mathbf{1}[f_j(D) + \nu \geq \tau + \rho]\, \mathrm{d}\rho\, \mathrm{d}\nu. \tag{22.16}$$
3. *Shift both noises.* Substitute $\rho = \rho' + g(D) - g(D')$ and $\nu = \nu' + (g(D) - g(D')) - (f_j(D) - f_j(D'))$. After cancelling, the indicators are those for D', and a translation leaves $\mathrm{d}\rho\, \mathrm{d}\nu$ unchanged, so only the densities are evaluated at shifted points.
4. *Size of the shifts.* Each term of g moves by at most Δ, so $|g(D) - g(D')| \leq \Delta$ by Lemma 22.2, whatever j is. The shift of ν is a difference of two such terms, at most 2Δ.
5. *Pay.* By step 1 of the proof of Lemma 22.3, a Laplace density of scale b changes by at most $e^{|x|/b}$ when its argument moves by x. That is e^{ε_1} for ρ and e^{ε_2} for ν. The output $\perp^k$ has only the first indicator and costs e^{ε_1}. Lemma 22.1 and Proposition 18.1 conclude.
6. *Monotone queries.* If all queries move the same way, g and f_j move the same way by Lemma 22.2, so the shift of ν is at most Δ.
7. *Accuracy.* With the equal split, the Laplace tail of Theorem 20.1 gives $\Pr[|\rho| \geq \alpha/2] = e^{-\alpha\varepsilon/(4\Delta)}$, and a union bound over the k query noises gives $k\, e^{-\alpha\varepsilon/(8\Delta)}$. Setting the second to $\beta/2$ gives Equation 22.15, and the first is then smaller. The union bound needs k fixed in advance, because the index where the run stops depends on the noise.
8. *Read off.* If all noises are below $\alpha/2$, a $\perp$ means $f_i < \tau + \rho - \nu_i < \tau + \alpha$, and the $\top$ means $f_j \geq \tau + \rho - \nu_j > \tau - \alpha$.

□

To find c queries above the threshold, the algorithm Sparse of Dwork and Roth restarts AboveThreshold with fresh threshold noise after each $\top$ and halts after the c-th. Each run gets a share of the budget and of the failure probability.

Corollary 22.1 (Sparse: c answers above)**.** *With c runs at budget ε/c each, Sparse is ε-DP, and with probability $1 - \beta$ all its answers are accurate to within*
$$\alpha_c = \frac{8c\Delta(\ln k + \ln(2c/\beta))}{\varepsilon}. \tag{22.17}$$
With runs at the largest ε_0 whose c-fold advanced composition (Advanced and optimal composition) is at most ε at slack δ, it is (ε, δ)-DP, with margin Equation 22.15 at ε_0 and β/c.

Proof.

1. *Privacy.* The runs form an adaptive composition of c AboveThresholds, so basic composition at ε/c gives ε, and advanced composition at ε_0 gives (ε, δ).
2. *Accuracy.* Apply Theorem 22.6 to each run at its budget and failure probability β/c, and take a union bound over the runs.

□

The advanced margin grows like $\sqrt{c}$ instead of c.

The $\perp$ answers are free because step 4 pays for all of them through one maximum, whether there are 3 or 3 million. The run must halt because that payment covers one $\top$, and each further $\top$ against the same ρ needs its own shift. Query noise $\mathrm{Lap}(2c\Delta/\varepsilon_2)$ pays for c shifts at once (Section J.5). The split of ε matters only for accuracy. The proof is tight in the limit. On the pair of $2m$ queries built in Section 22.7, the largest privacy loss is 0.994011, 0.999981 and 1.000000 at $m = 10$, 20 and 40. Figure 22.6 shows one run.

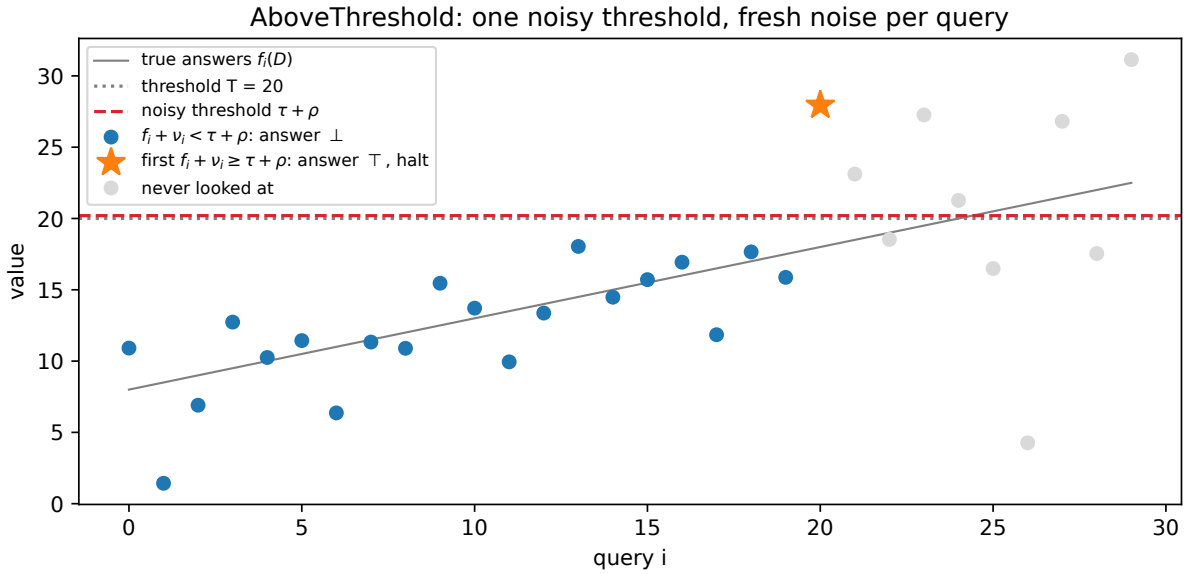


Figure 22.6.: One run of AboveThreshold with $\varepsilon = 1$ on 30 queries whose true answers rise from 8 to 22.5, with threshold $\tau = 20$. Each query's noisy answer is compared with the same noisy threshold. The first one above it is reported and the run halts; later queries are never looked at.

Example 22.6. You monitor $k = 1000$ counts ($\Delta = 1$) with threshold $\tau = 500$ at $\varepsilon = 1$. With $\beta = 0.05$, Equation 22.15 gives $\alpha = 8(\ln 1000 + \ln 40) = 84.77$. In the script's stream every count is 400, below $500 - \alpha$, except query 700, at 600, above $500 + \alpha$. So with probability at least 0.95 the output is query 700. In 10,000 simulated runs it always was, and the margin the noises needed had a 95% quantile of 78.59, so the union bound is nearly tight. For the first c alarms, Equation 22.17 gives a margin of 1,032 at $c = 10$. Advanced composition at $\delta = 10^{-6}$ gives the worse 1,746 there, and wins only at $c = 100$, with 6,506 against 12,161.

22.7. When the sparse vector technique goes wrong

The sparse vector technique is easy to break. Lyu, Su and Li (2017) found several published variants, each a harmless-looking optimization of AboveThreshold, that were not private at all

(Table 22.4). Each removes an ingredient that one step of the proof of Theorem 22.6 needs, and finding that step is how to review an implementation.

Table 22.4.: Variants of the sparse vector technique and their actual privacy, from Lyu, Su & Li (2017).

Variant (as numbered by Lyu, Su & Li)	What it changes	Privacy shown by Lyu, Su & Li
Algorithm 1 (theirs); Algorithm 2 (Dwork & Roth)	Correct, with c answers above	ε
Algorithm 3 (Roth's 2011 lecture notes)	Outputs the noisy value $f_i + \nu_i$ for each $\top$	∞
Algorithm 4 (Lee & Clifton 2014)	Query noise does not grow with c	$\frac{1+6c}{4}\varepsilon$
Algorithm 5 (Stoddard et al. 2014)	No noise on the queries	∞
Algorithm 6 (Chen et al. 2015)	Never halts	∞

The **no-halt variant** keeps the noise of AboveThreshold at the equal split but answers every query. One pair of neighbours and one output with too large a ratio suffice to break it. The pair below makes each answer of $\perp^m\top^m$ a little more likely under D, and without a halt the $2m$ factors multiply. Take $\Delta = 1$ and

$$f(D) = (\underbrace{0,\dots,0}_{m},\underbrace{1,\dots,1}_{m}), \qquad f(D') = (\underbrace{1,\dots,1}_{m},\underbrace{0,\dots,0}_{m}). \tag{22.18}$$

Proposition 22.3 (Without the halt, the loss is unbounded)**.** *On this pair, the no-halt variant has*

$$\ln \frac{\Pr[\perp^m\top^m \mid D]}{\Pr[\perp^m\top^m \mid D']} \geq \frac{m\varepsilon}{4}, \tag{22.19}$$

so it is not ε'-DP for any ε'.

Proof. Once the shared threshold is fixed the answers are independent, so the plan is to fix it, multiply, bound the ratio at every threshold, and average. Let $b = 4/\varepsilon$ be the query-noise scale $2\Delta/\varepsilon_2$ at the equal split, F its Laplace CDF and $\bar{F} = 1 - F$.

1. *Fix the threshold.* Given $\tau + \rho = t$, a query of value f says $\perp$ with probability $F(t - f)$ and $\top$ with probability $\bar{F}(t - f)$.
2. *Multiply.* Under D the output needs m zeros below and m ones above, and under D' the values swap:

$$a_D(t) = F(t)^m\,\bar{F}(t-1)^m, \qquad a_{D'}(t) = F(t-1)^m\,\bar{F}(t)^m. \tag{22.20}$$

3. *A power.* Their ratio is $\omega(t)^m$, with $\omega(t) = \frac{F(t)}{F(t-1)} \cdot \frac{\bar{F}(t-1)}{\bar{F}(t)}$, and both factors are at least 1.
4. *Bound ω.* The CDF is $\frac{1}{2}e^{x/b}$ for $x \leq 0$ and the survival function $\frac{1}{2}e^{-x/b}$ for $x \geq 0$. So the first factor is $e^{1/b}$ when $t \leq 0$, and the second is $e^{1/b}$ when $t \geq 1$. For $0 < t < 1$, $F(t) \geq \frac{1}{2}$ makes the first factor at least $e^{(1-t)/b}$, and likewise the second is at least $e^{t/b}$. So $\omega(t) \geq e^{1/b}$ at every threshold.

5. *Average.* The threshold has the same distribution under both datasets, so averaging $a_D \geq e^{m/b} a_{D'}$ gives Equation 22.19. The pair exists for every m, so the loss is unbounded.

□

AboveThreshold's proof fails here because each $\top$ after the first needs its own shift, and the halt is what limits them to one. Without query noise (Algorithm 5), even the halting version fails, because two queries can give an output that is impossible on D'. Publishing $f_j + \nu_j$ (Algorithm 3) releases the noise the proof integrates over. The no-halt variant adds as much noise per query as AboveThreshold, so a reviewer who checks only noise scales will not find the bug. An audit will find it. Under ε-DP every test, a rule that says "D" or "D'" from the output, fires at most e^{ε} times as often under D as under D', by the definition, and a simple sign test breaks that (Figure 22.7, right).

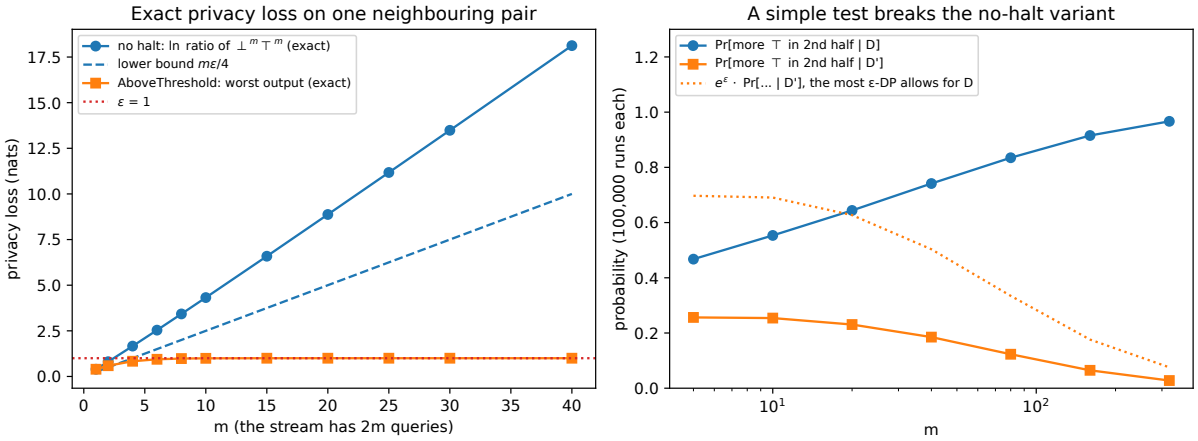


Figure 22.7.: Left: privacy loss on the neighbouring pair $f(D) = (0^m, 1^m)$, $f(D') = (1^m, 0^m)$ with $\varepsilon = 1$, computed exactly by integrating over the threshold noise. For the no-halt variant, the loss of the output $\bot^m \top^m$ grows linearly in m, above the lower bound $m\varepsilon/4$. For AboveThreshold, the largest loss over all outputs levels off at ε. Right: the no-halt variant under a simple test, the probability that the second half has more $\top$ answers than the first. Once the probability under D exceeds e^{ε} times the probability under D' (dotted), no ε-DP mechanism could produce it.

Example 22.7. Table 22.5 gives the script's exact values for $\varepsilon = 1$ and $\tau = 0$: the largest loss over all outputs for AboveThreshold, and the loss of $\bot^m \top^m$ for the no-halt variant.

Table 22.5.: Exact privacy loss of AboveThreshold and of its no-halt variant on the counterexample, with $\varepsilon = 1$.

m	1	5	10	20	40
AboveThreshold, largest loss over outputs	0.3958	0.8998	0.9940	1.0000	1.0000
No halt, loss of $\bot^m\top^m$	0.3958	2.0985	4.3220	8.8741	18.1246
Lower bound $m\varepsilon/4$	0.25	1.25	2.50	5.00	10.00

At $m = 1$ the two coincide, because AboveThreshold halts after the $\top$ anyway. Then the no-halt loss grows by about 0.45 nats (units of natural logarithm) per pair of queries. The lower bound is loose by a factor of a little under 2, because step 4 bounded ω at its smallest. The sign test asks whether the second half has more $\top$ answers. Its log-ratio is 0.77 at $m = 10$ and 1.40 at $m = 40$, so from $m = 40$ on it alone proves that the variant is not 1-DP.

22.8. Code

The script code/selection.py computes every number in this chapter and draws every figure. Exponential-mechanism probabilities are exact sums, the noisy max and the sparse vector technique are integrated on a grid over one noise variable, and simulations check the integrals. From the repository root:

```
uv run python code/part03-dp-foundations/selection.py
```

Table 22.6.: The script's output with the default seed.

Experiment	Result
Laplace threshold shift, $b = s = 1$	Ratio 1.6321 at $t = 0$, 2.5357 at $t = 0.7$, 2.7183 at $t = 1$ and $t = 2$
Exponential mechanism on the vote, $\varepsilon = 1$	Table 22.1; normalizer ratio 1.1790; largest loss 0.3353 (monotone weights: 0.6585)
Exponential mechanism on 50 models	Table 22.2; $\Pr[\text{gap} \geq 13.82] = 9.12 \times 10^{-4}$ against $e^{-3} = 0.0498$; tied profile at gap 20: 0.0022 against 0.0023
Laplace report-noisy-max, 50 models, $\varepsilon = 1$	$b = 2$: Pr[best] 0.4241, expected gap 1.40 (simulated 1.39). $b = 1$: 0.6463 and 0.54
Laplace report-noisy-max on the vote, $b = 1$	Largest loss 0.6674 ($b = 2$: 0.3626)

Experiment	Result
Gumbel-max, five candidates	Within 0.0008 of the exponential mechanism over 10^6 runs; Laplace at total variation 0.0216
AboveThreshold, $k = 1000$, $\beta = 0.05$	$\alpha = 84.77$; simulated 95% quantile 78.59; alarm stream correct in 10,000 of 10,000 runs
AboveThreshold against no halt	Table 22.5; sign test log-ratios 0.77, 1.40, 2.63 at $m = 10, 40, 160$
Sparse, $k = 1000$, $\beta = 0.05$	Margin 1,032 ($c = 10$) and 12,161 ($c = 100$); with advanced composition at $\delta = 10^{-6}$, 1,746 and 6,506
Threshold learner, $n = 1000$	Excess error 0.0024 ($\varepsilon = 1$), 0.0206 ($\varepsilon = 0.1$), 0.0008 non-private; records for $\alpha = 0.05$: 33,913 (sampling), 793 (privacy at $\varepsilon = 1$)
Private median of 1,000 gradient norms	Median 0.9804; mean distance 0.0013 ($\varepsilon = 1$) and 0.0157 ($\varepsilon = 0.1$); imbalance bound 26.4 and 264.5

tests/test_selection.py checks that the output still matches.

22.9. Summary

Table 22.7.: The results of this chapter, and where each is used.

Result	Statement	Use
Theorem 22.1	Pick r with probability $\propto e^{\varepsilon u(D,r)/(2\Delta)}$; ε-DP, and $e^{\varepsilon u/\Delta}$ for monotone scores	Model selection; private multiplicative weights; fix the candidates first
Theorem 22.2	Gap $\leq \frac{2\Delta}{\varepsilon}(\ln\lvert\mathcal{R}\rvert + t)$ w.p. $1 - e^{-t}$	Candidates cost $\ln\lvert\mathcal{R}\rvert$ in accuracy, nothing in privacy
Proposition 22.1	A data-independent normalizer allows $e^{\varepsilon u/\Delta}$	Why the Laplace mechanism needs no factor 2
Theorem 22.3	Excess error $\leq 2\sqrt{\ln(2\lvert\mathcal{H}\rvert/\beta)/(2n)} + 2\ln(\lvert\mathcal{H}\rvert/\beta)/(\varepsilon n)$	A finite class from $O(\ln\lvert\mathcal{H}\rvert)$ records
Proposition 22.2	Piece j w.p. $\propto \ell_j e^{-\varepsilon\lvert 2j-n\rvert/2}$; imbalance $\leq \frac{2}{\varepsilon}(\ln(B/\ell^\star) + t)$	A private clipping norm or quantile; fix the range first
Lemma 22.1, Lemma 22.2, Lemma 22.3	Freeze data-independent noise; a maximum moves by at most Δ; a shifted Laplace tail changes by at most $e^{\lvert s\rvert/b}$	Every noisy-max proof

Result	Statement	Use
Theorem 22.4	$\arg\max_r(u + \mathrm{Lap}(2\Delta/\varepsilon))$ is ε-DP; $\mathrm{Lap}(\Delta/\varepsilon)$ for monotone scores	Vote counts in private prediction; release the index only
Theorem 22.5	A Gumbel noisy max is the exponential mechanism	Stable sampling; top-c at $c\varepsilon$
Theorem 22.6	AboveThreshold is $(\varepsilon_1 + \varepsilon_2)$-DP for any number of $\perp$; margin Equation 22.15	Monitoring; the reusable holdout of adaptive data analysis
Corollary 22.1	c runs: margin Equation 22.17, or $\sqrt{c}$ growth with advanced composition	Advanced composition once $c \gg 2\ln(1/\delta)$
Proposition 22.3	Without the halt the loss grows like $m\varepsilon/4$	Draw fresh query noise, keep one threshold per run, halt, never publish noisy values

22.10. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3 (report-noisy-max, Claim 3.9), §3.4 (the exponential mechanism, Theorems 3.10 and 3.11) and §3.6 (the sparse vector technique: AboveThreshold, Theorems 3.23 and 3.24, and the algorithms Sparse and NumericSparse, with Theorem 3.26 for their accuracy). The proofs above follow theirs. Their version of Sparse with $\delta > 0$ uses a closed-form per-run budget where this chapter solves for the largest one.
- McSherry & Talwar, "Mechanism Design via Differential Privacy" (FOCS 2007). The exponential mechanism, with the continuous version and its use in auctions.
- Lyu, Su & Li, "Understanding the Sparse Vector Technique for Differential Privacy" (Proceedings of the VLDB Endowment, 2017). The general algorithm with c answers above and a budget split, the monotone improvement, the survey of incorrect variants in Table 22.4, and the optimal split. The best single source on the technique.
- Dwork, Naor, Reingold, Rothblum & Vadhan, "On the Complexity of Differentially Private Data Release" (STOC 2009). Where the sparse vector idea first appears.
- Kasiviswanathan, Lee, Nissim, Raskhodnikova & Smith, "What Can We Learn Privately?" (FOCS 2008; SIAM Journal on Computing 2011). The private learner of Equation 22.7: any finite class is learnable privately with $O(\ln|\mathcal{H}|)$ records.
- Alon, Livni, Malliaris & Moran, "Private PAC Learning Implies Finite Littlestone Dimension" (STOC 2019). Why the class must be finite: thresholds over an infinite domain cannot be learned privately.
- Smith, "Privacy-Preserving Statistical Estimation with Optimal Convergence Rates" (STOC 2011), and Gillenwater, Joseph & Kulesza, "Differentially Private Quantiles" (ICML 2021). The exponential mechanism for medians and quantiles over an interval, sampled exactly piece by piece.

- Dwork, Feldman, Hardt, Pitassi, Reingold & Roth, "The Reusable Holdout: Preserving Validity in Adaptive Data Analysis" (Science 2015). Thresholdout, an application of the sparse vector technique to validation sets reused across many experiments, treated in Adaptive data analysis.
- Durfee & Rogers, "Practical Differentially Private Top-k Selection with Pay-what-you-get Composition" (NeurIPS 2019). The one-shot Gumbel top-c and its equivalence to repeated exponential mechanisms.
- McKenna & Sheldon, "Permute-and-Flip: A New Mechanism for Differentially Private Selection" (NeurIPS 2020), and Ding et al., "The Permute-and-Flip Mechanism is Identical to Report-Noisy-Max with Exponential Noise" (2021).
- Maddison, Tarlow & Minka, "A* Sampling" (NeurIPS 2014): the Gumbel-max trick in machine learning, with its history going back to Gumbel and to Yellott (1977).
- Near & Abuah, *Programming Differential Privacy*, the chapters on the exponential mechanism and the sparse vector technique: short implementations alongside Dwork & Roth.

23. Query release and private multiplicative weights

Many private releases are a large batch of simple statistics: every cross-tabulation of a census file, or the hundreds of counts on a dashboard. Each is a **linear query**, the average over the records of a function that gives each record a value between 0 and 1. Adding noise to every answer makes the noise grow with the number of queries, in proportion to it under pure differential privacy (DP) and like its square root under approximate DP (The Laplace and Gaussian mechanisms). With millions of queries, either is useless. Yet the reconstruction attack breaks privacy only when the answers are within about $1/\sqrt{n}$ of the truth, for n records. This chapter fills the gap with an error that grows only with the logarithms of the number of queries and of the number of possible records.

The one idea is to learn the data from its mistakes. Keep a public guess of the data, a histogram that starts out uniform, find a query that the guess gets badly wrong, measure that one query privately, and nudge the guess toward the answer. Each nudge is a step of the **multiplicative weights** method from online learning, and its guarantee says that about $\ln N/\alpha^2$ nudges suffice for error α on every query, where N is the number of possible records. The number of queries does not appear, so the privacy budget is spent on corrections instead of on queries, and the final guess can be published as a **synthetic dataset**.

The running example is a table of $n = 10{,}000$ records with eight yes/no attributes, drawn from a mixture of three groups, so there are $N = 2^8 = 256$ possible records. The queries are every cell of every marginal table: for each set of attributes and each combination of their values, the fraction of records with those values. That gives $2^8 - 1 = 255$ tables, and a cell fixes each attribute to 0 or 1 or leaves it out, which gives $3^8 - 1 = 6560$ cells. The uniform histogram, which uses no data, has a largest error of 0.232 over these queries.

23.1. Linear queries as inner products

Every algorithm in this chapter treats a dataset as a histogram and a query as a vector, so that answering a query is an inner product. This section fixes that view first, because it lets a guessed histogram answer every query.

Definition 23.1 (Universe, histogram and linear query)**.** The **universe** $\mathcal{X}$ is the set of all possible records, and $N = |\mathcal{X}|$ is its size. A dataset $D = (x_1, \dots, x_n)$ holds n records from the universe. Its **histogram** p gives, for each element x of the universe, the fraction $p(x)$ of the records that equal x. A **linear query** is a function $q\colon \mathcal{X} \to [0,1]$, and its answer on D is the average of its values over the records,

$$q(D) = \frac{1}{n}\sum_{i=1}^{n} q(x_i). \tag{23.1}$$

The set of queries to answer is the **workload** $\mathcal{Q}$, with $|\mathcal{Q}|$ queries. Neighbouring datasets $D \sim D'$ differ by replacing one record, so n is public.

The answer depends on the records only through how many of them take each value, which is what the histogram records.

Proposition 23.1 (A linear query is an inner product with the histogram)**.** *For every linear query q and dataset D with histogram p,*

$$q(D) = \sum_{x \in \mathcal{X}} q(x)\, p(x) = \langle q, p \rangle, \tag{23.2}$$

the inner product of two vectors indexed by the universe. Replacing one record moves the answer by at most $1/n$, so q has sensitivity $1/n$.

Proof.

1. Group the records by value. Each element x of the universe contributes $q(x)$ once for every record equal to it, and there are $np(x)$ such records, so $\sum_i q(x_i) = \sum_x np(x)q(x)$. Dividing by n gives Equation 23.2.
2. Replacing one record x_i by x_i' changes the sum by $q(x_i') - q(x_i)$, which lies in $[-1, 1]$ because q takes values in $[0, 1]$. Divided by n, the answer moves by at most $1/n$.

□

Two consequences carry the chapter. Any distribution $\hat{p}$ over the universe, not only the histogram of a real dataset, gives an answer $\langle q, \hat{p} \rangle$ to every query, so a histogram is a synthetic dataset. And the **error** of $\hat{p}$ on q, the distance between its answer and the true one, is $|\langle q, \hat{p} - p \rangle|$, which is linear in the difference of the two histograms. The target throughout is an error of at most α on every query of the workload, except with a small failure probability β, and all logarithms are natural.

Example 23.1. Take two yes/no attributes, so the universe is $\{00, 01, 10, 11\}$, and four records $01, 11, 11, 10$. Table 23.1 lists the histogram and the query "the first attribute is 1". The inner product is $0 + 0 + \frac{1}{4} + \frac{1}{2} = \frac{3}{4}$, and indeed three of the four records have a first attribute of 1. The running example is the same picture with 256 elements, where each marginal cell is a query with a 1 on the elements that match it, so the script answers all 6,560 queries at once with one matrix product.

Table 23.1.: A four-record dataset as a histogram, and one query.

Element x	00	01	10	11
Records equal to x	0	1	1	2
$p(x)$	0	1/4	1/4	1/2
$q(x)$: first attribute is 1	0	0	1	1

23.2. Noise on every answer

The direct way to release a workload is to add noise to each answer. Before improving on it, it helps to know what it costs, because the two sample sizes of this section are the targets that the rest of the chapter beats.

Write $f(D)$ for the vector of the $|\mathcal{Q}|$ answers. By Proposition 23.1, replacing one record moves each coordinate by at most $1/n$, so the ℓ_1 sensitivity of f is at most $|\mathcal{Q}|/n$ and its ℓ_2 sensitivity at most $\sqrt{|\mathcal{Q}|}/n$. The square root comes from the norm: the ℓ_1 norm adds the moves, while the ℓ_2 norm adds their squares and takes a square root. The Laplace mechanism adds noise of scale $b = \Delta_1 f/\varepsilon$ to every coordinate, and each noise exceeds t in absolute value with probability $e^{-t/b}$ (Theorem 20.1). The Gaussian mechanism is calibrated through $\mu = \mu(\varepsilon, \delta)$, the largest ratio of sensitivity to noise standard deviation that the exact calibration allows (Corollary 20.1).

Proposition 23.2 (Records needed with noise on every answer)**.** *Let the workload have $|\mathcal{Q}|$ queries, and let z be the number that a standard normal variable exceeds with probability $\beta/(2|\mathcal{Q}|)$. The Laplace mechanism on the answer vector, with scale $|\mathcal{Q}|/(n\varepsilon)$, is ε-DP, and with probability at least $1-\beta$ its largest error is at most α once*

$$n \geq \frac{|\mathcal{Q}| \ln(|\mathcal{Q}|/\beta)}{\alpha\varepsilon}. \tag{23.3}$$

The Gaussian mechanism with standard deviation $\sqrt{|\mathcal{Q}|}/(n\mu)$ on every answer is (ε, δ)-DP, and with probability at least $1-\beta$ its largest error is at most α once

$$n \geq \frac{\sqrt{|\mathcal{Q}|}\, z}{\mu\alpha}. \tag{23.4}$$

Proof.

1. *Laplace.* Each answer gets noise of scale $b = |\mathcal{Q}|/(n\varepsilon)$, which exceeds $b\ln(|\mathcal{Q}|/\beta)$ in absolute value with probability $\beta/|\mathcal{Q}|$. The union bound, which says that the probability that at least one of several events happens is at most the sum of their probabilities, gives probability at most β that some answer's noise exceeds that level. Setting $b\ln(|\mathcal{Q}|/\beta) = \alpha$ and solving for n gives Equation 23.3.
2. *Gaussian.* A normal variable exceeds z standard deviations in absolute value with probability $2\Phi(-z)$, where Φ is the standard normal cumulative distribution function, and the one-sided tail $\beta/(2|\mathcal{Q}|)$ that defines z makes this $\beta/|\mathcal{Q}|$. By the union bound, some answer's noise exceeds σz with probability at most β, where $\sigma = \sqrt{|\mathcal{Q}|}/(n\mu)$ is the standard deviation. Setting $\sigma z = \alpha$ and solving for n gives Equation 23.4.

□

The records needed grow like $|\mathcal{Q}|$ under pure DP and like $\sqrt{|\mathcal{Q}|}$, times a slowly growing z, under approximate DP. For a few hundred queries on a large dataset that is fine, and nothing in this chapter beats it there. But a table with 30 yes/no attributes has $3^{30} - 1 \approx 2 \times 10^{14}$ marginal cells, too many for independent noise.

A workload with structure does better. Replacing one record lowers one cell of a marginal table by $1/n$ and raises another by $1/n$, so each table contributes only $2/n$ to the ℓ_1 sensitivity, whatever its number of cells, and $2/n^2$ to the squared ℓ_2 sensitivity.

Example 23.2. With $n = 10{,}000$ and $\varepsilon = 1$, the 255 marginal tables give an ℓ_1 sensitivity of $2 \times 255/n = 0.0510$, so the Laplace mechanism adds noise of scale 0.0510 to each of the 6,560 cells. Its largest error, averaged over 20 runs, is 0.497, worse than the 0.232 of the uniform histogram, which uses no data. The Gaussian mechanism at $\delta = 10^{-6}$ has ℓ_2 sensitivity $\sqrt{2 \times 255}/n = 0.00226$. With $\mu = 0.2367$ its noise has standard deviation 0.0095, and its average largest error is 0.037. Both use the structure of the marginals: treated as 6,560 unrelated queries, the ℓ_1 sensitivity would be $|\mathcal{Q}|/n = 0.656$, about 13 times larger.

23.3. Multiplicative weights and its regret bound

Forget privacy for two sections. The method behind this chapter comes from a prediction game, and the question is how well a learner can do against any sequence of outcomes when all it may do is reweight a fixed set of advisers. The answer is a regret bound that depends on the number of advisers only through a logarithm, and it drives every algorithm that follows.

Each day, N **experts** predict a yes/no outcome, and you must decide whom to follow before you see it. If one expert is always right, the **halving algorithm** follows the majority of the experts who have never erred and drops each one that errs. Each of your mistakes drops at least half of them, so you err at most $\log_2 N$ times. In 20 simulated runs with 1,024 experts it never made more than 6 mistakes, against the bound $\log_2 1024 = 10$. Real experts are never perfect, so multiplicative weights keeps every expert but shrinks its weight each time it does badly.

Definition 23.2 (The experts game and multiplicative weights)**.** The game has T rounds. In round t the learner chooses a probability distribution p_t over the N experts. Then a **loss vector** ℓ_t is revealed, which gives each expert x a loss $\ell_t(x)$ in $[-1, 1]$, and the learner pays the average loss under its distribution, $\langle \ell_t, p_t \rangle$. The losses may be chosen by an adversary who knows the learner's rule. **Multiplicative weights** keeps a positive weight $w_t(x)$ on each expert, starting from $w_1(x) = 1$, and has a **learning rate** (step size) $\eta \le \frac{1}{2}$. In round t it plays $p_t(x) = w_t(x)/W_t$, where $W_t = \sum_x w_t(x)$ is the total weight, and after the losses are revealed it updates each weight by

$$w_{t+1}(x) = w_t(x)(1 - \eta \ell_t(x)). \tag{23.5}$$

The learner's **regret** against a distribution π over the experts, fixed in hindsight, is its total loss minus the total loss of π, that is $\sum_t \langle \ell_t, p_t \rangle - \sum_t \langle \ell_t, \pi \rangle$.

An expert with a positive loss loses weight, and one with a negative loss gains it. The proof of the regret bound takes the logarithm of factors $1 - x$ twice: from above, when it turns the total weight into the learner's loss, and from below, when it turns one expert's weight into that expert's loss. The next lemma supplies both bounds, and its range $|x| \le \frac{1}{2}$ is why the learning rate is at most $\frac{1}{2}$.

Lemma 23.1 (Two bounds on the logarithm)**.** *For every $x < 1$, $\ln(1 - x) \le -x$. For $|x| \le \frac{1}{2}$,*

$$\ln(1 - x) \ge -x - x^2. \tag{23.6}$$

Proof.

1. *Upper bound.* The logarithm is concave, so it lies below its tangent line at any point. The tangent of $\ln(1-x)$ at $x=0$ is $-x$, which gives $\ln(1-x) \le -x$.
2. *Lower bound.* Let $g(x) = \ln(1-x) + x + x^2$, so that $g(0) = 0$ and the claim is that g is never negative on the range. Its derivative is

$$g'(x) = -\frac{1}{1-x} + 1 + 2x = \frac{x(1-2x)}{1-x}. \tag{23.7}$$

   On $[0, \frac{1}{2}]$ every factor is non-negative, so g rises from 0 and stays non-negative. On $[-\frac{1}{2}, 0]$ the numerator is at most 0 and the denominator positive, so g falls toward $g(0) = 0$ and is non-negative there too.

□

At the end of the range the bound is close, since $\ln \frac{1}{2} = -0.6931$ lies just above -0.75. Outside the range it can fail, and at $x = 0.9$ it does, because $\ln 0.1 = -2.30$ lies below -1.71. A loss outside $[-1, 1]$ pushes $\eta\ell_t(x)$ out of the range in the same way, and at $\eta\ell_t(x) \ge 1$ a weight can even become zero or negative, so losses are rescaled into $[-1, 1]$ first.

Theorem 23.1 (The regret bound of multiplicative weights)**.** *Run multiplicative weights with learning rate $\eta \le \frac{1}{2}$ for T rounds against any sequence of loss vectors with entries in $[-1, 1]$. Then for every distribution π over the experts,*

$$\sum_{t=1}^{T} \langle \ell_t, p_t \rangle - \sum_{t=1}^{T} \langle \ell_t, \pi \rangle \le \eta T + \frac{\ln N}{\eta}. \tag{23.8}$$

When $T \ge 4 \ln N$, the learning rate $\eta = \sqrt{\ln N / T}$ is allowed and gives regret at most $2\sqrt{T \ln N}$.

Proof. The plan is to bound $\ln W_{T+1}$ from above by the learner's total loss and from below by any one expert's total loss, to compare the two bounds, and then to average the comparison over the experts.

1. *The total weight shrinks with the learner's loss.* Summing Equation 23.5 over the experts gives $W_{t+1} = \sum_x w_t(x) - \eta \sum_x w_t(x)\ell_t(x)$. Factoring out W_t turns each weight into $p_t(x)$, so

$$W_{t+1} = W_t(1 - \eta \langle \ell_t, p_t \rangle). \tag{23.9}$$

2. *Upper bound.* Unrolling step 1 from $W_1 = N$ makes W_{T+1} equal to N times the product of the factors. Each $\eta\langle \ell_t, p_t \rangle$ is below 1, so the upper bound of Lemma 23.1 applies to every factor:

$$\ln W_{T+1} = \ln N + \sum_{t=1}^{T} \ln\left(1 - \eta \langle \ell_t, p_t \rangle\right) \le \ln N - \eta \sum_{t=1}^{T} \langle \ell_t, p_t \rangle. \tag{23.10}$$

3. *Lower bound from one expert.* The total weight is at least the weight of any single expert x, and that weight is the product of its own factors. Each $\eta\ell_t(x)$ lies in $[-\frac{1}{2}, \frac{1}{2}]$, so the lower bound of Lemma 23.1 applies, and $\ell_t(x)^2 \le 1$ bounds each squared term:

$$\ln W_{T+1} \ge \ln w_{T+1}(x) = \sum_{t=1}^{T} \ln\left(1 - \eta \ell_t(x)\right) \ge -\eta \sum_{t=1}^{T} \ell_t(x) - \eta^2 T. \tag{23.11}$$

4. *Compare.* The upper bound of step 2 is at least the lower bound of step 3, so $\eta \sum_t \langle \ell_t, p_t \rangle \le \ln N + \eta \sum_t \ell_t(x) + \eta^2 T$. Dividing by η gives Equation 23.8 for the comparison with the single expert x.
5. *Any distribution.* The inequality of step 4 holds for every expert x, and its left side does not depend on x. Multiply it by $\pi(x)$ and add over the experts. The $\pi(x)$ sum to 1, so the left side and the constant terms are unchanged, and $\sum_x \pi(x)\ell_t(x) = \langle \ell_t, \pi \rangle$, which is Equation 23.8.
6. *Choose the learning rate.* A larger η raises ηT and lowers $\ln N/\eta$, so the bound is smallest where the two terms are equal, at $\eta = \sqrt{\ln N/T}$, where it equals $2\sqrt{T \ln N}$. The lemma needs $\eta \le \frac{1}{2}$, that is $T \ge 4 \ln N$.

□

The average regret per round, $2\sqrt{\ln N/T}$, goes to zero, so over many rounds the learner does as well as the best fixed distribution. Nothing in the proof assumed that the losses were random or independent of the weights, so an adversary who knows the rule gains nothing, and the number of experts enters only through $\ln N$. The sequences met in practice are rarely as hostile as the worst case, as Section 23.4 shows.

The weights also look familiar. Each factor $1 - \eta\ell$ is close to $e^{-\eta\ell}$ when η is small, so the distribution is close to a softmax of minus the cumulative losses, the shape of the exponential mechanism. The version with the factor $e^{-\eta\ell}$ exactly is called Hedge, and online learning also views it as follow the regularized leader (FTRL) with an entropy penalty; DP-FTRL uses a squared-norm penalty to train models privately.

Example 23.3. The smallest case shows the mechanics. Take $N = 2$ experts and $\eta = \frac{1}{2}$ over $T = 3$ rounds, with losses $\ell_1 = (1, 0)$, $\ell_2 = (0, 1)$ and $\ell_3 = (1, 0)$. Table 23.2 follows the weights. The learner pays $\frac{1}{2} + \frac{2}{3} + \frac{1}{2} = \frac{5}{3}$ in total. Expert 2 lost only 1, so the regret against it is $\frac{2}{3}$, and the bound Equation 23.8 is $\frac{1}{2} \times 3 + \ln 2/\frac{1}{2} = 1.5 + 1.386 = 2.886$. The running example enters in the next section, where the experts are the 256 elements of the universe.

Table 23.2.: Multiplicative weights on two experts with learning rate 1/2.

Round t	Weights w_t	Distribution p_t	Loss ℓ_t	Learner pays $\langle \ell_t, p_t \rangle$
1	$(1, 1)$	$(1/2, 1/2)$	$(1, 0)$	1/2
2	$(1/2, 1)$	$(1/3, 2/3)$	$(0, 1)$	2/3
3	$(1/2, 1/2)$	$(1/2, 1/2)$	$(1, 0)$	1/2

23.4. From regret to a mistake bound

Now turn the game into an algorithm for releasing answers, still without privacy. The question is how many corrections a guessed histogram needs before it answers every query to within α, and the answer must not depend on the number of queries. Let the experts be the elements of the universe, so that p_t is a guessed histogram, and compare it with the true histogram p, one particular distribution π. If each round's regret against p is the guess's error on one query, the regret bound limits the number of large errors.

Theorem 23.2 (The mistake bound)**.** *Run multiplicative weights over the universe, starting from the uniform histogram, with learning rate $\eta = \alpha/2$. In each round, look for a query $q_t \in \mathcal{Q}$ on which the current guess has error more than α. If there is none, stop: the guess is within α on every query. Otherwise let s_t be $+1$ if the guess's answer is too high and -1 if it is too low, and use the loss $\ell_t = s_t q_t$. The number U of rounds before the algorithm stops satisfies*

$$U < \frac{4 \ln N}{\alpha^2}, \tag{23.12}$$

whatever the workload and however the queries are chosen.

Proof. The plan is to show that the regret against the true histogram grows by more than α in every round, to bound the same regret by Equation 23.8, and to compare the two.

1. *Each round's regret is the error.* The regret bound holds for every distribution π, and the loss of p is the true answer times s_t, so by Equation 23.2 $\langle \ell_t, p_t - p \rangle = s_t(\langle q_t, p_t \rangle - \langle q_t, p \rangle)$. The sign s_t makes the bracket positive, so this is the error of p_t on q_t, more than α.
2. *Total regret is large.* Adding over the U rounds, the regret against p is more than αU.
3. *Total regret is small.* The losses $s_t q_t$ have entries in $[-1, 1]$, and $\eta = \alpha/2 \le \frac{1}{2}$, so Theorem 23.1 with $\pi = p$ and $T = U$ bounds the same regret by $\eta U + \ln N/\eta = \alpha U/2 + 2 \ln N/\alpha$.
4. *Combine.* The regret is more than αU by step 2 and at most $\alpha U/2 + 2 \ln N/\alpha$ by step 3, so

$$\alpha U < \frac{\alpha U}{2} + \frac{2 \ln N}{\alpha}. \tag{23.13}$$

   Subtracting $\alpha U/2$ from both sides and dividing by $\alpha/2$ gives Equation 23.12.

□

If the guess's answer is too high, the loss is $+q_t$, and the weights of the records that the query counts shrink, which lowers the answer. A few such corrections fix every query at once because each one moves the guess closer to the true histogram in relative entropy, a distance that starts at no more than $\ln N$. The algorithm never looks at the queries it gets right, so their number never enters, and doubling the universe adds only $4 \ln 2/\alpha^2$ corrections.

The choice of query and its sign are the only places where the algorithm touches the data, and both use only answers of sensitivity $1/n$, which makes it easy to privatize. The bound is very loose, because in practice the first corrections fix errors several times larger than α, and many queries at once.

Example 23.4. With $\alpha = 0.05$ the bound Equation 23.12 allows $4 \ln 256/0.05^2 = 8872$ corrections. The algorithm needs 148 (Figure 23.1). The first three queries it corrects have errors 0.232, 0.228 and 0.226, more than four times α. After 148 rounds the regret is 16.70, against a bound of 225.5.

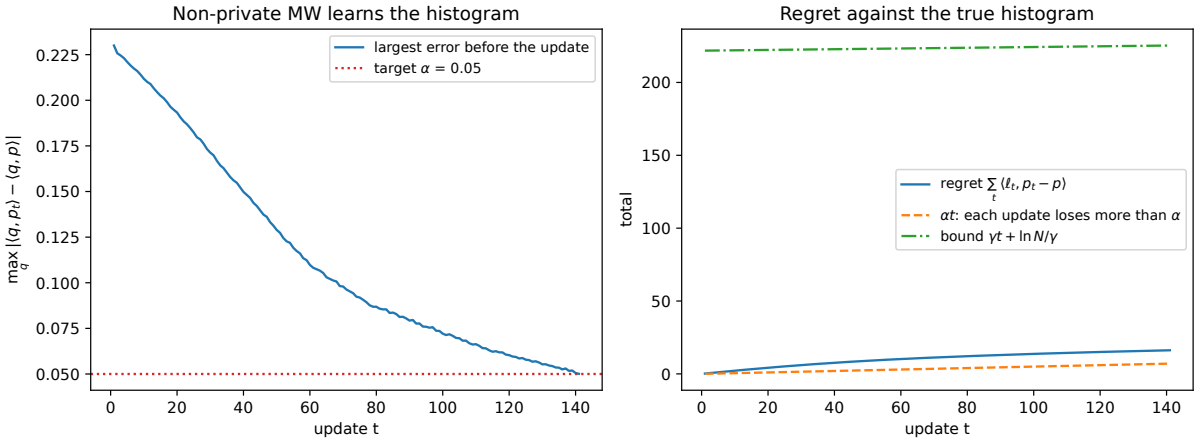


Figure 23.1.: Non-private multiplicative weights on the running example, $\alpha = 0.05$, $\eta = \alpha/2$. Left: the largest error over the 6,560 queries before each update. Right: the regret against the true histogram and its bound $\eta t + \ln N/\eta$.

23.5. Private multiplicative weights

The algorithm of Section 23.4 touches the data in two places, the choice of a badly answered query and the sign of its error. This section makes both private, which gives an error that grows only with the logarithms of $|\mathcal{Q}|$ and N. The exponential mechanism picks a query whose error is close to the largest, and the Laplace mechanism measures its answer, which gives the sign. Both can miss, so the algorithm cannot tell when to stop. Instead it runs a fixed number of rounds T and releases the average of its guesses. This is the **multiplicative weights exponential mechanism (MWEM)** of Hardt, Ligett & McSherry (2012), the practical form of private multiplicative weights (Hardt & Rothblum 2010).

Definition 23.3 (Private multiplicative weights, offline)**.** Input: the dataset D, the workload $\mathcal{Q}$, a number of rounds $T \geq 4 \ln N$ and a per-step budget ε_0. Start from the uniform histogram p_1 and set $\eta = \sqrt{\ln N / T}$. For $t = 1, \dots, T$:

1. *Select.* Run the exponential mechanism over $\mathcal{Q}$ with budget ε_0 and score $|\langle q, p_t \rangle - q(D)|$, the error of the current guess on q. Call the chosen query q_t.
2. *Measure.* Release $m_t = q_t(D) + \mathrm{Lap}(1/(n\varepsilon_0))$.
3. *Update.* Let $s_t = +1$ if $\langle q_t, p_t \rangle > m_t$ and -1 otherwise, and apply Equation 23.5 with the loss $s_t q_t$ to get p_{t+1}.

Output the average histogram $\bar{p} = \frac{1}{T} \sum_{t=1}^{T} p_t$.

The measurement needs its own budget, because the exponential mechanism releases only the index of the chosen query, never its exact score.

Three earlier results do the private work. The exponential mechanism over the $|\mathcal{Q}|$ queries, with a score of sensitivity $1/n$, is ε_0-DP, and except with probability β' the chosen score is within $\frac{2}{n\varepsilon_0}(\ln |\mathcal{Q}| + \ln(1/\beta'))$ of the best (Selection). The Laplace mechanism with scale $b = 1/(n\varepsilon_0)$ is ε_0-DP, and its noise exceeds $b \ln(1/\beta')$ in absolute value with probability β' (Theorem 20.1).

Advanced composition (Advanced and optimal composition) makes k adaptive ε_0-DP steps (ε', δ)-DP with $\varepsilon' = \varepsilon_0\sqrt{2k\ln(1/\delta)} + k\varepsilon_0\tanh(\varepsilon_0/2)$. The privacy step below needs a closed-form per-step budget from it.

Lemma 23.2 (A closed-form per-step budget)**.** *Let $\varepsilon > 0$ and $\delta \in (0,1)$ satisfy $\varepsilon \le 8\ln(1/\delta)$. If each of k adaptively chosen steps is ε_0-DP with*

$$\varepsilon_0 = \frac{\varepsilon}{2\sqrt{2k\ln(1/\delta)}}, \tag{23.14}$$

then the whole is (ε, δ)-DP.

Proof.

1. Substitute ε_0 into the advanced composition bound. The first term, $\varepsilon_0\sqrt{2k\ln(1/\delta)}$, is exactly $\varepsilon/2$.
2. The second term, $k\varepsilon_0\tanh(\varepsilon_0/2)$, is at most $k\varepsilon_0^2/2$, because tanh never exceeds its argument. Substituting ε_0 turns $k\varepsilon_0^2/2$ into $\varepsilon^2/(16\ln(1/\delta))$, and the condition $\varepsilon \le 8\ln(1/\delta)$ makes this at most $\varepsilon/2$.
3. The two terms add to at most ε, so the composition is (ε, δ)-DP.

□

The condition is mild, since at $\delta = 10^{-6}$ it allows any ε up to 110. When ε_0 is small the second term is far below $\varepsilon/2$, so the closed form leaves up to half the budget unspent (Example 23.5).

Theorem 23.3 (Privacy and accuracy of private multiplicative weights)**.** *The algorithm of Definition 23.3 is $2T\varepsilon_0$-DP. So $\varepsilon_0 = \varepsilon/(2T)$ makes it ε-DP, and ε_0 from Equation 23.14 with $k = 2T$ makes it (ε, δ)-DP when $\varepsilon \le 8\ln(1/\delta)$. With probability at least $1-\beta$, the largest error of the average histogram over the workload is at most*

$$\max_{q\in\mathcal{Q}} \left|\langle q, \bar{p}\rangle - q(D)\right| \le 2\sqrt{\frac{\ln N}{T}} + \frac{2}{n\varepsilon_0}\Big(\ln|\mathcal{Q}| + 2\ln\frac{2T}{\beta}\Big). \tag{23.15}$$

Proof. The plan is to check that each private step is ε_0-DP, to define the event on which every selection and every measurement works as intended, and on that event to run the regret argument of Theorem 23.2 with a deficit per round for the two private steps.

1. *Privacy.* Given the history, the guess p_t is a fixed histogram, so the score moves by at most $1/n$ between neighbours (Proposition 23.1) and the selection is ε_0-DP. The measurement is a Laplace mechanism of scale $1/(n\varepsilon_0)$ on an answer of sensitivity $1/n$, so it is ε_0-DP too. The update and the average are post-processing, and composition allows the $2T$ steps to be adaptive (Advanced and optimal composition).
2. *The good event.* Write $\mathrm{err}_t(q) = |\langle q, p_t\rangle - q(D)|$ for the error of round t's guess on query q. Give each of the $2T$ private steps the failure probability $\beta' = \beta/(2T)$. The utility bound of the exponential mechanism then says that the selection picks a query whose error is within

$$c_E = \frac{2}{n\varepsilon_0}\Big(\ln|\mathcal{Q}| + \ln\frac{2T}{\beta}\Big) \tag{23.16}$$

of the largest, except with probability $\beta/(2T)$. The Laplace tail says that the measurement's noise is below

$$c_L = \frac{1}{n\varepsilon_0} \ln \frac{2T}{\beta}, \tag{23.17}$$

also except with probability $\beta/(2T)$. The union bound over the $2T$ steps makes all of this hold at once except with probability β.

3. *Each round's regret is nearly the largest error.* If the sign s_t is right, then the round's regret $\langle s_t q_t, p_t - p\rangle$ is $\mathrm{err}_t(q_t)$, as in step 1 of Theorem 23.2. If it is wrong, the guess's answer lies between the measured and the true answers, which are at most c_L apart, so $\mathrm{err}_t(q_t) \le c_L$, and the regret $-\mathrm{err}_t(q_t)$ falls short of $\mathrm{err}_t(q_t)$ by at most $2c_L$. Either way the round's regret is at least $\mathrm{err}_t(q_t) - 2c_L$, and by step 2 the chosen query's error is within c_E of the largest, so

$$\langle s_t q_t, p_t - p\rangle \ge \max_q \mathrm{err}_t(q) - c_E - 2c_L. \tag{23.18}$$

4. *Sum and use the regret bound.* Adding over the T rounds, the regret against the true histogram is at least $\sum_t \max_q \mathrm{err}_t(q) - T(c_E + 2c_L)$. By Theorem 23.1 with $\eta = \sqrt{\ln N/T}$ it is at most $2\sqrt{T \ln N}$. Comparing the two and dividing by T,

$$\frac{1}{T}\sum_{t=1}^{T} \max_q \mathrm{err}_t(q) \le 2\sqrt{\frac{\ln N}{T}} + c_E + 2c_L. \tag{23.19}$$

5. *The average histogram.* For any query q, the error of the average is the average of the errors by Equation 23.2, $\langle q, \bar{p}\rangle - q(D) = \frac{1}{T}\sum_t \langle q, p_t - p\rangle$. Its absolute value is at most $\frac{1}{T}\sum_t \mathrm{err}_t(q)$, which is at most the left side of step 4. Finally $c_E + 2c_L = \frac{2}{n\varepsilon_0}(\ln|\mathcal{Q}| + 2\ln\frac{2T}{\beta})$, which gives Equation 23.15.

□

The algorithm releases the average guess because the regret bound controls the sum of the rounds' largest errors, and says nothing about any single round. The constants are poor: on the running example the bound is above 1, while the real error is far smaller (Example 23.5).

The first term of Equation 23.15 falls with more rounds, and the second grows, because each round gets less budget. Balancing the two turns the bound into a sample size.

Corollary 23.1 (Records needed by private multiplicative weights)**.** *Write $L = \ln|\mathcal{Q}| + 2\ln(2T/\beta)$ for the logarithmic term of Equation 23.15, treated as a constant because it depends on T only through a logarithm. Under (ε, δ)-DP with the budget of Equation 23.14 for $2T$ steps, there is a number of rounds T at which the error bound is at most α once*

$$n \ge \frac{64\sqrt{\ln N \, \ln(1/\delta)}\, L}{\alpha^2 \varepsilon}. \tag{23.20}$$

Proof.

1. *The shape of the bound.* With $k = 2T$ steps, Equation 23.14 gives $\varepsilon_0 = \varepsilon/(4\sqrt{T\ln(1/\delta)})$, so the second term of Equation 23.15 is $8\sqrt{\ln(1/\delta)}\, L\sqrt{T}/(n\varepsilon)$. The bound therefore has the shape $A/\sqrt{T} + B\sqrt{T}$, with $A = 2\sqrt{\ln N}$ from the regret and $B = 8\sqrt{\ln(1/\delta)}\, L/(n\varepsilon)$ from the noise.

2. *Balance.* A sum of this shape is smallest where its two terms are equal, at $T = A/B$, and there it equals $2\sqrt{AB}$. Substituting A and B,

$$2\sqrt{AB} = 8(\ln N \ln(1/\delta))^{1/4}\sqrt{\frac{L}{n\varepsilon}}. \tag{23.21}$$

3. *Solve for n.* Setting this to α and squaring gives $64\sqrt{\ln N \ln(1/\delta)}\, L/(n\varepsilon) = \alpha^2$, which rearranges to Equation 23.20.

□

Under pure DP, $\varepsilon_0 = \varepsilon/(2T)$ makes the second term $4LT/(n\varepsilon)$, linear in T. Minimizing $2\sqrt{\ln N/T} + 4LT/(n\varepsilon)$ over T and setting the minimum to α gives $n \geq 108\, L \ln N/(\alpha^3\varepsilon)$. The queries enter only through L, which grows like $\ln|\mathcal{Q}|$, where the baseline paid $|\mathcal{Q}|$ or $\sqrt{|\mathcal{Q}|}$. The price is accuracy, because at a fixed n the error falls like $n^{-1/2}$, or $n^{-1/3}$ under pure DP, against $1/n$ for independent noise, so the method wins only when the queries far outnumber the records. No method can do much better on every workload, since errors below about $1/\sqrt{n}$ on many queries let the reconstruction attack recover the data. Up to constants it is close to optimal, by the packing and fingerprinting arguments of Lower bounds (Bun, Ullman & Vadhan 2014).

The running time is proportional to N, which is 2^{50} for 50 attributes. Some such cost is unavoidable, since Ullman & Vadhan (2011) used probabilistically checkable proof (PCP) techniques to show that, under standard cryptographic assumptions, no efficient algorithm releases private synthetic data accurate for every two-way marginal. Practical methods keep the select–measure–update loop with a compact model in place of the histogram (McKenna, Mullins, Sheldon & Miklau 2022).

Example 23.5. With $n = 10{,}000$, $\varepsilon = 1$ and pure DP, the number of rounds that minimizes Equation 23.15 is $T = 38$, so each step gets $\varepsilon_0 = 1/76 = 0.0132$. Table 23.3 compares three choices of T, averaged over 20 runs.

Table 23.3.: Private multiplicative weights on the running example, $n = 10{,}000$, $\varepsilon = 1$.

Rounds T	Budget per step ε_0	Largest error	Bound Equation 23.15
10	0.0500	0.098	1.572
38	0.0132	0.078	1.120
100	0.0050	0.123	1.486

The bound picks a good T, although its values are useless as numbers. With 38 rounds the largest error is 0.078, against 0.497 for Laplace noise and 0.037 for Gaussian noise, so here the Gaussian mechanism wins. Figure 23.2 shows that Gaussian noise wins above a few thousand records, because 6,560 queries are not many enough.

With $\delta > 0$, advanced composition gives each step more budget once T is large. At $T = 100$ and $\delta = 10^{-6}$, pure DP gives each step $\varepsilon_0 = 0.00500$. The closed form Equation 23.14 gives 0.00673, although in total that budget spends only 0.505ε, while solving the advanced composition bound exactly for ε_0 gives 0.01322. At $T = 1000$ the same three budgets are 0.00050, 0.00213 and 0.00418.

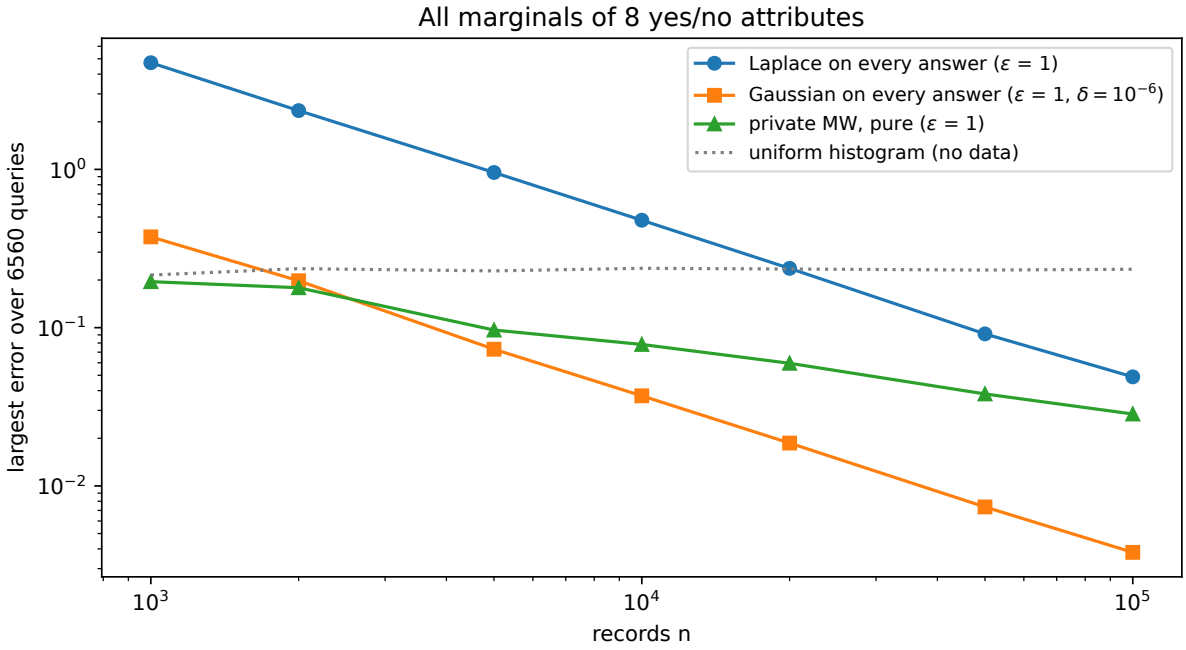
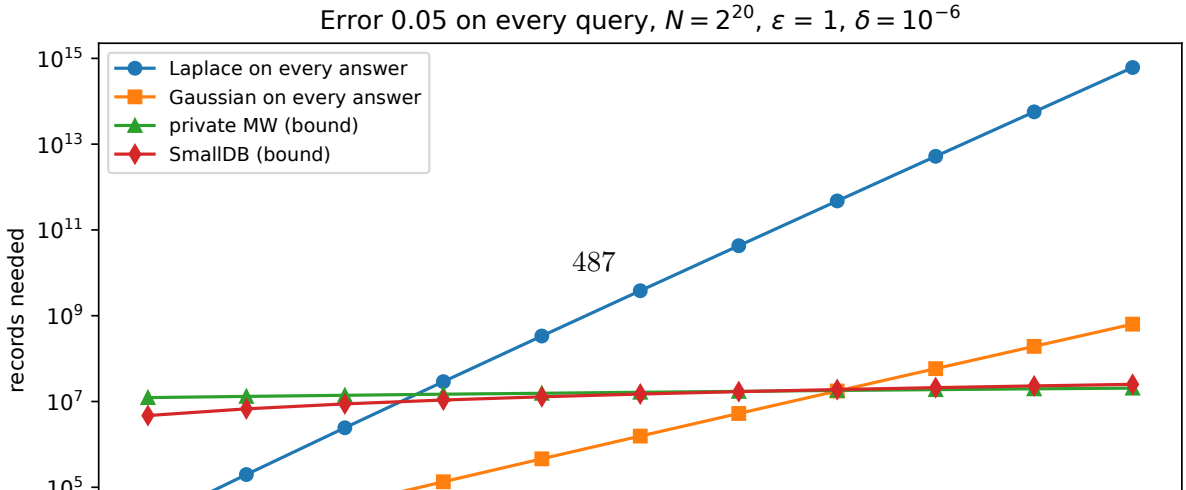


Figure 23.2.: Largest error over the 6,560 marginal queries of the running example as the number of records grows, averaged over 10 runs: Laplace and Gaussian noise on every answer, and private multiplicative weights with pure DP and its best number of rounds. The dotted line is the uniform histogram.

The guarantees separate only at a much larger scale. Table 23.4 gives the records needed for error 0.05 on every query with probability 0.95, over a universe of $N = 2^{20}$ and a workload with no structure. The Laplace and SmallDB columns are pure DP, and the other two are (ε, δ)-DP. For private multiplicative weights the table solves Equation 23.15 for n after a search over T. At 10^6 queries the search finds $T = 81{,}552$ and $L = 43.81$, and needs 1.56×10^7 records, while the closed form Equation 23.20 gives 1.55×10^7. That bound hardly moves as the queries grow a millionfold, and it crosses the Gaussian column near 10^9 queries (Figure 23.3).

Table 23.4.: Records needed for error 0.05 on every query with probability 0.95, $\varepsilon = 1$, $\delta = 10^{-6}$, $N = 2^{20}$.

Queries $\lvert\mathcal{Q}\rvert$	Laplace	Gaussian	Private multiplicative weights (bound)	SmallDB (bound)
10^3	1.98×10^5	1.08×10^4	1.31×10^7	6.74×10^6
10^6	3.36×10^8	4.61×10^5	1.56×10^7	1.29×10^7
10^9	4.74×10^{11}	1.76×10^7	1.81×10^7	1.90×10^7



pay only for the queries the guess gets wrong without searching for them. The sparse vector technique of Selection waits for a bad query to arrive instead, which is the original form of private multiplicative weights (Hardt & Rothblum 2010).

Its basic form, AboveThreshold, compares each query's noisy value with a noisy threshold, answers $\perp$ ("below") or $\top$ ("above"), and halts at the first $\top$, while the $\perp$ answers cost nothing. With probability $1-\beta$, every $\perp$ lies below the threshold plus a **margin** and every $\top$ above it minus the margin, which for K queries of sensitivity Δ at budget ε is $8\Delta(\ln K+\ln(2/\beta))/\varepsilon$ (Equation 22.15). The variant Sparse restarts it after each $\top$. Here the question asked is whether the guess's error is large, so only the few queries it gets wrong cost privacy.

Definition 23.4 (Online private multiplicative weights)**.** Input: the dataset, a target error α, privacy parameters (ε,δ), and a stream of at most K queries, with K fixed in advance. Set the budget of updates to $c=\lceil 16\ln N/\alpha^2\rceil$, the learning rate to $\eta=\alpha/4$, and the guess $\hat{p}$ to the uniform histogram. Give each of the at most c runs of AboveThreshold inside Sparse, and each of the at most c measurements, the budget ε_0 of Equation 23.14 with $k=2c$ steps. For each incoming query f_i (the letter f marks queries of the stream), the score is the current guess's error on it, $g_i=|\langle f_i,\hat{p}\rangle - f_i(D)|$. Run Sparse on these scores with threshold $3\alpha/4$, allowing up to c answers $\top$, and act on its answer:

- If Sparse answers $\perp$, answer $\langle f_i,\hat{p}\rangle$ from the guess.
- If it answers $\top$, measure $m_i=f_i(D)+\mathrm{Lap}(1/(n\varepsilon_0))$, answer m_i, and update $\hat{p}$ by Equation 23.5 with the loss $s_i f_i$, where s_i is the sign of $\langle f_i,\hat{p}\rangle - m_i$.
- After the c-th $\top$, halt.

The threshold sits at $3\alpha/4$ because a $\top$ must certify an error of at least $\alpha/2$, twice the learning rate, so that each update gains more regret than the $\eta=\alpha/4$ per round that Theorem 23.1 allows, while a $\perp$ must certify an error below α. A margin of $\alpha/4$ gives both, so accuracy needs enough records to keep every margin and every measurement's noise at most $\alpha/4$.

Theorem 23.4 (Privacy and accuracy of the online algorithm)**.** *The algorithm of Definition 23.4 is (ε,δ)-DP when $\varepsilon\le 8\ln(1/\delta)$. If*

$$n\ge\frac{32(\ln K+\ln(4c/\beta))}{\alpha\varepsilon_0},\qquad \varepsilon_0=\frac{\varepsilon}{2\sqrt{4c\ln(1/\delta)}},\tag{23.22}$$

then with probability at least $1-\beta$ every answer is within α of the truth, and the algorithm answers the whole stream without using up its c answers $\top$.

Proof. The plan is to check the scores' sensitivity, compose the steps, and, on the event where every margin and noise is at most $\alpha/4$, show that every answer is accurate and every update is a real correction, so that the mistake bound limits their number.

1. *Sensitivity.* Between two updates the guess $\hat{p}$ is fixed, because it changes only after a measurement, which is a released output. Given the history, each score g_i is the distance between a fixed number and an answer of sensitivity $1/n$, so it has sensitivity $1/n$.

2. *Privacy.* The runs of AboveThreshold and the measurements alternate: a run ends with its $\top$, that query is measured, then the next run starts with the updated guess. So the algorithm is a sequence of at most $2c$ steps, each ε_0-DP and each chosen as a function of the earlier outputs, and by Lemma 23.2 with $k = 2c$ it is (ε, δ)-DP.
3. *The good event.* Half of the failure probability goes to the c runs, $\beta/(2c)$ each, and half to the c measurements. With budget ε_0, sensitivity $1/n$ and failure probability $\beta/(2c)$, the margin of one run is $8(\ln K + \ln(4c/\beta))/(n\varepsilon_0)$ by Equation 22.15, and Equation 23.22 says exactly that this is at most $\alpha/4$. A measurement's noise exceeds $\ln(2c/\beta)/(n\varepsilon_0)$ with probability $\beta/(2c)$ by the Laplace tail, and that level is below the margin of a run, so it is at most $\alpha/4$ too. The union bound over the c runs and the c measurements gives total failure probability β.
4. *Answers.* On the good event, a $\perp$ means $g_i < 3\alpha/4 + \alpha/4 = \alpha$, so the guess's answer is within α. A $\top$ is answered by the measurement, which is within $\alpha/4$.
5. *Updates are real corrections.* A $\top$ means $g_i \geq 3\alpha/4 - \alpha/4 = \alpha/2$. The measurement's noise is at most $\alpha/4$, less than the error, so the measured answer lies on the same side of the guess's answer as the true one, and the sign s_i is right. As in step 1 of Theorem 23.2, the update's regret against the true histogram is $g_i \geq \alpha/2$.
6. *Few updates.* After U updates the regret is at least $\alpha U/2$, and by Equation 23.8 with $\eta = \alpha/4$ it is at most $\alpha U/4 + 4\ln N/\alpha$. So $\alpha U/4 \leq 4\ln N/\alpha$, which is $U \leq 16\ln N/\alpha^2 \leq c$, and the budget of c answers $\top$ is never used up.

□

Because $\sqrt{c}$ is about $4\sqrt{\ln N}/\alpha$, the records needed grow like $\sqrt{\ln N \ln(1/\delta)}/(\alpha^2\varepsilon)$ times a logarithm, as offline, but the logarithm is of the stream length K instead of $|\mathcal{Q}|$. A $\top$ query must be measured, because the guess is off by at least $\alpha/2$ there and the measurement gives the sign.

The guarantee holds even when the analyst chooses each query after the earlier answers, because every step of the proof conditions on the history, which is the setting of Adaptive data analysis. The budget c is set by the worst case, so it is far larger than a real stream needs.

Example 23.6. On the running example, with $\alpha = 0.05$, the budget is $c = 35{,}490$ corrections, against the 148 of Example 23.4. For a query server over a universe of $N = 2^{20}$ records, the budget is $c = 88{,}723$. A stream of $K = 10^6$ queries at $\varepsilon = 1$ and $\delta = 10^{-6}$, with probability 0.95, needs 8.39×10^7 records by Equation 23.22. That is about five times the offline bound in Table 23.4, the price of the worst-case budget and the wider margin.

23.7. SmallDB: the exponential mechanism over small datasets

Blum, Ligett & Roth (2008) were the first to answer many linear queries with error that grows only with $\ln|\mathcal{Q}|$, and their algorithm answers a different question: what can the exponential mechanism achieve in a single choice, when running time is ignored? **SmallDB** makes that choice among whole small datasets, like the classifier selection of Selection with datasets in place of classifiers.

The proof first needs some small dataset that answers every query well, so that the exponential mechanism has a good candidate. The next lemma shows that a random sample of the data is one with positive probability.

Lemma 23.3 (A small dataset with small error exists)**.** *Let $m > 2\ln(2|\mathcal{Q}|)/\alpha^2$. Then some dataset of m records from the universe has error at most $\alpha/2$ on every query of $\mathcal{Q}$.*

Proof.

1. Draw m records independently from the true histogram p and call the result $\hat{D}$. For one query q, the answer $q(\hat{D})$ is an average of m independent values in $[0,1]$ with mean $q(D)$.
2. Hoeffding's inequality (Concentration inequalities) says that such an average is more than s away from its mean with probability at most $2e^{-2ms^2}$, which at $s = \alpha/2$ is $2e^{-m\alpha^2/2}$.
3. By the union bound, some query is more than $\alpha/2$ off with probability at most $2|\mathcal{Q}|e^{-m\alpha^2/2}$, and with $m > 2\ln(2|\mathcal{Q}|)/\alpha^2$ this is below 1. So with positive probability no query is off by more than $\alpha/2$, and a dataset with that property exists.

□

Theorem 23.5 (SmallDB)**.** *Let $m = \lceil 2\ln(2|\mathcal{Q}|)/\alpha^2 \rceil$ (or $2\ln(2|\mathcal{Q}|)/\alpha^2 + 1$ if that ratio is an integer), and run the exponential mechanism with budget ε over every dataset $\hat{D}$ of m records from the universe, with the score $-\max_{q\in\mathcal{Q}} |q(\hat{D}) - q(D)|$, minus the candidate's largest error. The mechanism is ε-DP, and the chosen dataset has error at most α on every query with probability at least $1-\beta$ once*

$$n \geq \frac{4(m\ln N + \ln(1/\beta))}{\alpha\varepsilon} \approx \frac{8\ln(2|\mathcal{Q}|)\ln N}{\alpha^3\varepsilon}. \tag{23.23}$$

Proof. The plan is to show that the best candidate scores at least $-\alpha/2$, to count the candidates, and to apply the utility bound of the exponential mechanism.

1. *Privacy.* Each $q(D)$ has sensitivity $1/n$ by Proposition 23.1, so a candidate's largest error moves by at most $1/n$ between neighbours, the score has sensitivity $1/n$, and the exponential mechanism is ε-DP.
2. *A good candidate exists.* By Lemma 23.3, some candidate has error at most $\alpha/2$ on every query, so the best score is at least $-\alpha/2$.
3. *Count the candidates.* A dataset of m records is a list of m elements of the universe, so there are at most N^m candidates, and $\ln N^m = m\ln N$.
4. *The selection is nearly the best.* The utility bound of the exponential mechanism, with N^m candidates, sensitivity $1/n$ and failure probability β, says that the chosen dataset's score is within $\frac{2}{n\varepsilon}(m\ln N + \ln(1/\beta))$ of the best, except with probability β. Its error is therefore at most $\alpha/2$ plus that amount.
5. *Records.* Asking that the amount be at most $\alpha/2$ gives the first form of Equation 23.23. Substituting $m \approx 2\ln(2|\mathcal{Q}|)/\alpha^2$ and dropping the smaller term $\ln(1/\beta)$ gives the approximate form.

□

SmallDB has the same logarithm of $|\mathcal{Q}|$ as private multiplicative weights, and under pure DP the same α^{-3}, which comes from m, growing like $1/\alpha^2$, times the $1/(\alpha\varepsilon)$ of the exponential mechanism's error. It is simpler to analyse, because it makes one choice instead of T, but hopeless to run, because the exponential mechanism must weigh N^m candidates.

Example 23.7. For the 6,560 marginal queries at $\alpha = 0.1$, a dataset of $m = 1897$ records suffices, and there are $256^{1897} \approx 10^{4568}$ candidates. Equation 23.23 asks for 4.21×10^5 records at $\varepsilon = 1$. For a universe of 2^{20} and $\alpha = 0.05$, the last column of Table 23.4 shows that SmallDB's bound is close to that of private multiplicative weights.

23.8. Code

The script code/query_release.py computes every number in this chapter and draws every figure: it runs the baselines and every algorithm on the running example and evaluates every bound. From the repository root:

```
uv run python code/part03-dp-foundations/query_release.py
```

Table 23.5.: The script's output with the default seed.

Experiment	Result
Running example	$N = 256$, $\lvert\mathcal{Q}\rvert = 6560$ cells in 255 tables; uniform histogram error 0.232
Baselines, $n = 10{,}000$	Laplace scale 0.0510, largest error 0.497; Gaussian $\mu = 0.2367$, standard deviation 0.0095, largest error 0.037
Halving, 1,024 experts	At most 6 mistakes in 20 runs; bound 10
Non-private multiplicative weights, $\alpha = 0.05$	148 corrections against the bound 8,872; regret 16.70 against 225.5
Private multiplicative weights, $\varepsilon = 1$	Table 23.3; best $T = 38$
Budget per step at $\delta = 10^{-6}$	$T = 100$: 0.00500, 0.00673, 0.01322; $T = 1000$: 0.00050, 0.00213, 0.00418
Records needed	Table 23.4; closed form 1.55×10^7 at 10^6 queries
SmallDB, $\alpha = 0.1$	$m = 1897$; 10^{4568} candidates; 4.21×10^5 records
Online private multiplicative weights, $\alpha = 0.05$	$c = 88{,}723$ for $N = 2^{20}$, 8.39×10^7 records for $K = 10^6$; $c = 35{,}490$ for $N = 256$

tests/test_query_release.py checks that the output still matches.

23.9. Summary

Table 23.6.: The results of this chapter, and where each is used.

Result	Statement	Use
Proposition 23.1	$q(D) = \langle q, p \rangle$, with sensitivity $1/n$	A histogram is a synthetic dataset
Proposition 23.2	Laplace: $n \geq \lvert\mathcal{Q}\rvert \ln(\lvert\mathcal{Q}\rvert/\beta)/(\alpha\varepsilon)$; Gaussian: $n \geq \sqrt{\lvert\mathcal{Q}\rvert}\, z/(\mu\alpha)$	For up to thousands of queries, Gaussian noise scaled to the workload's structure
Lemma 23.1	$-x - x^2 \leq \ln(1-x) \leq -x$ for $\lvert x\rvert \leq \frac{1}{2}$	Keep $\eta \leq \frac{1}{2}$ and losses in $[-1, 1]$
Theorem 23.1	Regret $\leq \eta T + \ln N/\eta$, and $\leq 2\sqrt{T \ln N}$ at $\eta = \sqrt{\ln N/T}$	Learning with exponentially many experts; DP-FTRL
Theorem 23.2	$U < 4 \ln N/\alpha^2$ corrections at $\eta = \alpha/2$	Spend the budget on corrections, not on queries
Lemma 23.2	$\varepsilon_0 = \varepsilon/(2\sqrt{2k \ln(1/\delta)})$ gives (ε, δ)-DP for $\varepsilon \leq 8 \ln(1/\delta)$	A closed-form per-step budget; the exact one is up to twice as large
Theorem 23.3	Error $\leq 2\sqrt{\ln N/T} + \frac{2}{n\varepsilon_0}(\ln\lvert\mathcal{Q}\rvert + 2 \ln \frac{2T}{\beta})$	Synthetic data for a fixed workload; release the average and choose T by the bound
Corollary 23.1	$n \geq 64\sqrt{\ln N \ln(1/\delta)}\, L/(\alpha^2\varepsilon)$; $108 L \ln N/(\alpha^3\varepsilon)$ under pure DP	When the queries far outnumber the records; time proportional to N
Theorem 23.4	$c = \lceil 16 \ln N/\alpha^2 \rceil$ corrections; $n \geq 32(\ln K + \ln(4c/\beta))/(\alpha\varepsilon_0)$	Interactive query servers with adaptive queries
Theorem 23.5	$n \geq 4(m \ln N + \ln(1/\beta))/(\alpha\varepsilon)$ with $m = \lceil 2 \ln(2\lvert\mathcal{Q}\rvert)/\alpha^2 \rceil$; time N^m	What the exponential mechanism can do when running time is ignored

23.10. References

- Hardt & Rothblum, "A Multiplicative Weights Mechanism for Privacy-Preserving Data Analysis" (FOCS 2010). Private multiplicative weights in its online form, with the sparse vector technique, and the $\sqrt{\ln N} \ln\lvert\mathcal{Q}\rvert/\alpha^2$ sample complexity.
- Hardt, Ligett & McSherry, "A Simple and Practical Algorithm for Differentially Private Data Release" (NeurIPS 2012). MWEM, the offline form with the exponential mechanism and an averaged output, and its experiments on real tables.
- Blum, Ligett & Roth, "A Learning Theory Approach to Non-Interactive Database Privacy" (STOC 2008; Journal of the ACM 2013). SmallDB, the first release with error logarithmic in the number of queries.
- Arora, Hazan & Kale, "The Multiplicative Weights Update Method: a Meta-Algorithm and Applications" (Theory of Computing, 2012). The regret bound of Equation 23.8 and the many uses of the method.

- Littlestone & Warmuth, "The Weighted Majority Algorithm" (Information and Computation, 1994). The halving and weighted majority algorithms.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), Chapter 4. SmallDB and online private multiplicative weights, with full proofs.
- Gupta, Roth & Ullman, "Iterative Constructions and Private Data Release" (TCC 2012). The general select–measure–update framework behind both algorithms.
- Bun, Ullman & Vadhan, "Fingerprinting Codes and the Price of Approximate Differential Privacy" (STOC 2014). Lower bounds showing that private multiplicative weights is close to optimal for some query families.
- Ullman & Vadhan, "PCPs and the Hardness of Generating Private Synthetic Data" (TCC 2011). Why efficient private synthetic data for all two-way marginals is impossible in general.
- McKenna, Mullins, Sheldon & Miklau, "AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data" (VLDB 2022). The select–measure–update loop with graphical models and Gaussian measurements, as used in practice.

24. Beyond global sensitivity

The Laplace and Gaussian mechanisms add noise scaled to the **global sensitivity** of a query f, the most that changing one record can move f over every pair of neighbouring datasets that could exist (Definition 20.1). For counts that is the right scale, but the median of ages in $[0, 100]$ has global sensitivity 100. On a dataset with half its people at 0 and half at 100, one person moves the median from one end to the other. Yet on the dataset you actually hold, the median of 1,001 ages moves by about a tenth of a year when one person changes. This chapter asks whether you can add noise on the scale of the data you have and still satisfy differential privacy (DP).

The answer is yes, and one idea organizes every method: the noise scale is itself a function of the data, so it must be private too. Smooth sensitivity, propose-test-release and inverse sensitivity repair the leak for a statistic like the median. Stability-based release handles discrete answers, subsample-and-aggregate handles black-box estimators, and clipping handles unbounded data, at the price of a bias.

The running example is the median of $n = 1{,}001$ synthetic ages, drawn from a normal distribution with mean 38 and standard deviation 14 and clipped to $[0, 100]$, released with $\varepsilon = 1$ and, where needed, $\delta = 10^{-6}$. Its median is 36.96, and one changed age moves it by at most 0.1386 years. Values lie in a range $[0, \Lambda]$, with $\Lambda = 100$ for the ages, and are sorted as $x_1 \le ... \le x_n$. Datasets have a fixed size n, and their neighbours $D \sim D'$ differ by one replaced record (The definition of differential privacy), so the **Hamming distance** $d(D, D'')$ to any third dataset counts the records in which they differ. Every proof bounds the privacy loss of an output y, how much more likely y is under D than under D',

$$L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}, \tag{24.1}$$

and pure ε-DP holds exactly when it never exceeds ε for any neighbours in either order.

24.1. Local sensitivity, and why it leaks

Global sensitivity takes a maximum over all pairs of neighbours, including pathological ones. The natural fix is to take the maximum only over the neighbours of the dataset you hold. This section computes that quantity for the median and shows why noise proportional to it is not private.

Definition 24.1 (Local sensitivity)**.** The **local sensitivity** of a real-valued query f at a dataset D is the most that one record can move f starting from D itself,

$$\mathrm{LS}_f(D) = \max_{D' \sim D} |f(D) - f(D')|. \tag{24.2}$$

The global sensitivity is its worst case over all datasets, $\Delta f = \max_D \mathrm{LS}_f(D)$, so $\mathrm{LS}_f(D) \le \Delta f$ always.

For the median, and for its smooth sensitivity in Section 24.2, you need to know how far an order statistic can travel when k records change.

Lemma 24.1 (Changing k records moves each order statistic by at most k places)**.** *Sort the values of D as $x_1 \le ... \le x_n$, all in $[0, \Lambda]$, and pad with $x_i = 0$ for $i \le 0$ and $x_i = \Lambda$ for $i > n$. If D'' differs from D in at most k records and has sorted values $x''_1 \le ... \le x''_n$, then for every rank j,*

$$x_{j-k} \le x''_j \le x_{j+k}. \tag{24.3}$$

Proof.

1. *Upper bound.* At least $j + k$ values of D are at most x_{j+k}, and the padding makes this true when $j + k > n$. Changing k records removes at most k of them, so at least j values of D'' are still at most x_{j+k}, which says that $x''_j \le x_{j+k}$.
2. *Lower bound.* At least $n - j + k + 1$ values of D are at least x_{j-k}, and at least $n - j + 1$ of them survive k changes, so $x''_j \ge x_{j-k}$.

□

Proposition 24.1 (Local and global sensitivity of the median)**.** *Let n be odd and $m = (n + 1)/2$, so that the median of D is x_m. Then*

$$\mathrm{LS}_{\mathrm{med}}(D) = \max\left(x_m - x_{m-1},\ x_{m+1} - x_m\right), \qquad \Delta\,\mathrm{med} = \Lambda. \tag{24.4}$$

Proof.

1. *The median of a neighbour lies between x_{m-1} and x_{m+1}.* This is Lemma 24.1 with $k = 1$ and $j = m$.
2. *Both ends are reached.* Replacing x_m by Λ makes x_{m+1} the m-th smallest value. Replacing it by 0 puts the new 0 first, and x_{m-1} becomes the m-th smallest.
3. *The global sensitivity.* With m values at 0 and $m - 1$ at Λ, the gap above the median is Λ, and no gap can exceed Λ.

□

Now add Laplace noise scaled to the local sensitivity, $M_{\mathrm{LS}}(D) = f(D) + \mathrm{Lap}(\mathrm{LS}_f(D)/\varepsilon)$. The proof of Theorem 20.1 needs the same scale on both sides of the pair, but by Equation 24.4 one record can change this scale from 0 to Λ, and then no (ε, δ) guarantee survives.

Proposition 24.2 (Noise scaled to the local sensitivity is not private)**.** *For the median with $n = 5$ and $\Lambda = 100$, there is a neighbouring pair on which M_{LS} is not (ε, δ)-DP for any finite ε and any $\delta < 1$ (Nissim, Raskhodnikova & Smith 2007, Example 1).*

Proof.

1. *A dataset that gets no noise.* $D = (0, 0, 0, 0, 100)$ has median 0 and both gaps next to it are 0, so $\mathrm{LS}_{\mathrm{med}}(D) = 0$, the noise has scale 0, and $M_{\mathrm{LS}}(D) = 0$ exactly.
2. *Its neighbour gets wide noise.* $D' = (0, 0, 0, 100, 100)$ also has median 0, but the gap above it is 100. So $M_{\mathrm{LS}}(D')$ has a continuous distribution, which equals 0 with probability 0.

3. *The inequality fails.* The output 0 has probability 1 under D and 0 under D', so DP would need $1 \le e^{\varepsilon} \cdot 0 + \delta$, which no $\delta < 1$ satisfies.

□

Example 24.1. The leak needs neither a degenerate dataset nor different answers. Take $\Lambda = 100$, $\varepsilon = 1$ and the neighbours of Table 24.1, which have the same median but local sensitivities six times apart.

Table 24.1.: Two neighbouring datasets with the same median and very different local sensitivities.

Dataset	Values	Median	LS
D	10, 20, 30, 31, 90	30	$\max(30-20, 31-30) = 10$
D' (31 replaced by 90)	10, 20, 30, 90, 90	30	$\max(30-20, 90-30) = 60$

The releases are $30 + \mathrm{Lap}(10)$ and $30 + \mathrm{Lap}(60)$. The narrow density decays six times faster, so the log-ratio of the two Laplace densities grows with the distance from the centre:

$$L_{D',D}(y) = \ln \frac{20}{120} + |y-30|\Big(\frac{1}{10} - \frac{1}{60}\Big) = -1.792 + \frac{|y-30|}{12}. \tag{24.5}$$

No pure ε holds (Figure 24.1). The loss exceeds 1 once $|y-30| > 33.50$, which happens with probability 0.572 under D' and 0.0351 under D. The smallest δ for which the pair is $(1,\delta)$-DP, its privacy profile of Divergences, is $0.572 - e \times 0.0351 = 0.477$ with D' in the numerator, and the script finds 0.215 with the roles swapped. An adversary tells the two apart by the width of the noise, not its centre.

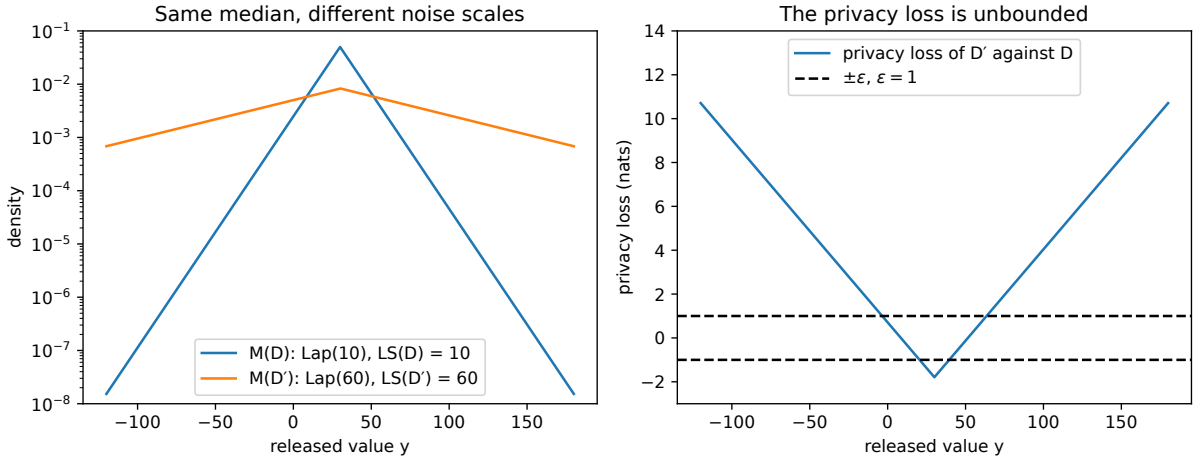


Figure 24.1.: Left: the two releases of Example 24.1, with Laplace noise of scales 10 and 60. Right: their privacy loss, which crosses $\varepsilon = 1$ at distance 33.5.

24.2. Smooth sensitivity

The leak came from a noise scale that jumps between neighbours. So use a scale that cannot jump: an upper bound on the local sensitivity that changes by at most a factor e^{β} between neighbours, since stretching noise by e^{β} moves its log-density by about β. This section finds the smallest such bound (Nissim, Raskhodnikova & Smith 2007) and computes it for the median; Section 24.3 says which noise it can be used with.

Definition 24.2 (Smooth upper bound and smooth sensitivity)**.** Let $\beta > 0$. A function S of the dataset is a **β-smooth upper bound** on LS_f if

$$S(D) \geq \mathrm{LS}_f(D) \quad \text{and} \quad S(D) \leq e^{\beta} S(D') \quad \text{for all } D \sim D'. \tag{24.6}$$

A dataset D'' at distance k with a large local sensitivity forces such a bound up at D too, discounted by a factor $e^{-\beta}$ per step. So the **β-smooth sensitivity** of f takes the maximum over every dataset D'' of size n,

$$S^*_{f,\beta}(D) = \max_{D''} \ \mathrm{LS}_f(D'')\, e^{-\beta\, d(D,D'')}. \tag{24.7}$$

Proposition 24.3 (The smooth sensitivity is the smallest smooth upper bound)**.** *$S^*_{f,\beta}$ is a β-smooth upper bound on LS_f, and every β-smooth upper bound S satisfies $S(D) \geq S^*_{f,\beta}(D)$ for every D.*

Proof.

1. *Upper bound.* The term $D'' = D$ of Equation 24.7, at distance 0, contributes $\mathrm{LS}_f(D)$, and a maximum is at least each of its terms.
2. *Smooth.* A path from D' to D'' can go through D first, so $d(D', D'') \leq 1 + d(D, D'')$. Each term of $S^*_{f,\beta}(D')$ is then at least $e^{-\beta}$ times the term for the same D'' in $S^*_{f,\beta}(D)$. Maximizing over D'' gives $S^*_{f,\beta}(D') \geq e^{-\beta} S^*_{f,\beta}(D)$, and exchanging the two neighbours gives the other direction.
3. *Smallest.* Let S be any β-smooth upper bound and D'' a dataset at distance k from D. Walking from D'' to D in k steps, S falls by at most a factor $e^{-\beta}$ per step, so $S(D) \geq e^{-\beta k} S(D'') \geq e^{-\beta k} \mathrm{LS}_f(D'')$. Maximizing over D'' gives $S(D) \geq S^*_{f,\beta}(D)$.

□

The smooth sensitivity is positive even where the local sensitivity is 0. On the degenerate pair of Proposition 24.2 it is already $100e^{-\beta}$ at D, the discounted local sensitivity of its neighbour D'.

You cannot enumerate all datasets, but all those at distance k get the same discount, so you need only the largest local sensitivity within each distance (a dataset closer than k also enters $A^{(k)}$, with too small a weight, but the term for its own distance already counts it at full weight):

$$S^*_{f,\beta}(D) = \max_{k=0,\dots,n} e^{-k\beta} A^{(k)}(D), \qquad A^{(k)}(D) = \max_{D'':\ d(D,D'')\leq k} \mathrm{LS}_f(D''). \tag{24.8}$$

For the median, $A^{(k)}$ is the widest gap that k changes can open next to the median. You open one by moving the k values between two order statistics $k+1$ ranks apart out to 0 or Λ, so that the emptied window ends up next to the median.

Proposition 24.4 (The smooth sensitivity of the median)**.** *For the median, with the sorting and padding of Lemma 24.1, the largest local sensitivity within k changes is the widest of $k+2$ windows of the sorted data,*

$$A^{(k)}(D) = \max_{0 \le t \le k+1} (x_{m+t} - x_{m+t-k-1}), \qquad k = 0, \dots, n, \tag{24.9}$$

where the window with index t runs from rank $m+t-k-1$ to rank $m+t$ (Nissim, Raskhodnikova & Smith 2007, Proposition 3.4). Since $A^{(n)} = \Lambda$ and each $A^{(k)}$ costs one subtraction per window, $S^*_{\mathrm{med},\beta}$ takes $O(n^2)$ operations.

Proof. Steps 1 to 4 show that no dataset within distance k has a gap next to its median wider than the widest window, and step 5 shows that every window is reached.

1. *What the gap looks like.* Let D'' be within distance k of D, with sorted values $x''_1 \le \dots \le x''_n$. By Equation 24.4 its local sensitivity is one of its two gaps next to the median. Take the upper one, (x''_m, x''_{m+1}), since the reflection $x \mapsto \Lambda - x$ turns lower gaps into upper ones and maps the windows of Equation 24.9 to themselves.
2. *The gap contains at most k original values.* Assume the gap is positive, and let x_ℓ be the largest value of D at or below x''_m. No value of D'' lies strictly inside the gap, so the values of D inside it, say $x_{\ell+1}, \dots, x_{\ell+j}$, were all changed, and $j \le k$. The next value $x_{\ell+j+1}$ is then at or above x''_{m+1}, so the gap is at most $x_{\ell+j+1} - x_\ell$.
3. *The gap sits near rank m.* Exactly m values of D'' are at or below x''_m. Each is an unchanged value among $x_1, \dots, x_\ell$ or one of at most k new values, so $m \le \ell + k$. And at least $\ell - (k-j)$ of $x_1, \dots, x_\ell$ are unchanged, because j of the k changes were used inside the gap, and these unchanged values are among the m, so $\ell - (k-j) \le m$.
4. *Fit it in a window.* If $\ell \le m$, the window from rank ℓ to rank $\ell + k + 1$ has $t = \ell + k + 1 - m$ in $[0, k+1]$ by step 3, and it contains the span of step 2 because $j \le k$. If $\ell > m$, the window with $t = k+1$ runs from rank m to rank $m+k+1$, which is at least $\ell + j + 1$ by the second inequality of step 3, and contains it.
5. *Every window is reached.* The window with index t has k values strictly between its ends, at ranks $\ell = m + t - k - 1$ and $u = m + t$. Moving $k + 1 - t$ of them to 0 and the rest to Λ gives x_ℓ rank m (or $m - 1$ when $t = 0$), so the median sits at one end of an empty interval of length $x_u - x_\ell$. With $k = n$ and $t = m$ the window runs from the padded $x_0 = 0$ to $x_{n+1} = \Lambda$, so $A^{(n)} = \Lambda$ and no larger k is needed.

□

Example 24.2. Take $D = (1, 3, 4, 6, 9)$ on $[0, 10]$, with median 4. Table 24.2 evaluates Equation 24.9 with $\beta = 0.5$ and checks it against brute force over all $11^5 = 161{,}051$ datasets with values in $\{0, \dots, 10\}$.

Table 24.2.: The smooth sensitivity of the median of $D = (1, 3, 4, 6, 9)$ on $[0, 10]$ with $\beta = 0.5$: the formula for $A^{(k)}$ against brute force over all datasets, and the discounted values whose maximum is $S^*_{0.5}$.

k	Windows $x_{m+t} - x_{m+t-k-1}$, $t = 0, \dots, k+1$	$A^{(k)}$	Brute force	$e^{-0.5k} A^{(k)}$
0	$4-3,\ 6-4$	2	2	2.00
1	$4-1,\ 6-3,\ 9-4$	5	5	**3.03**
2	$4-0,\ 6-1,\ 9-3,\ 10-4$	6	6	2.21
3	$4-0,\ 6-0,\ 9-1,\ 10-3,\ 10-4$	8	8	1.79
4	$\dots,\ 9-0,\ 10-1,\ \dots$	9	9	1.22
5	$\dots,\ 10-0,\ \dots$	10	10	0.82

The local sensitivity is 2, the gap from 4 up to 6. But moving the 6 up to 10 opens a gap of $9 - 4 = 5$, and even after the discount $e^{-0.5} = 0.6065$ that beats 2, so $S^*_{0.5}(D) = 3.03$ at $k = 1$ (Figure 24.2, left). For the 1,001 ages the formula gives $S^*_{\text{med},1/6} = 0.1632$ at $k = 7$ and $S^*_{\text{med},0.0322} = 0.5144$ at $k = 21$, since a smaller β discounts distant datasets less. Section 24.3 says where these two values of β come from.

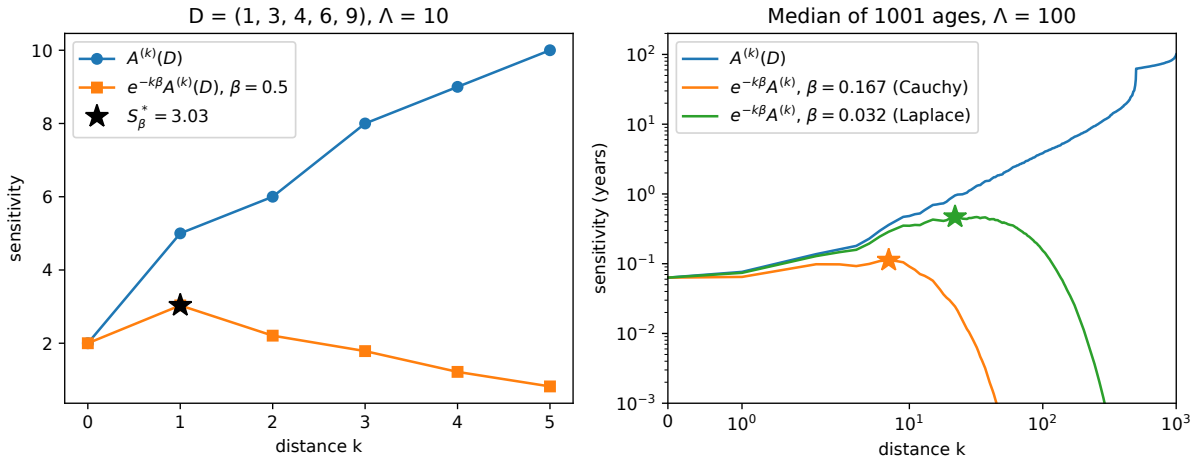


Figure 24.2.: Left: $A^{(k)}$ and $e^{-k\beta} A^{(k)}$ for the walkthrough dataset, with the star at $S^*_{0.5} = 3.03$. Right: the same for 1,001 synthetic ages (a fresh draw) with $\beta = 1/6$ and $\beta = 0.032$.

24.3. Admissible noise for smooth sensitivity

A smooth bound still changes by up to a factor e^{β}. This section asks which noise tolerates that, in the mechanism

$$M(D) = f(D) + \frac{S(D)}{\alpha} Z, \tag{24.10}$$

where Z has a fixed density h and the constant $\alpha > 0$ sets how many noise scales one record's shift of f is worth. The answer depends on the tail of h, because a light-tailed density and a stretched copy of it have a log-ratio that grows without bound, as in Example 24.1.

Theorem 24.1 (Pure DP with Cauchy-like noise)**.** *Let $h_\gamma(z) \propto 1/(1+|z|^\gamma)$ with $\gamma > 1$, a density whose tails decay like $|z|^{-\gamma}$; $\gamma = 2$ is the standard Cauchy. Let S be a β-smooth upper bound on LS_f, and let*

$$\gamma\alpha + (\gamma+1)\beta \le \varepsilon. \tag{24.11}$$

Then Equation 24.10 with $Z \sim h_\gamma$ is ε-DP.

Proof. The plan is to split the privacy loss at an output y into three pieces, for the scales, the centres and the stretch of the noise draw, and to bound each.

1. *Write out the privacy loss.* The density of $M(D)$ at y is $\frac{\alpha}{S(D)}h(z)$, where $z = \alpha\frac{y-f(D)}{S(D)}$ is the draw that produces y from D, and likewise for D' with z'. Substituting y in terms of z shows that z' is z shifted, then stretched:

$$z' = e^\kappa(z+v), \qquad e^\kappa = \frac{S(D)}{S(D')}, \quad v = \alpha\,\frac{f(D)-f(D')}{S(D)}. \tag{24.12}$$

   Inserting $h(z+v)$ in the middle of the ratio splits the log-ratio of the densities into three terms:

$$L_{D,D'}(y) = \underbrace{\ln\frac{S(D')}{S(D)}}_{\text{scale}} + \underbrace{\ln\frac{h(z)}{h(z+v)}}_{\text{sliding}} + \underbrace{\ln\frac{h(z+v)}{h(e^\kappa(z+v))}}_{\text{dilation}}. \tag{24.13}$$

2. *Size of the shift and the stretch.* Smoothness gives $|\kappa| \le \beta$. One record moves f by at most $\mathrm{LS}_f(D) \le S(D)$, so $|v| \le \alpha$, the only use of the upper-bound property.
3. *Scale term.* It is at most β, by smoothness.
4. *Sliding term.* Write $\psi(u) = \ln(1+u^\gamma)$, so that $-\ln h_\gamma(z) = \psi(|z|)$ plus a constant. Its slope $\gamma u^{\gamma-1}/(1+u^\gamma)$ is at most γ, because $u^{\gamma-1} \le 1+u^\gamma$ both for $u \le 1$ and for $u > 1$. The arguments $|z|$ and $|z+v|$ differ by at most $|v|$, so by the mean value theorem the term is at most $\gamma|v| \le \gamma\alpha$.
5. *Dilation term.* With $w = |z+v|$ and $\kappa \ge 0$, the inequality $(1+ca)/(1+a) \le c$ for $c \ge 1$ gives

$$\frac{h(w)}{h(e^\kappa w)} = \frac{1+e^{\kappa\gamma}w^\gamma}{1+w^\gamma} \le e^{\kappa\gamma} \tag{24.14}$$

   for every w, however far in the tail; that is what the power-law tail buys. For $\kappa < 0$ the argument moves towards 0, where h is larger, and the ratio is at most 1. So the term is at most $\gamma\beta$.
6. *Add up.* The loss is at most $\beta + \gamma\alpha + \gamma\beta \le \varepsilon$ at every y, which is pure ε-DP.

□

Nissim, Raskhodnikova & Smith (2007, Lemma 2.7) take $\alpha = \beta = \varepsilon/(2(\gamma+1))$, which for the Cauchy gives the release $f(D) + \frac{6}{\varepsilon}S^*_{f,\varepsilon/6}(D)\,Z$ and, by Equation 24.11, a loss of at most $0.8333\,\varepsilon$. For Laplace noise the dilation term grows with the draw, so the condition gains a term that charges for draws above $\ln(1/\delta)$.

Theorem 24.2 (Approximate DP with Laplace noise)**.** *Let $Z \sim \mathrm{Lap}(1)$, with density $\frac{1}{2}e^{-|z|}$, let S be a β-smooth upper bound on LS_f, let $\delta \in (0,1)$, and let*

$$\beta + e^{\beta}\alpha + (e^{\beta}-1)\ln(1/\delta) \le \varepsilon. \tag{24.15}$$

Then Equation 24.10 is (ε,δ)-DP.

Proof.

1. *Reuse the split.* Steps 1 to 3 of the proof of Theorem 24.1 did not use the shape of h. So the loss is the scale term, at most β, plus $\ln(h(z)/h(z'))$ with $z' = e^{\kappa}(z+v)$, $|\kappa| \le \beta$ and $|v| \le \alpha$.
2. *Bound the shape term by the draw.* For the Laplace density that log-ratio is $|z'| - |z|$. The triangle inequality and the bounds on κ and v give

$$\ln \frac{h(z)}{h(z')} \le e^{\beta}(|z|+\alpha) - |z| = (e^{\beta}-1)|z| + e^{\beta}\alpha. \tag{24.16}$$

3. *The rare draws pay δ.* A $\mathrm{Lap}(1)$ variable exceeds $\ln(1/\delta)$ in absolute value with probability exactly δ. Outside that event the loss is at most the left side of Equation 24.15, hence at most ε. By Lemma 20.2 the smallest δ that works for a pair is at most the probability that the loss exceeds ε, so the mechanism is (ε,δ)-DP.

□

Nissim, Raskhodnikova & Smith (2007, Lemma 2.9) split the budget as $\alpha = \varepsilon/2$ and $\beta = \varepsilon/(2\ln(2/\delta))$. With either split β must shrink like $1/\ln(1/\delta)$, because the term $(e^{\beta}-1)\ln(1/\delta)$ of Equation 24.15 must stay below ε. That term is there because Laplace noise cannot give pure DP. Pure DP needs h at a point and at a stretched point to stay within a bounded ratio however far out the point is, and applying that repeatedly shows that h can decay at most like a power. A density that tolerates the shift and the rescaling is called **admissible**.

Heavy tails have a price. A standard Cauchy variable has $\Pr[|Z| \le t] = \frac{2}{\pi}\arctan t$, so $|Z|$ has median 1, but about one release in 16 is off by more than ten times that ($\Pr[|Z| > 10] = 0.0635$, against $e^{-10} \approx 4.5 \times 10^{-5}$ for Laplace noise).

Example 24.3. Release the median of the 1,001 ages with $\varepsilon = 1$, and compare the options by their median absolute error, which for noise bZ is b for the Cauchy and $b\ln 2$ for the Laplace. With global sensitivity it is $100\ln 2 = 69.31$ years. Cauchy noise with $\alpha = \beta = 1/6$ uses $S^*_{\mathrm{med},1/6} = 0.1632$, barely above the local sensitivity, so its error is $6 \times 0.1632 = 0.979$ (simulated: 0.980). Laplace noise with $\delta = 10^{-6}$ and $\alpha = 1/2$ allows $\beta = 0.0322$ by Equation 24.15, close to the 0.0345 of Nissim, Raskhodnikova & Smith. Then $S^*_{\mathrm{med},0.0322} = 0.5144$, the noise scale is 1.029 and the error is 0.713.

Both smooth releases are 70 to 100 times more accurate than the global one, and the gain grows with n (Figure 24.3). Below about 20 points the smooth noise is worse, because a handful of changes can then open a gap of nearly the whole range.

24.4. Propose-test-release

Smooth sensitivity needs a formula like Equation 24.9, and those are known for only a few functions. Dwork & Lei (2009) take a cruder route. Propose a bound b on the local sensitivity, check privately that the data are far from any dataset where it fails, and only then release with noise of scale b/ε. The check counts how many changes it takes before something bad happens, and the proof needs such a count to have sensitivity 1.

Lemma 24.2 (Distances to a set have sensitivity 1)**.** *For any set $\mathcal{B}$ of datasets, let* $\operatorname{dist}(D, \mathcal{B}) = \min_{D'' \in \mathcal{B}} d(D, D'')$ *be the number of changes needed to reach the nearest of them. Then* $|\operatorname{dist}(D, \mathcal{B}) - \operatorname{dist}(D', \mathcal{B})| \le 1$ *for all $D \sim D'$.*

Proof.

1. *One direction.* A path from D' into $\mathcal{B}$ can go through D first, so $d(D', D'') \le 1 + d(D, D'')$ for every D''. Minimizing over $D'' \in \mathcal{B}$ gives $\operatorname{dist}(D', \mathcal{B}) \le 1 + \operatorname{dist}(D, \mathcal{B})$.
2. *The other.* Exchange D and D'.

□

So a distance to a set can be released with $\mathrm{Lap}(1/\varepsilon)$ noise, and propose-test-release compares it with a threshold that noise alone lifts a distance of 0 above only with probability $\delta/2$.

Theorem 24.3 (Propose-test-release)**.** *Fix a proposed bound $b > 0$, $\varepsilon > 0$ and $\delta \in (0, 1)$. Let* $\operatorname{dist}_b(D) = \min\{d(D, D'') : \mathrm{LS}_f(D'') > b\}$ *be the number of records that must change before the local sensitivity can exceed b.* ***Propose-test-release*** *outputs*

- $\perp$ *if* $\operatorname{dist}_b(D) + \mathrm{Lap}(1/\varepsilon) \le \ln(1/\delta)/\varepsilon$*;*
- $f(D) + \mathrm{Lap}(b/\varepsilon)$ *otherwise.*

It is $(2\varepsilon, \delta/2)$-DP. For the median, $\operatorname{dist}_b(D) = \min\{k : A^{(k)}(D) > b\}$ *with $A^{(k)}$ from Equation 24.9.*

Proof. The plan is to split neighbouring pairs into two cases. If the proposal is right for the pair, the release is an ordinary Laplace mechanism. If it is wrong, both datasets sit at an unstable point and the test almost always refuses.

1. *The test is ε-DP.* dist_b is a distance to a set, so it has sensitivity 1 by Lemma 24.2, and adding $\mathrm{Lap}(1/\varepsilon)$ is the Laplace mechanism. Whether the test passes is post-processing of that noisy value.
2. *Case 1: $|f(D) - f(D')| \le b$.* The release $f(D) + \mathrm{Lap}(b/\varepsilon)$ has privacy loss at most ε by Theorem 20.1. The output is a function of two independent draws, each with loss at most ε, so by basic composition the mechanism is 2ε-DP on this pair.
3. *Case 2: $|f(D) - f(D')| > b$.* Then both local sensitivities exceed b, because each neighbour is in the other's maximum, so $\operatorname{dist}_b(D) = 0$. The test passes only if $\mathrm{Lap}(1/\varepsilon)$ exceeds $\ln(1/\delta)/\varepsilon$, which has probability $\frac{1}{2}e^{-\ln(1/\delta)} = \delta/2$.

4. *Bound case 2.* For any set S of outputs, $M(D)$ lands in S by refusing, if $\perp \in S$, or by releasing, with probability at most $\delta/2$. The refusal is post-processing of the ε-DP test, so its probability is at most $e^{\varepsilon} \le e^{2\varepsilon}$ times that under D', and

$$\Pr[M(D) \in S] \le e^{2\varepsilon} \Pr[M(D') \in S] + \delta/2. \tag{24.17}$$

5. *Combine.* Case 1 satisfies the same inequality, so every pair does.

□

When the data are stable the noise scale is set by the proposal, so it reveals nothing, and otherwise $\perp$ says only that the data were probably near an unstable dataset. The threshold is large, $\ln(10^6) = 13.82$ at $\varepsilon = 1$ and $\delta = 10^{-6}$, so the data must be stable against more than a dozen changes. The method needs only the distance to instability, which is often easier to bound than $S^*_{f,\beta}$, but it pays 2ε and can refuse.

The proposal must not depend on the data. Setting b to twice the observed local sensitivity makes the noise scale a function of the data again, and the leak of Section 24.1 returns.

Example 24.4. On the 1,001 ages, propose $b = 1$ year. The first k with $A^{(k)} > 1$ is 21, so $\mathrm{dist}_1(D) = 21$, and the test passes with probability $1 - \frac{1}{2}e^{-(21-13.82)} = 0.999621$. The release then has median absolute error $\ln 2 = 0.693$ years, at a total budget of 2. On 101 ages $\mathrm{dist}_1 = 1$, and the pass probability is $\frac{1}{2}e^{-(13.82-1)} = 1.36 \times 10^{-6}$. Figure 24.3 (right) shows it switching from 0 to 1 between $n = 500$ and $n = 1000$.

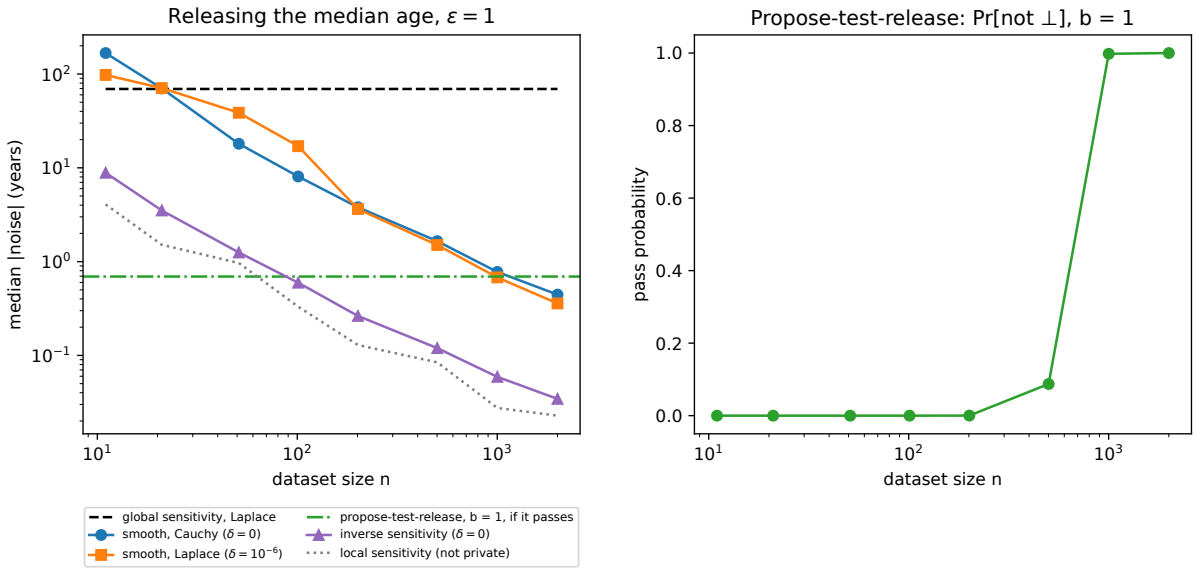


Figure 24.3.: Left: median absolute noise of each release of the median age with $\varepsilon = 1$, averaged over 10 synthetic datasets per n; the dotted line is the local sensitivity. Right: the probability that propose-test-release with $b = 1$ answers.

24.5. Inverse sensitivity

Smooth sensitivity and propose-test-release spend their effort on choosing a noise scale safely. Inverse sensitivity (Asi & Duchi 2020) has no noise scale. It samples each candidate output with a probability that falls exponentially in the number of records you would have to change to make it the true answer. That count is a distance to a set, so by Lemma 24.2 no smooth bound, test or refusal is needed.

Definition 24.3 (Inverse sensitivity and its mechanism)**.** The **inverse sensitivity** of f at D toward an output y is the number of records you must change to make f equal to y,

$$\operatorname{len}_f(D, y) = \min\{d(D, D'') : f(D'') = y\}. \tag{24.18}$$

The **inverse-sensitivity mechanism** samples y from $[0, \Lambda]$ with the density

$$p_D(y) = \frac{e^{-\varepsilon \operatorname{len}_f(D,y)/2}}{\int_0^\Lambda e^{-\varepsilon \operatorname{len}_f(D,y')/2}\,\mathrm{d}y'}. \tag{24.19}$$

This is the exponential mechanism with the score $-\operatorname{len}_f(D, y)$, and its proof is the exponential mechanism's.

Theorem 24.4 (The inverse-sensitivity mechanism is pure DP)**.** *If every output in* $[0, \Lambda]$ *is the value of* f *on some dataset of size* n*, so that every count* $\operatorname{len}_f(D, y)$ *is finite, then the mechanism of Equation 24.19 is* ε*-DP.*

Proof.

1. *The penalty has sensitivity 1.* For each y, $\operatorname{len}_f(D, y)$ is the distance from D to the non-empty set of datasets on which f equals y, so by Lemma 24.2 it changes by at most 1 between neighbours.
2. *Each weight moves by at most* $e^{\varepsilon/2}$*.* So the weight of each output changes by a factor between $e^{-\varepsilon/2}$ and $e^{\varepsilon/2}$, and so does the normalizer, an integral of such weights.
3. *Pointwise, hence pure DP.* The density is a weight over the normalizer, so $p_D(y)/p_{D'}(y) \le e^{\varepsilon/2} \cdot e^{\varepsilon/2} = e^{\varepsilon}$ at every y.

□

The factor 2 is needed because a weight and the normalizer can move in opposite directions. For the median the count has a closed form, and the density is a staircase, flat on each gap between sorted values.

Proposition 24.5 (The count for the median)**.** *Let* n *be odd, so that the median has rank* $m = (n + 1)/2$*, with the sorting and padding of Lemma 24.1. For a candidate* y *in the open gap* (x_i, x_{i+1})*,*

$$\operatorname{len}_{\text{med}}(D, y) = \begin{cases} m - i & \text{if } i < m, \\ i - m + 1 & \text{if } i \ge m. \end{cases} \tag{24.20}$$

Proof.

1. *Above the median.* Take $i \geq m$, so i values of D lie below y. A dataset with median y has at most $m-1$ values below y, and each change removes at most one, so at least $i-m+1$ changes are needed. That many suffice: move one value from below y to y and $i-m$ others to Λ.
2. *Below the median.* The reflection $x \mapsto \Lambda - x$ turns the gap with index $i < m$ into the gap with index $n - i \geq m$, and step 1 gives $(n-i) - m + 1 = m - i$, since $n + 1 = 2m$.

□

There is no noise scale to leak. Where the data are dense, the outputs a few steps from the median are also a short distance from it, so the error is a few gaps times a factor that grows with $1/\varepsilon$. Asi & Duchi (2020) show that, up to logarithmic factors, no ε-DP mechanism does much better on a dataset and its close neighbours, so the method is near-optimal dataset by dataset.

It has two limits. The outer gaps $(0, x_1)$ and (x_n, Λ) can be long and are m steps away, which is negligible for a thousand points but not for eleven. And the count must be computed: closed forms like Equation 24.20 are known for medians, quantiles and a few other statistics.

Example 24.5. Take the walkthrough dataset $D = (1, 3, 4, 6, 9)$ on $[0, 10]$ with $\varepsilon = 1$. Table 24.3 lists its six gaps, their counts from Equation 24.20, and their probabilities, each its length times $e^{-\text{len}/2}$ over the total. The two gaps next to the median are one change away and together get probability $0.1477 + 0.2955 = 0.4432$.

Table 24.3.: The inverse-sensitivity release of the median of $D = (1, 3, 4, 6, 9)$ on $[0, 10]$ with $\varepsilon = 1$: each gap between sorted values, the number of changes that make a point of it the median, and its probability.

Gap	$(0, 1)$	$(1, 3)$	$(3, 4)$	$(4, 6)$	$(6, 9)$	$(9, 10)$
Length	1	2	1	2	3	1
$\text{len}_{\text{med}}(D, y)$	3	2	1	1	2	3
Probability	0.0544	0.1792	0.1477	0.2955	0.2688	0.0544

On the 1,001 ages the median absolute error, computed exactly from the density, is 0.0783 years (simulated: 0.0784). That is pure DP with no refusals, twelve times more accurate than smooth sensitivity with Cauchy noise (0.979), and even below the local sensitivity 0.1386, because a typical release lands only a few short gaps from the median. On 101 ages the error grows to 0.696 years, because the gaps are ten times wider.

24.6. Stability-based release

Some statistics take discrete values, such as the most common diagnosis or the model a selection procedure picks. On most datasets one record cannot change them, so their local sensitivity is 0. Propose-test-release with $b = 0$ then gives the **stability-based release**: test privately that the data are far from any dataset where one change flips the answer, and if so release the answer exactly (Dwork & Roth 2014, ch. 7; Thakurta & Smith 2013).

Theorem 24.5 (Stability-based release)**.** *Let f be a discrete statistic, and let s be a* ***stability score****: a function of the data with sensitivity 1 that equals 0 on every dataset that has a neighbour where f gives a different answer. The stability-based release outputs*

$$M(D) = \begin{cases} \perp & \text{if } s(D) + \mathrm{Lap}(1/\varepsilon) \le \ln(1/\delta)/\varepsilon, \\ f(D) & \text{otherwise,} \end{cases} \tag{24.21}$$

and it is $(\varepsilon, \delta/2)$-DP. By Lemma 24.2, the distance to the set of datasets that have a neighbour with a different answer is one such score.

Proof. The proof is that of Theorem 24.3, except that nothing but the test depends on the data when the answers agree.

1. *The test is ε-DP*, as in step 1 of the proof of Theorem 24.3.
2. *Neighbours with the same answer.* The release is the same function of the test's outcome on both, so it is post-processing of an ε-DP output. The second ε that propose-test-release paid for its noisy answer is not needed.
3. *Neighbours with different answers.* Then $s(D) = s(D') = 0$, and the test passes with probability $\delta/2$. Step 4 of the proof of Theorem 24.3, with e^{ε} in place of $e^{2\varepsilon}$, gives $\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \delta/2$ for every set S.

□

For the **mode** of a categorical dataset, the most frequent value with ties broken by a fixed order, a score can be read off the lead of the winner.

Proposition 24.6 (A stability score for the mode)**.** *Let the lead g be the top count minus the second-highest count. Under replacement,*

$$s(D) = \max\big(0,\ \lceil g/2 \rceil - 1\big) \tag{24.22}$$

is a stability score for the mode.

Proof.

1. *Sensitivity 1.* One replacement lowers one count by at most 1 and raises another by at most 1, so the lead moves by at most 2. Halving and rounding up turns that into a move of at most 1, since $\lceil (g+2)/2 \rceil = \lceil g/2 \rceil + 1$.
2. *Zero where the answer can flip.* If $g \ge 3$, after one change the mode keeps at least $c_1 - 1$ records, where c_1 is its count, and every other value has at most $c_2 + 1 \le c_1 - 2$, where c_2 is the second-highest count. So a dataset whose neighbour has a different mode has $g \le 2$, and then $s = 0$.

□

The release is exact, and the cost of privacy is the chance of a refusal, which depends on the lead and not on the number of possible values. The mode must lead by about $2\ln(1/\delta)/\varepsilon$ records, whether there are ten categories or a billion. Under pure DP, the packing lower bound of Lower bounds needs a lead of about $\ln N/\varepsilon$ to choose among N values, so this is how approximate DP gets around it. An exact release next to a flip happens with probability $\delta/2$, and that is what δ pays for.

Example 24.6. The median is not discrete, so take a mode, with $\varepsilon = 1$ and $\delta = 10^{-6}$ (threshold 13.82). Leads of 10, 30 and 40 records give scores 4, 14 and 19, and the release answers with probability 2.73×10^{-5}, 0.5842 and 0.9972. So a lead of 35 is needed for probability 0.95, whatever the number of categories. When a neighbour adds or removes a record instead, one change moves the lead by 1 rather than 2, so the lead needed halves.

24.7. Subsample-and-aggregate

Sometimes you cannot analyse f at all, because it is a trained model or a pipeline you did not write. Subsample-and-aggregate (Nissim, Raskhodnikova & Smith 2007; Smith 2011) runs f as a black box on disjoint pieces of the data and privately combines the answers. One record can change only one answer, so the aggregator needs to be private only with respect to one input.

Theorem 24.6 (Subsample-and-aggregate)**.** *Split the n records into B disjoint blocks of n/B records by a partition fixed in advance. Compute the block answers $a_j = f(\text{block } j)$, clamp each to $[0, R]$, and release*

$$M(D) = \frac{1}{B}\sum_{j=1}^{B} \operatorname{clip}_{[0,R]}(a_j) + \operatorname{Lap}\Big(\frac{R}{B\varepsilon}\Big). \tag{24.23}$$

This is ε-DP for any f. When the clipping is inactive and the blocks are independent samples, its mean squared error as an estimate of a target θ is, with a the output of f on one block,

$$\underbrace{\big(\mathbb{E}[a] - \theta\big)^2}_{\text{bias of } f \text{ on } n/B \text{ records}} + \underbrace{\frac{\operatorname{Var}(a)}{B}}_{\text{sampling}} + \underbrace{2\Big(\frac{R}{B\varepsilon}\Big)^2}_{\text{noise}}. \tag{24.24}$$

Proof. The privacy part bounds the sensitivity of the average; the error comes from squaring a sum of three independent pieces.

1. *One record touches one block.* The partition does not depend on the values, so replacing a record changes at most one block answer.
2. *The aggregate has sensitivity R/B.* After clamping, that answer moves by at most R, and the average divides by B.
3. *Privacy.* Laplace noise of scale $R/(B\varepsilon)$ on a quantity of sensitivity R/B is ε-DP by Theorem 20.1.
4. *Error.* The released value minus θ is the bias $\mathbb{E}[a] - \theta$, plus the mean-zero average of the $a_j - \mathbb{E}[a]$, with variance $\operatorname{Var}(a)/B$, plus the Laplace noise, with variance $2(R/(B\varepsilon))^2$. The cross terms of the square each contain an independent mean-zero factor and average to zero, which leaves Equation 24.24.

□

The noise falls like $1/B$, while the sampling term hardly moves, because a block estimate has about B times the variance of f on all records and averaging divides by B again. The bias of f on small blocks grows with B, so the best B balances bias and noise. The partition must not depend on the values, or one record could reshuffle many records between blocks.

The proof never looks inside f, so the method suits trained models, as in private aggregation of teacher ensembles (Papernot et al. 2017). But the range R is still a global bound, and a loose R means large noise. Smith (2011) shows that for asymptotically normal estimators a version of the method loses nothing in the leading-order error.

Example 24.7. The running example has no black-box statistic, so take one: estimate the rate $\lambda = 1$ of $n = 10{,}000$ independent exponential inter-arrival times with $\varepsilon = 1$. The estimate $1/\bar{x}$, one over the sample mean, has infinite global sensitivity, because tiny times make it huge.

On a block of $n_b = n/B$ records the block sum G is gamma with shape n_b and rate λ, and the block estimate is $a = n_b/G$. Integrating $1/g$ and $1/g^2$ against the gamma density leaves gamma densities with shapes $n_b - 1$ and $n_b - 2$, which gives

$$\mathbb{E}[a] = \frac{n_b \lambda}{n_b - 1}, \qquad \text{bias } \frac{\lambda}{n_b - 1}, \qquad \operatorname{Var}(a) = \frac{n_b^2 \lambda^2}{(n_b - 1)^2 (n_b - 2)}. \tag{24.25}$$

With $R = 5$, Table 24.4 and Figure 24.4 show the trade-off; at $B = 10$ the noise standard deviation is $\sqrt{2} \times 0.5 = 0.7071$.

Table 24.4.: Error of subsample-and-aggregate for the rate of an exponential distribution, $n = 10{,}000$, $\lambda = 1$, $R = 5$, $\varepsilon = 1$: the three terms of Equation 24.24, their combination, and a simulation with 1,000 repetitions.

Blocks B	Block size	Bias	Sampling sd	Noise sd	Root-mean-square error	Simulated
10	1,000	0.0010	0.0100	0.7071	0.7072	0.7205
100	100	0.0101	0.0102	0.0707	0.0722	0.0733
250	40	0.0256	0.0105	0.0283	0.0396	0.0401
1,000	10	0.1111	0.0124	0.0071	0.1120	0.1114

Among the divisors of n shown, the best B is 250, with error 0.0396, where bias and noise are of similar size. A non-private estimate has standard deviation about $\lambda/\sqrt{n} = 0.0100$, so privacy at $\varepsilon = 1$ costs a factor of four, for an estimator whose sensitivity you never had to bound.

24.8. Clipping

The mean is the most common statistic of all, and on unbounded data its global sensitivity is infinite. **Clipping** replaces each value x by $\min(x, C)$ before averaging, so the sensitivity becomes C/n. But clipping changes the quantity you estimate, and the choice of C trades that bias against the noise. Applied to per-example gradients, the same move drives private model training in later parts.

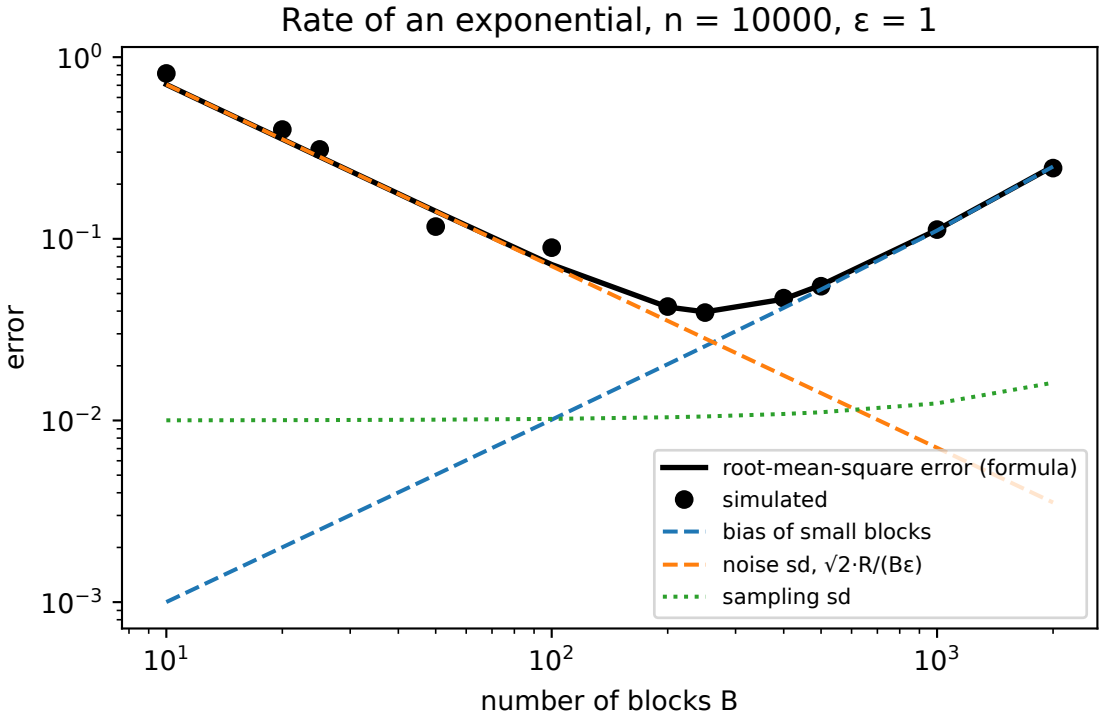


Figure 24.4.: Root-mean-square error of subsample-and-aggregate for the exponential rate against the number of blocks B, with its bias, noise and sampling components.

Theorem 24.7 (The clipped mean and its best clipping norm)**.** *Take non-negative data $X_1, \dots, X_n$ drawn independently from a distribution with mean μ, and write $(x - C)_+ = \max(x - C, 0)$ for the part of a value above C. The clipped mean*

$$M(D) = \frac{1}{n}\sum_{i=1}^{n} \min(X_i, C) + \mathrm{Lap}\Big(\frac{C}{n\varepsilon}\Big) \tag{24.26}$$

is ε-DP, and its mean squared error as an estimate of μ is

$$\mathrm{err}(C)^2 = \underbrace{\mathbb{E}[(X - C)_+]^2}_{\text{bias}^2} + \underbrace{\frac{\mathrm{Var}(\min(X, C))}{n}}_{\text{sampling}} + \underbrace{\frac{2C^2}{n^2\varepsilon^2}}_{\text{noise}}. \tag{24.27}$$

The clipping norm that minimizes it solves

$$\Pr[X > C]\Big(\mathbb{E}[(X - C)_+] - \frac{C - \mathbb{E}[\min(X, C)]}{n}\Big) = \frac{2C}{n^2\varepsilon^2}. \tag{24.28}$$

Proof. The privacy part is one line, the error splits as in Equation 24.24, and the best C is where the derivative of the error vanishes.

1. *Sensitivity.* Each clipped value lies in $[0, C]$, so one replacement moves the mean by at most C/n, and the noise makes it ε-DP by Theorem 20.1.
2. *The bias.* Clipping removes exactly the excess, $x - \min(x, C) = (x - C)_+$, so $\mu - \mathbb{E}[\min(X, C)] = \mathbb{E}[(X - C)_+]$.
3. *Error decomposition.* As in step 4 of the proof of Theorem 24.6, the error is the bias, a mean-zero sampling term with variance $\mathrm{Var}(\min(X, C))/n$ and independent noise with variance $2(C/(n\varepsilon))^2$, which gives Equation 24.27.
4. *Derivatives in C.* Raising C slightly raises $\min(X, C)$ by that amount exactly when $X > C$, so $\mathbb{E}[\min(X, C)]$ has derivative $\Pr[X > C]$ and $\mathbb{E}[\min(X, C)^2]$ has derivative $2C\Pr[X > C]$. By the chain rule the three terms have derivatives $-2\,\mathbb{E}[(X - C)_+]\Pr[X > C]$, then $\frac{2}{n}\Pr[X > C](C - \mathbb{E}[\min(X, C)])$, then $4C/(n^2\varepsilon^2)$.
5. *Set the sum to zero* and divide by 2, which is Equation 24.28.

□

The bias does not shrink with n, but the noise does, so the best C grows with n and moves into the tail. Clipping from above always underestimates. Clipping to an interval around a centre cuts both sides, and the two cuts cancel for a symmetric distribution, which is why Private mean estimation first estimates the centre privately.

Choosing C is itself a data-dependent decision. Setting it to the observed maximum or the empirical 99th percentile brings back the leak of Section 24.1. Choose C from domain knowledge, or spend part of the budget on a private quantile by inverse sensitivity (Section 24.5).

Example 24.8. The ages are already bounded, so take incomes, in thousands, with $\ln X \sim \mathcal{N}(\ln 40, 0.8^2)$. The median is 40 and the mean is $e^{\ln 40 + 0.64/2} = 55.085$, pulled up by the heavy right tail. The averages of X and X^2 below C that Equation 24.27 needs have a closed form in the normal distribution function (Probability distributions), and Table 24.5 uses it with $n = 1000$ and $\varepsilon = 1$.

Table 24.5.: Error of the clipped private mean of log-normal incomes, $n = 1000$, $\varepsilon = 1$, for four clipping norms: the three terms of Equation 24.27, their combination, and a simulation with 5,000 repetitions.

Clipping norm C	$\Pr[X > C]$	Bias	Sampling sd	Noise sd	Root-mean-square error	Simulated
40 (the median)	0.5000	23.415	0.335	0.057	23.417	23.411
257.23 (99th percentile)	0.0100	0.924	1.462	0.364	1.767	1.747
331.34 (optimal)	0.0041	0.438	1.538	0.469	1.666	1.649
1000	0.0000	0.006	1.645	1.414	2.169	2.134

Clipping at the median cuts back half the incomes, and the bias of 23 dwarfs everything. The optimum, $C^* = 331.34$, cuts off 0.4% of the population and has error 1.666, only 1% above the standard deviation 1.649 of the unclipped, non-private mean. That the simulated error also prints as 1.649 is a coincidence of 5,000 repetitions, whose own standard error is about 1%. Above the optimum the noise takes over (Figure 24.5, left). The optimum grows with n: 177.5 at $n = 100$, 557.3 at $n = 10{,}000$ and 876.3 at $n = 100{,}000$ (Figure 24.5, right).

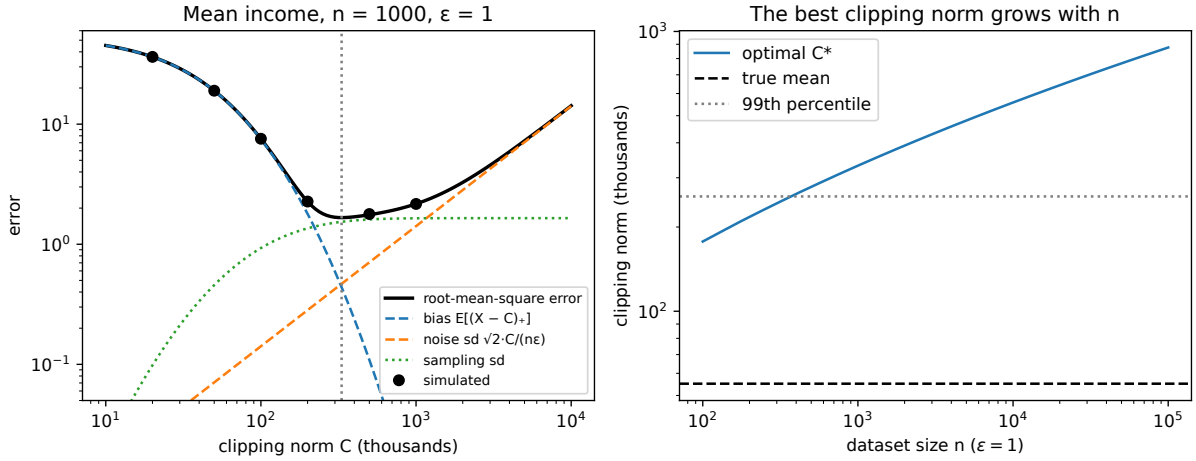


Figure 24.5.: Left: root-mean-square error of the clipped mean of incomes against C, with its components; the dotted line marks $C^* = 331$. Right: the optimal clipping norm against the dataset size.

24.9. Code

The script code/sensitivity.py computes every number in this chapter and draws its figures. From the repository root:

```
uv run python code/part03-dp-foundations/sensitivity.py
```

Table 24.6.: The script's output with seed 0.

Experiment	Result
Local-sensitivity noise, Table 24.1	smallest δ at $\varepsilon = 1$: 0.477 and 0.215
$A^{(k)}$, Table 24.2	Formula equals brute force; $S^*_{0.5} = 3.03$
Noise parameters	Cauchy loss bound 0.8333; Laplace $\beta = 0.0322$
Median of the ages, median error	Cauchy 0.979 (simulated 0.980); Laplace 0.713; global 69.31
Propose-test-release, $b = 1$, pass probability	0.999621 at $n = 1001$; 1.36×10^{-6} at $n = 101$
Subsample-and-aggregate	Best $B = 250$, error 0.0396
Inverse sensitivity, median error	0.0783 (simulated 0.0784); 0.696 on 101 ages
Stable mode, pass probability	0.5842 at a lead of 30; lead 35 for 0.95
Clipped mean of incomes	$C^* = 331.34$, error 1.666 (simulated 1.649)

tests/test_sensitivity.py checks that the output still matches this table.

24.10. Summary

Table 24.7.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 24.1	The most one record can move f from D	Never a noise scale on its own
Proposition 24.1	The larger gap next to the median; globally Λ	Medians and quantiles
Proposition 24.2	Noise scaled to LS_f is not private	Ask what a data-dependent scale reveals
Proposition 24.3	$S^*_{f,\beta}$ is the smallest β-smooth upper bound	Data-dependent noise with a proof
Proposition 24.4	$A^{(k)}$ is the widest of $k + 2$ windows	Pays off once n is in the hundreds
Theorem 24.1	Cauchy-like noise is ε-DP under Equation 24.11	Pure DP needs polynomial tails
Theorem 24.2	Laplace noise is (ε, δ)-DP under Equation 24.15	Light tails need δ and $\beta \approx \varepsilon/(2\ln(1/\delta))$
Lemma 24.2	A distance to a set of datasets has sensitivity 1	Any "how many changes until something bad" count

Result	Statement	Use
Theorem 24.3	Test the distance to instability, then add $\mathrm{Lap}(b/\varepsilon)$	Statistics with no formula for S^*; fix b before looking
Theorem 24.4	Exponential mechanism on the number of changes	Medians, quantiles, clipping norms
Theorem 24.5	Test a stability score, then release $f(D)$ exactly	The mode once its lead exceeds about $2\ln(1/\delta)/\varepsilon$
Theorem 24.6	Clamped average of block answers plus Laplace noise	Black-box estimators; teacher ensembles in Part 5
Theorem 24.7	Clipped mean plus Laplace noise; best C from Equation 24.28	Means and gradient clipping; never choose C from the raw data

24.11. References

- Nissim, Raskhodnikova & Smith, "Smooth Sensitivity and Sampling in Private Data Analysis" (STOC 2007; full version on the authors' pages): local and smooth sensitivity, admissible noise (Lemmas 2.6–2.9), the median (Proposition 3.4) and the original sample-and-aggregate framework
- Dwork & Lei, "Differential Privacy and Robust Statistics" (STOC 2009): propose-test-release, and private robust estimators of scale, location and regression built on it
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 7: propose-test-release, stability-based release and subsample-and-aggregate in textbook form
- Near & Abuah, *Programming Differential Privacy*, chapter on local sensitivity: the same ideas with short code
- Asi & Duchi, "Instance-optimality in Differential Privacy via Approximate Inverse Sensitivity Mechanisms" (NeurIPS 2020): the inverse-sensitivity mechanism and its near-optimality on every dataset
- Thakurta & Smith, "Differentially Private Feature Selection via Stability Arguments, and the Robustness of the Lasso" (COLT 2013): stability-based release, used for model selection
- Smith, "Privacy-Preserving Statistical Estimation with Optimal Convergence Rates" (STOC 2011): subsample-and-aggregate for asymptotically normal estimators
- Bun & Steinke, "Average-Case Averages: Private Algorithms for Smooth Sensitivity and Mean Estimation" (NeurIPS 2019): better noise distributions for smooth sensitivity, under concentrated DP
- Amin, Kulesza, Muñoz Medina & Vassilvitskii, "Bounding User Contributions: A Bias-Variance Trade-off in Differential Privacy" (ICML 2019): the clipping trade-off when users contribute many records
- Papernot, Abadi, Erlingsson, Goodfellow & Talwar, "Semi-supervised Knowledge Transfer for Deep Learning from Private Training Data" (ICLR 2017): subsample-and-aggregate for model training

25. Discrete and floating-point-safe mechanisms

Every privacy proof so far was a statement about real numbers, such as the proof in the Laplace and Gaussian mechanisms that compares two densities on the real line. A computer has only floating-point numbers, a finite set with uneven gaps, and random bits, so the program that "samples Lap(b)" produces a distribution that the proof says nothing about. Mironov (2012) showed that the gap matters: the textbook Laplace sampler has outputs that only one of two neighbouring datasets can produce, and an observer who sees one knows which dataset was used.

The one idea of this chapter is that **differential privacy (DP) is a property of the output distribution the code actually produces**, so the code must produce exactly the distribution the proof is about. One way there keeps floating point and rounds the output to a coarse grid, so that every output is reachable from every input. The other way samples integer noise for integer-valued queries exactly, from uniform random integers, which gives the discrete Laplace and the discrete Gaussian mechanisms.

The running example is a hospital's count of patients with a rare diagnosis, with sensitivity $\Delta_1 f = 1$ under add/remove neighbours, released at $\varepsilon = 1$. In a toy floating-point model of the Laplace mechanism, almost half of the releases name the true count with certainty. The discrete Laplace releases the exact count with probability 0.4621, yet no output identifies the count, and the discrete Gaussian at $\sigma = 1$ has $\delta = 0.1414$ at $\varepsilon = 1$, close to the continuous Gaussian's 0.1269.

25.1. Floating-point Laplace breaks differential privacy

The textbook way to sample Lap(b) is inverse cumulative distribution function (CDF) sampling. Draw U uniform on $(0, 1)$ and a fair sign $\xi \in \{-1, +1\}$, and output $\xi \cdot b \ln(1/U)$. The magnitude exceeds a level z exactly when $U < e^{-z/b}$, so it is exponential with mean b (Probability distributions), and the sign mirrors it into Lap(b). In floating point, U, the logarithm and the sum with $f(D)$ are each rounded. This section asks whether the rounded mechanism is still DP, and the answer is no, for every ε.

The tool that shows it also audits any implementation. If some outputs can only come from D, seeing one of them reveals D with certainty, and since no factor e^{ε} can pay for that, δ has to.

Proposition 25.1 (The support argument)**.** *Let M be (ε, δ)-DP, let $D \sim D'$ be neighbouring datasets, and write $\mathcal{Y}_D$ for the set of outputs that $M(D)$ can produce. Let $S = \mathcal{Y}_D \setminus \mathcal{Y}_{D'}$ be the outputs reachable from D but not from D'. Then, whatever ε is,*

$$\delta \geq \Pr[M(D) \in S]. \tag{25.1}$$

In particular, if $\Pr[M(D) \in S] > 0$, then M is not ε-DP for any finite ε.

Proof.

1. *Apply the definition to the event S.* The definition of (ε, δ)-DP in the definition of differential privacy, applied to the set S, says that

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{25.2}$$

   By the choice of S the multiplicative term is 0 however large e^{ε} is, which leaves Equation 25.1.
2. *Pure DP.* Pure ε-DP is the case $\delta = 0$, where Equation 25.1 forces $\Pr[M(D) \in S] = 0$. So a positive probability rules out every finite ε.

□

The floating-point sampler has such outputs because of how floats are spaced. A float with a p-bit significand (IEEE 754 doubles have $p = 53$) is $m \cdot 2^e$ for an integer $m < 2^p$, so floats are dense near 0 and sparse far from it. With $f(D) = 0$, the outputs near 0 are the fine-grained noise values themselves. With $f(D') = 1$, an output near 0 needs a noise value near -1, where floats are about 2^{-p} apart, and adding 1 keeps that coarse spacing. So many outputs of $M(D)$ near 0 are impossible under $M(D')$, and the mismatch of grids elsewhere works in both directions. In the narrow window of Figure 25.1 (left), 3 of the 9 outputs of $M(D)$ cannot come from $M(D')$, and 4 of the 10 outputs of $M(D')$ cannot come from $M(D)$.

Doubles have too many values to enumerate, so the chapter's script (Section 25.7) uses p-bit floats with p from 6 to 14. Its U is a real uniform rounded down to a p-bit float, the logarithm and the sum are each rounded to p bits, and every output's probability is computed exactly (Table 25.1).

Table 25.1.: The toy floating-point Laplace sampler with $b = 1$, $f(D) = 0$ and $f(D') = 1$. S is the set of outputs of $M(D)$ that $M(D')$ cannot produce, and S' the reverse.

Significand bits p	Outputs of $M(D)$	Outputs of $M(D')$	$\Pr[M(D) \in S]$	$\Pr[M(D') \in S']$
8	1,604	1,409	0.4795	0.2070
12	25,646	22,477	0.4791	0.2033
14	102,586	89,859	0.4826	0.2053

More precision does not help. The right panel of Figure 25.1 stays at about 48% from 6 to 14 bits, because adding bits refines both grids together. Mironov (2012) found the same effect at $p = 53$ in several DP libraries of the time. The leak is not in where the output lands on the real line but in which float it is, so printing the output to a few decimals would hide it. That is the idea behind snapping.

A second, smaller leak comes from the tails, which are cut off. A uniform made of 53 random bits, a positive multiple of 2^{-53}, is never below 2^{-53}, so the noise magnitude never exceeds $b \ln(2^{53}) \approx 36.74\, b$, and outputs between $f(D) + 36.74b$ and $f(D') + 36.74b$ are reachable from one input only. By Equation 25.1, their probability of about 2^{-53} is a floor on δ that rules out pure DP. Jin et al. (2022) found such leaks in libraries released after Mironov's paper.

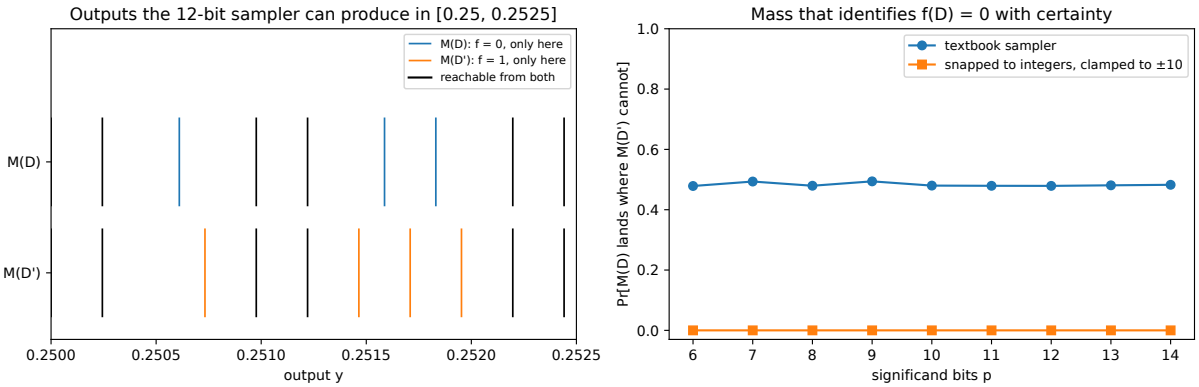


Figure 25.1.: The toy floating-point Laplace sampler with $b = 1$. Left: at 12 bits, the outputs of $f(D) = 0$ (top) and $f(D') = 1$ (bottom) in a narrow window, coloured by whether the other input can produce them. Right: $\Pr[M(D) \in S]$ against p, textbook and snapped (Section 25.2).

Example 25.1. The hospital releases its count with the textbook sampler and $b = 1$, so the exact mechanism would be ε-DP with $\varepsilon = \Delta_1 f/b = 1$. An analyst who knows that the count is 0 or 1, say whether one particular patient is in the data, checks whether the released float can be produced from 1. In the 12-bit toy the check identifies the count 0 with certainty 47.91% of the time, since $\Pr[M(D) \in S] = 0.4791$ in Table 25.1. By Proposition 25.1, no ε with a δ below 0.4791 describes the release, although the proof promised that one output moves the analyst's odds by at most a factor e.

25.2. The snapping mechanism

If the output must be a float, its low-order bits have to go. This section states the snapping mechanism of Mironov (2012), which removes them, gives its guarantee, whose proof is Mironov's, and checks it on the toy of Section 25.1.

In words, the mechanism clamps the true answer into a range $[-B, B]$, adds Laplace noise computed in floating point, rounds to the nearest multiple of a power of 2 at least as large as the noise scale, and clamps again. The clamps keep rounding errors small next to the grid.

Definition 25.1 (The snapping mechanism)**.** Let f have sensitivity 1 and let b be the noise scale. Let Λ be the smallest power of 2 with $\Lambda \geq b$, and let $[-B, B]$ be the clamping range. Write $\oplus$, $\otimes$ and LN for floating-point addition, multiplication and logarithm. Write $\lfloor \cdot \rceil_\Lambda$ for rounding to the nearest multiple of Λ, and clamp_B for the map onto $[-B, B]$. With a fair sign ξ and an ideal uniform double U^*, the **snapping mechanism** outputs

$$M(D) = \mathrm{clamp}_B\Big(\big\lfloor \mathrm{clamp}_B(f(D)) \oplus \xi \otimes b \otimes \mathrm{LN}(U^*) \big\rceil_\Lambda\Big). \tag{25.3}$$

The guarantee is the exact Laplace mechanism's $\varepsilon = 1/b$ plus two small terms that pay for floating-point error. One floating-point operation errs by at most a fixed fraction of the number it produces, the **unit roundoff** of doubles, written $\omega = 2^{-53}$. Clamping keeps every number below B in size, so the terms grow with $B\omega$.

Theorem 25.1 (Privacy of the snapping mechanism)**.** *If $b < B < 2^{46}b$, the snapping mechanism is ε-DP with*

$$\varepsilon = \frac{1 + 12B\omega}{b} + \frac{2^{-49}B}{b}, \qquad \omega = 2^{-53}. \tag{25.4}$$

The proof is Mironov's (2012, Theorem 1 and §5). Clamping the input keeps every sum where floats are at most about $B\omega$ apart. Rounding makes the output one of the multiples of Λ in $[-B, B]$, the same set for every input, so Proposition 25.1 has nothing to bite on. In exact arithmetic, each grid point collects an interval of noise values. The two inputs give it the probabilities of two intervals shifted by at most 1, whose ratio is at most $e^{1/b}$, as post-processing of the Laplace mechanism (Post-processing, group privacy and composition). Floating-point error moves each interval's endpoints by a relative amount of order ω, which gives the extra terms.

The price is accuracy. Rounding to Λ, which is below $2b$, adds up to $\Lambda/2 \le b$ of error, and clamping biases answers near $\pm B$. The extra terms in ε are tiny, about 3.1×10^{-14} with $b = 1$ and $B = 10$. The guarantee assumes a correctly behaved logarithm and an ideal uniform U^*, which a library has to supply.

The snapped toy is measured by the **privacy loss** of the definition of differential privacy, the log of the ratio of an output's probabilities under the two inputs:

$$L_{D,D'}(y) = \ln \frac{\Pr[M(D) = y]}{\Pr[M(D') = y]}. \tag{25.5}$$

That chapter showed that for discrete outputs, pure ε-DP is the same as $|L_{D,D'}(y)| \le \varepsilon$ at every output y.

Example 25.2. Snap the toy sampler of the running example to integers, $\Lambda = 1$ for $b = 1$, and clamp to $[-10, 10]$. Both inputs now reach the same 21 outputs, the integers from -10 to 10, so $\Pr[M(D) \in S] = 0$ at every precision (Figure 25.1, right). The largest privacy loss over those outputs is 1.0010 at $p = 12$ and 1.0002 at $p = 14$, against $\varepsilon = 1$ for the exact mechanism, and the excess shrinks with precision as the ω terms predict, with $\omega = 2^{-p}$ in the toy.

25.3. An exact coin with an exponential bias

Both exact samplers below need a coin that comes up 1 with probability $e^{-\lambda}$ for a rational $\lambda \ge 0$. The obvious code, `random() < exp(-lam)`, is inexact twice over, because `exp` rounds and `random()` has only 53 bits. The exact samplers use one primitive instead, a uniform integer in $\{0, 1, \dots, d-1\}$, which Python's `random.Random.randrange(d)` and `secrets.SystemRandom().randrange(d)` provide exactly, by drawing random bits and redrawing when the result is d or more. Algorithm 1 of Canonne, Kamath and Steinke (2020), after von Neumann (1951), builds the $e^{-\lambda}$ coin from these integers.

Its first step flips coins whose bias is a fraction, and the lemma says that such coins are exact.

Lemma 25.1 (An exact coin with rational bias)**.** *For integers $0 \le m \le d$, let W be uniform on $\{0, \dots, d-1\}$. Then $\Pr[W < m] = m/d$, so the indicator $\mathbf{1}[W < m]$, which is 1 when $W < m$ and 0 otherwise, is an exact* $\mathrm{Bern}(m/d)$ *draw.*

Proof.

1. There are m favourable values, 0 to $m-1$, among d equally likely ones, so $\Pr[W < m] = m/d$. No step rounds, because comparing two integers is exact.

□

No fraction equals e^{-1}, so the lemma alone cannot make the coin. The way around is the power series $e^{-\lambda} = 1 - \lambda + \frac{\lambda^2}{2!} - \frac{\lambda^3}{3!} + \cdots$. Flip coins with success probabilities $\lambda, \lambda/2, \lambda/3, \ldots$ until one fails, so that the chance of surviving k flips is $\lambda^k/k!$, the k-th term of the series. The parity of the stopping index then supplies the alternating signs.

Theorem 25.2 (Algorithm 1: an exact coin with an exponential bias)**.** *Let λ be rational with $0 \le \lambda \le 1$. Draw independent coins $I_k \sim \mathrm{Bern}(\lambda/k)$ for $k = 1, 2, \ldots$ and stop at the first $I_K = 0$, so that K is the index of the first failure. Output 1 if K is odd and 0 if K is even. Then the survival probabilities, the output probability and the mean number of flips are*

$$\Pr[K > k] = \frac{\lambda^k}{k!}, \qquad \Pr[K \text{ odd}] = e^{-\lambda}, \qquad \mathbb{E}[K] = e^{\lambda}. \tag{25.6}$$

For rational $\lambda > 1$, draw $\lfloor \lambda \rfloor$ coins with probability e^{-1} this way, stopping at the first 0, then one coin with probability $e^{-(\lambda - \lfloor \lambda \rfloor)}$. The output, which is 1 only if every coin is 1, is an exact $\mathrm{Bern}(e^{-\lambda})$ *draw.*

Proof. The plan is to compute the survival probabilities, turn them into point probabilities, add the odd ones, read the mean off the survival probabilities, and reduce a large exponent to several small ones.

1. *The survival probabilities.* The event $K > k$ means that the first k coins all came up 1. They are independent with probabilities $\lambda/1, \lambda/2, \ldots, \lambda/k$, so $\Pr[K > k]$ is their product, $\lambda^k/k!$. For $\lambda = m/d$, each $\lambda/j = m/(dj)$ is a ratio of integers, at most 1 since $\lambda \le 1$, so each coin is exact by Lemma 25.1.
2. *The point probabilities.* The event $K = k$ means that the coins survived $k-1$ flips but not k, so
$$\Pr[K = k] = \Pr[K > k-1] - \Pr[K > k] = \frac{\lambda^{k-1}}{(k-1)!} - \frac{\lambda^k}{k!}. \tag{25.7}$$
3. *Add over odd k.* Over $k = 1, 3, 5, \ldots$, every power λ^j appears exactly once, with a plus sign when $j = k-1$ is even and a minus sign when $j = k$ is odd. So the sum is the series
$$\sum_{k \text{ odd}} \Pr[K = k] = \sum_{j \ge 0} \frac{(-\lambda)^j}{j!} = e^{-\lambda}. \tag{25.8}$$
4. *The mean.* A variable K on $\{1, 2, \ldots\}$ is a count of survivals, $K = \sum_{k \ge 0} \mathbf{1}[K > k]$, since exactly K of the indicators $\mathbf{1}[K > 0], \mathbf{1}[K > 1], \ldots$ are 1. Taking expectations and using step 1,
$$\mathbb{E}[K] = \sum_{k \ge 0} \Pr[K > k] = \sum_{k \ge 0} \frac{\lambda^k}{k!} = e^{\lambda}. \tag{25.9}$$
Each flip uses one uniform integer, so a coin costs $e^{\lambda} \le e$ uniform draws on average.

5. *Large* λ. A product of probabilities is the probability that independent coins all come up 1, and $e^{-\lambda} = (e^{-1})^{\lfloor\lambda\rfloor}\, e^{-(\lambda-\lfloor\lambda\rfloor)}$, so the output is Bern$(e^{-\lambda})$. Each coin has an exponent of at most 1, so steps 1 to 3 cover it. Stopping at the first 0 does not change the output, which is 0 as soon as one coin is 0.

□

The coin is exact, because no step rounds and the only randomness is uniform integers. It costs at most e draws for $\lambda \le 1$ and below $\frac{e}{1-e^{-1}} + 1 \approx 5.3$ for any λ. The whole-unit coins stop at the first 0, so on average fewer than $1/(1-e^{-1})$ of them are flipped, at most e draws each, and the last coin, reached with probability at most e^{-1}, costs at most e. The cost jumps at each integer (Figure 25.2), from $e = 2.7183$ just below 1 to $e + e^{-1} = 3.0862$ at 1, because there the final coin for the fractional part 0 still costs a draw.

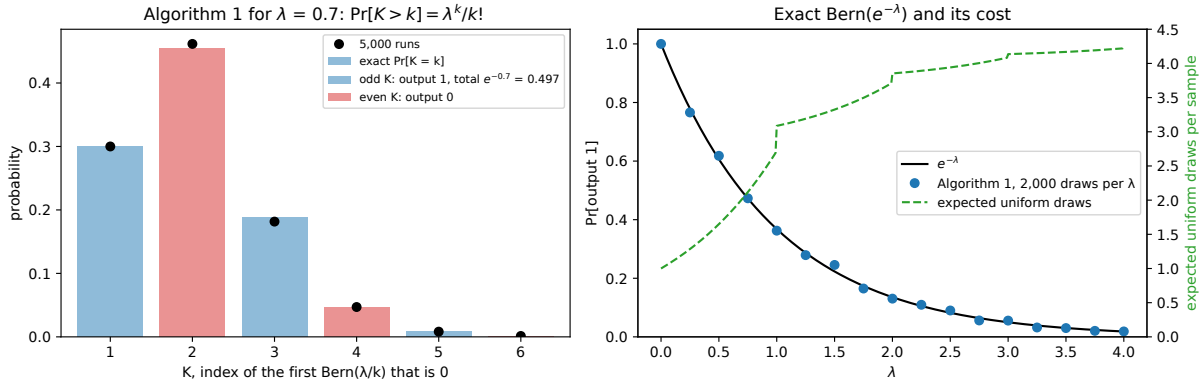


Figure 25.2.: Left: the distribution of K for $\lambda = 0.7$, exact and from 5,000 runs, odd and even K coloured. Right: the frequency of 1s against $e^{-\lambda}$, with the expected draws per coin.

Example 25.3. To see the parity trick outcome by outcome, take $\lambda = 0.7$ (Table 25.2). Each row is the chance of surviving $k-1$ flips minus the chance of surviving k, and the odd rows add up to $0.3000 + 0.1878 + 0.0086 + \cdots = 0.4966 = e^{-0.7}$. Over 10,000 runs the script sees $K = 1$ in a fraction 0.2987 of them, against 0.3000, and outputs 1 with frequency 0.4897.

Table 25.2.: Algorithm 1 for $\lambda = 0.7$, outcome by outcome. The output is the parity of the index of the first 0.

K	Coins seen	$\Pr[K = k]$	Output
1	$I_1 = 0$	$1 - 0.7 = 0.3000$	1
2	$I_1 = 1$, $I_2 = 0$	$0.7 - 0.7^2/2 = 0.4550$	0
3	$I_1 = I_2 = 1$, $I_3 = 0$	$0.245 - 0.0572 = 0.1878$	1
4	three 1s, then 0	0.0472	0
5	four 1s, then 0	0.0086	1

The running count's sampler in Section 25.4 needs only the coin with $\lambda = 1$. Over 10,000 draws it comes up 1 with frequency 0.3709, against $e^{-1} = 0.3679$, at 3.096 uniform draws each, against the expected 3.086.

25.4. The discrete Laplace mechanism

Most queries in DP are counts or histograms, like the running example. For them the natural noise is integer-valued too, so that the output has no low-order bits to leak and the noise can be sampled exactly. This section defines the discrete Laplace distribution, proves that adding it, the **geometric mechanism** of Ghosh, Roughgarden and Sundararajan (2009), has the continuous guarantee, and samples it exactly.

Keep the Laplace density $e^{-|z|/b}$ only at the integers, and divide by the sum of those values. Each step away from 0 then multiplies the probability by the same factor $r = e^{-1/b}$.

Definition 25.2 (The discrete Laplace distribution)**.** The **discrete Laplace** distribution with scale $b > 0$, written $\mathrm{Lap}_{\mathbb{Z}}(b)$, has the probability mass function (PMF)

$$\Pr[Z = z] = \frac{1-r}{1+r}\, r^{|z|}, \qquad z \in \mathbb{Z}, \qquad r = e^{-1/b}. \tag{25.10}$$

The constant in front and the variance both come from sums of geometric series.

Proposition 25.2 (Normalization and variance of the discrete Laplace)**.** *The probabilities in Equation 25.10 add up to 1, the mean of Z is 0, and*

$$\mathrm{Var}(Z) = \frac{2r}{(1-r)^2}. \tag{25.11}$$

Proof.

1. *Normalize.* The weights $r^{|z|}$ are 1 at $z = 0$ and appear twice, at $\pm z$, for each $z \geq 1$. Summing the geometric series gives
$$\sum_{z \in \mathbb{Z}} r^{|z|} = 1 + \frac{2r}{1-r} = \frac{1+r}{1-r}, \tag{25.12}$$
and dividing by it gives the constant in Equation 25.10.
2. *Mean.* The PMF gives z and $-z$ the same probability, so their contributions to the mean cancel, and the variance is $\mathbb{E}[Z^2]$.
3. *Variance.* Differentiating the geometric series $\sum_{z \geq 0} r^z = \frac{1}{1-r}$ twice in r, multiplying by r after each step, gives
$$\sum_{z \geq 1} z^2 r^z = \frac{r(1+r)}{(1-r)^3}. \tag{25.13}$$
Counting $\pm z$ twice,
$$\mathbb{E}[Z^2] = \frac{1-r}{1+r} \cdot 2 \cdot \frac{r(1+r)}{(1-r)^3} = \frac{2r}{(1-r)^2}. \tag{25.14}$$

□

Shifting the input by one unit changes each output's probability by at most the factor $e^{1/b}$, so the guarantee is the continuous one.

Theorem 25.3 (The discrete Laplace mechanism)**.** *Let f be integer-valued, with values in $\mathbb{Z}^d$ and ℓ_1 sensitivity $\Delta_1 f$, and let $b > 0$. The mechanism $M(D) = f(D) + Z$, with independent $\mathrm{Lap}_{\mathbb{Z}}(b)$ noise on each coordinate, is ε-DP with*

$$\varepsilon = \frac{\Delta_1 f}{b}, \tag{25.15}$$

and for a scalar f whose answers differ by $\Delta_1 f$, the privacy loss reaches $\pm\varepsilon$ at every output above both answers.

Proof.

1. *The privacy loss.* Take $d = 1$ first, and neighbours with true answers $f(D) = a$ and $f(D') = a'$. An output y requires noise $y - a$ under D and $y - a'$ under D'. The constant $\frac{1-r}{1+r}$ is the same for both, so it cancels in the ratio, and with $\ln r = -1/b$

$$L_{D,D'}(y) = \ln \frac{r^{|y-a|}}{r^{|y-a'|}} = \frac{|y - a'| - |y - a|}{b}. \tag{25.16}$$

2. *Bound it.* By the triangle inequality, two numbers' distances to y differ by at most their distance to each other, so in both directions

$$\big||y - a'| - |y - a|\big| \le |a - a'| \le \Delta_1 f. \tag{25.17}$$

   So $|L_{D,D'}(y)| \le \Delta_1 f/b$ at every integer y, which is pure DP with Equation 25.15 by the pointwise criterion of Section 25.2. For $y \ge \max(a, a')$ the difference of distances is exactly $|a - a'|$, so the bound is reached.
3. *Vectors.* With independent noise per coordinate, the privacy loss is a sum of per-coordinate losses. Each is at most $|a_i - a'_i|/b$ by step 2, and these add up to at most $\Delta_1 f/b$.

□

Step 1 is where integers matter, because a non-integer shift would give different inputs different supports, the floating-point failure again.

For rational $b = t/s$, Algorithm 2 of Canonne, Kamath and Steinke (2020) samples $\mathrm{Lap}_{\mathbb{Z}}(b)$ exactly. It assembles a one-sided geometric variable from a remainder W below t and a quotient V, divides it by s, and attaches a random sign. The proof needs two tools. The first, used by the remainder step and the sign step, reshapes one distribution into another by throwing some draws away.

Lemma 25.2 (Rejection sampling)**.** *Let q be a PMF you can sample from, the **proposal**, and let $h(x) \in [0, 1]$ be an acceptance probability for each value x. Repeat: draw $Y \sim q$ and accept it with probability $h(Y)$, until a draw is accepted. Then the output has PMF $q(x)h(x)/A$, where $A = \sum_x q(x)h(x)$ is the probability that one round accepts, and the number of rounds is geometric with mean $1/A$.*

Proof.

1. *One round.* A round proposes x with probability $q(x)$ and accepts it with probability $h(x)$, so it outputs x with probability $q(x)h(x)$, and something with probability A.

2. *Add up the rounds.* The rounds are independent, so the output is x when the first j rounds output nothing and round $j+1$ outputs x, for some $j \geq 0$. That is a geometric series with ratio $1 - A < 1$:
$$\sum_{j \geq 0} (1-A)^j q(x) h(x) = \frac{q(x)h(x)}{A}. \tag{25.18}$$
3. *The cost.* Each round succeeds with probability A independently, so the number of rounds is geometric with mean $1/A$.

□

So the output is exact as long as each acceptance is an exact coin, and A never has to be computed. The second tool, for the quotient step, says that counting coin flips gives a geometric variable.

Lemma 25.3 (Counting successes gives a geometric variable)**.** *Flip independent coins with success probability κ and let V be the number of successes before the first failure. Then $\Pr[V = v] = \kappa^v(1-\kappa)$ for $v = 0, 1, 2, \ldots$, and $\Pr[V \geq v] = \kappa^v$.*

Proof.

1. The event $V = v$ means exactly v successes followed by one failure, and independent flips multiply, which gives $\kappa^v(1-\kappa)$. The event $V \geq v$ says that the first v flips were successes, which has probability κ^v. This is the geometric distribution of Probability distributions, counted from 0.

□

Theorem 25.4 (Algorithm 2: exact sampling of the discrete Laplace)**.** *Input integers $s, t \geq 1$, so that $b = t/s$. Repeat until a value is returned:*

1. *Draw W uniform on $\{0, \ldots, t-1\}$ and accept it with probability $e^{-W/t}$ (Algorithm 1); on rejection, start over.*
2. *Count the successes V of independent coins with probability e^{-1} before the first failure (Algorithm 1 with $\lambda = 1$).*
3. *Set $G = W + tV$ and $Y = \lfloor G/s \rfloor$.*
4. *Draw a fair bit J. If $J = 1$ and $Y = 0$, start over. Otherwise return $(1-2J)Y$.*

The output is exactly $\mathrm{Lap}_{\mathbb{Z}}(b)$, and the mean number of passes through the loop is $1/(A_W\,(1 - \frac{1}{2}(1 - e^{-s/t})))$, where $A_W = \frac{1}{t}\sum_{w<t} e^{-w/t}$ is the acceptance probability of step 1.

Proof. The plan follows the algorithm's own order: the accepted remainder, the quotient, their combination, the division and the sign, then the cost.

1. *W given acceptance.* By Lemma 25.2 with a uniform proposal and acceptance $e^{-w/t}$, the accepted W has $\Pr[W = w] \propto e^{-w/t}$ on $\{0, \ldots, t-1\}$.
2. *V is geometric.* By Lemma 25.3 with $\kappa = e^{-1}$, $\Pr[V = v] = (1 - e^{-1})e^{-v}$.
3. *$G = W + tV$ is geometric with ratio $e^{-1/t}$.* Each $g \geq 0$ is $w + tv$ for exactly one pair, $w = g \bmod t$ and $v = \lfloor g/t \rfloor$, and the independent weights multiply to $e^{-w/t}e^{-v} = e^{-g/t}$. So $\Pr[G = g] \propto e^{-g/t}$.

4. $Y = \lfloor G/s \rfloor$ *is geometric with ratio* $e^{-s/t} = e^{-1/b}$. The event $Y = y$ collects $g = sy, \dots, sy + s - 1$, so its weight is a factor that does not depend on y times $e^{-sy/t} = r^y$:

$$\sum_{j=0}^{s-1} e^{-(sy+j)/t} = e^{-sy/t} \sum_{j=0}^{s-1} e^{-j/t}. \tag{25.19}$$

5. *The sign.* A fair sign gives $\pm y$ probability $\frac{1}{2}\Pr[Y = y]$ each, but 0 would get both halves, since $+0$ and -0 are the same output. Rejecting the case $J = 1$, $Y = 0$ leaves $\frac{1}{2}\Pr[Y = y]$ for every $z = \pm y$, including 0, and after renormalizing by Lemma 25.2, $\Pr[Z = z] \propto r^{|z|}$, which is Equation 25.10.
6. *The cost.* A pass survives step 1 with probability A_W, and step 4 unless $J = 1$ and $Y = 0$, which has probability $\frac{1}{2}(1 - e^{-s/t})$ by Lemma 25.3. By step 3 of Lemma 25.2, the mean number of passes is one over the product of the two survival probabilities.

□

To follow steps 3 and 4 by hand, take $t = 3$ and $s = 2$, so $b = 1.5$ (Table 25.3). Then $\Pr[Y = 0] = 0.4866$ and $\Pr[Y = 1] = 0.2498$, and their ratio 0.5134 is $e^{-2/3}$, as step 4 says.

Table 25.3.: Algorithm 2 with $t = 3$, $s = 2$. Each g is one pair (w, v); pairs of consecutive g merge into one y.

g	$w = g \bmod 3$	$v = \lfloor g/3 \rfloor$	$\Pr[G = g] = (1 - e^{-1/3})e^{-g/3}$	$y = \lfloor g/2 \rfloor$
0	0	0	0.2835	0
1	1	0	0.2031	0
2	2	0	0.1455	1
3	0	1	0.1043	1

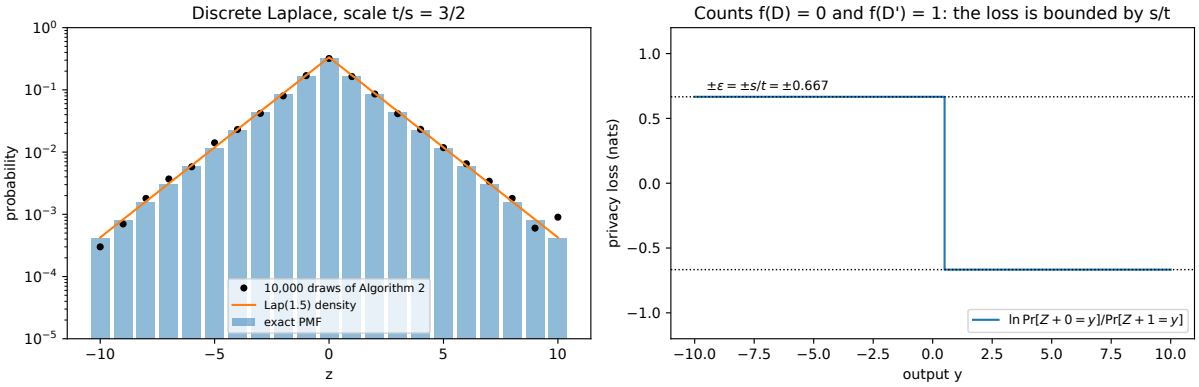


Figure 25.3.: Left: the PMF of $\mathrm{Lap}_{\mathbb{Z}}(1.5)$ on a log scale, exact and from 10,000 draws of Algorithm 2. Right: the privacy loss between counts 0 and 1, bounded by $\varepsilon = 2/3$.

The discrete Laplace gives the continuous guarantee with slightly less noise. At $b = 1$ its variance is 1.8413 against 2, and the gap tends to $\frac{1}{6}$ as b grows. Rounding $f(D) + \mathrm{Lap}(b)$ to an integer would also be ε-DP, but it is a different distribution and needs a floating-point sampler.

For a single count, the discrete Laplace is the best possible mechanism. Ghosh, Roughgarden and Sundararajan (2009) show that every user, whatever their loss and prior, can post-process its output to do as well as with any other ε-DP mechanism.

Example 25.4. Release the running count at $\varepsilon = 1$, so $b = 1$ and $r = e^{-1} = 0.3679$. The exact count is released when the noise is 0, which happens with probability $\frac{1-r}{1+r} = \frac{0.6321}{1.3679} = 0.4621$. The noise has variance $\frac{2r}{(1-r)^2} = 1.8413$, a standard deviation of 1.357. Algorithm 2 with $s = t = 1$ always accepts in step 1, so a pass is a run of e^{-1} coins and a fair bit, and a sample takes 1.462 passes on average. The script's 10,000 draws have variance 1.8305 and cost 11.49 uniform draws each, and over the scales in Table 25.6 the cost stays between 10 and 12.

25.5. The discrete Gaussian mechanism

The Gaussian mechanism is the workhorse of private machine learning, because its noise grows with the ℓ_2 sensitivity, far smaller than the ℓ_1 sensitivity in high dimension. Its integer analogue (Canonne, Kamath & Steinke 2020) served the 2020 US Census (Abowd et al., 2022) and suits noise added before secure aggregation, which works with integers (Kairouz, Liu & Steinke, 2021). This section compares it with the continuous Gaussian of the Laplace and Gaussian mechanisms in exact $\delta(\varepsilon)$, Rényi divergence and variance.

As with the discrete Laplace, keep the bell curve at the integers and normalize.

Definition 25.3 (The discrete Gaussian distribution)**.** The **discrete Gaussian** with parameter $\sigma > 0$ and integer location μ, written $\mathcal{N}_{\mathbb{Z}}(\mu, \sigma^2)$, gives each integer x the probability

$$\Pr[X = x] = \frac{e^{-(x-\mu)^2/(2\sigma^2)}}{\mathcal{Z}_\sigma}, \qquad \mathcal{Z}_\sigma = \sum_{y \in \mathbb{Z}} e^{-y^2/(2\sigma^2)}. \tag{25.20}$$

The parameter σ^2 is the variance of the bell curve, not quite that of X (Proposition 25.3). An integer shift only relabels the terms of $\mathcal{Z}_\sigma$, so the normalizer cancels in the privacy loss of $M(D) = f(D) + X$ for integer-valued f. Then the smallest δ at each ε has the continuous Gaussian's formula, with discrete tails.

Theorem 25.5 (Privacy loss and exact δ of the discrete Gaussian mechanism)**.** *Let f be integer-valued with sensitivity Δ, an integer, and let $M(D) = f(D) + X$ with $X \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$. For a neighbouring pair whose answers differ by exactly Δ, the privacy loss at an integer output y is*

$$L_{D,D'}(y) = \frac{(y - f(D'))^2 - (y - f(D))^2}{2\sigma^2}, \tag{25.21}$$

and the smallest δ for which the pair satisfies (ε, δ)-DP is

$$\delta(\varepsilon) = \Pr\Big[X > \frac{\varepsilon\sigma^2}{\Delta} - \frac{\Delta}{2}\Big] - e^{\varepsilon} \Pr\Big[X > \frac{\varepsilon\sigma^2}{\Delta} + \frac{\Delta}{2}\Big]. \tag{25.22}$$

Proof. The plan is to cancel the normalizer, write the smallest δ as a sum over the outputs where the loss exceeds ε, and evaluate that sum under each input. Take $f(D) = 0$ and $f(D') = \Delta$, since an integer shift of both changes nothing.

1. *The normalizer cancels.* Under D the output y has probability $e^{-y^2/(2\sigma^2)}/\mathcal{Z}_\sigma$, and under D' it has $e^{-(y-\Delta)^2/(2\sigma^2)}/\mathcal{Z}_\sigma$. So the ratio is the ratio of the two exponentials, and
$$L_{D,D'}(y) = \frac{(y-\Delta)^2 - y^2}{2\sigma^2} = \frac{\Delta^2 - 2\Delta y}{2\sigma^2}, \tag{25.23}$$
the formula of the continuous Gaussian, now only at integers y.
2. *The smallest δ as a sum.* The definition asks $\Pr[M(D) \in S] - e^\varepsilon \Pr[M(D') \in S] \le \delta$ for every set S. The left side is a sum over $y \in S$ of the terms $\Pr[M(D) = y] - e^\varepsilon \Pr[M(D') = y]$, so it is largest when S holds exactly the outputs with a positive term. The smallest valid δ is therefore the sum of the positive terms, the hockey-stick divergence of Divergences. A term is positive exactly when $L_{D,D'}(y) > \varepsilon$, which by step 1 means $y < \frac{\Delta}{2} - \frac{\varepsilon\sigma^2}{\Delta}$.
3. *The sum under D.* Under D the output is X, so those outputs have the probability of a lower tail of X. The PMF gives x and $-x$ the same probability, so the lower tail equals the mirrored upper tail:
$$\Pr\left[X < \frac{\Delta}{2} - \frac{\varepsilon\sigma^2}{\Delta}\right] = \Pr\left[X > \frac{\varepsilon\sigma^2}{\Delta} - \frac{\Delta}{2}\right]. \tag{25.24}$$
4. *The sum under D'.* Under D' the output is $X + \Delta$, so the same outputs need X below $-\frac{\Delta}{2} - \frac{\varepsilon\sigma^2}{\Delta}$, and by the same symmetry their probability is the second tail in Equation 25.22. Subtracting e^ε times it from step 3 gives Equation 25.22.

□

Canonne, Kamath and Steinke (2020, Theorem 7) show that a pair with a smaller shift has a smaller δ, so Equation 25.22 is the privacy profile of the mechanism, its smallest δ at each ε over all neighbouring pairs. Because the proof needs integer shifts, a real-valued query is first scaled by $1/h$ for a small step h and randomly rounded to an integer, and the accounting uses the sensitivity of the rounded query.

The curve $\delta(\varepsilon)$ is close to the continuous one but not equal to it. Moving y by one moves the privacy loss by Δ/σ^2, so the loss takes values on a lattice. As ε crosses a lattice value, a whole output enters or leaves the sum in step 2 of the proof, which puts a kink in $\delta(\varepsilon)$ (Figure 25.4). At $\varepsilon = 1$ the discrete curve is above the continuous one, 0.6032 against 0.5099 at $\sigma = 0.5$, but just past a kink it can be below, and the gap closes as σ grows.

The two remaining comparisons rest on one question: how does the sum of a Gaussian bump over the integers depend on where the bump is centred? The Rényi proof needs it largest at an integer centre, and the variance proof needs it in closed form. The tool is the Poisson summation formula (Stein & Shakarchi 2003, ch. 5), which for a smooth, fast-decaying g with Fourier transform $\hat{g}(k) = \int g(x)e^{-2\pi ikx}\,\mathrm{d}x$ says
$$\sum_{x\in\mathbb{Z}} g(x) = \sum_{k\in\mathbb{Z}} \hat{g}(k). \tag{25.25}$$

Lemma 25.4 (A Gaussian bump summed over the integers)**.** *For every real centre c and every $\sigma > 0$,*
$$\sum_{x\in\mathbb{Z}} e^{-(x-c)^2/(2\sigma^2)} = \sqrt{2\pi}\,\sigma \sum_{k\in\mathbb{Z}} e^{-2\pi^2\sigma^2k^2}\cos(2\pi kc). \tag{25.26}$$
The sum is largest when c is an integer, where it equals $\mathcal{Z}_\sigma$.

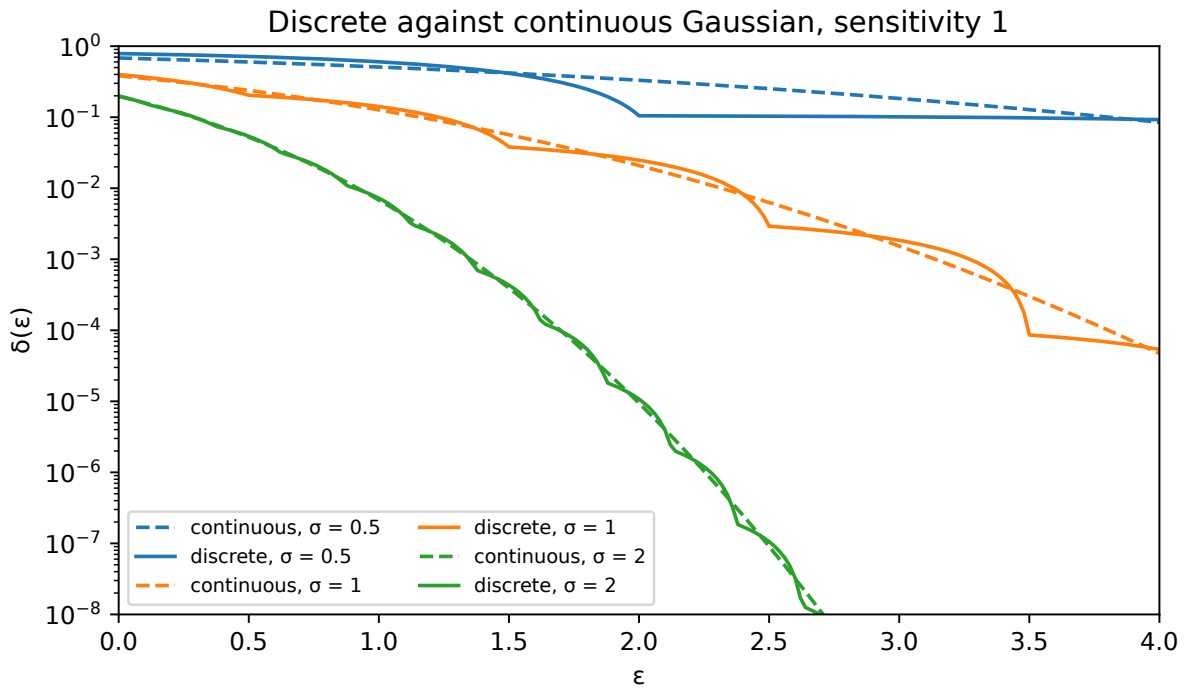


Figure 25.4.: Discrete against continuous Gaussian for sensitivity 1: $\delta(\varepsilon)$ from Equation 25.22 (solid) and for the continuous Gaussian (dashed), for $\sigma = 0.5$, 1 and 2.

Proof.

1. *Transform the bump.* The Fourier transform of $e^{-x^2/(2\sigma^2)}$ is again a Gaussian, $\sqrt{2\pi}\,\sigma e^{-2\pi^2\sigma^2k^2}$, and shifting a function by c multiplies its transform by the phase $e^{-2\pi ikc}$. So for $g(x) = e^{-(x-c)^2/(2\sigma^2)}$,
$$\hat{g}(k) = \sqrt{2\pi}\,\sigma e^{-2\pi^2\sigma^2k^2} e^{-2\pi ikc}. \tag{25.27}$$
2. *Pair k with $-k$.* Poisson summation adds $\hat{g}(k)$ over all integers k. The terms for k and $-k$ have the same real coefficient and phases that add to $2\cos(2\pi kc)$, which gives Equation 25.26.
3. *Where it is largest.* Every coefficient $e^{-2\pi^2\sigma^2k^2}$ is positive and $\cos \le 1$, with $\cos(2\pi kc) = 1$ for all k exactly when c is an integer. So the right side is largest at integer c, and there the left side is $\mathcal{Z}_\sigma$ by relabelling.

□

So moving the centre off the integers can only lower the sum. At $\sigma = 0.5$ the script finds 1.271342 at $c = 0$ and 1.235287 at $c = 0.5$, while at $\sigma = 1$ both are 2.506628, $\sqrt{2\pi}$ to six decimals.

The second comparison is in Rényi divergence. For two distributions P and Q on the integers, the **Rényi divergence** of order $\alpha > 1$ is

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \sum_x P(x)^\alpha Q(x)^{1-\alpha}, \tag{25.28}$$

an average of the privacy loss that weights large losses more as α grows (Divergences). For continuous Gaussians with variance σ^2 and means μ and ν it is $\alpha(\mu - \nu)^2/(2\sigma^2)$ (Equation 3.19).

For discrete ones, completing a square turns the sum into a bump summed over the integers, which the lemma bounds.

Theorem 25.6 (Rényi divergence of two discrete Gaussians)**.** *For integers μ and ν and every $\alpha > 1$,*

$$D_\alpha(\mathcal{N}_{\mathbb{Z}}(\mu,\sigma^2)\,\|\,\mathcal{N}_{\mathbb{Z}}(\nu,\sigma^2)) \le \frac{\alpha(\mu-\nu)^2}{2\sigma^2}, \tag{25.29}$$

with equality when α is an integer.

Proof. The plan is to write out the sum, complete the square in its exponent, and apply Lemma 25.4 to what is left. Write $p_\mu(x) = e^{-(x-\mu)^2/(2\sigma^2)}/\mathcal{Z}_\sigma$, with the same normalizer for every integer μ.

1. *Write out the summand.* In $\sum_x p_\mu(x)^\alpha p_\nu(x)^{1-\alpha}$ the powers of the normalizer combine to $1/\mathcal{Z}_\sigma$, and the exponents add up to
$$-\frac{\alpha(x-\mu)^2 + (1-\alpha)(x-\nu)^2}{2\sigma^2}. \tag{25.30}$$
2. *Complete the square.* With $c = \alpha\mu + (1-\alpha)\nu$ the x^2 and x terms match, and the constant is what is left over:
$$\alpha(x-\mu)^2 + (1-\alpha)(x-\nu)^2 = (x-c)^2 - \alpha(\alpha-1)(\mu-\nu)^2. \tag{25.31}$$
3. *Factor the sum.* Substituting step 2 into step 1 gives
$$\sum_x p_\mu(x)^\alpha p_\nu(x)^{1-\alpha} = e^{\alpha(\alpha-1)(\mu-\nu)^2/(2\sigma^2)} \cdot \frac{1}{\mathcal{Z}_\sigma} \sum_x e^{-(x-c)^2/(2\sigma^2)}. \tag{25.32}$$
4. *Apply the lemma.* The last factor is the bump sum centred at c divided by the bump sum centred at an integer, which by Lemma 25.4 is at most 1, with equality when c is an integer. For integer α the centre c is an integer, because μ and ν are. Taking $\frac{1}{\alpha-1}\ln$ of step 3 gives Equation 25.29.

□

So the discrete Gaussian is never worse than the continuous one in Rényi divergence. It satisfies the same zero-concentrated DP (Canonne, Kamath & Steinke 2020, Theorem 4), the guarantee of Part 4 that bounds every D_α by a constant times α.

The third comparison is the variance. Restricting the bell curve to integers pulls a little mass towards 0, and the proof reads the variance off the derivative of the normalizer, which the lemma gives in closed form.

Proposition 25.3 (Variance of the discrete Gaussian)**.** *For $X \sim \mathcal{N}_{\mathbb{Z}}(0,\sigma^2)$,*

$$\operatorname{Var}(X) = \sigma^2 - \frac{\sum_{k\neq 0} 4\pi^2\sigma^4 k^2 e^{-2\pi^2\sigma^2k^2}}{1 + \sum_{k\neq 0} e^{-2\pi^2\sigma^2k^2}} < \sigma^2. \tag{25.33}$$

Proof. The plan is to express the variance through the derivative of the normalizer, evaluate the normalizer by Lemma 25.4, and differentiate. Let $\tau = 1/(2\sigma^2)$ and $\mathcal{Z}(\tau) = \sum_x e^{-\tau x^2}$.

1. *Variance from the normalizer.* The mean is 0 because x and $-x$ have the same probability, so $\mathrm{Var}(X) = \mathbb{E}[X^2] = \sum_x x^2 e^{-\tau x^2}/\mathcal{Z}(\tau)$. Differentiating $e^{-\tau x^2}$ in τ brings down a factor $-x^2$, so
$$\mathbb{E}[X^2] = -\frac{\mathrm{d}}{\mathrm{d}\tau} \ln \mathcal{Z}(\tau). \tag{25.34}$$
2. *Poisson summation at $c = 0$.* With $\sigma = 1/\sqrt{2\tau}$, Equation 25.26 gives
$$\mathcal{Z}(\tau) = \sqrt{\pi/\tau}\,(1 + \theta(\tau)), \qquad \theta(\tau) = \sum_{k \neq 0} e^{-\pi^2 k^2/\tau}. \tag{25.35}$$
3. *Differentiate.* The log splits into $\frac{1}{2}\ln\pi - \frac{1}{2}\ln\tau + \ln(1+\theta(\tau))$, so
$$-\frac{\mathrm{d}}{\mathrm{d}\tau}\ln\mathcal{Z} = \frac{1}{2\tau} - \frac{\theta'(\tau)}{1+\theta(\tau)}, \qquad \theta'(\tau) = \sum_{k\neq 0} \frac{\pi^2 k^2}{\tau^2} e^{-\pi^2 k^2/\tau} > 0. \tag{25.36}$$
4. *Substitute $\tau = 1/(2\sigma^2)$.* Then $1/(2\tau) = \sigma^2$ and $\pi^2 k^2/\tau = 2\pi^2\sigma^2 k^2$, which turns step 3 into Equation 25.33. The correction is positive, so $\mathrm{Var}(X) < \sigma^2$.

□

The deficit is invisible in practice. Its leading term comes from $k = \pm 1$, so the relative deficit $1 - \mathrm{Var}(X)/\sigma^2$ is about $8\pi^2\sigma^2 e^{-2\pi^2\sigma^2}$, which is 2.11×10^{-7} at $\sigma = 1$. Only below $\sigma \approx 0.6$ does it show, as $\mathrm{Var}(X) = 0.8601\,\sigma^2$ at $\sigma = 0.5$ (Figure 25.5).

One property does not carry over: the sum of two independent discrete Gaussians is not exactly a discrete Gaussian. For $\sigma \geq 1$ it is very close, and distributed noise addition, where each client adds a share, relies on that.

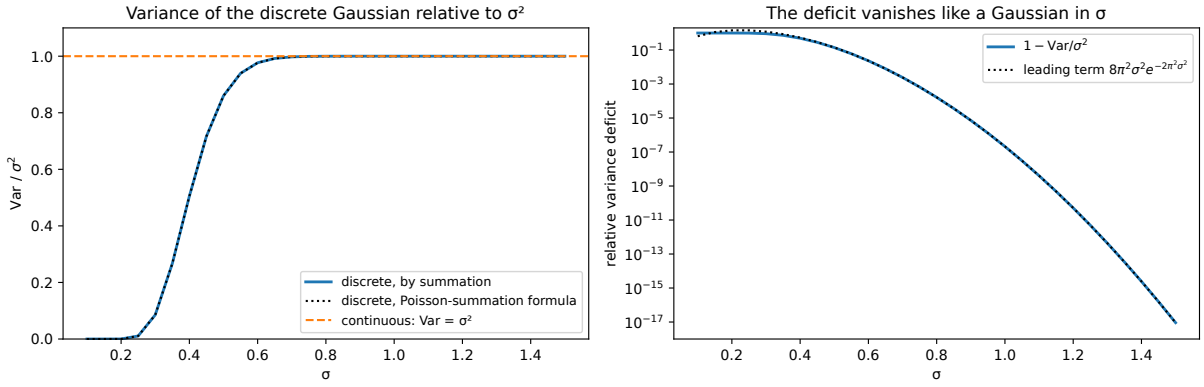


Figure 25.5.: Left: $\mathrm{Var}(X)/\sigma^2$ for $X \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$, by direct summation and from Equation 25.33. Right: the relative deficit $1 - \mathrm{Var}(X)/\sigma^2$ on a log scale, with its leading term.

Example 25.5. Release the running count with $\mathcal{N}_{\mathbb{Z}}(0, 1)$ noise, so $\sigma = 1$ and $\Delta = 1$. The privacy loss at an integer output y is $(1 - 2y)/2$, so the kinks are at $\varepsilon = 0.5, 1.5, \ldots$ (Figure 25.4). At $\varepsilon = 1$ the outputs with loss above ε are $y \leq -1$, and Equation 25.22 gives $\delta(1) = 0.1414$, against 0.1269 for continuous $\mathcal{N}(0, 1)$ noise. In Table 25.4, integer orders match $\alpha/(2\sigma^2)$ exactly, because their centre c is an integer, while the order 2.5 has $c = (1 - \alpha) \cdot 1 = -1.5$ and at $\sigma = 0.5$ falls below the continuous value. At $\sigma \geq 1$ the lemma's slack is below the four decimals shown.

Table 25.4.: Rényi divergences between discrete Gaussians shifted by 1, by direct summation, with the continuous value $\alpha/(2\sigma^2)$ in parentheses.

σ	D_2 (continuous)	$D_{2.5}$ (continuous)	D_5 (continuous)
0.5	4.0000 (4.0000)	4.9808 (5.0000)	10.0000 (10.0000)
1	1.0000 (1.0000)	1.2500 (1.2500)	2.5000 (2.5000)
2	0.2500 (0.2500)	0.3125 (0.3125)	0.6250 (0.6250)

25.6. Sampling the discrete Gaussian exactly

The privacy analysis is only as good as the sampler, and inverse-CDF sampling of $\mathcal{N}_{\mathbb{Z}}(0,\sigma^2)$ would need the irrational normalizer and a floating-point uniform. Algorithm 3 of Canonne, Kamath and Steinke (2020) instead proposes from the discrete Laplace, whose tails are heavier, and thins the proposals with exact $e^{-\lambda}$ coins. By Lemma 25.2, proposal times acceptance only has to be proportional to $e^{-y^2/(2\sigma^2)}$, so the acceptance probability is a Gaussian factor shifted to cancel the Laplace factor.

Theorem 25.7 (Algorithm 3: exact sampling of the discrete Gaussian)**.** *Input a rational $\sigma^2 > 0$ and let $t = \lfloor \sigma \rfloor + 1$. Repeat: draw $Z \sim \mathrm{Lap}_{\mathbb{Z}}(t)$ with Algorithm 2 ($s = 1$), and accept it with probability*

$$\exp\Big(-\frac{(|Z| - \sigma^2/t)^2}{2\sigma^2} \Big) \tag{25.37}$$

using Algorithm 1. Output the first accepted Z. The output is exactly $\mathcal{N}_{\mathbb{Z}}(0,\sigma^2)$, and the acceptance probability per proposal is

$$A = \frac{1-r}{1+r}\, e^{-\sigma^2/(2t^2)}\, \mathcal{Z}_\sigma, \qquad r = e^{-1/t}, \tag{25.38}$$

which tends to $\sqrt{\pi/2}\, e^{-1/2} = 0.7602$ as σ grows.

Proof. The plan is to check the shape of proposal times acceptance, that the acceptance is a probability and that every quantity is rational, and then to average the acceptance over proposals.

1. *Proposal times acceptance.* The proposal has $\Pr[Z = y] \propto e^{-|y|/t}$, and multiplying by the acceptance probability adds the exponents. Expanding the square gives

$$-\frac{|y|}{t} - \frac{(|y| - \sigma^2/t)^2}{2\sigma^2} = -\frac{|y|}{t} - \frac{y^2}{2\sigma^2} + \frac{|y|}{t} - \frac{\sigma^2}{2t^2} = -\frac{y^2}{2\sigma^2} - \frac{\sigma^2}{2t^2}. \tag{25.39}$$

   The $|y|/t$ terms cancel, leaving $e^{-y^2/(2\sigma^2)}$ times a constant, so by Lemma 25.2 the output is $\mathcal{N}_{\mathbb{Z}}(0,\sigma^2)$.
2. *The acceptance is a probability.* Its exponent is minus a square, so it is at most 1.
3. *Everything is rational.* The integer t is computed as $\lfloor \sqrt{\lfloor \sigma^2 \rfloor} \rfloor + 1$ with an integer square root. An integer n has $n^2 \le \sigma^2$ exactly when $n^2 \le \lfloor \sigma^2 \rfloor$, so this equals $\lfloor \sigma \rfloor + 1$ without computing σ. So $\lambda = (|y| - \sigma^2/t)^2/(2\sigma^2)$ is a ratio of integers, and Algorithm 1 applies.

4. *The acceptance rate.* By Lemma 25.2, A is the average acceptance over proposals. With $\Pr[Z = y] = \frac{1-r}{1+r} r^{|y|}$, step 1 turns each term of that average into $\frac{1-r}{1+r} e^{-\sigma^2/(2t^2)} e^{-y^2/(2\sigma^2)}$, and the sum over y is Equation 25.38.

□

To follow the algorithm by hand, take $\sigma^2 = 4$, so $t = 3$ and $\frac{1-r}{1+r} = 0.1651$ (Table 25.5). Dividing each product by the first gives $e^{-y^2/8}$, for instance $0.1167/0.1322 = 0.883 = e^{-1/8}$.

Table 25.5.: Algorithm 3 for $\sigma^2 = 4$, by hand. Each acceptance is one coin of Algorithm 1, with λ kept as a fraction.

y	Proposal $0.1651\, r^{\lvert y\rvert}$	$\lambda = (\lvert y\rvert - 4/3)^2/8$	Acceptance $e^{-\lambda}$	Product
0	0.1651	2/9	0.8007	0.1322
1	0.1183	1/72	0.9862	0.1167
2	0.0848	1/18	0.9460	0.0802
4	0.0435	8/9	0.4111	0.0179
6	0.0223	49/18	0.0657	0.0015

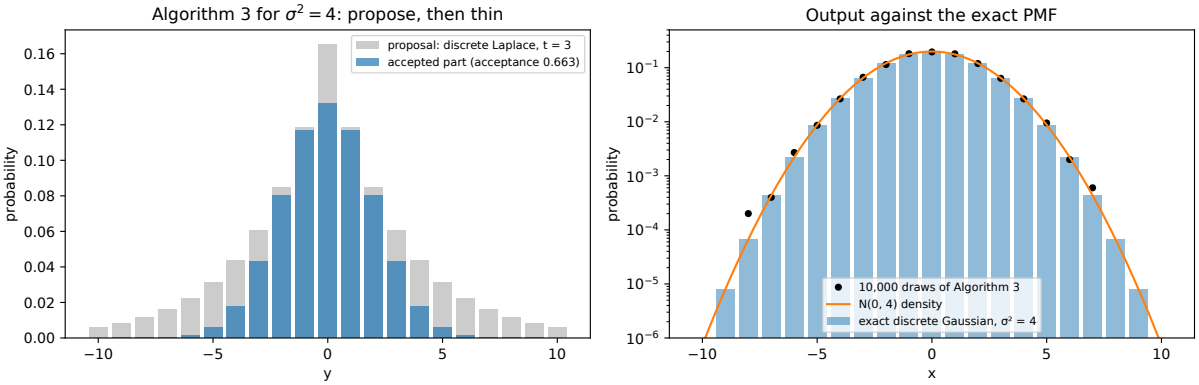


Figure 25.6.: Algorithm 3 for $\sigma^2 = 4$. Left: the discrete Laplace proposal with $t = 3$ and its accepted part, A times the discrete Gaussian PMF. Right: 10,000 draws against the exact PMF on a log scale.

The sampler is exact and cheap. The acceptance rate runs from 0.5185 at $\sigma^2 = \frac{1}{4}$ to 0.7532 at $\sigma^2 = 100$, so a draw takes fewer than two proposals and 15 to 27 uniform integers (Table 25.6). The choice of t affects only the cost, because step 1 of the proof works for every integer $t \geq 1$. For large σ, $\frac{1-r}{1+r} \approx \frac{1}{2t}$ and $\mathcal{Z}_\sigma \approx \sqrt{2\pi}\sigma$, so A is about $\frac{\sqrt{2\pi}\sigma}{2t} e^{-\sigma^2/(2t^2)}$. The derivative of its logarithm in t is $-\frac{1}{t} + \frac{\sigma^2}{t^3}$, which vanishes at $t = \sigma$, and $\lfloor \sigma \rfloor + 1$ is an integer next to it.

Exactness is not timing safety. The number of rounds depends on the output, so the running time leaks information about the noise, and Jin et al. (2022) show that such leaks are exploitable. A deployment pads the running time or runs a fixed number of rounds.

Example 25.6. Release the running count with $\mathcal{N}_{\mathbb{Z}}(0,1)$ noise, as in Example 25.5. The proposal scale is $t = 2$ and the acceptance rate 0.5418, so a draw takes 1.85 proposals on average, and the script's 10,000 draws have variance 0.9977 and cost 24.8 uniform integers each. For a histogram of 1,000 such counts, where one record changes one bin by 1, add $\mathcal{N}_{\mathbb{Z}}(0,100)$ to each bin. The 999 unchanged bins add nothing to the privacy loss, so Equation 25.22 with $\Delta = 1$ and $\sigma = 10$ gives the release's $\delta(\varepsilon)$. At 15.6 uniform integers per bin, the histogram needs about 15,600 of them and no floating-point operation.

25.7. Code

The script code/discrete_mechanisms.py implements Algorithms 1 to 3 with `fractions.Fraction` and integers, drawing uniform integers from a seeded `random.Random`. It also enumerates the toy floating-point sampler exactly and computes the discrete Gaussian's quantities by summation. Each sampler is compared with its exact distribution by the total variation (TV) distance, half the sum of the absolute differences between exact probabilities and observed frequencies. The "noise floor" is that distance for 10,000 draws from the exact PMF drawn with NumPy. From the repository root:

```
uv run python code/part03-dp-foundations/discrete_mechanisms.py
```

Table 25.6.: The script's output with the default seed.

Experiment	Result
Float Laplace, $p = 12$	$\Pr[M(D) \in S] = 0.4791$; snapped to integers: 0, with $\max\|L\| = 1.0010$
Algorithm 1, $\lambda = \frac{1}{2}, 1, \frac{5}{2}, 4$	Frequencies 0.6024, 0.3709, 0.0869, 0.0168 against 0.6065, 0.3679, 0.0821, 0.0183; draws per coin 1.662, 3.096, 3.976, 4.218 against 1.649, 3.086, 3.941, 4.240
Algorithm 2, $b = 1$	Variance 1.8305 (exact 1.8413); TV to exact 0.0114 (noise floor 0.0074); 11.49 draws per sample
Algorithm 2, $b = 3/2$	Variance 4.3834 (exact 4.3370); TV 0.0119 (noise floor 0.0243); 12.08 draws per sample
Algorithm 2, $b = 10$	Variance 195.2985 (exact 199.8334); TV 0.0339 (noise floor 0.0331); 10.25 draws per sample
Algorithm 3, $\sigma^2 = \frac{1}{4}$	Variance 0.2189 (exact 0.2150); TV 0.0028 (noise floor 0.0045); acceptance 0.5185; 26.6 draws
Algorithm 3, $\sigma^2 = 1$	Variance 0.9977 (exact 1.0000); TV 0.0076 (noise floor 0.0042); acceptance 0.5418; 24.8 draws

Experiment	Result
Algorithm 3, $\sigma^2 = 4$	Variance 4.0826 (exact 4.0000); TV 0.0154 (noise floor 0.0142); acceptance 0.6629; 18.9 draws
Algorithm 3, $\sigma^2 = 100$	Variance 99.4386 (exact 100.0000); TV 0.0243 (noise floor 0.0325); acceptance 0.7532; 15.6 draws
Discrete Gaussian variance	$\mathrm{Var}/\sigma^2 = 0.010728, 0.860051, 1.000000$ at $\sigma = 0.25, 0.5, 1$, by summation and by Equation 25.33 alike

tests/test_discrete_mechanisms.py checks that the output still matches.

25.8. Summary

Table 25.7.: The results of this chapter, and where each is used.

Result	Statement	Use
Proposition 25.1	$\delta \geq \Pr[M(D) \in \mathcal{Y}_D \setminus \mathcal{Y}_{D'}]$ for every ε	Auditing an implementation with one event
Example 25.1	The textbook floating-point sampler leaks about 48% of its mass	Never ship `f(D) + laplace(b)` in floating point
Theorem 25.1	Rounding to $\Lambda \geq b$ and clamping gives $\varepsilon = \frac{1+12B\omega}{b} + \frac{2^{-49}B}{b}$	Laplace noise when the output must be a float
Theorem 25.2	The parity of the first 0 among $\mathrm{Bern}(\lambda/k)$ coins is $\mathrm{Bern}(e^{-\lambda})$	The coin both samplers need; `exp` or `random()` in a sampler breaks exactness
Theorem 25.3	$f(D) + \mathrm{Lap}_{\mathbb{Z}}(b)$ is ε-DP with $\varepsilon = \Delta_1 f/b$	Counts and histograms, with variance about $2b^2 - \frac{1}{6}$
Theorem 25.4	Remainder, quotient and sign give $\mathrm{Lap}_{\mathbb{Z}}(t/s)$ exactly	10 to 12 uniform draws per sample
Theorem 25.5	The continuous $\delta(\varepsilon)$ with discrete tails	Census releases and secure aggregation
Theorem 25.6	$D_\alpha \leq \frac{\alpha(\mu-\nu)^2}{2\sigma^2}$, with equality at integer α	The same zero-concentrated DP as the continuous Gaussian (Part 4)
Proposition 25.3	$\mathrm{Var}(X) < \sigma^2$, by $O(\sigma^4 e^{-2\pi^2\sigma^2})$	Invisible for $\sigma \geq 1$; sums are only nearly discrete Gaussian

Result	Statement	Use
Theorem 25.7	Propose $\mathrm{Lap}_{\mathbb{Z}}(\lfloor\sigma\rfloor+1)$ and thin with exact coins	Fewer than two proposals per draw; the running time is a separate side channel

25.9. References

- Mironov, "On Significance of the Least Significant Bits for Differential Privacy" (CCS 2012). The floating-point attack on the textbook Laplace sampler, the snapping mechanism and its proof (Theorem 1 and §5).
- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020; journal version in the Journal of Privacy and Confidentiality, 2022). Algorithms 1–3, the exact $\delta(\varepsilon)$ (Theorem 7), the Rényi and concentrated-DP guarantee (Theorem 4), the variance and tail bounds via Poisson summation. Read §5 alongside the code here.
- Ghosh, Roughgarden & Sundararajan, "Universally Utility-Maximizing Privacy Mechanisms" (STOC 2009; SIAM Journal on Computing, 2012). The geometric mechanism and its optimality for a single count.
- von Neumann, "Various Techniques Used in Connection with Random Digits" (1951). The original trick behind Algorithm 1: exponential probabilities from comparisons of uniform variables.
- Jin, McMurtry, Rubinstein & Ohrimenko, "Are We There Yet? Timing and Floating-Point Attacks on Differential Privacy Systems" (IEEE S&P 2022). Floating-point and timing attacks on DP libraries released after Mironov's paper, including attacks on samplers whose running time depends on the output.
- Kairouz, Liu & Steinke, "The Distributed Discrete Gaussian Mechanism for Federated Learning with Secure Aggregation" (ICML 2021). The discrete Gaussian when each client adds its own noise, and how close sums of discrete Gaussians are to a discrete Gaussian.
- Abowd et al., "The 2020 Census Disclosure Avoidance System TopDown Algorithm" (Harvard Data Science Review, 2022). The largest deployment of the discrete Gaussian.
- Stein & Shakarchi, *Fourier Analysis: An Introduction* (2003), ch. 5. The Poisson summation formula and the theta function behind Equation 25.26.

26. Local, shuffle and distributed trust models

Every mechanism so far in this part assumed a **trusted curator**, who holds the raw data, runs the mechanism and releases only its output. The guarantee of differential privacy (DP) is about that output. It says nothing about the curator's database, the engineer who can read it or the court order that can demand it. As Threat models put it, trust decides who the adversary is. A browser vendor collecting usage statistics is exactly the party its users do not want to see their data.

This chapter asks how much accuracy each weaker trust assumption costs, and what you trust instead. There are four answers (Figure 26.1). In the **central** model the curator is trusted, and the noise is added once, to the answer. In the **local** model nobody is trusted, and each person randomizes their own record before it leaves their device. In the **shuffle** model a shuffler is trusted only to unlink reports from senders. In the **distributed** model cryptography is trusted, and each person adds a share of the noise to a sum that a protocol reveals only in total.

What separates the models is where the noise enters. The further upstream it enters, the fewer parties you trust and the more noise reaches the answer, because n independent noises of one size add up to about $\sqrt{n}$ times that size. The running example is the telemetry of an app, starting with one yes/no question put to $n = 10{,}000$ users, of whom 3,000 hold a 1. At $\varepsilon = \ln 3$, a trusted curator answers with standard deviation 1.287 on the count, while randomized response on every device has 86.6. The chapter proves that no local protocol avoids that factor of 67.3, and shows how shuffling and distributed noise win the accuracy back.

Table 26.1.: The symbols of this chapter.

Symbol	Meaning
ε_0; (ε, δ)	the local level of one report; the central guarantee
R, $p_R(y \mid x)$	a local randomizer, and the probability that it reports y on the record x
p, q	the probabilities that a report supports a value its sender holds, or does not hold (q is not a sampling rate here)
k, f_v, C_v	the domain size, the frequency of value v, the number of reports that support v
w, m	the number of hash buckets; the bound on a numeric value
T, b_0, b_m, α	the number of rounds; the kept bits and secret offset of α-point rounding

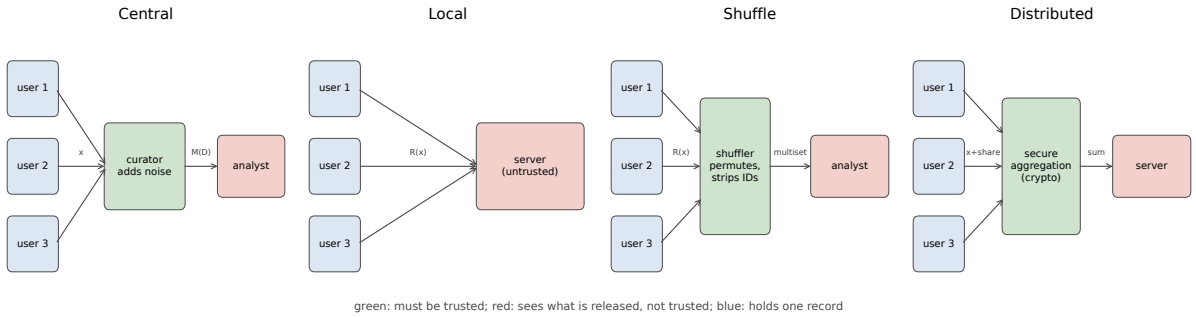


Figure 26.1.: The four trust models. Blue boxes hold one record each, green boxes must be trusted, and red boxes see the release and are not trusted.

26.1. Central and local differential privacy

In the **local model** there is no curator, and each person sends only a randomized version of their own record. This section asks what such a randomizer promises, to whom, and what it costs on the simplest task, counting one bit.

A report must look about the same whatever its owner's record is, because whoever receives it may know whose it is. So the requirement is the DP definition applied to a dataset of one record. In every model of this chapter, $D \sim D'$ means that the datasets differ by **replacing** one record. Each report arrives from a known device, so whether a person took part is visible anyway, and only their value can be hidden.

Definition 26.1 (Local randomizer)**.** Let $\mathcal{X}$ be the domain of one record and $\mathcal{Y}$ a set of reports. A randomizer $R\colon \mathcal{X} \to \mathcal{Y}$ is an **ε_0-DP local randomizer** if for every pair of records $x, x' \in \mathcal{X}$ and every event $S \subseteq \mathcal{Y}$,

$$\Pr[R(x) \in S] \le e^{\varepsilon_0} \Pr[R(x') \in S]. \tag{26.1}$$

Write $p_R(y \mid x)$ for the probability that R reports y on x. The **local protocol** releases $(R(x_1), \dots, R(x_n))$, with independent randomness for each person.

Replacing one record changes only one report, so the transcript should be exactly as private as one report.

Proposition 26.1 (What a local protocol promises)**.** *If R is an ε_0-DP local randomizer, the transcript of the local protocol is ε_0-DP under replace neighbours, and each report is ε_0-DP against anyone who sees it, including its collector.*

Proof.

1. *The transcript factorizes.* The randomness is independent, so a transcript has probability $\prod_i p_R(y_i \mid x_i)$.
2. *Neighbours differ in one factor.* If D and D' differ in record j, the other factors cancel in the ratio, which leaves $p_R(y_j \mid x_j)/p_R(y_j \mid x'_j) \le e^{\varepsilon_0}$ by Equation 26.1. Single outcomes extend to events by addition, as for randomized response.

3. *Who sees it does not matter.* Equation 26.1 is about the distribution of one report, so it holds for every observer, and any estimate computed from the reports is covered by post-processing.

□

So a breach of the collector, an insider or a subpoena reveals only reports that are already private. That is why Google's RAPPOR in Chrome, Apple's telemetry and Microsoft's telemetry in Windows used local DP. But each report must hide its owner from everyone, so each carries the noise that the central mechanism adds once in total. Table 26.2 compares the four models, and the sections below derive its last column.

Table 26.2.: The four trust models and the order of the error of a count.

Model	Who must be trusted	What the untrusted party sees	Error of a count, ε fixed
Central	The curator, with all raw records	$M(D)$	Order $1/\varepsilon$
Local	Nobody	Every report $R(x_i)$	Order $\sqrt{n}/\varepsilon$
Shuffle	The shuffler, to unlink reports	The multiset of reports	Order $\sqrt{\ln(1/\delta)}/\varepsilon$
Distributed	Cryptography, and enough honest clients	The noisy sum	Order $\sqrt{\ln(1/\delta)}/\varepsilon$

Every local estimator in this chapter is built the same way. A randomizer shrinks the signal, so the mean of a report moves less than the true value does, and the next lemma undoes the shrinking and says what that costs.

Lemma 26.1 (Debiasing a linear randomizer)**.** *Suppose a report Y has mean $\mathbb{E}[Y] = c_0 + c_1\vartheta$ for an unknown quantity ϑ, a known offset c_0 and a known slope $c_1 \neq 0$. Then*

$$\hat{\vartheta} = \frac{Y - c_0}{c_1} \tag{26.2}$$

is unbiased for ϑ, with $\mathrm{Var}(\hat{\vartheta}) = \mathrm{Var}(Y)/c_1^2$.

Proof.

1. Linearity of expectation gives $\mathbb{E}[\hat{\vartheta}] = (c_0 + c_1\vartheta - c_0)/c_1 = \vartheta$.
2. Subtracting a constant leaves a variance unchanged, and scaling by $1/c_1$ multiplies it by $1/c_1^2$.

□

A strongly private randomizer has a small slope, so the estimator multiplies the noise variance by a large $1/c_1^2$. That is where the cost of local privacy comes from. The variances below also use the fact from Distributions that variances of independent reports add. A 0/1 report has variance equal to its probability of a 1 times its probability of a 0.

The simplest local randomizer is **randomized response** from the definition of differential privacy. It reports the true bit with probability $t = e^{\varepsilon_0}/(1+e^{\varepsilon_0})$ and the flipped bit otherwise, so its two rows differ by the factor $t/(1-t) = e^{\varepsilon_0}$.

Proposition 26.2 (Randomized response on a fraction)**.** *Let a fraction π of n people hold a 1, and let each report their bit by randomized response at level ε_0. With $\bar{y}$ the mean of the reports, the estimate*

$$\hat{\pi} = \frac{\bar{y} - (1-t)}{2t-1} \tag{26.3}$$

is unbiased for π, and for every dataset

$$\mathrm{sd}(\hat{\pi}) = \frac{e^{\varepsilon_0/2}}{(e^{\varepsilon_0}-1)\sqrt{n}}. \tag{26.4}$$

Proof.

1. *The mean.* A holder of a 1 reports a 1 with probability t, and a holder of a 0 with probability $1-t$. So the reports have mean $(1-t)+\pi(2t-1)$, and Lemma 26.1 gives the estimate.
2. *The variance.* Every report has variance $t(1-t)$ whatever its bit, so the mean of n reports has variance $t(1-t)/n$, and the debiasing divides it by $(2t-1)^2$.
3. *In terms of ε_0.* Substituting t gives $t(1-t) = e^{\varepsilon_0}/(1+e^{\varepsilon_0})^2$ and $2t-1 = (e^{\varepsilon_0}-1)/(e^{\varepsilon_0}+1)$, and the factors $(1+e^{\varepsilon_0})^2$ cancel.

□

Example 26.1. Of the app's $n = 10{,}000$ users, 3,000 hold a 1, and each uses randomized response at $\varepsilon_0 = \ln 3$, so $t = 0.75$. There $e^{\varepsilon_0}/(e^{\varepsilon_0}-1)^2 = 3/4$, so Equation 26.4 gives a standard deviation of $\sqrt{0.75/10{,}000} = 0.00866$, and 20,000 simulated surveys give a mean of 0.3000 and a standard deviation of 0.00861. On the count, that is $10{,}000 \times 0.00866 = 86.6$ people.

A trusted curator would add Laplace noise to the count instead. One person moves the count by at most 1, so by Theorem 20.1 the noise at the same level $\varepsilon = \ln 3 = 1.0986$ has standard deviation $\sqrt{2}/\varepsilon = 1.287$. The local survey is $86.6/1.287 = 67.3$ times noisier.

26.2. The price of local privacy

Is the factor 67 a weakness of randomized response, or of the local model? This section shows that it is the model. No local protocol estimates a fraction with error below order $1/((e^{\varepsilon_0}-1)\sqrt{n})$, while the central Laplace mechanism reaches order $1/(n\varepsilon)$. The shuffle and distributed models exist to escape this gap.

The proof measures what the reports reveal with the **Kullback–Leibler (KL) divergence** of Divergences, $D_{\mathrm{KL}}(Q_0 \| Q_1) = \sum_y q_0(y) \ln \frac{q_0(y)}{q_1(y)}$, which adds over independent observations.

The proof compares two populations by their **total variation (TV) distance** $\mathrm{TV}(P_0, P_1) = \frac{1}{2}\sum_x |P_0(x) - P_1(x)|$. The key step is that a randomizer changes each report's probability by at most a factor e^{ε_0}, so the reports of two populations differ only a little, however different the populations are.

Lemma 26.2 (A local randomizer contracts KL divergence)**.** *Let each record be drawn from P_0 or P_1 on a finite domain, and let R be an ε_0-DP local randomizer. Write Q_0 and Q_1 for the distributions of one report, $q_j(y) = \sum_x P_j(x)\, p_R(y \mid x)$. Then*

$$D_{\mathrm{KL}}(Q_0 \| Q_1) \le (e^{\varepsilon_0} - 1)^2\, \mathrm{TV}(P_0, P_1)^2. \tag{26.5}$$

Proof. The plan is to bound $q_0(y) - q_1(y)$ by a multiple of $q_1(y)$ for each report y, then turn that into a bound on KL with $\ln u \le u - 1$.

1. *A band.* Let $\underline{p}(y) = \min_x p_R(y \mid x)$. By Equation 26.1, $g(x) = p_R(y \mid x) - \underline{p}(y)$ lies in $[0, G]$ with $G = (e^{\varepsilon_0} - 1)\underline{p}(y)$.
2. *Only the difference matters.* The differences $P_0(x) - P_1(x)$ sum to 0, so subtracting the floor $\underline{p}(y)$, which does not depend on x, changes nothing:
$$q_0(y) - q_1(y) = \sum_x \big(P_0(x) - P_1(x)\big)\, g(x). \tag{26.6}$$
3. *Bound the sum.* The differences sum to 0, so the positive ones and the negative ones each total $\mathrm{TV}(P_0, P_1)$. With g between 0 and G, the sum lies within $\pm \mathrm{TV} \cdot G$. Also $q_1(y)$ is an average of numbers that are each at least $\underline{p}(y)$, so $\underline{p}(y) \le q_1(y)$ and
$$|q_0(y) - q_1(y)| \le (e^{\varepsilon_0} - 1)\, \mathrm{TV}(P_0, P_1)\, q_1(y). \tag{26.7}$$
4. *To KL.* The concave curve $\ln u$ lies below its tangent $u - 1$ at $u = 1$. Applied to $u = q_0(y)/q_1(y)$,
$$D_{\mathrm{KL}}(Q_0 \| Q_1) \le \sum_y q_0 \Big(\frac{q_0}{q_1} - 1\Big). \tag{26.8}$$
5. *Complete the square.* Since $\sum_y q_0 = \sum_y q_1 = 1$, the right side equals $\sum_y (q_0 - q_1)^2 / q_1$. Squaring step 3 and summing bounds it by $(e^{\varepsilon_0} - 1)^2\, \mathrm{TV}^2 \sum_y q_1$, and the last sum is 1.

□

The lower bound below turns little information into large error. Its last step needs a second lemma, that a good estimator is a good test. By Hypothesis testing, every test between two distributions has error rates summing to at least $1 - \mathrm{TV}$.

Lemma 26.3 (A test built from an estimator)**.** *Suppose an estimator $\hat{\vartheta}$ is within $a/2$ of the truth with probability at least $1 - \beta$, both when the truth is ϑ_0 and when it is $\vartheta_0 + a$. Then the two distributions of the data have TV distance at least $1 - 2\beta$.*

Proof.

1. Decide $\vartheta_0 + a$ when $\hat{\vartheta} > \vartheta_0 + a/2$, the midpoint.

2. The test errs only when the estimate misses the truth by at least $a/2$, which has probability at most β under each value.
3. Every test has error rates summing to at least $1 - \mathrm{TV}$, so $1 - \mathrm{TV} \le 2\beta$.

□

The theorem compares two populations whose fractions differ by a small gap a. Their reports are too alike to tell apart, so no estimator is accurate to half the gap under both.

Theorem 26.1 (The local lower bound for a fraction)**.** *Let each of n people apply an ε_0-DP local randomizer, possibly different per person, to a bit drawn from* $\mathrm{Bern}(\frac{1}{2})$ *or from* $\mathrm{Bern}(\frac{1}{2} + a)$*. No estimator built from the reports is within $a/2$ of the true fraction with probability at least $1 - \beta$ under both unless*

$$\frac{a}{2} \ge \frac{1 - 2\beta}{(e^{\varepsilon_0} - 1)\sqrt{2n}}. \tag{26.9}$$

Proof.

1. *Add over people.* The reports are independent, so KL adds over people. The two populations have TV distance a, so by Lemma 26.2 the n reports have KL at most $n(e^{\varepsilon_0} - 1)^2 a^2$.
2. *Convert to TV.* TV does not add over people, which is why the proof went through KL. **Pinsker's inequality** (Divergences) says $\mathrm{TV} \le \sqrt{D_{\mathrm{KL}}/2}$, so for the distributions Q_0^n, Q_1^n of all n reports
$$\mathrm{TV}(Q_0^n, Q_1^n) \le a(e^{\varepsilon_0} - 1)\sqrt{n/2}. \tag{26.10}$$
3. *Turn information into error.* Lemma 26.3 forces this TV to be at least $1 - 2\beta$, so $a \ge (1 - 2\beta)\sqrt{2}/((e^{\varepsilon_0} - 1)\sqrt{n})$, and halving gives Equation 26.9.

□

Randomized response is optimal up to a constant, since dividing Equation 26.4 by Equation 26.9 leaves $\sqrt{2}\, e^{\varepsilon_0/2}/(1 - 2\beta)$. The contraction is loose by a constant. For one report at $\varepsilon_0 = 1$, comparing Bern(0.5) with Bern(0.6), the exact KL is 0.00429 and Equation 26.5 gives 0.02952. Duchi, Jordan & Wainwright (2018) apply the contraction to many local problems. Bounds of the same order hold even when each randomizer may depend on earlier reports.

Example 26.2. At $\varepsilon = 1$ and $n = 1{,}000$ (Figure 26.2), randomized response has standard deviation 3.03×10^{-2} and the lower bound at $\beta = 0.1$ is 1.04×10^{-2}. Their ratio 2.91 is the constant $\sqrt{2}\, e^{1/2}/(1 - 2\beta) = 1.4142 \times 1.6487/0.8$. The central Laplace mechanism gives $\sqrt{2}/1000 = 1.41 \times 10^{-3}$, 21 times less. That ratio, $e^{1/2}/((e-1)\sqrt{n})$ divided by $\sqrt{2}/n$, is $0.678\sqrt{n}$, so it is 215 at $n = 100{,}000$ and 2146 at $n = 10^7$. For a standard deviation of 0.001, randomized response needs 920,674 people and the central mechanism 1,415. So local DP suits only the frequent items of very large populations.

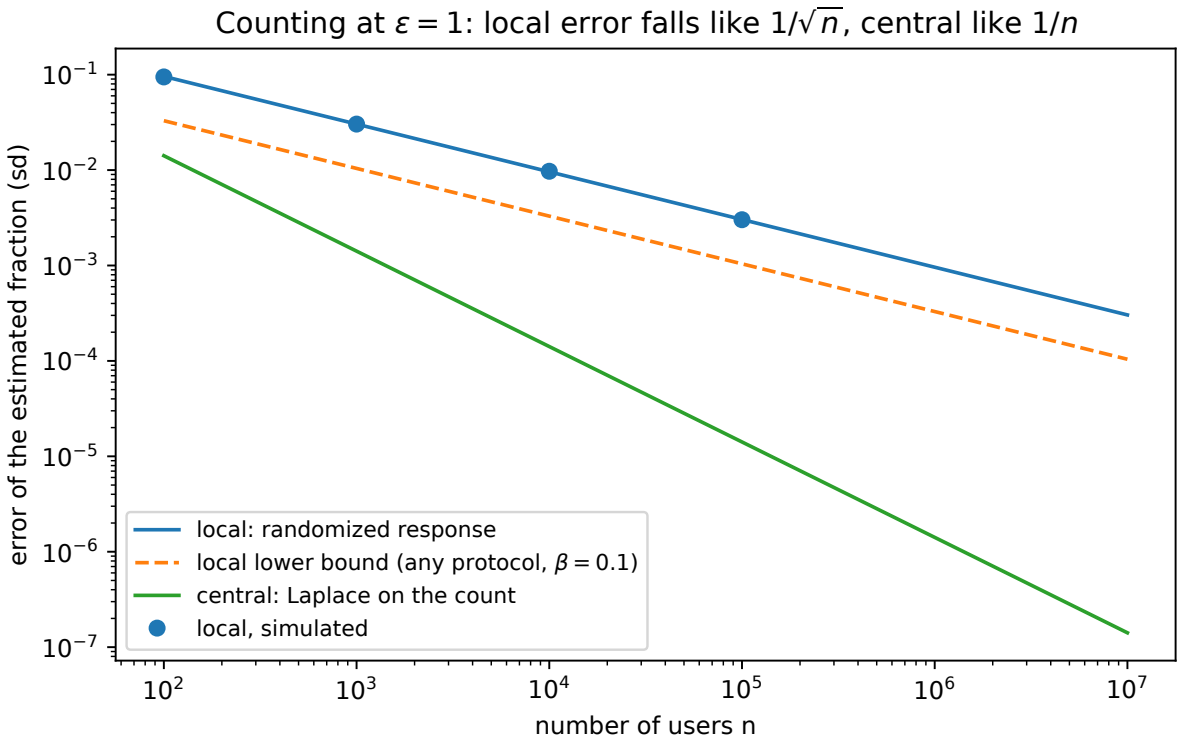


Figure 26.2.: Standard deviation of an estimated fraction at $\varepsilon = 1$. Randomized response (line, simulated dots) falls like $1/\sqrt{n}$, a constant above the local lower bound; the central Laplace mechanism falls like $1/n$.

26.3. Frequency estimation

In **frequency estimation** each person holds one of k values, such as an emoji, and the analyst wants the fraction f_v holding each value v. It is the workhorse of deployed local DP. This section compares two generalizations of randomized response, which behave very differently as k grows.

Both fit one template. For each value v, a report either **supports** v or not, and it supports v more often when its sender holds v, so Lemma 26.1 unshrinks the count of supports.

Proposition 26.3 (A pure frequency oracle)**.** *Suppose that for each value v a report supports v with probability p when its sender holds v and $q < p$ otherwise, independently across people. Let C_v count the reports that support v. Then the estimate $\hat{f}_v$ below is unbiased, with the exact variance shown:*

$$\hat{f}_v = \frac{C_v/n - q}{p - q}, \qquad \mathrm{Var}(\hat{f}_v) = \frac{f_v p(1-p) + (1-f_v)q(1-q)}{n(p-q)^2}. \tag{26.11}$$

Proof.

1. *Unbiased.* The holders support v with probability p and the others with q, so $\mathbb{E}[C_v/n] = q + f_v(p-q)$, and Lemma 26.1 applies.
2. *Variance.* The independent supports have variances $p(1-p)$ and $q(1-q)$, which add. Dividing by n^2 for the mean and by $(p-q)^2$ for the debiasing gives Equation 26.11.

□

What decides utility is the variance at $f_v = 0$. Most values of a large domain are rare, and below this noise floor a value cannot be told from an absent one.

Proposition 26.4 (k-ary randomized response and unary encoding)**.** ***k-ary randomized response*** *reports the true value with probability $p = \frac{e^{\varepsilon_0}}{e^{\varepsilon_0}+k-1}$ and each other value with probability $q = \frac{1}{e^{\varepsilon_0}+k-1}$, and a report supports the value it names. It is an ε_0-DP local randomizer, with rare-value variance*

$$\frac{e^{\varepsilon_0} + k - 2}{n(e^{\varepsilon_0} - 1)^2}. \tag{26.12}$$

Unary encoding *writes the value as k bits with only bit v set, sends each 1 as 1 with probability p and each 0 as 1 with probability q, and supports v if bit v is sent as 1. It is an ε_0-DP local randomizer with*

$$\varepsilon_0 = \ln \frac{p(1-q)}{q(1-p)}. \tag{26.13}$$

*The **symmetric** choice $q = 1 - p$, which is basic one-shot RAPPOR, has rare-value variance $e^{\varepsilon_0/2}/(n(e^{\varepsilon_0/2} - 1)^2)$. The **optimized** choice $p = \frac{1}{2}$, $q = 1/(e^{\varepsilon_0} + 1)$ (Wang et al. 2017) has $4e^{\varepsilon_0}/(n(e^{\varepsilon_0} - 1)^2)$. k-ary randomized response beats optimized unary encoding exactly when $k < 3e^{\varepsilon_0} + 2$.*

Proof. The plan is to check both privacy levels, then put each scheme's p and q into Equation 26.11 at $f_v = 0$.

1. *k-ary randomized response.* A report has probability p under the record it names and q under any other, so two records differ by at most $p/q = e^{\varepsilon_0}$.
2. *Unary encoding.* Changing x to x' changes two bits, and the others cancel. Bit x contributes p/q or $(1-p)/(1-q)$, and bit x' contributes q/p or $(1-q)/(1-p)$. Since $p > q$, the largest product is $\frac{p}{q} \cdot \frac{1-q}{1-p}$, so the two bits together cost ε_0, not $2\varepsilon_0$.
3. *Rare-value variances.* At $f_v = 0$, Equation 26.11 is $q(1-q)/(n(p-q)^2)$. For k-ary randomized response the factors $(e^{\varepsilon_0} + k - 1)^2$ cancel. For the optimized choice, $p - q = (e^{\varepsilon_0} - 1)/(2(e^{\varepsilon_0} + 1))$, and the squared 2 leaves the 4. The symmetric choice is binary randomized response at $\varepsilon_0/2$ per bit, so Proposition 26.2 gives its variance.
4. *Crossover.* The two variances share a denominator, so compare $e^{\varepsilon_0} + k - 2$ with $4e^{\varepsilon_0}$.

□

k-ary randomized response reports the truth with probability about e^{ε_0}/k, so its variance grows linearly in k (Figure 26.3). Unary encoding pays ε_0 once for two bits, so its variance does not depend on k, at the price of a k-bit report. The optimized choice rarely invents a 1, which matters because almost all bits are 0. Every entry of Table 26.3 is still divided by n, so the gap of Theorem 26.1 remains.

Table 26.3.: n times the rare-value variance at $\varepsilon_0 = 1$. k-ary randomized response wins below $k = 3e + 2 = 10.15$.

k	k-ary randomized response	Symmetric unary	Optimized unary
2	0.921	3.918	3.683
8	2.953	3.918	3.683
32	11.082	3.918	3.683
128	43.596	3.918	3.683

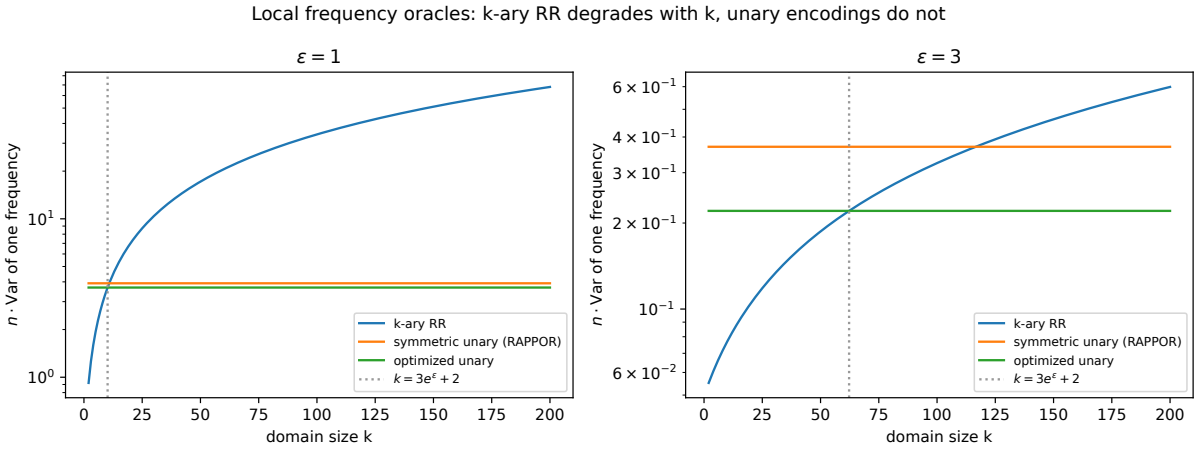


Figure 26.3.: n times the rare-value variance against k, at $\varepsilon_0 = 1$ and 3. The dotted line is the crossover $k = 3e^{\varepsilon_0} + 2$.

Example 26.3. The app's keyboard wants the frequency of each of $k = 128$ emoji at $\varepsilon_0 = 1$ from $n = 10^6$ users. k-ary randomized response gives a rare emoji a standard deviation of $\sqrt{43.596/10^6} = 0.0066$, and optimized unary encoding gives $\sqrt{3.683/10^6} = 0.0019$. An estimate must stand about three standard deviations above 0, so even with optimized unary encoding, emoji used by fewer than about 0.6% of users are lost. With $k = 8$, below the crossover, k-ary randomized response is better, at $\sqrt{2.953/10^6} = 0.0017$.

26.4. Large and unknown domains

A unary report carries one bit per possible value, which is absurd for all web addresses. And an oracle answers only for values the analyst names in advance, while a keyboard app wants words nobody listed. This section removes both limits. Hashing makes reports short, and growing prefixes finds the frequent values without a dictionary, the **heavy hitters** problem.

In **local hashing** each person draws a hash function h from a public family that maps the domain to w buckets, so that two values collide with probability $1/w$. They send the index of h and a w-ary randomized response of $h(x)$, with the p and q of Proposition 26.4 at w in place of k. A report supports v if its bucket equals $h(v)$.

Proposition 26.5 (Local hashing)**.** *Let $\hat{g}_v$ be the estimate of Equation 26.11 applied to the supports of v, which estimates the share of people whose own bucket is $h(v)$. Then the estimate below is unbiased for f_v, with the stated variance at $f_v = 0$:*

$$\hat{f}_v = \frac{w\,\hat{g}_v - 1}{w - 1}, \qquad \operatorname{Var}(\hat{f}_v)\big|_{f_v=0} = \frac{(e^{\varepsilon_0} + w - 1)^2}{n\,(w-1)\,(e^{\varepsilon_0} - 1)^2}. \tag{26.14}$$

It is smallest at $w = e^{\varepsilon_0} + 1$, where it equals optimized unary encoding's $4e^{\varepsilon_0}/(n(e^{\varepsilon_0} - 1)^2)$.

Proof. The plan is to undo two shrinkings with Lemma 26.1, the randomized response and then the collisions, and then to minimize the rare-value variance.

1. *The own bucket.* A holder of v lands in $h(v)$, and anyone else does with probability $1/w$. So a person's bucket is $h(v)$ with probability $g_v = \frac{1}{w} + f_v \frac{w-1}{w}$.
2. *The randomized response.* A report equals $h(v)$ with probability p if the bucket is $h(v)$ and q otherwise, so Lemma 26.1 makes $\hat{g}_v$ unbiased for g_v.
3. *The collisions.* Step 1 has offset $1/w$ and slope $(w-1)/w$, so Lemma 26.1 gives $\hat{f}_v$, with $(w/(w-1))^2$ times the variance of $\hat{g}_v$.
4. *A rare value.* At $f_v = 0$ a report supports v with probability exactly $1/w$, so each support has variance $(w-1)/w^2$. Dividing by n and $(p-q)^2$ and multiplying by $w^2/(w-1)^2$ leaves $1/(n(w-1)(p-q)^2)$, and $p - q = (e^{\varepsilon_0} - 1)/(e^{\varepsilon_0} + w - 1)$.
5. *The best width.* With $u = w - 1$, the part that depends on w is $e^{2\varepsilon_0}/u + 2e^{\varepsilon_0} + u$. Two positive terms with a fixed product have the smallest sum when equal, at $u = e^{\varepsilon_0}$.

□

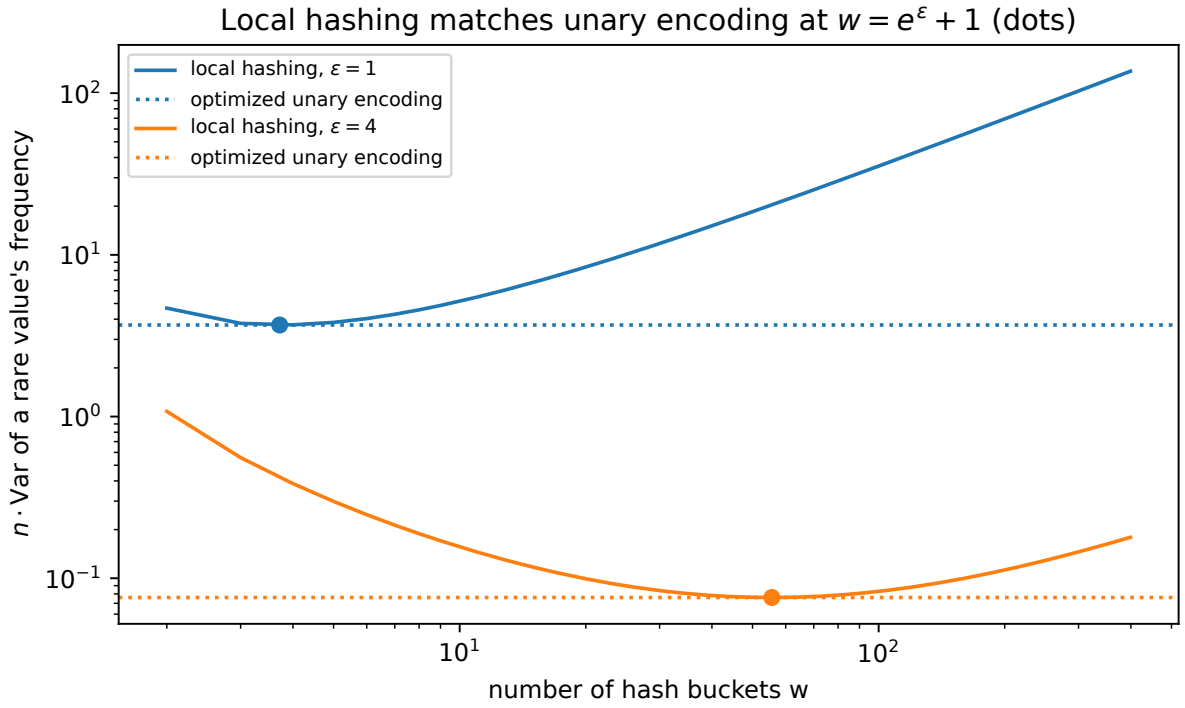


Figure 26.4.: n times the rare-value variance of local hashing against w, at $\varepsilon_0 = 1$ and 4. Dots mark $w = e^{\varepsilon_0} + 1$; dotted lines are optimized unary encoding.

Too few buckets waste the signal on collisions, and too many spread it thin (Figure 26.4). At $\varepsilon_0 = 4$, with $w = 56$, a report needs about 6 bits. The price is the server's work, which Hadamard response (Acharya, Sun & Zhang 2019) makes fast at a constant factor in variance.

To find unlisted values, write each as a string of L symbols from an alphabet of A symbols, and split the people at random into L groups. Group ℓ reports its first ℓ symbols, its **prefix**, by local hashing. The analyst keeps the prefixes whose estimate passes a threshold, extends each by every symbol, and moves to the next group. Write s for the standard deviation of an absent value's estimate, and Φ for the standard normal distribution function.

Proposition 26.6 (Heavy hitters by growing prefixes)**.** *If c_ℓ candidates are tested at level ℓ, then under the normal approximation the threshold*

$$\tau_\ell = z_\ell \, s, \qquad \text{where} \quad c_\ell \, (1 - \Phi(z_\ell)) = \beta, \tag{26.15}$$

admits on average β false candidates. Every prefix of a string is at least as frequent as the string, and each level tests only A extensions of each kept prefix.

Proof.

1. *The threshold.* An absent value's estimate averages many independent reports, so it is close to normal with mean 0, and exceeds $z\,s$ with probability about $1 - \Phi(z)$. Linearity of expectation sums this over the c_ℓ candidates.

2. *Prefixes.* Each holder of a string holds all its prefixes, so a prefix is at least as frequent as every string that starts with it.

□

The search is the TreeHist protocol of Bassily et al. (2017), and Bassily & Smith (2015) show that no local protocol finds values rarer than order $\sqrt{\ln k/n}/\varepsilon_0$.

Example 26.4. A million users each type one of 200 six-letter words with Zipf frequencies, at $\varepsilon_0 = 4$. The domain has $26^6 = 308{,}915{,}776$ strings. Local hashing uses $w = 56$, the integer nearest $e^4 + 1 = 55.60$, from 16,384 hash functions, so that collisions average out. Each of the six levels sees about 166,666 people, giving $s = 6.75 \times 10^{-4}$, and no level tests more than 2,054 candidates. At the last level, $\beta = 0.1$ puts the threshold at 3.80 standard deviations, a frequency of 0.0026.

The search returns 50 strings, all real words (Figure 26.5). The rarest word found has frequency 0.0033, and the most frequent word missed has 0.0035, lost to noise at one level. Testing all 26^6 strings would need 6.18 standard deviations, and at 3.80 it would raise about 22,418 false alarms.

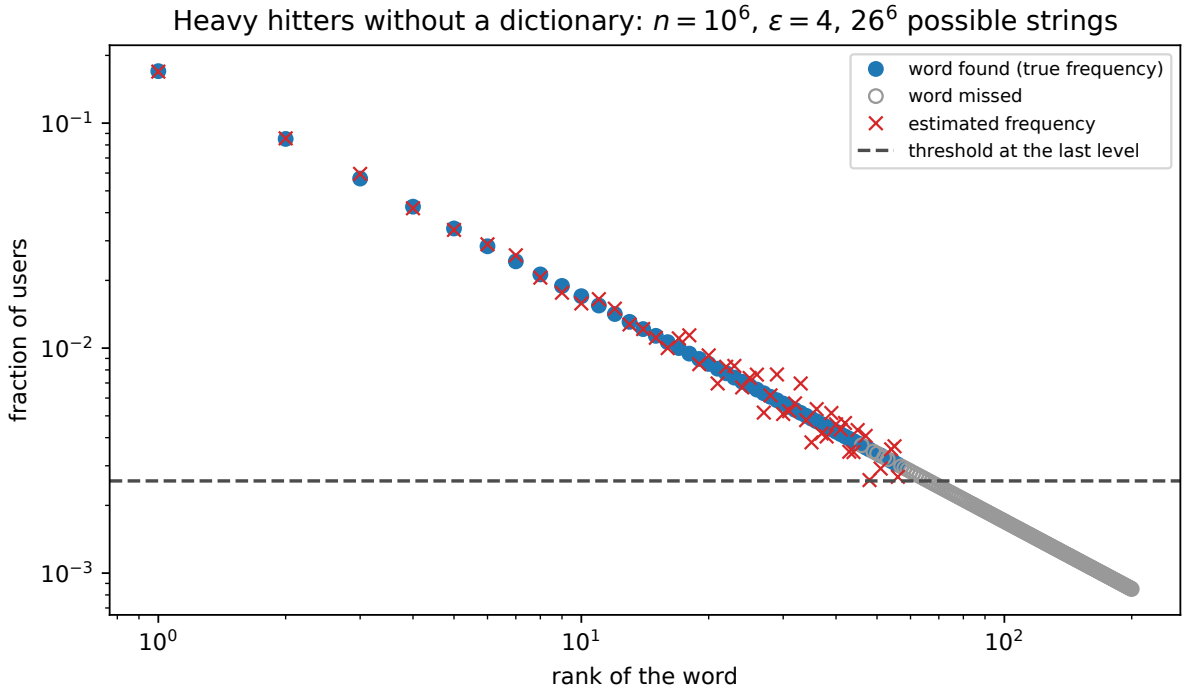


Figure 26.5.: The 200 words by rank: true frequency (circles, filled when found) and the estimate for each word found (crosses). The dashed line is the last level's threshold.

26.5. Estimating a mean

Telemetry also collects numbers, such as minutes in the app. Each person holds x_i between 0 and a known bound m, and the analyst wants the mean $\mu = \frac{1}{n}\sum_i x_i$. The questions are how accurate a local mean can be, and how few bits each phone must send.

Adding Laplace noise to each value is the obvious randomizer. Ding, Kulkarni & Yekhanin (2017) send one bit instead, whose probability of a 1 rises linearly from randomized response's lie probability at $x = 0$ to its truth probability at $x = m$.

Proposition 26.7 (Local means)**.** *Reports $x_i + \mathrm{Lap}(m/\varepsilon_0)$ are ε_0-DP, and their average is unbiased with standard deviation $\sqrt{2}\,m/(\varepsilon_0\sqrt{n})$. A bit Y_i with*

$$\Pr[Y_i = 1] = \frac{1}{e^{\varepsilon_0}+1} + \frac{x_i}{m}\cdot\frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1} \tag{26.16}$$

is also ε_0-DP, and the estimate below is unbiased with the stated bound for all data:

$$\hat{\mu} = \frac{m}{n}\sum_{i=1}^{n}\frac{(e^{\varepsilon_0}+1)Y_i - 1}{e^{\varepsilon_0}-1}, \qquad \mathrm{sd}(\hat{\mu}) \le \frac{m}{2\sqrt{n}}\cdot\frac{e^{\varepsilon_0}+1}{e^{\varepsilon_0}-1}. \tag{26.17}$$

Proof. The plan is to handle the Laplace reports in one step, then show that the bit is private, unbiased and of bounded variance.

1. *Laplace reports.* One value moves by at most m, so Theorem 20.1 applies, and the average of n variances $2m^2/\varepsilon_0^2$ has variance $2m^2/(n\varepsilon_0^2)$.
2. *Private.* For every value, both outcome probabilities of the bit lie between $1/(e^{\varepsilon_0}+1)$ and $e^{\varepsilon_0}/(e^{\varepsilon_0}+1)$, whose ratio is e^{ε_0}.
3. *Unbiased.* The mean of Y_i is linear in x_i, so Lemma 26.1 makes each summand of $\hat{\mu}$ unbiased for x_i.
4. *Variance.* A 0/1 variable has variance at most $1/4$, the summand scales it by $(m(e^{\varepsilon_0}+1)/(e^{\varepsilon_0}-1))^2$, and the average divides by n.

□

Both errors are of order $m/(\varepsilon_0\sqrt{n})$, against $m/(\varepsilon_0 n)$ centrally, and Duchi, Jordan & Wainwright (2018) show that no local protocol beats this order. The bit wins on communication, since a Laplace report needs about $\log_2 m$ bits. Its bound is 0.765 times the Laplace standard deviation at $\varepsilon_0 = 1$, and the two are equal at $\varepsilon_0 = 2.324$.

The bound m must be fixed before collection, because every randomizer uses it. A device whose value exceeds m clips it, which biases the mean down, so a bound set too low trades noise for bias.

Example 26.5. A million users spend between 0 and $m = 1{,}440$ minutes a day in the app, with a mean of 82.74. At $\varepsilon_0 = 1$ the one-bit estimate has standard deviation 1.420 minutes, within the bound 1.558 of Equation 26.17. In 200 simulated collections the estimates average 82.61, with standard deviation 1.415. Laplace reports would give 2.036 minutes, at about 11 bits each.

26.6. Collecting every day

Telemetry asks the same question every day, and basic composition (Post-processing, group privacy and composition) prices T reports of one record at $T\varepsilon_0$. This section shows that the price is real, then gives the two fixes deployments use, **memoization** and **α-point rounding**.

Proposition 26.8 (Fresh reports leak)**.** *Suppose a person reports the same bit every day by randomized response at level ε_0, with fresh randomness. After T days with k ones, the likelihood ratio between bit 1 and bit 0 is $e^{\varepsilon_0(2k-T)}$, and the majority of the reports is wrong with a probability that falls exponentially in T.*

Proof.

1. *The ratio.* A sequence with k ones has probability $t^k(1-t)^{T-k}$ under bit 1 and $(1-t)^k t^{T-k}$ under bit 0, and $t/(1-t) = e^{\varepsilon_0}$.
2. *The majority.* It errs when more than half the reports flip, a binomial tail with flip probability below $\frac{1}{2}$, which falls exponentially by the Chernoff bound of Concentration inequalities.

□

The fix is to randomize once. In memoization the person draws one permanent randomized response per value and reuses it. RAPPOR encodes the value as a **Bloom filter**, a bit vector in which H hash functions each set one bit, and the permanent response replaces each bit by a fair coin with probability ϕ.

Proposition 26.9 (Memoization)**.** *All reports of an unchanged value, however many, are together as private as its permanent response. For RAPPOR that level, written ε_∞ because it covers unboundedly many reports, is*

$$\varepsilon_\infty = 2H \ln \frac{1-\phi/2}{\phi/2}. \tag{26.18}$$

Proof.

1. *Post-processing.* The reports are a function of the permanent response and of randomness independent of the data.
2. *RAPPOR.* A 1 is reported as 1 with probability $1-\phi/2$ and a 0 with probability $\phi/2$. Two values' filters differ in at most $2H$ bits, and the others cancel.

□

Each new value costs another ε_∞. RAPPOR's homepage collection used $H = 2$ and $\phi = 0.75$, giving $\varepsilon_\infty = 4\ln(5/3) = 2.043$. A fixed response would let the collector link a sender's reports across days, like a cookie, so RAPPOR re-randomizes it before each report, and with the deployment's re-randomization probabilities one report is then ε_1-DP with $\varepsilon_1 = 0.5343$.

A value that drifts, such as minutes, would need a new response every day. Ding, Kulkarni & Yekhanin (2017) instead draw, once, a bit b_0 by Equation 26.16 at $x = 0$, a bit b_m at $x = m$ and an offset α uniform on $[0, m)$. In round t the person sends b_m if $x_t + \alpha \geq m$ and b_0 otherwise, so

that the random offset picks b_m with probability x_t/m. **Output perturbation** then flips each sent bit with a small probability κ, to blur the days on which the sent bit switches.

Proposition 26.10 (α-point rounding)**.** *With values in* $[0, m]$*, and* b_0*,* b_m *and* α *drawn once, independently of the data:*

1. *Each report has the distribution of Equation 26.16, so each round's* $\hat{\mu}$ *satisfies Equation 26.17.*
2. *The reports depend on the data only through the days with* $x_t + \alpha \geq m$*, and with probability* $2e^{\varepsilon_0}/(e^{\varepsilon_0}+1)^2$ *the kept bits agree, so every report is the same.*
3. *Values that stay in a band of width* d *change the report only if* $m - \alpha$ *falls in the band, which has probability* d/m*.*
4. *With output perturbation one round is* ε_0'*-DP, with*

$$\varepsilon_0' = \ln \frac{(1-2\kappa)\frac{e^{\varepsilon_0}}{e^{\varepsilon_0}+1} + \kappa}{(1-2\kappa)\frac{1}{e^{\varepsilon_0}+1} + \kappa}, \tag{26.19}$$

and if, for every offset, two sequences of values select different kept bits on at most r *days, their transcripts have likelihood ratio at most* $((1-\kappa)/\kappa)^r$*.*

Proof. The plan is to average over the offset for item 1, read items 2 and 3 off the construction, and multiply daily ratios for item 4.

1. *Unbiased.* For a value x, b_m is sent with probability $\Pr[\alpha \geq m - x] = x/m$, so the report is 1 with probability $(1 - x/m)/(e^{\varepsilon_0}+1) + (x/m)\, e^{\varepsilon_0}/(e^{\varepsilon_0}+1)$.
2. *Dependence.* Each report is b_0 or b_m, chosen by $x_t + \alpha \geq m$. Both kept bits are 1 with probability $e^{\varepsilon_0}/(e^{\varepsilon_0}+1)^2$, and both 0 with the same probability. The choice switches only if $m - \alpha$, uniform on $(0, m]$, lies in the band.
3. *One round.* The flip maps a probability π of a 1 to $\kappa + (1-2\kappa)\pi$, still linear, so the worst ratio is between the ends of Equation 26.16.
4. *Transcripts.* Fix the kept bits and offset. A day sending the same kept bit under both sequences cancels, and a differing day has ratio at most $(1-\kappa)/\kappa$. The flips are independent, so the ratios multiply, for every offset.

□

So the collector learns nothing about people whose kept bits agree, and about the others only when their value crosses a secret threshold, so the guarantee protects steady behaviour, while a person whose behaviour changes reveals roughly when. Microsoft deployed the protocol in Windows 10 at $\varepsilon_0 = 1$ every six hours with $\kappa = 0.2$, where Equation 26.19 gives 0.569 per round, while the paper reports a more conservative 0.686.

Example 26.6. Fresh reports at $\varepsilon_0 = \ln 3$ leak fast. After 100 days with 75 ones the likelihood ratio is 3^{50}, about 7.18×10^{23}, while a memoized report leaves the majority wrong with probability $1 - t = 0.25$ (Figure 26.6).

For the app's minutes, with $m = 1{,}440$ and $\varepsilon_0 = 1$, the kept bits agree with probability $2e/(e+1)^2 = 0.3932$. Let each person's minutes wobble by up to 15 either side of their level for 30 days. By item 3, the fraction whose report ever changes is $(1 - 0.3932)$ times the expected range of their values over m, which is 0.0118. A simulation of 100,000 people gives 0.0119, against 1.0000

with fresh one-bit reports. On one freshly simulated day the α-point estimate is 82.17 minutes against that day's true mean of 82.68, within the standard deviation of 1.42. With $\kappa = 0.2$, each change costs $\ln(0.8/0.2) = 1.386$.

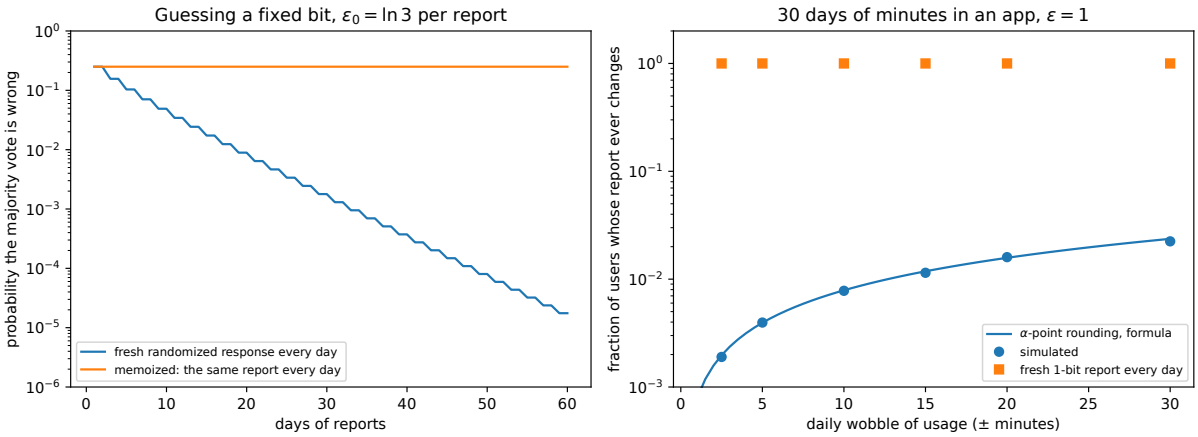


Figure 26.6.: Left: how often the majority of T daily reports is wrong, fresh or memoized. Right: the fraction of reports that ever change in 30 days, α-point or fresh.

26.7. The shuffle model

The local model loses $\sqrt{n}$ because the analyst knows whose report is whose. In the **shuffle model** (Bittau et al. 2017; Cheu et al. 2019) each person sends a randomized report to a **shuffler**, which strips identifiers and forwards a random permutation. The shuffler is trusted only to unlink, since the reports it sees are ε_0-DP. This section asks how far the central ε then falls.

Call the person whose privacy is at stake the target. The more reports the target's report is mixed with, the less the multiset reveals. The bound to compute with is Theorem 3.1 of Feldman, McMillan & Talwar (2021), in this book's notation and for replace neighbours. Its randomizers may depend on earlier outputs, which covers adaptive protocols.

Theorem 26.2 (Amplification by shuffling)**.** *Let $R_i(z_{1:i-1}, \cdot)$, for $i = 1, \dots, n$, be an ε_0-DP local randomizer for every value of the earlier outputs $z_{1:i-1}$. Let M_{sh} draw a uniform random permutation Π, compute $z_i = R_i(z_{1:i-1}, x_{\Pi(i)})$ in turn and output $z_{1:n}$. For any $\delta \in [0,1]$ with $\varepsilon_0 \le \ln\left(\frac{n}{16\ln(2/\delta)}\right)$, M_{sh} is (ε, δ)-DP with*

$$\varepsilon \le \ln\left(1 + \frac{e^{\varepsilon_0} - 1}{e^{\varepsilon_0} + 1}\left(\frac{8\sqrt{e^{\varepsilon_0}\ln(4/\delta)}}{\sqrt{n}} + \frac{8e^{\varepsilon_0}}{n}\right)\right). \tag{26.20}$$

The proof is in Amplification by shuffling and by iteration, which reduces the shuffled output to two binomial counts, and Section L.3 bounds their hockey-stick divergence. Its first step, the lemma below, carries the intuition that other people's reports can look exactly like the target's.

Lemma 26.4 (Clones)**.** *Let R be an ε_0-DP local randomizer and x^0, x^1 the two values the target might hold. Then for every other value x there is a distribution $\mathrm{LO}(x)$, the leftover, with*

$$R(x) = \frac{e^{-\varepsilon_0}}{2} R(x^0) + \frac{e^{-\varepsilon_0}}{2} R(x^1) + (1 - e^{-\varepsilon_0})\,\mathrm{LO}(x). \tag{26.21}$$

Proof.

1. *A floor.* By Equation 26.1, $p_R(y \mid x)$ is at least $e^{-\varepsilon_0} p_R(y \mid x^0)$ and at least $e^{-\varepsilon_0} p_R(y \mid x^1)$, so it is at least half their sum.
2. *The leftover.* Subtracting the two pieces leaves a non-negative mass of total $1 - e^{-\varepsilon_0}$, which rescales to $\mathrm{LO}(x)$.

□

So with probability $e^{-\varepsilon_0}$ each other person sends a **clone**, a report distributed like one of x^0 or x^1. About $(n-1)e^{-\varepsilon_0}$ clones, split at random between the two values, hide the target after shuffling. The theorem's condition, $ne^{-\varepsilon_0} \geq 16\ln(2/\delta)$, asks for enough expected clones.

Example 26.7. For 4-ary randomized response at $\varepsilon_0 = \ln 3$ (Figure 26.7), $p = \frac{1}{2}$ and $q = \frac{1}{6}$. Take a target with $x^0 = 1$ or $x^1 = 2$. Another person with value 3 reports $(\frac{1}{6}, \frac{1}{6}, \frac{1}{2}, \frac{1}{6})$, the clone part is $\frac{1}{6}(R(1) + R(2)) = (0.1111, 0.1111, 0.0556, 0.0556)$, and the leftover has mass $0.6667 = 1 - e^{-\varepsilon_0}$.

For central $(1, 10^{-6})$, bisection on Equation 26.20 allows each user $\varepsilon_0 = 3.504$ at $n = 10^4$, 5.701 at 10^5 and 7.992 at 10^6. At 7.992, 8-ary randomized response keeps the truth with probability 0.998. For adding up bits as in Cheu et al. (2019), only about 676 of a million people then send a random bit, since binary randomized response at that level sends one with probability $2/(e^{\varepsilon_0} + 1)$, and the count has standard deviation 18.4, against 960 for local randomized response at $\varepsilon_0 = 1$.

The factor before the bracket of Equation 26.20 is $\tanh(\varepsilon_0/2)$. For large n the $1/n$ term is negligible and $\ln(1+u) \approx u$, so the bound is about $8\tanh(\varepsilon_0/2)\sqrt{e^{\varepsilon_0}\ln(4/\delta)/n}$ (Figure 26.8). So a target ε allows e^{ε_0} of order $\varepsilon^2 n/\ln(1/\delta)$, and binary randomized response at that level has error about $1/\sqrt{ne^{\varepsilon_0}} = \sqrt{\ln(1/\delta)}/(\varepsilon n)$ on a fraction. Times n, that is the shuffle row of Table 26.2. The constants are loose (Table 26.4). The numeric column computes the paper's clone-count comparison exactly, and the exact column is shuffled binary randomized response, below which no general bound can go. Reason with the closed form, but deploy with a numerical accountant.

Table 26.4.: Central ε after shuffling $n = 10^4$ reports at $\delta = 10^{-6}$; the condition requires $\varepsilon_0 \leq 3.763$.

ε_0	Closed form, Equation 26.20	Numeric (Theorem 3.2)	Exact, shuffled binary randomized response
0.5	0.0939	0.0204	0.0162
1	0.2140	0.0530	0.0357
2	0.5009	0.1550	0.0870
3	0.8241	0.3207	0.1692

The amplification lasts only while the shuffler does its job, because a shuffler that colludes with the analyst leaves only ε_0. Fake senders hide nobody, so only honest senders count in n.

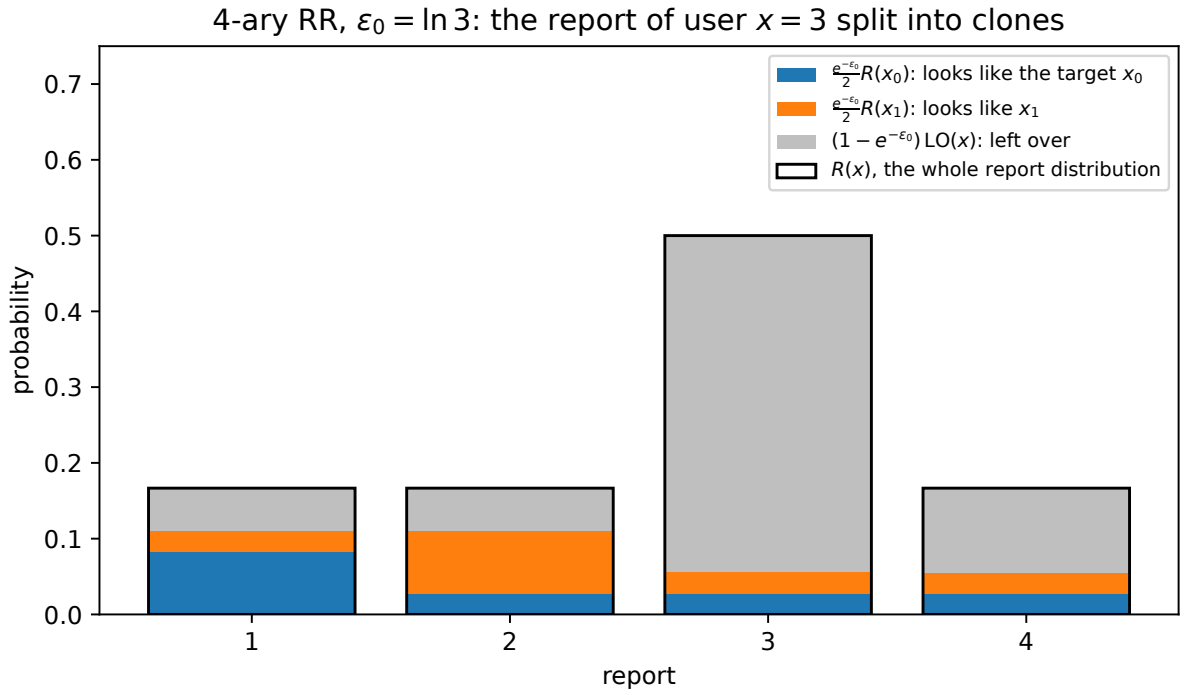


Figure 26.7.: The report distribution of value 3 under 4-ary randomized response at $\varepsilon_0 = \ln 3$, split into clones of value 1 (blue) and value 2 (orange) and a leftover (grey).

26.8. Distributed noise

The shuffle model still adds noise to every report. In the **distributed model** each client adds a small share of the central noise, and a **secure aggregation** protocol reveals only the sum. The question is what that costs, and what dishonest clients do to it.

Secure aggregation is built from the additive secret sharing of Cryptography basics, with a practical version by Bonawitz et al. (2017). The one property used is **simulation**: everything the server sees could be generated from the sum alone. The noise works because independent Gaussian shares add to a Gaussian whose variance is the sum of theirs.

Proposition 26.11 (Distributed Gaussian noise)**.** *Let n clients hold vectors x_i whose sum $f(D)$ has ℓ_2 sensitivity $\Delta_2 f$, and let each send $x_i + Z_i$ through secure aggregation, with independent $Z_i \sim \mathcal{N}(0, \frac{\sigma^2}{n} I)$. The server's view is (ε, δ)-DP whenever the Gaussian mechanism with noise σ on f is. If an adversary knows the noise of t clients, the guarantee is that of noise $\sigma\sqrt{(n-t)/n}$.*

Proof.

1. *The shares add up.* Independent normals add their variances, so $\sum_i Z_i \sim \mathcal{N}(0, \sigma^2 I)$.
2. *Only the sum is revealed.* By simulation the view is a function of the sum and independent randomness, so post-processing applies.
3. *Colluders.* Removing what they know leaves $n - t$ shares of variance σ^2/n, whose privacy is the exact curve of Theorem 20.3.

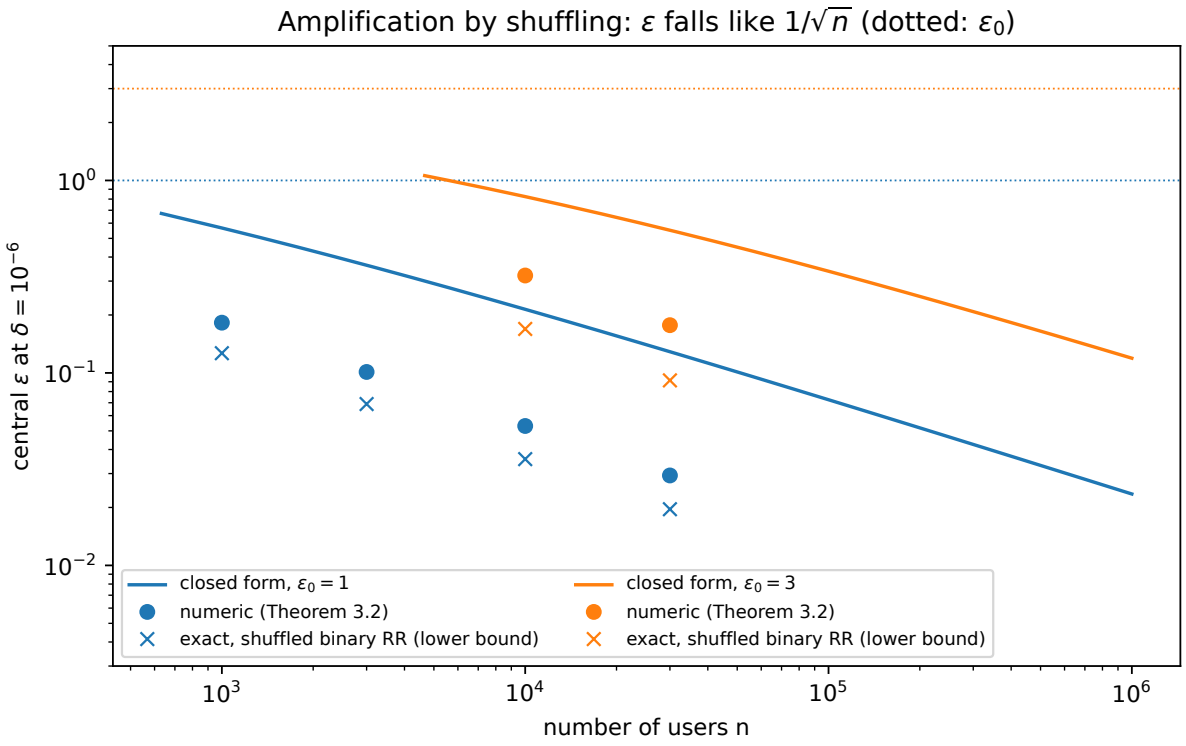


Figure 26.8.: Central ε at $\delta = 10^{-6}$ after shuffling, against n: closed form (lines), numeric bound (dots), exact binary randomized response (crosses). Dotted lines mark ε_0.

□

Secure aggregation adds integers modulo a large number, because only on a finite set can shares be uniformly random. **Skellam noise** (Agarwal, Kairouz & Liu 2021) fits. $\mathrm{Sk}(\mu)$ is the difference of two independent Poisson(μ) variables, with variance 2μ. Sums of independent Poisson variables are Poisson, because their moment generating functions multiply (Distributions), so n shares $\mathrm{Sk}(\mu/n)$ add to exactly $\mathrm{Sk}(\mu)$. Its privacy is close to a Gaussian's of the same variance, which this chapter uses as the approximation. The distributed discrete Gaussian (Kairouz, Liu & Steinke 2021) is the alternative.

Distributed noise gives central Gaussian accuracy without a curator, as long as enough clients are honest (Figure 26.9). If at most a fraction ρ may collude or drop out, each client adds variance $\sigma^2/(n(1-\rho))$, so the honest ones alone supply σ^2. Malicious inputs are another problem, since secure aggregation does not check them.

Example 26.8. A server aggregates the app's histogram through 100 clients at $(\varepsilon, \delta) = (1, 10^{-6})$. A user who changes value moves one count down and another up, so $\Delta_2 f = \sqrt{2}$. The analytic calibration of Corollary 20.1 gives total noise $\sigma = 5.975$, so each client adds standard deviation $5.975/\sqrt{100} = 0.597$. With 80%, 50% or 10% of the clients honest, ε rises to 1.128, 1.455 or 3.525. The script also checks that 100 Skellam shares of total variance 25 sum to mean 0.004 and variance 25.15, and that sharing $(3, -2)$, $(5, 7)$ and $(0, 1)$ modulo 2^{20} reveals $(8, 6)$.

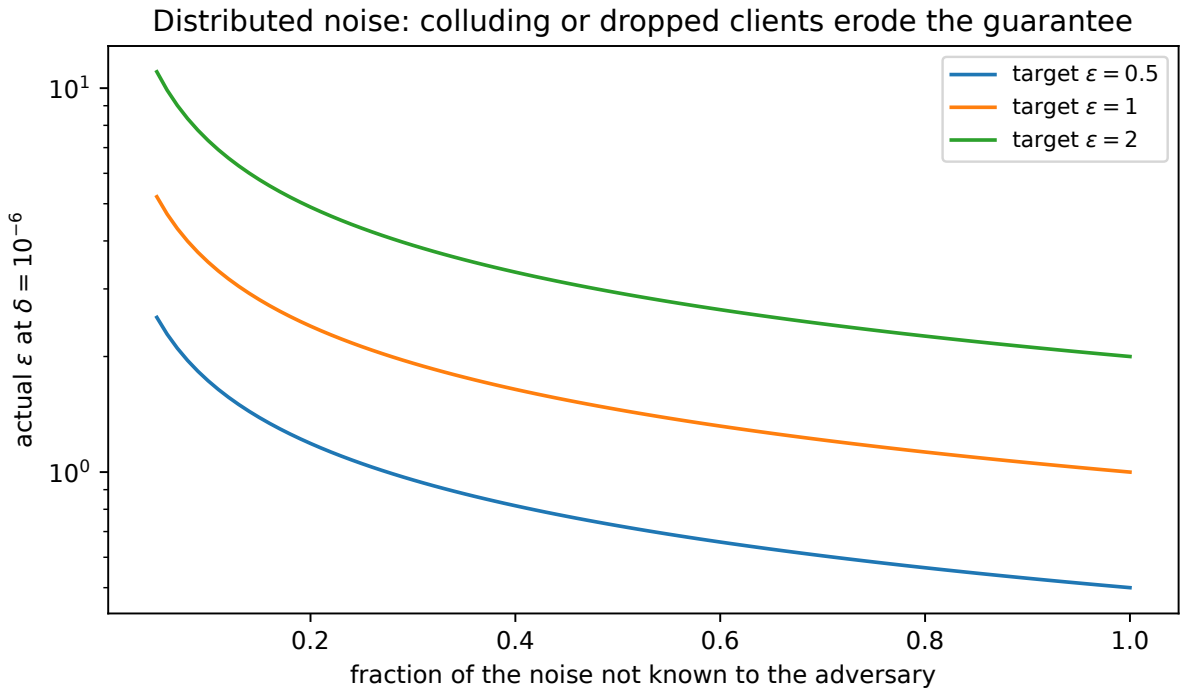


Figure 26.9.: The actual ε at $\delta = 10^{-6}$ against the fraction of the noise the adversary does not know.

26.9. Code

The script code/trust_models.py computes every number in this chapter and draws its figures. Its central comparison estimates an 8-value histogram at $(1, 10^{-6})$ in all four models, exactly and in 200 simulated runs (Figure 26.10). From the repository root:

```
uv run python code/part03-dp-foundations/trust_models.py
```

Table 26.5.: Output of the chapter's script (seed 0). The matched rows are the exact expected squared ℓ_2 errors; the simulated ones agree within Monte Carlo error.

Experiment	Result
Randomized response, $\varepsilon = \ln 3$, $n = 10^4$	sd 0.00866 (simulated mean 0.3000, sd 0.00861); count sd local 86.6, central 1.287, ratio 67.3
Fraction sd at $\varepsilon = 1$	$n = 10^3$: local 3.03×10^{-2}, central 1.41×10^{-3}, ratio 21, lower bound 1.04×10^{-2}; ratio 215 at 10^5, 2146 at 10^7
One report, Bern(0.5) against Bern(0.6)	KL 0.00429; bound 0.02952
Frequency oracles, $\varepsilon_0 = 1$	Table 26.3; both unary encodings have $\varepsilon_0 = 1.0000$
Local hashing, best w	$n \cdot$ Var 3.6917 ($w = 4$) and 0.0760 ($w = 56$); optimized unary 3.6827 and 0.0760
Heavy hitters, $\varepsilon_0 = 4$	50 found, all real; rarest 0.0033; threshold 0.0026; 22,418 false alarms if every string were tested
One-bit mean, $\varepsilon_0 = 1$	sd 1.420 (bound 1.558, simulated 1.415); Laplace 2.036; equal at $\varepsilon_0 = 2.324$
Repeated collection	Ratio 7.18×10^{23}; $\varepsilon_\infty = 2.043$, $\varepsilon_1 = 0.5343$; α-point changes 0.0119 (formula 0.0118)
Clones and shuffling	Leftover 0.6667; Table 26.4; largest ε_0 for $\varepsilon = 1$: 1.460 (10^3, the cap), 3.504, 5.701, 7.992
Distributed, 100 clients	$\sigma = 5.975$, 0.597 per client; ε at 80%, 50%, 10% honest: 1.128, 1.455, 3.525
Matched, $n = 10^3$	Central 6.40×10^{-5}; distributed 2.86×10^{-4}; shuffle 9.35×10^{-3}; local 2.71×10^{-2}
Matched, $n = 10^4$	Central 6.40×10^{-7}; distributed 2.86×10^{-6}; shuffle 4.88×10^{-5}; local 2.71×10^{-3}
Matched, $n = 10^5$	Central 6.40×10^{-9}; distributed 2.86×10^{-8}; shuffle 4.76×10^{-7}; local 2.71×10^{-4}

tests/test_trust_models.py checks that the output still matches.

26.10. Summary

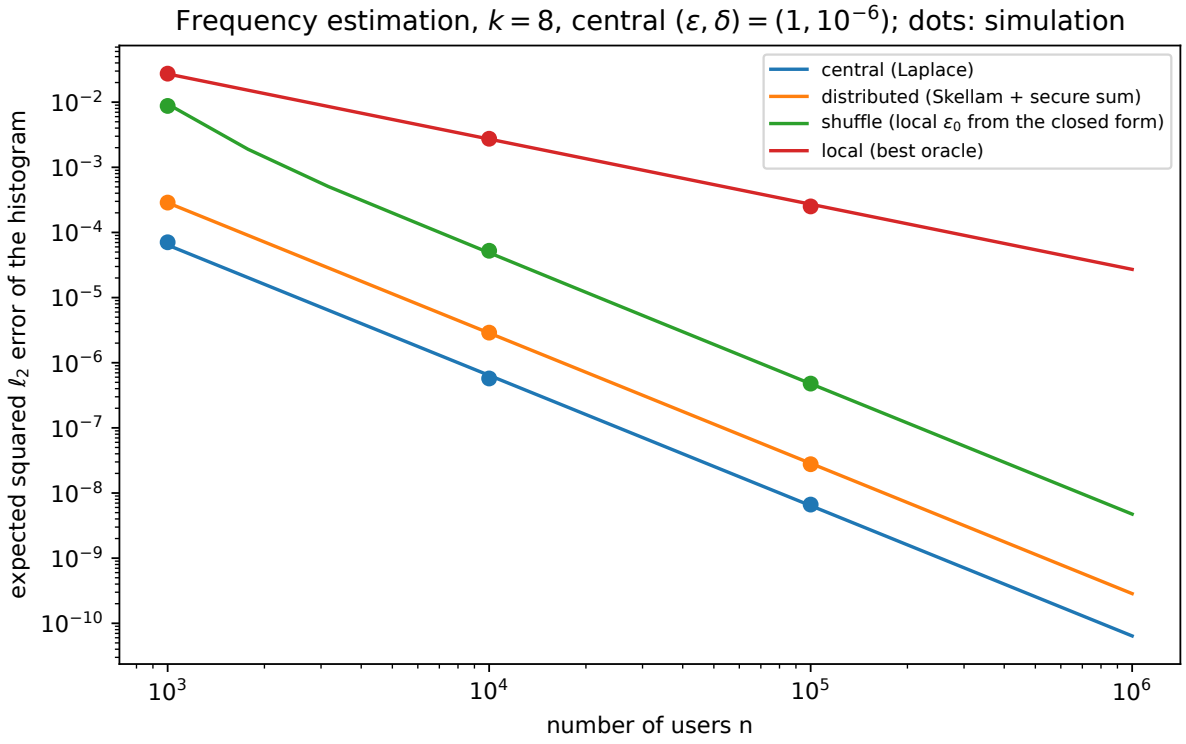


Figure 26.10.: Expected squared ℓ_2 error of the 8-value histogram at $(1, 10^{-6})$ in the four models: exact (lines) and simulated (dots).

Table 26.6.: The results of this chapter, and where each is used.

Result	Statement	Use
Proposition 26.1	Local randomizers make the transcript ε_0-DP against everyone	State the trust model next to (ε, δ)
Lemma 26.1	$(Y - c_0)/c_1$ is unbiased, variance $\mathrm{Var}(Y)/c_1^2$	Every local estimator
Proposition 26.2	$\mathrm{sd} = e^{\varepsilon_0/2}/((e^{\varepsilon_0} - 1)\sqrt{n})$	Counting one bit
Theorem 26.1	Error $\geq (1 - 2\beta)/((e^{\varepsilon_0} - 1)\sqrt{2n})$	Local DP costs about $\sqrt{n}$; use it for frequent items of large populations
Proposition 26.4	Rare variance grows with k for k-ary, is flat for unary	k-ary when $k < 3e^{\varepsilon_0} + 2$, else optimized unary
Proposition 26.5, Proposition 26.6	Best at $w = e^{\varepsilon_0} + 1$; prefixes grown level by level	Huge or unknown domains
Proposition 26.7	One bit per person, sd at most Equation 26.17	Numeric telemetry; fix m in advance
Proposition 26.9, Proposition 26.10	One permanent response per value; α-point rounding for drifting values	Memoize daily collection; fresh noise is averaged away (Proposition 26.8)
Theorem 26.2	Equation 26.20, about $8 \tanh(\varepsilon_0/2)\sqrt{e^{\varepsilon_0} \ln(4/\delta)/n}$	ε_0 up to about $\ln(\varepsilon^2 n / \ln(1/\delta))$; deploy with a numerical bound
Proposition 26.11	Shares $\mathcal{N}(0, \sigma^2/n)$ sum to $\mathcal{N}(0, \sigma^2)$	Over-provision for collusion and dropouts

26.11. References

- Warner, "Randomized Response: A Survey Technique for Eliminating Evasive Answer Bias" (*Journal of the American Statistical Association* 1965): the original local randomizer.
- Kasiviswanathan, Lee, Nissim, Raskhodnikova & Smith, "What Can We Learn Privately?" (FOCS 2008): defines the local model and relates it to statistical-query learning.
- Duchi, Jordan & Wainwright, "Minimax Optimal Procedures for Locally Private Estimation" (*Journal of the American Statistical Association* 2018; conference version 2013): the KL contraction behind the local lower bound, and optimal local estimators.
- Chan, Shi & Song, "Optimal Lower Bound for Differentially Private Multi-Party Aggregation" (2012): the $\Omega(\sqrt{n})$ lower bound for counting without a trusted party.
- Erlingsson, Pihur & Korolova, "RAPPOR: Randomized Aggregatable Privacy-Preserving Ordinal Response" (CCS 2014): the first large deployment, with memoization.
- Wang, Blocki, Li & Jha, "Locally Differentially Private Protocols for Frequency Estimation" (USENIX Security 2017): the frequency-oracle framework, optimized unary encoding and local hashing.

- Bassily & Smith, "Local, Private, Efficient Protocols for Succinct Histograms" (STOC 2015): heavy hitters with short reports, and the matching lower bound on the error.
- Bassily, Nissim, Stemmer & Thakurta, "Practical Locally Private Heavy Hitters" (NeurIPS 2017), and Bun, Nelson & Stemmer, "Heavy Hitters and the Structure of Local Privacy" (PODS 2018; ACM Transactions on Algorithms 2019): heavy-hitter protocols built by growing prefixes or hashing, and the conversion of approximate local protocols to pure ones.
- Acharya, Sun & Zhang, "Hadamard Response: Estimating Distributions Privately, Efficiently, and with Little Communication" (AISTATS 2019): frequency estimation with the Hadamard matrix.
- Apple Differential Privacy Team, "Learning with Privacy at Scale" (Apple Machine Learning Journal, 2017): the count-mean sketch, its Hadamard variant and the sequence fragment puzzle, as deployed in iOS and macOS.
- Ding, Kulkarni & Yekhanin, "Collecting Telemetry Data Privately" (NeurIPS 2017): the one-bit mean, α-point rounding with memoization, output perturbation, and Microsoft's deployment in Windows 10.
- Bittau et al., "Prochlo: Strong Privacy for Analytics in the Crowd" (2017): the encode–shuffle–analyze architecture.
- Cheu, Smith, Ullman, Zeber & Zhilyaev, "Distributed Differential Privacy via Shuffling" (EUROCRYPT 2019): the shuffle model as a model of computation, and its separations from local and central.
- Erlingsson, Feldman, Mironov, Raghunathan, Talwar & Thakurta, "Amplification by Shuffling: From Local to Central Differential Privacy via Anonymity" (SODA 2019): the first amplification theorem.
- Balle, Bell, Gascón & Nissim, "The Privacy Blanket of the Shuffle Model" (CRYPTO 2019): a different decomposition of the randomizer, and tight bounds for specific protocols.
- Feldman, McMillan & Talwar, "Hiding Among the Clones: A Simple and Nearly Optimal Analysis of Privacy Amplification by Shuffling" (FOCS 2021): the closed form used here, the clone reduction and its numerical version.
- Bonawitz et al., "Practical Secure Aggregation for Privacy-Preserving Machine Learning" (CCS 2017): the secure aggregation protocol used in federated learning.
- Kairouz, Liu & Steinke, "The Distributed Discrete Gaussian Mechanism for Federated Learning with Secure Aggregation" (ICML 2021) and Agarwal, Kairouz & Liu, "The Skellam Mechanism for Differentially Private Federated Learning" (NeurIPS 2021): integer noise that survives secure aggregation.

27. Lower bounds: what privacy costs

The earlier chapters of this part built mechanisms and measured their error. The Laplace mechanism releases a histogram over N bins with error about $\ln N/\varepsilon$ per bin, and the Gaussian mechanism releases d attribute means with error about $\sqrt{d\ln(1/\delta)}/(n\varepsilon)$ each. These **upper bounds** leave a question open: is that error the price of privacy, or of a clumsy mechanism? A **lower bound** answers it, because it says that *no* mechanism satisfying differential privacy (DP) can be more accurate.

One idea proves every lower bound here: accuracy is an attack. An accurate output tells an attacker which of many datasets was used, whether a person is in it, or what a secret bit was. DP limits every such attack, because no test can tell the outputs on two neighbouring datasets apart much better than chance. So if the attack built from an accurate mechanism would beat that limit, the mechanism cannot be both accurate and private, and solving the inequality for the error or for the number of records n gives the bound.

The running example is a survey in which each of n people answers d yes/no questions, and the release is the vector of the d attribute means, under replace neighbours. One question with $N = 10^6$ possible answers, a ZIP code, gives a histogram under add/remove neighbours. The examples use $\varepsilon = 1$, a failure probability $\beta = 0.05$ and $\delta = 10^{-6}$. Packing shows that under pure DP the histogram's error grows like $\ln N/\varepsilon$. Fingerprinting shows that under approximate DP the d means need n growing like $\sqrt{d}$, through a membership inference attack that you will run. The Dinur–Nissim attack of Reconstruction attacks shows that about n counting queries need error of order $\sqrt{n}$. Table 27.1 lists the symbols.

Table 27.1.: The symbols of this chapter.

Symbol	Meaning
M, $M(D)$	the mechanism and its random output on the dataset D
$D \sim D'$	neighbours: one record added or removed (histograms) or replaced (elsewhere)
ε, δ	the privacy parameters; pure DP is $\delta = 0$
α, β	the error and failure probability of an accuracy guarantee
n, d, N	the numbers of records, of attributes, and of histogram bins
m, k	the number of candidates in a packing, and their distance in records from the reference
p_j, x_{ij}, $\bar{x}$	the population mean of attribute j, person i's answer (± 1), and the sample means

27.1. Packing: pure DP pays the logarithm of the number of candidates

Suppose the output of a mechanism must point to one of m candidates, such as the most common ZIP code. For each candidate i you can build a dataset D_i on which an accurate mechanism lands in a region G_i of outputs almost always. Run the mechanism on one reference dataset D_0 instead, and on average each region gets at most a share $1/m$ of its probability. But DP says the output on D_0 cannot look very different from the output on a nearby D_i. For large m the two clash, and the clash is **packing**, the private version of Fano's method in Information theory.

Every bound in this chapter starts from an accuracy guarantee.

Definition 27.1 ((α, β)-accuracy)**.** A mechanism M is (α, β)**-accurate** for a quantity if, on *every* dataset, its output is within the **error** α of that quantity with probability at least $1 - \beta$, where β is the **failure probability**. These are not the error rates of a test, which other chapters write α and β.

The packing lemma compares D_0 with datasets k records away, so it uses group privacy, proved in Post-processing, group privacy and composition. Each of the k steps multiplies the bound by e^{ε} and adds a slack δ, which later steps multiply again. If M is (ε, δ)-DP and D, D'' differ in k records, then for every set of outputs S

$$\Pr[M(D) \in S] \le e^{k\varepsilon} \Pr[M(D'') \in S] + \delta_k, \qquad \delta_k = \delta \sum_{j=0}^{k-1} e^{j\varepsilon} \le k\, e^{(k-1)\varepsilon} \delta. \tag{27.1}$$

The slack grows geometrically: at $\varepsilon = 1$ and $\delta = 10^{-6}$, $\delta_5 = 8.6 \times 10^{-5}$ is small, while $\delta_{20} = 282$ bounds a probability by more than 1. So the lemma is stated for pure DP, where it says that m demands of size $(1 - \beta)e^{-k\varepsilon}$ must fit into a total of 1.

Lemma 27.1 (Packing lemma)**.** *Let $D_0, D_1, \dots, D_m$ be datasets with D_0 within k records of every D_i, and let $G_1, \dots, G_m$ be disjoint sets of outputs. If M is ε-DP and $\Pr[M(D_i) \in G_i] \ge 1 - \beta$ for every i, then*

$$m(1 - \beta)e^{-k\varepsilon} \le 1, \qquad \textit{that is,} \qquad k \ge \frac{\ln\left(m(1 - \beta)\right)}{\varepsilon}. \tag{27.2}$$

Proof.

1. *Each region gets a share of $M(D_0)$.* Accuracy says that G_i is likely under D_i, and group privacy Equation 27.1 with $\delta = 0$ says that it cannot be much less likely under D_0:

$$1 - \beta \le \Pr[M(D_i) \in G_i] \le e^{k\varepsilon} \Pr[M(D_0) \in G_i]. \tag{27.3}$$

   So $M(D_0)$ puts probability at least $(1 - \beta)e^{-k\varepsilon}$ on each G_i.
2. *The shares fit into one distribution.* The regions are disjoint, so no output is counted twice and the m shares add up to at most 1. Overlapping regions would let one output serve several candidates.
3. *Combine.* Adding the m bounds of step 1 and using step 2 gives the first form, and taking logarithms gives the second. Figure 27.1 draws this step.

□

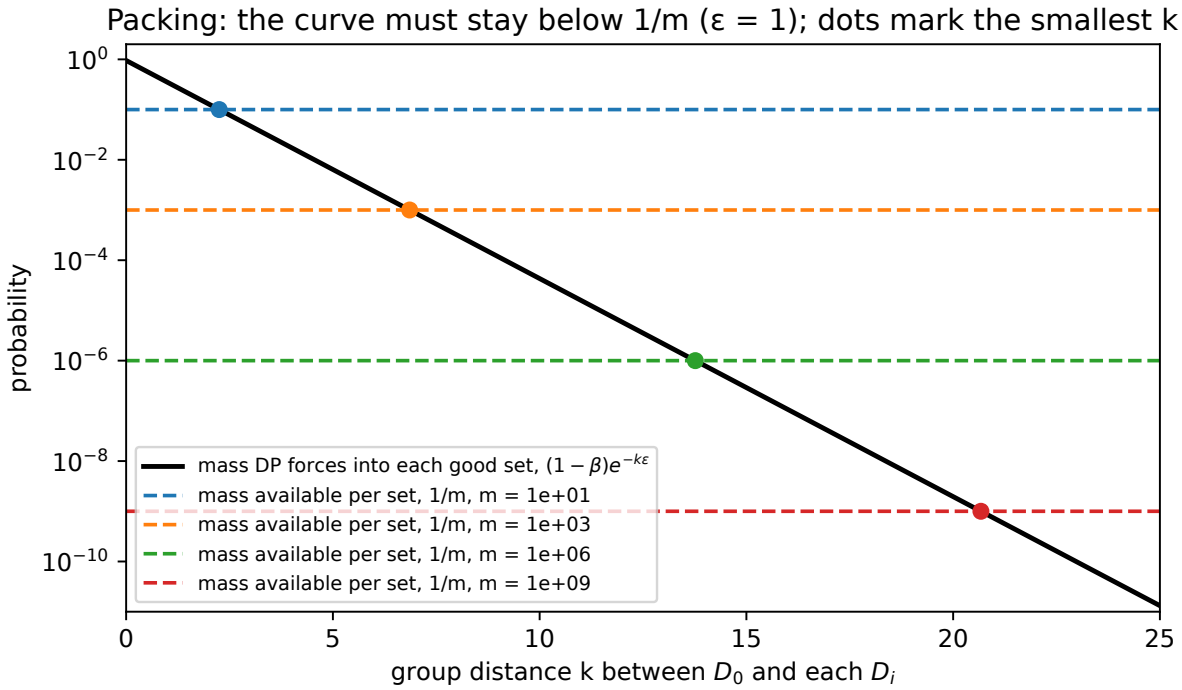


Figure 27.1.: The packing argument at $\varepsilon = 1$, $\beta = 0.05$. Black: the share $(1-\beta)e^{-k\varepsilon}$ that $M(D_0)$ must put on each good region. Dashed: the average share available with m regions. Accuracy at group distance k is impossible left of each dot.

Two consequences follow by choosing the datasets. The **mode** is the most frequent element of the data. A **histogram** is the vector of counts of N elements, and its ℓ_∞ error is its largest error over the bins. A histogram accurate to within α finds a spike of height above 2α, so it inherits the mode's bound, halved.

Theorem 27.1 (Mode and histogram lower bounds)**.** *Let M be ε-DP under add/remove neighbours, over a universe of N elements. If M returns the most frequent element with probability at least $1-\beta$ whenever that element appears k times and every other element appears 0 times, then*

$$k \geq \frac{\ln\left(N(1-\beta)\right)}{\varepsilon}. \tag{27.4}$$

If M releases a histogram over the N bins that is (α, β)-accurate in ℓ_∞ error, then

$$\alpha \geq \frac{1}{2}\left(\frac{\ln\left(N(1-\beta)\right)}{\varepsilon} - 1\right). \tag{27.5}$$

Proof. The plan is to build the candidates once, apply the lemma to the mode, and then show that an accurate histogram separates the same candidates.

1. *The candidates.* Take $D_0 = \varnothing$, the empty dataset, and for each element x let D_x be k copies of x. Each D_x is k additions from D_0, and there are $m = N$ of them.
2. *The mode.* On D_x a correct answer lands in $G_x = \{x\}$, and these regions are disjoint, so Lemma 27.1 gives Equation 27.4.
3. *Disjoint regions for the histogram.* The true histogram of D_x is $k\,e_x$, with k in bin x and 0 elsewhere, and a correct answer lies in the ball G_x of ℓ_∞ radius α around it. Two centres differ by k. A point in both balls would put the centres within 2α of each other, by the triangle inequality, so the balls are disjoint when $k > 2\alpha$.
4. *Pick the smallest such k.* The lemma's bound is strongest for the closest datasets, so take $k = \lfloor 2\alpha \rfloor + 1 \leq 2\alpha + 1$.
5. *Apply the lemma.* With step 4 it gives

$$2\alpha + 1 \geq \frac{\ln(N(1-\beta))}{\varepsilon}, \tag{27.6}$$

and subtracting 1 and halving gives Equation 27.5.

□

Both bounds are matched, up to constants, by mechanisms you already have. The Laplace mechanism adds independent $\mathrm{Lap}(1/\varepsilon)$ noise to every count, since one record moves one count by 1. Report-noisy-max adds the same noise and returns the bin with the largest noisy count (Selection).

Proposition 27.1 (Laplace histograms and report-noisy-max match packing)**.** *Under ε-DP with add/remove neighbours, the Laplace histogram over N bins has ℓ_∞ error, at failure probability β, of exactly $-\ln\left(1-(1-\beta)^{1/N}\right)/\varepsilon$, which is at most $\ln(N/\beta)/\varepsilon$. When one element leads by k records and every other count is 0, report-noisy-max returns the mode except with probability at most $(N-1)e^{-k\varepsilon}(2+k\varepsilon)/4$.*

Proof.

1. *One Laplace coordinate.* A $\mathrm{Lap}(b)$ variable, with density $\frac{1}{2b}e^{-|z|/b}$, exceeds α in absolute value with probability $e^{-\alpha/b}$, here $e^{-\alpha\varepsilon}$.
2. *All N bins at once.* The noises are independent, so all stay within α with probability $(1-e^{-\alpha\varepsilon})^N$, and setting this to $1-\beta$ gives the exact error. A union bound over the bins puts the failure probability at most $Ne^{-\alpha\varepsilon}$, and setting that to β gives $\ln(N/\beta)/\varepsilon$.
3. *Report-noisy-max.* It fails only if some other bin's noise exceeds bin x's noise by k or more. The difference of two independent $\mathrm{Lap}(1/\varepsilon)$ variables has density $\frac{\varepsilon}{4}(1+\varepsilon|z|)e^{-\varepsilon|z|}$, a convolution that Section E.6 works out. Substituting $u=\varepsilon z$, its tail beyond k is

$$\frac{1}{4}\int_{k\varepsilon}^{\infty}(1+u)e^{-u}\,\mathrm{d}u=\frac{e^{-k\varepsilon}(2+k\varepsilon)}{4}. \tag{27.7}$$

   A union bound over the $N-1$ other bins gives the claim.

□

Both the lower bound and the Laplace error grow like $\ln N/\varepsilon$ (Figure 27.2), with slopes $1/(2\varepsilon)$ and $1/\varepsilon$, so Laplace is optimal up to a factor of about 2. The lemma loses that factor in step 2, which counts only the mass inside the good regions, while the Laplace output on $\varnothing$ falls almost entirely outside every ball G_x. Under replace neighbours the Laplace error doubles while the lower bound barely moves.

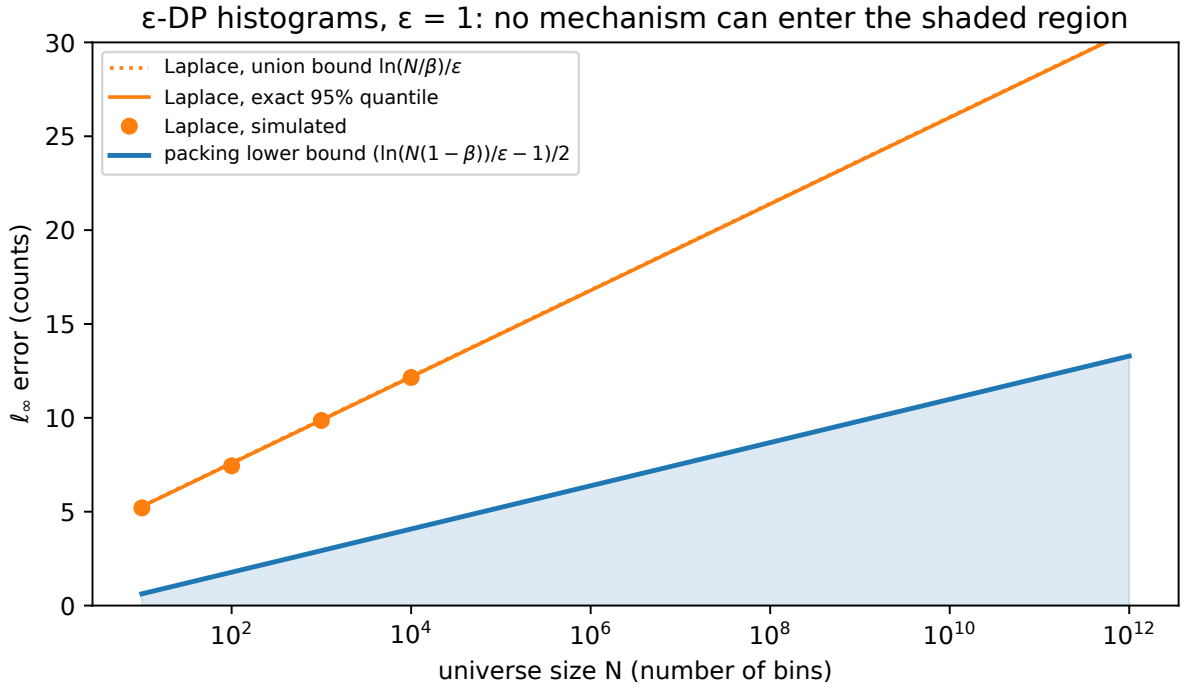


Figure 27.2.: ℓ_∞ error of ε-DP histograms over N bins at $\varepsilon=1$, failure probability 0.05. Orange: the Laplace mechanism, exact (line), union bound (dotted) and simulated (dots). Blue: the packing lower bound, with no ε-DP mechanism in the shaded region.

The lemma needs $\delta = 0$, because with slack the share in step 1 turns negative once $e^{-k\varepsilon}$ falls below about $k\delta$. What survives is a capped bound.

Proposition 27.2 (Packing under approximate DP)**.** *Let $D_0, \dots, D_m$ and the disjoint sets $G_1, \dots, G_m$ be as in Lemma 27.1, with $\Pr[M(D_i) \in G_i] \geq 1 - \beta$ for every i, but let M be (ε, δ)-DP. Then at least one of*

$$e^{-k\varepsilon}(1-\beta) \leq 2k\delta \qquad \text{or} \qquad m\, e^{-k\varepsilon}(1-\beta) \leq 2 \tag{27.8}$$

holds. In words, the group distance k must be at least about $\min(\ln m, \ln(1/\delta))/\varepsilon$.

Proof.

1. *The share with slack.* Group privacy Equation 27.1 applied to D_i, D_0 and G_i gives

$$\Pr[M(D_0) \in G_i] \geq e^{-k\varepsilon}(1-\beta) - e^{-k\varepsilon}\delta_k. \tag{27.9}$$

   The subtracted part is a sum of k terms $\delta e^{(j-k)\varepsilon}$, each at most $\delta e^{-\varepsilon} \leq \delta$, so the share is at least $e^{-k\varepsilon}(1-\beta) - k\delta$.
2. *Two cases.* If the first condition fails, $k\delta$ is less than half of $e^{-k\varepsilon}(1-\beta)$, so each share is more than $\frac{1}{2}e^{-k\varepsilon}(1-\beta)$.
3. *Fit into 1.* The m shares add up to at most 1, as in the lemma, which is the second condition.

□

The cap is real, because a mechanism reaches it. The stability-based histogram (Vadhan 2017) noises only the non-empty bins and releases those above a threshold of order $\ln(1/\delta)/\varepsilon$, so its error does not grow with N (Section J.3). On the mode's packing datasets, the stability-based release reports the mode once it leads by about $\ln(1/\delta)/\varepsilon$ replacements, however many values there are.

Example 27.1. Table 27.2 compares the bounds at $\varepsilon = 1$ and $\beta = 0.05$. For $N = 10^6$ locations, the most common one must lead by at least $\ln(10^6 \times 0.95)/1 = 13.76$ records before an ε-DP release can report it reliably, and report-noisy-max needs 18.44. Every ε-DP histogram has, at the 95% level, an error of at least $\frac{1}{2}(13.76 - 1) = 6.38$ in some bin, and Laplace has 16.79. Simulated at $N = 1000$, the Laplace error is 9.84 against the exact 9.88, and report-noisy-max at the lead 11.09 fails 2.28% of the time against the union bound's 5%.

Table 27.2.: Packing lower bounds against the mechanisms that achieve them, $\varepsilon = 1$, $\beta = 0.05$, add/remove neighbours, in records.

N	Mode: lower bound on lead k	Report-noisy-max needs	Histogram: lower bound on error	Laplace error (exact)	Laplace error (union bound)
10^3	6.86	11.09	2.93	9.88	9.90
10^6	13.76	18.44	6.38	16.79	16.81
10^9	20.67	25.65	9.84	23.69	23.72

Each factor of 1000 in N adds $\ln 1000 = 6.9$ to every column that grows like $\ln N$, and half that to the histogram lower bound.

27.2. Sample complexity: how many records privacy costs

An error of 6 records is large in a dataset of 50 and negligible in one of a million, so a lower bound on error is a lower bound on the number of records. This section translates the packing bounds into records, where privacy can be compared with sampling error, a cost you pay anyway. It is also where pure and approximate DP part ways once the output has many dimensions.

For the histogram, an error of α in each frequency is an error of αn in each count.

Proposition 27.3 (Records for a private histogram)**.** *An ε-DP histogram over N bins whose ℓ_∞ error is at most αn with probability $1-\beta$, on every dataset of up to n records, needs*

$$n \geq \frac{\ln\left(N(1-\beta)\right) - \varepsilon}{2\alpha\varepsilon}. \tag{27.10}$$

Without privacy, estimating all N frequencies to within α with probability $1-\beta$ takes at most $n = \ln(2N/\beta)/(2\alpha^2)$ samples.

Proof.

1. *The private bound.* The packing datasets have $k \leq n$ records, so the guarantee covers them, and Equation 27.5 with the error αn gives

$$\alpha n \geq \frac{1}{2}\left(\frac{\ln(N(1-\beta))}{\varepsilon} - 1\right). \tag{27.11}$$

   Multiplying by 2ε and dividing by $2\alpha\varepsilon$ gives Equation 27.10.
2. *The sampling bound.* By Hoeffding's inequality (Concentration inequalities), an average of n independent variables in $[0,1]$ is more than α from its mean with probability at most $2e^{-2n\alpha^2}$. Each frequency is such an average, so a union bound over the N bins gives a failure probability of at most $2Ne^{-2n\alpha^2}$. Setting this to β and solving for n gives the claim.

□

So privacy adds a term of order $\ln N/(\alpha\varepsilon)$ to a statistical cost of order $\ln N/\alpha^2$, which Fano's method shows is really needed. Their ratio is about α/ε, so privacy dominates only for coarse answers at strong privacy.

The survey's d means make the candidates multiply. Records are $x \in \{-1,+1\}^d$, a person's d yes/no answers, and the release is the vector of means $\bar{x} \in [-1,1]^d$. Datasets made of copies of one corner of the cube $\{\pm 1\}^d$ give 2^d candidates, so packing charges $\ln 2^d = d\ln 2$.

Theorem 27.2 (Records for d marginals under pure DP)**.** *Any ε-DP mechanism under replace neighbours that releases the d attribute means with ℓ_∞ error at most α with probability $1-\beta$ needs*

$$n \geq \frac{1}{\alpha}\left(\frac{\ln\left((2^d-1)(1-\beta)\right)}{\varepsilon} - 1\right) \approx \frac{d\ln 2}{\alpha\varepsilon}. \tag{27.12}$$

The approximation holds for large d.

Proof. This is packing at a fixed size n, so the candidates replace records rather than add them.

1. *Candidates.* Fix a corner $u_0 \in \{\pm 1\}^d$ and let D_0 be n copies of it. For each of the $m = 2^d - 1$ other corners u, let D_u be D_0 with k records replaced by u.
2. *Disjoint regions.* The mean of D_u is $u_0 + \frac{k}{n}(u - u_0)$. Two corners differ in some coordinate by 2, so their means differ there by $2k/n$. As in step 3 of the proof of Theorem 27.1, the balls of radius α around the means are disjoint when $k > \alpha n$.
3. *Apply the lemma* with $k = \lfloor \alpha n \rfloor + 1 \le \alpha n + 1$:

$$\alpha n + 1 \ge \frac{\ln\left((2^d - 1)(1 - \beta)\right)}{\varepsilon}. \tag{27.13}$$

   Subtracting 1 and dividing by α gives Equation 27.12.

□

Two mechanisms come close. By Theorem 20.3 of the Laplace and Gaussian mechanisms, Gaussian noise of standard deviation σ on a query with ℓ_2 sensitivity Δ_2 is (ε, δ)-DP exactly when Δ_2/σ is at most the root $\mu(\varepsilon, \delta)$ of Equation 20.17, which is 0.2367 at $\varepsilon = 1$ and $\delta = 10^{-6}$.

Proposition 27.4 (Laplace and Gaussian upper bounds for d marginals)**.** *Under replace neighbours, the Laplace mechanism releases the d means with ℓ_∞ error at most α with probability $1 - \beta$, at ε-DP, once*

$$n = \frac{2d \ln(d/\beta)}{\alpha\varepsilon}. \tag{27.14}$$

The Gaussian mechanism does so at (ε, δ)-DP once $n = 2\sqrt{d}\, z/(\mu(\varepsilon, \delta)\, \alpha)$, where z is the number with $\Phi(-z) = \beta/(2d)$ and Φ is the standard normal cumulative distribution function.

Proof.

1. *Sensitivities.* Replacing one record moves each of the d means by at most $2/n$, so the ℓ_1 sensitivity is $\Delta_1 = 2d/n$ and the ℓ_2 sensitivity is $\Delta_2 = 2\sqrt{d}/n$.
2. *Laplace.* The noise ξ_j in coordinate j is $\mathrm{Lap}(b)$ with $b = \Delta_1/\varepsilon = 2d/(n\varepsilon)$, and it exceeds α with probability $e^{-\alpha/b}$. A union bound over the coordinates gives

$$\Pr\left[\max_j |\xi_j| > \alpha\right] \le d e^{-\alpha/b}. \tag{27.15}$$

   Setting this to β gives $\alpha n\varepsilon/(2d) = \ln(d/\beta)$, and solving for n gives the claim.
3. *Gaussian.* The noise has standard deviation $\sigma = \Delta_2/\mu(\varepsilon, \delta)$. A normal variable exceeds z standard deviations in absolute value with probability $2\Phi(-z)$, so a union bound gives

$$\Pr\left[\max_j |\xi_j| > \sigma z\right] \le 2d\, \Phi(-z) = \beta. \tag{27.16}$$

   Setting $\sigma z = \alpha$, that is $2\sqrt{d}\, z/(n\mu) = \alpha$, and solving for n gives the claim.

□

Pure DP pays linearly in the dimension, and Laplace matches it up to the factor $2\ln(d/\beta)/\ln 2$. The Gaussian requirement grows only like $\sqrt{d \ln d}$, because its noise follows Δ_2 rather than Δ_1, so for large d it beats *every* pure-DP mechanism. That is no contradiction, because under approximate DP (Proposition 27.2) the corners force only $k \geq 13$ at $\delta = 10^{-6}$, against $\ln(2^{1000} \times 0.95) = 693.10$ for pure DP at $d = 1000$. Approximate DP needs the idea of Section 27.3.

Packing also prices a question every analyst meets: what does it cost not to know where the data lie? Write the unknown mean as c, because μ is the Gaussian ratio here.

Proposition 27.5 (A mean known to lie in a range)**.** *Let the records be drawn from a normal distribution with unknown mean c and variance 1, and let $|c| \leq R$ be all that is known. An ε-DP mechanism under replace neighbours that returns an estimate closer than α to c, with probability at least $1 - \beta$ whatever c is, needs*

$$n \geq \frac{\ln\left(\lfloor R/\alpha \rfloor (1-\beta)\right)}{\varepsilon}, \tag{27.17}$$

where $\lfloor R/\alpha \rfloor$ is R/α rounded down.

Proof. This is the packing lemma with random datasets.

1. *Candidates.* Place $m = \lfloor R/\alpha \rfloor$ means $c_1, \ldots, c_m$ in $[-R, R]$, spaced at least 2α apart, and let G_j be the open interval of radius α around c_j. These intervals are disjoint.
2. *Every dataset is close to a reference.* Fix a reference dataset D_0 of size n. Any dataset D of size n is at most n replacements away, so group privacy with $\delta = 0$ gives
$$\Pr[M(D) \in G_j] \leq e^{n\varepsilon} \Pr[M(D_0) \in G_j]. \tag{27.18}$$
3. *Average over the data.* Draw the records of D from the normal distribution with mean c_j. Averaged over D, the left side is at least $1 - \beta$ by accuracy, while the right side does not depend on D. So $\Pr[M(D_0) \in G_j] \geq (1-\beta)e^{-n\varepsilon}$.
4. *Combine* as in the lemma: the m shares fit into a total of 1, which is Equation 27.17.

□

At $\alpha = 0.1$, $\beta = 0.05$ and $\varepsilon = 1$, the bound is $n \geq 4.55$ for $R = 10$, 9.16 for $R = 10^3$ and 16.07 for $R = 10^6$. Clipping to $[-R, R]$ needs records linear in R, and Private mean estimation closes the gap by first locating the data with a histogram. In d dimensions the cost is about $d \ln(R/\alpha)/\varepsilon$.

Example 27.2. Table 27.3 gives the numbers for the survey at $\varepsilon = 1$ and $\beta = 0.05$. A histogram over $N = 10^6$ cells to within $\alpha = 0.1$ needs at least $(13.76 - 1)/(2 \times 0.1 \times 1) = 63.8$, so 64 records under ε-DP, and at most 875 for sampling error. At $\alpha = 0.01$ the numbers are 638 and 87,522, so privacy is the smaller cost in both cases.

Table 27.3.: Records needed to release d attribute means with ℓ_∞ error 0.1 (on the scale $[-1, 1]$) with probability 0.95, at $\varepsilon = 1$ and, for the Gaussian, $\delta = 10^{-6}$.

	Pure DP, packing (lower)	Pure DP, Laplace (upper)	Approximate DP, Gaussian (upper)
$d = 100$	683	15,202	2,941
$d = 1000$	6,921	198,070	10,836
$d = 10{,}000$	69,304	2,441,215	38,570
$d = 100{,}000$	693,137	29,017,315	134,299

The packing column grows tenfold with each tenfold increase in d, and the Gaussian column only 3.7-, 3.6- and 3.5-fold, close to $\sqrt{10}$. So from $d = 2731$ attributes on, the Gaussian mechanism needs fewer records than any pure-DP mechanism, which is one reason private machine learning uses approximate DP.

27.3. Fingerprinting: the √d lower bound for approximate DP

Packing fails under approximate DP because group privacy breaks down at large distances. Fingerprinting compares only neighbours, so δ never blows up. It asks whether an accurate release of one random dataset's d means agrees with a member's record more than with a stranger's. The **fingerprinting lemma** shows that accuracy forces agreement of order d in total, and DP caps each record's share, so n must be large. The idea comes from fingerprinting codes in cryptography, which Bun, Ullman & Vadhan (2014) brought to DP; this version follows Steinke & Ullman (2016).

Definition 27.2 (The random survey)**.** Draw attribute means $p_1, \dots, p_d$ independently and uniformly from $[-1, 1]$. Given p, draw n records independently, each coordinate $x_{ij} \in \{\pm 1\}$ independently with mean p_j, that is, $+1$ with probability $(1 + p_j)/2$ and -1 otherwise.

The means are random because, for a fixed p, a mechanism that ignores the data and outputs p would be accurate up to sampling error yet tell nothing about any record. A mechanism that must be accurate for every p has to learn p from the records, and so must correlate with them.

The lemma is about one attribute, so drop the index j and write $x = (x_1, \dots, x_n)$ for its column. It says that an estimate's correlation with the records' deviations $x_i - p$ cannot be small unless the estimate is inaccurate.

Lemma 27.2 (Fingerprinting lemma)**.** *For the column x of one attribute of Definition 27.2, with mean $\bar{x}$, and any estimate $f\colon \{\pm 1\}^n \to \mathbb{R}$ of p computed from it,*

$$\mathbb{E}\Big[f(x)\sum_{i=1}^{n}(x_i - p)\Big] = 2\,\mathbb{E}[p\,f(x)] \geq \frac{2}{3} - 2\,\mathbb{E}[|f(x) - \bar{x}|], \tag{27.19}$$

where the expectations are over p and the column.

Proof. The plan is to show that the correlation is the slope of the average estimate $g(p) = \mathbb{E}[f(x) \mid p]$, weighted by $1 - p^2$, and then to average that slope by integrating by parts. An accurate estimate rises from about -1 to about $+1$, so its slope cannot be small on average.

1. *How the data respond to p.* The column has probability $\prod_i \frac{1+x_i p}{2}$. Its log-derivative in p, the **score** of statistics, is $\sum_i \frac{x_i}{1+x_i p}$. Each term equals $\frac{x_i - p}{1-p^2}$, as you can check for $x_i = 1$ and for $x_i = -1$.
2. *Differentiate the average estimate.* Since $g(p) = \sum_x f(x) \Pr[x \mid p]$ and the derivative of $\Pr[x \mid p]$ is $\Pr[x \mid p]$ times its log-derivative, g' is the average of f times the score. Inserting step 1 and multiplying by $1 - p^2$ gives

$$(1 - p^2)\, g'(p) = \mathbb{E}\Big[f(x) \sum_i (x_i - p) \;\Big|\; p \Big]. \tag{27.20}$$

3. *Average over p and integrate by parts.* The density of p is $\frac{1}{2}$ on $[-1, 1]$. Integration by parts moves the derivative from g onto $1 - p^2$, whose derivative is $-2p$:

$$\frac{1}{2} \int_{-1}^{1} (1 - p^2) g'(p)\, \mathrm{d}p = \frac{1}{2} \Big[(1 - p^2) g(p) \Big]_{-1}^{1} + \int_{-1}^{1} p\, g(p)\, \mathrm{d}p. \tag{27.21}$$

   The boundary term vanishes because $1 - p^2 = 0$ at ± 1 and g, a polynomial, is finite there. The remaining integral is $2\,\mathbb{E}[p\, g(p)]$, which equals $2\,\mathbb{E}[p\, f(x)]$ because g is the average of f given p. This is the equality.
4. *Compare with the sample mean.* The sample mean has average p given p, so $\mathbb{E}[p\, \bar{x}] = \mathbb{E}[p^2] = \frac{1}{3}$. Any other estimate differs from it by $f - \bar{x}$, and $|p| \le 1$, so

$$\mathbb{E}[p\, f] = \tfrac{1}{3} + \mathbb{E}[p(f - \bar{x})] \ge \tfrac{1}{3} - \mathbb{E}[|f - \bar{x}|]. \tag{27.22}$$

   Multiplying by 2 gives the inequality.

□

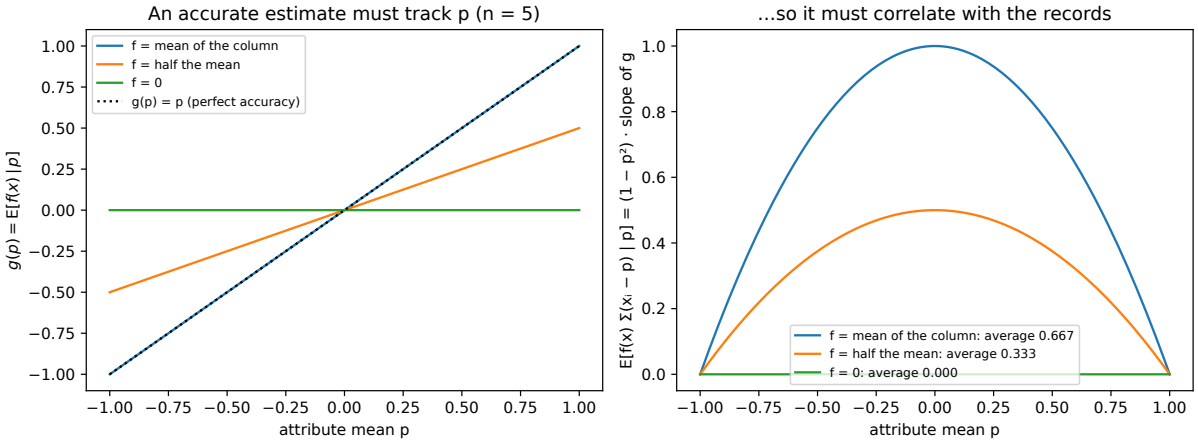


Figure 27.3.: The fingerprinting lemma for one attribute and $n = 5$ records. Left: the average estimate $g(p)$ for the sample mean, half the sample mean, and the constant 0. Right: each estimate's correlation with the records' deviations, $(1 - p^2)g'(p)$.

The lower bound below needs DP to bound the *average* of a score, while the definition bounds the probability of each event. The bridge is that an average is an integral of tail probabilities, and DP bounds each tail.

Lemma 27.3 (DP bounds the average of a bounded score)**.** *If M is (ε, δ)-DP, $D \sim D'$, and h is a function of the output with $|h| \le B$, then*

$$\mathbb{E}[h(M(D))] \le \mathbb{E}[h(M(D'))] + (e^{\varepsilon} - 1)\,\mathbb{E}[|h(M(D'))|] + 2\delta B. \tag{27.23}$$

Proof. The plan is to write the mean as an integral of tails, bound each tail by the definition, and integrate back.

1. *The mean as an integral of tails.* A variable Y with $0 \le Y \le B$ has $\mathbb{E}[Y] = \int_0^B \Pr[Y > t]\,\mathrm{d}t$, because each value y is counted once for every level t below it. For Y equal to 1, 2 or 3 with probability $\frac{1}{3}$ each, the integral is $1 + \frac{2}{3} + \frac{1}{3} = 2$. Splitting h into its positive part h_+ and negative part h_- gives

$$\mathbb{E}[h] = \int_0^B \Pr[h > t]\,\mathrm{d}t - \int_0^B \Pr[h < -t]\,\mathrm{d}t. \tag{27.24}$$

2. *Bound the positive part from above.* The definition applied to the event $\{h > t\}$ gives

$$\Pr[h(M(D)) > t] \le e^{\varepsilon} \Pr[h(M(D')) > t] + \delta. \tag{27.25}$$

3. *Bound the negative part from below,* since it is subtracted. The definition with D' on the left and the event $\{h < -t\}$, solved for the term on D, gives

$$\Pr[h(M(D)) < -t] \ge e^{-\varepsilon}(\,\Pr[h(M(D')) < -t] - \delta). \tag{27.26}$$

4. *Integrate over $t \in [0, B]$.* Write $h' = h(M(D'))$. By step 1 the integrated tails are means again, and each δ integrates to δB:

$$\mathbb{E}[h(M(D))] \le e^{\varepsilon}\,\mathbb{E}[h'_+] - e^{-\varepsilon}\,\mathbb{E}[h'_-] + (1 + e^{-\varepsilon})\delta B. \tag{27.27}$$

5. *Pull out the mean on D'.* Since $\mathbb{E}[h'] = \mathbb{E}[h'_+] - \mathbb{E}[h'_-]$, the first two terms equal

$$\mathbb{E}[h'] + (e^{\varepsilon} - 1)\mathbb{E}[h'_+] + (1 - e^{-\varepsilon})\mathbb{E}[h'_-]. \tag{27.28}$$

   The coefficient $1 - e^{-\varepsilon}$ is at most $e^{\varepsilon} - 1$, because their difference is $e^{\varepsilon} + e^{-\varepsilon} - 2 \ge 0$, so the middle terms are at most $(e^{\varepsilon} - 1)\mathbb{E}[|h'|]$. Finally $1 + e^{-\varepsilon} \le 2$.

□

So a score centred on D', with mean absolute value 1, has mean at most $e - 1 = 1.72$ on D at $\varepsilon = 1$, plus $2\delta B$. The lower bound plays the two lemmas against each other through an attack that scores each record x_i by its correlation with the release:

$$Z_i = \langle M(D),\ x_i - p \rangle = \sum_{j=1}^{d} M(D)_j\,(x_{ij} - p_j). \tag{27.29}$$

A member should score high, because the release follows the data, and a stranger should score around 0.

Theorem 27.3 (The fingerprinting lower bound)**.** *Let M be (ε, δ)-DP under replace neighbours, with outputs in $[-1,1]^d$, and let its average error on the random survey of Definition 27.2 be $\mathbb{E}[\|M(D) - \bar{x}\|_1] \le \alpha' d$, where α' is the average error per attribute. Then*

$$n \ge \frac{(2/3 - 2\alpha')\, d}{(e^\varepsilon - 1)\sqrt{d} + 4\delta d}. \tag{27.30}$$

Proof. The plan is that accuracy makes the members' scores total order d, while DP keeps each within a multiple of a stranger's spread $\sqrt{d}$, so the n members must carry the total.

1. *Accuracy makes the scores large on average.* Fix an attribute j and condition on the other columns, which leaves column j and p_j distributed as in the lemma. Apply Lemma 27.2 to f, the average of $M(D)_j$ over the mechanism's coins. That averaging leaves the correlation unchanged and cannot increase the error. Summing over j and swapping the sums over j and i gives
$$\sum_{i=1}^{n} \mathbb{E}[Z_i] \ge \frac{2}{3} d - 2\,\mathbb{E}[\|M(D) - \bar{x}\|_1] \ge \Big(\frac{2}{3} - 2\alpha'\Big) d. \tag{27.31}$$
2. *A stranger's score is small.* Let $D^{(i)}$ be D with record i replaced by a fresh one, and let $Z_i' = \langle M(D^{(i)}), x_i - p\rangle$ score x_i against a release that never saw it. Given p, the deviations $x_{ij} - p_j$ are independent of that release, with mean 0 and variance at most 1. So $\mathbb{E}[Z_i'] = 0$, the cross terms of $Z_i'^2$ average to 0, and $\mathbb{E}[Z_i'^2] \le d$ because each output coordinate lies in $[-1,1]$. By the Cauchy–Schwarz inequality, $\mathbb{E}[|Z_i'|] \le \sqrt{d}$.
3. *DP keeps a member's score close to a stranger's.* D and $D^{(i)}$ differ only in record i, so they are neighbours under replace. The score $h(y) = \langle y, x_i - p\rangle$ has $|h| \le 2d$, so Lemma 27.3 with $B = 2d$, averaged over the data and combined with step 2, gives
$$\mathbb{E}[Z_i] \le (e^\varepsilon - 1)\sqrt{d} + 4\delta d. \tag{27.32}$$
4. *Combine.* Summing step 3 over the n records bounds the total from above, and step 1 bounds it from below:
$$\Big(\frac{2}{3} - 2\alpha'\Big) d \le n((e^\varepsilon - 1)\sqrt{d} + 4\delta d), \tag{27.33}$$
which rearranges to Equation 27.30.

□

An (α, β)-accurate mechanism in ℓ_∞, clipped to $[-1,1]^d$, has $\alpha' \le \alpha + 2\beta$, since a coordinate off by more than α is off by at most 2. The $\sqrt{d}$ is the gap between signal and noise: a member scores of order d/n, and DP lets that exceed a stranger's spread $\sqrt{d}$ only by a constant factor. So δ must stay far below $1/\sqrt{d}$, which keeps the term $4\delta d$ of Equation 27.30 below $(e^\varepsilon - 1)\sqrt{d}$, and the term $4\delta d$ cannot be removed, because a mechanism that publishes each record with probability δ is $(0, \delta)$-DP (The definition of differential privacy).

The rate matches the Gaussian mechanism, but the constants are poor. At $\alpha = 0.1$, $\beta = 0.05$, $\varepsilon = 1$, $\delta = 10^{-6}$ the bound gives $n \ge 5.0$ for $d = 1000$ and $n \ge 15.8$ for $d = 10{,}000$, against the Gaussian mechanism's 10,836 and 38,570 (Figure 27.4). Published versions of this lower bound gain a factor $1/\alpha$ by padding the dataset with fixed records. Drawing p from a sharper distribution gains a further $\sqrt{\ln(1/\delta)}$ (Steinke & Ullman 2016). The last gap, $\sqrt{\ln d}$, comes from the union bound in the Gaussian upper bound, not from privacy.

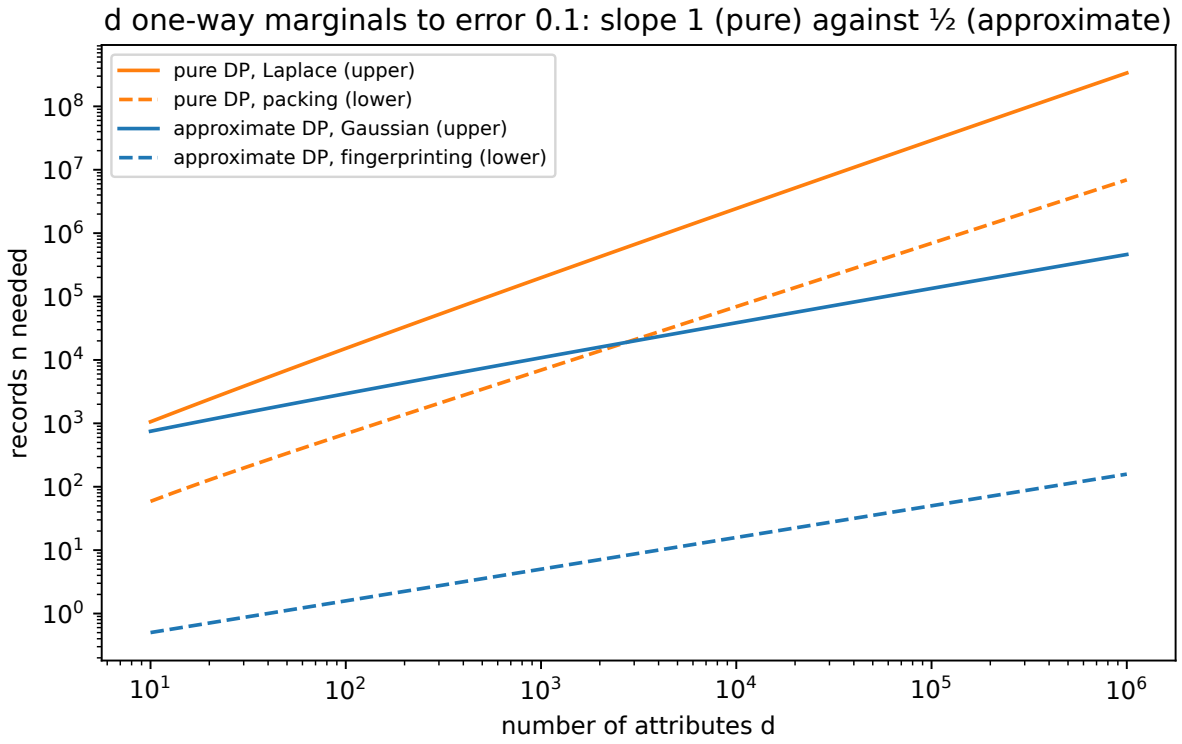


Figure 27.4.: Records needed to release d attribute means to ℓ_∞ error 0.1 with probability 0.95, at $\varepsilon = 1$ and $\delta = 10^{-6}$, on log–log axes. Orange: pure DP (Laplace upper bound, packing lower bound), slope 1. Blue: approximate DP (Gaussian upper bound, fingerprinting lower bound), slope ½.

Example 27.3. For $n = 5$ the script sums over all $2^5 = 32$ columns and integrates over p exactly, for the three estimates of Figure 27.3. For the sample mean, both sides of Equation 27.19 are 0.6667. For half the sample mean, the correlation is 0.3333 and the error is $\frac{1}{2}\mathbb{E}[|\bar{x}|] = 0.3000$, so the bound is $\frac{2}{3} - 0.6 = 0.0667$. For $f = 0$ the bound, $\frac{2}{3} - 1.2 = -0.5333$, is vacuous. So the lemma is tight for unbiased estimators and loose for shrunk ones.

27.4. Tracing attacks: the lower bound is a membership inference attack

In 2008 Homer et al. showed that the published allele frequencies of a genetic study revealed whether a given person's DNA was in it, and the US National Institutes of Health withdrew such summaries that year. Their test is, up to details, the score Z_i of Theorem 27.3. This section runs it on the random survey and compares its power with what DP allows. Dwork et al. (2015) proved it **robust**: it works on noisy means and with only a small reference sample.

The attacker sees $\hat{p} = \bar{x} + \xi$, the true means with Gaussian noise of standard deviation σ per coordinate, a target record x^* and the population means p. It scores $Z(x^*) = \langle \hat{p}, x^* - p \rangle$ and says "member" when the score is large. Each member pulls the release toward its own record by $1/n$ of its deviation, which shifts its score up. The attack's **advantage** is its true positive rate (TPR), how often it says "member" for a member, minus its false positive rate (FPR), how often it does so for a non-member.

Proposition 27.6 (Mean gap and spread of the tracing score)**.** *On the random survey of Definition 27.2, with the release $\hat{p} = \bar{x} + \xi$,*

$$\mathbb{E}[Z \mid \textit{member}] - \mathbb{E}[Z \mid \textit{non-member}] = \frac{2d}{3n}, \qquad \mathrm{Var}(Z \mid \textit{non-member}) = d\left(\frac{2}{15} + \frac{8}{15n} + \frac{2\sigma^2}{3}\right). \tag{27.34}$$

If the score is approximately normal with this mean gap κ and standard deviation s under both hypotheses, the best threshold has advantage $\mathrm{TPR} - \mathrm{FPR} = 2\Phi(\kappa/(2s)) - 1$.

Proof. For p uniform on $[-1, 1]$, $\mathbb{E}[p^2] = \frac{1}{3}$, $\mathbb{E}[p^4] = \frac{1}{5}$ and $\mathbb{E}[(1-p^2)^2] = \frac{8}{15}$. A ± 1 variable with mean p has square 1 and variance $1 - p^2$.

1. *Non-member mean.* A fresh x^* is independent of $\hat{p}$ given p, with deviations of mean 0, so $\mathbb{E}[Z] = 0$.
2. *Member mean.* For $x^* = x_i$, coordinate j contributes $\mathbb{E}[\bar{x}_j(x_{ij} - p_j)]$. In $\bar{x}_j$, other people's records are independent of x_{ij} and contribute 0. The member's own term contributes $\frac{1}{n}\mathbb{E}[x_{ij}(x_{ij} - p_j)] = \frac{1}{n}(1 - p_j^2)$. Averaging over p_j and summing over the d coordinates gives $\frac{2d}{3n}$.
3. *Non-member variance.* The deviations $x_j^* - p_j$ are independent of each other and of $\hat{p}$, so the cross terms of Z^2 vanish and
$$\mathrm{Var}(Z) = \sum_j \mathbb{E}[\hat{p}_j^2(1 - p_j^2)]. \tag{27.35}$$
Given p, the mean square of $\hat{p}_j$ is its squared mean plus the sampling and noise variances, $p_j^2 + \frac{1-p_j^2}{n} + \sigma^2$. Multiplying by $1 - p_j^2$ and averaging gives, per coordinate,
$$\mathbb{E}[p^2 - p^4] + \frac{1}{n}\mathbb{E}[(1-p^2)^2] + \sigma^2\,\mathbb{E}[1 - p^2] = \frac{2}{15} + \frac{8}{15n} + \frac{2\sigma^2}{3}. \tag{27.36}$$

4. *Advantage of a threshold.* For $\mathcal{N}(0, s^2)$ against $\mathcal{N}(\kappa, s^2)$, the Neyman–Pearson lemma says that the best test thresholds the density ratio, which grows with the score, at the crossing point $\kappa/2$. A member exceeds it with probability $\Phi(\frac{\kappa}{2s})$ and a non-member with probability $\Phi(-\frac{\kappa}{2s}) = 1 - \Phi(\frac{\kappa}{2s})$, so the advantage is $2\Phi(\frac{\kappa}{2s}) - 1$. This is the total variation (TV) distance of the two normals, which Hypothesis testing shows to be the best advantage of any test.

□

Without noise, the signal-to-noise ratio $\kappa/s \approx \frac{2d}{3n}\big/\sqrt{2d/15}$ is of order $\sqrt{d}/n$, so tracing reaches advantage 0.5, where $\kappa/(2s) = \Phi^{-1}(0.75) = 0.674$, once $d \approx 0.55\,n^2$. Noise adds $2\sigma^2/3$ per coordinate, so the attack fails when $\sigma \gg \sqrt{d}/n$, the order of the Gaussian mechanism's $\sigma = 2\sqrt{d}/(n\mu)$. Aggregates are not safe because they are aggregates; what protects members is the ratio of n to $\sqrt{d}$, or noise.

DP bounds the attack for each target, because the game is a test between the neighbours $M(D)$ and $M(D^{(i)})$. By Corollary 18.1 of the definition of differential privacy, its advantage is at most $(e^\varepsilon - 1 + 2\delta)/(e^\varepsilon + 1)$ (Equation 18.20), which is 0.4621 at $\varepsilon = 1$ and $\delta = 0$. For the Gaussian mechanism, step 4 with $\kappa = \Delta_2$ and $s = \sigma$ bounds the TV distance of the two outputs by $2\Phi(\Delta_2/(2\sigma)) - 1$.

Example 27.4. The script draws 200 datasets of $n = 50$ records with $d = 2000$ attributes and scores the 50 members and 50 fresh non-members of each. Without noise, Equation 27.34 predicts a mean gap of $2 \times 2000/(3 \times 50) = 26.67$ and a spread of $\sqrt{2000(\frac{2}{15} + \frac{8}{750})} = 16.97$; the simulation gives 26.70 and 16.92. The normal approximation gives the advantage $2\Phi(0.786) - 1 = 0.568$. Table 27.4 shows three noise levels, the last being the $\sigma = 6.67$ of the $(1, 10^{-5})$-DP Gaussian mechanism.

Table 27.4.: The tracing attack on released attribute means, $n = 50$, $d = 2000$. The advantage is the best TPR − FPR over thresholds.

Noise σ	Advantage (simulated)	Advantage (normal approximation)	Worst-case TV bound	TPR at 1% FPR
0	0.578	0.568	1.000	0.242
1.00	0.264	0.259	0.629	0.052
6.67 (DP, $\varepsilon = 1$)	0.049	0.044	0.107	0.014

The worst-case TV column uses $\Delta_2 = 2\sqrt{2000}/50 = 1.79$. Without noise, a quarter of the members are identified at a 1% false-alarm rate from the means of 50 people, a failure in the sense of Membership inference. With DP noise, at $\delta = 10^{-5}$, the advantage is 0.049, under both the TV bound of 0.107 and the generic bound, still about 0.462. The TPR at 1% FPR is 0.014, under the bound $e^1 \times 0.01 + 10^{-5} = 0.0272$ that the definition gives for the event "the test fires". Figure 27.5 shows the scores and Figure 27.6 the whole curve, which fades out around $\sigma = \sqrt{2000}/50 = 0.89$.

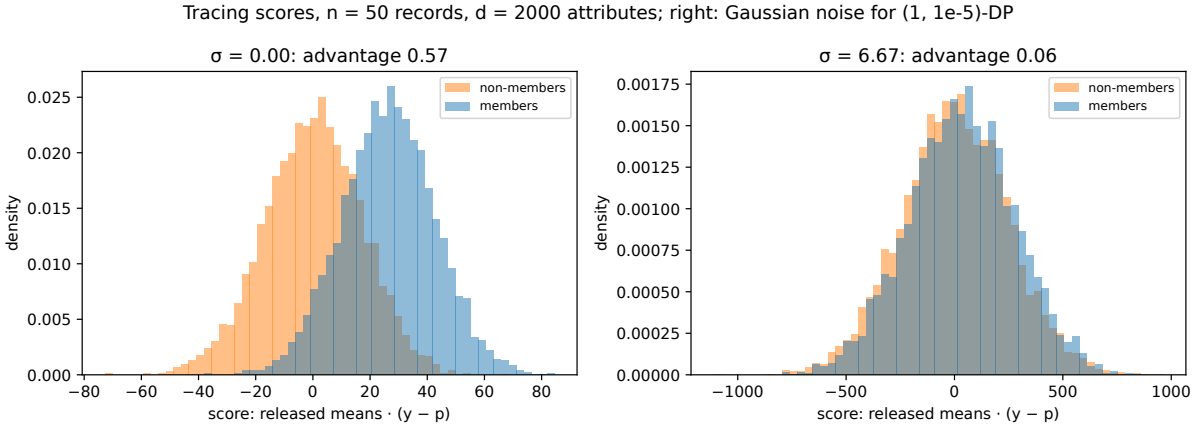


Figure 27.5.: Tracing scores of members (blue) and non-members (orange), $n = 50$, $d = 2000$. Left: exact means, with the members shifted by $2d/(3n) = 26.7$. Right: means with the noise of a $(1, 10^{-5})$-DP Gaussian mechanism.

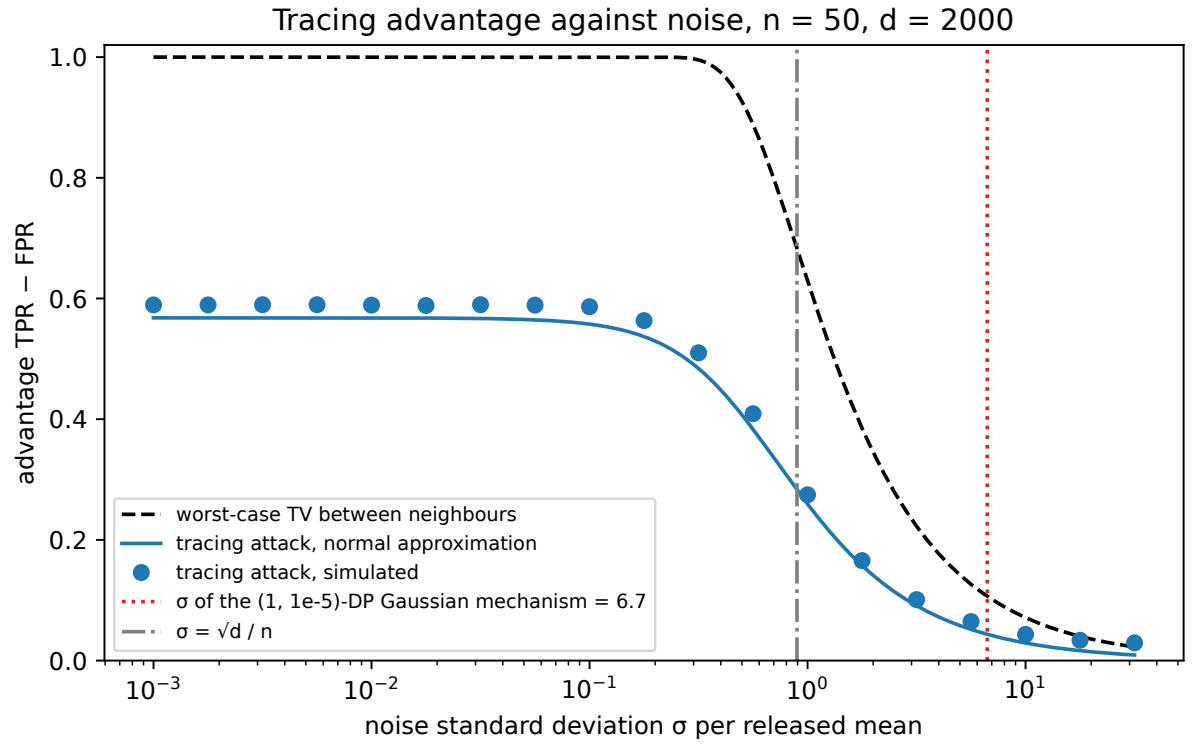


Figure 27.6.: Advantage of the tracing attack against the noise standard deviation, $n = 50$, $d = 2000$. Dots: simulation; solid: the normal approximation from Equation 27.34; dashed: the worst-case TV distance between neighbours' outputs, which bounds every attack.

27.5. Dinur–Nissim as a lower bound

Reconstruction attacks showed that n secret bits can be almost entirely reconstructed from about n random subset sums answered with error $o(\sqrt{n})$, error that grows more slowly than $\sqrt{n}$. Each query counts the 1s in a random subset of people, and the attacker fits the bits to the answers by least squares. Combined with DP's limit on guessing any single bit, it becomes a lower bound: a release that reveals most bits cannot be private.

The limit on guessing is Corollary 18.1 of the definition of differential privacy. If D and D' differ only in one uniformly random secret bit, every guess of that bit from the output of an (ε, δ)-DP mechanism is right with probability at most $(e^\varepsilon + \delta)/(e^\varepsilon + 1)$, which is $e/(e+1) = 0.7311$ at $\varepsilon = 1$ and $\delta = 0$. Read the other way, recovering a fraction r of the bits proves a lower bound on ε.

Theorem 27.4 (Reconstruction bounds ε from below)**.** *Let the database be n independent uniform bits, and call two databases neighbours when they differ in one bit. If M is (ε, δ)-DP, then the expected fraction r of bits that any reconstruction procedure recovers from M's output satisfies*

$$r \le \frac{e^\varepsilon + \delta}{e^\varepsilon + 1}, \qquad \text{so} \qquad \varepsilon \ge \ln \frac{r - \delta}{1 - r}. \tag{27.37}$$

In particular, a mechanism that answers $Q = O(n)$ random subset-sum queries with error $o(\sqrt{n})$ is not (ε, δ)-DP for any fixed ε and any $\delta < \frac{1}{2}$. Error $\Omega(\sqrt{n})$, at least a constant times $\sqrt{n}$, is necessary.

Proof.

1. *One bit at a time.* Fix i and every bit but x_i, which stays uniform because the bits are independent. Its two values give neighbouring databases, and the guess of x_i is a post-processing of M's output, so it is a test between them, right with probability at most $(e^\varepsilon + \delta)/(e^\varepsilon + 1)$. The uniform prior matters, because with a biased one guessing the likelier value already beats $\frac{1}{2}$.
2. *Average over the bits.* The expected fraction recovered is the average over i of the probability that bit i is guessed right, so it obeys step 1. Multiplying by $e^\varepsilon + 1$ gives $e^\varepsilon(1 - r) \ge r - \delta$, and dividing by $1 - r$ and taking logarithms gives the second form.
3. *Insert the attack.* By the Dinur–Nissim theorem, in the sharper form of Dwork, McSherry and Talwar (2007) that needs only $O(n)$ queries, errors of $o(\sqrt{n})$ let the decoder miss only $o(n)$ bits. So $r \to 1$, and the bound forces $\varepsilon \to \infty$.

□

The Gaussian mechanism matches it. Changing one bit changes by 1 the answer of every query whose subset contains that person, so the answers move by at most $\sqrt{Q}$ in ℓ_2, and σ proportional to $\sqrt{Q}/\mu(\varepsilon, \delta)$ suffices, of order $\sqrt{n}$ for $Q = 4n$. So independent noise costs error of order $\sqrt{Q}$ per query under approximate DP, and of order Q under pure DP. For many more than n queries, private multiplicative weights (Query release) answers every query from a small private synthetic dataset, with error that grows only like a small power of $\ln Q$.

The bound is crude, because least squares is not the best attack, so the ε it proves is far below the one the Gaussian mechanism guarantees. The true privacy loss lies between them. A lower

bound from one attack is never a guarantee, and auditing, in a later part, narrows the gap with stronger attacks.

Example 27.5. The script answers $Q = 800$ random subset sums of $n = 200$ uniform bits with Gaussian noise of standard deviation σ, decodes by least squares and rounds at $\frac{1}{2}$, over 40 databases. Table 27.5 gives the fraction recovered, the ε it implies by Equation 27.37 at $\delta = 10^{-6}$, and the ε the Gaussian mechanism guarantees with $\mu = \sqrt{800}/\sigma$. For instance, recovering 0.9607 of the bits proves $\varepsilon \geq \ln\left((0.9607 - 10^{-6})/(1 - 0.9607)\right) = 3.20$.

Table 27.5.: Reconstruction from 800 noisy subset sums of 200 bits, and the privacy loss it proves, at $\delta = 10^{-6}$.

$\sigma/\sqrt{n}$	Fraction recovered	ε at least (attack)	ε at most (Gaussian mechanism)
0.1	1.0000	∞	294.17
0.25	0.9607	3.20	69.24
0.5	0.8058	1.42	26.36
1	0.6710	0.71	11.00
2	0.5810	0.33	4.89

At $\sigma = 0.1\sqrt{200} = 1.4$ every bit is recovered, so the release is not $(\varepsilon, 10^{-6})$-DP for any finite ε. A mechanism with $\varepsilon = 1$ lets an attacker recover at most 73.11% of the bits, which the table places between $0.5\sqrt{n}$ and $\sqrt{n}$ of noise per answer. Recovering 67% of the bits at $\sigma = \sqrt{n}$ proves only $\varepsilon \geq 0.71$, while the guarantee there is $\varepsilon \leq 11.00$. Figure 27.7 shows the whole range.

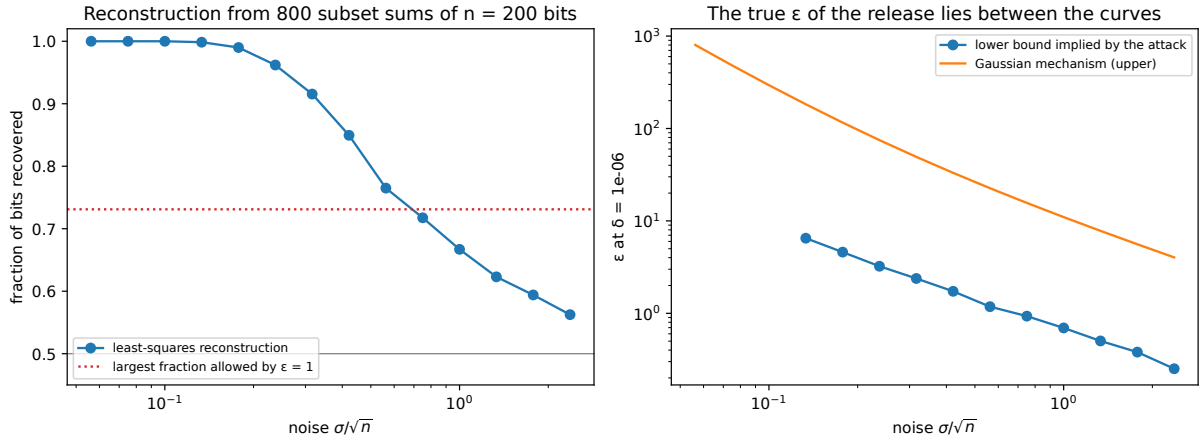


Figure 27.7.: Left: fraction of 200 secret bits recovered by least squares from 800 noisy subset sums, against $\sigma/\sqrt{n}$; dotted, the most any $\varepsilon = 1$ mechanism allows. Right: the ε the reconstruction proves (blue) and the ε the Gaussian mechanism guarantees (orange), at $\delta = 10^{-6}$.

27.6. Code

The script code/lower_bounds.py computes every number in this chapter and draws every figure. It checks the Laplace histogram, report-noisy-max, the tracing attack and the reconstruction attack against their formulas by simulation, and the fingerprinting lemma by exact summation. From the repository root:

```
uv run python code/part03-dp-foundations/lower_bounds.py
```

Table 27.6.: The script's output with the default seed.

Experiment	Result
What DP allows, $\varepsilon = 1$	Advantage at most 0.4621; bit guessed at most 0.7311 of the time; group $\delta_5 = 8.6 \times 10^{-5}$, $\delta_{20} = 282$ at $\delta = 10^{-6}$
Mode and histogram, $N = 10^6$	Lead $k \geq 13.76$ (noisy-max needs 18.44); error ≥ 6.38 (Laplace 16.79); full table in Table 27.2
Simulations at $N = 1000$	Laplace 95% error 9.84 (exact 9.88); noisy-max failure 0.0228 at lead 11.09 (union bound 0.05)
Packing under $\delta = 10^{-6}$, 2^d candidates	Forces $k \geq 7$ at $d = 10$ and $k \geq 13$ at $d = 100$ and 1000 (pure: 6.88, 69.26, 693.10); Equation 27.8: 7, 11, 11; $\ln(1/\delta)/\varepsilon = 13.82$
Packing for a mean, $\alpha = 0.1$	$n \geq 4.55$, 9.16, 16.07 for $R = 10$, 10^3, 10^6; under $\delta = 10^{-6}$: 4, 9, 11
Sample complexity, histogram $N = 10^6$	$\alpha = 0.1$: 64 private, 875 sampling. $\alpha = 0.01$: 638 private, 87,522 sampling
Sample complexity, marginals	Table 27.3; the Gaussian beats the pure-DP lower bound from $d = 2731$; $\mu(1, 10^{-6}) = 0.2367$
Fingerprinting lower bound	$n \geq 1.6$, 5.0, 15.8, 50.1 at $d = 10^2, 10^3, 10^4, 10^5$
Fingerprinting lemma, $n = 5$	Mean: 0.6667 (bound 0.6667). Half mean: 0.3333 (bound 0.0667). Zero: 0 (bound -0.5333)
Tracing, $n = 50$, $d = 2000$	Gap 26.70 (predicted 26.67), spread 16.92 (16.97); Table 27.4
Dinur–Nissim, $n = 200$, $Q = 800$	Table 27.5

tests/test_lower_bounds.py checks that the output still matches.

27.7. Summary

Table 27.7.: The results of this chapter, and where each is used.

Result	Statement	Use
Lemma 27.1	m disjoint good sets within k of D_0: $k \geq \ln(m(1-\beta))/\varepsilon$	Outputs that single out one of many candidates
Theorem 27.1	Mode: lead $k \geq \ln(N(1-\beta))/\varepsilon$; histogram: $\alpha \geq \frac{1}{2}(\ln(N(1-\beta))/\varepsilon - 1)$	Pure DP pays $\ln(\text{candidates})/\varepsilon$
Proposition 27.1	Laplace: $\alpha \leq \ln(N/\beta)/\varepsilon$; noisy-max fails with probability $\leq (N-1)e^{-k\varepsilon}(2+k\varepsilon)/4$	Both optimal up to a factor of about 2
Proposition 27.2	$e^{-k\varepsilon}(1-\beta) \leq 2k\delta$ or $me^{-k\varepsilon}(1-\beta) \leq 2$	Packing forces at most about $\ln(1/\delta)/\varepsilon$
Proposition 27.3	$n \geq (\ln(N(1-\beta)) - \varepsilon)/(2\alpha\varepsilon)$; sampling $\approx \ln(2N/\beta)/(2\alpha^2)$	Privacy adds records, mostly for coarse answers
Theorem 27.2	$n \gtrsim d \ln 2/(\alpha\varepsilon)$	Pure DP pays linearly in the dimension
Proposition 27.4	Laplace: $n = 2d\ln(d/\beta)/(\alpha\varepsilon)$; Gaussian: $n = 2\sqrt{d}\,z/(\mu\alpha)$	The Gaussian beats pure DP from $d = 2731$
Proposition 27.5	$n \geq \ln(\lfloor R/\alpha \rfloor(1-\beta))/\varepsilon$	The cost of not knowing where the data lie
Lemma 27.2	$\mathbb{E}[f\sum_i(x_i - p)] = 2\mathbb{E}[pf] \geq \frac{2}{3} - 2\mathbb{E}[\lvert f - \bar{x}\rvert]$	Accuracy forces correlation with the records
Lemma 27.3	$\mathbb{E}[h(M(D))] \leq \mathbb{E}[h(M(D'))] + (e^\varepsilon - 1)\mathbb{E}[\lvert h(M(D'))\rvert] + 2\delta B$	DP for averages, not only events
Theorem 27.3	$n \geq (\frac{2}{3} - 2\alpha')d/((e^\varepsilon - 1)\sqrt{d} + 4\delta d)$	Approximate DP pays $\sqrt{d}$; keep $\delta \ll 1/\sqrt{d}$
Proposition 27.6	Gap $2d/(3n)$; variance $d(\frac{2}{15} + \frac{8}{15n} + \frac{2\sigma^2}{3})$	Advantage 0.5 once $d \approx 0.56\,n^2$; $\sigma \gg \sqrt{d}/n$ stops it
Theorem 27.4	Recovering a fraction r proves $\varepsilon \geq \ln\frac{r-\delta}{1-r}$	Error $o(\sqrt{n})$ on $O(n)$ queries is never private

27.8. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 8: packing lower bounds and reconstruction attacks as lower bounds, at textbook depth.
- Vadhan, "The Complexity of Differential Privacy" (2017), §5: packing, fingerprinting and reconstruction lower bounds side by side; also the stability-based histogram for approximate DP.

- Hardt & Talwar, "On the Geometry of Differential Privacy" (STOC 2010): packing generalized to any set of linear queries through volume arguments, with matching upper bounds for pure DP.
- Beimel, Brenner, Kasiviswanathan & Nissim, "Bounds on the Sample Complexity for Private Learning and Private Data Release" (Machine Learning, 2014): the sample-complexity view of packing.
- Bun, Ullman & Vadhan, "Fingerprinting Codes and the Price of Approximate Differential Privacy" (STOC 2014): the first $\sqrt{d}$ lower bound for approximate DP, through fingerprinting codes.
- Steinke & Ullman, "Between Pure and Approximate Differential Privacy" (Journal of Privacy and Confidentiality, 2016): the simplified fingerprinting lemma and the tight bound $\Omega(\sqrt{d \ln(1/\delta)}/(\alpha\varepsilon))$ for one-way marginals.
- Dwork, Smith, Steinke, Ullman & Vadhan, "Robust Traceability from Trace Amounts" (FOCS 2015): tracing attacks that tolerate noisy statistics and estimated population means.
- Dwork, Smith, Steinke & Ullman, "Exposed! A Survey of Attacks on Private Data" (Annual Review of Statistics and Its Application, 2017): reconstruction and tracing attacks for a statistics audience.
- Homer et al., "Resolving Individuals Contributing Trace Amounts of DNA to Highly Complex Mixtures Using High-Density SNP Genotyping Microarrays" (PLoS Genetics, 2008): the tracing attack on genomic summary statistics.
- Kamath & Ullman, "A Primer on Private Statistics" (2020, arXiv): fingerprinting lower bounds for private mean estimation, with the score-and-integrate-by-parts proof used here.
- Dinur & Nissim, "Revealing Information while Preserving Privacy" (PODS 2003): the reconstruction attack and the $\sqrt{n}$ threshold.
- Boneh & Shaw, "Collusion-Secure Fingerprinting for Digital Data" (IEEE Transactions on Information Theory, 1998) and Tardos, "Optimal Probabilistic Fingerprint Codes" (STOC 2003): the cryptographic codes that gave fingerprinting its name.

28. Private mean estimation

The mean is the statistic that private machine learning computes most often. Every step of private gradient descent clips per-example gradients, averages them and adds noise, and a survey that reports the share of people with some property releases a mean of yes/no answers. Earlier chapters released the mean of the dataset in hand. An analyst usually wants something else: the mean of the **population** the data were drawn from. So this chapter asks how many records a private estimate of a population mean needs, and what each thing you do not know about the data costs.

Two guarantees with different shapes meet here. Privacy must hold for every dataset, including ones that your assumptions say cannot happen, because a guarantee that fails when an assumption fails protects nobody. Accuracy, on the other hand, is a statement about data drawn from a distribution, such as "the records are normal with variance 1". So an assumption about the data may make a mechanism more accurate, but it may never enter the privacy proof. Every mechanism below respects that split: it clips the data to a range, which makes it private whatever the data are, and the assumption only says how often the clipping changes nothing.

One idea then organizes the chapter: spend a small part of the budget to learn where the data are, then clip there and spend the rest on the mean. The noise of a clipped mean is proportional to the width of the clipping range, so a range set by your prior ignorance makes the noise pay for that ignorance in full. Finding the data privately first costs a number of records that grows only like the logarithm of the range, and the final step can then clip to a narrow window and add little noise.

The running example is a survey of $n = 1000$ people at $\varepsilon = 1$, and each section changes what is known about the records: yes/no answers first, then normal measurements whose mean could be anywhere up to a bound R, then vectors, then incomes with a heavy tail. Throughout, datasets have a fixed size n, and two datasets $D \sim D'$ are **neighbours** when they differ in one replaced record, as in Beyond global sensitivity. The records $X_1, \dots, X_n$ are drawn independently from a distribution whose mean μ is the target, and every statement of the form "the data lie in this range" is allowed to fail with probability $\beta = 0.05$. An estimate $\hat{\mu}$ is judged by its **root-mean-square error** (RMSE), the square root of the average of $(\hat{\mu} - \mu)^2$ over the data and the mechanism's noise. The guarantees are those of the definition of differential privacy (DP): a mechanism is (ε, δ)-DP when changing one record changes the probability of any set of outputs by at most a factor e^{ε} plus δ, and pure ε-DP when $\delta = 0$. Table 28.1 lists the symbols the chapter uses.

Table 28.1.: The symbols of this chapter.

Symbol	Meaning
n, ε, δ	the number of records, fixed and public; the parameters of DP

Symbol	Meaning
μ, σ^2	the mean and the variance of the distribution the records come from
α, β	the target RMSE; the probability that a "the data lie in this range" statement fails
w, R	the width of a known range; a bound on $\lvert\mu\rvert$, or on the length $\lVert\mu\rVert$
τ_1, τ_d	how far n normal records can lie from μ, in one and in d dimensions
b, m	the width and the number of the bins of the coarse step
d, r_t, c_t, s_t	the dimension; the radius, the centre and the noise standard deviation of round t of shrinking balls
$\kappa(\varepsilon, \delta)$	the largest ratio of ℓ_2 sensitivity to noise standard deviation at which Gaussian noise is (ε, δ)-DP
τ, τ_b	the clipping half-width for heavy-tailed data, and its worst-case choice

28.1. Bounded data

Start where nothing is unknown except the mean. The records lie in a known interval, such as yes/no answers coded 0 and 1, so no clipping is needed and no assumption about the data enters at all. The question is how many more records a private estimate needs than a non-private one. The answer is the best case of the chapter, and every later section tries to recover it when the data are not bounded.

Every estimator in this chapter has the same form: clip each record to an interval, average, and add Laplace noise. The lemma below says what such an estimator costs in privacy and in error, and every later proof starts from it. Its privacy part is the Laplace mechanism of the Laplace and Gaussian mechanisms applied to a mean. Its error part is the three-way split of the clipping section of Beyond global sensitivity: the clipped average varies from sample to sample, clipping shifts its expected value away from μ, and the noise adds its own spread.

Lemma 28.1 (A clipped mean with Laplace noise)**.** *Let each record X_i be replaced by a clipped value Y_i that depends on X_i alone and lies in an interval of width w, and release the average of the clipped values plus Laplace noise ξ,*

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} Y_i + \xi, \qquad \xi \sim \mathrm{Lap}\Big(\frac{w}{n\varepsilon}\Big). \tag{28.1}$$

The release is ε-DP for every dataset. When the records are independent draws from a distribution with mean μ, the mean squared error of the release splits into three parts,

$$\mathbb{E}\big[(\hat{\mu}-\mu)^2\big] = \underbrace{\frac{\operatorname{Var}(Y)}{n}}_{\text{sampling}} + \underbrace{\big(\mathbb{E}[Y]-\mu\big)^2}_{\text{bias}^2} + \underbrace{\operatorname{Var}(\xi)}_{\text{noise}}, \qquad \operatorname{Var}(\xi) = \frac{2w^2}{n^2\varepsilon^2}. \tag{28.2}$$

Proof. The plan is to bound the sensitivity of the clipped average, calibrate the Laplace noise to it, then square the error and watch the cross terms vanish.

1. *Sensitivity.* Replacing one record changes one clipped value Y_i, by at most w since both values lie in the interval, and the other values stay put because each depends on its own record alone. So the sum moves by at most w and the average by at most w/n, which is its sensitivity under replace neighbours, as for the mean with public n in Proposition 20.2.
2. *Privacy.* By Theorem 20.1, Laplace noise with scale equal to the sensitivity divided by ε, here $w/(n\varepsilon)$, makes the release ε-DP. Nothing in steps 1 and 2 assumed anything about the data, because the clipping, not an assumption, keeps every Y_i in the interval.
3. *The three parts.* Write $\hat{\mu}-\mu$ as a sum of three pieces: the average of $Y_i-\mathbb{E}[Y]$ over the records, the constant $\mathbb{E}[Y]-\mu$, and the noise ξ. The first and the last have mean 0 and are independent of each other, so when the sum is squared and averaged, every cross term contains a mean-zero factor and vanishes. The average of n independent copies has $1/n$ times the variance of one, which gives the sampling part.
4. *The noise.* By Theorem 20.1 a Laplace variable with scale s has variance $2s^2$, which with $s = w/(n\varepsilon)$ is $2w^2/(n^2\varepsilon^2)$.

□

With the lemma in hand, the bounded case is a matter of reading off the error and solving for n. Bounded records need no clipping, so the bias is zero, and the only question is how the sampling term and the noise term add up.

Theorem 28.1 (Private mean of bounded data)**.** *Let the records be independent draws from a distribution on an interval of width w with variance σ^2, and release their average plus* $\operatorname{Lap}(w/(n\varepsilon))$ *noise. The release is ε-DP, and its RMSE is*

$$\operatorname{RMSE} = \sqrt{\frac{\sigma^2}{n} + \frac{2w^2}{n^2\varepsilon^2}}. \tag{28.3}$$

The RMSE is at most α once

$$n \geq \frac{\sigma^2 + \sqrt{\sigma^4 + 8w^2\alpha^2/\varepsilon^2}}{2\alpha^2}, \qquad \textit{which is at most} \qquad \frac{\sigma^2}{\alpha^2} + \frac{\sqrt{2}\,w}{\alpha\varepsilon}. \tag{28.4}$$

Proof.

1. *Privacy and error.* The records already lie in the interval, so Lemma 28.1 applies with $Y_i = X_i$ and width w, and the release is ε-DP. In Equation 28.2 the bias is 0, the sampling term is σ^2/n and the noise term is $2w^2/(n\varepsilon)^2$, and their sum is the square of Equation 28.3.

2. *Solve for n.* Setting the square of Equation 28.3 equal to α^2 and multiplying by n^2 gives the quadratic $\alpha^2 n^2 - \sigma^2 n - 2w^2/\varepsilon^2 = 0$, whose positive root is the first form of Equation 28.4. The RMSE falls as n grows, so every n above the root reaches α.
3. *The simpler bound.* The square root of a sum is at most the sum of the square roots, so $\sqrt{\sigma^4 + 8w^2\alpha^2/\varepsilon^2} \le \sigma^2 + 2\sqrt{2}\, w\alpha/\varepsilon$. Inserting this and dividing by $2\alpha^2$ gives the second form.

□

The second form of Equation 28.4 reads as two costs added together. Sampling needs about σ^2/α^2 records whatever you do, and privacy adds about $w/(\alpha\varepsilon)$ more. The ratio of the privacy term to the sampling term is $\sqrt{2}\, w\alpha/(\varepsilon\sigma^2)$, so for a small target error α privacy costs a lower-order number of records. Halving ε doubles the privacy term and leaves the sampling term alone.

The privacy term grows with the width w and not with the variance σ^2, because the noise is set by the worst change one record can make, which is the width, whatever the data look like. The variance describes typical data, and typical data enter only the sampling term. That split between worst-case noise and typical-case sampling is the pattern of every section below.

Example 28.1. Take yes/no answers, so $w = 1$, and the worst case for sampling, a population that is half yes, so $\sigma = 0.5$. At $\varepsilon = 1$ the private RMSE is 0.0520 for 100 people, against 0.0500 without privacy. For 1,000 people it is 0.0159 against 0.0158, and at 10,000 both are 0.0050. To reach $\alpha = 0.05$ you need 108 people instead of 100, and to reach $\alpha = 0.01$ you need 2,508 instead of 2,500. Privacy costs eight people at a sample size where sampling already needs 2,500.

28.2. Gaussian data with a known range

Now let the records be unbounded, but drawn from a normal distribution with unknown mean μ and variance 1, and suppose all you know about the mean is that $|\mu| \le R$. The bound R measures your ignorance. A height in centimetres might have R of a few hundred, while a feature of a model you have never looked at might need $R = 10^6$. Unbounded records have no finite sensitivity, so the estimator must clip, and the obvious choice is to clip to the whole range where the data could possibly be. This section works out what that costs, and the answer is the baseline that the next section improves on.

Clipping changes a record only if the record lies outside the range. To show that this is rare, the proof needs to know how far n normal draws can stray from their mean, which is the lemma below.

Lemma 28.2 (Normal draws stay close to their mean)**.** *Let $X_1, \dots, X_n$ be independent draws from a normal distribution with mean μ and variance 1, and let $\beta \in (0, 1)$. With probability at least $1 - \beta$, every draw lies within*

$$\tau_1 = \sqrt{2 \ln(2n/\beta)} \tag{28.5}$$

of μ.

Proof.

1. *One draw.* A standard normal variable exceeds a level t in absolute value with probability at most $2e^{-t^2/2}$, which is the Chernoff bound of Concentration inequalities applied to each of its two tails. Each $X_i - \mu$ is standard normal.
2. *All draws.* The probability that at least one of the n draws lies further than t from μ is at most the sum of their n probabilities, by the union bound, which is $2ne^{-t^2/2}$. Setting this equal to β and solving for t gives τ_1.

□

The bound is loose. Concentration inequalities proves a tail bound half as large for one draw, but the factor 2 changes τ_1 only slightly, so the chapter keeps the simpler form. For $n = 1000$ and $\beta = 0.05$ it gives $\tau_1 = 4.604$, and in 20,000 simulated datasets the largest distance exceeded τ_1 in a fraction 0.0039 of them, not 0.05.

The naive estimator widens the range $[-R, R]$ by τ_1 on each side and clips there, so that with high probability it clips nothing.

Proposition 28.1 (The naive estimator)**.** *Let the records be independent draws from a normal distribution with mean μ, where $|\mu| \le R$, and variance 1, and let τ_1 be as in Equation 28.5. Clip every record to $[-R - \tau_1, R + \tau_1]$, average, and add* $\mathrm{Lap}(2(R + \tau_1)/(n\varepsilon))$ *noise. The release is ε-DP for every dataset. With probability at least $1 - \beta$ the clipping changes no record. On that event the estimate is the plain average plus noise, and, ignoring the rare event where it fails, the error has no bias and*

$$\mathrm{RMSE} = \sqrt{\frac{1}{n} + \frac{8(R + \tau_1)^2}{n^2\varepsilon^2}}. \tag{28.6}$$

Proof.

1. *Privacy.* The clipping interval has width $w = 2(R + \tau_1)$, so Lemma 28.1 gives ε-DP for every dataset.
2. *Clipping changes nothing, usually.* By Lemma 28.2, with probability at least $1 - \beta$ every record lies within τ_1 of μ. Since $|\mu| \le R$, every record then lies in $[-R - \tau_1, R + \tau_1]$, where clipping leaves it alone. This is why the interval is widened by τ_1: a mean near R has half its records above R, and clipping them would bias the estimate toward 0.
3. *Error.* On that event the clipped average is the plain average, so Equation 28.2 has no bias, a sampling term $1/n$, and the noise term $2w^2/(n^2\varepsilon^2)$ with $w = 2(R + \tau_1)$, which is $8(R + \tau_1)^2/(n^2\varepsilon^2)$. Their sum is the square of Equation 28.6.

□

The noise grows linearly in R, so reaching an error α takes about $1/\alpha^2 + R/(\alpha\varepsilon)$ records. The estimator pays for R in full: doubling your uncertainty about where the data are doubles the noise, even though the data themselves have not changed. Normality entered only in step 2, to show that the clipping usually changes nothing, while the privacy of step 1 holds for every dataset.

The lower-bounds chapter shows that paying for R in full is not the price of privacy. Its packing argument gives a lower bound of only $\ln(R/\alpha)/\varepsilon$ records, so a cleverer mechanism should pay logarithmically, not linearly, for the range.

Example 28.2. Take $n = 1000$ records at $\varepsilon = 1$, with $\beta = 0.05$ so that $\tau_1 = 4.604$, and put the true mean at $\mu = R/2$. For $R = 10$, the noise standard deviation is 0.0413 and the RMSE is 0.0520, against 0.0316 without privacy. The error grows with the range, to 2.8416 for $R = 10^3$ and to 2828.44 for $R = 10^6$. That last estimate is useless, although the 1,000 records pin the mean down to within about 0.03. The script also computes the exact error, without conditioning on the event of Lemma 28.2, from the mean and variance of a clipped normal variable, and the two agree to the digits quoted.

28.3. Coarse then fine

The naive estimator wastes its noise on the empty parts of $[-R, R]$. Karwa & Vadhan (2018) split the work in two. A **coarse step** spends a small part of the budget to find, privately, a short interval that holds the data. A **fine step** then clips to that interval and spends the rest of the budget on the mean. The coarse step is a selection problem among about R candidate intervals, and selection costs only the logarithm of the number of candidates, which is how the range drops out of the noise.

Two earlier results carry the proof. The first is the utility bound of the exponential mechanism (Selection). When the mechanism chooses among m candidates with a score of sensitivity 1 at budget ε_1, it picks a candidate whose score is at least g below the best with probability at most $m\,e^{-\varepsilon_1 g/2}$, which is Equation 22.4 with $t = \varepsilon_1 g/2 - \ln m$. The second is adaptive basic composition (Post-processing, group privacy and composition). If a first step is ε_1-DP and a second step, which may use the first step's output, is ε_2-DP for each value of that output, then together they are $(\varepsilon_1 + \varepsilon_2)$-DP. That is what allows the coarse step to choose the clipping window of the fine step.

Theorem 28.2 (Coarse then fine)**.** *Let the records be independent draws from a normal distribution with mean μ, where $|\mu| \le R$, and variance 1, let τ_1 be as in Equation 28.5, and split the budget as $\varepsilon = \varepsilon_1 + \varepsilon_2$. Cut $[-R - b, R + b]$ into $m = \lceil 2(R + b)/b \rceil$ bins of width $b = 2\tau_1$, where $\lceil \cdot \rceil$ rounds up.*

1. Coarse step. *Choose a bin J with the exponential mechanism at ε_1, with the number of records in the bin as its score.*
2. Fine step. *Clip every record to the **window** made of bin J and its two neighbours, an interval of width $3b$. Average, and add* $\mathrm{Lap}(3b/(n\varepsilon_2))$ *noise.*

The mechanism is ε-DP for every dataset. With probability at least $1 - \beta - m e^{-\varepsilon_1 n/4}$, no record is clipped. On that event the estimate is the plain average plus noise, and, ignoring the rare event where it fails,

$$\mathrm{RMSE} = \sqrt{\frac{1}{n} + \frac{18 b^2}{n^2 \varepsilon_2^2}}, \tag{28.7}$$

which does not depend on R. The coarse step fails with probability at most β once

$$n \ge \frac{4 \ln(m/\beta)}{\varepsilon_1}. \tag{28.8}$$

Proof. The plan is to show that the two steps are private by composition, that the data fill at most two bins, that the coarse step therefore picks a non-empty bin, that the window around a non-empty bin holds every record, and finally to read the error off Equation 28.2.

1. *Privacy.* Replacing one record takes it out of one bin and puts it into another, so every count changes by at most 1, and the coarse step is the exponential mechanism with a score of sensitivity 1, which is ε_1-DP. For each value of J, the fine step is a clipped mean over a window of width $3b$, which is ε_2-DP by Lemma 28.1. By adaptive basic composition the two steps together are ε-DP. The window depends on the data only through J, which was chosen privately, so no leak of the kind seen in Beyond global sensitivity arises.
2. *The data fill at most two bins.* By Lemma 28.2, with probability at least $1 - \beta$ every record lies within $\tau_1 = b/2$ of μ. Then the records span an interval of length at most b, which meets at most two neighbouring bins. So the fullest bin holds at least $n/2$ records, and every other bin except one is empty.
3. *The coarse step picks a non-empty bin.* An empty bin has score 0, at least $n/2$ below the best. The utility bound with the m bins as candidates and the gap $g = n/2$ bounds the chance of picking one by $me^{-\varepsilon_1 n/4}$. Setting this equal to β and solving for n gives Equation 28.8.
4. *The window holds every record.* A non-empty bin J contains a record within $b/2$ of μ, so μ lies within $b/2$ of the bin. Every record lies within $b/2$ of μ, hence within b of the bin, which is inside the window. Clipping to bin J alone would not do, because the data can straddle two bins and J may be the less full one.
5. *Error.* On the events of steps 2 and 3, the fine step clips nothing, so Equation 28.2 has no bias, a sampling term $1/n$, and the noise term with $w = 3b$ and budget ε_2, which is $18b^2/(n^2\varepsilon_2^2)$. That is Equation 28.7.

□

The range R now enters only through the number of bins m, and only inside a logarithm in Equation 28.8. So an error α needs about $1/\alpha^2 + 1/(\alpha\varepsilon) + \ln R/\varepsilon$ records in total, up to factors of $\sqrt{\ln n}$ hidden in b. The packing bound Equation 27.17 of the lower-bounds chapter says a pure-DP mechanism must pay that logarithm, so the coarse step is optimal up to constants. The fine step pays for a window of width $3b = 6\sqrt{2\ln(2n/\beta)}$, a few standard deviations, which is where the $\ln n$ factors come from. The bins cannot be much narrower, because step 2 needs the data to fill at most two of them, so that the fullest bin leads every empty one by $n/2$.

The split has a cost when you already know a lot. The fine step gets only ε_2 and clips to a window wider than the spread of the data, so for small R the naive estimator wins. With $n = 1000$, $\varepsilon = 1$ and a quarter of the budget for the coarse step, the crossover is at $R = 13.81$ (Figure 28.2). The quarter is a simple default rather than the best split.

Example 28.3. Take $n = 1000$, $\varepsilon = 1$ and $\beta = 0.05$ again, with $\varepsilon_1 = 0.25$ and $\varepsilon_2 = 0.75$. The bins have width $b = 9.207$ and the window 27.62. Table 28.2 compares the two estimators. The fine step's noise has standard deviation 0.0521 and the RMSE is 0.0609 for every R, while the naive estimator's error grows with R. The number of records the coarse step needs, by Equation 28.8, grows from 74 to 245 as R grows by a factor of 10^5. The bound on its failure probability is below 10^{-21} in all three cases, because 1,000 records is far more than it needs.

Table 28.2.: The naive and coarse-then-fine estimators of a normal mean, $n = 1000$, $\varepsilon = 1$, $\beta = 0.05$, a quarter of the budget for the coarse step: the formulas Equation 28.6 and Equation 28.7, and simulations with 4,000 repetitions, whose own error is about 2%.

Range R	Bins m	Coarse step needs $n \geq$	Naive RMSE	Coarse then fine: RMSE	Simulated, naive	Simulated, coarse then fine
10	5	74	0.0520	0.0609	0.0517	0.0588
10^3	220	135	2.8416	0.0609	2.7615	0.0606
10^6	217,223	245	2828.44	0.0609	2803.08	0.0596

Figure 28.1 shows the coarse step on one dataset with $R = 100$ and $\mu = 37$, which has 24 bins. On the script's own draw of such a dataset, 876 and 124 records fall into the two non-empty bins. The exponential mechanism picks the fuller one with a probability that prints as 1 to twelve digits, and any given empty bin with probability about 2.8×10^{-48}.

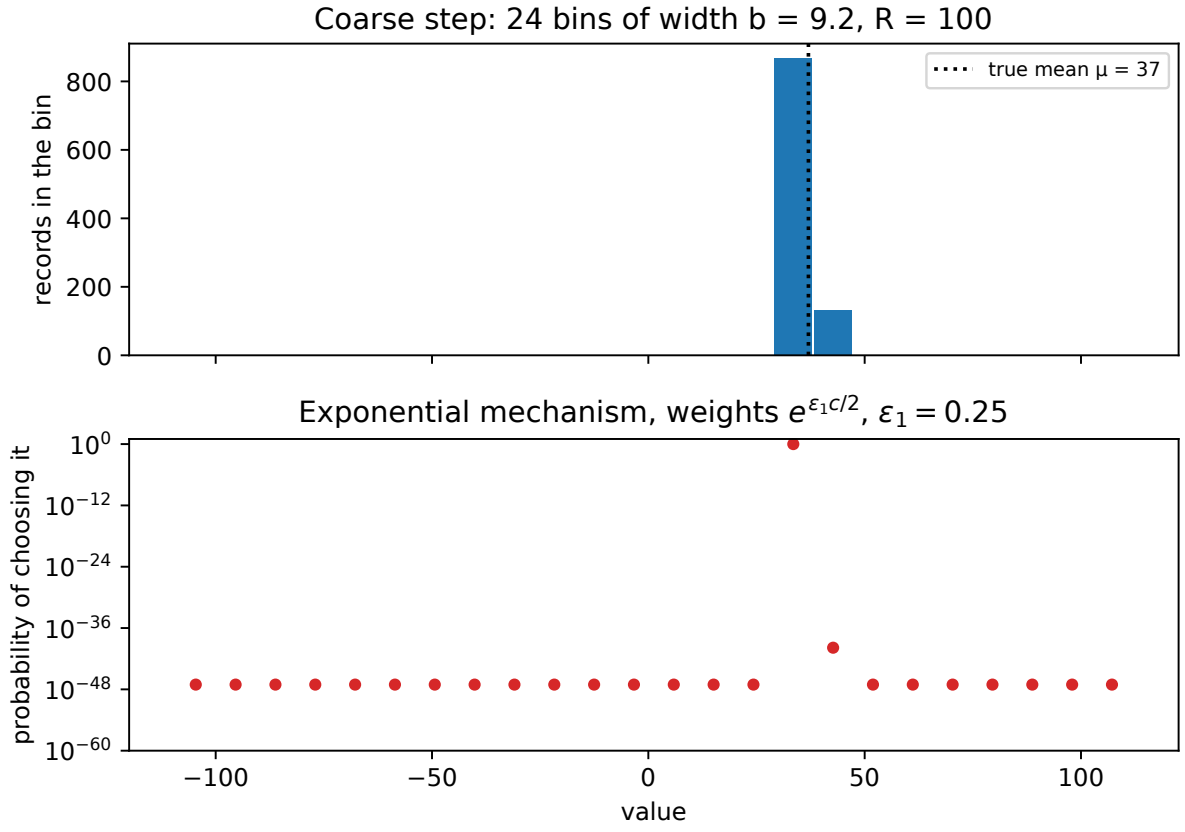


Figure 28.1.: Top: the coarse histogram of 1,000 normal records with mean 37, in 24 bins of width 9.2 covering a range with $R = 100$ (a fresh draw). Bottom: the probability that the exponential mechanism with $\varepsilon_1 = 0.25$ picks each bin, on a log scale.

Under approximate DP the range can go altogether. The stability-based release reports the fullest bin exactly, with high probability, once it leads the others by about $2\ln(1/\delta)/\varepsilon$ records, whatever

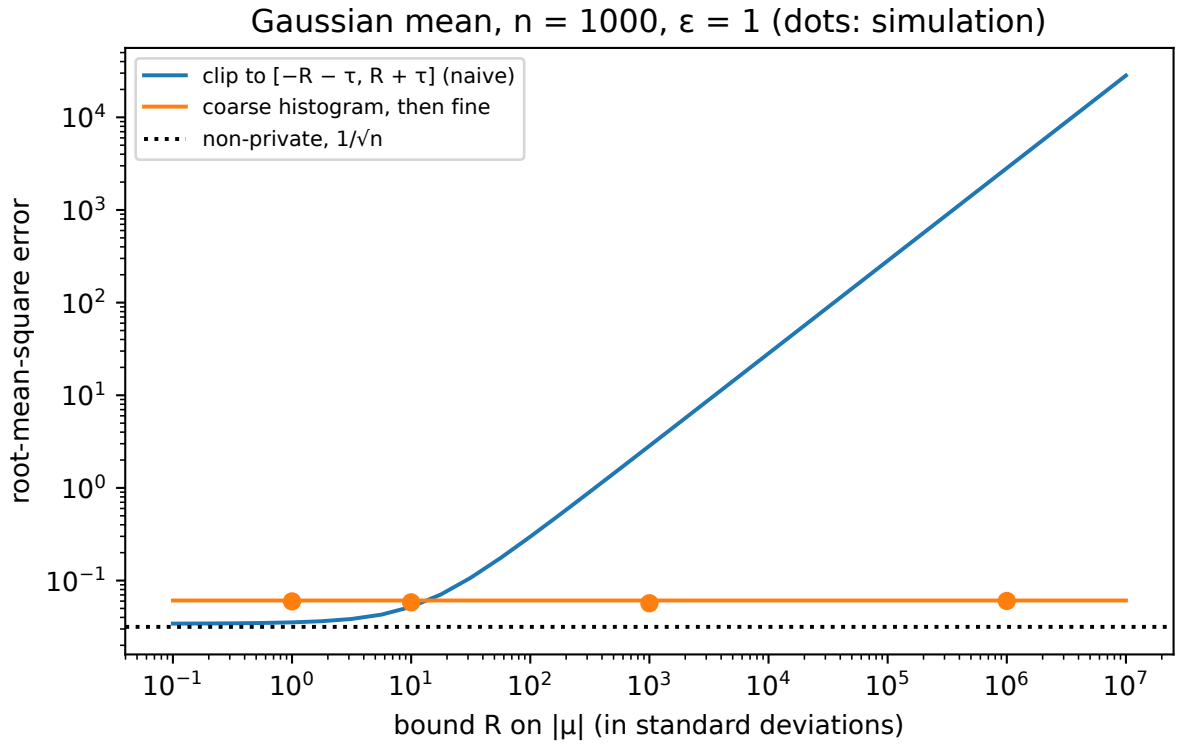


Figure 28.2.: Root-mean-square error of the naive estimator and of coarse then fine, against the bound R on the mean, for $n = 1000$ normal records and $\varepsilon = 1$. The dots are simulations of coarse then fine; the dotted line is the non-private error $1/\sqrt{n}$.

the number of bins. Its test is noisy, so a smaller lead gives a refusal rather than a wrong bin. Karwa & Vadhan use a stability-based histogram of this kind, so their (ε, δ)-DP coarse step needs a number of records of order $\ln(1/\delta)/\varepsilon$ and works even with no bound R at all, which matches the packing bound under approximate DP, Equation 27.8. Section J.3 in the appendices proves the histogram's guarantee for added or removed records; under this chapter's replaced records two bins change, so the histogram is run at $\varepsilon/2$ and $\delta/2$, which changes only the constant.

The same trick handles an unknown variance. Differences of pairs of records, $X_2 - X_1$ and so on, have mean 0 and a spread set by the variance alone. Put the sizes $|X_2 - X_1|, |X_4 - X_3|, \ldots$ into bins whose edges double, from $[1, 2)$ to $[2, 4)$ and so on, scaled to the possible range of the standard deviation σ. Most sizes fall within a factor of a few of σ, so the fullest bins locate σ within a constant factor. With σ known to lie between $\sigma_{\min}$ and $\sigma_{\max}$ there are about $\log_2(\sigma_{\max}/\sigma_{\min})$ bins, and choosing a full one with the exponential mechanism costs, by the same utility bound, records of order the logarithm of that number divided by ε (Karwa & Vadhan 2018 give the details).

28.4. Many dimensions: shrinking balls

Now each record is a vector in d dimensions, drawn from a normal distribution with mean μ and identity covariance, and all you know is that the length of μ, $\|\mu\|$, is at most R. Error is measured by the length of $\hat{\mu} - \mu$. The coarse step of Section 28.3 does not carry over cheaply, because a histogram over cells of a grid would need about $(R/b)^d$ cells, and the logarithm of that is $d \ln(R/b)$, a cost linear in d. Biswas, Dong, Kamath & Ullman (2020) shrink a ball instead. Keep a ball that contains μ, clip the data to a slightly larger ball, take a noisy mean, and use it to draw a smaller ball. A few rounds take the ball from radius R down to the spread of the data.

Two tools replace the one-dimensional ones. The noise is Gaussian rather than Laplace, because one record moves a vector average in every coordinate at once, and Gaussian noise is calibrated to the ℓ_2 sensitivity, the most one record can move the answer in length (Proposition 20.3). Write $\kappa(\varepsilon, \delta)$ for the largest ratio of ℓ_2 sensitivity to noise standard deviation at which Gaussian noise is (ε, δ)-DP, the root that Corollary 20.1 finds by bisection and calls μ^*; this page writes it κ because μ is the mean here. Gaussian noise of standard deviation $\Delta_2/\kappa(\varepsilon, \delta)$ on a query of ℓ_2 sensitivity Δ_2 is then (ε, δ)-DP, and so is any larger noise. The second tool is the d-dimensional version of Lemma 28.2, which the proof needs twice: once to say how far records stray from μ, and once to bound the length of a round's noise.

Lemma 28.3 (Normal vectors stay close to their mean)**.** *Let Z be a standard normal vector in d dimensions, that is, a vector of d independent standard normal coordinates. For every $t \geq 0$,*

$$\Pr\left[\|Z\| > \sqrt{d} + t\right] \leq e^{-t^2/2}. \tag{28.9}$$

So k independent such vectors all have length at most $\sqrt{d} + \sqrt{2\ln(k/\beta)}$ with probability at least $1 - \beta$.

Proof.

1. *One vector.* The length $\|Z\|$ changes by at most the length of the move when Z moves, and such functions of a normal vector concentrate around their average. That average is at most $\sqrt{d}$, because the mean of a length is at most the square root of its mean square, which is d. The tail bound is Boucheron, Lugosi & Massart (2013, ch. 5), and Section E.5 in the appendices proves the same bound with the Chernoff method alone.
2. *Several vectors.* Give each of the k vectors a share β/k of the failure probability. Setting $e^{-t^2/2} = \beta/k$ gives $t = \sqrt{2\ln(k/\beta)}$, and the probability that at least one vector is longer is at most the sum of the k shares, by the union bound, which is β.

□

The mechanism below keeps two lengths fixed before it sees the data: a radius τ_d that every record stays within, from the lemma with $k = n$, and a margin u that bounds the sampling error and the noise of each step, from the lemma with $k = 1$.

Theorem 28.3 (Shrinking balls)**.** *Let the records be independent draws from a normal distribution in d dimensions with mean μ, where $\|\mu\| \le R$, and identity covariance, and let $T \ge 0$ be a number of rounds. Two lengths set the scale. The first bounds how far records lie from the mean,*

$$\tau_d = \sqrt{d} + \sqrt{2\ln(2n/\beta)}, \tag{28.10}$$

and the second, u, is a margin for the $T+1$ steps of the mechanism, T rounds and a final step:

$$u = \sqrt{2\ln(4(T+1)/\beta)}. \tag{28.11}$$

Give each step a budget $(\varepsilon_t, \delta_t)$ such that all the steps add up to ε and δ. Start from the ball of radius $r_0 = R$ around the centre $c_0 = 0$. In round $t = 0, \dots, T-1$:

1. *Move every record that lies outside the ball of radius $r_t + \tau_d$ around c_t onto its surface.*
2. *Release the average plus Gaussian noise with standard deviation $s_t = 2(r_t + \tau_d)/(n\,\kappa(\varepsilon_t, \delta_t))$ in each coordinate, and call the result c_{t+1}.*
3. *Set the next radius to*

$$r_{t+1} = \frac{\sqrt{d} + u}{\sqrt{n}} + s_t(\sqrt{d} + u). \tag{28.12}$$

A final step does steps 1 and 2 once more, with the rest of the budget and noise s_T, and releases the noisy average. The mechanism is (ε, δ)-DP for every dataset. With probability at least $1 - \beta$, every ball contains μ and no record is ever moved. On that event the final release is the plain average plus noise, and, ignoring the rare event where it fails,

$$\mathrm{RMSE} = \sqrt{\frac{d}{n} + d\,s_T^2}. \tag{28.13}$$

Proof. The plan is to show that each step is a Gaussian mechanism and compose them, then that, as long as the current ball holds μ, nothing is clipped and the next ball holds μ too, and finally to read the error off the last step.

1. *Privacy of one step.* After the move of step 1 every record lies in a ball of radius $r_t + \tau_d$, whose diameter is $2(r_t + \tau_d)$. Replacing one record therefore moves the sum by a vector of length at most $2(r_t + \tau_d)$, and the average by that divided by n, which is the ℓ_2 sensitivity. The noise s_t is this sensitivity divided by $\kappa(\varepsilon_t, \delta_t)$, so the step is $(\varepsilon_t, \delta_t)$-DP for each value of the centre, which was chosen privately. Adaptive basic composition (Post-processing, group privacy and composition) adds the budgets of the $T + 1$ steps.
2. *Records stay within τ_d of the mean.* Each $X_i - \mu$ is a standard normal vector, so Lemma 28.3 with $k = n$ vectors and failure probability $\beta/2$ says that, with probability at least $1 - \beta/2$, every record lies within $\sqrt{d} + \sqrt{2\ln(2n/\beta)} = \tau_d$ of μ.
3. *The balls keep the mean.* Suppose μ lies in the ball of round t and the event of step 2 holds. Then every record lies within τ_d of μ, hence inside the clipping ball, and nothing moves. So $c_{t+1} - \mu$ is the sampling error $\bar{X} - \mu$ plus the noise. The sampling error is a normal vector with variance $1/n$ in each coordinate, so $\sqrt{n}$ times it is a standard normal vector, and the noise divided by s_t is one too. By Lemma 28.3 with $k = 1$ and failure probability $\beta/(4(T + 1))$ for each of the two, the sampling error has length at most $(\sqrt{d} + u)/\sqrt{n}$ and the noise has length at most $s_t(\sqrt{d} + u)$, so μ lies in the next ball, of radius Equation 28.12. The $2(T + 1)$ shares of $\beta/(4(T + 1))$ and the $\beta/2$ of step 2 add up to β, which is the total failure probability.
4. *Final error.* On that event the final release is $\bar{X}$ plus independent noise. The expected squared length of $\bar{X} - \mu$ is d/n, the sum of d variances of $1/n$, and that of the noise is $d\, s_T^2$, and the cross term has mean 0 as in Lemma 28.1. That gives Equation 28.13.

□

In a round, the radius drops from r_t to about $2(r_t + \tau_d)\sqrt{d}/(n\,\kappa(\varepsilon_t, \delta_t))$. When n is larger than a constant times $\sqrt{d}/\kappa(\varepsilon_t, \delta_t)$, which is of order $\sqrt{d\ln(1/\delta)}/\varepsilon_t$, that is a reduction by a constant factor, and about $\ln R$ rounds bring the radius down to the order of $\tau_d \approx \sqrt{d}$. Below that the ball cannot shrink, because the records themselves are spread about $\sqrt{d}$ around μ and the clipping ball must leave them alone. In the final step the sensitivity is then about $2\sqrt{d}/n$, so $d\, s_T^2$ is about $4d^2/(n\kappa)^2$, and an error α needs n of order $d/(\alpha\kappa)$, that is, $d\sqrt{\ln(1/\delta)}/(\alpha\varepsilon)$. Reaching an error α therefore takes about $d/\alpha^2 + d\sqrt{\ln(1/\delta)}/(\alpha\varepsilon) + \sqrt{d}\ln R/\varepsilon$ records, up to logarithmic factors.

Without privacy the error is $\sqrt{d/n}$. The privacy noise is about $d/(n\varepsilon)$ up to logarithmic factors, so it grows like d instead of $\sqrt{d}$, and it is larger than the sampling error unless n is above about d/ε^2. That is not a weakness of the method. The fingerprinting lower bound says that releasing d means with an error α' in each coordinate needs about $\sqrt{d}/(\alpha'\varepsilon)$ records. An ℓ_2 error α spread over d coordinates is an error of about $\alpha/\sqrt{d}$ in each, so substituting $\alpha' = \alpha/\sqrt{d}$ gives $d/(\alpha\varepsilon)$. Kamath, Li, Singhal & Ullman (2019) prove the matching lower bound directly for normal data.

The range costs $\sqrt{d}\ln R/\varepsilon$, which is less than the $d\ln R/\varepsilon$ of a grid histogram, because each round makes progress in all d directions at once. The rounds compose here by basic composition, which adds the budgets; the tighter accountants of Part 4 make several rounds cheaper, and the original method uses one of them. This is the shape of every private average of vectors, including the average of clipped gradients in private training. There the clipping norm is a fixed number rather than a shrinking ball, and choosing it is the clipping trade-off of Beyond global sensitivity in many dimensions.

Example 28.4. The running example needs more records here, because an error below 1 in $d = 1000$ dimensions is out of reach for 1,000 records even without privacy, where the sampling

error alone is $\sqrt{d/n} = 1$. Take $n = 100{,}000$ records with $\|\mu\| \le 1000$, at $\varepsilon = 1$ and $\delta = 10^{-6}$, with $\beta = 0.05$ and a quarter of the budget for the rounds. Here $\kappa(1, 10^{-6}) = 0.2367$. Table 28.3 compares the naive estimator, which clips to the ball of radius $R + \tau_d$, with shrinking balls. The script tries every number of rounds up to 29 and keeps the best, which is two for each d. For $d = 100$ the radius falls from 1000 to 8.54 after one round and to 0.243 after the second. The ratio of the privacy noise to $d/(n\varepsilon)$ falls from 32.2 at $d = 10$ to 14.2 at $d = 1000$, as the terms in $\sqrt{d}$ give way to the term in d. A simulation at $d = 100$ with 40 datasets gives an RMSE of 0.0363 against the formula's 0.0366, and every ball contained μ in all 40.

Table 28.3.: The mean of $n = 100{,}000$ normal vectors in d dimensions with $\|\mu\| \le 1000$, at $(1, 10^{-6})$-DP, by the naive estimator and by shrinking balls: the ℓ_2 RMSE from Equation 28.13.

d	Non-private $\sqrt{d/n}$	Naive RMSE	Rounds	Radii	Shrinking balls: RMSE
10	0.0100	0.2697	2	1000, 4.12, 0.0725	0.0105
100	0.0316	0.8586	2	1000, 8.54, 0.243	0.0366
1000	0.1000	2.7730	2	1000, 22.9, 1.43	0.1736

28.5. Heavy tails: clip into the data

Normal data rarely stray far, so every estimator so far could clip without touching a record. Many real quantities have heavy tails: incomes, session lengths, the norms of per-example gradients. Suppose all you know is that the variance is at most σ^2. Chebyshev's inequality then only says that a record is more than $\sigma\sqrt{n/\beta}$ from the mean with probability at most β/n, so a clipping range wide enough to touch no record has half-width $\sigma\sqrt{n/\beta}$, and its noise is enormous. The way out is to clip into the data on purpose and accept a bias, and the question is how wide to clip.

The estimator clips around a centre that a coarse step has placed near the mean; to keep the formulas clean, the theorem takes the centre to be μ itself, and writes τ for the clipping half-width. The bias of clipping is the average amount by which records stick out of the window, and a variance bound controls that amount: a record that sticks out contributes to the variance a squared distance at least τ times its distance, so the excess of all such records together is at most σ^2/τ.

Theorem 28.4 (Clipping into heavy-tailed data)**.** *Let the records be independent draws from any distribution with mean μ and variance at most σ^2. Clip every record to $[\mu - \tau, \mu + \tau]$ for a half-width $\tau > 0$, average, and add* $\mathrm{Lap}(2\tau/(n\varepsilon))$ *noise. The release is ε-DP for every τ and every dataset, and*

$$\mathrm{RMSE} \le \frac{\sigma}{\sqrt{n}} + \frac{\sigma^2}{\tau} + \frac{2\sqrt{2}\,\tau}{n\varepsilon}. \tag{28.14}$$

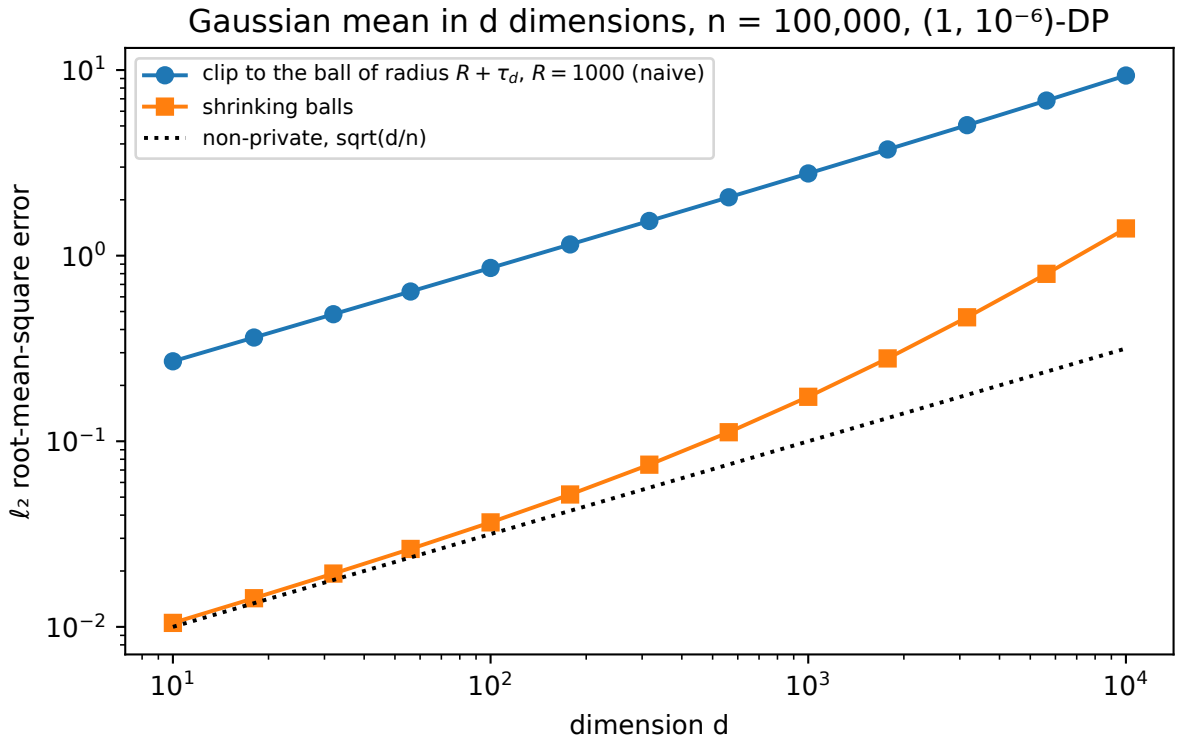


Figure 28.3.: The ℓ_2 root-mean-square error of the mean of 100,000 normal vectors against the dimension d, at $(1, 10^{-6})$-DP with $\|\mu\| \leq 1000$: clipping to the ball of radius $R + \tau_d$, shrinking balls, and the non-private error $\sqrt{d/n}$.

The right side is smallest at

$$\tau_b = \sigma\sqrt{\frac{n\varepsilon}{2\sqrt{2}}}, \qquad \textit{where it equals} \qquad \frac{\sigma}{\sqrt{n}} + 2\sigma\sqrt{\frac{2\sqrt{2}}{n\varepsilon}}. \tag{28.15}$$

Proof. The plan is to bound the three parts of Equation 28.2 one at a time, add them, and minimize over τ.

1. *Privacy.* The clipping interval has width 2τ, so Lemma 28.1 gives ε-DP for every dataset, whatever τ is.
2. *Bias.* Clipping moves a record X toward μ by $(|X-\mu|-\tau)_+$, the amount by which its distance from μ exceeds τ, or 0. So the bias is at most the average of that amount. The amount is positive only where $|X-\mu| > \tau$, and there it is at most $|X-\mu| \le (X-\mu)^2/\tau$, because dividing $|X-\mu|$ by τ gives a number above 1. Averaging, the bias is at most $\mathbb{E}[(X-\mu)^2]/\tau \le \sigma^2/\tau$.
3. *Sampling.* A variance is half the average squared difference between two independent copies of a variable. Clipping to an interval never moves two values further apart, so the clipped record varies at most as much as the record itself, and its variance is at most σ^2. The sampling term of Equation 28.2 is at most σ^2/n.
4. *Noise.* The clipping interval has width 2τ, so by Equation 28.2 the noise standard deviation is $\sqrt{2}\cdot 2\tau/(n\varepsilon)$.
5. *Add.* By Equation 28.2, the RMSE is the length of a vector with these three components, and a length is at most the sum of its components. That gives Equation 28.14.
6. *Minimize.* The two terms that depend on τ are σ^2/τ, falling, and $2\sqrt{2}\,\tau/(n\varepsilon)$, rising. The derivative of their sum, $-\sigma^2/\tau^2 + 2\sqrt{2}/(n\varepsilon)$, is zero at τ_b, and there the two terms are equal, each $\sigma\sqrt{2\sqrt{2}/(n\varepsilon)}$. That gives Equation 28.15.

□

Requiring the second term of Equation 28.15 to be at most α and squaring shows that an error α needs about $\sigma^2/\alpha^2 + 8\sqrt{2}\,\sigma^2/(\alpha^2\varepsilon)$ records, of order $1/(\alpha^2\varepsilon)$. Without clipping into the data, at $\tau = \sigma\sqrt{n/\beta}$, it would need of order $1/(\beta\alpha^2\varepsilon^2)$, because the noise term $2\sqrt{2}\,\sigma/(\varepsilon\sqrt{n\beta})$ must then fall below α. With heavy tails, then, privacy stops being a lower-order cost. For bounded data the privacy term in Equation 28.4 had a single $1/\alpha$, while here it has $1/\alpha^2$, the same power as sampling, divided by ε. Kamath, Singhal & Ullman (2020) show that this is unavoidable: no ε-DP mechanism does better for every distribution with bounded variance. Section J.4 in the appendices derives this lower bound, and its k-th moment version, from a two-point distribution and group privacy. They also show what more knowledge buys. If the k-th moment $\mathbb{E}[|X-\mu|^k]$ is bounded for some $k > 2$, the cost falls to about $1/(\varepsilon\,\alpha^{k/(k-1)})$ records, and as k grows it approaches the $1/(\alpha\varepsilon)$ of bounded data.

The bound Equation 28.14 is a worst case, and it is tight up to a constant factor. A sharper version of step 2 gives $\sigma^2/(4\tau)$, because $(|y|-\tau)_+ \le y^2/(4\tau)$ for every y, and a distribution that puts a little mass at distance 2τ on one side of the mean attains that. So the worst case hides its variance in a few far-out records on one side. For an ordinary heavy-tailed distribution the bias falls faster than σ^2/τ, the best τ is smaller than τ_b, and the error is lower. But τ_b uses only n, ε and the variance bound, so it can be set without looking at the data; if the variance bound is four times too large, τ_b doubles, the release stays private and only the accuracy suffers. Setting τ

from the data, such as to the observed maximum, would bring back the leak of Beyond global sensitivity.

Example 28.5. Take a Pareto distribution with shape 3, the law of $x_m U^{-1/3}$ for a uniform U on $(0, 1)$, scaled so that its variance is 1. Its smallest value is $x_m = 1.1547$ and its mean is 1.7321, and its third moment is infinite. With $n = 1000$ and $\varepsilon = 1$, Table 28.4 compares three choices of τ. Clipping wide enough to touch nothing costs an RMSE of 0.4012. The worst-case choice $\tau_b = 18.80$ gives 0.0607, and the best τ for this particular distribution, 5.36, gives 0.0329, close to the non-private 0.0316. The bound Equation 28.15 promises 0.1380, which is loose for this distribution, as predicted: for this law the average excess beyond a point falls like the inverse square of the point rather than its inverse. At $n = 100{,}000$ the pattern repeats, with RMSE 0.0401, 0.0062 and 0.0032 for the three choices.

Table 28.4.: Clipped private means of Pareto records with variance 1, $n = 1000$, $\varepsilon = 1$: the exact terms of Equation 28.2 for three clipping half-widths τ around the mean, and simulations with 4,000 repetitions.

τ	Bias	Sampling sd	Noise sd	RMSE	Simulated
141.42 (no record clipped)	0.0000	0.0313	0.4000	0.4012	0.3997
18.80 (worst-case best, τ_b)	−0.0018	0.0293	0.0532	0.0607	0.0591
5.36 (best for this distribution)	−0.0153	0.0249	0.0152	0.0329	0.0325

28.6. Code

The script code/mean_estimation.py computes every number in this chapter and draws its figures. Each estimator and each error formula is a small typed function. The exact errors use the mean and variance of a clipped normal or Pareto variable, and the simulations run the mechanisms on fresh data. From the repository root:

```
uv run python code/part03-dp-foundations/mean_estimation.py
```

Table 28.5.: The script's output with seed 0.

Experiment	Result
Spread of 1,000 normal records, $\beta = 0.05$	$\tau_1 = 4.604$; exceeded in a fraction 0.0039 of 20,000 simulated datasets

Experiment	Result
Survey proportion, $\varepsilon = 1$	108 records for RMSE 0.05 (100 without privacy); 2,508 for 0.01 (2,500)
Naive estimator, $n = 1000$	RMSE 0.0520, 2.8416 and 2828.44 for $R = 10$, 10^3, 10^6
Coarse then fine, $n = 1000$	RMSE 0.0609 for every R (simulated 0.0588, 0.0606, 0.0596); coarse step needs 74, 135, 245 records; beats naive from $R = 13.81$
Shrinking balls, $n = 100{,}000$, $R = 1000$	RMSE 0.0105, 0.0366, 0.1736 for $d = 10$, 100, 1000, against 0.0100, 0.0316, 0.1000 without privacy; simulated 0.0363 at $d = 100$
Pareto records, $n = 1000$	RMSE 0.4012 without clipping into the data, 0.0607 at τ_b, 0.0329 at the best τ; bound 0.1380

tests/test_mean_estimation.py checks that the output still matches this table.

28.7. Summary

Table 28.6.: The results of this chapter, and where each is used.

Result	Statement	Use
Lemma 28.1	Clip to width w, average, add $\text{Lap}(w/(n\varepsilon))$: ε-DP for every dataset; error^2 = sampling + bias^2 + $2w^2/(n\varepsilon)^2$	Privacy comes from the clipping alone; let assumptions about the data enter only through how often clipping changes nothing
Theorem 28.1	RMSE $= \sqrt{\sigma^2/n + 2w^2/(n\varepsilon)^2}$; error α from $\sigma^2/\alpha^2 + \sqrt{2}\,w/(\alpha\varepsilon)$ records	Proportions, surveys, clipped sums: privacy is a lower-order cost
Lemma 28.2	n normal draws lie within $\tau_1 = \sqrt{2\ln(2n/\beta)}$ of μ with probability $1 - \beta$	Widening a clipping range so that it touches no record
Proposition 28.1	Clip to $[-R - \tau_1, R + \tau_1]$: error α from about $1/\alpha^2 + R/(\alpha\varepsilon)$ records	The baseline; never clip to a range set by your ignorance
Theorem 28.2	Exponential mechanism on bins of width $2\tau_1$, then clip to 3 bins: $1/\alpha^2 + 1/(\alpha\varepsilon) + \ln R/\varepsilon$ records; the coarse step needs $4\ln(m/\beta)/\varepsilon_1$	Means with unknown location; choosing a clipping range privately with a quarter of the budget or less; under approximate DP a stability-based histogram replaces $\ln R$ by $\ln(1/\delta)$

Result	Statement	Use
Lemma 28.3	$\Pr[\|Z\| > \sqrt{d} + t] \le e^{-t^2/2}$ for a standard normal vector	The radius τ_d and the margin u of shrinking balls
Theorem 28.3	Shrinking balls with Gaussian noise, radius Equation 28.12: $d/\alpha^2 + d\sqrt{\ln(1/\delta)}/(\alpha\varepsilon) + \sqrt{d}\ln R/\varepsilon$ records	Private averages of vectors, such as gradients: budget about $d\sqrt{\ln(1/\delta)}/(\alpha\varepsilon)$ records for privacy on top of d/α^2 for sampling
Theorem 28.4	Clip to $\mu \pm \tau_b$ with $\tau_b = \sigma\sqrt{n\varepsilon/(2\sqrt{2})}$: $\sigma^2/\alpha^2 + 8\sqrt{2}\,\sigma^2/(\alpha^2\varepsilon)$ records; with a bounded k-th moment, $1/\alpha^2 + 1/(\varepsilon\,\alpha^{k/(k-1)})$	Incomes, session lengths, gradient norms: clip into the data, with a width set from n, ε and a variance bound, never from the data

28.8. References

- Karwa & Vadhan, "Finite Sample Differentially Private Confidence Intervals" (ITCS 2018): coarse then fine for normal means and variances, under pure and approximate DP
- Biswas, Dong, Kamath & Ullman, "CoinPress: Practical Private Mean and Covariance Estimation" (NeurIPS 2020): shrinking confidence balls for means and covariances, with code
- Kamath, Li, Singhal & Ullman, "Privately Learning High-Dimensional Distributions" (COLT 2019): the $d/(\alpha\varepsilon)$ cost for normal means in d dimensions, with a matching lower bound
- Kamath, Singhal & Ullman, "Private Mean Estimation of Heavy-Tailed Distributions" (COLT 2020): the $1/(\alpha^2\varepsilon)$ rate for bounded variance and its generalization to k-th moments, with lower bounds
- Kamath & Ullman, "A Primer on Private Statistics" (arXiv 2020): the univariate and multivariate cases and the lower bounds, at textbook depth
- Kamath, *CS 860: Algorithms for Private Data Analysis* (University of Waterloo, lectures on private mean estimation): the ladder of rates from bounded to heavy-tailed data that organizes this chapter
- Bun & Steinke, "Average-Case Averages: Private Algorithms for Smooth Sensitivity and Mean Estimation" (NeurIPS 2019): mean estimation by smooth sensitivity of a trimmed mean
- Boucheron, Lugosi & Massart, *Concentration Inequalities* (Oxford, 2013), ch. 5: concentration of the length of a normal vector

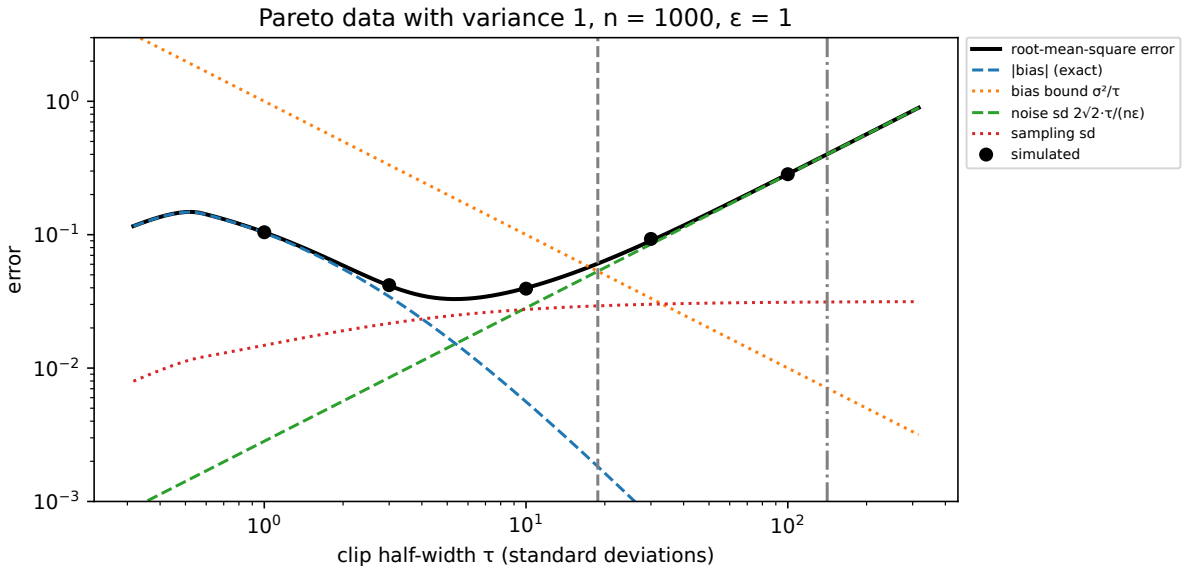


Figure 28.4.: Error of the clipped private mean of Pareto records with variance 1, $n = 1000$, $\varepsilon = 1$, against the clipping half-width τ: the RMSE, the exact bias and its bound σ^2/τ, the noise and the sampling standard deviations, and simulated points. The dashed vertical line is τ_b and the dash-dotted one the width that clips no record.

Part IV.

Privacy accounting

The previous part defined differential privacy (DP), built the basic mechanisms and proved how (ε, δ) guarantees compose. This part answers the question that decides whether a trained model's privacy claim means anything: **after thousands of noisy steps, each on a random subset of the data, how large is ε, really?** The answer is never read off a single theorem. It comes from an **accountant**, a procedure that tracks some summary of each step's privacy loss, combines the summaries, and converts the result into the (ε, δ) you report. The running case throughout is stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, called DP-SGD, and the workhorse summary is Rényi differential privacy (RDP). Two accountants given the same training run can report values of ε that differ by a large factor. Both can be correct upper bounds, but only the tighter one tells you what the noise actually bought.

Who this part is for

You are expected to have read the prerequisites and the part on DP foundations. From Divergences you need the Rényi divergence D_α, the hockey-stick divergence E_γ and the data-processing inequality. From Hypothesis testing you need trade-off functions $T(P, Q)$ and the Gaussian trade-off curve. From Concentration inequalities you need Hoeffding's lemma, and from Optimization the contraction of a gradient step. From the DP foundations you need randomized response and the hypothesis-testing reading of DP, basic composition, the privacy loss random variable with advanced and optimal composition from Composition, the Gaussian mechanism and its exact $\delta(\varepsilon)$ from the Laplace and Gaussian mechanisms, and the local and shuffle models from Trust models. Each chapter restates the facts it uses in one line and links to them.

The common thread

Composition is easy for the privacy loss itself. If you run k mechanisms, the privacy loss of the whole run is the sum of the k per-step losses, and its distribution is a convolution. Composition is hard for (ε, δ), because a pair of numbers forgets almost everything about that distribution. Advanced composition recovers part of what is lost. The rest of the field rests on one move: **track a richer summary of the privacy loss, one that composes simply, and convert to (ε, δ) only once, at the end.**

The summaries differ in how much they keep, and Table 28.7 lists them.

Table 28.7.: The chapters of this part, the question each one answers, the summary of the privacy loss it tracks, and the earlier chapters it uses.

Chapter	The question it answers	Summary it tracks	Builds on
Rényi DP	Which summary of the privacy loss adds up under composition?	The curve $\varepsilon(\alpha)$ of Rényi divergences	2, 3, 18–21

Chapter	The question it answers	Summary it tracks	Builds on
From Rényi DP to (ε, δ) and to tests	How much does converting an RDP curve to (ε, δ) lose, and what is unavoidable?	The same curve, read as a constraint on tests	3, 4, 18, 21, 29
Concentrated differential privacy	When does one number ρ describe the whole curve, and when not?	A line $\rho\alpha$ above the curve	2, 3, 19–21, 25, 29, 30; one formula from 33
f-DP and Gaussian differential privacy	What is the most informative description of a mechanism's privacy?	The full trade-off function; μ for Gaussian ones	3, 4, 18, 21, 29, 30
Amplification by subsampling	How much privacy does using each record only with probability q buy?	RDP of the subsampled Gaussian	3, 18–21, 29, 30
Numerical accounting with privacy loss distributions	How do you compute the exact ε of a composition, with error bars?	The discretized distribution of the privacy loss	3, 18, 20, 21, 29, 30, 32, 33
Amplification by shuffling and by iteration	What other sources of uncertainty amplify privacy?	Bounds for shuffled reports and hidden iterates	3, 6, 7, 18, 26, 29, 30, 32, 33
Adaptive composition: filters, odometers and individual accounting	What changes when the privacy parameters are chosen on the fly, or differ per record?	Running sums checked by filters and odometers	2, 21, 29, 31, 33
Choosing an accountant	Which accountant should you use for DP-SGD, and what should you report?	All of the above, side by side	3, 21, 29–36

Read the table from top to bottom as a ladder. An RDP curve is a lossy summary of the trade-off function, a zero-concentrated DP (zCDP) parameter ρ is a lossy summary of the RDP curve, and (ε, δ) is a lossy summary of all three. Numerical accounting keeps the whole privacy loss distribution and loses only discretization error, which it bounds. Amplification (chapters 33 and 35) is a separate axis. It is about which pair of distributions you account for in the first place, and every accountant above must use the pair that matches the sampling scheme the training code actually runs.

The running example

Every chapter that can compute it reports the same number: the ε at $\delta = 10^{-5}$ of DP-SGD with Poisson sampling rate $q = 0.01$, noise multiplier $\sigma = 1$ and up to $T = 10{,}000$ steps. That is about one hundred epochs at a typical setting. A single Gaussian mechanism with $\sigma = 1$ serves as the small example. Watching the same number move from chapter to chapter shows what each tool gains. Choosing an accountant puts all the values side by side.

How to read this part

Order. Read chapters 29 and 30 first: every later chapter uses the RDP curve and its conversion. Chapter 30 ends by reading the curve as a constraint on tests, and chapter 32 develops that trade-off view in full, so read it next. Chapters 33 and 34 form the practical core and should be read in order. Chapter 31 can be read any time after chapter 30, except its section on subsampling, which uses one formula from chapter 33; read it before chapter 36, which uses its conversion. Chapter 35 needs chapter 33. Chapter 36 needs chapters 31 and 33. Chapter 37 closes the part and uses all the others, so read it last.

Structure of a chapter. The chapters have the shape of the earlier parts. Each opens with the question it answers and the running example below, then gives one section per topic, whose definitions, theorems and worked examples are numbered, each theorem followed by its proof. A Code section, a Summary table of the chapter's results and References close it.

Run the code. Each chapter has a script in `code/` that prints every number its text quotes, except values attributed to an external library or paper, and regenerates its figures, for example

```
uv run python code/part04-privacy-accounting/renyi_dp.py
```

None of the scripts uses an accounting library. Each accountant is implemented from its definition in a few dozen lines of NumPy, so you can read exactly what it computes.

Notation

The part uses the notation of the DP foundations part, extended in Table 28.8.

Table 28.8.: Notation used throughout this part, in addition to that of the earlier parts.

Symbol	Meaning
M, $D \sim D'$	A mechanism; neighbouring datasets (add/remove-one unless replace-one is named)
y, $P = M(D)$, $Q = M(D')$	An output; the two output distributions being compared

Symbol	Meaning
$D' = D \cup \{x\}$; FPR, FNR	As in the DP foundations part, under add/remove D' is the dataset that holds the record x and D the one without it. A test of P against Q has type I error FPR, its false positive rate, the rate of saying D' when the data were D, and type II error FNR, its false negative rate, the rate of saying D when they were D'. A guarantee covers both orders of the pair; a chapter that needs one order says so, as the subsampling chapters do for the remove direction, which puts the dataset with the record first
$L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}$	The privacy loss of output y; $L_{D,D'}(Y)$ with $Y \sim M(D)$ is the privacy loss random variable
$\delta(\varepsilon) = E_{e^\varepsilon}(P\|Q)$	The smallest δ for a given ε, as a hockey-stick divergence
D_α, $\varepsilon(\alpha)$	Rényi divergence of order $\alpha > 1$; the RDP curve of a mechanism
ρ	The zCDP parameter
$T(P, Q)$, G_μ, μ	Trade-off function; the Gaussian trade-off function $T(\mathcal{N}(0,1), \mathcal{N}(\mu,1))$; the parameter of Gaussian differential privacy, so a Gaussian mechanism with noise multiplier σ has $\mu = 1/\sigma$
$\Delta_2 f$, σ	ℓ_2 sensitivity; noise multiplier (noise standard deviation per unit of sensitivity)
q, k, T	Sampling rate; number of mechanisms composed; number of steps of one iterative algorithm, such as DP-SGD
h	Grid width of a discretized privacy loss distribution

Two symbols do double duty, and each chapter warns you when it happens. α is the Rényi order throughout this part, but the f-DP literature uses it for the type I error, so the chapters of this part write the errors as false positive and false negative rates, FPR and FNR. q is the sampling rate, so the chapters in which a sampling rate appears write output densities with capital letters, such as $Q(y)$ or $P_q(y)$, never $q(y)$.

What you will be able to do

By the end of this part you should be able to:

- compute the RDP curve of the Gaussian, Laplace and randomized-response mechanisms, compose them, and convert the result to (ε, δ) with both the classic and the improved conversion;
- explain what an RDP curve, a zCDP parameter and a trade-off function each keep and lose about the privacy loss;
- derive privacy amplification by subsampling, and compute the RDP of the Poisson-subsampled Gaussian;
- build a privacy-loss-distribution accountant with pessimistic and optimistic error bounds, and use it to check any other accountant;
- say when shuffling or hiding intermediate iterates gives extra privacy, and when it does not apply;
- run a privacy filter, and compute per-record privacy spend in DP-SGD;
- pick an accountant for a DP-SGD run, state its assumptions, and report ε in a way that someone else can check.

29. Rényi differential privacy

You train a model with 1000 steps of full-batch noisy gradient descent. Each step releases a quantity whose **sensitivity** is 1, meaning that changing one person's record moves it by at most 1, and adds Gaussian noise whose standard deviation is $\sigma = 20$ times that sensitivity. The number σ is the **noise multiplier**. What is the total (ε, δ) differential privacy (DP) guarantee at $\delta = 10^{-5}$? Advanced and optimal composition turns each step into a single pair $(\varepsilon_0, \delta_0)$ and composes the steps at a cost that grows like $\sqrt{k}$ in their number k, which here gives $\varepsilon = 68.01$. The true answer is $\varepsilon = 7.51$, because 1000 Gaussian steps behave like one Gaussian step with noise multiplier $\sigma/\sqrt{1000}$.

The overestimate comes from the first move. A Gaussian step has a whole curve of valid pairs $(\varepsilon_0, \delta_0)$, and collapsing it to one point throws away what composition needs. **Rényi differential privacy (RDP)** keeps a curve instead, one that adds. For each **order** $\alpha > 1$ it records the Rényi divergence between the outputs on neighbouring datasets, an exponential moment of the privacy loss. Curves add under composition, because a moment of a product of independent likelihood ratios is a product of moments, and the summed curve converts to (ε, δ) once, at the end, by a Chernoff bound. For the 1000 steps this gives $\varepsilon = 8.84$, 18% above the exact 7.51.

Two running examples carry the chapter: randomized response, which reports one person's bit truthfully with probability $t = 0.75$, and the 1000 Gaussian steps. Neighbouring datasets $D \sim D'$ differ by adding or removing one record, and for randomized response they are the two values of one person's bit. Table 29.1 lists the symbols.

Table 29.1.: The symbols of this chapter.

Symbol	Meaning
M; $D \sim D'$	a mechanism; two neighbouring datasets
P, Q; p, q	the output distributions $M(D)$ and $M(D')$; their densities or mass functions
$r = p/q$; $L = \ln r$	the likelihood ratio of an output; the privacy loss
α; D_α	the Rényi order, always above 1; the Rényi divergence of that order
$\varepsilon_M(\alpha)$, or $\varepsilon(\alpha)$	the RDP curve of M
σ; b; t	the Gaussian noise multiplier; the Laplace scale; the truth probability of randomized response
k; μ	the number of composed mechanisms; the shift of a Gaussian pair in standard deviations

Symbol	Meaning
$K_M(\lambda)$	the log-moment of the moments accountant, with $\lambda = \alpha - 1$

29.1. Rényi differential privacy

Pure ε-DP bounds the privacy loss at every output, and (ε, δ)-DP lets that bound fail on outputs of total probability δ. Both summarize the distribution of the privacy loss by one point of its upper tail. Before any accounting can start, you need a summary of the whole tail that composition preserves, and this section defines it.

Fix a mechanism M and neighbours $D \sim D'$, and write $P = M(D)$ and $Q = M(D')$ for the two output distributions, with densities or mass functions p and q. The **likelihood ratio** $r = p/q$ says how many times likelier an output is under D than under D'. Its logarithm $L = \ln r$ is the **privacy loss** of Advanced and optimal composition (Definition 21.1). The Rényi divergence D_α of order $\alpha > 1$ (Definition 3.2) is the logarithm of a moment of r, divided by $\alpha - 1$. By Proposition 3.1 that moment can be taken under either distribution, and under P it is an exponential moment of the loss:

$$e^{(\alpha-1)D_\alpha(P\|Q)} = \mathbb{E}_Q\big[r^\alpha\big] = \mathbb{E}_P\big[e^{(\alpha-1)L}\big]. \tag{29.1}$$

The second equality is the change of measure of Lemma 3.2, which turns an average under P into an average under Q with each output reweighted by r. An exponential moment weights large losses more heavily than small ones, and the more so the larger the order. So small orders see the typical loss and large orders the worst case, and a bound at every order describes the whole upper tail.

Definition 29.1 (Rényi differential privacy)**.** Let $\alpha > 1$. A mechanism M is (α, ε)**-RDP** if for all neighbouring datasets $D \sim D'$

$$D_\alpha(M(D) \,\|\, M(D')) \le \varepsilon. \tag{29.2}$$

The **RDP curve** of M records, at each order, the smallest ε that works, which is the divergence of the worst pair of neighbours:

$$\varepsilon_M(\alpha) = \sup_{D \sim D'} D_\alpha(M(D) \,\|\, M(D')). \tag{29.3}$$

The curve inherits the shape of each pair's divergence. Divergences showed that this divergence rises from the Kullback–Leibler (KL) divergence at order 1 to the max-divergence D_∞, the largest privacy loss, as the order grows. The top of that range is pure DP, so pure DP is one point of the curve.

Proposition 29.1 (The two ends of the RDP curve)**.** *Let M be a mechanism, and write $\varepsilon_M(\infty)$ for the supremum over neighbours of $D_\infty(M(D)\|M(D'))$. Then:*

1. *M is ε-DP if and only if $\varepsilon_M(\infty) \le \varepsilon$.*
2. *$\varepsilon_M(\alpha)$ is non-decreasing in α, so an ε-DP mechanism is (α, ε)-RDP at every order.*

3. *At every order, $\varepsilon_M(\alpha)$ is at least the largest KL divergence $D_{\mathrm{KL}}(M(D)\|M(D'))$ over neighbours, and so the limit of the curve as $\alpha \to 1$ is at least that KL divergence too.*

Proof.

1. *Pure DP is the max-divergence.* ε-DP says that $\Pr_P[S] \le e^{\varepsilon} \Pr_Q[S]$ for every event S and every pair of neighbours. By Theorem 3.2, the largest log-ratio $\ln(\Pr_P[S]/\Pr_Q[S])$ over events is $D_\infty(P\|Q)$. So the condition holds for all pairs exactly when every D_∞ is at most ε, which is $\varepsilon_M(\infty) \le \varepsilon$.
2. *Monotonicity.* Each pair's D_α is non-decreasing in α, by the monotonicity of Theorem 3.2. If every function of a family is at least as large at each $\alpha' > \alpha$ as at α, so is the largest of them. Together with step 1, this gives $\varepsilon_M(\alpha) \le \varepsilon_M(\infty) \le \varepsilon$ for an ε-DP mechanism.
3. *The KL end.* The monotonicity of Theorem 3.2 covers all orders, and KL is the order 1, so it gives $D_\alpha \ge D_{\mathrm{KL}}$ for each pair. Taking the supremum over pairs keeps the inequality, and a limit of numbers that are all at least some value is at least that value.

□

The limit at order 1 equals the largest KL for every mechanism of this chapter, though in general the limit and the supremum over pairs need not commute. An RDP guarantee is a curve, not a number, and every point of it is a valid guarantee.

A mechanism whose loss is usually small but occasionally large has a small KL divergence but large high-order divergences, so it is caught at large orders. Orders below 1 exist as divergences but are not used for privacy, because the conversion of Section 29.6 needs the moment of Equation 29.1 with a positive exponent $\alpha - 1$.

Example 29.1 (Randomized response, outcome by outcome)**.** Randomized response (Definition 18.4) reports one person's bit truthfully with probability $t = 0.75$ and flips it otherwise. Table 29.2 lists its outputs when the true bit is 1 (P) and 0 (Q). The likelihood ratio of each output is 3 or 1/3, so the mechanism is ε-DP with $\varepsilon = \ln \frac{t}{1-t} = \ln 3$.

Table 29.2.: Randomized response with truth probability $t = 0.75$. The last column gives the terms of $\mathbb{E}_Q[r^2]$, the moment of Equation 29.1 at order 2.

Output y	$p(y)$ (bit is 1)	$q(y)$ (bit is 0)	$r(y)$	$L(y)$	$p(y)^2/q(y)$
1	0.75	0.25	3	1.0986	2.2500
0	0.25	0.75	1/3	−1.0986	0.0833

At order 2, Equation 29.1 says that e^{D_2} is the sum of $p(y)^2/q(y)$ over the outputs, which is the last column:

$$e^{D_2} = 2.2500 + 0.0833 = 2.3333, \qquad \varepsilon(2) = \ln 2.3333 = 0.8473. \tag{29.4}$$

The same recipe works at any order, and it gives Equation 29.7 of the next section. Swapping the two bits turns the terms into $q(y)^2/p(y)$, which are 0.0833 and 2.2500, the same sum, so both directions of the pair have the same curve. Table 29.3 gives it at a few orders.

Table 29.3.: The RDP curve of randomized response with $t = 0.75$.

α	1 (KL)	1.5	2	5	10	64	∞
$\varepsilon(\alpha)$	0.5493	0.7340	0.8473	1.0267	1.0666	1.0940	1.0986

The curve rises from the KL value to the pure DP level (Figure 29.1). At the bottom is the average loss under P, which is $\ln 3$ with probability 0.75 and $-\ln 3$ with probability 0.25, so it is $0.5 \ln 3 = 0.5493$. At the top is the largest loss, $\ln 3 = 1.0986$, and the chapter's script (Section 29.7) finds 0.5494 at $\alpha = 1.0001$ and 1.0983 at $\alpha = 1000$. For a pure DP mechanism like this one the curve tells you little beyond $\ln 3$. It matters most when D_∞ is large or infinite.

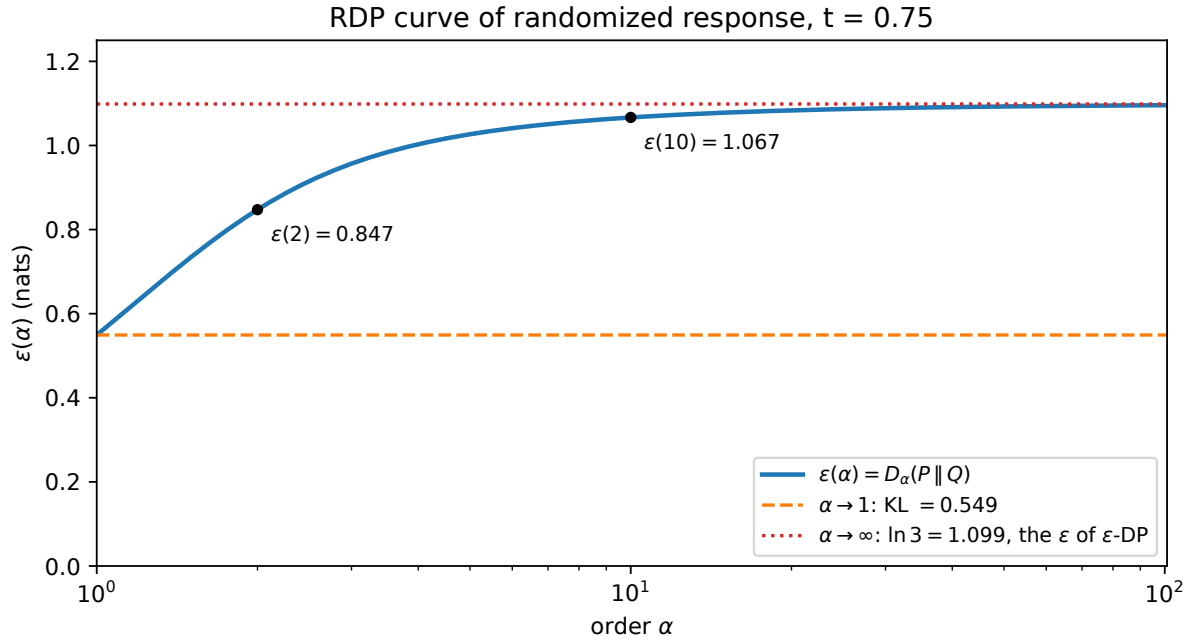


Figure 29.1.: The RDP curve $\varepsilon(\alpha)$ of randomized response with truth probability 0.75, on a log scale in α. It starts at the KL divergence 0.549 as $\alpha \to 1$ and rises to the pure DP level $\ln 3 = 1.099$ as $\alpha \to \infty$.

29.2. RDP curves of the basic mechanisms

To account for a mechanism you need its curve, and this section computes it for the three mechanisms every accountant starts from. A mechanism that adds noise to a query's answer produces, on two neighbours, the same noise distribution centred at two different answers. So its curve comes from the Rényi divergence between the noise and a shifted copy of itself, which Divergences computed for the normal and the Laplace distributions. The only new step is the supremum over neighbours, and for these mechanisms the worst pair is the one whose answers are furthest apart.

The Gaussian mechanism adds $\mathcal{N}(0, \sigma^2 I_d)$ noise to a query f whose ℓ_2 sensitivity $\Delta_2 f$, the largest Euclidean distance between its answers on neighbours, is 1. The Laplace mechanism of Theorem 20.1 adds noise with density $\frac{1}{2b}e^{-|z|/b}$ to a real-valued query whose sensitivity $\Delta_1 f$, the largest $|f(D) - f(D')|$ over neighbours, is 1. The three formulas are Corollaries 3 and 2 and Table 2 of Mironov (2017), and Section B.4 in the appendices derives the Gaussian curve from scratch by completing the square.

Theorem 29.1 (RDP curves of the Gaussian, Laplace and randomized response mechanisms)**.** *Let $\alpha > 1$. The Gaussian mechanism with noise multiplier σ, on a query with $\Delta_2 f = 1$, has the curve*

$$\varepsilon(\alpha) = \frac{\alpha}{2\sigma^2}. \tag{29.5}$$

The Laplace mechanism with scale b, on a real-valued query with $\Delta_1 f = 1$, has the curve

$$\varepsilon(\alpha) = \frac{1}{\alpha - 1} \ln\left(\frac{\alpha}{2\alpha - 1} e^{(\alpha-1)/b} + \frac{\alpha - 1}{2\alpha - 1} e^{-\alpha/b}\right). \tag{29.6}$$

Randomized response with truth probability $t \in (\frac{1}{2}, 1)$ has the curve

$$\varepsilon(\alpha) = \frac{1}{\alpha - 1} \ln\Big(t^\alpha (1-t)^{1-\alpha} + (1-t)^\alpha t^{1-\alpha}\Big). \tag{29.7}$$

Proof. The plan is the same for both noise mechanisms: reduce a pair of neighbours to the noise against a shifted copy, read its divergence off Divergences, and show that the largest shift is the worst. Randomized response needs only the sum.

1. *Gaussian, one pair.* On neighbours the outputs are $\mathcal{N}(f(D), \sigma^2 I_d)$ and $\mathcal{N}(f(D'), \sigma^2 I_d)$, whose d coordinates are independent. Coordinate i is a pair of normals whose means are some distance s_i apart, with divergence $\alpha s_i^2/(2\sigma^2)$ by Proposition 3.3. Rényi divergences add over independent coordinates, by the additivity of Theorem 3.2, so the pair has

$$D_\alpha = \frac{\alpha \sum_i s_i^2}{2\sigma^2} = \frac{\alpha \, \|f(D) - f(D')\|_2^2}{2\sigma^2}. \tag{29.8}$$

2. *Gaussian, worst pair.* The divergence grows with the distance between the answers, which is at most $\Delta_2 f = 1$, so the supremum over neighbours is Equation 29.5.
3. *Laplace, one pair.* Translating the output is a relabelling that changes no divergence, so the pair is $\mathrm{Lap}(0, b)$ against $\mathrm{Lap}(s, b)$, with $s = |f(D) - f(D')| \le 1$. Proposition 3.4 gives its divergence, which is Equation 29.6 with $1/b$ replaced by the shift in scale units, s/b. The reverse pair has the same divergence, because the reflection $y \mapsto s - y$ swaps the two distributions.
4. *Laplace, worst pair.* You need the divergence to grow with the shift. Write $x = s/b$ and let $g(x)$ be the argument of the logarithm, $\frac{\alpha}{2\alpha-1}e^{(\alpha-1)x} + \frac{\alpha-1}{2\alpha-1}e^{-\alpha x}$. Differentiating each exponential gives

$$g'(x) = \frac{\alpha(\alpha - 1)}{2\alpha - 1}(e^{(\alpha-1)x} - e^{-\alpha x}), \tag{29.9}$$

which is positive for $x > 0$, because the first exponent is positive and the second negative. The logarithm and the factor $1/(\alpha - 1)$ keep the divergence increasing, so the worst pair has $s = 1$, and the supremum is Equation 29.6.

5. *Randomized response.* There is only one pair of inputs, the two values of the bit, so no supremum is needed. In the sum of $p^\alpha q^{1-\alpha}$, output 1 contributes $t^\alpha(1-t)^{1-\alpha}$ and output 0 contributes $(1-t)^\alpha t^{1-\alpha}$. Taking the logarithm and dividing by $\alpha - 1$ gives Equation 29.7.

□

Two shapes appear in Figure 29.2, and which one you get depends on whether the privacy loss is bounded. A bounded loss makes the curve level off. For randomized response the loss is $\pm \ln 3$, and for Laplace with answers 0 and 1 it is $(|y-1| - |y|)/b$, which the triangle inequality keeps within $1/b$ of zero. Every D_α is at most the largest loss, so the two curves rise towards $\ln 3$ and $1/b$ and stay below them. The Laplace curve approaches $1/b$ like $\frac{1}{b} - \frac{\ln 2}{\alpha - 1}$: the first term of Equation 29.6 dominates and its coefficient tends to $\frac{1}{2}$, so the logarithm is close to $(\alpha - 1)/b - \ln 2$. At $\alpha = 512$ with $b = 1$, the formula and this approximation both give 0.9986.

An unbounded loss makes the curve grow without bound. The Gaussian privacy loss is a straight line in the output (Lemma 20.3), so it has no maximum, and the Gaussian mechanism is not ε-DP for any ε. It is still RDP at every finite order, which is all that composition and conversion need. A curve that is a straight line through the origin is the mark of a normal privacy loss, and Concentrated DP builds a definition on such lines.

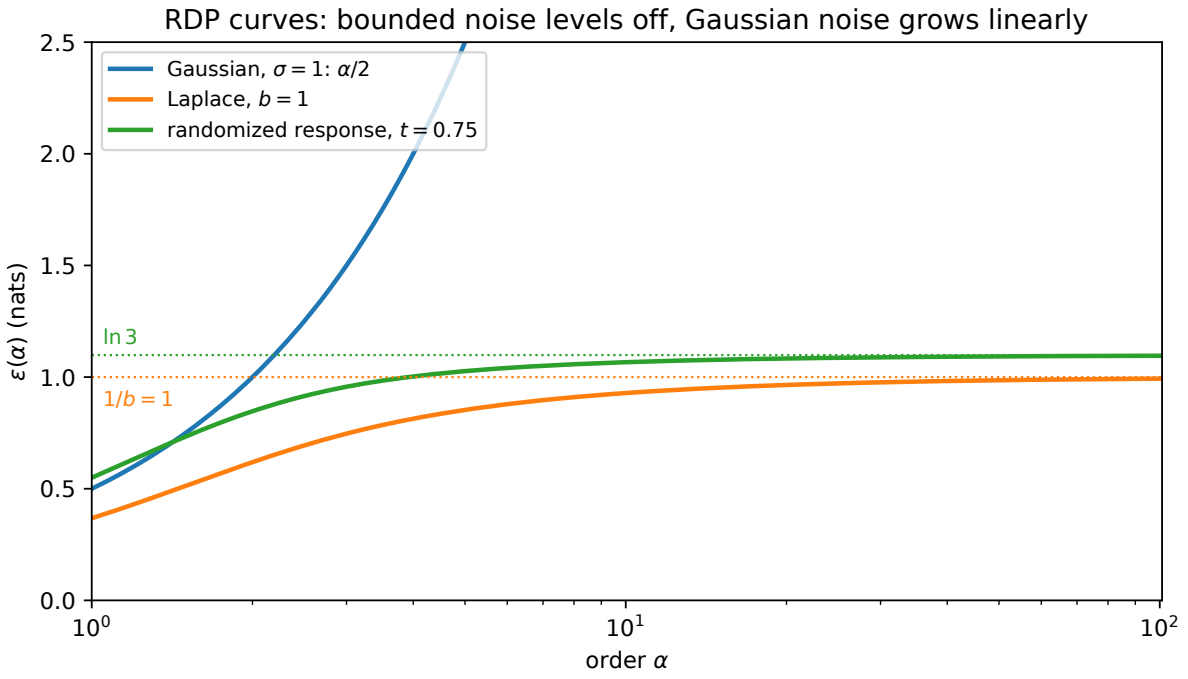


Figure 29.2.: RDP curves of the Gaussian mechanism ($\sigma = 1$), the Laplace mechanism ($b = 1$) and randomized response (truth probability 0.75), all with sensitivity 1, on a log scale in α, so the straight line $\alpha/2$ looks curved. The Laplace and randomized response curves level off at their pure DP levels $1/b$ and $\ln 3$ (dotted). The Gaussian curve $\alpha/2$ grows linearly.

Example 29.2. Table 29.4 gives the three curves at four orders, from the script. The Gaussian row is $\alpha/2$, read off at each order.

Table 29.4.: RDP curves of the three basic mechanisms with sensitivity 1.

Mechanism	$\varepsilon(1.5)$	$\varepsilon(2)$	$\varepsilon(10)$	$\varepsilon(64)$	$\varepsilon(\infty)$
Gaussian, $\sigma = 1$	0.7500	1.0000	5.0000	32.0000	∞
Laplace, $b = 1$	0.5129	0.6191	0.9287	0.9891	1
Randomized response, $t = 0.75$	0.7340	0.8473	1.0666	1.0940	1.0986

You can check the Laplace row by hand at order 2. The coefficients in Equation 29.6 are $\frac{2}{3}$ and $\frac{1}{3}$, the prefactor $1/(\alpha - 1)$ is 1, and with $b = 1$

$$\varepsilon(2) = \ln\left(\tfrac{2}{3}e + \tfrac{1}{3}e^{-2}\right) = 0.6191, \tag{29.10}$$

the same number as in Part 1's Laplace example. As $\alpha \to 1$ the Laplace curve tends to the KL divergence of Proposition 3.4, $1/b + e^{-1/b} - 1 = 0.3679$. Both noise curves agree with direct numerical integration of $p^{\alpha}q^{1-\alpha}$ (Table 29.8).

29.3. Composition

Training a model runs thousands of mechanisms, each chosen after seeing the outputs of the earlier ones, and you need the curve of the whole run. For independent mechanisms the answer is the additivity of Theorem 3.2: the moment of Equation 29.1 splits into a product, and the logarithm turns it into a sum. Adaptive composition is not a product of independent distributions, because the second mechanism depends on the first output. The theorem below shows that the curves still add, as long as each step's curve is bounded whatever the earlier outputs were.

The result is Mironov (2017), Proposition 1. Section B.4 in the appendices derives it from scratch, and Section B.5 compares it with the other accountants on one mechanism.

Theorem 29.2 (Adaptive composition of RDP curves)**.** *Let M_1 have RDP curve ε_1. Let $M_2(\cdot\,; y_1)$ be a mechanism chosen using the first output y_1, and let ε_2 be a curve, fixed in advance, that bounds the RDP curve of $M_2(\cdot\,; y_1)$ for every y_1. The **adaptive composition** M releases both outputs, $M(D) = (y_1, y_2)$, with $y_1 \sim M_1(D)$ and $y_2 \sim M_2(D; y_1)$. Then for every $\alpha > 1$*

$$\varepsilon_M(\alpha) \le \varepsilon_1(\alpha) + \varepsilon_2(\alpha). \tag{29.11}$$

By induction, treating the first $k-1$ steps as one mechanism, k adaptively chosen mechanisms have a curve of at most $\sum_{i=1}^{k} \varepsilon_i(\alpha)$.

Proof. The plan is to factor the joint density, write the moment as a double integral, bound the inner integral uniformly, and recognize the outer one. Fix neighbours $D \sim D'$ and an order $\alpha > 1$. Write p_1, q_1 for the densities of y_1 under D and D', and $p_2(\cdot \mid y_1), q_2(\cdot \mid y_1)$ for those of y_2 given y_1.

1. *Factor the joint densities.* By the definition of the composition, y_1 is drawn first and y_2 is drawn given y_1, so $p(y_1, y_2) = p_1(y_1)\, p_2(y_2 \mid y_1)$, and the same holds for q.
2. *Write the moment as a double integral.* By Equation 29.1, the moment is the integral of $p^\alpha q^{1-\alpha}$. A power of a product is the product of the powers, so the integrand splits into a factor for y_1 and a factor for y_2 given y_1:

$$e^{(\alpha-1)D_\alpha(P\|Q)} = \int p_1^\alpha q_1^{1-\alpha} \left(\int p_2(y_2 \mid y_1)^\alpha q_2(y_2 \mid y_1)^{1-\alpha} \, \mathrm{d}y_2 \right) \mathrm{d}y_1. \tag{29.12}$$

3. *Bound the inner integral.* With y_1 fixed, the inner integral is $e^{(\alpha-1)D_\alpha}$ for the two outputs of $M_2(\cdot\,; y_1)$ on D and D', by Equation 29.1 again. That divergence is at most $\varepsilon_2(\alpha)$, and the exponential is increasing because $\alpha - 1 > 0$. So, whatever y_1 is,

$$\int p_2(y_2 \mid y_1)^\alpha q_2(y_2 \mid y_1)^{1-\alpha} \, \mathrm{d}y_2 \le e^{(\alpha-1)\varepsilon_2(\alpha)}. \tag{29.13}$$

4. *Recognize the outer integral.* Pull that constant out. What is left is $\int p_1^\alpha q_1^{1-\alpha} \, \mathrm{d}y_1$, which by Equation 29.1 is $e^{(\alpha-1)D_\alpha(P_1\|Q_1)}$, at most $e^{(\alpha-1)\varepsilon_1(\alpha)}$. So

$$e^{(\alpha-1)D_\alpha(P\|Q)} \le e^{(\alpha-1)\varepsilon_2(\alpha)}\, e^{(\alpha-1)\varepsilon_1(\alpha)}. \tag{29.14}$$

5. *Take logarithms.* Dividing by $\alpha - 1$ gives $D_\alpha(P\|Q) \le \varepsilon_1(\alpha) + \varepsilon_2(\alpha)$ for this pair. The right side does not depend on the pair, so the supremum over neighbours satisfies the same bound.

□

The bound is exact for independent mechanisms, because then the inner divergence in step 3 is the same for every y_1 and replacing it by its worst case loses nothing. The factorization of step 2 is why composition works for RDP and not for (ε, δ) directly. The smallest δ at a given ε is the hockey-stick divergence $\int (p - e^\varepsilon q)_+$, where $(x)_+ = \max(x, 0)$, and a positive part of a difference does not factorize over the steps the way the moment of step 2 does.

One condition remains. The mechanisms can be chosen adaptively, but each step's curve bound ε_i must be fixed in advance and must not depend on the data. When the budget itself is chosen on the fly, you need the filters and odometers of Adaptive composition.

Example 29.3. To see the steps on numbers, take an adaptive pair of randomized responses. The first step reports the bit with truth probability 0.75. The second reports it again, with truth probability 0.75 if the first output was 1 and 0.6 if it was 0. Table 29.5 lists the four joint outcomes when the true bit is 1 (P) and 0 (Q). Each joint probability is the first step's probability times the second's, as in step 1.

Table 29.5.: Two adaptive randomized responses: the second truth probability depends on the first output. The last column gives the terms of the moment at order 2.

(y_1, y_2)	$p(y_1, y_2)$	$q(y_1, y_2)$	p^2/q
(1, 1)	$0.75 \times 0.75 = 0.5625$	$0.25 \times 0.25 = 0.0625$	5.0625
(1, 0)	$0.75 \times 0.25 = 0.1875$	$0.25 \times 0.75 = 0.1875$	0.1875
(0, 1)	$0.25 \times 0.6 = 0.15$	$0.75 \times 0.4 = 0.3$	0.0750

(y_1, y_2)	$p(y_1, y_2)$	$q(y_1, y_2)$	p^2/q
(0, 0)	$0.25 \times 0.4 = 0.1$	$0.75 \times 0.6 = 0.45$	0.0222

The terms sum to 5.3472, so $D_2 = \ln 5.3472 = 1.6766$. The bound adds the first step's curve to the worst curve the second step can have. At order 2 the second step's curve is 0.8473 on the branch $y_1 = 1$ and 0.1542 on the branch $y_1 = 0$, by Equation 29.7 with $t = 0.75$ and $t = 0.6$. So the bound is $0.8473 + \max(0.8473, 0.1542) = 1.6946$. It holds, and it is loose by $1.6946 - 1.6766 = 0.0180$, because step 3 replaced the inner divergence on the branch $y_1 = 0$ by its worst case. When the second step does not adapt, with truth probability 0.75 on both branches, the joint distribution is a product and $D_2 = 1.6946$ exactly.

For the opening's Gaussian steps with $\sigma = 20$, each step has the curve $\alpha/(2 \cdot 20^2) = \alpha/800$, and k steps have $k\alpha/800$. Setting $k\alpha/800 = \alpha/(2\sigma'^2)$ shows that this is the curve of one Gaussian step with noise multiplier $\sigma' = 20/\sqrt{k}$. At $k = 1000$ the curve is $1.2500\,\alpha$, one Gaussian with $\sigma' = 0.6325$, and the function `compose_rdp`, which adds 1000 copies of the curve on the grid of orders, returns 2.5000 at $\alpha = 2$. Different mechanisms compose just as easily, order by order: a Laplace step with $b = 1$ followed by a Gaussian step with $\sigma = 1$ has $\varepsilon(2) = 0.6191 + 1 = 1.6191$, from Table 29.4.

29.4. Post-processing and group privacy

Two properties of DP should carry over to RDP. Post-processing matters because a released model or its predictions must not leak more than the training run that produced them. Group privacy matters because records are correlated: a family shares a household, and one user contributes many rows. For ε-DP, a group of k records gets $k\varepsilon$ (Theorem 19.2), because the likelihood ratios along a chain of k neighbours multiply. For RDP the answer is weaker. The Rényi divergence has no plain triangle inequality, only one that bounds each link of the chain at a higher order, so the order of the group guarantee shrinks as the group grows.

Post-processing is the data-processing inequality of Theorem 3.5, applied to every pair of neighbours: a map that acts on the output alone can only make the two output distributions harder to tell apart.

Theorem 29.3 (Post-processing)**.** *Let M be a mechanism and g a randomized map applied to its output that does not read the data. Then $\varepsilon_{g \circ M}(\alpha) \le \varepsilon_M(\alpha)$ for every $\alpha > 1$.*

Proof.

1. For each pair of neighbours, $g \circ M(D)$ is $M(D)$ passed through the channel g, the same channel whichever dataset was used. So D_α can only decrease, by Theorem 3.5.
2. The inequality holds for every pair, so it holds for the supremum.

□

Step 1 is where g must ignore the data: a map that reads the data is not a channel, and it could simply print a record.

Group privacy walks from D to D' through a dataset that differs from each in one record, so it needs a bound on the divergence across two steps. The weak triangle inequality gives one, at the price of higher orders on the two legs. Two exponents u and v choose how the cost is split between the legs, and the proof separates the legs with Hölder's inequality.

Lemma 29.1 (Weak triangle inequality (Mironov 2017, Proposition 11))**.** *Let P, R and Q be distributions, let $\alpha > 1$, and let $u, v > 1$ with $\frac{1}{u} + \frac{1}{v} = 1$. Then*

$$D_\alpha(P\|Q) \le \frac{\alpha - 1/u}{\alpha - 1} D_{u\alpha}(P\|R) + D_{v(\alpha-1)+1}(R\|Q). \tag{29.15}$$

The even split $u = v = 2$ doubles the order on the first leg and nearly doubles it on the second:

$$D_\alpha(P\|Q) \le \frac{\alpha - 1/2}{\alpha - 1} D_{2\alpha}(P\|R) + D_{2\alpha-1}(R\|Q). \tag{29.16}$$

Proof. The plan is to write the moment for P against Q as an average under R of a product of two likelihood ratios, separate the two factors with Hölder's inequality, and recognize each factor as a Rényi moment. Write p, p_R and q for the densities of P, R and Q.

1. *Insert the middle distribution.* The exponents of p_R in $(p/p_R)^\alpha (p_R/q)^{\alpha-1} p_R$ add to 0, so the product is $p^\alpha q^{1-\alpha}$. With Equation 29.1 this gives
$$e^{(\alpha-1)D_\alpha(P\|Q)} = \mathbb{E}_R\Big[\Big(\frac{p}{p_R}\Big)^\alpha \Big(\frac{p_R}{q}\Big)^{\alpha-1}\Big]. \tag{29.17}$$
2. *Separate the two ratios.* Hölder's inequality bounds the average of a product of two non-negative quantities X and Y by a product of their higher moments, $\mathbb{E}_R[XY] \le (\mathbb{E}_R X^u)^{1/u} (\mathbb{E}_R Y^v)^{1/v}$, and with $u = v = 2$ it is the Cauchy–Schwarz inequality. Apply it with $X = (p/p_R)^\alpha$ and $Y = (p_R/q)^{\alpha-1}$.
3. *The first factor.* $\mathbb{E}_R[X^u]$ is the moment of the ratio p/p_R at order $u\alpha$ under R. By Equation 29.1 at order $u\alpha$, it equals $e^{(u\alpha-1)D_{u\alpha}(P\|R)}$.
4. *The second factor.* $\mathbb{E}_R[Y^v]$ is an average under R, and the change of measure from R to Q multiplies the integrand by p_R/q. So it equals $\mathbb{E}_Q\big[(p_R/q)^{v(\alpha-1)+1}\big]$, which by Equation 29.1 is $e^{v(\alpha-1)D_{v(\alpha-1)+1}(R\|Q)}$.
5. *Combine.* Raising the factors to the powers $1/u$ and $1/v$ and taking logarithms gives
$$(\alpha-1)D_\alpha(P\|Q) \le \frac{u\alpha - 1}{u} D_{u\alpha}(P\|R) + (\alpha-1) D_{v(\alpha-1)+1}(R\|Q). \tag{29.18}$$
Dividing by $\alpha - 1$ gives Equation 29.15, since $\frac{u\alpha-1}{u} = \alpha - \frac{1}{u}$. With $u = v = 2$ the second order is $2\alpha - 1$, which gives Equation 29.16.

□

Applied to a group of two records, the lemma halves the order and costs a constant factor in ε. Repeating it handles groups of 2^c records, where c is the number of doublings.

Theorem 29.4 (Group privacy in RDP (Mironov 2017, Proposition 2))**.** *Let M be (α, ε)-RDP, let $c \geq 1$ be an integer, and suppose $\alpha \geq 2^{c+1}$. Then for any two datasets that differ in 2^c records, M satisfies the RDP bound of order $\alpha/2^c$ with ε replaced by $3^c\varepsilon$.*

Proof. The plan is to prove the case of two records by walking through an intermediate dataset, and then to repeat it for larger groups.

1. *Two records, the walk.* Let D and D' differ in two records, and let D_1 be the dataset in between, a neighbour of both. Put $P = M(D)$, $R = M(D_1)$ and $Q = M(D')$, and let $\beta = \alpha/2 \geq 2$ be the order of the group guarantee. Applying Equation 29.16 at order β turns its orders into $2\beta = \alpha$ and $2\beta - 1 = \alpha - 1$:

$$D_\beta(P\|Q) \leq \frac{\beta - 1/2}{\beta - 1} D_\alpha(P\|R) + D_{\alpha-1}(R\|Q). \tag{29.19}$$

2. *Two records, the legs.* Both legs are pairs of neighbours, so $D_\alpha(P\|R) \leq \varepsilon$. By the monotonicity of Proposition 29.1, $D_{\alpha-1}(R\|Q) \leq D_\alpha(R\|Q) \leq \varepsilon$.
3. *Two records, the factor.* The factor $\frac{\beta-1/2}{\beta-1} = 1 + \frac{1/2}{\beta-1}$ decreases in β and equals $\frac{3}{2}$ at $\beta = 2$, so it is at most $\frac{3}{2}$. Hence
$$D_\beta(P\|Q) \leq \tfrac{3}{2}\varepsilon + \varepsilon = \tfrac{5}{2}\varepsilon \leq 3\varepsilon. \tag{29.20}$$
Mironov states the bound with the constant 3, while the argument gives $\frac{5}{2}$.
4. *Larger groups.* A group of 2^c records is two groups of 2^{c-1} records, so each round of steps 1 to 3 halves the order and multiplies ε by 3. After c rounds the order is $\alpha/2^c$ and the bound is $3^c\varepsilon$, and the condition $\alpha \geq 2^{c+1}$ keeps the order of the last round at least 2.

□

Each doubling of the group costs half the order as well as a factor in ε. The reason is step 2 of the lemma's proof: the divergence across two steps involves a product of two likelihood ratios, and controlling the average of a product needs higher moments of each factor. On the Gaussian the exact answer is known: a group of 2^c records shifts a query of sensitivity 1 by up to 2^c, so the exact group curve is $\alpha(2^c)^2/(2\sigma^2)$, which Table 29.6 compares with the bounds at $\sigma = 1$. For two records the exact value at order β is $4\beta/(2\sigma^2) = 2\varepsilon(2\beta)$ and the theorem gives $3\varepsilon(2\beta)$, a factor 1.5, and the factor compounds to 1.5^c for 2^c records.

So use the mechanism's own curve when you can: for the Gaussian it is the curve with sensitivity 2^c. For a mechanism that is already ε-DP, the pure DP group bound $k\varepsilon$ holds at every order by Proposition 29.1, and it can beat the RDP route.

Table 29.6.: Group privacy for the Gaussian mechanism with $\sigma = 1$: the exact curve on inputs 2^c records apart against the two bounds.

Group size 2^c, order	Exact	Weak triangle, iterated	Proposition 2: $3^c\varepsilon(2^c\beta)$	Proposition 2 / exact
2, $\beta = 2$	4.0000	4.5000	6.0000	1.5000
2, $\beta = 4$	8.0000	8.1667	12.0000	1.5000
2, $\beta = 10$	20.0000	20.0556	30.0000	1.5000
4, $\beta = 2$	16.0000	18.5000	36.0000	2.2500

Group size 2^c, order	Exact	Weak triangle, iterated	Proposition 2: $3^c\varepsilon(2^c\beta)$	Proposition 2 / exact
8, $\beta = 2$	64.0000	74.5000	216.0000	3.3750

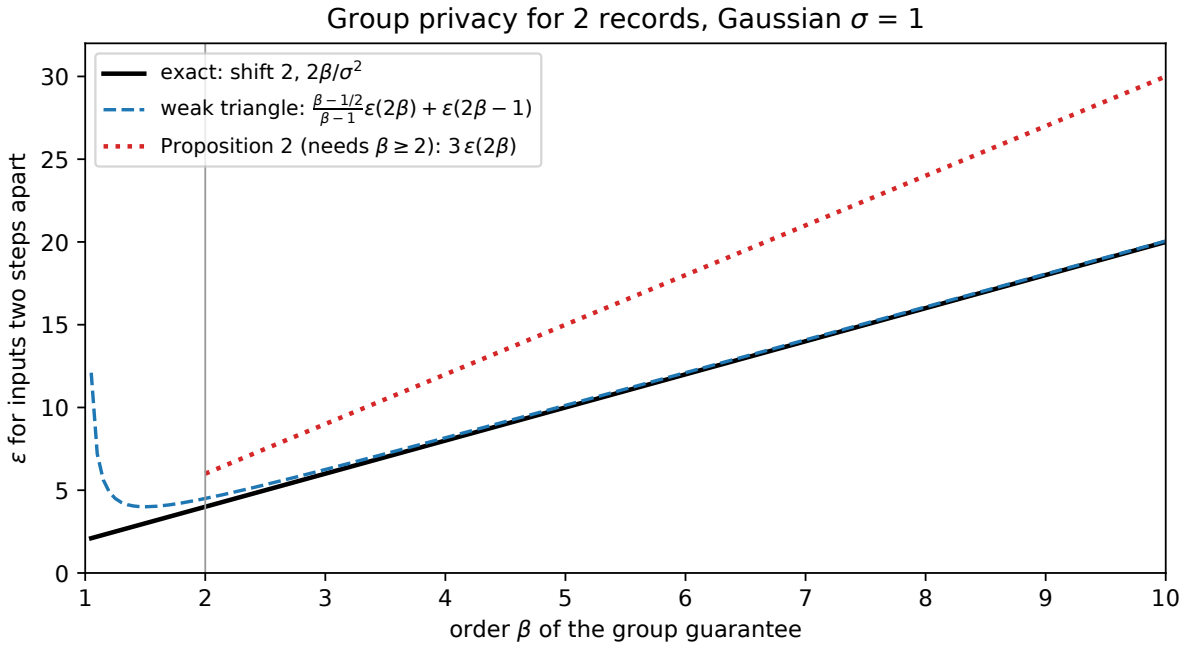


Figure 29.3.: RDP of the Gaussian mechanism ($\sigma = 1$) on inputs two records apart, as a function of the order β of the group guarantee. Black: the exact curve 2β. Dashed: the weak triangle inequality with $u = v = 2$. Dotted: Proposition 2's bound $3\varepsilon(2\beta)$, drawn from $\beta = 2$, where its condition $\alpha = 2\beta \geq 4$ starts to hold.

Example 29.4. Post-processing first. Pass the output of randomized response with $t = 0.75$ through a channel that flips it with probability 0.2. When the bit is 1, the final output is 1 if the first output was 1 and kept, or 0 and flipped, which has probability $0.75 \times 0.8 + 0.25 \times 0.2 = 0.65$. When the bit is 0, the same event has probability $0.25 \times 0.8 + 0.75 \times 0.2 = 0.35$. So the two output distributions are now closer, and D_2 drops from 0.8473 to $\ln(0.65^2/0.35 + 0.35^2/0.65) = 0.3333$, and D_∞ drops from 1.0986 to $\ln(0.65/0.35) = 0.6190$.

For group privacy, check the first row of Table 29.6 by hand, two records at $\beta = 2$ with $\sigma = 1$. The exact value is $2 \cdot 2^2/2 = 4$. The weak triangle inequality gives $\frac{1.5}{1}\varepsilon(4) + \varepsilon(3) = 1.5 \times 2 + 1.5 = 4.5$, and the theorem gives $3\varepsilon(4) = 3 \times 2 = 6$, which is $6/4 = 1.5$ times the exact value. Figure 29.3 draws the whole two-record case. The weak-triangle bound hugs the exact line 2β for $\beta \geq 3$ and blows up near $\beta = 1$, where its factor $\frac{\beta - 1/2}{\beta - 1}$ does. For the Laplace mechanism with $b = 1$ at order 2, two records apart, the exact value is 1.5958, the weak triangle inequality gives 1.9674 and the theorem 2.4411, while the pure DP group bound $2/b = 2$ beats the theorem.

29.5. The moments accountant

Abadi et al. (2016) introduced the **moments accountant** to train neural networks with DP, before RDP existed, and many libraries still describe their accounting in its terms. For each $\lambda > 0$, in practice the integers up to 32, it tracks the logarithm of the moment generating function of the privacy loss, adds these numbers over the training steps, and converts the total to (ε, δ) with a tail bound. This section shows that it tracks the same object as the RDP curve, so everything in this chapter applies to it.

Definition 29.2 (Log-moment of a mechanism)**.** For $\lambda > 0$, the **log-moment** of a mechanism M is the cumulant generating function of the privacy loss of the worst pair of neighbours,

$$K_M(\lambda) = \sup_{D \sim D'} \ln \mathbb{E}_{Y \sim M(D)}\big[e^{\lambda L(Y)}\big]. \tag{29.21}$$

Abadi et al. write it $\alpha_M(\lambda)$, which clashes with the Rényi order.

The moment inside the logarithm is the one in Equation 29.1 with $\alpha - 1 = \lambda$. So the log-moment is the RDP curve one order up, multiplied by λ.

Proposition 29.2 (The moments accountant is RDP)**.** *For every mechanism M and every $\lambda > 0$,*

$$K_M(\lambda) = \lambda\, \varepsilon_M(\lambda + 1), \qquad \textit{equivalently} \qquad \varepsilon_M(\alpha) = \frac{K_M(\alpha - 1)}{\alpha - 1}. \tag{29.22}$$

Proof.

1. *The moment is a Rényi moment.* Fix neighbours. By Equation 29.1 at the order $\alpha = \lambda + 1$, $\mathbb{E}_P[e^{\lambda L}] = e^{\lambda D_\alpha(P\|Q)}$, and its logarithm is $\lambda D_\alpha(P\|Q)$.
2. *Take the supremum.* The factor $\lambda > 0$ does not change which pair of neighbours is worst, so the supremum of the left side is $K_M(\lambda)$ and that of the right side is $\lambda\, \varepsilon_M(\alpha) = \lambda\, \varepsilon_M(\lambda + 1)$.

□

So the two parts of Abadi et al.'s Theorem 2 are results of this chapter. Composability, $K_M(\lambda) \le \sum_i K_{M_i}(\lambda)$, is Equation 29.11 at order $\lambda + 1$ multiplied by λ. The tail bound says that M is (ε, δ)-DP with $\delta = \min_\lambda \exp\big(K_M(\lambda) - \lambda\varepsilon\big)$. It is the conversion proved in the next section, Section 29.6, read in the other direction, fixing ε and solving for δ. So the two accountants give the same numbers with different bookkeeping.

The identity itself is simple, so what was new in 2016 was a bound on $K_M(\lambda)$ for the *subsampled* Gaussian step of private stochastic gradient descent (SGD), called DP-SGD, in which each step uses a random fraction of the data. Amplification by subsampling derives it in RDP form.

Example 29.5. For randomized response with $t = 0.75$ at $\lambda = 1$, the moment $\mathbb{E}_P[e^L] = 0.75 \times 3 + 0.25 \times \frac{1}{3}$ has the same two terms, 2.2500 and 0.0833, as the last column of Table 29.2, so its logarithm is $1 \cdot \varepsilon(2) = 0.8473$. At $\lambda = 2$, the direct sum and $2\varepsilon(3)$ both give 1.9136.

For the Gaussian pair $P = \mathcal{N}(0,1)$ and $Q = \mathcal{N}(\mu,1)$, the privacy loss under P is normal with mean $\mu^2/2$ and variance μ^2 (Lemma 20.3). A normal variable with mean m and variance s^2 has the cumulant generating function $m\lambda + s^2\lambda^2/2$, so

$$K(\lambda) = \lambda\frac{\mu^2}{2} + \frac{\lambda^2\mu^2}{2} = \lambda\cdot\frac{(\lambda+1)\mu^2}{2} = \lambda\,\varepsilon(\lambda+1), \tag{29.23}$$

with $\varepsilon(\alpha) = \alpha\mu^2/2$, as Equation 29.22 says. At $\mu = 1$ the formula gives $K(1) = \frac{1}{2} + \frac{1}{2} = 1.0000$ and $K(2) = 1 + 2 = 3.0000$. Averaging $e^{\lambda L}$ over a million draws gives 1.0000 and 3.0003.

29.6. From RDP to (ε, δ)

After composing, you hold a curve, but what you report is an (ε,δ) pair, because that guarantee has a direct meaning for every event. The conversion rests on one observation. Outputs whose privacy loss is at most some threshold c already satisfy the e^c bound, so only outputs with a larger loss can break it, and their total probability can be charged to δ. That probability is a tail of the privacy loss. The Chernoff bound controls a tail with an exponential moment, which is exactly what each point of the RDP curve provides.

The tool is the Chernoff bound of Theorem 2.3 in Concentration inequalities. For any random variable X, any level c and any $\lambda > 0$, it says that $\Pr[X \ge c]$ is at most $e^{-\lambda c}\,\mathbb{E}[e^{\lambda X}]$. The converted ε is then the RDP value plus a tail term, which pays for δ and shrinks as the order grows. This is Mironov (2017), Proposition 3, and Section D.3 in the appendices derives it from scratch.

Theorem 29.5 (From RDP to (ε, δ)-DP)**.** *Let M be $(\alpha,\varepsilon(\alpha))$-RDP for some order $\alpha > 1$. Then for every $0 < \delta < 1$, M is $(\varepsilon_\delta,\delta)$-DP with*

$$\varepsilon_\delta = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha-1}. \tag{29.24}$$

If you hold the whole curve, the best order gives $\varepsilon_\delta = \min_\alpha\big[\varepsilon(\alpha) + \ln(1/\delta)/(\alpha-1)\big]$.

Proof. The plan is to split every event into outputs with small loss, which satisfy the e^c bound, and outputs with large loss, whose probability is a tail, and then to bound the tail with Chernoff. Fix neighbours with privacy loss L, and a threshold c to be chosen.

1. *Split the event.* For any event S, $\Pr_P[S] = \Pr_P[S\cap\{L\le c\}] + \Pr_P[S\cap\{L>c\}]$.
2. *Small losses.* On $\{L \le c\}$ the definition of L gives $p \le e^c q$. Integrating over $S\cap\{L\le c\}$, the first term is at most $e^c\Pr_Q[S]$.
3. *Large losses.* The set $S\cap\{L>c\}$ lies inside $\{L>c\}$, so the second term is at most $\Pr_P[L>c]$.
4. *Chernoff on the loss.* Apply Theorem 2.3 to L under P with $\lambda = \alpha - 1$:

$$\Pr_P[L > c] \le e^{-(\alpha-1)c}\,\mathbb{E}_P\big[e^{(\alpha-1)L}\big]. \tag{29.25}$$

   By Equation 29.1 the average is $e^{(\alpha-1)D_\alpha(P\|Q)}$, at most $e^{(\alpha-1)\varepsilon(\alpha)}$, so

$$\Pr_P[L > c] \le e^{(\alpha-1)(\varepsilon(\alpha)-c)}. \tag{29.26}$$

5. *Choose the threshold.* Setting $(\alpha-1)(\varepsilon(\alpha)-c) = \ln\delta$ gives $c = \varepsilon(\alpha)+\ln(1/\delta)/(\alpha-1)$. Then $\Pr_P[S] \le e^c \Pr_Q[S] + \delta$ for every event and every pair of neighbours, which is (c,δ)-DP.

□

For the Gaussian curve the best order has a closed form, because the curve grows with α while the tail term falls, and the minimum is where they balance.

Corollary 29.1 (The best order for a Gaussian curve)**.** *For the Gaussian curve $\alpha/(2\sigma^2)$ and $0 < \delta < 1$, the right side of Equation 29.24, over all real orders $\alpha > 1$, is smallest at*

$$\alpha^* = 1 + \sigma\sqrt{2\ln(1/\delta)}, \qquad \varepsilon_\delta = \frac{1}{2\sigma^2} + \frac{\sqrt{2\ln(1/\delta)}}{\sigma}. \tag{29.27}$$

Proof.

1. *Substitute.* Write $a = \alpha - 1 > 0$ and $\ell = \ln(1/\delta)$. The Gaussian curve is $(a+1)/(2\sigma^2)$, so Equation 29.24 becomes $\varepsilon_\delta(a) = \frac{a+1}{2\sigma^2} + \frac{\ell}{a}$.
2. *Set the derivative to zero.* The derivative $\frac{1}{2\sigma^2} - \frac{\ell}{a^2}$ vanishes at $a = \sigma\sqrt{2\ell}$. The function is a line plus ℓ/a, whose second derivative $2\ell/a^3$ is positive, so this is the minimum.
3. *Evaluate.* At that a, both $\frac{a}{2\sigma^2}$ and $\frac{\ell}{a}$ equal $\frac{\sqrt{2\ell}}{2\sigma}$, so they add to $\sqrt{2\ell}/\sigma$. With the constant $\frac{1}{2\sigma^2}$, this is Equation 29.27.

□

The conversion gives something away in two places. Step 3 of the conversion's proof charges the whole tail $\Pr_P[L > c]$ to δ. The exact smallest δ at level c is the average $\mathbb{E}_P[(1 - e^{c-L})_+]$ (Lemma 21.1), which charges an output with loss above c only the fraction $1 - e^{c-L}$ of its probability. Step 4 then bounds the tail with a Chernoff bound, which overestimates it. From RDP to (ε, δ) and to tests removes much of both losses with sharper conversions.

The order trades two terms against each other. Small orders make the curve small but the tail term $\ln(1/\delta)/(\alpha-1)$ large, and large orders do the opposite (Figure 29.4, left). A steeper composed curve makes large orders more expensive, so at $\delta = 10^{-5}$ one Gaussian step with $\sigma = 20$ wants $\alpha \approx 97$ while 1000 such steps want $\alpha \approx 4$. Because the best order ranges this widely, this part optimizes over one fixed grid that spans the range: 99 orders from 1.1 to 10.9 in steps of 0.1, the 52 integers from 12 to 63, and 128, 256 and 512, which is 154 orders:

```
ORDERS = [1 + x / 10 for x in range(1, 100)] + list(range(12, 64)) + [128, 256, 512]
```

Example 29.6. Take one Gaussian step with $\sigma = 1$ at $\delta = 10^{-5}$. Here $\ln(1/\delta) = 11.5129$ and $\sqrt{2\ln(1/\delta)} = 4.7985$. The curve is $\alpha/2$, and Equation 29.24 adds $11.5129/(\alpha-1)$. At $\alpha = 2$ the sum is $1.0000 + 11.5129 = 12.5129$, dominated by the tail term, and at $\alpha = 20$ it is $10.0000 + 0.6059 = 10.6059$, dominated by the curve. The grid's best order is 5.8, next to the continuous optimum $\alpha^* = 1 + 4.7985 = 5.7985$, and both give $\varepsilon = 5.2985$. The exact ε comes from the privacy profile of Theorem 20.3 with $\mu = 1/\sigma = 1$. Solving $\delta_\mu(\varepsilon) = 10^{-5}$ by bisection gives 4.3772, so the conversion is 21% too large ($5.2985/4.3772 = 1.21$). Read the other way, the

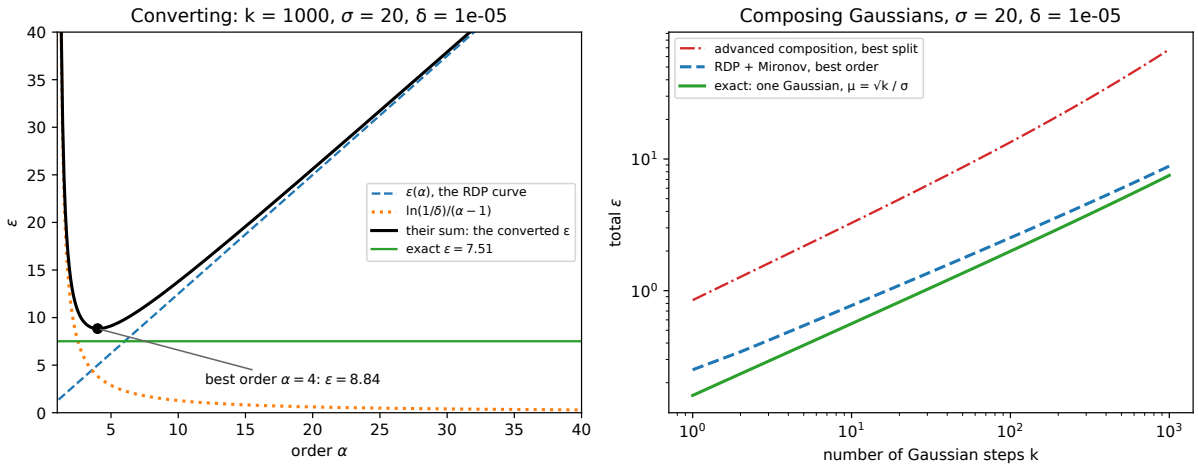


Figure 29.4.: Left: converting the curve of 1000 Gaussian steps with $\sigma = 20$ at $\delta = 10^{-5}$. The converted ε (black) is the sum of the RDP curve (dashed) and the tail term $\ln(1/\delta)/(\alpha - 1)$ (dotted). Its minimum over the grid, 8.84 at $\alpha = 4$, sits above the exact value 7.51 (green). Right: total ε against the number of steps k on log axes, for advanced composition, RDP with Mironov's conversion, and the exact value.

mechanism at $\varepsilon = 5.2985$ has $\delta = 1.300 \times 10^{-7}$, which is 76.9 times smaller than the conversion promises.

Now the opening's problem, k Gaussian steps with $\sigma = 20$ at $\delta = 10^{-5}$. Table 29.7 compares four ways to account for them. The exact value is available because k steps are one Gaussian with $\mu = \sqrt{k}/\sigma$. The two routes of Part 3 turn each step into one point $(\varepsilon_0, \delta_0)$ of its exact curve and compose the points. Basic composition (Theorem 19.3) spends δ/k per step and reports $k\varepsilon_0$. Advanced composition in its sharp form (Equation 21.17) spends a fraction s of δ on the k per-step slacks and the rest on the tail of the summed loss. The script picks the best s on the grid Part 3 uses, and at every k it lands on $s = 10^{-1/4} \approx 0.56$.

Table 29.7.: Total ε at $\delta = 10^{-5}$ for k Gaussian steps with $\sigma = 20$, four ways.

k	Basic	Advanced	RDP + Mironov (best order)	Exact	RDP / exact	Advanced / exact
1	0.160	0.847	0.251 (128)	0.160	1.566	5.29
10	1.892	3.270	0.771 (31)	0.561	1.374	5.83
100	21.510	13.423	2.524 (10.6)	1.993	1.267	6.73
1000	238.578	68.008	8.838 (4)	7.511	1.177	9.05

Advanced composition is 5 to 9 times the exact value, and it gets looser as k grows, because each step's curve was collapsed to one point before composing. At $k = 1$ it is even worse than basic composition, because it pays for the tail of a sum that has only one term. RDP is 18% to 57% above exact, and it gets tighter as k grows. Only at $k = 1$ does the grid cost anything, 0.251 against 0.241 at the continuous optimum 97.0, because the grid jumps from 63 to 128. The remaining gap is the conversion's slack, which depends on the composed $\mu = \sqrt{k}/\sigma$. At the

continuous optimum, Mironov's conversion is 1.07 times exact at $\mu = 10$, 1.21 at $\mu = 1$, 1.42 at $\mu = 0.1$ and 1.76 at $\mu = 0.01$, so the most private composed mechanisms, with small μ, pay the most.

The running example of Part 4's later chapters runs 10,000 steps of DP-SGD with $\sigma = 1$. If each step used the whole dataset, even the exact guarantee at $\delta = 10^{-5}$ would be $\varepsilon = 5425.5$, no guarantee at all, and RDP on `ORDERS` gives 5615.1 at the lowest order 1.1. DP-SGD is private only because each step touches a random 1% of the data, and Amplification by subsampling computes the curve of such a step.

29.7. Code

The script code/renyi_dp.py computes every number in this chapter and draws every figure. It checks the curves against numerical integration, composes them with `compose_rdp` on the grid `ORDERS`, converts with `rdp_to_dp_mironov`, and compares with the exact Gaussian privacy curve, inverted by bisection. Later chapters import these functions. From the repository root:

```
uv run python code/part04-privacy-accounting/renyi_dp.py
```

Table 29.8.: The script's output with the default seed.

Experiment	Result
Randomized response, $t = 0.75$	$\varepsilon(2) = 0.8473$; KL 0.5493; $\varepsilon(\infty) = 1.0986$; curve in Table 29.3
Laplace curve against numerical integration	Largest difference 1.3×10^{-8} over $b \in \{0.5, 1, 4\}$ and four orders
Gaussian curve against numerical integration, $\sigma = 1$	1.0000 at $\alpha = 2$; 5.0000 at $\alpha = 10$
Adaptive randomized response, order 2	Exact 1.6766; bound 1.6946
Group privacy, Gaussian, two records, order 2	Exact 4.0000; weak triangle 4.5000; Proposition 2: 6.0000
Moments accountant, Gaussian $\mu = 1$	$K(1) = 1.0000$, $K(2) = 3.0000$; Monte Carlo 1.0000 and 3.0003
One Gaussian, $\sigma = 1$, $\delta = 10^{-5}$	RDP 5.2985 at $\alpha = 5.8$; exact 4.3772
1000 Gaussians, $\sigma = 20$, $\delta = 10^{-5}$	Advanced 68.01; RDP 8.84 at $\alpha = 4$; exact 7.51

tests/test_renyi_dp.py checks that the output still matches.

29.8. Summary

Table 29.9.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 29.1	(α, ε)-RDP: $D_\alpha(M(D)\|M(D')) \le \varepsilon$ for all $D \sim D'$; the curve $\varepsilon_M(\alpha)$ is the worst pair's divergence	Every accountant in this part stores the curve on the grid `ORDERS`, from 1.1 to 512
Proposition 29.1	ε-DP iff $\varepsilon_M(\infty) \le \varepsilon$; the curve is non-decreasing; it is at least the largest KL	Pure DP is the point $\alpha = \infty$, so an ε-DP mechanism is RDP at every order
Theorem 29.1	Gaussian $\alpha/(2\sigma^2)$; Laplace Equation 29.6; randomized response Equation 29.7; all exact	The inputs to every accountant; the Gaussian curve is the base of DP-SGD accounting
Theorem 29.2	$\varepsilon_M(\alpha) \le \sum_i \varepsilon_i(\alpha)$, adaptively, with equality for products	Compose in RDP and convert once at the end; k Gaussian steps with noise multiplier σ are one with $\sigma/\sqrt{k}$
Theorem 29.3	$\varepsilon_{g \circ M}(\alpha) \le \varepsilon_M(\alpha)$ for g that ignores the data	Releasing models and predictions
Lemma 29.1	$D_\alpha(P\|Q) \le \frac{\alpha-1/u}{\alpha-1} D_{u\alpha}(P\|R) + D_{v(\alpha-1)+1}(R\|Q)$	Equality for Gaussians at the best u
Theorem 29.4	2^c records: order $\alpha/2^c$ with $3^c\varepsilon$, if $\alpha \ge 2^{c+1}$	Loose by 1.5 per doubling on the Gaussian; for the Gaussian, use the curve with the group's sensitivity
Proposition 29.2	$K_M(\lambda) = \lambda\, \varepsilon_M(\lambda + 1)$ (Definition 29.2)	Reading Abadi et al. (2016) and the libraries built on the moments accountant
Theorem 29.5	$(\varepsilon(\alpha) + \ln(1/\delta)/(\alpha - 1), \delta)$-DP, at the best order	18–57% above exact in Table 29.7 and 1.76 times exact at composed $\mu = 0.01$; for reporting, use the next chapter or a numerical accountant
Corollary 29.1	$\alpha^* = 1 + \sigma\sqrt{2\ln(1/\delta)}$, $\varepsilon_\delta = \frac{1}{2\sigma^2} + \frac{\sqrt{2\ln(1/\delta)}}{\sigma}$	The best order shrinks as the composed curve grows; keep a grid from just above 1 to a few hundred

29.9. References

- Mironov, "Rényi Differential Privacy" (IEEE Computer Security Foundations Symposium, CSF 2017; arXiv:1702.07476). The source of this chapter: the definition (Definition 4), adaptive composition (Proposition 1), weak group privacy (Proposition 2), the conversion (Proposition 3), the curves of randomized response, Laplace and Gaussian (Table 2, Corollaries 2 and 3) and the weak triangle inequality (Proposition 11, Corollary 4). Short and readable.
- Abadi, Chu, Goodfellow, McMahan, Mironov, Talwar & Zhang, "Deep Learning with Differential Privacy" (CCS 2016). DP-SGD and the moments accountant; Theorem 2 is composability and the tail bound. Read it for the subsampled Gaussian bound and the experiments.
- van Erven & Harremoës, "Rényi Divergence and Kullback–Leibler Divergence" (IEEE Transactions on Information Theory, 2014). The properties of D_α that RDP inherits, proved for general measures.
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising" (ICML 2018). The exact privacy curve of the Gaussian mechanism used here as the ground truth (Theorem 8).
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 3. Basic composition (Theorem 3.16) and advanced composition (Theorem 3.20), the baseline that RDP improves.
- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (arXiv 2019). The curves that make DP-SGD accounting work, and the reference implementation behind most RDP accountants; used in Amplification by subsampling.

30. From Rényi DP to (ε, δ) and to tests

Privacy accountants track Rényi differential privacy (RDP), because it composes by adding curves (the previous chapter). Take two neighbouring datasets $D \sim D'$ that differ by adding or removing your record, with D' the one that holds it. A mechanism M then produces two output distributions, $P = M(D)$ and $Q = M(D')$. For each **order** $\alpha > 1$ the Rényi divergence $D_\alpha(P\|Q)$ measures how far apart they are, and the **RDP curve** $\varepsilon(\alpha)$ bounds it over all neighbouring pairs. But nobody reports the curve. A paper, a model card or a regulator asks for (ε, δ)-differential privacy (DP), or for a statement about attacks: how much better than guessing can a test tell whether your record was used? So every RDP analysis ends with a conversion, and the conversion decides the number you publish. This chapter asks how much the standard conversion loses and where, which (ε, δ) a curve supports at best and at which order, and what a curve says about tests.

One idea organizes the answers. The **privacy loss** of an output y is $L(y) = \ln(P(y)/Q(y))$, the logarithm of how much more likely y is under D than under D'. The first section recalls two facts about it. The smallest δ at a given ε is an average of a function of the loss, while the RDP curve records only a few other averages of it, the values of its moment generating function (MGF). So a conversion asks for the largest δ consistent with those few averages. The classic answer, Mironov's conversion, wastes much of what the curve says, and a sharper one recovers almost all of it. The last section shows that what remains, 5 to 10% of ε for the Gaussian mechanism, belongs to the curve itself and not to any formula.

The running example is the Gaussian mechanism on a query of sensitivity 1, with noise of standard deviation σ (the noise multiplier), run k times. After dividing by σ, each output shifts by $1/\sigma$ between the two datasets, so the k outputs differ by a vector of length $\mu = \sqrt{k}/\sigma$. Only the component along that vector carries evidence, so the composed mechanism is equivalent to the pair $\mathcal{N}(0, 1)$ and $\mathcal{N}(\mu, 1)$ (Theorem 20.3), and its RDP curve is $\varepsilon(\alpha) = \alpha\mu^2/2$, the sum of k single-step curves $\alpha/(2\sigma^2)$ (Rényi DP). At $\mu = 1$, a single release at $\sigma = 1$, the exact answer at $\delta = 10^{-5}$ is $\varepsilon = 4.377$ (Theorem 20.3), while Mironov's conversion gives 5.299 and the improved one 4.729. Every number uses $\delta = 10^{-5}$ and the part's grid of 154 orders: 1.1 to 10.9 in steps of 0.1, every integer from 12 to 63, then 128, 256 and 512.

30.1. Where Mironov's conversion loses

Rényi DP derived Mironov's conversion, the one in the original moments accountant and in much older code. Before you can improve it, you need to know where it loses. This section rewrites its proof as two inequalities on the privacy loss, so that the cost of each can be measured on the Gaussian. Section D.3 in the appendices derives the same conversion from scratch, with the Gaussian's best order in closed form.

Two results from earlier chapters turn a conversion into a statement about the loss. The first says what δ is. For one pair of datasets, taken in the order (D, D'), the smallest δ at level ε is the hockey-stick divergence, the largest value of $P(S) - e^{\varepsilon}Q(S)$ over sets S of outputs. Writing $(x)_+ = \max(x, 0)$ and factoring P out of $(P - e^{\varepsilon}Q)_+$ turns it into an average of a function of the loss (Lemma 21.1). The outputs that contribute are those with $L > \varepsilon$, which gives the second form (Lemma 20.2):

$$\delta(\varepsilon) = \mathbb{E}_P\big[(1 - e^{\varepsilon - L})_+\big] = \Pr_P[L > \varepsilon] - e^{\varepsilon} \Pr_Q[L > \varepsilon]. \tag{30.1}$$

So an output contributes to δ only if its loss exceeds ε, and then only the fraction $1 - e^{\varepsilon - L}$ of its probability. An output with loss $\varepsilon + \ln 2$ contributes half its probability, and only an output with infinite loss contributes all of it. For the Gaussian at $\mu = 1$ and $\varepsilon = 1$, a Riemann sum of the average and the closed form Equation 20.17 both give $\delta = 1.2694 \times 10^{-1}$.

The second result says what an RDP curve knows. The Rényi divergence of order α is $1/(\alpha - 1)$ times the logarithm of $\mathbb{E}_P[e^{(\alpha-1)L}]$ (Equation 3.13), so a bound $\varepsilon(\alpha)$ on it is a bound on the MGF of the loss at the point $\alpha - 1$:

$$\mathbb{E}_P\big[e^{(\alpha-1)L}\big] \le e^{(\alpha-1)\varepsilon(\alpha)}. \tag{30.2}$$

Every conversion bounds the average in Equation 30.1 using only the averages in Equation 30.2. A DP guarantee is the largest of these δ over all neighbouring pairs, taken in both orders. RDP also bounds the divergence in both orders, so every bound below, proved for one pair (P, Q), holds just as well for (Q, P) and for every other pair.

Mironov's proof makes two moves. It replaces δ by the probability that the loss exceeds ε, and then it bounds that probability by the MGF.

Proposition 30.1 (Mironov's conversion, in two steps)**.** *Let M be $(\alpha, \varepsilon(\alpha))$-RDP for some order $\alpha > 1$, and fix a neighbouring pair with privacy loss L. For every ε,*

$$\delta(\varepsilon) \overset{(1)}{\le} \Pr_P[L > \varepsilon] \overset{(2)}{\le} e^{(\alpha-1)(\varepsilon(\alpha)-\varepsilon)}. \tag{30.3}$$

Hence, for every $\delta \in (0, 1)$, the mechanism M is (ε, δ)-DP with

$$\varepsilon = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1}. \tag{30.4}$$

Proof.

1. *Drop the fraction.* Inequality (1) rounds the fraction of Equation 30.1 up to 1. The positive part $(1 - e^{\varepsilon - L})_+$ is 0 when $L \le \varepsilon$ and below 1 otherwise, so it is at most the indicator $\mathbf{1}[L > \varepsilon]$. Averaging both sides under P gives (1).
2. *Bound the tail by the MGF.* Inequality (2) is Markov's inequality (Theorem 2.1): a quantity that cannot be negative exceeds a level with probability at most its average divided by the level. The quantity $e^{(\alpha-1)L}$ exceeds $e^{(\alpha-1)\varepsilon}$ exactly when $L > \varepsilon$, so

$$\Pr_P[L > \varepsilon] \le \mathbb{E}_P\big[e^{(\alpha-1)L}\big]\, e^{-(\alpha-1)\varepsilon}, \tag{30.5}$$

   and Equation 30.2 bounds the average by $e^{(\alpha-1)\varepsilon(\alpha)}$.

3. *Solve for ε.* Setting the right side of (2) equal to δ and taking logarithms gives $(\alpha-1)(\varepsilon(\alpha)-\varepsilon) = -\ln(1/\delta)$, which rearranges to Equation 30.4. The same argument applied to (Q, P) covers the other order of the pair.

□

The two steps lose in different places. Step 1 ignores how far past ε the loss goes, so an output with loss just above ε counts fully toward the tail but almost nothing toward δ. It is tight only when every output with $L > \varepsilon$ has infinite loss, as for a mechanism that sometimes publishes the record outright. Step 2 replaces a tail by an MGF, and Markov's inequality is tight only for a variable that puts all its mass on 0 and on the level itself. A smooth loss distribution is nothing like that.

For the Gaussian, the loss under P is normal with mean $\mu^2/2$ and standard deviation μ (Lemma 20.3). The tail $\Pr_P[L > \varepsilon]$ reaches 10^{-5} at 4.2649 standard deviations above the mean. Minimizing Mironov's ε over the order, which Rényi DP did in closed form (Corollary 29.1 with $\sigma = 1/\mu$), gives $\mu^2/2+\mu\sqrt{2\ln(1/\delta)}$, which sits $\sqrt{2\ln(10^5)} \approx 4.799$ standard deviations above the mean. So step 2 moves the level up by a fixed $4.799 - 4.2649 \approx 0.534$ standard deviations, and in units of ε its cost grows like μ.

Example 30.1. You release a statistic of sensitivity 1 with the Gaussian mechanism at $\sigma = 1$, so $\mu = 1$ and $\varepsilon(\alpha) = \alpha/2$. Mironov's conversion gives $\alpha/2 + \ln(10^5)/(\alpha - 1)$ at each order. The first term grows with the order and the second shrinks, so the best order lies in between. Over all real orders it is $\alpha = 5.799$, with $\varepsilon = 5.299$, and the grid's order 5.8 gives the same 5.299. The exact answer, from Equation 20.17, is 4.377, so Mironov's number overstates ε by 0.921.

Table 30.1 splits that excess between the two steps, and repeats the split for 100 such releases, where $\mu = 10$. Its first three rows give the ε at which each member of Equation 30.3 equals 10^{-5}, and the last two their differences. The middle row needs the tail. Under P the loss is $\mu^2/2 - \mu y$ with y standard normal, so $L > \varepsilon$ when $y < \mu/2 - \varepsilon/\mu$, and $\Pr_P[L > \varepsilon] = \Phi(\mu/2 - \varepsilon/\mu)$, where Φ is the standard normal distribution function. This tail equals 10^{-5} when $\mu/2 - \varepsilon/\mu = -4.2649$, which puts the level at

$$\varepsilon = \frac{\mu^2}{2} + 4.2649\,\mu, \tag{30.6}$$

about 4.765 at $\mu = 1$ and 92.649 at $\mu = 10$.

Table 30.1.: Mironov's conversion for the Gaussian at $\delta = 10^{-5}$, split into its two steps.

	$\mu = 1$ (one step, $\sigma = 1$)	$\mu = 10$ (100 steps, $\sigma = 1$)
Exact ε	4.377	91.817
After step 1: ε where the tail equals δ	4.765	92.649
After step 2, Mironov	5.299 at $\alpha = 5.8$	98.026 at $\alpha = 1.5$
Loss from step 1	0.388	0.832
Loss from step 2	0.534	5.377 (0.041 of it from the grid)

At $\mu = 10$, step 2 costs 5.377 of the total excess $98.026 - 91.817 = 6.209$. Of that, 0.041 is the grid's, since over all real orders Mironov's conversion gives 97.985 at $\alpha = 1.480$. Measured in δ instead, at a fixed ε, the two losses multiply. Back at $\mu = 1$ and Mironov's $\varepsilon = 5.299$, the exact δ is 1.300×10^{-7}, while the tail is 7.992×10^{-7}, 6.1 times larger. Mironov's bound is 1.000×10^{-5}, 12.5 times larger again, so you certify $\delta = 10^{-5}$ for a release whose true δ is 1.3×10^{-7}.

30.2. The improved conversion

Both of Mironov's steps together replace the integrand $(1 - e^{\varepsilon - z})_+$ of Equation 30.1, as a function of a loss value z, by an exponential $c\, e^{(\alpha-1)z}$ that lies above it. The bound has to be an exponential, because that is the only function of the loss whose average Equation 30.2 tells you. Mironov's constant $c = e^{-(\alpha-1)\varepsilon}$ makes the exponential equal 1 at $z = \varepsilon$, so it passes through the corner of the indicator. But the integrand is well below 1 near ε, so a lower exponential still lies above it. This section finds the lowest one, and that is the whole improvement.

The result was found three times. It is Theorem 21 of Balle, Barthe, Gaboardi, Hsu & Sato (2020) and Proposition 12 of Canonne, Kamath & Steinke (2020), who also give the δ form, and Asoodeh, Liao, Calmon, Kosut & Sankar proved it independently (2020, Lemma 1). Its δ is Mironov's multiplied by a factor below 1 that depends only on the order.

Theorem 30.1 (The improved conversion)**.** *Let M be $(\alpha, \varepsilon(\alpha))$-RDP for some order $\alpha > 1$. For every ε, the mechanism M is (ε, δ)-DP with*

$$\delta = \frac{e^{(\alpha-1)(\varepsilon(\alpha)-\varepsilon)}}{\alpha}\Big(1 - \frac{1}{\alpha}\Big)^{\alpha-1}. \tag{30.7}$$

Equivalently, for every $\delta \in (0, 1)$, it is (ε, δ)-DP with

$$\varepsilon = \varepsilon(\alpha) + \ln\frac{\alpha - 1}{\alpha} - \frac{\ln\delta + \ln\alpha}{\alpha - 1}. \tag{30.8}$$

Proof. The plan is to reduce the conversion to one constant, find the smallest constant for which the exponential stays above the integrand, and then solve for ε.

1. *Reduce to a constant.* Suppose $(1 - e^{\varepsilon - z})_+ \le c\, e^{(\alpha-1)z}$ for every real z. Then the same holds with z replaced by the random loss L, and averaging under P keeps the inequality. The left side averages to $\delta(\varepsilon)$ by Equation 30.1, and the right side to at most $c\, e^{(\alpha-1)\varepsilon(\alpha)}$ by Equation 30.2, so
$$\delta(\varepsilon) \le c\, e^{(\alpha-1)\varepsilon(\alpha)}. \tag{30.9}$$
2. *Write the smallest constant.* The condition of step 1 says that c is at least $(1 - e^{\varepsilon - z})_+\, e^{-(\alpha-1)z}$ at every z, so the smallest valid constant c^* is the largest value of that expression. It is 0 for $z \le \varepsilon$, and for $z > \varepsilon$ it is
$$g(z) = e^{-(\alpha-1)z} - e^{\varepsilon - \alpha z}. \tag{30.10}$$
3. *Find the maximum.* To locate the peak of g, differentiate it and factor out $e^{-\alpha z}$:
$$g'(z) = e^{-\alpha z}\big(\alpha e^{\varepsilon} - (\alpha - 1)e^{z}\big). \tag{30.11}$$
The bracket equals $e^{\varepsilon} > 0$ at $z = \varepsilon$ and decreases as z grows, so g first rises and then falls. Its maximum is where the bracket vanishes, at $e^{z^*} = \frac{\alpha}{\alpha-1}e^{\varepsilon}$, that is at the point $z^* = \varepsilon - \ln(1 - 1/\alpha)$.

4. *Evaluate it.* Write $g(z^*) = e^{-\alpha z^*}(e^{z^*} - e^{\varepsilon})$. At z^* the difference $e^{z^*} - e^{\varepsilon}$ is $e^{\varepsilon}/(\alpha - 1)$, and the first factor is $e^{-\alpha\varepsilon}(1 - 1/\alpha)^{\alpha}$. Multiplying the two gives

$$c^* = \frac{e^{-(\alpha-1)\varepsilon}}{\alpha - 1}\Big(1 - \frac{1}{\alpha}\Big)^{\alpha}. \tag{30.12}$$

5. *Collect.* Step 1 with $c = c^*$ bounds δ by $c^* e^{(\alpha-1)\varepsilon(\alpha)}$. Since $(1 - 1/\alpha)/(\alpha - 1) = 1/\alpha$, the constant in front simplifies to $\frac{1}{\alpha}(1 - \frac{1}{\alpha})^{\alpha-1}$, which is Equation 30.7.
6. *Solve for ε.* Setting Equation 30.7 equal to δ and taking logarithms gives

$$\ln \delta = (\alpha - 1)(\varepsilon(\alpha) - \varepsilon) - \ln \alpha + (\alpha - 1) \ln\Big(1 - \frac{1}{\alpha}\Big). \tag{30.13}$$

   Dividing by $\alpha - 1$ and solving for ε, with $\ln(1 - 1/\alpha) = \ln \frac{\alpha-1}{\alpha}$, gives Equation 30.8. As for Mironov's conversion, the pair (Q, P) follows from the same curve.

□

Figure 30.1 draws the two exponentials. Mironov's passes through the corner of the indicator at $z = \varepsilon$, while the lowest one touches the integrand at z^*, slightly to the right of ε. The chapter's script (Section 30.6) checks step 4 by brute force: at $\alpha = 6$ and $\varepsilon = 4$ the formula gives $c^* = 1.380552 \times 10^{-10}$ at $z^* = 4.1823$, and the maximum of g over a grid of two million points agrees to all seven digits. One practical point remains. When $\varepsilon(\alpha)$ is tiny, Equation 30.8 can be negative, and then you report 0, as the script and Canonne et al. do. That is safe because $\delta(\varepsilon)$ only falls as ε grows, so a bound at a negative level also holds at 0.

Both conversions add a term to the same $\varepsilon(\alpha)$, so their difference at one order does not depend on the mechanism. That difference is what an accountant gains by switching.

Corollary 30.1 (What the improved conversion saves)**.** *At every order $\alpha > 1$, for every RDP curve and every δ, the improved ε of Equation 30.8 is smaller than Mironov's Equation 30.4 by*

$$s(\alpha) = \frac{\ln \alpha}{\alpha - 1} + \ln \frac{\alpha}{\alpha - 1}. \tag{30.14}$$

At a fixed ε, the improved δ is Mironov's times $\frac{1}{\alpha}(1 - \frac{1}{\alpha})^{\alpha-1}$. Over a grid of orders, the best improved ε is below the best Mironov ε by at least $s(\alpha_M)$, where α_M is the order Mironov's conversion picks.

Proof.

1. *The saving in ε.* Subtracting Equation 30.8 from Equation 30.4 at the same order cancels both $\varepsilon(\alpha)$ and $\ln(1/\delta)/(\alpha - 1)$. What remains is $\ln \alpha/(\alpha - 1) - \ln \frac{\alpha-1}{\alpha}$, which is $s(\alpha)$.
2. *The factor in δ.* Mironov's δ at level ε is $e^{(\alpha-1)(\varepsilon(\alpha)-\varepsilon)}$, the right side of Equation 30.3, and Equation 30.7 multiplies it by the stated factor.
3. *Over a grid.* The improved conversion takes its own best order, so its minimum is at most its value at α_M. By step 1 that value is Mironov's minimum minus $s(\alpha_M)$.

□

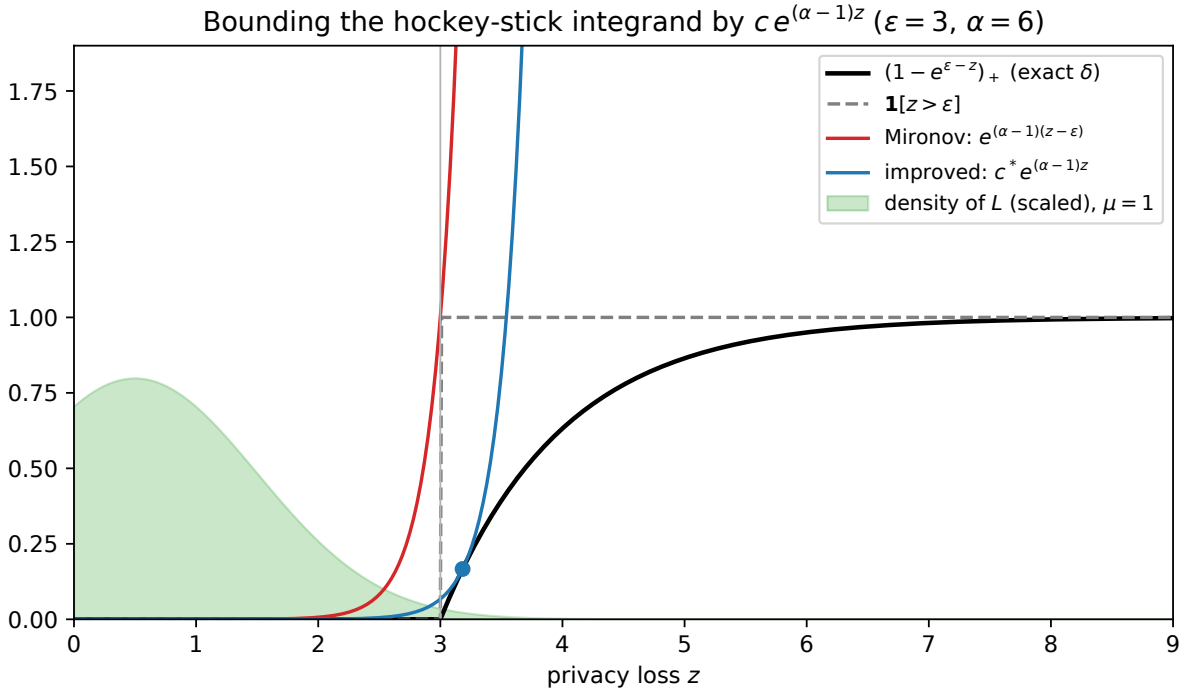


Figure 30.1.: The hockey-stick integrand $(1 - e^{\varepsilon - z})_+$ (black) at $\varepsilon = 3$, the indicator $\mathbf{1}[z > \varepsilon]$ that Mironov's step 1 replaces it with (dashed), and the two exponentials of order $\alpha = 6$ that bound it: Mironov's, through the corner of the indicator (red), and the smallest one, which touches the integrand at $z^* = \varepsilon - \ln(1 - 1/\alpha)$ (blue). The shaded curve is the density of the Gaussian privacy loss at $\mu = 1$: the conversions average the exponentials against it.

The factor in δ behaves like $1/(e\alpha)$ for large orders, because $(1-1/\alpha)^{\alpha-1}$ tends to $1/e$. So at large orders the gain is large in δ but small in ε. Table 30.2 gives values. At order 2, for instance, the factor is $\frac{1}{2}\cdot\frac{1}{2}=0.25$ and the saving is $\ln 2+\ln 2=1.3863$. If two accountants report ε about 0.4 apart from the same curve at a best order near 10, one of them uses Mironov's conversion. Opacus and `dp_accounting` both use the improved one.

Table 30.2.: What the improved conversion gains over Mironov's at the same order.

α	δ factor	$1/(e\alpha)$	ε saving $s(\alpha)$
2	0.2500	0.1839	1.3863
10	0.0387	0.0368	0.3612
100	0.0037	0.0037	0.0566

Example 30.2. For the mean released with $\sigma=1$, the improved conversion over the grid gives $(4.729, 10^{-5})$-DP at order 5.4. Mironov's conversion gave 5.299 at order 5.8, and the exact value is 4.377. So the improved conversion removes 0.570 of the 0.921 excess, using the same curve and one extra line of code. Corollary 30.1 promised at least $s(5.8)=\frac{\ln 5.8}{4.8}+\ln\frac{5.8}{4.8}=0.555$. The actual saving is a little more, because the improved conversion moves to a better order. At $\varepsilon=4.729$ the exact Gaussian δ is 2.097×10^{-6}, so the guarantee holds with room to spare.

The improved ε is below even the 4.765 of step 1 in Table 30.1. That is possible because its exponential, unlike the indicator, dips below 1 just above ε, where the integrand is small. For 100 releases, at $\mu=10$, it gives 96.116 against Mironov's 98.026 and the exact 91.817.

30.3. RDP as a constraint on tests

An (ε,δ) guarantee limits attacks through a single straight line in the plane of a test's two errors, and an RDP curve says more. A **test** looks at the output and decides between the hypothesis "it came from P", so your record was not used, and "it came from Q" (Definition 4.1). Its two errors are usually written α and β, but α is the Rényi order on this page, so they are written by their names. Its false positive rate FPR is the probability that it rejects P although the output came from P. Its false negative rate FNR is the probability that it accepts although the output came from Q, and its **power** is $1-\mathrm{FNR}$. This section finds the region of error pairs that an RDP curve allows, following Balle et al. (2020), and from it the best possible conversion.

The tool is data processing. A test turns the output into a coin, and that is post-processing, which cannot increase a Rényi divergence (Theorem 3.5). So each order of the curve constrains the coin, and through it the pair of errors. The **trade-off function** $T(P,Q)(\mathrm{FPR})$ of Definition 4.2 is the smallest FNR that any test reaches at false positive rate FPR, so it describes the most powerful attack at each rate.

Theorem 30.2 (The test region of an RDP curve)**.** *Let M have RDP curve $\varepsilon(\cdot)$ at the orders of a grid, and let $P=M(D)$ and $Q=M(D')$ on neighbouring datasets. Every test between P and Q, with errors* FPR *and* FNR*, satisfies at every order α of the grid*

$$\mathrm{FPR}^{\alpha}(1-\mathrm{FNR})^{1-\alpha}+(1-\mathrm{FPR})^{\alpha}\mathrm{FNR}^{1-\alpha}\le e^{(\alpha-1)\varepsilon(\alpha)},\qquad (1-\mathrm{FNR})^{\alpha}\mathrm{FPR}^{1-\alpha}+\mathrm{FNR}^{\alpha}(1-\mathrm{FPR})^{1-\alpha}\le e^{(\alpha-} \tag{30.15}$$

Call the set of pairs $(\mathrm{FPR}, \mathrm{FNR})$ *that satisfy all of these the* ***RDP privacy region****, and let* $f_{\mathrm{RDP}}(\mathrm{FPR})$ *be the smallest* FNR *in it at false positive rate* FPR*. Then:*

1. Trade-off bound. *For every* FPR*,* $T(P, Q)(\mathrm{FPR}) \geq f_{\mathrm{RDP}}(\mathrm{FPR})$*, so no test is more powerful than the region allows.*
2. Optimal conversion. *Every pair consistent with the curve has*

$$\delta(\varepsilon) \leq \delta_{\mathrm{opt}}(\varepsilon) = \max_{\mathrm{FPR}} \left[1 - \mathrm{FPR} - e^{\varepsilon} f_{\mathrm{RDP}}(\mathrm{FPR})\right], \tag{30.16}$$

and some two-outcome pair consistent with the curve attains it. So δ_{opt} *is the best* δ *that any conversion using only these RDP values can certify.*

Proof. The plan is to compute the Rényi divergence of a test's outcome and apply the curve to it, which proves claim 1, and then to rewrite δ as a statement about tests, which the region bounds.

1. *The outcome of a test is a coin.* Table 30.3 lists its distribution under each hypothesis.

Table 30.3.: The outcome of a test with errors $(\mathrm{FPR}, \mathrm{FNR})$, as a two-point distribution under each hypothesis.

Outcome	Probability under P	Probability under Q
reject	FPR	$1 - \mathrm{FNR}$
accept	$1 - \mathrm{FPR}$	FNR

2. *Compute its Rényi divergence.* For a two-point distribution, the sum in the definition of D_α has one term per row of Table 30.3:

$$D_\alpha(\text{coin under } P \,\|\, \text{coin under } Q) = \frac{1}{\alpha - 1} \ln \left[\mathrm{FPR}^\alpha (1 - \mathrm{FNR})^{1-\alpha} + (1 - \mathrm{FPR})^\alpha \mathrm{FNR}^{1-\alpha}\right]. \tag{30.17}$$

3. *Apply the curve.* By data processing this divergence is at most $D_\alpha(P\|Q)$, which is at most $\varepsilon(\alpha)$. Multiplying by $\alpha - 1 > 0$ and exponentiating gives the first inequality of Equation 30.15. The second comes the same way from the bound on $D_\alpha(Q\|P)$, as the same sum with the columns of the table swapped. So every test's errors lie in the region, and its FNR is at least f_{RDP} at its FPR, which is claim 1.

4. *Write* δ *as a best test.* Let S be the set of outputs on which a test accepts, so that $P(S) = 1 - \mathrm{FPR}$ and $Q(S) = \mathrm{FNR}$. By Lemma 20.2, $\delta(\varepsilon)$ is the largest $P(S) - e^\varepsilon Q(S)$ over all events, so

$$\delta(\varepsilon) = \max_{\text{tests}} \left[1 - \mathrm{FPR} - e^{\varepsilon} \mathrm{FNR}\right]. \tag{30.18}$$

Every test has $\mathrm{FNR} \geq f_{\mathrm{RDP}}(\mathrm{FPR})$ by step 3, and replacing FNR by $f_{\mathrm{RDP}}(\mathrm{FPR})$ can only increase the bracket. That gives the bound in Equation 30.16.

5. *Attain the bound.* Let $(\mathrm{FPR}^*, \mathrm{FNR}^*)$ maximize Equation 30.16, with $\mathrm{FNR}^* = f_{\mathrm{RDP}}(\mathrm{FPR}^*)$. Build a mechanism whose output is just the coin of Table 30.3 with these errors: on D it accepts with probability $1 - \mathrm{FPR}^*$, and on D' with probability FNR^*. By step 2, its Rényi divergences in both directions are the logarithms of the left sides of Equation 30.15, divided by $\alpha - 1$, at a point of the region, so the mechanism satisfies the curve. The test

that accepts exactly when the coin shows "accept" has errors $(\mathrm{FPR}^*, \mathrm{FNR}^*)$, so by step 4 this mechanism's $\delta(\varepsilon)$ is at least $1 - \mathrm{FPR}^* - e^{\varepsilon}\mathrm{FNR}^*$, which is $\delta_{\mathrm{opt}}(\varepsilon)$.

□

Computing f_{RDP} uses the shape of the constraints. For a fixed FPR, both left sides of Equation 30.15 equal 1 at $\mathrm{FNR} = 1 - \mathrm{FPR}$, where the test ignores the output and the two coins are identical. Elsewhere they are at least 1, because each is $e^{(\alpha-1)D_\alpha}$ for a divergence that is never negative, so that point is their minimum. Both are also convex in FNR, because each term has the form $x^\alpha y^{1-\alpha}$, which is jointly convex for $\alpha > 1$ (Section F.5 proves it). So on $[0, 1 - \mathrm{FPR}]$ they decrease, the constraints hold above a cut-off and fail below it, and $f_{\mathrm{RDP}}(\mathrm{FPR})$ is that cut-off. The script finds it by bisection.

The region is an intersection of convex sets, so f_{RDP} is convex and the objective of Equation 30.16 is concave in FPR, which a one-dimensional search maximizes. For a single order this optimum is close to the improved conversion. At order 2, where $\varepsilon(2) = 1$ for the running example, Mironov gives $1 + \ln(10^5) = 12.513$, the improved conversion 11.127 and the optimum from that one value 10.668. At orders 8 and 32 the improved conversion and the optimum agree to three decimals (5.214 and 16.228). Balle et al. indeed derive the improved conversion as the straight line that just touches the region of one order, the same tangency that Figure 30.1 shows between the exponential and the integrand.

The region is not the set an (ε, δ) guarantee allows. An (ε, δ) pair says that every test has $1 - \mathrm{FPR} \le e^{\varepsilon}\mathrm{FNR} + \delta$, and the same with the two errors swapped: straight lines, and the best it can do is touch the curved region at one point. A whole trade-off curve is the envelope of such lines, one for each ε, which is the duality that Gaussian DP develops. Nor does the region characterize RDP. A test squeezes the output into one bit, and information spread over many outputs is lost when they are merged into "accept" and "reject". So a pair can have every test inside the region and still violate the curve: the region is what RDP implies about tests, not an equivalent definition.

Example 30.3. For the Gaussian at $\mu = 1$ the true trade-off curve is known, so you can see how loose the region is. The best test between $\mathcal{N}(0, 1)$ and $\mathcal{N}(1, 1)$ rejects when the output exceeds a threshold, because the likelihood ratio increases with the output (Theorem 4.1). Its false negative rate at false positive rate FPR is the Gaussian trade-off function of Theorem 4.3,

$$G_1(\mathrm{FPR}) = \Phi\big(\Phi^{-1}(1 - \mathrm{FPR}) - 1\big). \tag{30.19}$$

At $\mathrm{FPR} = 0.1$, $\Phi^{-1}(0.9) = 1.2816$, so $G_1 = \Phi(0.2816) = 0.6109$ and the true power is 0.3891. Figure 30.2 plots the largest power the RDP curve allows against this true power, and Table 30.4 compares them at three rates. The curve allows 1.24 to 1.36 times the power the Gaussian has; at $\mathrm{FPR} = 0.1$, for instance, $0.48376/0.38914 = 1.24$.

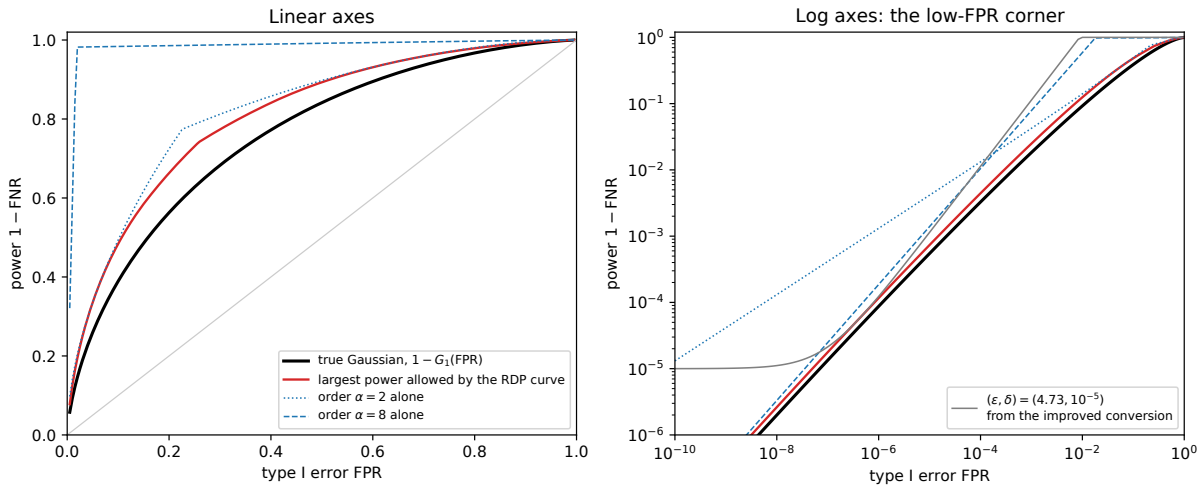


Figure 30.2.: Largest power $1-\mathrm{FNR}$ at false positive rate FPR allowed by the Gaussian's RDP curve at $\mu = 1$ (red, all orders of the grid together), by the single orders 2 and 8 (blue), and achieved by the Gaussian itself (black). Left: linear axes. Right: log axes, with the (ε, δ) line $1-\mathrm{FNR} \le e^{\varepsilon}\mathrm{FPR} + \delta$ from the improved conversion (grey), which touches the red curve.

Table 30.4.: Power of the best test against the Gaussian mechanism at $\mu = 1$, true and allowed by its RDP curve.

False positive rate FPR	True power $1 - G_1(\mathrm{FPR})$	Largest power allowed by the RDP curve	Ratio
0.1	3.8914×10^{-1}	4.8376×10^{-1}	1.24
10^{-3}	1.8298×10^{-2}	2.4910×10^{-2}	1.36
10^{-6}	8.7218×10^{-5}	1.1636×10^{-4}	1.33

Each order guards a different range of FPR. At small FPR the second constraint of order α keeps its first term, $(1-\mathrm{FNR})^{\alpha}\mathrm{FPR}^{1-\alpha}$, below $e^{(\alpha-1)\varepsilon(\alpha)}$. So the power is at most a constant times $\mathrm{FPR}^{(\alpha-1)/\alpha}$, a line of slope $(\alpha-1)/\alpha$ on log axes. As FPR shrinks, a line of slope near 1 falls faster than one of slope near $\frac{1}{2}$, so the constraint of a large order is the tighter, or binding, one at small FPR, while small orders bind in the bulk. At $\mathrm{FPR} = 10^{-6}$, order 2 alone allows power 1.3118×10^{-3} and order 8 allows 1.8622×10^{-4}. At $\mathrm{FPR} = 0.1$ the roles swap, with 0.4932 for order 2 and 0.9838 for order 8. The exact boundaries of the two orders cross at $\mathrm{FPR} \approx 1.87 \times 10^{-4}$, and at $\mathrm{FPR} = 10^{-3}$ the binding order of the grid is 3.7.

Now suppose an auditor runs a membership test against the mean released with $\sigma = 1$, at false positive rate 10^{-3}. The RDP curve alone guarantees that no test detects membership with probability above 2.4910×10^{-2}, and the true best test reaches 1.8298×10^{-2}. The $(4.729, 10^{-5})$ guarantee of the improved conversion only caps the power at $e^{4.729} \times 10^{-3} + 10^{-5} = 0.1131$, more than four times the region's bound. Reading an RDP result through one (ε, δ) pair throws away most of what it says about low false positive rates.

30.4. Choosing the order

Both conversions hold at every order, so you take the best one. Each converted ε is the RDP curve, which grows with the order, plus a term in $\ln(1/\delta)/(\alpha-1)$, which shrinks with it, so the best order balances the two. An accountant evaluates the curve only on a grid of orders, and where the best order falls tells you what that grid must contain. For the Gaussian the curve is a line through the origin, and the best order has a closed form or nearly so.

Proposition 30.2 (Best orders for a linear curve)**.** *Let $\varepsilon(\alpha) = \rho\alpha$ with slope $\rho > 0$, which is $\rho = \mu^2/2 = k/(2\sigma^2)$ for the composed Gaussian, and let $0 < \delta < 1$. Over real orders $\alpha > 1$, Mironov's conversion is smallest at $\alpha^* = 1 + \sqrt{\ln(1/\delta)/\rho}$, where it equals*

$$\varepsilon^* = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{30.20}$$

The improved conversion is convex for $1 < \alpha < 1/\delta$, and its smallest value there is at the root of

$$\rho(\alpha-1)^2 = \ln\frac{1}{\alpha\delta}. \tag{30.21}$$

Proof. Each step sets a derivative to zero, as for any smooth minimum, and the last checks that the stationary point is a minimum.

1. *Mironov.* The derivative of $\rho\alpha + \ln(1/\delta)/(\alpha-1)$ is $\rho - \ln(1/\delta)/(\alpha-1)^2$, which is increasing in α. Setting it to zero gives $(\alpha-1)^2 = \ln(1/\delta)/\rho$, which is α^*. At α^* the curve term is $\rho\alpha^* = \rho + \sqrt{\rho\ln(1/\delta)}$, and the second term $\ln(1/\delta)/(\alpha^*-1)$ is $\sqrt{\rho\ln(1/\delta)}$ too, so the two contribute equally beyond ρ.
2. *Improved, the derivative.* Write Equation 30.8 as $\rho\alpha + \ln(1-\frac{1}{\alpha}) + \frac{\ln(1/\delta)-\ln\alpha}{\alpha-1}$ and differentiate term by term. The middle term has derivative $\frac{1}{\alpha(\alpha-1)}$, and by the quotient rule the last one has derivative
$$-\frac{1}{\alpha(\alpha-1)} - \frac{\ln(1/\delta)-\ln\alpha}{(\alpha-1)^2}. \tag{30.22}$$
The terms $\frac{1}{\alpha(\alpha-1)}$ cancel, leaving $\rho - \ln(1/(\alpha\delta))/(\alpha-1)^2$. Setting it to zero gives Equation 30.21.
3. *A minimum.* The derivative of the expression of step 2 is $\frac{1}{\alpha(\alpha-1)^2} + \frac{2\ln(1/(\alpha\delta))}{(\alpha-1)^3}$, which is positive whenever $\alpha < 1/\delta$. So the objective is convex there, and its stationary point is its minimum.

□

The script solves Equation 30.21 by iterating $\alpha \leftarrow 1 + \sqrt{\ln(1/(\alpha\delta))/\rho}$ from Mironov's α^*. The only difference from Mironov's condition is the extra α inside the logarithm, which pulls the best order slightly lower. Ignoring that slowly varying $\ln\alpha$, the best order is

$$\alpha^* - 1 \approx \sqrt{\frac{\ln(1/\delta)}{\rho}} = \sigma\sqrt{\frac{2\ln(1/\delta)}{k}}. \tag{30.23}$$

More steps or less noise push the best order toward 1, with $\alpha^* - 1$ shrinking like $1/\sqrt{k}$. A smaller δ pushes it up, but only through $\sqrt{\ln(1/\delta)}$: at $\mu = 1$, moving from $\delta = 10^{-5}$ to 10^{-10} doubles

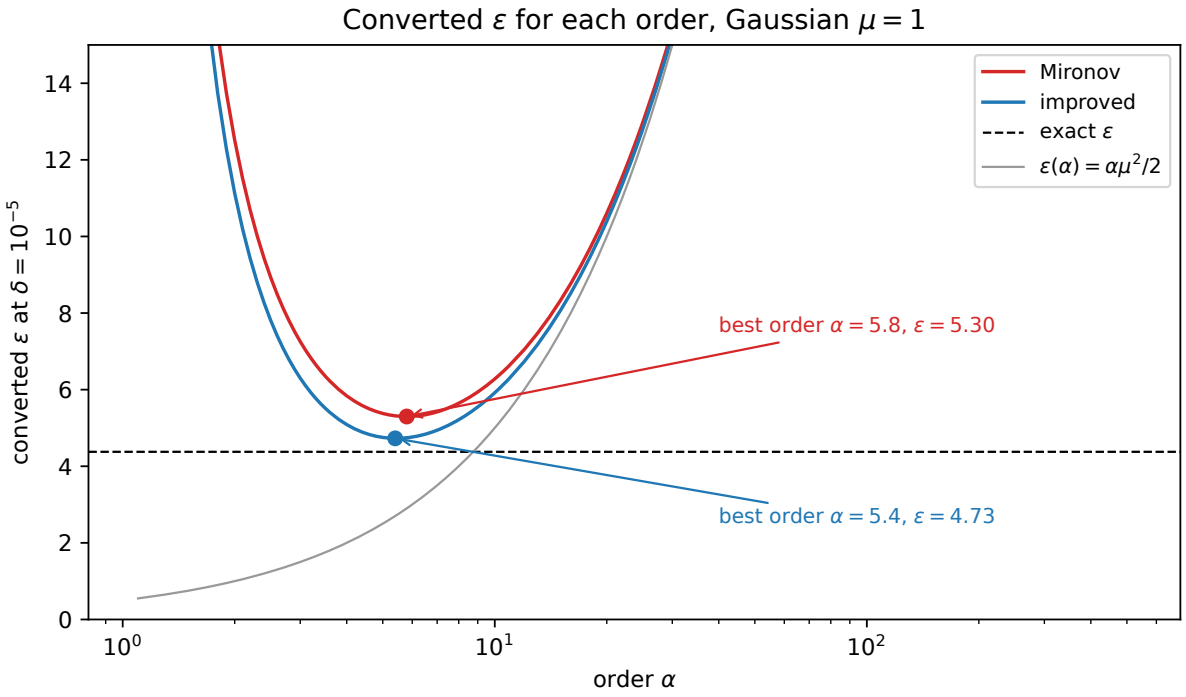


Figure 30.3.: Converted ε at $\delta = 10^{-5}$ for each order of the grid, for the Gaussian at $\mu = 1$, with Mironov's conversion (red) and the improved one (blue). Dots mark the best order of each. The dashed line is the exact ε, and the grey curve is the RDP curve $\varepsilon(\alpha) = \alpha/2$ that both conversions add to.

$\ln(1/\delta)$ but moves the improved best order only from 5.432 to 7.483. Figure 30.3 shows the two converted curves for $\mu = 1$. Both are U-shaped, and the improved one sits below Mironov's everywhere by $s(\alpha)$ of Equation 30.14.

Example 30.4. Table 30.5 shows how the best order moves over four settings. In the first row, one release at $\sigma = 10$, the slope is $\rho = 1/(2 \cdot 100) = 0.005$. Mironov's best order is then $1 + \sqrt{\ln(10^5)/0.005} = 1 + \sqrt{2302.6} = 48.985$, and the improved conversion's extra $\ln \alpha$ brings it down to 40.525.

Table 30.5.: Best orders for the composed Gaussian at $\delta = 10^{-5}$.

σ	k	ρ	Best order, improved (continuous / grid)	ε continuous / grid	Mironov's best order
10	1	0.005	40.525 / 41	0.375 / 0.375	48.985
1	1	0.5	5.432 / 5.4	4.728 / 4.729	5.799
1	100	50	1.472 / 1.5	96.035 / 96.116	1.480
1	10,000	5000	1.048 / 1.1	5475.791 / 5611.778	1.048

You plan $k = 100$ full-batch noisy gradient steps at $\sigma = 1$, so $\rho = 100/2 = 50$. Equation 30.21 puts the best order at 1.472, and the grid's 1.5 costs $96.116 - 96.035 = 0.081$ in ε. A single release at $\sigma = 10$ instead wants order 40.525, and the grid's integer orders match it to three decimals of ε. One grid serves both because it is dense near 1 and runs from 1.1 to 512.

The grid needs orders near 1 for long runs with little noise. At $\rho = 5000$ the best order is 1.048, below the grid's smallest order 1.1, and the grid's answer is $5611.778 - 5475.791 = 135.987$ larger than the continuous optimum. The grid also needs large orders for a small total slope: one release at $\sigma = 100$ has $\rho = 5 \times 10^{-5}$ and best order 338.312. Subsampled mechanisms (subsampling), which use each record only in a random fraction of the steps, have tiny curves, so like this release they need large orders, which is why the grid ends at 128, 256 and 512.

30.5. The slack no conversion removes

The improved conversion beats Mironov's, and the optimal conversion of Equation 30.16 is the best possible from the curve. Yet for the Gaussian both stay above the exact ε. Is that a weakness of the formulas, or of the curve as a summary? This section shows that it is the summary. A different mechanism satisfies the same curve and has a larger δ, and a conversion that sees only the curve must cover both. It also looks at the opposite direction, from (ε, δ) to RDP, which matters when one analysis mixes pure-DP and RDP components.

The idea is to build the worst mechanism the curve allows and compute its δ. By Theorem 30.2, the largest δ consistent with a set of orders is attained by a two-outcome pair. Table 30.6 shows one, for the Gaussian's curve at $\mu = 1$ and the level $\varepsilon = 4.377$ where the Gaussian has $\delta = 10^{-5}$.

Like the coin of Table 30.3, it has two outcomes: a rare one, much likelier under D, that betrays D, and a common one that carries almost all the mass. The privacy loss column is $\ln(P/Q)$ for each outcome. For the rare outcome it is $\ln(2.2779 \times 10^{-4}/2.2998 \times 10^{-6}) = 4.5956$, and for the common one it is a tiny negative number, because P puts slightly less mass there than Q.

Table 30.6.: A two-outcome mechanism that satisfies the Gaussian's RDP curve at $\mu = 1$ at every order and has nearly the largest δ at $\varepsilon = 4.377$ that any such mechanism can have.

Outcome	$P = M(D)$	$Q = M(D')$	Privacy loss L
rare	2.2779×10^{-4}	2.2998×10^{-6}	4.5956
common	rest	rest	-2.255×10^{-4}

Proposition 30.3 (The curve does not determine δ)**.** *The two-outcome mechanism of Table 30.6 satisfies $D_\alpha(P\|Q) \le \alpha/2$ and $D_\alpha(Q\|P) \le \alpha/2$ at every order $\alpha > 1$ that the script scans, and its δ at $\varepsilon = 4.377$ is 4.4698×10^{-5}. So no conversion that sees only the RDP curve $\alpha/2$ can certify $(4.377, 10^{-5})$-DP, although the Gaussian mechanism at $\mu = 1$ satisfies it.*

Proof. The plan is to build the pair from the optimal conversion, push it strictly inside the curve at every real order, check that numerically, and compute its δ.

1. *Build the pair.* On the part's grid, at $\mu = 1$ and $\varepsilon = 4.377$, the pair that attains Equation 30.16 has $\delta = 4.4700 \times 10^{-5}$. But it satisfies the curve only at the grid's orders, because its Rényi divergence is a smooth function of the order and can poke above $\alpha/2$ between two grid orders. So the script recomputes δ_{opt} with the constraints of a dense set of orders, steps of 0.002 from 1.01 to 30 and every integer up to 512. It then raises the probability of the rare outcome under Q by a relative 10^{-5}, which pushes the pair strictly inside the curve.
2. *Check the curve.* On a scan of α from $1 + 10^{-8}$ to 10^6, refined around the maximum, the largest value of $D_\alpha(P\|Q) - \alpha/2$ is -9.97×10^{-6}, at $\alpha = 5.0955$, so the curve holds everywhere scanned. The reverse divergences stay about 0.5 below the curve. Beyond the scan, $D_\alpha(P\|Q)$ tends to the largest privacy loss, 4.5956, while $\alpha/2$ grows without bound. The rounded values in the table also satisfy the curve.
3. *Compute its δ.* By Equation 30.1 only outcomes with $L > \varepsilon$ contribute, and here that is only the rare outcome. So $\delta(4.377) = P(\text{rare}) - e^{4.377}\, Q(\text{rare})$. With $e^{4.377} \approx 79.6$ this is 2.2779×10^{-4} minus about 1.831×10^{-4}, which is 4.4698×10^{-5} with unrounded values.
4. *Conclude.* A certificate computed from the curve alone must also hold for this pair, which has about 4.5 times the Gaussian's δ at the same ε. So no such certificate can give $(4.377, 10^{-5})$. For comparison, the improved conversion's δ at that ε is 4.4701×10^{-5}, within 10^{-8} of this pair's.

□

The pair puts almost all its mass at a loss near 0 and a sliver at 4.5956, where the improved conversion's exponential nearly touches the integrand, so for this pair that bound is nearly an equality. It is very unlike the Gaussian loss, which spreads its tail smoothly over all large values. The RDP curve cannot tell the two apart, because it only sees MGF values (Equation 30.2).

The opposite direction asks when a guarantee of another kind gives an RDP curve, so that a pure-DP or (ε, δ) component can join an RDP analysis.

Proposition 30.4 (From (ε, δ) and from ε-DP to RDP)**.**

1. *For every $\delta \in (0,1)$, some mechanism is $(0,\delta)$-DP and has $D_\alpha(P\|Q) = \infty$ at every order $\alpha > 1$. So (ε,δ)-DP with $\delta > 0$ implies no RDP bound at any order.*
2. *Every ε-DP mechanism is $(\alpha, \min(\varepsilon, \alpha\varepsilon^2/2))$-RDP for every $\alpha > 1$ (Bun & Steinke 2016, Proposition 3.3).*

Proof.

1. *A rare catastrophe.* Take the mechanism that outputs nothing, except that with probability δ it publishes your record. Order the pair with the dataset that holds you first, $P = M(D')$ and $Q = M(D)$, which a guarantee must also cover. Then P puts mass δ on an output that Q never produces, and otherwise matches Q. Moving mass δ changes the probability of any event by at most δ, so $P(S) \le Q(S) + \delta$ for every event S, and the mechanism is $(0,\delta)$-DP.
2. *Its divergence is infinite.* The sum $\sum P^\alpha Q^{1-\alpha}$ in the Rényi divergence contains the term $\delta^\alpha \cdot 0^{1-\alpha}$ for the published record. For $\alpha > 1$ that term is infinite, so $D_\alpha(P\|Q) = \infty$ at every order.
3. *The bound ε.* An ε-DP mechanism has privacy loss at most ε at every output, so $e^{(\alpha-1)L} \le e^{(\alpha-1)\varepsilon}$ everywhere. Its average under P is then at most $e^{(\alpha-1)\varepsilon}$. That average is $e^{(\alpha-1)D_\alpha(P\|Q)}$ (Equation 3.13), so taking logarithms and dividing by $\alpha - 1$ gives $D_\alpha(P\|Q) \le \varepsilon$.
4. *The bound $\alpha\varepsilon^2/2$.* This bound says that the logarithm of the MGF of an ε-DP loss grows at most quadratically. Concentrated DP proves it with Hoeffding's lemma, together with the sharper bound $\alpha\varepsilon\tanh(\varepsilon/2)$, which randomized response attains as $\alpha \to 1$.

□

So RDP, like pure DP, rules out catastrophic events however rare, and to get a curve from an (ε,δ) mechanism you need to know how its δ arises. For randomized response at $\varepsilon = 0.5$, the bound of claim 2 is 0.2500 at order 2 and 0.5000 at orders 4 and 10. The script gives $D_2 = 0.2273$, $D_4 = 0.3519$ and $D_{10} = 0.4473$, each below its bound.

Example 30.5. Table 30.7 and Figure 30.4 compare all four answers for the Gaussian with $\sigma = 10$ per step. Over $k = 1$ to 10,000 steps, $\mu = \sqrt{k}/10$ runs from 0.1 to 10 and the exact ε from 0.341 to 91.817, the range where accounting choices matter. Because everything depends only on μ, the row $k = 100$, where $\mu = 1$, repeats the single release at $\sigma = 1$ of the earlier sections.

Table 30.7.: ε at $\delta = 10^{-5}$ for the Gaussian with $\sigma = 10$ composed k times: exact, the best any conversion from the RDP grid can give, the improved conversion and Mironov's.

k	μ	Exact	Optimal from the curve	Improved	Mironov	Ratios to exact (opt. / impr. / Mir.)
1	0.100	0.341	0.375	0.375	0.485	1.10 / 1.10 / 1.42
10	0.316	1.199	1.308	1.308	1.568	1.09 / 1.09 / 1.31

k	μ	Exact	Optimal from the curve	Improved	Mironov	Ratios to exact (opt. / impr. / Mir.)
100	1.000	4.377	4.729	4.729	5.299	1.08 / 1.08 / 1.21
1000	3.162	17.857	19.054	19.054	20.175	1.07 / 1.07 / 1.13
10,000	10.000	91.817	96.116	96.116	98.026	1.05 / 1.05 / 1.07

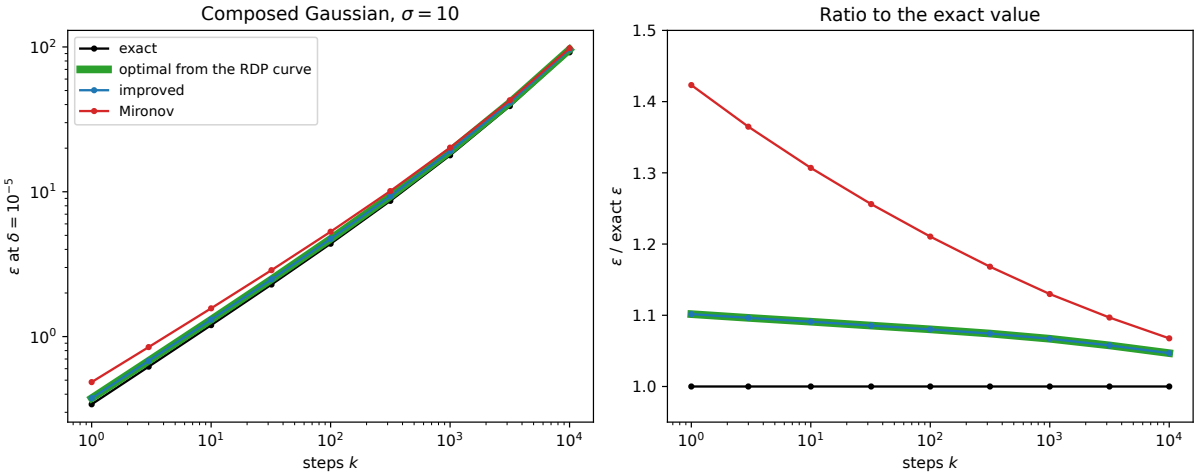


Figure 30.4.: Left: ε at $\delta = 10^{-5}$ for the Gaussian with $\sigma = 10$ composed k times, exact (black), optimal from the RDP curve (wide green), improved (blue, on top of the green) and Mironov (red). Right: each divided by the exact value. The improved conversion is optimal for the curve; the remaining 5 to 10% belongs to the RDP summary itself.

Over the whole grid, unlike at the single order 2 of Section 30.3, the optimal and improved conversions coincide: at $\mu = 1$ they are 4.728505 and 4.728507, a difference of 2×10^{-6}. The optimal conversion can use all orders at once, but that helps only if more than one order holds back the worst mechanism. For the worst pair at $\varepsilon = 4.377$, only the constraint at order 5.1 is active, and at every other order of the grid the pair's divergence stays at least 0.0011 below the curve. Mironov's excess shrinks fast with k, from 42% to 7%, and the unavoidable excess slowly, from 10% to 5%, because for large μ the term $\mu^2/2$ that every method shares dominates.

Suppose your paper reports $(4.729, 10^{-5})$-DP from an RDP accountant for this mechanism, and a reviewer asks whether a better conversion would lower it. The answer is no, by Proposition 30.3. The same construction at $\delta = 10^{-5}$ gives a pair inside the curve at every real order whose δ reaches 10^{-5} only at $\varepsilon = 4.728376$. The improved conversion at the best real order, 5.4318, gives 4.728387, which is a valid certificate. So the best value any method can certify from the curve lies between 4.728376 and 4.728387, which is 4.7284 to four decimals, and a method that sees only the grid's orders must report at least 4.728505. A lower number needs an accountant that sees the mechanism, and for the Gaussian that number is 4.377.

When the slack matters, change the summary, not the formula. To get the exact 4.377 you must track more than an MGF: the trade-off function (Gaussian DP) or the full privacy loss distribution (numerical accounting). The 5 to 10% is for the plain Gaussian. Subsampled Gaussian steps, as in private training, have curves that are not lines, and there the gap to the exact value can be much larger, as choosing an accountant measures.

30.6. Code

The script code/rdp_conversions.py computes every number in this chapter and draws its figures. Each conversion is a small typed function (`rdp_to_dp_mironov`, `rdp_to_dp_improved`, `rdp_to_dp_tradeoff`), and `continuous_worst_pair` builds the pair of Table 30.6. From the repository root:

```
uv run python code/part04-privacy-accounting/rdp_conversions.py
```

Table 30.8.: Results printed by the script (seed 0; nothing is random).

Experiment	Result
δ as an average, Gaussian $\mu = 1$, $\varepsilon = 1$	closed form 1.2694×10^{-1}; Riemann sum 1.2694×10^{-1}
Mironov at $\mu = 1$	5.299 at order 5.8; continuous optimum 5.299 at 5.799
Optimal constant at $\alpha = 6$, $\varepsilon = 4$	formula 1.380552×10^{-10}; grid 1.380552×10^{-10}
Improved at $\mu = 1$	4.729 at order 5.4; exact 4.377
Single order 2, $\varepsilon(2) = 1$	Mironov 12.513, improved 11.127, optimal 10.668
Power at FPR $= 10^{-3}$, $\mu = 1$	true 1.8298×10^{-2}; RDP allows 2.4910×10^{-2}
Best order, $\rho = 5000$	continuous 1.048; grid 1.1
Optimal from the curve vs improved, $\mu = 1$	4.728505 and 4.728507
Worst two-outcome pair at $\varepsilon = 4.377$	grid orders only: $\delta = 4.4700 \times 10^{-5}$; every real order: $\delta = 4.4698 \times 10^{-5}$, largest excess over $\alpha/2$ is -9.97×10^{-6}
Worst pair at $\delta = 10^{-5}$, every real order	$\varepsilon = 4.728376$; improved at order 5.4318: 4.728387
Randomized response $\varepsilon = 0.5$, order 2	$D_2 = 0.2273 \leq 0.2500$

tests/test_rdp_conversions.py checks that the output still matches Table 30.8 and every other number in the text.

30.7. Summary

Table 30.9.: The results of this chapter, and where each is used.

Result	Statement	Use
Proposition 30.1	$\varepsilon = \varepsilon(\alpha) + \ln(1/\delta)/(\alpha-1)$, by dropping the fraction and then Markov	The original moments accountant and older code; overstates the Gaussian's ε by 7 to 42%
Theorem 30.1	$\varepsilon = \varepsilon(\alpha) + \ln\frac{\alpha-1}{\alpha} - \frac{\ln\delta+\ln\alpha}{\alpha-1}$, from the lowest exponential above the integrand	Always use it: every modern RDP accountant (Opacus, `dp_accounting`) does
Corollary 30.1	Saves $s(\alpha) = \frac{\ln\alpha}{\alpha-1} + \ln\frac{\alpha}{\alpha-1}$ in ε, a factor near $1/(e\alpha)$ in δ	About 0.36 at order 10; two accountants that differ by this use different conversions
Theorem 30.2	Each order bounds every test's $(\mathrm{FPR}, \mathrm{FNR})$; $\delta_{\mathrm{opt}} = \max_{\mathrm{FPR}}[1 - \mathrm{FPR} - e^{\varepsilon} f_{\mathrm{RDP}}(\mathrm{FPR})]$	Read an RDP guarantee as a bound on attacks with the whole region, not one (ε, δ) line; Gaussian DP
Proposition 30.2	Mironov's best order $1 + \sqrt{\ln(1/\delta)/\rho}$; improved $\rho(\alpha-1)^2 = \ln(1/(\alpha\delta))$	Design the order grid; if the best order sits at either end of the grid, extend it
Proposition 30.3	A two-outcome pair with the Gaussian's curve has 4.5 times its δ	Expect 5 to 10% excess on the plain Gaussian and more with subsampling; for less, use a numerical accountant
Proposition 30.4	(ε, δ) with $\delta > 0$ gives no RDP; ε-DP gives $\min(\varepsilon, \alpha\varepsilon^2/2)$	Mixing pure-DP and RDP components in one analysis

30.8. References

- Mironov, "Rényi Differential Privacy" (IEEE CSF 2017): the definition, composition, the Gaussian and Laplace curves, and the conversion of Equation 30.4 (Proposition 3).
- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020; arXiv:1905.09982): privacy regions of divergences, the test region of Equation 30.15 (§5.1.1), and the improved conversion as a tangent line (Theorem 21).
- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020; arXiv:2004.00010): the short proof of the improved conversion through the optimal constant c^* (Proposition 12), and its use for concentrated DP (Corollary 13).
- Asoodeh, Liao, Calmon, Kosut & Sankar, "A Better Bound Gives a Hundred Rounds: Enhanced Privacy Guarantees via f-Divergences" (IEEE International Symposium on Information Theory, ISIT 2020; arXiv:2001.05990): an independent proof of the improved conversion (Lemma 1, Eq. 20), applied to noisy gradient descent.

- Asoodeh, Liao, Calmon, Kosut & Sankar, "Three Variants of Differential Privacy: Lossless Conversion and Applications" (IEEE Journal on Selected Areas in Information Theory, 2021; arXiv:2008.06529): the optimal conversion from one Rényi order through the joint range of two f-divergences, and RDP as a trade-off between the two errors of a test.
- Bun & Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (TCC 2016): ε-DP implies $\varepsilon^2/2$-zero-concentrated DP (Proposition 3.3) and the standard zero-concentrated conversion.
- Steinke, "Composition of Differential Privacy & Privacy Amplification by Subsampling" (2022 survey, arXiv:2210.00597): a unified treatment of privacy loss, RDP, conversions and their tightness.
- Opacus (`opacus/accountants/analysis/rdp.py`) and Google's `dp_accounting` (`rdp_privacy_accountant.py`): production implementations of the improved conversion.

31. Concentrated differential privacy

The first chapter of this part describes a mechanism by a whole curve. Take two neighbouring datasets $D \sim D'$, which differ by adding or removing one record. At each order $\alpha > 1$, the **Rényi differential privacy (RDP) curve** $\varepsilon(\alpha)$ is the largest Rényi divergence D_α between the output distributions $M(D)$ and $M(D')$ over all such pairs. For the Gaussian mechanism, which adds normal noise of standard deviation σ to a query that one record moves by at most 1, the curve is the line $\alpha/(2\sigma^2)$, fixed by its slope. This chapter asks when one number can stand for the whole curve, and what that number buys.

The answer is **zero-concentrated differential privacy (zCDP)**, the most widely used form of concentrated differential privacy (CDP). A mechanism is ρ-zCDP when its RDP curve lies under the line $\rho\alpha$. RDP curves add under composition, and lines through the origin add to a line, so composition adds the parameters. Every section asks how well a line fits the curve. It fits the Gaussian exactly. A pure differential privacy (DP) mechanism has a curve that flattens at ε, because no Rényi divergence exceeds its largest log-ratio of probabilities, so a line fits it only near $\alpha = 1$. For the subsampled Gaussian of differentially private stochastic gradient descent (SGD), DP-SGD for short, the line fits so badly that zCDP throws away everything subsampling gained. **Truncated CDP** repairs this by asking for the line only on small orders.

Two running examples carry the chapter. The first is one Gaussian step with $\sigma = 1$, which is 0.5-zCDP; at $\delta = 10^{-5}$ its exact ε is 4.377, and zCDP reports 4.728. The second is a DP-SGD run of $T = 10{,}000$ steps, each on a random sample that includes each record with probability $q = 0.01$, with $\sigma = 1$. At the same δ, RDP accounting puts that run at $\varepsilon = 6.719$, while zCDP reports 5475.8.

31.1. The definition of zCDP

You add Gaussian noise to a count, a mean or a gradient, again and again, and you want one number that says how much privacy you have spent so far. Bun and Steinke (2016) turned the Gaussian's straight RDP curve into a definition: describe a mechanism by the slope of a line that lies above its RDP curve.

Definition 31.1 (Zero-concentrated differential privacy)**.** A mechanism M is **ρ-zCDP** if, for all neighbouring datasets $D \sim D'$ and all orders $\alpha > 1$,

$$D_\alpha(M(D) \,\|\, M(D')) \le \rho\alpha. \tag{31.1}$$

The name comes from what the definition says about the privacy loss: it is concentrated around its mean like a normal variable, and "zero" says the line has no intercept. Fix a neighbouring pair and write $P = M(D)$ and $Q = M(D')$, with $P(y)$ and $Q(y)$ their probabilities or densities

at an output y. Capital letters are used because q is the sampling rate later in this chapter. The **privacy loss** $L(y) = \ln(P(y)/Q(y))$ says how much more likely y is under D than under D', and drawing y from P makes it a random variable L. Divergences shows that the Rényi divergence is a rescaled logarithm of the moment generating function (MGF) of this loss (Equation 3.13):

$$(\alpha - 1)D_\alpha(P\|Q) = \ln \mathbb{E}_P\big[e^{(\alpha-1)L}\big]. \tag{31.2}$$

So zCDP bounds every exponential moment of the loss, and exponential moments control how often the loss is large.

Proposition 31.1 (The moment and tail form)**.** *If M is ρ-zCDP, then for every neighbouring pair the privacy loss L satisfies*

$$\ln \mathbb{E}[e^{(\alpha-1)L}] \le (\alpha - 1)\alpha\rho \quad \textit{for all } \alpha > 1, \tag{31.3}$$

and its upper tail falls off like a normal tail around ρ:

$$\Pr[L > \rho + t] \le e^{-t^2/(4\rho)} \quad \textit{for all } t > 0. \tag{31.4}$$

Proof.

1. *The moment form.* Multiply Equation 31.1 by $\alpha - 1$, which is positive, so the inequality keeps its direction. By Equation 31.2 the left side becomes $\ln \mathbb{E}[e^{(\alpha-1)L}]$, which gives Equation 31.3.
2. *A tail bound at each order.* The positive variable $e^{(\alpha-1)L}$ is large exactly when L is, so the Chernoff bound (Theorem 2.3) applies Markov's inequality to it. For any level τ, it gives

   $$\Pr[L > \tau] \le \mathbb{E}\big[e^{(\alpha-1)L}\big]e^{-(\alpha-1)\tau}, \tag{31.5}$$

   which Equation 31.3 bounds by $e^{(\alpha-1)(\alpha\rho-\tau)}$.
3. *The best order.* Use the order that gives the smallest bound. Put $\tau = \rho + t$ and write $\lambda = \alpha - 1$, so that $\alpha\rho - \tau = \lambda\rho - t$. The exponent becomes $\lambda^2\rho - \lambda t$, a parabola in λ whose smallest value, at $\lambda = t/(2\rho)$, is $-t^2/(4\rho)$.

□

Three properties make the definition useful, and each is inherited from a property of the Rényi divergence.

Theorem 31.1 (The Gaussian mechanism, composition and post-processing)**.**

1. *The Gaussian mechanism, which adds $\mathcal{N}(0, \sigma^2 I_d)$ noise to a query f with ℓ_2 sensitivity $\Delta_2 f$, is ρ-zCDP with $\rho = (\Delta_2 f)^2/(2\sigma^2)$. With the part's noise multiplier, which measures σ in units of the sensitivity, $\rho = 1/(2\sigma^2)$.*
2. *If M_1 is ρ_1-zCDP and M_2 is ρ_2-zCDP, possibly chosen after seeing the output of M_1, the pair is $(\rho_1 + \rho_2)$-zCDP.*
3. *Any function of the output of a ρ-zCDP mechanism is ρ-zCDP.*

Proof.

1. *The Gaussian.* For two normal distributions with the same variance σ^2, D_α is α times the squared distance between the means, divided by $2\sigma^2$ (Equation 3.19). This holds in any dimension, because rotating the coordinates so that the means differ along one axis leaves the other axes identical under both, and identical independent coordinates add nothing (additivity in Theorem 3.2). On neighbours the distance is at most $\Delta_2 f$, which gives the claimed ρ.
2. *Composition.* The RDP curves of an adaptive composition add (Rényi DP), so the pair's curve is at most $\rho_1\alpha + \rho_2\alpha$, a line of slope $\rho_1 + \rho_2$.
3. *Post-processing.* The Rényi divergence obeys the data-processing inequality (Theorem 3.5), so passing the output through any function cannot raise the curve.

□

For the Gaussian the definition holds with equality in every moment. Its privacy loss is normal with mean ρ and variance 2ρ (Lemma 20.3), and the logarithm of the MGF of that normal at $\lambda = \alpha - 1$ is $\lambda\rho + \lambda^2\rho = (\alpha-1)\alpha\rho$. So a ρ-zCDP mechanism is one whose privacy loss is, in every exponential moment, no worse than $\mathcal{N}(\rho, 2\rho)$. For other mechanisms the smallest valid ρ is the largest ratio $\varepsilon(\alpha)/\alpha$, the rest of the curve is forgotten, and the later sections measure that cost.

The tail bound Equation 31.4 is loose only by a polynomial factor. For the Gaussian the exact tail is a standard normal tail at $z = t/\sqrt{2\rho}$, while the bound is $e^{-z^2/2}$. The Mills ratio of Concentration says that a normal tail at z is about $e^{-z^2/2}/(z\sqrt{2\pi})$, so the exact tail divided by the bound is about $1/(z\sqrt{2\pi}) = \sqrt{\rho/\pi}/t$.

Example 31.1. One Gaussian step with $\sigma = 1$ is 0.5-zCDP, and ten of them are 5-zCDP. A million simulated privacy losses (seed 0) have mean 0.4990 and variance 1.0013, against $\rho = 0.5$ and $2\rho = 1$. Their $\ln \mathbb{E}[e^L]$ is 1.0000, against $(\alpha-1)\alpha\rho = 1$ at $\alpha = 2$. Table 31.1 compares the tail with its bound. At $t = 4$ their ratio is 0.094, against $\sqrt{0.5/\pi}/4 = 0.100$ from the Mills ratio.

Table 31.1.: Upper tail of the Gaussian privacy loss, $\sigma = 1$: exact, simulated (10^6 draws) and the bound Equation 31.4.

t	Exact $\Pr[L > \rho + t]$	Simulated	zCDP bound $e^{-t^2/(4\rho)}$
1	1.587×10^{-1}	1.586×10^{-1}	6.065×10^{-1}
2	2.275×10^{-2}	2.261×10^{-2}	1.353×10^{-1}
4	3.167×10^{-5}	2.700×10^{-5}	3.355×10^{-4}

31.2. Pure DP implies zCDP

A real pipeline mixes Gaussian steps with ε-DP ones, such as a Laplace count, a randomized-response survey bit or an exponential-mechanism selection. To put them in one budget, you need their ρ. The privacy loss of such a mechanism lies in $[-\varepsilon, \varepsilon]$, and a bounded variable has Gaussian-like tails, so ρ should be of order ε^2. The question is the constant.

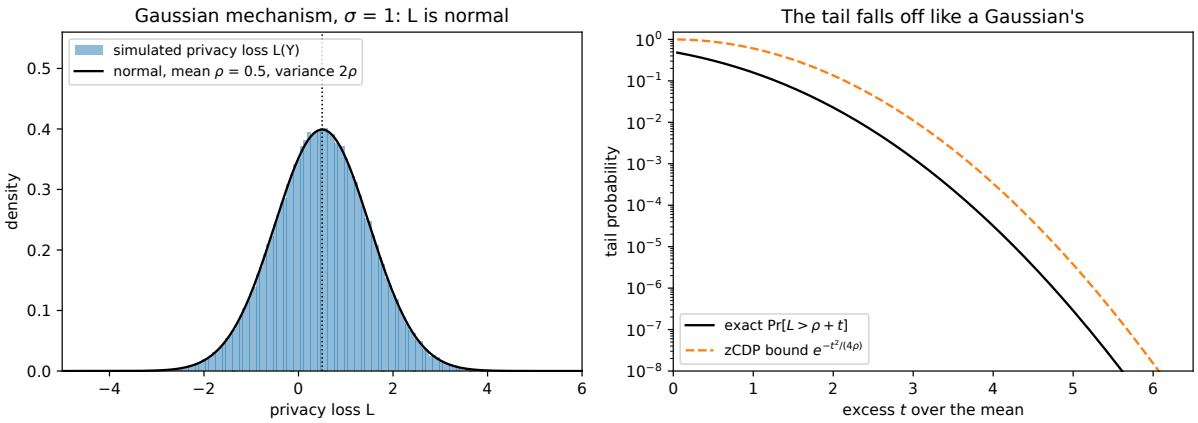


Figure 31.1.: The privacy loss of the Gaussian mechanism with $\sigma = 1$. Left: a histogram of $L(Y)$ from simulation against the normal density with mean $\rho = 0.5$ and variance $2\rho = 1$. Right: the exact upper tail $\Pr[L > \rho + t]$ against the zCDP bound $e^{-t^2/(4\rho)}$, on a log scale.

The tool is Hoeffding's lemma from Concentration (Lemma 2.2). A variable X confined to an interval of length w has an MGF no larger than that of a normal variable with the same mean and variance $w^2/4$:

$$\ln \mathbb{E}[e^{\lambda X}] \le \lambda \mathbb{E}[X] + \frac{\lambda^2 w^2}{8} \quad \text{for every real } \lambda. \tag{31.6}$$

Applied once to bound the mean of the loss and once to bound its exponential moments, it gives the constant $\frac{1}{2}$.

Theorem 31.2 (Pure DP implies zCDP)**.** *If M is ε-DP, then M is $(\varepsilon^2/2)$-zCDP (Bun & Steinke 2016, Proposition 1.4), and its RDP curve lies under both the line and the cap ε:*

$$\varepsilon(\alpha) \le \min\left(\varepsilon, \ \frac{\alpha\varepsilon^2}{2}\right). \tag{31.7}$$

Proof. The plan is to bound the mean of the loss with Hoeffding's lemma, then its exponential moments with the same lemma, and add the two. The cap comes separately.

1. *The loss is bounded.* Pure DP holds output by output (Proposition 18.1), so ε-DP for both orders of the pair gives $e^{-\varepsilon} \le P(y)/Q(y) \le e^{\varepsilon}$ wherever either probability is positive. So $L \in [-\varepsilon, \varepsilon]$, and P and Q have the same support.
2. *The mean is at most $\varepsilon^2/2$.* Apply Hoeffding's lemma to $X = -L$, which lies in an interval of length 2ε, with $\lambda = 1$:
$$\ln \mathbb{E}_P[e^{-L}] \le -\mathbb{E}_P[L] + \frac{\varepsilon^2}{2}. \tag{31.8}$$
The left side is 0, because $e^{-L} = Q(Y)/P(Y)$ averages to 1 under P when the supports agree (Lemma 3.2). So $\mathbb{E}_P[L] \le \varepsilon^2/2$.
3. *The exponential moments.* Apply the lemma again, now to L itself with $\lambda = \alpha - 1$:
$$\ln \mathbb{E}_P[e^{(\alpha-1)L}] \le (\alpha-1)\mathbb{E}_P[L] + \frac{(\alpha-1)^2\varepsilon^2}{2}. \tag{31.9}$$

4. *Combine.* Putting the mean bound of step 2 into step 3 bounds the right side by $(\alpha-1)\varepsilon^2/2 + (\alpha-1)^2\varepsilon^2/2 = (\alpha-1)\alpha\,\varepsilon^2/2$. By Equation 31.2, that is $D_\alpha \le \alpha\varepsilon^2/2$.
5. *The cap.* D_α is non-decreasing in α and tends to the max-divergence D_∞, the largest log-ratio of probabilities over events (Theorem 3.2). ε-DP bounds that log-ratio by ε.

□

Hoeffding treats the loss as an arbitrary bounded variable. A sharper route uses the fact that **randomized response**, which reports a bit truthfully with probability $e^\varepsilon/(1+e^\varepsilon)$ and flips it otherwise, is the worst ε-DP mechanism: every other one is a post-processing of it. Its MGF is a ratio of hyperbolic cosines, and step 3 of the proof below bounds it with one inequality: $\ln\cosh x$ is a concave function of x^2, so it lies below its tangent.

Lemma 31.1 (A tangent bound for ln cosh)**.** *For every $a > 0$ and every real x,*

$$\ln\cosh x - \ln\cosh a \le (x^2 - a^2)\frac{\tanh a}{2a}. \tag{31.10}$$

Proof.

1. *Change variable.* Let $h(s) = \ln\cosh\sqrt{s}$ for $s \ge 0$, so that the claim is $h(x^2) - h(a^2) \le (x^2-a^2)h'(a^2)$. By the chain rule, $h'(s) = \tanh(\sqrt{s})/(2\sqrt{s})$, which at $s = a^2$ is $\tanh(a)/(2a)$.
2. *The derivative decreases.* The function $\tanh(y)/y$ decreases for $y > 0$, because its derivative has the sign of $y - \sinh y\cosh y$, which is negative. So h' decreases, and h is concave.
3. *Tangent line.* A concave function lies below each of its tangent lines, and the claim is the tangent at $s = a^2$.

□

Theorem 31.3 (The sharp constant)**.** *Every ε-DP mechanism is $\varepsilon\tanh(\varepsilon/2)$-zCDP:*

$$\varepsilon(\alpha) \le \alpha\,\varepsilon\tanh(\varepsilon/2). \tag{31.11}$$

No smaller ρ holds for randomized response. More generally, the exact RDP curve of randomized response is a valid RDP curve for every ε-DP mechanism.

Proof. The plan is to reduce every ε-DP pair to randomized response, compute its MGF exactly, bound it with Lemma 31.1, and show that the bound is reached as $\alpha \to 1$.

1. *Reduce to randomized response.* Let $P^\star$ and $Q^\star$ be the output distributions of randomized response on the bits 0 and 1. Every ε-DP pair is a post-processing of them: there is a channel K with $P = P^\star K$ and $Q = Q^\star K$ (Theorem 21.2 with $\delta = 0$). By the data-processing inequality (Theorem 3.5), $D_\alpha(P\|Q) \le D_\alpha(P^\star\|Q^\star)$ at every order, which is the last claim.
2. *The MGF of randomized response.* Under $P^\star$ the truthful answer has probability $p = e^\varepsilon/(1+e^\varepsilon)$ and likelihood ratio $p/(1-p) = e^\varepsilon$. So the loss is $+\varepsilon$ with probability p and $-\varepsilon$ otherwise. With $\lambda = \alpha - 1$, averaging $e^{\lambda L}$ over the two values gives

$$\mathbb{E}[e^{\lambda L}] = \frac{e^{(\lambda+1)\varepsilon} + e^{-\lambda\varepsilon}}{1+e^\varepsilon}. \tag{31.12}$$

Factoring $e^{\varepsilon/2}$ out of the numerator and the denominator turns each into twice a hyperbolic cosine:

$$\mathbb{E}[e^{\lambda L}] = \frac{\cosh\left((\lambda+\frac{1}{2})\varepsilon\right)}{\cosh(\varepsilon/2)}. \tag{31.13}$$

3. *Bound it.* Take $x = (\lambda+\frac{1}{2})\varepsilon$ and $a = \varepsilon/2$ in Lemma 31.1. Then $x^2 - a^2 = \varepsilon^2\lambda(\lambda+1)$ and $\tanh(a)/(2a) = \tanh(\varepsilon/2)/\varepsilon$, so

$$\ln \mathbb{E}[e^{\lambda L}] \le (\alpha-1)\alpha\,\varepsilon\tanh(\varepsilon/2). \tag{31.14}$$

   By Equation 31.2 and step 1, this gives Equation 31.11.
4. *No smaller ρ.* As $\alpha \to 1$, any RDP curve tends to the Kullback–Leibler (KL) divergence, the mean loss (Theorem 3.2). For randomized response that mean is $\varepsilon(2p-1) = \varepsilon(e^\varepsilon - 1)/(e^\varepsilon+1) = \varepsilon\tanh(\varepsilon/2)$. So $\varepsilon(\alpha)/\alpha$ tends to $\varepsilon\tanh(\varepsilon/2)$, and any smaller slope fails near $\alpha = 1$.

□

Since $\tanh x \le x$, the sharp constant is below $\varepsilon^2/2$, and the two agree as $\varepsilon \to 0$. At $\varepsilon = 0.5$ the sharp ρ is 0.1225 against 0.1250, and at $\varepsilon = 1$ it is 0.4621 against 0.5000. Mechanisms other than randomized response can need less: the Laplace mechanism needs 0.3679 at $\varepsilon = 1$.

Any line fits poorly far from $\alpha = 1$, because the true curve flattens at the cap (Figure 31.2). At $\varepsilon = 1$ the line $\alpha\varepsilon^2/2$ crosses the cap at $\alpha = 2$, and at $\varepsilon = 3$ its slope 4.5 puts it above the cap at every order. Since every ε-DP pair is a post-processing of randomized response, an RDP accountant can instead charge a pure-DP step the curve of randomized response, while a zCDP accountant can keep only a line. For large ε, as in local DP, account the mechanism by its curve or by ε, not by a ρ.

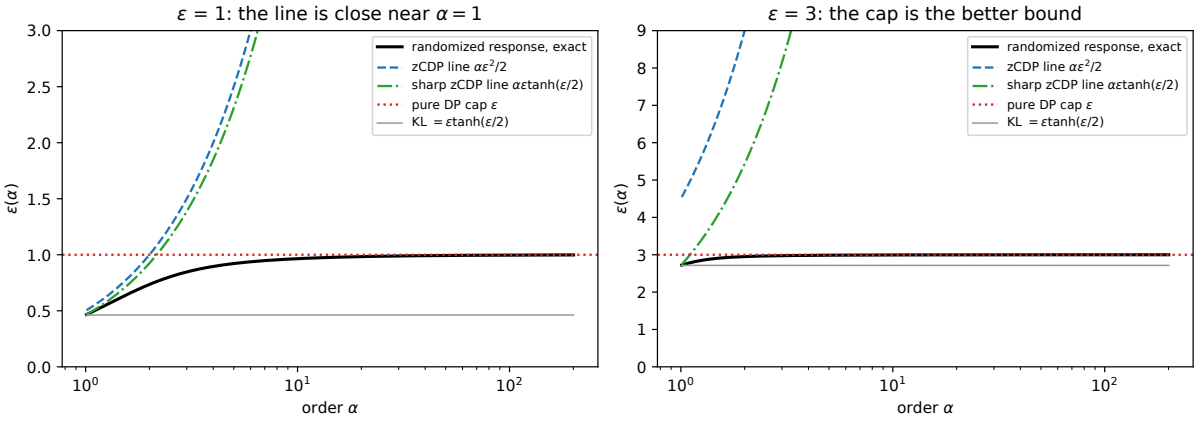


Figure 31.2.: The RDP curve of randomized response (black) against the zCDP line $\alpha\varepsilon^2/2$ (dashed), the sharp zCDP line $\alpha\varepsilon\tanh(\varepsilon/2)$ (dash-dotted) and the pure-DP cap ε (dotted), for $\varepsilon = 1$ (left) and $\varepsilon = 3$ (right). The grey line is the KL divergence $\varepsilon\tanh(\varepsilon/2)$, the curve's value at $\alpha \to 1$.

The exponential mechanism, which selects one candidate and favours those with higher scores, costs four times less than $\varepsilon^2/2$. Its privacy loss on a neighbouring pair lies in an interval of length ε, not 2ε, which is the **bounded-range** property of Durfee and Rogers (2019). Hoeffding's lemma then pays a quarter of the variance term.

Proposition 31.2 (Bounded-range mechanisms)**.** *If, for every neighbouring pair taken in either order, the privacy loss of M lies in an interval of length ε, then M is $(\varepsilon^2/8)$-zCDP. An ε-DP exponential mechanism has this property, so it is $(\varepsilon^2/8)$-zCDP (Cesar & Rogers 2021).*

Proof.

1. *The mean.* The loss is finite in both orders, so P and Q have the same support and $\mathbb{E}_P[e^{-L}] = 1$, as in step 2 of Theorem 31.2. Hoeffding's lemma with length ε and $\lambda = 1$ then gives $0 \le -\mathbb{E}_P[L] + \varepsilon^2/8$.
2. *The exponential moments.* The lemma with $\lambda = \alpha - 1$ adds $(\alpha-1)^2\varepsilon^2/8$ to $(\alpha-1)\mathbb{E}_P[L]$.
3. *Combine.* With step 1 the sum is at most $(\alpha-1)\alpha\,\varepsilon^2/8$, which by Equation 31.2 is $D_\alpha \le \alpha\varepsilon^2/8$.

□

Example 31.2. Randomized response at $\varepsilon = 1$ reports the truth with probability $e/(1+e) = 0.7311$. Table 31.2 compares its exact curve with the bounds. The third column puts the sharp mean bound $\varepsilon\tanh(1/2) = 0.4621$, which holds for every ε-DP pair (Theorem 21.3), into step 4 of the Hoeffding proof in place of $\varepsilon^2/2$. That line starts lower but keeps the slope 0.5, which comes from Hoeffding's variance term, so it sits $0.5 - 0.4621 = 0.0379$ below the zCDP line at every order.

Table 31.2.: Randomized response at $\varepsilon = 1$: the exact RDP curve against the Hoeffding bound with the sharp mean, the zCDP line and the line capped at ε.

α	Exact $\varepsilon(\alpha)$	$\varepsilon\tanh(\varepsilon/2) + (\alpha-1)\varepsilon^2/2$	$\alpha\varepsilon^2/2$	$\min(\varepsilon, \alpha\varepsilon^2/2)$
1.5	0.6273	0.7121	0.7500	0.7500
2	0.7353	0.9621	1.0000	1.0000
5	0.9217	2.4621	2.5000	1.0000
10	0.9652	4.9621	5.0000	1.0000
50	0.9936	24.9621	25.0000	1.0000

Up to $\alpha = 2$ the zCDP line is within 36% of the exact value ($1.0000/0.7353 = 1.360$). At $\alpha = 50$ it is 25 times too large ($25.0000/0.9936 = 25.2$), while the cap is within 1% ($1.0000/0.9936 = 1.0064$).

31.3. From zCDP to (ε, δ)

Whatever you account in, you report (ε, δ). Mironov's conversion turns an RDP curve into such a guarantee (From Rényi DP to (ε, δ) and to tests). At each order α, a mechanism with curve value $\varepsilon(\alpha)$ is $(\varepsilon(\alpha) + \ln(1/\delta)/(\alpha-1), \delta)$-DP, and you take the best order. The improved conversion of the same chapter avoids part of the slack of Markov's inequality, and subtracts $\ln(\alpha)/(\alpha-1) - \ln(1 - 1/\alpha)$ from this. For a ρ-zCDP mechanism the curve is the line $\rho\alpha$, so the best order can be found by hand. Section D.4 in the appendices derives the result from scratch, with worked numbers.

Theorem 31.4 (From zCDP to (ε, δ))**.** *A ρ-zCDP mechanism is (ε, δ)-DP for every $\delta \in (0, 1)$ with*

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}, \tag{31.15}$$

reached at the order $\alpha^* = 1 + \sqrt{\ln(1/\delta)/\rho}$ (Bun & Steinke 2016, Proposition 1.3). It is also (ε, δ)-DP with the smaller value

$$\varepsilon = \min_{\alpha > 1} \left[\rho\alpha + \frac{\ln(1/\delta)}{\alpha - 1} + \ln\left(1 - \frac{1}{\alpha}\right) - \frac{\ln \alpha}{\alpha - 1} \right] \tag{31.16}$$

(Canonne, Kamath & Steinke 2020, Proposition 12 and Corollary 13).

Proof. Write $\ell = \ln(1/\delta)$.

1. *Plug in the line.* Mironov's conversion with $\varepsilon(\alpha) = \rho\alpha$ says that every $\alpha > 1$ gives a valid ε:

$$g(\alpha) = \rho\alpha + \frac{\ell}{\alpha - 1}. \tag{31.17}$$

   The first term grows with the order and the second shrinks, so the best order lies in between.
2. *Minimize.* The derivative $\rho - \ell/(\alpha - 1)^2$ is zero at $\alpha - 1 = \sqrt{\ell/\rho}$. Both terms of g are convex for $\alpha > 1$, so this stationary point is the minimum.
3. *Evaluate.* At α^* the first term is $\rho + \sqrt{\rho\ell}$ and the second is $\ell/\sqrt{\ell/\rho} = \sqrt{\rho\ell}$, and their sum is Equation 31.15.
4. *The improved form.* Putting $\varepsilon(\alpha) = \rho\alpha$ into the improved conversion gives the bracket of Equation 31.16 at each order, and every order is valid.

□

For calibration you need the reverse: the largest ρ whose conversion stays within a target ε at a given δ. The simple ε grows with ρ, and solving Equation 31.15 for ρ shows that the target is met by every

$$\rho \le \left(\sqrt{\varepsilon + \ln(1/\delta)} - \sqrt{\ln(1/\delta)} \right)^2. \tag{31.18}$$

Keep budgets in ρ and convert once, at the end: ε grows like the square root of ρ, so adding the ε's of two halves of a budget overstates the whole.

How much do the conversions lose? The Gaussian answers this, because the zCDP line is exact for it, so any gap is the price of the conversion alone. After dividing by σ, a ρ-zCDP Gaussian mechanism is the pair $\mathcal{N}(0, 1)$ against $\mathcal{N}(\mu, 1)$ with $\mu = \sqrt{2\rho}$, and Theorem 20.3 gives its exact smallest δ at each ε (Equation 20.17). At $\rho = 0.5$ the simple formula is 21% too large ($5.299/4.377 = 1.210$), and the improved conversion 8% ($4.728/4.377 = 1.080$), and both gaps shrink as ρ grows (Figure 31.3). The part's accountants minimize over a fixed list of orders, `ORDERS`, and the improved conversion on that list agrees with a fine grid to within 0.001 for $\rho \le 1$.

Example 31.3. k Gaussian steps with $\sigma = 1$ are $(k/2)$-zCDP, since each costs $\frac{1}{2}$ and composition adds. Table 31.3 converts a range of ρ at $\delta = 10^{-5}$, where $\ln(1/\delta) = 11.5129$. The row $\rho = 0.5$ is the single step of the running example. Its best order for the simple formula is $\alpha^* = 1 + \sqrt{11.5129/0.5} = 5.799$, and the simple ε is $0.5 + 2\sqrt{0.5 \times 11.5129} = 5.299$. The row $\rho = 5$ is ten steps, and $\rho = 50$ a hundred.

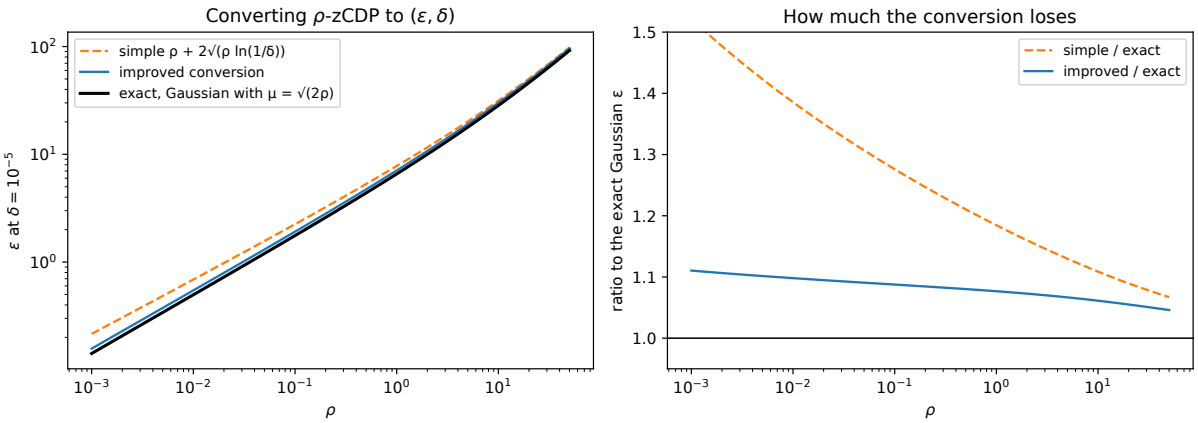


Figure 31.3.: Left: ε at $\delta = 10^{-5}$ for a ρ-zCDP mechanism, from the simple formula Equation 31.15, the improved conversion Equation 31.16, and the exact value for the Gaussian mechanism with $\mu = \sqrt{2\rho}$. Right: each conversion divided by the exact Gaussian value.

Table 31.3.: ε at $\delta = 10^{-5}$ for ρ-zCDP: simple formula, improved conversion on a fine grid of orders and on the part's `ORDERS`, and the exact ε of the Gaussian mechanism with the same ρ.

ρ	$\mu = \sqrt{2\rho}$	Simple Equation 31.15	Improved, fine grid	Improved, `ORDERS`	Exact Gaussian
0.01	0.1414	0.689	0.546	0.546	0.497
0.1	0.4472	2.246	1.914	1.914	1.760
0.5	1.0000	5.299	4.728	4.729	4.377
1	1.4142	7.786	7.077	7.077	6.573
5	3.1623	20.174	19.047	19.054	17.857
50	10.0000	97.985	96.035	96.116	91.817

To make a single Gaussian step $(1, 10^{-5})$-DP through Equation 31.18, take $\rho = (\sqrt{12.5129} - \sqrt{11.5129})^2 = 0.02082$, that is $\sigma = 1/\sqrt{2\rho} = 4.901$.

31.4. Group privacy

A household or a user with many records contributes several records at once. Group privacy asks how the guarantee degrades for datasets joined by a chain of m neighbouring steps, writing m because k counts composed mechanisms. For pure DP the answer is $m\varepsilon$ (Theorem 19.2). For RDP the chain needs a triangle inequality, which the Rényi divergence lacks at a fixed order, so Mironov's group bound gives up orders: for a group of 2^c, a curve value ε at order α becomes $3^c\varepsilon$ at order $\alpha/2^c$ (Rényi DP). zCDP constrains every order at once, so it can pay for the chain with higher orders.

The proof walks along the chain one record at a time. Each step must pass from P to Q through an intermediate distribution R, and the tool is a triangle-like inequality that pays for the detour with higher orders on the two legs (Bun & Steinke 2016, Lemma 5.2).

Lemma 31.2 (A triangle-like inequality for Rényi divergence)**.** *For distributions P, Q, R, every order $\alpha > 1$ and every $m > 1$,*

$$D_\alpha(P\|Q) \le \frac{m\alpha}{m\alpha - 1} D_{\frac{m\alpha-1}{m-1}}(P\|R) + D_{m\alpha}(R\|Q). \tag{31.19}$$

Proof. The plan is to write the sum that defines $D_\alpha(P\|Q)$ as an average under R of a product of two ratios, split the product with Hölder's inequality, and recognize each factor as a Rényi divergence of one leg.

1. *Insert R.* Assume R is positive wherever P or Q is, since otherwise the right side is infinite. Multiplying out one term shows $R \cdot P^\alpha R^{-\alpha} \cdot R^{\alpha-1} Q^{1-\alpha} = P^\alpha Q^{1-\alpha}$. So the sum that defines the divergence is an average under R (Lemma 3.2), with the ratios evaluated at an output drawn from R:

$$e^{(\alpha-1)D_\alpha(P\|Q)} = \sum_y P(y)^\alpha Q(y)^{1-\alpha} = \mathbb{E}_R\big[(P/R)^\alpha (R/Q)^{\alpha-1}\big]. \tag{31.20}$$

2. *Split with Hölder.* Hölder's inequality bounds the average of a product of two positive variables X_1 and X_2 by a product of power means, for exponents u and v with $\frac{1}{u} + \frac{1}{v} = 1$:

$$\mathbb{E}[X_1 X_2] \le (\mathbb{E}[X_1^u])^{1/u} (\mathbb{E}[X_2^v])^{1/v}. \tag{31.21}$$

   The second average will be a divergence of order $v(\alpha-1)+1$, so choose $v = \frac{m\alpha-1}{\alpha-1}$ to make that order $m\alpha$, and the matching $u = \frac{m\alpha-1}{\alpha(m-1)}$. Then

$$\mathbb{E}_R\big[(P/R)^\alpha (R/Q)^{\alpha-1}\big] \le \big(\mathbb{E}_R[(P/R)^{u\alpha}]\big)^{1/u} \big(\mathbb{E}_R[(R/Q)^{v(\alpha-1)}]\big)^{1/v}. \tag{31.22}$$

3. *Recognize two divergences.* Write $\alpha_1 = u\alpha = \frac{m\alpha-1}{m-1}$ and $\alpha_2 = v(\alpha-1)+1 = m\alpha$. Multiplying out as in step 1, the first average is $\sum_y R^{1-\alpha_1} P^{\alpha_1} = e^{(\alpha_1-1)D_{\alpha_1}(P\|R)}$, and the second is $\sum_y R^{\alpha_2} Q^{1-\alpha_2} = e^{(\alpha_2-1)D_{\alpha_2}(R\|Q)}$.
4. *Take logarithms and divide by $\alpha-1$.* The second leg contributes $D_{m\alpha}(R\|Q)$, since $(\alpha_2-1)/v = \alpha - 1$. The first contributes $D_{\alpha_1}(P\|R)$ times $\frac{\alpha_1-1}{u(\alpha-1)}$. Since $\alpha_1 - 1 = \frac{m(\alpha-1)}{m-1}$, that coefficient is $\frac{m\alpha}{m\alpha-1}$, which gives Equation 31.19.

□

Theorem 31.5 (Group privacy)**.** *If M is ρ-zCDP and D, D' are joined by a chain of m neighbouring steps, then for every $\alpha > 1$*

$$D_\alpha(M(D)\,\|\,M(D')) \le m^2 \rho\, \alpha, \tag{31.23}$$

so M is $m^2\rho$-zCDP for groups of size m (Bun & Steinke 2016, Proposition 1.9). The Gaussian mechanism attains the bound.

Proof. The proof is by induction on m, and the case $m = 1$ is the definition.

1. *Split the chain.* Let $\hat{D}$ be $m-1$ steps from D and one step from D'. Apply Equation 31.19 with $P = M(D)$, $R = M(\hat{D})$ and $Q = M(D')$.
2. *The group of $m-1$.* By the induction hypothesis at order $\alpha_1 = \frac{m\alpha-1}{m-1}$, the first term is at most $\frac{m\alpha}{m\alpha-1}(m-1)^2\rho\,\alpha_1 = m(m-1)\rho\alpha$. The factor $m\alpha-1$ in α_1 cancels the coefficient's denominator.
3. *The last step.* The second term is a single neighbouring step, so ρ-zCDP at order $m\alpha$ bounds it by $m\rho\alpha$. The sum is $(m^2-m+m)\rho\alpha = m^2\rho\alpha$.
4. *Tightness.* For the Gaussian mechanism a group of m shifts the mean by $m\Delta_2 f$, so Equation 3.19 gives exactly $\alpha m^2(\Delta_2 f)^2/(2\sigma^2) = m^2\rho\alpha$.

□

The bound is quadratic in m, which matches pure DP: with $\rho \approx \varepsilon^2/2$, the group's $m^2\rho$ is about $(m\varepsilon)^2/2$. The RDP route multiplies ε by 3^c while dividing the order by 2^c, so its slope is $6^c\rho$ instead of $m^2\rho = 4^c\rho$ (Table 31.4). The induction can raise the order at each step only because zCDP controls every order; a bound known only up to some order would run out.

Strong group privacy is also a weakness. Bun and Steinke use it to prove lower bounds on the sample size of zCDP algorithms as strong as those for pure DP. These rule out **stability-based** algorithms, which release an exact answer when the data are far from any dataset where it would change, and Section 31.5 weakens the definition to escape them.

Example 31.4. Table 31.4 takes the Gaussian mechanism with $\sigma = 1$ ($\rho = 0.5$) and groups of size m. The RDP route starts from a high order, 64, where $\varepsilon(64) = 0.5 \times 64 = 32$. For $m = 2^c$ it gives order $64/2^c$, $\varepsilon = 3^c \times 32$ and slope $6^c \times 0.5$. The triangle-like inequality is an equality along the chain of normals with means 0, 1 and 2, where both sides of Equation 31.19 are 6 at $\alpha = 3$ and $m = 2$.

Table 31.4.: Group privacy of the Gaussian mechanism with $\sigma = 1$. zCDP gives slope $m^2\rho$ at every order, and its conversion is within about 8% of the exact value. Mironov's RDP group bound, started from order 64, gives slope $6^c\rho$ and only on orders up to $64/m$.

Group size m	zCDP $m^2\rho$	ε at 10^{-5} (improved)	Exact Gaussian, shift m	RDP route: order, ε, slope
1	0.5	4.728	4.377	—
2	2	10.725	9.997	32, 96, 3
4	8	25.919	24.382	16, 288, 18
8	32	68.616	65.319	8, 864, 108

31.5. Approximate zCDP

Some useful mechanisms are (ε, δ)-DP but not zCDP at all. Propose-test-release and stability-based histograms are of this kind: they fail with probability about δ. The lower bounds of Section 31.4 rule them out of zCDP, yet you still want them in one budget with Gaussian steps. Bun and Steinke's answer is to let zCDP fail on an event of small probability.

Definition 31.2 (Approximate zCDP)**.** A mechanism M is **δ-approximately ρ-zCDP** if, for all neighbours $D \sim D'$, there are events E and E' of probability at least $1-\delta$, for the runs on D and on D', such that for every $\alpha > 1$

$$D_\alpha(M(D)|_E \,\|\, M(D')|_{E'}) \le \rho\alpha \quad \text{and} \quad D_\alpha(M(D')|_{E'} \,\|\, M(D)|_E) \le \rho\alpha. \tag{31.24}$$

Here $M(D)|_E$ is the output distribution conditioned on E, and the events may depend on the mechanism's internal randomness, not only on its output (Bun & Steinke 2016, Definition 1.12).

Three properties make it a ledger with two columns, ρ and the probability δ_a that zCDP fails.

Proposition 31.3 (The approximate zCDP ledger)**.**

1. *An (ε,δ)-DP mechanism is δ-approximately $(\varepsilon^2/2)$-zCDP (Bun & Steinke 2016, Proposition 1.13 and Corollary 8.7).*
2. *A δ_1-approximately ρ_1-zCDP step followed by a δ_2-approximately ρ_2-zCDP step is $(\delta_1+\delta_2-\delta_1\delta_2)$-approximately $(\rho_1+\rho_2)$-zCDP (their Lemma 8.2). For many steps the failure probability is $1-\prod_i(1-\delta_i) \le \sum_i \delta_i$.*
3. *A δ_a-approximately ρ-zCDP mechanism is $(\varepsilon, \delta_a + (1-\delta_a)\delta')$-DP, where ε is any conversion of ρ-zCDP at δ' (their Lemma 8.8).*

Proof. The first two parts have short reasons. For the third, the usual conversion applies on the good event, and the bad event can add at most its probability.

1. *Absorbing (ε,δ)-DP.* Any (ε,δ)-DP pair behaves, with probability $1-\delta$, like an ε-DP pair, which Theorem 31.2 turns into $\varepsilon^2/2$. Section J.2 in the appendices derives the decomposition and the composition of mixed pipelines.
2. *Composition.* The conditioned mechanisms compose as zCDP (Theorem 31.1), and both good events hold with probability at least $(1-\delta_1)(1-\delta_2) = 1-(\delta_1+\delta_2-\delta_1\delta_2)$.
3. *Split on the event.* Let $\Pr[E], \Pr[E'] \ge 1-\delta_a$, fix a set S of outputs, and write $P_E(S) = \Pr[M(D) \in S \mid E]$. The probability of S outside E is at most 1, so

$$\Pr[M(D) \in S] \le P_E(S)\Pr[E] + 1 - \Pr[E] = 1 - \Pr[E](1 - P_E(S)). \tag{31.25}$$

   This bound shrinks as $\Pr[E]$ grows, so it is at most its value $P_E(S)(1-\delta_a)+\delta_a$ at the smallest allowed $\Pr[E]$.
4. *Use the conditional zCDP.* The conditioned pair is ρ-zCDP, so $P_E(S) \le e^\varepsilon \Pr[M(D') \in S \mid E'] + \delta'$.
5. *Remove the conditioning on D'.* Dropping the outputs outside E' can only lower a probability, so $\Pr[M(D') \in S] \ge (1-\delta_a)\Pr[M(D') \in S \mid E']$.
6. *Combine.* Putting step 4 into step 3 and using step 5 gives

$$\Pr[M(D) \in S] \le e^\varepsilon \Pr[M(D') \in S] + (1-\delta_a)\delta' + \delta_a. \tag{31.26}$$

□

The ρ column composes like zCDP, so after conversion ε grows like the square root of the number of steps, while the δ_a column adds up like δ in basic composition. The ledger is worth keeping only when some steps are genuinely (ε,δ)-DP and not zCDP.

Example 31.5. A pipeline runs 10 stability-based steps, each $(0.5, 10^{-7})$-DP, and 10 Gaussian steps with $\sigma = 1$. Each stability step enters the ρ column as $0.5^2/2 = 0.125$ and the δ_a column as 10^{-7}. The totals are $\rho = 10 \times 0.125 + 10 \times 0.5 = 6.25$ and $\delta_a = 1 - (1 - 10^{-7})^{10} = 1.0000 \times 10^{-6}$. The improved conversion at $\delta' = 10^{-5} - \delta_a$ gives $\varepsilon = 22.100$, with total $\delta_a + (1 - \delta_a)\delta' \le 10^{-5}$. Without the ledger you would convert the Gaussians alone to $\varepsilon = 19.119$ at the remaining $\delta = 10^{-5} - 10^{-6} = 9 \times 10^{-6}$ and add the stability steps by basic composition, for $19.119 + 5 = 24.119$. The ledger saves about 2, because each $\varepsilon = 0.5$ step costs $\varepsilon^2/2$ in ρ, not ε.

31.6. Subsampling, and truncated CDP

DP-SGD, the running example of this part, runs each step on a Poisson sample of the data, in which each record is used independently with probability q, the sampling rate. Subsampling makes a step much more private than the Gaussian it contains, because a record is usually not in the batch. This section asks whether zCDP sees that gain, and what to do when it does not.

Take add/remove neighbours, sensitivity 1 and noise multiplier σ, in one dimension, which is enough because only the shift along the record's direction carries evidence. Without the record the step outputs $Q = \mathcal{N}(0, \sigma^2)$. With it, the record is in the sample with probability q, shifting the mean by 1, so the output is the mixture $P = (1 - q)\mathcal{N}(0, \sigma^2) + q\,\mathcal{N}(1, \sigma^2)$. The RDP curve is $D_\alpha(P \| Q)$, because the other order is never larger (Amplification by subsampling, Equation 33.17; Section L.1). On integer orders it has a closed form. The divergence averages a product of α likelihood ratios, as if the step ran α times, and the term j below collects the runs in which the record was sampled j times.

Lemma 31.3 (The subsampled Gaussian on integer orders)**.** *For every integer $\alpha \ge 2$, the curve is a binomial sum:*

$$\varepsilon(\alpha) = \frac{1}{\alpha - 1} \ln A_\alpha, \qquad A_\alpha = \sum_{j=0}^{\alpha} \binom{\alpha}{j} (1 - q)^{\alpha - j} q^j \exp\left(\frac{j^2 - j}{2\sigma^2}\right) \tag{31.27}$$

(Mironov, Talwar & Zhang 2019).

Proof.

1. *Write the ratio.* Let r be the likelihood ratio of $\mathcal{N}(1, \sigma^2)$ to Q. Then $P/Q = 1 - q + q\,r$, and writing the divergence as an average under Q of the ratio to the power α (Proposition 3.1) gives $A_\alpha = e^{(\alpha - 1) D_\alpha(P \| Q)} = \mathbb{E}_Q[(1 - q + q\,r)^\alpha]$.
2. *Expand.* For integer α, the binomial theorem expands the power into the terms $\binom{\alpha}{j}(1 - q)^{\alpha - j} q^j\, \mathbb{E}_Q[r^j]$.
3. *Each moment.* By the same form of Proposition 3.1, $\mathbb{E}_Q[r^j] = e^{(j-1)D_j}$ for the two normals, and Equation 3.19 gives $D_j = j/(2\sigma^2)$, so the moment is $e^{(j^2 - j)/(2\sigma^2)}$.

□

Proposition 31.4 (No amplification for zCDP)**.** *For the Poisson-subsampled Gaussian with any $q \in (0, 1]$, the smallest ρ with $\varepsilon(\alpha) \le \rho\alpha$ for all $\alpha > 1$ is $1/(2\sigma^2)$, the same as without subsampling. On integer orders the curve obeys*

$$\varepsilon(\alpha) \ge \frac{\alpha}{2\sigma^2} + \frac{\alpha \ln q}{\alpha - 1}. \tag{31.28}$$

Proof. Write $\rho_0 = 1/(2\sigma^2)$ for the slope without subsampling. The plan is to show that ρ_0 is enough, because subsampling never hurts, and that nothing smaller is enough, because at large orders subsampling stops helping.

1. *Upper bound, by convexity.* The ratio $1 - q + q\,r$ is an average of 1 and r, and $x \mapsto x^\alpha$ is convex, so the α-th power of that average is at most $(1 - q) + q\,r^\alpha$. Averaging under Q, with $\mathbb{E}_Q[r^\alpha] = e^{(\alpha-1)\alpha\rho_0}$ from the Gaussian curve, gives

   $$A_\alpha \le 1 - q + q\,e^{(\alpha-1)\alpha\rho_0}. \tag{31.29}$$

   An average of 1 and a number at least 1 is at most that number, so $A_\alpha \le e^{(\alpha-1)\alpha\rho_0}$, that is $D_\alpha(P\|Q) \le \rho_0\alpha$.
2. *The other order.* zCDP needs both orders of the pair, so $D_\alpha(Q\|P)$ needs the same bound. The sum $\sum_y V(y)(U(y)/V(y))^\alpha$ is jointly convex in the pair of distributions (U, V): its value at a mixture of pairs is at most the mixture of its values (Section F.5 proves it). Writing (Q, P) as the mixture $(1-q)(Q, Q) + q\,(Q, \mathcal{N}(1, \sigma^2))$, and using the Gaussian curve for the second pair, gives the same bound.
3. *Lower bound, by one term.* Every term of Equation 31.27 is positive, so keeping only $j = \alpha$, the runs in which the record was always sampled, gives $A_\alpha \ge q^\alpha e^{(\alpha^2-\alpha)\rho_0}$. Taking logarithms and dividing by $\alpha - 1$ gives Equation 31.28.
4. *Let the order grow.* Dividing Equation 31.28 by α gives $\varepsilon(\alpha)/\alpha \ge \rho_0 + \ln q/(\alpha - 1)$, which tends to ρ_0, because the penalty for a rarely sampled record, $\ln q/(\alpha - 1)$, vanishes as the order grows. So a line above the whole curve needs $\rho \ge \rho_0$, and step 1 shows that ρ_0 suffices.

□

The curve bends, and a line cannot (Figure 31.4). For $q = 0.01$ and $\sigma = 1$, the curve up to $\alpha = 8$ is tiny and almost linear, with slope close to $q^2(e^{1/\sigma^2} - 1)/2 = 8.5914 \times 10^{-5}$. Expanding $(1 + q(r - 1))^\alpha$ in powers of q instead, the linear term averages to 0, so $A_\alpha - 1$ starts with $\binom{\alpha}{2}q^2(e^{1/\sigma^2} - 1)$, and dividing its logarithm by $\alpha - 1$ gives that slope. Between $\alpha = 8$ and $\alpha = 16$ the curve jumps by more than three orders of magnitude, and beyond that it follows Equation 31.28, which turns positive at $\alpha = 1 + 2\sigma^2 \ln(1/q) = 10.21$. A line above the whole curve needs the slope $\rho_0 = 0.5$. A line that only has to clear the curve up to $\alpha = 8$ can have slope 1.1171×10^{-4}, which is 4476 times smaller.

Only the small orders matter for DP-SGD. After T steps the curve $T\varepsilon(\alpha)$ stays small up to the bend, while the conversion's term $\ln(1/\delta)/(\alpha - 1)$ falls with the order, so Mironov's and the improved conversion both pick an order just below the bend, $\alpha = 4$ on the integer orders of `ORDERS`. The large orders that set the zCDP slope are never used.

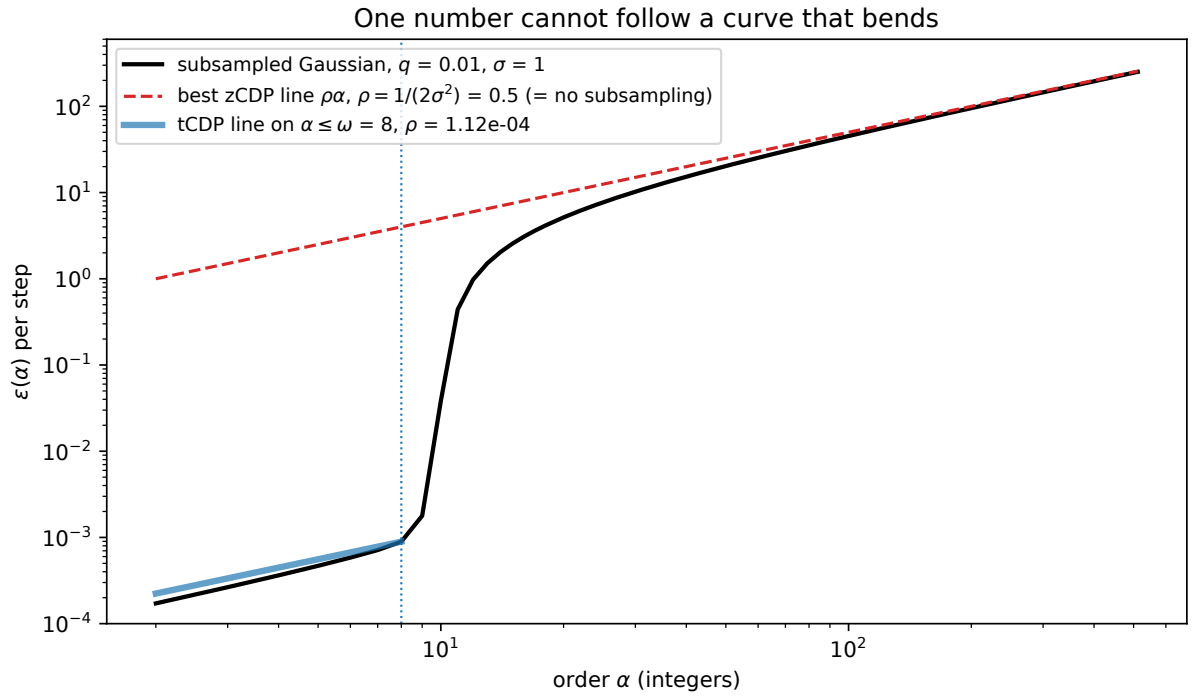


Figure 31.4.: The RDP curve of one Poisson-subsampled Gaussian step with $q = 0.01$, $\sigma = 1$, on integer orders (black), against the best zCDP line $\rho\alpha$ with $\rho = 1/(2\sigma^2) = 0.5$ (red dashed), which is the unsubsampled Gaussian's line $\alpha/2$, and the tCDP line on orders up to $\omega = 8$ (blue). Both axes are logarithmic.

Definition 31.3 (Truncated concentrated differential privacy)**.** A mechanism is (ρ, ω)**-tCDP** if, for all neighbours $D \sim D'$ and all orders $\alpha \in (1, \omega)$ below the cut-off $\omega > 1$,

$$D_\alpha(M(D) \,\|\, M(D')) \le \rho\alpha \tag{31.30}$$

(Bun, Dwork, Rothblum & Steinke 2018, Definition 1). Plain zCDP is the case $\omega = \infty$.

Composition and conversion work as for zCDP, on the covered orders.

Proposition 31.5 (Composition and conversion of tCDP)**.** *A (ρ_1, ω_1)-tCDP step followed by a (ρ_2, ω_2)-tCDP step is $(\rho_1 + \rho_2, \min(\omega_1, \omega_2))$-tCDP (their Lemma 2). A (ρ, ω)-tCDP mechanism is (ε, δ)-DP with*

$$\varepsilon = \begin{cases} \rho + 2\sqrt{\rho \ln(1/\delta)} & \text{if } \ln(1/\delta) \le (\omega - 1)^2 \rho, \\ \rho\omega + \dfrac{\ln(1/\delta)}{\omega - 1} & \text{otherwise} \end{cases} \tag{31.31}$$

(their Lemma 6).

Proof.

1. *Composition.* On orders below both cut-offs the two curves add (Rényi DP), and above the smaller cut-off one of them says nothing.
2. *The endpoint counts.* D_α is non-decreasing and lower semicontinuous in α, so $D_\omega \le \lim_{\alpha \uparrow \omega} D_\alpha \le \rho\omega$, and Mironov's conversion may use every order in $(1, \omega]$.
3. *Constrained minimum.* As in Theorem 31.4, the conversion is the convex function $g(\alpha) = \rho\alpha + \ln(1/\delta)/(\alpha - 1)$. If its minimizer $\alpha^* = 1 + \sqrt{\ln(1/\delta)/\rho}$ is at most ω, which is the first condition, it gives Equation 31.15. Otherwise g decreases on all of $(1, \omega]$, since a convex function decreases before its minimizer, and the best order is ω.

□

Unlike zCDP, tCDP is amplified by subsampling, provided the cut-off stays below about $\ln(1/q)/\rho$. The theorem uses sampling of a fixed size, not the Poisson sampling of DP-SGD, so it is a guide rather than an accountant here.

Theorem 31.6 (Subsampling amplifies tCDP)**.** *Let $\rho, q \in (0, 0.1]$ with $\ln(1/q) \ge 3\rho(2 + \log_2(1/\rho))$, and let M be (ρ, ω')-tCDP with $\omega' \ge \ln(1/q)/(2\rho) \ge 3$. Running M on a uniformly random subset of a fraction q of the records, of fixed size and without replacement, is $(13q^2\rho, \omega)$-tCDP with $\omega = \ln(1/q)/(4\rho)$ (Bun, Dwork, Rothblum & Steinke 2018, Theorem 11).*

The proof, in Section 3 of that paper, expands the Rényi sum of the two mixtures in powers of q. The second-order term carries the factor q^2, and the higher terms can be controlled only up to orders of about $\ln(1/q)/\rho$, which is where the cut-off comes from. Because the theorem covers every tCDP mechanism, its constant 13 is generous: with $\sigma = \sqrt{5}$ and $q = 0.01$ its ρ is about 11 times what the subsampled curve needs.

Example 31.6. Table 31.5 lists the per-step curve of the running DP-SGD example, $q = 0.01$ and $\sigma = 1$, on integer orders, which is what Equation 31.27 computes.

Table 31.5.: The RDP curve of one DP-SGD step ($q = 0.01$, $\sigma = 1$) on integer orders: its value, its slope from the origin, the lower bound Equation 31.28, and the Gaussian without subsampling.

α	$\varepsilon(\alpha)$	$\varepsilon(\alpha)/\alpha$	Lower bound Equation 31.28	No subsampling, $\alpha/2$
2	1.7181×10^{-4}	8.5907×10^{-5}	-8.2103	1
4	3.6315×10^{-4}	9.0789×10^{-5}	-4.1402	2
8	8.9364×10^{-4}	1.1171×10^{-4}	-1.2631	4
10	3.8270×10^{-2}	3.8270×10^{-3}	-0.1169	5
12	9.7801×10^{-1}	8.1501×10^{-2}	0.9762	6
16	3.0879	1.9299×10^{-1}	3.0878	8
64	2.7322×10^{1}	4.2690×10^{-1}	27.3217	32
512	2.5139×10^{2}	4.9099×10^{-1}	251.3858	256

The steepest ratio on the integer orders of `ORDERS` is 0.4910, at $\alpha = 512$, but zCDP needs the line at every order, and the ratio tends to 0.5. So zCDP must charge $\rho = 0.5$ per step, as without subsampling. After 10,000 steps $\rho = 5000$, and Equation 31.15 gives $\varepsilon = 5000 + 2\sqrt{5000 \times 11.5129} = 5479.9$ at $\delta = 10^{-5}$, or 5475.8 with the improved conversion. RDP accounting on the integer orders of `ORDERS` puts the same run at $\varepsilon = 7.469$ with Mironov's conversion and 6.719 with the improved one, and fractional orders do slightly better (Table 33.6).

Table 31.6 takes the per-step slope on integer orders up to ω, composes 10,000 steps by adding the slopes, and converts with Equation 31.31.

Table 31.6.: tCDP accounting of the running example, $q = 0.01$, $\sigma = 1$, $T = 10{,}000$, $\delta = 10^{-5}$, for several truncation points ω. Slopes are taken on integer orders up to ω.

ω	ρ per step	ρ after 10^4 steps	ε at 10^{-5}
2	8.5907×10^{-5}	0.8591	13.231
3	8.8213×10^{-5}	0.8821	8.403
4	9.0789×10^{-5}	0.9079	7.469
5	9.3733×10^{-5}	0.9373	7.507
6	9.7250×10^{-5}	0.9725	7.665
8	1.1171×10^{-4}	1.1171	8.289
10	3.8270×10^{-3}	38.2704	80.252
16	1.9299×10^{-1}	1929.9067	2228.027

The best integer choice, $\omega = 4$, reproduces RDP with Mironov's conversion, 7.469. There $\ln(1/\delta) = 11.51$ exceeds $(\omega - 1)^2\rho = 9 \times 0.9079 = 8.17$, so the second branch of Equation 31.31 applies; the first would give 7.374 but needs the order 4.561, beyond ω. A smaller ω forbids the orders the conversion needs, and once ω passes the bend near $\alpha = 10$, tCDP collapses towards zCDP.

31.7. Code

The script code/concentrated_dp.py computes every number in this chapter and draws every figure, checking the Gaussian privacy loss by simulation and evaluating the other curves exactly. From the repository root:

```
uv run python code/part04-privacy-accounting/concentrated_dp.py
```

Table 31.7.: The script's output with the default seed.

Experiment	Result
Gaussian privacy loss, $\sigma = 1$	Mean 0.4990, variance 1.0013, $\ln \mathbb{E}[e^L] = 1.0000$; Table 31.1
Pure DP to zCDP	Smallest ρ of randomized response 0.1225, 0.4621 at $\varepsilon = 0.5$, 1; Table 31.2
Bounded range, $\varepsilon^2/8$	Fraction used 0.9965, 0.9864, 0.8348 at $\varepsilon = 0.5$, 1, 4
zCDP to $(\varepsilon, 10^{-5})$	Table 31.3; at $\rho = 0.5$, 1.210 and 1.080 times the exact 4.377
Group privacy, $\sigma = 1$	Table 31.4; triangle-like inequality: both sides 6
Approximate zCDP pipeline	$\rho = 6.25$, $\delta_a = 1.0000 \times 10^{-6}$, $\varepsilon = 22.100$; without it 24.119
DP-SGD, $T = 10^4$	zCDP: 5479.9 (simple), 5475.8 (improved); tCDP, $\omega = 4$: 7.469; RDP: 7.469 (Mironov), 6.719 (improved)

tests/test_concentrated_dp.py checks that the output still matches.

31.8. Summary

Table 31.8.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 31.1	$D_\alpha \le \rho\alpha$ for every order and every neighbouring pair	Keep budgets in ρ; convert once, at the end
Proposition 31.1	$\ln \mathbb{E}[e^{(\alpha-1)L}] \le (\alpha-1)\alpha\rho$; $\Pr[L > \rho + t] \le e^{-t^2/(4\rho)}$	The loss is no worse than $\mathcal{N}(\rho, 2\rho)$
Theorem 31.1	Gaussian: $\rho = (\Delta_2 f)^2/(2\sigma^2)$; ρ's add; post-processing is free	$\sigma = 1$ costs $\rho = 0.5$, about $\varepsilon = 4.7$ at $\delta = 10^{-5}$

Result	Statement	Use
Theorem 31.2	ε-DP gives $(\varepsilon^2/2)$-zCDP, curve $\leq \min(\varepsilon, \alpha\varepsilon^2/2)$	Pure-DP steps in a zCDP budget; sharp only as $\varepsilon \to 0$
Theorem 31.3	ε-DP gives $\varepsilon \tanh(\varepsilon/2)$-zCDP; curve $\leq$ that of randomized response	For large ε any line passes the cap almost at once: keep the RDP curve or ε
Proposition 31.2	Bounded range ε gives $(\varepsilon^2/8)$-zCDP	The exponential mechanism; nearly tight
Theorem 31.4	$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}$, or the improved minimum over orders	Report with the improved one; calibrate by Equation 31.18
Lemma 31.2	A triangle inequality for D_α, paid for with orders $\frac{m\alpha-1}{m-1}$ and $m\alpha$	Walking a chain of datasets
Theorem 31.5	Groups of m: $m^2\rho$	Users with several records; exact for the Gaussian
Proposition 31.3	(ε, δ)-DP gives δ-approximate $(\varepsilon^2/2)$-zCDP; ρ and δ_a add	Pipelines with stability-based steps
Lemma 31.3	Subsampled Gaussian on integer orders: a binomial sum	Exact per-step curve of DP-SGD
Proposition 31.4	Smallest $\rho = 1/(2\sigma^2)$ for every q	Never account a subsampled mechanism in zCDP
Proposition 31.5	$(\sum \rho_i, \min \omega_i)$; convert by Equation 31.31	One number for DP-SGD with a small ω; matches RDP on the running example
Theorem 31.6	Sampling a fraction q gives $(13q^2\rho, \ln(1/q)/(4\rho))$-tCDP	Generic but loose; prefer RDP

31.9. References

- Bun & Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (TCC 2016; arXiv:1605.02065). The source for this chapter: the definition (1.1), the Gaussian (1.6), pure DP to zCDP (1.4, proved as 3.3), conversion (1.3, 3.5, 3.6), group privacy (1.9, 5.3), approximate zCDP (1.12, Section 8), and the lower bounds. Read Sections 1–3 first.
- Dwork & Rothblum, "Concentrated Differential Privacy" (2016, arXiv:1603.01887). The original two-parameter definition and its motivation from advanced composition.
- Bun, Dwork, Rothblum & Steinke, "Composable and Versatile Privacy via Truncated CDP" (STOC 2018). Truncated CDP: definition, composition, conversion (Lemma 6), amplification

by subsampling (Theorem 11) and the sinh-normal mechanism.

- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020). The discrete Gaussian and its zCDP guarantee, and the improved conversion from RDP and zCDP to (ε, δ) (Proposition 12, Corollary 13), with a numerically stable way to compute it.
- Cesar & Rogers, "Bounding, Concentrating, and Truncating: Unifying Privacy Loss Composition for Data Analytics" (Algorithmic Learning Theory 2021). Composition of pure DP, bounded-range (exponential mechanism) and CDP steps in one analysis, including the $\varepsilon^2/8$ bound for bounded-range mechanisms.
- Durfee & Rogers, "Practical Differentially Private Top-k Selection with Pay-what-you-get Composition" (NeurIPS 2019). Introduces the bounded-range property of the exponential mechanism.
- Dong, Durfee & Rogers, "Optimal Differential Privacy Composition for Exponential Mechanisms" (ICML 2020). Exact composition of bounded-range mechanisms, and the mean bound that Cesar & Rogers use.
- Mironov, "Rényi Differential Privacy" (CSF 2017). The RDP framework this chapter restricts to lines, and the RDP group-privacy bound used for comparison.
- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (2019, arXiv:1908.10530). The subsampled-Gaussian curve Equation 31.27.
- Abowd et al., "The 2020 Census Disclosure Avoidance System TopDown Algorithm" (Harvard Data Science Review, 2022; arXiv:2204.08986). zCDP accounting with the discrete Gaussian at national scale, and the role of invariants.

32. f-DP and Gaussian differential privacy

Every privacy definition in this part answers one question. How well can an adversary who sees the output of a mechanism M tell whether it ran on a dataset D or on a neighbour D' with one record added or removed? Differential privacy (DP) in its (ε, δ) form answers with two numbers, Rényi differential privacy (RDP) with a curve over orders α (Rényi DP), and concentrated DP with one number ρ (concentrated DP). But Part 1 already gave the complete answer. It is the **trade-off function**, which gives for each rate of false alarms the smallest rate of misses any test can reach (Hypothesis testing). This chapter makes that curve the privacy guarantee itself.

The one idea is that the whole trade-off curve between $M(D)$ and $M(D')$ is the most informative description of a mechanism's privacy, and every other notion is a lossy summary of it. Two facts make the curve practical. Gaussian noise gives a one-parameter family of curves G_μ that is closed under composition, and a central limit theorem (CLT) says that composing many mechanisms that each leak little gives a curve close to some G_μ.

Throughout, $P = M(D)$ and $Q = M(D')$ are the two output distributions. Their densities, or probabilities for discrete outputs, are written $P(y)$ and $Q(y)$, since the letter q is kept for the sampling rate. The **privacy loss** of an output y is $L(y) = \ln \frac{P(y)}{Q(y)}$. The Rényi order needs the letter α, so this chapter writes the type I and type II errors of a test as its **false positive rate** FPR and **false negative rate** FNR.

The running example is a count released with Gaussian noise of standard deviation 1, so one record moves the centre of the output by one standard deviation. An attacker who accepts 5% false positives then misses the record with probability 0.7405, and the release is $(4.3772, 10^{-5})$-DP (Section 32.3 derives both numbers). The last sections carry the same reasoning to private training, with sampling rate $q = 0.01$, noise multiplier $\sigma = 1$ and up to 10,000 steps.

32.1. f-DP

A definition of privacy should say what an adversary cannot do. The strongest such statement bounds the adversary's type II error at every type I error from below: however it tunes its test, it must miss at least this often. That bound is a curve, so the definition takes a curve as its parameter.

A test ϕ looks at the output y and fires, deciding that the data was D', with probability $\phi(y)$. So $\text{FPR} = \mathbb{E}_P[\phi]$ and $\text{FNR} = 1 - \mathbb{E}_Q[\phi]$. The trade-off function $T(P, Q)$ gives the smallest FNR at each FPR (Definition 4.2). Part 1 proved that it is non-increasing, convex, continuous and at most $1 - \text{FPR}$ (Theorem 4.2). Every function with these properties is the curve of some pair (Section K.1 in the appendices), and is called a trade-off function.

Definition 32.1 (f-DP)**.** Let f be a trade-off function. A mechanism M is **f-differentially private (f-DP)** if no neighbouring pair is easier to tell apart than f allows,

$$T(M(D), M(D')) \geq f \quad \text{for all neighbours } D \sim D'. \tag{32.1}$$

The definition is Dong, Roth & Su's (2022, Definition 2.3), cited as Dong et al. from here on, and the curve inequality holds at every FPR. The old definition is a special case of it, because Part 3 proved that M is (ε, δ)-DP exactly when the curve of every neighbouring pair lies above two straight lines (Theorem 18.3),

$$f_{\varepsilon,\delta}(\mathrm{FPR}) = \max\left(0,\ 1-\delta-e^{\varepsilon}\mathrm{FPR},\ e^{-\varepsilon}(1-\delta-\mathrm{FPR})\right). \tag{32.2}$$

So (ε, δ)-DP is $f_{\varepsilon,\delta}$-DP (Wasserman & Zhou 2010; Dong et al., Proposition 2.5). The second line is the first reflected across the diagonal $\mathrm{FPR} = \mathrm{FNR}$, and setting $\mathrm{FPR} = 1-\delta-e^{\varepsilon}\mathrm{FPR}$ shows that they meet there at $\mathrm{FPR} = \frac{1-\delta}{1+e^{\varepsilon}}$. The curve starts at $1-\delta$ on the FNR-axis, not at 1, and that gap is what δ means: a test may catch D' with probability δ at no cost in false alarms.

Because the neighbouring relation is symmetric, a guarantee for (D, D') also holds for (D', D), and the next result turns that into a guarantee with the reflected curve. The **inverse** $f^{-1}(\mathrm{FPR}) = \inf\{t \in [0,1] : f(t) \leq \mathrm{FPR}\}$ has the graph of f reflected across the diagonal. A curve with $f = f^{-1}$ is **symmetric**, as $f_{\varepsilon,\delta}$ is.

Proposition 32.1 (Symmetry and post-processing)**.** *If M is f-DP, then M is $\max(f, f^{-1})$-DP. If K is a randomized map that does not look at the data, the mechanism that outputs $K(M(D))$ is also f-DP.*

Proof.

1. *Swap the roles.* Since $D' \sim D$ is also a neighbouring pair, $T(Q, P) \geq f$. A test ϕ of P against Q with errors $(\mathrm{FPR}, \mathrm{FNR})$ gives the opposite test $1-\phi$ of Q against P, whose errors are $(\mathrm{FNR}, \mathrm{FPR})$. So $T(Q, P)$ is the inverse of $T(P, Q)$.
2. *Reflect.* A curve lies above f when its achievable error pairs sit inside the region above f, and reflecting both regions across the diagonal keeps that inclusion. So $T(Q, P) \geq f$ turns into $T(P, Q) \geq f^{-1}$, and with $T(P, Q) \geq f$ this gives $\max(f, f^{-1})$. That maximum keeps all four properties of a trade-off function.
3. *Post-processing.* A test ϕ on the processed output is the test $y \mapsto \mathbb{E}[\phi(K(y))]$ on the raw output, with the same two errors. So the curve after processing is no lower than before (Dong et al., Lemma 2.9).

□

Post-processing can only raise the curve, and Blackwell's theorem (Dong et al., Theorem 2.10) gives the converse: $T(P, Q) \leq T(P', Q')$ exactly when some randomized map sends P to P' and Q to Q' at once. A divergence also shrinks under such maps, but it is one number computed from the pair, so it is a coarser summary than the curve.

Example 32.1. Three mechanisms carry the label "$\varepsilon = 1$". Binary randomized response with $\varepsilon = 1$ reports the true bit with probability $e/(1+e) = 0.7311$ (randomized response). With bit 0 under D and 1 under D', the only useful test fires on output 1, with FPR = FNR = 0.2689. Mixing it with "never fire" and "always fire" gives a curve with one corner at $(0.2689, 0.2689)$, exactly the corner $1/(1+e)$ of $f_{1,0}$. The Laplace mechanism with sensitivity 1 and scale 1 is also $(1, 0)$-DP, and the running example, Gaussian noise with $\sigma = 1$, is shown for comparison.

Table 32.1.: Smallest type II error at three type I errors for three mechanisms. The Laplace curve is Part 1's Equation 4.12 for a unit shift at scale 1.

Type I error FPR	$f_{1,0}$ (randomized response)	Laplace, $\varepsilon = 1$	Gaussian, $\sigma = 1$ (G_1)
0.01	0.9728	0.9728	0.9076
0.1	0.7282	0.7282	0.6109
0.25	0.3204	0.3679	0.3724

The $f_{1,0}$ column is the line $1 - e\,\mathrm{FPR}$, so at FPR = 0.1 it is $1 - 0.2718 = 0.7282$. Randomized response sits exactly on $f_{1,0}$: it is the worst $(1, 0)$-DP mechanism. The Laplace mechanism matches it at small FPR, where both have slope $-e$, but is more private in the middle (0.3679 against 0.3204 at FPR = 0.25). The label "$(1, 0)$-DP" cannot tell the two apart, but their curves can (Figure 32.1). The Gaussian curve crosses the others: it allows more power at small FPR and less at 0.25, which no single ε can express.

32.2. Gaussian differential privacy

The Gaussian mechanism is the workhorse of private machine learning, so its curve deserves a name. This section shows that Gaussian noise always produces one of Part 1's Gaussian curves, a family with one parameter. Using that family as the yardstick gives Gaussian differential privacy (GDP), a one-number definition that is exact for the Gaussian mechanism.

Part 1 found the curve of two shifted normals (Theorem 4.3). The best test of $\mathcal{N}(0, 1)$ against $\mathcal{N}(\mu, 1)$ fires when the output exceeds a threshold, and it misses when a draw from the shifted normal stays below it. Write Φ for the standard normal distribution function and Φ^{-1} for its inverse.

Definition 32.2 (Gaussian differential privacy)**.** For $\mu \geq 0$, the **Gaussian trade-off function** is

$$G_\mu(\mathrm{FPR}) = T(\mathcal{N}(0,1), \mathcal{N}(\mu,1))(\mathrm{FPR}) = \Phi(\Phi^{-1}(1-\mathrm{FPR}) - \mu). \tag{32.3}$$

A mechanism is μ**-GDP** if it is G_μ-DP (Dong et al., Definition 2.6).

The Gaussian mechanism releases a vector, while Part 1's curve is for one number. The proof that the mechanism is GDP needs a step that reduces the vector to one number, and the next lemma provides it. With independent noise of equal variance in every coordinate, only the direction of the shift carries evidence.

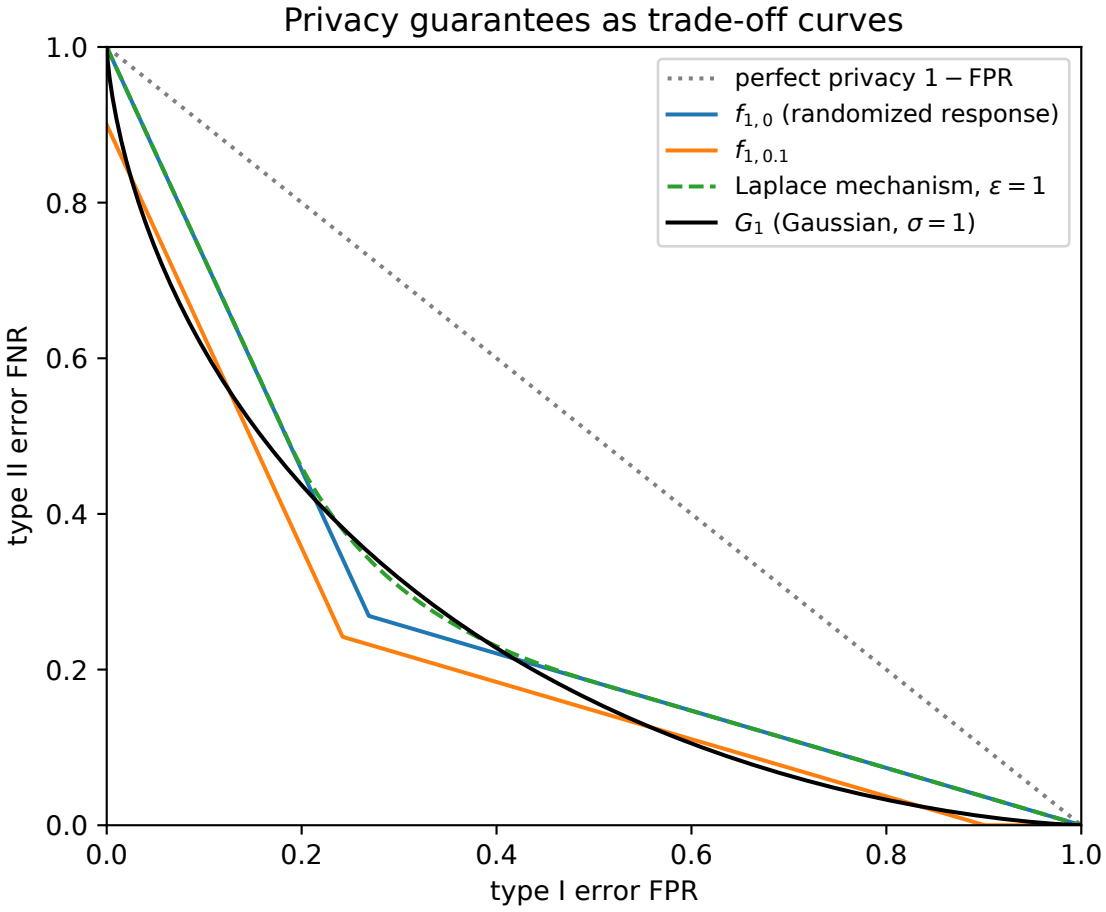


Figure 32.1.: Trade-off curves of $(1, 0)$-DP (randomized response), $(1, 0.1)$-DP, Laplace noise with $\varepsilon = 1$ and Gaussian noise with $\sigma = 1$; the dotted diagonal is perfect privacy.

Lemma 32.1 (A Gaussian shift in d dimensions is a shift in one)**.** *Let I_d be the $d \times d$ identity matrix. For every $m \in \mathbb{R}^d$,*

$$T(\mathcal{N}(0, I_d), \mathcal{N}(m, I_d)) = G_{\|m\|_2}. \tag{32.4}$$

Proof. The plan is to show that the likelihood ratio depends on one number, find that number's law, and read off Part 1's curve.

1. *Write the likelihood ratio.* Expanding $\|y - m\|^2 = \|y\|^2 - 2\langle m, y\rangle + \|m\|^2$ in the two normal densities cancels $\|y\|^2$, which leaves
$$\frac{Q(y)}{P(y)} = \exp\Big(\langle m, y\rangle - \frac{1}{2}\|m\|^2\Big). \tag{32.5}$$
2. *One number decides.* The ratio depends on y only through $s = \langle m, y\rangle/\|m\|$, the projection of y on the direction of the shift. By the Neyman–Pearson lemma (Theorem 4.1) the best tests are thresholds on s.
3. *Find the law of s.* The inner product of a unit vector with a standard normal vector is standard normal, since its variance is the squared length of the unit vector. So $s \sim \mathcal{N}(0, 1)$ under the first hypothesis, and the mean m adds $\langle m, m\rangle/\|m\| = \|m\|$ under the second.
4. *Read off the curve.* Every best test is a threshold test of $\mathcal{N}(0, 1)$ against $\mathcal{N}(\|m\|, 1)$, whose curve is $G_{\|m\|}$ by Theorem 4.3.

□

In two dimensions the best boundary is perpendicular to the shift (Figure 32.3, left). The Gaussian mechanism adds noise to a query, written g because f already names a trade-off function. Write $\Delta_2 g$ for its ℓ_2 sensitivity, the largest $\|g(D) - g(D')\|_2$ over neighbours, and σ for the **noise multiplier**, the noise standard deviation in units of $\Delta_2 g$.

Theorem 32.1 (The Gaussian mechanism is GDP)**.** *The mechanism $M(D) = g(D) + \mathcal{N}(0, \sigma^2(\Delta_2 g)^2 I)$ is $1/\sigma$-GDP. This is tight: a neighbouring pair with $\|g(D) - g(D')\|_2 = \Delta_2 g$ has trade-off function exactly $G_{1/\sigma}$.*

Proof. The plan is to turn the two outputs into the pair of Lemma 32.1 by invertible maps, which cannot change any test's errors, and read off the shift.

1. *Standardize and centre.* Divide the output by $\sigma\Delta_2 g$ and subtract $g(D)/(\sigma\Delta_2 g)$. Both maps can be undone, so no test's errors change. Now $P = \mathcal{N}(0, I)$, and Lemma 32.1 gives $T(P, Q) = G_\nu$ with $\nu = \|g(D) - g(D')\|_2/(\sigma\Delta_2 g)$.
2. *Bound the shift.* By the definition of sensitivity, $\nu \le 1/\sigma$.
3. *Compare curves.* G_μ decreases as μ grows, because Φ is increasing, so $G_\nu \ge G_{1/\sigma}$ for every neighbouring pair.
4. *Tightness.* A pair at full sensitivity has $\nu = 1/\sigma$ and gives equality.

□

For the Gaussian mechanism, μ-GDP is not a bound with slack; it is the curve, and every (ε, δ) pair follows from it (Section 32.3). The parameter μ is the distance between the two output means in units of the noise standard deviation, set by the worst pair. Doubling it to $\mu = 2$ cuts the miss rate of the best test at a 5% false-alarm rate from 0.7405 to 0.3612. Dong et al. suggest that $\mu \le 0.5$ is a reasonable guarantee and $\mu \ge 6$ promises almost nothing (their discussion after Definition 2.6).

Example 32.2. A count query has $\Delta_2 g = 1$, since adding or removing one record changes the count by 1. Released with $\sigma = 1$ it is 1-GDP, and with $\sigma = 4$ it is 0.25-GDP. For $\sigma = 1$, Table 32.2 evaluates G_1. At FPR $= 0.01$, for example, the threshold is $\Phi^{-1}(0.99) = 2.3263$, and subtracting the shift gives $G_1(0.01) = \Phi(1.3263) = 0.9076$.

Table 32.2.: The 1-GDP guarantee of the Gaussian mechanism with $\sigma = 1$, read as attack errors.

Type I error FPR	Smallest type II error $G_1(\text{FPR})$	Best power $1 - G_1(\text{FPR})$
0.01	0.9076	0.0924
0.05	0.7405	0.2595
0.1	0.6109	0.3891

A simulation of the threshold test at FPR $= 0.05$ with 1,000,000 draws per hypothesis gives $\widehat{\text{FPR}} = 0.0502$ and $\widehat{\text{FNR}} = 0.7408$, against the exact 0.7405. Read as a membership attack, the first row says that whatever the attacker knows about the rest of the data, at a 1% false-positive rate it detects a record at most 9.24% of the time.

32.3. Duality with (ε, δ)

Regulators, papers and libraries report (ε, δ). So you need to turn a curve into (ε, δ) pairs without losing anything, and you need the profile of μ-GDP in closed form. This section does both.

The idea is geometric. An (ε, δ) guarantee is the line $\text{FNR} = 1 - \delta - e^{\varepsilon}\text{FPR}$ and its mirror image, and a curve satisfies it when it lies above the line. For a slope $-e^{\varepsilon}$, the smallest δ that works is the one where the line just touches the curve. The line of slope u through the point $(\text{FPR}, f(\text{FPR}))$ meets the FNR-axis at $f(\text{FPR}) - u\,\text{FPR}$, and the touching line has the lowest of these intercepts. Minus that intercept is the **convex conjugate**

$$f^*(u) = \sup_{\text{FPR}\in[0,1]} \big(u\,\text{FPR} - f(\text{FPR})\big). \tag{32.6}$$

Theorem 32.2 (Duality between f-DP and (ε, δ)-DP)**.** *Let f be a symmetric trade-off function. A mechanism is f-DP if and only if it is $(\varepsilon, \delta(\varepsilon))$-DP for every $\varepsilon \ge 0$, where*

$$\delta(\varepsilon) = 1 + f^*(-e^{\varepsilon}). \tag{32.7}$$

For any pair P, Q, symmetric or not, $1 + T(P, Q)^*(-e^{\varepsilon})$ is the hockey-stick divergence $E_{e^{\varepsilon}}(Q\|P)$, the largest amount by which an event's probability under Q exceeds e^{ε} times its probability under P.

Proof. The plan is to find the smallest δ for one line, show that the lines rebuild the curve, and identify the conjugate as a divergence (Dong et al., Propositions 2.11 and 2.12).

1. *One pair is one line.* By Equation 32.2, $f \geq f_{\varepsilon,\delta}$ requires $\delta \geq 1 - f(\mathrm{FPR}) - e^{\varepsilon}\mathrm{FPR}$ for every FPR. The smallest such δ is the supremum of the right side, which is $1 + f^*(-e^{\varepsilon})$. The mirror line gives the same condition, because f is symmetric.
2. *f-DP implies every $(\varepsilon, \delta(\varepsilon))$.* If $T(M(D), M(D')) \geq f$ and $f \geq f_{\varepsilon,\delta(\varepsilon)}$, then $T(M(D), M(D')) \geq f_{\varepsilon,\delta(\varepsilon)}$.
3. *The lines rebuild the curve.* The line for ε touches f where its slope is $-e^{\varepsilon}$, and as ε runs over $[0, \infty)$ these slopes cover every value from -1 down. A symmetric curve crosses the diagonal with slope -1, or at a corner whose slopes include -1, and is steeper to its left, so the lines and their mirror images touch every point. A convex function is the supremum of its touching lines, so $\sup_{\varepsilon} f_{\varepsilon,\delta(\varepsilon)} = f$, and a mechanism with every $(\varepsilon, \delta(\varepsilon))$ has $T \geq f$.
4. *The conjugate is a hockey-stick divergence.* If each detection of D' earns one unit and each false alarm costs $\gamma \geq 1$, a test's gain is $1 - \mathrm{FNR} - \gamma\,\mathrm{FPR} = \int \phi\,(Q - \gamma P)$. The best test fires exactly where $Q > \gamma P$ and gains $E_{\gamma}(Q\|P)$, by Part 1's best event (Lemma 3.4) with Q in the place of P. The same maximum can be taken in two stages: at each FPR the largest power is $1 - T(P, Q)(\mathrm{FPR})$, and then the best FPR is chosen, so

$$\max_{\mathrm{FPR}\in[0,1]} \big(1 - T(P,Q)(\mathrm{FPR}) - \gamma\,\mathrm{FPR}\big) = E_{\gamma}(Q\|P). \tag{32.8}$$

   With $\gamma = e^{\varepsilon}$ the left side is $1 + f^*(-e^{\varepsilon})$.

□

The whole **privacy profile** $\varepsilon \mapsto \delta(\varepsilon)$ loses nothing, because its lines rebuild the curve, while one point keeps only one touching line (Figure 32.2, right). So "1-GDP" is stronger than "$(4.3772, 10^{-5})$-DP", the point the example below computes: at $\mathrm{FPR} = 0.1$ the single point allows $\mathrm{FNR} = e^{-4.3772}(1 - 10^{-5} - 0.1) = 0.0113$, while $G_1(0.1) = 0.6109$.

For G_{μ} the touching point has a closed form, found with Part 1's slope rule: the slope of a curve at a test's threshold τ is minus the likelihood ratio there (Proposition 4.1). The result is the exact Gaussian profile that Part 3 derived as a hockey-stick divergence (Theorem 20.3), and Section D.5 in the appendices derives it from the threshold test in plain steps.

Corollary 32.1 (The privacy profile of μ-GDP)**.** *A mechanism is μ-GDP if and only if it is $(\varepsilon, \delta(\varepsilon))$-DP for every $\varepsilon \geq 0$, with*

$$\delta(\varepsilon) = \Phi\Big(-\frac{\varepsilon}{\mu} + \frac{\mu}{2}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\varepsilon}{\mu} - \frac{\mu}{2}\Big). \tag{32.9}$$

Proof. The plan is to check that Theorem 32.2 applies, find the test where G_{μ} has slope $-e^{\varepsilon}$, and read off its errors (Dong et al., Corollary 2.13).

1. *The curve is symmetric.* The reflection $y \mapsto \mu - y$ swaps $\mathcal{N}(0,1)$ and $\mathcal{N}(\mu,1)$, and swapping the pair inverts the curve (Proposition 32.1, step 1), so $G_{\mu} = G_{\mu}^{-1}$ and $\delta(\varepsilon)$ is the maximum of $1 - G_{\mu}(\mathrm{FPR}) - e^{\varepsilon}\mathrm{FPR}$.

2. *Find the touching point.* That expression is concave, so its maximum is where $G'_\mu = -e^\varepsilon$. By Proposition 4.1 this happens where the likelihood ratio Q/P at the threshold, $e^{\mu\tau-\mu^2/2}$ by step 1 of the proof of Lemma 32.1, equals e^ε, so
$$\tau = \frac{\varepsilon}{\mu} + \frac{\mu}{2}. \tag{32.10}$$
3. *Read off the errors.* The test fires when $y > \tau$, so $\mathrm{FPR}^* = \Phi(-\tau)$, and its power is $1 - \mathrm{FNR}^* = \Phi(\mu - \tau)$.
4. *Substitute.* Putting these into $\delta = 1 - \mathrm{FNR}^* - e^\varepsilon \mathrm{FPR}^*$ gives Equation 32.9.

□

At $\varepsilon = 0$ the profile is $2\Phi(\mu/2) - 1$, the total variation (TV) distance, which is the best $1 - \mathrm{FPR} - \mathrm{FNR}$ of any test (Theorem 4.5). For small δ, the first term of Equation 32.9 must be a small normal tail, which needs $\frac{\varepsilon}{\mu} - \frac{\mu}{2}$ of about $\sqrt{2\ln(1/\delta)}$. So the ε you pay grows roughly like $\mu\sqrt{2\ln(1/\delta)}$. At $\mu = 1$, the targets 10^{-4}, 10^{-5} and 10^{-6} give $\varepsilon = 3.8044$, 4.3772 and 4.8866, and at $\mu = 2$ the increments are about twice as large.

Example 32.3. For the running example, $\mu = 1$ and $\varepsilon = 1$ put the touching threshold at $\tau = 1.5000$. The best test there has type I error $\Phi(-1.5) = 0.066807$ and power $\Phi(-0.5) = 0.308538$, so $\delta(1) = 0.308538 - e \times 0.066807 = 0.308538 - 0.181601 = 0.126937$. Part 3's Table 20.4 has the same two numbers as the probability under P that the loss exceeds 1 and e times that probability under Q; here they are the best test's power and the price of its false alarms. Maximizing $1 - G_1(\mathrm{FPR}) - e\,\mathrm{FPR}$ over a grid of 20,001 values gives the same 0.126937, at $\mathrm{FPR} = 0.0668$.

Going from a target δ to ε has no closed form. But $\delta(\varepsilon)$ decreases in ε, so the chapter's script (Section 32.8) solves for it by bisection (Table 32.3).

Table 32.3.: The ε of μ-GDP at $\delta = 10^{-5}$.

μ	0.5	1	2
ε at $\delta = 10^{-5}$	1.9931	4.3772	9.9973

As a check on step 3 of the duality, the supremum of 1001 lines $f_{\varepsilon,\delta(\varepsilon)}$ is within 4.9×10^{-6} of G_1 (Table 32.9).

32.4. Composition

Training runs a mechanism thousands of times on the same data. Composing (ε, δ) guarantees directly is lossy (composition). This section shows that composing curves is exact, and that for Gaussian curves it is a one-line formula.

An adversary who sees two outputs, drawn with independent randomness, faces a pair of joint distributions: $P_1 \times P_2$, both drawn on D, against $Q_1 \times Q_2$, both drawn on D'. The curve of that pair defines the composition of curves.

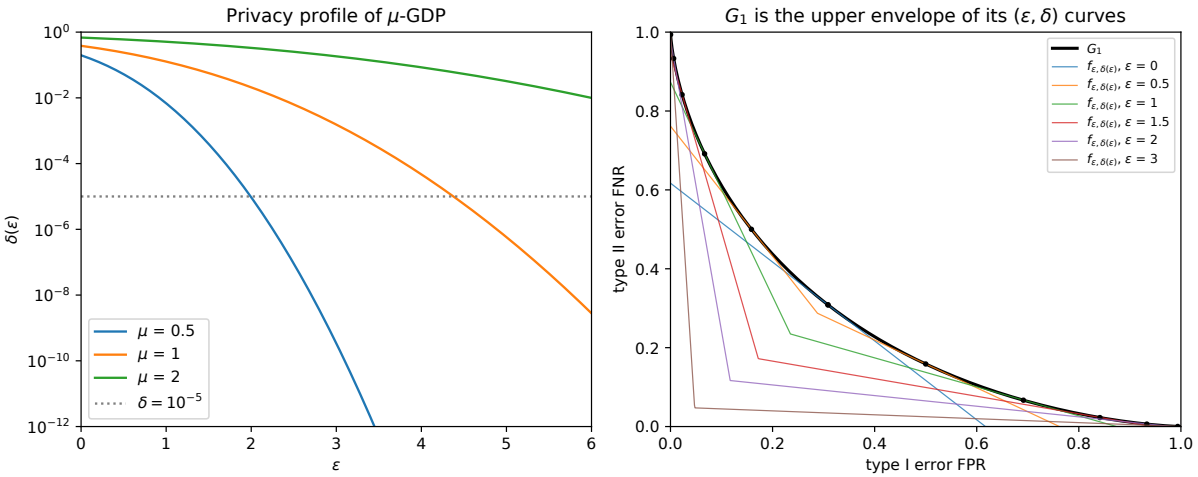


Figure 32.2.: Left: the privacy profile of μ-GDP for $\mu = 0.5, 1, 2$. Right: G_1 as the upper envelope of six of its $(\varepsilon, \delta(\varepsilon))$ curves.

Definition 32.3 (Tensor product)**.** The **tensor product** of trade-off functions $f_1 = T(P_1, Q_1)$ and $f_2 = T(P_2, Q_2)$ is $f_1 \otimes f_2 = T(P_1 \times P_2, Q_1 \times Q_2)$.

It does not depend on which pairs represent f_1 and f_2 (Dong et al., Definition 3.1; Section K.2 in the appendices proves it). The theorem below says that this product is the exact guarantee of a composition. Its Gaussian case follows from Lemma 32.1, because independent Gaussian coordinates form a Gaussian vector.

Theorem 32.3 (Composition of f-DP)**.** *If each mechanism M_i is f_i-DP, even when it is chosen adaptively from the earlier outputs, then $(M_1, \dots, M_k)$ is $f_1 \otimes \cdots \otimes f_k$-DP, and no better bound holds in general. For Gaussian curves the shifts add like the sides of a right-angled triangle:*

$$G_{\mu_1} \otimes \cdots \otimes G_{\mu_k} = G_\mu, \qquad \mu = \sqrt{\mu_1^2 + \cdots + \mu_k^2}. \tag{32.11}$$

Proof. The plan is to prove the non-adaptive case through Blackwell's theorem, cite the adaptive case, show tightness, and compute the Gaussian product (Dong et al., Theorem 3.2 and Corollary 3.3).

1. *Non-adaptive composition.* Let $(\tilde{P}_i, \tilde{Q}_i)$ be a pair whose curve is exactly f_i. Since $f_i \le T(P_i, Q_i)$, the pair with the lower curve can be processed into the other: by Blackwell's theorem some randomized map K_i sends $\tilde{P}_i$ to P_i and $\tilde{Q}_i$ to Q_i. Applied to the coordinates independently, the maps send the product pair $(\prod \tilde{P}_i, \prod \tilde{Q}_i)$ to $(\prod P_i, \prod Q_i)$. Post-processing cannot help a test (Proposition 32.1), so $T(\prod P_i, \prod Q_i) \ge \bigotimes f_i$.
2. *Adaptive composition.* When M_2 depends on the output of M_1, step 1 does not apply. Instead, fix the first output and look at the attacker's test on the second. Replacing it by the best test of a fixed pair with curve f_2, at the same type I error, keeps that error and does not raise the type II error, because M_2 is f_2-DP on every branch. So the attacker does no better than on the product pair. Section K.2 in the appendices derives this step by step, as Dong et al. do in their Appendix C.

3. *Tightness.* If the mechanisms ignore each other and some neighbouring pair attains each f_i, the composed pair is the product pair.
4. *Gaussian curves.* $\bigotimes G_{\mu_i}$ is the curve of $\mathcal{N}(0, I_k)$ against $\mathcal{N}(m, I_k)$ with $m = (\mu_1, \dots, \mu_k)$. By Lemma 32.1 this is $G_{\|m\|_2}$.

□

Under composition μ^2 adds, just as variances of independent noises add. Full-batch noisy gradient descent with noise multiplier σ composes T mechanisms that are each $1/\sigma$-GDP, so it is $\sqrt{T}/\sigma$-GDP, and by step 3 no accountant can improve on that. Zero-concentrated DP is the same arithmetic with $\rho = \mu^2/2$ (concentrated DP), but μ gives the exact curve and ρ only an RDP bound on it (Section 32.7).

For other curves the tensor product is hard. For $f_{\varepsilon,\delta}$ curves it is the optimal composition theorem (Theorem 21.4). When the $(\varepsilon_i, \delta_i)$ differ, computing it exactly is #P-complete (Murtagh & Vadhan 2016), a class of counting problems believed to have no efficient algorithm. So numerical accountants compute $\bigotimes f_i$ another way. The composed privacy loss is the sum of the per-step losses (Lemma 21.2), so the privacy loss distribution (PLD) accountants of numerical accounting discretize its law and convolve.

Example 32.4. By Equation 32.11, two releases of the running example compose to $G_{\sqrt{2}} = G_{1.4142}$, and four to G_2. A hundred releases with $\sigma = 10$ give $G_{\sqrt{100\times0.01}} = G_1$. A simulation of the threshold test on the projection, with $\mu = (1, 1)$ at FPR $= 0.05$ gives $\widehat{\text{FPR}} = 0.0497$ and $\widehat{\text{FNR}} = 0.5913$, against $G_{\sqrt{2}}(0.05) = 0.5912$. That sits between one release, $G_1(0.05) = 0.7405$, and one release with twice the shift, $G_2(0.05) = 0.3612$ (Figure 32.3).

Curves of the (ε, δ) family, by contrast, leave that family under composition. Run randomized response with $\varepsilon = 1$ on the same bit twice, the product Part 3 read as a sum of privacy losses (Table 21.1). The output 00 has probability $e^2/(1+e)^2 = 0.5344$ under P (bit 0) and $1/(1+e)^2 = 0.0723$ under Q (bit 1), and Table 32.4 lists all four outputs.

Table 32.4.: Two independent runs of randomized response with $\varepsilon = 1$.

Outputs	P	Q	Q/P
00	0.5344	0.0723	0.1353
01	0.1966	0.1966	1.0000
10	0.1966	0.1966	1.0000
11	0.0723	0.5344	7.3891

The best tests add outcomes in decreasing order of Q/P, each raising FPR by its P and lowering FNR by its Q. So $f_{1,0} \otimes f_{1,0}$ is the polygon through $(0, 1)$, $(0.0723, 0.4656)$, $(0.2689, 0.2689)$, $(0.4656, 0.0723)$ and $(1, 0)$, with three slopes, $-e^2$, -1 and $-e^{-2}$. An $f_{\varepsilon,\delta}$ curve has only two, so no single pair describes it. Each segment lies on a line of one such curve: the two outer ones on $f_{2,0}$, and the middle one on the line $1 - 0.4621 - \text{FPR}$ of $f_{0,0.4621}$. So the polygon is the supremum of these two curves, with $0.4621 = \tanh(1/2)$.

Composing points instead of curves loses. Four 1-GDP releases are 2-GDP, $\varepsilon = 9.9973$ at $\delta = 10^{-5}$. Giving each release a quarter of the δ budget turns it into the point $(4.6902, 2.5 \times 10^{-6})$ of 1-GDP,

and adding the points gives $\varepsilon = 18.7607$ at the same total δ, almost twice as large; Part 3's Figure 21.4 shows the loss for many steps.

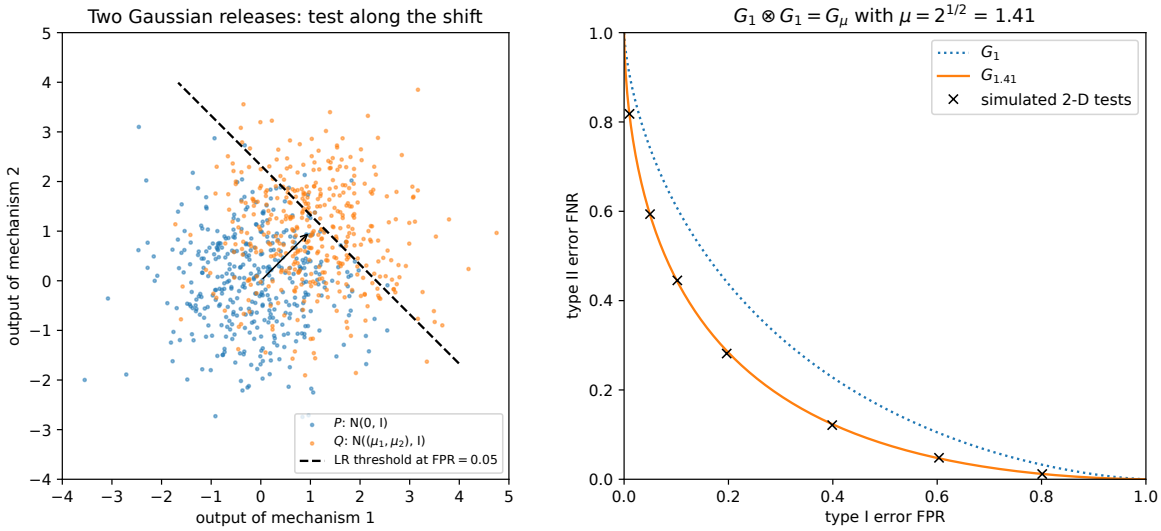


Figure 32.3.: Left: 400 draws each from $\mathcal{N}(0, I_2)$ and $\mathcal{N}((1,1), I_2)$; the dashed line is the best test's boundary at FPR $= 0.05$, perpendicular to the shift. Right: G_1, $G_{\sqrt{2}}$ and simulated errors of the projected test (crosses).

32.5. The central limit theorem for composition

Most mechanisms are not Gaussian, so Equation 32.11 does not apply to them. But the loss of a composition is a sum of independent per-step losses, and such sums are nearly normal. This section turns that into a statement about curves, which is what lets one μ summarize a long training run.

A CLT needs the mean, the variance and a third moment of the loss. Dong et al. write them as integrals of the log-slope of the curve, because the slope encodes the privacy loss.

Definition 32.4 (Moment functionals of a curve)**.** For a trade-off function f define (Dong et al., §3.2)

$$\mathrm{kl}(f) = -\int_0^1 \ln|f'(t)|\,\mathrm{d}t, \qquad \kappa_2(f) = \int_0^1 \ln^2|f'(t)|\,\mathrm{d}t, \tag{32.12}$$

$$\kappa_3(f) = \int_0^1 \big|\ln|f'(t)|\big|^3\,\mathrm{d}t, \qquad \bar{\kappa}_3(f) = \int_0^1 \big|\ln|f'(t)| + \mathrm{kl}(f)\big|^3\,\mathrm{d}t. \tag{32.13}$$

The proof of the CLT starts by reading these as moments of the **privacy loss random variable (PRV)**, the loss of an output drawn from P (Definition 21.1), using Part 1's slope rule (Proposition 4.1).

Lemma 32.2 (The functionals are moments of the privacy loss)**.** *For $f = T(P, Q)$, with L the privacy loss of an output drawn from P,*

$$\mathrm{kl}(f) = \mathbb{E}_P[L] = D_{\mathrm{KL}}(P\|Q), \quad \kappa_2(f) = \mathbb{E}_P[L^2], \quad \kappa_3(f) = \mathbb{E}_P|L|^3, \quad \bar{\kappa}_3(f) = \mathbb{E}_P\big|L - \mathbb{E}_P L\big|^3. \tag{32.14}$$

Proof.

1. *The slope is the threshold loss.* A small loss means a large ratio Q/P, so the best test with $\mathrm{FPR} = t$ fires when L is below its t-quantile $l(t)$ under P. By Proposition 4.1 the slope there is minus the likelihood ratio $Q/P = e^{-L}$, so $|f'(t)| = e^{-l(t)}$ and $-\ln|f'(t)| = l(t)$.
2. *Integrate over quantiles.* If U is uniform on $[0, 1]$, then $l(U)$ has the law of L, as when uniform numbers fed into an inverse distribution function produce samples. So $\int_0^1 h(l(t))\,\mathrm{d}t = \mathbb{E}_P[h(L)]$ for any function h.
3. *Take moments.* The choices $h(x) = x$, x^2, $|x|^3$ and $|x - \mathbb{E}_P L|^3$ give the four identities; the first is the Kullback–Leibler (KL) divergence.

□

The theorem matches the composition to the Gaussian curve whose loss has the same ratio of mean to standard deviation. For k curves, collect each functional into a vector, $\mathrm{kl} = (\mathrm{kl}(f_1), \dots, \mathrm{kl}(f_k))$ and so on, and write $\|\cdot\|_1$ for the sum of the entries and $\|\cdot\|_2^2$ for the sum of their squares.

Theorem 32.4 (Central limit theorem for composition)**.** *Let $f_1, \dots, f_k$ be symmetric trade-off functions with $\kappa_3(f_i) < \infty$, and put*

$$\mu = \frac{2\|\mathrm{kl}\|_1}{\sqrt{\|\kappa_2\|_1 - \|\mathrm{kl}\|_2^2}}, \qquad \xi = \frac{0.56\,\|\bar{\kappa}_3\|_1}{\left(\|\kappa_2\|_1 - \|\mathrm{kl}\|_2^2\right)^{3/2}}. \tag{32.15}$$

If $\xi < \frac{1}{2}$, then for all $\mathrm{FPR} \in [\xi, 1 - \xi]$ the composed curve lies in a band around G_μ, shifted by ξ both across and up or down:

$$G_\mu(\mathrm{FPR} + \xi) - \xi \le f_1 \otimes \cdots \otimes f_k(\mathrm{FPR}) \le G_\mu(\mathrm{FPR} - \xi) + \xi. \tag{32.16}$$

Proof. The plan is to find the mean and variance of the composed loss, approximate it by a normal with an explicit error, and match a Gaussian pair (Dong et al., Theorem 3.4 and Appendix D).

1. *Add the losses.* For product distributions the densities multiply, so the composed loss is a sum of independent per-step losses. By Lemma 32.2 it has mean $K = \|\mathrm{kl}\|_1$ under P and variance $V = \|\kappa_2\|_1 - \|\mathrm{kl}\|_2^2$.
2. *Approximate by a normal.* The Berry–Esseen theorem bounds the gap between the distribution function of a sum and the matching normal by a constant times the summed third central moments over $V^{3/2}$. With the constant 0.56 that is ξ.
3. *Match a Gaussian pair.* Swapping P and Q negates the loss, and a symmetric curve is unchanged by the swap, so by Lemma 32.2, step 2, the loss under Q has the law of minus the loss under P, with mean $-K$. The best tests therefore compare two nearly normal laws $2K/\sqrt{V}$ standard deviations apart. The pair behind G_μ has loss $\mathcal{N}(\mu^2/2, \mu^2)$ under P (Lemma 20.3), whose two laws are μ standard deviations apart, so the match is $\mu = 2K/\sqrt{V}$.

4. *From the loss to the curve.* A best test's two errors are values of the two distribution functions of the loss, each within ξ of a normal value by step 2. Eliminating the threshold gives the band, as steps 3 to 6 of Section K.4 in the appendices show.

□

As the number of curves grows, each leaking less, the band closes and the composition converges to some G_μ (Dong et al., Theorem 3.5). For k pieces that are $(\varepsilon_i, 0)$-DP, with $\sum_i \varepsilon_i^2 \to \mu^2$ and $\max_i \varepsilon_i \to 0$, the limit is G_μ (their Theorem 3.6), the $\sqrt{k}$ behaviour of advanced composition with the exact constant. That is why GDP is the natural family, while two runs of randomized response already leave the (ε, δ) family (Table 32.4).

The band ξ is a bound but a wide one, so in practice G_μ itself is used as the estimate, and that is an approximation, not a bound. It can lie above or below the true curve, and its error is largest in the tails, where small δ lives.

Example 32.5. Take k mechanisms, each ε_0-DP with $\varepsilon_0 = 1/\sqrt{k}$, so that the limit is the running example's G_1. For randomized response the summed loss is ε_0 times the number of truthful runs minus the number of flipped ones, so the exact profile is a finite binomial sum (Table 32.5).

Table 32.5.: $\delta(1)$ for k-fold randomized response with $\varepsilon_0 = 1/\sqrt{k}$, exactly and from the CLT. The limit G_1 has $\delta(1) = 0.1269$.

k	ε_0	Exact $\delta(1)$	μ, ξ of Equation 32.15	GDP $\delta(1)$
1	1.0000	0.0000	1.0422, 0.7663	0.1420
10	0.3162	0.1323	1.0042, 0.1837	0.1284
100	0.1000	0.1257	1.0004, 0.0562	0.1271
1000	0.0316	0.1269	1.0000, 0.0177	0.1270

At $k = 1$ the theorem says nothing, since $\xi = 0.7663 > \frac{1}{2}$. At $k = 10$ the true 0.1323 lies *above* the CLT's 0.1284, so the CLT under-reports δ there, and from $k = 100$ on it is accurate at this moderate ε, 0.1271 against 0.1257. At the small δ of a reported guarantee the picture changes (Table 32.6).

Table 32.6.: ε at $\delta = 10^{-5}$ for k-fold randomized response with $\varepsilon_0 = 1/\sqrt{k}$.

k	Randomized response, exact ε	GDP with the μ of Equation 32.15	GDP with $\mu = 1$	Basic composition $k\varepsilon_0$
10	3.1599	4.3983	4.3772	3.1623
100	4.3068	4.3793	4.3772	10.0000
1000	4.3683	4.3774	4.3772	31.6228

At $k = 10$ the true loss never exceeds 3.1623, so the exact ε is far below the Gaussian's 4.3983, whose tail ignores that bound. By $k = 1000$ the exact value and the two GDP values agree to within 0.01. In the tail the CLT over-reported ε at every k (Figure 32.4).

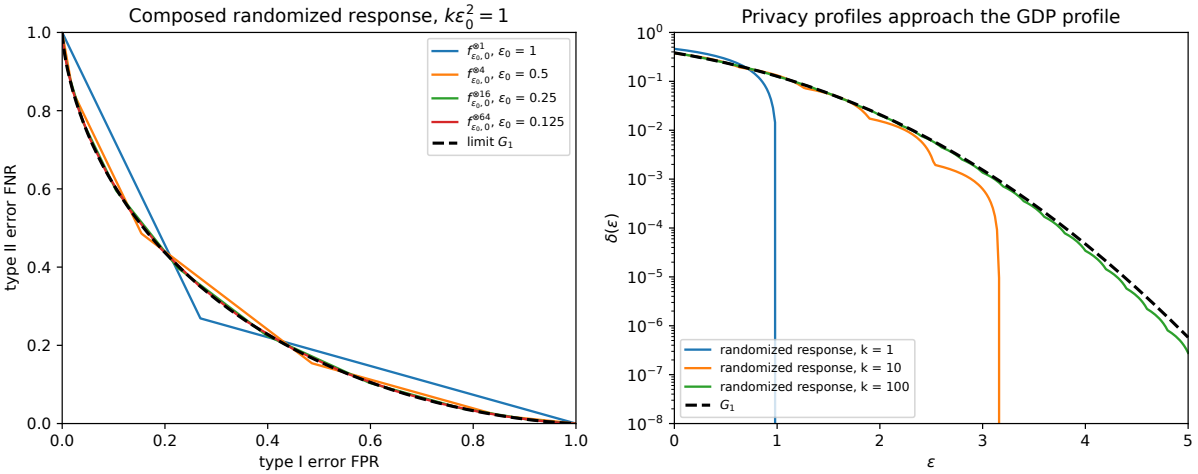


Figure 32.4.: Left: the exact curve of k-fold randomized response with $\varepsilon_0 = 1/\sqrt{k}$, for $k = 1, 4, 16, 64$, approaching G_1. Right: their privacy profiles on a log scale; each drops to 0 at $\varepsilon = \sqrt{k}$, while the GDP profile has a Gaussian tail.

32.6. The subsampled Gaussian and DP-SGD

Private training runs stochastic gradient descent (SGD) with differential privacy, DP-SGD for short (subsampling). Each of its T steps puts each record in the batch independently with probability q, clips the gradients and adds Gaussian noise with noise multiplier σ. A step leaks little when q is small, so the CLT should apply, and this section finds the μ it gives.

Under add/remove neighbours, one step reduces to a Gaussian against a mixture, because the added record shifts the sum by at most one sensitivity with probability q (subsampling derives it). In units of the sensitivity the pair is $P = \mathcal{N}(0, \sigma^2)$, the record absent, and $Q = (1-q)\mathcal{N}(0, \sigma^2) + q\mathcal{N}(1, \sigma^2)$, the record present.

Theorem 32.5 (The subsampled Gaussian CLT)**.** *Under add/remove neighbours, let $T \to \infty$ and $q\sqrt{T} \to c$ with σ fixed. Then the trade-off curve of T steps of this subsampled Gaussian mechanism converges to G_μ with*

$$\mu = q\sqrt{T(e^{1/\sigma^2} - 1)}. \tag{32.17}$$

Proof. The plan is to compute the mean and variance of one step's loss for small q, add them over T steps, and use $\mu = 2K/\sqrt{V}$ from Theorem 32.4 (Bu, Dong, Long & Su 2020, Theorem 5).

1. *Write the likelihood ratio.* The ratio of the $\mathcal{N}(1, \sigma^2)$ density to the $\mathcal{N}(0, \sigma^2)$ density is $e^{(y-1/2)/\sigma^2}$, by the cancellation of squares in Lemma 32.1. So

$$\frac{Q(y)}{P(y)} = 1 + qW(y), \qquad W(y) = e^{(y-1/2)/\sigma^2} - 1, \tag{32.18}$$

where W measures how much the output favours "the record was in the batch".

2. *Find the moments of W under P.* The mean is 0, because averaging a density ratio under its denominator gives 1. The moment generating function $\mathbb{E}[e^{\lambda Y}] = e^{\lambda^2\sigma^2/2}$ of $Y \sim \mathcal{N}(0, \sigma^2)$, at $\lambda = 2/\sigma^2$, gives $\mathbb{E}_P[(W+1)^2] = \mathbb{E}_P[e^{2Y/\sigma^2}]\, e^{-1/\sigma^2} = e^{2/\sigma^2} e^{-1/\sigma^2} = e^{1/\sigma^2}$. Expanding the square,
$$\mathbb{E}_P[W^2] = \mathbb{E}_P[(W+1)^2] - 2\,\mathbb{E}_P[W+1] + 1 = e^{1/\sigma^2} - 1. \tag{32.19}$$
3. *Expand the loss.* For small q, $L = -\ln(1+qW) = -qW + \frac{q^2W^2}{2} + O(q^3W^3)$. The first term has mean 0, so $\mathbb{E}_P[L] \approx \frac{q^2}{2}(e^{1/\sigma^2} - 1)$, and to leading order $\mathrm{Var}_P(L) \approx q^2(e^{1/\sigma^2} - 1)$.
4. *Sum over T steps.* Means and variances of independent losses add, so $K \approx \frac{Tq^2}{2}(e^{1/\sigma^2} - 1)$ and $V \approx Tq^2(e^{1/\sigma^2} - 1)$. Then
$$\mu = \frac{2K}{\sqrt{V}} = \frac{Tq^2(e^{1/\sigma^2} - 1)}{q\sqrt{T(e^{1/\sigma^2} - 1)}} = q\sqrt{T(e^{1/\sigma^2} - 1)}. \tag{32.20}$$
5. *Justify the limit.* With $q\sqrt{T} \to c$, each step's moments are $O(1/T)$ and $\sum \kappa_3 = O(Tq^3) \to 0$, the conditions of the limit form of the CLT. That form assumes symmetric curves, and this pair is not symmetric, so Section K.5 in the appendices treats P and Q separately, bounds the remainders by $O(q^3)$ and proves the limit.

□

Subsampling buys a factor of about q: full batches give $\sqrt{T}/\sigma$-GDP, and Equation 32.17 multiplies this by $q\sigma\sqrt{e^{1/\sigma^2} - 1}$, which is $q \times 1.3108$ at $\sigma = 1$. The neighbouring relation matters here: Dong et al. (Corollary 5.4) analyse fixed-size batches under replace-one neighbours, where a neighbour swaps one record for another, and get a different constant, $\mu = 1.7101$ at $T = 10{,}000$ in the running example (subsampling explains why not to mix them).

The result is a limit, used as an approximation at finite T, and it is not a bound. The errors of the CLT and of the expansion in step 3 add up, and neither is controlled in the tail where $\delta = 10^{-5}$ lives. Gopi, Lee & Wutschitz (2021) showed that the CLT value can lie below the true ε, and the example confirms it. Numerical accounting computes certified values, and choosing an accountant compares the accountants on this example.

Example 32.6. The running DP-SGD example has $q = 0.01$, $\sigma = 1$ and $\delta = 10^{-5}$. Then $\sqrt{e-1} = \sqrt{1.7183} = 1.3108$, so $\mu = 0.01 \times 1.3108\sqrt{T}$, which is 0.1311 at $T = 100$. Table 32.7 turns each μ into ε with Equation 32.9 and compares it with a numerical accountant.

Table 32.7.: The running DP-SGD example: the CLT's μ and ε, and a numerical bracket for the true ε.

Steps T	$q\sqrt{T}$	CLT μ	CLT ε at 10^{-5}	Numerical accountant ε	Full batch, $\mu = \sqrt{T}$
100	0.10	0.1311	0.4575	[0.7175, 0.7185]	10.0
1,000	0.32	0.4145	1.6177	[1.8232, 1.8332]	31.6
10,000	1.00	1.3108	6.0071	[6.1377, 6.2377]	100.0

The bracket rounds each step's loss down or up to a lattice of spacing $h = 10^{-5}$, so its width is Th by construction. The informative number is its lower end, because the true ε is at least that. In all three rows the CLT value lies below it, by a ratio of 0.638 at $T = 100$, 0.887 at $T = 1{,}000$ and 0.979 at $T = 10{,}000$, where $q\sqrt{T}$ is largest.

The exact per-step moments, computed by quadrature, are smaller than the leading terms, and they give $\varepsilon = 5.9184$ at $T = 10{,}000$, or 5.8379 with the roles of D and D' swapped. Both lie below the CLT's 6.0071, so a "more exact" μ is not more conservative; only a certified accountant is.

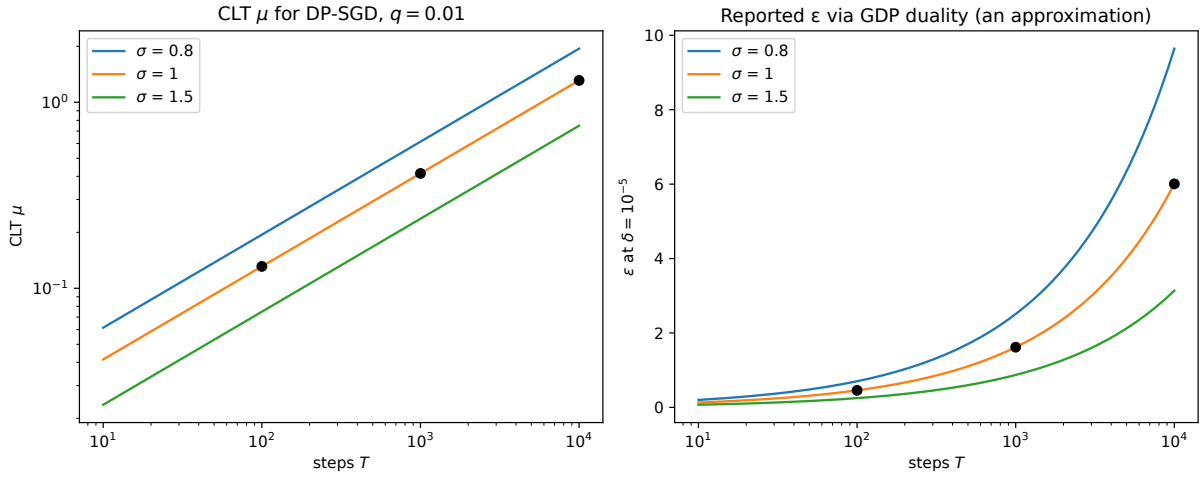


Figure 32.5.: The CLT for DP-SGD with $q = 0.01$ and three noise multipliers: μ (left) and the ε at $\delta = 10^{-5}$ (right) against the number of steps. Dots mark the running example.

Figure 32.5 shows how strongly ε depends on σ: $e^{1/\sigma^2} - 1$ is 3.7707 at $\sigma = 0.8$ and 0.5596 at $\sigma = 1.5$.

32.7. RDP as a lossy summary of the curve

Rényi DP tracks $\varepsilon(\alpha) = D_\alpha(P\|Q)$ over orders $\alpha > 1$, and Conversions (Section 30.3) turned it into the region of test errors it allows. This section reads that region as f-DP, to see why RDP accounting leaves slack.

The construction rests on one observation. A test maps P to a coin Bern(FPR) that lands 1 with probability FPR, and Q to the coin Bern$(1 - \text{FNR})$. Processing cannot increase a Rényi divergence (Theorem 3.5), so the two coins must stay within the RDP budget at every order. The lowest FNR that passes this check is the curve that RDP certifies.

Theorem 32.6 (What an RDP curve certifies)**.** *Let $\varepsilon(\alpha)$ be an RDP curve, known at the orders of the part's grid (the fixed list of orders its accountants evaluate), for both (D, D') and (D', D). Let $f_{\text{RDP}}(\text{FPR})$ be the smallest* FNR *with*

$$D_\alpha(\text{Bern}(\text{FPR}) \,\|\, \text{Bern}(1 - \text{FNR})) \le \varepsilon(\alpha) \quad \textit{and} \quad D_\alpha(\text{Bern}(1 - \text{FNR}) \,\|\, \text{Bern}(\text{FPR})) \le \varepsilon(\alpha) \tag{32.21}$$

at every order α of the grid (Equation 30.15). Then:

1. *The trade-off curve of a pair determines its RDP curve.*
2. *f_{RDP} is a symmetric trade-off function, and a mechanism with this RDP curve is f_{RDP}-DP. At each* FPR*, no higher value follows from the RDP values alone.*
3. *The optimal conversion Equation 30.16 of Conversions is the privacy profile $1 + f^*_{\mathrm{RDP}}(-e^\varepsilon)$ of f_{RDP}.*

Proof. The plan is to read the RDP curve off the law of the loss, push each test through the data-processing inequality, check the shape of f_{RDP}, exhibit a pair at each point, and match the conversion with the duality.

1. *The curve determines the divergence.* By Lemma 32.2, step 2, the curve fixes the law of L under P, and $D_\alpha(P\|Q) = \frac{1}{\alpha-1}\ln \mathbb{E}_P[e^{(\alpha-1)L}]$ is a functional of that law.
2. *RDP implies f_{RDP}-DP.* By the data-processing inequality the two coins of any test satisfy the constraints, so every point of the true curve is feasible and $T(P,Q) \geq f_{\mathrm{RDP}}$.
3. *Shape.* Each constraint bounds the logarithm of a sum of terms $x^\alpha y^{1-\alpha}$, so it bounds the sum itself, and each term is jointly convex for $\alpha > 1$ (the paragraph after Theorem 30.2; Section F.5). So the feasible set and its lower boundary are convex. Two equal coins have divergence 0, so $f_{\mathrm{RDP}}(\mathrm{FPR}) \leq 1 - \mathrm{FPR}$ and $f_{\mathrm{RDP}}(1) = 0$, and a convex function with its minimum at 1 is non-increasing. Relabelling the coins' outcomes swaps the two constraints, so the feasible set is symmetric about the diagonal.
4. *No higher value.* For each feasible point, the pair of coins $(\mathrm{Bern}(\mathrm{FPR}), \mathrm{Bern}(1-\mathrm{FNR}))$ is itself a mechanism with these RDP values, and its test "fire on 1" has errors $(\mathrm{FPR}, \mathrm{FNR})$ (step 5 of the proof of Theorem 30.2). Different values of FPR need different pairs.
5. *The conversion is the conjugate.* Equation 30.16 maximizes $1 - \mathrm{FPR} - e^\varepsilon f_{\mathrm{RDP}}(\mathrm{FPR})$, with the roles of the two errors swapped from Theorem 32.2. Since f_{RDP} is symmetric the swap changes nothing, and the maximum is Equation 32.7 applied to f_{RDP}.

□

The results form a ladder. For the Gaussian mechanism, G_μ is the exact curve, f_{RDP} is what its RDP curve keeps, and an (ε, δ) pair is one touching line of f_{RDP}. The optimal conversion removes the gap between the line and f_{RDP}, but no conversion removes the gap between f_{RDP} and the true curve (Section 30.5). RDP loses because different laws of the loss share the same exponential moments at the grid's orders, and f_{RDP} must allow the worst of them. Conversions shows that the extra power it grants an attacker persists down to $\mathrm{FPR} = 10^{-6}$ (Table 30.4). So GDP accounting for full-batch noisy gradient descent beats RDP accounting, although both compose exactly.

Example 32.7. For the running example, $\mu = 1$, the RDP curve is $\varepsilon(\alpha) = \alpha/2$ (Proposition 3.3). The Gaussian pair has these RDP values, so claim 2 gives $G_1 \geq f_{\mathrm{RDP}}$, and the script, computing f_{RDP} by bisection on FNR, finds it strictly below G_1 everywhere it looked. In Table 32.8 the best advantage $1 - \mathrm{FPR} - \mathrm{FNR}$ of a symmetric curve is reached at its diagonal point, where its slope is -1, and equals $1 - 2\mathrm{FPR}$ there, which for G_1 is $1 - 2 \times 0.3085 = 0.3829$.

Table 32.8.: The trade-off curve of 1-GDP against the best curve its RDP curve implies over the part's grid of orders, read at the diagonal and through the duality.

Quantity	Exact curve G_1	f_{RDP} from $\varepsilon(\alpha) = \alpha/2$
FNR at G_1's diagonal point, FPR $= \Phi(-1/2) = 0.3085$	0.3085	0.2201
Diagonal point, FPR = FNR	0.3085	0.2585
Best advantage	0.3829	0.4830
$\delta(0) = 1 - 2\text{FPR}$ at the diagonal point		
$\delta(1)$	0.1269	0.2149

The RDP summary allows an attacker an advantage of 0.4830 where the best possible is 0.3829, and the best $\delta(1)$ it certifies is 0.2149, 1.69 times the exact 0.1269. More orders can only raise f_{RDP}, but not by much: at FPR $= 0.3085$, a dense set of orders in $[1.01, 3000]$ raises it from 0.2201 to 0.2219, still far below 0.3085 (Figure 32.6).

32.8. Code

The script code/gaussian_dp.py computes every number in this chapter and draws every figure, and later chapters import its functions, such as `gdp_delta` and `gdp_eps`. From the repository root:

```
uv run python code/part04-privacy-accounting/gaussian_dp.py
```

Table 32.9.: Results printed by the script with seed 0.

Experiment	Result
Simulated threshold test on G_1 at FPR $= 0.05$	$\widehat{\text{FPR}} = 0.0502$, $\widehat{\text{FNR}} = 0.7408$; exact 0.7405
$\delta(1)$ of 1-GDP: formula, hockey-stick integral, conjugate on a grid	0.126937 each
ε at $\delta = 10^{-5}$ for $\mu = 0.5, 1, 2$	1.9931, 4.3772, 9.9973
Envelope of 1001 (ε, δ) lines against G_1	max gap 4.9×10^{-6}
Simulated 2-D test, $\mu = (1, 1)$, FPR $= 0.05$	$\widehat{\text{FNR}} = 0.5913$; $G_{\sqrt{2}}(0.05) = 0.5912$
Four 1-GDP releases at $\delta = 10^{-5}$	exact 9.9973; composing points 18.7607
DP-SGD, $T = 10{,}000$	CLT $\mu = 1.3108$, $\varepsilon = 6.0071$; numerical $[6.1377, 6.2377]$
1-GDP vs its RDP curve: best advantage; $\delta(1)$	0.3829 vs 0.4830; 0.1269 vs 0.2149

tests/test_gaussian_dp.py checks that the output still matches Table 32.9 and every other number in the text.

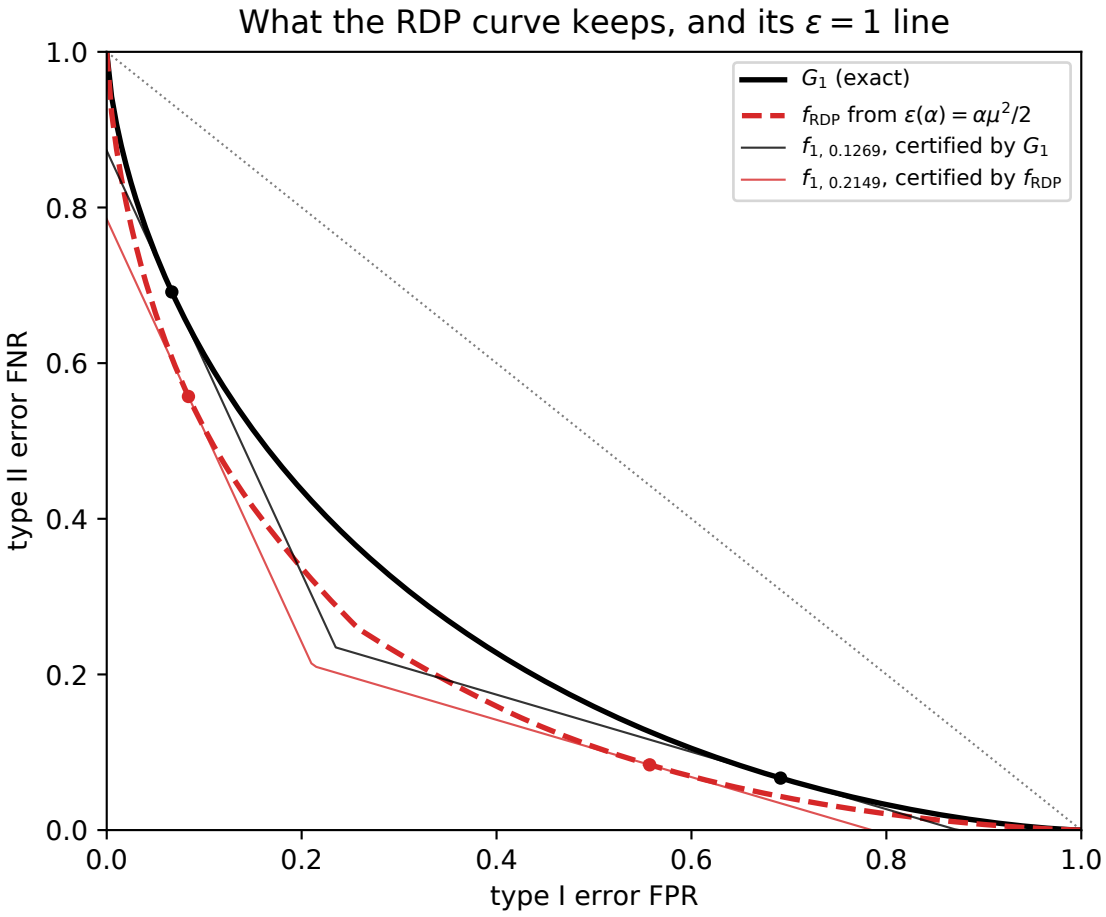


Figure 32.6.: G_1 (black) and the curve f_{RDP} that its RDP curve certifies (red, dashed), each with its touching (ε, δ) curve at $\varepsilon = 1$: $f_{1,0.1269}$ and $f_{1,0.2149}$.

32.9. Summary

Table 32.10.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 32.1	$T(M(D), M(D')) \geq f$ for all neighbours; (ε, δ)-DP is $f_{\varepsilon,\delta}$-DP	Every guarantee as a bound on every attacker's errors; summarize the curve as late as possible
Proposition 32.1	f-DP implies $\max(f, f^{-1})$-DP and survives post-processing	Working with symmetric curves
Definition 32.2	$G_\mu(\text{FPR}) = \Phi(\Phi^{-1}(1-\text{FPR}) - \mu)$	One number; $\mu \leq 0.5$ is reasonable, $\mu \geq 6$ promises almost nothing
Theorem 32.1	Noise multiplier σ gives exactly $1/\sigma$-GDP	Every Gaussian release, with no slack
Theorem 32.2	f-DP $\iff$ $(\varepsilon, 1 + f^*(-e^\varepsilon))$-DP for all $\varepsilon \geq 0$	Reporting a curve as (ε, δ); quote the profile, not one point
Corollary 32.1	$\delta(\varepsilon) = \Phi(-\frac{\varepsilon}{\mu} + \frac{\mu}{2}) - e^\varepsilon \Phi(-\frac{\varepsilon}{\mu} - \frac{\mu}{2})$	$\mu = 1$ is $\varepsilon \approx 4.4$ at $\delta = 10^{-5}$, $\mu = 0.5$ about 2
Theorem 32.3	$\bigotimes f_i$, tight; $\bigotimes G_{\mu_i} = G_{\sqrt{\sum \mu_i^2}}$	Gaussian noise without subsampling: account in GDP, μ^2 adds, the profile is exact
Theorem 32.4	$\bigotimes f_i \approx G_\mu$ with $\mu = 2K/\sqrt{V}$, band ξ	Quick estimates; good in the bulk, unreliable in the tails
Theorem 32.5	$\mu \approx q\sqrt{T(e^{1/\sigma^2} - 1)}$	A quick DP-SGD estimate, never the reported ε; it under-reports
Theorem 32.6	RDP gives f_{RDP}-DP, below the true curve; the optimal conversion is its conjugate	Why RDP accounting leaves slack no conversion removes

32.10. References

- Dong, Roth & Su, "Gaussian Differential Privacy" (Journal of the Royal Statistical Society Series B, 2022; arXiv:1905.02383): the source of f-DP and GDP, their duality, composition and CLTs.
- Bu, Dong, Long & Su, "Deep Learning with Gaussian Differential Privacy" (Harvard Data Science Review, 2020): the Poisson-subsampling CLT of Equation 32.17 (Theorem 5) and its use for DP-SGD.

- Wasserman & Zhou, "A Statistical Framework for Differential Privacy" (Journal of the American Statistical Association, 2010): the hypothesis-testing reading of (ε, δ)-DP.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015): optimal composition through testing regions and Blackwell's theorem.
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy" (ICML 2018): Theorem 8 is the exact profile Equation 32.9 of the Gaussian mechanism.
- Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021): the PRV accountant, and examples where the GDP CLT under-reports ε.
- Murtagh & Vadhan, "The Complexity of Computing the Optimal Composition of Differential Privacy" (TCC 2016): why accountants discretize.

33. Amplification by subsampling

Training a neural network with differential privacy (DP) runs stochastic gradient descent (SGD) for ten thousand steps or more, with per-example clipping and Gaussian noise, a method called DP-SGD. Each step clips every example's gradient to norm C, sums the clipped gradients and adds Gaussian noise whose standard deviation is the **noise multiplier** σ times C. If every record were in every batch, ten thousand steps with $\sigma = 1$ would cost $\varepsilon = 5425.5$ at $\delta = 10^{-5}$ (Rényi DP), a guarantee nobody would report. This chapter asks how much cheaper a step becomes when it looks only at a small random batch, and for which ways of drawing the batch the answer holds.

The one idea is that the output becomes a mixture. Neighbours differ by adding or removing one record, unless a section says otherwise. Write P_0 for a step's output distribution without the record and P_1 for it with the record. If the batch contains the record only with probability q, the **sampling rate**, the step on the dataset with the record draws from P_1 with probability q and from P_0 otherwise. So every likelihood ratio between the two runs is pulled toward 1:

$$\frac{(1-q)P_0 + qP_1}{P_0} = (1-q) + q\,\frac{P_1}{P_0}, \tag{33.1}$$

a weighted average of 1 and the old ratio, once the rest of the batch is fixed.

The running example is DP-SGD with $q = 0.01$, $\sigma = 1$, $\delta = 10^{-5}$ and up to $T = 10{,}000$ steps. Sampling turns a step that costs $\varepsilon = 1$ into one that costs 0.0170, and with Rényi differential privacy (RDP) the whole run costs $\varepsilon = 6.71$ instead of 5425.5.

33.1. Amplification for (ε, δ)-DP under Poisson sampling

You have a mechanism M that is (ε, δ)-DP for add/remove neighbours, and you run it on a random sample of the data. What are the privacy parameters of the combined mechanism? This question comes first because every later answer reuses its argument: fix everything except whether the record was drawn, compare a mixture with its parts, and average the rest back in. The tight answer below is due to Li, Qardaji & Su (2012) and Balle, Barthe & Gaboardi (2018).

Definition 33.1 (Poisson subsampling)**.** Let $q \in [0, 1]$. **Poisson sampling** with rate q keeps each record of a dataset D independently with probability q and returns the kept records, $S_q(D)$. For a mechanism M, the **subsampled mechanism** $M \circ S_q$ runs M on $S_q(D)$.

The argument rests on one inequality about mixtures. Write $P(A)$ for the probability that P gives to an event A, and say that P is (ε, δ)**-close** to Q when $P(A) \le e^{\varepsilon} Q(A) + \delta$ for every event A. Picture two runs that both draw from a common distribution R with probability $1 - q$, while with probability q the first draws from P and the second from P'. In the opening's picture, R

and P' are both P_0 and P is P_1. Where the runs agree the likelihood ratio is 1, and where they differ it is at most e^{ε}, up to the slack δ. So the mixture's ratio should be at most the weighted average $1 + q(e^{\varepsilon} - 1)$, and the slack should shrink to $q\delta$. The one subtlety is that P must be close to R as well as to P', or a likely event of P could be one that R never produces.

Lemma 33.1 (The mixture lemma)**.** *Let R, P and P' be distributions on the same outputs, and let $\varepsilon \geq 0$, $\delta \in [0, 1]$ and $q \in [0, 1]$. Suppose that for every event A,*

$$P(A) \leq e^{\varepsilon} P'(A) + \delta \qquad \text{and} \qquad P(A) \leq e^{\varepsilon} R(A) + \delta. \tag{33.2}$$

Then for every event A,

$$(1 - q)R(A) + qP(A) \leq e^{\varepsilon'} \left[(1 - q)R(A) + qP'(A)\right] + q\delta, \qquad e^{\varepsilon'} = 1 + q(e^{\varepsilon} - 1). \tag{33.3}$$

Proof. The plan is to treat the three probabilities of A as unknowns. Making $P'(A)$ and $R(A)$ smaller makes the inequality harder to satisfy, so it is enough to check it at the smallest values the assumptions allow. Fix A and write $a = P(A)$, $b = P'(A)$ and $c = R(A)$. Write F for the gap, the left side of Equation 33.3 minus the bracketed term times $e^{\varepsilon'}$. The claim is that F is at most $q\delta$.

1. *Expand F.* The two terms in c combine to $(1 - q)c(1 - e^{\varepsilon'})$, and $1 - e^{\varepsilon'} = -q(e^{\varepsilon} - 1)$ by the definition of ε'. The term in b is $-e^{\varepsilon'} qb$. So

$$F = qa - q(1 - q)(e^{\varepsilon} - 1)\, c - q\big(1 + q(e^{\varepsilon} - 1)\big)\, b. \tag{33.4}$$

2. *Find the worst b and c.* Both coefficients are negative, so F is largest when b and c are as small as allowed. Solving each assumption for b or c gives the lower bound $e^{-\varepsilon}(a - \delta)$ for both, and both are also at least 0, being probabilities.
3. *Case $a \leq \delta$.* Dropping the two negative terms leaves $F \leq qa \leq q\delta$.
4. *Case $a > \delta$.* Put $b = c = e^{-\varepsilon}(a - \delta)$ into step 1. With the common factor q taken out, the two coefficients add up to $(1 - q)(e^{\varepsilon} - 1) + 1 + q(e^{\varepsilon} - 1) = e^{\varepsilon}$, so

$$F \leq qa - q\, e^{-\varepsilon}(a - \delta)\, e^{\varepsilon} = q\delta. \tag{33.5}$$

□

Balle, Barthe & Gaboardi (2018, Theorem 2) call this **advanced joint convexity**. On 20,000 random triples (R, P, P') on four outcomes, the chapter's script finds some that meet it with equality to four digits. In the theorem, R, P and P' will be outputs for one fixed choice of the other records, and the next lemma averages that choice back in.

Lemma 33.2 (Averaging over a shared choice)**.** *Let $P = \sum_t w_t P_t$ and $Q = \sum_t w_t Q_t$ be averages of distributions with the same non-negative weights w_t summing to 1, and let $\gamma > 0$. If $P_t(A) \leq \gamma Q_t(A) + \delta$ for every t and every event A, then*

$$P(A) \leq \gamma Q(A) + \delta \quad \text{for every event } A. \tag{33.6}$$

Proof.

1. *Write the excess as an average.* For a fixed event A, both averages use the same weights, so the excess of the averages is the average of the excesses:
$$P(A) - \gamma Q(A) = \sum_t w_t [P_t(A) - \gamma Q_t(A)]. \tag{33.7}$$
2. *Bound the average.* Each bracket is at most δ, and so is any weighted average of them.

□

This is **joint convexity** of the hockey-stick divergence of Definition 3.4, the largest excess $P(A) - \gamma Q(A)$ over events, for a mixing choice that both runs share. With both lemmas the theorem follows, and it says that δ is paid only when the record is drawn.

Theorem 33.1 (Amplification by Poisson subsampling)**.** *Let M be (ε, δ)-DP for add/remove neighbours, and let $q \in [0, 1]$. Then $M \circ S_q$ is (ε', δ')-DP for add/remove neighbours with*
$$\varepsilon' = \ln\left(1 + q(e^\varepsilon - 1)\right), \qquad \delta' = q\delta. \tag{33.8}$$
For every ε and q, some ε-DP mechanism attains this ε'.

Proof. The plan is to fix every random choice except whether the added record x is drawn, apply Lemma 33.1 once for each order of the pair, average with Lemma 33.2, and finally exhibit a mechanism that attains the bound. Let $D' = D \cup \{x\}$.

1. *Condition on the rest of the sample.* Let $\tilde{D}$ be the Poisson sample of D. Each record has its own coin, so the sample of D' is $\tilde{D} \cup \{x\}$ with probability q and $\tilde{D}$ otherwise, independently of $\tilde{D}$. Given $\tilde{D}$, write $P_0 = M(\tilde{D})$ and $P_1 = M(\tilde{D} \cup \{x\})$, so that the output on D' is the mixture $(1-q)P_0 + qP_1$.
2. *Bound the direction with the mixture first.* Apply Lemma 33.1 with $R = P' = P_0$ and $P = P_1$. Both of its assumptions then hold because $\tilde{D}$ and $\tilde{D} \cup \{x\}$ are neighbours, and its right side becomes $e^{\varepsilon'} P_0(A) + q\delta$. Here the dataset with the record comes first and its neighbour removes it, which Numerical accounting calls the **remove** direction.
3. *Bound the direction with the mixture second.* Apply the lemma with $R = P = P_0$ and $P' = P_1$. Its first assumption is DP of M again, and its second, $P_0(A) \le e^\varepsilon P_0(A) + \delta$, always holds. Its left side becomes $P_0(A)$, so $P_0(A) \le e^{\varepsilon'}[(1-q)P_0(A) + qP_1(A)] + q\delta$.
4. *Average over $\tilde{D}$.* Both bounds hold for every $\tilde{D}$, so Lemma 33.2 carries them to the subsampled mechanism.
5. *Attain the bound.* Randomized response reports a bit truthfully with probability $e^\varepsilon/(1+e^\varepsilon)$ and flipped otherwise (Theorem 18.1). Let M apply it to the bit "the sample contains a flagged record", take D with no flagged record, and let x be flagged. Then P_1 gives the output 1 exactly e^ε times the probability that P_0 gives it, for every $\tilde{D}$. At the output 1 the likelihood ratio of the mixture to P_0 is therefore
$$\frac{(1-q)P_0(1) + qP_1(1)}{P_0(1)} = 1 + q(e^\varepsilon - 1). \tag{33.9}$$
With $\delta = 0$, the event that the output is 1 forces $e^{\varepsilon'}$ to be at least this ratio.

□

The formula has two regimes (Figure 33.1). For small ε, $e^{\varepsilon} - 1 \approx \varepsilon$ and the logarithm is close to its argument minus 1, so $\varepsilon' \approx q\varepsilon$. For large ε the 1 inside the logarithm stops mattering, so $\varepsilon' \approx \varepsilon - \ln(1/q)$, because in the worst case the record was drawn. At $q = 0.01$ the switch comes near $\varepsilon = \ln(1/q) = 4.6$, so Table 33.1 gives 0.9059 at $\varepsilon = 5$, not the 0.05 of a factor of 100.

The δ always shrinks by the factor q, because the failure event of M only matters when the record was drawn. The bound is tight for one step, but composing per-step pairs is not, as Section 33.5 shows.

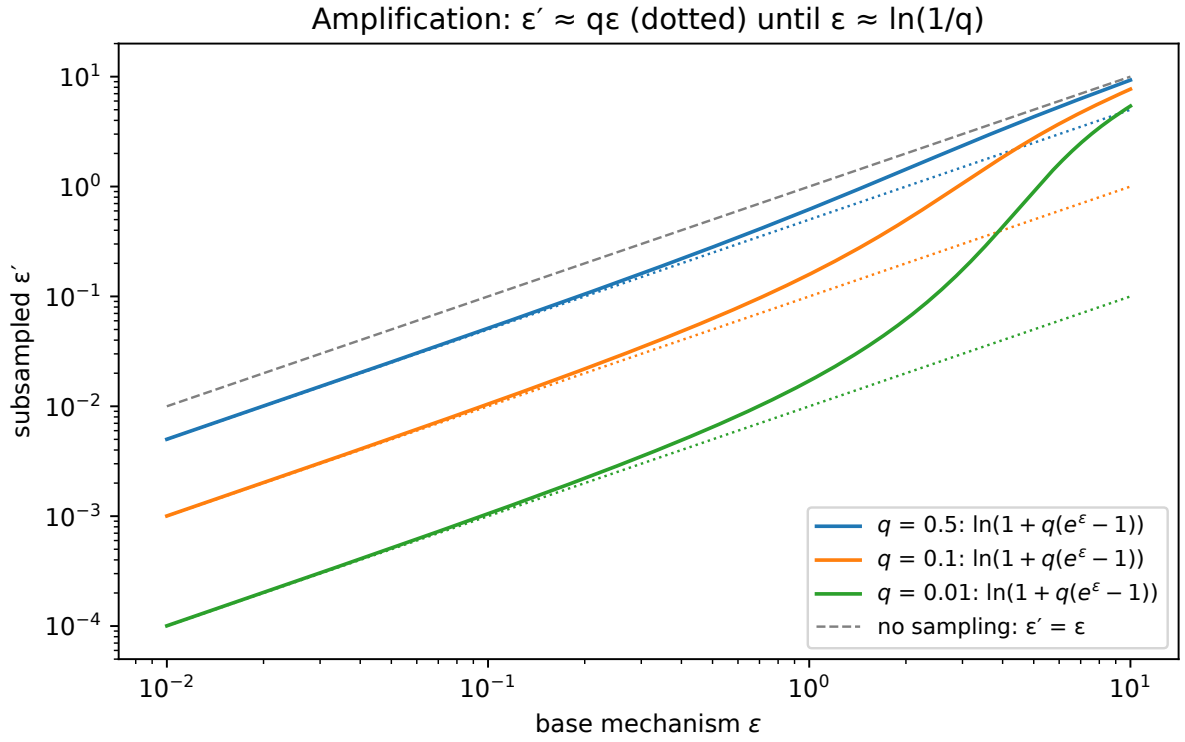


Figure 33.1.: The subsampled ε' against the base ε for three sampling rates, with $q\varepsilon$ dotted and no sampling dashed.

Table 33.1.: The amplified $\varepsilon' = \ln(1 + q(e^{\varepsilon} - 1))$ for three sampling rates.

q	$\varepsilon = 0.1$	$\varepsilon = 1$	$\varepsilon = 5$	$\varepsilon = 10$
0.01	0.0011	0.0170	0.9059	5.3993
0.1	0.0105	0.1586	2.7563	7.6978
0.5	0.0512	0.6201	4.3136	9.3069

Example 33.1. Take the randomized response of step 5 of the proof of Theorem 33.1, with $\varepsilon = 1$ and $q = 0.1$. When the record is not drawn the true bit is 0, which is reported as 0 with probability $e/(1 + e) = 0.7311$ and as 1 with probability 0.2689. When it is drawn the two probabilities swap. The mixture weights the two cases by 0.9 and 0.1, so it reports 1 with probability $0.9 \times 0.2689 + 0.1 \times 0.7311 = 0.3152$ and 0 with probability 0.6848.

Without sampling the likelihood ratios are e and $1/e$. With sampling the largest ratio of the mixture to P_0 is $0.3152/0.2689 = 1.1718 = 1 + 0.1(e-1)$, so $\varepsilon' = \ln 1.1718 = 0.1586$, as Equation 33.8 says, against $q\varepsilon = 0.1$. The largest ratio the other way, $0.7311/0.6848 = 1.0675$, is smaller. The mixture never puts less than 0.9 times P_0's probability on any output, so this ratio can never exceed $1/0.9$.

33.2. Sampling schemes and neighbouring relations

Poisson sampling gives batches of random size, but most training code draws batches of a fixed size. Does Theorem 33.1 still apply to them? This section says which pair of outputs to account for under each scheme, because accounting for one scheme while running another can multiply ε several times over. A fixed batch size pairs naturally with **replace-one** neighbours, datasets of the same size that differ in one record's value, because adding a record would change the size.

Definition 33.2 (Fixed-size batches)**. Sampling without replacement** draws a batch of m of the n records of a dataset uniformly at random. **Shuffle and partition** puts the records in a uniformly random order once per epoch and uses consecutive blocks of m records as the epoch's batches.

A uniformly random batch contains a given record with probability m/n, which plays the role of q. There is no independent coin for the differing record, though, so the proof uses a **coupling**: it builds the batch in a way that has such a coin, and shares the coin and the other records between the two datasets (Balle, Barthe & Gaboardi 2018, Theorem 9).

Theorem 33.2 (Amplification by sampling without replacement)**.** *Let M be (ε, δ)-DP for replace-one neighbours on datasets of size m, and let $q = m/n$. Running M on a batch of m records drawn without replacement from n is (ε', δ')-DP for replace-one neighbours on datasets of size n, with*

$$\varepsilon' = \ln\left(1 + q(e^{\varepsilon} - 1)\right), \qquad \delta' = q\delta. \tag{33.10}$$

Proof. The plan is to draw the batch in two stages that expose a coin for the differing record, write both outputs as the mixtures of Lemma 33.1, check its assumptions, and average. Let D and D' differ in record i, which is x in D and x' in D'.

1. *Build a batch that avoids i.* Draw B, a uniform set of $m-1$ of the other $n-1$ records, then z, a uniform record among the $n-m$ others not in B. Each set of m records that avoids i arises from m choices of z, equally often, so $B \cup \{z\}$ is a uniform such set.
2. *Write both outputs as mixtures.* A uniform batch contains i with probability q. Given that it does, the rest of it is distributed as B, and otherwise the batch is distributed as $B \cup \{z\}$. So flip a coin with probability q, and use $B \cup \{x\}$ in D and $B \cup \{x'\}$ in D' on heads, and $B \cup \{z\}$ in both on tails. Given (B, z), the outputs are $(1-q)R + qP$ and $(1-q)R + qP'$, with $R = M(B \cup \{z\})$, $P = M(B \cup \{x\})$ and $P' = M(B \cup \{x'\})$.
3. *Check the assumptions.* The three batches have size m and differ pairwise in one element, so each of P and P' is (ε, δ)-close to the other two.
4. *Apply the lemma and average.* Lemma 33.1 gives the bound in each direction, and Lemma 33.2 averages over (B, z).

□

Each combination of sampling scheme and neighbouring relation needs its own base guarantee (Table 33.2), and the **zero-out** relation of its last row sets the target's gradient to 0. The last column applies them to DP-SGD's sum of gradients clipped to norm C, with noise σC. Replacing a record moves the sum by up to $2C$, twice what adding one does, since the new gradient may point the opposite way. A Gaussian mechanism's privacy depends only on the shift in units of the noise, so this is the same as a shift of C with noise multiplier $\sigma/2$. Lebeda, Regehr, Kamath & Steinke (2024, Proposition 9) show that for fixed-size batches this shift is the worst case under add and under remove too.

Table 33.2.: What to account for under each combination of sampling scheme and neighbouring relation.

Sampling	Neighbours	Pair to account for	Base guarantee needed	Gaussian sum, clipping norm C
Poisson, rate q	add/remove	$(1-q)P_0 + qP_1$ against P_0	add/remove	means differ by C: noise multiplier σ
Fixed size m of n	replace-one	$(1-q)R + qP$ against $(1-q)R + qP'$, $q = m/n$	replace-one	means differ by $2C$: noise multiplier $\sigma/2$
Fixed size m of n	add/remove	dominated by the Poisson pair at $\sigma/2$	replace-one	$\sigma/2$
Poisson, rate q	replace-one	both sides mixtures	$(2\varepsilon, (1+e^{\varepsilon})\delta)$ by group privacy	means differ by up to $2C$
Shuffle and partition	zero-out	one batch per epoch	no simple formula (Proposition 33.1)	measured, not derived

Poisson sampling under replace-one neighbours is the row where both outputs are mixtures, $(1-q)R + qP$ and $(1-q)R + qP'$, given the rest of the sample $\tilde{D}$. Here $R = M(\tilde{D})$, and P and P' add x or x', so each is one step from R but two from the other. Lemma 33.1 needs one guarantee for all these pairs, so it must use the weaker one. Group privacy (Theorem 19.2) gives $(2\varepsilon, (1+e^{\varepsilon})\delta)$ for datasets two steps apart, and the lemma with Lemma 33.2 then gives

$$\varepsilon' = \ln\left(1 + q(e^{2\varepsilon} - 1)\right), \qquad \delta' = q(1+e^{\varepsilon})\delta. \tag{33.11}$$

For small q this is about $e^{\varepsilon} + 1$ times the add/remove value, since $e^{2\varepsilon} - 1 = (e^{\varepsilon} - 1)(e^{\varepsilon} + 1)$. It is loose, because it charges the pairs one step apart the two-step price. For pure DP, the ratio of the two mixtures at an output is largest where $P = e^{\varepsilon}R$ and $P' = e^{-\varepsilon}R$, which gives the exact value

$$\varepsilon' = \ln\left(1 + q(e^{\varepsilon} - 1)\right) - \ln\left(1 - q(1 - e^{-\varepsilon})\right), \tag{33.12}$$

which the Laplace mechanism on a sum attains. For small ε the exact value is about twice the add/remove value. At $\varepsilon = 1$ and $q = 0.01$ it is 0.02338, against 0.06193 for the group-privacy bound and 0.01704 for add/remove: a ratio of 1.37, not 3.64.

Shuffle and partition is worse, because in one epoch the record is in exactly one batch, so no step's output is a mixture with "not used". The only uncertainty is which batch held the record, and the other records can give that away. Chua et al. (2024) build a bad case under zero-out, with scalar gradients: every other record has gradient -1 and the target has $+1$. Adding the batch size to each batch sum cancels the other records, so batches without the target output $\mathcal{N}(0, \sigma^2)$, while the target's batch outputs $\mathcal{N}(2, \sigma^2)$, or $\mathcal{N}(1, \sigma^2)$ when the target is zeroed out. A natural test says "target present" when the largest batch output exceeds a threshold τ.

Proposition 33.1 (A lower bound for shuffle and partition)**.** *In the epoch above, with b batches and noise multiplier σ, let Φ be the standard normal distribution function. For every threshold τ and every ε, the smallest $\delta(\varepsilon)$ for which the epoch is $(\varepsilon, \delta(\varepsilon))$-DP satisfies*

$$\delta(\varepsilon) \geq \Big[1 - \Phi\Big(\frac{\tau - 2}{\sigma}\Big)\Phi\Big(\frac{\tau}{\sigma}\Big)^{b-1}\Big] - e^{\varepsilon}\Big[1 - \Phi\Big(\frac{\tau - 1}{\sigma}\Big)\Phi\Big(\frac{\tau}{\sigma}\Big)^{b-1}\Big]. \tag{33.13}$$

Proof.

1. *Compute the event's probability.* The largest output stays below τ only if every batch output does, and the batches' noises are independent. With the target present, its batch stays below τ with probability $\Phi(\frac{\tau-2}{\sigma})$, and each of the $b-1$ others with probability $\Phi(\frac{\tau}{\sigma})$. With the target zeroed out, the first factor becomes $\Phi(\frac{\tau-1}{\sigma})$.
2. *Remove the batch index.* Neither product depends on which batch holds the target, so averaging over the random order changes nothing.
3. *Bound $\delta(\varepsilon)$.* The smallest $\delta(\varepsilon)$ is the largest value of $P(A) - e^{\varepsilon}Q(A)$ over events A, the hockey-stick divergence of Definition 3.4. One particular event gives a value no larger.

□

Maximizing over τ gives the best such bound, and so a lower bound on ε at a given δ. Under zero-out a Poisson step's sum also moves by at most C, so the Poisson accountant applies to the same relation. With $\sigma = 0.4$ and $b = 10{,}000$ batches, the script reproduces Chua et al.'s check, $\delta(4) \geq 0.226$ for shuffling against 0.244 for a fixed batch order, where a Poisson accountant gives $\delta(4) \leq 1.18 \times 10^{-5}$.

Example 33.2. In the running example one epoch is $b = 100$ batches, and Table 33.3 compares three batch samplers over it. Deterministic batches use the record once, in a known batch, so the epoch is a plain Gaussian mechanism with an exact ε. Shuffling gets the lower bound of Proposition 33.1, and Poisson sampling the RDP upper bound.

Table 33.3.: The ε at $\delta = 10^{-5}$ of one epoch of 100 batches under three batch samplers. Shuffling is a lower bound and Poisson an upper bound, so wherever the shuffled column exceeds the Poisson column, the Poisson number is provably wrong for shuffled training.

σ	Deterministic batches (exact)	Shuffled batches (lower bound)	Poisson sampling (upper bound)
0.5	10.00	10.00	8.03
1	4.38	4.06	1.21
1.5	2.75	1.46	0.47
2	1.99	0.48	0.26
2.5	1.55	0.20	0.17
3	1.27	0.10	0.14

At $\sigma = 1$, shuffled training has $\varepsilon \geq 4.06$ after one epoch, close to the 4.38 of a fixed order, while the Poisson accountant reports at most 1.21. The gap closes as σ grows (Figure 33.2), and at $\sigma = 3$ this test is too weak to show one. Over 10,000 steps, fixed-size sampled batches give $\varepsilon = 47.42$ by RDP, with noise multiplier $\sigma/2 = 0.5$, against 6.71 for Poisson sampling.

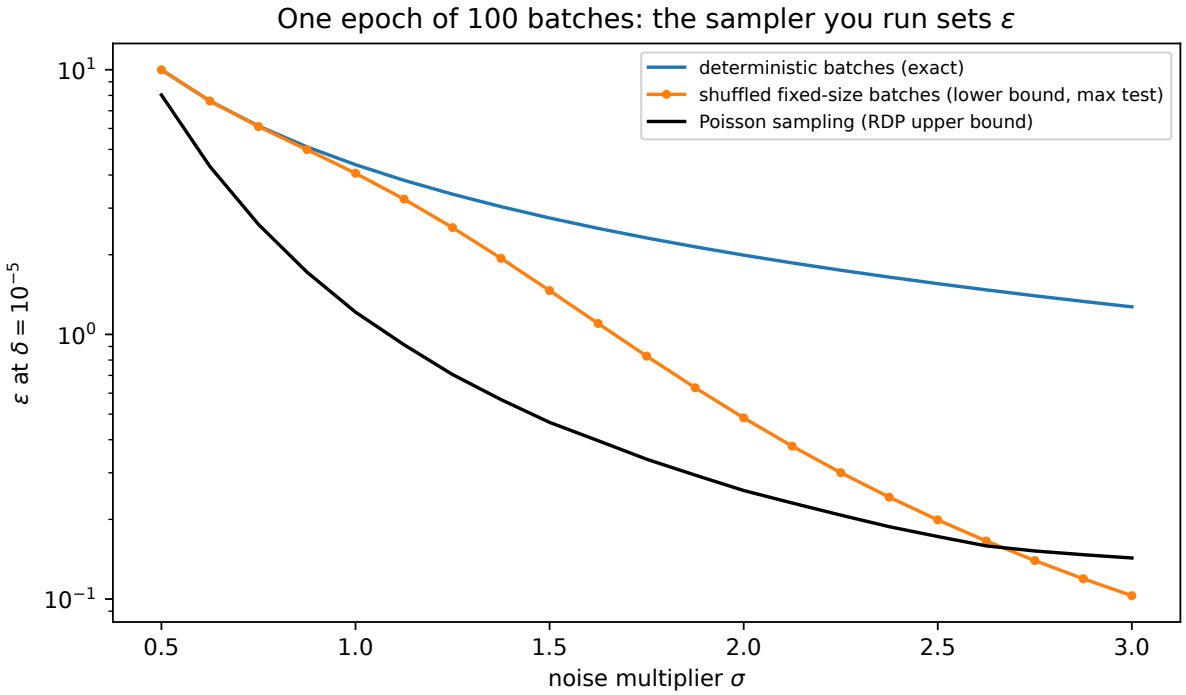


Figure 33.2.: The ε at $\delta = 10^{-5}$ of one epoch of 100 batches against the noise multiplier: deterministic batches (exact), shuffled batches (a lower bound) and Poisson sampling with $q = 0.01$ (an RDP upper bound).

33.3. RDP of the Poisson-subsampled Gaussian

Per-step (ε, δ) amplification composes badly, as Section 33.5 will show, while RDP composes well. A mechanism is $(\alpha, \varepsilon(\alpha))$-RDP when the Rényi divergence of order $\alpha > 1$ between its outputs on any two neighbouring datasets is at most $\varepsilon(\alpha)$, and these curves add under composition (Rényi DP). So DP-SGD's accountant needs the curve of one subsampled Gaussian step, a Rényi divergence between a Gaussian mixture and a Gaussian, which has an exact finite sum at integer orders. Figure 33.3 shows the pair and its **privacy loss**, the logarithm of the likelihood ratio at each output.

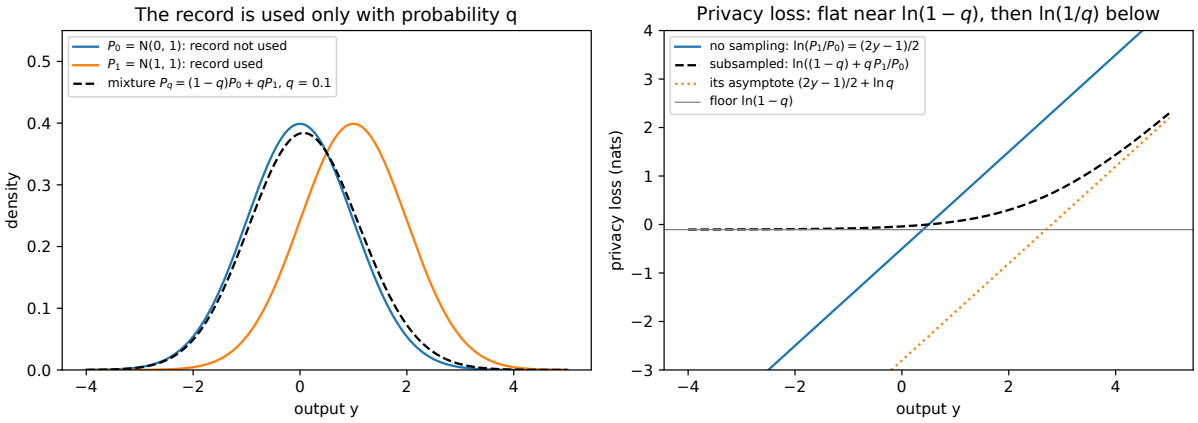


Figure 33.3.: Left: one subsampled Gaussian step with $q = 0.1$; the mixture (dashed) is close to P_0. Right: the privacy loss, flat near $\ln(1-q)$ on the left and $\ln(1/q)$ below the unsubsampled loss in the right tail.

The Rényi divergence of Definition 3.2 is a log-average of a power of the likelihood ratio, in which a larger order weights large ratios more heavily. For $\alpha > 1$,

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \mathbb{E}_{y\sim Q}\Big[\Big(\frac{P(y)}{Q(y)}\Big)^\alpha\Big]. \tag{33.14}$$

The proof below starts, like that of Theorem 33.1, by fixing the rest of the sample, so its first step needs the Rényi counterpart of Lemma 33.2.

Lemma 33.3 (Averaging cannot increase a Rényi divergence)**.** *Let $P = \sum_t w_t P_t$ and $Q = \sum_t w_t Q_t$ be averages with the same weights w_t, and let $\alpha > 1$. Then*

$$D_\alpha(P\|Q) \le \max_t D_\alpha(P_t\|Q_t). \tag{33.15}$$

Proof.

1. *Write the divergence as an integral.* By the definition, $e^{(\alpha-1)D_\alpha(P\|Q)}$ is the integral of $Q\, g(P/Q)$ with the convex function $g(r) = r^\alpha$.
2. *Use joint convexity.* A function of the form $Q\, g(P/Q)$ with g convex, called the **perspective** of g, is jointly convex in the pair (P, Q), as Section F.5 in the appendices derives. So the integral at the averaged pair is at most the average of the integrals:

$$\int Q\, g(P/Q)\, \mathrm{d}y \le \sum_t w_t \int Q_t\, g(P_t/Q_t)\, \mathrm{d}y. \tag{33.16}$$

3. *Take the largest term.* An average is at most its largest term, and the logarithm and division by $\alpha - 1 > 0$ keep the order.

□

This is the **quasi-convexity** of the Rényi divergence (van Erven & Harremoës 2014, Theorem 13). The theorem and its proof are due to Mironov, Talwar & Zhang (2019).

Theorem 33.3 (RDP of the subsampled Gaussian)**.** *Let $P_0 = \mathcal{N}(0, \sigma^2)$, $P_1 = \mathcal{N}(1, \sigma^2)$ and $P_q = (1-q)P_0 + qP_1$. The Poisson-subsampled Gaussian mechanism with ℓ_2 sensitivity 1 and noise multiplier σ is $(\alpha, \varepsilon(\alpha))$-RDP for add/remove neighbours, for every $\alpha > 1$, with*

$$\varepsilon(\alpha) = \max\{D_\alpha(P_q \| P_0),\ D_\alpha(P_0 \| P_q)\} = D_\alpha(P_q \| P_0) = \frac{\ln A_\alpha}{\alpha - 1}, \tag{33.17}$$

where

$$A_\alpha = \mathbb{E}_{y \sim P_0}\Big[\Big((1-q) + q\frac{P_1(y)}{P_0(y)}\Big)^\alpha\Big], \qquad \frac{P_1(y)}{P_0(y)} = e^{(2y-1)/(2\sigma^2)}. \tag{33.18}$$

For integer α,

$$A_\alpha = \sum_{j=0}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha-j} q^j \exp\Big(\frac{j^2 - j}{2\sigma^2}\Big). \tag{33.19}$$

Proof. The plan is to remove every coordinate except the one along which the record moves the output, compare the two directions of the remaining pair, and evaluate the larger one.

1. *Condition on the rest of the sample.* Let f be the clipped gradient sum divided by C, so that its sensitivity is 1, and let $\tilde{D}$ be the rest of the sample. Given $\tilde{D}$, one output is $\mathcal{N}(f(\tilde{D}), \sigma^2 I)$ and the other mixes it, with weight $1-q$, with $\mathcal{N}(f(\tilde{D} \cup \{x\}), \sigma^2 I)$. By Lemma 33.3 it suffices to bound one $\tilde{D}$.
2. *Reduce to one dimension.* Shifting or rotating both distributions changes no likelihood ratio, so rotate the difference of the means onto the first axis. The other coordinates are identical in both runs and independent of the first, and Rényi divergences of independent coordinates add (Theorem 3.2), so they contribute 0.
3. *Reduce to length 1.* Rescaling the output by $1/c$ turns a difference of length $c < 1$ into one of length 1 with noise $\sigma/c > \sigma$. That is the length-1 pair with extra independent noise added, which cannot increase a divergence (Theorem 3.5). What is left is the pair (P_q, P_0) in both orders.
4. *Compare the two directions.* Mironov, Talwar & Zhang (2019, Theorem 5) prove that the order with the mixture first is the larger, for every $\alpha \geq 1$, using the reflection $y \mapsto 1 - y$, which swaps P_0 and P_1. This step is cited, and Section L.1 in the appendices derives it.
5. *Write the first direction as an expectation.* The definition with $Q = P_0$ makes $(\alpha-1)D_\alpha(P_q \| P_0)$ the logarithm of the average of $(P_q/P_0)^\alpha$ under P_0. Dividing the mixture by P_0 gives $(1-q) + q\,P_1/P_0$, which is Equation 33.18. In the Gaussian ratio the normalizing constants cancel, leaving
$$\frac{P_1(y)}{P_0(y)} = \exp\Big(\frac{y^2 - (y-1)^2}{2\sigma^2}\Big) = e^{(2y-1)/(2\sigma^2)}. \tag{33.20}$$
6. *Expand for integer α.* Write r for the ratio P_1/P_0. The binomial theorem expands $(1-q+qr)^\alpha$ into the sum of $\binom{\alpha}{j}(1-q)^{\alpha-j}q^j r^j$ over j, and the average of a sum is the sum of the averages.

7. *Compute each moment.* By the definition of D_j, the average of r^j under P_0 is $e^{(j-1)D_j(P_1\|P_0)}$. Two normals one unit apart have $D_j(P_1\|P_0) = j/(2\sigma^2)$, by Equation 3.19. So the j-th moment is $e^{(j^2-j)/(2\sigma^2)}$, which gives Equation 33.19.

□

For fractional α the sum does not terminate, so the script integrates Equation 33.18 numerically, and the reverse direction too. The dominance of step 4 holds for Rényi divergences only, and for $\delta(\varepsilon)$ Numerical accounting computes both directions.

The curve has two regimes (Figure 33.4). Low orders average the privacy loss over the bulk of the outputs, where sampling flattens it, so the curve stays close to the $j = 2$ term of Equation 33.19, the lowest power of q that costs anything. High orders look at the far tail, where sampling saves only $\ln(1/q)$. Every term of Equation 33.19 is positive, so A_α is at least its last term, and taking logarithms makes this precise:

$$\varepsilon(\alpha) \geq \frac{\alpha \ln q}{\alpha - 1} + \frac{\alpha}{2\sigma^2}, \tag{33.21}$$

the unsubsampled curve minus about $\ln(1/q)$, a bound Concentrated DP also uses.

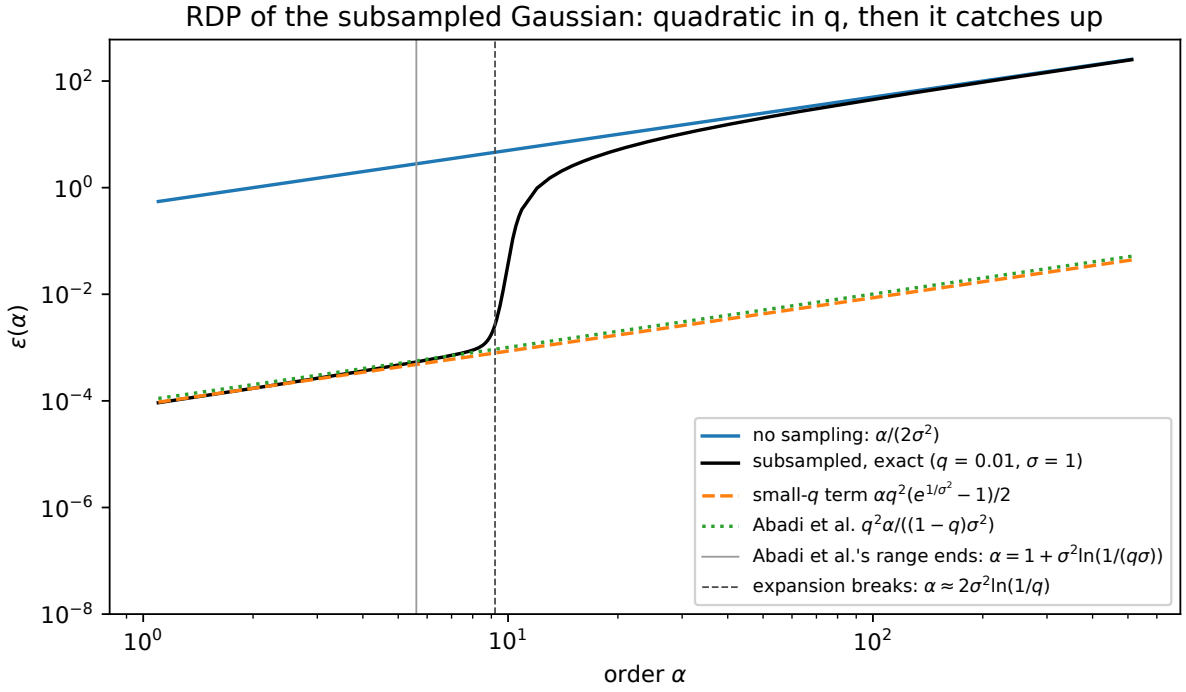


Figure 33.4.: The RDP curve of the subsampled Gaussian with $q = 0.01$, $\sigma = 1$ (black), against the unsubsampled curve, the small-q term and Abadi et al.'s leading term, whose range ends at the solid vertical line. The dashed line marks $\alpha = 2\sigma^2 \ln(1/q)$.

Example 33.3. Take $q = 0.01$ and $\sigma = 1$. At $\alpha = 2$, Equation 33.19 has three terms, one for each number j of factors r. Their coefficients are $(0.99)^2 = 0.9801$, $2 \times 0.99 \times 0.01 = 0.0198$ and $(0.01)^2 = 0.0001$, which add up to $(1-q+q)^2 = 1$. The exponential factors are 1, 1 and e, so they change only the last term, which becomes 0.000272. So $A_2 = 1 + 0.0001 \times 1.71828 = 1.00017183$

and $\varepsilon(2) = \ln A_2 = 1.7181 \times 10^{-4}$, against 1 without sampling. The terms $j = 0$ and 1 cost nothing, because they have the factor 1 that a mixture of identical distributions would have.

The binomial sum is exact but exists only at integer orders, while the integral covers every order but could be wrong if its grid were too coarse. Table 33.4 compares them, and gives the reverse direction $D_\alpha(P_0 \| P_q)$ beside them.

Table 33.4.: The two ways of computing $D_\alpha(P_q \| P_0)$, and the reverse direction, at $q = 0.01$, $\sigma = 1$. A dash marks a value the script does not print, and the sum exists only at integer orders.

α	Binomial sum	Integral	Reverse direction
2	1.718134×10^{-4}	1.718134×10^{-4}	1.6022×10^{-4}
8	—	8.9364×10^{-4}	5.7239×10^{-4}
10	3.827042×10^{-2}	3.827042×10^{-2}	—
32	1.124628×10^{1}	1.124628×10^{1}	1.6923×10^{-3}

The two computations agree to all printed digits. The curve stays near the $j = 2$ term up to $\varepsilon(8) = 8.9364 \times 10^{-4}$, against 4 without sampling, then climbs to $\varepsilon(16) = 3.0879$ and $\varepsilon(32) = 11.246$, against 8 and 16. The reverse direction is smaller at every order, because it looks at the left tail, where P_0/P_q is at most $1/(1-q)$. At every order the accountant uses, it stays smaller for $\sigma = 0.5$ and $\sigma = 2$ too, by a margin of at least 5.09×10^{-4} and 5.67×10^{-8}.

33.4. Small q and the moments accountant

Abadi et al. (2016) introduced DP-SGD together with the **moments accountant**, which tracks the cumulant generating function $K_M(\lambda) = \ln \mathbb{E}[e^{\lambda L}]$ of the privacy loss L, the log-likelihood ratio of an output. Rényi DP shows that this is the RDP curve in disguise, $K_M(\lambda) = \lambda\, \varepsilon(\lambda + 1)$. Abadi et al. bounded K_M for small q, and this section derives the leading term and where it stops holding. The idea is that of Example 33.3: the terms $j = 0$ and 1 cost nothing, and every term from $j = 3$ on carries q^3 or a higher power.

Proposition 33.2 (The subsampled Gaussian for small q)**.** *For $q \to 0$ at fixed α and σ, the RDP curve of Theorem 33.3 is*

$$\varepsilon(\alpha) = \frac{\alpha q^2 (e^{1/\sigma^2} - 1)}{2} + O(q^3) \approx \frac{\alpha q^2}{2\sigma^2} \quad \textit{for large } \sigma. \tag{33.22}$$

Proof. The plan is to subtract the part of the sum that costs nothing, keep the lowest power of q, and expand the logarithm.

1. *Subtract what costs nothing.* Without the exponential factors, Equation 33.19 is $(1-q+q)^\alpha = 1$. Subtracting this 1 leaves each term multiplied by its factor minus 1, and the terms $j = 0$ and 1 vanish because their factor is $e^0 = 1$:

$$A_\alpha = 1 + \sum_{j=2}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha-j} q^j \big(e^{(j^2-j)/(2\sigma^2)} - 1\big). \tag{33.23}$$

2. *Keep the lowest power of q.* The $j = 2$ term is $\binom{\alpha}{2} q^2 (1-q)^{\alpha-2} (e^{1/\sigma^2} - 1)$, and every other term carries q^3 or more.
3. *Take the logarithm.* For small y, $\ln(1+y) = y + O(y^2)$, and $(1-q)^{\alpha-2} = 1 + O(q)$, so

$$\ln A_\alpha = \frac{\alpha(\alpha-1)}{2} q^2 (e^{1/\sigma^2} - 1) + O(q^3). \tag{33.24}$$

4. *Divide by $\alpha - 1$.* This gives Equation 33.22 by Equation 33.17. For large σ, $e^{1/\sigma^2} - 1 \approx 1/\sigma^2$.

□

The cost per step is quadratic in q, not linear as in Theorem 33.1, so the total over E epochs of E/q steps is about $E\, q\, \alpha/(2\sigma^2)$, which shrinks as batches get smaller. Abadi et al. (2016, Lemma 3) prove a bound of the same shape for $\sigma \geq 1$, $q < 1/(16\sigma)$ and integer $\lambda \leq \sigma^2 \ln \frac{1}{q\sigma}$. Divided by λ, with $\alpha = \lambda + 1$, its leading term reads

$$\varepsilon(\alpha) \lesssim \frac{q^2 \alpha}{(1-q)\sigma^2} \quad \text{for} \quad \alpha \leq 1 + \sigma^2 \ln \frac{1}{q\sigma}. \tag{33.25}$$

For large σ this is about twice Equation 33.22, a valid but loose bound, and the source of the "$\varepsilon(\alpha) \approx q^2\alpha/\sigma^2$" you will see quoted.

The real limit of the expansion is its range of orders. Each next term multiplies the power of q by q and the exponential by e^{j/σ^2}, so the terms first fall and later grow. The expansion fails when the last term, $q^\alpha e^{\alpha(\alpha-1)/(2\sigma^2)}$, overtakes the $j = 2$ term, about $q^2 e^{1/\sigma^2}$. Ignoring the binomial coefficients, the difference of their logarithms factors as $(\alpha - 2)(\frac{\alpha+1}{2\sigma^2} - \ln \frac{1}{q})$, which turns positive at $\alpha = 2\sigma^2 \ln(1/q) - 1$. So the expansion holds up to about

$$\alpha \approx 2\sigma^2 \ln(1/q), \tag{33.26}$$

and the curve leaves the quadratic regime abruptly there. Abadi et al.'s range has the same form with about half the constant, so it is conservative.

Example 33.4. At $q = 0.01$, $\ln(1/q) = 4.605$, so Equation 33.26 puts the breakdown at 9.2 for $\sigma = 1$ and 147.4 for $\sigma = 4$. Table 33.5 compares the exact curve with Equation 33.22 and Equation 33.25. For instance, at $\alpha = 2$ and $\sigma = 1$ the Abadi column is $0.0002/0.99 = 2.0202 \times 10^{-4}$.

Table 33.5.: The exact RDP of the subsampled Gaussian against its small-q expansion and Abadi et al.'s leading term, $q = 0.01$, $\sigma = 1$ unless marked.

α	Exact	Leading term	Abadi	Exact / leading	Exact / leading, $\sigma = 4$
2	1.7181×10^{-4}	1.7183×10^{-4}	2.0202×10^{-4}	1.00	1.00
4	3.6315×10^{-4}	3.4366×10^{-4}	4.0404×10^{-4}	1.06	1.00
6	5.8350×10^{-4}	5.1548×10^{-4}	6.0606×10^{-4}	1.13	1.00
8	8.9364×10^{-4}	6.8731×10^{-4}	8.0808×10^{-4}	1.30	1.00
10	3.8270×10^{-2}	8.5914×10^{-4}	1.0101×10^{-3}	44.54	1.01
32	1.1246×10^{1}	2.7493×10^{-3}	3.2323×10^{-3}	4090.67	1.02

At $\sigma = 1$ the exact curve first exceeds twice the leading term at $\alpha = 8.9$, just below 9.2, and at $\sigma = 4$ at 147.4. Up to $\alpha = \sigma^2 \ln(1/q)$ the leading term is within 8% (1.08 at $\alpha = 4.6$ for $\sigma = 1$, 1.05 at $\alpha = 73.7$ for $\sigma = 4$). Beyond the breakdown it is 2246 times too small at $\alpha = 16$ for $\sigma = 1$, so an accountant that used it there would under-report ε. Abadi et al.'s range ends at $\alpha = 5.6$ for $\sigma = 1$, and at $\alpha = 8$ their value is already too small.

Abadi et al.'s headline $\varepsilon \approx 1.26$, for $\sigma = 4$ and 10,000 steps, matches the exact curve converted as in Section 33.5: 1.2586, or 1.0355 with the improved conversion.

33.5. Accounting for DP-SGD

Everything is now in place to account for the running example. How large is its ε, and how much does the accounting route matter? The RDP accountant adds the per-step curve over the T steps and converts the total to (ε, δ) once. Mironov's conversion adds $\ln(1/\delta)/(\alpha-1)$ to the RDP value, and the improved conversion of From RDP to (ε, δ) subtracts two small corrections. Section C.4 and Section C.5 in the appendices derive the result from scratch.

Theorem 33.4 (The RDP accountant for DP-SGD)**.** *Let DP-SGD run T steps that may each depend on the earlier outputs. Each step draws a Poisson sample with rate q, clips every per-example gradient to norm C and adds $\mathcal{N}(0, \sigma^2 C^2 I)$ to the sum. With $\varepsilon(\alpha)$ from Equation 33.17, DP-SGD is (ε, δ)-DP for add/remove neighbours with*

$$\varepsilon = \min_{\alpha} \left[T\,\varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1} \right] \quad \textit{(Mironov)}, \qquad \varepsilon = \min_{\alpha} \left[T\,\varepsilon(\alpha) + \ln \frac{\alpha - 1}{\alpha} - \frac{\ln \delta + \ln \alpha}{\alpha - 1} \right] \quad \textit{(improved)}, \tag{33.27}$$

with the minimum over the grid of orders shared by this part.

Proof.

1. *Identify one step.* Dividing the noisy sum by C makes one step the mechanism of Theorem 33.3, whose curve is $\varepsilon(\alpha)$.
2. *Compose.* RDP curves add under adaptive composition (Rényi DP), so the T steps have the curve $T\,\varepsilon(\alpha)$.
3. *Convert.* Mironov's conversion is Markov's inequality applied to $e^{\lambda L}$ and solved for ε, and the improved one is Balle et al. (2020, Theorem 21). Each holds at every order, so the smallest value over the grid holds too.

□

The naive routes amplify each step in (ε, δ) form and then compose. Each step gets a share δ_0 of δ, and is $(\varepsilon_0, \delta_0)$-DP for the ε_0 that solves the exact Gaussian profile of Theorem 20.3. Then Equation 33.8 turns this into $(\varepsilon_1, q\delta_0)$. Basic composition (Theorem 19.3) adds the parameters to $(T\varepsilon_1, Tq\delta_0)$, which fixes $\delta_0 = \delta/(Tq)$. Advanced composition (Corollary 21.1) grows like $\sqrt{T}$ at the price of an extra slack δ', whose share of δ the script optimizes.

For one step RDP loses, 0.96 against 0.20, because the conversion costs $\ln(1/\delta)/(\alpha-1)$ however small the curve is. RDP pays that cost once, so it wins from $T = 4$ on (Figure 33.5). The best order falls from 9.9 to 4.1 as T grows, because the growing term $T\,\varepsilon(\alpha)$ pulls the minimum toward

the low orders, where the exact curve is 1.06 times the quadratic term at $T = 10{,}000$, against 30.61 at $T = 1$.

The improved conversion saves from 27.5% at $T = 1$ to 9.6% at $T = 10{,}000$, where it gives 6.71 against 7.43, still an upper bound. Numerical accounting computes the tight value, and Choosing an accountant compares all the accountants of this part.

Example 33.5. Table 33.6 and Figure 33.5 give ε for the running example under the four routes.

Table 33.6.: The ε at $\delta = 10^{-5}$ of DP-SGD with $q = 0.01$, $\sigma = 1$ under four accounting routes.

Steps T	RDP, Mironov	RDP, improved	Best order (improved)	Per step, basic	Per step, advanced
1	1.32	0.96	9.9	0.20	1.13
10	1.41	1.04	9.4	3.64	7.88
100	1.61	1.21	8.8	58.01	78.10
1,000	2.54	2.10	7.8	839.39	1258.64
10,000	7.43	6.71	4.1	11296.93	24379.55

Follow the per-step route at $T = 10{,}000$ to see where it loses. With basic composition it must spend $\delta_0 = 10^{-5}/100 = 10^{-7}$ per step, which forces $\varepsilon_0 = 5.35$. That is in the regime where amplification only subtracts (Figure 33.1), so the amplified $\varepsilon_1 = 1.130$ is only about five times smaller, and basic composition then multiplies it by 10,000. Advanced composition is worse at every T here, because the second term of its bound (Theorem 21.1), $T\varepsilon_1(e^{\varepsilon_1} - 1)$, alone exceeds $T\varepsilon_1$ once $\varepsilon_1 > \ln 2$. Its sharper form, with $\tanh(\varepsilon_1/2)$ in place of $e^{\varepsilon_1} - 1$, gives 6433, still useless.

33.6. Amplification for a general mechanism

For (ε, δ)-DP, Equation 33.8 is one formula, tight over the whole class of (ε, δ)-DP mechanisms. Is there an RDP analogue, a map from any mechanism's RDP curve and q to the subsampled curve that is tight? No, and the reason is also why accountants treat the Gaussian as a special case. The expansion of Theorem 33.3 still works for one direction of any mechanism.

Proposition 33.3 (The first direction for any mechanism)**.** *Let M be $(\alpha, \varepsilon(\alpha))$-RDP for add/remove neighbours at every order, let P_0 and P_1 be its outputs on a neighbouring pair, and let $\alpha \geq 2$ be an integer. Then*

$$D_\alpha((1-q)P_0 + qP_1 \| P_0) \leq \frac{1}{\alpha - 1} \ln \left[(1-q)^{\alpha-1}(\alpha q - q + 1) + \sum_{j=2}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha-j} q^j e^{(j-1)\varepsilon(j)} \right]. \tag{33.28}$$

Proof.

1. *Expand.* Steps 5 and 6 of the proof of Theorem 33.3 never used the Gaussian, so they expand the divergence into the moments of $r = P_1/P_0$ under P_0.

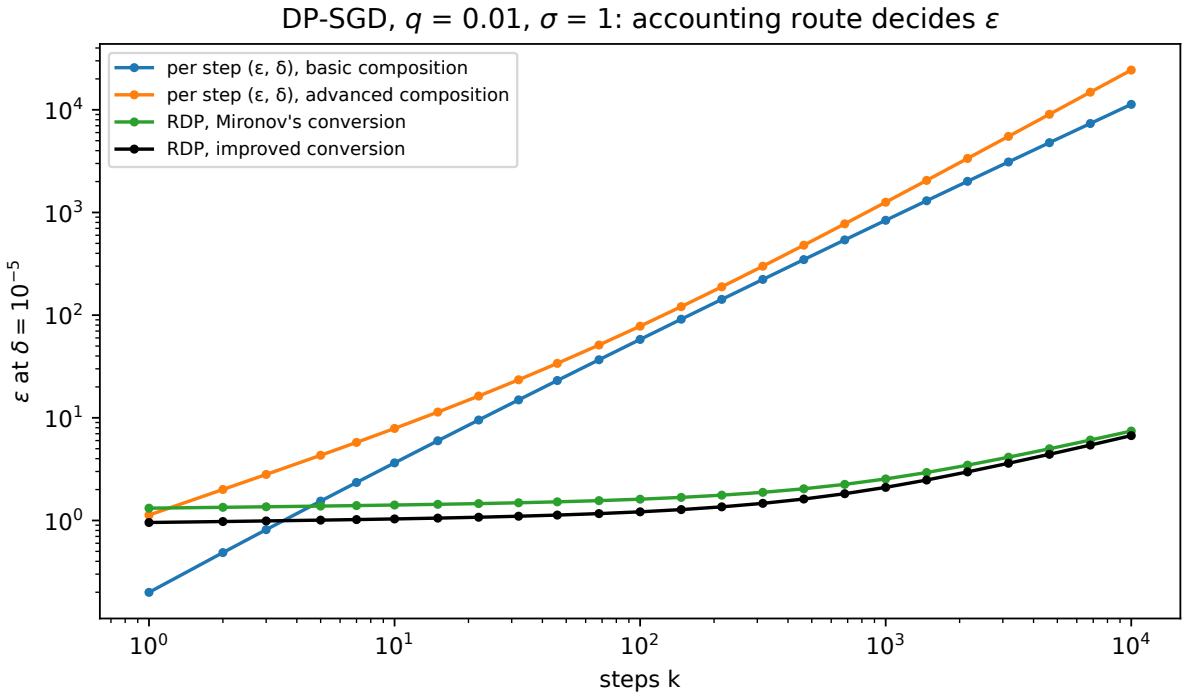


Figure 33.5.: The ε at $\delta = 10^{-5}$ of DP-SGD with $q = 0.01$, $\sigma = 1$ against the number of steps, under four accounting routes.

2. *Bound each moment.* By the definition of D_j, the j-th moment is $e^{(j-1)D_j(P_1\|P_0)}$, which RDP of M bounds by $e^{(j-1)\varepsilon(j)}$ for $j \geq 2$.
3. *Collect the first two terms.* The moments for $j = 0$ and 1 are 1, and their terms add to $(1-q)^\alpha + \alpha q(1-q)^{\alpha-1} = (1-q)^{\alpha-1}(\alpha q - q + 1)$.

□

This is not a tight general answer, for two reasons. Each $\varepsilon(j)$ is a maximum over pairs, taken separately for each j, so the bound is attained only if one pair is the worst at every order at once, as for the Gaussian. And the other direction has no finite expansion and can be the larger one.

Theorem 33.5 (General RDP amplification (Zhu & Wang 2019))**.** *Let M be $(\alpha, \varepsilon(\alpha))$-RDP for add/remove neighbours at every order, and let q be the Poisson rate. For every integer $\alpha \geq 2$, $M \circ S_q$ is RDP at order α with at most*

$$\frac{1}{\alpha-1}\ln\left[(1-q)^{\alpha-1}(\alpha q - q + 1) + \binom{\alpha}{2}q^2(1-q)^{\alpha-2}e^{\varepsilon(2)} + 3\sum_{j=3}^{\alpha}\binom{\alpha}{j}(1-q)^{\alpha-j}q^j e^{(j-1)\varepsilon(j)}\right]. \tag{33.29}$$

The proof is Zhu & Wang's (2019, Theorem 5). It bounds the first direction as Proposition 33.3 does and covers the second with the factor 3. Without that factor the expression is a lower bound for some mechanisms (their Theorem 6), and exact for the Gaussian and Laplace mechanisms (their Theorem 8 and Proposition 10). Wang, Balle & Kasiviswanathan (2019) give a bound of the same form for fixed-size batches.

Example 33.6. Table 33.7 evaluates Equation 33.29 on the running example's step, with the Gaussian's curve $\varepsilon(j) = j/(2\sigma^2)$, against the exact curve of Equation 33.17.

Table 33.7.: Zhu & Wang's general bound applied to the Gaussian against the exact RDP, $q = 0.01$, $\sigma = 1$.

α	2	4	8	16	32
General bound Equation 33.29	1.7181×10^{-4}	4.1880×10^{-4}	1.4036×10^{-3}	3.1611	11.282
Exact Equation 33.17	1.7181×10^{-4}	3.6315×10^{-4}	8.9364×10^{-4}	3.0879	11.246
Ratio	1.00	1.15	1.57	1.02	1.00

The factor 3 costs nothing at $\alpha = 2$, where there are no terms with $j \geq 3$, and adds only about $\ln 3/(\alpha - 1)$ at high orders, where the last term dominates. It costs most in between, a factor 1.57 at $\alpha = 8$, near DP-SGD's best orders for 100 to 1000 steps (Table 33.6). So accountants use Equation 33.19 for the Gaussian.

33.7. Code

The script code/subsampling.py computes every number in this chapter and draws every figure. It computes the subsampled Gaussian's RDP both by the binomial sum and by integration, and reproduces checks from Chua et al. (2024) and Lebeda et al. (2024). From the repository root:

```
uv run python code/part04-privacy-accounting/subsampling.py
```

Table 33.8.: The script's output with the default seed.

Experiment	Result
Mixture lemma, 20,000 random triples	Largest ratio of the left side's excess to $q\delta$: 1.0000
Replace-one under Poisson, $\varepsilon = 1$, $q = 0.01$	Group-privacy bound 0.06193 (ratio 3.64 to add/remove's 0.01704); exact pure-DP value 0.02338 (ratio 1.37)
One epoch, 100 batches, $\sigma = 1$	Poisson ≤ 1.21; shuffled ≥ 4.06; deterministic 4.38
Chua et al. check, $\sigma = 0.4$, 10,000 batches	Shuffled $\delta(4) \geq 0.226$; deterministic 0.244
Lebeda et al.'s setting, $q = 0.001$, $\sigma = 0.8$, 10,000 steps	RDP: Poisson 1.38, fixed-size 15.30
Subsampled Gaussian RDP, $q = 0.01$, $\sigma = 1$	Sum and integral agree to seven digits (Table 33.4)

Experiment	Result
Small-q breakdown, $q = 0.01$	Exact reaches twice the leading term at $\alpha = 8.9$ ($\sigma = 1$) and 147.4 ($\sigma = 4$)
Abadi et al. example, $\sigma = 4$	Mironov 1.2586 at $\alpha = 20$; improved 1.0355 at $\alpha = 17$
Running example, $T = 10{,}000$	RDP improved 6.71 at $\alpha = 4.1$; Mironov 7.43; fixed-size batches 47.42; RDP beats per-step basic composition from $T = 4$

tests/test_subsampling.py checks that the output still matches.

33.8. Summary

Table 33.9.: The results of this chapter, and where each is used.

Result	Statement	Use
Lemma 33.1	A shared part of weight $1 - q$ turns (ε, δ) into $(\varepsilon', q\delta)$	Every amplification result
Lemma 33.2	A bound for each shared choice holds for the average	Fixing the rest of the sample
Theorem 33.1	Poisson, add/remove: $\varepsilon' = \ln(1 + q(e^{\varepsilon} - 1))$, $\delta' = q\delta$, tight per step	A small ε is multiplied by q, a large one loses at most $\ln(1/q)$; replace-one costs about twice as much for small ε (Equation 33.12)
Theorem 33.2	Fixed size m of n, replace-one: the same with $q = m/n$	Fixed-size batches: account at $\sigma/2$
Proposition 33.1	A largest-batch-output test bounds $\delta(\varepsilon)$ from below	No amplification theorem for shuffling; account for the sampler you run
Lemma 33.3	The same for D_α	Reducing a step to one pair
Theorem 33.3	$\varepsilon(\alpha) = \ln A_\alpha/(\alpha - 1)$, a binomial sum at integer orders; exact	The RDP accountant for DP-SGD
Proposition 33.2	$\varepsilon(\alpha) \approx \alpha q^2(e^{1/\sigma^2} - 1)/2$	Quick estimates up to $\alpha \approx 2\sigma^2 \ln(1/q)$, where it breaks abruptly
Theorem 33.4	Compose $T\,\varepsilon(\alpha)$, convert once at the best order	Never compose per-step (ε, δ); 6.71 on the running example
Proposition 33.3	Any mechanism's first direction, by moments	Exact when one pair is worst at every order

Result	Statement	Use
Theorem 33.5	Any mechanism, Poisson: a factor 3 on the terms $j \geq 3$	Other mechanisms; up to 1.57 times exact here

33.9. References

- Kasiviswanathan, Lee, Nissim, Raskhodnikova & Smith, "What Can We Learn Privately?" (FOCS 2008; SIAM Journal on Computing 2011). The first use of sampling to amplify privacy, in private learning. Read it for the origin of the idea; the bound there is not tight.
- Li, Qardaji & Su, "On Sampling, Anonymization, and Differential Privacy: Or, k-Anonymization Meets Differential Privacy" (ACM Symposium on Information, Computer and Communications Security, ASIACCS 2012). The first proof of the tight (ε, δ) amplification for Poisson sampling.
- Balle, Barthe & Gaboardi, "Privacy Amplification by Subsampling: Tight Analyses via Couplings and Divergences" (NeurIPS 2018). Advanced joint convexity (Theorem 2), the tight (ε, δ) amplification for Poisson sampling (Theorem 8), fixed-size sampling with replace-one neighbours (Theorem 9) and sampling with replacement, and tightness via randomized response (Theorem 13). The reference for Section 33.1 and Section 33.2.
- Abadi, Chu, Goodfellow, McMahan, Mironov, Talwar & Zhang, "Deep Learning with Differential Privacy" (CCS 2016). DP-SGD and the moments accountant; Lemma 3 is the small-q bound of Section 33.4. Read it to see where the practice started and which assumptions it made, including Poisson sampling.
- van Erven & Harremoës, "Rényi Divergence and Kullback-Leibler Divergence" (IEEE Transactions on Information Theory, 2014). The properties of the Rényi divergence, including joint quasi-convexity (Theorem 13), used in Lemma 33.3.
- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (arXiv 2019). The reduction to one dimension (Theorem 4), the dominance of the first direction (Theorem 5 and Corollary 7), and the numerically stable computation of A_α (Section 3.3). The reference for Section 33.3 and the basis of most RDP accountants.
- Wang, Balle & Kasiviswanathan, "Subsampled Rényi Differential Privacy and Analytical Moments Accountant" (AISTATS 2019). General RDP amplification for sampling without replacement (Theorem 9) and a lower bound (Proposition 11).
- Zhu & Wang, "Poisson Subsampled Rényi Differential Privacy" (ICML 2019). General RDP amplification for Poisson sampling (Theorem 5), the matching lower bound (Theorem 6), and exactness under a condition that the Gaussian and Laplace mechanisms satisfy (Theorem 8 and Proposition 10).
- Steinke, "Composition of Differential Privacy & Privacy Amplification by Subsampling" (arXiv 2022). A self-contained survey chapter with clean proofs of composition and of subsampling; the best second reading after this chapter.
- Chua, Ghazi, Kamath, Kumar, Manurangsi, Sinha & Zhang, "How Private are DP-SGD Implementations?" (ICML 2024). Shows that shuffle-and-partition batching can be much less private than the Poisson accountant reports, with the largest-batch-output test of Proposition 33.1.
- Lebeda, Regehr, Kamath & Steinke, "Avoiding Pitfalls for Privacy Accounting of Subsampled

Mechanisms under Composition" (arXiv:2405.20769, 2024; IEEE SaTML 2025). Fixed-size against Poisson sampling (the factor two in the shift, and $\varepsilon \approx 1$ against $\varepsilon > 10$), and worst-case datasets that fail under composition.

- Zhu, Dong & Wang, "Optimal Accounting of Differential Privacy via Characteristic Function" (AISTATS 2022). Dominating pairs for subsampled mechanisms (Theorem 11), used by Lebeda et al. and by Numerical accounting.
- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020). The improved conversion used in Equation 33.27 (Theorem 21).

34. Numerical accounting with privacy loss distributions

You trained a model with differential privacy (DP) by 10,000 steps of stochastic gradient descent (SGD). In this algorithm, DP-SGD, each step sampled every record independently with probability $q = 0.01$, clipped each gradient to a fixed norm, and added Gaussian noise whose standard deviation is $\sigma = 1$ times that norm. You need the smallest ε for which the whole run is $(\varepsilon, 10^{-5})$-DP for datasets that differ by adding or removing one record. Rényi differential privacy (RDP), from Rényi DP, gives an upper bound of unknown slack, 6.7127, and the Gaussian central limit theorem (CLT) of f-DP and Gaussian DP gives an approximation, 6.0071. This chapter brackets the truth, here between 6.1377 and 6.2377, and a finer grid narrows the bracket at a known cost.

The one idea is to track the whole distribution of the privacy loss. Composition showed that $\delta(\varepsilon)$ is an average of a non-decreasing function of the loss, and that the loss of a composition is the sum of the step losses. For independent steps the total loss therefore has a convolution as its distribution, which a computer can evaluate once the loss sits on a grid of width h. Rounding every loss up can only raise δ and rounding down can only lower it, so the two roundings bracket the truth, and after k steps the bracket is at most kh wide in ε.

The running example is this run, for up to $T = 10{,}000$ steps. Because q is the sampling rate, the densities of the two output distributions P and Q are written $P(y)$ and $Q(y)$ rather than p and q. Table 34.1 lists the symbols.

Table 34.1.: The symbols of this chapter.

Symbol	Meaning
P, Q; $P(y)$, $Q(y)$	Output distributions on D and on its neighbour D'; their densities
$L(y)$	Privacy loss of the output y, the log of $P(y)/Q(y)$
ω	Privacy loss distribution, the law of $L(Y)$ with $Y \sim P$
μ	Shift between two Gaussians, in standard deviations
h; τ	Grid width of a discretized loss; tail mass cut off by truncation
k; T	Number of composed steps; number of DP-SGD steps
q; σ	Sampling rate; noise multiplier

Symbol	Meaning
Φ	Standard normal distribution function

34.1. The privacy loss distribution

The guarantee you report is $\delta(\varepsilon) = E_{e^\varepsilon}(P\|Q)$, the **hockey-stick divergence** of Divergences, the smallest δ for which $P(S) \le e^\varepsilon Q(S) + \delta$ holds for every set S of outputs. Computed directly it is a sum over outputs, and the output space of a training run is enormous. This section shows that the one-dimensional distribution of the privacy loss (Definition 21.1) carries all of $\delta(\varepsilon)$.

Definition 34.1 (Privacy loss distribution)**.** Let P and Q be the output distributions of a mechanism on neighbouring datasets $D \sim D'$, with densities $P(y)$ and $Q(y)$. The **privacy loss** of an output y is $L(y) = \ln \frac{P(y)}{Q(y)}$, which is $+\infty$ where $Q(y) = 0 < P(y)$ and $-\infty$ where $P(y) = 0 < Q(y)$. The **privacy loss distribution (PLD)** ω of the pair is the law of $L(Y)$ with $Y \sim P$. Its atom $\omega(\{+\infty\})$ is the P-mass of the outputs that are impossible under D'.

An output adds to δ only if its loss ℓ exceeds ε, and then it adds the fraction $1 - e^{\varepsilon-\ell}$ of its probability, so δ is an average of that weight over the PLD. Equivalently, it is the gap $P(S) - e^\varepsilon Q(S)$ on the set S of outputs whose loss exceeds ε, which is the set that makes the gap largest. The Q-probability of that set needs no extra information, because reweighting by the likelihood ratio recovers it from ω.

Theorem 34.1 (δ is an average over the PLD)**.** *Let ω be the PLD of a pair (P, Q), including its atom at $+\infty$. Then for every real ε,*

$$\delta(\varepsilon) = \mathbb{E}_{L\sim\omega}\big[(1 - e^{\varepsilon - L})_+\big] = \Pr_P[L > \varepsilon] - e^\varepsilon \Pr_Q[L > \varepsilon], \tag{34.1}$$

where $(x)_+ = \max\{x, 0\}$ and an infinite loss gets the weight 1. The law of L under Q is fixed by ω: it is $e^{-\ell}\,\omega(\mathrm{d}\ell)$ on the finite losses, plus an atom at $-\infty$ of size $1 - \mathbb{E}_\omega[e^{-L}]$.

Proof. The plan is to take the first form from the composition chapter, split its weight into two terms, and move the second term from P to Q by reweighting.

1. *The first form.* Lemma 21.1 proved it: each term $(P(y) - e^\varepsilon Q(y))_+$ of the hockey-stick sum equals $P(y)(1 - e^{\varepsilon - L(y)})_+$, and outputs with $Q(y) = 0 < P(y)$ add their whole probability.
2. *Split the weight.* When $L > \varepsilon$ the bracket $1 - e^{\varepsilon - L}$ is positive, and otherwise its positive part is 0, so
$$(1 - e^{\varepsilon - L})_+ = \mathbf{1}[L > \varepsilon] - e^\varepsilon\, e^{-L}\, \mathbf{1}[L > \varepsilon]. \tag{34.2}$$
The first term averages to $\Pr_P[L > \varepsilon]$.
3. *Reweight from P to Q.* An output with $P(y) > 0$ has Q-probability equal to its P-probability times $Q(y)/P(y) = e^{-L(y)}$. This is the change of measure of Lemma 3.2 with the roles of P and Q swapped, and it reaches only the outputs that P can produce. For any function g of the loss it gives
$$\mathbb{E}_Q\big[g(L)\,\mathbf{1}[P > 0]\big] = \mathbb{E}_P\big[g(L)\, e^{-L}\big]. \tag{34.3}$$

4. *The second form.* Take g to be the indicator of a loss above ε. The outputs with $P(y) = 0$ have $L = -\infty$ and are not in that event, so the factor $\mathbf{1}[P > 0]$ can be dropped, and the second term of step 2 averages to $e^{\varepsilon} \Pr_Q[L > \varepsilon]$.
5. *The law under Q.* With g the indicator of a set A of finite losses, step 3 gives $\Pr_Q[L \in A] = \int_A e^{-\ell}\omega(\mathrm{d}\ell)$. With $g = 1$ it gives the Q-mass of the outputs where $P > 0$, which is $\mathbb{E}_\omega[e^{-L}]$. The rest of Q sits where $P = 0$, at $L = -\infty$.

□

So two pairs with the same PLD have the same $\delta(\varepsilon)$, whatever their output spaces. The weight $(1 - e^{\varepsilon-\ell})_+$ is 0 up to $\ell = \varepsilon$ and then rises towards 1, so $\delta(\varepsilon)$ is the upper tail of the PLD beyond ε, down-weighted near ε (Figure 34.1). Because the weight is at most 1, $\delta(\varepsilon) \le \Pr_P[L > \varepsilon]$, and because losses just above ε get a weight close to 0, that tail bound is loose.

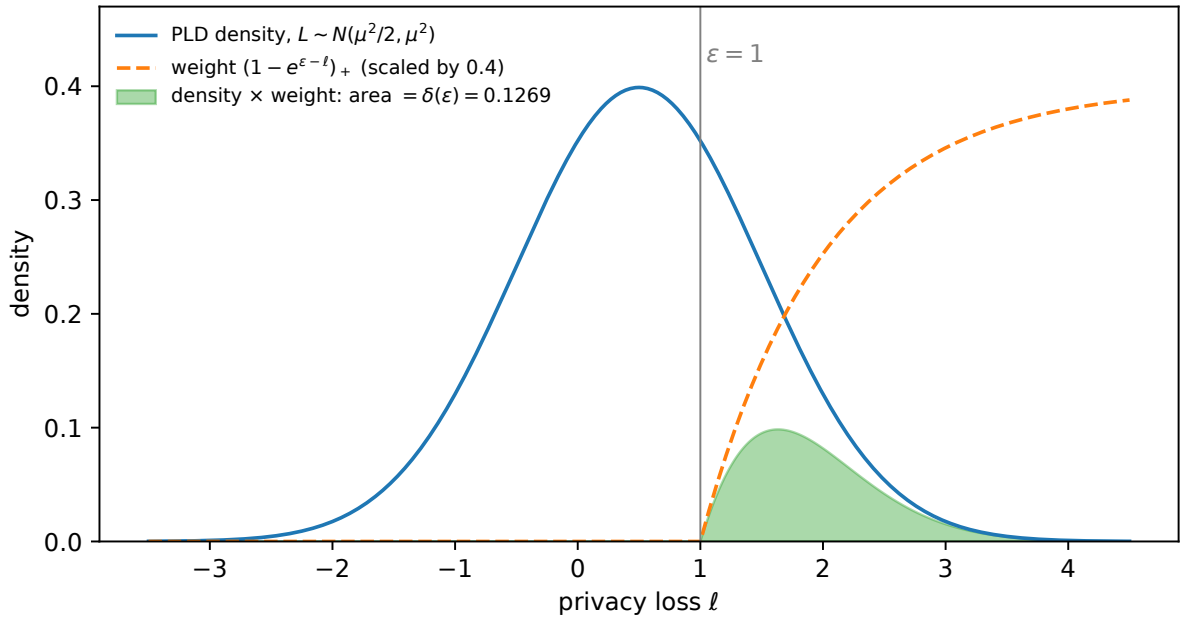


Figure 34.1.: The Gaussian PLD for $\mu = 1$ (blue), the weight $(1 - e^{\varepsilon-\ell})_+$ at $\varepsilon = 1$ (orange, scaled by 0.4 to fit), and their product (green). The green area is $\delta(1) = 0.1269$.

Not every distribution is a PLD. By step 5, $\mathbb{E}_\omega[e^{-L}]$ is a Q-mass, so it is at most 1. Section 34.3 will round a PLD and must know that the result still belongs to a real pair, so it needs the converse, that every distribution with this property is a PLD.

Proposition 34.1 (Which distributions are PLDs)**.** *A distribution ω on $\mathbb{R} \cup \{+\infty\}$ is the PLD of some pair of distributions if and only if $\mathbb{E}_\omega[e^{-L}] \le 1$, where $e^{-\infty} = 0$.*

Proof.

1. *Necessity.* By Theorem 34.1, $\mathbb{E}_\omega[e^{-L}]$ is the Q-mass of the outputs where $P > 0$, and no mass exceeds 1.

2. *Build P.* For the converse, let the outputs be the losses themselves, and take $P = \omega$ on the real line, with its atom at $+\infty$, if any, on an extra point y_+.
3. *Build Q.* Put $Q(\mathrm{d}\ell) = e^{-\ell}\omega(\mathrm{d}\ell)$ on the finite losses, of total mass at most 1 by assumption, and the missing mass $1 - \mathbb{E}_\omega[e^{-L}]$ on a second extra point y_-, where P is zero.
4. *Check the loss.* At each finite ℓ the ratio of the two masses is $1/e^{-\ell}$, so the loss is ℓ, and at y_+, where Q is zero, it is $+\infty$. So the loss under P has law ω.

□

For the **Gaussian pair** $P = \mathcal{N}(0, \sigma^2)$, $Q = \mathcal{N}(s, \sigma^2)$ of a query whose answer moves by s between the neighbours, write $\mu = s/\sigma$ for the shift in noise standard deviations. The loss is then linear in the output, so it is normal, with mean $\mu^2/2$ and variance μ^2 under P and mean $-\mu^2/2$ under Q (Lemma 20.3). Inserting the two normal tails into Equation 34.1 gives the exact profile of Theorem 20.3, which is also the curve of Gaussian differential privacy (GDP) with parameter μ in f-DP and Gaussian DP:

$$L \sim \mathcal{N}(\tfrac{\mu^2}{2}, \mu^2) \text{ under } P, \qquad \delta(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big). \tag{34.4}$$

The law under Q agrees with Theorem 34.1, since tilting a normal density by $e^{-\ell}$ moves its mean down by its variance.

Example 34.1. With sensitivity 1 and $\sigma = 1$, $\mu = 1$ and the PLD is $\mathcal{N}(0.5, 1)$. A million samples in the chapter's script (Section 34.7) give a mean loss of 0.4990, a variance of 1.0013 and $\mathbb{E}[e^{-L}] = 1.0015$, against the exact 0.5, 1 and 1. The green area of Figure 34.1 is Equation 34.4 at $\varepsilon = 1$:

$$\delta(1) = \Phi(-0.5) - e\,\Phi(-1.5) = 0.3085 - 0.1816 = 0.1269. \tag{34.5}$$

Table 34.2 checks Equation 34.4 against numerical integration over the PLD density and against Monte Carlo. Solving $\delta(\varepsilon) = 10^{-5}$ shows that the mechanism is $(4.3772, 10^{-5})$-DP.

Table 34.2.: δ(ε) of the Gaussian pair with $\mu = 1$, computed three ways.

ε	Closed form Equation 34.4	Quadrature over the PLD	Monte Carlo (10^6 draws)
0	0.382925	0.382925	0.382644
0.5	0.238422	0.238422	0.238748
1	0.126937	0.126937	0.126967
2	0.020924	0.020924	0.020922

34.2. Composition is convolution

DP-SGD releases T noisy gradients, and more generally a composition of k mechanisms outputs a tuple $(y_1, \dots, y_k)$, whose $\delta(\varepsilon)$ you want. The bounds of Composition, including the optimal composition theorem (Theorem 21.4), see each step only through its pair $(\varepsilon_i, \delta_i)$, so they must cover the worst PLD with those values. With the PLDs themselves, composition is exact. The loss of a tuple is the sum of the per-step losses (Lemma 21.2), and the law of a sum of independent variables is the **convolution** of their laws.

Theorem 34.2 (Composition is convolution)**.** *Let k steps run independently, step i with the pair (P_i, Q_i) and the PLD ω_i. The product pair $(P_1 \times \cdots \times P_k,\ Q_1 \times \cdots \times Q_k)$ has PLD*

$$\omega_{1..k} = \omega_1 * \omega_2 * \cdots * \omega_k, \qquad \omega^{*k} \text{ for } k \text{ identical steps}, \tag{34.6}$$

where $*$ is the convolution of distributions and a sum that contains $+\infty$ is $+\infty$. For k Gaussian steps with shift μ the result is again Gaussian,

$$\mathcal{N}(\tfrac{\mu^2}{2}, \mu^2)^{*k} = \mathcal{N}(\tfrac{k\mu^2}{2}, k\mu^2), \tag{34.7}$$

the PLD of one Gaussian pair with shift $\sqrt{k}\,\mu$.

Proof. The plan is to follow the probability of the tuple from a product, to a sum of losses, to a convolution.

1. *Losses add.* By Lemma 21.2, the loss of the tuple is $\sum_i L_i(y_i)$, with $L_i = \ln(P_i/Q_i)$. A sum that contains $+\infty$ is $+\infty$, because once one step gives the dataset away, the whole run does.
2. *The terms are independent.* Under the product of the P_i, the outputs Y_i are independent with $Y_i \sim P_i$, and each $L_i(Y_i)$ depends only on its own Y_i, with law ω_i.
3. *A sum's law is the convolution.* The chance that two independent terms sum to ℓ adds, over every split $\ell = a + b$, the chance of a times the chance of b, which is what convolution computes. Adding one term at a time gives Equation 34.6.
4. *Gaussian steps.* A sum of independent normals is normal, with the means and the variances added, which gives mean $k\mu^2/2$ and variance $k\mu^2$.

□

In DP-SGD each gradient depends on the model, which depends on the earlier outputs, so step 2 fails: the second step's pair, and with it its loss, depends on the first output. The way around is to replace each step by one fixed pair that is at least as easy to tell apart, whatever happened before. Comparing pairs at every level of strictness uses the hockey-stick divergence with a general weight $\gamma \geq 0$ in place of e^{ε},

$$E_\gamma(P\|Q) = \sum_y \big(P(y) - \gamma Q(y)\big)_+. \tag{34.8}$$

Definition 34.2 (Dominating pair)**.** A pair (P, Q) **dominates** a mechanism M if $E_\gamma(M(D)\|M(D')) \leq E_\gamma(P\|Q)$ for all neighbours $D \sim D'$ and all $\gamma \geq 0$ (Zhu, Dong & Wang 2022, Definition 7).

Every (ε, δ) statement true of a dominating pair is then true of the mechanism, and domination survives adaptive composition.

Theorem 34.3 (Dominating pairs compose)**.** *Run k mechanisms in sequence, each chosen as a function of the earlier outputs. If (P_i, Q_i) dominates step i for every value of the earlier outputs, then $(P_1 \times \cdots \times P_k,\ Q_1 \times \cdots \times Q_k)$ dominates the adaptive composition. So the convolution of the dominating PLDs gives a valid $\delta(\varepsilon)$ for the adaptive run.*

Its proof is in Zhu, Dong & Wang (2022, Theorem 10), and Section K.3 in the appendices derives it step by step. It compares the adaptive run with the fixed pairs one step at a time, and that comparison needs E_γ for every $\gamma \geq 0$, not only for $\gamma \geq 1$.

Equation 34.6 has no slack at all, so the only errors in a PLD accountant are the ones you introduce to compute the convolution, by discretization (Section 34.3) and in floating point (Section 34.4). The bound is only as tight as the dominating pair, though. For the Gaussian mechanism with ℓ_2 sensitivity 1, the pair $\mathcal{N}(0, \sigma^2)$, $\mathcal{N}(1, \sigma^2)$ dominates and is attained by some neighbouring datasets, so the result is tight, while a subsampled mechanism needs one pair for adding a record and one for removing it (Section 34.5).

Example 34.2. One Gaussian step with $\sigma = 1$ has $\mu = 1$ and PLD $\mathcal{N}(0.5, 1)$. Ten steps add ten means and ten variances, so their PLD is $\mathcal{N}(5, 10)$, a single Gaussian pair with $\mu = \sqrt{10}$. Solving Equation 34.4 with that μ for $\delta = 10^{-5}$, the composition is $(17.8566, 10^{-5})$-DP. For 100 steps, $\mu = \sqrt{100} = 10$ and $\varepsilon = 91.8173$. Section 34.4 recovers both numbers from a discretized PLD.

34.3. Pessimistic and optimistic discretization

A computer cannot convolve continuous distributions, so the loss must sit on a finite grid $\{ih : i \in \mathbb{Z}\}$ of width h, and any rounding changes δ. The question is whether the error can have a known sign for every number of compositions, so that the answer is still a guarantee. It can, because the weight $(1 - e^{\varepsilon - \ell})_+$ never decreases as the loss grows, so moving every loss up can only raise δ and moving every loss down can only lower it.

Definition 34.3 (Pessimistic and optimistic PLDs)**.** Let ω be a PLD, $h > 0$ a grid width and $\tau \geq 0$ a tail mass. The **pessimistic** PLD $\omega^{\uparrow}$ moves every loss $\ell \in ((i-1)h, ih]$ up to ih. It may also truncate each tail, of mass at most τ, moving the top one to $+\infty$ and the bottom one up to the lowest kept grid point. The **optimistic** PLD $\omega^{\downarrow}$ moves every loss $\ell \in [ih, (i+1)h)$ down to ih. It may move the top tail down to the highest kept grid point, and drop the bottom tail, which leaves a total mass just below 1.

To compare averages before and after such moves, the next proof builds the rounded loss from the same random draw as the true one, a **coupling**, and its step 3 needs one lemma.

Lemma 34.1 (Monotone coupling)**.** *Let g be non-decreasing, and let X and X' be random variables built on the same random draw with $X \leq X'$ for every outcome. Then $\mathbb{E}[g(X)] \leq \mathbb{E}[g(X')]$.*

Proof. For every outcome $g(X) \leq g(X')$, because g is non-decreasing. Averaging an inequality that holds outcome by outcome keeps it. □

Write $\delta_\nu(\varepsilon)$ for Equation 34.1 evaluated on a distribution ν.

Theorem 34.4 (The discretization sandwich)**.** *Let ω be the PLD of a pair (P, Q). For every number of steps k and every ε,*

$$\delta_{(\omega^{\downarrow})^{*k}}(\varepsilon) \;\leq\; \delta_{\omega^{*k}}(\varepsilon) \;\leq\; \delta_{(\omega^{\uparrow})^{*k}}(\varepsilon), \tag{34.9}$$

and the same holds for convolutions of different PLDs, each rounded the same way. Moreover:

1. *$\omega^{\uparrow}$ is the PLD of a pair that dominates (P, Q), so the upper bound also holds for adaptive composition.*
2. *Without truncation, let $\varepsilon^{\uparrow}(\delta)$ and $\varepsilon^{\downarrow}(\delta)$ be the values of ε at which the composed pessimistic and optimistic curves reach δ. Then $\varepsilon^{\uparrow}(\delta) \le \varepsilon^{\downarrow}(\delta) + kh$.*

Proof. The plan is to build both roundings from the same draw of the loss, add up k steps, and apply Lemma 34.1 to the weight of Equation 34.1. Validity, domination and the width then follow from the same coupling.

1. *Couple one step.* Draw $L \sim \omega$ and set $L^{\uparrow} = h\lceil L/h \rceil$ and $L^{\downarrow} = h\lfloor L/h \rfloor$, which have the laws $\omega^{\uparrow}$ and $\omega^{\downarrow}$. A number lies between its two roundings, which are at most one grid step apart, so for every outcome
$$L^{\downarrow} \le L \le L^{\uparrow} \le L^{\downarrow} + h. \tag{34.10}$$
2. *Add k steps.* Coupling each of k independent steps as in step 1 and adding the inequalities gives
$$\sum_i L_i^{\downarrow} \le \sum_i L_i \le \sum_i L_i^{\uparrow} \le \sum_i L_i^{\downarrow} + kh, \tag{34.11}$$
and by Theorem 34.2 each sum has the convolved law of its PLDs.
3. *Apply the lemma.* By Equation 34.1, $\delta(\varepsilon)$ is the average of $g_\varepsilon(\ell) = (1 - e^{\varepsilon - \ell})_+$ at the total loss, and g_ε is non-decreasing. Lemma 34.1 applied to the first two inequalities of step 2 gives Equation 34.9.
4. *Truncation keeps the sign.* The pessimistic truncation moves mass up, so Lemma 34.1 still applies. The optimistic truncation moves mass down, which the lemma covers in the other direction, or deletes it, which removes a non-negative term from δ. Both can be repeated after every convolution.
5. *The pessimistic PLD belongs to a pair.* Rounding up raises every loss and so lowers e^{-L}, and mass moved to $+\infty$ contributes $e^{-\infty} = 0$. So $\mathbb{E}_{\omega^{\uparrow}}[e^{-L}] \le \mathbb{E}_{\omega}[e^{-L}] \le 1$, and Proposition 34.1 builds a pair $(P^{\uparrow}, Q^{\uparrow})$ with PLD $\omega^{\uparrow}$ (Doroshenko et al. 2022, Lemma 4.2, prove it for this rounding).
6. *That pair dominates.* Step 3 with $k = 1$ says $E_{e^\varepsilon}(P\|Q) \le E_{e^\varepsilon}(P^{\uparrow}\|Q^{\uparrow})$ for every real ε, that is for every $\gamma = e^{\varepsilon} > 0$. At $\gamma = 0$ both divergences are the total mass 1. So $(P^{\uparrow}, Q^{\uparrow})$ dominates (P, Q) at every $\gamma \ge 0$, and Theorem 34.3 extends the bound to adaptive composition.
7. *The width.* Write $\delta^{\uparrow}$ and $\delta^{\downarrow}$ for the composed pessimistic and optimistic curves. Adding a constant c to the loss shifts its δ curve to the right by c, because g_ε sees only the difference $\varepsilon - \ell$:
$$\mathbb{E}[g_\varepsilon(X + c)] = \mathbb{E}[g_{\varepsilon - c}(X)]. \tag{34.12}$$
The last inequality of step 2 and Lemma 34.1 then give $\delta^{\uparrow}(\varepsilon) \le \delta^{\downarrow}(\varepsilon - kh)$. So wherever the optimistic curve has dropped to δ, the pessimistic curve has dropped to it at most kh further right.

□

Rounding down raises e^{-L}, so $\mathbb{E}[e^{-L}]$ can exceed 1, and then $\omega^{\downarrow}$ is no pair's PLD. Its δ still lies below that of k non-adaptive copies of (P, Q), so it bounds the mechanism's δ from below whenever some neighbours actually produce (P, Q), as they do for the Gaussian and the subsampled Gaussian. Each truncation of mass τ adds at most τ to δ, since a weight never exceeds 1, and a few dozen truncations at $\tau = 10^{-15}$ are far below any reported δ.

For a continuous PLD the width is almost exactly kh. Such a PLD puts no mass on the grid points, so every loss moves by exactly h between its two roundings, and a fixed width at 10^4 steps needs a grid 10^4 times finer than for one step. When the losses spread evenly inside each cell, each rounding moves a loss by about $h/2$ on average, so the midpoint of the two sides is a nearly unbiased estimate, but only the two sides are guarantees.

Example 34.3. Randomized response with $\varepsilon_0 = \ln 3$ (The definition of differential privacy) reports the true bit with probability $\frac{3}{4}$. The true report has loss $\ln 3 = 1.0986$, with probability 0.7500, and the flipped one has loss -1.0986, with probability 0.2500. On the grid $h = 0.5$, rounding up gives the losses $\{-1, 1.5\}$ and rounding down gives $\{-1.5, 1\}$. The rounded-up PLD is valid:

$$\mathbb{E}[e^{-L}] = 0.75\, e^{-1.5} + 0.25\, e^{1} = 0.1673 + 0.6796 = 0.8469 \le 1. \tag{34.13}$$

The rounded-down one has $\mathbb{E}[e^{-L}] = 1.3963$, which no pair can have.

Table 34.3 follows the four outcomes of three steps, indexed by the number j of steps with a positive loss. The exact total is $(2j-3)\ln 3$, the rounded-up total is $1.5j - (3-j)$, and the rounded-down total is $j - 1.5(3-j)$. The gap between the two roundings is $3 \times 0.5 = 1.5$ in every row, as step 7 of the proof predicts.

Table 34.3.: Three-fold randomized response with $\varepsilon_0 = \ln 3$: each outcome's total loss, exact and rounded on the grid $h = 0.5$.

j	Probability	Exact total loss	Rounded up	Rounded down
0	0.0156	-3.2958	-3	-4.5
1	0.1406	-1.0986	-0.5	-2
2	0.4219	1.0986	2	0.5
3	0.4219	3.2958	4.5	3

Feeding each column into Equation 34.1 gives the three δ curves. At $\varepsilon = 0$ only the two positive exact totals count, $3\ln 3$ for $j = 3$ and $\ln 3$ for $j = 2$. Both have probability $\frac{27}{64}$, since $(\frac{3}{4})^3 = 3(\frac{3}{4})^2\frac{1}{4}$, and their weights are $1 - \frac{1}{27}$ and $1 - \frac{1}{3}$, so

$$\delta(0) = \tfrac{27}{64} \cdot \tfrac{26}{27} + \tfrac{27}{64} \cdot \tfrac{2}{3} = \tfrac{44}{64} = 0.6875. \tag{34.14}$$

Table 34.4 and Figure 34.2 show that the pessimistic and optimistic values bracket the exact ones at every ε.

Table 34.4.: $\delta(\varepsilon)$ of three-fold randomized response, exact and from the rounded PLDs on the grid $h = 0.5$.

ε	Exact δ	Pessimistic	Optimistic
0	0.6875	0.7820	0.5669
1	0.4190	0.6758	0.3648
2	0.3064	0.3872	0.2667

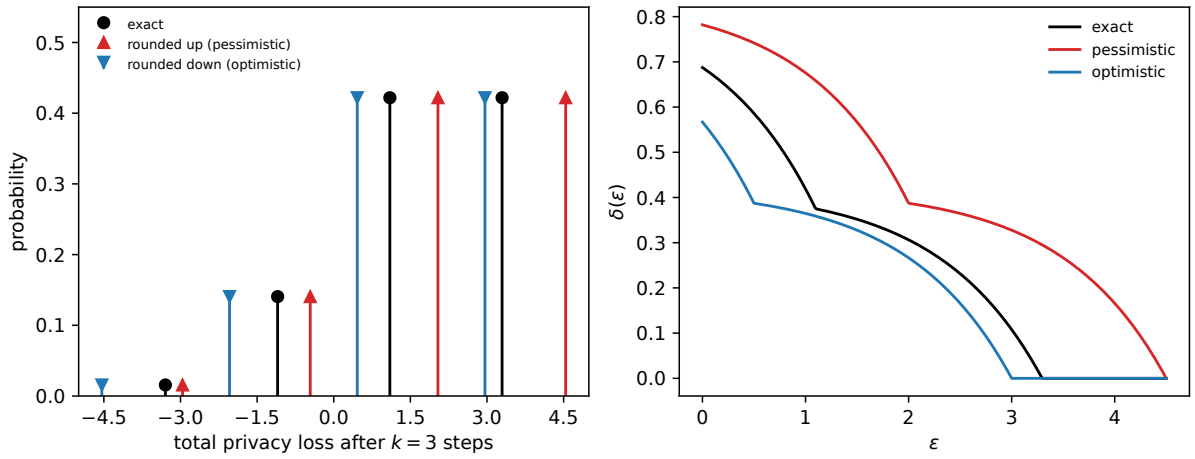


Figure 34.2.: Left: the PLD of three-fold randomized response with $\varepsilon_0 = \ln 3$ (black), rounded up (red) and down (blue) on the grid $h = 0.5$, offset sideways so that they do not overlap. Right: the three $\delta(\varepsilon)$ curves; the pessimistic one lies above the exact one and the optimistic one below it.

34.4. Composition by FFT

A discretized PLD is a vector of masses $m_0, \dots, m_{n-1}$ at the losses $(o + i)h$, where the integer offset o says where the vector starts on the grid, plus an atom at $+\infty$. Convolving two such vectors directly costs a number of operations proportional to the product of their lengths. On the grid $h = 10^{-5}$, one DP-SGD step of the running example already has 329590 bins, and after 10,000 steps the composed PLD has 2103251. The fast Fourier transform (FFT), brought to PLD accounting by Koskela, Jälkö and Honkela (2020), makes this cheap.

The proof of Proposition 34.2 below needs one tool, which turns a convolution into a pointwise product. The discrete Fourier transform attaches to index i the power ζ^i of a fixed complex number ζ, and powers multiply by adding exponents, so indices add the way losses add.

Lemma 34.2 (The discrete convolution theorem)**.** *For a length N, let $\zeta = e^{-2\pi \mathrm{i}/N}$, and let the discrete Fourier transform of $a_0, \dots, a_{N-1}$ be $\hat{a}_m = \sum_{i=0}^{N-1} a_i \zeta^{im}$. Then $\hat{a}_m \hat{b}_m$ is the transform of the **circular convolution***

$$c_n = \sum_{i+j \equiv n \bmod N} a_i b_j. \tag{34.15}$$

Proof.

1. *Multiply the transforms.* Expanding the product $\hat{a}_m \hat{b}_m$ gives one term per pair (i, j), and the powers combine as $\zeta^{im}\zeta^{jm} = \zeta^{(i+j)m}$.
2. *Group by the index sum.* Since $\zeta^N = 1$, exponents matter only modulo N, so grouping the pairs by $i + j$ modulo N gives

$$\hat{a}_m \, \hat{b}_m = \sum_{n=0}^{N-1} \Big(\sum_{i+j \equiv n \bmod N} a_i b_j \Big) \zeta^{nm}, \tag{34.16}$$

which is the transform of c.

□

The FFT computes a transform of length N in $O(N \log N)$ operations instead of $O(N^2)$. Two discretized PLDs on a common grid then compose as follows.

Proposition 34.2 (Convolution of discretized PLDs)**.** *Let a and b be the mass vectors of two discretized PLDs on the same grid, with lengths n_a and n_b, offsets o_a and o_b, and atoms a_∞ and b_∞ at $+\infty$. Their convolution has offset $o_a + o_b$, atom $a_\infty + b_\infty - a_\infty b_\infty$, and masses*

$$c = \mathrm{IDFT}_N(\mathrm{DFT}_N(a) \cdot \mathrm{DFT}_N(b)), \qquad N \geq n_a + n_b - 1, \tag{34.17}$$

where DFT_N transforms the vectors padded with zeros to length N and IDFT_N is its inverse. For k copies of one vector m of length n, with $N \geq k(n-1)+1$,

$$\omega^{*k} = \mathrm{IDFT}_N(\mathrm{DFT}_N(m)^k). \tag{34.18}$$

If N is smaller, the result is the PLD of the total loss reduced modulo Nh into the window of N grid points.

Proof. The plan is to turn adding losses into adding indices, handle the atom, and then apply Lemma 34.2.

1. *Grid losses add as indices.* A loss $(o_a + i)h$ plus a loss $(o_b + j)h$ is $(o_a + o_b + i + j)h$. So the mass at index n of the sum collects every pair with $i + j = n$, which is the discrete convolution $c_n = \sum_{i+j=n} a_i b_j$.
2. *The atom.* The sum is finite only when both terms are, which has probability $(1-a_\infty)(1-b_\infty)$. So it is $+\infty$ with probability $a_\infty + b_\infty - a_\infty b_\infty$.
3. *Padding.* By Lemma 34.2 the product of transforms gives the circular convolution. With $N \geq n_a + n_b - 1$, every pair of nonzero entries has $i + j \leq n_a + n_b - 2 < N$, so no index sum wraps and the circular convolution is c.
4. *Powers.* Each further convolution multiplies the transform by one more copy, which gives $\mathrm{DFT}(m)^k$. The composed vector's largest index is $k(n-1)$, so it has $k(n-1)+1$ entries, and that is the padding needed.
5. *Wraparound.* With N too small, Lemma 34.2 still holds but indices add modulo N. Mass at total index $n \geq N$ lands at $n - N$, so the total loss is reduced modulo Nh.

□

The one-shot k-th power needs a window k times wider than one step. The script instead uses **repeated squaring**, which lets it trim the vector between convolutions. It forms $\omega^{*2}, \omega^{*4}, \omega^{*8}, \ldots$ by convolving each with itself and multiplies the powers in the binary expansion of k, so that $\omega^{*10} = \omega^{*8} * \omega^{*2}$, at most $2 \log_2 k$ convolutions in all. After each one it truncates both tails at mass $\tau = 10^{-15}$ in the safe directions of Definition 34.3. Since all but τ of the mass lies within a fixed number of standard deviations of the mean, the vector then grows like the composed PLD's standard deviation, $\sqrt{k}$ times one step's, and not like the worst-case range $k(n-1)$.

The FFT is not exact in floating point, since each mass has an absolute error of order 10^{-16} times the largest mass. Tiny tail masses therefore come out as noise, sometimes negative, which the script sets to zero. Summed over millions of bins, that noise alone can exceed τ, so a truncation

that may cut only a tail of mass at most τ never trims the ends, and the script also cuts every loss below -100 or above 200 in the safe direction. That noise is far below a δ of 10^{-5} but is not part of the certified bound, and the privacy loss random variable (PRV) accountant of Gopi, Lee and Wutschitz (2021) also leaves it out, as negligible for δ above 10^{-10} (their Section 6).

To choose the grid, pick h from the width kh you can accept. Pick the range of losses to keep from the tails, which Chebyshev's inequality (Theorem 2.2) bounds from the composed variance alone. Two published refinements allow coarser grids: the PRV accountant rounds to the nearest grid point and recentres the mean, so its errors grow like $\sqrt{k}\,h$ (Gopi et al., Lemma 5.3), and connect the dots (Doroshenko et al. 2022) finds the tightest pessimistic PLD on a given grid.

Example 34.4. On the grid $h = 10^{-3}$, truncated at $\tau = 10^{-15}$ per tail, the Gaussian PLD $\mathcal{N}(0.5, 1)$ has 15884 bins. After ten steps by repeated squaring it has 50257, a ratio of $3.16 = \sqrt{10}$, while the one-shot power needs a window of $10 \times (15884 - 1) + 1 = 158831$. Both give the same pessimistic $\delta(1) = 0.81891018$, against the exact 0.81851782 from Equation 34.4 with $\mu = \sqrt{10}$. Table 34.5 turns this into ε at $\delta = 10^{-5}$ and compares it with Example 34.2.

Table 34.5.: Composed Gaussian steps with $\mu = 1$ on the grid $h = 10^{-3}$: ε at $\delta = 10^{-5}$.

k	Exact ε	Pessimistic	Optimistic	Mean of the pessimistic PLD
10	17.8566	17.8616	17.8516	5.0050 (exact 5)
100	91.8173	91.8673	91.7673	50.0500 (exact 50)

The width is kh, and the exact value sits in the middle, $17.8616 - 17.8566 = 0.005 = kh/2$, because each rounding moves a step's loss by $h/2$ on average. The last column shows that shift in the pessimistic means.

Without padding, wraparound appears. On an unpadded circular window from loss 0 to 10, centred at the mean, the tenth power gives $\delta(1) = 0.83478562$, wrong in the second decimal. Losses below 0 wrap to the top of the window and losses above 10 to the bottom. Width 20 gives 0.81890091, and width 40 gives 0.81891018, the padded value.

34.5. The subsampled Gaussian

One DP-SGD step with Poisson sampling includes each record independently with probability q, sums the clipped gradients of the sampled records, and adds Gaussian noise. Look at it from the one record that may be missing, and measure the noisy sum along that record's gradient, in units of the clipping norm, the bound on each gradient's length. Without the record, this sum is a Gaussian centred at 0. With it, the record is sampled with probability q and then shifts the centre by the sensitivity 1, so the output is a mixture. Amplification by subsampling derives this reduction, and this section gives its PLD in closed form, so that only rounding separates the grid from the truth.

Under add/remove, each direction gives its own pair. Write $D' = D \cup \{x\}$ for the dataset that holds the record, as in the rest of the part. The **remove** direction puts D' first, $P = M(D')$ and

$Q = M(D)$, because its neighbour D is obtained by removing x. The **add** direction is the order $P = M(D)$, $Q = M(D')$.

Theorem 34.5 (Dominating pairs of the subsampled Gaussian)**.** *Let one step of DP-SGD use Poisson sampling at rate q, noise multiplier σ and sensitivity 1, under add/remove neighbours. The pair*

$$P = (1-q)\,\mathcal{N}(0,\sigma^2) + q\,\mathcal{N}(1,\sigma^2), \qquad Q = \mathcal{N}(0,\sigma^2) \tag{34.19}$$

dominates the remove direction, and the same pair with P and Q swapped dominates the add direction.

Zhu, Dong & Wang (2022, Theorem 11 and the paragraph after it) prove each pair dominates its case, and Section L.2 in the appendices derives it step by step from two exact hockey-stick identities. Their mixture component sits at 0 and their base Gaussian at 1, and the reflection $y \mapsto 1-y$ turns that into the form above. By Theorem 34.3, a T-step run has the guarantee $\max\{\delta_{\mathrm{rem},T}(\varepsilon), \delta_{\mathrm{add},T}(\varepsilon)\}$. Neither pair dominates the other in general, so an accountant that uses only the remove pair covers only half of the add/remove relation.

The ratio of the mixture density to the base density $\mathcal{N}(0,\sigma^2)$ is $1-q$ plus q times the ratio of the shifted Gaussian to the base, an exponential in y. Swapping P and Q flips the sign of the log of that ratio.

Proposition 34.3 (The PLD of the subsampled Gaussian)**.** *For the pairs of Theorem 34.5, with $Y \sim P$ of the respective pair, the privacy losses are*

$$L_{\mathrm{rem}}(y) = \ln\Big((1-q) + q\,e^{(2y-1)/(2\sigma^2)}\Big), \qquad L_{\mathrm{add}}(y) = -L_{\mathrm{rem}}(y). \tag{34.20}$$

The remove loss is increasing, takes values in $(\ln(1-q), \infty)$, and the output with loss ℓ is

$$y(\ell) = \sigma^2 \ln\frac{e^\ell - 1 + q}{q} + \frac{1}{2}, \tag{34.21}$$

with $y(\ell) = -\infty$ for $\ell \le \ln(1-q)$. A bin of remove losses is therefore an interval of outputs, with mass

$$\Pr_P\left[L_{\mathrm{rem}} \in (a,b]\right] = (1-q)\Big[\Phi\Big(\tfrac{y(b)}{\sigma}\Big) - \Phi\Big(\tfrac{y(a)}{\sigma}\Big)\Big] + q\Big[\Phi\Big(\tfrac{y(b)-1}{\sigma}\Big) - \Phi\Big(\tfrac{y(a)-1}{\sigma}\Big)\Big], \tag{34.22}$$

and for the add pair $\Pr_P[L_{\mathrm{add}} \in (a,b]] = \Phi(y(-a)/\sigma) - \Phi(y(-b)/\sigma)$. One step's $\delta(\varepsilon)$ is a difference of two normal tails at the output $y(\varepsilon)$.

Proof. The plan is to divide the mixture by the base, read off the loss and its inverse, and turn loss bins into output intervals; step 6 then gives the single-step δ.

1. *Divide the mixture by the base.* Write φ for the standard normal density, so that $\mathcal{N}(c,\sigma^2)$ has density proportional to $\varphi((y-c)/\sigma)$, with the same constant for both centres. Dividing each part of the mixture by the base gives

$$\frac{P(y)}{Q(y)} = (1-q) + q\,\frac{\varphi((y-1)/\sigma)}{\varphi(y/\sigma)}. \tag{34.23}$$

2. *The ratio of two Gaussians.* The densities are exponentials of minus a square, so their ratio is the exponential of the difference of the squares:
$$\frac{\varphi((y-1)/\sigma)}{\varphi(y/\sigma)} = \exp\left(\frac{y^2-(y-1)^2}{2\sigma^2}\right) = e^{(2y-1)/(2\sigma^2)}. \tag{34.24}$$
Taking logs of step 1 gives L_{rem}, and swapping P and Q flips the sign, which gives L_{add}.
3. *The loss is increasing.* The exponential is positive and increasing in y. As $y \to -\infty$ it vanishes and the loss tends to $\ln(1-q)$, and as $y \to \infty$ the loss tends to ∞.
4. *Invert it.* To solve $e^{\ell} = (1-q) + q\,e^{(2y-1)/(2\sigma^2)}$ for y, subtract $1-q$, divide by q, take logs, multiply by σ^2 and add $\frac{1}{2}$.
5. *Bins are intervals.* Because the map is increasing, the bin $\{L_{\text{rem}} \in (a,b]\}$ is the interval $\{Y \in (y(a), y(b)]\}$. Its probability under the mixture is $1-q$ times that under $\mathcal{N}(0,\sigma^2)$ plus q times that under $\mathcal{N}(1,\sigma^2)$. For the add pair, $L_{\text{add}} \in (a,b]$ exactly when $Y \in [y(-b), y(-a))$, with $Y \sim \mathcal{N}(0,\sigma^2)$.
6. *Single-step δ in closed form.* For the remove pair, $L > \varepsilon$ exactly when $Y > y(\varepsilon)$, so by Equation 34.1
$$\delta(\varepsilon) = \Pr_P[Y > y(\varepsilon)] - e^{\varepsilon}\,\Pr_Q[Y > y(\varepsilon)], \tag{34.25}$$
two normal tail probabilities. The add pair is the same with the event $Y < y(-\varepsilon)$.

□

The two directions have very different losses (Figure 34.3). The remove loss is bounded below by $\ln(1-q)$ and has an unbounded upper tail, because a rare large output betrays that the record was used. The add loss is its mirror image, bounded above by $-\ln(1-q)$, so a single add step has $\delta(\varepsilon) = 0$ once $\varepsilon \geq -\ln(1-q)$.

In the running example the remove direction gives the larger ε at every T of Table 34.7. That is an observation, not a theorem, so the accountant computes both and takes the larger.

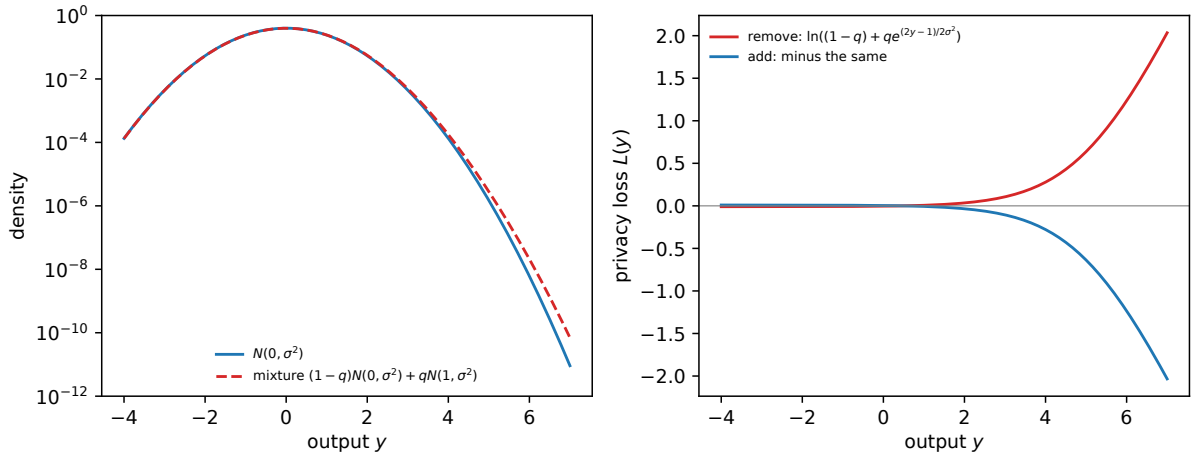


Figure 34.3.: Left: the densities of $\mathcal{N}(0,\sigma^2)$ and of the mixture $(1-q)\mathcal{N}(0,\sigma^2) + q\mathcal{N}(1,\sigma^2)$ for $q = 0.01$, $\sigma = 1$, on a log scale. They differ only in the far right tail. Right: the privacy loss as a function of the output for the remove pair (red) and the add pair (blue). Both are nearly flat near zero and steep in the right tail.

Example 34.5. For $q = 0.01$ and $\sigma = 1$, the lowest possible remove loss is $\ln(0.99) = -0.0101$. Except for 10^{-15} of mass, the remove loss is in $[-0.0101, 3.2858]$ and the add loss in $[-2.8926, 0.0101]$, which on the grid $h = 10^{-5}$ is 329590 and 290268 bins. The rounded-up remove PLD has $\mathbb{E}[e^{-L}] = 0.999995$, below 1 and so valid, as step 5 of the proof of Theorem 34.4 requires. Table 34.6 compares the pessimistic PLD's δ with the closed form of step 6 above.

Table 34.6.: One subsampled-Gaussian step ($q = 0.01$, $\sigma = 1$): $\delta(\varepsilon)$ exactly and from the pessimistic PLD, $h = 10^{-5}$.

ε	Remove: exact δ	Remove: pessimistic	Add: exact δ	Add: pessimistic
0	3.829249e-03	3.830792e-03	3.829249e-03	3.832688e-03
0.5	2.214519e-07	2.214633e-07	0	0
1	2.732009e-09	2.732117e-09	0	0

At $\varepsilon = 0$ both directions have the same δ, the total variation distance $\sum_y (P(y) - Q(y))_+$. It is unchanged by swapping the pair, because both distributions sum to 1, so P exceeds Q in total by as much as Q exceeds P. At $\delta = 10^{-5}$, bisection on the closed form gives one step $\varepsilon = 0.1995$ in the remove direction and 0.0091 in the add direction.

34.6. Accounting for DP-SGD

To answer the opening question, the accountant discretizes each direction's PLD up and down (Section 34.5), composes it T times by FFT (Section 34.4), and reads off ε, which can be done exactly because between two grid points $\delta(\varepsilon)$ has a simple form.

Proposition 34.4 (Reading ε off a discretized PLD)**.** *Let a discretized PLD have masses m_ℓ at the grid losses ℓ and an atom m_∞ at $+\infty$, and let ε lie between two consecutive grid points. Write A for the total mass of the grid points above ε, and B for the sum of those masses times $e^{-\ell}$. Then*

$$\delta(\varepsilon) = m_\infty + A - e^{\varepsilon} B, \tag{34.26}$$

so on that interval the equation $\delta(\varepsilon) = \delta$ has the root $\varepsilon = \ln\big((m_\infty + A - \delta)/B\big)$.

Proof.

1. *The weights.* Between two grid points the set of grid losses above ε does not change. So the weight of Equation 34.1 is $1 - e^{\varepsilon - \ell}$ on those losses, 1 on the atom, and 0 elsewhere.
2. *Sum them.* Summing the weights against the masses gives

$$\delta(\varepsilon) = m_\infty + \sum_{\ell > \varepsilon} m_\ell - e^{\varepsilon} \sum_{\ell > \varepsilon} m_\ell \, e^{-\ell}, \tag{34.27}$$

which is the stated form. It is linear in e^{ε}, so setting it equal to δ and solving for e^{ε} gives the root.

□

Two accountants from earlier chapters serve as comparisons. The RDP accountant of Amplification by subsampling tracks, for each order $\alpha > 1$, the per-step Rényi divergence $\frac{1}{\alpha-1}\ln A_\alpha$ of the remove pair. Here A_α is the likelihood ratio of Proposition 34.3 raised to the power α and averaged under the base,

$$A_\alpha = \mathbb{E}_{y\sim\mathcal{N}(0,\sigma^2)}\Big[\big((1-q)+q\,e^{(2y-1)/(2\sigma^2)}\big)^\alpha\Big]. \tag{34.28}$$

The script multiplies the divergence by T, converts it to (ε,δ) with the improved conversion of From Rényi DP to (ε, δ) and to tests, and takes the best order on a grid of orders. For integer orders the binomial theorem turns A_α into a finite sum, which agrees with numerical integration on $\ln A_2 = 0.00017181$ and $\ln A_{10} = 0.34443377$.

The GDP central limit theorem of f-DP and Gaussian DP replaces the T steps by one Gaussian pair. Heuristically, the average squared deviation of the likelihood ratio from 1 is $e^{1/\sigma^2}-1$ for one Gaussian step, subsampling scales it by q^2, and μ^2 adds over steps, which gives the shift of Theorem 32.5

$$\mu = q\sqrt{T(e^{1/\sigma^2}-1)}, \tag{34.29}$$

and ε then comes from Equation 34.4. At $T = 10{,}000$ this gives $\mu = 0.01\times100\times1.3108 = 1.3108$.

Example 34.6. Table 34.7 gives ε at $\delta = 10^{-5}$ from the pessimistic PLD, an upper bound valid for adaptive composition, and from the optimistic PLD, a lower bound for the non-adaptive run, each the larger over the two directions. At $T = 1000$ the true ε lies between 1.8232 and 1.8332, and at $T = 10{,}000$ the bracket is $[6.1377, 6.2377]$, wider because its width is Th (Figure 34.4).

Table 34.7.: DP-SGD with $q = 0.01$, $\sigma = 1$: ε at $\delta = 10^{-5}$ after T steps, from the PLDs on the grid $h = 10^{-5}$ (the larger direction), the add direction alone, RDP, and the GDP central-limit approximation.

T	PLD upper	PLD lower	Width	Add direction, upper	RDP	GDP CLT
1	0.1995	0.1994	1.0e-05	0.0091	0.9555	0.0368
10	0.3800	0.3799	1.0e-04	0.0655	1.0353	0.1304
100	0.7185	0.7175	1.0e-03	0.3387	1.2141	0.4575
1000	1.8332	1.8232	1.0e-02	1.4217	2.1014	1.6177
10000	6.2377	6.1377	1.0e-01	5.6615	6.7127	6.0071

Table 34.8 repeats $T = 1000$ on three grids. The width equals Th to four decimals, as Theorem 34.4 predicts, and on the coarsest grid the upper bound 2.3279 is worse than RDP's 2.1014. Each tenfold refinement of h makes the vectors about ten times longer, up to 767721 bins after 1000 steps at $h = 10^{-5}$.

Table 34.8.: DP-SGD after $T = 1000$ steps: the PLD bounds on ε at $\delta = 10^{-5}$ for three grid widths.

h	Upper	Lower	Width	Th
10^{-3}	2.3279	1.3279	1.0000	1.0000

h	Upper	Lower	Width	Th
10^{-4}	1.8782	1.7782	0.1000	0.1000
10^{-5}	1.8332	1.8232	0.0100	0.0100

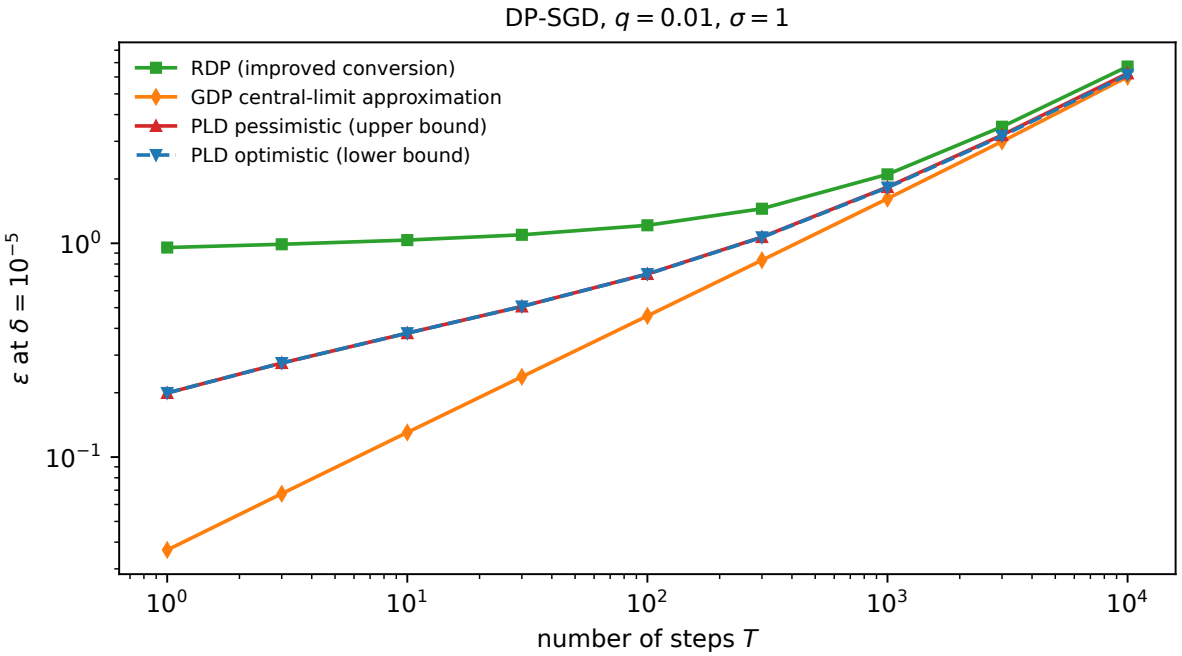


Figure 34.4.: ε at $\delta = 10^{-5}$ against the number of steps T for DP-SGD with $q = 0.01$, $\sigma = 1$: RDP (green), the CLT (orange), and the pessimistic (red) and optimistic (blue) PLD bounds on the grid $h = 10^{-5}$, which overlap at this scale.

RDP is a valid bound, but loose. It exceeds the PLD upper bound by $0.9555 - 0.1995 = 0.7560$ at $T = 1$ and by $6.7127 - 6.2377 = 0.4750$ at $T = 10{,}000$, because its conversion pays about $\ln(1/\delta)/(\alpha - 1)$ whatever the curve, the price of turning one moment of the loss into a tail bound.

The CLT is not a bound, and here it is too optimistic. It falls below the PLD *lower* bound at every T, by $1.8232 - 1.6177 = 0.2055$ at $T = 1000$ and by $6.1377 - 6.0071 = 0.1306$ at $T = 10{,}000$. The subsampled PLD has a heavy right tail (Figure 34.5), and at $\delta = 10^{-5}$ you read exactly that tail, where the Gaussian approximation is worst. So report the pessimistic ε with the optimistic one as its error bar, and pick h for the largest T you will run, since a bracket of 0.01 at $T = 10{,}000$ needs $Th \le 0.01$, that is $h \le 10^{-6}$.

34.7. Code

The script code/numerical_accounting.py implements the accountant in NumPy, with the RDP and CLT comparisons, and checks each piece against something exact, such as a closed form or the one-shot power. From the repository root:

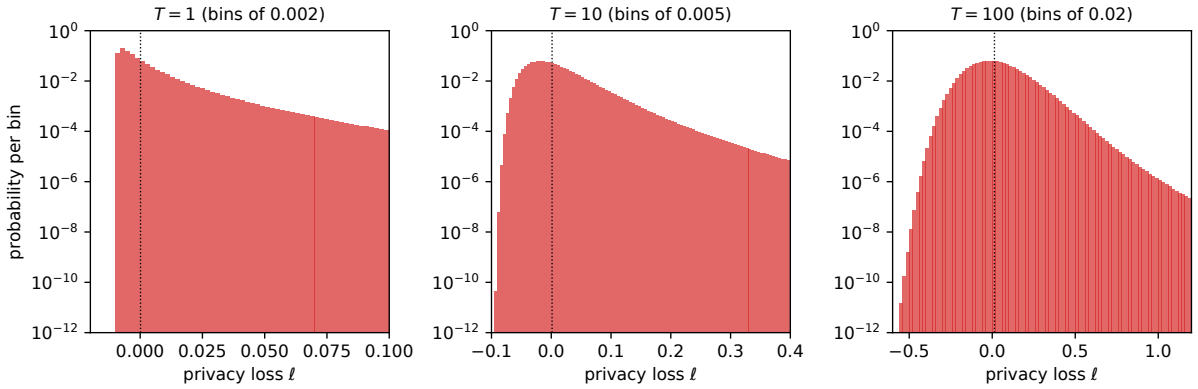


Figure 34.5.: The pessimistic remove-direction PLD of DP-SGD with $q = 0.01$, $\sigma = 1$ after 1, 10 and 100 steps (grid $h = 10^{-4}$, re-binned for display; log scale). The dotted line is the mean. One step piles its mass just above $\ln(1 - q) = -0.0101$; convolution moves the bulk towards a bell shape, but the right tail stays heavier than the left.

```
uv run python code/part04-privacy-accounting/numerical_accounting.py
```

It takes from half a minute to a few minutes, nearly all of it the $T = 10{,}000$ compositions of the two directions, each rounded up and down.

Table 34.9.: The script's output with the default seed.

Experiment	Result
Gaussian PLD, $\mu = 1$	Mean 0.4990, variance 1.0013, $\mathbb{E}[e^{-L}]$ 1.0015; Table 34.2; $\varepsilon = 4.3772$ at $\delta = 10^{-5}$
Finite pair $(0.1, 0.2, 0.3, 0.4)$ against its reverse, $\varepsilon = 0.5$	Both forms of Equation 34.1 give 0.2351
Randomized response, three steps, $h = 0.5$	Table 34.3 and Table 34.4; $\mathbb{E}[e^{-L}]$ 0.8469 (up), 1.3963 (down)
Gaussian steps on $h = 10^{-3}$	Table 34.5; squaring and one-shot power agree, $\delta(1) = 0.81891018$
Wraparound, $k = 10$	Window 10: 0.83478562; window 20: 0.81890091; window 40: 0.81891018
One subsampled step	Table 34.6; exact ε 0.1995 (remove) and 0.0091 (add)
RDP check	$\ln A_2 = 0.00017181$ and $\ln A_{10} = 0.34443377$ by both methods
DP-SGD	Table 34.7 and Table 34.8

tests/test_numerical_accounting.py checks that the output still matches.

34.8. Summary

Table 34.10.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 34.1	ω is the law of $L(Y)$, $Y \sim P$, atom at $+\infty$ included	What a numerical accountant tracks
Theorem 34.1	$\delta(\varepsilon) = \mathbb{E}_\omega[(1 - e^{\varepsilon - L})_+]$; law under Q is $e^{-\ell}\omega(\mathrm{d}\ell)$	(ε, δ) from any PLD
Proposition 34.1	A PLD if and only if $\mathbb{E}_\omega[e^{-L}] \le 1$	The pessimistic PLD is a real pair's
Equation 34.4	Gaussian PLD $\mathcal{N}(\mu^2/2, \mu^2)$ and its closed-form δ	Checking the code; Gaussian DP
Theorem 34.2	Independent steps: $\omega_1 * \cdots * \omega_k$	Exact composition
Theorem 34.3	Products of dominating pairs dominate the adaptive run	DP-SGD
Lemma 34.1	Pointwise $X \le X'$ gives $\mathbb{E}[g(X)] \le \mathbb{E}[g(X')]$ for non-decreasing g	The sign of rounding errors
Theorem 34.4	$\delta^\downarrow \le \delta \le \delta^\uparrow$ for every k; width at most kh	Report the pessimistic ε, the optimistic one as error bar; choose h from kh
Lemma 34.2	Product of transforms is circular convolution	Convolution in $O(N \log N)$
Proposition 34.2	Pad to $n_a + n_b - 1$, or losses wrap modulo Nh	Never let an FFT wrap silently
Theorem 34.5	Remove: mixture against $\mathcal{N}(0, \sigma^2)$; add: swapped	Compute both directions, take the larger
Proposition 34.3	$L = \pm \ln\left((1-q) + q\, e^{(2y-1)/(2\sigma^2)}\right)$	Exact bin masses for DP-SGD
Proposition 34.4	$\delta(\varepsilon) = m_\infty + A - e^\varepsilon B$ between grid points	Exact ε; RDP over-reports, the CLT can under-report

34.9. References

- Sommer, Meiser & Mohammadi, "Privacy Loss Classes: The Central Limit Theorem in Differential Privacy" (Proceedings on Privacy Enhancing Technologies, PoPETs 2019). Defines the privacy loss distribution, proves that composition convolves PLDs, and shows that composed PLDs tend to Gaussians. Read it for the PLD formalism of Section 34.1 and Section 34.2.

- Meiser & Mohammadi, "Tight on Budget? Tight Bounds for r-Fold Approximate Differential Privacy" (CCS 2018). The privacy buckets: discretize the loss into buckets, round up for a certified upper bound and down for a lower bound. The origin of Section 34.3.
- Koskela, Jälkö & Honkela, "Computing Tight Differential Privacy Guarantees Using FFT" (AISTATS 2020). FFT-based PLD accounting on a fixed window, with error analysis for the truncation and wraparound. Read it for Section 34.4.
- Koskela, Jälkö, Prediger & Honkela, "Tight Differential Privacy for Discrete-Valued Mechanisms and for the Subsampled Gaussian Mechanism Using FFT" (AISTATS 2021). Rigorous upper and lower bounds with the FFT accountant, applied to the subsampled Gaussian; the source for the setting of Section 34.5 and Section 34.6.
- Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021). The PRV accountant: nearest-point discretization with a mean-matching shift, explicit error bounds, and a running time of order $\sqrt{k}$ (up to logarithmic factors) for k identical steps. Read it when you need certified error bars on a fast accountant.
- Doroshenko, Ghazi, Kamath, Kumar & Manurangsi, "Connect the Dots: Tighter Discrete Approximations of Privacy Loss Distributions" (PoPETs 2022). Frames discretization with dominating pairs, proves rounding up gives a valid PLD (Lemma 4.2) and builds the optimal pessimistic PLD on a grid. Read it for the cleanest account of why the pessimistic bound is valid.
- Zhu, Dong & Wang, "Optimal Accounting of Differential Privacy via Characteristic Function" (AISTATS 2022). Dominating pairs (Definition 7), their adaptive composition (Theorem 10), the two directions of Poisson subsampling (Theorem 11), and accounting through the characteristic function of the privacy loss.
- Google, `dp_accounting` library documentation (the `pld` module). A production PLD accountant with pessimistic and optimistic estimates and connect-the-dots discretization; use it to check the numbers here.

35. Amplification by shuffling and by iteration

Amplification by subsampling showed that uncertainty is worth privacy: a step that uses your record only with probability q leaks roughly q times as much. This chapter asks what two other kinds of uncertainty are worth. The first is uncertainty about **which report is yours**. In the shuffle model of Trust models, each of n users randomizes their own data with a local randomizer that satisfies differential privacy (DP) at level ε_0, and a shuffler mixes the reports before the analyst sees them. The second is uncertainty about **when your record was used**. Noisy gradient descent (GD) on a convex loss forgets, so when only the final model is released, a record used early is better hidden than composition says.

One idea organizes every result. You show that what the adversary sees can be generated from a much simpler pair of distributions, by a procedure that does not depend on your record. By the data-processing inequality, the simple pair's privacy then holds for the real output. For shuffling the simple pair is a pair of counts, and for iteration it is a pair of Gaussians, or a pair that a chain of noisy steps keeps close.

The numbers come from two running examples. A company asks 10,000 users a yes-or-no question through randomized response at $\varepsilon_0 = 2$, a weak local guarantee, yet behind a shuffler the reports are $(0.0870, 10^{-6})$-DP. A model is trained for 10,000 steps of full-batch noisy GD, with noise ten times as large as the change one record makes to a step. Releasing every iterate gives no guarantee at all, while releasing only the final model of a contracting run gives $\varepsilon = 6.5534$. Throughout, neighbouring datasets $D \sim D'$ are replace-one: they differ in the value of one record, yours. The shuffling here is of anonymous reports, not the shuffle-and-partition batching of Amplification by subsampling, for which no comparable theorem holds.

35.1. Shuffled randomized response, exactly

Before bounding shuffling for every randomizer, it pays to compute it exactly for one. For binary randomized response the privacy of the shuffled reports can be computed in milliseconds, and that value is the yardstick for every general bound in Section 35.2. Randomized response, from the definition of differential privacy, reports a bit truthfully with probability $p = e^{\varepsilon_0}/(1 + e^{\varepsilon_0})$ and flipped otherwise (that chapter writes t for p). One report is ε_0-DP, because changing the bit makes any report likelier by a factor of at most $p/(1-p) = e^{\varepsilon_0}$.

Definition 35.1 (Shuffled randomized response)**.** Each of n users holds a bit and sends its randomized-response report to a **shuffler**, which outputs the n reports in a uniformly random order. The analyst sees only this shuffled sequence, whose order carries no information, so the page calls it the **bag** of reports.

Earlier chapters supply the two tools. The first computes δ: for two distributions P and Q on a finite set, the smallest δ at a given ε is the hockey-stick divergence $E_{e^\varepsilon}(P\|Q)$, which by Lemma 20.2 sums $P(y) - e^\varepsilon Q(y)$ over the outcomes y whose privacy loss $\ln(P(y)/Q(y))$ exceeds ε. The exact $\delta(\varepsilon)$ of a pair is the larger of the two directions, $E_{e^\varepsilon}(P\|Q)$ and $E_{e^\varepsilon}(Q\|P)$. The second, the data-processing inequality of Theorem 3.5, says that a map that does not depend on the data cannot move two distributions further apart.

Once the order is gone, only the number of 1-reports can tell the analyst anything, and your report moves that number by at most one.

Theorem 35.1 (The count is the whole view)**.** *In shuffled randomized response, let the other $n-1$ users all hold 0, and let your bit be 0 in D and 1 in D'. Write b for the probability mass function of $\mathrm{Bin}(n-1, 1-p)$. Then the bag has the same divergences as the number Y of 1-reports, whose distributions on D and D' are*

$$P(y) = p\,b(y) + (1-p)\,b(y-1), \qquad Q(y) = (1-p)\,b(y) + p\,b(y-1), \qquad y = 0, \dots, n. \quad (35.1)$$

Proof. The plan is to show that the count carries everything, then to split it into your report and the others', and to condition on yours.

1. *The bag is the count.* Given the count Y, every arrangement of Y ones and $n-Y$ zeros is equally likely under both datasets, because the shuffle is uniform. So the bag can be rebuilt from the count with randomness that does not depend on the data, and the data-processing inequality, applied in both directions, gives the two the same divergences.
2. *Split the count.* Write $Y = S + B$, where S counts the 1-reports among the others and B is your report. Each other user reports 1 with probability $1-p$, independently, so $S \sim \mathrm{Bin}(n-1, 1-p)$ on both datasets.
3. *Your report.* You hold 0 in D, so $\Pr[B=0] = p$, and you hold 1 in D', so $\Pr[B=0] = 1-p$.
4. *Condition on your report.* The count is y either when $B=0$ and $S=y$, or when $B=1$ and $S=y-1$, so by the law of total probability

$$\Pr[Y=y] = \Pr[B=0]\,b(y) + \Pr[B=1]\,b(y-1), \quad (35.2)$$

and the two values of $\Pr[B=0]$ from step 3 give Equation 35.1.

□

The pair assumes that the other users all hold 0. If m of them hold 1, a guarantee must hold for the worst m. The chapter's script (Section 35.6) scans every m at $n = 1000$, and $m \le 30$ at $n = 10^4$ and 10^5. The pair $m = 0$ is the worst or within 10^{-4} of the worst found, so the chapter calls its ε exact.

Shuffling never lowers the pure-DP level. At the count $y = 0$ the ratio $P(0)/Q(0)$ is e^{ε_0} for every n, and only the probability of that outcome shrinks. So shuffling amplifies by moving the bad outcomes into δ.

How does the privacy improve with the number of users? The count is a sum of many independent bits, so it is close to normal. Two normals with the same variance are as hard to tell apart as two unit-variance normals a distance μ apart, where μ is their mean difference in standard deviations. Such a pair satisfies Gaussian differential privacy (GDP) of f-DP and Gaussian DP with parameter

μ, written μ-**GDP**, and its exact $\delta(\varepsilon)$ is the curve of Theorem 20.3. The next result computes the two moments that set μ.

Proposition 35.1 (Moments of the shuffled count)**.** *In the setting of Theorem 35.1, the count Y has variance $np(1-p)$ on both datasets, and its means differ by $2p-1$. So a pair of normals with these moments is μ-GDP with*

$$\mu = \frac{2p-1}{\sqrt{np(1-p)}} = \frac{2\sinh(\varepsilon_0/2)}{\sqrt{n}}. \tag{35.3}$$

Proof.

1. *The variance.* Every report, yours included, is a bit that is 1 with probability p or $1-p$, so each has variance $p(1-p)$. The n reports are independent, so their variances add to $np(1-p)$ on both datasets.
2. *The mean difference.* Only your report changes between the datasets, and its mean moves from $1-p$ to p, a difference of $2p-1$.
3. *Simplify.* In terms of ε_0, the numerator is $2p-1 = (e^{\varepsilon_0}-1)/(e^{\varepsilon_0}+1)$ and the square root is $\sqrt{p(1-p)} = e^{\varepsilon_0/2}/(e^{\varepsilon_0}+1)$. The factors $e^{\varepsilon_0}+1$ cancel in the ratio, and dividing top and bottom by $e^{\varepsilon_0/2}$ leaves $e^{\varepsilon_0/2} - e^{-\varepsilon_0/2}$, which is $2\sinh(\varepsilon_0/2)$.

□

So the privacy falls like $1/\sqrt{n}$, with a constant that grows like $e^{\varepsilon_0/2}$. The approximation is good when there are many 1-reports to hide among: at $n = 10^5$, $\varepsilon_0 = 2$ and $\delta = 10^{-6}$ it gives $\varepsilon = 0.0244$ against the exact 0.0247 (Table 35.2). At $n = 1000$ and $\varepsilon_0 = 4$, the others send on average only $999/(1+e^4) = 18.0$ 1-reports, a count far from normal, and it gives 0.9667 against the exact 1.6910. The exact computation costs milliseconds, so use it.

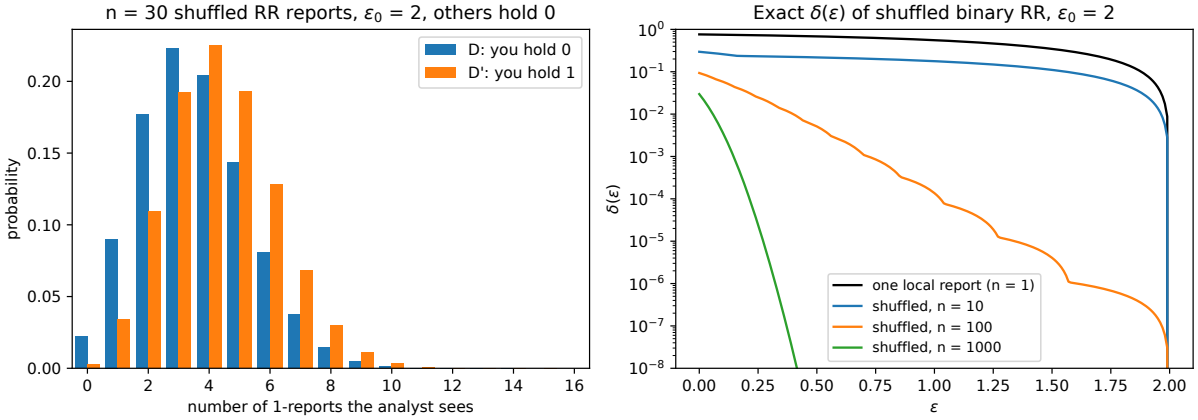


Figure 35.1.: Left: the number of 1-reports among $n = 30$ shuffled reports with $\varepsilon_0 = 2$, when you hold 0 (blue) and 1 (orange); the others hold 0. Right: the exact $\delta(\varepsilon)$ for one unshuffled report and for $n = 10, 100, 1000$ shuffled reports; all reach 0 only at ε_0, but for large n they are negligible far earlier.

Example 35.1. Start with the smallest case, $n = 3$ users at $\varepsilon_0 = \ln 3$, so $p = \frac{3}{4}$. The two other users hold 0, so their count has $b = (\frac{9}{16}, \frac{6}{16}, \frac{1}{16})$, and Table 35.1 fills in Equation 35.1. At $\varepsilon = 0.5$, where $e^{0.5} = 1.6487$, only $y = 0$ has a ratio P/Q above $e^{0.5}$, so only it enters the sum:

$$E_{e^{0.5}}(P\|Q) = \frac{27 - 9e^{0.5}}{64} = 0.1900. \tag{35.4}$$

In the other direction the outcomes $y = 2$ and $y = 3$ enter and give 0.0861, so $\delta(0.5) = 0.1900$. A single unshuffled report has $\delta(0.5) = \frac{3}{4} - \frac{1}{4}e^{0.5} = 0.3378$, so even two other users nearly halve δ.

Table 35.1.: Shuffled binary randomized response with $n = 3$ users and $\varepsilon_0 = \ln 3$; the two other users hold 0.

Count y	0	1	2	3
$b(y)$, others' 1-reports	9/16	6/16	1/16	0
$P(y)$, you hold 0	27/64	27/64	9/64	1/64
$Q(y)$, you hold 1	9/64	33/64	19/64	3/64
$P(y)/Q(y)$	3.0000	0.8182	0.4737	0.3333

Now the survey. Its 10,000 users answer at $\varepsilon_0 = 2$, so each report is the truth with probability $p = e^2/(1+e^2) = 0.8808$. Each report alone is only 2-DP, but the shuffled bag is $(0.0870, 10^{-6})$-DP, and the normal approximation Equation 35.3 gives 0.0844 (Table 35.2).

35.2. The clone bound for any local randomizer

Randomized response on bits is one randomizer. In practice users send histograms, k-ary responses or noisy vectors, so you need one bound for every ε_0-DP local randomizer R, a randomized map from a user's data to a report. Erlingsson et al. (2019) and Balle et al. (2019) gave early bounds. The one to use today is that of Feldman, McMillan & Talwar (2021), and its central reduction is short enough to prove here.

The reduction rests on clones. Lemma 26.4 shows that any ε_0-DP randomizer, run on another user's data, produces with probability $e^{-\varepsilon_0}$ a report distributed exactly like one of your two possible reports. After the shuffle, the adversary can only count the reports that look like your D-version and those that look like your D'-version, and the clones blur both counts. Write x^0 and x^1 for the values you hold in D and in D'.

Theorem 35.2 (The clone reduction)**.** *Let every user apply the same ε_0-DP local randomizer R, and shuffle the reports. Let the number of clones be $N_c \sim \mathrm{Bin}(n-1, e^{-\varepsilon_0})$, let $A \sim \mathrm{Bin}(N_c, \frac{1}{2})$ given N_c, and let the bit $F \sim \mathrm{Bern}(\frac{e^{\varepsilon_0}}{e^{\varepsilon_0}+1})$ be independent of both. Then the shuffled output on D and on D' is the same post-processing of*

$$\tilde{P} = (A + F,\ N_c - A + 1 - F) \qquad \text{and} \qquad \tilde{Q} = (A + 1 - F,\ N_c - A + F). \tag{35.5}$$

Each pair counts the reports that look like your D-version and like your D'-version, and $F = 1$ when your own report points to your true value.

Proof. The plan is to split every other user's report into clones and a leftover, to show that after the shuffle only the numbers of clones matter, and then to replace your own report by a randomized-response bit.

1. *Every report contains clones of yours.* By Lemma 26.4, for every value x there is a leftover distribution $\mathrm{LO}(x)$ with
$$R(x) = \frac{e^{-\varepsilon_0}}{2} R(x^0) + \frac{e^{-\varepsilon_0}}{2} R(x^1) + (1 - e^{-\varepsilon_0})\mathrm{LO}(x). \tag{35.6}$$
2. *Read the mixture as a three-sided coin.* A mixture is sampled by first choosing a component with its weight. So each other user flips a coin with faces "0", "1" and "leftover", and draws from $R(x^0)$, $R(x^1)$ or $\mathrm{LO}(x)$. The users flip independently, so the number of clones has the law of N_c, and given N_c the number of 0-clones has the law of A.
3. *After the shuffle, only the counts matter.* Give yourself the face 0 on D and 1 on D'. Someone who knows every other user's data, but not your bit, can produce the shuffled bag from the numbers of 0-faces and 1-faces alone, by drawing that many reports from $R(x^0)$ and $R(x^1)$, a leftover report for every other user, and shuffling. Those numbers are $(A+1, N_c - A)$ on D and $(A, N_c - A + 1)$ on D', so by the data-processing inequality the output is no more distinguishable than these counts (Lemma 3.3 of the paper).
4. *Your own report is a randomized response.* Kairouz, Oh & Viswanath (2015) show that any ε_0-DP randomizer on two inputs is a post-processing of binary randomized response. So there are two distributions Q_0 and Q_1 such that $R(x^0)$ draws from Q_0 with probability $\frac{e^{\varepsilon_0}}{e^{\varepsilon_0}+1}$ and from Q_1 otherwise, and $R(x^1)$ does the reverse. Your report therefore comes from the distribution of your true value exactly when $F = 1$.
5. *Recount by Q_0 and Q_1.* The weights in step 4 are symmetric, so a clone, a fair mix of $R(x^0)$ and $R(x^1)$, is also a fair mix of Q_0 and Q_1. Count reports drawn from Q_0 and from Q_1 instead of faces. The clones give $(A, N_c - A)$, and your report adds 1 to the first count on D when $F = 1$ and to the second when $F = 0$, the reverse on D'. That is Equation 35.5. This step is Lemma 3.4 of the paper, and Section L.3 in the appendices gives the full argument for a fixed randomizer.

□

Computing the exact $\delta(\varepsilon)$ of this pair of two-dimensional counts, with Lemma 20.2, gives the **numeric clone bound**. Feldman et al. also bound it in closed form, with Chernoff bounds on N_c and on A (their Lemma 3.5), which gives Theorem 26.2 of Trust models. For any $\delta \in [0,1]$ with $\varepsilon_0 \le \ln\left(\frac{n}{16\ln(2/\delta)}\right)$, the shuffled output is (ε, δ)-DP with
$$\varepsilon \le \ln\left(1 + \frac{e^{\varepsilon_0} - 1}{e^{\varepsilon_0} + 1}\left(\frac{8\sqrt{e^{\varepsilon_0}\ln(4/\delta)}}{\sqrt{n}} + \frac{8e^{\varepsilon_0}}{n}\right)\right). \tag{35.7}$$

Section L.3 carries out a version of the Chernoff calculation. The condition, $ne^{-\varepsilon_0} \ge 16\ln(2/\delta)$, asks for enough expected clones that N_c stays close to its mean except with probability about δ. At $\delta = 10^{-6}$ it allows ε_0 up to 1.460 for $n = 1000$, 3.763 for $n = 10^4$ and 6.066 for $n = 10^5$.

The closed form scales as Proposition 35.1 predicts for the count. Given n_c clones, the first coordinates of the pair, $A + F$ and $A + 1 - F$, have means that differ by $2\Pr[F = 1] - 1 = \frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1}$, while A has standard deviation $\sqrt{n_c}/2$. With $n_c \approx ne^{-\varepsilon_0}$, the ratio of the two is $2/(1 + e^{-\varepsilon_0})$

times the μ of Equation 35.3. The factor exceeds 1 because the clone argument counts only the fraction $e^{-\varepsilon_0}$ of reports guaranteed to look like yours. In Table 35.2 the numeric clone bound is 1.44 to 2.00 times the exact ε wherever there are many expected clones, close to that factor, and 2.36 at $n = 1000$ and $\varepsilon_0 = 4$, with only about $999e^{-4} = 18.3$ of them. No general bound can go below the exact column, since binary randomized response is one of the randomizers it covers.

The closed form is looser again, because its Chernoff constants are safe but not sharp. Where its condition holds it is 3.10 to 4.75 times the numeric clone bound, and 4.47 to 7.55 times the exact ε. The first bound, of Erlingsson et al. (2019), grows like $e^{3\varepsilon_0}$ instead of $e^{\varepsilon_0/2}$. So reason about scaling with the closed form, and deploy with the numeric bound.

Table 35.2.: Central ε at $\delta = 10^{-6}$ after shuffling n reports that are each ε_0-DP. "n/a" marks where the condition of Equation 35.7 fails; "—" marks where the bound of Erlingsson et al. is at least ε_0.

n	ε_0	Exact, binary randomized response	Normal approx., Equation 35.3	Clones, numeric	Clones, closed form	Erlingsson et al.
1,000	1	0.1266	0.1212	0.1824	0.5662	—
1,000	2	0.3233	0.2890	0.5455	n/a	—
1,000	4	1.6910	0.9667	3.9900	n/a	—
10,000	1	0.0357	0.0352	0.0530	0.2140	—
10,000	2	0.0870	0.0844	0.1550	0.5009	—
10,000	4	0.3146	0.2816	0.6009	n/a	—
100,000	1	0.0101	0.0101	0.0153	0.0726	0.4285
100,000	2	0.0247	0.0244	0.0450	0.1862	—
100,000	4	0.0847	0.0822	0.1698	0.5346	—

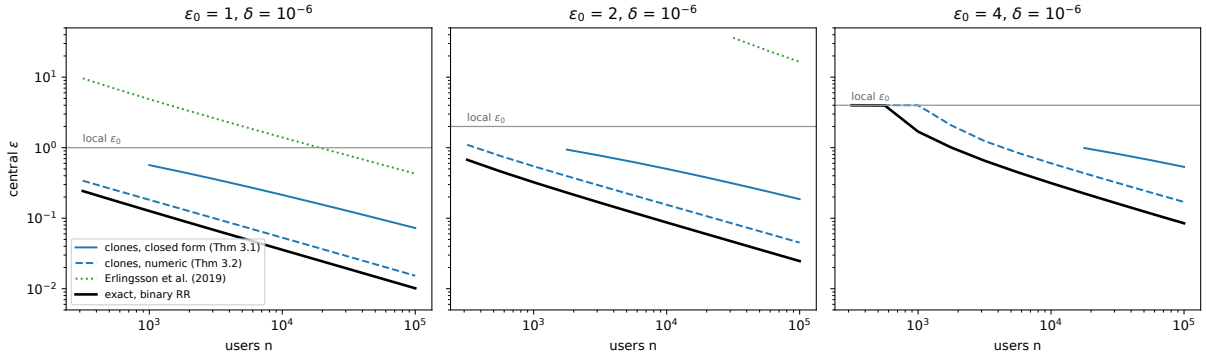


Figure 35.2.: Central ε at $\delta = 10^{-6}$ against the number of users n, for $\varepsilon_0 = 1, 2, 4$: the closed-form clone bound (where its condition holds), the numeric clone bound, the bound of Erlingsson et al. (where it is below 50) and the exact ε of shuffled binary randomized response. The grey line is the local ε_0.

Example 35.2. For the survey of 10,000 users at $\varepsilon_0 = 2$, Table 35.2 gives 0.0870 exactly, 0.1550 from the numeric clone bound and 0.5009 from the closed form.

The choice of accountant can decide whether a deployment is feasible. Suppose $n = 10^5$ users send a bit and you want central $\varepsilon \le 0.2$ at $\delta = 10^{-6}$. The closed form allows $\varepsilon_0 = 2$ (0.1862), but the numeric clone bound allows $\varepsilon_0 = 4$ (0.1698), where each report is the truth with probability $p = 0.9820$. If a fraction π of the population holds 1, the debiased estimate of π has variance

$$\frac{1}{n}\left(\pi(1-\pi) + \frac{p(1-p)}{(2p-1)^2}\right). \tag{35.8}$$

The noise term is $\frac{0.9820 \times 0.0180}{0.9640^2} = 0.0190$, against up to 0.25 for the sampling term you would pay without any privacy, so it costs little accuracy.

35.3. Amplification by iteration

The second half of the chapter turns from hiding which report is yours to hiding when your record was used. Projected noisy stochastic gradient descent (SGD) makes one pass over the data. At step t it takes a gradient step on record x_t, adds Gaussian noise and projects back onto a convex set. If every iterate is released, x_t is protected only by the noise of step t, but if only the final iterate is released, the noise of every later step also covers it. The obstacle is that the later steps are maps applied to random points, so their variances do not simply add. Feldman, Mironov, Talwar & Thakurta (2018) overcame it with a divergence that allows a bounded shift.

The argument needs every step to be a map that never pushes two points further apart.

Definition 35.2 (Contractive noisy iteration)**.** A map ψ is a **contraction** if $\|\psi(u) - \psi(v)\| \le \|u - v\|$ for all points u and v. A **contractive noisy iteration** is a sequence $X_t = \psi_t(X_{t-1}) + Z_t$, for $t = 1, \dots, T$, where each ψ_t is a contraction and $Z_t \sim \mathcal{N}(0, \sigma^2 I)$ is fresh noise of standard deviation σ.

A gradient step $\theta \mapsto \theta - \eta \nabla \ell(\theta)$ on a convex, β-smooth loss ℓ with learning rate $\eta \le 2/\beta$ is a contraction, by Lemma 7.2. So is the projection Π_K onto a closed convex set K, which maps each point to its nearest point of K, and so is their composition.

Lemma 35.1 (Projection is a contraction)**.** *Let K be a closed convex set with projection Π_K. Then $\|\Pi_K u - \Pi_K v\| \le \|u - v\|$ for all points u and v.*

Proof.

1. *Two obtuse angles.* Step 2 of the proof of Lemma 6.2 shows that the nearest point makes an obtuse angle with every other point of the set: $\langle u - \Pi_K u, \theta - \Pi_K u \rangle \le 0$ for every $\theta \in K$. Take $\theta = \Pi_K v$, and then exchange the roles of u and v.
2. *Add them.* Adding the two inequalities and collecting terms gives

$$\|\Pi_K u - \Pi_K v\|^2 \le \langle u - v, \Pi_K u - \Pi_K v \rangle. \tag{35.9}$$

3. *Cauchy–Schwarz.* The right side is at most $\|u - v\| \, \|\Pi_K u - \Pi_K v\|$. Dividing by $\|\Pi_K u - \Pi_K v\|$, when it is not 0, gives the claim.

□

The two runs, on D and on D', are pushed apart at the step that uses your record, and after that the contractions keep the gap from growing while the noise blurs it. So the quantity to track is how far apart the runs might be, a **shift**, and the divergence that would remain if one run could be moved by that shift. To move a random variable by at most z means to pair each of its values with one at most z away, a pairing called a **coupling**.

Definition 35.3 (Shifted Rényi divergence)**.** For random vectors X and X', an order $\alpha > 1$ and a shift $z \geq 0$, the **shifted Rényi divergence** is

$$D_\alpha^{(z)}(X\|X') = \inf\{D_\alpha(Y\|X') : \text{some coupling of } X \text{ and } Y \text{ has } \|X - Y\| \leq z \text{ almost surely}\}, \tag{35.10}$$

where D_α is the Rényi divergence of Divergences. With $z = 0$ it is D_α itself.

The noise pays off a shift at the price it charges for hiding one. Two normals of standard deviation σ whose means are a apart have, by Proposition 3.3,

$$R_\alpha(a) = D_\alpha(\mathcal{N}(0,\sigma^2)\,\|\,\mathcal{N}(a,\sigma^2)) = \frac{\alpha a^2}{2\sigma^2}. \tag{35.11}$$

Lemmas 20 and 21 of Feldman et al. drive everything. The first lets the noise pay off part of the shift at this price.

Lemma 35.2 (Shift reduction)**.** *Let $Z \sim \mathcal{N}(0,\sigma^2 I)$ be independent of X and of X'. For all $z \geq 0$ and $a \geq 0$,*

$$D_\alpha^{(z)}(X + Z\,\|\,X' + Z) \leq D_\alpha^{(z+a)}(X\|X') + R_\alpha(a). \tag{35.12}$$

The proof couples the two variables and splits each displacement into a part carried forward and a part of length at most a that the noise pays for. Section L.4 in the appendices derives it step by step. On two point masses, $X = 0$ and $X' = z$, the lemma is exact: moving $X + Z$ by $z - a$ towards $X' + Z$ leaves two normals a distance a apart, at divergence $R_\alpha(a)$.

The second lemma handles the gradient steps, which differ between the two runs at the step that uses your record. Different maps can only add the gap between them to the shift, at no cost.

Lemma 35.3 (Contraction)**.** *Let ψ be any map and ψ' a contraction, with $\|\psi(u) - \psi'(u)\| \leq s$ at every point u. Then for every $z \geq 0$,*

$$D_\alpha^{(z+s)}(\psi(X)\,\|\,\psi'(X')) \leq D_\alpha^{(z)}(X\|X'). \tag{35.13}$$

Proof.

1. *Start from a coupling.* Take any Y coupled with X so that $\|X - Y\| \leq z$ almost surely.
2. *Map it.* By the triangle inequality through $\psi'(X)$, and because ψ' is a contraction,

$$\|\psi(X) - \psi'(Y)\| \leq \|\psi(X) - \psi'(X)\| + \|\psi'(X) - \psi'(Y)\| \leq s + z. \tag{35.14}$$

3. *Data processing.* So $\psi'(Y)$ is an admissible move of $\psi(X)$ for the shift $z + s$. Applying the same map ψ' to Y and to X' cannot increase their divergence (Theorem 3.5), so $D_\alpha^{(z+s)}(\psi(X)\|\psi'(X')) \leq D_\alpha(Y\|X')$. Taking the infimum over Y gives the claim.

□

Chained over T steps, the lemmas become a piece of bookkeeping. The shift grows by s_t when the two maps differ at step t, and at each step you choose how much of it, a_t, the noise pays off.

Theorem 35.3 (Amplification by iteration)**.** *Let $X_t = \psi_t(X_{t-1}) + Z_t$ and $X'_t = \psi'_t(X'_{t-1}) + Z'_t$ be contractive noisy iterations from the same starting point, with noise of the same distribution, and let $s_t = \sup_u \|\psi_t(u) - \psi'_t(u)\|$. Choose payments $a_1, \dots, a_T \geq 0$ such that the unpaid shift $z_t = \sum_{i \leq t}(s_i - a_i)$ is never negative and $z_T = 0$. Then*

$$D_\alpha(X_T \| X'_T) \leq \sum_{t=1}^{T} R_\alpha(a_t). \tag{35.15}$$

Proof. The proof is an induction on the claim that, with the unpaid shift allowed, the divergence after step k is at most what has been paid so far: $D_\alpha^{(z_k)}(X_k \| X'_k) \leq \sum_{i \leq k} R_\alpha(a_i)$.

1. *Start.* At $k = 0$ both runs sit at the same point and $z_0 = 0$, so both sides are 0.
2. *Peel off the noise.* Step $k+1$ adds noise to $\psi_{k+1}(X_k)$ and to $\psi'_{k+1}(X'_k)$, so shift reduction with payment a_{k+1} gives
$$D_\alpha^{(z_{k+1})}(X_{k+1} \| X'_{k+1}) \leq D_\alpha^{(z_{k+1}+a_{k+1})}(\psi_{k+1}(X_k) \,\|\, \psi'_{k+1}(X'_k)) + R_\alpha(a_{k+1}). \tag{35.16}$$
3. *Peel off the maps.* By the definition of the unpaid shift, $z_{k+1} + a_{k+1} = z_k + s_{k+1}$. So the contraction lemma, with the gap s_{k+1}, bounds the shifted divergence on the right by $D_\alpha^{(z_k)}(X_k \| X'_k)$.
4. *Close the induction.* The claim for step k bounds that by $\sum_{i \leq k} R_\alpha(a_i)$, which proves the claim for step $k+1$. At $k = T$ the shift is $z_T = 0$, and the shifted divergence is the ordinary one.

□

The contractions never enter the cost. For one pass of noisy SGD the runs differ at a single step, and because the price is quadratic, small equal payments are cheapest. The theorem states its guarantee in Rényi differential privacy (RDP), and measures the noise per unit of the gradient's replace-one sensitivity $2G$, as the **noise multiplier** $\tilde{\sigma} = \sigma/(2G)$.

Theorem 35.4 (One pass of noisy SGD)**.** *Let the losses $\ell(\cdot\,; x)$ be convex, G-Lipschitz and β-smooth, and let K be a closed convex set. Run projected noisy SGD once over $x_1, \dots, x_n$, with $\theta_t = \Pi_K(\theta_{t-1} - \eta(\nabla \ell(\theta_{t-1}; x_t) + Z_t))$, $Z_t \sim \mathcal{N}(0, \sigma^2 I)$ and $\eta \leq 2/\beta$. Then releasing only θ_n is $(\alpha, \frac{2\alpha G^2}{\sigma^2(n-t+1)})$-RDP for the record x_t, for every $\alpha > 1$ (Theorem 23 of Feldman et al.). With $\sigma = 2G\tilde{\sigma}$, the RDP curve of the record at step t is*

$$\varepsilon_t(\alpha) = \frac{\alpha}{2\tilde{\sigma}^2} \cdot \frac{1}{n - t + 1} : \tag{35.17}$$

one step's Gaussian RDP curve, divided by the number of noisy steps from t to the end.

Proof. We write the pass as a contractive noisy iteration, find the one step where the runs differ, and pay as cheaply as possible.

1. *A contractive noisy iteration.* Write X_t for the iterate before projection, so that $\theta_t = \Pi_K(X_t)$. Then $X_t = \psi_t(X_{t-1}) - \eta Z_t$, with $\psi_t(u) = \Pi_K(u) - \eta\nabla\ell(\Pi_K(u); x_t)$. That map is a projection followed by a gradient step, a contraction by Lemma 35.1 and Lemma 7.2. The noise $-\eta Z_t$ has standard deviation $\eta\sigma$, and $\theta_n = \Pi_K(X_n)$ is a post-processing of X_n.
2. *The shift.* The runs on D and D' use different records only at step t. There the maps differ by at most $s_t = 2\eta G$, since each gradient has norm at most G, and at every other step they agree.
3. *Equal payments are cheapest.* Pay nothing before step t, and pay $a_t, \dots, a_n$ with $\sum_k a_k = s_t$ from step t on. Cauchy–Schwarz, applied to $(a_t, \dots, a_n)$ and a vector of $n - t + 1$ ones, gives

$$\Big(\sum_k a_k\Big)^2 \le (n - t + 1)\sum_k a_k^2, \tag{35.18}$$

   with equality for the even schedule $a_k = s_t/(n - t + 1)$. That schedule keeps every unpaid shift non-negative.
4. *Add up the price.* Each payment costs $\alpha a_k^2/(2\eta^2\sigma^2)$ by Equation 35.11 with noise $\eta\sigma$. So Theorem 35.3 with the even schedule gives

$$D_\alpha(X_n \| X_n') \le \frac{\alpha s_t^2}{2\eta^2\sigma^2(n - t + 1)}. \tag{35.19}$$

5. *Put in the shift.* With $s_t = 2\eta G$ the learning rate cancels, because it scales the shift and the noise alike, and the bound is $\frac{2\alpha G^2}{\sigma^2(n-t+1)}$. Substituting $\sigma = 2G\tilde{\sigma}$ gives Equation 35.17.

□

The even schedule loses nothing when the maps after step t are translations and the runs start from one point at step t. The last iterates are then Gaussians a distance s apart, and Table 35.3 and Figure 35.3 show the even schedule matching their exact divergence, while uneven schedules cost more.

The bound is exact in the worst case, when step t maps every point to one point and erases the noise added before it. For typical runs it is loose, because that earlier noise also hides the shift, and a map that strictly shrinks distances shrinks the shift for free. Read as a Gaussian pair, the bound gives the record at step t the parameter $\mu = s/(\sigma\sqrt{n - t + 1})$. Figure 35.4 compares it with the exact μ of a one-dimensional run of 100 steps with $s/\sigma = 1$, computed in Section 35.5. When each step shrinks distances by the factor 0.95, the exact μ at $t = 50$ is 0.0240, against the bound's $1/\sqrt{51} = 0.1400$.

Early records are safe and late ones are not. A uniformly random stopping time spreads this out, and Theorem 26 of Feldman et al. then gives every record $2\ln n$ times the guarantee of the first (Section L.5 derives a version). Several passes need Theorem 35.3 with several shifts, and Altschuler & Talwar (2022) combine iteration with subsampling.

Table 35.3.: Paying off a shift of 1 over four Gaussian steps with $\sigma = 1$, at $\alpha = 2$. The total equals $D_2(\mathcal{N}(0,4)\|\mathcal{N}(1,4)) = 0.25$.

Step	1	2	3	4	Total
Shift before the step	1	0.75	0.5	0.25	
Shift paid, a_k	0.25	0.25	0.25	0.25	1
Cost $\alpha a_k^2/(2\sigma^2)$	0.0625	0.0625	0.0625	0.0625	0.2500

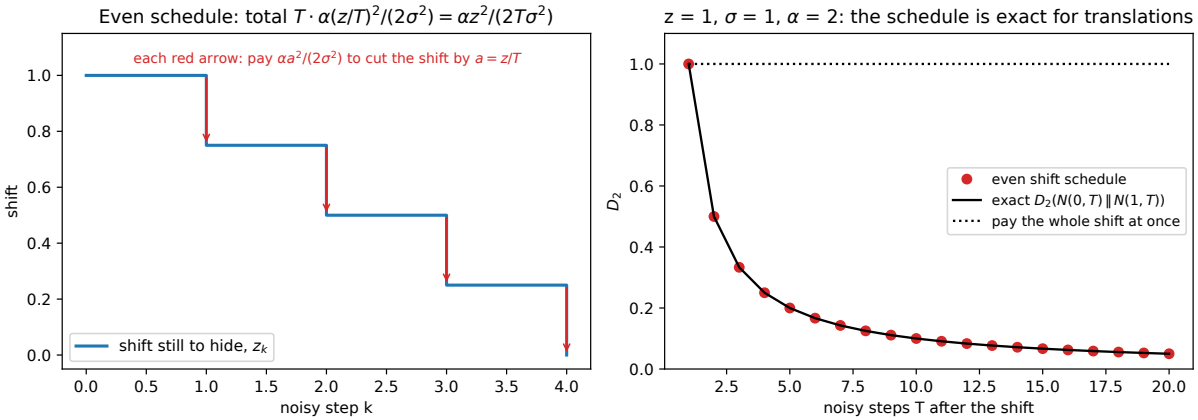


Figure 35.3.: Left: the even shift schedule over $T = 4$ noisy steps; each red arrow is a payment of $a = z/T$. Right: the total cost of the even schedule (dots) against the exact $D_2(\mathcal{N}(0,T)\|\mathcal{N}(1,T))$ (line), for a shift $z = 1$ and $\sigma = 1$; the dotted line pays the whole shift in one step.

Example 35.3. Projected noisy SGD makes one pass over $n = 10{,}000$ records with noise multiplier $\tilde{\sigma} = 1$. Mironov's conversion from From RDP to (ε, δ) turns an RDP curve into $\varepsilon = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1}$, minimized over a grid of orders α up to 512, and at $\delta = 10^{-5}$ each position gets its own answer.

The last record gets $\varepsilon = 5.2985$, from the RDP curve $\alpha/2$ of a single Gaussian mechanism, which is also what every record gets if all iterates are released (its exact value by Theorem 20.3 is 4.3772). The record used 100 steps before the end gets $\varepsilon = 0.4849$, the record in the middle gets $\varepsilon = 0.0707$, and the first record gets $\varepsilon = 0.0481$, at the largest order of the grid. So a report must state the worst record's value, or randomize the order and the stopping time.

35.4. Many steps on a bounded set

Full-batch noisy GD uses every record at every step, so the RDP of T steps grows linearly in T under composition. Theorem 35.3 does not help by itself, because every step adds a fresh shift s and the cheapest schedule pays s at every step, which is composition again. Yet if the parameters live in a bounded set, two runs can never be further apart than its diameter, however long they

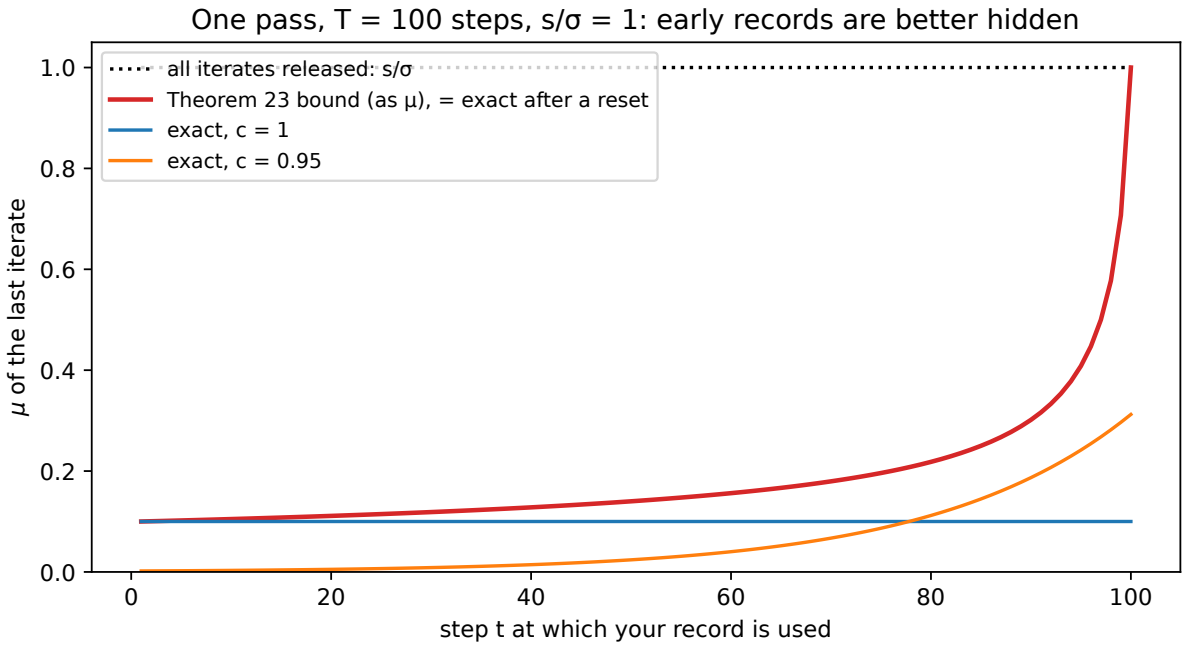


Figure 35.4.: The μ of the last iterate against the step t at which your record is used, for $T = 100$ steps with $s/\sigma = 1$. Releasing all iterates gives $\mu = 1$ (dotted). The bound of Theorem 35.4 (red) is exact when the run is reset just before step t; without shrinking (blue) the earlier noise also helps, and with steps that shrink distances by 0.95 (orange) the shift itself decays.

run. Altschuler & Talwar (2022) turned this into a bound that stops growing, and this section derives its full-batch form.

The idea is to ignore everything before some late step. At that step the two runs are at most the diameter d_K apart, and only the later steps need to pay off that distance plus the new shifts.

Proposition 35.2 (A bounded set caps the cost)**.** *Let two contractive noisy iterations have noise standard deviation σ and maps $\psi_t = g_t \circ \Pi_K$ and $\psi'_t = g'_t \circ \Pi_K$. Here Π_K projects onto a closed convex set K of diameter d_K, and g_t and g'_t are contractions that differ by at most s at every point. After T steps, from a common start,*

$$D_\alpha(X_T \| X'_T) \le \frac{\alpha}{2\sigma^2} \min\left\{ Ts^2,\ \min_{m\in\{1,\dots,T\}} \frac{(d_K + ms)^2}{m} \right\} \le \frac{2\alpha d_K s}{\sigma^2} \quad \text{when } d_K/s \text{ is an integer.} \tag{35.20}$$

Proof. We start the accounting m steps before the end, pay evenly, compare with composition, and choose m.

1. *Start the accounting late.* Fix $\tau = T - m$. The projected points $\Pi_K(X_\tau)$ and $\Pi_K(X'_\tau)$ both lie in K, at most d_K apart, so moving each onto its partner shows that their shifted divergence with shift d_K is 0. This is the only place where boundedness enters.
2. *Run the induction from there.* The induction in the proof of Theorem 35.3, started at step τ from shift d_K and divergence 0, applies with the contractions g_t. Its payments must total $d_K + ms$, the starting gap plus s per step.
3. *Pay evenly.* Paying $a = (d_K + ms)/m$ at every step pays d_K/m more than the step adds, so the unpaid shift falls from d_K to 0 and never goes negative. The total price is
$$m \cdot \frac{\alpha a^2}{2\sigma^2} = \frac{\alpha(d_K + ms)^2}{2m\sigma^2}. \tag{35.21}$$
4. *Or start at the beginning.* From the common start, paying s per step costs $\alpha T s^2/(2\sigma^2)$, the composition of T Gaussian steps. The better of the two choices gives the first bound.
5. *Choose m.* A short window means few, large payments, and a long one means more shift to pay. Expanding the square shows the trade-off:
$$\frac{(d_K + ms)^2}{m} = \frac{d_K^2}{m} + 2d_K s + ms^2. \tag{35.22}$$
Over real $m > 0$ the falling and the growing terms balance at $m = d_K/s$, where the sum is $4d_K s$. When d_K/s is an integer and $T \ge d_K/s$, that m is allowed. When $T < d_K/s$, the composition branch Ts^2 is below $d_K s$. Either way the bound is at most $\frac{\alpha}{2\sigma^2} \cdot 4d_K s = \frac{2\alpha d_K s}{\sigma^2}$.

□

The bound follows composition until $T = 4d_K/s$ and stays flat after that. For other d_K/s, taking $m = \lceil d_K/s \rceil$ gives at most $\frac{\alpha}{2\sigma^2}(4d_K s + s^2)$. For full-batch GD on n records the shift is $s = 2\eta G/n$. Theorem 3.1 of Altschuler & Talwar proves the same shape for mini-batch noisy SGD, and their Theorem 4.1 shows it tight up to constants. All these results need **only the last iterate to be released**. Checkpoints or a model released after each epoch show the run before it has forgotten, and the accounting is back to composition. They also need contractive steps, and so convexity. For

non-convex losses, Annamalai (2024) constructs a loss for which the last iterate of differentially private SGD (DP-SGD) leaks as much as all iterates together. Nasr et al. (2021) found that their attacks fall short of the DP-SGD bound when the adversary sees only the final model, but that observation is not a guarantee. So do not report an ε that relies on a hidden state for deep learning.

Example 35.4. Take the parameter set $K = [0, 1]$, so $d_K = 1$, with per-step shift $s = 0.01$ and noise $\sigma = 0.1$, so that $d_K/s = 100$ is an integer. Each composed step costs $\frac{\alpha s^2}{2\sigma^2} = 0.01$ at $\alpha = 2$. The plateau of Equation 35.20 is $2\alpha d_K s/\sigma^2 = 2\alpha$, which is 4.0000 at $\alpha = 2$, reached at $T = 4d_K/s = 400$ steps. After 2000 steps composition charges 20.0000, five times the bound.

How loose is the constant? With a linear loss the two runs are random walks drifting in opposite directions and projected onto the interval, the construction behind Altschuler & Talwar's lower bound. The script computes their D_2 on a grid of 1000 cells (500 cells give 0.3779 against 0.3786 at $T = 2000$). It flattens at 0.3786 after about 100 steps (Table 35.4 and Figure 35.5, right). This walk is one pair of datasets, so its divergence is a lower bound on the worst case, and the constant of the bound is at most $4.0000/0.3786 = 10.57$ times too large.

Table 35.4.: D_2 of the last iterate of full-batch noisy GD on a linear loss over $K = [0, 1]$, with $s = 0.01$ and $\sigma = 0.1$.

Steps T	10	100	400	1000	2000
Composition, $\alpha T s^2/(2\sigma^2)$	0.1000	1.0000	4.0000	10.0000	20.0000
Bounded-domain bound, Equation 35.20	0.1000	1.0000	4.0000	4.0000	4.0000
Projected random walks, numerical	0.0961	0.3693	0.3786	0.3786	0.3786

35.5. Noisy gradient descent on a quadratic

All the results so far are bounds. In one dimension with a quadratic loss, each step of noisy GD is a linear map plus Gaussian noise, so the last iterate is Gaussian on both datasets and its privacy can be computed exactly. The computation shows the plateau directly, shows that it needs contraction, and gives a yardstick for the bounds on strongly convex losses.

Full-batch noisy GD on the loss $\ell(\theta; x) = \frac{\lambda}{2}(\theta - x)^2$ has the averaged gradient $\lambda(\theta - \bar{x})$, where $\bar{x}$ is the mean of the records. So a step multiplies θ by a **contraction factor** $c = 1 - \eta\lambda$ and adds the offset $\eta\lambda\bar{x}$ and noise. Replacing one of n records by one that differs by v moves the offset by $s = \eta\lambda v/n$, at every step from the one where the record joins.

Theorem 35.5 (Exact privacy of the last iterate)**.** *Let $\theta_k = c\,\theta_{k-1} + b_k + \xi_k$ for $k = 1, \dots, T$, from a fixed θ_0, with $0 < c \le 1$, offsets b_k and independent noise $\xi_k \sim \mathcal{N}(0, \sigma^2)$. Let the offsets*

on D and D' differ by s at every step $k \geq t$ and agree before. Then the last iterate is exactly μ_T-GDP, with

$$\mu_T = \frac{s}{\sigma} \cdot \frac{\sum_{k=t}^{T} c^{T-k}}{\sqrt{\sum_{j=0}^{T-1} c^{2j}}}, \qquad \text{and for } t = 1,\ c < 1: \qquad \lim_{T\to\infty} \mu_T = \frac{s}{\sigma}\sqrt{\frac{1+c}{1-c}}. \tag{35.23}$$

Releasing all iterates instead is $\sqrt{T-t+1}\, s/\sigma$-GDP, and for $c = 1$ and $t = 1$ the two agree.

Proof. Unroll the run into a sum, read off its mean and variance, and divide; then take the limit.

1. *Unroll.* Substituting each step into the next shows, by induction on T, that each step's offset and noise are multiplied by c once for every later step:
$$\theta_T = c^T \theta_0 + \sum_{k=1}^{T} c^{T-k}(b_k + \xi_k). \tag{35.24}$$
2. *The noise.* The noise part is a sum of independent normals, so it is normal with mean 0 and variance $\sigma^2 \sum_{j=0}^{T-1} c^{2j}$, the same on both datasets.
3. *The means.* The start and the offsets before step t are the same in both runs and cancel, so the means differ by $s \sum_{k\geq t} c^{T-k}$.
4. *A Gaussian pair.* Two normals with the same variance are μ-GDP with μ their mean difference in standard deviations, which gives the first formula.
5. *The limit.* For $t = 1$ both sums are geometric, $(1-c^T)/(1-c)$ on top and $(1-c^{2T})/(1-c^2)$ under the root. As $c^T \to 0$, the ratio tends to $\sqrt{1-c^2}/(1-c)$, and writing $1-c^2 = (1-c)(1+c)$ gives $\sqrt{(1+c)/(1-c)}$.
6. *All iterates.* Releasing every iterate composes $T-t+1$ Gaussian steps, each s/σ-GDP given the past. GDP composes by adding squares (f-DP and Gaussian DP), which gives $\sqrt{T-t+1}\, s/\sigma$. For $c = 1$ every power of c is 1, and step 4 gives $(T-t+1)s/(\sigma\sqrt{T})$, which for $t = 1$ is $\sqrt{T}s/\sigma$.

□

Contraction is what forgets. With $c < 1$, the offset of step k reaches the end damped by c^{T-k} while fresh noise keeps arriving, so the signal saturates at $s/(1-c)$ and the noise at $\sigma/\sqrt{1-c^2}$. With $c = 1$ the signal grows like T and μ like $\sqrt{T}$ (Figure 35.5, left). The plateau arrives after a few multiples of $1/(1-c)$ steps. With $c = 0.99$, μ is 9.6101 at $T = 100$ and 14.1061 at $T = 1000$, against the limit $\sqrt{199} = 14.1067$. On a λ-strongly convex, β-smooth loss with $\eta = 1/\beta$, that is the condition number β/λ.

Chourasia, Ye & Shokri (2021, Corollary 1) bound the RDP of the last iterate of full-batch noisy GD on smooth strongly convex losses. In the notation of this section its limit is $\frac{2\alpha s^2}{(1-c)\sigma^2}$. The exact limiting RDP is $\alpha\mu^2/2$ with the limit of Equation 35.23, so the ratio of exact to bound is $\frac{1+c}{4}$. At $c = 0.99$, $\alpha = 2$ and $s/\sigma = 1$ that is 199.0000 against 400.0000, so the bound is within a factor of about 2.

Example 35.5. Run $T = 10{,}000$ steps of full-batch noisy GD with noise ten times the per-step shift, $\sigma/s = 10$, on a quadratic with $c = 0.99$. Releasing every iterate is $\mu = \sqrt{10{,}000}/10 = 10.0000$-GDP, which Theorem 20.3 turns into $\varepsilon = 91.8173$ at $\delta = 10^{-5}$, no guarantee at all. Releasing only

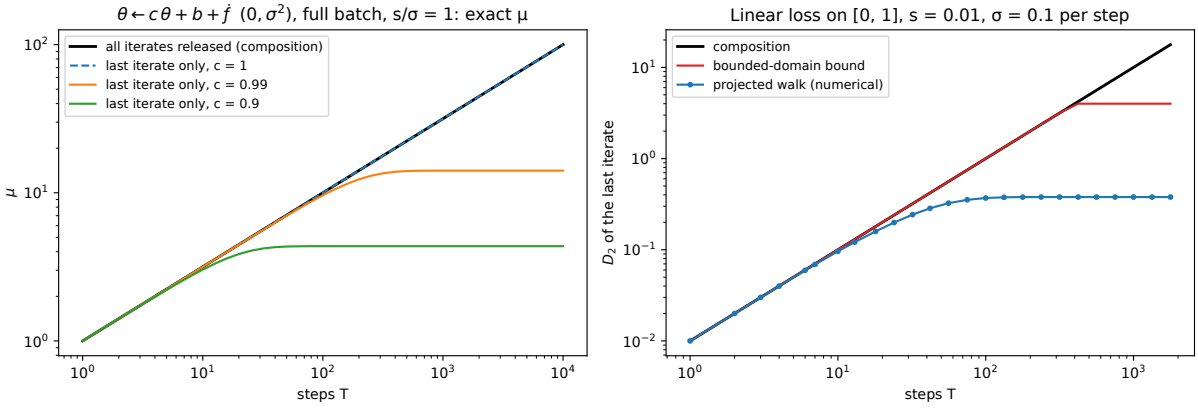


Figure 35.5.: Left: the exact μ of the last iterate of one-dimensional full-batch noisy GD with $s/\sigma = 1$, against the number of steps T, for $c = 1$ (dashed, on the black composition line), $c = 0.99$ and $c = 0.9$. Right: D_2 of the last iterate for a linear loss on $[0, 1]$ with $s = 0.01$ and $\sigma = 0.1$: composition, the bound Equation 35.20, and two projected random walks.

the final model is, after this many steps, at the limit of Equation 35.23, $\mu = 0.1 \times 14.1067 = 1.4107$, which is $\varepsilon = 6.5534$. The noise is the same, and the difference lies entirely in what the adversary may see.

35.6. Code

The script code/shuffling_iteration.py computes every number in this chapter and draws every figure. It computes shuffled randomized response and the clone pair exactly, in log space, and checks the closed forms for Gaussian pairs by Monte Carlo. From the repository root:

```
uv run python code/part04-privacy-accounting/shuffling_iteration.py
```

Table 35.5.: The script's output with the default seed.

Experiment	Result
Shuffled randomized response, $n = 3$, $\varepsilon_0 = \ln 3$	$\delta(0.5) = 0.1900$ against 0.3378 for one report; simulation within 0.0015 of the exact count distribution
Worst dataset pair, all m at $n = 1000$, $m \le 30$ at $n = 10^4, 10^5$	The pair $m = 0$ is within 4.5×10^{-5} of the largest ε found
Clone decomposition, $\varepsilon_0 = \ln 3$	Clone part $(0.1667, 0.1667)$, leftover $(0.875, 0.125)$
Shuffling at $\delta = 10^{-6}$	Table 35.2; numeric clone bound 1.44 to 2.36 times the exact value
Shift reduction, $z = 1$, $\sigma = 1$, $\alpha = 2$	Even schedule equals the exact D_2 at $T = 1, 2, 4, 10$

Experiment	Result
One pass over 10^4 records, $\tilde{\sigma} = 1$	ε from 0.0481 (first record) to 5.2985 (last) at $\delta = 10^{-5}$
Bounded domain, $d_K = 1$, $s = 0.01$, $\sigma = 0.1$	Table 35.4; plateau 4.0000 against 0.3786 for the projected random walks
Quadratic, $s/\sigma = 1$	Limits 4.3589 ($c = 0.9$) and 14.1067 ($c = 0.99$); μ estimated from 200,000 simulated runs ($c = 0.9$, $T = 50$) 4.3381 against the exact 4.3365

tests/test_shuffling_iteration.py checks that the output still matches.

35.7. Summary

Table 35.6.: The results of this chapter, and where each is used.

Result	Statement	Use
Theorem 35.1	Shuffled binary randomized response is the count pair Equation 35.1	Compute it exactly, in milliseconds: the yardstick for every shuffling bound. Pure DP is not amplified
Proposition 35.1	$\mu \approx 2\sinh(\varepsilon_0/2)/\sqrt{n}$	Sizing a deployment; wrong when the others send few 1-reports
Theorem 35.2	Output is a post-processing of the clone pair Equation 35.5	Any local randomizer: deploy with the numeric clone bound, 1.44 to 2.36 times the exact value
Equation 35.7	Closed form, for $\varepsilon_0 \le \ln \frac{n}{16\ln(2/\delta)}$	Reasoning about scaling only: 4.47 to 7.55 times the exact value
Lemma 35.1	Projection onto a convex set is a contraction	So is a projected gradient step with $\eta \le 2/\beta$
Lemma 35.2	$D_\alpha^{(z)}(X + Z \Vert X' + Z) \le D_\alpha^{(z+a)}(X \Vert X') + \frac{\alpha a^2}{2\sigma^2}$	Noise pays off a shift at a quadratic price
Lemma 35.3	A contraction adds the maps' gap to the shift, at no cost	Steps on different records
Theorem 35.3	$D_\alpha(X_T \Vert X'_T) \le \sum_t R_\alpha(a_t)$ for any feasible payments	Pay evenly, never before the shift arrives
Theorem 35.4	$\varepsilon_t(\alpha) = \frac{\alpha}{2\tilde{\sigma}^2(n-t+1)}$, last iterate only	One-pass convex training; state the worst record

Result	Statement	Use
Proposition 35.2	$\leq \frac{\alpha}{2\sigma^2}\min\{Ts^2, \min_m(d_K + ms)^2/m\} \leq \frac{2\alpha d_K s}{\sigma^2}$	Long convex runs releasing only the final model; never with checkpoints or non-convex losses
Theorem 35.5	μ_T of Equation 35.23, limit $\frac{s}{\sigma}\sqrt{\frac{1+c}{1-c}}$	Exact yardstick: without contraction the last iterate reveals as much as all of them

35.8. References

- Feldman, McMillan & Talwar, "Hiding Among the Clones: A Simple and Nearly Optimal Analysis of Privacy Amplification by Shuffling" (FOCS 2021; arXiv:2012.12803). Theorems 3.1 and 3.2 and Lemmas 3.3–3.5 are the clone bound and reduction used here; Section 6 gives the numerical method, and their code computes it. Also covers approximate-DP randomizers (Theorem 3.8) and RDP of shuffled mechanisms.
- Erlingsson, Feldman, Mironov, Raghunathan, Talwar & Thakurta, "Amplification by Shuffling: From Local to Central Differential Privacy via Anonymity" (SODA 2019; arXiv:1811.12469). The first amplification-by-shuffling theorem (Theorem 7 of the arXiv version), for adaptive randomizers; read it for the swap argument and for the encode–shuffle–analyse motivation.
- Balle, Bell, Gascón & Nissim, "The Privacy Blanket of the Shuffle Model" (CRYPTO 2019). Writes each randomizer as a mixture with an input-independent "blanket" and derives amplification bounds from it, including a closed form tailored to k-ary randomized response.
- Cheu, Smith, Ullman, Zeber & Zhilyaev, "Distributed Differential Privacy via Shuffling" (EUROCRYPT 2019). An early formal study of the shuffle model; its analysis of shuffled binary randomized response already has the $e^{\varepsilon_0/2}/\sqrt{n}$ dependence.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015). Shows that every ε_0-DP randomizer on two inputs is a post-processing of binary randomized response, the fact used in step 4 of the clone reduction.
- Feldman, Mironov, Talwar & Thakurta, "Privacy Amplification by Iteration" (FOCS 2018; arXiv:1808.06651). Shifted Rényi divergence, the shift-reduction lemma (Lemma 20), the contraction lemma (Lemma 21), Theorem 22, and noisy SGD (Theorems 23, 24 and 26).
- Altschuler & Talwar, "Privacy of Noisy Stochastic Gradient Descent: More Iterations without More Privacy Loss" (NeurIPS 2022; arXiv:2205.13710). The bounded-domain plateau (Proposition 3.2, Theorem 3.1; the full-batch form Equation 35.20 is this chapter's specialization of Proposition 3.2), a matching lower bound (Theorem 4.1), and extensions to strongly convex losses and cyclic batches.
- Chourasia, Ye & Shokri, "Differential Privacy Dynamics of Langevin Diffusion and Noisy Gradient Descent" (NeurIPS 2021; arXiv:2102.05855). Converging RDP of the last iterate of noisy GD for smooth strongly convex losses (Corollary 1), proved through Langevin diffusion and log-Sobolev inequalities.
- Ye & Shokri, "Differentially Private Learning Needs Hidden State (Or Much Faster Convergence)" (NeurIPS 2022; arXiv:2203.05363). Extends the converging bound to mini-batch noisy SGD with shuffle-and-partition and sampling without replacement.

- Nasr, Song, Thakurta, Papernot & Carlini, "Adversary Instantiation: Lower Bounds for Differentially Private Machine Learning" (IEEE S&P 2021; arXiv:2101.04535). Attacks that match the DP-SGD bound for strong adversaries and fall short when the adversary sees only the final model.
- Annamalai, "It's Our Loss: No Privacy Amplification for Hidden State DP-SGD With Non-Convex Loss" (ACM Workshop on Artificial Intelligence and Security, AISec 2024; arXiv:2407.06496). A non-convex loss for which the last iterate leaks as much as all iterates.

36. Adaptive composition: filters, odometers and individual accounting

Every composition theorem so far assumes that the privacy parameters of the steps are fixed before the run starts. The mechanisms may be chosen after seeing earlier outputs, as in Composition (Theorem 21.5), but the per-step budgets and the number of steps must be known in advance. Real systems break this. You train until the validation loss stops improving, or lower the noise when training stalls, so the number of steps and their budgets depend on the outputs. And when a model is trained with differential privacy (DP) by stochastic gradient descent (SGD), each record's gradient is scaled down to a maximum norm, and a record whose gradient is small costs less than the worst case, by an amount that depends on the current model.

Plugging the parameters that occurred into a fixed-parameter theorem fails, because an analyst who chooses when to stop can make the bound fail about three times as often as it claims. So the question splits in two. A **privacy filter** decides before each step whether the run may continue within a budget fixed in advance. A **privacy odometer** reports how much has been spent, and it must hold at every step at once, because you may stop at any of them. The one idea behind both is a quantity built from the privacy loss that never goes up on average, since stopping it at a time of your choosing cannot raise its mean. In Rényi differential privacy (RDP) this makes adaptivity almost free: if the spends at a fixed order add up to at most a budget on every path, the run has that budget.

The running example is full-batch noisy gradient descent whose noise has standard deviation $\sigma = 10$ times the sensitivity, the noise multiplier. One hundred steps planned in advance are $(5.299, 10^{-5})$-DP, best at the Rényi order 5.8, where the RDP budget is $B = 2.900$ (Example 36.2). The chapter lowers the noise mid-run, reports the spend with an odometer, and tracks each record's own spend. The pure-DP sections use randomized response instead, because a Gaussian step is not pure DP.

Table 36.1.: The symbols of this chapter.

Symbol	Meaning
t; y_t; $y_{<t}$	a step; the output it releases; the outputs released before it
L_t; $L_{1:t}$	the privacy loss of step t given the past; the loss of the first t steps
ε_t; $\varepsilon_t(\alpha)$	the pure-DP parameter of step t; its RDP spend at order α
T	the last step a run takes, decided from the outputs

Symbol	Meaning
(ε_g, δ); U_t	a filter's global budget; an odometer's running bound
N_t; B	the supermartingale of the Rényi filter; its budget at order α
$S_t(\alpha)$; Λ	the spend of the first t steps at order α; a finite grid of orders

36.1. When the parameters are chosen during the run

Before building anything, you need to see why the obvious approach fails. Advanced composition bounds the privacy loss of k steps of ε-DP, with k and ε fixed, except with probability δ (Equation 21.17). The shortcut is to apply it **ex post**, reporting advanced composition of the parameters used when you stop. Rogers, Roth, Ullman & Vadhan (2016) showed that this is not valid, because a bound that fails with probability δ at each fixed time can fail far more often at a time chosen from the outputs.

A run has steps $t = 1, 2, ...$, and step t releases y_t from a mechanism chosen from the earlier outputs $y_{<t}$. On neighbours $D \sim D'$, which differ by adding or removing one record, write $P_t(y_t \mid y_{<t})$ and $Q_t(y_t \mid y_{<t})$ for the probability of y_t after that history on D and on D'. The privacy loss of step t is $L_t = \ln\left(P_t(y_t \mid y_{<t})/Q_t(y_t \mid y_{<t})\right)$, and by Lemma 21.2 the loss of the first t steps is the sum $L_{1:t}$. Step t is ε_t-DP whatever the history, where ε_t may depend on $y_{<t}$ but not on y_t. Such a parameter is **predictable**. The **analyst** is whoever makes these choices from the outputs. Probabilities are under P, the distribution of the whole transcript on D, unless written otherwise.

Definition 36.1 (Privacy filter)**.** A **privacy filter** with budget (ε_g, δ) is a rule that, before each step, looks at the parameters $\varepsilon_1, \dots, \varepsilon_t$ so far, including the one about to run, and says *continue* or *halt*. It is valid if the stopped run $(y_1, \dots, y_T)$ is (ε_g, δ)-DP for every analyst, however the analyst picks the steps and their parameters.

An odometer makes a claim at every step. Since you may stop at any of them, the claim you report is only safe if it held at all times at once.

Definition 36.2 (Privacy odometer)**.** A **privacy odometer** with confidence δ is a sequence of functions $U_t(\varepsilon_1, \dots, \varepsilon_t)$ of the parameters spent so far. It is valid if, for every analyst and every pair of neighbours, the probability that the loss ever exceeds the running bound is at most δ:

$$\Pr\left[\exists\, t : L_{1:t} > U_t\right] \le \delta. \tag{36.1}$$

The natural candidate for U_t is advanced composition of the parameters used so far, which is correct at every fixed time and still not an odometer. The simplest mechanism shows it. Randomized response at level ε reports a person's bit truthfully with probability $e^\varepsilon/(1 + e^\varepsilon)$ and flips it otherwise.

Proposition 36.1 (The plug-in odometer fails)**.** *Let every step be randomized response at level ε on the same person's bit, and let*

$$U_t = t\varepsilon \tanh(\varepsilon/2) + \varepsilon\sqrt{2t\ln(1/\delta)}. \tag{36.2}$$

At every fixed t, $\Pr[L_{1:t} > U_t] \le \delta$. But the probability that $L_{1:t} > U_t$ at some $t \le n$ tends to 1 as $n \to \infty$, so U_t is not a valid odometer.

Proof. The plan is to prove the fixed-time bound with Azuma's inequality, then to show that an analyst who stops at the first crossing turns the crossing probability into the failure probability, and that this probability tends to 1.

1. *The loss is a walk.* Take a person whose bit is 1 in D and 0 in D'. A report of 1 has probability $e^\varepsilon/(1+e^\varepsilon)$ on D and $1/(1+e^\varepsilon)$ on D', so its loss is $+\varepsilon$, and a report of 0 has loss $-\varepsilon$. By Proposition 21.1 the mean loss per step under D is $\varepsilon\tanh(\varepsilon/2)$. So $L_{1:t}$ is a random walk with steps $\pm\varepsilon$ and a slight upward drift.
2. *Centre it.* Subtracting the drift gives $M_t = L_{1:t} - t\varepsilon\tanh(\varepsilon/2)$, a martingale whose increments lie in intervals of length 2ε.
3. *The fixed-time bound.* Azuma's inequality (Theorem 2.7) bounds the chance that such a martingale exceeds s after t steps by $\exp(-2s^2/(4t\varepsilon^2))$. Setting this to δ gives $s = \varepsilon\sqrt{2t\ln(1/\delta)}$, and adding the drift back gives $\Pr[L_{1:t} > U_t] \le \delta$.
4. *The analyst's rule.* Run until $L_{1:t} > U_t$, then stop. The loss is ε times the number of reports of 1 minus the number of reports of 0, a function of the outputs, so this rule decides from what has been released.
5. *The event the odometer must control.* The claim $L_{1:T} \le U_T$ fails exactly when the walk crossed the boundary at some $t \le n$, because that is when the analyst stops. So the failure probability is $\Pr[\exists\, t \le n : L_{1:t} > U_t]$, a probability over the whole path.
6. *The crossing probability tends to 1.* The boundary sits $\varepsilon\sqrt{2t\ln(1/\delta)}$ above the mean, a fixed multiple of $\sqrt{t}$. The centred steps are $\pm\varepsilon$ minus their mean, so their variance is the mean square minus the squared mean, $v = \varepsilon^2 - \varepsilon^2\tanh^2(\varepsilon/2)$. The law of the iterated logarithm, which we cite (Durrett, in the References), says that a centred walk with step variance v exceeds $\frac{1}{2}\sqrt{2vt\ln\ln t}$ infinitely often with probability 1. That curve grows faster than any fixed multiple of $\sqrt{t}$, because $\ln\ln t$ grows without bound, so the walk crosses the boundary with probability 1.

□

The theorem was never wrong; its use was. An analyst who stops when the loss happens to be high selects exactly the paths where a fixed-time bound fails. Rogers et al. (2016, Theorem 6.1) turn this construction into a lower bound: every valid odometer pays an extra $\ln\ln n$ inside the logarithm, where n is the number of steps.

Basic composition does not suffer from this, because $|L_{1:t}| \le \sum_{j\le t}\varepsilon_j$ holds for every output and leaves no bad event to select. Filters escape by another route. A filter fixes the budget in advance and only asks whether the next step fits, so there is no running claim to catch out, and Section 36.3 answers that question with no $\ln\ln$ loss.

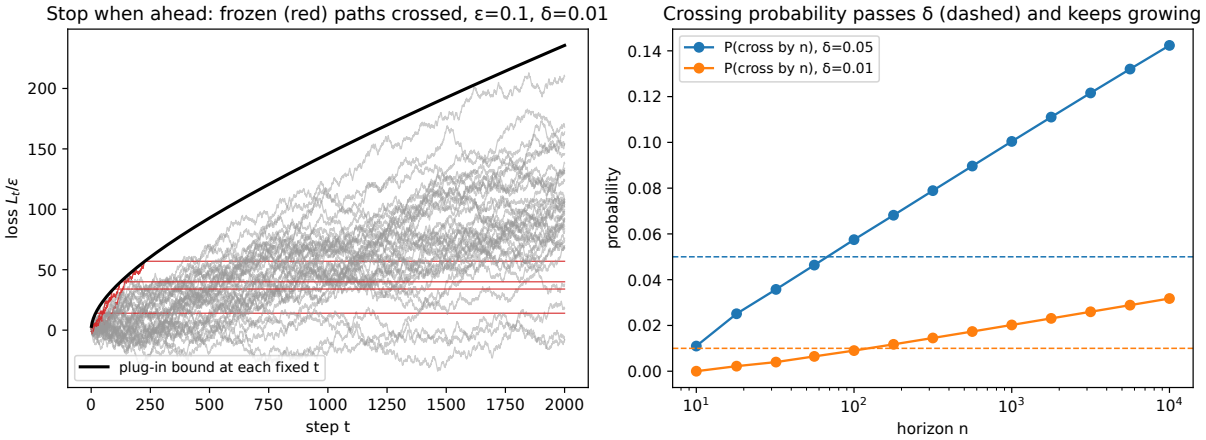


Figure 36.1.: Left: forty runs of randomized response with $\varepsilon = 0.1$, the loss in units of ε, and the plug-in advanced-composition bound at $\delta = 0.01$. Red paths stop at their first crossing. Right: the exact probability of a crossing by step n, for $\delta = 0.05$ and 0.01 (dashed).

Example 36.1. Take randomized response with $\varepsilon = 0.1$, whose mean loss per step is $0.1 \times \tanh(0.05) = 0.004996$. The crossing probability is exact: propagate the distribution of $L_{1:t}/\varepsilon$, an integer walk, step by step, and remove at each t the mass above U_t/ε (Table 36.2).

Table 36.2.: The exact probability $\Pr[\exists\, t \le n : L_{1:t} > U_t]$ that randomized response with $\varepsilon = 0.1$ crosses the plug-in bound by step n. The last column is the largest probability of exceeding it at one fixed step $t \le 1000$, and that step.

Horizon n	10	100	1000	10,000	worst single fixed $t \le 1000$
$\delta = 0.05$	0.0110	0.0575	0.1004	0.1424	0.0110 ($t = 17$)
$\delta = 0.01$	0.0000	0.0090	0.0202	0.0318	0.0016 ($t = 21$)
$\delta = 10^{-5}$	0	3.12×10^{-6}	1.80×10^{-5}	3.69×10^{-5}	8.18×10^{-7} ($t = 758$)

With $\delta = 0.01$, the plug-in bound after $t = 100$ steps is $100 \times 0.004996 + 0.1\sqrt{200 \ln 100} = 3.534$. In Figure 36.1 (left), a few runs cross the boundary early, where it is narrow, and the analyst freezes them there. By $n = 10^4$ a fraction 0.0318 of runs have crossed, so a report "$\varepsilon = U_T$ with $\delta = 0.01$" is wrong $0.0318/0.01 = 3.18$ times as often as it claims. At $\delta = 0.05$ the factor is $0.1424/0.05 = 2.85$, although the bound fails at a single fixed time $t \le 1000$ with probability at most 0.0110. A small δ does not cure it: at $\delta = 10^{-5}$ the factor is 3.69.

36.2. The Rényi filter

Rényi differential privacy describes a mechanism by a curve. The Rényi divergence of order $\alpha > 1$ is $D_\alpha(P\|Q) = \frac{1}{\alpha-1} \ln \mathbb{E}_P[e^{(\alpha-1)L}]$, a log of an exponential moment of the privacy loss L (Equation 31.2), and a mechanism is $(\alpha, \varepsilon(\alpha))$-RDP if this is at most $\varepsilon(\alpha)$ for every pair of neighbours. RDP composition adds curves fixed in advance (Equation 29.11). This section asks what happens when the spend $\varepsilon_t(\alpha)$ depends on the earlier outputs. Feldman & Zrnic (2021) and Lécuyer (2021) showed that nothing is lost at a fixed order, and every later section rests on that.

The proof stops a random process at a time chosen from the outputs, so it needs two tools. The first extends Lemma 21.2 to a run whose length is decided by its outputs.

Lemma 36.1 (The loss of a stopped run)**.** *Let the run stop after step T, where the decision to stop after y_t is a function of $y_1, \dots, y_t$ alone. Then the privacy loss of the stopped transcript $(y_1, \dots, y_T)$ is $L_{1:T} = \sum_{t \le T} L_t$.*

Proof.

1. *Chain rule.* The transcript has probability $\prod_{t \le T} P_t(y_t \mid y_{<t})$ under P, times the probability of the stopping decisions, and likewise with Q_t under Q.
2. *The stopping decisions cancel.* Each decision is a function of outputs already released, so given them it has probability 1 or 0, the same under P and Q, and it cancels from the ratio.
3. *Take logs.* The log of the remaining ratio of products is the sum of the step losses.

□

The second tool says that stopping early cannot raise the mean of a process that never goes up on average. Picture your wealth in a game that is fair or unfavourable to you, where you may quit at any time by looking at the past but not at the future.

Definition 36.3 (Supermartingale and stopping time)**.** A process $N_0, N_1, \dots$, with N_t a function of $y_{\le t}$, is a **supermartingale** if $\mathbb{E}[N_t \mid y_{<t}] \le N_{t-1}$ for every t, and a **martingale** if equality holds. A **stopping time** T is a random step such that the event $\{T \ge t\}$, "the run takes step t", is decided by $y_{<t}$.

Lemma 36.2 (Optional stopping)**.** *Let N be a supermartingale and T a stopping time with $T \le n$ for a fixed n. Then*

$$\mathbb{E}[N_T] \le \mathbb{E}[N_0]. \tag{36.3}$$

If $N_t \ge 0$ for every t, the horizon can be dropped: $\mathbb{E}[N_T \mathbf{1}[T < \infty]] \le \mathbb{E}[N_0]$ for every stopping time T.

Proof. The idea is to write the stopped process as a running total of the increments it collects, and to show that each has non-positive mean. Write $T \wedge t = \min(T, t)$.

1. *Write the stopped process as a sum.* The stopped process gains $N_t - N_{t-1}$ only if the run takes step t, which the indicator $\mathbf{1}[T \geq t]$ records:

$$N_{T\wedge t} = N_{T\wedge(t-1)} + \mathbf{1}[T \geq t](N_t - N_{t-1}). \tag{36.4}$$

2. *Each collected increment has non-positive mean.* The indicator is decided by $y_{<t}$, so given $y_{<t}$ it comes out of the conditional expectation, and what remains, $\mathbb{E}[N_t \mid y_{<t}] - N_{t-1}$, is non-positive. This is where the stopping decision must not look at y_t.
3. *Add up the steps.* Taking expectations and summing over $t \leq n$ gives $\mathbb{E}[N_{T\wedge n}] \leq \mathbb{E}[N_0]$, and $T \wedge n = T$.
4. *Drop the horizon.* For $N \geq 0$, apply step 3 to $T \wedge n$ and let $n \to \infty$. Fatou's lemma, which says that the mean of a limit of non-negative variables is at most the limit of their means, gives the second bound.

□

The horizon matters only for processes that can go negative. With both tools in hand, the result says: fix an order and a budget, add up the RDP spends as the run goes, and stop before the sum would pass the budget.

Theorem 36.1 (The Rényi filter (Feldman & Zrnic 2021; Lécuyer 2021))**.** *Fix an order $\alpha > 1$ and a budget B. Let step t be $(\alpha, \varepsilon_t(\alpha))$-RDP conditionally on $y_{<t}$, with $\varepsilon_t(\alpha)$ predictable. Run step t only if*

$$\sum_{j\leq t} \varepsilon_j(\alpha) \leq B. \tag{36.5}$$

Then the stopped run is (α, B)-RDP. Equivalently, if $\sum_t \varepsilon_t(\alpha) \leq B$ on every path, the adaptive composition is (α, B)-RDP.

Proof. The proof builds a supermartingale from the privacy loss and the spends, stops it with the filter, and reads the RDP bound off its mean. The quantity to use is $e^{(\alpha-1)L}$, whose mean is what the Rényi divergence measures, divided by what the spends allow. Fix $D \sim D'$ and define

$$N_t = \exp\Big((\alpha-1)\Big(L_{1:t} - \sum_{j\leq t} \varepsilon_j(\alpha)\Big)\Big), \qquad N_0 = 1. \tag{36.6}$$

1. *One step's moment is bounded by its spend.* Given $y_{<t}$, step t is a fixed mechanism, so the moment form of D_α and the RDP assumption give $\mathbb{E}[e^{(\alpha-1)L_t} \mid y_{<t}] = e^{(\alpha-1)D_\alpha(P_t\|Q_t)} \leq e^{(\alpha-1)\varepsilon_t(\alpha)}$.
2. *N_t is a supermartingale.* Each step multiplies N by $e^{(\alpha-1)(L_t - \varepsilon_t(\alpha))}$. Both N_{t-1} and $\varepsilon_t(\alpha)$ are functions of $y_{<t}$, so they come out of the conditional expectation, and step 1 bounds what remains by 1. This is where predictability is used.
3. *Stop it.* Whether step t runs depends on spends that are functions of $y_{<t}$, so the filter's last step T is a stopping time, and Lemma 36.2 gives $\mathbb{E}[N_T] \leq 1$.
4. *Use the budget on every path.* Undoing the definition, $e^{(\alpha-1)L_{1:T}} = N_T\, e^{(\alpha-1)\sum_{j\leq T}\varepsilon_j(\alpha)}$, and the filter keeps the spend at most B on every path. Taking expectations,

$$\mathbb{E}\big[e^{(\alpha-1)L_{1:T}}\big] \leq e^{(\alpha-1)B}\,\mathbb{E}[N_T] \leq e^{(\alpha-1)B}. \tag{36.7}$$

5. *Read off the divergence.* By Lemma 36.1, $L_{1:T}$ is the privacy loss of the stopped transcript, so the left side is $e^{(\alpha-1)D_\alpha}$ of the stopped run, and $D_\alpha \le B$. Swapping D and D' gives the other direction.

□

Step 4 needs the budget on every path, not on average, because paths with a large spend may also have a large N_T. When every path spends exactly B and each step's moment equals its RDP parameter, as for Gaussian steps, N is a martingale and the bound is attained. In Figure 36.2, adaptive Gaussian steps (shift 0.3 after a positive output, 0.6 otherwise, in units of the noise) are stopped by a filter with $\alpha = 2$ and $B = 1$. If the last step's shift is chosen so that it spends exactly what is left, 10^6 Monte Carlo paths give $D_2 = 1.0004$. If the filter simply stops, $D_2 = 0.9305$, because no path uses the whole budget.

The price is the order, which must be fixed before the run. You can track several orders and stop when any one is exhausted, but continuing while *some* order still fits is a choice made after seeing the spends, which Section 36.4 prices. At the end, Mironov's conversion (Equation 29.24) turns a DP target (ε_g, δ) into the budget $B = \varepsilon_g - \ln(1/\delta)/(\alpha - 1)$.

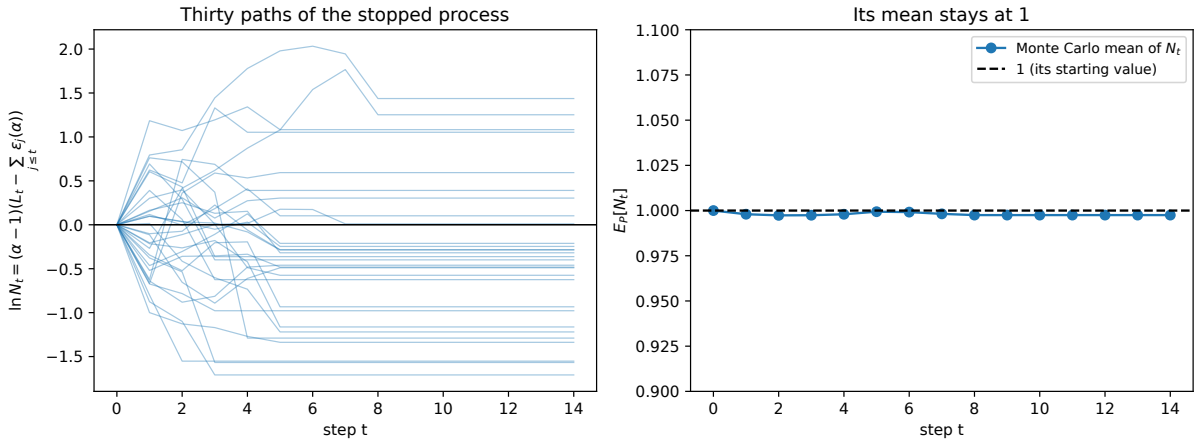


Figure 36.2.: Adaptive Gaussian steps under a Rényi filter with $\alpha = 2$ and $B = 1$. Left: $\ln N_t$ along thirty paths, each frozen when stopped. Right: the mean of N_t over 2×10^5 paths stays at 1.

Example 36.2. You train with full-batch noisy gradient descent and want to lower the noise when progress stalls. With $(\varepsilon_g, \delta) = (5.299, 10^{-5})$ and the order $\alpha = 5.8$, the RDP budget is $B = 5.299 - \ln(10^5)/4.8 = 2.900$. A Gaussian step with noise multiplier σ_t spends $\alpha/(2\sigma_t^2)$ (Equation 29.5). With $\sigma_t = 10$ throughout, each step spends $5.8/200 = 0.0290$, which gives $2.900/0.0290 = 100$ steps. If you switch to $\sigma_t = 5$ after a plateau at step 60, each later step costs $5.8/50 = 0.1160$, and the filter allows $(2.900 - 60 \times 0.0290)/0.1160 = 1.160/0.1160 = 10$ more. Either way the run is $(5.8, 2.900)$-RDP, whatever rule decided the switch.

The filter can also beat fixed-curve composition outright when branches trade costs. Two steps, as in Table 29.5 of Rényi DP, are too few to show it, but three are enough.

Example 36.3. Take $\alpha = 2$, so $N_t = e^{L_{1:t} - \text{spend}}$, and one person's bit, 1 in D and 0 in D'. Step 1 is randomized response with $\varepsilon = \ln 3$, keep-probability $p = 3/4$, and RDP at order 2 (Equation 29.7) $\ln(p^2/(1-p) + (1-p)^2/p) = 0.8473$. After $y_1 = 1$, steps 2 and 3 are randomized response with $\ln 3$ (strong) and then $\ln 1.5$ (weak, keep-probability 0.6, RDP 0.1542), and after $y_1 = 0$ they come in the other order. So every path spends $B = 2 \times 0.8473 + 0.1542 = 1.8487$ (computed before rounding), while fixed-curve composition charges each later step its worst branch, $3 \times 0.8473 = 2.5419$.

Table 36.3.: Three adaptive randomized-response steps at $\alpha = 2$, in which every path spends the same.

(y_1, y_2, y_3)	P	Q	$L_{1:3}$	spend $\sum_j \varepsilon_j(2)$	N_3	$P \times N_3$
(1, 1, 1)	0.3375	0.0250	+2.6027	1.8487	2.1254	0.7173
(1, 1, 0)	0.2250	0.0375	+1.7918	1.8487	0.9446	0.2125
(1, 0, 1)	0.1125	0.0750	+0.4055	1.8487	0.2362	0.0266
(1, 0, 0)	0.0750	0.1125	−0.4055	1.8487	0.1050	0.0079
(0, 1, 1)	0.1125	0.0750	+0.4055	1.8487	0.2362	0.0266
(0, 1, 0)	0.0375	0.2250	−1.7918	1.8487	0.0262	0.0010
(0, 0, 1)	0.0750	0.1125	−0.4055	1.8487	0.1050	0.0079
(0, 0, 0)	0.0250	0.3375	−2.6027	1.8487	0.0117	0.0003

In the first row of Table 36.3, $P = 0.75 \times 0.75 \times 0.6 = 0.3375$ and $Q = 0.25 \times 0.25 \times 0.4 = 0.0250$, so $N_3 = e^{2.6027 - 1.8487} = 2.1254$. The last column sums to 1.0000, because randomized response meets step 1 of the proof with equality. So the joint divergence $D_2 = \ln \sum P^2/Q = 1.8487$ is exactly B, well below the fixed-curve 2.5419.

36.3. Filters for pure differential privacy

You run an interactive system with a global budget (ε_g, δ), and each query comes with its own ε_t, chosen after seeing the earlier answers. Before each query you need a rule that says whether it may run. Summing the ε_t is valid however you stop, but the total grows like the number of steps k, and advanced composition, which grows like $\sqrt{k}$, cannot be applied ex post. This section gives filters that grow like $\sqrt{k}$. Its examples use pure-DP steps, since Gaussian steps are not pure DP.

Every filter below continues while one statistic of $\varepsilon_1, \ldots, \varepsilon_t$ is at most ε_g. The simplest statistic is the plain sum, which works because a pure-DP step bounds the loss of every single output.

Proposition 36.2 (The basic filter (Rogers et al. 2016, Theorem 3.4))**.** *Let each step be ε_t-DP conditionally on the past, with ε_t predictable. The filter that continues while $\sum_{j \le t} \varepsilon_j \le \varepsilon_g$ gives an ε_g-DP stopped run, and $U_t = \sum_{j \le t} \varepsilon_j$ is a valid odometer with $\delta = 0$.*

Proof.

1. *Each step's loss is bounded on every output.* An ε_t-DP step makes no output more than e^{ε_t} times likelier on one neighbour than on the other, so $|L_t| \le \varepsilon_t$.

2. *So is the stopped run's loss.* By Lemma 36.1, $|L_{1:T}| \le \sum_{t \le T} \varepsilon_t \le \varepsilon_g$ on every path, and the same bound holds at every t, which is the odometer.
3. *A bounded loss is pure DP.* A loss of at most ε_g on every transcript y means $P(y) \le e^{\varepsilon_g} Q(y)$, and summing over the transcripts in any event E gives $P(E) \le e^{\varepsilon_g} Q(E)$.

□

To grow like $\sqrt{k}$, the statistic needs a mean term plus a spread term like advanced composition's $\sqrt{2V \ln(1/\delta)}$, where $V = \sum_t \varepsilon_t^2$ plays the role of a variance. Because V is random, Rogers et al. pad it by a constant x and multiply it by a logarithmic factor in V/x.

Theorem 36.2 (The Rogers et al. filter (2016, Theorem 5.1))**.** *Under the assumptions of Proposition 36.2, let $\delta < 1/e$ and $x = \varepsilon_g^2/(28.04 \ln(1/\delta))$. The filter that continues while*

$$K = \sum_t \frac{\varepsilon_t(e^{\varepsilon_t} - 1)}{2} + \sqrt{2(V + x)\Big(1 + \tfrac{1}{2}\ln\big(\tfrac{V}{x} + 1\big)\Big)\ln(1/\delta)} \le \varepsilon_g \tag{36.8}$$

gives an (ε_g, δ)-DP stopped run.

The proof is in Rogers et al.'s Section 5. Its centred loss has random increment ranges $2\varepsilon_t$, so Azuma's inequality no longer applies, and they use a **self-normalized** inequality instead (their Theorem 4.4), which bounds the martingale divided by $\sqrt{V + x}$ whatever V turns out to be. The extra x and the logarithmic factor are its price, while the constant 28.04 and the condition $\delta < 1/e$ are constants of their proof.

Zero-concentrated differential privacy (zCDP) avoids that price. A mechanism is ρ-zCDP if it is $(\alpha, \alpha\rho)$-RDP at every order $\alpha > 1$, so its RDP curve lies below a line through the origin with slope ρ. In zCDP, adaptive budgets add up for free at every order by Theorem 36.1, and the conversion to (ε, δ)-DP happens once, at the end.

Theorem 36.3 (The Whitehouse et al. filter (2023, Theorem 2))**.** *Under the same assumptions, let $\delta \in (0, 1)$. The filter that continues while*

$$\sqrt{2\ln(1/\delta)\textstyle\sum_t \varepsilon_t^2} + \tfrac{1}{2}\textstyle\sum_t \varepsilon_t^2 \le \varepsilon_g \tag{36.9}$$

gives an (ε_g, δ)-DP stopped run. Equivalently, it continues while $\frac{1}{2}\sum_t \varepsilon_t^2 \le \rho^*$, where

$$\rho^* = \Big(\sqrt{\ln(1/\delta) + \varepsilon_g} - \sqrt{\ln(1/\delta)}\Big)^2. \tag{36.10}$$

Proof. Measure each step in zCDP, apply the Rényi filter at every order at once, then convert. Write $\ell = \ln(1/\delta)$.

1. *Each step is zCDP.* An ε_t-DP step is $(\alpha, \alpha\varepsilon_t^2/2)$-RDP for every $\alpha > 1$ (Equation 31.7), because its loss lies in $[-\varepsilon_t, \varepsilon_t]$. So it is $\varepsilon_t^2/2$-zCDP.
2. *The rule is a Rényi filter at every order.* Multiplying $\frac{1}{2}\sum_t \varepsilon_t^2 \le \rho^*$ by α gives $\sum_t \alpha\varepsilon_t^2/2 \le \alpha\rho^*$, a Rényi filter with budget $\alpha\rho^*$. By Theorem 36.1 the stopped run is $(\alpha, \alpha\rho^*)$-RDP. The rule does not mention α, so this holds at every order, and the stopped run is ρ^*-zCDP.

3. *Convert.* A ρ-zCDP mechanism is $(\rho + 2\sqrt{\rho\ell}, \delta)$-DP (Equation 31.15), Mironov's conversion at the best order.
4. *The choice of ρ^* makes this ε_g.* The equation $\rho + 2\sqrt{\rho\ell} = \varepsilon_g$ completes to $(\sqrt{\rho} + \sqrt{\ell})^2 = \varepsilon_g + \ell$, whose solution is the ρ^* of the statement (Equation 31.18).
5. *Rewrite the rule.* The map $\rho \mapsto \rho + 2\sqrt{\rho\ell}$ is increasing, so $\rho \le \rho^*$ is the same as $\rho + 2\sqrt{\rho\ell} \le \varepsilon_g$. With $\rho = \frac{1}{2}\sum_t \varepsilon_t^2$, the square root is $\sqrt{2\ell \sum_t \varepsilon_t^2}$, and the rule becomes Equation 36.9.

□

This is the budget-spending rule that Section 21.5 asked for, and adaptivity costs it essentially nothing. Its mean term $\varepsilon_t^2/2$ lies slightly above the sharp mean $\varepsilon_t \tanh(\varepsilon_t/2)$ of Equation 21.17 but below the $\varepsilon_t(e^{\varepsilon_t} - 1)$ of Equation 21.9, which is why Table 36.4 puts it below fixed-k advanced composition. For the same reason, a plug-in filter on Equation 21.9 is valid: it halts no later than the Whitehouse filter, and stopping earlier is post-processing.

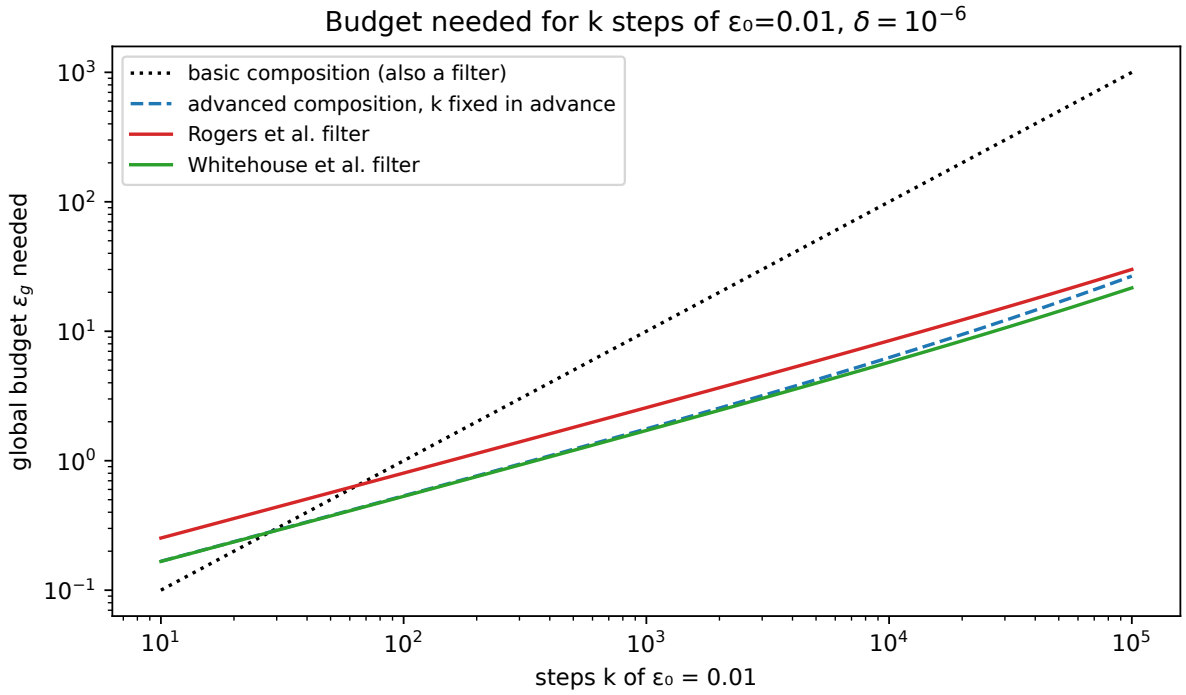


Figure 36.3.: Global budget needed to run k steps of $\varepsilon_0 = 0.01$ at $\delta = 10^{-6}$, on log–log axes, for the four rules of Table 36.4.

Example 36.4. Take k steps of ε_0 each at $\delta = 10^{-6}$. The budget each rule needs is its statistic after k steps, compared in Table 36.4 with advanced composition for a k fixed in advance (Equation 21.9) and plotted in Figure 36.3. The Rogers filter needs 1.40 to 1.51 times the Whitehouse budget.

Table 36.4.: Global budget ε_g needed for k steps of ε_0 at $\delta = 10^{-6}$. For the Rogers filter, the smallest ε_g at which Equation 36.8 holds.

ε_0	k	Basic	Advanced (fixed k)	Rogers filter	Whitehouse filter	Rogers / Whitehouse
0.01	100	1.000	0.536	0.803	0.531	1.51
0.01	1000	10.000	1.763	2.570	1.712	1.50
0.01	10,000	100.000	6.262	8.450	5.757	1.47
0.1	100	10.000	6.308	8.472	5.757	1.47
0.1	1000	100.000	27.140	30.244	21.623	1.40

Turned around, a budget of $\varepsilon_g = 1$ allows the numbers of steps in Table 36.5. For the Whitehouse filter, $\rho^* = (\sqrt{\ln(10^6) + 1} - \sqrt{\ln(10^6)})^2 = 0.017469$, which allows $\lfloor 2\rho^*/\varepsilon_0^2 \rfloor$ steps, and the Rogers filter lets through fewer than half as many.

Table 36.5.: Number of steps of ε_0 allowed by a global budget $\varepsilon_g = 1$ at $\delta = 10^{-6}$.

ε_0	Basic	Advanced (fixed k)	Rogers filter	Whitehouse filter
0.01	100	337	154	349
0.001	1000	33,785	15,481	34,937

36.4. Odometers

Early stopping and open-ended analyses have no budget to stop at. They need a running answer to "how much have I spent?" that stays valid however the stopping time is chosen, the odometer of Definition 36.2. This section builds odometers from the Rényi filter's supermartingale, at the cost of a union over the orders you might pick.

The tool is a time-uniform Markov inequality. Markov's inequality (Theorem 2.1) says that a non-negative variable with mean 1 is at least c with probability at most $1/c$. Ville's inequality gives the same bound for the event that a supermartingale is *ever* that large, because stopping at the first time it reaches c leaves the mean at most 1.

Lemma 36.3 (Ville's inequality)**.** *If $N_t \geq 0$ is a supermartingale with $N_0 = 1$, then for every $c > 0$,*

$$\Pr\left[\exists\, t : N_t \geq c\right] \leq \frac{1}{c}. \tag{36.11}$$

Proof.

1. *Stop at the first crossing.* Fix a horizon n and let T be the first t with $N_t \geq c$, or n if there is none. Whether $T \geq t$ depends only on $N_1, \ldots, N_{t-1}$, so T is a stopping time bounded by n.
2. *The stopped process keeps its mean.* By Lemma 36.2, $\mathbb{E}[N_T] \leq 1$.

3. *Apply Markov's argument to the stopped value.* If the process reaches c by time n, then $N_T \geq c$, and elsewhere $N_T \geq 0$. So N_T is at least c times the indicator of that event, and taking means gives $c \Pr[\exists\, t \leq n : N_t \geq c] \leq 1$.
4. *Drop the horizon.* These events grow with n, and their union is $\{\exists\, t : N_t \geq c\}$.

□

Each odometer below turns the running RDP spend at a few orders into a running ε. The orders form a finite grid Λ fixed in advance, with $|\Lambda|$ orders, and $S_t(\alpha) = \sum_{j \leq t} \varepsilon_j(\alpha)$ is the spend so far. The first odometer reports Mironov's conversion at the best order in the grid, with δ replaced by $\delta/|\Lambda|$ to pay for choosing the order late.

Theorem 36.4 (The Ville odometer)**.** *Let step t be $(\alpha, \varepsilon_t(\alpha))$-RDP conditionally on the past for each $\alpha \in \Lambda$, with predictable spends. With probability at least $1 - \delta$, simultaneously for all t,*

$$L_{1:t} \leq U_t = \min_{\alpha \in \Lambda} \Big[S_t(\alpha) + \frac{\ln(|\Lambda|/\delta)}{\alpha - 1} \Big]. \tag{36.12}$$

Proof. Ville's inequality, applied to the Rényi filter's supermartingale at each order, unpacks into a bound on the loss, and a union bound pays for all the orders.

1. *A supermartingale at each order.* Fix $\alpha \in \Lambda$. Steps 1 and 2 of the proof of Theorem 36.1 showed that N_t of Equation 36.6 is a non-negative supermartingale with $N_0 = 1$, using only that the spends are predictable.
2. *Ville.* Lemma 36.3 with $c = |\Lambda|/\delta$ gives

$$\Pr\left[\exists\, t : N_t \geq |\Lambda|/\delta\right] \leq \delta/|\Lambda|. \tag{36.13}$$

3. *Unpack.* Taking logarithms and dividing by $\alpha - 1$ turns $N_t \geq |\Lambda|/\delta$ into $L_{1:t} \geq S_t(\alpha) + \ln(|\Lambda|/\delta)/(\alpha - 1)$. So at this order the bound holds for all t at once, except with probability $\delta/|\Lambda|$.
4. *Union over the orders.* By the union bound (Lemma 2.3), the bound fails at some order with probability at most δ. Outside that event it holds at every order and time, so it holds at the minimizing order, chosen after the fact.

□

With a single order the Ville odometer is Mironov's fixed-time conversion, so at one fixed order a running bound is free. For Gaussian steps it is then exactly tight. In the limit of small steps the loss is a Brownian motion with drift, which crosses the boundary with probability exactly δ: N is then a martingale with continuous paths, so it equals $1/\delta$ at the crossing, and the inequalities in the proof of Lemma 36.3 become equalities. The script checks this at $\delta = 0.05$ and order 2, with 1000 Gaussian steps whose mean shifts by 0.2 noise widths. The crossing probability is 0.0461, a little below 0.0500 because whole steps miss crossings between them, while at any single fixed time it is at most 0.0072. Several orders add the union's $\ln|\Lambda|$. At $\delta = 10^{-5}$, with the grid of 154 orders that this part's accountants use, $\ln(1/\delta) = 11.513$ becomes $11.513 + \ln 154 = 16.550$.

An odometer is a tail bound, not a DP guarantee: "$\varepsilon = U_T$" means that this run's loss exceeds U_T with probability at most δ. A filter that stops before U_T passes ε_g is (ε_g, δ)-DP, by splitting any event on whether the loss exceeds ε_g (Section 29.6).

Lécuyer (2021, Section 4) builds a second odometer from Rényi filters with doubling budgets $B_f(\alpha) = 2^{f-1}\ln(2|\Lambda|/\delta)/(\alpha-1)$, for $f = 1, 2, \ldots$ Let $f(\alpha)$ be the first f whose budget covers $S_t(\alpha)$. The odometer reports that budget plus a conversion term whose confidence shrinks with f.

Theorem 36.5 (The Lécuyer odometer)**.** *Under the assumptions of Theorem 36.4, with probability at least $1-\delta$, simultaneously for all t,*

$$L_{1:t} \le \min_{\alpha\in\Lambda}\Big[B_{f(\alpha)}(\alpha) + \frac{\ln(2|\Lambda|f(\alpha)^2/\delta)}{\alpha-1}\Big]. \tag{36.14}$$

Proof.

1. *One filter at one order.* Fix α and f, and let T_f be the first step at which $S_t(\alpha)$ exceeds $B_f(\alpha)$. The supermartingale N_t stopped at T_f is still non-negative and starts at 1. So Lemma 36.3 with $c = 2|\Lambda|f^2/\delta$, unpacked as in the previous proof, bounds the loss by $B_f(\alpha) + \ln(2|\Lambda|f^2/\delta)/(\alpha-1)$ at every t before T_f, except with probability $\delta/(2|\Lambda|f^2)$.
2. *Union over filters and orders.* The failure probabilities add up to $\delta\sum_f 1/(2f^2) = \delta\pi^2/12 < \delta$.
3. *Pick the filter.* At time t, filter $f(\alpha)$ has not run out, so t comes before $T_{f(\alpha)}$, and step 1's bound holds at every order. Minimizing over the orders gives Equation 36.14.

□

Because the budgets double, the certified budget is at most twice the spend once the spend exceeds B_1. For pure-DP steps, Rogers et al. (2016, Theorem 6.5) and Whitehouse et al. (2023, Theorem 3) give advanced-composition odometers.

Example 36.5. Take the running example's steps with $\sigma = 10$, so $S_t(\alpha) = \alpha t/(2\sigma^2)$, at $\delta = 10^{-5}$ with the part's 154 orders. Table 36.6 and Figure 36.4 compare the fixed-time conversion with the two odometers and with a Ville odometer at the single order 5.8, chosen for $t = 100$. That last one spends 0.029 per step plus $\ln(10^5)/4.8 = 2.399$, so it is $0.29 + 2.399 = 2.689$ at $t = 10$ and $29 + 2.399 = 31.399$ at $t = 1000$.

Table 36.6.: Running ε at $\delta = 10^{-5}$ for t full-batch Gaussian steps with $\sigma = 10$. Only the last three columns are valid when t is chosen from the outputs.

Steps t	Fixed-time conversion	Ville odometer (154 orders)	Lécuyer odometer	Ville at $\alpha = 5.8$ only
10	1.568	1.869	1.916	2.689
100	5.299	6.253	6.507	5.299
300	9.812	11.465	11.892	11.099
1000	20.175	23.194	24.633	31.399

The Ville odometer costs 15–19% over the fixed-time value, 6.253 against 5.299 at $t = 100$. The single-order odometer is free at its design point and poor far from it. For the part's subsampled example ($q = 0.01$, $\sigma = 1$, 10^4 steps, $\delta = 10^{-5}$, with the curve of Amplification by subsampling), the fixed-time conversion gives 7.43, and an odometer valid whenever the run stops costs 8.82 (Ville) or 9.08 (Lécuyer).

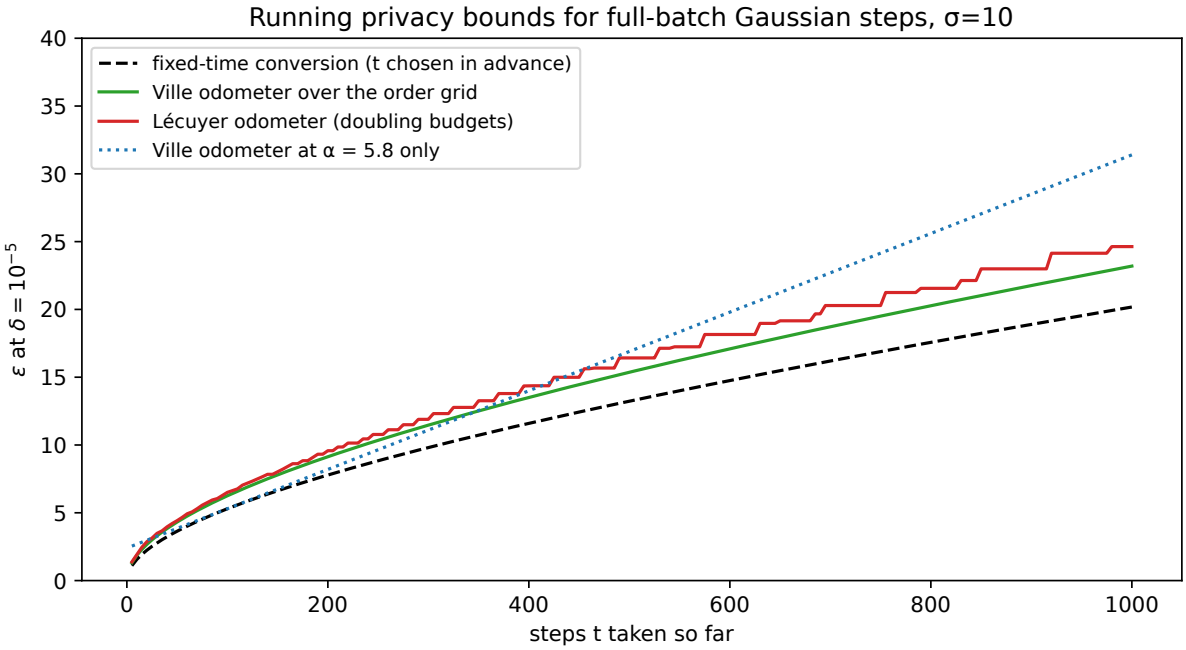


Figure 36.4.: Running privacy bounds at $\delta = 10^{-5}$ for full-batch Gaussian steps with $\sigma = 10$, for the four columns of Table 36.6; the dashed fixed-time conversion is not a valid odometer.

36.5. Individual privacy accounting in DP-SGD

A step of DP-SGD, the private form of SGD, **clips** each record's gradient to norm at most C, adds the clipped gradients and releases the sum with Gaussian noise. The usual analysis charges every record as if its clipped gradient always had norm C, but once the model fits a record, its gradient shrinks. Feldman & Zrnic (2021) observed that a record's actual cost is random and predictable, so the Rényi filter applies record by record. This section does the full-batch case, where every number is exact. A record's cost depends on how far it moves the released sum.

Proposition 36.3 (Individual RDP of a Gaussian step (Feldman & Zrnic 2021, Section 2))**.** *At step t, let the mechanism release $\sum_i \bar{g}_{i,t} + \mathcal{N}(0, \sigma^2 C^2 I)$, where $\bar{g}_{i,t}$ is record i's gradient at the current model, clipped to norm C. For the datasets with and without record i, the step's RDP at order α is*

$$\varepsilon_{i,t}(\alpha) = \frac{\alpha \|\bar{g}_{i,t}\|^2}{2\sigma^2 C^2} \le \frac{\alpha}{2\sigma^2}. \tag{36.15}$$

Proof.

1. *Reduce to one dimension.* With record i the output is $\mathcal{N}(m + \bar{g}_{i,t}, \sigma^2 C^2 I)$, and without it $\mathcal{N}(m, \sigma^2 C^2 I)$, where m sums the other clipped gradients. Rotate so that $\bar{g}_{i,t}$ lies along one axis. The noise is the same in every direction, so the other coordinates contribute nothing.
2. *Use the Gaussian divergence.* What remains is two normals with standard deviation σC, a shift of $\|\bar{g}_{i,t}\|/(\sigma C)$ in units of the noise. Two normals with shift μ in units of the noise have $D_\alpha = \alpha\mu^2/2$ in both directions (Equation 3.19), which gives the equality.

3. *Clip.* Clipping makes $\|\bar{g}_{i,t}\| \le C$, which gives the bound.

□

The filter then runs separately for each record: a record takes part only while its own spend fits in the budget.

Theorem 36.6 (The individual filter (Feldman & Zrnic 2021, Section 4.1))**.** *Fix α and B. At step t, let record i take part in the gradient sum only if $\sum_{j\le t} \varepsilon_{i,j}(\alpha) \le B$, counting only the steps it took part in. Then the whole run is (α, B)-RDP.*

Proof. It is enough to check that a record's spend is predictable and to apply the Rényi filter to each pair of neighbours.

1. *The spend is predictable.* The clipped gradient $\bar{g}_{i,t}$ is a function of record i and the current model, which is a function of $y_{<t}$. So by Proposition 36.3, $\varepsilon_{i,t}(\alpha)$ is the RDP of step t for the pair with and without record i, given $y_{<t}$. It must not depend on the other records, or removing record i would change which of them take part.
2. *Dropping a record costs it nothing.* Whether record i takes part depends only on its own past spends. If it is dropped, the outputs with and without it have the same law at that step, and its spend is 0.
3. *Apply the Rényi filter per record.* For the pair with and without record i, the spends are predictable and never sum past B, so Theorem 36.1 makes the transcript (α, B)-RDP for this pair. Every pair of neighbours differs in one record.

□

The filter gives the same guarantee with more training. Since no spend exceeds $\alpha/(2\sigma^2)$, every record survives at least as long as the worst-case run, and records whose gradients shrink keep taking part long after. A per-record ε computed from the realized spend is only an ex post estimate, for the reasons of Section 36.1, and it is sensitive, since a small spend says the record is well fitted. For Poisson-subsampled DP-SGD the argument is unchanged, with the subsampled-Gaussian curve of Amplification by subsampling.

There is a fairness cost. Records that the model fits poorly, often from under-represented groups, keep large gradients, spend their budget first and are dropped first, so the model sees even less of them. Yu et al. (2023) find that the classes a model serves worst also get the weakest individual guarantees.

Example 36.6. The simulation has 10,000 records, clip $C = 1$, $\sigma = 10$ and full-batch steps. **Typical records** (90%) have gradient norms $\|g_{i,t}\| = s_i e^{-t/40} e^{0.25 Z_{i,t}}$, with s_i uniform between 0.3 and 2 and $Z_{i,t}$ standard normal, so they shrink as the model fits them. **Hard records** (10%; 962 in this draw) have $\|g_{i,t}\| = s_i e^{0.25 Z_{i,t}}$ with s_i uniform between 0.7 and 1.5, and never shrink. The worst-case run is the running example: 100 steps, $\varepsilon = 5.299$ at $\delta = 10^{-5}$, order $\alpha^* = 5.8$, budget $B = 2.900$ and a per-step spend of at most 0.0290.

After the same 100 steps, record i's ε at order 5.8 is its own spend plus $\ln(10^5)/4.8$, a floor of 2.399 that no record can go below. Table 36.7 and Figure 36.5 show the distribution.

Table 36.7.: Individual ε at $\delta = 10^{-5}$ after 100 steps, at order 5.8 and, in the last column, at each record's best order in the part's grid.

Records	10th percentile	Median	90th percentile	Median, order chosen per record
All	2.56	3.25	3.81	2.75
Typical	2.55	3.17	3.67	2.61
Hard	4.23	5.02	5.26	5.01

The typical median is 3.17, against 5.299 for the worst case, and 16.3% of all records are below half the worst case. Hard records sit just under it, 5.02 at the median. Optimizing the order per record lowers the typical median to 2.61, because a small spend is best converted at a larger order, where the floor is smaller. All these values are ex post estimates, and the filter certifies only 5.299.

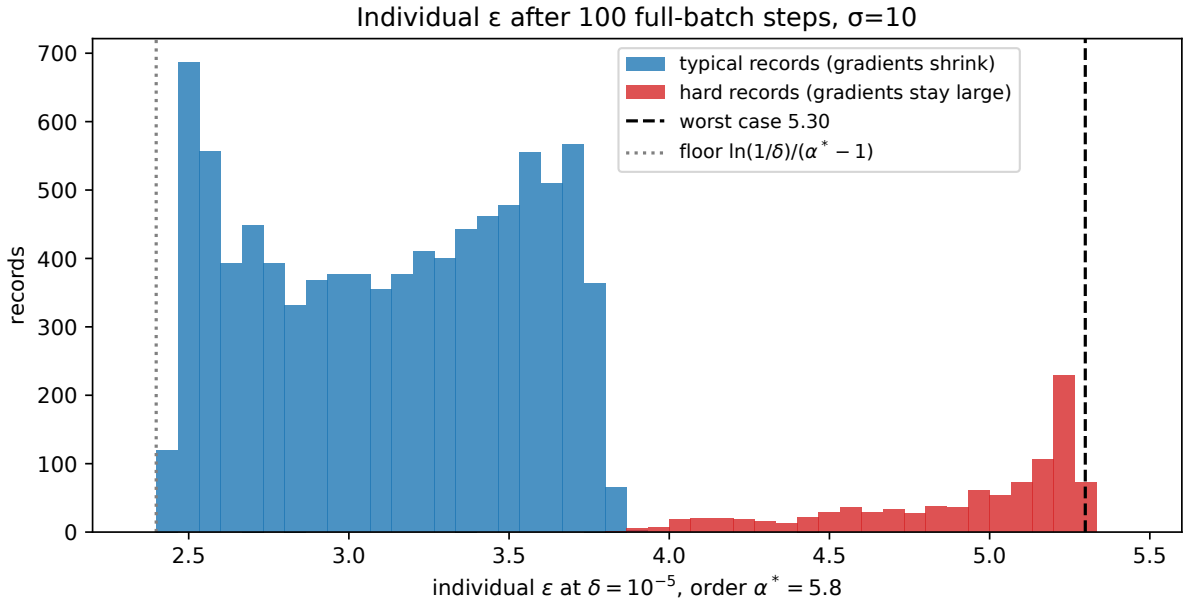


Figure 36.5.: Individual ε at $\delta = 10^{-5}$ and order $\alpha^* = 5.8$ after 100 full-batch steps with $\sigma = 10$. Typical records in blue, hard records in red, the worst case dashed and the floor $\ln(1/\delta)/(\alpha^* - 1)$ dotted.

Now run the individual filter for 300 steps with $B = 2.900$ at order 5.8. Everyone takes part in the first 100 steps, since $100 \times 0.0290 = 2.900$. Then hard records run out: the first is excluded at step 101, the median one at step 111, and by step 200 none takes part (0.1% are back at step 300, when a small gradient fits what was left). No typical record is ever excluded, and at step 300 their median spend is 0.778, about a quarter of the budget. Every record has the same $(5.8, 2.900)$-RDP guarantee as the 100-step worst-case run (Figure 36.6).

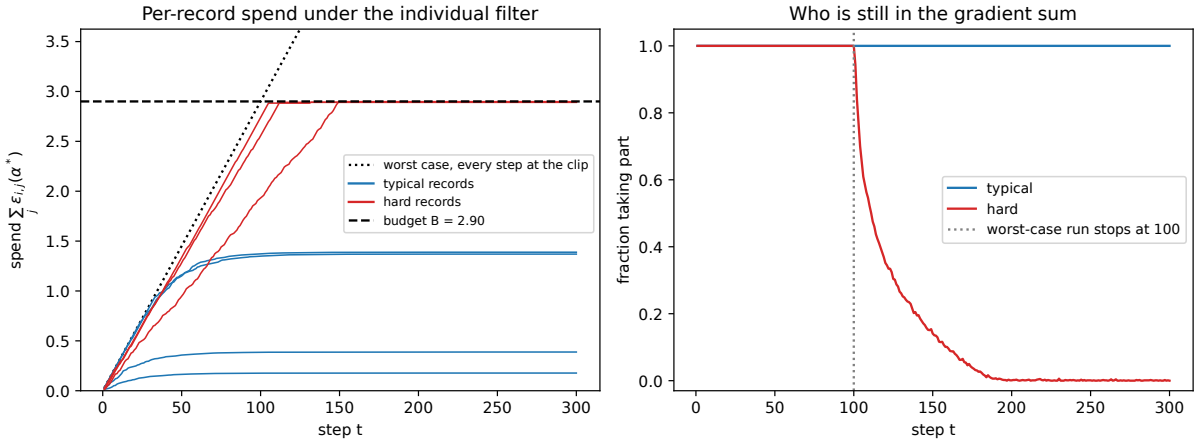


Figure 36.6.: Left: cumulative spend at order 5.8 under the individual filter, for four typical records (blue) and three hard records (red), against the worst case (dotted) and the budget $B = 2.90$ (dashed). Right: the fraction of each group taking part at each step.

36.6. Code

The script code/adaptive_composition.py computes every number in this chapter and draws every figure, with the plug-in failure exact by dynamic programming and the Rényi filter checked by Monte Carlo. From the repository root:

```
uv run python code/part04-privacy-accounting/adaptive_composition.py
```

Table 36.8.: The script's output with the default seed.

Experiment	Result
Plug-in advanced composition, randomized response $\varepsilon = 0.1$	Crossing by $n = 10^4$: 0.1424 at $\delta = 0.05$, 0.0318 at $\delta = 0.01$, 3.69×10^{-5} at $\delta = 10^{-5}$; at a single fixed $t \le 1000$ at most 0.0110, 0.0016 and 8.18×10^{-7}
Budget for $k = 1000$ steps of 0.01, $\delta = 10^{-6}$	Basic 10.000; advanced (fixed k) 1.763; Rogers filter 2.570; Whitehouse filter 1.712
Steps allowed at $\varepsilon_g = 1$, $\varepsilon_0 = 0.01$	Basic 100; advanced 337; Rogers 154; Whitehouse 349
Three-step randomized response, $\alpha = 2$	$\mathbb{E}[N_3] = 1.0000$; joint $D_2 = 1.8487 =$ budget; fixed-curve bound 2.5419
Adaptive Gaussian filter, $\alpha = 2$, $B = 1$, 10^6 paths	Last step spends the rest: $D_2 = 1.0004$; plain stop: $D_2 = 0.9305$
Odometers, $\sigma = 10$, $t = 100$, $\delta = 10^{-5}$	Fixed time 5.299; Ville 6.253; Lécuyer 6.507
Odometers, running example ($q = 0.01$, $\sigma = 1$, 10^4 steps)	Fixed time 7.43; Ville 8.82; Lécuyer 9.08
Ville crossing, Gaussian steps, $\delta = 0.05$	Monte Carlo 0.0461; Brownian limit 0.0500

Experiment	Result
Individual ε, 100 steps, $\sigma = 10$	Worst case 5.299; median typical 3.17, hard 5.02
Individual filter, 300 steps	Hard records excluded from step 101 (median 111); typical never; typical median spend 0.778 of 2.900

tests/test_adaptive_composition.py checks that the output still matches.

36.7. Summary

Table 36.9.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 36.1, Definition 36.2	A filter halts within a fixed budget; an odometer bounds the loss at all times	A filter if you have a budget, an odometer if not
Proposition 36.1	Advanced composition of realized parameters is eventually crossed	Never report a fixed-time bound at a data-dependent time
Lemma 36.2	A stopped supermartingale keeps its mean at most its start	Stopping cannot manufacture privacy loss
Theorem 36.1	$S_T(\alpha) \le B$ on every path gives (α, B)-RDP	Adaptive noise or stopping; fix the order before the run
Proposition 36.2	$\sum_t \varepsilon_t \le \varepsilon_g$ gives ε_g-DP	Few pure-DP queries; also an odometer
Theorem 36.2	Equation 36.8 $\le \varepsilon_g$ gives (ε_g, δ)-DP	1.40 to 1.51 times the Whitehouse budget
Theorem 36.3	$\sqrt{2\ln(1/\delta)V} + V/2 \le \varepsilon_g$ gives (ε_g, δ)-DP	Adaptive pure-DP queries at the cost of advanced composition
Lemma 36.3	$\Pr[\exists\, t : N_t \ge c] \le 1/c$	Markov's inequality for the whole path
Theorem 36.4	$U_t = \min_{\alpha\in\Lambda}[S_t(\alpha) + \ln(\lvert\Lambda\rvert/\delta)/(\alpha - 1)]$	Early stopping; 15–19% over fixed time here; keep the grid small
Theorem 36.5	Doubling Rényi filters, Equation 36.14	An alternative odometer
Proposition 36.3	$\varepsilon_{i,t}(\alpha) = \alpha\lVert\bar{g}_{i,t}\rVert^2/(2\sigma^2 C^2)$	A record's own cost in a clipped Gaussian sum

Result	Statement	Use
Theorem 36.6	Dropping a record once its spend reaches B gives (α, B)-RDP	Well-fitted records train longer; the worst-served are dropped first

36.8. References

- Rogers, Roth, Ullman & Vadhan, "Privacy Odometers and Filters: Pay-as-you-Go Composition" (NeurIPS 2016; arXiv:1605.08294). Defines filters and odometers, proves basic composition works for both (Theorem 3.4), gives the first advanced-composition filter (Theorem 5.1), the lower bound separating odometers from filters (Theorem 6.1) and an odometer (Theorem 6.5). Read Sections 1 and 3 for the model.
- Whitehouse, Ramdas, Rogers & Wu, "Fully-Adaptive Composition in Differential Privacy" (ICML 2023; arXiv:2203.05481). Filters that match advanced composition including constants (Theorems 1–2, through approximate zCDP), and odometers from time-uniform martingale concentration (Theorem 3). The place to look for steps with $\delta_t > 0$.
- Feldman & Zrnic, "Individual Privacy Accounting via a Rényi Filter" (NeurIPS 2021; arXiv:2008.11193). The fully adaptive RDP composition theorem and the Rényi filter (Sections 3–4), individual RDP and the individual filter, and private gradient descent with filtering (Section 5).
- Lécuyer, "Practical Privacy Filters and Odometers with Rényi Differential Privacy and Applications to Differentially Private Deep Learning" (2021, arXiv:2103.01379). An independent proof of the Rényi filter, the filter over a set of orders, and the RDP odometer with doubling budgets; experiments with adaptive training.
- Koskela, Tobaben & Honkela, "Individual Privacy Accounting with Gaussian Differential Privacy" (ICLR 2023). Individual filtering with Gaussian differential privacy and privacy loss distributions, tighter than RDP for the Gaussian mechanism.
- Yu, Kamath, Kulkarni, Liu, Yin & Zhang, "Individual Privacy Accounting for Differentially Private Stochastic Gradient Descent" (TMLR 2023). Output-specific per-example guarantees for DP-SGD, and the link between a record's training loss, its privacy and group fairness.
- Dwork, Rothblum & Vadhan, "Boosting and Differential Privacy" (FOCS 2010). Advanced composition with parameters fixed in advance, the theorem this chapter makes adaptive.
- Howard, Ramdas, McAuliffe & Sekhon, "Time-uniform Chernoff Bounds via Nonnegative Supermartingales" (Probability Surveys 2020). Ville's inequality and the time-uniform bounds behind the Whitehouse et al. odometers.
- Williams, *Probability with Martingales* (1991), chs. 10–11. Optional stopping (ch. 10) and martingale convergence (ch. 11), with proofs.
- Durrett, *Probability: Theory and Examples* (5th ed., Cambridge University Press, 2019), the chapter on Brownian motion. The law of the iterated logarithm used in Section 36.1, and Brownian hitting probabilities.

37. Choosing an accountant

You have trained a model with differential privacy (DP). The training loop was stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, called DP-SGD. At each step every record joins the batch independently with probability q (Poisson sampling), each record's gradient is clipped to norm at most C, and Gaussian noise of standard deviation σC is added to the sum. The **noise multiplier** σ thus measures the noise against one record's largest contribution. Your run used $q = 0.01$, $\sigma = 1$ and $T = 10{,}000$ steps, and you must now write "the model is $(\varepsilon, 10^{-5})$-DP" in a paper or a model card. That promises that for any two datasets $D \sim D'$ that differ by adding or removing one record, any event about what training released is at most e^{ε} times as likely on one as on the other, plus $\delta = 10^{-5}$.

The run is fixed, so its true ε is one number. Nobody can compute it exactly, and a procedure that bounds or estimates it is called an **accountant**. This part has built several, and they disagree. Composing per-step guarantees gives 11296.93, while Rényi differential privacy (RDP) yields 7.4287 or 6.7127, depending on how its curve is converted. The Gaussian differential privacy (GDP) central limit theorem (CLT) says 6.0071, and a privacy loss distribution (PLD) accountant brackets the truth in $[6.1377, 6.2377]$. Which one do you report, and what else must you say so that someone else can check it?

The numbers differ because the accountants make different kinds of statement. Some are upper bounds, proved for every pair of neighbouring datasets, and one is a proved lower bound, while others are approximations that may land below the truth. So the rule is short. **Report the tightest upper bound whose assumptions hold for the code you ran, and enough for someone else to recompute it.** The sections take its clauses in turn, and the chapter's script imports every accountant from the chapter of this part that derived it.

Table 37.1.: The symbols of this chapter.

Symbol	Meaning
q, σ, C, T, n	sampling rate, noise multiplier, clipping norm, steps, records
α, $\varepsilon(\alpha)$	Rényi order; the RDP curve of one step
μ; L; h	GDP parameter; privacy loss; grid width of a PLD
$\varepsilon^{\uparrow}_{\mathrm{PLD}}$, $\varepsilon^{\downarrow}_{\mathrm{PLD}}$	ε of the pessimistic and the optimistic PLD
$\varepsilon_{\mathrm{true}}$	the smallest ε for which the run is (ε, δ)-DP

37.1. Every accountant on one run

This is the question the part has built up to: on the running example, what does each accountant report, and which of its numbers can you trust? Earlier chapters computed some of the numbers. This section puts them in one table, adds the optimal conversion of an RDP curve, and sorts the accountants by the kind of statement each makes.

All accountants start from what one step reveals about one record. The other records contribute the same on both datasets, so look only at the noisy sum along that record's clipped gradient, in units of C. Without the record, the output is pure noise, $\mathcal{N}(0, \sigma^2)$. With it, the record joins the batch with probability q and then shifts the output by at most 1, and the worst case is exactly 1, so the output is the mixture $(1-q)\mathcal{N}(0,\sigma^2) + q\mathcal{N}(1,\sigma^2)$. The accountants differ in what they keep of this pair and in how they combine T copies of it.

Definition 37.1 (Upper bound, lower bound and approximation)**.** An **accountant** maps the parameters of a run and a target δ to a number ε_A. Write $\varepsilon_{\text{true}}$ for the smallest ε for which the run is (ε, δ)-DP. The accountant is an **upper bound** if $\varepsilon_A \geq \varepsilon_{\text{true}}$ for every run that satisfies its assumptions, and a **lower bound** if $\varepsilon_A \leq \varepsilon_{\text{true}}$ for every such run. It is an **approximation** if neither is proved.

A lower bound is not a privacy guarantee, but it tells you how far any valid upper bound can fall. The six accountants of the part are these.

1. **Per step, then composition** (Amplification by subsampling). One Gaussian step is $(\varepsilon_0, \delta_0)$-DP. Since the record is absent with probability $1-q$, subsampling dilutes e^{ε_0} to $1 + q(e^{\varepsilon_0} - 1)$, giving ε_1 (Equation 33.8). Then T steps compose, by basic composition to $T\varepsilon_1$, or by advanced composition, which grows like $\sqrt{T}$ at the price of an extra slack δ'.
2. **RDP, Mironov's conversion** (Rényi DP). RDP describes one step by the curve $\varepsilon(\alpha)$, the Rényi divergence of order $\alpha > 1$ between the two outputs (Equation 33.17). Curves add under composition, and Markov's inequality turns the composed curve into (ε, δ) at the best order:
$$\varepsilon = \min_{\alpha} \Big[T\,\varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1} \Big]. \tag{37.1}$$
3. **RDP, improved conversion** (Equation 30.8 in From Rényi DP to (ε, δ) and to tests). A sharper analysis of the same tail (Balle et al. 2020) saves, at each order, an amount that does not depend on the curve.
4. **RDP, optimal conversion** (the same chapter). It gives the smallest ε that holds at δ for every pair of distributions with the curve $T\,\varepsilon(\alpha)$.
5. **GDP central-limit approximation** (Equation 32.17 in f-DP and Gaussian DP). A mechanism is **μ-GDP** if telling its two outputs apart is no easier than telling $\mathcal{N}(0,1)$ from $\mathcal{N}(\mu,1)$. The loss of T steps is a sum of T small losses, so it is close to Gaussian. The approximation treats the run as μ-GDP with $\mu = q\sqrt{T(e^{1/\sigma^2} - 1)}$ and reads ε off the GDP profile Equation 32.9.
6. **PLD, pessimistic and optimistic** (Numerical accounting). The **privacy loss** L at an output is the log of how much more likely it is on one neighbour than on the other, and the PLD is its distribution. Losses add, so the PLD of T steps is a T-fold convolution, computed with the fast Fourier transform (FFT) on a grid of width $h = 10^{-5}$. Rounding every loss up gives the pessimistic PLD, whose ε is the "PLD upper" column below, and rounding down

gives the optimistic one, "PLD lower". Each is converted to ε with the record in the first dataset and in the second, and the larger value is kept (Section L.2 in the appendices).

Table 37.2 gives every accountant's ε on the running example.

Table 37.2.: DP-SGD with $q = 0.01$, $\sigma = 1$: ε at $\delta = 10^{-5}$ after T steps under every accountant of this part. Basic and advanced are per-step amplification followed by composition; the PLD columns use the grid $h = 10^{-5}$.

T	Basic	Advanced	RDP, Mironov	RDP, improved	RDP, optimal	GDP CLT	PLD upper	PLD lower
1	0.20	1.13	1.3175	0.9555	0.4899	0.0368	0.1995	0.1994
10	3.64	7.88	1.4145	1.0353	0.7951	0.1304	0.3800	0.3799
100	58.01	78.10	1.6118	1.2141	1.1538	0.4575	0.7185	0.7175
1,000	839.39	1258.64	2.5380	2.1014	2.1009	1.6177	1.8332	1.8232
10,000	11296.93	24379.55	7.4287	6.7127	6.7127	6.0071	6.2377	6.1377

The next theorem sorts the columns. It collects results proved earlier in the part, and what it adds is the two chains that order the bounds.

Theorem 37.1 (Which accountants are bounds)**.** *Assume that the run samples each record independently with probability q, clips each per-example gradient to norm C, adds Gaussian noise of standard deviation σC to the sum, releases every noisy gradient, and fixes q, σ and T in advance, with add/remove neighbours. Then per-step composition, RDP with each conversion and the pessimistic PLD are upper bounds, and the optimistic PLD is a lower bound. They satisfy*

$$\varepsilon_{\mathit{RDP,\ Mironov}} \geq \varepsilon_{\mathit{RDP,\ improved}} \geq \varepsilon_{\mathit{RDP,\ optimal}} \geq \varepsilon_{\mathit{true}}, \qquad \varepsilon^{\uparrow}_{\mathit{PLD}} \geq \varepsilon_{\mathit{true}} \geq \varepsilon^{\downarrow}_{\mathit{PLD}}, \tag{37.2}$$

and if the accountant drops no far tail of the loss distribution, the two PLD values differ by at most Th.

Proof. The plan is to ask, for each accountant, whether what it keeps of the per-step pair still supports a proof, and then to order the RDP conversions.

1. *Per-step composition.* One step is $(\varepsilon_1, q\delta_0)$-DP by the amplification theorem. Basic composition (Theorem 19.3) and advanced composition (Theorem 21.1) hold even when each step is chosen after seeing the earlier outputs (Theorem 21.5), which is how SGD works.
2. *RDP with Mironov's or the improved conversion.* The curve Equation 33.17 is the exact Rényi divergence of the pair. Curves add under adaptive composition (Equation 29.11), because the Rényi divergence is the log of a moment of the likelihood ratio, and given the earlier outputs each step adds fresh noise, so the moments multiply step by step. Both conversions are proved for every pair with the curve (Proposition 30.1, Theorem 30.1). The accountant minimizes over a finite grid of orders, which can only raise the minimum, so it costs tightness, never validity.
3. *The optimal conversion.* For each ε it takes the largest privacy profile $\delta(\varepsilon)$, the smallest δ that works at ε, over all pairs with the curve (Theorem 30.2). The real pair is one of them, so its profile is no larger, and $\varepsilon_{\mathrm{RDP,\ optimal}} \geq \varepsilon_{\mathrm{true}}$.

4. *The PLD sandwich.* Rounding every loss up describes a pair that leaks at least as much as the real one, a dominating pair, and composing dominating pairs gives a dominating pair (Section K.3 in the appendices). Rounding down works the other way, so the error keeps its sign through all T convolutions, which is Equation 34.9.
5. *The width.* Each step's loss moves by less than h between rounding down and up. After T steps the two composed losses differ by at most Th at every outcome, and ε is read off the loss, so the two ε values differ by at most Th too, unless a tail was dropped (Numerical accounting).
6. *Mironov above improved.* At every order the improved conversion is smaller by the saving of Corollary 30.1,
$$s(\alpha) = \frac{\ln \alpha}{\alpha - 1} + \ln \frac{\alpha}{\alpha - 1}, \tag{37.3}$$
whose two terms are positive for $\alpha > 1$. An expression that is larger at every order has a larger minimum.
7. *Improved above optimal.* The optimal conversion is the smallest ε valid for every pair with the curve, and the improved one is such an ε.

□

The theorem does not order the two chains against each other. The pessimistic PLD describes a rounded-up pair and may exceed the truth by up to Th, so a curve-based bound very close to the truth could fall below it. All that follows is $\varepsilon_{\text{RDP, optimal}} \geq \varepsilon^{\uparrow}_{\text{PLD}} - Th$. The script finds RDP optimal above the PLD upper bound at all nine values of T it tries, from 1 to 10,000, because the gap between them (at least 0.2677, Table 37.4) is far larger than Th (at most 0.1000), but that is an observation, not a theorem.

The CLT is missing from the theorem because it is not a bound. Equation 32.17 is a limit as $T \to \infty$ with $q\sqrt{T}$ fixed, and it keeps only the mean and variance of the per-step loss (f-DP and Gaussian DP). Neither the limit nor the two-moment summary controls its error in the tail of the loss, where $\delta = 10^{-5}$ is read, and here the CLT lies below the PLD lower bound at every T.

Example 37.1. Table 37.3 divides each ε by the PLD upper bound, which is within Th of the truth, and Figure 37.1 plots it.

Table 37.3.: Each accountant's ε divided by the PLD upper bound, from Table 37.2.

T	Basic	Advanced	RDP, Mironov	RDP, improved	RDP, optimal	GDP CLT	PLD lower
1	1.00	5.66	6.61	4.79	2.46	0.18	1.00
10	9.58	20.74	3.72	2.72	2.09	0.34	1.00
100	80.73	108.69	2.24	1.69	1.61	0.64	1.00
1,000	457.88	686.57	1.38	1.15	1.15	0.88	0.99
10,000	1811.07	3908.41	1.19	1.08	1.08	0.96	0.98

Per-step composition is right only at $T = 1$, where there is nothing to compose. At $T = 10{,}000$ it is off by a factor of over a thousand, because adding ε's assumes that every step leaks its worst case, while the loss of each step is positive on some outputs and negative on others, so the losses

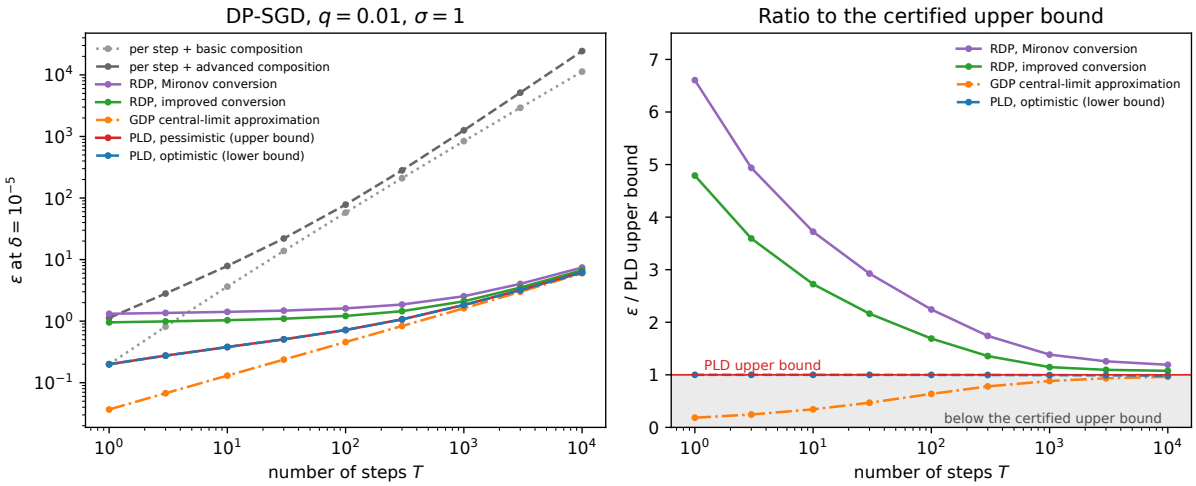


Figure 37.1.: Left: ε at $\delta = 10^{-5}$ against the number of steps T for DP-SGD with $q = 0.01$, $\sigma = 1$, under every accountant of Table 37.2 (the two PLD curves overlap). Right: RDP, the CLT and the PLD lower bound divided by the PLD upper bound; per-step composition is off the scale. Only the PLD lower bound may lie in the shaded region below 1, and the CLT lies there at every T.

mostly cancel. Advanced composition is worse still, because besides its $\sqrt{T}$ term it has a term $T\varepsilon_1(e^{\varepsilon_1} - 1)$ that grows like T, and the amplified ε_1 is too large for that term to be negligible.

RDP's excess has two sources, which Table 37.4 separates. The **conversion slack**, improved minus optimal, is what turning the curve into one (ε, δ) pair loses. The **curve slack**, optimal minus PLD upper, is what the curve loses by keeping moments of the loss rather than its whole distribution.

Table 37.4.: Where the RDP accountant (improved conversion) loses against the PLD upper bound. Conversion slack is improved minus optimal conversion; curve slack is optimal conversion minus PLD upper bound.

T	Conversion slack	Curve slack	Total	Excess over PLD upper
1	0.4656	0.2905	0.7561	379.1%
10	0.2402	0.4151	0.6554	172.5%
100	0.0603	0.4353	0.4956	69.0%
1,000	0.0005	0.2677	0.2681	14.6%
10,000	0.0000	0.4750	0.4750	7.6%

The conversion slack dominates short runs, 0.4656 of the 0.7561 excess at $T = 1$, because every conversion pays $\ln(1/\delta)/(\alpha - 1)$, about $11.5/(\alpha - 1)$ at $\delta = 10^{-5}$, however small the curve is. That toll is also why basic composition (0.20) beats RDP (0.9555) at $T = 1$, though both are valid. After many steps the curve term is large, so the best order trades it against a toll that is now small in comparison, and the conversion slack fades, to 0.0005 at $T = 1000$. The last 7.6% at $T = 10{,}000$ is curve slack, which only an accountant that keeps the whole distribution removes.

So after 1,000 steps a paper might say $\varepsilon = 2.54$ (RDP with Mironov's conversion, as in older code), 2.10 (the improved conversion, as in Opacus's RDP accountant), 1.83 (a PLD accountant) or 1.62 (the CLT). The first three are true, and 1.83 is the most informative. The fourth is false, because the optimistic PLD proves $\varepsilon \geq 1.8232$. At $T = 10{,}000$ the PLD bracket has width $6.2377 - 6.1377 = 0.1000$, exactly Th, and only a finer grid narrows it.

37.2. The inverse problem: noise for a target ε

Practitioners rarely compute ε after the fact. They fix a budget, say $\varepsilon = 3$ at $\delta = 10^{-5}$, fix the steps and the batch size, and ask how much noise to add. This section asks what each accountant's slack costs in noise. Dividing the noisy sum by the expected batch size qn leaves noise of standard deviation $\sigma C/(qn)$ in the averaged gradient. A loose accountant overstates ε at every σ, so it asks for a larger σ, and its slack turns directly into noise.

Definition 37.2 (The noise an accountant requires)**.** For an accountant A, with q and T fixed, write $\varepsilon_A(\sigma)$ for the ε it reports at δ when the noise multiplier is σ. The noise it requires for a target ε^* is

$$\sigma_A(\varepsilon^*) = \min\{\sigma > 0 : \varepsilon_A(\sigma) \leq \varepsilon^*\}. \tag{37.4}$$

This minimum is easy to find if more noise never raises the reported ε, because then it is where a decreasing function crosses a level. The reason to expect it is that a noisier run can be made from a less noisy one without touching the data.

Proposition 37.1 (More noise never raises a bound)**.** *Let $\sigma < \sigma'$. Per-step composition, RDP with each conversion and the exact PLD report $\varepsilon_A(\sigma') \leq \varepsilon_A(\sigma)$. The discretized PLD does so up to grid effects of order Th, and the CLT's ε falls strictly as σ grows.*

Proof.

1. *Post-processing.* Variances of independent Gaussians add, so adding fresh $\mathcal{N}(0, \sigma'^2 - \sigma^2)$ noise to each output of the run with σ gives exactly the run with σ', by a map that does not look at the data.
2. *Data processing.* By Theorem 3.5, such a map can only lower every divergence between the two outputs, including the Rényi divergences behind the curve and the hockey-stick divergence that defines $\delta(\varepsilon)$. Each listed accountant computes ε from these by a map that rises with them, so its ε falls.
3. *Grid and CLT.* Rounding can move the composed loss against the trend by up to Th. The CLT's $\mu = q\sqrt{T(e^{1/\sigma^2} - 1)}$ falls as σ grows, and Equation 32.9 gives a smaller ε for a smaller μ.

□

So any bracketing method finds $\sigma_A(\varepsilon^*)$. The script uses false position, which places each guess where the line through the two ends crosses the target, with the Illinois modification that stops one end from getting stuck, so it needs only a handful of evaluations. That matters for the PLD, where each is two FFT compositions of 10,000 steps.

The solver stops when the bracket is narrower than 10^{-4} and returns the upper end, where $\varepsilon_A \le \varepsilon^*$ was checked, so the returned σ meets the budget even if grid effects make it slightly larger than the minimum. For the PLD the bracket runs from 90% of the smaller of the RDP and CLT values to the RDP value, and since Theorem 37.1 does not order RDP against the PLD, the solver checks that the target really lies between the two ends. The optimal conversion is left out because it costs a search of its own at every σ. The CLT needs no search over σ: find the μ^* whose GDP profile gives ε^*, and invert Equation 32.17 for σ:

$$\sigma = \frac{1}{\sqrt{\ln\left(1 + \mu^{*2}/(q^2 T)\right)}}. \tag{37.5}$$

Table 37.5.: The noise multiplier σ each accountant requires for a target ε at $\delta = 10^{-5}$, with $T = 10{,}000$ steps at $q = 0.01$. PLD on the grid $h = 10^{-5}$; the PLD lower column is the σ at which the optimistic bound reaches the target.

Target ε	Advanced	RDP, Mironov	RDP, improved	GDP CLT	PLD upper	PLD lower
1	24.682	4.974	4.126	3.796	3.988	3.654
3	10.505	1.854	1.662	1.549	1.584	1.547
8	5.711	0.962	0.917	0.868	0.885	0.880

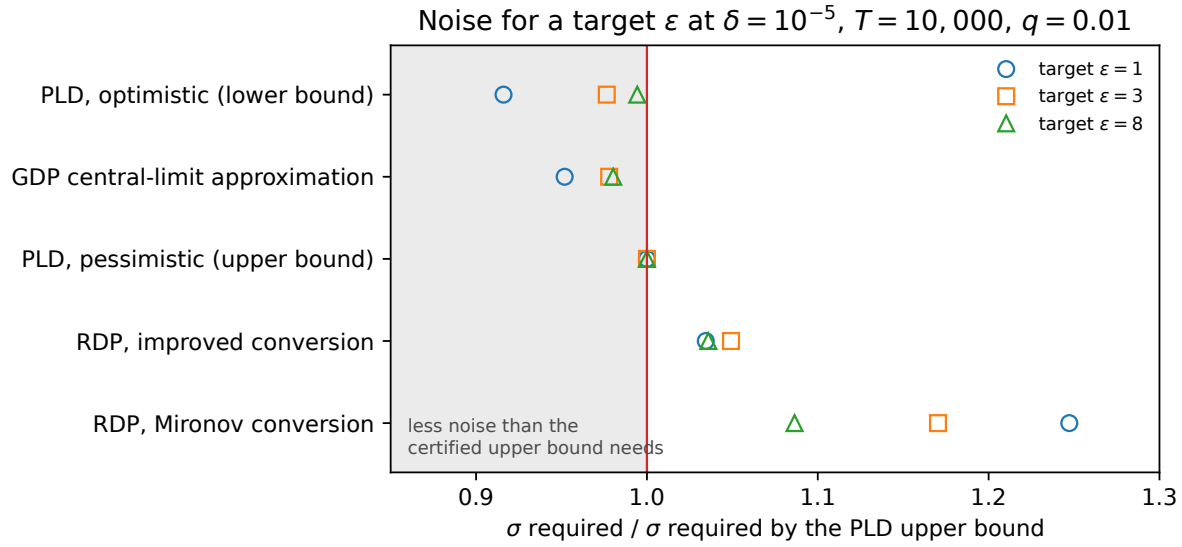


Figure 37.2.: For each target ε at $\delta = 10^{-5}$ ($T = 10{,}000$, $q = 0.01$), the noise multiplier each accountant requires, divided by the one the PLD upper bound requires. The CLT and the PLD lower bound fall in the shaded region, asking for less noise than any certified accountant can justify; advanced composition is off the chart.

Figure 37.2 shows each σ of Table 37.5 relative to the PLD upper bound's. Advanced composition asks for 6.19 to 6.63 times the noise, a level at which DP-SGD rarely trains a useful model. Mironov's conversion costs 8.6 to 24.7% more, most at the strictest target, where its fixed toll is

the largest share of ε, and the improved conversion 3.4 to 4.9% more. That is less than RDP's 7.6% excess in ε, because here a small relative rise in σ buys a larger relative fall in ε. As a check, the PLD upper bound's ε at its own $\sigma = 3.9884$ is 1.0000, the target.

The CLT asks for 2.0 to 4.8% less noise than the certified bound, and at $\varepsilon = 8$ that is provably too little. At its $\sigma = 0.8676$ the optimistic PLD already gives 8.2594, so the run is not $(8, 10^{-5})$-DP, and an honest PLD report at that σ would read 8.3594. At $\varepsilon = 1$ and 3 the grid cannot tell, since the PLD brackets at the CLT's σ, $[0.9550, 1.0550]$ and $[2.9938, 3.0938]$, contain the target. The width $Th = 0.1$ is 10% of $\varepsilon = 1$, so the PLD pins σ only to $[3.654, 3.988]$ there.

Example 37.2. With $n = 50{,}000$ records and $q = 0.01$, the expected batch holds 500 records. At $\varepsilon = 8$ the RDP accountant sets $\sigma = 0.917$, so the averaged gradient carries noise of standard deviation $0.917C/500$, while the PLD accountant sets $\sigma = 0.885$. The RDP accountant thus asks for 3.6% more noise for the same certified budget (the ratio 1.036 in Figure 37.2). Seen from the other side, at $\sigma = 0.8851$, where the PLD certifies $\varepsilon = 8$, the RDP accountant reports 8.6330. At a fixed σ the tighter accountant buys a longer run instead, 20% more epochs at $\varepsilon = 3$.

37.3. Where each accountant is tight

DP-SGD is one mechanism. You will also release statistics with the Gaussian mechanism, answer queries with pure-DP mechanisms ($\delta = 0$), shuffle locally randomized reports, or stop training adaptively. This section asks which accountant is tight for which mechanism. An accountant is tight exactly when the summary it keeps describes the mechanism's privacy loss without loss, so the answer changes with the mechanism. Zero-concentrated differential privacy (zCDP) joins the list here, summarizing the RDP curve by one line $\rho\alpha$ above it. Table 37.6 collects the answers from the earlier chapters.

Table 37.6.: Which accountant to use for which mechanism, with the chapter or section that establishes it.

Mechanism	Use	Why, and how tight	Avoid
Gaussian, no subsampling	GDP or zCDP with Equation 32.9	Exact; RDP's conversion loses 5 to 10% (conversions)	Per-step composition
Poisson-subsampled Gaussian (DP-SGD)	Pessimistic PLD, optimistic as error bar	Bracket of width Th; RDP 7.6 to 69% above it here for $T \geq 100$	zCDP, which cannot express amplification (Concentrated DP); the CLT
Pure-DP steps (randomized response, Laplace, exponential mechanism)	PLD, or RDP with the mechanism's own curve	The loss takes few values, so the PLD is exact or nearly; zCDP is loose for large ε (Concentrated DP)	Advanced composition

Mechanism	Use	Why, and how tight	Avoid
Noise or stopping chosen during the run	RDP filter (Adaptive composition)	Exact at the order it tracks	Fixed-parameter theorems
Local randomizers behind a shuffler	Numeric bound of Shuffling and iteration	The closed form is 4.47 to 7.55 times the exact value there	The subsampling accountant
Only the last iterate released	Amplification by iteration (Shuffling and iteration)	Needs a convex, smooth loss and no released checkpoints	Such claims for deep learning

The ranking in every row follows one ladder. An (ε, δ) pair keeps two numbers, zCDP one line, RDP a curve, GDP the whole trade-off function when it is Gaussian, and the PLD the entire distribution of the loss. Each rung loses information that the next keeps, and lost information is filled in with a worst case. To measure that cost you need mechanisms whose true ε is known, and the next result gives one whose composed loss is exactly Gaussian and one whose loss takes few values.

Proposition 37.2 (Two mechanisms with an exact ε)**.**

(a) T Gaussian steps of sensitivity 1 and noise multiplier σ, without subsampling, are exactly μ-GDP with $\mu = \sqrt{T}/\sigma$, so Equation 32.9 gives their exact ε. Their RDP curve $\alpha\mu^2/2$ is also exact, and equals the zCDP line with $\rho = \mu^2/2$.

(b) k rounds of randomized response, each ε_0-DP, have composed loss $(k - 2J)\varepsilon_0$ with $J \sim \mathrm{Bin}(k, 1/(1 + e^{\varepsilon_0}))$, so the exact $\delta(\varepsilon)$ is a finite sum.

Proof.

1. *One Gaussian step.* The two outputs $\mathcal{N}(0, \sigma^2)$ and $\mathcal{N}(1, \sigma^2)$, divided by σ, are exactly $\mathcal{N}(0, 1)$ and $\mathcal{N}(1/\sigma, 1)$, so one step is $1/\sigma$-GDP with nothing lost.
2. *Gaussian composition.* Squares of μ add under composition, as variances do (f-DP and Gaussian DP), so $\mu^2 = T/\sigma^2$. The Rényi divergence of two normals μ standard deviations apart is $\alpha\mu^2/2$ (Proposition 3.3), a line through the origin, so the best zCDP line is the curve itself.
3. *Randomized response.* Each round's loss is $\pm\varepsilon_0$ and losses of independent rounds add (Lemma 21.2), which gives Proposition 21.1. The values lie on a lattice, so nothing is rounded, and $\delta(\varepsilon)$, an average over the loss (Lemma 21.1), is a finite binomial sum.

□

So when the composed pair is exactly Gaussian, GDP costs nothing. When subsampling makes it a mixture, no closed form is exact, and only the PLD is both certified and tight. Adaptivity is where RDP is the natural tool. If the noise or the stopping time depends on released outputs, the per-step pair is not fixed in advance as Theorem 37.1 assumed, and the Rényi filter of Adaptive composition handles that while losing nothing at the order it tracks.

Example 37.3. One hundred Gaussian steps with $\sigma = 10$ compose to $\mu = \sqrt{100}/10 = 1$, and Equation 32.9 gives the exact $\varepsilon = 4.3772$ at $\delta = 10^{-5}$. Converting the equally exact RDP curve gives 4.7285 with the improved conversion, 8.0% above, and 5.2985 with Mironov's. One hundred rounds of randomized response with $\varepsilon_0 = 0.1$ have exact $\varepsilon = 4.3068$. Against that, RDP with the exact curve reaches 4.6154. zCDP must use a ρ whose line lies above the curve, and the smallest is $\varepsilon_0 \tanh(\varepsilon_0/2)$ per round, the charge that every ε_0-DP step satisfies (Equation 31.11), which yields 4.7263. Advanced composition with $\delta' = \delta$ ends at 5.8502, and basic composition at $100 \times 0.1 = 10.0000$.

Suppose a team releases the 100 Gaussian counts and reports $\varepsilon = 5.2985$ from an old RDP accountant with Mironov's conversion. The release is exactly $(4.3772, 10^{-5})$-DP, so the report is 21.0% above the truth ($5.2985/4.3772 \approx 1.210$), and the team could have answered more queries at no cost in privacy.

37.4. Matching the accountant to the code

Every number so far assumes a model of the training run, and the accountant cannot check it against your code. It takes q, σ and T and trusts that the code does what the theorem assumes. Ponomareva et al. (2023, Section 5.3.1) put it in one heading, "privacy accounting assumptions should match training reality", and note that many papers since Abadi et al. (2016) have reported the Poisson number while training with shuffled fixed-size batches. This section asks what such a mismatch costs, and the answer is that the number then describes a different algorithm. Check each item of Table 37.7 against the code before reporting.

Table 37.7.: A checklist for matching the accountant to the training code.

Item	What the accountant assumes	What to check	If it does not match
Sampling scheme	Poisson at rate q	Independent inclusion, variable batch sizes?	Fixed-size: Proposition 37.3; shuffled: Proposition 37.4
Neighbouring relation	Add/remove	Or replace-one, or zero-out (one gradient set to 0)?	Replace-one costs about twice as much (subsampling)
Privacy unit	One example	Several examples per person?	Clip and sample per user
Released outputs	Every noisy gradient	Checkpoints or logs exposed?	Already covered
Tuning	Not counted	Settings chosen on private results?	Proposition 37.5, or tune on public data
Adaptive parameters	Fixed in advance	Early stopping, schedules?	A filter (Adaptive composition)
Implementation	Exact real-valued noise	Floating-point sampling?	Outside the accountant's scope

The first row is the most common mismatch. A loader that draws a random batch of exactly qn records looks like Poisson sampling. But a record cannot enter a full batch without pushing another out, so in effect one record is swapped for another, and the batch sum can move by $2C$ rather than C.

Proposition 37.3 (Fixed-size batches (Lebeda et al. 2024))**.** *If each step draws a fresh uniformly random batch of exactly qn records, then under add/remove neighbours every ε certified for the Poisson pair with rate q and noise multiplier $\sigma/2$ holds for the run with noise multiplier σ.*

The proof is Proposition 9 of Lebeda, Regehr, Kamath & Steinke (2024), which shows with Zhu, Dong & Wang (2022, Theorem 11) that the shift of $2C$ is the worst case under the add and the remove relation separately. A shift of $2C$ against noise σC is a shift of C against $(\sigma/2)C$, and Amplification by subsampling derives the coupling behind it.

A loader that shuffles once per epoch and cuts the data into batches uses each record exactly once per epoch. The record is then never absent by chance, which is what amplification feeds on, and no amplification theorem holds for every dataset. What stays certain is the record's own steps, one per epoch, under the **zero-out** relation, in which one record's gradient is replaced by zero.

Proposition 37.4 (The guarantee without amplification)**.** *Suppose each of E epochs shuffles the data and cuts it into consecutive batches, with noise multiplier σ. Under zero-out neighbours the run is $\sqrt{E}/\sigma$-GDP, and under replace-one neighbours it is $2\sqrt{E}/\sigma$-GDP.*

Proof. The plan is to fix the shuffles, find the one step per epoch that differs, and compose those steps.

1. *Fix the shuffles.* They do not depend on the data. Publishing them too can only help an observer, and the real run drops them, a post-processing (Theorem 19.1), so it suffices to prove the bound with them fixed.
2. *One step per epoch differs.* With the shuffles fixed, only the batch that holds the record differs, and its sum moves by at most C under zero-out. Every other step, given the earlier outputs, has the same distribution on both datasets.
3. *That step is Gaussian.* A shift of C against noise σC is $1/\sigma$-GDP, by part (a) of Proposition 37.2.
4. *Compose.* The E differing steps compose adaptively to $\mu^2 = E/\sigma^2$ (f-DP and Gaussian DP). Under replace-one, the old gradient leaves and a new one comes in, so the shift is up to $2C$ and μ doubles.

□

Under add/remove there is no clean bound, because inserting a record shifts every later batch. Ponomareva et al. recommend reporting this guarantee whenever the sampling assumption fails. Tuning breaks the accountant's assumptions differently, because the release depends on several runs while the accountant counts one.

Proposition 37.5 (Tuning on the private data composes)**.** *Training m configurations on the private data and releasing the best costs at most the m runs composed. With RDP, that is the curve $mT\,\varepsilon(\alpha)$.*

Proof.

1. *Post-processing.* The best run is a function of the m trained models, so by Theorem 19.1 releasing it is no less private than releasing all m.
2. *Composition.* The m runs use the same data, so their RDP curves add (Equation 29.11).

□

Papernot & Steinke (2022) do much better when the number of configurations is random, and NIST SP 800-226 (Section 4.3) recommends tuning on non-sensitive data.

Example 37.4. At $T = 10{,}000$, fixed-size batches accounted honestly by Proposition 37.3 give 47.42 with RDP instead of 6.71, and 43.42 with the PLD instead of 6.24. Shuffle and partition for 100 epochs at $\sigma = 1$ is $\mu = \sqrt{100} = 10$ by Proposition 37.4, and Equation 32.9 gives $\varepsilon = 91.8173$ for zero-out neighbours, or 284.39 at $\mu = 20$ under replace-one. Ten tuning runs compose to $\varepsilon = 27.23$ with RDP, and 100 runs to 145.17.

So a script whose `DataLoader` has `shuffle=True` and `batch_size=500` on 50,000 records for 100 epochs, calling an RDP accountant with `sample_rate=0.01`, fails the checklist's first row, and its certified guarantee is 91.8173, not 6.71. Switching the loader to Poisson sampling, a few lines, makes the assumptions true and the guarantee 6.24 with a PLD accountant.

The sampling scheme is the largest single risk. Its factor of 6.96 (43.42/6.24) from one line in the data loader exceeds the choice between RDP and the PLD, at most 1.19 here (Table 37.3). The other rows decide what the number means. Example-level DP-SGD protects each example, not each user, which NIST SP 800-226 (Section 2.4) calls a privacy hazard, and floating-point noise can break a mechanism that is correct on paper (Section 4.3).

37.5. What to report

An ε without its assumptions cannot be checked, compared or even read, since the same number means different things for one example or one user, with or without tuning, at $\delta = 10^{-5}$ or 10^{-10}. This section asks what a statement must contain so that a reader can interpret and recompute it. NIST SP 800-226, "Guidelines for Evaluating Differential Privacy Guarantees" (Near, Darais, Lefkovitz & Howarth, 2025), answers for anyone evaluating a claim, and Ponomareva et al. (2023, Section 5.3.3) answer for machine learning.

Definition 37.3 (A privacy statement)**.** Following Ponomareva et al. (2023, Section 5.3.3), a **privacy statement** gives four things.

1. *The DP setting:* central or local, and who is trusted.
2. *The instantiated definition:* which data accesses are covered (one run, or tuning too), the mechanism's output, the unit of privacy, and the neighbouring relation.
3. *Accounting details:* the type of accounting, its assumptions and whether they hold, and the formal guarantee for the final model and for the tuning.
4. *Transparency:* code when possible, using standard libraries.

For DP-SGD it adds what the accountant needs: the sampling scheme, q (or the batch size and n), T (or the epochs), σ, the clipping norm C, and the accountant's version and discretization.

The clipping norm is listed although ε does not depend on it, because C and σ together give the actual noise σC and let a reader check the implementation.

The second parameter needs as much care. Both documents want δ much smaller than $1/n$ for n units of privacy, because δ is a chance that the promise fails. A mechanism that publishes each record in full with probability δ is $(0, \delta)$-DP, yet at $\delta = 1/n$ it exposes someone with probability about $1 - 1/e = 0.632$ (Proposition 18.3). Ponomareva et al. (Section 5.3.2) suggest $\delta = n^{-1.1}$. NIST (Section 2.3) gives $\delta \leq 1/n^2$ as a common recommendation, calls values above 10^{-5} suspicious, and asks for the original RDP, zCDP or GDP parameters alongside any conversion.

So a report should give ε at more than one δ, and these cannot be derived from one pair. Many mechanisms share a single (ε, δ) pair and behave differently below it; one can be $(0, \delta)$-DP but satisfy no guarantee at all at a smaller δ. A PLD accountant composes the distribution once, and each further δ costs one more evaluation of $\delta(\varepsilon)$.

Table 37.8.: The running example's privacy profile: ε at several δ for $T = 10{,}000$, $q = 0.01$, $\sigma = 1$, $n = 50{,}000$.

δ	PLD upper	PLD lower	RDP, improved
2.0×10^{-5} $(1/n)$	6.0068	5.9068	6.4884
1.0×10^{-5}	6.2377	6.1377	6.7127
6.8×10^{-6} $(n^{-1.1})$	6.3641	6.2641	6.8353
1.0×10^{-6}	6.9574	6.8574	7.4140
4.0×10^{-10} $(1/n^2)$	9.0502	8.9378	9.4540

Example 37.5. With $n = 50{,}000$ records, the expected batch is $qn = 500$, and $T = 10{,}000$ steps are $Tq = 100$ epochs. The conventions give $1/n = 2.0 \times 10^{-5}$, $n^{-1.1} = 6.8 \times 10^{-6}$ and $1/n^2 = 4.0 \times 10^{-10}$, and Table 37.8 gives ε at each. The usual $\delta = 10^{-5}$ is half of $1/n$, so it is not "much smaller than $1/n$", and at $1/n^2$ the certified ε rises to 9.0502. The PLD bracket has width 0.1000, the Th of Theorem 37.1, down to $\delta = 10^{-6}$. At $1/n^2$ it is $9.0502 - 8.9378 = 0.1124$, because the accountant cuts off the far tails of each step's PLD and moves up to 10^{-15} of mass to the side that keeps each bound safe, of order 10^{-11} over 10^4 steps, a few percent of 4×10^{-10}. The RDP accountant's original parameter is its curve, $\varepsilon(4.1) = 3.7336$ at the best order, which converts to 6.7127.

> **Privacy guarantee.** The model was trained with DP-SGD in the central model: the training server is trusted to run the code below, and the guarantee covers every intermediate noisy gradient, hence every checkpoint and the final model. **Unit of privacy:** one training example; each person contributes at most one example. **Neighbouring relation:** add or remove one example. **Training:** Poisson sampling with rate $q = 0.01$ ($n = 50{,}000$ examples, expected batch size 500), $T = 10{,}000$ steps (100 epochs), per-example clipping norm $C = 1$, Gaussian noise multiplier $\sigma = 1$. **Accountant:** privacy loss distributions, pessimistic discretization on the grid $h = 10^{-5}$, both add and remove directions (this book's `numerical_accounting.py`). **Guarantee:** $(6.24, 10^{-5})$-DP and $(9.06, 4.0 \times 10^{-10})$-DP. The optimistic discretization

gives 6.1377 and 8.9378, so the accountant's own error is at most 0.1000 and 0.1124. **Not covered:** hyperparameters were tuned on a public proxy dataset. **If the sampling assumption failed** (for example, shuffled fixed-size batches), the guarantee without amplification is $(91.82, 10^{-5})$-DP for zero-out neighbours.

Upper bounds are rounded up, never to the nearest value, so 9.0502 becomes 9.06, because 9.05 would be a claim the accountant has not certified.

The statement gives the accountant's error bar, which RDP cannot give for lack of a lower bound, and the fallback for a reader who doubts an item of Table 37.7.

37.6. Checking your accountant

An accountant is code, and code has bugs. This section asks how to test one, yours or a library's, before you trust its number. A good test is a case where the truth is known exactly, or a property every correct accountant must have, so that a failure is a bug and not slack. The results of this chapter supply four.

Proposition 37.6 (Four properties of every correct accountant)**.** *Under the assumptions of Theorem 37.1:*

1. *With $q = 1$ the run is exactly $\sqrt{T}/\sigma$-GDP, and every upper bound is at least the ε that Equation 32.9 gives.*
2. *On the plain Gaussian pair, the PLD bracket contains that exact ε, if no tail is truncated.*
3. *The chains of Equation 37.2 hold at every T, and no upper bound falls below the optimistic PLD.*
4. *The composed pessimistic PLD, evaluated at its own ε, gives back δ, and no accountant's ε decreases as T grows.*

Proof. The plan is to derive each property from a result already proved, so that a violation can only be a bug.

1. *Full batches.* With $q = 1$ every step is the plain Gaussian mechanism, so Proposition 37.2 gives the truth, and an upper bound is at least the truth (Definition 37.1).
2. *The Gaussian PLD.* By Theorem 37.1 the two PLD values bracket the truth, here the closed form.
3. *The chains.* Theorem 37.1 proves them, and an upper bound below a lower bound would put the truth on both sides.
4. *Round trip and growth.* Solving $\delta(\varepsilon) = \delta$ and evaluating $\delta(\varepsilon)$ at the solution are inverse operations on one decreasing profile. The T-step transcript is the $(T + 1)$-step one with its last output dropped, a data-independent map, so by Theorem 3.5 it is no easier to tell apart.

□

Example 37.6. For $\sigma = 1$, $T = 10$ and $q = 1$, the exact μ is $\sqrt{10} = 3.1623$ and the exact ε is 17.8566. The RDP accountant gives 19.0536, above it as a bound must be. The CLT formula gives $\mu = \sqrt{10(e-1)} = 4.1452$ and $\varepsilon = 25.5853$, and fails because its derivation assumes a small q. On the plain Gaussian with $\mu = 1$, $T = 10$ and $h = 10^{-3}$, the PLD bracket $[17.8516, 17.8616]$ contains the closed form 17.8566. Both chains hold at all nine values of T, the round trip gives back $\delta = 1.000 \times 10^{-5}$, and the PLD upper bound increases with T.

37.6.1. Comparing with independent libraries

How closely should two independent implementations agree? This book's accountants use no library, so that every line can be read, but production code should use a maintained one and compare it with a second: Google's `dp_accounting` (RDP and PLD), Opacus (RDP, GDP and its default, the privacy loss random variable (PRV) accountant), or Microsoft's `prv_accountant` (Gopi, Lee & Wutschitz 2021), which returns a lower bound, an estimate and an upper bound. Two RDP accountants differ only in their orders and their conversion. Opacus uses the improved conversion and the same orders as this book's accountant except the three largest (128, 256 and 512), and the running example's best orders, 4.1 to 9.9, lie in both grids. A gap of about $s(\alpha)$ (Equation 30.14), 0.3612 at order 10, means that one of them uses Mironov's conversion.

Fractional orders are a known weak spot, because there the subsampled Gaussian's RDP is an infinite series with terms of both signs. `dp_accounting` 0.6.0 counts the negative terms as positive and returns a loose bound, so the fixed-size value 47.42 of Example 37.4 comes out as 49.43 there, while Opacus keeps the signs. PLD and PRV brackets from two libraries must overlap, since every certified upper bound lies above every certified lower bound. A cross-check run outside this book's scripts agrees. Opacus 1.6.0, `dp_accounting` 0.6.0 and `prv_accountant` 0.2.0 match this chapter's RDP and PLD numbers to the printed precision, and their tight PLD estimate at $T = 10{,}000$ is about 6.19, inside the bracket. These values, like the 49.43, come from the libraries, not from the book's scripts.

37.7. Code

The script code/choosing_an_accountant.py imports every accountant from the chapter that derived it and implements only the comparison, the inverse problem and the checks. Its public functions are `accountant_table(q, sigma, delta, ks)`, one `AccountantRow` per T, and `sigma_for_target(eps_target, q, steps, delta, accountant)`, which solves Equation 37.4. It runs in about two and a half minutes, mostly PLD compositions:

```
uv run python code/part04-privacy-accounting/choosing_an_accountant.py
```

Table 37.9.: The script's output with the default seed (nothing is random).

Experiment	Result
Every accountant, $T = 1$ to 10,000	Table 37.2, Table 37.3, Table 37.4; both chains hold, RDP optimal $\geq$ PLD upper, CLT below PLD lower, at every T
Noise for a target ε	Table 37.5; PLD $[8.2594, 8.3594]$ at the CLT's σ for $\varepsilon = 8$
Other mechanisms	Gaussian $\mu = 1$: 4.3772 exact, 4.7285 RDP; randomized response: 4.3068 exact, 4.6154 RDP, 4.7263 zCDP, 5.8502 advanced
Mismatches	Fixed-size: 47.42 (RDP), 43.42 (PLD); no amplification: 91.8173 (zero-out), 284.39 (replace-one); 10 tuning runs: 27.23
Privacy profile	Table 37.8
Sanity checks	$q = 1$: 17.8566 exact, 19.0536 RDP, 25.5853 CLT; Gaussian PLD $[17.8516, 17.8616]$

tests/test_choosing_an_accountant.py checks that the output still matches, and calls the two public functions directly.

37.8. Summary

Table 37.10.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 37.1	Upper bounds are at least the true ε, lower bounds at most; approximations have no proof	Report only upper bounds; never a CLT number
Theorem 37.1	Per-step, RDP and pessimistic PLD are upper bounds, optimistic PLD a lower bound; Equation 37.2; PLD width $\leq Th$	Report the pessimistic PLD with the optimistic one beside it; RDP (improved) for quick bounds, 5 to 10% high on long runs, far more on short ones
Definition 37.2, Proposition 37.1	$\sigma_A(\varepsilon^*)$ is the root of a decreasing function	Calibrate σ with the accountant you will report
Proposition 37.2	Plain Gaussian steps are $\sqrt{T}/\sigma$-GDP; randomized response composes on a lattice	GDP for the plain Gaussian, PLD for pure DP; both as test cases

Result	Statement	Use
Proposition 37.3	Fixed-size batches: the Poisson pair with $\sigma/2$	Check the sampler first, the costliest error in practice
Proposition 37.4	Shuffle and partition: $\sqrt{E}/\sigma$-GDP, zero-out	The fallback to state
Proposition 37.5	Tuning composes all runs	Tune on public data
Definition 37.3	Setting, definition, accounting, code, DP-SGD parameters	ε at several δ, one well below $1/n$; round up
Proposition 37.6	Four properties of a correct accountant	Run them; cross-check a second library

37.9. References

- Ponomareva, Hazimeh, Kurakin, Xu, Denison, McMahan, Vassilvitskii, Chien & Thakurta, "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (Journal of Artificial Intelligence Research 77, 2023; arXiv:2303.00654). Section 5.3.1 on matching accounting to data processing (Table 3), Section 5.3.2 on setting δ, Section 5.3.3 on what to report, Section 5.2.1 on target ε tiers. The practical companion to this chapter.
- Near, Darais, Lefkovitz & Howarth, "Guidelines for Evaluating Differential Privacy Guarantees" (NIST Special Publication 800-226, March 2025). The unit of privacy (Section 2.4), the choice of δ and reporting original parameters of RDP, zCDP and GDP guarantees (Section 2.3), floating-point and hyperparameter-tuning hazards (Section 4.3). Read it for what an evaluator will ask.
- Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021). The PRV accountant with certified error bounds, and examples where the CLT under-reports; the basis of `prv_accountant` and Opacus's default accountant.
- Doroshenko, Ghazi, Kamath, Kumar & Manurangsi, "Connect the Dots: Tighter Discrete Approximations of Privacy Loss Distributions" (PoPETs 2022). The pessimistic PLD discretization used in Google's `dp_accounting`.
- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (arXiv 2019). The RDP of the subsampled Gaussian, computed exactly; the core of every RDP accountant for DP-SGD.
- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020). Theorem 21 is the improved conversion that Opacus and `dp_accounting` use.
- Dong, Roth & Su, "Gaussian Differential Privacy" (Journal of the Royal Statistical Society Series B, 2022), and Bu, Dong, Long & Su, "Deep Learning with Gaussian Differential Privacy" (Harvard Data Science Review, 2020). The GDP CLT for DP-SGD; read with this chapter's warning that it is not a bound.
- Lebeda, Regehr, Kamath & Steinke, "Avoiding Pitfalls for Privacy Accounting of Subsampled Mechanisms under Composition" (arXiv:2405.20769, 2024; IEEE SaTML 2025). Fixed-size batches versus Poisson sampling, and the factor that the wrong accountant costs.

- Chua, Ghazi, Kamath, Kumar, Manurangsi, Sinha & Zhang, "How Private are DP-SGD Implementations?" (ICML 2024). Shuffled batches can be much less private than the Poisson accountant reports.
- Papernot & Steinke, "Hyperparameter Tuning with Renyi Differential Privacy" (ICLR 2022). Privacy cost of tuning with a random number of trials, far below naive composition.
- Google, `dp_accounting` library; Meta, Opacus; Microsoft, `prv_accountant`. Documentation of the production RDP, PLD and PRV accountants to check this book's numbers against.

Part V.

Differential privacy in machine learning

The previous parts built the guarantee and the tools to account for it. This part trains models with it. It asks the questions that decide whether a private model is worth deploying: **how do you train with differential privacy (DP), how much accuracy does it cost, which knobs move that cost, and how do you check that the trained model is as private as the accountant claims?** The answers are mostly about one algorithm, stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, called DP-SGD, and about everything around it: how its hyperparameters are chosen, what replaces it when its assumptions fail, how public data and the choice of privacy unit change it, who pays for its noise, which ε to target, and how to audit the result. The last chapter turns the guarantee to a different use: the same noise that protects people also keeps a reused validation or test set honest.

Who this part is for

You are expected to have read the earlier parts. From Optimization you need SGD, strong convexity and the sensitivity of regularized empirical risk minimization (ERM). From Membership inference you need the likelihood-ratio attack (LiRA) and the true and false positive rates of an attack. From the DP foundations you need the Gaussian mechanism (The Laplace and Gaussian mechanisms), clipping (Beyond global sensitivity), the exponential mechanism and report-noisy-max (Selection) and the lower bounds of Lower bounds. From the accounting part you need Gaussian differential privacy (GDP) and trade-off functions (f-DP and Gaussian DP), amplification by subsampling and the Rényi differential privacy (RDP) accountant for DP-SGD (Amplification by subsampling), and what to report (Choosing an accountant). The last chapter also uses the replace-one argument of Algorithmic stability and generalization and the sparse vector technique of Selection. Each chapter restates the facts it uses and links to them.

The common thread

A private training algorithm has two halves. The **privacy half** says which quantity has bounded sensitivity and how much noise covers it; it is a theorem, and the accountants of the previous part turn it into a number. The **utility half** says how much the noise and the bounding cost; it is mostly measured, rarely proved. Every chapter of this part moves one of the two.

The first two chapters build the algorithm: Private ERM compares three places to inject noise into training and shows why, in practice, only noise on the gradients survives non-convex models, and DP-SGD adds clipping, minibatches and accounting. The next three ask what else spends the budget or could spend it better: hyperparameter search; mechanisms that do not rely on random batches, such as the follow the regularized leader (FTRL) algorithm run on noisy gradient sums, DP-FTRL; and private prediction instead of private training, such as private aggregation of teacher ensembles (PATE). The next three change the setting: public data, a different privacy unit, and the effect on under-represented groups. The next two close the loop between theory and practice: which ε to choose, and how to test a claim empirically. The last chapter uses the privacy half for a different purpose. A mechanism that reveals little about any one record cannot be overfitted, so DP also lets one holdout set answer many adaptively chosen questions, the situation of every hyperparameter search and leaderboard. Table 37.11 lists them.

Table 37.11.: The chapters of this part, the question each answers, and the earlier chapters it uses.

Chapter	The question it answers	Builds on
Private empirical risk minimization	Where can noise enter training, what does each choice cost, and what rate can a convex model reach?	6, 7, 20, 24, 27, 32
DP-SGD	How do clipping, noise and sampling make deep-learning training private, and what does each knob do?	6, 20, 24, 33, 37, 38
Hyperparameter tuning and private selection	How much privacy does tuning leak, and how do you pay for it honestly?	22, 29, 39
DP-FTRL and matrix factorization	How do you train privately without trusting random batches?	20, 32, 39
Private prediction: PATE and in-context learning	When is it better to privatize answers than weights?	22, 24, 29, 39
Public data and private fine-tuning	How much does a public pretrained model buy, and why does fine-tuning fewer parameters help?	39, 40
User-level differential privacy	What changes when one person contributes many records?	19, 39
Disparate impact	Who pays for the noise and the clipping?	39
Choosing ε	Which ε should you target, and how do you justify it?	18, 32, 37, 39
Auditing private training	Is the model as private as the accountant says?	10, 32, 39, 46
Adaptive data analysis	How many adaptively chosen questions can one holdout set answer, and how do you reuse it safely?	7, 19, 20, 22, 23, 31, 40

The running examples

Two models recur. The first is **regularized logistic regression** in $d = 10$ dimensions on synthetic data, where the exact minimizer can be computed and every method can be compared with the non-private optimum; Private ERM introduces it. The second is a **small neural network** trained with DP-SGD on synthetic data, small enough that every experiment of the part runs on a laptop in seconds, yet non-convex, so that clipping and noise behave as they do in deep learning;

DP-SGD introduces it. The full-scale experiment, DP-SGD on CIFAR-10 audited with LiRA, needs a GPU and PyTorch, which are not dependencies of the repository; the chapters scale it down and say how.

How to read this part

Order. Read chapters 38 and 39 first, in order: every later chapter uses DP-SGD. Chapters 40 and 41 change how the algorithm is run, and can be read in either order. Chapters 42 to 45 are independent of each other. Chapter 46 uses the vocabulary of all the earlier ones, and chapter 47 should come after the others, because it checks what the others claim. Chapter 48 can be read at any point after chapter 40, whose hyperparameter search is its motivating example. It needs DP-SGD only for that example.

Structure of a chapter. The chapters have the shape of the earlier parts. Each opens with the question it answers and its running example, then gives one section per topic, whose definitions, theorems and worked examples are numbered, each theorem followed by its proof. A Code section, a Summary table of the chapter's results and References close it.

Run the code. Each chapter has a script in `code/` that prints every number its text quotes, except values attributed to a paper, and regenerates its figures, for example

```
uv run python code/part05-dp-ml/dp_erm.py
```

Every model is written in NumPy, gradients included, so you can see exactly what each algorithm computes.

Notation

The part uses the notation of the earlier parts, extended in Table 37.12.

Table 37.12.: Notation used throughout this part, in addition to that of the earlier parts.

Symbol	Meaning
$D = (z_1, \dots, z_n)$, $z_i = (x_i, y_i)$	A training set of n records with features x_i and labels y_i; a neighbouring dataset D' adds or removes one record, $D \sim D'$
$\theta \in \mathbb{R}^d$, $\ell(\theta; z)$, $\mathcal{L}(\theta; D)$	The model parameters; the loss on one record; the empirical risk, the training objective (with its regularizer, when there is one)
θ_t, θ^*, $\tilde{\theta}$	The iterate at step t; the non-private minimizer; a private release
$g_i = \nabla_\theta \ell(\theta; z_i)$, $\bar{g}_i$	The gradient of record i's loss, a per-example gradient; the clipped one

Symbol	Meaning
C	The clipping norm: per-example gradients are rescaled to length at most C
σ	The noise multiplier: the noise standard deviation in units of C
q, $B = qn$, T	The sampling rate; the expected batch size; the number of steps
η	The learning rate
$\mu(\varepsilon, \delta)$	The largest ratio of sensitivity to noise standard deviation for which a Gaussian mechanism is (ε, δ)-DP

What you will be able to do

By the end of this part you should be able to:

- train a convex model privately by output, objective or gradient perturbation, and predict its excess risk;
- implement DP-SGD from scratch, with per-example clipping, Poisson sampling and an RDP accountant, and explain what each hyperparameter trades;
- account for hyperparameter search, and run a private selection procedure;
- explain when DP-FTRL or a matrix-factorization mechanism beats DP-SGD, and why;
- train a teacher ensemble and answer queries with PATE or a private in-context learner;
- decide whether to fine-tune all parameters or a few, with a public pretrained model;
- account for user-level privacy, and bound each user's contribution;
- measure and explain the disparate impact of private training on small groups;
- choose an ε and justify it to someone who will hold you to it;
- audit a private training run with canaries and turn the attack's success into an empirical lower bound on ε;
- explain why a reused holdout set overfits, size a holdout set for adaptively chosen questions, and reuse one safely with Gaussian noise or Thresholdout.

38. Private empirical risk minimization

Most of machine learning is **empirical risk minimization (ERM)**. You choose the parameters θ that minimize the average loss on the training data, usually plus a **ridge** penalty on the squared length of θ. The trained weights are a function of every training record, and Membership inference showed that they give some of those records away. An exact minimizer can even release a record outright: the median of an odd number of values minimizes the average absolute loss, and it *is* one of the values. Differential privacy (DP) promises the opposite: the distribution of whatever you release changes by at most a factor e^{ε}, plus a slack δ, when one record is replaced. So randomness has to enter somewhere between the data and the released weights, and this chapter asks where.

The idea that organizes the answer is the one behind every mechanism of this book. Find a quantity that one record can move only by a known amount, its **sensitivity**, and add noise large enough to hide a move of that size. There are three natural places for the noise. **Output perturbation** trains as usual and adds noise to the minimizer, which a ridge penalty keeps from moving far. **Objective perturbation** adds a random linear term to the objective and minimizes it exactly, which needs a bound on how far one record moves the objective's gradient. **Gradient perturbation** adds noise to every gradient during training, so it needs that bound only at the points it visits. The earlier in training the noise enters, the fewer assumptions it needs, and the third needs no convexity at all, which is why the next chapter builds on it.

The running example is regularized logistic regression in $d = 10$ dimensions, with a ridge of weight 0.01. The features are uniform on the unit sphere, and the labels come from a logistic model whose true weights have length 5. Every method targets $\varepsilon = 1$, with $\delta = 10^{-5}$ for those that need a δ, on 1,000 to 100,000 records. At 10,000 records, output perturbation with Gaussian noise raises the training loss above the exact minimizer's by 9.274×10^{-4}, and objective perturbation by ten times less.

38.1. Output perturbation

The simplest private trainer computes the exact minimizer and adds noise to it, as Chaudhuri & Monteleoni (2008) did for logistic regression. This section asks how much noise hides one record's effect on the minimizer, what the noise costs in accuracy, and what changes when the minimizer is only approximate.

A training set D holds n records. Each record $z_i = (x_i, y_i)$ pairs a feature vector $x_i \in \mathbb{R}^d$ with a label $y_i \in \{-1, +1\}$, and $\ell(\theta; z)$ is the loss of the parameters on one record. The objective adds a ridge penalty of weight λ to the average loss,

$$\mathcal{L}(\theta; D) = \frac{1}{n} \sum_{i=1}^{n} \ell(\theta; z_i) + \frac{\lambda}{2} \|\theta\|^2, \tag{38.1}$$

and its minimizer is $\theta^*(D)$.

Neighbouring datasets $D \sim D'$ differ by **replacing** one record, z in D by z' in D', so both have size n. The **excess risk** of a released $\tilde{\theta}$ is $\mathcal{L}(\tilde{\theta}; D) - \mathcal{L}(\theta^*; D)$, how much worse it fits the objective than the exact minimizer.

Two earlier results do most of the work. First, if each loss is convex with gradients of length at most L, replacing one record moves the minimizer by at most $2L/(\lambda n)$ (Theorem 6.5 in Optimization). That is because the objective's gradient moves by at most $2L/n$, and the ridge's curvature λ turns a small gradient into a small distance: where the gradient has length g, the minimizer is within g/λ.

The second result says how much Gaussian noise hides a move of $\Delta_2 f$, the most that one record can move a function f in Euclidean length. With normal noise of standard deviation σ per coordinate, only the ratio $\mu = \Delta_2 f/\sigma$ matters, because a rotation puts the move along one axis without changing the noise. The release is (ε, δ)-DP exactly when

$$\delta \geq \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big), \tag{38.2}$$

where Φ is the standard normal distribution function (Theorem 20.3). The right side grows with μ, so each budget has a largest admissible ratio, written $\mu(\varepsilon, \delta)$, and the least noise is $\sigma = \Delta_2 f/\mu(\varepsilon, \delta)$ (Corollary 20.1). For $\varepsilon = 1$ and $\delta = 10^{-5}$, $\mu = 0.2681$, so $\sigma = 3.7306\,\Delta_2 f$, while the classic formula $\Delta_2 f\sqrt{2\ln(1.25/\delta)}/\varepsilon$ of Theorem 20.2 gives 30% more, $4.8448\,\Delta_2 f$.

Gaussian noise can never give pure ε-DP, with $\delta = 0$, because the log ratio of two shifted Gaussian densities grows without bound in the tails. Pure DP needs a density that falls off only exponentially in every direction, and the theorem below offers this noise as its pure option.

Lemma 38.1 (Noise with density proportional to $e^{-|b|/s}$)**.** *Let a noise vector $b \in \mathbb{R}^d$ have density $\nu(b) \propto e^{-\|b\|/s}$, with scale $s > 0$. If f has ℓ_2 sensitivity $\Delta_2 f$ and $s = \Delta_2 f/\varepsilon$, then $f(D) + b$ is ε-DP. The length $\|b\|$ has the gamma distribution with shape d and scale s, the direction is uniform and independent of the length, and*

$$\mathbb{E}\|b\| = ds, \qquad \mathbb{E}\|b\|^2 = d(d+1)s^2. \tag{38.3}$$

Proof.

1. *Privacy.* At an output y the two densities share their constant, so their ratio is $e^{(\|y-f(D')\|-\|y-f(D)\|)/s}$. By the triangle inequality the difference of distances is at most $\|f(D) - f(D')\| \leq \Delta_2 f$. So the ratio is at most $e^{\Delta_2 f/s} = e^{\varepsilon}$ at every output, which is pure ε-DP by the pointwise criterion of the definition of differential privacy.
2. *Length and direction.* The density depends on b only through its length r. The sphere of radius r has surface area proportional to r^{d-1}, so the length has density proportional to

$$r^{d-1}e^{-r/s}, \tag{38.4}$$

   the gamma density with shape d and scale s. At a given length every direction has the same density, so the direction is uniform and independent of the length.
3. *Moments.* A gamma variable with shape d and scale s has mean ds and variance ds^2. The second moment is the variance plus the squared mean, $ds^2 + d^2s^2 = d(d+1)s^2$.

□

A Gaussian vector's squared length has mean $d\sigma^2$, against $d(d+1)s^2$ here, so in high dimension the pure-DP noise is longer by a factor that grows with d.

Theorem 38.1 (Output perturbation)**.** *Let each loss $\ell(\cdot; z)$ be convex and differentiable, with gradients of length at most L, let $\lambda > 0$, and let neighbours replace one record. Release*

$$\tilde{\theta} = \theta^*(D) + Z, \qquad Z \sim \mathcal{N}(0, \sigma^2 I), \qquad \sigma = \frac{2L}{\lambda n \, \mu(\varepsilon, \delta)}. \tag{38.5}$$

This release is (ε, δ)-DP. Replacing Z by noise b with density $\propto e^{-\|b\|/s}$ and $s = 2L/(\lambda n \varepsilon)$ gives pure ε-DP.

Proof.

1. *Sensitivity.* By Theorem 6.5 the minimizer moves by at most $\Delta_2 \theta^* = 2L/(\lambda n)$ when one record is replaced.
2. *Gaussian noise.* With $\sigma = \Delta_2 \theta^* / \mu(\varepsilon, \delta)$ the ratio of sensitivity to noise is $\mu(\varepsilon, \delta)$, which satisfies Equation 38.2, so the release is (ε, δ)-DP.
3. *Pure noise.* With $s = \Delta_2 \theta^* / \varepsilon$, Lemma 38.1 makes the release ε-DP.

□

How much accuracy does the noise cost? Section M.1 answers exactly, and a second-order answer is simpler. Near the minimizer the objective is a bowl, whose shape is given by the **Hessian** $H = \nabla^2 \mathcal{L}(\theta^*)$, the matrix of second derivatives. A small random step costs half its squared length times the bowl's curvature averaged over the step's direction.

Proposition 38.1 (Excess risk of output perturbation)**.** *Let $\mathcal{L}$ be twice differentiable with Hessian H at θ^*, and let the noise Z have a uniformly random direction, independent of its length. To second order in Z,*

$$\mathbb{E}[\mathcal{L}(\tilde{\theta}) - \mathcal{L}(\theta^*)] \approx \frac{1}{2} \mathbb{E}[Z^\top H Z] = \frac{\mathbb{E}\|Z\|^2}{2d} \operatorname{tr} H, \tag{38.6}$$

which is $\frac{\sigma^2}{2} \operatorname{tr} H$ for the Gaussian.

Proof.

1. *Expand around the minimizer.* The gradient vanishes at θ^*, so a second-order Taylor expansion leaves only the quadratic term,

$$\mathcal{L}(\theta^* + Z) - \mathcal{L}(\theta^*) \approx \tfrac{1}{2} Z^\top H Z. \tag{38.7}$$

2. *Average over the direction.* Write $Z = \|Z\| u$ with u a unit vector. The d squared coordinates of u add up to 1 and are exchangeable, so each averages $1/d$. Flipping the sign of one coordinate leaves a uniform direction uniform, so the product of two different coordinates averages to zero. Hence

$$\mathbb{E}[u^\top H u] = \frac{1}{d} \sum_i H_{ii} = \frac{\operatorname{tr} H}{d}. \tag{38.8}$$

3. *Put the length back.* The length is independent of the direction, so $\mathbb{E}[Z^\top H Z] = \mathbb{E}\|Z\|^2 \operatorname{tr} H/d$. For the Gaussian $\mathbb{E}\|Z\|^2 = d\sigma^2$.

□

Every solver stops at an approximate minimizer, so the theorem's exact θ^* never exists in practice. What a solver can guarantee is that the gradient's length is below a tolerance τ, and the next result adds that slack to the sensitivity.

Proposition 38.2 (Sensitivity of an approximate minimizer)**.** *In the setting of Theorem 38.1, let the optimizer return, on every dataset, a point θ with $\|\nabla \mathcal{L}(\theta; D)\| \le \tau$. Then its outputs θ and θ' on two neighbouring datasets satisfy*

$$\|\theta - \theta'\| \le \frac{2L}{\lambda n} + \frac{2\tau}{\lambda}. \tag{38.9}$$

Proof.

1. *Each output is near its minimizer.* The objective is λ-strongly convex, so part (a) of Corollary 6.1 puts θ within τ/λ of $\theta^*(D)$, and θ' within τ/λ of $\theta^*(D')$.
2. *The minimizers are near each other.* By Theorem 6.5 they are at most $2L/(\lambda n)$ apart.
3. *Add the three distances.* The triangle inequality along the path $\theta \to \theta^*(D) \to \theta^*(D') \to \theta'$ gives Equation 38.9.

□

The noise is calibrated to the worst case, and the damage is measured on the actual case. The sensitivity uses only the ridge's curvature λ, while the damage is weighted by H, which includes the data's own. If every eigenvalue of H is close to some h, then $\operatorname{tr} H \approx dh$, and substituting the σ of Equation 38.5 into $\frac{\sigma^2}{2} \operatorname{tr} H$ gives a Gaussian excess risk of about $\frac{2dL^2}{\mu^2 n^2} \cdot \frac{h}{\lambda^2}$, so a small ridge buys a better non-private fit and costs a lot of noise. The excess risk falls like $1/n^2$, faster than the error of order $1/n$ that sampling causes even without privacy, so for large n privacy costs less than sampling does.

The method also needs a nearly exact solution. To keep the extra term of Equation 38.9 below the exact one, the optimizer must reach $\tau \le L/n$, a tolerance that shrinks with the dataset. The stopping rule must be a gradient check, not a step count, because a fixed number of steps may leave a large gradient on some dataset, and the proof needs the bound on every one. Wu et al. (2017) bound the sensitivity of stochastic gradient descent (SGD) run for a fixed number of passes instead.

Example 38.1. Start with one feature, $n = 20$ records and $\lambda = 0.5$ (Figure 38.1). Replacing the last record, which has $x = 1$, by the same feature with the opposite label moves the minimizer from 0.2128 to 0.1296, by 0.0832. With $L = 1$, since a logistic gradient is never longer than its unit feature, the bound is $2/(0.5 \times 20) = 0.2$, and the noise is calibrated to the bound, $\sigma = 3.7306 \times 0.2 = 0.7461$, so the two output densities overlap almost completely.

In the running example ($d = 10$, $\lambda = 0.01$, $L = 1$), $n = 10{,}000$ gives sensitivity $2/(0.01 \times 10{,}000) = 0.02$ and $\sigma = 0.07461$ per coordinate, next to a minimizer of length 2.886. The Hessian

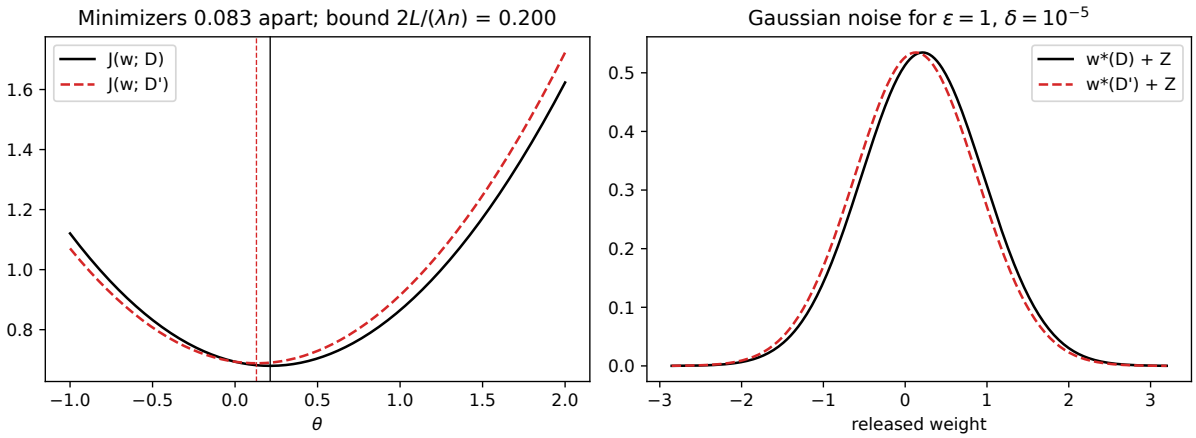


Figure 38.1.: Left: two neighbouring regularized logistic objectives in one dimension ($n = 20$, $\lambda = 0.5$), their minimizers (vertical lines) 0.083 apart, below the bound $2L/(\lambda n) = 0.2$. Right: the released weight's density under each dataset with Gaussian noise for $\varepsilon = 1$, $\delta = 10^{-5}$; the noise must hide a shift as large as the bound, not the actual shift.

at θ^* has eigenvalues between 0.0263 and 0.0330, and trace 0.3129. Equation 38.6 predicts $\frac{1}{2} \times 0.07461^2 \times 0.3129 = 8.711 \times 10^{-4}$, and 20 simulated releases average 9.274×10^{-4}. The pure-DP version, with $s = 0.02$, predicts 6.885×10^{-4} and averages 8.321×10^{-4}. The gap is wider because the squared length of gamma noise varies a lot, and 20 draws average it only roughly.

Stopping early is costly. A gradient tolerance of $\tau = 10^{-6}$ gives sensitivity 0.0202, essentially exact, and $\tau = 10^{-4}$ doubles it to 0.04. At $\tau = 10^{-3}$ it is 0.22, eleven times the exact value, which multiplies the excess risk by about 121.

38.2. Objective perturbation

Output perturbation wastes the data's curvature, because it pays for the worst case λ and is judged on the actual H. Chaudhuri, Monteleoni & Sarwate (2011) put the noise inside the optimization instead. This section asks whether a randomly tilted objective, minimized exactly, is private, and what it costs.

The tilt is a random linear term $\frac{1}{n}\langle b, \theta\rangle$. It acts as a constant force, and all of the objective's curvature resists it, so where the data add curvature the minimizer moves less. The output is no longer something fixed plus noise, so the proof must track how the minimization stretches the noise. It bounds that stretching only when each record's Hessian has rank one, so that replacing a record changes it by little. So the loss must depend on θ through one inner product, $\ell(\theta; (x, y)) = \psi(y\langle\theta, x\rangle)$, for a scalar function ψ of the **margin** $m = y\langle\theta, x\rangle$. Logistic regression has $\psi(m) = \ln(1 + e^{-m})$, whose derivative lies in $(-1, 0)$ and whose second derivative is at most $c = \frac{1}{4}$. The budget ε is split in two. One part pays for the difference in stretching between the two datasets, which the bound c on ψ'' controls, and the rest, written ε', goes to the noise.

Theorem 38.2 (Objective perturbation)**.** *Let the loss be $\ell(\theta;(x,y)) = \psi(y\langle\theta,x\rangle)$, with ψ convex and twice differentiable, $|\psi'| \le 1$ and $0 \le \psi'' \le c$, and let every feature have $\|x\| \le 1$. Let $\lambda > 0$ and let neighbours replace one record. Set*

$$\varepsilon' = \varepsilon - 2\ln\Big(1 + \frac{c}{n\lambda}\Big), \tag{38.10}$$

draw b with density $\nu(b) \propto e^{-\varepsilon'\|b\|/2}$, and release

$$\tilde\theta = \arg\min_\theta \Big[\mathcal{L}(\theta;D) + \frac{1}{n}\langle b,\theta\rangle\Big]. \tag{38.11}$$

If $\varepsilon' > 0$, the release is ε-DP (Chaudhuri, Monteleoni & Sarwate 2011, Algorithm 2). If $\varepsilon' \le 0$, adding $\lambda_+ = \frac{c}{n(e^{\varepsilon/4}-1)} - \lambda$ to the ridge and drawing the noise with $\varepsilon/2$ in place of ε' makes it ε-DP.

Proof. The plan is to write the density of $\tilde\theta$ explicitly, then bound the ratio of the two densities, for D and D', at every point. The ratio splits into a noise factor and a stretching factor, and each consumes part of ε.

1. *From the output back to the noise.* The perturbed objective is λ-strongly convex, so it has exactly one minimizer, where its gradient is zero: $\nabla\mathcal{L}(\tilde\theta;D) + b/n = 0$. Solving for b gives

$$b = \phi_D(\tilde\theta) = -n\nabla\mathcal{L}(\tilde\theta;D) = -\sum_{i=1}^{n}\nabla\ell(\tilde\theta;z_i) - n\lambda\tilde\theta. \tag{38.12}$$

   Every θ is the output for exactly one noise vector, $\phi_D(\theta)$, so the map from noise to output is invertible and ϕ_D is its inverse.
2. *The density of the output.* When a random vector with density ν is mapped smoothly and invertibly, the image has density ν at the preimage times the absolute determinant of the inverse map's Jacobian. The determinant corrects for stretching: a small box around θ and its preimage around b carry the same probability, so their densities differ by the ratio of their volumes. Applied to D and D', the ratio of the two output densities at any θ is

$$\frac{p_D(\theta)}{p_{D'}(\theta)} = \underbrace{\frac{\nu(\phi_D(\theta))}{\nu(\phi_{D'}(\theta))}}_{\text{noise}} \cdot \underbrace{\frac{|\det\nabla\phi_D(\theta)|}{|\det\nabla\phi_{D'}(\theta)|}}_{\text{Jacobian}}. \tag{38.13}$$

3. *The noise factor is at most $e^{\varepsilon'}$.* The sums in ϕ_D and $\phi_{D'}$ share every term except the replaced record's, so $\phi_{D'}(\theta) - \phi_D(\theta) = \nabla\ell(\theta;z) - \nabla\ell(\theta;z')$, of length at most 2, since each gradient $\psi'(m)\,y\,x$ has length at most $|\psi'|\,\|x\| \le 1$. This is the setting of Lemma 38.1 with sensitivity 2 and scale $s = 2/\varepsilon'$, so the noise factor is at most $e^{2/s} = e^{\varepsilon'}$.
4. *The Jacobian is the Hessian.* Differentiating ϕ_D gives $\nabla\phi_D(\theta) = -A$, with

$$A = \sum_i \psi''(m_i)\,x_i x_i^\top + n\lambda I, \tag{38.14}$$

   where m_i is the margin of record i. The chain rule gives each loss's Hessian as $\psi''(m_i)$ times $y_i^2 x_i x_i^\top = x_i x_i^\top$. Every term is positive semi-definite, so all eigenvalues of A are at least $n\lambda$. For D' the matrix is $A' = A + E$, with $E = \psi''(m')x'x'^\top - \psi''(m)xx^\top$: one rank-one term in, one out.

5. *The Jacobian factor is at most* $(1 + c/(n\lambda))^2$. The factor is $\det(A)/\det(A')$. Bound first its reciprocal, $\det(A')/\det(A)$, which is $\det(I + A^{-1}E)$, which equals $\det(I + K)$ for the similar, symmetric matrix $K = A^{-1/2}EA^{-1/2}$. With $\tilde{x} = A^{-1/2}x$, K is the difference of the rank-one terms $\psi''(m')\tilde{x}'\tilde{x}'^{\top}$ and $\psi''(m)\tilde{x}\tilde{x}^{\top}$. A rank-one matrix $vv^{\top}$ has one nonzero eigenvalue, $\|v\|^2$, and here $\|\tilde{x}\|^2 = x^{\top}A^{-1}x \le 1/(n\lambda)$. So K has at most two nonzero eigenvalues κ_1 and κ_2. The term added has one eigenvalue, $\psi''(m')\|\tilde{x}'\|^2 \le c/(n\lambda)$, and can raise no eigenvalue by more than that. The term subtracted can likewise lower none by more than $c/(n\lambda)$. So each κ lies between $-a$ and a, with $a = c/(n\lambda)$. Hence

$$\det(I + K) = (1 + \kappa_1)(1 + \kappa_2) \le (1 + a)^2. \tag{38.15}$$

   The same argument with D and D' exchanged bounds $\det(A)/\det(A')$, the Jacobian factor itself, by $(1 + a)^2$, because A' also has all eigenvalues at least $n\lambda$.
6. *Add the two budgets.* The log of the density ratio is at most $\varepsilon' + 2\ln(1 + c/(n\lambda))$, which is ε by Equation 38.10, at every θ and in both directions. A bound on the density ratio everywhere is pure ε-DP.
7. *When* $\varepsilon' \le 0$. The Jacobian factor alone would then exceed ε, and a larger ridge shrinks it. With $\lambda + \lambda_+$ in place of λ, $a = e^{\varepsilon/4} - 1$, so the Jacobian takes $2\ln(1 + a) = \varepsilon/2$ and the noise gets the other half.

□

The analysis is safe but not tight. The noise factor reaches $e^{\varepsilon'}$ only if both records have full-length gradients in opposite directions. A logistic gradient has full length only at a very negative margin, and two records whose gradients point in opposite directions have margins of opposite sign, so at most one of them can. The Jacobian bound charges both records the full curvature c, but the record that arrives raises the determinant while the one that leaves lowers it, so $\ln(1 + a)$ suffices, half the charge.

The method needs exact minimization, which no computer delivers, and even rounding can leak (Discrete mechanisms). It also needs twice-differentiable losses, so a support vector machine's hinge loss must be smoothed first. Iyengar et al. (2019) tolerate inexact solvers by adding a little output noise.

The excess risk follows the second-order argument of output perturbation, except that the noise now enters as a force, which the curvature H converts into a displacement by dividing by it.

Proposition 38.3 (Excess risk of objective perturbation)**.** *Under the assumptions of Proposition 38.1, with the noise b of Theorem 38.2, to second order in b,*

$$\mathbb{E}\big[\mathcal{L}(\tilde{\theta}) - \mathcal{L}(\theta^*)\big] \approx \frac{1}{2n^2}\mathbb{E}\big[b^{\top}H^{-1}b\big] = \frac{\mathbb{E}\|b\|^2}{2dn^2}\operatorname{tr} H^{-1}. \tag{38.16}$$

Proof.

1. *The displacement.* To first order in b the gradient of $\mathcal{L}$ at $\tilde{\theta}$ is $H(\tilde{\theta} - \theta^*)$, and the perturbed minimizer makes it cancel the force:

$$\tilde{\theta} - \theta^* = -\frac{1}{n}H^{-1}b. \tag{38.17}$$

2. *Its cost.* Substituting the displacement into the quadratic of step 1 of Proposition 38.1 gives $\frac{1}{2n^2} b^\top H^{-1} H H^{-1} b = \frac{1}{2n^2} b^\top H^{-1} b$.
3. *Average over the direction.* The direction of b is uniform and independent of its length (Lemma 38.1), so step 2 of Proposition 38.1 applies with H^{-1} in place of H.

□

Curvature now helps. Output perturbation's noise scales like $1/(\lambda n)$ and is weighted by H, while here the noise does not depend on n or λ, is divided by n, and is weighted by H^{-1}. If every eigenvalue of H is close to h, pure-DP output perturbation's excess risk is about $(h/\lambda)^2$ times objective perturbation's, when $\varepsilon' \approx \varepsilon$, because both use the same noise shape. When the ridge supplies all the curvature the two are equivalent, and when the data supply more, objective perturbation wins quadratically.

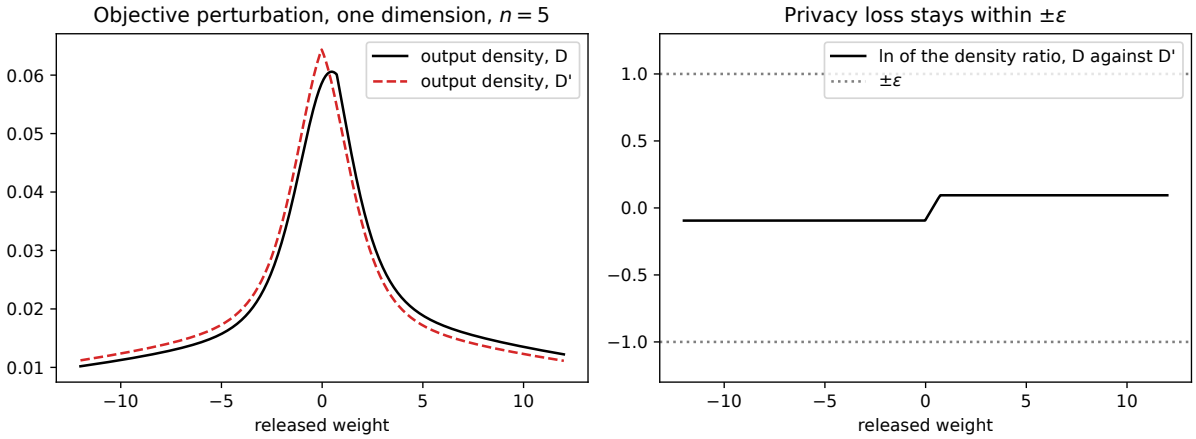


Figure 38.2.: Objective perturbation in one dimension, $n = 5$, $\lambda = 0.1$, $\varepsilon = 1$. Left: the exact densities of the released weight under a dataset and its neighbour, computed by change of variables. Right: their log ratio, the privacy loss, which stays well inside $\pm\varepsilon$.

Example 38.2. In one dimension the chapter's script computes both output densities exactly on a grid. Take logistic loss ($c = \frac{1}{4}$) on $n = 5$ records with features in $[-1, 1]$, $\lambda = 0.1$ and $\varepsilon = 1$. Then $c/(n\lambda) = 0.5$, the Jacobian budget is $2 \ln 1.5 = 0.8109$, and the noise gets $\varepsilon' = 0.1891$ (Figure 38.2). Over 500 random datasets and replacements, the largest log ratio anywhere on the grid is 0.3779. Its noise part never exceeds 0.0992, about half its budget of 0.1891, and its Jacobian part never exceeds 0.3395, below even $\ln 1.5 = 0.4055$, which is half the Jacobian budget of 0.8109.

In the running example at $n = 10{,}000$, the Jacobian budget is $2 \ln(1 + 0.25/100) = 0.0050$, so the noise gets $\varepsilon' = 0.9950$. The noise has $\mathbb{E}\|b\|^2 = d(d+1)(2/\varepsilon')^2$, and $\operatorname{tr} H^{-1} = 320.7$, so Equation 38.16 predicts an excess risk of 7.127×10^{-5}, and 20 simulated releases average 8.156×10^{-5}. Pure-DP output perturbation with the same ε had 8.321×10^{-4}, so objective perturbation is ten times better. The rule $(h/\lambda)^2$ predicts this, because the objective's curvature, between 0.0263 and 0.0330, is about three times the ridge's 0.01, and $3^2 = 9$.

38.3. Gradient perturbation: noisy gradient descent

The first two methods need the exact minimizer, but real training is iterative and stops early. Gradient perturbation, proposed by Williams & McSherry (2010), makes each step of the optimizer private and composes the steps. This section asks how much noise per step makes a whole run of full-batch gradient descent private, and how accurate the run is.

Each step will be a Gaussian mechanism applied at a point that earlier steps chose. T such adaptive Gaussian mechanisms, with ratios $\mu_1, \dots, \mu_T$, are exactly as private as one Gaussian mechanism with ratio $\sqrt{\mu_1^2 + \cdots + \mu_T^2}$ (f-DP and Gaussian differential privacy). Given the past, each step's privacy loss is normal with variance μ_t^2 and mean $\mu_t^2/2$, so their sum is normal with variance $\sum_t \mu_t^2$ and half that as its mean, which is the privacy loss of one Gaussian mechanism.

Theorem 38.3 (Noisy gradient descent)**.** *Let every per-record gradient have length at most L, with the loss convex or not, and let neighbours replace one record. Start from $\theta_0 = 0$ and run T steps of*

$$\theta_{t+1} = \theta_t - \eta(\nabla \mathcal{L}(\theta_t; D) + \xi_t), \qquad \xi_t \sim \mathcal{N}(0, \sigma^2 I) \text{ independent}, \qquad \sigma = \frac{2L}{n} \cdot \frac{\sqrt{T}}{\mu(\varepsilon, \delta)}, \tag{38.18}$$

with learning rate η. Releasing all the iterates $\theta_1, \dots, \theta_T$ is (ε, δ)-DP.

Proof.

1. *One step is a Gaussian mechanism.* Given θ_t, the step releases $\nabla \mathcal{L}(\theta_t; D) + \xi_t$. The ridge term $\lambda\theta_t$ is the same on both datasets, and the averages differ only in the replaced record. So the gradient moves by at most $\frac{1}{n}\|\nabla \ell(\theta_t; z) - \nabla \ell(\theta_t; z')\| \le 2L/n$, and the step's ratio is $\mu_{\text{step}} = 2L/(n\sigma)$. The point θ_t depends on the data only through earlier released outputs, which is what adaptive composition allows.
2. *Compose.* By the composition of Gaussian mechanisms, the T steps are as private as one Gaussian with ratio $\sqrt{T} \cdot 2L/(n\sigma)$. Setting this to $\mu(\varepsilon, \delta)$ gives the σ of Equation 38.18. The iterates are computed from the noisy gradients and θ_0 alone, so releasing them is post-processing.
3. *No convexity was used.* Steps 1 and 2 bounded the gradient's change by $2L/n$ at the point θ_t, whatever θ_t is. Nothing required a minimizer, convexity or smoothness.

□

Convexity enters only for accuracy. On a λ-strongly convex, β-**smooth** objective, one whose curvature lies between λ and β everywhere, gradient descent with learning rate $\eta = 1/\beta$ shrinks the error by a fixed factor per step, while the noise keeps the iterate from settling.

Proposition 38.4 (Excess risk of noisy gradient descent)**.** *Let the objective be λ-strongly convex and β-smooth, and let $\eta = 1/\beta$. The excess risk of the last iterate is approximately*

$$\mathbb{E}[\mathcal{L}(\theta_T) - \mathcal{L}(\theta^*)] \approx \left(1 - \frac{\lambda}{\beta}\right)^T (\mathcal{L}(\theta_0) - \mathcal{L}(\theta^*)) + \frac{d\sigma^2}{4\beta}. \tag{38.19}$$

Proof.

1. *Optimization error.* Without noise, each step shrinks $\mathcal{L}(\theta_t) - \mathcal{L}(\theta^*)$ by at least a factor $1-\lambda/\beta$. Theorem 6.1 proves the same factor for the distance on a quadratic, and Section G.1 proves it for the excess risk of every such objective. Over T steps this gives the first term.
2. *How the noise moves the error.* Near the minimizer the gradient is about $H(\theta_t - \theta^*)$, so the error $e_t = \theta_t - \theta^*$ evolves as
$$e_{t+1} \approx (I - \eta H)e_t - \eta\xi_t. \tag{38.20}$$
3. *The steady state in one direction.* Along an eigenvector of H with curvature h, the error's variance v settles where one step leaves it unchanged, $v = (1-\eta h)^2 v + \eta^2\sigma^2$. Solving gives $v = \eta\sigma^2/(h(2-\eta h))$.
4. *Its cost.* That direction costs $\frac{1}{2}hv = \eta\sigma^2/(2(2-\eta h))$, about $\eta\sigma^2/4$ when ηh is small and at most twice that when ηh reaches 1. Summing over the d directions with $\eta = 1/\beta$ gives $d\sigma^2/(4\beta)$.

□

The curvature h cancels in step 4, because flat directions wander farther but cost less per unit of wandering. What does not cancel is T, so the step count is a privacy parameter: every step spends budget, so the noise floor grows like T, while too few steps leave optimization error (Figure 38.3). Equation 38.19 can be minimized from public quantities: λ, the curvature bound $\beta = \frac{1}{4} + \lambda$ of logistic loss with unit features, $\mathcal{L}(\theta_0) = \ln 2$ at $\theta_0 = 0$, d, n and the budget. This **public rule** for T is pessimistic, because it knows only that the curvature is at least λ. Picking T by validation score is itself a computation on the data (Hyperparameter tuning).

Example 38.3. For $n = 10{,}000$, the public rule picks $T = 218$. That gives per-step noise $\sigma = 0.01102$ on the mean gradient and an estimated excess risk of 1.301×10^{-3}, and simulated runs average 1.384×10^{-3}. Table 38.1 gives the rule's choice at every dataset size. The step count grows slowly, like $\ln n$, because the optimization error shrinks geometrically while its target, the noise floor, shrinks like $1/n^2$.

Table 38.1.: Noisy gradient descent on the running example: the step count chosen by minimizing Equation 38.19 with public quantities only, the resulting noise, and the estimated excess risk.

n	Steps T (public rule)	Per-step σ on the mean gradient	Estimated excess risk
1,000	100	0.07461	6.725×10^{-2}
3,000	156	0.03106	1.080×10^{-2}
10,000	218	0.01102	1.301×10^{-3}
30,000	274	0.004117	1.779×10^{-4}
100,000	335	0.001366	1.930×10^{-5}

Figure 38.3 shows that the rule overshoots, because the data's actual curvature, between 0.0263 and 0.0330, is well above 0.01. Over 10 datasets the excess risk is smallest near $T = 50$, at 2.875×10^{-4}, five times less than the rule's, and it rises to 6.018×10^{-3} at $T = 1{,}000$. But you may use that only if you learned it privately or from similar public data.

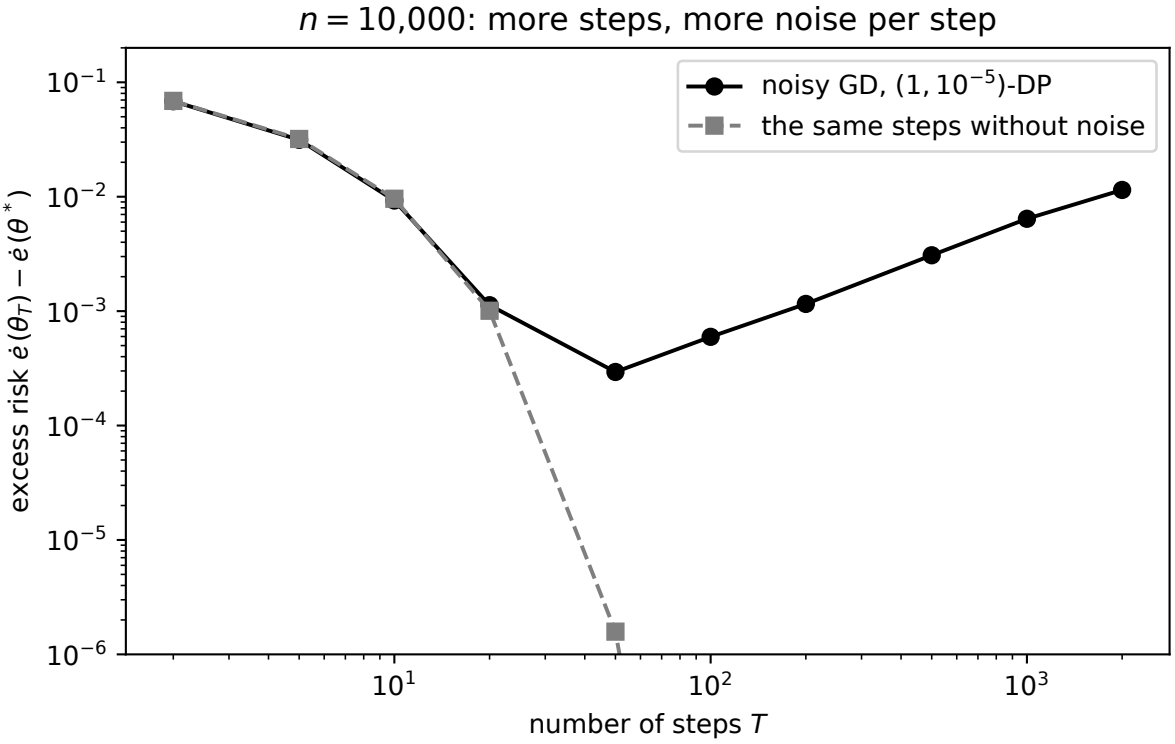


Figure 38.3.: Excess risk of noisy gradient descent at $n = 10{,}000$, $(\varepsilon, \delta) = (1, 10^{-5})$, against the number of steps T, averaged over datasets, with the same steps without noise (dashed). The noiseless error falls geometrically and reaches machine precision; the noisy run is limited by noise that grows with T, so its error is smallest near $T = 50$.

38.4. Noisy projected gradient descent without a ridge

Every method so far leaned on the ridge, for its privacy or for its accuracy. But a linear classifier trained without a ridge inside a fixed ball of weights is convex with bounded gradients, not strongly convex. This section asks how accurate a private trainer can be on such a model, and where the dimension enters.

The tool is the guarantee of projected SGD from Optimization. Projected SGD keeps the weights in a closed convex set $\mathcal{W}$ fixed in advance, here the ball of radius R, by replacing each new point with its **projection** $\Pi_{\mathcal{W}}$, the nearest point of the set. It steps along a random vector g_t that is right on average given the past, with squared length at most G^2 on average. If the objective is convex and its minimizer over $\mathcal{W}$ lies within R of the start, then T steps with learning rate $\eta = R/(G\sqrt{T})$ make the average $\bar{\theta}_T$ of the visited points satisfy

$$\mathbb{E}[\mathcal{L}(\bar{\theta}_T)] - \mathcal{L}(\theta^*) \le \frac{RG}{\sqrt{T}} \tag{38.21}$$

(Theorem 6.2). Independent noise with mean zero only adds its second moment to G^2 (Proposition 6.4).

Theorem 38.4 (Noisy projected gradient descent)**.** *Let the loss be convex in θ with per-record gradients of length at most L, let the objective be the plain average $\mathcal{L}(\theta; D) = \frac{1}{n}\sum_i \ell(\theta; z_i)$, with no ridge, and let neighbours replace one record. Keep the weights in the ball $\mathcal{W} = \{\|\theta\| \le R\}$. Start from $\theta_1 = 0$ and run T steps of*

$$\theta_{t+1} = \Pi_{\mathcal{W}}\Big(\theta_t - \eta\big(\nabla\mathcal{L}(\theta_t; D) + \xi_t\big)\Big), \qquad \xi_t \sim \mathcal{N}(0, \sigma^2 I), \qquad \sigma = \frac{2L}{n}\cdot\frac{\sqrt{T}}{\mu(\varepsilon,\delta)}, \tag{38.22}$$

with $\eta = R/(G\sqrt{T})$ and $G^2 = L^2 + d\sigma^2$, and release $\bar{\theta} = \frac{1}{T}\sum_{t=1}^{T}\theta_t$. The release is (ε,δ)-DP, and its excess risk over the best weights in the ball is at most

$$\mathbb{E}[\mathcal{L}(\bar{\theta}; D)] - \min_{\theta\in\mathcal{W}} \mathcal{L}(\theta; D) \le RL\sqrt{\frac{1}{T} + \frac{4d}{n^2\mu(\varepsilon,\delta)^2}}. \tag{38.23}$$

Once $T \ge n^2\mu^2/(4d)$ this is at most $2\sqrt{2}\,RL\sqrt{d}/(n\mu)$, and for $\varepsilon < 1$

$$\mathbb{E}[\mathcal{L}(\bar{\theta}; D)] - \min_{\theta\in\mathcal{W}} \mathcal{L}(\theta; D) \le \frac{4RL\sqrt{d\ln(1.25/\delta)}}{n\varepsilon}. \tag{38.24}$$

Proof. The plan is to prove privacy as for noisy gradient descent, check the two conditions of the SGD guarantee for the noisy gradient, and then see that the step count drops out of the noise term.

1. *Privacy.* Each step releases a noisy gradient whose noiseless part moves by at most $2L/n$, as in step 1 of the proof of Theorem 38.3, and the T steps compose into one Gaussian with ratio $\mu(\varepsilon,\delta)$. The projection uses only the public ball, and the average uses only the iterates, so both are post-processing.
2. *The noisy gradient is unbiased.* Write $g_t = \nabla\mathcal{L}(\theta_t; D) + \xi_t$. Given the past, ξ_t is fresh noise with mean zero, so g_t averages to the true gradient.

3. *Its second moment is at most $L^2 + d\sigma^2$.* The gradient of $\mathcal{L}$ is an average of vectors of length at most L, so its squared length is at most L^2. The noise is independent with mean zero, so by Proposition 6.4 it adds its own second moment, one σ^2 per coordinate, $d\sigma^2$ in all.
4. *Apply the SGD guarantee.* The minimizer over the ball and the start $\theta_1 = 0$ both lie in the ball, so they are at most R apart. With $\eta = R/(G\sqrt{T})$, Theorem 6.2 bounds the excess risk of $\bar{\theta}$ by
$$\frac{RG}{\sqrt{T}} = R\sqrt{\frac{L^2 + d\sigma^2}{T}}. \tag{38.25}$$
5. *The step count cancels from the noise term.* By Equation 38.22, $\sigma^2 = 4L^2T/(n^2\mu^2)$, so $d\sigma^2/T = 4L^2d/(n^2\mu^2)$ whatever T is. Factoring out L^2 gives Equation 38.23.
6. *Choose T.* The first term under the root falls with T and the second does not. When $T \geq n^2\mu^2/(4d)$ the first is at most the second, and the root is at most $\sqrt{8d/(n^2\mu^2)} = 2\sqrt{2}\,\sqrt{d}/(n\mu)$.
7. *In terms of ε.* For $\varepsilon < 1$ the classic noise $\Delta_2 f\sqrt{2\ln(1.25/\delta)}/\varepsilon$ is (ε, δ)-DP (Theorem 20.2). The exact calibration uses the least noise that is (ε, δ)-DP, so its ratio is at least the classic one,
$$\frac{1}{\mu} \leq \frac{\sqrt{2\ln(1.25/\delta)}}{\varepsilon}. \tag{38.26}$$
Substituting this into step 6 gives $2\sqrt{2} \cdot \sqrt{2} = 4$ in front, which is Equation 38.24.

□

Privacy noise is just more gradient variance. The SGD guarantee cares only about the second moment of g_t, and Gaussian noise raises it by one σ^2 for every coordinate the optimizer moves, which Public data and private fine-tuning exploits by training fewer coordinates. Unlike in Section 38.3, more steps never hurt, because step 5 cancels the growth of the noise. The step count only has to reach $n^2\mu^2/(4d)$, but each full-batch step costs n gradient evaluations, so Bassily, Smith & Thakurta use one random record per step instead, which by Amplification by subsampling also needs less noise.

The rate cannot be beaten. As long as $\sqrt{d} \leq n\varepsilon$, every (ε, δ)-DP algorithm has excess risk of order $RL\sqrt{d}/(n\varepsilon)$ on some dataset, by the fingerprinting argument of Lower bounds (Bassily, Smith & Thakurta 2014). Section M.1 derives this lower bound with explicit constants, and Equation 38.24 matches it up to $\sqrt{\ln(1/\delta)}$ and constants. It is a worst case, reached by losses nearly linear near their minimizer.

Example 38.4. Take the running example's data, with $d = 10$ and $L = 1$, drop the ridge, and keep the weights in the ball of radius $R = 5$. At $\varepsilon = 1$ and $\delta = 10^{-5}$ the exact ratio is $\mu = 0.2681$, and the classic ratio $\varepsilon/\sqrt{2\ln(1.25/\delta)}$ is smaller, 0.2064, as step 7 requires. With a thousand records the rule $T = \lceil n^2\mu^2/(4d) \rceil$ gives 1,797 steps, and the noise on the mean gradient is $\sigma = 0.3163$. The noise adds $d\sigma^2 = 10 \times 0.3163^2$ to the second moment, so $G^2 = 2.0004$, about twice the noiseless $L^2 = 1$. The learning rate is then 0.0834, and Equation 38.23 evaluates to 0.1668.

As T grows, the first term under the root vanishes and the bound falls to $2RL\sqrt{d}/(n\mu) = 0.1180$, a factor $\sqrt{2}$ below its value at the threshold. It is 0.5137 after 100 steps, 0.1973 after 1,000 and 0.1190 after 100,000, within 1% of that limit. Table 38.2 runs the algorithm for three dataset sizes, five datasets each.

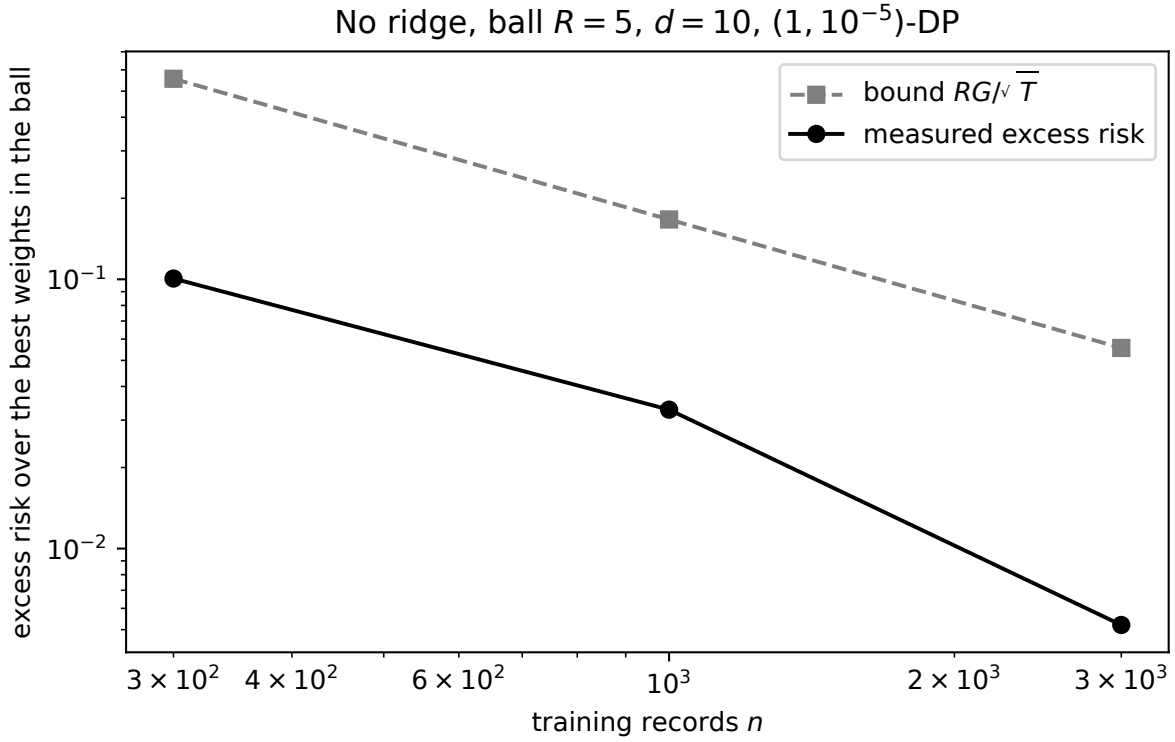


Figure 38.4.: Noisy projected gradient descent on unregularized logistic regression in the ball of radius 5, $d = 10$, $(1, 10^{-5})$-DP, with $T = \lceil n^2\mu^2/(4d) \rceil$ steps: the measured excess risk over the best weights in the ball (circles) against the bound Equation 38.23 (squares), averaged over 3 datasets drawn separately from those of the table. The bound falls like $1/n$; the measured excess risk falls faster.

Table 38.2.: Noisy projected gradient descent without a ridge on the running example's data, ball of radius 5, $(1, 10^{-5})$-DP: mean excess risk over 5 datasets against the bound.

n	Steps T	σ	Measured excess risk	Bound Equation 38.23	Bound / measured
300	162	0.3166	1.119×10^{-1}	5.558×10^{-1}	5.0
1,000	1,797	0.3163	3.028×10^{-2}	1.668×10^{-1}	5.5
3,000	16,167	0.3162	7.297×10^{-3}	5.561×10^{-2}	7.6

The bound holds at every n and is loose by a factor of 5 to 8. Its log-log slope in n is -1.00, while the measured slope is -1.18, steeper because the logistic loss has curvature near its minimizer that the bound does not use (Figure 38.4). The rule's σ is the same at every n, because $T \propto n^2$ and $\sigma \propto \sqrt{T}/n$.

38.5. How the three compare

Each method is private by its own argument, so which should you use? Theory answers by rates, how the excess risk falls with n, d and ε, and simulation answers for the running example. This section shows that they agree on the rate and differ in the constant, which is what you choose by.

With a guaranteed curvature the rate improves. The convex rate of Section 38.4 falls like $\sqrt{d}/(n\varepsilon)$. For a λ-strongly convex objective the rate squares this to $d/(n\varepsilon)^2$ and divides it by λ.

Theorem 38.5 (Optimal rate for strongly convex objectives)**.** *For λ-strongly convex objectives whose losses have gradients of length at most L, the best expected excess risk achievable under (ε, δ)-DP is of order*

$$\frac{L^2\, d}{\lambda\, n^2 \varepsilon^2}, \tag{38.27}$$

up to logarithmic factors in n and $1/\delta$.

Bassily, Smith & Thakurta (2014) prove both sides, the upper bound with noisy stochastic gradient descent and the lower bound with fingerprinting codes, and Section M.1 proves both with explicit constants. Output and objective perturbation fall at this rate in n because their noise has second moment proportional to $1/n^2$. Noisy gradient descent loses a logarithm, because pushing its optimization error below a noise floor $\propto T/n^2$ takes $T \propto \ln n$ steps.

Table 38.3.: Mean excess risk $\mathcal{L}(\tilde{\theta}) - \mathcal{L}(\theta^*)$ over 20 datasets of the running example ($d = 10$, $\lambda = 0.01$, $\varepsilon = 1$, $\delta = 10^{-5}$ for the Gaussian methods), and the slope of a straight-line fit of ln excess risk against $\ln n$.

n	Output, Gaussian	Output, pure	Objective, pure	Noisy gradient descent (public T)
1,000	8.170×10^{-2}	7.060×10^{-2}	1.009×10^{-2}	6.558×10^{-2}

n	Output, Gaussian	Output, pure	Objective, pure	Noisy gradient descent (public T)
3,000	1.211×10^{-2}	7.206×10^{-3}	6.303×10^{-4}	8.888×10^{-3}
10,000	9.274×10^{-4}	8.321×10^{-4}	8.156×10^{-5}	1.384×10^{-3}
30,000	9.973×10^{-5}	8.943×10^{-5}	7.135×10^{-6}	1.538×10^{-4}
100,000	8.543×10^{-6}	6.379×10^{-6}	8.798×10^{-7}	1.621×10^{-5}
Slope in $\ln n$	-2.01	-2.00	-2.01	-1.79

On the running example (Table 38.3, Figure 38.5) all four releases fall like $1/n^2$, and noisy gradient descent a little slower, -1.79, because its step count grows with $\ln n$. Objective perturbation is about ten times better than output perturbation at every n, the $(h/\lambda)^2$ of Section 38.2. Noisy gradient descent with the public step rule is the worst of the three for $n \geq 10{,}000$, and tuned privately to $T \approx 50$ it would sit between the two perturbation methods. Accuracy on held-out data suffers even less. The non-private model has accuracy 0.747 at $n = 1{,}000$, which output perturbation drops to 0.681, noisy gradient descent to 0.695 and objective perturbation only to 0.736. By $n = 10{,}000$ all four private releases are within 0.002 of the non-private 0.753.

The pure-DP column is slightly better because, by Lemma 38.1, at $d = 10$ and $\varepsilon = 1$ its noise has mean squared length $d(d+1)(\Delta_2 f)^2 = 110\,(\Delta_2 f)^2$, against $d \times 3.7306^2(\Delta_2 f)^2 = 139.2\,(\Delta_2 f)^2$ for the Gaussian. At $d = 100$ it is $10{,}100\,(\Delta_2 f)^2$ against $1{,}392\,(\Delta_2 f)^2$, the price of $\delta = 0$ that Lower bounds proves unavoidable.

A deployed model is judged not on the training objective but by its **population risk**, the expected loss on a new record from the same distribution. For convex Lipschitz losses over a ball, Bassily, Feldman, Talwar & Thakurta (2019) show that the best population excess risk is of order

$$LR\Big(\frac{1}{\sqrt{n}} + \frac{\sqrt{d\ln(1/\delta)}}{n\varepsilon}\Big). \tag{38.28}$$

The first term is what any learner pays for seeing only n samples, so once n exceeds about $d\ln(1/\delta)/\varepsilon^2$, privacy adds less than sampling already costs.

Example 38.5. Equation 38.27 predicts how much data you need. Suppose an excess risk of 10^{-4} is acceptable, and output perturbation gives 9.274×10^{-4} at $n = 10{,}000$. The rate $1/n^2$ says that multiplying n by $\sqrt{9.274} = 3.05$ brings it to about 10^{-4}, and Table 38.3 confirms it, with 9.973×10^{-5} at $n = 30{,}000$. Halving ε costs as much as halving n, because every noise scale in this chapter is proportional to $1/(n\varepsilon)$.

38.6. When the model is not convex

A neural network's loss is not convex in its parameters, and the privacy proofs of output and objective perturbation both need a unique minimizer that moves only a little when one record changes. This section asks what survives of the three methods, and why the next chapter keeps only one.

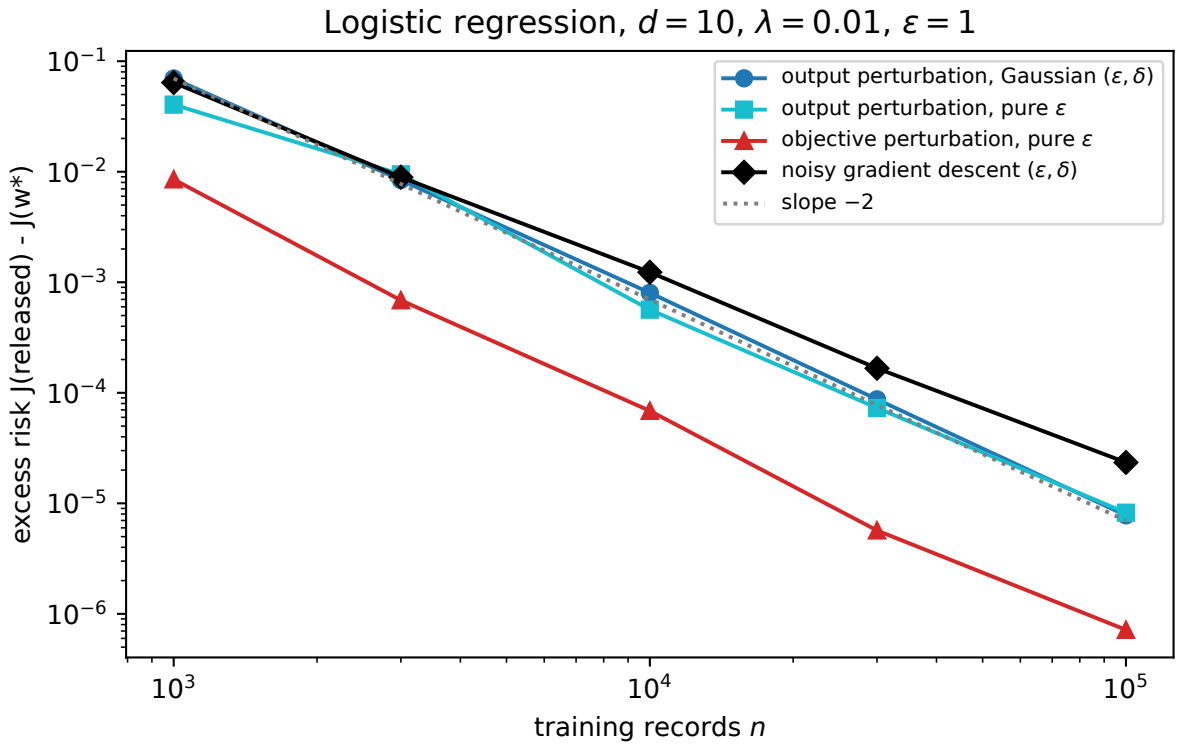


Figure 38.5.: Excess risk of the four private releases against the dataset size on the running example, log-log, with a line of slope -2. All fall like $1/n^2$; objective perturbation is about ten times better than output perturbation throughout.

Without convexity the minimizer can jump. Optimization averages the per-record loss $\frac{1}{4}(\theta^2 - 1)^2 - y\theta$, with labels $y = \pm 1$, into a risk with two wells, minimized in the well of the majority label. With 501 labels $+1$ and 500 labels -1, changing one label flips the majority and moves the minimizer by 2.0010. No $1/n$ appears, so output perturbation would need noise whose scale does not shrink with the dataset.

Objective perturbation breaks at step 1 of its proof, because the perturbed objective can have several minimizers and the map from noise to output is no longer invertible. Neel, Roth, Vietri & Wu (2020) prove a variant private without convexity, but only when an oracle returns the global minimizer of the perturbed objective. Gradient descent on a neural network finds a local minimum, about which that guarantee says nothing. Gradient perturbation survives, once clipping supplies the gradient bound its proof needs, just as Beyond global sensitivity clips a value before averaging it.

Proposition 38.5 (Noisy gradient descent with clipping)**.** *For any differentiable loss, convex or not, replace each per-record gradient by its clipped version* $\nabla \ell(\theta; z_i) \min(1, C/\|\nabla \ell(\theta; z_i)\|)$*, for a clipping norm* C*. Noisy gradient descent with the average of clipped gradients in place of* $\nabla \mathcal{L}$*, and with* $L = C$ *in Equation 38.18, is* (ε, δ)*-DP for replace-one neighbours.*

Proof.

1. *The clipped gradients are bounded.* A clipped gradient has length at most C, and is unchanged when it is already shorter.
2. *The step's sensitivity.* The average of clipped gradients loses one term of length at most C/n and gains another, so it moves by at most $2C/n$.
3. *The rest of the proof.* The proof of Theorem 38.3 used only this bound at the visited points (its step 3), so it goes through with $L = C$.

□

The non-convex case loses the guarantee of accuracy, not of privacy, so whether a network learns becomes an empirical question. Clipping also changes what is optimized, because the average of clipped gradients is not the gradient of $\mathcal{L}$, and the next chapter measures that bias. Random minibatches, which make each step cheaper and more private, complete DP-SGD, stochastic gradient descent with per-example clipping and Gaussian noise.

Example 38.6. Take the double-well loss with $n = 1{,}001$ records. Output perturbation at $\varepsilon = 1$ needs $\sigma = 3.7306 \times 2.0010 = 7.465$, more than three times the distance between the wells, whatever the dataset size. The release hides which well the data chose, as privacy demands when one record decides it, and so is useless. Noisy gradient descent with clipping still moves toward the well the noisy average gradient points to. When many records agree, that gradient is large compared with the noise, and the method finds the majority well with high probability.

38.7. Code

The script code/part05-dp-ml/dp_erm.py runs the three methods on regularized logistic regression with exact Newton solves, computes the exact one-dimensional densities of objective perturbation, and checks noisy projected gradient descent against its bound. From the repository root:

```
uv run python code/part05-dp-ml/dp_erm.py
```

Table 38.4.: The script's output with the default seed.

Experiment	Result
Gaussian calibration, $\varepsilon = 1$, $\delta = 10^{-5}$	$\mu = 0.2681$; exact $\sigma = 3.7306\,\Delta_2 f$ against classic $4.8448\,\Delta_2 f$
Output perturbation, one dimension	Minimizers 0.0832 apart, bound 0.2
Output perturbation, $n = 10{,}000$	Predicted excess risk 8.711×10^{-4}, measured 9.274×10^{-4}
Approximate minimizer, $\tau = 10^{-3}$	Sensitivity 0.22 against 0.02 exact
Objective perturbation, one dimension, 500 datasets	Largest log ratio 0.3779 against $\varepsilon = 1$; noise part 0.0992 of 0.1891, Jacobian part 0.3395 of 0.8109
Objective perturbation, $n = 10{,}000$	Predicted 7.127×10^{-5}, measured 8.156×10^{-5}
Noisy gradient descent, $n = 10{,}000$	Public rule $T = 218$, measured 1.384×10^{-3}; best $T \approx 50$ gives 2.875×10^{-4}
Noisy projected gradient descent, no ridge, $n = 1{,}000$	$T = 1{,}797$, bound 0.1668, measured 3.028×10^{-2}
Slopes in $\ln n$	-2.01, -2.00, -2.01, -1.79 with a ridge; -1.18 measured and -1.00 for the bound without

tests/test_dp_erm.py checks that the output still matches.

38.8. Summary

Table 38.5.: The results of this chapter, and where each is used.

Result	Statement	Use
Lemma 38.1	Noise $\propto e^{-\|b\|/s}$ with $s = \Delta_2 f/\varepsilon$ is ε-DP; $\mathbb{E}\|b\|^2 = d(d+1)s^2$	Pure-DP output and objective perturbation; costs a factor growing like d against the Gaussian

Result	Statement	Use
Theorem 38.1	$\theta^* + \mathcal{N}(0, \sigma^2 I)$ with $\sigma = 2L/(\lambda n \mu)$ is (ε, δ)-DP	Convex models trained to convergence; the baseline the others are measured against
Proposition 38.1	Excess risk $\approx \mathbb{E}\|Z\|^2 \operatorname{tr} H/(2d)$	Predicts the cost of privacy; it pays for the ridge's curvature and falls like $1/n^2$
Proposition 38.2	Sensitivity $2L/(\lambda n) + 2\tau/\lambda$ at gradient tolerance τ	Stop on a gradient check with τ well below L/n, never on a step count
Theorem 38.2	Minimizing $\mathcal{L} + \langle b, \theta \rangle / n$ is ε-DP, with $\varepsilon - 2\ln(1 + c/(n\lambda))$ left for the noise	Private logistic regression and smoothed support vector machines; needs an exact minimizer
Proposition 38.3	Excess risk $\approx \mathbb{E}\|b\|^2 \operatorname{tr} H^{-1}/(2dn^2)$	Beats output perturbation by about $(h/\lambda)^2$ when the data add curvature h
Theorem 38.3	Noise $\sigma = (2L/n)\sqrt{T}/\mu$ on the mean gradient; all iterates (ε, δ)-DP, convex or not	The privacy argument of DP-SGD; every iterative private trainer
Proposition 38.4	Optimization error shrinking like $(1 - \lambda/\beta)^T$, plus a noise floor $d\sigma^2/(4\beta)$	The step count is part of the budget; choose it from public quantities, or pay for choosing it
Theorem 38.4	No ridge, ball of radius R: excess risk $\leq 4RL\sqrt{d \ln(1.25/\delta)}/(n\varepsilon)$	The $\sqrt{d}/(n\varepsilon)$ rate, optimal up to logarithms; noise is more gradient variance; Public data
Theorem 38.5	Strongly convex: $L^2 d/(\lambda n^2 \varepsilon^2)$, up to logarithms	All three methods reach it; halving ε costs as much as halving the data
Proposition 38.5	Clipped noisy gradient descent is private for any loss	Why deep learning uses gradient perturbation, with clipping

38.9. References

- Chaudhuri & Monteleoni, "Privacy-preserving logistic regression" (NeurIPS 2008). Output and objective perturbation for logistic regression, the first private ERM algorithms.
- Chaudhuri, Monteleoni & Sarwate, "Differentially Private Empirical Risk Minimization" (JMLR 2011). The general analysis of both: output perturbation (Algorithm 1), objective perturbation (Algorithm 2) and their excess-risk bounds. The reference for Section 38.1 and Section 38.2.

- Kifer, Smith & Thakurta, "Private Convex Empirical Risk Minimization and High-dimensional Regression" (COLT 2012). Objective perturbation with Gaussian noise, constraints and non-smooth losses.
- Williams & McSherry, "Probabilistic Inference and Differential Privacy" (NeurIPS 2010). The first use of noisy gradient steps for private learning.
- Song, Chaudhuri & Sarwate, "Stochastic gradient descent with differentially private updates" (IEEE GlobalSIP 2013). Gradient perturbation for SGD, with minibatches.
- Bassily, Smith & Thakurta, "Private Empirical Risk Minimization: Efficient Algorithms and Tight Error Bounds" (FOCS 2014). Noisy stochastic gradient methods that attain the optimal rates, and the matching lower bounds; the reference for Section 38.4 and Section 38.5.
- Bassily, Feldman, Talwar & Thakurta, "Private Stochastic Convex Optimization with Optimal Rates" (NeurIPS 2019). The optimal population risk under DP, the rate quoted in Section 38.5.
- Neel, Roth, Vietri & Wu, "Oracle Efficient Private Non-Convex Optimization" (ICML 2020). Objective perturbation without convexity, given an optimization oracle; the caveat of Section 38.6.
- Mironov, "On Significance of the Least Significant Bits for Differential Privacy" (CCS 2012). Floating-point noise sampling breaks DP; why exact minimization is a real assumption.
- Wu, Li, Kumar, Chaudhuri, Jha & Naughton, "Bolt-on Differential Privacy for Scalable Stochastic Gradient Descent-based Analytics" (SIGMOD 2017). Output perturbation for SGD run for a fixed number of passes, without an exact minimizer.
- Iyengar, Near, Song, Thakkar, Thakurta & Wang, "Towards Practical Differentially Private Convex Optimization" (IEEE S&P 2019). Approximate minima perturbation, an objective perturbation that tolerates inexact solvers, and a benchmark of all three methods.
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising" (ICML 2018). The exact calibration Equation 38.2.
- Dong, Roth & Su, "Gaussian Differential Privacy" (JRSS-B 2022). The composition of Gaussian mechanisms used in Section 38.3.

39. DP-SGD

Private empirical risk minimization ended with one method left standing for non-convex models: gradient descent with Gaussian noise on every step. This chapter asks what that method computes exactly, why it is private for any model, and what its privacy costs in accuracy. The method is stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, which Abadi et al. (2016) introduced to give deep learning a guarantee of differential privacy (DP), and which is called DP-SGD. Nearly every private deep-learning system runs it, through libraries such as Opacus, TensorFlow Privacy and JAX Privacy.

The one idea is that the privacy analysis never looks inside the model. Each step releases a noisy sum of clipped per-example gradients. Clipping rescales each record's gradient to length at most C, the clipping norm, so adding or removing one record changes the sum by at most C, whatever the network, the loss or the current parameters. The privacy proof needs nothing more, so everything else affects only accuracy, and the knobs fall into two groups. The noise multiplier σ, which is the noise's standard deviation in units of C, the sampling rate and the number of steps set the privacy parameter ε through an accountant, and nothing else does. The clipping norm and the learning rate leave ε alone, and decide how much of the gradient survives and how far the noise moves the weights. Privacy then costs accuracy in two ways: a **bias** from clipping, which changes the direction of the average gradient, and a **variance** from the noise, which makes each step partly random.

The running example takes a configuration whose privacy Part 4 computed without training anything: a sampling rate of 0.01, a noise multiplier of 1 and 10,000 steps. Amplification by subsampling reports $\varepsilon = 6.71$ at $\delta = 10^{-5}$ for it, and this chapter uses it to train a network. Inputs are 50-dimensional standard normal vectors, and each label is the sign of the output of a fixed random **teacher** network with 8 hidden tanh units, so a perfect classifier exists. The **student** has one hidden layer of 64 tanh units and a logistic loss, with $d = 3{,}329$ parameters including biases. It trains on $n = 10{,}000$ records and is tested on 20,000 more. DP-SGD runs with a clipping norm of 0.5 and a learning rate of 0.5. Each step draws on average 100 records, so an epoch is 100 steps and the 10,000 steps make 100 epochs. The dataset is fixed, and only the training randomness changes between runs. Table 39.1 lists the symbols of the chapter.

Table 39.1.: The symbols of this chapter.

Symbol	Meaning
$D = (z_1, \dots, z_n)$	The training set: n records $z_i = (x_i, y_i)$
$\theta \in \mathbb{R}^d$	The model's d parameters; θ_t after step t
$\ell(\theta; z)$	The loss of the parameters θ on the record z
$g_i = \nabla_\theta \ell(\theta; z_i)$	The per-example gradient of record i; $\bar{g}_i$ after clipping

Symbol	Meaning
C	The clipping norm
σ	The noise multiplier: the noise standard deviation in units of C
q; $B = qn$	The sampling rate; the expected batch size
T; η	The number of steps; the learning rate
S_t; G_t	The batch of step t; its noisy sum of clipped gradients
$\varepsilon_{q,\sigma}(\alpha)$	The Rényi DP curve of one step at order α

39.1. The algorithm and its guarantee

What exactly does DP-SGD compute, and which (ε, δ) does it satisfy? The answer has to be precise, because each detail of the algorithm is a hypothesis of the proof, and an implementation that changes a detail can lose the guarantee without any visible symptom. Two datasets are neighbours here when one is the other with one record added or removed. Write $D \sim D'$ for such a pair, with $D' = D \cup \{z\}$ the dataset D with the record z added. The previous chapter used replacement instead. Add/remove suits DP-SGD because its batches are drawn by an independent coin for each record, so adding a record just adds one more coin.

The algorithm takes the noisy gradient descent of the previous chapter and changes three things. Each step uses a random batch instead of the whole dataset, each record's gradient is clipped, and the noise is scaled to the clipping norm.

Definition 39.1 (DP-SGD)**.** Fix in advance the clipping norm $C > 0$, the noise multiplier $\sigma > 0$ and the sampling rate $q \in (0, 1]$, as well as the number of steps T and the learning rate η. Write $B = qn$ for the expected batch size. Start from initial parameters θ_0 chosen independently of the data. For each step $t = 0, \dots, T-1$:

1. *Sample.* Put each record in the batch S_t independently with probability q. This is **Poisson sampling**.
2. *Per-example gradients.* For each record i of the batch, compute its **per-example gradient** $g_i = \nabla_\theta \ell(\theta_t; z_i)$ from the record z_i and θ_t alone.
3. *Clip.* Rescale each gradient to length at most C,

$$\bar{g}_i = g_i \min\Big(1, \frac{C}{\|g_i\|}\Big). \tag{39.1}$$

4. *Noisy sum.* Add fresh Gaussian noise ξ_t, with standard deviation σC in every coordinate, to the clipped sum, and call the result G_t:

$$G_t = \sum_{i \in S_t} \bar{g}_i + \xi_t, \qquad \xi_t \sim \mathcal{N}(0, \sigma^2 C^2 I_d). \tag{39.2}$$

5. *Step.* Set $\theta_{t+1} = \theta_t - \eta\, G_t / B$. The divisor is the expected batch size, a public number, not the size of the batch actually drawn.

The output is the sequence of parameters $\theta_1, \dots, \theta_T$.

The proof needs one fact about step 4: how far one record can move the clipped sum. Its answer is why the guarantee ignores the model.

Lemma 39.1 (The clipped sum has sensitivity C)**.** *Fix the parameters θ_t and the set of other records drawn into the batch. If a record z is added to the batch, the clipped sum $\sum_{i \in S_t} \bar{g}_i$ changes by a vector of length at most C, whatever the model, the loss and θ_t.*

Proof.

1. *The other terms do not move.* Each term $\bar{g}_i$ is computed from its own record and θ_t alone, by step 2 of Definition 39.1, so every term of the other records is the same with or without z.
2. *The new term is short.* The only change is the extra term $\bar{g}_z$. Clipping multiplies the length of g_z by $\min(1, C/\|g_z\|)$, so the new term's length is

$$\|\bar{g}_z\| = \min(\|g_z\|, C) \le C. \tag{39.3}$$

□

With the sensitivity fixed at C and the noise at σC, a step on which z is drawn is a Gaussian mechanism whose shift, measured in noise standard deviations, is $C/(\sigma C) = 1/\sigma$. Its privacy depends on that ratio alone (The Laplace and Gaussian mechanisms, Theorem 20.3), so the clipping norm cancels.

Poisson sampling then makes the step's output on D' a mixture. The record z is left out with probability $1 - q$, and then the output is distributed as on D. It is drawn with probability q, and then the output is shifted. Amplification by subsampling computes the Rényi differential privacy (RDP) curve of that mixture against the unshifted Gaussian. That curve is the Rényi divergence between the two distributions as a function of its order $\alpha > 1$, which the accountant chooses later. At order α the divergence is the logarithm of the α-th moment of the ratio of the two densities, divided by $\alpha - 1$. For integer orders the mixture's ratio is $1 - q$ plus q times a Gaussian ratio, so its α-th power expands by the binomial theorem, with one term for each number j of the α factors that take the shifted component, and

$$\varepsilon_{q,\sigma}(\alpha) = \frac{1}{\alpha - 1} \ln \sum_{j=0}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha - j} q^j e^{(j^2 - j)/(2\sigma^2)}, \tag{39.4}$$

and a numerical integral gives the other orders. Abadi et al. (2016) introduced this curve as the moments accountant.

The accountant tracks a curve rather than one (ε, δ) pair because curves add under composition. A pair only bounds the probability that the privacy loss exceeds ε, and says nothing about how large the loss is on that rare event, so pairs compose loosely. The theorem below composes the curves and converts the total. Section C.5 derives the same result from scratch, from clipping to the final ε, with worked numbers.

Theorem 39.1 (Privacy of DP-SGD)**.** *Run DP-SGD (Definition 39.1) with neighbours $D \sim D'$ that differ by adding or removing one record. Assume that:*

1. *each per-example gradient depends only on its own record and the current parameters;*
2. *the initial parameters θ_0 and the values of C, σ, q, T and the denominator B are fixed before the data are read;*
3. *every batch is drawn by Poisson sampling.*

Then for every $0 < \delta < 1$, releasing all of $\theta_1, \dots, \theta_T$ is (ε, δ)-DP with

$$\varepsilon = \min_{\alpha > 1} \Big[T\varepsilon_{q,\sigma}(\alpha) + \ln \frac{\alpha - 1}{\alpha} - \frac{\ln \delta + \ln \alpha}{\alpha - 1} \Big]. \tag{39.5}$$

The guarantee does not depend on the loss, the model, C or η.

Proof. The plan is to show that one step is a subsampled Gaussian mechanism with ratio $1/\sigma$, that the parameter update is post-processing of it, and then to compose the T steps and convert the total.

1. *One record moves the clipped sum by at most C.* This is Lemma 39.1, and it uses the first assumption.
2. *One step is the subsampled Gaussian.* Only the shift between the two outputs matters, by Lemma 20.1, and rotating the space puts that shift on one axis, where the other coordinates carry the same noise under both datasets. A shift shorter than C gives a pair that is a post-processing of the pair with shift C, by adding extra noise, so it is at least as private, as in step 4 of the proof of Theorem 20.3. So after rescaling by C, the worst case is the one-dimensional pair of the mixture $(1-q)\mathcal{N}(0, \sigma^2) + q\mathcal{N}(1, \sigma^2)$ and $\mathcal{N}(0, \sigma^2)$, whose curve is Equation 39.4. Poisson sampling is what makes the step on D' this mixture, because z is drawn with probability q independently of the other records. The reverse direction, $\mathcal{N}(0, \sigma^2)$ against the mixture, has a smaller curve at every order (Amplification by subsampling), so Equation 39.4 covers both.
3. *The update is post-processing.* The denominator $B = qn$ is a fixed public number, so θ_{t+1} is computed from θ_t and G_t alone. Post-processing cannot raise an RDP curve, by Theorem 29.3.
4. *Compose.* Step t is evaluated at θ_t, which depends on the earlier noisy sums. Its curve is bounded by $\varepsilon_{q,\sigma}$ whatever θ_t is, and that bound is fixed in advance, so adaptive composition applies (Theorem 29.2). The T steps together have the curve $T\varepsilon_{q,\sigma}(\alpha)$.
5. *Convert.* The conversion of Balle et al. (2020) turns a curve value at each order into an (ε, δ) guarantee. From Rényi DP to (ε, δ) proves it. Taking the best order gives Equation 39.5. Every iterate is a function of the released noisy sums, so releasing all of them, or only the last, is covered.

□

Each assumption is a condition on the code, and each has a common way to fail. Batch normalization computes means and variances over the batch, so one record changes every other record's gradient and the sum can move by much more than C. Private models therefore use normalizations that act on one example at a time, such as group or layer normalization. Choosing C as the median gradient norm of the current batch makes the noise depend on the data, so C

must come from public data or a private estimate. Dividing by the realized batch size $|S_t|$ instead of B also breaks the proof, because $|S_t|$ changes when z is added. Shuffling the data and cutting it into fixed batches is a different mechanism, which Amplification by subsampling shows can be much less private than the Poisson accountant reports. Finally, the accountant must see every step, including the steps of runs discarded during tuning, which is the subject of Hyperparameter tuning.

The guarantee is for one record. A person with many records needs user-level privacy, and preprocessing that uses the data, such as normalizing features by their mean, must itself be private or use public statistics.

Example 39.1. For the running example's configuration, Equation 39.5 gives $\varepsilon = 6.71$ at $\delta = 10^{-5}$, the same value Part 4 reports. The best order is $\alpha = 4.1$. Table 39.2 gives the noise multiplier that the same q and T need for other budgets.

Table 39.2.: The noise multiplier that DP-SGD with $q = 0.01$ and $T = 10{,}000$ steps needs for each budget, from the RDP accountant.

ε at $\delta = 10^{-5}$	0.5	1	2	4	6.71	8
Noise multiplier σ	7.72	4.13	2.28	1.36	1	0.917

How large is the noise? The averaged update G_t/B carries noise with standard deviation $\sigma C/B = 1 \times 0.5/100 = 0.005$ on each of the 3,329 coordinates. A Gaussian vector with d independent coordinates has length close to $\sqrt{d}$ times their standard deviation, so the noise has length about $0.005 \times \sqrt{3{,}329} = 0.288$. The signal is the averaged clipped gradient, whose length is at most $C = 0.5$. In the run its length is 0.070 at the first step, and between 0.015 and 0.027 from step 1,000 on, so from then on the noise is about ten times longer than the signal.

Training works anyway, because the signal points in roughly the same direction from one step to the next, while the noise is independent. So over t steps the signals add up to about t times their length, while the noise adds up to only about $\sqrt{t}$ times its length, since independent vectors add in squares. With a typical signal length of 0.026, signal overtakes noise when $t \cdot 0.026 > \sqrt{t} \cdot 0.288$, that is after $t > (0.288/0.026)^2 \approx 123$ steps. The final model reaches test accuracy 0.9589, against 0.9627 for non-private SGD.

39.2. Clipping: bias, noise and the choice of C

The clipping norm does not affect privacy, but it controls both forms of cost. A small C clips more gradients, which biases the update, and a large C adds more noise, because the noise is σC. To choose C you need to know what the update estimates on average, and how a run changes when C changes.

Averaged over the sampling and the noise, each record enters the update with a weight that clipping sets. The denominator B is chosen so that the sampling rate cancels.

Proposition 39.1 (The expected update)**.** *In DP-SGD (Definition 39.1), with θ_t fixed, the averaged update is an unbiased estimate of the average clipped gradient,*

$$\mathbb{E}\Big[\frac{G_t}{B}\Big] = \frac{1}{n}\sum_{i=1}^{n} \min\Big(1, \frac{C}{\|g_i\|}\Big) g_i, \tag{39.6}$$

and its noise has standard deviation $\sigma C/B$ in each coordinate.

Proof.

1. *Average over the sampling.* Each record is in the batch with probability q, so the expected clipped sum is $\sum_{i=1}^{n} q\,\bar{g}_i$. The noise has mean zero.
2. *Divide by $B = qn$.* The q cancels and leaves $\frac{1}{n}\sum_i \bar{g}_i$, which is Equation 39.6 by the definition of $\bar{g}_i$.
3. *The noise.* The noise ξ_t has standard deviation σC in each coordinate, and dividing by B divides it by B.

□

The update is therefore not the gradient of the training loss but of a reweighted one. Each record's weight $\min(1, C/\|g_i\|)$ falls as its gradient grows, so records the model fits badly, such as rare examples, members of small groups or mislabelled points, are down-weighted, while well-fitted records keep their full weight. The weights change as training proceeds, and Disparate impact measures who pays for them. Dividing by $|S_t|$ instead of B would not give this average, because the expectation of a ratio of random quantities is not the ratio of their expectations.

When C is below every gradient norm, clipping turns each gradient into C times a unit vector. The clipping norm then plays the same role as the learning rate.

Proposition 39.2 (Scale invariance)**.** *Suppose that every per-example gradient of a run is clipped, that is $\|g_i\| \geq C$ at every step. Write the noise as $\xi_t = \sigma C \zeta_t$ with $\zeta_t \sim \mathcal{N}(0, I_d)$. Then the update is*

$$\theta_{t+1} = \theta_t - \frac{\eta C}{B}\Big(\sum_{i \in S_t} \frac{g_i}{\|g_i\|} + \sigma\zeta_t\Big), \tag{39.7}$$

which depends on η and C only through their product ηC. So two runs with the same ηC and the same seed, in both of which every gradient is clipped, produce the same parameters.

Proof.

1. *Clipped gradients are scaled unit vectors.* If $\|g_i\| \geq C$, then $\min(1, C/\|g_i\|) = C/\|g_i\|$, so $\bar{g}_i = C g_i/\|g_i\|$.
2. *Factor out C.* Every term of the noisy sum now carries the factor C, so $G_t = C(\sum_{i\in S_t} g_i/\|g_i\| + \sigma\zeta_t)$.
3. *Substitute into the step.* Step 5 of Definition 39.1 subtracts $\eta G_t/B$, which gives Equation 39.7. The batch, the unit vectors and ζ_t involve neither η nor C, so the run depends on them only through ηC.

□

Below the gradient norms, DP-SGD is normalized SGD, which averages unit vectors, plus noise. Lowering C further changes nothing once η is scaled up to match. This is why practitioners often clip aggressively and tune only the learning rate, and it explains why De et al. (2022) find that small clipping norms work well. With the Adam optimizer, C drops out of the step length altogether, because Adam divides each coordinate of the update by its own running scale, which carries the same factor C.

Above the gradient norms, noise grows and signal does not. Once C exceeds most gradient norms, the clipped average stops growing with C, but the noise $\sigma C/B$ keeps growing in proportion, so the signal-to-noise ratio falls like $1/C$. So raising C beyond the point where it clips almost nothing only adds noise.

Example 39.2. Gradient norms change during training, so a fixed C visits both regimes. In the DP-SGD run of the running example the median per-example gradient norm is 3.592 at the first step and 0.006 at step 1,000, and it rounds to 0.000 from step 3,000 on. Non-private training behaves the same way (Figure 39.1, right). The logistic loss flattens on records classified with a wide margin, so their gradients vanish. With $C = 0.5$, every gradient is clipped at the start, and later between 9% and 25% of the batch are, the few records still near the decision boundary. So a fixed C is in the scale-invariant regime early, and later above most gradient norms, where clipping changes little.

Table 39.3 trains the running example at $\varepsilon = 6.71$ with seven clipping norms, once with the learning rate fixed at 0.5 and once with ηC fixed at $0.5 \times 0.5 = 0.25$.

Table 39.3.: Test accuracy of the running example at $\sigma = 1$ for seven clipping norms, one run each. With ηC fixed, small clipping norms all give the same accuracy.

C	0.01	0.03	0.1	0.3	1	3	10
Fixed $\eta = 0.5$	0.9244	0.9510	0.9567	0.9607	0.9583	0.9523	0.8347
Fixed $\eta C = 0.25$	0.9627	0.9623	0.9605	0.9599	0.9590	0.9558	0.9494

With the learning rate fixed, a small C is too slow, reaching 0.9244 at $C = 0.01$, because every step has length proportional to $\eta C = 0.005$. A large C is too noisy, reaching 0.8347 at $C = 10$. With ηC fixed, accuracy is flat for $C \leq 0.3$, as scale invariance predicts, and falls slowly for large C, where the noise grows while the clipped gradients do not. The best accuracy, 0.9627 at $C = 0.01$ with a learning rate of 25, equals the non-private model's.

39.3. Batch size and the number of steps at a fixed budget

Suppose the privacy budget (ε, δ) and the compute budget, a number of epochs E, are both fixed. Each epoch is $1/q$ steps, so $T = E/q$. Should you take many small steps or few large ones? The accountant says how much noise each choice needs, and the noise on the averaged gradient decides how useful each step is.

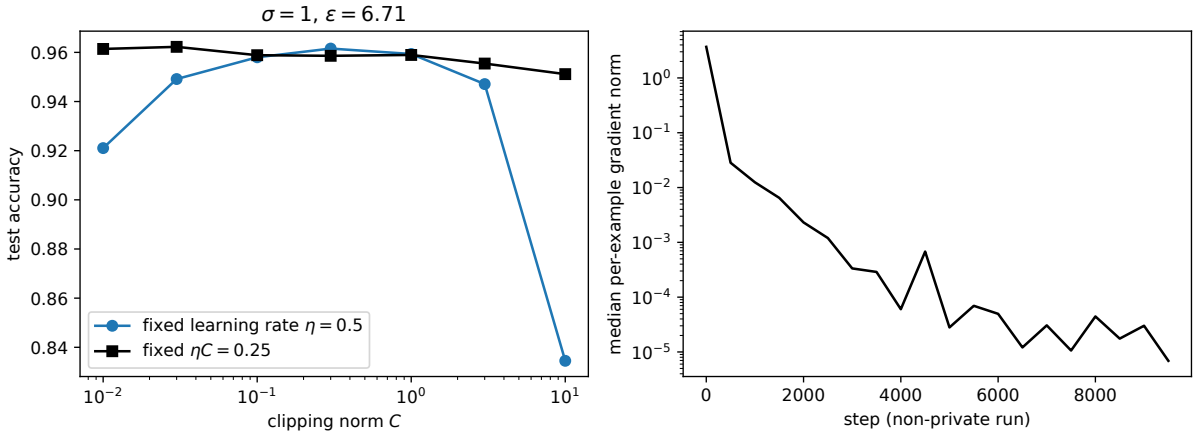


Figure 39.1.: Left: test accuracy of the running example at $\sigma = 1$ ($\varepsilon = 6.71$) against the clipping norm, with the learning rate fixed at $\eta = 0.5$ (circles) and with ηC fixed at 0.25 (squares). Right: the median per-example gradient norm during non-private training, on a log scale; it falls by about five orders of magnitude.

For small q, the terms $j = 0$ and $j = 1$ of Equation 39.4 cost nothing. Their exponent $j^2 - j$ is zero, and with every exponential set to 1 the sum is the binomial expansion of $\big((1-q)+q\big)^{\alpha} = 1$, whose logarithm is zero. Among the other terms the term $j = 2$, which carries q^2, dominates (Amplification by subsampling derives the approximation). That term is enough to see the trade.

Proposition 39.3 (Batch size at a fixed budget)**.** *Assume that the per-step RDP curve is approximately*

$$\varepsilon_{q,\sigma}(\alpha) \approx \frac{\alpha}{2} q^2 (e^{1/\sigma^2} - 1), \tag{39.8}$$

as it is for small q and orders that are not too large. Fix ε, δ and the number of epochs E, so that $T = E/q$. Where σ is well above 1, keeping ε fixed as q changes requires

$$\sigma \propto \sqrt{q}, \qquad \text{so} \qquad \frac{\sigma C}{B} \propto \frac{C}{\sqrt{qn}\,\sqrt{n}} = \frac{C}{\sqrt{Bn}}, \tag{39.9}$$

and the noise on the averaged gradient falls like $1/\sqrt{B}$. Where σ is near or below 1, the per-step cost is larger by the factor $\sigma^2(e^{1/\sigma^2} - 1) > 1$, so small batches need more noise than Equation 39.9 says.

Proof. The plan is to approximate the per-step cost and add it over all the steps. The total gives the noise multiplier that keeps ε fixed, and the last step measures what the approximation leaves out.

1. *Per-step cost.* For σ well above 1, the exponent $1/\sigma^2$ is small and $e^{1/\sigma^2} - 1 \approx 1/\sigma^2$. So the per-step curve is about $\frac{\alpha q^2}{2\sigma^2}$.
2. *Total cost.* Curves add under composition, so $T = E/q$ steps have the curve

$$\frac{E}{q} \cdot \frac{\alpha q^2}{2\sigma^2} = \frac{\alpha E q}{2\sigma^2}. \tag{39.10}$$

The conversion Equation 39.5 depends on the run only through this curve, so ε stays fixed when q/σ^2 does, that is when $\sigma \propto \sqrt{q}$.

3. *Noise per averaged gradient.* The noise on the averaged gradient is $\sigma C/B$ with $B = qn$. With $\sigma \propto \sqrt{q}$ and n fixed it is proportional to $\sqrt{q}/q = 1/\sqrt{q}$, and since $q = B/n$ that is Equation 39.9.
4. *Small σ.* The exponential grows faster than its tangent at 0, so $e^{1/\sigma^2} - 1$ exceeds the approximation $1/\sigma^2$ of step 1. The ratio of the two is the factor $\sigma^2(e^{1/\sigma^2} - 1)$, which is above 1 and grows as σ falls. By step 2, small batches need small σ, so they are the ones that pay this factor, as extra noise beyond the $\sqrt{q}$ law.

□

The accountant's numbers, in Table 39.4 of the example below, show that amplification is most efficient at large σ. The ratio $\sigma/\sqrt{q}$, which Equation 39.9 says is constant, falls from 22.3 at $B = 10$ to 10.8 at $B = 10{,}000$. Over the same range the factor $\sigma^2(e^{1/\sigma^2} - 1)$ falls from 3.23 to 1, so the small batches pay most of the extra noise through it. The rest comes from the high orders that the conversion uses, where the approximation of Proposition 39.3 does not hold and sampling buys little (Amplification by subsampling).

Large batches help, but not for free. For a fixed number of epochs they give less noise per step and fewer steps, and fewer steps means less progress unless the learning rate grows in proportion. The best learning rate does grow with the batch: in Table 39.5, below, it goes from 0.25 at $B = 30$ to between 1 and 2 at $B = 100$ to $B = 1{,}000$. De et al. (2022) and Anil et al. (2022) train with batches of thousands to hundreds of thousands, and report that large batches are a main ingredient of accurate private models. Large batches also parallelize well, which matters more on accelerators than in this small example.

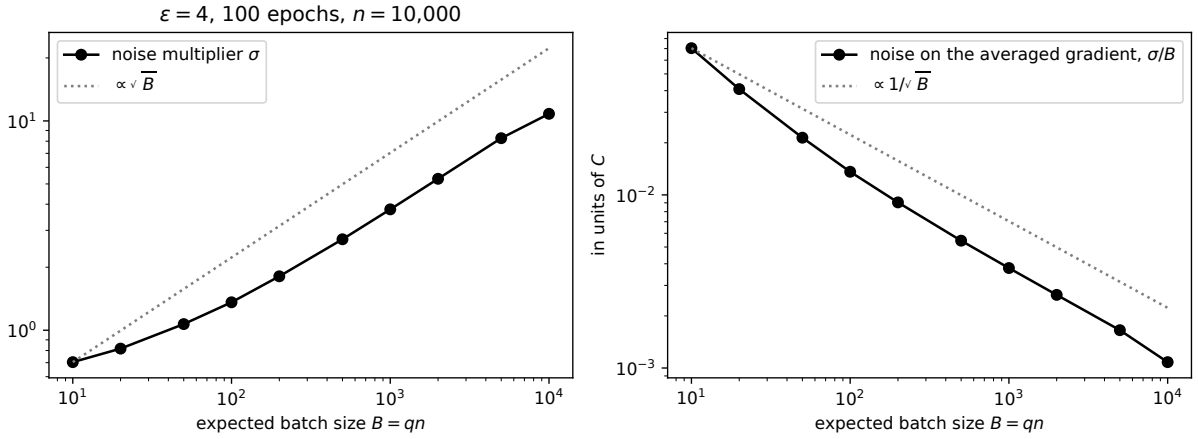


Figure 39.2.: DP-SGD at $\varepsilon = 4$, $\delta = 10^{-5}$, for 100 epochs on $n = 10{,}000$ records, as a function of the expected batch size. Left: the noise multiplier the accountant requires, against the $\sqrt{B}$ law of Equation 39.9 ($\sigma \propto \sqrt{q}$ at fixed n). Right: the noise standard deviation on the averaged gradient, in units of C, against the $1/\sqrt{B}$ law. Small batches need more noise than the law predicts.

Example 39.3. Table 39.4 fixes $\varepsilon = 4$, $\delta = 10^{-5}$ and 100 epochs on $n = 10{,}000$ records, and finds the noise multiplier for each sampling rate. For full batches, $q = 1$, there is no sampling, and

the 100 Gaussian steps compose exactly into a single Gaussian mechanism with ratio $\sqrt{100}/\sigma$ (f-DP and Gaussian DP).

Table 39.4.: Noise needed for $\varepsilon = 4$ at $\delta = 10^{-5}$ over 100 epochs, for seven sampling rates. The noise on the averaged gradient, in units of C, falls faster than $1/\sqrt{B}$.

q	B	Steps T	σ	Noise per averaged gradient σ/B	$\sigma/\sqrt{q}$	$\sigma^2(e^{1/\sigma^2} - 1)$
0.001	10	100,000	0.704	0.0704	22.3	3.23
0.003	30	33,333	0.909	0.0303	16.6	1.95
0.01	100	10,000	1.36	0.0136	13.6	1.33
0.03	300	3,333	2.16	0.0072	12.5	1.12
0.1	1,000	1,000	3.78	0.00378	12	1.04
0.3	3,000	333	6.44	0.00215	11.8	1.01
1	10,000	100	10.8	0.00108	10.8	1

From $B = 10$ to $B = 1{,}000$, a hundredfold increase, the noise per averaged gradient falls from 0.0704 to 0.00378. That is a factor of 18.6, against the factor of 10 that $1/\sqrt{B}$ predicts. Table 39.5 trains four of these configurations with three learning rates each.

Table 39.5.: Test accuracy of the running example at $\varepsilon = 4$ and 100 epochs, for four batch sizes and three learning rates each, one run each.

B	Learning rates and test accuracy
30	$\eta = 0.25$: 0.9544; 0.5: 0.9487; 1: 0.9453
100	$\eta = 0.5$: 0.9522; 1: 0.9569; 2: 0.9465
300	$\eta = 1$: 0.9591; 2: 0.9565; 4: 0.9577
1,000	$\eta = 2$: 0.9589; 4: 0.9565; 8: 0.9576

At its best learning rate, each batch size reaches between 0.9544 and 0.9591. The two largest batches, 300 and 1,000, do best, with 30 and 100 times fewer steps than $B = 10$ would take. Choosing the best of three learning rates after looking at test accuracy is itself a leak. It is done here only to compare batch sizes, and Hyperparameter tuning shows how to do it privately.

39.4. Per-example gradient norms without per-example gradients

Ordinary backpropagation computes the gradient of the *sum* of the batch's losses, never the gradient of each record. Clipping needs each record's gradient norm, and forming all B per-example gradients costs B times the memory of the model's gradient, which for large models is prohibitive. Can the norms, and the clipped sum, be had more cheaply?

The tool is the shape of a dense layer's gradient. Take a dense layer that maps an input a to Wa, and write v for the gradient of the loss with respect to the layer's output, the **back-propagated**

error. The gradient with respect to the weights W is then the outer product $va^\top$ (Gradient inversion, Proposition 14.1). That chapter uses the outer product to reconstruct inputs from gradients, and here it makes per-example norms cheap.

Proposition 39.4 (Ghost norms)**.** *Consider a dense layer, and for record i write a_i for its input to the layer and v_i for its back-propagated error, so that the record's gradient for the layer is $v_i a_i^\top$. Then*

$$\left\| v_i a_i^\top \right\|_F = \|v_i\| \, \|a_i\|, \tag{39.11}$$

where the Frobenius norm $\|\cdot\|_F$ is the square root of the sum of squared entries. The whole network's per-example gradient norm is the square root of the sum of these squared norms over the layers. Write c_i for the clipping factor of record i,

$$c_i = \min\Big(1, \frac{C}{\|g_i\|}\Big). \tag{39.12}$$

Stack the errors as the rows of a matrix V and the inputs as the rows of a matrix A. If $c \circ V$ is V with row i multiplied by c_i, the clipped sum for the layer is $\sum_i c_i v_i a_i^\top = (c \circ V)^\top A$. Now let the layer act on a sequence of N tokens, with input a_{is} and error v_{is} at token s. Record i's gradient is then $\sum_s v_{is} a_{is}^\top$, and

$$\Big\| \sum_s v_{is} a_{is}^\top \Big\|_F^2 = \sum_{s,s'} (a_{is} \cdot a_{is'})(v_{is} \cdot v_{is'}). \tag{39.13}$$

Proof. The plan is to compute the norm of one outer product, add over layers, write the clipped sum as one matrix product, and expand the square for sequences.

1. *One token.* The entry (j,k) of $va^\top$ is $v_j a_k$, so the squared Frobenius norm factors into two sums,
$$\sum_{j,k} v_j^2 a_k^2 = \Big(\sum_j v_j^2\Big)\Big(\sum_k a_k^2\Big) = \|v\|^2 \|a\|^2. \tag{39.14}$$
2. *Layers.* The full gradient is the concatenation of the layers' gradients, so its squared length is the sum of theirs.
3. *The clipped sum.* The sum $\sum_i c_i v_i a_i^\top$ is linear in the errors, so it is the usual batch product of errors and inputs with each record's error scaled by c_i. One backward pass computes the errors, the norms come from Equation 39.11, and one more matrix product per layer gives the clipped sum.
4. *Sequences.* Drop the record index i. The squared Frobenius norm of a matrix M is the trace of $M^\top M$, so the square of a sum of outer products expands into one trace per pair of tokens,
$$\Big\| \sum_s v_s a_s^\top \Big\|_F^2 = \sum_{s,s'} \operatorname{tr}\left(a_s v_s^\top v_{s'} a_{s'}^\top\right). \tag{39.15}$$
Inside each trace, $v_s^\top v_{s'}$ is the number $v_s \cdot v_{s'}$, and what remains is $\operatorname{tr}(a_s a_{s'}^\top) = a_s \cdot a_{s'}$. That gives Equation 39.13, the entrywise product of two $N \times N$ Gram matrices, summed.

□

This is **ghost clipping**, in which the per-example gradients are "ghosts" that are never formed. Goodfellow (2015) gave the norms for single inputs, Lee & Kifer (2021) clipped with them, and Li et al. (2022) extended them to sequences. For one token per record, ghost clipping stores $B(d_{\text{in}} + d_{\text{out}})$ numbers per layer instead of $Bd_{\text{in}}d_{\text{out}}$, where d_{in} and d_{out} are the layer's input and output widths. For sequences the Gram matrices cost $2BN^2$, which beats the explicit gradients only when $2N^2 < d_{\text{in}}d_{\text{out}}$. So long sequences in narrow layers favour forming the gradients, and Bu, Mao & Xu (2022) choose per layer. A convolutional layer's per-example gradient is a sum of outer products over spatial positions, so Equation 39.13 applies with positions in place of tokens.

Either way, privacy is untouched. Ghost norms are exact computations of the same quantities, so the clipped sum, and with it Theorem 39.1, is unchanged.

Example 39.4. For the 256 records of a test batch of the running example, the norms from Equation 39.11 agree with explicitly formed per-example gradients to a relative difference of 2.2×10^{-16}, which is rounding error. For a random 16-token sequence, Equation 39.13 gives 33.144228, and the explicit gradient's norm is the same. Table 39.6 compares the memory for several layers.

Table 39.6.: Numbers stored to clip one dense layer, explicitly and with ghost norms.

Layer	Explicit $Bd_{\text{in}}d_{\text{out}}$	Ghost	Ratio
Running example, first layer: $B = 100$, $51 \to 64$ (50 inputs and a bias)	326,400	11,500	28.4
$B = 32$, one token, $768 \to 768$	18,874,368	49,152	384.0
$B = 32$, 512 tokens, $768 \to 768$	18,874,368	16,777,216	1.1
$B = 32$, 512 tokens, $768 \to 3{,}072$	75,497,472	16,777,216	4.5
$B = 8$, 4,096 tokens, $4{,}096 \to 4{,}096$	134,217,728	268,435,456	0.5

39.5. Microbatches and vectorized per-example gradients

Deep-learning frameworks are built to compute the gradient of a batch's summed loss in one parallel pass, and per-example clipping breaks that pattern. Computed naively, one record at a time, DP-SGD can run tens or hundreds of times slower than ordinary training. Ghost norms (Section 39.4) are one remedy, and two others are common. Vectorization computes the per-example gradients in parallel after all, while **microbatching** clips groups of records instead of single records. Vectorization changes nothing about privacy or accuracy, so the question is what microbatching costs.

Definition 39.2 (Microbatched DP-SGD)**.** Choose a group size m, and fix in advance a rule that assigns every record to one of B/m groups, for example a hash of a record identifier. A Poisson batch of expected size B then splits into about B/m **microbatches** of about m records each. Microbatched DP-SGD runs Definition 39.1, but clips the mean gradient of each microbatch instead of each record's gradient. It adds Gaussian noise to the sum of the clipped means and divides the result by B/m.

Under per-example clipping, the added record's term goes from absent to present, a change of at most C. A microbatch's clipped mean instead goes from one vector of length at most C to another, and that change can be twice as long.

Proposition 39.5 (What microbatches cost)**.** *Under add/remove neighbours, the sum of clipped microbatch means has sensitivity $2C$ as soon as a record can share its group with others. So the accountant's ε for the same q, σ and T holds only with noise of standard deviation $2\sigma C$. The update's noise then has standard deviation*

$$\frac{2\sigma C}{B/m} = 2m \cdot \frac{\sigma C}{B} \tag{39.16}$$

per coordinate, $2m$ times that of per-example clipping.

Proof. The plan is to locate the one microbatch a record can change, bound how far its clipped mean moves, and then scale the noise to match the accountant.

1. *One record touches one microbatch.* The assignment of records to groups is fixed before training and does not depend on which other records are present. A position in the dataset would not do, because adding a record shifts the positions of the records after it. So adding or removing z changes only the microbatch of z's group, and every other clipped mean is the same on both datasets.
2. *That clipped mean moves by up to $2C$.* If the group has other sampled records, its clipped mean changes from one vector of length at most C to another. The two can point in opposite directions, so they differ by up to $2C$. Only when z is alone in its group does the change shrink to C, the per-example case, and a worst-case bound must take $2C$.
3. *The accountant.* Step 2 of the proof of Theorem 39.1 needs the noise to be σ times the sensitivity, so the noise must be $\sigma \cdot 2C$. The rest of that proof is unchanged.
4. *Per coordinate.* The update divides the noisy sum by the public number B/m, and dividing $2\sigma C$ by it gives Equation 39.16.

□

Microbatching trades accuracy for speed at a steep rate. A batch now has only B/m clipped terms to carry signal instead of B, while the total noise has doubled to $2\sigma C$. A microbatch's mean gradient is also shorter than its records' gradients when they disagree, so clipping removes less of it, but that does not make up for a factor $2m$ in the noise. Some libraries offer microbatches as a speed option, so check what yours assumes. Whether its accounting uses sensitivity C or $2C$, and whether its microbatches are fixed in advance or cut by position from the sampled batch, decides whether the reported ε is valid. Cut by position, adding one record shifts every later record into a different microbatch, and step 1 fails.

Vectorization costs nothing, because it keeps per-example clipping. JAX's `vmap` computes the per-example gradients in parallel by vectorizing over the examples of a batch, and Opacus by attaching hooks that record the per-example quantities during backpropagation. These are exact computations of the same per-example gradients, so they change neither the privacy nor the accuracy. Subramani, Vadivelu & Kamath (2021) show that vectorization combined with just-in-time compilation in many cases brings DP-SGD's running time close to that of ordinary training.

Example 39.5. At the running example's $\sigma = 1$, $C = 0.5$ and $B = 100$, the noise on the averaged update is 0.0050 per coordinate with per-example clipping. With microbatches of $m = 4$ records it is 0.0400, eight times more, and with $m = 16$ it is 0.1600, thirty-two times more, at the same $\varepsilon = 6.71$. The test accuracy falls with it, from 0.9589 for per-example clipping (Example 39.1) to 0.9336 with microbatches of 4 and 0.7459 with microbatches of 16, one run each.

39.6. The running example: privacy against accuracy

The earlier sections each changed one knob. This one puts the pieces together and asks, for the running example, how much accuracy each budget costs and where the loss comes from.

Example 39.6. Table 39.7 trains the running example with $q = 0.01$, $C = 0.5$, $\eta = 0.5$ and $T = 10{,}000$ steps, at each budget's noise multiplier from Table 39.2, three times each. Each run starts from a fresh random initialization. The non-private baseline uses the same sampling and learning rate with neither clipping nor noise. In a single run its accuracy is already at its best after 1,000 steps, at 0.9627, and stays there, at 0.9617 after 10,000 steps. So the baseline runs 1,000 steps, and its three runs average 0.9610.

Table 39.7.: Test accuracy of the running example under DP-SGD at five budgets, with $\delta = 10^{-5}$.

	Noise multiplier	Test accuracy (mean ± standard deviation over 3 runs)
$\varepsilon = 0.5$	7.72	0.8209 ± 0.0180
$\varepsilon = 1$	4.13	0.9175 ± 0.0161
$\varepsilon = 2$	2.28	0.9441 ± 0.0051
$\varepsilon = 4$	1.36	0.9548 ± 0.0019
$\varepsilon = 8$	0.917	0.9604 ± 0.0019
Clipped, no noise	0	0.9649 ± 0.0009
Non-private SGD, 1,000 steps	—	0.9610 ± 0.0005

Part 4 asked how private the running configuration is, and answered $\varepsilon = 6.71$. This example adds the other half: on this task it costs 0.4 points of accuracy, 0.9589 against 0.9627 in the single runs of Example 39.1. Halving ε to about 3 needs σ between 1.36 and 2.28, and by the table costs about one more point of accuracy, while reaching $\varepsilon = 1$ costs four points. Choosing ε asks which of these trades is worth making, and Auditing checks whether the trained model is as private as the accountant says.

Clipping costs nothing here, and noise costs everything. Clipping without noise reaches 0.9649, slightly above non-private SGD, because on this problem clipping only limits the influence of the few records with large gradients. The whole loss comes from the noise, and it is steeply non-linear in ε. From $\varepsilon = 8$ to 4 the accuracy drops by half a point, from 2 to 1 by 2.7 points, and from 1 to 0.5 by almost 10. The signal-against-noise argument of Example 39.1 explains the collapse. At $\varepsilon = 1$ the noise multiplier is 4.13, so the noise on the averaged gradient has length about $4.13 \times 0.5/100 \times \sqrt{3{,}329} = 1.19$, against a signal of about 0.026 if the signal stays near its length at $\sigma = 1$. Signal then overtakes noise only after about $(1.19/0.026)^2 \approx 2{,}100$ steps, so the 10,000 steps contain only a few independent useful moves. Figure 39.4 shows the result: with $\sigma = 4$ the model reaches its accuracy early and then wanders.

Run-to-run variation grows with the noise. The standard deviation over runs is 0.0019 at $\varepsilon = 8$ and 0.018 at $\varepsilon = 0.5$. A single private run is a sample from a wide distribution, so comparing two configurations by one run each, as Table 39.3 does, is reliable only for differences well above that spread.

Architecture choices change under privacy. The running example uses tanh units deliberately, because Papernot, Thakurta, Song, Chien & Erlingsson (2021) find that bounded activations such as tanh train better than ReLU under DP-SGD. With the same guarantee, $(2.93, 10^{-5})$ on the handwritten digits of MNIST, their tanh network reaches 98.1% test accuracy against 96.6% for ReLU. On the CIFAR-10 images at $(7.53, 10^{-5})$ it reaches 66.2% against 61.6%. Model size changes too. The noise vector has length about $\sigma C\sqrt{d}/B$, which grows with the number of parameters, while the clipped signal cannot exceed C. So the best private model is often smaller than the best non-private one, unless pretraining on public data has already done most of the work (Public data and private fine-tuning). Larger models can still win when the batch size and the clipping norm are tuned with them. De, Berrada, Hayes, Smith & Balle (2022) train a 40-layer wide residual network from scratch on CIFAR-10 to 81.4% test accuracy at $(8, 10^{-5})$. Very large batches and small clipping norms are among their practices. These numbers come from the papers, not from this chapter's script.

39.7. Code

The script code/dp_sgd.py implements the network and its per-example gradients by ghost norms. It also implements DP-SGD with Poisson sampling and optional microbatches, and the RDP accountant of Part 4, rewritten in a few dozen lines so the part stands on its own. Later chapters import its model and accountant: `make_task()`, `train_dpsgd(rng, data, config)`, `accuracy`, `clipped_grad_sum`, `dpsgd_epsilon(q, sigma, steps, delta)` and `noise_multiplier_for(q, steps, eps, delta)`. From the repository root:

```
uv run python code/part05-dp-ml/dp_sgd.py
```

Table 39.8.: The script's output with the default seed.

Experiment	Result
Accountant, $q = 0.01$, $\sigma = 1$, 10,000 steps	$\varepsilon = 6.71$ at $\alpha = 4.1$, as in Part 4

Experiment	Result
Noise on the averaged gradient	0.005 per coordinate, length about 0.288; signal 0.015 to 0.027
Ghost norms	Agree with explicit gradients to 2.2×10^{-16}
Microbatches of 4 and 16	Noise 8 and 32 times larger; accuracy 0.9336 and 0.7459
Running example, $\varepsilon = 6.71$	0.9589 against 0.9627 non-private
Clipping sweep	With ηC fixed, 0.9599 to 0.9627 for $C \leq 0.3$
Batch sweep, $\varepsilon = 4$	Noise per averaged gradient falls 18.6 times from $B = 10$ to 1,000
Privacy against accuracy	From 0.8209 at $\varepsilon = 0.5$ to 0.9604 at $\varepsilon = 8$

tests/test_dp_sgd.py checks that the output still matches.

39.8. Summary

Table 39.9.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 39.1	Poisson batches; clip each g_i to length C; add $\mathcal{N}(0, \sigma^2 C^2 I)$ to the sum; step by $\eta G_t / B$	Every chapter of this part; check an implementation for Poisson sampling, per-example gradients, no batch statistics, a public denominator and a fixed C
Lemma 39.1	One record moves the clipped sum by at most C	Why the guarantee ignores the model
Theorem 39.1	RDP curve $T\varepsilon_{q,\sigma}(\alpha)$, converted by Equation 39.5	Only σ, q and T set ε, so C and η are free for accuracy; the running example has $\varepsilon = 6.71$ at $\delta = 10^{-5}$
Example 39.1	Noise $\sigma C / B$ per coordinate, length about $\sigma C \sqrt{d} / B$; signal overtakes noise after about (noise length/signal length)^2 steps	Judging whether a configuration can learn at all
Proposition 39.1	$\mathbb{E}[G_t / B] = \frac{1}{n} \sum_i \min(1, C / \lvert g_i \rvert) g_i$	Divide by B, not $\lvert S_t \rvert$; the reweighting is the source of disparate impact
Proposition 39.2	With every gradient clipped, the run depends on ηC only	Clip small and tune ηC: below the typical gradient norm, C stops mattering

Result	Statement	Use
Proposition 39.3	$\sigma \propto \sqrt{q}$ and noise $\propto 1/\sqrt{B}$ at fixed ε and epochs, for small q and σ well above 1	Prefer large batches at a fixed number of epochs, and scale the learning rate up with them
Proposition 39.4	$\|v_i a_i^\top\|_F = \|v_i\| \, \|a_i\|$; for sequences, the summed entrywise product of two Gram matrices	Per-example norms at scale; ghost norms per layer while $2N^2 < d_{\text{in}} d_{\text{out}}$
Proposition 39.5	Sensitivity $2C$; noise $2m\sigma C/B$ per coordinate at the same ε	Speed up per-example clipping by vectorizing or by ghost norms, not by microbatches
Example 39.6	Accuracy from 0.8209 at $\varepsilon = 0.5$ to 0.9604 at $\varepsilon = 8$, against 0.9610 non-private	Expect accuracy close to non-private at large ε and a collapse below some threshold; locate it for your task before promising a budget

39.9. References

- Abadi, Chu, Goodfellow, McMahan, Mironov, Talwar & Zhang, "Deep Learning with Differential Privacy" (CCS 2016). DP-SGD and the moments accountant; the paper this chapter implements.
- Song, Chaudhuri & Sarwate, "Stochastic gradient descent with differentially private updates" (IEEE GlobalSIP 2013), and Bassily, Smith & Thakurta, "Private Empirical Risk Minimization: Efficient Algorithms and Tight Error Bounds" (FOCS 2014). The noisy SGD that DP-SGD extends with clipping.
- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (arXiv 2019). The accountant of Equation 39.4.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023). The practitioner's reference for everything in this chapter: what to clip, how to tune, what to report.
- De, Berrada, Hayes, Smith & Balle, "Unlocking High-Accuracy Differentially Private Image Classification through Scale" (arXiv 2022). Large batches, small clipping norms and other practices that close much of the gap on image classification.
- Anil, Ghazi, Gupta, Kumar & Manurangsi, "Large-Scale Differentially Private BERT" (Findings of EMNLP 2022). Very large batches for private language-model pretraining.
- Goodfellow, "Efficient Per-Example Gradient Computations" (arXiv 2015). The norm trick of Equation 39.11.
- Lee & Kifer, "Scaling up Differentially Private Deep Learning with Fast Per-Example Gradient Clipping" (PoPETs 2021). Clipping with the norm trick and a second backward pass, without forming per-example gradients.
- Subramani, Vadivelu & Kamath, "Enabling Fast Differentially Private SGD via Just-in-Time Compilation and Vectorization" (NeurIPS 2021). Why per-example clipping is slow in standard frameworks, and how vectorization removes most of the cost.

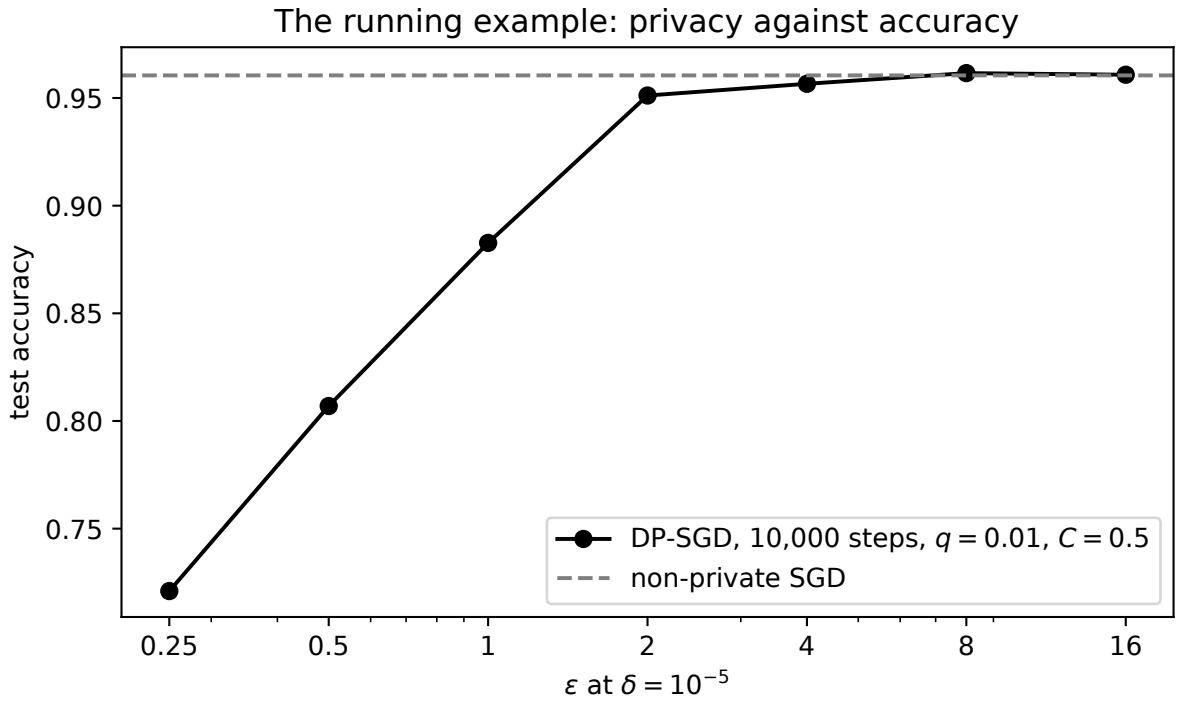


Figure 39.3.: Test accuracy of DP-SGD on the running example at $\delta = 10^{-5}$ against ε (log scale), with the non-private model dashed. Accuracy is close to non-private above $\varepsilon = 4$ and falls steeply below $\varepsilon = 1$.

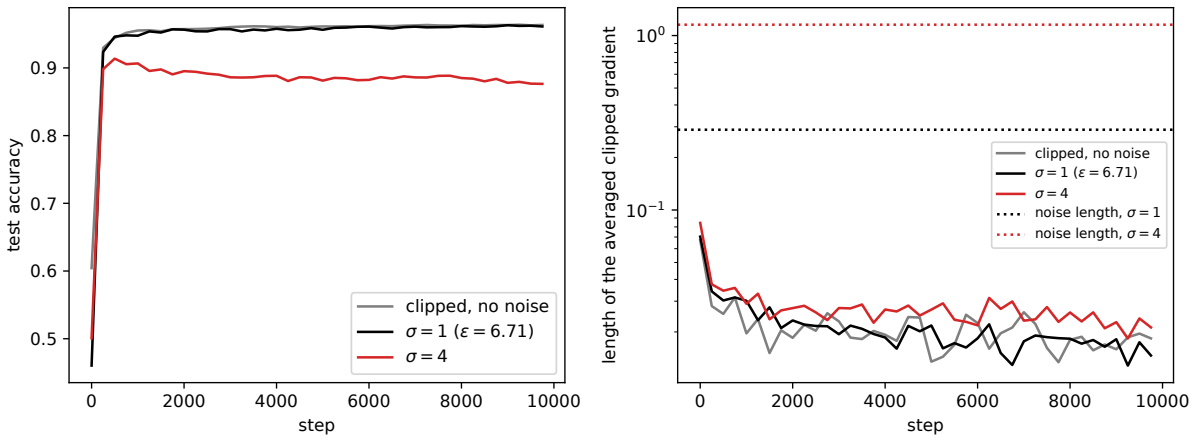


Figure 39.4.: Left: test accuracy during training with clipping alone, with $\sigma = 1$ ($\varepsilon = 6.71$) and with $\sigma = 4$. Right: the length of the averaged clipped gradient at each step (solid, log scale) against the length of the noise on it (dotted). The noise is about ten times longer than the signal at $\sigma = 1$, and about forty times at $\sigma = 4$.

- Papernot, Thakurta, Song, Chien & Erlingsson, "Tempered Sigmoid Activations for Deep Learning with Differential Privacy" (AAAI 2021). Bounded activations train better than ReLU under DP-SGD.
- Li, Tramèr, Liang & Hashimoto, "Large Language Models Can Be Strong Differentially Private Learners" (ICLR 2022). Ghost clipping for sequence models, Equation 39.13.
- Bu, Mao & Xu, "Scalable and Efficient Training of Large Convolutional Neural Networks with Differential Privacy" (NeurIPS 2022). Choosing per layer between ghost norms and explicit gradients.
- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020). The conversion of Equation 39.5.

40. Hyperparameter tuning and private selection

The previous chapter proved a guarantee for one training run. The algorithm is stochastic gradient descent (SGD) with per-example clipping and Gaussian noise. Run once with a fixed learning rate, clipping norm, batch size and number of steps, it satisfies differential privacy (DP) with the (ε, δ) reported by its accountant, the routine that adds up the privacy cost of the steps, and it is called DP-SGD. But in practice nobody trains once. A practitioner tries a grid of learning rates, scores each trained model on a validation set, and ships the best one. The validation set and the training set are private data, and every run touched them. So this chapter asks how much the search leaks, and how to pay for it honestly.

The answer turns on one idea. The model you ship was chosen by comparing all the runs, so it is a function of all of them, and with a fixed number of runs it can cost as much as all of them together. If instead the *number* of runs is random, drawn before the search starts, then the event "every run came out badly" is mostly explained by "there were few runs" rather than by the data. The cost then falls to a constant multiple of one run, plus a term that grows only with the logarithm of the expected number of runs.

The running example is that of DP-SGD: a network with 3,329 parameters trained on $n = 10{,}000$ records, with a sampling rate q of 0.01, a clipping norm C of 0.5, a noise multiplier σ of 1 and 10,000 steps. One run costs $\varepsilon = 6.71$ at $\delta = 10^{-5}$. Charging for every run in Rényi differential privacy (RDP) turns a grid of 10 runs into $\varepsilon = 27.23$, less than ten times 6.71 because the conversion to (ε, δ) pays its δ term once. A random number of runs with mean 10 costs between 10.08 and 13.67, depending on its distribution (Section 40.4).

40.1. The cost of releasing the best run

A choice among eight learning rates carries at most three bits, which seems harmless. But the released model is a whole private run, picked *because* of how it came out, and the comparison depends on every run. This section measures how much that can cost, because the rest of the chapter is about avoiding it.

Call one **run** a randomized algorithm Q that trains a model on the private data with hyperparameters taken from a fixed set of **candidates**, and outputs the model with its **score**, such as a validation accuracy. Outputs are ordered by score, and $Q(D)$ also denotes the distribution of one run's output on the dataset D.

The **best-of-K** algorithm M draws a number of runs K, independently of the data, runs Q independently K times, and releases the best output. Each run draws its candidate independently from a fixed distribution, as random search does. Neighbouring datasets $D \sim D'$ differ by adding or removing one record. Every result of this chapter needs the distribution of the best of K draws, which the next lemma gives.

Lemma 40.1 (The best of a random number of draws)**.** *Let one draw have a distribution Q on outcomes ordered from best to worst. Write $Q(\preceq y)$ for the probability that a draw is no better than y, and $Q(\prec y)$ for the probability that it is strictly worse. Let the number of draws K be random and independent of the draws, with generating function*

$$f(x) = \mathbb{E}[x^K] = \sum_k \Pr[K = k]\, x^k. \tag{40.1}$$

If $K = 0$, the release is a fixed output that does not depend on the data. Then for every other outcome y, the best of the K draws is y with probability

$$\Pr[M = y] = f(Q(\preceq y)) - f(Q(\prec y)). \tag{40.2}$$

A fixed $K = k$ has $f(x) = x^k$, and a Poisson number of draws with mean $\mathbb{E}[K]$ has $f(x) = e^{\mathbb{E}[K](x-1)}$. A geometric number on $\{1, 2, \dots\}$, with $\Pr[K = k] = \omega(1-\omega)^{k-1}$ for a stopping probability $\omega \in (0, 1)$, has $f(x) = \omega x/(1 - (1 - \omega)x)$.

Proof. The plan is to count outcomes for a fixed number of draws, then average over K, then sum the three series.

1. *A fixed number k of draws.* The best of k independent draws is no better than y exactly when every draw is, which has probability $Q(\preceq y)^k$.
2. *The best is exactly y.* That happens when every draw is no better than y but not every draw is strictly worse. The second event is contained in the first, so subtracting its probability gives
$$\Pr[\text{best of } k = y] = Q(\preceq y)^k - Q(\prec y)^k. \tag{40.3}$$
3. *Average over K.* The number of draws is independent of the draws, so averaging step 2 over K replaces each power x^k by its average $\sum_k \Pr[K = k]\, x^k = f(x)$. The term for $K = 0$ is the same constant in both generating functions and cancels.
4. *The three generating functions.* A fixed k gives one term, the Poisson weights sum by the exponential series, and the geometric weights by the geometric series.

□

A die shows the lemma at work. The best of two rolls is a 6 with probability $1 - (5/6)^2 = 0.3056$. If you roll once or twice with probability $\frac{1}{2}$ each, then $f(x) = \frac{1}{2}x + \frac{1}{2}x^2$, and the best is a 6 with probability $f(1) - f(5/6) = 0.2361$, the average of the one-roll and two-roll answers.

The simplest run that shows the cost has two outcomes, as in randomized response (Definition 18.4). With "bad" the worse outcome, Lemma 40.1 says that the best of k is bad only when all k runs are bad, so the likelihood ratios of the k independent runs multiply.

The proposition below also bounds the cost in Rényi terms, to show that large orders see it too. For distributions P and P' on a finite set, the **Rényi divergence** of order $\alpha > 1$ is

$$D_\alpha(P \| P') = \frac{1}{\alpha - 1} \ln \sum_y P(y)^\alpha P'(y)^{1-\alpha}, \tag{40.4}$$

and an RDP curve $\varepsilon(\alpha)$ bounds it between outputs on neighbours (Rényi differential privacy, Definition 29.1).

Proposition 40.1 (The best of k randomized responses)**.** *Let one run be randomized response with two outcomes, "good" and "bad", with good the better. On D the run comes out good with probability $\frac{1}{1+e^{\varepsilon}}$, and on its neighbour D' with probability $\frac{e^{\varepsilon}}{1+e^{\varepsilon}}$, so one run is exactly ε-DP. Releasing the best of k runs is exactly $k\varepsilon$-DP, because*

$$\frac{\Pr_D[\textit{best of } k \textit{ is bad}]}{\Pr_{D'}[\textit{best of } k \textit{ is bad}]} = \frac{\left(e^{\varepsilon}/(1+e^{\varepsilon})\right)^k}{\left(1/(1+e^{\varepsilon})\right)^k} = e^{k\varepsilon}. \tag{40.5}$$

At every order $\alpha > 1$ the release also has (Papernot & Steinke 2022, Proposition 17)

$$D_\alpha(M(D)\|M(D')) \geq k\varepsilon - \frac{k\ln(1+e^{-\varepsilon})}{\alpha-1}. \tag{40.6}$$

Proof. We compute the probability of "bad" on each dataset, compare the other outcome, and then keep only the bad term of the Rényi sum.

1. *Bad needs every run bad.* No outcome is worse than "bad", so Lemma 40.1 with $f(x) = x^k$ gives $\Pr[\text{best is bad}] = Q(\text{bad})^k$ on each dataset.
2. *The ratio for bad.* One run is bad with probability $e^{\varepsilon}/(1+e^{\varepsilon})$ on D and $1/(1+e^{\varepsilon})$ on D'. Raising both to the power k gives Equation 40.5.
3. *The ratio for good.* The good outcome has the ratio $(1-Q_D(\text{bad})^k)/(1-Q_{D'}(\text{bad})^k)$, which lies between $e^{-\varepsilon}$ and 1. The two datasets play symmetric roles, so the largest log ratio over outcomes and over both orders of the pair is $k\varepsilon$.
4. *The Rényi version.* Every term of the Rényi sum is positive, so the sum is at least its bad term. With the probabilities of step 2,

$$e^{(\alpha-1)D_\alpha} \geq \Pr_D[\text{bad}]^\alpha \Pr_{D'}[\text{bad}]^{1-\alpha} = e^{k\varepsilon\alpha}(1+e^{\varepsilon})^{-k}. \tag{40.7}$$

   Taking logarithms, dividing by $\alpha-1$ and writing $\ln(1+e^{\varepsilon}) = \varepsilon + \ln(1+e^{-\varepsilon})$ gives the bound.

□

The whole loss sits on one event, that every run failed, whose ratio is the product of k per-run ratios. Figure 40.1 (right) shows the privacy loss, the log ratio of an output's probabilities on D and D', for $\varepsilon = 1$. One run has losses ± 1, while the best of 10 has a loss of 10 with probability 0.0436. A DP-SGD run that sometimes diverges is such a run, with a "bad" outcome whose probability depends a little on the data.

The bound for a fixed k is therefore tight. Composition says that the best of k costs at most $k\varepsilon$ (Section 40.2), and Proposition 40.1 exhibits a run that attains it. No analysis that knows only each run's guarantee can promise less, so a better guarantee needs a different procedure. The next sections change one thing about it: the number of runs.

Example 40.1. With $\varepsilon = 1$, a run is good with probability 0.2689 on D and 0.7311 on D'. The largest log ratio is 2.0000 for $k = 2$, 3.0000 for $k = 3$ and 10.0000 for $k = 10$, exactly $k\varepsilon$. At order $\alpha = 2$, the Rényi divergence of the best of 10 is 6.8683. That lies between the lower bound of Proposition 40.1, 6.8674, and the composition bound, ten times one run's 0.7353, which is 7.3533. Composition overstates the exact value by only 7%, so the best of 10 costs almost as much as all 10 runs.

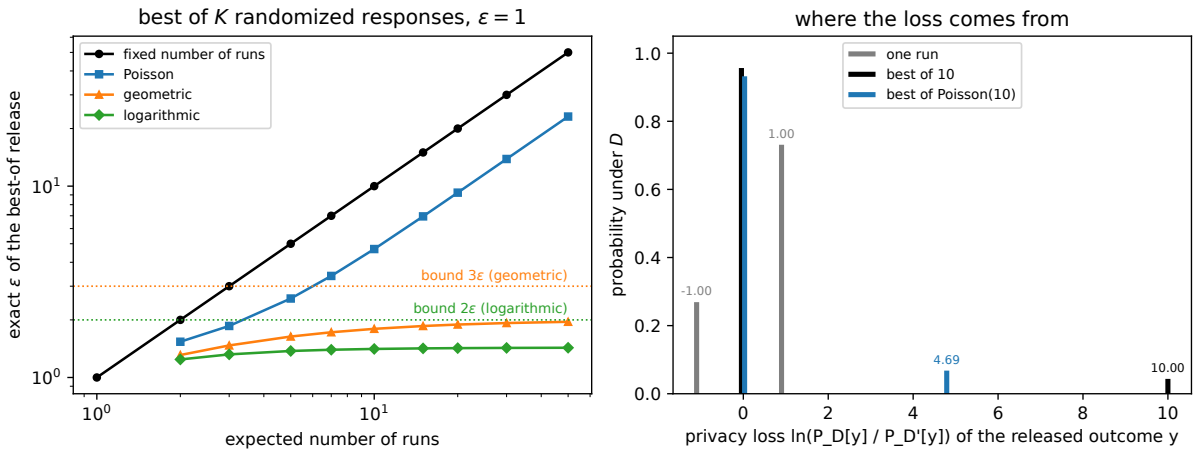


Figure 40.1.: Left: the exact ε of releasing the best of K randomized-response runs, each 1-DP, against the expected number of runs, for a fixed number of runs, a Poisson, a geometric and a logarithmic number. A fixed number grows linearly; the geometric and logarithmic numbers stay below their bounds 3ε and 2ε. Right: the privacy loss of the released outcome under D for one run, the best of 10 runs and the best of a Poisson number with mean 10; the best of 10 puts a loss of 10 on an event of probability 0.0436.

40.2. Paying by composition

The safe answer is to charge for every run. This section says what that costs, because it is the baseline every cleverer method must beat. The best output is computed from the k outputs alone, so it leaks no more than all of them, and they together cost the sum of their RDP curves.

Proposition 40.2 (Composition of k runs)**.** *Let each of k runs have RDP curve $\varepsilon(\alpha)$, and let the runs use independent randomness. Then releasing anything computed from the k outputs alone, in particular the best, has RDP curve*

$$\varepsilon_M(\alpha) \le k\,\varepsilon(\alpha), \tag{40.8}$$

even if each candidate is chosen after seeing the earlier results. For DP-SGD, k runs of T steps at noise multiplier σ have the curve of one run of kT steps.

Proof.

1. *The runs compose.* The curves of adaptively chosen mechanisms add (Theorem 29.2), so the k outputs together have curve $k\varepsilon(\alpha)$.
2. *The release is post-processing.* The best output is computed from the k outputs without reading the data, so its curve is no larger than theirs (Theorem 29.3).
3. *DP-SGD.* Write $\varepsilon_{q,\sigma}(\alpha)$ for the curve of one subsampled Gaussian step. One run of T steps then has curve $T\varepsilon_{q,\sigma}(\alpha)$, so k runs have $kT\varepsilon_{q,\sigma}(\alpha)$, the curve of one run of kT steps.

□

Every ε in this chapter comes from a curve by the conversion of From RDP to (ε, δ) (Theorem 30.1), minimized over the order. In (ε, δ) terms, k runs cost less than k times one run's ε, because the conversion pays its $\ln(1/\delta)$ term once. So 10 runs of the running example cost 4.06 times one run, not 10. But the curve grows linearly, and so does ε once the budget is large: from 30 to 100 runs, ε grows by a factor of 2.5.

Raising the noise keeps the budget: at a total of $\varepsilon = 6.71$, each of 10 runs needs $\sigma = 2.45$ instead of 1, which costs accuracy (DP-SGD). For two or three candidates, composition costs little more than the alternatives below. Figure 40.2 shows that the methods with a random number of runs pull away from composition only from a few runs on.

Example 40.2. Table 40.1 takes the running example, $q = 0.01$, $\sigma = 1$ and 10,000 steps, for which one run costs $\varepsilon = 6.71$ at $\delta = 10^{-5}$, and composes k runs.

Table 40.1.: Composition of k DP-SGD runs of the running example, at $\delta = 10^{-5}$.

Runs k	1	3	10	30	100
ε of k runs at $\sigma = 1$	6.71	12.73	27.23	58.25	145.17
Ratio to one run	1.00	1.90	4.06	8.68	21.63
σ per run for a total of 6.71	1	1.46	2.45	4.12	7.44

40.3. Random stopping

Liu & Talwar (2019) asked whether the best of many ε-DP runs can cost a constant multiple of ε, whatever the number of runs. They found that it can, with one change to the procedure: the number of runs becomes random. This section proves their result for pure DP runs, where the mechanism behind the Rényi versions of Section 40.4 is easiest to see.

Definition 40.1 (Random stopping)**.** Fix a stopping probability $\omega \in (0, 1)$. Repeat the following: run Q on the data and keep its output, then, with probability ω, stop and release the best output kept so far. The number of runs K is then geometric, with $\Pr[K = j] = \omega(1-\omega)^{j-1}$ for $j \geq 1$ and mean $1/\omega$.

The probability of each output turns out to be a ratio built from three probabilities of one run. Each moves by at most a factor e^{ε} between neighbours, and the stopping probability cannot amplify those moves.

Theorem 40.1 (Random stopping is 3ε-DP (Liu & Talwar 2019, Theorem 3.2))**.** *Let Q be ε-DP, and let M be random stopping with probability ω. For an outcome y, write p for the probability that one run outputs y, $p_>$ for the probability that one run is strictly better than y, and $p_\geq = p_> + p$. Then*

$$\Pr[M = y] = \frac{\omega\, p}{(\omega + (1-\omega)p_>)(\omega + (1-\omega)p_\geq)}, \tag{40.9}$$

and M is 3ε-DP for every ω. For every fraction $\pi \in (0,1]$, the release falls below the top π-quantile of one run's score with probability

$$\Pr[\textit{miss}] = \frac{\omega(1-\pi)}{\omega + (1-\omega)\pi} \le \frac{\omega}{\pi}. \tag{40.10}$$

Proof. The plan is to apply Lemma 40.1 with the geometric generating function, simplify the result to Equation 40.9, bound each of its three factors between neighbours, and finally sum a geometric series for the utility.

1. *Apply the lemma.* One run is no better than y with probability $1-p_>$ and strictly worse with probability $1-p_\ge$. With $f(x) = \omega x/(1-(1-\omega)x)$, and since $1-(1-\omega)(1-p_>) = \omega+(1-\omega)p_>$, Equation 40.2 gives

$$\Pr[M = y] = \frac{\omega(1-p_>)}{\omega + (1-\omega)p_>} - \frac{\omega(1-p_\ge)}{\omega + (1-\omega)p_\ge}. \tag{40.11}$$

2. *Simplify.* Over the common denominator, the numerator is ω times $(1-p_>)(\omega+(1-\omega)p_\ge) - (1-p_\ge)(\omega+(1-\omega)p_>)$. The terms with ω give $\omega(p_\ge - p_>) = \omega p$, and the terms with $1-\omega$ give $(1-\omega)(p_\ge - p_>) = (1-\omega)p$. Their sum is p, which gives Equation 40.9.
3. *Bound each factor.* Write primes for the same quantities on D'. Each of p, $p_>$ and $p_\ge$ is the probability of an event about one run's output, so ε-DP gives $p \le e^\varepsilon p'$, $p'_> \le e^\varepsilon p_>$ and $p'_\ge \le e^\varepsilon p_\ge$. The constant ω does not depend on the data, so $\omega+(1-\omega)p'_> \le e^\varepsilon(\omega+(1-\omega)p_>)$, and the same holds with $p_\ge$.
4. *Multiply.* The ratio of output probabilities is the product of the numerator ratio and the two inverted denominator ratios, each at most e^ε by step 3:

$$\frac{\Pr[M(D) = y]}{\Pr[M(D') = y]} = \frac{p}{p'} \cdot \frac{\omega + (1-\omega)p'_>}{\omega + (1-\omega)p_>} \cdot \frac{\omega + (1-\omega)p'_\ge}{\omega + (1-\omega)p_\ge} \le e^{3\varepsilon}. \tag{40.12}$$

   The same argument with D and D' swapped bounds the inverse ratio, and this holds for every outcome, so the release is 3ε-DP.
5. *Utility.* The release is below the top π-quantile only if every run is. Summing over the number of runs with the geometric series gives

$$\Pr[\text{miss}] = \sum_{j\ge 1} \omega(1-\omega)^{j-1}(1-\pi)^j = \frac{\omega(1-\pi)}{1-(1-\omega)(1-\pi)}, \tag{40.13}$$

   and the denominator equals $\omega + (1-\omega)\pi$.
6. *The simple bound.* The miss probability is at most ω/π exactly when $\pi(1-\pi) \le \omega(1-\pi)+\pi$, which holds because $\pi(1-\pi) \le \pi$.

□

This is why the disaster of a fixed k disappears. With $f(x) = x^k$ the ratio for the worst outcome is a k-th power, while here it is a product of three bounded ratios. The event "every run was bad" is now mostly the event "the coin stopped early", which does not depend on the data. The factor 3 is attained in the limit, by an outcome with something above it and something below it (Example 40.3). On randomized response the good outcome has nothing above it, so one factor cannot move and the cost tends to 2ε.

The utility guarantee is about quantiles, and it costs runs. Equation 40.10 does not promise the best candidate, only a release in the top π fraction of one run's outcomes with probability at least $1-\omega/\pi$. To land in the top $1/m$ of a grid of m with probability 0.9, set $\pi = 1/m$ and $\omega/\pi = 0.1$, which gives $\omega = 0.1/m$, a mean of $10m$ runs.

Example 40.3. On randomized response with $\varepsilon = 1$, geometric stopping has an exact ε of 1.3106 at $\omega = 0.5$, 1.7957 at $\omega = 0.1$, 1.9769 at $\omega = 0.01$ and 1.9977 at $\omega = 0.001$, approaching 2. To reach 3, take three outcomes, best first, with probabilities $(a, b, 1-a-b)$ on D and $(ae^{\varepsilon}, be^{-\varepsilon}, \dots)$ on D', and let $\omega \ll b \ll a$ be small. For the middle outcome, the numerator ratio of Equation 40.9 is e^{ε}. Since ω is negligible next to $p_{>}$, and $p_{\geq}$ is close to $p_{>}$, both denominator ratios approach e^{ε} too. Table 40.2 shows the exact ε approaching 3, and that of the logarithmic number of runs of Section 40.4 approaching 2.

Table 40.2.: Exact ε of the best-of release on a three-outcome mechanism that is exactly 1-DP, computed from Equation 40.2. The middle outcome attains the maximum, and Equation 40.9 gives the same value.

a	b	ω	Geometric stopping	Logarithmic number, same ω
0.01	0.001	0.1	1.2598	1.1299
0.01	0.0001	0.01	2.2316	1.6158
0.01	10^{-6}	10^{-4}	2.9874	1.9937
0.001	10^{-9}	10^{-6}	2.9987	1.9994

For utility, take a grid of 8 candidates with one best. With $\omega = 1/8$, a mean of 8 runs, the chance of never trying the top eighth is 0.4667. With $\omega = 1/80$, a mean of 80 runs, it falls to 0.0805 against a bound of 0.1. A target ten times finer, the top 1/80 of a finer grid, is then still missed with probability 0.4969.

40.4. Rényi bounds for a random number of runs

DP-SGD is not usefully ε-DP for any small ε, because its guarantee is an RDP curve that converts to (ε, δ). Papernot & Steinke (2022) extended random stopping to RDP and to a family of distributions for the number of runs, which lets you trade more runs against a lower privacy cost. This section proves their generic bound and its Poisson case, and states a third case. (They write λ for the order, μ for the Poisson mean and γ for the stopping probability.)

The generic bound separates one run's divergence from a factor that depends only on the derivative f' of the generating function at two points. Those points are acceptance probabilities of a test that reads only a run's output, which is what makes the bound useful: any privacy guarantee of the run keeps them close.

Theorem 40.2 (The generic bound (Papernot & Steinke 2022, Lemma 7))**.** *Let K have generating function f and be independent of the runs, and let M release the best of K independent runs of*

Q. For a pair of neighbours, write Q, Q' for the two run distributions, and M, M' also for the two best-of distributions. Then for every order $\alpha > 1$ there are numbers $s, s' \in [0,1]$ such that

$$D_\alpha(M\|M') \le D_\alpha(Q\|Q') + \frac{1}{\alpha-1}\ln\Big[f'(s)^\alpha f'(s')^{1-\alpha}\Big], \tag{40.14}$$

where s and s' are the probabilities that one and the same randomized test (step 5 of the proof) accepts a run's output, on D and on D'.

Proof. Take the outputs finite and $\Pr[K=0]=0$ to keep the steps short; the paper handles the general case, including the Poisson distribution, for which K can be 0. We write each best-of probability as one run's probability times an average of f', put that into the Rényi sum, bound the averages by their worst case, and recognize the worst case as a test.

1. *The best-of probability is an average of f'.* By Equation 40.2 and the fundamental theorem of calculus,
$$M(y) = f(Q(\preceq y)) - f(Q(\prec y)) = \int_{Q(\prec y)}^{Q(\preceq y)} f'(x)\,\mathrm{d}x. \tag{40.15}$$
The interval has length $Q(y)$, so $M(y) = Q(y)\,c(y)$, where $c(y)$ is the average of f' over that interval, the expectation of $f'(X)$ for X uniform on it. Likewise $M'(y) = Q'(y)\,c'(y)$.
2. *Put it in the Rényi sum.* Substituting step 1 into the definition of the divergence gives
$$e^{(\alpha-1)D_\alpha(M\|M')} = \sum_y M(y)^\alpha M'(y)^{1-\alpha} = \sum_y Q(y)^\alpha Q'(y)^{1-\alpha}\,c(y)^\alpha c'(y)^{1-\alpha}. \tag{40.16}$$
3. *Bound the averages by a worst case.* Couple X and X' at the same relative position $t \in [0,1]$ in their two intervals. The function $(u,v) \mapsto u^\alpha v^{1-\alpha}$ is jointly convex for $\alpha > 1$, so by Jensen's inequality $c(y)^\alpha c'(y)^{1-\alpha}$ is at most the average of $f'(X)^\alpha f'(X')^{1-\alpha}$. That average is at most the largest value of $f'(s)^\alpha f'(s')^{1-\alpha}$ over t, with $s = Q(\prec y) + tQ(y)$ and $s' = Q'(\prec y) + tQ'(y)$.
4. *Pull the worst case out of the sum.* Bounding every factor $c^\alpha c'^{1-\alpha}$ by the largest one over y and t leaves $\sum_y Q(y)^\alpha Q'(y)^{1-\alpha} = e^{(\alpha-1)D_\alpha(Q\|Q')}$ times that maximum. Taking logarithms and dividing by $\alpha-1$ gives Equation 40.14.
5. *The pair comes from one test.* The test "accept an output worse than y, accept y itself with probability t, reject anything better" accepts with probability s under Q and s' under Q'. The test reads only the run's output, so any privacy guarantee of Q constrains the pair (s, s'), by post-processing (Theorem 19.1, Theorem 29.3).

□

The first term is one run's divergence. The second is what the distribution of K adds, and each bound below comes from choosing f and bounding it.

For a Poisson number of runs, f' is an exponential, so the second term is linear in s and s'. An $(\hat\varepsilon, \hat\delta)$-DP guarantee of the run, applied to the event that the test rejects, then bounds it.

Theorem 40.3 (A Poisson number of runs (Papernot & Steinke 2022, Theorem 6))**.** *Let K be Poisson with mean $\mathbb{E}[K]$. Let Q be $(\alpha, \varepsilon(\alpha))$-RDP, and also $(\hat\varepsilon, \hat\delta)$-DP for some $\hat\varepsilon$ with $e^{\hat\varepsilon} \le 1 + \frac{1}{\alpha-1}$. Then*

$$\varepsilon_M(\alpha) \le \varepsilon(\alpha) + \mathbb{E}[K]\hat\delta + \frac{\ln \mathbb{E}[K]}{\alpha-1}. \tag{40.17}$$

Proof.

1. *Compute f'.* With $f(x) = e^{\mathbb{E}[K](x-1)}$, the derivative is $f'(x) = \mathbb{E}[K]e^{\mathbb{E}[K](x-1)}$. Taking the logarithm of the second term of Equation 40.14 and regrouping the exponents gives
$$\ln\Big[f'(s)^{\alpha} f'(s')^{1-\alpha}\Big] = \ln \mathbb{E}[K] + \mathbb{E}[K]\big[(\alpha-1)(1-s') - \alpha(1-s)\big]. \tag{40.18}$$
2. *Use the DP guarantee on the rejection event.* The numbers $1-s'$ and $1-s$ are the probabilities that the test of Theorem 40.2 rejects, on D' and on D. The run is $(\hat{\varepsilon}, \hat{\delta})$-DP, so $1 - s' \le e^{\hat{\varepsilon}}(1-s) + \hat{\delta}$, and the bracket is at most
$$(1-s)\big[(\alpha-1)e^{\hat{\varepsilon}} - \alpha\big] + (\alpha-1)\hat{\delta}. \tag{40.19}$$
3. *The condition removes the first term.* The condition $e^{\hat{\varepsilon}} \le 1 + \frac{1}{\alpha-1}$ is the same as $(\alpha-1)e^{\hat{\varepsilon}} \le \alpha$, so the first term is not positive and the bracket is at most $(\alpha-1)\hat{\delta}$. Dividing by $\alpha - 1$, and adding $D_\alpha(Q\|Q') \le \varepsilon(\alpha)$ in Equation 40.14, gives Equation 40.17.

□

The Poisson bound pays $\mathbb{E}[K]\hat{\delta}$ for the chance of many runs. That is linear in the expected number of runs, but multiplied by the run's $\hat{\delta}$ at a small $\hat{\varepsilon}$, so it is small only when one run is very private. On randomized response with $\varepsilon = 0.5$ and mean 10, the run has $\hat{\delta} = 0$ at $\hat{\varepsilon} = 0.5$, so Equation 40.17 applies at the order where $e^{0.5} = 1 + 1/(\alpha-1)$, which is $\alpha = 2.5415$. There the exact divergence is 0.4421, the bound 1.7654 and composition of 10 runs 2.7165.

A bound that stays a constant multiple of one run, whatever the run's $\hat{\delta}$, needs another distribution for K, from a family that contains the geometric distribution.

Definition 40.2 (Truncated negative binomial number of runs)**.** For a shape $\nu > -1$ and $\omega \in (0,1)$, the **truncated negative binomial** distribution on $\{1, 2, \dots\}$ has
$$\Pr[K=k] = \frac{(1-\omega)^k}{\omega^{-\nu} - 1} \prod_{i=0}^{k-1} \frac{i+\nu}{i+1}, \qquad \mathbb{E}[K] = \frac{\nu(1-\omega)}{\omega(1-\omega^{\nu})}. \tag{40.20}$$

The shape ν sets how skewed K is: the smaller ν, the more often $K = 1$. The case $\nu = 1$ is the geometric distribution. The limit $\nu = 0$ is the **logarithmic distribution**, with $\Pr[K=k] = \frac{(1-\omega)^k}{k\ln(1/\omega)}$ and mean $\frac{1/\omega - 1}{\ln(1/\omega)}$.

The theorem for this family uses the run's RDP guarantee at a second order $\hat{\alpha}$, chosen freely, to keep the rejection probabilities close.

Theorem 40.4 (A truncated negative binomial number of runs (Papernot & Steinke 2022, Theorem 2))**.** *Let K follow the distribution of Definition 40.2, and let Q be $(\alpha, \varepsilon(\alpha))$-RDP and $(\hat{\alpha}, \varepsilon(\hat{\alpha}))$-RDP for a second order $\hat{\alpha} \ge 1$. Then*
$$\varepsilon_M(\alpha) \le \varepsilon(\alpha) + (1+\nu)\Big(1 - \frac{1}{\hat{\alpha}}\Big)\varepsilon(\hat{\alpha}) + \frac{(1+\nu)\ln(1/\omega)}{\hat{\alpha}} + \frac{\ln \mathbb{E}[K]}{\alpha - 1}. \tag{40.21}$$

If Q is ε-DP, letting both orders grow gives $(2+\nu)\varepsilon$-DP (their Corollary 3). That is 3ε for the geometric distribution, as in Theorem 40.1, and 2ε for the logarithmic.

The proof follows that of Theorem 40.3, now with $f'(x) = \omega^{\nu+1}\mathbb{E}[K]\,(1-(1-\omega)x)^{-\nu-1}$. Step 2 now bounds the rejection probabilities through the run's RDP guarantee at the order $\hat{\alpha}$. Papernot & Steinke (2022, Appendix B.2) carry out the algebra, and Appendix Section M.3 does it step by step, with their Corollaries 3 and 4.

Each term of Equation 40.21 pays for something. The first is one run. The last is the price of the number of runs, and it is logarithmic, so ten times more runs in expectation adds only $\ln 10/(\alpha-1)$. The middle terms pay for how far the distribution of K lets an adversary tell "many runs, all bad" from "few runs", and they are a constant times the run's own privacy at another order. For a run with a linear RDP curve, $\varepsilon(\alpha) = \rho\alpha$, Papernot & Steinke's Corollary 4 turns the bound into a closed form whose overhead grows like $\sqrt{\ln \mathbb{E}[K]}$.

The bounds hold for every run with the given curve, so they can be loose for one run. On randomized response with $\varepsilon = 0.5$ and a logarithmic number of runs with mean 10, Table 40.3 compares the exact Rényi divergence of the release with Equation 40.21 and with composition of 10 runs. Randomized response is pure DP, so the bound keeps falling as $\hat{\alpha}$ grows, and the table gives its limit, in which the two middle terms become $(1+\nu)\varepsilon = 0.5$. At order 2, Equation 40.21 is worse than composition, because its constant terms dominate at small orders. At orders 5 and 20 it is 2.6 and 4.3 times below composition, and 3.7 and 1.7 times above the exact value.

Table 40.3.: Randomized response with $\varepsilon = 0.5$, best of a logarithmic number of runs with mean 10: the exact Rényi divergence against the bound and against composition.

Order α	Exact D_α	Bound Equation 40.21	Composition of 10 runs
2	0.1438	3.0299	2.2734
5	0.3912	1.4599	3.8424
20	0.6379	1.0962	4.7505

Example 40.4. The chapter's script (Section 40.8) evaluates Equation 40.17 and Equation 40.21 on the RDP curve of one DP-SGD run of the running example, with the best $\hat{\alpha}$, and with the $\hat{\delta}$ that Equation 30.7 gives at $\hat{\varepsilon} = \ln(1 + 1/(\alpha-1))$. It uses that a bound at a larger order holds at a smaller one (Proposition 29.1), and converts with Equation 30.8 (Table 40.4, Figure 40.2).

Table 40.4.: The ε at $\delta = 10^{-5}$ of tuning with DP-SGD runs of the running example (one run: $\varepsilon = 6.71$), by the number of runs and how it is chosen.

Expected runs	3	10	30	100	1,000
Composition	12.73	27.23	58.25	145.17	1035.93
Poisson	8.93	13.67	26.43	67.77	302.57
Geometric ($\nu = 1$)	9.22	11.31	12.85	14.32	16.76
$\nu = 0.5$	9.03	10.75	11.98	13.15	15.09
Logarithmic ($\nu = 0$)	8.75	10.08	11.04	11.95	13.46

For 10 runs in expectation, the logarithmic distribution costs 10.08, 1.5 times one run, against 27.23 for composition, and from 10 to 1,000 runs it adds only 3.38 more. The Poisson bound

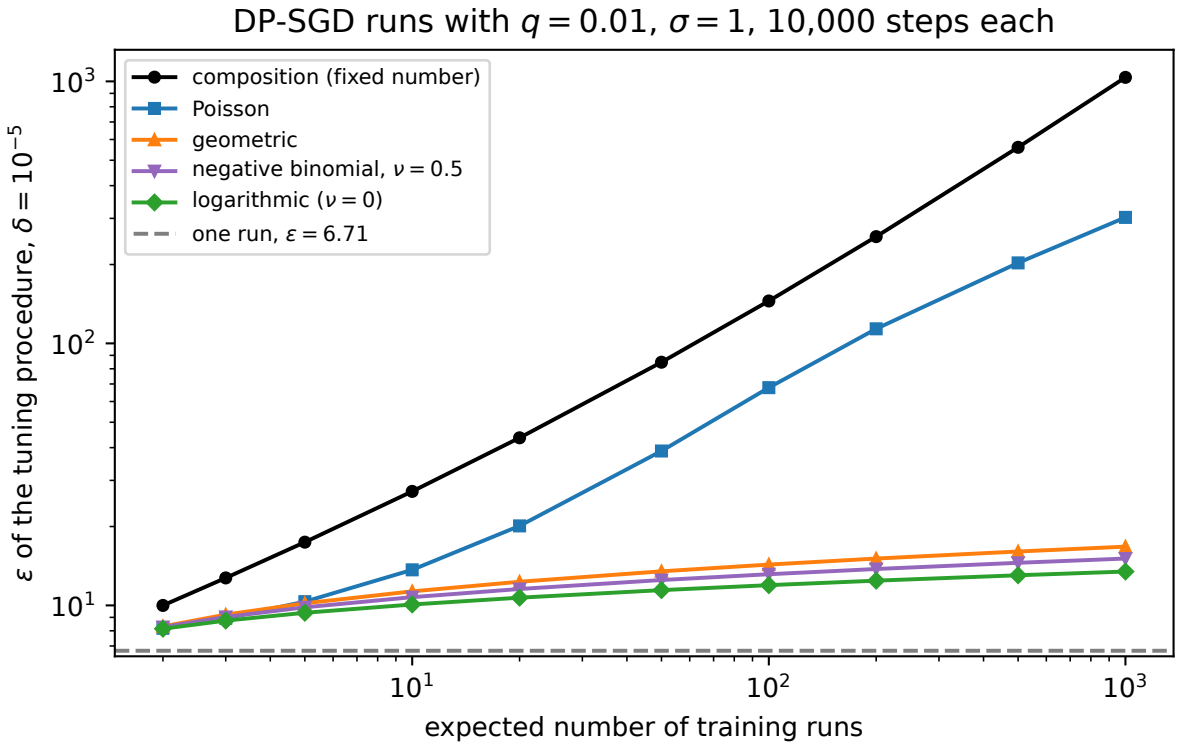


Figure 40.2.: The ε at $\delta = 10^{-5}$ of releasing the best of several DP-SGD runs of the running example ($q = 0.01$, $\sigma = 1$, 10,000 steps, one run $\varepsilon = 6.71$), against the expected number of runs, by composition, by the Poisson bound and by the truncated negative binomial bound with $\nu = 1$ (geometric), 0.5 and 0 (logarithmic). Both axes are logarithmic.

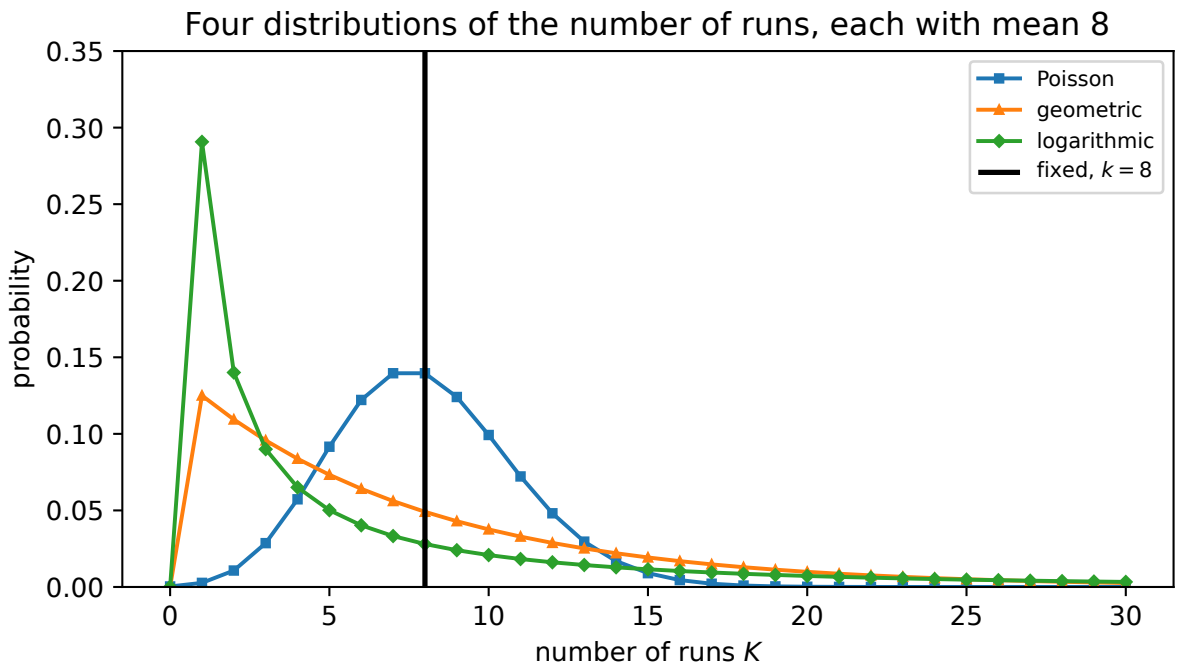


Figure 40.3.: Four distributions of the number of runs, each with mean 8: Poisson, geometric, logarithmic and a fixed 8. The logarithmic distribution stops after one run with probability 0.29.

starts well, at 8.93 for 3 runs, but grows nearly linearly, because a run at $\varepsilon = 6.71$ has a large $\hat{\delta}$ at the small $\hat{\varepsilon}$ the condition forces.

The cheaper bounds come with a more skewed number of runs. With mean 10, the logarithmic distribution has $\omega = 0.02692$ and stops after a single run with probability 0.2692, against 0.1000 for the geometric and 0.1562 for $\nu = 0.5$. The Poisson distribution, by contrast, is concentrated near its mean: with mean 10, $\Pr[K \le 1] = 4.99 \times 10^{-4}$. Figure 40.3 shows the four distributions with mean 8.

40.5. Selecting by a noisy validation score

The methods so far treat each run as a black box that outputs a model and its score. Often, though, the candidates are already fixed, such as public pretrained checkpoints, and only the choice among them uses private data, a held-out validation set. That is a selection problem, which Selection solved. This section says how those tools apply to tuning, and what they leave out: the cost of training the candidates, and the protection of the validation set.

Let the score of candidate r be its number of correct predictions $u(r)$ on a private validation set of m records. Adding or removing one validation record adds or removes one prediction, which is right or wrong, so every count moves by at most $\Delta u = 1$. The **exponential mechanism** picks r from the candidate set $\mathcal{R}$ with probability proportional to $e^{\varepsilon u(r)/(2\Delta u)}$. It is ε-DP in the

validation records (Theorem 22.1). Its expected gap to the best count is at most $\frac{2\Delta u}{\varepsilon}(\ln|\mathcal{R}|+1)$ (Theorem 22.2), an accuracy gap of $\frac{2}{\varepsilon m}(\ln|\mathcal{R}|+1)$.

When the candidates were themselves trained on private data, the cost has two parts, which fall on different records.

Proposition 40.3 (Trained candidates and a disjoint validation set)**.** *Let k candidates be trained on a private training set by runs with RDP curves $\varepsilon_1(\alpha),\dots,\varepsilon_k(\alpha)$. Let a selection mechanism choose among them on a private validation set disjoint from the training set, with RDP curve $\varepsilon_{sel}(\alpha)$ in the validation records for every fixed set of candidates. Then releasing the selected model costs*

$$\varepsilon_M(\alpha) \le \max\Big(\sum_{j=1}^{k}\varepsilon_j(\alpha),\ \varepsilon_{sel}(\alpha)\Big). \tag{40.22}$$

Proof.

1. *A training record.* If the neighbours differ in a training record, the validation set and the selection's noise are the same on both and do not depend on it. The release is then a post-processing (Theorem 29.3) of the k trained models, which compose (Theorem 29.2) to $\sum_j \varepsilon_j(\alpha)$.
2. *A validation record.* If the neighbours differ in a validation record, the trained models have the same distribution on both. Given the models, the release is the selection applied to fixed candidates, with curve at most $\varepsilon_{\text{sel}}(\alpha)$, and mixing both sides over the same distribution of models keeps the divergence below that worst case.
3. *Combine.* Neighbours differ in one record, which lies in one of the two sets, so the larger bound covers every pair, as in parallel composition (Theorem 19.4).

□

Selection is cheap, and training is not. The expensive part is that each candidate was *trained* on private data, which Equation 40.22 still charges by composition. If the candidates do not depend on private data, as with public checkpoints, that term vanishes, and choosing among them costs $\ln|\mathcal{R}|$ in utility and no linear cost in privacy.

The validation set needs protection under every method. The best-of methods of Section 40.3 and Section 40.4 assume that the score is part of each run's private output. If the score is an exact validation accuracy, the validation records are not protected at all. Adding Gaussian noise of standard deviation σ_{val} to each validation count makes each score a Gaussian mechanism with curve $\alpha/(2\sigma_{\text{val}}^2)$ (Theorem 29.1). By step 3 of the proof of Proposition 40.3, the counts then cost the larger of the training curve and their own composed curve. With $\sigma_{\text{val}} = 20$ correct predictions, one noisy count alone has $\varepsilon = 0.182$ at $\delta = 10^{-5}$, and the eight counts of a search, composed, have 0.546, far below eight training runs.

Example 40.5. In the experiment of Section 40.6, eight DP-SGD runs at $\sigma = 2.07$, with learning rates from 0.0625 to 8, get these counts out of $m = 5{,}000$: 4,486, 4,604, 4,681, 4,694, 4,600, 4,470, 4,232 and 4,123. Table 40.5 selects among them with the exponential mechanism.

Table 40.5.: The exponential mechanism selecting among eight trained models by their correct predictions on 5,000 private validation records.

ε_{sel}	Pr[best]	Expected gap (correct predictions)	Accuracy gap	Bound $2(\ln 8 + 1)/\varepsilon_{\text{sel}}$
0.01	0.2478	87.62	1.752%	615.9
0.1	0.6484	5.61	0.112%	61.6
1	0.9985	0.02	0.000%	6.2

At $\varepsilon_{\text{sel}} = 0.1$ the selection costs about a tenth of a point of accuracy. The training costs much more. Each run at $\sigma = 2.07$ and 1,000 steps has $\varepsilon = 0.657$, and the eight together 1.994. Selecting instead by the eight noisy counts of the best-of methods leaves the total at 1.994, because the counts' composed curve is below the training curve at the orders the conversion uses.

40.6. Tuning the running example

This section puts the strategies side by side on the running example, at the same total budget, and asks what each delivers in accuracy, which is the comparison a practitioner faces.

Definition 40.3 (Six ways to tune the learning rate)**.** Each run trains the running example's network with DP-SGD ($q = 0.01$, $C = 0.5$) for 1,000 steps, with one of eight learning rates $\eta \in \{0.0625, 0.125, \dots, 8\}$, and is scored by its correct predictions on 5,000 private validation records plus Gaussian noise with $\sigma_{\text{val}} = 20$. Each strategy releases one model and chooses its noise multiplier σ so that the whole procedure meets $\varepsilon = 2$ at $\delta = 10^{-5}$.

1. *No tuning:* one run with the learning rate 0.5 that DP-SGD used.
2. *The reporting bug:* the eight runs at the σ of one run, the best released, and the single-run ε reported, though the search really spends more.
3. *Proxy data:* the grid run on a related public task, then one private run with its best learning rate.
4. *Composition:* the eight runs at the σ for which eight runs together cost 2 (Proposition 40.2).
5. *Poisson random search:* K runs with learning rates drawn uniformly from the grid, K Poisson with mean 8 (Theorem 40.3).
6. *Logarithmic random search:* the same with K logarithmic (Theorem 40.4).

Runs take 1,000 steps instead of 10,000 to keep the script fast. Of the running example's 20,000 held-out records, 5,000 form the validation set and 15,000 the test set. The proxy task is labelled by a different random network and touches no private data, so it costs nothing (Theorem 19.1). The script trains three runs per learning rate for each σ and repeats each procedure 4,000 times by drawing from this pool, with fresh validation noise and a fresh K, which understates the variety of a real search a little.

Example 40.6. Table 40.6 gives the result. Suppose one paper reports "test accuracy 0.948 at $\varepsilon = 1.97$" for a model chosen from eight learning rates, and another 0.943 at $\varepsilon = 1.99$ after

an honest grid search. The first is the reporting bug of Table 40.6, which spends 5.60, so the difference is the accounting. At its honest cost, the first paper's model should be compared with methods at $\varepsilon = 5.60$.

Table 40.6.: Tuning the learning rate of the running example (1,000 steps per run) at a total budget of $\varepsilon = 2$, $\delta = 10^{-5}$. Test accuracy: mean and standard deviation over repetitions of the whole procedure; runs: the average number of private training runs.

Strategy	σ	ε of one run	Total ε	Test accuracy	Runs on private data
No tuning: learning rate 0.5 from DP-SGD	1.03	1.97	1.97	0.9509 ± 0.0015	1
Grid of 8, single-run ε reported	1.03	1.97	5.60	0.9479 ± 0.0030	8
Tune on proxy data, then one run	1.03	1.97	1.97	0.9487 ± 0.0025	1
Grid of 8, composition	2.07	0.66	1.99	0.9430 ± 0.0022	8
Random search, Poisson, mean 8	1.53	0.98	1.99	0.9433 ± 0.0060	8.01
Random search, logarithmic, mean 8	1.36	1.17	2.00	0.9372 ± 0.0200	7.95

The honest strategies cost about a point of accuracy here, and the bug hides it. At the same total budget, composition reaches 0.9430 and Poisson random search 0.9433, against 0.9509 without tuning, while the bug claims 1.97 and spends up to 5.60. Even with its budget misreported, the grid search, at 0.9479, is below the untuned learning rate. The learning rate 0.5 is the best of the grid, and the noisy validation count sometimes picks a neighbour. It is the best because DP-SGD chose its settings with knowledge of this very task, a leak that no accountant in the book has charged for. Tuning on proxy data is nearly free, reaching 0.9487 for the cost of one private run.

Among the honest searches, Poisson random search allows $\sigma = 1.53$ instead of composition's 2.07, so each run costs 0.98 instead of 0.66. But each learning rate is drawn a Poisson number of times with mean 1, so it is missed with probability $e^{-1} = 0.3679$, and its accuracy varies more from one search to the next, 0.0060 against 0.0022, so at this small grid the lower noise and the missed learning rates roughly cancel. The logarithmic distribution allows the least noise, $\sigma = 1.36$, but with mean 8 it stops after a single run with probability 0.2907 and then releases a random learning

rate. That is why its accuracy is the lowest and most variable, 0.9372 ± 0.0200. Figure 40.4 repeats the comparison at $\varepsilon = 1$ and 4.

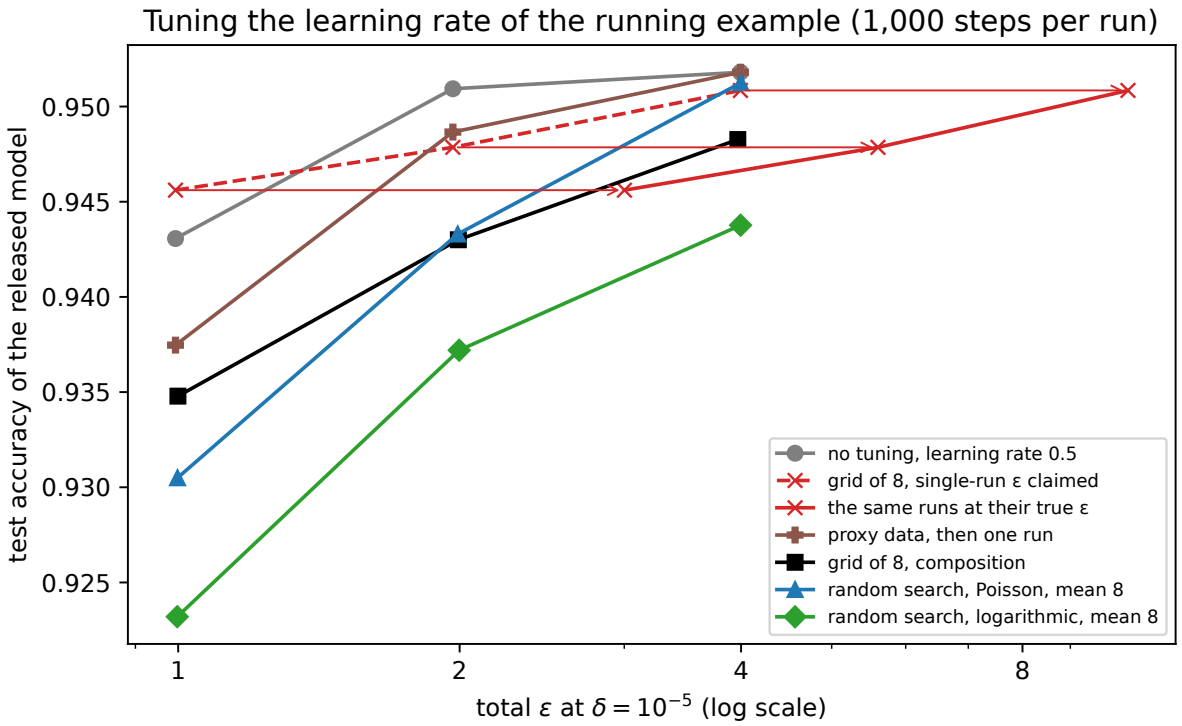


Figure 40.4.: Test accuracy of the model released by each tuning strategy against the total ε at $\delta = 10^{-5}$ it costs (1,000 DP-SGD steps per run). The red dashed line places the grid search at the single-run ε that the reporting bug claims, and the arrows move each point to its honest cost by composition. Points at $\varepsilon = 2$ use three runs per learning rate; those at 1 and 4 one run.

40.7. What to do and what to report

The results of this chapter turn into a short procedure, and into a rule for what a reported ε must cover.

Definition 40.4 (An honest tuning procedure)**.**

1. *Spend private runs only where you must.* Tune on public or proxy data when it exists, and reuse hyperparameters from similar tasks. Shrink the search: below the typical gradient norm only the product of learning rate and clipping norm matters (DP-SGD).
2. *Fix the search before touching the private data.* Write down the candidate set, the distribution of the number of runs, the score and the validation set.
3. *Make the score private.* Use a held-out validation set and a noisy count.
4. *Account for the whole search.* For a few candidates, compose (Proposition 40.2). For more, use random search with Theorem 40.3 or Theorem 40.4. Count every run on the private data, including crashed runs, debugging runs and evaluations for early stopping.

5. *Report both numbers.* Give the ε of the released run, and the ε of the whole procedure with how it was computed: the candidate set, the distribution of K, the accountant, δ and the neighbouring relation.

The cheapest fix is usually upstream: in Table 40.6, the strategies that spend the whole budget on one private run beat every honest search. Tuning without noise does not escape the problem, because the hyperparameters are then a non-private function of the data. Papernot & Steinke (2022, Section 2) build a support vector machine whose best regularization weight reveals whether a few outliers were in the training set, a membership inference through a single number. Retraining the chosen configuration with fresh noise does not make the tuning free either, because the configuration is still a function of the tuning runs.

Fixing the search in advance is part of the guarantee. The theorems of Section 40.3 and Section 40.4 require that each run draws its candidate independently from a fixed distribution, which is random search, and that K is drawn independently of the data. Adaptive methods such as Bayesian optimization, which pick the next candidate from the scores so far, are not covered and need Adaptive composition. An engineer who looks at the first results and decides to "try a few more" has changed the procedure, and the theorem no longer applies.

Both numbers in step 5 matter. The released run's ε says what the model alone would leak if its hyperparameters were public, while the procedure's ε is what the release guarantees. Ponomareva et al. (2023, Section 5.3.3) ask for both. Auditing cannot catch the bug, because an audit tests the trained model, not how it was chosen.

Example 40.7. For the Poisson row of Table 40.6, a complete report reads: "DP-SGD with Poisson sampling rate 0.01, clipping norm 0.5, noise multiplier 1.53, 1,000 steps; one run satisfies $(0.98, 10^{-5})$-DP. The learning rate was selected by random search over $\{0.0625, 0.125, \dots, 8\}$ with a Poisson number of runs with mean 8, scored by correct predictions on a disjoint validation set of 5,000 records with Gaussian noise of standard deviation 20; the whole procedure satisfies $(1.99, 10^{-5})$-DP by the Rényi bound of Papernot & Steinke (2022, Theorem 6), add/remove neighbours, record level."

40.8. Code

The script code/hyperparameter_tuning.py computes the exact best-of-K distributions of Equation 40.2, evaluates the bounds on the RDP accountant it imports from code/dp_sgd.py, and runs the tuning experiment, about 200 DP-SGD runs, in two to four minutes. From the repository root:

```
uv run python code/part05-dp-ml/hyperparameter_tuning.py
```

Table 40.7.: The script's output with the default seed.

Experiment	Result
Best of 10 randomized responses, $\varepsilon = 1$	Exactly $\varepsilon = 10$; loss 10 with probability 0.0436

Experiment	Result
Composition, running example	27.23 for 10 runs, 145.17 for 100
Geometric stopping, three outcomes	Exact ε up to 2.9987, below the bound 3
Logarithmic number of runs, running example	10.08 for 10 runs, 13.46 for 1,000
Exponential mechanism over 8 models	$\Pr[\text{best}] = 0.6484$ at $\varepsilon = 0.1$
Tuning at $\varepsilon = 2$	Honest search 0.9430 to 0.9433; bug claims 1.97 and spends 5.60

tests/test_hyperparameter_tuning.py checks that the output still matches.

40.9. Summary

Table 40.8.: The results of this chapter, and where each is used.

Result	Statement	Use
Lemma 40.1	$\Pr[M = y] = f(Q(\preceq y)) - f(Q(\prec y))$, with $f(x) = \mathbb{E}[x^K]$	Every best-of computation; needs K independent of the runs
Proposition 40.1	The best of k randomized responses is exactly $k\varepsilon$-DP	The best of k runs is no cheaper than k runs unless k is random
Proposition 40.2	$\varepsilon_M(\alpha) \le k\varepsilon(\alpha)$, valid always, even adaptively	Two or three candidates; report the cost of the search, not of the run
Theorem 40.1	Random stopping is 3ε-DP; it misses the top π fraction with probability at most ω/π	Pure-DP runs; $10m$ runs on average to reach the top $1/m$ with probability 0.9
Theorem 40.2	$\varepsilon(\alpha) + \frac{1}{\alpha-1}\ln[f'(s)^\alpha f'(s')^{1-\alpha}]$, with s, s' from one test	The source of every bound for a random number of runs
Theorem 40.3	$\varepsilon(\alpha) + \mathbb{E}[K]\hat{\delta} + \frac{\ln \mathbb{E}[K]}{\alpha-1}$, if $e^{\hat{\varepsilon}} \le 1 + \frac{1}{\alpha-1}$	Small searches at small per-run budgets
Theorem 40.4	One run, plus a multiple of one run at order $\hat{\alpha}$, plus $\frac{\ln \mathbb{E}[K]}{\alpha-1}$; pure DP: $(2+\nu)\varepsilon$	Large searches, at the price of often running few candidates
Proposition 40.3	$\max(\sum_j \varepsilon_j(\alpha), \varepsilon_{\text{sel}}(\alpha))$ for a disjoint validation set	Validation scores must be noisy; a few thousand records make that cheap
Definition 40.3	Six strategies at a total $\varepsilon = 2$	Honest search costs about a point; the bug claims 1.97 and spends 5.60

Result	Statement	Use
Definition 40.4	Tune off the private data, fix the search, noise the score, count every run, report both numbers	Adaptive searches need adaptive composition

40.10. References

- Liu & Talwar, "Private Selection from Private Candidates" (STOC 2019; arXiv:1811.07971). Random stopping and the 3ε bound of Section 40.3; threshold and (ε, δ) variants; the fixed-k counterexample (their Appendix B).
- Papernot & Steinke, "Hyperparameter Tuning with Renyi Differential Privacy" (ICLR 2022; arXiv:2110.03620). The generic bound, the Poisson and truncated negative binomial theorems of Section 40.4, the lower bound of Section 40.1, and an experiment on image classification that recommends the Poisson distribution for small searches.
- Chaudhuri & Vinterbo, "A Stability-Based Validation Procedure for Differentially Private Machine Learning" (NeurIPS 2013). The first improvement over composition for tuning, for learning algorithms that are stable.
- Mohapatra, Sasy, He, Kamath & Thakkar, "The Role of Adaptive Optimizers for Honest Private Hyperparameter Selection" (arXiv:2111.04906, 2021). Honest tuning with composition, and optimizers that need less tuning.
- McSherry & Talwar, "Mechanism Design via Differential Privacy" (FOCS 2007). The exponential mechanism of Section 40.5.
- Papernot, Thakurta, Song, Chien & Erlingsson, "Tempered Sigmoid Activations for Deep Learning with Differential Privacy" (arXiv:2007.14191, 2020). Architecture choices that are best without privacy are not best with it, so tuning cannot simply be done non-privately.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023). What to tune, how, and what to report.
- Bergstra & Bengio, "Random Search for Hyper-Parameter Optimization" (JMLR 2012). Why random search, which the random-K theorems require, is a sound tuning method in its own right.

41. DP-FTRL and matrix factorization

Stochastic gradient descent (SGD) with differential privacy (DP), the DP-SGD of the previous chapter, owes much of its privacy to **Poisson sampling**. A record is in each batch only with probability q, and the accountant credits the run for that uncertainty. On the running example of this part, with $q = 0.01$, this amplification turns a noise multiplier of 1 into $\varepsilon = 6.71$ over 10,000 steps. But fixed batches, streaming pipelines and federated servers cannot honestly claim Poisson sampling, and Amplification by subsampling shows that the Poisson accountant then understates ε. Without amplification, for $\varepsilon = 2$ over five passes, the noise multiplier grows from 0.928 to 4.458 and test accuracy falls from 0.9460 to 0.8941.

This chapter asks how to train privately without trusting random batches. The one idea is that training releases running sums of gradients, so noise added at one step may partly cancel the noise added before it. Releasing all running sums is described by a matrix A, and writing $A = LR$ splits the release into an encoder R, which sets the privacy, and a decoder L, which sets the error. The first such algorithm ran the follow the regularized leader (FTRL) optimizer on sums noised through a binary tree, and is called DP-FTRL. That tree, the square-root factorization and the best known mechanisms are all choices of L and R.

The running example is the part's network from DP-SGD, with $d = 3{,}329$ weights, $n = 10{,}000$ records, clipping norm $C = 0.5$ and $\delta = 10^{-5}$. The fixed-order runs cut one public random permutation, drawn independently of the data, into 100 batches of 100, so each record takes part once per epoch, 100 steps apart. Table 41.1 lists the symbols.

Table 41.1.: The symbols of this chapter.

Symbol	Meaning
T, t; E	number of steps and a step; number of epochs
x_t, stacked as the rows of x; S_t, $\hat{S}_t$	clipped gradient sum of step t; its prefix sum $x_1 + \cdots + x_t$, and the private estimate
A	the $T \times T$ lower-triangular matrix of ones, so that Ax stacks the prefix sums
$A = LR$; r_j, ℓ_t	a factorization into encoder R and decoder L; column j of R, row t of L
z; s	Gaussian noise; its standard deviation in units of the clipping norm C
μ; $\mu(\varepsilon, \delta)$	sensitivity over noise standard deviation; the largest μ whose Gaussian release is (ε, δ)-DP
$\Delta_2 R$; e_t	the encoder's sensitivity in units of C; the error of $\hat{S}_t$ per unit of privacy

Symbol	Meaning
c_k; h_k	the coefficients of the square-root factorization and of its inverse

41.1. Training is a stream of prefix sums

What does a training run have to release, and what does privacy cost when nothing is sampled? The answer turns training into the private release of running sums, a classic problem with much better mechanisms than fresh noise on every step. The reason is that gradient descent adds up its steps. Write B for the batch size and η for the learning rate.

Proposition 41.1 (Training releases prefix sums)**.** *Run gradient descent from θ_0 with a constant learning rate η and no momentum, $\theta_t = \theta_{t-1} - \frac{\eta}{B}\hat{x}_t$, where $\hat{x}_t$ is the possibly noisy clipped gradient sum of step t. Then*

$$\theta_t = \theta_0 - \frac{\eta}{B}\hat{S}_t, \qquad \hat{S}_t = \sum_{s \le t} \hat{x}_s, \tag{41.1}$$

so every model is post-processing of the prefix sums, and its noise is η/B times theirs. The FTRL algorithm with linear losses $\langle \hat{x}_s, \theta \rangle$, $\theta_t = \arg\min_\theta \left(\langle \hat{S}_t, \theta \rangle + \frac{B}{2\eta}\|\theta - \theta_0\|^2 \right)$, produces the same weights. Releasing all T prefix sums is the linear workload

$$S = Ax, \qquad A = \begin{pmatrix} 1 & & & \\ 1 & 1 & & \\ \vdots & & \ddots & \\ 1 & 1 & \cdots & 1 \end{pmatrix} \in \mathbb{R}^{T \times T}, \tag{41.2}$$

where the x_t are stacked as the rows of the $T \times d$ matrix x, and row t of Ax is S_t.

Proof.

1. *Unroll the recursion.* The updates $\theta_u - \theta_{u-1} = -\frac{\eta}{B}\hat{x}_u$ for $u = 1, \dots, t$ telescope when added, which gives Equation 41.1.
2. *FTRL.* The FTRL objective is a linear term plus a quadratic. Its gradient is $\hat{S}_t + \frac{B}{\eta}(\theta - \theta_0)$, and setting it to zero gives $\theta = \theta_0 - \frac{\eta}{B}\hat{S}_t$, the same point.
3. *Post-processing.* Every model is a fixed function of the released prefix sums, and post-processing never weakens a guarantee (Theorem 19.1).

□

Why release every prefix sum, and not only the last? Training is adaptive, since the gradient x_t is computed at θ_{t-1}, which depends on the earlier noisy prefix sums. So the intermediate models must be private too, and the workload is all T rows of A, released one at a time as the stream arrives. With momentum or a learning-rate schedule the weights are still fixed linear combinations of the noisy gradients, with another lower-triangular matrix in place of A, which Denisov, McMahan, Rush, Smith & Thakurta (2022) factor too. This chapter keeps plain gradient descent.

Two facts from earlier chapters price every mechanism here. The first is about privacy. Release a matrix $f(D)$ plus independent normal noise of standard deviation sC on every entry, and measure the change in f between neighbours in the Frobenius norm, the length of the matrix read as one long vector. Then only the ratio μ of that change to sC matters, and the release is (ε, δ)-DP exactly when δ is at least the Gaussian privacy profile of Theorem 20.3 at μ. Such a release satisfies μ-Gaussian differential privacy (GDP). The profile grows with μ, so each target allows a largest ratio $\mu(\varepsilon, \delta)$, and at $\delta = 10^{-5}$ these are $\mu(2, \delta) = 0.502$ and $\mu(8, \delta) = 1.666$.

The second fact is about error. A weighted sum of independent normals is normal, with variance the sum of the variances times the weights squared (Proposition 1.11). So if z has independent entries of standard deviation sC and ℓ is a fixed row vector, each coordinate of ℓz is normal with variance

$$s^2 C^2 \|\ell\|^2 = s^2 C^2 \sum_j \ell_j^2. \tag{41.3}$$

Neighbouring datasets $D \sim D'$ differ by adding or removing one record, whose slot in the public order is then empty in one stream. (Under replace-one neighbours, a contribution could change by $2C$, and every sensitivity would double.) The two obvious mechanisms put the noise at opposite ends, on each step's gradient sum or on each prefix sum.

Proposition 41.2 (The two naive mechanisms)**.** *Let a record take part in one step τ, fixed in advance, and add a vector of length at most C to x_τ. Then each of the following is μ-GDP.*

- ***Independent noise on each step**, $\hat{S}_t = \sum_{u \le t}(x_u + z_u)$ with noise of standard deviation C/μ per coordinate. The error of $\hat{S}_t$ has standard deviation $\sqrt{t}\, C/\mu$.*
- ***Independent noise on each prefix sum**, $\hat{S}_t = S_t + z_t$ with noise of standard deviation $\sqrt{T}\, C/\mu$ per coordinate, which is also the error of every $\hat{S}_t$.*

Proof. The plan is to find, for each mechanism, how far one record moves the noisy quantities, which fixes the noise by the first fact, and then to read the error off Equation 41.3.

1. *Independent noise on each step: privacy.* The record changes only x_τ, by a vector of length at most C, so the noisy gradients $x_u + z_u$ form one Gaussian mechanism with ratio μ, and the prefix sums are post-processing of them.
2. *Its error.* The error is $\hat{S}_t - S_t = z_1 + \cdots + z_t$, which is Equation 41.3 with $\ell = (1, \dots, 1, 0, \dots, 0)$, carrying t ones. So its standard deviation is $\sqrt{t}\, C/\mu$.
3. *Independent noise on each prefix sum: privacy.* The record's gradient appears in every S_t with $t \ge \tau$. So $T - \tau + 1$ of the released vectors move by the same vector, and the total Frobenius change is $\sqrt{T - \tau + 1}\, C$. The worst position, $\tau = 1$, gives $\sqrt{T}\, C$, and the noise must be $\sqrt{T}\, C/\mu$.
4. *Its error.* Each $\hat{S}_t$ carries one noise vector of that standard deviation.

□

DP-SGD run in a fixed order without sampling is the first mechanism, so its model at step t carries a random walk of t noise vectors. Neither mechanism uses the structure of A, because the first ignores that the noise accumulates and the second that most of S_t was already released in S_{t-1}.

Example 41.1. Take one epoch, $T = 100$ steps, and normalize $\mu = 1$ and $C = 1$. Independent noise on each step gives the final prefix sum an error standard deviation of $\sqrt{100} = 10$. Averaged over the 100 prefix sums, its root-mean-square error is

$$\sqrt{(1 + 2 + \cdots + 100)/100} = \sqrt{50.5} = 7.11. \tag{41.4}$$

Independent noise on each prefix sum has error 10 on every one.

At $\varepsilon = 2$, $\mu = 0.502$, so the first mechanism needs a noise scale of $1/0.502 = 1.994$ per step, and its final prefix sum carries noise of standard deviation 19.94, both in units of C. DP-SGD with Poisson sampling at the same ε needs only $\sigma = 0.827$, and its final prefix sum carries $0.827\sqrt{100} = 8.27$. Amplification buys a factor of 2.4 here, and correlated noise can beat even that without any sampling.

41.2. The matrix mechanism

The two naive mechanisms put the noise on the inputs or on the answers. The **matrix mechanism** of Li, Hay, Rastogi, Miklau & McGregor (2010) puts it in between, on a linear transformation of the inputs from which the answers can be recovered linearly. This section asks how private and how accurate such a release is, even on an adaptive stream, and how accurate any such release can be.

Definition 41.1 (Matrix mechanism)**.** A **factorization** of the prefix-sum matrix is a pair $A = LR$, with an **encoder** $R \in \mathbb{R}^{m \times T}$ and a **decoder** $L \in \mathbb{R}^{T \times m}$. Write r_j for column j of R and ℓ_t for row t of L. The matrix mechanism with noise scale s releases

$$\hat{S} = L(Rx + z) = Ax + Lz, \qquad z_{jk} \sim \mathcal{N}(0, s^2 C^2) \text{ independent.} \tag{41.5}$$

It is **streaming** when L and R are lower-triangular, so that $\hat{S}_t$ needs only the inputs and noise up to step t.

The encoder decides where each step's gradient lands, so it sets the privacy, and the decoder decides how much noise reaches each answer, so it sets the error.

Theorem 41.1 (Privacy and error of the matrix mechanism)**.** *Let a record take part in one step, at a position fixed in advance. Then the release Equation 41.5 is μ-GDP with*

$$\mu = \frac{\Delta_2 R}{s}, \qquad \Delta_2 R = \max_j \|r_j\|, \tag{41.6}$$

the largest Euclidean length of a column of R. If the factorization is streaming, this holds even when each x_t is chosen after seeing $\hat{S}_1, \dots, \hat{S}_{t-1}$. Each coordinate of $\hat{S}_t - S_t$ is normal with mean zero and standard deviation $sC\|\ell_t\|$. At a fixed privacy level μ this is

$$\frac{C}{\mu}\, e_t, \qquad e_t = \Delta_2 R\, \|\ell_t\|, \tag{41.7}$$

*and the **error per unit of privacy** e_t does not change if R is multiplied by any number $c > 0$ and L divided by it.*

Proof. The plan is to compute how far one record moves the noisy encoding $Rx + z$, extend that to adaptive inputs, and then read the error off the decoder.

1. *Reduce to the encoding.* L is fixed, so $\hat{S}$ is post-processing of $Rx + z$ (Theorem 19.1), and $LRx = Ax$ by the factorization.
2. *Sensitivity for fixed inputs.* Neighbouring streams differ in the record's contribution g to x_τ, a vector of length at most C, so row τ of x changes by g and nothing else. Then Rx changes by $r_\tau g^\top$, column τ of R times the row g. Its entries are $r_{j\tau} g_k$, so its squared Frobenius norm factorizes, and its Frobenius norm is $\|r_\tau\| \, \|g\| \le \|r_\tau\| C$. The worst position gives $\Delta_2 R \, C$, and with noise of standard deviation sC the ratio is $\Delta_2 R / s$.
3. *Adaptive inputs.* With lower-triangular factors, row j of $Rx + z$ depends on $x_1, \dots, x_j$ only, so it can be released at step j. Because R is lower-triangular, rows before τ do not involve x_τ, so the earlier outputs have the same distribution under both streams, and conditioning on them fixes g. Given the same history, the other records' gradients agree in both runs, so each later row of $Rx + z$ has its mean shifted by the matching row of $r_\tau g^\top$. These are Gaussian mechanisms released one after another, with squared shifts adding up to at most $\|r_\tau\|^2 C^2$. Adaptive Gaussian steps compose into one Gaussian mechanism whose squared ratio is the sum of theirs (Gaussian DP), so the ratio is at most $\Delta_2 R / s$ again. Denisov et al. (2022, Theorem 2.1) prove the general statement, and Section H.5 in the appendices proves the Gaussian case for several participations.
4. *Error.* The error is $\hat{S}_t - S_t = \ell_t z$, so Equation 41.3 gives standard deviation $sC\|\ell_t\|$. Substituting $s = \Delta_2 R / \mu$ from Equation 41.6 gives Equation 41.7. Replacing R by cR and L by L/c multiplies $\Delta_2 R$ by c and $\|\ell_t\|$ by $1/c$, so their product is unchanged.

□

So privacy lives in the columns and error in the rows. Insisting on streaming factors costs nothing, because any factorization has a lower-triangular one with the same distribution of $\hat{S}$ (Denisov et al. 2022, Proposition 2.2), which Section H.5 derives by Gram–Schmidt.

When R is square and invertible, the mechanism is noisy SGD with correlated noise. Then $L = AR^{-1}$ and $\hat{S} = A(x + R^{-1}z)$, so the mechanism adds the noise $\zeta = R^{-1}z$ to the gradients and then sums. An implementation generates each ζ_t on the fly from the stored past noise.

How small can the error be? The squared row lengths of L add up to its squared Frobenius norm $\|L\|_F^2$, so the root-mean-square error over the T prefix sums is

$$\Big(\frac{1}{T}\sum_t e_t^2\Big)^{1/2} = \frac{\Delta_2 R \, \|L\|_F}{\sqrt{T}}. \tag{41.8}$$

The bound below uses the **trace norm** $\|A\|_* = \sum_k \nu_k(A)$, the sum of the singular values $\nu_k(A)$ of A.

Theorem 41.2 (The trace-norm lower bound)**.** *For every factorization $A = LR$,*

$$\frac{\Delta_2 R \, \|L\|_F}{\sqrt{T}} \ge \frac{\|A\|_*}{T}. \tag{41.9}$$

For the prefix-sum matrix, $\nu_k(A) = 1/(2\sin\frac{(2k-1)\pi}{4T+2})$ for $k = 1, \dots, T$.

Proof. The plan is to bound $\|A\|_*$ by the product $\|L\|_F\|R\|_F$, then bound $\|R\|_F$ by its longest column.

1. *Duality.* For any matrix U whose largest singular value is at most 1, $\operatorname{tr}(U^\top A) \le \|A\|_*$, with equality for a U built from the singular vectors of A. This dual description of the trace norm is proved in Section H.3 in the appendices.
2. *Cauchy–Schwarz.* The trace $\operatorname{tr}(U^\top LR)$ is the Frobenius inner product of $L^\top U$ and R, so the Cauchy–Schwarz inequality gives
$$\operatorname{tr}(U^\top LR) \le \|L^\top U\|_F \, \|R\|_F. \tag{41.10}$$
3. *Remove U.* The matrix $U^\top$ does not lengthen any vector, since its singular values are at most 1. Applied to each column of L, this gives $\|L^\top U\|_F = \|U^\top L\|_F \le \|L\|_F$. With step 1, $\|L\|_F\|R\|_F \ge \|A\|_*$.
4. *Bound $\|R\|_F$ by the sensitivity.* R has T columns, each of length at most $\Delta_2 R$, so $\|R\|_F^2 \le T(\Delta_2 R)^2$. With step 3, this gives $\Delta_2 R\,\|L\|_F \ge \|A\|_*/\sqrt{T}$, which is Equation 41.9 after dividing by $\sqrt{T}$.
5. *The singular values.* The inverse A^{-1} has 1 on the diagonal and -1 just below it. The eigenvalues of $(A^{-1})^\top A^{-1}$ are $4\sin^2\frac{(2k-1)\pi}{4T+2}$, because that product is the second-difference matrix with a 1 in its last diagonal entry, whose eigenvectors are sine waves; Section H.3 derives them. The singular values of A are the reciprocals of their square roots.

□

Step 4 is tight only when all columns of R are equally long, so unequal columns waste privacy on the short ones. The chapter's script (Section 41.7) checks the closed form of the singular values against a numerical decomposition at $T = 256$, and they agree to 2.0×10^{-13}.

Example 41.2. Table 41.2 collects the error per unit of privacy for one participation. At $T = 100$ the naive mechanisms have root-mean-square errors 7.11 and 10, while Equation 41.9 says no factorization can go below 2.18. The other rows are derived in Section 41.3 and Section 41.4.

Table 41.2.: Error of each released prefix sum per unit of privacy ($\mu = 1$, $C = 1$), for one participation per record. The optimal factorization is normalized to columns of length 1 and was computed for $T \le 256$ only.

Mechanism	$T = 100$: $\Delta_2 R$	root-mean-square e_t	max e_t	$T = 10{,}000$: $\Delta_2 R$	root-mean-square e_t	max e_t
Independent noise ($R = I$)	1	7.11	10.00	1	70.71	100.00
Noise on prefix sums ($R = A$)	10	10.00	10.00	100	100.00	100.00
Binary tree	2.828	5.05	6.93	3.873	9.84	13.96
Square root	1.591	2.37	2.53	2.000	3.84	4.00
Optimal (Frobenius)	1	2.24	2.45	—	—	—

Mechanism	$T = 100$: $\Delta_2 R$	root-mean-square e_t	max e_t	$T = 10{,}000$: $\Delta_2 R$	root-mean-square e_t	max e_t
Lower bound Equation 41.9	—	2.18	—	—	3.63	—

Over a hundredfold longer run the independent-noise error grows tenfold, the tree's by a factor of 2 and the square root's by 1.6. At $T = 10{,}000$, independent noise is 18 times worse than the square root.

41.3. Tree aggregation and DP-FTRL

The first mechanism with error far below $\sqrt{T}$ was the **binary tree**, found independently by Dwork, Naor, Pitassi & Rothblum (2010) and by Chan, Shi & Song (2010) for counting events in a stream. Kairouz, McMahan, Song, Thakkar, Thakurta & Xu (2021) used it to release the prefix sums of gradients, with FTRL as the optimizer, and called the result **DP-FTRL**. This section asks how the tree works as a factorization and how good it is.

Definition 41.2 (Binary tree mechanism)**.** Build a binary tree over the steps $1, \dots, T$. Level 0 has one node per step, and a node at level l covers one of the blocks $1, \dots, 2^l$, then $2^l + 1, \dots, 2 \cdot 2^l$, and so on, of 2^l consecutive steps. There are $H = \lceil \log_2 T \rceil + 1$ levels. The mechanism releases each node's sum of gradients plus independent noise, and estimates each prefix sum as

$$\hat{S}_t = \sum_{\text{nodes in the decomposition of } t} (\text{node sum} + \text{noise}), \tag{41.11}$$

where the decomposition of t takes one node for each 1 in the binary expansion of t.

Proposition 41.3 (Error of the tree)**.** *The binary tree is a streaming factorization of A, with*

$$\Delta_2 R = \sqrt{H}, \qquad e_t = \sqrt{H}\sqrt{\operatorname{popcount}(t)}, \tag{41.12}$$

where $\operatorname{popcount}(t)$ *is the number of 1s in the binary expansion of t, at most $H - 1$ for $t \le T$. Both factors grow like $\sqrt{\ln T}$, so the error grows like $\ln T$.*

Proof. The plan is to check the decomposition, write the tree as an encoder and decoder, and count.

1. *The decomposition.* Write t in binary. Going from the highest bit down, each 1 at bit l takes the next block of 2^l steps. That block starts right after the sum of the higher bits, which is a multiple of 2^{l+1}, so it is a node of the tree. For example, $7 = 4 + 2 + 1$ uses the nodes covering steps 1 to 4, 5 and 6, and 7, while 8 uses a single node covering 1 to 8.
2. *The factorization.* R has one row per node, with 1 in the columns of the steps the node covers, so it is $m \times T$ with m about $2T$. Row t of L has a 1 for each node of the decomposition of t. Then $LR = A$, because by step 1 those nodes cover each step $1, \dots, t$ exactly once.

3. *Sensitivity.* A step lies in exactly one node per level, so column j of R has H ones and length $\sqrt{H}$.
4. *Error.* Row t of L has popcount(t) ones, so $\|\ell_t\| = \sqrt{\text{popcount}(t)}$, and Equation 41.7 gives Equation 41.12.
5. *Streaming.* Each node can be released once its last step has arrived, and $\hat{S}_t$ uses only completed nodes. With the rows of R ordered by completion time, the factors are streaming, so Theorem 41.1 applies to adaptive training.

□

The tree sums at most $\log_2 t + 1$ noise terms for the t-th prefix instead of t, each somewhat larger because every step is hidden in H nodes. That trade is a large win for long runs but not for short ones: the tree first wins at $T = 28$.

The tree is not the end. Its error jumps around, since $t = 127 = 1111111_2$ uses seven nodes and $t = 128$ uses one (Figure 41.2, left), and every step pays for being in top-level nodes that few prefixes use. Honaker (2015) combined all nodes with weights to reduce the variance, but that estimator uses nodes completed after step t, so it cannot be used during training. The square-root factorization of the next section has smaller error everywhere.

Example 41.3. For $T = 100$ the tree has $H = 8$ levels, so $\Delta_2 R = \sqrt{8} = 2.828$. The largest popcount up to 100 is 6, at $t = 63$, so the largest error per unit of privacy is $\sqrt{8}\sqrt{6} = 6.93$. The root-mean-square error is 5.05 (Table 41.2). For $T = 10{,}000$ there are 15 levels and $\Delta_2 R = 3.873$. The maximum error is then 13.96 and the root-mean-square error 9.84, against 70.71 for independent noise.

DP-FTRL runs FTRL, which by Equation 41.1 is gradient descent on noisy prefix sums, with the prefix sums released by the tree. Kairouz et al. (2021) also add momentum and restart the tree after a number of steps, and they report that DP-FTRL without amplification compares favourably with amplified DP-SGD. Xu et al. (2023) train production language models for the Gboard keyboard with it in federated learning, where devices cannot be Poisson-sampled.

41.4. The square-root factorization

The tree has a tall, thin encoder and a sparse decoder. A symmetric choice is $L = R$, a matrix whose square is A. This section asks whether A has a lower-triangular square root, what it looks like, and how close it comes to the best possible factorization.

Power series find the square root. A lower-triangular **Toeplitz** matrix is constant along each diagonal, so it is fixed by its first column $(c_0, \dots, c_{T-1})$, identified with the series $c(u) = \sum_k c_k u^k$ in a formal variable u. The proof of the next theorem needs one fact, which turns a matrix equation into an equation between series.

Lemma 41.1 (Lower-triangular Toeplitz matrices multiply like power series)**.** *The product of the lower-triangular Toeplitz matrices of two series $a(u)$ and $b(u)$ is the lower-triangular Toeplitz matrix of $a(u)b(u)$, truncated after u^{T-1}.*

Proof.

1. Entry (i,k) of the first matrix is a_{i-k} and entry (k,j) of the second is b_{k-j}, both zero above the diagonal. So entry (i,j) of the product is $\sum_k a_{i-k}b_{k-j}$ over $j \le k \le i$. This sum depends only on $i-j$, and it is the coefficient of u^{i-j} in $a(u)b(u)$.

□

Theorem 41.3 (The square-root factorization)**.** *The lower-triangular matrix $R = A^{1/2}$ with $R^2 = A$ is the Toeplitz matrix with entries*

$$R_{ij} = c_{i-j} \text{ for } i \ge j, \qquad c_k = \binom{2k}{k} 4^{-k} : \quad c = 1, \tfrac{1}{2}, \tfrac{3}{8}, \tfrac{5}{16}, \tfrac{35}{128}, \dots, \tag{41.13}$$

the coefficients of the power series $(1-u)^{-1/2} = \sum_k c_k u^k$, computed by $c_0 = 1$ and $c_k = c_{k-1}\frac{2k-1}{2k}$. With $L = R$,

$$(\Delta_2 R)^2 = \|\ell_T\|^2 = \sum_{k<T} c_k^2, \qquad 1 + \frac{\ln \frac{2T+1}{3}}{\pi} \le \sum_{k<T} c_k^2 \le 1 + \frac{0.8 + \ln \frac{4T-3}{5}}{\pi}, \tag{41.14}$$

and the largest error per unit of privacy is $\max_t e_t = e_T = \sum_{k<T} c_k^2$, which grows like $\ln(T)/\pi$. As noisy SGD, the noise added to the gradients is $\zeta = R^{-1}z$, where R^{-1} is the Toeplitz matrix of the coefficients h_k of $(1-u)^{1/2}$, $h = 1, -\frac{1}{2}, -\frac{1}{8}, -\frac{1}{16}, -\frac{5}{128}, \dots$.

Proof. The plan is to write A as a series, take its square root as a series, and then read the sensitivity and the error off the coefficients.

1. *A is the series* $1/(1-u)$. Its first column is all ones, and $1 + u + u^2 + \cdots = 1/(1-u)$.
2. *A square root.* The series $(1-u)^{-1/2}$ squares to $1/(1-u)$, so by Lemma 41.1 its Toeplitz matrix squares to A. Its coefficients come from the binomial series. The k-th one is

$$\frac{\frac{1}{2} \cdot \frac{3}{2} \cdots \frac{2k-1}{2}}{k!} = \frac{1 \cdot 3 \cdots (2k-1)}{2^k k!}. \tag{41.15}$$

   Multiplying top and bottom by $2 \cdot 4 \cdots 2k = 2^k k!$ gives $\frac{(2k)!}{4^k (k!)^2} = \binom{2k}{k} 4^{-k}$, and the ratio of consecutive terms is $\frac{2k-1}{2k}$.
3. *The inverse.* By Lemma 41.1 again, R^{-1} is the matrix of $1/(1-u)^{-1/2} = (1-u)^{1/2}$. The binomial series gives its coefficients $1, -\frac{1}{2}, -\frac{1}{8}, -\frac{1}{16}, -\frac{5}{128}, \dots$, all negative after the first.
4. *Sensitivity and error.* Column j of R holds $c_0, \dots, c_{T-j}$, so the first column is the longest, with squared length $\sum_{k<T} c_k^2$. Row t of $L = R$ holds $c_{t-1}, \dots, c_0$, so its squared length is $\sum_{k<t} c_k^2$, largest at $t = T$.
5. *Bounds.* A Wallis-type inequality gives $\frac{1}{\pi(k+1/2)} \le c_k^2 \le \frac{1}{\pi(k+1/4)}$ for $k \ge 1$. Section H.3 in the appendices proves it, and since both bounding functions decrease, each sum compares with an integral. For the lower bound,

$$\sum_{k=1}^{T-1} \frac{1}{k+1/2} \ge \int_1^T \frac{\mathrm{d}v}{v+1/2} = \ln \frac{2T+1}{3}. \tag{41.16}$$

For the upper bound, the first term is kept apart and the rest is bounded by an integral,

$$\sum_{k=1}^{T-1} \frac{1}{k+1/4} \le \frac{1}{5/4} + \int_1^{T-1} \frac{\mathrm{d}v}{v+1/4} = 0.8 + \ln \frac{4T-3}{5}. \tag{41.17}$$

Multiplying both sums by $1/\pi$ and adding the $k = 0$ term, 1, gives Equation 41.14.

□

The noise added to step t's gradient is $z_t - \frac{1}{2}z_{t-1} - \frac{1}{8}z_{t-2} - \ldots$, fresh noise minus a decaying share of the noise already injected. Each step's noise is a little larger than independent noise, by the factor $\left(\sum_k h_k^2\right)^{1/2}$ at the same noise scale. The script sums 100,000 coefficients and finds $\sum_k h_k^2 = 1.2732 = 4/\pi$, so the factor tends to 1.128. But the negative coefficients pull the accumulated noise back toward zero, so the running sums no longer drift (Figure 41.1).

The square root is close to optimal. Against the lower bound of 2.18 at $T = 100$ and 3.63 at $T = 10{,}000$, its root-mean-square error is 2.37 and 3.84, within 9% and 6%, and Section H.3 proves that the ratio tends to 1. Fichtenberger, Henzinger & Upadhyay (2023) and Henzinger, Upadhyay & Upadhyay (2023) bound its maximum and mean squared errors, and note that the factorization already appears in Bennett (1977). The closed form in Theorem 1 of the arXiv version of the first paper, $1 + \ln \frac{4T-3}{5}/\pi$, omits the first term of the upper comparison in step 5, so it gives 2.392 against the true 2.531 at $T = 100$. A finer expansion, stated here without proof, gives $\sum_{k<T} c_k^2 \approx (\ln T + \gamma + 4\ln 2)/\pi$ with γ Euler's constant, which is 2.532 at $T = 100$.

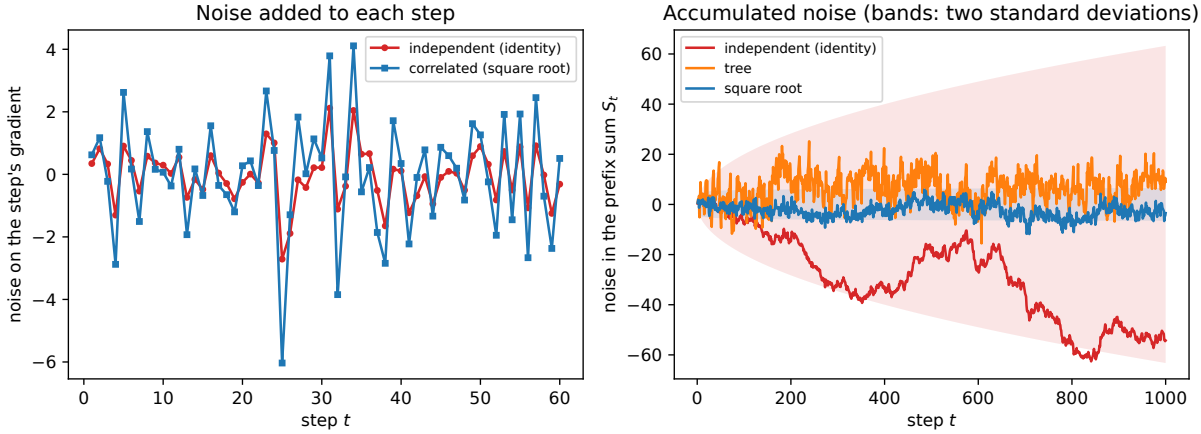


Figure 41.1.: Left: one coordinate of the noise added to each of the first 60 steps' gradients, independent (identity factorization) and square-root correlated, both calibrated to $\mu = 1$ for one participation. Right: the noise in the prefix sums over 1,000 steps, for independent noise, the binary tree and the square root, with bands of two standard deviations. Independent noise drifts like a random walk; the square root's stays within a narrow band.

Example 41.4. At $T = 4$ the coefficients are $c = (1, \frac{1}{2}, \frac{3}{8}, \frac{5}{16})$, so

$$R = \begin{pmatrix} 1 & & & \\ \frac{1}{2} & 1 & & \\ \frac{3}{8} & \frac{1}{2} & 1 & \\ \frac{5}{16} & \frac{3}{8} & \frac{1}{2} & 1 \end{pmatrix}. \tag{41.18}$$

Row 4 times column 1 of R is $\frac{5}{16} + \frac{3}{8} \cdot \frac{1}{2} + \frac{1}{2} \cdot \frac{3}{8} + \frac{5}{16} = 1$, as $R^2 = A$ requires, and the script confirms $R^2 = A$ exactly. The first column has squared length

$$1 + \frac{1}{4} + \frac{9}{64} + \frac{25}{256} = \frac{381}{256} = 1.488, \tag{41.19}$$

so $\Delta_2 R = 1.220$, and the last prefix sum has error 1.488 per unit of privacy, against $\sqrt{4} = 2$ for independent noise. At $T = 100$ the same computation gives $\sum_k c_k^2 = 2.531$, so $\Delta_2 R = 1.591$. The final prefix sum's error is then 2.53, a quarter of independent noise's 10.

41.4.1. The optimal factorization

How much better than the square root can any factorization do? By scale invariance the best encoder may have columns of length at most 1, and it then minimizes $\|L\|_F$. Write $X = R^\top R$ for the inner products of the columns of R. For a given X the best decoder, the least-squares solution of $LR = A$, gives $\|L\|_F^2 = \operatorname{tr}(A^\top A X^{-1})$, which is convex in X.

Theorem 41.4 (The optimal factorization)**.** *The problem of minimizing* $\operatorname{tr}(A^\top A X^{-1})$ *over positive definite* X *with diagonal at most 1 has the solution*

$$X = V^{-1/2}(V^{1/2} A^\top A V^{1/2})^{1/2} V^{-1/2}, \tag{41.20}$$

where V *is the diagonal matrix of a positive vector* v *that is a fixed point of* $v \mapsto \operatorname{diag}\left((V^{1/2} A^\top A V^{1/2})^{1/2}\right)$*. The optimal value is* $\sum_i v_i$*. In words, the optimal* X *is the matrix square root of* $A^\top A$*, rescaled on both sides by a diagonal matrix chosen so that every column of* R *has length exactly 1.*

The proof is Denisov et al. (2022, Theorem 3.2), who iterate the fixed-point map to find v, and Section H.4 in the appendices gives it in full. A lower-triangular R with $R^\top R = X$ comes from a Cholesky factorization of X with rows and columns reversed.

The script's iteration converges in 155 steps at $T = 100$, to a root-mean-square error of 2.24 against 2.37 for the square root, and in 193 steps at $T = 256$, to 2.52 against 2.67. Each iteration costs cubic time, so the square root, free and nearly as good, is the usual default for long runs. Figure 41.2 shows the errors across one run and against the horizon, and Figure 41.3 shows that the square root and the optimal factors look alike at $T = 16$.

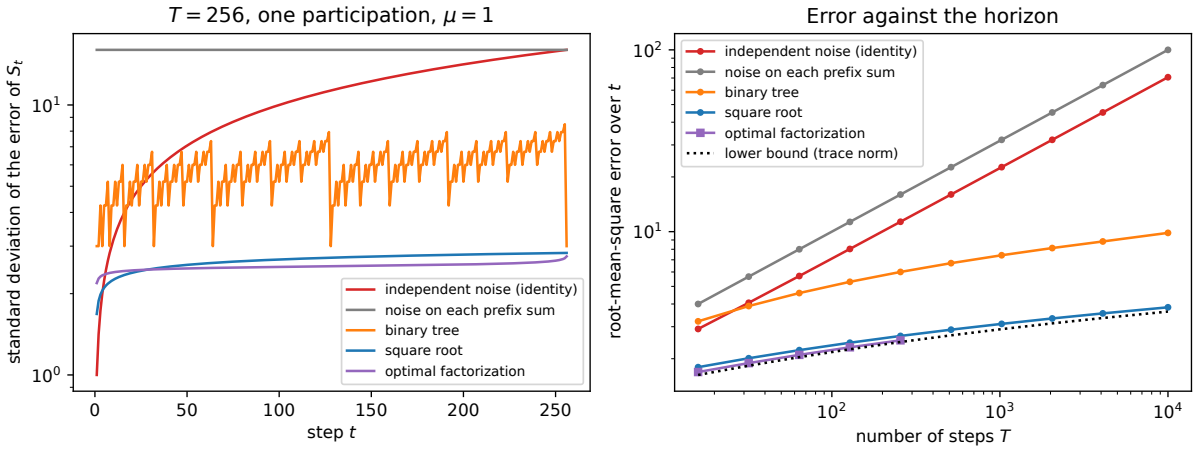


Figure 41.2.: Left: standard deviation of the error of each released prefix sum S_t for $T = 256$, one participation, at $\mu = 1$ and $C = 1$, for five factorizations. Right: the root-mean-square error over a run against its length T, with the trace-norm lower bound of Equation 41.9 dotted; the optimal factorization is computed up to $T = 256$.

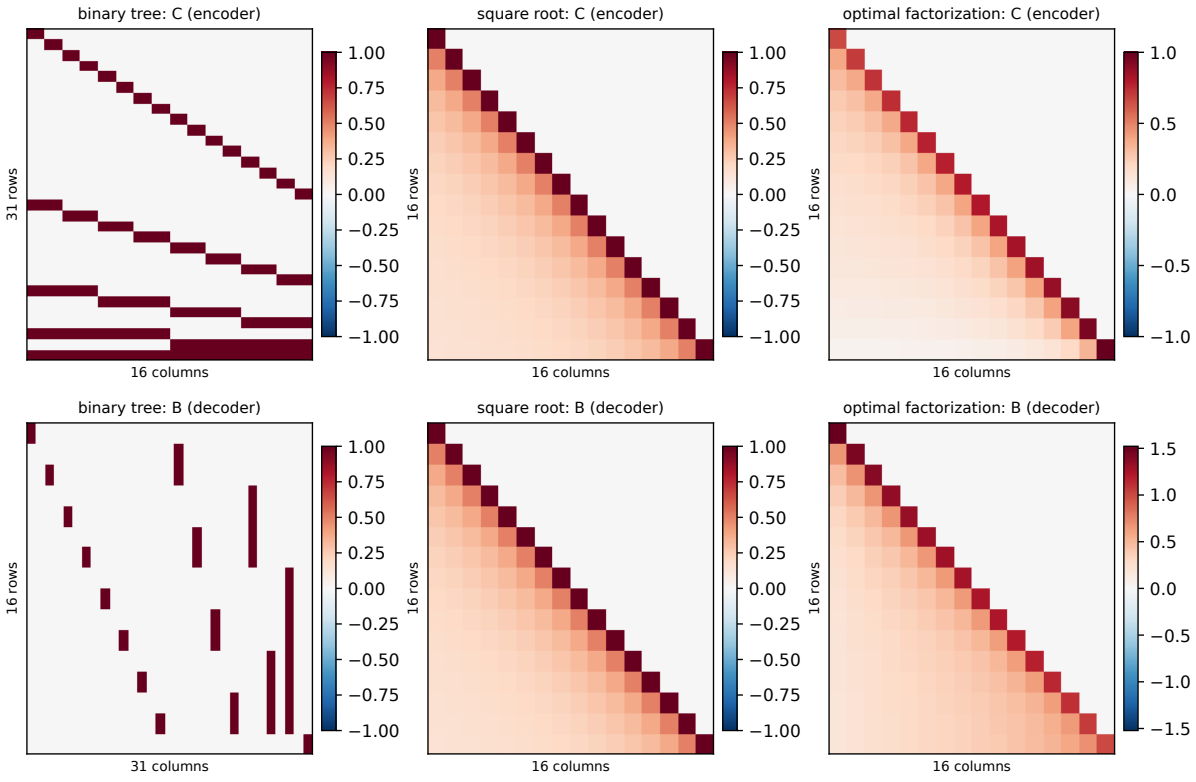


Figure 41.3.: The encoder R (top) and decoder L (bottom) of three factorizations of the 16×16 prefix-sum matrix. The tree's encoder has one row per node (31 rows), and its decoder picks the nodes of each prefix. The square root uses the same matrix for both; the optimal factorization's encoder has columns of length exactly 1.

41.5. Several participations

So far each record took part in one step. Training usually makes several passes, and then a record's gradient enters x at several steps, each time through a different column of R. This section asks how large the sensitivity is then, and how the factorization should change to keep it small. The answer depends on which steps a record can occupy and on how the columns of R overlap.

Definition 41.3 (Participation pattern, separation and band)**.** A record's **participation pattern** is the set of steps $\tau_1 < \cdots < \tau_k$ at which it contributes, with vectors $g_1, \dots, g_k$ of length at most C, possibly chosen adaptively. The pattern has b**-min-separation** if any two participations are at least b steps apart. A lower-triangular encoder is b**-banded** if its nonzero entries lie on the main diagonal and the $b-1$ diagonals below it, that is $R_{ij} = 0$ whenever $i - j \geq b$.

Theorem 41.5 (Sensitivity under several participations)**.** *For a record with participation pattern $\tau_1, \dots, \tau_k$, the change in Rx is $\sum_i r_{\tau_i} g_i^\top$. If $R^\top R \geq 0$ entrywise, the sensitivity in units of C is*

$$(\Delta_2 R)^2 = \frac{1}{C^2} \max_{\text{patterns}} \sup_{\|g_i\| \leq C} \Big\| \sum_i r_{\tau_i} g_i^\top \Big\|_F^2 = \max_{\text{patterns}} \Big\| \sum_i r_{\tau_i} \Big\|^2, \tag{41.21}$$

the largest length of a sum of participating columns, and the release is μ-GDP with $\mu = \Delta_2 R / s$. For independent noise, $R = I$, this gives $\Delta_2 R = \sqrt{k}$. For a b-banded encoder and patterns with b-min-separation, whatever the sign of the entries of $R^\top R$,

$$\Delta_2 R = \Big(\sum_i \|r_{\tau_i}\|^2 \Big)^{1/2} \leq \sqrt{k} \max_j \|r_j\|. \tag{41.22}$$

Proof. The plan is to expand the squared norm of the change into inner products, find the worst contributions, and evaluate the two special cases.

1. *The change in Rx.* Row τ_i of x changes by g_i, so Rx changes by $\sum_i r_{\tau_i} g_i^\top$, a sum of the column-times-row products of Theorem 41.1.
2. *Expand the norm.* The Frobenius inner product of two such products, $ab^\top$ and $a'b'^\top$, is $(a \cdot a')(b \cdot b')$. So
$$\Big\| \sum_i r_{\tau_i} g_i^\top \Big\|_F^2 = \sum_{i,j} (r_{\tau_i} \cdot r_{\tau_j})(g_i \cdot g_j). \tag{41.23}$$
3. *The worst contributions.* Each $g_i \cdot g_j$ lies in $[-C^2, C^2]$. The entries of $R^\top R$ are the inner products of columns, so if they are nonnegative, the sum is largest when every $g_i \cdot g_j = C^2$, that is when all g_i are the same vector of length C. The sum is then $C^2 \| \sum_i r_{\tau_i} \|^2$. The condition holds for the identity, the tree and the square root, because their R has nonnegative entries. Adaptive choices of the g_i cannot exceed this supremum, and Denisov et al. (2022, Theorem 2.1) state their adaptive result for exactly this sensitivity, and Section H.5 in the appendices proves it for Gaussian noise and a lower-triangular R.
4. *Independent noise.* The columns of I are orthonormal, so the squared length of a sum of k of them is k.
5. *Banded encoders.* Column τ of a b-banded R is nonzero only in rows $\tau, \dots, \tau + b - 1$. Participations at least b apart have columns with disjoint supports, so the cross terms of step 2 vanish whatever the g_i, leaving $\sum_i \|r_{\tau_i}\|^2 \|g_i\|^2$.

□

Correlated noise makes repeated participation more expensive. Under independent noise, k participations cost $\sqrt{k}$, as Gaussian composition says. Under the square root, the columns of nearby steps overlap and point in similar directions, so a record's contributions pile up and the sensitivity grows faster than $\sqrt{k}$. Under the tree it is worse still, because a top node holds several participations of the same record. For the optimal factorization Equation 41.21 applies only if X has no negative entries. Banding restores $\sqrt{k}$ by cutting the square root's encoder to its first b diagonals, which removes the overlaps. Its decoder $L = AR^{-1}$ is less accurate for one participation, but with many the saving in sensitivity dominates. Choquette-Choo and coauthors (2023) optimize factorizations under a participation constraint and add amplification to banded ones.

So the data order is part of the mechanism. A fixed permutation repeated every epoch guarantees a separation of exactly n/B steps, while reshuffling every epoch can put a record one step apart across an epoch boundary, which breaks the banded bound. The order must be fixed independently of the data, but it need not be random.

Example 41.5. Table 41.3 takes the running example's order, with separation $b = 100$, so over E epochs every record takes part E times. The banded encoder keeps the first 100 coefficients of the square root. Each cell gives the sensitivity and then e_T of Equation 41.7 with that sensitivity.

Table 41.3.: Sensitivity (in units of C) and error of the final prefix sum per unit of privacy, for E participations 100 steps apart, $T = 100E$.

Epochs E	Independent: $\Delta_2 R$ / final e_T	Tree	Square root	Banded square root
1	1.000 / 10.00	2.828 / 4.90	1.591 / 2.53	1.591 / 2.53
2	1.414 / 20.00	4.690 / 8.12	2.531 / 4.20	2.250 / 4.16
5	2.236 / 50.00	8.944 / 21.91	5.080 / 8.86	3.558 / 8.56
10	3.162 / 100.00	16.186 / 39.65	9.154 / 16.54	5.031 / 15.69
20	4.472 / 200.00	30.430 / 74.54	17.191 / 32.09	7.115 / 29.85

The banded sensitivity at 20 epochs is $\sqrt{20} \times 1.591 = 7.115$, as step 5 of the proof of Theorem 41.5 predicts. The full square root's is 17.191, more than twice as large, because its long columns overlap. Yet the final errors are close, 29.85 against 32.09, because the banded decoder is less accurate. Both are between a sixth and a seventh of independent noise's 200.

41.6. The running example: matrix mechanisms against DP-SGD

The previous sections compared mechanisms by the error in released sums. This section asks whether that ranking carries over to trained networks, and how mechanisms without sampling compare with DP-SGD with sampling at the same ε. The comparison needs one number per method computed before training, the noise its final model carries.

Proposition 41.4 (Noise at equal privacy)**.** *Fix a target (ε, δ) and E participations 100 steps apart, so $T = 100E$. A fixed-order mechanism with encoder R and the multi-epoch sensitivity $\Delta_2 R$ of Theorem 41.5 needs the noise scale*

$$s = \frac{\Delta_2 R}{\mu(\varepsilon, \delta)}, \tag{41.24}$$

and its final prefix sum carries noise of standard deviation $s\|\ell_T\|$ per coordinate, in units of C. For independent noise this is $s = \sqrt{E}/\mu(\varepsilon, \delta)$. DP-SGD with noise multiplier σ carries $\sigma\sqrt{T}$ in its final prefix sum.

Proof.

1. *Noise scale.* By Theorem 41.5 the release is μ-GDP with $\mu = \Delta_2 R/s$, and it meets the target exactly when $\mu \le \mu(\varepsilon, \delta)$. The smallest such s is the one stated, and for $R = I$ the sensitivity is $\sqrt{E}$.
2. *Final prefix noise.* By Theorem 41.1 the error of $\hat{S}_T$ has standard deviation $sC\|\ell_T\|$, which is $s\|\ell_T\|$ in units of C.
3. *DP-SGD.* Its final prefix sum is a sum of T independent noise vectors of standard deviation σ in units of C, so by Equation 41.3 it carries $\sigma\sqrt{T}$.

□

For DP-SGD, the Rényi differential privacy (RDP) accountant of DP-SGD finds the smallest σ for $q = 0.01$, T steps and the target ε. Training then runs Equation 41.1 with each factorization's noise added to the gradients. For the square-root and banded encoders that noise is $R^{-1}z$, a convolution in time, and for the tree it is the difference of consecutive tree estimates.

Example 41.6. Table 41.4 gives the noise each method needs for $\varepsilon = 2$ and 8 at $\delta = 10^{-5}$, and the noise its final prefix sum carries. Table 41.5 gives the test accuracy at the better of two learning rates, 0.5 and 2, averaged over three runs.

Table 41.4.: Noise needed by each method at equal ε ($\delta = 10^{-5}$, 100 steps per epoch) and the resulting noise in the final prefix sum, all in units of C. DP-SGD uses Poisson sampling with $q = 0.01$ and the RDP accountant; the others use a fixed order and exact Gaussian accounting.

ε, epochs	DP-SGD σ	Independent s	Tree s	Square root s	Banded s	Final prefix noise: DP-SGD / independent / tree / square root / banded
2, 1	0.827	1.994	5.639	3.172	3.172	8.27 / 19.94 / 9.77 / 5.05 / 5.05

ε, epochs	DP-SGD σ	Independent s	Tree s	Square root s	Banded s	Final prefix noise: DP-SGD / independent / tree / square root / banded
2, 5	0.928	4.458	17.833	10.129	7.093	20.75 / 99.69 / 43.68 / 17.67 / 17.06
2, 10	1.03	6.305	32.273	18.251	10.031	32.57 / 199.38 / 79.05 / 32.98 / 31.28
8, 1	0.501	0.600	1.698	0.955	0.955	5.01 / 6.00 / 2.94 / 1.52 / 1.52
8, 5	0.573	1.342	5.369	3.049	2.135	12.81 / 30.01 / 13.15 / 5.32 / 5.14
8, 10	0.616	1.898	9.716	5.495	3.020	19.48 / 60.02 / 23.80 / 9.93 / 9.42

Table 41.5.: Test accuracy (mean ± standard deviation over three runs) at equal ε, $\delta = 10^{-5}$, batch 100, clipping norm 0.5, best of learning rates 0.5 and 2. Without noise, fixed-order training with clipping reaches 0.9420, 0.9580 and 0.9606 after 1, 5 and 10 epochs.

ε, epochs	DP-SGD (Poisson)	Independent	Tree	Square root	Banded square root
2, 1	0.9343 ± 0.0044	0.9066 ± 0.0031	0.9134 ± 0.0038	0.9361 ± 0.0046	0.9358 ± 0.0019
2, 5	0.9460 ± 0.0021	0.8941 ± 0.0071	0.9183 ± 0.0084	0.9430 ± 0.0011	0.9456 ± 0.0019
2, 10	0.9493 ± 0.0012	0.8641 ± 0.0130	0.9096 ± 0.0074	0.9398 ± 0.0029	0.9491 ± 0.0024
8, 1	0.9371 ± 0.0016	0.9395 ± 0.0018	0.9412 ± 0.0006	0.9461 ± 0.0022	0.9434 ± 0.0017
8, 5	0.9524 ± 0.0026	0.9420 ± 0.0029	0.9461 ± 0.0012	0.9497 ± 0.0025	0.9542 ± 0.0014

ε, epochs	DP-SGD (Poisson)	Independent	Tree	Square root	Banded square root
8, 10	0.9520 ± 0.0006	0.9410 ± 0.0032	0.9484 ± 0.0004	0.9530 ± 0.0015	0.9549 ± 0.0024

At $\varepsilon = 2$, fixed-order SGD with independent noise loses 2.8 points to DP-SGD after one epoch and 8.5 points after ten. It gets worse with more epochs, because its noise scale grows like $\sqrt{E}$, from 1.994 to 6.305. The tree recovers between a quarter and a half of the gap, and the square-root encoders nearly all of it. At ten epochs the banded encoder reaches 0.9491, within 0.0002 of DP-SGD.

Matrix mechanisms can even beat amplified DP-SGD for short runs at large ε, where amplification gains little. At $\varepsilon = 8$ and one epoch, the square root's final prefix sum carries 1.52 against DP-SGD's 5.01, and it wins outright. DP-SGD keeps its lead at small ε over many epochs, where its noise multiplier grows only from 0.827 to 1.03 while every fixed-order mechanism pays for ten participations, and only banding keeps up. In every row, the method with the least final-prefix noise is the most accurate or within the run-to-run spread of it.

The comparison is slightly tilted toward the fixed-order methods. DP-SGD is accounted with the RDP accountant, which overstates ε and so gives it more noise than it needs (Choosing an accountant), while the others use the exact Gaussian curve. Choosing the learning rate by test accuracy is itself a leak (Hyperparameter tuning), used here only to compare methods at their best. The runs are short, but the gaps of Table 41.3 widen with more epochs. Figure 41.4 shows the same pattern across budgets at five epochs.

A federated server cannot sample devices with a known rate, but if it can ensure that a device contributes at most once every b rounds, the banded matrix mechanism gives a provable guarantee, and at the running example's scale it is as accurate as DP-SGD with sampling would be. The privacy unit there is a user, which User-level differential privacy treats.

41.7. Code

The script code/matrix_mechanisms.py builds the identity, prefix, tree, square-root and optimal factorizations and computes their sensitivities and errors. Its structured formulas agree with dense matrices at $T = 256$ to a relative difference of 3.3×10^{-16}. It also computes multi-epoch sensitivities and trains the running example with each mechanism's noise, against DP-SGD from code/dp_sgd.py. From the repository root:

```
uv run python code/part05-dp-ml/matrix_mechanisms.py
```

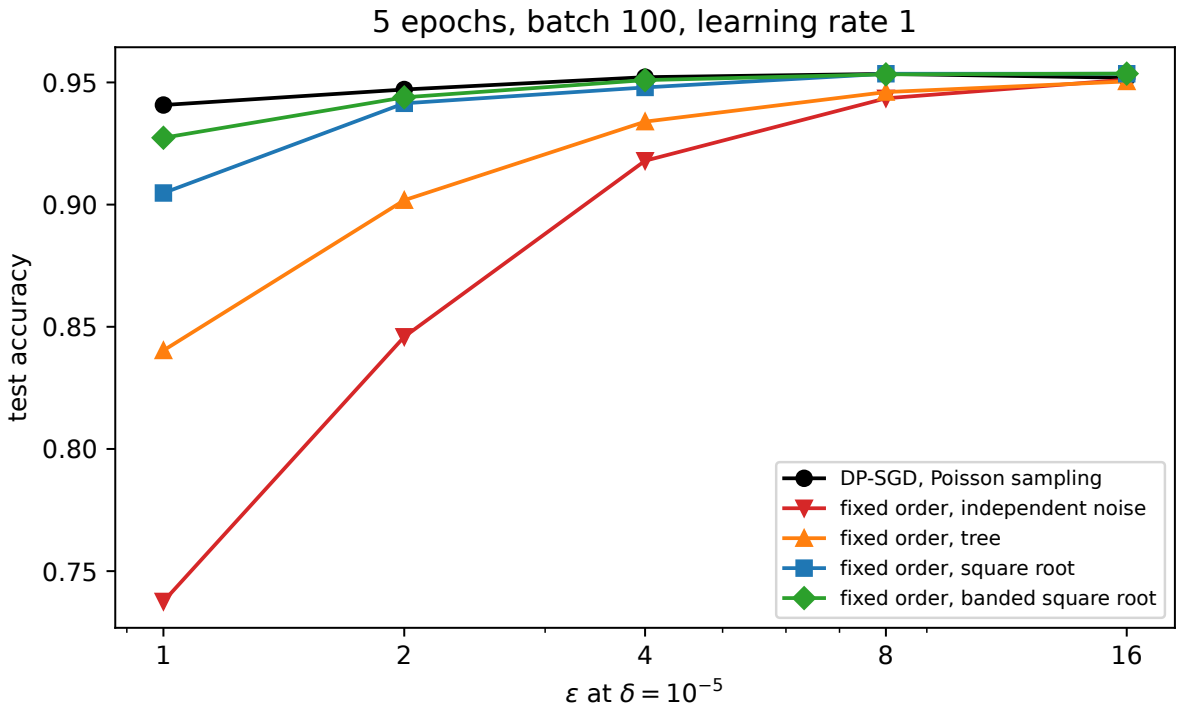


Figure 41.4.: Test accuracy of the running example after 5 epochs (500 steps of batch 100) at learning rate 1, against ε at $\delta = 10^{-5}$, one run per point. DP-SGD uses Poisson sampling; the other four use a fixed data order and differ only in how their noise is correlated.

Table 41.6.: The script's output with the default seed.

Experiment	Result
Square-root coefficients and inverse	$c = 1, 0.5, 0.375, 0.3125, ...$; $h = 1, -0.5, -0.125, -0.0625, ...$; $R^2 = A$ exactly at $T = 4$
One participation, $T = 100$	Root-mean-square error 7.11 (independent), 5.05 (tree), 2.37 (square root), 2.24 (optimal), bound 2.18
One participation, $T = 10{,}000$	70.71, 9.84, 3.84; bound 3.63
20 epochs, sensitivity	4.472 (independent), 30.430 (tree), 17.191 (square root), 7.115 (banded)
Accuracy, $\varepsilon = 2$, 10 epochs	DP-SGD 0.9493, independent 0.8641, tree 0.9096, square root 0.9398, banded 0.9491
Accuracy, $\varepsilon = 8$, 1 epoch	DP-SGD 0.9371, independent 0.9395, square root 0.9461

tests/test_matrix_mechanisms.py checks that the output still matches. The script takes about two minutes of processor time, most of it for the 198 training runs behind Table 41.5.

41.8. Summary

Table 41.7.: The results of this chapter, and where each is used.

Result	Statement	Use
Proposition 41.1	$\theta_t = \theta_0 - \frac{\eta}{B}\hat{S}_t$; the workload is Ax	Every model is post-processing of the prefix sums
Proposition 41.2	Noise per step: error $\sqrt{t}\,C/\mu$; noise per prefix sum: $\sqrt{T}\,C/\mu$	The first is DP-SGD without amplification
Definition 41.1	$\hat{S} = L(Rx + z) = Ax + Lz$	With R invertible, noisy SGD with noise $R^{-1}z$
Theorem 41.1	$\mu = \Delta_2 R/s$ with $\Delta_2 R = \max_j \|r_j\|$; error $\frac{C}{\mu}\Delta_2 R\|\ell_t\|$	Comparing correlated-noise schemes, also on adaptive streams
Theorem 41.2	$\Delta_2 R\,\|L\|_F/\sqrt{T} \geq \|A\|_*/T$	No factorization beats it; 2.18 at $T = 100$
Proposition 41.3	$\Delta_2 R = \sqrt{H}$, $e_t = \sqrt{H}\sqrt{\text{popcount}(t)}$	DP-FTRL; worth it only beyond a few dozen steps

Result	Statement	Use
Theorem 41.3	$c_k = \binom{2k}{k}4^{-k}$; $(\Delta_2 R)^2 = \max_t e_t = \sum_{k<T} c_k^2 \approx (\ln T + 3.35)/\pi$	The default for one pass: within about 10% of the best error, at no cost
Theorem 41.4	$X = R^\top R$ from a fixed point in v	Slightly better than the square root, at cubic cost per iteration
Theorem 41.5	$\Delta_2 R = \max \lVert \sum_i r_{\tau_i} \rVert$ if $R^\top R \geq 0$; banded: $(\sum_i \lVert r_{\tau_i} \rVert^2)^{1/2}$	Compute it for your actual participation pattern; for several passes, fix the order and use a band no wider than the separation
Proposition 41.4	$s = \Delta_2 R/\mu(\varepsilon, \delta)$; final noise $s\lVert \ell_T \rVert$, against $\sigma\sqrt{T}$	Compare methods by final-prefix noise. Without honest Poisson sampling, fix the order and use a matrix mechanism; keep DP-SGD for small ε over many epochs

41.9. References

- Dwork, Naor, Pitassi & Rothblum, "Differential Privacy Under Continual Observation" (STOC 2010), and Chan, Shi & Song, "Private and Continual Release of Statistics" (ACM Transactions on Information and System Security, 2011; conference version 2010). The binary tree for private running counts.
- Li, Hay, Rastogi, Miklau & McGregor, "Optimizing Linear Counting Queries Under Differential Privacy" (PODS 2010); journal version Li, Miklau, Hay, McGregor & Rastogi, "The Matrix Mechanism" (VLDB Journal 2015). Factoring a query workload to choose where noise goes.
- Kairouz, McMahan, Song, Thakkar, Thakurta & Xu, "Practical and Private (Deep) Learning without Sampling or Shuffling" (ICML 2021). DP-FTRL with tree aggregation; the comparison with amplified DP-SGD.
- Denisov, McMahan, Rush, Smith & Thakurta, "Improved Differential Privacy for SGD via Optimal Private Linear Operators on Adaptive Streams" (NeurIPS 2022). The matrix mechanism for adaptive streams (Theorem 2.1), lower-triangular factorizations, and the fixed point for the optimal factorization.
- Fichtenberger, Henzinger & Upadhyay, "Constant Matters: Fine-grained Error Bound on Differentially Private Continual Observation" (ICML 2023), and Henzinger, Upadhyay & Upadhyay, "Almost Tight Error Bounds on Differentially Private Continual Counting" (SODA 2023). The square-root factorization and how close it is to optimal, for the maximum and the mean squared error.
- Choquette-Choo, McMahan, Rush & Thakurta, "Multi-Epoch Matrix Factorization Mechanisms for Private Machine Learning" (ICML 2023). Sensitivity under several participations and factorizations optimized for it.

- Choquette-Choo, Ganesh, McKenna, McMahan, Rush, Thakurta & Xu, "(Amplified) Banded Matrix Factorizations: A Unified Approach to Private Training" (NeurIPS 2023). Banded encoders, min-separation, and amplification for matrix mechanisms.
- Honaker, "Efficient Use of Differentially Private Binary Trees" (2015). The variance-optimal, but not streaming, estimator from a tree.
- Xu, Zhang, Andrew, Choquette-Choo, Kairouz, McMahan, Rosenstock & Zhang, "Federated Learning of Gboard Language Models with Differential Privacy" (ACL 2023, industry track). DP-FTRL in a production federated system.

42. Private prediction: PATE and in-context learning

Training with differential privacy (DP) by stochastic gradient descent (SGD) with clipped and noisy gradients, DP-SGD, makes the weights private, so every later query to the model is free post-processing. The price is paid during training, and sometimes it cannot be paid. The model may be a large language model (LLM) that you can prompt but not train, or a model with no gradients. This chapter asks when it is better to make each *answer* private instead, and how.

The idea that organizes the chapter is one record, one vote. Split the private records into disjoint shards, train an ordinary model, a **teacher**, on each, and answer a query by a noisy vote of the teachers. A record lives in one shard, so it moves at most one vote, whatever the teachers are, as in the subsample-and-aggregate idea of Beyond global sensitivity. Privacy is therefore easy, and accuracy comes down to the gap between the top two vote counts against noise that grows like the square root of the number of answers. Private aggregation of teacher ensembles (PATE), by Papernot, Abadi, Erlingsson, Goodfellow and Talwar (2017), turns a few private answers into a private model.

The running example is the part's task (DP-SGD): 50-dimensional standard normal inputs labelled ± 1 by a fixed random network, with $n = 10{,}000$ private records. Its 20,000 test records are split into a **public pool**, whose inputs are treated as public and unlabelled, and an **evaluation set**, 10,000 each. The teachers are regularized logistic regressions (Private empirical risk minimization), each fitted on one shard of consecutive records. Neighbours $D \sim D'$ differ by adding or removing one record, $\delta = 10^{-5}$ throughout, and Table 42.1 lists the symbols.

Table 42.1.: The symbols of this chapter.

Symbol	Meaning
k; m	number of teachers; number of labels
$N_i(x)$; i^*; g	votes for label i on query x; plurality label; gap between the top two counts
b; σ	Laplace scale of LNMax; Gaussian standard deviation of GNMax
Q	number of answered queries
α; D_α	Rényi order; Rényi divergence
$\rho(\varepsilon, \delta)$	largest slope of a linear Rényi curve that converts to (ε, δ)
$\tilde{\beta}$; ε_0	bound on the chance of missing the plurality; pure-DP parameter of one step

42.1. The noisy vote: LNMax and GNMax

A query arrives, the teachers vote, and you must release one label. This section asks which mechanism releases it and how the cost of many answers grows with their number, since that growth is what limits private prediction.

Definition 42.1 (Teacher ensemble and noisy vote)**.** Split the private dataset D into k disjoint shards by a rule fixed before looking at the data, such as a hash of a record identifier, and train one **teacher** on each shard. On a query x with m possible labels, let $N_i(x)$ be the number of teachers that vote for label i. The label with the most votes is the **plurality** i^*, and the **gap** g is the top count minus the runner-up. **LNMax** (Laplace noisy max) releases

$$\arg\max_i \,(N_i(x) + Z_i), \qquad Z_i \sim \mathrm{Lap}(b), \tag{42.1}$$

and **GNMax** (Gaussian noisy max) releases the same label with $Z_i \sim \mathcal{N}(0, \sigma^2)$, the noise independent across labels. Only the label is released, never the noisy counts.

Every privacy statement below starts from one fact: one record can move the vote histogram $(N_1(x), \dots, N_m(x))$ only a little, whatever the teachers do. The cost theorem below needs this in the ℓ_1 norm, the sum of absolute changes, and the ℓ_2 norm, their Euclidean length.

Lemma 42.1 (One record changes one vote)**.** *With the shards fixed in advance, adding or removing one record changes at most one teacher. On every query, the vote histogram then moves by at most 2 in ℓ_1 norm and by at most $\sqrt{2}$ in ℓ_2 norm.*

Proof.

1. The record lands in the one shard its hash points to, and every other shard, and so every other teacher, is unchanged. Shards cut by position would not do, since one inserted record would shift every later record into the next shard. The script cuts by position only because, on its one fixed dataset, that is the same as a fixed assignment.
2. The changed teacher may vote differently, so one count goes down by 1 and another goes up by 1, or nothing changes. One vote is replaced, so this is the change of a histogram under replace in Proposition 20.2, with ℓ_1 norm $1 + 1 = 2$ and ℓ_2 norm $\sqrt{1^2 + 1^2} = \sqrt{2}$.

□

The cost of many answers is counted in Rényi differential privacy (RDP), whose curves add under composition. The improved conversion of Theorem 30.1 then turns a composed RDP curve $\varepsilon(\alpha)$ into (ε, δ)-DP, adding to the curve at each order the price of the conversion and keeping the best order:

$$\varepsilon = \min_{\alpha} \left[\varepsilon(\alpha) + \ln \frac{\alpha - 1}{\alpha} - \frac{\ln \delta + \ln \alpha}{\alpha - 1}\right]. \tag{42.2}$$

A curve that is a line $\rho\alpha$ enters only through its slope, so write $\rho(\varepsilon, \delta)$ for the largest slope that converts to a given budget. At $\delta = 10^{-5}$ it is $\rho = 0.0306$ for $\varepsilon = 1$ and $\rho = 0.3731$ for $\varepsilon = 4$.

Theorem 42.1 (The cost of noisy votes)**.** *Under add/remove neighbours, with the shards fixed in advance:*

1. *Each LNMax answer is $(2/b)$-DP (Papernot et al. 2017).*
2. *Each GNMax answer is $(\alpha, \alpha/\sigma^2)$-RDP for every $\alpha > 1$ (Papernot, Song, Mironov, Raghunathan, Talwar & Erlingsson 2018, Proposition 8).*
3. *Q GNMax answers, even on adaptively chosen queries, are $(\alpha, Q\alpha/\sigma^2)$-RDP. A budget (ε, δ) therefore needs the noise*

$$\sigma = \sqrt{\frac{Q}{\rho(\varepsilon, \delta)}}. \tag{42.3}$$

Proof. The plan is to read LNMax as report-noisy-max and GNMax as a Gaussian mechanism followed by post-processing, then to compose and solve for the noise.

1. *LNMax.* LNMax is report-noisy-max (Theorem 22.4) with the counts as scores, each moving by at most 1. By Lemma 42.1 one count may go up while another goes down, so the scores are not monotone, and the theorem's scale $b = 2/\varepsilon$ applies. Solving for ε gives $2/b$.
2. *GNMax.* Adding $\mathcal{N}(0, \sigma^2)$ to every count is the Gaussian mechanism on a histogram of ℓ_2 sensitivity $\sqrt{2}$, so its noise multiplier is $\sigma/\sqrt{2}$. The Gaussian curve of Theorem 29.1, α over twice the squared multiplier, gives

$$\varepsilon(\alpha) = \frac{\alpha}{2(\sigma/\sqrt{2})^2} = \frac{\alpha}{\sigma^2}. \tag{42.4}$$

   The argmax reads only this output, so by post-processing (Theorem 29.3) it cannot raise the curve.
3. *Compose.* Each answer is $(\alpha, \alpha/\sigma^2)$-RDP whatever the earlier answers were, so by adaptive composition (Theorem 29.2) the curves of Q answers add to $Q\alpha/\sigma^2$.
4. *Calibrate.* That curve is the line with slope Q/σ^2. Setting the slope to $\rho(\varepsilon, \delta)$ and solving for σ gives Equation 42.3.

□

What the teachers are never enters the cost, so a teacher can even be a human annotator. Only the label may be released, though, because the winner's noisy count would publish the noise that the report-noisy-max proof keeps hidden.

An LNMax answer enters the RDP accountant as an ε_0-DP step with $\varepsilon_0 = 2/b$, charged the curve of randomized response, which Theorem 31.3 shows bounds every such step. That charges every answer for its worst case, while Gaussian curves add without slack, so Gaussian noise wins for many answers. The RDP accountant is itself a little loose for GNMax, whose Q answers are exactly one Gaussian shift (f-DP and Gaussian DP). At the σ the accountant picks for $\varepsilon = 1$, the exact ε is 0.915, and for $\varepsilon = 4$ it is 3.698, so the accountant overstates ε by about 9% and 8%.

Example 42.1. Table 42.2 gives the noise for four numbers of answers at $\varepsilon = 1$ and $\varepsilon = 4$, with the Laplace scale also shown as a standard deviation, $\sqrt{2}b$.

Table 42.2.: Noise per count for Q answers at $(\varepsilon, 10^{-5})$, from the RDP accountant, with LNMax charged the randomized-response curve.

Budget	Answers Q	GNMax σ	LNMax b	LNMax standard deviation $\sqrt{2}b$
$\varepsilon = 1$	1	5.72	2.02	2.85
	100	57.2	79.7	113
	1,000	181	255	361
	10,000	572	809	1,140
$\varepsilon = 4$	1	1.64	0.501	0.709
	100	16.4	22.7	32.1
	1,000	51.8	73.1	103
	10,000	164	231	327

For a single answer Laplace needs less noise, and from 100 answers on σ is smaller than $\sqrt{2}b$ by a factor close to 2. From 100 to 10,000 answers σ grows tenfold, from 57.2 to 572, as Equation 42.3 says. Noise of 572 on each count swamps the vote of any ensemble of fewer than a few thousand teachers.

42.2. How accurate is a noisy vote

The cost of an answer does not depend on the teachers, but its accuracy depends almost only on them. This section asks how likely the noisy vote is to return the plurality label and the right label, which you need to choose the number of teachers. Whether GNMax picks one label over another depends only on the difference of their two noises, and that difference is $\mathcal{N}(0, 2\sigma^2)$, because the variances of independent variables add.

Proposition 42.1 (Accuracy of a noisy vote)**.** *Let Φ be the standard normal distribution function. On two labels with gap g, GNMax returns the plurality label with probability*

$$\Pr[\textit{plurality}] = \Phi\Big(\frac{g}{\sqrt{2}\,\sigma}\Big). \tag{42.5}$$

On m labels, it misses the plurality with probability at most

$$\tilde{\beta} = \sum_{i \neq i^*} \frac{1}{2} \operatorname{erfc}\Big(\frac{N_{i^*} - N_i}{2\sigma}\Big), \tag{42.6}$$

where erfc *is the complementary error function (Papernot et al. 2018, Proposition 7). If a fraction $a(x)$ of the teachers vote for the correct label on a two-label query x, the answer is correct with probability $\Phi(k(2a(x) - 1)/(\sqrt{2}\sigma))$, and the expected accuracy is the average of this over queries.*

Proof.

1. *Two labels.* GNMax returns label 1 when $N_1 + Z_1 > N_2 + Z_2$, that is, when the noise difference $Z_2 - Z_1$ is below $N_1 - N_2$. That difference is $\mathcal{N}(0, 2\sigma^2)$, so the event has probability $\Phi((N_1 - N_2)/(\sqrt{2}\sigma))$, and with label 1 the plurality, $N_1 - N_2 = g$.

2. *Many labels.* GNMax misses the plurality only if some other label beats it, and the chance of a union of events is at most the sum of their chances. By step 1, label i beats the plurality with probability $\Phi(-(N_{i^*} - N_i)/(\sqrt{2}\sigma))$, which equals $\frac{1}{2}\operatorname{erfc}((N_{i^*} - N_i)/(2\sigma))$ because $\Phi(-u) = \frac{1}{2}\operatorname{erfc}(u/\sqrt{2})$.
3. *Toward the truth.* The gap toward the correct label is the number of teachers that are right minus the number that are wrong, $ka - k(1-a) = k(2a-1)$. Step 1 with this signed gap gives the chance of a correct answer, and averaging over queries gives the expected accuracy.

□

For a fixed number of answers and budget, σ is fixed, while the signal $k(2a-1)$ grows with k. But each teacher then sees n/k records and is right less often, so $2a-1$ shrinks, and which effect wins depends on how fast a teacher's accuracy falls with its shard size.

The noiseless plurality vote is an ordinary ensemble, and its accuracy is the ceiling of every private answer, since noise only pulls answers toward a coin flip. Equation 42.5 is exact, because two labels have one competitor, while Equation 42.6 is loose when many labels are nearly tied with the plurality.

Example 42.2. Take $k = 100$ teachers and the noise for 1,000 answers at $\varepsilon = 4$, $\sigma = 51.8$ (Figure 42.1). A query split 98 to 2 gets the plurality label with probability 0.9051, one split 70 to 30 with probability 0.7076, and one split 52 to 48 with probability 0.5218.

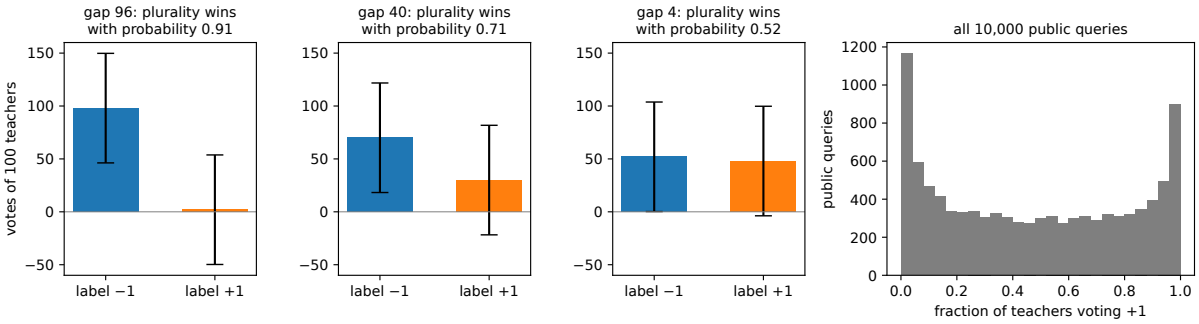


Figure 42.1.: Left: the votes of 100 teachers on three public queries, with bars of one noise standard deviation ($\sigma = 51.8$) and the probability of returning the plurality. Right: the fraction of teachers voting +1 across the 10,000 public queries.

Table 42.3 splits the 10,000 private records among 25 to 500 teachers.

Table 42.3.: Teachers on the running example, on the evaluation set: mean teacher accuracy, noiseless plurality accuracy, and the median gap as a fraction of k.

Teachers k	Records each	Mean teacher accuracy	Plurality vote accuracy	Median $\lvert g\rvert/k$
25	400	0.8954	0.9403	1.000
50	200	0.8518	0.9389	0.880
100	100	0.7896	0.9384	0.660
250	40	0.7031	0.9336	0.408

Teachers k	Records each	Mean teacher accuracy	Plurality vote accuracy	Median $\lvert g\rvert/k$
500	20	0.6459	0.9339	0.276

The teachers get much worse as their shards shrink, from 0.8954 to 0.6459, but the plurality vote stays between 0.9336 and 0.9403, because their errors are close to independent. The absolute median gap grows, from 25 to $0.276 \times 500 = 138$, so here larger ensembles give more accurate noisy answers (Table 42.4).

Table 42.4.: Expected accuracy of Q GNMax answers on the evaluation set, from Equation 42.5.

Budget	Answers Q	σ	$k = 25$	$k = 50$	$k = 100$	$k = 250$	$k = 500$
$\varepsilon = 1$	100	57.2	0.5961	0.6651	0.7461	0.8250	0.8588
	1,000	180.9	0.5308	0.5546	0.5889	0.6489	0.6987
	10,000	572.1	0.5097	0.5173	0.5285	0.5498	0.5709
$\varepsilon = 4$	100	16.4	0.7899	0.8803	0.9064	0.9185	0.9237
	1,000	51.8	0.6059	0.6805	0.7641	0.8378	0.8682
	10,000	163.7	0.5340	0.5602	0.5979	0.6625	0.7140

With 100 answers at $\varepsilon = 4$, 500 teachers reach 0.9237, but with 1,000 answers at $\varepsilon = 1$ only 0.6987.

42.3. Private answers against a private model

You have a budget (ε, δ). You can spend it on training one model with DP-SGD, whose answers are then free, or on answering queries one at a time with a teacher ensemble. This section asks which gives more accurate answers, and how the choice depends on the number of queries you expect. Write $A_{\text{model}}(\varepsilon)$ for the accuracy of a privately trained model and $A_{\text{vote}}(Q, \varepsilon)$ for the expected accuracy of Q private answers at the same budget.

Proposition 42.2 (The break-even number of queries)**.** *Suppose that $A_{\text{vote}}(Q, \varepsilon)$ decreases in Q, as it does when the plurality is correct on most queries. Then private answers are more accurate than a private model exactly when*

$$Q < Q^*(\varepsilon) = \max\{Q : A_{\text{vote}}(Q, \varepsilon) \geq A_{\text{model}}(\varepsilon)\}, \tag{42.7}$$

with $Q^* = 0$ when even one answer is worse than the model.

Proof.

1. *The model is constant in Q.* Every query to a released model is post-processing of its weights (Theorem 19.1), so it costs nothing and is as accurate as the model.

2. *The vote falls toward* $\frac{1}{2}$. By Equation 42.3, σ grows like $\sqrt{Q}$, and by Proposition 42.1 each answer is correct with probability $\Phi(g/(\sqrt{2}\sigma))$ for its signed gap g. That chance falls toward $\frac{1}{2}$ as σ grows when the gap is positive, which is why the hypothesis holds when most gaps are positive.
3. *Break-even.* A decreasing function crosses a constant at most once, and the last Q before the crossing is Q^*.

□

DP-SGD pays a fixed price, which at small ε is a much worse model, while the vote's first answers are almost as good as the noiseless ensemble. So private prediction wins when the budget is small and the queries are few. At large ε the ensemble's ceiling decides, and a million queries are out of reach either way, since they need a σ a hundred times that of 100 answers.

The comparison assumes that you *can* run DP-SGD. With LLMs behind an interface, or parties who will not pool their data, private prediction is the only private option.

Example 42.3. The DP-SGD baseline is the configuration of DP-SGD, retrained twice per budget. Table 42.5 gives its accuracy and the break-even number of queries for the best number of teachers.

Table 42.5.: DP-SGD on the running example against GNMax answers, with the break-even number of queries.

ε	DP-SGD accuracy (2 runs)	Break-even Q^*	Best k
0.5	0.8164 ± 0.0060	58	500
1	0.9128 ± 0.0207	18	500
2	0.9438 ± 0.0039	0	—
4	0.9573 ± 0.0024	0	—
8	0.9605 ± 0.0004	0	—

At $\varepsilon = 0.5$, 500 teachers answer up to 58 queries more accurately than the private model, and at $\varepsilon = 1$ up to 18. From $\varepsilon = 2$ on, DP-SGD beats even the noiseless vote, because the ensemble's ceiling, 0.9336 to 0.9403, is below its 0.9438 (Figure 42.2).

42.4. PATE: from private answers to a private model

Private answers cost budget forever, while a private model costs it once. PATE spends the budget on labelling a fixed number of public, unlabelled inputs with private answers, then trains a new model on them and releases only that model. This section asks how private the released model is, and how good it can be.

Definition 42.2 (PATE)**.** Train teachers as in Definition 42.1 and label Q public inputs $x_1, \dots, x_Q$ by GNMax with noise σ. Then train a **student** on these inputs and labels, by any algorithm, possibly with other public data. The student is released, and the teachers never are.

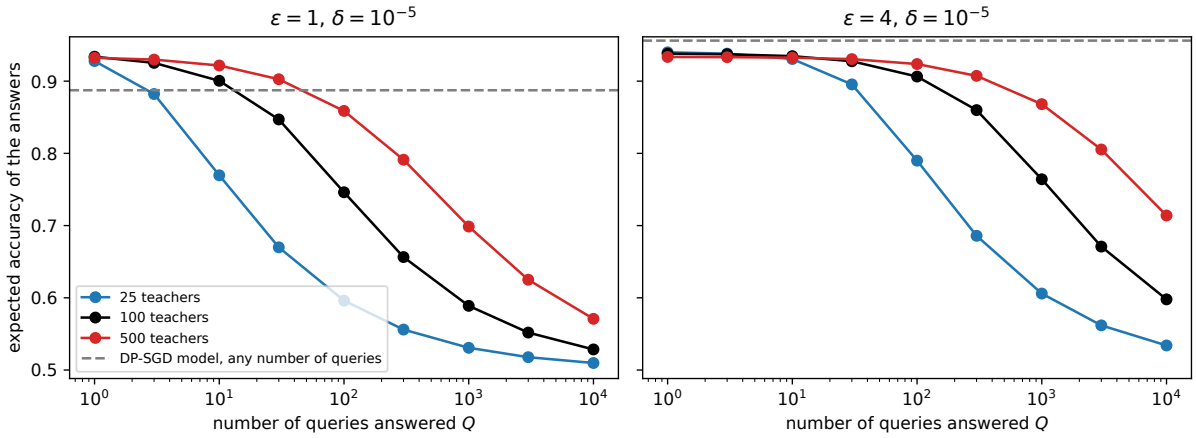


Figure 42.2.: Expected accuracy of Q GNMax answers for 25, 100 and 500 teachers, at $\varepsilon = 1$ (left) and $\varepsilon = 4$ (right). Dashed: DP-SGD at the same budget.

The student never sees a private record, only public inputs and private labels, so its privacy should be that of the labels.

Theorem 42.2 (Privacy of the PATE student)**.** *The student of Definition 42.2, and every prediction it ever makes, is $(\alpha, Q\alpha/\sigma^2)$-RDP, and so (ε, δ)-DP with ε from Equation 42.2, even when each input to be labelled is chosen after seeing the earlier labels. The cost depends on nothing about the student.*

Proof.

1. *The only private output is the labels.* The student's training sees the public inputs, which do not depend on the private data, and the Q labels, never the teachers or the records.
2. *The labels are Q GNMax answers.* By Theorem 42.1 they cost $Q\alpha/\sigma^2$, and adaptive composition lets the student choose which inputs to ask about.
3. *Post-processing.* The student's weights are a function of the labels and of public data, so by Theorem 29.3 they are as private as the labels, and so is every prediction computed from them.

□

Three things limit the student. It is trained on Q examples rather than n, its labels are noisy, and the plurality vote has the ensemble's ceiling. More labels mean more noise per label by Equation 42.3, so each ε has a best Q. The semi-supervised students of Papernot et al. (2017, 2018) also learn from unlabelled public inputs, which this chapter's student does not.

PATE needs public, unlabelled inputs from the same distribution as the private data. The teachers are non-private models of their shards and must stay secret (Part 2 attacks them).

Example 42.4. Take $k = 250$ teachers, whose plurality vote labels the public pool with accuracy 0.9322. The student is the part's network, trained non-privately for 2,000 steps on the first Q public inputs and their GNMax labels (Table 42.6; the first two rows have no privacy).

Table 42.6.: Test accuracy of a PATE student with 250 teachers and Q labels, one run each; in parentheses, the fraction of correct GNMax labels.

Labels	$Q = 250$	$Q = 500$	$Q = 1{,}000$	$Q = 2{,}000$
True labels (not private)	0.8726	0.8990	0.9188	0.9410
Noiseless plurality (not private)	0.8913	0.9106	0.9229	0.9297
GNMax, $\varepsilon = 1$	0.8238 (0.7760)	0.7595 (0.6880)	0.6995 (0.6350)	0.6749 (0.6260)
GNMax, $\varepsilon = 2$	0.7980 (0.8040)	0.8393 (0.8140)	0.8141 (0.7490)	0.7318 (0.6885)
GNMax, $\varepsilon = 4$	0.8633 (0.8720)	0.8754 (0.8740)	0.8898 (0.8360)	0.8259 (0.7840)
GNMax, $\varepsilon = 8$	0.8950 (0.9160)	0.9037 (0.9040)	0.9049 (0.8870)	0.8884 (0.8525)

The best student reaches 0.8238 at $\varepsilon = 1$ and 0.9049 at $\varepsilon = 8$, while DP-SGD, at 0.9128 to 0.9605, is better at every budget (Figure 42.3). With true labels the student needs 2,000 examples to reach 0.9410, but at $\varepsilon = 4$, 2,000 labels need $\sigma = 73.2$ by Equation 42.3, and only 0.7840 of them are right. Choosing the best Q by test accuracy is itself a leak, which Hyperparameter tuning shows how to pay for.

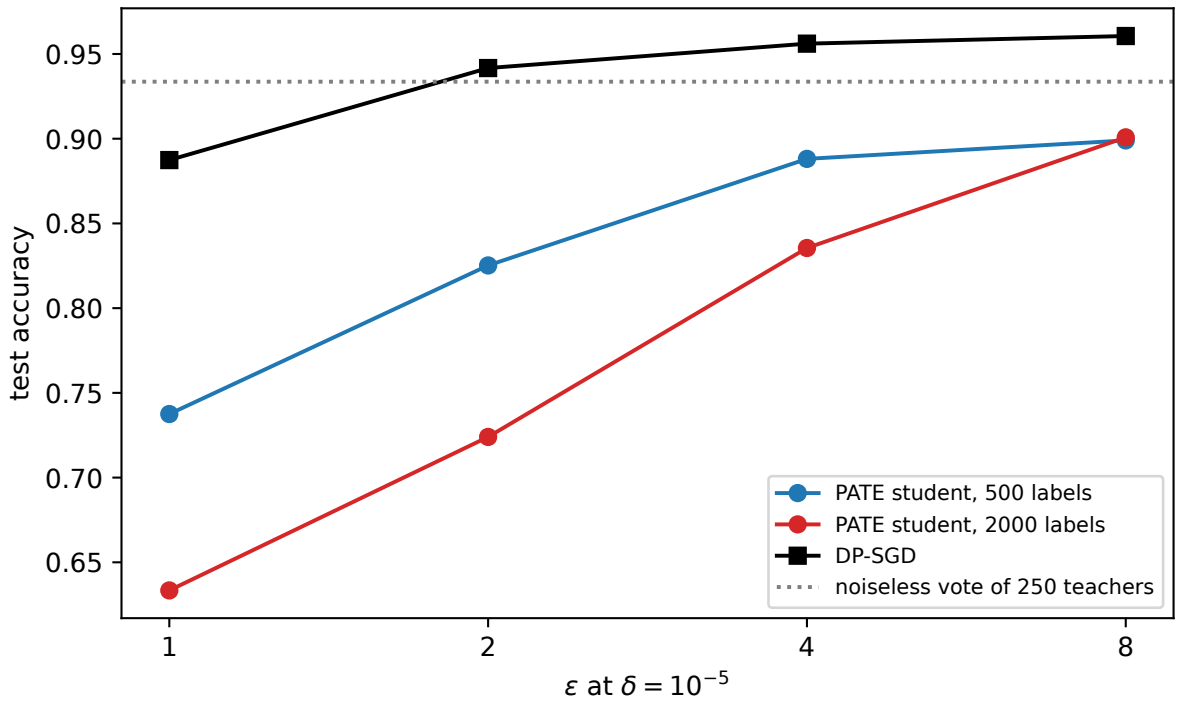


Figure 42.3.: Test accuracy of the PATE student (250 teachers, 500 or 2,000 labels) and of DP-SGD against ε. Dotted: the noiseless plurality vote.

PATE loses here because the task is binary, so a wrong label is the other label, and because the teachers are weak (0.7031 each with 40 records), so many gaps are small against the noise. Its

published successes are on tasks where the teachers agree on most inputs. On a handwritten-digit benchmark (MNIST), Papernot et al. (2018, Table 1) report a student with 98.5% accuracy against a non-private 99.2%, at $\varepsilon = 1.97$ with 286 answered queries.

42.5. Data-dependent analysis and Confident-GNMax

The cost α/σ^2 of a GNMax answer is a worst case over all vote histograms, including ties that one teacher decides. When 249 of 250 teachers agree, one record can hardly change the answer. This section asks whether such an answer can be charged less, and whether a mechanism can answer only such queries.

The charge is a Rényi divergence, which RDP bounds for every pair of neighbours (Definition 29.1). Between two distributions P and P' on a discrete set of outputs, the **Rényi divergence** of order α is

$$D_\alpha(P\|P') = \frac{1}{\alpha-1}\ln\sum_y P(y)^\alpha P'(y)^{1-\alpha}. \tag{42.8}$$

If the mechanism almost always returns one output on the actual dataset, that output dominates the sum, and one record can hardly change its probability. The bound below makes this exact for a pure-DP step.

Theorem 42.3 (The data-dependent bound, pure-DP form)**.** *Let M be ε_0-DP, and suppose that on the actual dataset D some output i^* has $\Pr[M(D)\neq i^*] = \beta < e^{-\varepsilon_0}$. Then for every neighbour D' (Papernot et al. 2017, Theorem 3, written in RDP),*

$$D_\alpha(M(D)\|M(D')) \le \frac{1}{\alpha-1}\ln\Big[(1-\beta)^\alpha(1-e^{\varepsilon_0}\beta)^{1-\alpha} + \beta\, e^{(\alpha-1)\varepsilon_0}\Big]. \tag{42.9}$$

The bound also holds with any upper bound $\tilde\beta \ge \beta$ below $e^{-\varepsilon_0}$ in place of β. For LNMax, $\varepsilon_0 = 2/b$.

Proof. The plan is to split the sum of Equation 42.8 into the likely output and the rest, bound each part with pure DP, and then pass from the true failure probability to its bound.

1. *Split the sum.* Write $P = M(D)$ and $P' = M(D')$. By Equation 42.8, $e^{(\alpha-1)D_\alpha}$ is the sum of $P(y)^\alpha P'(y)^{1-\alpha}$ over outputs. Split it into the term of i^* and the rest, and write β' for $P'(\text{not } i^*)$.
2. *The unlikely outputs.* For $y \neq i^*$ the term is $P(y)\big(P(y)/P'(y)\big)^{\alpha-1}$. Pure DP bounds the ratio by e^{ε_0}, so these terms add up to at most $\beta\, e^{(\alpha-1)\varepsilon_0}$.
3. *The likely output.* Its term is $(1-\beta)^\alpha(1-\beta')^{1-\alpha}$. Pure DP applied to the event "not i^*" gives $\beta' \le e^{\varepsilon_0}\beta$, so $1-\beta' \ge 1-e^{\varepsilon_0}\beta > 0$. The exponent $1-\alpha$ is negative, so a larger base gives a smaller power, and the term is at most $(1-\beta)^\alpha(1-e^{\varepsilon_0}\beta)^{1-\alpha}$.
4. *Collect, and pass to $\tilde\beta$.* Adding the two parts and taking $\frac{1}{\alpha-1}\ln$ gives Equation 42.9 with the true β. The right side increases in β for every $\beta < e^{-\varepsilon_0}$, because it is convex in β with a non-negative slope at 0 (Appendix Section M.2 proves both). So the largest value over all $\beta \le \tilde\beta$ is at $\tilde\beta$.

□

For small β the bound is about β times a constant, so an answer costs in proportion to the chance that the noise overturns it. GNMax has a Gaussian form of the same bound. Because GNMax is not pure DP, the proof bounds the unlikely outputs with Rényi divergences at two higher orders α_1 and α_2 instead of a ratio, but the bracket keeps the two parts of Equation 42.9: the likely output, weighted by $1-\tilde{\beta}$, and the rest, weighted by $\tilde{\beta}$.

Theorem 42.4 (The data-dependent bound, Gaussian form)**.** *Let M be GNMax with noise σ, and $\tilde{\beta}$ the bound of Equation 42.6 on the actual dataset D. Set $\alpha_2 = \sigma\sqrt{\ln(1/\tilde{\beta})}$, $\alpha_1 = \alpha_2 + 1$, $\varepsilon_1 = \alpha_1/\sigma^2$ and $\varepsilon_2 = \alpha_2/\sigma^2$. If $\alpha \le \alpha_1$, $\alpha_2 > 1$ and $\tilde{\beta} \le e^{(\alpha_2-1)\varepsilon_2}\big/\big(\frac{\alpha_1}{\alpha_1-1}\cdot\frac{\alpha_2}{\alpha_2-1}\big)^{\alpha_2}$, then for every neighbour D' (Papernot et al. 2018, Theorem 6, with the orders of their Proposition 10)*

$$D_\alpha(M(D)\|M(D')) \le \frac{1}{\alpha-1}\ln\Big[(1-\tilde{\beta})A^{\alpha-1} + \tilde{\beta}B^{\alpha-1}\Big], \quad A = \frac{1-\tilde{\beta}}{1-(\tilde{\beta}e^{\varepsilon_2})^{(\alpha_2-1)/\alpha_2}}, \quad B = \frac{e^{\varepsilon_1}}{\tilde{\beta}^{1/(\alpha_1-1)}}. \tag{42.10}$$

Each answer is charged the smaller of this and α/σ^2, and the charges add up over answers.

The Gaussian form follows the plan of Theorem 42.3, and its proof is in Papernot et al. (2018, Appendix A). Appendix Section M.2 derives it step by step.

Figure 42.4 draws the cost of one GNMax answer with $\sigma = 40$ at order 8 against the gap. Up to gaps of 100 the bound is no better than the data-independent $\alpha/\sigma^2 = 0.005$, and then it falls steeply, to 1.23×10^{-3} at a gap of 150 and 8.37×10^{-5} at 200. The exact cost on two labels is five times lower at a gap of 150 (2.42×10^{-4}), even though the exact cost takes the worse of the two directions while the bound covers only $D_\alpha(M(D)\|M(D'))$. So Theorem 42.4 is valid but not tight.

The data-dependent ε is itself private information, since one record changes one teacher, whose votes can change on *every* query. Papernot et al. (2018, Appendix B) therefore release it with noise scaled to its **smooth sensitivity** (Beyond global sensitivity), and pay for that release. On MNIST their data-dependent $\varepsilon = 2.00$ becomes 2.52 ± 0.385 once sanitized (their Table 2). Unsanitized, a data-dependent ε is a measurement, not a guarantee.

A query with strong consensus is cheap under this analysis and more likely to be labelled right, so a noisy check of the top count can pick out the queries worth answering.

Definition 42.3 (Confident-GNMax)**.** For each query, release $\perp$ (no answer) unless $\max_i N_i + \mathcal{N}(0,\sigma_1^2) \ge \tau_1$, with check noise σ_1 and threshold τ_1. If the check passes, release a GNMax answer with noise σ_2 (Papernot et al. 2018, Algorithm 1).

Proposition 42.3 (Cost of Confident-GNMax)**.** *With data-independent accounting, a pool of Q_{pool} queries, of which Q_{ans} are answered, costs*

$$\varepsilon(\alpha) = \Big(\frac{Q_{\text{pool}}}{2\sigma_1^2} + \frac{Q_{\text{ans}}}{\sigma_2^2}\Big)\alpha, \tag{42.11}$$

provided this total is kept below a budget fixed in advance.

Proof.

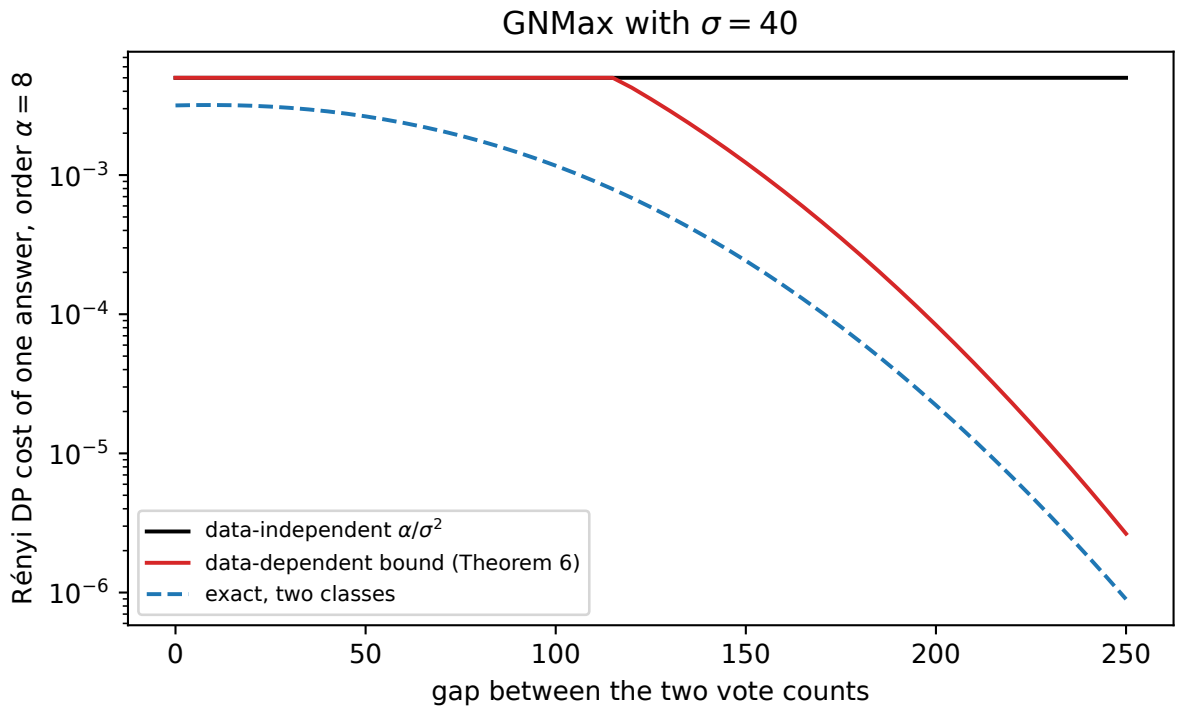


Figure 42.4.: The RDP cost at order 8 of one GNMax answer with $\sigma = 40$ against the gap, log scale: data-independent, the bound of Equation 42.10, and the exact cost on two labels (dashed).

1. *The check.* One record moves one count up or down by 1, which moves the maximum by at most 1. So the check is a Gaussian mechanism with sensitivity 1, $(\alpha, \alpha/(2\sigma_1^2))$-RDP by Theorem 29.1, on every query.
2. *The answer.* It is GNMax, $(\alpha, \alpha/\sigma_2^2)$-RDP by Theorem 42.1, run only when the check passes.
3. *Compose.* Whether an answer runs depends on a released output, so its cost is chosen adaptively. The Rényi filter (Theorem 36.1) says that adaptive RDP costs that stay below a budget fixed in advance compose as if fixed, and summing the two lines gives Equation 42.11.

□

The check is paid on every query, so σ_1 is chosen much larger than σ_2. The price of skipping the expensive, likely-wrong queries is that the student never sees labels near the decision boundary.

Example 42.5. Take first the pure-DP form. For an LNMax answer with $\varepsilon_0 = 0.1$ at order 8, the randomized-response curve charges 0.0367, and Equation 42.9 gives 0.0235 at $\tilde{\beta} = 0.1$ and 0.00025 at $\tilde{\beta} = 0.001$.

For GNMax with 250 teachers and $\sigma = 40$, Table 42.7 compares the costs of answering the first Q public queries.

Table 42.7.: ε at $\delta = 10^{-5}$ for Q GNMax answers, 250 teachers, $\sigma = 40$.

Answers Q	Data-independent ε	Data-dependent ε (not sanitized)
250	2.45	1.93
1,000	5.38	4.37
4,000	12.30	10.23

The saving is about 20% at every Q, because only 0.2879 of the public queries have a gap of at least 150. LNMax at the same noise standard deviation costs more at every Q (Figure 42.5). If one teacher switched all its votes on the first 1,000 queries, the data-dependent ε would move from 4.375 to 4.389.

Table 42.8 runs Confident-GNMax with the 250 teachers and the parameters Papernot et al. use on MNIST.

Table 42.8.: Confident-GNMax with 250 teachers, $\tau_1 = 200$, $\sigma_1 = 150$, $\sigma_2 = 40$; the student is trained on the answered queries.

Pool Q_{pool}	Answered Q_{ans}	Label accuracy	Data-independent ε	Data-dependent ε	Student accuracy
1,000	459	0.8693	3.59	2.82	0.8894
4,000	1,773	0.8917	7.87	6.28	0.9043

At a data-independent $\varepsilon = 3.59$, the student reaches 0.8894, about what plain GNMax reached at $\varepsilon = 4$ (0.8898). The check costs $1{,}000/(2 \times 150^2) = 0.022$ in slope, against $459/40^2 = 0.287$ for the answers.

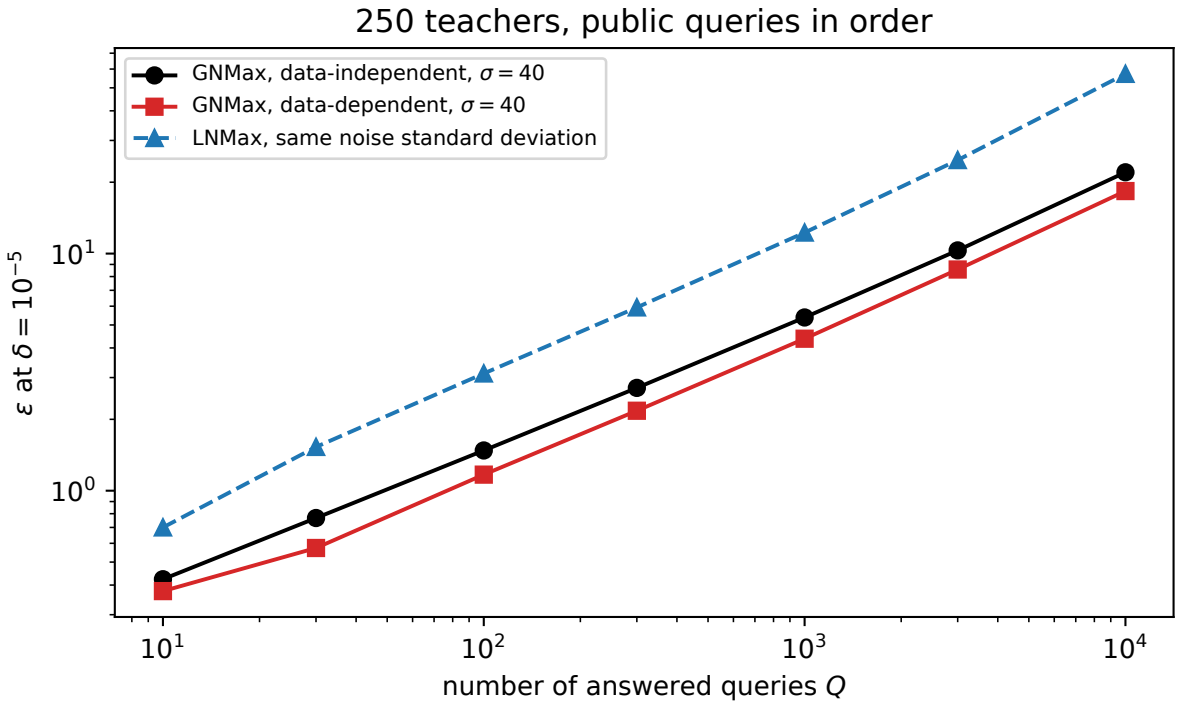


Figure 42.5.: ε for Q answers with 250 teachers, log–log: GNMax with $\sigma = 40$, data-independent and data-dependent, and LNMax with the same noise standard deviation.

42.6. Stable aggregation

The data-dependent ε depends on the votes, so publishing it needs a separate private release. This section asks whether agreement can be made cheap with a budget fixed before any vote is seen. The idea is to test privately whether a vote is so lopsided that one record cannot overturn it, and to release the exact plurality when it is. Only failed tests cost budget, because they run through the sparse vector technique of Selection. This is the stability-based aggregation of Thakurta & Smith (2013), applied to PATE by Bassily, Thakkar & Thakurta (2018).

The **distance to instability** $\mathrm{dist}(x)$ of a query x is the largest number j such that changing the votes of *any* j teachers leaves the same label strictly on top, the stability score of Theorem 24.5 applied to the votes.

Lemma 42.2 (Distance to instability)**.** *With g the gap between the top two counts,*

$$\mathrm{dist}(x) = \left\lceil \frac{g}{2} \right\rceil - 1, \quad \textit{and } 0 \textit{ when } g = 0. \tag{42.12}$$

Adding or removing one record moves the distance of every query by at most 1.

Proof.

1. *The formula.* One changed teacher moves one vote between labels, which lowers the gap by at most 2. After j changes the label stays strictly on top while $g - 2j \geq 1$, that is, for $j \leq (g-1)/2$, and the largest such whole number is $\lceil g/2 \rceil - 1$.

2. *Sensitivity.* A record changes one teacher (Lemma 42.1). If any j changes from D' keep the label on top, then any $j-1$ changes from D do, because D is one change away from D'. This is the argument of Lemma 24.2, and it holds for every query at once.

□

AboveThreshold halts at the first high value, so stable aggregation, defined next, runs it on the negated distances to halt at the first low distance.

Definition 42.4 (Stable aggregation)**.** Fix a budget ε_0 per run, a cutoff c and a pool of Q_{pool} public queries. Go through the queries in order, running AboveThreshold (Definition 22.2) at budget ε_0 on the values $-\operatorname{dist}(x_i)$ with threshold $-\tau$, threshold noise $\xi \sim \operatorname{Lap}(2/\varepsilon_0)$ and fresh query noise $\nu_i \sim \operatorname{Lap}(4/\varepsilon_0)$. A query with $-\operatorname{dist}(x_i) + \nu_i \geq -\tau + \xi$ is **unstable**: it gets no label, and the run restarts with fresh threshold noise. Every other query is **stable** and gets the exact plurality label. Stop after c unstable answers or at the end of the pool. The slack is split into δ_{bad}, for the event that a query at distance 0 is declared stable, and δ', for the RDP conversion of Theorem 42.5, and τ is chosen so that

$$Q_{\text{pool}} \cdot \Pr[\nu - \xi > \tau] \leq \delta_{\text{bad}}, \qquad \Pr[\nu - \xi > \tau] = \frac{b_1^2 e^{-\tau/b_1} - b_2^2 e^{-\tau/b_2}}{2(b_1^2 - b_2^2)}, \tag{42.13}$$

with $b_1 = 4/\varepsilon_0$ and $b_2 = 2/\varepsilon_0$ the two Laplace scales.

Appendix Section E.6 derives the tail formula. A label can leak only through a query at distance 0 that is declared stable, and the threshold makes that rarer than δ_{bad}.

Theorem 42.5 (Privacy of stable aggregation)**.** *The labels that stable aggregation releases are together $(\varepsilon, \delta' + \delta_{bad})$-DP, where ε is the better of basic composition and the RDP conversion at δ' of c composed ε_0-DP steps,*

$$\varepsilon = \min\Big\{c\varepsilon_0,\ \min_{\alpha}\Big[c\,\varepsilon_{\text{RR}}(\alpha) + \ln\frac{\alpha-1}{\alpha} - \frac{\ln\delta' + \ln\alpha}{\alpha-1}\Big]\Big\}, \tag{42.14}$$

with ε_{RR} the RDP curve of randomized response at ε_0 (Theorem 29.1). The budget does not depend on how many stable labels are released.

Proof. The plan is to price the stable/unstable answers, show that the exact labels add nothing outside a bad event, and bound that event by δ_{bad}.

1. *The answers cost c runs.* By Lemma 42.2 the values $-\operatorname{dist}(x_i)$ have sensitivity 1, so each run of AboveThreshold is ε_0-DP however many queries it reads (Theorem 22.6). Basic composition (Theorem 19.3) of at most c runs gives $c\varepsilon_0$. Alternatively, Theorem 31.3 gives each run the randomized-response curve, the curves add, and Equation 42.2 at δ' gives the second term. Either bound is valid, so the smaller one is.
2. *Exact labels are safe at distance at least 1.* At such a query, one change of teacher leaves the same label on top, so the plurality is the same on D and D'. When every labelled query has distance at least 1 on D, the output is therefore fixed by which queries were declared stable, in the same way for both datasets.

3. *The bad event.* A query at distance 0 is declared stable when $\nu_i - \xi < -\tau$. The difference of two independent symmetric variables is symmetric, so this has probability $\Pr[\nu - \xi > \tau]$, and a union bound over at most Q_{pool} queries with Equation 42.13 bounds the bad event by δ_{bad}.
4. *Put the two together.* For a set S of outputs, the chance of landing in S on D is at most the chance of doing so without the bad event, plus δ_{bad}. Without it, the output lands in S only if the stable/unstable answers land in a set S' whose labels, the same on both datasets by step 2, put the output in S. Step 1 bounds the chance of S' on D by e^{ε} times its chance on D', plus δ', which gives $(\varepsilon, \delta' + \delta_{\text{bad}})$-DP.

□

Nothing needs sanitizing, as long as c is fixed in advance, and the labels are exact, so the label noise that hurt the students of Section 42.4 is gone. What depends on the data is how many labels the budget buys.

The threshold is the obstacle. It must stop every query at distance 0 across the pool, so τ grows like $4\ln(Q_{\text{pool}}/\delta_{\text{bad}})/\varepsilon_0$, and a query is declared stable reliably only when the plurality leads by about 2τ votes. So the method pays only with many teachers that are nearly unanimous on almost every query, and deployed PATE systems use Confident-GNMax instead.

Example 42.6. Take $\varepsilon_0 = 1$ and $\delta_{\text{bad}} = \delta' = 5 \times 10^{-6}$, a total δ of 10^{-5}. For a pool of 1,000 queries Equation 42.13 gives $\tau = 74.8$, and for 10,000 queries 84.0, a lead of about $2\tau \approx 170$ votes. The budget of Equation 42.14 is $\varepsilon = 20.00$ for $c = 20$, from basic composition, and even 100 unstable answers cost 83.45 by the RDP route rather than 100.

With 250 teachers the largest distance on the pool is 124, and only 0.1996 of the queries reach 84.0, against 0.3923 with 500 teachers. Table 42.9 runs the procedure.

Table 42.9.: Stable aggregation on the public pool, $\delta_{\text{bad}} = \delta' = 5 \times 10^{-6}$: queries read before the c-th unstable answer, labels released, their accuracy, and the budget.

Teachers	ε_0	Cutoff c	Queries read	Labels released	Label accuracy	ε
250	1	20	23	3	1.0000	20.00
250	2	20	41	21	0.9524	40.00
500	1	20	34	14	1.0000	20.00
500	2	20	54	34	0.9412	40.00

The labels are accurate, but only 3 to 34 at a budget of 20 or more.

42.7. Private in-context learning

An LLM can learn a task from examples in its prompt, without training: show it four labelled movie reviews and it labels the fifth. If these examples, the **exemplars**, are private records, every answer leaks them. Duan, Dziedzic, Papernot and Boenisch (NeurIPS 2023) show this with a

membership inference attack on GPT-3, and DP-SGD is not an option behind an interface. This section asks how to answer privately from a prompt. The answer is to treat each prompt built from a group of exemplars as a teacher.

Definition 42.5 (Private in-context learning and PromptPATE)**. Private in-context learning** (Wu, Panda, Wang & Mittal, ICLR 2024; Duan et al. 2023) splits the private exemplars into disjoint groups by a rule fixed in advance, prompts the model once per group with that group's exemplars and the query, and releases the GNMax label of the answers. In the version of Wu et al., each query first draws a fresh Poisson sample of the exemplars, each record included independently with probability q, and splits the sample into groups by the fixed rule. **PromptPATE** (Duan et al.) labels a few public inputs with Confident-GNMax over prompted models and uses them as the exemplars of one public prompt.

Theorem 42.6 (The cost of private in-context learning)**.** *A PromptPATE prompt costs only its labels, however often it is used. For answers to queries, each is $(\alpha, \alpha/\sigma^2)$-RDP without subsampling. With Poisson subsampling at rate q, Q answers have Q times the curve of the Poisson-subsampled Gaussian at rate q and noise multiplier $\sigma/\sqrt{2}$ (Amplification by subsampling), the accountant DP-SGD uses:*

$$\varepsilon(\alpha) = Q\,\varepsilon_{q,\,\sigma/\sqrt{2}}(\alpha). \tag{42.15}$$

Proof. The plan is to show that one exemplar moves one vote, so that the earlier analysis applies, then to add the subsampling and compose.

1. *One exemplar, one prompt.* The groups are fixed by a rule that does not depend on the data, which places each exemplar without looking at the others, so adding or removing an exemplar changes one group's prompt and at most one answer. As in Lemma 42.1, the histogram has ℓ_2 sensitivity $\sqrt{2}$.
2. *Without subsampling.* GNMax on this histogram is $(\alpha, \alpha/\sigma^2)$-RDP by Theorem 42.1, whatever the model computes from its prompt.
3. *With subsampling.* The record is in the query's sample with probability q. If it is not, the output is distributed as without it. If it is, the histogram shifts by at most $\sqrt{2}$ against noise σ on each coordinate, which, rotated onto one axis and divided by $\sqrt{2}$, is the subsampled Gaussian with sensitivity 1 and noise multiplier $\sigma/\sqrt{2}$.
4. *Compose.* The Q answers use independent samples, so their curves add (Theorem 29.2), which is Equation 42.15.
5. *PromptPATE.* The deployed prompt is a function of public inputs and private labels, so by Theorem 29.3 it costs what the labels cost.

□

Everything about private prediction carries over, and the difference is the teacher. A prompted LLM learns from very few examples, because most of what it knows comes from pretraining, so its votes agree far more than those of small independent models, and high consensus makes data-dependent costs small. Within one query the groups must still be disjoint, or a record in two groups moves two votes.

With a real LLM the loss is small: on a movie-review sentiment task, Duan et al. report 92.7% with GPT-3 and PromptPATE at $(\varepsilon, \delta) = (0.147, 10^{-6})$, against 95.2% without privacy.

Example 42.7. No language model is available here, so the script uses a **stand-in** without pretraining: given exemplars (x_j, y_j), it labels a query x by the sign of $\langle \theta, x \rangle$, with $\theta = \sum_j y_j x_j$. An attacker who scores a candidate record (x, y) by $y\langle \theta, x \rangle$ has an area under the curve (AUC) of 1.000 against a 1-shot learner, 0.996 against a 4-shot learner and 0.889 against a 16-shot learner (400 trials each).

For private answers, each query samples the private records at rate $q = 0.04$ and splits the sample by record index into 25 groups of about 16. One 16-shot learner is right on 0.6232 of the evaluation set, and the 25-vote without noise on 0.8090. Table 42.10 adds the noise from Equation 42.15.

Table 42.10.: Private in-context learning with the stand-in: 25 groups of about 16 exemplars per query, sampling rate 0.04, accuracy on 2,000 evaluation queries.

Budget	Answers Q	σ	Accuracy
$\varepsilon = 1$	100	2.75	0.7780
	1,000	7.39	0.6800
	10,000	22.93	0.5615
$\varepsilon = 4$	100	1.30	0.8080
	1,000	2.36	0.7910
	10,000	6.65	0.7025

Subsampling makes the noise small, $\sigma = 7.39$ for 1,000 answers at $\varepsilon = 1$ against 181 without it (Table 42.2). But 25 voters have small gaps, so accuracy still falls from 0.8090 to 0.6800 (Figure 42.6).

A student prompt avoids the per-query cost. Split the records into 625 disjoint 16-shot learners, whose noiseless vote is right on 0.9366 of the evaluation set, and label the first Q public inputs with GNMax at $\varepsilon = 1$. With 16 labelled inputs ($\sigma = 22.9$) the student prompt reaches 0.6315, and with 256 ($\sigma = 91.5$) 0.7954, about the accuracy of 100 private answers but for any number of queries. An LLM needs a handful of exemplars rather than hundreds, which is why PromptPATE can stop at a very small ε.

42.8. Code

The script code/private_prediction.py implements every mechanism and accountant of this chapter, trains the students and the DP-SGD baselines, and simulates private in-context learning. It imports the running example and DP-SGD from code/dp_sgd.py, and the logistic regression from code/dp_erm.py. From the repository root:

```
uv run python code/part05-dp-ml/private_prediction.py
```

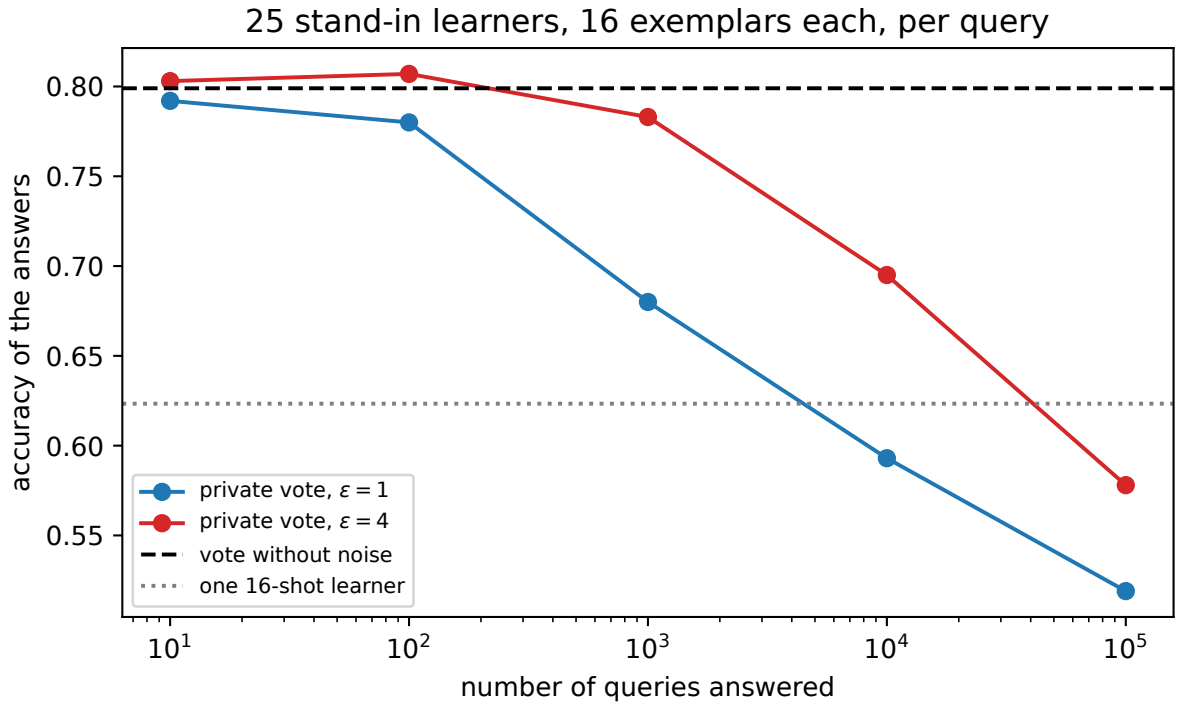


Figure 42.6.: Accuracy of the stand-in's private answers against Q at $\varepsilon = 1$ and $\varepsilon = 4$. Dashed: the vote without noise; dotted: one 16-shot learner.

Table 42.11.: The script's output with the default seed.

Experiment	Result
Noise for Q answers	$\sigma \propto \sqrt{Q}$: 57.2 for 100 answers at $\varepsilon = 1$, 572 for 10,000
Teachers	Plurality accuracy 0.9336 to 0.9403 for 25 to 500 teachers
Answers against DP-SGD	Answers win for up to 58 queries at $\varepsilon = 0.5$, 18 at $\varepsilon = 1$, none from $\varepsilon = 2$
PATE student, 250 teachers	Best 0.8238 at $\varepsilon = 1$ to 0.9049 at $\varepsilon = 8$; DP-SGD 0.9128 to 0.9605
Data-dependent analysis	About 20% lower ε on this task; Theorem 42.4 above the exact two-label cost
Confident-GNMax	459 of 1,000 queries answered, student 0.8894 at $\varepsilon = 3.59$
Stable aggregation	Threshold 84.0 at $\varepsilon_0 = 1$; 14 exact labels for $\varepsilon = 20$ with 500 teachers
In-context learning, stand-in	Membership AUC 0.889 for a 16-shot learner; 0.7780 accuracy for 100 private answers at $\varepsilon = 1$

tests/test_private_prediction.py checks that the output still matches.

42.9. Summary

Table 42.12.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 42.1	Disjoint shards fixed by a hash; release only the noisy argmax label	Count your queries first
Lemma 42.1	One record moves the histogram by 2 in ℓ_1, $\sqrt{2}$ in ℓ_2	Every cost here
Theorem 42.1	LNMax $(2/b)$-DP; GNMax $(\alpha, \alpha/\sigma^2)$-RDP; $\sigma = \sqrt{Q/\rho(\varepsilon,\delta)}$	Gaussian beyond a handful of answers
Proposition 42.1	Two labels: $\Phi(g/(\sqrt{2}\sigma))$; m labels: failure at most $\tilde{\beta}$	Add teachers while gaps grow
Proposition 42.2	Answers win for $Q < Q^*(\varepsilon)$	Private prediction or DP-SGD
Theorem 42.2	The student costs its Q labels	Needs public inputs; teachers stay secret
Theorem 42.3	Equation 42.9, for $\beta < e^{-\varepsilon_0}$	Cost proportional to the chance of being overturned
Theorem 42.4	Equation 42.10 under its conditions	Not a guarantee until sanitized
Proposition 42.3	$(Q_{\text{pool}}/(2\sigma_1^2) + Q_{\text{ans}}/\sigma_2^2)\alpha$	$\sigma_1 \gg \sigma_2$
Lemma 42.2	dist $= \lceil g/2 \rceil - 1$, sensitivity 1	The stability score
Theorem 42.5	Exact labels, $(\varepsilon, \delta' + \delta_{\text{bad}})$ from c runs	Check the share of queries above the threshold first
Theorem 42.6	$Q\,\varepsilon_{q,\sigma/\sqrt{2}}(\alpha)$	Never answer users from a prompt of private exemplars without a noisy vote

42.10. References

- Papernot, Abadi, Erlingsson, Goodfellow & Talwar, "Semi-supervised Knowledge Transfer for Deep Learning from Private Training Data" (ICLR 2017). PATE: teachers on disjoint shards, LNMax, a student trained on privately labelled public data, and the first data-dependent analysis (their Theorem 3, Equation 42.9).
- Papernot, Song, Mironov, Raghunathan, Talwar & Erlingsson, "Scalable Private Learning with PATE" (ICLR 2018). GNMax and its RDP (Proposition 8), the union bound Equation 42.6 (Proposition 7), the data-dependent bound Equation 42.10 (Theorem 6,

Proposition 10), Confident- and Interactive-GNMax, and the smooth-sensitivity release of the data-dependent ε (Appendix B).

- Nissim, Raskhodnikova & Smith, "Smooth Sensitivity and Sampling in Private Data Analysis" (STOC 2007). Sample-and-aggregate, of which PATE is an instance, and smooth sensitivity.
- Duan, Dziedzic, Papernot & Boenisch, "Flocks of Stochastic Parrots: Differentially Private Prompt Learning for Large Language Models" (NeurIPS 2023). The membership attack on prompts and PromptPATE.
- Wu, Panda, Wang & Mittal, "Privacy-Preserving In-Context Learning for Large Language Models" (ICLR 2024). Disjoint exemplar groups with Poisson subsampling and a Gaussian noisy vote for classification; embedding- and keyword-space aggregation for text generation.
- Thakurta & Smith, "Differentially Private Feature Selection via Stability Arguments, and the Robustness of the Lasso" (COLT 2013). The distance-to-instability test that releases a stable answer exactly.
- Bassily, Thakkar & Thakurta, "Model-Agnostic Private Learning" (NeurIPS 2018). PATE with stable aggregation and the sparse vector technique, and accuracy guarantees for the student; Section 42.6.
- Dwork & Feldman, "Privacy-preserving Prediction" (COLT 2018). Private prediction as a learning problem in its own right, with sample-complexity bounds for answering predictions privately.
- Mironov, "Rényi Differential Privacy" (CSF 2017). The Gaussian mechanism's RDP and composition used throughout.
- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020). The conversion of Equation 42.2.

43. Public data and private fine-tuning

The previous chapters trained models with differential privacy (DP) from random weights, by stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, called DP-SGD (DP-SGD). On this part's running example, described below, it costs little at a privacy budget of $\varepsilon = 8$ and a great deal at $\varepsilon = 1$ and below. The most accurate private models are not trained that way. They are **pretrained** without privacy on a large **public** dataset, such as text or images from the web, and then **fine-tuned** privately on the sensitive data (Tramèr & Boneh 2021; Yu et al. 2022; Li et al. 2022). This chapter asks why that helps so much. It also asks why fine-tuning only a small part of the model, such as the last layer or a low-rank update, often beats fine-tuning all of it under DP.

The one idea is that DP-SGD pays for noise in every trainable direction, while the useful part of the gradient, the signal, does not grow with the number of directions. The noise has the same size in each coordinate it updates, so its length grows with the number of trainable weights. The signal has length at most the clipping norm, and in practice it is concentrated in a few directions. Public data move the starting point close to a good model, and they tell you, at no privacy cost, which few directions to train. Training only those cuts the noise without cutting the signal, unless the task needs a direction that was left out.

The running example is the part's. Its inputs are 50-dimensional standard normal vectors, each labelled by the sign of a fixed random teacher network with 8 hidden tanh units. The network being trained, the student, has 64 hidden tanh units and $p = 3{,}329$ weights, and it trains on $n = 10{,}000$ private records. A public task labels 50,000 more inputs with a teacher close to the private one (Section 43.5). Fine-tuning runs DP-SGD for $T = 10{,}000$ steps, with sampling rate $q = 0.01$ and clipping norm $C = 0.5$, at $\delta = 10^{-5}$, and neighbouring private datasets $D \sim D'$ differ by adding or removing one record. Pretrained on the public task, the student is right on 0.9035 of the private test inputs before any private training. At $\varepsilon = 1$, private training from scratch reaches 0.9246, and private fine-tuning of a small low-rank adapter reaches 0.9616. Table 43.1 lists the symbols.

Table 43.1.: The symbols of this chapter.

Symbol	Meaning
n, p, d	Private records; weights of the model; trainable weights, the rest frozen
C, σ, q, $B = qn$	Clipping norm, noise multiplier, sampling rate, expected batch size
T, η	Number of steps; learning rate
$s = \sigma C/B$	Noise standard deviation per coordinate on the averaged gradient

Symbol	Meaning
H, h_j, d_{eff}	Curvature of the loss at the start; its eigenvalues; the effective dimension
D_{pub}, D	Public dataset; private dataset

43.1. The cost of each trainable direction

Every bound on private learning in this part grows with the dimension. Private empirical risk minimization shows that the best excess risk under (ε, δ)-DP grows like $\sqrt{d}$ for convex Lipschitz losses and like d for strongly convex ones. Yet large pretrained language models with hundreds of millions of weights are fine-tuned privately with good results. Before choosing which weights to train, you need to know what each trainable direction really costs.

The privacy side does not depend on that choice. DP-SGD proves that each step's clipped sum has sensitivity C whatever the model, as long as clipping acts on the vector of trainable coordinates, and its accountant turns q, σ and T alone into (ε, δ). Accuracy does depend on the choice, because the noise lands on every trainable coordinate. The next lemma measures that noise, in all d coordinates and in a subspace. Its second half is what later proofs use to treat one direction at a time.

Lemma 43.1 (Gaussian noise in d coordinates and in a subspace)**.** *Let $\xi \sim \mathcal{N}(0, s^2 I_d)$ be a vector of d independent normal coordinates with mean 0 and standard deviation s. Its squared length has mean ds^2 and relative spread $\sqrt{2/d}$, so its length is close to $s\sqrt{d}$ when d is large. In every orthonormal basis the coordinates of ξ are again independent $\mathcal{N}(0, s^2)$. So if P_m is the orthogonal projection onto a fixed m-dimensional subspace, $P_m\xi$ has standard deviation s along each of m orthonormal directions of the subspace, and length close to $s\sqrt{m}$.*

Proof.

1. *The squared length.* It is a sum of d independent terms ξ_i^2, each with mean s^2 and variance $\mathbb{E}[\xi_i^4] - s^4 = 2s^4$, since a normal's fourth moment is $3s^4$. So the sum has mean ds^2 and standard deviation $\sqrt{2d}\, s^2$, a share $\sqrt{2/d}$ of the mean, and Chebyshev's inequality (Theorem 2.2) keeps it near the mean.
2. *Rotation invariance.* The density of ξ is proportional to $e^{-\|\xi\|^2/(2s^2)}$, which depends only on the length. A change of orthonormal basis keeps lengths, so the density keeps its product form in the new coordinates. Section E.1 in the appendices proves this in full.
3. *Projection.* Choose the new basis so that its first m vectors span the subspace. Then $P_m\xi$ keeps those m coordinates, which by step 2 are independent $\mathcal{N}(0, s^2)$, and step 1 with m in place of d gives their length.

□

In DP-SGD the noise on the averaged gradient is the step's noise $\xi_t \sim \mathcal{N}(0, \sigma^2 C^2 I_d)$ divided by the expected batch size B, the fixed denominator DP-SGD uses. Its standard deviation per

coordinate is $s = \sigma C/B$, so by Lemma 43.1 its length is

$$\frac{\|\xi_t\|}{B} \approx s\sqrt{d} = \frac{\sigma C\sqrt{d}}{B}. \tag{43.1}$$

What the noise costs is the loss it adds, and its length alone does not say that. To find the loss, follow the noise on a loss that is quadratic near the pretrained weights, a local model that holds while fine-tuning stays near them. There the noise's contribution to the weights is a sum of independent Gaussian kicks, each shrunk by the curvature. Steep directions forget old kicks, while flat ones accumulate kicks but charge little for them.

Theorem 43.1 (The noise cost of T noisy steps)**.** *Let the training loss be a quadratic in the weights θ, $\mathcal{L}(\theta) = \mathcal{L}(\theta^*) + \frac{1}{2}(\theta-\theta^*)^\top H(\theta-\theta^*)$. Assume that its curvature H, a symmetric matrix on the d trainable coordinates, has eigenvalues h_j with $0 \le h_j < 2/\eta$, so that each step shrinks every direction's error. Run T steps of noisy gradient descent $\theta_{t+1} = \theta_t - \eta(\nabla\mathcal{L}(\theta_t) + \xi_t)$ with independent $\xi_t \sim \mathcal{N}(0, s^2 I_d)$, and the same steps without noise from the same start. The noise raises the expected loss after T steps by the* ***noise cost***

$$\mathcal{E} = \sum_{j=1}^{d} \frac{\eta s^2}{2(2-\eta h_j)}\Big(1-(1-\eta h_j)^{2T}\Big) \approx \frac{\eta s^2}{4}\, d_{\mathit{eff}}, \qquad d_{\mathit{eff}} = \sum_{j=1}^{d}\Big(1-(1-\eta h_j)^{2T}\Big), \tag{43.2}$$

where the approximation holds when $\eta h_j \ll 1$. The ***effective dimension*** *d_{eff} satisfies*

$$d_{\mathit{eff}} \le \min\big(d,\ 2\eta T \operatorname{tr} H\big). \tag{43.3}$$

Proof. The plan is to write the noise's effect on the weights as a linear recursion, turn the loss increase into a variance, compute that variance one eigenvector at a time, and bound the sum.

1. *Separate the noise.* The gradient of the quadratic is $H(\theta-\theta^*)$. Write $\bar\theta_t$ for the iterate without noise and $e_t = \theta_t - \bar\theta_t$ for the difference. Subtracting the two update rules, the terms with θ^* cancel, so

$$e_{t+1} = (I-\eta H)e_t - \eta\xi_t, \qquad e_0 = 0. \tag{43.4}$$

   So e_t is a linear function of the noise alone, a Gaussian vector with mean zero.
2. *The loss increase as a variance.* Expanding the quadratic around $\bar\theta_T$ gives the loss without noise, a term linear in e_T and a quadratic term:

$$\mathcal{L}(\theta_T) = \mathcal{L}(\bar\theta_T) + \nabla\mathcal{L}(\bar\theta_T)^\top e_T + \tfrac{1}{2}e_T^\top H e_T. \tag{43.5}$$

   The middle term has mean zero, because e_T has mean zero and $\bar\theta_T$ is not random. So $\mathcal{E} = \frac{1}{2}\mathbb{E}[e_T^\top H e_T]$.
3. *One direction at a time.* In the eigenbasis of H the recursion splits into one scalar recursion per eigenvector, and by Lemma 43.1 the noise coordinates in that basis are still independent $\mathcal{N}(0, s^2)$. Along an eigenvector with curvature h, the error obeys $e_{t+1} = (1-\eta h)e_t - \eta\xi$. Each kick shrinks by the factor $1-\eta h$ at every later step, and the kicks are independent, so their variances add up to a geometric sum:

$$v_T = \eta^2 s^2 \sum_{t=0}^{T-1}(1-\eta h)^{2t} = \eta^2 s^2\, \frac{1-(1-\eta h)^{2T}}{1-(1-\eta h)^2}. \tag{43.6}$$

4. *Its cost.* The quadratic term of step 2 charges the direction $\frac{1}{2}hv_T$. The denominator of v_T factors as $\eta h(2-\eta h)$, so for $h > 0$ the factor h cancels, and for $h = 0$ both sides below are 0:

$$\frac{1}{2}hv_T = \frac{\eta s^2}{2(2-\eta h)}\Big(1-(1-\eta h)^{2T}\Big). \tag{43.7}$$

   Summing over the d directions gives Equation 43.2, and when $\eta h \ll 1$ the first factor is $\eta s^2/4$.
5. *Which directions pay.* When $2\eta hT \gg 1$ the bracket is close to 1. The direction is at its steady state, the case Private empirical risk minimization analyses, and it costs $\eta s^2/4$ whatever its curvature, because stiffer directions wander less but charge more. When $2\eta hT \ll 1$ the bracket is about $2\eta hT$, since $(1-x)^{2T} \approx 1-2Tx$ for small x. The noise there has random-walked a distance of variance $T\eta^2 s^2$, charged at the small rate $h/2$.
6. *The bound.* Each bracket is at most 1, which gives $d_{\text{eff}} \le d$. Bernoulli's inequality $(1-x)^{2T} \ge 1-2Tx$ holds for $0 \le x \le 1$, and for larger x its right side is negative while the even power on the left is not. So each bracket is at most $2\eta h_j T$, and summing over j gives $d_{\text{eff}} \le 2\eta T \operatorname{tr} H$.

□

So the count of trainable weights is an upper bound, not the price. Noise in a direction where the loss is flat costs nothing, however long it is. By step 5, what you pay for is roughly the number of directions with $2\eta hT$ above 1, that is with curvature above $1/(2\eta T)$, the directions the noise has time to fill. A pretrained model whose loss is curved in few directions can therefore be fine-tuned in all its weights at a cost set by those few. Li, Liu, Hashimoto, Inan, Kulkarni, Lee & Thakurta (2022) explain the success of private fine-tuning of large language models this way. And since d_{eff} grows with ηT, a longer or more aggressive run fills more flat directions with noise.

Fewer trainable weights still help, for two reasons. Freezing weights restricts H to fewer coordinates. By Cauchy's interlacing theorem, the k-th largest eigenvalue of the restricted matrix is at most the k-th largest of the full one, so d_{eff} has fewer and smaller terms. When every $\eta h_j \le 1$, each term grows with h_j, so d_{eff} can only fall. And with fewer coordinates each per-example gradient is shorter, so fewer gradients are shrunk by clipping before they are averaged, and the signal can be longer. Against both stands the **bias** of freezing: a direction the task needs but that is frozen cannot move. Note too that a loss increase is not an accuracy loss, so use Equation 43.2 to compare strategies, not to predict accuracy.

Example 43.1. Table 43.2 gives the noise length Equation 43.1 for the three trainable sets of Section 43.5, with $B = 100$ and $C = 0.5$, and the noise multiplier σ that the accountant gives for each budget.

Table 43.2.: Length of the noise on the averaged gradient, $\sigma C\sqrt{d}/B$, for three numbers of trainable weights.

ε	σ	All weights, $d = 3{,}329$	Rank-4 adapter and last layer, $d = 525$	Last layer, $d = 65$
0.5	7.72	2.227	0.884	0.311

ε	σ	All weights, $d = 3{,}329$	Rank-4 adapter and last layer, $d = 525$	Last layer, $d = 65$
1	4.13	1.191	0.473	0.166
2	2.28	0.658	0.261	0.092
8	0.917	0.265	0.105	0.037

The signal is the average clipped gradient over the private records at the pretrained weights. Its length is 0.0130 when all weights are trained, 0.0207 for the adapter and 0.0250 for the last layer alone (Figure 43.1). The last layer's 65 coordinates carry a *longer* signal than all 3,329 together, because each record's gradient restricted to them is shorter and less often clipped. At $\varepsilon = 1$ the noise is 92.0 times the signal for all weights, 22.9 times for the adapter and 6.7 times for the last layer. A signal of constant direction outgrows a random walk of noise after $(\text{noise}/\text{signal})^2$ steps (DP-SGD), here 8,455, 523 and 44 steps.

The curvature tells a less pessimistic story. The chapter's script (Section 43.7), as a diagnostic outside any private release, measures the **Gauss–Newton curvature** of the private loss, the Hessian of the logistic loss with the network replaced by its linearization at the pretrained weights. It is positive semi-definite, and near a good fit it is close to the full Hessian. Estimated from 2,000 private records, it has largest eigenvalue 0.723 and trace 16.310 over all weights, and 0.142 and 0.681 for the last layer. With $\eta = 0.1$ and $T = 10{,}000$, only 406 of the 3,329 directions have curvature above $1/(2\eta T) = 5 \times 10^{-4}$, and $d_{\text{eff}} = 451$, against 60.2 of 65 for the last layer. The full model's noise is $\sqrt{3{,}329/65} = 7.2$ times longer than the last layer's, so counting weights would charge it the squared ratio, $3{,}329/65 = 51$ times the cost, but Equation 43.2 charges only $451/60.2 = 7.5$ times. At $\varepsilon = 1$, where $s = 0.0206$, Equation 43.2 predicts a loss increase of 0.0048 for all weights at $\eta = 0.1$, against 0.0355 if every direction had reached its steady state. For the last layer alone it predicts 0.00064.

43.2. The last layer on frozen features

The simplest way to use a pretrained model is to freeze everything except the last layer. For the running example that is logistic regression on the 64 hidden activations plus a bias, 65 weights, and it turns private deep learning back into the convex problem of Private empirical risk minimization. Tramèr & Boneh (2021) showed that this recipe beats end-to-end private training on standard image benchmarks at moderate budgets, even with handcrafted features, and that features learned on public data do better still. This section asks what the private part costs, and what limits it.

Definition 43.1 (Linear probing)**.** Let $\phi\colon \mathcal{X} \to \mathbb{R}^d$ be a feature map computed from public data only, here the pretrained network's $d - 1$ hidden tanh activations and a constant 1. **Linear probing** freezes ϕ and trains a linear classifier $\theta \in \mathbb{R}^d$ on the features, with labels $y \in \{-1, 1\}$ and the logistic loss

$$\ell(\theta; (x, y)) = \ln\left(1 + e^{-y\langle \theta, \phi(x)\rangle}\right). \tag{43.8}$$

The privacy of a probe rests on one fact: as far as the private data are concerned, the features are a fixed function. Step 3 of the probe's proof needs this, and so do Section 43.4 and Section 43.6, so the next lemma states it once.

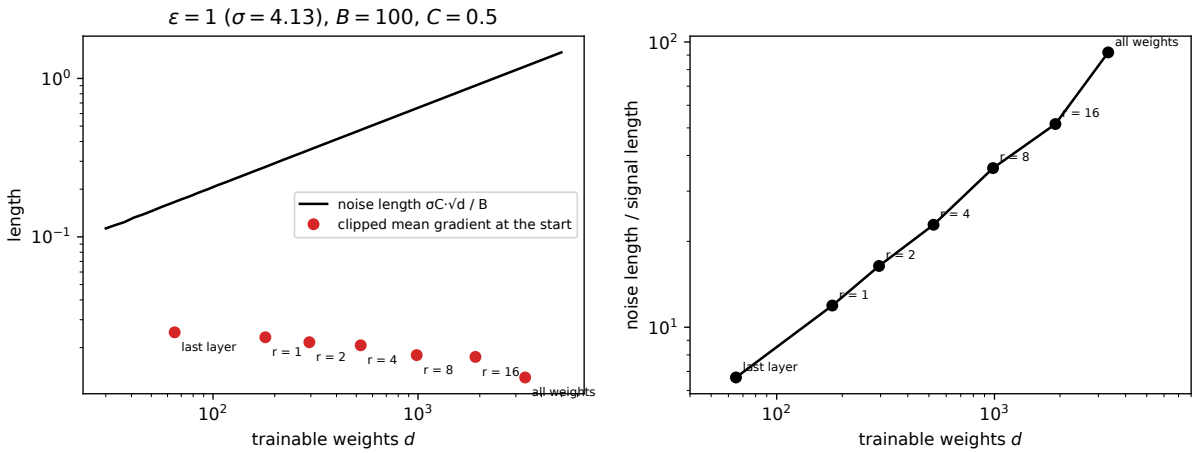


Figure 43.1.: Left: at $\varepsilon = 1$ ($\sigma = 4.13$, $B = 100$, $C = 0.5$), the noise length $\sigma C\sqrt{d}/B$ (line) against the number of trainable weights d, and the length of the average clipped gradient at the pretrained model (points) for the last layer, low-rank adapters of ranks 1 to 16, and all weights. Right: their ratio. The noise grows like $\sqrt{d}$ while the signal shrinks, because longer gradient vectors are clipped more.

Lemma 43.2 (Computations on public data are free)**.** *Let a be the output of any computation that reads only the public dataset D_{pub}, with coins independent of the private dataset D. Suppose that for every fixed value of a the mechanism $D \mapsto M(D; a)$ is (ε, δ)-DP. Then for every fixed D_{pub}, the mechanism $D \mapsto M(D; a)$ with a computed from D_{pub} is (ε, δ)-DP.*

Proof.

1. *The public output does not move.* Fix D_{pub}, neighbours $D \sim D'$ and an event S. The computation never reads the private data, so a has the same distribution ρ under D and under D'.
2. *Average the guarantee.* Each probability is an average over $a \sim \rho$ of the probability for a fixed a. The DP inequality holds for each fixed a, and averaging both sides over the same ρ keeps it:

$$\mathbb{E}_{a\sim\rho} \Pr[M(D; a) \in S] \le \mathbb{E}_{a\sim\rho}\left[e^{\varepsilon} \Pr[M(D'; a) \in S] + \delta\right] = e^{\varepsilon}\, \mathbb{E}_{a\sim\rho} \Pr[M(D'; a) \in S] + \delta. \tag{43.9}$$

□

In DP-SGD this is the assumption that the initial weights do not depend on the private data.

Proposition 43.1 (The cost of a private linear probe)**.** *In the setting of Definition 43.1, the following hold.*

1. *Every per-record gradient has length at most $\|\phi(x)\| < \sqrt{d}$, so the loss is $\sqrt{d}$-Lipschitz in θ and clipping is not needed for privacy.*

2. *Write $\mathcal{L}_{\mathcal{D}}$ for the population risk, $\mathcal{L}_{best}$ for the best population risk of any model, θ^{ϕ} for the best linear model on the features and $\tilde{\theta}$ for the private probe. The excess risk splits into a* ***feature gap*** *and a* ***privacy cost****,*

$$\mathcal{L}_{\mathcal{D}}(\tilde{\theta}) - \mathcal{L}_{best} = \underbrace{\mathcal{L}_{\mathcal{D}}(\theta^{\phi}) - \mathcal{L}_{best}}_{features} + \underbrace{\mathcal{L}_{\mathcal{D}}(\tilde{\theta}) - \mathcal{L}_{\mathcal{D}}(\theta^{\phi})}_{privacy}, \tag{43.10}$$

   and the feature gap depends on neither n nor ε.
3. *The privacy cost is that of private convex empirical risk minimization (ERM) in d dimensions. For losses that are L-Lipschitz over a ball of radius R, noisy projected gradient descent keeps it below a constant times $LR\sqrt{d\ln(1/\delta)}/(n\varepsilon)$, and no private algorithm does better than order $LR\sqrt{d}/(n\varepsilon)$.*

Proof.

1. *The gradient bound.* By the chain rule, $\nabla_{\theta}\ell = \ell'(u)\, y\, \phi(x)$, where $u = y\langle\theta, \phi(x)\rangle$ is the margin and $\ell'(u) = -1/(1+e^{u})$ lies in $(-1, 0)$. Each tanh activation lies in $(-1, 1)$ and the constant is 1, so $\|\phi(x)\|^2 < (d-1) + 1 = d$.
2. *The split.* Adding and subtracting $\mathcal{L}_{\mathcal{D}}(\theta^{\phi})$ gives Equation 43.10. Its first difference compares two non-private quantities, so it contains no noise and does not depend on n or ε.
3. *The privacy term.* By Lemma 43.2 the features are a fixed function, so the probe is ordinary convex ERM on the records $(\phi(x_i), y_i)$, with Lipschitz constant $\sqrt{d}$, or C if you clip. Private empirical risk minimization proves the upper bound for noisy projected gradient descent and cites the lower bound of Bassily, Smith & Thakurta (2014).

□

Features cap accuracy, and privacy only decides how close you get to the cap. The feature gap is paid even with infinite data and no privacy, while the privacy cost falls like $\sqrt{d}/(n\varepsilon)$. With few poor features the probe is private, cheap and bad, which is the point of Tramèr & Boneh's title: private learning needs better features, or much more data. End-to-end training can learn better features, but it pays for every weight to do so.

So linear probing is the safe first baseline. The problem is convex, so the optimizer cannot wander into a bad region, and every method of Private empirical risk minimization applies. Clipping still pays, because the bound $\sqrt{d}$ is reached only by a record with every activation saturated and a large negative margin. A record classified with a wide margin has $|\ell'(u)|$ close to 0, so clipping at a smaller C biases only a few gradients and divides the noise by $\sqrt{d}/C$.

Example 43.2. The pretrained network's 64 hidden units plus a constant give $d = 65$ features. The largest feature norm on the private training records is 7.035, below the bound $\sqrt{65} = 8.062$. The pretrained network with its public last layer labels 0.9035 of the private test inputs correctly. Fitting the last layer exactly and without privacy on the 10,000 private records raises this to 0.9536, the probe's ceiling θ^{ϕ} in Equation 43.10. A network trained without privacy from scratch reaches 0.9606, so the feature gap is 0.007 in accuracy. The private probe reaches 0.9478 at $\varepsilon = 0.5$ and 0.9514 at $\varepsilon = 8$ (Table 43.5). At every budget it is within 0.006 of its ceiling, and at no budget can it pass it.

43.3. Low-rank adapters

The last layer is often not enough, because features pretrained on one task are not exactly right for another. Training all weights fixes the features but pays for every weight. Low-rank adaptation (LoRA) sits in between (Hu et al. 2022). Yu et al. (2022) showed that private fine-tuning of language models with it matches or beats private full fine-tuning at a fraction of the cost. To run it under DP-SGD you need to know how many weights it trains, what its per-example gradients are, and where it can move each matrix.

Definition 43.2 (Low-rank adapter)**.** Take a pretrained $d_{\text{out}} \times d_{\text{in}}$ weight matrix W_0 and a rank r. A **low-rank adapter** freezes W_0 and trains two factors $U \in \mathbb{R}^{d_{\text{out}} \times r}$ and $V \in \mathbb{R}^{d_{\text{in}} \times r}$, so that the layer uses

$$W = W_0 + UV^\top. \tag{43.11}$$

Training starts from $U = 0$ and a random V, so the model starts exactly at the pretrained one. The adapter trains

$$d = r(d_{\text{in}} + d_{\text{out}}) \tag{43.12}$$

weights, against $d_{\text{in}} d_{\text{out}}$ for the matrix.

The adapter saves noise only while d is below $d_{\text{in}} d_{\text{out}}$. For the matrix itself, record i's gradient is the outer product $\nu_i a_i^\top$ of the gradient ν_i with respect to the layer's output and the layer's input a_i. Its length is $\|\nu_i\| \, \|a_i\|$, so clipping can compute it without forming the gradient, which is ghost clipping (DP-SGD). The factors' gradients follow by the chain rule and stay outer products.

Proposition 43.2 (Gradients and steps of a low-rank adapter)**.** *For record i, let ν_i be the gradient of its loss with respect to the layer's output and a_i the layer's input. The adapter's per-example gradients are outer products,*

$$\nabla_U \ell_i = \nu_i \, (V^\top a_i)^\top, \qquad \nabla_V \ell_i = a_i \, (U^\top \nu_i)^\top, \tag{43.13}$$

with lengths $\|\nu_i\| \, \|V^\top a_i\|$ and $\|a_i\| \, \|U^\top \nu_i\|$. Take one step of size η/B on both factors with the noisy sums of clipped gradients (the bar marks clipping) $G_U = \sum_i \overline{\nabla_U \ell_i} + \xi_U$ and $G_V = \sum_i \overline{\nabla_V \ell_i} + \xi_V$. It changes W to W' with

$$W' - W = -\frac{\eta}{B}(G_U V^\top + U G_V^\top) + O\Big(\frac{\eta^2}{B^2}\Big). \tag{43.14}$$

From $U = 0$, the change $W' - W$ has rank at most r, and to first order its rows lie in the span of the columns of V. The privacy guarantee is that of DP-SGD with the same q, σ and T.

Proof. The plan is to push the matrix gradient through the product $UV^\top$, read the lengths off the outer products, expand one step, and set $U = 0$.

1. *The gradient for U.* Write $G_i = \nu_i a_i^\top$ for record i's gradient with respect to W, through which alone the loss depends on the factors. A small change $\mathrm{d}U$ changes W by $\mathrm{d}U \, V^\top$, and so the loss by $\operatorname{tr}(G_i^\top \mathrm{d}U \, V^\top)$. The cyclic property of the trace moves $V^\top$ to the front:

$$\operatorname{tr}(G_i^\top \mathrm{d}U \, V^\top) = \operatorname{tr}(V^\top G_i^\top \mathrm{d}U) = \langle G_i V, \mathrm{d}U \rangle. \tag{43.15}$$

   So $\nabla_U \ell_i = G_i V = \nu_i (V^\top a_i)^\top$.

2. *The gradient for V.* A change $\mathrm{d}V$ changes W by $U\,\mathrm{d}V^\top$ and the loss by $\mathrm{tr}(G_i^\top U\,\mathrm{d}V^\top) = \langle G_i^\top U, \mathrm{d}V\rangle$, so $\nabla_V \ell_i = G_i^\top U = a_i(U^\top \nu_i)^\top$.
3. *Lengths.* An outer product of two vectors has length equal to the product of their lengths. So record i's whole gradient norm comes from vectors of dimension d_{out}, d_{in} and r, without forming a matrix.
4. *The step.* After the step, $W' = W_0 + (U - \frac{\eta}{B}G_U)(V - \frac{\eta}{B}G_V)^\top$. Expanding the product and subtracting $W = W_0 + UV^\top$ gives

$$W' - W = -\frac{\eta}{B}(G_U V^\top + U G_V^\top) + \frac{\eta^2}{B^2} G_U G_V^\top, \tag{43.16}$$

   which is Equation 43.14.
5. *From $U = 0$.* By step 2, $\nabla_V \ell_i = 0$ for every record, so $G_V = \xi_V$ is pure noise and the first-order change is $-\frac{\eta}{B}G_U V^\top$, whose rows are combinations of the columns of V. The second-order term $\frac{\eta^2}{B^2}G_U \xi_V^\top$ tilts them slightly. The whole change is a $d_{\text{out}} \times r$ matrix times an $r \times d_{\text{in}}$ one, so its rank is at most r.
6. *Privacy.* Clipping acts on the vector of all trainable coordinates, so the clipped sum has sensitivity C and the guarantee of DP-SGD applies.

□

An adapter is DP-SGD in a smaller space that moves. Its noise lives in $d = r(d_{\text{in}} + d_{\text{out}})$ coordinates, so Equation 43.1 applies with that d, and noise in the other directions of W never arises. But the restriction costs something at the start. By step 5 of the proof, with $U = 0$ the first update is the true gradient times $VV^\top$, which for a random V of small rank is close to a multiple of the projection onto a random r-dimensional subspace. By symmetry, such a subspace captures on average only r/d_{in} of a fixed vector's energy, since each of the d_{in} directions of a random basis gets an equal share. Once $U \neq 0$ the gradient for V is no longer zero, so V turns toward the directions the gradients use. So low-rank adaptation works when the change the new task needs is itself close to low rank, as Hu et al. (2022) observed for language models.

Full fine-tuning is not ruled out. Li, Tramèr, Liang & Hashimoto (2022) fine-tuned all the weights of language models with DP-SGD and matched or beat the parameter-efficient methods, those that train few weights such as adapters, once the hyperparameters were right. Theorem 43.1 explains why this can work: full fine-tuning pays d_{eff}, not p. Which wins depends on the model, the data and the tuning. The adapter's other advantages are certain, since it stores only d numbers per task and clips much smaller gradients.

Example 43.3. The running example's first layer maps 51 inputs (50 features and a constant) to 64 hidden units, which is 3,264 weights. A rank-4 adapter trains $4 \times (51 + 64) = 460$ of them, and with the 65 weights of the last layer that gives $d = 525$. Rank 1 gives 180 and rank 16 gives 1,905. On 256 private records, the ghost norms of Equation 43.13 agree with explicitly formed per-example gradients to a relative difference of 3.3×10^{-16}.

43.4. Gradients near a public subspace

Theorem 43.1 says noise costs little in directions the loss does not use, and near a good fit those are the directions the private gradients do not point along. If you knew the directions

the gradients do use, you could add noise only there. Public data give an estimate: compute per-example gradients on public records, find their top principal directions, and project the private gradients onto them. Zhou, Wu & Banerjee (2021) call this **projected DP-SGD**. The **gradient embedding perturbation** of Yu, Zhang, Chen & Liu (2021) also releases the part outside the subspace with separate noise. This section asks how much a projection saves and what it loses.

A projection chosen from public data is a fixed map, so it shrinks the noise and the signal but leaves the guarantee alone.

Proposition 43.3 (Projected DP-SGD)**.** *Let P_m be the orthogonal projection onto an m-dimensional subspace computed from public data only. Each step releases*

$$P_m\Big(\sum_{i\in S_t}\bar g_i\Big)+\mathcal{N}(0,\sigma^2C^2P_m), \tag{43.17}$$

the projected sum of the clipped gradients $\bar g_i$ of the batch S_t plus noise of standard deviation σC along each of m orthonormal directions of the subspace. This has the same (ε,δ) as DP-SGD with the same q, σ and T. Let $\bar g_D$ be the average clipped gradient over the private records, and $f_m=\|P_m\bar g_D\|^2/\|\bar g_D\|^2$ the share of its energy in the subspace. The kept signal has length $\sqrt{f_m}\,\|\bar g_D\|$, the noise on the averaged gradient has length about $s\sqrt{m}$, and the signal outgrows the noise after about

$$T_{even}(m)=\Big(\frac{s\sqrt{m}}{\sqrt{f_m}\,\|\bar g_D\|}\Big)^2 \tag{43.18}$$

steps, against $(s\sqrt{p}/\|\bar g_D\|)^2$ without projection. The lost share $1-f_m$ is a bias that no number of steps recovers.

Proof.

1. *Sensitivity.* A projection never lengthens a vector, because $x=P_mx+(x-P_mx)$ with orthogonal parts, so $\|x\|^2=\|P_mx\|^2+\|x-P_mx\|^2$. Adding or removing one record therefore changes the projected sum by at most C.
2. *The mechanism.* In an orthonormal basis of the subspace, Equation 43.17 is a vector of m coordinates and sensitivity at most C, plus independent $\mathcal{N}(0,\sigma^2C^2)$ noise on each. With Poisson sampling it is the subsampled Gaussian of DP-SGD. By Lemma 43.2 the public subspace is a fixed parameter, so the accountant of DP-SGD gives the same (ε,δ).
3. *Signal and noise.* Dividing by B, the kept signal is $P_m\bar g_D$, of length $\sqrt{f_m}\,\|\bar g_D\|$ by the definition of f_m. The noise has m coordinates of standard deviation s, so by Lemma 43.1 its length is about $s\sqrt{m}$.
4. *Break-even.* Over j steps a signal of constant direction adds up to j times its length, and independent noise to $\sqrt{j}$ times its length. Setting $j\sqrt{f_m}\,\|\bar g_D\|=\sqrt{j}\,s\sqrt{m}$ and solving for j gives Equation 43.18.

□

A projection trades bias for variance, and the public data fix the exchange rate. The noise falls by $\sqrt{p/m}$ and the signal by $\sqrt{f_m}$. If the private gradients lie near the public subspace, f_m is close to 1 for $m\ll p$ and the trade is excellent. If the public data are unrelated, $f_m\approx m/p$ as for a random subspace, and the projection removes as much signal as noise. The subspace must come

from public data. One found from the private gradients themselves leaks information about the records, and Lemma 43.2 no longer applies.

A fixed subspace is also a cap, since training can then move only within it. So projected DP-SGD recomputes the subspace as training proceeds, and gradient embedding perturbation pays a little more noise to release the residual. The same concentration explains why full fine-tuning can work: gradients near a low-dimensional subspace come from a loss curved in few directions, which makes d_{eff} small.

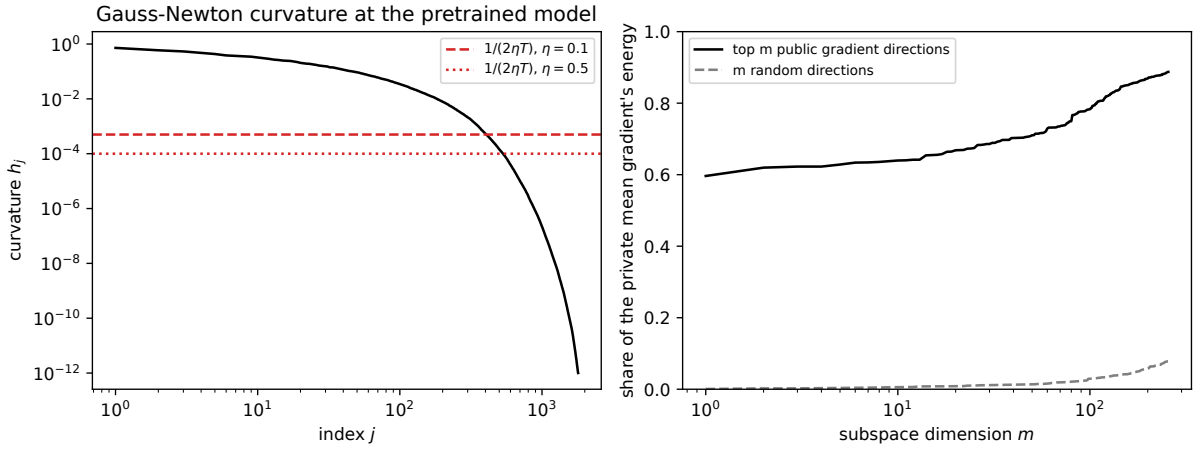


Figure 43.2.: Left: eigenvalues of the Gauss–Newton curvature of the private loss at the pretrained weights, estimated from 2,000 private records, on log scales, with the thresholds $1/(2\eta T)$ for $\eta = 0.1$ and 0.5 and $T = 10{,}000$; directions below a threshold contribute little noise cost. Right: the fraction of the energy of the private average clipped gradient that lies in the top m principal directions of 500 public per-example gradients (solid), and in m random directions (dashed).

Example 43.4. At the pretrained weights the average clipped private gradient has length 0.0130. Table 43.3 gives the share of its energy captured by the top principal directions of 500 clipped public per-example gradients, and by random subspaces of the same dimension.

Table 43.3.: How much of the private average clipped gradient lies in public subspaces, at the pretrained weights, and the steps after which the kept signal outgrows the noise at $\varepsilon = 1$ ($s = 0.0206$).

Dimension m	1	4	16	64	256	3,329 (all)
Public top-m directions, f_m	0.596	0.623	0.655	0.732	0.887	1
m random directions	0.001	0.001	0.004	0.022	0.080	1

Dimension m	1	4	16	64	256	3,329 (all)
Break-even steps at $\varepsilon = 1$, Equation 43.18	4	16	62	222	733	8,455

A single public direction holds 60% of the private gradient's energy, and 64 directions, 2% of the dimensions, hold 73%. At $\varepsilon = 1$ the kept signal outgrows the noise after 222 steps in the 64-dimensional subspace, against 8,455 steps for all weights. The cost is the 27% of the energy outside it, which a fixed projection never learns. The curvature in the left panel of Figure 43.2 tells the same story of few important directions. Its eigenvalues fall from 0.723 to below 5×10^{-4} within the first 406 directions, so a run with $\eta = 0.1$ fills only the directions above that threshold line.

43.5. Pretraining and fine-tuning the running example

The previous sections predict which strategy should win. This section measures how much a related public dataset buys on the running example at each budget, and which part of the model to fine-tune. The public task is built with care, because an identical task would leave nothing to learn privately and an unrelated one would teach nothing.

Definition 43.3 (The public task and pretraining)**.** The private teacher has a first layer W_1^{priv} (8 hidden units, 51 inputs) and an output layer W_2^{priv}. The **public teacher** has first layer $W_1^{\text{priv}} + 0.15Z_1$ and output layer $\cos(0.3)\, W_2^{\text{priv}} + \sin(0.3)\, Z_2$, where Z_1 and Z_2 are fixed standard normal draws. So it has the same hidden units, each slightly perturbed, read out with weights turned by 0.3 radians toward a random vector. It labels 50,000 inputs from the private input distribution. **Pretraining** trains the 64-unit student on them by SGD without privacy, for 10,000 steps with batches of 100 and learning rate 0.5.

Definition 43.4 (The four fine-tuning strategies)**.** From the pretrained weights, DP-SGD with $q = 0.01$, $C = 0.5$ and $T = 10{,}000$ trains one of three sets. **Full** trains all 3,329 weights, **last layer** trains $d = 65$, and **adapter** trains a rank-4 adapter $W_1 = W_1^{\text{pre}} + UV^\top$ on the first layer and the last layer, $d = 525$. The baseline, **scratch**, trains all weights from a random initialization, as in DP-SGD. The private test set of 20,000 records is split in half. Each method's learning rate is chosen from three values on the first half at $\varepsilon = 1$ and used at every budget, and accuracy is reported on the second half.

The four strategies differ in where they start and what they train, but not in what they cost, which is what makes the comparison fair.

Proposition 43.4 (One guarantee for every strategy)**.** *At each budget, the four strategies of Definition 43.4 are (ε, δ)-DP with respect to the private training set, with the same noise multiplier: $\sigma = 7.72$, 4.13, 2.28 and 0.917 for $\varepsilon = 0.5$, 1, 2 and 8 at $\delta = 10^{-5}$.*

Proof.

1. *Fixed starting points.* The pretrained weights and the adapter's random V come from public data and independent coins, so by Lemma 43.2 they are fixed parameters of each private run.
2. *The same accountant.* Each method clips the vector of its trainable coordinates, so each step's clipped sum has sensitivity C and the guarantee of DP-SGD applies. It depends only on q, σ and T, which are the same for every method, so the accountant finds the same σ for each budget.

□

The proposition covers the training set and not the learning-rate choice, which looked at the private validation half without paying for it. Hyperparameter tuning shows how to tune privately. Here the comparison is of methods at their best, as in DP-SGD.

Example 43.5. The public teacher agrees with the private one on 0.9004 of the private test inputs. Pretraining reaches 0.9861 training accuracy on the public task, and 0.9035 on the private task, so it is useful but not sufficient. Table 43.4 shows the learning-rate selection, and Table 43.5 the results.

Table 43.4.: Learning-rate selection at $\varepsilon = 1$, one run each, on the validation half of the private test set.

Method	Learning rates tried, accuracy on the validation half	Chosen
Scratch	0.25: 0.9318; 0.5: 0.9303; 1: 0.8582	0.25
Full	0.03: 0.9559; 0.1: 0.9567; 0.3: 0.9433	0.1
Last layer	0.1: 0.9479; 0.3: 0.9499; 1: 0.9460	0.3
Adapter, rank 4	0.03: 0.9556; 0.1: 0.9619; 0.3: 0.9472	0.1

Table 43.5.: Test accuracy on the report half of the private test set, mean and standard deviation over two runs, $\delta = 10^{-5}$. The pretrained model without fine-tuning has 0.9035. Non-private: SGD for 1,000 steps from scratch and from the pretrained weights, and the exact logistic regression on the frozen features.

	Scratch	Full	Last layer	Adapter, rank 4
Trainable weights d	3,329	3,329	65	525
$\varepsilon = 0.5$	0.8589 ± 0.0075	0.9395 ± 0.0006	0.9478 ± 0.0010	0.9464 ± 0.0005
$\varepsilon = 1$	0.9246 ± 0.0050	0.9599 ± 0.0013	0.9497 ± 0.0000	0.9616 ± 0.0001
$\varepsilon = 2$	0.9518 ± 0.0010	0.9663 ± 0.0001	0.9519 ± 0.0005	0.9653 ± 0.0002

	Scratch	Full	Last layer	Adapter, rank 4
$\varepsilon = 8$	0.9611 ± 0.0012	0.9677 ± 0.0001	0.9514 ± 0.0005	0.9655 ± 0.0001
Non-private	0.9606	0.9704	0.9536	—

Pretraining is worth about a factor of eight in ε. Every fine-tuning method at $\varepsilon = 0.5$ (0.9395 to 0.9478) beats training from scratch at $\varepsilon = 1$ (0.9246). The adapter at $\varepsilon = 1$ (0.9616) matches training from scratch at $\varepsilon = 8$ (0.9611), and full fine-tuning at $\varepsilon = 2$ (0.9663) beats it.

Which weights to train depends on the budget. At $\varepsilon = 0.5$ the last layer is best, because its noise is $\sqrt{3{,}329/65} = 7.2$ times shorter than full fine-tuning's and its ceiling, 0.9536, is not yet binding. From $\varepsilon = 1$ on, it is stuck near that ceiling (0.9497 to 0.9519), while the adapter and full fine-tuning keep improving, since they can correct the features. The adapter is never more than 0.0022 below the best method, so the middle choice is the robust one. Full fine-tuning is behind at $\varepsilon = 0.5$, by 0.0083, but best at $\varepsilon = 2$ and 8, as Li et al. (2022) found for language models.

The learning rate matters as much as the method. In Table 43.4 full fine-tuning loses 1.3 percentage points at a learning rate three times too large, and the adapter 1.5 points. Their best learning rate, 0.1, is 2.5 times smaller than for training from scratch, which is Theorem 43.1 at work: a larger η fills more flat directions with noise and undoes what pretraining provided.

The adapter's rank matters less. At $\varepsilon = 1$, two runs each give 0.9590 ± 0.0038 at rank 1 ($d = 180$), 0.9624 ± 0.0023 at rank 4 ($d = 525$) and 0.9586 ± 0.0031 at rank 16 ($d = 1{,}905$). A higher rank follows more of the directions the task needs but adds noise, and over a sixteenfold range of ranks neither force wins: the differences are about the size of the run-to-run spread. Rank 4's mean over these two seeds, 0.9624, even differs from the 0.9616 of Table 43.5 by about as much, so differences between pretrained methods smaller than about 0.005 should not be read as rankings.

In the single runs of Figure 43.3 the order of the pretrained methods at $\varepsilon = 0.5$ and 1 changes, as the run-to-run spread predicts.

43.6. What public data does not buy

Everything so far treated the public data as free, and formally that is right. But "public" is a label someone chose, and the guarantee says nothing about the records under that label. Tramèr, Kamath & Carlini (2024) argue that the field's reliance on web-scraped pretraining data needs scrutiny. Before you report a guarantee for a fine-tuned model, you need to know exactly what it covers.

Theorem 43.2 (What the guarantee covers)**.** *Let $M(D_{pub}, D)$ pretrain on D_{pub} without privacy and fine-tune on D with an (ε, δ)-DP algorithm whose randomness is independent of the pretraining. Then for every fixed D_{pub}, the mechanism $D \mapsto M(D_{pub}, D)$ is (ε, δ)-DP. There is no guarantee of any kind with respect to D_{pub}. A person with a record x in both datasets is protected only for the copy in D, while the copy in D_{pub} can be revealed completely.*

Proof.

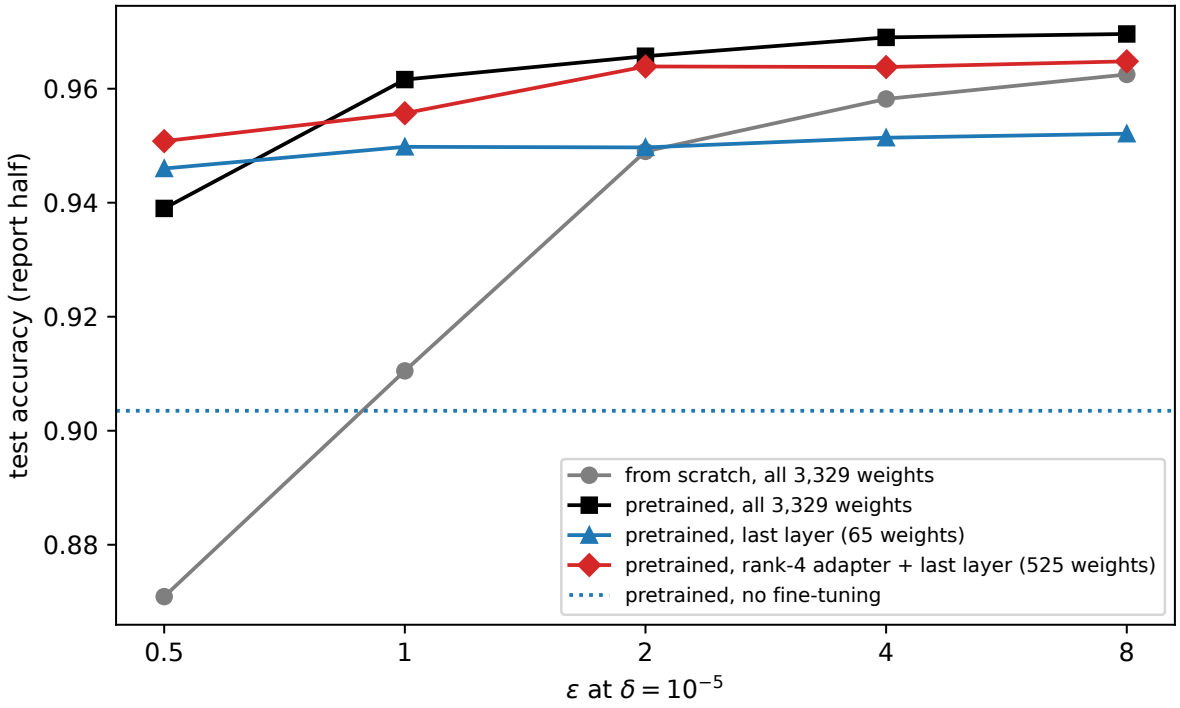


Figure 43.3.: Test accuracy of the four strategies against ε at $\delta = 10^{-5}$, one run each with the learning rates of Table 43.4 and a different seed from Table 43.5, with the pretrained model's accuracy before fine-tuning (dotted). The pretrained methods are far ahead of training from scratch at small budgets; the last layer stays near its ceiling from $\varepsilon = 1$ on, and training from scratch passes it at larger budgets.

1. *Fixed public data.* With D_{pub} fixed, pretraining produces weights θ_0, or a distribution over them, that do not depend on D. Fine-tuning is (ε, δ)-DP for every fixed θ_0, so Lemma 43.2 gives the first claim.
2. *Nothing for the public data.* Changing one record of D_{pub} can change θ_0 arbitrarily, and the fine-tuned model inherits whatever θ_0 encodes. Training without privacy can memorize individual records (Memorization and extraction), and no noise was calibrated to that change.
3. *A record in both.* DP compares D with D without x, with D_{pub} the same in both. So a test of whether x was in D obeys Theorem 18.3: its true positive rate is at most e^{ε} times its false positive rate plus δ. A test of whether x is in the training data at all can use the copy in D_{pub}, which nothing bounds.

□

The first gap is overlap. Web-scale "public" datasets contain personal data, such as posts, profiles and leaked documents. A private set of medical notes or emails may overlap with them, and the people in both are not protected at all. Deduplicating against the public data protects copies, not people, since near-duplicates and paraphrases survive exact matching. Public is not the same as non-sensitive either, since a model can link public records in ways their publication did not allow. And large models are often fine-tuned by whoever has the compute, a trust question DP does not address (Trust models).

The second gap is distribution shift. Public data help as much as they resemble the private task. Benchmarks whose "private" set is close to well-curated public data can overstate what the method does on genuinely sensitive data, such as medical images, rare languages or internal logs. Table 43.5 used a public task close to the private one by design. The gain shrinks as the public teacher drifts away, and for the last layer it turns into a loss against training from scratch. So when you report ε, say what the public data are, whether overlap with the private data was checked, and that the guarantee covers the private data given the pretrained model.

Example 43.6. Take the guarantee at $\varepsilon = 1$ and $\delta = 10^{-5}$. Any test of whether a record was in the private fine-tuning set that has false positive rate 0.01 has true positive rate at most $e \times 0.01 + 10^{-5} = 0.0272$. If the same record is also in the public set, a membership test against the pretrained model, such as the likelihood-ratio attack of Membership inference, is bounded by nothing. It may still fail in practice, but no theorem says it will. If instead the record appears twice in the private set, group privacy (Theorem 19.2) bounds the pair by $(2\varepsilon, 2e^{\varepsilon}\delta) = (2, 5.44 \times 10^{-5})$, a weaker but real guarantee. So a copy in the public set is worse than a duplicate in the private set.

43.7. Code

The script code/public_data.py builds the public task, pretrains the student, implements the last-layer and low-rank-adapter versions of DP-SGD with their per-example gradients (ghost norms included), estimates the curvature and the public gradient subspace, and runs the comparison. It takes about four minutes on a laptop. From the repository root:

```
uv run python code/part05-dp-ml/public_data.py
```

Table 43.6.: The script's output with the default seed.

Experiment	Result
Public task	Teacher agreement 0.9004; pretrained student 0.9035 on the private task
Noise length at $\varepsilon = 1$	1.191, 0.473 and 0.166 for $d = 3{,}329$, 525 and 65
Signal at the pretrained weights	0.0130, 0.0207 and 0.0250; break-even 8,455, 523 and 44 steps
Curvature	Largest eigenvalue 0.723, trace 16.310; $d_{\text{eff}} = 451$ against 60.2 for the last layer at $\eta = 0.1$
Public subspace	60% of the private gradient's energy in one direction, 73% in 64
Adapter ghost norms	Agree with explicit gradients to 3.3×10^{-16}
Fine-tuning, $\varepsilon = 0.5$	Last layer 0.9478, adapter 0.9464, full 0.9395, scratch 0.8589
Fine-tuning, $\varepsilon = 1$	Adapter 0.9616, full 0.9599, last layer 0.9497, scratch 0.9246

tests/test_public_data.py checks that the output still matches.

43.8. Summary

Table 43.7.: The results of this chapter, and where each is used.

Result	Statement	Use
Lemma 43.1	Squared length ds^2; a projection keeps m independent coordinates	Noise length $\sigma C\sqrt{d}/B$: count the trainable weights first
Theorem 43.1	Cost $\approx \frac{\eta s^2}{4} d_{\text{eff}}$, $d_{\text{eff}} \le \min(d, 2\eta T \operatorname{tr} H)$, on a quadratic	Think in effective dimension: noise where the loss is flat is cheap; keep η small
Lemma 43.2	A computation on public data is a fixed parameter of the mechanism	Pretraining and tuning on public data cost nothing
Proposition 43.1	Convex ERM with $L \le \sqrt{d}$; feature gap plus privacy cost of order $\sqrt{d}/(n\varepsilon)$	Best at very small ε; measure its ceiling on public data that resemble the private task

Result	Statement	Use
Proposition 43.2	$r(d_{\text{in}} + d_{\text{out}})$ weights; outer-product gradients; first step of rank r	At moderate budgets, the robust middle choice
Proposition 43.3	Same (ε, δ); break-even $(s\sqrt{m}/(\sqrt{f_m}\|\bar{g}_D\|))^2$; bias $1 - f_m$	Projected DP-SGD; the subspace must come from public data
Proposition 43.4	Every strategy has the σ of DP-SGD with the same q and T	A related public model is worth about a factor of eight in ε here
Theorem 43.2	(ε, δ)-DP in D for every fixed D_{pub}; nothing for D_{pub}	Check for overlap, and say what the public data are when you report ε

43.9. References

- Tramèr & Boneh, "Differentially Private Learning Needs Better Features (or Much More Data)" (ICLR 2021). Linear models on handcrafted ScatterNet features beat end-to-end private networks at moderate budgets; features from public data do better. The source of Section 43.2.
- Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang & Chen, "LoRA: Low-Rank Adaptation of Large Language Models" (ICLR 2022). The adapter of Section 43.3, without privacy.
- Yu, Naik, Backurs, Gopi, Inan, Kamath, Kulkarni, Lee, Manoel, Wutschitz, Yekhanin & Zhang, "Differentially Private Fine-tuning of Language Models" (ICLR 2022). Private fine-tuning with LoRA and other parameter-efficient methods; reports, for example, 87.8% accuracy on a language-inference benchmark at $\varepsilon = 6.7$ against 90.2% without privacy, for a large RoBERTa model.
- Li, Tramèr, Liang & Hashimoto, "Large Language Models Can Be Strong Differentially Private Learners" (ICLR 2022). Full fine-tuning with the right hyperparameters matches parameter-efficient methods; ghost clipping for sequence models.
- Li, Liu, Hashimoto, Inan, Kulkarni, Lee & Thakurta, "When Does Differentially Private Learning Not Suffer in High Dimensions?" (NeurIPS 2022). Dimension-free bounds when gradients concentrate in few directions, and the measurement that fine-tuning gradients are dominated by a few principal components. The background of Section 43.1.
- Zhou, Wu & Banerjee, "Bypassing the Ambient Dimension: Private SGD with Gradient Subspace Identification" (ICLR 2021). Projected DP-SGD with a subspace estimated from public gradients.
- Yu, Zhang, Chen & Liu, "Do Not Let Privacy Overbill Utility: Gradient Embedding Perturbation for Private Learning" (ICLR 2021). Projection onto a public anchor subspace plus a separately noised residual.
- Tramèr, Kamath & Carlini, "Position: Considerations for Differentially Private Learning with Large-Scale Public Pretraining" (ICML 2024). What the public-pretraining paradigm does and does not protect, and how benchmarks can mislead. The source of Section 43.6.

- Bassily, Smith & Thakurta, "Private Empirical Risk Minimization: Efficient Algorithms and Tight Error Bounds" (FOCS 2014). The rates for private convex ERM used in Section 43.2.
- De, Berrada, Hayes, Smith & Balle, "Unlocking High-Accuracy Differentially Private Image Classification through Scale" (arXiv 2022). Private fine-tuning of image models pretrained on large public datasets, among other practices.

44. User-level differential privacy

Every differential privacy (DP) guarantee in this part so far protects one record. Stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, DP-SGD, reports $\varepsilon = 6.71$ at $\delta = 10^{-5}$ for the part's running example. So an adversary who sees the trained model learns little about whether any single training example was used. But people do not contribute one record each. A user of a messaging app sends hundreds of messages and a patient makes dozens of visits, and what an adversary learns about all of a person's records together can be much more than what it learns about one. This chapter asks what a record-level ε says about a person who contributed k records, and how to train a model with a guarantee for the person instead.

The one idea is that the neighbouring relation is the privacy unit. A DP guarantee compares the outputs on two neighbouring datasets, and whatever differs between neighbours is what is protected. If neighbours differ in one record, records are protected, and if they differ in all of one person's records, people are. That leaves two routes to a guarantee for people. The first keeps the record as the unit and pays for the person afterwards, by group privacy or by an accountant that knows the person has k records. The second makes the person the unit from the start. It samples people, clips each person's total contribution, and runs the usual accountant with people in place of records.

The running example is the part's network, with one hidden layer of 64 tanh units and 3,329 weights. It learns labels produced by a fixed teacher network from 10,000 training records of 50 features, and is tested on 20,000 more. This chapter assigns the records to $U = 1{,}000$ users. The user sizes are drawn first, giving every user one record and spreading the other 9,000 in proportion to log-normal weights. So sizes are very uneven, with a mean of 10 records, a median of 4 and a largest user of 348 (Figure 44.1). Each user then receives a nearest-neighbour cluster of records of that size around a random record, so their records resemble each other the way one person's messages do. By the direct accountant of Section 44.2, the record-level model protects a user with two records only at $\varepsilon = 15.54$, and a user with ten at $\varepsilon = 146.20$. Training with the user as the unit instead reaches a test accuracy of 0.9347 at user-level $\varepsilon = 4$, against 0.9612 without privacy.

Table 44.1.: The symbols of this chapter.

Symbol	Meaning
U, n_u	the number of users; the number of records of user u
k	the number of records of one user, or the cap on it

Symbol	Meaning
q, σ, C, T, B	the sampling rate, noise multiplier, clipping norm, number of steps and expected batch size of DP-SGD, counted in whichever unit is sampled
H, b_m	the number of a user's records in a Poisson batch, $H \sim \mathrm{Bin}(k, q)$, and its probabilities
P_0, $P_{(k)}$	one step's output without and with the user, in units of C
α, A_α	a Rényi order; the Rényi moment, the average of the α-th power of a likelihood ratio
$\varepsilon_{q,\sigma}(\alpha)$, $\varepsilon_k(\alpha)$	the Rényi differential privacy (RDP) curve of one step for one record, and for a user with k records
$\bar{g}_u$, $\tilde{g}_u$	user u's vector, such as their average gradient, and its clipped version
n_k	the number of records kept by a cap of k per user

44.1. The privacy unit

A model trained with record-level DP-SGD at $\varepsilon = 6.71$ is released, and one person in its training set contributed k of its records. This section asks what the guarantee promises that person. The question has to come first, because a deployment cannot say "your data are protected" until it knows. And the answer depends on a neighbouring relation that the engineer chose, not the person.

The definition of DP never says what a neighbour is, so a guarantee for people is the same definition with a coarser relation.

Definition 44.1 (Example-level and user-level DP)**.** Let each record of a dataset belong to one user. Two datasets are **example-level neighbours**, $D \sim D'$, if they differ by adding or removing one record. They are **user-level neighbours** if they differ by adding or removing all the records of one user. A mechanism M is **user-level** (ε, δ)-DP if for every pair of user-level neighbours D and D' and every event S,

$$\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{44.1}$$

It is **example-level** (ε, δ)-DP if the same holds for every pair of example-level neighbours.

User-level neighbours are a few example-level steps apart, so an example-level guarantee does say something about users. The tool is group privacy, Theorem 19.2 in Post-processing, group privacy and composition. For two datasets joined by a path of k neighbouring steps, it says that the guarantee holds with $k\varepsilon$ in place of ε, while the slacks δ pile up in a geometric sum.

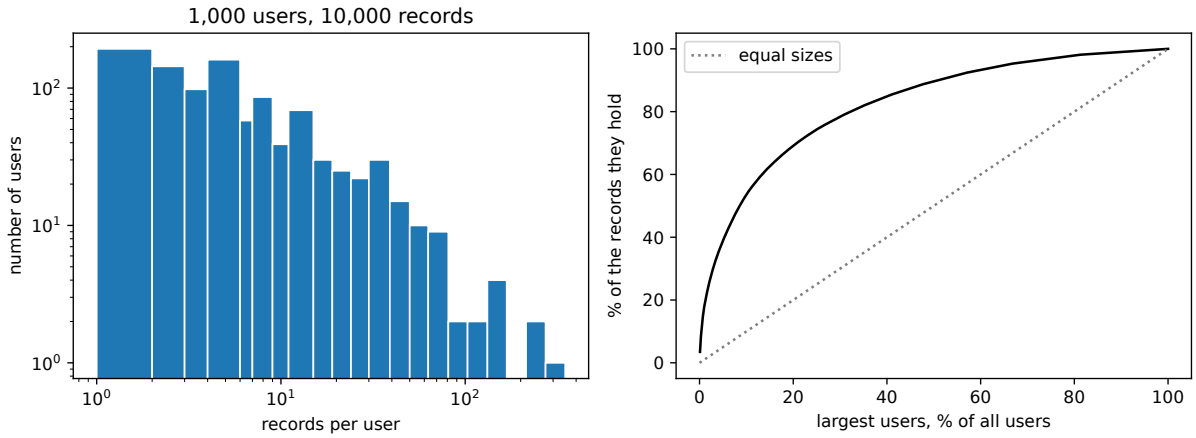


Figure 44.1.: Left: the number of records per user in the running example's population of 1,000 users, on log–log axes; 193 users have one record and the largest has 348. Right: the share of all records held by the largest users. The largest 10 users hold 17.9% of the records and the smallest half of the users 10.3%.

Proposition 44.1 (A record-level guarantee read for users)**.** *Let M be example-level (ε, δ)-DP. Then M is user-level $(k\varepsilon, \delta_k)$-DP for users with at most k records, where*

$$\delta_k = \delta \sum_{j=0}^{k-1} e^{j\varepsilon} = \delta \, \frac{e^{k\varepsilon} - 1}{e^{\varepsilon} - 1}. \tag{44.2}$$

No better bound holds for every example-level (ε, δ)-DP mechanism.

Proof.

1. *User-level neighbours are joined by a short path.* Let D' be D plus the k records $x_1, \dots, x_k$ of one user. Adding them one at a time gives the datasets $D, D \cup \{x_1\}, \dots, D'$, each an example-level neighbour of the next. A user with fewer records gives a shorter path, and both $k\varepsilon$ and δ_k grow with k, so the bound covers them too.
2. *Apply group privacy along the path.* Theorem 19.2, applied to this path, gives the bound with $k\varepsilon$ and δ_k. The path can be walked in either direction, so the bound holds both ways.
3. *Nothing better holds in general.* Proposition 19.1 builds an (ε, δ)-DP mechanism, the alarm mechanism, that meets Equation 44.2 with equality on one event when δ is small enough. A guarantee known only as "(ε, δ)-DP" could come from that mechanism.

□

User-level DP is not a new privacy notion. It is DP with a coarser neighbouring relation, and the proofs of post-processing, composition and the accountants of Part 4 hold for any fixed relation, so they all hold for it. What must be recomputed is the sensitivity, how far one unit can move the computation. The unit should follow the question "whose participation do I want to hide?", and Choosing an accountant lists the unit among the things a guarantee must report. For RDP the step from records to groups is weaker than for (ε, δ), because Theorem 29.4 halves the order and triples ε each time the group doubles.

The price of group privacy grows linearly in k for ε and exponentially in k for δ. For pure DP a user with k records gets $k\varepsilon$, which is already weak. At $\varepsilon = 1$ per record, a user with 10 records gets $\varepsilon = 10$, a likelihood ratio of $e^{10} \approx 22{,}000$. With $\delta > 0$ it is worse, because Equation 44.2 multiplies each δ by up to $e^{(k-1)\varepsilon}$. To keep δ_k at 10^{-5}, the per-record δ must be tiny, and a smaller δ costs a larger per-record ε. DP-SGD's accountant can convert its curve at any δ, so this trade can be made, but it is a race between a falling δ and a rising $e^{(k-1)\varepsilon}$. For small k the falling δ wins. For larger k the rising ε wins at every δ, and group privacy gives no $(\varepsilon, 10^{-5})$ guarantee at all.

Example 44.1. The running example's model is example-level $(6.71, 10^{-5})$-DP. Read naively through Proposition 44.1, a user with two records gets $\varepsilon = 2 \times 6.71 = 13.42$ with $\delta_2 = 10^{-5}(1 + e^{6.71}) = 0.0082$. That is a probability of almost one in a hundred that the guarantee fails outright, which is no guarantee. Re-converting the accountant's curve at a smaller per-record δ does better. A search over per-record δ, each with its own per-record ε, finds the one that makes $\delta_2 = 10^{-5}$. Its per-record ε is 9.30, so the user gets $(18.61, 10^{-5})$. For five records the same procedure gives a per-record ε of 23.62 and a user-level ε of 118.12. For ten records it gives nothing, because no per-record δ makes δ_{10} as small as 10^{-5}. In the running example's population, 244 of the 1,000 users have at least ten records, and 30 have at least 50.

44.2. A user with k records in example-level DP-SGD

Group privacy treats DP-SGD as a black box, and Example 44.1 shows what that costs. But we know exactly how a user with k records enters each step of DP-SGD. Each of the user's records is drawn into the batch independently with probability q, so each step sees a random number of them, between 0 and k. This section computes the user's guarantee directly from that structure and asks how fast it grows with k.

The record-level accountant is the starting point. Amplification by subsampling compares, in units of C, the output without a record, $P_0 = \mathcal{N}(0, \sigma^2)$, with the mixture $(1-q)P_0 + q\mathcal{N}(1, \sigma^2)$. It shows that the step's RDP curve is $\varepsilon_{q,\sigma}(\alpha) = \ln A_\alpha/(\alpha - 1)$, where at integer orders the Rényi moment is the binomial sum

$$A_\alpha = \sum_{j=0}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha-j} q^j \exp\Big(\frac{j^2 - j}{2\sigma^2}\Big). \tag{44.3}$$

RDP curves add over the T steps, even when each step depends on earlier outputs (Theorem 29.2). DP-SGD converts the total to (ε, δ) at the best order:

$$\varepsilon = \min_{\alpha} \Big[T\varepsilon_{q,\sigma}(\alpha) + \ln \frac{\alpha - 1}{\alpha} - \frac{\ln \delta + \ln \alpha}{\alpha - 1}\Big]. \tag{44.4}$$

A user enters a step in the same way, except that the shift is no longer 0 or 1. It is the number of the user's records that are drawn, which is binomial. The worst case is when all the drawn records push the sum by the full C in the same direction.

Theorem 44.1 (The worst case for a user with k records (Charles et al. 2024, Theorem 3))**.** *Run example-level DP-SGD with Poisson sampling rate q, clipping norm C and noise multiplier σ, on*

a dataset in which one user owns k records. Let $H \sim \mathrm{Bin}(k, q)$ be the number of the user's records in a batch, with probabilities $b_m = \binom{k}{m} q^m (1-q)^{k-m}$. In units of C, compare

$$P_0 = \mathcal{N}(0, \sigma^2) \qquad \text{and} \qquad P_{(k)} = \sum_{m=0}^{k} b_m \, \mathcal{N}(m, \sigma^2). \tag{44.5}$$

Then the T steps satisfy, for this user, the RDP bound $T\varepsilon_k(\alpha)$ at every order $\alpha > 1$, with

$$\varepsilon_k(\alpha) = \max\left\{ D_\alpha(P_{(k)} \| P_0),\ D_\alpha(P_0 \| P_{(k)}) \right\}. \tag{44.6}$$

Some loss whose gradients are aligned attains this curve.

Proof. The plan is to count the user's records in a step, find what they do to the noisy sum, and then cite the step that shows the aligned case is the worst.

1. *How many records are drawn.* Poisson sampling draws each record independently with probability q. So the number of the user's k records in the batch is a sum of k independent coins, $H \sim \mathrm{Bin}(k, q)$, independent of which other records are drawn.
2. *What they do to the sum.* Fix the other records' contribution, which is the same with and without the user. Given $H = m$, the user adds m clipped gradients of length at most C, so their sum has length at most mC, with equality when they all point the same way. Along that direction and in units of C, the output is then $\mathcal{N}(m, \sigma^2)$ with probability b_m, which is the mixture $P_{(k)}$.
3. *The aligned case is the worst.* That no other configuration of the user's gradients, in any dimension, gives a larger divergence is Theorem 3 of Charles et al. (2024), which also justifies the reduction to one dimension. Section M.4 in the appendices proves it for $D_\alpha(P_{(k)} \| P_0)$ at integer orders. The reason is that gradients pointing in different directions add in squares rather than in length, so they shift the sum less.
4. *Compose.* RDP curves add over the T steps by Theorem 29.2, which gives $T\varepsilon_k(\alpha)$.

□

The divergence $D_\alpha(P_{(k)} \| P_0)$ has a closed form at integer orders, which checks the numerical accountant and shows where a heavy user's cost comes from. The idea is that the α-th power of a mixture over m becomes an average over α independent copies of H. The last step of the proof needs the moment generating function of a normal variable, $\mathbb{E}[e^{sY}] = e^{s^2\sigma^2/2}$ for $Y \sim \mathcal{N}(0, \sigma^2)$, which Proposition 1.10 proves.

Proposition 44.2 (The Rényi moment of a user with k records)**.** *Let $\alpha \geq 2$ be an integer, and let $H_1, \ldots, H_\alpha$ be independent copies of $H \sim \mathrm{Bin}(k, q)$. Then $D_\alpha(P_{(k)} \| P_0) = \ln A_\alpha / (\alpha - 1)$, with*

$$A_\alpha = \mathbb{E}\Big[\exp\Big(\frac{1}{\sigma^2} \sum_{1 \leq i < j \leq \alpha} H_i H_j \Big) \Big]. \tag{44.7}$$

For $k = 1$ this is the binomial sum Equation 44.3.

Proof. The plan is to write the divergence as an average of a power of the likelihood ratio, turn the power into a product over independent copies, and then average over the output.

1. *The divergence as an average.* By the definition of the Rényi divergence, $A_\alpha = e^{(\alpha-1)D_\alpha(P_{(k)}\|P_0)}$ is the average of $(P_{(k)}(y)/P_0(y))^\alpha$ over $y \sim P_0$.
2. *The ratio of two Gaussians.* Two normal densities with the same variance differ only in the square in their exponents. So for $P_m = \mathcal{N}(m, \sigma^2)$ the ratio is the exponential of a linear function of y:
$$\frac{P_m(y)}{P_0(y)} = \exp\Big(\frac{-(y-m)^2 + y^2}{2\sigma^2}\Big) = \exp\Big(\frac{2my - m^2}{2\sigma^2}\Big). \tag{44.8}$$
3. *The mixture's ratio is an average over H.* Dividing the mixture by P_0 term by term with Equation 44.8 gives
$$\frac{P_{(k)}(y)}{P_0(y)} = \sum_m b_m \exp\Big(\frac{2my - m^2}{2\sigma^2}\Big) = \mathbb{E}_H\Big[\exp\Big(\frac{2Hy - H^2}{2\sigma^2}\Big)\Big]. \tag{44.9}$$
4. *A power of an average is an average over independent copies.* Multiplying out α copies of a sum $\sum_m b_m f(m)$ gives one term for each choice $(m_1, \dots, m_\alpha)$, with weight $b_{m_1} \cdots b_{m_\alpha}$. That weight is the joint distribution of independent copies. A product of exponentials is the exponential of the sum, so with $V = H_1 + \cdots + H_\alpha$ the total over the copies,
$$\Big(\frac{P_{(k)}(y)}{P_0(y)}\Big)^\alpha = \mathbb{E}\Big[\exp\Big(\frac{2Vy - \sum_i H_i^2}{2\sigma^2}\Big)\Big]. \tag{44.10}$$
5. *Average over the output.* Everything is positive, so the average over y and the average over the copies can be exchanged. For fixed copies, the normal moment generating function at $s = V/\sigma^2$ gives $\mathbb{E}_{y\sim P_0}[e^{Vy/\sigma^2}] = e^{V^2/(2\sigma^2)}$, so
$$A_\alpha = \mathbb{E}\Big[\exp\Big(\frac{V^2 - \sum_i H_i^2}{2\sigma^2}\Big)\Big]. \tag{44.11}$$
6. *Only products of different copies remain.* Expanding the square gives $V^2 = \sum_i H_i^2 + 2\sum_{i<j} H_i H_j$. The squares cancel, which leaves Equation 44.7.
7. *One record.* For $k = 1$ each copy is 0 or 1. If J of the α copies are 1, the sum $\sum_{i<j} H_i H_j$ counts the pairs among them, $(J^2 - J)/2$, and $J \sim \mathrm{Bin}(\alpha, q)$. Averaging over J gives Equation 44.3.

□

The closed form also gives a quick estimate. When kq is small, most copies are 0, and a light user should look like a single record that is drawn whenever any of the user's records is.

Corollary 44.1 (A light user looks like one record sampled at rate kq)**.** *For an integer order α and small kq, to leading order in kq,*
$$\varepsilon_k(\alpha) \approx \frac{\alpha}{2}(kq)^2(e^{1/\sigma^2} - 1), \tag{44.12}$$
which is the curve of one record sampled at rate kq to the same order.

Proof. The plan is to find the first term of Equation 44.7 that costs privacy, and then compare it with one record.

1. *Terms with at most one non-zero copy are free.* For them $\sum_{i<j} H_i H_j = 0$, so they contribute their probability times 1. All the probabilities add to 1, so only the excess of the other factors over 1 counts.
2. *The first costly term.* The most likely way for a product to be non-zero is for exactly two copies to equal 1. There are $\binom{\alpha}{2}$ pairs of copies, each copy is 1 with probability $kq(1-q)^{k-1} \approx kq$ for small q, and the factor is e^{1/σ^2}, so

$$A_\alpha \approx 1 + \binom{\alpha}{2} (kq)^2 (e^{1/\sigma^2} - 1). \tag{44.13}$$

3. *Compare with one record.* This is the first three terms of Equation 44.3 with q replaced by kq.
4. *Take the logarithm.* Using $\ln(1+x) \approx x$ and dividing by $\alpha - 1$ gives Equation 44.12.

□

For non-integer orders, and for the reverse direction $D_\alpha(P_0 \| P_{(k)})$, this chapter's script integrates numerically instead.

So in the regime where amplification works well, only the probability that some record of the user is in the batch matters, about kq. Halving the sampling rate compensates for doubling the user's records. The per-step curve grows like $(kq)^2$, and the converted ε roughly like its square root, so the user's ε grows roughly linearly in k. Beyond that regime it grows faster. For $k = 50$ and $q = 0.01$, half of all steps include one of the user's records, and subsampling stops helping, because it helps only while the unit is usually absent. And the higher terms of Equation 44.7 grow like $e^{H_i H_j/\sigma^2}$, with $H_i H_j$ as large as k^2. A step that draws all k records acts like a mechanism with noise multiplier σ/k, and the accountant retreats to orders close to 1, where the conversion is expensive.

Group privacy is the wrong tool for DP-SGD, because it ignores the structure of the mechanism. Example 44.2 shows that from two records on, the direct accountant gives a smaller ε, and a finite one for every k. The direct accountant assumes the worst case over gradients, all of them aligned, and one might hope real users are milder. But a user's records resemble each other, so their gradients do partly align, and Theorem 44.1 says some loss attains the bound. An auditor should therefore treat it as the guarantee rather than a pessimistic estimate.

Example 44.2. Table 44.2 applies the accountant of Theorem 44.1 to the running example's DP-SGD run, with $q = 0.01$, $\sigma = 1$, $T = 10{,}000$ and $\delta = 10^{-5}$.

Table 44.2.: The user-level ε at $\delta = 10^{-5}$ of the running example's example-level DP-SGD, for a user with k records. "None" means that group privacy gives no $(\varepsilon, 10^{-5})$ guarantee.

Records of the user k	1	2	5	10	50
User accountant, Equation 44.5	6.72	15.54	52.36	146.20	2161.43
Best order α	4	2.5	1.65	1.3	1.05

Records of the user k	1	2	5	10	50
Group privacy, Equation 44.2	6.71	18.61	118.12	none	none
One record at rate kq	6.71	15.32	50.11	134.50	1673.08

The first column checks the accountant. With one record it is the DP-SGD accountant, 6.72 against 6.71, and the difference comes from the coarser grid of orders this chapter's script uses (its best order is 4, against 4.1 on the finer grid). At $\alpha = 4.1$ the two curves agree to all printed digits, 3.733633×10^{-4}. For $k = 2$ and 5 at $\alpha = 2$, the numerical integral and Equation 44.7 agree to seven digits, at 6.992273×10^{-4} and 4.626224×10^{-3}. The reverse direction is smaller, at 6.119238×10^{-4} and 3.412123×10^{-3}.

Already at two records, the person behind them is protected only at $\varepsilon = 15.54$, more than twice the record's 6.71. At ten, $\varepsilon = 146.20$ means no protection at all. The rate-kq estimate of Corollary 44.1 is within 1.5% at $k = 2$ (15.32 against 15.54) and 8% at $k = 10$ (134.50 against 146.20). So rerunning the record-level accountant with q replaced by kq is a quick estimate of a user's exposure. In every column it errs on the low side, because it drops the terms in which several of the user's records are drawn together.

44.3. User-level DP-SGD

Accounting for users after the fact charges a heavy user for every one of their records, and Table 44.2 shows the bill. The alternative changes the algorithm, so that the unit DP-SGD samples, clips and accounts for is the user. McMahan, Ramage, Talwar & Zhang (2018) introduced it for federated learning, where each user's device computes an update on its own data. This section states the algorithm and asks what it guarantees.

Definition 44.2 (User-level DP-SGD)**.** Fix U users, a user sampling rate q, a clipping norm C, a noise multiplier σ and a learning rate η, and start from weights θ_0 chosen independently of the data. For $t = 0, \dots, T-1$:

1. *Sample users.* Put each user in the set S_t independently with probability q.
2. *Per-user update.* For each $u \in S_t$, compute one vector $\bar{g}_u$ from user u's records alone. It can be their average gradient $\bar{g}_u = \frac{1}{n_u} \sum_{i \in u} \nabla_\theta \ell(\theta_t; z_i)$, or the change in weights after a few local steps of SGD on those records.
3. *Clip per user.* Set $\tilde{g}_u = \bar{g}_u \min(1, C/\|\bar{g}_u\|)$.
4. *Noisy sum.* Set $G_t = \sum_{u \in S_t} \tilde{g}_u + \xi_t$ with $\xi_t \sim \mathcal{N}(0, \sigma^2 C^2 I)$.
5. *Step.* Set $\theta_{t+1} = \theta_t - \eta\, G_t/(qU)$.

The proof of DP-SGD used four properties of the clipped sum. Each one still holds with users in place of records, so the accountant should not change.

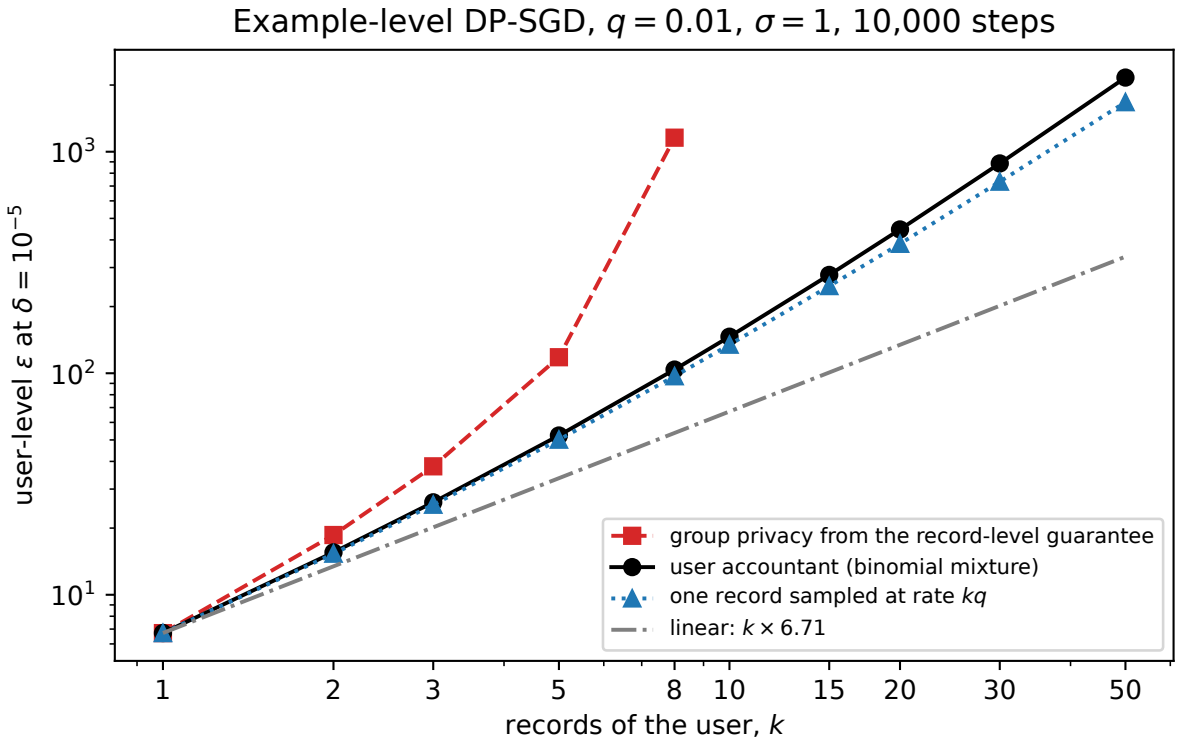


Figure 44.2.: The user-level ε at $\delta = 10^{-5}$ of the running example's DP-SGD ($q = 0.01$, $\sigma = 1$, 10,000 steps; $\varepsilon = 6.71$ per record) for a user with k records. The direct accountant of Equation 44.5 (black) is close to a single record sampled at rate kq (blue, dotted) and grows faster than linearly (grey). Group privacy from the record-level guarantee (red, dashed) is larger, and gives no guarantee at all for $k = 10$ and above.

Theorem 44.2 (User-level DP-SGD keeps the record-level accountant)**.** *In Definition 44.2, suppose each $\bar{g}_u$ is computed from user u's records alone and the number of users U is public. Then releasing $\theta_1, \dots, \theta_T$ is user-level (ε, δ)-DP, with ε given by Equation 44.4 and q the user sampling rate.*

Proof. The plan is to check the four properties that the proof for records uses, one at a time.

1. *One user moves the clipped sum by at most C.* Fix the weights θ_t and the other sampled users. Each $\tilde{g}_v$ is computed from user v's records alone, so adding user u leaves every other term unchanged and adds $\tilde{g}_u$, of length at most C. However many records u has, they enter only through $\tilde{g}_u$.
2. *One step is a subsampled Gaussian mechanism over users.* User u enters S_t with probability q, independently of the other users. So the output with u is a mixture: the same as without u with probability $1 - q$, and shifted by $\tilde{g}_u$ with probability q. After rescaling by C and rotating the shift onto one axis, the worst case is the record-level pair, whose curve is $\varepsilon_{q,\sigma}(\alpha)$.
3. *The division is post-processing.* Dividing by qU uses only public numbers, as dividing by n does in example-level DP-SGD.
4. *Compose and convert.* RDP composition over the T adaptive steps and Equation 44.4 finish the proof.

□

The count that matters is the number of units. User-level DP-SGD has the privacy of example-level DP-SGD on a dataset of U records. To keep an expected batch of B units, its sampling rate must be B/U, larger than B/n by the average number of records per user, so amplification is weaker. On the running example, with 1,000 steps and an expected batch of 100 units, a record-level $\varepsilon = 4$ needs $\sigma = 0.778$ at $q = 0.01$. A user-level $\varepsilon = 4$ needs $\sigma = 3.78$ at $q = 0.1$. With $C = 0.5$, the noise on the averaged update, $\sigma C/B$, is then 0.0189 against 0.00389, about five times larger. That is the honest price of protecting people. There are ten times fewer of them than records, so the sampling rate is ten times larger, and the noise must grow to make up for the weaker amplification.

In return, the records inside a user are free. The guarantee does not depend on n_u, so a user's records can be averaged, used for several local steps or subsampled, as long as the result is one clipped vector computed from that user's data alone. More records per user then improve each $\bar{g}_u$ while the noise stays fixed (Levy et al. 2021). The proof does assume two things. Step 1 rules out any computation that mixes users, such as normalizing features by an average over users, and it fails if one person's records are spread over several accounts, since the unit is then the account and the person is again a group. Step 2 needs users to be Poisson-sampled, and iterating over users in a shuffled order is a different mechanism, as it is for records.

A common variant keeps the code for per-example gradients. It replaces steps 2 and 3 by per-record clipping and a cap: keep at most k records per user, clip each record's gradient to C, and sum them. One user can then move the sum by kC, so the variant should pay for the cap in noise.

Proposition 44.3 (Per-record clipping inside user sampling)**.** *Run Definition 44.2 with users sampled whole at rate q, but let each user's vector be the sum of at most k of their records'*

gradients, each clipped to C. Then each step has the RDP curve of noise multiplier σ/k, whose Rényi moment at integer order α is

$$A_\alpha = \sum_{j=0}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha-j} q^j \exp\Big(\frac{k^2(j^2-j)}{2\sigma^2}\Big). \tag{44.14}$$

Proof.

1. *The user's shift.* The user's vector is a sum of at most k vectors of length at most C, so its length is at most kC. The user is still sampled as a whole with probability q.
2. *The pair.* In units of C, step 2 of the proof of Theorem 44.2 now compares $\mathcal{N}(0,\sigma^2)$ with $(1-q)\mathcal{N}(0,\sigma^2) + q\mathcal{N}(k,\sigma^2)$.
3. *Rescale.* Dividing the output by k is an invertible map, so it changes no divergence. It gives the pair with shift 1 and noise σ/k, whose moment is Equation 44.3 with σ/k in place of σ, which is Equation 44.14.

□

This chapter's script checks Equation 44.14 at $k = 2$ and $\alpha = 3$, where it and the binomial sum at σ/k both give 8.219438×10^{-2}. So clipping each record and summing at most k of them reuses existing code but pays a factor k in noise multiplier. Averaging before clipping costs no extra noise, whatever the user's size, and it gives every user a vector of the same kind whatever their size, which keeps the clipping norm meaningful across users. Summing a user's gradients and then clipping the sum is also valid, with sensitivity C, and Section 44.5 shows that it gives heavy users more weight.

Example 44.3. Table 44.3 gives the noise multiplier for four user-level budgets, with 1,000 steps and an expected batch of 100 sampled units. The column "record-level" is the noise that protects only records. Column (a) is example-level DP-SGD on all 10,000 records at $q = 0.01$, accounted by Theorem 44.1 for the largest user, with $k = 348$. Column (b) is user-level DP-SGD at $q = 0.1$. Column (c) keeps at most 2 records per user, 1,807 records in all, and runs example-level DP-SGD on them at $q = 100/1{,}807 = 0.0553$, accounted for $k = 2$ (Section 44.4).

Table 44.3.: Noise multipliers for 1,000 steps with an expected batch of 100 units, at $\delta = 10^{-5}$.

User-level ε	Record-level (not user-level)	(a) All records, $k = 348$	(b) User-level DP-SGD	(c) At most 2 records per user
1	1.52	449	12.9	14.4
2	1.03	237	6.89	7.62
4	0.778	128	3.78	4.18
8	0.616	70.3	2.18	2.4

Column (a) is the cost of keeping the record as the unit when one user has 348 records. In each step this user has on average $348 \times 0.01 = 3.48$ records in the batch, so subsampling hardly hides them, and the noise must cover a shift of several clipping norms in nearly every step. At $\varepsilon = 4$ the noise on the averaged gradient has standard deviation $128 \times 0.5/100 = 0.64$ per coordinate. Its length is then about $0.64 \times \sqrt{3{,}329} = 36.9$, against a clipped average of length at most 0.5.

44.4. Contribution bounding

User-level DP-SGD needs a data pipeline that groups records by user and computes one vector per user. Many training pipelines are built around records instead, with shuffled files of examples, per-example gradient code and fixed-length batches. This section asks whether you can keep such a pipeline and still protect users. You can, if you limit how many records each user contributes, and the question becomes what limit to choose.

Definition 44.3 (Contribution bounding)**. Contribution bounding** with cap k keeps at most k records of each user and discards the rest. The records kept for user u are chosen from u's own records, with randomness independent of everything else, for example uniformly at random. The bounded dataset has $n_k = \sum_u \min(n_u, k)$ records.

After bounding, a user-level neighbour is a group of at most k records of one user, and Theorem 44.1 already accounts for exactly that.

Theorem 44.3 (Bounding gives a user-level guarantee)**.** *Run example-level DP-SGD with sampling rate q on the dataset bounded at cap k. The result is user-level (ε, δ)-DP, with ε given by Equation 44.4 applied to the curve $T\varepsilon_k(\alpha)$ of Theorem 44.1 for a user with k records.*

Proof.

1. *Bounding acts on each user separately.* Write the bounded dataset as the union over users of $\pi(D_u)$, where D_u is user u's records and π keeps at most k of them. Adding user u adds $\pi(D_u)$ and changes nothing else, because π looks only at u's records and uses independent randomness. So user-level neighbours become datasets that differ in at most k records, all of one user. A rule that looked at other users' data, such as keeping the records most similar to the rest of the dataset, would break this step.
2. *Account for those records.* Example-level DP-SGD draws each of the user's at most k records independently with probability q, so Theorem 44.1 gives the pair Equation 44.5 for one step. A user with fewer records is a k-record user some of whose clipped gradients are 0, and the worst case of Theorem 44.1 covers every gradient of length at most C, including 0.
3. *Compose and convert,* as in Theorem 44.1 and Equation 44.4.

□

The cap trades the noise the accountant requires against the data you throw away. Both sides of the trade follow from results above, the noise from Corollary 44.1 and the data from how the expected update weights each record.

Proposition 44.4 (What the cap costs)**.** *Under contribution bounding with cap k, two things change as k grows. When kq is small and σ large, holding the user-level budget fixed at a fixed q requires a noise multiplier that grows in proportion to kq. And the expected update gives user u weight $\min(n_u, k)/n_k$ instead of n_u/n, so every user with more than k records loses weight.*

Proof.

1. *The noise.* For large σ, $e^{1/\sigma^2} - 1 \approx 1/\sigma^2$, so the per-step curve of Equation 44.12 becomes about $\frac{\alpha}{2}(kq/\sigma)^2$. It depends on k, q and σ only through kq/σ, so holding it fixed means $\sigma \propto kq$. Doubling k then requires either halving q, which halves the batch, or doubling σ.
2. *The weights.* Before bounding, the expected update of DP-SGD averages over all n records, each with weight $1/n$. After it, the average is over the n_k kept records, of which user u has $\min(n_u, k)$. A user with $n_u > k$ therefore loses weight, and their records beyond k are not used at all.

□

Choosing k is a bias–variance trade-off. Amin, Kulesza, Muñoz Medina & Vassilvitskii (2019) studied this choice for private empirical risk minimization, and showed that the best cap depends on the distribution of user sizes. A cap above most users' sizes buys little data at a large noise cost, and a cap below them throws away most of the data, the same trade as choosing a clipping norm. The cap depends on the data, so it should come from public knowledge of the population or be chosen privately. Choosing it by trying several caps and reading test accuracy, as the example below does to show the trade-off, spends privacy that the accountant does not see (Hyperparameter tuning).

Which records to keep also matters. By step 1 of the proof of Theorem 44.3, any rule that looks only at the user's own records is allowed, and Chua et al. (2024) report that keeping each user's longest records worked best on one of their language datasets. Bounding also combines with user-level DP-SGD that averages each user's records, where a cap bounds compute and costs data but no noise.

Example 44.4. Table 44.4 fixes the user-level budget at $\varepsilon = 4$ and $\delta = 10^{-5}$, with 1,000 steps and an expected batch of 100 records. For each cap the sampling rate is $q = 100/n_k$, and kq is roughly the chance that a user with k kept records has one in a batch.

Table 44.4.: Contribution bounding at user-level $\varepsilon = 4$: example-level DP-SGD on at most k records per user, accounted for k records, one run each.

Cap k	Records kept	Sampling rate q	kq	Noise multiplier σ	Test accuracy
1	1,000 (10.0%)	0.1	0.100	3.78	0.8917
2	1,807 (18.1%)	0.0553	0.111	4.18	0.8920
5	3,506 (35.1%)	0.0285	0.143	5.34	0.8790
10	5,063 (50.6%)	0.0198	0.198	7.34	0.8276
20	6,631 (66.3%)	0.0151	0.302	11.2	0.7562
50	8,437 (84.4%)	0.0119	0.593	21.8	0.6401

Raising the cap from 1 to 50 multiplies the data by 8.4 and the noise multiplier by 5.8, and accuracy falls from 0.8917 to 0.6401. Since kq grows by $0.593/0.100 = 5.93$, proportionality to kq would predict $3.78 \times 5.93 = 22.4$ at the largest cap, close to the accountant's 21.8, although at $kq = 0.593$ the small-kq regime of Proposition 44.4 no longer holds. On this task the noise dominates, because a dataset of 1,000 records, one per user, already holds most of what the network can learn privately at this budget. Every extra record per user costs more in noise than

it adds in signal, and the caps 1 and 2 are tied. The row $k = 1$ is user-level DP-SGD on one record per user in disguise, since sampling one record per user at rate 0.1 is sampling users at rate 0.1, with the same $\sigma = 3.78$.

44.5. Heterogeneous users

User sizes are rarely equal. In the running example's population, the 30 users with at least 50 records each hold 30.6% of the records, while the smallest half of the users hold 10.3% (Figure 44.1). Example-level training weights each user by their number of records, and user-level training does not. This section asks what objective user-level DP-SGD optimizes, and what that means for heavy users.

Averaging over the sampling cancels the factor q, and the noise has mean zero, so the expected update should be a plain average over users of their clipped vectors.

Proposition 44.5 (The expected user-level update)**.** *In Definition 44.2, averaged over the user sampling and the noise, the update direction is*

$$\mathbb{E}\Big[\frac{G_t}{qU}\Big] = \frac{1}{U}\sum_{u=1}^{U} \min\Big(1, \frac{C}{\|\bar{g}_u\|}\Big)\bar{g}_u, \tag{44.15}$$

in which each user's term has length at most C/U, whatever their number of records. Example-level DP-SGD gives user u's records together the weight n_u/n instead.

Proof. The plan is to average over the sampling and the noise, then bound one user's weight, then compare it with example-level training.

1. *Average over the sampling.* User u is in S_t with probability q, so the expected sum of the sampled $\tilde{g}_u$ is $q\sum_u \tilde{g}_u$. The noise has mean zero.
2. *Divide by qU.* The q cancels, which leaves $\frac{1}{U}\sum_u \tilde{g}_u$. That is Equation 44.15, by the definition of $\tilde{g}_u$.
3. *The weight of one user.* Clipping gives $\|\tilde{g}_u\| \le C$ for every user, so each term has length at most C/U. The share of any group of users in the update is therefore at most their share of the users.
4. *Example-level.* The same computation for DP-SGD over records (DP-SGD) gives each record weight $1/n$, so user u's records together have weight n_u/n.

□

User-level training therefore changes the objective, not only the noise. With averaged per-user gradients, the model minimizes a clipped version of the average over users of each user's average loss, $\frac{1}{U}\sum_u \frac{1}{n_u}\sum_{i\in u} \ell(\theta; z_i)$, rather than the average over records. For a keyboard model, weighting people equally keeps a few very active users from defining everyone's suggestions. For a model meant to fit the overall distribution of records, it under-weights heavy users' content, which matters when heavy users differ from light ones.

Clipping caps heavy users a second time. A heavy user's average gradient is a good estimate, typically short once the model fits their data. But if their data are unusual, the gradient is long

and is clipped to C, the user-level analogue of what clipping does to records with large gradients (Disparate impact). Summing a user's gradients before clipping sits between record weighting and user weighting. A heavy user's vector is then up to n_u times longer than one record's, so their influence rises with n_u until it reaches C.

Example 44.5. The 30 users with at least 50 records hold 30.6% of the records, and so 30.6% of the weight in the expected example-level update. Under user-level DP-SGD, Proposition 44.5 caps their weight at $30/1{,}000 = 3.0\%$. To see whether that hurts the model where heavy users' data are, the script splits the test set by the owner of each test point's nearest training record. A heavy user's records were chosen as the nearest neighbours of one seed in 50 dimensions, so they spread widely, and 68.3% of test points have a heavy user's record as their nearest neighbour. At user-level $\varepsilon = 4$, user-level DP-SGD reaches 0.9361 on those points and 0.9319 on the others, against 0.9607 and 0.9622 for the non-private model. On this task, down-weighting heavy users costs nothing measurable, because one teacher network labels every record, so the light users' records teach the same function. On data where heavy users' records follow their own patterns, the same reweighting would cost accuracy on those patterns.

44.6. Comparing the strategies on the running example

Three ways to train the running example with a user-level guarantee are now on the table. Strategy (a) runs example-level DP-SGD on all records, accounted for the heaviest user by Theorem 44.1 with $k = 348$. Strategy (b) is user-level DP-SGD, with the guarantee of Theorem 44.2. Strategy (c) bounds contributions to two records per user and runs example-level DP-SGD, with the guarantee of Theorem 44.3 at $k = 2$. This section asks which model is best at the same user-level (ε, δ).

Example 44.6. Table 44.5 trains each strategy for 1,000 steps with an expected batch of 100 units, $C = 0.5$ and $\eta = 0.5$, at the noise multipliers of Table 44.3, one run each.

Table 44.5.: Test accuracy at the same user-level budget, $\delta = 10^{-5}$, one run each.

User-level ε	(a) Example-level, all records	(b) User-level DP-SGD	(c) At most 2 records per user
1	0.5318	0.8065	0.7084
2	0.5582	0.9028	0.8374
4	0.5713	0.9347	0.8810
8	0.5478	0.9442	0.9333
Non-private, 1,000 steps	0.9612		

The cap $k = 2$ is the best cap at $\varepsilon = 4$ in Table 44.4, tied with $k = 1$, and choosing it from test accuracy is a leak that a real deployment would have to pay for. Run-to-run variation is of the order of a point at these budgets, which is why the cap of 2 scores 0.8810 here and 0.8920 in its separate run of Table 44.4. The cap $k = 1$ and user-level DP-SGD on one random record per user are the same algorithm with the same σ, and two runs of it gave 0.8917 and 0.9070.

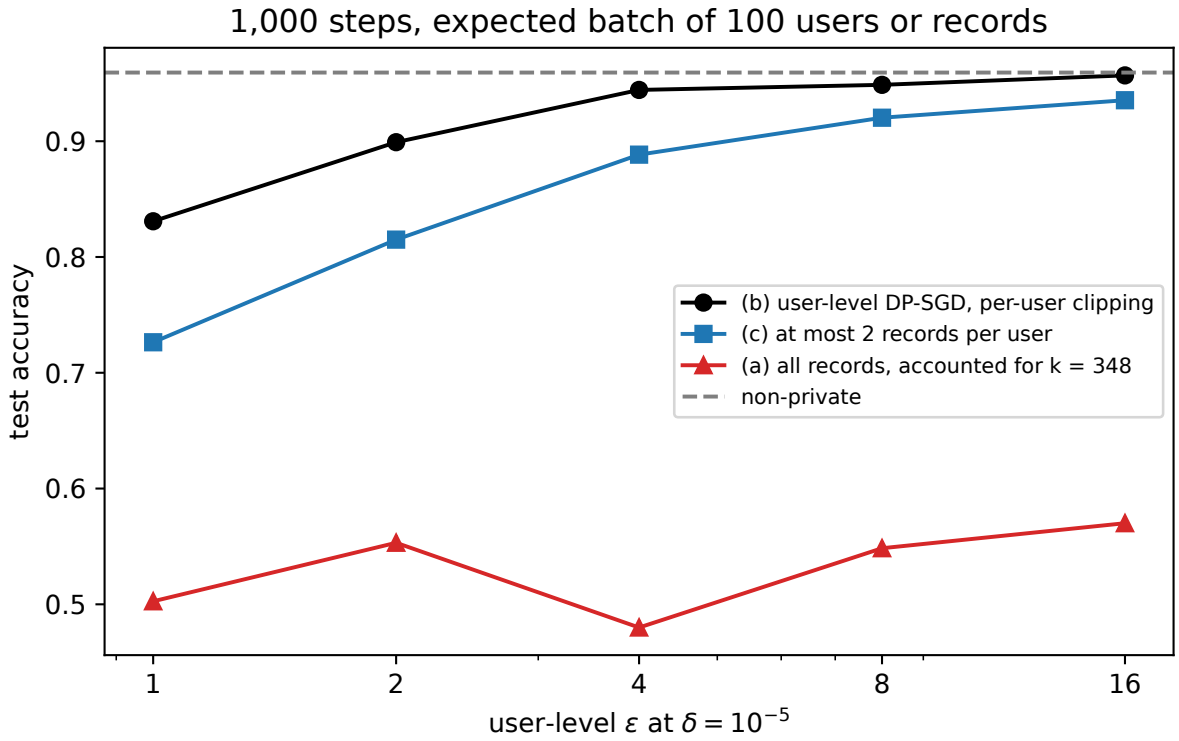


Figure 44.3.: Test accuracy of the running example against the user-level ε at $\delta = 10^{-5}$, for the three strategies (1,000 steps, expected batch 100, one run each; the runs differ from Table 44.5 in their random seeds). User-level DP-SGD is best at every budget; example-level DP-SGD on all records, accounted for its heaviest user, stays near chance.

Keeping the record as the unit fails. Strategy (a) is at chance, between 0.53 and 0.57, at every budget up to $\varepsilon = 8$, because the heaviest user dictates the noise for everyone, and a σ between 70.3 and 449 drowns every gradient. This is Figure 44.2 in practice: an example-level pipeline on data with heavy users needs either an enormous ε or a cap.

Changing the unit wins. User-level DP-SGD is the best strategy at every budget, with 0.9347 at $\varepsilon = 4$ against 0.9612 for the non-private model. It needs a little less noise than the cap of 2, and unlike the cap it uses every record, through the per-user average. Its advantage is largest at small budgets, 9.8 points at $\varepsilon = 1$ against 1.1 points at $\varepsilon = 8$. Its cost is compute, since each step processes the records of about 100 users, about 1,000 records, ten times what the example-level strategies process. Bounding is the fallback, and strategy (c) keeps the example-level pipeline and loses 5 to 10 points against (b) below $\varepsilon = 8$. It comes close at $\varepsilon = 8$, where the noise is small enough that the discarded data, not the noise, limits accuracy.

Published comparisons on language models agree. Chua et al. (2024) find that user-level DP-SGD generally beats contribution bounding, especially at small budgets, and Charles et al. (2024) find that user-level sampling usually wins, though not always.

At $\varepsilon = 4$ user-level DP-SGD loses 2.7 points against the non-private model, while the record-level model of DP-SGD reached 0.9548 at the same ε. The difference between 0.9548 and 0.9347 is the price of the unit, since the guarantee now covers the person with 348 records as well as the one with a single record.

44.7. Code

The script code/user_level.py builds the user population and implements the user accountant of Theorem 44.1 by numerical integration. It checks the accountant against Equation 44.7 and Equation 44.3, compares it with group privacy, and trains the three strategies with the part's network. It imports the model, DP-SGD and the record-level accountant from code/dp_sgd.py. From the repository root:

```
uv run python code/part05-dp-ml/user_level.py
```

It takes about a minute and a half, most of it spent in user-level DP-SGD, which processes about 1,000 records per step, and in finding the noise multiplier for the user with 348 records.

Table 44.6.: The script's output with the default seed.

Experiment	Result
Population	1,000 users; mean 10, median 4, largest 348 records; 193 users with one record
Accountant checks	$k = 1$ matches the subsampled Gaussian; integral matches Equation 44.7 to seven digits
Running example, user with k records	$\varepsilon = 15.54$ at $k = 2$, 146.20 at $k = 10$; group privacy 18.61 and none

Experiment	Result
Noise at user-level $\varepsilon = 4$	$\sigma = 128$ (all records), 3.78 (user-level), 4.18 (cap 2)
Contribution bounding at $\varepsilon = 4$	0.8917 at $k = 1$ down to 0.6401 at $k = 50$
Strategies at $\varepsilon = 4$	0.5713 (all records), 0.9347 (user-level), 0.8810 (cap 2); non-private 0.9612
Heavy users	30.6% of records, at most 3.0% of the user-level update

tests/test_user_level.py checks that the output still matches.

44.8. Summary

Table 44.7.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 44.1	User-level neighbours differ in all the records of one user	Name the privacy unit with every ε
Proposition 44.1	Example-level (ε, δ) gives user-level $(k\varepsilon, \delta\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1})$, tightly	Any mechanism's guarantee for users; not for DP-SGD, where it fails from $k = 10$
Theorem 44.1	A user with k records in example-level DP-SGD: $\mathcal{N}(0, \sigma^2)$ against $\sum_m b_m \mathcal{N}(m, \sigma^2)$, tight	Auditing what an existing DP-SGD model guarantees to heavy users
Proposition 44.2	$A_\alpha = \mathbb{E}[\exp(\sigma^{-2}\sum_{i<j} H_i H_j)]$ at integer α	Checking the numerical accountant
Corollary 44.1	$\varepsilon_k(\alpha) \approx \frac{\alpha}{2}(kq)^2(e^{1/\sigma^2} - 1)$ for small kq	Quick estimate: rerun the record-level accountant at rate kq, and expect the truth to be a little worse
Definition 44.2, Theorem 44.2	Sample users, clip one vector per user: DP-SGD's accountant with users as units	The default for user-level training; it pays for the number of people, not their records, and uses all their data
Proposition 44.3	Per-record clipping, at most k records, users sampled whole: noise multiplier σ/k	Reusing per-example gradient code, at a factor k in noise
Definition 44.3, Theorem 44.3	At most k records per user, chosen per user; account for k records	Keeping a record-based pipeline; choose the cap from public knowledge

Result	Statement	Use
Proposition 44.4	Noise grows like kq; users above the cap lose weight	Choosing k as a bias–variance trade-off
Proposition 44.5	$\mathbb{E}[G_t/(qU)] = \frac{1}{U}\sum_u \min(1, C/\|\bar{g}_u\|)\bar{g}_u$	Deciding how heavy users count: averaging weights people equally, summing before clipping weights records until the clip

44.9. References

- McMahan, Ramage, Talwar & Zhang, "Learning Differentially Private Recurrent Language Models" (ICLR 2018). User-level DP-SGD in federated learning: sample users, clip each user's update, add Gaussian noise, account with users as the unit.
- Charles, Ganesh, McKenna, McMahan, Mitchell, Pillutla & Rush, "Fine-Tuning Large Language Models with User-Level Differential Privacy" (arXiv 2407.07737, 2024). The tight accountant for example-level sampling (their Theorem 3, the mixture of Equation 44.5), and an empirical comparison of example-level and user-level sampling for language models.
- Chua, Ghazi, Huang, Kamath, Kumar, Liu, Manurangsi, Sinha & Zhang, "Mind the Privacy Unit! User-Level Differential Privacy for Language Model Fine-Tuning" (Conference on Language Modeling 2024; arXiv 2406.14322). Contribution bounding with group privacy against user-wise DP-SGD, with record-selection rules and the choice of the cap.
- Amin, Kulesza, Muñoz Medina & Vassilvitskii, "Bounding User Contributions: A Bias-Variance Trade-off in Differential Privacy" (ICML 2019). The cap on records per user as a bias–variance trade-off, and how the best cap depends on the distribution of user sizes.
- Levy, Sun, Amin, Kale, Kulesza, Mohri & Suresh, "Learning with User-Level Privacy" (NeurIPS 2021). Theory of user-level private learning when each user holds several samples.
- Mironov, "Rényi Differential Privacy" (CSF 2017). Group privacy for RDP.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (Foundations and Trends in Theoretical Computer Science, 2014). Group privacy for pure and approximate DP.

45. Disparate impact

The chapter on private gradient descent trained the part's running example with differential privacy (DP) at $\varepsilon = 4$ and lost less than one point of test accuracy: 0.9548 against 0.9610 without privacy. That number is an average over the test set, so it says nothing about who loses the point. Bagdasaryan, Poursaeed & Shmatikov (2019) asked that question. They found that stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, DP-SGD, concentrates the accuracy it loses on the classes and subgroups that are under-represented in the training data. In their gender classifier the drop was much larger on Black faces than on white faces. They called this the **disparate impact** of DP on model accuracy. This chapter asks why it happens, how to measure it honestly, and what can be done about it.

The answer lies in the two rules of DP-SGD, which are blind to groups and for that very reason are not neutral. Clipping caps each record's pull, its contribution to the update, so a group's pull is at most its share of the records times the cap, however badly the model fits it. The noise is the same for everyone, while the part of the update that serves a group grows with the group's size, so a small group's signal is weaker against the same noise. A small group the model finds hard suffers from both, and the two effects compound, because the noise then acts on a pull that clipping has already shrunk.

The running example is the part's two-layer network on 50-dimensional inputs, with a minority added. Of the $n = 10{,}000$ training records, 9,500 come from the running example's training set. The other 500, a share of 5%, lie in another region of input space, where inputs are longer, 8.89 on average against 7.04, which lengthens their gradients. They also follow a rule of their own, one the majority does not teach. Without privacy the model reaches 0.9529 on the majority and 0.9411 on the minority, each measured on 20,000 fresh test records. DP-SGD keeps the running example's settings from DP-SGD, with neighbours that differ by adding or removing one record. At $\varepsilon = 4$ the majority loses 0.59 points of accuracy and the minority 9.34. Table 45.1 lists the symbols of the chapter.

Table 45.1.: The symbols of this chapter. A subscript i always names a record and a subscript g or h a group, so g_i is a record's gradient and $\bar{g}_g$ a group's average gradient.

Symbol	Meaning
q; C; η; T	the sampling rate, 0.01; the clipping norm, 0.5; the learning rate, 0.5; the number of steps, 10,000
δ	the slack of every guarantee, 10^{-5}
g_i; θ	the gradient of record i's loss; the model's weights
g, h (as subscripts)	a group of records; n_g records of group g, and its **share** $s_g = n_g/n$

Symbol	Meaning
$B = qn$; σ; $\tau = \sigma C/B$	the expected batch size; the noise multiplier; the noise per coordinate on the averaged update
$\bar{g}_g$; m_g	group g's average gradient; its average clipped gradient
w_g; c_g; $\tilde{s}_g$	the group's clipping weight; the fraction of its average gradient that survives clipping; its effective share
acc_g; π_g	a model's accuracy on test records of group g; the group's privacy cost
d_g; λ; L_g	in the quadratic model of Section 45.3, the number of the group's own weights, the curvature of its loss, and its loss

45.1. Measuring disparate impact

Before explaining the effect, you need to say what exactly to measure, and to check that it happens on the running example. A gap between the groups' accuracies is not enough, because the non-private model may already have one, and DP did not cause that part. The right quantity compares each group with itself, trained with and without privacy.

Definition 45.1 (Privacy cost and disparate impact)**.** Train a model on the same data with the same recipe, once without privacy and once with DP. The **privacy cost** of group g is the accuracy the group loses when the recipe is made private,

$$\pi_g = \text{acc}_g(\text{non-private}) - \mathbb{E}[\text{acc}_g(\text{private})], \tag{45.1}$$

where the expectation is over the randomness of private training. DP has **disparate impact** on groups g and h when $\pi_g \neq \pi_h$, and the **privacy cost gap** $|\pi_g - \pi_h|$ measures how much.

Esipova et al. (2023) use this definition, and Bagdasaryan et al. (2019) compare the same differences. The experiment below is designed to make the comparison fair. Every row trains on the same records with the same recipe, and differs only in the clipping and in the noise multiplier that the accountant of DP-SGD sets for each ε. Test records are drawn independently, so a measured accuracy near 0.9 on 20,000 of them is a binomial proportion with standard deviation $\sqrt{0.9 \times 0.1/20{,}000} = 0.0021$, far below the differences in Table 45.2. Training is random, so each row averages three runs. And because the non-private model serves both groups within 1.2 points, the private model is compared with a baseline that is already fair.

Example 45.1. Table 45.2 gives each group's accuracy on the running example over three training runs per row. At $\varepsilon = 4$ the majority loses 0.59 points of accuracy and the minority 9.34, 15.7 times as much. At $\varepsilon = 8$, a budget at which DP-SGD found the running example nearly as accurate as non-private training, the majority loses nothing (its accuracy is 0.20 points higher) and the minority loses 8.58 points. At $\varepsilon = 1$ the ratio falls to 4.6 only because the majority starts

to lose too, 7.07 points against the minority's 32.78. Yet a test set drawn like the training set, 95% majority, would report a single loss of $0.95 \times 0.0059 + 0.05 \times 0.0934 = 0.0103$ at $\varepsilon = 4$, about one point, and hide the minority's loss of more than nine.

Table 45.2.: Test accuracy of each group (mean ± standard deviation over 3 runs) of the running example with a 5% minority, trained with DP-SGD at four budgets ($\delta = 10^{-5}$), with clipping alone, and without privacy. The privacy cost is Equation 45.1.

Training	Majority accuracy	Minority accuracy	Gap	Privacy cost: majority	minority
Non-private SGD	0.9529 ± 0.0018	0.9411 ± 0.0052	0.0119	—	—
Clipping only, $\sigma = 0$	0.9581 ± 0.0015	0.9007 ± 0.0048	0.0574	−0.0051	0.0404
$\varepsilon = 8$, $\sigma = 0.917$	0.9550 ± 0.0006	0.8553 ± 0.0110	0.0997	−0.0020	0.0858
$\varepsilon = 4$, $\sigma = 1.36$	0.9470 ± 0.0037	0.8477 ± 0.0211	0.0993	0.0059	0.0934
$\varepsilon = 2$, $\sigma = 2.28$	0.9366 ± 0.0082	0.7574 ± 0.0379	0.1791	0.0163	0.1836
$\varepsilon = 1$, $\sigma = 4.13$	0.8822 ± 0.0032	0.6133 ± 0.0173	0.2689	0.0707	0.3278

Both causes of the effect are already visible in the table. The clipping-only row has no noise at all, yet the minority loses 4.04 points while the majority gains 0.51, so clipping alone is disparate. Adding noise then widens the gap steadily as ε falls (Figure 45.1), and the next two sections explain each part. The minority's runs also vary more, a point Section 45.4 returns to.

The published experiments found the same pattern. Esipova et al. (2023) undersampled class 8 of the MNIST digits to about 1% of the training set. At $\varepsilon = 5.90$, DP-SGD cost class 2, which kept its usual size, 8.9 points, but class 8 57.9 points (their Table 2).

45.2. Clipping reweights the groups

Clipping without noise gives no privacy at all, yet in Table 45.2 it costs the minority 4.04 points. This section asks how a rule that treats every record the same way can treat groups differently. Each DP-SGD step adds up the clipped gradients of a Poisson batch, adds Gaussian noise, and divides the noisy sum G_t by the public expected batch size $B = qn$. Because each record enters the batch with probability q, DP-SGD shows that the average update is the average gradient with each record weighted by how much of it clipping keeps:

$$\mathbb{E}\Big[\frac{G_t}{B}\Big] = \frac{1}{n}\sum_{i=1}^{n} \min\Big(1, \frac{C}{\|g_i\|}\Big) g_i. \tag{45.2}$$

The factor in front of each gradient is what the rest of the chapter tracks, record by record and then group by group.

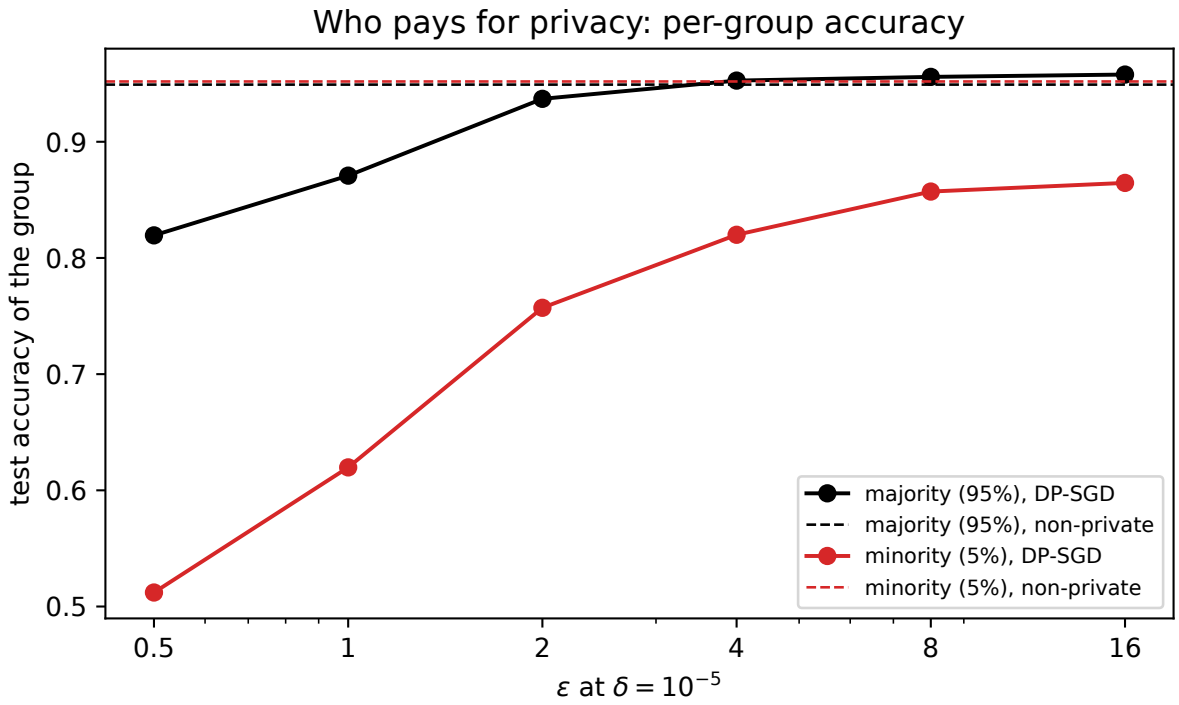


Figure 45.1.: Test accuracy of each group of the running example against ε (log scale, one run per point, $\delta = 10^{-5}$), with each group's non-private accuracy dashed. The majority is close to its non-private accuracy down to $\varepsilon = 2$; the minority falls far below it at every budget.

Definition 45.2 (Clipping weights)**.** A record's **clipping weight** is $\min(1, C/\|g_i\|)$, the factor clipping multiplies its gradient by. A group's clipping weight w_g is the average of its records' clipping weights. Group g's average gradient and average clipped gradient are

$$\bar{g}_g = \frac{1}{n_g}\sum_{i\in g} g_i, \qquad m_g = \frac{1}{n_g}\sum_{i\in g} \min\Big(1, \frac{C}{\|g_i\|}\Big) g_i. \tag{45.3}$$

Splitting the average update by group shows each group's pull. Because each clipped gradient is at most C long, a group's pull is capped by its share, however large its gradients are. When the gradients within a group point the same way, the cap turns into a reweighting of the groups.

Proposition 45.1 (Clipping reweights the groups)**.** *For any gradients, the expected update of DP-SGD is a share-weighted sum of the groups' average clipped gradients, and each group's term is at most its share times C long:*

$$\mathbb{E}\Big[\frac{G_t}{B}\Big] = \sum_g s_g\, m_g, \qquad \|s_g\, m_g\| \le s_g\, C. \tag{45.4}$$

Suppose in addition that the records within each group have parallel gradients, and let $c_g = \|m_g\|/\|\bar{g}_g\|$ be the fraction of group g's average gradient that survives clipping. Then the expected update is the gradient of a reweighted mixture of the groups,

$$\mathbb{E}\Big[\frac{G_t}{B}\Big] = \Big(\sum_h s_h\, c_h\Big) \sum_g \tilde{s}_g\, \bar{g}_g, \qquad \tilde{s}_g = \frac{s_g\, c_g}{\sum_h s_h\, c_h}, \tag{45.5}$$

*where the **effective shares** $\tilde{s}_g$ are non-negative and add up to 1. If all of group g's records also have the same gradient norm, then $c_g = w_g$.*

Proof. The plan is to regroup the sum, bound each group's part, and then use parallel gradients to pull a scalar out of each group's part.

1. *Split the sum.* Group the records of the expected update and multiply and divide each group's part by n_g:
$$\frac{1}{n}\sum_g \sum_{i\in g} \min\Big(1, \frac{C}{\|g_i\|}\Big) g_i = \sum_g \frac{n_g}{n}\cdot m_g = \sum_g s_g m_g. \tag{45.6}$$
2. *Bound the pull.* Each clipped gradient has length at most C, and the length of an average is at most the average of the lengths, by the triangle inequality. So $\|m_g\| \le C$, and $\|s_g m_g\| \le s_g C$.
3. *Parallel gradients.* If every g_i in group g points along the same unit vector e_g, then $g_i = \|g_i\| e_g$ and its clipped version is $\min(\|g_i\|, C)\, e_g$. Averaging, m_g and $\bar{g}_g$ both point along e_g, so $m_g = c_g \bar{g}_g$.
4. *Factor out the total.* Substituting step 3 into step 1 and multiplying and dividing by $\sum_h s_h c_h$ gives Equation 45.5. Each $\tilde{s}_g$ is a non-negative term divided by the sum of all such terms, so they add up to 1.
5. *Equal norms.* If all of group g's records have the same norm, the ratio of clipped length to length is the same for each record, namely its clipping weight. So c_g is that common weight, which is also the group's average w_g.

□

Equation 45.5 separates two things clipping does. The factor $\sum_h s_h c_h$ shortens the update, which a larger learning rate can undo. The effective shares change *which* mixture of the groups is being descended, and no learning rate undoes that. Esipova et al. (2023) find this error of direction far more harmful than the error of length, especially to the under-represented class (their Table 1). The effective shares also move the end point. Clipped gradient descent stops where $\sum_g s_g m_g = 0$, so it stops at the optimum of the reweighted mixture, which leans toward the groups whose gradients clipping cuts least.

A group's effective share falls when its gradients are long compared with C, for two reasons. The model may fit the group badly, since a misclassified record has a large loss derivative and a record classified with a wide margin a small one. Or the group's inputs may be large, since a dense layer's per-example gradient norm is the error's norm times the input's norm (DP-SGD). Tran, Dinh & Fioretto (2021) identify both factors, and a small group the model has not yet learned has both. That sets up a feedback loop. As the majority is learned, its gradients fall below C and its weight rises toward 1, while the minority keeps long gradients and a low weight. So the minority is learned more slowly, and its gradients stay long.

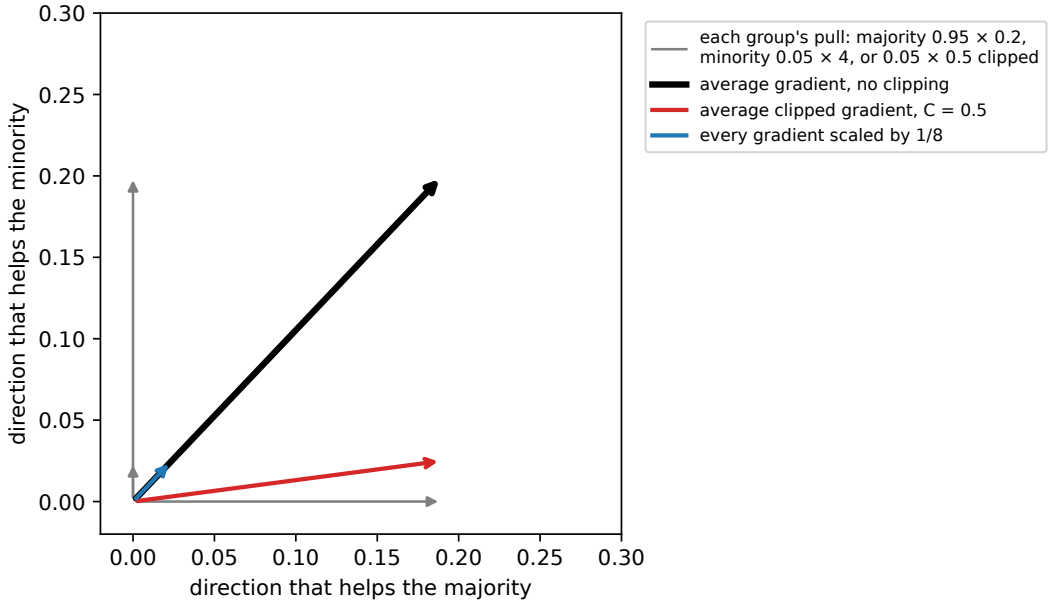


Figure 45.2.: The clipping example in two dimensions: 95 majority records with gradient (0.2, 0) and 5 minority records with gradient (0, 4). Without clipping the two groups pull about equally hard (black); clipping at C = 0.5 cuts the minority's pull eightfold and turns the update toward the majority (red). Scaling every gradient by the same factor 1/8 instead keeps the direction (blue).

The smallest case shows the reweighting exactly, before the network shows the feedback loop.

Example 45.2. Take 95 majority records whose gradient is $(0.2, 0)$, already short, and 5 minority records whose gradient is $(0, 4)$, long because the model fits them badly (Figure 45.2). Without clipping the average gradient is $(0.95 \times 0.2, 0.05 \times 4) = (0.19, 0.2)$, of length 0.2759. The five minority records pull as hard as the ninety-five others, since the minority's component is a fraction $0.2/0.39 = 0.513$ of the sum of the two components. Clipping at $C = 0.5$ leaves the

majority's gradients untouched and shrinks the minority's to length 0.5, so the average becomes $(0.19, 0.05 \times 0.5) = (0.19, 0.025)$, of length 0.1916, and the minority's fraction falls to 0.116. The update has turned by 39.0 degrees toward the majority.

Here $c_{\text{maj}} = 1$ and $c_{\text{min}} = 0.5/4 = 0.125$, so Equation 45.5 gives the minority an effective share of $0.05 \times 0.125/(0.05 \times 0.125 + 0.95) = 0.0065$ instead of 0.05. Scaling every gradient by C/C_{max} instead, with $C_{\text{max}} = 4$ the largest norm, gives $(0.0238, 0.0250)$. That update is eight times shorter, but at an angle of 0.0 degrees to the true average.

Example 45.3. In the running example, Table 45.3 follows each group's gradients during training with clipping and no noise, on 1,000 majority and the 500 minority training records. At step 0 both groups are fully clipped, with similar weights, and the minority's effective share, 0.0495, is its true share. By step 2,500 the majority's median gradient norm has fallen to 3.8×10^{-4}, while the minority's is still 2.7 and 70% of its records are clipped. The minority's effective share has fallen to 0.0251, half its true share, exactly while it most needs to be learned, with test accuracy 0.6724 against the majority's 0.9569. The share recovers only as the minority is learned, to 0.0387 at the end.

Table 45.3.: Per-group gradient statistics during training with clipping at $C = 0.5$ and no noise. "Clipped" is the fraction of records with gradient norm above C. The effective share is $s\,w_{\text{min}}/(s\,w_{\text{min}} + (1-s)\,w_{\text{maj}})$ with $s = 0.05$, Equation 45.5 with $c_g = w_g$, which treats each group's gradients as parallel and of equal norm.

Step	Median norm: majority	Median norm: minority	Clipped: majority	Clipped: minority	Weight w_g: majority	Weight w_g: minority	Effective share of the minority	Test accuracy: majority	Test accuracy: minority
0	3.4	3.7	1.00	1.00	0.165	0.163	0.0495	0.5064	0.5019
2,500	3.8×10^{-4}	2.7	0.18	0.70	0.869	0.424	0.0251	0.9569	0.6724
5,000	1.4×10^{-4}	0.71	0.14	0.54	0.891	0.579	0.0331	0.9607	0.8137
7,500	7.7×10^{-5}	0.30	0.13	0.45	0.906	0.651	0.0364	0.9621	0.8661
10,000	7.0×10^{-5}	0.16	0.11	0.39	0.917	0.702	0.0387	0.9613	0.8900

The same happens with noise (Figure 45.3). At $\varepsilon = 4$ the minority's effective share is 0.0261 at step 2,500 and 0.0358 at the end, where its weight is 0.637 against the majority's 0.902. The noise keeps the minority's gradients long, so clipping keeps cutting them back.

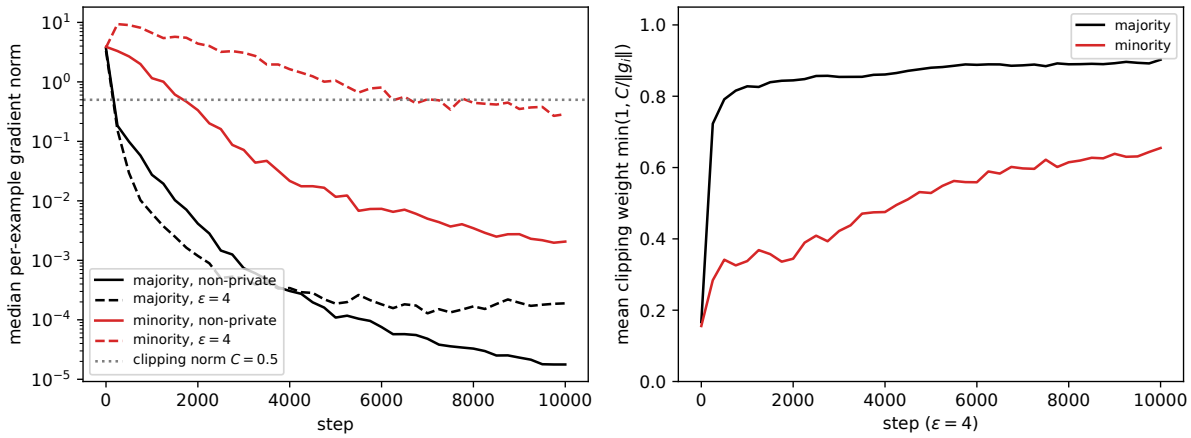


Figure 45.3.: Left: the median per-example gradient norm of each group during non-private training (solid) and DP-SGD at $\varepsilon = 4$ (dashed), on a log scale, with the clipping norm dotted. The majority's gradients fall far below C within the first thousand steps; the minority's stay above it much longer. Right: each group's mean clipping weight during DP-SGD at $\varepsilon = 4$.

45.3. The noise floor of a small group

Clipping explains the clipping-only row of Table 45.2, but not why the gap keeps widening as ε falls. The noise is added to the sum and is the same for everyone, so this section asks why it should hurt a small group more than a large one. The reason is that the noise competes with each group's signal, and a group's signal in the sum is proportional to its size. On the averaged update the noise has standard deviation $\tau = \sigma C/B$ per coordinate, whatever the data. In the running example at $\varepsilon = 4$, $\tau = 1.36 \times 0.5/100 = 0.0068$.

DP-SGD explains why training works at all when each step's noise is ten times longer than its signal. The signal keeps pointing the same way, so it adds up in proportion to the number of steps, while independent noise adds up only in proportion to its square root. Apply the same argument to the part of the signal that serves one group. That part is proportional to the group's share when each of its records pulls about as hard as any other, and the argument gives a first law.

Proposition 45.2 (Break-even time of a group's signal)**.** *Suppose each step moves the weights by a signal that keeps pointing the same way, plus noise that is independent from step to step, has mean zero and has root-mean-square length ℓ_{noise}. Let ℓ_{sig} be the length of the signal the whole population shares, and suppose the part that serves group g has length $s_g \ell_{sig}$. Then after t steps the group's accumulated signal is longer than the root-mean-square length of the accumulated noise exactly when $t > t_g$, where*

$$t_g = \frac{1}{s_g^2}\left(\frac{\ell_{noise}}{\ell_{sig}}\right)^2. \tag{45.7}$$

That is $1/s_g^2$ times as many steps as a signal the whole population shares needs.

Proof.

1. *The signal adds up.* The group's signal points the same way at every step, so after t steps its length is $t\, s_g \ell_{\text{sig}}$.
2. *The noise adds up more slowly.* The noise vectors are independent with mean zero, so the cross terms of the squared length of their sum average to zero, and the squared lengths add. The sum has mean squared length $t\, \ell_{\text{noise}}^2$, so its root-mean-square length is $\ell_{\text{noise}}\sqrt{t}$.
3. *Compare.* The signal is longer when $t\, s_g \ell_{\text{sig}} > \ell_{\text{noise}}\sqrt{t}$. Dividing both sides by $\sqrt{t}\, s_g \ell_{\text{sig}}$ and squaring gives $t > t_g$.

□

The law compares the group's signal with the whole noise vector, as DP-SGD does for the full signal, so it is a rough clock, not an exact threshold.

The break-even law says when a group's signal can emerge, but not where training settles once it has. For that, take a model simple enough to solve exactly. Each group has its own block of weights, which no other group's loss touches, and its loss is quadratic in them. Step 3 of the proof below needs one fact about a number that is pulled toward zero at each step and kicked by fresh noise.

Lemma 45.1 (A shrinking recursion with noise settles at a fixed variance)**.** *Fix a pull strength a with $0 < a < 2$. Let the noise terms ζ_t be independent of each other and of the start x_0, with mean zero and variance ω^2, and let a number evolve as*

$$x_{t+1} = (1 - a)x_t + \zeta_t. \tag{45.8}$$

Then the variance of x_t converges to

$$V = \frac{\omega^2}{a(2 - a)}, \tag{45.9}$$

and the distance of the variance from V shrinks by the factor $(1-a)^2$ at every step, so the starting point is forgotten after about $1/a$ steps.

Proof.

1. *One step of variance.* The noise ζ_t is independent of x_t, so the variances of the two terms add, and the constant factor $1 - a$ multiplies the variance by its square. Writing V_t for the variance of x_t,
$$V_{t+1} = (1 - a)^2 V_t + \omega^2. \tag{45.10}$$
2. *The fixed point.* The V of the statement solves $V = (1 - a)^2 V + \omega^2$, because $1 - (1 - a)^2 = a(2 - a)$.
3. *Convergence.* Subtracting the fixed-point equation from step 1 gives $V_{t+1} - V = (1-a)^2(V_t - V)$. Since $0 < a < 2$, the factor $(1 - a)^2$ is below 1, so V_t converges to V. For small a the factor after t steps is $(1 - a)^{2t} \approx e^{-2at}$, which is small once t is a few times $1/a$.

□

A weak pull lets the noise accumulate, and halving the pull roughly doubles the variance. The theorem below applies the lemma to each weight of a group, and the variance it settles at is the group's noise floor. Section M.5 in the appendices derives a per-step decomposition of a group's loss change, due to Tran, Dinh & Fioretto (2021), and recovers the same floor from it.

Theorem 45.1 (The noise floor of a group)**.** *Suppose group g's loss depends only on its own block θ_g of d_g weights, which no other group's loss touches, as $L_g = \frac{\lambda}{2}\|\theta_g - \theta_g^*\|^2$ with curvature $\lambda > 0$ and optimum θ_g^*. The training objective is the share-weighted average $\sum_g s_g L_g$, and gradient descent with learning rate η, where $0 < \eta s_g \lambda < 2$, adds independent noise of standard deviation τ per coordinate to the averaged gradient at each step. Then group g's expected loss settles at*

$$\mathbb{E}[L_g] = \frac{d_g\,\eta\,\tau^2}{2\,s_g\,(2 - \eta s_g \lambda)} \approx \frac{d_g\,\eta\,\tau^2}{4\,s_g}, \tag{45.11}$$

and it takes about $1/(\eta s_g \lambda)$ steps to get there. If clipping multiplies the group's gradient by a constant factor c_g, as in Proposition 45.1, the same holds with $s_g c_g$ in place of s_g.

Proof. The plan is to write one step for a single weight of the group, recognize the recursion of Lemma 45.1, and add up the resulting variances into a loss.

1. *The group's gradient.* Only group g's loss depends on θ_g, and it enters the objective with weight s_g. So the objective's gradient with respect to θ_g is $s_g\lambda(\theta_g - \theta_g^*)$.
2. *One step.* Write $x = \theta_g - \theta_g^*$ for the distance from the optimum and $\tau\xi$ for the noise, with ξ standard normal. The update $\theta_g \leftarrow \theta_g - \eta(s_g\lambda x + \tau\xi)$ moves each coordinate as

$$x \leftarrow (1 - \eta s_g \lambda)x - \eta\tau\xi. \tag{45.12}$$

3. *Stationary variance.* This is the recursion of Lemma 45.1 with $a = \eta s_g \lambda$ and noise variance $\omega^2 = \eta^2\tau^2$. So each coordinate settles at variance

$$\frac{\eta^2\tau^2}{\eta s_g \lambda(2 - \eta s_g \lambda)} = \frac{\eta\tau^2}{s_g\lambda(2 - \eta s_g \lambda)}, \tag{45.13}$$

   and gets there after about $1/a = 1/(\eta s_g \lambda)$ steps.
4. *Excess loss.* The loss is $\frac{\lambda}{2}$ times the sum of the squared coordinates over the group's d_g weights, so its expectation is $\frac{\lambda}{2}d_g$ times step 3's variance, which is Equation 45.11. For a small group $\eta s_g \lambda$ is small, so $2 - \eta s_g \lambda \approx 2$, which gives the approximate form.
5. *With clipping.* If clipping multiplies the group's gradient by c_g, the group's term in step 1 becomes $s_g c_g \lambda(\theta_g - \theta_g^*)$, while the noise is unchanged. Steps 2 to 4 then hold with $s_g c_g$ in place of s_g.

□

The floor scales like one over the share. A group with a twentieth of the records sits at about twenty times the excess loss of a group with all of them, and takes twenty times longer to get there. The curvature λ cancels, because a sharper loss is pulled back harder but also pays more for each unit of noise. The share does not cancel, because it scales the group's pull back to the optimum while the noise stays the same size. Waiting does not help either, because at a fixed budget more steps need more noise per step. DP-SGD shows that, for a small sampling rate, doubling the steps doubles the noise variance per step. The break-even count of Equation 45.7 is proportional to that variance, so it doubles too, and grows as fast as the run. Finally, the two mechanisms compound, so a group that is 5% of the data and whose gradients clipping halves pays the floor of a group of 2.5%.

The quadratic model is exact for what it assumes, as the simulation below confirms, but a network is not quadratic and its groups do not use disjoint weights. Where the minority's task overlaps the majority's, the majority's gradient does part of the minority's work, and the penalty is smaller. Tran, Dinh & Fioretto's decomposition needs no disjoint weights, but it leaves the role of the share implicit, while the quadratic model makes it explicit.

Example 45.4. Take the noise of the running example at $\varepsilon = 4$, $\tau = 0.0068$, with $d_g = 51$ weights per group, $\lambda = 1$ and $\eta = 0.5$. Table 45.4 compares Equation 45.11 with a simulation of 40,000 noisy steps, averaged over the second half.

Table 45.4.: The noise floor of a group in the quadratic model with $\varepsilon = 4$ noise ($\tau = 0.0068$, $d_g = 51$, $\lambda = 1$, $\eta = 0.5$).

Share s_g	Excess loss, Equation 45.11	Simulated	Steps to settle, $1/(\eta s_g)$
0.95	4.07×10^{-4}	4.07×10^{-4}	2
0.2	1.55×10^{-3}	1.55×10^{-3}	10
0.05	5.97×10^{-3}	5.99×10^{-3}	40
0.01	2.96×10^{-2}	2.97×10^{-2}	200

The formula and the simulation agree to within 1%. The floor grows from 4.07×10^{-4} for a group with 95% of the data to 2.96×10^{-2} for a group with 1%, a factor of 73. The approximate form of Equation 45.11 would say 95, the ratio of the shares. The exact form is lower because the factor $2 - \eta s_g \lambda$ is 1.525 for the large group rather than 2.

The break-even law is harsher. In DP-SGD the averaged clipped gradient of the running example has length about 0.026 during training. At $\varepsilon = 4$ the noise has length $\tau\sqrt{3{,}329} = 0.392$ over the network's 3,329 weights. So a signal shared by the whole population emerges after $(0.392/0.026)^2 = 228$ steps, while one that is 20% of it needs 5,693 steps. At 5% the count is 91,085, nine times the entire run, and at 1% it is 2,277,115. The running example's minority still reaches 0.85 within the run at $\varepsilon = 4$, so much of what it learns must come from directions the majority's gradient also pushes. The law applies to the part of its rule that the rest of the data does not teach.

The network is not the quadratic model, so the last check varies the minority's share in the network itself.

Example 45.5. Table 45.5 varies the minority's share with everything else fixed. The minority's privacy cost falls steeply with its share, from 34.88 points at 1% to 9.34 at 5% and 2.80 at 20%. Going from 1% to 5% divides the cost by $0.3488/0.0934 = 3.7$, and going from 5% to 20% divides it by $0.0934/0.0280 = 3.3$. Equation 45.11 predicts that the loss falls in proportion to the share, by 5 and by 4, and the cost falls a little less because accuracy is not loss and it saturates. At 1% the private model is at chance on the minority, 0.5009, and cannot lose more. At 20% the minority's cost is within three points of the majority's, which stays below one point throughout (Figure 45.4, left).

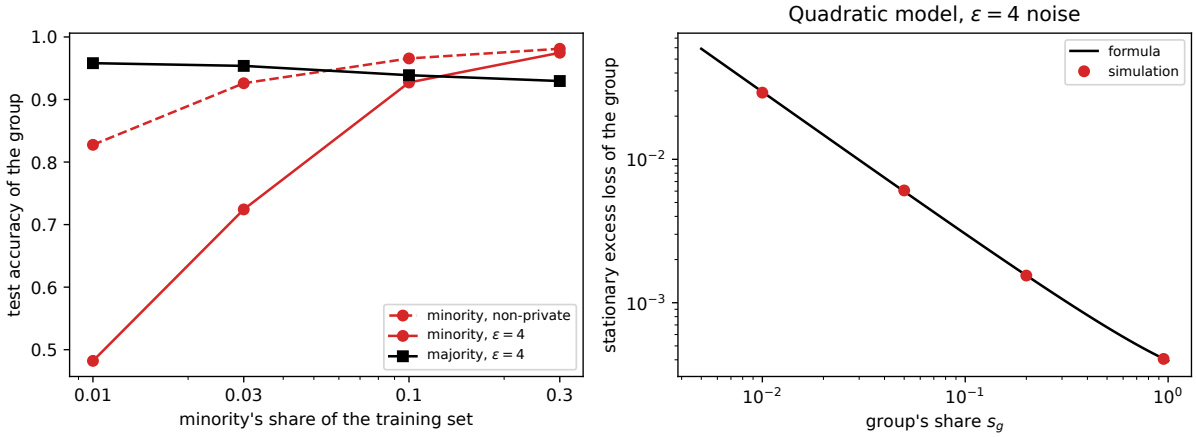


Figure 45.4.: Left: each group's test accuracy of the running example against the minority's share of the 10,000 training records (one run per point), non-private and at $\varepsilon = 4$. Right: the stationary excess loss of a group in the quadratic model, Equation 45.11, against its share, with the noise of $\varepsilon = 4$, compared with simulation.

Table 45.5.: Each group's test accuracy for three sizes of the minority, non-private and at $\varepsilon = 4$ (mean $\pm$ standard deviation over 2 runs; the 5% row is from Table 45.2, over 3 runs).

Minority share	Records	Non-private: majority	minority	$\varepsilon = 4$: majority	minority	Privacy cost: majority	minority
1%	100	0.9556 ± 0.0015	0.8498 ± 0.0021	0.9558 ± 0.0036	0.5009 ± 0.0102	-0.0002	0.3488
5%	500	0.9529 ± 0.0018	0.9411 ± 0.0052	0.9470 ± 0.0037	0.8477 ± 0.0211	0.0059	0.0934
20%	2,000	0.9395 ± 0.0016	0.9787 ± 0.0004	0.9355 ± 0.0019	0.9507 ± 0.0038	0.0041	0.0280

45.4. Measuring per-group accuracy

Every number so far was measured on 20,000 test records per group. Real evaluations are not so generous. A test split of 2,000 records with a 5% minority holds 100 minority records. This section asks how precisely a group's accuracy, and the gap between two groups, can be measured, and how much a single training run says.

Given the trained model, each test record of a group is classified correctly or not, independently, with probability equal to the group's true accuracy p_g. This needs the test records to be drawn independently from the group. Then the number correct is binomial, and the **Clopper–Pearson interval** of Theorem 4.7, built from exact binomial tail probabilities, covers the true accuracy with probability at least 95%. For the gap between two groups, the two counts come from different test records, so their variances add. Training adds a second source of variation, since a different run gives a different model.

Proposition 45.3 (Measuring a gap between groups)**.** *Fix a trained model with true accuracies p_g and p_h on groups g and h, and measure them as $\hat{p}_g$ and $\hat{p}_h$ on disjoint sets of n_g^{test} and n_h^{test} test records. Then $\hat{p}_g - \hat{p}_h$ has mean $p_g - p_h$ and variance $p_g(1-p_g)/n_g^{test} + p_h(1-p_h)/n_h^{test}$, and for large counts an approximate 95% interval for the gap is*

$$(\hat{p}_g - \hat{p}_h) \pm 1.96\sqrt{\frac{\hat{p}_g(1-\hat{p}_g)}{n_g^{test}} + \frac{\hat{p}_h(1-\hat{p}_h)}{n_h^{test}}}. \tag{45.14}$$

If the model itself is the output of a random training run, the variance of one run's measured accuracy on group g is the average test-set variance plus the run-to-run variance of the true accuracy:

$$\mathrm{Var}(\hat{p}_g) = \mathbb{E}\Big[\frac{p_g(1-p_g)}{n_g^{test}}\Big] + \mathrm{Var}(p_g). \tag{45.15}$$

Proof.

1. *The gap.* Given the model, each count is binomial, so $\hat{p}_g$ has mean p_g and variance $p_g(1-p_g)/n_g^{\text{test}}$. The two counts come from disjoint test records, so they are independent, and the variance of a difference of independent estimates is the sum of their variances. For large counts the difference is approximately normal by the central limit theorem, and 1.96 standard deviations on each side cover 95%. Replacing the true accuracies by the measured ones in the variance gives Equation 45.14.
2. *Two sources.* Over training runs the true accuracy p_g is itself random. The law of total variance splits the variance of $\hat{p}_g$ into the average of its variance given the model, which is step 1's test-set variance, plus the variance of its mean given the model, which is $\mathrm{Var}(p_g)$.

□

The second term is the one a larger test set cannot touch, because only repeated training estimates it. In Table 45.2 the run-to-run standard deviation at $\varepsilon = 4$ is 0.0211 for the minority against 0.0037 for the majority, almost six times as large. So a comparison of two training recipes on one run each can easily be reversed for the small group. DP-SGD draws the same lesson for overall accuracy, and for a small group it is sharper. The first term, the test-set variance, grows as the group's test records shrink, so small groups need intervals. A 5-point gap between groups can be seen only with a few hundred minority test records, and a 1-point gap only with thousands. Per-group accuracies on private test data are themselves statistics of that data, and publishing them needs its own privacy analysis (Choosing ε).

Example 45.6. Train the running example at $\varepsilon = 4$ once. On all 20,000 minority test records its accuracy is 0.8641. Table 45.6 measures it on random subsets.

Table 45.6.: Minority accuracy of one model trained at $\varepsilon = 4$, measured on random subsets of the minority's test records, with the majority on 1,900 records for comparison.

Test records	Correct	Estimate	95% Clopper–Pearson interval
100 minority	88	0.880	$[0.800, 0.936]$
500 minority	424	0.848	$[0.813, 0.878]$

Test records	Correct	Estimate	95% Clopper–Pearson interval
2,000 minority	1,742	0.871	$[0.856, 0.885]$
1,900 majority	1,812	0.954	$[0.943, 0.963]$

With 100 minority records, the split that a 2,000-record test set with a 5% minority would give, the estimate is 0.880. That is 1.6 points above the true 0.8641, and the interval is 13.6 points wide. The non-private model's 0.9411 from Table 45.2 lies only just above its upper end, 0.936. So on this test set alone the minority's privacy cost could be anywhere from about half a point to fourteen points. With 500 records the interval is 6.5 points wide, and the majority's, from 1,900 records at a higher accuracy, is 2 points wide.

By Equation 45.14 the gap between the groups, measured on the 1,900 and the 100 records, is $0.954 - 0.880 = 0.074$, with a standard error of $\sqrt{0.954 \times 0.046/1{,}900 + 0.88 \times 0.12/100} = 0.0329$. Its 95% interval is therefore $0.074 \pm 1.96 \times 0.0329 = 0.074 \pm 0.064$, from 0.010 to 0.138. The gap is real, but it could be anywhere from one point to fourteen. And this is one training run. By Table 45.2 the minority's accuracy at $\varepsilon = 4$ varies by 0.0211 from run to run on top of this.

45.5. Mitigations and what they cost

Both mechanisms come from DP-SGD's two knobs, the clipping norm and the noise. This section asks whether you can set the knobs, or change the algorithm, so that small groups pay less without spending more privacy. Each fix moves cost around rather than removing it, and it is worth knowing where to.

The first idea is to stop clipping from turning the update. Global scaling multiplies every per-example gradient by the same factor $C/C_{\max}$, where $C_{\max}$ is a bound that no gradient's norm exceeds, and then adds noise σC to the sum. Nothing is clipped, so the direction is right, as the blue arrow of Figure 45.2 shows. Write S_t for the batch and ξ for a standard normal vector.

Proposition 45.4 (Global scaling is a large clipping norm)**.** *Suppose no per-example gradient is longer than $C_{\max}$. Then one step of global scaling with learning rate η is one step of DP-SGD with the clipping norm set to $C_{\max}$, noise multiplier σ and learning rate $\eta C/C_{\max}$:*

$$\frac{\eta}{B}\Big(\frac{C}{C_{\max}}\sum_{i\in S_t} g_i + \sigma C\xi\Big) = \frac{\eta C/C_{\max}}{B}\Big(\sum_{i\in S_t} g_i + \sigma C_{\max}\xi\Big). \tag{45.16}$$

Both have the same privacy guarantee.

Proof.

1. *Factor out the scale.* Both terms inside the left-hand bracket carry the factor $C/C_{\max}$, since $\sigma C = (C/C_{\max})\,\sigma C_{\max}$. Moving it in front of the bracket gives the right-hand side.
2. *Recognize DP-SGD.* With the clipping norm set to $C_{\max}$ and no gradient longer than $C_{\max}$, DP-SGD clips nothing, so its clipped sum is $\sum_i g_i$ and its noise is $\sigma C_{\max}\xi$. That is the right-hand bracket, with learning rate $\eta C/C_{\max}$.

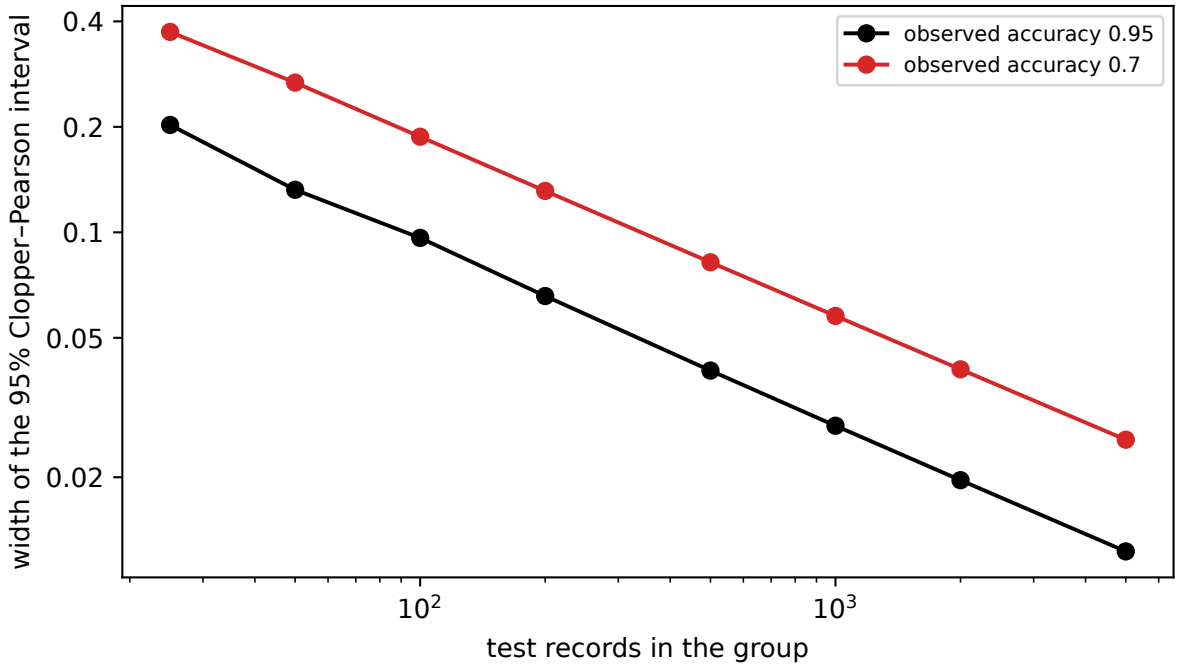


Figure 45.5.: Width of the 95% Clopper–Pearson interval for a group's accuracy against the number of test records in the group, for observed accuracies 0.95 and 0.7, on log scales. The width falls like one over the square root of the number of records.

3. *Privacy.* Each scaled gradient is at most C long and the noise is σC, so the ratio of sensitivity to noise is $1/\sigma$, as for DP-SGD at the same σ, and the same accountant applies.

□

So global scaling gets the direction right and pays in noise. It is the large-C end of the clipping-norm curve of DP-SGD with ηC fixed, where the noise is as large as possible relative to the gradients. The bound must be fixed in advance, because a bound read off the training data would make the sensitivity depend on the data. Esipova et al. (2023) instead clip the rare gradients above $C_{\max}$ and adapt it from a privately estimated count of the gradients that exceed it. They report a smaller privacy cost gap than DP-SGD and earlier mitigations, without group labels.

Group-aware methods do need group labels. Xu, Du & Wu (2021) give groups that are clipped more a larger clipping norm, and Tran, Dinh & Fioretto (2021) penalize the differences between groups in the terms of their decomposition. Both need each training record's group, which is itself sensitive and may not be lawful to collect, and their per-group statistics must be computed privately. The simplest group-aware fix, multiplying a group's loss by a weight, fails outright, because clipping scales each long gradient back to length C whatever the weight.

To give a group more pull, you can give it more records in each batch instead, by sampling its records at a higher rate. Clipping does not undo that, but privacy charges for it. Write $\varepsilon_{\text{acct}}(q, \sigma, T, \delta)$ for the ε that the accountant of DP-SGD gives at sampling rate q, noise multiplier σ, T steps and slack δ.

Proposition 45.5 (Oversampling a group costs that group's privacy)**.** *Suppose group g's records are sampled at rate vq, with $v > 1$, and the others at rate q, by a rule fixed in advance, and the noisy sum is still divided by $B = qn$. Then group g's term in the expected update grows by the factor v, to $v\, s_g m_g$. Each of its records has the privacy of the subsampled Gaussian mechanism (Gaussian noise on a Poisson-sampled batch) at rate vq, and the algorithm is (ε_g, δ)-DP with*

$$\varepsilon_g = \varepsilon_{acct}(vq, \sigma, T, \delta) \;>\; \varepsilon_{acct}(q, \sigma, T, \delta). \tag{45.17}$$

So a fixed target ε requires the noise multiplier for rate vq.

Proof.

1. *The pull.* Record i enters the batch with probability q_i, so the expected clipped sum is $\sum_i q_i \min(1, C/\|g_i\|) g_i$. With $q_i = vq$ for group g and division by qn, regrouping as in Proposition 45.1 gives group g the term $v\, s_g m_g$.
2. *One record's privacy.* The add/remove analysis of a record x in DP-SGD compares the output with and without x. With x present, the output is a mixture of two distributions, one for the steps where x is in the batch and one for the steps where it is not. The other records' terms are the same in both, so the mixture's weights are x's own inclusion probability and its complement. A record sampled at rate vq therefore has the privacy of the subsampled Gaussian at rate vq. The rate may depend on the record's own group, which is part of the record, but not on anything learned from the data.
3. *The guarantee.* (ε, δ)-DP must hold for every record, so the algorithm's ε is the largest of the per-record values, which is the oversampled group's. It exceeds the others' because the accountant's ε grows with the sampling rate.

□

Oversampling therefore buys little at a fixed budget. Fixed at $\varepsilon = 4$ for every record, sampling the minority four times as often requires $\sigma = 4.71$ instead of 1.36. The minority's pull grows by 4 and the noise by $4.71/1.36 = 3.46$, so its signal-to-noise ratio improves by only $4/3.46 = 1.16$, while the majority's falls by 3.46. The majority's records end at $\varepsilon = 0.86$, which is wasted budget. Oversampling without changing σ would silently give the minority $\varepsilon = 20.96$, the weakest privacy for the group already worst served.

Example 45.7. Table 45.7 compares three changes on the running example at $\varepsilon = 4$ for every record.

Table 45.7.: Mitigations at $\varepsilon = 4$ on the running example with a 5% minority (mean ± standard deviation). The first row is from Table 45.2 (3 runs); the clipping norm rows keep $\eta C = 0.25$, 2 runs each; the oversampling row samples minority records at rate $4q$ and sets σ so that they meet $\varepsilon = 4$, 3 runs.

Change	σ	Majority accuracy	Minority accuracy	Gap
None: $C = 0.5$, $\eta = 0.5$	1.36	0.9470 ± 0.0037	0.8477 ± 0.0211	0.0993

Change	σ	Majority accuracy	Minority accuracy	Gap
Smaller clipping norm: $C = 0.05$, $\eta = 5$	1.36	0.9541 ± 0.0008	0.8313 ± 0.0157	0.1229
Larger clipping norm: $C = 5$, $\eta = 0.05$	1.36	0.9463 ± 0.0027	0.8133 ± 0.0166	0.1330
Oversample the minority four times	4.71	0.8669 ± 0.0079	0.8270 ± 0.0111	0.0399

None of the three changes helps the minority. A ten times smaller clipping norm clips the minority more, and its accuracy falls by 1.64 points while the majority's rises by 0.71, the clipping mechanism at work. A ten times larger one clips almost nothing, so the direction is right. But with ηC fixed, the learning rate is ten times smaller, so the noise in each step, $\eta\sigma C/B$, is unchanged while every gradient shorter than 0.5 moves the weights ten times less. By the end of training more than half of the minority's gradients are that short (Table 45.3), so its signal falls against the noise and it loses 3.44 points, the noise mechanism at work. These differences are of the size of the run-to-run spread, 0.0157 to 0.0211, so their sizes are uncertain.

Oversampling narrows the gap from 9.93 to 3.99 points, but by **levelling down**. The minority's accuracy is 2.07 points lower than without it, and the majority's 8.01 points lower. A smaller gap is not by itself a fairer model, so report both groups' accuracy and privacy cost, never the gap alone.

Some of the cost is unavoidable. Learning a group's own rule privately includes estimating statistics of that group's n_g records, such as its average features, and private estimation has a price that no algorithm avoids. The fingerprinting bound, Theorem 27.3 in Lower bounds, shows that releasing the means of d attributes with constant accuracy under (ε, δ)-DP needs at least about $\sqrt{d}/(e^{\varepsilon} - 1)$ records, whatever the mechanism. A smaller group is a smaller dataset, and no clever clipping makes it larger. Feldman (2020) argues the deeper point for any learner. When the data have a long tail of small subpopulations, fitting them well requires memorizing their few examples, which is exactly what DP forbids. What helps is more data from the group, a larger ε with its own costs, or a starting point that already serves the group, such as a public pretrained model (Public data and private fine-tuning).

45.6. Code

The script code/disparate_impact.py trains the running example with a minority group, using the model, clipping and accountant of DP-SGD, samples each group at its own rate and logs per-group gradient statistics. It also computes the two-dimensional example, the quadratic model and the Clopper–Pearson intervals. From the repository root:

```
uv run python code/part05-dp-ml/disparate_impact.py
```

Table 45.8.: The script's output with the default seed.

Experiment	Result
Per-group accuracy, 5% minority	Privacy cost at $\varepsilon = 4$: 0.0059 majority, 0.0934 minority; clipping alone: -0.0051 and 0.0404
Two-dimensional clipping example	Minority's fraction of the update 0.513 $\to$ 0.116; the update turns by 39.0 degrees
Clipping weights during training	Minority's effective share 0.0495 at the start, 0.0251 at step 2,500, 0.0387 at the end
Quadratic noise floor	Formula and simulation agree; 4.07×10^{-4} at share 0.95, 2.96×10^{-2} at 0.01
Group size at $\varepsilon = 4$	Minority privacy cost 0.3488 at 1%, 0.0934 at 5%, 0.0280 at 20%
Clopper–Pearson on 100 minority records	88 correct, $[0.800, 0.936]$, against a true accuracy of 0.8641
Mitigations at $\varepsilon = 4$	No change of C helps the minority; oversampling four times needs $\sigma = 4.71$ and levels both groups down

tests/test_disparate_impact.py checks that the output still matches.

45.7. Summary

Table 45.9.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 45.1	$\pi_g = \text{acc}_g(\text{non-private}) - \mathbb{E}[\text{acc}_g(\text{private})]$	Report every group's accuracy and privacy cost, never the gap alone; choosing ε
Proposition 45.1	$\mathbb{E}[G_t/B] = \sum_g s_g m_g$, $\lVert s_g m_g \rVert \le s_g C$; effective shares $s_g c_g / \sum_h s_h c_h$	Clipping alone is disparate: groups with large inputs or an untaught rule lose share
Proposition 45.2	A fraction s_g of the signal needs $1/s_g^2$ times the steps	Training longer at a fixed ε does not help a small group
Lemma 45.1	$x_{t+1} = (1-a)x_t + \zeta_t$ settles at variance $\omega^2/(a(2-a))$	Any noisy descent near an optimum
Theorem 45.1	$\mathbb{E}[L_g] \approx d_g \eta \tau^2/(4 s_g)$, with $s_g c_g$ under clipping	A group's privacy cost grows like one over its share

Result	Statement	Use
Proposition 45.3	$\hat{p}_g - \hat{p}_h$ plus or minus 1.96 standard errors; run-to-run variance adds	Clopper–Pearson intervals and the spread over several runs for every group
Proposition 45.4	Scaling by $C/C_{\max}$ is DP-SGD with the clipping norm set to $C_{\max}$	A larger C with ηC fixed trades the clipping part for the noise part
Proposition 45.5	Records sampled at vq get the accountant's ε at vq	Account at the highest rate; upweighting a loss is undone by clipping; a group too small needs more data or a public model

45.8. References

- Bagdasaryan, Poursaeed & Shmatikov, "Differential Privacy Has Disparate Impact on Model Accuracy" (NeurIPS 2019). The observation this chapter explains: DP-SGD's accuracy loss falls on under-represented and complex groups, and existing gaps grow.
- Tran, Dinh & Fioretto, "Differentially Private Empirical Risk Minimization under the Fairness Lens" (NeurIPS 2021). Per-step decompositions of a group's excess risk into clipping and noise terms, for output perturbation and DP-SGD, and the role of gradient norms, input norms, the loss Hessian and the distance to the decision boundary.
- Esipova, Ghomi, Luo & Cresswell, "Disparate Impact in Differential Privacy from Gradient Misalignment" (ICLR 2023). Separates clipping's direction error from its length error, finds the direction error the main cause, and proposes global scaling with an adaptive threshold.
- Xu, Du & Wu, "Removing Disparate Impact on Model Accuracy in Differentially Private Stochastic Gradient Descent" (KDD 2021). Group-dependent clipping that equalizes the groups' clipping bias.
- Feldman, "Does Learning Require Memorization? A Short Tale about a Long Tail" (STOC 2020). Why fitting small subpopulations well requires memorizing their examples, and so conflicts with privacy.
- Clopper & Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial" (Biometrika, 1934). The exact interval of Section 45.4.

46. Choosing ε

Every model trained with differential privacy (DP) ships with a budget ε. DP is the guarantee that one record changes the probability of any output by at most a factor e^{ε}, up to a slack δ. An earlier chapter trained the part's running example, a small neural network on 10,000 records, with stochastic gradient descent (SGD) made private by clipping and Gaussian noise, which is DP-SGD. At $\delta = 10^{-5}$ its test accuracy was 0.9175 at $\varepsilon = 1$, 0.9441 at $\varepsilon = 2$, 0.9548 at $\varepsilon = 4$ and 0.9604 at $\varepsilon = 8$, against 0.9610 without privacy. Those accuracies are half of a decision. The other half is what each budget means for the people in the data: what an attacker who sees the model can learn about one of them, and how sure it can be.

The one idea of this chapter is that (ε, δ) is one straight line above a curve. An attacker who asks whether a record was in the training set runs a test with two error rates, and DP bounds the true positive rate by a line in the false positive rate. A training algorithm has a whole trade-off curve, the best true positive rate at each false positive rate, which lies below that line. At $\varepsilon = 8$ the line allows a test that catches every member while almost never accusing a non-member, yet the curve of DP-SGD caps the true positive rate at 0.0589 when the false positive rate is 10^{-3}. So choosing ε well means computing the curve and reading it where your threat lies, as a bound on membership tests, on beliefs or on reconstruction.

The running example is DP-SGD with sampling rate $q = 0.01$, $T = 10{,}000$ steps and $n = 10{,}000$ records, at six budgets from $\varepsilon = 0.5$ to 16. Each budget has the noise multiplier σ that the Rényi differential privacy (RDP) accountant of DP-SGD requires at $\delta = 10^{-5}$. Table 46.1 lists the symbols that recur.

Table 46.1.: The symbols of this chapter.

Symbol	Meaning
P, Q	Output distribution without and with the target record
L	Privacy loss $\ln(Q(y)/P(y))$ of an output y
ω	Distribution of L under Q
$\delta(\varepsilon)$	Privacy profile, the smallest δ at each ε
TPR^*	Trade-off curve, the best true positive rate at each false positive rate

46.1. What one (ε, δ) pair says about a membership test

A model card says "trained with $(\varepsilon, \delta) = (8, 10^{-5})$". What does that number alone forbid an attacker to do? This question comes first because it is the only one you can answer for a model whose training algorithm you have not seen.

Definition 46.1 (Membership test)**.** Fix the training set without the target, D^-, and a target record z^*, so that D^- and $D^- \cup \{z^*\}$ are add/remove neighbours, the relation DP-SGD with Poisson sampling uses. The attacker sees an output y, the model or all its iterates, drawn from $Q = M(D^- \cup \{z^*\})$ if z^* is a member and from $P = M(D^-)$ if not. A **membership test** says "member" on a set S of outputs. Its **false positive rate (FPR)** is $P(S)$, and its **true positive rate (TPR)** is $Q(S)$. A randomized test, which says "member" with probability $\phi(y)$, has rates $\mathbb{E}_P[\phi]$ and $\mathbb{E}_Q[\phi]$. The **privacy loss** of an output is $L(y) = \ln(Q(y)/P(y))$, with $Q(y)$ and $P(y)$ the two densities.

Both rates are probabilities of one event, so DP bounds them directly. By Theorem 18.3, a mechanism is (ε, δ)-DP exactly when every test obeys two lines, the definition applied to S and to its complement with the pair swapped. That theorem names the errors α and β, but this page keeps α for the Rényi order of Section 46.2. Since a probability is also at most 1, every membership test satisfies

$$\mathrm{TPR} \le \min\left\{1,\ e^{\varepsilon}\,\mathrm{FPR} + \delta,\ 1 - e^{-\varepsilon}(1 - \delta - \mathrm{FPR})\right\}. \tag{46.1}$$

The **advantage** $\mathrm{TPR} - \mathrm{FPR}$ measures how much a test beats guessing. It is largest where the two lines cross, and Corollary 18.1 bounds it there:

$$\mathrm{TPR} - \mathrm{FPR} \le \frac{e^{\varepsilon} - 1 + 2\delta}{e^{\varepsilon} + 1}. \tag{46.2}$$

The line that matters is $e^{\varepsilon}\mathrm{FPR} + \delta$, which binds at low FPR. Membership inference judges attacks by their TPR at an FPR of 10^{-3} or below, because an attack that flags members while almost never flagging non-members confidently identifies people. Once $e^{\varepsilon}\mathrm{FPR} + \delta$ reaches 1, the pair allows a perfect TPR, and at $\varepsilon = 8$ that happens from an FPR of $(1 - 10^{-5})e^{-8} = 3.35 \times 10^{-4}$ on. So at $\varepsilon = 8$ the pair says nothing where attacks are judged. An audit refutes a claimed budget only when its measured point lies above the line.

The advantage, by contrast, is read where the two error rates are equal. It is 0.4621 at $\varepsilon = 1$ and 0.9640 at $\varepsilon = 4$, numbers that make $\varepsilon = 4$ sound hopeless. But they are tight only for randomized response, whose curve is exactly the two lines, and DP-SGD is not that mechanism.

Example 46.1. Table 46.2 evaluates Equation 46.1 and Equation 46.2 at the six budgets, all at $\delta = 10^{-5}$. For $\varepsilon = 1$ and an FPR of 0.01, the line $e^{\varepsilon}\mathrm{FPR} + \delta$ gives $e \times 0.01 + 10^{-5} = 0.02719$.

Table 46.2.: The largest TPR and advantage of any membership test that the pair $(\varepsilon, 10^{-5})$ allows.

ε	TPR at FPR 10^{-3}	TPR at FPR 10^{-2}	TPR at FPR 10^{-1}	Advantage
0.5	0.001659	0.0165	0.1649	0.2449

ε	TPR at FPR 10^{-3}	TPR at FPR 10^{-2}	TPR at FPR 10^{-1}	Advantage
1	0.002728	0.02719	0.2718	0.4621
2	0.007399	0.0739	0.7389	0.7616
4	0.05461	0.546	0.9835	0.9640
8	0.9997	0.9997	0.9997	0.9993
16	1	1	1	1.0000

Read this way, $\varepsilon \le 2$ is strong at low FPR, $\varepsilon = 4$ only at very low FPR, and $\varepsilon \ge 8$ is no guarantee against membership inference. For DP-SGD the last two conclusions are far too pessimistic.

46.2. The whole curve of DP-SGD

The pair is one point of a privacy profile, the δ an accountant reports at each ε. The accountant could equally have reported ε at $\delta = 10^{-6}$ or 10^{-3}, and each such pair is another line that every test obeys. The best TPR an attacker can reach is the curve that all those lines enclose, and for DP-SGD it can be computed. Here it is computed exactly and compared with the line and with two cheaper summaries, from RDP and from a Gaussian limit.

DP-SGD reduced a run to one pair of distributions per step, the worst case over models, data and directions. In units of the clipping norm, a step's output is $\mathcal{N}(0, \sigma^2)$ without the target and the mixture $(1-q)\mathcal{N}(0,\sigma^2) + q\,\mathcal{N}(1,\sigma^2)$ with it. The log of the ratio of the two densities is the privacy loss of one step,

$$L_1(y) = \ln\left((1-q) + q\, e^{(2y-1)/(2\sigma^2)}\right). \tag{46.3}$$

Definition 46.2 (Trade-off curve, privacy profile and privacy loss distribution)**.** Let P and Q be as in Definition 46.1. The **trade-off curve** $\mathrm{TPR}^*(\mathrm{FPR})$ is the largest TPR of any test whose false positive rate is at most FPR. The **privacy profile** $\delta(\varepsilon)$ is the smallest δ with $Q(S) \le e^{\varepsilon} P(S) + \delta$ for every event S. The **privacy loss distribution (PLD)** ω is the distribution of $L(y)$ when y is drawn from Q.

The PLD fixes the other two, because the best tests compare the loss with a threshold. Theorem 32.2 proved a version for curves that treat adding and removing the record alike. A membership test needs only adding, which the proof below handles (Dong, Roth & Su 2022).

Theorem 46.1 (The trade-off curve from the privacy loss)**.** *Let P and Q have positive densities, privacy loss L and PLD ω, and write $\mathbb{E}_\omega[X;\ A]$ for the average of X times the indicator of the event A. The best tests say "member" when L exceeds a threshold τ, and they trace the trade-off curve*

$$(\mathrm{FPR}(\tau), \mathrm{TPR}(\tau)) = (\mathbb{E}_\omega[e^{-L};\ L > \tau],\ \omega(L > \tau)), \qquad \tau \in \mathbb{R}. \tag{46.4}$$

The curve is the lower envelope of the lines of the privacy profile,

$$\mathrm{TPR}^*(\mathrm{FPR}) = \inf_{\varepsilon \in \mathbb{R}} \left(e^{\varepsilon}\mathrm{FPR} + \delta(\varepsilon)\right), \qquad \delta(\varepsilon) = Q[L > \varepsilon] - e^{\varepsilon} P[L > \varepsilon], \tag{46.5}$$

and the line at ε touches it at the threshold test with $\tau = \varepsilon$, whose FPR is at most $e^{-\varepsilon} Q[L > \varepsilon]$. For T steps of DP-SGD, ω is the T-fold convolution of the one-step PLD. Rounding each one-step loss up, or down, to a grid gives a curve above, or below, the true one.

Proof. The plan is to find the best tests, show that every line of the profile lies above them and touches them once, and then compute the PLD of the composed run.

1. *The best tests threshold the loss.* The likelihood ratio $Q(y)/P(y)$ is $e^{L(y)}$, which increases with L. By the Neyman–Pearson lemma (Theorem 4.1), the test with the largest TPR at a given FPR says "member" where $L(y) > \tau$, randomizing where $L = \tau$. Its TPR is $Q[L > \tau]$, which is $\omega(L > \tau)$.
2. *Its FPR by a change of measure.* Both densities are positive, so the definition of L gives $P(y) = e^{-L(y)}Q(y)$. The P-probability of an event is therefore the Q-average of e^{-L} over it (Lemma 4.2):
$$P[L > \tau] = \mathbb{E}_Q\big[e^{-L};\ L > \tau\big]. \tag{46.6}$$
With step 1, this is Equation 46.4.
3. *Every line lies above the curve.* By the definition of the profile, every test has $\mathrm{TPR} \le e^{\varepsilon}\mathrm{FPR} + \delta(\varepsilon)$. Taking the smallest right-hand side over ε gives "$\le$" in Equation 46.5.
4. *The profile is a tail of the loss.* We want the event that makes $Q(S) - e^{\varepsilon}P(S)$ largest. Since $P(y) = e^{-L(y)}Q(y)$, adding an output with $L(y) \le \varepsilon$ adds $Q(y) - e^{\varepsilon}P(y) = Q(y)(1 - e^{\varepsilon - L(y)})$, which is not positive, and removing one with $L(y) > \varepsilon$ removes a positive amount. So the best event is $\{L > \varepsilon\}$, which gives the formula for $\delta(\varepsilon)$, as in Lemma 20.2.
5. *Each line touches the curve.* By step 4 the threshold test with $\tau = \varepsilon$ has $\mathrm{TPR} = e^{\varepsilon}\mathrm{FPR} + \delta(\varepsilon)$, so it lies on the line. Every FPR is reached by some threshold test, with randomization at the threshold, so the infimum is attained at every FPR. On its event $e^{-L} < e^{-\varepsilon}$, so by step 2 its FPR is at most $e^{-\varepsilon}Q[L > \varepsilon]$.
6. *Composition adds losses.* The steps use independent noise, so the total loss is the sum of the per-step losses (Lemma 19.2). The worst-case pair is the same at every step, so under Q these losses are independent with one distribution, and ω is its T-fold convolution (Numerical accounting).
7. *A guaranteed bracket.* Rounding each one-step loss up to a grid makes every output look more revealing, so the rounded pair is less private and its curve lies above the true one (Numerical accounting). Rounding down gives a curve below it.

□

The RDP accountant that set σ also gives a curve, at no extra cost. A test turns the two distributions into two coins, and processing cannot increase a Rényi divergence, so the coins must stay within the run's RDP budget at every order α.

Proposition 46.1 (A trade-off bound from the RDP curve)**.** *If $D_\alpha(Q\|P) \le \varepsilon(\alpha)$ at each order $\alpha > 1$, as the RDP accountant of the whole run certifies, then every test satisfies*
$$\mathrm{TPR} \le \min_{\alpha > 1} \big(\mathrm{FPR} \cdot e^{\varepsilon(\alpha)}\big)^{(\alpha-1)/\alpha}. \tag{46.7}$$

Proof.

1. *Reduce the test to two coins.* The test maps Q to a coin that says "member" with probability TPR, and P to one that does so with probability FPR. Post-processing cannot increase a Rényi divergence (Theorem 29.3), so at each order
$$e^{(\alpha-1)\varepsilon(\alpha)} \ge \mathrm{TPR}^{\alpha}\mathrm{FPR}^{1-\alpha} + (1-\mathrm{TPR})^{\alpha}(1-\mathrm{FPR})^{1-\alpha}. \tag{46.8}$$

2. *Drop a term.* The second term is not negative, so dropping it keeps the inequality, and $\mathrm{TPR}^{\alpha} \le e^{(\alpha-1)\varepsilon(\alpha)}\mathrm{FPR}^{\alpha-1}$.
3. *Solve for TPR.* The power $1/\alpha$ gives the bound at order α, and since it holds at every order, it holds at the best one.

□

Theorem 32.6 keeps the dropped term and gives the best curve that RDP allows; Equation 46.7 is looser but closed-form at each order. The Gaussian summary is a limit. By Theorem 32.5, as T grows with $q\sqrt{T}$ fixed, the curve approaches that of $\mathcal{N}(0,1)$ against $\mathcal{N}(\mu,1)$ (Theorem 4.3). With Φ the standard normal distribution function, this Gaussian differential privacy (GDP) approximation is

$$\mathrm{TPR}^{*}(\mathrm{FPR}) \approx 1-\Phi(\Phi^{-1}(1-\mathrm{FPR})-\mu), \qquad \mu = q\sqrt{T(e^{1/\sigma^2}-1)}, \tag{46.9}$$

and it is not a bound. Table 46.3 gives each run's σ and what the three accountants report. Against the upper end of the PLD bracket, the RDP accountant overstates ε by between 0.035 (at $\varepsilon = 0.5$) and 0.264 (at $\varepsilon = 4$), and by 0.58 at $\varepsilon = 8$.

Table 46.3.: The running example's noise multipliers at $q = 0.01$, $T = 10{,}000$, $\delta = 10^{-5}$, and the ε each accountant reports; runs are named by their target. The PLD bracket runs from the loss rounded down, for adding the record, to the loss rounded up, for adding or removing it.

Target ε (RDP)	σ	RDP ε	PLD ε, bracket	GDP μ	GDP ε
0.5	7.72	0.500	[0.445, 0.465]	0.130	0.454
1	4.13	0.999	[0.903, 0.923]	0.246	0.909
2	2.28	1.998	[1.825, 1.845]	0.461	1.819
4	1.36	3.993	[3.629, 3.729]	0.847	3.615
8	0.917	7.997	[7.317, 7.417]	1.511	7.116
16	0.695	15.955	[14.582, 14.682]	2.632	14.097

The pair overstates the attacker, and more so the larger ε: at an FPR of 10^{-3} the factor is about 1.2 at $\varepsilon = 1$, about 4 at $\varepsilon = 4$ and about 17 at $\varepsilon = 8$ (Table 46.4). Figure 46.1 shows why. The curve of DP-SGD is smooth and close to a Gaussian curve, and by Theorem 46.1 the line reported at $\delta = 10^{-5}$ touches it at an FPR of at most $e^{-\varepsilon}Q[L > \varepsilon]$, below 10^{-4} for every run here (Table 46.8). Everywhere else the line is a loose summary.

The RDP bound stays looser than the exact curve, because one Rényi order summarizes the loss by one moment. GDP is close but not on the safe side. Up to $\varepsilon = 8$ it lies inside the exact bracket at an FPR of 10^{-3}, but at $\varepsilon = 16$ it overshoots, because $\mu = 2.632$ is far from the regime where the limit holds. Use it to explain, not to certify.

Example 46.2. Table 46.4 gives the four answers at an FPR of 10^{-3}. At $\varepsilon = 8$ the pair allows 0.9997, the RDP bound 0.09016 and the curve at most 0.0589.

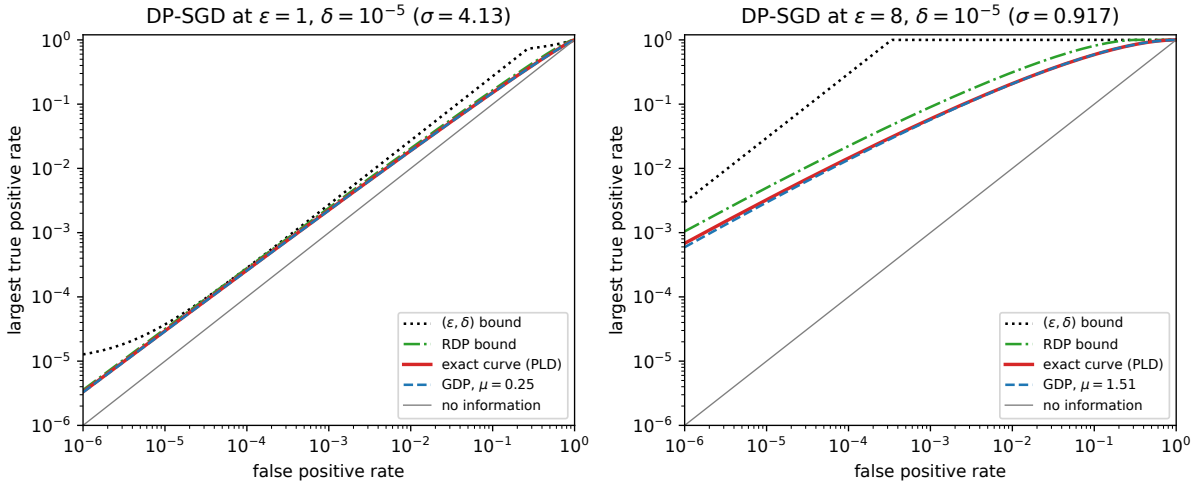


Figure 46.1.: The largest TPR of a membership test against the running example at $\varepsilon = 1$ (left) and 8 (right), on log axes: from the pair (dotted), from the RDP curve (dash-dotted), exactly from the pessimistic PLD (solid red), and from GDP (dashed). The grey diagonal is an attacker with no information.

Table 46.4.: The largest TPR of a membership test against the running example's DP-SGD at FPR 10^{-3}, $\delta = 10^{-5}$.

ε	Pair, Equation 46.1	RDP, Equation 46.7	Exact (PLD bracket)	GDP, Equation 46.9
0.5	0.001659	0.00161	[0.001524, 0.001554]	0.001537
1	0.002728	0.002427	[0.002209, 0.00225]	0.002224
2	0.007399	0.005033	[0.004267, 0.004342]	0.004273
4	0.05461	0.01668	[0.01215, 0.01312]	0.01244
8	0.9997	0.09016	[0.05555, 0.0589]	0.05719
16	1	0.4971	[0.2714, 0.2808]	0.3234

Table 46.5 gives the exact bracket at higher FPR, and the advantage. There the pair allows 0.9640 at $\varepsilon = 4$, while DP-SGD allows at most 0.3429.

Table 46.5.: The exact trade-off curve of the running example at two more false positive rates, and its largest advantage.

ε	Exact TPR at FPR 10^{-2}	Exact TPR at FPR 10^{-1}	Exact advantage
0.5	[0.01391, 0.01418]	[0.1236, 0.126]	[0.0472, 0.0567]
1	[0.01859, 0.01893]	[0.1489, 0.1515]	[0.0933, 0.1023]

ε	Exact TPR at FPR 10^{-2}	Exact TPR at FPR 10^{-1}	Exact advantage
2	[0.03088, 0.03141]	[0.2043, 0.2076]	[0.1778, 0.1859]
4	[0.06739, 0.0724]	[0.3211, 0.3416]	[0.3094, 0.3429]
8	[0.1987, 0.2092]	[0.5682, 0.5901]	[0.5271, 0.5498]
16	[0.5447, 0.5588]	[0.8637, 0.8752]	[0.7647, 0.7758]

The ratio TPR/FPR, the factor by which a flag is more likely on a member, grows only slowly as the FPR falls (Figure 46.2). At $\varepsilon = 8$ it is about 21 at an FPR of 10^{-2} and about 59 at 10^{-3}, nowhere near $e^8 \approx 2{,}981$.

The curve is the worst case of an attacker that sees every iterate and targets a record whose gradient is as large as clipping allows, and Nasr et al. (2021) showed that such an attacker reaches it. A typical record seen through the final model is safer. Table 46.6 runs the loss-threshold attack of Membership inference on one model per budget, with the 10,000 training records as members and the 20,000 test records as non-members.

Table 46.6.: A loss-threshold membership attack on one trained model per budget. The best advantage is the largest TPR − FPR over 999 thresholds.

ε	TPR at FPR 10^{-3}	TPR at FPR 10^{-2}	TPR at FPR 10^{-1}	Best advantage
0.5	0.0010	0.0102	0.0989	0.0155
1	0.0013	0.0108	0.1019	0.0085
2	0.0011	0.0096	0.0983	0.0059
4	0.0008	0.0072	0.0963	0.0116
8	0.0007	0.0098	0.1007	0.0162
16	0.0013	0.0102	0.0992	0.0158
Non-private	0.0004	0.0099	0.0956	0.0221

The attack is at chance at every budget, and nearly so without privacy, because the network generalizes. That is no guarantee, and Auditing private training builds attacks that come close to the curve.

46.3. The Bayesian reading

People who are not statisticians rarely think in error rates. They ask “if someone sees the model, how sure can they be that I was in the data?”, which is a question about posterior probabilities. Read as beliefs, the curve answers it, and the answer depends on what the attacker believed before.

Bayes’ rule in odds form says that the posterior odds of membership are the prior odds times the likelihood ratio $e^{L(y)}$ (Theorem 4.6). So an attacker with prior p believes afterwards that the

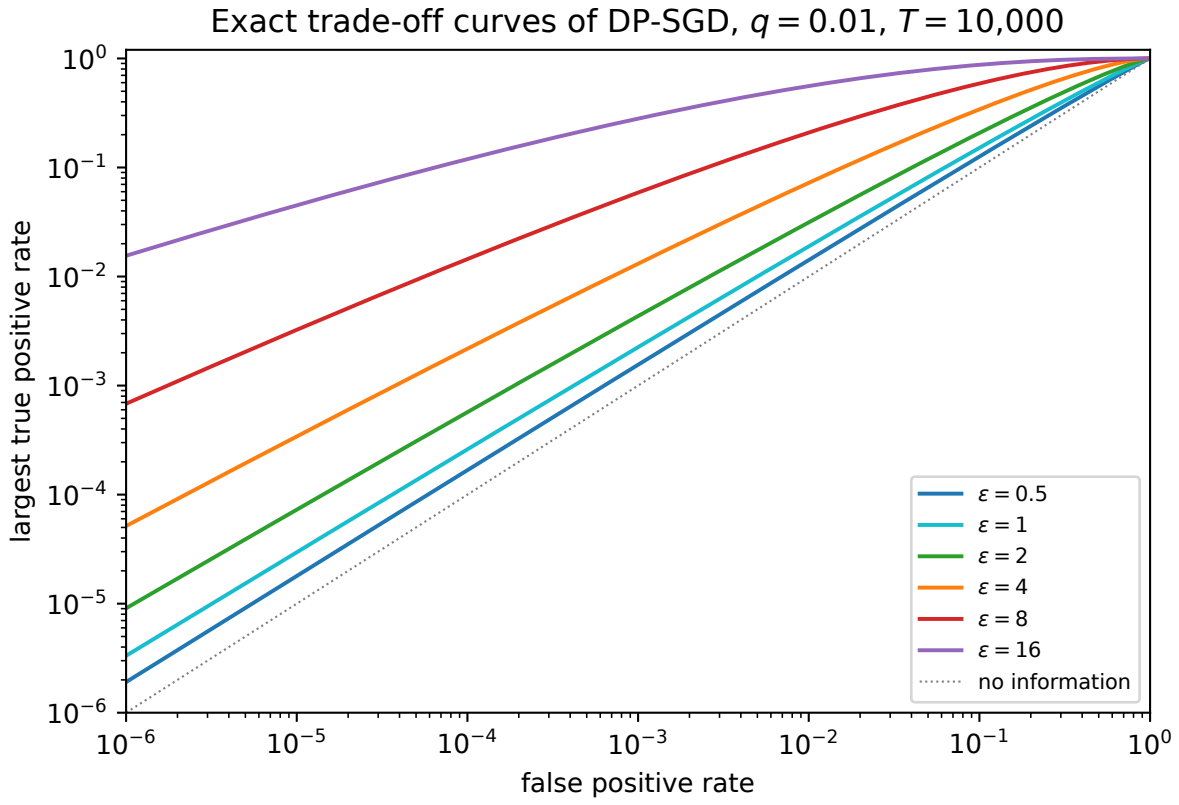


Figure 46.2.: The exact trade-off curves (pessimistic PLD) of the running example at the six budgets, on log axes.

target is a member with probability $pe^L/(pe^L + 1 - p)$, which grows with the loss. Wherever the loss is at most ε, Theorem 18.4 bounds this posterior:

$$\Pr[\text{member} \mid y] \le \frac{p\, e^{\varepsilon}}{p\, e^{\varepsilon} + 1 - p}. \tag{46.10}$$

Under (ε, δ)-DP the same theorem lets the bound fail only on rare outputs, those whose loss exceeds a higher level. An attacker who acts only on a flag, the test's verdict "member", also faces the base rate p, by the same odds rule.

Proposition 46.2 (The precision of a flag)**.** *Let a membership test have rates TPR and FPR, and let the attacker's prior be p. A flag ("the test said member") is right with probability*

$$\Pr[\textit{member} \mid \textit{flag}] = \frac{p\,\mathrm{TPR}}{p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR}} \le \frac{p\,(e^{\varepsilon}\mathrm{FPR} + \delta)}{p\,(e^{\varepsilon}\mathrm{FPR} + \delta) + (1-p)\,\mathrm{FPR}}, \tag{46.11}$$

where the bound holds if the mechanism is (ε, δ)-DP. With $\delta = 0$ the bound equals Equation 46.10.

Proof.

1. *The odds of a flag.* A flag has probability TPR for a member and FPR for a non-member, so the posterior odds are $\frac{p}{1-p} \cdot \frac{\mathrm{TPR}}{\mathrm{FPR}}$. Odds o convert to the probability $o/(1+o)$, which gives the equality.
2. *The bound.* The precision increases with TPR, so replacing TPR by its bound from Equation 46.1 can only increase it.
3. *Pure DP.* With $\delta = 0$ the FPR cancels, leaving $pe^{\varepsilon}/(pe^{\varepsilon} + 1 - p)$. A flag is never more convincing than the most convincing single output.

□

DP bounds the change of belief, not the belief, as Table 46.7 shows at three priors. An attacker who thinks there is a 1% chance that you are in the data can reach at most 2.67% at $\varepsilon = 1$, but 35.55% at $\varepsilon = 4$ and 96.79%, near certainty, at $\varepsilon = 8$.

Table 46.7.: The largest posterior probability of membership when the privacy loss is at most ε, Equation 46.10.

ε	Prior 0.01	Prior 0.1	Prior 0.5
0.5	0.0164	0.1548	0.6225
1	0.0267	0.2320	0.7311
2	0.0695	0.4509	0.8808
4	0.3555	0.8585	0.9820
8	0.9679	0.9970	0.9997
16	1.0000	1.0000	1.0000

But only outputs whose loss is ε attain the bound, and for DP-SGD the realized loss, drawn from ω, is usually much smaller (Table 46.8). In the $\varepsilon = 8$ run, a worst-case member's total loss is at most 1.136 half the time, which moves a prior of 1% to at most 0.0305. Only once in a thousand

does it exceed 5.971 (posterior 0.7984). The chance that the loss exceeds the reported ε, in the column $Q[L > \varepsilon]$, is of the order of δ.

Table 46.8.: The total privacy loss of a worst-case member, from the pessimistic PLD (so slightly overstated), and the posterior it produces from a prior of 1%.

Run	Median loss	Loss exceeded w.p. 10^{-3}	Loss exceeded w.p. 10^{-5}	$Q[L > \varepsilon]$	Posterior from prior 0.01: median, 10^{-3}, 10^{-5}
$\varepsilon = 0.5$	0.018	0.421	0.575	1.10×10^{-4}	0.0102, 0.0152, 0.0176
$\varepsilon = 1$	0.040	0.802	1.093	5.04×10^{-5}	0.0104, 0.0220, 0.0293
$\varepsilon = 2$	0.115	1.549	2.099	2.52×10^{-5}	0.0112, 0.0454, 0.0762
$\varepsilon = 4$	0.401	3.059	4.092	1.59×10^{-5}	0.0149, 0.1771, 0.3768
$\varepsilon = 8$	1.136	5.971	7.910	7.86×10^{-6}	0.0305, 0.7984, 0.9649
$\varepsilon = 16$	3.032	11.661	15.314	3.78×10^{-6}	0.1732, 0.9991, 1.0000

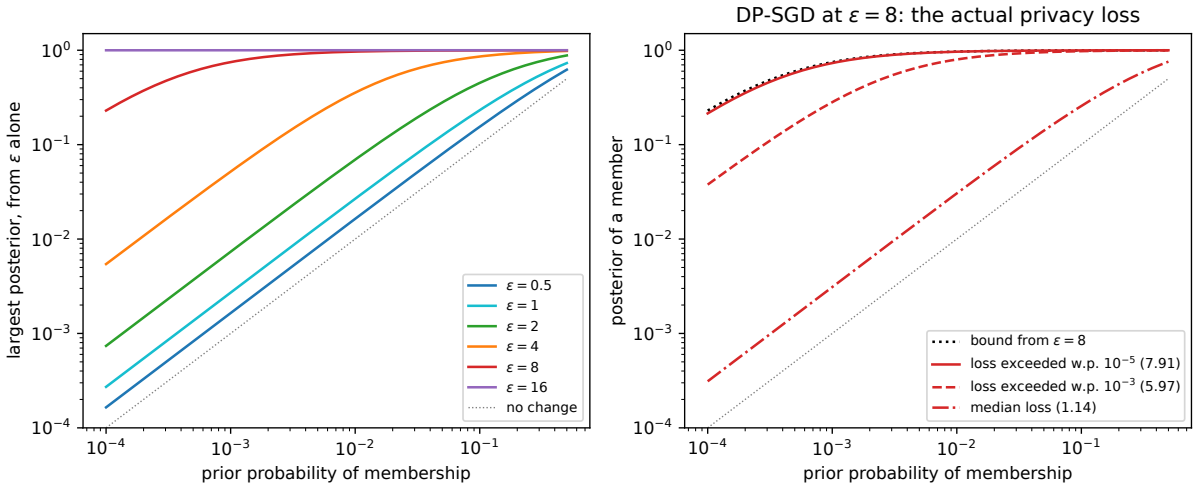


Figure 46.3.: Left: the largest posterior against the prior, Equation 46.10, for the six budgets. Right: for the $\varepsilon = 8$ run, the bound (dotted) and the posterior of a member whose loss is the median of the PLD or is exceeded with probability 10^{-3} or 10^{-5}.

Example 46.3. An insider who knows the training set holds you or a colleague, not both, has prior 0.5. Against the $\varepsilon = 2$ run, Table 46.7 caps its posterior at 0.8808. A stranger with prior 0.01 reaches at most 0.0695 against the same run and, for a typical member, $0.01e^{0.115}/(0.01e^{0.115} + 0.99) = 0.0112$. The guarantee is the same, and what it protects against depends on what the attacker knew before.

Base rates also make flags unconvincing, because in Equation 46.11 the prior enters through $p/(1-p)$. At $\varepsilon = 8$, prior 1% and an FPR of 10^{-2}, the pair allows a precision of 0.5024, while the exact curve allows 0.1744, so five flags out of six are false.

46.4. Reconstruction

Membership is a strict standard for the defender, because the attacker is handed the target record and only has to decide whether it was used. Many harms need more, such as recovering a diagnosis or a face the attacker does not know. The same curve bounds them too, read at a false positive rate set by the attacker's uncertainty beforehand.

Definition 46.3 (Reconstruction robustness (Balle, Cherubin & Hayes 2022))**.** An **informed adversary** knows every training record except the target Z, knows a prior π from which Z was drawn, sees the trained model, and outputs a guess. It succeeds if the guess is within an error radius ξ of Z, as measured by an error function err. The **baseline** is the best success without the model,

$$\kappa = \sup_{z_0} \Pr_{Z\sim\pi}\left[\mathrm{err}(Z, z_0) \le \xi\right], \tag{46.12}$$

for example $\kappa = 1/m$ when Z is uniform over m candidates and only an exact guess counts.

The proof below averages one membership test per candidate over the prior, and moving that average inside the curve needs the curve to be concave. Theorem 4.2 proved that every trade-off function, the smallest false negative rate at each FPR, is convex and non-increasing, and TPR^* is one minus such a function, so it is concave and non-decreasing.

Theorem 46.2 (Reconstruction is membership at the baseline)**.** *If every neighbouring pair of M has trade-off curve at or below TPR^*, then every informed reconstruction attack succeeds with probability at most*

$$\Pr[\textit{success}] \le \mathrm{TPR}^*(\kappa). \tag{46.13}$$

With the pair (ε, δ) this is $e^{\varepsilon}\kappa + \delta$, and with one RDP order it is $(\kappa e^{\varepsilon(\alpha)})^{(\alpha-1)/\alpha}$.

Proof. The plan is to bound each candidate's test by the curve, then average over the prior, using concavity to bring the average inside.

1. *One test per candidate.* Fix D^- and the attack $\mathcal{A}$. For a candidate z, let S_z be the set of models θ for which $\mathcal{A}(\theta)$ is within ξ of z. The attack succeeds on z exactly when the model trained on $D^- \cup \{z\}$ falls in S_z.
2. *Bound each test by the curve.* "Say member when $\theta \in S_z$" is a membership test for z, with $Q_z = M(D^- \cup \{z\})$ and $P = M(D^-)$. Its FPR is $\mathrm{FPR}_z = P(S_z)$, so $Q_z(S_z) \le \mathrm{TPR}^*(\mathrm{FPR}_z)$.
3. *Average over the prior.* The success probability is the average of $Q_Z(S_Z)$ over $Z \sim \pi$. Because the curve is concave, Jensen's inequality moves the average inside it:

$$\mathbb{E}_Z[Q_Z(S_Z)] \le \mathbb{E}_Z[\mathrm{TPR}^*(\mathrm{FPR}_Z)] \le \mathrm{TPR}^*(\mathbb{E}_Z[\mathrm{FPR}_Z]). \tag{46.14}$$

4. *The average FPR is at most the baseline.* The average FPR is the chance that the attack succeeds on a model trained without Z. That model is independent of Z, and for each fixed model the guess is a fixed z_0 (a randomized attack averages such guesses), which succeeds with probability at most κ. Since the curve is non-decreasing, Equation 46.13 follows.
5. *The special cases.* Substituting the line of Equation 46.1, or Equation 46.7 at one order, for TPR^* gives the closed forms.

□

Balle, Cherubin & Hayes state the pure-DP and RDP forms, and the (ε, δ) form is a corollary given here. With the exact curve of DP-SGD it is the bound of Hayes, Mahloujifar & Balle (2023), who found it close to their best attack. They also found that hyperparameters with the same ε have different curves and so different reconstruction success. A large prior uncertainty is a small FPR, which is where the curve of DP-SGD lies far below the line.

A small baseline is an honest assumption only if you can defend it. If the attacker can narrow the target to two candidates, $\kappa = 0.5$ and reconstruction is as easy as membership at an FPR of 0.5, so a defensible claim states the prior. Guo, Karrer, Chaudhuri & van der Maaten (2022) bound continuous targets differently. Under RDP of order 2 at level ε, an unbiased reconstruction of a target in an interval of width w has expected squared error at least $w^2/(4(e^\varepsilon - 1))$, and Fisher information gives a tighter bound. Section N.4 derives the first and shows why it needs unbiasedness.

Example 46.4. Choosing one of m equally likely candidates is reconstruction with $\kappa = 1/m$, so its success bound is the curve at an FPR of $1/m$, read from Table 46.4 and Table 46.5. At $\varepsilon = 8$ the pair gives no bound for any m up to 1,000, but the curve caps the success among 1,000 candidates at 0.0589, against 0.001 by guessing. With only ten candidates the attacker already knows a lot: at $\varepsilon = 4$ the curve allows 0.3416 against a baseline of 0.1. Among a thousand candidates even $\varepsilon = 16$ keeps the success at 0.2808, as Balle, Cherubin & Hayes also observed in experiments.

46.5. Interpreting δ

Every number so far was at $\delta = 10^{-5}$. What does δ mean, how small must it be, and what does a smaller one cost in ε for DP-SGD? The answer depends on the losses at which a mechanism uses up its δ.

The worst way to spend it is to publish records. Proposition 18.3 showed that the mechanism that publishes each record in full, independently with probability δ, is $(0, \delta)$-DP. On n records it publishes at least one with probability

$$1 - (1 - \delta)^n \approx 1 - e^{-n\delta}. \tag{46.15}$$

So $\delta = 1/n$ permits exposing a record with probability about 0.63, and a meaningful guarantee needs $n\delta$ well below 1. That mechanism spends δ at infinite loss, since a published record is identified with certainty. So the National Institute of Standards and Technology (NIST) prefers guarantees that rule out such failure, such as zero-concentrated differential privacy (zCDP). DP-SGD spends δ at moderate losses, because $\delta(\varepsilon)$ is at most the tail $Q[L > \varepsilon]$ (Theorem 46.1),

which is thin (Table 46.8). So for DP-SGD δ is not a probability of catastrophe, and shrinking it raises ε only moderately.

How small? NIST SP 800-226 (Near, Darais, Lefkovitz & Howarth 2025) calls $\delta \leq 1/n^2$ a common recommendation and δ above 10^{-5} suspicious. The running example has $n\delta = 0.1$, acceptable for DP-SGD but not by the $1/n^2 = 10^{-8}$ rule. If one person can contribute many records, n is the number of people, and a per-record guarantee is not a per-person one (User-level differential privacy).

Table 46.9.: The ε of three runs of the running example at five values of δ, from the RDP accountant.

σ (run)	$\delta = 10^{-4}$	10^{-5}	10^{-6}	10^{-8}	10^{-10}
4.13 ($\varepsilon = 1$)	0.852	0.999	1.129	1.354	1.548
1.36 ($\varepsilon = 4$)	3.501	3.993	4.432	5.209	5.890
0.917 ($\varepsilon = 8$)	7.088	7.997	8.821	10.301	11.627

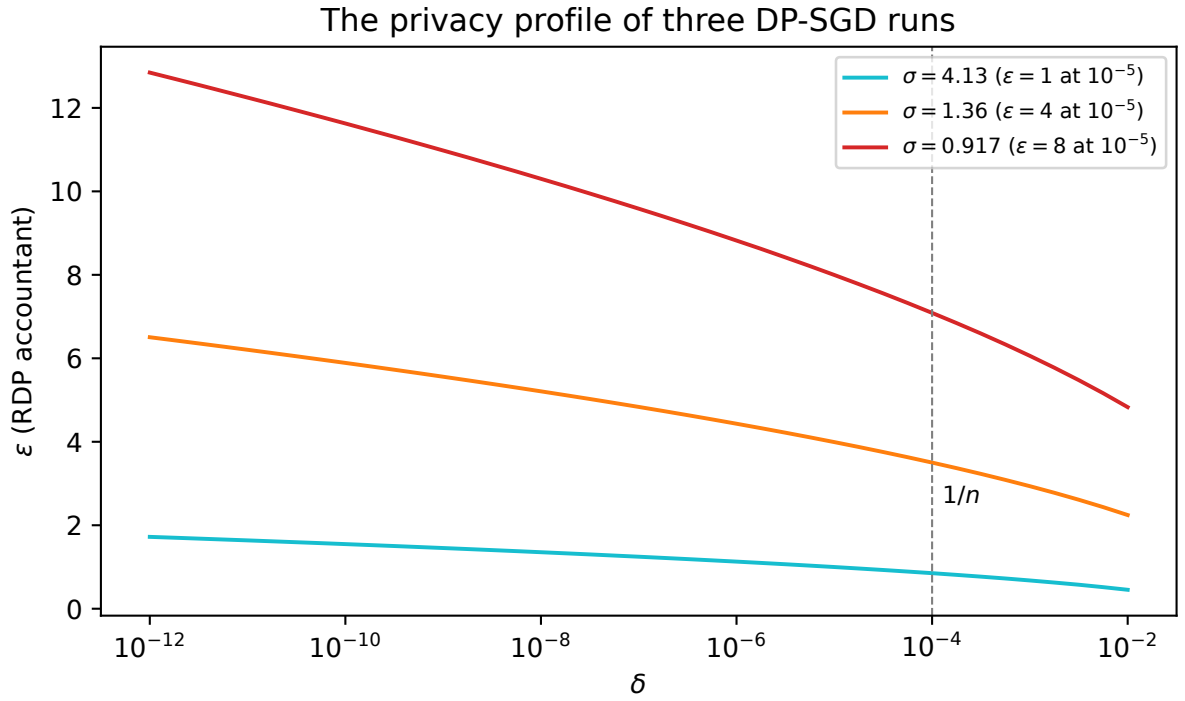


Figure 46.4.: The ε of three runs, as a function of δ, of the running example, from the RDP accountant. The dashed line marks $\delta = 1/n$.

Example 46.5. For $n = 10{,}000$, Equation 46.15 gives the release mechanism an exposure probability of 0.6321 at $\delta = 10^{-4} = 1/n$, 0.09516 at $\delta = 10^{-5}$ and 0.00995 at 10^{-6}. For DP-SGD the same reductions cost little (Table 46.9, Figure 46.4). From 10^{-5} to 10^{-6} the $\varepsilon = 4$ run's budget rises from 3.993 to 4.432, the $1/n^2$ rule makes it $(5.209, 10^{-8})$-DP, and at 10^{-10} it reaches 5.890. Every factor of 100 in δ costs between about 0.2 and 0.9 in ε for the $\varepsilon = 1$ and $\varepsilon = 4$ runs, so when in doubt, use a smaller δ.

46.6. Deployed values and their units

"Industry uses ε between 1 and 10" is a common justification. What have real deployments published, and in which units? Comparing your budget with theirs needs one notion first, because an ε means nothing without it.

Definition 46.4 (Privacy unit and period)**.** The **privacy unit** of a guarantee is what one neighbouring change is: one record, one user's data for a day, or one user's data forever, under add/remove or replace-one. The **period** is the time over which the budget is spent, counting every release and training run that touched the data.

Table 46.10 lists deployments documented in a primary source. The unit, period and accounting differ from row to row, so the values are not comparable without conversion. A zCDP guarantee with parameter ρ converts by the simple bound of Theorem 31.4 (Bun & Steinke 2016),

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}, \tag{46.16}$$

and tighter numerical conversions give somewhat smaller values. The Gboard models were trained on users' devices by follow the regularized leader (FTRL) with noise on sums of gradients (DP-FTRL and matrix factorization).

Table 46.10.: Published parameters of real deployments, as stated in the sources in the references, except the RAPPOR ε_∞, computed here.

Deployment	Privacy unit and period	Published parameters
US Census Bureau, 2020 redistricting data (zCDP)	One person's or housing unit's record, replace-one; one release	$\rho = 2.56$ for persons, 0.07 for housing units; reported as $\varepsilon = 17.14$ plus 2.47, 19.61 in total, at $\delta = 10^{-10}$
Google Gboard language models (zCDP)	One device's data, whole training run	ρ from 0.25 to 1.86, that is ε from 4.49 to 13.69 at $\delta = 10^{-10}$; Spanish model (2022) $(8.9, 10^{-10})$; later models down to $\varepsilon = 0.994$
Apple, local DP on iOS and macOS	One donation; limited donations per day	Emoji $\varepsilon = 4$, one per day; QuickType $\varepsilon = 8$, two per day; Safari domains $\varepsilon = 4$, two per day
Google RAPPOR in Chrome, local	One report (ε_1); one unchanging value (ε_∞)	$\varepsilon_1 = 0.5343$; $\varepsilon_\infty = 2.043$
Microsoft Windows telemetry, local	One user, one round every 6 hours	$\varepsilon = 1$ per round and app; 0.686 after output perturbation; 1.672 across all apps
LinkedIn Audience Engagements API	One member, per analyst, per month	$(0.15, 10^{-10})$ per result; $(34.9, 7 \times 10^{-9})$ per month

Deployment	Privacy unit and period	Published parameters
Facebook URLs dataset (zCDP)	One action; or one user per kind of action, up to its 99th percentile	Per action ε up to 0.45 at $\delta = 10^{-5}$; per user ρ about 0.005, so that $\varepsilon = 0.45$

The conversions are short. With $\rho = 2.56$ and $\delta = 10^{-10}$, Equation 46.16 gives $2.56 + 2\sqrt{2.56 \times 23.026} = 17.915$, the persons' figure of Kazan & Reiter (2022); the Bureau's 17.14 uses a tighter conversion. Local reports add up over time, so QuickType's two donations of $\varepsilon = 8$ cost up to 16 per day, and Tang, Korolova et al. (2017) found that this budget resets daily. RAPPOR reuses one randomized answer for a value that does not change, so all its reports cost at most $\varepsilon_\infty = 2H \ln \frac{1-f/2}{f/2}$, for H hash functions and a probability f of replacing a bit by a coin. At $H = 2$ and $f = 0.75$ that is $4 \ln(5/3) = 2.043$, but a value that changes is reported afresh (Local, shuffle and distributed trust models). In the Facebook release a user with k actions costs k times one action, because zCDP composes by adding ρ.

Only the Gboard rows train a model. They give device-level guarantees for a whole run, with ε between about 1 and 14 at $\delta = 10^{-10}$, while the running example's 1 to 8 are at a much larger δ and per record. NIST notes that many deployments have used $1 < \varepsilon \leq 20$, but practice is no evidence that a value is safe. To compare budgets, convert both to the same unit, neighbouring relation, accounting and δ, as Desfontaines' list of real-world uses does, or say that you did not.

The deployments also show what a published ε leaves out, as four questions to ask of any budget. Does it renew? Rogers et al. (2021) count one RAPPOR report every 30 minutes as $0.534 \times 48 \times 30 = 769$ per month for a user whose value changes, and Microsoft's four rounds a day as $0.686 \times 4 \times 30 = 82.3$. What sits outside it? Apple's encrypted channel and stripped IP addresses protect users, but ε does not count them. What does it assume? LinkedIn's budget per analyst holds only if analysts do not collude, which a contract enforces, not noise. Whom does it cover? The Facebook release protects whole users only up to the 99th percentile of activity. LinkedIn even chose its value from an attack, accepting the 0.115 chance that an analyst who averages a count over 30 days recovers it.

Example 46.6. Is the $\varepsilon = 8$ run "less than half the Census budget"? The Census 17.14 is replace-one at $\delta = 10^{-10}$, the run's 7.997 add/remove at $\delta = 10^{-5}$. At $\delta = 10^{-10}$ the run is at 11.627 (Table 46.9). Replacing a record is a removal and an addition, so in the worst case replace-one doubles an add/remove guarantee (Proposition 18.2). Doubled, the run's 11.627 exceeds 17.14, so after converting the two are of the same order, a weaker claim than the slogan.

46.7. Justifying a choice

You must now pick a target for the running example and write a statement that a sceptical reviewer will check. The earlier sections supply the pieces, and a good statement states what is claimed before giving the number.

Definition 46.5 (A defensible privacy statement)**.** A **defensible privacy statement** for a trained model gives six things.

1. **The unit**, with the period, counting every run and hyperparameter search that touched the data (Hyperparameter tuning).
2. **The threat**: membership, attribute inference or reconstruction, the attacker's prior, and what it observes.
3. **The whole guarantee**: the curve from a tight accountant, with δ well below $1/n$, read at the threat's operating points.
4. **The evidence**: an audit (Auditing private training) consistent with the claim, and a check of the code against the proof's assumptions (DP-SGD).
5. **The utility**: the accuracy curve against ε, and the smallest budget beyond which accuracy stops rising steeply.
6. **The comparison**: the tiers of Ponomareva et al. (2023), in which $\varepsilon \leq 1$ is strong, $\varepsilon \leq 10$ is reasonable for many applications, and above 10 the guarantee alone is not enough.

The first four items answer failures seen earlier. Without the unit, one number can mean protection of a single click or of a whole history. The threat decides where to read the curve, and the pair alone can overstate the attacker 17-fold, which tempts a reader to dismiss DP as vacuous. Whether the code runs the algorithm of the proof is an empirical fact, which audits check.

The utility item matters because the accuracy curve is steeply non-linear (Figure 46.5). Each doubling of ε buys less accuracy and more risk, read as the exact TPR at an FPR of 1%. Going from $\varepsilon = 1$ to 2 gains 2.66 percentage points and moves that TPR from at most 0.01893 to at most 0.03141, while the step from 2 to 4 gains 1.07 points for a TPR of at most 0.0724. The step from 4 to 8 gains only 0.56 points yet nearly triples the TPR, to at most 0.2092. So past $\varepsilon = 4$, accuracy is bought with a much larger increase in worst-case risk. This is also why the tiers are reasonable: below 1 every mechanism is strong by Equation 46.1, from 1 to 10 DP-SGD's curve lies far below the line, and above 10 even the curve allows a TPR above 0.27 at an FPR of 10^{-3}, at $\varepsilon = 16$.

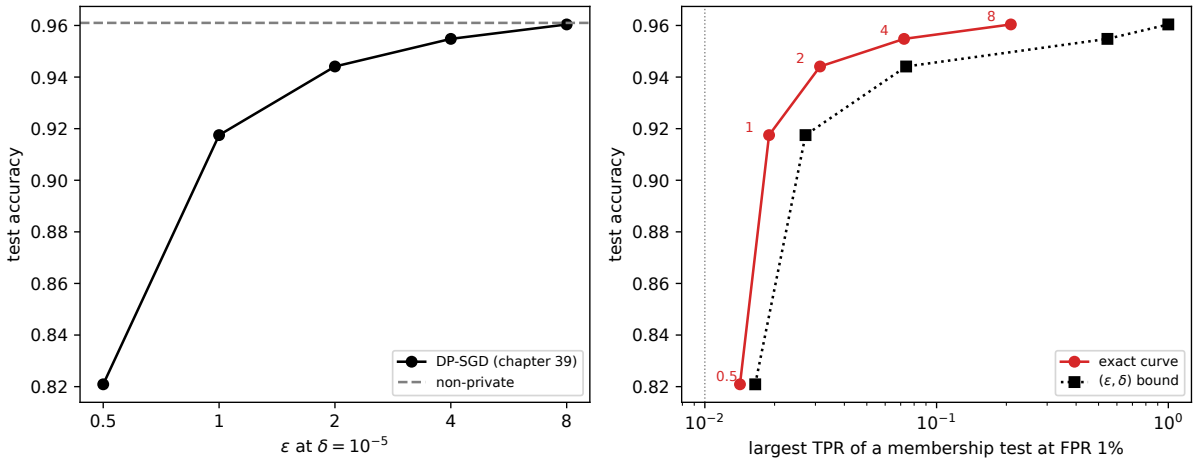


Figure 46.5.: Left: test accuracy of the running example against ε, with the non-private model dashed. Right: the same accuracies against the largest membership TPR at FPR 1%, from the exact curve (circles) and from the pair (squares).

Example 46.7. Following Definition 46.5, a team might choose the $\varepsilon = 4$ run and write:

> The model was trained with DP-SGD (Poisson sampling rate 0.01, noise multiplier 1.36, clipping norm 0.5, 10,000 steps) on 10,000 records, one record per person. The privacy unit is one record under add/remove neighbours; no other run touched the training data. The RDP accountant gives $(3.993, 10^{-5})$-DP and $(5.209, 10^{-8})$-DP; a PLD accountant gives $\varepsilon \leq 3.729$ at $\delta = 10^{-5}$. Against an attacker who knows every other record and sees every iterate, any membership test has TPR at most 0.01312 at FPR 10^{-3} and at most 0.0724 at FPR 10^{-2}, and any attack that picks a record out of 100 equally likely candidates succeeds with probability at most 0.0724. Test accuracy is 0.9548, against 0.9610 without privacy.

Every number in it is checkable, and a reviewer who thinks $\varepsilon = 4$ is too large can now argue with the TPR at a specific FPR rather than with the number.

46.8. Code

The script code/choosing_epsilon.py calibrates the noise multipliers with the RDP accountant, discretizes the one-step loss pessimistically and optimistically, composes it 10,000 times by fast Fourier transform (FFT), and reads off the curves and bounds of this chapter. It also runs the loss-threshold attack and checks the deployment conversions, in two to three minutes. From the repository root:

```
uv run python code/part05-dp-ml/choosing_epsilon.py
```

Table 46.11.: The script's output with the default seed.

Experiment	Result
Accountants	RDP ε = target; PLD brackets 0.02 to 0.1 wide and below it; GDP ε below the PLD lower end except at $\varepsilon = 0.5$ and 1 (Table 46.3)
Membership at FPR 10^{-3}	Pair 0.002728 / 0.05461 / 0.9997 at $\varepsilon = 1$ / 4 / 8; exact curve at most 0.00225 / 0.01312 / 0.0589
Advantage	Pair 0.9640 at $\varepsilon = 4$; exact at most 0.3429
Bayes	Median total loss 0.401 at $\varepsilon = 4$, 1.136 at $\varepsilon = 8$; $Q[L > \varepsilon]$ between 3.78×10^{-6} and 1.10×10^{-4}
Reconstruction, 1,000 candidates	At most 0.0589 at $\varepsilon = 8$, 0.2808 at $\varepsilon = 16$
δ	$\varepsilon = 4$ run: 3.993 at 10^{-5}, 5.209 at 10^{-8}, 5.890 at 10^{-10}
Conversions	$\rho = 2.56$ gives 17.915 at $\delta = 10^{-10}$ by Equation 46.16; RAPPOR $\varepsilon_\infty = 2.043$

Experiment	Result
Deployment checks	Facebook clicks: $\rho = 0.0053$, $\varepsilon = 0.454$ per user; Microsoft one round by Ding et al.'s Theorem 6: 0.569 (reported 0.686); LinkedIn averaging attack at $\varepsilon = 0.15$: 0.115; monthly totals 769 (RAPPOR) and 82.3 (Microsoft)
Loss-threshold attack	TPR within 0.004 of the FPR at every budget and every FPR (Table 46.6)

tests/test_choosing_epsilon.py checks that the output still matches.

46.9. Summary

Table 46.12.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 46.1	$\text{FPR} = P(S)$, $\text{TPR} = Q(S)$	Every reading of a budget
Equation 46.1, Equation 46.2	$\text{TPR} \le e^{\varepsilon}\text{FPR} + \delta$; advantage $\le \frac{e^{\varepsilon}-1+2\delta}{e^{\varepsilon}+1}$	$\varepsilon \le 1$ is strong for any mechanism
Theorem 46.1	Curve $(\mathbb{E}_{\omega}[e^{-L}; L > \tau], \omega(L > \tau))$	Report the TPR at FPR 10^{-3} and 10^{-2}, not just the pair
Proposition 46.1	$\text{TPR} \le \min_{\alpha}(\text{FPR}\, e^{\varepsilon(\alpha)})^{(\alpha-1)/\alpha}$	A free, valid bound from RDP
Equation 46.9	$\text{TPR} \approx 1 - \Phi(\Phi^{-1}(1-\text{FPR}) - \mu)$	To explain, not to certify
Equation 46.10	Posterior $\le pe^{\varepsilon}/(pe^{\varepsilon} + 1 - p)$	State the prior
Proposition 46.2	Precision $p\,\text{TPR}/(p\,\text{TPR} + (1-p)\text{FPR})$	Base rates dominate at small priors
Theorem 46.2	Reconstruction success $\le \text{TPR}^*(\kappa)$	Large budgets can still prevent reconstruction
Equation 46.15	Exposure $1 - (1-\delta)^n$	δ well below $1/n$; for DP-SGD a smaller δ is cheap
Definition 46.4, Equation 46.16	Unit and period; $\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}$	Convert before comparing; ask how a budget renews, what it assumes, whom it covers
Definition 46.5	Unit, threat, curve, evidence, utility, comparison	The smallest budget past the bend in accuracy; above 10, show the curve and an audit

46.10. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (Foundations and Trends in Theoretical Computer Science, 2014). The definition and its testing and Bayesian readings.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015), and Dong, Roth & Su, "Gaussian Differential Privacy" (JRSS B, 2022). The testing view of DP, trade-off curves, the duality of Equation 46.5 and the central limit theorem for composition.
- Koskela, Jälkö & Honkela, "Computing Tight Differential Privacy Guarantees Using FFT" (AISTATS 2020), and Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021). Numerical PLD accounting, which Equation 46.4 reuses for the curve.
- Nasr, Song, Thakurta, Papernot & Carlini, "Adversary Instantiation: Lower Bounds for Differentially Private Machine Learning" (IEEE S&P 2021). Attacks that reach the accountant's guarantee when the adversary controls the target's gradient and sees every iterate.
- Balle, Cherubin & Hayes, "Reconstructing Training Data with Informed Adversaries" (IEEE S&P 2022). Reconstruction robustness and its bounds from pure DP, RDP and zCDP.
- Hayes, Mahloujifar & Balle, "Bounding Training Data Reconstruction in DP-SGD" (NeurIPS 2023). The reconstruction bound from the trade-off curve of DP-SGD, Equation 46.13, and experiments showing it is nearly tight.
- Guo, Karrer, Chaudhuri & van der Maaten, "Bounding Training Data Reconstruction in Private (Deep) Learning" (ICML 2022). Lower bounds on the mean squared error of reconstruction from RDP and Fisher information.
- Near, Darais, Lefkovitz & Howarth, *Guidelines for Evaluating Differential Privacy Guarantees* (NIST SP 800-226, March 2025). Privacy units, $\delta \leq 1/n^2$, the hazards of large ε and of event-level units, and the case for publishing parameters.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023). The three tiers of ε for machine learning and what to report.
- Dwork, Kohli & Mulligan, "Differential Privacy in Practice: Expose your Epsilons!" (Journal of Privacy and Confidentiality, 2019). Interviews with practitioners and the proposal of an Epsilon Registry.
- Desfontaines, "A list of real-world uses of differential privacy" (blog, desfontain.es, updated 2025). A maintained list of deployments, converted to a common δ and unit.
- Abowd et al., "The 2020 Census Disclosure Avoidance System TopDown Algorithm" (Harvard Data Science Review, 2022), and US Census Bureau, "Census Bureau Sets Key Parameters to Protect Privacy in 2020 Census Results" (press release, June 2021). The redistricting data's ρ and ε. Cumings-Menon et al., "Disclosure Avoidance for the 2020 Census Demographic and Housing Characteristics File" (HDSR, 2023). Kazan & Reiter, "Assessing Statistical Disclosure Risk for Differentially Private, Hierarchical Count Data" (arXiv 2022), for the 17.91 conversion.
- Xu, Zhang, Andrew, Choquette-Choo, Kairouz, McMahan, Rosenstock & Zhang, "Federated Learning of Gboard Language Models with Differential Privacy" (ACL 2023 Industry Track); McMahan & Thakurta, "Federated Learning with Formal Differential Privacy Guarantees" (Google AI blog, 2022); Xu & Zhang, "Advances in private training for production on-device

language models" (Google Research blog, 2024). The Gboard rows.

- Apple Differential Privacy Team, "Learning with Privacy at Scale" (Apple Machine Learning Journal, 2017), and Apple, "Differential Privacy Overview" (2017). Tang, Korolova, Bai, Wang & Wang, "Privacy Loss in Apple's Implementation of Differential Privacy on MacOS 10.12" (arXiv 2017). The Apple rows.
- Erlingsson, Pihur & Korolova, "RAPPOR: Randomized Aggregatable Privacy-Preserving Ordinal Response" (CCS 2014). Ding, Kulkarni & Yekhanin, "Collecting Telemetry Data Privately" (NeurIPS 2017), §6 for the deployment and Theorems 6 and 7 for the per-round values. Rogers et al., "LinkedIn's Audience Engagements API: A Privacy Preserving Data Analytics System at Scale" (Journal of Privacy and Confidentiality, 2021), §8 for the non-collusion assumption, the attack behind $\varepsilon = 0.15$, and Table 2, which puts deployments on a monthly scale. Messing et al., "Facebook Privacy-Protected Full URLs Data Set" (Harvard Dataverse, 2020), the privacy section and its table of noise per kind of action. The other rows of Table 46.10.
- Bun & Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (TCC 2016). The conversion Equation 46.16.

47. Auditing private training

Every chapter of this part ended with a number. Train with stochastic gradient descent (SGD) under differential privacy (DP), that is, with per-example clipping and Gaussian noise (DP-SGD), and an accountant turns the configuration into a guarantee. That guarantee concludes a proof about an algorithm on paper, but what ships is code, and the code can quietly differ from the algorithm in a dozen places. The noise may be scaled for the averaged gradient but added to the sum, or the accountant may be handed the wrong dataset size. None of these changes the accuracy much, so the usual tests miss them. This chapter asks whether the trained model is as private as the accountant says.

An **audit** answers by attacking. It plants records of its own, called **canaries**, trains with and without them, and runs a membership test. It measures the test's true positive rate (TPR), how often it flags a canary that was trained on, and its false positive rate (FPR), how often it flags one that was not. The one idea is that DP caps how often any test can be right, so measured error rates imply a **lower bound** on ε. Such a bound cannot prove privacy, but it disproves any claim below it. The chapter then makes the bound bite. It puts confidence intervals on the measured rates, chooses the strongest attack, and gets many trials out of one or two training runs.

The running example is the part's network, a two-layer tanh network trained on the 10,000 records of DP-SGD with sampling rate $q = 0.01$, clipping norm $C = 0.5$, noise multiplier $\sigma = 1$, learning rate 0.5 and $T = 10{,}000$ steps. The Rényi differential privacy (RDP) accountant reports $\varepsilon = 6.71$ at $\delta = 10^{-5}$, and this is the **claim** every audit below tests. Neighbours $D \sim D'$ differ by adding or removing one record, and Table 47.1 lists the symbols.

Table 47.1.: The symbols of this chapter.

Symbol	Meaning
D, $D' = D \cup \{z^*\}$	the dataset without and with the canary z^*
FPR, TPR	how often the attack says "canary in" without and with the canary
n; k_0, k_1	trials per hypothesis; false and true positives
β; ε_{lo}	the probability that an audit's bound is wrong (5%); the bound
q, σ, C, B, η, T	sampling rate, noise multiplier, clipping norm, expected batch size (100), learning rate, steps
x_t; Λ	the canary's projection at step t; the likelihood-ratio statistic
μ	the parameter of Gaussian differential privacy

Symbol	Meaning
m, r, W	canaries, guesses and correct guesses of a one-run audit

47.1. From an attack to a lower bound on ε

You ran an attack and counted how often it was right. This section asks what that lets you conclude about ε, with what confidence, and how much n trials could ever conclude. Every later audit reports its result through this bound.

Definition 47.1 (Audit trials and error rates)**.** Let M be a training algorithm, D a dataset and z^* a **canary** record, and write $D' = D \cup \{z^*\}$. A **trial** runs M on D or on D' with fresh randomness and computes a score s from the output. The attack says "canary in" when $s \geq \tau$, for a threshold τ. Its **false positive rate** and **true positive rate** are

$$\mathrm{FPR} = \Pr[s \geq \tau \mid D], \qquad \mathrm{TPR} = \Pr[s \geq \tau \mid D'], \tag{47.1}$$

and its false negative rate is $\mathrm{FNR} = 1 - \mathrm{TPR}$. An audit runs n trials on each dataset and counts k_0 false positives and k_1 true positives.

The definition of DP limits these rates. By Theorem 18.3 in The definition of differential privacy, if M is (ε, δ)-DP then every test, randomized or not, satisfies both

$$\mathrm{TPR} \leq e^{\varepsilon}\,\mathrm{FPR} + \delta, \qquad 1 - \mathrm{FPR} \leq e^{\varepsilon}\,(1 - \mathrm{TPR}) + \delta. \tag{47.2}$$

The first inequality caps how often the test flags present canaries against how often it flags absent ones. The second swaps the two datasets and applies the cap to the verdict "canary out".

The audit sees only counts, not rates. If the trials are independent, k_0 is a binomial count whose success probability is the FPR, and Theorem 4.7 in Hypothesis testing turns such a count into an interval. Its proof shows that each end, on its own, misses the true rate with probability at most half the interval's failure probability. So write $\overline{\mathrm{FPR}}$ for the **one-sided Clopper–Pearson upper bound** on the FPR and $\underline{\mathrm{TPR}}$ for the lower bound on the TPR, each at level $\beta/2$. They solve

$$\Pr\left[\mathrm{Bin}(n, \overline{\mathrm{FPR}}) \leq k_0\right] = \frac{\beta}{2}, \qquad \Pr\left[\mathrm{Bin}(n, \underline{\mathrm{TPR}}) \geq k_1\right] = \frac{\beta}{2}. \tag{47.3}$$

Putting these pessimistic ends into Equation 47.2, and taking the larger of the two caps, gives a bound that holds with probability $1 - \beta$.

Theorem 47.1 (A certified lower bound on ε)**.** *Suppose the $2n$ trials are independent and the threshold τ was chosen without looking at them. Let $\overline{\mathrm{FPR}}$ and $\underline{\mathrm{TPR}}$ be the one-sided Clopper–Pearson bounds at level $\beta/2$, and let*

$$\varepsilon_{\mathrm{lo}} = \max\left\{ \ln \frac{\underline{\mathrm{TPR}} - \delta}{\overline{\mathrm{FPR}}},\ \ln \frac{1 - \overline{\mathrm{FPR}} - \delta}{1 - \underline{\mathrm{TPR}}} \right\}. \tag{47.4}$$

Then, with probability at least $1 - \beta$, M is not (ε, δ)-DP for any $\varepsilon < \varepsilon_{\mathrm{lo}}$.

Proof. The plan is to solve the two caps for ε, check that pessimistic rates can only lower the result, and then bound the probability that the rates are not pessimistic.

1. *The true rates bound ε.* Dividing each inequality of Equation 47.2 by the factor in front of e^{ε} gives
$$e^{\varepsilon} \geq \frac{\mathrm{TPR} - \delta}{\mathrm{FPR}}, \qquad e^{\varepsilon} \geq \frac{1 - \mathrm{FPR} - \delta}{1 - \mathrm{TPR}}. \tag{47.5}$$
2. *Pessimistic rates lower both fractions.* Both fractions grow with the TPR and shrink with the FPR. So a larger FPR and a smaller TPR can only lower them.
3. *The two bounds hold together with probability $1 - \beta$.* Each bound fails with probability at most $\beta/2$, so by the union bound at least one fails with probability at most β. This needs binomial counts, which is why the trials must be independent and the threshold fixed before the counting.
4. *Combine.* When both bounds hold, step 2 says that $\varepsilon_{\mathrm{lo}}$ is at most the larger logarithm of the two fractions at the true rates. Step 1 says that this is at most the ε of any valid guarantee. So, over the randomness of the trials, the bound exceeds the ε of a valid guarantee with probability at most β.

□

The bound says that no smaller ε is possible, never that this ε is enough. A weak attack gives a small $\varepsilon_{\mathrm{lo}}$ for a leaky model as easily as for a private one. So a bound above the claim refutes it, while a bound below it says only that this attacker did no better. The second form in the maximum rewards tests that rarely miss present canaries.

The hypothesis on the threshold matters in practice. Sweeping the threshold over the same trials and reporting the best bound makes the confidence level wrong, because among many thresholds one is lucky. Every audit in this chapter chooses its threshold on half of the trials and counts on the other half.

Even an attack that never errs certifies only a limited ε, because the bound uses the pessimistic end of each interval. The next result computes that ceiling, which tells you how many trials to budget.

Corollary 47.1 (The ceiling of an audit)**.** *Under the hypotheses of Theorem 47.1, the largest bound that n trials per hypothesis can certify is that of a perfect attack, with $k_0 = 0$ and $k_1 = n$:*

$$\varepsilon_{\max}(n) = \ln \frac{(\beta/2)^{1/n} - \delta}{1 - (\beta/2)^{1/n}} \approx \ln \frac{n}{\ln(2/\beta)}. \tag{47.6}$$

Proof.

1. *A perfect attack gives the best ends.* Fewer successes push both Clopper–Pearson ends down, so $k_0 = 0$ gives the smallest $\overline{\mathrm{FPR}}$ and $k_1 = n$ the largest $\underline{\mathrm{TPR}}$. By step 2 of the proof of Theorem 47.1, these give the largest bound.
2. *Solve for the ends.* With $k_0 = 0$ the upper bound solves $(1 - \overline{\mathrm{FPR}})^n = \beta/2$, so $\overline{\mathrm{FPR}} = 1 - (\beta/2)^{1/n}$. With $k_1 = n$ the lower bound solves $\underline{\mathrm{TPR}}^n = \beta/2$, so $\underline{\mathrm{TPR}} = (\beta/2)^{1/n}$. Substituting into either form of Equation 47.4 gives the exact expression.

3. *Approximate.* Write the power as an exponential, $(\beta/2)^{1/n} = e^{-\ln(2/\beta)/n}$, which is about $1 - \ln(2/\beta)/n$ for large n. So $\overline{\text{FPR}}$ is about $\ln(2/\beta)/n$ while $\underline{\text{TPR}}$ is about 1, and their ratio is about $n/\ln(2/\beta)$. With $\beta = 0.05$, $\ln(2/\beta) = \ln 40 = 3.69$.

□

So each factor of e in the bound costs a factor of e in trials. The reason is that a false positive rate cannot be told apart from zero with fewer than about 3/FPR trials, the **rule of three**. For example, with no false positive in 3,000 trials the 95% upper bound is 9.9808×10^{-4}. Table 47.2 shows what that means. Certifying the running example's 6.71 needs at least 3,038 trials per hypothesis, so 6,076 training runs, even for an attack that never errs. For a real network each run is a full training job.

Table 47.2.: The most a perfect attack can certify with n trials per hypothesis, at 95% confidence and $\delta = 10^{-5}$. Conversely, certifying $\varepsilon = 1$, 2, 4, 6.71 and 8 takes at least 12, 30, 204, 3,038 and 10,999 trials per hypothesis.

Trials per hypothesis n	100	1,000	10,000	100,000	1,000,000
Largest certifiable ε, Equation 47.6	3.28	5.60	7.90	10.21	12.51

Example 47.1. Suppose 1,000 runs with the canary and 1,000 without give 500 true positives and 1 false positive. The point estimate is $\ln \frac{0.5 - 10^{-5}}{0.001} = 6.21$. The Clopper–Pearson bounds at $\beta/2 = 0.025$ are $\overline{\text{FPR}} = 0.00556$ and $\underline{\text{TPR}} = 0.4685$, so one false positive in 1,000 is consistent with a true FPR five times larger. The certified bound is $\ln \frac{0.4685 - 10^{-5}}{0.00556} = 4.43$, almost two less than the point estimate. With ten times the trials and the same rates, the bounds are $\overline{\text{FPR}} = 0.00184$ and $\underline{\text{TPR}} = 0.4902$, and the certified bound rises to 5.59. The price of confidence falls, slowly, as the counts grow.

47.2. Canaries and threat models

A bound is only as strong as the attack behind it, and DP is a promise about the worst pair of datasets and the worst test. A typical training record scored by its loss gives a valid but nearly useless bound, since Membership inference shows that most records are barely exposed. This section asks which canary and which view of training make the strongest audit of DP-SGD, and what that audit faces.

Definition 47.2 (Canaries and threat models)**.** An **input-space canary** is a record (x, y) inserted into the training data like any other, such as a mislabelled example, chosen so that its gradient is large and points where few others do. A **gradient-space canary** is a gradient the auditor writes directly, as if it controlled the training code. A **Dirac canary** (Nasr et al. 2023) is a gradient-space canary equal to C on one coordinate and zero elsewhere, on a coordinate where the other records' gradients are zero. In the **black-box** threat model the attacker sees only the final model, and in the **white-box** model it sees every iterate $\theta_0, \ldots, \theta_T$.

A white-box gradient audit tests the *mechanism*, that is, whether the code clips and adds the noise the proof assumes. A black-box input audit tests the *deployment*, what an attacker with a released model can learn. A white-box attacker can recover every step's noisy gradient sum G_t, which the lemma below defines. DP-SGD moves the weights by the learning rate η times G_t divided by the expected batch size B, so $G_t = -B(\theta_{t+1} - \theta_t)/\eta$. The proof of the next theorem needs to know what this sum looks like along one direction, and the lemma below works it out.

Lemma 47.1 (One DP-SGD step seen along one direction)**.** *Let one DP-SGD step release $G_t = \sum_i \bar{g}_i + \xi_t$, the sum of the clipped gradients $\bar{g}_i$ of the records sampled at step t plus noise $\xi_t \sim \mathcal{N}(0, \sigma^2 C^2 I)$, with Poisson sampling at rate q. Let u be a fixed unit vector. Suppose one record's clipped gradient is exactly Cu at every step, while no other record's gradient has a component along u. Then the projection $x_t = u^\top G_t / C$ satisfies*

$$x_t \sim \mathcal{N}(0, \sigma^2) \text{ without the record}, \qquad x_t \sim (1-q)\,\mathcal{N}(0, \sigma^2) + q\,\mathcal{N}(1, \sigma^2) \text{ with it}, \tag{47.7}$$

independently over the steps.

Proof.

1. *The noise and the other records.* The projection of an isotropic Gaussian vector on a unit vector is a one-dimensional Gaussian with the same standard deviation, so $u^\top \xi_t / C$ is $\mathcal{N}(0, \sigma^2)$. The other records add nothing, because their gradients have no component along u.
2. *The record.* Its gradient Cu adds exactly 1 to x_t when it is sampled, which happens with probability q, independently of the noise.
3. *Independence.* The sampling coin and the noise are drawn afresh at every step, and the record's contribution does not depend on the weights, so different steps are independent.

□

This is the pair that the accountant of Amplification by subsampling analyses, the worst case for one step under add/remove neighbours. A Dirac canary meets the lemma's hypotheses exactly. The strength of a test against this pair is measured on the scale of **Gaussian differential privacy (GDP)**. A mechanism is **μ-GDP** (Definition 32.2) when no test separates its neighbours better than one separates two unit-variance normals whose means are μ apart. That pair's best true positive rate at a given false positive rate is

$$\mathrm{TPR}_\mu(\mathrm{FPR}) = 1 - \Phi\big(\Phi^{-1}(1 - \mathrm{FPR}) - \mu\big), \tag{47.8}$$

where Φ is the standard normal distribution function.

Theorem 47.2 (The white-box Dirac audit)**.** *Let a Dirac canary sit on a coordinate j that no other record's gradient touches, let the attacker see every iterate, and let $x_t = G_{t,j}/C$. Then $x_1, \dots, x_T$ follow Equation 47.7 independently, and the most powerful test at every FPR thresholds the* ***likelihood-ratio statistic***

$$\Lambda = \sum_{t=1}^{T} \ln\Big(1 - q + q\, e^{(x_t - 1/2)/\sigma^2}\Big). \tag{47.9}$$

No attacker who sees the whole training run does better. For large T with $q\sqrt{T}$ fixed, the trade-off curve of Λ approaches that of μ-GDP with $\mu = q\sqrt{T(e^{1/\sigma^2} - 1)}$. The ***sum of projections*** *$\sum_t x_t$ has a curve close to μ-GDP with $\mu_{\text{sum}} = q\sqrt{T}/\sigma$.*

Proof. The plan is to show that the projections follow the lemma's pair, to multiply the steps' likelihood ratios, to check that the rest of the run adds no evidence, and then to read off both statistics' strength.

1. *The canary is the only difference.* Every other record contributes 0 to coordinate j. The canary's gradient has norm exactly C, so clipping leaves it unchanged, and it is the same at every step whatever the weights are. So Lemma 47.1 applies with u the unit vector of coordinate j.
2. *The likelihood ratio of one step.* The squares x^2 cancel in the ratio of two normal densities with the same variance, so $\mathcal{N}(1, \sigma^2)$ over $\mathcal{N}(0, \sigma^2)$ at x is $e^{(x-1/2)/\sigma^2}$. The mixture puts weight $1 - q$ on the null density and q on the shifted one, so its ratio to the null is
$$1 - q + q\, e^{(x-1/2)/\sigma^2}. \tag{47.10}$$
3. *Multiply over the steps.* Independence makes the joint likelihood ratio the product of the per-step ratios, and its logarithm is Equation 47.9. By the Neyman–Pearson lemma (Theorem 4.1), thresholding it is the most powerful test at every FPR.
4. *The rest of the run carries no evidence.* The whole run is a function of the x_t and of randomness with the same distribution under both hypotheses, namely the other records' sampling and the noise on the other coordinates. The weights depend on past x_t, but given those, everything else is drawn the same way with or without the canary.
5. *The two statistics in Gaussian terms.* For Λ, the subsampled Gaussian central limit theorem (CLT), Theorem 32.5 in f-DP and Gaussian DP, gives the curve of T steps of this pair, which the most powerful test attains. For the sum, the mean shifts by the expected number of times the canary is sampled, qT, while the noise has standard deviation $\sigma\sqrt{T}$, and their ratio is $q\sqrt{T}/\sigma$. With the canary the sum also has a little extra variance, $Tq(1 - q)$.

□

The likelihood ratio beats the sum because it weighs each step by its evidence. A step with $x_t = 3$ is much stronger evidence than three steps with $x_t = 1$, since a value three noise standard deviations out is very unlikely without the canary. The ratio of the two μ is $\sigma\sqrt{e^{1/\sigma^2} - 1}$, which is 1.311 at $\sigma = 1$ and tends to 1 as σ grows.

Black-box audits with input canaries are weaker for three reasons. The final model is one function of all the noisy sums, so by post-processing it carries at most as much information as they do, and usually much less. An input canary's gradient shrinks or turns once the model fits it or gives up on it. And it shares directions with the other records' gradients, which act as extra noise. Nasr, Song, Thakurta, Papernot & Carlini (2021) built a ladder of adversaries for DP-SGD and found the bound tight only for the strongest, which sees every iterate and chooses a worst-case dataset and canary.

Example 47.2. At $q = 0.01$, $\sigma = 1$ and $T = 10{,}000$, the likelihood-ratio statistic has $\mu = 0.01 \times \sqrt{10{,}000 \times 1.7183} = 1.3108$. The sum of projections has $\mu_{\text{sum}} = 0.01 \times 100 = 1.00$, or

0.9951 with the extra variance. The Gaussian privacy profile of Corollary 32.1 converts these to $\varepsilon = 6.01$ and 4.35 at $\delta = 10^{-5}$. So choosing the sum lowers the ε the audit can ever reach from 6.01 to 4.35, before a single trial is run.

47.3. Tight audits of the mechanism

The white-box Dirac audit faces the accountant's own pair. This section asks whether, with enough trials, it recovers the accountant's 6.71, and how close it gets with the trials an auditor can afford. The answer explains why audits stop short of their claims.

The smallest ε that a pair of output distributions allows can be read straight off the definition. Only likelihood-ratio thresholds need to be checked, because the Neyman–Pearson lemma says they are the best tests.

Theorem 47.3 (A perfect audit measures the true ε)**.** *Let P and Q be the distributions of the training output without and with the canary. The smallest ε such that $Q(S) \le e^{\varepsilon} P(S) + \delta$ for every event S is*

$$\varepsilon^*(\delta) = \sup_S \ \ln \frac{Q(S) - \delta}{P(S)}, \tag{47.11}$$

the supremum over events with $Q(S) > \delta$, and it may be taken over likelihood-ratio thresholds only. So an audit that thresholds the likelihood ratio, and knows its rates exactly, computes $\varepsilon^*(\delta)$.

Proof.

1. *The supremum is the definition solved for ε.* If ε is at least the supremum, every event's log-ratio is at most ε, which rearranges to $Q(S) \le e^{\varepsilon} P(S) + \delta$. If ε is below it, some event violates the inequality.
2. *Only likelihood-ratio thresholds matter.* For a fixed $P(S)$, the ratio grows with $Q(S)$. By the Neyman–Pearson lemma (Theorem 4.1), among events with a given $P(S)$ the one with the largest $Q(S)$ is a likelihood-ratio threshold, randomized on ties.
3. *A perfect audit.* Reading off exact rates at every likelihood-ratio threshold evaluates the ratio of step 1 at every candidate of step 2, so it computes the supremum.

□

For the Dirac canary this pair is the one the accountant bounds, so a perfect audit converges to the true ε of DP-SGD, not to what an accountant reports. The RDP accountant's 6.71 is a loose upper bound. The numerical accountant of f-DP and Gaussian DP puts the true value between 6.14 and 6.24, and the CLT approximation gives 6.01. So even an audit with infinitely many trials stops about half a unit below 6.71, and that gap belongs to the accountant.

Real audits fall much further short, because of where on the curve the supremum is reached. For a μ-GDP curve that point is explicit.

Proposition 47.1 (Where ε lives on a Gaussian curve)**.** *Let a mechanism's trade-off curve be the μ-GDP curve Equation 47.8, and let $\varepsilon = \varepsilon^*(\delta)$. The supremum Equation 47.11 is reached at the threshold whose rates are*

$$\mathrm{FPR}^* = \Phi\Big(-\frac{\varepsilon}{\mu} - \frac{\mu}{2}\Big), \qquad \mathrm{TPR}^* = \Phi\Big(-\frac{\varepsilon}{\mu} + \frac{\mu}{2}\Big). \tag{47.12}$$

Proof.

1. *The supremum is a tangent.* By Equation 47.11, ε is set by the steepest line through $(0, \delta)$ that still meets the concave curve of TPR against FPR. At the optimum that line touches the curve, so the curve's slope there is e^{ε}.
2. *The slope is the likelihood ratio.* By Proposition 4.1 in Hypothesis testing, the slope at a threshold is the likelihood ratio there. For $\mathcal{N}(\mu, 1)$ against $\mathcal{N}(0, 1)$ at threshold τ it is $e^{\mu\tau - \mu^2/2}$.
3. *Solve for the threshold.* Setting the slope to e^{ε} gives $\tau = \varepsilon/\mu + \mu/2$. Its FPR is $1 - \Phi(\tau) = \Phi(-\tau)$, and its TPR is $1 - \Phi(\tau - \mu) = \Phi(\mu - \tau)$.

□

For the running example, $\mu = 1.3108$ and $\varepsilon = 6.01$ give $\mathrm{FPR}^* = 8.1 \times 10^{-8}$ and $\mathrm{TPR}^* = 4.3 \times 10^{-5}$. So certifying the full ε at $\delta = 10^{-5}$ means measuring a false positive rate below one in ten million. With n trials you can measure rates down to about $3/n$, and there the curve's log-ratio of TPR to FPR is far below ε. For $\mu = 1.3108$, Equation 47.8 gives 2.74 at FPR 10^{-2}, 3.63 at 10^{-3} and 4.38 at 10^{-4}. Seeing ten false positives near 10^{-7} takes about 1.2×10^8 trials without the canary. Figure 47.1 shows the measured curve well inside the $(6.71, 10^{-5})$ limit.

Nasr, Hayes, Steinke, Balle, Tramèr, Jagielski, Carlini & Terzis (2023) avoid the tail by assuming the curve's shape. If the mechanism is μ-GDP, one test at a moderate FPR, where trials are cheap, bounds μ from below, and the Gaussian shape carries that bound into the tail.

Proposition 47.2 (A lower bound on μ from one test)**.** *If M is μ-GDP, every test satisfies*

$$\mu \geq \Phi^{-1}(\mathrm{TPR}) - \Phi^{-1}(\mathrm{FPR}). \tag{47.13}$$

Under the hypotheses of Theorem 47.1, the value μ_{lo} of the right-hand side at $\underline{\mathrm{TPR}}$ and $\overline{\mathrm{FPR}}$ satisfies $\mu \geq \mu_{\mathrm{lo}}$ with probability at least $1 - \beta$.

Proof.

1. *Rearrange the curve.* Under μ-GDP every test's TPR is at most Equation 47.8. The normal distribution is symmetric, so $1 - \Phi(a) = \Phi(-a)$ and $\Phi^{-1}(1 - p) = -\Phi^{-1}(p)$, and the cap becomes
$$\mathrm{TPR} \leq \Phi\big(\Phi^{-1}(\mathrm{FPR}) + \mu\big). \tag{47.14}$$
Applying the increasing function Φ^{-1} to both sides gives Equation 47.13.
2. *Pessimistic rates.* The right-hand side of Equation 47.13 grows with the TPR and shrinks with the FPR, so the Clopper–Pearson ends can only lower it. They hold together with probability at least $1 - \beta$, as in step 3 of the proof of Theorem 47.1.

□

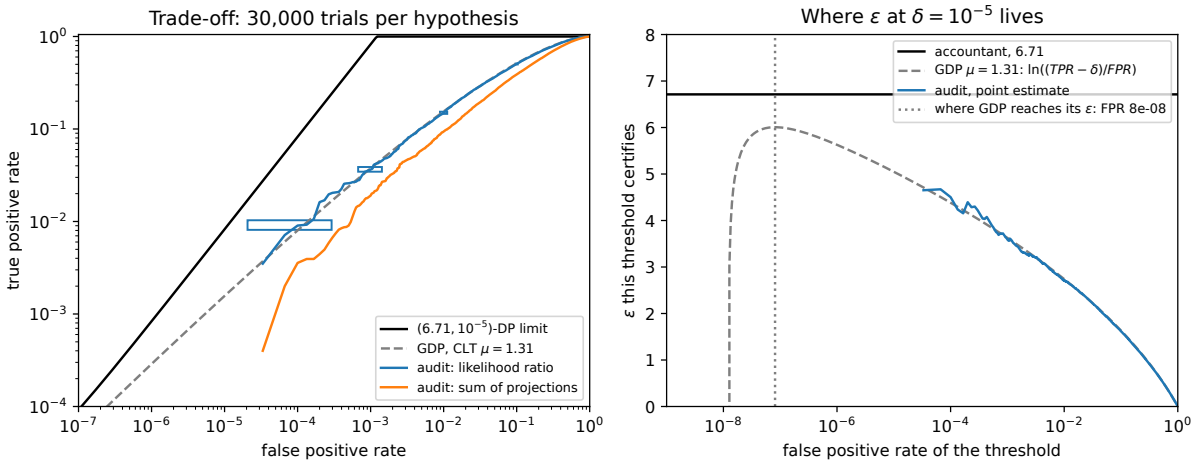


Figure 47.1.: Left: the trade-off of the white-box audit of the running example on log axes, from 30,000 trials per hypothesis. The black line is the most any test can achieve under $(6.71, 10^{-5})$-DP; the dashed grey curve is μ-GDP with the CLT's $\mu = 1.31$. The blue curve is the likelihood-ratio statistic, the orange one the sum of projections; boxes are 95% Clopper–Pearson regions at four thresholds. Right: the ε a threshold would certify if its rates were known exactly, $\ln((\mathrm{TPR} - \delta)/\mathrm{FPR})$, against its false positive rate. The Gaussian curve reaches its ε of 6.01 only at FPR 8×10^{-8} (dotted line), far below the rates a feasible audit can measure.

The bound on μ is rigorous, but its conversion to ε is valid only if the curve really is Gaussian. The subsampled Gaussian of DP-SGD has a heavier tail, so the μ that Equation 47.13 gives depends on where you read it. In the example below, the measured rates give 1.300 at FPR 10^{-1}, 1.303 at 10^{-2}, 1.371 at 10^{-3} and 1.492 at 2×10^{-4}, against the CLT's 1.3108. For this reason Nasr et al. mostly insert the canary at every step, which makes each step an exact Gaussian mechanism, as Section 47.6 does.

Example 47.3. The script simulates the white-box Dirac audit directly from Equation 47.7. Each trial draws 10,000 projections, adds the canary on a $\mathrm{Bin}(10{,}000, 0.01)$ number of steps when it is present, and computes Λ. By Theorem 47.2 this is statistically identical to training the network with a Dirac canary, and thousands of times cheaper.

With 50,000 trials per hypothesis, the measured curve is heavier-tailed than the Gaussian. At FPR 10^{-2} the point value of $\ln((\mathrm{TPR} - \delta)/\mathrm{FPR})$ is 2.73, as the Gaussian predicts, but at 10^{-4} it is 4.84, above the Gaussian's 4.38. Table 47.3 gives the certified bounds, with thresholds chosen on the first half of the trials and counts taken on the second.

Table 47.3.: The white-box Dirac audit of the running example with the likelihood-ratio statistic, against the accountant's $\varepsilon = 6.71$. All bounds hold with probability 95%.

Trials per hypothesis	1,000	3,000	10,000	50,000
$\varepsilon_{\mathrm{lo}}$, Equation 47.4	1.64	1.88	2.84	3.63

Trials per hypothesis	1,000	3,000	10,000	50,000
μ_{lo}, Equation 47.13 with Clopper–Pearson	0.795	1.064	1.147	1.185
ε of μ_{lo}-GDP (assumes the Gaussian shape)	3.36	4.70	5.13	5.33
Ceiling, Equation 47.6, at half the trials	4.91	6.01	7.21	8.82

The rigorous bound reaches 3.63 with 50,000 trials per hypothesis, about half the claim, while the Gaussian-shape estimate climbs to 5.33. The sum of projections needs far more trials for less, 1.97 with 10,000 trials per hypothesis and 3.26 with 100,000. With 1,000,000 it falls to 2.27 in this run, because bounds read in the far tail are noisy, as Figure 47.2 shows. Every number in Table 47.3 is below the claim, as it must be for a correct implementation.

47.4. One-run auditing

The audits so far need one training run per trial, thousands in all, which is out of the question for a model that takes a day to train. Can a single training run give a valid bound? It can, if each canary is a trial.

Definition 47.3 (One-run audit)**.** Insert m canaries, each independently with probability $\frac{1}{2}$, and write $S_i = +1$ if canary i is in the training set and $S_i = -1$ if not. Train once and score every canary. Guess $\hat{S}_i = +1$ ("in") for the $r/2$ highest scores and $\hat{S}_i = -1$ ("out") for the $r/2$ lowest, and abstain, $\hat{S}_i = 0$, on the rest. Let W be the number of correct guesses among the r made.

Each guess is about one fair coin, made from an output that is private in that coin. DP lets the output make one value of the coin at most e^{ε} times as likely as the other, so no guess can be right more often than $e^{\varepsilon}/(1+e^{\varepsilon})$. Steinke, Nasr & Jagielski (2023) turn that into a bound on the total count.

Theorem 47.4 (One-run auditing under pure DP)**.** *If training is ε-DP, then for every v*

$$\Pr[W \geq v] \leq \Pr\left[\mathrm{Bin}(r, p_{\varepsilon}) \geq v\right], \qquad p_{\varepsilon} = \frac{e^{\varepsilon}}{1+e^{\varepsilon}}. \tag{47.15}$$

Proof. The plan is to show that each guess is right with probability at most p_{ε}, even given the earlier guesses, and then to compare the count with a binomial.

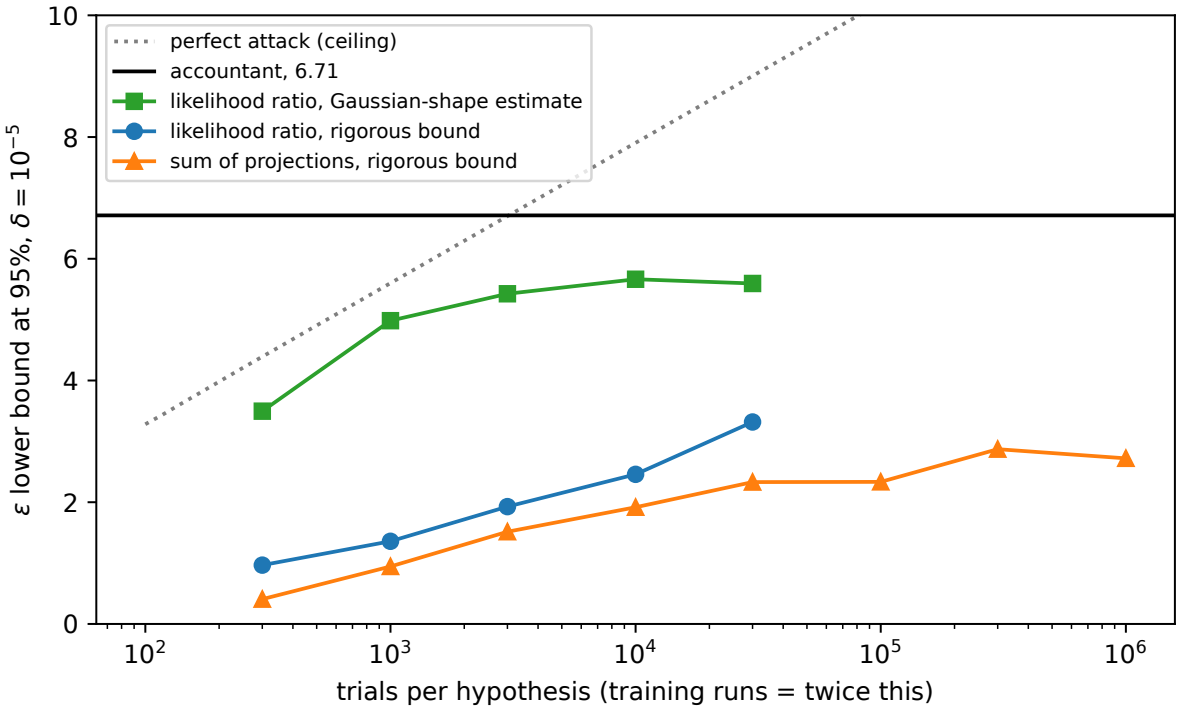


Figure 47.2.: The certified ε lower bound of the white-box audit of the running example against the number of trials per hypothesis, at 95% confidence and $\delta = 10^{-5}$: the likelihood-ratio statistic (circles), the sum of projections (triangles) and the Gaussian-shape estimate from the likelihood ratio (squares), with the accountant's 6.71 and the ceiling of a perfect attack. The bounds grow roughly like the logarithm of the number of trials.

1. *Each inclusion bit is protected.* Fix every inclusion bit except S_i. Changing S_i adds or removes one record, so the run is ε-DP as a function of S_i, and by post-processing (Theorem 19.1) so are the guesses. For every vector of guesses s,

$$\Pr[\hat{S} = s \mid S_i = +1, S_{-i}] \le e^{\varepsilon} \Pr[\hat{S} = s \mid S_i = -1, S_{-i}]. \tag{47.16}$$

2. *Average over the later bits.* Order the guessed canaries in a fixed order, and condition only on the earlier bits $S_{<i}$. The later bits are independent of S_i, so averaging step 1 over them puts the same weights on both sides, and the inequality holds given $S_{<i}$.
3. *No guess is right more than p_ε of the time.* Given $S_{<i}$, write a for the probability of s when $S_i = +1$ and b for it when $S_i = -1$. With the prior $\frac{1}{2}$ on S_i, Bayes' rule gives $\Pr[S_i = +1 \mid \hat{S} = s, S_{<i}] = a/(a+b)$. The fraction grows with a, and step 2 says $a \le e^{\varepsilon} b$, so

$$\frac{a}{a+b} \le \frac{e^{\varepsilon} b}{e^{\varepsilon} b + b} = p_\varepsilon. \tag{47.17}$$

   The same holds for $S_i = -1$. The earlier bits fix whether the earlier guesses were right, so each guess is right with probability at most p_ε given the earlier outcomes.
4. *The count is dominated by a binomial.* Write the j-th guess's correctness as $\mathbf{1}[U_j \le p_j]$, with U_j independent uniform numbers and $p_j \le p_\varepsilon$ its conditional probability from step 3. Then $\mathbf{1}[U_j \le p_j] \le \mathbf{1}[U_j \le p_\varepsilon]$, and the right-hand indicators sum to a $\mathrm{Bin}(r, p_\varepsilon)$ count, so Equation 47.15 follows. Section N.1 in the appendices builds this coupling step by step and proves the domination lemma it relies on.

□

Under (ε, δ)-DP each guess can be right a little more often. Write $F(v) = \Pr[\mathrm{Bin}(r, p_\varepsilon) \ge v]$ for the binomial tail of Equation 47.15.

Theorem 47.5 (One-run auditing under approximate DP)**.** *If training is (ε, δ)-DP, then for every v*

$$\Pr[W \ge v] \le F(v) + 2m\delta \max_{i=1,\dots,m} \frac{F(v-i) - F(v)}{i}. \tag{47.18}$$

The proof is Steinke, Nasr & Jagielski (2023, Corollary 5.4), which tracks how the additive δ of step 1 accumulates over the guesses. Either theorem gives an audit in the same way. Observing v correct guesses rejects every ε whose right-hand side is below β, and the largest such ε is the bound.

The binomial is exactly the count of ε-randomized response on the true bits (Theorem 18.1), so the bound is tight for that mechanism. In the paper's worked example, 75 correct guesses out of 100 give the lower bound 0.702 at 95% confidence. The script reproduces it, and the paper's 0.699 and 0.673 for $\delta = 10^{-4}$ with 100 and 1,000 canaries. The ceiling is again logarithmic, about $\ln(r/\ln(1/\beta))$ when all guesses are right, with $\ln(1/\beta)$ because the test is one-sided. That is 3.49 for 100 canaries, 5.81 for 1,000 and 8.11 for 10,000 at $\delta = 0$, and certifying 6.71 needs about 2,500 perfect guesses. So a one-run audit saves runs, not trials.

The δ term grows with the number of canaries, since the correction is at most $2m\delta$. With 10,000 canaries all guessed right it lowers the bound from 8.11 to 7.83, and when $2m\delta$ approaches β it swallows the whole failure probability. Even with white-box Dirac canaries, one run is not tight

for DP-SGD. Each canary is guessed once, so the audit reads the trade-off curve near FPR $1/m$ at best. And the binomial is calibrated to randomized response, whose curve has a sharp corner, while the curve of DP-SGD is smooth.

Example 47.4. Simulate one run of the running example with 1,000 Dirac canaries on distinct untouched coordinates, about half of them included, each scored by Λ. The auditor tries six list lengths r and pays for trying them with a Bonferroni correction. Each test runs at level $\beta/6$, so that the chance that any of the six is wrong is at most β. Table 47.4 gives the result.

Table 47.4.: One-run audit of the running example's mechanism with 1,000 white-box Dirac canaries, at 95% overall confidence and $\delta = 10^{-5}$.

Guesses r	20	50	100	200	500	1,000
Correct, white-box Dirac canaries	20	50	92	185	425	745
ε_{lo} (Bonferroni over the six)	1.21	2.22	1.58	1.88	1.43	0.90

All 50 most confident guesses are right, which certifies 2.22, while the 100-guess list makes 8 mistakes and certifies less. With 5,000 canaries the best bound is 2.53, from 479 correct out of 500. There the correction $2m\delta = 0.1$ exceeds the per-test level $\beta/8$ of its eight list lengths, and 20 correct guesses out of 20 certify nothing at all. One run certifies 2.22, against 2.84 for 10,000 runs per hypothesis in Table 47.3, at a twenty-thousandth of the cost.

47.5. Black-box audit of the trained network

The white-box Dirac audit tests the mechanism. What does an attacker learn from the released network itself, with canaries planted as ordinary records? This section runs the one-run audit of Section 47.4, so each audit costs one training run. The canaries must be records the model can learn only by memorizing them, since a correctly labelled record has nearly the same loss whether or not it was trained on.

Definition 47.4 (Black-box audit with mislabelled canaries)**.** Take 1,000 records from the held-out test set of the running example and flip their labels, which makes them **mislabelled canaries**. Include each in the 10,000 training records with probability $\frac{1}{2}$, train once with DP-SGD at the claimed configuration, and score each canary by minus its loss on the final model. Guess with the six list lengths of Example 47.4, and bound ε with Equation 47.18, Bonferroni-corrected. As controls, audit a model trained with clipping but no noise, and one trained without clipping or noise.

The bound is valid, because including a canary is adding a record, and DP-SGD's guarantee holds for every dataset. But it should be far below 6.71. The attacker sees one function of the 10,000 noisy sums, the canary's gradient changes as it is fitted, and the 10,000 real records pull the other way. The no-noise control separates the attacker's weakness from the noise. A model trained without noise has no finite ε, so if its audit also gives a small bound, the audit is what keeps the bound small.

Example 47.5. Table 47.5 gives the three audits, one training run each.

Table 47.5.: Black-box one-run audit of the running example's network with 1,000 mislabelled canaries, one training run each, at 95% overall confidence and $\delta = 10^{-5}$. Test accuracy is on the 19,000 test records not used as canaries.

Training	Test accuracy	Mean loss, in	Mean loss, out	Correct of $r = 100$	Correct of $r = 1{,}000$	Best ε_{lo}
DP-SGD, $\sigma = 1$, claim $\varepsilon = 6.71$	0.9645	5.565	6.013	59	556	0.13
Clipping only, $\sigma = 0$, $\varepsilon = \infty$	0.9638	5.511	6.155	67	589	0.24
Non-private, no clipping	0.9155	0.023	9.041	91	967	2.96

The non-private model memorizes the canaries it trained on. Their mean loss is 0.023, against 9.041 for those it did not see, so 967 of 1,000 guesses are right and the audit certifies 2.96. A thousand perfect guesses would certify 5.78 even without the Bonferroni correction, so 33 mistakes cost almost three units. Memorizing also costs almost five points of test accuracy, 0.9155 against 0.9645.

DP-SGD does not memorize. The mean losses of included and excluded canaries differ by less than half a unit, 59 of the 100 most confident guesses are right, and the certified bound is 0.13, from 293 of 500. The clipping-only model is barely easier, at 0.24. Clipping alone bounds each canary's pull on the weights to C per step, against the combined pull of 10,000 correctly labelled records, and that is what defeats this attacker. It does not make clipping private, since a white-box Dirac canary would identify membership in that model perfectly. Figure 47.3 shows the loss distributions, and how the fraction of correct guesses falls as the list grows.

Nothing here contradicts 6.71 or confirms it. The black-box bound of 0.13 says that a realistic attacker with one model extracts very little, while the white-box audits of Section 47.3 say that an attacker who sees every step extracts much more. An honest privacy statement reports both, with the threat model of each, and Definition 46.5 lists what else it must contain.

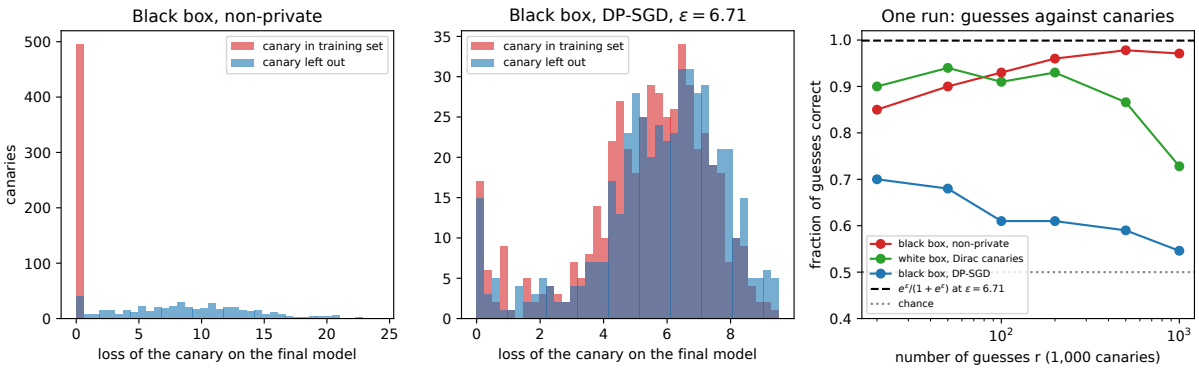


Figure 47.3.: One-run audits with 1,000 canaries. Left and middle: the loss of each mislabelled canary on the final model, for canaries in the training set (red) and left out (blue), without privacy and with DP-SGD at $\varepsilon = 6.71$. Right: the fraction of the r guesses that are correct, against r, for the non-private network (black box), the DP-SGD network (black box) and the mechanism with white-box Dirac canaries; the dashed line is the most a 6.71-DP algorithm allows on average, $e^{\varepsilon}/(1+e^{\varepsilon})$.

47.6. Finding bugs

An audit bound below the claim proves nothing, but a bound above it proves, with confidence $1-\beta$, that the claim is false. That makes audits a test suite for private training code, which passes silently on a correct implementation and fails loudly on a broken one. This section asks which bugs such a test catches, and how cheaply.

Definition 47.5 (Per-step and end-to-end audits)**.** A **per-step audit** inserts a Dirac canary at every step in one run and leaves it out in another, and treats each step's projection x_t as one trial, so it gets T trials per hypothesis from two runs. An **end-to-end audit** runs the many-trial protocol of Section 47.3, with one training run per trial.

With the canary always present there is no sampling coin to blur it, so each step of a correct implementation is a plain Gaussian mechanism. Its privacy is then known exactly, and a measured μ above it is evidence of a bug.

Proposition 47.3 (A per-step audit tests the noise)**.** *In a per-step audit of a correct implementation, the projections are independent across steps, with*

$$x_t \sim \mathcal{N}(0,\sigma^2) \text{ without the canary}, \qquad x_t \sim \mathcal{N}(1,\sigma^2) \text{ with it}, \tag{47.19}$$

so each step is exactly μ-GDP with $\mu = 1/\sigma$. If the bound μ_{lo} of Proposition 47.2, computed from these trials with a threshold chosen on other trials, exceeds $1/\sigma$, the implementation does not add the noise it claims, with confidence $1-\beta$.

Proof.

1. *One step is a Gaussian mechanism.* With the canary in every batch, Lemma 47.1 with $q = 1$ gives Equation 47.19. The means differ by 1 in noise of standard deviation σ, so by Theorem 4.3 the trade-off curve is Equation 47.8 with $\mu = 1/\sigma$.

2. *The steps are independent trials.* The noise is fresh at every step and the canary's gradient is fixed, so the counts at a fixed threshold are binomial and Proposition 47.2 applies.
3. *A refutation.* By Proposition 47.2, a correct implementation gives $\mu_{\text{lo}} \le 1/\sigma$ with probability at least $1-\beta$. So observing $\mu_{\text{lo}} > 1/\sigma$ rejects, at level β, the hypothesis that the code adds noise σC to a sum with sensitivity C.

□

A per-step audit tests the noise and the clipping, but it never exercises the sampling code, and it compares the measured noise with the claimed σ, not with the claimed ε. So a bug in how batches are drawn, or in what the accountant is told, passes it. Nasr et al. (2023) used per-step audits to catch clipping applied after averaging, noise drawn from a seed with only 100 possible values, and a wrong noise scale. An end-to-end audit tests the claimed ε directly, so it catches an accounting bug as readily as a noise bug, but only when the true ε exceeds the claim by more than the audit's shortfall. For the running example that shortfall is about three units at 100,000 runs. Tramèr, Terzis, Steinke, Song, Jagielski & Carlini (2022) used such an audit to refute the guarantee of an open-source implementation, at a confidence they report as $1-10^{-10}$.

No canary audit catches everything, because it tests one dataset, one canary and one attack. Batches drawn by shuffling but accounted as Poisson can have an ε several times the reported one, by a lower bound in Amplification by subsampling. Yet a per-step audit never sees the sampling, and an end-to-end audit sees the gap only where it exceeds the shortfall. Audits also catch errors only in the unsafe direction. Noise σC added to the averaged gradient is B times more noise than claimed, so every audit passes, although the bug wastes accuracy. Passing an audit is evidence, never a certificate.

Example 47.6. Table 47.6 audits three implementations of the running example's noise, each with two runs of 10,000 steps.

Table 47.6.: Per-step audits of three implementations of the running example's DP-SGD step, from 10,000 steps with and 10,000 without a Dirac canary, at 95% confidence. The last column is what the RDP accountant would report for the noise the audit measured.

Implementation	Noise on the sum	Claimed per-step μ	Audited μ_{lo}	Accountant at $\sigma = 1/\mu_{\text{lo}}$	Verdict
Correct	σC	1	0.886	5.38	consistent
Variance passed as a standard deviation	$\sigma^2 C^2 = 0.5\,C$	1	1.745	29.27	refuted
Noise scaled for the mean, added to the sum	$\sigma C/B = 0.01\,C$	1	6.359	over 4,700	refuted

The correct implementation passes, with 0.886 below the claimed 1. The two noise bugs fail decisively. Passing the variance $\sigma^2 C^2 = 0.25$ where the code expects the standard deviation

$\sigma C = 0.5$ halves the noise, and the audit measures 1.745. Scaling the noise for the averaged gradient, whose sensitivity is C/B, and adding it to the sum makes the noise $B = 100$ times too small. Then 10,000 steps separate the hypotheses perfectly, and 6.359 is the most Equation 47.13 can certify from the 5,000 error-free trials per hypothesis of the evaluation half. The same bug also fails an end-to-end audit with the sum of projections, which with 10,000 runs per hypothesis gives $\varepsilon_{\text{lo}} = 7.21 > 6.71$. Figure 47.4 shows why the two noise bugs are easy to see.

A fourth bug lies in the accounting and leaves the code correct. The accountant is told that the dataset has 20,000 records instead of 10,000, as can happen when augmented copies are counted. It computes $q = 0.005$ and reports $\varepsilon = 3.06$. The per-step audit measures the noise, which is right, and composing its μ_{lo} at the claimed $q = 0.005$ gives 2.47, which does not contradict 3.06. The end-to-end audit of Table 47.3 does, because the code really samples at $q = 0.01$ and its 3.63 exceeds 3.06.

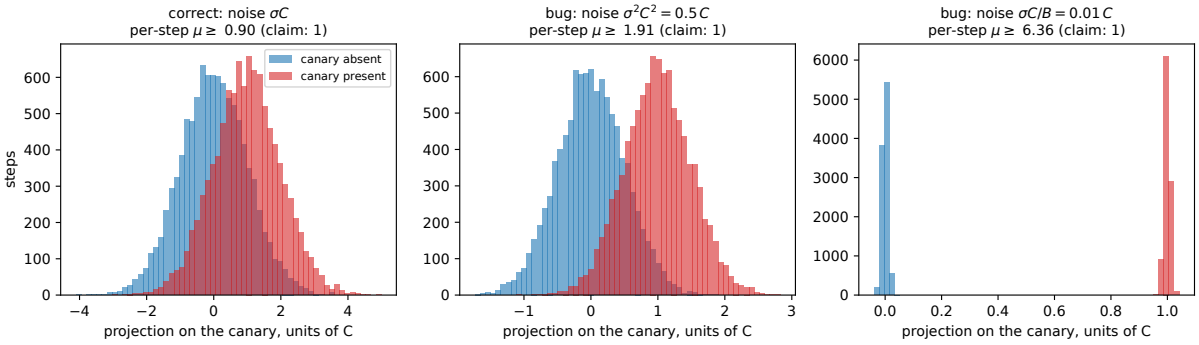


Figure 47.4.: Per-step audits of three implementations: the projections of 10,000 steps on a Dirac canary, without it (blue) and with it at every step (red), in units of C. A correct implementation has noise of standard deviation σC; the two bugs shrink it by a factor of 2 and of 100. The titles give the certified lower bound on the per-step μ; the claim is 1.

47.7. Code

The script code/auditing.py computes every bound of this chapter, simulates the white-box audits from Equation 47.7, and trains the black-box networks with the model and DP-SGD of code/dp_sgd.py. It takes a few minutes to run from the repository root:

```
uv run python code/part05-dp-ml/auditing.py
```

Table 47.7.: The script's output with the default seed.

Experiment	Result
Clopper–Pearson, $k = 10$ of 10,000	$[0.00048, 0.00184]$, as in Part 1
One-run bound, the paper's examples	0.702, 0.699 and 0.673, as published

Experiment	Result
Ceiling of a perfect attack	5.60 with 1,000 trials, 7.90 with 10,000; 3,038 needed for 6.71
White-box Dirac audit, likelihood ratio	$\varepsilon_{\text{lo}} = 3.63$ with 50,000 trials per hypothesis
Same, Gaussian-shape estimate	5.33, against the CLT's 6.01
One run, 1,000 white-box Dirac canaries	2.22
One run, black box, mislabelled canaries	0.13 with DP-SGD, 0.24 with clipping only, 2.96 non-private
Per-step audits	Correct 0.886 against a claim of 1; bugs 1.745 and 6.359

tests/test_auditing.py checks that the output still matches.

47.8. Summary

Table 47.8.: The results of this chapter, and where each is used.

Result	Statement	Use
Definition 47.1	Trials on D and $D' = D \cup \{z^*\}$; the attack's FPR and TPR	Every audit
Theorem 47.1	ε_{lo} from the Clopper–Pearson ends of both rates, with probability $1-\beta$	Choose the threshold on other trials; it refutes a claim, never proves one
Corollary 47.1	$\varepsilon_{\max}(n) \approx \ln(n/\ln(2/\beta))$: 5.60 at $n = 1{,}000$	Budget at least $e^{\varepsilon}\ln(2/\beta)$ trials per hypothesis, in practice far more
Definition 47.2	Input, gradient and Dirac canaries; black-box and white-box attackers	Report the threat model with the number
Lemma 47.1	$\mathcal{N}(0,\sigma^2)$ against $(1-q)\mathcal{N}(0,\sigma^2) + q\,\mathcal{N}(1,\sigma^2)$ per step	The pair the accountant analyses
Theorem 47.2	Λ is optimal, with $\mu = q\sqrt{T(e^{1/\sigma^2}-1)}$; the sum has $q\sqrt{T}/\sigma$	Test code with the strongest attack
Theorem 47.3	$\varepsilon^*(\delta) = \sup_S \ln \frac{Q(S)-\delta}{P(S)}$	A perfect audit finds the true ε, not the accountant's bound

Result	Statement	Use
Proposition 47.1	Reached at FPR $\Phi(-\varepsilon/\mu - \mu/2)$, 8.1×10^{-8} here	Why audits at small δ stop short
Proposition 47.2	$\mu \geq \Phi^{-1}(\text{TPR}) - \Phi^{-1}(\text{FPR})$	Rigorous for μ; gives ε only if the curve is Gaussian
Definition 47.3	m canaries in by fair coins; W correct of r guesses	Auditing one real training run
Theorem 47.4	W is dominated by $\text{Bin}(r, \frac{e^\varepsilon}{1+e^\varepsilon})$	Independent fair coins; tight for randomized response
Theorem 47.5	The same plus at most $2m\delta$	Pay for trying several list lengths with Bonferroni
Definition 47.4	Mislabelled canaries scored by their loss	A small bound describes the attacker, not the model
Definition 47.5	Canary at every step; or one run per trial	Audit the noise per step and the claim end to end
Proposition 47.3	Correct code gives $\mu = 1/\sigma$ per step	A cheap test of noise and clipping, blind to sampling and accounting

47.9. References

- Jagielski, Ullman & Oprea, "Auditing Differentially Private Machine Learning: How Private is Private SGD?" (NeurIPS 2020). An early audit of DP-SGD: poisoning canaries, Clopper–Pearson intervals, and the lower bound of Equation 47.4.
- Nasr, Song, Thakurta, Papernot & Carlini, "Adversary Instantiation: Lower Bounds for Differentially Private Machine Learning" (IEEE S&P 2021). A ladder of threat models from black box to gradient canaries with every iterate visible; tight bounds for the strongest.
- Nasr, Hayes, Steinke, Balle, Tramèr, Jagielski, Carlini & Terzis, "Tight Auditing of Differentially Private Machine Learning" (USENIX Security 2023). Dirac canaries, per-step audits with the canary at every step, Gaussian-shape bounds, and three bugs found.
- Steinke, Nasr & Jagielski, "Privacy Auditing with One (1) Training Run" (NeurIPS 2023). The one-run audit of Section 47.4; Corollary 5.4 is Equation 47.18, and its appendix gives code for the bound.
- Tramèr, Terzis, Steinke, Song, Jagielski & Carlini, "Debugging Differential Privacy: A Case Study for Privacy Auditing" (arXiv 2022). An audit that refuted the guarantee of a published implementation.
- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022). The likelihood ratio attack (LiRA) and the case for reporting TPR at a low FPR, which an audit's bound formalizes.
- Dong, Roth & Su, "Gaussian Differential Privacy" (JRSS B 2022). Trade-off curves, μ-GDP and the CLT for composition used in Section 47.3.

- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015). The hypothesis-testing reading of (ε, δ)-DP behind Equation 47.2.

48. Adaptive data analysis

Every chapter so far has used differential privacy (DP) to protect the people in a dataset. This chapter uses it for a different purpose: to keep a data analysis honest. Machine learning reuses its evaluation data all the time. You tune hyperparameters on a validation set, look at the scores, and try new settings, and competition teams adjust to every leaderboard score. In each case the next question asked of the data depends on the answers to the previous ones. **How many such questions can one holdout set answer before its answers stop telling you anything about new data?**

The idea that organizes the answer is that a question chosen from answers that reveal little about any one record cannot be overfitted to the holdout set. Overfitting a holdout set means choosing questions that fit its particular records, and to do that the analyst must learn something about those records from the earlier answers. A DP mechanism limits what any sequence of answers reveals about a single record, and that limit survives whatever the analyst computes from the answers. So the proof of Algorithmic stability and generalization, which replaces one record at a time by a fresh one, still works with privacy in place of stability.

The running example is a search over 1,000 hyperparameter configurations on a validation set of 10,000 records, each configuration chosen from the scores of the earlier ones. The error of a score is how far it lies from the model's accuracy on new data. If the scores were questions fixed in advance, their largest error would be at most 0.0195 on average. Chosen adaptively, they can be much worse: an analyst who attacks the set can make a model that only guesses at random score 0.0712 above its true accuracy. Of the guarantees that hold for any analyst, splitting the set so that each record answers one question bounds the largest error only by 0.6165, and Gaussian noise on each answer by 0.4095. When the analyst also has a training set, the **reusable holdout** does better in practice, because it adds noise only when the two sets disagree. The chapter's symbols are collected in Table 48.1.

Table 48.1.: The symbols of this chapter.

Symbol	Meaning
$D = (x_1, \dots, x_n)$; $\mathcal{D}$	the sample of n records; the population it is drawn from
f; $f(D)$; $f(\mathcal{D})$	a statistical query; its empirical value; its population value
k; f_j, a_j	the number of queries; the j-th query and its answer
α; β	the accuracy target; the failure probability
$D \sim D'$; $D^{(i)}$	samples that differ in one record; D with x_i replaced by a fresh draw x_i'
W; f_W	an algorithm that reads the sample and outputs a query; that query

Symbol	Meaning
σ; ρ	the standard deviation of Gaussian noise; the parameter of zero-concentrated DP
D_t, D_h; τ, b, B	Thresholdout's training and holdout sets; its threshold, Laplace noise scale and budget of detections
Δ	the sensitivity, the most that replacing one record moves a query

48.1. Reusing a holdout set overfits

A holdout set gives an unbiased verdict on a model that was fixed before the holdout set was drawn. The trouble starts on the second question, once the analyst has seen an answer and lets it shape the next one. This section makes the setting precise and shows how quickly exact answers go wrong.

The questions are averages over records, such as an accuracy, so they are the questions of Part 1's stability chapter (Definition 7.4), here called queries.

Definition 48.1 (Statistical queries and adaptive analysis)**.** The dataset $D = (x_1, \dots, x_n)$ is a sample of n records drawn independently from a distribution $\mathcal{D}$, the population. Each record holds features and label together. A **statistical query** is a function f from one record to $[0, 1]$, such as "does this classifier get this record right?". Its **empirical value** and its **population value** are

$$f(D) = \frac{1}{n}\sum_{i=1}^{n} f(x_i), \qquad f(\mathcal{D}) = \mathbb{E}_{x\sim\mathcal{D}}[f(x)]. \tag{48.1}$$

An **analyst** asks k queries $f_1, \dots, f_k$ one at a time, and a mechanism M answers query f_j with a number a_j. The analysis is **adaptive** when each query may depend on the earlier answers. The answers are accurate to a target α when the largest error $\max_j |a_j - f_j(\mathcal{D})|$ is at most α, either on average or with probability at least $1 - \beta$ for a small failure probability β.

When the queries are all fixed before the sample is drawn, exact empirical answers are accurate, and the number of queries enters only through its logarithm. Theorem 7.5 proves that the expected largest error then obeys

$$\mathbb{E}\Big[\max_{j\le k} \big|f_j(D) - f_j(\mathcal{D})\big|\Big] \le \sqrt{\frac{\ln(2k)}{2n}}. \tag{48.2}$$

A version with high probability follows from Hoeffding's inequality (Theorem 2.5), which bounds the chance that one error exceeds α by $2e^{-2n\alpha^2}$, and the union bound over the k queries (Lemma 2.3). So all k errors stay below α with probability at least $1 - \beta$ once

$$n \ge \frac{\ln(2k/\beta)}{2\alpha^2}. \tag{48.3}$$

At $\alpha = \beta = 0.05$, ten queries need 1,199 records. A thousand queries need 2,120, and a million need only 3,501.

That argument needs each query to be fixed while the records are drawn, so that its values on the records are independent draws with mean $f_j(\mathcal{D})$. One query built from earlier answers is enough to break it.

Proposition 48.1 (The point-query attack)**.** *Let the records be uniform on N values, labelled $1, \dots, N$, and let a mechanism answer every query with its exact empirical value. An analyst asks the N point queries $f_v(x) = \mathbf{1}[x = v]$, which are fixed in advance. Then it asks one query chosen from their answers: "is this record's value one of those that occur in the sample?". With these $k = N + 1$ queries, the largest error is*

$$\max_j \left| a_j - f_j(\mathcal{D}) \right| \geq 1 - \frac{n}{N} = 1 - \frac{n}{k-1}. \tag{48.4}$$

Proof.

1. *The last query can be built.* The answer to f_v is positive exactly when the value v occurs in the sample, so the point queries' answers name the set of values that occur.
2. *Its empirical value.* Every record's value occurs in the sample by definition, so the last query is 1 on every record, and its answer is 1.
3. *Its population value.* The last query is 1 exactly on the distinct values of the sample, of which there are at most n. A fresh record takes each value with probability $1/N$, so the population value is at most n/N.
4. *The error.* The last answer exceeds its population value by at least $1 - n/N$, which is Equation 48.4.

□

The bound holds for every sample, and it stays above $\frac{1}{2}$ until $n \geq (k-1)/2$. So with exact answers n must grow linearly in k, against $\ln k$ for fixed queries. Each point query is harmless, but together they reveal the sample, and a later query exploits it.

Example 48.1. With $n = 1{,}000$ records on $N = 2{,}000$ values, the 2,000 point queries are all within 0.0035 of their population values in one run of the chapter's script. The last query has empirical value 1 and population value 0.3905, against an expected $1-(1-1/2{,}000)^{1{,}000} = 0.3935$. So its error is about 0.61, while Equation 48.2 would bound the average largest error of 2,001 fixed queries by 0.0644. With $N = 10{,}000$ values the error is about 0.90.

Real analysts leak more slowly than this attack, but they leak. In Freedman's paradox (Freedman 1983), which Part 1 runs in Example 7.9, features screened and then tested on the same sample look predictive even when all are noise. The leaderboard version is an attack of Blum & Hardt (2015).

Definition 48.2 (The boosting attack)**.** The holdout labels are n fair coins, independent of everything else, so no submission can beat an accuracy of $\frac{1}{2}$ on new data. The attacker submits k random labellings of the holdout records and reads each one's exact accuracy. It keeps the labellings that scored above $\frac{1}{2}$, and submits their majority vote on each record.

Each kept labelling agrees with the holdout labels slightly more often than chance, and the vote adds these small agreements up. A normal approximation predicts that the vote's reported accuracy exceeds its true accuracy of $\frac{1}{2}$ by about

$$\frac{\sqrt{k/(2n)}}{\pi}. \tag{48.5}$$

So the overfitting grows like $\sqrt{k/n}$, while the bound Equation 48.2 for fixed queries grows only like $\sqrt{\ln k/n}$.

Example 48.2. Table 48.2 runs the attack, averaged over 20 runs. In the running example's regime, 1,000 submissions to a holdout set of 10,000 records, the final submission reports 0.5712 for a coin flip. Its overfitting of 0.0712 is more than three times the bound 0.0195 for 1,001 fixed queries, the 1,000 submissions and the vote.

Table 48.2.: The boosting attack against a leaderboard that reports exact holdout accuracy, averaged over 20 runs. The final submission's true accuracy is exactly 0.5.

Holdout n	Submissions k	Reported accuracy of the final submission	Overfitting	Approximation $\sqrt{k/(2n)}/\pi$	Bound Equation 48.2 for $k+1$ fixed queries
1,000	10	0.5224	0.0224	0.0225	0.0393
1,000	100	0.5739	0.0739	0.0712	0.0515
1,000	1,000	0.7182	0.2182	0.2251	0.0617
10,000	100	0.5227	0.0227	0.0225	0.0163
10,000	1,000	0.5712	0.0712	0.0712	0.0195
10,000	10,000	0.7142	0.2142	0.2251	0.0223

The approximation matches the simulation until the overfitting is large, where it overstates it, because the normal approximation behind it treats the vote's gain as growing without limit (Figure 48.1). For both holdout sizes the attack beats the fixed-query bound once k is a few dozen, and with $k = n$ it reports 71% to 72%.

48.2. Sample splitting and its cost

The attacks work because a query can depend on the records that answer it. The obvious fix is to make that impossible, by never letting a record answer a query it helped to choose. This section asks how much data that costs, because every later method must beat it.

Definition 48.3 (Sample splitting)**.** Split the sample into k parts $D_1, \dots, D_k$ of n/k records each, and answer query j with its empirical value on part j only, $a_j = f_j(D_j)$.

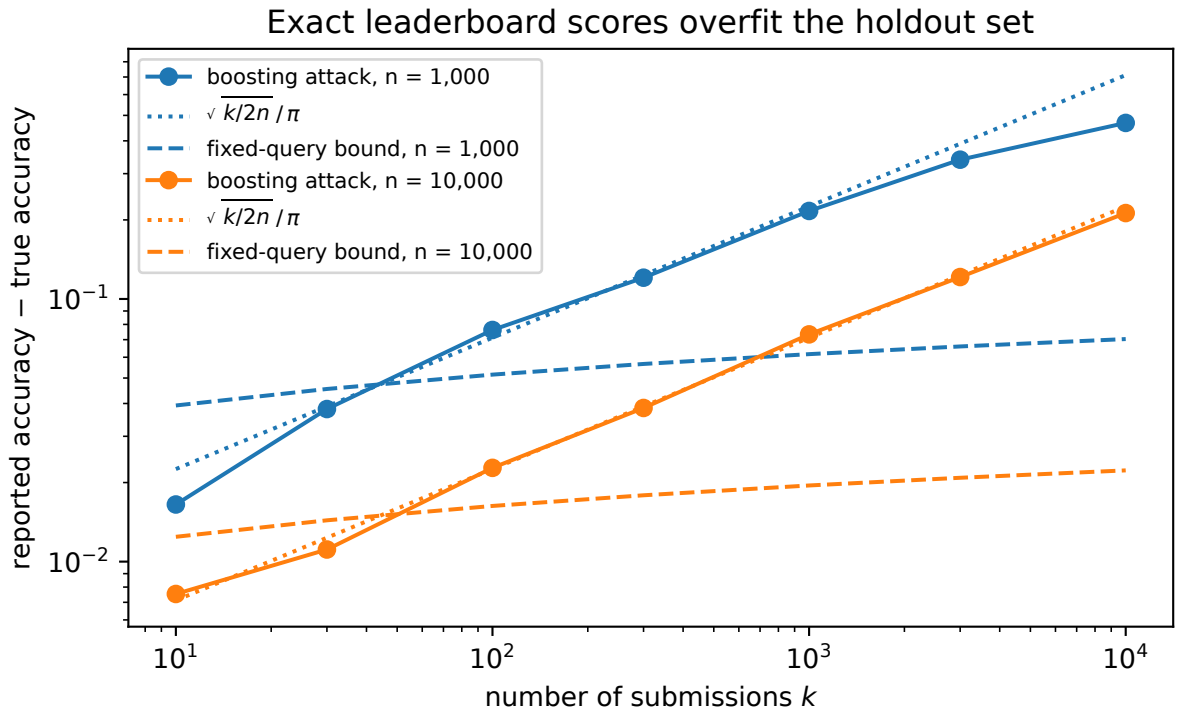


Figure 48.1.: Overfitting of the boosting attack (reported minus true accuracy) against the number of submissions, for holdout sets of 1,000 and 10,000 records, with the approximation $\sqrt{k/(2n)}/\pi$ (dotted) and the bound for fixed queries (dashed). The attack grows like $\sqrt{k}$, the bound like $\sqrt{\ln k}$.

Fixing the earlier parts and the analyst's own coins fixes query j, while part j is still a fresh sample. So the argument for fixed queries applies to each part with n/k records, and Proposition 7.3 proves that, however the queries are chosen,

$$\mathbb{E}\Big[\max_{j\le k}\big|f_j(D_j)-f_j(\mathcal{D})\big|\Big]\le\sqrt{\frac{k\ln(2k)}{2n}}. \tag{48.6}$$

In the same way, Hoeffding's inequality and the union bound make all errors below α with probability $1-\beta$ once $n\ge k\ln(2k/\beta)/(2\alpha^2)$. The data needed grows linearly in k.

Splitting buys validity by using each record once, as if it collected fresh data for every query, so it suits analyses with few rounds of adaptivity. If the queries come in r rounds, and each round is fixed before its answers are seen, one part per round suffices, and the bound becomes $\sqrt{r\ln(2k)/(2n)}$. A final test set, used once after all decisions are made, is the extreme case $r=1$.

When every query is adaptive, though, the cost is steep. At $\alpha=0.05$ and $k=1{,}000$, the bound Equation 48.6 needs 1,520,181 records, against 1,521 for the bound Equation 48.2 on average if the queries were fixed in advance.

Example 48.3. In the running example, 1,000 configurations are scored one after another on 10,000 records. If each score were a query fixed in advance, Equation 48.2 would bound the average largest error by 0.0195. The search is adaptive, so that bound does not apply. Splitting the set into 1,000 parts of 10 records restores validity, but Equation 48.6 is then $\sqrt{1{,}000\ln 2{,}000/20{,}000}=0.6165$, which is useless for scores between 0 and 1.

48.3. Differential privacy implies generalization

The point-query attack learns individual records from the answers and then asks a query that fits them. A mechanism that hides individual records should stop it without throwing away data, and this section proves that it does. Every later section applies the result.

The proof swaps one record for a fresh one, so the guarantees here are for the **replace-one relation**. Two samples are neighbours, written $D\sim D'$, when they differ in one record. Write $D^{(i)}$ for the sample with x_i replaced by an independent fresh draw x_i', so that $D\sim D^{(i)}$. Under this relation every statistical query has sensitivity $1/n$, since one record moves an average of n values in $[0,1]$ by at most $1/n$. A mechanism that is ε-DP for adding or removing a record is 2ε-DP for replacing one (Proposition 18.2).

Three facts about DP from Part 3 carry the proof. The first says that DP bounds averages as well as probabilities. For an (ε,δ)-DP mechanism M, neighbours $D\sim D'$ and every function g of the output with values in $[0,1]$, Lemma 19.1 gives

$$\mathbb{E}[g(M(D))]\le e^{\varepsilon}\,\mathbb{E}[g(M(D'))]+\delta. \tag{48.7}$$

The second is post-processing (Theorem 19.1): any computation on the output that does not touch the data is as private as the output. The third is adaptive composition (Theorem 19.3): mechanisms chosen one after another from earlier outputs have parameters that add up, or grow more slowly under the sharper bounds of Composition. The analyst's choices are post-processing

of earlier answers, so the **transcript**, the whole list of queries and answers, inherits the answering mechanism's guarantee, however clever the analyst is.

The first result says that a query chosen privately has nearly the same expected value on the sample as on the population. Its proof needs one tool from Part 1, Lemma 7.1: swapping two independent draws from one distribution, both independent of everything else, leaves the expectation of any function of all of them unchanged. Write $f_{W(D)}$ for the query that a randomized algorithm W outputs on the sample D, shortened to f_W when the sample is clear.

Lemma 48.1 (Transfer lemma)**.** *Let W be an algorithm that reads a sample and outputs a statistical query f_W, and let W be (ε, δ)-DP under the replace-one relation. Then, with expectations over the sample and the coins of W,*

$$\mathbb{E}[f_W(D)] \le e^{\varepsilon}\,\mathbb{E}[f_W(\mathcal{D})] + \delta, \qquad \mathbb{E}[f_W(\mathcal{D})] \le e^{\varepsilon}\,\mathbb{E}[f_W(D)] + \delta, \tag{48.8}$$

and therefore, since a query's values lie in $[0,1]$,

$$\Big|\mathbb{E}[f_W(D)] - \mathbb{E}[f_W(\mathcal{D})]\Big| \le e^{\varepsilon} - 1 + \delta. \tag{48.9}$$

Proof. The plan is to write both expectations as averages over records x_i of a chosen query's value on x_i. On the sample side the query was chosen with x_i in the sample, and on the population side without it, so DP can compare the two choices.

1. *The empirical value is an average over records.* By definition $f_W(D) = \frac{1}{n}\sum_i f_W(x_i)$, so linearity of expectation gives

$$\mathbb{E}[f_W(D)] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}[f_{W(D)}(x_i)]. \tag{48.10}$$

2. *The population value is the same average after a swap.* The fresh record x_i' is one that W never saw, so averaging over x_i' alone turns its value on x_i' into the population value:

$$\mathbb{E}[f_{W(D)}(\mathcal{D})] = \mathbb{E}[f_{W(D)}(x_i')]. \tag{48.11}$$

   The records x_i and x_i' are independent draws from $\mathcal{D}$, independent of the other records and of the coins of W. So by Lemma 7.1, exchanging them, which turns D into $D^{(i)}$ and x_i' into x_i, leaves the expectation unchanged:

$$\mathbb{E}[f_{W(D)}(x_i')] = \mathbb{E}[f_{W(D^{(i)})}(x_i)]. \tag{48.12}$$

   In words, the population value of a query chosen from D equals the value on x_i of a query chosen without x_i.
3. *DP compares the two choices.* Hold D and x_i' fixed. Then $D \sim D^{(i)}$, and $g(f) = f(x_i)$ is a function of the output of W with values in $[0,1]$. So Equation 48.7, with $\mathbb{E}_W$ averaging over the coins of W, gives

$$\mathbb{E}_W[f_{W(D)}(x_i)] \le e^{\varepsilon}\,\mathbb{E}_W[f_{W(D^{(i)})}(x_i)] + \delta, \tag{48.13}$$

   and averaging over D and x_i' keeps the inequality.

4. *Combine.* Averaging step 3 over i turns its left side into $\mathbb{E}[f_W(D)]$ by step 1, and its right side into $e^{\varepsilon}\,\mathbb{E}[f_W(\mathcal{D})] + \delta$ by step 2. That is the first bound. Subtracting $\mathbb{E}[f_W(\mathcal{D})]$ from both sides leaves $(e^{\varepsilon} - 1)\,\mathbb{E}[f_W(\mathcal{D})] + \delta$, which is at most $e^{\varepsilon} - 1 + \delta$ because a query's value is at most 1.
5. *The other direction.* The replace-one relation is symmetric, so steps 3 and 4 hold with D and $D^{(i)}$ exchanged, which gives the second bound. The two directions together give Equation 48.9.

□

Steps 1 and 2 together are Part 1's identity for the expected gap (Theorem 7.1), with the chosen query in place of a trained model: the gap is an average over records of how much the chosen query's value on a record changes when that record is replaced. Part 1 bounded each term by **uniform stability**, the largest change in one fixed algorithm's loss when one record is replaced, while DP bounds it through the distribution of the output. Uniform stability does not survive the analyst. Keeping the best of several nearly tied queries is a choice that one record can flip, and then the chosen query's value on that record jumps, much as for the 1-nearest-neighbour rule of Proposition 7.2. DP survives post-processing and adaptive composition, so every query the analyst builds from private answers is chosen privately, which is all the lemma needs.

For small ε, $e^{\varepsilon} - 1$ is about ε, so the lemma's cost is linear in ε, and that is the right order in general. Take a sample of one record, a fair bit, and let W release the bit with probability $e^{\varepsilon}/(1 + e^{\varepsilon})$ and its flip otherwise. If W then outputs the query "is the record equal to the released bit?", that query's empirical value exceeds its population value by about $\varepsilon/4$ on average. Because the cost grows with ε, a guarantee to within 0.05 needs ε around 0.05 or below.

The theorem turns the lemma into accuracy for every answer of a private mechanism. Its idea is to let a private algorithm pick the query with the worst error. The lemma then says that this worst query cannot be much further from the population than from the sample.

Theorem 48.1 (Transfer theorem)**.** *Let M answer k adaptively chosen statistical queries, and let the whole interaction be (ε, δ)-DP in the sample under the replace-one relation. Suppose its answers are accurate on the sample on average, $\mathbb{E}[\max_j |a_j - f_j(D)|] \le \alpha$. Then they are accurate on the population up to the lemma's term:*

$$\mathbb{E}\Big[\max_{j \le k} \big|a_j - f_j(\mathcal{D})\big|\Big] \le \alpha + e^{\varepsilon} - 1 + \delta. \tag{48.14}$$

Proof. The plan is to turn the largest absolute error into the signed error of one query, to choose that query privately, and to split its error into the lemma's part and the sample's part.

1. *Add the complements.* For each query f_j, also consider $1 - f_j$, answered by $1 - a_j$. These answers are computed from the original ones, so by post-processing they cost no privacy. A complement's signed error is the original's with the sign flipped, $(1 - f_j(\mathcal{D})) - (1 - a_j) = -(f_j(\mathcal{D}) - a_j)$. So over the $2k$ queries, the largest signed error $f(\mathcal{D}) - a$ equals the largest absolute error of the original k.

2. *Pick the worst query privately.* Let W read the transcript of queries and answers and output the query, among the $2k$, with the largest $f(\mathcal{D}) - a$. DP restricts only how an output depends on the sample. W also uses the distribution $\mathcal{D}$, but that is a fixed constant, like a number written into its code, and W exists only inside the proof, so no one has to run it. Its only input that depends on the sample is the transcript, so by post-processing it is (ε, δ)-DP.
3. *Split the worst error.* By step 1 the left side of Equation 48.14 is $\mathbb{E}[f_W(\mathcal{D}) - a_W]$, where a_W is the answer to the chosen query. Adding and subtracting its empirical value gives

$$\mathbb{E}[f_W(\mathcal{D}) - a_W] = \mathbb{E}[f_W(\mathcal{D}) - f_W(D)] + \mathbb{E}[f_W(D) - a_W]. \tag{48.15}$$

4. *Bound each part.* The first part is at most $e^{\varepsilon} - 1 + \delta$ by the second bound of Lemma 48.1, since W is DP. The second is at most $\mathbb{E}[\max_j |f_j(D) - a_j|] \le \alpha$, because a complement's error on the sample equals the original's. Adding them gives Equation 48.14.

□

The theorem holds only on average, and Section 48.4 cites versions with high probability.

Step 1 of the proof is its least obvious move, and Table 48.3 shows it on two queries. The largest absolute error, 0.12, is on f_1, whose answer is too high, so among f_1 and f_2 alone the largest signed error would be 0.05. The complement $1 - f_1$ carries the error 0.12 with a positive sign, and W picks it.

Table 48.3.: Two queries and their complements. The largest signed error over all four, 0.12, equals the largest absolute error of the original two.

Query	Answer a	Population value $f(\mathcal{D})$	Signed error $f(\mathcal{D}) - a$
f_1	0.62	0.50	−0.12
f_2	0.40	0.45	+0.05
$1 - f_1$	0.38	0.50	+0.12
$1 - f_2$	0.60	0.55	−0.05

Example 48.4. The lemma is easiest to see on picking the best-looking candidate, with a small sample so that the bias is visible. Take $k = 1{,}000$ queries that are all pure noise, each the rate at which a random feature agrees with a random label on $n = 1{,}000$ records. Every population value is $\frac{1}{2}$, and the empirical values are independent with standard deviation $1/(2\sqrt{n}) = 0.0158$. Choosing the query with the largest empirical value, with no privacy, overstates it by 0.0509 on average over 2,000 runs. The queries themselves are fixed in advance, so this is at most their largest error, and Equation 48.2 bounds that by 0.0616.

Now choose with the exponential mechanism of Selection, which picks query j with probability proportional to $e^{\varepsilon n f_j(D)/2}$. Its score has sensitivity $1/n$, so it is ε-DP by Theorem 22.1. The script computes its selection bias exactly for each sample, as the probability-weighted average of the empirical values, and Table 48.4 compares it with the lemma.

Table 48.4.: Selection bias of the exponential mechanism over 1,000 null queries on 1,000 records, averaged over 2,000 runs, against the transfer lemma.

ε	Measured bias $\mathbb{E}[f_W(D)] - \frac{1}{2}$	Lemma $e^{\varepsilon} - 1$	Sharper $(e^{\varepsilon} - 1) \cdot \frac{1}{2}$
0.01	0.00124	0.01005	0.00503
0.1	0.01245	0.10517	0.05259
1	0.04991	1.71828	0.85914
No privacy (argmax)	0.0509		

The last column is the sharper bound $(e^{\varepsilon} - 1)\,\mathbb{E}[f_W(\mathcal{D})] + \delta$ from step 4 of the lemma's proof, with population value $\frac{1}{2}$ and $\delta = 0$. For small ε the measured bias is $\varepsilon/8$, a quarter of that bound. At $\varepsilon = 1$ the mechanism behaves like the argmax, and the lemma's bound, 1.72, says nothing (Figure 48.2).

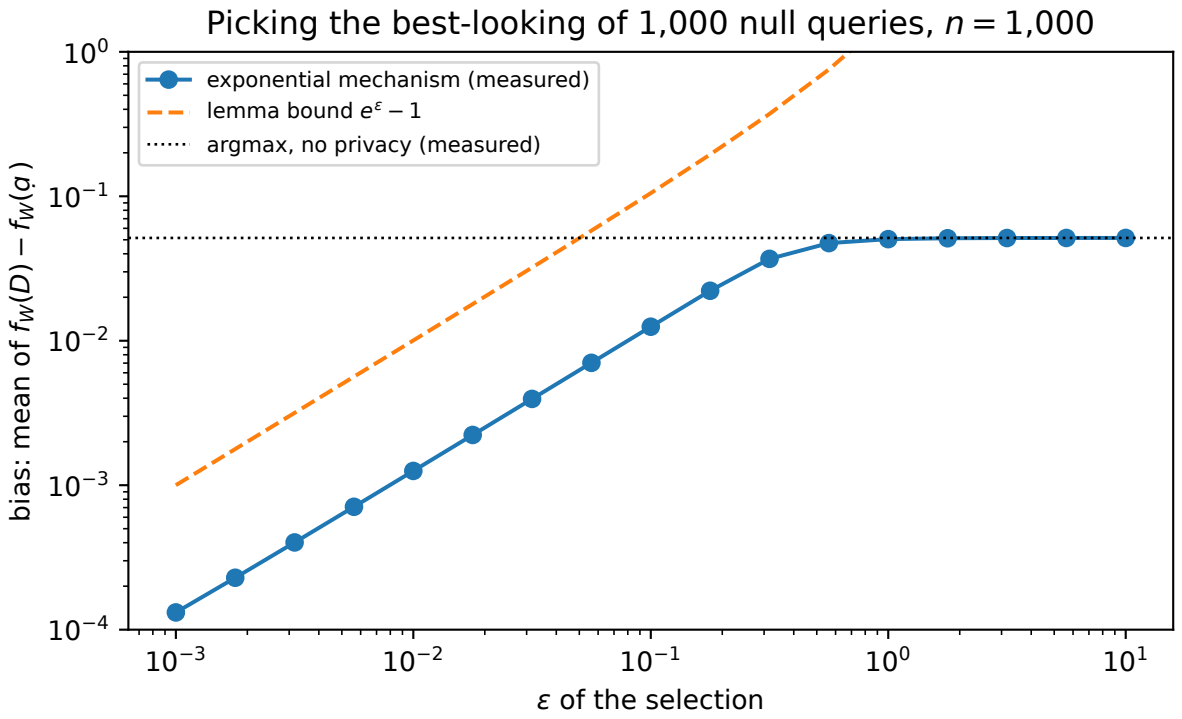


Figure 48.2.: Selection bias of the exponential mechanism choosing among 1,000 noise queries on 1,000 records, against the ε of the choice, with the transfer lemma's bound $e^{\varepsilon} - 1$ and the bias of the non-private argmax.

48.4. Answering with Gaussian noise: the $\sqrt{k}$ rate

The transfer theorem makes any private mechanism that is accurate on the sample accurate on the population, and the simplest such mechanism adds Gaussian noise to each empirical value.

This section asks how many records that needs for k adaptive queries, and how it compares with splitting.

The privacy of Gaussian answers is easiest to track in **zero-concentrated differential privacy** (zCDP), which measures privacy by one number ρ. It bounds the Rényi divergence between the outputs on neighbours, at every order, by ρ times that order. By Theorem 31.1, Gaussian noise of standard deviation σ on a query of sensitivity $1/n$ is ρ-zCDP with $\rho = 1/(2n^2\sigma^2)$, and the values of ρ add under adaptive composition. By Theorem 31.4, a ρ-zCDP mechanism is (ε, δ)-DP for every $\delta > 0$ with

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{48.16}$$

Theorem 48.2 (Gaussian answers to adaptive queries)**.** *Answer each query with $a_j = f_j(D) + \xi_j$, where the noises ξ_j are independent and normal with mean 0 and standard deviation σ. Then for every $\delta > 0$,*

$$\mathbb{E}\Big[\max_j \big|a_j - f_j(\mathcal{D})\big|\Big] \le \sigma\sqrt{2\ln(2k)} + e^{\varepsilon} - 1 + \delta, \qquad \varepsilon = \rho + 2\sqrt{\rho\ln(1/\delta)}, \quad \rho = \frac{k}{2n^2\sigma^2}. \tag{48.17}$$

Proof.

1. *Privacy.* Each answer is the Gaussian mechanism on a query of sensitivity $1/n$, so it is ρ_1-zCDP with $\rho_1 = 1/(2n^2\sigma^2)$. The analyst chooses each query from earlier answers, and adaptive composition adds the k values to $\rho = k\rho_1$. Equation 48.16 converts this to (ε, δ)-DP.
2. *Accuracy on the sample.* The errors on the sample are the noises ξ_j. A normal variable with variance σ^2 has moment generating function exactly $e^{s^2\sigma^2/2}$. The proof of Theorem 7.5 bounds an expected maximum of $2k$ variables using only such a bound for each, $e^{s^2/(8n)}$ there, and not their independence. Run with $e^{s^2\sigma^2/2}$ for the $2k$ values $\pm\xi_j$, its steps give $\mathbb{E}[\max_j |\xi_j|] \le \sigma\sqrt{2\ln(2k)}$.
3. *Transfer.* Theorem 48.1, with the accuracy of step 2 and the privacy of step 1, gives Equation 48.17.

□

More noise lowers the transfer term but raises the noise term, so there is a best noise level, and it sets the rate. To see it, keep only the leading part of the transfer term. For small ε, $e^{\varepsilon} - 1$ is about ε, and for small ρ the conversion is dominated by its square-root term.

Corollary 48.1 (The square-root rate)**.** *Replace the transfer term of Equation 48.17 by its leading part $2\sqrt{\rho\ln(1/\delta)}$, and leave the δ term aside. The resulting bound is smallest at $\sigma = (k\ln(1/\delta)/\ln(2k))^{1/4}/\sqrt{n}$, and there it equals α when*

$$n = \frac{8\sqrt{k\ln(2k)\ln(1/\delta)}}{\alpha^2}. \tag{48.18}$$

With δ of order α, so that the δ term also stays below α, the number of records grows like the square root of k:

$$n = O\Big(\frac{\sqrt{k\,\ln k\,\ln(1/\alpha)}}{\alpha^2}\Big). \tag{48.19}$$

Proof.

1. *The leading transfer term.* With $\rho = k/(2n^2\sigma^2)$, the leading part is

$$2\sqrt{\rho \ln(1/\delta)} = \frac{\sqrt{2k \ln(1/\delta)}}{n\sigma}. \tag{48.20}$$

2. *Balance the two terms.* The bound is now $\sigma A + B/\sigma$, with $A = \sqrt{2\ln(2k)}$ from the noise and $B = \sqrt{2k\ln(1/\delta)}/n$ from step 1. Its derivative $A - B/\sigma^2$ vanishes at $\sigma = \sqrt{B/A}$, which is the stated noise level, and there the bound equals $2\sqrt{AB}$.
3. *Solve for n.* Setting $2\sqrt{AB} = \alpha$ gives $AB = \alpha^2/4$. Since $AB = 2\sqrt{k\ln(2k)\ln(1/\delta)}/n$, this is the stated n, and $\ln(1/\delta)$ of order $\ln(1/\alpha)$ gives Equation 48.19.

□

The $\sqrt{k}$ is the $\sqrt{k}$ of composition. Under zCDP, ρ grows like k, so the ε of k answers grows like $\sqrt{k}$ times that of one. So the best error grows like $k^{1/4}/\sqrt{n}$, up to logarithms, and keeping it at α needs n to grow like $\sqrt{k}$, while splitting needs n to grow like k. The approximate n is slightly below the exact optimum, because the proof dropped terms that add to the bound.

The constants are large. Table 48.5 solves each expected-error bound for the smallest n that reaches $\alpha = 0.05$. For one query the Gaussian bound needs 7,300 records, against 139 for the other two, and splitting stays cheaper up to a few hundred adaptive queries. Beyond that the square root wins: at a million queries, 33 million records against 2.9 billion. At the optimum the error splits almost equally between the noise, 0.0242, and the transfer term, 0.0258, with $\varepsilon = 0.0236$ and $\delta = 0.0019$ for every k. Here δ protects no one, because it is only a parameter of the bound.

Table 48.5.: Records needed to bring each bound on the expected largest error down to 0.05. The fixed-query column is valid only for queries chosen in advance; the other two hold for adaptive queries.

Queries k	Fixed in advance Equation 48.2	Sample splitting Equation 48.6	Gaussian noise Equation 48.17
1	139	139	7,300
10	600	5,992	47,985
100	1,060	105,967	201,800
1,000	1,521	1,520,181	764,337
10,000	1,981	19,806,976	2,758,964
1,000,000	2,902	2,901,731,548	33,393,779

Three results go further, and this chapter cites them without proofs. The first is a **high-probability** transfer theorem (Bassily et al. 2016). If an (ε, δ)-DP mechanism is within α of the empirical values with probability $1 - \beta$, with ε of order α and δ of order $\alpha\beta$, then it is within a constant times α of the population values with probability at least $1 - c\beta$, for a constant c. Section N.2 in the appendices derives the shorter proof of Jung et al. (2020). With Gaussian noise it gives $n = \tilde{O}(\sqrt{k}/\alpha^2)$, where $\tilde{O}$ hides logarithmic factors.

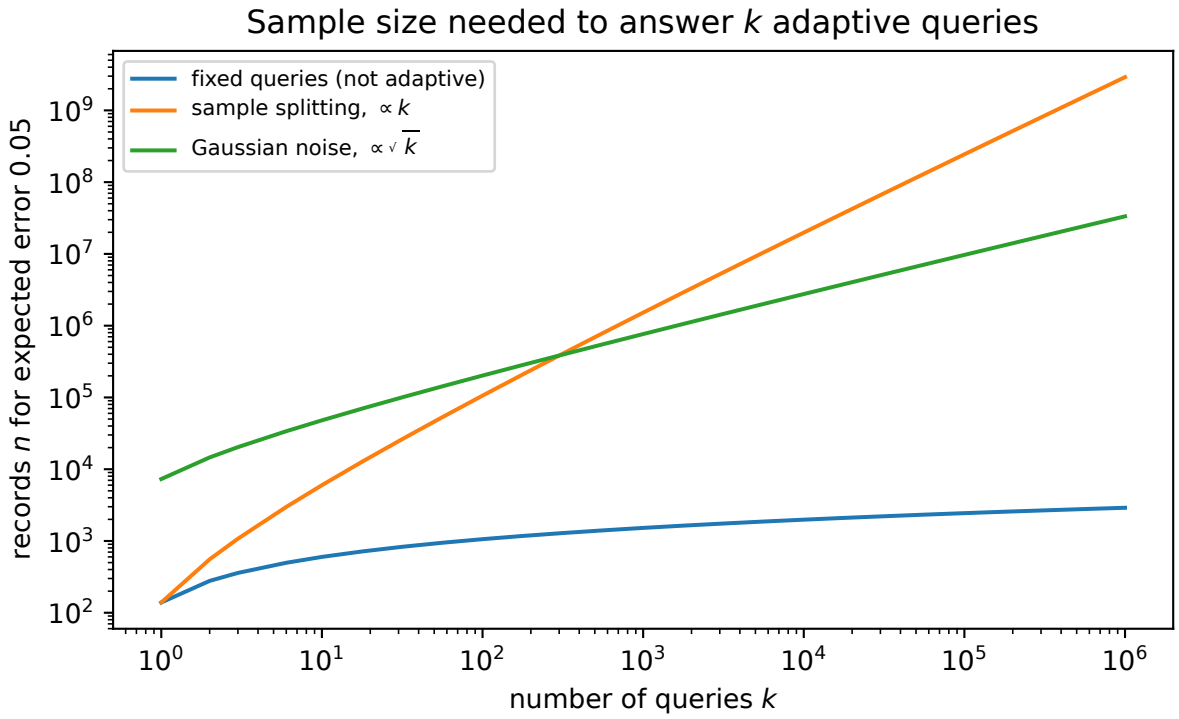


Figure 48.3.: Records needed for an expected largest error of 0.05 against the number of queries, for queries fixed in advance, sample splitting and Gaussian noise with the transfer theorem. The slopes on this log-log scale are about 0, 1 and $\frac{1}{2}$.

The second result helps when the queries are very many and the records take few distinct values. Then private multiplicative weights, the online algorithm of Query release, needs a number of records that grows only like $\ln k$ (Bassily et al. 2016), at the price of a worse power of $1/\alpha$. The third is a lower bound, which assumes a standard conjecture of cryptography. For records with many features, no mechanism that runs in polynomial time answers more than about n^2 adaptive queries accurately (Hardt & Ullman 2014; Steinke & Ullman 2015). Read the other way, k queries need n of order $\sqrt{k}$, so the Gaussian rate is the best such a mechanism reaches in general.

Example 48.5. With $n = 10{,}000$ records and $k = 1{,}000$ adaptive queries, the running example, the best Gaussian bound Equation 48.17 is 0.4095. It uses $\sigma = 0.0513$, so that the noise contributes 0.2001 and the transfer term $e^{\varepsilon} - 1 + \delta$ contributes 0.2093, with $\varepsilon = 0.1674$ and $\delta = 0.027$. Splitting gives 0.6165, so at this size every guarantee for adaptive queries is weak. The theorem guards against an analyst who attacks on purpose, and Section 48.5 shows that a typical analyst is much easier to defend against.

48.5. The reusable holdout: Thresholdout

In machine learning the analyst usually has two sets, a training set to explore freely and a holdout set only for confirming what the training set suggests. Most of the time the two agree, and then the holdout set's answer adds nothing. Only disagreements, the sign of overfitting to the training set, carry information about the holdout set. This section shows how to pay privacy only for those.

The **reusable holdout** of Dwork et al. (2015) is built on Thresholdout, which follows the pattern of AboveThreshold, the sparse vector technique of Selection, which compares a stream of noisy query values with a noisy threshold and pays privacy only when one crosses it.

Definition 48.4 (Thresholdout)**.** Thresholdout holds a training set D_t, a holdout set D_h of n records, a threshold τ, a Laplace noise scale b and a budget of B detections. It starts with a noisy threshold $\hat{\tau} = \tau + \zeta$, where $\zeta \sim \mathrm{Lap}(2b)$, and answers each query f in two steps.

1. Draw fresh noise $\nu \sim \mathrm{Lap}(4b)$, and compare the gap between the two sets with the noisy threshold: is $|f(D_h) - f(D_t)| + \nu > \hat{\tau}$?
2. If not, answer with the training value $f(D_t)$. If so, count one **overfitting detection**, answer with $f(D_h) + \xi$, where $\xi \sim \mathrm{Lap}(b)$, and redraw the threshold noise ζ.

The run stops at its B-th detection.

Privacy is spent only at detections. A comparison that comes out below the threshold releases a training value, and the only thing it reveals about the holdout set is that the noisy gap stayed below, which AboveThreshold already pays for once per run.

Theorem 48.3 (Privacy of Thresholdout)**.** *Suppose replacing one holdout record moves every query's value $f(D_h)$ by at most Δ. Then a run of Thresholdout that stops at its B-th detection is ε-DP in the holdout records, with*

$$\varepsilon = \frac{2B\Delta}{b}, \tag{48.21}$$

however many queries were answered below the threshold. For statistical queries $\Delta = 1/n$.

Proof. The plan is to see the stretch between two detections as one run of AboveThreshold, price one such run, and add up the runs.

1. *One detection is one run of AboveThreshold.* Between two detections, Thresholdout compares a stream of values $|f(D_h) - f(D_t)|$ with one noisy threshold and stops at the first that exceeds it. By Theorem 22.6, that is $(\varepsilon_1 + \varepsilon_2)$-DP when the threshold noise has scale Δ/ε_1 and the query noise scale $2\Delta/\varepsilon_2$, for any number of answers below the threshold.
2. *The sensitivity of the compared values.* The training set does not depend on the holdout set, so hold it fixed. Replacing one holdout record moves $f(D_h)$ by at most Δ, so it moves $|f(D_h) - f(D_t)|$ by at most Δ too.
3. *The cost of one detection.* The threshold noise has scale $2b = \Delta/\varepsilon_1$, so $\varepsilon_1 = \Delta/(2b)$. The query noise has scale $4b = 2\Delta/\varepsilon_2$, so $\varepsilon_2 = \Delta/(2b)$. The noisy answer is the Laplace mechanism with scale b, which costs Δ/b, so one detection costs $\Delta/(2b) + \Delta/(2b) + \Delta/b = 2\Delta/b$.
4. *The answers below the threshold.* They are training values, which do not depend on the holdout set. Their only link to it is the comparison, which step 1 already paid for.
5. *All detections.* Each detection closes one run, and the next run starts with fresh threshold noise. Basic composition (Theorem 19.3) over the B closed runs gives Equation 48.21.

□

So only overfitting is charged: a query whose training and holdout values agree costs nothing, however many such queries there are. An analysis that ends before the B-th detection leaves one run open, which costs only its threshold term $\Delta/(2b)$, less than the detection it never made.

Every answer is within about τ plus noise of the holdout value. So Theorem 48.1 applies with the holdout set as the sample, and the answers are accurate on the population to within that, plus $e^{\varepsilon} - 1$. A smaller τ gives sharper answers but more detections. The formal guarantee needs large holdout sets, though. For statistical queries $\varepsilon = 2B/(nb)$, and for the transfer term to stay below a target α this must be at most about α, so B detections need n of order $B/(b\alpha)$ records.

Example 48.6. The running search has no separate training set, so this example is the experiment of Dwork et al. (2015), scaled down. A training set and a holdout set each have $n = 2{,}000$ records with 5,000 features, each $+1$ or -1, and a label ± 1. In the first setting every feature is independent of the label, so no classifier can beat 50%. In the second, 20 features agree with the label with probability $\frac{1}{2} + 1.5/\sqrt{n}$, a **correlation** of $3/\sqrt{n} = 0.067$. The correlation of a ± 1 feature with a ± 1 label is the average of their product, which is twice the agreement rate minus 1. The analyst works in three steps.

1. Compute each feature's correlation with the label on the training set, and keep the features where it exceeds $1/\sqrt{n}$ in size.
2. Ask the holdout set for the correlations of those features, and keep the ones whose holdout correlation also exceeds $1/\sqrt{n}$ in size, with the same sign.
3. Rank them by training correlation. For each m from 10 to 200, build the classifier that takes the sign of the sum of the top m features, each multiplied by the sign of its training correlation, and ask the holdout set for its accuracy.

The holdout queries are answered exactly, with Gaussian noise of standard deviation $1/\sqrt{n}$ (the sampling error of one correlation), or by Thresholdout with $\tau = 4/\sqrt{n}$ and $b = 1/\sqrt{n}$. The script computes each classifier's true accuracy exactly, from the distribution of the features, and Table 48.6 averages 10 repetitions for $m = 200$.

Table 48.6.: The reusable-holdout experiment with 200 selected features, averaged over 10 repetitions.

Setting	Holdout answered	Training accuracy	Reported holdout accuracy	True accuracy
No signal	Exactly	0.6980	0.6842	0.5000
No signal	With Gaussian noise	0.7172	0.6135	0.5000
No signal	By Thresholdout	0.7664	0.5217	0.5000
20 informative features	Exactly	0.7146	0.7039	0.5355
20 informative features	With Gaussian noise	0.7322	0.6386	0.5347
20 informative features	By Thresholdout	0.7759	0.5471	0.5312

Answered exactly, the holdout set reports 68% for a classifier that cannot beat 50%. Gaussian noise of the size of the sampling error removes only part of the bias. Thresholdout reports 52%, within its threshold of the truth. That threshold is $4/\sqrt{n}$ on a correlation, and since an accuracy is (1 + correlation)/2, it is $2/\sqrt{n} = 0.045$ on an accuracy. Thresholdout works in two ways. Most holdout correlations in step 2 come back as training values, so the selection barely depends on the holdout set. Then the accuracy queries of step 3 have training accuracies far above their holdout accuracies, so they trigger detections and come back as noisy holdout values. With 20 informative features, Thresholdout's report, 0.5471, is close to the true accuracy of 0.5312, and its selection is about as good as the others (Figure 48.4).

The privacy accounting tells the other half of the story. Each repetition asks about 1,587 holdout queries, and Thresholdout records 325.8 detections on average. One record moves a correlation by at most $\Delta = 2/n$, so each detection costs $2\Delta/b = 0.0894$ by Equation 48.21, and an average repetition costs $\varepsilon = 29.1$, plus $\Delta/(2b) = 0.022$ for the run left open at the end. Each Gaussian answer is ρ-zCDP with $\rho = \Delta^2/(2\sigma^2) = 2/n$ by Theorem 31.1, with $\sigma = 1/\sqrt{n}$, so all of them together have $\rho = 1.59$, which is $\varepsilon = 10.1$ at $\delta = 10^{-5}$. Neither is near the small ε the transfer theorem needs, yet both help, because this analyst only screens features and does not attack the holdout set.

48.6. What it means for machine-learning practice

Hyperparameter search, early stopping on a validation loss, competition leaderboards and benchmark test sets all ask one set of records many questions chosen from earlier answers. This section turns the earlier results into rules for such work.

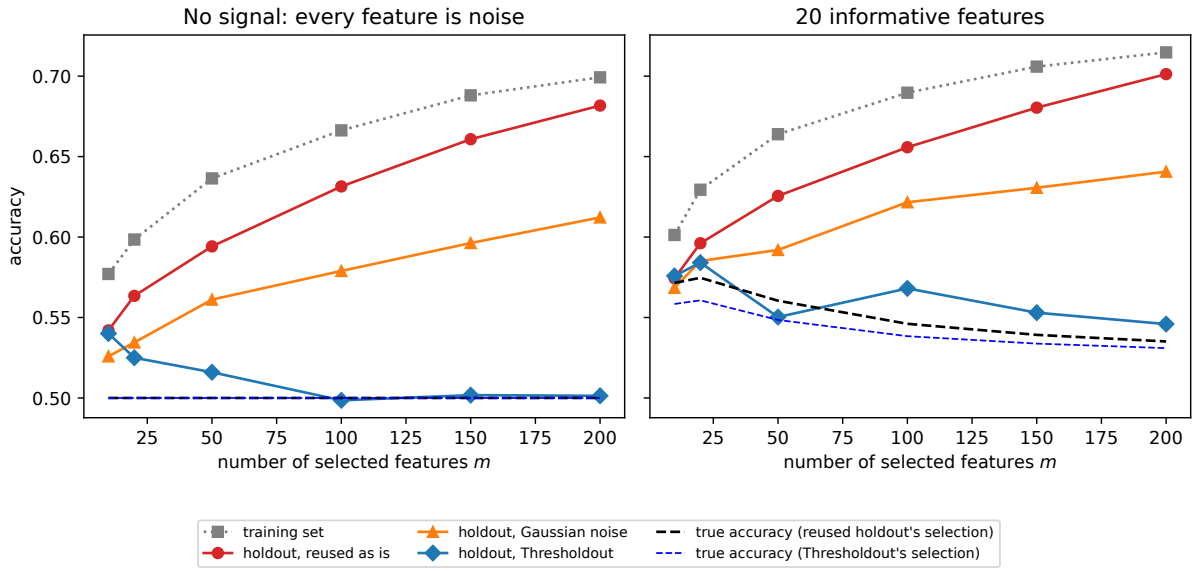


Figure 48.4.: Accuracy of a classifier built from the top m features against m: on the training set, as reported by a holdout set reused as is, with Gaussian noise and through Thresholdout, and the true accuracy. Left: every feature is noise. Right: 20 features carry signal.

1. **Count adaptive rounds, not queries.** Questions chosen before any answer was seen are cheap, since the records that k of them need grow only like $\ln k$. So the best of k models trained without the validation set has a score that overstates the truth by at most $\sqrt{\ln(2k)/(2n)}$ on average (Equation 48.2).
2. **Keep a final test set and use it once.** One query is never adaptive. If the analysis has a few rounds, give each round its own part of the data (Section 48.2).
3. **Answer through noise when the rounds are many.** Gaussian noise (Section 48.4), or Thresholdout when there is a training set (Section 48.5), answers many more adaptive queries than splitting.
4. **Private tuning generalizes for free.** A private search chooses its winner privately, so its validation score is nearly unbiased (Corollary 48.2).
5. **For leaderboards, release less.** Blum & Hardt's (2015) **Ladder** reports a new score only when a submission beats the best so far by a margin, rounded to that margin. A leaderboard that changes rarely reveals few bits, and few bits cannot encode much about individual records. So its error against the population is of order $n^{-1/3}$, up to a factor that grows like the cube root of $\ln(kn)$, which Section N.3 in the appendices proves.

Rule 4 needs one more step from the transfer lemma, because a search outputs a configuration, not a query. The step is to see the winner's correctness on a record as a query.

Corollary 48.2 (A private search's winner is nearly unbiased)**.** *Let a hyperparameter search be (ε, δ)-DP in the validation set under the replace-one relation, and let it output a trained model. Then the model's accuracy on the validation set overstates its accuracy on the population by at most $e^{\varepsilon} - 1 + \delta$ on average.*

Proof.

1. *The winner defines a query.* "Is this record classified correctly by the winner?" maps a record to $\{0, 1\}$, so it is a statistical query. Its empirical value is the validation accuracy and its population value the true accuracy.
2. *The query is chosen privately.* The query is a function of the search's output alone, so by post-processing the algorithm that outputs it is (ε, δ)-DP.
3. *Apply the lemma.* Lemma 48.1 bounds the difference of the two expectations by $e^{\varepsilon} - 1 + \delta$.

□

The bound holds on average, and only for accuracy on the records the search was private in.

Real practice is usually milder than the worst case. New test sets for CIFAR-10 and ImageNet, collected after years of reuse of the old ones, lowered accuracy, by 3 to 15 points on CIFAR-10 and 11 to 14 points on ImageNet, but largely preserved the ranking of models, and the authors attribute the drop to the new images rather than to adaptive overfitting (Recht et al. 2019). Across more than a hundred Kaggle competitions, Roelofs et al. (2019) found little adaptive overfitting, perhaps because similar models ask similar questions (Mania et al. 2019). Reuse is dangerous with few records, many truly adaptive rounds, and an incentive to climb the leaderboard. A validation set for tuning and a test set used once is splitting with two rounds, and it is often enough.

Example 48.7. In the running example, a team reports the best of 1,000 configurations on a validation set of 10,000 records. If the configurations were all trained and fixed before the validation set was opened, rule 1 bounds the expected optimism of the winner's score by 0.0195. If the search was sequential, that bound no longer holds, and Table 48.5 shows that 10,000 records are far too few to certify 1,000 adaptive scores to within 0.05. The honest options are a fresh test set for the final model, or a private search. With $\varepsilon = 0.05$ and a negligible δ, Corollary 48.2 bounds the optimism by $e^{0.05} - 1 = 0.051$. That bound is cheap only because it covers the winner alone. The price is paid elsewhere, in the noise that keeps the search at $\varepsilon = 0.05$, which makes it worse at finding the best configuration.

48.7. Code

The script code/part05-dp-ml/adaptive_data_analysis.py computes every number in this chapter and draws its figures. It runs the two attacks, the selection-bias experiment and the reusable-holdout experiment, and solves the three expected-error bounds for the sample size each needs. From the repository root:

```
uv run python code/part05-dp-ml/adaptive_data_analysis.py
```

Table 48.7.: The script's output with seed 0.

Experiment	Result
Fixed queries, $\alpha = \beta = 0.05$	1,199 records for 10 queries, 3,501 for a million
Point-query attack, $n = 1{,}000$	Error 0.61 with 2,001 queries, 0.90 with 10,001

Experiment	Result
Boosting attack	Overfitting 0.2182 at $k = n = 1{,}000$; approximation $\sqrt{k/(2n)}/\pi$ within 0.01 while the overfitting is small
Selection bias, 1,000 null queries	Argmax 0.0509; exponential mechanism 0.00124 at $\varepsilon = 0.01$, against $e^{\varepsilon} - 1 = 0.01005$
Records for error 0.05, $k = 1{,}000$	Fixed 1,521; splitting 1,520,181; Gaussian 764,337
Reusable holdout, no signal, $m = 200$	Reported 0.6842 exactly, 0.6135 with Gaussian noise, 0.5217 with Thresholdout; true 0.5

tests/test_adaptive_data_analysis.py checks that the output still matches the numbers in this chapter.

48.8. Summary

Table 48.8.: The results of this chapter, and where each is used.

Result	Statement	Use
Equation 48.2	$\mathbb{E}\max_j \lvert f_j(D) - f_j(\mathcal{D})\rvert \le \sqrt{\ln(2k)/(2n)}$ for queries fixed in advance	Count adaptive rounds, not queries
Proposition 48.1	One adaptive query after N point queries has error $\ge 1 - n/N$	Exact answers need n linear in k
Definition 48.2	Overfitting $\approx \sqrt{k/(2n)}/\pi$	On a leaderboard, release improvements, not scores
Equation 48.6	$\mathbb{E}\max_j \lvert f_j(D_j) - f_j(\mathcal{D})\rvert \le \sqrt{k\ln(2k)/(2n)}$	Few rounds; keep a test set used once, at the end
Lemma 48.1	$\lvert\mathbb{E} f_W(D) - \mathbb{E} f_W(\mathcal{D})\rvert \le e^{\varepsilon} - 1 + \delta$ for (ε, δ)-DP W	A privately chosen query cannot be overfitted
Theorem 48.1	Population error $\le \alpha + e^{\varepsilon} - 1 + \delta$	Privacy at level ε costs about ε in accuracy
Theorem 48.2	$\sigma\sqrt{2\ln(2k)} + e^{\varepsilon} - 1 + \delta$ with $\rho = k/(2n^2\sigma^2)$	Sizing a holdout set for an interactive analysis
Corollary 48.1	$n \approx 8\sqrt{k\ln(2k)\ln(1/\delta)}/\alpha^2$	Records grow like $\sqrt{k}$, against k for splitting
Theorem 48.3	$\varepsilon = 2B\Delta/b$ for B detections	Confirming training-set findings
Corollary 48.2	A private search's winner overstates by at most $e^{\varepsilon} - 1 + \delta$	Private hyperparameter tuning

48.9. References

- Dwork, Feldman, Hardt, Pitassi, Reingold & Roth, "Preserving Statistical Validity in Adaptive Data Analysis" (STOC 2015). The first connection between DP and adaptive data analysis, and the transfer results of Section 48.3.
- Dwork, Feldman, Hardt, Pitassi, Reingold & Roth, "The Reusable Holdout: Preserving Validity in Adaptive Data Analysis" (*Science* 349, 2015), and "Generalization in Adaptive Data Analysis and Holdout Reuse" (NeurIPS 2015). Thresholdout, its guarantee and the experiment of Section 48.5.
- Bassily, Nissim, Smith, Steinke, Stemmer & Ullman, "Algorithmic Stability for Adaptive Data Analysis" (STOC 2016). The high-probability transfer theorem, the $\tilde{O}(\sqrt{k}/\alpha^2)$ rate for the Gaussian mechanism and the rate for private multiplicative weights.
- Jung, Ligett, Neel, Roth, Sharifi-Malvajerdi & Shenfeld, "A New Analysis of Differential Privacy's Generalization Guarantees" (ITCS 2020). A short proof of the high-probability transfer theorem.
- Hardt & Ullman, "Preventing False Discovery in Interactive Data Analysis Is Hard" (FOCS 2014), and Steinke & Ullman, "Interactive Fingerprinting Codes and the Hardness of Preventing False Discovery" (COLT 2015). The lower bound: about n^2 adaptive queries for efficient mechanisms.
- Blum & Hardt, "The Ladder: A Reliable Leaderboard for Machine Learning Competitions" (ICML 2015). The boosting attack and the Ladder.
- Mania, Miller, Schmidt, Hardt & Recht, "Model Similarity Mitigates Test Set Overuse" (NeurIPS 2019). Why similar models overfit a reused test set less.
- Freedman, "A Note on Screening Regression Equations" (*The American Statistician* 37, 1983). Freedman's paradox.
- Recht, Roelofs, Schmidt & Shankar, "Do ImageNet Classifiers Generalize to ImageNet?" (ICML 2019), and Roelofs, Shankar, Recht, Fridovich-Keil, Hardt, Miller & Schmidt, "A Meta-Analysis of Overfitting in Machine Learning" (NeurIPS 2019). How much adaptive overfitting happens in practice.
- Kamath, *CS 860: Algorithms for Private Data Analysis*, University of Waterloo (2020), lectures 16A and 16B. The point-query attack, the three regimes of Table 48.5 and the proof of the expectation transfer theorem, following notes by Nikolov and Steinke.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.6. The sparse vector technique behind Thresholdout.

A. How much noise to add

Every differentially private release in this book answers the same practical question. You have a number, or a list of numbers, computed from people's data, and you want to publish it with random noise added. How much noise is enough? This appendix answers that question from scratch for the mechanisms the book uses most. They are Laplace noise for pure differential privacy (DP), Gaussian noise for approximate DP, which allows a small failure probability, randomized response for a single bit, and the exponential mechanism for choosing among candidates. It assumes only that you know what a probability density, an average, a logarithm and an exponential are. Everything else, including the definition of differential privacy itself, is built up on the page.

The results sit together because they all rest on one quantity, the **privacy loss**. It measures how much more likely an output is when one person's record is in the data than when it is changed. Each mechanism is calibrated by bounding the privacy loss, everywhere for pure DP and except on outputs of small total probability for approximate DP. The chapters derive the same formulas with more general tools. Table A.1 shows where each one is stated and used, and its symbols are defined in the sections it points to.

Table A.1.: The calibration formulas derived in this appendix and the chapter sections that state or use them.

Section of this appendix	What it derives	Where the chapters state or use it
Section A.1	The definition, the privacy loss and the exact δ	Section 18.1, Section 18.2, Section 18.4
Section A.2	Sensitivity of counts, sums, means and histograms	Section 20.1
Section A.3	Laplace noise of scale Δ_1/ε gives ε-DP	Section 20.2
Section A.4	Randomized response and its unbiased estimate	Section 18.3
Section A.5	The exponential mechanism and its utility	Section 22.1, Section 22.2
Section A.6	The classic Gaussian formula $\sigma = \Delta_2\sqrt{2\ln(1.25/\delta)}/\varepsilon$	Section 20.3
Section A.7	The exact Gaussian $\delta(\varepsilon)$ and the smallest σ	Section 20.4, Section 20.5
Section A.8	Computing δ in code, and how ε, δ, σ and Δ_2 trade off	Section 20.4, Section 46.5

A.1. What differential privacy promises, and the privacy loss

A.1.1. The question

Before you can ask how much noise is enough, you need to know what "enough" means. What exactly must the noisy output satisfy, and how do you check it for one mechanism?

A.1.2. The answer

A dataset is a collection of records, one per person. Two datasets D and D' are **neighbours** if they differ in one person's record. The book uses two versions of this. Under **add/remove**, D' is D with one record added or removed, so the two have different numbers of records. The size n is then itself protected, because the output must not reveal which size is true. Under **replace-one**, D' is D with one record changed, so n is the same on both and treated as public. This appendix uses add/remove unless it says otherwise, as the chapters do for counts and histograms.

A **mechanism** M is a randomized algorithm: run on a dataset D, it returns a random output $M(D)$. Write $p_D(y)$ for the probability (or density) that $M(D)$ outputs y, and $p_{D'}(y)$ for the same on the neighbour. An **event** S is any set of outputs, such as "the released count is above 1240". The privacy parameter $\varepsilon \geq 0$ says how much one record may change the probability of an event: by at most a factor e^{ε}, so a small ε means a factor close to 1. Formally, M is **ε-differentially private** (ε-DP) if for every pair of neighbours and every event S,

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S]. \tag{A.1}$$

The weaker version allows a small slack δ on top of the factor. A mechanism is **(ε, δ)-differentially private** if, for every pair of neighbours and every event S,

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{A.2}$$

Read it as "the factor e^{ε} holds, except for a δ of probability". Both definitions must hold with D and D' swapped too, since a neighbour of D' is D.

The quantity that decides both is the **privacy loss** of an output y. It is the log of how much more likely y is on D than on D':

$$L(y) = \ln \frac{p_D(y)}{p_{D'}(y)}. \tag{A.3}$$

A loss of 0 means y is equally likely on both datasets and says nothing about which one was used. A loss of ε means y is e^{ε} times more likely on D. Two facts, proved below, turn the definitions into statements about L. First, if $L(y) \leq \varepsilon$ for every output y and every ordered pair of neighbours, the mechanism is ε-DP. Second, call the outputs where the loss exceeds ε the **bad set**, $B = \{y : L(y) > \varepsilon\}$. The smallest δ that works for one pair at a given ε is the probability of the bad set on D, minus e^{ε} times its probability on D':

$$\delta(\varepsilon) = \Pr[M(D) \in B] - e^{\varepsilon} \Pr[M(D') \in B]. \tag{A.4}$$

A.1.3. A worked example

Take a mechanism with three possible outputs, and the probabilities in Table A.2 on a dataset D and its neighbour D'.

Table A.2.: A mechanism with three outputs on two neighbouring datasets. The last row uses $e^{\varepsilon} = 2$.

Output y	1	2	3
$p_D(y)$	0.5	0.3	0.2
$p_{D'}(y)$	0.2	0.3	0.5
Ratio $p_D(y)/p_{D'}(y)$	2.5	1	0.4
Privacy loss $L(y)$	0.9163	0	-0.9163
$p_D(y) - 2\,p_{D'}(y)$	0.1	-0.3	-0.8

The largest ratio is 2.5, at output 1, so this pair is ε-DP with $\varepsilon = \ln 2.5 = 0.9163$ and with nothing smaller. The script that comes with this appendix (see the Code section at the end) checks all 7 non-empty events, and none has a larger ratio of probabilities than 2.5.

Now ask for a smaller ε, so that the factor e^{ε} is 2 instead of 2.5. Output 1 then breaks the factor, because its ratio is 2.5. It is the only output in the bad set, because only there is the ratio above 2. So Equation A.4 gives $\delta = 0.5 - 2 \times 0.2 = 0.1$. As a check, the script computes $\Pr[M(D) \in S] - 2\Pr[M(D') \in S]$ for every event S. The largest value is indeed 0.1000, reached by the event that holds output 1 alone. Figure A.1 draws it: δ is the red part of the first bar, the probability that sticks out above e^{ε} times the neighbour's.

Note that δ is not the probability of the bad set, which here is 0.5. Most of that probability is allowed by the factor e^{ε}, and only the excess counts.

A.1.4. Why it works

1. **A pointwise bound gives the event bound.** Suppose $L(y) \le \varepsilon$ at every output, that is $p_D(y) \le e^{\varepsilon} p_{D'}(y)$. The probability of an event is the sum of the probabilities of its outputs, so summing this inequality over the outputs in S gives $\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S]$. For a density, the sum is an integral, and the same step works. So bounding the loss at every output is enough for ε-DP.
2. **The slack for one event is a sum over its outputs.** Moving the e^{ε} term to the left in Equation A.2, the event S needs $\delta \ge \Pr[M(D) \in S] - e^{\varepsilon} \Pr[M(D') \in S]$. The right side is the sum, over the outputs y in S, of $p_D(y) - e^{\varepsilon} p_{D'}(y)$.
3. **The worst event takes exactly the outputs with a positive term.** To make that sum as large as possible, include every output whose term is positive and leave out every other one. A term is positive exactly when $p_D(y) > e^{\varepsilon} p_{D'}(y)$, that is when $L(y) > \varepsilon$. So the worst event is the bad set B, and the smallest δ that covers every event is its sum, Equation A.4.
4. **The probability of the bad set is always a valid δ.** Dropping the term $e^{\varepsilon} \Pr[M(D') \in B]$, which is never negative, gives $\delta(\varepsilon) \le \Pr[M(D) \in B] = \Pr[L > \varepsilon]$, where the loss is computed at an output drawn from $M(D)$. This simpler bound is what the classic Gaussian formula of Section A.6 uses.

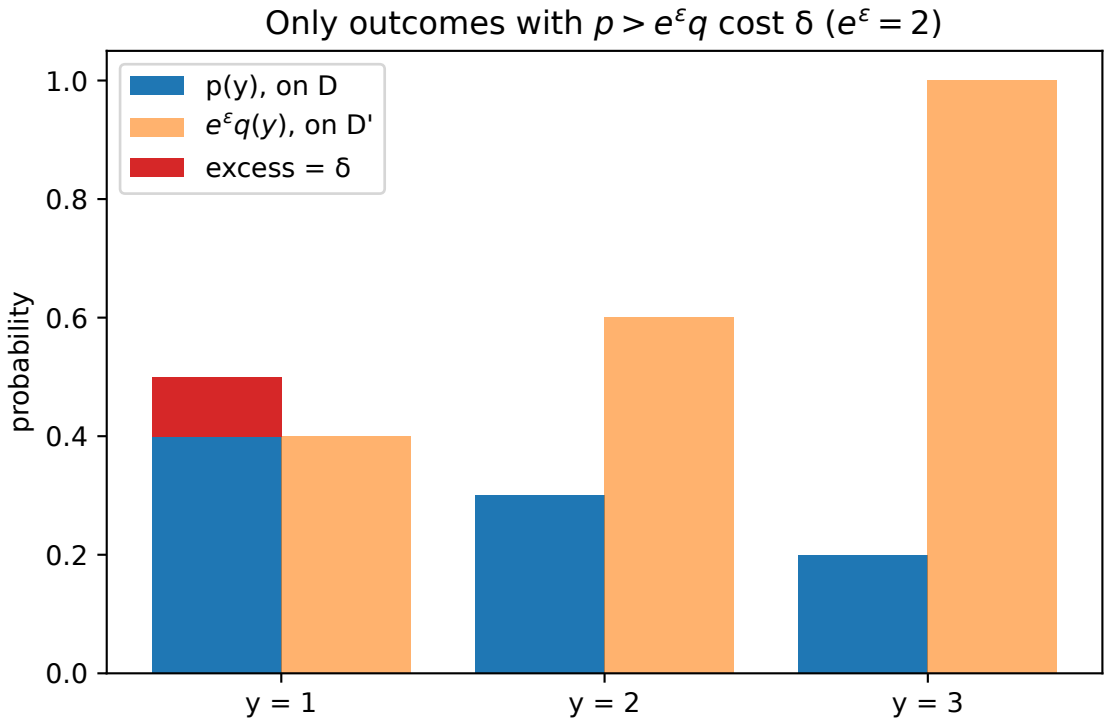


Figure A.1.: The three-output example. Blue bars are the probabilities on D, orange bars are $e^{\varepsilon} = 2$ times the probabilities on D'. Only output 1 has more probability on D than the orange bar allows, and the excess, in red, is the exact $\delta = 0.1$.

A.1.5. Checking it

The script lists every event of the three-output example and confirms both numbers by brute force. The largest ratio over events is 2.5000, the same as the largest ratio over single outputs. The largest excess at $e^{\varepsilon} = 2$ is 0.1000, the same as the sum over the bad set. The probability of the bad set, 0.5000, is five times larger than the exact δ, which shows how loose step 4 can be.

A.1.6. Recipe and pitfalls

To check a mechanism, write down its privacy loss for a pair of neighbours. If the loss never exceeds ε, you have ε-DP. If it does, the exact δ is the excess mass of Equation A.4, maximized over pairs of neighbours.

The most common mistake is to apply δ to each output instead of each event. "Every output has $p_D(y) \le e^{\varepsilon} p_{D'}(y) + \delta$" pays the slack once per output, and with many outputs that allows a mechanism that reveals everything (Section 18.2 gives the example). The second mistake is to forget the neighbouring relation. A replacement is a removal followed by an addition, so a guarantee under replace-one can cost up to twice the ε of one under add/remove. Every guarantee should therefore say which relation it uses. Section 18.4 writes the same exact δ as a divergence, and Section D.1 explains what ε means for an attacker.

A.2. Sensitivity: how much one person can move the answer

A.2.1. The question

Noise has to hide the effect of any one person. So before choosing the noise you need to know how far one person can move the answer. How do you compute that for the queries you actually run?

A.2.2. The answer

Let f be the query, a function that takes a dataset and returns d numbers. The **sensitivity** of f is the largest change that one person can cause, over all pairs of neighbours. Because the answer is a list of numbers, the change $v = f(D) - f(D')$ is a list too, and its size can be measured in two ways. The ℓ_1 **norm** $\|v\|_1 = |v_1| + \cdots + |v_d|$ adds up the absolute changes, and Laplace noise is calibrated to it. The ℓ_2 **norm** $\|v\|_2 = \sqrt{v_1^2 + \cdots + v_d^2}$ is the straight-line length of the change, and Gaussian noise is calibrated to it. So there are two sensitivities, one for each norm:

$$\Delta_1 = \max_{D \sim D'} \|f(D) - f(D')\|_1, \qquad \Delta_2 = \max_{D \sim D'} \|f(D) - f(D')\|_2. \tag{A.5}$$

Table A.3 gives them for the standard queries. Values are **clipped** to a range $[0, C]$: any value above C is replaced by C before the query runs, so no record can contribute more than C.

Table A.3.: Sensitivities of the standard queries under the two neighbouring relations.

Query	Add/remove Δ_1	Add/remove Δ_2	Replace-one Δ_1	Replace-one Δ_2
Count of records	1	1	0	0
Sum of values clipped to $[0, C]$	C	C	C	C
Mean of n clipped values, n public	no useful bound	no useful bound	C/n	C/n
Histogram, each record in one bin	1	1	2	$\sqrt{2}$
d counts that one record can all change by 1	d	$\sqrt{d}$	d	$\sqrt{d}$

A.2.3. A worked example

A study releases the mean age of $n = 1000$ patients, with ages clipped to $[0, 100]$, under replace-one. One patient's age can change from 0 to 100 at most, so the sum moves by at most $C = 100$. The mean is the sum divided by the fixed n, so it moves by at most $100/1000 = 0.1$ years. That is the sensitivity, $\Delta_1 = \Delta_2 = 0.1$, however old the patients actually are.

A.2.4. Why it works

Each row asks what the single worst record can do.

1. **Count.** Adding or removing a person changes the count by exactly 1. Replacing a person leaves it unchanged, which is why under replace-one the count of all records is public anyway.
2. **Sum of clipped values.** Adding a person with value x in $[0, C]$ changes the sum by x, at most C. Replacing x by x' changes it by $|x - x'|$, again at most C. Both bounds are reached with $x = C$ and $x' = 0$.
3. **Mean.** Under replace-one, the mean is the sum divided by the public n, so its sensitivity is the sum's divided by n, which is C/n. Under add/remove the divisor changes too, and the change depends on n, which is private. The script finds that for datasets of 1 to 3 values in $[0, 3]$ one record can move the mean by 1.5, from the mean of $\{0\}$ to the mean of $\{0, 3\}$. The usual fix is to release a noisy sum and a noisy count separately and divide them.
4. **Histogram.** Under add/remove the person lands in one bin, so one bin changes by 1. Both norms of the change are 1. Under replace-one one bin loses 1 and another gains 1, so the ℓ_1 norm is $1 + 1 = 2$ and the ℓ_2 norm is $\sqrt{1^2 + 1^2} = \sqrt{2}$.
5. **Counts that one person can all touch.** Suppose each person answers d yes/no questions and you release the number of "yes" answers to each. One person who says yes to all d moves every count by 1, so the ℓ_1 norm of the change is d and the ℓ_2 norm is $\sqrt{d \cdot 1^2} = \sqrt{d}$.

The last row shows why the two norms matter. When one person spreads their effect over many coordinates, Δ_1 grows like d but Δ_2 only like $\sqrt{d}$. Since the noise is scaled to the sensitivity, as the next sections show, this is why Gaussian noise wins for long vectors (Section 20.5).

A.2.5. Checking it

The script lists every dataset of 1 to 3 records and every neighbour under both relations, and takes the largest change. Values are in $\{0, 1, 2, 3\}$, so $C = 3$ and the histogram has 4 bins, and the vector query uses $d = 3$ questions. It reproduces every row of Table A.3. For example, the histogram has ℓ_1 sensitivity 1 under add/remove and 2 under replace-one, with ℓ_2 sensitivity 1.4142 under replace-one. The three counts have 3 and 1.7321. For the mean of exactly 3 records under replace-one it finds $3/3 = 1.0000$.

A.2.6. Recipe and pitfalls

First fix the neighbouring relation, then ask what the worst single record can do to each coordinate, and add the changes up in the norm your noise needs. Compute the sensitivity before you look at the data, because it is a worst case over everything the domain allows, not over the data you hold.

Unbounded data has infinite sensitivity. If ages, incomes or gradients can be as large as anyone likes, one record can move a sum by any amount, and no finite noise hides it. Clip every value to a fixed range first, and calibrate to the clipping norm. A second pitfall is a person who contributes many records: if one person can add 5 ratings, a count of ratings has sensitivity 5 per person, not 1. Section C.1 applies the same reasoning to clipped gradients.

A.3. Laplace noise: scale Δ_1/ε gives ε-DP

A.3.1. The question

You want to publish a count, or a few numbers, with pure ε-DP. How wide must the Laplace noise be, and how far will the published answer be from the truth?

A.3.2. The answer

The **Laplace distribution** is a two-sided exponential, peaked at 0. With scale b, its typical size, it puts density $\frac{1}{2b}e^{-|z|/b}$ at each value z. The **Laplace mechanism** adds independent Laplace noise to each of the d numbers, with the scale set to the ℓ_1 sensitivity divided by ε:

$$M(D) = f(D) + (Z_1, \dots, Z_d), \qquad Z_i \sim \mathrm{Lap}(b), \qquad b = \frac{\Delta_1}{\varepsilon}. \tag{A.6}$$

It is ε-DP. For one number, the error is the noise itself, and it exceeds a threshold τ with probability exactly $e^{-\tau/b}$. So with probability 0.95 the error is at most $b \ln 20$.

A.3.3. A worked example

A hospital has 1,234 patients with diabetes. On the neighbouring dataset D' there is one more, so the two true counts are 1234 and 1235, and $\Delta_1 = 1$. With $\varepsilon = 0.5$, the scale is $b = 1/0.5 = 2$.

The privacy loss at an output y is the log of the ratio of the two Laplace densities. The factors $\frac{1}{2b}$ cancel, and the log of a ratio of exponentials is the difference of the exponents, so

$$L(y) = \frac{|y - 1235| - |y - 1234|}{b}. \tag{A.7}$$

Table A.4 evaluates it at five outputs.

Table A.4.: The Laplace mechanism for a count with $b = 2$: the two densities and the privacy loss at five outputs.

Output y	1230	1234	1234.5	1235	1240
$p_D(y)$	0.0338	0.2500	0.1947	0.1516	0.0124
$p_{D'}(y)$	0.0205	0.1516	0.1947	0.2500	0.0205
Ratio	1.6487	1.6487	1.0000	0.6065	0.6065
$L(y)$	0.5	0.5	0	-0.5	-0.5

At $y = 1230$ the loss is $(5 - 4)/2 = 0.5$, and at $y = 1240$ it is $(5 - 6)/2 = -0.5$. Below both true counts the ratio is stuck at $e^{0.5} = 1.6487$, above both it is stuck at $e^{-0.5} = 0.6065$, and between them it moves from one to the other. It never goes further, however far out the output lands.

For accuracy, the 95% error bound is $b \ln 20 = 2 \times 2.9957 = 5.99$. So the published count is within about 6 of 1234 in 95% of releases. The bound scales with $1/\varepsilon$, so it is 29.96 at $\varepsilon = 0.1$, where the scale is 10, and 3.00 at $\varepsilon = 1$.

A.3.4. Why it works

1. **Only the shift matters.** Write $a = f(D)$ and $a' = f(D')$ for the two true answers. For one number, the privacy loss at y is $(|y - a'| - |y - a|)/b$, as in the example.
2. **Bound it with the triangle inequality.** The distance from y to a' is at most the distance from y to a plus the distance from a to a', so $|y - a'| - |y - a| \le |a - a'|$. In the example, with $y = 1230$, this says $5 - 4 \le 1$.
3. **The shift is at most the sensitivity.** So $L(y) \le |a - a'|/b \le \Delta_1/b$, and with $b = \Delta_1/\varepsilon$ that is ε. This holds at every output y.
4. **Several numbers.** With d coordinates and independent noise, the density of the output is the product of the d coordinate densities. The log of a product is a sum, so the privacy loss is the sum of the coordinate losses. Each is at most $|a_i - a'_i|/b$ by step 2, and their sum is at most $\|a - a'\|_1/b \le \Delta_1/b = \varepsilon$.
5. **Conclude.** The loss is at most ε at every output, for every pair of neighbours in either order. So by step 1 of Section A.1 the mechanism is ε-DP.
6. **Accuracy.** The absolute error $|Z|$ has density $\frac{1}{b}e^{-z/b}$ for $z \ge 0$, twice the Laplace density on one side. Its integral from τ to infinity is $e^{-\tau/b}$. Setting $e^{-\tau/b} = 0.05$ and solving gives $\tau = b \ln 20$. The same density has average $\int_0^\infty z\frac{1}{b}e^{-z/b}\,\mathrm{d}z = b$, so the typical error is b.

The bound of step 2 is reached, with equality, at every output below both true answers. So with a smaller scale b the loss there would be $1/b > \varepsilon$, and those outputs would break the factor e^{ε}. For the count above, the event "the output is at most 1234" has probability 0.5 on D and $\frac{1}{2}e^{-1/b} = 0.3033$ on D', a ratio of exactly $e^{0.5}$.

A.3.5. Checking it

Figure A.2 draws the two densities and their ratio for the example. The ratio is flat at e^{ε} on the left and at $e^{-\varepsilon}$ on the right, and it never leaves that band. Figure A.3 draws the privacy loss itself. It is flat outside the two true answers and a straight line between them, and it stays between -0.5 and 0.5. On a grid of 70,001 outputs from 1200 to 1270, the script finds the loss in exactly $[-0.5000, 0.5000]$.

For several numbers, the script takes three yes/no counts that one person can all raise by 1, so $\Delta_1 = 3$ and $b = 3/0.5 = 6$. Over 100,000 random outputs, the largest privacy loss is 0.5000, the ε that step 4 promises.

For accuracy, a million draws of Lap(2) give a mean absolute error of 2.0026 against the exact 2. Their 95th percentile is 5.9938 against $b \ln 20 = 5.9915$. The error exceeds 5.9915 in a fraction 0.0501 of draws.

A.3.6. Recipe and pitfalls

1. Compute the ℓ_1 sensitivity Δ_1 under your neighbouring relation.
2. Set $b = \Delta_1/\varepsilon$ and add independent Lap(b) noise to every coordinate.
3. Quote the error: b on average, and $b \ln 20$ with probability 0.95 for each coordinate.

The noise does not depend on the number of people, so a count of a million people and a count of ten get the same noise. Small groups are therefore expensive. Use fresh noise for every release, because reusing the same noise draw lets an analyst subtract it out. Releasing the same count k times with fresh noise costs $k\varepsilon$, which Section B.1 derives. Section 20.2 adds the bound on the largest error over d coordinates.

A.4. Randomized response: one bit, ε-DP

A.4.1. The question

Each person holds one sensitive bit, such as "I have used a drug", and nobody is trusted to see the true bits. How often should each person tell the truth, and how do you estimate the fraction of ones from the noisy reports?

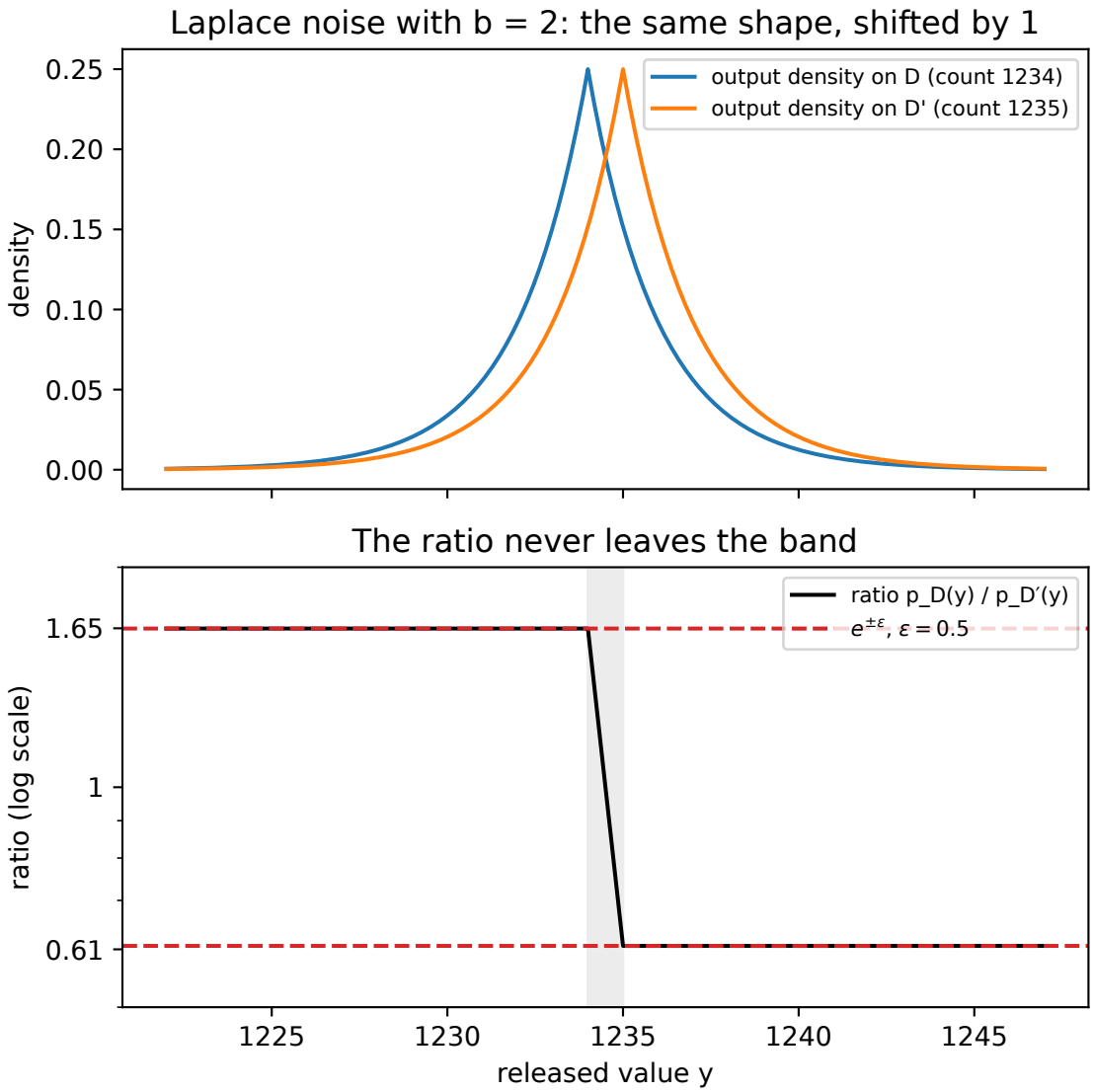


Figure A.2.: Top: the output densities of a count released with Laplace noise of scale 2, on a dataset with true count 1234 and on its neighbour with 1235. Bottom: their ratio on a log scale, which stays between $e^{-0.5}$ and $e^{0.5}$ at every output and reaches both ends.

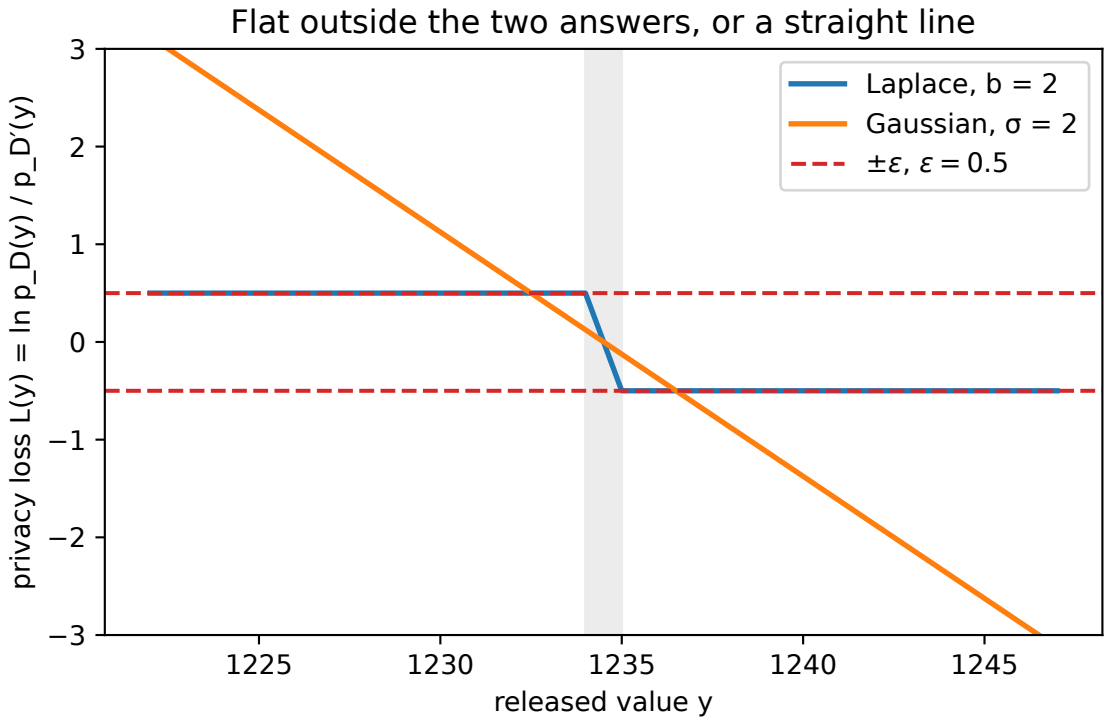


Figure A.3.: The privacy loss $L(y)$ of the same count against the released value, for Laplace noise of scale 2 and for Gaussian noise of standard deviation 2. The Laplace loss is flat outside the two true answers and never leaves $[-0.5, 0.5]$. The Gaussian loss is a straight line that leaves any band.

A.4.2. The answer

Each person reports their true bit with probability t and the flipped bit with probability $1 - t$, where

$$t = \frac{e^{\varepsilon}}{1 + e^{\varepsilon}}. \tag{A.8}$$

This is ε-DP under replace-one. If $\bar{y}$ is the fraction of reported ones among n people, an unbiased estimate of the true fraction π of ones is

$$\hat{\pi} = \frac{\bar{y} - (1 - t)}{2t - 1}, \qquad \mathrm{sd}(\hat{\pi}) = \frac{\sqrt{t(1 - t)}}{(2t - 1)\sqrt{n}}. \tag{A.9}$$

A.4.3. A worked example

At $\varepsilon = 0.5$, $t = e^{0.5}/(1 + e^{0.5}) = 0.6225$. A true 1 is reported as 1 with probability 0.6225, and a true 0 is reported as 1 with probability 0.3775. Their ratio is $0.6225/0.3775 = 1.6487 = e^{0.5}$.

Survey $n = 1000$ people, of whom a fraction $\pi = 0.3$ have the bit. The expected fraction of reported ones is $0.3 \times 0.6225 + 0.7 \times 0.3775 = 0.4510$. The estimate subtracts $1 - t = 0.3775$ and divides by $2t - 1 = 0.2449$, which turns this back into 0.3. Its standard deviation is $\sqrt{0.6225 \times 0.3775}/(0.2449\sqrt{1000}) = 0.0626$.

A.4.4. Why it works

1. **Privacy.** There are two inputs and two outputs. A report of 1 has probability t if the bit is 1 and $1 - t$ if it is 0, so its privacy loss is $\ln(t/(1 - t))$. With $t = e^{\varepsilon}/(1 + e^{\varepsilon})$, the ratio is $t/(1 - t) = e^{\varepsilon}$. A report of 0 has the inverse ratio, so $|L| \le \varepsilon$ at both outputs.
2. **Many people.** People flip their own coins independently, so the probability of all the reports is a product with one factor per person. Under replace-one, neighbours differ in one person's bit, so every other factor is the same on both datasets and cancels in the ratio. The loss is that one person's loss.
3. **The expected report.** A person with bit 1 reports 1 with probability t, and a person with bit 0 with probability $1 - t$. Averaging over the population gives $\mathbb{E}[\bar{y}] = \pi t + (1 - \pi)(1 - t) = (1 - t) + \pi(2t - 1)$.
4. **Unbias it.** Solving that equation for π gives $\hat{\pi}$. It is a linear function of $\bar{y}$, so its average is the true π.
5. **The spread.** Each report is a 0/1 variable with variance $t(1 - t)$, whatever the true bit, and the reports are independent. So $\bar{y}$ has variance $t(1 - t)/n$. Dividing by $2t - 1$ divides the standard deviation by $2t - 1$.

A.4.5. Checking it

Ten thousand simulated surveys give a mean estimate of 0.2996 and a standard deviation of 0.0629, against 0.3 and 0.0626 from the formulas.

A.4.6. Recipe and pitfalls

Set $t = e^{\varepsilon}/(1+e^{\varepsilon})$, collect the reports, and compute $\hat{\pi}$. Expect an error of about $1/(\varepsilon\sqrt{n})$ for small ε. The reason is that $e^{\varepsilon} \approx 1+\varepsilon$ there, so $t \approx (1+\varepsilon)/2$, which makes $2t-1 \approx \varepsilon/2$ and $t(1-t) \approx \frac{1}{4}$.

The price of trusting nobody is large. If one trusted party could see the true bits, it could add $\mathrm{Lap}(1/\varepsilon)$ noise to their count instead, and the error on the fraction would fall like $1/n$, not like $1/\sqrt{n}$. The guarantee also holds only under replace-one. Under add/remove the neighbour has one report more or fewer, and the number of reports, which is published, shows the difference at once. Section 18.3 compares the two designs, and Warner's original survey design is the same mechanism.

A.5. The exponential mechanism: choosing the best candidate

A.5.1. The question

Sometimes the output is not a number but a choice: the best model, the most common diagnosis, the winner of a vote. Adding noise to a name makes no sense. How do you choose privately, and how good is the choice?

A.5.2. The answer

Fix a list of candidates $\mathcal{R}$ in advance, and write $|\mathcal{R}|$ for their number. For each candidate r, a score $u(D, r)$ says how good it is on dataset D, where higher is better. Let Δu be the sensitivity of the score, the most that one person can change any one candidate's score.

The **exponential mechanism** gives each candidate a weight $\exp(\varepsilon u(D,r)/(2\Delta u))$, which grows with its score. It then picks a candidate with probability equal to its share of the total weight $W(D)$, the **normalizer**:

$$\Pr[M_E(D) = r] = \frac{\exp\left(\frac{\varepsilon\, u(D,r)}{2\Delta u}\right)}{W(D)}, \qquad W(D) = \sum_{s \in \mathcal{R}} \exp\left(\frac{\varepsilon\, u(D,s)}{2\Delta u}\right). \tag{A.10}$$

Dividing by the normalizer makes the probabilities add up to 1. The mechanism is ε-DP, and its choice is rarely far below the best score $\mathrm{OPT}(D)$. For every $t > 0$,

$$\Pr\left[u(D, M_E(D)) \le \mathrm{OPT}(D) - \frac{2\Delta u}{\varepsilon}(\ln|\mathcal{R}| + t)\right] \le e^{-t}. \tag{A.11}$$

A.5.3. A worked example

Six people vote among options A, B, C and D. On D the counts are $(3, 2, 1, 0)$, and on the neighbour D' one more person votes for B, so the counts are $(3, 3, 1, 0)$. The score is the count, so $\Delta u = 1$. Take $\varepsilon = 1$, so each weight is $e^{u/2}$.

On D the weights are $e^{1.5} = 4.4817$, $e^{1} = 2.7183$, $e^{0.5} = 1.6487$ and $e^{0} = 1$, which sum to $W(D) = 9.8487$. So A is chosen with probability $4.4817/9.8487 = 0.4551$. On D' B's weight rises to $e^{1.5}$ and the normalizer to $W(D') = 11.6121$. Table A.5 lists all four probabilities.

Table A.5.: The exponential mechanism with $\varepsilon = 1$ on a vote and its neighbour with one more vote for B.

Option	A	B	C	D
$\Pr[M_E(D) = r]$	0.4551	0.2760	0.1674	0.1015
$\Pr[M_E(D') = r]$	0.3859	0.3859	0.1420	0.0861
Ratio	1.1790	0.7151	1.1790	1.1790

For A, C and D the score did not change, so the ratio is just the change in the normalizer, $11.6121/9.8487 = 1.1790$. For B, its own weight is also $e^{1/2}$ times larger on D' than on D, so its ratio is $1.1790 \times e^{-1/2} = 1.1790 \times 0.6065 = 0.7151$. The largest privacy loss on this pair is $|\ln 0.7151| = 0.3353$, well below $\varepsilon = 1$.

A.5.4. Why it works

Privacy. The probability of r is a weight divided by a total, and one person can change both. So split the ratio of the probabilities on D and D' into two factors:

$$\frac{\Pr[M_E(D) = r]}{\Pr[M_E(D') = r]} = \exp\Big(\frac{\varepsilon\big(u(D, r) - u(D', r)\big)}{2\Delta u}\Big) \cdot \frac{W(D')}{W(D)}. \tag{A.12}$$

1. **The own-weight factor.** One person changes r's score by at most Δu, so the exponent is at most $\varepsilon\Delta u/(2\Delta u) = \varepsilon/2$. The factor is at most $e^{\varepsilon/2}$.
2. **The normalizer factor.** Every candidate's score on D' is at most Δu above its score on D, so every term of $W(D')$ is at most $e^{\varepsilon/2}$ times the matching term of $W(D)$. Adding up the terms, $W(D') \le e^{\varepsilon/2} W(D)$.
3. **Multiply.** The ratio is at most $e^{\varepsilon/2} \cdot e^{\varepsilon/2} = e^{\varepsilon}$. The candidate and the pair were arbitrary, and the same argument works with D and D' swapped, so by step 1 of Section A.1 the mechanism is ε-DP.

The 2 in the exponent is there because both factors can be large at once. One person can raise r's score and lower all the others, so each factor needs its own half of the budget.

Utility. Call a candidate poor if its score is at least c below the best.

1. **Poor candidates are light.** Each has weight at most $e^{\varepsilon(\mathrm{OPT}-c)/(2\Delta u)}$, since the weight grows with the score.
2. **The normalizer is heavy.** It contains the weight of a best candidate, so $W(D) \ge e^{\varepsilon\,\mathrm{OPT}/(2\Delta u)}$.
3. **Divide.** There are at most $|\mathcal{R}|$ poor candidates, so together they have probability at most $|\mathcal{R}|\, e^{\varepsilon(\mathrm{OPT}-c)/(2\Delta u)} / e^{\varepsilon\,\mathrm{OPT}/(2\Delta u)} = |\mathcal{R}|\, e^{-\varepsilon c/(2\Delta u)}$.
4. **Solve for c.** Setting $|\mathcal{R}|\, e^{-\varepsilon c/(2\Delta u)} = e^{-t}$ and taking logs gives $c = \frac{2\Delta u}{\varepsilon}(\ln|\mathcal{R}| + t)$.

A.5.5. Checking it

Figure A.4 shows the vote. On the right, each ratio is drawn as its two factors, and both stay below $e^{\varepsilon/2}$. Only B's score moved, so for A, C and D the own-weight factor is exactly 1.

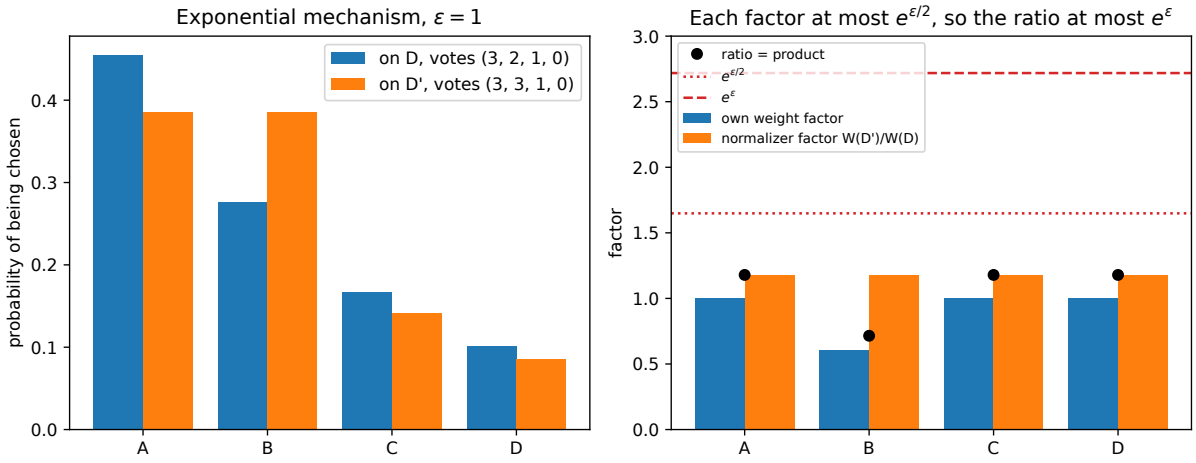


Figure A.4.: Left: the probability that the exponential mechanism with $\varepsilon = 1$ chooses each option, on a vote with counts (3, 2, 1, 0) and on its neighbour with one more vote for B. Right: each ratio of the two probabilities, split into the change of the option's own weight and the change of the normalizer. Each factor stays below $e^{\varepsilon/2}$, so the ratio stays below e^{ε}.

The utility bound is weak for so small an example. The probability of choosing C or D, at least $c = 2$ below A, is 0.2689. Step 3 with $|\mathcal{R}| = 4$ bounds it by $4e^{-1} = 1.4715$, which says nothing. It helps when there are many candidates. Take 50 models scored by their correct predictions, $900, 899, \dots, 851$, so that $\Delta u = 1$. With $\varepsilon = 1$ and $t = 3$, the bound says the gap exceeds $2(\ln 50 + 3) = 13.82$ with probability at most $e^{-3} = 0.0498$. The exact probability is 9.12×10^{-4}, much smaller, because most models are far below the best and much lighter than the bound assumes.

A.5.6. Recipe and pitfalls

1. Fix the list of candidates before looking at the data, and choose a score $u(D, r)$.
2. Find the score's sensitivity Δu.
3. Sample r with probability proportional to $\exp(\varepsilon u(D, r)/(2\Delta u))$.

The candidate list must not depend on the data, because the proof compares the same list on D and D'. Watch numerical overflow too: subtract the largest score before taking exponentials, which changes no probability. When every score moves the same way under each change of one person, as counts under add/remove do, the two factors cannot both be large, and the 2 can be dropped (Section 22.1). Section 22.2 adds the expected gap and the applications.

A.6. Gaussian noise, the classic formula

A.6.1. The question

Gaussian noise is calibrated to the ℓ_2 sensitivity, which is much smaller than the ℓ_1 sensitivity for long vectors such as gradients. But its privacy loss has no upper limit, so it can never give pure ε-DP. How much Gaussian noise gives (ε, δ)-DP, and where does the textbook formula come from?

A.6.2. The answer

Add independent normal noise with mean 0 and standard deviation σ to each coordinate. For $0 < \varepsilon < 1$ and $0 < \delta < 1$, the **Gaussian mechanism** with

$$\sigma = \frac{\Delta_2 \sqrt{2 \ln(1.25/\delta)}}{\varepsilon} \tag{A.13}$$

is (ε, δ)-DP. The proof below needs $\delta \leq 0.7448$, which every useful δ satisfies.

A.6.3. A worked example

Release the diabetes count with $\Delta_2 = 1$, $\varepsilon = 0.5$ and $\delta = 10^{-5}$. Then $\ln(1.25/10^{-5}) = \ln 125{,}000 = 11.7361$, and its double has square root $c = \sqrt{23.4722} = 4.8448$. So $\sigma = 4.8448/0.5 = 9.6896$. The 95% error of normal noise is 1.96 standard deviations, here 18.99, against 5.99 for Laplace noise at the same ε. For a single count Laplace noise is better, and Gaussian noise pays off only for long vectors.

A.6.4. Why it works

The plan is to find the privacy loss, which turns out to be a normal random variable. Then show that it exceeds ε with probability at most δ, and that this is enough. Write $\Phi(x)$ for the standard normal cumulative distribution function, the probability that a normal variable with mean 0 and standard deviation 1 is below x, and $\varphi(x) = e^{-x^2/2}/\sqrt{2\pi}$ for its density.

1. **The privacy loss is a straight line in the noise.** Take one number, with true answers a on D and $a + s$ on D', where s is the shift between them. An output from $M(D)$ is $y = a + z$, where z is the noise. Its density on D is proportional to $e^{-z^2/(2\sigma^2)}$ and on D' to $e^{-(z-s)^2/(2\sigma^2)}$, because y sits $z - s$ away from D''s answer. The log of the ratio is the difference of the exponents:

$$L = \frac{(z-s)^2 - z^2}{2\sigma^2} = \frac{s^2 - 2sz}{2\sigma^2}. \tag{A.14}$$

2. **So the loss is normal.** The noise z is normal with mean 0 and standard deviation σ, and L is a constant minus a multiple of z. So L is normal with mean $s^2/(2\sigma^2)$ and standard deviation $2s\sigma/(2\sigma^2) = s/\sigma$. Write $\mu = s/\sigma$ for the shift measured in noise standard deviations. Then L has mean $\mu^2/2$ and standard deviation μ. For a vector the same holds with $s = \|f(D) - f(D')\|_2$. The reason is that expanding the squared distances in the same way leaves only the noise's component along the direction of the shift, and that component is a single normal number with standard deviation σ.
3. **A small chance of a large loss is enough.** Let B be the bad set, the outputs with $L > \varepsilon$, and suppose its probability on D is at most δ. Split any event S into its part inside B and its part outside. The part inside has probability at most δ, because it is part of B. On the part outside, every output has $p_D(y) \le e^{\varepsilon} p_{D'}(y)$, so summing over it gives at most $e^{\varepsilon} \Pr[M(D') \in S]$. Adding the two parts gives

$$\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \delta, \tag{A.15}$$

   which is Equation A.2. The pair in the other order has the same loss distribution, by the symmetry of the normal curve, so it needs nothing new.
4. **The bad event is a normal tail.** Standardize the loss by subtracting its mean $\mu^2/2$ and dividing by its standard deviation μ. The result is a standard normal variable, and $L > \varepsilon$ exactly when it exceeds the point $t = (\varepsilon - \mu^2/2)/\mu = \varepsilon/\mu - \mu/2$. By the symmetry of the normal curve, $\Pr[L > \varepsilon] = \Phi(-t)$. A larger shift makes t smaller and the tail heavier, so the worst pair has the largest shift, $s = \Delta_2$.
5. **Put in the formula.** Write $c = \sqrt{2\ln(1.25/\delta)}$, so Equation A.13 is $\sigma = c\Delta_2/\varepsilon$. Then $\mu = \Delta_2/\sigma = \varepsilon/c$, and

$$t = \frac{\varepsilon}{\mu} - \frac{\mu}{2} = c - \frac{\varepsilon}{2c}. \tag{A.16}$$

6. **Bound the normal tail.** For $t > 0$, every point z beyond t has $z/t \ge 1$, so the tail $\int_t^\infty \varphi(z)\,\mathrm{d}z$ is at most $\int_t^\infty \frac{z}{t}\varphi(z)\,\mathrm{d}z$. The derivative of $\varphi(z)$ is $-z\varphi(z)$, so the second integral equals $\varphi(t)/t$. This is the **Mills-ratio bound**:

$$\Phi(-t) \le \frac{e^{-t^2/2}}{t\sqrt{2\pi}}. \tag{A.17}$$

7. **Expand t^2.** Squaring step 5 gives $t^2 = c^2 - \varepsilon + \varepsilon^2/(4c^2)$, which is at least $c^2 - \varepsilon$. So $e^{-t^2/2} \le e^{-c^2/2} e^{\varepsilon/2}$. By the definition of c, $e^{-c^2/2} = \delta/1.25$. Together with step 6,

$$\Pr[L > \varepsilon] \le \delta \cdot \frac{e^{\varepsilon/2}}{1.25\sqrt{2\pi}\,t}. \tag{A.18}$$

8. **This is where 1.25 and $\varepsilon < 1$ come in.** The bound is at most δ when the fraction is at most 1, that is when $t \ge e^{\varepsilon/2}/(1.25\sqrt{2\pi})$. For $\varepsilon < 1$, the right side is below $e^{1/2}/(1.25\sqrt{2\pi}) = 0.5262$. It remains to show that t is that large. For $\varepsilon < 1$, step 5 gives $t = c - \varepsilon/(2c) \ge c - 1/(2c)$. This lower bound grows with c, and it equals 0.5262 at $c = 1.0176$, the positive root of $c^2 - 0.5262c - \frac{1}{2} = 0$. Since c grows as δ shrinks, $c \ge 1.0176$ holds whenever $\delta \le 1.25e^{-1.0176^2/2} = 0.7448$. So the constant 1.25 absorbs the factor $e^{\varepsilon/2}/(t\sqrt{2\pi})$, and the condition $\varepsilon < 1$ keeps $e^{\varepsilon/2}$ below $e^{1/2}$.

Step 3 is the reason a tail probability is enough. The rare outputs with a large loss are paid for by δ, and every other output keeps the factor e^{ε}.

A.6.5. Checking it

The proof made three approximations in a row. Step 4 replaced the exact δ by the bad-set probability, step 6 replaced that by the Mills bound, and step 7 loosened it further with the constants. Figure A.5 follows each of them for ε up to 1 at $\delta = 10^{-5}$. At $\varepsilon = 0.5$ the numbers are in Table A.6.

Table A.6.: Each step of the classic proof at $\varepsilon = 0.5$, $\delta = 10^{-5}$, $\Delta_2 = 1$, where $t = 4.7932$.

Quantity at the classic $\sigma = 9.6896$	Value
Target δ	10^{-5}
After expanding t^2, step 7	8.55×10^{-7}
Mills bound $\varphi(t)/t$, step 6	8.54×10^{-7}
Bad event $\Pr[L > \varepsilon] = \Phi(-t)$, step 4	8.21×10^{-7}
Exact $\delta(\varepsilon)$, from Section A.7	1.61×10^{-8}

So the classic σ delivers a δ 622 times smaller than asked for. The constants of step 8 cost a factor of 11.7, and dropping the correction term of Equation A.4 costs a factor of 51.0. The Mills ratio itself costs almost nothing.

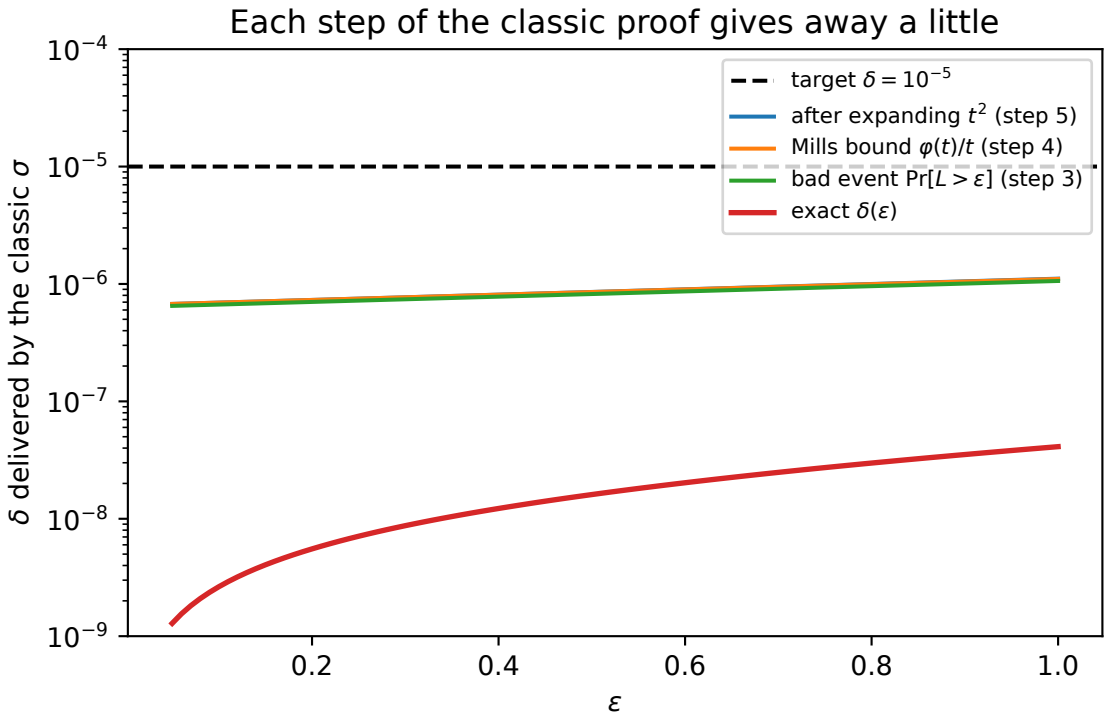


Figure A.5.: The δ that the classic σ delivers at each ε up to 1, for a target $\delta = 10^{-5}$, at each step of the proof. The bound after step 7, the Mills bound and the bad-event probability nearly coincide, about ten times below the target. The exact δ is a further 25.8 times smaller at $\varepsilon = 1$ and 505.9 times smaller at $\varepsilon = 0.05$.

A.6.6. Recipe and pitfalls

1. Compute the ℓ_2 sensitivity Δ_2.
2. For $\varepsilon < 1$, set $\sigma = \Delta_2\sqrt{2\ln(1.25/\delta)}/\varepsilon$.
3. Add independent normal noise with that σ to every coordinate.

The formula is valid but wasteful, as the table shows. The proof says nothing for $\varepsilon \geq 1$, and Section A.7 shows that the formula eventually becomes wrong there. Choose δ much smaller than $1/n$. A mechanism that publishes each person's record in full with probability δ satisfies the definition, and with n people and $\delta = 1/n$ it publishes about one person's record every time. Section 20.3 gives the same proof with the vector case in full.

A.7. Gaussian noise, the exact calibration

A.7.1. The question

The classic formula overshoots by a large factor, and it says nothing for $\varepsilon \geq 1$. What is the smallest σ that gives (ε, δ)-DP, for any ε?

A.7.2. The answer

Measure the sensitivity in units of the noise, $\mu = \Delta_2/\sigma$. The Gaussian mechanism is (ε, δ)-DP exactly when $\delta \geq \delta(\varepsilon)$, where

$$\delta(\varepsilon) = \Phi\Big(\frac{\Delta_2}{2\sigma} - \frac{\varepsilon\sigma}{\Delta_2}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\Delta_2}{2\sigma} - \frac{\varepsilon\sigma}{\Delta_2}\Big) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big). \tag{A.19}$$

This $\delta(\varepsilon)$ falls as σ grows. So the smallest σ for a target (ε, δ) is the one solution of $\delta(\varepsilon) = \delta$, which a computer finds by bisection.

A.7.3. A worked example

To see the two terms, take a case where both terms are large enough to see, $\mu = 1$ and $\varepsilon = 1$. The first term is $\Phi(0.5 - 1) = \Phi(-0.5) = 0.3085$, the probability of the bad set on D. The second is $e \cdot \Phi(-1.5) = 0.1816$, the correction. Their difference is the exact $\delta = 0.1269$.

Now the count again, with $\Delta_2 = 1$, $\varepsilon = 0.5$ and $\delta = 10^{-5}$. Bisection finds $\sigma = 7.0318$, and putting it back into Equation A.19 gives $\delta = 1.000 \times 10^{-5}$. The classic formula gave 9.6896, which is 1.378 times more noise than needed. At the classic σ the true δ is 1.61×10^{-8}. With the exact σ the 95% error is $1.96 \times 7.0318 = 13.78$, against 18.99 with the classic one.

A.7.4. Why it works

1. **Start from the exact δ.** By Equation A.4, the smallest δ for one pair of neighbours is $\Pr[M(D) \in B] - e^{\varepsilon} \Pr[M(D') \in B]$, where B is the set of outputs with $L > \varepsilon$.
2. **The first term.** Step 4 of Section A.6 found $\Pr[M(D) \in B] = \Phi(-t)$ with $t = \varepsilon/\mu - \mu/2$, which is $\Phi(\mu/2 - \varepsilon/\mu)$.
3. **The second term.** Measure the output in units of σ, starting from D's true answer. The bad set is the outputs below $\mu/2 - \varepsilon/\mu$, by step 4 of Section A.6. On D' the output is centred at μ instead of 0, so it falls below that point with probability $\Phi(\mu/2 - \varepsilon/\mu - \mu) = \Phi(-\mu/2 - \varepsilon/\mu)$.
4. **Combine.** Subtracting e^{ε} times the second term from the first gives Equation A.19 for this pair.
5. **The largest shift is the worst pair, and more noise always helps.** The plan is to show that δ grows with μ, by computing its derivative. Write $\alpha = \mu/2 - \varepsilon/\mu$ and $\beta = -\mu/2 - \varepsilon/\mu$ for the two arguments, so that $\delta = \Phi(\alpha) - e^{\varepsilon}\Phi(\beta)$. Their derivatives in μ are $\frac{1}{2} + \varepsilon/\mu^2$ and $-\frac{1}{2} + \varepsilon/\mu^2$, which differ by exactly 1. Also $\alpha^2 - \beta^2 = (\alpha - \beta)(\alpha + \beta) = \mu \cdot (-2\varepsilon/\mu) = -2\varepsilon$, so $\varphi(\beta) = e^{-\varepsilon}\varphi(\alpha)$. The derivative of δ in μ is therefore

$$\varphi(\alpha)\Big(\frac{1}{2} + \frac{\varepsilon}{\mu^2}\Big) - e^{\varepsilon}e^{-\varepsilon}\varphi(\alpha)\Big(-\frac{1}{2} + \frac{\varepsilon}{\mu^2}\Big) = \varphi(\alpha) > 0. \tag{A.20}$$

   So δ grows with the shift μ, and the worst pair of neighbours is the one with the largest shift, Δ_2. And since $\mu = \Delta_2/\sigma$, a larger σ means a smaller μ and a smaller δ.
6. **"Exactly when."** The worst pair needs $\delta \geq \delta(\varepsilon)$ by step 1, and every other pair needs less by step 5. So $\delta \geq \delta(\varepsilon)$ is both necessary and enough.
7. **Solve by bisection.** As σ grows from near 0 to very large, $\delta(\varepsilon)$ falls steadily from near 1 to near 0, so it crosses the target exactly once. Bisection keeps an interval of σ values with the crossing inside, evaluates δ at its midpoint, and keeps the half that still contains the crossing. Each step halves the interval, and the script takes its midpoints in $\ln \sigma$ because σ can range over many orders of magnitude.

Intuitively, step 5 says that adding more noise can only make the two datasets harder to tell apart. A Gaussian mechanism with a larger σ is the one with a smaller σ plus extra independent noise, and anything you can detect after adding that extra noise yourself, you could detect before.

A.7.5. Checking it

Figure A.6 draws the derivation at $\mu = 1$ and $\varepsilon = 1$. On the left, the bad set is the outputs left of -0.5σ, where the density on D rises above e^{ε} times the density on D'. The red area between the two curves is δ, and the orange area under it is the correction that the classic proof throws away. On the right, the same δ appears as the part of the privacy loss distribution beyond $L = \varepsilon$.

The script checks the closed form in three ways. First, it integrates the excess $p_D - e^{\varepsilon}p_{D'}$ over the bad set numerically (by Simpson's rule, on a fine grid), and at $\mu = 1$, $\varepsilon = 1$ this gives 0.1269. The same δ is also the average of $(1 - e^{\varepsilon - L})_+$ over losses drawn on D, where $(x)_+$ means $\max(x, 0)$ (Section 18.4 shows why). A Monte Carlo average of it over a million random draws gives 0.1270. Second, over 22 pairs of ε and μ, the largest relative difference between the closed form and the numerical integral is 3.1×10^{-6}. Third, a numerical derivative in μ agrees with $\varphi(\alpha)$ of step 5 to within 10^{-9} on a grid of 16 pairs.

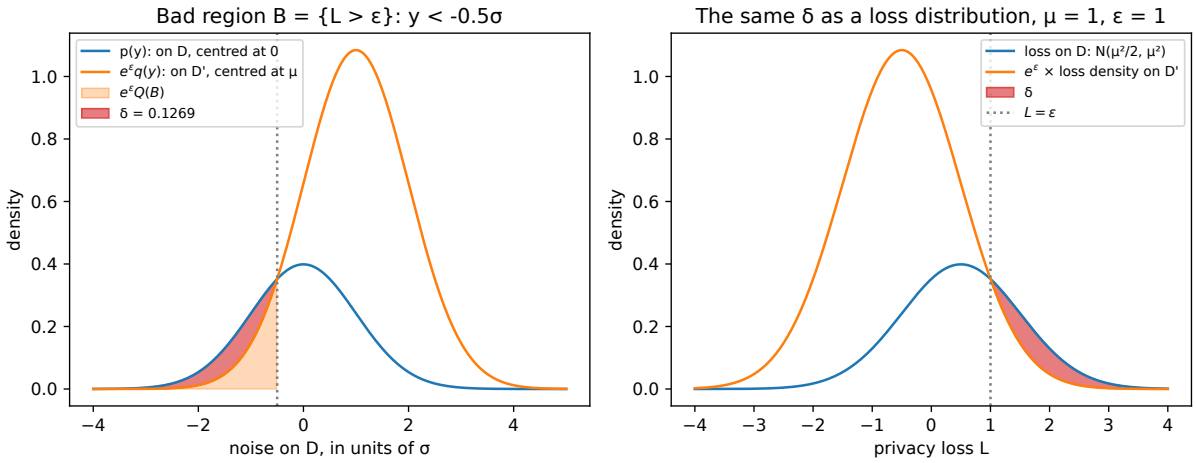


Figure A.6.: The exact δ of the Gaussian mechanism at $\mu = 1$, $\varepsilon = 1$. Left: the output density on D and e^ε times the density on D', with the output measured in units of σ. On the bad set, left of the dotted line, the red excess is $\delta = 0.1269$ and the orange area is the correction $e^\varepsilon \Pr[M(D') \in B] = 0.1816$. Right: the same excess in terms of the privacy loss, which is normal with mean $\mu^2/2$ and standard deviation μ on D.

Table A.7 compares the two calibrations at $\delta = 10^{-5}$, and Figure A.7 draws them over a wider range.

Table A.7.: The classic and the exact Gaussian noise at $\delta = 10^{-5}$.

ε	Classic σ/Δ_2	Exact σ/Δ_2	Classic / exact
0.1	48.4481	30.7496	1.576
0.25	19.3792	13.2855	1.459
0.5	9.6896	7.0318	1.378
1	4.8448	3.7306	1.299
2	2.4224	1.9938	1.215
4	1.2112	1.0812	1.120
8	0.6056	0.6002	1.009

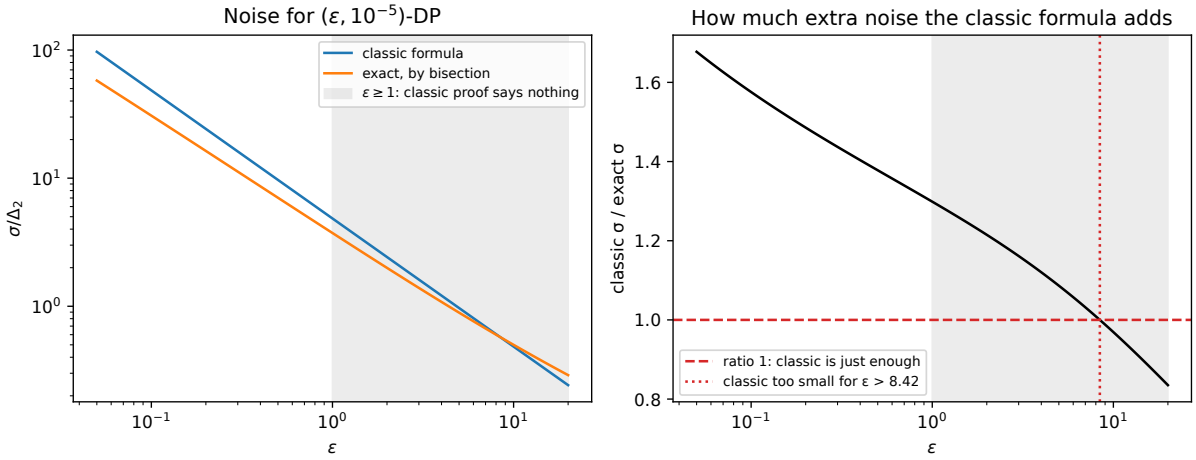


Figure A.7.: Left: the noise σ/Δ_2 needed for $(\varepsilon, 10^{-5})$-DP by the classic formula and by the exact calibration, on log axes. Right: their ratio. For ε from 0.05 to 1 the classic formula uses 1.30 to 1.68 times the necessary noise. Beyond $\varepsilon = 1$, in the shaded region, its proof says nothing, and beyond $\varepsilon = 8.42$ it adds too little noise.

For $\varepsilon < 1$ the classic formula is valid but loose, by a factor that grows as ε shrinks. Above 1 its proof no longer applies. The formula still happens to give enough noise at $\delta = 10^{-5}$ up to $\varepsilon = 8.42$, but beyond that point its true δ exceeds the target, so it is wrong there, not just unproved.

a whole curve of (ε, δ) pairs, so quote μ or the curve rather than one pair. Section 20.4 derives the same formula through a divergence between the two output distributions, and Section B.3 shows that many Gaussian releases combine into one. Section A.8 works through the formula by hand and in code, and shows how the noise changes when you move ε, δ or the number of releases.

A.8. How ε, δ, σ and the sensitivity trade off

A.8.1. The question

The last section gave a formula for the exact δ of the Gaussian mechanism and a way to solve it for σ. In practice you start from what you can give up, a privacy budget of ε and δ, or from the noise you can afford, and you want to know how the four quantities move together. They are the privacy parameter ε, the slack δ, the noise standard deviation σ and the ℓ_2 sensitivity Δ_2. How do you compute δ from normal probabilities, by hand and in code? And if you halve ε, make δ a thousand times smaller or release the answer ten times, how much more noise do you need?

A.8.2. The answer

Everything follows from the exact formula of Section A.7. Write $\Phi(x)$ for the probability that a standard normal variable is below x, and $\mu = \Delta_2/\sigma$ for the sensitivity measured in units of the noise. The mechanism is (ε, δ)-DP exactly when δ is at least

$$\delta(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big). \tag{A.21}$$

Six facts about it answer the questions.

1. **Only the ratio $\mu = \Delta_2/\sigma$ matters.** The formula sees Δ_2 and σ only through μ. Doubling the sensitivity and doubling σ therefore gives the same guarantee, and you can solve once for $\Delta_2 = 1$ and multiply the resulting σ by Δ_2.
2. **One σ gives a whole curve.** For a fixed μ, $\delta(\varepsilon)$ falls as ε grows. So a Gaussian mechanism is $(\varepsilon, \delta(\varepsilon))$-DP for every ε at once, not for one pair.
3. **The noise falls roughly like $1/\varepsilon$, then more slowly.** Write z_δ for the point a standard normal variable exceeds with probability δ, so that $\Phi(-z_\delta) = \delta$. The exact δ is never more than the probability of the bad set, a normal tail. So asking only that this tail be δ gives a simple **tail rule**, which always gives enough noise:

   $$\sigma_{\text{tail}} = \Delta_2\,\frac{z_\delta + \sqrt{z_\delta^2 + 2\varepsilon}}{2\varepsilon}, \tag{A.22}$$

   The rule is about $\Delta_2 z_\delta/\varepsilon$ when ε is small compared with $z_\delta^2/2$, and about $\Delta_2/\sqrt{2\varepsilon}$ when it is large. At $\delta = 10^{-5}$ it uses 1.39 times the exact noise for $\varepsilon = 0.1$. The excess shrinks as ε grows, to 1.02 times at $\varepsilon = 32$.
4. **The noise grows only slowly as δ shrinks.** The point z_δ grows like $\sqrt{2\ln(1/\delta)}$, and by fact 3 the noise is roughly proportional to z_δ. Near $\delta = 10^{-5}$, each factor of 10 in δ adds only about 13% to 15% more noise at ε of 0.5 or 1.

5. **Repeating costs $\sqrt{k}$.** Releasing k answers, each with sensitivity Δ_2 and its own independent noise σ, gives exactly the guarantee of one release with $\mu\sqrt{k}$. Then k releases need $\sqrt{k}$ times the noise of one for the same (ε, δ).
6. **Choose δ from the number of people n.** Take δ well below $1/n$, for example $1/n^2$. The reason, shown in step 7 below, is that a mechanism that publishes each record with probability δ is $(0, \delta)$-DP, and with $\delta = 1/n$ it publishes someone's record with probability about 0.63.

A.8.3. A worked example

By hand. Take $\Delta_2 = 1$, $\sigma = 4$ and $\varepsilon = 1$. Table A.8 follows the formula one number at a time, for a dataset D and a neighbour D'. First compute μ, then the two arguments of Φ, which Section A.7 called α and β. Then look up Φ of each in a normal table or compute it, and finally combine them. The second term, $e^{\varepsilon}\Phi(\beta)$, is the **correction** that turns the probability of the bad set into the exact δ.

Table A.8.: The exact δ of the Gaussian mechanism with $\Delta_2 = 1$, $\sigma = 4$ and $\varepsilon = 1$, one step at a time.

Step	Formula	Value
Sensitivity in noise units	$\mu = \Delta_2/\sigma$	0.2500
First argument	$\alpha = \mu/2 - \varepsilon/\mu = 0.125 - 4$	-3.8750
Second argument	$\beta = -\mu/2 - \varepsilon/\mu = -0.125 - 4$	-4.1250
Probability of the bad set on D	$\Phi(\alpha)$	5.3312×10^{-5}
Same set on D'	$\Phi(\beta)$	1.8537×10^{-5}
Correction	$e^{\varepsilon}\,\Phi(\beta) =$ $2.7183 \times 1.8537 \times 10^{-5}$	5.0388×10^{-5}
Exact δ	$\Phi(\alpha) - e^{\varepsilon}\Phi(\beta)$	2.9243×10^{-6}

So noise of standard deviation 4 on a count, whose sensitivity is 1, gives $(1, 2.92 \times 10^{-6})$-DP. Notice how much the correction removes. The first term, the probability of the bad set, is 18.2 times the final δ, so using it alone would overstate δ eighteen-fold.

By computer. Most languages have no Φ built in, but they all have the **complementary error function** erfc, which Python calls `math.erfc`. In terms of it,

$$\Phi(x) = \tfrac{1}{2}\operatorname{erfc}\left(-x/\sqrt{2}\right), \qquad 1 - \Phi(x) = \tfrac{1}{2}\operatorname{erfc}\left(x/\sqrt{2}\right). \tag{A.23}$$

Use these forms, and never compute a small tail as 1 minus something close to 1. A double-precision number carries about 16 significant digits. So when $\Phi(x)$ is within 10^{-16} of 1, the subtraction $1 - \Phi(x)$ keeps none of the digits that matter. Table A.9 shows this at $x = 8$ and $x = 9$. The textbook form $\Phi(x) = \frac{1}{2}(1 + \operatorname{erf}(x/\sqrt{2}))$, with the plain error function erf, has the same problem for negative x, because erf is then close to -1.

Table A.9.: The probability that a standard normal variable exceeds x, computed three ways in double precision. Only the last is right.

Way of computing the upper tail at x	$x = 8$	$x = 9$
$1 - \Phi(x)$, by subtraction	6.6613×10^{-16}	0
$\Phi(-x)$ with the erf form	6.1062×10^{-16}	0
$\frac{1}{2}\operatorname{erfc}(x/\sqrt{2})$	6.2210×10^{-16}	1.1286×10^{-19}

The same cancellation spoils δ itself when δ is tiny. Then α and β are far out in the lower tail, where erf is close to -1 and the erf form adds 1 to it. The script that comes with this appendix calibrates σ at $\varepsilon = 1$ for four targets. It puts each σ back into the formula, computing Φ once with erfc and once with erf, and Table A.10 shows the result. With erfc, every target comes back correct to five digits. With erf, the target 10^{-12} is already off in the fifth digit. The error then grows fast, so that 10^{-15} comes back 4% too small and 10^{-18} comes back 55 times too large.

Table A.10.: The exact δ at $\varepsilon = 1$, recomputed at the calibrated σ with Φ from erfc and from erf. The last column is how much larger the first term is than the difference.

Target δ	σ/Δ_2	δ with erfc	δ with erf	First term / δ
10^{-9}	5.4953	1.0000×10^{-9}	1.0000×10^{-9}	32.5
10^{-12}	6.5578	1.0000×10^{-12}	9.9997×10^{-13}	45.4
10^{-15}	7.4870	1.0000×10^{-15}	9.6226×10^{-16}	58.5
10^{-18}	8.3220	1.0000×10^{-18}	5.5511×10^{-17}	71.7

Even with erfc, the formula subtracts two nearly equal terms, and a subtraction loses digits too. The last column says how many. When the first term is r times the difference, the subtraction loses about $\log_{10} r$ digits. Here r is at most 72, and $\log_{10} 72 = 1.86$, so the subtraction loses at most 2 of the 16 digits. To check this independently, the script also integrates the excess $p_D - e^{\varepsilon} p_{D'}$ over the bad set directly. Measure the noise in units of σ as y, so that its privacy loss is $L(y) = \mu^2/2 - \mu y$. The integrand is then $\varphi(y)\,(1 - e^{\varepsilon - L(y)})$, which is never a difference of two tails. That integral agrees with the erfc formula to about one part in 10^{12} at all four targets.

Scaling. Now take a sum of values clipped to $[0, 2]$, so that $\Delta_2 = 2$, with $\sigma = 8$. The ratio is still $\mu = 0.25$, so δ is again 2.9243×10^{-6}, and the same holds for $\Delta_2 = 0.5$ with $\sigma = 2$.

The curve. The same $\sigma = 4$ gives a whole curve of guarantees. It is $(1, 2.92 \times 10^{-6})$-DP, and also $(2, 5.09 \times 10^{-17})$-DP and $(0.9263, 10^{-5})$-DP. Figure A.8 draws the curve for four noise levels. Each falls steeply as ε grows, and each crosses $\delta = 10^{-5}$ at its own ε.

A.8.4. Why it works

1. **Only μ matters.** In the formula, Δ_2 and σ appear only in $\mu/2$ and ε/μ. So any sensitivity and noise with the same ratio give the same curve. In particular, the smallest σ for a target is Δ_2 times the smallest σ for a sensitivity of 1. The reason behind it is that measuring the output in units of σ turns every Gaussian mechanism into one with noise 1 and shift μ.

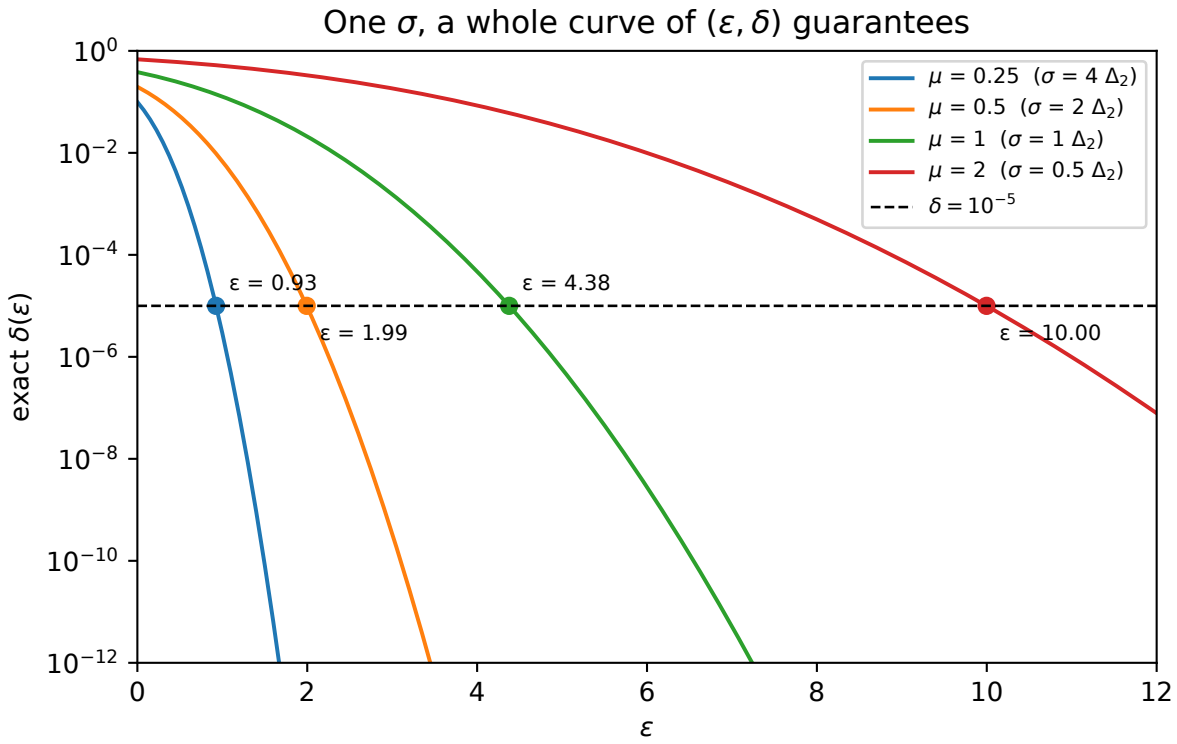


Figure A.8.: The exact $\delta(\varepsilon)$ of the Gaussian mechanism against ε, on a log scale, for four noise levels $\mu = \Delta_2/\sigma$. A single σ is $(\varepsilon, \delta(\varepsilon))$-DP for every ε on its curve. The dots mark the ε each one gives at $\delta = 10^{-5}$: 0.93 for $\sigma = 4\Delta_2$, 1.99 for $2\Delta_2$, 4.38 for Δ_2 and 10.00 for $\Delta_2/2$.

2. **The curve falls.** Write $\alpha = \mu/2 - \varepsilon/\mu$ and $\beta = -\mu/2 - \varepsilon/\mu$ as before, and φ for the standard normal density. Step 5 of Section A.7 showed that $e^{\varepsilon}\varphi(\beta) = \varphi(\alpha)$. Both arguments fall by $1/\mu$ for each unit of ε, so differentiating in ε gives

$$\frac{\mathrm{d}\delta}{\mathrm{d}\varepsilon} = -\frac{\varphi(\alpha)}{\mu} - e^{\varepsilon}\Phi(\beta) + \frac{e^{\varepsilon}\varphi(\beta)}{\mu} = -e^{\varepsilon}\,\Phi(\beta) < 0. \tag{A.24}$$

   The first and last terms cancel, and what is left is negative. So a larger ε always comes with a smaller δ, and one mechanism traces a curve. The same step 5 showed that δ grows with μ. More noise means a smaller μ, so it lowers the whole curve.
3. **A safe rule from the bad set alone.** Step 4 of Section A.1 showed that the exact δ is never more than the probability of the bad set, which step 4 of Section A.6 computed:

$$\Pr[L > \varepsilon] = \Phi(\alpha). \tag{A.25}$$

   So any σ that makes $\Phi(\alpha) = \delta$ is enough. Since $\Phi(-z_\delta) = \delta$, that means $\alpha = -z_\delta$, which is $\mu/2 - \varepsilon/\mu = -z_\delta$. Multiplying by 2μ gives the quadratic $\mu^2 + 2z_\delta\mu - 2\varepsilon = 0$, whose positive root is $\mu = \sqrt{z_\delta^2 + 2\varepsilon} - z_\delta$. Then $\sigma = \Delta_2/\mu$, and multiplying the top and bottom by $\sqrt{z_\delta^2 + 2\varepsilon} + z_\delta$ gives Equation A.22. Solving the same equation for ε instead gives the matching rule for the ε of a given σ:

$$\varepsilon_{\text{tail}} = z_\delta\mu + \frac{\mu^2}{2}. \tag{A.26}$$

4. **Its two regimes.** When 2ε is small compared with z_δ^2, the square root is close to z_δ, so $\sigma_{\text{tail}} \approx \Delta_2 z_\delta/\varepsilon$, which is proportional to $1/\varepsilon$. When 2ε is large compared with z_δ^2, the square root is close to $\sqrt{2\varepsilon}$ and dominates z_δ, so $\sigma_{\text{tail}} \approx \Delta_2/\sqrt{2\varepsilon}$. The bend between the two is near $\varepsilon = z_\delta^2/2$. At $\delta = 10^{-5}$, where $z_\delta = 4.2649$, that is $\varepsilon = 9.09$. The classic formula has the same $1/\varepsilon$ shape with $\sqrt{2\ln(1.25/\delta)}$ in place of z_δ, but it never bends, which is why it fails at large ε (Section A.7).
5. **Why δ costs little.** By the Mills-ratio bound of Section A.6, $\Phi(-z_\delta)$ is close to $e^{-z_\delta^2/2}/(z_\delta\sqrt{2\pi})$. So δ shrinks like $e^{-z_\delta^2/2}$, and z_δ grows only like $\sqrt{2\ln(1/\delta)}$. Dividing δ by 10 adds about $2\ln 10 = 4.61$ to z_δ^2. At $\delta = 10^{-5}$ this makes z_δ grow from 4.2649 to 4.7534, a factor of 1.115, and the tail rule's noise of about $\Delta_2 z_\delta/\varepsilon$ grows by the same factor. The exact noise grows a little faster, as the checks below show.
6. **Repeating adds the losses.** Suppose the k questions are fixed in advance, each answer has sensitivity Δ_2, and each gets independent noise σ. Together the k answers are one vector, with ℓ_2 sensitivity $\sqrt{k\Delta_2^2} = \sqrt{k}\,\Delta_2$. Step 2 of Section A.6 showed that a vector release behaves like one number shifted by the ℓ_2 length of the change. So the k releases have the exact curve of one release with $\mu\sqrt{k}$. To keep $\mu\sqrt{k}$ at the value one release needs, σ must grow by $\sqrt{k}$. When a question may depend on earlier answers, the k answers are no longer one fixed vector query, so this argument does not apply as it stands. Section B.3 proves the same result for that case.
7. **Why δ must be below $1/n$.** Take the mechanism that publishes each record in full with probability δ, independently. One record changes the output only when its own coin publishes it, so the two probabilities of any event differ by at most δ, and the mechanism is $(0, \delta)$-DP. Each of n records stays hidden with probability $1 - \delta$, so someone is exposed with probability $1 - (1 - \delta)^n$. With $\delta = 1/n$ this is about 0.63. The reason is that $n\ln(1 - 1/n)$ is close to -1 for large n, so

$$(1 - 1/n)^n = e^{n\ln(1-1/n)} \approx e^{-1}, \tag{A.27}$$

and $1 - e^{-1} = 0.63$. A guarantee that allows this mechanism promises little, so $n\delta$ must be far below 1.

A.8.5. Checking it

Table A.11 is the reference table of this section. It gives the exact σ/Δ_2 for five values of ε and four of δ, each found by bisection and checked by putting it back into the formula. Read along a row to see the cost of a smaller δ, and up a column to see the cost of a smaller ε.

Table A.11.: The smallest σ/Δ_2 for (ε, δ)-DP, from the exact calibration. Multiply by Δ_2 to get σ.

ε	$\delta = 10^{-3}$	$\delta = 10^{-5}$	$\delta = 10^{-7}$	$\delta = 10^{-9}$
0.1	17.4044	30.7496	41.3295	50.2098
0.5	4.6101	7.0318	8.9957	10.6739
1	2.5747	3.7306	4.6787	5.4953
2	1.4452	1.9938	2.4491	2.8445
4	0.8231	1.0812	1.2978	1.4878

Moving along a row, to a smaller δ, changes the noise much less than moving up a column, to a smaller ε. At $\varepsilon = 0.5$, for example, a δ ten thousand times smaller costs 1.518 times the noise. Halving ε instead, to 0.25, costs 1.889 times the noise (Table A.7 has the value at 0.25). Figure A.9 shows the whole plane. On the left each line is one noise level, and the lines are nearly vertical because σ depends mostly on ε. On the right, the product $\varepsilon\sigma/\Delta_2$ would be a flat line if σ were exactly proportional to $1/\varepsilon$.

Noise against ε. Table A.12 compares the tail rule with the exact calibration at $\delta = 10^{-5}$. The rule is always enough, as step 3 promises. Its excess over the exact noise is 39% at $\varepsilon = 0.1$, and it shrinks steadily as ε grows. The reason is the correction term that the rule ignores. At $\varepsilon = 0.1$ and the exact σ, the probability of the bad set is 111 times the final δ, so the correction removes almost all of it. At $\varepsilon = 8$ it is only 3.6 times the final δ, and the correction matters much less. In the exact calibration, the product $\varepsilon\sigma/\Delta_2$ grows by only a factor of 1.213 from $\varepsilon = 0.1$ to $\varepsilon = 1$. So "σ is proportional to $1/\varepsilon$" is a fair first guess below $\varepsilon = 1$. It fails at large ε, where the product keeps growing, up to 6.5579 at $\varepsilon = 32$.

Table A.12.: The noise for $(\varepsilon, 10^{-5})$-DP: the exact calibration, the tail rule of Equation A.22 and its two limits, with $z_\delta = 4.2649$.

ε	Exact σ/Δ_2	$\varepsilon\sigma/\Delta_2$	Tail rule	Tail / exact	z_δ/ε	$1/\sqrt{2\varepsilon}$
0.1	30.7496	3.0750	42.7658	1.391	42.6489	2.2361
0.5	7.0318	3.5159	8.6454	1.229	8.5298	1.0000
1	3.7306	3.7306	4.3791	1.174	4.2649	0.7071
2	1.9938	3.9876	2.2439	1.125	2.1324	0.5000
4	1.0812	4.3246	1.1728	1.085	1.0662	0.3536
8	0.6002	4.8018	0.6320	1.053	0.5331	0.2500

ε	Exact σ/Δ_2	$\varepsilon\sigma/\Delta_2$	Tail rule	Tail / exact	z_δ/ε	$1/\sqrt{2\varepsilon}$
32	0.2049	6.5579	0.2083	1.016	0.1333	0.1250

The last two columns show the two regimes of step 4. At $\varepsilon = 0.1$ the rule is almost exactly z_δ/ε. At $\varepsilon = 32$ the rule has moved well away from z_δ/ε and towards $1/\sqrt{2\varepsilon}$, but it is still above that limit, because z_δ is not yet small next to $\sqrt{2\varepsilon} = 8$.

Noise against δ. Starting from $\delta = 10^{-5}$, making δ ten times smaller raises the exact noise by a factor of 1.146 at $\varepsilon = 0.5$. There σ/Δ_2 goes from 7.0318 to 8.0576, while at $\varepsilon = 1$ the factor is 1.132. Both are a little above the 1.115 that step 5 predicted from z_δ alone, so the estimate from z_δ alone is slightly low.

The privacy a noise level buys. Turning the question around, Table A.13 gives the ε that a chosen σ buys at $\delta = 10^{-5}$, exactly and by the tail rule $\varepsilon_{\text{tail}} = z_\delta\mu + \mu^2/2$. For large σ the $z_\delta\mu$ term dominates, so ε falls like $1/\sigma$, the inverse of the noise rule. Again the rule is safe, because it never reports a smaller ε than the exact one.

Table A.13.: The ε that a given noise level buys at $\delta = 10^{-5}$.

σ/Δ_2	0.5	1	2	5	10	20
Exact ε	9.9973	4.3772	1.9931	0.7255	0.3407	0.1600
Tail rule $z_\delta\mu + \mu^2/2$	10.5298	4.7649	2.2574	0.8730	0.4315	0.2145

Repeating. Suppose ten releases must together be $(1, 10^{-5})$-DP. Then by step 6 each needs $\sqrt{10}$ times the noise of one release, so $\sigma = 11.7973$. As a check, putting $\mu = \sqrt{10}/11.7973$ back into the formula gives $\delta = 1.000 \times 10^{-5}$. The tempting alternative is to split the budget instead. Basic composition, which Section B.1 derives, says that the εs and the δs of separate releases add up, so ten $(0.1, 10^{-6})$-DP releases are together $(1, 10^{-5})$-DP. That needs $\sigma = 36.3047$, which is 3.08 times more noise. With 100 releases the gap grows to 9.70 times. Splitting gives each release only ε/k, and the noise grows roughly like $1/\varepsilon$, so it grows like k rather than $\sqrt{k}$.

Choosing δ from n. Table A.14 gives the noise at $\varepsilon = 1$ for four choices of δ, and how often the publish-a-record mechanism of step 7 would expose someone. A fixed $\delta = 10^{-5}$ is fine for a thousand people, with exposure 0.00995. For a million people it is ten times $1/n$, and the exposure is 0.99995. A rule sometimes used to get "a little below $1/n$", $\delta = 1/n^{1.1}$, is not far enough below, since its exposure is still 0.328 for ten thousand people. By contrast, $\delta = 1/n^2$ makes $n\delta = 1/n$, which keeps the exposure near $1/n$, and its noise is modest. For ten thousand people it needs only 1.367 times the noise of $\delta = 10^{-5}$.

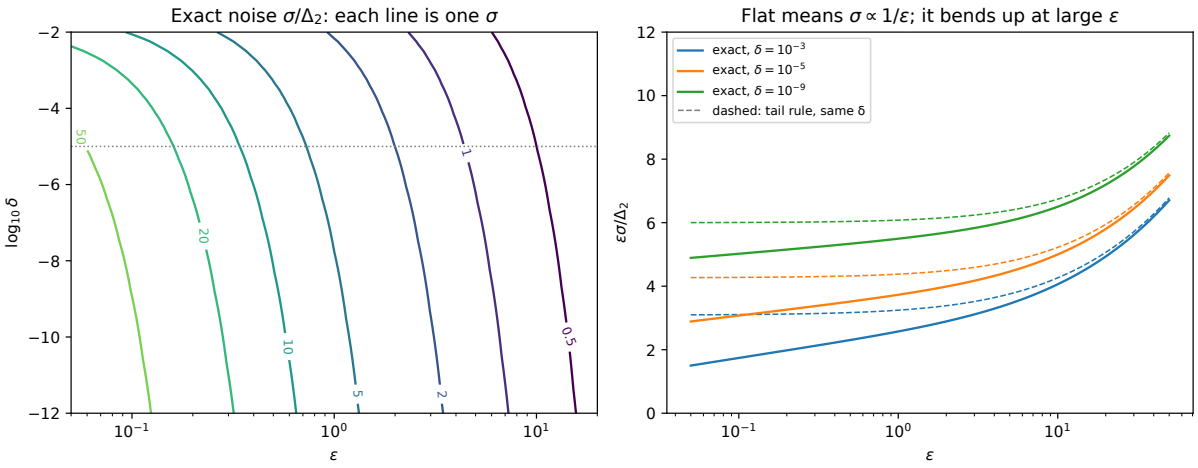


Figure A.9.: Left: contour lines of the exact σ/Δ_2 over ε (log scale) and $\log_{10}\delta$, with $\delta = 10^{-5}$ dotted. The lines are nearly vertical, so ε decides the noise far more than δ does. Right: the product $\varepsilon\sigma/\Delta_2$ against ε for three values of δ, exact (solid) and by the tail rule (dashed). The tail rule is nearly flat for small ε, where it is close to z_δ/ε, and the exact product sags below it there, because the correction term matters most at small ε. Both rise at large ε, where the noise falls more slowly than $1/\varepsilon$.

Table A.14.: The exact σ/Δ_2 at $\varepsilon = 1$ for four ways of choosing δ, and the probability $1-(1-\delta)^n$ that publishing each record with probability δ exposes someone. At $\delta = 1/n$ that probability is 0.632 for every n shown.

n	$\delta = 10^{-5}$	$\delta = 1/n$	$\delta = 1/n^{1.1}$	$\delta = 1/n^2$	Exposure at 10^{-5}	Exposure at $1/n^{1.1}$
1,000	3.7306	2.5747	2.7660	4.2247	0.00995	0.394
10,000	3.7306	3.1857	3.4106	5.1003	0.0952	0.328
100,000	3.7306	3.7306	3.9833	5.8678	0.632	0.271
1,000,000	3.7306	4.2247	4.5013	6.5578	0.99995	0.222

A.8.6. Recipe and pitfalls

Decide the quantities in this order, because the later steps use the earlier choices.

1. **Fix the neighbouring relation and compute** Δ_2, as in Section A.2. Clip first if the data are unbounded.
2. **Pick** δ **from the number of people** n, well below $1/n$. The choice $\delta = 1/n^2$ is cheap, as Table A.14 shows, and Section 46.5 discusses other rules.
3. **Pick** ε**.** It decides the noise far more than δ does, so spend your judgement here.
4. **Count the releases** k that must share the budget. If they are Gaussian releases of the same sensitivity, solve for one and multiply σ by $\sqrt{k}$.
5. **Solve for** σ **by bisection** on the exact formula with $\Delta_2 = 1$, computing Φ with erfc, and multiply by Δ_2. Table A.11 gives a first estimate, and Equation A.22 gives a safe closed form if you cannot run bisection.

6. **Check by plugging back.** Put $\mu = \Delta_2/\sigma$ into the formula and confirm that $\delta(\varepsilon)$ is at most your target.
7. **Report μ, or the curve,** not only one pair, so that a reader can find δ at any ε.

The common mistakes follow from the facts above. Quoting ε without δ says nothing, because one Gaussian mechanism has every ε on its curve. A δ of $1/n$ or more allows a mechanism that publishes a whole record. Computing $1-\Phi(x)$ by subtraction, or Φ with erf, silently loses accuracy from about $\delta = 10^{-12}$ and gives nonsense by 10^{-18}. The classic formula is unproved above $\varepsilon = 1$ and wrong above 8.42 at $\delta = 10^{-5}$. And forgetting composition is the most expensive mistake of all, because k releases at the σ of one are only (ε, δ)-DP for a much larger ε. Splitting the budget evenly fixes that, but at about k times the noise of one release instead of $\sqrt{k}$ times. Section B.3 and Section B.5 compare the ways to account for repeated releases.

A.9. All the recipes in one table

Table A.15 collects the calibrations of this appendix. Every row assumes you have fixed the neighbouring relation first, since the sensitivity depends on it.

Table A.15.: The calibration recipes of this appendix.

Mechanism	Sensitivity	Noise or rule	Guarantee
Laplace	Δ_1, the ℓ_1 norm	$\mathrm{Lap}(b)$ per coordinate, $b = \Delta_1/\varepsilon$	ε-DP; 95% error $b \ln 20$ per coordinate
Randomized response	one bit, under replace-one	truth with probability $e^\varepsilon/(1+e^\varepsilon)$	ε-DP; error about $1/(\varepsilon\sqrt{n})$
Exponential mechanism	Δu of the score	choose r with probability $\propto e^{\varepsilon u(D,r)/(2\Delta u)}$	ε-DP; gap $\le \frac{2\Delta u}{\varepsilon}(\ln\lvert\mathcal{R}\rvert + t)$ with probability $1 - e^{-t}$
Gaussian, classic	Δ_2, the ℓ_2 norm	$\sigma = \Delta_2\sqrt{2\ln(1.25/\delta)}/\varepsilon$	(ε, δ)-DP for $\varepsilon < 1$; loose
Gaussian, exact	Δ_2, the ℓ_2 norm	$\sigma = \Delta_2/\mu$ with $\delta(\varepsilon) = \delta$ by bisection	(ε, δ)-DP for every ε; tight
Gaussian, k releases	Δ_2 of each release	$\sqrt{k}$ times the exact σ of one, with δ well below $1/n$	(ε, δ)-DP for all k together

A.10. Code

The script computes every number on this page and draws every figure. It checks the three-output example against all events, finds every sensitivity by brute force over small datasets, and simulates the Laplace error and randomized response. For the Gaussian mechanism it compares the closed form with numerical integration and with Monte Carlo, checks the derivative of step 5, and solves the exact calibration by bisection. For the trade-offs it computes Φ three ways and checks tiny values of δ against an integral over the bad set. It also tabulates the noise as each quantity moves. From the repository root:

```
uv run python code/appendix-derivations/apx_core_calibration.py
```

Table A.16 summarizes its output with the default random seed, 0.

Table A.16.: The script's output with the default seed.

Check	Result
Three outputs, every event	Largest ratio 2.5000; largest excess at $e^{\varepsilon} = 2$ is 0.1000; bad set 0.5000
Sensitivities, every dataset of 1 to 3 records	Every row of Table A.3; mean under add/remove up to 1.5000
Laplace loss, count, $b = 2$	In exactly $[-0.5000, 0.5000]$; event ratio 1.6487
Laplace loss, three counts, $b = 6$	Largest over 100,000 outputs 0.5000
Laplace error, $b = 2$	Mean 2.0026 (exact 2); 95th percentile 5.9938 (formula 5.9915)
Randomized response, $\varepsilon = 0.5$, $n = 1000$	Mean 0.2996, standard deviation 0.0629 (formula 0.0626)
Exponential mechanism, vote	Largest privacy loss 0.3353 at $\varepsilon = 1$
Classic chain, $\varepsilon = 0.5$, $\delta = 10^{-5}$	8.55×10^{-7}, 8.54×10^{-7}, 8.21×10^{-7}, exact 1.61×10^{-8}
Exact δ, $\mu = 1$, $\varepsilon = 1$	0.1269 (Simpson 0.1269, Monte Carlo 0.1270)
Exact calibration, $\varepsilon = 0.5$, $\delta = 10^{-5}$	$\sigma = 7.0318$ against classic 9.6896
Classic formula fails at $\delta = 10^{-5}$	For $\varepsilon > 8.42$
Exact δ by hand, $\Delta_2 = 1$, $\sigma = 4$, $\varepsilon = 1$	2.9243×10^{-6} (bad-set integral the same)
Φ from erf at $\delta = 10^{-18}$	5.5511×10^{-17}, against 1.0000×10^{-18} from erfc
Tail rule against exact, $\delta = 10^{-5}$	1.391 times at $\varepsilon = 0.1$, 1.016 times at $\varepsilon = 32$
Ten releases at $(1, 10^{-5})$	$\sigma = 11.7973$ each, against 36.3047 when the budget is split

tests/test_appendix_core_calibration.py checks that the output still matches. Three things stand out. Every formula agrees with brute force or simulation to within sampling error. The Laplace and exponential mechanisms meet their bounds exactly on some outputs, so their noise cannot be reduced. And the classic Gaussian formula is safe but wasteful for small ε, and unsafe for very large ε, which is why the exact calibration should be the default.

A.11. References

- Dwork, McSherry, Nissim & Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (TCC 2006): sensitivity and the Laplace mechanism.
- Warner, "Randomized Response: A Survey Technique for Eliminating Evasive Answer Bias" (1965): randomized response.
- McSherry & Talwar, "Mechanism Design via Differential Privacy" (FOCS 2007): the exponential mechanism.

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014): §3.3 for the Laplace mechanism, §3.4 for the exponential mechanism and Appendix A for the classic Gaussian proof.
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising" (ICML 2018): the exact $\delta(\varepsilon)$ of the Gaussian mechanism and its calibration by bisection.
- Near & Abuah, *Programming Differential Privacy*: the same mechanisms with code, and the practical pitfalls of sensitivity.

B. Composing private releases

A private system almost never releases one number. A dashboard publishes a hundred counts, an analyst asks one query after another, and a training run takes thousands of noisy steps. Each release on its own may be private, but an attacker sees all of them together, so this appendix answers the practical questions that follow. What is the total privacy cost of a number k of releases? Why is that answer correct? How much noise must each release get so that the whole set stays within a budget? And which of the several ways to add up the cost should you use?

Everything here rests on one quantity, so here it is in three sentences. A mechanism M is **differentially private** if changing one person's record, which turns a dataset D into a **neighbouring** dataset D', barely changes the distribution of its output. The **privacy loss** of an output y is the logarithm of how much more likely y is under D than under D'. For a budget ε, pure ε-differential privacy (DP) says this loss is between $-\varepsilon$ and ε for every output. Approximate (ε, δ)-DP asks instead that every set S of outputs, for every pair of neighbours, satisfies

$$\Pr_{D}[S] \leq e^{\varepsilon} \Pr_{D'}[S] + \delta, \tag{B.1}$$

where $\Pr_D[S]$ is the chance that the output lands in S when the data are D. The small slack δ lets rare outputs break the factor e^{ε}. Section A.1 in the appendices states both definitions in full, with examples.

The one idea of this page is that **the privacy loss of several releases is the sum of their separate losses**. Basic composition bounds that sum by the largest value it can take. Advanced composition uses the fact that the terms partly cancel, like the steps of a random walk. For Gaussian noise the sum is again exactly Gaussian, and Rényi DP tracks a set of averages of the loss that add just as simply. A method that adds up the cost of many releases this way is called an **accountant**. Table B.1 lists the derivations and the chapter sections that use them.

Table B.1.: The derivations of this appendix and the chapter sections that state or use them.

Derivation	Where the chapters state or use it
Basic composition: budgets add (Section B.1)	Section 19.3; every accountant starts here
Advanced composition: $\sqrt{k}$ instead of k (Section B.2)	Section 21.1, Section 21.2
k Gaussian releases are one Gaussian release (Section B.3)	Section 19.3, Section 20.4, Section 32.4
Rényi DP of the Gaussian, and why it adds (Section B.4)	Section 29.2, Section 29.3, Section 29.6
Comparing the four ways to add up the cost (Section B.5)	Section 21.4, Section 37.1

Notation. The k mechanisms are $M_1, \dots, M_k$, run on the same dataset, and their outputs are $y_1, \dots, y_k$. For one mechanism, $p(y)$ is the probability (or density) of the output y when the data are D, and $q(y)$ is the same when the data are D'. The privacy loss is

$$L(y) = \ln \frac{p(y)}{q(y)}. \tag{B.2}$$

For Gaussian noise, σ is the standard deviation of the noise and Δ is the ℓ_2 sensitivity. That is the largest ordinary (Euclidean) distance by which one record can move the true answer, which may be a vector of numbers. Their ratio, the shift in units of the noise, is $\mu = \Delta/\sigma$. The accounting chapters take $\Delta = 1$ and call σ the noise multiplier, so their σ is this page's σ with $\Delta = 1$. The standard normal cumulative distribution function is Φ, and logarithms are natural.

Neighbours can be defined in two ways: by adding or removing one record, or by replacing one record with another. The choice changes the sensitivity of a query, but not any argument on this page, as long as every release uses the same relation.

B.1. Basic composition: the budgets add

B.1.1. The question

You release k answers computed from the same dataset, each with its own fresh noise, and each private on its own. What guarantee holds for all k answers together, including when you choose each query after reading the earlier answers?

B.1.2. The answer

If release i is $(\varepsilon_i, \delta_i)$-DP, then the k releases together are

$$\Big(\sum_{i=1}^{k} \varepsilon_i,\ \sum_{i=1}^{k} \delta_i\Big)\text{-DP}. \tag{B.3}$$

Here ε_i is the budget of release i, and δ_i its slack. The same holds when release i is chosen using the earlier outputs, as long as its budget is fixed in advance. The chapter states this as Equation 19.11 in Section 19.3.

B.1.3. A worked example

A dashboard publishes 10 counts, such as the number of patients in each of 10 clinics. One person changes each count by at most 1, so each count has sensitivity 1. Laplace noise of scale b makes a release with sensitivity Δ_1 private at $\varepsilon = \Delta_1/b$ (Section A.3). So each count gets Laplace noise of scale $b = 1/0.1 = 10$, which makes it 0.1-DP, and by Equation B.3 the 10 counts together are $10 \times 0.1 = 1$-DP.

It helps to see why one noisy count is 0.1-DP. Say the true count is 0 on D and 1 on D'. The Laplace density with scale b is $\frac{1}{2b}e^{-|z|/b}$, and the two densities centred at 0 and 1 have the same

constant $\frac{1}{2b}$ in front. So the constants cancel in the ratio, and the privacy loss of an output y is the difference of the two exponents,

$$L(y) = \frac{|y-1| - |y|}{b}. \tag{B.4}$$

For $y \le 0$ this is $((1-y)-(-y))/b = 1/b = 0.1$. For $y \ge 1$ it is -0.1, and in between it moves in a straight line from one to the other (Figure B.1, left). So the loss never leaves $[-0.1, 0.1]$.

The bound of 1 for the 10 counts is reached. Each count's loss is $+0.1$ exactly when its output is at most 0, its true value on D. Under D the noise is symmetric around 0, so this happens with probability $\frac{1}{2}$, and all 10 counts do it at once with probability $(\frac{1}{2})^{10} = 0.000977$. A simulation of a million dashboards finds the largest total loss to be exactly 1.0000, reached in a fraction 0.000997 of them. So for pure DP, basic composition is exact in the worst case. But that case is rare, and the average total loss on D is only 0.0484, as the checking section below shows. The gap between the worst and the typical loss is what the next section exploits.

B.1.4. Why it works

Pure DP, outputs with independent noise. The plan is to write the probability of all k outputs as a product, so that the ratio of two such probabilities becomes a product of ratios, and the log of the ratio becomes a sum.

1. **The joint probability is a product.** Each release uses its own fresh noise, so with the dataset held fixed the outputs are independent. The probability of seeing $y_1, \dots, y_k$ together is the product of the separate probabilities, $p(y_1, \dots, y_k) = p_1(y_1) \cdots p_k(y_k)$, and the same holds for q.
2. **The ratio is a product of ratios.** Dividing one product by the other, term by term,

$$\frac{p(y_1, \dots, y_k)}{q(y_1, \dots, y_k)} = \frac{p_1(y_1)}{q_1(y_1)} \cdots \frac{p_k(y_k)}{q_k(y_k)}. \tag{B.5}$$

3. **Each factor is bounded.** Release i is ε_i-DP, so its ratio lies between $e^{-\varepsilon_i}$ and e^{ε_i}.
4. **Multiply the bounds and take logs.** The product lies between $e^{-\sum \varepsilon_i}$ and $e^{\sum \varepsilon_i}$. The logarithm of a product is the sum of the logarithms, so the total loss is $L = L_1(y_1) + \cdots + L_k(y_k)$, and $|L| \le \sum_i \varepsilon_i$ at every output. That is pure $\sum_i \varepsilon_i$-DP.

Figure B.1 draws this for two counts. Each count's loss is flat at $+\varepsilon$ on one side, flat at $-\varepsilon$ on the other, with a ramp in between, and the loss of the pair is the sum of the two. It reaches 2ε only in the corner where both outputs favour D, that is, where both have the largest loss.

Adaptive releases. Now suppose you choose the second query after reading the first answer. The outputs are no longer independent, so step 1 needs a different reason. The **chain rule** of probability gives it. The chance of seeing y_1 and then y_2 is the chance of y_1, times the chance of y_2 given that the first output was y_1:

$$p(y_1, y_2) = p_1(y_1)\, p_2(y_2 \mid y_1). \tag{B.6}$$

The ratio is again a product of two ratios, and the second one, $p_2(y_2 \mid y_1)/q_2(y_2 \mid y_1)$, is the ratio of the mechanism you chose after seeing y_1. If every mechanism you might choose is ε_2-DP, that

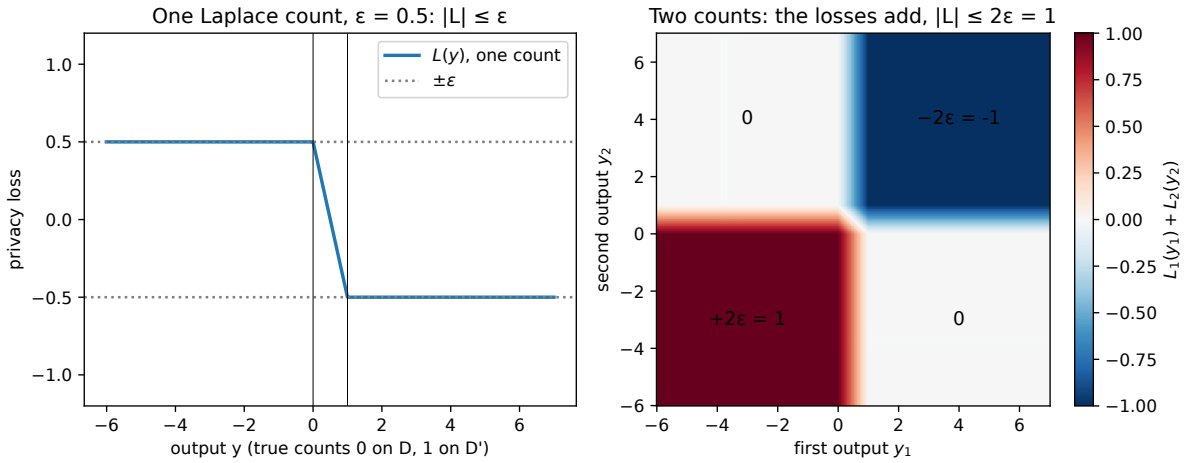


Figure B.1.: Left: the privacy loss of one count released with Laplace noise at $\varepsilon = 0.5$, as a function of the output, when the true count is 0 on D and 1 on D'. It is flat at $\pm\varepsilon$ outside $[0, 1]$. Right: the total loss of two such counts over the plane of the two outputs. It is the sum of two such steps, so it reaches 2ε only in the lower-left corner, where both outputs favour D. The figure uses $\varepsilon = 0.5$ rather than 0.1 so that the shading is visible.

ratio lies between $e^{-\varepsilon_2}$ and e^{ε_2}, whatever y_1 was. So steps 2 to 4 go through unchanged. With k releases the chain rule has k factors, one per release.

A tiny case makes this concrete. **Randomized response** at level ε reports a person's secret bit truthfully with probability $e^{\varepsilon}/(1 + e^{\varepsilon})$ and flips it otherwise. Ask once at level $\ln 3$, which tells the truth with probability $\frac{3}{4}$. If the answer is 1, ask again at $\ln 3$. If it is 0, ask again at $\ln 2$, which tells the truth with probability $\frac{2}{3}$. Take D to be the dataset where the person's bit is 1 and D' the one where it is 0. Table B.2 lists the four outcomes, as computed by this appendix's script (see the Code section).

Table B.2.: Two randomized-response answers about one bit, where the level of the second depends on the first answer. Each probability is a product of a first-step and a second-step factor, so each loss is a sum of two step losses.

Answers (y_1, y_2)	Probability on D	Probability on D'	Privacy loss L
$(1, 1)$	$\frac{3}{4} \cdot \frac{3}{4} = 0.5625$	$\frac{1}{4} \cdot \frac{1}{4} = 0.0625$	$\ln 3 + \ln 3 = 2.1972$
$(1, 0)$	$\frac{3}{4} \cdot \frac{1}{4} = 0.1875$	$\frac{1}{4} \cdot \frac{3}{4} = 0.1875$	$\ln 3 - \ln 3 = 0$
$(0, 1)$	$\frac{1}{4} \cdot \frac{2}{3} = 0.1667$	$\frac{3}{4} \cdot \frac{1}{3} = 0.2500$	$-\ln 3 + \ln 2 = -0.4055$
$(0, 0)$	$\frac{1}{4} \cdot \frac{1}{3} = 0.0833$	$\frac{3}{4} \cdot \frac{2}{3} = 0.5000$	$-\ln 3 - \ln 2 = -1.7918$

Each row's loss is the first answer's loss plus the second answer's loss, where the second loss belongs to whichever mechanism was chosen. The largest loss is $2.1972 = 2 \ln 3$. That equals the basic bound $\ln 3 + \ln 3$, in which the second budget is $\ln 3$, the larger of the two levels the second step could use.

Approximate DP. An (ε, δ)-DP mechanism may have outputs whose loss exceeds ε, or is even infinite, as long as they are rare. The Gaussian mechanism is an example: its loss is a straight line in the output, so it has no largest value. So the product argument fails, and a different one is needed. It rests on a fact that this page uses again later, the **bad-event fact**: for any threshold τ and any set S of outputs,

$$\Pr_{D}[S] \le e^{\tau} \Pr_{D'}[S] + \Pr_{D}[L > \tau]. \tag{B.7}$$

To see it, split S into the outputs whose loss is at most τ and the rest. On the first part $\ln(p/q) \le \tau$, so $p \le e^{\tau} q$ output by output. Adding up over that part gives at most e^{τ} times its probability under D', which is at most $e^{\tau} \Pr_{D'}[S]$. The second part lies inside the event $L > \tau$, so its probability is at most $\Pr_{D}[L > \tau]$. So if the loss exceeds τ with probability at most δ when the data are D, then $\Pr_{D}[S] \le e^{\tau} \Pr_{D'}[S] + \delta$ for every S. DP also asks for the same bound with D and D' swapped, and the same argument gives it.

Now suppose each release satisfies a **tail condition**: whatever the earlier outputs were, its loss exceeds ε_i with probability at most δ_i. Then the composition is $(\sum \varepsilon_i, \sum \delta_i)$-DP, in three steps.

1. **A large total needs a large term.** If every L_i is at most ε_i, the sum is at most $\sum_i \varepsilon_i$. So the total can exceed $\sum_i \varepsilon_i$ only if at least one term exceeds its own ε_i.
2. **Union bound.** The chance that at least one of several events happens is at most the sum of their chances. So

$$\Pr\left[L > \sum_i \varepsilon_i\right] \le \sum_i \Pr[L_i > \varepsilon_i] \le \sum_i \delta_i. \tag{B.8}$$

   In the adaptive case, the chance that term i exceeds ε_i is an average, over the earlier outputs, of a conditional chance that is at most δ_i. So it is at most δ_i too.
3. **Bad-event fact.** Equation B.7 with $\tau = \sum_i \varepsilon_i$ gives the result.

The tail condition is a **sufficient condition**: it is enough for (ε, δ)-DP, but it asks for more. The classic Gaussian calibration proves exactly this condition (Section 20.3), so the argument covers it. But some (ε, δ)-DP mechanisms do not satisfy it. For example, take a mechanism with two outputs. Output A has probability $\frac{1}{2}$ on D and $\frac{1}{2}e^{-1.1} = 0.1664$ on D', and output B takes the rest, $\frac{1}{2}$ and 0.8336. Output A has loss 1.1, above 1, and it happens half the time, so the tail condition at $\varepsilon = 1$ fails. Yet the only set that breaks the factor e is $\{A\}$, and it breaks it by just $\frac{1}{2} - e \times 0.1664 = 0.0476$. So the mechanism is $(1, 0.0476)$-DP. Basic composition still holds for releases that are only known to be $(\varepsilon_i, \delta_i)$-DP. Section 19.3 proves it with a two-step argument on events, and Section J.1 gives a second route through four-point randomized response.

B.1.5. Checking it

A simulation of a million runs of the 10-count dashboard backs up the worked example. The largest total loss is 1.0000, reached in a fraction 0.000997 of runs against the exact $(\frac{1}{2})^{10} = 0.000977$. The mean loss is 0.0484. That matches the exact value, which is 10 times the mean loss of one count, $\varepsilon + e^{-\varepsilon} - 1$ at $\varepsilon = 0.1$, a formula you get by integrating the loss of Figure B.1 against the Laplace density. The script also recomputes Table B.2 from the two randomized-response steps, and the two-output pair above. So the bound is exact for the worst output, and very loose for the typical one.

B.1.6. Recipe and pitfalls

To share a total budget (ε, δ) among k releases with basic composition, follow three steps.

1. Give release i a share ε_i and δ_i, so that the shares add up to ε and δ. Equal shares ε/k and δ/k are the simplest choice.
2. Calibrate each release to its share, for example with Laplace noise of scale Δ_1/ε_i, where Δ_1 is its ℓ_1 sensitivity, the largest total of the absolute changes one record can make to its answers.
3. Draw fresh, independent noise for every release.

Several things go wrong in practice. Reusing one noise draw for two different queries breaks step 1 of the derivation, because the outputs are no longer independent given the data. For example, subtracting two answers that share one noise draw cancels the noise exactly. Publishing the same noisy answer twice is fine, though, since the copy is computed from the output without touching the data. Choosing the budgets themselves from the answers, such as "stop when the answer looks good", is not covered, because each budget must be fixed in advance (Section 36.3 handles it). The δ_i add up, so k releases at $\delta_i = 10^{-5}$ give $k \times 10^{-5}$. And for many releases the bound is far too large, which is the subject of the next section.

B.2. Advanced composition: $\sqrt{k}$ instead of k

B.2.1. The question

You publish 100 counts, each 0.1-DP. Basic composition says the total is $\varepsilon = 10$. That is almost no guarantee, because it allows some output to be up to $e^{10} \approx 22{,}000$ times more likely on one dataset than on its neighbour. An attacker who sees that output can multiply their odds on which dataset it was by that much. If you accept a tiny slack δ', can you claim a much smaller ε?

B.2.2. The answer

Yes. Take k releases that are each ε-DP and any slack $\delta' > 0$. Together they are (ε', δ')-DP with

$$\varepsilon' = \varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon(e^{\varepsilon} - 1). \tag{B.9}$$

The first term grows like $\sqrt{k}$, and the second is small when ε is small, because $e^{\varepsilon} - 1 \approx \varepsilon$. When each release is (ε, δ)-DP instead, the total slack becomes $k\delta + \delta'$. This is the theorem of Dwork, Rothblum and Vadhan, stated as Equation 21.9 in Section 21.2. The chapter also gives a sharper form whose second term is $k\varepsilon\tanh(\varepsilon/2)$. Here $\tanh(x) = (e^{2x} - 1)/(e^{2x} + 1)$ is the hyperbolic tangent, which is close to x for small x, so this term is about half the one above.

B.2.3. A worked example

Take $k = 100$, $\varepsilon = 0.1$ and $\delta' = 10^{-5}$. Then $\ln(1/\delta') = 11.5129$, so the first term is

$$0.1\sqrt{200 \times 11.5129} = 0.1\sqrt{2302.6} = 4.7985. \tag{B.10}$$

The second term is $100 \times 0.1 \times (e^{0.1} - 1) = 1.0517$. Their sum is $\varepsilon' = 5.8502$, against 10 for basic composition. The sharper tanh form gives 5.2981. The best bound that holds for every 0.1-DP release, the optimal composition of Section 21.4, gives 4.3068.

B.2.4. Why it works

The idea: a random walk. The total loss is a sum of k terms, one per release, and each lies between $-\varepsilon$ and ε. Basic composition assumes every term is $+\varepsilon$ at once. But the terms are random and partly cancel, like the steps of a walk that goes up or down at random. After k steps such a walk is typically about $\sqrt{k}$ steps from where it started, not k. The reason is that the variances of independent steps add, so the variance after k steps is k times that of one step, and the typical distance, its square root, grows like $\sqrt{k}$. The walk also drifts upward a little, because each term has a small positive average when the data are D, as step 2 shows. So the total loss is a small drift plus a fluctuation of order $\varepsilon\sqrt{k}$. A tail bound says how far above the drift the fluctuation can go except with probability δ', and the bad-event fact turns that into (ε', δ')-DP.

Randomized response shows the walk in its plainest form. At level ε each answer has loss exactly $+\varepsilon$ if it is truthful and $-\varepsilon$ if it is flipped. So the total loss of k answers is a walk with steps of $\pm\varepsilon$, which Figure B.2 draws. The chapter shows that randomized response is the worst case among all ε-DP mechanisms (Section 21.4). That is, no k releases that are each ε-DP need a larger δ, at any total ε, than k randomized responses do. So it is the right one to look at.

The derivation has seven steps.

1. **The loss is a sum of bounded terms.** By Section B.1 the total loss is $L = L_1 + \cdots + L_k$, where L_i is the loss of release i given the earlier outputs. Each release is ε-DP whatever the history, so $-\varepsilon \le L_i \le \varepsilon$.

2. **Each term has a small average.** Write m_i for the average of L_i on D, given the earlier outputs. The claim is that, for one ε-DP release with output probabilities p and q,

$$m = \sum_y p(y) \ln \frac{p(y)}{q(y)} \le \varepsilon(e^{\varepsilon} - 1). \tag{B.11}$$

   The proof uses one inequality about the logarithm of a positive number x,

$$\ln x \le x - 1. \tag{B.12}$$

   It holds because the line $x - 1$ touches the curve $\ln x$ at $x = 1$ and the curve bends down away from it. There are three sub-steps. The first one adds a term to m so that the positive and negative losses can be paired up, output by output.

- *Add a term that is never negative.* Apply $\ln x \le x - 1$ with $x = p/q$ and average under q. This gives $\sum_y q \ln(p/q) \le \sum_y q(p/q - 1) = \sum_y p - \sum_y q = 0$. Flipping the sign, and using $-\ln(p/q) = \ln(q/p)$, the sum $\sum_y q \ln(q/p)$ is never negative. Adding it to m can only make m larger, and the two sums combine term by term:

$$m \le \sum_y p \ln \frac{p}{q} + \sum_y q \ln \frac{q}{p} = \sum_y (p - q) \ln \frac{p}{q}. \tag{B.13}$$

- *Use the bound on the loss.* Each $|\ln(p/q)|$ is at most ε, so $m \le \varepsilon \sum_y |p - q|$.
- *Bound the total difference.* Write $|p - q| = q\,|p/q - 1|$. The ratio p/q lies between $e^{-\varepsilon}$ and e^{ε}, so $|p/q - 1|$ is at most $e^{\varepsilon} - 1$, its value at the upper end. At the lower end it is only $1 - e^{-\varepsilon}$, which is smaller. The $q(y)$ add up to 1, so $\sum_y |p - q| \le e^{\varepsilon} - 1$.

For randomized response, with two outputs, the average is easy to compute directly. Write $t = e^{\varepsilon}/(1 + e^{\varepsilon})$ for the chance of a truthful answer on D, whose loss is $+\varepsilon$, while a flipped answer has loss $-\varepsilon$. So $m = \varepsilon t - \varepsilon(1 - t) = \varepsilon(2t - 1)$. Substituting t gives $2t - 1 = (e^{\varepsilon} - 1)/(e^{\varepsilon} + 1)$, which is $\tanh(\varepsilon/2)$, so $m = \varepsilon \tanh(\varepsilon/2)$. At $\varepsilon = 0.1$ this is 0.0050, while the bound of Equation B.11 is 0.0105. At $\varepsilon = 1$ they are 0.4621 and 1.7183. So the bound holds and is about twice too large for small ε, which is where the chapter's sharper tanh form comes from.

3. **Centre the terms.** Write $X_i = L_i - m_i$. Given the earlier outputs, X_i has average zero, and it lies in an interval of length 2ε, from $-\varepsilon - m_i$ to $\varepsilon - m_i$. The total loss is the drift plus the sum of these centred terms, $L = \sum_i m_i + \sum_i X_i$, and the drift is at most $k\varepsilon(e^{\varepsilon} - 1)$ by step 2.

4. **One centred term has a small exponential average.** The next two steps bound the average of $e^{\lambda X}$ for a number $\lambda > 0$ of our choosing, because an exponential magnifies large values, and step 6 turns that into a tail bound. Hoeffding's lemma says that a variable X with average zero, which stays in an interval of length d, satisfies

$$\mathbb{E}\big[e^{\lambda X}\big] \le e^{\lambda^2 d^2/8} \qquad \text{for every real } \lambda. \tag{B.14}$$

The proof compares X with a variable that sits only at the two ends of the interval, which has the largest exponential average, and then bounds that average. Hoeffding's lemma (Lemma 2.2) gives the details. With $d = 2\varepsilon$ the right side is $e^{\lambda^2\varepsilon^2/2}$.

5. **The bounds multiply, even when releases adapt.** Write $W_j = X_1 + \cdots + X_j$ for the running sum, so that $e^{\lambda W_k} = e^{\lambda W_{k-1}} e^{\lambda X_k}$. Take the average of this product in two stages, first over the last output with the earlier ones fixed and then over the earlier ones. With the earlier outputs fixed, $e^{\lambda W_{k-1}}$ is a fixed number, and step 4 bounds the average of $e^{\lambda X_k}$ by $e^{\lambda^2\varepsilon^2/2}$ whatever the history was. So

$$\mathbb{E}\big[e^{\lambda W_k}\big] \le e^{\lambda^2\varepsilon^2/2}\,\mathbb{E}\big[e^{\lambda W_{k-1}}\big] \le \ldots \le e^{k\lambda^2\varepsilon^2/2}. \tag{B.15}$$

This is where adaptivity is handled. Each release may depend on the past, but the bound on its own term does not. This argument is Azuma's inequality.

6. **Turn the exponential average into a tail bound.** Markov's inequality says that a quantity that is never negative exceeds a level a with probability at most its average divided

by a. Apply it to $e^{\lambda W_k}$ at the level $a = e^{\lambda s}$, for a value s to be chosen. For $\lambda > 0$ the event $W_k \geq s$ is the event $e^{\lambda W_k} \geq e^{\lambda s}$, so

$$\Pr[W_k \geq s] \leq e^{-\lambda s + k\lambda^2\varepsilon^2/2}. \tag{B.16}$$

The exponent is a parabola in λ. Its derivative $-s + k\lambda\varepsilon^2$ is zero at $\lambda = s/(k\varepsilon^2)$, where the exponent equals $-s^2/(2k\varepsilon^2)$. Setting $e^{-s^2/(2k\varepsilon^2)} = \delta'$ and solving gives the level

$$s = \varepsilon\sqrt{2k\ln(1/\delta')}. \tag{B.17}$$

7. **Convert.** If $L > \varepsilon'$, then $\sum_i X_i = L - \sum_i m_i > \varepsilon' - k\varepsilon(e^\varepsilon - 1) = s$. So $\Pr[L > \varepsilon'] \leq \delta'$, and the bad-event fact Equation B.7 with $\tau = \varepsilon'$ gives (ε', δ')-DP. The same argument with D and D' swapped covers the other direction.

When each release is (ε, δ)-DP with $\delta > 0$, its loss can exceed ε on rare outputs, so step 1 fails. The fix uses a fact proved in the chapter: every such release behaves like a processed version of a simple mechanism that, with probability δ, announces which dataset it ran on, and otherwise runs ε-DP randomized response. Then the k announcements cost at most $k\delta$ by the union bound, and the rest is the argument above. Section 21.4 and Section J.1 carry this out.

B.2.5. Checking it

The script computes the loss of 100 randomized-response answers at $\varepsilon = 0.1$ exactly, from the binomial count of truthful answers. Its mean is 0.4996 and its standard deviation 0.9988, so the bound $\varepsilon' = 5.8502$ sits more than five standard deviations above the mean. The exact chance that the loss exceeds 5.8502 is 1.022×10^{-8}, far below the $\delta' = 10^{-5}$ the theorem promises. The exact δ at that ε', the smallest slack that works (Section B.3, step 4, shows how to compute it), is smaller still, 1.956×10^{-9}. So advanced composition is valid but conservative, for three reasons. Hoeffding's lemma allows for a larger spread than the walk has, and step 2 doubles the drift. Then the bad-event fact counts every output above ε' in full.

Figure B.2 draws 25 runs of the walk over 1000 answers. Basic composition is the straight line $i\varepsilon$, which leaves the plot at $i = 400$. The runs stay in a band around the drift whose width grows like $\sqrt{i}$. In 100,000 simulated runs of 1000 answers, the largest total loss was 18.4, while basic composition says 100 and advanced composition 25.6914.

B.2.6. Recipe and pitfalls

To spend a total budget $(\varepsilon_{\text{total}}, \delta')$ on k releases that are each ε-DP, set the leading term of Equation B.9 equal to the target. That gives each release roughly

$$\varepsilon \approx \frac{\varepsilon_{\text{total}}}{\sqrt{2k\ln(1/\delta')}}. \tag{B.18}$$

Then evaluate Equation B.9 with that ε, and lower it a little if the second term pushes the total over the target.

Advanced composition is worse than basic composition when k is small. Its first term is $\varepsilon\sqrt{k}$ times $\sqrt{2\ln(1/\delta')}$, which is 4.80 at $\delta' = 10^{-5}$, while basic composition is $\varepsilon k = \varepsilon\sqrt{k} \times \sqrt{k}$. So $\sqrt{k}$

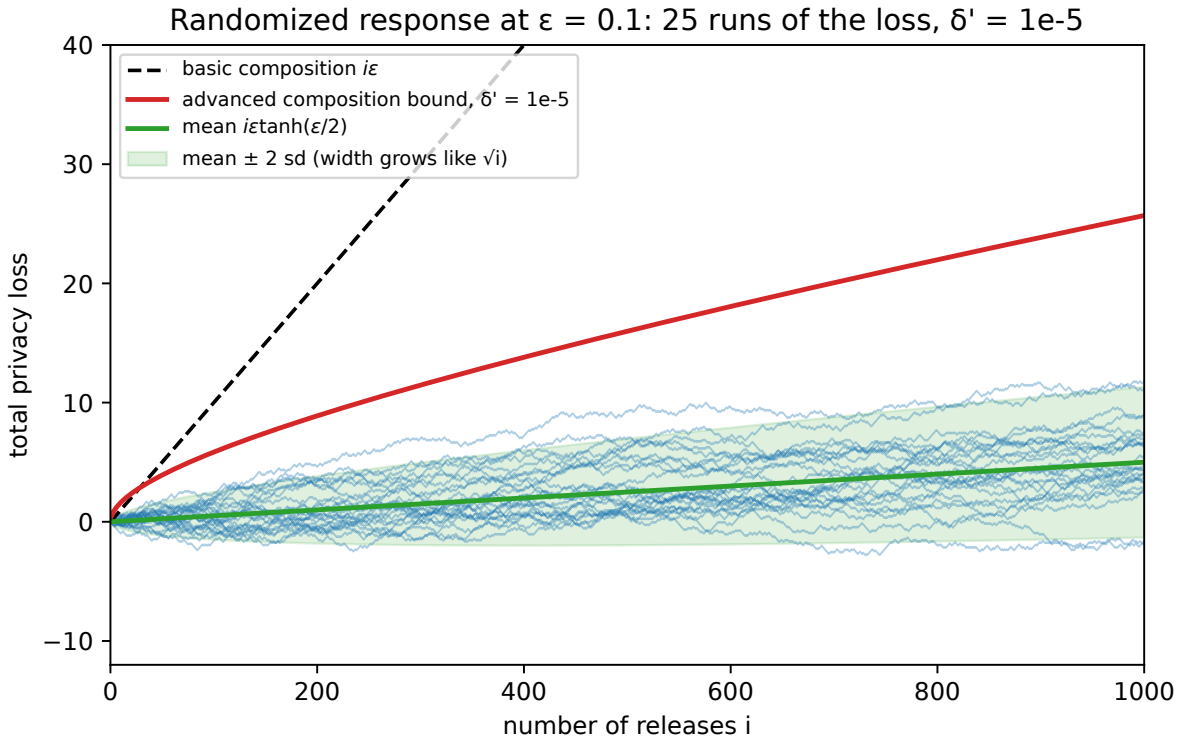


Figure B.2.: The total privacy loss of i randomized-response answers at $\varepsilon = 0.1$, for 25 simulated runs (thin lines). The green line is the mean loss and the shaded band the mean plus or minus two standard deviations, whose width grows like $\sqrt{i}$. The red curve is the advanced composition bound at $\delta' = 10^{-5}$, and the dashed line is the basic bound $i\varepsilon$.

must beat 4.80 and more. Setting the two bounds equal and solving for k shows that at $\varepsilon = 0.1$ advanced composition wins only once k exceeds $2\ln(1/\delta')/(2-e^{\varepsilon})^2 = 28.76$. So always compute both and keep the smaller. For $\varepsilon \geq \ln 2$ the second term alone is at least $k\varepsilon$, so the $e^{\varepsilon}-1$ form never wins, and you need the tanh form of the chapter. And when you know more than each release's ε, for example that the noise is Gaussian, the next two sections do much better.

B.3. Gaussian releases: k of them are one

B.3.1. The question

You add Gaussian noise of standard deviation σ to k answers, each with sensitivity Δ. What is the exact (ε, δ) guarantee of all k together? And how much noise must each answer get so that the whole set meets a total budget?

B.3.2. The answer

The k releases are exactly as private as **one** Gaussian release whose sensitivity is $\sqrt{k}\,\Delta$, with the same noise σ. In units of the noise, that single release has the shift

$$\mu_k = \frac{\sqrt{k}\,\Delta}{\sigma}. \tag{B.19}$$

So the smallest valid δ at a given ε is the exact Gaussian formula with μ_k in place of Δ/σ:

$$\delta(\varepsilon) = \Phi\Big(\frac{\mu_k}{2} - \frac{\varepsilon}{\mu_k}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu_k}{2} - \frac{\varepsilon}{\mu_k}\Big). \tag{B.20}$$

Read backwards, this gives the noise for k releases at a total budget. If σ_1 is the noise that makes one release (ε, δ)-DP, then

$$\sigma_k = \sqrt{k}\,\sigma_1 \tag{B.21}$$

makes k releases together (ε, δ)-DP. The single-release formula is Equation 20.17 in Section 20.4, and Section A.7 derives it.

B.3.3. A worked example

You want 10 counts with sensitivity 1 to be $(1, 10^{-5})$-DP together. One count at $(1, 10^{-5})$ needs $\sigma_1 = 3.7306$, from the exact calibration. By Equation B.21 each of the 10 counts gets $\sigma_{10} = \sqrt{10} \times 3.7306 = 11.797$. Putting $\mu_{10} = \sqrt{10}/11.797$ back into Equation B.20 and solving for ε at $\delta = 10^{-5}$ gives 1.0000, as it should.

Compare this with basic composition. Splitting the budget evenly gives each count $(0.1, 10^{-6})$, and the exact calibration for that share needs $\sigma = 36.305$. That is 3.08 times more noise for the same total guarantee. With 100 counts the two answers are $\sigma_{100} = 37.306$ and 362.02, a factor of 9.70.

B.3.4. Why it works

The idea. The privacy loss of one Gaussian release is itself a normal random variable. Losses add, and a sum of independent normal variables is normal, with the means added and the variances added. The sum of k losses turns out to be exactly the loss of one release with $\sqrt{k}$ times the shift. Since the smallest δ depends only on the distribution of the loss, the two have the same δ at every ε.

1. **The loss of one release is normal.** Take one count whose true value is 0 on D and Δ on D', and write the output on D as $y = z$, where z is the noise. The same output y sits at distance $z - \Delta$ from the centre on D'. The normal density with standard deviation σ is proportional to $e^{-z^2/(2\sigma^2)}$, so the loss is a difference of two exponents:

$$L = \frac{(z-\Delta)^2 - z^2}{2\sigma^2} = \frac{\Delta^2 - 2\Delta z}{2\sigma^2}. \tag{B.22}$$

   This is a straight line in the noise z. Its mean is $\Delta^2/(2\sigma^2) = \mu^2/2$, because z has mean 0. Its variance is $(\Delta/\sigma^2)^2\sigma^2 = \mu^2$, because multiplying z by a constant multiplies the variance by its square. A straight-line function of a normal variable is normal, so on D

$$L \sim \mathcal{N}\Big(\frac{\mu^2}{2},\ \mu^2\Big), \qquad \mu = \frac{\Delta}{\sigma}. \tag{B.23}$$

2. **Add k independent losses.** With fresh noise for each release, the k losses are independent, and the total loss is their sum (Section B.1). Means add, and so do variances of independent terms. A sum of independent normal variables is normal (Section E.1 proves it). So the total loss is normal with mean $k\mu^2/2$ and variance $k\mu^2$.
3. **Recognize one release.** A single release with shift $\mu_k = \sqrt{k}\,\mu$ has, by step 1, a loss with mean $\mu_k^2/2 = k\mu^2/2$ and variance $\mu_k^2 = k\mu^2$. That is the same normal distribution as in step 2.
4. **The exact δ depends only on the loss.** By the definition of (ε, δ)-DP in the opening, the smallest δ at a given ε is the largest gap $\Pr_D[S] - e^\varepsilon \Pr_{D'}[S]$ over sets of outputs. The gap is largest when S holds exactly the outputs with $p > e^\varepsilon q$, so the smallest δ is $\sum_y (p - e^\varepsilon q)_+$, where $(x)_+ = \max(x, 0)$. Factor p out of each term, and note that $e^\varepsilon q/p = e^{\varepsilon - L}$. This gives

$$\delta(\varepsilon) = \sum_y p(y)\big(1 - e^{\varepsilon - L(y)}\big)_+ = \mathbb{E}_D\big[(1 - e^{\varepsilon - L})_+\big]. \tag{B.24}$$

   The right side is an average over the loss alone. So two mechanisms whose losses have the same distribution on D have the same $\delta(\varepsilon)$ for every ε. By steps 2 and 3, the k releases have the $\delta(\varepsilon)$ of one release at μ_k, which is Equation B.20. The other direction, with D and D' swapped, gives the same value, because the Gaussian pair is symmetric.

There is a second way to see the same result. Stack the k answers into one vector. In the worst case, changing one record moves every answer by Δ, so the vector moves by $(\Delta, \dots, \Delta)$, whose length is $\sqrt{k\Delta^2} = \sqrt{k}\,\Delta$. Gaussian noise with the same σ in every coordinate looks the same in every direction. So you can turn the axes until the shift points along the first one, and the other $k - 1$ coordinates then carry no evidence at all. What is left is one Gaussian release with shift $\sqrt{k}\,\Delta$.

Adaptive queries. Suppose each query is chosen after seeing the earlier answers, and each still has sensitivity Δ and noise σ. If every query moves by exactly Δ, step 1 gives the same

normal distribution for each loss whatever the history was. A term whose distribution is the same whatever happened before is independent of what happened before, so steps 2 to 4 hold unchanged. Queries that move by less than Δ can only help, but proving that for an adaptive sequence takes the composition theorem of Gaussian DP (Section 32.4, Section K.3), which this page does not repeat.

B.3.5. Checking it

To test this, the script simulates 400,000 runs of 16 counts with noise $\sigma = 4$. Each count has $\mu = 0.25$, so the 16 together should behave like one release with $\mu_{16} = 4 \times 0.25 = 1$. Its loss should then have mean $\frac{1}{2}$ and variance 1, and the simulated total loss has mean 0.4971 and variance 1.0001. Averaging $(1 - e^{1-L})_+$ over the simulated losses estimates $\delta(1)$ as 0.1264. The formula Equation B.20 with $\mu_{16} = 1$ gives 0.1269. Figure B.3 shows the simulated losses for 1, 4 and 16 counts against the normal curve of a single release.

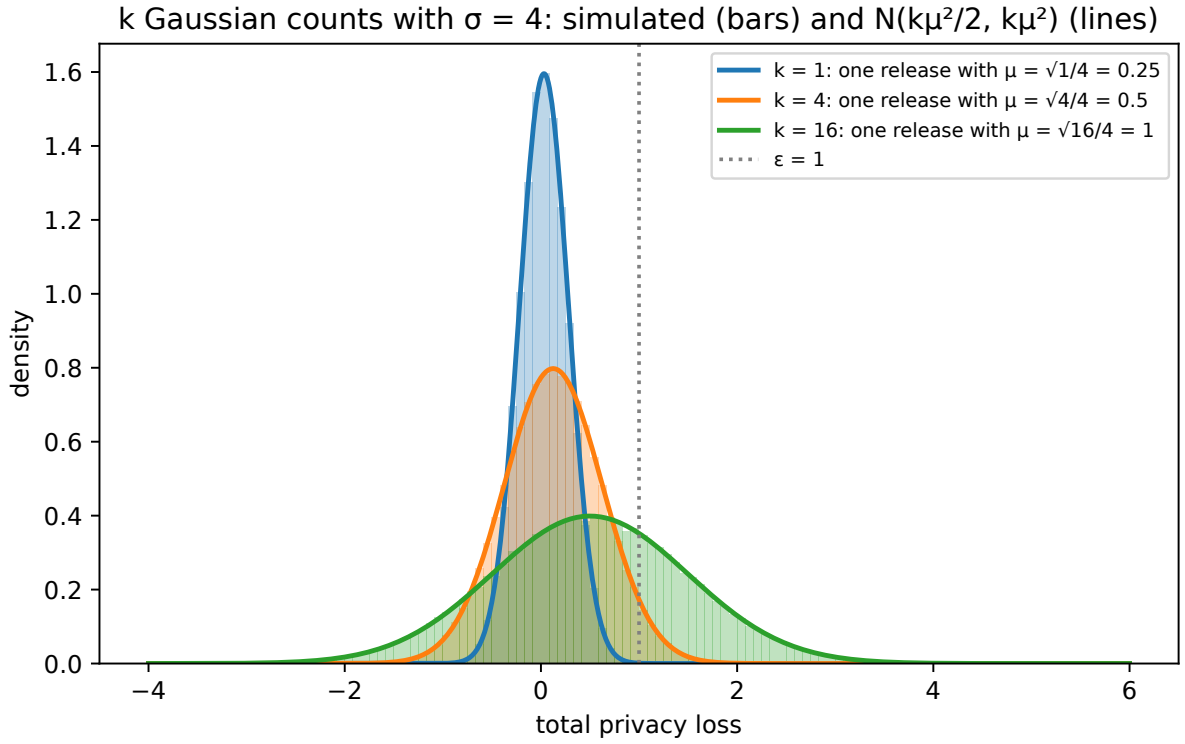


Figure B.3.: The total privacy loss of k Gaussian counts with noise $\sigma = 4$ and sensitivity 1, for $k = 1$, 4 and 16. Bars are simulated losses; lines are the normal distribution $\mathcal{N}(k\mu^2/2, k\mu^2)$ of one release with shift $\mu_k = \sqrt{k}/4$. The mean grows like k and the width like $\sqrt{k}$.

The result is exact, not a bound. Nothing was approximated in steps 1 to 4.

B.3.6. Recipe and pitfalls

To release k answers with Gaussian noise under a total budget (ε, δ), follow three steps.

1. Find σ_1, the exact noise for one release of sensitivity Δ at (ε, δ), by solving Equation 20.17 (Section A.7).
2. Multiply by the square root of the number of releases, $\sigma_k = \sqrt{k}\,\sigma_1$.
3. Add fresh noise $\mathcal{N}(0, \sigma_k^2)$ to each answer.

The other way round, given σ and k, put $\mu_k = \sqrt{k}\,\Delta/\sigma$ into Equation B.20 and solve for ε at your δ. When the releases have different sensitivities Δ_i or noise levels σ_i, the same steps give one release with $\mu^2 = \sum_i (\Delta_i/\sigma_i)^2$, because means and variances still add.

The sensitivity Δ must be the ℓ_2 sensitivity of each release under the neighbouring relation you claim. For example, a sum of values between $-C$ and C, for a clipping norm C, has sensitivity C under add/remove but $2C$ under replace-one. The result needs plain Gaussian noise. With subsampling, where each step uses only a random subset of the records, as in private stochastic gradient descent (SGD), the loss of a step is no longer normal, and Section C.5 shows what to do instead. Laplace releases do not combine into one Laplace release, so for them you need advanced composition or the accountants below.

B.4. Rényi DP: a summary of the loss that adds

B.4.1. The question

Many pipelines mix mechanisms, or use noise whose loss is not normal, so the exact shortcut of the previous section does not apply. You want a summary of each mechanism that you can add up across releases, and convert to (ε, δ) once at the end. What is that summary for the Gaussian mechanism, and why does it add?

B.4.2. The answer

Rényi differential privacy (RDP) summarizes the loss by averages of exponentials of it. A single number cannot capture both the typical and the rare large losses, so RDP keeps one such average for each **order** $\alpha > 1$, a dial that sets how much weight large losses get. For each order, the Rényi divergence between the two output distributions P (on D) and Q (on D') is

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \int p(y)^\alpha q(y)^{1-\alpha}\, \mathrm{d}y. \tag{B.25}$$

The integral is the average, on D, of $e^{(\alpha-1)L}$, because $p^\alpha q^{1-\alpha} = p\,(p/q)^{\alpha-1}$. So it weighs outputs with a large loss heavily, more so for larger α.

A mechanism is $(\alpha, \varepsilon(\alpha))$-RDP when, for every neighbouring pair,

$$D_\alpha(P\|Q) \le \varepsilon(\alpha). \tag{B.26}$$

The function $\alpha \mapsto \varepsilon(\alpha)$ is its **RDP curve**. The Gaussian mechanism with sensitivity Δ and noise σ has a straight line for its curve,

$$\varepsilon(\alpha) = \frac{\alpha \Delta^2}{2\sigma^2}, \tag{B.27}$$

and RDP curves add under composition. So k Gaussian releases have the curve $k\alpha\Delta^2/(2\sigma^2)$. The chapter states these as Equation 29.5 and Equation 29.11.

B.4.3. A worked example

Take $\Delta = 1$, $\sigma = 1$ and $\alpha = 2$. The formula gives $D_2 = 2 \times 1/(2 \times 1) = 1$. Undoing the logarithm and the factor $1/(\alpha - 1)$ in the definition, the integral inside is $e^{(\alpha-1)D_\alpha} = e^{(2-1)\times 1} = e = 2.7183$. You can check this with the derivation below, which shows the integrand is a normal bump with exactly that area (Figure B.4). Two such releases have $D_2 = 1 + 1 = 2$.

B.4.4. Why it works

The Gaussian curve, by completing the square. Put p at centre 0 and q at centre Δ, both with standard deviation σ. Write $c = 1/(\sigma\sqrt{2\pi})$ for the constant in front of the normal density.

1. **Combine the exponents.** Since $\alpha + (1 - \alpha) = 1$, the constants combine to a single c, and

$$p^\alpha q^{1-\alpha} = c \, \exp\Big(- \frac{\alpha y^2 + (1-\alpha)(y-\Delta)^2}{2\sigma^2} \Big). \tag{B.28}$$

2. **Expand the numerator.** Multiply out $(y - \Delta)^2 = y^2 - 2y\Delta + \Delta^2$ and collect terms:

$$\alpha y^2 + (1-\alpha)(y-\Delta)^2 = y^2 - 2(1-\alpha)\Delta y + (1-\alpha)\Delta^2. \tag{B.29}$$

3. **Complete the square.** The first two terms are the start of $\big(y - (1-\alpha)\Delta\big)^2$, which also contains $(1-\alpha)^2\Delta^2$. Adding and subtracting that term gives

$$y^2 - 2(1-\alpha)\Delta y + (1-\alpha)\Delta^2 = \big(y - (1-\alpha)\Delta\big)^2 + (1-\alpha)\Delta^2(1 - (1-\alpha)). \tag{B.30}$$

   The last term simplifies to $(1-\alpha)\alpha\Delta^2 = -\alpha(\alpha-1)\Delta^2$.

4. **Integrate.** Putting this back into step 1, the exponent is $-\big(y - (1-\alpha)\Delta\big)^2/(2\sigma^2)$ plus the constant $+\alpha(\alpha-1)\Delta^2/(2\sigma^2)$, whose sign flips because of the minus in front of the fraction. So

$$\int p^\alpha q^{1-\alpha} \, \mathrm{d}y = e^{\alpha(\alpha-1)\Delta^2/(2\sigma^2)} \int c \, e^{-(y-(1-\alpha)\Delta)^2/(2\sigma^2)} \, \mathrm{d}y = e^{\alpha(\alpha-1)\Delta^2/(2\sigma^2)}. \tag{B.31}$$

   The integral that remains is a normal density centred at $(1-\alpha)\Delta$, so it equals 1.

5. **Take the logarithm and divide by** $\alpha - 1$**.** This gives $D_\alpha = \alpha\Delta^2/(2\sigma^2)$. The value grows with the shift, so the worst neighbouring pair is the one whose answers are Δ apart, which gives Equation B.27.

Why the curves add. For two independent releases the joint density is a product, $p(y_1, y_2) = p_1(y_1)p_2(y_2)$, and the same for q. A power of a product is the product of the powers, so the integrand splits into a factor for y_1 and a factor for y_2. The double integral of such a product is the product of the two single integrals:

$$\iint p_1^{\alpha} q_1^{1-\alpha}\, p_2^{\alpha} q_2^{1-\alpha}\, \mathrm{d}y_1\, \mathrm{d}y_2 = \int p_1^{\alpha} q_1^{1-\alpha}\, \mathrm{d}y_1 \cdot \int p_2^{\alpha} q_2^{1-\alpha}\, \mathrm{d}y_2. \tag{B.32}$$

Taking the logarithm turns this product into a sum, and dividing by $\alpha - 1$ gives D_{α} of the pair as the sum of the two divergences. When the second release is chosen from the first output, do the inner integral over y_2 first, with y_1 fixed. That inner integral is $e^{(\alpha-1)D_{\alpha}}$ for the mechanism chosen after y_1, so it is at most $e^{(\alpha-1)\varepsilon_2(\alpha)}$ whatever y_1 was. Replacing it by that bound leaves a constant times the first release's integral. So the curves still add, now as an upper bound (Section 29.3). Unlike the composition of (ε, δ) guarantees, nothing like δ is added at each step; the only slack comes in the one conversion at the end.

For k Gaussian releases the sum is $k\alpha\Delta^2/(2\sigma^2) = \alpha(\sqrt{k}\,\Delta)^2/(2\sigma^2)$, the curve of one release with $\sqrt{k}$ times the shift. That agrees with Section B.3. Other mechanisms have their own curves. The Laplace curve, for example, levels off at the pure ε of the mechanism (Equation 29.6).

Converting to (ε, δ). At the end you convert the summed curve once. For any order α, the mechanism is (ε, δ)-DP with

$$\varepsilon = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1}, \tag{B.33}$$

and you take the order that makes this smallest. The reason is the bad-event fact Equation B.7, with the tail $\Pr[L > \tau]$ bounded by Markov's inequality applied to $e^{(\alpha-1)L}$, whose average is what RDP controls. Section D.3 derives it. For example, take 1000 Gaussian steps with sensitivity 1 and noise $\sigma = 20$. Their summed curve is $1000\alpha/800 = 1.25\alpha$. At $\delta = 10^{-5}$ you minimize $1.25\alpha + 11.5129/(\alpha - 1)$ over the grid of orders that the accounting chapters use, which runs from 1.1 to 512. The best is $\varepsilon = 8.838$, at $\alpha = 4$. The exact answer from Section B.3 is 7.511, so the conversion costs 18%.

B.4.5. Checking it

A numerical integral of $p^{\alpha}q^{1-\alpha}$ on a fine grid, which uses no formula for the result, confirms the curve. It gives 1.0000 at $\alpha = 2$ and 5.0000 at $\alpha = 10$ with $\sigma = 1$, and 0.0625 at $\alpha = 2$ with $\sigma = 4$. These match $\alpha/(2\sigma^2)$ to every digit shown. Figure B.4 shows why step 4 works. The product p^2q^{-1} is a normal bump centred at $(1 - \alpha)\Delta = -1$, on the far side of p from q, and its area is e^1.

B.4.6. Recipe and pitfalls

To account with RDP, follow three steps.

1. Write down each release's RDP curve on a grid of orders, such as $\alpha\Delta^2/(2\sigma^2)$ for a Gaussian release.
2. Add the curves order by order.
3. Convert once, at the order that gives the smallest ε.

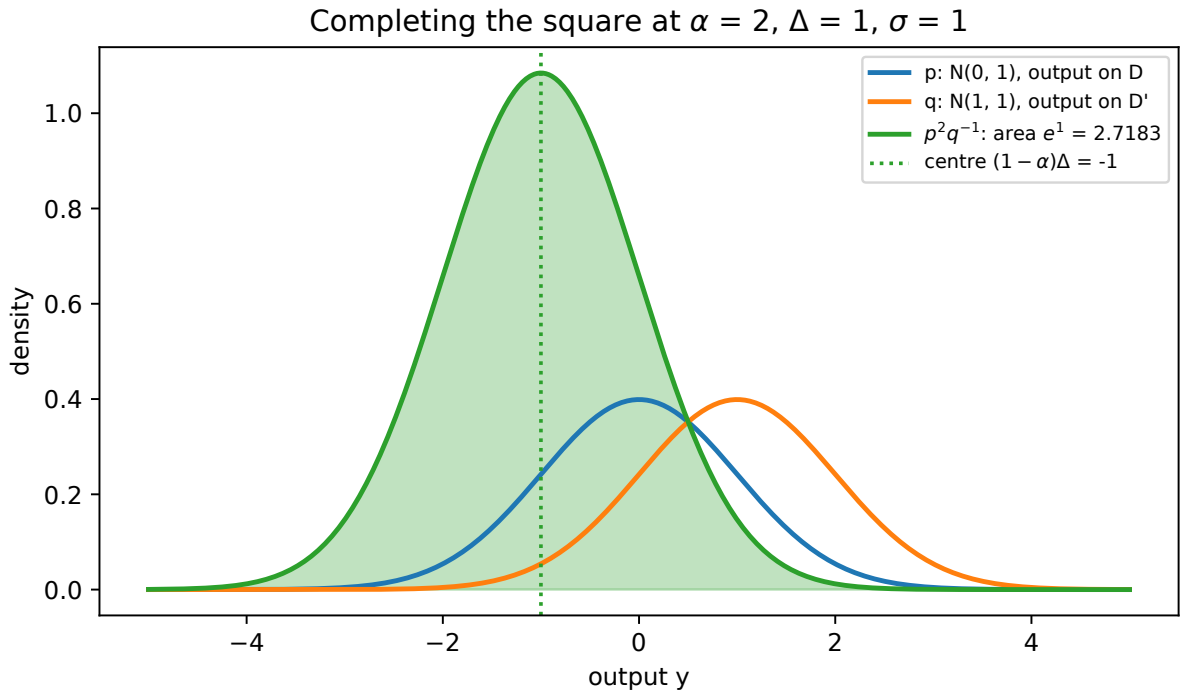


Figure B.4.: The two output densities of a Gaussian release with $\Delta = 1$ and $\sigma = 1$ (blue on D, orange on D'), and the integrand $p^\alpha q^{1-\alpha}$ for $\alpha = 2$ (green). Completing the square shows that the integrand is a normal bump centred at $(1-\alpha)\Delta = -1$, scaled up by $e^{\alpha(\alpha-1)\Delta^2/(2\sigma^2)} = e$, so its area is $e = 2.7183$ and $D_2 = 1$.

Do not convert each release to (ε, δ) first and then compose, because that throws away the curve and lands you back at basic or advanced composition. The grid of orders must reach small orders, because a steep summed curve makes large orders expensive, and the best order for many steps can be 2 or less. The converted ε overstates the exact one by an amount that shrinks slowly with k, from 57% at one step to 10% at 10,000 steps in the comparison below. When every release is a plain Gaussian, use the exact result of Section B.3 instead.

B.5. Comparing the four accountants

B.5.1. The question

You now have four ways to add up the cost of k releases: basic, advanced, RDP and exact. For the same mechanism, how different are their answers, and which should you use?

B.5.2. The answer

Fix one mechanism, a Gaussian release with sensitivity 1 and noise $\sigma = 20$, and a total $\delta = 10^{-5}$. Table B.3 gives the total ε of k releases under each accountant, from the script. Each accountant assumes something different. Two of them use the release's exact privacy curve, the smallest δ for each ε from Equation B.20 with $k = 1$.

- **Basic** gives each release the share δ/k, reads its ε_0 off the release's exact curve, and adds the k values of ε_0.
- **Advanced** spends a fraction κ of δ on the releases, $\kappa\delta/k$ each, and the rest on δ'. It reads ε_0 off the exact curve, applies Equation B.9, and keeps the best κ on a grid from $10^{-1/4}$ down to 10^{-10}.
- **RDP** adds the curves Equation B.27 and converts once.
- **Exact** is Equation B.20 with $\mu_k = \sqrt{k}/20$.

Table B.3.: Total ε at $\delta = 10^{-5}$ for k Gaussian releases with sensitivity 1 and noise $\sigma = 20$, under four accountants.

k	Basic	Advanced	RDP	Exact	RDP ÷ exact
1	0.160	0.864	0.251	0.160	1.57
10	1.892	3.502	0.771	0.561	1.37
100	21.510	16.462	2.524	1.993	1.27
1,000	238.578	105.863	8.838	7.511	1.18
10,000	2,601.968	938.234	36.513	33.104	1.10

B.5.3. A worked example

Take $k = 1000$, the size of a short training run. Basic composition gives each step $\delta = 10^{-8}$ and reports $\varepsilon = 238.578$. Advanced composition reports 105.863, or 68.008 with the chapter's sharper tanh form. RDP reports 8.838, and the exact answer is 7.511. So the two accountants that see

each release only as one $(\varepsilon_0, \delta_0)$ pair overstate ε by a factor of 9 to 32, while RDP overstates it by 18%.

B.5.4. Why it works

The order of the columns follows how much each accountant knows about a release. Basic and advanced composition know only one point $(\varepsilon_0, \delta_0)$ of each release's privacy curve. A Gaussian release has a whole curve of valid pairs, and collapsing it to one point throws away how the release behaves at every other point. Advanced composition still wins over basic for large k, because it uses the cancellation of Section B.2. RDP keeps a whole curve of exponential moments, which adds exactly, and loses only in the one conversion at the end. The exact accountant keeps the whole distribution of the loss, which for Gaussian noise is a normal distribution described by two numbers.

At $k = 1$ basic composition equals the exact answer, since there is nothing to add. Advanced composition is worst there, because its $\sqrt{2\ln(1/\delta')}$ factor is paid even for one release. As k grows, basic composition grows like k, advanced like $\sqrt{k}$ until its drift term takes over, and RDP and exact both like $\sqrt{k}$.

B.5.5. Checking it

Figure B.5 plots the four accountants for k from 1 to 10,000, on log axes, and their ratio to the exact answer. The basic line has slope 1, so it grows like k. The RDP and exact lines run parallel, with slope one half, so they grow like $\sqrt{k}$. RDP's excess shrinks from 1.57 times the exact value at one release to 1.10 times at 10,000, because the conversion's extra term $\ln(1/\delta)/(\alpha - 1)$ matters less as the curve grows. The exact line is the true value, so every other line lies above it. The script's value at $k = 1000$, 7.511, is also the one the RDP chapter quotes for this mechanism.

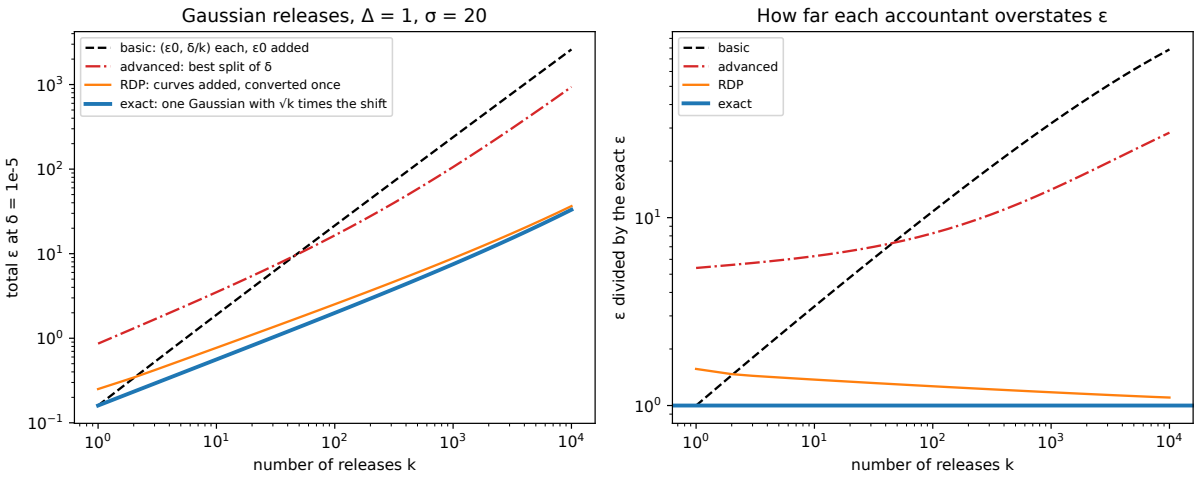


Figure B.5.: Left: the total ε at $\delta = 10^{-5}$ of k Gaussian releases with sensitivity 1 and noise $\sigma = 20$, on log axes, under basic composition (dashed), advanced composition with the best split of δ (dash-dot), RDP converted once (orange) and the exact answer (thick blue). Right: each accountant's ε divided by the exact one.

B.5.6. Recipe and pitfalls

Choose the accountant by what you know about each release.

1. **Every release is a plain Gaussian.** Use the exact result of Section B.3. It is a closed formula and the true answer.
2. **The releases mix mechanisms, or are subsampled, as in private training.** Use RDP, or a numerical accountant of the privacy loss distribution (PLD), which computes the exact answer to a chosen precision (Section 34.1). Section C.5 works through the subsampled case.
3. **You know only each release's** $(\varepsilon_0, \delta_0)$**.** Use optimal composition (Section 21.4), with advanced composition as a quick check by hand.
4. **Only a few releases, or pure DP with different budgets.** Basic composition is simple and close enough, and it is exact for the worst case of pure DP.

Report one accountant, chosen before you look at the results, and state which one it was and what δ it used. Every accountant needs the same neighbouring relation for every release, and none of them covers budgets chosen on the fly. For those cases, see Section 36.3.

B.6. Code

The script `code/appendix-derivations/apx_core_composition.py` computes every number on this page and draws its five figures. It simulates the 10-count Laplace dashboard and rebuilds the adaptive randomized-response table. It computes the advanced bound and checks it against the exact loss of composed randomized response, and simulates the random walk. For Gaussian releases it simulates the summed loss, checks Equation B.20 and the $\sqrt{k}$ noise recipe, and integrates the Rényi divergence numerically. Finally, it runs the four accountants on one mechanism. Run it with

```
uv run python code/appendix-derivations/apx_core_composition.py
```

Table B.4 collects the main numbers it prints.

Table B.4.: The main numbers printed by the appendix's script, with the default seed 0.

Check	Printed values
10 Laplace counts at $\varepsilon = 0.1$	largest loss 1.0000; reached in 0.000997 of runs (exact 0.000977); mean 0.0484
Two-output pair	$(1, 0.0476)$-DP, yet $\Pr[L > 1] = 0.50$
Mean loss of one step, $\varepsilon = 0.1$ and 1	bound 0.0105 and 1.7183; randomized response 0.0050 and 0.4621
Advanced, $k = 100$, $\varepsilon = 0.1$, $\delta' = 10^{-5}$	$4.7985 + 1.0517 = 5.8502$; tanh form 5.2981; optimal 4.3068; basic 10
Exact tail at 5.8502	$\Pr[L > \varepsilon'] = 1.022 \times 10^{-8}$; exact $\delta = 1.956 \times 10^{-9}$

Check	Printed values
16 Gaussian counts, $\sigma = 4$	loss mean 0.4971 (0.5), variance 1.0001 (1); $\delta(1)$ 0.1264 simulated, 0.1269 exact
Noise for $(1, 10^{-5})$ in total	$\sigma_1 = 3.7306$; $\sigma_{10} = 11.797$ against 36.305 by basic splitting
Rényi divergence, numerical against formula	1.0000, 5.0000, 0.0625
1000 Gaussian steps, $\sigma = 20$	RDP 8.838 at $\alpha = 4$; exact 7.511

Three things stand out. Basic composition is exact for the worst case of pure DP but very loose for the typical case. Advanced composition is valid with a large margin: in the worked example the true δ is smaller than the promised one by a factor of $10^{-5}/(1.956 \times 10^{-9}) = 5{,}112$. And for Gaussian noise the exact answer is as easy to compute as any bound, so there is no reason to use a bound.

B.7. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014). Basic and advanced composition, with the proofs this page follows.
- Dwork, Rothblum & Vadhan, "Boosting and Differential Privacy" (2010). The original advanced composition theorem.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (2015). Optimal composition, and why randomized response is the worst case.
- Hoeffding, "Probability Inequalities for Sums of Bounded Random Variables" (1963), and Azuma, "Weighted Sums of Certain Dependent Random Variables" (1967). The tail bounds behind advanced composition.
- Mironov, "Rényi Differential Privacy" (2017). The definition of RDP, the Gaussian and Laplace curves, composition and the conversion to (ε, δ).
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising" (2018). The exact $\delta(\varepsilon)$ of the Gaussian mechanism used in Equation B.20.
- Dong, Roth & Su, "Gaussian Differential Privacy" (2022). Why Gaussian releases compose exactly, including adaptive ones.
- Abadi et al., "Deep Learning with Differential Privacy" (2016). The moments accountant, the form of RDP accounting used in private training.

C. Accounting for DP-SGD

This appendix follows one training run from the clipped gradient to its final guarantee of differential privacy (DP). The run is stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, which is called DP-SGD. A training algorithm is **differentially private** when adding or removing one record changes the probability of any set of outcomes by at most a factor e^{ε}, plus a small slack δ. The **sensitivity** of a computation is the most that one record can move its result. The **Gaussian mechanism** adds normal noise to such a result, and its privacy depends only on the sensitivity divided by the noise's standard deviation. Appendix Section A.7 derives the Gaussian mechanism's exact guarantee, and Section B.4 derives the Rényi divergence, the tool that adds up the cost of many steps. This page uses both and restates each where it is needed.

The five sections answer the questions a practitioner asks, in the order a training run raises them. Clipping fixes the sensitivity, and the noise then turns each step into a Gaussian mechanism. Random batches make each step cheaper, and a curve built from the Rényi divergence measures that cheaper step exactly. Finally, an accountant adds the steps up and converts the total into one (ε, δ). Every number uses the book's running example, so you can check each one against the chapters. It trains on 10,000 records for 100 epochs. Each batch is drawn at random and holds 100 records on average, and the run and reports its guarantee at $\delta = 10^{-5}$. Table C.1 maps each section to the chapter sections that use its formula.

Table C.1.: The five derivations of this appendix and the chapter sections that use them.

Section	Formula it derives	Used in
Section C.1	the clipped sum has sensitivity C	Section 39.1, Section 39.5, Section 33.2
Section C.2	one step has ratio $1/\sigma$; T unsampled steps have $\sqrt{T}/\sigma$	Section 39.1, Section 39.3, Section 20.4
Section C.3	$\varepsilon' = \ln\left(1 + q(e^{\varepsilon} - 1)\right)$	Section 33.1
Section C.4	the binomial sum for the subsampled Gaussian's Rényi curve	Section 33.3, Section 33.4, Equation 39.4
Section C.5	$\varepsilon = \min_{\alpha} \left[T\varepsilon(\alpha) + \ln(1/\delta)/(\alpha - 1)\right]$	Section 33.5, Section 29.6, Section 37.2

The notation is that of the DP-SGD chapter, and each symbol is introduced where it first appears. Two datasets are **neighbours** when one of them is the other with one record added. Write D for a dataset and $D' = D \cup \{x\}$ for the same dataset with the record x added. This **add/remove**

relation is the one the book uses for DP-SGD. The reason is the way DP-SGD draws its batches, by an independent coin flip for each record, so that adding a record simply adds one more coin (Section 18.1, and Section C.3).

C.1. Clipping gives the sum a sensitivity

C.1.1. The question

The noise has to hide any one record's contribution, so you need a bound on how far one record can move the sum of gradients. But a gradient can have any length. A mislabelled record far from the decision boundary can have a gradient a thousand times longer than a typical one, and nothing about the model bounds it. How do you get a bound that holds for every model and every record?

C.1.2. The answer

Write g_i for the **per-example gradient** of record i, the gradient of that record's own loss at the current weights. You rescale each one so that its length is at most a number C that you choose, the **clipping norm**. Rescaling keeps the direction and caps the length:

$$\bar{g}_i = g_i \min\Big(1, \frac{C}{\|g_i\|}\Big), \qquad \bar{G}(D) = \sum_{i \in D} \bar{g}_i. \tag{C.1}$$

Here $\|g_i\|$ is the Euclidean length of the gradient and $\bar{G}(D)$ is the **clipped sum** over the records of a dataset D. Its ℓ_2 sensitivity is the largest length of $\bar{G}(D') - \bar{G}(D)$ over neighbouring datasets, and it is

$$\Delta_2 \bar{G} = C \quad \text{under add/remove}, \qquad \Delta_2 \bar{G} = 2C \quad \text{under replace-one}. \tag{C.2}$$

Under **replace-one**, the other common relation, neighbouring datasets have the same size and differ in the value of one record. The book uses the add/remove relation for DP-SGD, so the sensitivity is C.

C.1.3. A worked example

Take $C = 1$ and a dataset D of two records with two-dimensional gradients. The first is $(3, 4)$, of length 5, so clipping divides it by 5 and gives $(0.60, 0.80)$. The second is $(0.3, -0.4)$, of length 0.5, which is shorter than C and stays as it is. The clipped sum is $\bar{G}(D) = (0.90, 0.40)$.

Now add a third record x with gradient $(-2.4, 0.7)$, of length 2.5. Clipping divides it by 2.5 and gives $(-0.96, 0.28)$, so $\bar{G}(D') = (-0.06, 0.68)$. The sum moved by $(-0.96, 0.28)$, which is exactly the new record's clipped gradient. Its length is $\sqrt{0.96^2 + 0.28^2} = 1.00$, which is C.

Under the replace-one relation, change the first record's gradient from $(3, 4)$ to $(-3, -4)$ instead. Its clipped term goes from $(0.60, 0.80)$ to $(-0.60, -0.80)$, so the sum becomes $(-0.30, -1.20)$. It moved by $(-1.20, -1.60)$, of length 2.00, which is $2C$.

C.1.4. Why it works

1. **A clipped gradient has length at most C.** If $\|g_i\| \le C$, the factor $\min(1, C/\|g_i\|)$ is 1 and the gradient is unchanged. Otherwise the factor is $C/\|g_i\|$, and the length becomes $\|g_i\| \cdot C/\|g_i\| = C$. Either way $\|\bar{g}_i\| \le C$, and the direction is the same because the factor is positive.
2. **The other records' terms do not change.** Each record's gradient is computed from that record alone and from the current weights. The weights come from the earlier noisy steps, which are already released, so they are fixed when this step runs. Adding x therefore leaves every other term of the sum exactly as it was.
3. **Add/remove: the sum moves by one clipped gradient.** By step 2, $\bar{G}(D') - \bar{G}(D) = \bar{g}_x$, and by step 1 its length is at most C. That is the first half of Equation C.2.
4. **Replace-one: the sum moves by a difference of two.** If record x is replaced by x', the sum moves by $\bar{g}_{x'} - \bar{g}_x$. The triangle inequality says that the length of a difference is at most the sum of the lengths, so it is at most $C + C = 2C$. The worked example reaches $2C$, because two gradients that point in opposite directions give the largest difference.

Nothing in these steps used the loss, the architecture or the data. That is why clipping gives a sensitivity for every model.

C.1.5. Checking it

This appendix's script, described in the Code section at the end, draws 20,000 random datasets of five two-dimensional gradients, with lengths that are often far above C, and a random new record for each. The largest change of the clipped sum, in units of C, is 1.0000 when the record is added and 2.0000 when it replaces another. Both bounds are reached, so neither can be improved.

Figure C.1 shows the mechanism. On the left, each gradient outside the circle of radius C is pulled back onto it, along its own direction, and those inside are left alone. On the right, adding one record moves the clipped sum by that record's clipped gradient, so the new sum always lands inside a circle of radius C around the old one.

C.1.6. Recipe and pitfalls

Clip every per-example gradient to a norm C that is fixed before training, and the sum has sensitivity C under add/remove. The clipping norm is not a privacy parameter. The next section sets the noise to σC, a fixed multiple σ of the clipping norm. So a larger C means more noise in the same proportion, and the two changes cancel in ε (Section C.2). The clipping norm affects only accuracy, which Section 39.2 studies.

Step 2 is where implementations break the argument. Batch normalization makes each record's gradient depend on the other records in the batch, so one record can move every term of the sum. A clipping norm computed from the current batch, such as its median gradient length, has the same flaw in another place. The noise level then depends on the data, so it can itself reveal something about the records.

Two other cases double the sensitivity. Clipping the mean gradient of a group of records, a microbatch, has sensitivity $2C$. The group's clipped mean is a vector of length at most C both

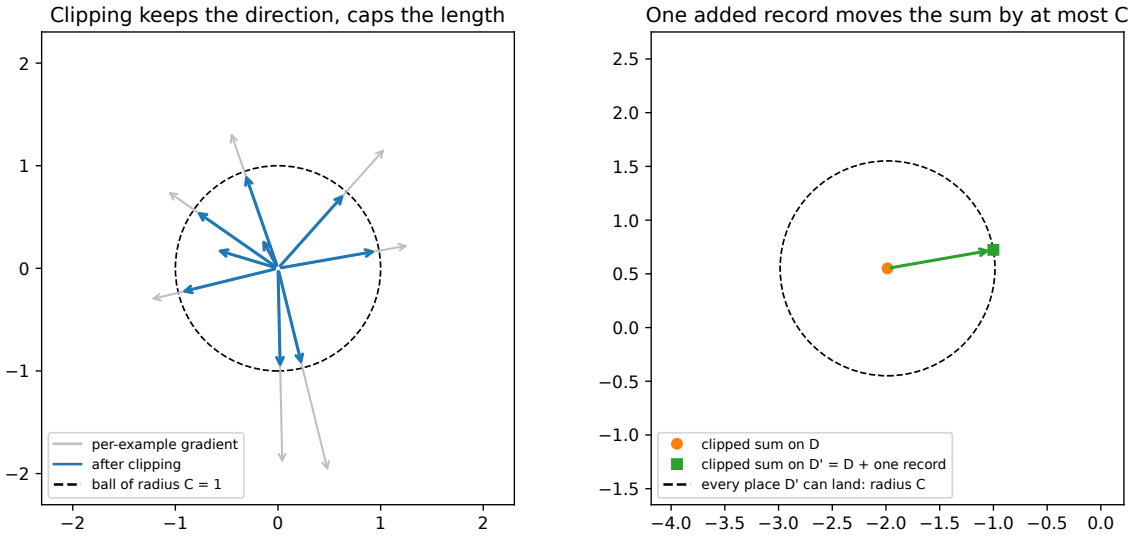


Figure C.1.: Left: nine random per-example gradients (grey) and the same gradients after clipping to norm $C = 1$ (blue), with the ball of radius C dashed. Right: the clipped sum on D and on D plus one record, with the circle of radius C around the first, inside which the second must land.

before and after one member is added, and the two vectors can point in opposite directions, as in the replace-one example above (Section 39.5). The replace-one relation also has sensitivity $2C$. In both cases the program that adds up the privacy cost, the **accountant**, must be told, because it assumes C (Section 33.2).

C.2. One noisy step is a Gaussian mechanism

C.2.1. The question

Each step releases the clipped sum plus noise. How much noise makes that one step private, and what do the rest of the update and the later steps cost?

C.2.2. The answer

DP-SGD adds normal noise to the clipped sum, independently in every coordinate. Its standard deviation is σC, where the number σ is the **noise multiplier**, the noise's standard deviation in units of the sensitivity. So step number t of training releases

$$G_t = \bar{G} + \xi_t, \qquad \xi_t \sim \mathcal{N}(0, \sigma^2 C^2 I). \tag{C.3}$$

Here I is the identity matrix, which says that the noise ξ_t has the same spread in every direction.

A Gaussian mechanism is described by one ratio, the sensitivity divided by the noise's standard deviation, which the book calls μ. For one step it is

$$\mu = \frac{\Delta_2 \bar{G}}{\sigma C} = \frac{C}{\sigma C} = \frac{1}{\sigma}. \tag{C.4}$$

A larger ratio means the two output distributions overlap less, so an observer can tell them apart more easily. The exact trade-off between ε and δ is the probability mass where the distribution with the record exceeds e^{ε} times the one without it. For the mechanism with ratio μ that mass gives the condition

$$\delta \geq \delta_{\mu}(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big), \tag{C.5}$$

where $\Phi(z)$ is the probability that a standard normal variable is at most z (Section 20.4, and Section A.7 for a derivation from scratch). Without sampling, every record is used in every step. If the run has T steps, they act together like one Gaussian mechanism with ratio

$$\mu_T = \frac{\sqrt{T}}{\sigma}. \tag{C.6}$$

C.2.3. A worked example

Take $\sigma = 1$, so one step has $\mu = 1$. Solving Equation C.5 for $\delta = 10^{-5}$ gives $\varepsilon = 4.38$ for a single step, which is already a large privacy cost. The cost grows quickly with the number of steps if every record is used in each one. For example, 100 steps have $\mu_T = \sqrt{100} = 10$ and $\varepsilon = 91.82$, and the running example's 10,000 steps have $\mu_T = 100$ and $\varepsilon = 5425.51$. Nobody would report such a number, and the next two sections show how random batches bring it down to single digits.

The DP-SGD chapter has a full-batch row in Table 39.4, where every batch is the whole dataset, so its 100 epochs are 100 steps. For a target of $\varepsilon = 4$ that row uses $\sigma = 10.8$. So $\mu_T = \sqrt{100}/10.8 = 0.926$, and the script gives $\varepsilon = 4.005$, the target up to the rounding of σ.

C.2.4. Why it works

1. **Only the ratio matters.** Compare the noisy sum on D and on D'. The two are normal distributions with the same spread, centred s apart, where the shift s is the length of $\bar{G}(D') - \bar{G}(D)$. Rotate the space so that this shift lies along the first axis. The noise has the same spread in every direction, so rotating it leaves it unchanged. Every other coordinate then has the same noise on both datasets and tells an observer nothing. What is left is one coordinate, $\mathcal{N}(0, \sigma^2 C^2)$ against $\mathcal{N}(s, \sigma^2 C^2)$. Dividing the output by σC changes no probability of any event, so this is the pair $\mathcal{N}(0, 1)$ against $\mathcal{N}(s/(\sigma C), 1)$.
2. **The worst shift gives ratio $1/\sigma$.** By Section C.1 the shift is at most C, so the ratio is at most $C/(\sigma C) = 1/\sigma$. A smaller ratio is never less private, because the two distributions then overlap more. Notice that the clipping norm cancels, which is why it is not a privacy parameter.
3. **The rest of the update is free.** The learning rate, the division by the expected batch size B, momentum, Adam and the new weights are all computed from G_t and from earlier released values, without looking at the data again. Such a computation is called **post-processing**, and it cannot reveal more than its input. The reason is that an attacker who

sees G_t could run the same computation herself, so any event about the processed output is an event about G_t, and its probability obeys the same bound (Section 19.1).

4. **Unsampled steps add up like one Gaussian.** The **privacy loss** of an output y is the logarithm of how much more likely y is on D' than on D. For the pair $\mathcal{N}(\mu, 1)$ against $\mathcal{N}(0, 1)$, the loss is

$$\ln \frac{e^{-(y-\mu)^2/2}}{e^{-y^2/2}} = \mu y - \frac{\mu^2}{2}. \tag{C.7}$$

With $y \sim \mathcal{N}(\mu, 1)$, this is a normal variable with mean $\mu^2/2$ and variance μ^2. Each step draws fresh noise, independent of the earlier steps. So over T steps the joint likelihood ratio is the product of the steps' ratios, and the losses add. Each step's loss has this same distribution whatever happened before, so the sum is normal with mean $T\mu^2/2$ and variance $T\mu^2$. That is exactly the loss of one Gaussian mechanism with ratio $\sqrt{T}\mu = \sqrt{T}/\sigma$. Since δ for a given ε is computed from the distribution of the loss alone, the two have the same guarantee, and Section B.3 proves it in full.

C.2.5. Checking it

The full-batch check above recovers the chapter's $\varepsilon = 4$ to three digits. It also confirms step 4, because the chapter solved its row for 100 composed steps, while here the 100 steps are one Gaussian with $\mu_T = 10/\sigma$. The script solves Equation C.5 by bisection on ε, working with logarithms, because at $\mu_T = 100$ both terms underflow to zero in ordinary arithmetic. The value at 10,000 steps, 5425.51, agrees with the 5425.5 quoted in Amplification by subsampling.

C.2.6. Recipe and pitfalls

Set the noise to σC, and one step is a Gaussian mechanism with ratio $1/\sigma$, whatever C is. Keep everything after the noise a function of the noisy sum and of public numbers. Dividing by the number of records actually in the batch is the common mistake here, because that count depends on the data and changes when x is added. Divide by the expected batch size B instead, the sampling rate times the number of records, which is public. And never compose unsampled steps for a long run, because ε grows at least like $\mu_T^2/2 = T/(2\sigma^2)$, the average privacy loss, and reaches the thousands.

C.3. Random batches make each step cheaper

C.3.1. The question

DP-SGD does not use every record in every step. Each step draws a random batch, and with $q = 0.01$ your record is in it only one time in a hundred. How much cheaper is a step that probably did not see you?

C.3.2. The answer

Under **Poisson sampling**, each record enters the batch independently with probability q, the **sampling rate**, by its own coin flip. If a step is ε-DP when it runs on whatever batch it is given, then the step run on a Poisson sample is ε'-DP under add/remove, with

$$\varepsilon' = \ln\left(1 + q(e^{\varepsilon} - 1)\right). \tag{C.8}$$

This is **amplification by subsampling**. When ε is small, $e^{\varepsilon} - 1$ is close to ε, and the logarithm of a number close to 1 is close to that number minus 1. So for small ε

$$\varepsilon' \approx q\varepsilon, \tag{C.9}$$

and the step becomes about $1/q$ times cheaper. When the step also has a slack δ, the same argument shrinks that slack to $q\delta$ (Equation 33.8).

C.3.3. A worked example

Take $q = 0.01$ and a step with $\varepsilon = 1$. Then $e - 1 = 1.71828$, so $1 + q(e^{\varepsilon} - 1) = 1.017183$ and $\varepsilon' = \ln 1.017183 = 0.017037$. The step costs about 0.017 instead of 1, almost $q\varepsilon = 0.01$. Table C.2 shows how the saving shrinks for larger ε.

Table C.2.: Amplification by Poisson sampling at rate $q = 0.01$. For small ε the cost shrinks by about q; for large ε it only drops by about $\ln(1/q) = 4.6$.

Base ε	0.1	1	5	10
Sampled ε', $q = 0.01$	0.0011	0.0170	0.9059	5.3993
$q\varepsilon$	0.0010	0.0100	0.0500	0.1000

C.3.4. Why it works

The idea is that with probability $1 - q$ your record is not used, and then the output has exactly the distribution it would have without you. Only the remaining fraction q of the time can the output differ. So every likelihood ratio becomes a weighted average of 1, with weight $1 - q$, and the old ratio, with weight q.

A tiny case shows it. Suppose a step's output y is three times as likely when your record is used. Under sampling, y's probability with you in the dataset is $(1 - q) \cdot p + q \cdot 3p$, where p is its probability without you. With $q = 0.01$ that is $0.99p + 0.03p = 1.02p$, so the ratio has fallen from 3 to 1.02. Here are the steps in general.

1. **Fix everyone else's coin.** Let U be the set of other records that were drawn. Each record has its own coin, so given U, record x is drawn with probability q, independently of U.

2. **Write both outputs given U.** Let P_0 be the output distribution of the step run on U, and P_1 that of the step run on $U \cup \{x\}$. On D the output is drawn from P_0. On D' it is drawn from P_0 when the coin of x says no, and from P_1 when it says yes. So for every set of outputs S,
$$P_{D'}(S \mid U) = (1-q)P_0(S) + qP_1(S). \tag{C.10}$$
3. **Use the step's own guarantee.** The sets U and $U \cup \{x\}$ are neighbours, and the step is ε-DP, so $P_1(S) \le e^{\varepsilon} P_0(S)$ for every S.
4. **Put it into the mixture.** Replacing $P_1(S)$ by the larger $e^{\varepsilon}P_0(S)$ gives
$$P_{D'}(S \mid U) \le (1-q)P_0(S) + qe^{\varepsilon}P_0(S) = (1 + q(e^{\varepsilon}-1))P_0(S). \tag{C.11}$$
5. **Average over U.** Both datasets draw the other records' coins with the same probabilities. The bound of step 4 holds for every U with the same factor, so it holds for the averages over U too. This is Equation C.8 in the direction "D' against D".
6. **Check the other direction.** DP also needs $P_D(S) \le e^{\varepsilon'} P_{D'}(S)$. The step's guarantee holds with either dataset first, so it also gives $P_1(S) \ge e^{-\varepsilon}P_0(S)$, so the mixture is at least $(1 - q + qe^{-\varepsilon})P_0(S)$. Dividing, $P_0(S) \le P_{D'}(S)/(1 - q + qe^{-\varepsilon})$. To compare the two factors, write $a = e^{\varepsilon}$ and multiply out:
$$(1 - q + q/a)(1 + q(a-1)) = 1 + q(1-q)\frac{(a-1)^2}{a} \ge 1. \tag{C.12}$$
So $1/(1 - q + qe^{-\varepsilon}) \le 1 + q(e^{\varepsilon} - 1)$, and this direction is never the worse one.

C.3.5. Checking it

The script takes a count with Laplace noise, whose density falls like $e^{-\varepsilon|y-\text{count}|}$, with $\varepsilon = 1$. Adding the record moves the count by 1, so the noisy count's density ratio between "used" and "not used" ranges from e^{-1} to e. It computes the mixture's likelihood ratio on a grid of outputs. At $q = 0.01$ the largest log ratio, with the record against without it, is 0.017037, equal to Equation C.8, so the bound is reached and cannot be improved. The other direction's largest log ratio is 0.006341, smaller, as step 6 says. Figure C.2 shows the same at $q = 0.1$, where the effect is easier to see. The ratio without sampling climbs to e, while the mixture's ratio stays below $1 + 0.1(e - 1) = 1.172$.

C.3.6. Recipe and pitfalls

For a step that is ε-DP on its batch, Poisson sampling at rate q makes it $\ln(1 + q(e^{\varepsilon} - 1))$-DP. For small ε this is about $q\varepsilon$. For large ε it is about $\varepsilon - \ln(1/q)$, because when the record is drawn, which happens with probability q, it is fully exposed.

The proof needs Poisson sampling, because step 1 needs an independent coin for your record. A batch of exactly B records drawn at random has no such coin. When your record enters such a batch it pushes another one out, which is a replacement, so the analysis needs the replace-one relation and twice the sensitivity. Shuffling the data and cutting it into batches is worse, because then your record is in exactly one batch per epoch and is never "not used" (Section 33.2).

Finally, do not add up these per-step numbers over a long run. The slacks add up too, so with 10,000 steps each step may only have a tiny share of δ. A Gaussian step with a tiny δ needs a

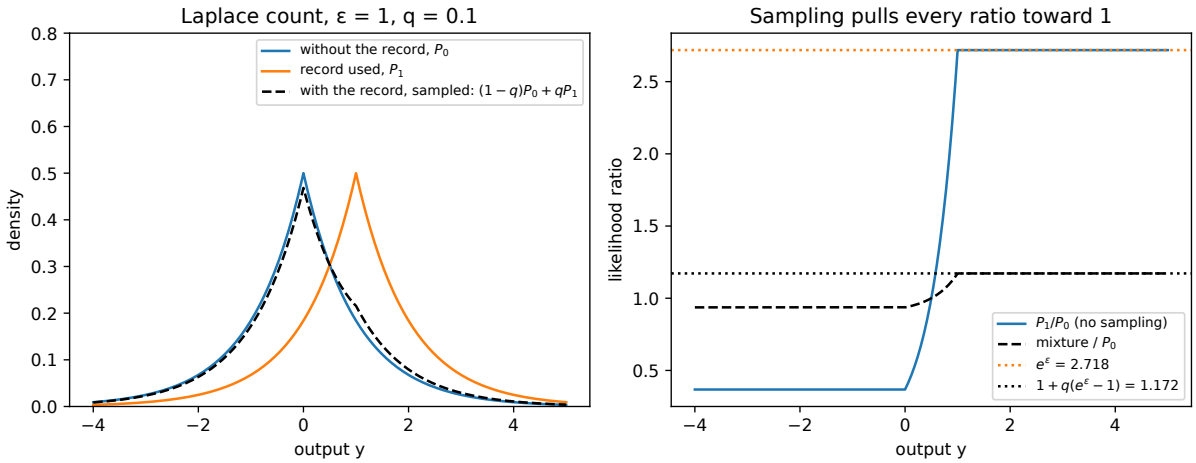


Figure C.2.: Left: a Laplace count with $\varepsilon = 1$ without the record (P_0), with the record used (P_1), and the Poisson-sampled output with the record, the mixture $0.9P_0 + 0.1P_1$ (dashed). Right: the likelihood ratios P_1/P_0 and mixture/P_0 against the output, with the bounds e^ε and $1 + q(e^\varepsilon - 1)$ dotted.

large ε, and there amplification only subtracts $\ln(1/q)$. Section 33.5 shows that the total runs into the thousands, and the next section gives the quantity that composes well.

C.4. The Rényi curve of a sampled Gaussian step

C.4.1. The question

A single (ε, δ) pair says how often the privacy loss is large, but not how large it gets when it is, so pairs add up badly over 10,000 steps. A curve that summarizes the whole distribution of the loss composes by simple addition. What is that curve for one sampled Gaussian step?

C.4.2. The answer

The **Rényi divergence** of order $\alpha > 1$ between two distributions P and Q is a log-average of a power of their likelihood ratio,

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \mathbb{E}_{y\sim Q}\left[\left(\frac{P(y)}{Q(y)}\right)^\alpha\right]. \tag{C.13}$$

A mechanism has **Rényi differential privacy (RDP)** curve $\varepsilon(\alpha)$ when this divergence between its outputs on any two neighbouring datasets is at most $\varepsilon(\alpha)$, for every order α. A larger order weights large ratios more heavily. For the plain Gaussian mechanism with noise multiplier σ the curve is the straight line $\alpha/(2\sigma^2)$ (Section B.4).

For one DP-SGD step, divide the noisy sum by C. Then the output without the record is $P_0 = \mathcal{N}(0, \sigma^2)$, and with the record it is the mixture $P_q = (1-q)P_0 + qP_1$ with $P_1 = \mathcal{N}(1, \sigma^2)$.

This one-dimensional pair is the worst case, as the derivation below explains. Its RDP curve is $\varepsilon_{q,\sigma}(\alpha) = D_\alpha(P_q \| P_0)$, and for integer α it is a finite sum,

$$\varepsilon_{q,\sigma}(\alpha) = \frac{1}{\alpha - 1} \ln A_\alpha, \qquad A_\alpha = \sum_{j=0}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha-j} q^j e^{(j^2-j)/(2\sigma^2)}. \tag{C.14}$$

Here $\binom{\alpha}{j}$ is the binomial coefficient, the number of ways to choose j of α factors. For small q the curve is close to its leading term,

$$\varepsilon_{q,\sigma}(\alpha) \approx \frac{\alpha q^2 (e^{1/\sigma^2} - 1)}{2} \approx \frac{\alpha q^2}{2\sigma^2} \quad \text{for large } \sigma. \tag{C.15}$$

C.4.3. A worked example

Take the running example, $q = 0.01$ and $\sigma = 1$, at order $\alpha = 2$. The sum has three terms, one for each j (Table C.3).

Table C.3.: The three terms of A_2 for $q = 0.01$ and $\sigma = 1$.

j	$\binom{2}{j}(1-q)^{2-j} q^j$	$e^{(j^2-j)/2}$	Term
0	$0.99^2 = 0.9801$	1	0.980100
1	$2 \times 0.99 \times 0.01 = 0.0198$	1	0.019800
2	$0.01^2 = 0.0001$	$e = 2.718$	0.000272

They add up to $A_2 = 1.00017183$, so $\varepsilon_{q,\sigma}(2) = \ln A_2/(2-1) = 1.7181 \times 10^{-4}$. Without sampling the curve at $\alpha = 2$ is $2/(2\sigma^2) = 1$, so sampling made this order almost six thousand times cheaper.

C.4.4. Why it works

First, why one dimension and one shift of exactly 1 is enough. Given which other records were drawn, the step's output with the record is a mixture of "not drawn" and "drawn", as in Section C.3. As in Section C.2, rotating the space leaves one coordinate in which the two components differ, by at most C, and a smaller shift is never worse. Averaging over the other records' coins cannot increase the divergence. The reason is that the average of $(P/Q)^\alpha$ weighted by Q is a convex function of the pair (P, Q), so a mixture of pairs is never further apart than the worst pair in it. Section 33.3 gives these reduction steps in full, so what remains is the pair P_q and P_0. The derivation then takes five steps.

1. **Write the mixture's ratio.** Dividing the mixture by P_0 gives $P_q/P_0 = (1-q) + q\,r$, where $r = P_1/P_0$ is the Gaussian likelihood ratio. The normalizing constants cancel, and

$$r(y) = \frac{e^{-(y-1)^2/(2\sigma^2)}}{e^{-y^2/(2\sigma^2)}} = e^{(2y-1)/(2\sigma^2)}. \tag{C.16}$$

   So by the definition, $A_\alpha = \mathbb{E}_{y \sim P_0}\big[(1 - q + q\,r(y))^\alpha\big]$.

2. **Expand the power.** For a whole number α, the binomial theorem expands the power of a sum of two numbers into $\alpha + 1$ terms. At $\alpha = 2$ it is the familiar $(a + b)^2 = a^2 + 2ab + b^2$. Here the two numbers are $1 - q$ and qr, so

$$(1 - q + qr)^\alpha = \sum_{j=0}^{\alpha} \binom{\alpha}{j} (1 - q)^{\alpha - j} q^j r^j. \tag{C.17}$$

3. **Average term by term.** The average of a sum is the sum of the averages, and $(1-q)^{\alpha-j}q^j$ is a constant. So A_α is the same sum with each r^j replaced by its average $\mathbb{E}[r^j]$ under P_0.
4. **Each average is a Gaussian moment generating function (MGF).** The MGF of a random variable y is the average of $e^{\lambda y}$, as a function of a number λ. For $y \sim \mathcal{N}(0, \sigma^2)$ it is

$$\mathbb{E}[e^{\lambda y}] = e^{\lambda^2 \sigma^2 / 2}. \tag{C.18}$$

   To see why, complete the square in the exponent of the integrand, $\lambda y - y^2/(2\sigma^2) = -(y - \lambda\sigma^2)^2/(2\sigma^2) + \lambda^2\sigma^2/2$. After you take out the constant $e^{\lambda^2\sigma^2/2}$, what is left is a normal density centred at $\lambda\sigma^2$, and it integrates to 1. Now $r^j = e^{-j/(2\sigma^2)} e^{(j/\sigma^2)y}$, so taking $\lambda = j/\sigma^2$ gives

$$\mathbb{E}[r^j] = e^{-j/(2\sigma^2)}\, e^{j^2/(2\sigma^2)} = e^{(j^2 - j)/(2\sigma^2)}. \tag{C.19}$$

   For example, at $j = 2$ and $\sigma = 1$ this is $e^{-1} \cdot e^2 = e$, the factor in the last row of Table C.3.
5. **Assemble.** Putting each average back into the sum gives A_α of Equation C.14, and dividing its logarithm by $\alpha - 1$ gives the curve.

The small-q form comes from looking at which terms matter. Without the exponential factors, the terms add up to $(1 - q + q)^\alpha = 1$. The factors for $j = 0$ and $j = 1$ are $e^0 = 1$, so those two terms cost nothing, and

$$A_\alpha = 1 + \sum_{j=2}^{\alpha} \binom{\alpha}{j} (1 - q)^{\alpha - j} q^j \big(e^{(j^2 - j)/(2\sigma^2)} - 1\big). \tag{C.20}$$

For small q the $j = 2$ term dominates, because every later term carries q^3 or a higher power. That term is about $\frac{\alpha(\alpha-1)}{2} q^2 (e^{1/\sigma^2} - 1)$, so A_α is 1 plus a small number. The logarithm of 1 plus a small number is close to that small number, so dividing by $\alpha - 1$ gives Equation C.15. Its second form follows because $e^{1/\sigma^2} - 1$ is close to $1/\sigma^2$ when σ is large. Both forms are approximations, not bounds. You will also see the curve quoted as $q^2\alpha/\sigma^2$, which is the leading term of the upper bound Abadi et al. (2016) proved for small q, and about twice Equation C.15 for large σ (Section 33.4).

Last, the direction. The guarantee must hold with either dataset first, so the curve is the larger of $D_\alpha(P_q \| P_0)$ and $D_\alpha(P_0 \| P_q)$. The first one puts the dataset with the record first, and it is called the **remove** direction. It is always the larger for the subsampled Gaussian, which is why accountants compute only it. Appendix Section L.1 derives this step by step, and the script checks it below.

C.4.5. Checking it

The curve can be computed in two independent ways, and the script does both: by the binomial sum, and by averaging $(1 - q + qr)^\alpha$ numerically on a fine grid of outputs. At $\alpha = 2$, 8 and 32 the two agree to all seven printed digits: 1.718134×10^{-4}, 8.936439×10^{-4} and 11.24628. A Monte

Carlo average of r^2 over two million normal draws gives 2.7084, against the exact $e = 2.7183$. The other direction is smaller at each of the three orders: 1.6022×10^{-4}, 5.7239×10^{-4} and 1.6923×10^{-3}.

Table C.4 compares the exact curve with its small-q form at the running example's q and σ.

Table C.4.: The RDP curve of one DP-SGD step with $q = 0.01$, $\sigma = 1$, against its small-q form and against no sampling.

α	2	4	8	16	32
Exact Equation C.14	1.7181×10^{-4}	3.6315×10^{-4}	8.9364×10^{-4}	3.0879	11.246
Small-q form Equation C.15	1.7183×10^{-4}	3.4366×10^{-4}	6.8731×10^{-4}	1.3746×10^{-3}	2.7493×10^{-3}
Exact / small-q	1.00	1.06	1.30	2246.32	4090.67
No sampling, $\alpha/(2\sigma^2)$	1	2	4	8	16

Up to $\alpha = 8$ the approximation is within 30%, but then it fails abruptly, and at $\alpha = 16$ it is 2246 times too small. Figure C.3 shows why. The terms with large j grow like $q^j e^{j^2/(2\sigma^2)}$, and the last one overtakes the $j = 2$ term near $\alpha = 2\sigma^2 \ln(1/q)$ (Section 33.4). So the exact curve follows the small-q line, then climbs steeply near $\alpha = 2\sigma^2 \ln(1/q) = 9.2$ and runs toward the unsampled line. At high orders the divergence looks at the rare outputs where the record was clearly used, and there sampling saves only about $\ln(1/q)$. With more noise the approximation holds further: at $\alpha = 8$ the exact curve is 1.019 times the small-q form for $\sigma = 2$ and 1.004 times for $\sigma = 4$.

C.4.6. Recipe and pitfalls

For integer orders, compute Equation C.14 with logarithms, since the terms span many orders of magnitude. For orders between the integers, average $(1 - q + qr)^\alpha$ numerically, as the chapters' accountants do. Use Equation C.15 only for estimates at orders below $2\sigma^2 \ln(1/q)$, never as the curve itself.

The curve assumes Poisson sampling and the add/remove relation, like Section C.3. It also assumes the sensitivity is exactly the clipping norm. If something in your code makes the sensitivity $2C$, such as microbatches or the replace-one relation, give the accountant the noise multiplier $\sigma/2$ instead of σ, because the ratio is then $2C/(\sigma C) = 1/(\sigma/2)$.

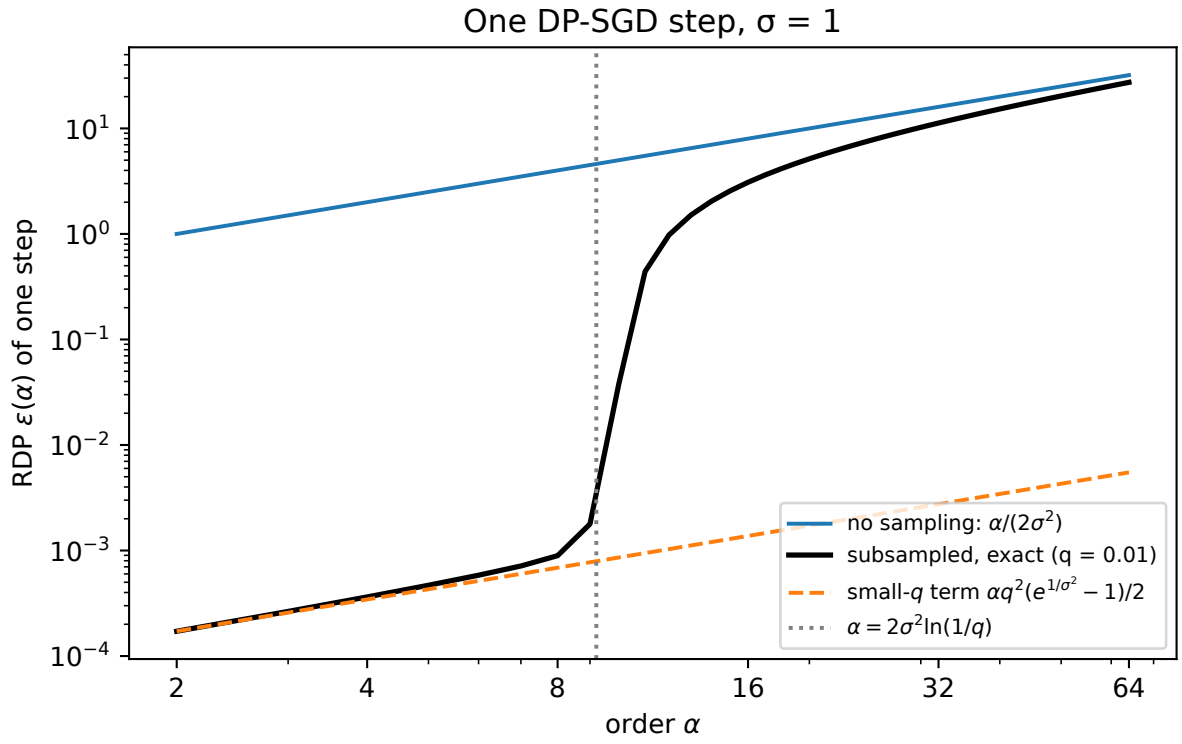


Figure C.3.: The RDP curve of one DP-SGD step against the order α, on log scales, for $q = 0.01$ and $\sigma = 1$: exact (black), the small-q term $\alpha q^2(e^{1/\sigma^2} - 1)/2$ (dashed) and no sampling, $\alpha/(2\sigma^2)$. The dotted line marks $\alpha = 2\sigma^2 \ln(1/q)$, where the small-$q$ form breaks.

C.5. From batch, noise and epochs to the final ε

C.5.1. The question

You train with n records, an expected batch of B, noise multiplier σ and a number of epochs E. What (ε, δ) do you report? And the reverse question, which a practitioner asks more often: for a target ε, how much noise do you need?

C.5.2. The answer

The sampling rate is $q = B/n$, and one epoch is $n/B = 1/q$ steps, so there are $T = E\,n/B$ steps in all. The RDP accountant multiplies the per-step curve by T and converts it to (ε, δ) at the best order, using Mironov's conversion:

$$\varepsilon = \min_{\alpha} \Big[T\,\varepsilon_{q,\sigma}(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1} \Big]. \tag{C.21}$$

The second term is the price of converting, which this page calls the conversion **toll**. The minimum runs over a grid of orders. The book's grid has the orders from 1.1 to 10.9 in steps of 0.1, followed by whole orders up to 512. It takes $\varepsilon_{q,\sigma}(\alpha)$ from Equation C.14 at whole orders and from the numerical average between them.

C.5.3. A worked example

The running example has $n = 10{,}000$ records and $B = 100$, so the sampling rate is $q = 0.01$. Each of its 100 epochs has 100 steps, so $T = 10{,}000$. The script finds that the best order is $\alpha = 4.3$, where one step costs $\varepsilon_{q,\sigma}(4.3) = 3.9400 \times 10^{-4}$. Multiplying by the 10,000 steps gives a composed curve of 3.9400 at that order. The conversion then adds $\ln(10^5)/(4.3 - 1) = 11.5129/3.3 = 3.4888$. So $\varepsilon = 3.9400 + 3.4888 = 7.4287$, which you would report as $(7.43, 10^{-5})$. Table C.5 gives ε after each number of epochs.

Table C.5.: The ε at $\delta = 10^{-5}$ of the running example ($q = 0.01$, $\sigma = 1$) after each number of epochs. "Every record in every step" composes unsampled Gaussian steps exactly (Section C.2); the RDP columns use Mironov's conversion, Equation C.21, and the improved conversion of Balle et al. (2020).

Epochs	Steps T	Every record in every step	RDP, Mironov	Best order for Mironov	RDP, improved
1	100	91.8	1.61	8.9	1.21
5	500	344.5	2.07	8.3	1.65
10	1,000	633.9	2.54	7.9	2.10
20	2,000	1189.8	3.34	7.2	2.87
50	5,000	2800.6	5.18	5.5	4.59
100	10,000	5425.5	7.43	4.3	6.71

Sampling turns thousands into single digits. The **improved conversion** of From Rényi DP to (ε, δ) subtracts two small corrections from the conversion term $\ln(1/\delta)/(\alpha - 1)$ and gives 6.71 at 100

epochs, the number the chapters report. A **privacy loss distribution (PLD)** accountant tracks the whole distribution of the loss instead of one curve, and Section 34.6 reports 6.2377 for the same run as its upper bound.

C.5.4. Why it works

1. **The curves add.** By the definition, the Rényi divergence of the whole run is set by the average of the α-th power of its likelihood ratio, and step 2 below writes that as the average of $e^{(\alpha-1)L}$ for the privacy loss L. The run's likelihood ratio is the product of the steps' ratios, so $e^{(\alpha-1)L}$ is a product of per-step factors. Average this product one step at a time, starting from the last. With everything before it held fixed, the last step is a subsampled Gaussian step, so its factor averages to at most $e^{(\alpha-1)\varepsilon_{q,\sigma}(\alpha)}$, whatever the earlier steps did. That number is a constant, so it comes out of the average, and the next step is treated the same way. After T steps the average is at most $e^{(\alpha-1)T\varepsilon_{q,\sigma}(\alpha)}$, so the composed curve is $T\varepsilon_{q,\sigma}(\alpha)$, even though each step depends on the earlier ones (Section B.4).
2. **The average of a power is the Rényi divergence.** Let L be the privacy loss, $\ln(P(y)/Q(y))$ with y drawn from P, where P is the output distribution on the dataset with the record. Then
$$\mathbb{E}_{y\sim P}\big[e^{(\alpha-1)L}\big] = \int P\Big(\frac{P}{Q}\Big)^{\alpha-1}\mathrm{d}y = \int Q\Big(\frac{P}{Q}\Big)^{\alpha}\mathrm{d}y = e^{(\alpha-1)D_\alpha(P\|Q)}. \tag{C.22}$$
3. **Markov's inequality bounds the chance of a large loss.** For a variable that cannot be negative, the chance that it is at least a is at most its average divided by a. Apply it to $e^{(\alpha-1)L}$ at the level $a = e^{(\alpha-1)\varepsilon}$. With step 2 and the composed curve of step 1, this gives
$$\Pr[L > \varepsilon] \le e^{(\alpha-1)(T\varepsilon_{q,\sigma}(\alpha)-\varepsilon)}. \tag{C.23}$$
4. **A rare large loss is all that δ has to cover.** Split any set of outputs S into the outputs where the loss is at most ε and those where it is larger. On the first part, P is at most e^{ε} times Q, by the definition of L. The second part has probability at most the chance of a loss above ε. Adding the two parts,
$$P(S) \le e^{\varepsilon}Q(S) + \Pr[L > \varepsilon], \tag{C.24}$$
so the run is (ε,δ)-DP as soon as the chance of a loss above ε is at most δ.
5. **Solve for ε.** Set the bound of step 3 equal to δ, take logarithms and divide by $\alpha-1$. This gives
$$\varepsilon = T\varepsilon_{q,\sigma}(\alpha) + \frac{\ln(1/\delta)}{\alpha-1}. \tag{C.25}$$
It holds at every order, so the smallest value over the orders also holds, which is Equation C.21. Appendix Section D.3 derives the conversion, and its improved form, in more detail.

The minimum sits between the extremes because the two terms pull in opposite directions. Figure C.4 shows them for the running example. The composed curve $T\varepsilon_{q,\sigma}(\alpha)$ rises with the order, slowly at first and then steeply past $\alpha \approx 8$, where the curve leaves its small-q form. The conversion toll $\ln(1/\delta)/(\alpha-1)$ falls. Their sum is smallest at $\alpha = 4.3$, where it is 7.43. A longer run makes the first term larger and pushes the best order down, from 8.9 after one epoch to 4.3 after 100.

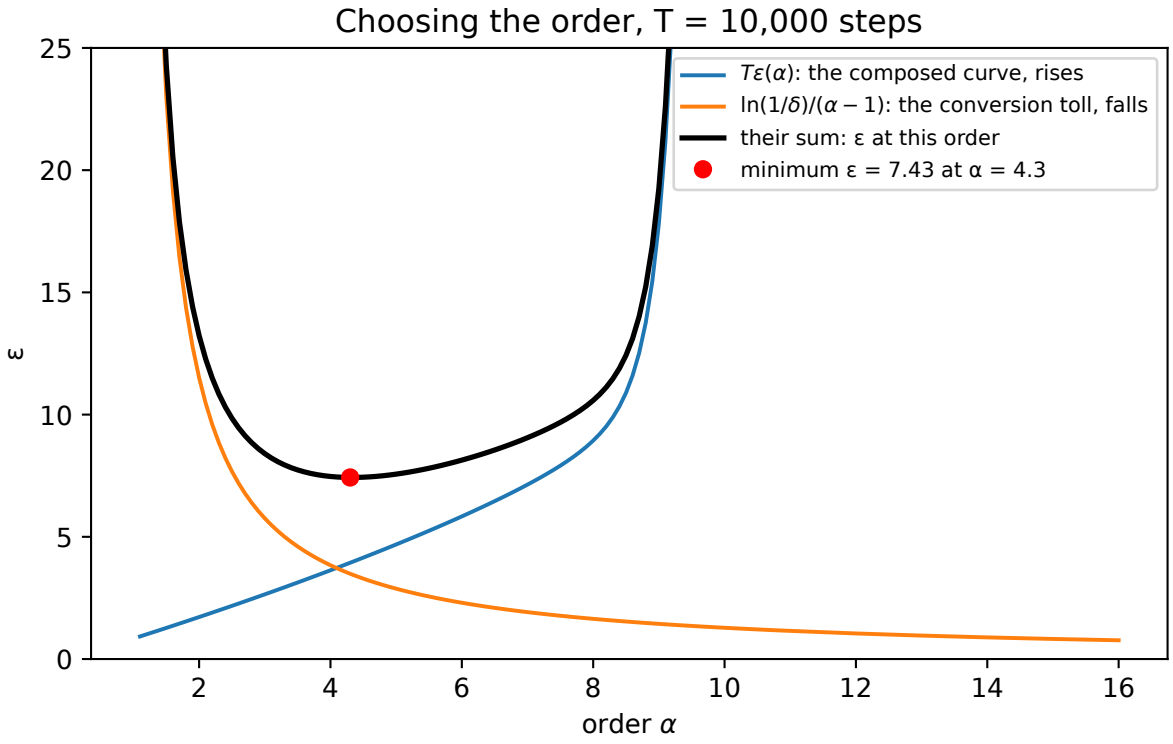


Figure C.4.: The two terms of Equation C.21 for the running example after $T = 10{,}000$ steps, against the order α: the composed curve $T\varepsilon(\alpha)$, the conversion toll $\ln(1/\delta)/(\alpha - 1)$, and their sum, with its minimum, $\varepsilon = 7.43$ at $\alpha = 4.3$, marked.

C.5.5. Checking it

The script's accountant reproduces the chapters' numbers to every printed digit: 7.4287 with Mironov's conversion and 6.71 with the improved one, as in Section 33.5. Restricting the grid to whole orders, where Equation C.14 is exact, costs little, giving 7.4692 at $\alpha = 4$. Figure C.5 plots ε against the number of epochs. Without sampling it grows nearly in proportion to the number of steps, while the RDP accountant grows slowly. The PLD upper bounds from Section 34.6, marked as crosses, sit below the RDP curves. A PLD bound is proved for every pair of neighbouring datasets, like the RDP bound, and it is the tightest such number for this run. Both are therefore valid guarantees, and with the improved conversion RDP overstates ε by about 8% at 100 epochs, since $6.71/6.24 = 1.08$.

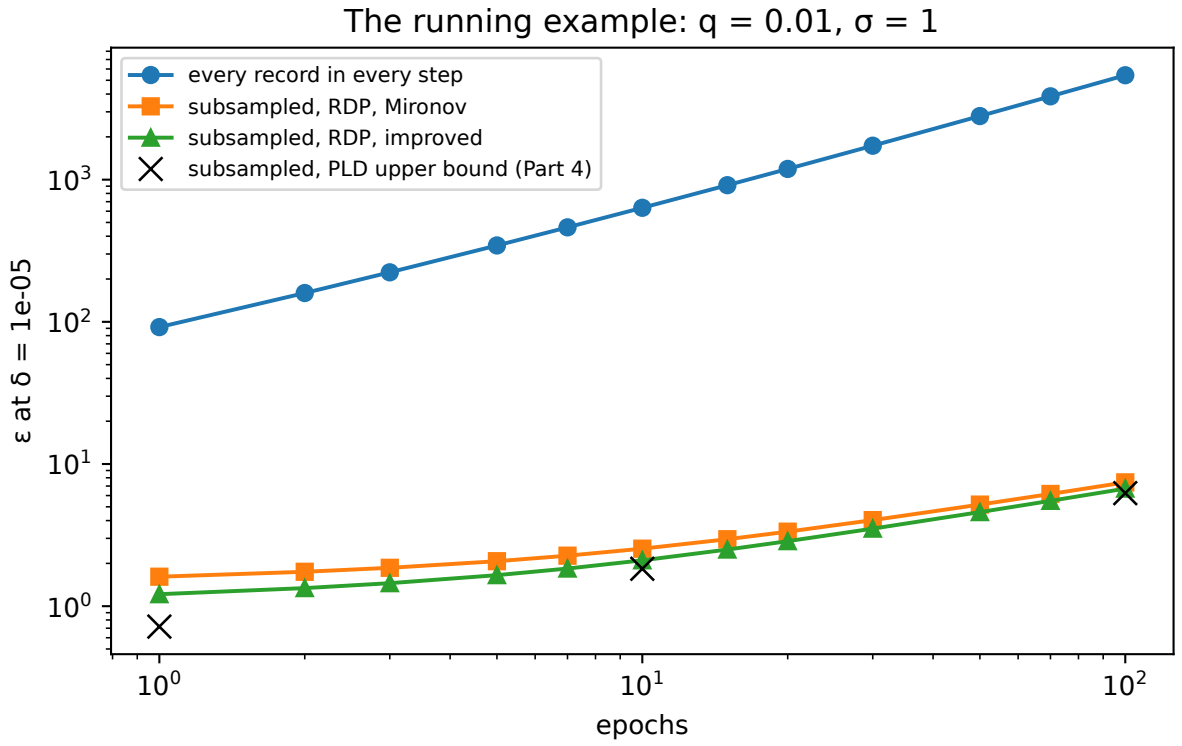


Figure C.5.: The ε at $\delta = 10^{-5}$ of the running example against the number of epochs, on log scales: every record in every step (exact Gaussian composition), the subsampled run with the RDP accountant under Mironov's and the improved conversion, and the PLD upper bound that Part 4's numerical accountant reports (crosses).

C.5.6. Recipe and pitfalls

To get ε from a training configuration, follow these steps.

1. Compute the sampling rate q, which is B/n, and the number of steps $T = E\,n/B$.
2. Compute the per-step curve $\varepsilon_{q,\sigma}(\alpha)$ on a grid of orders, with Equation C.14 at whole orders.
3. Multiply the curve by T.

4. At each order add the toll $\ln(1/\delta)/(\alpha - 1)$, or use the improved conversion, and report the smallest value with its δ.

To go the other way, from a target ε to a noise multiplier, search over σ. More noise never makes a step less private, so ε falls as σ grows, and bisection finds the smallest σ that meets the target. Table C.6 shows the result for the running example's q and T. The improved column matches Table 39.2 of the DP-SGD chapter.

Table C.6.: The noise multiplier needed for each target ε with $q = 0.01$ and $T = 10{,}000$ steps, found by bisection on σ.

Target ε at $\delta = 10^{-5}$	1	2	4	8
σ, Mironov's conversion	4.974	2.617	1.484	0.962
σ, improved conversion	4.126	2.278	1.358	0.917

Most mistakes in reported numbers come from a mismatch between the accountant and the code, not from the accountant itself. Five are common.

- **Sampling must be Poisson, or the analysis must match the sampler.** The whole saving rests on the independent coin of Section C.3. If the code draws batches of exactly B records, a new record pushes another one out, so the sum has sensitivity $2C$ and the accountant needs $\sigma/2$. For the running example that gives $\varepsilon = 47.42$ instead of 6.71, with the improved conversion. Shuffling and cutting into batches has no simple amplification at all (Section 33.2).
- **State the neighbouring relation.** The numbers here are for adding or removing one record. A replace-one guarantee is a different statement and needs twice the sensitivity.
- **The clipping norm is not a privacy parameter.** Changing C changes accuracy, not ε, because the noise scales with it. A paper that reports C in place of σ, q and T has not said how private the model is.
- **Report ε with δ.** The same run is $(5.89, 10^{-3})$, $(7.43, 10^{-5})$ and $(8.71, 10^{-7})$ under Mironov's conversion. An ε without its δ means nothing, and δ should be well below $1/n = 10^{-4}$. A mechanism that published one random record in full would satisfy $\delta = 1/n$, so a larger δ allows a real leak.
- **Count every run, including tuning.** Each training run that looks at the private data spends budget, including the runs you discard. Choosing the best of several runs by a score computed on private data leaks more than one run does (Section 40.1, and Section M.3 for the derivation).

C.6. Code

The script `apx_core_dp_sgd.py` computes every number on this page and draws its figures. It rewrites the accountant in a few dozen lines rather than importing the chapters' code, so it is an independent check. It clips random gradients to test the sensitivity, solves the exact Gaussian curve with logarithms, checks amplification on a Laplace count, computes the subsampled Gaussian's curve by the binomial sum and by numerical averaging, and converts it to ε and back to σ. From the repository root, run:

```
uv run python code/appendix-derivations/apx_core_dp_sgd.py
```

The script is code/appendix-derivations/apx_core_dp_sgd.py, and tests/test_appendix_core_dp_sgd.py checks that its output still matches this page. Table C.7 summarizes the output with the default seed.

Table C.7.: The script's output with the default seed.

Check	Result
Clipped sum, 20,000 random datasets	Largest change: $1.0000\,C$ (add/remove), $2.0000\,C$ (replace-one)
Unsampled Gaussian steps, $\sigma = 1$	$\varepsilon = 4.38$, 91.82 and 5425.51 after 1, 100 and 10,000 steps
Full batch, $\sigma = 10.8$, 100 steps	$\varepsilon = 4.005$, the chapter's target of 4
Amplification, Laplace count, $\varepsilon = 1$, $q = 0.01$	Largest log ratio 0.017037, equal to the formula; other direction 0.006341
Subsampled Gaussian, $q = 0.01$, $\sigma = 1$	Binomial sum and numerical average agree to seven digits; remove direction larger
Small-q form	Within 30% up to $\alpha = 8$; 2246 times too small at $\alpha = 16$
Running example, 100 epochs	$\varepsilon = 7.43$ (Mironov, $\alpha = 4.3$), 6.71 (improved)
Noise for $\varepsilon = 1$	$\sigma = 4.974$ (Mironov), 4.126 (improved)
Fixed-size batches	$\varepsilon = 47.42$ against 6.71 for Poisson

The main thing to take from it is how much each idea is worth on the running example. Clipping and the noise make one step cost $\varepsilon = 4.38$, so 10,000 steps without sampling cost 5425.51. Sampling, measured by the Rényi curve and converted once, brings the same run down to 7.43, and the improved conversion to 6.71. Two independent ways of computing the curve agree to seven digits, so the remaining gap to the PLD accountant comes from the conversion and from keeping only a curve, not from numerical error.

C.7. References

- Abadi, Chu, Goodfellow, McMahan, Mironov, Talwar & Zhang, "Deep Learning with Differential Privacy" (CCS 2016). DP-SGD with per-example clipping, and the moments accountant, which is the RDP accountant of Section C.5 in another notation.
- Mironov, "Rényi Differential Privacy" (CSF 2017). The definition of RDP, its composition and the conversion of Equation C.21.
- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (arXiv 2019). The reduction to one dimension, the dominance of the remove direction and the binomial sum of Equation C.14.
- Balle, Barthe & Gaboardi, "Privacy Amplification by Subsampling: Tight Analyses via Couplings and Divergences" (NeurIPS 2018). The tight amplification of Equation C.8, with δ, and the other sampling schemes.

- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020). The improved conversion behind the 6.71.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023). What to clip, how to choose the knobs and what to report, for practitioners.

D. What ε means, and converting between definitions

A **mechanism** M is a randomized algorithm: it takes a dataset and returns a random output. Two datasets D and D' are **neighbours**, written $D \sim D'$, when one is the other with one person's record added or removed. The definition of privacy has two parameters, a multiplicative limit ε and a small additive slack δ. The mechanism is (ε, δ)**-differentially private** when, for every pair of neighbours and every set S of outputs,

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{D.1}$$

The **privacy loss** of an output y is the log of how much more likely y is when the data are D than when they are D'. Write $p(y)$ and $q(y)$ for the probability (or density) of y under $M(D)$ and $M(D')$. Then the privacy loss L is

$$L(y) = \ln \frac{p(y)}{q(y)}. \tag{D.2}$$

Pure differential privacy (DP), the case $\delta = 0$, is written ε-DP. It is the same as asking that $|L(y)| \leq \varepsilon$ for every output. One direction holds because a single output is itself a set of outputs, and the other because a set's probability is the sum over its outputs. The absolute value appears because neighbours come in both orders, so the definition also holds with D and D' swapped. Section A.1 builds these definitions slowly, with examples.

This appendix answers two practical questions. The first is what a number like $\varepsilon = 1$ promises. The first two sections read the definition as a limit on any test an attacker can run, and as a limit on how far an attacker's belief can move. The second question is how to report (ε, δ) when your accountant tracks something else. An **accountant** is the code that adds up the privacy cost of many releases from the same data, which is called composition. Accountants track Rényi differential privacy (RDP), zero-concentrated differential privacy (zCDP) or Gaussian differential privacy (GDP), because those compose more simply, and the last three sections convert each of them to (ε, δ). Table D.1 lists the five results and the chapter sections that state or use them.

Table D.1.: The results of this appendix and the chapter sections that state or use them.

Section	What it derives	Stated or used in
Section D.1	Every test has $\text{FPR} + e^{\varepsilon}\,\text{FNR} \geq 1 - \delta$	Section 18.5, Section 18.6, Section 32.1
Section D.2	Posterior odds are at most e^{ε} times the prior odds	Section 18.7
Section D.3	RDP gives $\varepsilon = \varepsilon(\alpha) + \ln(1/\delta)/(\alpha - 1)$	Section 29.6, Section 30.1, Section 30.2

Section	What it derives	Stated or used in
Section D.4	ρ-zCDP gives $\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}$	Section 31.1, Section 31.3
Section D.5	μ-GDP: the curve G_μ and its exact (ε, δ)	Section 32.2, Section 32.3, Section 32.4, Section 20.4

A running example. Most sections below use one mechanism, the Gaussian mechanism. It adds normal noise to a query, and the noise has standard deviation σ. Here the query's **sensitivity** Δ is the most its value can move between neighbours, measured as a Euclidean distance when the query returns a vector.

Dividing the output by σ, and then subtracting the query's value on D, changes nothing about privacy, because you can undo both steps. After them, the output on D is a unit normal centred at 0, and the output on D' is a unit normal centred a distance $\mu = \Delta/\sigma$ away, for the neighbours whose query values are the full sensitivity apart. A smaller shift is harder to detect, so this pair is the worst one (Section D.5 shows why). So the two output distributions are

$$P = \mathcal{N}(0, 1) \text{ on } D, \qquad Q = \mathcal{N}(\mu, 1) \text{ on } D'. \tag{D.3}$$

Taking the log of the ratio of the two normal densities, the y^2 terms cancel and the privacy loss is a straight line in the output,

$$L(y) = -\frac{y^2}{2} + \frac{(y-\mu)^2}{2} = \frac{\mu^2}{2} - \mu y. \tag{D.4}$$

When y is drawn from the distribution P, so that y is standard normal, the loss is a shifted and scaled normal variable, which is again normal. It is therefore normal with mean $\mu^2/2$ and variance μ^2. Throughout, the standard normal distribution function Φ gives the probability that a standard normal variable Z is at most x,

$$\Phi(x) = \Pr[Z \le x], \tag{D.5}$$

and Φ^{-1} is its inverse. Unless a section says otherwise, the numbers use $\delta = 10^{-5}$, and logarithms are natural.

D.1. What ε means for any test

D.1.1. The question

An attacker sees the output and must guess whether the data were D or D', for example whether your record was used. It may know everything else and compute anything. What does $\varepsilon = 1$ stop it from doing?

D.1.2. The answer

Any guessing rule can be written as a set S of outputs: the attacker says "D'" when the output lands in S, and "D" otherwise. It can be wrong in two ways. Its **false positive rate (FPR)** is how often it says "D'" when the data were D, so $\text{FPR} = \Pr[M(D) \in S]$. Its **false negative rate (FNR)** is how often it says "D" when the data were D', so $\text{FNR} = \Pr[M(D') \notin S]$. If M is (ε, δ)-DP, then every test, against every pair of neighbours, satisfies both

$$\text{FPR} + e^{\varepsilon}\,\text{FNR} \geq 1 - \delta \qquad \text{and} \qquad e^{\varepsilon}\,\text{FPR} + \text{FNR} \geq 1 - \delta. \tag{D.6}$$

The chapter on the definition (Section 18.5) writes α and β for the two error rates. This page needs α for a Rényi order, so it writes them FPR and FNR. There are two inequalities because the definition holds in both orders of the pair.

D.1.3. A worked example

Take $\varepsilon = 1$ and $\delta = 0$, and an attacker who accepts 5% false positives. The first inequality says $0.05 + e\,\text{FNR} \geq 1$, so $\text{FNR} \geq 0.95/e = 0.3495$. The second says $e \times 0.05 + \text{FNR} \geq 1$, so $\text{FNR} \geq 1 - 0.1359 = 0.8641$. Both must hold, so the attacker misses at least **86.4%** of the time, and it catches D' at most 13.6% of the time.

Notice which inequality did the work. The first one alone would allow a miss rate as low as 35%. At a low FPR the second one is much stronger. Low FPR is exactly where attackers want to operate, because an attack that rarely raises a false alarm is one whose accusations can be trusted. For comparison, a coin that says "D'" 5% of the time, ignoring the output, catches D' 5% of the time. So $\varepsilon = 1$ lets the best attacker catch D' at most $0.136/0.05 = e \approx 2.7$ times as often as that coin.

Other false positive rates give other limits. At $\text{FPR} = 0.01$ the attacker misses at least 0.9728 of the time, and at $\text{FPR} = 0.25$ at least 0.3204. With $\delta = 10^{-5}$ the limit at $\text{FPR} = 0.05$ drops only to 0.864076, because δ lowers each of the two bounds by exactly δ.

D.1.4. Why it works

Each inequality is the definition applied to one set. The proof also uses one rule twice: the probability that the output is outside S is one minus the probability that it is inside, so $\Pr[M(D) \notin S] = 1 - \text{FPR}$ and $\Pr[M(D') \in S] = 1 - \text{FNR}$.

1. **The first inequality.** Apply the definition to the set of outputs where the test says "D", the complement of S. It gives $\Pr[M(D) \notin S] \leq e^{\varepsilon}\Pr[M(D') \notin S] + \delta$. The left side is $1 - \text{FPR}$ and the probability on the right is FNR. So $1 - \text{FPR} \leq e^{\varepsilon}\,\text{FNR} + \delta$, which rearranges to the first inequality.
2. **The second inequality.** If D and D' are neighbours, so are D' and D, so the definition also holds with the two datasets swapped. Apply it in that order to S itself: $\Pr[M(D') \in S] \leq e^{\varepsilon}\Pr[M(D) \in S] + \delta$. The left side is $1 - \text{FNR}$ and the probability on the right is FPR, which gives the second inequality.

3. **Tests that flip a coin.** An attacker may also randomize, saying "D'" with some probability that depends on the output. In that case, think of its coin U as part of the output. Releasing $(M(D), U)$, with U independent of the data, is still (ε, δ)-DP, because for each value of U the inequality holds for the matching slice of the set, and averaging both sides over U keeps it. So steps 1 and 2 apply to randomized tests too.

A tiny case shows that the bound is reached. In randomized response, one person reports a bit: the true bit with probability $e^\varepsilon/(1+e^\varepsilon)$, the flipped bit otherwise. Each report is exactly e^ε times as likely under one value of the bit as under the other, so the mechanism is ε-DP. Let the bit be 1 under D and 0 under D', and let the test say "D'" when the report is 0. It errs when the report is flipped, in either world, so $\mathrm{FPR} = \mathrm{FNR} = 1/(1+e^\varepsilon)$, which is 0.2689 at $\varepsilon = 1$. Then

$$\mathrm{FPR} + e^\varepsilon \,\mathrm{FNR} = \frac{1+e^\varepsilon}{1+e^\varepsilon} = 1, \tag{D.7}$$

so the first inequality holds with equality, and by symmetry so does the second. So randomized response meets both bounds at once, and neither inequality can be strengthened if it is to hold for every ε-DP mechanism.

D.1.5. Checking it

The script for this page (see Section D.7) lists every possible test against three people who each run randomized response with $\varepsilon = \ln 3$, for neighbours that differ in the first person's bit. There are 8 report vectors, and a test is any subset of them, so there are $2^8 = 256$ tests. The smallest slack, the amount by which a test clears the tighter of the two inequalities, is exactly 0: no test breaks the bound, and several tests meet it.

Figure D.1 draws the forbidden region, with the FPR across and the FNR up. Each pair of lines is the lower edge of where a test can be, and a smaller ε pushes that edge up towards the diagonal $\mathrm{FPR} + \mathrm{FNR} = 1$, which is what a test that ignores the output achieves. The dots are randomized response, which sits on the corner of each region.

D.1.6. Recipe and pitfalls

To turn a guarantee into an attack limit, pick the false positive rate you care about. The attacker's true positive rate (TPR) is the fraction of the time it correctly says "D'", so $\mathrm{TPR} = 1 - \mathrm{FNR}$. The second inequality caps it at

$$\mathrm{TPR} \le e^\varepsilon \,\mathrm{FPR} + \delta. \tag{D.8}$$

Auditing, which tests a privacy claim by attacking the system, runs this in reverse. If an attack measures a TPR and an FPR against a trained model, the mechanism's ε must be at least $\ln\left((\mathrm{TPR} - \delta)/\mathrm{FPR}\right)$.

Three things go wrong in practice. The bound is for the worst mechanism with this ε, and most mechanisms sit strictly inside it, so a weak attack says little about how private the mechanism really is. At very small FPR the δ term dominates, since at $\mathrm{FPR} = 0$ the bound still allows a TPR of δ, so δ must be tiny. And the guarantee covers only the mechanism's own kind of neighbours. A guarantee for neighbours that add or remove one record says less about neighbours that replace one record with another. Section 18.5 proves the converse, that the two inequalities for all tests are the same as (ε, δ)-DP, and Section 18.6 turns them into a bound on membership advantage.

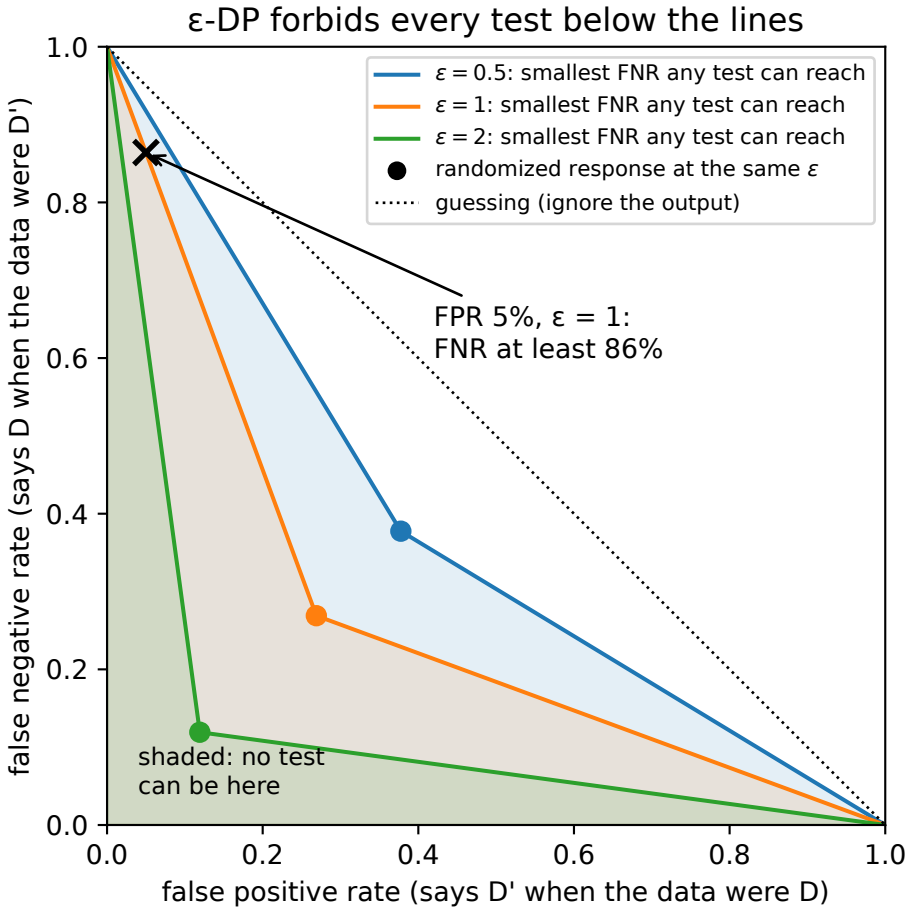


Figure D.1.: The false positive and false negative rates that tests can reach under ε-DP with $\delta = 0$, for $\varepsilon = 0.5$, 1 and 2. The shaded area below each pair of lines is forbidden for every test. The dots mark randomized response at the same ε, the cross marks the worked example at 5% false positives, and the dotted diagonal is guessing without looking at the output.

D.2. What ε means for an attacker's belief

D.2.1. The question

People often ask a different question: "after the release, how sure can someone be about my record?" Take the strongest attacker. It knows every record except yours, so it knows the data are one of two neighbours, D or D', which differ only in your record. Before the release it believes D with some prior probability π. How large can its belief be afterwards?

D.2.2. The answer

The **odds** of D are its probability divided by the probability of D', so the prior odds are $\pi/(1-\pi)$. If M is ε-DP, then after any output y the posterior odds are within a factor e^{ε} of the prior odds, and the posterior probability is capped:

$$\frac{\Pr[D \mid y]}{\Pr[D' \mid y]} \le e^{\varepsilon}\,\frac{\pi}{1-\pi}, \qquad \Pr[D \mid y] \le \frac{e^{\varepsilon}\pi}{e^{\varepsilon}\pi + 1 - \pi}. \tag{D.9}$$

The same argument with $e^{-\varepsilon}$ gives the matching lower limits.

D.2.3. A worked example

Take $\varepsilon = 1$ and a prior of 1%, so the prior odds are $0.01/0.99 = 0.010101$. The output can multiply them by at most e, which brings them to 0.027457. Any odds o turn back into a probability as $o/(1+o)$, so the posterior is at most $0.027457/1.027457 = 0.0267$. In words, a suspicion of 1% can rise to at most **2.7%**. With a prior of 50% the prior odds are 1, so the posterior odds are at most e and the posterior is at most $e/(1+e) = 0.7311$, or **73%**.

Table D.2 gives the cap for other values. Small priors stay small unless ε is large: at $\varepsilon = 4$ a prior of 1% can rise to 36%.

Table D.2.: The largest posterior probability of D after one output of an ε-DP mechanism, from Equation D.9.

ε	Prior 0.01	Prior 0.5
0.5	0.0164	0.6225
1	0.0267	0.7311
2	0.0695	0.8808
4	0.3555	0.9820

D.2.4. Why it works

The argument is Bayes' rule written in odds. A table with the two hypotheses as rows shows it (Table D.3). Each row multiplies the prior by the chance of seeing y under that hypothesis, and the posterior divides each product by their sum c.

Table D.3.: Bayes' rule for two hypotheses, where $c = \pi\, p(y) + (1-\pi)\, q(y)$ is the total chance of seeing y.

Hypothesis	Prior	Chance of y	Prior × chance	Posterior
D	π	$p(y)$	$\pi\, p(y)$	$\pi\, p(y)/c$
D'	$1-\pi$	$q(y)$	$(1-\pi)\, q(y)$	$(1-\pi)\, q(y)/c$

1. **Divide the two posteriors.** The sum c is the same in both rows, so it cancels in the ratio. What is left is the prior odds times the ratio of the chances, which is $e^{L(y)}$:

$$\frac{\Pr[D \mid y]}{\Pr[D' \mid y]} = \frac{\pi}{1-\pi} \cdot \frac{p(y)}{q(y)} = \frac{\pi}{1-\pi}\, e^{L(y)}. \tag{D.10}$$

2. **Use pure DP.** Under ε-DP, $L(y) \le \varepsilon$ for every output, so $e^{L(y)} \le e^{\varepsilon}$. This gives the odds bound of Equation D.9.
3. **Turn odds into a probability.** The map $o \mapsto o/(1+o)$ grows with o, so the largest odds give the largest probability. Put in $o = e^{\varepsilon}\pi/(1-\pi)$ and multiply top and bottom by $1-\pi$ to get the probability bound.

With numbers, take randomized response at $\varepsilon = 1$, your bit 1 under D, and a report of 1. Then $p(y) = e/(1+e) = 0.7311$ and $q(y) = 0.2689$. With a prior of 1%, the products are $0.01 \times 0.7311 = 0.007311$ and $0.99 \times 0.2689 = 0.266252$. Their sum is $c = 0.273563$, so the posterior is $0.007311/0.273563 = 0.0267$, exactly the cap.

With δ. Under (ε, δ)-DP the privacy loss is not bounded everywhere, so the bound cannot hold for every output. It is tempting to say that it holds except on outputs of total probability δ, but that is false in general. For example, Section D.5 shows that the Gaussian running example with $\mu = 1$ is $(4.3772, 10^{-5})$-DP. Yet, when the data are D, its privacy loss exceeds 4.3772 with probability 5.284×10^{-5}, more than five times δ.

What does hold is the same bound at a larger level $\varepsilon' > \varepsilon$, except on a set of outputs whose probability is a small multiple of δ. To prove it, let B be the set of outputs whose loss exceeds ε', and write $\Pr_P[B]$ and $\Pr_Q[B]$ for its probability under D and D'.

1. The definition, applied to B, says $\Pr_P[B] - e^{\varepsilon} \Pr_Q[B] \le \delta$.
2. On B each output has $q(y) < e^{-\varepsilon'} p(y)$, because its loss exceeds ε'. Adding over B gives $\Pr_Q[B] < e^{-\varepsilon'} \Pr_P[B]$.
3. Put step 2 into step 1. Since $\Pr_Q[B]$ is less than $e^{-\varepsilon'} \Pr_P[B]$, replacing it by $e^{-\varepsilon'} \Pr_P[B]$ makes the left side smaller, and the left side was already at most δ. So $\Pr_P[B]\,(1 - e^{\varepsilon - \varepsilon'}) < \delta$, and therefore

$$\Pr_P[B] < \frac{\delta}{1 - e^{\varepsilon - \varepsilon'}}. \tag{D.11}$$

Outside B the loss is at most ε', so steps 1 to 3 of the pure case go through with ε' in place of ε. So the posterior bound holds at level ε', except on outputs whose probability under D is at most $\delta/(1 - e^{\varepsilon - \varepsilon'})$. With $\varepsilon' = \varepsilon + 1$ that exception is $\delta/(1 - e^{-1}) = 1.582 \times 10^{-5}$. For the Gaussian example the true probability of a loss above $4.3772 + 1$ is 5.381×10^{-7}, well inside the bound.

D.2.5. Checking it

The script draws 2,000,000 people whose bit is 1 with probability 1% and runs randomized response at $\varepsilon = 1$. Among the reports of 1, the fraction that came from a true 1 is 0.0267, the cap of Equation D.9. Randomized response reaches the cap, so the cap cannot be lowered.

Figure D.2 shows the mechanism of the proof. In log odds, the output adds $L(y)$ to the prior's log odds, so ε-DP keeps the posterior inside a band of half-width ε around the diagonal. In probabilities the same band bends. The reason is that turning log odds back into a probability squeezes large log odds towards 1 and very negative ones towards 0, while leaving the middle stretched.

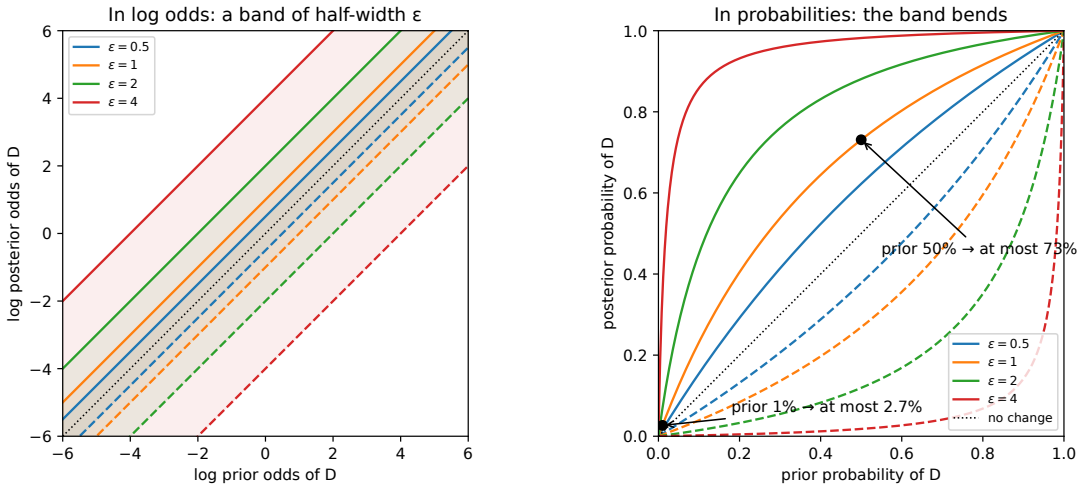


Figure D.2.: The largest (solid) and smallest (dashed) posterior of D after one output of an ε-DP mechanism, for $\varepsilon = 0.5$, 1, 2 and 4. Left: on the scale of log odds the posterior stays within a band of half-width ε around the prior (dotted diagonal). Right: the same band on the scale of probabilities, with the two worked examples at $\varepsilon = 1$ marked.

D.2.6. Recipe and pitfalls

To explain ε to someone who thinks in probabilities, ask for their prior π and give them the cap $e^{\varepsilon}\pi/(e^{\varepsilon}\pi + 1 - \pi)$. The guarantee limits the *change* in belief, not the belief itself. An attacker who is already 99% sure stays about that sure, and DP promises only that the output does not make it much surer. The bound is per person and per release. For correlated records, such as a household, first work out the guarantee for the whole group, and for repeated releases first add up their privacy cost. With $\delta > 0$, quote the bound at a larger level ε' together with the exception of Equation D.11. Section 18.7 states the result and discusses weaker attackers.

D.3. From Rényi DP to (ε, δ)

D.3.1. The question

Your accountant tracks RDP because RDP composes by adding. At the end it hands you a curve, one number for each order α (defined next), and you must report (ε, δ). How do you turn the curve into ε?

The **Rényi divergence** of order $\alpha > 1$ is a log average of a power of the likelihood ratio p/q. Written with the privacy loss, with y drawn from $P = M(D)$, it is

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \mathbb{E}_P\big[e^{(\alpha-1)L}\big]. \tag{D.12}$$

The expectation is the average of $(p/q)^{\alpha-1}$ weighted by p, which is the integral of $p^\alpha q^{1-\alpha}$, the form in which it is often defined. A mechanism has **RDP curve** $\varepsilon(\alpha)$ when $D_\alpha(M(D)\|M(D')) \le \varepsilon(\alpha)$ for every pair of neighbours. Multiplying by $\alpha - 1$ and exponentiating, this says that

$$\mathbb{E}_P\big[e^{(\alpha-1)L}\big] \le e^{(\alpha-1)\varepsilon(\alpha)}. \tag{D.13}$$

In words, the curve bounds the moment generating function (MGF) of the privacy loss, the average of $e^{\lambda L}$, at the point $\lambda = \alpha - 1$.

D.3.2. The answer

For every order $\alpha > 1$, a mechanism with RDP curve $\varepsilon(\alpha)$ is (ε, δ)-DP with

$$\varepsilon = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1}. \tag{D.14}$$

Every order gives a valid guarantee, so you report the smallest ε over the orders you have computed, usually a fixed grid. This is Mironov's conversion.

D.3.3. A worked example

For the Gaussian running example with $\mu = \Delta/\sigma = 1$, the curve is $\varepsilon(\alpha) = \alpha/2$ (step 4 below derives it). With $\delta = 10^{-5}$, the second term's numerator is $\ln(1/\delta) = 11.5129$. So at order 2 the conversion gives $1 + 11.5129 = 12.5129$, and at order 32 it gives $16 + 11.5129/31 = 16.3714$. Between the two, at order 5.8, it gives $2.9 + 11.5129/4.8 = 5.2985$, which is the best possible (step 5). Yet the exact ε of this mechanism at $\delta = 10^{-5}$ is 4.3772 (Section D.5), so the conversion overstates it by 0.9213.

D.3.4. Why it works

The proof has two moves. Markov's inequality turns the MGF bound into a bound on the chance that the loss is large. Then a small loss everywhere except on a rare set gives (ε, δ)-DP.

1. **Markov's inequality on the loss.** Fix a target level ε. Markov's inequality says that a quantity X that is never negative exceeds a level t with probability at most $\mathbb{E}[X]/t$. The reason is that those outcomes alone already contribute at least t times their probability to the average. The quantity $e^{(\alpha-1)L}$ is never negative, and it exceeds $e^{(\alpha-1)\varepsilon}$ exactly when $L > \varepsilon$, since $\alpha - 1 > 0$. So, using Equation D.13 in the last step,

$$\Pr_P[L > \varepsilon] \le \frac{\mathbb{E}_P\left[e^{(\alpha-1)L}\right]}{e^{(\alpha-1)\varepsilon}} \le e^{(\alpha-1)(\varepsilon(\alpha)-\varepsilon)}. \tag{D.15}$$

2. **Solve for ε.** Set the right side equal to δ and take logs: $(\alpha-1)(\varepsilon(\alpha)-\varepsilon) = -\ln(1/\delta)$. Solving for ε gives Equation D.14. So at that ε, the loss exceeds ε with probability at most δ.
3. **A rare large loss gives (ε, δ)-DP.** Take any set S of outputs and split it in two. On the part where $L \le \varepsilon$, each output has $p(y) \le e^{\varepsilon} q(y)$, so that part has P-probability at most e^{ε} times its Q-probability, which is at most $e^{\varepsilon} \Pr_Q[S]$. The other part lies inside the set where $L > \varepsilon$, which has P-probability at most δ by step 2. Adding the two parts gives $\Pr_P[S] \le e^{\varepsilon} \Pr_Q[S] + \delta$. RDP also bounds the divergence with the two datasets swapped, because swapped neighbours are still neighbours, so the same argument gives the other direction. Section A.6 uses the same split to calibrate the Gaussian mechanism. Note the direction: a rare large loss is enough for (ε, δ)-DP, but Section D.2 showed that (ε, δ)-DP does not force the large loss to be that rare.

A tiny case shows step 3 at work, before the Gaussian. Take three outputs with probabilities $p = (0.6, 0.39, 0.01)$ under D and $q = (0.6, 0.3999, 0.0001)$ under D', and $\varepsilon = 0.1$. Only the third output has a loss above 0.1, namely $\ln 100 = 4.605$, and its probability under D is 0.01. Step 3 then promises $(0.1, 0.01)$-DP. Checking all 8 sets directly, the largest value of $\Pr_P[S] - e^{0.1} \Pr_Q[S]$ is 0.00989, just below the promised 0.01.

4. **The Gaussian's curve.** Under P the loss is normal with mean $\mu^2/2$ and variance μ^2, so its MGF is that of a normal variable. For a normal variable X with mean m and variance v, that MGF is

$$\mathbb{E}[e^{\lambda X}] = e^{\lambda m + \lambda^2 v/2}. \tag{D.16}$$

   So with $\lambda = \alpha - 1$, $m = \mu^2/2$ and $v = \mu^2$, the exponent is $(\alpha-1)\mu^2/2 + (\alpha-1)^2\mu^2/2$, which factors as $(\alpha-1)\,\alpha\mu^2/2$. Dividing by $\alpha - 1$ then gives $\varepsilon(\alpha) = \alpha\mu^2/2$, a straight line through the origin. Section B.4 derives it in more detail and shows why curves add under composition.
5. **The best order.** The conversion is $g(\alpha) = \alpha\mu^2/2 + \ell/(\alpha-1)$, where $\ell = \ln(1/\delta)$. The first term grows with the order and the second shrinks, so the best order is in between. The slope is $g'(\alpha) = \mu^2/2 - \ell/(\alpha-1)^2$, which is zero when $(\alpha-1)^2 = 2\ell/\mu^2$. The first term is a straight line and the second curves upward, so their sum curves upward and this point is its minimum:

$$\alpha^* = 1 + \frac{\sqrt{2\ell}}{\mu}, \qquad \varepsilon = \frac{\mu^2}{2} + \mu\sqrt{2\ell}. \tag{D.17}$$

For the second formula, the first term at α^* is $\mu^2/2 + \mu\sqrt{2\ell}/2$, and the second is $\ell\mu/\sqrt{2\ell} = \mu\sqrt{2\ell}/2$. At $\mu = 1$, $\sqrt{2 \times 11.5129} = 4.7985$, so $\alpha^* = 5.7985$ and $\varepsilon = 0.5 + 4.7985 = 5.2985$.

The improved conversion. Step 3 is wasteful. The smallest valid δ is the largest value of $\Pr_P[S] - e^{\varepsilon} \Pr_Q[S]$ over sets S. That largest value comes from the set of outputs with $L > \varepsilon$, and each such output adds $p - e^{\varepsilon} q = p\,(1 - e^{\varepsilon - L})$. So the exact δ counts an output with $L > \varepsilon$ only in proportion $1 - e^{\varepsilon - L}$, which is small when L is just above ε, while step 3 charges it in full. The improved conversion (Balle et al. 2020; Canonne, Kamath & Steinke 2020) bounds that proportion by the smallest multiple of $e^{(\alpha-1)L}$ that lies above it, whose average Equation D.13 controls. It gives

$$\varepsilon = \varepsilon(\alpha) + \ln \frac{\alpha - 1}{\alpha} - \frac{\ln \delta + \ln \alpha}{\alpha - 1}. \tag{D.18}$$

Section 30.2 derives it. For the Gaussian with $\mu = 1$ its best order is 5.43, where it gives 4.7284. It saves 0.5701 of the 0.9213 by which Mironov's conversion overstates the exact 4.3772.

D.3.5. Checking it

The script averages e^{2L} over 2,000,000 Gaussian outputs; the log of that average, divided by 2, estimates the Rényi divergence of order 3 as 1.4965, against $\alpha\mu^2/2 = 1.5$ from step 4. At Mironov's $\varepsilon = 5.2985$ the chance that the loss exceeds ε is 7.992×10^{-7}, below the 10^{-5} that step 1 guarantees. The exact δ at that ε is smaller still, 1.300×10^{-7}. So the true δ is 76.9 times smaller than the 10^{-5} you report. At the improved conversion's 4.7284 the exact δ is 2.098×10^{-6}, so it too holds with room to spare.

Figure D.3 draws the conversion as a function of the order. The dashed line and the dashed hyperbola are its two terms, and their sum has a single minimum at $\alpha^* = 5.80$. The improved conversion lies below at every order, and neither reaches the exact value. The reason is that the RDP curve keeps only the MGF values of the privacy loss, so a conversion from it must allow for every loss distribution with those values.

D.3.6. Recipe and pitfalls

Compose in RDP, by adding the curves at each order of a fixed grid, such as the 154 orders from 1.1 to 512 that Part 4 uses. Then convert once, at the end, taking the smallest ε over the grid. Use the improved formula, because it is never worse and needs one more line of code. It can return a negative number when $\varepsilon(\alpha)$ is tiny, and then you report 0.

Three things go wrong. Converting each step to (ε, δ) and then adding the steps' ε's (basic composition) wastes most of the advantage of RDP. A grid can miss the best order. The conversion is flat near its minimum, so a grid point close to the best order loses little, but a grid with no point near it can lose a lot. And the curve must hold for every pair of neighbours under the relation you claim. Section 30.1 measures where each step of the proof loses, and Section 30.4 explains how to choose the grid.

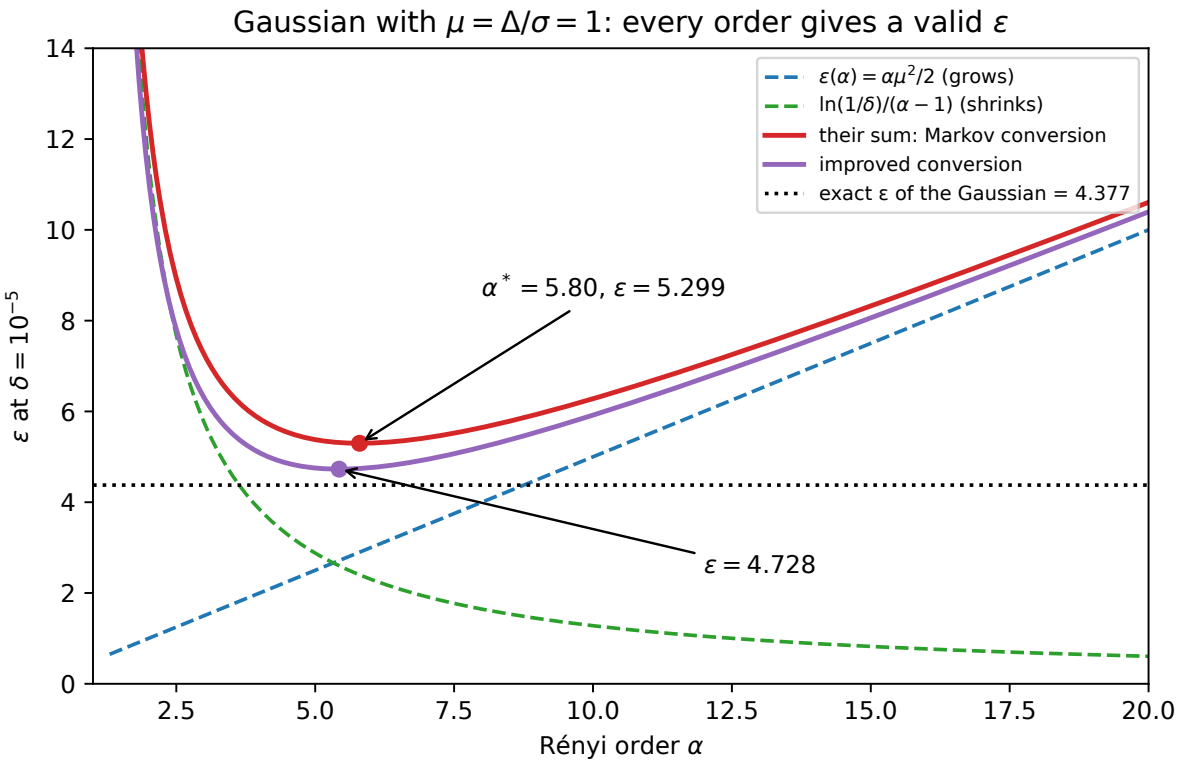


Figure D.3.: The two terms of Mironov's conversion for the Gaussian mechanism with $\mu = 1$ at $\delta = 10^{-5}$: the Rényi curve $\alpha/2$, which grows, and $\ln(1/\delta)/(\alpha - 1)$, which shrinks. Their sum (red) is smallest at $\alpha^* = 5.80$. The improved conversion (purple) and the exact ε of the Gaussian (dotted) are shown for comparison.

D.4. From zero-concentrated DP to (ε, δ)

D.4.1. The question

When every step adds Gaussian noise, the RDP curve is a straight line through the origin. A line is fixed by its slope, so you could keep one number instead of a curve. What is that number, and what (ε, δ) does it give?

D.4.2. The answer

A mechanism is **ρ-zCDP** when its RDP curve lies under the line of slope ρ through the origin: $\varepsilon(\alpha) \le \rho\alpha$ for every order $\alpha > 1$. The word "concentrated" in the name says that the privacy loss then has tails no heavier than a normal variable's. Such a mechanism is (ε, δ)-DP with

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{D.19}$$

Three facts make ρ easy to track. First, the Gaussian mechanism with sensitivity Δ and noise σ is ρ-zCDP with $\rho = \Delta^2/(2\sigma^2)$. Second, running several mechanisms adds their ρ's. Third, an ε-DP mechanism, such as a Laplace count, is $(\varepsilon^2/2)$-zCDP (Bun & Steinke 2016, Proposition 1.4). For calibration, though, you need the reverse. Solving Equation D.19 for ρ, a target (ε, δ) is met by any

$$\rho \le \left(\sqrt{\varepsilon + \ln(1/\delta)} - \sqrt{\ln(1/\delta)}\right)^2. \tag{D.20}$$

D.4.3. A worked example

You answer ten count queries, each with $\Delta = 1$ and Gaussian noise of $\sigma = 5$. Each costs $\rho = 1/(2 \times 25) = 0.02$, so the ten cost 0.2. You also release one Laplace count at $\varepsilon = 0.5$, which costs $0.5^2/2 = 0.125$. So the total is $\rho = 0.325$, and the whole release is $(\varepsilon, 10^{-5})$-DP with

$$\varepsilon = 0.325 + 2\sqrt{0.325 \times 11.5129} = 4.1937. \tag{D.21}$$

Going the other way, a single Gaussian query that must be $(1, 10^{-5})$-DP can have ρ up to $(\sqrt{12.5129} - \sqrt{11.5129})^2 = 0.02082$. That means $\sigma = 1/\sqrt{2\rho} = 4.901$ for a sensitivity of 1.

D.4.4. Why it works

1. **Plug the line into the RDP conversion.** By Equation D.14 with $\varepsilon(\alpha) = \rho\alpha$, every order gives a valid ε, namely $g(\alpha) = \rho\alpha + \ell/(\alpha - 1)$, with $\ell = \ln(1/\delta)$.
2. **Find where the slope is zero.** The slope is $g'(\alpha) = \rho - \ell/(\alpha - 1)^2$. It is zero when $(\alpha - 1)^2 = \ell/\rho$, that is, at $\alpha^* = 1 + \sqrt{\ell/\rho}$.
3. **Check that it is a minimum.** The term $\rho\alpha$ is a straight line, and $\ell/(\alpha - 1)$ curves upward for $\alpha > 1$. Their sum curves upward, so its only flat point is its lowest point.
4. **Evaluate.** At α^* the first term is $\rho + \rho\sqrt{\ell/\rho} = \rho + \sqrt{\rho\ell}$. The second is $\ell/\sqrt{\ell/\rho} = \sqrt{\rho\ell}$. Their sum is $\rho + 2\sqrt{\rho\ell}$, which is Equation D.19.
5. **The Gaussian's ρ.** Step 4 of Section D.3 showed that the Gaussian's curve is exactly the line $\alpha\mu^2/2$ with $\mu = \Delta/\sigma$. So $\rho = \mu^2/2 = \Delta^2/(2\sigma^2)$.

6. **Composition.** RDP curves add at each order when mechanisms run one after another (Section B.4). Lines through the origin add to a line whose slope is the sum of the slopes, so the ρ's add.
7. **Pure DP.** An ε-DP mechanism has a privacy loss between $-\varepsilon$ and ε. A variable confined to an interval has light tails, and Hoeffding's lemma, which bounds the MGF of such a variable by a normal one's, turns that into the line $\alpha\varepsilon^2/2$. Section 31.2 proves it, along with the sharper slope $\varepsilon\tanh(\varepsilon/2)$.
8. **The inverse.** Write $x = \sqrt{\rho}$, so that Equation D.19 reads $x^2 + 2x\sqrt{\ell} = \varepsilon$. Adding ℓ to both sides then completes the square, $(x + \sqrt{\ell})^2 = \varepsilon + \ell$, and taking the positive root and squaring it gives Equation D.20.

The one-step Gaussian with $\Delta = 1$ and $\sigma = 1$ is 0.5-zCDP, and it shows that this section and the last agree. Step 2 gives $\alpha^* = 1 + \sqrt{11.5129/0.5} = 5.7985$, and Equation D.19 gives $0.5 + 2\sqrt{0.5 \times 11.5129} = 5.2985$, the same as the minimum found for the curve $\alpha/2$ in Section D.3.

D.4.5. Checking it

For a Gaussian mechanism the zCDP line is exact, so the only slack is in the conversion. Table D.4 compares Equation D.19 with the improved RDP conversion on the same line, minimized over 20,001 orders, and with the exact ε of the Gaussian with $\mu = \sqrt{2\rho}$. The simple formula is 13% to 27% above the exact value, more so at small ρ. The improved conversion removes between 49% and 68% of that gap.

Table D.4.: The ε at $\delta = 10^{-5}$ of a ρ-zCDP mechanism, from the simple formula and the improved conversion, against the exact ε of the Gaussian mechanism with the same ρ.

ρ	$\mu = \sqrt{2\rho}$	Simple, Equation D.19	Improved conversion	Exact Gaussian
0.125	0.5000	2.5243	2.1657	1.9931
0.2	0.6325	3.2349	2.8136	2.5944
0.5	1.0000	5.2985	4.7284	4.3772
5	3.1623	20.1743	19.0473	17.8566

D.4.6. Recipe and pitfalls

Keep the budget in ρ, add it up, and convert once at the end. Do not add up the ε's of the parts instead. Because ε grows like the square root of ρ, the ε's of two halves add to more than the ε of the whole. Pure-DP steps enter at $\varepsilon^2/2$, which is close to the best possible slope for small ε and wasteful for large ones.

The big pitfall is subsampling. Differentially private stochastic gradient descent (SGD), DP-SGD for short, runs each step on a random sample of the data, which makes each step much more private at small orders. But the sampled step's RDP curve bends sharply upward at large orders. Any line that stays above the whole curve is then as steep as the line without sampling, so zCDP loses the whole gain. Accountants for DP-SGD therefore use RDP curves or numerical methods (Section 31.6). Section 31.3 states the conversion with its improved version.

D.5. Gaussian DP and its exact (ε, δ) curve

D.5.1. The question

For the Gaussian mechanism, both conversions above leave slack. Is there one number that describes its privacy exactly, and turns into the exact (ε, δ)?

D.5.2. The answer

A mechanism is **μ-GDP** when no pair of neighbours is easier to tell apart than $\mathcal{N}(0,1)$ from $\mathcal{N}(\mu,1)$. In terms of errors, every test with false positive rate FPR has a false negative rate of at least

$$G_\mu(\mathrm{FPR}) = \Phi(\Phi^{-1}(1-\mathrm{FPR}) - \mu). \tag{D.22}$$

The Gaussian mechanism is μ-GDP with $\mu = \Delta/\sigma$, with no slack. It is (ε, δ)-DP exactly when δ is at least

$$\delta(\varepsilon) = \Phi\Big(-\frac{\varepsilon}{\mu} + \frac{\mu}{2}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\varepsilon}{\mu} - \frac{\mu}{2}\Big). \tag{D.23}$$

When several GDP mechanisms run one after another, their μ's add like the sides of a right-angled triangle, $\mu = \sqrt{\mu_1^2 + \cdots + \mu_k^2}$.

D.5.3. A worked example

Take $\mu = 1$, the Gaussian mechanism with $\sigma = \Delta$. The best test says "D'" when the output is above a threshold (step 1 below). An attacker who accepts 5% false positives uses the threshold $\Phi^{-1}(0.95) = 1.6449$, and misses with probability at least $G_1(0.05) = \Phi(1.6449 - 1) = 0.7405$. At $\varepsilon = 1$, Equation D.23 gives $\delta(1) = \Phi(-0.5) - e\,\Phi(-1.5) = 0.308538 - e \times 0.066807 = 0.126937$. To report ε at $\delta = 10^{-5}$, solve Equation D.23 for ε by bisection, which repeatedly halves an interval that contains the answer. This works because $\delta(\varepsilon)$ falls as ε grows: a larger ε asks less of every test. Table D.5 gives the result for three values of μ.

Table D.5.: The exact δ at $\varepsilon = 1$ and the exact ε at $\delta = 10^{-5}$ of μ-GDP, from Equation D.23.

μ	0.5	1	2
$\delta(1)$	0.0068	0.1269	0.5099
ε at $\delta = 10^{-5}$	1.9931	4.3772	9.9973

Composition is just as quick. Four releases with $\sigma = 2\Delta$ each have $\mu_i = 0.5$, so together they have $\mu = \sqrt{4 \times 0.25} = 1$. They are 1-GDP, and so $(4.3772, 10^{-5})$-DP.

D.5.4. Why it works

The first three steps find the best test, which gives the curve G_μ. The next three find the (ε, δ) guarantee that the curve implies, and the last one handles composition.

1. **The best test thresholds the output.** The Neyman–Pearson lemma says that, among tests with the same false positive rate, the one with the fewest misses says "D'" on the outputs where $q(y)/p(y)$ is largest. The reason is that each output in the region costs $p(y)$ in false positives and buys $q(y)$ in detections, so the best buys go first. For the pair $\mathcal{N}(0,1)$ and $\mathcal{N}(\mu,1)$, the ratio is $q(y)/p(y) = e^{-L(y)} = e^{\mu y - \mu^2/2}$, which grows with y. So the best test says "D'" when y is above a threshold τ.
2. **Its two errors.** Under D, y is standard normal, so $\text{FPR} = \Pr[Z > \tau] = 1 - \Phi(\tau)$, and the threshold for a given FPR is $\tau = \Phi^{-1}(1 - \text{FPR})$. Under D', $y = \mu + Z$, so the test misses when $\mu + Z \le \tau$, which has probability $\Phi(\tau - \mu)$.
3. **Substitute.** Putting $\tau = \Phi^{-1}(1 - \text{FPR})$ into $\Phi(\tau - \mu)$ gives Equation D.22. For the Gaussian mechanism, a pair of neighbours whose query values differ by $s \le \Delta$ gives the pair $\mathcal{N}(0,1)$ and $\mathcal{N}(s/\sigma, 1)$, with shift $s/\sigma \le \Delta/\sigma$. A larger shift is easier to detect, because G_μ falls as μ grows, so the worst pair has $\mu = \Delta/\sigma$.
4. **One (ε, δ) guarantee is one line.** By Section D.1, (ε, δ)-DP asks every test to have $\text{FNR} \ge 1 - \delta - e^\varepsilon \text{FPR}$. The best tests have $\text{FNR} = G_\mu(\text{FPR})$, so the condition is $\delta \ge 1 - G_\mu(\text{FPR}) - e^\varepsilon \text{FPR}$ for every FPR, and the smallest valid δ is the largest value of the right side. The other inequality of Equation D.6 gives the same number. Reflecting the output, $y \mapsto \mu - y$, swaps the two normals, so it turns each test into one with the two error rates swapped, and the two inequalities into each other.
5. **Find the touching point.** Lowering the threshold by a small step h adds the outputs just below τ to the region. That raises FPR by about $p(\tau)h$ and lowers the miss rate by about $q(\tau)h$, so the curve's slope at that point is $-q(\tau)/p(\tau) = -e^{\mu\tau - \mu^2/2}$. As FPR grows the threshold drops, so this slope gets flatter. The quantity $1 - G_\mu(\text{FPR}) - e^\varepsilon \text{FPR}$ therefore rises while the miss rate falls faster than e^ε per unit of FPR, and falls afterwards. So it is largest where the slope equals $-e^\varepsilon$. Setting $\mu\tau - \mu^2/2 = \varepsilon$ gives $\tau = \varepsilon/\mu + \mu/2$.
6. **Read off δ.** At that threshold, $\text{FPR} = 1 - \Phi(\tau) = \Phi(-\tau)$ and the detection rate is $1 - \Phi(\tau - \mu) = \Phi(\mu - \tau)$. So $\delta(\varepsilon) = \Phi(\mu - \tau) - e^\varepsilon \Phi(-\tau)$, which is Equation D.23.
7. **Composition.** Each release adds independent noise, so k releases give a vector of k outputs, $\mathcal{N}(0, I)$ under D and $\mathcal{N}(\mu, I)$ under D' with $\mu = (\mu_1, \dots, \mu_k)$. Only the direction of μ carries evidence, and along it the shift is the length of μ, $\sqrt{\mu_1^2 + \cdots + \mu_k^2}$. Section B.3 proves this rotation step.

At $\mu = 1$ and $\varepsilon = 1$, step 5 gives $\tau = 1 + 0.5 = 1.5$. Step 6 then gives $\text{FPR} = \Phi(-1.5) = 0.066807$ and a detection rate of $\Phi(-0.5) = 0.308538$, which are the numbers of the worked example. Section A.7 reaches the same formula by a different route. There, $\Phi(-0.5)$ is the chance that the privacy loss exceeds ε, and $e\,\Phi(-1.5)$ is the correction subtracted from it. With $\mu = \Delta/\sigma$ the two derivations give the same curve.

D.5.5. Checking it

Searching over 20,001 thresholds for the largest $1 - G_1(\text{FPR}) - e\,\text{FPR}$ gives 0.126937, the value of Equation D.23. A simulation of the threshold test at $\text{FPR} = 0.05$ with 1,000,000 draws per world

gives a false positive rate of 0.0497 and a false negative rate of 0.7413, against the exact 0.7405.

Figure D.4 shows both halves of the derivation. On the left, each curve G_μ has a dashed line of slope $-e$ that just touches it. The line starts at $1-\delta(1)$ on the vertical axis, so the gap below 1 is the exact δ at $\varepsilon = 1$. On the right, the whole curve $\delta(\varepsilon)$ falls steeply, and the dots mark where it crosses 10^{-5}.

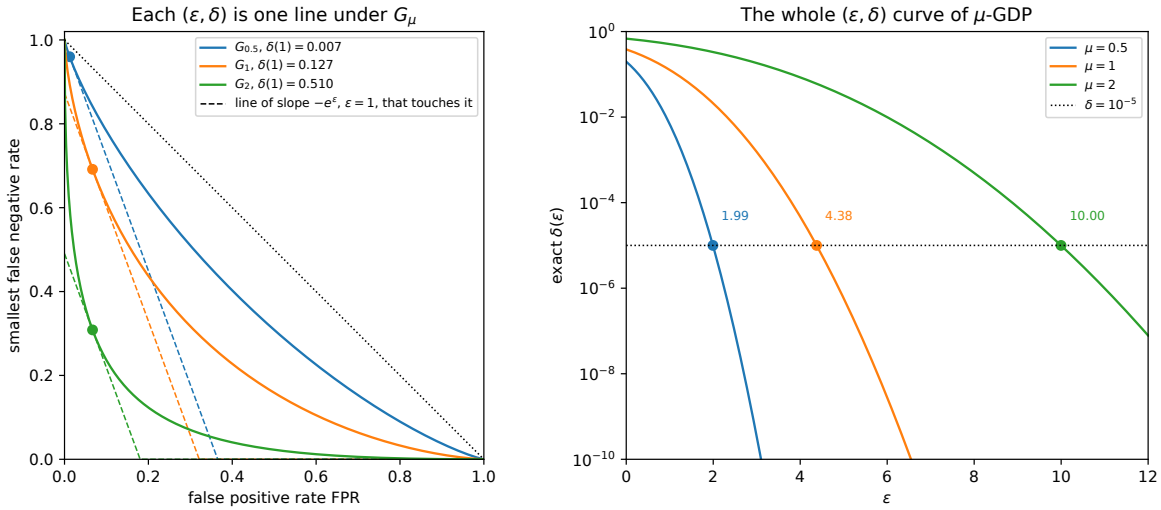


Figure D.4.: Left: the trade-off curves G_μ for $\mu = 0.5$, 1 and 2, the smallest false negative rate of any test at each false positive rate. Each dashed line has slope $-e$ and touches its curve at the dot; it meets the vertical axis at $1-\delta(1)$. Right: the exact $\delta(\varepsilon)$ of μ-GDP on a log scale, with the ε at $\delta = 10^{-5}$ marked.

D.5.6. Recipe and pitfalls

For a pipeline of Gaussian releases, compute $\mu_i = \Delta_i/\sigma_i$ for each, combine them as $\mu = \sqrt{\sum \mu_i^2}$, and solve Equation D.23 for ε at your δ. Nothing is lost along the way, so this ε is exact.

The formula is exact only for Gaussian noise with the ℓ_2 sensitivity Δ, the largest Euclidean distance between the query's values on neighbours. For subsampled DP-SGD, GDP comes from a central limit theorem and is only an approximation (Section 32.6). And one reported (ε, δ) pair keeps one touching line and forgets the rest of the curve. Section 32.3 shows that the whole family of lines rebuilds G_μ.

D.6. Summary

Table D.6 puts the four definitions side by side. Each one tracks a different summary of the privacy loss, and each converts to (ε, δ) at the end.

Table D.6.: The four ways to track privacy, how each composes and how each converts to (ε, δ).

Definition	What you track	How it composes	How to get (ε, δ)
(ε, δ)-DP	Two numbers	Add the ε's and the δ's (basic composition)	Nothing to do
RDP	A curve $\varepsilon(\alpha)$ over orders	Add the curves at each order	Smallest $\varepsilon(\alpha) + \ln(1/\delta)/(\alpha - 1)$ over orders, or the improved formula
ρ-zCDP	One number ρ	Add the ρ's	$\rho + 2\sqrt{\rho \ln(1/\delta)}$
μ-GDP	One number μ	Add the μ^2's	Solve Equation D.23 for ε

For the Gaussian mechanism with $\mu = 1$ at $\delta = 10^{-5}$, the three conversions give 5.2985 (Mironov's, which equals the zCDP formula here), 4.7284 (the improved conversion) and 4.3772 (GDP, exact).

D.7. Code

The script `code/appendix-derivations/apx_core_conversions.py` checks every number on this page. It enumerates every test against three people's randomized response, simulates the posterior and the threshold test, and checks the Gaussian's Rényi divergence by Monte Carlo. It also minimizes both RDP conversions over orders, inverts the exact Gaussian $\delta(\varepsilon)$ by bisection, and computes Φ^{-1} by bisection on Φ, which it evaluates with the standard library's complementary error function. Run it with

```
uv run python code/appendix-derivations/apx_core_conversions.py
```

Table D.7.: The checks the script prints, with seed 0.

Check	Printed	Expected
Least slack over all 256 tests, $\varepsilon = \ln 3$	0.000000	at least 0
Posterior from 2,000,000 randomized-response draws, prior 1%, $\varepsilon = 1$	0.0267	0.0267
Rényi divergence of order 3 by Monte Carlo, $\mu = 1$	1.4965	1.5

Check	Printed	Expected
Three outputs: largest $\Pr_P[S] - e^{0.1}\Pr_Q[S]$	0.00989	at most 0.01
$\Pr[L > 5.2985]$ for the Gaussian, $\mu = 1$	7.992×10^{-7}	at most 10^{-5}
$\delta(1)$ for $\mu = 1$ by search over thresholds	0.126937	0.126937
Threshold test at FPR $= 0.05$, $\mu = 1$, simulated miss rate	0.7413	0.7405

Every bound holds, and the ones that should be tight are tight. Randomized response reaches both the testing bound and the posterior cap. The Gaussian's privacy loss is normal, which is why its RDP curve, its zCDP parameter and its GDP curve are all exact descriptions. So for the RDP curve and the zCDP parameter, all the slack comes from the final conversion to (ε, δ).

D.8. References

- Dwork, C. & Roth, A. *The Algorithmic Foundations of Differential Privacy* (2014). The definitions, the privacy loss and the basic properties.
- Wasserman, L. & Zhou, S. "A statistical framework for differential privacy" (2010), and Kairouz, P., Oh, S. & Viswanath, P. "The composition theorem for differential privacy" (2015). The reading of (ε, δ)-DP as a limit on every test.
- Kasiviswanathan, S. P. & Smith, A. "On the 'semantics' of differential privacy: a Bayesian formulation" (2014). The posterior reading, including weaker attackers.
- Mironov, I. "Rényi differential privacy" (2017). RDP and the conversion of Equation D.14.
- Balle, B., Barthe, G., Gaboardi, M., Hsu, J. & Sato, T. "Hypothesis testing interpretations and Rényi differential privacy" (2020), and Canonne, C., Kamath, G. & Steinke, T. "The discrete Gaussian for differential privacy" (2020). The improved conversion.
- Bun, M. & Steinke, T. "Concentrated differential privacy: simplifications, extensions, and lower bounds" (2016). zCDP, its conversion and the bound for pure DP.
- Dong, J., Roth, A. & Su, W. "Gaussian differential privacy" (2022). f-DP, GDP, the exact (ε, δ) curve and composition.
- Balle, B. & Wang, Y.-X. "Improving the Gaussian mechanism for differential privacy: analytical calibration and optimal denoising" (2018). The exact δ of the Gaussian mechanism as a calibration rule.

E. Sums, limits and counting

This appendix proves six facts about sums of random variables and about counting that the chapters use without a full proof. The first finds the distribution of a sum of two independent variables by adding up the ways the sum can arise. That replaces a theorem the book takes as given, namely that the moment generating function of a distribution pins it down. The second counts: it finds the most likely value of a binomial count, how tall that peak is, and how large $n!$ is. The third uses the size of $k!$ to show how fast a Poisson count becomes unlikely far above its mean. The fourth proves the central limit theorem (CLT) by swapping one term at a time for a normal one. The fifth shows that the length of a normal vector in d dimensions rarely exceeds $\sqrt{d}$ by much. The sixth finds the tail of the difference of two Laplace variables. They sit together because each is a statement about a sum or a count. They also share their tools. The proofs add up the ways a sum can happen, compare neighbouring terms through their ratio, and apply Markov's inequality to an exponential.

Table E.1 lists where each result is stated or used.

Table E.1.: The derivations of this appendix and the chapter sections that state or use them.

Derivation	Where the chapters state or use it
Section E.1: sums by convolution, rotation invariance of the normal	Section 1.2 and the sections on the Poisson, Laplace and normal in Probability distributions; Section 3.3; Section 20.4; Section 27.1
Section E.2: Stirling's formula, the binomial mode and its height	Section 9.3; Section 9.5; Section 12.2; Section 13.3
Section E.3: the tail of a Poisson count	Section 1.4
Section E.4: the central limit theorem	The normal section of Probability distributions; Section 32.5 and every normal approximation in the book
Section E.5: the length of a normal vector	Section 28.4
Section E.6: the difference of two Laplace variables	Section 42.6; Section 27.1

E.1. A sum's distribution, found by convolution

E.1.1. What the chapter uses

Probability distributions identifies the distribution of a sum in four places. The sum of two independent Poisson counts is Poisson, and the difference of two independent exponential variables is Laplace. Any weighted sum of independent normal variables is normal, and the sum of n independent Exp(1) variables has the Gamma$(n, 1)$ density. Each time the chapter multiplies moment generating functions (MGFs), recognizes the product, and then relies on a theorem stated in Section 1.2: two distributions with the same MGF near 0 are the same. The chapter says that the proof of that theorem "needs more analysis than this book uses, so take it as given".

The same chapter states two facts about a normal vector $X \sim \mathcal{N}(0, \sigma^2 I_d)$, whose d coordinates are independent $\mathcal{N}(0, \sigma^2)$ variables. Its projection onto any unit vector is $\mathcal{N}(0, \sigma^2)$. And the projections onto two perpendicular unit vectors are independent, because "for jointly normal variables zero covariance means independence", which the chapter does not prove. Section 3.3 and Section 20.4 use a stronger form: rotating the space does not change the distribution of X. They use it to compare the noise X with a shifted copy $X + v$: a rotation turns the shift v to point along the first axis, which reduces the comparison to one dimension. Finally, Section 27.1 states that the difference of two independent Lap$(1/\varepsilon)$ variables has density $\frac{\varepsilon}{4}(1 + \varepsilon|z|)e^{-\varepsilon|z|}$.

This section proves all of these without the uniqueness theorem. The tool is the **convolution**, which adds up the ways a sum can arise. Take independent variables X and Y, and write f and g for their densities. The density of the sum $X + Y$ at a point s is then

$$f_{X+Y}(s) = \int f(x)\, g(s - x)\, dx, \tag{E.1}$$

and for counts the integral becomes a sum over x. In every line below the variables are independent. In the last one, each X_i is normal with mean μ_i and variance σ_i^2, and the weights a_i are constants. As in the chapter, Exp(b) and Lap(b) have scale b, and an equals sign between distributions means that the left side has the distribution on the right.

$$\begin{aligned} &\text{Poisson}(\mu_1) + \text{Poisson}(\mu_2) = \text{Poisson}(\mu_1 + \mu_2), \qquad && \text{Exp}(b) - \text{Exp}(b) = \text{Lap}(b), \\ &\text{Exp}(1) + \cdots + \text{Exp}(1)\ (n \text{ terms}) = \text{Gamma}(n, 1), \qquad && \textstyle\sum_i a_i X_i \sim \mathcal{N}\big(\sum_i a_i \mu_i, \sum_i a_i^2 \sigma_i^2\big). \end{aligned} \tag{E.2}$$

For the normal vector, the claim is that rotating X leaves its distribution unchanged, so the coordinates of X along any two perpendicular unit vectors are independent $\mathcal{N}(0, \sigma^2)$ variables.

E.1.2. The idea

To find the chance that a sum takes a value, list every way it can take that value and add up their chances. Roll two fair dice and ask for a total of 4. The first die shows 1, 2 or 3, and the second must then show 3, 2 or 1. Each pair has chance $\frac{1}{6} \cdot \frac{1}{6}$, because the dice are independent, so the total is 3/36. Table E.2 writes this out.

Table E.2.: The ways two dice can total 4. Adding the last row gives $\Pr[\text{total} = 4] = 3/36$.

First die x	1	2	3	4, 5, 6
Second die must show $4 - x$	3	2	1	impossible
Chance of this way	1/36	1/36	1/36	0

For a density the list of ways becomes an integral. The first variable takes a value x, the second must make up the rest, $s-x$, and independence multiplies the two densities. That is Equation E.1. Each identification in the chapter is one such integral, and each is short.

Rotation invariance has a different reason. The joint density of X depends on the point x only through its distance from the origin, so in two dimensions its contours are circles, and in d dimensions they are spheres. A rotation moves every point along its sphere and keeps every volume, so it leaves the density where it was. So a rotation that carries two perpendicular directions onto the first two coordinate axes does not change the distribution. Along the axes the coordinates are independent by definition, so they are independent along the two directions too.

E.1.3. Derivation

1. **Why Equation E.1 holds.** Fix a small width $\kappa > 0$ and the interval of that width around s. The sum lands in it when X is near some value x and Y lands within κ of $s-x$. Given $X = x$, that has chance about $g(s-x)\kappa$, because Y is independent of X and so its density does not change. Averaging over x with the density f gives $\kappa \int f(x)g(s-x)\,dx$, and dividing by κ gives the density. For counts the same step reads $\Pr[X+Y=k] = \sum_j \Pr[X=j]\Pr[Y=k-j]$, as in the dice.

2. **Poisson.** Take independent $N_1 \sim \text{Poisson}(\mu_1)$ and $N_2 \sim \text{Poisson}(\mu_2)$. The sum of step 1, with $\Pr[N=j] = e^{-\mu}\mu^j/j!$, is

$$\Pr[N_1 + N_2 = k] = \sum_{j=0}^{k} e^{-\mu_1}\frac{\mu_1^j}{j!}\,e^{-\mu_2}\frac{\mu_2^{k-j}}{(k-j)!} = \frac{e^{-(\mu_1+\mu_2)}}{k!}\sum_{j=0}^{k}\binom{k}{j}\mu_1^j\mu_2^{k-j}. \tag{E.3}$$

   The second form multiplies and divides by $k!$, and $\binom{k}{j} = k!/(j!(k-j)!)$. The binomial theorem says that the last sum is $(\mu_1+\mu_2)^k$. So $\Pr[N_1+N_2=k] = e^{-(\mu_1+\mu_2)}(\mu_1+\mu_2)^k/k!$, which is the Poisson$(\mu_1+\mu_2)$ probability.

3. **Exponential minus exponential.** Let $Y_1, Y_2 \sim \text{Exp}(b)$ be independent, each with density $f(y) = \frac{1}{b}e^{-y/b}$ for $y \geq 0$ and 0 for $y < 0$. The difference $Y_1 - Y_2$ equals x when $Y_1 = y$ and $Y_2 = y - x$, so the density of the difference at x is $\int f(y)f(y-x)\,dy$. Here y is the integration variable and x is the value of the difference. This is Equation E.1 for the sum $Y_1 + (-Y_2)$, whose second term has density $f(-z)$, so the copy of f appears shifted but not flipped. The integrand is non-zero only where both $y \geq 0$ and $y - x \geq 0$, that is for $y \geq \max(0, x)$, and on that range

$$f(y)f(y-x) = \frac{1}{b^2}e^{-y/b}e^{-(y-x)/b} = \frac{1}{b^2}e^{x/b}e^{-2y/b}. \tag{E.4}$$

Integrating $e^{-2y/b}$ from a point c to infinity gives $\frac{b}{2}e^{-2c/b}$, so with $c = \max(0, x)$ the density of the difference is $\frac{1}{2b}e^{x/b}e^{-2\max(0,x)/b}$. When $x \geq 0$ this is $\frac{1}{2b}e^{-x/b}$, and when $x < 0$ it is $\frac{1}{2b}e^{x/b}$. Together they make $\frac{1}{2b}e^{-|x|/b}$, which is the Lap(b) density.

4. **Sums of exponentials.** Write $f_n(x) = x^{n-1}e^{-x}/(n-1)!$ for $x \geq 0$, the Gamma$(n, 1)$ density. For $n = 1$ it is e^{-x}, the Exp(1) density. Suppose the sum of n independent Exp(1) variables has density f_n, and add one more. Use Equation E.1 with the sum at x and the integration variable y for the first term. Both terms are non-negative, so y can only lie between 0 and x:

$$\int_0^x \frac{y^{n-1}e^{-y}}{(n-1)!}\, e^{-(x-y)}\, dy = \frac{e^{-x}}{(n-1)!}\int_0^x y^{n-1}\, dy = \frac{e^{-x}}{(n-1)!}\cdot\frac{x^n}{n} = \frac{x^n e^{-x}}{n!}. \tag{E.5}$$

The exponentials combine to e^{-x}, which does not depend on y, so it leaves the integral. The result is f_{n+1}. By induction, the claim holds for every n.

5. **Sums of normals.** Start with two independent normals with mean 0, $X_1 \sim \mathcal{N}(0, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(0, \sigma_2^2)$, and write $v = \sigma_1^2 + \sigma_2^2$. In Equation E.1 the two exponents add to $-\frac{y^2}{2\sigma_1^2} - \frac{(s-y)^2}{2\sigma_2^2}$. Completing the square in y rewrites this sum as

$$-\frac{v}{2\sigma_1^2\sigma_2^2}\Big(y - \frac{\sigma_1^2}{v}s\Big)^2 - \frac{s^2}{2v}, \tag{E.6}$$

which you can check by expanding both forms. The first term is the exponent of a normal density in y with variance $\sigma_1^2\sigma_2^2/v$, and integrating it over y gives $\sqrt{2\pi\sigma_1^2\sigma_2^2/v}$. The two normalizing factors $1/(\sigma_1\sqrt{2\pi})$ and $1/(\sigma_2\sqrt{2\pi})$ remain. Multiplying the three numbers leaves $1/\sqrt{2\pi v}$, so the sum has density $e^{-s^2/(2v)}/\sqrt{2\pi v}$, the $\mathcal{N}(0, v)$ density. Two simpler facts extend this to weighted sums with any means. First, multiplying $X \sim \mathcal{N}(\mu, \sigma^2)$ by a constant $a > 0$ gives $\mathcal{N}(a\mu, a^2\sigma^2)$. The reason is that $aX \leq t$ is the event $X \leq t/a$. Differentiating its probability in t gives the density of X at t/a times $1/a$, and that is the $\mathcal{N}(a\mu, a^2\sigma^2)$ density. A negative a also flips the sign, which the symmetry of the normal about its mean absorbs. Second, adding a constant only moves the mean. With these two facts, applying the two-term rule one term at a time gives the rule for $\sum_i a_i X_i$.

6. **Rotation invariance.** The coordinates of $X \sim \mathcal{N}(0, \sigma^2 I_d)$ are independent, so its joint density is the product of d normal densities. The exponents add up to the squared length $\|x\|^2 = x_1^2 + \cdots + x_d^2$:

$$p(x) = \prod_{i=1}^{d} \frac{e^{-x_i^2/(2\sigma^2)}}{\sigma\sqrt{2\pi}} = (2\pi\sigma^2)^{-d/2}e^{-\|x\|^2/(2\sigma^2)}. \tag{E.7}$$

In this appendix a **rotation** means any square matrix O whose rows are perpendicular unit vectors, which is also called an orthogonal matrix (it includes reflections). It keeps every length, $\|Ox\| = \|x\|$, because the coordinates of Ox are the components of x along the rows of O. It also keeps every volume, because the factor by which a matrix scales volumes is $|\det O|$, and $\det O = \pm 1$ since $O^\top O = I$. In two dimensions, for example, O turns the plane by an angle ϕ, and $\cos^2\phi + \sin^2\phi = 1$ is the reason lengths are kept. Now let $Y = OX$. For any region A, Y lands in A exactly when X lands in the region $O^{-1}A$ that O maps onto A. Substituting $x = O^{-1}y$, which keeps volume, gives $\Pr[Y \in A] = \int_A p(O^{-1}y)\, dy$. By Equation E.7, $p(O^{-1}y)$ depends only on $\|O^{-1}y\| = \|y\|$, so it equals $p(y)$. So Y has the same density as X.

7. **Perpendicular projections are independent.** Let u and v be perpendicular unit vectors. Choose a rotation O whose first row is u and whose second row is v; the other rows are any unit vectors that complete a perpendicular set. The first two coordinates of $Y = OX$ are then $\langle u, X\rangle$ and $\langle v, X\rangle$. By step 6, $Y \sim \mathcal{N}(0, \sigma^2 I_d)$, so these two coordinates are independent $\mathcal{N}(0, \sigma^2)$ variables. This also proves the chapter's general remark, if "jointly normal" means "linear functions of one vector of independent standard normals". Two such variables $A = \langle a, G\rangle$ and $B = \langle b, G\rangle$, with $G \sim \mathcal{N}(0, I_d)$, have covariance $\mathbb{E}[AB] = \sum_{i,j} a_i b_j \mathbb{E}[G_i G_j] = \sum_i a_i b_i = \langle a, b\rangle$, because $\mathbb{E}[G_i G_j]$ is 1 when $i = j$ and 0 otherwise. When that is 0, $a/\|a\|$ and $b/\|b\|$ are perpendicular unit vectors, so A and B are multiples of two independent normals, and adding constant means keeps them independent.

8. **Two Laplace variables with the same scale.** The density $\frac{\varepsilon}{4}(1 + \varepsilon|z|)e^{-\varepsilon|z|}$ is the equal-scale case of the convolution in Section E.6 (step 5 there).

One more claim of the chapter needs no new proof. It says that the sum of two independent Lap(b) variables is not Laplace, because its MGF $1/(1 - b^2\lambda^2)^2$ is not a Laplace MGF. That argument needs only the easy direction, which is that two variables with the same distribution have the same MGF. So it does not depend on the uniqueness theorem.

E.1.4. Checking it

The script of this appendix (see the Code section at the end) evaluates each convolution numerically and compares it with the closed form. For the sum of two Poisson counts with means 2 and 3, the largest gap to Poisson(5) is 3.3×10^{-16}, which is rounding error. The difference of two exponential densities, integrated on a grid, is within 7.2×10^{-5} of the Laplace density. Similarly, five exponential densities convolved on a grid are within 1.1×10^{-3} of the Gamma density. Both of these gaps are the error of the grid, so they shrink as the grid gets finer. Finally, the sum of normals with variances 1 and 4 matches the normal density with variance 5 to 1.2×10^{-12}.

Figure E.1 draws the integral of step 3. In each of the first three panels, the blue curve is $f(y)$ and the orange curve is its copy shifted by x. The green area under their product is the density of the difference at x. At $x = 0$ the two curves coincide and the overlap is largest, 0.5. Moving the copy either way shrinks the overlap at the same exponential rate, which is why the result is the two-sided Laplace shape in the last panel.

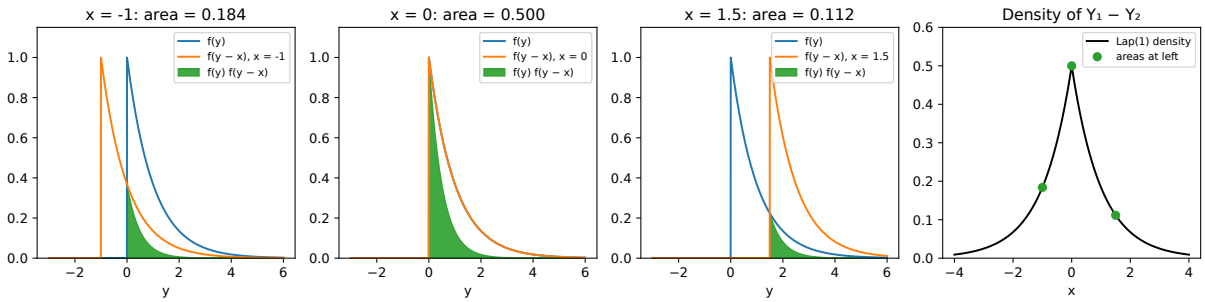


Figure E.1.: The density of $Y_1 - Y_2$ for independent Exp(1) variables at $x = -1$, 0 and 1.5, as the area under $f(y)f(y - x)$ (green), next to the Lap(1) density with those three areas marked.

For rotation invariance, the script draws a million points from $\mathcal{N}(0, I_2)$ and projects them onto $u = (\cos\phi, \sin\phi)$ and the perpendicular v, with $\phi = \pi/6$. The two projections have variances 0.9983 and 1.0010 and correlation -0.0005. Independence is a stronger claim than zero correlation, so the script also checks a joint probability. The chance that both projections exceed 1 is 0.0251, and the product of the two separate chances is $(1 - \Phi(1))^2 = 0.0252$, where Φ is the standard normal cumulative distribution function (CDF). Figure E.2 shows the circular contours and the two matching histograms.

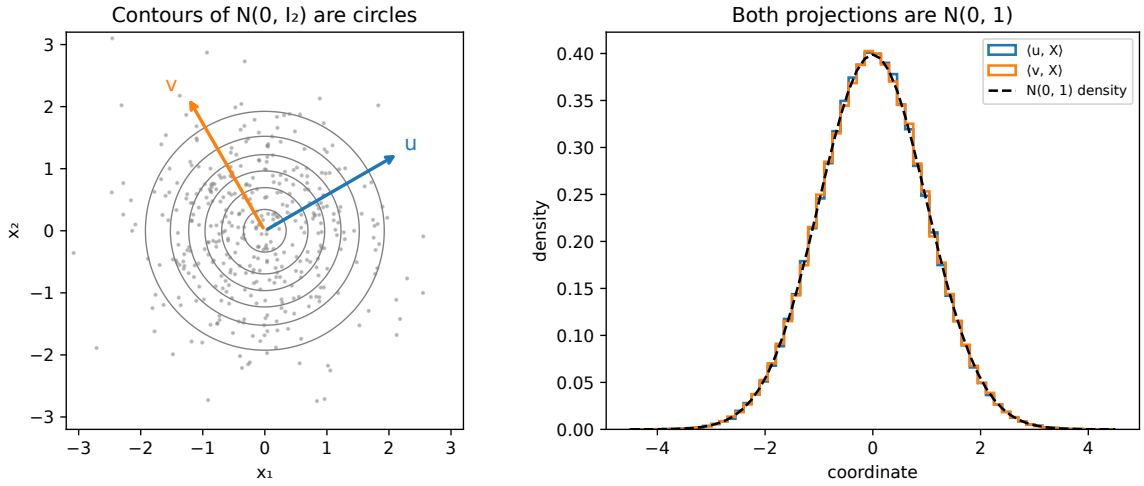


Figure E.2.: Left: contours of the $\mathcal{N}(0, I_2)$ density, which are circles, with 400 sample points and two perpendicular unit vectors u and v. Right: histograms of the projections $\langle u, X\rangle$ and $\langle v, X\rangle$ over 200,000 draws, against the $\mathcal{N}(0, 1)$ density.

E.1.5. What it does not cover

The derivations identify each sum that the book needs, one at a time. They do not prove the general theorem that an MGF finite near 0 determines the distribution. That theorem needs a way to recover a distribution from its transform, through complex numbers or a limit argument, and it stays cited to Blitzstein & Hwang (2019) and Billingsley (1995). Nothing in the book depends on it beyond the cases proved here.

E.2. Stirling's formula and the peak of a binomial

E.2.1. What the chapter uses

Four sections in the attacks part count with binomial probabilities. Write $X \sim \mathrm{Bin}(n, p)$ for the number of successes in n independent trials with success probability p. Write $\bar{p} = 1 - p$ and $p_k = \Pr[X = k] = \binom{n}{k} p^k \bar{p}^{n-k}$.

- Section 9.5 (Equation 9.17) says that the binomial weights "rise up to the middle value and fall after it". From that it shows that shifting $\mathrm{Bin}(m, \frac{1}{2})$ by one has total variation (TV)

distance equal to the largest weight, $\binom{m}{\lfloor m/2 \rfloor} 2^{-m}$. It then says that this is about $\sqrt{2/(\pi m)}$, without proof.

- Section 9.3 releases the exact sum K of m fair secret bits. Given K, the best guess sets every bit to the majority value, which gets $\max(K, m-K)$ bits right, so the expected fraction recovered is $\frac{1}{2} + \mathbb{E}|K - \frac{m}{2}|/m$. It approximates the fraction by $\frac{1}{2} + 1/\sqrt{2\pi m}$ through the central limit theorem and the mean absolute value $\sigma\sqrt{2/\pi}$ of a normal variable. That step assumes that the average distance $\mathbb{E}|K - \frac{m}{2}|$ follows the normal one, which the CLT alone does not give, because the CLT is a statement about probabilities, not averages.
- Section 12.2 (Equation 12.7) states that a binomial is unimodal. It then argues that the probability of the most likely count is close to the height $1/\sqrt{2\pi npq}$ of the normal curve, because "the density is nearly flat over a unit interval at the top".
- Section 13.3 (Equation 13.11) uses $\ln N! = N \ln N - N + O(\ln N)$, "Stirling's approximation of the factorial", without proof.

This section proves all of these. First, the binomial probabilities rise and then fall, with the most likely count at $\lfloor (n+1)p \rfloor$. Second, Stirling's formula holds with an explicit error r_n:

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n e^{r_n}, \qquad 0 < r_n \le \frac{1}{12n}. \tag{E.8}$$

Third, the fraction of bits recovered from the sum of m fair bits is exactly

$$\frac{1}{2} + \frac{1}{2}\binom{2j}{j} 4^{-j}, \qquad j = \lfloor m/2 \rfloor, \tag{E.9}$$

and $\binom{2j}{j} 4^{-j}\sqrt{\pi j} \to 1$, so the fraction is $\frac{1}{2} + 1/\sqrt{2\pi m}$ up to a relative error that vanishes.

Fourth, the height of the mode is close to the peak $1/\sqrt{2\pi npq}$ of the normal curve with the same mean and variance. The proof needs the count to have some spread, so it assumes $npq \ge 4$, and then

$$\left| \ln\left(\max_k p_k \cdot \sqrt{2\pi npq} \right) \right| \le \frac{3.5}{npq}. \tag{E.10}$$

E.2.2. The idea

The shape of the binomial comes from comparing each probability with the next one. Going from k to $k+1$ successes multiplies the probability by a ratio that gets smaller as k grows. While the ratio is above 1 the probabilities climb, and once it drops below 1 they fall, so there is one peak. With $n = 4$ and $p = \frac{1}{2}$ the ratios are $4, \frac{3}{2}, \frac{2}{3}, \frac{1}{4}$, and the probabilities $\frac{1}{16}, \frac{4}{16}, \frac{6}{16}, \frac{4}{16}, \frac{1}{16}$ climb twice and fall twice.

For the factorial, compare a sum with an integral. The logarithm $\ln n! = \ln 1 + \ln 2 + \cdots + \ln n$ is a sum of values of the slowly growing function $\ln x$, so it is close to $\int \ln x \, dx = x \ln x - x$. That gives $n \ln n - n$ with an error of order $\ln n$. Part of the error is exactly $\frac{1}{2} \ln n$, which becomes the $\sqrt{n}$ in Stirling's formula. What remains changes from n to $n+1$ by an amount that is always positive and adds up to a finite total, so it settles to a constant. A classical integral of $\sin^n$, due to Wallis, then shows that the constant is $\ln \sqrt{2\pi}$.

E.2.3. Derivation

1. **The ratio of neighbours.** Dividing the binomial formula at $k+1$ by the one at k, most factors cancel:
$$\frac{p_{k+1}}{p_k} = \frac{n-k}{k+1} \cdot \frac{p}{\bar{p}}. \tag{E.11}$$
As k grows the numerator $n-k$ falls and the denominator $k+1$ rises, so the ratio decreases. It is above 1 exactly when $(n-k)p > (k+1)\bar{p}$, which rearranges to $k < (n+1)p - 1$. So the probabilities rise while $k+1 < (n+1)p$ and do not rise after. The last count reached by a rise is the largest whole number not above $(n+1)p$, so the largest probability is at $k^* = \lfloor (n+1)p \rfloor$. When $(n+1)p$ is a whole number the ratio there is exactly 1, and $k^* - 1$ ties with k^*.

2. **Shifting by one costs the height of the peak.** This is the argument of Section 9.5, which step 1 completes. The shifted list is the distribution of $X+1$, with $\Pr[X+1=k] = p_{k-1}$, and the TV distance is the total of $p_{k-1} - p_k$ over the k where it is positive. By step 1 those are exactly the k after the peak, $k = k^*+1, \dots, n+1$, with $p_{n+1} = 0$. In the sum $(p_{k^*} - p_{k^*+1}) + (p_{k^*+1} - p_{k^*+2}) + \cdots + (p_n - p_{n+1})$, each negative part cancels the positive part of the next difference, and only p_{k^*} survives. So the TV distance equals $\max_k p_k$.

3. **A rough Stirling formula.** The function $\ln x$ increases, so on the interval $[k-1, k]$ it lies below $\ln k$, and on $[k, k+1]$ it lies above. Summing over $k = 2, \dots, n$ therefore gives
$$\int_1^n \ln x \, dx \le \ln n! \le \int_1^{n+1} \ln x \, dx, \tag{E.12}$$
since $\ln 1 = 0$. The integral of $\ln x$ is $x \ln x - x$, so the two sides are $n \ln n - n + 1$ and $(n+1)\ln(n+1) - n$. The upper side exceeds $n \ln n - n$ by $\ln(n+1) + n \ln(1 + 1/n)$, and the second part is at most 1 because $\ln(1+u) \le u$. So $\ln n!$ is $n \ln n - n$ up to an error between 0 and $\ln(n+1) + 1$, which gives the form Section 13.3 uses:
$$\ln n! = n \ln n - n + O(\ln n). \tag{E.13}$$

4. **The error settles to a constant.** Define $d_n = \ln n! - (n + \frac{1}{2}) \ln n + n$, the error left after taking out $n \ln n - n$ and also $\frac{1}{2} \ln n$. The extra $\frac{1}{2} \ln n$ is chosen because, as the next lines show, it makes the remaining error change by a summable amount. Subtracting d_{n+1} from d_n, the factorials leave $-\ln(n+1)$ and most terms cancel:
$$d_n - d_{n+1} = \left(n + \frac{1}{2}\right) \ln \frac{n+1}{n} - 1. \tag{E.14}$$
Write $u = 1/(2n+1)$. Then $\frac{n+1}{n} = \frac{1+u}{1-u}$ and $n + \frac{1}{2} = \frac{1}{2u}$. The power series $\ln \frac{1+u}{1-u} = 2(u + \frac{u^3}{3} + \frac{u^5}{5} + \cdots)$ comes from subtracting the series of $\ln(1-u)$ from that of $\ln(1+u)$, where the even powers cancel. Multiplying by $\frac{1}{2u}$ and subtracting 1 gives
$$d_n - d_{n+1} = \frac{u^2}{3} + \frac{u^4}{5} + \frac{u^6}{7} + \cdots. \tag{E.15}$$
Every term is positive, so d_n decreases. The denominators 5, 7, … are larger than 3, so the sum is at most $\frac{u^2}{3}(1 + u^2 + u^4 + \cdots) = \frac{u^2}{3(1-u^2)}$. Since $1 - u^2 = \frac{4n(n+1)}{(2n+1)^2}$, this equals $\frac{1}{12n(n+1)}$,

which is $\frac{1}{12}(\frac{1}{n} - \frac{1}{n+1})$. Adding these steps from n onwards, the right sides telescope, so $d_n - d_{n+k} < \frac{1}{12n}$ for every k. A decreasing sequence that never drops more than a fixed amount has a limit, so d_n tends to a constant d_∞, with $0 < d_n - d_\infty \le \frac{1}{12n}$. This is Equation E.8 with $r_n = d_n - d_\infty$, once $d_\infty = \ln\sqrt{2\pi}$.

5. **Wallis's integrals.** Let $I_n = \int_0^{\pi/2} \sin^n x\,dx$, so $I_0 = \pi/2$ and $I_1 = 1$. Integrating by parts, with $\sin^n x = \sin^{n-1} x \cdot \sin x$, gives $I_n = (n-1)\int_0^{\pi/2} \sin^{n-2} x \cos^2 x\,dx$. Writing $\cos^2 x = 1 - \sin^2 x$ turns this into $(n-1)(I_{n-2} - I_n)$, so $I_n = \frac{n-1}{n} I_{n-2}$. Running this down to I_0 or I_1 gives products of fractions such as $\frac{1\cdot3\cdots(2j-1)}{2\cdot4\cdots2j}$, which equals $\binom{2j}{j}4^{-j}$. So

$$I_{2j} = \frac{\pi}{2}\binom{2j}{j}4^{-j}, \qquad I_{2j+1} = \frac{1}{(2j+1)\binom{2j}{j}4^{-j}}. \tag{E.16}$$

Since $0 \le \sin x \le 1$, the integrals decrease in n. So $1 \le I_{2j}/I_{2j+1} \le I_{2j-1}/I_{2j+1} = \frac{2j+1}{2j}$, and the ratio tends to 1. The ratio is $\frac{\pi}{2}(2j+1)\left(\binom{2j}{j}4^{-j}\right)^2$, so

$$\binom{2j}{j}4^{-j} = \frac{1 + o(1)}{\sqrt{\pi j}}, \tag{E.17}$$

where $o(1)$ is a quantity that tends to 0 as j grows.

6. **The constant.** Write $\ln\binom{2j}{j} = \ln(2j)! - 2\ln j!$ and replace each $\ln n!$ by $d_n + (n+\frac{1}{2})\ln n - n$, which is the definition of step 4 solved for $\ln n!$. Writing $\ln(2j) = \ln 2 + \ln j$, the terms in $j\ln j$ and j cancel, and what is left is

$$\ln\left(\binom{2j}{j}4^{-j}\right) = d_{2j} - 2d_j + \tfrac{1}{2}\ln 2 - \tfrac{1}{2}\ln j. \tag{E.18}$$

As j grows, $d_{2j} - 2d_j \to -d_\infty$. By Equation E.17 the left side is $-\frac{1}{2}\ln(\pi j) + o(1)$. Matching the two gives $-d_\infty + \frac{1}{2}\ln 2 = -\frac{1}{2}\ln\pi$, so $d_\infty = \frac{1}{2}\ln(2\pi) = \ln\sqrt{2\pi}$.

7. **The fraction recovered, exactly.** For $K \sim \mathrm{Bin}(m, \frac{1}{2})$ one identity does all the work:

$$\left(k - \frac{m}{2}\right)\binom{m}{k} = \frac{m}{2}\left[\binom{m-1}{k-1} - \binom{m-1}{k}\right]. \tag{E.19}$$

It holds because $\binom{m-1}{k-1} = \frac{k}{m}\binom{m}{k}$ and $\binom{m-1}{k} = \frac{m-k}{m}\binom{m}{k}$. By symmetry about $\frac{m}{2}$, $\mathbb{E}|K - \frac{m}{2}|$ is twice the sum of $(k - \frac{m}{2})\binom{m}{k}2^{-m}$ over $k > \frac{m}{2}$. Let k_0 be the first whole number above $\frac{m}{2}$. Replacing each term by the identity, the sum from k_0 to m telescopes as in step 2, because the second binomial of each term cancels the first binomial of the next, and only $\frac{m}{2}\binom{m-1}{k_0-1}2^{-m}$ is left. So $\mathbb{E}|K - \frac{m}{2}|/m = \binom{m-1}{k_0-1}2^{-m}$. Now write $j = \lfloor m/2 \rfloor$. When $m = 2j$ is even, $k_0 = j+1$ and $\binom{2j-1}{j} = \frac{1}{2}\binom{2j}{j}$. When $m = 2j+1$ is odd, $k_0 = j+1$ again and $\binom{2j}{j}2^{-2j-1}$ appears directly. Either way $\mathbb{E}|K - \frac{m}{2}|/m = \frac{1}{2}\binom{2j}{j}4^{-j}$, which is Equation E.9. For $m = 10$ the fraction recovered is $\frac{1}{2} + \frac{1}{2}\cdot 252/1024 = 0.62305$. By Equation E.17 and $j \approx m/2$, the excess over $\frac{1}{2}$ is $\frac{1+o(1)}{2\sqrt{\pi m/2}} = \frac{1+o(1)}{\sqrt{2\pi m}}$, which is the chapter's approximation, now proved.

8. **The height of the peak for any p.** Write $k = k^*$ for the mode and $a = k/n$ for the fraction of successes there. Replace each of $n!$, $k!$ and $(n-k)!$ in p_k by Equation E.8. The square roots give $\sqrt{n/(2\pi k(n-k))}$, and the powers n^n, k^k, $(n-k)^{n-k}$, p^k and $\bar{p}^{n-k}$ combine into $\exp\big(-k\ln\frac{k}{np} - (n-k)\ln\frac{n-k}{n\bar{p}}\big)$. Multiplying by $\sqrt{2\pi np\bar{p}}$ and grouping gives three factors:

$$p_k\sqrt{2\pi np\bar{p}} = \underbrace{\sqrt{\frac{p\bar{p}}{a(1-a)}}}_{A} \cdot \underbrace{e^{-n\,\mathrm{kl}(a\|p)}}_{B} \cdot \underbrace{e^{r_n - r_k - r_{n-k}}}_{R}, \tag{E.20}$$

where $\mathrm{kl}(a\|p) = a\ln\frac{a}{p} + (1-a)\ln\frac{1-a}{\bar{p}}$ is the Kullback–Leibler (KL) divergence between coins with success probabilities a and p, which is never negative. The mode is within one count of the mean. By step 1, k^* lies between $(n+1)p - 1$ and $(n+1)p$, so the offset $\xi = k^* - np$ satisfies $|\xi| < 1$. Each factor is then close to 1.

- *Factor B.* The logarithm is concave, so by Jensen's inequality $\mathrm{kl}(a\|p) \le \ln\big(\frac{a^2}{p} + \frac{(1-a)^2}{\bar{p}}\big) = \ln\big(1 + \frac{(a-p)^2}{p\bar{p}}\big) \le \frac{(a-p)^2}{p\bar{p}}$. With $a - p = \xi/n$, this gives $0 \le -\ln B = n\,\mathrm{kl}(a\|p) \le \xi^2/(np\bar{p}) < 1/(np\bar{p})$.
- *Factor R.* By Equation E.8, $|\ln R| \le \frac{1}{12}\big(\frac{1}{n} + \frac{1}{k} + \frac{1}{n-k}\big)$. When $np\bar{p} \ge 4$, $k > np - 1 \ge np/2$ and $n - k > n\bar{p} - 1 \ge n\bar{p}/2$, so $|\ln R| \le \frac{1}{12}\cdot\frac{5}{np\bar{p}}$, using $\frac{1}{n}, \frac{1}{np}, \frac{1}{n\bar{p}} \le \frac{1}{np\bar{p}}$.
- *Factor A.* With $a = p + \xi/n$ and $1 - a = \bar{p} - \xi/n$, multiplying out gives $a(1-a) = p\bar{p}(1+x)$ with $x = \frac{\xi(\bar{p}-p)}{np\bar{p}} - \frac{\xi^2}{n^2 p\bar{p}}$, so $|x| \le \frac{2}{np\bar{p}} \le \frac{1}{2}$. For $|x| \le \frac{1}{2}$, $|\ln(1+x)| \le 2|x|$, so $|\ln A| = \frac{1}{2}|\ln(1+x)| \le \frac{2}{np\bar{p}}$.

Adding the three logarithms gives $(2 + 1 + \frac{5}{12})/(np\bar{p}) < 3.5/(np\bar{p})$, which is Equation E.10. With $n-1$ trials and the chapter's subgroup share t as the success probability, it gives Equation 12.7, and for $p = \frac{1}{2}$ the height $\sqrt{2/(\pi m)}$ of Section 9.5.

E.2.4. Checking it

Figure E.3 shows step 2 for $\mathrm{Bin}(10, \frac{1}{2})$. The green bars, where the shifted list is larger, start right after the peak and add up to 0.2461, the height of the mode. For $\mathrm{Bin}(100, \frac{1}{2})$ the same total is 0.0796, the value in Section 9.5. The script also checks step 1 by brute force. For every $n \le 60$ and eight values of p, $\lfloor (n+1)p \rfloor$ is a most likely count in all 480 cases.

Stirling's error is close to its bound everywhere (Figure E.4). At $n = 10$ the error r_n is 8.331×10^{-3} against $1/(12n) = 8.333 \times 10^{-3}$, and at $n = 1$ it is 0.0811 against 0.0833. So $1/(12n)$ is nearly the exact error, not just a bound. The rough bounds of step 3 are far weaker, since at $n = 10$ they only place $\ln 10! = 15.1044$ between 14.0259 and 16.3768.

For the fraction of bits recovered, Equation E.9 gives 0.62305, 0.53979 and 0.51261 for $m = 10$, 100 and 1000, matching the sums over the binomial table to the digits shown. The approximation $\frac{1}{2} + 1/\sqrt{2\pi m}$ gives 0.62616, 0.53989 and 0.51262. The script also confirms that $m = 11$ recovers the same fraction as $m = 10$, as Equation E.9 says. The ratio $\binom{2j}{j}4^{-j}\sqrt{\pi j}$ is 0.97535 at $j = 5$ and 0.99750 at $j = 50$, so it approaches 1 with an error close to $1/(8j)$.

Figure E.5 shows Equation E.10. The ratio of the mode height to $1/\sqrt{2\pi np\bar{p}}$ approaches 1 for $p = 0.5$, 0.2 and 0.05. At $n = 100$ it is 0.9975, 0.9956 and 0.9834. The right panel shows that the error falls like $1/(np\bar{p})$. The constant 3.5 of the proof is loose: over every $n \le 20{,}000$ and six values of p with $np\bar{p} \ge 4$, the script finds $np\bar{p}\,|\ln(\text{ratio})|$ at most 0.187.

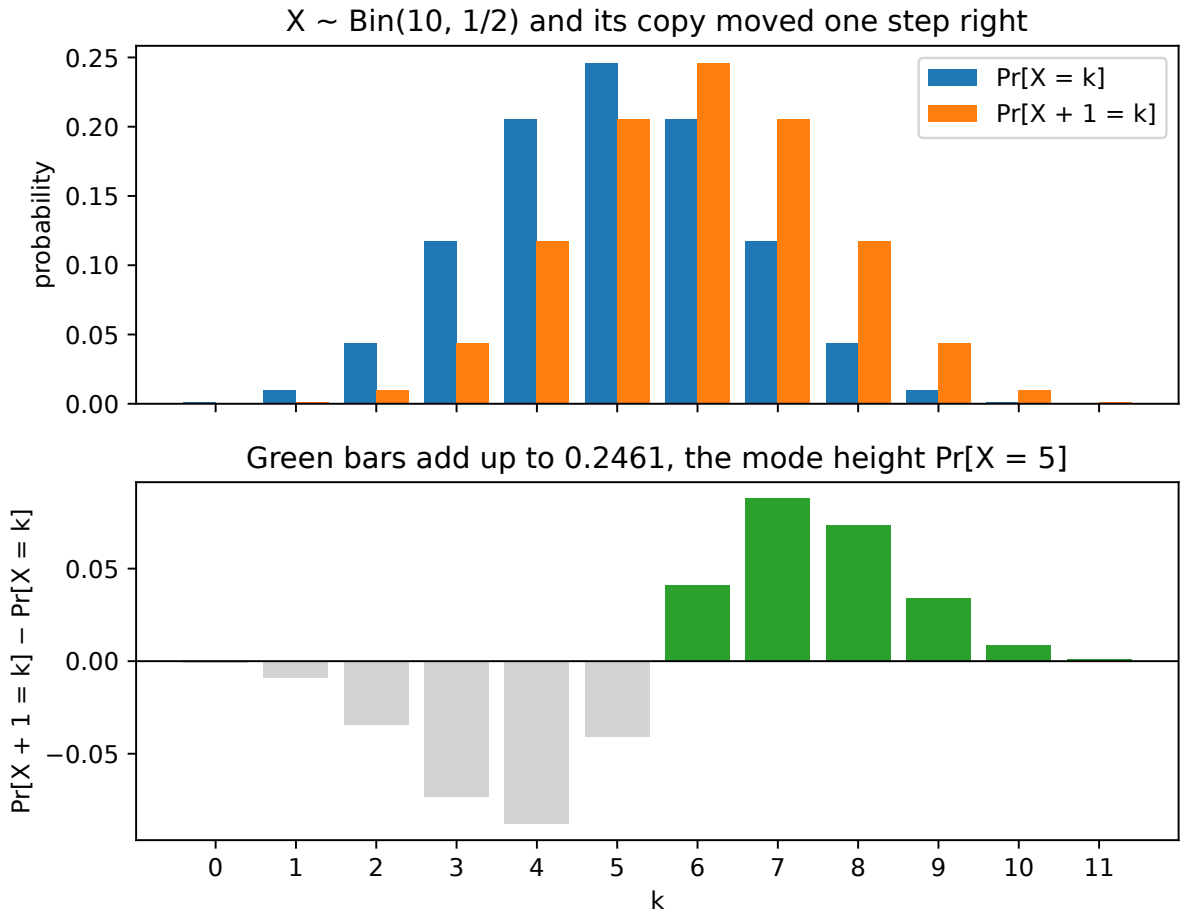


Figure E.3.: Top: the Bin(10, $\frac{1}{2}$) probabilities and the same list moved one place right. Bottom: their difference at each k. The positive differences (green) all lie after the peak and add up to the height of the mode.

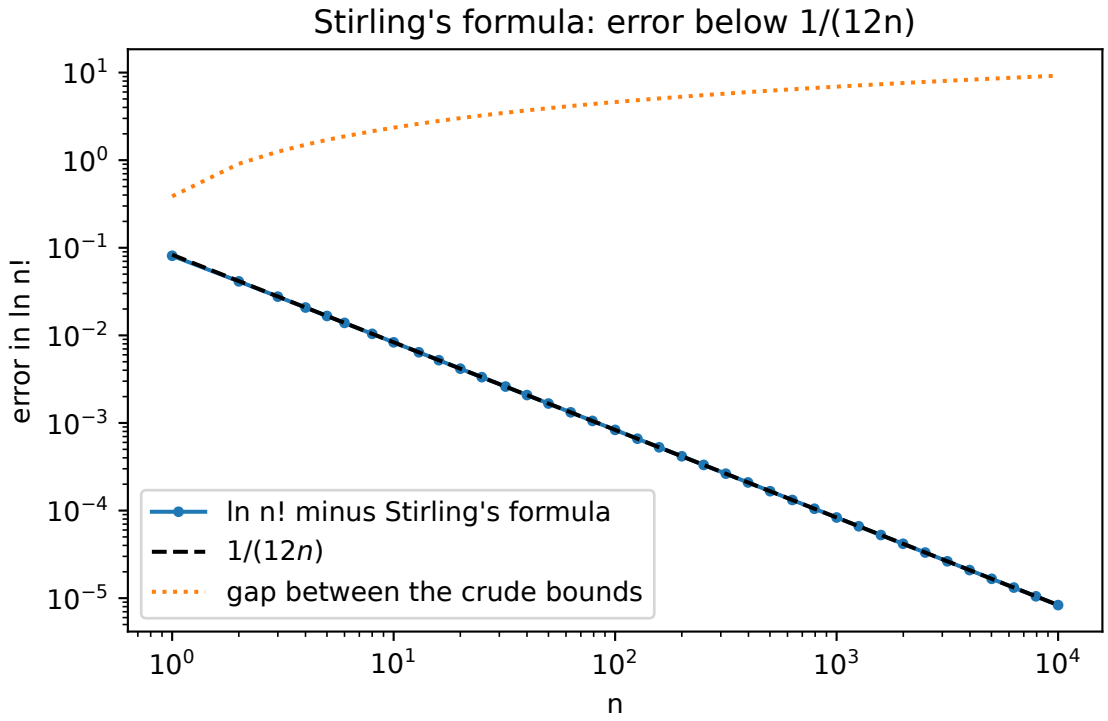


Figure E.4.: The error of Stirling's formula, $r_n = \ln n! - \ln\left(\sqrt{2\pi n}(n/e)^n\right)$, against n on log axes, with the bound $1/(12n)$ and the gap between the rough bounds of step 3.

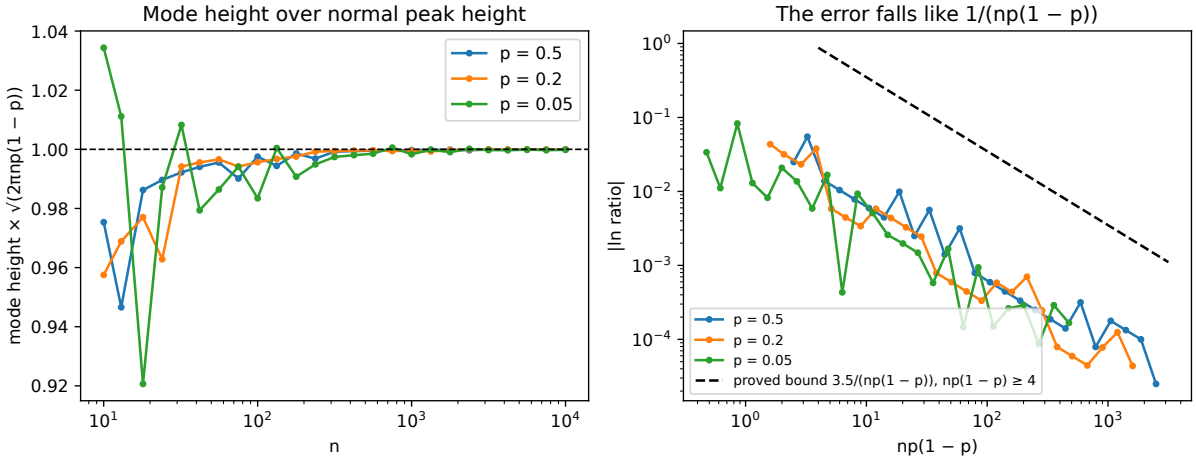


Figure E.5.: Left: the height of the binomial mode divided by the normal peak height $1/\sqrt{2\pi np\bar{p}}$, against n, for three values of p. Right: the absolute logarithm of that ratio against $np\bar{p}$, with the proved bound $3.5/(np\bar{p})$.

E.3. How fast a Poisson count falls far above its mean

E.3.1. What the chapter uses

Section 1.4 in Probability distributions defines a Poisson count $N \sim \text{Poisson}(\mu)$ with mean $\mu > 0$. It takes each value $k = 0, 1, 2, ...$ with probability

$$\Pr[N = k] = e^{-\mu}\frac{\mu^k}{k!}. \tag{E.21}$$

The chapter says that far above the mean these probabilities fall like $(e\mu/k)^k$, faster than any exponential but slower than the tail of a normal variable with the same variance, and it points here for the proof. This section proves that for every count $k \geq 1$,

$$\Pr[N = k] \leq \left(\frac{e\mu}{k}\right)^k. \tag{E.22}$$

It then shows that the whole tail beyond k obeys the same bound up to a constant factor, once k is larger than $e\mu$:

$$\Pr[N \geq k] \leq \frac{(e\mu/k)^k}{1 - e\mu/k}. \tag{E.23}$$

E.3.2. The idea

The probability of the count k has $k!$ in its denominator, so how fast the tail falls depends on how fast $k!$ grows. A lower bound on $k!$ gives an upper bound on the probability, and the lower bound comes from the power series of the exponential. That series writes e^k as a sum of positive terms $k^j/j!$, one for each $j = 0, 1, 2, ...$, and the term with $j = k$ is $k^k/k!$. A sum of positive terms is at least any one of its terms, so $k^k/k!$ is at most e^k. For $k = 3$, for example, the term is $27/6 = 4.5$, while the whole sum is $e^3 \approx 20.09$.

For the whole tail, each probability beyond k is at most the matching power of the single number $e\mu/k$. Once that number is below 1, the powers form a geometric series, and a geometric series adds up to its first term divided by one minus its ratio.

E.3.3. Derivation

1. **A lower bound on the factorial.** The power series of the exponential at k is $e^k = \sum_{j\geq 0} k^j/j!$. Every term is positive, so the sum is at least its term $j = k$, which is $k^k/k!$. Solving $k^k/k! \leq e^k$ for $k!$ gives
$$k! \geq \left(\frac{k}{e}\right)^k. \tag{E.24}$$
2. **Bound one probability.** Replacing $k!$ in the denominator by this smaller number can only make the fraction larger, so
$$\Pr[N = k] \leq e^{-\mu}\mu^k\left(\frac{e}{k}\right)^k. \tag{E.25}$$

3. **Drop the constant.** The mean μ is positive, so the factor $e^{-\mu}$ is below 1, and dropping it gives Equation E.22.
4. **Faster than any exponential.** To compare the bound with an exponential decay, write it as an exponential, using $x^k = e^{k \ln x}$ and $\ln(1/x) = -\ln x$:

$$\left(\frac{e\mu}{k}\right)^k = e^{-k \ln(k/(e\mu))}. \tag{E.26}$$

   The exponent is k times a logarithm that keeps growing with k. So for any constant $c > 0$, once $k > e\mu\, e^c$ the logarithm exceeds c and the bound is below e^{-ck}. That is what "faster than any exponential" means: the exponent grows like $k \ln k$, while that of e^{-ck} grows only like k.
5. **A matching lower bound.** The comparison with the normal needs to know that the probabilities are not much smaller than the bound. The factorial $k! = 1 \cdot 2 \cdots k$ is a product of k numbers, none larger than k, so $k! \le k^k$. Putting this larger number in the denominator gives

$$\Pr[N = k] \ge e^{-\mu} \left(\frac{\mu}{k}\right)^k = e^{-\mu - k \ln(k/\mu)}. \tag{E.27}$$

   So the exponent of $\Pr[N = k]$ lies between $k \ln(k/(e\mu))$ and $\mu + k \ln(k/\mu)$, and both grow like $k \ln k$.
6. **Slower than a normal.** A normal variable with mean μ and variance μ, the Poisson's own mean and variance, has density proportional to $e^{-(k-\mu)^2/(2\mu)}$ at k. Expanding the square, the leading term of the exponent is $k^2/(2\mu)$, so far out its tail falls like $e^{-k^2/(2\mu)}$. An exponent that grows like k^2 eventually overtakes one that grows like $k \ln k$, so by step 5 the Poisson probabilities fall more slowly than the normal tail.
7. **The whole tail.** Assume $k > e\mu$ and write $q = e\mu/k$, which is then below 1. The tail $\Pr[N \ge k]$ is the sum of $\Pr[N = j]$ over $j \ge k$, and by Equation E.22 each term is at most $(e\mu/j)^j$. For $j \ge k$ the base $e\mu/j$ is at most q, so each term is at most q^j. These terms form a geometric series with first term q^k and ratio q, so

$$\Pr[N \ge k] \le \sum_{j \ge k} q^j = \frac{q^k}{1 - q}, \tag{E.28}$$

   which is Equation E.23. Once $k \ge 2e\mu$, the ratio q is at most $\frac{1}{2}$, and the factor $1/(1-q)$ is at most 2.

E.3.4. Checking it

Take $\mu = 4$, the chapter's example of a count with standard deviation 2. Then $e\mu \approx 10.87$, so the tail bound Equation E.23 applies from $k = 11$, and its factor $1/(1-q)$ is at most 2 from $k = 22$. Table E.3 compares the bounds with the exact values at $k = 20$ and $k = 40$.

Table E.3.: Exact probabilities of $N \sim \text{Poisson}(4)$ against the two bounds of this section.

k	$\Pr[N = k]$	Bound Equation E.22	Ratio	$\Pr[N \geq k]$	Bound Equation E.23	Ratio
20	8.28×10^{-9}	5.09×10^{-6}	615	1.02×10^{-8}	1.11×10^{-5}	1093
40	2.71×10^{-26}	2.35×10^{-23}	867	3.01×10^{-26}	3.23×10^{-23}	1075

The bound on one probability is loose by a factor of several hundred, and the loss is easy to account for. By Stirling's formula Equation E.8, $k! = \sqrt{2\pi k}\,(k/e)^k e^{r_k}$, so step 1 drops the factor $\sqrt{2\pi k}\,e^{r_k}$ and step 3 drops e^{μ}. Their product is exactly the ratio in the table: $e^4\sqrt{40\pi} \approx 612.0$ at $k = 20$, and the factor $e^{r_{20}}$, which is at most $e^{1/240}$, brings it to 614.6. That factor grows only like $\sqrt{k}$, so in the exponent it is a small constant beside $k\ln(k/(e\mu))$, and the bound has the right rate. The tail bound adds the factor $1/(1-q)$, which is 2.19 at $k = 20$ and 1.37 at $k = 40$.

The comparison with the normal is stark. Take the normal variable with the same mean 4 and standard deviation 2. It lies above 20, eight standard deviations out, with probability about 6.22×10^{-16}, while the exact Poisson tail there is 1.02×10^{-8}. Above 40 the normal gives 9.74×10^{-73}, against 3.01×10^{-26} for the Poisson count. So a normal approximation to a count understates its far tail by many orders of magnitude. Figure E.6 draws all of these against k on a log scale. The two bounds run parallel to the exact tail, a constant factor above it, while the normal tail bends away below it.

E.3.5. What it does not cover

The Chernoff method of Concentration inequalities gives a sharper tail bound. It bounds the tail by the moment generating function $\exp(\mu(e^{\lambda} - 1))$ of the count, at the value of the free parameter λ that makes the bound smallest. Steps 3 and 4 of the proof of Theorem 2.4 do exactly that for the Poisson MGF, and give, for every $k > \mu$,

$$\Pr[N \geq k] \leq e^{-\mu}\left(\frac{e\mu}{k}\right)^k. \tag{E.29}$$

That keeps the factor $e^{-\mu}$ that step 3 here drops, and it does not need $k > e\mu$. For the count with mean 4 it gives 9.32×10^{-8} at $k = 20$, about 9 times the exact tail. This section's bound is the simpler one, and it shows where the rate $(e\mu/k)^k$ comes from: the growth of $k!$.

E.4. The central limit theorem, by swapping one term at a time

E.4.1. What the chapter uses

The normal section of Probability distributions states the CLT, illustrates it with sums of exponential variables, and cites the Berry–Esseen theorem (Shevtsova, 2011) for its rate. It gives no proof. Almost every normal approximation in the book rests on it, including the CLT for composition in Section 32.5, where the summands are not identically distributed.

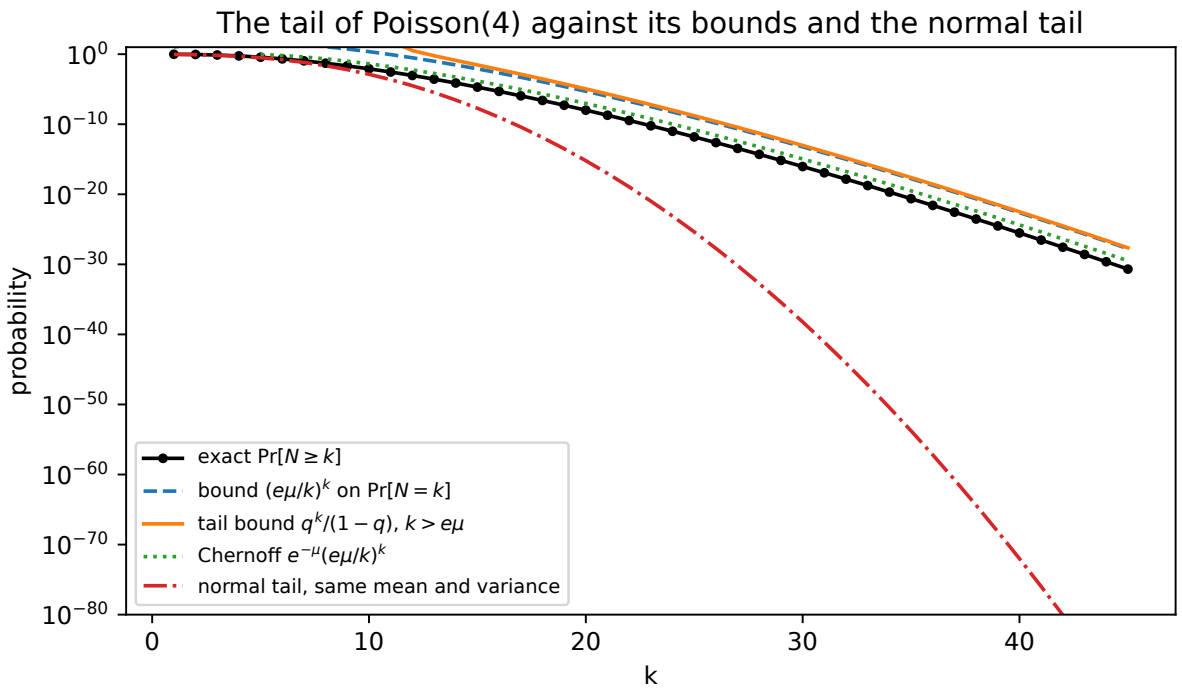


Figure E.6.: The tail $\Pr[N \geq k]$ of $N \sim \text{Poisson}(4)$ on a log scale (black), with the bound Equation E.22 on one probability (dashed), the tail bound Equation E.23 from $k = 11$ (orange), the Chernoff bound $e^{-\mu}(e\mu/k)^k$ (dotted), and the tail of a normal variable with mean 4 and variance 4 (dash-dotted).

Take independent $X_1, \dots, X_n$ with means μ_i, variances σ_i^2, and finite third moments. Write $Y_i = X_i - \mu_i$ for the centred terms and $s^2 = \sigma_1^2 + \cdots + \sigma_n^2$ for the variance of their sum. The quantity that controls the error is the **Lyapunov ratio**

$$L = \frac{\mathbb{E}|Y_1|^3 + \cdots + \mathbb{E}|Y_n|^3}{s^3}. \tag{E.30}$$

For n copies of one distribution with variance σ^2, it is $L = \mathbb{E}|X - \mu|^3/(\sigma^3\sqrt{n})$, which falls like $1/\sqrt{n}$. The theorem proved here says that the standardized sum $S = (Y_1 + \cdots + Y_n)/s$ has a CDF within $2L^{1/4}$ of the standard normal CDF Φ:

$$\big|\Pr[S \le z] - \Phi(z)\big| \le 2L^{1/4} \quad \text{for every } z. \tag{E.31}$$

So the CDF gap tends to 0 whenever L does, at least as fast as $n^{-1/8}$ for identical terms. That proves the CLT when the third moments are finite.

E.4.2. The idea

Compare the sum with a sum of normals, one term at a time. Next to each Y_i, put a normal variable G_i with the same mean 0 and the same variance σ_i^2. The sum of all the G_i, divided by s, is exactly $\mathcal{N}(0,1)$, by the normal-sum rule of Section E.1. Now swap the terms one at a time. Start with all normals, replace G_1 by Y_1, then G_2 by Y_2, and so on, until only the Y_i remain. The total change is the sum of the n changes from single swaps, so it is enough to show that each swap changes little.

A probability $\Pr[S \le z]$ is the average of an indicator, which jumps, and jumps are hard to expand. So the proof first compares averages $\mathbb{E}h(S)$ of a smooth **test function** h, and handles the indicator at the end. A single swap changes such an average little, because Y_i and G_i agree in their first two moments. Expand $h(W + Y_i/s)$ around the rest of the sum W as a Taylor series. The first-order and second-order terms depend on Y_i only through its mean and variance, so they are the same for Y_i and G_i and cancel. What is left is a third-order term of size $(\mathbb{E}|Y_i|^3 + \mathbb{E}|G_i|^3)/s^3$. There are n swaps of size about $n^{-3/2}$ each, so the total is about $n^{-1/2}$. The last step squeezes the indicator between two smooth ramps and pays for the width of the ramp.

E.4.3. Derivation

1. **The hybrids.** Let $G_1, \dots, G_n$ be independent normal variables, independent of the X_i, with $G_i \sim \mathcal{N}(0, \sigma_i^2)$. For $k = 0, \dots, n$, let

   $$H_k = \frac{Y_1 + \cdots + Y_k + G_{k+1} + \cdots + G_n}{s}. \tag{E.32}$$

   Then $H_n = S$, and $H_0 = (G_1 + \cdots + G_n)/s$ is exactly $\mathcal{N}(0,1)$ by step 5 of Section E.1, since its variance is $s^2/s^2 = 1$. Write $Z = H_0$.

2. **Telescoping.** For any test function h, the change from Z to S is a sum of single swaps, because the middle terms cancel in pairs:

   $$\mathbb{E}h(S) - \mathbb{E}h(Z) = \sum_{k=1}^{n} \big(\mathbb{E}h(H_k) - \mathbb{E}h(H_{k-1})\big). \tag{E.33}$$

3. **One swap.** The hybrids H_k and H_{k-1} differ only in term k. So if W stands for the other terms divided by s, then $H_k = W + Y_k/s$ while $H_{k-1} = W + G_k/s$. The rest W is independent of both Y_k and G_k. Now suppose that h has two derivatives, and that its second derivative h'' changes by at most $\gamma|a-b|$ between any two points a and b, for a constant γ. Taylor's theorem with this condition says that, for every w and y,

$$\left|h(w+y) - h(w) - h'(w)y - \tfrac{1}{2}h''(w)y^2\right| \le \frac{\gamma}{6}|y|^3. \tag{E.34}$$

   Apply it with $y = Y_k/s$, take expectations, and do the same with G_k. The term $h(W)$ is common to both. The first-order term averages to 0 for both, because W is independent of Y_k and Y_k has mean 0, so $\mathbb{E}[h'(W)Y_k] = \mathbb{E}[h'(W)]\,\mathbb{E}[Y_k] = 0$, and likewise for G_k. The second-order term averages to $\frac{1}{2}\mathbb{E}[h''(W)]\,\sigma_k^2$ for both, because Y_k and G_k have the same variance. So these terms cancel and

$$\left|\mathbb{E}h(H_k) - \mathbb{E}h(H_{k-1})\right| \le \frac{\gamma}{6s^3}(\mathbb{E}|Y_k|^3 + \mathbb{E}|G_k|^3). \tag{E.35}$$

4. **Sum the swaps.** For a normal variable, $\mathbb{E}|G_k|^3 = 2\sqrt{2/\pi}\,\sigma_k^3 \approx 1.596\,\sigma_k^3$, which you get by integrating $|x|^3$ against the normal density. And $\sigma_k^3 = (\mathbb{E}Y_k^2)^{3/2} \le \mathbb{E}|Y_k|^3$ by Jensen's inequality, because $x \mapsto x^{3/2}$ is convex. So each swap costs at most $\frac{\gamma}{6s^3}(1 + 2\sqrt{2/\pi})\mathbb{E}|Y_k|^3$. By step 2, with $c = (1 + 2\sqrt{2/\pi})/6 \approx 0.433$,

$$\left|\mathbb{E}h(S) - \mathbb{E}h(Z)\right| \le c\,\gamma\,L. \tag{E.36}$$

5. **A smooth ramp.** The function $\rho(u) = 6u^5 - 15u^4 + 10u^3$ rises from $\rho(0) = 0$ to $\rho(1) = 1$. Its first two derivatives are 0 at both ends, and its third derivative is at most 60 in size on $[0,1]$. Set $\rho = 0$ below 0 and $\rho = 1$ above 1. For a width $\kappa > 0$ and a point z, the upper ramp $h^+(x) = 1 - \rho((x-z)/\kappa)$ is 1 up to z and 0 beyond $z + \kappa$. Its second derivative changes by at most $\gamma = 60/\kappa^3$ per unit, so Equation E.36 applies with that γ. The lower ramp $h^-(x) = h^+(x + \kappa)$ is the same ramp moved left by κ.

6. **Sandwich the indicator.** At every point, $h^-(x) \le \mathbf{1}[x \le z] \le h^+(x)$. Averaging keeps the order, so

$$\Pr[S \le z] \le \mathbb{E}h^+(S) \le \mathbb{E}h^+(Z) + c\gamma L \le \Phi(z + \kappa) + c\gamma L. \tag{E.37}$$

   The last step uses $h^+ \le \mathbf{1}[x \le z + \kappa]$. The normal density is at most $1/\sqrt{2\pi}$, so $\Phi(z+\kappa) \le \Phi(z) + \kappa/\sqrt{2\pi}$. The lower ramp gives the matching bound from below, so

$$\left|\Pr[S \le z] - \Phi(z)\right| \le \frac{\kappa}{\sqrt{2\pi}} + \frac{60\,c\,L}{\kappa^3}. \tag{E.38}$$

7. **Choose the width.** A narrow ramp fits the indicator closely but has a large third derivative. The best balance sets the derivative in κ to zero, which gives $\kappa^4 = 180\,c\,L\sqrt{2\pi}$. At that width the second term is a third of the first, and the bound is $\frac{4}{3}\kappa/\sqrt{2\pi} = 1.99\,L^{1/4}$. That proves Equation E.31.

Many dimensions. Section G.6 and other sections use the CLT for random vectors. It follows from the one-dimensional theorem by the Cramér–Wold device: a sequence of random vectors converges in distribution to a normal vector if and only if every fixed linear combination of the coordinates converges to the matching normal variable. Each linear combination is a sum of independent scalar terms, so the theorem above applies to it. The Cramér–Wold device itself rests on a uniqueness theorem for the Fourier transform of a distribution, a relative of the MGF, so it stays cited (Billingsley, 1995).

E.4.4. Checking it

Figure E.7 follows the swaps for $n = 30$ Exp(1) summands, with $h = \sin$, whose third derivative is at most 1 in size. Every hybrid's average $\mathbb{E}\sin(H_k)$ is computed exactly from a closed form, with no simulation. The error grows in steps of at most 0.00122 to a total of 0.0365. Step 3 bounds each step by 0.00407 and the total by 0.1220, about three times the truth. For Exp(1) the bound uses $\mathbb{E}|X-1|^3 = 12/e - 2 = 2.4146$, against $\mathbb{E}|G|^3 = 1.5958$ for the normal.

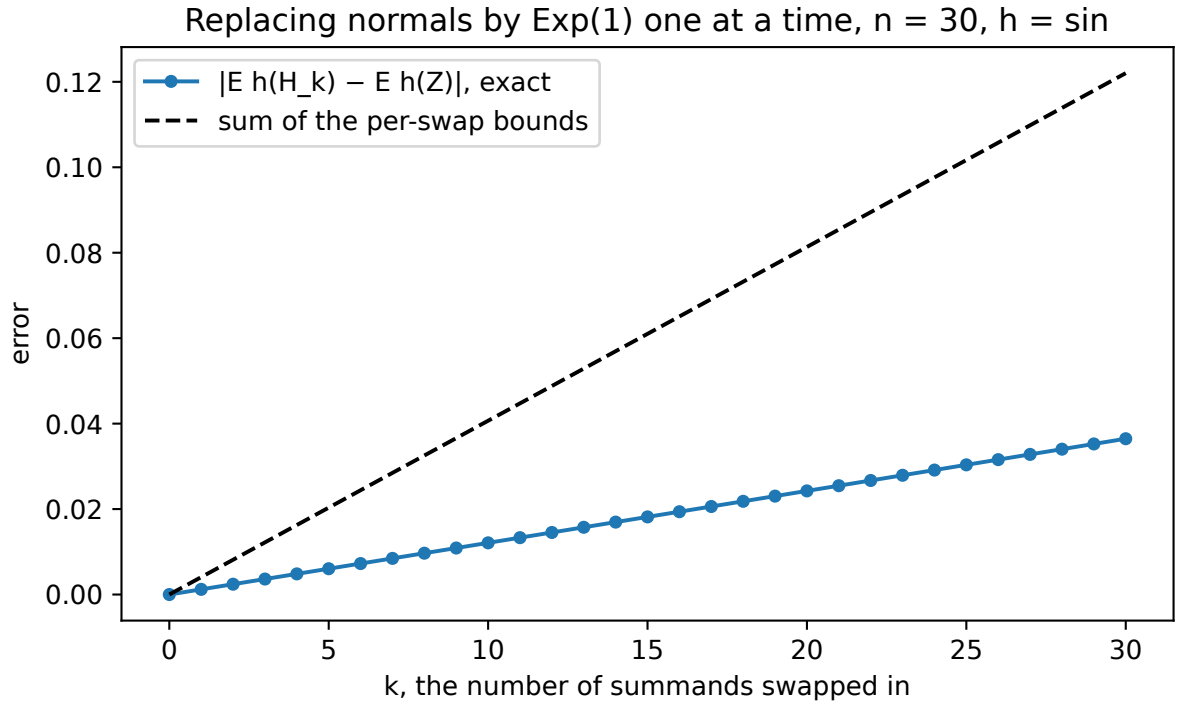


Figure E.7.: The error $|\mathbb{E}\,h(H_k) - \mathbb{E}\,h(Z)|$ after swapping k of $n = 30$ normal terms for centred Exp(1) terms, with $h = \sin$, computed exactly, against the sum of the per-swap bounds of step 3.

Figure E.8 draws the sandwich of step 6 at $z = 0$. The ramps lie on either side of the indicator, and the shaded area under the normal density is the price $\Phi(\kappa) - \Phi(0)$ of a ramp of width $\kappa = 0.5$.

The CDF bound is valid but slow. For 30 Exp(1) terms the Lyapunov ratio is $L = 0.4408$, and Equation E.31 gives 1.6201, which says nothing, while the true largest gap is 0.0243. Even at $n = 10^8$ the bound is still 0.2478. The Berry–Esseen bound $0.48L$ gives 0.2116 at $n = 30$. The proof gets the right answer in the limit, but not the right rate.

E.4.5. What it does not cover

The proof assumes finite third moments, but the CLT holds with finite variance alone. Lindeberg's original argument handles that case by cutting each term into a bounded part and a small remainder. The rate the proof gives is also weak. The Berry–Esseen theorem improves $L^{1/4}$ to a constant times L, with a constant below 0.48 for identical terms (Shevtsova, 2011) and a

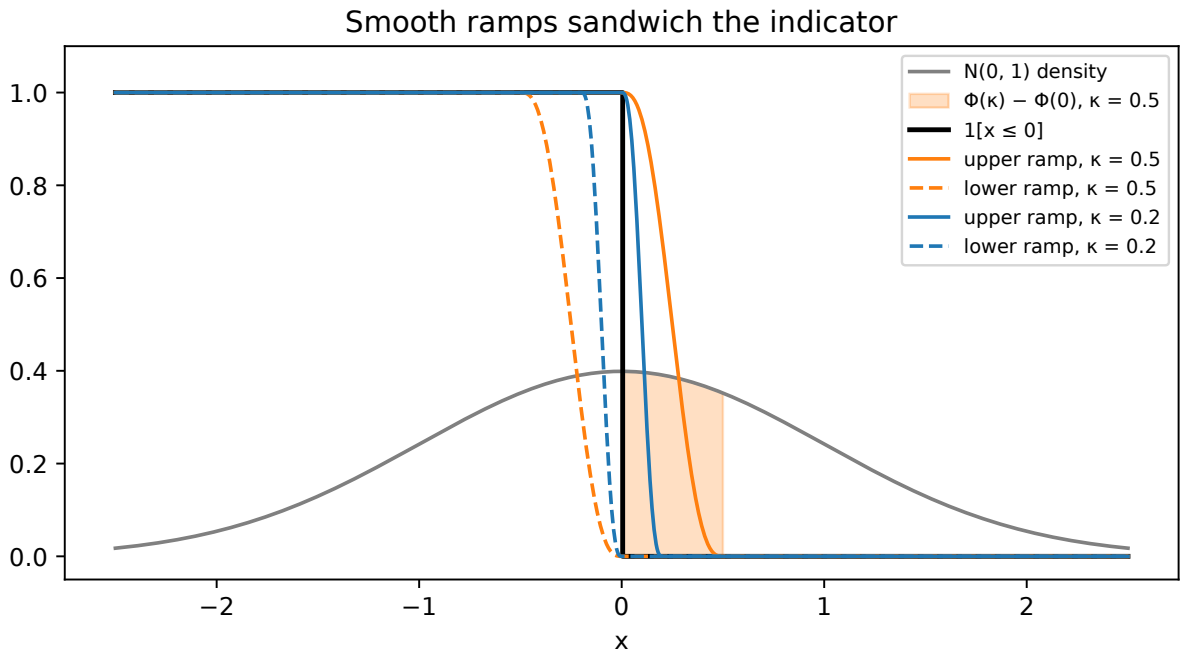


Figure E.8.: The indicator $\mathbf{1}[x \le 0]$ between the upper and lower smooth ramps of widths $\kappa = 0.5$ and 0.2, over the $\mathcal{N}(0,1)$ density. The shaded area is $\Phi(0.5) - \Phi(0)$.

constant of the same size for terms that are not identical. Its proof uses Fourier analysis, so it stays cited. The sandwich loses the rate because a smooth ramp of width κ must pay κ^{-3} in its third derivative.

E.5. The length of a normal vector stays near $\sqrt{d}$

E.5.1. What the chapter uses

Section 28.4 needs the chance that a standard normal vector in d dimensions is much longer than $\sqrt{d}$. Let $Z \sim \mathcal{N}(0, I_d)$, a vector of d independent standard normal coordinates, and let $t \ge 0$. The chapter uses

$$\Pr\left[\|Z\| \ge \sqrt{d} + t\right] \le e^{-t^2/2}. \tag{E.39}$$

It uses the bound to choose two lengths in its algorithm: a radius τ_d that every record stays within, and a margin for the noise in each round of Equation 28.12. The chapter cites Boucheron, Lugosi & Massart (2013, ch. 5), whose proof uses a general theorem for functions that change slowly when their input changes. This section proves Equation E.39 using only the Chernoff method.

E.5.2. The idea

The squared length $S = \|Z\|^2 = Z_1^2 + \cdots + Z_d^2$ is a sum of d independent terms, each with mean 1, so its mean is d. A sum of independent terms is the setting of the Chernoff bound: apply

Markov's inequality to $e^{\lambda(S-d)}$, which splits into a product of d identical factors. Each factor has a closed form, and a short series bound turns it into a simple expression. A well-chosen λ then makes the exponent of the bound exactly $-x$ at the level $2\sqrt{dx}+2x$, for any $x>0$. That gives a bound on the squared length, due to Laurent & Massart (2000). Taking square roots turns it into Equation E.39.

E.5.3. Derivation

1. **One factor.** For $\lambda < \frac{1}{2}$, combining the two exponents in the normal integral gives

$$\mathbb{E}\, e^{\lambda Z_1^2} = \int \frac{e^{-(1-2\lambda)x^2/2}}{\sqrt{2\pi}}\, dx = (1-2\lambda)^{-1/2}. \tag{E.40}$$

   The integrand is a normal density with variance $1/(1-2\lambda)$, without its factor $\sqrt{1-2\lambda}$. So the logarithm of the MGF of $Z_1^2 - 1$ is $-\lambda - \frac{1}{2}\ln(1-2\lambda)$.

2. **A simple upper bound.** The series $-\frac{1}{2}\ln(1-2\lambda) = \sum_{k\geq 1} \frac{(2\lambda)^k}{2k}$ starts with the term λ, which the $-\lambda$ cancels. For $0 \leq \lambda < \frac{1}{2}$ each later term is at most $\frac{1}{4}(2\lambda)^k$, because $2k \geq 4$. Summing the geometric series gives

$$-\lambda - \tfrac{1}{2}\ln(1-2\lambda) = \sum_{k\geq 2} \frac{(2\lambda)^k}{2k} \leq \frac{\lambda^2}{1-2\lambda}. \tag{E.41}$$

3. **The Chernoff bound.** The d terms are independent, so the MGF of $S-d$ is the product of d factors. Markov's inequality applied to $e^{\lambda(S-d)}$ gives, for any level $\ell > 0$ and any $0 < \lambda < \frac{1}{2}$,

$$\Pr[S - d \geq \ell] \leq \exp\Big(-\lambda\ell + \frac{d\lambda^2}{1-2\lambda} \Big). \tag{E.42}$$

4. **Choose the level and λ.** The level is written through a free parameter $x > 0$ because that makes the final exponent simple. Take $\ell = 2\sqrt{dx} + 2x$. Write $r = \sqrt{x/d}$, so that $\sqrt{dx} = dr$ and $x = dr^2$. Choose $\lambda = r/(1+2r)$, which is below $\frac{1}{2}$, so that $1 - 2\lambda = 1/(1+2r)$. Then $\frac{d\lambda^2}{1-2\lambda} = \frac{dr^2}{1+2r}$ and $\lambda\ell = \frac{2dr^2(1+r)}{1+2r}$. The exponent of step 3 is

$$\frac{-2dr^2(1+r) + dr^2}{1+2r} = -dr^2\,\frac{1+2r}{1+2r} = -x. \tag{E.43}$$

   So $\Pr[S - d \geq 2\sqrt{dx} + 2x] \leq e^{-x}$.

5. **Back to the length.** Take $x = t^2/2$. If $\|Z\| \geq \sqrt{d} + t$, then squaring gives $S \geq d + 2\sqrt{d}\,t + t^2$. Since $2\sqrt{dx} = \sqrt{2}\sqrt{d}\,t$, which is at most $2\sqrt{d}\,t$, and $2x = t^2$, it follows that $S - d \geq 2\sqrt{dx} + 2x$. So the event $\|Z\| \geq \sqrt{d} + t$ lies inside the event of step 4, and its probability is at most $e^{-x} = e^{-t^2/2}$.

E.5.4. Checking it

The squared length $\|Z\|^2$ has the chi-squared distribution, whose tail the script computes exactly with the incomplete gamma function. Table E.4 compares it with the bound.

Table E.4.: Exact $\Pr[\|Z\| \geq \sqrt{d}+t]$ for $Z \sim \mathcal{N}(0, I_d)$, against the bound of Equation E.39.

t	$d = 1$	$d = 10$	$d = 100$	$d = 1000$	bound $e^{-t^2/2}$
1	4.55×10^{-2}	6.75×10^{-2}	7.51×10^{-2}	7.76×10^{-2}	6.07×10^{-1}
2	2.70×10^{-3}	2.96×10^{-3}	2.63×10^{-3}	2.44×10^{-3}	1.35×10^{-1}
3	6.33×10^{-5}	3.84×10^{-5}	1.98×10^{-5}	1.37×10^{-5}	1.11×10^{-2}

The bound holds in every dimension, and over d up to 10,000 and t up to 6 the exact tail is never more than 0.498 of it. It is loose. At $t = 3$ it is 175 times the exact value for $d = 1$ and about 810 times for $d = 1000$. Figure E.9 shows that for large d the exact tail falls like e^{-t^2}, twice as fast in the exponent. Figure E.10 explains this. As d grows, $\|Z\| - \sqrt{d}$ settles to a normal distribution with variance $\frac{1}{2}$, so the spread of the length stays of order 1 while $\sqrt{d}$ grows. What matters for Section 28.4 is that the bound does not depend on d at all.

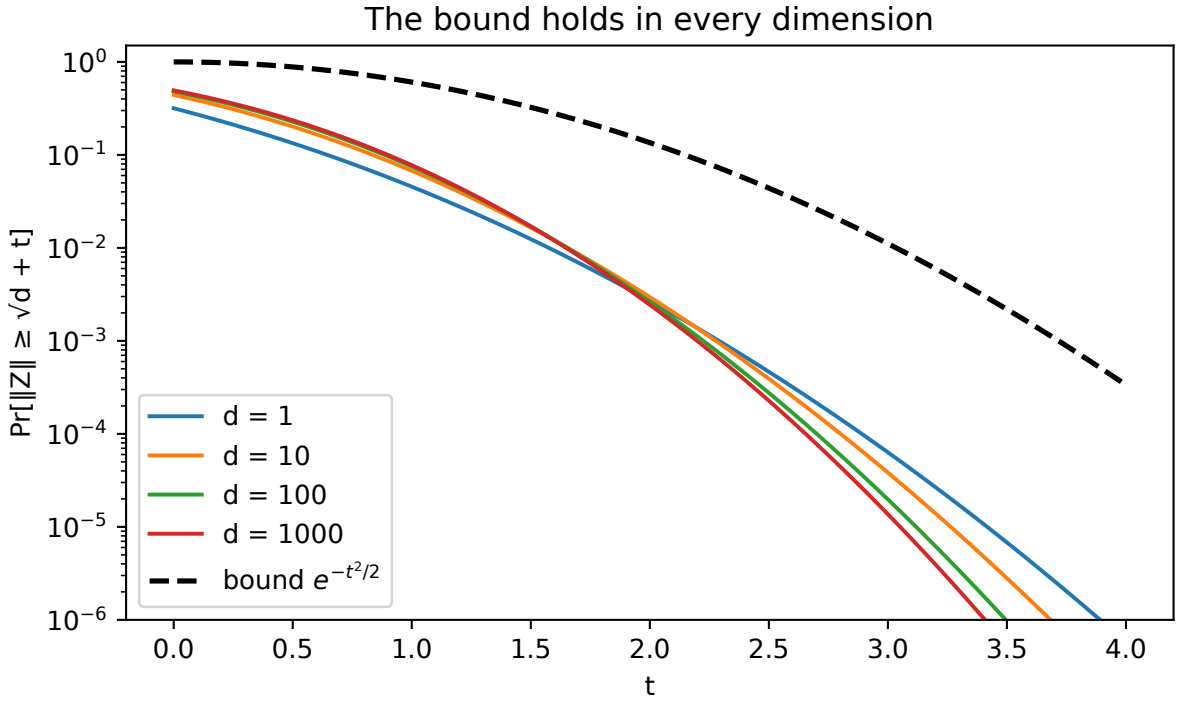


Figure E.9.: Exact $\Pr[\|Z\| \geq \sqrt{d}+t]$ for $d = 1$, 10, 100 and 1000 on a log scale, against the bound $e^{-t^2/2}$.

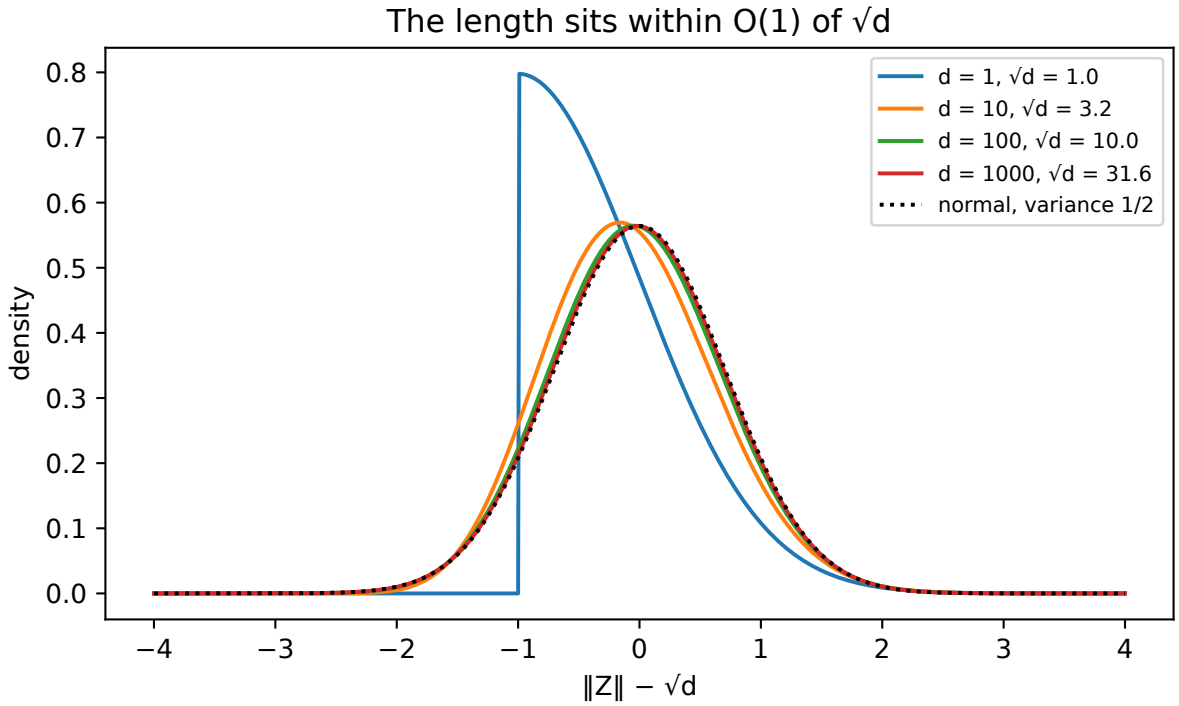


Figure E.10.: The density of $\|Z\| - \sqrt{d}$ for $d = 1$, 10, 100 and 1000, with a normal density of variance $\frac{1}{2}$ for comparison. The length sits within a distance of order 1 from $\sqrt{d}$ in every dimension.

E.6. The difference of two Laplace variables

E.6.1. What the chapter uses

Section 42.6 compares a Laplace noise $\nu \sim \mathrm{Lap}(b_1)$ with another, independent noise $\rho \sim \mathrm{Lap}(b_2)$, where $b_1 = 4/\varepsilon_0$ and $b_2 = 2/\varepsilon_0$ for a budget ε_0. It runs a test on each of P queries and wants the chance that any test wrongly passes to be at most a small number δ_{bad}. So it sets a threshold τ with $P \cdot \Pr[\nu - \rho > \tau] \le \delta_{\mathrm{bad}}$, using the tail of the difference given by Equation 42.13:

$$\Pr[\nu - \rho > \tau] = \frac{b_1^2 e^{-\tau/b_1} - b_2^2 e^{-\tau/b_2}}{2(b_1^2 - b_2^2)}, \qquad \tau \ge 0,\ b_1 \ne b_2. \tag{E.44}$$

The chapter says that the formula "comes from integrating the density of a sum of two Laplace variables, and the script checks it by simulation". Section 27.1 uses the case of equal scales, $b_1 = b_2 = 1/\varepsilon$, where the difference has density $\frac{\varepsilon}{4}(1 + \varepsilon|z|)e^{-\varepsilon|z|}$. This section does the integral.

E.6.2. The idea

A Laplace variable is symmetric, so $-\rho$ is again $\mathrm{Lap}(b_2)$, and the difference $\nu - \rho$ is a sum $\nu + \rho'$ of two independent Laplace variables, with $\rho' = -\rho \sim \mathrm{Lap}(b_2)$. Its density at s is the convolution Equation E.1. Each Laplace density has a corner at 0, so the integral splits into three ranges of the first variable y: below 0, between 0 and s, and above s. On each range both absolute values have a fixed sign, the integrand is a single exponential, and its integral is one line. Integrating the density from τ outwards then gives the tail.

E.6.3. Derivation

1. **Three pieces.** Fix $s \ge 0$, and write the density of $\nu + \rho'$ at s as $\frac{1}{4b_1b_2}\int e^{-|y|/b_1 - |s-y|/b_2}\,dy$.

   - For $y < 0$, the exponent is $y/b_1 - (s-y)/b_2$. Its integral over $y < 0$ is $e^{-s/b_2} \cdot \frac{b_1 b_2}{b_1 + b_2}$, because $\int_{-\infty}^{0} e^{y(1/b_1 + 1/b_2)}\,dy = \frac{1}{1/b_1 + 1/b_2}$.
   - For $y > s$, the exponent is $-y/b_1 - (y-s)/b_2$, and the same computation gives $e^{-s/b_1} \cdot \frac{b_1 b_2}{b_1 + b_2}$.
   - For $0 \le y \le s$, the exponent is $-y/b_1 - (s-y)/b_2 = -s/b_2 + y(1/b_2 - 1/b_1)$. With $b_1 \ne b_2$ its integral is $\frac{b_1 b_2}{b_1 - b_2}(e^{-s/b_1} - e^{-s/b_2})$.

2. **The density.** Dividing the three pieces by $4b_1b_2$ and collecting the two exponentials, the coefficient of e^{-s/b_1} is $\frac{1}{4}(\frac{1}{b_1+b_2} + \frac{1}{b_1-b_2}) = \frac{b_1}{2(b_1^2-b_2^2)}$, and that of e^{-s/b_2} is $-\frac{b_2}{2(b_1^2-b_2^2)}$. By symmetry, for every s,

   $$f(s) = \frac{b_1 e^{-|s|/b_1} - b_2 e^{-|s|/b_2}}{2(b_1^2 - b_2^2)}. \tag{E.45}$$

   As a check, at $s = 0$ this is $\frac{1}{2(b_1+b_2)}$, which is positive.

3. **The tail.** For $\tau \ge 0$, integrate f from τ to infinity, using $\int_\tau^\infty b\,e^{-s/b}\,ds = b^2 e^{-\tau/b}$. This gives Equation E.44. At $\tau = 0$ it gives $\frac{1}{2}$, as symmetry requires.

4. **Far out, the wider noise wins.** For large τ the term with the larger scale b_1 dominates, and the tail is about $\frac{b_1^2}{2(b_1^2-b_2^2)}e^{-\tau/b_1}$. With $b_1 = 2b_2$ that is $\frac{2}{3}e^{-\tau/b_1}$, which is $\frac{4}{3}$ times the tail $\frac{1}{2}e^{-\tau/b_1}$ of ν alone. So the narrower noise ρ adds only a constant factor. To make $P \cdot \Pr[\nu - \rho > \tau] \le \delta_{\text{bad}}$ for P queries, the threshold must be about $b_1 \ln(2P/(3\delta_{\text{bad}}))$, which is the $4\ln(P/\delta_{\text{bad}})/\varepsilon_0$ growth the chapter describes.

5. **Equal scales.** When $b_1 = b_2 = b$, the middle piece of step 1 has the constant integrand $e^{-s/b}$ over an interval of length s, so it is $s\,e^{-s/b}$. The other two pieces are $\frac{b}{2}e^{-s/b}$ each. Dividing by $4b^2$,

$$f(s) = \frac{1}{4b}\Big(1 + \frac{|s|}{b}\Big)e^{-|s|/b}. \tag{E.46}$$

   With $b = 1/\varepsilon$ this is the density $\frac{\varepsilon}{4}(1 + \varepsilon|z|)e^{-\varepsilon|z|}$ that Section 27.1 uses. Integrating from τ gives the tail $\frac{1}{4}(2 + \tau/b)e^{-\tau/b}$. It is also the limit of Equation E.44 as $b_1 \to b_2$. At $\tau = 3$ and $b = 1$, for example, it is 0.0622, and Equation E.44 with $b_1 = 1.001$ gives 0.0623. The factor $1 + |s|/b$ is why this density is not a Laplace density, as Section E.1 noted.

E.6.4. Checking it

At the chapter's budget $\varepsilon_0 = 1$, so $b_1 = 4$ and $b_2 = 2$, and at $\tau = 8$, Equation E.44 gives 0.0872. Numerical integration of the density of step 2 also gives 0.0872, and a million simulated pairs give 0.0867. Solving $P \cdot \Pr[\nu - \rho > \tau] = 5 \times 10^{-6}$ by bisection gives the chapter's thresholds, $\tau = 74.8$ for a pool of $P = 1{,}000$ queries and $\tau = 84.0$ for $P = 10{,}000$. The equal-scale density of step 5 matches a numerical convolution to 7.5×10^{-9}.

Figure E.11 shows the tail on a log scale. It runs parallel to the tail of ν alone and slightly above it, as step 4 says, while the tail of ρ alone falls away. The threshold sits where the tail reaches $\delta_{\text{bad}}/P = 5 \times 10^{-10}$, far beyond the reach of a simulation. That is why the closed form matters: it is the only way to set the threshold accurately.

E.7. Code

The script `code/appendix-derivations/apx_probability.py` checks every result of this appendix against an exact value, a numerical integral or a simulation, and draws the figures. Run it with

```
uv run python code/appendix-derivations/apx_probability.py
```

Table E.5 lists the main numbers it prints, with the default seed 0.

Table E.5.: The numbers printed by the appendix's script.

Check	Result
Poisson, exponential, Gamma and normal sums against their closed forms	largest gaps 3.3×10^{-16}, 7.2×10^{-5}, 1.1×10^{-3}, 1.2×10^{-12}

Check	Result
Projections of $\mathcal{N}(0, I_2)$ at $\phi = \pi/6$	variances 0.9983 and 1.0010, correlation -0.0005
Mode at $\lfloor (n+1)p \rfloor$	480 of 480 cases
Stirling error at $n = 10$ against $1/(12n)$	8.331×10^{-3} against 8.333×10^{-3}
Fraction recovered, $m = 10$, 100, 1000	0.62305, 0.53979, 0.51261
Shift TV of Bin(100, $\frac{1}{2}$) and its mode height	both 0.0796
Largest $np\bar{p}\,\lvert\ln(\text{ratio})\rvert$ for $np\bar{p} \geq 4$	0.187 (proved: at most 3.5)
Poisson(4) at $k = 20$ and 40: $\Pr[N = k]$ and its bound	8.28×10^{-9} and 5.09×10^{-6}; 2.71×10^{-26} and 2.35×10^{-23}
Poisson(4) at $k = 20$ and 40: $\Pr[N \geq k]$ and its bound	1.02×10^{-8} and 1.11×10^{-5}; 3.01×10^{-26} and 3.23×10^{-23}
Ratio $e^{\mu}\sqrt{2\pi k}\,e^{r_k}$ at $k = 20$ and 40	614.6 and 867.4 ($e^{\mu}\sqrt{2\pi k}$ alone: 612.0 and 865.6)
Normal tail with mean 4 and variance 4, above 20 and 40	6.22×10^{-16} and 9.74×10^{-73}
Chernoff bound on $\Pr[N \geq 20]$	9.32×10^{-8}, 9.1 times the exact tail
Swapping 30 Exp(1) terms, $h = \sin$	error 0.0365, bound 0.1220
CDF gap for 30 Exp(1) terms	exact 0.0243; bound 1.6201; Berry–Esseen 0.2116
Normal length tail, largest ratio to $e^{-t^2/2}$	0.498
Laplace difference tail at $\tau = 8$	formula 0.0872, integral 0.0872, simulation 0.0867
Thresholds for $P = 1{,}000$ and 10,000	74.8 and 84.0

Two lessons stand out. The closed forms match their numerical checks to the accuracy of the grid or the simulation, so the derivations prove the formulas the chapters quote. And the bounds differ a lot in how tight they are. Stirling's $1/(12n)$ is nearly exact, and the peak-height bound is about 19 times too large in its constant. The norm bound loses a factor of 2 in the exponent for large d, and the CLT bound is far too weak to use for numbers. In the chapters, the CLT is used for its limit, while concrete error sizes come from the exact computations or from Berry–Esseen.

E.8. References

- Blitzstein, J. K. & Hwang, J. (2019). *Introduction to Probability*, 2nd ed. Convolutions, the MGF and the uniqueness theorem at the level of this book.
- Billingsley, P. (1995). *Probability and Measure*, 3rd ed. Proofs of uniqueness for MGFs and characteristic functions, and the Cramér–Wold device.
- Feller, W. (1968). *An Introduction to Probability Theory and Its Applications*, Vol. 1, 3rd ed. The binomial ratio argument, Stirling's formula and Wallis's product.

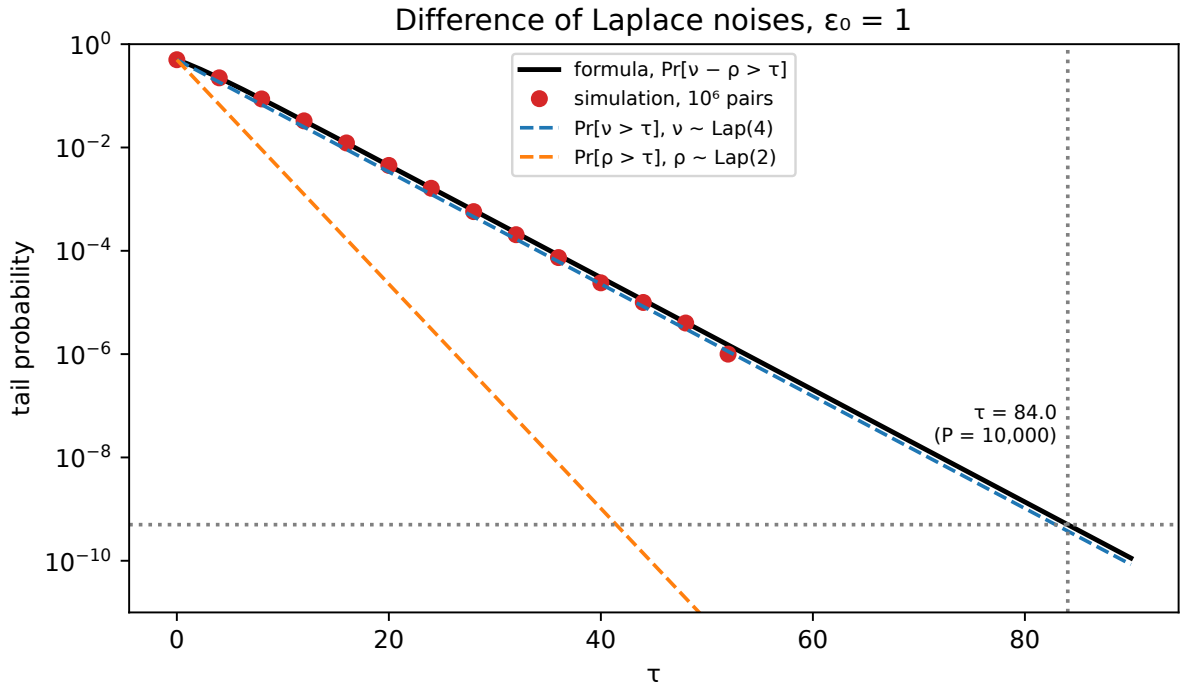


Figure E.11.: The tail $\Pr[\nu - \rho > \tau]$ for $\nu \sim \mathrm{Lap}(4)$ and $\rho \sim \mathrm{Lap}(2)$ on a log scale: the formula (line), one million simulated pairs (dots), and the tails of ν and ρ alone (dashed). The dotted lines mark the threshold for a pool of 10,000 queries and the level 5×10^{-10}.

- Robbins, H. (1955). "A remark on Stirling's formula". *American Mathematical Monthly* 62(1). The bounds $\frac{1}{12n+1} < r_n < \frac{1}{12n}$ on the error of Stirling's formula.
- Lindeberg, J. W. (1922). "Eine neue Herleitung des Exponentialgesetzes in der Wahrscheinlichkeitsrechnung". *Mathematische Zeitschrift* 15. The replacement argument, under the Lindeberg condition.
- Durrett, R. (2019). *Probability: Theory and Examples*, 5th ed. The CLT under the Lindeberg condition and its standard proof.
- Shevtsova, I. (2011). "On the absolute constants in the Berry–Esseen type inequalities for identically distributed summands". arXiv:1111.6554. The constant 0.48 quoted above.
- Laurent, B. & Massart, P. (2000). "Adaptive estimation of a quadratic functional by model selection". *Annals of Statistics* 28(5). The chi-squared tail bound of Section E.5, in Lemma 1.
- Boucheron, S., Lugosi, G. & Massart, P. (2013). *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Chapter 5 proves the same normal-length bound through Gaussian concentration of functions that change slowly.

F. Tail bounds, tests and divergences

Several chapters state a fact about tails, tests or divergences and then use it without a full proof. Some cite a book for the proof, some give a sketch that needs an extra condition, and one gives a reason that does not hold in general. This appendix derives six of these facts step by step. They belong together because they are built from the same three tools. The first is the moment generating function (MGF) of a random variable and its logarithm. The second is a change of measure, which rewrites a probability under one distribution as an average under another. The third is Jensen's inequality for convex functions.

Table F.1 lists the six results, where a chapter states each one, and where the book relies on it.

Table F.1.: The derivations of this appendix, the chapter section that states each result, and the sections that rely on it.

Result	Stated in	Relied on in
Bernstein's inequality	Concentration inequalities, Hoeffding section	The same chapter's example of a small error rate
Cramér's lower bound and the Chernoff–Stein lemma	Concentration inequalities, Chernoff section; Section 4.2	The claim that the Chernoff exponent cannot be improved
The Clopper–Pearson limits exist and are unique	Section 4.9	Section 47.1
Rényi divergence tends to the Kullback–Leibler divergence	Section 3.2	Section 5.4
Perspective functions are jointly convex	Section 30.3	Section 32.7, Section 31.6, Section 33.3
A noisy count has an increasing likelihood ratio	Section 9.5	Table 9.5

F.1. Bernstein's inequality: the variance sets the width

F.1.1. What the chapter uses

Concentration inequalities states Bernstein's inequality as Theorem 2.6, in its section on sub-Gaussian variables and Hoeffding's inequality. It says that "the proof is the same Chernoff recipe" and cites Boucheron, Lugosi & Massart (chapter 2) for the derivation.

The setting is an average of independent measurements. Take a number n of independent variables $X_1, \dots, X_n$, all with the same mean μ and the same variance σ^2. Assume that each one stays within a distance R of its mean, so that $|X_i - \mu| \le R$, and write $\bar{X}_n$ for their average. Then for every deviation $t > 0$,

$$\Pr\left[\bar{X}_n - \mu \ge t\right] \le \exp\left(-\frac{nt^2}{2\sigma^2 + \frac{2}{3}Rt}\right). \tag{F.1}$$

The chapter also turns this into a **width**. For a failure probability β, the width is a deviation that the average exceeds with probability at most β. The chapter says that the width is about

$$\sqrt{\frac{2\sigma^2 \ln(1/\beta)}{n}} + \frac{2R\ln(1/\beta)}{3n}. \tag{F.2}$$

The derivation below shows that this expression is more than an approximation: it is a valid width.

F.1.2. The idea

Hoeffding's lemma bounds the MGF of a bounded variable as if it were the worst variable the range allows, a fair coin on the two ends, which has the largest variance of any variable in the interval. Bernstein's inequality keeps the true variance instead. To see how, expand the MGF of a centred variable as a series. After the constant 1 and the mean, which is 0, its terms are the moments times powers of λ: first the variance, then the third moment, the fourth, and so on. A variable that never strays more than R from its mean has small higher moments, each at most the variance times a power of R. So the variance and the range together control the whole MGF. After that, the Chernoff recipe runs as usual: bound the MGF of each term of the sum, multiply these MGFs, choose the free parameter λ and simplify.

A rare event shows why this matters. Let X be a Bernoulli(0.01) variable, which is 1 with probability 0.01 and 0 otherwise. Its centred version $Y = X - 0.01$ is 0.99 with probability 0.01 and -0.01 otherwise. Its variance is $0.01 \times 0.99 = 0.0099$, while Hoeffding's lemma, which knows only that X lies in an interval of width 1, treats it as if its variance were $\frac{1}{4}$, about 25 times more. Its higher moments are just as small, as Table F.2 shows, and each stays below the bound $\sigma^2 R^{k-2}$ used in step 2 below.

Table F.2.: Roments of the centred Bernoulli(0.01) variable Y and the bound of step 2.

Roment k	2	3	4
$\mathbb{E}[Y^k]$	0.0099000	0.0097020	0.0096060
Bound $\sigma^2 R^{k-2}$, with $R = 0.99$	0.0099000	0.0098010	0.0097030

F.1.3. Derivation

1. **Centre the variables.** Subtract the mean from each variable, $Y_i = X_i - \mu$, so that each Y_i has mean 0, variance σ^2 and $|Y_i| \le R$. Then the event $\bar{X}_n - \mu \ge t$ is the event that the sum $S = Y_1 + \cdots + Y_n$ is at least nt.

2. **Bound each moment.** For $k \geq 2$, split $|Y|^k$ as $Y^2|Y|^{k-2}$. The second factor is at most R^{k-2}, because $|Y| \leq R$. Averaging gives

$$\mathbb{E}[Y^k] \leq \mathbb{E}\big[|Y|^k\big] \leq \mathbb{E}[Y^2]\, R^{k-2} = \sigma^2 R^{k-2}. \tag{F.3}$$

3. **Bound the MGF of one term.** Take $\lambda \geq 0$ and expand $e^{\lambda Y} = 1 + \lambda Y + \sum_{k\geq 2} \lambda^k Y^k/k!$. Because the variable is bounded, you may average the series term by term. The linear term then averages to 0, because Y has mean 0. Each later term is at most $\lambda^k \sigma^2 R^{k-2}/k!$ by step 2, and the direction of the inequality is kept because $\lambda^k \geq 0$. Writing $\lambda^k \sigma^2 R^{k-2}$ as $(\sigma^2/R^2)(\lambda R)^k$, the terms from $k = 2$ on add up to at most σ^2/R^2 times the sum $\sum_{k\geq 2} (\lambda R)^k/k!$, which is the exponential series without its first two terms, $e^{\lambda R} - 1 - \lambda R$. So

$$\mathbb{E}\big[e^{\lambda Y}\big] \leq 1 + \frac{\sigma^2}{R^2}(e^{\lambda R} - 1 - \lambda R) \leq \exp\left(\frac{\sigma^2}{R^2}(e^{\lambda R} - 1 - \lambda R)\right). \tag{F.4}$$

   The second inequality is $1 + x \leq e^x$. It holds because the line $1 + x$ is the tangent of the convex curve e^x at 0, and a convex curve lies above its tangents.

4. **Simplify the series.** Write $u = \lambda R$ and take $0 \leq u < 3$. Every factor of $k!$ from 3 up to k is at least 3, so $k! \geq 2 \cdot 3^{k-2}$. Each term of the series is therefore at most $\frac{u^2}{2}(u/3)^{k-2}$, and these terms form a geometric series with ratio $u/3 < 1$. Summing it gives

$$e^u - 1 - u \leq \frac{u^2}{2(1 - u/3)}. \tag{F.5}$$

   Put back $u = \lambda R$, multiply by σ^2/R^2 and take the logarithm of step 3's bound. The log-MGF of one term then satisfies

$$\ln \mathbb{E}\big[e^{\lambda Y}\big] \leq \frac{\sigma^2 \lambda^2}{2(1 - \lambda R/3)}, \qquad 0 \leq \lambda < 3/R. \tag{F.6}$$

   This is the Gaussian log-MGF $\sigma^2\lambda^2/2$ with the variance inflated by the factor $1/(1 - \lambda R/3)$, which is close to 1 for small λ.

5. **Chernoff for the sum.** The MGF of a sum of independent terms is the product of their MGFs, so the log-MGFs add and S has log-MGF at most n times the bound of step 4. Rarkov's inequality applied to $e^{\lambda S}$ (the master bound of the Chernoff method) then gives

$$\Pr[S \geq nt] \leq \exp\left(-\lambda n t + \frac{n\sigma^2\lambda^2}{2(1 - \lambda R/3)}\right). \tag{F.7}$$

6. **Choose λ.** Take $\lambda = t/(\sigma^2 + Rt/3)$. It is positive, and $\lambda R/3 = (Rt/3)/(\sigma^2 + Rt/3)$ is below 1, so step 4 applies. With this choice $1 - \lambda R/3 = \sigma^2/(\sigma^2 + Rt/3)$. The second term of the exponent becomes $nt^2/(2(\sigma^2 + Rt/3))$, and the first is $-nt^2/(\sigma^2 + Rt/3)$. Their sum is

$$-\frac{nt^2}{2(\sigma^2 + Rt/3)} = -\frac{nt^2}{2\sigma^2 + \frac{2}{3}Rt}, \tag{F.8}$$

   which is Equation F.1. This λ is not the exact minimizer of step 5, but it is the one that gives a clean formula.

7. **The width.** Write ℓ for $\ln(1/\beta)$. The bound equals β when $nt^2 = \ell(2\sigma^2 + \frac{2}{3}Rt)$, which is a quadratic equation in t with the positive root

$$t^* = \frac{R\ell}{3n} + \sqrt{\Big(\frac{R\ell}{3n}\Big)^2 + \frac{2\sigma^2\ell}{n}}. \tag{F.9}$$

The exponent $nt^2/(2\sigma^2 + \frac{2}{3}Rt) = nt/(2\sigma^2/t + \frac{2}{3}R)$ grows with t, so every $t \geq t^*$ has a bound of at most β. To simplify the root, use the fact that the square root of a sum of two non-negative numbers is at most the sum of their square roots, which you can see by squaring both sides. Applied to the square root in t^*, it gives

$$t^* \leq \sqrt{\frac{2\sigma^2\ell}{n}} + \frac{2R\ell}{3n}, \tag{F.10}$$

the chapter's width. So with probability at least $1 - \beta$, the average exceeds μ by less than this amount.

The proof also works when the variables have different variances σ_i^2. Step 5 then adds the individual bounds, and σ^2 becomes their average.

F.1.4. Checking it

The appendix's script, `apx_tails.py` (see the Code section), checks steps 3 and 4 for the centred Bernoulli(0.01) on a grid of λ from 0.01 to 2.99, which is just below $3/R = 3.03$. The exact log-MGF never exceeds the series bound $\frac{\sigma^2}{R^2}(e^{\lambda R} - 1 - \lambda R)$ of step 3, and the series bound never exceeds the simpler bound of step 4. Figure F.1 shows the three curves together with Hoeffding's $\lambda^2/8$. For small λ both Bernstein curves lie on the exact one, far below Hoeffding's, because they use the variance 0.0099 rather than $\frac{1}{4}$. Only near $\lambda = 3/R$ does the simplification of step 4 blow up.

Figure F.2 shows what this buys for the mean of $n = 1{,}000$ such variables, whose expected count is 10. Take $t = 0.01$, which asks for a count of 20 or more, twice the expected count. The exact binomial tail is 3.29×10^{-3}. Bernstein's bound is 2.26×10^{-2}, within a factor of 7 of it and close to the optimized Chernoff bound, 2.00×10^{-2}. (That bound uses the exact binomial MGF. Section F.2 shows that its exponent is the binary Kullback–Leibler divergence.) Hoeffding's bound is 0.819, which says almost nothing. The same holds for widths. At $\beta = 0.05$ the exact root t^* of step 7 is 0.00875, the chapter's simpler formula gives 0.00968, and Hoeffding's one-sided width $\sqrt{\ln(1/\beta)/(2n)} = \sqrt{\ln(20)/2000}$ is 0.0387, four times larger.

F.2. Chernoff's exponent is exact, for tails and for tests

F.2.1. What the chapter uses

Let $X_1, \ldots, X_n$ be independent copies of a variable X with mean μ, and let $S_n = X_1 + \cdots + X_n$. Write $K_X(\lambda) = \ln \mathbb{E}[e^{\lambda X}]$ for the log-MGF of one copy. The Chernoff method of Concentration inequalities bounds the chance that the sum reaches nt, for a level t above the mean, by

$$\Pr[S_n \geq nt] \leq e^{-nK_X^*(t)}, \qquad K_X^*(t) = \sup_{\lambda > 0} (\lambda t - K_X(\lambda)). \tag{F.11}$$

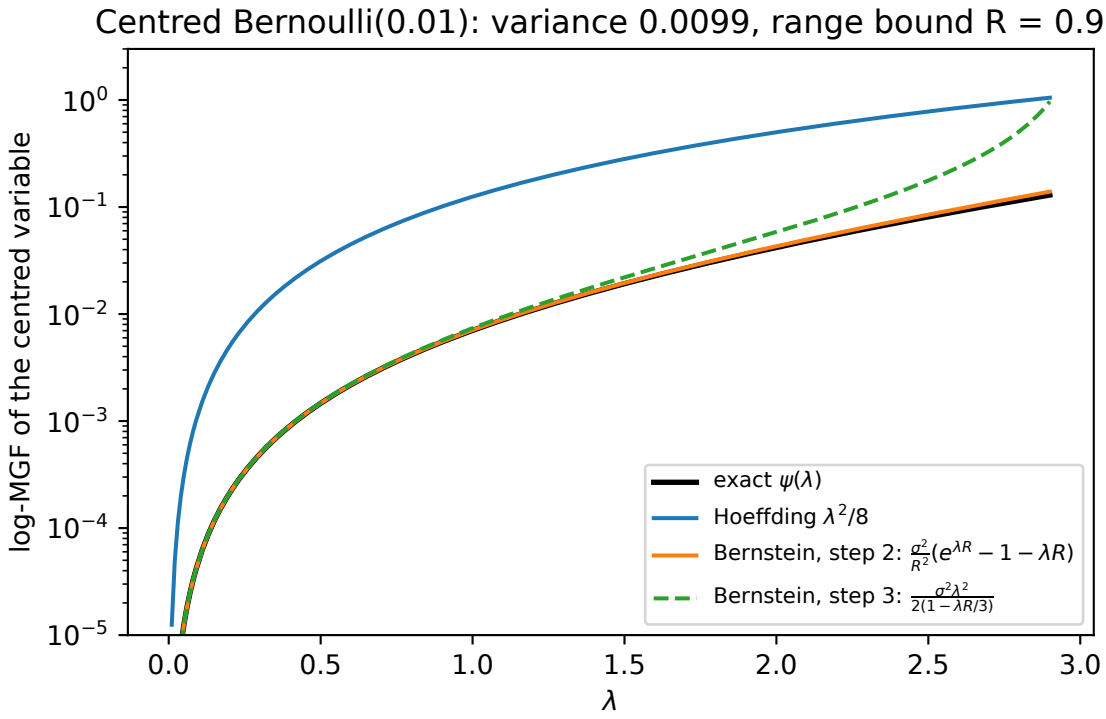


Figure F.1.: The log-MGF of the centred Bernoulli(0.01) variable as a function of λ (black), with Hoeffding's bound $\lambda^2/8$ and the two Bernstein bounds of steps 3 and 4 of the derivation, on a log scale.

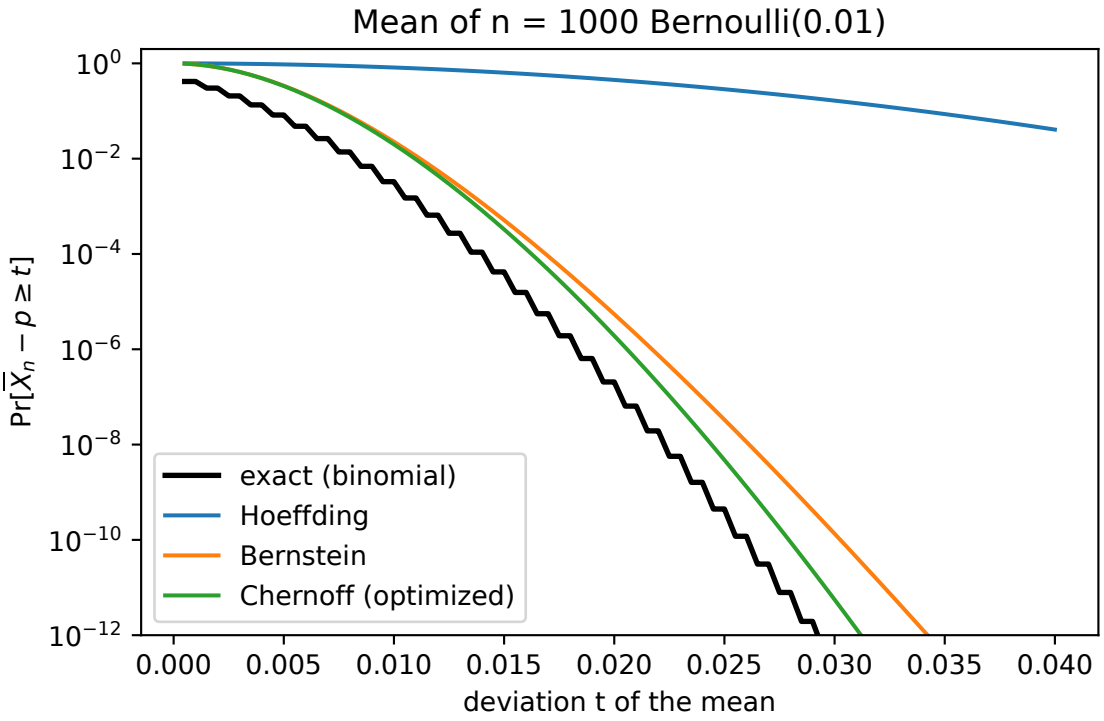


Figure F.2.: Tail of the mean of 1,000 Bernoulli(0.01) variables on a log scale: exact (black), and the Hoeffding, Bernstein and optimized Chernoff bounds.

The chapter then says that "Cramér's theorem says that … the probability of a large deviation decays exponentially in n with exactly the rate the optimized Chernoff bound gives, so no method can improve its exponent for large sums". It does not prove this. In symbols, for every t between the mean μ and the largest value $x_{\max}$ that X can take,

$$\lim_{n\to\infty} -\frac{1}{n}\ln \Pr[S_n \geq nt] = K_X^*(t). \tag{F.12}$$

Section 4.2 makes the same kind of claim about tests, and cites Cover & Thomas (chapter 11) for it. You observe n independent draws and test H_0, that they come from P, against H_1, that they come from Q. Hold the type I error at a level FPR in $(0, 1)$, and write $\mathrm{FNR}_n^*(\mathrm{FPR})$ for the smallest type II error of any test, randomized or not, whose type I error is at most FPR. The **Chernoff–Stein lemma** says that

$$\lim_{n\to\infty} -\frac{1}{n}\ln \mathrm{FNR}_n^*(\mathrm{FPR}) = D_{\mathrm{KL}}(P\|Q), \qquad D_{\mathrm{KL}}(P\|Q) = \mathbb{E}_P\Big[\ln\frac{p(X)}{q(X)}\Big], \tag{F.13}$$

where D_{KL} is the Kullback–Leibler (KL) divergence. The errors FPR and FNR are the type I and type II errors of Section 4.2, renamed because this page needs α for a Rényi order and β for a failure probability.

F.2.2. The idea

Chernoff's bound shows that the probability is *at most* about $e^{-nK_X^*(t)}$. To show that it is also *at least* about that, change the distribution so that the rare event becomes typical, and then pay for the change. The change is the **tilted** distribution of Divergences, which multiplies the probability of each outcome x by $e^{\lambda x}$ and renormalizes. With $\lambda > 0$ it moves mass toward large values. Pick λ so that the tilted mean sits just above t, so that most of the tilted sums land in a narrow window just above nt. Under the tilt, S_n then lands near nt most of the time, by Chebyshev's inequality. The price of switching back is the ratio of the two distributions on those samples, which is about $e^{-n(\lambda t - K_X(\lambda))}$, and that is the Chernoff exponent.

Here is a concrete case. A coin lands heads with probability 0.1, and you ask how likely 100 tosses are to give at least 20 heads, twice the expected number. Tilting by $e^{\lambda x}$, with $x = 1$ for heads and 0 for tails, turns it into a coin with heads probability $0.1e^{\lambda}/(0.9 + 0.1e^{\lambda})$. With $\lambda = \ln 2.25 = 0.8109$ that probability is exactly 0.2, so for the tilted coin 20 heads in 100 is the typical outcome, not a rare one.

The Chernoff–Stein lemma uses the same trick, with the two distributions already given. Under P, the evidence $\sum_i \ln(p(x_i)/q(x_i))$ of a sample is close to $n\,D_{\mathrm{KL}}(P\|Q)$, by the law of large numbers. So on most samples that P produces, Q gives a probability about $e^{-nD_{\mathrm{KL}}(P\|Q)}$ times smaller. A test with a small type I error must accept H_0 on most of P's typical samples. So under Q it accepts with probability about $e^{-nD_{\mathrm{KL}}(P\|Q)}$. No test can do better, because every test of small type I error must accept those samples. And the test that accepts exactly the samples with large evidence does that well.

F.2.3. Derivation

Two facts from the chapters do the work. The first is the **change of measure** (Lemma 3.2, Lemma 4.2). If P and Q' give positive probability to the same outcomes, then for any event A,

$$\Pr_P[X \in A] = \sum_{x \in A} p(x) = \sum_{x \in A} q'(x) \frac{p(x)}{q'(x)} = \mathbb{E}_{Q'}\Big[\frac{p(X)}{q'(X)} \mathbf{1}[X \in A]\Big], \tag{F.14}$$

because multiplying and dividing by $q'(x)$ changes nothing. The second is the **tilted distribution** with parameter λ, whose probabilities are

$$q_\lambda(x) = p(x)\, e^{\lambda x - K_X(\lambda)}. \tag{F.15}$$

They add up to 1 because $e^{K_X(\lambda)} = \sum_x p(x) e^{\lambda x}$. Its mean is $K'_X(\lambda)$ and its variance is $K''_X(\lambda)$, as the Rényi section of Divergences shows for the log-likelihood ratio. Here you only need these facts for a bounded X, where the sums are finite and the derivatives can be taken term by term.

Cramér's lower bound. Assume that X is bounded and not constant, and that $\mu < t < x_{\max}$.

1. **Tilt to just above t.** Fix a small $\kappa > 0$ with $t + \kappa < x_{\max}$. The tilted mean $K'_X(\lambda)$ equals μ at $\lambda = 0$. It increases with λ, because its derivative is a variance, which is positive for a variable that is not constant. It tends to $x_{\max}$ as $\lambda \to \infty$, because the weight $e^{\lambda x}$ favours the largest values more and more. So there is one $\lambda > 0$ with $K'_X(\lambda) = t + \kappa/2$.
2. **Change the measure of the whole sample.** Tilt each of the n copies. The ratio of the two probabilities of a sample is the product of the ratios of its draws, $\prod_i p(x_i)/q_\lambda(x_i) = e^{-\lambda S_n + n K_X(\lambda)}$. The change of measure therefore gives, for any event A about the sample,

$$\Pr[A] = \mathbb{E}_{Q_\lambda}\big[e^{-\lambda S_n + n K_X(\lambda)}\, \mathbf{1}[A]\big]. \tag{F.16}$$

3. **Restrict to a window.** Let A be the event $nt \le S_n \le n(t+\kappa)$. It is part of the event $S_n \ge nt$. On A, the sum is at most $n(t+\kappa)$, and $\lambda > 0$, so $e^{-\lambda S_n} \ge e^{-\lambda n(t+\kappa)}$. Hence

$$\Pr[S_n \ge nt] \ge \Pr[A] \ge e^{-n(\lambda(t+\kappa) - K_X(\lambda))} \Pr_{Q_\lambda}[A]. \tag{F.17}$$

4. **The window is likely under the tilt.** Under Q_λ, the average S_n/n has mean $t + \kappa/2$ and variance $K''_X(\lambda)/n$. The window contains every average within $\kappa/2$ of that mean. Chebyshev's inequality bounds the chance of falling outside, so

$$\Pr_{Q_\lambda}[A] \ge 1 - \frac{4K''_X(\lambda)}{n\kappa^2}, \tag{F.18}$$

   which tends to 1 as n grows.
5. **Let n grow.** Take logarithms in step 3 and divide by $-n$. The term $-\frac{1}{n}\ln \Pr_{Q_\lambda}[A]$ tends to 0, so

$$\limsup_{n\to\infty} -\frac{1}{n}\ln \Pr[S_n \ge nt] \le \lambda(t+\kappa) - K_X(\lambda). \tag{F.19}$$

6. **Let κ shrink.** As $\kappa \to 0$, the chosen λ tends to the λ_0 with $K'_X(\lambda_0) = t$, because K'_X is continuous and increasing. The right side of step 5 then tends to $\lambda_0 t - K_X(\lambda_0)$. The function $\lambda t - K_X(\lambda)$ is concave, since K_X is convex, and its slope $t - K'_X(\lambda)$ is zero at λ_0. So λ_0 is where it is largest, and $\lambda_0 t - K_X(\lambda_0) = K^*_X(t)$.

7. **Combine with Chernoff.** Chernoff's bound gives $-\frac{1}{n}\ln \Pr[S_n \geq nt] \geq K_X^*(t)$ for every n. Step 6 gives the matching upper limit, which proves Equation F.12.

For the coin, $K_X(\lambda) = \ln(1 - p + pe^{\lambda})$ with $p = 0.1$, and solving $K_X'(\lambda_0) = t$ gives $e^{\lambda_0} = t(1-p)/(p(1-t))$. Substituting, the exponent is the **binary KL divergence**

$$K_X^*(t) = \mathrm{kl}(t\|p) = t\ln\frac{t}{p} + (1-t)\ln\frac{1-t}{1-p}. \tag{F.20}$$

With $t = 0.2$ this is $\lambda_0 = \ln 2.25 = 0.8109$, $K_X(\lambda_0) = \ln 1.125 = 0.11778$, and $K_X^*(0.2) = 0.2 \times 0.8109 - 0.11778 = 0.04440$.

Chernoff–Stein, achievability. Assume that P and Q give positive probability to the same outcomes. Write $L(x)$ for the evidence $\ln(p(x)/q(x))$ of one draw x. Under P its mean is the divergence $D_{\mathrm{KL}}(P\|Q)$, written D_{KL} below, and assume that its variance V is finite. For a whole sample $x = (x_1, \ldots, x_n)$, write $p^n(x)$ and $q^n(x)$ for its probabilities under the two hypotheses. Each is a product over the draws, so the change of measure becomes

$$q^n(x) = p^n(x)\, e^{-\sum_i L(x_i)}. \tag{F.21}$$

1. **The test.** Fix $\kappa > 0$, and accept H_0 exactly when the average evidence is at least $D_{\mathrm{KL}} - \kappa$. Call that set of samples A_n.
2. **Its type I error.** It rejects when the average evidence falls more than κ below its mean D_{KL}. The average has variance V/n, so Chebyshev gives a type I error of at most $V/(n\kappa^2)$, which is below FPR once n is large.
3. **Its type II error.** On A_n the total evidence is at least $n(D_{\mathrm{KL}} - \kappa)$, so $q^n(x) \leq p^n(x)\, e^{-n(D_{\mathrm{KL}}-\kappa)}$. Summing over A_n gives

$$\mathrm{FNR}_n = \Pr_{Q^n}[A_n] \leq e^{-n(D_{\mathrm{KL}}-\kappa)} \Pr_{P^n}[A_n] \leq e^{-n(D_{\mathrm{KL}}-\kappa)}. \tag{F.22}$$

4. **The best test does at least as well.** So $\mathrm{FNR}_n^*(\mathrm{FPR}) \leq e^{-n(D_{\mathrm{KL}}-\kappa)}$ for large n, and $\liminf_n -\frac{1}{n}\ln \mathrm{FNR}_n^*(\mathrm{FPR}) \geq D_{\mathrm{KL}} - \kappa$ for every $\kappa > 0$.

Chernoff–Stein, converse. Now take any test, described by its probability $\phi(x)$ of rejecting H_0, with type I error $\mathbb{E}_{P^n}[\phi] \leq \mathrm{FPR}$.

5. **Restrict to typical samples.** Let B_n be the set of samples whose average evidence is at most $D_{\mathrm{KL}} + \kappa$. On B_n, $q^n(x) \geq p^n(x)\, e^{-n(D_{\mathrm{KL}}+\kappa)}$. The type II error is the probability of accepting under Q, and keeping only the samples in B_n can only lower it, so

$$\mathrm{FNR}_n = \sum_x (1-\phi(x))q^n(x) \geq e^{-n(D_{\mathrm{KL}}+\kappa)} \sum_{x\in B_n} (1-\phi(x))p^n(x). \tag{F.23}$$

6. **The test must accept most typical samples.** The last sum is the acceptance probability under P, minus the part outside B_n. That part is at most $\Pr_{P^n}[B_n^c]$, which Chebyshev bounds by $V/(n\kappa^2)$. So the sum is at least $1 - \mathrm{FPR} - V/(n\kappa^2)$.
7. **Let n grow.** Taking logarithms, $-\frac{1}{n}\ln \mathrm{FNR}_n \leq D_{\mathrm{KL}} + \kappa - \frac{1}{n}\ln\left(1 - \mathrm{FPR} - V/(n\kappa^2)\right)$, and the last term tends to 0. (For large n the factor $1 - \mathrm{FPR} - V/(n\kappa^2)$ is positive, so its logarithm is defined.) This holds for every test of level FPR, so $\limsup_n -\frac{1}{n}\ln \mathrm{FNR}_n^*(\mathrm{FPR}) \leq D_{\mathrm{KL}} + \kappa$. Since κ is arbitrary, steps 4 and 7 together prove Equation F.13.

F.2.4. Checking it

For the coin with $p = 0.1$ and $t = 0.2$, the script computes the exact exponent $-\frac{1}{n}\ln \Pr[S_n \geq 0.2n]$ from the binomial distribution. It is 0.06225 at $n = 100$, 0.04728 at $n = 1{,}000$ and 0.04481 at $n = 10{,}000$, and it approaches $\mathrm{kl}(0.2\|0.1) = 0.04440$ from above, as Chernoff's bound requires. The right panel of Figure F.3 also draws the bound of steps 3 and 4 for each n, with the window width $\kappa = 4\sqrt{t(1-t)/n}$. The factor 4 makes Chebyshev's bound in step 4 leave about three quarters of the tilted mass in the window. It closes in from the other side, but slowly: 0.23163, 0.09152 and 0.05800 at the same three sizes. It is slow because the window must shrink like $1/\sqrt{n}$, the size of the typical fluctuation. The left panel shows the tilt at $n = 100$. The original count sits around 10, while the tilted count sits inside the window from 20 up.

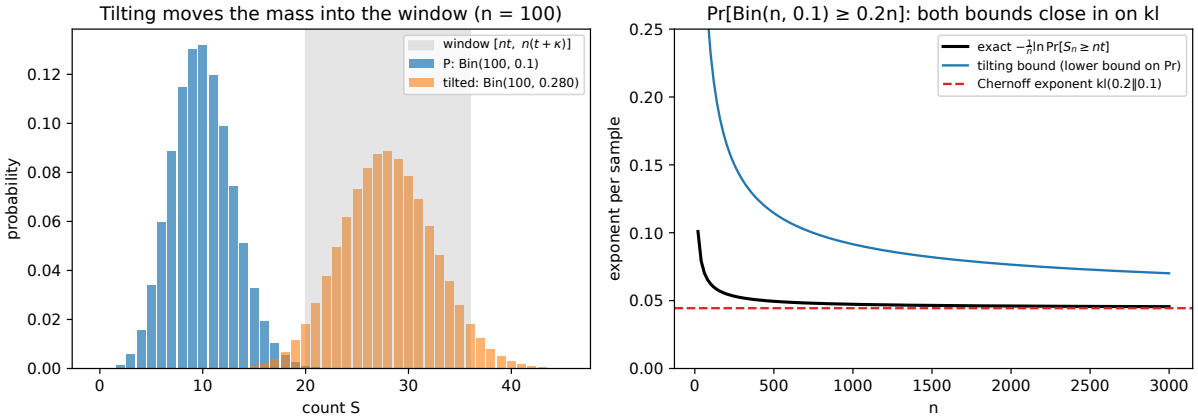


Figure F.3.: Left: the distribution of the number of heads in 100 tosses of a coin with heads probability 0.1, and of the tilted coin, with the window of step 3 shaded. Right: the exact exponent $-\frac{1}{n}\ln \Pr[S_n \geq 0.2n]$ (black), the bound from the tilting argument (blue) and the Chernoff exponent $\mathrm{kl}(0.2\|0.1)$ (dashed), against n.

For the Chernoff–Stein lemma the script uses the spam example of Table 4.1. An email's count of suspicious words, capped at 3, takes the values 0, 1, 2, 3 with probabilities 0.4, 0.3, 0.2, 0.1 under ham (P) and 0.1, 0.2, 0.3, 0.4 under spam (Q). There $D_{\mathrm{KL}}(P\|Q) = 0.4564$ and the variance of the evidence is $V = 0.8348$. The evidence of one count is $\pm\ln 4$ or $\pm\ln 1.5$, so the evidence of a sample is $i\ln 4 + j\ln 1.5$ for two whole numbers i and j. The script tracks the exact distribution of (i, j) under both hypotheses and builds the most powerful test of size exactly 0.05, as the Neyman–Pearson lemma prescribes. Table F.3 gives its type II error. Its last column gives the two bounds of the derivation. They use $\kappa = \sqrt{V/(0.05\,n)}$ for achievability, so that step 2 gives size 0.05, and $\kappa = \sqrt{2V/(0.95\,n)}$ for the converse, so that the factor in step 6 is 0.95/2. A negative lower bound says nothing, because an exponent is never negative.

Table F.3.: Type II error of the most powerful size-0.05 test on n spam-example observations, its exponent, and the two bounds of the derivation with κ chosen for each n. The limit is $D_{\mathrm{KL}}(P\|Q) = 0.4564$.

n	$\mathrm{FNR}_n^*(0.05)$	$-\frac{1}{n}\ln \mathrm{FNR}_n^*$	Bounds from the proof
10	7.702×10^{-2}	0.2564	$[-0.8357, 0.9501]$
50	1.160×10^{-7}	0.3194	$[-0.1214, 0.6588]$
200	3.959×10^{-33}	0.3730	$[0.1675, 0.5539]$

The exponent creeps up toward 0.4564, as Figure F.4 shows. Both bounds tend to KL, at the slow rate $1/\sqrt{n}$. The exact exponent is also far from its limit at moderate n. With 200 observations it is still 18% below KL, because the test must leave room for fluctuations of the evidence of order $\sqrt{n}$.

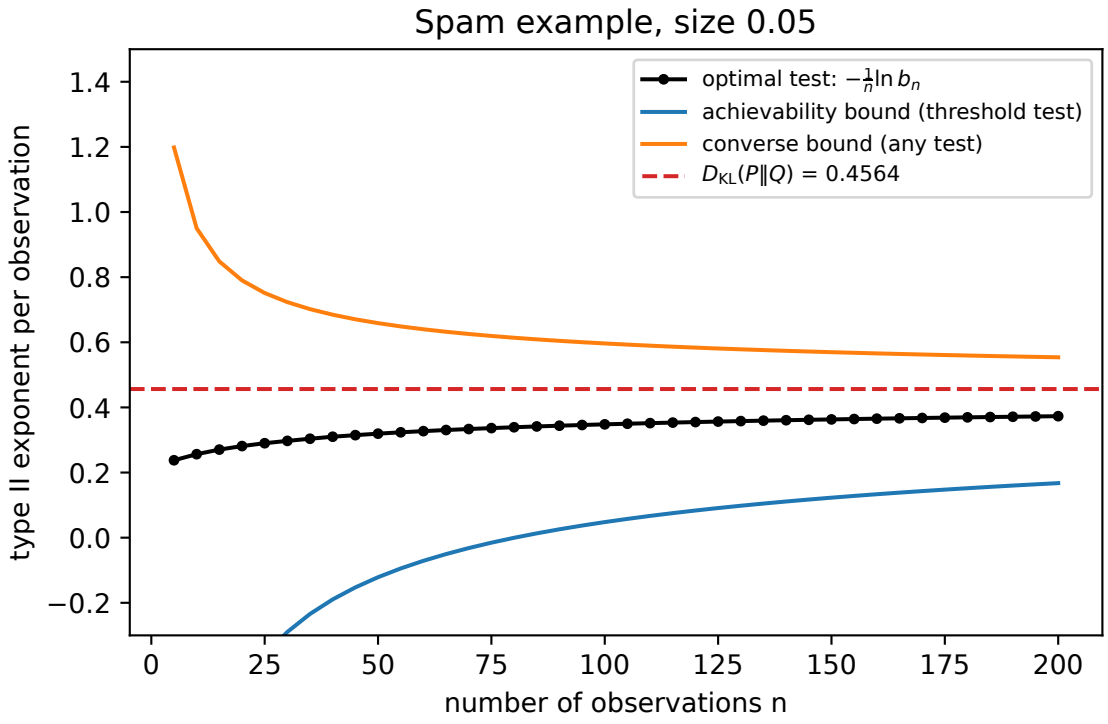


Figure F.4.: The type II exponent $-\frac{1}{n}\ln \mathrm{FNR}_n^*$ of the most powerful size-0.05 test on the spam example (black), between the achievability and converse bounds of the derivation, converging slowly to $D_{\mathrm{KL}}(P\|Q)$ (dashed).

F.2.5. What it does not cover

Cramér's theorem holds without boundedness. The upper bound is always Chernoff's, and the lower bound needs only a tilt with the right mean, which exists in more general settings; Dembo & Zeitouni prove it in full. The Chernoff–Stein argument above uses a finite variance V only for

Chebyshev's inequality. The weak law of large numbers gives the same two limits with only a finite KL divergence, which is the version Cover & Thomas prove. Neither proof says how fast the limit is reached. The examples above show that the approach is slow, with an error of order $1/\sqrt{n}$ in the exponent.

F.3. The Clopper–Pearson limits exist and are unique

F.3.1. What the chapter uses

Section 4.9 builds the Clopper–Pearson interval for a rate from k successes in n independent trials. Write $F_p(k) = \Pr[\mathrm{Bin}(n, p) \le k]$ for the probability of at most k successes when each trial succeeds with probability p. Step 1 of the chapter's derivation says that $F_p(k)$ is continuous and strictly decreasing in p for $k < n$, and concludes that for each k there is a single rate $u(k)$ with $F_{u(k)}(k) = \beta/2$. Here $1 - \beta$ is the confidence level, which the chapter writes $1 - \alpha$. The chapter gives a reason in one phrase, that a larger success probability makes small counts less likely, and points here for the proof. Theorem 47.1 in Auditing relies on the same step for its one-sided bounds.

The precise statement is this. For $0 \le k \le n-1$, the function $p \mapsto F_p(k)$ is continuous and strictly decreasing on $[0, 1]$, from $F_0(k) = 1$ to $F_1(k) = 0$. So for every level c in $(0, 1)$ exactly one rate $u(k)$ in $(0, 1)$ solves $F_{u(k)}(k) = c$. Moreover,

$$F_p(k) = \int_p^1 n \binom{n-1}{k} s^k (1-s)^{n-1-k}\, ds. \tag{F.24}$$

For $k = n$, $F_p(n) = 1$ for every p, and the chapter sets $u(n) = 1$.

F.3.2. The idea

Picture all the trials driven by one set of random numbers. Draw n numbers $U_1, \dots, U_n$ uniformly from $[0, 1]$, and say that trial i succeeds at rate p when $U_i \le p$. Each trial then succeeds with probability p, independently, so the count K_p of successes is $\mathrm{Bin}(n, p)$. Now raise p while keeping the same numbers. A trial that succeeded still succeeds, and some new ones may join, so the count never goes down. For example, with $n = 3$ and $U = (0.15, 0.62, 0.40)$, Table F.4 shows the count as p grows.

Table F.4.: One draw of three uniform numbers, and the count of successes they give at four rates. The count can only go up as p grows.

Rate p	0.1	0.3	0.5	0.7
Trials with $U_i \le p$	none	1	1, 3	1, 2, 3
Count K_p	0	1	2	3

On every draw the event "at most k successes" can only stop happening as p grows. So its probability $F_p(k)$ never increases. The count leaves that event when one more trial crosses over

while exactly k of the other $n-1$ have already succeeded, and that has positive probability for every p strictly between 0 and 1. So the decrease is strict. The derivation makes this exact by differentiating.

F.3.3. Derivation

1. **The endpoints.** At $p = 0$ no trial succeeds, so $F_0(k) = 1$. At $p = 1$ all n succeed, so $F_1(k) = 0$ for $k \le n-1$.
2. **Continuity.** $F_p(k) = \sum_{j=0}^{k} \binom{n}{j} p^j (1-p)^{n-j}$ is a polynomial in p, so it is continuous and can be differentiated.
3. **The derivative telescopes.** Write $g_j(p)$ for the probability of exactly j successes in $n-1$ trials, $\binom{n-1}{j} p^j (1-p)^{n-1-j}$, and set $g_{-1} = 0$. Differentiate one term of the sum by the product rule, and then use the identities $j\binom{n}{j} = n\binom{n-1}{j-1}$ and $(n-j)\binom{n}{j} = n\binom{n-1}{j}$. The result is
$$\frac{d}{dp}\binom{n}{j} p^j (1-p)^{n-j} = n(g_{j-1}(p) - g_j(p)). \tag{F.25}$$
Summing over $j = 0, \dots, k$, each g_j appears once with a plus sign and once with a minus sign, except $g_{-1} = 0$ and the last one. So
$$\frac{d}{dp} F_p(k) = -n \binom{n-1}{k} p^k (1-p)^{n-1-k}. \tag{F.26}$$
For example, with two trials and $k = 0$, $F_p(0) = (1-p)^2$ has derivative $-2(1-p)$, which is what the formula gives. The formula also has a reading in words. When p rises a little, one of the n trials may cross over, and the count leaves "at most k" exactly when the other $n-1$ trials hold exactly k successes.
4. **Strictly decreasing.** The derivative is negative for $0 < p < 1$. By the mean value theorem, for $p < p'$ the difference $F_{p'}(k) - F_p(k)$ equals $(p' - p)$ times the derivative at some point in between, which is negative.
5. **Existence and uniqueness.** A continuous function that goes from 1 to 0 takes every value c in between, by the intermediate value theorem. A strictly decreasing function takes each value at most once. So exactly one $u(k)$ has $F_{u(k)}(k) = c$. Strictness is also what the chapter needs when it turns the limit into a statement about the true rate: the true rate p is at least $u(K)$ exactly when $F_p(K) \le F_{u(K)}(K)$.
6. **The lower end.** For $k \ge 1$, the probability of at least k successes is $1 - F_p(k-1)$. By steps 1 to 4 it is continuous and strictly increasing in p, from 0 to 1. So the chapter's lower limit $l(k)$, the rate at which at least k successes have probability c, also exists and is unique.
7. **The Beta form.** Integrate the derivative of step 3 from p to 1, and use $F_1(k) = 0$. This gives Equation F.24. The integrand is the density of the Beta$(k+1, n-k)$ distribution, because its normalizing constant $1/B(k+1, n-k) = n!/(k!\,(n-1-k)!)$ equals $n\binom{n-1}{k}$. So $F_p(k)$ is the probability that a Beta$(k+1, n-k)$ variable exceeds p. The upper limit $u(k)$ is therefore the point that such a variable exceeds with probability c. This is how statistical software computes Clopper–Pearson limits.

F.3.4. Checking it

Figure F.5 draws $F_p(k)$ against p for $n = 1{,}000$ and $k = 0$, 1 and 10. Each curve falls from 1 to 0 and crosses the level 0.025 exactly once, at the upper limit: $u(0) = 0.003682$, $u(1) = 0.005559$ and $u(10) = 0.018313$. The first has a closed form, because $F_p(0) = (1-p)^n$ gives $u(0) = 1-0.025^{1/1000}$. The script checks the derivative of step 3 against finite differences, and the integral of step 7 by numerical integration. It also recomputes the limits that the chapters quote, by bisection. For 10 successes in 10,000 trials the interval is $[0.00048, 0.00184]$. For 1 success in 1,000 the one-sided 95% bound is 0.0047, and for none in 3,000 it is 9.9808×10^{-4}.

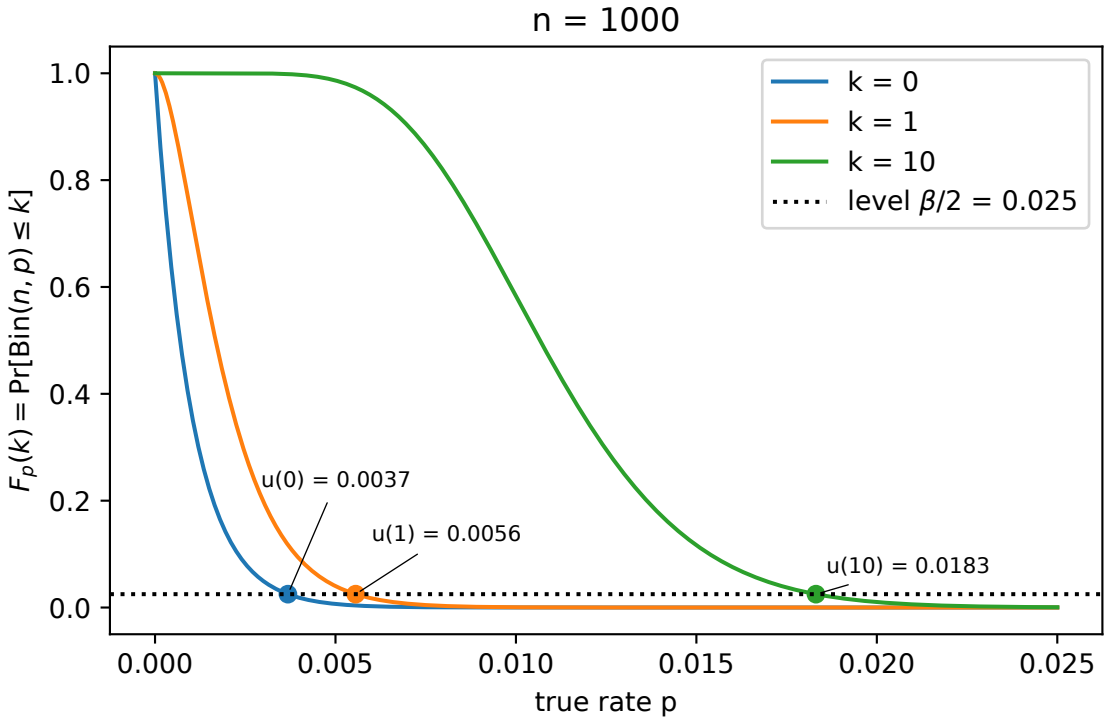


Figure F.5.: The probability $F_p(k)$ of at most k successes in 1,000 trials, as a function of the true rate p, for $k = 0$, 1 and 10. Each curve crosses the level 0.025 (dotted) once, at the upper limit $u(k)$.

Figure F.6 shows what the interval delivers. For $n = 100$ it gives the exact coverage of the 95% Clopper–Pearson interval at each true rate p, the probability that the interval built from a $\mathrm{Bin}(100, p)$ count contains p. The curve is a sawtooth, because it jumps each time p crosses an endpoint of one of the 101 intervals. It never drops below 0.95: its smallest value on the grid is 0.9504 and its average is 0.9645. The Wald interval $\hat{p} \pm 1.96\sqrt{\hat{p}(1-\hat{p})/n}$ of Section 4.9 falls far below 0.95 near the edges. At $p = 0.01$ it covers only 63.34% of the time, against 98.16% for Clopper–Pearson.

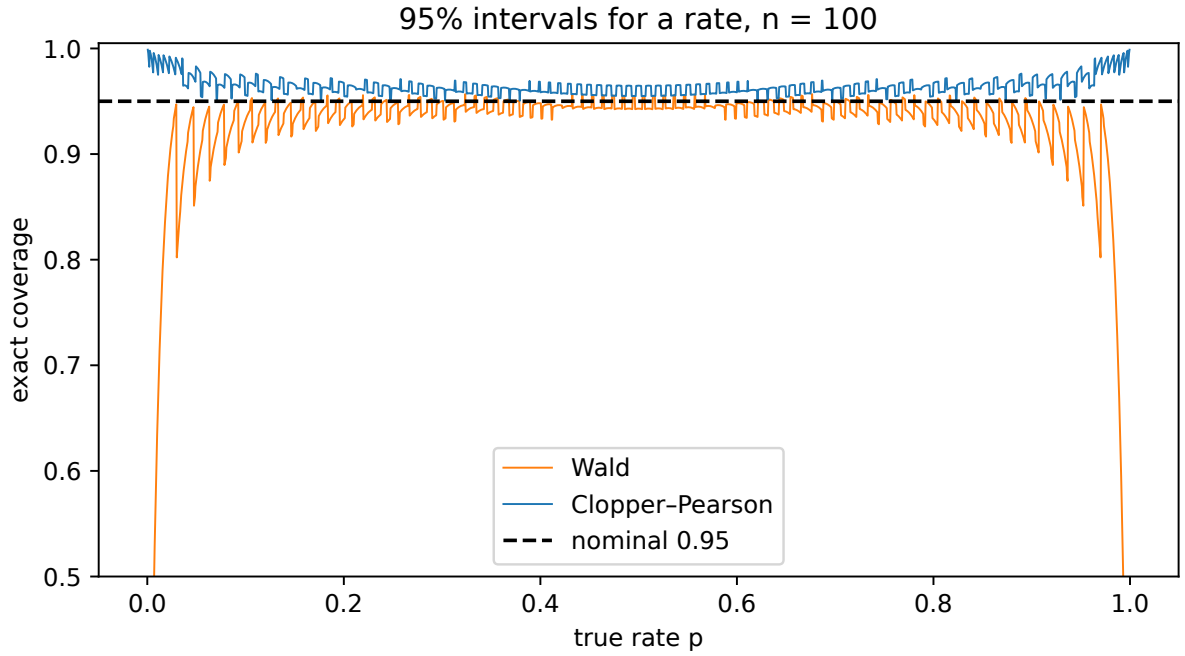


Figure F.6.: Exact coverage of 95% intervals for a rate from 100 trials, as a function of the true rate: Clopper–Pearson (blue) stays above the nominal 0.95 (dashed), and Wald (orange) falls far below it near 0 and 1.

F.4. Rényi divergence tends to KL at order 1

F.4.1. What the chapter uses

Section 3.2 writes the Rényi divergence of order α through the likelihood ratio $r = p/q$ and the log-MGF of the evidence $\ln r$ under P,

$$K_L(\lambda) = \ln \mathbb{E}_P[r^\lambda], \qquad D_\alpha(P\|Q) = \frac{K_L(\alpha - 1)}{\alpha - 1}. \tag{F.27}$$

That is Equation 3.13. Here K_L is the log-MGF of the evidence $L = \ln r$, not of a sum as in the previous section. Property 1 there says that $D_\alpha \to D_{\mathrm{KL}}(P\|Q)$ as $\alpha \to 1$, where $D_{\mathrm{KL}}(P\|Q) = \mathbb{E}_P[\ln r]$ is the KL divergence. The chapter proves it as the slope of K_L at 0, by passing a derivative inside an average. It adds: "This needs K_L to be finite near 0. van Erven & Harremoës prove the limit in general." Write $\lambda = \alpha - 1$. This section proves two statements without derivatives:

$$\lim_{\alpha \uparrow 1} D_\alpha(P\|Q) = D_{\mathrm{KL}}(P\|Q) \text{ always}, \qquad \lim_{\alpha \downarrow 1} D_\alpha(P\|Q) = D_{\mathrm{KL}}(P\|Q) \text{ if } D_{\alpha_0}(P\|Q) < \infty \text{ for some } \alpha_0 > 1. \tag{F.28}$$

The first holds even when the KL divergence is infinite.

Section 5.4 states the matching facts for the **Rényi entropy** $H_\alpha(X) = \frac{1}{1-\alpha} \log_2 \sum_x p(x)^\alpha$, measured in bits, of a variable with finitely many values. It says that H_α tends to the Shannon entropy H as $\alpha \to 1$, and to the min-entropy H_∞ as $\alpha \to \infty$. Both follow from the divergence limits.

F.4.2. The idea

The divergence $D_{1+\lambda}$ is $K_L(\lambda)/\lambda$, and two simple inequalities trap it between the KL divergence and a quantity whose limit is easy. The first is Jensen's inequality for the concave logarithm: the log of an average is at least the average of the log. The second is the tangent-line bound $\ln m \le m - 1$. Applied to the average $\mathbb{E}_P[r^\lambda]$, they give

$$\lambda D_{\mathrm{KL}} = \mathbb{E}_P[\ln r^\lambda] \le K_L(\lambda) \le \mathbb{E}_P[r^\lambda] - 1. \tag{F.29}$$

Dividing by λ, with care for its sign, puts $D_{1+\lambda}$ between D_{KL} and the average of the **power slope**

$$h_\lambda(r) = \frac{r^\lambda - 1}{\lambda}. \tag{F.30}$$

For each fixed value of the ratio r, the power slope moves steadily toward $\ln r$ as λ tends to 0. So its average moves toward the average of $\ln r$, which is the KL divergence, and no derivative of K_L is needed.

On the die of Table 3.1, r is 3 on a six and 0.6 otherwise. Table F.5 shows h_λ at both values. Each row increases from left to right, and the middle column, $\lambda = 0$, is $\ln r$.

Table F.5.: The power slope $h_\lambda(r) = (r^\lambda - 1)/\lambda$ at the die's two likelihood ratios. At $\lambda = 0$ it is $\ln r$.

λ	-0.5	-0.1	0	0.1	0.5
$h_\lambda(3)$	0.8453	1.0404	1.0986	1.1612	1.4641
$h_\lambda(0.6)$	-0.5820	-0.5241	-0.5108	-0.4980	-0.4508

F.4.3. Derivation

The derivation uses one theorem about averages, the **monotone convergence theorem**. Suppose random variables $Z_1 \le Z_2 \le \ldots$ increase to a limit Z at every outcome, and $\mathbb{E}[Z_1] > -\infty$. Then $\mathbb{E}[Z_k] \to \mathbb{E}[Z]$, and the limit may be $+\infty$. The same holds for a decreasing sequence if $\mathbb{E}[Z_1] < \infty$. For finitely many outcomes this is just the limit of a finite sum. For densities, it is a standard theorem of integration.

1. **The power slopes.** For a number $x > 0$, write $z = \ln x$, so that $x^\lambda = e^{\lambda z}$. Then the power slope is an integral,

$$h_\lambda(x) = \frac{e^{\lambda z} - 1}{\lambda} = \int_0^z e^{\lambda s}\, ds, \tag{F.31}$$

which runs backward when $z < 0$. At $\lambda = 0$ the integral is $z = \ln x$, and it changes continuously with λ. It is also non-decreasing in λ. If $z > 0$, every s in the integral is positive, so $e^{\lambda s}$ grows with λ. If $z < 0$, the integral is $-\int_z^0 e^{\lambda s}\, ds$ with negative s, so $e^{\lambda s}$ shrinks with λ and its negative grows. So as $\lambda \uparrow 0$, $h_\lambda(x)$ increases to $\ln x$, and as $\lambda \downarrow 0$ it decreases to $\ln x$.

2. **Below order 1, an upper bound.** Take $\lambda \in (-1, 0)$. Then $r^\lambda = (q/p)^{|\lambda|}$, and Jensen's inequality for the concave power $|\lambda| < 1$ gives $\mathbb{E}_P[r^\lambda] \le (\mathbb{E}_P[q/p])^{|\lambda|} \le 1$, so $K_L(\lambda)$ is not $+\infty$. (If P and Q share no outcome, $K_L(\lambda) = -\infty$ and every divergence is infinite, so there is nothing to prove.) Next, the logarithm is concave, so Jensen's inequality gives

$$K_L(\lambda) = \ln \mathbb{E}_P[r^\lambda] \ge \mathbb{E}_P[\ln r^\lambda] = \lambda D_{\mathrm{KL}}. \tag{F.32}$$

   Dividing by the negative λ flips the inequality, so $D_{1+\lambda}$ is at most D_{KL}. When the KL divergence is infinite this says nothing, and nothing is needed.
3. **Below order 1, a lower bound.** Since $\ln m \le m - 1$, $K_L(\lambda) \le \mathbb{E}_P[r^\lambda] - 1 = \lambda\, \mathbb{E}_P[h_\lambda(r)]$. Dividing by the negative λ flips it again:

$$\mathbb{E}_P[h_\lambda(r)] \le D_{1+\lambda} \le D_{\mathrm{KL}}, \qquad -1 < \lambda < 0. \tag{F.33}$$

4. **The limit from below.** As $\lambda \uparrow 0$, $h_\lambda(r)$ increases to $\ln r$ by step 1. The sequence starts above an integrable variable: at $\lambda = -1$, $h_{-1}(r) = 1 - 1/r$, and $\mathbb{E}_P[1/r] = \sum_{x:\, p(x)>0} q(x) \le 1$, so $\mathbb{E}_P[h_{-1}(r)] \ge 0$. So monotone convergence applies, and the average of the power slope tends to the average of $\ln r$, which is the KL divergence. Both sides of step 3 tend to D_{KL}, so $D_\alpha \to D_{\mathrm{KL}}$ as $\alpha \uparrow 1$. This includes an infinite KL divergence, where the lower bound alone grows without limit.
5. **Above order 1.** For $\lambda > 0$ the same two inequalities hold, and dividing by the positive λ keeps their direction:

$$D_{\mathrm{KL}} \le D_{1+\lambda} \le \mathbb{E}_P[h_\lambda(r)], \qquad \lambda > 0. \tag{F.34}$$

6. **The limit from above.** As $\lambda \downarrow 0$, $h_\lambda(r)$ decreases to $\ln r$. Monotone convergence now needs a finite starting point. That is the assumption: if $D_{\alpha_0} < \infty$ with $\lambda_0 = \alpha_0 - 1 > 0$, then $\mathbb{E}_P[r^{\lambda_0}] = e^{\lambda_0 D_{\alpha_0}}$ is finite, and so is $\mathbb{E}_P[h_{\lambda_0}(r)]$. Then $\mathbb{E}_P[h_\lambda(r)] \to D_{\mathrm{KL}}$ and $D_\alpha \to D_{\mathrm{KL}}$ as $\alpha \downarrow 1$.
7. **Rényi entropy.** Let U be the uniform distribution on the N values of X, which gives each value probability $1/N$. Measuring the divergence in bits too, with $\log_2$ in place of ln, the definition gives

$$D_\alpha(P\|U) = \frac{1}{\alpha - 1} \log_2 \sum_x p(x)^\alpha N^{\alpha-1} = \log_2 N - H_\alpha(X). \tag{F.35}$$

   The identity holds in any base used for both sides. In the same way, the KL divergence from the uniform distribution is $\log_2 N$ minus the Shannon entropy $H(X)$. On a finite set the likelihood ratio $Np(x)$ is at most N, so every $D_\alpha(P\|U)$ is finite. Both limits of steps 4 and 6 then apply, and $H_\alpha \to H$ as $\alpha \to 1$. As $\alpha \to \infty$, property 2 of Section 3.2 says that $D_\alpha(P\|U)$ tends to the logarithm of the largest ratio, $\log_2(N \max_x p(x))$. Subtracting it from $\log_2 N$ leaves the min-entropy,

$$\lim_{\alpha\to\infty} H_\alpha(X) = -\log_2 \max_x p(x) = H_\infty(X). \tag{F.36}$$

F.4.4. Checking it

Figure F.7 draws the sandwich for the die and for the pair $\mathcal{N}(0,1)$ against $\mathcal{N}(1,1)$, whose KL divergence is $\frac{1}{2}$. The black curve is $D_{1+\lambda}$ and the blue curve is $\mathbb{E}_P[h_\lambda(r)]$. The blue curve lies below the black on the left of 0 and above it on the right, and both meet the KL line at $\lambda = 0$.

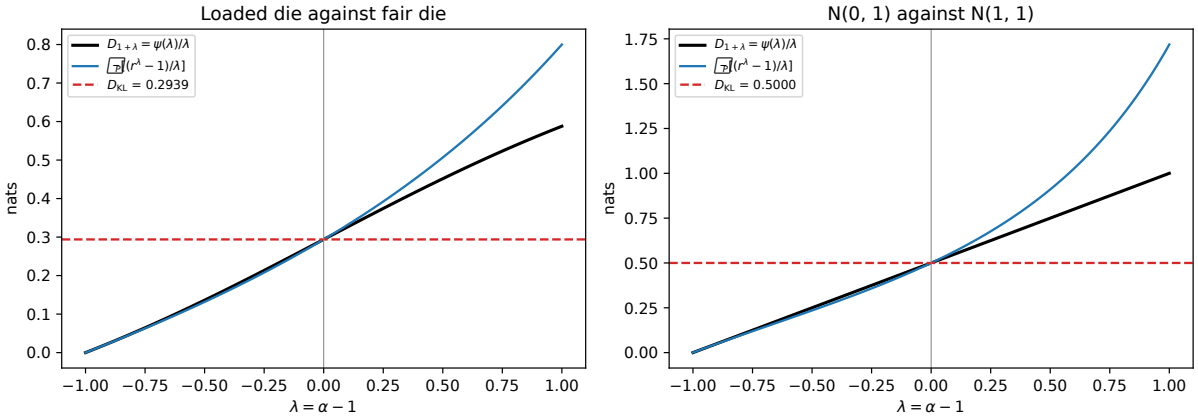


Figure F.7.: The Rényi divergence $D_{1+\lambda} = K_L(\lambda)/\lambda$ (black) and the bound $\mathbb{E}_P[(r^\lambda - 1)/\lambda]$ (blue) as functions of $\lambda = \alpha - 1$, with the KL divergence (dashed). Left: the loaded die against the fair die. Right: $\mathcal{N}(0, 1)$ against $\mathcal{N}(1, 1)$.

On the die, where $D_{\mathrm{KL}} = 0.2939$, the divergence at $\lambda = -0.01$ is 0.2907 with the lower bound 0.2902, and at $\lambda = +0.01$ it is 0.2971 with the upper bound 0.2976.

For the PIN example of Section 5.4, with 10% on one PIN and the rest uniform over the other 9,999, the Shannon entropy is $H = 12.4278$ bits. The script finds $H_{0.999} = 12.4310$ and $H_{1.001} = 12.4246$ on either side of it. As the order grows, $H_2 = 6.6322$ and $H_{100} = 3.3555$ approach the min-entropy $H_\infty = 3.3219$. The identity of step 7 holds to rounding error.

F.4.5. What it does not cover

Without the assumption of step 6, the limit from above can fail. Take outcomes $k = 1, 2, \ldots$ with $p(k)$ proportional to $e^{-\sqrt{k}}$, and let $q(k) = p(k)e^{-k}$, with the rest of Q's mass on an extra outcome that P never produces. Then $\ln r(k) = k$, so $D_{\mathrm{KL}} = \mathbb{E}_P[k]$ is finite. But $\mathbb{E}_P[r^\lambda]$ sums $e^{\lambda k - \sqrt{k}}$ times a constant, which diverges for every $\lambda > 0$. So $D_\alpha = \infty$ for every $\alpha > 1$, while the limit from below is the finite KL divergence. van Erven & Harremoës (2014) treat the general case, including orders near 0 and ∞.

F.5. Perspective functions are jointly convex

F.5.1. What the chapter uses

Let g be a convex function on $(0, \infty)$. Its **perspective** is a function G of two positive numbers x and y. It applies g to their ratio and multiplies the result by the denominator:

$$G(x, y) = y\, g(x/y). \tag{F.37}$$

The perspective has three properties. The first is that it is **jointly convex**. Take several points (x_t, y_t), indexed by t, and weights $w_t \geq 0$ that add up to 1. The value of G at the weighted

average of the points is at most the weighted average of its values:

$$G\Big(\sum_t w_t x_t,\ \sum_t w_t y_t\Big) \le \sum_t w_t\, G(x_t, y_t). \tag{F.38}$$

The second is that it is **positively homogeneous**, which means that scaling both arguments by a factor $c > 0$ scales the value by c. The third follows from the first two. The perspective is **subadditive**, so its value at a sum of two points is at most the sum of its values:

$$G(x_1 + x_2,\ y_1 + y_2) \le G(x_1, y_1) + G(x_2, y_2). \tag{F.39}$$

The Rényi divergence uses $g(t) = t^\alpha$ with $\alpha > 1$, whose perspective is $G(x, y) = x^\alpha y^{1-\alpha}$. Write the sum in the Rényi divergence as

$$F(P, Q) = \sum_y P(y)^\alpha Q(y)^{1-\alpha} = e^{(\alpha-1)D_\alpha(P\|Q)}. \tag{F.40}$$

Several chapters assert one of the properties above for this function, without a proof:

- Step 3 of the proof of Theorem 30.2, in the chapter on converting Rényi differential privacy (RDP) guarantees, uses that a test cannot increase the Rényi divergence, citing the data-processing inequality.
- The paragraph after the proof of Theorem 30.2 says that each constraint of Equation 30.15 is convex in the test's errors $(\mathrm{FPR}, \mathrm{FNR})$, so that the region they allow is convex and its lower boundary f_{RDP} is a convex function. It also uses that each constraint is smallest, equal to 1, at $\mathrm{FNR} = 1 - \mathrm{FPR}$. Step 3 of Section 32.7 repeats the convexity claim.
- Step 2 of the proof of Proposition 31.4 uses the joint convexity of F in the pair, for the pair (Q, P) written as a mixture.
- Step 2 of Section 33.3 uses it to show that a mixture of pairs with shared weights has a Rényi divergence no larger than the largest of its parts. It cites van Erven & Harremoës (2014, Theorem 13) for this **quasi-convexity**.

This section proves the three properties and these four uses.

F.5.2. The idea

The perspective is g of a ratio, weighted by the denominator. When you mix two points, the ratio of the mixture is an average of the two ratios, weighted by the denominators. A convex function of an average is at most the average of the function (Jensen), and multiplying back by the total denominator gives the joint convexity.

Take $\alpha = 2$, so $G(x, y) = x^2/y$, and the two points $(0.9, 0.3)$ and $(0.1, 0.9)$. Their values are $0.81/0.3 = 2.70$ and $0.01/0.9 = 0.0111$, and their ratios x/y are 3 and $\frac{1}{9}$. The midpoint is $(0.5, 0.6)$, whose ratio is $0.5/0.6 = 0.833$. That is the average of 3 and $\frac{1}{9}$ with weights $0.3/1.2 = \frac{1}{4}$ and $0.9/1.2 = \frac{3}{4}$, the shares of the two denominators. At the midpoint $G = 0.25/0.6 = 0.4167$, well below the average of the two values, $(2.70 + 0.0111)/2 = 1.3556$.

In the language of divergences, x is P's probability of an output and y is Q's. Merging two outputs replaces their likelihood ratios by an average weighted by Q, as in the data-processing section of Divergences, and a convex summary of the ratios can only go down.

F.5.3. Derivation

1. **Joint convexity.** Fix the points (x_t, y_t) and the weights w_t, and let $x = \sum_t w_t x_t$ and $y = \sum_t w_t y_t$. The numbers $v_t = w_t y_t / y$ are non-negative and add up to 1. They make the ratio of the mixture an average of the ratios,
$$\frac{x}{y} = \sum_t \frac{w_t y_t}{y} \cdot \frac{x_t}{y_t} = \sum_t v_t \frac{x_t}{y_t}. \tag{F.41}$$
Jensen's inequality for the convex g gives $g(x/y) \le \sum_t v_t\, g(x_t/y_t)$. Multiplying by y turns each $y\, v_t$ back into $w_t y_t$, which is Equation F.38.
2. **Homogeneity.** $G(cx, cy) = cy\, g(cx/cy) = c\, y\, g(x/y)$, because the ratio does not change.
3. **Subadditivity.** By homogeneity with $c = 2$, $G(x_1 + x_2, y_1 + y_2) = 2\, G(\frac{x_1+x_2}{2}, \frac{y_1+y_2}{2})$. By joint convexity with weights $\frac{1}{2}$, this is at most $G(x_1, y_1) + G(x_2, y_2)$.
4. **Zero probabilities.** For $G(x, y) = x^\alpha y^{1-\alpha}$, outputs where a probability is 0 follow the conventions $G(0, y) = 0$ and $G(x, 0) = \infty$ for $x > 0$. Steps 1 to 3 still hold, case by case. If some point has $y_t = 0 < x_t$, the right side is infinite, so there is nothing to check, while a point $(0, 0)$ adds nothing to either side. With $x_t = 0 < y_t$, the formula $y\, g(x/y)$ still applies, since $g(0) = 0$ and g is convex on $[0, \infty)$.
5. **Mixtures of pairs: quasi-convexity.** Let (P_t, Q_t) be pairs of distributions on the same outputs, and mix them with shared weights w_t. Apply step 1 at each output y, to the points $(P_t(y), Q_t(y))$, and add over y. Then
$$F\Big(\sum_t w_t P_t,\ \sum_t w_t Q_t\Big) \le \sum_t w_t\, F(P_t, Q_t) \le \max_t F(P_t, Q_t). \tag{F.42}$$
For $\alpha > 1$, $D_\alpha = \frac{1}{\alpha-1} \ln F$ increases with F. So the Rényi divergence of the mixed pair is at most the largest $D_\alpha(P_t \| Q_t)$. This is step 2 of Section 33.3. For densities, replace the sum over y by an integral, and the inequality survives because it holds at every y.
6. **The pair (Q, P) for a subsampled Gaussian.** In Section 31.6, $Q = \mathcal{N}(0, \sigma^2)$ and $P = (1-q)Q + q\,\mathcal{N}(1, \sigma^2)$. The pair (Q, P) is the mixture $(1-q)(Q, Q) + q\,(Q, \mathcal{N}(1, \sigma^2))$, with the shared weights $1-q$ and q. The first inequality of step 5 gives
$$F(Q, P) \le (1-q)F(Q, Q) + q\, F(Q, \mathcal{N}(1, \sigma^2)) = (1-q) + q\, e^{(\alpha-1)\alpha\rho_0}, \tag{F.43}$$
because $F(Q, Q) = 1$, and D_α between two normals with unit shift is $\alpha\rho_0$, where $\rho_0 = 1/(2\sigma^2)$. That is the bound the chapter states.
7. **Tests cannot increase the divergence.** A test rejects on output y with probability $\phi(y)$, which may lie strictly between 0 and 1. Split each output y into two pieces, "y and reject" and "y and accept". Their probabilities are $\phi(y)P(y)$ and $(1 - \phi(y))P(y)$ under P, and likewise under Q. Both pieces keep the ratio $P(y)/Q(y)$, so by homogeneity their two terms add up to $G(P(y), Q(y))$, and F does not change. Now merge all the "reject" pieces into one output and all the "accept" pieces into another. By subadditivity, applied one merge at a time, F can only go down. The two merged outputs are the test's coin. It rejects with probability $\mathrm{FPR} = \sum_y \phi(y)P(y)$ under P and $1 - \mathrm{FNR} = \sum_y \phi(y)Q(y)$ under Q, where FPR and FNR are its type I and type II errors. So
$$G(\mathrm{FPR}, 1 - \mathrm{FNR}) + G(1 - \mathrm{FPR}, \mathrm{FNR}) \le F(P, Q), \tag{F.44}$$
which is the data-processing step of the proof of Theorem 30.2, randomized tests included. The same split-and-merge argument works for any processing that sends each output y to a random output z, so it also proves the data-processing inequality for D_α.

8. **The region of allowed tests is convex.** The chapter's RDP curve $\varepsilon(\alpha)$ bounds the Rényi divergence of order α in both directions, and the coin of step 7 inherits both bounds. Written with G, the two constraints of Equation 30.15 are

$$c_1(\mathrm{FPR}, \mathrm{FNR}) = G(\mathrm{FPR}, 1-\mathrm{FNR}) + G(1-\mathrm{FPR}, \mathrm{FNR}) \le e^{(\alpha-1)\varepsilon(\alpha)}, \qquad c_2(\mathrm{FPR}, \mathrm{FNR}) = G(1-\mathrm{FNR}, \tag{F.45}$$

   Each term is G applied to numbers that depend linearly on $(\mathrm{FPR}, \mathrm{FNR})$. Mixing two error pairs mixes those numbers with the same weights, so step 1 makes each term, and their sum, convex in $(\mathrm{FPR}, \mathrm{FNR})$. The same holds for c_2, which swaps the roles of the two distributions. If two error pairs satisfy a convex constraint, so does every mixture of them, because its value is at most the mixture of two values that are both within the limit. So each constraint allows a convex set of pairs, and so does their intersection.
9. **The boundary is convex, and each constraint is smallest at** $\mathrm{FNR} = 1 - \mathrm{FPR}$**.** Let $f_{\mathrm{RDP}}(\mathrm{FPR})$ be the smallest allowed FNR at type I error FPR. For allowed pairs $(\mathrm{FPR}_1, \mathrm{FNR}_1)$ and $(\mathrm{FPR}_2, \mathrm{FNR}_2)$, the mixture is allowed, so f_{RDP} at the mixed FPR is at most the mixed FNR. Taking each FNR_i as small as allowed shows that f_{RDP} is convex. At $\mathrm{FNR} = 1 - \mathrm{FPR}$, homogeneity gives $c_1 = G(\mathrm{FPR}, \mathrm{FPR}) + G(1-\mathrm{FPR}, 1-\mathrm{FPR}) = \mathrm{FPR} + (1-\mathrm{FPR}) = 1$. Subadditivity merges the two terms, whose arguments add up to $(\mathrm{FPR} + 1 - \mathrm{FPR},\ 1 - \mathrm{FNR} + \mathrm{FNR}) = (1, 1)$, so $c_1 \ge G(1,1) = 1$ for every $(\mathrm{FPR}, \mathrm{FNR})$, so 1 is the smallest value. A convex function of FNR that is smallest at $1 - \mathrm{FPR}$ decreases on $[0, 1-\mathrm{FPR}]$, which is what the chapter needs to find $f_{\mathrm{RDP}}(\mathrm{FPR})$ by bisection on FNR.

F.5.4. Checking it

At $\alpha = 2$, the script draws 100,000 random pairs of points and weights. It finds no violation of joint convexity or of subadditivity. It also draws 20,000 random mixtures of three pairs of distributions on five outputs, and finds no mixture whose Rényi divergence exceeds the largest of its parts.

On the die, the loaded die gives a six probability 0.5 and each other face 0.1. Take the randomized test that rejects on a six, and with probability $\frac{1}{2}$ on a five. Its type I error is $\mathrm{FPR} = 0.5 + 0.05 = 0.55$, and it rejects under the fair die with probability $1 - \mathrm{FNR} = \frac{1}{6} + \frac{1}{12} = 0.25$. Its coin has $D_2 = 0.3920$, below $D_2(P\|Q) = 0.5878$ for the full roll, as step 7 requires.

For the subsampled Gaussian of step 6, with $q = 0.1$, $\sigma = 1$ and $\alpha = 2$, numerical integration gives $F(Q, P) = 1.0115$, below the bound $0.9 + 0.1e = 1.1718$. The bound is loose here because it charges the full Gaussian divergence to the fraction q of the mixture.

The right panel of Figure F.8 draws the region that order 2 alone allows for the Gaussian mechanism whose sensitivity equals its noise standard deviation ($\mu = 1$ in the chapters), whose RDP curve gives $\varepsilon(2) = 1$. At $\mathrm{FPR} = 0.1$ the boundary is $f_{\mathrm{RDP}}(0.1) = 0.5068$, so the largest power is 0.4932, the order-2 value that Section 30.3 reports. The script checks convexity on a grid of 199 values of FPR: the smallest second difference of f_{RDP} is positive. In the figure, the chord between two boundary points stays inside the region. The boundary has a kink where the binding constraint switches from c_2 to c_1, and convexity survives it.

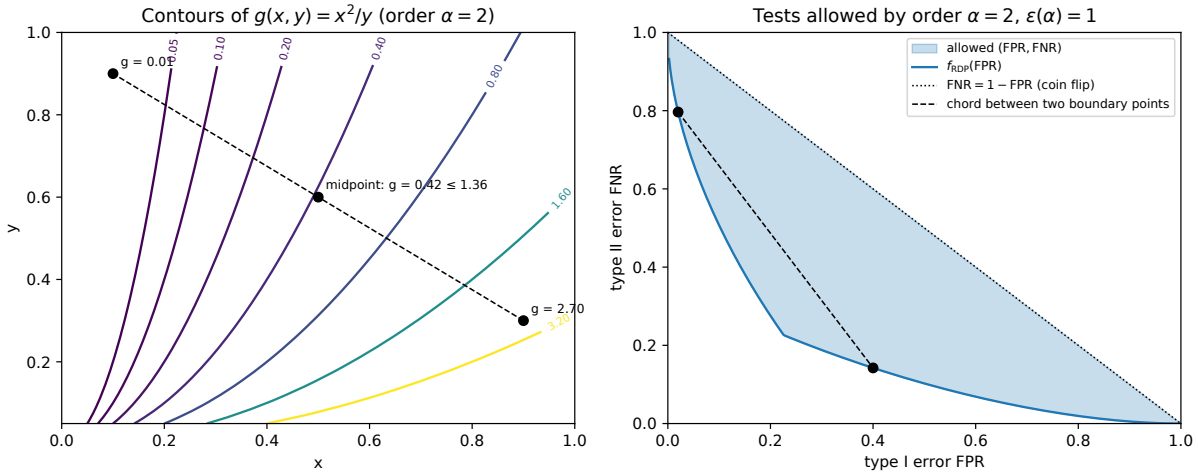


Figure F.8.: Left: contours of $G(x, y) = x^2/y$, the perspective of $g(t) = t^2$, with two points, the chord between them and its midpoint, where the function value is below the average of the two end values. Right: the pairs of type I and type II errors $(\mathrm{FPR}, \mathrm{FNR})$ that order $\alpha = 2$ with $\varepsilon(2) = 1$ allows (shaded), its lower boundary f_{RDP}, and a chord between two boundary points that stays inside.

F.6. A noisy count has an increasing likelihood ratio

F.6.1. What the chapter uses

The example of Section 9.5 releases a noisy count of $m = 100$ other records plus the target's bit b. The adversary sees $K + b + Z$, where the unknown count K of the others is $\mathrm{Bin}(m, \frac{1}{2})$ and the noise Z is normal with mean 0 and standard deviation $\sigma = 1$. Write $w_k = \binom{m}{k} 2^{-m}$ for the binomial weights and φ_σ for the normal density with standard deviation σ. In world 0 ($b = 0$) the observation has density

$$p(y) = f(y), \qquad f(y) = \sum_{k=0}^{m} w_k \, \varphi_\sigma(y - k), \tag{F.46}$$

and in world 1 ($b = 1$) it has the shifted density $q(y) = f(y - 1)$.

The chapter says that the likelihood ratio increases with y, so the best attack is a threshold, at $m/2 + \frac{1}{2} = 50.5$ by symmetry. A tempting reason is that f is log-concave, because a binomial and a normal are both log-concave and so is their sum. That reason is not valid in general, because the rule it quotes concerns two variables that both have log-concave densities on the real line. A binomial count lives on the integers and has no density there, and for such sums the rule fails. For example, a Bernoulli($\frac{1}{2}$) variable plus normal noise with $\sigma = 0.2$ has two separate peaks, as the left panel of Figure F.9 shows, so it is not log-concave. With $m = 100$ and $\sigma = 1$ the density does happen to be log-concave, but with $\sigma = 0.5$ it is not.

The attack needs less than log-concavity, and what it needs holds for every $\sigma > 0$:

$$\frac{q(y)}{p(y)} = \frac{f(y - 1)}{f(y)} \text{ is strictly increasing in } y. \tag{F.47}$$

The proof below uses two properties. The weights w_k must form a **log-concave sequence**, $w_k^2 \geq w_{k-1} w_{k+1}$, with no zero between two non-zero weights. The noise must be normal.

F.6.2. The idea

Both densities are mixtures over the unknown count of the others. Comparing the ratio at two observations $y < y'$ therefore means comparing two double sums, over the count j in one world and the count k in the other. Pair each term (j, k) with its mirror image (k, j). Each pair turns into a product of two factors. One compares the binomial weights, and it is non-negative because the binomial is log-concave. The other compares normal densities, and it is non-negative because a larger observation favours a larger centre. A product of two non-negative factors is non-negative, and so is the sum.

For a single Bernoulli($\frac{1}{2}$) record ($m = 1$), the weights are $w_0 = w_1 = \frac{1}{2}$, and $w_k = 0$ for every other k. The pairs with $j > k$ whose weight factor $w_{j-1} w_k - w_{k-1} w_j$ is not zero are listed in Table F.6. All three weight factors are positive, even though the density has two peaks.

Table F.6.: The weight factors of step 4 for one Bernoulli($\frac{1}{2}$) record, for the pairs $j > k$ where they are not zero.

Pair (j, k)	$w_{j-1} w_k$	$w_{k-1} w_j$	Weight factor
$(1, 0)$	$w_0 w_0 = \frac{1}{4}$	$w_{-1} w_1 = 0$	$\frac{1}{4}$
$(2, 0)$	$w_1 w_0 = \frac{1}{4}$	$w_{-1} w_2 = 0$	$\frac{1}{4}$
$(2, 1)$	$w_1 w_1 = \frac{1}{4}$	$w_0 w_2 = 0$	$\frac{1}{4}$

F.6.3. Derivation

Set $w_i = 0$ for i outside $0, \ldots, m$, and write φ for φ_σ. Substituting $j = k + 1$ in $q(y) = f(y - 1)$ writes both densities as sums over a centre:

$$p(y) = \sum_k w_k \, \varphi(y - k), \qquad q(y) = \sum_j w_{j-1} \varphi(y - j). \tag{F.48}$$

1. **What to show.** The ratio increases exactly when $q(y')/p(y') \geq q(y)/p(y)$ for every $y < y'$. Multiplying by the positive $p(y)p(y')$, this is $\Delta = q(y')p(y) - q(y)p(y') \geq 0$.
2. **Expand.** Multiplying out the sums,

$$\Delta = \sum_{j,k} a_{jk} b_{jk}, \qquad a_{jk} = w_{j-1} w_k, \qquad b_{jk} = \varphi(y' - j)\varphi(y - k) - \varphi(y - j)\varphi(y' - k). \tag{F.49}$$

3. **Pair each term with its mirror.** Swapping j and k swaps the two products in b_{jk}, so $b_{kj} = -b_{jk}$ and $b_{jj} = 0$. Split the sum into the terms with $j > k$ and those with $j < k$, and rename the indices in the second part. Then

$$\Delta = \sum_{j>k} (a_{jk} - a_{kj}) b_{jk} = \sum_{j>k} \left(w_{j-1} w_k - w_{k-1} w_j\right) b_{jk}. \tag{F.50}$$

4. **The weight factor is non-negative.** Take $j > k$. If $w_{k-1} = 0$ or $w_j = 0$, the subtracted product is 0 and the factor is at least 0. Otherwise $k-1$ and j both carry weight, and so does every index between them, because there are no internal zeros. Log-concavity, $w_i^2 \geq w_{i-1} w_{i+1}$, says that the ratio w_{i-1}/w_i never decreases as i grows. So $w_{j-1}/w_j \geq w_{k-1}/w_k$, which is $w_{j-1} w_k \geq w_{k-1} w_j$. For the binomial, $w_{i-1}/w_i = i/(m-i+1)$, which increases with i.
5. **The kernel factor is positive.** Take $j > k$ and $y < y'$. Compare the logarithms of the two products in b_{jk}. Expanding the squares in the normal densities, the terms in y^2, y'^2, j^2 and k^2 cancel, and

$$\ln \frac{\varphi(y'-j)\varphi(y-k)}{\varphi(y-j)\varphi(y'-k)} = \frac{(y-j)^2 + (y'-k)^2 - (y'-j)^2 - (y-k)^2}{2\sigma^2} = \frac{(y'-y)(j-k)}{\sigma^2} > 0. \tag{F.51}$$

   So $b_{jk} > 0$. In words, a larger observation is relatively more likely under the larger centre.
6. **Conclude.** Every term of step 3 is a non-negative weight factor times a positive kernel factor, so $\Delta \geq 0$. The pair $(1, 0)$ has weight factor $w_0^2 > 0$, so $\Delta > 0$ and the ratio is strictly increasing. This proves Equation F.47.
7. **The best attack is a threshold.** By the Neyman–Pearson lemma (Section 4.2), the most powerful test rejects where the likelihood ratio q/p is above a cut-off. When the ratio increases with y, that region is $\{y > \tau\}$ for some threshold τ.
8. **The threshold.** The binomial weights are symmetric, $w_k = w_{m-k}$, and so is the noise, so $f(m/2 + s) = f(m/2 - s)$ for every s. The ratio equals 1 where $f(y-1) = f(y)$. That happens when $y - 1$ and y sit symmetrically around $m/2$, at $y = m/2 + \frac{1}{2}$. Because the ratio is strictly increasing, it crosses 1 only there. The test with the best advantage rejects exactly where $q > p$, that is, where the ratio is above 1 (Section 4.6), which is $\{y > 50.5\}$ for $m = 100$.

F.6.4. Checking it

For the Bernoulli($\frac{1}{2}$) record with $\sigma = 0.2$, the script finds two peaks in the density. On a grid of 4,001 points the log-ratio still increases at every step. It also evaluates the double sum of step 3 directly and matches Δ to rounding error, both for this case and for $m = 100$. For $m = 100$ and $\sigma = 1$, the ratio crosses 1 at 50.5000, and the advantage of that threshold is 0.0779, the exact value in Table 9.5. The largest second difference of $\ln f$ on a grid is negative (-3.8×10^{-6}) at $\sigma = 1$ and positive ($+5.5 \times 10^{-5}$) at $\sigma = 0.5$. So the density is log-concave in the chapter's case but not for smaller noise. The ratio increases in both cases, as the right panel of Figure F.9 shows.

F.6.5. What it does not cover

The proof needs the kernel inequality of step 5 for the noise. Step 5 proves it for normal noise, and it holds for every log-concave noise density, such as the Laplace density (Karlin calls such shift kernels totally positive of order 2). For other noise the argument does not apply. It also needs log-concave weights without internal zeros. A count whose other records have unequal probabilities is a sum of independent Bernoulli variables, and such a count still has log-concave weights, so the result covers it too.

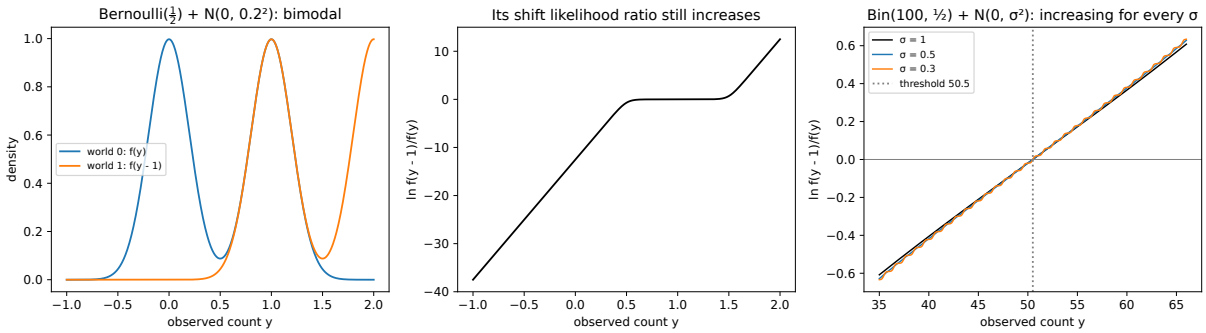


Figure F.9.: Left: the densities of a Bernoulli($\frac{1}{2}$) bit plus normal noise with standard deviation 0.2, in world 0 and shifted by 1 in world 1; each has two peaks. Middle: the logarithm of their likelihood ratio still increases with the observation. Right: the log-likelihood ratio of the chapter's noisy count of 100 records for three noise levels, increasing for each and crossing 0 at 50.5.

F.7. Code

The script `code/appendix-derivations/apx_tails.py` computes every number on this page and draws its figures. For each derivation it checks the inequalities numerically, compares each bound with the exact value, and recomputes the numbers the chapters quote. Run it with

```
uv run python code/appendix-derivations/apx_tails.py
```

Table F.7 lists the main numbers it prints.

Table F.7.: Numbers printed by the script for this appendix.

Section	Quantity	Value
Bernstein	Exact tail, mean of 1,000 Bernoulli(0.01) at $t = 0.01$	3.29×10^{-3}
Bernstein	Bernstein, optimized Chernoff, Hoeffding bounds there	2.26×10^{-2}, 2.00×10^{-2}, 0.819
Bernstein	Width at $\beta = 0.05$: exact root, chapter formula, Hoeffding	0.00875, 0.00968, 0.0387
Cramér	Exact exponent at $n = 100, 1{,}000, 10{,}000$; limit $\mathrm{kl}(0.2 \Vert 0.1)$	0.06225, 0.04728, 0.04481; 0.04440
Chernoff–Stein	Spam example: exponent at $n = 200$; limit KL	0.3730; 0.4564
Clopper–Pearson	$u(0)$, $u(1)$, $u(10)$ at $n = 1{,}000$, level 0.025	0.003682, 0.005559, 0.018313
Clopper–Pearson	Smallest coverage at $n = 100$: Clopper–Pearson, Wald	0.9504, 0.0488

Section	Quantity	Value
Rényi to KL	Die: $D_{0.99}$, KL, $D_{1.01}$	0.2907, 0.2939, 0.2971
Rényi entropy	PIN: $H_{0.999}$, H, $H_{1.001}$ (bits)	12.4310, 12.4278, 12.4246
Perspective	Violations of joint convexity, subadditivity, quasi-convexity	0, 0, 0
Perspective	$F(Q, P)$ and its bound, subsampled Gaussian	1.0115, 1.1718
Noisy count	Crossing point and advantage, $m = 100$, $\sigma = 1$	50.5000, 0.0779

Each bound sits on the right side of its exact value, and the limits are approached at the slow rates the proofs predict. The Wald coverage of 0.0488 occurs at the extreme rates of the grid, 0.0005 and 0.9995. At $p = 0.0005$, for example, the count is 0 in 95% of runs, and then the Wald interval is the single point $[0, 0]$, which misses p. The counts of violations are the numerical form of the convexity proofs.

F.8. References

- Boucheron, Lugosi & Massart, *Concentration Inequalities: A Nonasymptotic Theory of Independence* (2013). Chapter 2 derives Bernstein's inequality and its relatives by the Chernoff method.
- Dembo & Zeitouni, *Large Deviations Techniques and Applications* (2nd ed., 1998). Cramér's theorem in full generality, with the tilting argument used here.
- Cover & Thomas, *Elements of Information Theory* (2nd ed., 2006). Chapter 11 proves the Chernoff–Stein lemma with typical sets, and states the binary KL bounds.
- Clopper & Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial", *Biometrika* (1934). The original interval.
- Brown, Cai & DasGupta, "Interval estimation for a binomial proportion", *Statistical Science* (2001). The coverage of the Wald interval and its alternatives, with sawtooth plots like Figure F.6.
- van Erven & Harremoës, "Rényi divergence and Kullback–Leibler divergence", *IEEE Transactions on Information Theory* (2014). The limits of Rényi divergence in every order, and its convexity properties, in full generality.
- Boyd & Vandenberghe, *Convex Optimization* (2004). Section 3.2.6 introduces the perspective of a function and proves that it keeps convexity.
- Karlin, *Total Positivity* (1968). The theory behind step 5 of the noisy-count proof: kernels with the inequality of step 5 and the sums they preserve.

G. Optimization and estimation

This appendix proves seven results that the chapters on optimization, generalization and attacks use without deriving them in full. The first five are about gradient methods on convex functions. Convexity can be recognized in three ways, and on a strongly convex function gradient descent (GD) closes the gap to the minimum by a constant factor per step. On a convex smooth function, a small enough gradient step never pushes two points apart. On functions whose slope is bounded (Lipschitz functions), no gradient method can beat the rate of stochastic gradient descent (SGD), one over the square root of the number of steps. And when SGD trains a model, the best time to stop balances that rate against the stability that the non-expanding step gives. The other results treat a trained model as a random object, random because its training set is. Here the appendix finds the limit of the error of the 1-nearest-neighbour rule, and shows that the parameters fitted by logistic regression are approximately Gaussian, with a formula for how much one record moves them. It ends by showing when the best attack on the membership of a target record looks at the parameters only through that record's loss. Table G.1 lists where each result is stated or used.

Table G.1.: The derivations of this appendix and the chapter sections that state or use them.

Derivation	Where the book states or uses it
Three tests for convexity; linear convergence of GD	Section 6.1 and the GD section of Optimization
Co-coercivity of the gradient	The proof of Lemma 7.2; Section 35.3
Early stopping balances optimization error against stability	The SGD section of Algorithmic stability and generalization
Lower bound $RL/\sqrt{T}$ for convex Lipschitz optimization	The section on SGD for convex Lipschitz objectives in Optimization
Limit of the 1-nearest-neighbour error	Proposition 7.2
Fitted parameters are approximately Gaussian; influence of one record	Section 12.3, Section 10.3, Section 10.5, Section 11.3
The best membership attack depends on the loss	Section 9.2

G.1. Convexity three ways, and the linear rate of gradient descent

G.1.1. What the chapter uses

Optimization defines a convex function by its chords, saying that the straight segment between two points of the graph never dips below it. It then works with its tangents, Equation 6.2, which say

that the graph lies above each of its tangent planes. It proves that chords give tangents (Lemma 6.1) and points here for the other direction. In Section 6.1 it also states, citing Nesterov's *Lectures on Convex Optimization*, §2.1, that convexity and strong convexity can be read off the Hessian $\nabla^2 f$, the matrix of second derivatives, and that a strongly monotone gradient makes a function strongly convex. Strong monotonicity, Equation 6.6, says that $\langle \nabla f(\theta) - \nabla f(\theta'), \theta - \theta' \rangle \geq \lambda \|\theta - \theta'\|^2$: moving from θ' to θ raises the slope along the direction of travel by at least λ per unit of distance. Its logistic-regression example uses the Hessian test to conclude that the loss is convex. Finally, it proves the linear rate Equation 6.15, a geometric shrinking of the distance to the minimizer, only for a quadratic, while its interpretation says that GD converges linearly on every strongly convex smooth function.

Write f for a differentiable function of the parameters $\theta \in \mathbb{R}^d$, and $\lambda \geq 0$ for a level of curvature. The four claims proved here are these. First, tangents give chords. Suppose f lies above all its tangents, $f(\theta') \geq f(\theta) + \langle \nabla f(\theta), \theta' - \theta \rangle$ for all points θ and θ'. Then, for every mixing weight $\omega \in [0, 1]$,

$$f(\omega\theta' + (1-\omega)\theta) \leq \omega f(\theta') + (1-\omega) f(\theta). \tag{G.1}$$

Second, a Hessian with every eigenvalue at least λ makes f lie above each tangent by a bowl of curvature λ. In symbols, with I the identity matrix, if $\nabla^2 f(\theta) \succeq \lambda I$ everywhere, then

$$f(\theta') \geq f(\theta) + \langle \nabla f(\theta), \theta' - \theta \rangle + \frac{\lambda}{2} \|\theta' - \theta\|^2. \tag{G.2}$$

This inequality is what **λ-strongly convex** means, and with $\lambda = 0$ it is convexity. The Hessian test also works backwards for $\lambda = 0$: a convex twice-differentiable f has $\nabla^2 f \succeq 0$. Third, a strongly monotone gradient gives Equation G.2 without any second derivatives.

The fourth claim is about GD on the training objective. Take an empirical risk $\mathcal{L}(\theta; D)$, written $\mathcal{L}(\theta)$ here because the dataset D stays fixed. Let it be λ-strongly convex, with minimum value $\mathcal{L}^*$, and let it also be β-smooth, so that no curvature exceeds the number β, and write $\kappa = \beta/\lambda$ for its condition number. Then GD with the learning rate $\eta = 1/\beta$ shrinks the gap to the minimum by the factor $1 - 1/\kappa$ at every step, where θ_t is the point reached after t steps:

$$\mathcal{L}(\theta_t) - \mathcal{L}^* \leq \left(1 - \frac{1}{\kappa}\right)^t (\mathcal{L}(\theta_0) - \mathcal{L}^*). \tag{G.3}$$

And when $\mathcal{L}$ is twice differentiable with $\lambda I \preceq \nabla^2 \mathcal{L} \preceq \beta I$, the chapter's distance bound Equation 6.15 holds exactly as stated for the quadratic.

G.1.2. The idea

To get a chord from tangents, look at the mixed point $z = \omega\theta' + (1-\omega)\theta$ and draw the tangent there. The graph lies above this tangent at both ends, θ and θ', so it still lies above it after you average the two endpoint values with the weights $1-\omega$ and ω. The tangent is a straight line, so its weighted average over the two ends is its value at the weighted average of the ends, which is z itself. So the average of the function values is at least the tangent's value at z, which is $f(z)$. Take $f(\theta) = \theta^2$, $\theta = 0$, $\theta' = 2$ and $\omega = \frac{1}{2}$, so $z = 1$. The tangent at 1 is $1 + 2(\theta - 1)$. At the two ends it gives -1 and 3, below the function values 0 and 4. The two tangent values average to $1 = f(z)$, and the function values average to 2, which is at least 1.

The Hessian test works one line at a time. Restricted to the line from θ to θ', the function is a function of one variable whose second derivative is at least λ times the squared length of the step. Its slope therefore grows at least linearly along the line, and integrating twice gives the bowl.

The linear rate combines two inequalities. The chapter shows that a GD step with the learning rate $\eta = 1/\beta$ (the learning rate is also called the step size) lowers $\mathcal{L}$ by at least $\|\nabla\mathcal{L}\|^2/(2\beta)$. Strong convexity adds that the gradient cannot be small while the gap is large: the gap is at most $\|\nabla\mathcal{L}\|^2/(2\lambda)$. So each step removes at least the fraction $\lambda/\beta = 1/\kappa$ of the gap that remains.

G.1.3. Derivation

1. **Tangents give chords.** Let $z = \omega\theta' + (1-\omega)\theta$. The tangent inequality at z, applied towards θ and towards θ', gives $f(\theta) \geq f(z) + \langle \nabla f(z), \theta - z\rangle$ and $f(\theta') \geq f(z) + \langle \nabla f(z), \theta' - z\rangle$. Multiply the first by $1-\omega$ and the second by ω, and add. The inner products combine into $\langle \nabla f(z), (1-\omega)\theta + \omega\theta' - z\rangle$, which is zero because the vector on the right is $z - z$. What is left is $(1-\omega)f(\theta) + \omega f(\theta') \geq f(z)$, the chord inequality. Figure G.1 (left) draws the two gaps and their average.
2. **A curvature bound gives the bowl.** Fix θ and θ', and let $u = \theta' - \theta$ be the step between them. Walking a fraction $s \in [0,1]$ of that step gives the one-variable function $g(s) = f(\theta + su)$, the values of f along the segment. By the chain rule, $g'(s) = \langle \nabla f(\theta + su), u\rangle$ and $g''(s) = u^\top \nabla^2 f(\theta + su)\, u$. A symmetric matrix A satisfies $A \succeq \lambda I$ exactly when $u^\top A u \geq \lambda \|u\|^2$ for every u, so the curvature bound says $g''(s) \geq \lambda\|u\|^2$. Now integrate twice. By the fundamental theorem of calculus, the slope at s is the slope at 0 plus the integral of g'' from 0 to s, so $g'(s) \geq g'(0) + \lambda s\|u\|^2$. Integrating again over $s \in [0,1]$ gives $g(1) \geq g(0) + g'(0) + \frac{\lambda}{2}\|u\|^2$, which is Equation G.2 written in terms of f.
3. **And back: convexity gives a Hessian with no negative eigenvalue.** Step 5 of the proof of Proposition 6.1, with $\lambda = 0$, shows that a convex f has a monotone gradient, $\langle \nabla f(\theta + su) - \nabla f(\theta), su\rangle \geq 0$. In terms of g this says $s\,(g'(s) - g'(0)) \geq 0$. Divide by s^2 and let $s \to 0$: the ratio $(g'(s) - g'(0))/s$ tends to $g''(0) = u^\top \nabla^2 f(\theta) u$, so that is at least 0 for every direction u.
4. **Strong monotonicity gives the bowl.** Keep the same g. Strong monotonicity Equation 6.6, applied to the points $\theta + su$ and θ, says $\langle \nabla f(\theta + su) - \nabla f(\theta), su\rangle \geq \lambda s^2 \|u\|^2$. Dividing by $s > 0$ gives $g'(s) - g'(0) \geq \lambda s\|u\|^2$, which is the inequality of step 2 obtained without second derivatives. Integrating it over $s \in [0,1]$ again gives Equation G.2.
5. **A small gradient means a small gap.** Apply Equation G.2 at a point θ towards every other point θ'. The right side is a parabola in θ', and completing the square shows that its lowest value is $\mathcal{L}(\theta) - \|\nabla\mathcal{L}(\theta)\|^2/(2\lambda)$, reached at $\theta' = \theta - \nabla\mathcal{L}(\theta)/\lambda$. Every value of $\mathcal{L}$ is above the right side, and so above that lowest value. In particular $\mathcal{L}^* \geq \mathcal{L}(\theta) - \|\nabla\mathcal{L}(\theta)\|^2/(2\lambda)$, which rearranges to the **Polyak–Łojasiewicz inequality**:

$$\mathcal{L}(\theta) - \mathcal{L}^* \leq \frac{\|\nabla\mathcal{L}(\theta)\|^2}{2\lambda}. \tag{G.4}$$

6. **Each step removes a fixed fraction of the gap.** The chapter derives from the descent lemma Equation 6.5 that one GD step with the learning rate $\eta = 1/\beta$ gives $\mathcal{L}(\theta_{t+1}) \leq \mathcal{L}(\theta_t) - \|\nabla\mathcal{L}(\theta_t)\|^2/(2\beta)$. Subtract $\mathcal{L}^*$ from both sides. By Equation G.4, $\|\nabla\mathcal{L}(\theta_t)\|^2 \geq 2\lambda(\mathcal{L}(\theta_t) - \mathcal{L}^*)$, so the subtracted term is at least $\frac{\lambda}{\beta}(\mathcal{L}(\theta_t) - \mathcal{L}^*)$. Hence $\mathcal{L}(\theta_{t+1}) - \mathcal{L}^* \leq (1 - 1/\kappa)(\mathcal{L}(\theta_t) - \mathcal{L}^*)$, and repeating this t times gives Equation G.3.

7. **The distance bound, when second derivatives exist.** Write θ^* for the minimizer. It has zero gradient, so the GD step can be written as $\theta_{t+1} - \theta^* = (\theta_t - \theta^*) - \frac{1}{\beta}(\nabla\mathcal{L}(\theta_t) - \nabla\mathcal{L}(\theta^*))$. Step 4 of the proof of Proposition 6.1 writes the gradient difference as $\bar{H}_t(\theta_t - \theta^*)$, where $\bar{H}_t = \int_0^1 \nabla^2\mathcal{L}(\theta^* + s(\theta_t - \theta^*))\,ds$ is the average Hessian along the segment. For a unit vector u, $u^\top \bar{H}_t u$ is an average of numbers $u^\top\nabla^2\mathcal{L}\,u$ that all lie in $[\lambda, \beta]$, so it lies there too. The smallest and largest values of $u^\top A u$ over unit vectors are the extreme eigenvalues of a symmetric A, so every eigenvalue of $\bar{H}_t$ is in $[\lambda, \beta]$. Substituting $\bar{H}_t(\theta_t - \theta^*)$ for the gradient difference in the GD step gives $\theta_{t+1} - \theta^* = (I - \bar{H}_t/\beta)(\theta_t - \theta^*)$, a matrix with eigenvalues between 0 and $1 - 1/\kappa$ applied to the error. This is the quadratic's step 1 with the fixed H replaced by $\bar{H}_t$, and its steps 2 and 3 then go through unchanged and give Equation 6.15.

Step 7 also gives a faster rate for the gap. The gradient vanishes at θ^*, so the descent lemma at θ^* gives $\mathcal{L}(\theta_t) - \mathcal{L}^* \le \frac{\beta}{2}\|\theta_t - \theta^*\|^2$, and with Equation 6.15 the gap falls like $(1 - 1/\kappa)^{2t}$. That exponent is twice the one in Equation G.3. The slower rate is the price of an argument that uses only first derivatives.

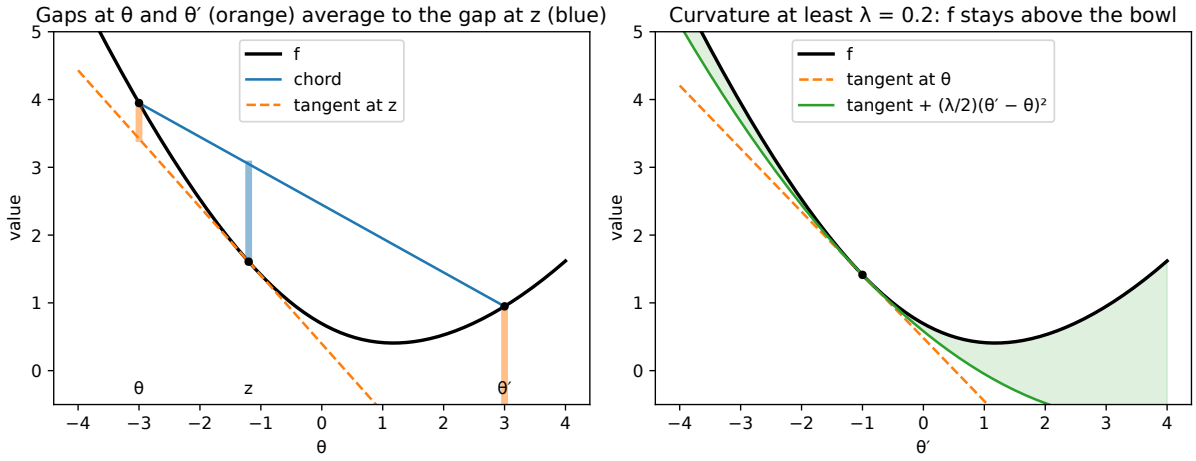


Figure G.1.: Left: the chord of $f(\theta) = \ln(1 + e^{-\theta}) + 0.1\theta^2$ between $\theta = -3$ and $\theta' = 3$, the tangent at the mixed point z, and the two gaps between f and the tangent at the ends (orange), whose weighted average is the gap between the chord and $f(z)$ (blue). Right: the same function with curvature at least $\lambda = 0.2$, which stays above its tangent plus the bowl $\frac{\lambda}{2}(\theta' - \theta)^2$.

G.1.4. Checking it

The script uses the logistic data of the optimization chapter: $n = 1{,}000$ records with x uniform on the unit sphere in $\mathbb{R}^5$ and labels from a logistic model. It adds a ridge penalty $\frac{\lambda}{2}\|\theta\|^2$ with $\lambda = 0.01$. Each x has length 1, so the logistic curvature is at most $\frac{1}{4}$, and the ridge adds λ. So $\beta = 0.26$ and $\kappa = 26$. Over 2,000 random pairs of points in the ball of radius 5, the smallest ratio of the tangent gap $\mathcal{L}(\theta') - \mathcal{L}(\theta) - \langle\nabla\mathcal{L}(\theta), \theta' - \theta\rangle$ to $\frac{1}{2}\|\theta' - \theta\|^2$ is 0.0372, above $\lambda = 0.01$ as Equation G.2 requires.

GD from $\theta_0 = 0$ satisfies both bounds with room to spare. Over the first 40 steps, the gap shrinks by a factor of at most 0.7131 per step and the distance by at most 0.8445, both below

$1 - 1/\kappa = 0.9615$. After 25 steps the gap is 1.22×10^{-5}, against 5.30×10^{-2} from Equation G.3. After 50 steps it is 2.59×10^{-9}, against 1.99×10^{-2}.

The bounds are loose here because they know only the curvature $\lambda = 0.01$ of the penalty, while the data add curvature of their own. At the minimizer, the smallest eigenvalue of $\nabla^2 \mathcal{L}$ is 0.0404. The rate of step 7 with that curvature in place of λ is $(1 - 0.0404/0.26)^2 = 0.713$ per step, and the measured rate between steps 20 and 40 is 0.712. So the form of the rate is exact, and only the curvature it is given is pessimistic. Figure G.2 shows the run.

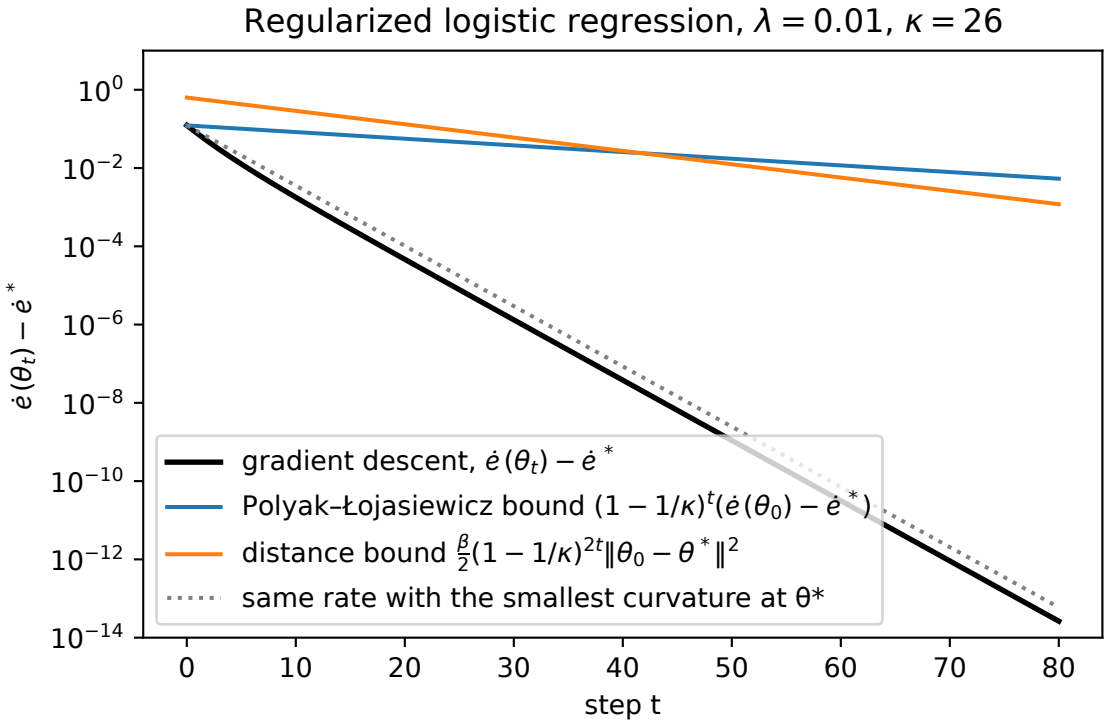


Figure G.2.: The gap $\mathcal{L}(\theta_t) - \mathcal{L}^*$ of GD on ridge-regularized logistic regression ($\lambda = 0.01$, $\kappa = 26$), with the Polyak–Łojasiewicz bound Equation G.3, the bound from the distance Equation 6.15, and the same rate computed with the smallest curvature at the minimizer, which tracks the run.

G.1.5. What it does not cover

The distance bound Equation 6.15 was proved here only for twice-differentiable $\mathcal{L}$. It holds without second derivatives as well. That proof applies the co-coercivity of the next section to $\mathcal{L} - \frac{\lambda}{2}\|\theta\|^2$, which is convex and $(\beta - \lambda)$-smooth (Nesterov 2018, Theorem 2.1.12), and it is longer than the short argument of step 7. The gap bound Equation G.3 needs only the descent lemma and Equation G.4, so it holds for any function that satisfies the Polyak–Łojasiewicz inequality, even one that is not convex (Karimi, Nutini & Schmidt 2016).

G.2. The gradient of a convex smooth function is co-coercive

G.2.1. What the chapter uses

Step 1 of the proof of Lemma 7.2 states that the gradient of a convex β-smooth function f is **co-coercive**. In words, two gradients that differ by a lot must come from points that are far apart along the direction of that difference. For all u and v,

$$\langle \nabla f(u) - \nabla f(v),\, u - v \rangle \geq \frac{1}{\beta} \|\nabla f(u) - \nabla f(v)\|^2. \tag{G.5}$$

The chapter cites Nesterov's Theorem 2.1.5 and points here. The lemma then derives from Equation G.5 that a gradient step $\psi(\theta) = \theta - \eta \nabla f(\theta)$ with $0 \leq \eta \leq 2/\beta$ is non-expansive, $\|\psi(u) - \psi(v)\| \leq \|u - v\|$. That fact drives the stability bound Equation 7.21 for SGD, and Section 35.3 uses the same contraction in amplification by iteration.

G.2.2. The idea

Subtract from f its tangent at v. The result, φ, is still convex and β-smooth, and its gradient is zero at v, so v is its lowest point. Now stand at u and take a gradient step on φ. By the descent lemma, the step lowers φ by at least $\|\nabla \varphi(u)\|^2/(2\beta)$, and no step can go below the lowest value $\varphi(v)$. So $\varphi(u)$ must sit at least $\|\nabla \varphi(u)\|^2/(2\beta)$ above $\varphi(v)$. In words, a convex smooth function cannot have a steep slope close to its minimum. This gives a tangent inequality with an extra term. Writing it once from u and once from v, and adding the two, gives Equation G.5.

A one-dimensional case makes the inequality concrete. Take the logistic loss

$$f(\theta) = \ln(1 + e^{-\theta}), \tag{G.6}$$

whose slope is $f'(\theta) = -\sigma(-\theta)$, where $\sigma(s) = 1/(1 + e^{-s})$ is the logistic function, and whose curvature is at most $\beta = \frac{1}{4}$. At the two points $u = 0$ and $v = 2$ the slopes are -0.5000 and -0.1192. The left side of Equation G.5 is $(-0.5000 + 0.1192)(0 - 2) = 0.7616$, and the right side is $0.3808^2/0.25 = 0.5800$, smaller as it should be. For the quadratic $f(\theta) = \frac{\beta}{2}\theta^2$ the slope difference is $\beta(u - v)$, so both sides equal $\beta(u - v)^2$, and the inequality is tight.

G.2.3. Derivation

1. **Shift the minimum to v.** Define $\varphi(\theta) = f(\theta) - \langle \nabla f(v), \theta \rangle$. Subtracting a linear function changes neither the curvature nor the differences between gradients, so φ is convex and β-smooth. Its gradient is $\nabla \varphi(\theta) = \nabla f(\theta) - \nabla f(v)$, which is zero at $\theta = v$. The tangent inequality Equation 6.2 for φ at v then reads $\varphi(\theta) \geq \varphi(v)$ for every θ, so v minimizes φ.
2. **A gradient step from u cannot go below the minimum.** Apply the descent lemma Equation 6.5 to φ at u, with the step $\theta' = u - \nabla \varphi(u)/\beta$. The computation is the chapter's GD step with $\beta = \beta$, and it gives $\varphi(\theta') \leq \varphi(u) - \|\nabla \varphi(u)\|^2/(2\beta)$. Since $\varphi(v) \leq \varphi(\theta')$ by step 1, we get $\varphi(v) \leq \varphi(u) - \|\nabla \varphi(u)\|^2/(2\beta)$.

3. **Return to f.** Write $\Delta = \nabla f(u) - \nabla f(v)$, which is $\nabla\varphi(u)$. The difference $\varphi(u) - \varphi(v)$ is $f(u) - f(v) - \langle \nabla f(v), u - v\rangle$, so step 2 becomes a tangent inequality with an extra term:

$$f(u) \geq f(v) + \langle \nabla f(v), u - v\rangle + \frac{1}{2\beta}\|\Delta\|^2. \tag{G.7}$$

4. **Swap and add.** The same inequality with u and v exchanged is $f(v) \geq f(u) + \langle \nabla f(u), v - u\rangle + \frac{1}{2\beta}\|\Delta\|^2$, because exchanging the points only flips the sign of Δ. Adding the two, the function values cancel and the inner products combine into $-\langle \Delta, u - v\rangle$, which leaves $0 \geq -\langle \Delta, u - v\rangle + \frac{1}{\beta}\|\Delta\|^2$. That is Equation G.5.
5. **The step is non-expansive.** This is the chapter's own argument, repeated so that the page stands alone. Expand $\psi(u) - \psi(v) = (u - v) - \eta\Delta$ as a square and replace $\langle \Delta, u - v\rangle$ by the smaller $\|\Delta\|^2/\beta$:

$$\|\psi(u) - \psi(v)\|^2 = \|u - v\|^2 - 2\eta\langle \Delta, u - v\rangle + \eta^2\|\Delta\|^2 \leq \|u - v\|^2 - \eta\Big(\frac{2}{\beta} - \eta\Big)\|\Delta\|^2. \tag{G.8}$$

   For $0 \leq \eta \leq 2/\beta$ the subtracted term is not negative, so the distance does not grow.

G.2.4. Checking it

The script draws 500 pairs (u, v) from a normal distribution with standard deviation 4 and evaluates both sides of Equation G.5 for the logistic loss. The smallest ratio of left to right side is 1.0006, so the inequality holds and is nearly tight. The tight pairs lie near 0, where the curvature of the logistic loss is close to $\frac{1}{4}$ and the loss looks like the quadratic of the example. For the five-dimensional logistic risk of the previous section, the smallest ratio over 2,000 pairs is 5.3704. That risk curves much less than $\frac{1}{4}$ in most directions, so the bound has slack there.

The non-expansive step behaves as step 5 says. Over the 500 pairs, the largest ratio $|\psi(u) - \psi(v)|/|u - v|$ is 0.9993 at $\eta = 4$ and 0.9988 at $\eta = 8 = 2/\beta$. At $\eta = 10$ it is 1.4985, so the step pushes some pairs apart. Figure G.3 shows both checks.

G.2.5. What it does not cover

Both convexity and smoothness are needed. Without convexity, step 1 fails, because a zero gradient no longer marks a minimum. A gradient step can then expand distances by up to the factor $1 + \eta\beta$, as on the concave function $-\frac{\beta}{2}\theta^2$, whose step multiplies every distance by exactly $1 + \eta\beta$. Without smoothness there is no descent lemma, and the gradient of a function with a kink, such as $|\theta|$, jumps by 2 between points arbitrarily close together, so no constant β can work.

G.3. When to stop SGD: balancing optimization against stability

G.3.1. What the chapter uses

Theorem 7.4 in Algorithmic stability and generalization bounds the stability of SGD. Two runs on training sets that differ in one record drift apart by at most $2L/n$ times the sum of the learning

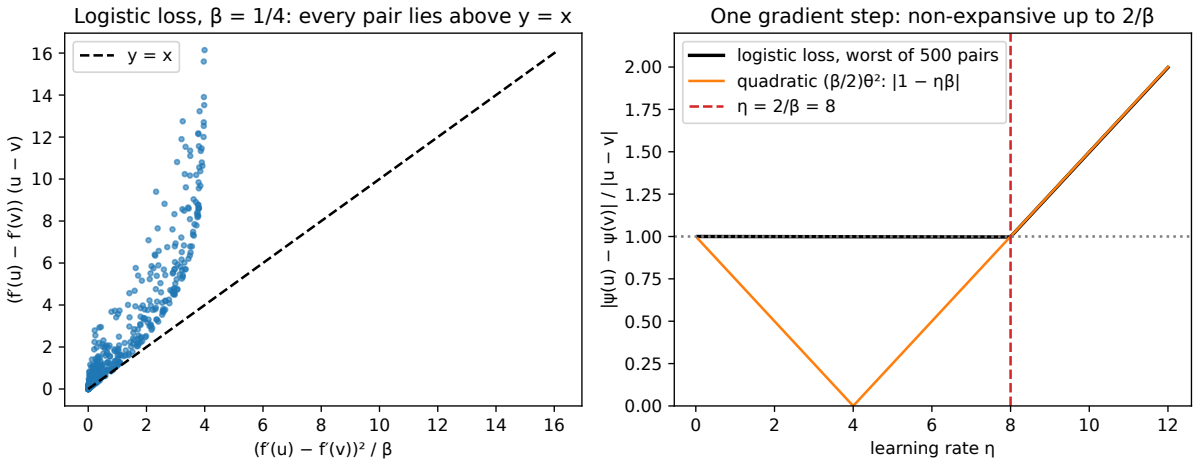


Figure G.3.: Left: for 500 random pairs, $(f'(u) - f'(v))(u - v)$ against $(f'(u) - f'(v))^2/\beta$ for the logistic loss, $\beta = \frac{1}{4}$. Every point lies above the line $y = x$. Right: the largest ratio of distances after and before one gradient step, over the same pairs, against the learning rate η, with the quadratic's factor $|1 - \eta\beta|$. Both cross 1 at $\eta = 2/\beta = 8$.

rates, so with a constant step η and T steps the uniform stability is

$$\gamma \leq \frac{2L^2\eta T}{n}, \tag{G.9}$$

where n is the number of training records and L bounds the slope of the loss. The chapter concludes that training longer lowers the training loss but raises γ, which makes a case for stopping early, and points here for the balance. This section makes that precise. For a loss that is convex, smooth and L-Lipschitz, a constant step small enough for that theorem, and SGD started at $\theta_0 = 0$, it shows that the expected excess test loss of the averaged weights $\bar{\theta}$ over the best weights θ^* is at most

$$\frac{\|\theta^*\|^2}{2\eta T} + \frac{2L^2\eta T}{n} + \frac{\eta L^2}{2}. \tag{G.10}$$

The first term falls with training time and the second rises with it. Their sum is smallest when $\eta T = \|\theta^*\|\sqrt{n}/(2L)$, and there the excess test loss is at most $2L\|\theta^*\|/\sqrt{n}$, plus the term $\eta L^2/2$.

G.3.2. The idea

The test loss of the trained weights is worse than that of the best weights for two separate reasons. SGD has not finished minimizing the training loss, which is the optimization error, and the training loss of the trained weights is an optimistic estimate of their test loss, which is the generalization gap. Each step of training shrinks the first and can grow the second. So there is a best training time, and finding it is a matter of adding the two bounds and minimizing the sum.

The two bounds come from two chapters. Optimization bounds the optimization error by a term that falls like $1/(\eta T)$, the cost of travelling from the starting point to θ^* with steps of size η. The stability chapter bounds the expected gap by γ, which grows like ηT. A sum $a/u + bu$ of a falling and a rising term is smallest where the two terms are equal, which is the whole calculation.

G.3.3. Derivation

The plan is to split the excess test loss into three pieces, bound each piece by a result the chapters prove, and then choose the training time that minimizes the sum of the bounds. Write $\mathcal{L}_{\mathcal{D}}(\theta)$ for the test loss of weights θ, the average loss on a new record, and $\mathcal{L}(\theta; D)$ for the training loss, the average loss on the training set D. Let θ^* be weights that minimize the test loss. They are fixed in advance, so they do not depend on D.

1. **Split the excess loss.** Add and subtract the training loss of $\bar{\theta}$ and of θ^*:

$$\mathcal{L}_{\mathcal{D}}(\bar{\theta}) - \mathcal{L}_{\mathcal{D}}(\theta^*) = \big[\mathcal{L}_{\mathcal{D}}(\bar{\theta}) - \mathcal{L}(\bar{\theta}; D)\big] + \big[\mathcal{L}(\bar{\theta}; D) - \mathcal{L}(\theta^*; D)\big] + \big[\mathcal{L}(\theta^*; D) - \mathcal{L}_{\mathcal{D}}(\theta^*)\big]. \tag{G.11}$$

   The first bracket is the generalization gap of $\bar{\theta}$, the second is its optimization error on the training set, and the third compares a fixed model's training loss with its test loss.

2. **The third piece has mean zero.** The weights θ^* do not depend on D, so by Proposition 7.1 their training loss is an unbiased estimate of their test loss, and the third bracket averages to zero over D.

3. **The averaged weights are as stable as the last ones.** Let $\theta_0, \theta_1, \ldots$ be the iterates of SGD on D, and θ'_t those of the coupled run on the neighbouring set $D^{(i)}$, as in the proof of Theorem 7.4. That proof shows that the distance $r_t = \|\theta_t - \theta'_t\|$ has $\mathbb{E}\, r_t \le 2L\eta t/n$, which is at most $2L\eta T/n$ for every $t \le T$. The averaged weights are $\bar{\theta} = \frac{1}{T}\sum_{t=0}^{T-1} \theta_t$. The distance between two averages is at most the average of the distances, by the triangle inequality, so $\mathbb{E}\|\bar{\theta} - \bar{\theta}'\|$ is also at most $2L\eta T/n$. The loss is L-Lipschitz, so the averaged weights are uniformly stable with $\gamma \le 2L^2\eta T/n$, as in step 5 of that proof.

4. **The first piece.** By Equation 7.7, a γ-uniformly stable algorithm has expected gap at most γ. With step 3, the first bracket averages to at most $2L^2\eta T/n$.

5. **The second piece.** Theorem 6.2 bounds the optimization error of SGD with constant step η, run on a convex objective whose random gradients are unbiased and have mean squared length at most G^2, from a start at distance at most R from θ^*. Here the objective is the training loss $\mathcal{L}(\cdot; D)$, and the set is all of $\mathbb{R}^d$, so there is no projection. A step that picks a record uniformly at random has a gradient whose average is the gradient of the training loss, so it is unbiased, and an L-Lipschitz loss gives $G = L$. Starting at $\theta_0 = 0$ makes that distance $R = \|\theta^*\|$. The theorem compares with a minimizer of the objective, but its proof never uses that θ^* minimizes anything: the convexity step holds for any fixed point. So, averaged over the steps' random choices,

$$\mathbb{E}\big[\mathcal{L}(\bar{\theta}; D) - \mathcal{L}(\theta^*; D)\big] \le \frac{\|\theta^*\|^2}{2\eta T} + \frac{\eta L^2}{2}. \tag{G.12}$$

6. **Add the three pieces.** Taking expectations in step 1 and using steps 2, 4 and 5 gives Equation G.10.

7. **Choose the training time.** For a fixed step η, training time enters Equation G.10 only through $u = \eta T$, the total distance the learning rates allow. The part that depends on it is

$$h(u) = \frac{\|\theta^*\|^2}{2u} + \frac{2L^2 u}{n}. \tag{G.13}$$

Its derivative $-\frac{\|\theta^*\|^2}{2u^2} + \frac{2L^2}{n}$ is negative for small u and positive for large u, and it is zero at

$$u^* = \frac{\|\theta^*\|\sqrt{n}}{2L}. \tag{G.14}$$

8. **The value at the balance.** Substituting u^*, each of the two terms of h equals $L\|\theta^*\|/\sqrt{n}$, so the excess test loss is at most $2L\|\theta^*\|/\sqrt{n}$, plus the term $\eta L^2/2$. Training much longer than u^* buys optimization accuracy that the growing stability term gives back, because past u^* the rising term of h grows faster than the falling term shrinks.

G.3.4. Checking it

The chapter's SGD experiment, Example 7.6, gives a concrete case. Its loss is the logistic loss on $n = 100$ records of the running example, with features on the unit sphere, so $L = 1$, and with step $\eta = 0.25$. In the running example a label is $+1$ with probability $\sigma(5x_1)$, where σ is the logistic function and x_1 the first feature. The logistic loss of weights θ is the negative log-likelihood of a model that says $+1$ with probability $\sigma(\langle\theta, x\rangle)$. At every x, the expected negative log-likelihood is smallest when the model's probability equals the true one, so the best weights are $\theta^* = (5, 0, \dots, 0)$, with $\|\theta^*\| = 5$.

Step 7 then gives $u^* = 5\sqrt{100}/2 = 25$. At $\eta = 0.25$ that is $T = 100$ steps, one pass over the 100 records, so the bound says to stop after one epoch. Its value there is $2 \cdot 5/10 = 1$, plus $\eta L^2/2 = 0.125$. Figure G.4 draws the two terms and their sum against u: they cross at $u^* = 25$, where each is 0.5, and the sum has its minimum there. In the experiment, though, the test loss keeps falling after one epoch, from 0.596 to 0.557 after five, and rises only after that (Table 7.4). So the bound stops too early, because its stability term is too pessimistic. After one epoch that term allows a gap of 0.5, while the measured gap is 0.036, and the measured gap grows far more slowly than linearly. The balance has the right form, but the true best time lies later than the bound says. The experiment measures the last iterate rather than the average, so the comparison is a check of the shape, not of the constant.

G.3.5. What it does not cover

The bound needs a convex loss, so it does not cover neural networks. Hardt, Recht & Singer (2016) also bound the stability of SGD on smooth non-convex losses, with steps that shrink like $1/t$. Their stability term then grows like a power of T below 1. But the optimization error of a non-convex loss has no bound of the kind step 5 uses, so there is nothing to balance it against. The bound also uses a constant step. With a step that shrinks over time, the sum of the learning rates replaces ηT in the stability term, and the balance is found the same way.

G.4. No gradient method beats $RL/\sqrt{T}$ on every convex Lipschitz function

G.4.1. What the chapter uses

The section on SGD for convex Lipschitz objectives in Optimization proves Equation 6.25, an error of at most $RL/\sqrt{T}$ after T steps. Here R is the distance from the start to the minimizer,

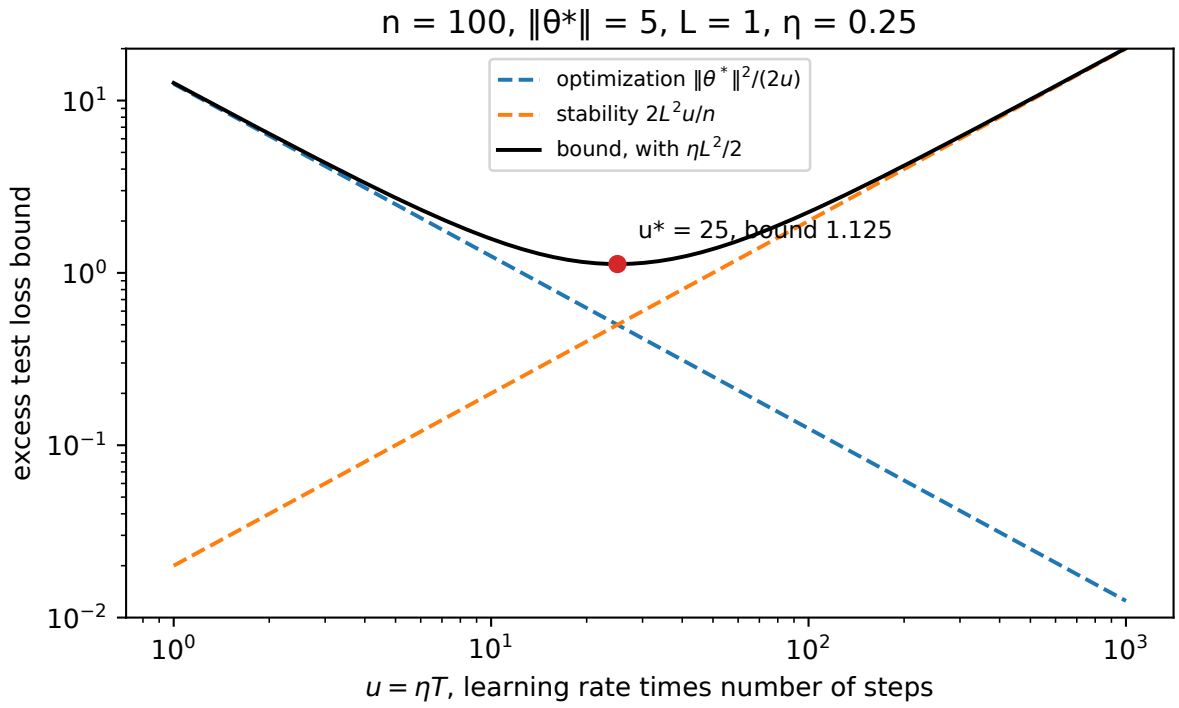


Figure G.4.: The bound Equation G.10 against $u = \eta T$, the learning rate times the number of steps, for $n = 100$, $\|\theta^*\| = 5$, $L = 1$ and $\eta = 0.25$, on log axes: the falling optimization term $\|\theta^*\|^2/(2u)$, the rising stability term $2L^2u/n$ (both dashed), and their sum plus $\eta L^2/2$ (solid), smallest at $u^* = 25$.

and L is the Lipschitz constant, the largest size of a gradient. Its interpretation then says that the rate cannot be improved: "there are convex L-Lipschitz functions on which every method that only queries gradients has error at least of order $RL/\sqrt{T}$ after T queries (Bubeck 2015, §3.5)."

This section proves that claim for the methods the book uses. A method is **span-restricted** if it starts at $\theta_1 = 0$ and every point it queries or outputs is a combination of the gradients it has received so far. Gradient descent and SGD, with a constant or a changing learning rate, with projection onto a ball centred at 0 (which only rescales a vector), and with averaged iterates, are all span-restricted. The claim is that in dimension $d \geq T + 1$ there is a convex function that is L-Lipschitz on the ball $\{\|\theta\| \leq R\}$, with its minimizer at distance R from the start, on which every span-restricted method, after T gradient queries, outputs a point θ with

$$f(\theta) - f^* \geq \frac{RL}{2(1 + \sqrt{T+1})}. \tag{G.15}$$

The right side is close to half of $RL/\sqrt{T}$ for large T, so the SGD guarantee is optimal up to a factor of about 2.

G.4.2. The idea

The hard function hides its minimum in many coordinates and reveals them one gradient at a time. It uses one more coordinate than the method has queries, $k = T + 1$. Its first term is a positive constant γ times the largest of the first k coordinates, and its second is a small bowl with a positive weight α. Both constants are chosen below:

$$f(\theta) = \gamma \max_{i \leq k} \theta_i + \frac{\alpha}{2}\|\theta\|^2. \tag{G.16}$$

Its lowest point spreads equal negative values over all k coordinates, which brings the largest coordinate down. But the gradient at a point whose later coordinates are all zero involves only one new coordinate, the first place where the largest value is reached. So after T gradients the method has touched at most T coordinates. The last coordinate is still zero, so the largest coordinate is at least zero, and f is at least zero. The minimum is negative, and its distance below zero is the bound.

Take $T = 2$, so $k = 3$, with $R = L = 1$. The constants are then $\gamma = 0.6340$ and $\alpha = 0.3660$, and the minimum is $f^* = -0.1830$. The first query is at $\theta_1 = 0$, where all three coordinates tie at 0, so the gradient points along the first coordinate. The method's next point has the form $(a, 0, 0)$. Whatever a is, the largest coordinate is first reached at the first or the second position, so the second gradient involves only the first two coordinates. Any output is then of the form $(a', b', 0)$, its third coordinate makes the largest coordinate at least 0, and $f \geq 0$. The error is at least 0.1830, which is $1/(2(1 + \sqrt{3}))$, the right side of Equation G.15.

G.4.3. Derivation

1. **The minimum.** The largest of k numbers is at least their average, so $f(\theta) \geq \sum_{i \leq k} \left(\frac{\gamma}{k}\theta_i + \frac{\alpha}{2}\theta_i^2\right) + \frac{\alpha}{2}\sum_{i>k}\theta_i^2$. Each term of the first sum is a parabola in θ_i whose lowest value is $-\gamma^2/(2\alpha k^2)$, at $\theta_i = -\gamma/(\alpha k)$. The terms of the second sum are at least 0. Adding up,

$f(\theta) \geq -\gamma^2/(2\alpha k)$. The point with $\theta_i = -\gamma/(\alpha k)$ for $i \leq k$ and 0 elsewhere reaches this value, because its first k coordinates are equal and so their largest equals their average. So

$$f^* = -\frac{\gamma^2}{2\alpha k}, \qquad \|\theta^*\| = \frac{\gamma}{\alpha\sqrt{k}}. \tag{G.17}$$

2. **The constants.** The max term changes by at most γ times the distance moved, since no coordinate changes by more than the distance. On the ball of radius R the quadratic term changes by at most αR times the distance, since $\frac{\alpha}{2}(\|\theta\|^2 - \|\theta'\|^2) = \frac{\alpha}{2}\langle\theta + \theta', \theta - \theta'\rangle$ and $\|\theta + \theta'\| \leq 2R$. So f is $(\gamma + \alpha R)$-Lipschitz on the ball. Asking for $\gamma + \alpha R = L$ and $\|\theta^*\| = R$, and solving the two equations, gives

$$\gamma = \frac{L\sqrt{k}}{1+\sqrt{k}}, \qquad \alpha = \frac{L}{R(1+\sqrt{k})}, \qquad -f^* = \frac{RL}{2(1+\sqrt{k})}. \tag{G.18}$$

3. **The gradients the method sees.** At a point θ, let i be the first index at which $\theta_1, \dots, \theta_k$ reach their largest value, and return $g = \gamma e_i + \alpha\theta$, where e_i is the i-th unit vector. This is a legitimate gradient for a function with a kink, a subgradient as in the chapter: its tangent stays below the graph. The reason is that $\max_j \theta'_j \geq \theta'_i = \max_j \theta_j + \langle e_i, \theta' - \theta\rangle$, while the quadratic term lies above its own tangent. Adding the two gives $f(\theta') \geq f(\theta) + \langle g, \theta' - \theta\rangle$.
4. **One new coordinate per query.** Write V_s for the vectors whose coordinates after the s-th are zero. The claim is that the s-th query point lies in V_{s-1} and its gradient in V_s. The first query is at $0 \in V_0$. Suppose the s-th query point θ_s is in V_{s-1}, with $s \leq k$. Its coordinates $s, \dots, k$ are zero, so its largest coordinate among the first k is at least 0. If that largest value is 0, coordinate s reaches it, so the first index that does is at most s. If it is positive, it is reached among the first $s-1$ coordinates. Either way $e_i \in V_s$, and $\alpha\theta_s \in V_{s-1}$, so $g_s \in V_s$. A span-restricted method then chooses θ_{s+1} in V_s.
5. **The error after T queries.** Any output lies in the span of $g_1, \dots, g_T$, which is inside V_T. Its coordinate $k = T + 1$ is zero, so its largest coordinate is at least 0 and $f(\theta) \geq \frac{\alpha}{2}\|\theta\|^2 \geq 0$. The error is therefore at least $-f^*$, which by step 2 with $k = T + 1$ is Equation G.15.

G.4.4. Checking it

The script runs the projected subgradient method of the chapter, with the learning rate $\eta = R/(L\sqrt{T})$ and the averaged iterate, against these gradients, with $R = L = 1$. Table G.2 shows that its error always lies between the two bounds.

Table G.2.: Error of the averaged iterate on the hard function, $R = L = 1$, against the lower and upper bounds.

T	Lower bound Equation G.15	Subgradient method	SGD guarantee $RL/\sqrt{T}$
10	0.1158	0.1285	0.3162
100	0.0452	0.0519	0.1000
1,000	0.0153	0.0178	0.0316
10,000	0.0050	0.0058	0.0100

No iterate ever has a negative value of f, as step 5 predicts. The method sits just above the lower bound, about 17% above it at $T = 10{,}000$ (a ratio of 1.17), so the lower bound is close to exact for this method. The upper bound is about twice the lower one, as the ratio $2(1 + \sqrt{T+1})/\sqrt{T}$ approaches 2. Figure G.5 shows the three on a log scale.

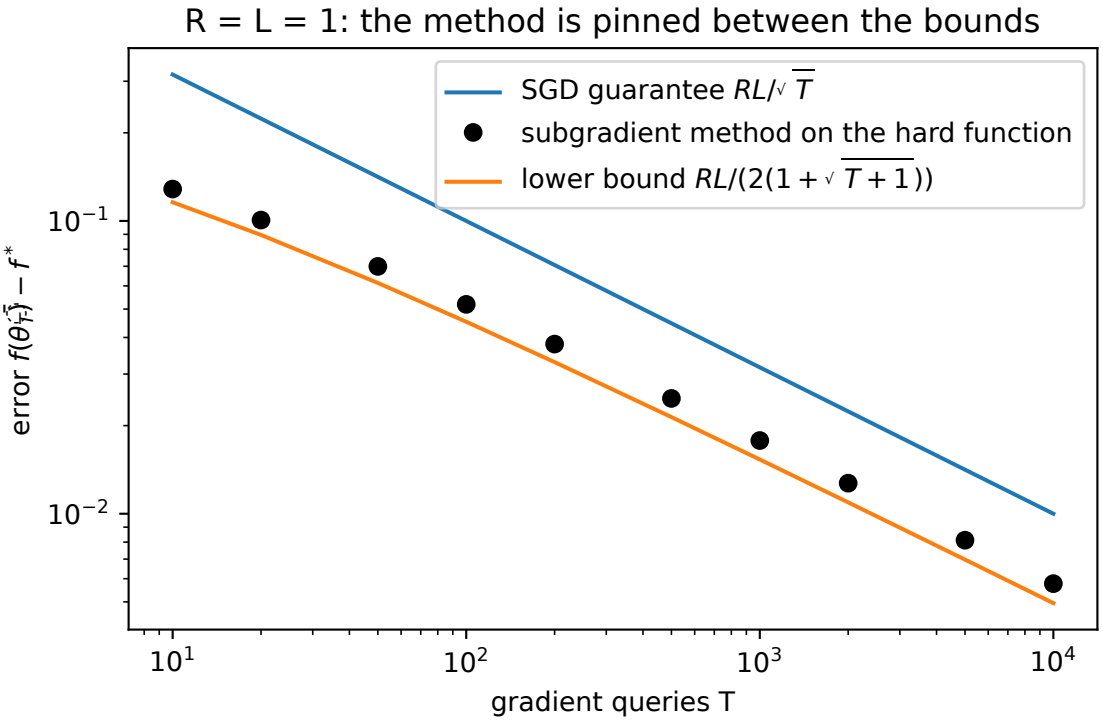


Figure G.5.: The lower bound $RL/(2(1 + \sqrt{T+1}))$, the SGD guarantee $RL/\sqrt{T}$, and the error of the projected subgradient method on the hard function, against the number of gradient queries T, with $R = L = 1$.

G.4.5. What it does not cover

The proof needs the method to stay in the span of its gradients. Nemirovski & Yudin (1983) remove that condition, and prove the same order of bound for every method that uses only gradients, by hiding the coordinates in a random rotation that the method cannot know in advance; their argument is longer and stays cited. The dimension must also be at least $T + 1$, because in low dimension, methods that cut away part of the space at each query, such as the ellipsoid method, have an error that shrinks like $e^{-cT/d}$ for a constant c (Bubeck 2015, ch. 2). Finally, the bound is for functions that are not smooth. On a smooth function GD with exact gradients does better than $1/\sqrt{T}$, and the $1/\sqrt{T}$ of SGD then comes from its gradient noise.

G.5. The 1-nearest-neighbour error tends to $\mathbb{E}[2p(1-p)]$

G.5.1. What the chapter uses

The nearest neighbour (NN) rule with one neighbour, 1-NN, predicts the label of the closest training point. Algorithmic stability and generalization states, citing Cover & Hart (1967), that its test error tends to $\mathbb{E}[2p(x)(1-p(x))]$ as the number n of training points grows. Here $p(x) = \Pr[y = 1 \mid x]$ is the probability that the label at x is $+1$. Step 3 of its proof (Proposition 7.2) says that for large n "the nearest neighbour of a test point x lies arbitrarily close to x" and cites this appendix for the precise argument. In its running example the limit is 0.338 and the Bayes error, the error of the best possible classifier, is 0.250.

The precise statement proved here is this. Let the training pairs (x_i, y_i) and the test pair (x, y) be independent draws from the same distribution. Assume that x lies in a closed and bounded set K, and that p is continuous on K. Ties between equally near neighbours may be broken by any rule that looks only at the points, not at their labels. Then the test error R_n of 1-NN, averaged over the training set and the test pair, satisfies

$$\lim_{n \to \infty} R_n = \mathbb{E}\big[2p(x)(1-p(x))\big]. \tag{G.19}$$

G.5.2. The idea

The argument has two parts. First, the nearest neighbour gets close. For a fixed test point, the chance that none of the n training points lands within a distance r of it is $(1-P)^n$, where P is the chance that one point lands there. As long as $P > 0$, this goes to zero. Second, the labels behave like two coins. Given where all the points are, the neighbour's label is $+1$ with probability $p(x')$, where x' is the neighbour, and the test label is $+1$ with probability $p(x)$, independently. They disagree with probability

$$e(x, x') = p(x)(1-p(x')) + (1-p(x))p(x'). \tag{G.20}$$

As x' approaches x, $e(x, x')$ approaches $2p(x)(1-p(x))$.

Table G.3 shows the two coins at a test point with $p(x) = 0.8$. A neighbour where $p = 0.7$ disagrees with probability 0.38. A neighbour at the same p disagrees with probability 0.32, which is $2 \times 0.8 \times 0.2$. Even a perfect neighbour disagrees often, because both labels are noisy.

Table G.3.: The chance that the 1-NN prediction is wrong at a test point with $p(x) = 0.8$, for two neighbours.

Neighbour's $p(x')$	Test $+1$, neighbour -1	Test -1, neighbour $+1$	Disagree
0.7	$0.8 \times 0.3 = 0.24$	$0.2 \times 0.7 = 0.14$	0.38
0.8	$0.8 \times 0.2 = 0.16$	$0.2 \times 0.8 = 0.16$	0.32

G.5.3. Derivation

1. **The error given the points.** Fix all the feature vectors, of the training set and the test point. The nearest neighbour x'_n is then fixed, since it depends only on the points. Given the points, the labels are independent, each $+1$ with probability p at its own point. The neighbour was chosen without looking at labels, so its label is still $+1$ with probability $p(x'_n)$. The prediction is wrong when the two labels differ, which has probability $e(x, x'_n)$. Averaging over the points gives $R_n = \mathbb{E}[e(x, x'_n)]$.
2. **The nearest neighbour gets close.** Fix the test point x and a radius $r > 0$, and write $P_r(x)$ for the probability that one training point lands within distance r of x. The training points are independent, so the probability that none of them does is

$$\Pr\left[\|x'_n - x\| > r \,\middle|\, x\right] = (1 - P_r(x))^n, \tag{G.21}$$

   which tends to 0 whenever $P_r(x) > 0$.
3. **Almost every test point has $P_r(x) > 0$.** Suppose some ball around x has probability zero. Then a smaller ball inside it, with a centre whose coordinates are fractions and a radius that is a fraction, also has probability zero and still contains x. Such a ball exists because fractions come arbitrarily close to any number: take a centre within $r/3$ of x and a radius between $r/3$ and $2r/3$, where r is the radius of the original ball. There are only countably many such balls, and each of them that has probability zero contributes nothing, so by the union bound their union has probability zero. So the test points with a zero-probability ball around them have probability zero together. For all other test points, step 2 applies for every r. The averaged probability $\Pr[\|x'_n - x\| > r] = \mathbb{E}[(1 - P_r(x))^n]$ then tends to 0. This uses the **bounded convergence theorem**: if quantities between 0 and 1 tend to 0 at almost every point, so does their average.
4. **The error converges.** Expanding e gives $e(x, x') - e(x, x) = (p(x') - p(x))(1 - 2p(x))$, so $|e(x, x') - e(x, x)| \le |p(x') - p(x)|$. A continuous function on a closed and bounded set is uniformly continuous. So for every tolerance $\xi > 0$ there is an $r > 0$ with $|p(x') - p(x)| \le \xi$ whenever $\|x' - x\| \le r$. Split the average according to whether the neighbour is within r. Inside, the difference is at most ξ, and outside it is at most 1, so

$$\left|R_n - \mathbb{E}[2p(1-p)]\right| \le \mathbb{E}\left|e(x, x'_n) - e(x, x)\right| \le \xi + \Pr\left[\|x'_n - x\| > r\right]. \tag{G.22}$$

   By step 3 the last term tends to 0. The limit of the left side is therefore at most ξ, for every $\xi > 0$, which proves Equation G.19.

G.5.4. Checking it

In the chapter's running example, x is uniform on the unit sphere in $\mathbb{R}^{10}$ and $p(x) = \sigma(5x_1)$. The script computes the Bayes error $\mathbb{E}[\min(p, 1-p)] = 0.250$ and the limit $\mathbb{E}[2p(1-p)] = 0.338$, by numerical integration over the first coordinate. The sphere looks the same from every point, so every test point sees the same P_r, so the median distance to the nearest neighbour can be computed exactly, by solving $(1 - P_r)^n = \frac{1}{2}$ for r. Table G.4 shows that it shrinks very slowly.

Table G.4.: Exact median distance from a test point to its nearest neighbour, for points uniform on the unit sphere in $\mathbb{R}^{10}$.

Training points n	100	4,000	10^6	10^9
Median distance to the nearest neighbour	0.758	0.489	0.261	0.121

The sphere in $\mathbb{R}^{10}$ is a nine-dimensional surface, so P_r grows like r^9 and the distance shrinks only like $n^{-1/9}$. That is why the error approaches its limit slowly. In the script's table, with 4,000 test points, the measured 1-NN error is 0.398 at $n = 100$ and 0.356 at $n = 4{,}000$. The average of $e(x, x'_n)$ over the same test points, which step 1 says has the same mean, is 0.392 and 0.358. Both are still above 0.338, because at $n = 4{,}000$ the typical neighbour is still about 0.49 away, where p differs noticeably. Figure G.6 shows both parts of the argument.

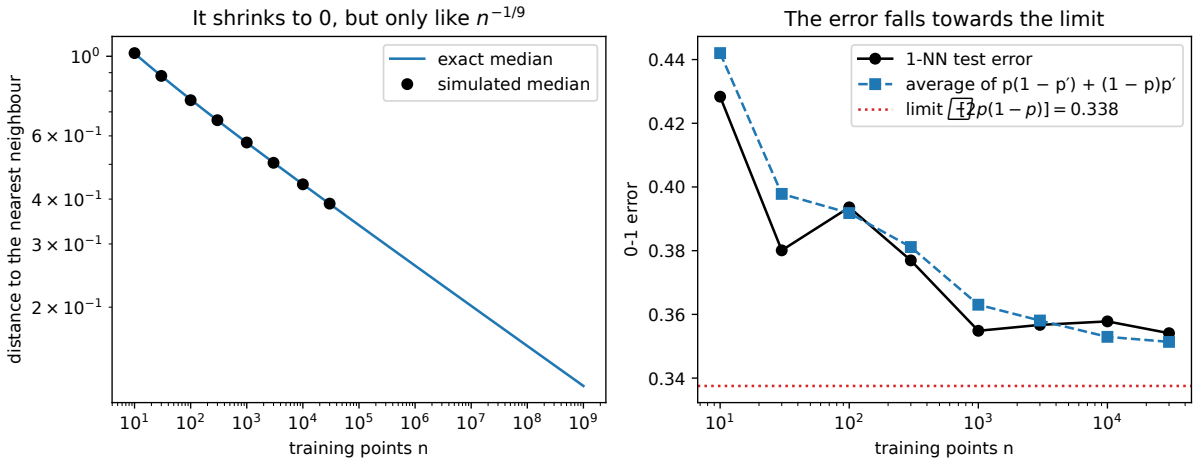


Figure G.6.: Left: the median distance from a test point to its nearest neighbour against n, exact (line) and simulated (dots), for points uniform on the unit sphere in $\mathbb{R}^{10}$. Right: the measured 1-NN test error and the average of $p(1-p') + (1-p)p'$ over the same 20,000 test points, with the limit $\mathbb{E}[2p(1-p)] = 0.338$.

G.5.5. What it does not cover

The limit holds without continuity of p, by a longer argument that approximates p by continuous functions, which is left to Devroye, Györfi & Lugosi (1996, ch. 5). The proof says nothing about the speed of convergence, and the example shows why none can be given without more assumptions: the speed depends on the dimension of the data. Finally, k-NN with k growing and k/n shrinking converges to the Bayes error itself, because averaging many neighbours' labels estimates $p(x)$ rather than drawing one more coin (Stone 1977).

G.6. Fitted parameters are approximately Gaussian, and one record moves them by its gradient

G.6.1. What the chapter uses

Section 12.3 states that parameters fitted by minimizing an average loss, which statisticians call **M-estimators**, are approximately Gaussian, centred on the population fit with covariance Σ_1/n. Here $\Sigma_1 = H^{-1}GH^{-1}$ is the **sandwich covariance**, H is the average curvature of the loss at the population fit, and G is the covariance of one record's gradient there. The chapter gives a four-step sketch and cites "the standard asymptotic normality of M-estimators (van der Vaart 1998, ch. 5)". The statement feeds the attack accuracy Equation 12.16 and the constant Equation 12.17. Section 10.3 uses the same fact to explain why the margin ϕ_z is close to Gaussian over retrainings. Section 10.5 explains in words which records have a large self-influence $\mu_{\text{in},z} - \mu_{\text{out},z}$, the numerator of the separation in Equation 10.12. And Section 11.3 asserts that as n grows the model approaches the population's probabilities.

This section proves the result for logistic regression, the model all four chapters use. A record is $z = (x, y)$, with a feature vector x, which may include a constant 1 for an intercept, and a label $y = \pm 1$. (The property-inference chapter writes the whole record as x; here the features get their own name.) The loss is $\ell(\theta; z) = \ln(1 + e^{-y\langle\theta, x\rangle})$. The fit $\hat{\theta}$ minimizes the training objective on the dataset $D = (z_1, \dots, z_n)$, here the average loss plus a ridge term, and the population fit θ^* minimizes its average over the data distribution $\mathcal{D}$:

$$\mathcal{L}(\theta; D) = \frac{1}{n}\sum_{i=1}^{n} \ell(\theta; z_i) + \frac{\lambda}{2}\|\theta\|^2, \qquad \mathcal{L}_{\mathcal{D}}(\theta) = \mathbb{E}_{z\sim\mathcal{D}}\, \ell(\theta; z) + \frac{\lambda}{2}\|\theta\|^2. \tag{G.23}$$

The ridge weight $\lambda \geq 0$ may be zero. (The property-inference chapter writes θ for the fit and $\theta(t)$ for the population fit. Here θ is any parameter vector, $\hat{\theta}$ the fit and θ^* the population fit.) The curvature and the gradient covariance are

$$H = \nabla^2 \mathcal{L}_{\mathcal{D}}(\theta^*), \qquad G = \mathbb{E}[g\, g^\top], \quad g = \nabla \ell(\theta^*; z) + \lambda\theta^*. \tag{G.24}$$

The assumptions are that the records are independent draws from one distribution, that $\mathbb{E}\|x\|^6$ is finite (true for bounded or Gaussian features, and needed so that the averages in step 2 have finite variance), and that H has no zero eigenvalue. Write $h > 0$ for its smallest eigenvalue. The claim has two parts. First, $\hat{\theta}$ is within a distance of order $1/\sqrt{n}$ of θ^*. Second, $\sqrt{n}(\hat{\theta} - \theta^*)$ is approximately Gaussian. It **converges in distribution** to a Gaussian, which means that for large n the probability that it lands in any box or half-space approaches the probability for the Gaussian:

$$\sqrt{n}\,(\hat{\theta} - \theta^*) \to \mathcal{N}(0,\ H^{-1}GH^{-1}). \tag{G.25}$$

The influence of one record is a statement about one fixed dataset, so it uses the membership-inference chapter's setting, with a ridge weight $\lambda > 0$ and the margin $\phi_z = y\langle\theta, x\rangle$ of a record $z = (x, y)$. Compare the fit $\hat{\theta}_D$ on a dataset D that contains z with the fit $\hat{\theta}_{-z}$ of the same objective with z's term removed. Training on z raises its margin by

$$\phi_z(\hat{\theta}_D) - \phi_z(\hat{\theta}_{-z}) = \frac{1}{n}\,\sigma(-\phi_z)\, x^\top H^{-1} x + O(1/n^2), \tag{G.26}$$

where ϕ_z on the right is the margin under $\hat{\theta}_{-z}$, and $\sigma(s) = 1/(1 + e^{-s})$ is the logistic function. Here H is the Hessian of the full training objective, with z's term included, at $\hat{\theta}_{-z}$, and "removed" means that the other terms keep their weight $1/n$.

G.6.2. The idea

The argument takes four moves. The first is that the fit is close to the population fit. With high probability the training objective curves upward by at least $h/2$ near θ^*. Its gradient at θ^* is an average of n independent terms with mean zero, so it has size about $1/\sqrt{n}$. A function that curves up by at least $h/2$ cannot have its minimum farther than about $2/h$ times the gradient away. The second move is that, near θ^*, the gradient is almost linear. The condition "gradient zero at $\hat{\theta}$" then becomes, exactly, $0 = \nabla\mathcal{L}(\theta^*; D) + \bar{H}(\hat{\theta} - \theta^*)$, where $\bar{H}$ is the average Hessian between the two points. Solving it gives $\hat{\theta} - \theta^* = -\bar{H}^{-1}\nabla\mathcal{L}(\theta^*; D)$. The third move is that the average gradient is approximately Gaussian, by the central limit theorem, with covariance G/n. The fourth is that $\bar{H}$ approaches H, and multiplying a Gaussian vector by the matrix $-H^{-1}$ multiplies its covariance by H^{-1} on both sides.

The simplest case shows why the covariance is a sandwich. For the loss $\frac{1}{2}(\theta - a)^2$ of a number a, the fit is the sample mean. The curvature is $H = 1$ and the gradient $\theta^* - a$ has variance $G = \mathrm{Var}(a)$, so the sandwich gives the familiar $\mathrm{Var}(a)/n$. Doubling the loss to $(\theta - a)^2$ doubles the curvature and quadruples the gradient variance. The two copies of H^{-1} undo the quadrupling, so the fit's variance does not change, as it should not, since the fit is still the mean.

G.6.3. Derivation

1. **The curvature of the logistic loss changes slowly.** The Hessian of one record's loss is $s(\langle\theta, x\rangle)\, xx^\top$, with $s(m) = \sigma(m)(1-\sigma(m))$. Its derivative is $s'(m) = \sigma(1-\sigma)(1-2\sigma)$, whose largest size is $c = 1/(6\sqrt{3}) \approx 0.0962$. To see this, write $\sigma = \frac{1}{2} + v$, so that $|s'| = 2|v|(\frac{1}{4} - v^2)$, which is largest at $v = 1/(2\sqrt{3})$. The matrix $xx^\top$ stretches no vector by more than $\|x\|^2$, and $|\langle\theta - \theta', x\rangle| \le \|\theta - \theta'\|\,\|x\|$, so the Hessian moves by at most $c\|x\|^3\|\theta - \theta'\|$. Averaging over the records, with $\hat{M}_3 = \frac{1}{n}\sum_i \|x_i\|^3$,

$$\big\|\nabla^2\mathcal{L}(\theta; D) - \nabla^2\mathcal{L}(\theta'; D)\big\| \le c\,\hat{M}_3\,\|\theta - \theta'\|. \tag{G.27}$$

2. **Three averages settle down.** Each of $\nabla^2\mathcal{L}(\theta^*; D)$, $\hat{M}_3$ and $\nabla\mathcal{L}(\theta^*; D)$ is an average of n independent terms. Their means are H, the third moment $M_3 = \mathbb{E}\|x\|^3$, and 0. The last mean is 0 because θ^* minimizes $\mathcal{L}_{\mathcal{D}}$, so $\mathbb{E}g = \nabla\mathcal{L}_{\mathcal{D}}(\theta^*) = 0$. Their variances are finite, because the sixth moment is. By Chebyshev's inequality, an average of n terms with variance v strays from its mean by more than a with probability at most $v/(na^2)$. (For a matrix, apply it to the sum of the squared entries, which is at least the square of the largest factor by which the matrix stretches a vector.) So each average is near its mean with probability tending to 1. Choose the radius $r = h/(8cM_3)$, whose role step 3 shows, and call E_n the event that all three are near their means in the following sense:

$$\big\|\nabla^2\mathcal{L}(\theta^*; D) - H\big\| \le \frac{h}{4}, \qquad \hat{M}_3 \le 2M_3, \qquad \big\|\nabla\mathcal{L}(\theta^*; D)\big\| < \frac{hr}{4}. \tag{G.28}$$

   Then E_n holds with probability tending to 1. For the gradient there is even an exact formula for the mean square. The cross terms of independent mean-zero vectors average to zero, so

$$\mathbb{E}\big\|\nabla\mathcal{L}(\theta^*; D)\big\|^2 = \frac{\operatorname{tr} G}{n}. \tag{G.29}$$

3. **The objective curves up near θ^*.** On E_n, take any θ within distance r of θ^*. By step 1 its Hessian differs from $\nabla^2\mathcal{L}(\theta^*; D)$ by at most $c \cdot 2M_3 \cdot r = h/4$, and that one differs from H by at most $h/4$. Since $H \succeq hI$, every eigenvalue of $\nabla^2\mathcal{L}(\theta; D)$ is at least $h - h/4 - h/4 = h/2$. So, by step 2 of Section G.1 applied to segments inside the ball, $\mathcal{L}(\cdot; D)$ is $(h/2)$-strongly convex on the ball of radius r around θ^*.
4. **The fit is inside the ball, and close.** On the sphere $\|\theta - \theta^*\| = r$, strong convexity and Cauchy–Schwarz give $\mathcal{L}(\theta; D) \geq \mathcal{L}(\theta^*; D) - \|\nabla\mathcal{L}(\theta^*; D)\|r + \frac{h}{4}r^2$. This is larger than $\mathcal{L}(\theta^*; D)$, by the third condition of E_n. A convex function that is higher on a whole sphere than at its centre cannot be lower anywhere outside. If some outside point θ had $\mathcal{L}(\theta; D) \leq \mathcal{L}(\theta^*; D)$, the segment from θ^* to θ would cross the sphere, and by the chord inequality the value at the crossing would be at most $\mathcal{L}(\theta^*; D)$. So the minimizer $\hat{\theta}$ is inside the ball. There, part (a) of Corollary 6.1, with strong convexity $h/2$ and $\nabla\mathcal{L}(\hat{\theta}; D) = 0$, gives

$$\|\hat{\theta} - \theta^*\| \leq \frac{2}{h}\,\|\nabla\mathcal{L}(\theta^*; D)\|, \tag{G.30}$$

   whose square has mean at most $4 \operatorname{tr} G/(h^2 n)$ by step 2. So the fit is within a distance of order $1/\sqrt{n}$ of the population fit, and in particular it converges to it.
5. **An exact linear equation for the error.** The fit has zero gradient. By the fundamental theorem of calculus along the segment from θ^* to $\hat{\theta}$, as in step 4 of the proof of Proposition 6.1, $0 = \nabla\mathcal{L}(\hat{\theta}; D) = \nabla\mathcal{L}(\theta^*; D) + \bar{H}_n(\hat{\theta} - \theta^*)$. Here $\bar{H}_n$ is the average of $\nabla^2\mathcal{L}(\cdot; D)$ along the segment, whose eigenvalues are at least $h/2$ by step 3, so it can be inverted:

$$\hat{\theta} - \theta^* = -\bar{H}_n^{-1}\,\nabla\mathcal{L}(\theta^*; D). \tag{G.31}$$

   This is the chapter's step 2 with "$\approx$" replaced by "$=$" and H replaced by $\bar{H}_n$.
6. **The average curvature approaches H.** By step 1, $\bar{H}_n$ differs from $\nabla^2\mathcal{L}(\theta^*; D)$ by at most $c\hat{M}_3\|\hat{\theta} - \theta^*\|/2$, which tends to 0 by step 4. By step 2, $\nabla^2\mathcal{L}(\theta^*; D)$ tends to H. The inverses follow, because $\bar{H}_n^{-1} - H^{-1} = \bar{H}_n^{-1}(H - \bar{H}_n)H^{-1}$ has size at most $\frac{2}{h} \cdot \frac{1}{h}\|H - \bar{H}_n\|$.
7. **The average gradient is approximately Gaussian.** The scaled gradient $\sqrt{n}\,\nabla\mathcal{L}(\theta^*; D) = \frac{1}{\sqrt{n}}\sum_i g_i$ is a scaled sum of independent vectors with mean 0 and covariance G. Take any fixed direction a. The numbers $\langle a, g_i\rangle$ are independent with mean 0 and variance $a^\top G a$, so the central limit theorem in one dimension (Section E.4) makes their scaled sum approximately $\mathcal{N}(0, a^\top G a)$. That is the distribution of $\langle a, Y\rangle$ for $Y \sim \mathcal{N}(0, G)$. The **Cramér–Wold device** says that the distributions of all the one-dimensional views $\langle a, \cdot\rangle$ determine the distribution of a vector, also in the limit (van der Vaart 1998, §2.2). So $\sqrt{n}\,\nabla\mathcal{L}(\theta^*; D)$ approaches $\mathcal{N}(0, G)$.
8. **Combine.** Multiply Equation G.31 by $\sqrt{n}$ and split it in two:

$$\sqrt{n}(\hat{\theta} - \theta^*) = -H^{-1}\sqrt{n}\,\nabla\mathcal{L}(\theta^*; D) + (H^{-1} - \bar{H}_n^{-1})\sqrt{n}\,\nabla\mathcal{L}(\theta^*; D). \tag{G.32}$$

   By step 7 the first term approaches $-H^{-1}Y$. A linear map A turns a Gaussian with covariance G into one with covariance $AGA^\top$, which here is $H^{-1}GH^{-1}$ because H is symmetric. In the second term, the matrix tends to 0 by step 6, while the vector has mean square $\operatorname{tr} G$ for every n. So their product tends to 0 in probability, by Chebyshev's inequality. Adding a term that tends to 0 in probability does not change the limiting distribution (Slutsky's lemma, van der Vaart 1998, Lemma 2.8). This proves Equation G.25. The event E_n fails with probability tending to 0, so it changes no limit.
9. **When the model is right, $G = H$.** Suppose $\lambda = 0$ and the labels really follow the model, $\Pr[y = 1 \mid x] = \sigma(\langle\theta^*, x\rangle) = p$. One record's gradient is $-\sigma(-y\langle\theta^*, x\rangle)\,y\,x$, so $gg^\top$ is

$(1-p)^2 xx^\top$ when $y=1$, which has probability p, and $p^2 xx^\top$ when $y=-1$. On average, given x, that is $p(1-p)^2 + (1-p)p^2 = p(1-p)$ times $xx^\top$. That is exactly the Hessian's weight s. So $G = H$ and the covariance is H^{-1}, the inverse of the Fisher information. The same case answers Section 11.3. The expected log-loss at each x is smallest when the predicted probability equals the true one, so the population fit of a correct model is the true parameter. By step 4 the fitted model then converges to the population's probabilities.

10. **The influence of one record.** Now fix the dataset D and the record $z = (x, y)$ in it, with $\lambda > 0$. The objective without z's term is $\mathcal{L}_{-z}$, and with it $\mathcal{L}(\cdot; D) = \mathcal{L}_{-z} + \frac{1}{n}\ell(\cdot; z)$. At $\hat{\theta}_{-z}$ the gradient of $\mathcal{L}_{-z}$ is zero, so the gradient of $\mathcal{L}(\cdot; D)$ there is just $\frac{1}{n}\nabla\ell(\hat{\theta}_{-z}; z)$. Step 5, applied to $\mathcal{L}(\cdot; D)$ around $\hat{\theta}_{-z}$ instead of around θ^*, gives exactly $\hat{\theta}_D - \hat{\theta}_{-z} = -\frac{1}{n}\bar{H}^{-1}\nabla\ell(\hat{\theta}_{-z}; z)$, with $\bar{H}$ the average Hessian of $\mathcal{L}(\cdot; D)$ along the segment. This move is small: $\mathcal{L}(\cdot; D)$ is λ-strongly convex and the logistic gradient has length at most $\|x\|$, so consequence (a) gives $\|\hat{\theta}_D - \hat{\theta}_{-z}\| \le \|x\|/(\lambda n)$. Replacing $\bar{H}$ by the Hessian H of $\mathcal{L}(\cdot; D)$ at $\hat{\theta}_{-z}$ therefore changes the matrix by $O(1/n)$, by step 1, and the move by $O(1/n^2)$. The logistic gradient is $\nabla\ell = -\sigma(-\phi_z)\, y\, x$, so the move is $\frac{1}{n}\sigma(-\phi_z)\, y\, H^{-1}x$. The margin $\phi_z = y\langle\theta, x\rangle$ changes by y times the inner product of the move with x, which is $\frac{1}{n}\sigma(-\phi_z)\, y^2\, x^\top H^{-1}x$. Since $y^2 = 1$, that is Equation G.26.

Step 10 turns the heuristic of Section 10.5 into a formula. The factor $\sigma(-\phi_z)$ is close to 1 when the model trained without z gets its label wrong, and close to 0 when it already predicts it confidently. The factor $x^\top H^{-1}x$ counts the length of x twice, once for how hard z pulls the weights and once for how much a move of the weights changes z's own margin, and divides by the curvature in that direction. In the chapter, the models without z train on the same number of records, with another record in z's place. That record pulls the fit by its own gradient. Averaged over which record it is, the pull is the average gradient of the loss, which is close to zero near the population fit, where only the small ridge term $\lambda\theta$ balances it. So, to first order, the self-influence $\mu_{\text{in},z} - \mu_{\text{out},z}$ is the average of Equation G.26 over training sets.

G.6.4. Checking it

The script checks the Gaussian picture on the logistic-regression toy of Section 12.3. The features are $x \sim \mathcal{N}(0, I_2)$ with an intercept. Records from the majority get the label $+1$ with probability $\sigma(2x_1 + x_2)$, records from a hidden subgroup with probability $\sigma(2x_1 - x_2)$, and the subgroup makes up a share $t = \frac{1}{2}$ of the population. Numerically, the largest of $|s'|$ is 0.09623, which is $1/(6\sqrt{3})$. Computed by quadrature, the population fit is $(1.704, 0, 0)$ for $(\theta_1, \theta_2, \text{intercept})$, and the sandwich has diagonal $(12.59, 6.94, 6.04)$, the chapter's values. The chapter trains its models with a ridge weight 10^{-3}. Including it in $\mathcal{L}_{\mathcal{D}}$, as the theorem allows, gives the population fit $(1.683, 0, 0)$ and the diagonal $(11.83, 6.76, 5.89)$. This is the right comparison for models trained that way.

Table G.5 compares n times the variance over 2,000 refits, with that ridge weight, against this sandwich. It also shows the share of refits whose θ_2 lands within 1.96 predicted standard deviations of its centre, which is 0.950 for an exact Gaussian.

Table G.5.: Scaled variances of logistic regressions refitted on 2,000 training sets at subgroup share $t = \frac{1}{2}$, against the sandwich covariance, which includes the ridge weight 10^{-3}.

n	$n\,\mathrm{Var}(\theta_1)$	$n\,\mathrm{Var}(\theta_2)$	$n\,\mathrm{Var}(\text{intercept})$	θ_2 within ± 1.96 sd
50	22.06	9.62	7.94	0.902
200	13.61	7.48	6.16	0.935
1,000	11.72	6.95	5.92	0.950
sandwich	11.83	6.76	5.89	0.950

At $n = 1{,}000$ every variance is within 3% of the sandwich, about the sampling error of a variance estimated from 2,000 refits. At $n = 50$ the variances are larger and the tails heavier, because the linearization of step 5 is not yet accurate. This also explains a remark in Section 12.3. There the chapter compares its refits with the sandwich without the ridge weight, and finds a gap of about 5% in θ_1. The ridge weight alone accounts for most of it, since it lowers the predicted value from 12.59 to 11.83. Figure G.7 shows the histograms becoming Gaussian, and Figure G.8 (left) shows the scaled variances settling on the sandwich.

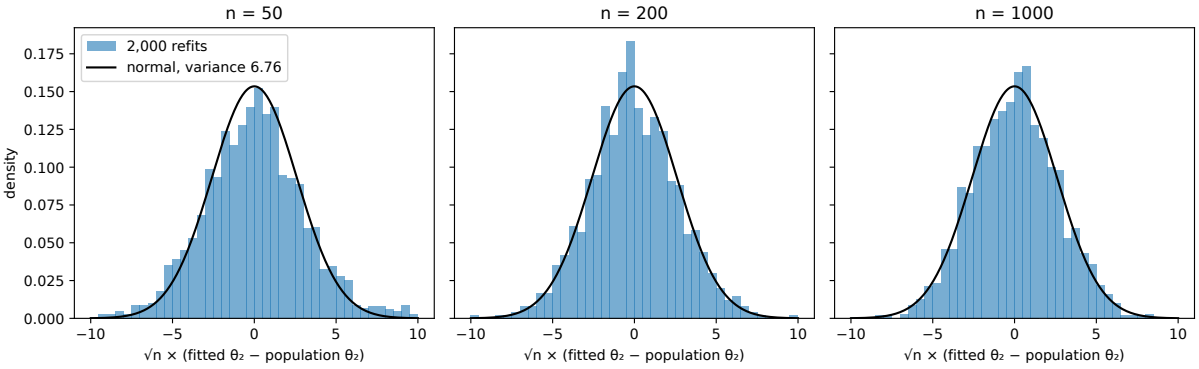


Figure G.7.: Histograms of $\sqrt{n}(\hat{\theta}_2 - \theta_2^*)$ over 2,000 logistic regressions refitted at subgroup share $t = \frac{1}{2}$, for $n = 50$, 200 and 1,000, with the normal density whose variance is the sandwich value.

Step 9 checks out as well, with $\lambda = 0$ as it requires. With the majority's rule alone, $t = 0$, the model is correct. Its population fit is $(2.000, 1.000, 0.000)$, the true weights, and G and H agree to rounding error, within 10^{-12}. At $t = \frac{1}{2}$ the model is wrong, because no single logistic rule describes the mixture. Then the diagonals differ: G has $(0.076, 0.191, 0.166)$ and H has $(0.078, 0.166, 0.166)$.

The influence formula is checked on the running example of the membership-inference chapter. That example has $d = 100$ features, 950 typical points and 50 outliers with five times longer x and a random label, and $\lambda = 10^{-3}$. One model is trained on $n = 500$ points. For 200 of them the script removes the point's term, refits, and compares the measured change in its margin with Equation G.26. For the 188 typical points, the mean change is 0.2287 against a predicted 0.2291, and the median ratio of measured to predicted is 1.003. For the 12 outliers, the mean change is 3.92 against a predicted 8.21, with a median ratio of 0.671. An outlier moves its own margin by several units, far outside the range where the linear expansion is accurate. The $O(1/n^2)$ term of

step 10 has a constant that grows with $\|x\|$ and with $1/\lambda$, and both are large for outliers. So the formula ranks the records correctly but overstates the largest influences. Figure G.8 (right) shows both groups.

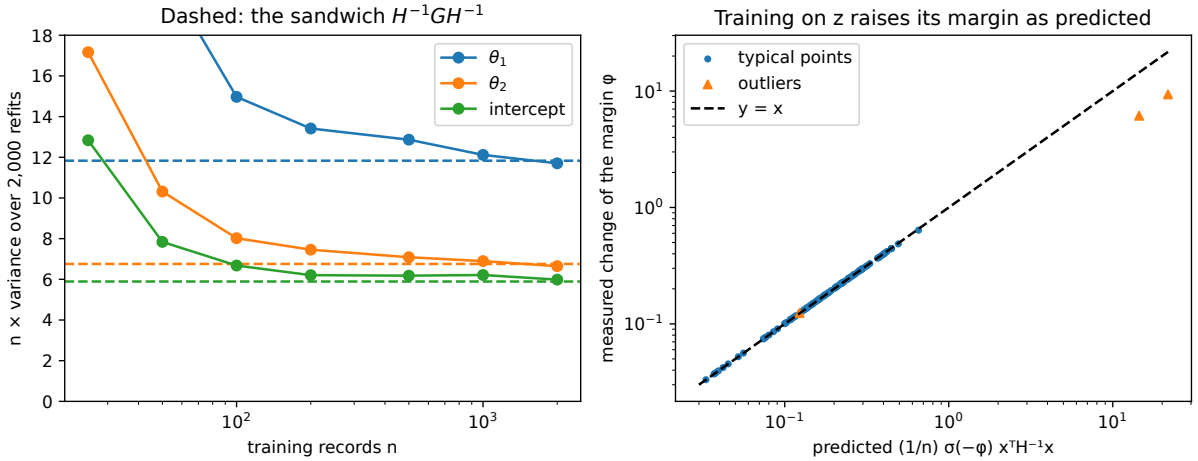


Figure G.8.: Left: n times the variance of each fitted parameter over 2,000 refits, against n, with the sandwich values (dashed). Right: for points of the membership-inference running example, the measured change of the margin ϕ_z when the point's term is added, against the first-order prediction Equation G.26. Typical points lie on the diagonal; outliers fall below it.

G.6.5. What it does not cover

The result is a limit, so it says nothing about how large n must be; a finite-sample version (a Berry–Esseen bound) needs further moment assumptions. The proof also needs H to be invertible and a minimizer to exist. Both can fail with $\lambda = 0$: when a line separates the classes perfectly, the loss keeps falling as the weights grow, so no minimizer exists at all. When the number of features grows with n, so that d/n does not go to zero, the sandwich is wrong: the fitted weights are too large in size, pushed away from zero, and their variance is larger than predicted (Sur & Candès 2019). This matters for the membership example, where $d/n = 0.2$. The general theory of M-estimators, for other losses and without convexity, is in van der Vaart (1998, ch. 5). For deep networks, which have many minima, the Gaussian shape of ϕ_z that Section 10.3 relies on is an empirical finding of Carlini et al., not a consequence of this theorem.

G.7. The best membership attack looks at the parameters through the loss

G.7.1. What the chapter uses

Section 9.2 studies a **centroid detector**: the model is the mean θ of n records, and an attacker who holds a record z plays a game against it. In the member world z is one of the n records, and in the non-member world all n records are fresh draws. The chapter gives the log likelihood ratio between the two worlds, saying only that "expanding the squares in the two Gaussian

densities gives" it. It then cites Sablayrolles et al. (2019): if training draws the parameters with a probability that decreases exponentially in the training loss, the Bayes-optimal membership attack depends on the parameters, to a good approximation, only through the target's loss, calibrated for each target, and white-box access then adds little. The chapter's chain Equation 9.5 compares access levels, and this result explains why, in its example, access to the weights (white-box) gives no better attack than access to the loss of each query, compared with what that loss would typically be (calibrated black-box).

The setting of Sablayrolles et al. is this. There are n candidate records $z_1, \dots, z_n$. Each is a member of the training set independently with probability π_1, and $m_i \in \{0, 1\}$ marks whether z_i is one. Training draws the parameters θ from a **Gibbs posterior** at a temperature $\nu > 0$, a density that falls exponentially in the total training loss:

$$p(\theta \mid m) = \frac{1}{Z(m)} \exp\Big(-\frac{1}{\nu} \sum_{i=1}^{n} m_i \, \ell(\theta; z_i) \Big), \tag{G.33}$$

where $Z(m)$ makes the density integrate to 1. Write m_{-1} for the memberships of the other candidates, and σ for the logistic function. The attacker knows the candidates and sees θ. The posterior probability that the target z_1 is a member is then exactly

$$\Pr[m_1 = 1 \mid \theta] = \mathbb{E}\Big[\sigma\Big(-\frac{\ell(\theta; z_1) - \tau(z_1, m_{-1})}{\nu} + \ln \frac{\pi_1}{1 - \pi_1} \Big) \,\Big|\, \theta \Big], \tag{G.34}$$

with the calibration

$$\tau(z_1, m_{-1}) = -\nu \ln \mathbb{E}_{\theta' \sim p(\cdot \mid m_1 = 0,\ m_{-1})} \big[e^{-\ell(\theta'; z_1)/\nu} \big]. \tag{G.35}$$

Inside the logistic function, θ appears only through the loss $\ell(\theta; z_1)$. It also appears in the outer average over the unknown m_{-1}, and that is where the result becomes an approximation. If τ hardly depends on the other memberships, it can be replaced by one number $\tau(z_1)$ per target, and the best attack thresholds $\ell(\theta; z_1) - \tau(z_1)$. That last step is Sablayrolles et al.'s approximation, not a theorem.

For the centroid detector no approximation is needed. Write $q(\theta \mid z)$ for the density of θ in the member world and $p(\theta \mid z)$ in the non-member world. The model is $\theta = \frac{1}{n} \sum_i x_i$ for n records $x_i \sim \mathcal{N}(0, I_d)$, and its loss is $\ell(\theta; z) = \frac{1}{2} \|\theta - z\|^2$. The log likelihood ratio between the member and non-member worlds is exactly

$$\ln \frac{q(\theta \mid z)}{p(\theta \mid z)} = -\frac{n}{n-1} \big(\ell(\theta; z) - \tau(z) \big) + \frac{d}{2} \ln \frac{n}{n-1}, \qquad \tau(z) = \frac{(n-1)\|z\|^2}{2n}. \tag{G.36}$$

G.7.2. The idea

Fix the memberships of all the other candidates for a moment. Then the only question is whether the term of z_1 is in the exponent or not. Adding it multiplies the density of θ by $e^{-\ell(\theta; z_1)/\nu}$ and changes the normalizing constant. The change of normalizing constant is a number that does not depend on θ, and τ is $-\nu$ times its logarithm. So, with the others fixed, the log of the posterior odds is a straight-line function of the loss. The attacker does not know the others, so it averages over them.

Two targets with the same loss show why the calibration matters. Take $\nu = 1$ and $\pi_1 = \frac{1}{2}$, so that the prior term $\ln(\pi_1/(1-\pi_1))$ is zero. A hard target has $\tau = 2$, because models trained without it give it a typical loss near 2. If the released model gives it loss 0.5, the posterior is $\sigma(2-0.5) = \sigma(1.5) = 0.8176$, likely a member. An easy target has $\tau = 0.3$, because every model fits it well. The same loss 0.5 gives $\sigma(0.3-0.5) = \sigma(-0.2) = 0.4502$, slightly less likely than not. A low loss is evidence of membership only relative to what the target's loss would be without it.

G.7.3. Derivation

1. **Bayes' rule with the others fixed.** Write $A = \pi_1\, p(\theta \mid m_1 = 1, m_{-1})$ and $B = (1-\pi_1)\, p(\theta \mid m_1 = 0, m_{-1})$ for the two joint weights. Bayes' rule says $\Pr[m_1 = 1 \mid \theta, m_{-1}] = A/(A+B)$. A ratio of this form equals $\sigma(\ln(A/B))$, because $1/(1+B/A) = 1/(1+e^{-\ln(A/B)})$. So
$$\Pr[m_1 = 1 \mid \theta, m_{-1}] = \sigma\Big(\ln \frac{p(\theta \mid m_1 = 1, m_{-1})}{p(\theta \mid m_1 = 0, m_{-1})} + \ln \frac{\pi_1}{1-\pi_1}\Big). \tag{G.37}$$
2. **The density ratio.** The two exponents differ by the target's term only, so the density ratio is $e^{-\ell(\theta;z_1)/\nu}\, Z_0/Z_1$, where Z_1 and Z_0 are the normalizing constants with and without the target. Write Z_1 as the integral of $e^{-\ell(\theta';z_1)/\nu}$ times the unnormalized density without the target. Dividing by Z_0 turns that integral into an average over θ' drawn from the posterior without the target:
$$\frac{Z_1}{Z_0} = \mathbb{E}_{\theta' \sim p(\cdot \mid m_1 = 0,\ m_{-1})}\big[e^{-\ell(\theta';z_1)/\nu}\big] = e^{-\tau(z_1, m_{-1})/\nu}. \tag{G.38}$$
So the log density ratio is $-(\ell(\theta; z_1) - \tau(z_1, m_{-1}))/\nu$, a function of θ only through the loss.
3. **Average over the unknown others.** By the law of total probability, $\Pr[m_1 = 1 \mid \theta]$ is the average of $\Pr[m_1 = 1 \mid \theta, m_{-1}]$ over m_{-1}, weighted by $\Pr[m_{-1} \mid \theta]$. Together with steps 1 and 2 that is Equation G.34. The weights depend on θ, which is why replacing $\tau(z_1, m_{-1})$ by a single $\tau(z_1)$ is an approximation.
4. **The centroid: two Gaussians.** If z is a member, $\theta = (z + S)/n$, where S, the sum of the other $n-1$ records, is $\mathcal{N}(0, (n-1)I_d)$. So θ is Gaussian with mean z/n and variance $(n-1)/n^2$ per coordinate. If it is not, all n records are fresh draws, so θ is Gaussian with mean 0 and variance $1/n$ per coordinate. The log of each density is minus the squared distance to the mean over twice the variance, plus $-\frac{d}{2}\ln$ of the variance. Their difference is
$$-\frac{n^2}{2(n-1)}\Big\|\theta - \frac{z}{n}\Big\|^2 + \frac{n}{2}\|\theta\|^2 + \frac{d}{2}\ln\frac{n}{n-1}. \tag{G.39}$$
5. **Expand the squares.** The first square is $\|\theta\|^2 - \frac{2}{n}\langle z, \theta\rangle + \frac{1}{n^2}\|z\|^2$. Collecting terms, $\|\theta\|^2$ has coefficient $\frac{n}{2} - \frac{n^2}{2(n-1)} = -\frac{n}{2(n-1)}$, and $\langle z, \theta\rangle$ has coefficient $\frac{n}{n-1}$. So the log ratio is $\frac{n}{2(n-1)}(2\langle z, \theta\rangle - \|\theta\|^2) - \frac{\|z\|^2}{2(n-1)} + \frac{d}{2}\ln\frac{n}{n-1}$, the chapter's formula with its constant made explicit.
6. **Write it with the loss.** Since $\ell(\theta; z) = \frac{1}{2}\|z\|^2 - \frac{1}{2}(2\langle z, \theta\rangle - \|\theta\|^2)$, the bracket of step 5 equals $\|z\|^2 - 2\ell(\theta; z)$. Substituting, the $\|z\|^2$ terms are $\frac{n}{2(n-1)}\|z\|^2 - \frac{1}{2(n-1)}\|z\|^2 = \frac{1}{2}\|z\|^2$, which is $\frac{n}{n-1}\tau(z)$. The result is Equation G.36. It has the Gibbs form exactly, with temperature $(n-1)/n$, and no approximation was needed. The reason is that the other records enter only through their sum S, whose distribution does not depend on which records they are.

The chapter's remark that the white-box and calibrated black-box attacks "agree to four decimals" follows from step 5. The calibrated score $\|z\|^2 - \|z-\theta\|^2$ is the bracket $2\langle z,\theta\rangle - \|\theta\|^2$, so the log ratio is a fixed multiple of it minus the term $\|z\|^2/(2(n-1))$. That term changes from target to target, so the two scores do not rank the games (pairs of model and target) in exactly the same order. But it changes very little: its standard deviation is 0.0504, against 0.757 for the log ratio itself, at $n = 100$ and $d = 50$.

G.7.4. Checking it

The script checks Equation G.34 exactly on a small Gibbs posterior. It uses 10 candidates $z_i \sim \mathcal{N}(0,1)$, the loss $\frac{1}{2}(\theta - z)^2$, $\nu = 1$, $\pi_1 = \frac{1}{2}$, and θ on a grid of 801 points in $[-4, 4]$. Enumerating all 2^9 memberships of the other candidates, the exact posterior and the right side of Equation G.34 differ by less than 10^{-12}, which is rounding error. Replacing $\tau(z_1, m_{-1})$ by the single value $\bar{\tau}(z_1) = -\nu \ln \mathbb{E}_{m_{-1}}[e^{-\tau(z_1,m_{-1})/\nu}]$, an average over memberships drawn from their prior, changes the posterior by up to 0.054 over the grid points that carry 99.9% of the probability of θ, so in this small example the approximation is reasonable but visible.

For the centroid, Equation G.36 matches the log ratio computed from the two densities to within 10^{-10}, rounding error. Over 100,000 games per world, the best advantage is 0.2804 for the exact likelihood ratio, 0.2800 for the calibrated score and 0.0422 for the raw loss. (The chapter's own run gives 0.2789 for the first two.) Figure G.9 shows why calibration matters. Against the raw loss, the exact log ratio is a cloud, because a target's loss mixes its membership with how typical it is. Against $\ell - \tau(z)$ it is a line of slope $-n/(n-1)$.

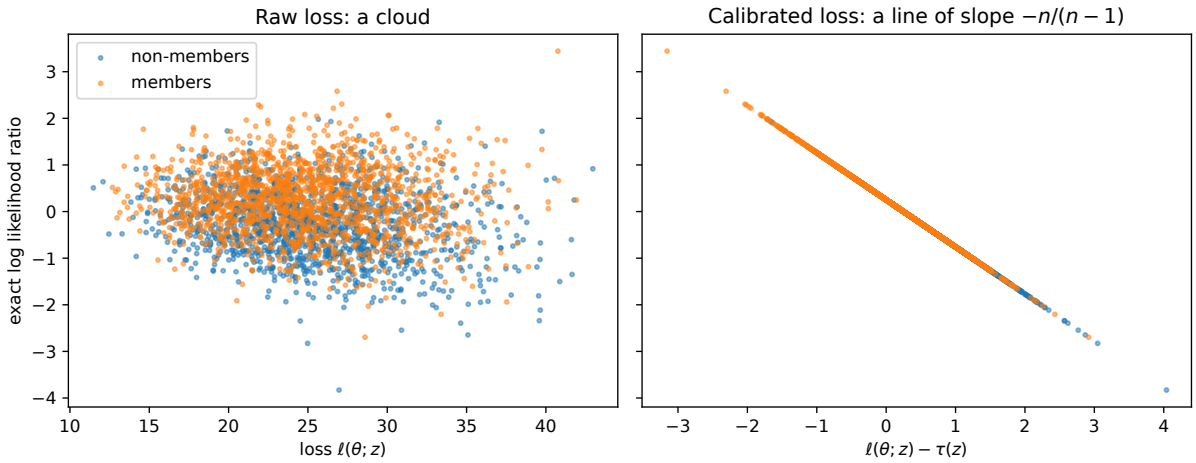


Figure G.9.: For 1,500 member and 1,500 non-member games of the centroid detector ($n = 100$, $d = 50$), the exact log likelihood ratio against the raw loss $\ell(\theta; z)$ (left) and against the calibrated loss $\ell(\theta; z) - \tau(z)$ (right).

G.7.5. What it does not cover

Real training is not a Gibbs posterior. SGD with a constant learning rate behaves like one only roughly, and an exactly trained model, like the centroid, is deterministic given its data. The

centroid escapes the problem because, from the attacker's view, the unknown other records make θ random, and the result has the Gibbs form anyway. In general the per-target calibration $\tau(z)$ must be estimated, which is what shadow models do in the per-example attacks of Membership inference.

G.8. Code

The script apx_optimization.py computes every number on this page and draws every figure. It checks the inequalities on random pairs and on gradient-descent runs. It runs the subgradient method against the resisting gradients and computes the exact nearest-neighbour distances on the sphere. It also refits logistic regressions thousands of times, compares leave-one-out refits with the influence formula, and enumerates a small Gibbs posterior. From the repository root:

```
uv run python code/appendix-derivations/apx_optimization.py
```

Table G.6 collects the main numbers it prints.

Table G.6.: The main numbers printed by the appendix's script.

Check	Result
Strong convexity of the ridge logistic risk, $\lambda = 0.01$	smallest ratio 0.0372 over 2,000 pairs
GD, largest gap ratio per step against $1 - 1/\kappa = 0.9615$	0.7131
GD gap after 50 steps, against Equation G.3	2.59×10^{-9} against 1.99×10^{-2}
Co-coercivity, smallest ratio of the two sides	1.0006 (logistic loss), 5.3704 (logistic risk)
Worst distance ratio after one step, $\eta = 8$ and $\eta = 10$	0.9988 and 1.4985
Early stopping, $n = 100$, $\|\theta^*\| = 5$, $L = 1$, $\eta = 0.25$: u^* and T^*	25 (also the grid minimum) and 100 steps
Early stopping: the two terms at u^*, and the bound	0.5 each; $1 + 0.125 = 1.125$
Hard function, $T = 10{,}000$: lower bound, method, SGD bound	0.0050, 0.0058, 0.0100
1-NN limit and Bayes error	0.338 and 0.250
1-NN error and plug-in average at $n = 4{,}000$	0.356 and 0.358
Sandwich diagonal at $t = \frac{1}{2}$, ridge 10^{-3}	$(11.83, 6.76, 5.89)$
n Var over 2,000 refits at $n = 1{,}000$	$(11.72, 6.95, 5.92)$
Influence, median measured/predicted: typical points, outliers	1.003, 0.671
Gibbs identity, largest error	below 10^{-12}
Centroid advantage: likelihood ratio, calibrated score, raw loss	0.2804, 0.2800, 0.0422

Every bound holds where it should, and the checks also show how loose each one is. The linear rate of GD is exact in form but pessimistic in its curvature, while co-coercivity is tight where the loss is most curved, and the lower bound is within 17% of what the subgradient method achieves. The Gaussian picture of a fitted model is accurate to sampling error at $n = 1{,}000$, while its first-order influence formula fails exactly for the records that matter most to membership attacks, the outliers.

G.9. References

- Yurii Nesterov, *Lectures on Convex Optimization*, 2nd ed. (2018), §2.1. The characterizations of convex and strongly convex functions, co-coercivity (Theorem 2.1.5), and the inequality that gives the distance rate of GD without second derivatives (Theorem 2.1.12).
- Boris Polyak, "Gradient methods for minimizing functionals" (1963), and Hamed Karimi, Julie Nutini & Mark Schmidt, "Linear convergence of gradient and proximal-gradient methods under the Polyak–Łojasiewicz condition" (2016). The inequality of step 5 in Section G.1 and how far it reaches beyond convexity.
- Moritz Hardt, Benjamin Recht & Yoram Singer, "Train faster, generalize better: Stability of stochastic gradient descent" (2016). The stability of SGD on convex and non-convex losses, which Section G.3 balances against its optimization error.
- Shai Shalev-Shwartz & Shai Ben-David, *Understanding Machine Learning* (2014), ch. 14. The optimization error of SGD on convex Lipschitz losses, used in Section G.3.
- Sébastien Bubeck, "Convex Optimization: Algorithms and Complexity" (2015), Theorem 3.13. The hard function of Section G.4, and the cutting-plane methods of ch. 2.
- Arkadi Nemirovski & David Yudin, *Problem Complexity and Method Efficiency in Optimization* (1983). The lower bound for every gradient method, not only span-restricted ones.
- Thomas Cover & Peter Hart, "Nearest neighbor pattern classification" (1967). The original 1-NN limit.
- Luc Devroye, László Györfi & Gábor Lugosi, *A Probabilistic Theory of Pattern Recognition* (1996), ch. 5. The 1-NN limit without continuity, and nearest-neighbour rules in general.
- Charles Stone, "Consistent nonparametric regression" (1977). k-NN rules that reach the Bayes error.
- Aad van der Vaart, *Asymptotic Statistics* (1998), ch. 2 and ch. 5. The Cramér–Wold device, Slutsky's lemma, and the asymptotic normality of M-estimators in general.
- Peter Huber, "The behavior of maximum likelihood estimates under nonstandard conditions" (1967). The sandwich covariance for a misspecified model.
- Pang Wei Koh & Percy Liang, "Understanding black-box predictions via influence functions" (2017). The influence formula of step 10 as a tool for explaining predictions.
- Pragya Sur & Emmanuel Candès, "A modern maximum-likelihood theory for high-dimensional logistic regression" (2019). What replaces the sandwich when d/n does not go to zero.
- Alexandre Sablayrolles, Matthijs Douze, Yann Ollivier, Cordelia Schmid & Hervé Jégou, "White-box vs black-box: Bayes optimal strategies for membership inference" (2019). The Gibbs-posterior result of Section G.7 and its approximation.

H. Random matrices and factorizations

Two chapters of the book lean on facts about matrices that they quote rather than prove. Reconstruction attacks decodes noisy answers by least squares, so its error depends on two properties of the random query matrix. The average of the inverse of its Gram matrix sets the error under random noise, and its smallest singular value sets the error under worst-case noise. The chapter on training with differential privacy (DP) by follow the regularized leader (FTRL), DP-FTRL and matrix factorization, releases the running sums of a training run by splitting the prefix-sum matrix into a product of two matrices. That chapter quotes several facts about this matrix and its factorizations, from its singular values to the form of the best factorization, and this appendix proves them. Each section starts by restating, in the chapter's notation, exactly what it proves. They sit together because each one is a statement about how a matrix stretches or shrinks vectors, and each proof comes down to an eigenvalue, a singular value or a projection.

The two chapters both call their matrix A, but it is a different matrix in each. In the first two sections, the query matrix A has one row for each of m queries and one column for each of n people. In the last three, A is the prefix-sum matrix, with one row and one column for each of T training steps. Table H.1 lists the results and where each chapter states or uses them.

Table H.1.: The derivations in this appendix and the chapter sections that use them.

Result	Where the chapter states or uses it
The average inverse of a Gram matrix with Gaussian rows is $\Sigma^{-1}/(m-n-1)$	Proposition 17.3, step 4 of its proof; Proposition 17.5, step 4 of its proof
A random subset matrix has $s_{\min}(A) \geq \sqrt{m}/14$ once m is of order $n \ln n$	Section 17.3, the worst-case bound Equation 17.13 (Proposition 17.2) and the paragraph after it
The prefix-sum matrix: singular values, trace-norm duality, the square root's coefficients, how close the square root is to optimal	Theorem 41.2 and its proof; Theorem 41.3, step 5 of its proof, and the comparison with the lower bound in Section 41.4
The optimal factorization for the root-mean-square error	Theorem 41.4
Lower-triangular factors lose nothing; several adaptive participations	Theorem 41.1, step 3 of its proof and the paragraph after it; Theorem 41.5, step 3 of its proof

H.1. The average inverse of a random Gram matrix

H.1.1. What the chapter uses

Least squares turns noisy answers into an estimate, and under random noise its error is set by the inverse of the **Gram matrix** $A^\top A$, the matrix of inner products between the columns of the query matrix. In Section 17.3, step 4 of the proof of Proposition 17.3 needs the average of that inverse. The chapter uses a result for Gaussian rows. Let Z be an $m \times n$ matrix whose m rows are independent, each a normal vector with mean 0 and covariance matrix Σ. Then $Z^\top Z$ is called a **Wishart matrix**, and, as long as $m \geq n + 2$,

$$\mathbb{E}\big[(Z^\top Z)^{-1}\big] = \frac{\Sigma^{-1}}{m - n - 1}. \tag{H.1}$$

Here m is the number of rows, n the number of columns and Σ^{-1} the inverse of the covariance. The chapter cites Muirhead (1982, ch. 3) for it. It applies it to rows that are random 0/1 queries q, by matching a normal row with the same second-moment matrix, $\Sigma = \mathbb{E}[qq^\top]$, and it says that for 0/1 rows this is only an approximation. Section 17.5 reuses it in its step 4, which gives Equation 17.23. The chapter checks the final recovery rates by simulation but not the formula itself.

H.1.2. The idea

Start with one column, $n = 1$. Then $Z^\top Z$ is a single number, the sum of m squared standard normal values, and its average is m. The naive guess for the average of its inverse is $1/m$. But an average of inverses is always larger than the inverse of the average, because small values blow up under $1/x$ much more than large values shrink. For example, a number that is 1 or 3 with equal chance has average 2, while the average of its inverses is $\frac{1}{2}(1 + \frac{1}{3}) = \frac{2}{3}$, not $\frac{1}{2}$. For the sum of m squared normals the exact answer is $1/(m - 2)$, so the naive guess is off by two rows. The formula says this is $1/(m - n - 1)$ with $n = 1$.

With more columns, and $\Sigma = I$ for now, there are two separate questions. Which matrix is the average? Symmetry answers that: flipping the sign of a column, or swapping two columns, does not change the distribution of Z, so the average inverse cannot prefer any column or any sign, and it must be a multiple of the identity. And what is the multiple? One diagonal entry of the inverse of a Gram matrix has a meaning of its own. It is one over the squared length of the part of the first column that the other columns cannot explain, the residual of regressing column 1 on the others. With Gaussian columns, that residual is a standard normal vector squeezed into a space of $m - n + 1$ dimensions, so its squared length is a sum of $m - n + 1$ squared normals. The one-column calculation then gives $1/(m - n + 1 - 2) = 1/(m - n - 1)$.

H.1.3. Derivation

Take $\Sigma = I$ first, so that the entries of Z are independent standard normal numbers. Write $W = Z^\top Z$ and $z_1, \ldots, z_n$ for the columns of Z.

1. **The inverse exists.** With $m \geq n$ Gaussian rows, the columns are linearly independent with probability 1. Add the columns one at a time: given the earlier ones, their span is a fixed subspace of lower dimension, and a column with a continuous distribution lands in it with probability 0. So W is invertible. (The stronger condition $m \geq n+2$ is needed only in step 6, to make the average finite.)
2. **Off-diagonal entries average to 0.** Flip the sign of column i of Z. The flipped matrix has the same distribution, because a standard normal entry and its negative have the same distribution. Flipping column i flips the sign of every inner product between column i and another column, so it flips row i and column i of W, except the diagonal entry. The inverse changes in the same way, since the flip is multiplication by a diagonal matrix Λ of signs on both sides, and $(\Lambda W \Lambda)^{-1} = \Lambda W^{-1} \Lambda$. So every entry of W^{-1} in row i, off the diagonal, has the same average as its negative, and that average, which is finite by steps 4 to 6 and the Cauchy–Schwarz inequality, is 0.
3. **Diagonal entries share one average.** Swapping columns i and k also leaves the distribution unchanged, and it swaps the diagonal entries i and k of W^{-1}. So all diagonal entries have the same average c, and $\mathbb{E}[W^{-1}] = cI$.
4. **A diagonal entry is one over a residual.** Let P be the orthogonal projection onto the span of $z_2, \dots, z_n$, so that $(I-P)z_1$ is the part of z_1 that the other columns cannot explain. Let w be the first column of W^{-1}, so $Ww = e_1$, where $e_1 = (1, 0, \dots, 0)$. Set $u = Zw$, a vector of length m. Then $Z^\top u = Z^\top Z w = Ww = e_1$, and this equation says two things: u is orthogonal to $z_2, \dots, z_n$, and $z_1^\top u = 1$. Now $u = w_1 z_1 + (\text{a combination of } z_2, \dots, z_n)$. Applying $I-P$ removes the combination and leaves u unchanged, since u is orthogonal to the other columns, so $u = w_1 (I-P) z_1$. Substituting this into $z_1^\top u = 1$ gives $w_1 \|(I-P)z_1\|^2 = 1$, because $z_1^\top (I-P) z_1 = \|(I-P)z_1\|^2$ for a projection. So
$$\big[W^{-1}\big]_{11} = \frac{1}{\|(I-P)z_1\|^2}. \tag{H.2}$$
5. **The residual is a sum of $m-n+1$ squared normals.** Fix the other columns. Then $I-P$ is a fixed projection onto a subspace of dimension $k = m-n+1$, the m dimensions minus the $n-1$ spanned by $z_2, \dots, z_n$. The column z_1 is independent of the others, and a standard normal vector keeps its distribution under any rotation, because its density depends only on the length of the vector. So rotate the coordinates until the subspace is spanned by the first k axes. In the new coordinates z_1 is still standard normal, and $\|(I-P)z_1\|^2$ is the sum of the squares of its first k coordinates, a **chi-squared** variable χ^2_k with k degrees of freedom. This holds for every value of the other columns, so it holds without conditioning too.
6. **The average of $1/\chi^2_k$.** The trick is to write $1/x$ as an integral of exponentials, $1/x = \int_0^\infty e^{-tx}\, dt$ for $x > 0$, because exponentials of a sum of independent squares are easy to average. For one standard normal number g and a number $t \geq 0$,
$$\mathbb{E}\big[e^{-tg^2}\big] = (1+2t)^{-1/2}, \tag{H.3}$$
because e^{-tg^2} times the normal density is a normal density with variance $1/(1+2t)$, missing only its normalizing factor. A χ^2_k variable X is a sum of k independent such squares, so the averages multiply and $\mathbb{E}[e^{-tX}] = (1+2t)^{-k/2}$. Exchanging the average and the integral, which is allowed because everything is positive,
$$\mathbb{E}\Big[\frac{1}{X}\Big] = \int_0^\infty (1+2t)^{-k/2}\, dt = \frac{1}{k-2} \quad \text{for } k \geq 3. \tag{H.4}$$

The integral converges only when $k/2 > 1$. Putting in $k = m-n+1$ gives $c = 1/(m-n-1)$, and the condition $k \geq 3$ becomes $m \geq n+2$.

7. **General covariance.** Write $\Sigma^{1/2}$ for the symmetric square root of Σ, the matrix whose square is Σ. A normal row with covariance Σ can be written as a standard normal row times $\Sigma^{1/2}$, so $Z = Z_0\Sigma^{1/2}$ with Z_0 standard. Then $Z^\top Z = \Sigma^{1/2} Z_0^\top Z_0 \Sigma^{1/2}$, its inverse is $\Sigma^{-1/2}(Z_0^\top Z_0)^{-1}\Sigma^{-1/2}$, and averaging, with the fixed matrices taken outside the average, and using steps 3 and 6 gives Equation H.1.

H.1.4. Checking it

The script draws Gaussian matrices with $n = 50$ columns and averages m times the diagonal of $(Z^\top Z)^{-1}$ over 4,000 draws. The formula predicts $m/(m-n-1)$. It also draws 0/1 matrices, as in the chapter, whose second-moment matrix is $\Sigma = \frac{1}{4}(I + \mathbf{1}\mathbf{1}^\top)$. The diagonal of Σ^{-1} is $4n/(n+1)$, as computed in Section 17.3, step 3. Dividing by it puts both kinds of rows on the same scale, so that both are predicted to give $m/(m-n-1)$. Table H.2 shows the result.

Table H.2.: m times the average diagonal entry of the inverse Gram matrix, $n = 50$ columns, 4,000 draws per entry.

Rows m	60	100	200	400
Predicted, $m/(m-n-1)$	6.667	2.041	1.342	1.146
Gaussian rows, simulated	6.661	2.041	1.342	1.146
0/1 rows, simulated and rescaled	6.456	2.000	1.329	1.140

The Gaussian rows match the formula within the simulation noise. The 0/1 rows fall a little below it, by $6.667 - 6.456 = 0.211$ at $m = 60$, about 3%, and by less as m grows. So for subset queries the chapter's approximation is slightly pessimistic: it predicts a slightly larger error than the attack suffers. Figure H.1 shows the same comparison over a range of m. The gap between the curve $m/(m-n-1)$ and the naive line at 1 is the correction that step 4 of the chapter makes, and it is large only when m is close to n.

Steps 4 and 5 can be checked one at a time. For one Gaussian matrix with $m = 60$ rows, the script regresses the first column on the others and finds a squared residual of 4.931982. One over the first diagonal entry of the inverse is the same 4.931982, as Equation H.2 says. Figure H.2 repeats this for 20,000 matrices and compares the histogram with the χ^2 density with $m-n+1 = 11$ degrees of freedom. Step 6 for $k = 11$ predicts $1/9 = 0.1111$, and a million simulated χ^2_{11} values give 0.1112. Finally, the symmetry of steps 2 and 3 shows up in simulation as an average inverse whose off-diagonal entries are tiny. With $n = 10$, $m = 30$ and 20,000 draws, their average size is 0.0013 of the diagonal for Gaussian rows. For 0/1 rows the ratio is 0.1000, because 0/1 entries have a nonzero mean and flipping a column's sign is no longer a symmetry. That is still the shape the approximation predicts, since Σ^{-1} is proportional to $I - \frac{1}{n+1}\mathbf{1}\mathbf{1}^\top$, whose off-diagonal entries are $\frac{1}{n} = 0.1$ times its diagonal ones.

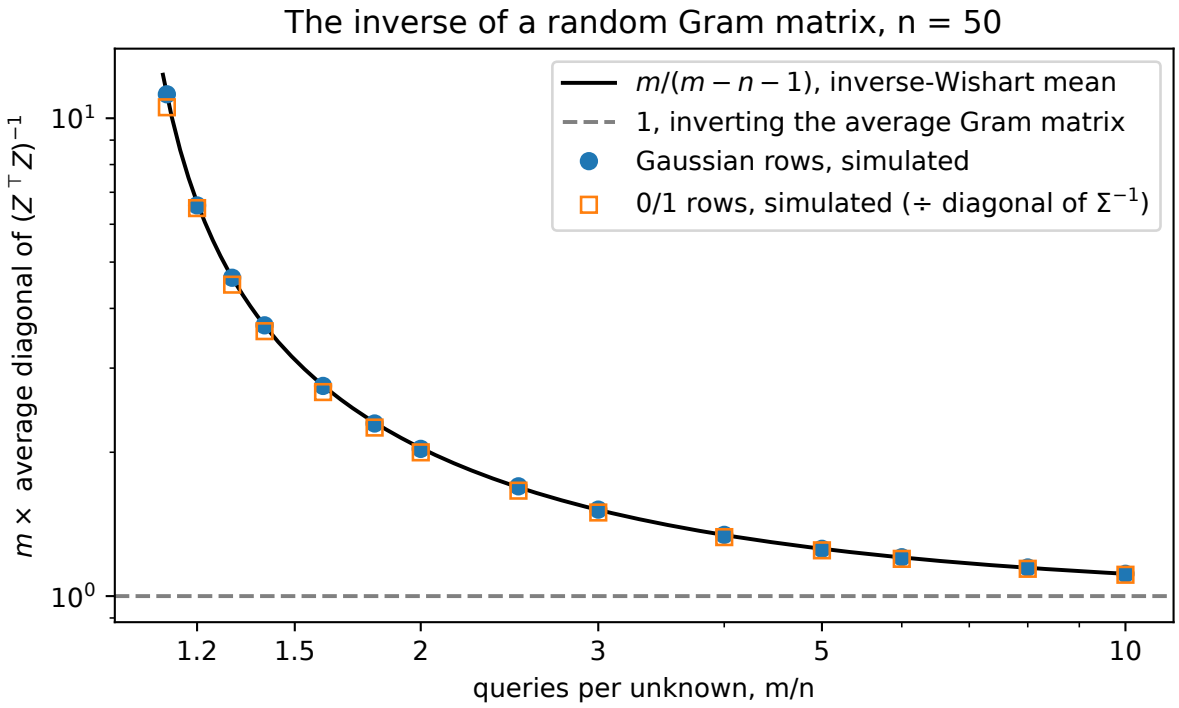


Figure H.1.: The average diagonal entry of $(Z^\top Z)^{-1}$, multiplied by m, against the number of rows per column m/n for $n = 50$. Circles are Gaussian rows and squares are 0/1 rows rescaled by the diagonal of Σ^{-1}. The black curve is $m/(m-n-1)$ and the dashed line is the naive value 1, from inverting the average Gram matrix.

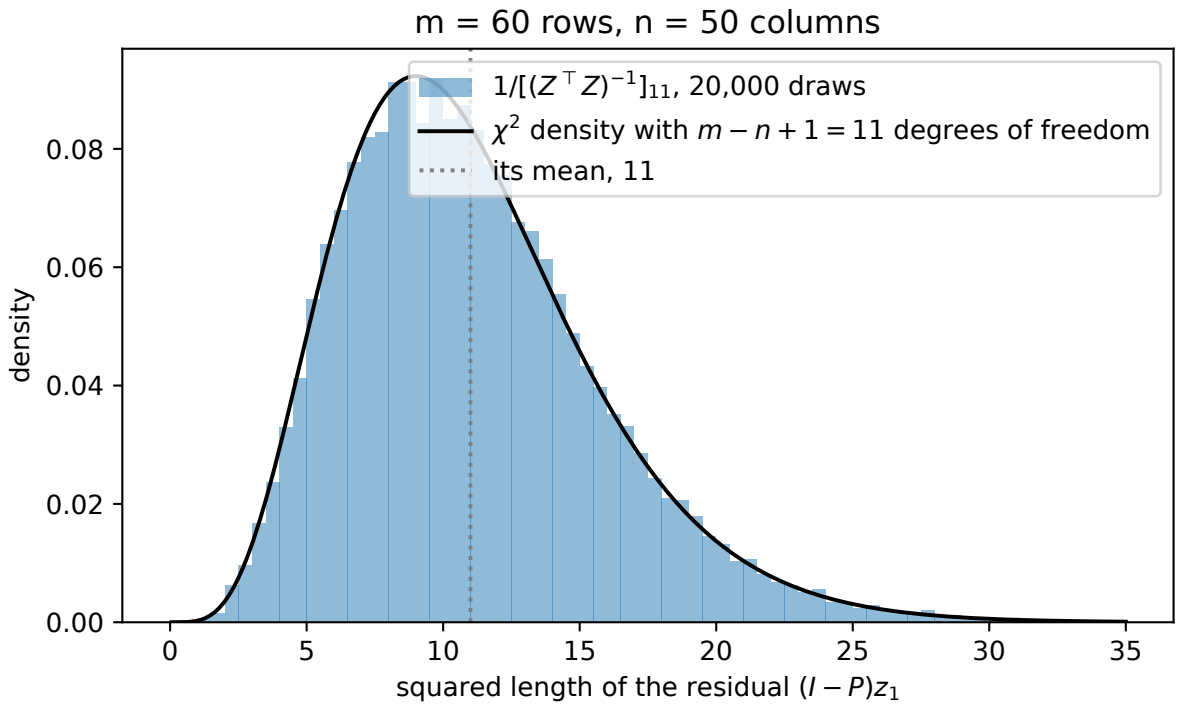


Figure H.2.: Histogram of $1/[(Z^\top Z)^{-1}]_{11}$ over 20,000 Gaussian matrices with $m = 60$ rows and $n = 50$ columns, against the χ^2 density with $m-n+1 = 11$ degrees of freedom. The two agree, as steps 4 and 5 predict.

H.1.5. What it does not cover

The proof needs Gaussian rows in two places. Step 2 needs each column to keep its distribution when its sign flips, and step 5 needs rotation invariance. Rows of 0/1 entries have neither property, which is why the chapter calls the formula an approximation and checks the attack by simulation. There is also a technical gap for 0/1 rows. A 0/1 matrix is singular with a small but positive probability, for example when two columns happen to be equal, so the average of its inverse is not even defined. The rescaled values in Table H.2 are averages over draws that happened to be invertible, which at these sizes is every draw.

The condition $m \geq n + 2$ is needed. At $m = n + 1$ the residual is a χ^2_2 variable, whose inverse has an infinite average, because χ^2_2 puts too much probability near 0. Even for $m \geq n+2$, the diagonal entries of the inverse have a heavy right tail when m is close to n, which makes simulations there noisy.

H.2. A random subset matrix shrinks no direction much

H.2.1. What the chapter uses

The worst-case bound of Section 17.3, Equation 17.13, says that least squares gets at most $4mE^2/s_{\min}(A)^2$ bits wrong when every answer has error at most E. Here $s_{\min}(A)$, the **smallest singular value**, is the smallest length $\|Ax\|$ over unit vectors x. The bound is useful only if $s_{\min}(A)$ is of order $\sqrt{m}$. The paragraph after Proposition 17.2 says that for random rows $s_{\min}$ "concentrates near $\sqrt{m} - \sqrt{n}$ times the scale of the rows" and cites Vershynin, *High-Dimensional Probability*, ch. 4. It then suggests by a heuristic that $s_{\min}(A) \approx \frac{1}{2}(\sqrt{m} - \sqrt{n})$ for random subsets, checks this at one size, and points here for a proof.

This section proves a version with a worse constant and an extra logarithm, but a complete proof. Let A be an $m \times n$ matrix whose rows are independent uniformly random subsets $q_1, \dots, q_m$, so each entry is an independent fair 0 or 1. For a failure probability β,

$$s_{\min}(A) \geq \frac{\sqrt{m}}{14} \quad \text{with probability at least } 1 - \beta, \quad \text{once} \quad m \geq 128\Big(n \ln(80n + 1) + \ln \frac{1}{\beta}\Big). \tag{H.5}$$

Substituted into Equation 17.13, this gives at most $4mE^2 \cdot 196/m = 784E^2$ wrong bits. So with $m = O(n \log n)$ queries, error $E = o(\sqrt{n})$ leaves $o(n)$ wrong bits.

H.2.2. The idea

The proof reuses a lemma from the chapter's linear-programming section, Section 17.2, in three moves. The lemma says that one random query catches any fixed direction: for any fixed vector x, the squared answer $\langle q, x\rangle^2$ exceeds $\|x\|^2/16$ with probability at least $\frac{3}{16}$. Over m independent queries, Hoeffding's inequality then makes it nearly certain that a good share of the queries catch x, and each caught query adds a fixed amount to $\|Ax\|^2$. That handles one direction. To handle all directions at once, round every unit vector to a nearby point of a fine grid, use the union bound over the finitely many grid points, and pay for the rounding with a crude bound on how much A can stretch the small difference. Table H.3 lists the chain of constants.

Table H.3.: How the lemma for one query becomes a bound on the smallest singular value.

Move	What it gives	Holds
Lemma, one query	$\langle q, x\rangle^2 > \|x\|^2/16$	with probability $\geq \frac{3}{16}$, for each fixed x
Hoeffding, m queries	more than $m/8$ such queries, so $\|Ax\|^2 > m\|x\|^2/128$	except with probability $e^{-m/128}$, for each fixed x
Union bound over the grid	the same for every grid point	except with probability $(80n+1)^n e^{-m/128}$
Rounding	$\|Ax\| \geq \sqrt{m}/14$ for every unit x	whenever the previous row holds

The all-ones direction deserves a word, because subset queries have a nonzero mean. Every entry of A is $\frac{1}{2}$ on average, so A stretches the all-ones vector a lot: each answer $\langle q, \mathbf{1}\rangle$ is about $n/2$. That direction gives the largest singular value, not the smallest, and the lemma already allows for it, since a nonzero mean only pushes $\langle q, x\rangle$ away from 0.

H.2.3. Derivation

1. **One query catches a fixed direction.** This is the chapter's lemma, whose proof holds for every vector x (the chapter adds an assumption about large coordinates only afterwards). Fix a nonzero x with n coordinates, and write a random subset as $q_i = (1 + s_i)/2$ with independent fair signs s_i, so that a person is in the subset when the sign is $+1$. The answer $Y = \langle q, x\rangle$ is then the constant $\frac{1}{2}\sum_i x_i$ plus the sum $\frac{1}{2}\sum_i s_i x_i$ of terms with random signs. Write τ for the standard deviation of that sum, so $\tau^2 = \frac{1}{4}\|x\|^2$. Counting which terms survive in the expansion of the fourth power shows that a random-sign sum has fourth moment at most 3 times its squared variance, as for a normal variable, and the chapter's step 3 shows that adding the constant keeps $\mathbb{E}Y^4 \leq 3(\mathbb{E}Y^2)^2$. The Paley–Zygmund inequality (Equation 17.5) says that a non-negative variable W exceeds $\xi\,\mathbb{E}W$ with probability at least $(1-\xi)^2(\mathbb{E}W)^2/\mathbb{E}[W^2]$, for any level ξ between 0 and 1. Applied to $W = Y^2$, the ratio of moments is at least $\frac{1}{3}$. Finally, $\mathbb{E}Y^2$ is τ^2 plus the squared constant, so it is at least τ^2, and

$$\Pr\left[|Y| > \sqrt{\xi}\,\tau\right] \geq \frac{(1-\xi)^2}{3}. \tag{H.6}$$

   With $\xi = \frac{1}{4}$ the bound reads

$$\Pr\left[\langle q, x\rangle^2 > \frac{\|x\|^2}{16}\right] \geq \frac{3}{16}. \tag{H.7}$$

2. **Many queries catch it, by Hoeffding's inequality.** Call query j *good* for x when $\langle q_j, x\rangle^2 > \|x\|^2/16$, and let N be the number of good queries. The queries are independent, so N is a sum of m independent 0/1 terms, each 1 with probability $p \geq \frac{3}{16}$. Hoeffding's inequality (Concentration inequalities) says that the average of m independent terms in $[0, 1]$ falls more than t below its mean with probability at most e^{-2mt^2}. The event $N \leq m/8$ means the average falls at least $p - \frac{1}{8} \geq \frac{1}{16}$ below its mean, so with $t = \frac{1}{16}$ it has probability at most $e^{-2m/256} = e^{-m/128}$.

3. **Many good queries make $\|Ax\|$ large.** $\|Ax\|^2 = \sum_j \langle q_j, x\rangle^2$ is a sum of non-negative terms, and each good query contributes more than $\|x\|^2/16$. If $N > m/8$, then $\|Ax\|^2 > \frac{m}{8} \cdot \frac{\|x\|^2}{16} = \frac{m\|x\|^2}{128}$.
4. **A finite grid.** Every unit vector has its coordinates in $[-1, 1]$. Take the grid of points in $[-1, 1]^n$ whose coordinates are multiples of $h = 1/(40n)$. Each coordinate takes $2/h + 1 = 80n+1$ values, so there are $(80n+1)^n$ grid points. Rounding each coordinate of a unit vector x to the nearest multiple of h gives a grid point y with $|x_i - y_i| \le h/2$, so $\|x-y\| \le \frac{h}{2}\sqrt{n} = \frac{1}{80\sqrt{n}}$, and hence $\|y\| \ge 1 - \frac{1}{80\sqrt{n}}$.
5. **Union bound over the grid.** By steps 2 and 3, each nonzero grid point fails $\|Ay\|^2 > m\|y\|^2/128$ with probability at most $e^{-m/128}$, and the point 0 does not matter, because no unit vector is rounded to it. The union bound says that the chance that at least one of several events happens is at most the sum of their chances. So every grid point satisfies it except with probability at most $(80n + 1)^n e^{-m/128}$. Taking logarithms, this is at most β exactly when $m \ge 128(n \ln(80n + 1) + \ln(1/\beta))$, the condition of Equation H.5.
6. **A crude bound on stretching.** Every entry of A is 0 or 1, so $\|Av\|^2 = \sum_j \langle q_j, v\rangle^2 \le \sum_j \|q_j\|^2\|v\|^2 \le mn\|v\|^2$, by the Cauchy–Schwarz inequality and because each query has at most n ones, so $\|q_j\|^2 \le n$. So A stretches any vector by at most $\sqrt{mn}$.
7. **From the grid to every unit vector.** Outside the bad event of step 5, take any unit vector x and its grid point y. By the triangle inequality and steps 3 to 6,

$$\|Ax\| \ge \|Ay\| - \|A(x-y)\| \ge \sqrt{\frac{m}{128}}\Big(1 - \frac{1}{80\sqrt{n}}\Big) - \sqrt{mn} \cdot \frac{1}{80\sqrt{n}}. \tag{H.8}$$

   The right side equals $\sqrt{m}$ times $\frac{1}{\sqrt{128}} - \frac{1}{80}(1 + \frac{1}{\sqrt{128n}})$, which grows with n, so it is smallest at $n = 1$, where it is $\sqrt{m}\,(\frac{79}{80\sqrt{128}} - \frac{1}{80}) = 0.0748\sqrt{m}$, which is more than $\sqrt{m}/14 = 0.0714\sqrt{m}$. The smallest $\|Ax\|$ over unit vectors is $s_{\min}(A)$, which gives Equation H.5.
8. **The consequence for least squares.** Substituting $s_{\min}(A)^2 \ge m/196$ into Equation 17.13 bounds the number of wrong bits by $4mE^2/(m/196) = 784E^2$. If $E = o(\sqrt{n})$, then $E^2 = o(n)$, so the wrong bits are $o(n)$.

H.2.4. Checking it

The proof is far from tight. Figure H.3 plots the simulated $s_{\min}(A)/\sqrt{m}$ against m/n for $n = 50$, 100 and 200. It falls below the proved floor of $1/14$ only at $m = 1.25n$, and from about $m = 4n$ on it sits on the cited asymptotic $\frac{1}{2}(1 - \sqrt{n/m})$. At the chapter's size, $n = 500$ and $m = 2000$, the script finds $s_{\min}(A) = 11.44$, which is $0.2558\sqrt{m}$. The asymptotic predicts $\frac{1}{2}(1 - \sqrt{1/4}) = 0.25$ times $\sqrt{m}$, and the proof guarantees $\sqrt{m}/14$. The price of the proof is the number of queries it asks for. For $n = 100$ people and failure probability $\beta = 0.01$, Equation H.5 asks for 115,627 queries, about 1,156 per person. The simulation shows the floor holding from $m = 1.5n$ on.

Figure H.4 shows where the slack comes from. For one matrix with $n = 100$ and $m = 400$, it plots $\|Ax\|^2/m$ for 4,000 random unit directions. Random directions land between about 0.2 and 3. For a unit vector the average of $\langle q, x\rangle^2$ is $\frac{1}{4}(1 + (\sum_i x_i)^2)$, the variance of one entry plus a share of the mean that grows with how far x points along the all-ones direction. The weakest direction is lower, and the all-ones direction is about a hundred times higher, because of the nonzero mean of the entries. The proof's levels, $\frac{1}{128}$ for a grid point and $\frac{1}{196}$ for every direction, are well below all

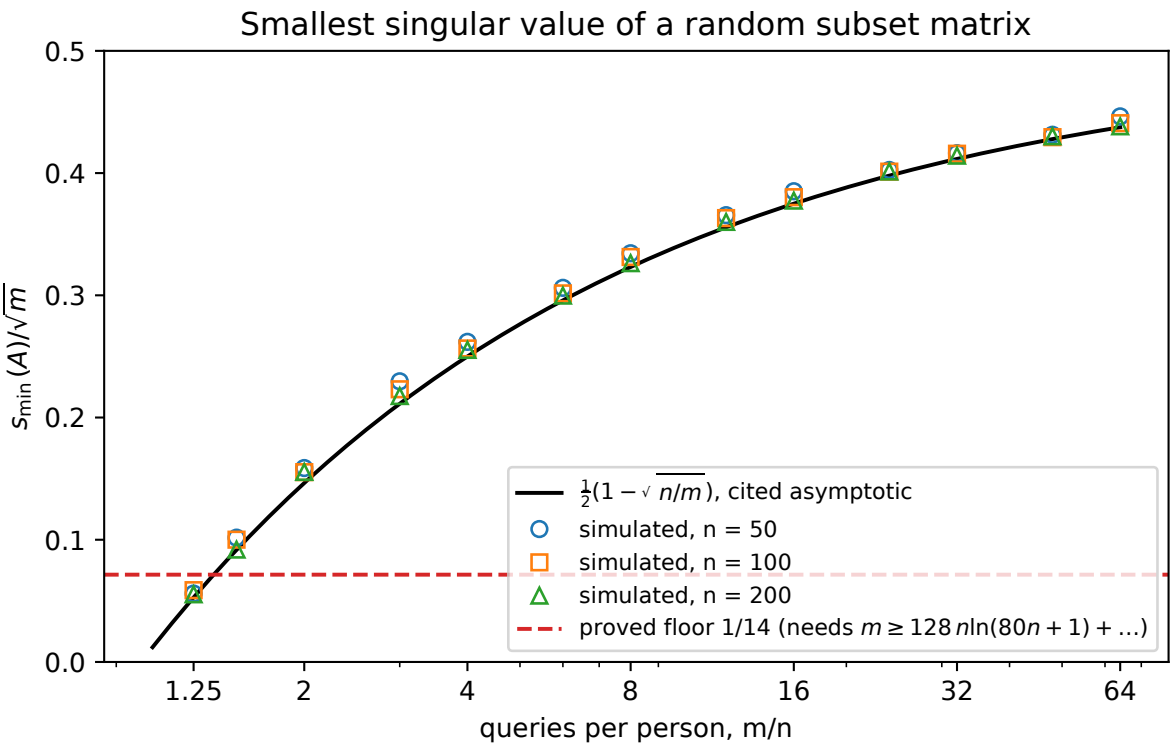


Figure H.3.: The smallest singular value of a random $m \times n$ subset matrix, divided by $\sqrt{m}$, against m/n, for three values of n (three draws each). The black curve is the cited asymptotic $\frac{1}{2}(1-\sqrt{n/m})$, and the dashed line is the floor $1/14$ proved here, which the proof guarantees only once m is of order $128\, n \ln(80n)$.

of them. The lemma itself has slack too. For the weakest direction of this matrix, 0.3075 of the queries are good, and no random direction of the 4,000 has fewer than 0.5550 good queries. The lemma guarantees only $\frac{3}{16} = 0.1875$ on average.

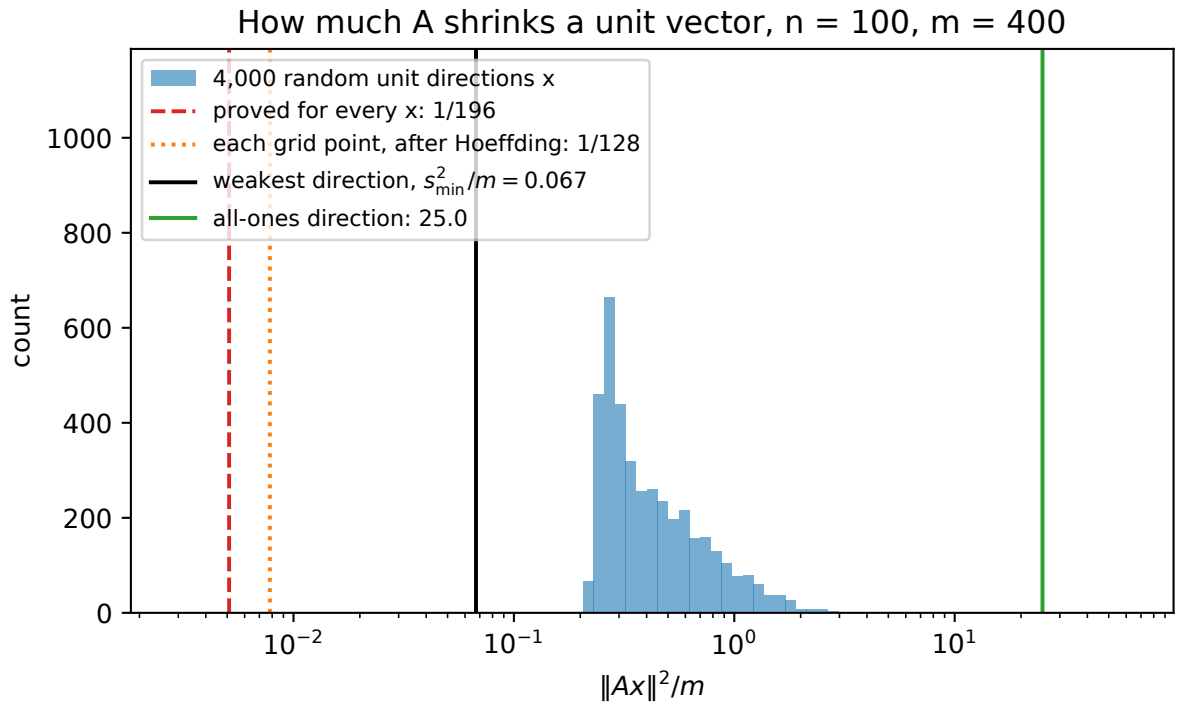


Figure H.4.: Histogram of $\|Ax\|^2/m$ over 4,000 random unit vectors x, for one random subset matrix with $n = 100$ and $m = 400$ (log scale). The vertical lines mark the weakest direction $s_{\min}^2/m$, the all-ones direction, and the two levels of the proof: $\frac{1}{128}$ for each grid point after Hoeffding's inequality, and $\frac{1}{196}$ for every direction.

H.2.5. What it does not cover

The proof gives $m = O(n \log n)$, not $m = 4n$. The logarithm comes from the union bound over a grid fine enough to absorb the crude stretching bound of step 6. For matrices whose entries have mean 0, called centred, a sharper argument bounds the stretching by about $\sqrt{m} + \sqrt{n}$, so it can use a much coarser grid of directions (a net), and it needs only $m = O(n)$ (Vershynin, ch. 4). For subset matrices it would also have to treat the all-ones direction separately, and this appendix does not carry that out. The sharp constant, $s_{\min} \approx \frac{1}{2}(\sqrt{m} - \sqrt{n})$ for large m and n at a fixed ratio, is the Bai–Yin law for matrices with independent centred entries (Bai and Yin 1993; Vershynin, ch. 4). Subset matrices are not centred, but they differ from $\frac{1}{2}$ times a random-sign matrix by the rank-one matrix $\frac{1}{2}\mathbf{1}\mathbf{1}^\top$, and on vectors orthogonal to $\mathbf{1}$ the two act the same. Turning that observation into a proof of the sharp constant takes a perturbation argument for the one remaining direction, which stays cited. So the chapter's statement with $m = 4n$ rests on the cited asymptotics and on its simulation, while the $O(n \log n)$ version is proved here.

H.3. The prefix-sum matrix

H.3.1. What the chapter uses

In DP-FTRL and matrix factorization, A is the $T \times T$ **prefix-sum matrix**: lower-triangular, with every entry on and below the diagonal equal to 1, so that row t of Ax is the running sum $x_1 + \cdots + x_t$. A factorization $A = LR$ releases $L(Rx + z)$, where z is Gaussian noise. Its **sensitivity** $\Delta_2 R$ is the largest length of a column of R, and its error per unit of privacy at step t is $e_t = \Delta_2 R \, \|\ell_t\|$, with ℓ_t the t-th row of L. The chapter uses four facts about this matrix.

The first two are in the proof of Theorem 41.2. Every factorization has a root-mean-square error (RMSE) of at least $\|A\|_*/T$ (Equation 41.9), where the RMSE is $\left(\frac{1}{T}\sum_t e_t^2\right)^{1/2}$, the typical error over the T released sums, where the **trace norm** $\|A\|_*$ is the sum of the singular values. The proof uses the duality

$$\operatorname{tr}(U^\top A) \le \|A\|_* \quad \text{for every } U \text{ whose largest singular value is at most 1,} \tag{H.9}$$

which the chapter states without proof. It also states the singular values, with $\omega_k = (2k-1)\pi/(2T+1)$,

$$\sigma_k(A) = \frac{1}{2\sin\frac{(2k-1)\pi}{4T+2}} = \frac{1}{2\sin(\omega_k/2)}, \qquad k = 1, \dots, T, \tag{H.10}$$

and says that the eigenvalues of $(A^{-1})^\top A^{-1}$ are $4\sin^2(\omega_k/2)$, checking this numerically.

The other two are in Section 41.4. The square-root factorization uses the lower-triangular matrix with entries $r_k = \binom{2k}{k}4^{-k}$ on its k-th diagonal (the chapter writes them c_k, keeping r_τ for the columns of R), and step 5 of the proof of Theorem 41.3 bounds its sensitivity and largest error, $\sum_{k<T} r_k^2$, with Equation 41.14. That step quotes the Wallis-type inequality

$$\frac{1}{\pi(k+\frac{1}{2})} \le r_k^2 \le \frac{1}{\pi(k+\frac{1}{4})} \tag{H.11}$$

from Fichtenberger, Henzinger and Upadhyay (2023) and checks it numerically for $k < 100{,}000$. Finally, the chapter compares the square root with the lower bound at $T = 100$ and $T = 10{,}000$ and finds it within 9% and 6%. This section proves the duality, the singular values and the Wallis bounds, and it explains the comparison by proving that the ratio tends to 1:

$$\frac{\ln(2T+1)}{\pi} \le \frac{\|A\|_*}{T} \le \text{RMSE of any factorization}, \qquad \text{RMSE of the square root} \le 1 + \frac{0.8 + \ln\frac{4T-3}{5}}{\pi}. \tag{H.12}$$

H.3.2. The idea

The singular values of A are hard to see directly, but its inverse is simple. Taking running sums is undone by taking differences, so A^{-1} has 1 on the diagonal and -1 just below it. Then $(A^{-1})^\top A^{-1}$ is the matrix of a second difference, $-v_{j-1} + 2v_j - v_{j+1}$, the same matrix that describes a vibrating string. Its eigenvectors are sine waves, and the two ends of the matrix decide which frequencies fit. For $T = 2$ the whole computation fits in a few lines:

$$A^{-1} = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}, \qquad (A^{-1})^\top A^{-1} = \begin{pmatrix} 2 & -1 \\ -1 & 1 \end{pmatrix}. \tag{H.13}$$

The eigenvalues of the second matrix are $(3 \pm \sqrt{5})/2$, that is 2.618 and 0.382. The formula proved below gives the same two numbers, as $4\sin^2(3\pi/10)$ and $4\sin^2(\pi/10)$. The singular values of A are one over their square roots, the golden ratio 1.618 and its inverse 0.618.

The duality is a statement about lining up. The singular value decomposition writes A as a sum of simple pieces, one for each singular value σ_k, and each piece maps one input direction v_k to one output direction u_k. A matrix U that stretches no vector can line up with each piece at best perfectly, so it collects at most σ_k from each, and $\|A\|_*$ in total. The Wallis bounds come from squeezing. Two sequences built from r_k^2, one decreasing and one increasing, close in on the same limit $1/\pi$, so each stays on its side. Finally, the near-optimality of the square root follows by putting the bounds side by side. The lower bound on every factorization and the upper bound on the square root both grow like $(\ln T)/\pi$, so their ratio tends to 1.

H.3.3. Derivation

The singular values.

1. **The inverse.** Write N for the matrix with ones just below the diagonal and zeros elsewhere, so that $Nx = (0, x_1, \dots, x_{T-1})$ shifts a vector down by one. Taking the differences $x_t - x_{t-1}$ of running sums gives back the x_t, so $A^{-1} = I - N$.
2. **The product.** $(A^{-1})^\top A^{-1} = (I - N^\top)(I - N) = I - N - N^\top + N^\top N$. The matrix $N^\top N$ is diagonal with entries $1, \dots, 1, 0$, because N maps the last basis vector to 0 and every other basis vector to the next one. So the product $M = (A^{-1})^\top A^{-1}$ is tridiagonal, with -1 next to the diagonal and 2 on the diagonal, except for a 1 in the bottom-right corner.
3. **Interior rows.** Try $v_j = \sin(j\omega)$ for $j = 1, \dots, T$. Row j of Mv, for $1 < j < T$, is $-v_{j-1} + 2v_j - v_{j+1}$. The identity $\sin((j-1)\omega) + \sin((j+1)\omega) = 2\cos\omega \sin(j\omega)$ turns this into $(2 - 2\cos\omega)v_j = 4\sin^2(\omega/2)\, v_j$. So $\nu = 4\sin^2(\omega/2)$ is the eigenvalue, if the first and last rows agree.
4. **The first row.** Row 1 is $2v_1 - v_2$, which is the interior formula with $v_0 = 0$. The sine satisfies $\sin(0) = 0$, so row 1 holds for every ω.
5. **The last row.** Row T is $-v_{T-1} + v_T$, which is the interior formula with $v_{T+1} = v_T$. So the sine must take the same value at T and $T+1$, which means it must be symmetric about $T + \frac{1}{2}$, a peak or a trough there. That requires $(T + \frac{1}{2})\omega$ to be an odd multiple of $\frac{\pi}{2}$, that is, $\omega_k = (2k-1)\pi/(2T+1)$. The values $k = 1, \dots, T$ give T different angles in $(0, \pi)$, where $\sin^2(\omega/2)$ increases, hence T different eigenvalues, which is all of them.
6. **Back to A.** The eigenvalues of $M = (AA^\top)^{-1}$ are the squares of the singular values of A^{-1}, which are one over the singular values of A. So $\sigma_k(A) = 1/\sqrt{4\sin^2(\omega_k/2)} = 1/(2\sin(\omega_k/2))$, which is Equation H.10, largest at $k = 1$.

The duality. Write the singular value decomposition $A = \sum_k \sigma_k u_k v_k^\top$, with orthonormal u_k and orthonormal v_k. Since $\operatorname{tr}(U^\top u_k v_k^\top) = v_k^\top U^\top u_k = \langle Uv_k, u_k \rangle$, linearity of the trace gives

$$\operatorname{tr}(U^\top A) = \sum_k \sigma_k \langle Uv_k, u_k \rangle \le \sum_k \sigma_k \|Uv_k\|\, \|u_k\| \le \sum_k \sigma_k = \|A\|_*. \tag{H.14}$$

The middle step is the Cauchy–Schwarz inequality, and the last uses $\|Uv_k\| \le 1$, because U stretches no vector and v_k has length 1. Equality holds for $U = \sum_k u_k v_k^\top$, which maps each v_k exactly onto u_k. This is Equation H.9.

The Wallis bounds.

1. **Two integrals.** Let $I_n = \int_0^{\pi/2} \sin^n x\,dx$. Integrating by parts gives $I_n = \frac{n-1}{n} I_{n-2}$, and $I_0 = \frac{\pi}{2}$, $I_1 = 1$. Unrolling the recursion, $I_{2k} = \frac{\pi}{2}\cdot\frac{1}{2}\cdot\frac{3}{4}\cdots\frac{2k-1}{2k} = \frac{\pi}{2} r_k$, because the square root's coefficients satisfy $r_k = r_{k-1}\frac{2k-1}{2k}$ (Equation 41.13). In the same way $I_{2k+1} = \frac{2}{3}\cdot\frac{4}{5}\cdots\frac{2k}{2k+1} = \frac{1}{(2k+1)r_k}$.
2. **The lower bound.** On $[0, \frac{\pi}{2}]$, $0 \le \sin x \le 1$, so higher powers are smaller and I_n decreases in n. From $I_{2k+1} \le I_{2k}$ you get $\frac{1}{(2k+1)r_k} \le \frac{\pi}{2} r_k$, that is, $r_k^2 \ge \frac{2}{\pi(2k+1)} = \frac{1}{\pi(k+\frac{1}{2})}$.
3. **A first upper bound, and the limit.** From $I_{2k} \le I_{2k-1} = \frac{1}{2kr_k}$, the same way, $r_k^2 \le \frac{1}{\pi k}$ for $k \ge 1$. Together with step 2, $k\,r_k^2$ is squeezed between $\frac{k}{\pi(k+1/2)}$ and $\frac{1}{\pi}$, so it tends to $\frac{1}{\pi}$. This is Wallis's product in disguise.
4. **Two monotone sequences.** Let $a_k = (k+\frac{1}{2})r_k^2$ and $b_k = (k+\frac{1}{4})r_k^2$. Both tend to $\frac{1}{\pi}$ by step 3. Using $r_{k+1}/r_k = (2k+1)/(2k+2)$,

$$\frac{a_{k+1}}{a_k} = \frac{(2k+3)(2k+1)}{(2k+2)^2} = \frac{4k^2+8k+3}{4k^2+8k+4} < 1, \qquad \frac{b_{k+1}}{b_k} = \frac{(4k+5)(2k+1)^2}{(4k+1)(2k+2)^2} = \frac{16k^3+36k^2+24k+5}{16k^3+36k^2+24k+4} > \tag{H.15}$$

   So a_k decreases and b_k increases.
5. **Conclude.** A decreasing sequence stays above its limit, so $a_k \ge \frac{1}{\pi}$, which is the lower bound of Equation H.11 again. An increasing sequence stays below its limit, so $b_k \le \frac{1}{\pi}$, which is the upper bound. Both hold for every $k \ge 0$.

How close the square root is to optimal.

1. **A floor under every factorization.** For $0 < x \le \frac{\pi}{2}$, $\sin x \le x$, so each singular value in Equation H.10 is at least $\frac{1}{\omega_k} = \frac{2T+1}{(2k-1)\pi}$. The function $\frac{1}{2x-1}$ decreases, so each term $\frac{1}{2k-1}$ is at least its integral from k to $k+1$. Adding these up, and then dividing the sum of the singular values by T,

$$\sum_{k=1}^{T} \frac{1}{2k-1} \ge \frac{1}{2}\ln(2T+1), \qquad \frac{\|A\|_*}{T} \ge \frac{2T+1}{2\pi T}\ln(2T+1) \ge \frac{\ln(2T+1)}{\pi}. \tag{H.16}$$

2. **A ceiling over the square root.** For the square root, $L = R$. Row t holds $r_{t-1}, \ldots, r_0$, so each row of L is no longer than the last one, whose squared length is $\sum_{k<T} r_k^2 = (\Delta_2 R)^2$ (step 4 of the proof of Theorem 41.3). So every $e_t \le (\Delta_2 R)^2$, and the RMSE, an average of the e_t^2 under a square root, is at most the largest e_t, which is $\sum_{k<T} r_k^2$, which Equation 41.14 bounds by $1 + (0.8 + \ln\frac{4T-3}{5})/\pi$. With the Wallis bounds proved above, that step of the chapter is now complete.
3. **The ratio tends to 1.** By Equation 41.9, the optimal RMSE lies between $\|A\|_*/T$ and the square root's RMSE, so all three lie in the band of Equation H.12. Both ends of the band are $(\ln T)/\pi$ plus a constant, so the square root's RMSE divided by the lower bound tends to 1 as T grows, like $1 + O(1/\ln T)$.

H.3.4. Checking it

The script compares Equation H.10 with a numerical singular value decomposition at $T = 256$, and the largest relative gap is 2.7×10^{-15}. Each $\sin(j\omega_k)$ vector satisfies $Mv = \nu v$ to within 1.4×10^{-14}. Figure H.5 draws the first three eigenvectors for $T = 16$. Each starts from 0 just

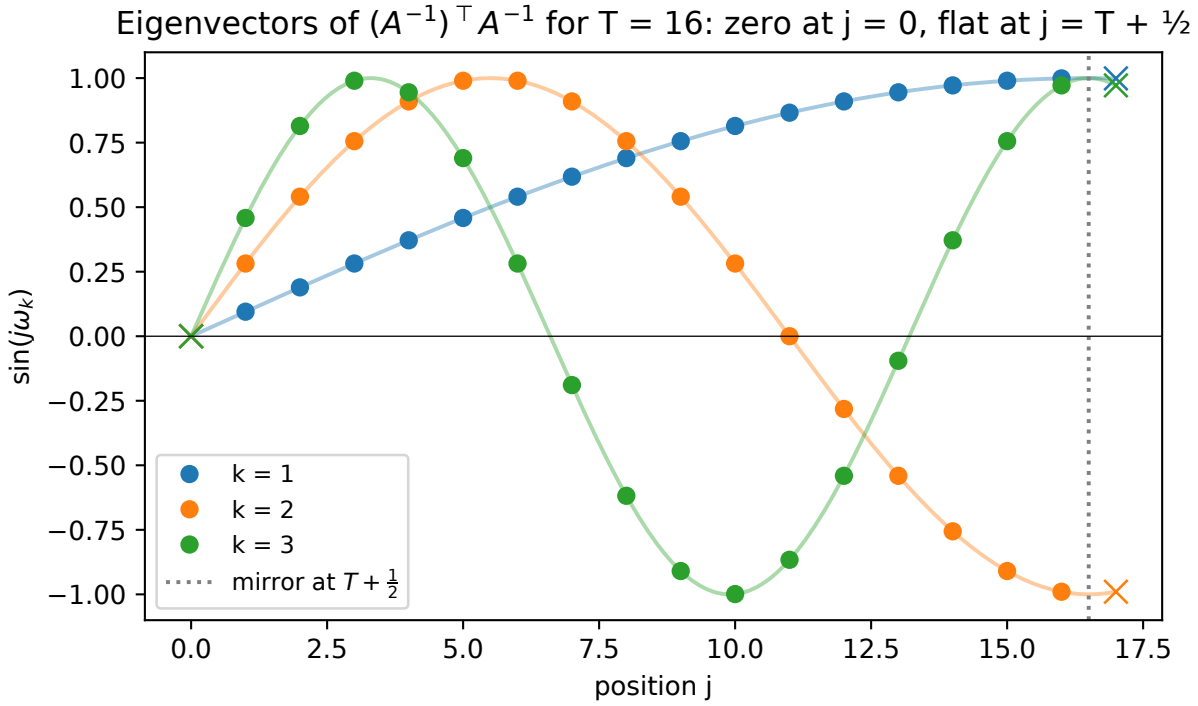


Figure H.5.: The first three eigenvectors $v_j = \sin(j\omega_k)$ of $(A^{-1})^\top A^{-1}$ for $T = 16$, as dots, with the sine curves they sample. Crosses mark the extra points $j = 0$, where every curve is 0, and $j = T + 1$, where every curve repeats its value at $j = T$, because it is symmetric about the dotted line at $T + \frac{1}{2}$.

before position 1 and is flat across the mirror at $T + \frac{1}{2}$, which is how steps 4 and 5 select the angles.

For the duality at $T = 16$, the trace norm is 26.0552. The largest $\operatorname{tr}(U^\top A)$ over 2,000 random matrices U, each scaled so that its largest singular value is 1, is 4.9448, well below it, and the aligned choice $U = \sum_k u_k v_k^\top$ attains 26.0552, the trace norm itself.

For the Wallis bounds, the script checks the two ratio inequalities of step 4 in exact integer arithmetic for every $k < 10^6$, and Equation H.11 itself in floating point for every $k < 10^4$. A floating-point check cannot go much further, because b_k approaches $\frac{1}{\pi}$ like $1/(32\pi k^2)$, which soon drowns in rounding error. At $k = 9999$, a_k is still above $\frac{1}{\pi}$ by 8.0×10^{-6} and b_k below it by 9.9×10^{-11}. Figure H.6 shows the two sequences closing in on $\frac{1}{\pi}$ from either side.

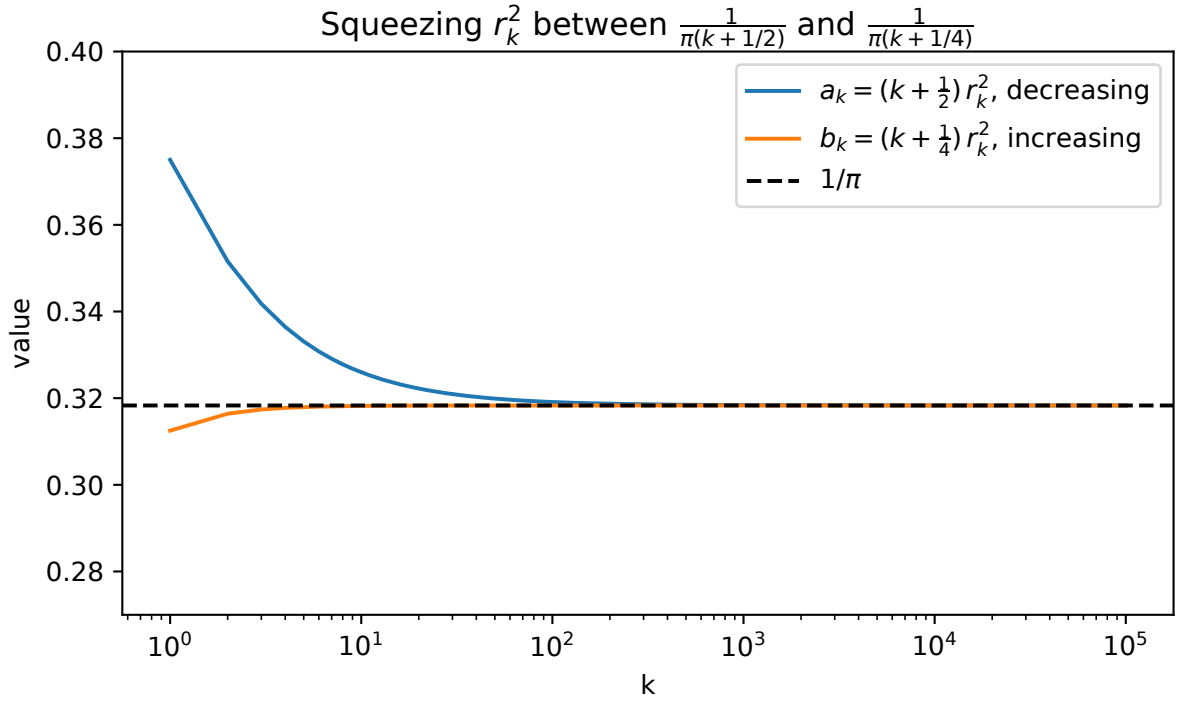


Figure H.6.: The sequences $a_k = (k + \frac{1}{2})r_k^2$ and $b_k = (k + \frac{1}{4})r_k^2$ against k (log scale). The first decreases and the second increases, and both tend to $1/\pi$ (dashed), which squeezes r_k^2 between $\frac{1}{\pi(k+1/2)}$ and $\frac{1}{\pi(k+1/4)}$.

Table H.4 gives the band of Equation H.12 and the two curves inside it. The chapter's "within 9% and 6%" are the ratios 1.089 and 1.056. The ratio keeps falling slowly, to 1.040 at $T = 10^6$, as the $1 + O(1/\ln T)$ of step 3 predicts. The proved band is wide. At $T = 100$ the ceiling divided by the floor is 1.568, so the band only guarantees a ratio below that, while the actual ratio is 1.089, because both $\sin x \le x$ and the bound of each row by the last one give away a constant. Figure H.7 shows the band and both curves from $T = 10$ to 10^6. Every factorization has its RMSE above the blue curve, and the optimal one has its RMSE between the blue and the orange curve.

Table H.4.: The lower bound on every factorization and the square root's RMSE, per unit of privacy, with the proved floor and ceiling of Equation H.12.

T	Floor $\frac{\ln(2T+1)}{\pi}$	Lower bound $\frac{\|A\|_*}{T}$	Square root RMSE	Ratio	Ceiling	$\frac{\ln T}{\pi}$
100	1.688	2.178	2.371	1.089	2.647	1.466
10,000	3.152	3.634	3.836	1.056	4.115	2.932
1,000,000	4.618	5.100	5.302	1.040	5.581	4.398

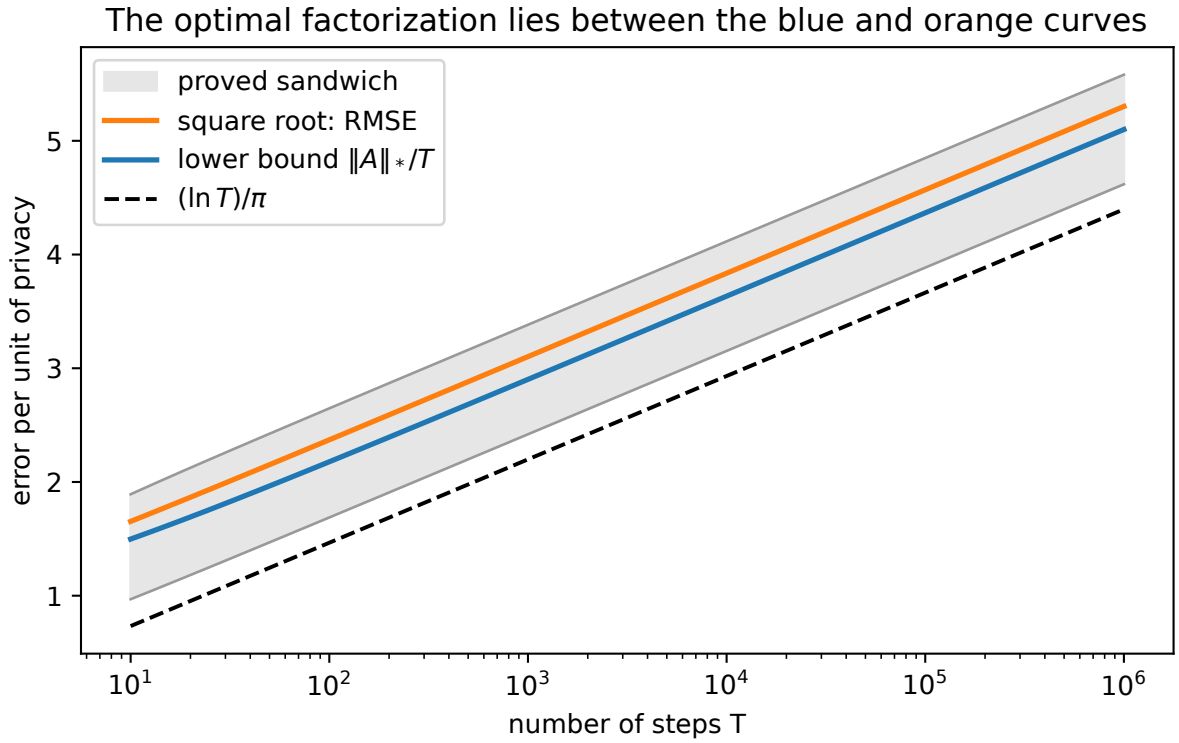


Figure H.7.: The trace-norm lower bound $\|A\|_*/T$ (blue) and the square root's RMSE (orange) against the number of steps T, per unit of privacy, with $(\ln T)/\pi$ dashed. The grey band is the proved sandwich of Equation H.12. The optimal RMSE lies between the blue and orange curves, which approach each other in ratio.

H.3.5. What it does not cover

The chapter also quotes the finer approximation $\sum_{k<T} r_k^2 \approx (\ln T + \gamma + 4\ln 2)/\pi$, with γ Euler's constant. It follows from a longer expansion of r_k^2 in powers of $1/k$, and it stays unproved here, as in the chapter. The ratio of step 3 tends to 1 only slowly, so the comparison says that the square root is nearly optimal for the RMSE, not that it is optimal at any finite T. The next section finds the optimum.

H.4. The optimal factorization for the root-mean-square error

H.4.1. What the chapter uses

In Theorem 41.4 and the text around it, the chapter looks for the factorization $A = LR$ with the smallest RMSE. Scaling the encoder R up and the decoder L down by the same factor changes nothing. So the chapter fixes $\Delta_2 R \le 1$, which means that every column of R has length at most 1. It then minimizes the **Frobenius norm** $\|L\|_F$, the square root of the sum of the squared entries of L, and the RMSE per unit of privacy is $\|L\|_F/\sqrt{T}$. The encoder enters only through the matrix $X = R^\top R$ of inner products of its columns, whose diagonal holds the squared column lengths. The chapter states that the problem is

$$\text{minimize } \operatorname{tr}(A^\top A X^{-1}) \quad \text{over positive definite } X \text{ with } X_{ii} \le 1 \text{ for every } i, \tag{H.17}$$

that it is convex, and that its solution is

$$X = V^{-1/2}\big(V^{1/2}A^\top A V^{1/2}\big)^{1/2}V^{-1/2}, \qquad v = \operatorname{diag}\Big(\big(V^{1/2}A^\top A V^{1/2}\big)^{1/2}\Big), \tag{H.18}$$

with V the diagonal matrix of the vector v, so that v is a fixed point of the map on the right. It cites Denisov et al. (2022, Theorem 3.2) for this, sketches the Lagrangian step and iterates the map. A matrix is **positive definite** when it is symmetric and $x^\top X x > 0$ for every nonzero x, and $Y^{1/2}$ is the positive definite square root of such a Y.

H.4.2. The idea

There are four steps. First, once the encoder R is chosen, the best decoder is forced. It is the one that undoes R with no wasted motion, and its size depends on R only through $X = R^\top R$. That turns a problem about two matrices into one about X. Second, the objective is convex in X: along any straight line through the positive definite matrices it bends upward, in the same way that $1/x$ does for positive x. Third, at the optimum the gradient of the objective must be balanced by a price κ_i on each column-length constraint, and solving that balance gives Equation H.18. Fourth, convexity makes any balanced point the global minimum.

The smallest case shows the answer. At $T = 2$, $A^\top A$ has rows $(2, 1)$ and $(1, 1)$. A feasible X with both columns of length 1 has rows $(1, \rho)$ and $(\rho, 1)$, and the objective is $(3 - 2\rho)/(1 - \rho^2)$. Setting its derivative to zero gives $\rho^2 - 3\rho + 1 = 0$, so $\rho = (3 - \sqrt{5})/2 = 0.382$ and the minimum is 2.618, the golden ratio squared. Table H.5 compares it with the other factorizations at $T = 2$.

Table H.5.: The four quantities for $T = 2$, per unit of privacy. The RMSE is $\Delta_2 R\|L\|_F/\sqrt{T}$.

Factorization	$(\Delta_2 R)^2$	$\|L\|_F^2$	RMSE
Independent noise, $R = I$	1	3	1.2247
Square root, $r = (1, \frac{1}{2})$	1.25	2.25	1.1859
Optimum, $\rho = 0.382$	1	2.618	1.1441
Lower bound $\|A\|_*/T = \sqrt{5}/2$			1.1180

H.4.3. Derivation

Write $B = A^\top A$. It is positive definite because A is invertible.

1. **The best decoder for a given encoder.** The encoder R has some number of rows and T columns. The equation $LR = A$ can only hold if R has rank T, because A has rank T, and then $X = R^\top R$ is positive definite. Let $R^+ = X^{-1}R^\top$, so that $R^+R = I$, and let $\Pi = RR^+$, the orthogonal projection onto the column space of R. Any decoder with $LR = A$ satisfies $L\Pi = LRR^+ = AR^+$. Split $L = L\Pi + L(I - \Pi)$. Each row of $L\Pi$ lies in the column space of R and each row of $L(I - \Pi)$ is orthogonal to it, so the squared lengths add, and $\|L\|_F^2 = \|AR^+\|_F^2 + \|L(I - \Pi)\|_F^2 \geq \|AR^+\|_F^2$, with equality for $L = AR^+$. And

$$\|AR^+\|_F^2 = \operatorname{tr}\left(AR^+(R^+)^\top A^\top\right) = \operatorname{tr}\left(AX^{-1}A^\top\right) = \operatorname{tr}(BX^{-1}), \tag{H.19}$$

   because $R^+(R^+)^\top = X^{-1}R^\top RX^{-1} = X^{-1}$, and the trace does not change when a product is rotated. So the problem over (L, R) is Equation H.17 over X.
2. **The objective is convex.** Take positive definite X, a symmetric direction H, and $X(t) = X + tH$. Differentiating $X(t)X(t)^{-1} = I$ gives $\frac{d}{dt}X^{-1} = -X^{-1}HX^{-1}$, and differentiating again gives $2X^{-1}HX^{-1}HX^{-1}$. So the second derivative of $f(t) = \operatorname{tr}(BX(t)^{-1})$ at $t = 0$ is

$$f''(0) = 2\operatorname{tr}\left(X^{-1}BX^{-1} \cdot HX^{-1}H\right). \tag{H.20}$$

   Both $X^{-1}BX^{-1}$ and $HX^{-1}H$ are positive semidefinite, and the trace of a product of two positive semidefinite matrices is at least 0, because $\operatorname{tr}(GH) = \operatorname{tr}(G^{1/2}HG^{1/2})$ is a sum of non-negative diagonal entries. So f bends upward along every line. The positive definite matrices form a convex set, and the constraints $X_{ii} \leq 1$ are linear, so the problem is convex.
3. **The balance condition.** Give each constraint a price $\kappa_i \geq 0$ and let K be the diagonal matrix of the prices. The gradient of $\operatorname{tr}(BX^{-1})$ with respect to X is $-X^{-1}BX^{-1}$, by step 2, and that of $\sum_i \kappa_i X_{ii}$ is K. At an optimum such prices exist, because the constraints are linear, and the two gradients balance:

$$X^{-1}BX^{-1} = K, \qquad \text{that is,} \qquad XKX = B. \tag{H.21}$$

   Since B is positive definite, so is $K = X^{-1}BX^{-1}$, so every price is positive. The optimality conditions also require each price times the slack of its constraint, $\kappa_i(1 - X_{ii})$, to be 0, which is called complementary slackness. So a positive price forces $X_{ii} = 1$, for every i. All columns of the optimal encoder have length exactly 1.
4. **Solving the balance.** Multiply Equation H.21 by $K^{1/2}$ on both sides: $(K^{1/2}XK^{1/2})^2 = K^{1/2}BK^{1/2}$. The matrix $K^{1/2}XK^{1/2}$ is positive definite, and a positive definite matrix has only one positive definite square root, so $K^{1/2}XK^{1/2} = (K^{1/2}BK^{1/2})^{1/2}$, which is the first half of Equation H.18 with $V = K$. Its diagonal entries read $\kappa_i X_{ii} = \left[(K^{1/2}BK^{1/2})^{1/2}\right]_{ii}$, and with $X_{ii} = 1$ this is the fixed point $v = \operatorname{diag}\left((V^{1/2}BV^{1/2})^{1/2}\right)$, the second half.
5. **A balanced point is the minimum.** A convex function lies above its tangent, so for any feasible Y,

$$\operatorname{tr}(BY^{-1}) \geq \operatorname{tr}(BX^{-1}) - \operatorname{tr}\left(X^{-1}BX^{-1}(Y - X)\right) = \operatorname{tr}(BX^{-1}) - \sum_i \kappa_i(Y_{ii} - 1) \geq \operatorname{tr}(BX^{-1}), \tag{H.22}$$

   using Equation H.21, $X_{ii} = 1$, $Y_{ii} \leq 1$ and $\kappa_i > 0$. So a fixed point of Equation H.18 gives the optimum.

6. **The optimum exists, and its value is the sum of the prices.** The feasible matrices have entries in $[-1, 1]$, since $|X_{ij}| \le \sqrt{X_{ii}X_{jj}}$, and the objective grows without bound near a singular X, so the minimum is attained at a positive definite matrix, where step 3 applies. Its value follows from Equation H.21, which gives $BX^{-1} = XK$:

$$\|L\|_F^2 = \operatorname{tr}(BX^{-1}) = \operatorname{tr}(XK) = \sum_i v_i, \qquad \text{RMSE} = \sqrt{\frac{1}{T}\sum_i v_i}. \tag{H.23}$$

7. **A lower-triangular encoder.** Any R with $R^\top R = X$ gives the same RMSE, and streaming needs a lower-triangular one. Let J be the matrix that reverses the order of coordinates. The Cholesky factorization writes the positive definite matrix JXJ as $GG^\top$ with G lower-triangular. Then $R = JG^\top J$ is lower-triangular, because reversing both rows and columns turns the upper-triangular $G^\top$ into a lower-triangular matrix, and $R^\top R = JGJJG^\top J = J(JXJ)J = X$. The decoder $L = AR^{-1}$ is lower-triangular too.

That the iteration $v \leftarrow \operatorname{diag}\left((V^{1/2}BV^{1/2})^{1/2}\right)$ converges to the fixed point is shown by Denisov et al. (2022), and it stays cited. The script observes it.

H.4.4. Checking it

The script runs the iteration from $v = \mathbf{1}$ with the chapter's stopping rule, a relative change below 10^{-10}. It reproduces the chapter's counts, 155 iterations at $T = 100$ and 193 at $T = 256$, and the RMSE 2.2355 and 2.5244, against 2.3707 and 2.6688 for the square root. At $T = 64$ it checks every step. The balance condition Equation H.21 holds to 5.5×10^{-13}, the diagonal of X equals 1 to 8.4×10^{-11}, the encoder has no nonzero entry above the diagonal, and $LR = A$ to 6.7×10^{-16}. The RMSE computed from L, 2.0999, equals $\sqrt{\sum_i v_i/T}$ as step 6 says. Moving X in 200 random feasible directions never lowers the objective: the smallest increase is 3.4857.

Figure H.8 shows the iteration and what it buys. On the left, the quantity $\max_i |X_{ii} - 1|$ for the current v falls geometrically, and the number of iterations grows slowly with T. On the right, the square root spends its privacy unevenly. Its first column is the longest and fixes the sensitivity, and its later columns are shorter, so the noise that hides them is larger than they need. The optimum makes every column exactly as long as the first, as step 3 requires.

H.5. Streaming: triangular factors and adaptive participations

H.5.1. What the chapter uses

Two facts in DP-FTRL and matrix factorization concern streams. There the mechanism must release its estimate $\hat{S}_t$ of the running sum at step t, and the gradients are chosen after seeing earlier outputs. Recall the setting. Each step's gradient sum is a vector of d numbers, one per model weight, so x and the noise z are matrices with one row per step. One record adds a vector of length at most the clipping norm C to each step it takes part in, and the noise has standard deviation sC, where s is the noise scale. Under "Streaming needs lower-triangular factors" in Section 41.2, the chapter says that for any factorization there is a lower-triangular one with the same distribution of the release $\hat{S} = L(Rx + z)$, citing Denisov et al. (2022, Proposition 2.2).

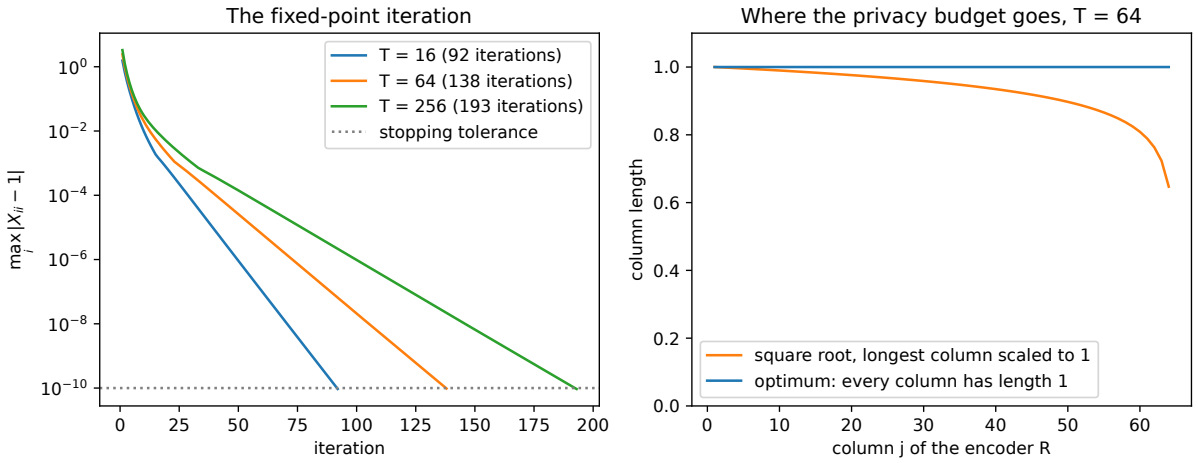


Figure H.8.: Left: the gap $\max_i |X_{ii} - 1|$ of the fixed-point iteration against the iteration number, for $T = 16$, 64 and 256 (log scale), down to the stopping tolerance 10^{-10}. Right: the column lengths of the encoder R for $T = 64$, for the square root scaled so its longest column has length 1, and for the optimum, where every column has length 1.

Precisely, if $A = LR$ with L of size $T \times m$ and R of size $m \times T$, there are lower-triangular $T \times T$ matrices L' and R' with

$$L'R' = A, \qquad L'L'^{\top} = LL^{\top}, \qquad \Delta_2 R' \le \Delta_2 R. \tag{H.24}$$

Here $\Delta_2 R$ is the sensitivity when each record takes part in one step. When a record takes part at several steps $\tau_1, \dots, \tau_k$, contributing $g_1, \dots, g_k$, the encoder's output changes by $\sum_i r_{\tau_i} g_i^{\top}$, where r_τ is column τ of R. The sensitivity of Equation 41.21 is the largest Frobenius norm of that change, over the allowed patterns and over all g_i of length at most C, divided by C. The same inequality holds for it.

Step 3 of the proof of Theorem 41.1 proves that a record that takes part in one step stays μ-Gaussian differential privacy (GDP) when the inputs are chosen adaptively. In step 3 of the proof of Theorem 41.5, a record takes part at several steps, and its later contributions g_i may depend on outputs it has already influenced. The chapter there says that such adaptive choices cannot do worse than the largest change over fixed g_i, and it cites Denisov et al. (2022, Theorem 2.1). This section proves that claim for Gaussian noise. With a lower-triangular R, a fixed participation pattern, and the sensitivity of Equation 41.21, the release is μ-GDP with $\mu = \Delta_2 R / s$, however the contributions are chosen.

Recall what μ-GDP means. The neighbouring datasets differ by adding or removing one record: D is the dataset without the record and $D' = D \cup \{x\}$ the dataset with it. Let P be the distribution of all the outputs on D, when the record is absent, and Q their distribution on D', when it is present. The release is μ-GDP when telling P from Q is no easier than telling $\mathcal{N}(0,1)$ from $\mathcal{N}(\mu,1)$. The best trade-off between the two errors of a test for that normal pair is a curve called G_μ. By the Neyman–Pearson lemma, how easy that is depends only on the **privacy loss** $L_{D,D'} = \ln(p/q)$, the log of the ratio of the two densities at the output, written with its pair of datasets because L alone is the decoder on this page, and on its distribution under P and under Q (f-DP and Gaussian DP). For the normal pair the privacy loss is normal, $\mathcal{N}(\mu^2/2, \mu^2)$ under P.

H.5.2. The idea

For the triangular factors, the release depends on the decoder L only through the covariance $LL^\top$ of its noise Lz. Running Gram–Schmidt on the rows of L, in order, writes each row as a combination of the first few orthonormal directions, which is a lower-triangular matrix of coefficients. The orthonormal directions can then be moved into the encoder, where they cannot lengthen any column.

For adaptivity, release the rows of $Rx + z$ one at a time. Given what has been released so far, each new row is a Gaussian mechanism, and the record shifts its mean by some amount μ_j, measured in units of the noise. So the privacy loss of that row is a normal variable with mean $\mu_j^2/2$ and variance μ_j^2. The shifts depend on the past, because the record's later gradients do, but their squares always add up to at most μ^2. If they added up to exactly μ^2 on every path, the total privacy loss would be exactly $\mathcal{N}(\mu^2/2, \mu^2)$, the loss of a single Gaussian mechanism. To arrange that, imagine one extra output that tops the sum up to μ^2. Throwing that output away afterwards is post-processing, which cannot make privacy worse.

H.5.3. Derivation

Lower-triangular factors.

1. **The decoder has full rank.** A has rank T and $A = LR$, so L has rank T and $m \geq T$.
2. **Gram–Schmidt on the rows.** Orthonormalize the rows $\ell_1, \dots, \ell_T$ of L in order, giving orthonormal rows $o_1, \dots, o_T$, the rows of a $T \times m$ matrix O with $OO^\top = I$. By construction ℓ_t is a combination of $o_1, \dots, o_t$ only, so $L = L'O$ with L' lower-triangular, and invertible because L has rank T.
3. **The new encoder.** Set $R' = OR$. Then $L'R' = L'OR = LR = A$, and $R' = L'^{-1}A$ is lower-triangular as a product of lower-triangular matrices. The noise covariance is unchanged, because $L'L'^\top = L'OO^\top L'^\top = LL^\top$. So $L'z'$, with z' of size $T \times d$, has the same distribution as Lz, and the release has the same distribution.
4. **No more sensitivity.** A matrix with orthonormal rows does not lengthen vectors: $\|Ov\|^2 = \sum_t \langle o_t, v\rangle^2 \leq \|v\|^2$, by Bessel's inequality, because the o_t are orthonormal. Each column of R' is O times the matching column of R, so $\Delta_2 R' \leq \Delta_2 R$. For several participations, the change $\sum_i r_{\tau_i} g_i^\top$ becomes $O \sum_i r_{\tau_i} g_i^\top$, whose Frobenius norm is no larger for the same reason. This gives Equation H.24.

Adaptive participations. Take R lower-triangular of size $T \times T$, noise z with independent $\mathcal{N}(0, \varsigma^2)$ entries where $\varsigma = sC$, and release Y_j, row j of $Rx + z$, at step j. The record takes part at fixed steps $\tau_1 < \cdots < \tau_k$, and its contribution g_i at step τ_i may depend on $Y_1, \dots, Y_{\tau_i - 1}$. The other records' gradients at step t are functions of $Y_1, \dots, Y_{t-1}$, the same functions with and without the record.

1. **Each row is a Gaussian mechanism given the past.** Given $Y_1, \dots, Y_{j-1}$, the mean of Y_j is the same function of the past under P and Q, except that under Q it is shifted by $\Delta_j = \sum_i R_{j\tau_i} g_i^\top$. Only participations with $\tau_i \leq j$ contribute, because R is lower-triangular, and their g_i are already fixed by the past.

2. **The privacy loss adds up.** The density of all outputs is the product of the conditional densities, so the privacy loss is a sum $L_{D,D'} = \sum_j L_j$ of the privacy losses L_j of the single rows. For one row, the normal densities give $L_j = \frac{\|\Delta_j\|^2}{2\varsigma^2} - \frac{\langle \Delta_j, z_j \rangle}{\varsigma^2}$, with $z_j = Y_j$ minus its mean under P. Under P, given the past, z_j is normal with independent coordinates, so L_j is normal with mean $\mu_j^2/2$ and variance μ_j^2, where $\mu_j = \|\Delta_j\|/\varsigma$.
3. **The shifts are bounded on every path.** The squared shifts add up to $\sum_j \|\Delta_j\|^2 = \|\sum_i r_{\tau_i} g_i^\top\|_F^2$, the change in Rx. The g_i have length at most C, so this is at most $(\Delta_2 R)^2 C^2$ by the definition Equation 41.21, whatever values the g_i took. Dividing by ς^2, $\sum_j \mu_j^2 \le \mu^2$ with $\mu = \Delta_2 R/s$.
4. **Top up to exactly μ^2.** Add one more output Y_{T+1}, normal with variance 1, whose mean is 0 under P and $\mu_{T+1} = \sqrt{\mu^2 - \sum_{j \le T} \mu_j^2}$ under Q. Its shift depends on the past, which is allowed, because the argument below conditions on the past at every row. Now $\sum_j \mu_j^2 = \mu^2$ exactly on every path. The original release is the padded one with Y_{T+1} thrown away, and by post-processing it is at least as private.
5. **The total loss is exactly normal under P.** The tool is the moment generating function, the average of $e^{\lambda L_{D,D'}}$ for a real number λ. A normal variable with mean a and variance b has $\mathbb{E}[e^{\lambda N}] = e^{\lambda a + \lambda^2 b/2}$. So, given the past, step 2 gives

$$\mathbb{E}\big[e^{\lambda L_j}\big] = e^{c\mu_j^2}, \qquad c = \frac{\lambda + \lambda^2}{2}. \tag{H.25}$$

   In other words, the factor $e^{\lambda L_j - c\mu_j^2}$ has average 1 given the past, whatever μ_j the past chose. Multiply these factors over all rows, averaging the last one first, then the one before, and so on: each average is 1, so the product has average 1. The product is $e^{\lambda L_{D,D'} - c\sum_j \mu_j^2}$, and $\sum_j \mu_j^2 = \mu^2$ is a constant on every path, so

$$\mathbb{E}_P\big[e^{\lambda L_{D,D'}}\big] = e^{c\mu^2} = \exp\Big(\lambda \frac{\mu^2}{2} + \lambda^2 \frac{\mu^2}{2}\Big). \tag{H.26}$$

   This is the moment generating function of a normal variable with mean $\mu^2/2$ and variance μ^2. A moment generating function that is finite for every λ determines the distribution, so $L_{D,D'} \sim \mathcal{N}(\mu^2/2, \mu^2)$ under P. Here the padding matters. Without it, the constant μ^2 would be the path-dependent $\sum_j \mu_j^2$, and it could not be taken out of the average.
6. **So is the loss under Q, and the release is μ-GDP.** The distribution of $L_{D,D'}$ under Q follows from that under P, because $q = e^{-L_{D,D'}} p$: for any set S of values, $\Pr_Q[L_{D,D'} \in S] = \mathbb{E}_P\big[e^{-L_{D,D'}}\, \mathbf{1}[L_{D,D'} \in S]\big]$. The single Gaussian mechanism with ratio μ has the same law of $L_{D,D'}$ under its P, hence under its Q too. By Neyman–Pearson, the two pairs have the same trade-off curve G_μ. The padded release is therefore exactly μ-GDP, and the release itself, by step 4, is μ-GDP.

H.5.4. Checking it

For the triangular factors, the script starts from the binary tree of Section 41.3 at $T = 64$, whose encoder has one row for each of its 127 nodes. Gram–Schmidt on the rows of its decoder gives 64×64 lower-triangular factors with $L'R' = A$ and $L'L'^\top = LL^\top$, both exactly. The sensitivity stays 2.6458, which is $\sqrt{7}$, because the first column of the tree's encoder already lies in the span

of the decoder's rows. So 64 noisy numbers can replace the tree's 127 node sums, with the same distribution of every released prefix sum.

For adaptivity, the script simulates the square-root encoder at $T = 8$, one coordinate, with $C = 1$ and $s = \Delta_2 R$, so that $\mu = 1$. The record takes part at steps 1 and 5, with $g_1 = 1$ and $g_2 = \tanh(Y_4)$, a contribution chosen from an output the record has already influenced. Over a million runs under P, the padded privacy loss has mean 0.4996 and variance 0.9985, against $\mu^2/2 = 0.5$ and $\mu^2 = 1$. Without the padding the loss has mean 0.2681 and variance 0.5207, smaller, because $|g_2| < 1$ keeps the total shift below its worst case. Figure H.9 shows both histograms against the normal density.

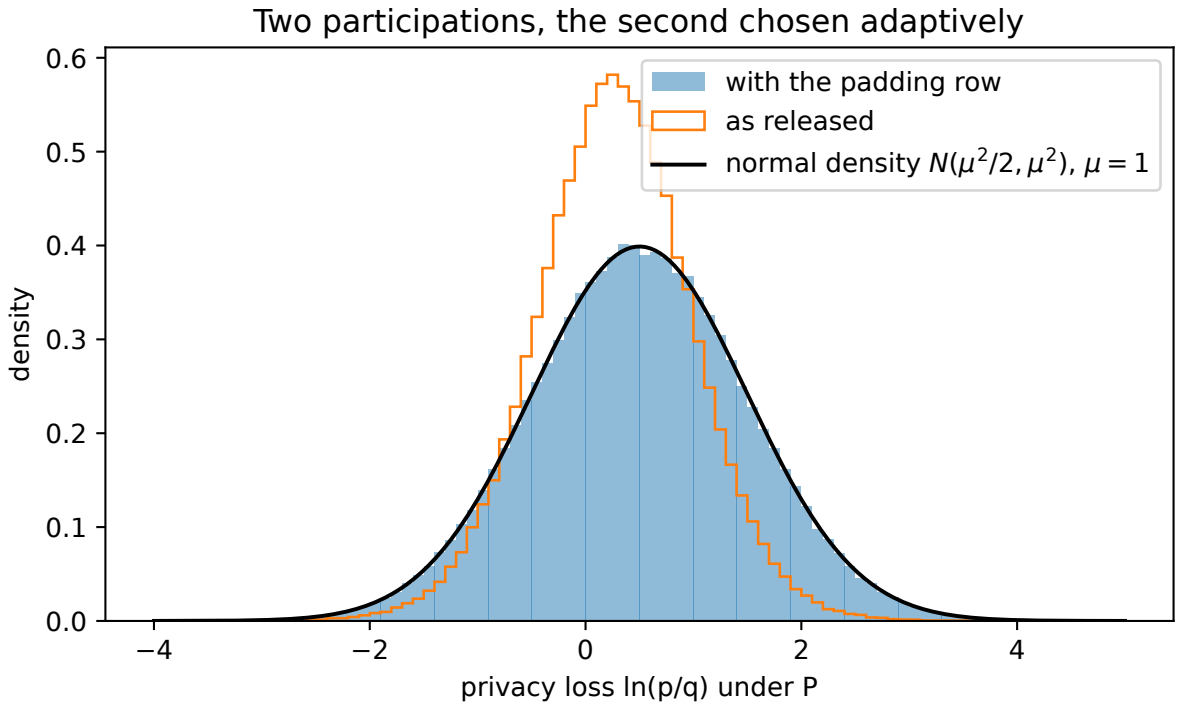


Figure H.9.: Histogram of the privacy loss $\ln(p/q)$ under P for a two-participation run of the square-root mechanism at $T = 8$, whose second contribution depends on an earlier output, with $\mu = 1$. Filled: with the padding row of step 4, which matches $\mathcal{N}(\mu^2/2, \mu^2)$ (black). Outline: the release itself, which is more concentrated, so more private.

H.5.5. What it does not cover

The adaptive proof assumes a square lower-triangular encoder released one row per step, a participation pattern fixed in advance, and Gaussian noise. The binary tree has more rows than steps. It fits after ordering its rows by the step at which each node is complete, and then the same argument applies with several rows released at some steps. Denisov et al. (2022, Theorem 2.1) prove a more general statement, which also covers (ε, δ) guarantees and non-Gaussian losses, and that version stays cited.

H.6. Code

The script `code/appendix-derivations/apx_matrices.py` checks each derivation numerically and draws the figures. For the Wishart mean it simulates Gaussian and 0/1 matrices, checks the regression identity of step 4 and the χ^2 average of step 6. For the smallest singular value it measures $s_{\min}$, the share of good queries and the number of queries the proof needs. For the prefix-sum matrix it checks the eigenvectors and singular values, the duality, the monotonicity of a_k and b_k for $k < 10^6$, and the sandwich up to $T = 10^6$. It also runs the fixed-point iteration, checks every condition of the optimum, and checks the triangular reduction and the adaptive privacy loss. Run it with

```
uv run python code/appendix-derivations/apx_matrices.py
```

Table H.6 collects the main numbers it prints with the default seed.

Table H.6.: The main numbers printed by the script.

Check	Result
m times average inverse diagonal, $n = 50$, $m = 100$: predicted, Gaussian, 0/1	2.041, 2.041, 2.000
$\mathbb{E}[1/\chi^2_{11}]$: Monte Carlo against 1/9	0.1112 against 0.1111
$s_{\min}(A)/\sqrt{m}$ at $n = 500$, $m = 2000$: simulated, asymptotic, proved	0.2558, 0.25, 0.0714
Queries the proof needs at $n = 100$, $\beta = 0.01$	115,627
Closed-form singular values against numerical, $T = 256$	2.7×10^{-15}
Square root against lower bound, $T = 100$ and 10^4	1.089 and 1.056
Optimal RMSE at $T = 100$ and 256, iterations	2.2355 (155), 2.5244 (193)
Adaptive privacy loss, padded: mean and variance	0.4996 and 0.9985

Every bound holds, and the simulations agree with the exact statements to within their noise. The two bounds that are far from tight are the smallest-singular-value floor, by a factor of three to six in $s_{\min}$ and much more in the number of queries, and the band of Equation H.12, which is wide at small T even though its ratio tends to 1.

H.7. References

- R. J. Muirhead, *Aspects of Multivariate Statistical Theory*, 1982. Chapter 3 covers the Wishart distribution and the mean of its inverse; the source the chapter cites.
- R. Vershynin, *High-Dimensional Probability*, 2018. Chapter 4 proves bounds on the singular values of random matrices with nets, the same plan as Section H.2 with sharper constants.

- Z. D. Bai and Y. Q. Yin, "Limit of the smallest eigenvalue of a large dimensional sample covariance matrix", *Annals of Probability*, 1993. The sharp constant $\sqrt{m} - \sqrt{n}$ for centred entries.
- S. Denisov, H. B. McMahan, J. Rush, A. Smith and A. Thakurta, 2022, on noisy stochastic gradient descent (SGD): "Improved differential privacy for SGD via optimal private linear operators on adaptive streams". The optimal factorization (Theorem 3.2), lower-triangular factors (Proposition 2.2) and adaptive streams (Theorem 2.1).
- H. Fichtenberger, M. Henzinger and J. Upadhyay, "Constant matters: fine-grained error bound on differentially private continual observation", 2023. The square-root factorization and the Wallis-type bounds on its coefficients.
- J. Dong, A. Roth and W. Su, "Gaussian differential privacy", 2022. Gaussian differential privacy, the Neyman–Pearson view of privacy and composition.

I. Pseudorandom functions and learning with errors

This appendix proves two results that Cryptography basics states without a full proof. The first is that the GGM tree, named after Goldreich, Goldwasser and Micali, is a pseudorandom function (PRF) whenever the pseudorandom generator (PRG) it is built from is secure. The second is about learning with errors (LWE), a problem in which a secret vector is hidden behind noisy linear equations. When the arithmetic is done modulo a prime, an algorithm that can only *recognize* such equations can be turned into one that *finds* the secret. Both proofs are reductions: each turns an algorithm that solves one problem into an algorithm that solves another. Both also rest on the same move. The new algorithm takes the input it must judge, called its challenge, and uses it to build exactly the situation the old algorithm expects to see. So the heart of each proof is a check that two distributions are identical. Table I.1 lists where each result comes from.

Table I.1.: The two derivations in this appendix and the chapter section that states each one.

Derivation	Where the chapter states it	What the chapter gives
Section I.1: the GGM tree is a PRF, with advantage at most nQ times the PRG's	Section 8.5.1	The theorem with its proof cited to Goldreich, and the idea in one paragraph
Section I.2: search LWE reduces to decision LWE for prime p polynomial in d	Definition 8.12	One sentence describing Regev's idea

The notation is the chapter's, with two letters changed. The security parameter λ is the length of keys and seeds in bits, and U_k stands for the uniform distribution on strings of k bits. An **adversary** is an algorithm that tries to break a scheme, usually by asking it questions called queries; it is written $\mathcal{A}$, and a second adversary built from it in a reduction is written $\mathcal{B}$. The number of queries Q and a bound ν on an advantage get letters of their own. Cryptography texts often write these q and ε, but in this book q is a sampling rate and ε a privacy parameter. The modulus in learning with errors is p.

A **distinguisher** receives input from one of two distributions, its two "worlds", and outputs a bit. Its **advantage** is the absolute difference between how often it outputs 1 in the two worlds. A procedure is **efficient** if its running time grows at most polynomially in λ, and a quantity is **negligible** if it shrinks faster than one over any polynomial in λ.

I.1. The GGM tree is a pseudorandom function

I.1.1. What the chapter uses

A pseudorandom generator stretches a short random seed into a longer string that efficient algorithms cannot tell from random. Here the generator doubles the length: $G\colon \{0,1\}^{\lambda} \to \{0,1\}^{2\lambda}$. Write its output as a left half $G_0(s)$ followed by a right half $G_1(s)$, each as long as the seed, so that $G(s) = (G_0(s), G_1(s))$. A PRG distinguisher $\mathcal{B}$ receives one 2λ-bit string and outputs a bit, and its advantage is

$$\mathrm{Adv}_{\mathrm{PRG}}(\mathcal{B}) = \big|\Pr[\mathcal{B}(G(U_\lambda)) = 1] - \Pr[\mathcal{B}(U_{2\lambda}) = 1]\big|. \tag{I.1}$$

The GGM tree turns G into a keyed function on n-bit inputs. Picture a binary tree of depth n in which every node carries a λ-bit **label**. The root's label is the key k. A node's left child gets the left half G_0 of G applied to the node's label, and its right child gets the right half G_1. An input $x = x_1 x_2 \cdots x_n$ is then read as directions from the root to a leaf. At level i, the bit x_i says go left if it is 0 and right if it is 1, and the output is the label of the leaf you reach,

$$F_k(x) = G_{x_n}\big(\cdots G_{x_2}(G_{x_1}(k))\cdots\big). \tag{I.2}$$

A node is named by the bits that lead to it, so node 01 is the right child of the left child of the root, and its **depth** is the length of its name.

In the PRF game, a fair coin decides what sits inside a box. It is either F_k with a uniform key, or a truly random function f from n-bit inputs to λ-bit outputs, which gives each input an independent uniform output. An adversary $\mathcal{A}$ sends up to Q queries, each chosen after seeing the answers to the earlier ones, and then outputs a bit. The box that answers the queries is called an oracle, and $\mathcal{A}^{O}$ means $\mathcal{A}$ run with oracle O. The adversary's advantage is

$$\mathrm{Adv}_{\mathrm{PRF}}(\mathcal{A}) = \big|\Pr[\mathcal{A}^{F_k} = 1] - \Pr[\mathcal{A}^{f} = 1]\big|, \tag{I.3}$$

where the probabilities are also over the adversary's own random choices.

The result says that any such adversary can be turned into a PRG distinguisher whose advantage is smaller by exactly the factor nQ, and which is about as fast. Precisely, take any adversary $\mathcal{A}$ that makes at most Q queries. Then there is a PRG distinguisher $\mathcal{B}$ that runs $\mathcal{A}$ once and, apart from that, evaluates G fewer than nQ times, such that

$$\mathrm{Adv}_{\mathrm{PRF}}(\mathcal{A}) = nQ \cdot \mathrm{Adv}_{\mathrm{PRG}}(\mathcal{B}). \tag{I.4}$$

The running time matters because PRG security only limits fast distinguishers. So if no distinguisher with $\mathcal{B}$'s running time has PRG advantage above some ν, then no Q-query adversary has PRF advantage above $nQ\,\nu$, which is the chapter's $\mathrm{Adv}_{\mathrm{PRF}} \le nQ \cdot \mathrm{Adv}_{\mathrm{PRG}}$. The chapter gives a proof sketch and cites Goldreich, *Foundations of Cryptography*, Vol. 1, §3.6, for the full proof.

I.1.2. The idea

The proof walks from the real function to a random one in n steps, one level of the tree at a time. Each intermediate world is called a **hybrid**, because it is part real and part random. The j-th hybrid, H_j, gives every node at depth j a fresh uniform label and computes everything below it with G as usual. The first hybrid, H_0, is the real F_k, because the only node at depth 0 is the root and its uniform label is the key. The last, H_n, is a random function, because every leaf gets its own uniform label. Going from H_{j-1} to H_j changes a single thing. In H_{j-1}, the two children of a node at depth $j-1$ carry the two halves of G applied to a uniform label, while in H_j they carry two uniform labels. Telling these apart is the PRG's game of output against uniform string, played once for every node at depth $j-1$ that the adversary's queries pass through. Figure I.1 shows the four hybrids for $n = 3$ and three queries.

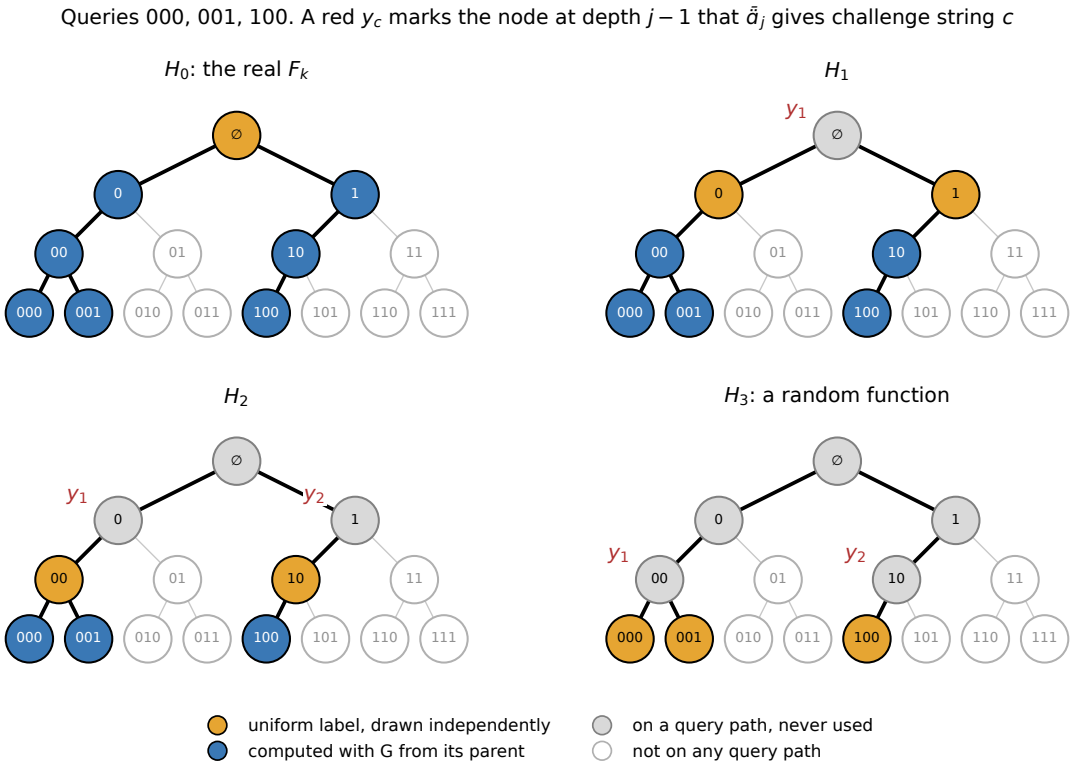


Figure I.1.: The GGM tree of depth 3 in the hybrids H_0 to H_3, for the queries 000, 001 and 100. Orange nodes get independent uniform labels, blue nodes are computed with G from their parent, and grey nodes lie on a query path but are never used. A red y_c marks the node at depth $j-1$ to which the reduction $\mathcal{B}_j$ assigns its c-th challenge string.

The subtle part is that the adversary chooses its queries as it goes. A reduction $\mathcal{B}_j$ that wants to use the adversary to break the PRG receives Q challenge strings, which are either all outputs of G on independent uniform seeds or all uniform. But it cannot know in advance which nodes at depth $j-1$ the queries will pass through. So it hands out its challenge strings like cloakroom tickets. The first time a query passes through a new node at depth $j-1$, that node gets the next unused string and keeps it for the rest of the game. Table I.2 follows $\mathcal{B}_2$ through the three queries of Figure I.1.

Table I.2.: How $\mathcal{B}_2$ assigns its challenge strings y_1, y_2, y_3 to the nodes at depth 1 as the queries arrive. The third string is never needed.

Query	Node at depth 1	First visit?	String used	Node at depth 2	Its label
000	0	yes	y_1	00	left half of y_1
001	0	no	y_1	00	left half of y_1
100	1	yes	y_2	10	left half of y_2

If each y_c is G applied to its own uniform seed, then the labels at depth 2 are halves of G applied to independent uniform labels at depth 1, which is H_1. If each y_c is uniform, the labels at depth 2 are uniform, which is H_2. The one thing left to check is that handing out values on first use gives the adversary the same distribution of answers as drawing them all in advance. It does, because a value nobody has looked at yet is still a fresh draw, and step 4 below proves this carefully.

I.1.3. Derivation

The proof has four parts. Steps 1 to 3 set up the hybrids and show that two neighbouring hybrids differ only in the distribution of one table. Steps 4 and 5 build the reduction $\mathcal{B}_j$, which judges Q strings at once, and show that it recreates either neighbour exactly. Steps 6 and 7 turn it into a distinguisher that judges a single string, and add up the levels. Step 8 restates the result for efficient adversaries.

1. **Define the hybrids.** For $j = 0, 1, \ldots, n$, hybrid H_j gives every node at depth j an independent uniform λ-bit label $L(x_1 \cdots x_j)$, all drawn before the game starts. It answers the query x by walking down from the depth-j node on its path:

$$H_j(x) = G_{x_n}(\cdots G_{x_{j+1}}(L(x_1 \cdots x_j)) \cdots). \tag{I.5}$$

   With $j = 0$ the only label is the root's, which is a uniform key, so H_0 is F_k. With $j = n$ the answer is the leaf's own label, independent and uniform for each input, so H_n is a random function. The table of 2^j labels can be huge, but that does not matter. The hybrids are thought experiments, and only the reduction has to be efficient.

2. **Split the advantage into levels.** Write $h_j = \Pr[\mathcal{A}^{H_j} = 1]$ for how often the adversary says 1 in hybrid H_j. In the sum $\sum_{j=1}^{n}(h_{j-1} - h_j)$, each intermediate h_j appears once with a plus sign and once with a minus sign, so the sum telescopes:

$$h_0 - h_n = \sum_{j=1}^{n}(h_{j-1} - h_j). \tag{I.6}$$

   The chapter states this telescoping, with absolute values, as Lemma 8.2. The left side is $\Pr[\mathcal{A}^{F_k} = 1] - \Pr[\mathcal{A}^{f} = 1]$, whose absolute value is $\mathrm{Adv}_{\mathrm{PRF}}(\mathcal{A})$.

3. **Write two neighbouring hybrids as one table.** Fix $j \geq 1$ and look at a node w at depth $j - 1$. In H_{j-1}, its two children $w0$ and $w1$ get the two halves of $G(L(w))$, so together they carry the string $G(L(w))$. Different nodes w have independent uniform labels, so these strings are independent, each distributed as $G(U_\lambda)$. In H_j, the children carry

two independent uniform labels, so together they carry a string distributed as $U_{2\lambda}$, again independently for each w. Below depth j both hybrids compute with G in the same way. So the two hybrids are one and the same world, except for the distribution P that the children's strings come from. In that world, a table T gives each node w at depth $j-1$ its own independent draw $T(w)$ from P. The query x is answered by taking the half of $T(x_1 \cdots x_{j-1})$ that bit x_j selects and walking down with G:

$$W_j^P(x) = G_{x_n}(\cdots G_{x_{j+1}}(\text{half } x_j \text{ of } T(x_1 \cdots x_{j-1}))\cdots). \tag{I.7}$$

With $P = G(U_\lambda)$ this world is H_{j-1}, and with $P = U_{2\lambda}$ it is H_j. For $j = 1$ the only node at depth 0 is the root, so the table has a single entry.

4. **Lazy sampling gives the same view.** There are two ways to fill the table T. In the *advance* version, every entry is drawn before the game starts. In the *lazy* version, each entry is drawn from P the first time a query needs it, and reused afterwards. The adversary's **view** is everything it sees: its own random choices (its coins) and all the answers it receives. The claim is that the view has the same distribution in both versions. Think of dice rolled earlier and hidden under cups: lifting a cup now shows a number that is exactly as random as rolling the die now, provided nobody peeked.

   The proof goes one query at a time. Call the **history** after i queries the adversary's coins together with the table entries used so far. Every answer, and therefore every next query, is a function of the history. If the next query needs an entry that was already used, both versions answer with that same entry, which is part of the history. If it needs a new entry, then in the advance version that entry was drawn independently of the other entries and of the adversary's coins, so given the history it is still distributed as P. In the lazy version it is a fresh draw from P, so the same holds. So in both versions, the history after $i+1$ queries is the history after i queries plus one entry with the same conditional distribution. The probability of a whole history is the product of these conditional probabilities, one per query (the chain rule), so the histories have the same distribution after every query. The same then holds for anything computed from them, including the adversary's output.

5. **Build $\mathcal{B}_j$, which judges Q strings at once.** The reduction $\mathcal{B}_j$ receives Q strings $y_1, \ldots, y_Q$ of 2λ bits each. They are drawn independently from one distribution P, which is either $G(U_\lambda)$ or $U_{2\lambda}$. It runs $\mathcal{A}$ and keeps a ticket table, which starts empty. When a query x arrives, $\mathcal{B}_j$ looks at the node $w = x_1 \cdots x_{j-1}$ at depth $j-1$ on its path, and if w has no ticket yet, gives it the next unused index c. It then answers with the half of y_c that x_j selects, walked down with G exactly as in step 3. At the end, $\mathcal{B}_j$ outputs whatever $\mathcal{A}$ outputs. Each query passes through one node at depth $j-1$, so at most Q nodes ever need a ticket and the strings never run out.

   Now compare this with step 4. The string y_c is first read when the c-th new node appears. Until then, everything $\mathcal{B}_j$ has done depends only on $y_1, \ldots, y_{c-1}$ and on $\mathcal{A}$'s coins, and y_c is independent of all of them. So $\mathcal{B}_j$ fills the table of step 3 lazily, in the sense of step 4, and $\mathcal{A}$'s view inside $\mathcal{B}_j$ is exactly its view in W_j^P. With step 3, this gives

$$\Pr\left[\mathcal{B}_j(G(U_\lambda)^Q) = 1\right] = h_{j-1}, \qquad \Pr\left[\mathcal{B}_j(U_{2\lambda}^Q) = 1\right] = h_j, \tag{I.8}$$

   where $G(U_\lambda)^Q$ means Q independent outputs of G on uniform seeds, and $U_{2\lambda}^Q$ means Q independent uniform strings.

6. **Go from Q strings to one.** The chapter's hybrid lemma for many samples does this, and here is its argument for this case. Write g_i for the probability that $\mathcal{B}_j$ outputs 1 on a list of Q independent strings whose first i are uniform and whose other $Q-i$ are outputs of G. The two ends are the cases of step 5: $g_0 = h_{j-1}$ (all from G) and $g_Q = h_j$ (all uniform). The mixed lists in between match no hybrid, and they do not need to. The one-string distinguisher $\mathcal{B}'_j$ takes a challenge string z and picks a position i uniformly from 1 to Q. It puts uniform strings before position i, puts z at position i, and after it puts outputs of G on seeds it draws itself, which is possible because G is public. Then it outputs $\mathcal{B}_j$'s answer on the whole list. If z comes from G, the list starts with $i-1$ uniform strings, so $\mathcal{B}'_j$ says 1 with probability g_{i-1}; if z is uniform, the probability is g_i. Averaging over i gives $\frac{1}{Q}\sum_{i=1}^{Q}(g_{i-1} - g_i)$, which telescopes as in step 2 to $\frac{1}{Q}(g_0 - g_Q)$:

$$\Pr[\mathcal{B}'_j(G(U_\lambda)) = 1] - \Pr[\mathcal{B}'_j(U_{2\lambda}) = 1] = \frac{1}{Q}\,(h_{j-1} - h_j). \tag{I.9}$$

7. **Pick the level at random and add up.** The final distinguisher $\mathcal{B}$ picks j uniformly from $\{1, \ldots, n\}$ and runs $\mathcal{B}'_j$. Averaging step 6 over j and using Equation I.6,

$$\Pr[\mathcal{B}(G(U_\lambda)) = 1] - \Pr[\mathcal{B}(U_{2\lambda}) = 1] = \frac{1}{n}\sum_{j=1}^{n}\frac{1}{Q}\,(h_{j-1} - h_j) = \frac{h_0 - h_n}{nQ}. \tag{I.10}$$

   Taking absolute values gives Equation I.4. As for the cost, $\mathcal{B}$ runs $\mathcal{A}$ once. It computes at most $Q-1$ outputs of G for the list, and at most $n-1$ evaluations of G per query to walk down the tree. That is at most $(Q-1) + Q(n-1) = nQ - 1$ evaluations, fewer than nQ.

8. **Conclude for efficient adversaries.** Suppose n and Q grow at most polynomially in λ. Then an efficient $\mathcal{A}$ gives an efficient $\mathcal{B}$. If $\mathcal{A}$'s advantage were not negligible, it would be at least λ^{-c} for some exponent c and infinitely many λ. Dividing by the polynomial nQ keeps it above one over a polynomial, so $\mathcal{B}$'s advantage $\mathrm{Adv}_{\mathrm{PRF}}(\mathcal{A})/(nQ)$ would not be negligible either, and G would not be a PRG. So if G is a PRG, the GGM tree is a PRF.

Adaptivity matters only in step 5. A reduction that fixed in advance which nodes get challenge strings would have to cover all 2^{j-1} nodes at depth $j-1$. That can be exponentially many strings, and the loss factor would grow with it, from Q to 2^{j-1}. The ticket rule lets the adversary's own queries decide where the Q strings go, and step 4 is why that changes nothing in what the adversary sees.

I.1.4. Checking it

The script runs the whole reduction on a toy with $\lambda = 8$, $n = 3$ and $Q = 3$. Its PRG is broken on purpose, so that every advantage is large enough to measure: $G(s) = (s, f(s))$ for a fixed random permutation f of the 8-bit strings. So the left half is the seed itself, and the right half is a public function of it. In the real F_k, every left step keeps the label and every right step applies f. So $F_k(000) = k$, $F_k(001) = f(k)$ and $F_k(100) = f(k)$, while $F_k(110) = f(f(k))$ because it takes two right steps.

The adversary uses these relations, and it chooses its third query adaptively so that the check also covers the subtle part of step 5. It asks 000 and then 001, and for its third query it asks 100 if the first answer starts with a 0 bit, and 110 otherwise. Check 1 asks whether the second

answer is f of the first. Check 2 asks whether the third answer is f of the first, after 100, or f applied twice to the first, after 110. The adversary flips a coin to choose one check, and outputs 1, meaning "random function", if that check fails. A uniform label passes a fixed equality only with probability 2^{-8}, so a check on labels that are independent fails with probability $1 - 2^{-8}$. Both checks pass in H_0, so $h_0 = 0$. In H_1 and H_2, the first two leaves still hang below one node and are computed from it with G, so check 1 passes. But check 2 compares labels from different branches of the tree, which are independent, so it fails with probability $1 - 2^{-8}$. That gives $h_1 = h_2 = \frac{1}{2}(1 - 2^{-8}) = 0.4980$. In H_3 both checks fail, so $h_3 = 1 - 2^{-8} = 0.9961$. Table I.3 compares these with simulation.

Table I.3.: How often the adversary says "random" in each hybrid, and in each reduction $\mathcal{B}_j$ fed outputs of the toy G or uniform strings (seed 0, 200,000 runs per entry).

j	Exact h_j	Simulated h_j	$\mathcal{B}_j$ on 3 outputs of G (should be h_{j-1})	$\mathcal{B}_j$ on 3 uniform strings (should be h_j)
0	0.0000	0.0000		
1	0.4980	0.4994	0.0000	0.5005
2	0.4980	0.4985	0.4979	0.4993
3	0.9961	0.9962	0.4983	0.9964

Each $\mathcal{B}_j$ lands on the hybrid that step 5 predicts, to within simulation noise of about 0.001, even though the adversary's third query depends on what it has seen. Figure I.2 draws the same comparison. The single-sample distinguisher $\mathcal{B}$ of step 7 says 1 with probability 0.5535 on outputs of G and 0.6641 on uniform strings. Its advantage is therefore 0.1105, against the predicted $0.9961/9 = 0.1107$. Multiplying back by $nQ = 9$ gives 0.995, which matches the adversary's PRF advantage $h_3 - h_0 = 0.9961$ as Equation I.4 says it should.

Since Equation I.4 is an equality, the factor nQ between the two advantages is lost inside $\mathcal{B}$ itself, in the averaging over levels and positions. The toy shows why. The adversary's rate jumps by 0.498 at levels 1 and 3 and does not move at level 2, but $\mathcal{B}$ spreads its effort evenly over all levels and all Q positions, most of which carry no signal. So the factor nQ comes from how the proof is organized, just like the factor for m samples in the chapter's hybrid lemma.

I.1.5. What it does not cover

The proof needs every input to have the same length n. With inputs of different lengths the GGM tree is not a PRF, because $F_k(0)$ is the label of the node that $F_k(00)$ hangs from. An adversary then checks whether $F_k(00) = G_0(F_k(0))$, which always holds for the real function and almost never for a random one. The proof also assumes that G is deterministic, because a node's label must give the same children every time a query passes through it. It assumes as well that G doubles its input exactly; a generator that stretches by only one bit can be applied repeatedly to its own output to reach double length. Finally, Equation I.4 compares advantages. To turn it into a concrete guarantee, such as "no attacker running for 2^{128} steps succeeds", you need a concrete bound on the advantage of PRG distinguishers with $\mathcal{B}$'s running time.

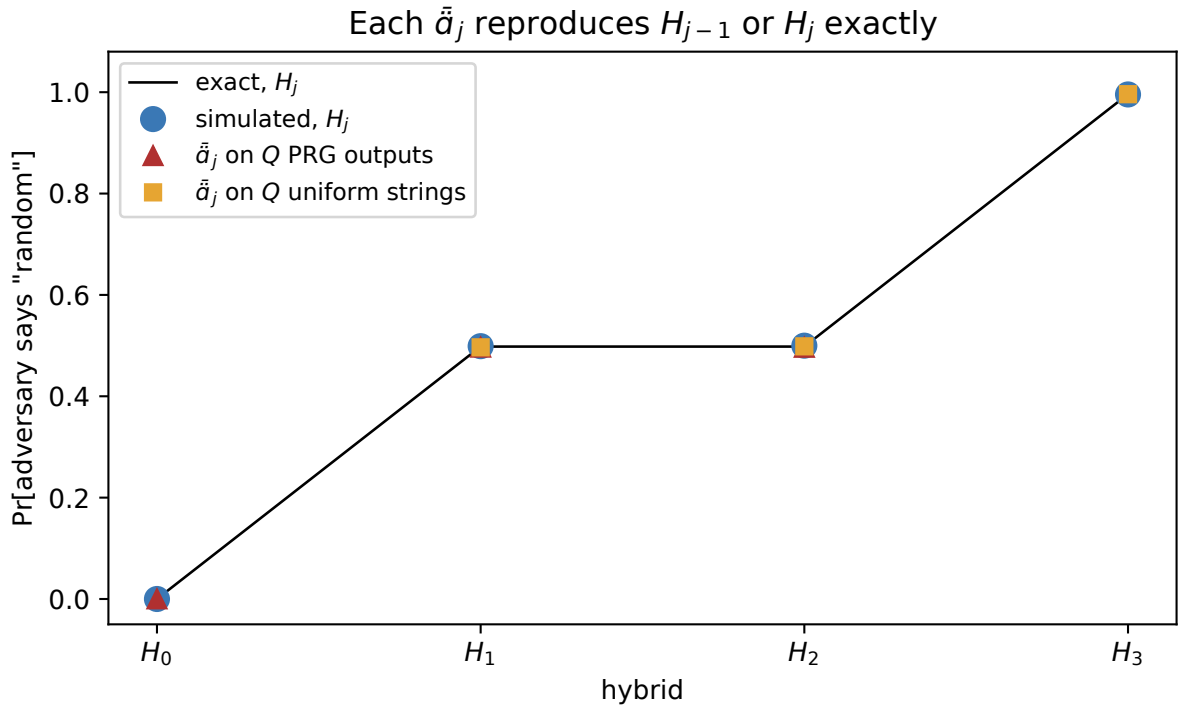


Figure I.2.: How often the toy adversary says "random" in the hybrids H_0 to H_3, exactly and by simulation, and in each reduction $\mathcal{B}_j$. Red triangles are $\mathcal{B}_j$ fed outputs of G, placed at H_{j-1}; orange squares are $\mathcal{B}_j$ fed uniform strings, placed at H_j. Every marker sits on the curve.

I.2. Search LWE reduces to decision LWE

I.2.1. What the chapter uses

Learning with errors hides a secret vector behind linear equations with small errors. Fix a dimension d, a modulus p and an error distribution χ on integers that are small compared with p. Write $\mathbb{Z}_p$ for the integers $0, \dots, p-1$ with arithmetic mod p, and $\mathbb{Z}_p^d$ for vectors of d such numbers. The secret s is a vector in $\mathbb{Z}_p^d$. An **LWE sample** is a pair made of a uniform random vector a in $\mathbb{Z}_p^d$ and a number b. To make b, draw an error e from χ and set

$$b = \langle a, s \rangle + e \bmod p, \qquad \langle a, s \rangle = \sum_{j=1}^{d} a_j s_j. \tag{I.11}$$

Every sample has its own fresh a and e, independent of the other samples, and the secret is the same in all of them.

Search LWE asks for s, given as many samples as the algorithm needs. **Decision LWE** asks whether m given pairs are LWE samples or uniform pairs in $\mathbb{Z}_p^d \times \mathbb{Z}_p$. The chapter shows that search is at least as hard as decision. For the converse it says only this: Regev, who introduced LWE, proves it for a prime p that is at most polynomial in d, by guessing the coordinates of s one at a time and testing each guess with the decision algorithm.

Here is that converse in a form this appendix proves. Assume that p is prime. Assume also that a decision algorithm $\mathcal{A}$, given m pairs, has advantage at least ν for every secret, where the probabilities are over the pairs and over $\mathcal{A}$'s own random choices:

$$\big|\Pr[\mathcal{A}(m \text{ LWE samples with secret } s) = 1] - \Pr[\mathcal{A}(m \text{ uniform pairs}) = 1]\big| \geq \nu \quad \text{for every } s \in \mathbb{Z}_p^d. \tag{I.12}$$

Then, for any failure probability β, there is a search algorithm that finds all of s with probability at least $1 - \beta$. It calls $\mathcal{A}$ at most $(dp+1)N$ times and uses $dpNm$ LWE samples, where the number of repetitions per test is

$$N = \left\lceil \frac{8}{\nu^2} \ln \frac{2(dp+1)}{\beta} \right\rceil. \tag{I.13}$$

The advantage ν is **noticeable** if it is at least one over some polynomial in d. In that case, and when p, m and $\mathcal{A}$'s running time are also polynomial in d, the search runs in polynomial time. The value of N comes from step 6 of the derivation.

I.2.2. The idea

The test for one guess works like this. Take an LWE sample and a guess g for the first coordinate s_1. Pick a random number r, add it to the first coordinate of a, and add rg to b. If the guess is right, the change to b is exactly the change to $\langle a, s \rangle$. So the new pair is again an LWE sample with the same secret and the same small error. If the guess is wrong, the error picks up the extra term $r(g - s_1)$. When p is prime, that term is a random multiple of a nonzero number, so it is uniform. It drowns the error, and together with the shifted first coordinate it makes the whole pair uniform. Feeding the new pairs to $\mathcal{A}$ therefore asks $\mathcal{A}$ "LWE or uniform?", and the true answer is "LWE" exactly when the guess is right. Repeating the test many times measures how

often $\mathcal{A}$ says 1, and comparing that rate with $\mathcal{A}$'s rate on uniform pairs, which the search draws itself, tells the right guess from the wrong ones. Trying all p guesses for each of the d coordinates recovers s.

Table I.4 makes the wrong-guess case concrete with $p = 5$, $s_1 = 3$ and error $e = 1$. After the change, the new b minus the inner product of the new a with s is $e + r(g - s_1) \bmod 5$. With the right guess $g = 3$ it stays at $e = 1$ for every r. With the wrong guess $g = 1$ it becomes $1 - 2r \bmod 5$, which takes each of the five values exactly once as r runs through $0, \ldots, 4$. Since r is uniform, the result is uniform.

Table I.4.: The value $b - \langle a, s \rangle$ after re-randomizing with guess g, for $p = 5$, $s_1 = 3$, $e = 1$ and each value of r. The wrong guess hits every element of $\mathbb{Z}_5$ once.

r	0	1	2	3	4
Right guess $g = 3$: $1 + 0 \cdot r$	1	1	1	1	1
Wrong guess $g = 1$: $1 - 2r \bmod 5$	1	4	2	0	3

I.2.3. Derivation

Fix a coordinate ℓ and a guess $g \in \mathbb{Z}_p$. Steps 1 to 4 show what one re-randomized sample looks like, and steps 5 to 8 turn this into a search with a failure probability you choose.

1. **Re-randomize.** Take an LWE sample (a, b) with $b = \langle a, s \rangle + e$. Draw a fresh r uniform in $\mathbb{Z}_p$, independent of everything else. Let a' be a with r added to its ℓ-th coordinate, and let $b' = b + rg \bmod p$. Adding r to a_ℓ adds rs_ℓ to the inner product, so $\langle a', s \rangle = \langle a, s \rangle + rs_\ell$. Subtracting this from b' leaves the **residual**

$$b' - \langle a', s \rangle = e + r(g - s_\ell) \bmod p. \tag{I.14}$$

2. **The new vector is still uniform.** For each fixed r, adding r to one coordinate is a bijection of $\mathbb{Z}_p^d$. By Lemma 8.3, a bijection maps the uniform distribution to itself. Since a is uniform and independent of r and e, the vector a' is uniform and independent of e.

3. **A right guess gives an LWE sample.** If $g = s_\ell$, the residual Equation I.14 is just e, so $b' = \langle a', s \rangle + e$. With step 2, a' is uniform and e is an independent draw from χ, so (a', b') is an LWE sample with the same secret s.

4. **A wrong guess gives a uniform pair.** Now let $c = g - s_\ell \neq 0$. This is where p must be prime. Every nonzero number mod a prime has an inverse; for example, $2 \cdot 3 = 6 \equiv 1 \pmod 5$. Fix the error e and write $v = e + rc$ for the residual. The map that sends (a, r) to (a', v) can be undone. From the residual you recover $r = (v - e)c^{-1}$, and then you recover a by subtracting r from coordinate ℓ of a'. So this map is a bijection of $\mathbb{Z}_p^d \times \mathbb{Z}_p$. The pair (a, r) is uniform there and independent of e, so by Fact 4 the pair (a', v) is uniform, whatever e is. Finally, $b' = \langle a', s \rangle + v$. The map from (a', v) to (a', b') is another bijection, undone by subtracting $\langle a', s \rangle$ from b'. So by Fact 4 once more, (a', b') is uniform on $\mathbb{Z}_p^d \times \mathbb{Z}_p$.

5. **The test feeds $\mathcal{A}$ exactly one of its two worlds.** Each re-randomized pair uses its own fresh sample and its own r, so m of them are independent. By steps 3 and 4, they are m LWE samples with secret s when the guess is right, and m uniform pairs when it is wrong. Write α_{LWE} and α_{U} for the probabilities that $\mathcal{A}$ outputs 1 in these two cases. Then a run of the test outputs 1 with probability α_{LWE} for the right guess and α_{U} for every wrong one, and $|\alpha_{\mathrm{LWE}} - \alpha_{\mathrm{U}}| \geq \nu$ by assumption.

6. **Estimate the rates.** The search does not know α_{LWE} or α_{U}, so it estimates them by repetition. For the baseline, it draws m uniform pairs itself, runs $\mathcal{A}$ on them, and repeats N times. The fraction of 1s is its estimate $\hat{\alpha}_{\mathrm{U}}$. For each coordinate ℓ and guess g, it repeats the test N times with fresh samples and fresh r, and the fraction of 1s is $\hat{\alpha}_{\ell,g}$. Each estimate is an average of N independent 0/1 values, so it concentrates around its mean. Hoeffding's inequality, from Concentration inequalities, says that such an average lands at least t away from its mean with probability at most $2e^{-2Nt^2}$. Step 7 needs every estimate within $\nu/4$, so take $t = \nu/4$, which gives $2e^{-N\nu^2/8}$. There are $dp + 1$ estimates in all. The union bound says that the chance that at least one of several events happens is at most the sum of their chances. So some estimate is off by $\nu/4$ or more with probability at most

$$2(dp + 1)\, e^{-N\nu^2/8} \leq \beta, \tag{I.15}$$

   Taking logarithms, this holds when $N\nu^2/8 \geq \ln(2(dp + 1)/\beta)$, which is Equation I.13.

7. **Decide each guess.** The search keeps the guess g for coordinate ℓ when $|\hat{\alpha}_{\ell,g} - \hat{\alpha}_{\mathrm{U}}| > \nu/2$. Suppose every estimate is within $\nu/4$ of its mean, which happens with probability at least $1 - \beta$. For the right guess, the true gap is at least ν and the two estimation errors together are less than $\nu/2$, so the measured gap is more than $\nu/2$ and the guess is kept. For a wrong guess the true gap is 0, so the measured gap is less than $\nu/2$ and the guess is dropped. So exactly one guess survives for each coordinate, and it is s_ℓ.

8. **Count the cost.** The search calls $\mathcal{A}$ N times for the baseline and N times for each of the dp pairs of coordinate and guess, which is $(dp + 1)N$ calls. Each test call uses m fresh LWE samples, so the search uses $dpNm$ samples. By Equation I.13, N grows like $1/\nu^2$ and only like the logarithm of dp/β. So the cost is polynomial in d when p, m and $1/\nu$ are. Even a failure probability as small as $\beta = 2^{-d}$ is affordable, because then $\ln(1/\beta) = d \ln 2$ is only linear in d.

Each assumption has its place. Primality is used in step 4, a polynomial p in step 8 (the search tries every element of $\mathbb{Z}_p$), a noticeable ν in step 8 as well, and "for every secret" in step 5. The coordinates are tested independently of one another, which is the "one at a time" of the chapter.

I.2.4. Checking it

The script first checks steps 3 and 4 directly, with $d = 4$, $p = 97$ and errors drawn from a normal distribution with standard deviation 3, rounded to the nearest integer. It re-randomizes 20,000 samples with the right guess for s_1 and with a guess that is off by 1. Figure I.3 shows the residuals. With the right guess they form the narrow bump of the error, and 0.760 of them lie within 3 of zero, against the exact 0.757 for the rounded normal. With the wrong guess they are flat, and 0.071 of them lie within 3 of zero. A uniform residual lands on each of the 7 values from -3 to 3 with probability $1/97$, so the exact share is $7/97 = 0.072$.

The third panel shows why the modulus must be prime. With $p = 96$ and a guess off by 48, the term $48r \bmod 96$ is only ever 0 or 48, because $48r$ is a multiple of 96 when r is even. So the residual is the error, or the error shifted by half the modulus, and it forms two bumps instead of a flat line. A decision algorithm that looks for small residuals would accept this wrong guess about half as often as the right one, and the test of step 7 could keep it.

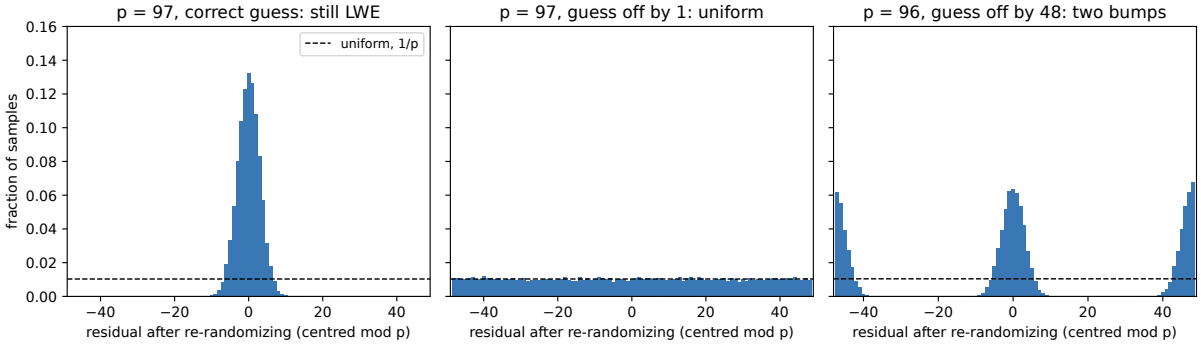


Figure I.3.: Histograms of the residual $b' - \langle a', s \rangle$ after re-randomizing 20,000 LWE samples ($d = 4$, rounded normal errors with standard deviation 3). Left: prime $p = 97$ and the right guess, which keeps the error's bump. Middle: $p = 97$ and a guess off by 1, which is uniform. Right: composite $p = 96$ and a guess off by 48, which leaves two bumps.

It then runs the whole search. A real decision algorithm for these parameters would be hard to write, so the script uses a stand-in. The stand-in sees one pair per call ($m = 1$) and is built with the secret, but the search treats it as a black box and never reads s. On 90% of calls it ignores its input and answers with a fair coin flip. On the other 10% it says 1 when the residual is within 3 of zero, which happens with probability 0.7567 for an LWE sample and 7/97 for a uniform pair. Its advantage is therefore $\nu = 0.1 \times (0.7567 - 7/97) = 0.0684$. With $\beta = 0.01$, Equation I.13 asks for $N = 19{,}230$ repetitions per test, which makes 7,480,470 calls to the stand-in in total.

The search recovers the secret $(6, 27, 27, 28)$ exactly. The smallest gap for a right guess is 0.0708, and the largest gap for a wrong one is 0.0166, both measured against the threshold $\nu/2 = 0.0342$. Figure I.4 shows a second run with another secret. Run on ten more random secrets, the search recovers every one of them.

The bound behind Equation I.13 is conservative. It asks every estimate to be within $\nu/4$ with probability $1 - \beta$ at once, but in practice the largest gap for a wrong guess, 0.0166, is only about half the threshold. The right guess clears the threshold with room to spare too. A search that stops testing once a guess is found, or that adds repetitions only for guesses near the threshold, would need fewer calls.

I.2.5. What it does not cover

The proof assumes that $\mathcal{A}$ works for every secret. Some decision algorithms work only on average over a random secret. The first step in handling them is to make the secret random: pick a uniform vector u and replace each b by $b + \langle a, u \rangle$, which gives samples with the random secret $s + u$. But for a given u the algorithm may still fail, so the search must try several u and check which ones work. Regev (2009) carries this out, and those details stay cited. The proof also

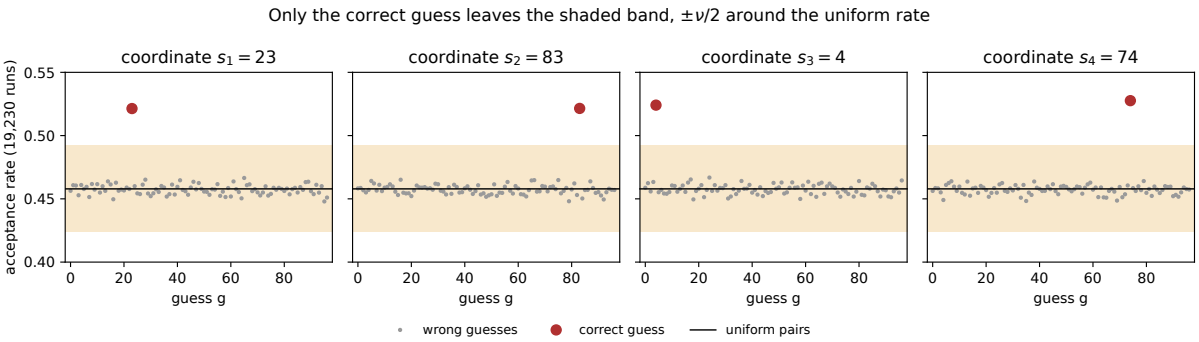


Figure I.4.: Acceptance rate of the stand-in decision algorithm on re-randomized samples, for every guess g of each of the four coordinates of a secret in $\mathbb{Z}_{97}^4$. Grey dots are wrong guesses, the red dot is the right one, the black line is the rate on uniform pairs and the shaded band is $\pm\nu/2$ around it.

assumes that a lower bound ν on $\mathcal{A}$'s advantage is known. Without one, the search can try $\nu = 1/2, 1/4, 1/8, \ldots$ in turn, which multiplies the cost by a logarithmic factor.

For moduli that are not prime, or are too large to try every guess, other reductions exist. Peikert (2009) handles moduli that are products of distinct small primes, and Micciancio & Mol (2011) give reductions that use no more samples than the decision algorithm, for a wide class of moduli. Brakerski, Langlois, Peikert, Regev & Stehlé (2013) show that decision LWE is hard for any modulus by relating moduli to one another rather than by guessing. These stay cited, because each needs lattice and number theory well beyond this appendix.

I.3. Code

The script apx_cryptography.py computes every number on this page and draws its figures. For the GGM tree it builds the toy PRG, runs the adaptive adversary in each hybrid, runs each reduction $\mathcal{B}_j$ on outputs of G and on uniform strings, and measures the single-sample distinguisher of step 7. For LWE it prints Table I.4, re-randomizes samples with right and wrong guesses, and runs the full search with the stand-in decision algorithm. From the repository root:

```
uv run python code/appendix-derivations/apx_cryptography.py
```

Table I.5.: The numbers the script prints (seed 0; 200,000 runs per GGM entry, 20,000 samples for the residuals).

Quantity	Value
GGM: exact h_0, h_1, h_2, h_3	0.0000, 0.4980, 0.4980, 0.9961
GGM: single-sample $\mathcal{B}$, rate on $G(U_\lambda)$ and on $U_{2\lambda}$	0.5535, 0.6641
GGM: advantage of $\mathcal{B}$, measured and predicted $\mathrm{Adv}_{\mathrm{PRF}}(\mathcal{A})/(nQ)$	0.1105, 0.1107

Quantity	Value
LWE: residual within 3 of zero, right guess (exact)	0.760 (0.757)
LWE: residual within 3 of zero, wrong guess (exact)	0.071 (0.072)
LWE search: ν, repetitions N, calls to the stand-in	0.0684, 19,230, 7,480,470
LWE search: smallest right gap, largest wrong gap, threshold $\nu/2$	0.0708, 0.0166, 0.0342
LWE search: random secrets recovered exactly	10 of 10

Two things are worth taking from these numbers. In the GGM check, the reductions recreate the hybrids exactly even against an adaptive adversary, so the factor nQ is lost only in the averaging of steps 6 and 7. In the LWE check, re-randomizing turns a right guess into LWE samples and a wrong one into exactly uniform pairs, and the only real cost is the number of repetitions needed to see a weak decision algorithm's small advantage.

I.4. References

- Goldreich, Goldwasser & Micali, "How to Construct Random Functions", *Journal of the ACM*, 1986. The GGM construction and its original proof.
- Goldreich, *Foundations of Cryptography*, Vol. 1: *Basic Tools*, 2001, §3.6. The careful textbook proof of the GGM theorem, with the hybrids over the levels of the tree.
- Katz & Lindell, *Introduction to Modern Cryptography*, 3rd ed., 2020. A readable proof of the GGM theorem, in its chapter on theoretical constructions of symmetric-key primitives.
- Bellare & Rogaway, "The Security of Triple Encryption and a Framework for Code-Based Game-Playing Proofs", *EUROCRYPT*, 2006. Game-playing proofs, including lazy sampling of random tables.
- Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography", *Journal of the ACM*, 2009 (conference version 2005). Defines LWE, and its search-to-decision lemma is the reduction proved here, including the step for decision algorithms that work only on average.
- Peikert, "Public-Key Cryptosystems from the Worst-Case Shortest Vector Problem", *STOC*, 2009. Search-to-decision for moduli that are products of small primes.
- Micciancio & Mol, "Pseudorandom Knapsacks and the Sample Complexity of LWE Search-to-Decision Reductions", *CRYPTO*, 2011. Search-to-decision reductions that preserve the number of samples, for a wide class of moduli.
- Brakerski, Langlois, Peikert, Regev & Stehlé, "Classical Hardness of Learning with Errors", *STOC*, 2013. Hardness of decision LWE for any modulus.
- Hoeffding, "Probability Inequalities for Sums of Bounded Random Variables", *Journal of the American Statistical Association*, 1963. The concentration bound behind the number of repetitions.

J. Mechanisms and their limits

This appendix proves eight results that the chapters on differential privacy (DP) state about specific mechanisms but do not derive in full. Three of them are about what the slack δ of approximate DP means. The first shows that every (ε, δ)-DP pair of output distributions splits into an ε-DP pair and a leftover of mass δ, and so is a post-processing of the worst (ε, δ)-DP pair, called four-point randomized response. The second uses the same split to show that (ε, δ)-DP implies an approximate form of zero-concentrated differential privacy (zCDP). The third, the stability-based histogram, shows what the slack buys: an error that does not grow with the number of bins. The other five results pin down single mechanisms. One is a lower bound for estimating means of heavy-tailed data. Three are privacy proofs for selection. They cover the sparse vector technique with several answers above the threshold, Gumbel top-c selection, and a direct proof for the Gumbel noisy max. The last is a check that the "alarm" mechanism, which the group-privacy chapter uses to show its bound is tight, really is (ε, δ)-DP.

The derivations share one habit. Each proof fixes a neighbouring pair, writes down the two output distributions, and then compares them directly: it splits them into parts, changes variables in the noise, or bounds them one outcome at a time. Table J.1 maps each derivation to the chapter section that states or uses it.

Table J.1.: The derivations of this appendix and the chapter sections that rely on them.

Derivation	Where the chapter states or uses it
Every (ε, δ) pair is post-processed four-point randomized response	Section 21.4 (Table 21.3, Equation 21.15), Section 21.5
(ε, δ)-DP implies δ-approximate $(\varepsilon^2/2)$-zCDP	Section 31.5
The stability-based histogram	Section 27.1, Section 28.3
A lower bound for heavy-tailed means	Section 28.5
Sparse vector with c answers above	Section 22.6
Gumbel top-c is the exponential mechanism without replacement	Section 22.5
A Gumbel noisy max is private by the Laplace bar argument	Section 22.4, Section 22.5
The alarm mechanism is (ε, δ)-DP	Section 19.2

J.1. Every approximate-DP pair is post-processed four-point randomized response

J.1.1. What the chapter uses

Take a mechanism that is (ε, δ)-DP, fix two neighbouring datasets D and D', and write P and Q for the distributions of its output on them. For now the outputs are discrete, with probabilities $p(y)$ and $q(y)$ at an output y; the last part of this section extends the argument to densities. Approximate DP says that the hockey-stick divergence is at most δ in both directions. That divergence is the mass of one distribution that sticks out above e^{ε} times the other, as in Equation 18.13, so

$$E_{e^{\varepsilon}}(P\|Q) = \sum_{y} \big(p(y) - e^{\varepsilon} q(y)\big)_{+} \le \delta, \qquad E_{e^{\varepsilon}}(Q\|P) = \sum_{y} \big(q(y) - e^{\varepsilon} p(y)\big)_{+} \le \delta. \tag{J.1}$$

Here the positive part $(x)_{+}$ equals x when x is positive and 0 otherwise.

The **four-point pair** $(P^{\star}, Q^{\star})$ of Table 21.3 is the worst such pair, in the sense that every test tells it apart as well as (ε, δ)-DP allows. It has four outputs: "flag D", 0, 1 and "flag D'". With probability δ it announces the dataset, by "flag D" on D or "flag D'" on D'. Otherwise it runs randomized response at level ε, which reports 0 with probability $t = e^{\varepsilon}/(1+e^{\varepsilon})$ on D and $1-t$ on D', and reports 1 otherwise. So $P^{\star}$ gives the four outputs the probabilities δ, $(1-\delta)t$, $(1-\delta)(1-t)$ and 0, and $Q^{\star}$ gives them 0, $(1-\delta)(1-t)$, $(1-\delta)t$ and δ. A **channel** K is a table of probabilities $K(y \mid z)$ that turns an input z into an output y. The claim is that some channel produces the pair (P, Q) from the four-point pair:

$$p(y) = \sum_{z} P^{\star}(z) K(y \mid z), \qquad q(y) = \sum_{z} Q^{\star}(z) K(y \mid z) \qquad \text{for every output } y. \tag{J.2}$$

The composition chapter derives the channel for $\delta = 0$ (Equation 21.12). For $\delta > 0$ it explains why a channel must exist, through Blackwell's theorem (a pair that is at least as hard to test as another at every level is a post-processing of it), and cites Kairouz, Oh & Viswanath (2015) for its construction. Optimal composition Equation 21.15, advanced composition with $\delta > 0$, and basic composition with $\delta > 0$ in Section 21.5 all rest on this channel.

J.1.2. The idea

Split each distribution into two parts. The two **pure parts**, one inside P and one inside Q, are chosen so that at every output each is at most e^{ε} times the other. The **leftover** of each distribution is everything else, and the split is arranged so that it has mass exactly δ. The pure parts then form an ε-DP pair, so the chapter's $\delta = 0$ channel produces them from plain randomized response. The leftovers are what the flags are for: the channel turns "flag D" into a draw from P's leftover, and "flag D'" into a draw from Q's.

The only care needed is in the masses. The four-point pair gives each side exactly $1 - \delta$ of plain randomized response, so the two pure parts must both have mass exactly $1 - \delta$. The obvious pure parts, $\min\{p, e^{\varepsilon} q\}$ and $\min\{q, e^{\varepsilon} p\}$, can have different masses. So the proof first moves the heavier one down to the lighter one's mass, by mixing in a scaled copy of the other one so that the factor e^{ε} survives, and then scales both down to $1 - \delta$.

A small case shows every step. Take three outputs with $P = (0.6, 0.3, 0.1)$, $Q = (0.2, 0.3, 0.5)$ and $e^{\varepsilon} = 2$. The mass of P above $2Q$ is $0.6 - 0.4 = 0.2$, and the mass of Q above $2P$ is $0.5 - 0.2 = 0.3$, so the pair is $(\ln 2, 0.3)$-DP, with δ the larger of the two. Table J.2 follows the construction.

Table J.2.: The construction on three outputs with $e^{\varepsilon} = 2$ and $\delta = 0.3$.

	Output 1	Output 2	Output 3	Total
p	0.6	0.3	0.1	1
q	0.2	0.3	0.5	1
$m_P = \min\{p, 2q\}$	0.4	0.3	0.1	$A = 0.8$
$m_Q = \min\{q, 2p\}$	0.2	0.3	0.2	$B = 0.7$
$\hat{m}_P$, the heavier part moved down to mass B	1/3	4/15	1/10	0.7
$\hat{m}_Q = m_Q$	0.2	0.3	0.2	0.7
$R_D = (p - \hat{m}_P)/0.3$	8/9	1/9	0	1
$R_{D'} = (q - \hat{m}_Q)/0.3$	0	0	1	1

The rows m_P and m_Q are the first guesses for the pure parts, with masses A and B. The heavier one, m_P, is moved down by step 4 of the derivation below: it is mixed with $m_Q/e^{\varepsilon} = m_Q/2$, with weight $\omega^* = \frac{2}{9}$ on $m_Q/2$, chosen so that the mass falls to B. So on output 1 the row $\hat{m}_P$ is $\frac{7}{9} \times 0.4 + \frac{2}{9} \times 0.1 = \frac{1}{3}$. The lighter mass $B = 0.7$ is already $1 - \delta$, so no further scaling is needed, and both pure parts end with mass 0.7. They also stay within a factor 2 of each other on every output, because the ratios $\hat{m}_P/\hat{m}_Q$ are $\frac{5}{3}$, $\frac{8}{9}$ and $\frac{1}{2}$. What is left over has mass 0.3 on each side, so dividing it by $\delta = 0.3$ gives two distributions. The section's checks below confirm that the channel built from these rows gives back P and Q exactly.

J.1.3. Derivation

Assume $0 < \delta < 1$ and $\varepsilon > 0$. Write $\gamma = e^{\varepsilon}$ to keep the formulas short.

1. **Pure parts.** Define, for every output y,

$$m_P(y) = \min\{p(y), \gamma q(y)\}, \qquad m_Q(y) = \min\{q(y), \gamma p(y)\}. \tag{J.3}$$

   Each is non-negative and at most its own distribution, $m_P \le p$ and $m_Q \le q$.
2. **They are within a factor γ.** The claim is $m_P \le \gamma m_Q$ at every output. The minimum m_P is at most γq by definition. To compare it with $\gamma m_Q = \min\{\gamma q, \gamma \cdot \gamma p\}$, note that it is also at most p, which is at most $\gamma^2 p$ because $\gamma \ge 1$. So m_P is at most both γq and $\gamma \cdot \gamma p$, and hence at most γ times the smaller one, which is γm_Q. Swapping the roles gives $m_Q \le \gamma m_P$.
3. **Their masses.** At each output, $p - \min\{p, \gamma q\} = (p - \gamma q)_+$: either the minimum is p and both sides are 0, or the minimum is $\gamma q < p$ and both sides are $p - \gamma q$. Summing over outputs, the mass of m_P is

$$A = \sum_y m_P(y) = 1 - E_\gamma(P \| Q) \ge 1 - \delta, \tag{J.4}$$

and in the same way the mass of m_Q is $B = 1 - E_\gamma(Q\|P) \geq 1 - \delta$.

4. **Move the heavier part down.** Say $A \geq B$; otherwise swap the names P and Q. For a weight ω between 0 and 1, mix m_P with the scaled-down $\gamma^{-1}m_Q$:

$$m_P^\omega = (1-\omega)\, m_P + \omega\, \gamma^{-1} m_Q. \tag{J.5}$$

Three properties hold for every ω, because each holds for both pieces of the mix, and a mix of two numbers lies between them. First, the mix fits inside p: $m_P \leq p$ by step 1, and $\gamma^{-1}m_Q \leq \gamma^{-1}\gamma p = p$ because $m_Q \leq \gamma p$. Second, it is at most γm_Q: $m_P \leq \gamma m_Q$ by step 2, and $\gamma^{-1}m_Q \leq \gamma m_Q$ because $\gamma \geq 1$. Third, it is at least $\gamma^{-1}m_Q$, which is the reverse bound $m_Q \leq \gamma m_P^\omega$. This holds because step 2 gives $m_Q \leq \gamma m_P$, that is $m_P \geq \gamma^{-1}m_Q$, and the second piece equals $\gamma^{-1}m_Q$. The mass of the mix is $(1-\omega)A + \omega\gamma^{-1}B$. It falls in a straight line from A at $\omega = 0$ to $\gamma^{-1}B$ at $\omega = 1$, and $\gamma^{-1}B \leq B \leq A$. So it passes through B, at

$$\omega^* = \frac{A - B}{A - \gamma^{-1}B} \tag{J.6}$$

(and $\omega^* = 0$ when $A = B$). Now $m_P^{\omega^*}$ and m_Q both have mass B.

5. **Scale both to mass $1 - \delta$.** Multiply both by $(1-\delta)/B$, which is at most 1 by step 3:

$$\hat{m}_P = \frac{1-\delta}{B}\, m_P^{\omega^*}, \qquad \hat{m}_Q = \frac{1-\delta}{B}\, m_Q. \tag{J.7}$$

Scaling both by the same factor keeps the ratio bounds of step 4 between $m_P^{\omega^*}$ and m_Q, and scaling down keeps $\hat{m}_P \leq p$ and $\hat{m}_Q \leq q$. Dividing by $1 - \delta$ then gives two distributions,

$$\bar{P} = \frac{\hat{m}_P}{1-\delta}, \qquad \bar{Q} = \frac{\hat{m}_Q}{1-\delta}, \tag{J.8}$$

with $\bar{P}(y) \leq \gamma\bar{Q}(y)$ and $\bar{Q}(y) \leq \gamma\bar{P}(y)$ at every output. So $(\bar{P}, \bar{Q})$ is an ε-DP pair.

6. **The leftovers are distributions.** Define the leftovers $R_D = (p - \hat{m}_P)/\delta$ and $R_{D'} = (q - \hat{m}_Q)/\delta$. Each is non-negative because of step 5, and each sums to $(1 - (1-\delta))/\delta = 1$, so each is a distribution. Rearranging the definitions gives the split

$$P = (1-\delta)\bar{P} + \delta R_D, \qquad Q = (1-\delta)\bar{Q} + \delta R_{D'}. \tag{J.9}$$

7. **The channel.** The channel sends "flag D" to R_D and "flag D'" to $R_{D'}$. It sends the reports 0 and 1 to the two rows of the $\delta = 0$ channel Equation 21.12, built for the pure pair $(\bar{P}, \bar{Q})$:

$$K(y \mid 0) = \frac{\gamma\bar{P}(y) - \bar{Q}(y)}{\gamma - 1}, \qquad K(y \mid 1) = \frac{\gamma\bar{Q}(y) - \bar{P}(y)}{\gamma - 1}. \tag{J.10}$$

The composition chapter shows that these rows are distributions when the pair is ε-DP: each entry is non-negative because no output is more than γ times likelier under one side, and each row sums to $(\gamma - 1)/(\gamma - 1) = 1$. It also shows that mixing the rows with weights t and $1 - t$ gives $\bar{P}$, and with weights $1 - t$ and t gives $\bar{Q}$.

8. **Check Equation J.2.** Feed $P^\star$ through K, one input at a time. The input "flag D" has probability δ, and the channel sends it to R_D. The reports 0 and 1 have probabilities $(1-\delta)t$ and $(1-\delta)(1-t)$, so by step 7 their rows mix to $(1-\delta)\bar{P}$. The input "flag D'" has probability 0 under $P^\star$ and adds nothing. So the total is $\delta R_D + (1-\delta)\bar{P}$, which is P by Equation J.9. The same steps with $Q^\star$, whose report 1 has probability $(1-\delta)t$, give $(1-\delta)\bar{Q} + \delta R_{D'} = Q$.

J.1.4. Checking it

The script runs the construction on three kinds of pairs: the small example, many random pairs, and a binned normal pair. On the small example of Table J.2 it finds $\omega^* = 0.2222$ and the channel rows $K(\cdot \mid 0) = (0.6667, 0.3333, 0)$ and $K(\cdot \mid 1) = (0.0952, 0.4762, 0.4286)$. Feeding $P^\star$ and $Q^\star$ through them returns $(0.6, 0.3, 0.1)$ and $(0.2, 0.3, 0.5)$. It then draws 2,000 random pairs for each of three settings: 3 outputs at $\varepsilon = 0.5$, 8 outputs at $\varepsilon = 1$ and 20 outputs at $\varepsilon = 2$. A quarter of the pairs have an output that only one side can produce. Such an output has an infinite privacy loss, so all of its mass must go into the leftover. Each pair gets its own exact δ. Over all 6,000 pairs, $P^\star K$ and $Q^\star K$ differ from P and Q by at most 2.2×10^{-16}, and every ratio bound of step 5 holds to within rounding. The smallest channel entry is -1.4×10^{-13}, which is rounding too: the leftover rows divide a difference by δ, and δ is sometimes small.

Figure J.1 shows the split on a larger pair. P is a normal distribution with mean 0 and standard deviation 1, Q one with mean 1 and standard deviation 0.8, each reported in eight bins. At $\varepsilon = 1$ the two hockey-stick divergences are 0.2259 and 0.0844, so $\delta = 0.2259$. The red leftover of P sits in the left bins, where p exceeds $e^\varepsilon q$, while the leftover of Q sits on the right, where q exceeds $e^\varepsilon p$. In both panels the blue pure parts stay under the dashed line.

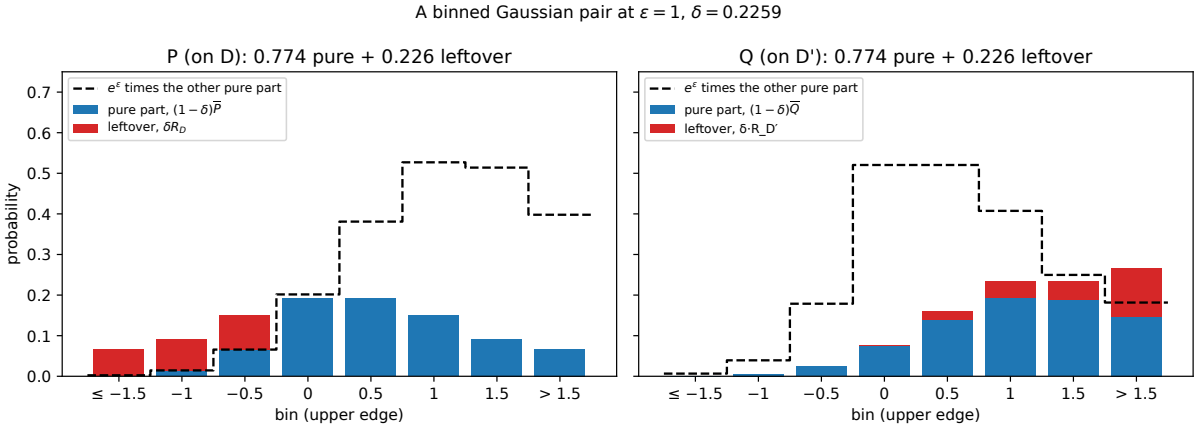


Figure J.1.: The split of Equation J.9 for a pair of binned normal distributions at $\varepsilon = 1$, $\delta = 0.2259$. Blue: the pure parts $(1-\delta)\overline{P}$ and $(1-\delta)\overline{Q}$. Red: the leftovers δR_D and $\delta R_{D'}$. Dashed: e^ε times the other side's pure part, which each blue bar must stay under.

Figure J.2 shows the same pair as hypothesis tests. On the left are trade-off curves: for each false positive rate x, the smallest false negative rate any test reaches. The pair's curve lies above the boundary $f_{\varepsilon,\delta}(x) = \max\{0, 1 - \delta - e^\varepsilon x, e^{-\varepsilon}(1 - \delta - x)\}$ of Equation 18.18, which is exactly the curve of the four-point pair. Its pure part lies above the higher boundary $f_{\varepsilon,0}$ of plain randomized response, because removing the leftover removes the outputs that tests can exploit. On the right is step 4. Here Q's first pure part is the heavier one, with mass $B = 0.9156$ against $A = 0.7741$. So step 4 runs with the roles of P and Q swapped: Q's part is mixed down, and it reaches A at $\omega^* = 0.2243$.

The result is exact, not a bound, so there is no slack to measure. The construction is not unique, because other splits work too. But no split can use a δ below the pair's own: every split needs a leftover of mass at least $\max\{E_\gamma(P\|Q), E_\gamma(Q\|P)\}$, because a leftover smaller than the mass sticking out above γ times the other side would leave some of that mass in the pure part.

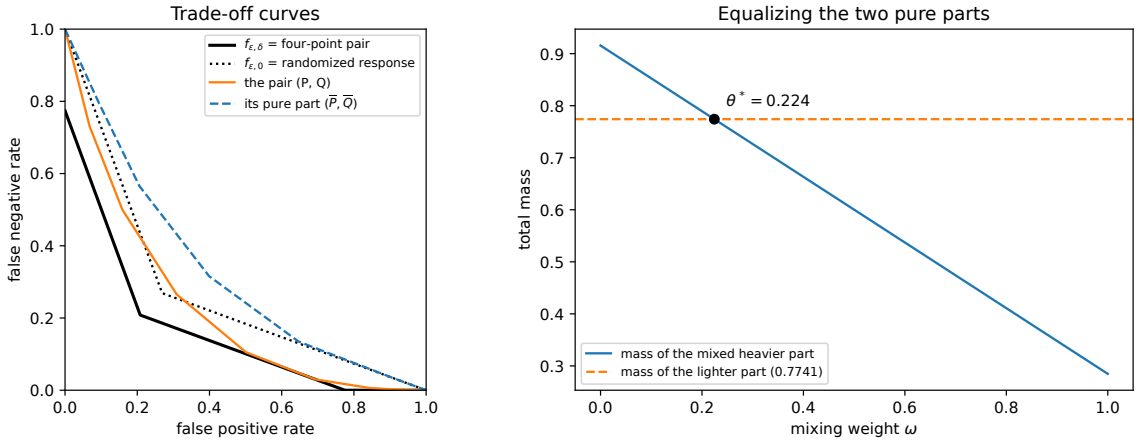


Figure J.2.: Left: trade-off curves of the binned normal pair (orange) and of its pure part (blue dashed), against the boundaries $f_{\varepsilon,\delta}$ (black), attained by the four-point pair, and $f_{\varepsilon,0}$ (dotted), attained by randomized response. Right: the mass of the mixed heavier pure part against the mixing weight ω; it falls in a straight line and meets the lighter part's mass at $\omega^* = 0.224$.

J.1.5. What it does not cover

The proof needs $\delta > 0$ to divide by in step 6, and the case $\delta = 0$ is the chapter's own derivation. It also needs $\varepsilon > 0$ to divide by $\gamma - 1$ in step 7. With $\varepsilon = 0$ the pure parts are equal, $\bar{P} = \bar{Q}$, randomized response reports 0 with probability $\frac{1}{2}$ on both sides, and both rows of the channel are simply $\bar{P}$.

Continuous outputs need one change. Replace the probabilities $p(y)$ and $q(y)$ by densities with respect to a common reference distribution, for example $(P + Q)/2$. Both have densities between 0 and 2 with respect to it, so every quantity above is finite. Every step above works output by output, so it goes through with sums replaced by integrals.

J.2. Approximate DP implies approximate zCDP

J.2.1. What the chapter uses

Zero-concentrated differential privacy describes a mechanism by a slope ρ. It asks that the Rényi divergence D_α of every order $\alpha > 1$, between the outputs on neighbouring datasets, lie under the line $\rho\alpha$ through the origin (Equation 31.1). The approximate version allows this to fail on an event of probability at most δ. Precisely, a mechanism M is **δ-approximately ρ-zCDP** if every pair of neighbours $D \sim D'$ has an event E for the run on D and an event E' for the run on D' with the following two properties. (The events may depend on how the run produced its output, as the idea below explains, and $M(D)|_E$ is the output distribution conditioned on E.) Each event has probability at least $1 - \delta$, and for every order $\alpha > 1$,

$$D_\alpha(M(D)|_E \,\|\, M(D')|_{E'}) \le \rho\alpha \qquad \text{and} \qquad D_\alpha(M(D')|_{E'} \,\|\, M(D)|_E) \le \rho\alpha. \tag{J.11}$$

The chapter states in Section 31.5 that an (ε, δ)-DP mechanism is δ-approximately $(\varepsilon^2/2)$-zCDP (Bun & Steinke 2016, Proposition 1.13). As the reason, it says only that any (ε, δ)-DP pair decomposes so that "with probability $1 - \delta$ it behaves like an ε-DP pair". It also states that approximate zCDP composes, with slack $\delta_1 + \delta_2 - \delta_1\delta_2$ for two steps (their Lemma 8.2), and it uses both facts in an example pipeline that mixes stability-based steps with Gaussian steps.

J.2.2. The idea

The decomposition is Equation J.9 of the previous section. It says that you can produce $M(D)$ in two stages. First flip a coin that says "pure" with probability $1 - \delta$. If it does, draw the output from the pure part $\bar{P}$, and otherwise draw it from the leftover R_D. The output then has exactly the distribution $P = (1 - \delta)\bar{P} + \delta R_D$, so this two-stage run is the same mechanism. Do the same on D' with $\bar{Q}$ and $R_{D'}$. Now let E and E' be the events that the coins say "pure". Conditioned on them, the outputs have distributions $\bar{P}$ and $\bar{Q}$, an ε-DP pair, and the chapter has already shown that every ε-DP pair is $(\varepsilon^2/2)$-zCDP.

Note that these events are about how the output was produced, not only about the output itself. That is enough for the chapter, because its conversion from approximate zCDP back to (ε, δ)-DP uses E only through its probability and through the conditional distribution.

J.2.3. Derivation

1. **Split the pair.** Fix neighbours $D \sim D'$, with output distributions P and Q. Because the mechanism is (ε, δ)-DP, both hockey-stick divergences are at most δ. For $0 < \delta < 1$ and $\varepsilon > 0$, the previous section then gives Equation J.9, with $(\bar{P}, \bar{Q})$ an ε-DP pair. If $\delta = 0$, take $\bar{P} = P$ and $\bar{Q} = Q$; if $\varepsilon = 0$, the previous section's last part covers it; and $\delta = 1$ makes the claim empty.
2. **Realize the mechanism in two stages.** Run $M(D)$ by drawing a coin C that equals "pure" with probability $1 - \delta$, then an output from $\bar{P}$ if C is "pure" and from R_D otherwise. The output has distribution $(1 - \delta)\bar{P} + \delta R_D = P$. Run $M(D')$ the same way with $\bar{Q}$ and $R_{D'}$.
3. **Pick the events.** Let E be "C is pure" in the run on D, and E' the same in the run on D'. Each has probability exactly $1 - \delta$. Conditioned on E the output is drawn from $\bar{P}$, and conditioned on E' from $\bar{Q}$.
4. **Apply the pure-DP bound.** The pair $(\bar{P}, \bar{Q})$ is ε-DP, so by Equation 31.7 of Section 31.2,

$$D_\alpha(\bar{P}\|\bar{Q}) \le \frac{\alpha\varepsilon^2}{2} \qquad \text{and} \qquad D_\alpha(\bar{Q}\|\bar{P}) \le \frac{\alpha\varepsilon^2}{2} \qquad \text{for every } \alpha > 1. \tag{J.12}$$

   That chapter proves it with Hoeffding's lemma. The privacy loss of an ε-DP pair lies in $[-\varepsilon, \varepsilon]$, and a Rényi divergence is the logarithm of an exponential moment of the privacy loss. Hoeffding's lemma bounds that moment by a normal one, and the result is the line $\alpha\varepsilon^2/2$. Steps 3 and 4 together are the definition with $\rho = \varepsilon^2/2$. The sharper Equation 31.11 gives $\rho = \varepsilon \tanh(\varepsilon/2)$ the same way.

Composition, in the case the chapter uses. The pipeline of Section 31.5 mixes steps that are $(\varepsilon_i, \delta_i)$-DP with steps that are ρ_i-zCDP, each chosen after seeing the earlier outputs. For a zCDP step take $\delta_i = 0$ and the coin always "pure". For an $(\varepsilon_i, \delta_i)$-DP step take $\rho_i = \varepsilon_i^2/2$ and the two-stage run of step 2, built for the mechanism that the earlier outputs (the history) selected.

5. **The coins ignore the history.** Whatever the earlier outputs were, step i's coin says "pure" with probability exactly $1-\delta_i$. So the event "every coin says pure" has probability $\prod_i(1-\delta_i) = 1-\delta_a$, which defines the slack δ_a.
6. **Condition on all coins.** The probability of a sequence of outputs together with "every coin says pure" is a product over steps of $(1-\delta_i)$ times the pure distribution of step i given the history. Dividing by $\prod_i(1-\delta_i)$ leaves the product of the pure distributions. So conditioned on all coins being pure, the run is an adaptive composition of the pure parts, on D and on D' alike.
7. **Add the slopes.** Each pure part is ρ_i-zCDP for every history. Rényi divergences of an adaptive composition add, even when each step is chosen from the earlier outputs (Rényi DP; Theorem 31.1), so the conditioned run is $(\sum_i \rho_i)$-zCDP. The whole pipeline is therefore δ_a-approximately $(\sum_i \rho_i)$-zCDP, with $\delta_a = 1-\prod_i(1-\delta_i)$. For two steps that is $\delta_1+\delta_2-\delta_1\delta_2$.

J.2.4. Checking it

The script computes the Rényi divergences of the pure parts from the previous section on a grid of orders from 1.001 to 20.9, in both directions. For the three-output example, with $\varepsilon = \ln 2$ and $\rho = \varepsilon^2/2 = 0.2402$, the largest value of $D_\alpha/(\alpha\rho)$ is 0.4275. For the binned normal pair, with $\rho = 0.5$, it is 0.4306. Both are well under 1, the most the bound allows, and the slack has two sources. First, the Rényi divergence of an ε-DP pair never exceeds ε, so as α grows its curve flattens while the line $\alpha\varepsilon^2/2$ keeps rising. Second, these pure parts are milder than randomized response, the worst ε-DP pair.

J.2.5. What it does not cover

The events here depend on the coin that chooses between the pure part and the leftover, not only on the output. If an event had to be a set of outputs, the argument would not apply as it stands, because the leftover and the pure part can share outputs. The chapter's derivation in Section 31.5 uses the events only through their probabilities and conditional distributions, so it holds for these events.

The composition argument above needs each good event to have probability exactly $1-\delta_i$ given any history, which is what the two-stage run provides. Bun & Steinke's Lemma 8.2 covers general approximately-zCDP steps, whose good events only have probability at least $1-\delta_i$ and may depend on the history. That general case stays cited. For the pipelines of the chapter, which mix (ε, δ)-DP steps with zCDP steps, the special case is all that is used.

J.3. The stability-based histogram

J.3.1. What the chapter uses

A histogram over a universe of N bins releases a count for each bin. Under pure DP, the packing argument of Section 27.1 shows that the ℓ_∞ error, the largest error over the bins, must grow like $\ln N/\varepsilon$ (Equation 27.5). Under approximate DP, the chapter says, "the stability-based histogram (Vadhan 2017) adds noise only to the bins that are non-empty … its error does not grow with N

at all". It gives no proof. In Section 28.3, the same mechanism locates the data roughly before a mean is estimated, without any bound on where the data might be.

The **stability-based histogram** with parameters ε and δ works on a dataset of n records. Each record falls in one bin, and c_x is the number of records in bin x. The mechanism compares noisy counts with a threshold, set to

$$\tau = 1 + \frac{\ln\left(1/(2\delta)\right)}{\varepsilon}. \tag{J.13}$$

For every bin with $c_x \geq 1$, it draws a fresh Laplace noise $Z_x \sim \mathrm{Lap}(1/\varepsilon)$, and it releases the pair $(x, c_x + Z_x)$ if the noisy count exceeds τ. It releases nothing else, and a bin that is not released is read as 0. The chapter needs two claims about it. The first is privacy: under add/remove neighbours, where D' adds or removes one record, the mechanism is (ε, δ)-DP. The second is accuracy. Suppose k bins are non-empty, and allow a failure probability β. Then with probability at least $1 - \beta$ every bin's estimate is within

$$\tau + \frac{\ln(k/\beta)}{\varepsilon} = 1 + \frac{\ln\left(1/(2\delta)\right) + \ln(k/\beta)}{\varepsilon} \tag{J.14}$$

of its true count. Since $k \leq n$, this is of order $(\ln(1/\delta) + \ln(n/\beta))/\varepsilon$, with no N in it.

J.3.2. The idea

Adding one record changes one bin, from count c to $c + 1$, and two cases can happen. If the bin already had records, both datasets put Laplace noise on it. That is the ordinary Laplace mechanism on a count, which is ε-DP, and the threshold only decides what to print, so it changes nothing. If the bin was empty, the smaller dataset never even looks at it, so it cannot be released. The larger dataset releases it only if a count of 1 plus noise clears τ, and the threshold is set so that this happens with probability exactly δ. So apart from an event of probability δ, the two runs print the same thing. That is the argument of Section 18.8 for the mechanism that releases each record with probability δ.

Accuracy comes from the same threshold. A bin that is not released has a count below about τ, so reading it as 0 costs at most about τ. A released bin has the usual Laplace error. Only the k non-empty bins have any noise at all, so the union bound runs over k bins, not over N.

J.3.3. Derivation

Privacy. Fix a dataset D and add one record, in bin x, to get D'.

1. **Only bin x differs.** Every other bin has the same count in D and D'. Its noise is drawn the same way, independently of bin x, so its part of the output has the same distribution on both sides.
2. **Bin x was non-empty.** Then $c_x(D) \geq 1$, and both runs add $\mathrm{Lap}(1/\varepsilon)$ to a count that changes by 1. That is the Laplace mechanism with sensitivity 1, which is ε-DP. Comparing the noisy count with τ is a function of it, which is post-processing, so bin x's part of the output is ε-DP.

3. **Bin x was empty.** On D nothing about bin x is ever printed, while on D' bin x has count 1 and is printed exactly when $1 + Z_x > \tau$. A $\mathrm{Lap}(1/\varepsilon)$ variable exceeds a level $s \geq 0$ with probability $\frac{1}{2}e^{-\varepsilon s}$. Here the level is $\tau - 1$, which is $\ln(1/(2\delta))/\varepsilon$ by Equation J.13, so bin x is printed with probability

$$\Pr[Z_x > \tau - 1] = \tfrac{1}{2}e^{-\ln(1/(2\delta))} = \delta. \tag{J.15}$$

4. **Couple the two runs in the empty case.** Run the mechanism on D' by first running it on D with the same noise, then drawing Z_x for the new bin. The two outputs differ only if bin x is printed, which has probability δ. So for any set S of outputs, $\Pr[M(D') \in S] \leq \Pr[M(D) \in S] + \delta$. In the other direction, the outputs agree whenever bin x is not printed. That depends only on Z_x, which is drawn independently of everything else, so the outputs agree with probability $1 - \delta$ whatever $M(D)$ printed, and $\Pr[M(D') \in S] \geq (1 - \delta)\Pr[M(D) \in S] \geq \Pr[M(D) \in S] - \delta$. Both directions are $(0, \delta)$-DP.
5. **Put the bins back together.** Step 4 already compares whole outputs, but step 2 compares only bin x's part. To finish step 2, fix the output w of all the other bins. For each w, bin x's part satisfies the ε-DP inequality. The other bins' output w has the same distribution on D and D', so averaging the inequality over w with these common weights keeps it. Either way the pair satisfies (ε, δ)-DP, in both orders.

Accuracy. Let the k non-empty bins have noises Z_x.

6. **Error of one bin.** An empty bin is never printed and its estimate 0 is exact. A printed bin has error $|Z_x|$. A non-empty bin that is not printed has $c_x + Z_x \leq \tau$, so its error, the count c_x itself, is at most $\tau - Z_x \leq \tau + |Z_x|$. Either way the error is at most $\tau + |Z_x|$.
7. **All k noises at once.** A $\mathrm{Lap}(1/\varepsilon)$ variable exceeds s in absolute value with probability $e^{-\varepsilon s}$. The chance that at least one of k events happens is at most the sum of their chances (the union bound), so

$$\Pr\left[\max_x |Z_x| > s\right] \leq ke^{-\varepsilon s}. \tag{J.16}$$

The right side equals β at $s = \ln(k/\beta)/\varepsilon$. Together with step 6, this shows that the ℓ_∞ error is at most Equation J.14 with probability at least $1 - \beta$.

J.3.4. Checking it

Take $\varepsilon = 1$ and $\delta = 10^{-6}$, so $\tau = 1 + \ln(5 \times 10^5) = 14.1224$. Figure J.3 shows the probability that a bin with true count c is printed. Below τ it falls by a factor e^ε for each record removed, and at $c = 1$ it is exactly 10^{-6}. That point is the whole δ the mechanism spends.

For accuracy, take $k = 10$ non-empty bins and $\beta = 0.05$. The bound Equation J.14 is $14.1224 + \ln 200 = 19.4207$. The script also computes the exact 95% quantile of the error when all ten bins hold the same count, and takes the worst count. The worst is a count of 18. Each such bin is suppressed with probability $\frac{1}{2}e^{-(18-\tau)} = 0.0104$, and then its whole count is the error. With ten bins, at least one is suppressed with probability about 0.1, which is more than 0.05, so the 95% quantile is exactly 18. A simulation of 200,000 runs at that count gives the same 18, so the bound is loose by about 1.4 records. The union bound costs little, and the slack has two small sources. A bin is suppressed only when its noise is negative, so only one of the two Laplace tails matters, which is worth $\ln 2 = 0.69$. And counts are whole numbers, so the worst count cannot sit exactly where the bound puts it.

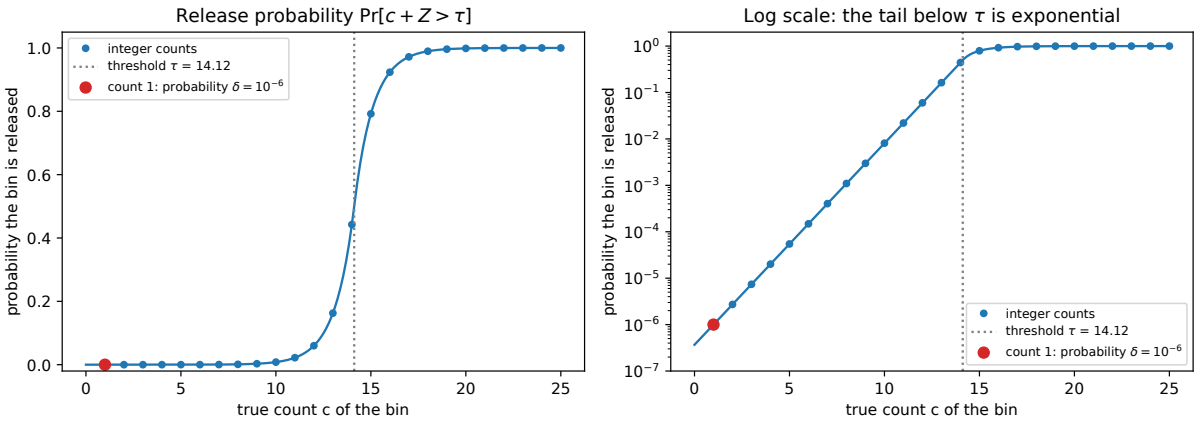


Figure J.3.: The probability that the stability-based histogram prints a bin, against the bin's true count, at $\varepsilon = 1$ and $\delta = 10^{-6}$, on a linear scale (left) and a log scale (right). The dotted line is the threshold $\tau = 14.12$. At count 1 the probability is exactly δ, which is all the privacy that empty bins cost.

Figure J.4 compares the error with pure DP as the universe grows. The Laplace histogram must add noise to all N bins. All N noises stay below s in size with probability $(1 - e^{-\varepsilon s})^N$, so setting this to 0.95 gives the exact 95% error, $-\ln(1 - 0.95^{1/N})/\varepsilon$, which is 16.79 at $N = 10^6$ and 30.60 at $N = 10^{12}$. The stability-based histogram stays at 19.42 for every N. It beats the Laplace histogram from $N = 1.394 \times 10^7$. It even falls below the packing lower bound Equation 27.5, $\frac{1}{2}(\ln(0.95N)/\varepsilon - 1)$, which holds for every pure-DP mechanism, but only from $N = 2.114 \times 10^{17}$, because $\ln(1/\delta)$ is a large constant.

J.3.5. What it does not cover

The proof is for add/remove neighbours, which is the relation of the lower-bounds chapter. Under replace neighbours, as in Section 28.3, one record leaves one bin and joins another. Each of the two bins behaves as in step 2 or step 4, independently, and basic composition gives $(2\varepsilon, 2\delta)$-DP. So run the mechanism with $\varepsilon/2$ and $\delta/2$.

The threshold Equation J.13 is the smallest that makes step 3 cost exactly δ. A larger threshold is also private, at the price of a larger error. The bound counts only bins that hold data. A universe with N bins, or even infinitely many, never enters, because the mechanism never draws noise for an empty bin.

J.4. A lower bound for heavy-tailed means

J.4.1. What the chapter uses

Section 28.5 estimates the mean of data whose variance is at most σ^2, from n records, under replace neighbours. It clips each record to an interval narrower than the data's spread, accepting a bias, and reaches an error a with about $\sigma^2/a^2 + 8\sqrt{2}\,\sigma^2/(a^2\varepsilon)$ records, by Equation 28.15. It

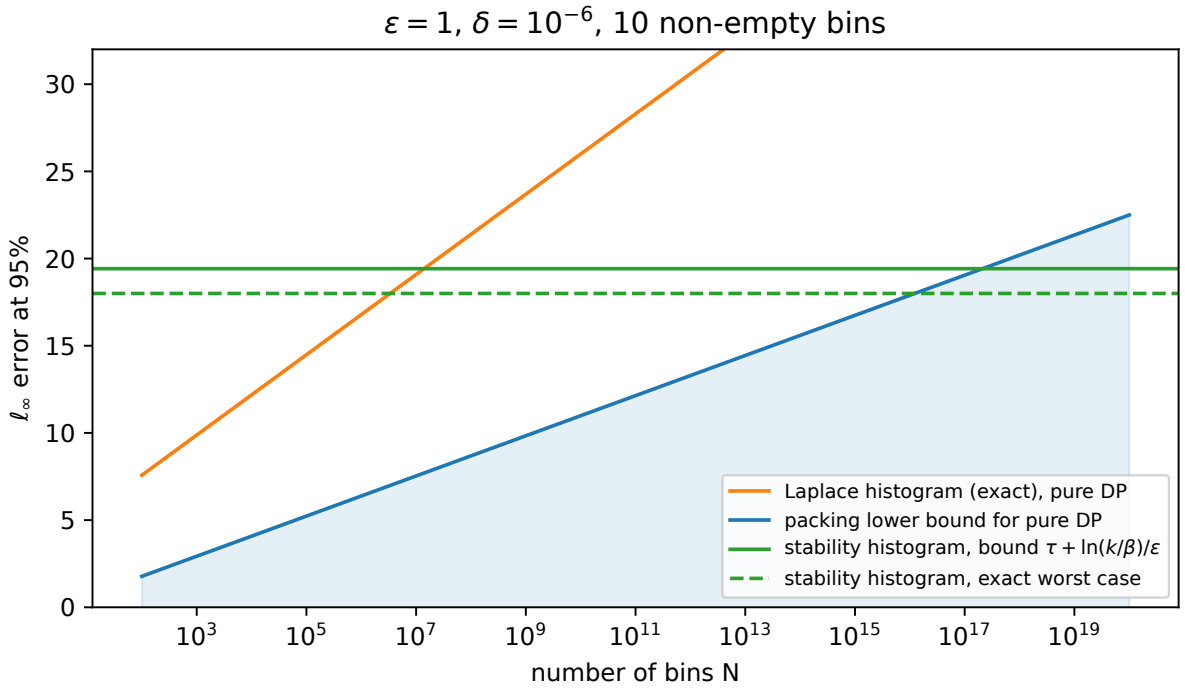


Figure J.4.: The ℓ_∞ error at the 95% level against the number of bins N, at $\varepsilon = 1$, for data in 10 non-empty bins. Orange: the Laplace histogram (exact), a pure-DP mechanism. Blue: the packing lower bound for every pure-DP mechanism, with the impossible region shaded. Green: the stability-based histogram with $\delta = 10^{-6}$, its bound (solid) and its exact worst case (dashed), neither of which depends on N.

then says that "Kamath, Singhal & Ullman (2020) show that this is unavoidable", and that if the k-th moment $\mathbb{E}|X-\mu|^k$ is bounded the cost falls to about $1/(\varepsilon a^{k/(k-1)})$ records. It gives no proof.

This section proves the claim for mechanisms that are pure ε-DP. Fix a whole number $k \geq 2$ and a bound m_k on the k-th central moment, the average of $|X-\mu|^k$ for a record X with mean μ. For the variance, $k=2$ and $m_2 = \sigma^2$. The data may come from any distribution that satisfies

$$\mathbb{E}|X-\mu|^k \leq m_k. \tag{J.17}$$

Suppose an ε-DP mechanism M takes n records from such a distribution and returns an estimate within less than a of μ with probability at least $1-\beta$. The probability is over both the data and the mechanism's noise, and the target error must satisfy $a \leq m_k^{1/k}/2$. Then

$$n \geq \frac{\ln\left((1-\beta)/\beta\right)}{\ln\left(1+p(e^\varepsilon-1)\right)} \geq \frac{\ln\left((1-\beta)/\beta\right)}{p(e^\varepsilon-1)}, \qquad p = \frac{(2a)^{k/(k-1)}}{m_k^{1/(k-1)}}. \tag{J.18}$$

For the variance, with $k=2$, the simpler bound reads

$$n \geq \frac{\sigma^2 \ln\left((1-\beta)/\beta\right)}{4a^2(e^\varepsilon-1)}. \tag{J.19}$$

For small ε, $e^\varepsilon - 1$ is close to ε, so this is of order $\sigma^2/(a^2\varepsilon)$, the rate of the chapter's estimator. For general k, with the moment bound m_k held fixed, it is of order $1/(\varepsilon a^{k/(k-1)})$.

J.4.2. The idea

Build two distributions that a good estimator must tell apart but that DP says it cannot. The first, P_0, puts all its mass at 0, so its mean is 0. The second, P_1, is the same except that a small probability p moves to a far point v. Its mean is pv, which we set to $2a$. An accurate estimator must answer below a on data from P_0 and above a on data from P_1. The far point lets the mean move by $2a$ while the moment stays within its bound, because the mass out there is tiny.

Now look at the data. A sample from P_0 is all zeros. A sample from P_1 is all zeros except for about np records equal to v, so it is only about np replacements away from the all-zeros sample. Group privacy says that ε-DP outputs on datasets J replacements apart differ by at most a factor $e^{J\varepsilon}$. If np is small, the two samples look too much alike for the estimator to answer differently, so n must be at least about $1/(p\varepsilon)$. The more spread the moment bound allows, the farther the point v can be, the smaller p becomes, and the larger n must be.

Figure J.5 draws the two distributions for $k=2$, $\sigma=1$ and $a=0.15$.

J.4.3. Derivation

1. **The far point.** Let P_1 put probability p on a value $v>0$ and $1-p$ on 0, with $pv = 2a$ and $pv^k = m_k$. (Step 2 shows that pv^k bounds the central moment.) Dividing the second equation by the first gives $v^{k-1} = m_k/(2a)$, so

$$v = \left(\frac{m_k}{2a}\right)^{1/(k-1)}, \qquad p = \frac{2a}{v} = \frac{(2a)^{k/(k-1)}}{m_k^{1/(k-1)}}. \tag{J.20}$$

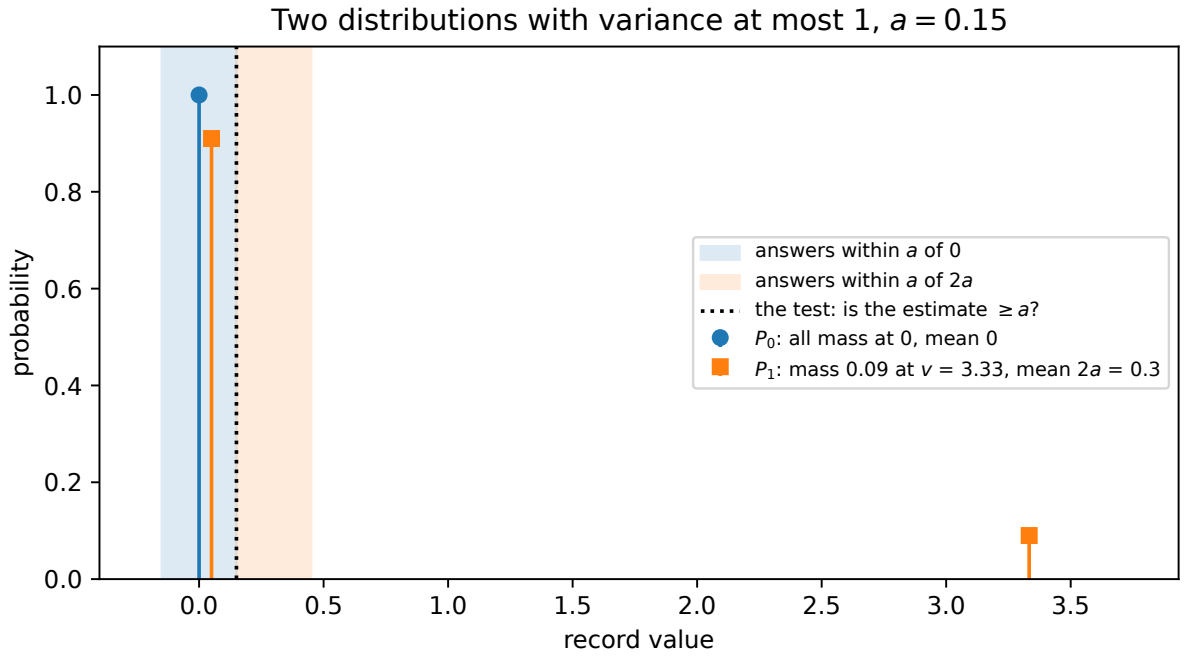


Figure J.5.: The two distributions of the lower bound for variance at most 1 and $a = 0.15$. P_0 (blue) puts all its mass at 0. P_1 (orange, its stem at 0 drawn slightly to the right) moves mass $p = 0.09$ to $v = 3.33$, which shifts the mean to $2a = 0.3$. An estimator accurate to a must answer in the blue band on P_0 and in the orange band on P_1, so the test "is the estimate at least a?" separates them.

For p to be a probability you need $v \geq 2a$, which is $m_k \geq (2a)^k$, the condition $a \leq m_k^{1/k}/2$. For $k = 2$ this is $v = \sigma^2/(2a)$ and $p = 4a^2/\sigma^2$.

2. P_1 **meets the moment bound.** Its mean is $\mu = pv$. The value 0 is at distance pv from it and the value v at distance $(1-p)v$, so

$$\mathbb{E}|X - \mu|^k = (1-p)(pv)^k + p\big((1-p)v\big)^k = p(1-p)v^k\big(p^{k-1} + (1-p)^{k-1}\big). \tag{J.21}$$

Both p and $1-p$ lie in $[0,1]$, and raising such a number to the power $k-1 \geq 1$ cannot increase it. So the last bracket is at most $p + (1-p) = 1$, and the moment is at most $p(1-p)v^k \leq pv^k = m_k$. P_0 has moment 0.

3. **The datasets.** A sample of n records from P_0 is the all-zeros dataset D_0. A sample D_1 from P_1 has some number J of records equal to v, where J follows a binomial distribution with n trials and success probability p. You get D_1 from D_0 by replacing J records, so the two are at distance J.
4. **The test.** Let S be the event "the estimate is at least a". On P_0 the mean is 0, and an estimate within less than a of it is below a. So accuracy on P_0 gives $\Pr[M(D_0) \in S] \leq \beta$. On P_1 the mean is $2a$, and an estimate within less than a of it is above a. So accuracy on P_1, averaged over the random dataset D_1, gives $\mathbb{E}\Pr[M(D_1) \in S] \geq 1 - \beta$.
5. **Group privacy.** For pure DP, two datasets at distance J satisfy $\Pr[M(D_1) \in S] \leq e^{J\varepsilon}\Pr[M(D_0) \in S]$ (Equation 19.6 with $\delta = 0$). The reason is that the privacy loss adds up along the path of J single replacements, at most ε per step. With step 4, $\Pr[M(D_1) \in S] \leq e^{J\varepsilon}\beta$ for every possible D_1.
6. **Average over the datasets.** Combining steps 4 and 5 gives $1 - \beta \leq \beta\,\mathbb{E}[e^{\varepsilon J}]$. Each of the n records independently contributes a factor e^ε with probability p and 1 otherwise, so the average of $e^{\varepsilon J}$ is $(1 - p + pe^\varepsilon)^n$. Hence

$$1 - \beta \leq \beta\big(1 + p(e^\varepsilon - 1)\big)^n. \tag{J.22}$$

7. **Solve for n.** Take logarithms and divide by $\ln(1 + p(e^\varepsilon - 1))$ to get the first bound of Equation J.18. Since $\ln(1+x) \leq x$, the denominator is at most $p(e^\varepsilon - 1)$, which gives the second, simpler bound.

J.4.4. Checking it

Take a variance bound $\sigma^2 = 1$ and $\varepsilon = 1$, and ask for error $a = 0.05$ with failure probability $\beta = 0.05$. Step 1 gives $v = 10$ and $p = 0.01$, and the variance of P_1 is 0.99, within the bound. Equation J.18 then says that every ε-DP estimator needs at least 172.8 records, or 171.4 by the simpler form. The chapter's clipped mean reaches a root-mean-square error of 0.05 with 7,616 records, by Equation 28.15. The two use different measures of error and the lower bound's constants are loose, so the gap of a factor 44 says little. What matters is the rate. Lighter tails make the bound smaller. With a k-th moment bound of 1 and the same a, it falls to 55.6 records for $k = 3$ and to 38.4 for $k = 4$. For data in $[0,1]$ the far point can be at most 1, so the same argument with $v = 1$ and $p = 2a$ gives 18.6. The script also checks step 2 on a grid of a for $k = 2, 3, 4, 6$, and the largest k-th central moment of P_1 never exceeds 1. It comes closest at the smallest a, where it prints as 1.0000.

Figure J.6 plots these bounds against a on log–log axes. For $k = 2$ the lower bound and the chapter's estimator both have slope -2, so they differ by a constant factor at every a. A larger k gives a smaller slope, $-k/(k-1)$, which approaches -1, the slope of bounded data.

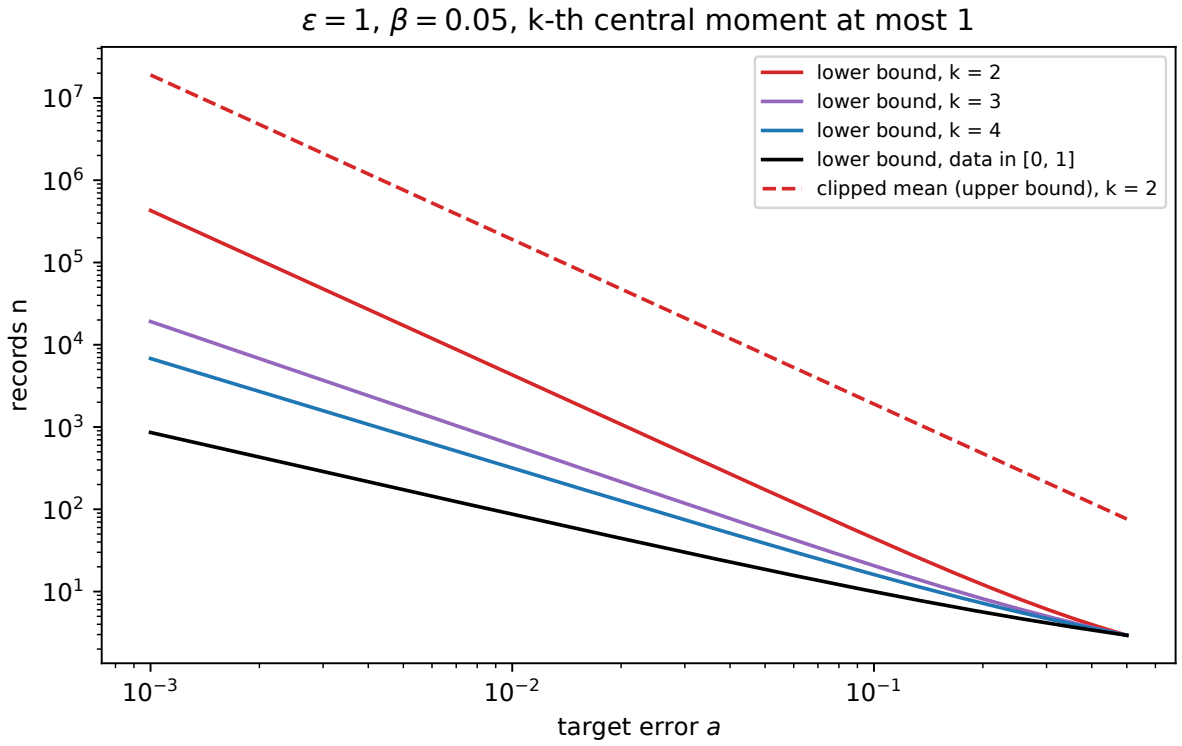


Figure J.6.: Records needed for error a at $\varepsilon = 1$, $\beta = 0.05$, on log–log axes, with k-th central moment at most 1. Solid: the lower bound Equation J.18 for $k = 2$, 3 and 4, and the same argument for data in $[0, 1]$. Dashed: the chapter's clipped mean Equation 28.15 for $k = 2$, which reaches root-mean-square error a. The $k = 2$ lines are parallel, so the chapter's estimator has the optimal rate.

J.4.5. What it does not cover

The proof needs pure DP. Step 5 multiplies by $e^{J\varepsilon}$, and under approximate DP group privacy adds a slack that grows like $e^{J\varepsilon}\delta$, which ruins the averaging in step 6 once J is large. Lower bounds under approximate DP need different tools and are not proved here, and Kamath, Singhal & Ullman (2020) is the place to start. The bound is also only the privacy part of the cost. The sampling part, about σ^2/a^2 records, comes from the usual non-private lower bounds.

J.5. Sparse vector with c answers above

J.5.1. What the chapter uses

Section 22.6 proves that AboveThreshold, which halts at the first query above a noisy threshold, is $(\varepsilon_1 + \varepsilon_2)$-DP, where ε_1 is the budget spent on the threshold's noise and ε_2 the budget spent on the queries' noise. To find c queries above the threshold, it says, "Lyu, Su & Li's Algorithm 1 keeps one threshold noise and uses $\nu_i \sim \mathrm{Lap}(2c\Delta/\varepsilon_2)$, so that the c shifts together cost e^{ε_2}". It gives no proof. The algorithm takes a dataset D, queries $f_1, f_2, \ldots$ of sensitivity Δ that may be chosen adaptively, a threshold τ and budgets $\varepsilon_1, \varepsilon_2 > 0$.

1. Draw the threshold noise $\xi \sim \mathrm{Lap}(\Delta/\varepsilon_1)$ once.
2. For each query i, draw fresh noise $\nu_i \sim \mathrm{Lap}(2c\Delta/\varepsilon_2)$. If $f_i(D) + \nu_i \geq \tau + \xi$, output $\top$, and halt if this is the c-th $\top$. Otherwise output $\bot$.

The claim is that this is $(\varepsilon_1 + \varepsilon_2)$-DP for any number of queries. If the queries are monotone, meaning one record moves all of them the same way, noise $\nu_i \sim \mathrm{Lap}(c\Delta/\varepsilon_2)$ suffices.

J.5.2. The idea

The proof is the chapter's AboveThreshold proof with c answers $\top$ instead of one. Every $\bot$ answer says that its noisy query is below the noisy threshold. As in the chapter, all of them together are one condition: the largest noisy $\bot$ query is below the threshold. The proof compares the two datasets by a change of variables in the noise. If the threshold noise on D' is shifted by the amount the largest $\bot$ query moved, at most Δ, that condition holds on D' exactly when it held on D. A Laplace density changes by at most a factor e^{ε_1} under such a shift, however many $\bot$ answers there are. Each $\top$ answer needs its own noise shifted to stay above the shifted threshold, by at most 2Δ: Δ for its own query's move and Δ for the threshold's. With noise scale $2c\Delta/\varepsilon_2$, one such shift costs $e^{\varepsilon_2/c}$. There are at most c of them, because the run halts at the c-th, so together they cost e^{ε_2}.

J.5.3. Derivation

The proof reuses four facts of the selection chapter. Pure DP can be checked one output at a time (Proposition 18.1). Noise drawn the same way on both datasets can be fixed and averaged over at the end (Lemma 22.1). A Laplace density with scale b changes by at most a factor $e^{|s|/b}$ when shifted by s (Lemma 22.3). And a maximum of numbers that each move by at most Δ moves by at most Δ (Lemma 22.2). Fix neighbours $D \sim D'$.

1. **Fix an output and the $\perp$ noises.** An output is a sequence of answers that ends either at the c-th $\top$ or at the end of the stream. Let B be the positions of its $\perp$ answers and A those of its $\top$ answers, so A has at most c elements. Fix the noises ν_i for $i \in B$. By Fact 2 it is enough to bound the conditional probability, because these noises do not depend on the data. The queries are fixed too, since each can depend only on earlier answers, which the output fixes.
2. **Write the output as conditions on ξ and the $\top$ noises.** Let $g(D) = \max_{i \in B} (f_i(D) + \nu_i)$ be the largest noisy $\perp$ query. All $\perp$ answers happen exactly when $g(D) < \tau + \xi$. Each $\top$ answer $i \in A$ happens when $f_i(D) + \nu_i \geq \tau + \xi$. Write p_ξ and p_ν for the Laplace densities of the threshold noise and of a query noise. The probability of the output is the integral of these densities over the region where all the conditions hold:

$$\int p_\xi(r)\, \mathbf{1}[g(D) < \tau + r] \prod_{i \in A} \Big(p_\nu(v_i)\, \mathbf{1}[f_i(D) + v_i \geq \tau + r] \Big)\, \mathrm{d}r \prod_{i \in A} \mathrm{d}v_i. \tag{J.23}$$

3. **Shift the noises.** Substitute $r = r' + g(D) - g(D')$ and, for each $i \in A$, $v_i = v_i' + (g(D) - g(D')) - (f_i(D) - f_i(D'))$. The first condition becomes $g(D') < \tau + r'$, and each $\top$ condition becomes $f_i(D') + v_i' \geq \tau + r'$. These are exactly the conditions for D'. The substitution is a translation, so it does not change the volume element. Only the densities change: they are now evaluated at shifted points.
4. **Size of the shifts.** The ν_i for $i \in B$ are fixed, so g is a maximum of numbers that each move by at most Δ, and $|g(D) - g(D')| \leq \Delta$ by Fact 4. Each $\top$ shift is a difference of two terms of size at most Δ, so it is at most 2Δ. If there are no $\perp$ answers, there is no g: shift r by 0 and each v_i by $-(f_i(D) - f_i(D'))$, at most Δ.
5. **Pay for the shifts.** By Fact 3, the threshold noise, with scale Δ/ε_1 and shift at most Δ, costs a factor at most e^{ε_1}. Each $\top$ noise, with scale $2c\Delta/\varepsilon_2$ and shift at most 2Δ, costs at most $e^{2\Delta\varepsilon_2/(2c\Delta)} = e^{\varepsilon_2/c}$. There are at most c of them, so together they cost at most e^{ε_2}. Pulling the factors out of the integral leaves the integral for D':

$$\Pr[\text{output} \mid D, \nu_B] \leq e^{\varepsilon_1 + \varepsilon_2} \Pr[\text{output} \mid D', \nu_B]. \tag{J.24}$$

6. **Collect.** Fact 2 removes the conditioning on ν_B, and Fact 1 turns the bound for every output into $(\varepsilon_1 + \varepsilon_2)$-DP.

Monotone queries. If all queries move the same way, by at most Δ, then g moves the same way as each f_i, also by at most Δ (Fact 4). The $\top$ shift in step 3 is the difference of two numbers of the same sign, each at most Δ in size, so it is at most Δ. With noise scale $c\Delta/\varepsilon_2$ each shift costs $e^{\varepsilon_2/c}$, and the rest is unchanged.

J.5.4. Checking it

The script computes the exact privacy loss of one output on one neighbouring pair, by integrating over the threshold noise on a fine grid, as the selection chapter's script does. The pair extends the counterexample of the selection chapter. On D the first m queries have value 0 and the last c have value 1, while on D' the first m have value 1 and the last c have value 0, with $\Delta = 1$, $\tau = 0$ and $\varepsilon_1 = \varepsilon_2 = 0.5$. The output is m answers $\perp$ followed by c answers $\top$, which is complete because the run halts at the c-th $\top$. On this pair every shift in step 4 can approach its largest size.

Figure J.7 shows the loss against m. For $c = 1$ it is 0.3958 at $m = 1$ and 0.9940 at $m = 10$, which are the chapter's AboveThreshold values. More answers above make the loss climb more slowly, so for $c = 4$ it is only 0.2670 at $m = 1$ and 0.7602 at $m = 10$. But for every c it reaches 1.0000, which is $\varepsilon_1 + \varepsilon_2$, by $m = 40$, and it never goes above, so the bound is tight. The dashed curve shows why the noise must grow with c. With $\nu_i \sim \mathrm{Lap}(2\Delta/\varepsilon_2)$ and $c = 4$, each $\top$ costs a full e^{ε_2}, and the loss reaches $2.5000 = \varepsilon_1 + 4\varepsilon_2$.

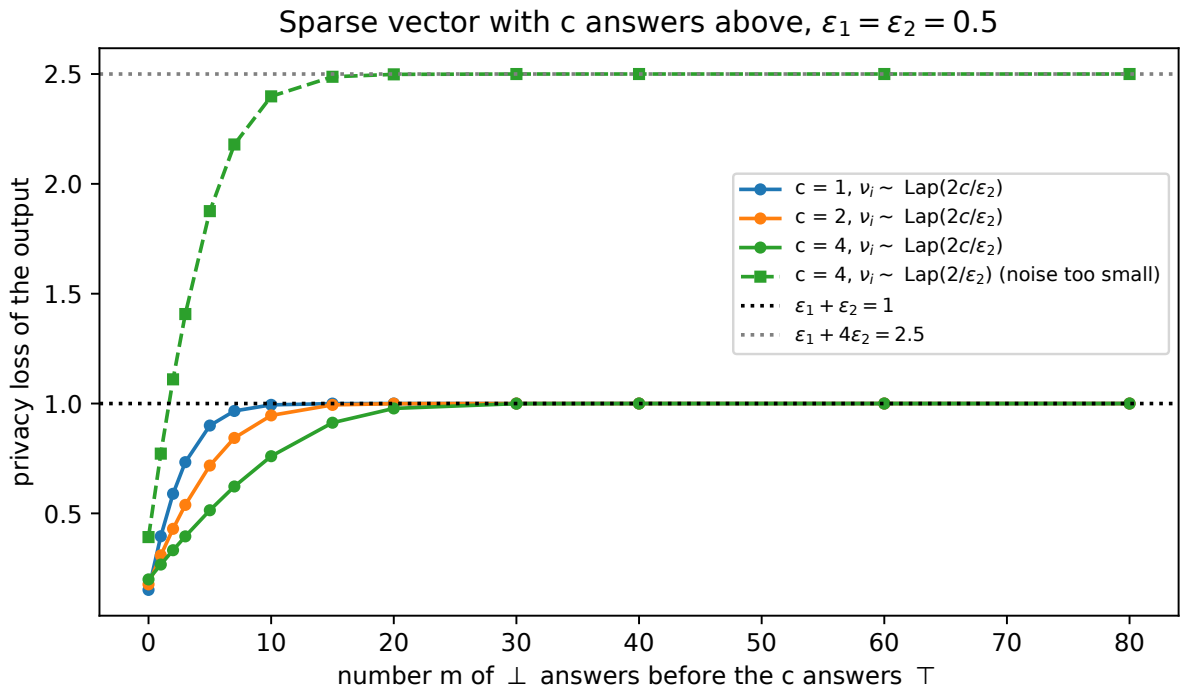


Figure J.7.: The exact privacy loss of the output "m answers $\perp$, then c answers $\top$" on a worst-case neighbouring pair, against m, with $\varepsilon_1 = \varepsilon_2 = 0.5$. With query noise $\mathrm{Lap}(2c/\varepsilon_2)$ the loss approaches $\varepsilon_1 + \varepsilon_2 = 1$ from below for $c = 1$, 2 and 4. With noise $\mathrm{Lap}(2/\varepsilon_2)$ that does not grow with c (dashed, $c = 4$), it approaches $\varepsilon_1 + 4\varepsilon_2 = 2.5$.

J.6. Gumbel top-c is the exponential mechanism without replacement

J.6.1. What the chapter uses

Section 22.5 proves the Gumbel-max trick Equation 22.12. Add independent Gumbel noise of scale κ to each score u_r and take the largest. Then candidate i wins with probability $e^{u_i/\kappa} / \sum_r e^{u_r/\kappa}$, which is the exponential mechanism Equation 22.2 when $\kappa = 2\Delta/\varepsilon$. The chapter then states without proof that "taking the c largest of $u_r + G_r$ gives the same distribution as running the exponential mechanism c times, removing each winner (Durfee & Rogers 2019)", so its privacy is $c\varepsilon$.

Write $\lambda_r = e^{u_r/\kappa}$ for the weight of candidate r. The claim is that for any distinct candidates

$i_1, \dots, i_c$,

$$\Pr\left[\text{the top } c \text{ of } u_r + G_r, \text{ in order, are } i_1, \dots, i_c\right] = \prod_{j=1}^{c} \frac{\lambda_{i_j}}{\sum_{r \notin \{i_1, \dots, i_{j-1}\}} \lambda_r}. \tag{J.25}$$

Each factor is the probability that the exponential mechanism, run on the candidates not yet chosen, picks i_j.

J.6.2. The idea

Turn each noisy score into a waiting time, so that a higher score means an earlier arrival. The time $E_r = e^{-(u_r+G_r)/\kappa}$ does this, because e^{-x} falls as x rises. For Gumbel noise these times turn out to be independent exponential variables, and candidate r's time has rate λ_r: on average it arrives after $1/\lambda_r$. The top c noisy scores are then the first c arrivals of a race.

Exponential races have a simple rule. The chance that r arrives first is its share of the total rate, $\lambda_r / \sum_s \lambda_s$. And exponential waiting times have no memory: once one candidate has arrived, the others' remaining waits are fresh exponential times with the same rates. So the race restarts among the rest, and the next arrival is again picked in proportion to the rates. For example, with rates 3, 2 and 1, the first candidate wins the first round with probability 3/6. The race then restarts between the other two, and the second candidate wins it with probability 2/3. The ordered pair (first, second) therefore has probability $\frac{3}{6} \cdot \frac{2}{3} = \frac{1}{3}$.

J.6.3. Derivation

1. **Arrival times are exponential.** The Gumbel distribution with scale κ has cumulative distribution function $\Pr[G \le g] = \exp(-e^{-g/\kappa})$. For $s > 0$, $E_r > s$ means $u_r + G_r < -\kappa \ln s$, that is $G_r < -\kappa \ln s - u_r$. So

   $$\Pr[E_r > s] = \exp\left(-e^{\ln s + u_r/\kappa}\right) = \exp(-s\lambda_r). \tag{J.26}$$

   That is the survival function of an exponential variable with rate λ_r. The E_r are independent because the G_r are.
2. **Order reverses.** The map $x \mapsto e^{-x/\kappa}$ is decreasing, so the largest noisy score has the smallest arrival time. The top c noisy scores, in decreasing order, are the first c arrivals, in order.
3. **The first arrival.** Let $\Lambda = \sum_r \lambda_r$. Candidate i arrives first at a time near s when E_i is near s and every other E_r exceeds s. Integrating over s,

   $$\Pr[i \text{ first}] = \int_0^\infty \lambda_i e^{-\lambda_i s} \prod_{r \ne i} e^{-\lambda_r s}\, \mathrm{d}s = \int_0^\infty \lambda_i e^{-\Lambda s}\, \mathrm{d}s = \frac{\lambda_i}{\Lambda}. \tag{J.27}$$

4. **No memory.** Given $E_r > s$, the extra wait $E_r - s$ exceeds x with probability $e^{-\lambda_r(s+x)}/e^{-\lambda_r s} = e^{-\lambda_r x}$. So it is again exponential with rate λ_r, whatever s is. The other candidates' times are independent of E_i, so given that i arrives first at time s, their extra waits are independent exponentials with their own rates, and their order does not depend on s.

5. **Restart the race.** By step 4, the probability that i arrives first and the rest arrive in a given order is the integral of step 3 times the probability of that order in a race among the others alone. That is λ_i/Λ times a race with i removed. Applying this c times gives Equation J.25.
6. **The same as c exponential mechanisms.** Round j of the exponential mechanism without replacement picks i_j with probability λ_{i_j} over the sum of the remaining weights. That is the j-th factor of Equation J.25, so the two procedures have the same distribution of ordered outputs.
7. **Privacy.** Each round is the exponential mechanism on a set of candidates fixed by the earlier outputs, with scores of sensitivity Δ, so each is ε-DP. Basic composition, which holds even when each step depends on the earlier outputs (Section 19.3), makes the c rounds $c\varepsilon$-DP. Releasing the top c as an unordered set is post-processing of the ordered list, so it is $c\varepsilon$-DP too.

J.6.4. Checking it

Take six candidates with scores $(4, 3, 3, 1, 0, 0)$ and scale $\kappa = 2$, the chapter's five candidates with a sixth. Equation J.25 gives probabilities for all 30 ordered pairs, and they sum to 1. For candidate 1 followed by candidate 2 it gives 0.1313, and in a million draws of Gumbel noise the top two came out in that order with frequency 0.1313. Over all 30 pairs the largest difference between frequency and formula is 0.0003, about the sampling error of a million draws. Step 1 is checked separately: the average arrival time of each candidate, times its rate, is 1 to within 0.001, as it must be for an exponential variable.

Figure J.8 shows the 30 pairs from a fresh run, and Figure J.9 shows the race. Each curve is the probability that a candidate has not yet arrived by time s. A higher score gives a steeper curve and an earlier typical arrival. One draw's arrivals are marked with their rank, and the first two are the top two noisy scores.

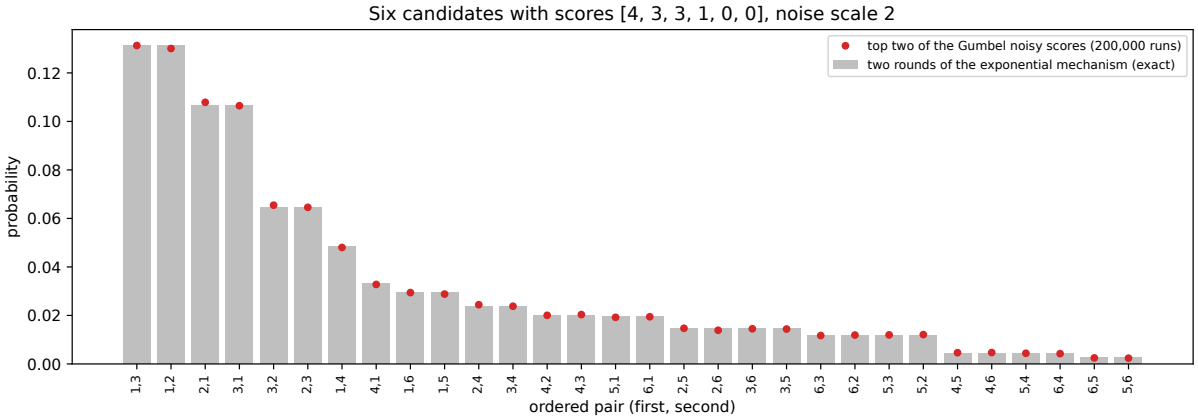


Figure J.8.: Probabilities of the 30 ordered pairs (first, second) for six candidates with scores $(4, 3, 3, 1, 0, 0)$ and Gumbel scale 2. Grey bars: two rounds of the exponential mechanism, Equation J.25. Red dots: frequencies of the top two noisy scores in 200,000 runs.

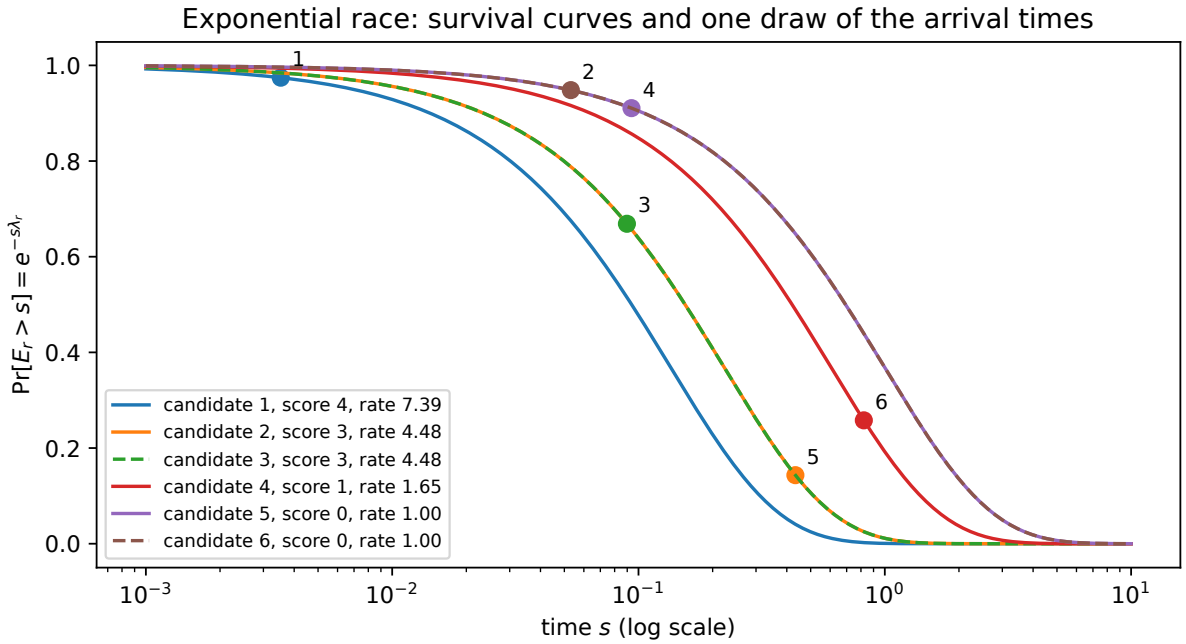


Figure J.9.: The exponential race for the same six candidates. Each curve is $\Pr[E_r > s] = e^{-s\lambda_r}$ with rate $\lambda_r = e^{u_r/2}$, on a log time axis; tied candidates share a curve (one dashed). The dots are one draw of the arrival times $E_r = e^{-(u_r+G_r)/2}$, numbered in order of arrival.

J.7. A Gumbel noisy max is private by the Laplace bar argument

J.7.1. What the chapter uses

Section 22.4 proves that report-noisy-max with Laplace noise is private by a bar argument. Fix two neighbouring datasets D and D', a candidate i, and the noise of every other candidate. Then i wins exactly when its own noise clears a bar $t(D)$, and moving to D' moves the bar by at most 2Δ, or within $[-\Delta, \Delta]$ when the scores are monotone. Lemma 22.3 pays for the move: for Laplace noise Z of scale b, $\Pr[Z > t - s] \le e^{|s|/b} \Pr[Z > t]$. Section 22.5 proves the Gumbel noisy max private in a different way. It shows that the Gumbel noisy max is the exponential mechanism (Theorem 22.5), so it inherits that mechanism's privacy.

This section gives a second proof, which does not need the softmax formula. The bar argument works with Gumbel noise of scale $\kappa = 2\Delta/\varepsilon$ as well, because the Gumbel tail obeys the same bound. For $G \sim \text{Gumbel}(\kappa)$ and all real t and s,

$$\Pr[G > t - s] \le e^{|s|/\kappa} \Pr[G > t]. \tag{J.28}$$

The same proof shows that monotone scores allow the smaller scale $\kappa = \Delta/\varepsilon$.

J.7.2. The idea

The Laplace proof compares densities: shifting the Laplace density by s changes its height by at most $e^{|s|/b}$ anywhere. That fails for the Gumbel density. Its left tail falls like $\exp(-e^{-g/\kappa})$, a double exponential. So the density a fixed distance to the right of a point, divided by the density at the point, grows without bound as the point moves far to the left. But the bar argument never needs the density. It needs only the tail $\Pr[G > t]$, the chance of clearing a bar, and the Gumbel tail is well behaved.

Write the tail as $1 - e^{-y}$, with $y = e^{-t/\kappa}$. Lowering the bar by $s > 0$ multiplies y by $e^{s/\kappa}$. The function $1 - e^{-y}$ is zero at $y = 0$ and bends downward, so multiplying y by a factor of at least 1 multiplies the function by at most that factor. For example, at $y = 1$, doubling y takes $1 - e^{-y}$ from 0.6321 to 0.8647, a factor of only 1.37. Raising the bar instead only lowers the tail.

J.7.3. Derivation

1. **The tail as a function of y.** The Gumbel distribution of scale κ has $\Pr[G \le g] = \exp(-e^{-g/\kappa})$, so with $\phi(y) = 1 - e^{-y}$ and $y(t) = e^{-t/\kappa}$,

$$\Pr[G > t] = 1 - \exp\big(-e^{-t/\kappa}\big) = \phi(y(t)). \tag{J.29}$$

2. **Lowering the bar multiplies y.** Moving the bar from t to $t - s$ gives $y(t-s) = e^{-(t-s)/\kappa} = e^{s/\kappa}\, y(t)$.
3. **A raised bar.** If $s \le 0$, the bar $t - s$ is at least t, so the tail can only fall, and $\Pr[G > t - s] \le \Pr[G > t]$. Since $e^{|s|/\kappa} \ge 1$, Equation J.28 holds.
4. **A lowered bar.** If $s > 0$, write $c = e^{s/\kappa} > 1$. The function ϕ is concave, because its second derivative $-e^{-y}$ is negative, so its graph lies above every chord. The point y lies on the chord from 0 to cy, at the fraction $1/c$ of the way, so

$$\phi(y) \ge \frac{1}{c}\,\phi(cy) + \Big(1 - \frac{1}{c}\Big)\phi(0) = \frac{\phi(cy)}{c}, \tag{J.30}$$

   using $\phi(0) = 0$. So $\phi(cy) \le c\,\phi(y)$, which by steps 1 and 2 is Equation J.28.
5. **The bar argument.** Run the proof of Theorem 22.4 with Gumbel noise G_i in place of the Laplace noise Z_i. Its steps 1 to 4 freeze the other candidates' noise, write the event that i wins as $G_i > t(D)$, and bound the move $s = t(D') - t(D)$ of the bar. None of them uses the shape of the noise, only that it is drawn the same way on both datasets. Step 5 pays for the move with Equation J.28 in place of Lemma 22.3, which gives the factor $e^{2\Delta/\kappa} = e^{\varepsilon}$ at $\kappa = 2\Delta/\varepsilon$. With monotone scores the move lies in $[-\Delta, \Delta]$, and the factor is $e^{\Delta/\kappa} = e^{\varepsilon}$ at $\kappa = \Delta/\varepsilon$.

J.7.4. Checking it

Take $\kappa = 1$ and $s = 1$, the setting in which the chapter checks Lemma 22.3. Table J.3 gives the ratio of the two tails at four bars.

Table J.3.: The factor by which lowering the bar by 1 raises the tail of Gumbel(1), against the bound $e = 2.7183$.

Bar t	-2	0	2	5
$\Pr[G > t-1]/\Pr[G > t]$	1.0006	1.4776	2.4317	2.7026

The ratio climbs towards e as the bar moves right, but never reaches it. Far to the right y is small, and there $1 - e^{-y}$ is nearly y itself, so multiplying y by e nearly multiplies the tail by e. Far to the left both tails are close to 1, so their ratio is too. On a grid of bars from -10 to 20 and shifts from -5 to 5, the ratio never exceeds $e^{|s|}$. The Laplace tail, by contrast, reaches the factor e exactly once both bars are above 0, as the chapter shows. Figure J.10 draws both ratios against the bar.

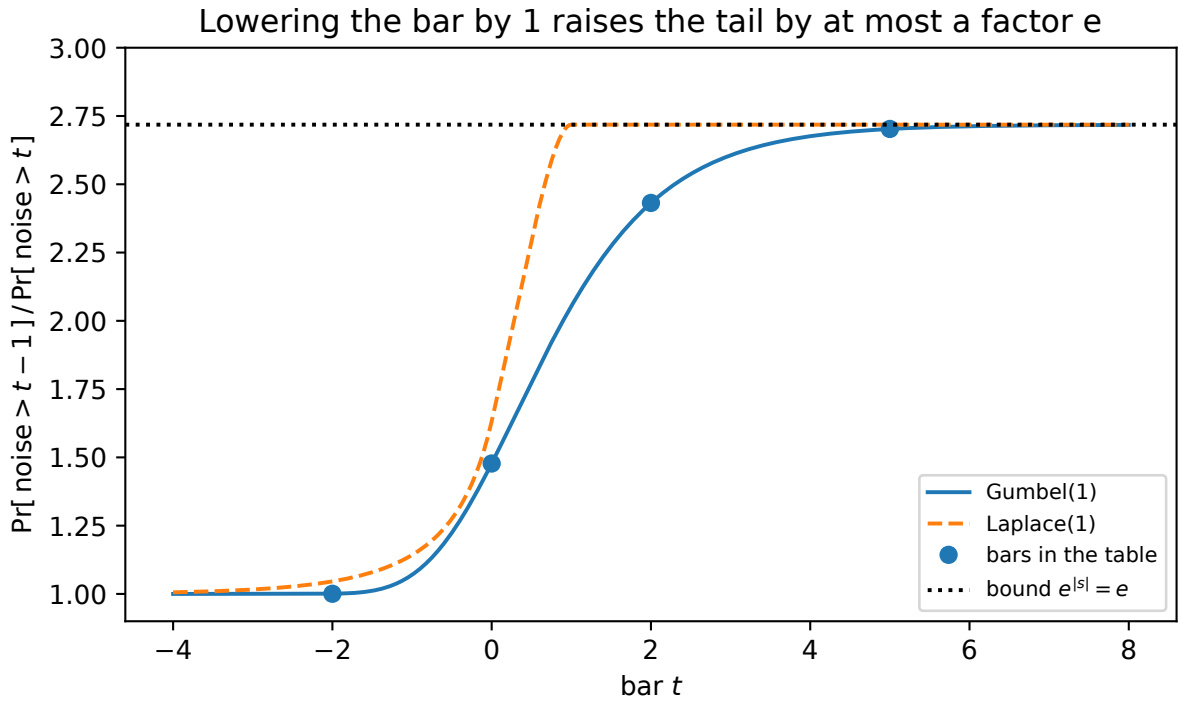


Figure J.10.: The factor $\Pr[\text{noise} > t-1]/\Pr[\text{noise} > t]$ against the bar t, for Gumbel(1) noise (solid, with the four bars of Table J.3 as dots) and for Lap(1) noise (dashed), with the bound e (dotted). The Gumbel factor rises towards e without reaching it; the Laplace factor equals e from $t = 1$ on.

The chapter's picture of the bar argument lowers the bar from 0.7 by 1. With Gumbel noise of scale 1, the winning probability rises from 0.3914 to 0.7407, a ratio of 1.8925, below e as the bound requires.

J.7.5. What it does not cover

The bar argument covers the release of one winner. Releasing the top c noisy scores needs the race argument of Section J.6, which shows that it is c exponential mechanisms in a row. The argument also covers only the index of the winner, as for Laplace noise. Releasing the winning noisy score as well would publish G_i, the noise the proof averages over.

J.8. The alarm mechanism is (ε, δ)-DP, with one condition

J.8.1. What the chapter uses

Section 19.2 shows that the group-privacy bound Equation 19.6 is attained, using a mechanism whose input is a count, where neighbouring counts differ by 1. The mechanism has a parameter K, the largest group it is built for, and on a count $c \le K$ it outputs "alarm" with probability

$$a_c = \delta \sum_{j=0}^{K-c-1} e^{j\varepsilon} = \delta \, \frac{e^{(K-c)\varepsilon} - 1}{e^{\varepsilon} - 1}, \tag{J.31}$$

and "quiet" otherwise. Above K it always outputs "quiet", so $a_c = 0$ for $c \ge K$. The chapter shows that the event "alarm" meets the definition with equality, and states the condition on K under which the other events are also within (ε, δ). This section proves that condition.

For $\varepsilon > 0$ and $K \ge 1$, the alarm mechanism is (ε, δ)-DP if and only if

$$a_0 \le \frac{e^{\varepsilon} + \delta}{e^{\varepsilon} + 1}. \tag{J.32}$$

Since a_0 grows with K, this caps K.

J.8.2. The idea

There are only two outputs, so there are only two events to check, "alarm" and "quiet", each in both orders of a pair of neighbouring counts. The alarm probabilities fall as the count rises, by the rule $a_c = e^{\varepsilon} a_{c+1} + \delta$. Three of the four checks then hold for any K. The fourth asks that "quiet" on the higher count is not too much likelier than on the lower one. If a_0 is close to 1, "quiet" is rare on count 0. Since $a_1 = (a_0 - \delta)/e^{\varepsilon}$ is smaller by about a factor e^{ε}, "quiet" is much more common on count 1, and the ratio of the two "quiet" probabilities breaks the bound.

J.8.3. Derivation

Fix neighbouring counts c and $c+1$. The alarm probability has fewer terms at $c+1$, so $a_c \ge a_{c+1}$. For $c \ge K$ both are 0, the two output distributions are equal, and there is nothing to check. So take $c \le K - 1$, where splitting off the first term of the sum gives

$$a_c = e^{\varepsilon} a_{c+1} + \delta. \tag{J.33}$$

The empty event has probability 0 on both sides, and the event "alarm or quiet" has probability 1 on both sides, which is at most $e^{\varepsilon} + \delta$. So both pass, and four checks remain.

1. **"Alarm", count c against $c+1$.** The definition asks $a_c \le e^\varepsilon a_{c+1} + \delta$. This holds with equality, by the rule above.
2. **"Alarm", count $c+1$ against c.** It asks $a_{c+1} \le e^\varepsilon a_c + \delta$, which holds because $a_{c+1} \le a_c$.
3. **"Quiet", count c against $c+1$.** It asks $1 - a_c \le e^\varepsilon(1 - a_{c+1}) + \delta$. This holds because $1 - a_c \le 1 - a_{c+1}$ and $1 - a_{c+1} \ge 0$.
4. **"Quiet", count $c+1$ against c.** It asks $1 - a_{c+1} \le e^\varepsilon(1 - a_c) + \delta$. Substitute the rule for a_c. The right side becomes $e^\varepsilon - e^{2\varepsilon} a_{c+1} - (e^\varepsilon - 1)\delta$. Moving terms across, the check becomes $(e^{2\varepsilon} - 1)a_{c+1} \le (e^\varepsilon - 1)(1 - \delta)$. Since $e^{2\varepsilon} - 1 = (e^\varepsilon - 1)(e^\varepsilon + 1)$ and $e^\varepsilon - 1 > 0$, it is

$$a_{c+1} \le \frac{1 - \delta}{e^\varepsilon + 1}. \tag{J.34}$$

5. **The worst pair.** The largest a_{c+1} with $c \le K - 1$ is a_1, so all checks pass exactly when $a_1 \le (1 - \delta)/(e^\varepsilon + 1)$. By the rule, $a_0 = e^\varepsilon a_1 + \delta$, and this condition is the same as $a_0 \le (e^\varepsilon(1 - \delta) + \delta(e^\varepsilon + 1))/(e^\varepsilon + 1)$. The numerator simplifies to $e^\varepsilon + \delta$, which gives Equation J.32. The right side of Equation J.32 is below 1, so the condition also makes a_0 a valid probability.

At the cap the pair of counts 0 and 1 is exactly the four-point pair of Section J.1, with "flag D" merged into report 0 and "flag D'" merged into report 1. Then $a_0 = \delta + (1 - \delta)t$ and $a_1 = (1 - \delta)(1 - t)$, with $t = e^\varepsilon/(1 + e^\varepsilon)$, as extreme as (ε, δ)-DP allows.

J.8.4. Checking it

In the chapter's example, $\varepsilon = 0.5$, $\delta = 10^{-5}$ and $K = 10$. The cap is $(e^{0.5} + 10^{-5})/(e^{0.5} + 1) = 0.6225$, and $a_0 = 2.272 \times 10^{-3}$, far below it. So the chapter's mechanism is $(0.5, 10^{-5})$-DP, and the script's check over every event agrees: the largest amount by which any event exceeds the bound is 2.5×10^{-20}, which is rounding on the event that holds with equality. The largest allowed size at these parameters is $K = 21$, where $a_0 = 0.5598$. At $K = 22$, $a_0 = 0.9229$ is still a probability, but the event "quiet" on counts 1 against 0 exceeds the bound by 0.3132. So the condition is real, and it does not follow from $a_0 \le 1$.

J.8.5. What it does not cover

The derivation divides by $e^\varepsilon - 1$ in step 4, so it needs $\varepsilon > 0$. At $\varepsilon = 0$ the rule reads $a_c = a_{c+1} + \delta$, check 4 asks $a_c - a_{c+1} \le \delta$, which holds with equality, and the only condition left is $a_0 \le 1$.

J.9. Code

The script apx_mechanisms.py computes every number on this page and draws its figures. It builds the four-point channel and checks it on 6,000 random pairs. It computes the stability-based histogram's error exactly and by simulation, and the sparse vector technique's privacy loss by numerical integration. It also evaluates the heavy-tail lower bound, compares Gumbel top-two selection with the exponential mechanism, and checks every event of the alarm mechanism. From the repository root:

```
uv run python code/appendix-derivations/apx_mechanisms.py
```

Table J.4 shows the output with seed 0.

Table J.4.: The script's output with seed 0.

Check	Result
Four-point channel, three-output example	$\omega^* = 0.2222$; $P^\star K = (0.6, 0.3, 0.1)$ and $Q^\star K = (0.2, 0.3, 0.5)$ exactly
Four-point channel, binned normal pair at $\varepsilon = 1$	$\delta = 0.2259$; masses 0.7741 and 0.9156; $\omega^* = 0.2243$
Four-point channel, 6,000 random pairs	reproduces P and Q to 2.2×10^{-16}; smallest channel entry -1.4×10^{-13} (rounding)
Pure parts as zCDP	largest $D_\alpha/(\alpha\varepsilon^2/2)$ is 0.4275 and 0.4306
Stability-based histogram, $\varepsilon = 1$, $\delta = 10^{-6}$, 10 bins	$\tau = 14.1224$; error bound 19.4207; exact worst 18.0000, simulated 18.0000
Laplace histogram over N bins	16.7857 at $N = 10^6$, 30.6012 at $N = 10^{12}$; above the stability bound from $N = 1.394 \times 10^7$
Heavy-tail lower bound, $a = 0.05$	172.8 records for $k = 2$, 55.6 for $k = 3$, 38.4 for $k = 4$, 18.6 for bounded data; the clipped mean uses 7,616
Sparse vector, $c = 1, 2, 4$	loss 0.9940, 0.9459, 0.7602 at $m = 10$; 1.0000 at $m = 40$; 2.5000 with noise that ignores c
Gumbel top two, six candidates	largest difference from Equation J.25 0.0003 over 30 pairs, from 10^6 draws
Gumbel tail, bar lowered by 1, at $t = -2, 0, 2, 5$	1.0006, 1.4776, 2.4317, 2.7026, against $e = 2.7183$
Gumbel winning probability, bar lowered from 0.7 by 1	0.3914 to 0.7407, a ratio of 1.8925
Gumbel tail ratio over $e^{\lvert s\rvert}$, bars -10 to 20, shifts -5 to 5	largest 1.0000, reached at $s = 0$; never above 1
Alarm mechanism, $\varepsilon = 0.5$, $\delta = 10^{-5}$	cap 0.6225; $a_0 = 2.272 \times 10^{-3}$ at $K = 10$; largest K is 21; at $K = 22$ an event fails by 0.3132

tests/test_appendix_mechanisms.py checks that the output still matches this table.

What to take from it. The four-point construction is exact: it reproduces every pair to rounding, including pairs whose outputs only one side can produce. The stability-based histogram's error is flat in N, and its bound is within about 1.4 records of the exact worst case, so what approximate DP costs here is the constant $\ln(1/\delta)/\varepsilon$, in place of a $\ln N$ that grows with the universe. The sparse vector proof is tight: on a worst-case pair the loss approaches $\varepsilon_1 + \varepsilon_2$ for every c, and noise that ignores c really does cost c times more. And the alarm mechanism's hidden condition matters at $K = 22$ but not at the chapter's $K = 10$.

J.10. References

- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015; IEEE Transactions on Information Theory, 2017): the four-point pair, the channel for $\delta > 0$, and optimal composition.
- Blackwell, "Equivalent Comparisons of Experiments" (Annals of Mathematical Statistics, 1953): one experiment is a post-processing of another exactly when it is harder to test at every level.
- Bun & Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (TCC 2016): approximate zCDP, Proposition 1.13 and the general composition Lemma 8.2.
- Vadhan, "The Complexity of Differential Privacy" (in *Tutorials on the Foundations of Cryptography*, 2017): the stability-based histogram and its accuracy, in the textbook form used here.
- Karwa & Vadhan, "Finite Sample Differentially Private Confidence Intervals" (ITCS 2018): a stability-based histogram as the coarse step of mean estimation without a range bound.
- Kamath, Singhal & Ullman, "Private Mean Estimation of Heavy-Tailed Distributions" (COLT 2020): upper and lower bounds for distributions with a bounded k-th moment.
- Lyu, Su & Li, "Understanding the Sparse Vector Technique for Differential Privacy" (VLDB 2017): Algorithm 1 with c answers above, its proof, and the broken variants.
- Durfee & Rogers, "Practical Differentially Private Top-k Selection with Pay-what-you-get Composition" (NeurIPS 2019): Gumbel top-k as repeated exponential mechanisms.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014): AboveThreshold, the exponential mechanism and group privacy.

K. Composition of trade-off functions

Two chapters of Part 4 compose differential privacy (DP) guarantees step by step. They are f-DP and Gaussian DP and Numerical accounting, and they lean on five results that they state but do not prove, and this appendix derives all five. Each result is about how well an attacker can tell two output distributions apart, which the chapters measure by a curve of the attacker's two kinds of error, the trade-off curve defined below.

The first result says that any curve with the four properties of a trade-off curve really is the curve of some pair of distributions. The next two say that running mechanisms one after another, even when each one is chosen after seeing the earlier outputs, leaks no more than running fixed, independent copies of the worst step. One of them says it with trade-off curves and the other with hockey-stick divergences, a second measure of the same leak. The last two are central limit theorems: when there are many small steps, the composed curve comes close to the curve of two normal distributions with the same variance. The first three results share one move. You take a test that an attacker runs on one pair of distributions and replace it, piece by piece, by a test on another pair that does at least as well. The central limit theorems instead follow the best tests of the composed pair and watch their errors approach those of the two normal distributions.

Table K.1 lists the five derivations and where the book states or uses each one. Each section restates the terms it needs, so the table only points the way.

Table K.1.: The derivations of this appendix and the chapter sections that state or use them.

Derivation	What it proves	Stated or used in
Section K.1	Every convex, non-increasing, continuous $f \le 1 - \alpha$ is the trade-off curve of a pair	Section 4.3, Section 32.1, Section 32.4
Section K.2	Composition of f-DP, adaptive steps included, and the tensor product is well defined	Section 32.4 (Equation 32.11)
Section K.3	Dominating pairs survive adaptive composition	Section 34.2, Section 34.3, Section 37.1
Section K.4	The f-DP central limit theorem with its Berry–Esseen band	Section 32.5 (Equation 32.15)
Section K.5	The central limit theorem for the Poisson-subsampled Gaussian	Section 32.6 (Equation 32.17), Section 34.6, Section 37.1

Notation. The notation is that of f-DP and Gaussian DP. A mechanism M runs either on a dataset D or on a neighbouring dataset D', written $D \sim D'$, which adds or removes one record (the results hold for any neighbouring relation). Write $P = M(D)$ for the distribution of its output on D and $Q = M(D')$ for the distribution on D'. Their densities, or probabilities for discrete outputs, are p and q.

A **test** ϕ looks at the output y and fires, deciding that the data was D', with probability $\phi(y)$. How often it fires on D is its type I error $\alpha = \mathbb{E}_P[\phi]$. How often it fires on D' is its **power** $\mathbb{E}_Q[\phi]$, and how often it stays silent there is its type II error $\beta = 1 - \mathbb{E}_Q[\phi]$.

The **trade-off function** $T(P, Q)$ gives, at each type I error α, the smallest type II error that any test with type I error at most α can reach. A **pair** always means such a pair of distributions (P, Q), and its curve means $T(P, Q)$. A mechanism is f-DP when the curve of every neighbouring pair lies on or above the curve f, so that no test does better than f allows. The privacy loss of an output measures how much more likely the output is on D than on D',

$$L(y) = \ln \frac{p(y)}{q(y)}. \tag{K.1}$$

One fact from Hypothesis testing carries most of the weight, so here it is once. The **Neyman–Pearson lemma** (Section 4.2) says that the best test at any type I error is a likelihood-ratio test. Such a test fires where q/p is above a threshold and stays silent where it is below. On outputs exactly at the threshold it may fire with any probability, and this coin flip is what lets the test spend its type I error exactly when the threshold outputs carry positive mass. For every $\alpha \in (0, 1]$ such a test exists with type I error exactly α, and its type II error is $T(P, Q)(\alpha)$. At $\alpha = 0$ the best test fires only on outputs that P never produces. In terms of the loss, $q/p = e^{-L}$, so the best tests fire when L is small.

K.1. Every trade-off function has a canonical pair

K.1.1. What the chapter uses

f-DP and Gaussian DP calls f a trade-off function when it is the curve $T(P, Q)$ of some pair (Section 32.1). Every such curve has four properties: it is non-increasing, convex and continuous, and it lies on or below the line $1 - \alpha$ of the test that ignores the output and fires at random. The composition theorem of the next section needs the converse. It starts from curves $f_1, \dots, f_k$, one per step, that are only known to have the four properties, and it needs a pair of distributions behind each one. The converse says that the four properties are enough, and it builds the pair.

Let $f\colon [0, 1] \to [0, 1]$ have the four properties. Let P_f be the uniform distribution on $[0, 1]$, and let Q_f put mass $f(1 - x)$ on each interval $[0, x]$,

$$Q_f([0, x]) = f(1 - x) \quad \text{for } x \in [0, 1]. \tag{K.2}$$

This may leave Q_f short of total mass 1. So Q_f also has an **atom**, a single point carrying positive probability, at a point outside $[0, 1]$, say at 2, and the atom carries the missing mass $1 - f(0)$. Then

$$T(P_f, Q_f) = f. \tag{K.3}$$

The pair (P_f, Q_f) is the **canonical pair** of f. Hypothesis testing already gives this construction in one paragraph of Section 4.3. The f-DP chapter cites only the forward statement, that every curve of a pair has the four properties. This section spells the converse out step by step, because the next section depends on it.

K.1.2. The idea

Take P_f uniform, so that a test which fires on an interval of length α has type I error exactly α. Then shape Q_f so that firing on the top interval $(1-\alpha, 1]$ misses exactly $f(\alpha)$ of Q_f. That forces the mass of Q_f below $1-\alpha$ to be $f(\alpha)$, which is the formula above. A value $f(0)$ below 1 means that some test reaches type II error $f(0) < 1$ with no type I error at all. So Q_f needs mass $1 - f(0)$ in a place where P_f has none, and the atom outside $[0, 1]$ supplies exactly that mass. What remains is to check that these top-interval tests are the best tests. Step 2 below shows that convexity makes the density of Q_f rise from left to right. Since P_f has the flat density 1, the likelihood ratio rises too, and the Neyman–Pearson lemma says the best tests fire where the ratio is highest, which is at the top.

A Gaussian example makes the construction concrete. The Gaussian curve G_μ is the curve of the normal distribution $\mathcal{N}(0, 1)$ against the normal distribution $\mathcal{N}(\mu, 1)$ with its mean shifted by μ. A mechanism whose neighbouring pairs all have curves on or above G_μ satisfies Gaussian differential privacy (GDP) with parameter μ, written μ-GDP. For $\mu = 1$ the canonical pair is easy to recognize. Let Z be standard normal and Φ its distribution function. Then P_f is the law, meaning the distribution, of $\Phi(Z)$, which is uniform on $[0, 1]$ because for every $x \in [0, 1]$

$$\Pr[\Phi(Z) \le x] = \Pr[Z \le \Phi^{-1}(x)] = x. \tag{K.4}$$

And Q_f is the law of $\Phi(Z + 1)$. Applying the increasing function Φ to both $\mathcal{N}(0, 1)$ and $\mathcal{N}(1, 1)$ is a one-to-one relabelling of the outputs, so it changes no test's errors and the pair still has curve G_1. Figure K.1 shows this pair and the test at $\alpha = 0.1$.

K.1.3. Derivation

1. **Q_f is a probability distribution.** Write $F(x) = f(1-x)$. As x grows, $1-x$ shrinks, and f is non-increasing, so F is non-decreasing. Property $f(\alpha) \le 1-\alpha$ at $\alpha = 1$ forces $f(1) = 0$, so $F(0) = 0$, and $F(1) = f(0) \le 1$. The atom adds the missing mass $1 - f(0)$, so the total is 1.
2. **Q_f has a non-decreasing density on $[0, 1]$.** This step uses three standard facts about a convex function, which are cited (Rockafellar 1970, §§23–24). It has a slope $f'(\alpha)$ at every point except at most countably many kinks, its slope never decreases as α grows, and it is the integral of its slope. So F has density $F'(x) = -f'(1-x)$, which is minus the slope of f at $1-x$. Now let x grow. Then $1-x$ shrinks, and because the slope of f never decreases as its argument grows, it can only fall, that is become more negative, as its argument shrinks. So $f'(1-x)$ can only become more negative, and the density $-f'(1-x)$ can only grow. At a kink the density has a jump, and either one-sided value can stand for it.
3. **The likelihood ratio is non-decreasing.** The density of P_f is 1 on $[0, 1]$, so the likelihood ratio q/p equals the density of Q_f there. At the atom, P_f has no mass, so the ratio is infinite.

4. **The top-interval tests are likelihood-ratio tests.** Fix $\alpha \in (0, 1]$ and let ϕ_α fire on the atom and on every $x > 1 - \alpha$. Take the threshold τ to be the density just to the left of $1 - \alpha$, which is well defined even when $1 - \alpha$ is a kink, because a non-decreasing function has a limit from each side. By step 3, the ratio is at least τ wherever ϕ_α fires and at most τ wherever it is silent. Outputs with ratio exactly τ are ties, and the lemma allows any behaviour on ties. So ϕ_α is a likelihood-ratio test, and no test with type I error at most α has a smaller type II error.
5. **Read off the errors.** The type I error of ϕ_α is the uniform mass of $(1 - \alpha, 1]$, which is α, since the atom has no P_f-mass. Its type II error is the Q_f-mass where it is silent, $Q_f([0, 1 - \alpha]) = F(1 - \alpha) = f(\alpha)$. So $T(P_f, Q_f)(\alpha) = f(\alpha)$ for $\alpha \in (0, 1]$. At $\alpha = 0$ the best test fires on the atom only, and it misses $F(1) = f(0)$.

K.1.4. Checking it

The script of this appendix, apx_composition.py, coarsens each canonical pair into 1000 equal cells of $[0, 1]$, plus the atom. It then computes the trade-off curve of this finite pair exactly, by sorting the cells by likelihood ratio. Reporting only the cell that the output fell in is post-processing, a function applied to the output, and no test can gain from seeing less, so the coarse curve can only lie above f. It still touches f at the cell edges, because a test that fires on the top cells fires on a top interval, and that test is still available with the same errors. For $f = G_1$ the largest gap is 2.0×10^{-3}, near $\alpha = 0$, where G_1 is steepest. The second example is the curve of an (ε, δ)-DP guarantee, which the chapter writes as a curve made of two straight lines,

$$f_{\varepsilon,\delta}(\alpha) = \max\left(0,\ 1 - \delta - e^{\varepsilon}\alpha,\ e^{-\varepsilon}(1 - \delta - \alpha)\right). \tag{K.5}$$

With $\varepsilon = 1$ and $\delta = 0.1$ its largest gap is 8.3×10^{-5}. Both gaps shrink as the cells shrink, which is what Equation K.3 predicts.

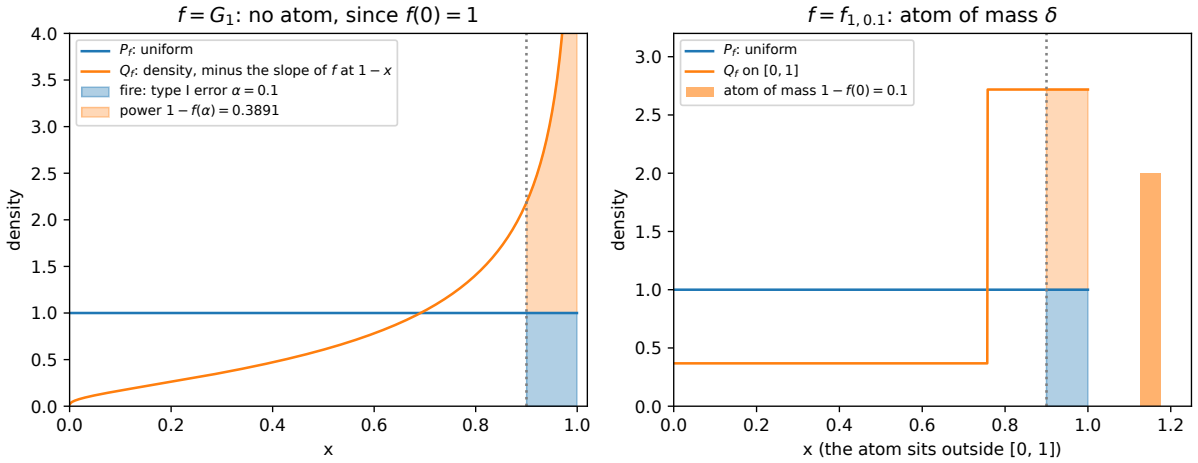


Figure K.1.: The canonical pair of two trade-off functions. Left: for $f = G_1$, the uniform P_f and the density of Q_f, which is minus the slope of f at $1 - x$; the test that fires on $(0.9, 1]$ has type I error 0.1 (blue area) and power $1 - G_1(0.1) = 0.3891$ (orange area). Right: for $f_{1,0.1}$, the density of Q_f steps from e^{-1} to e at $x = 1 - 0.2420$, and the missing mass $\delta = 0.1$ sits in an atom outside $[0, 1]$ that every good test fires on.

In Figure K.1 (right), the density of Q_f on $[0,1]$ has two levels, e^{-1} and e, which are minus the two slopes $-e^{-1}$ and $-e$ of $f_{1,0.1}$. The step sits at $1-\alpha^*$, where α^* is the corner of $f_{1,0.1}$, the point where its two lines meet. Setting $1-\delta-e\,\alpha = e^{-1}(1-\delta-\alpha)$ with $\delta = 0.1$ gives $\alpha^* = 0.9/(1+e) = 0.2420$. The atom carries the $\delta = 0.1$ of mass that a test can catch with no type I error.

K.2. Composition of f-DP, adaptive steps included

K.2.1. What the chapter uses

When an attacker sees the outputs of two mechanisms, it faces two joint distributions, one for D and one for D'. The simplest case is two independent mechanisms. Their joint output then has the **product** distribution $P \times P'$, under which the two coordinates are drawn independently, one from P and one from P'. The chapter defines the **tensor product** of two trade-off functions $f = T(P,Q)$ and $g = T(P',Q')$ as the curve of the product pair,

$$f \otimes g = T(P \times P', Q \times Q'). \tag{K.6}$$

The composition theorem then says that composing f_i-DP mechanisms gives an $f_1 \otimes \cdots \otimes f_k$-DP mechanism, even when each mechanism is chosen after seeing the earlier outputs. In symbols, suppose M_1 is f_1-DP and, for every value y_1 of its output, the second mechanism $M_2(\cdot\,; y_1)$ is f_2-DP. Then the pair of outputs (y_1, y_2) satisfies

$$T\big((M_1,M_2)(D),(M_1,M_2)(D')\big) \geq f_1 \otimes f_2 \quad \text{for all neighbours } D \sim D', \tag{K.7}$$

and the same holds for k steps with $f_1 \otimes \cdots \otimes f_k$. Section 32.4 states this theorem (Dong, Roth & Su 2022, Theorem 3.2) and leaves three gaps. The chapter proves the non-adaptive case through Blackwell's theorem, a cited result that compares two pairs through post-processing. For the adaptive case, the chapter reports that Dong et al. prove it "with the Neyman–Pearson lemma alone" and uses the result without a proof. Finally, the chapter cites without proof the fact that $f \otimes g$ does not depend on which pairs represent f and g (their Definition 3.1), which is what makes the tensor product well defined. This section proves the non-adaptive case, the adaptive case and this independence of the pairs, using only the Neyman–Pearson lemma.

K.2.2. The idea

Take the adaptive example of Rényi DP (Table 29.5), which uses randomized response, a mechanism that reports a person's bit truthfully with some keep-probability and flips it otherwise. One person's bit is 1 in D and 0 in D'. The first step keeps the bit with probability 0.75, and the second step's keep-probability depends on the first output, 0.75 if it was 1 and 0.6 if it was 0. Each step's curve lies on or above f, the curve of randomized response with keep-probability 0.75. The reason is that a keep-probability closer to $\frac{1}{2}$ makes the report closer to a fair coin, and so hides the bit better. So the theorem promises that the joint curve lies on or above $f \otimes f$, the curve of two independent runs with keep-probability 0.75. Table K.2 lists the four joint outputs.

Table K.2.: Two adaptive randomized responses: keep-probability 0.75, then 0.75 or 0.6 depending on y_1.

(y_1, y_2)	p, bit 1 (D)	q, bit 0 (D')	q/p
(1, 1)	0.5625	0.0625	0.111
(1, 0)	0.1875	0.1875	1
(0, 1)	0.15	0.3	2
(0, 0)	0.1	0.45	4.5

An attacker's joint test can be read one branch at a time, where a branch is one value of the first output. For each first output y_1, what the test does with y_2 is a test of the second step's pair on that branch. The plan is to swap each branch's pair for a fixed pair whose curve is exactly f, here randomized response with keep-probability 0.75 itself, and each branch's test for the best test of that fixed pair at the same type I error. The type I error does not change, and the type II error cannot grow, because f lies on or below that branch's curve. After the swap every branch uses the same fixed pair, so the second output no longer depends on the first, and the joint pair is a product. The first coordinate is then swapped in the same way, now one value of the second output at a time. The result is a test on the product pair that is at least as good as the attacker's test, so no attacker beats $f \otimes f$.

Here it is on one test. The test that fires only on $(0, 0)$ has type I error $p(0, 0) = 0.1$ and type II error $1 - 0.45 = 0.55$. On the branch $y_1 = 1$ it never fires, and on the branch $y_1 = 0$ it fires when $y_2 = 0$. That branch keeps the bit with probability 0.6, so the branch test fires with probability 0.4 on D and 0.6 on D'. The best test of the fixed pair at type I error 0.4 always fires on output 0, which has probability 0.25 on D and 0.75 on D'. To spend the remaining 0.15 of type I error, it also fires on output 1 with probability 0.2, since output 1 has probability 0.75 on D. That adds $0.2 \times 0.25 = 0.05$ of power, so the test's power is $0.75 + 0.05 = 0.8$, more than the branch test's 0.6.

Swapping it in, the joint type I error stays $0.25 \times 0.4 = 0.1$, since the first output is 0 with probability 0.25 on D. The power becomes $0.75 \times 0.8 = 0.6$, since the first output is 0 with probability 0.75 on D'. So the type II error drops from 0.55 to 0.4, which is exactly the value of $f \otimes f$ at 0.1. To check that value, take the best test of the product pair at type I error 0.1. It fires on $(0, 0)$, which costs 0.0625 of type I error and brings 0.5625 of power. It spends the remaining 0.0375 on the two mixed outputs, where $q/p = 1$, so they add 0.0375 of power, and the type II error is $1 - 0.6 = 0.4$.

K.2.3. Derivation

Write the two joint distributions with the first output's law and the second output's law given the first: $P(y_1, y_2) = P_1(y_1)P_2(y_2 \mid y_1)$ and $Q(y_1, y_2) = Q_1(y_1)Q_2(y_2 \mid y_1)$. The assumption is that $T(P_1, Q_1) \geq f_1$, and that $T(P_2(\cdot \mid y_1), Q_2(\cdot \mid y_1)) \geq f_2$ for every y_1. By Section K.1 there are pairs $(\tilde{P}_i, \tilde{Q}_i)$ with curve exactly f_i, for example the canonical pairs.

1. **The replacement step.** One pair can be swapped for another whose curve is lower, one test at a time. Suppose $T(P, Q) \geq T(\tilde{P}, \tilde{Q})$. Then for every test ϕ of P against Q there is a

test $\tilde{\phi}$ of $\tilde{P}$ against $\tilde{Q}$ with

$$\mathbb{E}_{\tilde{P}}[\tilde{\phi}] = \mathbb{E}_P[\phi] \qquad \text{and} \qquad 1 - \mathbb{E}_{\tilde{Q}}[\tilde{\phi}] \le 1 - \mathbb{E}_Q[\phi]. \tag{K.8}$$

In words, $\tilde{\phi}$ has the same type I error and a type II error no larger. To find it, let $\alpha = \mathbb{E}_P[\phi]$ and take $\tilde{\phi}$ to be the likelihood-ratio test of $\tilde{P}$ against $\tilde{Q}$ with type I error exactly α, which exists by the Neyman–Pearson lemma. For $\alpha = 0$ it is the test that fires on every output where $\tilde{p} = 0$, which is best because firing anywhere else would cost type I error. Its type II error is $T(\tilde{P}, \tilde{Q})(\alpha)$, which is at most $T(P, Q)(\alpha)$ by assumption. That is at most the type II error of ϕ, because ϕ is one of the tests the minimum in $T(P, Q)(\alpha)$ ranges over.

2. **Swap the second coordinate, one branch at a time.** Fix any joint test $\phi(y_1, y_2)$. For a fixed y_1, the function $\phi(y_1, \cdot)$ is a test of $P_2(\cdot \mid y_1)$ against $Q_2(\cdot \mid y_1)$, and that pair's curve is at least $f_2 = T(\tilde{P}_2, \tilde{Q}_2)$. Step 1 gives a test $\tilde{\phi}_{y_1}$ of $\tilde{P}_2$ against $\tilde{Q}_2$ with the same type I error on this branch and at least the same power. Now replace the second output too. Write z_2 for an output of the fixed pair, drawn from $\tilde{P}_2$ or $\tilde{Q}_2$ independently of y_1, and define the new joint test $\phi'(y_1, z_2) = \tilde{\phi}_{y_1}(z_2)$. It is a test of the pair $P_1 \times \tilde{P}_2$ against $Q_1 \times \tilde{Q}_2$, in which the second output no longer depends on the first.
3. **Average over the first output.** The joint type I error of ϕ is the average over $y_1 \sim P_1$ of the branch type I errors, and step 2 kept each of these. So ϕ' has the same joint type I error:
$$\mathbb{E}_{P_1 \times \tilde{P}_2}[\phi'] = \mathbb{E}_{y_1 \sim P_1}\big[\mathbb{E}_{\tilde{P}_2}\tilde{\phi}_{y_1}\big] = \mathbb{E}_{y_1 \sim P_1}\big[\mathbb{E}_{P_2(\cdot \mid y_1)}\phi(y_1, \cdot)\big] = \mathbb{E}_P[\phi]. \tag{K.9}$$
The joint power is the average over $y_1 \sim Q_1$ of the branch powers, and step 2 raised or kept each of them. So $\mathbb{E}_{Q_1 \times \tilde{Q}_2}[\phi'] \ge \mathbb{E}_Q[\phi]$, and the joint type II error did not grow.
4. **Swap the first coordinate.** The pair is now a product. So for each fixed z_2, the function $\phi'(\cdot, z_2)$ is a test of P_1 against Q_1, whatever z_2 is. Step 1, with $T(P_1, Q_1) \ge f_1 = T(\tilde{P}_1, \tilde{Q}_1)$, swaps it for a test of $\tilde{P}_1$ against $\tilde{Q}_1$ with the same type I error and at least the same power. Averaging over $z_2 \sim \tilde{P}_2$ and $z_2 \sim \tilde{Q}_2$, as in step 3, gives a test ϕ'' of $\tilde{P}_1 \times \tilde{P}_2$ against $\tilde{Q}_1 \times \tilde{Q}_2$ with the type I error of ϕ and a type II error no larger. The order of the two swaps matters. Before step 2 the first output and the second were dependent, so fixing the second output would have changed the distribution of the first, and the branch test would no longer be a test of P_1 against Q_1.
5. **Conclude.** Take any α and any joint test ϕ with type I error at most α. Steps 2 to 4 give a test ϕ'' on the product pair with the same type I error and a type II error no larger. So the type II error of ϕ is at least $T(\tilde{P}_1 \times \tilde{P}_2, \tilde{Q}_1 \times \tilde{Q}_2)$ evaluated at that type I error. Trade-off curves are non-increasing, so it is also at least that curve at α, which is $f_1 \otimes f_2(\alpha)$. This is Equation K.7.
6. **The tensor product does not depend on the pairs.** Steps 2 to 5 never used that $P_2(\cdot \mid y_1)$ changes with y_1. So they also prove the non-adaptive statement: if $T(P_i, Q_i) \ge T(\tilde{P}_i, \tilde{Q}_i)$ for $i = 1, 2$, then $T(P_1 \times P_2, Q_1 \times Q_2) \ge T(\tilde{P}_1 \times \tilde{P}_2, \tilde{Q}_1 \times \tilde{Q}_2)$. If both pairs have curve exactly f_i, the assumption holds in both directions, so the two product curves are each at least the other, and they are equal.
7. **Many steps, by induction.** The k-fold tensor product $f_1 \otimes \cdots \otimes f_k$ is the curve of the k-fold product pair, and step 6 shows that it does not depend on the pairs chosen. Suppose the first $k-1$ steps together are $f_1 \otimes \cdots \otimes f_{k-1}$-DP. That curve is the curve of the product pair $\tilde{P}_1 \times \cdots \times \tilde{P}_{k-1}$ against $\tilde{Q}_1 \times \cdots \times \tilde{Q}_{k-1}$. Treat the first $k-1$ outputs as one first output, and step k, chosen after seeing all of them, as the second step. Steps 2 to 5 then give the curve of $\tilde{P}_1 \times \cdots \times \tilde{P}_k$ against $\tilde{Q}_1 \times \cdots \times \tilde{Q}_k$, which is $f_1 \otimes \cdots \otimes f_k$.

K.2.4. Checking it

For the example of Table K.2, the script computes both curves exactly, by sorting the outputs by likelihood ratio. Each curve is made of straight pieces that meet at corners, written as (type I error, type II error). The adaptive pair's corners are at $(0.1, 0.55)$, $(0.25, 0.25)$ and $(0.4375, 0.0625)$, while the product's are at $(0.0625, 0.4375)$, $(0.25, 0.25)$ and $(0.4375, 0.0625)$. On a grid of 100,001 type I errors the adaptive curve is never below the product curve. The two touch from $\alpha = 0.25$ on and differ most, by 0.2812, at $\alpha = 0.0625$. The gap comes from the branch $y_1 = 0$, which keeps the bit only with probability 0.6 and so leaks less than the theorem charges for it.

The script also checks the replacement itself. It draws 20,000 random tests on the four joint outputs, with each $\phi(y_1, y_2)$ uniform on $[0, 1]$, and runs steps 2 and 4 on each one. No test had its type I error changed by more than 3.3×10^{-16}, which is rounding, and none had its type II error raised. On average the type II error fell by 0.4013, because random tests are far from optimal. Figure K.2 shows the replacement on single tests, and it shows that step 4 matters too. For example, the test that fires only on $(1, 1)$, the output most typical of D, has type II error 0.9375 at type I error 0.5625. Step 2 lowers the type II error to 0.7708, and step 4, which swaps the first coordinate for the best test of the fixed pair, lowers it to 0.1042.

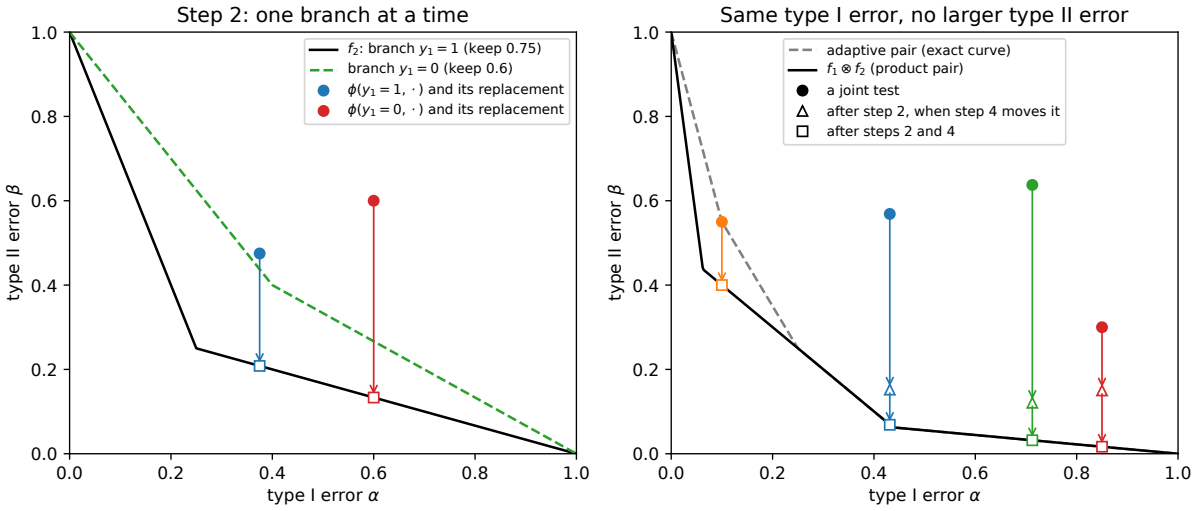


Figure K.2.: Test replacement for two adaptive randomized responses. Left: step 2 on each branch, where the attacker's test (dot) is swapped for the best test of the fixed pair with curve f_2 at the same type I error (square). On branch $y_1 = 1$ the branch pair is the fixed pair itself, and on branch $y_1 = 0$ (keep 0.6) its curve lies above f_2. Right: four joint tests on the adaptive pair, after step 2 (triangles, shown when step 4 moves them further) and after steps 2 and 4 (squares); each moves straight down, onto or above $f_1 \otimes f_2$. The adaptive pair's own curve (grey, dashed) lies above $f_1 \otimes f_2$.

K.2.5. What it does not cover

With continuous outputs there is a technical condition. Step 2 chooses one test per branch, and the averages in step 3 make sense only if the joint test ϕ' is a measurable function of (y_1, z_2), which roughly means that its averages are well defined. Choosing the canonical pair as $(\tilde{P}_2, \tilde{Q}_2)$

settles this. Its best test at type I error α is "fire when $z_2 > 1-\alpha$ or z_2 is the atom". So $\phi'(y_1, z_2)$ is that formula with α replaced by the branch's type I error, which is itself an average of ϕ over y_2 and so a measurable function of y_1. For finite outputs, as in the examples, there is nothing to check.

The proof does not use Blackwell's theorem, which the chapter cites for the non-adaptive case. Blackwell's theorem says more. The curve of one pair lies on or above the curve of another exactly when the first pair can be obtained by post-processing the second, that is by applying one random function to the outputs of both its distributions. That stronger statement stays cited. The chapter also claims that no better bound than $f_1 \otimes \cdots \otimes f_k$ holds in general, and this needs no new argument. Suppose the steps ignore each other's outputs, and each f_i is exactly the curve of some neighbouring pair of step i. Then the joint pair is that product pair, so its curve equals the tensor product, as the chapter shows in step 3 of its derivation.

K.3. Dominating pairs survive adaptive composition

K.3.1. What the chapter uses

Numerical accounting compares pairs through the **hockey-stick divergence** with a general weight $\gamma \geq 0$,

$$E_\gamma(P\|Q) = \sum_y \big(P(y) - \gamma Q(y)\big)_+, \tag{K.10}$$

where $P(y)$ and $Q(y)$ are the probabilities of output y and $(x)_+$ is x when x is positive and 0 otherwise. This section writes sums, so it assumes outputs that take finitely or countably many values, as in its examples. For continuous outputs every sum becomes an integral of densities, and nothing else changes. At the weight $\gamma = e^\varepsilon$ the divergence is the smallest δ for which the pair satisfies the (ε, δ)-DP inequality $P(S) \leq e^\varepsilon Q(S) + \delta$ for every set of outputs S.

A pair (P, Q) **dominates** a mechanism M if $E_\gamma(M(D)\|M(D')) \leq E_\gamma(P\|Q)$ for all neighbours and all $\gamma \geq 0$, so that every (ε, δ) statement true of the pair is true of the mechanism. The theorem used in Section 34.2 says that domination survives composition. Suppose the pair (P_i, Q_i) dominates step i for every value of the earlier outputs. Then for every $\gamma \geq 0$,

$$E_\gamma(M(D) \,\|\, M(D')) \leq E_\gamma(P_1 \times \cdots \times P_k \,\|\, Q_1 \times \cdots \times Q_k), \tag{K.11}$$

where M is the adaptive composition of the k steps. The chapter cites this theorem and does not prove it: "Its proof is in Zhu, Dong & Wang (2022, Theorem 10)." The numerical-accounting chapter works with the privacy loss distribution (PLD), the distribution of the privacy loss L when the output is drawn from P, and it rounds every loss up to a grid to get a pessimistic PLD that belongs to a pair dominating the mechanism. The theorem is what lets that pessimistic PLD bound adaptive runs, in step 6 of Section 34.3. It also makes the PLD accountants of Section 37.1 valid for the differentially private form of stochastic gradient descent (SGD), called DP-SGD, whose steps depend on the model so far.

K.3.2. The idea

Unlike the previous section, this section's tests ϕ fire for P, as in the numerical-accounting chapter. The reason is that $E_\gamma(P\|Q)$ rewards finding P, so a test that fires for P makes the formulas shorter. Think of E_γ as a best net gain. A test earns one unit each time it fires on a draw from P and pays γ each time it fires on a draw from Q. Its average gain is $\mathbb{E}_P[\phi] - \gamma\,\mathbb{E}_Q[\phi]$, and step 1 below shows that the best gain over all tests is $E_\gamma(P\|Q)$. Domination says that, whatever the price γ, the dominating pair offers at least the best gain the mechanism offers.

A picture turns this into the replacement step of Section K.2. Draw each test as a point in the plane, with how often it fires under Q across and how often it fires under P up. The points of all tests fill a region, and the region's upper edge g gives, for each budget c of firing under Q, the most firing under P that a test can reach. Step 2 shows that this edge is concave, bending downward like an arch, because mixing two tests gives a point on the segment between them.

Now take a line of slope γ that touches the edge from above without crossing it. It meets the vertical axis at height E_γ, because the height of a line of slope γ through a test's point is exactly that test's gain, and the touching line passes through the best test. A concave edge is the lowest of its touching lines, so the values E_γ at all slopes $\gamma \geq 0$ fix the edge. If the dominating pair has every intercept at least as high, its edge is everywhere at least as high, and its region contains the other one. So every test on the dominated pair has a counterpart on the dominating pair that fires at least as often under P and at most as often under Q.

That counterpart plays the role of the replacement step of Section K.2. There the counterpart had the same type I error and no larger type II error, and here it has two inequalities instead, one in each error. Both inequalities can only raise the gain $\mathbb{E}_P[\phi] - \gamma\,\mathbb{E}_Q[\phi]$, because $\gamma \geq 0$, so the branch-by-branch swap of the previous section goes through unchanged.

A small case shows the touching lines. Randomized response with keep-probability 0.75 has two outputs. The best tests fire first on the output that favours P, then on both. So the upper edge runs from $(0, 0)$ to $(0.25, 0.75)$ with slope 3, and then to $(1, 1)$ with slope $1/3$. A line of slope γ between $1/3$ and 3 touches the corner $(0.25, 0.75)$ and meets the vertical axis at $0.75 - 0.25\gamma$. The hockey-stick formula gives the same value, because $E_\gamma = (0.75 - 0.25\gamma)_+ + (0.25 - 0.75\gamma)_+$ and the second term is 0 once $\gamma \geq 1/3$. For example, the intercept and the formula both give 0.5 at $\gamma = 1$ and 0.25 at $\gamma = 2$.

K.3.3. Derivation

1. **The best gain is the hockey-stick divergence.** For every $\gamma \geq 0$,

$$E_\gamma(P\|Q) = \max_\phi \left(\mathbb{E}_P[\phi] - \gamma\,\mathbb{E}_Q[\phi]\right), \tag{K.12}$$

   where ϕ ranges over tests with values in $[0, 1]$. The gain is $\sum_y \phi(y)(P(y) - \gamma Q(y))$, and each term is largest when $\phi(y) = 1$ where $P(y) > \gamma Q(y)$ and $\phi(y) = 0$ elsewhere. That choice gives $\sum_y (P(y) - \gamma Q(y))_+$. This is Lemma 3.4, and it is also Equation 32.8 with the roles of the two distributions swapped.

2. **The upper edge of the region.** For a pair $(\tilde{P}, \tilde{Q})$ and $c \in [0,1]$, let

$$g(c) = \max\{\mathbb{E}_{\tilde{P}}[\tilde{\phi}] : \mathbb{E}_{\tilde{Q}}[\tilde{\phi}] \le c\}. \tag{K.13}$$

This is the Neyman–Pearson problem with the two distributions swapped, so the maximum is reached by a likelihood-ratio test that fires where $\tilde{p}/\tilde{q}$ is large. A bigger budget allows more tests, so g is non-decreasing. To see that g is concave, take tests $\tilde{\phi}_1$ and $\tilde{\phi}_2$ that reach g at budgets c_1 and c_2, and a weight $\lambda \in [0,1]$. The mixed test $\lambda\tilde{\phi}_1 + (1-\lambda)\tilde{\phi}_2$ mixes both of their averages (Hypothesis testing, the fact that mixing tests mixes their errors). So it fires under $\tilde{Q}$ at most $\lambda c_1 + (1-\lambda)c_2$ of the time and under $\tilde{P}$ exactly $\lambda g(c_1) + (1-\lambda)g(c_2)$ of the time. Then g at the mixed budget is at least the mixed value, which is what concave means.

3. **The edge is the lowest of its touching lines.** For every $c \in [0,1]$,

$$g(c) = \inf_{\gamma \ge 0} [E_\gamma(\tilde{P}\|\tilde{Q}) + \gamma c]. \tag{K.14}$$

The right side is never below $g(c)$. For any test with $\mathbb{E}_{\tilde{Q}}[\tilde{\phi}] \le c$, step 1 gives $\mathbb{E}_{\tilde{P}}[\tilde{\phi}] \le E_\gamma + \gamma\,\mathbb{E}_{\tilde{Q}}[\tilde{\phi}] \le E_\gamma + \gamma c$, because $\gamma \ge 0$. To see that the right side comes down to $g(c)$, use a line that touches g at c. For $0 < c < 1$, let γ^* be the slope of g just to the right of c. It is finite, because the slope of a concave function never increases, so it is at most the slope of the chord from 0 to c, which is at most $1/c$. It is at least 0 because g is non-decreasing. A concave function lies below each of its touching lines (Rockafellar 1970, §23), so $g(c') \le g(c) + \gamma^*(c' - c)$ for every c'. Now any test $\tilde{\phi}$ has $\mathbb{E}_{\tilde{P}}[\tilde{\phi}] \le g(c')$ with $c' = \mathbb{E}_{\tilde{Q}}[\tilde{\phi}]$, so its gain at price γ^* is at most $g(c') - \gamma^* c' \le g(c) - \gamma^* c$. By step 1, $E_{\gamma^*} + \gamma^* c \le g(c)$. The two ends of the budget need their own argument. At $c = 1$, the slope $\gamma^* = 0$ works, since $E_0 = 1 = g(1)$. At $c = 0$, the edge is $g(0) = \tilde{P}[\tilde{q} = 0]$, the mass that $\tilde{P}$ puts where $\tilde{Q}$ puts none. Then E_γ falls to exactly this value as $\gamma \to \infty$, because every output with $\tilde{q} > 0$ eventually contributes 0. So at $c = 0$ the infimum is reached only in the limit, which is why Equation K.14 uses inf rather than min.

4. **Domination puts one region inside the other.** Suppose $E_\gamma(P\|Q) \le E_\gamma(\tilde{P}\|\tilde{Q})$ for every $\gamma \ge 0$. Then for every test ϕ of P against Q there is a test $\tilde{\phi}$ of $\tilde{P}$ against $\tilde{Q}$ with

$$\mathbb{E}_{\tilde{P}}[\tilde{\phi}] \ge \mathbb{E}_P[\phi] \qquad \text{and} \qquad \mathbb{E}_{\tilde{Q}}[\tilde{\phi}] \le \mathbb{E}_Q[\phi]. \tag{K.15}$$

The reason: by step 1 and the assumption, $\mathbb{E}_P[\phi] \le E_\gamma(P\|Q) + \gamma\,\mathbb{E}_Q[\phi] \le E_\gamma(\tilde{P}\|\tilde{Q}) + \gamma\,\mathbb{E}_Q[\phi]$ for every $\gamma \ge 0$. Taking the smallest right side, Equation K.14 gives $\mathbb{E}_P[\phi] \le g(\mathbb{E}_Q[\phi])$. The test that reaches g at the budget $c = \mathbb{E}_Q[\phi]$ is the $\tilde{\phi}$ wanted.

5. **Swap branch by branch.** Take two adaptive steps. The letters P_i and Q_i already name the dominating pairs, so the run's own laws get new letters, A for the laws on D and B for those on D'. On D the first output has law A_1, and the second, given y_1, has law $A_2(\cdot \mid y_1)$. On D' the laws are B_1 and $B_2(\cdot \mid y_1)$. So the joint laws are $A_1(y_1)A_2(y_2 \mid y_1)$ and $B_1(y_1)B_2(y_2 \mid y_1)$. The fixed pair (P_2, Q_2) dominates $(A_2(\cdot \mid y_1), B_2(\cdot \mid y_1))$ on every branch, and (P_1, Q_1) dominates (A_1, B_1). Fix $\gamma \ge 0$ and a joint test ϕ. Its gain is

$$\sum_{y_1} \Big(A_1(y_1)\,\mathbb{E}_{A_2(\cdot|y_1)}\phi(y_1,\cdot) - \gamma\, B_1(y_1)\,\mathbb{E}_{B_2(\cdot|y_1)}\phi(y_1,\cdot) \Big). \tag{K.16}$$

On each branch, step 4 gives a test $\tilde{\phi}_{y_1}$ of P_2 against Q_2 that fires at least as often under P_2 and at most as often under Q_2. The weights $A_1(y_1)$ and $\gamma\, B_1(y_1)$ are not negative, so the swap raises the first part of each term or keeps it, and lowers the subtracted part or keeps it. So the gain does not fall, and the new test is a test of $A_1 \times P_2$ against $B_1 \times Q_2$.

That pair is a product, so for each fixed second output the test is a test of A_1 against B_1. Step 4 swaps each of these tests for a test of P_1 against Q_1 in the same way. The weights are now the probabilities of the second output under P_2 and γQ_2, which are again not negative. The result is a test of $P_1 \times P_2$ against $Q_1 \times Q_2$ with at least the gain of ϕ.

6. **Conclude, and induct.** Step 1 says that no test of the product pair gains more than E_γ of the product pair, so neither does ϕ. Step 1 also says that the best gain over joint tests ϕ is E_γ of the run's joint pair, so that divergence is at most E_γ of the product pair, which is Equation K.11 for two steps. Treating the first $k-1$ steps as one step, as in step 7 of Section K.2, extends it to k steps.

7. **The same theorem in the language of curves.** Domination at every $\gamma \geq 0$ turns out to be the same as an order between trade-off curves, and the proof goes both ways through step 4. To use the curves, switch back to tests that fire for Q, as in Section K.2, by reading $1-\phi$ and $1-\tilde{\phi}$, which fire exactly when ϕ and $\tilde{\phi}$ stay silent. First suppose that the pair $(\tilde{P}, \tilde{Q})$ dominates (P, Q). For any ϕ, step 4 gives a $\tilde{\phi}$, and the type I errors of $1-\phi$ and $1-\tilde{\phi}$ are $1-\mathbb{E}_P[\phi]$ and $1-\mathbb{E}_{\tilde{P}}[\tilde{\phi}]$, while their type II errors are $\mathbb{E}_Q[\phi]$ and $\mathbb{E}_{\tilde{Q}}[\tilde{\phi}]$. So the test $1-\tilde{\phi}$ is no worse in either error, and since trade-off curves are non-increasing this gives $T(P, Q) \geq T(\tilde{P}, \tilde{Q})$.

   Now suppose instead that $T(P, Q) \geq T(\tilde{P}, \tilde{Q})$. Take any test ϕ that fires for P. The replacement step of Section K.2, applied to $1-\phi$, gives a test χ of $\tilde{P}$ against $\tilde{Q}$ with the same type I error and a type II error no larger. Reading $\tilde{\phi} = 1-\chi$ back as a test that fires for P, it fires under $\tilde{P}$ exactly as often as ϕ fires under P, and under $\tilde{Q}$ at most as often as ϕ fires under Q. So its gain is at least that of ϕ at every $\gamma \geq 0$, and by step 1, $E_\gamma(P\|Q) \leq E_\gamma(\tilde{P}\|\tilde{Q})$.

   Together, the two directions say that domination at every $\gamma \geq 0$ is the same as $T(M(D), M(D')) \geq T(P, Q)$ for all neighbours. So the theorem of this section is the theorem of Section K.2, read through hockey-stick divergences.

K.3.4. Checking it

The three-step adaptive run of Adaptive composition (Table 36.3) is a good test case. The first step is randomized response with keep-probability 0.75. Then come a strong step (keep 0.75) and a weak step (keep 0.6), in an order set by the first output: if it is 1 the strong step comes second and the weak one third, and if it is 0 the order is reversed. Randomized response with keep-probability 0.75 dominates both kinds of step, so the theorem says the product of three such pairs dominates the run. The script checks E_γ of both at 502 values of γ in $[0, e^4]$, including 100 values below 1, and the product is never below the run. Table K.3 lists four of them.

Table K.3.: $\delta(\varepsilon) = E_{e^\varepsilon}(P\|Q)$ of the three-step adaptive run and of the product of its dominating pairs.

$\varepsilon = \ln\gamma$	0	1	2	3
Adaptive run, $\delta(\varepsilon)$	0.5750	0.3926	0.1528	0.0000
Product of three dominating pairs	0.6875	0.4190	0.3064	0.1080

The adaptive run's δ reaches 0 at its largest privacy loss. Along any path, the first step and the strong step each multiply the likelihood ratio by at most $0.75/0.25 = 3$, and the weak step by at most $0.6/0.4 = 1.5$. So the largest loss is $\ln(3 \times 3 \times 1.5) = \ln 13.5 = 2.6027$. The product's δ reaches 0 only at $3\ln 3 = 3.2958$, because it charges for three strong steps while no path of the run uses more than two (Figure K.3). The script also checks step 4 directly. Of 100,000 random tests on the run's eight transcripts, the possible triples of outputs, none lies above the product's upper edge g, and the closest comes within 0.0726 of it.

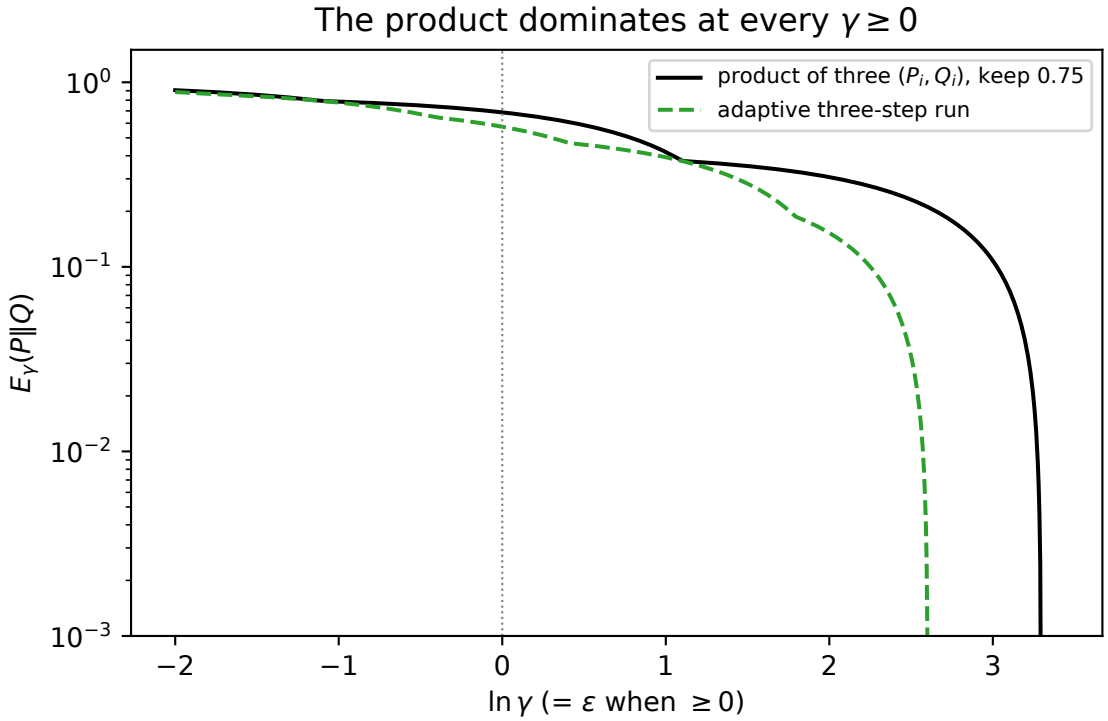


Figure K.3.: The hockey-stick divergence $E_\gamma(P\|Q)$ against $\ln\gamma$ for the three-step adaptive run (dashed) and for the product of three randomized responses with keep-probability 0.75 (solid). The product lies on or above the run at every $\gamma \geq 0$, including $\gamma < 1$ (left of the dotted line).

Figure K.4 draws the regions for a pair with a curved edge. The dominating pair has $P = \mathcal{N}(1, 1)$ and $Q = \mathcal{N}(0, 1)$, and the dominated pair has $P = \mathcal{N}(0.6, 1)$ and the same Q, so its region lies

inside. Three touching lines of the outer region, with slopes 0.5, 1 and e, meet the vertical axis at $E_\gamma = 0.5953$, 0.3829 and 0.1269. The inner pair's values are 0.5253, 0.2358 and 0.0190, all lower, as domination requires.

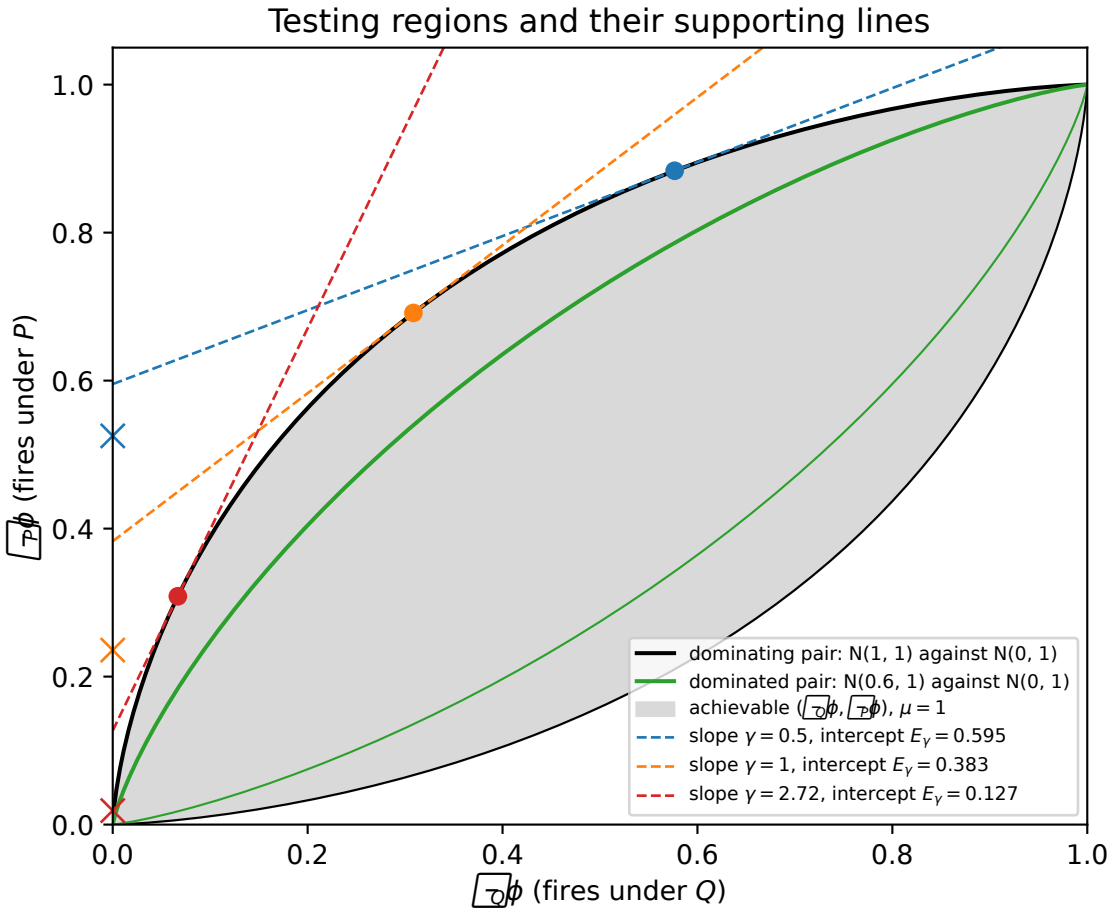


Figure K.4.: Testing regions in the plane of $(\mathbb{E}_Q\phi, \mathbb{E}_P\phi)$. The grey region holds every test of $\mathcal{N}(1,1)$ against $\mathcal{N}(0,1)$; the region of $\mathcal{N}(0.6,1)$ against $\mathcal{N}(0,1)$ (green) lies inside it. Each dashed line has slope γ and touches the outer edge at a dot; its height at 0 is E_γ of the outer pair. Crosses on the vertical axis mark E_γ of the inner pair, below each line.

K.3.5. What it does not cover

The theorem needs domination at every $\gamma \geq 0$, not only at $\gamma \geq 1$. In step 3 the touching line at a large budget c can have a slope below 1, so step 4 uses those values of γ. The values of E_γ below $\gamma = 1$ describe the reversed pair (Q, P), because

$$E_\gamma(P\|Q) = 1 - \gamma + \gamma\, E_{1/\gamma}(Q\|P). \tag{K.17}$$

To see it, split each term as $(P - \gamma Q)_+ = (P - \gamma Q) + (\gamma Q - P)_+$, and note that $(\gamma Q - P)_+ = \gamma(Q - P/\gamma)_+$. Summing over y, the first part gives $1 - \gamma$, since P and Q each sum to 1, and

the second gives $\gamma\, E_{1/\gamma}(Q\|P)$. So domination below $\gamma = 1$ is domination of the reversed pair at $1/\gamma > 1$. The theorem also assumes one fixed pair per step for every history. A pair that may change with the history is not covered, and that version stays with Zhu, Dong & Wang (2022).

K.4. The f-DP central limit theorem

K.4.1. What the chapter uses

f-DP and Gaussian DP states the central limit theorem (CLT) for composition in Section 32.5. The theorem describes the composed curve through a few averages of each step's privacy loss L_i, with the output drawn from P_i. The chapter computes these averages from the curve f_i alone, and step 1 below shows that they are moments of the loss. So read $\mathrm{kl}(f_i)$ as the mean of L_i, $\kappa_2(f_i)$ as its second moment $\mathbb{E}_P[L_i^2]$, $\kappa_3(f_i)$ as its third absolute moment $\mathbb{E}_P|L_i|^3$, and $\bar\kappa_3(f_i)$ as its third absolute moment about the mean, $\mathbb{E}_P|L_i - \mathbb{E}_P L_i|^3$. The theorem also assumes that every curve is symmetric, $f_i = T(P_i, Q_i) = T(Q_i, P_i)$, which means that telling D from D' is exactly as hard as telling D' from D.

Summing over the steps gives the moments of the composed loss $L = L_1 + \cdots + L_k$ under P. Its mean is $K = \sum_i \mathrm{kl}(f_i)$, and its variance is $V = \sum_i \kappa_2(f_i) - \sum_i \mathrm{kl}(f_i)^2$, each step's second moment minus its squared mean, summed. The chapter writes these sums as $\|\mathrm{kl}\|_1$ and $\|\kappa_2\|_1 - \|\mathrm{kl}\|_2^2$, and the summed third central moments as $\|\bar\kappa_3\|_1 = \sum_i \bar\kappa_3(f_i)$. With these it sets

$$\mu = \frac{2K}{\sqrt{V}}, \qquad w = \frac{0.56\,\|\bar\kappa_3\|_1}{V^{3/2}}, \tag{K.18}$$

where 0.56 is the constant of the Berry–Esseen theorem that step 3 states. In this section w is the width of a band. Dong, Roth & Su write it γ, which this book keeps for the level of the hockey-stick divergence. The band surrounds the Gaussian curve G_μ, the curve of $\mathcal{N}(0,1)$ against $\mathcal{N}(\mu,1)$, which is $G_\mu(\alpha) = \Phi(\Phi^{-1}(1-\alpha) - \mu)$ with Φ the standard normal distribution function. The theorem says that if $w < \frac{1}{2}$, then for all $\alpha \in [w, 1-w]$ the composed curve lies in the band

$$G_\mu(\alpha + w) - w \le f_1 \otimes \cdots \otimes f_k(\alpha) \le G_\mu(\alpha - w) + w. \tag{K.19}$$

This is Dong, Roth & Su (2022), Theorem 3.4.

The chapter also states a limit form for many steps that each leak little (their Theorem 3.5). Take, for each k, a fresh list of k symmetric curves. Suppose that as k grows the summed means tend to a limit, $\sum \mathrm{kl} \to K_\infty$, the largest mean tends to 0, the summed second moments tend to some $s^2 > 0$, and the summed third absolute moments tend to 0. Then the composed curve tends to $G_{2K_\infty/s}$. Their Theorem 3.6 applies this to steps that are ε_i-DP. The chapter explains why the theorem should hold and refers to Dong et al., Appendix D, for the proof. Its sketch explains in outline why $\mu = 2K/\sqrt{V}$ and why the curve is close to G_μ, and points here for the details. This section proves the band from the Berry–Esseen theorem, which stays cited, and shows that the argument forces the value $\mu = 2K/\sqrt{V}$ rather than choosing it.

K.4.2. The idea

The best test fires when the composed privacy loss L is below some threshold τ. Its type I error is $\Pr_P[L < \tau]$, a distribution function of L under P, and its power is $\Pr_Q[L < \tau]$, a distribution function of L under Q. The Berry–Esseen theorem, stated in step 3, says each is within w of a normal distribution function. Step 2 shows that symmetry makes the law of L under Q the mirror image of its law under P: mean $-K$ instead of K, with the same spread $\sqrt{V}$. So, measured in units of $\sqrt{V}$ from the mean under P, the two normal approximations are one curve and the same curve shifted left by $2K/\sqrt{V}$. Two normal curves one standard deviation wide and μ apart are exactly the Gaussian pair $\mathcal{N}(0,1)$ against $\mathcal{N}(\mu,1)$ behind G_μ. So each best test has a type I error within w, and a type II error within w, of the matching test of the Gaussian pair, and that is what the band says.

For an example, compose $k = 100$ randomized responses, each ε_0-DP with $\varepsilon_0 = 0.1$, so that each reports the bit truthfully with probability $e^{\varepsilon_0}/(1 + e^{\varepsilon_0})$. Each step's loss is then ± 0.1, and step 8 shows that its mean is $\varepsilon_0 \tanh(\varepsilon_0/2) = 0.1 \tanh(0.05) = 0.004996$. Its variance is $0.1^2 - 0.004996^2$, so over the 100 steps $K = 0.4996$, $V = 0.9975$ and $\mu = 2 \times 0.4996/\sqrt{0.9975} = 1.0004$. Figure K.6 (right) shows the two step-shaped distribution functions of $(L - K)/\sqrt{V}$, one under P and one shifted left by about 1 under Q.

K.4.3. Derivation

Let (P_i, Q_i) be pairs with curves f_i. By Section K.2, the composed curve $f_1 \otimes \cdots \otimes f_k$ is the curve of the product pair $P = \prod P_i$ against $Q = \prod Q_i$, whose privacy loss is $L = \sum_i L_i$ with independent terms.

1. **The functionals are moments of the loss.** The chapter computes kl, κ_2 and $\bar{\kappa}_3$ as integrals over $\alpha \in [0,1]$ of powers of $\ln|f'(\alpha)|$, and the link to the loss is the slope rule. Let $\tau(\alpha)$ be the threshold of the best test of size α, the point below which L has probability α under P. Moving that threshold to take in one more sliver of outputs near y raises the type I error by $p(y)\,\mathrm{d}y$ and lowers the type II error by $q(y)\,\mathrm{d}y$. So the curve's slope there is $-q/p = -e^{-\tau(\alpha)}$, and taking logs gives

$$-\ln|f'(\alpha)| = \tau(\alpha). \tag{K.20}$$

   The function τ is the inverse of the distribution function of L under P. So feeding a uniform α into τ draws a sample of L under P, just as feeding uniform random numbers into any inverse distribution function draws from that distribution. So averaging a function of $\tau(\alpha)$ over α uniform on $[0,1]$ averages that function of L under P. Applied to each step's curve f_i, this gives $\mathrm{kl}(f_i) = \mathbb{E}_P[L_i]$, $\kappa_2(f_i) = \mathbb{E}_P[L_i^2]$ and $\bar{\kappa}_3(f_i) = \mathbb{E}_P|L_i - \mathbb{E}_P L_i|^3$. Means and variances of independent terms add, so K and V are the mean and variance of L under P. The same step shows that the curve fixes the law of the loss under P, because it fixes the function $\tau(\alpha)$.

2. **Symmetry mirrors the law.** Swap the roles of P_i and Q_i. The swapped pair has privacy loss $\ln(q_i/p_i) = -L_i$, read under Q_i. Its curve is $T(Q_i, P_i) = f_i$, the same curve, so by step 1 its loss has the same law: $-L_i$ under Q_i has the law of L_i under P_i. Adding the independent steps, $-L$ under Q has the law of L under P. So under Q the loss has mean $-K$, variance V and the same third absolute central moments.

3. **Berry–Esseen under both laws.** The Berry–Esseen theorem bounds how far a standardized sum of independent terms can be from normal. Take such a sum S, and write m for its mean and V for its variance. Write ρ_3 for the sum of the third absolute central moments of its terms. Then for every x (Shevtsova 2010, with the constant 0.56)

$$\Big|\Pr\Big[\frac{S-m}{\sqrt{V}} \le x\Big] - \Phi(x)\Big| \le \frac{0.56\,\rho_3}{V^{3/2}}. \tag{K.21}$$

The same bound holds with $<$ in place of $\le$. To see it, let x rise towards a point x_0 from below. Then $\Pr[\cdot \le x]$ rises to $\Pr[\cdot < x_0]$, while $\Phi(x)$ moves continuously to $\Phi(x_0)$, so the bound survives the limit. Under P, with $S = L$, the right side is exactly the band width w defined above. Under Q the bound follows from step 2, which turns a statement about L under Q into one about $-L$ under P. For every y,

$$\Pr_Q\Big[\frac{L+K}{\sqrt{V}} < y\Big] = \Pr_P\Big[\frac{L-K}{\sqrt{V}} > -y\Big] = 1 - \Pr_P\Big[\frac{L-K}{\sqrt{V}} \le -y\Big], \tag{K.22}$$

which is within w of $1 - \Phi(-y) = \Phi(y)$, by the symmetry of the normal density.

4. **The errors of a best test.** By the Neyman–Pearson lemma, a best test fires when $L < \tau$ and, if L has an atom at τ, fires there with some probability λ. Its type I error α lies between $\Pr_P[L < \tau]$ and $\Pr_P[L \le \tau]$. Write $x = (\tau - K)/\sqrt{V}$ for the threshold in standard units under P. Both ends are within w of $\Phi(x)$ by step 3, so $|\alpha - \Phi(x)| \le w$. Its power lies between $\Pr_Q[L < \tau]$ and $\Pr_Q[L \le \tau]$. The event $L < \tau$ is the event $(L+K)/\sqrt{V} < x + 2K/\sqrt{V}$, so by step 3 the power is within w of $\Phi(x + \mu)$ with $\mu = 2K/\sqrt{V}$. In terms of the type II error $\beta = 1 - \text{power}$, and using $1 - \Phi(t) = \Phi(-t)$,

$$|\alpha - \Phi(x)| \le w, \qquad |\beta - \Phi(-x - \mu)| \le w. \tag{K.23}$$

This is where μ comes from. Under P and under Q the loss has the same spread, and its two means K and $-K$ are $2K/\sqrt{V}$ standard deviations apart. The Gaussian pair behind G_μ has exactly this picture, with means μ apart.

5. **Remove the threshold.** The Gaussian curve is $G_\mu(\alpha) = \Phi(-\Phi^{-1}(\alpha) - \mu)$, because $\Phi^{-1}(1-\alpha) = -\Phi^{-1}(\alpha)$. From $\Phi(x) \ge \alpha - w$ and the fact that Φ is increasing, $x \ge \Phi^{-1}(\alpha - w)$. Then $\Phi(-x-\mu) \le \Phi(-\Phi^{-1}(\alpha - w) - \mu) = G_\mu(\alpha - w)$, so $\beta \le G_\mu(\alpha - w) + w$. In the same way, $\Phi(x) \le \alpha + w$ gives $x \le \Phi^{-1}(\alpha + w)$ and $\beta \ge G_\mu(\alpha + w) - w$. The first bound needs $\alpha - w \ge 0$ and the second needs $\alpha + w \le 1$, so that Φ^{-1} is applied inside $[0, 1]$, which is why the chapter states the band on $[w, 1-w]$. At the ends, where Φ^{-1} is infinite, the bound reads $\beta \le 1 + w$ or $\beta \ge -w$ and holds trivially.

6. **Every point of the curve.** For every type I error α, the Neyman–Pearson lemma gives a best test of size exactly α, and its type II error is the composed curve at α. Steps 4 and 5 apply to that test, so Equation K.19 holds at every $\alpha \in [w, 1-w]$.

7. **The limit form.** Take a triangular array, a fresh list of k symmetric curves for each k, with the chapter's four conditions. First, $V \to s^2$. Each mean kl_i is a Kullback–Leibler divergence and so cannot be negative, which means each squared mean is at most the largest mean times the mean. So $\sum_i \mathrm{kl}_i^2 \le \max_i \mathrm{kl}_i \sum_i \mathrm{kl}_i \to 0 \cdot K_\infty = 0$, and V has the same limit as $\sum_i \kappa_2$.

Second, $w \to 0$, because a third absolute central moment is at most 8 times the third absolute moment. To see it, measure size by $\|X\|_3 = (\mathbb{E}|X|^3)^{1/3}$, which obeys the triangle

inequality. The mean m_i of L_i is at most $\|L_i\|_3$ in size, since an average is at most the average size and the average size is at most $\|L_i\|_3$. So $\|L_i - m_i\|_3 \le \|L_i\|_3 + |m_i| \le 2\|L_i\|_3$, and cubing gives the factor 8. Hence $\sum \bar{\kappa}_3 \le 8 \sum \kappa_3 \to 0$, so $w \to 0$ and $\mu \to 2K_\infty/s$.

Third, the band closes. Since μ converges, it stays below some bound $\mu_{\max}$. The function $(\mu, \alpha) \mapsto G_\mu(\alpha)$ is uniformly continuous on $[0, \mu_{\max}] \times [0, 1]$, so both edges of the band tend to $G_{2K_\infty/s}$ uniformly in α. Below $\alpha = w$, the composed curve and G_μ are both non-increasing and at most 1, so each lies between its value at w and 1. Both values at w tend to 1 as $w \to 0$, so the curve converges there too, and the same argument with 0 in place of 1 covers $\alpha > 1 - w$.

8. **Steps that are ε_i-DP.** Such a step is $f_{\varepsilon_i,0}$-DP, and the curve $f_{\varepsilon,0}$ is that of randomized response. Its loss is $+\varepsilon$ with probability $t = e^\varepsilon/(1+e^\varepsilon)$ and $-\varepsilon$ otherwise. So $\mathrm{kl} = \varepsilon(2t-1)$, and $2t - 1 = (e^\varepsilon - 1)/(e^\varepsilon + 1) = \tanh(\varepsilon/2)$. The loss always has size ε, so $\kappa_2 = \varepsilon^2$ and $\kappa_3 = \varepsilon^3$. For $x \ge 0$, $x - x^3/3 \le \tanh x \le x$, and with $x = \varepsilon/2$ this gives

$$\left|\varepsilon \tanh \frac{\varepsilon}{2} - \frac{\varepsilon^2}{2}\right| \le \frac{\varepsilon^4}{24}. \tag{K.24}$$

If $\sum \varepsilon_i^2 \to \mu^2$ and $\max \varepsilon_i \to 0$, then $\sum \mathrm{kl} \to \mu^2/2$, because the error is at most $\max \varepsilon_i^2 \sum \varepsilon_i^2/24 \to 0$. Also $\sum \kappa_2 \to \mu^2$ and $\sum \kappa_3 \le \max \varepsilon_i \sum \varepsilon_i^2 \to 0$. Step 7, with $K_\infty = \mu^2/2$ and $s = \mu$, gives the limit $G_{2(\mu^2/2)/\mu} = G_\mu$. By Section K.2 the composed mechanism is $\bigotimes f_{\varepsilon_i,0}$-DP, so its guarantee tends to μ-GDP.

K.4.4. Checking it

The script composes k randomized responses with $\varepsilon_0 = 1/\sqrt{k}$, exactly, from the binomial law of the number of truthful reports. Table K.4 compares the composed curve with the band. Its "room" columns give the smallest distance, over $\alpha \in [w, 1-w]$, between the curve and each edge of the band.

Table K.4.: The band of Equation K.19 for k randomized responses with $\varepsilon_0 = 1/\sqrt{k}$. The last column is the distance between the distribution function F of $(L-K)/\sqrt{V}$ under P and Φ, the quantity that Berry–Esseen bounds by w.

k	μ	w	Smallest room above the lower edge	Smallest room below the upper edge	$\max_\alpha \lvert f(\alpha) - G_\mu(\alpha)\rvert$	Actual $\sup_x \lvert F(x) - \Phi(x)\rvert$
10	1.0042	0.1837	0.2151	0.2420	0.0137	0.1331
100	1.0004	0.0562	0.0611	0.0647	0.0014	0.0405
1000	1.0000	0.0177	0.0187	0.0193	0.0001	0.0127

The exact curve always stays inside the band, with room to spare (Table K.4). The Berry–Esseen step is nearly sharp here. Dividing the last column by the w column gives about 0.72 at every k, so the distribution function of the loss really is 0.72 times w away from Φ, because it jumps at each of its $k+1$ values. The curve is much closer to G_μ than that, 0.0137 at $k = 10$ and 0.0001 at $k = 1000$. The reason is that a jump moves both errors of a test at once. Just past an atom of

the loss, both distribution functions overshoot their normal curves. So the type I error grows and the type II error shrinks together, and a point that moves right and down at once moves roughly along the falling curve G_μ rather than away from it. So the band is valid but wide, and its width shrinks only like $1/\sqrt{k}$. At $k = 1$, $w = 0.7663$ is above $\frac{1}{2}$ and the theorem says nothing.

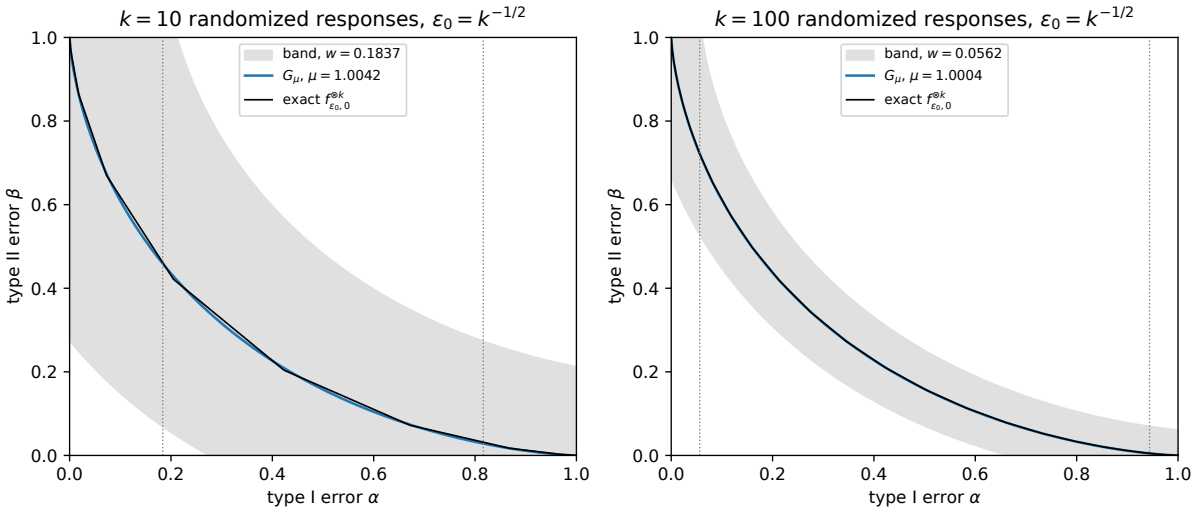


Figure K.5.: The band of the f-DP central limit theorem for k randomized responses with $\varepsilon_0 = k^{-1/2}$. Grey: the region between $G_\mu(\alpha + w) - w$ and $G_\mu(\alpha - w) + w$; blue: G_μ; black: the exact composed curve. Dotted lines mark $\alpha = w$ and $\alpha = 1 - w$, where the band starts to apply. Left: $k = 10$, $w = 0.1837$. Right: $k = 100$, $w = 0.0562$.

Figure K.6 shows steps 3 and 4. Each step function stays within w of its normal curve, and the shift between the two normal curves is the μ of the band in Figure K.5.

K.4.5. What it does not cover

The proof needs every f_i to be symmetric, because step 2 gets the law under Q from the law under P. Without symmetry the two laws have to be handled separately, as in Section K.5. The Berry–Esseen theorem and its constant 0.56 are cited (Shevtsova 2010). The central limit theorem itself, without a rate, is derived in Section E.4. The band is additive, so it says nothing useful about type I errors below w. Those are the errors that decide a small δ. The δ that goes with ε is the largest value of $1 - f(\alpha) - e^\varepsilon \alpha$ over α, and for a large ε the cost $e^\varepsilon \alpha$ pushes that largest value to a tiny α. This is why the chapter treats the CLT only as an approximation in the tails.

K.5. The central limit theorem for the subsampled Gaussian

K.5.1. What the chapter uses

Each step of DP-SGD forms a batch by Poisson subsampling, which includes each record independently with probability q, the sampling rate. It clips each record's gradient to the clipping norm and adds Gaussian noise whose standard deviation, in units of the clipping norm, is the noise

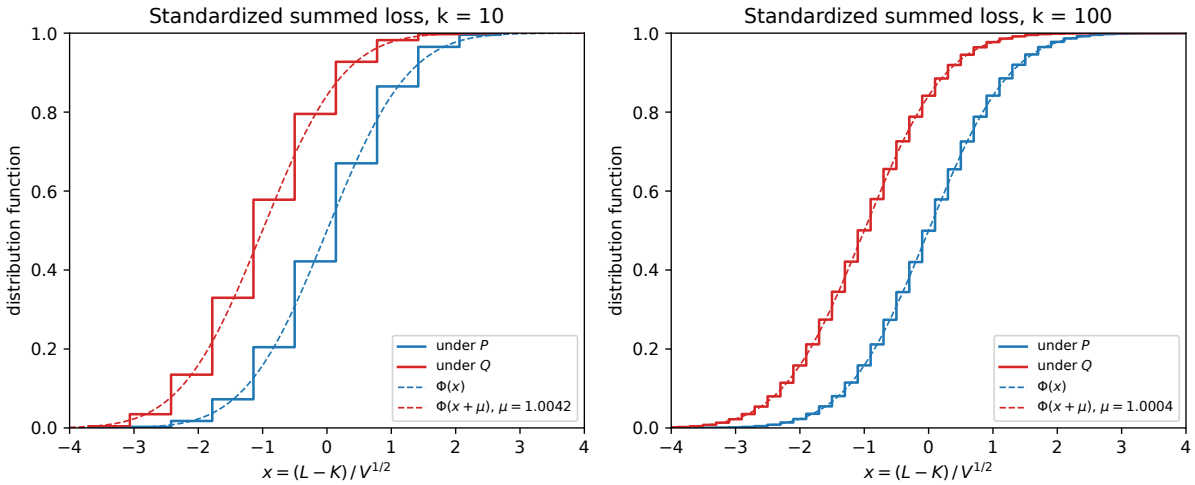


Figure K.6.: Distribution functions of the standardized composed loss $(L-K)/\sqrt{V}$ of k randomized responses, under P (blue steps) and under Q (red steps), against $\Phi(x)$ and $\Phi(x+\mu)$ (dashed). The law under Q is the mirror image of the law under P, shifted left by $\mu = 2K/\sqrt{V}$. Left: $k = 10$. Right: $k = 100$.

multiplier σ. The chapter uses add/remove neighbours, where D' is D with one target record added. The other records contribute the same to both outputs and can be subtracted, and the target record shifts the sum by at most one unit, and only when it lands in the batch. So one step reduces to the pair (Section 32.6)

$$P = \mathcal{N}(0, \sigma^2), \qquad Q = (1-q)\,\mathcal{N}(0, \sigma^2) + q\,\mathcal{N}(1, \sigma^2). \tag{K.25}$$

Under P the target record is absent, and under Q it is present and lands in the batch with probability q. This order of the two distributions is the **add direction**, and the reversed order (Q, P) is the **remove direction**. Since the letter q now names the sampling rate, this section writes the densities as $P(y)$ and $Q(y)$. The chapter states that if $k \to \infty$ and $q\sqrt{k} \to c$ with σ fixed, the k-fold composition is asymptotically μ-GDP with

$$\mu = q\sqrt{k(e^{1/\sigma^2} - 1)} \to c\sqrt{e^{1/\sigma^2} - 1}. \tag{K.26}$$

Write $v = e^{1/\sigma^2} - 1$, and write P^k and Q^k for the k-fold products of the pair. In the precise form proved here, with $c > 0$,

$$T(P^k, Q^k) \to G_{c\sqrt{v}} \quad \text{uniformly on } [0, 1]. \tag{K.27}$$

This is Bu, Dong, Long & Su (2020), Theorem 5.

The numerical-accounting chapter (Section 34.6) and Choosing an accountant (Section 37.1) use this formula as an approximation, but the chapter's derivation of it is labelled heuristic. It computes the leading terms of the mean and variance of one step's loss and plugs them into the $\mu = 2K/\sqrt{V}$ of the symmetric CLT, which leaves two gaps. The pair is not symmetric, so Equation K.19 does not apply as stated, and the remainder of the expansion is never bounded. This section closes both gaps.

K.5.2. The idea

The skeleton is the one of Section K.4. The best test fires when the summed loss is small, its errors are distribution functions of the loss under P and under Q, and Berry–Esseen makes each one nearly normal. What changes is that the law under Q is no longer a mirror image, so its mean and variance have to be computed on their own. They come out as mirror images to leading order. Under P one step's loss has mean about $+\frac{q^2}{2}v$, under Q about $-\frac{q^2}{2}v$, and both variances are about q^2v, with errors of order q^3. Over k steps the means become $\pm\frac{kq^2}{2}v \to \pm\frac{c^2}{2}v$ and the variances $kq^2v \to c^2v$, while the errors add up to order $kq^3 \to 0$. So, in standard units, the two laws become normal curves $c\sqrt{v}$ apart, which is the Gaussian pair with $\mu = c\sqrt{v}$.

At $q = 0.01$ and $\sigma = 1$, $v = e - 1 = 1.7183$. Computed to high accuracy by numerical integration, one step's loss has mean 8.1890×10^{-5} and variance 1.6009×10^{-4} under P. Under Q they are -8.3812×10^{-5} and 1.7163×10^{-4}. The leading terms are $\pm\frac{q^2}{2}v = \pm 8.5914 \times 10^{-5}$ and $q^2v = 1.7183 \times 10^{-4}$.

K.5.3. Derivation

1. **The loss.** The ratio of the $\mathcal{N}(1,\sigma^2)$ density to the $\mathcal{N}(0,\sigma^2)$ density at y is $e^{(y^2-(y-1)^2)/(2\sigma^2)} = e^{(y-1/2)/\sigma^2}$. So the likelihood ratio is $Q(y)/P(y) = 1 + qW$ with $W = e^{(y-1/2)/\sigma^2} - 1$, and the privacy loss is
$$L = \ln\frac{P(y)}{Q(y)} = -\ln(1+qW). \tag{K.28}$$
Since $W > -1$, the number $x = qW$ is above $-q$. So $x \geq -\frac{1}{2}$ whenever $q \leq \frac{1}{2}$, which step 3 needs and which every small sampling rate meets. From here on, $q \leq \frac{1}{2}$.

2. **All moments of W are finite.** Under P, $y \sim \mathcal{N}(0,\sigma^2)$, and $(1+W)^n = e^{ny/\sigma^2}e^{-n/(2\sigma^2)}$. The moment generating function $\mathbb{E}[e^{\lambda y}] = e^{\lambda^2\sigma^2/2}$ at $\lambda = n/\sigma^2$ gives $e^{n^2/(2\sigma^2)}$ for the first factor, so
$$\mathbb{E}_P\big[(1+W)^n\big] = e^{n(n-1)/(2\sigma^2)}. \tag{K.29}$$
Here n is any whole number. The first two cases give the mean and the second moment of W: the mean is 0, and $\mathbb{E}_P[W^2] = e^{1/\sigma^2} - 2 + 1 = v$. The absolute moments are bounded too, because $|W|^n \leq (1+W)^n + 1$ when $W \geq -1$ (a negative W has $|W| \leq 1$). So $\mathbb{E}_P|W|^n \leq e^{n(n-1)/(2\sigma^2)} + 1$. The later steps use moments up to the sixth, and all of them are finite. At $\sigma = 1$ this bound is 21.09 for the third moment and 404.43 for the fourth, the values that the checks below use. Averages under Q are averages under P weighted by the likelihood ratio, $\mathbb{E}_Q[h(W)] = \mathbb{E}_P[(1+qW)h(W)]$ for any function h, so they are finite as well.

3. **The logarithm with a bounded remainder.** For $x \geq -\frac{1}{2}$,
$$\ln(1+x) = x - \frac{x^2}{2} + R(x), \qquad |R(x)| \leq \frac{2}{3}|x|^3. \tag{K.30}$$
The remainder R is what is left after the first two terms. It starts at $R(0) = 0$, and its slope is $R'(x) = \frac{1}{1+x} - 1 + x = \frac{x^2}{1+x}$. For $x \geq 0$ that slope lies between 0 and x^2, so integrating from 0 gives $0 \leq R(x) \leq x^3/3$. For $-\frac{1}{2} \leq x < 0$ the denominator $1+x$ is at least $\frac{1}{2}$, so the

slope is at most $2x^2$, and integrating gives $|R(x)| \le \frac{2}{3}|x|^3$. The logarithm itself has slope $1/(1+x) \le 2$ on the same range, so

$$|\ln(1+x)| \le 2|x| \quad \text{for } x \ge -\tfrac{1}{2}. \tag{K.31}$$

4. **The means.** Under P, by steps 2 and 3, $L = -qW + \frac{q^2W^2}{2} - R(qW)$ has

$$\Big|\mathbb{E}_P[L] - \frac{q^2}{2}v\Big| \le \frac{2}{3}q^3\,\mathbb{E}_P|W|^3. \tag{K.32}$$

Under Q, weighting by $1+qW$ gives $\mathbb{E}_Q[L] = -\frac{q^2}{2}v + \frac{q^3}{2}\mathbb{E}_P[W^3] - \mathbb{E}_P[(1+qW)R(qW)]$, because $\mathbb{E}_P[(1+qW)(-qW + \frac{q^2W^2}{2})] = -q^2v + \frac{q^2}{2}v + \frac{q^3}{2}\mathbb{E}_P[W^3]$. The middle term is at most $\frac{q^3}{2}\mathbb{E}_P|W|^3$ in size. The last is at most $\frac{2}{3}q^3\,\mathbb{E}_P|W|^3 + \frac{2}{3}q^4\,\mathbb{E}_P[W^4]$, by the bound on R and $|1+qW| \le 1+q|W|$. Adding the two gives

$$\Big|\mathbb{E}_Q[L] + \frac{q^2}{2}v\Big| \le q^3\Big(\frac{7}{6}\,\mathbb{E}_P|W|^3 + \frac{2}{3}\,q\,\mathbb{E}_P[W^4]\Big). \tag{K.33}$$

5. **The variances.** Write $L = -qW + r$, where $|r| \le \frac{q^2W^2}{2} + \frac{2}{3}q^3|W|^3$. Then $\mathbb{E}_P[L^2] = q^2v - 2q\,\mathbb{E}_P[Wr] + \mathbb{E}_P[r^2]$. The middle term is at most $q^3\,\mathbb{E}_P|W|^3 + \frac{4}{3}q^4\,\mathbb{E}_P[W^4]$ in size, and the last is of order q^4. The squared mean is of order q^4 by step 4. So $\mathrm{Var}_P(L) = q^2v + O(q^3)$, where $O(q^3)$ means at most a constant times q^3, the constant depending only on σ through the moments of step 2. Under Q, $\mathbb{E}_Q[L^2] = \mathbb{E}_P[L^2] + q\,\mathbb{E}_P[WL^2]$, and $|L| \le 2q|W|$ by step 3 makes the extra term at most $4q^3\,\mathbb{E}_P|W|^3$. So $\mathrm{Var}_Q(L) = q^2v + O(q^3)$ as well.

6. **The third moments.** By step 3, $|L| \le 2q|W|$, so $\mathbb{E}_P|L|^3 \le 8q^3\,\mathbb{E}_P|W|^3$, and under Q the extra weight adds at most $8q^4\,\mathbb{E}_P[W^4]$. As in step 7 of Section K.4, a third absolute central moment is at most 8 times the third absolute moment. So under both laws $\mathbb{E}|L - \mathbb{E}L|^3 = O(q^3)$.

7. **Add up k steps.** Under P the summed loss has mean $K_P = k\,\mathbb{E}_P[L]$ and variance $V_P = k\,\mathrm{Var}_P(L)$, and likewise K_Q and V_Q under Q. With $q\sqrt{k} \to c$, $kq^2 \to c^2$ and $kq^3 = kq^2 \cdot q \to 0$. So steps 4 and 5 give

$$K_P \to \frac{c^2v}{2}, \qquad K_Q \to -\frac{c^2v}{2}, \qquad V_P \to c^2v, \qquad V_Q \to c^2v. \tag{K.34}$$

By step 6 the summed third central moments are $O(kq^3)$, while $V^{3/2}$ tends to $(c^2v)^{3/2} > 0$. So the Berry–Esseen bounds w_P and w_Q under the two laws, each the summed third central moments over $V^{3/2}$, tend to 0. That this ratio tends to 0 is Lyapunov's condition for a triangular array. Lyapunov's CLT would give convergence to the normal without a rate, and Berry–Esseen gives the rate.

8. **The errors of a best test.** As in step 4 of Section K.4, a best test fires when the summed loss S is below a threshold τ, and at τ itself it fires with some probability. Write $x = (\tau - K_P)/\sqrt{V_P}$ for the threshold in standard units under P. Then step 7's Berry–Esseen bound under P gives $|\alpha - \Phi(x)| \le w_P$, as before.

Under Q the loss must be standardized with its own mean and variance. Since $\tau = K_P + x\sqrt{V_P}$, the event $S < \tau$ is the event

$$\frac{S - K_Q}{\sqrt{V_Q}} < \omega_k x + \mu_k, \qquad \omega_k = \sqrt{\frac{V_P}{V_Q}}, \qquad \mu_k = \frac{K_P - K_Q}{\sqrt{V_Q}}. \tag{K.35}$$

By step 7, the scale ratio ω_k tends to 1 and the shift μ_k tends to $c^2 v/(c\sqrt{v}) = c\sqrt{v} = \mu$. So the Berry–Esseen bound under Q puts the power within w_Q of $\Phi(\omega_k x + \mu_k)$.

It remains to show that $\Phi(\omega_k x + \mu_k)$ differs from $\Phi(x + \mu)$ by at most some ξ_k that tends to 0 and does not depend on x. Fix a large number A and split the values of x in two. For $|x| \le A$ the difference is at most $(|\omega_k - 1|A + |\mu_k - \mu|)/\sqrt{2\pi}$, because the normal density is at most $1/\sqrt{2\pi}$. For $|x| > A$, take k large enough that $\omega_k \ge \frac{1}{2}$ and $0 \le \mu_k \le \mu + 1$. Then both arguments are at least $A/2$ when $x > A$ and at most $\mu + 1 - A/2$ when $x < -A$, so both values lie within $\Phi(\mu + 1 - A/2)$ of the same end, 1 or 0. Choosing A large first makes the second part small, and then choosing k large makes the first part small. So $|\beta - \Phi(-x - \mu)| \le w_Q + \xi_k$.

9. **Conclude.** Steps 5 and 6 of Section K.4, with $w = \max(w_P, w_Q + \xi_k)$, put the curve of P^k against Q^k in a band around G_μ whose width tends to 0. Step 7 there turns this into uniform convergence. The remove direction swaps the roles of P and Q. Its loss is $-L$, and its type I error is measured under Q instead of P. Its means are $-K_Q \to \frac{c^2 v}{2}$ under Q and $-K_P \to -\frac{c^2 v}{2}$ under P, so the same argument gives the same limit. Finally, DP-SGD chooses each step after seeing the model so far. But clipping bounds each step's effect, so each step's curve is at least the curve of this pair, as the chapter shows in Section 32.6. By Section K.2 the whole run is then $T(P, Q)^{\otimes k}$-DP, and that curve tends to G_μ.

K.5.4. Checking it

Step 4 says that $(\mathbb{E}_P[L] - \frac{q^2}{2}v)/q^3$ stays bounded, by $\frac{2}{3}\mathbb{E}_P|W|^3 \le 14.06$ at $\sigma = 1$. Computed by numerical integration, it is -3.619, -4.024, -4.292 and -4.453 at $q = 0.02$, 0.01, 0.005 and 0.0025. It settles near a constant, so the remainder really is of order q^3, and the bound holds with room. The same ratios for the mean under Q settle near 2.26, and for the variance under P near -13.2. At the running example's $q = 0.01$, the remainder is about 5% of the leading term for the mean under P (8.1890×10^{-5} against 8.5914×10^{-5}) and about 7% for the variance (1.6009×10^{-4} against 1.7183×10^{-4}).

The Berry–Esseen bound under P is 0.3045, 0.0963 and 0.0304 at $k = 100$, 1000 and 10,000. It shrinks like $1/\sqrt{k}$. The summed third moments grow like kq^3, while $V^{3/2}$ grows like $(kq^2)^{3/2} = k^{3/2}q^3$, so their ratio is of order $1/\sqrt{k}$ whatever q is. The actual distance at $k = 100$, computed from a numerical PLD, is between 0.0312 and 0.0343, so the bound is about nine times too large. Figure K.7 shows the standardized laws. The shift between the two curves is μ, which grows with k: it is 0.1311, 0.4145 and 1.3108.

The theorem controls the bulk, not the tail, and Figure K.8 shows what that costs at $\delta = 10^{-5}$. It plots $\delta(\varepsilon)$ for the add and the remove direction against the Gaussian profile with the CLT's μ. Each direction is computed with a numerical PLD, which rounds each step's loss to a grid of spacing $h = 10^{-5}$. Table K.5 gives the values of ε at $\delta = 10^{-5}$ as brackets. Rounding every loss up gives the upper end of a bracket, which is a valid bound, and rounding down gives the lower

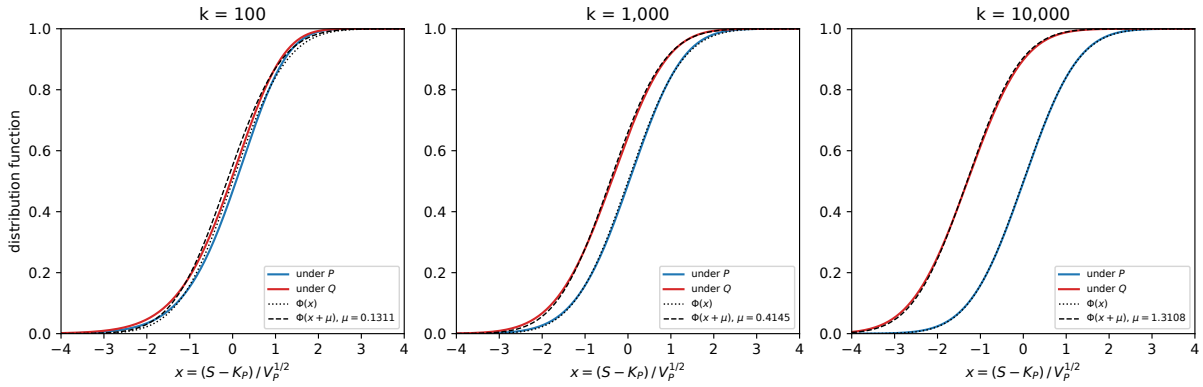


Figure K.7.: Distribution functions of the standardized summed loss $(S - K_P)/\sqrt{V_P}$ of k steps of the Poisson-subsampled Gaussian with $q = 0.01$ and $\sigma = 1$, under P (blue) and Q (red), computed from a numerical PLD, against $\Phi(x)$ and $\Phi(x + \mu)$ with $\mu = q\sqrt{k(e^{1/\sigma^2} - 1)}$. From left to right, $k = 100$, 1,000 and 10,000.

end. The two ends differ by at most kh, since each of the k steps contributes at most h, which is why the brackets widen to 0.1 at $k = 10{,}000$.

Table K.5.: ε at $\delta = 10^{-5}$ for the running DP-SGD example, from the CLT and from a numerical PLD in each direction of neighbours.

k	CLT ε	Add direction	Remove direction
100	0.4575	[0.3377, 0.3387]	[0.7175, 0.7185]
1,000	1.6177	[1.4117, 1.4217]	[1.8232, 1.8332]
10,000	6.0071	[5.5615, 5.6615]	[6.1377, 6.2377]

The two directions err in opposite ways (Table K.5). In the remove direction the loss is $\ln(1+qW)$ under the mixture. The rare outputs where the record was in the batch and the noise pushed y high make W large, so this loss has a long upper tail and δ lies above the Gaussian's. In the add direction the loss $-\ln(1+qW)$ can never exceed $-\ln(1-q)$ per step, so its upper tail is shorter than a Gaussian's. An accountant must take the worse direction, so the CLT under-reports ε at every k shown. Measured against ε itself, the gap shrinks as k grows, in line with the convergence the theorem proves, but at $\delta = 10^{-5}$ it is still 0.13 at $k = 10{,}000$.

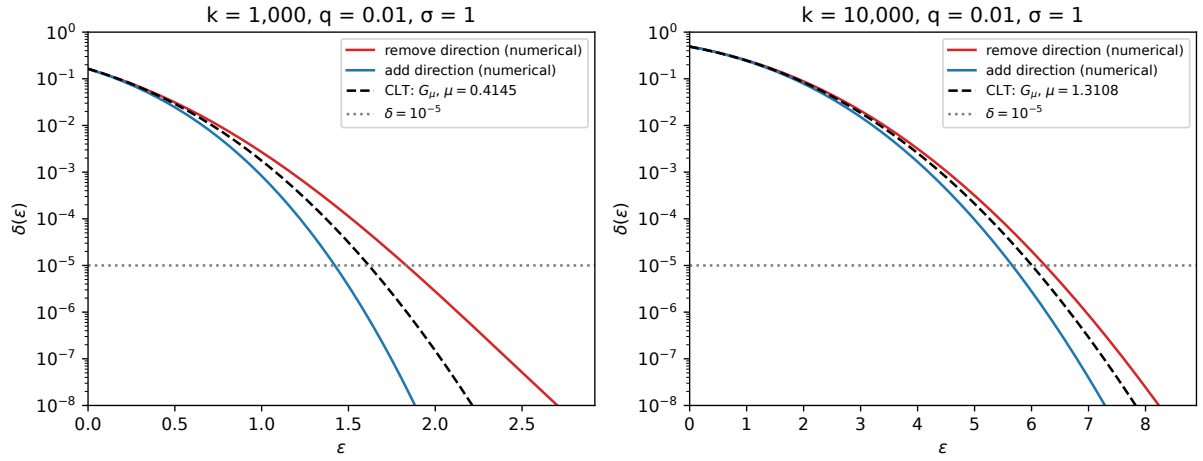


Figure K.8.: The privacy profile $\delta(\varepsilon)$ on a log scale for k steps of the Poisson-subsampled Gaussian with $q = 0.01$, $\sigma = 1$: the remove direction (red) and the add direction (blue) from a numerical PLD, rounded up, and the Gaussian profile of the CLT's μ (dashed). The dotted line is $\delta = 10^{-5}$. Left: $k = 1{,}000$. Right: $k = 10{,}000$.

K.5.5. What it does not cover

K.6. Code

The script code/appendix-derivations/apx_composition.py checks every derivation numerically. It builds the coarsened canonical pairs, computes exact trade-off curves of finite pairs by sorting likelihood ratios, and runs the test replacement on random tests. It computes hockey-stick divergences of the adaptive runs and their dominating products, the exact composed curves of randomized response against the Berry–Esseen band, and the moments and numerical PLDs of the subsampled Gaussian. It uses the standard library and NumPy only and runs in under a minute. From the repository root:

```
uv run python code/appendix-derivations/apx_composition.py
```

Output (seed 0):

Table K.6.: Results printed by the script with seed 0.

Check	Result
Canonical pair, 1000 cells: largest gap to f	G_1: 2.0×10^{-3}; $f_{1,0.1}$: 8.3×10^{-5}
Adaptive randomized response against $f \otimes f$	never below; largest gap 0.2812 at $\alpha = 0.0625$
Test replacement on 20,000 random tests	type I error unchanged (to 3.3×10^{-16}), type II never larger, mean drop 0.4013
Three-step run against the product, 502 values of γ	never above; $\delta(1)$ 0.3926 against 0.4190
100,000 random tests against the product's upper edge	none above; smallest margin 0.0726
CLT band, $k = 100$ randomized responses	$\mu = 1.0004$, $w = 0.0562$; curve inside, at most 0.0014 from G_μ
Subsampled Gaussian, mean remainder over q^3 at $q = 0.0025$	-4.453 under P, bound 14.06
DP-SGD, $k = 10{,}000$: ε at 10^{-5}	CLT 6.0071; remove direction $[6.1377, 6.2377]$

tests/test_appendix_composition.py checks that the output still matches Table K.6 and every other number on this page.

What to take from it. The replacement step never fails: on every random test the type I error is kept and the type II error does not grow, so composition is never worse than the product of the fixed pairs that bound each step. The adaptive examples show how loose that can be when the branches differ. The Berry–Esseen band always holds and is nearly sharp for the distribution functions, but the curve itself is far closer to G_μ. The subsampled-Gaussian expansions have remainders of order q^3, as proved. The CLT still under-reports ε at $\delta = 10^{-5}$, because the tail of the loss is longer than a Gaussian's in the direction that matters.

K.7. References

- Dong, Roth & Su, "Gaussian Differential Privacy" (Journal of the Royal Statistical Society Series B, 2022; arXiv:1905.02383). The source of f-DP, the tensor product (Definition 3.1), adaptive composition (Theorem 3.2, Appendix C) and the CLTs (Theorems 3.4–3.6, Appendix D). Read it for the original proofs, which this appendix follows in spirit.
- Zhu, Dong & Wang, "Optimal Accounting of Differential Privacy via Characteristic Function" (AISTATS 2022). Dominating pairs (Definition 7) and their adaptive composition (Theorem 10), including pairs that may depend on the history.
- Bu, Dong, Long & Su, "Deep Learning with Gaussian Differential Privacy" (Harvard Data Science Review, 2020). The CLT for the Poisson-subsampled Gaussian (Theorem 5), proved here in Section K.5.
- Shevtsova, "An improvement of convergence rate estimates in the Lyapunov theorem" (Doklady Mathematics, 2010). The constant 0.56 in the Berry–Esseen bound for sums of independent, non-identically distributed terms.
- Feller, *An Introduction to Probability Theory and Its Applications*, Volume II (1971), chapter 16, section 5. The Berry–Esseen theorem and Lyapunov's central limit theorem for triangular arrays, with proofs.
- Rockafellar, *Convex Analysis* (1970), §§23–24. One-sided derivatives and supporting lines of convex and concave functions, used in Section K.1 and in step 3 of Section K.3.

L. Subsampling, shuffling and iteration

Part 4 gets much of its privacy from the adversary's uncertainty. For example, a record that a training step samples only with some small probability may not have been used at all, and a report sent through a shuffler hides among the others. Likewise, a record used early in training is covered by the noise of every later step. The chapters state the results that turn this uncertainty into numbers, but several of the proofs are only cited or sketched. This appendix derives five of them in full. They sit together because they share one method: write what the adversary sees as an average, or as a post-processing, of something simpler, and then use two facts. The first is that averaging two runs with the same weights cannot make them easier to tell apart. The second is that processing the output cannot either.

Table L.1 lists the derivations and where the chapters state or use them.

Table L.1.: The derivations of this appendix and the chapter sections that state or use them.

Derivation	Stated or used in
The remove direction dominates in Rényi divergence	Section 33.3 (Equation 33.17); Section 31.6
Dominating pairs for Poisson subsampling, both directions	Section 34.5; Section 37.1
The clone reduction for shuffling, and a closed-form bound	Section 35.2 (Equation 35.5, Equation 35.7)
Shift reduction and contraction	Section 35.3 (Equation 35.12, Equation 35.13); Section 35.4
The random stopping time	Section 35.3

L.1. The remove direction dominates in Rényi divergence

L.1.1. What the chapter uses

Stochastic gradient descent (SGD) run with differential privacy (DP) is called DP-SGD. Each of its steps uses each record with probability q, the **sampling rate**. It adds Gaussian noise to a sum whose sensitivity is 1, and the noise multiplier σ is the standard deviation of that noise. The chapter first reduces the step to one dimension, the direction in which your clipped gradient moves the sum, and takes the worst case, where it moves the sum by the full sensitivity 1. Then the output without your record is the normal distribution $P_0 = \mathcal{N}(0, \sigma^2)$. With your record, the step uses it with probability q and then shifts the mean by 1, so the output is the mixture

$$P_q = (1-q)P_0 + qP_1, \qquad P_1 = \mathcal{N}(1, \sigma^2). \tag{L.1}$$

The neighbouring relation is add/remove, so a guarantee must cover both orders of the pair. The **remove** direction puts the output with your record first, (P_q, P_0), and the **add** direction puts it second, (P_0, P_q). Rényi differential privacy (RDP) measures each direction by a Rényi divergence. For two distributions P and Q and a number $\alpha > 1$, called the order, it is the log-average of a power of the likelihood ratio,

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \mathbb{E}_Q\Big[\Big(\frac{P}{Q}\Big)^\alpha\Big]. \tag{L.2}$$

Amplification by subsampling computes the RDP curve from the remove direction alone, because that direction has an exact binomial series. That is valid only if the remove direction is the larger one:

$$D_\alpha(P_q\|P_0) \;\geq\; D_\alpha(P_0\|P_q) \qquad \text{for every } \alpha > 1,\ q \in [0,1],\ \sigma > 0. \tag{L.3}$$

That chapter states this as part of Equation 33.17 in Section 33.3. Its step 8 cites Mironov, Talwar & Zhang (2019, Theorem 5) and checks the inequality numerically on a grid of orders. Concentrated DP (Section 31.6) cites this inequality to that chapter and to this section. This section supplies the proof, for every real order, not only integer ones.

Both divergences can be written as averages under P_0. Write $r(y) = P_1(y)/P_0(y)$ for the likelihood ratio of the two Gaussians, so that $P_q/P_0 = 1 - q + q\,r$. The remove moment is the chapter's Equation 33.18. The add moment is an average under P_q of $(P_0/P_q)^\alpha$. Moving that average to P_0 multiplies the integrand by P_q/P_0, and $(P_0/P_q)^\alpha \cdot P_q/P_0 = (P_q/P_0)^{1-\alpha}$. So the two moments are

$$A_\alpha = \mathbb{E}_{P_0}\big[(1 - q + q\,r)^\alpha\big] = e^{(\alpha-1)D_\alpha(P_q\|P_0)}, \qquad B_\alpha = \mathbb{E}_{P_0}\big[(1 - q + q\,r)^{1-\alpha}\big] = e^{(\alpha-1)D_\alpha(P_0\|P_q)}. \tag{L.4}$$

Since $\alpha - 1 > 0$, Equation L.3 says exactly that $A_\alpha \geq B_\alpha$.

L.1.2. The idea

The proof cuts the Gaussian pair into many small pieces, each of which is a randomized response. The mirror $y \mapsto 1 - y$ swaps the two Gaussians, because P_0 is centred at 0 and P_1 at 1. So group the outputs in mirrored pairs $\{\frac{1}{2} - z, \frac{1}{2} + z\}$. Within one such pair, P_0 prefers the point $\frac{1}{2} - z$ by some factor and P_1 prefers the other point by the same factor. That is binary randomized response, a coin that reports one of two answers and favours the true one. All three distributions also put the same total mass on each mirrored pair, because the mirror swaps P_0 and P_1 and keeps the pair. So both moments A_α and B_α are averages, with shared weights, of the same moments for randomized-response pairs. It is enough to prove $A_\alpha \geq B_\alpha$ for randomized response of every strength.

For randomized response the two moments are sums over two outputs, and they have a closed form. Their gap turns out to be a difference of two averages of the function tanh over two intervals with the same centre, one wider than the other. The common centre turns out never to be negative. Since tanh rises steeply near 0 and flattens out far to the right, widening an interval with such a centre can only lower the average, so the gap is never negative.

Take a concrete pair, in which P_0 reports "+" with probability 0.7311 and "−" with probability 0.2689, and P_1 is the reverse. With $q = 0.1$, the mixture $P_q = 0.9P_0 + 0.1P_1$ reports "+" with

probability 0.6848 (Table L.2). At $\alpha = 2$ the remove and add moments of this two-point pair, which we call A' and B' to keep them apart from the Gaussian A_α and B_α, are sums over the two outputs,

$$A' = \sum \frac{P_q^2}{P_0}, \qquad B' = \sum \frac{P_0^2}{P_q}. \tag{L.5}$$

The appendix's script (see Section L.6) gives $\ln A' = 0.010803$ and $\ln B' = 0.009846$, so here the remove direction is larger, as claimed.

Table L.2.: A randomized-response pair, with $P_0 = \pi_{0.5}$ in the notation of step 1, and its mixture at $q = 0.1$.

Output	P_0	P_1	$P_q = 0.9P_0 + 0.1P_1$
+	0.7311	0.2689	0.6848
−	0.2689	0.7311	0.3152

L.1.3. Derivation

1. **Name the two-point distributions.** For a real number u, let π_u put probability $e^u/(2\cosh u)$ on "+" and $e^{-u}/(2\cosh u)$ on "−". It is randomized response whose privacy loss is $\pm 2u$, since the two probabilities differ by the factor e^{2u}, and π_{-u} is the same coin reversed. The probability of "+" is $(1 + \tanh u)/2$, so $\tanh u$ is how strongly the coin leans to "+". In Table L.2, $P_0 = \pi_{0.5}$ and $P_1 = \pi_{-0.5}$.

2. **Split the Gaussian pair into mirrored pairs.** Fix $z > 0$ and compare the two points $\frac{1}{2} \mp z$. The Gaussian density is $e^{-y^2/(2\sigma^2)}$ up to a constant, so

$$\frac{P_0(\frac{1}{2} - z)}{P_0(\frac{1}{2} + z)} = \exp\Big(\frac{(\frac{1}{2} + z)^2 - (\frac{1}{2} - z)^2}{2\sigma^2}\Big) = e^{z/\sigma^2}. \tag{L.6}$$

   The mirror gives $P_1(y) = P_0(1 - y)$, so P_1 has the reverse ratio. Label the point $\frac{1}{2} - z$, on the side of P_0, as "+". Given that the output lies in the pair, P_0 is then π_a and P_1 is π_{-a}, with $a = z/(2\sigma^2)$.

3. **The pairs have the same weight under all three distributions.** Let $m(z) = P_0(\frac{1}{2} - z) + P_0(\frac{1}{2} + z)$ be the density of the pair under P_0. Under P_1 the two points swap, so the total is the same, and P_q is a mixture of the two. Write each density as $m(z)$ times a conditional probability on the pair. In $P_q(y)^\alpha P_0(y)^{1-\alpha}$ the powers of $m(z)$ add up to $\alpha + (1 - \alpha) = 1$, so

$$A_\alpha = \int_0^\infty m(z)\, A'(a)\, dz, \qquad B_\alpha = \int_0^\infty m(z)\, B'(a)\, dz, \qquad a = \frac{z}{2\sigma^2}, \tag{L.7}$$

   where $A'(a)$ and $B'(a)$ are the two moments of the randomized-response pair $(\pi_a, (1 - q)\pi_a + q\pi_{-a})$. If $A'(a) \geq B'(a)$ for every $a \geq 0$, then $A_\alpha \geq B_\alpha$.

4. **The mixture is again a randomized response.** Mixing averages the probabilities of "+", and so averages the leaning tanh. So $(1 - q)\pi_a + q\pi_{-a} = \pi_b$, where

$$\tanh b = (1 - q)\tanh a + q\tanh(-a) = (1 - 2q)\tanh a. \tag{L.8}$$

Since $|1-2q| \le 1$ and tanh is increasing, $-a \le b \le a$. In the example, $a = 0.5$ and $b = 0.3881$.

5. **Compute the two moments.** On "+" the term $\pi_b^{\alpha}\pi_a^{1-\alpha}$ is $e^{\alpha b+(1-\alpha)a}$ divided by $(2\cosh b)^{\alpha}(2\cosh a)^{1-\alpha}$, and on "−" the exponent changes sign. So the two exponentials add up to a hyperbolic cosine. Writing $\ell(u)$ for $\ln\cosh u$, the logarithms of the two moments are

$$\ln A' = \ell(\alpha b + (1-\alpha)a) - \alpha\,\ell(b) - (1-\alpha)\,\ell(a), \qquad \ln B' = \ell(\alpha a + (1-\alpha)b) - \alpha\,\ell(a) - (1-\alpha)\,\ell(b). \tag{L.9}$$

6. **Turn the gap into two averages of tanh.** Let $d = a-b \ge 0$. Then $\alpha b+(1-\alpha)a = a-\alpha d$ and $\alpha a + (1-\alpha)b = a + (\alpha-1)d$. Subtracting the two lines of step 5 and collecting the terms in $\ell(a)$ and $\ell(b)$ gives

$$\ln A' - \ln B' = (2\alpha-1)\big[\ell(a) - \ell(a-d)\big] - \big[\ell(a+(\alpha-1)d) - \ell(a-\alpha d)\big]. \tag{L.10}$$

The derivative of ℓ is tanh, so each bracket is an integral of tanh over an interval. The first is over $I_1 = [b, a]$, of length d. The second is over $I_2 = [a-\alpha d,\ a+(\alpha-1)d]$, of length $(2\alpha-1)d$. Writing each integral as length times average,

$$\ln A' - \ln B' = (2\alpha-1)\,d\,\Big[\operatorname{avg}_{I_1}\tanh - \operatorname{avg}_{I_2}\tanh\Big]. \tag{L.11}$$

Both intervals have the same centre, $c = a - d/2 = (a+b)/2$. For $\alpha \ge 1$, I_2 is at least as wide as I_1 and contains it.

7. **A wider interval has a smaller average.** Let $G(h)$ be the average of tanh over $[c-h, c+h]$. Pairing each point $c+x$ with its mirror $c-x$,

$$G(h) = \int_0^1 \tfrac{1}{2}\big[\tanh(c+hs) + \tanh(c-hs)\big]\,ds. \tag{L.12}$$

For $x \ge 0$ the bracket $\tanh(c+x) + \tanh(c-x)$ has derivative $\operatorname{sech}^2(c+x) - \operatorname{sech}^2(c-x)$. The function sech^2 is even and decreases as its argument moves away from 0. When $c \ge 0$ and $x \ge 0$, the point $c+x$ is at least as far from 0 as $c-x$, so the derivative is at most 0. So every bracket, and with it G, can only decrease as h grows.

8. **Conclude.** The centre satisfies $c = (a+b)/2 \ge 0$ because $b \ge -a$ (step 4). Step 7 then gives $\operatorname{avg}_{I_1}\tanh \ge \operatorname{avg}_{I_2}\tanh$, so Equation L.11 is at least 0. That proves $A'(a) \ge B'(a)$ for every a, and step 3 gives $A_\alpha \ge B_\alpha$, which is Equation L.3.

In the example, $c = 0.4440$. At $\alpha = 2$ the average of tanh is 0.416623 over I_1 and 0.413773 over the three-times-wider I_2. Multiplying their difference by $(2\alpha-1)d$ gives 0.000957, which is $\ln A' - \ln B' = 0.010803 - 0.009846$. Figure L.1 draws the two intervals for a larger example, where the difference is easy to see.

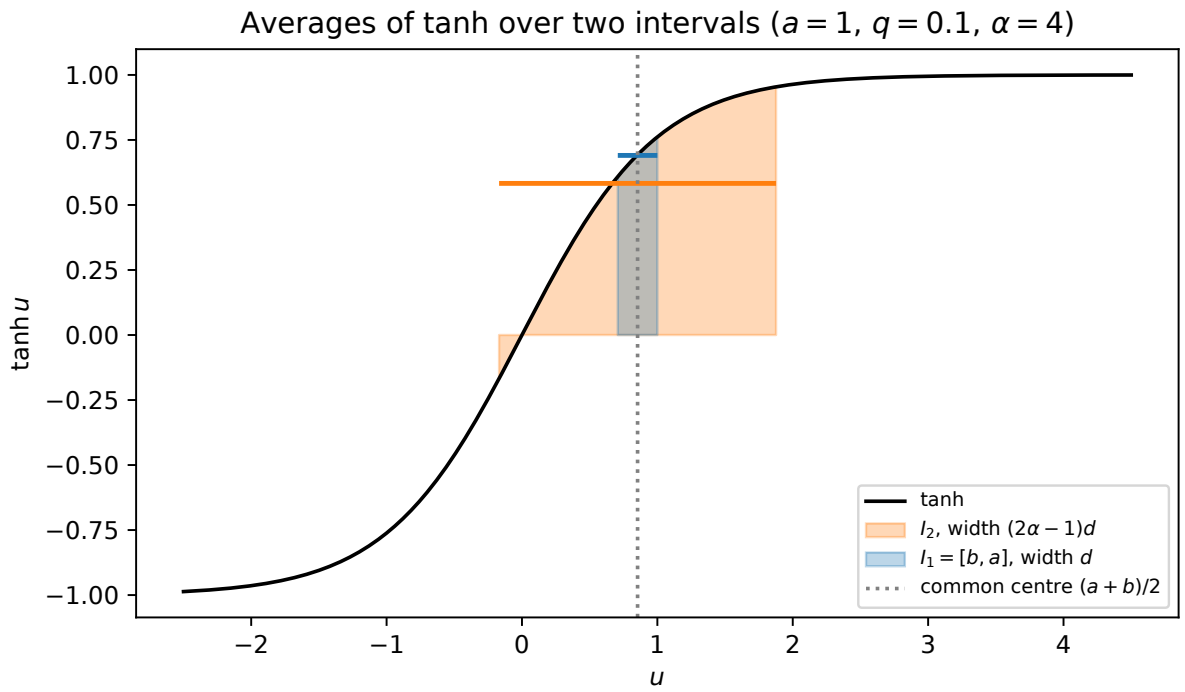


Figure L.1.: The function tanh with the two intervals of step 6 for $a = 1$, $q = 0.1$ and $\alpha = 4$. Both are centred at $(a + b)/2$. The horizontal bars are the averages of tanh over each interval. The wide interval reaches into the flat part on the right and the steep part on the left, so its average is lower.

L.1.4. Checking it

The script checks the pair inequality on a grid of 5,294,646 points. The grid has nine sampling rates from 0.001 to 0.9, 294 orders from 1.01 to 256, and 2,001 pair strengths a, up to likelihood ratios e^{2a} of 10^6. No point has a negative gap. The smallest value, -2.2×10^{-16}, is rounding at $a = 0$, where both moments equal 1 and their logarithms 0.

It also computes both directions for the Gaussian in two ways: by integrating over the output, as the chapter does, and by averaging the pair moments of step 3. Table L.3 shows that the two ways agree to every printed digit and that the remove direction is the larger.

Table L.3.: The two directions of the subsampled Gaussian with $q = 0.01$ and $\sigma = 1$. Integration over the output and the average over mirrored pairs give the same values.

α	$D_\alpha(P_q \Vert P_0)$	$D_\alpha(P_0 \Vert P_q)$
2	1.7181×10^{-4}	1.6022×10^{-4}
8	8.9364×10^{-4}	5.7239×10^{-4}
32	1.1246×10^{1}	1.6923×10^{-3}

Figure L.2 shows the decomposition and the gap for each mirrored pair. The gap is 0 only for the pair at $z = 0$, where the two points coincide, and it grows with the strength of the pair. Strong pairs sit far out in the tails of the Gaussians, and those are the outputs that high orders weight most. That is why the two directions differ by nearly four orders of magnitude at $\alpha = 32$.

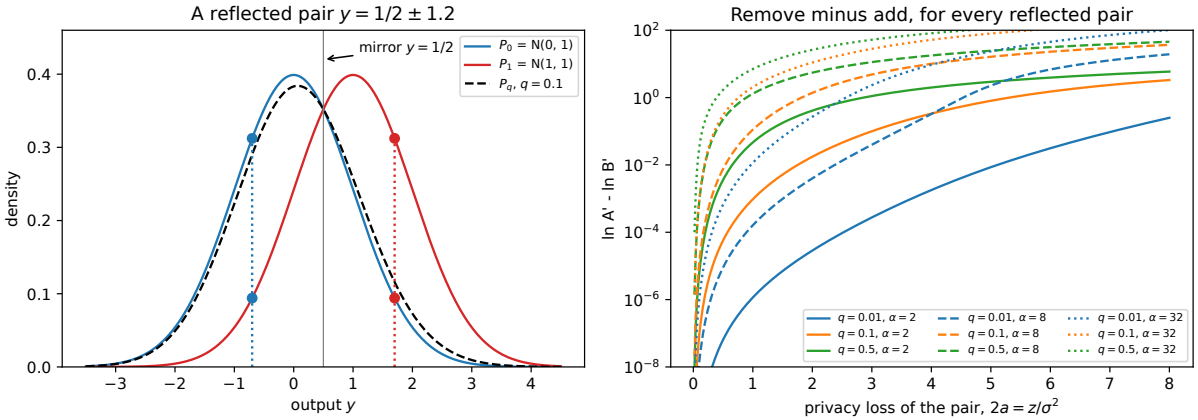


Figure L.2.: Left: P_0, P_1 and the mixture P_q for $q = 0.1$ and $\sigma = 1$, with one mirrored pair of outputs, $y = \frac{1}{2} \pm 1.2$. The density of P_0 at the left point equals that of P_1 at the right point, and the other way round. Right: the gap $\ln A' - \ln B'$ for each mirrored pair, against the pair's privacy loss $2a$, for three sampling rates and three orders. Every curve is positive.

L.1.5. What it does not cover

The proof uses only the mirror that swaps P_0 and P_1. So it covers every pair with such a mirror, for example the Laplace mechanism, but not a pair without one. It is also a statement about

Rényi divergences. For the hockey-stick divergence, which gives $\delta(\varepsilon)$, neither direction dominates in general, which is why Section 34.5 computes both. Mironov, Talwar & Zhang (2019, Theorem 5) state the same inequality, and the argument here is a self-contained route to it.

L.2. Dominating pairs for Poisson subsampling

L.2.1. What the chapter uses

Take add/remove neighbours, and write $D' = D \cup \{x\}$ for the dataset that holds your record x. A guarantee for a mechanism M must cover both orders of the pair, so there are two directions. The **remove** direction compares $M(D')$ against $M(D)$, with the dataset that holds the record first. The **add** direction compares $M(D)$ against $M(D')$.

One DP-SGD step draws a Poisson sample K of the data, keeping each record with probability q by its own coin. Write $g(u)$ for the clipped gradient of record u, divided by the clipping norm so that $\|g(u)\| \le 1$. The step outputs the noisy sum

$$M(D) = \sum_{u \in K} g(u) + \mathcal{N}(0, \sigma^2 I). \tag{L.13}$$

The gradients may depend on earlier steps, but they are fixed when the step runs.

The **hockey-stick divergence** at level γ is the largest amount by which P exceeds γ times Q on any event, $E_\gamma(P\|Q) = \max_A \left[P(A) - \gamma Q(A)\right]$. At $\gamma = e^\varepsilon$ it is the $\delta(\varepsilon)$ of the pair. The definition makes sense for every real γ, and for $\gamma \le 0$ the best event is the whole space, so $E_\gamma = 1 - \gamma$. A pair **dominates** one direction if its E_γ is at least the mechanism's E_γ in that direction, for every $\gamma \ge 0$ and all datasets. With $P_0 = \mathcal{N}(0, \sigma^2)$, $P_1 = \mathcal{N}(1, \sigma^2)$ and $P_q = (1-q)P_0 + qP_1$ as in Section L.1, the claim is that, for every $\gamma \ge 0$,

$$E_\gamma(M(D') \,\|\, M(D)) \le E_\gamma(P_q \| P_0), \qquad E_\gamma(M(D) \,\|\, M(D')) \le E_\gamma(P_0 \| P_q). \tag{L.14}$$

Numerical accounting states the two pairs in Section 34.5. It cites Zhu, Dong & Wang (2022, Theorem 11), and the sentence after its Theorem 34.5 says that they "prove each pair dominates its case". Choosing an accountant relies on both pairs in Section 37.1.

L.2.2. The idea

Two exact identities do the work, one for each order. Take a mixture $(1-q)Q + qP$ and one of its components, Q. On any event, the mixture's excess over γQ is q times the excess of P over $\gamma' Q$, at the stretched level $\gamma' = 1 + (\gamma - 1)/q$. The add direction, with the mixture second, has a similar identity. So the δ curve of the mixture against Q is the δ curve of P against Q, scaled by q and read at a different level. A larger base curve then gives a larger mixture curve, at every level.

The rest is bookkeeping. Fix everything except your record's coin, and the step is exactly such a mixture, with a Gaussian base pair whose shift is your clipped gradient. Of all such base pairs, the one with shift exactly 1 along one axis has the largest curve. Finally, averaging over the other records' coins cannot raise the curve.

For a feel of the stretch, take $q = 0.01$ and $\gamma = 1.5$. The remove identity reads the base pair at $\gamma' = 1 + 0.5/0.01 = 51$. For the subsampled step to make some event 1.5 times likelier, the base mechanism must make it 51 times likelier, which only happens far in its tail.

L.2.3. Derivation

1. **Fix the other records' coins.** Let K be the Poisson sample of D. Your record has its own coin, independent of K, so the sample of D' is K with probability $1-q$ and $K \cup \{x\}$ otherwise. Given K, $M(D)$ is $Q_K = \mathcal{N}(f_K, \sigma^2 I)$, with f_K the sum of the gradients in K. And $M(D')$ is $(1-q)Q_K + qP_K$, with $P_K = \mathcal{N}(f_K + v, \sigma^2 I)$ and $v = g(x)$, so $\|v\| \le 1$.

2. **Averaging over K cannot raise E_γ.** Both runs are averages over K with the same weights w_K. Write U_K and V_K for the two conditional outputs given K, in either order. For every event, $\sum_K w_K U_K(A) - \gamma \sum_K w_K V_K(A)$ is the weighted average of the excesses $U_K(A) - \gamma V_K(A)$, and an average is at most the largest term. So $E_\gamma(\sum_K w_K U_K \| \sum_K w_K V_K) \le \max_K E_\gamma(U_K \| V_K)$, for every real γ. This is Lemma 33.2 of Amplification by subsampling. It reduces both claims to one fixed K.

3. **The remove identity.** For any event A, collect the terms in $Q(A)$: $(1-q)Q(A) + qP(A) - \gamma Q(A) = q[P(A) - \gamma' Q(A)]$, where $q\gamma' = \gamma - 1 + q$. Taking the maximum over A on both sides gives

$$E_\gamma((1-q)Q + qP \,\|\, Q) = q\, E_{\gamma'}(P\|Q), \qquad \gamma' = 1 + \frac{\gamma - 1}{q}. \tag{L.15}$$

   This holds for every γ. When $\gamma < 1 - q$, the level γ' is negative and both sides equal $1 - \gamma$, since $q(1 - \gamma') = 1 - \gamma$.

4. **The add identity.** Now the mixture is second. Collect the terms in $Q(A)$ again: $Q(A) - \gamma(1-q)Q(A) - \gamma q P(A) = c\, Q(A) - \gamma q\, P(A)$, with $c = 1 - \gamma(1-q)$. If $c > 0$, factor it out. If $c \le 0$, every event gives at most 0, because $\gamma \ge 0$ makes the last term non-positive, and the empty event gives exactly 0. So

$$E_\gamma(Q \,\|\, (1-q)Q + qP) = \begin{cases} c\, E_{\gamma q/c}(Q\|P), & c = 1 - \gamma(1-q) > 0, \\ 0, & c \le 0. \end{cases} \tag{L.16}$$

5. **The base pair with shift 1 is the worst.** Compare (P_K, Q_K) with (P_1, P_0) at every level and in both orders. Shifting both distributions by $-f_K$ and rotating v onto the first axis is an invertible map, the same for both, so it changes no E_γ. The other coordinates are then independent and identical in both, so dropping them and adding them back are both post-processing, and they change nothing either. What is left is $\mathcal{N}(\|v\|, \sigma^2)$ against $\mathcal{N}(0, \sigma^2)$. If $\|v\| < 1$, this pair is a post-processing of (P_1, P_0). The map $y \mapsto \|v\|\, y + \mathcal{N}(0, (1 - \|v\|^2)\sigma^2)$ turns P_1 into $\mathcal{N}(\|v\|, \sigma^2)$ and P_0 into P_0. **Data processing** says no such map raises E_γ. The reason is that a test on the output pulls back to a test on the input with acceptance probability between 0 and 1, and the best such test is an event. So $E_\gamma(P_K \| Q_K) \le E_\gamma(P_1 \| P_0)$ and $E_\gamma(Q_K \| P_K) \le E_\gamma(P_0 \| P_1)$.

6. **Put the pieces together.** Both identities are increasing in the base divergence, because $q > 0$ and $c > 0$ where it matters. For the remove direction, steps 2, 3 and 5 give

$$E_\gamma(M(D') \| M(D)) \le \max_K q\, E_{\gamma'}(P_K \| Q_K) \le q\, E_{\gamma'}(P_1 \| P_0) = E_\gamma(P_q \| P_0), \tag{L.17}$$

where the last equality is Equation L.15 read backwards for the Gaussian pair. The add direction is the same with Equation L.16, and both sides are 0 when $c \leq 0$. This is Equation L.14.

7. **One base curve for both directions.** The mirror $y \mapsto 1 - y$ swaps P_0 and P_1 and is invertible, so $E_\gamma(P_0 \| P_1) = E_\gamma(P_1 \| P_0)$ for every γ. Both identities therefore read the same curve, the Gaussian δ curve Equation 34.4 with $\mu = 1/\sigma$, at different levels.

A bonus: amplification for any mechanism. The identity in step 3 used nothing about Gaussians. Let M be any (ε, δ)-DP mechanism, and in step 1 let $Q_K = M(K)$ and $P_K = M(K \cup \{x\})$ be its outputs on two neighbouring samples. Set $e^{\varepsilon'} = 1 + q(e^\varepsilon - 1)$. Then the stretched level is $\gamma' = 1 + (e^{\varepsilon'} - 1)/q = e^\varepsilon$, and Equation L.15 gives $E_{e^{\varepsilon'}}((1-q)Q_K + qP_K \| Q_K) = q\, E_{e^\varepsilon}(P_K \| Q_K) \leq q\delta$. With step 2, that is the remove half of Equation 33.8 in one line.

L.2.4. Checking it

The script computes one step with $q = 0.01$ and $\sigma = 1$ in two ways. One uses the closed forms of step 6 of Section 34.5, which find the output threshold where the loss crosses ε. The other uses the two identities with the Gaussian curve. Over a grid of ε from -0.5 to 3, the largest relative gap between the two is 7.4×10^{-14} in the remove direction. In the add direction it is 3.0×10^{-12}. At $\varepsilon = 0.5$, for example, both give the remove value 2.214519×10^{-7}, the exact value of Table 34.6. The bonus checks out too. With base $\varepsilon = 1$ the amplified level is $\varepsilon' = 0.017037$. There, the subsampled $\delta(\varepsilon')$ equals q times the base $\delta(1)$, and both are 1.269367×10^{-3}.

The script also checks the domination on a small dataset that is not a worst case. In one dimension, the other records have gradients 0.3 and -0.5, your record has 0.8, $q = 0.3$ and $\sigma = 1$. The two other records give four possible samples, and adding yours gives eight, so the real outputs are mixtures of four and eight Gaussians. The script computes their E_γ at five levels from 0.8 to 5, in both directions. The real divergence is never above the pair's, and its largest ratio to the pair's is 0.9687: your gradient is shorter than 1, so the bound is not reached.

Figure L.3 shows the stretch and the result. In the add direction $\delta(\varepsilon)$ drops to 0 at $\varepsilon = -\ln(1-q) = 0.0101$, where c reaches 0.

L.2.5. What it does not cover

The pairs dominate one step, for add/remove neighbours. Composing them over many adaptively chosen steps needs the dominating-pair composition theorem, which Section 34.2 states and Appendix Section K.3 derives. Replace-one neighbours, or fixed-size batches, give different pairs, as Section 33.2 explains.

L.3. The clone reduction for shuffling

L.3.1. What the chapter uses

In the shuffle model, n users each send one report through a shuffler, which hides the order. The neighbouring relation is replace-one: you hold x^0 in D and x^1 in D', and everyone else holds the

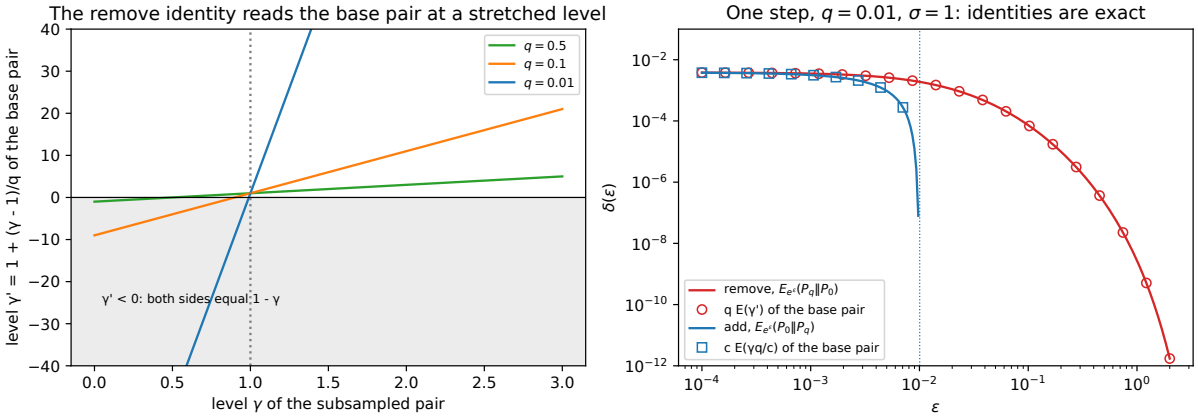


Figure L.3.: Left: the stretched level $\gamma' = 1 + (\gamma - 1)/q$ of the remove identity for three sampling rates. In the shaded region the level is negative and both sides equal $1 - \gamma$. Right: one step with $q = 0.01$ and $\sigma = 1$. The lines are $\delta(\varepsilon)$ of the remove pair (red) and of the add pair (blue), computed directly. The markers are the two identities applied to the Gaussian curve, and they fall on the lines. The dotted line marks $\varepsilon = -\ln(1-q)$.

same value in both. Every user applies the same local randomizer R, a randomized map from a value to a report, and $R(x)$ is the distribution of the report of a user who holds x. It is ε_0-**DP**: no report is more than e^{ε_0} times likelier under one input than under another.

Theorem 35.2 (Theorem 3.2 of Feldman, McMillan & Talwar 2021) says that the shuffled output on D and on D' is the same post-processing of the two random pairs of counts of Equation 35.5,

$$\tilde{P} = (A + F,\ N_c - A + 1 - F), \qquad \tilde{Q} = (A + 1 - F,\ N_c - A + F). \tag{L.18}$$

The number of clones N_c is binomial, $\mathrm{Bin}(n-1, e^{-\varepsilon_0})$. A clone is another user's report that is distributed like one of your two possible reports, and the idea below makes this precise. Given N_c, the number A of clones of the first kind is $\mathrm{Bin}(N_c, \frac{1}{2})$. The bit F, independent of both, says whether your own report favours your true value, and it is 1 with probability

$$p = \frac{e^{\varepsilon_0}}{e^{\varepsilon_0} + 1}. \tag{L.19}$$

Section 35.2 proves a weaker reduction, in which your own report counts fully on one side, with counts $(A + 1, N_c - A)$ against $(A, N_c - A + 1)$. That pair ignores the noise in your own report. Steps 4 and 5 of Theorem 35.2's proof then replace your exact contribution by the noisy bit F and redraw the other users' clones, resting on Lemma 3.4 of the paper. It also cites, without the calculation, the Chernoff bounds that give the closed form Equation 35.7. This section proves the full reduction for one fixed randomizer and derives a closed-form bound with explicit constants.

L.3.2. The idea

Two facts about randomized response carry the proof. First, your own report can be produced in two stages. A biased coin picks one of two "pure" report distributions, Q_0 or Q_1, and then a report is drawn from it. On D the coin picks Q_0 with probability p, and on D' it picks Q_1 with

probability p. So your report is a randomized-response bit followed by processing that does not depend on your data.

Second, every other user's report contains, with probability $e^{-\varepsilon_0}$, a draw from the average $\frac{1}{2}(R(x^0) + R(x^1))$. That average equals $\frac{1}{2}(Q_0 + Q_1)$. So such a **clone** can be produced by a fair coin that picks Q_0 or Q_1. After shuffling, all that matters is how many reports came from Q_0 and how many from Q_1. Those two counts are $\tilde{P}$ on D and $\tilde{Q}$ on D'.

Take three-valued randomized response with $\varepsilon_0 = \ln 4$. It reports the true value with probability $\frac{2}{3}$ and each other value with probability $\frac{1}{6}$, so $p = \frac{4}{5}$. You hold 0 on D and 1 on D'. Table L.4 lists the pieces, and Figure L.4 draws them.

Table L.4.: The clone decomposition for three-valued randomized response with $\varepsilon_0 = \ln 4$. The clone part of another user's $R(2)$ is $e^{-\varepsilon_0} = \frac{1}{4}$ times $\frac{1}{2}(Q_0 + Q_1)$, and the leftover is the rest, rescaled by $\frac{4}{3}$ to a distribution.

Report	$R(0)$	$R(1)$	Q_0	Q_1	$\frac{1}{2}(Q_0 + Q_1)$	$R(2)$	clone part of $R(2)$	leftover $\mathrm{LO}(2)$
0	2/3	1/6	5/6	0	5/12	1/6	5/48	1/12
1	1/6	2/3	0	5/6	5/12	1/6	5/48	1/12
2	1/6	1/6	1/6	1/6	1/6	2/3	1/24	5/6

You can check two rows by hand. On report 0, $pQ_0 + (1-p)Q_1 = \frac{4}{5} \cdot \frac{5}{6} + 0 = \frac{2}{3}$, which is $R(0)$. And $\frac{1}{2}(R(0) + R(1))$ on report 0 is $\frac{1}{2}(\frac{2}{3} + \frac{1}{6}) = \frac{5}{12}$, the same as $\frac{1}{2}(Q_0 + Q_1)$.

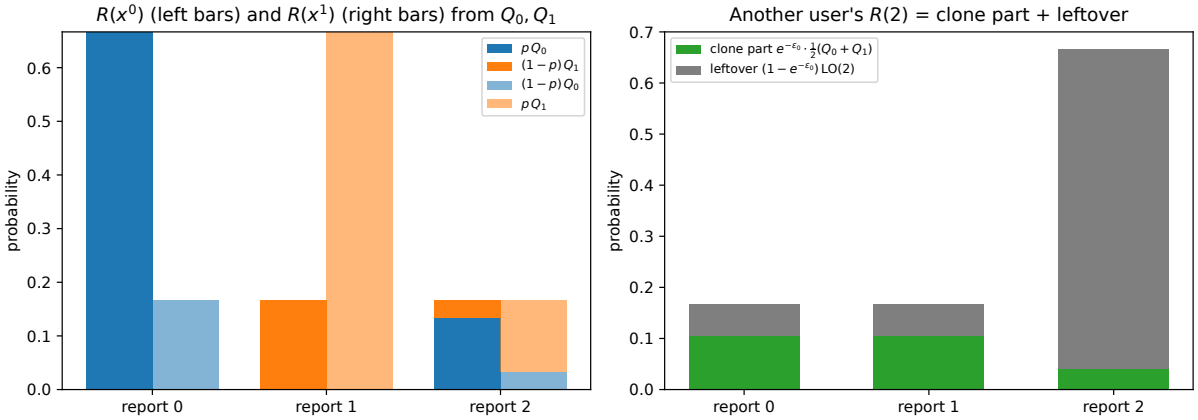


Figure L.4.: Left: the bars for $R(x^0)$ (solid) and $R(x^1)$ (pale) on the three reports, each split into its Q_0 part (blue) and its Q_1 part (orange), for three-valued randomized response with $\varepsilon_0 = \ln 4$. Right: another user's $R(2)$, split into the clone part $e^{-\varepsilon_0} \cdot \frac{1}{2}(Q_0 + Q_1)$ (green) and the leftover (grey).

L.3.3. Derivation

Steps 1 to 6 prove the reduction, and steps 7 to 13 bound the pair. Assume $\varepsilon_0 > 0$, since with $\varepsilon_0 = 0$ every report has the same distribution and there is nothing to prove.

1. **Define the two pure distributions.** Let

$$Q_0 = \frac{e^{\varepsilon_0} R(x^0) - R(x^1)}{e^{\varepsilon_0} - 1}, \qquad Q_1 = \frac{e^{\varepsilon_0} R(x^1) - R(x^0)}{e^{\varepsilon_0} - 1}. \tag{L.20}$$

   Each is non-negative on every report, because ε_0-DP gives $R(x^1)(y) \le e^{\varepsilon_0} R(x^0)(y)$, and the reverse. Each sums to $(e^{\varepsilon_0} - 1)/(e^{\varepsilon_0} - 1) = 1$. So both are distributions.

2. **Your report is randomized response followed by a draw.** Multiply out $pQ_0 + (1-p)Q_1$ over the common denominator $(e^{\varepsilon_0} + 1)(e^{\varepsilon_0} - 1)$. The terms in $R(x^1)$ are $-e^{\varepsilon_0} + e^{\varepsilon_0} = 0$, and the term in $R(x^0)$ is $e^{2\varepsilon_0} - 1$, so

$$R(x^0) = p\,Q_0 + (1-p)\,Q_1, \qquad R(x^1) = (1-p)\,Q_0 + p\,Q_1, \tag{L.21}$$

   where the second line is the same computation with x^0 and x^1 swapped.

3. **The average of your two report distributions is the average of the pure ones.** Adding the two definitions in step 1 gives $Q_0 + Q_1 = R(x^0) + R(x^1)$, because the factors $e^{\varepsilon_0} - 1$ cancel. So $\frac{1}{2}(R(x^0) + R(x^1)) = \frac{1}{2}(Q_0 + Q_1)$.

4. **Every other report contains a clone.** For another user holding x, ε_0-DP gives $R(x) \ge e^{-\varepsilon_0} R(x^0)$ and $R(x) \ge e^{-\varepsilon_0} R(x^1)$ on every report. Average the two and use step 3. The difference between $R(x)$ and $e^{-\varepsilon_0} \cdot \frac{1}{2}(Q_0 + Q_1)$ is non-negative with total mass $1 - e^{-\varepsilon_0}$, so it is that mass times a distribution $\mathrm{LO}(x)$, the leftover:

$$R(x) = \frac{e^{-\varepsilon_0}}{2}\,Q_0 + \frac{e^{-\varepsilon_0}}{2}\,Q_1 + (1 - e^{-\varepsilon_0})\,\mathrm{LO}(x). \tag{L.22}$$

   This is Equation 35.6 of the chapter, with the clones written in terms of Q_0 and Q_1.

5. **Count the faces.** Read each mixture as a coin with faces, followed by a draw from the face's distribution. Each other user's coin shows "Q_0" or "Q_1" with probability $e^{-\varepsilon_0}/2$ each, and "leftover" otherwise, independently. So the number of clones is $N_c \sim \mathrm{Bin}(n-1, e^{-\varepsilon_0})$, and $A \sim \mathrm{Bin}(N_c, \frac{1}{2})$ of them show Q_0. Your own coin (step 2) shows Q_0 with probability p on D and $1-p$ on D'. Let $F = 1$ when your coin shows the face on your side, Q_0 on D and Q_1 on D'. Then the numbers of Q_0 and Q_1 faces among all users are $(A+F,\ N_c - A + 1 - F)$ on D and $(A + 1 - F,\ N_c - A + F)$ on D'. These are $\tilde{P}$ and $\tilde{Q}$.

6. **The shuffled output is a post-processing of the counts.** Here is the map, which makes Lemma 3.3 of the paper explicit. It receives counts (N_0, N_1) and knows every other user's value. It sets $n_c = N_0 + N_1 - 1$, since one of the $N_0 + N_1$ pure draws is your own report, and picks a uniformly random set of n_c other users as clones. It then draws N_0 reports from Q_0 and N_1 reports from Q_1. Every other user who was not picked gets one report from their own leftover distribution. Finally, the map outputs all n reports in a uniformly random order.

   Three facts show that this reproduces the shuffled output exactly. First, in the real process each other user is a clone independently and with the same probability. So, given that there are n_c clones, which users they are is a uniformly random set of size n_c, independent of the faces. Second, given the faces, all reports are independent draws from Q_0, Q_1 or the leftovers. Third, the shuffler's random order removes every link between a report and its sender. The map uses only the counts, the other users' values and fresh randomness, which are the same on D and D'. So by data processing, every E_γ and D_α of the shuffled output is at most that of $(\tilde{P}, \tilde{Q})$.

7. **Condition on N_c.** Given $N_c = n_c$, the second count is $n_c + 1$ minus the first, so the first count a carries everything. The law of N_c is the same in both runs, so the pair is a shared-weight average over n_c of one-dimensional pairs.

8. **The likelihood ratio.** Write B_{n_c} for the probabilities of $\mathrm{Bin}(n_c, \frac{1}{2})$. Given n_c, $\tilde{P}(a) = p\,B_{n_c}(a-1) + (1-p)B_{n_c}(a)$ and $\tilde{Q}(a) = (1-p)B_{n_c}(a-1) + p\,B_{n_c}(a)$. Since $B_{n_c}(a-1)/B_{n_c}(a) = \binom{n_c}{a-1}/\binom{n_c}{a} = a/(n_c - a + 1)$, dividing gives $\frac{pa+(1-p)(n_c-a+1)}{(1-p)a+p(n_c-a+1)}$. Measure a from the middle, $a = \frac{n_c+1}{2} + t$, and write $b = 2p - 1$. Then

$$\frac{\tilde{P}(a)}{\tilde{Q}(a)} = \frac{\frac{n_c+1}{2} + b\,t}{\frac{n_c+1}{2} - b\,t}, \qquad b = \frac{e^{\varepsilon_0} - 1}{e^{\varepsilon_0} + 1}. \tag{L.23}$$

   So the privacy loss, the logarithm of this ratio, grows with the distance t above the middle.

9. **δ is at most the chance of a large loss.** In $E_{e^\varepsilon}(\tilde{P}\|\tilde{Q})$, an outcome contributes $\tilde{P}(a) - e^\varepsilon\tilde{Q}(a)$ only if its loss exceeds ε, and then at most $\tilde{P}(a)$. So $\delta(\varepsilon)$ is at most the $\tilde{P}$-probability that the loss exceeds ε.

10. **Hoeffding on A.** Under $\tilde{P}$ the distance above the middle is $t = (A - \frac{n_c}{2}) + (F - \frac{1}{2})$, which is at most $A - \frac{n_c}{2} + \frac{1}{2}$. Given $N_c = n_c$, the count A is a sum of n_c fair coins, so Hoeffding's inequality (Concentration) bounds the chance that it exceeds $n_c/2$ by more than s:

$$\Pr\left[A - \frac{n_c}{2} \geq s\right] \leq e^{-2s^2/n_c}. \tag{L.24}$$

    Choosing $s = \sqrt{n_c \ln(2/\delta)/2}$ makes this probability $\delta/2$.

11. **Chernoff on N_c.** The clone count has mean $\mu = (n-1)e^{-\varepsilon_0}$. The lower Chernoff bound of the same chapter says that it falls a fraction κ below its mean only rarely:

$$\Pr[N_c \leq (1-\kappa)\mu] \leq e^{-\kappa^2\mu/2}. \tag{L.25}$$

    Choosing $\kappa = \sqrt{2\ln(2/\delta)/\mu}$ makes this probability $\delta/2$, at the level $n_{c,\min} = (1-\kappa)\mu = \mu - \sqrt{2\mu\ln(2/\delta)}$.

12. **Combine the two tails.** By the union bound, the two failures of steps 10 and 11 together have $\tilde{P}$-probability at most δ. Outside them, $N_c > n_{c,\min}$ and, with n_c the value of N_c, $t \leq s + \frac{1}{2}$. Write the ratio Equation L.23 as $(1+u)/(1-u)$ with $u = b\,t/\frac{n_c+1}{2}$, which increases in u. The bound on t gives $u \leq h(n_c)$, where $h(n_c) = b(\sqrt{2n_c\ln(2/\delta)} + 1)/(n_c+1)$. Since the actual n_c is above $n_{c,\min}$, we want h to decrease, and it does for $n_c \geq 1$: its derivative has the sign of $\sqrt{2\ln(2/\delta)}\,\frac{1-n_c}{2\sqrt{n_c}} - 1$, which is negative there. So $u \leq h(n_{c,\min})$. The other direction, $E_{e^\varepsilon}(\tilde{Q}\|\tilde{P})$, is the same, because swapping the two counts and replacing A by $N_c - A$ turns $\tilde{P}$ into $\tilde{Q}$. So the pair, and with it the shuffled output, is (ε, δ)-DP with

$$\varepsilon = \ln\frac{1+x}{1-x}, \qquad x = b\,\frac{\sqrt{2n_{c,\min}\ln(2/\delta)} + 1}{n_{c,\min} + 1}, \qquad n_{c,\min} = \mu - \sqrt{2\mu\ln(2/\delta)}, \tag{L.26}$$

    provided $n_{c,\min} \geq 1$ and $x < 1$.

13. **The shape of Equation 35.7.** Suppose the expected number of clones is at least $8\ln(2/\delta)$. Then the Chernoff margin $\sqrt{2\mu\ln(2/\delta)}$ is at most $\mu/2$, so $n_{c,\min}$ is at least $\mu/2$. Bounding n_c+1 below by n_c and putting $n_c=\mu/2$ into the decreasing $h(n_c)$ gives

$$x \le \bar{x} = 2b\Big(\sqrt{\frac{\ln(2/\delta)}{\mu}} + \frac{1}{\mu}\Big). \tag{L.27}$$

If also $\bar{x} \le \frac{1}{2}$, then $1-\bar{x} \ge \frac{1}{2}$, so $\frac{1+\bar{x}}{1-\bar{x}} = 1 + \frac{2\bar{x}}{1-\bar{x}}$ is at most $1+4\bar{x}$. Substituting $1/\mu = e^{\varepsilon_0}/(n-1)$ gives

$$\varepsilon \le \ln\left(1 + \frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1}\left(\frac{8\sqrt{e^{\varepsilon_0}\ln(2/\delta)}}{\sqrt{n-1}} + \frac{8e^{\varepsilon_0}}{n-1}\right)\right). \tag{L.28}$$

This is Equation 35.7 with $\ln(2/\delta)$ in place of $\ln(4/\delta)$ and $n-1$ in place of n, under the two conditions just used. So the constant 8 of the paper is reproduced.

L.3.4. Checking it

The script checks steps 1 to 4 on 2,000 random randomizers that are 2-DP, each with four inputs and five reports. Q_0, Q_1 and every leftover are non-negative, and the identities of steps 2 and 3 hold to 1.1×10^{-16}. It then checks the whole reduction exactly on a small case: four users with three-valued randomized response at $\varepsilon_0=\ln 4$, the other three holding 2, 2 and 0. The exact law of the shuffled reports and the law produced by the map of step 6 from $(\tilde{P},\tilde{Q})$ agree to 8.3×10^{-17} on both D and D'.

Table L.5 compares the bounds at $\delta=10^{-6}$. The numeric column is the exact ε of the clone pair, and it reproduces Table 35.2.

Table L.5.: Central ε at $\delta=10^{-6}$ for n shuffled reports that are each ε_0-DP: the exact value for the clone pair, this section's closed form, its simplified form, and the closed form of Feldman, McMillan & Talwar. "n/a" marks where a condition fails.

n	ε_0	Clone pair, numeric	Equation L.26	Step 13, $\ln(1+4\bar{x})$	Equation 35.7
1,000	1	0.1824	0.3111	0.5565	0.5662
1,000	2	0.5455	1.0601	n/a	n/a
10,000	1	0.0530	0.0863	0.2096	0.2140
10,000	2	0.1550	0.2437	0.4919	0.5009
10,000	4	0.6009	1.0955	n/a	n/a
100,000	2	0.0450	0.0724	0.1823	0.1862
100,000	4	0.1698	0.2621	0.5251	0.5346

Equation L.26 is 1.54 to 1.94 times the numeric value, while the paper's closed form is 3.10 to 4.14 times it. The gain comes from keeping $\ln\frac{1+x}{1-x}$ and the exact $n_{c,\min}$ instead of simplifying them. The simplified form of step 13 is slightly below the paper's in every row, mainly because $\ln(2/\delta)$ is smaller than $\ln(4/\delta)$. All the closed forms are loose in another way too. At the ε of

Equation L.26, the pair's true δ is at most 3.84×10^{-11}, far below 10^{-6}, because each of steps 9 to 12 gives up some slack. Compute the numeric value to deploy.

Figure L.5 draws the two tails behind the bound for $n = 1000$ and $\varepsilon_0 = 2$. At the typical clone count, the loss stays below the closed-form ε across the whole Hoeffding window. The closed form must also cover clone counts as low as $n_{c,\min} = 72.6$, where the loss climbs faster, and that is where most of its slack comes from.

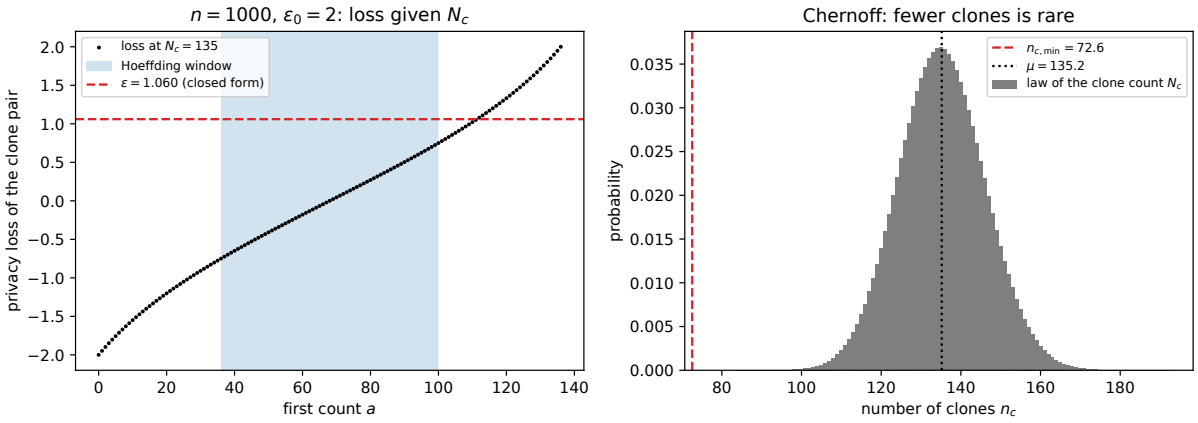


Figure L.5.: Left: the privacy loss of the clone pair against the first count a, given $N_c = 135$, the rounded mean, for $n = 1000$ and $\varepsilon_0 = 2$. The shaded band is the Hoeffding window of step 10 at $\delta = 10^{-6}$, and the dashed line is ε from Equation L.26. Right: the law of the clone count N_c, its mean μ and the Chernoff level $n_{c,\min}$ of step 11.

L.3.5. What it does not cover

The proof fixes one randomizer R that every user applies. Theorem 3.1 of the paper, as Theorem 26.2 states it, also allows each randomizer to be chosen adaptively from the earlier outputs. Then step 4 has to compare a user's randomizer with yours, and the paper needs a more careful argument, which stays cited. The numeric clone bound is exact for the pair, not for the mechanism. For binary randomized response the exact shuffled ε of Section 35.1 is smaller still.

L.4. Shift reduction and contraction

L.4.1. What the chapter uses

Amplification by iteration tracks two runs of a noisy iteration that use different records. To compare where they end up, it moves one run a little before comparing. To **couple** two random variables is to build them on one draw, so that each value of one is paired with a value of the other. The **shifted Rényi divergence** allows the first distribution to be moved by at most z in this way:

$$D_\alpha^{(z)}(X\|X') = \inf\{D_\alpha(Y\|X') : \text{some coupling of } X \text{ and } Y \text{ has } \|X - Y\| \le z \text{ almost surely}\}. \tag{L.29}$$

With $z = 0$ it is the ordinary D_α.

The iteration adds Gaussian noise at every step. Write $Z \sim \mathcal{N}(0, \sigma^2 I)$ for one step's noise, independent of everything else. The price of hiding a shift of length at most a under it is

$$R_\alpha(a) = \sup_{\|w\| \le a} D_\alpha(Z + w \,\|\, Z) = \frac{\alpha a^2}{2\sigma^2}, \tag{L.30}$$

the Gaussian cost Equation 35.11, which is the same in both orders. The two lemmas are Lemmas 20 and 21 of Feldman, Mironov, Talwar & Thakurta (2018). The first lets the noise pay off a shift $a \ge 0$. The second handles the gradient steps, which differ between the runs because they use different records. It lets two maps ψ and ψ' act on the runs, where ψ' is a **contraction**, $\|\psi'(u) - \psi'(v)\| \le \|u - v\|$, and the two maps differ by at most s at every point. In symbols,

$$D_\alpha^{(z)}(X + Z \,\|\, X' + Z) \le D_\alpha^{(z+a)}(X \| X') + R_\alpha(a), \qquad D_\alpha^{(z+s)}(\psi(X) \,\|\, \psi'(X')) \le D_\alpha^{(z)}(X \| X'). \tag{L.31}$$

These are Equation 35.12 and Equation 35.13 in Section 35.3. The chapter shows the first only for point masses, and says that the general proof uses a coupling and bounds a joint pair. It proves the second, Lemma 35.3, by the triangle inequality and the data-processing inequality. Theorem 35.3 and Theorem 35.4, and the plateau bound Equation 35.20 of Section 35.4, rest on both.

L.4.2. The idea

Contraction is the easy one. Move X to a nearby Y that is cheap to compare with X', and apply ψ' to Y. The maps can push the pair apart only by how much they differ, s. And applying the same map ψ' to Y and to X' cannot make them easier to tell apart.

For shift reduction, start from a coupling that moves each value of X by at most $z + a$. Split every move into two parts along the same direction. The **carried** part, of length at most z, is left for later, which is why the left side still allows a shift of z. The **paid** part, of length at most a, is absorbed into the noise. Absorbing it turns the noise Z into Z plus a random shift of length at most a. Such a noise is at most $R_\alpha(a)$ away from Z itself, and the chain rule adds that cost to the cost of Y.

Take X equal to 0 or 1 with probability $\frac{1}{2}$ each, and $X' = 1$. Moving every value of X to 1 makes it equal to X', so $D_\alpha^{(1)}(X \| X') = 0$. Now pay $a = 0.5$ of the shift with the noise. The value 0 needs a move of 1, split into 0.5 carried and 0.5 paid. The value 1 needs no move. The left panel of Figure L.6 draws this.

L.4.3. Derivation

Contraction.

1. **Pick a cheap nearby Y.** Fix $\xi > 0$. By the definition of the infimum, there is a Y and a coupling with $\|X - Y\| \le z$ and $D_\alpha(Y \| X') \le D_\alpha^{(z)}(X \| X') + \xi$.

2. **Carry the coupling through the maps.** Use the same coupling for $\psi(X)$ and $\psi'(Y)$. By the triangle inequality, $\|\psi(X) - \psi'(Y)\| \le \|\psi(X) - \psi'(X)\| + \|\psi'(X) - \psi'(Y)\| \le s + z$. The first term is at most s by the assumption on the two maps, and the second at most z because ψ' is a contraction.
3. **Apply the same map to both.** $D_\alpha(\psi'(Y)\|\psi'(X')) \le D_\alpha(Y\|X')$ by data processing: one map, applied to both, cannot raise a divergence.
4. **Conclude.** $\psi'(Y)$ is within $z + s$ of $\psi(X)$, so it is allowed in the infimum that defines $D_\alpha^{(z+s)}(\psi(X)\|\psi'(X'))$. That divergence is therefore at most $D_\alpha^{(z)}(X\|X') + \xi$, for every $\xi > 0$.

Only ψ' needs to be a contraction. The chapter assumes both maps are, which is more than enough.

Shift reduction.

1. **Pick a cheap nearby Y.** Fix $\xi > 0$ and take Y coupled to X with $\|X - Y\| \le z + a$ and $D_\alpha(Y\|X') \le D_\alpha^{(z+a)}(X\|X') + \xi$. Let the noise Z be independent of X, Y and X'.
2. **Split each move.** Let $W = X - Y$, and shorten it to length at most a:
$$W_2 = \min\Big(1, \frac{a}{\|W\|}\Big)W, \qquad W_1 = W - W_2, \tag{L.32}$$
with $W_2 = 0$ when $W = 0$. Both parts point along W. So $\|W_2\| \le a$ and $\|W_1\| = \max(\|W\| - a, 0) \le z$.
3. **Build the new nearby point.** Let $Y' = Y + W_2 + Z$. Then $(X+Z) - Y' = X - Y - W_2 = W_1$, so Y' is within z of $X + Z$, and
$$D_\alpha^{(z)}(X + Z \,\|\, X' + Z) \le D_\alpha(Y' \,\|\, X' + Z). \tag{L.33}$$
4. **Look at the pair before the sum.** Y' is the sum of Y and $W_2 + Z$, and $X' + Z$ is the sum of X' and Z. Adding the two coordinates is one map applied to both, so by data processing
$$D_\alpha(Y' \,\|\, X' + Z) \le D_\alpha((Y,\ W_2 + Z) \,\big\|\, (X',\ Z)). \tag{L.34}$$
5. **Use the chain rule.** The proof of Equation 29.11 in Rényi DP uses only one thing: each joint law is the law of a first coordinate times the law of the second given the first. It shows that the Rényi divergence of the joint pair is at most that of the first coordinates plus the largest divergence of the second coordinates given the first. Here the second coordinate of the right pair is Z, independent of X'. So
$$D_\alpha((Y,\ W_2 + Z) \,\big\|\, (X',\ Z)) \le D_\alpha(Y\|X') + \sup_y D_\alpha(\mathrm{Law}(W_2 + Z \mid Y = y) \,\big\|\, \mathcal{N}(0, \sigma^2 I)). \tag{L.35}$$
6. **Bound the noise with a random shift.** The noise is independent of Y and W_2. So given $Y = y$, the sum $W_2 + Z$ is a mixture of shifted Gaussians $\mathcal{N}(w, \sigma^2 I)$, one for each value w of the paid part, and each such w has length at most a. The moment of a Rényi divergence is an average of a convex function of the ratio,
$$e^{(\alpha-1)D_\alpha(P\|Q)} = \int Q\Big(\frac{P}{Q}\Big)^\alpha, \tag{L.36}$$
and $u \mapsto u^\alpha$ is convex for $\alpha > 1$, so the moment is convex in P for a fixed Q. The moment of a mixture is therefore at most the largest moment of its components. So the divergence in step 5 is at most the largest $D_\alpha(Z + w\|Z)$ over $\|w\| \le a$, which is $R_\alpha(a)$.
7. **Conclude.** Steps 3 to 6 give $D_\alpha^{(z)}(X + Z\|X' + Z) \le D_\alpha^{(z+a)}(X\|X') + \xi + R_\alpha(a)$, for every $\xi > 0$. This is Equation 35.12.

L.4.4. Checking it

In the example, $\sigma = 1$ and the total shift is 1, of which a is paid and $z = 1 - a$ carried. The cheap nearby point is $Y = 1$, with $D_\alpha(Y \| X') = 0$. The paid part is $W_2 = -a$ when $X = 0$ and 0 when $X = 1$, so the constructed $Y' = 1 + W_2 + Z$ is an equal mixture of $\mathcal{N}(1 - a, 1)$ and $\mathcal{N}(1, 1)$. It is compared with $X' + Z = \mathcal{N}(1, 1)$. Its order-2 divergence has a closed form. Write r for the likelihood ratio of $\mathcal{N}(1 - a, 1)$ to $\mathcal{N}(1, 1)$, which averages to 1 and has second moment e^{a^2} under $\mathcal{N}(1, 1)$. Then the second moment of the mixture's ratio is $\mathbb{E}[(\frac{1}{2} + \frac{1}{2}r)^2] = \frac{1}{4}(1 + 2 + e^{a^2})$, so

$$D_2(Y' \,\|\, X' + Z) = \ln\Big(1 + \tfrac{1}{4}(e^{a^2} - 1)\Big), \tag{L.37}$$

which is the subsampled Gaussian moment of Equation 33.19 at $\alpha = 2$, with $q = \frac{1}{2}$ and noise multiplier $1/a$. The lemma bounds it by $R_2(a) = a^2$. Table L.6 compares them, and the script confirms the closed form by numerical integration.

Table L.6.: The divergence of the constructed Y' in the example, against the price $R_2(a)$ that the shift-reduction lemma charges.

Paid shift a	Exact $D_2(Y' \| X' + Z)$	Bound $R_2(a)$
0.25	0.0160	0.0625
0.5	0.0686	0.2500
1	0.3574	1.0000

All the slack in this example comes from step 6. The paid shift is a only half the time, but the lemma charges the full $R_\alpha(a)$, as if it were paid every time. The lemma is tight when every value of X is moved by the full $z + a$, and then step 6 loses nothing. The script also checks the split of step 2 on 100,000 random moves in three dimensions, with $z = 0.6$ and $a = 0.4$. The largest carried part has length 0.6000 and the largest paid part 0.4000, as step 2 says.

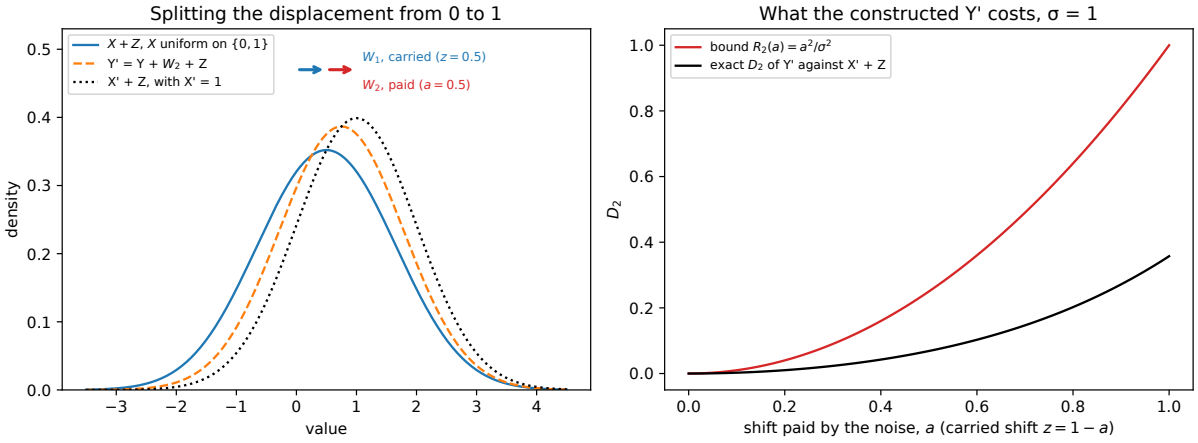


Figure L.6.: Left: the example with $\sigma = 1$ and $a = 0.5$. The densities of $X + Z$ (solid), of the constructed Y' (dashed) and of $X' + Z$ (dotted). The arrows split the move of the value 0 into the carried part W_1 and the paid part W_2. Right: the exact D_2 of Y' against $X' + Z$, and the lemma's price $R_2(a) = a^2$, as functions of the paid shift a.

L.5. The random stopping time

L.5.1. What the chapter uses

Projected noisy SGD makes one pass over n records, projecting each iterate back onto a convex set. The neighbouring relation is replace-one: the two datasets differ in the value of one record. Its losses are convex and β-smooth, the learning rate is at most $2/\beta$ so that each step is a contraction, and they are L-Lipschitz, which means each gradient has length at most L. It adds Gaussian noise of standard deviation σ to each gradient and releases only the last iterate. The chapter writes L_f for the Lipschitz constant, and this section writes L to keep formulas short. Theorem 23 in Section 35.3 gives the record used at step t of a run of T steps the curve

$$D_\alpha \le \frac{2\alpha L^2}{\sigma^2 (T - t + 1)}. \tag{L.38}$$

The last record is protected by only one step. A random stopping time fixes this. The chapter states Theorem 26 of Feldman, Mironov, Talwar & Thakurta (2018) without proof. Let the number of steps T be uniform on $\{1, \dots, n\}$ and independent of the data, and let the noise satisfy $\sigma \ge L\sqrt{2(\alpha - 1)\alpha}$. Then every record gets

$$\varepsilon(\alpha) = \frac{4\alpha L^2 \ln n}{n\sigma^2}. \tag{L.39}$$

This section derives a bound of this form. It reproduces the constant 4 for every $n \ge 37$.

L.5.2. The idea

The law of the output of the run that stops at a random time is an average over T of the laws of the runs that stop at T. Both datasets use the same weights $1/n$, because T does not depend on the data. Joint convexity then bounds the Rényi moment of the average by the average of the Rényi moments. A run that stops before your record is used leaks nothing. A run that stops k steps after your record is used leaks at most $2\alpha L^2/(\sigma^2 k)$, by Theorem 23. So for the first record the average leak is about $2\alpha L^2/\sigma^2$ times $\frac{1}{n}(1 + \frac{1}{2} + \cdots + \frac{1}{n})$, and that harmonic sum is about $\ln n$.

For $n = 1000$ and the first record, the average of $1/k$ over $k = 1, \dots, 1000$ is $H_{1000}/1000 = 0.0074855$, where $H_m = 1 + \frac{1}{2} + \cdots + \frac{1}{m}$ is the harmonic number. A run of fixed length gives the last record the full $1/1$.

L.5.3. Derivation

1. **The output is an average over the stopping time.** Fix the record at position t and the two datasets. Let P_T and Q_T be the laws of the iterate after T steps on D and on D'. The released output has law $P = \frac{1}{n}\sum_T P_T$ on D and $Q = \frac{1}{n}\sum_T Q_T$ on D'.

2. **Joint convexity of the moment.** For $\alpha > 1$ the moment of the Rényi divergence is an integral of $Q\, g(P/Q)$, with the convex function $g(u) = u^\alpha$. A function of the form $Q\, g(P/Q)$ is the **perspective** of g, and it is jointly convex in the pair (P, Q), as Appendix Section F.5 derives. The reason is Jensen's inequality. Take weights λ_i and pairs (P_i, Q_i), and let $\bar{P} = \sum_i \lambda_i P_i$ and $\bar{Q} = \sum_i \lambda_i Q_i$. Then $\sum_i \lambda_i Q_i\, g(P_i/Q_i)$ is $\bar{Q}$ times an average of the values $g(P_i/Q_i)$, with weights $\lambda_i Q_i/\bar{Q}$. That average is at least g of the averaged argument, which is $g(\bar{P}/\bar{Q})$. So
$$e^{(\alpha-1)D_\alpha(P\|Q)} \le \frac{1}{n}\sum_{T=1}^{n} e^{(\alpha-1)D_\alpha(P_T\|Q_T)}. \tag{L.40}$$

3. **The terms.** If $T < t$, your record has not been used, so $P_T = Q_T$ and the term is 1. If $T \ge t$, Theorem 23 for a run of T steps gives $D_\alpha(P_T\|Q_T) \le 2\alpha L^2/(\sigma^2 k)$, with $k = T - t + 1$. So the term is at most e^{x_k}, where
$$x_k = \frac{2\alpha(\alpha-1)L^2}{\sigma^2\, k}, \qquad k = 1, \dots, n-t+1. \tag{L.41}$$

4. **The noise condition keeps every exponent small.** The largest exponent is x_1, and $x_1 \le 1$ is exactly the condition $\sigma \ge L\sqrt{2(\alpha-1)\alpha}$.

5. **Replace each exponential by a line.** On $[0, 1]$ the convex function e^x lies below its chord from $(0, 1)$ to $(1, e)$, so $e^x \le 1 + (e-1)x$. Summing over k, the n terms add up to at most $n + (e-1)\frac{2\alpha(\alpha-1)L^2}{\sigma^2} H_{n-t+1}$. The harmonic number H_{n-t+1} is at most H_n, so the first record, $t = 1$, is the worst.

6. **Take logarithms.** Divide by n, use $\ln(1+u) \le u$, and divide by $\alpha - 1$:
$$D_\alpha(P\|Q) \le \frac{2(e-1)\,\alpha L^2\, H_n}{n\sigma^2}. \tag{L.42}$$

7. **Compare with the stated constant.** The stated bound is $4\alpha L^2 \ln n/(n\sigma^2)$. So it follows from Equation L.42 whenever the harmonic factor is small enough,
$$2(e-1)H_n \le 4\ln n. \tag{L.43}$$

   The script finds that this holds for every n from 37 to 10^6. Beyond 10^6 the bound $H_n \le 1 + \ln n$ is enough, because $2(e-1)(1+\ln n) \le 4\ln n$ once $n \ge 446$. For $n < 37$, though, the derivation gives $2(e-1)H_n$ in place of $4\ln n$. The chord matters here: with the cruder line $e^x \le 1 + 2x$, the constant would be $4H_n$, slightly above $4\ln n$.

L.5.4. Checking it

Take $n = 1000$, $\alpha = 2$, $L = 1$ and $\sigma = 2$, so $x_1 = 1.00$ and the noise condition holds with equality. At $\alpha = 2$, a run of fixed length n gives the first record the Rényi bound 0.0010 and the last record 1.0000. With the random stopping time, the average of step 2, before any approximation, gives at most 0.008527 to the first record and 0.001717 to the last. After the chord and the logarithm, Equation L.42 gives 0.012862 for every record, below the stated 0.013816. Figure L.7 shows the trade. A random stopping time gives up the excellent guarantee of the early records to protect the late ones.

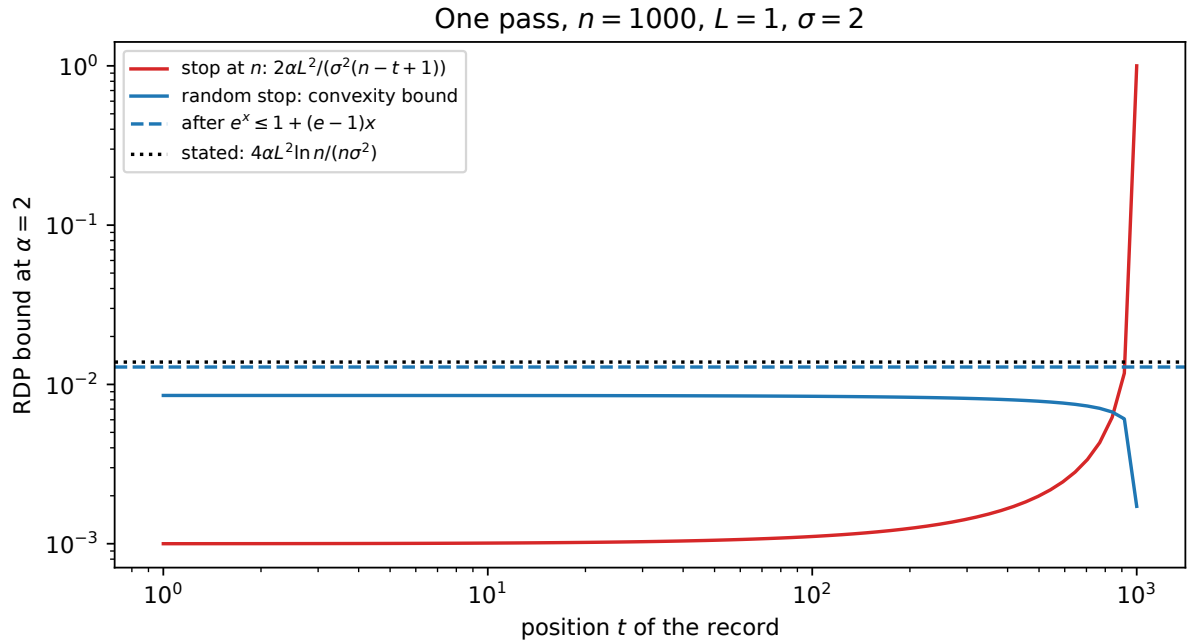


Figure L.7.: The RDP bound at $\alpha = 2$ for each record of a one-pass run with $n = 1000$, $L = 1$ and $\sigma = 2$, against the position t of the record. Red: the run always stops at n (Theorem 23). Blue: the random stopping time, from the convexity bound of step 2. Dashed: Equation L.42. Dotted: the stated bound $4\alpha L^2 \ln n/(n\sigma^2)$.

L.5.5. What it does not cover

The noise condition depends on the order: at $\alpha = 8$ it asks for $\sigma \ge L\sqrt{112}$. Larger orders need more noise, or a bound that does not use the chord. The result also needs one pass, a stopping time independent of the data, and the release of the final iterate only.

L.6. Code

The script `code/appendix-derivations/apx_amplification.py` computes every number on this page and draws its figures. It checks the randomized-response inequality on a grid and the Gaussian directions in two ways. It checks the two hockey-stick identities against closed forms, and domination on a small dataset. For the clones, it checks the decomposition on random randomizers, verifies the reduction exactly for four users, and computes the clone pair's exact ε next to the closed forms. It also evaluates the shift-reduction example and the random-stopping bounds. Run it with

```
uv run python code/appendix-derivations/apx_amplification.py
```

Table L.7 summarizes its output.

Table L.7.: The script's main results with the default seed.

Check	Result
Randomized-response gap on 5,294,646 grid points	no negative value
$D_\alpha(P_q\|P_0)$ and $D_\alpha(P_0\|P_q)$ at $\alpha = 32$, $q = 0.01$, $\sigma = 1$	11.246 and 1.6923×10^{-3}, two ways
Hockey-stick identities against closed forms, relative gap	7.4×10^{-14} (remove), 3.0×10^{-12} (add)
Three-record dataset, largest ratio of real to pair	0.9687
Clone decomposition on 2,000 random randomizers	no negative entry; identities to 1.1×10^{-16}
Exact reduction for four users	agreement to 8.3×10^{-17}
Clone closed form against numeric, $n = 10^4$, $\varepsilon_0 = 2$	0.2437 against 0.1550
Shift example at $a = 1$: exact D_2 against $R_2(a)$	0.3574 against 1.0000
Random stopping, $n = 1000$, $\alpha = 2$: Equation L.42 against the stated bound	0.012862 against 0.013816

Each derivation holds wherever the script can test it, and the closed forms are loose by the factors the sections report. For the clone bound, compute the pair numerically. For the stopping time, the stated constant is safe from $n = 37$ on.

L.7. References

- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (arXiv:1908.10530, 2019). The RDP of the subsampled Gaussian, its binomial series, and the dominance of the remove direction (Theorem 5) that Section L.1 proves by another route.
- Zhu, Dong & Wang, "Optimal Accounting of Differential Privacy via Characteristic Function" (AISTATS 2022). Dominating pairs (Definition 7), their composition (Theorem 10) and the pairs for Poisson subsampling (Theorem 11).
- Balle, Barthe & Gaboardi, "Privacy Amplification by Subsampling: Tight Analyses via Couplings and Divergences" (NeurIPS 2018). Advanced joint convexity and the mixture identities behind Section L.2.
- Feldman, McMillan & Talwar, "Hiding Among the Clones: A Simple and Nearly Optimal Analysis of Privacy Amplification by Shuffling" (FOCS 2021). The clone reduction (Theorem 3.2), the adaptive case and the closed form (Theorem 3.1).
- Feldman, Mironov, Talwar & Thakurta, "Privacy Amplification by Iteration" (FOCS 2018). The shifted Rényi divergence, Lemmas 20 and 21, Theorems 22 and 23, and the random stopping time (Theorem 26).
- van Erven & Harremoës, "Rényi Divergence and Kullback–Leibler Divergence" (IEEE Transactions on Information Theory, 2014). Convexity, data processing and the other properties of the Rényi divergence used throughout.

M. Private learning and tuning

This appendix derives five results that the chapters on private machine learning state but do not prove in full. All five are about learning under differential privacy (DP). Each one takes a guarantee for a small building block and follows it through a learning procedure. The block is one noisy gradient step, one noisy vote, one training run, one record of a user, or one step's noise. The chapters cite papers for these proofs, sketch them, or check them only numerically. Here each is derived step by step from facts the book has already met, and each is checked by a script. Table M.1 lists them.

Table M.1.: The derivations of this appendix and the chapter sections that state or use them.

Derivation	Where the chapter states or uses it
The best excess risk of private convex empirical risk minimization, from above and below, and exact bounds for output and objective perturbation	Section 38.5, Section 38.4, Section 38.1 and Section 38.2
The data-dependent Rényi bound for a noisy vote of teachers	Section 42.5
The Rényi bound for the best of a random number of training runs	Section 40.4
The worst case for a user with k records in private gradient descent with record-level sampling	Section 44.2
How one step of private gradient descent changes the loss of one group	Section 45.3

The page keeps one notation throughout, even where a source paper uses other letters. The model's parameters are θ, the Rényi order is always α, the learning rate is η and the sampling rate is q. The letter λ is a ridge coefficient in the first section and a curvature in the last, and μ is a ratio of sensitivity to noise. Each section names its letters again where they first appear.

M.1. The optimal rates of private convex learning

M.1.1. What the chapter uses

Private empirical risk minimization trains a model by **empirical risk minimization (ERM)**: it minimizes the average loss on n training records, plus a ridge penalty. For a dataset D the

objective is

$$\mathcal{L}(\theta; D) = \frac{1}{n}\sum_{i=1}^{n} \ell(\theta; z_i) + \frac{\lambda}{2}\|\theta\|^2. \tag{M.1}$$

Here θ is the vector of model parameters in d dimensions, z_i is the i-th record, and we write $\mathcal{L}(\theta)$ when the dataset is fixed. The loss ℓ is convex, and its gradient has length at most a constant L. The ridge coefficient is λ, and the minimizer of the objective $\mathcal{L}$ is θ^*.

A private algorithm releases parameters $\tilde{\theta}$, and its **excess risk** is $\mathcal{L}(\tilde{\theta}) - \mathcal{L}(\theta^*)$, how much worse they fit than the minimizer. Neighbouring datasets differ by replacing one record, and the guarantee is DP with privacy parameters ε and δ.

Section 38.5 states in Equation 38.27 that the best expected excess risk any (ε, δ)-DP algorithm can reach is of order

$$\frac{L^2 d}{\lambda n^2 \varepsilon^2}, \tag{M.2}$$

up to logarithms. Section 38.4 states the matching lower bound of order $RL\sqrt{d}/(n\varepsilon)$ for losses without a ridge, with weights in a ball of radius R. The chapter cites Bassily, Smith & Thakurta (2014) for both (Theorem 38.5). It also gives the excess risk of output perturbation, Equation 38.6, and of objective perturbation, Equation 38.16, only to second order in the noise.

This section proves three things. The first is an upper bound of the right order, with explicit constants, for noisy projected gradient descent. The second is a lower bound for every private algorithm, again with explicit constants, which proves the rate up to those constants. The third is a pair of exact two-sided bounds that replace the chapter's second-order formulas.

The upper bound uses the exact Gaussian calibration of the chapter. There, $\mu(\varepsilon, \delta)$ is the largest ratio of sensitivity to noise at which one Gaussian release is (ε, δ)-DP, given by Equation 38.2. The algorithm is noisy projected gradient descent, started from $\theta_1 = 0$ and kept in the ball of radius $R = L/\lambda$. It runs for T steps, and the learning rate at step t is $\eta_t = 2/(\lambda(t+1))$. Each gradient gets Gaussian noise of standard deviation $\sigma = (2L/n)\sqrt{T}/\mu(\varepsilon, \delta)$ per coordinate. The release $\tilde{\theta}$ is the average of the iterates in which iterate t has weight proportional to t. If the number of steps is $T \geq n^2\mu^2/d$, then

$$\mathbb{E}[\mathcal{L}(\tilde{\theta})] - \mathcal{L}(\theta^*) \leq \frac{16 L^2 d}{\lambda n^2 \mu(\varepsilon, \delta)^2} \leq \frac{32 L^2 d \ln(1.25/\delta)}{\lambda n^2 \varepsilon^2}. \tag{M.3}$$

The second form holds for $\varepsilon < 1$.

The lower bound uses a number K that measures how many records a private mean needs. The fingerprinting bound of Lower bounds says that estimating d means of ± 1 attributes privately to a constant accuracy needs a number of records proportional to

$$K = \frac{\sqrt{d}}{(e^{\varepsilon} - 1) + 4\delta\sqrt{d}}. \tag{M.4}$$

The denominator is how much one record can influence a correlation score under DP, and the numerator is the spread of that score. The proof below builds hard datasets with m real records out of n, for an integer m that it is free to choose. For every (ε, δ)-DP algorithm there is a dataset of n records, with linear losses whose gradients have length at most L, on which

$$\mathbb{E}[\mathcal{L}(\tilde{\theta})] - \mathcal{L}(\theta^*) \geq \frac{L^2}{2\lambda} \max_{1 \leq m \leq n} \left[\frac{m}{n}\left(\frac{1}{3} - \frac{m}{2K}\right)\right]^2 \geq \frac{L^2 K^2}{2592\, \lambda n^2} \quad \text{when } 3 \leq K \leq 3n. \tag{M.5}$$

The range for K makes the best choice of m, about $K/3$, a whole number between 1 and n. When δ is small enough that $4\delta\sqrt{d} \le e^{\varepsilon} - 1$, the number K is at least $\sqrt{d}/(2(e^{\varepsilon}-1))$. Then, still for $3 \le K \le 3n$, the bound is at least $L^2 d/(10368\,\lambda n^2 (e^{\varepsilon}-1)^2)$. For $\varepsilon \le 1$, $e^{\varepsilon}-1$ is between ε and 1.72ε, so this is of the order of Equation 38.27. Without a ridge, over the ball of radius R and with the same K, every private algorithm has excess risk at least

$$\max_{1 \le m \le n} \frac{mLR}{16n}\Big(\frac{2}{3} - \frac{m}{K}\Big)^2 \ge \frac{LRK}{729\,n} \quad \text{when } \tfrac{9}{2} \le K \le \tfrac{9}{2}n, \tag{M.6}$$

which is of order $RL\sqrt{d}/(n\varepsilon)$.

The exact bounds sandwich the excess risk between two multiples of the noise's average squared length. Write β_s for the largest curvature of $\mathcal{L}$, for example $\beta_s = \frac{1}{4} + \lambda$ for the chapter's logistic regression with features of length at most 1. Output perturbation adds noise Z to θ^*, and objective perturbation adds $\frac{1}{n}\langle b, \theta\rangle$ to the objective. Then

$$\frac{\lambda}{2}\mathbb{E}\|Z\|^2 \le \mathbb{E}\big[\mathcal{L}(\theta^* + Z) - \mathcal{L}(\theta^*)\big] \le \frac{\beta_s}{2}\mathbb{E}\|Z\|^2, \qquad \frac{\lambda\,\mathbb{E}\|b\|^2}{2\beta_s^2 n^2} \le \mathbb{E}\big[\mathcal{L}(\tilde{\theta}) - \mathcal{L}(\theta^*)\big] \le \frac{\mathbb{E}\|b\|^2}{2\lambda n^2}. \tag{M.7}$$

M.1.2. The idea

The upper bound treats privacy noise as one more source of gradient noise. Stochastic gradient descent (SGD) on a strongly convex objective converges at a rate that depends on the noise only through the average squared length G^2 of the noisy gradient. Privacy adds $d\sigma^2$ to G^2, one σ^2 per coordinate. The noise per step must grow like $\sqrt{T}$ to keep the total privacy fixed, so $d\sigma^2$ grows like T. But the convergence bound divides G^2 by T, so the number of steps cancels from the noise term. What is left is d times the noise of one full-budget release, which is the rate.

The lower bound turns a private trainer into a private mean estimator, which the fingerprinting argument says cannot be accurate with few records. Take the loss $-\langle\theta, x\rangle$ on records x, plus the ridge. Then the objective is a bowl centred on the mean of the records divided by λ, and the excess risk of any θ is $\frac{\lambda}{2}$ times its squared distance from that centre. A trainer with small excess risk therefore outputs a good estimate of the mean of the records.

The rate in n comes from padding with records equal to zero. Figure M.1 shows the move in two dimensions. Four real records sit at corners of a square, and four zero records sit at the origin. The mean of all eight is half the mean of the four real ones, so multiplying the trainer's output by $n/m = 2$ estimates the real mean. That multiplication also doubles the trainer's error. So a trainer whose error is e on n records gives an estimator whose error is $(n/m)e$ on m records. Fingerprinting says the estimator's error is at least a constant when m is about K or less, so the trainer's error is at least that constant times m/n, about K/n. Squaring a distance of order K/n gives an excess risk of order K^2/n^2.

The exact bounds hold because a strongly convex, smooth function lies between two parabolas around its minimizer. The excess risk of any point is at least $\frac{\lambda}{2}$ and at most $\frac{\beta_s}{2}$ times its squared distance from θ^*. The chapter's second-order formula sits between the two, with the true curvature in place of λ or β_s.

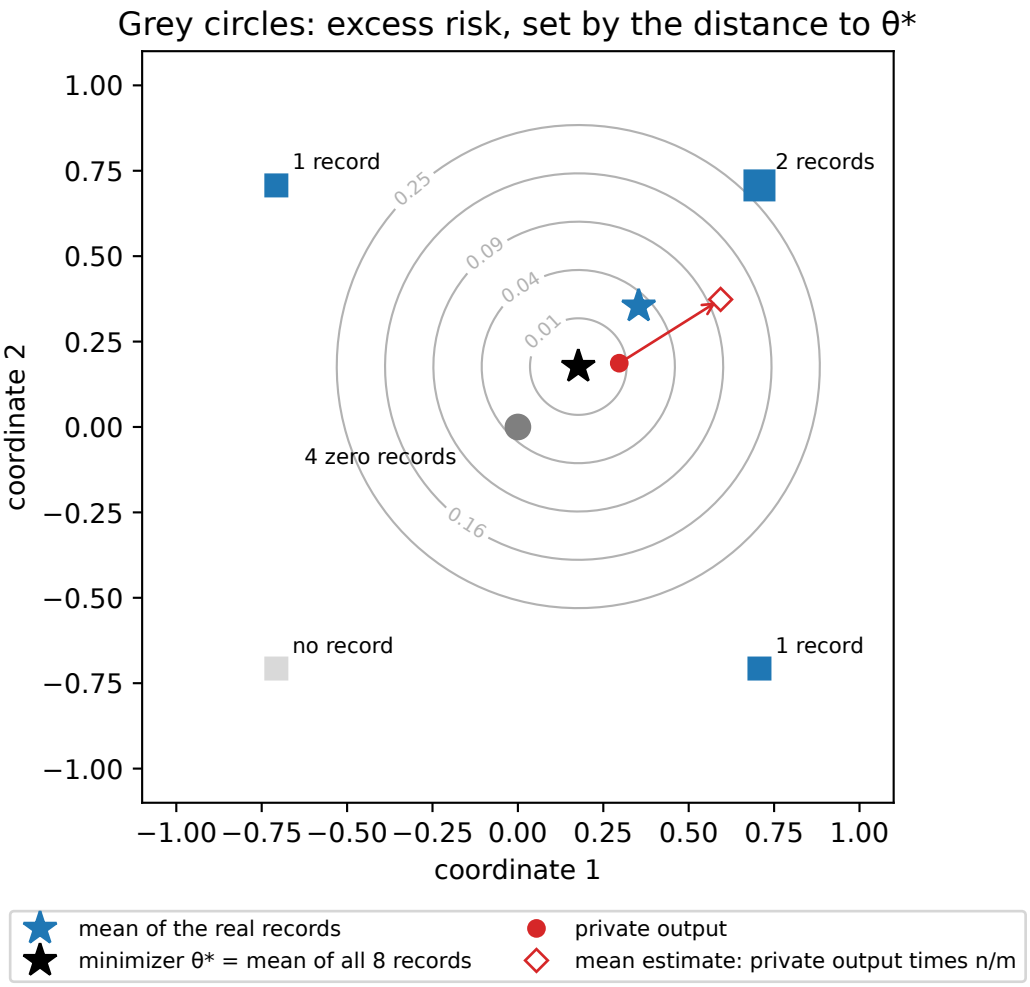


Figure M.1.: The reduction in two dimensions, with $L = \lambda = 1$. Four real records sit at corners of the square (one corner holds two), and four zero records at the origin. The minimizer θ^* of the objective is the mean of all eight, and the grey circles are levels of the excess risk. Multiplying a private output by $n/m = 2$ estimates the mean of the four real records, with twice the output's error.

M.1.3. Derivation

The upper bound. Three facts from earlier chapters do the work, and each is restated where it is used.

1. **Privacy.** Each step releases the noisy gradient $\nabla\mathcal{L}(\theta_t) + \xi_t$, where the noise ξ_t is fresh, normal, with mean zero and variance σ^2 in each coordinate. Replacing one record changes the average of the loss gradients by at most $2L/n$, because one gradient of length at most L leaves and another arrives. The ridge term $\lambda\theta_t$ is the same on both datasets. So each step is a Gaussian mechanism with ratio $2L/(n\sigma)$. By the composition of Gaussian mechanisms (f-DP and Gaussian differential privacy), used in step 2 of the proof of Theorem 38.3, T adaptive Gaussian steps are exactly as private as one Gaussian release with ratio $\sqrt{T} \cdot 2L/(n\sigma)$, and the choice of σ makes this $\mu(\varepsilon, \delta)$. The projection and the average use only released values, so they are post-processing.
2. **The ball contains the minimizer.** The gradient of $\mathcal{L}$ is zero at θ^*, so $\lambda\theta^* = -\frac{1}{n}\sum_i \nabla\ell(\theta^*; z_i)$. The right side is an average of vectors of length at most L, so $\|\theta^*\| \le L/\lambda = R$. The minimizer over the ball is therefore the overall minimizer.
3. **The noisy gradient's second moment.** On the ball, $\|\nabla\mathcal{L}(\theta)\| \le L + \lambda\|\theta\| \le 2L$. The noise has mean zero and is independent of θ_t, so the cross term vanishes when the square is expanded:
$$\mathbb{E}\|\nabla\mathcal{L}(\theta_t) + \xi_t\|^2 = \|\nabla\mathcal{L}(\theta_t)\|^2 + \mathbb{E}\|\xi_t\|^2 \le 4L^2 + d\sigma^2 = G^2. \tag{M.8}$$
4. **The bound for SGD on a strongly convex objective.** Optimization proves Equation 6.33. It is about a function F that is λ-strongly convex on a closed convex set containing its minimizer. The random step directions must be right on average, with average squared length at most G^2. Then projected steps with learning rates $2/(\lambda(t+1))$ and the average weighted by t satisfy
$$\mathbb{E}[F(\tilde{\theta}_T)] - F^* \le \frac{2G^2}{\lambda(T+1)}. \tag{M.9}$$
The proof follows the squared distance to the minimizer, and the noise enters only through G^2. The ridge makes $\mathcal{L}$ λ-strongly convex, and step 2 puts the minimizer in the ball, so the bound applies to $\mathcal{L}$.
5. **Substitute the noise.** The calibration gives $\sigma^2 = 4L^2T/(n^2\mu^2)$. So
$$\frac{2G^2}{\lambda(T+1)} = \frac{8L^2}{\lambda(T+1)} + \frac{8L^2d}{\lambda n^2\mu^2} \cdot \frac{T}{T+1} \le \frac{8L^2}{\lambda(T+1)} + \frac{8L^2d}{\lambda n^2\mu^2}. \tag{M.10}$$
The noise term no longer depends on T.
6. **Choose T.** When $T \ge n^2\mu^2/d$, the first term is at most the second, so the sum is at most $16L^2d/(\lambda n^2\mu^2)$. This is the first form of Equation M.3.
7. **In terms of ε.** For $\varepsilon < 1$ the classic Gaussian calibration is (ε, δ)-DP. It adds noise $\Delta\sqrt{2\ln(1.25/\delta)}/\varepsilon$ to a function whose sensitivity is Δ. The exact calibration uses the least noise that works, so its ratio is at least the classic one, and
$$\frac{1}{\mu^2} \le \frac{2\ln(1.25/\delta)}{\varepsilon^2}. \tag{M.11}$$
Substituting this doubles the constant to 32.

The other schedule of that section, learning rates $1/(\lambda t)$ with the uniform average, has the bound $G^2(1+\ln T)/(2\lambda T)$. The same substitution gives at most $4L^2 d(1+\ln T)/(\lambda n^2\mu^2)$ for $T \geq n^2\mu^2/d$. Since T grows like n^2, the factor $1 + \ln T$ grows like $\ln n$. This is the logarithm the chapter mentions, and the weighted average removes it.

The strongly convex lower bound. The plan is to build a family of hard datasets and an estimator, and then to apply the fingerprinting bound.

1. **A loss whose excess risk is a distance.** Take the linear loss $\ell(\theta; x) = -\langle \theta, x\rangle$ on records with $\|x\| \leq L$, so every loss gradient, $-x$, has length at most L. With the ridge, completing the square gives
$$\mathcal{L}(\theta) = \frac{\lambda}{2}\|\theta\|^2 - \langle \theta, \bar{x}\rangle = \frac{\lambda}{2}\Big\|\theta - \frac{\bar{x}}{\lambda}\Big\|^2 - \frac{\|\bar{x}\|^2}{2\lambda}, \tag{M.12}$$
where $\bar{x}$ is the mean of the records. So $\theta^* = \bar{x}/\lambda$, and the excess risk of any θ is exactly $\frac{\lambda}{2}\|\theta - \theta^*\|^2$.
2. **Hard datasets.** The fingerprinting bound uses random records $y_i \in \{-1, +1\}^d$ with random attribute means, as in Section 27.3, so the hard datasets are built from them. Fix an integer m between 1 and n. The dataset has m real records $x_i = (L/\sqrt{d})\, y_i$, each of length L, and $n - m$ records equal to zero. The zeros add nothing to the sum, so the mean of all n records is $\bar{x} = (m/n)(L/\sqrt{d})\, \bar{y}$, where $\bar{y}$ is the mean of the y_i.
3. **From a trainer to a mean estimator.** Given the m records y_i, build the dataset of step 2 and run the private trainer to get $\tilde{\theta}$. The estimator M rescales $\tilde{\theta}$ so that θ^* maps to $\bar{y}$, then clips each coordinate to $[-1, 1]$:
$$M = \mathrm{clip}_{[-1,1]^d}\Big(\frac{n\lambda\sqrt{d}}{mL}\,\tilde{\theta}\Big). \tag{M.13}$$
Replacing one y_i replaces one record of the padded dataset, so M is (ε, δ)-DP by post-processing. Clipping never moves a coordinate away from $\bar{y}$, because $\bar{y}$ lies in $[-1, 1]^d$. And by the Cauchy–Schwarz inequality the ℓ_1 norm is at most $\sqrt{d}$ times the Euclidean norm. So
$$\|M - \bar{y}\|_1 \leq \sqrt{d} \cdot \frac{n\lambda\sqrt{d}}{mL}\|\tilde{\theta} - \theta^*\| = \frac{dn\lambda}{mL}\|\tilde{\theta} - \theta^*\|. \tag{M.14}$$
4. **Average error.** Write E for the trainer's expected excess risk on these datasets. By step 1, $\mathbb{E}\|\tilde{\theta} - \theta^*\|^2 = 2E/\lambda$, and the average of a length is at most the square root of the average squared length. So the average error per attribute, $a = \mathbb{E}\|M - \bar{y}\|_1/d$, satisfies
$$a \leq \frac{n}{m} \cdot \frac{\sqrt{2\lambda E}}{L}. \tag{M.15}$$
5. **The fingerprinting bound.** Equation 27.30 says that an (ε, δ)-DP estimator of d attribute means from m records, with outputs in $[-1, 1]^d$ and average error a per attribute, needs
$$m \geq \Big(\frac{2}{3} - 2a\Big)K. \tag{M.16}$$
Its proof plays two facts against each other. An accurate estimate must correlate with the records by a total of order d. DP lets each record carry only about $(e^\varepsilon - 1)\sqrt{d} + 4\delta d$ of that total. Rearranging the inequality gives $a \geq \frac{1}{3} - \frac{m}{2K}$, and step 4 turns it into
$$\frac{\sqrt{2\lambda E}}{L} \geq \frac{m}{n}\Big(\frac{1}{3} - \frac{m}{2K}\Big). \tag{M.17}$$

6. **Choose m.** This holds for every m, so it holds for the best one. When the right side is negative it says nothing, so only values of m below $2K/3$ matter, where both sides are non-negative. Squaring and multiplying by $L^2/(2\lambda)$ gives the first form of Equation M.5. The bracket is largest near $m = K/3$. If $K \geq 3$, the integer $m = \lfloor K/3 \rfloor$ lies between $K/6$ and $K/3$, so $\frac{m}{n}(\frac{1}{3} - \frac{m}{2K}) \geq \frac{K}{6n} \cdot \frac{1}{6} = \frac{K}{36n}$. Squaring and multiplying by $L^2/(2\lambda)$ gives $L^2K^2/(2592\lambda n^2)$. The condition $K \leq 3n$ keeps this m at most n. The trainer's expected excess risk on the random datasets is at least this, so some dataset has at least this much.

The convex lower bound. Without a ridge in the objective, the same idea works if the ridge is moved into the loss and its size is chosen from m.

1. **The loss.** On the ball of radius R, take $\ell(\theta; x) = \frac{\lambda}{2}\|\theta\|^2 - \langle \theta, x \rangle$ with $\|x\| \leq L/2$ and $\lambda = mL/(2nR)$. Its gradient $\lambda\theta - x$ has length at most $\lambda R + L/2 \leq L$, because $m \leq n$. So it is a convex loss with gradients of length at most L on the ball.
2. **The datasets.** Use m real records $x_i = (L/(2\sqrt{d}))\, y_i$ and $n - m$ zero records. As before, $\mathcal{L}(\theta) = \frac{\lambda}{2}\|\theta - \bar{x}/\lambda\|^2$ plus a constant, and now $\bar{x}/\lambda = (R/\sqrt{d})\, \bar{y}$. That point has length at most R, so it is the minimizer over the ball, and the excess risk is again $\frac{\lambda}{2}\|\theta - \theta^*\|^2$.
3. **The estimator.** Output $M = \mathrm{clip}_{[-1,1]^d}((\sqrt{d}/R)\, \tilde{\theta})$. As in steps 3 and 4 above, $a \leq \sqrt{2E/\lambda}/R = \sqrt{4nE/(mLR)}$.
4. **Fingerprinting.** The condition $m \geq (\frac{2}{3} - 2a)K$ rearranges to $2a \geq \frac{2}{3} - \frac{m}{K}$, so

$$E \geq \frac{mLR}{16n}\Big(\frac{2}{3} - \frac{m}{K}\Big)^2, \tag{M.18}$$

   which is Equation M.6 before the choice of m.
5. **Choose m.** The right side is largest at $m = 2K/9$. If $K \geq 9/2$, the integer $m = \lfloor 2K/9 \rfloor$ lies between $K/9$ and $2K/9$, so $\frac{2}{3} - \frac{m}{K} \geq \frac{4}{9}$ and the bound is at least $\frac{LR}{16n} \cdot \frac{K}{9} \cdot \frac{16}{81} = \frac{LRK}{729n}$. The condition $K \leq \frac{9}{2}n$ keeps $m \leq n$.

The exact bounds. Two inequalities from Optimization give all four bounds. A β_s-smooth function lies below its tangent plus $\frac{\beta_s}{2}$ times the squared distance, by the descent lemma Equation 6.5. A λ-strongly convex function lies above its tangent plus $\frac{\lambda}{2}$ times the squared distance. Here β_s must bound the curvature of $\mathcal{L}$ everywhere, not only at θ^*. For the loss $g(y\langle \theta, x \rangle)$ with $0 \leq g'' \leq c$ and $\|x\| \leq 1$, the Hessian is at most $(c + \lambda)I$.

1. **Output perturbation.** At θ^* the gradient is zero, so the tangent is flat. The two inequalities give $\frac{\lambda}{2}\|Z\|^2 \leq \mathcal{L}(\theta^* + Z) - \mathcal{L}(\theta^*) \leq \frac{\beta_s}{2}\|Z\|^2$ for every noise vector Z. Averaging gives the first pair of Equation M.7. For Gaussian noise, $\mathbb{E}\|Z\|^2 = d\sigma^2$.
2. **Objective perturbation, from above.** The perturbed objective $\mathcal{L}_b(\theta) = \mathcal{L}(\theta) + \frac{1}{n}\langle b, \theta \rangle$ is λ-strongly convex and minimized at $\tilde{\theta}$. Strong convexity at its minimizer, where the gradient is zero, gives $\mathcal{L}_b(\theta^*) \geq \mathcal{L}_b(\tilde{\theta}) + \frac{\lambda}{2}\|\theta^* - \tilde{\theta}\|^2$. Writing out $\mathcal{L}_b$ and moving terms,

$$\mathcal{L}(\tilde{\theta}) - \mathcal{L}(\theta^*) \leq \frac{1}{n}\langle b, \theta^* - \tilde{\theta} \rangle - \frac{\lambda}{2}\|\tilde{\theta} - \theta^*\|^2 \leq \frac{\|b\|}{n}t - \frac{\lambda}{2}t^2, \qquad t = \|\tilde{\theta} - \theta^*\|, \tag{M.19}$$

   by the Cauchy–Schwarz inequality. The right side is a downward parabola in t whose largest value is $\|b\|^2/(2\lambda n^2)$. This holds for every draw of b, so averaging over b gives the upper bound of Equation M.7.

3. **Objective perturbation, from below.** The gradient of $\mathcal{L}_b$ is zero at its minimizer $\tilde{\theta}$, so the gradient of $\mathcal{L}$ is $-b/n$ there, and it is zero at θ^*. A β_s-smooth function's gradient changes by at most β_s per unit of distance, so $\|b\|/n \le \beta_s t$. Strong convexity of $\mathcal{L}$ at θ^* gives $\mathcal{L}(\tilde{\theta}) - \mathcal{L}(\theta^*) \ge \frac{\lambda}{2}t^2 \ge \frac{\lambda\|b\|^2}{2\beta_s^2 n^2}$. Averaging over b gives the lower bound.
4. **Compare with the chapter.** The second-order formulas average $\frac{1}{2}Z^\top HZ$ and $\frac{1}{2n^2}b^\top H^{-1}b$, where H is the Hessian at θ^*. Every eigenvalue of H lies between λ and β_s. So the exact upper bounds are the chapter's formulas with H replaced by its worst case, $\beta_s I$ for output perturbation and λI for objective perturbation.

Step 1 also shows what output perturbation achieves. The minimizer moves by at most $2L/(\lambda n)$ when one record is replaced (Fact 2 of the chapter), so the Gaussian noise has standard deviation $2L/(\lambda n\mu)$, and the upper bound is $2\beta_s dL^2/(\lambda^2 n^2\mu^2)$. This is the rate of Equation M.3 times the condition number β_s/λ, so output perturbation reaches the optimal rate only when β_s/λ is bounded by a constant. Objective perturbation with the chapter's pure-DP noise, whose density is proportional to $e^{-\varepsilon'\|b\|/2}$ with ε' the part of the budget left for the noise, has $\mathbb{E}\|b\|^2 = d(d+1)(2/\varepsilon')^2$, which grows like d^2, the price of $\delta = 0$.

M.1.4. Checking it

The script computes the bounds on the chapter's running example. It is logistic regression in $d = 10$ dimensions with ridge $\lambda = 0.01$ and $L = 1$. The budget is $\varepsilon = 1$ and $\delta = 10^{-5}$, so $\mu = 0.2681$. At $n = 10{,}000$ the rule $T = \lceil n^2\mu^2/d \rceil$ asks for 718,515 steps, with noise $\sigma = 0.6325$ on the mean gradient. The upper bound Equation M.3 is 2.2268×10^{-3}. The uniform average's bound is 8.0638×10^{-3}, larger by about $(1 + \ln T)/4$, the ratio of the two closed forms. That many full-batch steps are too slow for the script, so it runs the algorithm at smaller n, on two freshly drawn datasets each. With 300 records the measured excess risk is 2.2640×10^{-1}, and the bound is 10.9 times larger. The ratio grows to 16.5 at 1,000 records, where the measured value is 1.3489×10^{-2}, and to 23.0 at 3,000 records, where it is 1.0754×10^{-3}. It is loose for the reason the chapter gives: the logistic loss has more curvature than λ near its minimizer, which the bound does not use.

The lower bound is far below. The running example has $K = 1.8402$, which is below 3, so only the first form of Equation M.5 applies, at $m = 1$. At $n = 10{,}000$ it gives 1.8991×10^{-9}, a factor 1.17×10^6 below the upper bound. In high dimension the factor settles near 4.3×10^5: at $n = 10{,}000$ it is 4.27×10^5 for $d = 1{,}000$ and 4.28×10^5 for $d = 10{,}000$. The convex bound Equation M.6, with $R = 5$, is 8.6×10^3 times below the chapter's upper bound Equation 38.24 at both dimensions. These constants are as poor as those of the fingerprinting bound itself, which the lower-bounds chapter already calls loose. What the proof establishes is the shape: both bounds fall like $1/n^2$ and grow like d, as Figure M.2 shows.

The exact bounds bracket the measurements. At $n = 10{,}000$, output perturbation with Gaussian noise has the bounds 2.7835×10^{-4} and 7.2372×10^{-3} from Equation M.7. The chapter's second-order value is 8.7070×10^{-4}, and 20 releases average 9.6417×10^{-4}. Objective perturbation has the bounds 3.2872×10^{-7} and 2.2221×10^{-4}, the second-order value 7.1266×10^{-5}, and the measured 7.1178×10^{-5}. On 50 draws of the noise, every single release also obeys the per-draw inequalities of steps 2 and 3. The upper bounds are 7.5 and 3.1 times the measurements. The lower bounds are much weaker, because $\lambda = 0.01$ is far below the data's curvature. Figure M.3 shows why:

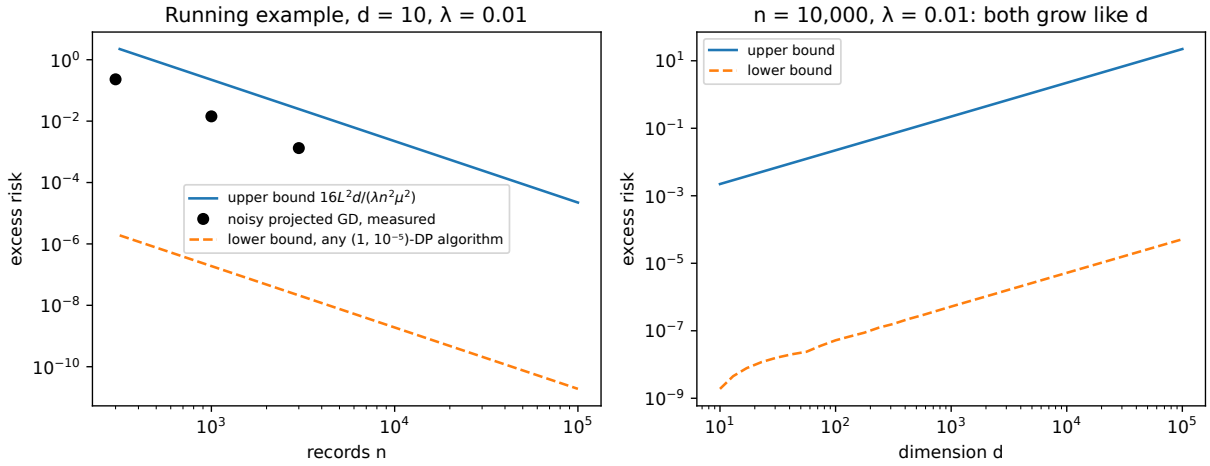


Figure M.2.: Left: excess risk against the number of records on the running example ($d = 10$, $\lambda = 0.01$, $(1, 10^{-5})$-DP), log–log. The upper bound Equation M.3 (solid), noisy projected gradient descent measured on two datasets per point (circles), and the lower bound Equation M.5 for every private algorithm (dashed). Right: the two bounds against the dimension at $n = 10{,}000$. Both fall like $1/n^2$ and grow like d; the constants are far apart.

along any direction, the objective lies between the two parabolas, and close to its second-order approximation.

M.1.5. What it does not cover

The lower bound loses large constants in three places: the fingerprinting bound itself, the factor $\sqrt{d}$ between the ℓ_1 and Euclidean errors, and the choice of m. It also loses a factor $\sqrt{\ln(1/\delta)}$, which Steinke & Ullman (2016) recover with a sharper distribution of the attribute means. It needs δ small compared with $1/\sqrt{d}$, so that K is of order $\sqrt{d}/(e^{\varepsilon} - 1)$, and it holds for replace neighbours. The upper bound uses full gradients, so it costs about $n^3\mu^2/d$ gradient evaluations. Bassily, Smith & Thakurta (2014) use one random record per step and subsampling, which is much cheaper and adds logarithmic factors; that analysis stays cited.

M.2. The data-dependent bound for a noisy vote

M.2.1. What the chapter uses

Private prediction answers a query with GNMax: each of many teachers votes for a label, Gaussian noise of standard deviation σ is added to each label's count, and the label with the highest noisy count is released. This is the vote of **private aggregation of teacher ensembles (PATE)**. GNMax satisfies **Rényi differential privacy (RDP)** with cost α/σ^2 at every order α. When the teachers agree, the answer is almost certain, and Section 42.5 charges it less with the bound Equation 42.10 of Papernot et al. (2018, Theorem 6).

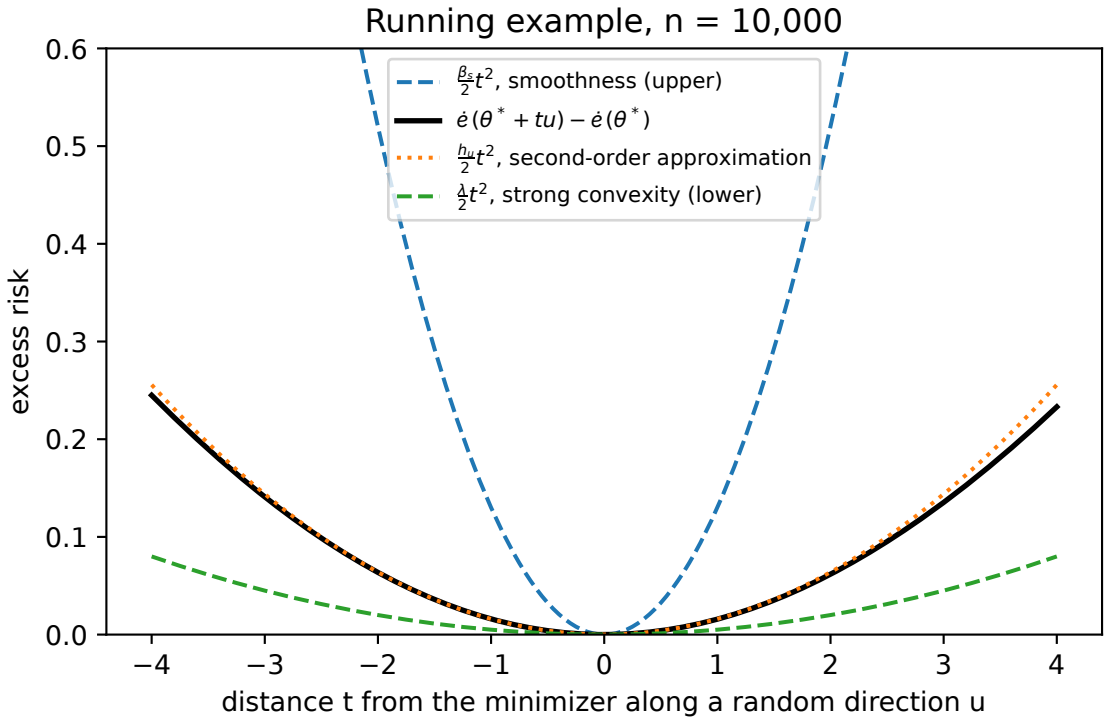


Figure M.3.: The running example's objective at $n = 10{,}000$ along a random direction u through its minimizer (black), between the parabolas of strong convexity, $\frac{\lambda}{2}t^2$, and smoothness, $\frac{\beta_s}{2}t^2$ (dashed). The second-order approximation $\frac{h_u}{2}t^2$, with h_u the curvature along u, is dotted.

Write $P = M(D)$ and $P' = M(D')$ for the distributions of the answer on the dataset and on a neighbour. Let i^* be the likely answer, and let $\tilde{\beta}$ be a known bound on the probability $\beta = P(\text{not } i^*)$ that the answer is something else, a failure probability (Papernot et al. write it q). The bound uses two auxiliary orders, $\alpha_2 = \sigma\sqrt{\ln(1/\tilde{\beta})}$ and $\alpha_1 = \alpha_2 + 1$, with the RDP costs of GNMax at those orders, $\varepsilon_1 = \alpha_1/\sigma^2$ and $\varepsilon_2 = \alpha_2/\sigma^2$. Theorem 6 requires $\alpha \le \alpha_1$, $\alpha_2 > 1$, and the condition

$$\tilde{\beta} \le \frac{e^{(\alpha_2-1)\varepsilon_2}}{\left(\frac{\alpha_1}{\alpha_1-1}\cdot\frac{\alpha_2}{\alpha_2-1}\right)^{\alpha_2}}. \tag{M.20}$$

Under these, the order-α divergence of the answer on the dataset from the answer on any neighbour satisfies

$$D_\alpha(P\|P') \le \frac{1}{\alpha-1}\ln\Big[(1-\tilde{\beta})A^{\alpha-1} + \tilde{\beta}B^{\alpha-1}\Big], \quad A = \frac{1-\tilde{\beta}}{1-(\tilde{\beta}e^{\varepsilon_2})^{(\alpha_2-1)/\alpha_2}}, \quad B = \frac{e^{\varepsilon_1}}{\tilde{\beta}^{1/(\alpha_1-1)}}. \tag{M.21}$$

Here D_α is the Rényi divergence of order α, defined in Equation 42.8 by

$$e^{(\alpha-1)D_\alpha(P\|P')} = \sum_y P(y)^\alpha P'(y)^{1-\alpha}. \tag{M.22}$$

The chapter derives the simpler pure-DP form Equation 42.9 and says of the Gaussian form that "its proof is in Papernot et al. (2018, Appendix A)". In step 4 of the proof of Theorem 42.3, it also uses that the pure-DP bound increases in β, which step 7 below proves.

This section proves the Gaussian bound with the true β in place of $\tilde{\beta}$. It then shows that the largest value over all $\beta \le \tilde{\beta}$ is a valid bound with no condition at all, and checks that this largest value is the value at $\tilde{\beta}$ in the chapter's setting. It also proves that the pure-DP bound always increases in β, which closes the gap in step 4.

M.2.2. The idea

Split the Rényi sum into the likely answer and all the others. On the likely answer, the two distributions are both close to 1, so the term is close to 1. It can only be large if the neighbour puts much more probability off i^* than P does. A Rényi guarantee at a high order α_2, applied to the single event "not i^*", forbids that. On the other answers, P has total mass only β. Their likelihood ratios could be large, but a Rényi guarantee at order $\alpha_1 \ge \alpha$ controls their average, and the average of a smaller power is controlled by the average of a larger one.

A small example shows the split. Take three answers with $P = (0.99, 0.008, 0.002)$ and $P' = (0.985, 0.01, 0.005)$, and order $\alpha = 2$. The Rényi sum is $0.99^2/0.985 + 0.008^2/0.01 + 0.002^2/0.005 = 0.99503 + 0.0064 + 0.0008 = 1.00223$. The first term is near $1 - \beta = 0.99$, inflated a little because P' has slightly less mass on i^*. The other two are $\beta = 0.01$ times an average ratio below 1. The bound treats the two parts separately and replaces each by its worst case.

M.2.3. Derivation

The orders α_1 and α_2 are fixed numbers in steps 1 to 4, chosen before β enters. The bound holds for any order $\alpha > 1$ and any orders with $\alpha \le \alpha_1$, $\alpha_2 > 1$ and $\beta e^{\varepsilon_2} < 1$. The last one keeps the denominator of A positive.

1. **Split the sum.** Write $\beta' = P'(\text{not } i^*)$. Then

$$e^{(\alpha-1)D_\alpha(P\|P')} = (1-\beta)^\alpha(1-\beta')^{1-\alpha} + \sum_{y\neq i^*} P(y)^\alpha P'(y)^{1-\alpha}. \tag{M.23}$$

2. **The neighbour cannot move much mass off i^*.** GNMax is $(\alpha_2, \varepsilon_2)$-RDP for every pair of neighbours, in either order, because the neighbour relation is symmetric. So $D_{\alpha_2}(P'\|P) \le \varepsilon_2$. Post-processing never increases a Rényi divergence, and reducing an answer to "is it i^*?" is post-processing. The reduced distributions are $(1-\beta', \beta')$ and $(1-\beta, \beta)$, and their Rényi sum is at least its "not i^*" term:

$$\beta'^{\alpha_2}\beta^{1-\alpha_2} \le e^{(\alpha_2-1)\varepsilon_2}, \qquad \text{so} \qquad \beta' \le (\beta e^{\varepsilon_2})^{(\alpha_2-1)/\alpha_2}. \tag{M.24}$$

3. **The likely answer's term.** The exponent $1-\alpha$ is negative, so a smaller base gives a larger power. Step 2 bounds $1-\beta'$ from below, so

$$(1-\beta)^\alpha(1-\beta')^{1-\alpha} \le (1-\beta)^\alpha\big(1-(\beta e^{\varepsilon_2})^{(\alpha_2-1)/\alpha_2}\big)^{1-\alpha} = (1-\beta)A^{\alpha-1}, \tag{M.25}$$

   with A as in the statement, β in place of $\tilde\beta$.

4. **The other answers.** Write their sum as β times an average under P given "not i^*". With the likelihood ratio $r(y) = P(y)/P'(y)$,

$$\sum_{y\neq i^*} P(y)^\alpha P'(y)^{1-\alpha} = \beta \sum_{y\neq i^*} \frac{P(y)}{\beta}\, r(y)^{\alpha-1}. \tag{M.26}$$

   Now set $s = (\alpha-1)/(\alpha_1-1)$, which lies in $(0,1]$ because $\alpha \le \alpha_1$. Then $r^{\alpha-1} = (r^{\alpha_1-1})^s$. The function $t \mapsto t^s$ is concave, so by Jensen's inequality the average of X^s is at most (average of $X)^s$. For example, with $s = \frac{1}{2}$, the average of $\sqrt{1}$ and $\sqrt{9}$ is 2, below $\sqrt{5}$. The average of r^{α_1-1} given "not i^*" is $\frac{1}{\beta}\sum_{y\neq i^*} P(y)^{\alpha_1}P'(y)^{1-\alpha_1}$. That sum is part of the full Rényi sum of order α_1, which RDP bounds by $e^{(\alpha_1-1)\varepsilon_1}$. So

$$\sum_{y\neq i^*} P(y)^\alpha P'(y)^{1-\alpha} \le \beta\Big(\frac{e^{(\alpha_1-1)\varepsilon_1}}{\beta}\Big)^s = \beta\Big(\frac{e^{\varepsilon_1}}{\beta^{1/(\alpha_1-1)}}\Big)^{\alpha-1} = \beta B^{\alpha-1}. \tag{M.27}$$

5. **Collect.** Adding steps 3 and 4 and taking $\frac{1}{\alpha-1}\ln$ gives the bound with the true β:

$$D_\alpha(P\|P') \le V(\beta) = \frac{1}{\alpha-1}\ln\Big[(1-\beta)A(\beta)^{\alpha-1} + \beta B(\beta)^{\alpha-1}\Big]. \tag{M.28}$$

6. **From β to $\tilde\beta$.** Only the bound $\tilde\beta \ge \beta$ is known, so take the largest value of V over all β in $(0, \tilde\beta]$. That is a valid bound, with no condition beyond those listed before step 1. When V increases in β, the largest value is $V(\tilde\beta)$, which is Equation 42.10. The condition of Theorem 6 is what Papernot et al. use to prove that V increases. The script checks it directly instead: at the chapter's $\sigma = 40$ and $\alpha = 8$, V increases on $(0, \tilde\beta]$ at every vote gap (the difference between the top two counts) where the condition holds.

7. **The pure-DP form increases in β.** For a mechanism that is pure DP with parameter ε_0, write $\kappa = e^{\varepsilon_0}$ for its largest ratio of probabilities. The same split gives the bracket of Equation 42.9,

$$f(\beta) = (1-\beta)^{\alpha}(1-\kappa\beta)^{1-\alpha} + \beta\,\kappa^{\alpha-1}, \tag{M.29}$$

on $0 \le \beta < 1/\kappa$. Its first term has the form $y\,\psi(x/y)$, with the power function $\psi(t) = t^{\alpha}$, $x = 1-\beta$ and $y = 1-\kappa\beta$. That is the **perspective** of the convex function ψ, and a perspective is jointly convex in (x, y) (Section F.5).

The reason is short. A convex function is the largest of its tangent lines, so $\psi(t) = \max(at+c)$ over the slopes a and intercepts c of its tangents. Then $y\psi(x/y)$ is the largest of the linear functions $ax + cy$, and a maximum of linear functions is convex. Here x and y are linear in β, so the first term is convex in β, and the second term is linear. So f is convex, and its slope only grows. At $\beta = 0$ the slope is

$$f'(0) = \kappa^{\alpha-1} + (\alpha-1)\kappa - \alpha, \tag{M.30}$$

which is 0 at $\kappa = 1$ and grows with κ. So $f' \ge 0$ on the whole range, and the bound is largest at $\beta = \tilde{\beta}$, in every case.

8. **Why $\alpha_2 = \sigma\sqrt{\ln(1/\tilde{\beta})}$.** The bound is small when A is close to 1, that is when step 2's bound on β' is close to $\tilde{\beta}$. With $\varepsilon_2 = \alpha_2/\sigma^2$, the logarithm of that bound is

$$\Big(1 - \frac{1}{\alpha_2}\Big)\Big(\ln\tilde{\beta} + \frac{\alpha_2}{\sigma^2}\Big) = \ln\tilde{\beta} + \frac{\alpha_2}{\sigma^2} + \frac{\ln(1/\tilde{\beta})}{\alpha_2} - \frac{1}{\sigma^2}. \tag{M.31}$$

The middle two terms trade a larger order against a larger cost, and their sum is smallest at $\alpha_2 = \sigma\sqrt{\ln(1/\tilde{\beta})}$, where it equals $2\sqrt{\ln(1/\tilde{\beta})}/\sigma$. So this choice makes the neighbour's failure probability as close to $\tilde{\beta}$ as the argument allows: at most $\tilde{\beta}\,e^{2\sqrt{\ln(1/\tilde{\beta})}/\sigma - 1/\sigma^2}$. The choice $\alpha_1 = \alpha_2 + 1$ is Papernot et al.'s, and any $\alpha_1 \ge \alpha$ gives a valid bound.

M.2.4. Checking it

Take the chapter's example: two labels, $\sigma = 40$, order $\alpha = 8$ and a gap of 150 votes. On two labels the difference of the two noises is normal with variance $2\sigma^2$, so the answer is wrong with probability $\tilde{\beta} = \beta = \frac{1}{2}\operatorname{erfc}(150/80) = 4.0050 \times 10^{-3}$. The orders are $\alpha_2 = 93.981$ and $\alpha_1 = 94.981$. Their costs are $\varepsilon_2 = 0.05874$ and $\varepsilon_1 = 0.05936$. Step 2 allows the neighbour a failure probability of at most 4.5014×10^{-3}. That is 1.1240 times $\tilde{\beta}$, which is the factor $e^{2\sqrt{\ln(1/\tilde{\beta})}/\sigma - 1/\sigma^2}$ of step 8. The likely answer's term is 0.999477 and the other answers' term is 9.1545×10^{-3}. The bound is 1.2278×10^{-3}, the value the chapter quotes, and the largest value over $\beta \le \tilde{\beta}$ is the same. One teacher that changes its vote moves the gap by 2, so the neighbours have gaps of 148 and 152. Against the worse of the two, the exact divergence in this direction is 2.2362×10^{-4}, 5.49 times smaller.

Over gaps from 0 to 300 in steps of 5, the condition holds from a gap of 20 on, and at all 57 such gaps V increases on $(0, \tilde{\beta}]$ and the exact two-label divergence lies below the bound. At $\sigma = 40$ the condition allows any $\tilde{\beta}$ up to 0.36321. With three labels and votes $(200, 40, 10)$, the union bound of Equation 42.6 gives $\tilde{\beta} = 2.7303 \times 10^{-3}$ and the bound 8.7544×10^{-4}. The exact worst divergence over the six ways one teacher can move its vote is 1.5188×10^{-4}, and the data-independent cost

is 5.0000×10^{-3}. The script also checks step 7 numerically: on a grid of orders and values of ε_0, the pure-DP bracket is convex and increasing in β.

Figure M.4 splits the bound into its two parts. Below a gap of 116 the bound exceeds α/σ^2 and is not used. Above it, the other answers' term is the larger one: the cost of an answer is mostly the chance that the noise overturns the vote, weighted by how much an overturned vote reveals. Figure M.5 shows the two facts behind step 6. With the orders fixed, V rises steadily up to $\tilde{\beta}$. And the largest value over $\beta \le \tilde{\beta}$, as a function of $\tilde{\beta}$, falls far below the data-independent cost as the vote becomes certain.

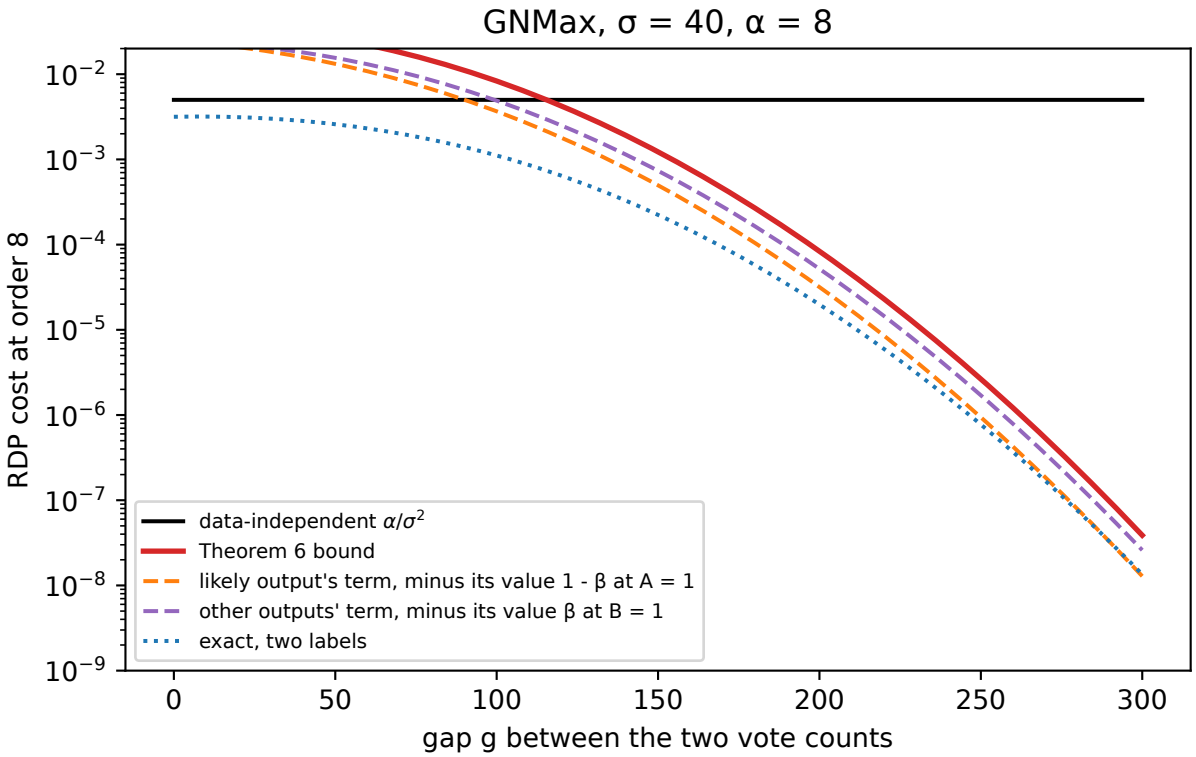


Figure M.4.: The data-dependent cost of one GNMax answer at order 8 with $\sigma = 40$, against the gap between the two vote counts, log scale. The bound (red) is the logarithm of 1 plus the two dashed parts, divided by $\alpha - 1$: the likely answer's term minus $1 - \tilde{\beta}$, and the other answers' term minus $\tilde{\beta}$. Black: the data-independent cost α/σ^2. Dotted: the exact cost on two labels.

M.2.5. What it does not cover

The proof that the condition of Theorem 6 makes V increasing stays cited to Papernot et al. (2018, Appendix A). Taking the largest value over $\beta \le \tilde{\beta}$ is a valid replacement that needs no condition, and it costs only a one-dimensional search. The union bound $\tilde{\beta}$ of Equation 42.6 for many labels is the paper's Proposition 7 and is not derived here. The bound is for one direction, $D_\alpha(M(D)\|M(D'))$ with D the actual dataset, which is the direction the chapter's accounting uses.

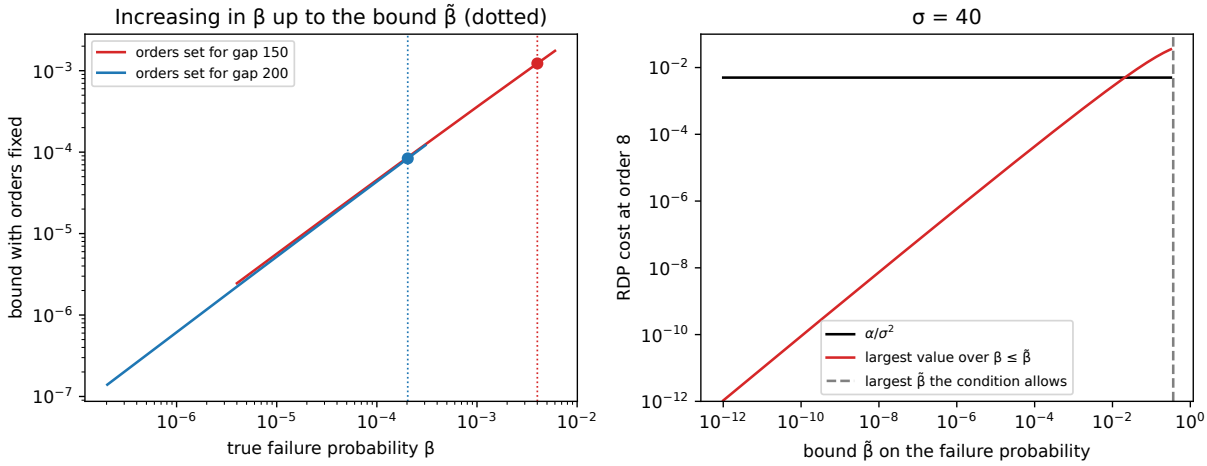


Figure M.5.: Left: the bound Equation M.28 as a function of the true failure probability β, with the orders set for gaps of 150 and 200 votes; it increases up to $\tilde{\beta}$ (dotted). Right: the largest value over $\beta \le \tilde{\beta}$ against $\tilde{\beta}$, at order 8 and $\sigma = 40$, with the data-independent cost and the largest $\tilde{\beta}$ the condition of Theorem 6 allows.

M.3. The best of a random number of runs

M.3.1. What the chapter uses

Hyperparameter tuning releases the best of K independent training runs, with K drawn at random before the search. Each run is an algorithm Q with RDP curve $\varepsilon(\alpha)$ at order α. The number of runs K follows a **truncated negative binomial** distribution with parameters $\nu > -1$ and $\omega \in (0, 1)$ (Papernot & Steinke write η for ν, γ for ω and λ for the order), whose generating function is $f(x) = \mathbb{E}[x^K]$. Its mean is $\mathbb{E}[K]$, and the case $\nu = 0$ is the logarithmic distribution. Section 40.4 states Papernot & Steinke's (2022) Theorem 2 as Equation 40.21: for every $\hat{\alpha} \ge 1$, the best-of release M satisfies

$$\varepsilon_M(\alpha) \le \varepsilon(\alpha) + (1+\nu)\Big(1 - \frac{1}{\hat{\alpha}}\Big)\varepsilon(\hat{\alpha}) + \frac{(1+\nu)\ln(1/\omega)}{\hat{\alpha}} + \frac{\ln \mathbb{E}[K]}{\alpha - 1}. \tag{M.32}$$

The chapter derives the generic bound Equation 40.14. For neighbours with run distributions Q, Q',

$$D_\alpha(M(D)\|M(D')) \le D_\alpha(Q\|Q') + \frac{1}{\alpha - 1}\ln\Big[f'(p)^\alpha f'(p')^{1-\alpha}\Big], \tag{M.33}$$

where p and p' are the probabilities that one and the same randomized test accepts a run's output, on D and on D'. The chapter takes outputs to be finite and $\Pr[K = 0] = 0$. For the truncated negative binomial case, Theorem 40.4 says that the proof follows the Poisson one, with the rejection probabilities bounded through the run's RDP at a second order $\hat{\alpha}$, and that "Papernot & Steinke (2022, Appendix B.2) carry out the algebra". The chapter also quotes their Corollary 3, $(2+\nu)\varepsilon$-DP for an ε-DP run, and Corollary 4 for zero-concentrated DP runs. This section carries out that algebra and derives both corollaries.

M.3.2. The idea

For this distribution, $f'(x)$ is a power of one simple quantity, $a = \omega + (1-\omega)(1-x)$. Here $1-x$ is the probability that the test rejects, and ω is a floor that comes from the distribution of K, not from the data. So the worst-case term of the generic bound is a power of $\omega a'^{\alpha-1}/a^{\alpha}$, with a' the same quantity on D'. That ratio is large only if a' is much larger than a.

Two facts stop that. The run's RDP guarantee says that the neighbour cannot reject much more often than D does: in Rényi terms the rejection probability on D' is at most a power of that on D. And a is never below ω, so dividing by it costs at most a power of $1/\omega$. The floor ω is the "few runs" explanation of the chapter: it caps how much the event "every run was rejected" can say about the data.

M.3.3. Derivation

The steps continue the chapter's proof of Theorem 40.2, whose steps 1 to 5 give Equation 40.14. Write $r = 1-p$ and $r' = 1-p'$ for the probabilities that the test rejects a run's output, on D and D'.

1. **The generating function.** The weights of the distribution are $\Pr[K=k] = (1-\omega)^k\binom{k+\nu-1}{k}/(\omega^{-\nu}-1)$ for $k \geq 1$, because $\prod_{j=0}^{k-1}\frac{j+\nu}{j+1} = \binom{k+\nu-1}{k}$. The negative binomial series $\sum_{k\geq 0}\binom{k+\nu-1}{k}z^k = (1-z)^{-\nu}$, with the $k=0$ term removed, gives

$$f(x) = \frac{(1-(1-\omega)x)^{-\nu}-1}{\omega^{-\nu}-1}, \qquad f'(x) = \mathbb{E}[K]\,\omega^{\nu+1}\,a(x)^{-(\nu+1)}, \quad a(x) = 1-(1-\omega)x. \tag{M.34}$$

   The mean is the slope at 1, as in the chapter:

$$\mathbb{E}[K] = f'(1) = \frac{\nu(1-\omega)}{\omega(1-\omega^{\nu})}. \tag{M.35}$$

   Since $a(1) = \omega$, the formula for f' is consistent at $x=1$. For $\nu = 0$ the same formulas hold in the limit, where f becomes $\ln(1-(1-\omega)x)/\ln\omega$.
2. **The worst-case term.** Write $a = a(p) = \omega + (1-\omega)r$ and $a' = a(p') = \omega + (1-\omega)r'$. The constant $\mathbb{E}[K]\omega^{\nu+1}$ appears with total power $\alpha + (1-\alpha) = 1$, so

$$f'(p)^{\alpha}f'(p')^{1-\alpha} = \mathbb{E}[K]\omega^{\nu+1}(a^{-\alpha}a'^{\alpha-1})^{\nu+1} = \mathbb{E}[K]\Big(\frac{\omega\,a'^{\alpha-1}}{a^{\alpha}}\Big)^{1+\nu}. \tag{M.36}$$

3. **The neighbour cannot reject much more often.** The run is $(\hat\alpha,\hat\varepsilon)$-RDP with $\hat\varepsilon = \varepsilon(\hat\alpha)$, for both orders of every pair of neighbours. The test's verdict, reject or accept, is post-processing of a run's output (Theorem 29.3), so $D_{\hat\alpha}$ between the verdict's distributions on D' and on D is at most $\hat\varepsilon$. Their Rényi sum is at least its "reject" term, so

$$r'^{\hat\alpha}r^{1-\hat\alpha} \leq e^{(\hat\alpha-1)\hat\varepsilon}, \qquad \text{so} \qquad r' \leq e^{s\hat\varepsilon}r^{s}, \quad s = 1-\frac{1}{\hat\alpha}. \tag{M.37}$$

4. **Carry it to a'.** Since $e^{s\hat\varepsilon} \geq 1$, step 3 gives

$$a' \leq \omega + (1-\omega)e^{s\hat\varepsilon}r^{s} \leq e^{s\hat\varepsilon}\big(\omega\cdot 1^{s} + (1-\omega)r^{s}\big) \leq e^{s\hat\varepsilon}\big(\omega+(1-\omega)r\big)^{s} = e^{s\hat\varepsilon}a^{s}. \tag{M.38}$$

The last inequality is Jensen's inequality for the concave function $t \mapsto t^s$, with weights ω and $1-\omega$ at the points 1 and r. For example, with $s = \frac{1}{2}$, $\omega = \frac{1}{2}$ and $r = 0$, the middle bracket is $\frac{1}{2}$ and the right side is $\sqrt{1/2} = 0.71$.

5. **Bound the ratio.** Substituting step 4,

$$\frac{\omega\, a'^{\alpha-1}}{a^{\alpha}} \le e^{(\alpha-1)s\hat{\varepsilon}}\, \omega\, a^{s(\alpha-1)-\alpha}. \tag{M.39}$$

The exponent of a is $s(\alpha-1)-\alpha = -1-(\alpha-1)/\hat{\alpha}$, which is negative. So the power is largest where a is smallest, and $a \ge \omega$. Replacing a by ω gives

$$\frac{\omega\, a'^{\alpha-1}}{a^{\alpha}} \le e^{(\alpha-1)(1-1/\hat{\alpha})\hat{\varepsilon}}\, \omega^{-(\alpha-1)/\hat{\alpha}}. \tag{M.40}$$

6. **Assemble.** Raise step 5 to the power $1+\nu > 0$, which keeps the inequality, and multiply by $\mathbb{E}[K]$ as in step 2. Taking $\frac{1}{\alpha-1}\ln$ gives

$$\frac{1}{\alpha-1}\ln\Big[f'(p)^{\alpha} f'(p')^{1-\alpha}\Big] \le (1+\nu)\Big(1-\frac{1}{\hat{\alpha}}\Big)\varepsilon(\hat{\alpha}) + \frac{(1+\nu)\ln(1/\omega)}{\hat{\alpha}} + \frac{\ln \mathbb{E}[K]}{\alpha-1}. \tag{M.41}$$

Adding $D_{\alpha}(Q\|Q') \le \varepsilon(\alpha)$ in Equation 40.14 gives Equation 40.21. At $\hat{\alpha} = 1$ step 3 says nothing, but then $s = 0$ and the trivial bound $a' \le 1 = a^0$ gives the same formula, with the term $(1+\nu)\ln(1/\omega)$.

7. **Corollary 3, pure DP.** An ε-DP run has $\varepsilon(\alpha) \le \varepsilon$ at every order, so the first term is at most ε. Let $\hat{\alpha} \to \infty$: the second term tends to $(1+\nu)\varepsilon$ and the third to 0. Then let $\alpha \to \infty$: the last term tends to 0, and the order-∞ divergence is the largest log ratio of probabilities. So the release is $(2+\nu)\varepsilon$-DP.

8. **Corollary 4, zero-concentrated DP.** A run that satisfies zero-concentrated DP with parameter ρ, a number that measures its privacy loss, has the RDP curve $\varepsilon(\alpha) = \rho\alpha$. Then $(1-1/\hat{\alpha})\varepsilon(\hat{\alpha}) = \rho(\hat{\alpha}-1)$, and the terms with $\hat{\alpha}$ are $(1+\nu)(\rho\hat{\alpha} + \ln(1/\omega)/\hat{\alpha}) - (1+\nu)\rho$. For positive numbers u and v, the sum $u\hat{\alpha} + v/\hat{\alpha}$ is smallest at $\hat{\alpha} = \sqrt{v/u}$, where it equals $2\sqrt{uv}$. Here the best second order is

$$\hat{\alpha} = \sqrt{\frac{\ln(1/\omega)}{\rho}}, \tag{M.42}$$

which is at least 1 when $\rho \le \ln(1/\omega)$. So

$$\varepsilon_M(\alpha) \le \rho\alpha + \frac{\ln \mathbb{E}[K]}{\alpha-1} + 2(1+\nu)\sqrt{\rho \ln(1/\omega)} - (1+\nu)\rho. \tag{M.43}$$

By the same rule, applied to $\rho(\alpha-1) + \ln \mathbb{E}[K]/(\alpha-1)$, the first two terms are smallest at the order

$$\alpha^* = 1 + \sqrt{\frac{\ln \mathbb{E}[K]}{\rho}}, \tag{M.44}$$

where they equal $\rho + 2\sqrt{\rho \ln \mathbb{E}[K]}$. Adding up gives the closed form of Papernot & Steinke's Corollary 4, which the chapter mentions, $2\sqrt{\rho \ln \mathbb{E}[K]} + 2(1+\nu)\sqrt{\rho \ln(1/\omega)} - \nu\rho$, at α^*. The Rényi divergence grows with the order, so the bound at α^* also holds at every order $\alpha \le \alpha^*$.

M.3.4. Checking it

The chapter checks Equation 40.21 on randomized response with $\varepsilon = 0.5$ and a logarithmic number of runs with mean 10, for which $\omega = 0.02692$. At order $\alpha = 5$ its terms are 0.3842 for one run, 0.5756 for $\ln \mathbb{E}[K]/(\alpha - 1)$, and, at $\hat{\alpha} = 100.95$, 0.4904 and 0.0358 for the two middle terms. Their sum is 1.4860, against an exact divergence of 0.3912. Figure M.6 draws the four terms against $\hat{\alpha}$.

The figure shows why $\hat{\alpha} = 100.95$ is not the best choice. Randomized response is pure DP, so as $\hat{\alpha}$ grows the second term rises to $(1 + \nu)\varepsilon = 0.5$ and the third falls to 0. The sum keeps falling, to its limit 1.4599, and that limit is the value in the chapter's table. A grid of orders that stops at $\hat{\alpha} = 100.95$ still gives a valid bound, just slightly above the best one. The same happens at the table's other orders: the limits, which the table gives, are 3.0299 at order 2 and 1.0962 at order 20, against 3.0561 and 1.1224 at the end of the grid.

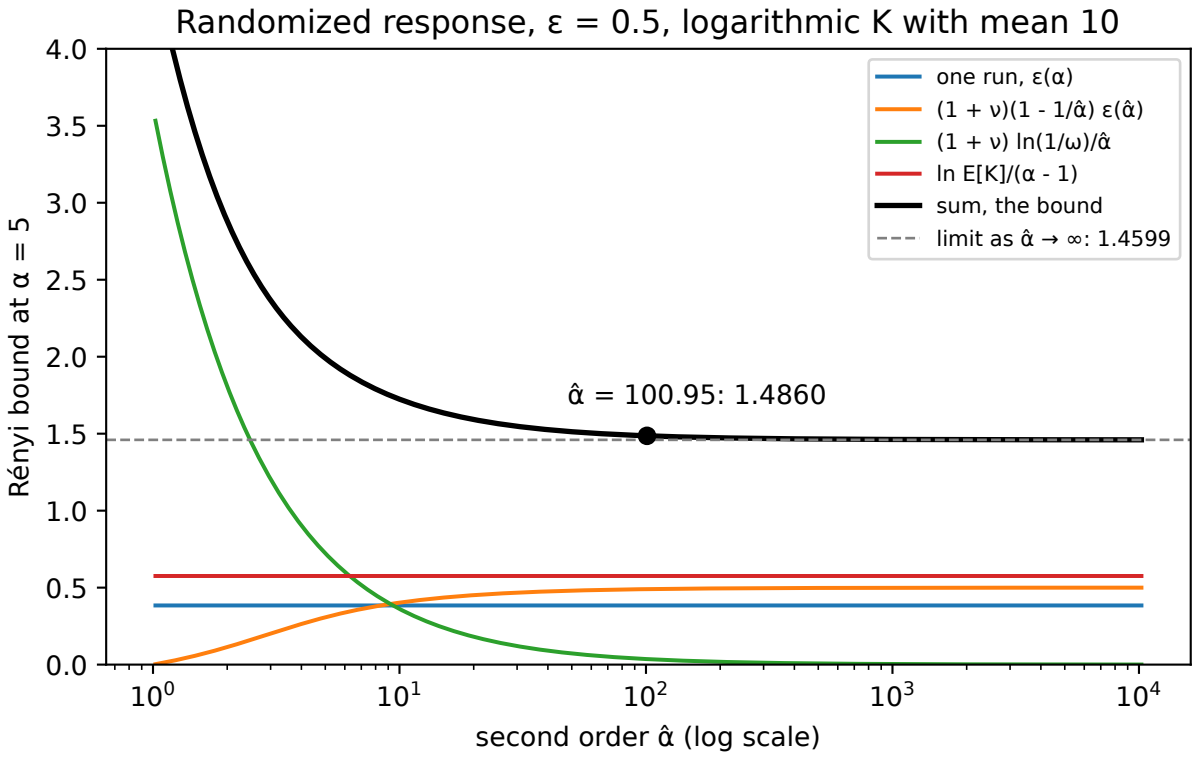


Figure M.6.: The four terms of Equation 40.21 at order $\alpha = 5$ against the second order $\hat{\alpha}$, for randomized response with $\varepsilon = 0.5$ and a logarithmic number of runs with mean 10. The sum (black) keeps falling toward its limit (dashed); the dot marks the end of the chapter's grid of orders.

The script also checks every step of the chain on 20,000 random cases of ω, α, $\hat{\alpha}$, $\hat{\varepsilon}$ and rejection probabilities that obey step 3, and none fails. For Corollary 4 with $\rho = 0.1$ and a geometric number of runs ($\nu = 1$) with mean 10, so $\omega = 0.1$, the closed form gives 2.7791. Minimizing the theorem over $\hat{\alpha}$ numerically at $\alpha^* = 5.7985$ gives the same, with the best $\hat{\alpha} = 4.7985$.

Figure M.7 shows where the slack is. The shaded region is every pair of rejection probabilities that order-4 RDP allows, and the red curve is the looser constraint that step 3 keeps. On the

shaded region the ratio $\omega a'^{\alpha-1}/a^{\alpha}$ is at most 7.67, while step 5 bounds it by 89.63. Most of the gap comes from replacing a by ω, which is exact only when the test rejects nothing on D.

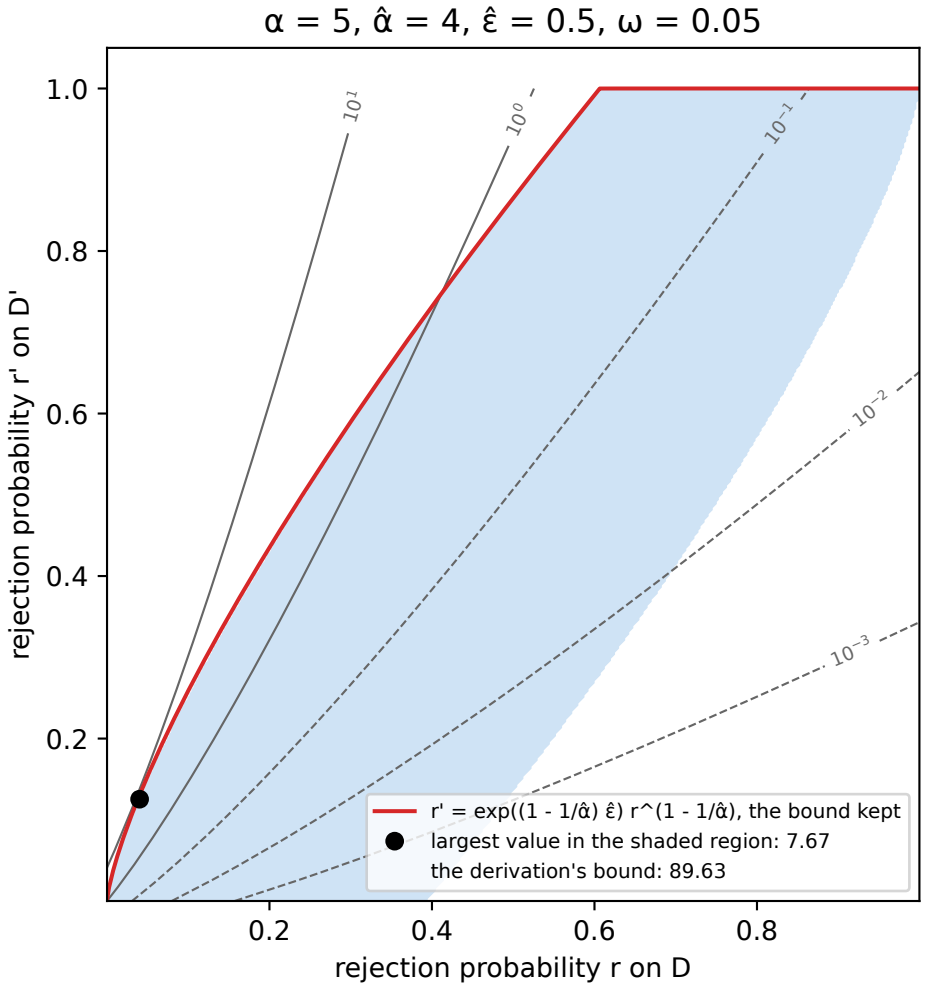


Figure M.7.: The unit square of rejection probabilities (r, r') on D and D'. Shaded: the pairs that order-$\hat{\alpha}$ RDP with $\hat{\alpha} = 4$ and $\hat{\varepsilon} = 0.5$ allows. Red: the bound $r' \le e^{(1-1/\hat{\alpha})\hat{\varepsilon}} r^{1-1/\hat{\alpha}}$ of step 3. Grey: levels of $\omega a'^{\alpha-1}/a^{\alpha}$ at $\alpha = 5$, $\omega = 0.05$, with its largest value on the shaded region marked.

M.3.5. What it does not cover

The derivation keeps the chapter's simplification to finite outputs and $\Pr[K = 0] = 0$. Papernot & Steinke (2022) handle the general case, where an empty search releases a fixed output, and that stays cited.

M.4. A user with many records in record-level DP-SGD

M.4.1. What the chapter uses

User-level differential privacy asks what DP-SGD, the private SGD of the earlier chapters, guarantees to a user who owns k records. DP-SGD samples each record into a batch independently with probability q, clips each record's gradient to length at most C, and adds Gaussian noise of standard deviation σC to the sum. In units of C, Section 44.2 compares the output without the user, $P_0 = \mathcal{N}(0, \sigma^2)$, with the output with the user, the mixture Equation 44.5:

$$P_{(k)} = \sum_{m=0}^{k} b_m \, \mathcal{N}(m, \sigma^2), \qquad b_m = \binom{k}{m} q^m (1-q)^{k-m}. \tag{M.45}$$

Its **Rényi moment** at integer order α is the average $A_\alpha = \mathbb{E}_{y\sim P_0}[(P_{(k)}(y)/P_0(y))^\alpha]$, so that $D_\alpha(P_{(k)} \| P_0) = \ln A_\alpha/(\alpha-1)$. The chapter computes it as Equation 44.7:

$$A_\alpha = \mathbb{E}\Big[\exp\Big(\frac{1}{\sigma^2} \sum_{1\le i<j\le\alpha} N_i N_j\Big)\Big], \tag{M.46}$$

where $N_1, \dots, N_\alpha$ are independent copies of the number $N \sim \mathrm{Bin}(k, q)$ of the user's records in a batch. The chapter derives A_α for this pair, in which every drawn record pushes the sum by the full C in the same direction. But it takes from Charles et al. (2024, Theorem 3) that this aligned case is the worst over all configurations of the user's gradients, and gives only the intuition.

This section proves that claim for the divergence $D_\alpha(P\|P_0)$ of the output with the user from the output without, at integer orders. The user's gradients may point in any directions in any number of dimensions. The Rényi moment is never larger than Equation 44.7, and it equals it when the gradients are aligned.

M.4.2. The idea

The Rényi moment of order α is an average over α independent draws of the user's subset of records. Each draw shifts the output by the sum v of the drawn gradients. The moment turns out to be the average of the exponential of the sum of inner products $\langle v_i, v_j\rangle$ between different draws. An inner product is at most the product of the lengths, and a sum of m gradients of length at most 1 has length at most m. So each inner product is at most the product of the two counts, which is exactly what Equation 44.7 has. Gradients that point the same way reach that maximum.

A user with two records and orthogonal gradients e_1 and e_2 shows the difference. Table M.2 lists the inner product of two independent draws for each pair of subsets, against the aligned case where both gradients are e_1.

Table M.2.: The inner product $\langle v_1, v_2 \rangle$ of the summed gradients of two independent draws of a two-record user's subset, for orthogonal and for aligned unit gradients. Draws that include no record give 0 in both cases.

Draw 1	Draw 2	Orthogonal gradients	Aligned gradients
$\{1\}$	$\{1\}$	1	1
$\{1\}$	$\{2\}$	0	1
$\{1\}$	$\{1,2\}$	1	2
$\{1,2\}$	$\{1,2\}$	2	4

Every entry of the orthogonal column is at most the aligned one, and the exponential of a larger number is larger. So the aligned moment is larger.

M.4.3. Derivation

1. **One step, with the past fixed.** By adaptive composition, it is enough to bound one step for every possible history. Given the history, the model is fixed, so the user's clipped gradients $g_1, \dots, g_k$ are fixed vectors in $\mathbb{R}^d$ with $\|g_a\| \le 1$ in units of C. The user's drawn subset S contains each record independently with probability q, and the user adds $v_S = \sum_{a \in S} g_a$ to the sum.
2. **The other records do not matter.** The other records' contribution U is random but has the same distribution with and without the user, and it is independent of S. So both outputs are mixtures over U of a pair shifted by U. The quantity $e^{(\alpha-1)D_\alpha(P\|Q)} = \sum_y Q(y)\big(P(y)/Q(y)\big)^\alpha$ is a perspective of $t \mapsto t^\alpha$, so it is jointly convex in the pair (Section F.5). Its value at a mixture of pairs is therefore at most the average of its values at the pairs. A shift by U changes nothing, so it suffices to take $U = 0$: compare $P_0 = \mathcal{N}(0, \sigma^2 I)$ with $P = \sum_S \pi(S)\,\mathcal{N}(v_S, \sigma^2 I)$, where $\pi(S) = q^{|S|}(1-q)^{k-|S|}$.
3. **The likelihood ratio.** Two normal densities with the same covariance $\sigma^2 I$ differ by $\|y\|^2 - \|y - v\|^2 = 2\langle v, y\rangle - \|v\|^2$ in the exponent, so

$$\frac{P(y)}{P_0(y)} = \sum_S \pi(S) \exp\Big(\frac{2\langle v_S, y\rangle - \|v_S\|^2}{2\sigma^2}\Big). \tag{M.47}$$

   This is the chapter's Equation 44.8, with vectors in place of numbers.
4. **The α-th power is an average over independent copies.** As in step 4 of the chapter, multiplying out α copies of the sum gives one term for every choice $(S_1, \dots, S_\alpha)$, with weight $\pi(S_1)\cdots\pi(S_\alpha)$. That is an average over independent subsets. With $u = v_{S_1} + \cdots + v_{S_\alpha}$,

$$\Big(\frac{P(y)}{P_0(y)}\Big)^\alpha = \mathbb{E}\Big[\exp\Big(\frac{2\langle u, y\rangle - \sum_i \|v_{S_i}\|^2}{2\sigma^2}\Big)\Big]. \tag{M.48}$$

5. **Average over y with the Gaussian moment generating function.** Fact 3 of the chapter gives the **moment generating function (MGF)** of a normal variable Y with mean 0 and variance σ^2. For every real s,

$$\mathbb{E}[e^{sY}] = e^{s^2\sigma^2/2}. \tag{M.49}$$

Under P_0 the coordinates of y are independent normal variables of this kind. So $\mathbb{E}[e^{\langle u,y\rangle/\sigma^2}]$ is a product of one-dimensional MGFs, one per coordinate j, with $s = u_j/\sigma^2$:

$$\mathbb{E}_{y\sim P_0}\big[e^{\langle u,y\rangle/\sigma^2}\big] = \prod_j e^{u_j^2/(2\sigma^2)} = e^{\|u\|^2/(2\sigma^2)}. \tag{M.50}$$

Everything is positive, so the two averages can be exchanged, and

$$A_\alpha = \mathbb{E}\Big[\exp\Big(\frac{\|u\|^2 - \sum_i \|v_{S_i}\|^2}{2\sigma^2}\Big)\Big] = \mathbb{E}\Big[\exp\Big(\frac{1}{\sigma^2}\sum_{i<j}\langle v_{S_i}, v_{S_j}\rangle\Big)\Big], \tag{M.51}$$

because $\|u\|^2 = \sum_i \|v_{S_i}\|^2 + 2\sum_{i<j}\langle v_{S_i}, v_{S_j}\rangle$ and the squares cancel.

6. **Bound each inner product.** By the Cauchy–Schwarz inequality, $\langle v_{S_i}, v_{S_j}\rangle \le \|v_{S_i}\|\,\|v_{S_j}\|$. By the triangle inequality, $\|v_S\| \le \sum_{a\in S}\|g_a\| \le |S|$. So $\langle v_{S_i}, v_{S_j}\rangle \le |S_i|\,|S_j|$.
7. **The aligned value.** The exponential is increasing and every weight is positive, so step 6 can be applied inside the average of Equation M.51. The counts $|S_1|, \dots, |S_\alpha|$ are independent $\mathrm{Bin}(k, q)$ variables, so the bound is Equation 44.7. Equality holds when every g_a is the same unit vector e, because then $v_S = |S|e$ and both inequalities of step 6 are equalities.
8. **Compose.** RDP curves add over steps, even when each step depends on the earlier outputs. So after T steps the user's divergence of this direction is at most $\frac{T}{\alpha-1}\ln A_\alpha$, with A_α from Equation 44.7.

M.4.4. Checking it

The script computes Equation M.51 exactly, by summing over all subsets, for $k = 5$, $q = 0.05$ and $\sigma = 1$. When the five gradients are aligned, $A_2 = 1.232011$, so the order-2 divergence is 0.208648, the value of Equation 44.7. When every pair of gradients meets at 60 degrees, $A_2 = 1.066665$, and at 90 degrees $A_2 = 1.021664$. Figure M.8 draws the divergence against the common angle, at orders 2 and 3. It is largest at angle 0, where it equals the aligned value. On 200 random configurations of four gradients of random length in three dimensions, no Rényi moment of order 2 or 3 exceeds the aligned one.

The identity of step 5 is checked against a direct integral. For three unit gradients in the plane at angles 0, 60 and 150 degrees, with $q = 0.3$ and $\sigma = 1$, a two-dimensional quadrature of $\mathbb{E}_{P_0}[(P/P_0)^2]$ gives 1.516754, and Equation M.51 gives 1.516754.

Figure M.9 shows which pairs of counts carry Equation 44.7. Pairs in which either copy draws no record contribute 0.948825 in total, their probability, because their weight $e^{N_1N_2/\sigma^2}$ is 1. The pair $(1, 1)$ adds 0.112710, the two mixed pairs $(1, 2)$ and $(2, 1)$ add 0.064501 together, and $(2, 2)$ adds 0.025084. Even the pair $(5, 5)$ contributes 7.032×10^{-3}. Its probability is only q^{10}, below 10^{-13}, but its weight is e^{25}. This is why the user's cost grows faster than linearly in k once the higher terms wake up, as the right panel shows.

M.4.5. What it does not cover

The argument needs an integer order, because step 4 multiplies out the α-th power. For non-integer orders, for the other direction $D_\alpha(P_0\|P)$, and for the privacy loss distribution (PLD) form that the tightest accountants use, the claim that the aligned pair is the worst case stays cited to Charles

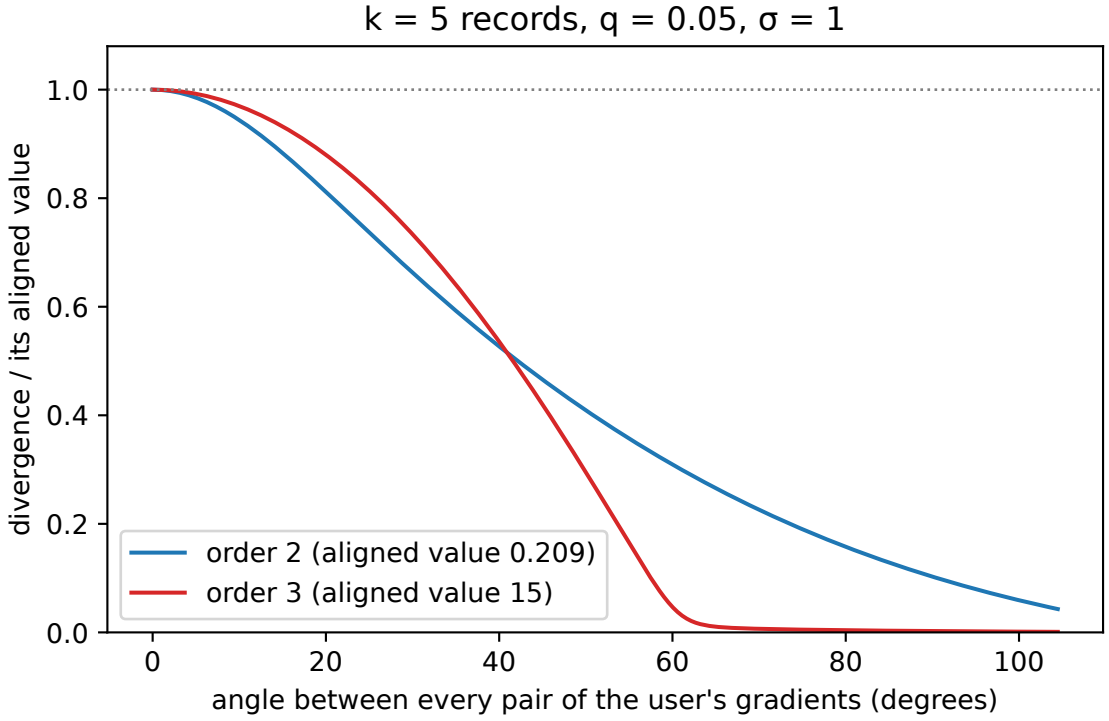


Figure M.8.: The Rényi divergence of the output with a user of five records from the output without, as a fraction of its aligned value, when every pair of the user's gradients meets at the same angle ($q = 0.05$, $\sigma = 1$). Orders 2 and 3. Both are largest when the gradients are aligned.

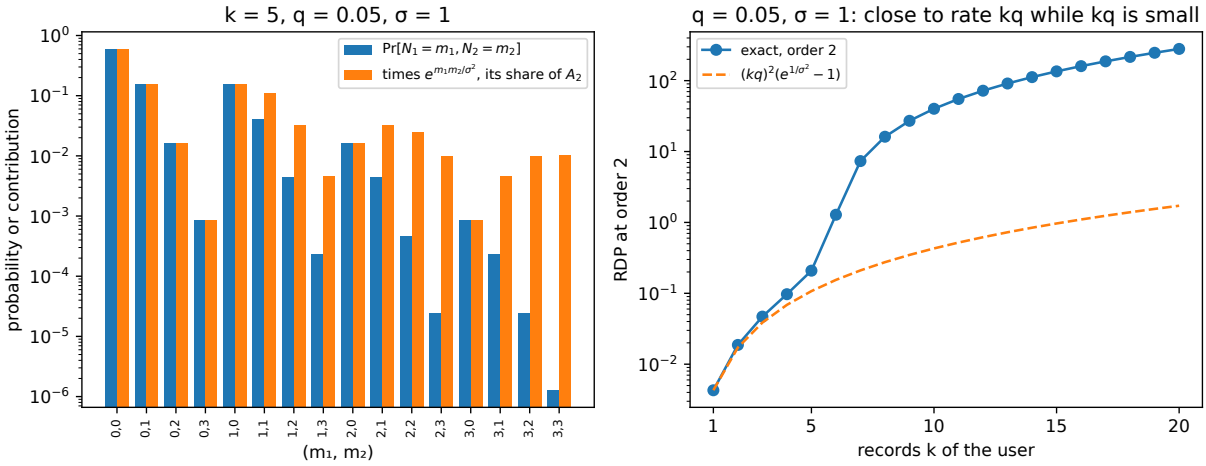


Figure M.9.: Left: the probability of each pair of counts (m_1, m_2) of a user's records in two independent batches ($k = 5$, $q = 0.05$), and its contribution to A_2 after weighting by $e^{m_1 m_2/\sigma^2}$ with $\sigma = 1$, log scale. Right: the exact order-2 RDP of one step against the number of records k, with the small-kq approximation $(kq)^2(e^{1/\sigma^2} - 1)$.

et al. (2024, Theorem 3). In the other direction the ratio is raised to the negative power $1-\alpha$, so the expansion of step 4 does not exist.

M.5. How one step of DP-SGD changes a group's loss

M.5.1. What the chapter uses

Disparate impact splits the records into groups g with shares s_g, and asks why DP-SGD hurts small groups more. In Section 45.3 it cites, without stating it, a result of Tran, Dinh & Fioretto (2021). The expected change in a group's loss in one step has a clipping term, driven by the group's gradient norms, and a noise term proportional to $\eta^2\sigma^2C^2$ times the trace of the group's loss Hessian. Here η is the learning rate, σ the noise multiplier and C the clipping norm. The chapter gives no formula. This section derives the decomposition and shows that it is exact for quadratic losses. It also shows that it contains the chapter's noise floor Equation 45.11.

Write $\mathcal{L}_g$ for group g's average loss, $\nabla\mathcal{L}_g$ for its gradient (the group's average gradient $\bar{g}_g$ of the chapter), and H_g for its Hessian. The unclipped training gradient is $\nabla\mathcal{L} = \sum_h s_h \nabla\mathcal{L}_h$, the share-weighted average of the groups' gradients.

One step moves the weights by $-\eta u$, with update $u = G_t/B + \tau\xi$. Here G_t is the sum of the batch's clipped gradients, B the expected batch size, ξ standard normal noise and $\tau = \sigma C/B$ the noise per coordinate. By Equation 45.4, the update's mean is $m = \sum_h s_h m_h$, with m_h the group's average clipped gradient. Write Σ_B for the covariance of G_t/B, which comes from sampling the batch, and $\Delta\mathcal{L}_g$ for the change in $\mathcal{L}_g$ over the step. Then, to second order in η,

$$\mathbb{E}[\Delta\mathcal{L}_g] \approx \underbrace{-\eta\langle\nabla\mathcal{L}_g, \nabla\mathcal{L}\rangle}_{\text{descent}} \underbrace{-\eta\langle\nabla\mathcal{L}_g, m-\nabla\mathcal{L}\rangle}_{\text{clipping}} + \frac{\eta^2}{2}\Big(\underbrace{m^\top H_g m}_{\text{curvature}} + \underbrace{\operatorname{tr}(H_g\Sigma_B)}_{\text{sampling}} + \underbrace{\tau^2 \operatorname{tr} H_g}_{\text{noise}}\Big), \quad \text{(M.52)}$$

and the approximation is exact when $\mathcal{L}_g$ is quadratic. The noise term is $\eta^2\sigma^2C^2 \operatorname{tr} H_g/(2B^2)$, the form Tran, Dinh & Fioretto give.

M.5.2. The idea

One step moves the weights by a random vector. A group's loss responds to the average move through its gradient, to first order, and to the spread of the move through its curvature, to second order. Clipping changes the average move. It shortens the gradients of groups with long gradients, so their share of the average falls and their loss falls more slowly. The noise does not change the average, but it adds spread. That spread is the same in every direction, so it raises a group's loss in proportion to the group's total curvature, the trace of its Hessian.

One dimension shows both effects. Take $\mathcal{L}(\theta) = \frac{h}{2}\theta^2$ and a step $\theta - \eta(m + \tau\xi)$, with ξ standard normal. The new loss averages to $\frac{h}{2}((\theta-\eta m)^2 + \eta^2\tau^2)$, because a squared normal variable has mean equal to its variance. Expanded, that is $\mathcal{L}(\theta) - \eta h\theta m + \frac{\eta^2}{2}(hm^2 + h\tau^2)$. Here $h\theta$ is the gradient, so the first-order term is the gradient times the average move, and the noise adds $\frac{\eta^2}{2}h\tau^2$ whatever the average move is.

M.5.3. Derivation

1. **The update's mean and spread.** By Equation 45.4, $\mathbb{E}[G_t/B] = m$. The noise $\tau\xi$ has mean zero, covariance $\tau^2 I$, and is independent of the batch. So u has mean m and covariance $\Sigma_B + \tau^2 I$.
2. **Expand the group's loss.** A second-order Taylor expansion around the current weights θ gives
$$\mathcal{L}_g(\theta - \eta u) = \mathcal{L}_g(\theta) - \eta\langle\nabla\mathcal{L}_g(\theta), u\rangle + \frac{\eta^2}{2}u^\top H_g u + R, \tag{M.53}$$
where the remainder R involves third derivatives. If $\mathcal{L}_g$ is quadratic, $R = 0$ and H_g is constant.
3. **Average.** The first-order term averages to $-\eta\langle\nabla\mathcal{L}_g, m\rangle$. For the second-order term, write $u = m + e$ with $\mathbb{E}[e] = 0$. Then $u^\top H_g u = m^\top H_g m + 2m^\top H_g e + e^\top H_g e$, and the middle term averages to zero. The last term averages to $\operatorname{tr}(H_g \operatorname{Cov}(u))$, because $\mathbb{E}[e^\top H_g e] = \sum_{jk}(H_g)_{jk}\mathbb{E}[e_j e_k]$. With step 1, that is $\operatorname{tr}(H_g\Sigma_B) + \tau^2 \operatorname{tr} H_g$.
4. **Split off clipping.** Write $m = \nabla\mathcal{L} + (m - \nabla\mathcal{L})$. The first part is what the step would do without clipping. The second, $m - \nabla\mathcal{L} = \sum_h s_h(m_h - \nabla\mathcal{L}_h)$, is the bias that clipping adds. A group whose records are all shorter than C has $m_h = \nabla\mathcal{L}_h$ and adds nothing to it. This gives the descent and clipping terms of Equation M.52.
5. **The sampling term.** Under Poisson sampling, each record i enters the batch independently with probability q, so $G_t = \sum_i b_i c_i$ with c_i the clipped gradient and b_i a coin with variance $q(1-q)$. With $B = qn$,
$$\Sigma_B = \frac{q(1-q)}{B^2}\sum_i c_i c_i^\top = \frac{1-q}{qn^2}\sum_i c_i c_i^\top. \tag{M.54}$$
It vanishes for full batches, $q = 1$.
6. **The noise term.** Substituting $\tau = \sigma C/B$ gives $\frac{\eta^2}{2}\tau^2 \operatorname{tr} H_g = \eta^2\sigma^2 C^2 \operatorname{tr} H_g/(2B^2)$. It does not depend on the group's share, its gradients or the clipping weights, only on its curvature.
7. **The chapter's noise floor.** Take the chapter's quadratic model: group g owns d_g weights θ_g, $\mathcal{L}_g = \frac{\lambda}{2}\|v\|^2$ with $v = \theta_g - \theta_g^*$, full batches and no clipping. On group g's weights the mean update is $s_g\lambda v$ and the group's gradient is λv, while $H_g = \lambda I$ on its d_g weights. With $a = \eta s_g\lambda$, Equation M.52 is exact and reads
$$\mathbb{E}[\Delta\mathcal{L}_g] = -\eta s_g\lambda^2\|v\|^2 + \frac{\eta^2}{2}(s_g^2\lambda^3\|v\|^2 + \tau^2 d_g\lambda) = -a(2-a)\mathcal{L}_g + \frac{\eta^2\tau^2 d_g\lambda}{2}. \tag{M.55}$$
The loss stops changing on average when the two terms balance, at
$$\mathbb{E}[\mathcal{L}_g] = \frac{\eta^2\tau^2 d_g\lambda}{2a(2-a)} = \frac{d_g\eta\tau^2}{2s_g(2-\eta s_g\lambda)}, \tag{M.56}$$
which is Equation 45.11. The descent term shrinks as the loss falls, and the noise term does not, which is why the loss settles at a floor rather than at zero. The same balance, direction by direction with share 1, gives the per-direction cost $\eta s^2/(2(2-\eta h_j))$ of Equation 43.2, where s is the noise per coordinate and h_j a curvature.

M.5.4. Checking it

The script uses the chapter's clipping example (Figure 45.2): 95 majority records with gradient $(0.2, 0)$ and 5 minority records with gradient $(0, 4)$. It gives each group the quadratic loss $\frac{1}{2}\|\theta - \theta_g^*\|^2$, whose Hessian is the identity, placed so that its gradient at the current weights is the group's gradient. All 100 records are in every batch, so the sampling term is zero. The noise is the running example's at $\varepsilon = 4$, $\sigma = 1.36$, with $B = 100$ and $\eta = 0.5$.

At $C = 0.5$ the majority is not clipped, so its clipping term is 0. Its descent term is -0.01900, its curvature term 0.00459 and its noise term 1.156×10^{-5}, for a total of -0.01440. The minority's descent term is -0.40000, but clipping adds back 0.35000. With the same curvature and noise terms, its total is -0.04540. So clipping has cancelled seven eighths of the minority's progress in this step.

The formula is exact here because the losses are quadratic. To check it, the script raises the noise multiplier to $\sigma = 100$, so that the noise term matters, and averages the minority's loss change over 2,000,000 noise draws. The average is 0.01707, against 0.01709 from Equation M.52. Step 7 is checked on the chapter's model with 51 weights per group, $\lambda = 1$, $\eta = 0.5$ and $\tau = 0.0068$. For a group with share 0.05 the floor is 5.9702×10^{-3}, and the expected change there is 0.

Figure M.10 shows the four terms against the clipping norm. For the minority, the clipping term is almost as large as the descent term until C approaches the minority's gradient length 4. The noise term grows like C^2 and is tiny per step at this budget. It matters over many steps because it does not shrink when the gradients do, which is the noise floor of step 7.

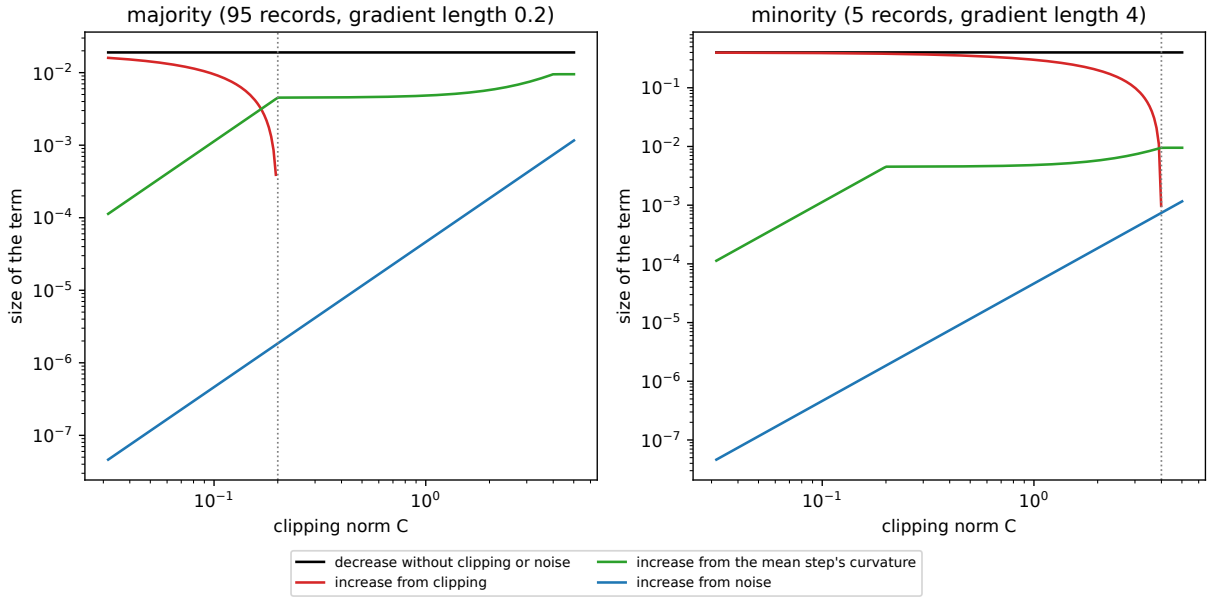


Figure M.10.: The terms of Equation M.52 for one full-batch step of the chapter's two-group example, against the clipping norm C, log–log: the loss decrease without clipping or noise, and the increases from clipping, from the curvature of the mean step and from the noise ($\sigma = 1.36$, $B = 100$, $\eta = 0.5$). The dotted line marks each group's gradient length.

M.5.5. What it does not cover

For a loss that is not quadratic, the remainder of step 2 is of order η^3 times the third derivatives, so Equation M.52 is accurate only for small steps. The decomposition is for one step at the current weights. It does not say how the terms evolve over training, which the chapter's feedback loop and noise floor address. Tran, Dinh & Fioretto (2021) also relate the trace of a group's Hessian to its distance from the decision boundary and its input norms, and that relation stays cited.

M.6. Code

The script apx_learning.py computes every number on this page and draws every figure. It runs noisy projected gradient descent and output and objective perturbation on the running example. It evaluates the data-dependent PATE bound against exact divergences on two and three labels. It sums the Rényi moment of a user's subsets exactly, and checks the identity against a quadrature. For the tuning and fairness sections, it checks every inequality on random cases and by Monte Carlo. From the repository root:

```
uv run python code/appendix-derivations/apx_learning.py
```

Table M.3 collects the main results.

Table M.3.: The main numbers the script prints, each against the value it is compared with.

Check	Value	Compared with
Upper bound Equation M.3, $n = 3{,}000$	2.4742×10^{-2}	measured 1.0754×10^{-3}
Lower bound Equation M.5, $n = 10{,}000$	1.8991×10^{-9}	upper bound 2.2268×10^{-3}
Output perturbation, $n = 10{,}000$	$[2.7835 \times 10^{-4},\ 7.2372 \times 10^{-3}]$	measured 9.6417×10^{-4}
Objective perturbation, $n = 10{,}000$	$[3.2872 \times 10^{-7},\ 2.2221 \times 10^{-4}]$	measured 7.1178×10^{-5}
PATE bound, gap 150	1.2278×10^{-3}	exact 2.2362×10^{-4}
PATE bound, votes $(200, 40, 10)$	8.7544×10^{-4}	exact 1.5188×10^{-4}
Tuning bound at order 5, limit in $\hat{\alpha}$	1.4599	exact 0.3912
Corollary 4, closed form	2.7791	numerical minimum 2.7791
User moment A_2, aligned	1.232011	60 degrees: 1.066665
Moment identity, quadrature	1.516754	formula 1.516754
Minority's step, formula	0.01709	Monte Carlo 0.01707

What to take from it. Every bound holds on every check. The upper bounds are 3.1 to 23.0 times the measured values. The lower bounds prove the rates but not useful constants. The two

places where the chapters relied on numerical evidence now have proofs, or a rigorous replacement. The pure-DP PATE bound always increases in β, and the Gaussian one can always be replaced by its largest value over $\beta \leq \tilde{\beta}$. The tuning bound for a pure-DP run is best in the limit of large $\hat{\alpha}$, slightly below its value at the end of any finite grid of orders, and the chapter's table now reports that limit.

M.7. References

- Bassily, Smith & Thakurta, "Private Empirical Risk Minimization: Efficient Algorithms and Tight Error Bounds" (FOCS 2014). The optimal rates and lower bounds of the first section, with stochastic gradient algorithms that use one record per step.
- Steinke & Ullman, "Between Pure and Approximate Differential Privacy" (Journal of Privacy and Confidentiality, 2016). The fingerprinting lemma used by the lower bounds, and the sharper version with the factor $\sqrt{\ln(1/\delta)}$.
- Lacoste-Julien, Schmidt & Bach, "A Simpler Approach to Obtaining an O(1/t) Convergence Rate for the Projected Stochastic Subgradient Method" (arXiv 2012). The weighted average that removes the logarithm from the strongly convex rate.
- Chaudhuri, Monteleoni & Sarwate, "Differentially Private Empirical Risk Minimization" (JMLR 2011). Output and objective perturbation, whose excess risk the exact bounds bracket.
- Papernot, Song, Mironov, Raghunathan, Talwar & Erlingsson, "Scalable Private Learning with PATE" (ICLR 2018). Theorem 6 and Proposition 10, and the proof of monotonicity under their condition (Appendix A).
- Papernot & Steinke, "Hyperparameter Tuning with Renyi Differential Privacy" (ICLR 2022). Theorem 2 and Corollaries 3 and 4, with the general case of an empty search (Appendix B).
- Charles, Ganesh, McKenna, McMahan, Mitchell, Pillutla & Rush, "Fine-Tuning Large Language Models with User-Level Differential Privacy" (arXiv 2407.07737, 2024). Theorem 3, the aligned worst case for both directions and all orders.
- Tran, Dinh & Fioretto, "Differentially Private Empirical Risk Minimization under the Fairness Lens" (NeurIPS 2021). The per-step decomposition of a group's loss change, and its link to the distance from the decision boundary.

N. Auditing, reconstruction and adaptive analysis

This appendix proves four results that the last chapters of Part 5 state and use but do not prove in full. Each one starts from a limit on how much an output can reveal and turns it into a limit on what someone can learn from it. Three of them start from a differential privacy (DP) guarantee, and the fourth, the Ladder, from the small number of different outputs it can produce.

A one-run audit plants test records, called canaries, in one training run and guesses which were used, and the first result limits how many of those guesses can be right. The transfer theorem is about an analyst who asks a dataset many questions, each chosen after seeing earlier answers, and it limits how far private answers can drift from the truth about the population. The Ladder is a rule for releasing a competition leaderboard, and its bound limits how far repeated submissions can push the leaderboard away from the truth. Finally, the reconstruction bound limits how close any unbiased estimate of a training record can get. The proofs share one habit, which is to bound something the attacker or the analyst chooses by a quantity that holds for every choice at once. For example, a sum of guesses is compared with a binomial count, and a classifier the analyst chose is compared with every classifier it could have submitted.

The four results sit together because they use the same few tools. Each proof conditions on what was observed, changes from one distribution to another through a likelihood ratio, or counts the possible outputs and applies a union bound. Table N.1 lists each derivation and the chapter section that uses it.

Table N.1.: The derivations of this appendix and the chapter sections that use them.

Derivation	What it proves	Where the chapter states or uses it
Section N.1	Correct guesses in a one-run audit are dominated by a binomial	Section 47.4, step 4 of the proof of Theorem 47.4
Section N.2	Private answers that are accurate on the sample are accurate on the population with high probability	Section 48.4, the first of the three cited results
Section N.3	The Ladder leaderboard's error is of order $(\ln k/n)^{1/3}$ against any analyst	Section 48.6, rule 5
Section N.4	An unbiased reconstruction has mean squared error at least $w^2/(4(e^{\varepsilon}-1))$	Section 46.4, the paragraph on Guo et al.

N.1. One-run auditing: correct guesses are dominated by a binomial

N.1.1. What the chapter uses

A one-run audit plants m canaries in the training set, each included by its own fair coin, and trains a model once. Write $S_i = +1$ if canary i is in the training set and $S_i = -1$ if it is not. The auditor then scores every canary, and it guesses "in", written $\hat{S}_i = +1$, for some canaries and "out", written $\hat{S}_i = -1$, for others. On the rest it abstains, written $\hat{S}_i = 0$. Each score is a number computed from the trained model, such as the canary's loss, and a high score suggests the canary was in. In the chapter the number of guesses r is fixed in advance: the auditor guesses "in" on the $r/2$ highest scores and "out" on the $r/2$ lowest. Let W be the number of correct guesses.

The privacy level ε of the training run is the number being tested. The chapter states that an ε-DP training run limits the count of correct guesses. That count reaches any level v no more often than a binomial count does:

$$\Pr[W \geq v] \leq \Pr\left[\mathrm{Bin}(r, p_\varepsilon) \geq v\right], \qquad p_\varepsilon = \frac{e^\varepsilon}{1 + e^\varepsilon}, \tag{N.1}$$

where $\mathrm{Bin}(r, p)$ is a binomial count of r independent trials that each succeed with probability p. This is Equation 47.15 in Section 47.4. The chapter derives it for pure DP, in which the slack δ of (ε, δ)-DP is zero, in four steps. Its step 4 writes whether each guess is right as an indicator that is 1 when a uniform random number falls below a threshold. The threshold depends on the earlier guesses, and the step compares the sum of the indicators with a binomial. It does not prove that this comparison is allowed. That missing piece is the **domination lemma** below. The version with δ, Equation 47.18, is cited to Steinke, Nasr & Jagielski (2023), and it stays cited here.

N.1.2. The idea

Each guess, on its own, is a guess about one fair coin, made from an output that is ε-DP in that coin. So no guess can be right more often than p_ε. The reason is that DP lets the output make one value of the coin at most e^ε times as likely as the other, and Bayes' rule turns odds of e^ε to 1 into the probability $e^\varepsilon/(1 + e^\varepsilon)$. The difficulty is that the guesses are not independent. The auditor ranks all the canaries together, so whether one guess is right can change the odds of the next. What saves the argument is that, once the guesses are put in some fixed order, the bound p_ε holds *given whether every earlier guess was right*, so the past can make a guess less likely to be right, but never more likely than p_ε. A count of successes in which every success has conditional probability at most p is never more likely to be large than a binomial count with success probability p.

A small case shows the last claim. Take $r = 2$ guesses and $p = 3/4$. The first guess is right with probability $3/4$. The second is right with probability $3/4$ after a correct first guess and only $1/2$ after a wrong one, so the past changes its odds, but never above $3/4$. Table N.2 compares the count with a $\mathrm{Bin}(2, 3/4)$ count.

Table N.2.: Two guesses whose second success probability depends on the first, against a binomial with the same largest success probability. Every tail of the adaptive count is at most the binomial's.

Correct guesses	0	1	2
Adaptive guesser	$\frac{1}{4} \cdot \frac{1}{2} = \frac{2}{16}$	$\frac{3}{4} \cdot \frac{1}{4} + \frac{1}{4} \cdot \frac{1}{2} = \frac{5}{16}$	$\frac{3}{4} \cdot \frac{3}{4} = \frac{9}{16}$
$\mathrm{Bin}(2, \frac{3}{4})$	$\frac{1}{16}$	$\frac{6}{16}$	$\frac{9}{16}$
Adaptive $\Pr[W \geq v]$, for $v = 1, 2$		$\frac{14}{16}$	$\frac{9}{16}$
		$\frac{15}{16}$	$\frac{9}{16}$
Binomial $\Pr[\mathrm{Bin} \geq v]$, for $v = 1, 2$			

Every tail of the adaptive count is at most the binomial's tail. The derivation shows why by building both counts from the same uniform random numbers, so that the adaptive count is never larger, outcome by outcome.

N.1.3. Derivation

The setting is the chapter's. The inclusion bits $S_1, \ldots, S_m$ are independent fair coins. Training produces an output from the dataset with the included canaries. The guesses $\hat{S}$ are computed from that output alone, possibly with extra randomness that is independent of the bits, and exactly r of them are non-zero. The first three steps are the chapter's, restated so that each says what it conditions on.

1. **Each inclusion bit is protected.** Fix every bit except S_i. Changing S_i adds or removes one record, so the training output is ε-DP as a function of S_i. The guesses are computed from the output without looking at the data, so they are ε-DP too, because post-processing cannot weaken privacy. So for every vector of guesses s,

$$\Pr[\hat{S} = s \mid S_i = +1, S_{-i}] \leq e^{\varepsilon} \Pr[\hat{S} = s \mid S_i = -1, S_{-i}], \tag{N.2}$$

   where S_{-i} stands for all the bits except S_i.

2. **Average over the later bits.** Write $S_{<i}$ for the bits $S_1, \ldots, S_{i-1}$, and $S_{>i}$ for the bits after i. Keep $S_{<i}$ fixed and average both sides of step 1 over $S_{>i}$. The later bits are independent of S_i, so the same weights appear on both sides, and the inequality survives:

$$\Pr[\hat{S} = s \mid S_i = +1, S_{<i}] \leq e^{\varepsilon} \Pr[\hat{S} = s \mid S_i = -1, S_{<i}]. \tag{N.3}$$

3. **No guess is right more than p_ε of the time, given the guesses and the earlier bits.** Write x and y for the two sides of step 2 without the factor e^{ε}, so that $x \leq e^{\varepsilon} y$. The bit S_i has prior probability $\frac{1}{2}$ of each value, and it is independent of $S_{<i}$. So Bayes' rule gives

$$\Pr[S_i = +1 \mid \hat{S} = s, S_{<i}] = \frac{\frac{1}{2}x}{\frac{1}{2}x + \frac{1}{2}y} = \frac{x}{x+y} \leq \frac{e^{\varepsilon} y}{e^{\varepsilon} y + y} = p_\varepsilon. \tag{N.4}$$

The inequality holds because $x/(x+y)$ grows with x. Swapping the roles of $+1$ and -1 gives the same bound for $\Pr[S_i = -1 \mid \hat{S} = s, S_{<i}]$. So whichever way canary i is guessed, the guess is right with probability at most p_ε, given the guesses and the earlier bits.

4. **Fix the guesses and put the guessed canaries in order.** Condition on the event $\hat{S} = s$ for one guess vector s. That fixes which r canaries are guessed. List them in index order, $i_1 < i_2 < \cdots < i_r$, and write $X_j = \mathbf{1}[S_{i_j} = s_{i_j}]$ for the indicator that the j-th guess is right. So $W = X_1 + \cdots + X_r$. The earlier indicators $X_1, \ldots, X_{j-1}$ are functions of the earlier bits $S_{<i_j}$ alone, because s is fixed. Step 3, applied to canary $i = i_j$, says that $X_j = 1$ has chance at most p_ε given $\hat{S} = s$ and the earlier bits $S_{<i_j}$. Knowing only the earlier indicators is knowing less than the earlier bits. So the chance of $X_j = 1$ given $\hat{S} = s$ and the earlier indicators is an average of those step-3 chances, taken over the values of $S_{<i_j}$ that produce the observed indicators. An average of numbers that are each at most p_ε is at most p_ε, so

$$\Pr[X_j = 1 \mid \hat{S} = s,\ X_1, \ldots, X_{j-1}] \le p_\varepsilon \qquad \text{for every } j \text{ and every past.} \tag{N.5}$$

Index order is the simplest choice. Any other order of the canaries works too, as long as it is chosen before the bits are drawn, because steps 1 to 3 can be redone with "earlier" meaning earlier in that order.

5. **The domination lemma.** Steps 1 to 4 leave a sequence of 0/1 variables in which each has conditional probability at most p of being 1, whatever happened before. The lemma says that their sum is never more likely to reach v than a binomial count.

> **Domination lemma.** Let $X_1, \ldots, X_r$ take values in $\{0, 1\}$, and suppose $\Pr[X_j = 1 \mid X_1, \ldots, X_{j-1}] \le p$ for every j and every value of the past. Then for every v,
>
> $$\Pr[X_1 + \cdots + X_r \ge v] \le \Pr\left[\mathrm{Bin}(r, p) \ge v\right]. \tag{N.6}$$

The proof builds both counts from the same random numbers. Write $g_j(x_1, \ldots, x_{j-1})$ for the conditional probability that $X_j = 1$ after the past $x_1, \ldots, x_{j-1}$, which is at most p by assumption. Draw independent numbers $U_1, \ldots, U_r$, each uniform on $[0, 1]$, and build two sequences from them.

- **A copy of the X's.** Set $Y_1 = \mathbf{1}[U_1 \le g_1]$ and, one step at a time, $Y_j = \mathbf{1}[U_j \le g_j(Y_1, \ldots, Y_{j-1})]$.
- **A binomial.** Set $B_j = \mathbf{1}[U_j \le p]$. The B_j are independent, each 1 with probability p, so their sum is $\mathrm{Bin}(r, p)$.

The sequence Y has the same distribution as X. The reason is that U_j is independent of the earlier U's, and so of $Y_1, \ldots, Y_{j-1}$. So given the past, Y_j is 1 with probability exactly g_j of that past, which is the conditional law of X_j. Multiplying these conditional probabilities along any 0/1 string gives the same probability for Y as for X, by the chain rule of probability. Next, compare the two sequences outcome by outcome. Since $g_j \le p$, whenever $U_j \le g_j$ also $U_j \le p$, so $Y_j \le B_j$ for every j and every outcome. Summing, $Y_1 + \cdots + Y_r \le B_1 + \cdots + B_r$ on every outcome. So whenever the Y count reaches v, so does the binomial count, and

$$\Pr\Big[\sum_j X_j \ge v\Big] = \Pr\Big[\sum_j Y_j \ge v\Big] \le \Pr\Big[\sum_j B_j \ge v\Big] = \Pr\left[\mathrm{Bin}(r, p) \ge v\right]. \tag{N.7}$$

This is the chapter's step 4 made precise. The uniform numbers U_j exist only inside the proof, and the copy Y is what lets the threshold g_j depend on the past.

6. **Average over the guesses.** Given $\hat{S} = s$, step 4 puts the correctness indicators in the lemma's setting with $p = p_\varepsilon$, so

$$\Pr[W \geq v \mid \hat{S} = s] \leq \Pr\left[\mathrm{Bin}(r, p_\varepsilon) \geq v\right]. \tag{N.8}$$

   The right side does not depend on s, because every guess vector has exactly r non-zero entries. So averaging over s, with weights $\Pr[\hat{S} = s]$, gives Equation 47.15.

If the number of guesses could vary with the output, step 6 would give a binomial with the number of guesses that s makes. The largest possible number of guesses then gives a bound that holds for every s, because a binomial count with more trials is more likely to reach v.

N.1.4. Checking it

Figure N.1 shows the binomial envelope for $r = 100$ guesses at four values of ε. The chapter's worked example observes $v = 75$ correct guesses. The shaded area beyond 75 is the most probability that an ε-DP training run can give to an audit at least this successful. The audit treats each ε as a claim to test. If an ε-DP run would reach 75 correct guesses with probability below $\beta = 0.05$, the audit rejects the claim that the run is ε-DP. Table N.3 lists the tail at each of the four values of ε.

Table N.3.: The binomial envelope for 100 guesses and its probability of 75 or more correct guesses, at four privacy levels.

ε	p_ε	Mean of $\mathrm{Bin}(100, p_\varepsilon)$	$\Pr[\mathrm{Bin}(100, p_\varepsilon) \geq 75]$	Rejected at $\beta = 0.05$?
0.5	0.6225	62.25	0.0048	Yes
1	0.7311	73.11	0.3827	No
$\ln 3$	0.7500	75.00	0.5535	No
2	0.8808	88.08	0.9999	No

The tail grows with ε, and it crosses 0.05 at $\varepsilon = 0.702$. That is the chapter's lower bound, recomputed by this page's script (see Section N.5). The script also reproduces the chapter's ceilings, the largest ε an audit can certify when every guess is right: 3.49 for 100 guesses, 5.81 for 1,000 and 8.11 for 10,000.

The script checks the domination lemma on an adaptive sequence it computes exactly. There are 100 indicators. The first is 1 with probability 0.75, and each later one is 1 with probability 0.75 after a success and 0.6 after a failure, in the spirit of the two-guess example of Table N.2. Their sum has mean 70.64, and its chance of reaching 75 is 0.2363, against 0.5535 for the binomial. Over every v from 0 to 100 its tail never exceeds the binomial's, except by floating-point rounding of 2.1×10^{-14}.

The bound is exact for randomized response, the mechanism that reports each inclusion bit and flips it with probability $1 - p_\varepsilon$. Here the reports play the role of the trained model. Guessing each canary's bit from its report is right with probability exactly p_ε, independently, so W is exactly $\mathrm{Bin}(r, p_\varepsilon)$. The left panel of Figure N.2 simulates 20,000 such audits at $\varepsilon = \ln 3$, each with 100

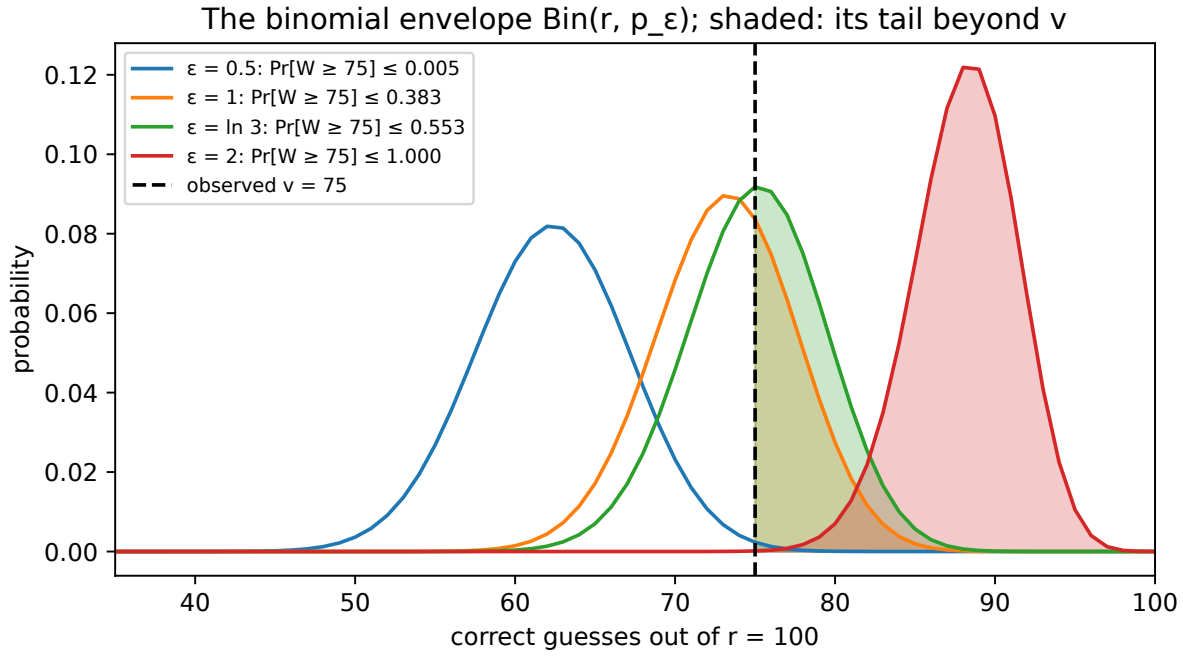


Figure N.1.: The binomial envelope $\mathrm{Bin}(100, p_\varepsilon)$ for $\varepsilon = 0.5$, 1, $\ln 3$ and 2, with the observed count of 75 correct guesses marked and each distribution's probability of 75 or more shaded.

guesses among 1,000 canaries, picked at random among the reports of each sign. The average count is 75.01, and the chance of 75 or more is 0.5544, against 0.5535 for the binomial.

The right panel scores each canary by S_i plus Laplace noise of scale $2/\varepsilon$. Changing S_i moves its score by 2, so this is ε-DP in each bit. Unlike randomized response, these scores differ in how informative they are, so the number of guesses matters. A Laplace density falls by the factor $e^{-x/b}$ over a distance x, where b is its scale. A score of 1 or more is a distance 2 closer to the centre $+1$ than to the centre -1. So it is exactly $e^{2/b} = e^{\varepsilon}$ times as likely when the canary is in as when it is out. A guess on such a confident score is therefore right with probability exactly p_ε, and the binomial is reached.

The simulation confirms this. With 1,000 canaries at $\varepsilon = 1$, where $p_\varepsilon = 0.731$, the average accuracy is 0.730 for 20 guesses and 0.732 for both 100 and 500 guesses. Guessing all 1,000 canaries includes scores near 0, which carry little information. Then the accuracy drops to 0.697. That is the chance that the noise, of scale $b = 2$ at $\varepsilon = 1$, does not move the score across 0, a distance 1 away, which is $1 - \frac{1}{2}e^{-1/2}$. So the auditor should guess only where the scores are confident, as the chapter does.

N.1.5. What it does not cover

The proof needs two things from the audit design. Each canary must be included by its own fair coin, because step 3 uses the prior $\frac{1}{2}$ and step 2 uses the independence of the bits. And the guesses must be computed from the training output alone, so that step 1 can use post-processing.

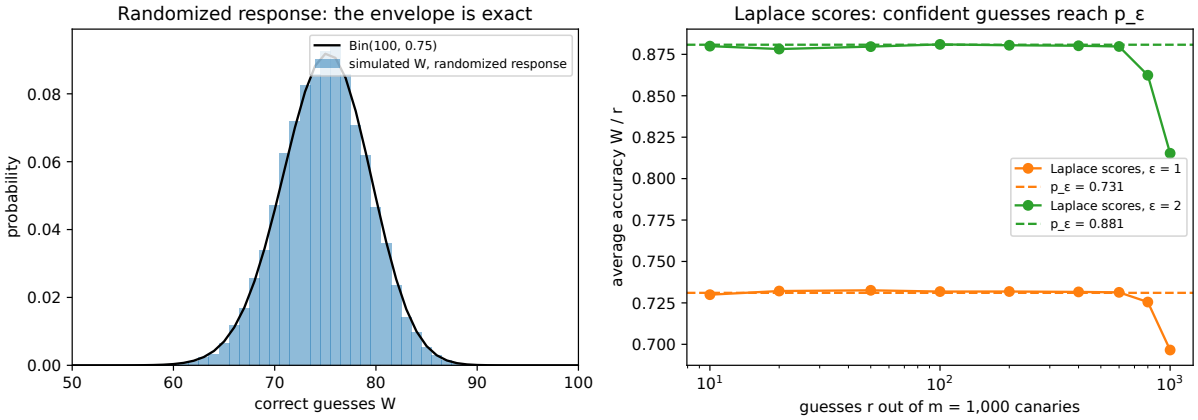


Figure N.2.: Left: the number of correct guesses in 20,000 simulated one-run audits of randomized response at $\varepsilon = \ln 3$, 100 guesses among 1,000 canaries, against the $\mathrm{Bin}(100, 0.75)$ envelope. Right: the average accuracy of one-run audits of Laplace scores against the number of guesses, at $\varepsilon = 1$ and 2, with p_ε dashed.

The proof covers pure DP only. Under (ε, δ)-DP, step 1 picks up an additive δ, and the slack can pile up across the guesses. Steinke, Nasr & Jagielski (2023) control it with a longer induction over the guesses, which gives the correction term of Equation 47.18. That argument is several pages long, so it stays cited.

N.2. Private answers stay accurate with high probability

N.2.1. What the chapter uses

The adaptive data analysis chapter proves the transfer theorem in expectation (Equation 48.14). A mechanism M answers k statistical queries that an analyst chooses one at a time, each from the earlier answers. A **statistical query** q maps one record to a number in $[0, 1]$. Its empirical value $q(D)$ is its average over the dataset D, a sample of n records drawn independently from a distribution $\mathcal{D}$. Its population value $q(\mathcal{D})$ is its average over $\mathcal{D}$. The answer to query q_j is a_j. If the whole interaction is (ε, δ)-DP in the sample, under the replace relation, and the answers are accurate on the sample on average, then they are accurate on the population on average.

Section 48.4 then states a high-probability version with citations and no proof. It credits the result to Bassily, Nissim, Smith, Steinke, Stemmer & Ullman (2016), and says that Jung, Ligett, Neel, Roth, Sharifi-Malvajerdi & Shenfeld (2020) give a shorter proof with explicit constants. This section gives that shorter proof. Call the answers (α, β)**-accurate on the sample** when, with probability at least $1 - \beta$, every answer is within α of its empirical value, so that $\max_j |a_j - q_j(D)| \le \alpha$. The theorem proved here is the following.

> **High-probability transfer theorem.** Suppose the interaction is (ε, δ)-DP in the sample under the replace relation, every answer lies in $[0, 1]$, and the answers are (α, β)-accurate on the sample. Then for every $c > 0$ and $d > 0$,
>
> $$\Pr\left[\max_{j \le k} |a_j - q_j(\mathcal{D})| > \alpha + c + e^{\varepsilon} - 1 + d\right] \le \frac{\beta}{c} + \frac{\delta}{d}. \tag{N.9}$$

The two free numbers c and d trade error for failure probability. The chapter's own statement keeps the failure probability of the order of β. This form instead divides β by c, which costs only a logarithm for Gaussian answers, as the end of the derivation shows.

N.2.2. The idea

The proof introduces one object, the **posterior** Q_π. Picture an observer who knows the distribution $\mathcal{D}$, the mechanism and the analyst's strategy, and who sees the whole transcript π of queries and answers, but not the sample. Ask that observer to describe a record picked at random from the sample. Their answer is a distribution over records, and that distribution is Q_π. Then a query's value under the posterior, $q(Q_\pi)$, is the observer's best guess of the query's empirical value. The proof compares each answer with the population in two hops, through the posterior. First, answers that are close to the sample are close to the observer's guess of the sample, because the observer's guess is an average over the samples that could have produced the transcript. Second, the observer's guess cannot be far from the population, because DP limits how much the transcript reveals about any one record. So the observer cannot believe that the sample's records are very different from fresh ones.

A small case makes the second hop concrete. Take $n = 20$ coin records, each 1 with probability $p = 0.3$, and one query, the average. The mechanism answers it with Laplace noise of scale $1/(n\varepsilon)$, which is ε-DP because one record moves the average by at most $1/n$. Figure N.3 plots the posterior mean of a sample record, $q(Q_\pi)$, against the observed answer a. With $\varepsilon = 4$ the answer is nearly exact, and the posterior follows it: after seeing $a = 0.6$, the observer believes the sample's records are 1 with probability 0.5973. With $\varepsilon = 1$ the observer believes 0.4989, and with $\varepsilon = 0.25$ only 0.3539, close to the population's 0.3. However strange the answer, the posterior cannot leave a band around the population value, and the band narrows as ε shrinks.

N.2.3. Derivation

The randomness is the sample, the mechanism's coins and the analyst's coins. The analyst's strategy and the distribution $\mathcal{D}$ are fixed and known to the proof, and they are not part of the private input. The transcript π lists every query and every answer, so each q_j and each a_j is a function of π. For simplicity the records take countably many values, so that the posterior is a list of probabilities. With densities, every sum below becomes an integral.

1. **The posterior and the resampling identity.** Pick an index I uniformly from $1, \dots, n$, independently of everything else, and define the posterior as the conditional distribution of the record x_I given the transcript:

$$Q_\pi(x) = \Pr[x_I = x \mid \pi] = \frac{1}{n} \sum_{i=1}^{n} \Pr[x_i = x \mid \pi]. \tag{N.10}$$

   For any query q that is a function of π, the empirical value $q(D)$ is the average of $q(x_i)$ over the records, so its conditional expectation is the average of q under the posterior:

$$\mathbb{E}[q(D) \mid \pi] = \frac{1}{n} \sum_{i=1}^{n} \mathbb{E}[q(x_i) \mid \pi] = \sum_{x} q(x)\, Q_\pi(x) = q(Q_\pi). \tag{N.11}$$

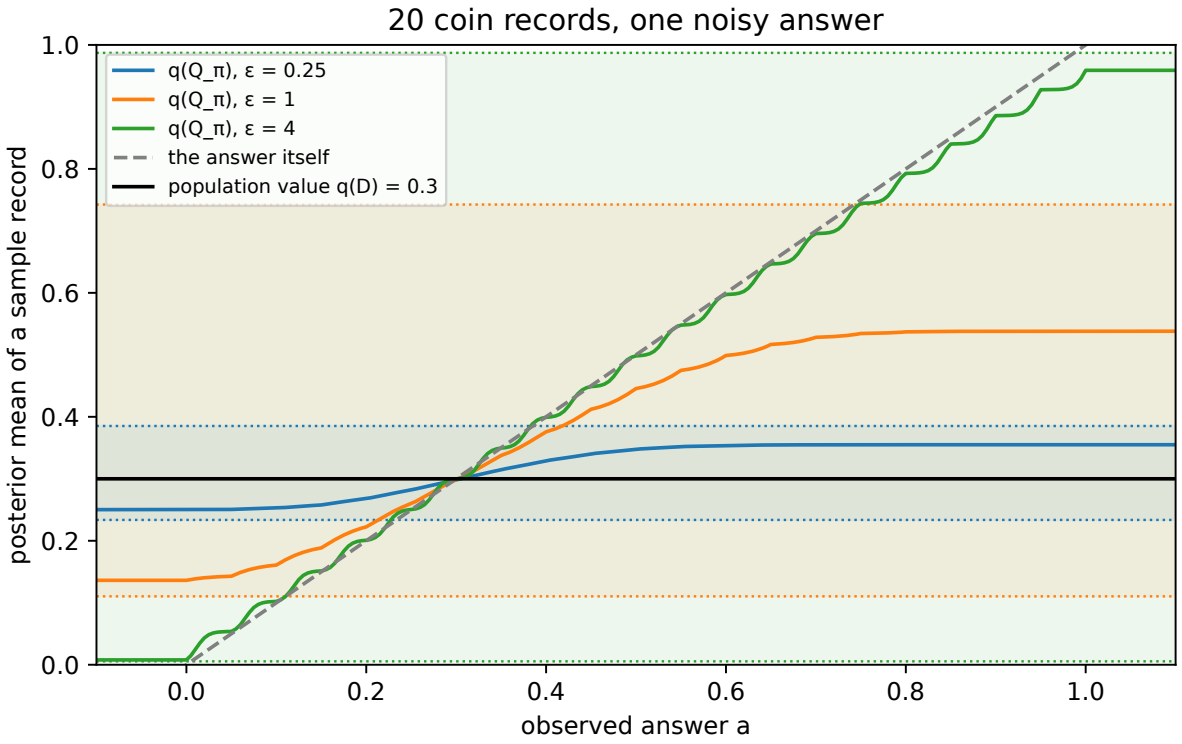


Figure N.3.: The posterior mean of a randomly chosen sample record, $q(Q_\pi)$, against the observed noisy answer a, for 20 coin records with mean 0.3 and Laplace noise at $\varepsilon = 0.25$, 1 and 4. Shaded with dotted edges: the band that pure DP allows, in the matching colour. The dashed diagonal is the answer itself and the black line the population value.

In words, the observer's best guess of a query's empirical value is the query's value under the posterior.

2. **Accuracy on the sample gives accuracy against the posterior.** Write F for the failure event $\max_j |a_j - q_j(D)| > \alpha$, which has probability at most β. The answer a_j is fixed by π, so by Equation N.11, $a_j - q_j(Q_\pi) = \mathbb{E}[a_j - q_j(D) \mid \pi]$. Outside F the gap $|a_j - q_j(D)|$ is at most α. Inside F it is at most 1, because both a_j and $q_j(D)$ lie in $[0, 1]$. So

$$\big|a_j - q_j(Q_\pi)\big| \le \mathbb{E}[|a_j - q_j(D)| \mid \pi] \le \alpha + \Pr[F \mid \pi]. \tag{N.12}$$

3. **The conditional failure probability is rarely large.** The conditional failure probability $\Pr[F \mid \pi]$ is a non-negative quantity whose average over transcripts is $\Pr[F] \le \beta$. Markov's inequality, which says that a non-negative quantity exceeds c with probability at most its average divided by c, gives

$$\Pr_\pi\big[\Pr[F \mid \pi] > c\big] \le \frac{\beta}{c}. \tag{N.13}$$

So, except with probability at most β/c, every answer is within $\alpha + c$ of its posterior value.

4. **Under pure DP, the posterior is never far from the population.** This warm-up is the case $\delta = 0$, and it explains the bands of Figure N.3. Fix a record i, a value x and a transcript π. Bayes' rule writes the observer's belief about record i through the chance of the transcript:

$$\Pr[x_i = x \mid \pi] = \frac{\Pr[\pi \mid x_i = x]\,\mathcal{D}(x)}{\Pr[\pi]}. \tag{N.14}$$

Replacing x_i by another value x' multiplies the chance of the transcript by at most e^ε, whatever the other records are. So it does so after averaging over the other records too, and $\Pr[\pi \mid x_i = x] \le e^\varepsilon \Pr[\pi \mid x_i = x']$. Averaging the right side over x' drawn from $\mathcal{D}$ gives $e^\varepsilon \Pr[\pi]$, because that is how x_i is drawn. Substituting into Bayes' rule, and then averaging over the record i, gives

$$\Pr[x_i = x \mid \pi] \le e^\varepsilon\,\mathcal{D}(x), \qquad Q_\pi(x) \le e^\varepsilon\,\mathcal{D}(x). \tag{N.15}$$

The same argument with x and x' swapped gives $Q_\pi(x) \ge e^{-\varepsilon}\mathcal{D}(x)$. Now average a query q with values in $[0, 1]$ against these two bounds, and do the same for $1 - q$:

$$e^{-\varepsilon} q(\mathcal{D}) \le q(Q_\pi) \le e^\varepsilon q(\mathcal{D}), \qquad e^{-\varepsilon}(1 - q(\mathcal{D})) \le 1 - q(Q_\pi) \le e^\varepsilon(1 - q(\mathcal{D})). \tag{N.16}$$

These confine $q(Q_\pi)$ to a band around $q(\mathcal{D})$. The upper bound of the first pair puts the band's top at most $(e^\varepsilon - 1)q(\mathcal{D})$ above the population value. The upper bound of the second pair puts its bottom at most $(e^\varepsilon - 1)(1 - q(\mathcal{D}))$ below it. Both distances are at most $e^\varepsilon - 1$.

5. **Under (ε, δ)-DP, the posterior is far from the population only rarely.** With $\delta > 0$ the pointwise bound of step 4 fails, so the proof bounds a probability instead. The tool is the one-sided transfer lemma that the chapter proves in Section 48.3. It says that if an (ε, δ)-DP mechanism M' reads the dataset and outputs a query $q_{M'}$, then the query's expected empirical value is not much larger than its expected population value:

$$\mathbb{E}[q_{M'}(D)] \le e^\varepsilon\,\mathbb{E}[q_{M'}(\mathcal{D})] + \delta. \tag{N.17}$$

The reason, in short, is a swap. The population value of a query is its value on a fresh record x_i' that the algorithm never saw. Swapping x_i' with the sample's record x_i leaves

the joint distribution unchanged, so it equals the value on x_i of a query chosen from a sample without x_i. And DP says that choosing from the sample with x_i instead of without it raises the average of any $[0,1]$-valued function of the choice by at most the factor e^{ε} plus δ (Equation 48.7).

Apply the lemma to a query built from the transcript. Among the $2k$ queries q_j and $1-q_j$, let q_π^* be the one with the largest gap $q(Q_\pi) - q(\mathcal{D})$. Adding the complements makes this largest signed gap equal to the largest absolute gap $\max_j |q_j(Q_\pi) - q_j(\mathcal{D})|$, because the complement $1-q$ has the same gap with the opposite sign. Fix a threshold τ, and let E be the event that this gap exceeds τ. Define the query

$$h_\pi(x) = q_\pi^*(x)\,\mathbf{1}[\pi \in E], \tag{N.18}$$

which is q_π^* on transcripts in E and the zero query elsewhere. The algorithm that outputs h_π computes it from the transcript and from the fixed $\mathcal{D}$, so it is post-processing of the interaction and is (ε,δ)-DP. The lemma applied to it, with $h_\pi(D) = q_\pi^*(D)\mathbf{1}[E]$ and $h_\pi(\mathcal{D}) = q_\pi^*(\mathcal{D})\mathbf{1}[E]$, gives

$$\mathbb{E}\big[q_\pi^*(D)\,\mathbf{1}[E]\big] \le e^{\varepsilon}\,\mathbb{E}\big[q_\pi^*(\mathcal{D})\,\mathbf{1}[E]\big] + \delta. \tag{N.19}$$

Both q_π^* and $\mathbf{1}[E]$ are fixed by π, so the resampling identity Equation N.11 turns the left side into $\mathbb{E}[q_\pi^*(Q_\pi)\mathbf{1}[E]]$. Subtract $\mathbb{E}[q_\pi^*(\mathcal{D})\mathbf{1}[E]]$ from both sides. On E the gap exceeds τ, which gives the left inequality below. On the right, $(e^{\varepsilon}-1)\mathbb{E}[q_\pi^*(\mathcal{D})\mathbf{1}[E]] \le (e^{\varepsilon}-1)\Pr[E]$ because $q_\pi^*(\mathcal{D}) \le 1$. So

$$\tau \Pr[E] \le \mathbb{E}\Big[(q_\pi^*(Q_\pi) - q_\pi^*(\mathcal{D}))\mathbf{1}[E]\Big] \le (e^{\varepsilon}-1)\Pr[E] + \delta. \tag{N.20}$$

Choose $\tau = e^{\varepsilon} - 1 + d$ and solve for $\Pr[E]$:

$$\Pr\Big[\max_j \big|q_j(Q_\pi) - q_j(\mathcal{D})\big| > e^{\varepsilon} - 1 + d\Big] \le \frac{\delta}{d}. \tag{N.21}$$

With $\delta = 0$ the right side is 0 for every d, which agrees with step 4.

6. **Combine.** Outside the two bad events of steps 3 and 5, which together have probability at most $\beta/c + \delta/d$ by the union bound, every answer satisfies

$$\big|a_j - q_j(\mathcal{D})\big| \le \big|a_j - q_j(Q_\pi)\big| + \big|q_j(Q_\pi) - q_j(\mathcal{D})\big| \le (\alpha + c) + (e^{\varepsilon} - 1 + d). \tag{N.22}$$

This is Equation N.9.

Gaussian answers. Now apply the theorem to the mechanism of Section 48.4. It answers query j with $q_j(D) + \xi_j$, where the noise ξ_j is normal with mean 0 and standard deviation σ, independently for each query. Here the answer is also clipped, that is, moved into $[0,1]$ if it falls outside. Clipping is post-processing, and it never moves an answer away from $q_j(D)$, which lies in $[0,1]$.

The privacy comes from step 1 of the chapter's derivation in Section 48.4. Zero-concentrated differential privacy (zCDP) measures privacy by one number ρ, a bound on the Rényi divergence of every order divided by the order. Gaussian noise of standard deviation σ on a value whose sensitivity is Δ gives $\rho = \Delta^2/(2\sigma^2)$. One record moves an average of n values in $[0,1]$ by at most

$1/n$, so each answer is zCDP with $\rho_1 = 1/(2n^2\sigma^2)$. Composition adds these to $\rho = k/(2n^2\sigma^2)$, and Equation 48.16 converts the total to (ε, δ)-DP with

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{N.23}$$

The sample accuracy comes from the normal tail. Each noise exceeds t in absolute value with probability at most $2e^{-t^2/(2\sigma^2)}$. So a union bound over the k answers gives, for any failure probability β',

$$\Pr\Big[\max_j |\xi_j| > \sigma\sqrt{2\ln(2k/\beta')}\Big] \le \beta'. \tag{N.24}$$

In other words, the answers are $(\sigma\sqrt{2\ln(2k/\beta')}, \beta')$-accurate on the sample.

Now fix a target total error α and failure probability β; here α is the total, not the sample-accuracy term of the theorem. The four error terms of Equation N.9 are the noise level $\sigma\sqrt{2\ln(2k/\beta')}$, then c, then $e^{\varepsilon} - 1$, then d. Give each a quarter of α, so $c = d = \alpha/4$. Split the failure probability equally too, with $\beta' = \delta = \alpha\beta/8$, so that β'/c and δ/d are both $\beta/2$. The noise term is then at most $\alpha/4$ when

$$\sigma = \frac{\alpha}{4\sqrt{2\ln\left(16k/(\alpha\beta)\right)}}. \tag{N.25}$$

For the transfer term, $e^{\varepsilon} - 1 \approx \varepsilon$ when ε is small, and step 4 of the chapter's derivation notes that for small ρ the privacy level is about $\varepsilon \approx \sqrt{2k\ln(1/\delta)}/(n\sigma)$. Setting this to $\alpha/4$ gives $n \approx 4\sqrt{2k\ln(1/\delta)}/(\alpha\sigma)$. Substituting the σ above and $\ln(1/\delta) = \ln(8/(\alpha\beta))$ gives

$$n \approx \frac{32\sqrt{k\,\ln\left(8/(\alpha\beta)\right)\,\ln\left(16k/(\alpha\beta)\right)}}{\alpha^2} = \tilde{O}\Big(\frac{\sqrt{k}}{\alpha^2}\Big), \tag{N.26}$$

where $\tilde{O}$ hides factors that are logarithmic in the number of queries and in $1/\alpha$ and $1/\beta$. This is the chapter's high-probability rate. It needs ε of order α and δ of order $\alpha\beta$, the same conditions the chapter quotes. The failure probability enters only through logarithms, so dividing it by c costs little.

N.2.4. Checking it

The pure-DP band of step 4 holds in the coin example. The script computes the posterior exactly on a grid of answers from -0.2 to 1.2, and Table N.4 compares the range of the posterior mean with the band.

Table N.4.: The posterior mean of a sample record in the coin example, with 20 records of mean 0.3 and one Laplace-noised answer, against the band of step 4.

ε	Posterior mean at $a = 0.6$	Range of the posterior mean	Band allowed by pure DP
0.25	0.3539	$[0.2502, 0.3550]$	$[0.2336, 0.3852]$
1	0.4989	$[0.1362, 0.5381]$	$[0.1104, 0.7425]$
4	0.5973	$[0.0078, 0.9590]$	$[0.0055, 0.9872]$

The posterior stays inside the band at every ε, but it does not fill it, because a single noisy average reveals too little about the records to push the posterior to the band's edge.

Table N.5 solves both bounds for the smallest number of records that brings the largest error down to $\alpha = 0.05$. The expectation column is the chapter's Table 48.5, recomputed. The high-probability column chooses the noise level σ and the numbers c and d. It also chooses how to split $\beta = 0.05$ between the two failure terms. It makes these choices with a numerical search, so it is accurate to about three digits and is rounded to three.

Table N.5.: Records that Gaussian answering needs for a largest error of 0.05, on average and with probability 0.95.

Queries k	Records, expected error Equation 48.17	Records, error with probability 0.95, Equation N.9	Ratio
1	7,300	38,400	5.3
100	201,800	455,000	2.3
1,000	764,337	1,530,000	2.0
1,000,000	33,393,779	56,500,000	1.7

The high-probability guarantee costs a factor of about 2 for a thousand queries, and the factor shrinks as k grows. At the optimum for $k = 1{,}000$ the noise term is 0.0243 and the transfer term $e^{\varepsilon} - 1$ is 0.0240. The slack terms are tiny, with $c = 0.0006$ and $d = 0.0012$, so the optimizer asks for $\delta = 3.89 \times 10^{-5}$ and a sample failure probability of 1.11×10^{-5}. This shows the logarithmic price at work: it is cheaper to make both failure probabilities very small than to give up error.

Figure N.4 runs the chapter's boosting analyst against Gaussian answers. Each record is a point with a label, and the labels are fair coins. A submission is a labelling of the points, and its query asks "does the submission agree with this record's label?", so its value is an accuracy. A fresh label is independent of every submission, so every query has population value exactly $\frac{1}{2}$. The analyst submits 99 random labellings and then the majority vote of those whose answer was above $\frac{1}{2}$. With $n = 10{,}000$ records, $k = 100$ queries and the noise level that minimizes the expectation bound, $\sigma = 0.0319$, the largest population error averages 0.0888 over 400 runs, and 95% of runs stay below 0.1125. The expectation bound is 0.2141, and the high-probability bound at the same σ is 0.3218. So the bounds are valid here, and loose by a factor of about 2.4 on average and about 2.9 at the 95% level. Part of the slack is the analyst, which is far from the worst case. The rest is the transfer step, which charges $e^{\varepsilon} - 1$ to every query.

N.2.5. What it does not cover

The theorem needs the answers to lie in $[0, 1]$, which clipping provides. It also needs the guarantee to hold under the replace relation, because the transfer lemma swaps a record for a fresh one. A mechanism that is (ε, δ)-DP for adding or removing a record is $(2\varepsilon, (1 + e^{\varepsilon})\delta)$-DP for replacing one. The form proved here divides the sample failure probability by c. Bassily et al. (2016) prove a form in which a failure probability β on the sample gives a failure probability of order β on the population, with a different argument, a "monitor" that picks the worst of many independent runs of the interaction. That form stays cited. For mechanisms whose accuracy depends on β only through $\ln(1/\beta)$, such as Gaussian noise, the two forms give the same rate.

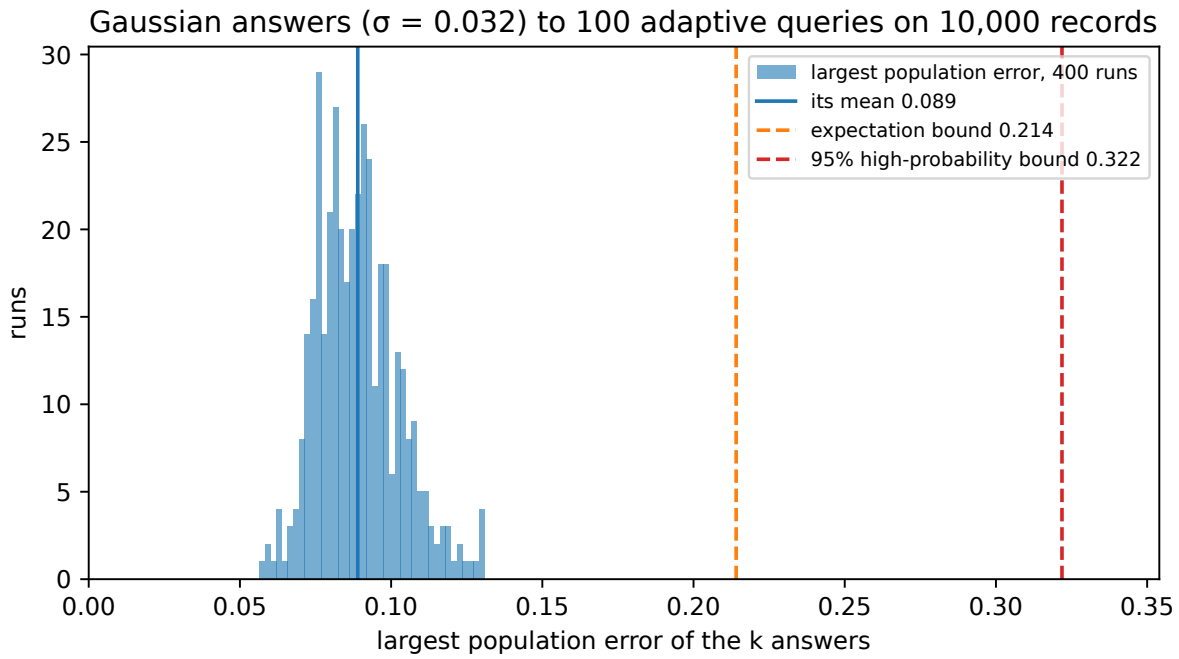


Figure N.4.: The largest population error of Gaussian answers to 100 adaptively chosen queries on 10,000 records, over 400 runs of the boosting analyst, with its mean, the expectation bound Equation 48.17 and the high-probability bound Equation N.9 at failure probability 0.05.

N.3. The Ladder: a leaderboard that rarely changes cannot be overfitted

N.3.1. What the chapter uses

Rule 5 of Section 48.6 says that the **Ladder** of Blum & Hardt (2015) keeps a competition leaderboard accurate against any analyst. The Ladder reports a new score only when a submission beats the best so far by a margin, rounded to that margin. The chapter states that its error against the population is of order $n^{-1/3}$, times a factor that grows like the cube root of $\ln(kn)$, where n is the size of the holdout set and k the number of submissions. It cites the paper and gives only the idea that a leaderboard that changes rarely reveals few bits.

To state the result, write the score as a loss in $[0, 1]$, where lower is better. For a classifier f, the empirical loss $\mathcal{L}(f; D)$ is its average loss on the n records of the holdout set D, and the population loss $\mathcal{L}_{\mathcal{D}}(f)$ is its average over the distribution. With the margin $\nu = 1/J$ for a whole number J, the Ladder releases values $R_0, R_1, \dots, R_k$ as follows.

- Start with $R_0 = 1$.
- When submission f_t arrives, compute $\mathcal{L}(f_t; D)$. If $\mathcal{L}(f_t; D) < R_{t-1} - \nu$, release R_t as $\mathcal{L}(f_t; D)$ rounded to the nearest multiple of ν. Otherwise, release $R_t = R_{t-1}$ again.

The leaderboard should show the best population loss among the submissions so far, so the **leaderboard error** is the largest gap $\max_t |R_t - \min_{i \le t} \mathcal{L}_{\mathcal{D}}(f_i)|$. The result proved here is the following.

> **Ladder bound.** Against any analyst, who may choose each submission from the values released so far, with probability at least $1 - \beta$ the leaderboard error is at most
>
> $$\nu + \sqrt{\frac{\ln(2k/\beta) + \ln \binom{k+J}{J}}{2n}} \;\le\; 3\Big(\frac{\ln(k+1)}{8n}\Big)^{1/3} + \sqrt{\frac{\ln(2k/\beta)}{2n}}, \qquad \text{(N.27)}$$
>
> where the right side holds at the best step $\nu = (\ln(k+1)/(8n))^{1/3}$, ignoring that $1/\nu$ must be a whole number.

The leading term is of order $(\ln k/n)^{1/3}$, and $\ln k \le \ln(kn)$, so this gives the chapter's rate.

N.3.2. The idea

An analyst overfits a holdout set by learning which submissions happen to do well on it and then combining them. To learn that, it must read the scores. The Ladder releases very little. Its released value moves at most $1/\nu$ times, because each move lowers it by at least ν and it starts at 1. Each move happens at one of k submissions and lands on one of $1/\nu$ rounded values. So the whole sequence of released values is one of a modest number of possible sequences, even though the analyst may react to it in any way it likes.

Once the analyst's own coins are fixed, it is a deterministic machine. Each submission is a function of the values released before it. So the set of every classifier it *could* submit, over all possible released sequences, is fixed before the holdout set is drawn, and it is not too large. A fixed set of classifiers is exactly what Hoeffding's inequality and a union bound handle, as in the fixed-query bound of Section 48.1. The adaptivity has been paid for by counting. The rounding to ν is the

price of keeping that count small, so the best ν balances rounding error against the logarithm of the count.

N.3.3. Derivation

1. **The released value falls by at least ν at each move.** All released values are multiples of ν, because $R_0 = 1 = J\nu$ and every new value is rounded to a multiple of ν. A move happens only if $\mathcal{L}(f_t; D) < R_{t-1} - \nu$. Rounding adds at most $\nu/2$, so $R_t < R_{t-1} - \nu/2$. Two multiples of ν that differ by more than $\nu/2$ differ by at least ν, so $R_t \le R_{t-1} - \nu$. The values stay at or above 0, since the loss does. So there are at most $J = 1/\nu$ moves.

2. **The number of possible released sequences.** A released sequence is fixed by the times of its moves and the values it moves to. With u moves, the times are u of the k submissions, and the values are u distinct multiples of ν among $0, \nu, \dots, (J-1)\nu$, which must appear in decreasing order. So the number of sequences is

$$N = \sum_{u=0}^{J} \binom{k}{u}\binom{J}{u} = \binom{k+J}{J} \le (k+1)^J. \tag{N.28}$$

   The middle step is Vandermonde's identity: choosing J items from $k+J$ is the same as choosing u from the first k and $J-u$ from the last J, summed over u, and $\binom{J}{J-u} = \binom{J}{u}$. The last step holds because $\binom{k+J}{J}$ is the product of the ratios $(k+i)/i$ for $i = 1, \dots, J$, and each ratio is at most $k+1$.

3. **The classifiers the analyst could submit.** Fix the analyst's coins, which are independent of the holdout set. Submission t is then a function of the values $R_1, \dots, R_{t-1}$. Those are the first $t-1$ entries of a released sequence, so they take at most N values. Let $\mathcal{F}$ be the set of all classifiers the analyst would submit at some time t after some possible history. It has at most kN members, and it is fixed before the holdout set is drawn, because it depends only on the analyst's strategy and coins.

4. **Every classifier in the set is accurate.** For one fixed classifier, the empirical loss is an average of n independent losses in $[0, 1]$. Hoeffding's inequality bounds the chance that it is more than γ from its mean by $2e^{-2n\gamma^2}$. The union bound over the at most kN members of $\mathcal{F}$ bounds the chance that any of them is off by more than γ by $2kNe^{-2n\gamma^2}$. Setting this to β and solving for γ shows that, with probability at least $1-\beta$,

$$\left|\mathcal{L}(f; D) - \mathcal{L}_{\mathcal{D}}(f)\right| \le \gamma = \sqrt{\frac{\ln(2kN/\beta)}{2n}} \qquad \text{for every } f \in \mathcal{F}. \tag{N.29}$$

   The analyst's actual submissions all belong to $\mathcal{F}$, whatever the holdout set turns out to be.

5. **The released value tracks the best empirical loss.** Write $m_t = \min_{i \le t} \mathcal{L}(f_i; D)$ for the best empirical loss so far. If the leaderboard has not moved by time t, then $R_t = 1$ and every submission had $\mathcal{L}(f_i; D) \ge 1 - \nu$, so $R_t - \nu \le m_t \le R_t$. Otherwise, let $t_0 \le t$ be the time of the last move. Then $R_t = R_{t_0}$ is within $\nu/2$ of $\mathcal{L}(f_{t_0}; D)$, so $m_t \le R_t + \nu/2$. For the other side, take any $i \le t$. If submission i moved the leaderboard, $\mathcal{L}(f_i; D) \ge R_i - \nu/2 \ge R_t - \nu/2$, because the released values never go up. If it did not, $\mathcal{L}(f_i; D) \ge R_{i-1} - \nu \ge R_t - \nu$. So

$$R_t - \nu \le m_t \le R_t + \frac{\nu}{2}. \tag{N.30}$$

6. **Combine.** On the event of step 4, each empirical loss is within γ of the population loss, so the smallest empirical loss so far is within γ of the smallest population loss so far. Together with step 5, $|R_t - \min_{i \le t} \mathcal{L}_{\mathcal{D}}(f_i)| \le \nu + \gamma$ for every t. Substituting $N = \binom{k+J}{J}$ gives the first form of Equation N.27. If the analyst's coins are random, the bound holds for each value of the coins, and so it holds on average over them too.

7. **The best step.** For a closed form, bound the number of histories by $\ln N \le J \ln(k+1) = \ln(k+1)/\nu$, and split the square root with $\sqrt{x+y} \le \sqrt{x} + \sqrt{y}$. Write $\kappa = \ln(k+1)/(2n)$ for short. Then the bound becomes

$$\nu + \sqrt{\frac{\kappa}{\nu}} + \sqrt{\frac{\ln(2k/\beta)}{2n}}. \tag{N.31}$$

   The first two terms trade off. Their derivative in ν is $1 - \frac{1}{2}\sqrt{\kappa}\,\nu^{-3/2}$, which is zero at $\nu^3 = \kappa/4$. There $\sqrt{\kappa/\nu} = 2\nu$, so the two terms add up to $3\nu = 3(\kappa/4)^{1/3} = 3(\ln(k+1)/(8n))^{1/3}$, which is the second form of Equation N.27.

N.3.4. Checking it

Figure N.5 plots the first form of the bound against ν for $n = 10{,}000$ records and $k = 1{,}000$ submissions, at $\beta = 0.05$. A large step makes the rounding term large, while a small one allows more moves, and so more possible histories and a larger union-bound term. The sum is smallest at $\nu = 1/32$, where it is 0.118. The closed form is looser, because it replaces $\binom{k+J}{J}$ by $(k+1)^J$. It puts the best step at 0.0442 and the bound at 0.1556. The closed form's step must be rounded to the form $1/J$. At $\nu = 1/23$, the nearest such step to 0.0442, the first form gives $0.0435 + 0.0769 = 0.1203$.

The script also runs the chapter's boosting attack against both kinds of leaderboard, with $n = 1{,}000$ coin labels and $k = 1{,}000$ submissions, over 20 runs. Every submission has population loss exactly $\frac{1}{2}$. Against a leaderboard that shows each submission's exact loss, the attacker keeps the submissions that scored below $\frac{1}{2}$, and its final majority vote makes the best shown loss wrong by 0.2207 on average. Against the Ladder with $\nu = 1/11$, the attacker sees only when the released value moves. The first submission moves the leaderboard from 1 to its loss near $\frac{1}{2}$, rounded to 5/11 or 6/11, both $1/22 = 0.0455$ away from $\frac{1}{2}$. Later random submissions almost never beat that by ν, so the attacker learns almost nothing, and the error stays at 0.0455, the rounding of that first move. The bound for these sizes is 0.2768 at $\nu = 1/11$, so it is loose by a factor of about 6 here.

N.3.5. What it does not cover

The proof needs the losses to lie in $[0, 1]$ and the holdout records to be independent draws, for Hoeffding's inequality. It controls the best score so far, which is what a leaderboard shows, not the score of every submission. Blum & Hardt (2015) also give a parameter-free variant that sets the margin from the data, which stays cited.

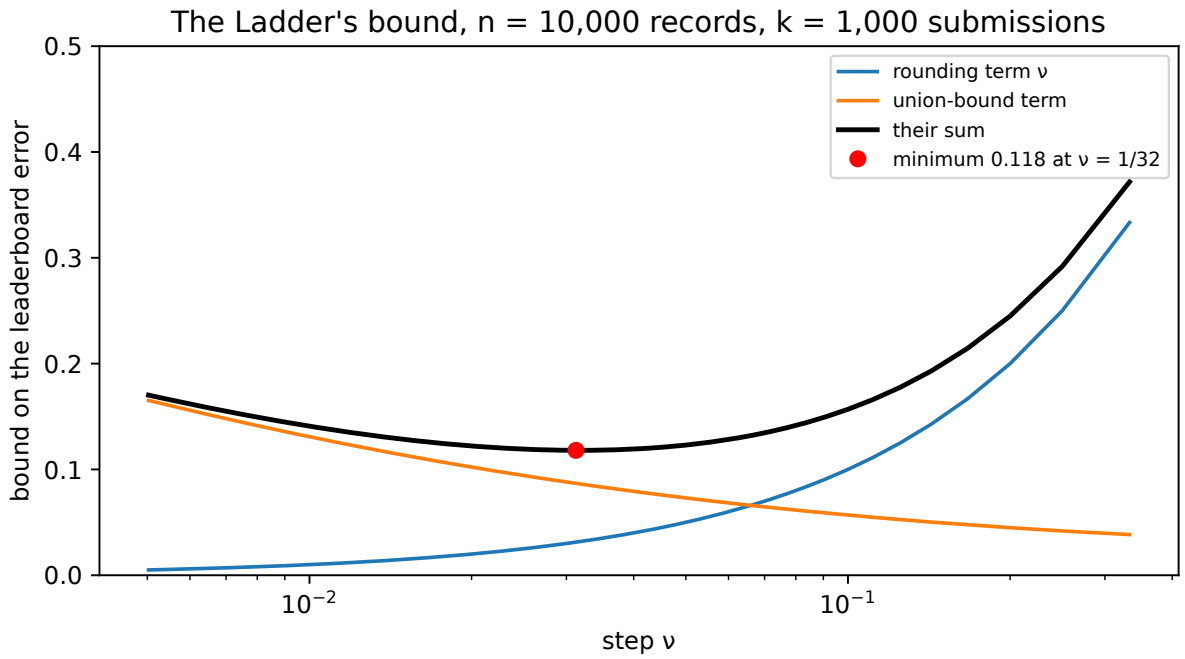


Figure N.5.: The Ladder's bound on the leaderboard error against the step ν, for $n = 10{,}000$ holdout records and $k = 1{,}000$ submissions at $\beta = 0.05$: the rounding term ν, the union-bound term, and their sum, with its minimum marked.

N.4. Unbiased reconstruction has a floor set by Rényi DP of order 2

N.4.1. What the chapter uses

Section 46.4 mentions a bound of Guo, Karrer, Chaudhuri & van der Maaten (2022) in one sentence. It says that under Rényi differential privacy (RDP) of order 2 at level ε, the expected mean squared error of any reconstruction is at least of order $1/(e^{\varepsilon} - 1)$. It adds that Fisher information gives a tighter, data-dependent bound for unbiased reconstructions. The chapter gives no proof. This section proves the first bound, and it has to correct the chapter's wording, because the bound holds for *unbiased* reconstructions, as in Guo et al., and not for every reconstruction.

The setting is a target record whose value is a number x in an interval of width w. The rest of the training data is fixed. The training run releases an output Y whose distribution P_x depends on x. The **Rényi divergence of order 2** between two distributions P and Q is

$$D_2(P\|Q) = \ln \mathbb{E}_Q\left[\left(\frac{dP}{dQ}\right)^2\right], \tag{N.32}$$

where dP/dQ is the likelihood ratio, the density of P divided by the density of Q. Training is $(2, \varepsilon)$-RDP when $D_2(P_{x'}\|P_x) \le \varepsilon$ for every pair of target values x and x', since changing the target is replacing one record. A reconstruction is a function $\hat{x}(Y)$ of the output. It is **unbiased** when its average equals the true value whatever that value is, $\mathbb{E}_x[\hat{x}] = x$ for every x. The result is

$$\mathbb{E}_x[(\hat{x} - x)^2] \ge \frac{w^2}{4(e^{\varepsilon} - 1)} \qquad \text{for every } x. \tag{N.33}$$

A target with several coordinates, coordinate ℓ in its own interval of width w_ℓ, gives the sum of these bounds, $\sum_\ell w_\ell^2/(4(e^\varepsilon-1))$.

N.4.2. The idea

An unbiased reconstruction must move its average by exactly $x'-x$ when the target moves from x to x'. The only thing that changes between the two cases is the distribution of the output, and under RDP that distribution changes little. So the reconstruction must amplify small differences in the output into a large difference in its average, and amplifying noise makes the reconstruction vary a lot. An inequality of Hammersley, Chapman and Robbins makes this precise. Moving the target reweights each output by the likelihood ratio, so the change in the reconstruction's average is a covariance between the reconstruction and that ratio. A covariance is at most the product of two spreads, by the Cauchy–Schwarz inequality. One spread is the reconstruction's error. The other is the spread of the likelihood ratio, which is called the χ^2 divergence and is set by RDP of order 2.

A concrete case is the Gaussian mechanism, which releases $Y = x + Z$ with Z normal of standard deviation σ and $x \in [0,1]$. The natural reconstruction is Y itself. It is unbiased, and its mean squared error is σ^2. Between two normals with the same σ and means a distance h apart, the order-2 divergence is h^2/σ^2, by completing the square in $\mathbb{E}[(dP/dQ)^2]$. Between targets 0 and 1 it is $1/\sigma^2$, so the mechanism is $(2,\varepsilon)$-RDP with $\varepsilon = 1/\sigma^2$, and its error is $1/\varepsilon$. At $\varepsilon = 1$ the error is 1, and the bound Equation N.33 is $1/(4(e-1)) = 0.1455$.

N.4.3. Derivation

Fix two target values x and x' in the interval, and write $\Lambda = dP_{x'}/dP_x$ for the likelihood ratio of the output. The divergence $D_2(P_{x'}\|P_x)$ is finite, so $P_{x'}$ gives no probability to outputs that P_x never produces, and the ratio is well defined.

1. **Change of measure.** An average under $P_{x'}$ is an average under P_x weighted by the likelihood ratio, $\mathbb{E}_{x'}[\hat{x}] = \mathbb{E}_x[\hat{x}\Lambda]$. The weights average to 1, $\mathbb{E}_x[\Lambda] = 1$, because the density of $P_{x'}$ integrates to 1.

2. **Write the change of the average as a covariance.** By unbiasedness, $\mathbb{E}_{x'}[\hat{x}] - \mathbb{E}_x[\hat{x}] = x'-x$. By step 1 the left side is $\mathbb{E}_x[\hat{x}(\Lambda-1)]$. Since $\mathbb{E}_x[\Lambda-1] = 0$, subtracting the constant x from $\hat{x}$ changes nothing, so

$$x' - x = \mathbb{E}_x\big[(\hat{x}-x)(\Lambda-1)\big]. \tag{N.34}$$

3. **Cauchy–Schwarz.** The average of a product is at most the square root of the product of the averages of the squares, so

$$(x'-x)^2 \le \mathbb{E}_x\big[(\hat{x}-x)^2\big] \cdot \mathbb{E}_x\big[(\Lambda-1)^2\big]. \tag{N.35}$$

   The first factor is the mean squared error at x. The second is the χ^2 **divergence** $\chi^2(P_{x'}\|P_x)$. This is the Hammersley–Chapman–Robbins inequality.

4. **The χ^2 divergence is set by order-2 RDP.** Expanding the square, $\mathbb{E}_x[(\Lambda-1)^2] = \mathbb{E}_x[\Lambda^2] - 2\mathbb{E}_x[\Lambda] + 1 = \mathbb{E}_x[\Lambda^2] - 1$. By the definition of D_2, $\mathbb{E}_x[\Lambda^2] = e^{D_2(P_{x'}\|P_x)}$, so

$$\chi^2(P_{x'}\|P_x) = e^{D_2(P_{x'}\|P_x)} - 1 \le e^{\varepsilon} - 1. \tag{N.36}$$

   Dividing step 3 by this gives, for every x' in the interval,

$$\mathbb{E}_x\big[(\hat{x}-x)^2\big] \ge \frac{(x'-x)^2}{e^{\varepsilon}-1}. \tag{N.37}$$

5. **Choose the farthest x'.** The bound holds for every x', so take the one farthest from x, which is an end of the interval. Wherever x lies, one of the two ends is at least $w/2$ away, so $(x'-x)^2 \ge w^2/4$, and Equation N.33 follows. At an end of the interval the other end is w away, and the bound is four times larger, $w^2/(e^{\varepsilon}-1)$.

6. **Several coordinates.** If the target has several coordinates, each in its own interval, apply steps 1 to 5 to each coordinate of $\hat{x}$, with x' differing from x in that coordinate only. Changing one coordinate is still a change of one record, so the RDP bound applies. The squared error is the sum of the coordinates' squared errors, so the bounds add.

N.4.4. Checking it

Table N.6 compares the bound with two unbiased mechanisms for a target in $[0,1]$, so $w=1$. The Gaussian mechanism has error $1/\varepsilon$, as above. The Laplace mechanism releases $x+Z$ with Laplace noise of scale b, whose error is $2b^2$. Its order-2 divergence between targets 0 and 1 is $\ln\left(\frac{2}{3}e^{1/b} + \frac{1}{3}e^{-2/b}\right)$ (Mironov 2017), which the script also checks by numerical integration at $b=1$, getting 0.619124 both ways. The script solves for the b that makes this equal to ε.

Table N.6.: The mean squared error of unbiased reconstructions of a target in $[0,1]$ from the Gaussian and Laplace mechanisms, against the bound Equation N.33, at the same RDP of order 2.

RDP level ε	Bound $1/(4(e^{\varepsilon}-1))$	Gaussian error $1/\varepsilon$	Laplace error $2b^2$	Gaussian over bound
0.1	2.377	10	17.36	4.21
0.5	0.3854	2	2.647	5.19
1	0.1455	1	1.023	6.87
2	0.03913	0.5	0.3458	12.8
4	0.004664	0.25	0.103	53.6
8	8.389×10^{-5}	0.125	0.02831	1,490

Both mechanisms stay above the bound, as they must. For small ε the Gaussian's error is about 4 times the bound. For large ε the gap grows without limit, because the bound falls like $e^{-\varepsilon}$ and the Gaussian's error only like $1/\varepsilon$. The left panel of Figure N.6 shows this on a log scale.

The right panel shows where the slack comes from, at $\varepsilon = 1$, so $\sigma = 1$. For the Gaussian, the true χ^2 divergence between targets a distance h apart is $e^{h^2/\sigma^2} - 1$, from the order-2 divergence above, so Equation N.37 with the mechanism's own pair gives $h^2/(e^{h^2/\sigma^2}-1)$. That rises to the

exact error σ^2 as h shrinks to 0. This limit is the Cramér–Rao bound, which uses the Fisher information, the rate at which the output distribution changes with the target. The RDP bound instead uses the worst-case χ^2, that of targets a full width apart, for every pair. At $h = \frac{1}{2}$, the page's choice for a target in the middle, the true divergence of that pair is $e^{1/4} - 1$, but the bound uses $e - 1$. So the slack comes from summarizing the mechanism by its worst pair.

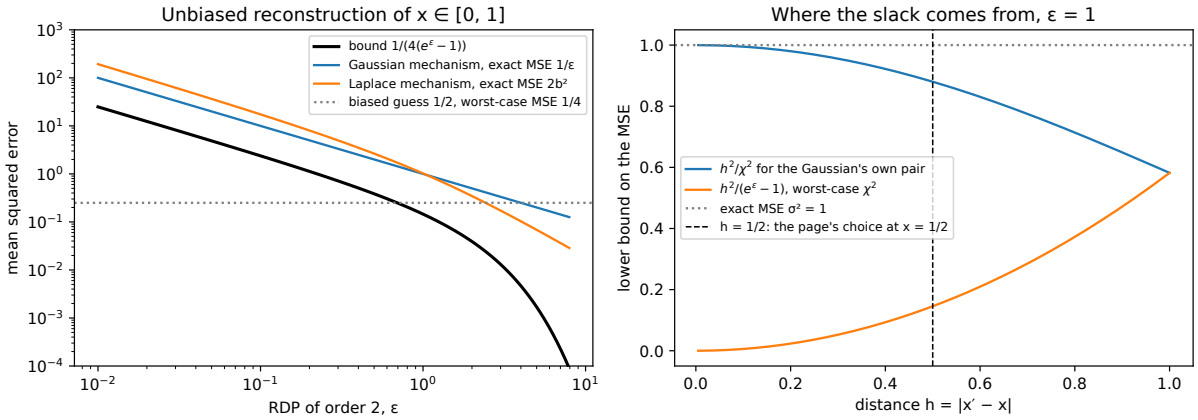


Figure N.6.: Left: the bound $1/(4(e^{\varepsilon}-1))$ on the mean squared error of an unbiased reconstruction of a target in $[0, 1]$, against the exact error of the Gaussian and Laplace mechanisms at the same RDP of order 2, and the worst-case error $\frac{1}{4}$ of the biased guess $\frac{1}{2}$. Right: at $\varepsilon = 1$, the Hammersley–Chapman–Robbins bound h^2/χ^2 against the distance h between the two targets, with the Gaussian's own χ^2 and with the worst-case $e^{\varepsilon} - 1$.

N.4.5. What it does not cover

The bound needs unbiasedness. Step 2 uses it at both x and x'. Without it, the bound fails. For example, the reconstruction that ignores the output and always answers $\frac{1}{2}$ has squared error at most $\frac{1}{4}$ for every target in $[0, 1]$. But Equation N.33 exceeds $\frac{1}{4}$ whenever $e^{\varepsilon} - 1 < 1$, that is, for $\varepsilon < \ln 2 = 0.693$. At $\varepsilon = 0.1$ the "bound" is 2.377, and the constant guess beats it by a factor of almost 10. So for small ε no such bound can hold for every reconstruction, and the chapter states it for unbiased ones.

A biased reconstruction still obeys a weaker statement. Steps 1 to 4 go through if, in step 2, the targets x and x' on the left are replaced by the averages $\mathbb{E}_x[\hat{x}]$ and $\mathbb{E}_{x'}[\hat{x}]$. The x inside the expectation is replaced by $\mathbb{E}_x[\hat{x}]$ too, which turns the squared error into the variance. They give

$$\left(\mathbb{E}_{x'}[\hat{x}] - \mathbb{E}_x[\hat{x}]\right)^2 \le \mathrm{Var}_x(\hat{x})\,(e^{\varepsilon} - 1), \tag{N.38}$$

where Var_x is the variance at x. In words, a reconstruction whose average follows the target closely must vary a lot, and one that varies little cannot follow the target. Guo et al. (2022) state their bound for unbiased reconstructions, and they also give the Fisher-information refinement for them, which is the limit of Equation N.37 as x' approaches x. That refinement needs the output distribution to change smoothly with the target, and it uses how fast the trained model's output distribution moves with the target rather than the RDP level, so it stays cited.

N.5. Code

The script code/appendix-derivations/apx_auditing.py computes every number on this page and draws its figures. It computes binomial tails exactly, solves for the one-run audit's ε bound, and computes the law of an adaptive sum exactly to check the domination lemma. It simulates one-run audits of randomized response and of Laplace scores. It computes the posterior of the coin example exactly, solves the expectation and high-probability bounds for the records that Gaussian answering needs, and simulates Gaussian answers to the boosting analyst. It evaluates the Ladder's bound, runs the boosting attack against an exact leaderboard and against the Ladder, and compares the reconstruction bound with the exact error of the Gaussian and Laplace mechanisms. From the repository root:

```
uv run python code/appendix-derivations/apx_auditing.py
```

Output (seed 0), summarized in Table N.7:

Table N.7.: Summary of the script's output.

Check	Result
One-run audit, $r = 100$, $v = 75$	$\varepsilon \geq 0.702$ at 95% confidence; tails 0.0048, 0.3827, 0.5535 and 0.9999 at $\varepsilon = 0.5$, 1, $\ln 3$ and 2
Domination lemma, adaptive sum of 100	$\Pr[X \geq 75] = 0.2363$ against 0.5535; never above the binomial
Randomized response, $\varepsilon = \ln 3$, 20,000 runs	Mean 75.01 correct; $\Pr[W \geq 75] = 0.5544$ against 0.5535
Laplace scores, $\varepsilon = 1$	Accuracy 0.730 to 0.732 for 20 to 500 guesses, against $p_\varepsilon = 0.731$; 0.697 for all 1,000
Coin posterior, $n = 20$	Inside the pure-DP band at $\varepsilon = 0.25$, 1 and 4
Records for error 0.05, $k = 1{,}000$	764,337 on average; about 1,530,000 with probability 0.95
Gaussian answers, $n = 10{,}000$, $k = 100$	Mean largest error 0.0888, 95% quantile 0.1125; bounds 0.2141 and 0.3218
Ladder, $n = 10{,}000$, $k = 1{,}000$	Best bound 0.118 at $\nu = 1/32$; closed form 0.1556
Boosting attack, $n = k = 1{,}000$	Error 0.2207 against an exact leaderboard, 0.0455 against the Ladder
Reconstruction, $\varepsilon = 1$	Bound 0.1455; Gaussian 1, Laplace 1.023

tests/test_appendix_auditing.py checks that the output still matches the numbers on this page.

What to take from it. The binomial envelope is exact for randomized response and for confident Laplace scores, and an adaptive guesser can only fall below it. The high-probability transfer theorem costs about a factor of 2 in records over the expectation version, because the failure probabilities enter only through logarithms. The Ladder stops the boosting attack almost

completely, while its worst-case bound is loose by a factor of about 6. The reconstruction bound is valid for unbiased reconstructions only, and it is loose by at least a factor of 4 for the Gaussian mechanism, because it charges every pair of targets the worst pair's divergence.

N.6. References

- Steinke, Nasr & Jagielski, "Privacy Auditing with One (1) Training Run" (NeurIPS 2023). The one-run audit, the binomial bound for pure DP and the δ correction, whose proof is cited above.
- Bassily, Nissim, Smith, Steinke, Stemmer & Ullman, "Algorithmic Stability for Adaptive Data Analysis" (STOC 2016). The first high-probability transfer theorem, by the monitor argument.
- Jung, Ligett, Neel, Roth, Sharifi-Malvajerdi & Shenfeld, "A New Analysis of Differential Privacy's Generalization Guarantees" (ITCS 2020). The posterior and resampling argument that this page follows. The constants here are derived on this page.
- Dwork, Feldman, Hardt, Pitassi, Reingold & Roth, "Preserving Statistical Validity in Adaptive Data Analysis" (STOC 2015). The transfer results in expectation that the chapter proves.
- Blum & Hardt, "The Ladder: A Reliable Leaderboard for Machine Learning Competitions" (ICML 2015). The Ladder, its guarantee, the boosting attack and a parameter-free variant.
- Guo, Karrer, Chaudhuri & van der Maaten, "Bounding Training Data Reconstruction in Private (Deep) Learning" (ICML 2022). The reconstruction bound from RDP of order 2 and its Fisher-information refinement, both for unbiased reconstructions.
- Hammersley, "On Estimating Restricted Parameters" (*Journal of the Royal Statistical Society B*, 1950), and Chapman & Robbins, "Minimum Variance Estimation without Regularity Assumptions" (*Annals of Mathematical Statistics*, 1951). The variance bound through a second distribution, used in the reconstruction proof.
- Mironov, "Rényi Differential Privacy" (CSF 2017). The Rényi divergences of the Laplace and Gaussian mechanisms used in the reconstruction check.

O. Glossary

Acronyms used in the book and the pages that use them.

AI Artificial intelligence. Used in Choosing ε.

API Application programming interface. Used in Threat models and privacy games, Memorization and training-data extraction, Choosing ε.

AUC Area under the curve. Used in Hypothesis testing, Threat models and attacks, Membership inference, Private prediction: PATE and in-context learning.

CDF Cumulative distribution function. Used in Probability distributions, Hypothesis testing, Threat models and privacy games, Syntactic anonymity and how it breaks, The Laplace and Gaussian mechanisms, Selection: exponential mechanism, report-noisy-max, sparse vector, Discrete and floating-point-safe mechanisms, Sums, limits and counting.

CDP Concentrated differential privacy. Used in Concentrated differential privacy.

CLT Central limit theorem. Used in Probability distributions, f-DP and Gaussian differential privacy, Numerical accounting with privacy loss distributions, Choosing an accountant, Auditing private training, Sums, limits and counting, Composition of trade-off functions.

CPA Chosen-plaintext attack. Used in Cryptography basics.

DLG Deep Leakage from Gradients. Used in Gradient inversion.

DNA Deoxyribonucleic acid. Used in Reconstruction attacks, Lower bounds: what privacy costs.

DP Differential privacy. Used in Differential privacy foundations, The definition of differential privacy, Post-processing, group privacy and composition, The Laplace and Gaussian mechanisms, Advanced and optimal composition, Selection: exponential mechanism, report-noisy-max, sparse vector, Query release and private multiplicative weights, Beyond global sensitivity, Discrete and floating-point-safe mechanisms, Local, shuffle and distributed trust models, Lower bounds: what privacy costs, Private mean estimation, Privacy accounting, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Concentrated differential privacy, f-DP and Gaussian differential privacy, Amplification by subsampling, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Adaptive composition: filters, odometers and individual accounting, Choosing an accountant, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Hyperparameter tuning and private selection, DP-FTRL and matrix factorization, Private prediction: PATE and in-context learning, Public data and private fine-tuning, User-level differential privacy, Disparate impact, Choosing ε, Auditing private training, Adaptive data analysis, How much noise to add, Composing private releases, Accounting for DP-SGD, What ε means, and converting between definitions, Random matrices and factorizations, Mechanisms and their limits, Composition of trade-off functions, Subsampling, shuffling and iteration, Private learning and tuning, Auditing, reconstruction and adaptive analysis.

DPI Data processing inequality. Used in Post-processing, group privacy and composition.

EMD Earth mover's distance. Used in Syntactic anonymity and how it breaks.

ERM Empirical risk minimization. Used in Optimization, Algorithmic stability and generalization, Differential privacy in machine learning, Private empirical risk minimization, Public data

and private fine-tuning, Private learning and tuning.

FL Federated learning. Used in Gradient inversion.

FNR False negative rate. Used in Hypothesis testing, The definition of differential privacy, What ε means, and converting between definitions.

FPR False positive rate. Used in Hypothesis testing, Threat models and attacks, Threat models and privacy games, Membership inference, Attribute inference and model inversion, Memorization and training-data extraction, The definition of differential privacy, Lower bounds: what privacy costs, Choosing ε, Auditing private training, What ε means, and converting between definitions.

FTRL Follow the regularized leader. Used in Query release and private multiplicative weights, Differential privacy in machine learning, DP-FTRL and matrix factorization, Choosing ε, Random matrices and factorizations.

GAN Generative adversarial network. Used in Divergences.

GD Gradient descent. Used in Optimization, Amplification by shuffling and by iteration, Optimization and estimation.

GDP Gaussian differential privacy. Used in f-DP and Gaussian differential privacy, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Choosing an accountant, Differential privacy in machine learning, DP-FTRL and matrix factorization, Choosing ε, Auditing private training, What ε means, and converting between definitions, Random matrices and factorizations, Composition of trade-off functions.

GPU Graphics processing unit. Used in Threat models and attacks, Differential privacy in machine learning.

HIV Human immunodeficiency virus. Used in Syntactic anonymity and how it breaks.

HMAC Hash-based message authentication code. Used in Cryptography basics.

IND Indistinguishability, as in IND-CPA. Used in Cryptography basics.

IP Internet Protocol, as in IP address. Used in Choosing ε.

KL Kullback–Leibler. Used in Prerequisites, Divergences, Hypothesis testing, Information theory, Advanced and optimal composition, Local, shuffle and distributed trust models, Rényi differential privacy, Concentrated differential privacy, f-DP and Gaussian differential privacy, Sums, limits and counting, Tail bounds, tests and divergences.

LLM Large language model. Used in Private prediction: PATE and in-context learning.

LP Linear programming. Used in Reconstruction attacks.

LR Likelihood ratio. Used in Hypothesis testing.

LWE Learning with errors. Used in Cryptography basics, Pseudorandom functions and learning with errors.

MAP Maximum a posteriori. Used in Information theory, Attribute inference and model inversion.

MGF Moment generating function. Used in Probability distributions, Concentration inequalities, From Rényi DP to (ε, δ) and to tests, Concentrated differential privacy, Accounting for DP-SGD, What ε means, and converting between definitions, Sums, limits and counting, Tail bounds, tests and divergences, Private learning and tuning.

ML Machine learning. Used in Membership inference, Choosing an accountant, DP-SGD, Hyperparameter tuning and private selection, Choosing ε, Accounting for DP-SGD.

MWEM Multiplicative weights exponential mechanism. Used in Query release and private multiplicative weights.

NN Nearest neighbour. Used in Algorithmic stability and generalization, Optimization and estimation.

NYC New York City. Used in Linkage and re-identification.

PAC Probably approximately correct. Used in Selection: exponential mechanism, report-noisy-

max, sparse vector.

PATE Private aggregation of teacher ensembles. Used in Differential privacy in machine learning, Private prediction: PATE and in-context learning, Private learning and tuning.

PCP Probabilistically checkable proof. Used in Query release and private multiplicative weights.

PIN Personal identification number. Used in Information theory, Tail bounds, tests and divergences.

PLD Privacy loss distribution. Used in f-DP and Gaussian differential privacy, Numerical accounting with privacy loss distributions, Choosing an accountant, Choosing ε, Composing private releases, Accounting for DP-SGD, Composition of trade-off functions, Private learning and tuning.

PMF Probability mass function. Used in Probability distributions, Discrete and floating-point-safe mechanisms.

PPT Probabilistic polynomial time. Used in Cryptography basics.

PRF Pseudorandom function. Used in Cryptography basics, Pseudorandom functions and learning with errors.

PRG Pseudorandom generator. Used in Cryptography basics, Pseudorandom functions and learning with errors.

PRP Pseudorandom permutation. Used in Cryptography basics.

PRV Privacy loss random variable. Used in f-DP and Gaussian differential privacy, Numerical accounting with privacy loss distributions, Choosing an accountant.

RDP Rényi differential privacy. Used in Privacy accounting, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Concentrated differential privacy, f-DP and Gaussian differential privacy, Amplification by subsampling, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Adaptive composition: filters, odometers and individual accounting, Choosing an accountant, Differential privacy in machine learning, DP-SGD, Hyperparameter tuning and private selection, DP-FTRL and matrix factorization, Private prediction: PATE and in-context learning, User-level differential privacy, Choosing ε, Auditing private training, Composing private releases, Accounting for DP-SGD, What ε means, and converting between definitions, Tail bounds, tests and divergences, Subsampling, shuffling and iteration, Private learning and tuning, Auditing, reconstruction and adaptive analysis.

RMSE Root-mean-square error. Used in Private mean estimation, Random matrices and factorizations.

ROC Receiver operating characteristic. Used in Hypothesis testing, Threat models and attacks, Threat models and privacy games, Membership inference.

SGD Stochastic gradient descent. Used in Optimization, Algorithmic stability and generalization, Gradient inversion, Privacy accounting, Rényi differential privacy, Concentrated differential privacy, f-DP and Gaussian differential privacy, Amplification by subsampling, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Adaptive composition: filters, odometers and individual accounting, Choosing an accountant, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Hyperparameter tuning and private selection, DP-FTRL and matrix factorization, Private prediction: PATE and in-context learning, Public data and private fine-tuning, User-level differential privacy, Disparate impact, Choosing ε, Auditing private training, Composing private releases, Accounting for DP-SGD, What ε means, and converting between definitions, Optimization and estimation, Random matrices and factorizations, Composition of trade-off functions, Subsampling, shuffling and iteration, Private learning and tuning.

SQL Structured Query Language. Used in Reconstruction attacks.

TPR True positive rate. Used in Hypothesis testing, Threat models and attacks, Threat models and privacy games, Membership inference, Attribute inference and model inversion, Memorization and training-data extraction, The definition of differential privacy, Lower bounds: what privacy costs, Choosing ε, Auditing private training, What ε means, and converting between definitions.

TV Total variation. Used in Divergences, Hypothesis testing, Cryptography basics, Threat models and privacy games, Property inference, Syntactic anonymity and how it breaks, The definition of differential privacy, The Laplace and Gaussian mechanisms, Discrete and floating-point-safe mechanisms, Local, shuffle and distributed trust models, Lower bounds: what privacy costs, f-DP and Gaussian differential privacy, Sums, limits and counting.

XOR Exclusive or. Used in Cryptography basics.

ZIP Zone Improvement Plan code, the US postal code. Used in Linkage and re-identification, Syntactic anonymity and how it breaks, Lower bounds: what privacy costs.

P. Index

AboveThreshold, 467–469, 471–474, 488, 489, 878, 996, 1243
adaptive composition, 409, 410, 450, 451, 453, 468, 601, 611, 623, 645, 671, 697, 708, 710, 718, 722, 747, 760, 763, 769, 770, 792, 808, 842, 866, 871, 992, 1223, 1244, 1269, 1312
advanced composition, 375, 442, 443, 448–451, 453, 454, 468, 469, 473, 474, 484, 486, 600, 620, 623, 661, 698, 745, 751, 758–760, 762–764, 767, 768, 770, 1034, 1043, 1052–1054
amplification by iteration, 731, 740, 769, 1291
amplification by shuffling, 549, 557, 601, 740, 1291
amplification by subsampling, 371, 601, 604, 621, 623, 642, 661, 702, 754, 762, 780, 808, 880, 1073, 1277, 1291
analytic Gaussian mechanism, 429–434, 553
approximate differential privacy, 374, 375, 377, 378, 390, 397, 398, 404, 406, 409, 411, 412, 416, 453, 493, 501, 563, 567, 571, 578, 579, 596, 597, 722, 740, 922, 973, 1319
approximate zCDP, 654, 660, 661, 760, 1243
attribute inference, 195, 196, 198, 236, 238, 253, 254, 340, 957
auxiliary knowledge, 199, 202, 203, 205, 215, 238, 239, 242
Azuma's inequality, 49, 442

background knowledge attack, 349
basic composition, 375, 408, 411, 412, 414, 416, 439–441, 443, 444, 449, 450, 452–454, 468, 502, 586, 591, 600, 620, 623, 655, 675, 698, 701, 760, 762, 763, 765, 770, 878, 879, 996, 1034, 1052, 1053, 1092, 1236
Bernoulli distribution, 11, 12, 14, 30, 39, 41, 43, 52, 105, 381, 888, 1123, 1126, 1127, 1144, 1146
binary symmetric channel, 109–111, 122, 123
binomial distribution, 11, 12, 24, 29, 100, 101, 212, 259, 260, 271, 410, 440, 446, 547, 655, 661, 675, 693–695, 700, 701, 769, 832, 834, 842, 851, 908–910, 914, 935, 941, 964, 971, 973, 977, 1055, 1065, 1073, 1094, 1096, 1099–1102, 1105, 1119, 1145, 1147, 1230, 1291, 1307, 1320, 1322–1325, 1341, 1342
black-box attack, 202, 204, 205, 215, 216, 236, 256, 272, 508, 513, 965, 974, 975, 979, 1175
blatantly non-private, 351, 369

canary, 197, 198, 236, 274, 281–285, 288, 289, 783, 962, 963, 965–968, 970, 971, 973–980, 1322, 1323, 1326
central limit theorem, 22, 30, 47, 208, 263, 454, 674–678, 680, 682, 683, 718, 719, 721, 725, 726, 728, 739, 763–768, 775–777, 935, 960, 967, 970, 979, 980, 1094, 1100, 1111, 1121, 1167, 1244, 1262, 1265–1269
central limit theorem for composition, 454, 674, 960
channel capacity, 123
Chernoff bound, 29, 30, 35–39, 41, 43, 45, 47, 49, 52, 53, 101, 547, 584, 590, 618, 644, 760, 1109, 1114, 1119, 1122, 1124, 1127, 1130, 1131, 1146, 1147, 1282–1284
clipping, 357, 358, 375, 462, 473, 507, 510, 511, 513, 581, 582, 584, 586,

591–596, 598, 600, 689, 756, 761, 773, 780, 781, 783, 801, 803, 805–807, 811, 812, 815, 817, 818, 820–823, 839, 840, 843, 858, 885, 890, 892, 893, 902, 905, 907, 911, 913, 914, 917, 921–925, 927–930, 932, 936, 938–941, 948, 958, 962, 967, 974, 975, 979, 980, 1055, 1058, 1059, 1072, 1073, 1266, 1297, 1316, 1317
Clopper–Pearson interval, 42, 97–99, 229, 935–937, 940, 941, 963–965, 969–971, 978–980, 1122, 1134, 1136, 1146
collision probability, 180, 181, 317
composition, 198, 344, 346, 348, 349, 371–373, 375, 397, 398, 408–414, 416, 417, 439–444, 446–454, 468, 469, 473–475, 484, 486, 502, 586, 591, 600, 601, 611, 612, 620, 622, 623, 641, 642, 644, 645, 650, 654, 655, 658, 661, 662, 671, 672, 674, 675, 682, 683, 697, 698, 701, 702, 705, 708–710, 718, 721, 722, 735, 736, 738, 740, 745, 747, 749, 751, 758–760, 762–770, 772, 777, 778, 792, 804, 808, 812, 826–828, 833, 834, 836–839, 841, 842, 866, 871, 878–880, 884, 908, 913, 945, 960, 980, 981, 992, 996, 1034, 1043, 1052–1054, 1071, 1073, 1076, 1084, 1088, 1090, 1092, 1093, 1201, 1219, 1223, 1236, 1243, 1244, 1269, 1291, 1296, 1312
composition attack, 198, 344, 346, 349
conditional entropy, 103, 107, 120, 340
contractive noisy iteration, 729, 731, 732, 735

data processing inequality, 64, 72–75, 77, 78, 108, 109, 120, 123, 257, 362, 371, 401, 402, 429, 446, 447, 600, 613, 645, 647, 679, 693, 724, 727, 730, 766, 774, 1140, 1141
deduplication, 289
Deep Leakage from Gradients, 292, 300–306, 309, 310
differencing attack, 415
differential privacy, 1, 195, 254, 369, 371–373, 377, 382, 385, 398, 416, 417, 435, 436, 445, 453, 454, 474, 475, 492, 493, 513, 515, 533, 556, 557, 578, 579, 597, 600, 601, 603, 606, 623, 641–643, 658, 661, 662, 664, 665, 682, 683, 702, 703, 711, 721, 722, 740, 741, 760, 777, 778, 780, 781, 785, 803, 804, 821, 823, 842, 862, 863, 884, 902, 903, 905, 922, 941, 960–962, 980, 981, 1001, 1032, 1033, 1054, 1073, 1074, 1093, 1140, 1201, 1243, 1269, 1291, 1296, 1319, 1342
Dinur–Nissim attack, 355, 369, 575, 577
discoverable memorization, 281, 288
discrete Gaussian distribution, 375, 524, 527–530, 532, 533, 557, 641, 662, 1093
discrete Laplace distribution, 375, 519–522, 530, 532
distance to instability, 513, 877, 884
distributed differential privacy, 557, 740
distribution inference, 272
dominating pair, 703, 708, 715, 721, 722, 764, 1244, 1254, 1256, 1269, 1270, 1291
double backpropagation, 301, 309
DP-SGD, 600–604, 621–623, 659–661, 677, 678, 680, 682, 697–702, 704, 715, 718–721, 741, 760, 763, 765, 768, 773, 777, 778, 780–783, 803, 806, 808–811, 813, 817, 818, 821–823, 827, 828, 833, 834, 836–840, 846, 857–862, 870–872, 880, 882, 883, 889, 892–894, 896, 897, 902, 905, 907, 910–919, 921, 922, 924, 925, 927, 930, 933, 936–938, 941, 943, 944, 947, 954, 957–960, 966, 974–977, 979, 980, 1066, 1067, 1073, 1266–1268

earth mover's distance, 342, 348
eccentricity, 322–324, 328–330
empirical risk minimization, 4, 142, 144, 147, 148, 151, 153, 154, 158, 168, 170, 780, 781, 786, 787, 803, 804, 821, 888, 891, 901, 903, 941, 1292, 1319
entropy, 6, 102–108, 110–113, 117–120, 122, 123, 221, 243, 291, 319, 338–340, 347, 348, 362, 363, 1122, 1138,

1147
equivalence class, 311, 331, 332
exponential mechanism, 372, 374, 375, 456–462, 464–467, 472–475, 483, 484, 490, 492, 513, 585–587, 596, 649, 661, 662, 768, 780, 836, 837, 841, 842, 990, 991, 1000, 1002, 1015, 1016, 1031–1033, 1216, 1236, 1243
extractable memorization, 198, 289

f-divergence, 70–73, 78, 641, 642
f-DP, 397, 398, 601, 603, 664, 668, 669, 671, 682, 737, 762, 769, 771, 780, 813, 967, 1093, 1244, 1262, 1269, 1296
FFT accountant, 722
fingerprinting code, 493, 579, 1001
fingerprinting lemma, 567, 568, 577, 579, 1319
frequency oracle, 375, 541, 554, 556, 557
fully adaptive composition, 760
Fundamental Law of Information Recovery, 196

Gaussian differential privacy, 398, 436, 454, 601, 603, 641, 665, 667–672, 674–677, 679, 680, 682, 683, 718, 721, 725, 736, 737, 760–764, 766–772, 774, 776, 777, 780, 804, 813, 845–847, 855, 857, 946, 947, 958–960, 962, 966, 967, 969–971, 976, 980, 1034, 1054, 1075, 1076, 1089–1094, 1122, 1140, 1198, 1201, 1244, 1253, 1261, 1266, 1269, 1296
Gaussian mechanism, 372, 373, 375, 403, 427–429, 431, 433, 435, 436, 443, 478, 479, 486, 524, 533, 551, 557, 565, 567, 573, 574, 576, 590, 600, 602, 603, 609, 610, 615, 616, 623, 626, 633, 637, 644, 646, 651–653, 662, 667, 668, 676, 683, 690, 693, 702, 722, 733, 760, 774, 777, 780, 783, 792, 804, 808, 813, 821, 845, 847, 866, 876, 884, 913, 938, 976, 992, 1001, 1022–1026, 1033, 1054, 1060, 1073, 1084, 1086, 1088, 1090, 1093, 1197, 1198, 1291, 1296, 1342
Gaussian trade-off function, 603, 632, 665
generalization gap, 149, 156, 198, 218, 234, 236, 253, 1156
geometric mechanism, 533
global sensitivity, 372, 435, 494, 495, 501, 508, 586, 780
gradient inversion, 196, 310
group privacy, 372, 375, 404, 406, 407, 412, 414–417, 559, 563, 566, 591, 615, 616, 621, 623, 652, 653, 660–662, 689, 700, 900, 906, 910–912, 920, 922, 1230, 1243
Gumbel-max trick, 465, 473, 475

hidden state, 740, 741
hockey-stick divergence, 6, 68, 69, 199, 371, 374, 401, 411, 421, 426, 431, 525, 600, 603, 668, 669, 690, 766, 1222, 1253, 1255, 1256
Hoeffding's inequality, 49, 363, 442, 649, 1146
homomorphic encryption, 310
honest-but-curious, 202, 203, 309

imputation baseline, 197, 240, 241, 253, 254, 340
individual privacy accounting, 756–760

k-anonymity, 195, 196, 198, 329, 332–334, 336, 349
Kullback–Leibler divergence, 55, 56, 59, 73, 78, 107, 441, 442, 446, 451, 538, 607, 608, 611, 623, 648, 1122, 1138, 1139, 1147, 1260, 1291

l-diversity, 196, 198, 332, 338, 340, 349
label leakage, 310
label-only attack, 202, 204, 205, 215, 216, 254
Laplace distribution, 5, 6, 18, 55, 64, 421, 422, 520
Laplace mechanism, 398, 421, 425, 431, 433, 435, 436, 443, 446, 454, 460, 473, 478, 479, 484, 502, 521, 539, 540, 562, 565, 609, 610, 616, 665, 702, 996, 1009, 1032, 1033, 1224, 1339, 1340
learning with errors, 171, 188–190, 193, 1202, 1211–1213, 1215

likelihood ratio attack, 196–198, 209, 222–226, 229–232, 234–236, 780, 781, 900, 980, 1148
likelihood-ratio test, 83, 196, 204, 1247, 1249, 1254
linkage attack, 311, 312, 349
Lipschitz, 5, 125–127, 129, 134, 135, 138, 142, 145, 147, 153, 154, 159, 160, 731, 890, 891, 1148, 1156, 1160, 1175
local differential privacy, 539, 556, 955
local randomizer, 374, 534, 535, 538, 539, 541, 549, 550, 556, 726, 739, 769
local sensitivity, 494, 495, 497–499, 501–503, 505, 512, 513
logit scaling, 236
loss attack, 216, 218, 220, 222, 224–226, 229, 235, 236, 948, 959

malicious adversary, 202, 203
martingale, 49–52, 442, 744, 746, 760
max-divergence, 58, 59, 63, 64, 371, 607, 647
McDiarmid's inequality, 50, 51, 53, 157, 318, 319, 328
membership advantage, 204, 213, 214, 239, 253, 261, 375, 387–389, 397, 398
membership inference, 195–197, 204, 216, 236, 254, 371, 573, 780, 900, 944, 948, 980, 1170, 1175
meta-classifier, 262, 266, 267, 269–272
Mills ratio, 45, 427, 428, 645, 1018, 1027
min-entropy, 6, 105, 110–113, 117–119, 122, 123, 243, 362, 1122, 1138
model inversion, 195, 236, 245, 253, 254, 340
moment generating function, 9–14, 17–19, 21, 24, 29, 30, 36, 37, 39, 40, 46, 47, 55, 59, 60, 165, 442, 625, 627, 638, 644, 647, 677, 909, 992, 1065, 1084, 1088, 1094, 1114, 1119, 1124, 1126, 1198, 1264, 1312
moments accountant, 606, 617, 621–623, 641, 702, 821, 1054, 1073
Mondrian, 334, 336, 338, 340, 341, 343, 346, 348, 349
mutual information, 4, 6, 106–109, 111, 115, 123, 362

Narayanan–Shmatikov attack, 396
negligible function, 173–175, 192
neighbouring datasets, 371, 374–377, 398, 413, 418, 419, 423, 434, 437, 450, 465, 476, 496, 514, 602, 605, 606, 630, 643, 689, 705, 770, 772, 773, 782, 787, 840, 1004, 1007, 1010, 1030, 1072
Neyman–Pearson lemma, 81, 100, 195, 196, 198, 204, 209, 221, 223, 286, 573, 667, 945, 967, 968, 1090, 1145, 1249, 1260
normal distribution, 5, 20, 177, 200, 219, 255, 260, 371, 383, 406, 429, 510, 524, 566, 583–585, 590, 626, 645, 690, 705, 867, 969, 1045, 1046, 1059, 1220

objective perturbation, 789–791, 800, 802–804, 1292, 1298, 1299, 1318, 1319
one-way marginal, 579
onion effect, 236
optimal composition, 372, 373, 375, 398, 414, 416, 417, 443, 447, 448, 450, 453, 454, 468, 484, 600, 683, 1053, 1054, 1243
optimistic discretization, 773
output perturbation, 548, 557, 786, 791, 799–804, 941, 955, 1298, 1299, 1318

packing lower bound, 560, 562, 563, 571, 578, 1227
Paley–Zygmund inequality, 353, 354, 1183
parallel composition, 412, 417, 836
permutation invariance, 272
permute-and-flip, 475
pessimistic discretization, 773
Pinsker's inequality, 57, 67, 74–78, 539
Poisson distribution, 12–15, 24, 29, 30, 37, 333–335, 348, 525–528, 533, 602, 604, 656, 657, 682, 684, 686, 687, 689–691, 693, 697, 699–702, 715, 722, 762, 768, 770–773, 777, 778, 783, 806, 808, 817, 820, 825, 827, 831–835, 837–842, 846, 857–860, 862, 880, 884, 894, 905, 907, 908, 938,

943, 958, 966, 1061, 1063, 1072, 1073, 1094, 1096, 1107–1109, 1118, 1119, 1244, 1267, 1269, 1270, 1277, 1291, 1316
Poisson sampling, 602, 682, 684, 686, 690, 691, 702, 715, 722, 772, 773, 777, 783, 806, 808, 820, 840, 846, 857, 860, 862, 880, 884, 894, 907, 908, 943, 958, 966, 1061, 1270, 1291, 1316
Poisson summation formula, 526, 528, 533
post-processing, 371, 372, 375, 401, 403, 413–417, 429, 445–447, 449, 453, 454, 484, 502, 503, 506, 536, 547, 551, 575, 591, 613, 616, 644, 645, 647, 660, 664, 671, 682, 726, 727, 732, 739, 740, 766, 771, 772, 792, 795, 808, 827, 831, 836, 844, 845, 847, 861, 866, 869, 871, 913, 945, 973, 989, 990, 999, 1059, 1198, 1224, 1236, 1243, 1277, 1281, 1296, 1297, 1303, 1307, 1322, 1330
privacy filter, 414, 604, 743, 760
privacy game, 4, 173, 195, 197, 205, 206, 208, 216, 218, 223, 254, 272, 371, 387
privacy loss, 371, 373, 374, 378, 381, 383, 389, 395, 398, 401–406, 408–413, 422–426, 434, 435, 437–441, 444, 451, 454, 457, 464, 471, 472, 496, 500, 502, 517, 521, 523, 524, 528, 531, 576, 600, 601, 603–605, 617, 618, 625, 629, 637, 638, 642, 644–646, 649, 660, 662, 672, 674, 692, 704, 705, 715, 716, 721, 722, 740, 742, 746–748, 759–762, 773, 777, 791, 827, 942–944, 950, 951, 961, 1002, 1004, 1009, 1012, 1013, 1017, 1022, 1032, 1037, 1043, 1046, 1053, 1060, 1069, 1088, 1093, 1198–1200, 1222, 1230, 1234, 1259, 1264, 1272, 1275, 1282, 1284, 1308
privacy loss distribution, 424, 434, 437, 601, 603, 604, 704–715, 717–722, 760–768, 772–778, 944–947, 949, 951, 958, 960, 1053, 1071, 1267
privacy loss random variable, 373, 374, 437, 600, 603
privacy odometer, 417, 454, 743, 760
privacy profile, 373, 374, 403, 428, 431, 435, 438, 454, 496, 619, 623, 669, 671, 676, 679, 763, 773, 776, 942, 944, 967, 1267
property inference, 195, 196, 256, 257, 271, 272
propose-test-release, 375, 502, 503, 506, 512, 513
PRV accountant, 683, 722, 777, 778
pseudorandom function, 179, 180, 183, 184, 186, 192, 193, 1202, 1205, 1207, 1214
pure differential privacy, 374, 377, 378, 380, 383, 390, 397, 398, 404, 408, 412, 431, 435, 436, 443, 486, 487, 492, 500, 504, 505, 512, 515, 521, 558, 564, 567, 571, 577–579, 607, 608, 610, 616, 622, 641, 646, 648, 660–662, 700, 739, 742, 750, 759, 768, 776, 787, 791, 802, 841, 864, 873, 876, 950, 960, 971, 1053, 1081, 1088, 1093, 1222, 1227, 1230, 1304, 1308, 1328, 1329, 1331, 1341, 1342

quantitative information flow, 123
quasi-identifier, 198, 311–313, 331–338, 340, 343, 348

randomized response, 380–382, 384, 387, 389, 393, 396, 398, 402, 403, 405, 409–411, 413, 415, 416, 439–441, 444–447, 449, 451–454, 535, 537, 539–543, 547, 550–552, 554, 556, 600, 604, 605, 607–612, 616, 617, 621–623, 638, 640, 647–649, 660, 661, 665, 666, 672, 675, 676, 686, 687, 702, 711, 712, 720, 723, 724, 726–728, 738–740, 744, 745, 749, 758, 768–770, 776, 826, 827, 830, 833, 840, 841, 867, 876, 878, 980, 1002, 1031, 1032, 1037, 1041, 1043, 1053, 1054, 1079, 1092, 1216, 1221, 1249, 1251, 1256, 1261–1263, 1268, 1272, 1280, 1281, 1291, 1309, 1326, 1341
RDP curve, 601–606, 608–611, 613, 620, 622, 628, 630, 632, 633, 635, 637, 639, 645–649, 657, 659, 661, 678–681, 694, 695, 697, 731, 733, 761, 769,

770, 772, 808, 812, 820, 827, 833, 836, 866, 876, 878, 908, 913, 945, 947, 1034, 1049, 1066, 1067, 1088, 1142, 1308, 1313
re-identification, 196, 322, 323, 330
reconstruction attack, 196, 369, 578, 579, 952
rejection sampling, 70, 521
report-noisy-max, 372, 375, 435, 464, 465, 472, 474, 475, 561–563, 780, 866, 883
ROC curve, 94, 100, 101, 210, 211, 216, 227, 228, 230, 234, 236
Rényi differential privacy, 371, 600–623, 625, 627, 628, 630–633, 635, 637–642, 645–649, 653, 657–662, 678–682, 690, 691, 693–703, 718–721, 731, 733, 740, 742, 747–750, 753, 755–757, 759–773, 775–778, 780, 783, 806, 808, 809, 812, 820, 821, 823, 827, 830–833, 836, 842, 857, 866, 867, 871, 873, 875, 876, 878–880, 883, 884, 905, 907, 908, 913, 922, 945–947, 952–954, 958–960, 977, 1034, 1048, 1049, 1051–1055, 1065–1069, 1071, 1073–1075, 1084, 1087, 1088, 1092, 1093, 1122, 1140–1143, 1223, 1270, 1286, 1290–1292, 1303, 1307, 1308, 1310, 1313, 1314, 1339, 1340, 1342
Rényi divergence, 58, 59, 62, 65, 72, 78, 527, 529, 600, 603, 605, 609, 623, 631, 637, 638, 645, 652, 692, 693, 702, 730, 740, 747, 762, 763, 766, 769, 826, 833, 864, 909, 945, 1054, 1069, 1092, 1122, 1139–1142, 1147, 1222, 1223, 1270, 1286, 1289, 1291, 1303, 1308, 1314, 1342
Rényi filter, 743, 747, 748, 750, 753, 756, 759, 760, 876

sample complexity, 486, 492, 577, 579, 884, 1215
sampling without replacement, 688, 701, 702, 740, 771–773, 777, 1073
secure aggregation, 4, 375, 532, 533, 551, 557
semantic security, 193
sensitivity, 4, 142, 144, 147, 148, 153, 330, 371–375, 398, 417–422, 424, 427, 428, 430–432, 434–436, 443, 448, 456, 460, 461, 467, 477, 479, 484, 488–490, 492, 494, 495, 497–499, 501–508, 510, 512, 513, 516, 521, 524, 526, 551, 565, 581, 582, 586, 591, 597, 603, 610, 611, 622, 626, 644, 665, 667, 693, 707, 715, 769, 780, 783, 785–789, 801–803, 807, 808, 817, 820, 821, 843, 847, 848, 850, 851, 855–857, 861, 862, 866, 876, 878, 880, 883, 884, 893, 894, 897, 937, 977, 983, 990, 992, 996, 1002, 1007, 1009, 1010, 1016, 1020, 1022–1025, 1027, 1030–1033, 1039, 1046, 1047, 1051, 1052, 1055, 1072, 1197, 1224, 1236, 1296
shadow model, 197, 221, 222, 225, 226, 229, 231–236, 262, 271, 289
shifted Rényi divergence, 730, 740, 1291
shuffle model, 536, 557, 600, 723, 740, 769, 1281
simulation paradigm, 193
Skellam mechanism, 557
skewness attack, 343, 344
smooth sensitivity, 375, 497–499, 505, 513, 597, 883, 884
smoothness, 5, 125, 127, 142, 147, 148, 500, 792, 1301
snapping mechanism, 516, 517, 533
sparse vector technique, 372, 374, 375, 435, 470, 474, 475, 492, 780, 884, 1001, 1216, 1242, 1243
staircase mechanism, 436
stochastic gradient descent, 3, 6, 125, 129, 131, 132, 134–143, 147, 148, 150, 159–162, 168, 170, 600–604, 621–623, 659–661, 676–678, 680, 682, 697–702, 704, 715, 718–721, 731, 733, 734, 739–741, 760, 762, 763, 765, 768, 773, 777, 778, 780–783, 795, 796, 803, 804, 806, 808–811, 813, 817, 818, 821–823, 827, 828, 833, 834, 836–840, 846, 851, 857–862, 870–872, 880, 882, 883, 889, 892–894, 896, 897, 902, 905, 907, 910–919, 921, 922, 924, 925, 927, 930, 933,

936–938, 941, 943, 944, 947, 954, 957–960, 966, 967, 974–977, 979, 980, 1053, 1055, 1057, 1059, 1062, 1065–1073, 1148, 1156, 1160, 1161, 1174, 1175, 1201, 1244, 1266–1269, 1292, 1296
strong convexity, 5, 125–130, 136, 144, 147, 148, 153, 170, 740, 780, 787, 789, 792, 798, 803, 1167, 1168, 1174, 1175, 1296, 1298, 1299, 1301, 1319
sub-Gaussian, 38–40, 52, 53
subsample-and-aggregate, 375, 507–509, 512, 513, 884
subsampled Gaussian mechanism, 601, 604, 623, 655–657, 661, 662, 676, 692–696, 700, 715, 717, 722, 768, 777, 808, 827, 880, 894, 913, 920, 938, 967, 1055, 1069, 1073, 1141, 1147, 1244, 1267–1269, 1275, 1291
subset-sum query, 350
sufficient statistic, 72, 77

t-closeness, 196, 198, 332, 341, 343, 349
threat model, 1, 4, 195, 197, 198, 200, 202, 205, 215, 371, 965, 979, 980, 1148
total variation distance, 6, 54, 65–67, 70, 76, 89–92, 195, 196, 198, 199, 227, 239, 341, 343, 344, 348, 398, 415, 428, 446, 467, 473, 573, 717, 1099
tracing attack, 196, 369, 375, 573, 574, 579
trade-off function, 6, 84–89, 92, 195, 198, 199, 209, 371, 385, 386, 398, 436, 445, 446, 454, 600, 601, 603, 604, 632, 664–668, 671, 673, 674, 676, 679, 680, 780, 942, 944, 947, 949, 952, 960, 966, 969, 976, 980, 1091, 1198, 1221, 1244, 1247, 1250, 1255
training-data extraction, 196
truncated concentrated differential privacy, 657–661, 1122, 1141, 1270
type I error, 80, 209, 603, 665, 668, 670, 671, 1130, 1142, 1247, 1249–1251, 1255, 1259, 1260, 1266, 1268
type II error, 80, 84, 85, 87, 209, 374, 603, 665, 668, 671, 1130, 1132, 1141, 1143, 1247, 1249, 1250, 1255, 1259, 1260

unary encoding, 541–544, 554, 556
uniform stability, 148, 150, 152, 157
union bound, 42, 97, 180–182, 186, 192, 346, 347, 355, 359, 362, 422, 425, 461, 462, 464, 468, 469, 478, 484, 489, 490, 562–565, 577, 584, 590, 753, 879, 883, 964, 1038, 1163, 1183, 1184, 1212, 1225, 1282, 1330, 1335, 1337
unit of privacy, 375, 397, 398, 772, 773, 777, 843, 846, 848, 850–852, 856, 905, 907, 910, 911, 913–919, 921, 922, 1192, 1193

variance, 3, 5, 7–10, 12, 14, 16–18, 20, 22, 24, 27, 29, 30, 34, 35, 38–42, 47, 50, 52, 53, 59, 65, 93, 98, 125, 133, 137–139, 142, 147, 168, 208, 219, 225–227, 229, 234–236, 260, 263–265, 271, 354, 357, 358, 364, 381, 383, 424, 426, 429, 432, 439, 440, 507, 510, 513, 520, 524, 527, 528, 531–533, 536, 537, 541–544, 546, 551, 553, 556, 566, 570, 572, 578, 581–585, 591, 592, 594, 595, 597, 598, 617, 645, 646, 649, 660, 667, 674, 676, 677, 707–709, 720, 725, 728, 737, 744, 766, 769, 785, 793, 803, 863, 886–888, 909, 922, 931, 932, 935, 940, 941, 967, 977, 992, 1013, 1045, 1054, 1060, 1084, 1097, 1107, 1109–1111, 1114, 1116, 1119, 1123, 1124, 1129, 1130, 1166, 1167, 1169, 1170, 1172, 1178, 1183, 1198, 1200, 1229, 1259, 1260, 1265, 1296, 1312, 1316, 1342
vulnerability, 111–114, 123, 240–242, 249, 253

weak triangle inequality, 614, 616, 623
white-box attack, 202, 204, 205, 215, 216, 236, 256, 965, 966, 970, 972, 974–976, 979, 1175

zero-concentrated differential privacy, 601, 603, 604, 643–654, 656–662, 750, 751, 760, 768–770, 776, 777, 955, 956, 958–961, 992, 997, 1076, 1087, 1088, 1092, 1093, 1216, 1223, 1242, 1243